NUMBERED SOLUTION COMPANION

Match each exercise by its four-digit problem number. Includes the original JavaScript source, approach hints and study prompts. Use PDF bookmarks to jump directly to a problem.

READING CODE: A QUICK MAP

Loop

Trace changing values after each iteration.

Map / Set

Track keys, values, membership checks and duplicates.

Recursion

Identify the base case and what each call returns.

Dynamic programming

Record the state definition and recurrence.

Complexity

Count repeated operations and extra data structures.

0001 Two Sum

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1. Two Sum
 * https://leetcode.com/problems/two-sum/
 * Difficulty: Easy
 *
 * Given an array of integers `nums` and an integer `target`, return indices
 * of the two numbers such that they add up to `target`.
 *
 * You may assume that each input would have exactly one solution, and you
 * may not use the same element twice.
 *
 * You can return the answer in any order.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
 */
var twoSum = function(nums, target) {
  const map = new Map();

  for (let i = 0; i < nums.length; i++) {
    const diff = target - nums[i];

    if (map.has(diff)) {
      return [map.get(diff), i];
    }

    map.set(nums[i], i);
  }
};
```

0002 Add Two Numbers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2. Add Two Numbers
 * https://leetcode.com/problems/add-two-numbers/
 * Difficulty: Medium
 *
 * You are given two non-empty linked lists representing two non-negative integers.
 *
 * The digits are stored in reverse order, and each of their nodes contains a single digit.
 * Add the two numbers and return the sum as a linked list.
 *
 * You may assume the two numbers do not contain any leading zero, except the number 0 itself.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} l1
 * @param {ListNode} l2
 * @return {ListNode}
```

```
 */
var addTwoNumbers = function(l1, l2) {
  const result = new ListNode();

  for (let tail = result, carry = 0; l1 || l2 || carry;) {
    const value = (l1?.val ?? 0) + (l2?.val ?? 0) + carry;
    tail.next = new ListNode(value % 10);
    tail = tail.next;
    carry = value >= 10 ? 1 : 0;
    [l1, l2] = [l1 && l1.next, l2 && l2.next];
  }

  return result.next;
};
```

0003 Longest Substring Without Repeating Characters

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3. Longest Substring Without Repeating Characters
 * https://leetcode.com/problems/longest-substring-without-repeating-characters/
 * Difficulty: Medium
 *
 * Given a string `s`, find the length of the longest substring without repeating characters.
 */

/**
 * @param {string} s
 * @return {number}
 */
var lengthOfLongestSubstring = function(s) {
  const map = {};
  let offset = 0;

  return s.split('').reduce((max, value, i) => {
    offset = map[value] >= offset ? map[value] + 1 : offset;
    map[value] = i;
    return Math.max(max, i - offset + 1);
  }, 0);
};
```

0004 Median Of Two Sorted Arrays

HARD

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 4. Median of Two Sorted Arrays
 * https://leetcode.com/problems/median-of-two-sorted-arrays/
 * Difficulty: Hard
 *
 * There are two sorted arrays nums1 and nums2 of size m and n respectively.
 *
 * Find the median of the two sorted arrays.
 * The overall run time complexity should be O(log (m+n)).
 *
 * You may assume nums1 and nums2 cannot be both empty.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var findMedianSortedArrays = function(nums1, nums2) {
```

```
  const sorted = [...nums1, ...nums2].sort((a, b) => a - b);
  const index = Math.floor((sorted.length - 1) / 2);

  return sorted.length % 2 === 0
    ? (sorted[index] + sorted[index + 1]) / 2
    : sorted[index];
};
```

0005 Longest Palindromic Substring

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 5. Longest Palindromic Substring
 * https://leetcode.com/problems/longest-palindromic-substring/
 * Difficulty: Medium
 *
 * Given a string `s`, return the longest palindromic substring in `s`.
 */

/**
 * @param {string} s
 * @return {string}
 */
var longestPalindrome = function(s) {
  let result = '';

  for (let i = 0; i < s.length; i++) {
    const palindrome1 = getExtendedPalindrome(s, i, i);
    const palindrome2 = getExtendedPalindrome(s, i, i + 1);
    const longerPalindrome = palindrome1.length > palindrome2.length
      ? palindrome1
      : palindrome2;

    if (longerPalindrome.length > result.length) {
      result = longerPalindrome;
    }
  }

  return result;
};

function getExtendedPalindrome(s, start, end) {
  while (start >= 0 && end < s.length && s[start] === s[end]) {
    start--;
    end++;
  }

  return s.slice(start + 1, end);
}
```

0006 Zigzag Conversion

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 6. ZigZag Conversion
 * https://leetcode.com/problems/zigzag-conversion/
 * Difficulty: Medium
 *
 * The string `"PAYPALISHIRING"` is written in a zigzag pattern on a given
 * number of rows like this: (you may want to display this pattern in a
 * fixed font for better legibility)
 *
 * > P   A   H   N
```

```
 * > A P L S I I G
 * > Y   I   R
 *
 * And then read line by line: `"PAHNAPLSIIGYIR"`
 */

/**
 * @param {string} s
 * @param {number} numRows
 * @return {string}
 */
var convert = function(s, numRows) {
  const order = [...new Array(numRows).keys()];
  order.push(...order.slice(1, -1).reverse());

  const rows = new Array(numRows).fill('');
  [...s].forEach((c, i) => (rows[order[i % order.length]] += c));

  return rows.join('');
};
```

0007 Reverse Integer

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 7. Reverse Integer
 * https://leetcode.com/problems/reverse-integer/
 * Difficulty: Easy
 *
 * Given a signed 32-bit integer x, return x with its digits reversed.
 * If reversing x causes the value to go outside the signed 32-bit integer
 * range [-2^31, 2^31 - 1], then return 0.
 */

/**
 * @param {number} x
 * @return {number}
 */
var reverse = function(x) {
  const reversed = String(Math.abs(x)).split('').reverse().join('');

  if (reversed > Math.pow(2, 31)) {
    return 0;
  }

  return reversed * Math.sign(x);
};
```

0008 String To Integer Atoi

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 8. String to Integer (atoi)
 * https://leetcode.com/problems/string-to-integer-atoi/
 * Difficulty: Medium
 *
 * Implement the myAtoi(string s) function, which converts a string to a
 * 32-bit signed integer (similar to C/C++'s atoi function).
 *
 * The algorithm for myAtoi(string s) is as follows:
 *
 * - Read in and ignore any leading whitespace.
 * - Check if the next character (if not already at the end of the string)
```

```
 *   is '-' or '+'. Read this character in if it is either. This determines
 *   if the final result is negative or positive respectively.
 *   Assume the result is positive if neither is present.
 * - Read in next the characters until the next non-digit charcter or the end
 *   of the input is reached. The rest of the string is ignored.
 * - Convert these digits into an integer (i.e. "123" -> 123, "0032" -> 32).
 *   If no digits were read, then the integer is 0. Change the sign as
 *   necessary (from step 2).
 * - If the integer is out of the 32-bit signed integer range [-231, 231 - 1],
 *   then clamp the integer so that it remains in the range. Specifically,
 *   integers less than -231 should be clamped to -231, and integers greater
 *   than 231 - 1 should be clamped to 231 - 1.
 * - Return the integer as the final result.
 */

/**
 * @param {string} s
 * @return {number}
 */
var myAtoi = function(s) {
  const parsed = +(s.trim().match(/^[-+]?\d+/g) || [0])[0];
  const clamped = Math.min(Math.max(parsed, (-2)**31), 2**31 - 1);

  return clamped;
};
```

0009 Palindrome Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 9. Palindrome Number
 * https://leetcode.com/problems/palindrome-number/
 * Difficulty: Easy
 *
 * Given an integer `x`, return `true` if `x` is palindrome integer.
 *
 * An integer is a palindrome when it reads the same backward as forward.
 * - For example, `121` is palindrome while `123` is not.
 */

/**
 * @param {number} x
 * @return {boolean}
 */
var isPalindrome = function(x) {
  if (x < 0) return false;
  return +String(x).split('').reverse().join('') === x;
};
```

0010 Regular Expression Matching

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 10. Regular Expression Matching
 * https://leetcode.com/problems/regular-expression-matching/
 * Difficulty: Hard
 *
 * Given an input string s and a pattern p, implement regular expression
 * matching with support for '.' and '*' where:
 *
 * '.' Matches any single character.
 * '*' Matches zero or more of the preceding element.
 *
```

```
 * The matching should cover the entire input string (not partial).
 */

/**
 * @param {string} s
 * @param {string} p
 * @return {boolean}
 */
var isMatch = function(s, p) {
  return new RegExp(`^${p}$`).test(s);
};
```

0011 Container With Most Water

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 11. Container With Most Water
 * https://leetcode.com/problems/container-with-most-water/
 * Difficulty: Medium
 *
 * Given n non-negative integers `a1, a2, ..., an ,` where each represents a point at
 * coordinate `(i, ai)`. `n` vertical lines are drawn such that the two endpoints of
 * the line `i` is at `(i, ai)` and `(i, 0)`. Find two lines, which, together with the
 * x-axis forms a container, such that the container contains the most water.
 *
 * Notice that you may not slant the container.
 */

/**
 * @param {number[]} height
 * @return {number}
 */
var maxArea = function(height) {
  let maxArea = 0;

  for (let left = 0, right = height.length - 1; left < right;) {
    maxArea = Math.max(
      maxArea,
      Math.min(height[left], height[right]) * (right - left)
    );

    if (height[left] < height[right]) {
      left++;
    } else {
      right--;
    }
  }

  return maxArea;
};
```

0012 Integer To Roman

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 12. Integer to Roman
 * https://leetcode.com/problems/integer-to-roman/
 * Difficulty: Medium
 *
 * Roman numerals are represented by seven different symbols: `I`, `V`, `X`,
 * `L`, `C`, `D` and `M`.
 *
 * Symbol       Value
```

```
 * I            1
 * V            5
 * X            10
 * L            50
 * C            100
 * D            500
 * M            1000
 *
 * For example, `2` is written as `II` in Roman numeral, just two one's
 * added together. `12` is written as `XII`, which is simply `X + II`.
 * The number `27` is written as `XXVII`, which is `XX + V + II`.
 *
 * Roman numerals are usually written largest to smallest from left to right.
 * However, the numeral for four is not `IIII`. Instead, the number four is
 * written as `IV`. Because the one is before the five we subtract it making
 * four. The same principle applies to the number nine, which is written as
 * `IX`. There are six instances where subtraction is used:
 *
 * - `I` can be placed before `V (5)` and `X (10)` to make 4 and 9.
 * - `X` can be placed before `L (50)` and `C (100)` to make 40 and 90.
 * - `C` can be placed before `D (500)` and `M (1000)` to make 400 and 900.
 *
 * Given an integer, convert it to a roman numeral.
 */

/**
 * @param {number} num
 * @return {string}
 */
var intToRoman = function(num) {
  const map = {
    M: 1000, CM: 900, D: 500, CD: 400, C: 100, XC: 90,
    L: 50, XL: 40, X: 10, IX: 9, V: 5, IV: 4, I: 1
  };

  return Object.entries(map).reduce((result, [letter, n]) => {
    result += letter.repeat(Math.floor(num / n));
    num %= n;
    return result;
  }, '');
};
```

0013 Roman To Integer

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 13. Roman to Integer
 * https://leetcode.com/problems/roman-to-integer/
 * Difficulty: Easy
 *
 * Roman numerals are represented by seven different symbols: `I`, `V`, `X`,
 * `L`, `C`, `D` and `M`.
 *
 * Symbol       Value
 * I            1
 * V            5
 * X            10
 * L            50
 * C            100
 * D            500
 * M            1000
 *
 * For example, `2` is written as `II` in Roman numeral, just two one's
 * added together. `12` is written as `XII`, which is simply `X + II`.
 * The number `27` is written as `XXVII`, which is `XX + V + II`.
 *
 * Roman numerals are usually written largest to smallest from left to right.
 * However, the numeral for four is not `IIII`. Instead, the number four is
 * written as `IV`. Because the one is before the five we subtract it making
```

```
 * four. The same principle applies to the number nine, which is written as
 * `IX`. There are six instances where subtraction is used:
 *
 * - `I` can be placed before `V (5)` and `X (10)` to make 4 and 9.
 * - `X` can be placed before `L (50)` and `C (100)` to make 40 and 90.
 * - `C` can be placed before `D (500)` and `M (1000)` to make 400 and 900.
 *
 * Given a roman numeral, convert it to an integer.
 */

/**
 * @param {string} s
 * @return {number}
 */
var romanToInt = function(s) {
  const map = { M: 1000, D: 500, C: 100, L: 50, X: 10, V: 5, I: 1 };

  return s.split('').reduce((result, v, i) => {
    result += map[v] >= map[s[i + 1]] || s.length === i + 1
      ? map[v]
      : -1 * map[v];
    return result;
  }, 0);
};
```

0014 Longest Common Prefix

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 14. Longest Common Prefix
 * https://leetcode.com/problems/longest-common-prefix/
 * Difficulty: Easy
 *
 * Write a function to find the longest common prefix string amongst an array of strings.
 *
 * If there is no common prefix, return an empty string `""`.
 */

/**
 * @param {string[]} strs
 * @return {string}
 */
var longestCommonPrefix = function(strs) {
  const prefixes = strs[0].split('').map((_, i) => strs[0].slice(0, i + 1));

  for (let i = strs.length - 1; i > -1; i--) {
    for (let j = prefixes.length - 1; j > -1; j--) {
      if (strs[i].substr(0, j + 1) !== prefixes[j]) {
        prefixes.pop();
      }
    }
  }

  return prefixes.pop() || '';
};
```

0015 3Sum

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 15. 3Sum
 * https://leetcode.com/problems/3sum/
```

```
 * Difficulty: Medium
 *
 * Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]]
 * such that i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0.
 *
 * Notice that the solution set must not contain duplicate triplets.
 */

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var threeSum = function(nums) {
  const result = [];
  nums.sort((a, b) => a - b);

  for (let i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] === nums[i - 1]) continue;
    let j = i + 1;
    let k = nums.length - 1;
    while (j < k) {
      const sum = nums[i] + nums[j] + nums[k];
      if (!sum) {
        result.push([nums[i], nums[j], nums[k]]);
        j++;
        k--;
        while (j < k && nums[j] === nums[j - 1]) {
          j++;
        }
        while (j < k && nums[k] === nums[k + 1]) {
          k--;
        }
      } else {
        sum < 0 ? j++ : k--;
      }
    }
  }

  return result;
};
```

0016 3Sum Closest

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 16. 3Sum Closest
 * https://leetcode.com/problems/3sum-closest/
 * Difficulty: Medium
 *
 * Given an integer array nums of length n and an integer target, find three integers
 * in nums such that the sum is closest to target.
 *
 * Return the sum of the three integers.
 *
 * You may assume that each input would have exactly one solution.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var threeSumClosest = function(nums, target) {
  const sums = new Set();

  nums.sort((a, b) => a - b);

  for (let i = 0; i < nums.length - 2; i++) {
```

```
    let j = i + 1;
    let k = nums.length - 1;

    while (j < k) {
      const sum = nums[i] + nums[j] + nums[k];
      sums.add(sum);
      if (sum > target) {
        k--;
      } else {
        j++;
      }
    }
  }

  return [...sums].reduce((p, c) => Math.abs(c - target) < Math.abs(p - target) ? c : p);
};
```

0017 Letter Combinations Of A Phone Number

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 17. Letter Combinations of a Phone Number
 * https://leetcode.com/problems/letter-combinations-of-a-phone-number/
 * Difficulty: Medium
 *
 * Given a string containing digits from 2-9 inclusive, return all possible
 * letter combinations that the number could represent. Return the answer
 * in any order.
 *
 * A mapping of digit to letters (just like on the telephone buttons) is
 * given below. Note that 1 does not map to any letters.
 */

/**
 * @param {string} digits
 * @return {string[]}
 */
var letterCombinations = function(digits) {
  if (!digits || !digits.length) return [];

  const map = {
    2: 'abc',
    3: 'def',
    4: 'ghi',
    5: 'jkl',
    6: 'mno',
    7: 'pqrs',
    8: 'tuv',
    9: 'wxyz'
  };

  if (digits.length === 1) {
    return map[digits].split('');
  }

  const result = [];
  const group1 = letterCombinations(digits.substr(0, 1));
  const group2 = letterCombinations(digits.substr(1));

  for (let i = 0; i < group1.length; i++) {
    for (let j = 0; j < group2.length; j++) {
      result.push(group1[i] + group2[j]);
    }
  }

  return result;
};
```

0018 4Sum

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 18. 4Sum
 * https://leetcode.com/problems/4sum/
 * Difficulty: Medium
 *
 * Given an array nums of n integers, return an array of all the unique
 * quadruplets [nums[a], nums[b], nums[c], nums[d]] such that:
 * - 0 <= a, b, c, d < n
 * - a, b, c, and d are distinct.
 * - nums[a] + nums[b] + nums[c] + nums[d] == target
 *
 * You may return the answer in any order.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[][]}
 */
var fourSum = function(nums, target) {
  const result = [];

  nums.sort((a, b) => a - b);

  for (let i = 0; i < nums.length - 3; i++) {
    for (let j = i + 1; j < nums.length - 2; j++) {
      let high = nums.length - 1;
      let low = j + 1;

      while (low < high) {
        const sum = nums[i] + nums[j] + nums[low] + nums[high];

        if (sum === target) {
          result.push([nums[i], nums[j], nums[low], nums[high]]);
          while (nums[low] === nums[low + 1]) {
            low++;
          }
          while (nums[high] === nums[high - 1]) {
            high--;
          }
          low++;
          high--;
        } else if (sum < target) {
          low++;
        } else {
          high--;
        }
      }
      while (nums[j] === nums[j + 1]) {
        j++;
      }
    }
    while (nums[i] === nums[i + 1]) {
      i++;
    }
  }

  return result;
};
```

0019 Remove Nth Node From End Of List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 19. Remove Nth Node From End of List
 * https://leetcode.com/problems/remove-nth-node-from-end-of-list/
 * Difficulty: Medium
 *
 * Given the head of a linked list, remove the nth node from the end of the list
 * and return its head.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} n
 * @return {ListNode}
 */
var removeNthFromEnd = function(head, n) {
  const result = new ListNode();
  let slow = result;
  let fast = result;
  slow.next = head;

  for (let i = 0; i <= n; i++) {
    fast = fast.next;
  }

  while (fast) {
    fast = fast.next;
    slow = slow.next;
  }

  slow.next = slow.next.next;

  return result.next;
};
```

0020 Valid Parentheses

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 20. Valid Parentheses
 * https://leetcode.com/problems/valid-parentheses/
 * Difficulty: Easy
 *
 * Given a string s containing just the characters '(', ')', '{', '}',
 * '[' and ']', determine if the input string is valid.
 *
 * An input string is valid if:
 * - Open brackets must be closed by the same type of brackets.
 * - Open brackets must be closed in the correct order.
 */

/**
 * @param {string} s
 * @return {boolean}
```

```
 */
var isValid = function(s) {
  const map = {
    '(': ')',
    '[': ']',
    '{': '}',
  };
  const stack = [];

  for (let i = 0; i < s.length; i++) {
    if (map[s[i]]) {
      stack.push(map[s[i]]);
    } else {
      if (stack.pop() !== s[i]) {
        return false;
      }
    }
  }

  return stack.length === 0;
};
```

0021 Merge Two Sorted Lists

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 21. Merge Two Sorted Lists
 * https://leetcode.com/problems/merge-two-sorted-lists/
 * Difficulty: Easy
 *
 * Merge two sorted linked lists and return it as a sorted list.
 * The list should be made by splicing together the nodes of the
 * first two lists.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} l1
 * @param {ListNode} l2
 * @return {ListNode}
 */
var mergeTwoLists = function(l1, l2) {
  if (!l1 || !l2) {
    return l1 || l2;
  }

  if (l1.val > l2.val) {
    [l2, l1] = [l1, l2];
  }

  l1.next = mergeTwoLists(l1.next, l2);

  return l1;
};
```

0022 Generate Parentheses

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 22. Generate Parentheses
 * https://leetcode.com/problems/generate-parentheses/
 * Difficulty: Medium
 *
 * Given n pairs of parentheses, write a function to generate all combinations of
 * well-formed parentheses.
 */

/**
 * @param {number} n
 * @return {string[]}
 */
var generateParenthesis = function(n) {
  const result = [];
  backtrack(result, '', 0, 0, n);
  return result;
};

function backtrack(result, str, open, close, max) {
  if (str.length === max * 2) {
    result.push(str);
    return;
  }

  if (open < max) backtrack(result, `${str}(`, open + 1, close, max);
  if (close < open) backtrack(result, `${str})`, open, close + 1, max);
}
```

0023 Merge K Sorted Lists

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 23. Merge k Sorted Lists
 * https://leetcode.com/problems/merge-k-sorted-lists/
 * Difficulty: Hard
 *
 * You are given an array of k linked-lists lists, each linked-list is sorted
 * in ascending order.
 *
 * Merge all the linked-lists into one sorted linked-list and return it.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode[]} lists
 * @return {ListNode}
 */
var mergeKLists = function(lists) {
  const map = new Map();
  for (let list of lists) {
    while (list) {
      map.set(list.val, (map.get(list.val) || 0) + 1);
      list = list.next;
    }
  }
  const sorted = [...map].sort(([a], [b]) => a - b);
  const result = new ListNode();
  let tail = result;
```

```
  for (let [key, value] of sorted) {
    while (value--) {
      tail.next = new ListNode(key);
      tail = tail.next;
    }
  }

  return result.next;
};
```

0024 Swap Nodes In Pairs

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 24. Swap Nodes in Pairs
 * https://leetcode.com/problems/swap-nodes-in-pairs/
 * Difficulty: Medium
 *
 * Given a linked list, swap every two adjacent nodes and return its head. You must solve the
 * problem without modifying the values in the list's nodes (i.e., only nodes themselves may
 * be changed.)
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var swapPairs = function(head) {
  if (!head || !head.next) return head;

  const result = head.next;
  head.next = result.next;
  result.next = head;
  head.next = swapPairs(head.next);

  return result;
};
```

0025 Reverse Nodes In K Group

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 25. Reverse Nodes in k-Group
 * https://leetcode.com/problems/reverse-nodes-in-k-group/
 * Difficulty: Hard
 *
 * Given the head of a linked list, reverse the nodes of the list k at a time,
 * and return the modified list.
 *
 * k is a positive integer and is less than or equal to the length of the linked
 * list. If the number of nodes is not a multiple of k then left-out nodes, in
 * the end, should remain as it is.
 *
 * You may not alter the values in the list's nodes, only nodes themselves may
 * be changed.
 */
```

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} k
 * @return {ListNode}
 */
var reverseKGroup = function(head, k) {
  const result = new ListNode(null, head);
  const stack = [];
  let tail = result;

  while (head) {
    for (let i = 0; i < k && head; i++) {
      stack.push(head);
      head = head.next;
    }
    if (stack.length === k) {
      while (stack.length) {
        tail.next = stack.pop();
        tail = tail.next;
      }
      tail.next = head;
    }
  }

  return result.next;
};
```

0026 Remove Duplicates From Sorted Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 26. Remove Duplicates from Sorted Array
 * https://leetcode.com/problems/remove-duplicates-from-sorted-array/
 * Difficulty: Easy
 *
 * Given an integer array nums sorted in non-decreasing order, remove the
 * duplicates in-place such that each unique element appears only once.
 * The relative order of the elements should be kept the same.
 *
 * Since it is impossible to change the length of the array in some languages,
 * you must instead have the result be placed in the first part of the array
 * nums. More formally, if there are k elements after removing the duplicates,
 * then the first k elements of nums should hold the final result. It does not
 * matter what you leave beyond the first k elements.
 *
 * Return k after placing the final result in the first k slots of nums.
 *
 * Do not allocate extra space for another array. You must do this by modifying
 * the input array in-place with O(1) extra memory.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var removeDuplicates = function(nums) {
  let i = 0;
  for (let j = 0; j < nums.length; j++) {
    if (nums[i] !== nums[j]) {
      nums[++i] = nums[j];
    }
```

```
  }
  return ++i;
};
```

0027 Remove Element

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 27. Remove Element
 * https://leetcode.com/problems/remove-element/
 * Difficulty: Easy
 *
 * Given an integer array nums and an integer val, remove all occurrences of val in nums
in-place.
 * The relative order of the elements may be changed.
 *
 * Since it is impossible to change the length of the array in some languages, you must
instead
 * have the result be placed in the first part of the array nums. More formally, if there are
 * k elements after removing the duplicates, then the first k elements of nums should hold the
 * final result. It does not matter what you leave beyond the first k elements.
 *
 * Return k after placing the final result in the first k slots of nums.
 *
 * Do not allocate extra space for another array. You must do this by modifying the input
 * array in-place with O(1) extra memory.
 */

/**
 * @param {number[]} nums
 * @param {number} val
 * @return {number}
 */
var removeElement = function(nums, val) {
  for (let i = nums.length - 1; i > -1; i--) {
    if (nums[i] === val) nums.splice(i, 1);
  }
};
```

0028 Implement Strstr

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 28. Implement strStr()
 * https://leetcode.com/problems/implement-strstr/
 * Difficulty: Easy
 *
 * Return the index of the first occurrence of needle in haystack, or -1 if needle
 * is not part of haystack.
 *
 * Clarification:
 * - What should we return when needle is an empty string? This is a great question
 *   to ask during an interview.
 *
 * - For the purpose of this problem, we will return 0 when needle is an empty string.
 *   This is consistent to C's strstr() and Java's indexOf().
 */

/**
 * @param {string} haystack
 * @param {string} needle
 * @return {number}
 */
```

```
var strStr = function(haystack, needle) {
  if (!needle) return 0;
  const split = haystack.split(needle);
  return split.length > 1 ? split[0].length : -1;
};
```

0029 Divide Two Integers

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 29. Divide Two Integers
 * https://leetcode.com/problems/divide-two-integers/
 * Difficulty: Medium
 *
 * Given two integers dividend and divisor, divide two integers without using multiplication,
 * division, and mod operator.
 *
 * The integer division should truncate toward zero, which means losing its fractional part.
 * For example, 8.345 would be truncated to 8, and -2.7335 would be truncated to -2.
 *
 * Return the quotient after dividing dividend by divisor.
 *
 * Note: Assume we are dealing with an environment that could only store integers within the
 * 32-bit signed integer range: [−231, 231 − 1]. For this problem, if the quotient is strictly
 * greater than 231 - 1, then return 231 - 1, and if the quotient is strictly less than -231,
 * then return -231.
 */

/**
 * @param {number} dividend
 * @param {number} divisor
 * @return {number}
 */
var divide = function(dividend, divisor) {
  if (divisor === -1 && dividend === Math.pow(-2, 31)) {
    return Math.pow(2, 31) - 1;
  }
  const isNegative = dividend > 0 ^ divisor > 0;
  let result = 0;

  dividend = Math.abs(dividend);
  subtract(Math.abs(divisor), 1);

  function subtract(n, quotient) {
    if (dividend > n) {
      subtract(n * 2, quotient * 2);
    }
    if (dividend >= n) {
      dividend -= n;
      result += quotient;
    }
  }

  return isNegative ? -result : result;
};
```

0030 Substring With Concatenation Of All Words

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 30. Substring with Concatenation of All Words
 * https://leetcode.com/problems/substring-with-concatenation-of-all-words/
 * Difficulty: Hard
```

```
 *
 * You are given a string s and an array of strings words. All the strings of words are of the
 same
 * length.
 *
 * A concatenated substring in s is a substring that contains all the strings of any
permutation of
 * words concatenated.
 *
 * - For example, if words = ["ab","cd","ef"], then "abcdef", "abefcd", "cdabef", "cdefab",
"efabcd",
 * and "efcdab" are all concatenated strings. "acdbef" is not a concatenated substring because
 it is
 * not the concatenation of any permutation of words.
 *
 * Return the starting indices of all the concatenated substrings in s. You can return the
answer in
 * any order.
 */

/**
 * @param {string} s
 * @param {string[]} words
 * @return {number[]}
 */
var findSubstring = function(s, words) {
  if (!words || words.length === 0) {
    return [];
  }

  const result = [];
  const map = {};
  const count = words[0].length;

  words.forEach(word => map[word] = ~~map[word] + 1);

  for (let i = 0; i < s.length - words.length * count + 1; i++) {
    const temp = Object.assign({}, map);

    for (let j = i; j < i + words.length * count; j += count) {
      const key = s.slice(j, j + count);
      if (!temp[key]) {
        break;
      } else if (--temp[key] === 0) {
        delete temp[key];
      }
    }

    if (Object.keys(temp).length === 0) {
      result.push(i);
    }
  }

  return result;
};
```

0031 Next Permutation

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 31. Next Permutation
 * https://leetcode.com/problems/next-permutation/
 * Difficulty: Medium
 *
 * Implement next permutation, which rearranges numbers into the
 * lexicographically next greater permutation of numbers.
 *
 * If such an arrangement is not possible, it must rearrange it
 * as the lowest possible order (i.e., sorted in ascending order).
```

```
 *
 * The replacement must be in place and use only constant extra memory.
 */

/**
 * @param {number[]} nums
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var nextPermutation = function(nums) {
  let i = nums.length - 2;

  while (i >= 0 && nums[i + 1] <= nums[i]) {
    i--;
  }

  if (i >= 0) {
    let cursor = nums.length - 1;
    while (nums[cursor] <= nums[i]) {
      cursor--;
    }
    swap(nums, i, cursor);
  }

  reverse(nums, i + 1);
};

function reverse(nums, start) {
  let i = start;
  let j = nums.length - 1;

  while (i < j) {
    swap(nums, i, j);
    i++;
    j--;
  }
}

function swap(nums, i, j) {
  const temp = nums[i];
  nums[i] = nums[j];
  nums[j] = temp;
}
```

0032 Longest Valid Parentheses

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 32. Longest Valid Parentheses
 * https://leetcode.com/problems/longest-valid-parentheses/
 * Difficulty: Hard
 *
 * Given a string containing just the characters '(' and ')', return the
 * length of the longest valid (well-formed) parentheses substring.
 */

/**
 * @param {string} s
 * @return {number}
 */
var longestValidParentheses = function(s) {
  if (!s.length) {
    return 0;
  }

  const stack = [-1];
  let max = 0;

  for (let i = 0; i < s.length; i++) {
    if (s[i] === '(') {
```

```
      stack.push(i);
    } else {
      stack.pop();
      if (!stack.length) {
        stack.push(i);
      } else {
        max = Math.max(max, i - stack[stack.length - 1]);
      }
    }
  }

  return max;
};
```

0033 Search In Rotated Sorted Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 33. Search in Rotated Sorted Array
 * https://leetcode.com/problems/search-in-rotated-sorted-array/
 * Difficulty: Medium
 *
 * There is an integer array nums sorted in ascending order (with distinct values).
 *
 * Prior to being passed to your function, nums is possibly rotated at an unknown pivot
 * index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1],
 * ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7]
 * might be rotated at pivot index 3 and become [4,5,6,7,0,1,2].
 *
 * Given the array nums after the possible rotation and an integer target, return the index
 * of target if it is in nums, or -1 if it is not in nums.
 *
 * You must write an algorithm with O(log n) runtime complexity.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var search = function(nums, target) {
  let start = 0;
  let end = nums.length;

  while (start < end) {
    const i = Math.floor((start + end) / 2);
    const middle = nums[i] < nums[0] === target < nums[0]
      ? nums[i]
      : target < nums[0] ? -Infinity : Infinity;

    if (middle < target) {
      start = i + 1;
    } else if (middle > target) {
      end = i;
    } else {
      return i;
    }
  }

  return -1;
};
```

0034 Find First And Last Position Of Element In Sorted Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 34. Find First and Last Position of Element in Sorted Array
 * https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/
 * Difficulty: Medium
 *
 * Given an array of integers nums sorted in non-decreasing order, find the starting and
 * ending position of a given target value.
 *
 * If target is not found in the array, return [-1, -1].
 *
 * You must write an algorithm with O(log n) runtime complexity.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
 */
var searchRange = function(nums, target) {
  let start = 0;
  let end = nums.length - 1;

  while (start <= end) {
    const middle = Math.floor((start + end) / 2);
    if (target > nums[middle]) {
      start = middle + 1;
    } else {
      end = middle - 1;
    }
  }

  if (nums[start] !== target) {
    return [-1, -1];
  }

  while (nums[start - 1] === target) start--;
  while (nums[end + 1] === target) end++;

  return [start, end];
};
```

0035 Search Insert Position

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 35. Search Insert Position
 * https://leetcode.com/problems/search-insert-position/
 * Difficulty: Easy
 *
 * Given a sorted array of distinct integers and a target value, return the
 * index if the target is found. If not, return the index where it would be
 * if it were inserted in order.
 *
 * You must write an algorithm with O(log n) runtime complexity.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var searchInsert = function(nums, target) {
  let start = 0;
  let end = nums.length - 1;

  while (start <= end) {
```

```
    const middle = Math.floor((start + end) / 2);
    if (target > nums[middle]) {
      start = middle + 1;
    } else {
      end = middle - 1;
    }
  }

  return start;
};
```

0036 Valid Sudoku

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 36. Valid Sudoku
 * https://leetcode.com/problems/valid-sudoku/
 * Difficulty: Medium
 *
 * Determine if a 9x9 Sudoku board is valid.
 * Only the filled cells need to be validated according to the following rules:
 *
 * - Each row must contain the digits 1-9 without repetition.
 * - Each column must contain the digits 1-9 without repetition.
 * - Each of the 9 3x3 sub-boxes of the grid must contain the digits 1-9 without repetition.
 *
 * The Sudoku board could be partially filled, where empty cells are filled with the character
 '.'.
 */

/**
 * @param {character[][]} board
 * @return {boolean}
 */
var isValidSudoku = function(board) {
  const rows = new Array(9).fill().map(() => new Set());
  const columns = new Array(9).fill().map(() => new Set());
  const boxes = new Array(9).fill().map(() => new Set());

  for (let i = 0; i < 9; i++) {
    for (let j = 0; j < 9; j++) {
      const value = board[i][j];
      const k = Math.floor(i / 3) * 3 + Math.floor(j / 3);
      const options = [rows[i], columns[j], boxes[k]];

      if (options.some(option => option.has(value))) return false;
      if (value !== '.') options.forEach(option => option.add(value));
    }
  }

  return true;
};
```

0037 Sudoku Solver

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 37. Sudoku Solver
 * https://leetcode.com/problems/sudoku-solver/
 * Difficulty: Hard
 *
 * Write a program to solve a Sudoku puzzle by filling the empty cells.
 *
```

```
 * A sudoku solution must satisfy all of the following rules:
 * - Each of the digits 1-9 must occur exactly once in each row.
 * - Each of the digits 1-9 must occur exactly once in each column.
 * - Each of the digits 1-9 must occur exactly once in each of the
 *   9 3x3 sub-boxes of the grid.
 * - The '.' character indicates empty cells.
 */

/**
 * @param {character[][]} board
 * @return {void} Do not return anything, modify board in-place instead.
 */
var solveSudoku = function(board) {
  for (let i = 0; i < board.length; i++) {
    for (let j = 0; j < board.length; j++) {
      if (board[i][j] === '.') {
        for (let k = 1; k < 10; k++) {
          if (isValid(board, i, j, k.toString())) {
            board[i][j] = k.toString();
            const solution = solveSudoku(board);
            if (solution !== false) {
              return solution;
            }
            board[i][j] = '.';
          }
        }
        return false;
      }
    }
  }

  return board;
};

function isValid(board, i, j, k) {
  for (let index = 0; index < board.length; index++) {
    if (board[i][index] === k) return false;
    if (board[index][j] === k) return false;
    const [x, y] = [
      3 * Math.floor(i / 3) + Math.floor(index / 3),
      3 * Math.floor(j / 3) + index % 3,
    ];
    if (board[x][y] === k) {
      return false;
    }
  }

  return true;
}
```

0038 Count And Say

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 38. Count and Say
 * https://leetcode.com/problems/count-and-say/
 * Difficulty: Medium
 *
 * The count-and-say sequence is a sequence of digit strings defined by the recursive formula:
 * - countAndSay(1) = "1"
 * - countAndSay(n) is the way you would "say" the digit string from countAndSay(n-1), which
is
 *   then converted into a different digit string.
 *
 * To determine how you "say" a digit string, split it into the minimal number of substrings
such
 * that each substring contains exactly one unique digit. Then for each substring, say the
number
 * of digits, then say the digit. Finally, concatenate every said digit.
```

```
 *
 * For example, the saying and conversion for digit string "3322251":
 * Given a positive integer n, return the nth term of the count-and-say sequence.
 */

/**
 * @param {number} n
 * @return {string}
 */
var countAndSay = function(n) {
  let result = '1';

  for (let i = 1; i < n; i++) {
    result = result.replace(/((\d)\2*)/g, '$1—')
                   .split('—')
                   .map(s => s ? `${s.length}${s[0]}` : '')
                   .join('');
  }

  return result;
};
```

0039 Combination Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 39. Combination Sum
 * https://leetcode.com/problems/combination-sum/
 * Difficulty: Medium
 *
 * Given an array of distinct integers candidates and a target integer target,
 * return a list of all unique combinations of candidates where the chosen
 * numbers sum to target. You may return the combinations in any order.
 *
 * The same number may be chosen from candidates an unlimited number of times.
 * Two combinations are unique if the frequency of at least one of the chosen
 * numbers is different.
 *
 * It is guaranteed that the number of unique combinations that sum up to target
 * is less than 150 combinations for the given input.
 */

/**
 * @param {number[]} candidates
 * @param {number} target
 * @return {number[][]}
 */
var combinationSum = function(candidates, target) {
  const result = [];
  backtrack(result, candidates, target);
  return result;
};

function backtrack(result, candidates, target, combination = [], offset = 0) {
  if (target < 0) {
    return;
  } else if (target === 0) {
    result.push(combination);
  } else {
    for (let i = offset; i < candidates.length; i++) {
      backtrack(result, candidates, target - candidates[i], [...combination, candidates[i]],
i);
    }
  }
}
```

0040 Combination Sum Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 40. Combination Sum II
 * https://leetcode.com/problems/combination-sum-ii/
 * Difficulty: Medium
 *
 * Given a collection of candidate numbers (candidates) and a target number (target),
 * find all unique combinations in candidates where the candidate numbers sum to target.
 *
 * Each number in candidates may only be used once in the combination.
 *
 * Note: The solution set must not contain duplicate combinations.
 */

/**
 * @param {number[]} candidates
 * @param {number} target
 * @return {number[][]}
 */
var combinationSum2 = function(candidates, target) {
  const result = [];
  candidates.sort((a, b) => a - b);
  backtrack(result, candidates, target);
  return result;
};

function backtrack(result, candidates, target, combination = [], offset = 0) {
  if (target < 0) {
    return;
  } else if (target === 0) {
    result.push(combination);
  } else {
    for (let i = offset; i < candidates.length; i++) {
      if (i > offset && candidates[i] === candidates[i - 1]) continue;
      backtrack(result, candidates, target - candidates[i], [...combination, candidates[i]], i
 + 1);
    }
  }
}
```

0041 First Missing Positive

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 41. First Missing Positive
 * https://leetcode.com/problems/first-missing-positive/
 * Difficulty: Hard
 *
 * Given an unsorted integer array nums, return the smallest missing positive integer.
 *
 * You must implement an algorithm that runs in O(n) time and uses constant extra space.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var firstMissingPositive = function(nums) {
  let i = 0;

  while (i < nums.length) {
```

```
    if (nums[i] > 0 && nums[i] <= nums.length && nums[nums[i] - 1] !== nums[i]) {
      [nums[nums[i] - 1], nums[i]] = [nums[i], nums[nums[i] - 1]];
    } else {
      i++;
    }
  }

  for (i = 0; i < nums.length; i++) {
    if (nums[i] !== i + 1) {
      return i + 1;
    }
  }

  return i + 1;
};
```

0042 Trapping Rain Water

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 42. Trapping Rain Water
 * https://leetcode.com/problems/trapping-rain-water/
 * Difficulty: Hard
 *
 * Given n non-negative integers representing an elevation map where the
 * width of each bar is 1, compute how much water it can trap after raining.
 */

/**
 * @param {number[]} height
 * @return {number}
 */
var trap = function(height) {
  let result = 0;

  for (let left = 0, right = height.length - 1, maxL = 0, maxR = 0; left < right;) {
    maxL = Math.max(maxL, height[left]);
    maxR = Math.max(maxR, height[right]);
    result += maxL < maxR ? maxL - height[left++] : maxR - height[right--];
  }

  return result;
};
```

0043 Multiply Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 43. Multiply Strings
 * https://leetcode.com/problems/multiply-strings/
 * Difficulty: Medium
 *
 * Given two non-negative integers num1 and num2 represented as strings,
 * return the product of num1 and num2, also represented as a string.
 *
 * - The length of both num1 and num2 is < 110.
 * - Both num1 and num2 contain only digits 0-9.
 * - Both num1 and num2 do not contain any leading zero, except the number 0 itself.
 * - You must not use any built-in BigInteger library or convert the inputs to integer
directly.
 */

/**
```

```
 * @param {string} num1
 * @param {string} num2
 * @return {string}
 */
var multiply = function(num1, num2) {
  const result = new Array(num1.length + num2.length).fill(0);
  for (let i = num1.length - 1; i >= 0; i--) {
    for (let j = num2.length - 1; j >= 0; j--) {
      const sum = num1[i] * num2[j] + result[i + j + 1];
      result[i + j] += Math.floor(sum / 10);
      result[i + j + 1] = sum % 10;
    }
  }
  return result.join('').replace(/^0+(?!$)/, '');
};

// one-liner alternative that breaks the rules:
/**
 * @param {string} num1
 * @param {string} num2
 * @return {string}
 */
var multiply = function(num1, num2) {
  return ((BigInt(num1) * BigInt(num2))).toString();
};
```

0044 Wildcard Matching

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 44. Wildcard Matching
 * https://leetcode.com/problems/wildcard-matching/
 * Difficulty: Hard
 *
 * Given an input string (s) and a pattern (p), implement wildcard pattern
 * matching with support for '?' and '*' where:
 *
 * - '?' Matches any single character.
 * - '*' Matches any sequence of characters (including the empty sequence).
 *
 * The matching should cover the entire input string (not partial).
 */

/**
 * @param {string} s
 * @param {string} p
 * @return {boolean}
 */
var isMatch = function(s, p) {
  let i = 0;
  let j = 0;
  let start = -1;
  let offset = -1;

  while (i < s.length) {
    if (j < p.length && s[i] === p[j] || p[j] === '?') {
      i++;
      j++;
    } else if (j < p.length && p[j] === '*') {
      start = j;
      offset = i;
      j++;
    } else if (start === -1) {
      return false;
    } else {
      j = start + 1;
      i = offset + 1;
      offset = i;
    }
```

```
  }

  for (let index = j; index < p.length; index++) {
    if (p[index] !== '*') {
      return false;
    }
  }

  return true;
};
```

0045 Jump Game Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 45. Jump Game II
 * https://leetcode.com/problems/jump-game-ii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of integers nums of length n.
 * You are initially positioned at nums[0].
 *
 * Each element nums[i] represents the maximum length of a forward
 * jump from index i. In other words, if you are at nums[i], you
 * can jump to any nums[i + j] where:
 * - 0 <= j <= nums[i] and
 * - i + j < n
 *
 * Return the minimum number of jumps to reach nums[n - 1].
 * The test cases are generated such that you can reach nums[n - 1].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var jump = function(nums) {
  let result = 0;
  let max = 0;
  let previous = 0;

  for (let index = 0; index < nums.length - 1; index++) {
    max = Math.max(max, index + nums[index]);
    if (index === previous) {
      result++;
      previous = max;
    }
  }

  return result;
};
```

0046 Permutations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 46. Permutations
 * https://leetcode.com/problems/permutations/
 * Difficulty: Medium
 *
 * Given an array nums of distinct integers, return all the possible permutations.
 * You can return the answer in any order.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var permute = function(nums) {
  const result = [];
  backtrack(result, nums);
  return result;
};

function backtrack(result, nums, order = []) {
  if (!nums.length) {
    result.push(order);
  } else {
    for (let i = 0; i < nums.length; i++) {
      backtrack(
        result,
        [...nums.slice(0, i), ...nums.slice(i + 1)],
        [...order, nums[i]]
      );
    }
  }
}
```

0047 Permutations Ii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 47. Permutations II
 * https://leetcode.com/problems/permutations-ii/
 * Difficulty: Medium
 *
 * Given a collection of numbers, nums, that might contain duplicates,
 * return all possible unique permutations in any order.
 */

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var permuteUnique = function(nums) {
  const result = new Set();
  backtrack(result, nums);
  return Array.from(result).map(s => s.split(','));
};

function backtrack(result, nums, order = []) {
  if (!nums.length) {
    result.add(order.join());
  } else {
    for (let i = 0; i < nums.length; i++) {
      backtrack(
        result,
        [...nums.slice(0, i), ...nums.slice(i + 1)],
        [...order, nums[i]],
      );
    }
  }
}
```

0048 Rotate Image

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 48. Rotate Image
 * https://leetcode.com/problems/rotate-image/
 * Difficulty: Medium
 *
 * You are given an n x n 2D matrix representing an image, rotate the image by
 * 90 degrees (clockwise).
 *
 * You have to rotate the image in-place, which means you have to modify the input
 * 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.
 */

/**
 * @param {number[][]} matrix
 * @return {void} Do not return anything, modify matrix in-place instead.
 */
var rotate = function(matrix) {
  matrix = matrix.reverse();

  for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < i; j++) {
      [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];
    }
  }

  return matrix;
};
```

0049 Group Anagrams

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 49. Group Anagrams
 * https://leetcode.com/problems/group-anagrams/
 * Difficulty: Medium
 *
 * Given an array of strings `strs`, group the anagrams together. You can return the
 * answer in any order.
 *
 * An Anagram is a word or phrase formed by rearranging the letters of a different
 * word or phrase, typically using all the original letters exactly once.
 */

/**
 * @param {string[]} strs
 * @return {string[][]}
 */
var groupAnagrams = function(strs) {
  const map = {};

  strs.forEach(str => {
    const key = [...str].sort();
    map[key] = map[key] ? [...map[key], str] : [str];
  });

  return Object.values(map);
};
```

0050 Powx N

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 50. Pow(x, n)
 * https://leetcode.com/problems/powx-n/
 * Difficulty: Medium
 *
 * Implement `pow(x, n)`, which calculates `x` raised to the power `n` (i.e., `x^n`).
 */

/**
 * @param {number} x
 * @param {number} n
 * @return {number}
 */
var myPow = function(x, n) {
  if (n === 0) return 1;
  if (n === 1) return x;
  if (n < 0) return 1 / myPow(x, -n);
  if (n % 2 === 0) return myPow(x * x, n / 2);

  return x * myPow(x * x, (n - 1) / 2);
};
```

0051 N Queens

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 51. N-Queens
 * https://leetcode.com/problems/n-queens/
 * Difficulty: Hard
 *
 * The n-queens puzzle is the problem of placing n queens on an n x n
 * chessboard such that no two queens attack each other.
 *
 * Given an integer n, return all distinct solutions to the n-queens
 * puzzle. You may return the answer in any order.
 *
 * Each solution contains a distinct board configuration of the n-queens'
 * placement, where 'Q' and '.' both indicate a queen and an empty space,
 * respectively.
 */

/**
 * @param {number} n
 * @return {string[][]}
 */
var solveNQueens = function(n) {
  const result = [];
  backtrack(result, n);
  return result;
};

function backtrack(result, n, board = [], size = 0) {
  if (n === size) {
    result.push(board.map(s => `${'.'.repeat(s)}Q${'.'.repeat(n - s - 1)}`));
  } else {
    for (let rows = 0; rows < n; rows++) {
      const isValid = !board.some((i, j) => {
        return i === rows || i === rows + size - j || i === rows - size + j;
      });
      if (isValid) {
        board.push(rows);
        backtrack(result, n, board, size + 1);
        board.pop();
      }
    }
  }
}
```

0052 N Queens Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 52. N-Queens II
 * https://leetcode.com/problems/n-queens-ii/
 * Difficulty: Hard
 *
 * The n-queens puzzle is the problem of placing n queens on an n x n
 * chessboard such that no two queens attack each other.
 *
 * Given an integer n, return the number of distinct solutions to the
 * n-queens puzzle.
 */

/**
 * @param {number} n
 * @return {number}
 */
var totalNQueens = function(n) {
  const result = [];
  backtrack(result, n);
  return result.length;
};

function backtrack(result, n, board = [], size = 0) {
  if (n === size) {
    result.push(board.map(s => `${'.'.repeat(s)}Q${'.'.repeat(n - s - 1)}`));
  } else {
    for (let rows = 0; rows < n; rows++) {
      const isValid = !board.some((i, j) => {
        return i === rows || i === rows + size - j || i === rows - size + j;
      });
      if (isValid) {
        board.push(rows);
        backtrack(result, n, board, size + 1);
        board.pop();
      }
    }
  }
}
```

0053 Maximum Subarray

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 53. Maximum Subarray
 * https://leetcode.com/problems/maximum-subarray/
 * Difficulty: Easy
 *
 * Given an integer array `nums`, find the contiguous subarray (containing at
 * least one number) which has the largest sum and return its sum.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubArray = function(nums) {
  for (let i = 1; i < nums.length; i++) {
    if (nums[i - 1] > 0) {
```

```
      nums[i] += nums[i - 1];
    }
  }

  return Math.max(...nums);
};
```

0054 Spiral Matrix

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 54. Spiral Matrix
 * https://leetcode.com/problems/spiral-matrix/
 * Difficulty: Medium
 *
 * Given an `m x n` `matrix`, return all elements of the `matrix` in spiral order.
 */

/**
 * @param {number[][]} matrix
 * @return {number[]}
 */
var spiralOrder = function(matrix) {
  const output = [];
  let top = 0;
  let right = matrix[0].length - 1;
  let bottom = matrix.length - 1;
  let left = 0;

  while (output.length < matrix.length * matrix[0].length) {
    for (let i = left; i <= right; i++) {
      output.push(matrix[top][i]);
    }
    top++;

    for (let i = top; i <= bottom; i++) {
      output.push(matrix[i][right]);
    }
    right--;

    for (let i = right; top <= bottom && i >= left; i--) {
      output.push(matrix[bottom][i]);
    }
    bottom--;

    for (let i = bottom; left <= right && i >= top; i--) {
      output.push(matrix[i][left]);
    }
    left++;
  }

  return output;
};
```

0055 Jump Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 55. Jump Game
 * https://leetcode.com/problems/jump-game/
 * Difficulty: Medium
 *
 * You are given an integer array nums. You are initially positioned at the
```

```
 * array's first index, and each element in the array represents your maximum
 * jump length at that position.
 *
 * Return true if you can reach the last index, or false otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var canJump = function(nums) {
  let max = nums[0];

  for (let i = 1; i < nums.length; i++) {
    if (max < i) {
      return false;
    }
    max = Math.max(nums[i] + i, max);
  }

  return true;
};
```

0056 Merge Intervals

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 56. Merge Intervals
 * https://leetcode.com/problems/merge-intervals/
 * Difficulty: Medium
 *
 * Given an array of intervals where intervals[i] = [starti, endi], merge all
 * overlapping intervals, and return an array of the non-overlapping intervals
 * that cover all the intervals in the input.
 */

/**
 * @param {number[][]} intervals
 * @return {number[][]}
 */
var merge = function(intervals) {
  intervals.sort(([a], [b]) => a - b);

  const result = [intervals.shift()];
  intervals.forEach(([first, second]) => {
    const [, last] = result[result.length - 1];
    if (first <= last) {
      result[result.length - 1][1] = Math.max(second, last);
    } else {
      result.push([first, second]);
    }
  });

  return result;
};
```

0057 Insert Interval

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 57. Insert Interval
 * https://leetcode.com/problems/insert-interval/
 * Difficulty: Medium
```

```
 *
 * You are given an array of non-overlapping intervals intervals where
 * intervals[i] = [starti, endi] represent the start and the end of the
 * ith interval and intervals is sorted in ascending order by starti.
 * You are also given an interval newInterval = [start, end] that
 * represents the start and end of another interval.
 *
 * Insert newInterval into intervals such that intervals is still sorted
 * in ascending order by starti and intervals still does not have any
 * overlapping intervals (merge overlapping intervals if necessary).
 *
 * Return intervals after the insertion.
 */

/**
 * @param {number[][]} intervals
 * @param {number[]} newInterval
 * @return {number[][]}
 */
var insert = function(intervals, newInterval) {
  const result = [];

  for (let i = 0; i < intervals.length; i++) {
    if (newInterval[1] < intervals[i][0]) {
      result.push(newInterval);
      return [...result, ...intervals.slice(i)];
    } else if (newInterval[0] > intervals[i][1]) {
      result.push(intervals[i]);
    } else {
      newInterval = [
        Math.min(newInterval[0], intervals[i][0]),
        Math.max(newInterval[1], intervals[i][1]),
      ];
    }
  }

  result.push(newInterval);
  return result;
};
```

0058 Length Of Last Word

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 58. Length of Last Word
 * https://leetcode.com/problems/length-of-last-word/
 * Difficulty: Easy
 *
 * Given a string `s` consists of upper/lower-case alphabets
 * and empty space characters ' ', return the length of last
 * word (last word means the last appearing word if we loop
 * from left to right) in the string.
 *
 * If the last word does not exist, return 0.
 *
 * Note: A word is defined as a maximal substring consisting
 * of non-space characters only.
 */

/**
 * @param {string} s
 * @return {number}
 */
var lengthOfLastWord = function(s) {
  return s.trim().split(/\s+/).pop().length;
};
```

0059 Spiral Matrix Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 59. Spiral Matrix II
 * https://leetcode.com/problems/spiral-matrix-ii/
 * Difficulty: Medium
 *
 * Given a positive integer n, generate an n x n matrix filled with
 * elements from 1 to n2 in spiral order.
 */

/**
 * @param {number} n
 * @return {number[][]}
 */
var generateMatrix = function(n) {
  const result = Array.from(new Array(n), () => new Array(n).fill(0));
  let top = 0;
  let right = n - 1;
  let bottom = n - 1;
  let left = 0;

  for (let count = 0; count < (n * n);) {
    for (let i = left; i <= right; i++) {
      result[top][i] = ++count;
    }
    top++;

    for (let i = top; i <= bottom; i++) {
      result[i][right] = ++count;
    }
    right--;

    for (let i = right; top <= bottom && i >= left; i--) {
      result[bottom][i] = ++count;
    }
    bottom--;

    for (let i = bottom; left <= right && i >= top; i--) {
      result[i][left] = ++count;
    }
    left++;
  }

  return result;
};
```

0060 Permutation Sequence

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 60. Permutation Sequence
 * https://leetcode.com/problems/permutation-sequence/
 * Difficulty: Hard
 *
 * The set [1, 2, 3, ..., n] contains a total of n! unique permutations.
 *
 * By listing and labeling all of the permutations in order, we get the
 * following sequence for n = 3:
 * - "123"
 * - "132"
 * - "213"
```

```
 * - "231"
 * - "312"
 * - "321"
 *
 * Given n and k, return the kth permutation sequence.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {string}
 */
var getPermutation = function(n, k) {
  const factorial = [1];
  for (let i = 1; i < n; i++) {
    factorial[i] = factorial[i-1] * i;
  }

  const values = new Array(n).fill(0).map((_, i) => i + 1);
  let result = '';
  k--;

  for (let i = n - 1; i >= 0; i--) {
    const index = Math.floor(k / factorial[i]);
    k = k % factorial[i];
    result += values[index];
    values.splice(index, 1);
  }

  return result;
};
```

0061 Rotate List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 61. Rotate List
 * https://leetcode.com/problems/rotate-list/
 * Difficulty: Medium
 *
 * Given the head of a linked list, rotate the list to the right by k places.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} k
 * @return {ListNode}
 */
var rotateRight = function(head, k) {
  if (!head || !head.next || !k) return head;
  let tailCopy = head;
  let tail = head;
  let count = 1;
  while (tail.next) {
    tail = tail.next;
    count++;
  }
  tail.next = head;
  for (let i = 1; i < count - k % count; i++) {
    tailCopy = tailCopy.next;
  }
  const result = tailCopy.next;
```

```
  tailCopy.next = null;
  return result;
};
```

0062 Unique Paths

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 62. Unique Paths
 * https://leetcode.com/problems/unique-paths/
 * Difficulty: Medium
 *
 * There is a robot on an m x n grid. The robot is initially located at the top-left
 * corner (i.e., grid[0][0]). The robot tries to move to the bottom-right corner
 * (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any
 * point in time.
 *
 * Given the two integers m and n, return the number of possible unique paths that
 * the robot can take to reach the bottom-right corner.
 */

/**
 * @param {number} m
 * @param {number} n
 * @return {number}
 */
var uniquePaths = function(m, n) {
  const row = new Array(n);

  for (let i = 0; i < n; i++) {
    row[i] = 1;
  }

  for (let i = 1; i < m; i++) {
    for (let j = 1; j < n; j++) {
      row[j] += row[j - 1];
    }
  }

  return row[n - 1];
};
```

0063 Unique Paths Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 63. Unique Paths II
 * https://leetcode.com/problems/unique-paths-ii/
 * Difficulty: Medium
 *
 * You are given an m x n integer array grid. There is a robot initially located at
 * the top-left corner (i.e., grid[0][0]). The robot tries to move to the bottom-right
 * corner (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at
 * any point in time.
 *
 * An obstacle and space are marked as 1 or 0 respectively in grid. A path that the
 * robot takes cannot include any square that is an obstacle.
 *
 * Return the number of possible unique paths that the robot can take to reach the
 * bottom-right corner.
 *
 * The testcases are generated so that the answer will be less than or equal to 2 * 109.
 */
```

```
/**
 * @param {number[][]} grid
 * @return {number}
 */
var uniquePathsWithObstacles = function(grid) {
  const cache = new Array(grid.length).fill(0).map(v => new Array(grid[0].length).fill(0));

  function traverse(x, y) {
    if (grid[x] === undefined || grid[x][y] === undefined || grid[x][y] === 1) return 0;
    if (x === grid.length - 1 && y === grid[0].length - 1) return 1;
    if (!cache[x][y]) {
      cache[x][y] = traverse(x + 1, y) + traverse(x, y + 1);
    }
    return cache[x][y];
  }

  return traverse(0, 0);
};
```

0064 Minimum Path Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 64. Minimum Path Sum
 * https://leetcode.com/problems/minimum-path-sum/
 * Difficulty: Medium
 *
 * Given a m x n grid filled with non-negative numbers, find a path from
 * top left to bottom right, which minimizes the sum of all numbers along
 * its path.
 *
 * Note: You can only move either down or right at any point in time.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minPathSum = function(grid) {
  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[0].length; j++) {
      if (i === 0 && j === 0) grid[i][j] = grid[i][j];
      else if (i === 0 && j !== 0) grid[i][j] = grid[i][j] + grid[i][j - 1];
      else if (i !== 0 && j === 0) grid[i][j] = grid[i][j] + grid[i - 1][j];
      else grid[i][j] = grid[i][j] + Math.min(grid[i - 1][j], grid[i][j - 1]);
    }
  }

  return grid[grid.length - 1][grid[0].length - 1];
};
```

0065 Valid Number

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 65. Valid Number
 * https://leetcode.com/problems/valid-number/
 * Difficulty: Hard
 *
 * Given a string s, return whether s is a valid number.
 *
 * For example, all the following are valid numbers: "2", "0089", "-0.1", "+3.14", "4.",
```

```
"-.9",
 * "2e10", "-90E3", "3e+7", "+6e-1", "53.5e93", "-123.456e789", while the following are not
 * valid numbers: "abc", "1a", "1e", "e3", "99e2.5", "--6", "-+3", "95a54e53".
 *
 * Formally, a valid number is defined using one of the following definitions:
 * - An integer number followed by an optional exponent.
 * - A decimal number followed by an optional exponent.
 *
 * An integer number is defined with an optional sign '-' or '+' followed by digits.
 * A decimal number is defined with an optional sign '-' or '+' followed by one of the
 * following definitions:
 * - Digits followed by a dot '.'.
 * - Digits followed by a dot '.' followed by digits.
 * - A dot '.' followed by digits.
 * - An exponent is defined with an exponent notation 'e' or 'E' followed by an integer
number.
 *
 * The digits are defined as one or more digits.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var isNumber = function(s) {
  return s.trim().length && !isNaN(s.trim()) && !s.includes('Infinity');
};
```

0066 Plus One

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 66. Plus One
 * https://leetcode.com/problems/plus-one/
 * Difficulty: Easy
 *
 * Given a non-empty array of digits representing a non-negative integer,
 * plus one to the integer.
 *
 * The digits are stored such that the most significant digit is at the
 * head of the list, and each element in the array contain a single digit.
 *
 * You may assume the integer does not contain any leading zero,
 * except the number 0 itself.
 */

/**
 * @param {number[]} digits
 * @return {number[]}
 */
var plusOne = function(digits) {
  let i = digits.length - 1;
  digits[i] += 1;
  while (digits[i] > 9) {
    digits[i--] = 0;
    if (digits[i]) {
      digits[i] += 1;
    } else {
      digits.unshift(1);
    }
  }
  return digits;
};

// alternative one-liner that breaks the rules:
/**
 * @param {number[]} digits
 * @return {number[]}
 */
```

```
var plusOne = function(digits) {
  return String(BigInt(digits.join('')) + BigInt(1)).split('');
};
```

0067 Add Binary

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 67. Add Binary
 * https://leetcode.com/problems/add-binary/
 * Difficulty: Easy
 *
 * Given two binary strings, return their sum (also a binary string).
 *
 * The input strings are both non-empty and contains only characters 1 or 0.
 */

/**
 * @param {string} a
 * @param {string} b
 * @return {string}
 */
var addBinary = function(a, b) {
  return (BigInt('0b' + a) + BigInt('0b' + b)).toString(2);
};
```

0068 Text Justification

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 68. Text Justification
 * https://leetcode.com/problems/text-justification/
 * Difficulty: Hard
 *
 * Given an array of strings words and a width maxWidth, format the text such that each
 * line has exactly maxWidth characters and is fully (left and right) justified.
 *
 * You should pack your words in a greedy approach; that is, pack as many words as you
 * can in each line. Pad extra spaces ' ' when necessary so that each line has exactly
 * maxWidth characters.
 *
 * Extra spaces between words should be distributed as evenly as possible. If the number
 * of spaces on a line does not divide evenly between words, the empty slots on the left
 * will be assigned more spaces than the slots on the right.
 *
 * For the last line of text, it should be left-justified, and no extra space is inserted
 * between words.
 *
 * Note:
 * - A word is defined as a character sequence consisting of non-space characters only.
 * - Each word's length is guaranteed to be greater than 0 and not exceed maxWidth.
 * - The input array words contains at least one word.
 */

/**
 * @param {string[]} words
 * @param {number} maxWidth
 * @return {string[]}
 */
var fullJustify = function(words, maxWidth) {
  const result = [[]];
  result[0].count = 0;
```

```
  for (const word of words) {
    let row = result[result.length - 1];
    if (row.length && row.count + row.length + word.length > maxWidth) {
      result.push([]);
      row = result[result.length - 1];
      row.count = 0;
    }
    row.push(word);
    row.count += word.length;
  }

  for (let i = 0; i < result.length; i++) {
    const row = result[i];
    if (row.length === 1 || i === result.length - 1) {
      result[i] = row.join(' ') + ' '.repeat(maxWidth - row.count - row.length + 1);
      continue;
    }

    const min = ' '.repeat(Math.floor((maxWidth - row.count) / (row.length - 1)));
    for (let j = 1; j < row.length; j++) {
      const delimiter = j <= (maxWidth - row.count) % (row.length - 1) ? ' ' : '';
      row[0] += min + delimiter + row[j];
    }

    result[i] = row[0];
  }

  return result;
};
```

0069 Sqrtx

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 69. Sqrt(x)
 * https://leetcode.com/problems/sqrtx/
 * Difficulty: Medium
 *
 * Given a non-negative integer x, return the square root of x rounded down to the nearest
 * integer. The returned integer should be non-negative as well.
 *
 * You must not use any built-in exponent function or operator.
 */

/**
 * @param {number} x
 * @return {number}
 */
var mySqrt = function(x) {
  let result = 1;

  while (result * result <= x) {
    result++;
  }

  return result - 1;
};
```

0070 Climbing Stairs

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 70. Climbing Stairs
```

```
 * https://leetcode.com/problems/climbing-stairs/
 * Difficulty: Easy
 *
 * You are climbing a staircase. It takes n steps to reach the top.
 *
 * Each time you can either climb 1 or 2 steps. In how many distinct
 * ways can you climb to the top?
 */

/**
 * @param {number} n
 * @return {number}
 */
var climbStairs = function(n) {
  const cache = new Map();
  const memo = n => {
    if (n < 4) return n;
    if (!cache.has(n)) {
      cache.set(n, memo(n - 2) + memo(n - 1));
    }
    return cache.get(n);
  };
  return memo(n);
};
```

0071 Simplify Path

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 71. Simplify Path
 * https://leetcode.com/problems/simplify-path/
 * Difficulty: Medium
 *
 * You are given an absolute path for a Unix-style file system, which always begins
 * with a slash '/'. Your task is to transform this absolute path into its simplified
 * canonical path.
 *
 * The rules of a Unix-style file system are as follows:
 * - A single period '.' represents the current directory.
 * - A double period '..' represents the previous/parent directory.
 * - Multiple consecutive slashes such as '//' and '///' are treated as a single slash '/'.
 * - Any sequence of periods that does not match the rules above should be treated as a
 *   valid directory or file name. For example, '...' and '....' are valid directory or
 *   file names.
 *
 * The simplified canonical path should follow these rules:
 * - The path must start with a single slash '/'.
 * - Directories within the path must be separated by exactly one slash '/'.
 * - The path must not end with a slash '/', unless it is the root directory.
 * - The path must not have any single or double periods ('.' and '..') used to denote
 *   current or parent directories.
 *
 * Return the simplified canonical path.
 */

/**
 * @param {string} path
 * @return {string}
 */
var simplifyPath = function(path) {
  const stack = [];
  for (const segment of path.split('/')) {
    if (segment === '.' || segment === '') continue;
    if (segment === '..') stack.pop();
    else stack.push(segment);
  }
  return `/${stack.join('/')}`;
};
```

0072 Edit Distance

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 72. Edit Distance
 * https://leetcode.com/problems/edit-distance/
 * Difficulty: Medium
 *
 * Given two strings word1 and word2, return the minimum number of operations required to
 * convert word1 to word2.
 *
 * You have the following three operations permitted on a word:
 * - Insert a character
 * - Delete a character
 * - Replace a character
 */

/**
 * @param {string} word1
 * @param {string} word2
 * @return {number}
 */
var minDistance = function(word1, word2) {
  const cache = new Array(word1.length + 1).fill(0).map(() => new Array(word2.length + 1));

  for (let i = 0; i <= word1.length; i++) {
    for (let j = 0; j <= word2.length; j++) {
      if (i === 0) {
        cache[0][j] = j;
      } else if (j === 0) {
        cache[i][0] = i;
      } else if (word1[i - 1] == word2[j - 1]) {
        cache[i][j] = cache[i - 1][j - 1];
      } else {
        cache[i][j] = Math.min(cache[i][j - 1], cache[i - 1][j - 1], cache[i - 1][j]) + 1;
      }
    }
  }

  return cache[word1.length][word2.length];
};
```

0073 Set Matrix Zeroes

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 73. Set Matrix Zeroes
 * https://leetcode.com/problems/set-matrix-zeroes/
 * Difficulty: Medium
 *
 * Given an `m x n` integer `matrix` matrix, if an element is `0`, set its entire
 * row and column to `0`'s, and return the matrix.
 *
 * You must do it in place.
 */

/**
 * @param {number[][]} matrix
 * @return {void} Do not return anything, modify matrix in-place instead.
 */
var setZeroes = function(matrix) {
  const columns = new Set();
  const rows = new Set();
```

```
  matrix.forEach((row, i) => {
    row.forEach((value, j) => {
      if (value === 0) {
        columns.add(i);
        rows.add(j);
      }
    });
  });

  [...columns].forEach(i => matrix[i].forEach((_, j) => matrix[i][j] = 0));
  matrix.forEach((_, i) => [...rows].forEach(j => matrix[i][j] = 0));

  return matrix;
};
```

0074 Search A 2D Matrix

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 74. Search a 2D Matrix
 * https://leetcode.com/problems/search-a-2d-matrix/
 * Difficulty: Medium
 *
 * Write an efficient algorithm that searches for a value in an m x n matrix.
 * This matrix has the following properties:
 *
 * Integers in each row are sorted from left to right.
 * The first integer of each row is greater than the last integer of the previous row.
 */

/**
 * @param {number[][]} matrix
 * @param {number} target
 * @return {boolean}
 */
var searchMatrix = function(matrix, target) {
  return matrix
    .filter(row => row[0] <= target && row[row.length - 1] >= target)
    .find(row => row.includes(target)) !== undefined;
};
```

0075 Sort Colors

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 75. Sort Colors
 * https://leetcode.com/problems/sort-colors/
 * Difficulty: Medium
 *
 * Given an array nums with n objects colored red, white, or blue, sort them in-place so
 * that objects of the same color are adjacent, with the colors in the order red, white,
 * and blue.
 *
 * We will use the integers 0, 1, and 2 to represent the color red, white, and blue,
 * respectively.
 *
 * You must solve this problem without using the library's sort function.
 */

/**
 * @param {number[]} nums
 * @return {void} Do not return anything, modify nums in-place instead.
```

```
 */
var sortColors = function(nums) {
  const count = nums.reduce((a, n) => ++a[n] && a, [0, 0, 0]);
  for (let i = 0, j = 0; i < count.length; i++) {
    for (let k = 0; k < count[i]; k++) {
      nums[j++] = i;
    }
  }
  return nums;
};
```

0076 Minimum Window Substring

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 76. Minimum Window Substring
 * https://leetcode.com/problems/minimum-window-substring/
 * Difficulty: Hard
 *
 * Given two strings s and t of lengths m and n respectively, return the minimum window
substring
 * of s such that every character in t (including duplicates) is included in the window. If
there
 * is no such substring, return the empty string "".
 *
 * The testcases will be generated such that the answer is unique.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {string}
 */
var minWindow = function(s, t) {
  const values = new Array(128).fill(0);
  let [start, end] = [-Infinity, Infinity];

  for (let i = 0; i < t.length; i++) {
    values[t.charCodeAt(i)]++;
  }

  for (let i = 0, j = 0, total = t.length; i < s.length; i++) {
    if (values[s.charCodeAt(i)] > 0) {
      total--;
    }
    values[s.charCodeAt(i)]--;
    while (!total) {
      if (end - start > i - j) {
        [start, end] = [j, i];
      }
      values[s.charCodeAt(j)]++;
      if (values[s.charCodeAt(j)] > 0) {
        total++;
      }
      j++;
    }
  }

  return end !== Infinity ? s.slice(start, end + 1) : '';
};
```

0077 Combinations

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 77. Combinations
 * https://leetcode.com/problems/combinations/
 * Difficulty: Medium
 *
 * Given two integers n and k, return all possible combinations of k numbers
 * out of the range [1, n].
 *
 * You may return the answer in any order.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number[][]}
 */
var combine = function(n, k) {
  const result = [];
  backtrack(result, n, k);
  return result;
};

function backtrack(result, n, k, combination = [], offset = 1) {
  if (combination.length === k) {
    result.push(combination);
  } else {
    while (offset <= n) {
      backtrack(result, n, k, [...combination, offset], ++offset);
    }
  }
}
```

0078 Subsets

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 78. Subsets
 * https://leetcode.com/problems/subsets/
 * Difficulty: Medium
 *
 * Given an integer array nums of unique elements, return all possible subsets (the power
set).
 *
 * The solution set must not contain duplicate subsets. Return the solution in any order.
 */

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var subsets = function(nums) {
  const result = [];
  dfs([], 0);

  function dfs(subset, start) {
    result.push(subset);
    for (let index = start; index < nums.length; index++) {
      dfs([...subset, nums[index]], index + 1);
    }
  }

  return result;
};
```

0079 Word Search

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 79. Word Search
 * https://leetcode.com/problems/word-search/
 * Difficulty: Medium
 *
 * Given an m x n grid of characters board and a string word, return true if word exists
 * in the grid.
 *
 * The word can be constructed from letters of sequentially adjacent cells, where adjacent
 * cells are horizontally or vertically neighboring. The same letter cell may not be used
 * more than once.
 */

/**
 * @param {character[][]} board
 * @param {string} word
 * @return {boolean}
 */
var exist = function(board, word) {
  function backtrack(x, y, k) {
    if (board[x][y] !== word[k]) {
      return false;
    } else if (k === word.length - 1) {
      return true;
    }

    board[x][y] = '-';

    for (const direction of [[-1, 0], [0, 1], [1, 0], [0, -1]]) {
      const [i, j] = [x + direction[0], y + direction[1]];
      if (i >= 0 && i < board.length && j >= 0 && j < board[0].length) {
        if (backtrack(i, j, k + 1)) {
          return true;
        }
      }
    }

    board[x][y] = word[k];

    return false;
  }

  for (let i = 0; i < board.length; i++) {
    for (let j = 0; j < board[0].length; j++) {
      if (backtrack(i, j, 0)) {
        return true;
      }
    }
  }

  return false;
};
```

0080 Remove Duplicates From Sorted Array Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 80. Remove Duplicates from Sorted Array II
 * https://leetcode.com/problems/remove-duplicates-from-sorted-array-ii/
 * Difficulty: Medium
```

```
 *
 * Given an integer array nums sorted in non-decreasing order, remove some duplicates in-place
 such
 * that each unique element appears at most twice. The relative order of the elements should
be kept
 * the same.
 *
 * Since it is impossible to change the length of the array in some languages, you must
instead have
 * the result be placed in the first part of the array nums. More formally, if there are k
elements
 * after removing the duplicates, then the first k elements of nums should hold the final
result.
 * It does not matter what you leave beyond the first k elements.
 *
 * Return k after placing the final result in the first k slots of nums.
 *
 * Do not allocate extra space for another array. You must do this by modifying the input
array
 * in-place with O(1) extra memory.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var removeDuplicates = function(nums) {
  let offset = 2;

  for (let i = 2; i < nums.length; i++) {
    if (nums[i] !== nums[offset - 2]) {
      nums[offset] = nums[i];
      offset++;
    }
  }

  return offset;
};
```

0081 Search In Rotated Sorted Array Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 81. Search in Rotated Sorted Array II
 * https://leetcode.com/problems/search-in-rotated-sorted-array-ii/
 * Difficulty: Medium
 *
 * There is an integer array nums sorted in non-decreasing order (not
 * necessarily with distinct values).
 *
 * Before being passed to your function, nums is rotated at an unknown
 * pivot index k (0 <= k < nums.length) such that the resulting array
 * is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]
 * (0-indexed). For example, [0,1,2,4,4,4,5,6,6,7] might be rotated at pivot
 * index 5 and become [4,5,6,6,7,0,1,2,4,4].
 *
 * Given the array nums after the rotation and an integer target, return
 * true if target is in nums, or false if it is not in nums.
 *
 * You must decrease the overall operation steps as much as possible.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {boolean}
 */
var search = function(nums, target) {
  return nums.includes(target);
```

```
};
```

0082 Remove Duplicates From Sorted List Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 82. Remove Duplicates from Sorted List II
 * https://leetcode.com/problems/remove-duplicates-from-sorted-list-ii/
 * Difficulty: Medium
 *
 * Given the head of a sorted linked list, delete all nodes that have duplicate
 * numbers, leaving only distinct numbers from the original list. Return the
 * linked list sorted as well.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var deleteDuplicates = function(head) {
  const result = new ListNode();
  let previous = null;
  let tail = result;

  while (head) {
    if (head.val !== head.next?.val && head.val !== previous) {
      tail.next = new ListNode(head.val);
      tail = tail.next;
    }
    previous = head.val;
    head = head.next;
  }

  return result.next;
};
```

0083 Remove Duplicates From Sorted List

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 83. Remove Duplicates from Sorted List
 * https://leetcode.com/problems/remove-duplicates-from-sorted-list/
 * Difficulty: Easy
 *
 * Given the head of a sorted linked list, delete all duplicates such that each
 * element appears only once. Return the linked list sorted as well.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
```

```
 * @param {ListNode} head
 * @return {ListNode}
 */
var deleteDuplicates = function(head) {
  const result = new ListNode();
  let tail = result;

  while (head) {
    if (head.val !== head.next?.val) {
      tail.next = new ListNode(head.val);
      tail = tail.next;
    }
    previous = head.val;
    head = head.next;
  }

  return result.next;
};
```

0084 Largest Rectangle In Histogram

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 84. Largest Rectangle in Histogram
 * https://leetcode.com/problems/largest-rectangle-in-histogram/
 * Difficulty: Hard
 *
 * Given an array of integers heights representing the histogram's bar height where the
 * width of each bar is 1, return the area of the largest rectangle in the histogram.
 */

/**
 * @param {number[]} heights
 * @return {number}
 */
var largestRectangleArea = function(heights) {
  const entries = [...heights, 0];
  const stack = [];
  let area = 0;

  for (let i = 0; i < entries.length; i++) {
    let updatedHeight = i;

    while (stack.length && stack[stack.length - 1][1] > entries[i]) {
      const [x, y] = stack.pop();
      area = Math.max(area, (i - x) * y);
      updatedHeight = x;
    }

    stack.push([updatedHeight, entries[i]]);
  }

  return area;
};
```

0085 Maximal Rectangle

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 85. Maximal Rectangle
 * https://leetcode.com/problems/maximal-rectangle/
 * Difficulty: Hard
 *
```

```
 * Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle
 * containing only 1's and return its area.
 */

/**
 * @param {character[][]} matrix
 * @return {number}
 */
var maximalRectangle = function(matrix) {
  if (!matrix?.length) return 0;
  let maxArea = 0;

  for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < matrix[0].length; j++) {
      if (matrix[i][j] === '1') {
        let minWidth = matrix[0].length;

        for (let k = i; k < matrix.length && matrix[k][j] === '1'; k++) {
          let width = 0;

          while (width < minWidth && j + width < matrix[0].length && matrix[k][j + width] ===
'1') {
            width++;
          }

          minWidth = Math.min(minWidth, width);
          maxArea = Math.max(maxArea, minWidth * (k - i + 1));
        }
      }
    }
  }

  return maxArea;
};
```

0086 Partition List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 86. Partition List
 * https://leetcode.com/problems/partition-list/
 * Difficulty: Medium
 *
 * Given the head of a linked list and a value x, partition it such that
 * all nodes less than x come before nodes greater than or equal to x.
 *
 * You should preserve the original relative order of the nodes in each
 * of the two partitions.
 */

/**
 * @param {ListNode} head
 * @param {number} x
 * @return {ListNode}
 */
var partition = function(head, x) {
  const result = [];
  const stack = [];

  while (head) {
    const target = head.val >= x ? result : stack;
    target.push(head.val);
    head = head.next;
  }

  return [...stack, ...result].reverse().reduce((a, b) => {
    return new ListNode(b, a);
  }, null);
};
```

0087 Scramble String

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 87. Scramble String
 * https://leetcode.com/problems/scramble-string/
 * Difficulty: Hard
 *
 * We can scramble a string s to get a string t using the following algorithm:
 *
 * 1. If the length of the string is 1, stop.
 * 2. If the length of the string is > 1, do the following:
 *   - Split the string into two non-empty substrings at a random index, i.e., if the string
 *     is s, divide it to x and y where s = x + y.
 *   - Randomly decide to swap the two substrings or to keep them in the same order. i.e.,
 *     after this step, s may become s = x + y or s = y + x.
 *   - Apply step 1 recursively on each of the two substrings x and y.
 *
 * Given two strings s1 and s2 of the same length, return true if s2 is a scrambled string
 * of s1, otherwise, return false.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var isScramble = function(s1, s2) {
  const n = s1.length;
  const dp = new Array(n).fill().map(() => new Array(n).fill().map(() => new
Array(n).fill(null)));

  function dfs(i1, j1, i2) {
    if (dp[i1][j1][i2] !== null) {
      return dp[i1][j1][i2];
    }
    if (i1 == j1) {
      return dp[i1][j1][i2] = s1[i1] === s2[i2];
    }

    const j2 = j1 - i1 + i2;
    let result = false;
    for (let i = i1; i < j1; i++) {
      result = result || (dfs(i1, i, i2) && dfs(i + 1, j1, i2 + i - i1 + 1))
        || (dfs(i1, i, j2 - i + i1) && dfs(i + 1, j1, i2));
    }

    return dp[i1][j1][i2] = result;
  }

  return dfs(0, n - 1, 0);
};
```

0088 Merge Sorted Array

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 88. Merge Sorted Array
 * https://leetcode.com/problems/merge-sorted-array/
 * Difficulty: Easy
 *
 * You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order,
 * and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2`
```

```
 * respectively.
 *
 * Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.
 *
 * The final sorted array should not be returned by the function, but instead be
 * stored inside the array `nums1`. To accommodate this, `nums1` has a length of `m + n`,
 * where the first `m` elements denote the elements that should be merged, and the
 * last `n` elements are set to `0` and should be ignored. `nums2` has a length of `n`.
 */

/**
 * @param {number[]} nums1
 * @param {number} m
 * @param {number[]} nums2
 * @param {number} n
 * @return {void} Do not return anything, modify nums1 in-place instead.
 */
var merge = function(nums1, m, nums2, n) {
  let i = m + n - 1;

  m--;
  n--;

  while (n >= 0) {
    nums1[i--] = nums1[m] > nums2[n] ? nums1[m--] : nums2[n--];
  }
};
```

0089 Gray Code

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 89. Gray Code
 * https://leetcode.com/problems/gray-code/
 * Difficulty: Medium
 *
 * An n-bit gray code sequence is a sequence of 2n integers where:
 * - Every integer is in the inclusive range [0, 2n - 1],
 * - The first integer is 0,
 * - An integer appears no more than once in the sequence,
 * - The binary representation of every pair of adjacent integers differs
 *   by exactly one bit, and
 * - The binary representation of the first and last integers differs by
 *   exactly one bit.
 * Given an integer n, return any valid n-bit gray code sequence.
 */

/**
 * @param {number} n
 * @return {number[]}
 */
var grayCode = function(n) {
  const result = [0];
  for (let i = 0; i < n; i++) {
    result.push(...result.map((v) => v | 1 << i).reverse());
  }
  return result;
};
```

0090 Subsets Ii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 90. Subsets II
 * https://leetcode.com/problems/subsets-ii/
 * Difficulty: Medium
 *
 * Given an integer array nums that may contain duplicates, return all possible subsets (the
power set).
 *
 * The solution set must not contain duplicate subsets. Return the solution in any order.
 */

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var subsetsWithDup = function(nums) {
  const seen = new Set();
  const result = [];
  dfs([], 0);

  function dfs(subset, start) {
    const key = subset.sort((a, b) => a - b).join('');
    if (!seen.has(key)) {
      result.push(subset);
      seen.add(key);
    }
    for (let index = start; index < nums.length; index++) {
      dfs([...subset, nums[index]], index + 1);
    }
  }

  return result;
};
```

0091 Decode Ways

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 91. Decode Ways
 * https://leetcode.com/problems/decode-ways/
 * Difficulty: Medium
 *
 * You have intercepted a secret message encoded as a string of numbers. The message
 * is decoded via the following mapping:
 * "1" -> 'A'
 * "2" -> 'B'
 * ...
 * "25" -> 'Y'
 * "26" -> 'Z'
 *
 * However, while decoding the message, you realize that there are many different
 * ways you can decode the message because some codes are contained in other codes
 * ("2" and "5" vs "25").
 *
 * For example, "11106" can be decoded into:
 * - "AAJF" with the grouping (1, 1, 10, 6)
 * - "KJF" with the grouping (11, 10, 6)
 * - The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6"
 *   is valid).
 *
 * Note: there may be strings that are impossible to decode.
 *
 * Given a string s containing only digits, return the number of ways to decode it.
 * If the entire string cannot be decoded in any valid way, return 0.
 *
 * The test cases are generated so that the answer fits in a 32-bit integer.
 */

/**
```

```
 * @param {string} s
 * @return {number}
 */
var numDecodings = function(s) {
  if (s == null || s.length === 0) return 0;
  if (s[0] === '0') return 0;

  const group = new Array(s.length + 1).fill(0);
  group[0] = 1;
  group[1] = 1;

  for (let i = 2; i <= s.length; i++) {
    const a = Number(s.slice(i - 1, i));
    if (a >= 1 && a <= 9) {
      group[i] += group[i - 1];
    }

    const b = Number(s.slice(i - 2, i));
    if (b >= 10 && b <= 26) {
      group[i] += group[i - 2];
    }
  }

  return group[s.length];
};
```

0092 Reverse Linked List Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 92. Reverse Linked List II
 * https://leetcode.com/problems/reverse-linked-list-ii/
 * Difficulty: Medium
 *
 * Given the head of a singly linked list and two integers left and right where
 * left <= right, reverse the nodes of the list from position left to position
 * right, and return the reversed list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} left
 * @param {number} right
 * @return {ListNode}
 */
var reverseBetween = function(head, left, right) {
  const result = new ListNode(0, head);
  let previous = result;
  let currentIndex = 1;

  while (currentIndex++ < left) {
    previous = previous.next;
  }

  const tail = previous.next;
  while (left++ < right) {
    const next = tail.next;
    tail.next = next.next;
    next.next = previous.next;
    previous.next = next;
  }
```

```
  return result.next;
};
```

0093 Restore Ip Addresses

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 93. Restore IP Addresses
 * https://leetcode.com/problems/restore-ip-addresses/
 * Difficulty: Medium
 *
 * A valid IP address consists of exactly four integers separated by single
 * dots. Each integer is between 0 and 255 (inclusive) and cannot have
 * leading zeros.
 *
 * For example, "0.1.2.201" and "192.168.1.1" are valid IP addresses, but
 * "0.011.255.245", "192.168.1.312" and "192.168@1.1" are invalid IP addresses.
 *
 * Given a string s containing only digits, return all possible valid IP
 * addresses that can be formed by inserting dots into s. You are not allowed
 * to reorder or remove any digits in s. You may return the valid IP addresses
 * in any order.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var restoreIpAddresses = function(s) {
  return backtrack([], s);
};

function backtrack(order, string, result = []) {
  const isValid = s => +s <= 255 && `${+s}` === s;

  if (order.length === 3 && isValid(string)) {
    result.push([...order, string].join('.'));
  } else {
    for (let index = 1; index < 4; index++) {
      const sliced = string.slice(0, index);
      if (isValid(sliced)) {
        backtrack([...order, sliced], string.slice(index), result);
      }
    }
  }

  return result;
}
```

0094 Binary Tree Inorder Traversal

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 94. Binary Tree Inorder Traversal
 * https://leetcode.com/problems/binary-tree-inorder-traversal/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return the inorder traversal of its nodes' values.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
```

```
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var inorderTraversal = function(root) {
  const result = [];
  const stack = [];

  while (root || stack.length) {
    if (root) {
      stack.push(root);
      root = root.left;
    } else {
      const { val, right } = stack.pop();
      result.push(val);
      root = right;
    }
  }

  return result;
};
```

0095 Unique Binary Search Trees Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 95. Unique Binary Search Trees II
 * https://leetcode.com/problems/unique-binary-search-trees-ii/
 * Difficulty: Medium
 *
 * Given an integer n, return all the structurally unique BST's (binary search
 * trees), which has exactly n nodes of unique values from 1 to n. Return the
 * answer in any order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number} n
 * @return {TreeNode[]}
 */
var generateTrees = function(n) {
  return backtrack(n);
};

function backtrack(n, j = 1, k = n, result = []) {
  for (let index = j; index <= k; index++) {
    for (const left of backtrack(n, j, index - 1)) {
      for (const right of backtrack(n, index + 1, k)) {
        result.push({ val: index, left, right });
      }
    }
  }

  return n ? result.length ? result : [null] : [];
}
```

0096 Unique Binary Search Trees

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 96. Unique Binary Search Trees
 * https://leetcode.com/problems/unique-binary-search-trees/
 * Difficulty: Medium
 *
 * Given an integer n, return the number of structurally unique BST's (binary
 * search trees) which has exactly n nodes of unique values from 1 to n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var numTrees = function(n) {
  return traverse(2 * n) / (traverse(n + 1) * traverse(n));
};

function traverse(n) {
  return n <= 0 ? 1 : n * traverse(n - 1);
}
```

0097 Interleaving String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 97. Interleaving String
 * https://leetcode.com/problems/interleaving-string/
 * Difficulty: Medium
 *
 * Given strings s1, s2, and s3, find whether s3 is formed by an interleaving of s1 and s2.
 *
 * An interleaving of two strings s and t is a configuration where s and t are divided into
 * n and m substrings respectively, such that:
 * - s = s1 + s2 + ... + sn
 * - t = t1 + t2 + ... + tm
 * - |n - m| <= 1
 * - The interleaving is s1 + t1 + s2 + t2 + s3 + t3 + ... or t1 + s1 + t2 + s2 + t3 + s3 +
...
 *
 * Note: a + b is the concatenation of strings a and b.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @param {string} s3
 * @return {boolean}
 */
var isInterleave = function(s1, s2, s3) {
  const values = new Array(s1.length + 1).fill().map(() => []);

  if (s1.length + s2.length !== s3.length) {
    return false;
  }

  for (let i = 0; i <= s1.length; i++) {
    for (let j = 0; j <= s2.length; j++) {
      values[i][j] = i && values[i - 1][j] && s3[i + j - 1] === s1[i - 1]
        || j && values[i][j - 1] && s3[i + j - 1] === s2[j - 1] || !i && !j;
    }
```

```
  }

  return Boolean(values.pop().pop());
};
```

0098 Validate Binary Search Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 98. Validate Binary Search Tree
 * https://leetcode.com/problems/validate-binary-search-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, determine if it is a valid binary search tree (BST).
 *
 * A valid BST is defined as follows:
 * - The left subtree of a node contains only nodes with keys less than the node's key.
 * - The right subtree of a node contains only nodes with keys greater than the node's key.
 * - Both the left and right subtrees must also be binary search trees.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isValidBST = function(root) {
  return traverse(root, null, null);
};

function traverse(root, min, max) {
  if (!root) {
    return true;
  }

  if ((min !== null && root.val <= min) || (max !== null && root.val >= max)) {
    return false;
  }

  return traverse(root.left, min, root.val) && traverse(root.right, root.val, max);
}
```

0099 Recover Binary Search Tree

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 99. Recover Binary Search Tree
 * https://leetcode.com/problems/recover-binary-search-tree/
 * Difficulty: Medium
 *
 * You are given the root of a binary search tree (BST), where the values
 * of exactly two nodes of the tree were swapped by mistake. Recover the
 * tree without changing its structure.
 */

/**
```

```
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {void} Do not return anything, modify root in-place instead.
 */
var recoverTree = function(root) {
  let [previous, small, large] = [null, null, null];

  dfs(root);
  [large.val, small.val] = [small.val, large.val];

  function dfs(root) {
    if (!root) return;
    dfs(root.left);
    if (previous != null && previous.val > root.val) {
      small = root;
      if (!large) {
        large = previous;
      } else {
        return false;
      }
    }
    previous = root;
    dfs(root.right);
  }
};
```

0100 Same Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 100. Same Tree
 * https://leetcode.com/problems/same-tree/
 * Difficulty: Easy
 *
 * Given the roots of two binary trees p and q, write a function to check if they are
 * the same or not.
 *
 * Two binary trees are considered the same if they are structurally identical, and
 * the nodes have the same value.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {boolean}
 */
var isSameTree = function(p, q) {
  const hasSameValue = p !== null && q !== null && p.val === q.val;
  const hasSameTree = hasSameValue && isSameTree(p.left, q.left) && isSameTree(p.right,
q.right);
  return (p === null && q === null) || hasSameTree;
};
```

0101 Symmetric Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 101. Symmetric Tree
 * https://leetcode.com/problems/symmetric-tree/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, check whether it is a mirror
 * of itself (i.e., symmetric around its center).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isSymmetric = function(root) {
  return isBalanced(root.left, root.right);
};

function isBalanced(a, b) {
  if (!a && !b) {
    return true;
  }
  if (!a || !b) {
    return false;
  }
  return a.val === b.val && isBalanced(a.left, b.right) && isBalanced(a.right, b.left);
}
```

0102 Binary Tree Level Order Traversal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 102. Binary Tree Level Order Traversal
 * https://leetcode.com/problems/binary-tree-level-order-traversal/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the level order traversal of its
 * nodes' values. (i.e., from left to right, level by level).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var levelOrder = function(root) {
```

```
  const result = [];

  traverse(result, root);

  return result;
};

function traverse(result, node, level = 0) {
  if (!node) {
    return [];
  }

  result[level] = result[level] || [];
  result[level].push(node.val);

  traverse(result, node.left, level + 1);
  traverse(result, node.right, level + 1);
}
```

0103 Binary Tree Zigzag Level Order Traversal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 103. Binary Tree Zigzag Level Order Traversal
 * https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the zigzag level order traversal of its nodes'
values.
 * (i.e., from left to right, then right to left for the next level and alternate between).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var zigzagLevelOrder = function(root) {
  const result = [];

  function traverse(node, depth) {
    if (node == null) return;
    if (result[depth] == null) result[depth] = [];

    if (depth % 2 === 0) {
      result[depth].push(node.val);
    } else {
      result[depth].unshift(node.val);
    }

    traverse(node.left, depth + 1);
    traverse(node.right, depth + 1);
  }

  traverse(root, 0);

  return result;
};
```

0104 Maximum Depth Of Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 104. Maximum Depth of Binary Tree
 * https://leetcode.com/problems/maximum-depth-of-binary-tree/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return its maximum depth.
 *
 * A binary tree's maximum depth is the number of nodes along the longest
 * path from the root node down to the farthest leaf node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxDepth = function(root) {
  if (!root) {
    return 0;
  }
  const [left, right] = [root.left, root.right].map(maxDepth);
  return 1 + Math.max(left, right);
};
```

0105 Construct Binary Tree From Preorder And Inorder Traversal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 105. Construct Binary Tree from Preorder and Inorder Traversal
 * https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/
 * Difficulty: Medium
 *
 * Given two integer arrays preorder and inorder where preorder is the preorder traversal of a
 * binary tree and inorder is the inorder traversal of the same tree, construct and return the
 * binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} preorder
 * @param {number[]} inorder
 * @return {TreeNode}
 */
var buildTree = function(preorder, inorder) {
  if (!preorder.length || !inorder.length) return null;
```

```
  const root = new TreeNode(preorder[0]);
  const mid = inorder.indexOf(preorder[0]);

  root.left = buildTree(preorder.slice(1, mid + 1), inorder.slice(0, mid));
  root.right = buildTree(preorder.slice(mid + 1), inorder.slice(mid + 1));

  return root;
};
```

0106 Construct Binary Tree From Inorder And Postorder Traversal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 106. Construct Binary Tree from Inorder and Postorder Traversal
 * https://leetcode.com/problems/construct-binary-tree-from-inorder-and-postorder-traversal/
 * Difficulty: Medium
 *
 * Given two integer arrays inorder and postorder where inorder is the inorder traversal of a
 * binary tree and postorder is the postorder traversal of the same tree, construct and return
 * the binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} inorder
 * @param {number[]} postorder
 * @return {TreeNode}
 */
var buildTree = function(inorder, postorder) {
  if (inorder.length === 0) return null;

  const root = postorder[postorder.length - 1];
  const index = inorder.indexOf(root);

  return {
    val: root,
    left: buildTree(inorder.slice(0, index), postorder.slice(0, index)),
    right: buildTree(inorder.slice(index + 1), postorder.slice(index, -1))
  };
};
```

0107 Binary Tree Level Order Traversal Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 107. Binary Tree Level Order Traversal II
 * https://leetcode.com/problems/binary-tree-level-order-traversal-ii/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the bottom-up level order traversal of its nodes'
values.
 * (i.e., from left to right, level by level from leaf to root).
 */

/**
 * Definition for a binary tree node.
```

```
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var levelOrderBottom = function(root) {
  if (!root) return [];

  const result = [];
  const stack = [root];

  while (stack.length > 0) {
    const level = [];
    const n = stack.length;

    for (let i = 0; i < n; i++) {
      const node = stack.shift();
      level.push(node.val);

      if (node.left) stack.push(node.left);
      if (node.right) stack.push(node.right);
    }

    result.unshift(level);
  }

  return result;
};
```

0108 Convert Sorted Array To Binary Search Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 108. Convert Sorted Array to Binary Search Tree
 * https://leetcode.com/problems/convert-sorted-array-to-binary-search-tree/
 * Difficulty: Easy
 *
 * Given an integer array nums where the elements are sorted in ascending order, convert it
 * to a height-balanced binary search tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var sortedArrayToBST = function(nums) {
  if (!nums.length) return null;

  const middleIndex = Math.floor(nums.length / 2);
  const root = new TreeNode(nums[middleIndex]);
  root.left = sortedArrayToBST(nums.slice(0, middleIndex));
  root.right = sortedArrayToBST(nums.slice(middleIndex + 1));

  return root;
};
```

0109 Convert Sorted List To Binary Search Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 109. Convert Sorted List to Binary Search Tree
 * https://leetcode.com/problems/convert-sorted-list-to-binary-search-tree/
 * Difficulty: Medium
 *
 * Given the head of a singly linked list where elements are sorted in ascending order,
 * convert it to a height-balanced binary search tree.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {ListNode} head
 * @return {TreeNode}
 */
var sortedListToBST = function(head) {
  if (!head) return null;
  if (!head.next) return new TreeNode(head.val);

  let fast = head;
  let slow = head;
  let previous = head;
  while (fast && fast.next) {
    previous = slow;
    slow = slow.next;
    fast = fast.next.next;
  }

  const root = new TreeNode(slow.val);
  previous.next = null;

  const newHead = slow.next;
  slow.next = null;

  root.left = sortedListToBST(head);
  root.right = sortedListToBST(newHead);

  return root;
};
```

0110 Balanced Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 110. Balanced Binary Tree
 * https://leetcode.com/problems/balanced-binary-tree/
 * Difficulty: Easy
```

```
 *
 * Given a binary tree, determine if it is height-balanced.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isBalanced = function(root) {
  if (!root) return true;
  return isBalanced(root.right) && isBalanced(root.left)
    && Math.abs(traverse(root.right) - traverse(root.left)) < 2;
};

function traverse(node, depth = 0) {
  if (!node) return depth;
  return Math.max(traverse(node.right, depth + 1), traverse(node.left, depth + 1));
}
```

0111 Minimum Depth Of Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 111. Minimum Depth of Binary Tree
 * https://leetcode.com/problems/minimum-depth-of-binary-tree/
 * Difficulty: Easy
 *
 * Given a binary tree, find its minimum depth.
 *
 * The minimum depth is the number of nodes along the shortest path from the
 * root node down to the nearest leaf node.
 *
 * Note: A leaf is a node with no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var minDepth = function(root) {
  if (!root) {
    return 0;
  }
  const [left, right] = [root.left, root.right].map(minDepth);
  return 1 + (Math.min(left, right) || Math.max(left, right));
};
```

0112 Path Sum

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 112. Path Sum
 * https://leetcode.com/problems/path-sum/
 * Difficulty: Easy
 *
 * Given the root of a binary tree and an integer targetSum, return true if the
 * tree has a root-to-leaf path such that adding up all the values along the
 * path equals targetSum.
 *
 * A leaf is a node with no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} targetSum
 * @return {boolean}
 */
var hasPathSum = function(root, targetSum) {
  if (!root) {
    return false;
  }
  const result = [];

  traverse(result, root);

  return result.includes(targetSum);
};

function traverse(result, node, sum = 0) {
  if (!node.left && !node.right) {
    result.push(sum + node.val);
  }

  if (node.left) traverse(result, node.left, sum + node.val);
  if (node.right) traverse(result, node.right, sum + node.val);
}
```

0113 Path Sum Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 113. Path Sum II
 * https://leetcode.com/problems/path-sum-ii/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths
where
 * the sum of the node values in the path equals targetSum. Each path should be returned as a
 * list of the node values, not node references.
 *
 * A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is
 a
```

```
 * node with no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} targetSum
 * @return {number[][]}
 */
var pathSum = function(root, targetSum) {
  if (!root) return [];
  return traverse([], targetSum, root);
};

function traverse(result, targetSum, node, history = []) {
  const values = [...history, node.val];

  if (!node.left && !node.right && values.reduce((sum, n) => sum + n, 0) === targetSum) {
    result.push(values);
  }

  if (node.left) traverse(result, targetSum, node.left, values);
  if (node.right) traverse(result, targetSum, node.right, values);

  return result;
}
```

0114 Flatten Binary Tree To Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 114. Flatten Binary Tree to Linked List
 * https://leetcode.com/problems/flatten-binary-tree-to-linked-list/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, flatten the tree into a "linked list":
 * - The "linked list" should use the same TreeNode class where the right child pointer points
 *   to the next node in the list and the left child pointer is always null.
 * - The "linked list" should be in the same order as a pre-order traversal of the binary
tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {void} Do not return anything, modify root in-place instead.
 */
var flatten = function(root) {
  if (root === null) return;

  if (root.left) {
    let previous = root.left;
    while (previous.right) {
      previous = previous.right;
    }
```

```
    const rightNode = root.right;
    root.right = root.left;
    previous.right = rightNode;
    root.left = null;
  }

  flatten(root.right);
};
```

0115 Distinct Subsequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 115. Distinct Subsequences
 * https://leetcode.com/problems/distinct-subsequences/
 * Difficulty: Hard
 *
 * Given two strings s and t, return the number of distinct subsequences of s which equals t.
 *
 * The test cases are generated so that the answer fits on a 32-bit signed integer.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {number}
 */
var numDistinct = function(s, t) {
  const nums = new Array(t.length + 1).fill(0);
  nums[0] = 1;

  for (let i = 0; i < s.length; i++) {
    for (let j = nums.length - 1; j >= 0; j--) {
      if (s[i] === t[j]) {
        nums[j + 1] += nums[j];
      }
    }
  }

  return nums[t.length];
};
```

0116 Populating Next Right Pointers In Each Node

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 116. Populating Next Right Pointers in Each Node
 * https://leetcode.com/problems/populating-next-right-pointers-in-each-node/
 * Difficulty: Medium
 *
 * You are given a perfect binary tree where all leaves are on the same level, and
 * every parent has two children. The binary tree has the following definition:
 *
 * Populate each next pointer to point to its next right node. If there is no next
 * right node, the next pointer should be set to NULL.
 *
 * Initially, all next pointers are set to NULL.
 */

/**
 * // Definition for a Node.
 * function Node(val, left, right, next) {
 *    this.val = val === undefined ? null : val;
```

```
 *    this.left = left === undefined ? null : left;
 *    this.right = right === undefined ? null : right;
 *    this.next = next === undefined ? null : next;
 * };
 */

/**
 * @param {Node} root
 * @return {Node}
 */
var connect = function(root) {
  const order = [];
  traverse(order, root);
  for (let level of order) {
    for (let i = 0; i < level.length - 1; i++) {
      level[i].next = level[i + 1];
    }
  }
  return root;
};

function traverse(order, node, level = 0) {
  if (!node) {
    return [];
  }

  order[level] = order[level] || [];
  order[level].push(node);

  traverse(order, node.left, level + 1);
  traverse(order, node.right, level + 1);
}
```

0117 Populating Next Right Pointers In Each Node Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 117. Populating Next Right Pointers in Each Node II
 * https://leetcode.com/problems/populating-next-right-pointers-in-each-node-ii/
 * Difficulty: Medium
 *
 * Given a binary tree, populate each next pointer to point to its next right node.
 * If there is no next right node, the next pointer should be set to NULL.
 *
 * Initially, all next pointers are set to NULL.
 */

/**
 * // Definition for a _Node.
 * function _Node(val, left, right, next) {
 *    this.val = val === undefined ? null : val;
 *    this.left = left === undefined ? null : left;
 *    this.right = right === undefined ? null : right;
 *    this.next = next === undefined ? null : next;
 * };
 */

/**
 * @param {_Node} root
 * @return {_Node}
 */
var connect = function(root) {
  if (root === null) return root;

  const depth = 0;
  const stack = [[root, depth]];

  while (stack.length) {
    const [node, depth] = stack.shift();
```

```
    if (stack.length) {
      const [nextNode, nextDepth] = stack[0];
      if (depth === nextDepth) {
        node.next = nextNode;
      }
    }
    if (node.left) {
      stack.push([node.left, depth + 1]);
    }
    if (node.right) {
      stack.push([node.right, depth + 1]);
    }
  }

  return root;
};
```

0118 Pascals Triangle

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 118. Pascal's Triangle
 * https://leetcode.com/problems/pascals-triangle/
 * Difficulty: Easy
 *
 * Given an integer numRows, return the first numRows of Pascal's triangle.
 *
 * In Pascal's triangle, each number is the sum of the two numbers directly
 * above it as shown:
 */

/**
 * @param {number} numRows
 * @return {number[][]}
 */
var generate = function(numRows) {
  const result = [[1]];

  for (let i = 1; i < numRows; i++) {
    result[i] = [1];
    result[i - 1].forEach((value, j, prev) => {
      result[i][j + 1] = (prev[j + 1] + value) || 1;
    });
  }

  return result;
};
```

0119 Pascals Triangle Ii

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 119. Pascal's Triangle II
 * https://leetcode.com/problems/pascals-triangle-ii/
 * Difficulty: Easy
 *
 * Given an integer rowIndex, return the rowIndexth (0-indexed) row of
 * the Pascal's triangle.
 *
 * In Pascal's triangle, each number is the sum of the two numbers
 * directly above it as shown:
 */
```

```
/**
 * @param {number} rowIndex
 * @return {number[]}
 */
var getRow = function(rowIndex) {
  return generateTriangle(rowIndex + 1)[rowIndex];
};

function generateTriangle(numRows) {
  const result = [[1]];

  for (let i = 1; i < numRows; i++) {
    result[i] = [1];
    result[i - 1].forEach((value, j, prev) => {
      result[i][j + 1] = (prev[j + 1] + value) || 1;
    });
  }

  return result;
};
```

0120 Triangle

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 120. Triangle
 * https://leetcode.com/problems/triangle/
 * Difficulty: Medium
 *
 * Given a triangle array, return the minimum path sum from top to bottom.
 *
 * For each step, you may move to an adjacent number of the row below. More
 * formally, if you are on index i on the current row, you may move to either
 * index i or index i + 1 on the next row.
 */

/**
 * @param {number[][]} triangle
 * @return {number}
 */
var minimumTotal = function(triangle) {
  for (let i = triangle.length - 2; i > -1; i--) {
    for (let j = 0; j < triangle[i].length; j++) {
      triangle[i][j] += Math.min( triangle[i + 1][j], triangle[i + 1][j + 1]);
    }
  }

  return triangle[0][0];
};
```

0121 Best Time To Buy And Sell Stock

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 121. Best Time to Buy and Sell Stock
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock/
 * Difficulty: Easy
 *
 * You are given an array prices where prices[i] is the price of a given stock
 * on the ith day.
 *
 * You want to maximize your profit by choosing a single day to buy one stock
 * and choosing a different day in the future to sell that stock.
```

```
 *
 * Return the maximum profit you can achieve from this transaction. If you
 * cannot achieve any profit, return 0.
 */

/**
 * @param {number[]} prices
 * @return {number}
 */
var maxProfit = function(prices) {
  let max = 0;

  for (let i = 0, min = prices[0]; i < prices.length; i++) {
    min = Math.min(min, prices[i]);
    max = Math.max(max, prices[i] - min);
  }

  return max;
};
```

0122 Best Time To Buy And Sell Stock Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 122. Best Time to Buy and Sell Stock II
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii/
 * Difficulty: Medium
 *
 * You are given an integer array prices where prices[i] is the price of a given
 * stock on the ith day.
 *
 * On each day, you may decide to buy and/or sell the stock. You can only hold at
 * most one share of the stock at any time. However, you can buy it then immediately
 * sell it on the same day.
 *
 * Find and return the maximum profit you can achieve.
 */

/**
 * @param {number[]} prices
 * @return {number}
 */
var maxProfit = function(prices) {
  return prices.reduce((result, price, i) => {
    return prices[i - 1] < price ? result + (price - prices[i - 1]) : result;
  }, 0);
};
```

0123 Best Time To Buy And Sell Stock Iii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 123. Best Time to Buy and Sell Stock III
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-iii/
 * Difficulty: Hard
 *
 * You are given an array prices where prices[i] is the price of a given stock on the ith day.
 *
 * Find the maximum profit you can achieve. You may complete at most two transactions.
 *
 * Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the
 * stock before you buy again).
 */
```

```
/**
 * @param {number[]} prices
 * @return {number}
 */
var maxProfit = function(prices) {
  const costs = [prices[0]];
  const profits = new Array(prices.length).fill(0);

  for (let i = 0; i < 2; i++) {
    for (let j = 1; j < prices.length; j++) {
      costs[j] = Math.min(costs[j - 1], prices[j] - profits[j]);
      profits[j] = Math.max(profits[j - 1], prices[j] - costs[j]);
    }
  }

  return profits[profits.length - 1];
};
```

0124 Binary Tree Maximum Path Sum

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 124. Binary Tree Maximum Path Sum
 * https://leetcode.com/problems/binary-tree-maximum-path-sum/
 * Difficulty: Hard
 *
 * A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the
 * sequence has an edge connecting them. A node can only appear in the sequence at most
 * once. Note that the path does not need to pass through the root.
 *
 * The path sum of a path is the sum of the node's values in the path.
 *
 * Given the root of a binary tree, return the maximum path sum of any non-empty path.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxPathSum = function(root) {
  let result = -Infinity;

  function traverse(node) {
    if (!node) return 0;
    const leftValue = Math.max(traverse(node.left), 0);
    const rightValue = Math.max(traverse(node.right), 0);
    result = Math.max(result, node.val + leftValue + rightValue);
    return node.val + Math.max(leftValue, rightValue);
  }

  traverse(root);

  return result;
};
```

0125 Valid Palindrome

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 125. Valid Palindrome
 * https://leetcode.com/problems/valid-palindrome/
 * Difficulty: Easy
 *
 * A phrase is a palindrome if, after converting all uppercase letters into lowercase letters
and
 * removing all non-alphanumeric characters, it reads the same forward and backward.
Alphanumeric
 * characters include letters and numbers.
 *
 * Given a string s, return true if it is a palindrome, or false otherwise.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var isPalindrome = function(s) {
  const string = s.replace(/[^A-Z\d]+/ig, '').toLowerCase();
  return string.split('').reverse().join('') === string;
};
```

0126 Word Ladder Ii

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 126. Word Ladder II
 * https://leetcode.com/problems/word-ladder-ii/
 * Difficulty: Hard
 *
 * A transformation sequence from word beginWord to word endWord using a dictionary wordList
 * is a sequence of words beginWord -> s1 -> s2 -> ... -> sk such that:
 * - Every adjacent pair of words differs by a single letter.
 * - Every si for 1 <= i <= k is in wordList. Note that beginWord does not need to be in
wordList.
 * - sk == endWord
 *
 * Given two words, beginWord and endWord, and a dictionary wordList, return all the shortest
 * transformation sequences from beginWord to endWord, or an empty list if no such sequence
 * exists. Each sequence should be returned as a list of the words [beginWord, s1, s2, ...,
sk].
 */

/**
 * @param {string} beginWord
 * @param {string} endWord
 * @param {string[]} wordList
 * @return {string[][]}
 */
var findLadders = function(beginWord, endWord, wordList) {
  const set = new Set(wordList);
  const queue = [beginWord];
  const group = [];
  let isEnd = false;

  while (queue.length && !isEnd) {
    group.push([...queue]);
```

```
    const limit = queue.length;
    for (let i = 0; i < limit && !isEnd; i++) {
      const from = queue.shift();
      for (const word of set) {
        if (!isValid(from, word)) {
          continue;
        } else if (word === endWord) {
          isEnd = true;
          break;
        }
        queue.push(word);
        set.delete(word);
      }
    }
  }

  if (!isEnd) {
    return [];
  }

  const result = [[endWord]];
  for (let i = group.length - 1; i >= 0; i--) {
    const limit = result.length;
    for (let j = 0; j < limit; j++) {
      const path = result.shift();
      for (const word of group[i]) {
        if (!isValid(path[0], word)) {
          continue;
        }
        result.push([word, ...path]);
      }
    }
  }

  return result;

  function isValid(a, b) {
    let count = 0;
    for (let i = 0; i < a.length && count < 2; i++) {
      if (a[i] !== b[i]) {
        count++;
      }
    }
    return count === 1;
  }
};
```

0127 Word Ladder

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 127. Word Ladder
 * https://leetcode.com/problems/word-ladder/
 * Difficulty: Hard
 *
 * A transformation sequence from word beginWord to word endWord using a dictionary
 * wordList is a sequence of words beginWord -> s1 -> s2 -> ... -> sk such that:
 * - Every adjacent pair of words differs by a single letter.
 * - Every si for 1 <= i <= k is in wordList. Note that beginWord does not need to
 *   be in wordList.
 * - sk == endWord
 * Given two words, beginWord and endWord, and a dictionary wordList, return the
 * number of words in the shortest transformation sequence from beginWord to
 * endWord, or 0 if no such sequence exists.
 */

/**
 * @param {string} beginWord
```

```
 * @param {string} endWord
 * @param {string[]} wordList
 * @return {number}
 */
var ladderLength = function(beginWord, endWord, wordList) {
  const set = new Set(wordList);
  let queue = [beginWord];
  let count = 1;

  while (queue.length) {
    const group = [];

    for (const word of queue) {
      if (word === endWord) {
        return count;
      }

      for (let i = 0; i < word.length; i++) {
        for (let j = 0; j < 26; j++) {
          const str = word.slice(0, i) + String.fromCharCode(j + 97) + word.slice(i + 1);
          if (set.has(str)) {
            group.push(str);
            set.delete(str);
          }
        }
      }
    }

    queue = group;
    count++;
  }

  return 0;
};
```

0128 Longest Consecutive Sequence

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 128. Longest Consecutive Sequence
 * https://leetcode.com/problems/longest-consecutive-sequence/
 * Difficulty: Medium
 *
 * Given an unsorted array of integers nums, return the length of the longest consecutive
 * elements sequence.
 *
 * You must write an algorithm that runs in O(n) time.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var longestConsecutive = function(nums) {
  const set = new Set(nums);
  let result = 0;

  for (const num of set) {
    if (!set.has(num - 1)) {
      let count = 1;
      let n = num;

      while (set.has(n + 1)) {
        n++;
        count++;
      }
      result = Math.max(result, count);
    }
```

```
  }

  return result;
};
```

0129 Sum Root To Leaf Numbers

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 129. Sum Root to Leaf Numbers
 * https://leetcode.com/problems/sum-root-to-leaf-numbers/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree containing digits from 0 to 9 only.
 *
 * Each root-to-leaf path in the tree represents a number.
 *
 * - For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123.
 *
 * Return the total sum of all root-to-leaf numbers. Test cases are generated
 * so that the answer will fit in a 32-bit integer.
 *
 * A leaf node is a node with no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var sumNumbers = function(root) {
  function traverse(node, value) {
    if (!node) {
      return null;
    }
    value += node.val;
    if (!node.left && !node.right) {
      return Number(value);
    }
    return traverse(node.left, value) + traverse(node.right, value);
  }

  return traverse(root, '');
};
```

0130 Surrounded Regions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 130. Surrounded Regions
 * https://leetcode.com/problems/surrounded-regions/
 * Difficulty: Medium
 *
 * You are given an m x n matrix board containing letters 'X' and 'O', capture regions
 * that are surrounded:
 * - Connect: A cell is connected to adjacent cells horizontally or vertically.
```

```
 * - Region: To form a region connect every 'O' cell.
 * - Surround: The region is surrounded with 'X' cells if you can connect the region with
 * 'X' cells and none of the region cells are on the edge of the board.
 *
 * To capture a surrounded region, replace all 'O's with 'X's in-place within the original
 * board. You do not need to return anything.
 */

/**
 * @param {character[][]} board
 * @return {void} Do not return anything, modify board in-place instead.
 */
var solve = function(board) {
  for (let i = 0; i < board.length; i++) {
    for (let j = 0; j < board[0].length; j++) {
      if (board[i][j] === 'O'
          && (!i || !j || i === board.length - 1 || j === board[0].length - 1)) {
        dfs(i, j);
      }
    }
  }

  for (let i = 0; i < board.length; i++) {
    for (let j = 0; j < board[0].length; j++) {
      if (board[i][j] === '-') {
        board[i][j] = 'O';
      } else {
        board[i][j] = 'X';
      }
    }
  }

  function dfs(i, j) {
    if (i < 0 || i >= board.length || j < 0 || j >= board[i].length
        || board[i][j] === '-' || board[i][j] === 'X') {
      return;
    }

    board[i][j] = '-';
    dfs(i - 1, j);
    dfs(i + 1, j);
    dfs(i, j - 1);
    dfs(i, j + 1);
  }
};
```

0131 Palindrome Partitioning

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 131. Palindrome Partitioning
 * https://leetcode.com/problems/palindrome-partitioning/
 * Difficulty: Medium
 *
 * Given a string s, partition s such that every substring of the partition
 * is a palindrome. Return all possible palindrome partitioning of s.
 */

/**
 * @param {string} s
 * @return {string[][]}
 */
var partition = function(s) {
  return backtrack([], s);
};

function backtrack(order, string, result = []) {
  if (!string.length) {
    result.push(order);
```

```
  }

  for (let index = 1; index <= string.length; index++) {
    const sliced = string.slice(0, index);
    if (isPalindrome(sliced)) {
      backtrack([...order, sliced], string.slice(index), result);
    }
  }

  return result;
}

function isPalindrome(input) {
  let right = input.length - 1;
  let left = 0;

  while (left < right) {
    if (input[left] !== input[right]) {
      return false;
    }
    left++;
    right--;
  }

  return true;
}
```

0132 Palindrome Partitioning Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 132. Palindrome Partitioning II
 * https://leetcode.com/problems/palindrome-partitioning-ii/
 * Difficulty: Hard
 *
 * Given a string s, partition s such that every substring of the partition is a palindrome.
 *
 * Return the minimum cuts needed for a palindrome partitioning of s.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minCut = function(s) {
  const isPalindrome = new Array(s.length).fill().map(() => new Array(s.length).fill(false));
  const partitions = new Array(s.length).fill(0);

  for (let i = 0; i < s.length; i++) {
    let offset = i;
    for (let j = 0; j <= i; j++) {
      if (s[j] === s[i] && (i - j <= 1 || isPalindrome[j + 1][i - 1])) {
        isPalindrome[j][i] = true;
        offset = j === 0 ? 0 : Math.min(offset, partitions[j - 1] + 1);
      }
    }
    partitions[i] = offset;
  }

  return partitions[s.length - 1];
};
```

0133 Clone Graph

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 133. Clone Graph
 * https://leetcode.com/problems/clone-graph/
 * Difficulty: Medium
 *
 * Given a reference of a node in a connected undirected graph.
 * Return a deep copy (clone) of the graph.
 * Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.
 *
 * Test case format:
 * - For simplicity, each node's value is the same as the node's index (1-indexed).
 *   For example, the first node with val == 1, the second node with val == 2, and so on.
 *   The graph is represented in the test case using an adjacency list.
 *
 * - An adjacency list is a collection of unordered lists used to represent a finite graph.
 *   Each list describes the set of neighbors of a node in the graph.
 *
 * - The given node will always be the first node with val = 1. You must return the copy of
 *   the given node as a reference to the cloned graph.
 */

/**
 * // Definition for a Node.
 * function Node(val, neighbors) {
 *    this.val = val === undefined ? 0 : val;
 *    this.neighbors = neighbors === undefined ? [] : neighbors;
 * };
 */

/**
 * @param {Node} node
 * @return {Node}
 */
var cloneGraph = function(node) {
  const map = new Map();
  return node && cloneNode(node, map);
};

function cloneNode(node, map) {
  const cloned = new Node(node.val, node.neighbors);
  map.set(node.val, cloned);

  cloned.neighbors = node.neighbors && node.neighbors.map(n => {
    return map.get(n.val) || cloneNode(n, map);
  });

  return cloned;
}
```

0134 Gas Station

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 134. Gas Station
 * https://leetcode.com/problems/gas-station/
 * Difficulty: Medium
 *
 * There are n gas stations along a circular route, where the amount of gas at the ith station
 is gas[i].
 *
 * You have a car with an unlimited gas tank and it costs cost[i] of gas to travel from the
ith station to
 * its next (i + 1)th station. You begin the journey with an empty tank at one of the gas
stations.
 *
 * Given two integer arrays gas and cost, return the starting gas station's index if you can
```

```
travel around
 * the circuit once in the clockwise direction, otherwise return -1. If there exists a
solution, it is
 * guaranteed to be unique
 */

/**
 * @param {number[]} gas
 * @param {number[]} cost
 * @return {number}
 */
var canCompleteCircuit = function(gas, cost) {
  let total = 0, tank = 0, index = 0;

  for (let i = 0; i < gas.length; i++) {
    tank += gas[i] - cost[i];
    total += gas[i] - cost[i];
    if (tank < 0) {
      tank = 0;
      index = i + 1;
    }
  }

  return total >= 0 ? index : -1;
};
```

0135 Candy

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 135. Candy
 * https://leetcode.com/problems/candy/
 * Difficulty: Hard
 *
 * There are n children standing in a line. Each child is assigned a rating value given
 * in the integer array ratings.
 *
 * You are giving candies to these children subjected to the following requirements:
 * - Each child must have at least one candy.
 * - Children with a higher rating get more candies than their neighbors.
 *
 * Return the minimum number of candies you need to have to distribute the candies to
 * the children.
 */

/**
 * @param {number[]} ratings
 * @return {number}
 */
var candy = function(ratings) {
  const data = new Array(ratings.length).fill(1);

  for (let i = 1; i < ratings.length; i++) {
    if (ratings[i - 1] < ratings[i]) {
      data[i] = data[i - 1] + 1;
    }
  }

  for (let i = ratings.length - 1; i > 0; i--) {
    if (ratings[i - 1] > ratings[i]) {
      data[i - 1] = data[i - 1] > data[i] + 1 ? data[i - 1] : data[i] + 1;
    }
  }

  return data.reduce((sum, n) => sum + n, 0);
};
```

0136 Single Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 136. Single Number
 * https://leetcode.com/problems/single-number/
 * Difficulty: Easy
 *
 * Given a non-empty array of integers nums, every element appears twice
 * except for one. Find that single one.
 *
 * You must implement a solution with a linear runtime complexity and
 * use only constant extra space.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var singleNumber = function(nums) {
  return nums.reduce((result, num) => result ^= num, 0);
};
```

0137 Single Number Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 137. Single Number II
 * https://leetcode.com/problems/single-number-ii/
 * Difficulty: Medium
 *
 * Given an integer array nums where every element appears three times except for one, which
appears
 * exactly once. Find the single element and return it.
 *
 * You must implement a solution with a linear runtime complexity and use only constant extra
space.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var singleNumber = function(nums) {
  let a = 0, b = 0;

  nums.forEach(c => {
    const offset = (~a & b & c) | (a & ~b & ~c);
    b = (~a & ~b & c) | (~a & b & ~c);
    a = offset;
  });

  return a | b;
};
```

0138 Copy List With Random Pointer

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 138. Copy List with Random Pointer
 * https://leetcode.com/problems/copy-list-with-random-pointer/
 * Difficulty: Medium
 *
 * A linked list of length n is given such that each node contains an additional random
pointer,
 * which could point to any node in the list, or null.
 *
 * Construct a deep copy of the list. The deep copy should consist of exactly n brand new
nodes,
 * where each new node has its value set to the value of its corresponding original node. Both
 * the next and random pointer of the new nodes should point to new nodes in the copied list
such
 * that the pointers in the original list and copied list represent the same list state. None
of
 * the pointers in the new list should point to nodes in the original list.
 *
 * For example, if there are two nodes X and Y in the original list, where X.random --> Y,
then
 * for the corresponding two nodes x and y in the copied list, x.random --> y.
 *
 * Return the head of the copied linked list.
 *
 * The linked list is represented in the input/output as a list of n nodes. Each node is
 * represented as a pair of [val, random_index] where:
 * - val: an integer representing Node.val
 * - random_index: the index of the node (range from 0 to n-1) that the random pointer points
 *   to, or null if it does not point to any node.
 *
 * Your code will only be given the head of the original linked list.
 */

/**
 * // Definition for a Node.
 * function Node(val, next, random) {
 *    this.val = val;
 *    this.next = next;
 *    this.random = random;
 * };
 */

/**
 * @param {Node} head
 * @return {Node}
 */
var copyRandomList = function(head) {
  const map = new Map();

  function copy(reference) {
    if (reference === null) return null;
    const instance = map.get(reference);
    if (instance) {
      return instance;
    }

    const node = new Node(reference.val);
    map.set(reference, node);
    node.next = copy(reference.next);
    node.random = copy(reference.random);

    return node;
  }

  return copy(head);
};
```

0139 Word Break

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 139. Word Break
 * https://leetcode.com/problems/word-break/
 * Difficulty: Medium
 *
 * Given a string s and a dictionary of strings wordDict, return true if s can be segmented
into
 * a space-separated sequence of one or more dictionary words.
 *
 * Note that the same word in the dictionary may be reused multiple times in the segmentation.
 */

/**
 * @param {string} s
 * @param {string[]} wordDict
 * @return {boolean}
 */
var wordBreak = function(s, wordDict) {
  const result = [1, ...new Array(s.length + 1).fill(0)];

  for (let i = 1; i <= s.length; i++) {
    for (let j = 0; j < i; j++) {
      if (result[j] && wordDict.includes(s.slice(j, i))) {
        result[i] = 1;
        break;
      }
    }
  }

  return result[s.length];
};
```

0140 Word Break Ii

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 140. Word Break II
 * https://leetcode.com/problems/word-break-ii/
 * Difficulty: Hard
 *
 * Given a string s and a dictionary of strings wordDict, add spaces in s to
 * construct a sentence where each word is a valid dictionary word. Return
 * all such possible sentences in any order.
 *
 * Note that the same word in the dictionary may be reused multiple times in
 * the segmentation.
 */

/**
 * @param {string} s
 * @param {string[]} wordDict
 * @return {string[]}
 */
var wordBreak = function(s, wordDict) {
  const result = [];
  backtrack(result, s, wordDict, '');
  return result;
};

function backtrack(result, s, wordDict, substr) {
  if (!s.length) {
    result.push(substr);
    return;
  }

  wordDict.forEach(word => {
```

```
    if (s.length >= word.length && word === s.substring(0, word.length)) {
      backtrack(
        result,
        s.substring(word.length),
        wordDict,
        substr.length ? `${substr} ${word}` : word,
      );
    }
  });
}
```

0141 Linked List Cycle

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 141. Linked List Cycle
 * https://leetcode.com/problems/linked-list-cycle/
 * Difficulty: Easy
 *
 * Given head, the head of a linked list, determine if the linked list has a cycle in it.
 *
 * There is a cycle in a linked list if there is some node in the list that can be reached
 * again by continuously following the next pointer. Internally, pos is used to denote the
 * index of the node that tail's next pointer is connected to. Note that pos is not passed
 * as a parameter.
 *
 * Return true if there is a cycle in the linked list. Otherwise, return false.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */

/**
 * @param {ListNode} head
 * @return {boolean}
 */
var hasCycle = function(head) {
  while (head) {
    if (head.visited) {
      return true;
    }
    head.visited = 1;
    head = head.next;
  }

  return false;
};
```

0142 Linked List Cycle Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 142. Linked List Cycle II
 * https://leetcode.com/problems/linked-list-cycle-ii/
 * Difficulty: Medium
 *
 * Given the head of a linked list, return the node where the cycle begins.
 * If there is no cycle, return null.
```

```
 *
 * There is a cycle in a linked list if there is some node in the list that
 * can be reached again by continuously following the next pointer. Internally,
 * pos is used to denote the index of the node that tail's next pointer is
 * connected to (0-indexed). It is -1 if there is no cycle. Note that pos is
 * not passed as a parameter.
 *
 * Do not modify the linked list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */

/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var detectCycle = function(head) {
  while (head) {
    if (head.visited) {
      return head;
    }
    head.visited = 1;
    head = head.next;
  }
  return head;
};
```

0143 Reorder List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 143. Reorder List
 * https://leetcode.com/problems/reorder-list/
 * Difficulty: Medium
 *
 * You are given the head of a singly linked-list. The list can be represented as:
 *
 * L0 → L1 → … → Ln - 1 → Ln
 * Reorder the list to be on the following form:
 *
 * L0 → Ln → L1 → Ln - 1 → L2 → Ln - 2 → …
 * You may not modify the values in the list's nodes. Only nodes themselves may be changed.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {void} Do not return anything, modify head in-place instead.
 */
var reorderList = function(head) {
  if (!head || !head.next || !head.next.next) {
    return head;
  }

  let list1 = head;
  let list2 = head;
```

```
  while (list2.next && list2.next.next) {
    list1 = list1.next;
    list2 = list2.next.next;
  }

  let center1 = list1;
  let center2 = list1.next;
  while (center2.next) {
    const temp = center2.next;
    center2.next = temp.next;
    temp.next = center1.next;
    center1.next = temp;
  }

  list1 = head;
  list2 = center1.next;
  while (list1 != center1) {
    center1.next = list2.next;
    list2.next = list1.next;
    list1.next = list2;
    list1 = list2.next;
    list2 = center1.next;
  }
};
```

0144 Binary Tree Preorder Traversal

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 144. Binary Tree Preorder Traversal
 * https://leetcode.com/problems/binary-tree-preorder-traversal/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return the preorder traversal of its nodes' values.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var preorderTraversal = function(root) {
  return root
    ? [root.val, ...preorderTraversal(root.left), ...preorderTraversal(root.right)]
    : [];
};
```

0145 Binary Tree Postorder Traversal

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 145. Binary Tree Postorder Traversal
 * https://leetcode.com/problems/binary-tree-postorder-traversal/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return the postorder traversal
```

```
 * of its nodes' values.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var postorderTraversal = function(root) {
  if (!root) {
    return [];
  }

  return [
    ...postorderTraversal(root.left),
    ...postorderTraversal(root.right),
    root.val
  ];
};
```

0146 Lru Cache

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 146. LRU Cache
 * https://leetcode.com/problems/lru-cache/
 * Difficulty: Medium
 *
 * Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.
 *
 * Implement the LRUCache class:
 * - LRUCache(int capacity) Initialize the LRU cache with positive size capacity.
 * - int get(int key) Return the value of the key if the key exists, otherwise return -1.
 * - void put(int key, int value) Update the value of the key if the key exists. Otherwise,
 *   add the key-value pair to the cache. If the number of keys exceeds the capacity from
 *   this operation, evict the least recently used key.
 *
 * The functions get and put must each run in O(1) average time complexity.
 */

/**
 * @param {number} capacity
 */
var LRUCache = function(capacity) {
  this.cache = new Map();
  this.capacity = capacity;
};

/**
 * @param {number} key
 * @return {number}
 */
LRUCache.prototype.get = function(key) {
  if (!this.cache.has(key)) {
    return -1;
  }

  const value = this.cache.get(key);
  this.cache.delete(key);
  this.cache.set(key, value);
  return this.cache.get(key);
};
```

```
/**
 * @param {number} key
 * @param {number} value
 * @return {void}
 */
LRUCache.prototype.put = function(key, value) {
  if (this.cache.has(key)) {
    this.cache.delete(key);
  }
  this.cache.set(key, value);
  if (this.cache.size > this.capacity) {
    this.cache.delete(this.cache.keys().next().value);
  }
};
```

0147 Insertion Sort List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 147. Insertion Sort List
 * https://leetcode.com/problems/insertion-sort-list/
 * Difficulty: Medium
 *
 * Given the head of a singly linked list, sort the list using insertion sort, and
 * return the sorted list's head.
 *
 * The steps of the insertion sort algorithm:
 * - Insertion sort iterates, consuming one input element each repetition and growing
 *   a sorted output list.
 * - At each iteration, insertion sort removes one element from the input data, finds
 *   the location it belongs within the sorted list and inserts it there.
 * - It repeats until no input elements remain.
 *
 * The following is a graphical example of the insertion sort algorithm. The partially
 * sorted list (black) initially contains only the first element in the list. One
 * element (red) is removed from the input data and inserted in-place into the sorted
 * list with each iteration.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var insertionSortList = function(head) {
  const result = new ListNode(0);

  while (head) {
    const tail = head;
    let current = result;
    head = head.next;

    while (current) {
      if (!current.next || tail.val <= current.next.val) {
        [current.next, tail.next] = [tail, current.next];
        break;
      }
      current = current.next;
    }
  }

  return result.next;
```

```
};
```

0148 Sort List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 148. Sort List
 * https://leetcode.com/problems/sort-list/
 * Difficulty: Medium
 *
 * Given the head of a linked list, return the list after sorting it in ascending order.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var sortList = function(head) {
  if (!head || !head.next) return head;
  let fast = head;
  let slow = head;
  while (fast.next && fast.next.next) {
    fast = fast.next.next;
    slow = slow.next;
  }
  const middle = slow.next;
  slow.next = null;
  return mergeList(sortList(head), sortList(middle));
};

function mergeList(a, b) {
  const ordered = new ListNode(-1);
  let list = ordered;

  while (a && b) {
    list.next = a.val < b.val ? a : b;
    list = list.next;
    if (a.val < b.val) {
      a = a.next;
    } else {
      b = b.next;
    }
  }

  list.next = a ?? b;
  return ordered.next;
}
```

0149 Max Points On A Line

HARD

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 149. Max Points on a Line
 * https://leetcode.com/problems/max-points-on-a-line/
 * Difficulty: Hard
 *
```

```
 * Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane,
 * return the maximum number of points that lie on the same straight line.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var maxPoints = function(points) {
  let max = 0;

  points.forEach(x => {
    const slopes = new Map();

    points.forEach(y => {
      if (x === y) return;
      const slope = y[0] - x[0] !== 0 ? (y[1] - x[1]) / (y[0] - x[0]) : Infinity;
      slopes.set(slope, (slopes.get(slope) || 0) + 1);
      max = Math.max(max, slopes.get(slope));
    });
  });

  return max + 1;
};
```

0150 Evaluate Reverse Polish Notation

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 150. Evaluate Reverse Polish Notation
 * https://leetcode.com/problems/evaluate-reverse-polish-notation/
 * Difficulty: Medium
 *
 * You are given an array of strings tokens that represents an arithmetic expression in a
 * Reverse Polish Notation.
 *
 * Evaluate the expression. Return an integer that represents the value of the expression.
 *
 * Note that:
 * - The valid operators are '+', '-', '*', and '/'.
 * - Each operand may be an integer or another expression.
 * - The division between two integers always truncates toward zero.
 * - There will not be any division by zero.
 * - The input represents a valid arithmetic expression in a reverse polish notation.
 * - The answer and all the intermediate calculations can be represented in a 32-bit integer.
 */

/**
 * @param {string[]} tokens
 * @return {number}
 */
var evalRPN = function(tokens) {
  const stack = [];
  const operators = {
    '+': (a, b) => a + b,
    '-': (a, b) => a - b,
    '*': (a, b) => a * b,
    '/': (a, b) => a / b >= 0 ? Math.floor(a / b) : Math.ceil(a / b),
  };

  tokens.forEach(token => {
    if (operators[token]) {
      const item = stack.pop();
      stack.push(operators[token](stack.pop(), item));
    } else {
      stack.push(Number(token));
    }
  });
```

```
  return stack.pop();
};
```

0151 Reverse Words In A String

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 151. Reverse Words in a String
 * https://leetcode.com/problems/reverse-words-in-a-string/
 * Difficulty: Medium
 *
 * Given an input string, reverse the string word by word.
 */

/**
 * @param {string} s
 * @return {string}
 */
var reverseWords = function(s) {
  return s.trim().replace(/\s+/g, ' ').split(/\s/).reverse().join(' ');
};
```

0152 Maximum Product Subarray

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 152. Maximum Product Subarray
 * https://leetcode.com/problems/maximum-product-subarray/
 * Difficulty: Medium
 *
 * Given an integer array nums, find the contiguous subarray within an array
 * (containing at least one number) which has the largest product.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxProduct = function(nums) {
  let result = nums[0];
  let min = 1;
  let max = 1;
  for (let n of nums) {
    [min, max] = [Math.min(n, min * n, max * n), Math.max(n, min * n, max * n)];
    result = Math.max(result, max);
  }
  return result;
};
```

0153 Find Minimum In Rotated Sorted Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 153. Find Minimum in Rotated Sorted Array
 * https://leetcode.com/problems/find-minimum-in-rotated-sorted-array/
 * Difficulty: Medium
```

```
 *
 * Suppose an array of length n sorted in ascending order is rotated between 1 and n times.
 * For example, the array nums = [0,1,2,4,5,6,7] might become:
 * - [4,5,6,7,0,1,2] if it was rotated 4 times.
 * - [0,1,2,4,5,6,7] if it was rotated 7 times.
 *
 * Notice that rotating an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array
 * [a[n-1], a[0], a[1], a[2], ..., a[n-2]].
 *
 * Given the sorted rotated array nums of unique elements, return the minimum element of this
 * array.
 *
 * You must write an algorithm that runs in O(log n) time.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMin = function(nums) {
  let left = 0;
  let right = nums.length - 1;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);

    if (nums[mid] <= nums[right]) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return nums[left];
};
```

0154 Find Minimum In Rotated Sorted Array Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 154. Find Minimum in Rotated Sorted Array II
 * https://leetcode.com/problems/find-minimum-in-rotated-sorted-array-ii/
 * Difficulty: Hard
 *
 * Suppose an array of length n sorted in ascending order is rotated between 1 and n times.
 * For example, the array nums = [0,1,4,4,5,6,7] might become:
 * - [4,5,6,7,0,1,4] if it was rotated 4 times.
 * - [0,1,4,4,5,6,7] if it was rotated 7 times.
 *
 * Notice that rotating an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array
 * [a[n-1], a[0], a[1], a[2], ..., a[n-2]].
 *
 * Given the sorted rotated array nums that may contain duplicates, return the minimum element
 * of this array.
 *
 * You must decrease the overall operation steps as much as possible.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMin = function(nums) {
  for (let i = 0; i < nums.length - 1; i++) {
    if (nums[i] > nums[i + 1]) {
      return nums[i + 1];
    }
  }
  return nums[0];
```

```
};
```

0155 Min Stack

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 155. Min Stack
 * https://leetcode.com/problems/min-stack/
 * Difficulty: Medium
 *
 * Design a stack that supports push, pop, top, and retrieving the minimum element in constant
 time.
 *
 * Implement the MinStack class:
 * - MinStack() initializes the stack object.
 * - void push(int val) pushes the element val onto the stack.
 * - void pop() removes the element on the top of the stack.
 * - int top() gets the top element of the stack.
 * - int getMin() retrieves the minimum element in the stack.
 *
 * You must implement a solution with O(1) time complexity for each function.
 */


var MinStack = function() {
  this.stack = [];
};

/**
 * @param {number} val
 * @return {void}
 */
MinStack.prototype.push = function(val) {
  this.stack.push({ val, min: this.stack.length ? Math.min(val, this.getMin()) : val });
};

/**
 * @return {void}
 */
MinStack.prototype.pop = function() {
  this.stack.pop();
};

/**
 * @return {number}
 */
MinStack.prototype.top = function() {
  return this.stack[this.stack.length - 1].val;
};

/**
 * @return {number}
 */
MinStack.prototype.getMin = function() {
  return this.stack.length ? this.stack[this.stack.length - 1].min : 0;
};
```

0156 Binary Tree Upside Down

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 156. Binary Tree Upside Down
 * https://leetcode.com/problems/binary-tree-upside-down/
```

```
 * Difficulty: Medium
 *
 * Given the root of a binary tree, turn the tree upside down and return the new root.
 *
 * You can turn a binary tree upside down with the following steps:
 * - The original left child becomes the new root.
 * - The original root becomes the new right child.
 * - The original right child becomes the new left child.
 *
 * The mentioned steps are done level by level. It is guaranteed that every right node has
 * a sibling (a left node with the same parent) and has no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var upsideDownBinaryTree = function(root) {
  if (!root || !root.left) return root;

  const newRoot = upsideDownBinaryTree(root.left);
  root.left.left = root.right;
  root.left.right = root;
  root.left = null;
  root.right = null;

  return newRoot;
};
```

0157 Read N Characters Given Read4

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 157. Read N Characters Given Read4
 * https://leetcode.com/problems/read-n-characters-given-read4/
 * Difficulty: Easy
 *
 * Given a file and assume that you can only read the file using a given method read4,
implement
 * a method to read n characters.
 *
 * Method read4:
 * - The API read4 reads four consecutive characters from file, then writes those characters
into
 *   the buffer array buf4.
 *
 * The return value is the number of actual characters read.
 *
 * Note that read4() has its own file pointer, much like FILE *fp in C.
 */

/**
 * @param {function} read4()
 * @return {function}
 */
var solution = function(read4) {
  let cache = [];
  let cacheIndex = 0;
  let cacheSize = 0;

  /**
```

```
    * @param {character[]} buf Destination buffer
    * @param {number} n Number of characters to read
    * @return {number} The number of actual characters read
    */
  return function(buf, n) {
    let charsRead = 0;

    while (charsRead < n) {
      if (cacheIndex >= cacheSize) {
        cache = new Array(4);
        cacheSize = read4(cache);
        cacheIndex = 0;
        if (cacheSize === 0) break;
      }

      buf[charsRead++] = cache[cacheIndex++];
    }

    return charsRead;
  };
};
```

0158 Read N Characters Given Read4 Ii Call Multiple Times

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 158. Read N Characters Given read4 II - Call Multiple Times
 * https://leetcode.com/problems/read-n-characters-given-read4-ii-call-multiple-times/
 * Difficulty: Hard
 *
 * Given a file and assume that you can only read the file using a given method read4,
implement
 * a method read to read n characters. Your method read may be called multiple times.
 *
 * Method read4:
 * - The API read4 reads four consecutive characters from file, then writes those characters
into
 *   the buffer array buf4.
 *
 * The return value is the number of actual characters read.
 *
 * Note that read4() has its own file pointer, much like FILE *fp in C.
 */

/**
 * @param {function} read4()
 * @return {function}
 */
var solution = function(read4) {
  let cache = [];
  let cacheIndex = 0;
  let cacheSize = 0;

  /**
    * @param {character[]} buf Destination buffer
    * @param {number} n Number of characters to read
    * @return {number} The number of actual characters read
    */
  return function(buf, n) {
    let charsRead = 0;

    while (charsRead < n) {
      if (cacheIndex >= cacheSize) {
        cache = new Array(4);
        cacheSize = read4(cache);
        cacheIndex = 0;
        if (cacheSize === 0) break;
      }
```

```
      buf[charsRead++] = cache[cacheIndex++];
    }

    return charsRead;
  };
};
```

0159 Longest Substring With At Most Two Distinct Characters

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 159. Longest Substring with At Most Two Distinct Characters
 * https://leetcode.com/problems/longest-substring-with-at-most-two-distinct-characters/
 * Difficulty: Medium
 *
 * Given a string s, return the length of the longest substring that contains at most two
 * distinct characters.
 */

/**
 * @param {string} s
 * @return {number}
 */
var lengthOfLongestSubstringTwoDistinct = function(s) {
  const map = new Map();
  let maxLength = 0;
  let start = 0;

  for (let end = 0; end < s.length; end++) {
    map.set(s[end], (map.get(s[end]) || 0) + 1);

    while (map.size > 2) {
      map.set(s[start], map.get(s[start]) - 1);
      if (map.get(s[start]) === 0) {
        map.delete(s[start]);
      }
      start++;
    }

    maxLength = Math.max(maxLength, end - start + 1);
  }

  return maxLength;
};
```

0160 Intersection Of Two Linked Lists

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 160. Intersection of Two Linked Lists
 * https://leetcode.com/problems/intersection-of-two-linked-lists/
 * Difficulty: Medium
 *
 * Given the heads of two singly linked-lists headA and headB, return the
 * node at which the two lists intersect. If the two linked lists have no
 * intersection at all, return null.
 *
 * For example, the following two linked lists begin to intersect at node c1:
 */

/**
 * Definition for singly-linked list.
```

```
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */

/**
 * @param {ListNode} headA
 * @param {ListNode} headB
 * @return {ListNode}
 */
var getIntersectionNode = function(headA, headB) {
  if (!headA || !headB) {
    return null;
  }

  let a = headA;
  let b = headB;

  while (a !== b) {
    a = a === null ? headB : a.next;
    b = b === null ? headA : b.next;
  }

  return a;
};
```

0161 One Edit Distance

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 161. One Edit Distance
 * https://leetcode.com/problems/one-edit-distance/
 * Difficulty: Medium
 *
 * Given two strings s and t, return true if they are both one edit distance apart, otherwise
 * return false.
 *
 * A string s is said to be one distance apart from a string t if you can:
 * - Insert exactly one character into s to get t.
 * - Delete exactly one character from s to get t.
 * - Replace exactly one character of s with a different character to get t.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isOneEditDistance = function(s, t) {
  if (s === t) return false;
  const sLength = s.length;
  const tLength = t.length;
  if (Math.abs(sLength - tLength) > 1) return false;

  if (sLength > tLength) return isOneEditDistance(t, s);

  let i = 0;
  while (i < sLength && s[i] === t[i]) i++;

  if (sLength === tLength) {
    return i < sLength && s.slice(i + 1) === t.slice(i + 1);
  }

  return s.slice(i) === t.slice(i + 1);
};
```

0162 Find Peak Element

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 162. Find Peak Element
 * https://leetcode.com/problems/find-peak-element/
 * Difficulty: Medium
 *
 * A peak element is an element that is strictly greater than its neighbors.
 *
 * Given a 0-indexed integer array nums, find a peak element, and return its index.
 * If the array contains multiple peaks, return the index to any of the peaks.
 *
 * You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is
 * always considered to be strictly greater than a neighbor that is outside the array.
 *
 * You must write an algorithm that runs in O(log n) time.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findPeakElement = function(nums) {
  let left = 0;

  for (let right = nums.length - 1, middle = 0; left < right;) {
    middle = Math.floor((right + left) / 2);
    if (nums[middle] > nums[middle + 1]) {
      right = middle;
    } else {
      left = middle + 1;
    }
  }

  return left;
};
```

0163 Missing Ranges

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 163. Missing Ranges
 * https://leetcode.com/problems/missing-ranges/
 * Difficulty: Easy
 *
 * You are given an inclusive range [lower, upper] and a sorted unique integer array nums,
 * where all elements are within the inclusive range.
 *
 * A number x is considered missing if x is in the range [lower, upper] and x is not in nums.
 *
 * Return the shortest sorted list of ranges that exactly covers all the missing numbers.
 * That is, no element of nums is included in any of the ranges, and each missing number
 * is covered by one of the ranges.
 */

/**
 * @param {number[]} nums
 * @param {number} lower
 * @param {number} upper
 * @return {number[][]}
 */
var findMissingRanges = function(nums, lower, upper) {
```

```
  const result = [];
  let prev = lower - 1;

  for (let i = 0; i <= nums.length; i++) {
    const curr = i < nums.length ? nums[i] : upper + 1;
    if (curr - prev > 1) {
      result.push([prev + 1, curr - 1]);
    }
    prev = curr;
  }

  return result;
};
```

0164 Maximum Gap

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 164. Maximum Gap
 * https://leetcode.com/problems/maximum-gap/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the maximum difference between two successive elements
in
 * its sorted form. If the array contains less than two elements, return 0.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumGap = function(nums) {
  nums.sort((a, b) => a - b);
  let result = 0;
  for (let i = 1; i < nums.length; i++) {
    result = Math.max(result, nums[i] - nums[i - 1]);
  }
  return result;
};
```

0165 Compare Version Numbers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 165. Compare Version Numbers
 * https://leetcode.com/problems/compare-version-numbers/
 * Difficulty: Medium
 *
 * Given two version strings, version1 and version2, compare them. A version string
 * consists of revisions separated by dots '.'. The value of the revision is its
 * integer conversion ignoring leading zeros.
 *
 * To compare version strings, compare their revision values in left-to-right order.
 * If one of the version strings has fewer revisions, treat the missing revision
 * values as 0.
 *
 * Return the following:
 * - If version1 < version2, return -1.
 * - If version1 > version2, return 1.
 * - Otherwise, return 0.
 */

/**
```

```
 * @param {string} version1
 * @param {string} version2
 * @return {number}
 */
var compareVersion = function(version1, version2) {
  const [v1, v2] = [version1.split(/\./), version2.split(/\./)];

  while (v1.length || v2.length) {
    const [a, b] = [v1.shift(), v2.shift()].map(s => Number(s ?? 0));
    if (a < b) return -1;
    if (a > b) return 1;
  }

  return 0;
};
```

0166 Fraction To Recurring Decimal

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 166. Fraction to Recurring Decimal
 * https://leetcode.com/problems/fraction-to-recurring-decimal/
 * Difficulty: Medium
 *
 * Given two integers representing the numerator and denominator of a fraction, return
 * the fraction in string format.
 *
 * If the fractional part is repeating, enclose the repeating part in parentheses.
 *
 * If multiple answers are possible, return any of them.
 *
 * It is guaranteed that the length of the answer string is less than 104 for all the
 * given inputs.
 */

/**
 * @param {number} numerator
 * @param {number} denominator
 * @return {string}
 */
var fractionToDecimal = function(numerator, denominator) {
  if (numerator === 0) return '0';

  const map = new Map();
  let result = '';

  if (Math.sign(numerator) !== Math.sign(denominator)) {
    result += '-';
  }

  numerator = Math.abs(numerator);
  denominator = Math.abs(denominator);
  result += Math.floor(numerator / denominator);

  let remainder = numerator % denominator;
  while (remainder) {
    result = result.indexOf('.') === -1 ? `${result}.` : result;
    if (map.has(remainder)) {
      const index = map.get(remainder);
      result = `${result.slice(0, index)}(${result.slice(index)})`;
      break;
    }
    map.set(remainder, result.length);
    remainder *= 10;
    result += Math.floor(remainder / denominator);
    remainder %= denominator;
  }

  return result;
```

```
};
```

0167 Two Sum Ii Input Array Is Sorted

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 167. Two Sum II - Input Array Is Sorted
 * https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/
 * Difficulty: Easy
 *
 * Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order,
 * find two numbers such that they add up to a specific target number. Let these two numbers
 * be numbers[index1] and numbers[index2] where 1 <= index1 < index2 <= numbers.length.
 *
 * Return the indices of the two numbers, index1 and index2, added by one as an integer array
 * [index1, index2] of length 2.
 *
 * The tests are generated such that there is exactly one solution. You may not use the same
 * element twice.
 */

/**
 * @param {number[]} numbers
 * @param {number} target
 * @return {number[]}
 */
var twoSum = function(numbers, target) {
  const map = new Map();

  for (let i = 0; i < numbers.length; i++) {
    const diff = target - numbers[i];

    if (map.has(diff)) {
      return [map.get(diff) + 1, i + 1];
    }

    map.set(numbers[i], i);
  }
};
```

0168 Excel Sheet Column Title

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 168. Excel Sheet Column Title
 * https://leetcode.com/problems/excel-sheet-column-title/
 * Difficulty: Easy
 *
 * Given an integer columnNumber, return its corresponding column
 * title as it appears in an Excel sheet.
 */

/**
 * @param {number} columnNumber
 * @return {string}
 */
var convertToTitle = function(columnNumber) {
  const n = columnNumber - 1;
  return n >= 0 && n < 26
    ? String.fromCharCode(65 + n)
    : convertToTitle(parseInt(n / 26)) + convertToTitle((n % 26) + 1);
};
```

0169 Majority Element

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 169. Majority Element
 * https://leetcode.com/problems/majority-element/
 * Difficulty: Easy
 *
 * Given an array nums of size n, return the majority element.
 *
 * The majority element is the element that appears more than ■n / 2■ times.
 * You may assume that the majority element always exists in the array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var majorityElement = function(nums) {
  const map = new Map();
  nums.forEach(n => map.set(n, (map.get(n) || 0) + 1));

  const sorted = [...map].sort(([,a], [,b]) => b - a);
  return sorted[0][0];
};
```

0170 Two Sum Iii Data Structure Design

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 170. Two Sum III - Data structure design
 * https://leetcode.com/problems/two-sum-iii-data-structure-design/
 * Difficulty: Easy
 *
 * Design a data structure that accepts a stream of integers and checks if it has a pair of
 * integers that sum up to a particular value.
 *
 * Implement the TwoSum class:
 * - TwoSum() Initializes the TwoSum object, with an empty array initially.
 * - void add(int number) Adds number to the data structure.
 * - boolean find(int value) Returns true if there exists any pair of numbers whose sum is
equal
 *   to value, otherwise, it returns false.
 */

var TwoSum = function() {
  this.numCount = new Map();
};

/**
 * @param {number} number
 * @return {void}
 */
TwoSum.prototype.add = function(number) {
  this.numCount.set(number, (this.numCount.get(number) || 0) + 1);
};

/**
 * @param {number} value
 * @return {boolean}
 */
TwoSum.prototype.find = function(value) {
  for (const num of this.numCount.keys()) {
```

```
    const complement = value - num;
    if (complement === num) {
      if (this.numCount.get(num) > 1) return true;
    } else if (this.numCount.has(complement)) {
      return true;
    }
  }
  return false;
};
```

0171 Excel Sheet Column Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 171. Excel Sheet Column Number
 * https://leetcode.com/problems/excel-sheet-column-number/
 * Difficulty: Easy
 *
 * Given a string columnTitle that represents the column title as appears in
 * an Excel sheet, return its corresponding column number.
 */

/**
 * @param {string} columnTitle
 * @return {number}
 */
var titleToNumber = function(columnTitle) {
  return columnTitle.split('').reduce((result, c, i) => {
    return result + (c.charCodeAt() - 64) * Math.pow(26, columnTitle.length - (i + 1));
  }, 0);
};
```

0172 Factorial Trailing Zeroes

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 172. Factorial Trailing Zeroes
 * https://leetcode.com/problems/factorial-trailing-zeroes/
 * Difficulty: Medium
 *
 * Given an integer n, return the number of trailing zeroes in n!.
 *
 * Note that n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1.
 */

/**
 * @param {number} n
 * @return {number}
 */
var trailingZeroes = function(n) {
  return n < 5 ? 0 : Math.floor(n / 5) + trailingZeroes(n / 5);
};
```

0173 Binary Search Tree Iterator

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 173. Binary Search Tree Iterator
 * https://leetcode.com/problems/binary-search-tree-iterator/
 * Difficulty: Medium
 *
 * Implement the BSTIterator class that represents an iterator over the in-order traversal
 * of a binary search tree (BST):
 * - BSTIterator(TreeNode root) Initializes an object of the BSTIterator class. The root of
 *   the BST is given as part of the constructor. The pointer should be initialized to a
 *   non-existent number smaller than any element in the BST.
 * - boolean hasNext() Returns true if there exists a number in the traversal to the right
 *   of the pointer, otherwise returns false.
 * - int next() Moves the pointer to the right, then returns the number at the pointer.
 *
 * Notice that by initializing the pointer to a non-existent smallest number, the first call
 * to next() will return the smallest element in the BST.
 *
 * You may assume that next() calls will always be valid. That is, there will be at least a
 * next number in the in-order traversal when next() is called.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 */
var BSTIterator = function(root) {
  this.stack = [];
  this.root = root;
};

/**
 * @return {number}
 */
BSTIterator.prototype.next = function() {
  while (this.root) {
    this.stack.push(this.root);
    this.root = this.root.left;
  }
  this.root = this.stack.pop();

  const result = this.root.val;
  this.root = this.root.right;
  return result;
};

/**
 * @return {boolean}
 */
BSTIterator.prototype.hasNext = function() {
  return this.root || this.stack.length;
};
```

0174 Dungeon Game

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 174. Dungeon Game
 * https://leetcode.com/problems/dungeon-game/
 * Difficulty: Hard
 *
 * The demons had captured the princess and imprisoned her in the bottom-right corner of a
dungeon.
```

```
 * The dungeon consists of m x n rooms laid out in a 2D grid. Our valiant knight was initially
 * positioned in the top-left room and must fight his way through dungeon to rescue the
princess.
 *
 * The knight has an initial health point represented by a positive integer. If at any point
his
 * health point drops to 0 or below, he dies immediately.
 *
 * Some of the rooms are guarded by demons (represented by negative integers), so the knight
loses
 * health upon entering these rooms; other rooms are either empty (represented as 0) or
contain
 * magic orbs that increase the knight's health (represented by positive integers).
 *
 * To reach the princess as quickly as possible, the knight decides to move only rightward or
 * downward in each step.
 *
 * Return the knight's minimum initial health so that he can rescue the princess.
 *
 * Note that any room can contain threats or power-ups, even the first room the knight enters
and
 * the bottom-right room where the princess is imprisoned.
 */

/**
 * @param {number[][]} dungeon
 * @return {number}
 */
var calculateMinimumHP = function(dungeon) {
  const dp = new Array(dungeon.length).fill(0).map(() => new
Array(dungeon[0].length).fill(0));
  return traverse(dungeon, dungeon.length, dungeon[0].length, 0, 0, dp);
};

function traverse(dungeon, i, j, row, col, dp) {
  if (row === i - 1 && col === j - 1) {
    return Math.max(1, 1 - dungeon[row][col]);
  } else if (row >= i || col >= j) {
    return Infinity;
  } else if (dp[row][col]) {
    return dp[row][col];
  }
  const right = traverse(dungeon, i, j, row, col + 1, dp);
  const down = traverse(dungeon, i, j, row + 1, col, dp);
  dp[row][col] = Math.max(1, Math.min(right, down) - dungeon[row][col]);
  return dp[row][col];
};
```

0179 Largest Number

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 179. Largest Number
 * https://leetcode.com/problems/largest-number/
 * Difficulty: Medium
 *
 * Given a list of non-negative integers nums, arrange them such that they
 * form the largest number.
 *
 * Note: The result may be very large, so you need to return a string
 * instead of an integer.
 */

/**
 * @param {number[]} nums
 * @return {string}
 */
var largestNumber = function(nums) {
  return nums
```

```
    .sort((a, b) => `${b}${a}` - `${a}${b}`)
    .join('')
    .replace(/^0+/, 0);
};
```

0186 Reverse Words In A String Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 186. Reverse Words in a String II
 * https://leetcode.com/problems/reverse-words-in-a-string-ii/
 * Difficulty: Medium
 *
 * Given a character array s, reverse the order of the words.
 *
 * A word is defined as a sequence of non-space characters. The words in s will be separated
 * by a single space.
 *
 * Your code must solve the problem in-place, i.e. without allocating extra space.
 */

/**
 * @param {character[]} s
 * @return {void} Do not return anything, modify s in-place instead.
 */
var reverseWords = function(s) {
  let left = 0;
  let right = s.length - 1;

  while (left < right) {
    [s[left], s[right]] = [s[right], s[left]];
    left++;
    right--;
  }

  left = 0;
  for (let i = 0; i <= s.length; i++) {
    if (i === s.length || s[i] === ' ') {
      right = i - 1;
      while (left < right) {
        [s[left], s[right]] = [s[right], s[left]];
        left++;
        right--;
      }
      left = i + 1;
    }
  }
};
```

0187 Repeated Dna Sequences

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 187. Repeated DNA Sequences
 * https://leetcode.com/problems/repeated-dna-sequences/
 * Difficulty: Medium
 *
 * The DNA sequence is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and
'T':
 * - For example, "ACGAATTCCG" is a DNA sequence.
 *
 * When studying DNA, it is useful to identify repeated sequences within the DNA.
 *
```

```
 * Given a string s that represents a DNA sequence, return all the 10-letter-long sequences
 * (substrings) that occur more than once in a DNA molecule. You may return the answer in any
order.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var findRepeatedDnaSequences = function(s) {
  let substring = s.slice(0, 10);
  const set = new Set([substring]);
  const result = new Set();

  for (let i = 10; i < s.length; i++) {
    substring = substring.slice(1) + s[i];
    if (set.has(substring)) {
      result.add(substring);
    }
    set.add(substring);
  }

  return [...result];
};
```

0188 Best Time To Buy And Sell Stock Iv

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 188. Best Time to Buy and Sell Stock IV
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-iv/
 * Difficulty: Hard
 *
 * You are given an integer array prices where prices[i] is the price of a given stock on
 * the ith day, and an integer k.
 *
 * Find the maximum profit you can achieve. You may complete at most k transactions: i.e.
 * you may buy at most k times and sell at most k times.
 *
 * Note: You may not engage in multiple transactions simultaneously (i.e., you must sell
 * the stock before you buy again).
 */

/**
 * @param {number} k
 * @param {number[]} prices
 * @return {number}
 */
var maxProfit = function(k, prices) {
  if (prices.length < 2 || k === 0) return 0;

  if (k >= prices.length / 2) {
    let profit = 0;
    for (let i = 1; i < prices.length; i++) {
      profit += prices[i] > prices[i - 1] ? (prices[i] - prices[i - 1]) : 0;
    }
    return profit;
  }

  const buy = new Array(k + 1).fill(-Infinity);
  const sell = new Array(k + 1).fill(0);
  for (let i = 0; i < prices.length; i++) {
    for (let j = k; j >= 1; j--) {
      sell[j] = Math.max(sell[j], buy[j] + prices[i]);
      buy[j] = Math.max(buy[j], sell[j - 1] - prices[i]);
    }
  }

  return sell[k];
```

```
};
```

0189 Rotate Array

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 189. Rotate Array
 * https://leetcode.com/problems/rotate-array/
 * Difficulty: Medium
 *
 * Given an array, rotate the array to the right by k steps, where k is non-negative.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var rotate = function(nums, k) {
  nums.unshift(...nums.splice((k % nums.length) * -1));
};
```

0190 Reverse Bits

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 190. Reverse Bits
 * https://leetcode.com/problems/reverse-bits/
 * Difficulty: Easy
 *
 * Reverse bits of a given 32 bits unsigned integer.
 */

/**
 * @param {number} n - a positive integer
 * @return {number} - a positive integer
 */
var reverseBits = function(n) {
  return parseInt([...n.toString(2)].reverse().join('').padEnd(32, '0'), 2);
};
```

0191 Number Of 1 Bits

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 191. Number of 1 Bits
 * https://leetcode.com/problems/number-of-1-bits/
 * Difficulty: Easy
 *
 * Write a function that takes an unsigned integer and returns the number of
 * '1' bits it has (also known as the Hamming weight).
 */

/**
 * @param {number} n - a positive integer
 * @return {number}
```

```
 */
var hammingWeight = function(n) {
  return n.toString(2).match(/1/g)?.length || 0;
};
```

0198 House Robber

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 198. House Robber
 * https://leetcode.com/problems/house-robber/
 * Difficulty: Medium
 *
 * You are a professional robber planning to rob houses along a street. Each house has a
certain
 * amount of money stashed, the only constraint stopping you from robbing each of them is that
 * adjacent houses have security systems connected and it will automatically contact the
police
 * if two adjacent houses were broken into on the same night.
 *
 * Given an integer array nums representing the amount of money of each house, return the
maximum
 * amount of money you can rob tonight without alerting the police.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var rob = function(nums) {
  let previous = 0;
  let current = 0;

  for (const n of nums) {
    const temp = previous;
    previous = current;
    current = Math.max(temp + n, previous);
  }

  return Math.max(current, previous);
};
```

0199 Binary Tree Right Side View

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 199. Binary Tree Right Side View
 * https://leetcode.com/problems/binary-tree-right-side-view/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, imagine yourself standing on the right side of it, return
the
 * values of the nodes you can see ordered from top to bottom.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
```

```
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var rightSideView = function(root, result = [], depth = 0) {
  if (!root) return result;
  result[depth] = root.val;
  rightSideView(root.left, result, depth + 1);
  return rightSideView(root.right, result, depth + 1);
};
```

0200 Number Of Islands

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 200. Number of Islands
 * https://leetcode.com/problems/number-of-islands/
 * Difficulty: Medium
 *
 * Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water),
 * return the number of islands.
 *
 * An island is surrounded by water and is formed by connecting adjacent lands horizontally
 * or vertically. You may assume all four edges of the grid are all surrounded by water.
 */

/**
 * @param {character[][]} grid
 * @return {number}
 */
var numIslands = function(grid) {
  let count = 0;

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[i].length; j++) {
      if (grid[i][j] === '1') {
        count++;
        dfs(i, j);
      }
    }
  }

  function dfs(i, j) {
    if (i < 0 || i >= grid.length || j < 0 || j >= grid[i].length || grid[i][j] === '0') {
      return;
    }

    grid[i][j] = '0';

    dfs(i + 1, j);
    dfs(i - 1, j);
    dfs(i, j + 1);
    dfs(i, j - 1);
  }

  return count;
};
```

0201 Bitwise And Of Numbers Range

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 201. Bitwise AND of Numbers Range
```

```
 * https://leetcode.com/problems/bitwise-and-of-numbers-range/
 * Difficulty: Medium
 *
 * Given two integers left and right that represent the range [left, right], return the
 * bitwise AND of all numbers in this range, inclusive.
 */

/**
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
var rangeBitwiseAnd = function(left, right) {
  let count = 0;
  while (left !== right) {
    left >>= 1;
    right >>= 1;
    count++;
  }
  return left << count;
};
```

0202 Happy Number

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 202. Happy Number
 * https://leetcode.com/problems/happy-number/
 * Difficulty: Easy
 *
 * Write an algorithm to determine if a number n is happy.
 *
 * A happy number is a number defined by the following process:
 * - Starting with any positive integer, replace the number by the sum of the squares of its
digits.
 * - Repeat the process until the number equals 1 (where it will stay), or it loops endlessly
in a
 *   cycle which does not include 1.
 * - Those numbers for which this process ends in 1 are happy.
 *
 * Return true if n is a happy number, and false if not.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isHappy = function(n) {
  while (n !== 1) {
    n = [...String(n)].reduce((sum, digit) => sum + digit ** 2, 0);
    if (n === 4) {
      return false;
    }
  }
  return true;
};
```

0203 Remove Linked List Elements

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 203. Remove Linked List Elements
 * https://leetcode.com/problems/remove-linked-list-elements/
```

```
 * Difficulty: Easy
 *
 * Given the head of a linked list and an integer val, remove all the
 * nodes of the linked list that has Node.val == val, and return the
 * new head.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} val
 * @return {ListNode}
 */
var removeElements = function(head, val) {
  if (!head) return null;
  head.next = removeElements(head.next, val);
  return head.val === val ? head.next : head;
};
```

0204 Count Primes

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 204. Count Primes
 * https://leetcode.com/problems/count-primes/
 * Difficulty: Medium
 *
 * Given an integer n, return the number of prime numbers that are strictly less than n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countPrimes = function(n) {
  const nonPrimes = [];
  let result = 0;
  for (let i = 2; i < n; i++) {
    if (!nonPrimes[i]) {
      result++;
      for (let j = 2; i * j < n; j++) {
        nonPrimes[i * j] = 1;
      }
    }
  }
  return result;
};
```

0205 Isomorphic Strings

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 205. Isomorphic Strings
 * https://leetcode.com/problems/isomorphic-strings/
 * Difficulty: Easy
 *
 * Given two strings s and t, determine if they are isomorphic.
```

```
 *
 * Two strings s and t are isomorphic if the characters in s can be replaced to get t.
 *
 * All occurrences of a character must be replaced with another character while preserving
 * the order of characters. No two characters may map to the same character, but a character
 * may map to itself.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isIsomorphic = function(s, t) {
  for (let i = 0; i < s.length; i++) {
    if (s.indexOf(s[i], i + 1) !== t.indexOf(t[i], i + 1)) {
      return false;
    }
  }
  return true;
};
```

0206 Reverse Linked List

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 206. Reverse Linked List
 * https://leetcode.com/problems/reverse-linked-list/
 * Difficulty: Easy
 *
 * Given the head of a singly linked list, reverse the list, and return the reversed list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var reverseList = function(head) {
  let prev = null;
  let tail = head;

  while (tail) {
    const next = tail.next;
    tail.next = prev;
    prev = tail;
    tail = next;
  }

  return prev;
};
```

0207 Course Schedule

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 207. Course Schedule
```

```
 * https://leetcode.com/problems/course-schedule/
 * Difficulty: Medium
 *
 * There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1.
 * You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you
 * must take course bi first if you want to take course ai.
 *
 * For example, the pair [0, 1], indicates that to take course 0 you have to first take course
 1.
 *
 * Return true if you can finish all courses. Otherwise, return false.
 */

/**
 * @param {number} numCourses
 * @param {number[][]} prerequisites
 * @return {boolean}
 */
var canFinish = function(numCourses, prerequisites) {
  const graph = new Map();
  const seen = new Set();

  prerequisites.forEach(([course, prerequisite]) => {
    graph.set(course, graph.get(course) || []);
    graph.get(course).push(prerequisite);
  });

  function dfs(course) {
    if (seen.has(course)) {
      return false;
    }
    seen.add(course);
    for (const prerequisite of graph.get(course) || []) {
      if (!dfs(prerequisite)) return false;
    }
    seen.delete(course);
    graph.set(course, []);
    return true;
  }

  for (let i = 0; i < numCourses; i++) {
    if (!dfs(i)) {
      return false;
    }
  }

  return true;
};
```

0208 Implement Trie Prefix Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 208. Implement Trie (Prefix Tree)
 * https://leetcode.com/problems/implement-trie-prefix-tree/
 * Difficulty: Medium
 *
 * A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently
store
 * and retrieve keys in a dataset of strings. There are various applications of this data
structure,
 * such as autocomplete and spellchecker.
 *
 * Implement the Trie class:
 * - Trie() Initializes the trie object.
 * - void insert(String word) Inserts the string word into the trie.
 * - boolean search(String word) Returns true if the string word is in the trie (i.e., was
inserted
 *   before), and false otherwise.
```

```
 * - boolean startsWith(String prefix) Returns true if there is a previously inserted string
word
 *   that has the prefix prefix, and false otherwise.
 */

var Trie = function() {
  this.root = {};
};

/**
 * @param {string} word
 * @return {void}
 */
Trie.prototype.insert = function(word) {
  let node = this.root;
  for (const char of word) {
    if (!node[char]) {
      node[char] = {};
    }
    node = node[char];
  }
  node.isWord = true;
};

/**
 * @param {string} word
 * @return {boolean}
 */
Trie.prototype.search = function(word) {
  const node = this.find(word);
  return node != null && node.isWord === true;
};

/**
 * @param {string} prefix
 * @return {boolean}
 */
Trie.prototype.startsWith = function(prefix) {
  return this.find(prefix) !== null;
};

/**
 * @param {string} word
 * @return {boolean}
 */
Trie.prototype.find = function(word) {
  let node = this.root;
  for (const char of word) {
    node = node[char];
    if (!node) {
      return null;
    }
  }
  return node;
};
```

0209 Minimum Size Subarray Sum

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 209. Minimum Size Subarray Sum
 * https://leetcode.com/problems/minimum-size-subarray-sum/
 * Difficulty: Medium
 *
 * Given an array of positive integers nums and a positive integer target, return the
 * minimal length of a subarray whose sum is greater than or equal to target. If there
 * is no such subarray, return 0 instead.
 */
```

```
/**
 * @param {number} target
 * @param {number[]} nums
 * @return {number}
 */
var minSubArrayLen = function(target, nums) {
  let result = Infinity;

  for (let right = 0, sum = 0, left = 0; right < nums.length; right++) {
    sum += nums[right];
    while (sum >= target) {
      result = Math.min(result, right - left + 1);
      sum -= nums[left];
      left++;
    }
  }

  return result === Infinity ? 0 : result;
};
```

0210 Course Schedule Ii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 210. Course Schedule II
 * https://leetcode.com/problems/course-schedule-ii/
 * Difficulty: Medium
 *
 * There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1.
 * You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you
 * must take course bi first if you want to take course ai.
 *
 * For example, the pair [0, 1], indicates that to take course 0 you have to first take course
 1.
 *
 * Return the ordering of courses you should take to finish all courses. If there are many
valid
 * answers, return any of them. If it is impossible to finish all courses, return an empty
array.
 */

/**
 * @param {number} numCourses
 * @param {number[][]} prerequisites
 * @return {number[]}
 */
var findOrder = function(numCourses, prerequisites) {
  const graph = Array(numCourses).fill().map(() => []);
  const seen = new Set();
  const path = new Set();
  const result = [];

  prerequisites.forEach(([c, p]) => graph[p].push(c));

  for (let i = 0; i < numCourses; i++) {
    if (!seen.has(i) && !dfs(i)) {
      return [];
    }
  }

  return result;

  function dfs(course) {
    if (path.has(course)) return false;
    if (seen.has(course)) return true;

    path.add(course);
    for (const c of graph[course]) {
```

```
      if (!dfs(c)) {
        return false;
      }
    }
    path.delete(course);
    seen.add(course);
    result.unshift(course);
    return true;
  }
};
```

0211 Design Add And Search Words Data Structure

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 211. Design Add and Search Words Data Structure
 * https://leetcode.com/problems/design-add-and-search-words-data-structure/
 * Difficulty: Medium
 *
 * Design a data structure that supports adding new words and finding if a string matches
 * any previously added string.
 *
 * Implement the WordDictionary class:
 * - WordDictionary() Initializes the object.
 * - void addWord(word) Adds word to the data structure, it can be matched later.
 * - bool search(word) Returns true if there is any string in the data structure that
 *   matches word or false otherwise. word may contain dots '.' where dots can be
 *   matched with any letter.
 */


var WordDictionary = function() {
  this.root = {};
};

/**
 * @param {string} word
 * @return {void}
 */
WordDictionary.prototype.addWord = function(word) {
  let node = this.root;

  for (const character of word) {
    node[character] = node[character] || {};
    node = node[character];
  }

  node.isTail = true;
};

/**
 * @param {string} word
 * @return {boolean}
 */
WordDictionary.prototype.search = function(word) {
  return dfs(this.root, 0);

  function dfs(node, i) {
    if (word.length === i) {
      return node.isTail;
    }
    if (word[i] === '.') {
      for (const key in node) {
        if (dfs(node[key], i + 1)) {
          return true;
        }
      }
    } else if (node[word[i]] && dfs(node[word[i]], i + 1)) {
      return true;
```

```
    }

    return false;
  }
};
```

0212 Word Search Ii

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 212. Word Search II
 * https://leetcode.com/problems/word-search-ii/
 * Difficulty: Hard
 *
 * Given an m x n board of characters and a list of strings words, return all words on the
board.
 *
 * Each word must be constructed from letters of sequentially adjacent cells, where adjacent
cells
 * are horizontally or vertically neighboring. The same letter cell may not be used more than
once
 * in a word.
 */

/**
 * @param {character[][]} board
 * @param {string[]} words
 * @return {string[]}
 */
var findWords = function(board, words) {
  const root = {};
  const result = new Set();
  const m = board.length;
  const n = board[0].length;

  words.forEach(w => w.split('').reduce((n, c) => n[c] = n[c] || {}, root).word = w);

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      dfs(i, j, root);
    }
  }

  return Array.from(result);

  function dfs(i, j, node) {
    const char = board[i][j];
    const next = node[char];
    if (!next) return;
    if (next.word) {
      result.add(next.word);
    }
    board[i][j] = '#';
    for (const [di, dj] of [[0, 1], [1, 0], [0, -1], [-1, 0]]) {
      if (i + di >= 0 && i + di < m && j + dj >= 0 && j + dj < n && board[i + di][j + dj] !==
'#') {
        dfs(i + di, j + dj, next);
      }
    }
    board[i][j] = char;
  }
};
```

0213 House Robber Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 213. House Robber II
 * https://leetcode.com/problems/house-robber-ii/
 * Difficulty: Medium
 *
 * You are a professional robber planning to rob houses along a street. Each house has a
certain
 * amount of money stashed. All houses at this place are arranged in a circle. That means the
 * first house is the neighbor of the last one. Meanwhile, adjacent houses have a security
system
 * connected, and it will automatically contact the police if two adjacent houses were broken
into
 * on the same night.
 *
 * Given an integer array nums representing the amount of money of each house, return the
maximum
 * amount of money you can rob tonight without alerting the police.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var rob = function(nums) {
  if (nums.length === 1) {
    return nums[0];
  }

  return Math.max(
    handlePermutation(nums, 0, nums.length - 1),
    handlePermutation(nums, 1, nums.length)
  );
};

function handlePermutation(nums, start, end) {
  let previous = 0, current = 0;

  for (let i = start; i < end; i++) {
    const temp = previous;
    previous = current;
    current = Math.max(temp + nums[i], previous);
  }

  return Math.max(current, previous);
}
```

0214 Shortest Palindrome

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 214. Shortest Palindrome
 * https://leetcode.com/problems/shortest-palindrome/
 * Difficulty: Hard
 *
 * You are given a string s. You can convert s to a palindrome by adding characters
 * in front of it.
 *
 * Return the shortest palindrome you can find by performing this transformation.
 */
```

```javascript
/**
 * @param {string} s
 * @return {string}
 */
var shortestPalindrome = function(s) {
  const reversed = s.split('').reverse().join('');
  for (let i = s.length; i > 0; i--) {
    if (s.slice(0, i) === reversed.slice(s.length - i)) {
      return reversed.slice(0, reversed.length - i) + s;
    }
  }
  return '';
};
```

0215 Kth Largest Element In An Array

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```javascript
/**
 * 215. Kth Largest Element in an Array
 * https://leetcode.com/problems/kth-largest-element-in-an-array/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return the kth largest element in the array.
 *
 * Note that it is the kth largest element in the sorted order, not the kth distinct element.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var findKthLargest = function(nums, k) {
  return nums.sort((a, b) => a - b)[nums.length - k];
};
```

0216 Combination Sum Iii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```javascript
/**
 * 216. Combination Sum III
 * https://leetcode.com/problems/combination-sum-iii/
 * Difficulty: Medium
 *
 * Find all valid combinations of k numbers that sum up to n such that the following
 * conditions are true:
 * - Only numbers 1 through 9 are used.
 * - Each number is used at most once.
 *
 * Return a list of all possible valid combinations. The list must not contain the same
 * combination twice, and the combinations may be returned in any order.
 */

/**
 * @param {number} k
 * @param {number} n
 * @return {number[][]}
 */
var combinationSum3 = function(k, n) {
  const result = [];

  function backtrack(history, sum, start) {
    if (sum > n) return;
```

```
    if (history.length === k && sum === n) {
      result.push(history);
      return;
    }
    for (let i = start; i < 10; i++) {
      backtrack([...history, i], sum + i, i + 1);
    }
  }

  backtrack([], 0, 1);

  return result;
};
```

0217 Contains Duplicate

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 217. Contains Duplicate
 * https://leetcode.com/problems/contains-duplicate/
 * Difficulty: Easy
 *
 * Given an integer array `nums`, return `true` if any value appears at least
 * twice in the array, and return `false` if every element is distinct.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var containsDuplicate = function(nums) {
  return new Set(nums).size !== nums.length;
};
```

0218 The Skyline Problem

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 218. The Skyline Problem
 * https://leetcode.com/problems/the-skyline-problem/
 * Difficulty: Hard
 *
 * A city's skyline is the outer contour of the silhouette formed by all the buildings in that
 * city when viewed from a distance. Given the locations and heights of all the buildings,
 * return the skyline formed by these buildings collectively.
 *
 * The geometric information of each building is given in the array buildings where
 * buildings[i] = [lefti, righti, heighti]:
 * - lefti is the x coordinate of the left edge of the ith building.
 * - righti is the x coordinate of the right edge of the ith building.
 * - heighti is the height of the ith building.
 *
 * You may assume all buildings are perfect rectangles grounded on an absolutely flat surface
 * at height 0.
 *
 * The skyline should be represented as a list of "key points" sorted by their x-coordinate
 * in the form [[x1,y1],[x2,y2],...]. Each key point is the left endpoint of some horizontal
 * segment in the skyline except the last point in the list, which always has a y-coordinate
 * 0 and is used to mark the skyline's termination where the rightmost building ends. Any
ground
 * between the leftmost and rightmost buildings should be part of the skyline's contour.
 *
 * Note: There must be no consecutive horizontal lines of equal height in the output skyline.
```

```
 * For instance, [...,[2 3],[4 5],[7 5],[11 5],[12 7],...] is not acceptable; the three lines
 * of height 5 should be merged into one in the final output as such: [...,[2 3],[4 5],[12
7],...]
 */

/**
 * @param {number[][]} buildings
 * @return {number[][]}
 */
var getSkyline = function(buildings) {
  const result = [];
  const events = buildings.flatMap(([l, r, h]) => [[l, -h], [r, h]]);

  events.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  for (let i = 0, previous = 0, heights = [0]; i < events.length; i++) {
    const [x, h] = events[i];
    if (h < 0) {
      heights.push(-h);
    } else {
      heights.splice(heights.indexOf(h), 1);
    }
    const value = Math.max(...heights);
    if (value !== previous) {
      result.push([x, previous = value]);
    }
  }

  return result;
};
```

0219 Contains Duplicate Ii

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 219. Contains Duplicate II
 * https://leetcode.com/problems/contains-duplicate-ii/
 * Difficulty: Easy
 *
 * Given an integer array `nums` and an integer `k`, return `true` if there are
 * two distinct indices `i` and `j` in the array such that `nums[i] == nums[j]`
 * and `abs(i - j) <= k`.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
var containsNearbyDuplicate = function(nums, k) {
  const map = new Map();

  for (let i = 0; i < nums.length; i++) {
    if (Math.abs(i - map.get(nums[i])) <= k) {
      return true;
    }
    map.set(nums[i], i);
  }

  return false;
};
```

0220 Contains Duplicate Iii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 220. Contains Duplicate III
 * https://leetcode.com/problems/contains-duplicate-iii/
 * Difficulty: Hard
 *
 * You are given an integer array nums and two integers indexDiff and valueDiff.
 *
 * Find a pair of indices (i, j) such that:
 * - i != j,
 * - abs(i - j) <= indexDiff.
 * - abs(nums[i] - nums[j]) <= valueDiff, and
 *
 * Return true if such pair exists or false otherwise.
 */

/**
 * @param {number[]} nums
 * @param {number} indexDiff
 * @param {number} valueDiff
 * @return {boolean}
 */
var containsNearbyAlmostDuplicate = function(nums, indexDiff, valueDiff) {
  if (valueDiff < 0) return false;

  const map = new Map();

  for (let i = 0; i < nums.length; i++) {
    const limit = valueDiff + 1;
    const item = Math.floor(nums[i] / limit);

    if (map.has(item)
      || map.has(item - 1) && Math.abs(nums[i] - map.get(item - 1)) < limit
      || map.has(item + 1) && Math.abs(nums[i] - map.get(item + 1)) < limit) {
      return true;
    }

    map.set(item, nums[i]);

    if (i >= indexDiff) {
      map.delete(Math.floor(nums[i - indexDiff] / limit));
    }
  }

  return false;
};
```

0221 Maximal Square

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 221. Maximal Square
 * https://leetcode.com/problems/maximal-square/
 * Difficulty: Medium
 *
 * Given an m x n binary matrix filled with 0's and 1's, find the largest square containing
only
 * 1's and return its area.
 */

/**
 * @param {character[][]} matrix
 * @return {number}
 */
var maximalSquare = function(matrix) {
  const dp = [...new Array(matrix.length + 1)].map(() => new Array(matrix[0].length +
1).fill(0));
```

```
  let max = 0;

  for (let i = 1; i < dp.length; i++) {
    for (let j = 1; j < dp[0].length; j++) {
      if (matrix[i - 1][j - 1] === '1') {
        max = Math.max(max, dp[i][j] = Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
+ 1);
      }
    }
  }

  return max ** 2;
};
```

0222 Count Complete Tree Nodes

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 222. Count Complete Tree Nodes
 * https://leetcode.com/problems/count-complete-tree-nodes/
 * Difficulty: Easy
 *
 * Given the root of a complete binary tree, return the number of the nodes in the tree.
 *
 * According to Wikipedia, every level, except possibly the last, is completely filled
 * in a complete binary tree, and all nodes in the last level are as far left as possible.
 * It can have between 1 and 2h nodes inclusive at the last level h.
 *
 * Design an algorithm that runs in less than O(n) time complexity.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var countNodes = function(root) {
  return !root ? 0 : 1 + countNodes(root.left) + countNodes(root.right);
};
```

0223 Rectangle Area

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 223. Rectangle Area
 * https://leetcode.com/problems/rectangle-area/
 * Difficulty: Medium
 *
 * Given the coordinates of two rectilinear rectangles in a 2D plane, return the total area
 * covered by the two rectangles.
 *
 * The first rectangle is defined by its bottom-left corner (ax1, ay1) and its top-right
corner
 * (ax2, ay2).
 *
 * The second rectangle is defined by its bottom-left corner (bx1, by1) and its top-right
```

```
corner
 * (bx2, by2).
 */

/**
 * @param {number} ax1
 * @param {number} ay1
 * @param {number} ax2
 * @param {number} ay2
 * @param {number} bx1
 * @param {number} by1
 * @param {number} bx2
 * @param {number} by2
 * @return {number}
 */
var computeArea = function(ax1, ay1, ax2, ay2, bx1, by1, bx2, by2) {
  return (ax2 - ax1) * (ay2 - ay1) + (bx2 - bx1) * (by2 - by1)
    - Math.max(0, Math.min(ax2, bx2) - Math.max(ax1, bx1))
    * Math.max(0, Math.min(ay2, by2) - Math.max(ay1, by1));
};
```

0224 Basic Calculator

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 224. Basic Calculator
 * https://leetcode.com/problems/basic-calculator/
 * Difficulty: Hard
 *
 * Given a string s representing a valid expression, implement a basic calculator to
 * evaluate it, and return the result of the evaluation.
 *
 * Note: You are not allowed to use any built-in function which evaluates strings as
 * mathematical expressions, such as eval().
 */

/**
 * @param {string} s
 * @return {number}
 */
var calculate = function(s) {
  const stack = [];
  let result = 0;

  for (let i = 0, sign = 1; i < s.length; i += 1) {
    if (s[i] >= '0' && s[i] <= '9') {
      let value = 0;
      while (s[i] >= '0' && s[i] <= '9') {
        value = (value * 10) + (s[i] - '0');
        i += 1;
      }
      result += value * sign;
      i -= 1;
    } else if (s[i] === '+') {
      sign = 1;
    } else if (s[i] === '-') {
      sign = -1;
    } else if (s[i] === '(') {
      stack.push(result, sign);
      result = 0;
      sign = 1;
    } else if (s[i] === ')') {
      result = stack.pop() * result;
      result += stack.pop();
    }
  }

  return result;
};
```

0225 Implement Stack Using Queues

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 225. Implement Stack using Queues
 * https://leetcode.com/problems/implement-stack-using-queues/
 * Difficulty: Easy
 *
 * Implement a last-in-first-out (LIFO) stack using only two queues.
 * The implemented stack should support all the functions of a normal
 * stack (push, top, pop, and empty).
 *
 * Implement the MyStack class:
 *
 * - void push(int x) Pushes element x to the top of the stack.
 * - int pop() Removes the element on the top of the stack and returns it.
 * - int top() Returns the element on the top of the stack.
 * - boolean empty() Returns true if the stack is empty, false otherwise.
 */


var MyStack = function() {
  this.data = [];
};

/**
 * @param {number} x
 * @return {void}
 */
MyStack.prototype.push = function(x) {
  this.data.push(x);
};

/**
 * @return {number}
 */
MyStack.prototype.pop = function() {
  return this.data.pop();
};

/**
 * @return {number}
 */
MyStack.prototype.top = function() {
  return this.data[this.data.length - 1];
};

/**
 * @return {boolean}
 */
MyStack.prototype.empty = function() {
  return !this.data.length;
};

/**
 * Your MyStack object will be instantiated and called as such:
 * var obj = new MyStack()
 * obj.push(x)
 * var param_2 = obj.pop()
 * var param_3 = obj.top()
 * var param_4 = obj.empty()
 */
```

0226 Invert Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 226. Invert Binary Tree
 * https://leetcode.com/problems/invert-binary-tree/
 * Difficulty: Easy
 *
 * Invert a binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} node
 * @return {TreeNode}
 */
var invertTree = function(node) {
  if (node) [node.left, node.right] = [invertTree(node.right), invertTree(node.left)];
  return node;
};
```

0227 Basic Calculator Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 227. Basic Calculator II
 * https://leetcode.com/problems/basic-calculator-ii/
 * Difficulty: Medium
 *
 * Given a string s which represents an expression, evaluate this expression and return its
value.
 *
 * The integer division should truncate toward zero.
 *
 * You may assume that the given expression is always valid. All intermediate results will be
in
 * the range of [-231, 231 - 1].
 *
 * Note: You are not allowed to use any built-in function which evaluates strings as
mathematical
 * expressions, such as eval().
 */

/**
 * @param {string} s
 * @return {number}
 */
var calculate = function(s) {
  const stack = [];

  for (let i = 0, num = 0, sign = '+'; i < s.length; i++) {
    if (s[i] >= '0' && s[i] <= '9') num = num * 10 + (s[i] - '0');
    if ((s[i] < '0' && s[i] !== ' ') || i === s.length - 1) {
      if (sign === '+') stack.push(num);
      else if (sign === '-') stack.push(-num);
      else if (sign === '*') stack.push(stack.pop() * num);
      else if (sign === '/') stack.push(Math.trunc(stack.pop() / num));
```

```
      sign = s[i];
      num = 0;
    }
  }

  return stack.reduce((a, b) => a + b, 0);
};
```

0228 Summary Ranges

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 228. Summary Ranges
 * https://leetcode.com/problems/summary-ranges/
 * Difficulty: Easy
 *
 * You are given a sorted unique integer array nums.
 *
 * A range [a,b] is the set of all integers from a to b (inclusive).
 *
 * Return the smallest sorted list of ranges that cover all the numbers in the
 * array exactly. That is, each element of nums is covered by exactly one of
 * the ranges, and there is no integer x such that x is in one of the ranges
 * but not in nums.
 *
 * Each range [a,b] in the list should be output as:
 * - "a->b" if a != b
 * - "a" if a == b
 */

/**
 * @param {number[]} nums
 * @return {string[]}
 */
var summaryRanges = function(nums) {
  const result = [];
  for (let i = 0, n = nums[0]; i < nums.length; i++) {
    if (nums[i] + 1 !== nums[i + 1]) {
      result.push(nums[i] === n ? `${n}` : `${n}->${nums[i]}`);
      n = nums[i + 1];
    }
  }
  return result;
};
```

0229 Majority Element Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 229. Majority Element II
 * https://leetcode.com/problems/majority-element-ii/
 * Difficulty: Medium
 *
 * Given an integer array of size n, find all elements that appear more than ■ n/3 ■ times.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var majorityElement = function(nums) {
  const map = new Map();
  nums.forEach(n => map.set(n, (map.get(n) || 0) + 1));
```

```
  return [...map]
    .filter(([key, value]) => value > nums.length / 3)
    .map(([key]) => key);
};
```

0230 Kth Smallest Element In A Bst

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 230. Kth Smallest Element in a BST
 * https://leetcode.com/problems/kth-smallest-element-in-a-bst/
 * Difficulty: Medium
 *
 * Given the root of a binary search tree, and an integer k, return the kth smallest value
 * (1-indexed) of all the values of the nodes in the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} k
 * @return {number}
 */
var kthSmallest = function(root, k) {
  const result = [];
  dfs(root);
  return result[k - 1];

  function dfs(node) {
    if (!node || result.length > k) return null;
    dfs(node.left);
    result.push(node.val);
    dfs(node.right);
  }
};
```

0231 Power Of Two

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 231. Power of Two
 * https://leetcode.com/problems/power-of-two/
 * Difficulty: Easy
 *
 * Given an integer n, return true if it is a power of two. Otherwise, return false.
 *
 * An integer n is a power of two, if there exists an integer x such that n == 2x.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isPowerOfTwo = function(n) {
  if (n > 1) {
```

```
    while (n % 2 === 0) {
      n /= 2
    }
  }
  return n === 1;
};
```

0232 Implement Queue Using Stacks

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 232. Implement Queue using Stacks
 * https://leetcode.com/problems/implement-queue-using-stacks/
 * Difficulty: Easy
 *
 * Implement a first in first out (FIFO) queue using only two stacks.
 * The implemented queue should support all the functions of a normal
 * queue (push, peek, pop, and empty).
 *
 * Implement the MyQueue class:
 * - void push(int x) Pushes element x to the back of the queue.
 * - int pop() Removes the element from the front of the queue and returns it.
 * - int peek() Returns the element at the front of the queue.
 * - boolean empty() Returns true if the queue is empty, false otherwise.
 */


var MyQueue = function() {
  this.data = [];
};

/**
 * @param {number} x
 * @return {void}
 */
MyQueue.prototype.push = function(x) {
  this.data.push(x);
};

/**
 * @return {number}
 */
MyQueue.prototype.pop = function() {
  return this.data.shift();
};

/**
 * @return {number}
 */
MyQueue.prototype.peek = function() {
  return this.data[0];
};

/**
 * @return {boolean}
 */
MyQueue.prototype.empty = function() {
  return !this.data.length;
};

/**
 * Your MyQueue object will be instantiated and called as such:
 * var obj = new MyQueue()
 * obj.push(x)
 * var param_2 = obj.pop()
 * var param_3 = obj.peek()
 * var param_4 = obj.empty()
 */
```

0233 Number Of Digit One

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 233. Number of Digit One
 * https://leetcode.com/problems/number-of-digit-one/
 * Difficulty: Hard
 *
 * Given an integer n, count the total number of digit 1 appearing in all non-negative
 * integers less than or equal to n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countDigitOne = function(n) {
  if (n <= 0) {
    return 0;
  } else if (n < 10) {
    return 1;
  }

  const base = 10 ** (n.toString().length - 1);
  const answer = parseInt(n / base);

  return countDigitOne(base - 1) * answer
    + (answer === 1 ? (n - base + 1) : base) + countDigitOne(n % base);
};
```

0234 Palindrome Linked List

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 234. Palindrome Linked List
 * https://leetcode.com/problems/palindrome-linked-list/
 * Difficulty: Easy
 *
 * Given the `head` of a singly linked list, return `true` if it is a palindrome.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {boolean}
 */
var isPalindrome = function(head) {
  let a = '', b = '';

  while (head) {
    a = a + head.val;
    b = head.val + b;
    head = head.next;
  }

  return a === b;
};
```

0235 Lowest Common Ancestor Of A Binary Search Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 235. Lowest Common Ancestor of a Binary Search Tree
 * https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/
 * Difficulty: Easy
 *
 * Given a binary search tree (BST), find the lowest common ancestor (LCA) of two
 * given nodes in the BST.
 *
 * According to the definition of LCA on Wikipedia: “The lowest common ancestor is
 * defined between two nodes p and q as the lowest node in T that has both p and q
 * as descendants (where we allow a node to be a descendant of itself).”
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */

/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, p, q) {
  if (p.val > root.val && q.val > root.val) {
    return lowestCommonAncestor(root.right, p, q);
  }
  if (p.val < root.val && q.val < root.val) {
    return lowestCommonAncestor(root.left, p , q);
  }
  return root;
};
```

0236 Lowest Common Ancestor Of A Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 236. Lowest Common Ancestor of a Binary Tree
 * https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/
 * Difficulty: Medium
 *
 * Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.
 *
 * According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined
 * between two nodes p and q as the lowest node in T that has both p and q as descendants
 * (where we allow a node to be a descendant of itself).”
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
```

```
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, p, q) {
  if (!root || root === p || root === q) return root;
  const [l, r] = [lowestCommonAncestor(root.left, p, q), lowestCommonAncestor(root.right, p,
q)];
  return l && r ? root : l ?? r;
};
```

0237 Delete Node In A Linked List

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 237. Delete Node in a Linked List
 * https://leetcode.com/problems/delete-node-in-a-linked-list/
 * Difficulty: Easy
 *
 * Write a function to delete a node in a singly-linked list. You will not be
 * given access to the head of the list, instead you will be given access to
 * the node to be deleted directly.
 *
 * It is guaranteed that the node to be deleted is not a tail node in the list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */
/**
 * @param {ListNode} node
 * @return {void} Do not return anything, modify node in-place instead.
 */
var deleteNode = function(node) {
  node.val = node.next.val;
  node.next = node.next.next;
};
```

0238 Product Of Array Except Self

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 238. Product of Array Except Self
 * https://leetcode.com/problems/product-of-array-except-self/
 * Difficulty: Medium
 *
 * Given an integer array nums, return an array answer such that answer[i] is equal to the
product
 * of all the elements of nums except nums[i].
 *
 * The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit integer.
 *
 * You must write an algorithm that runs in O(n) time and without using the division
operation.
 */

/**
 * @param {number[]} nums
```

```
 * @return {number[]}
 */
var productExceptSelf = function(nums) {
  const emptyResult = new Array(nums.length).fill(0);
  const zeroCount = nums.filter(n => n === 0).length;
  if (zeroCount > 1) {
    return emptyResult;
  }
  const product = nums.reduce((product, n) => product * (n === 0 ? 1 : n), 1);
  if (zeroCount === 1) {
    emptyResult[nums.indexOf(0)] = product;
    return emptyResult;
  }
  return nums.map(n => product / n);
};
```

0239 Sliding Window Maximum

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 239. Sliding Window Maximum
 * https://leetcode.com/problems/sliding-window-maximum/
 * Difficulty: Hard
 *
 * You are given an array of integers nums, there is a sliding window of size k which is
moving
 * from the very left of the array to the very right. You can only see the k numbers in the
 * window. Each time the sliding window moves right by one position.
 *
 * Return the max sliding window.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var maxSlidingWindow = function(nums, k) {
  const result = [];
  const queue = [];

  for (let i = 0; i < nums.length; i++) {
    while (queue.length && queue[0] < i - k + 1) {
      queue.shift();
    }
    while (queue.length && nums[queue[queue.length - 1]] < nums[i]) {
      queue.pop();
    }
    queue.push(i);
    if (i >= k - 1) {
      result.push(nums[queue[0]]);
    }
  }

  return result;
};
```

0240 Search A 2D Matrix Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 240. Search a 2D Matrix II
 * https://leetcode.com/problems/search-a-2d-matrix-ii/
```

```
 * Difficulty: Medium
 *
 * Write an efficient algorithm that searches for a value target in an m x n integer matrix
 * matrix. This matrix has the following properties:
 * - Integers in each row are sorted in ascending from left to right.
 * - Integers in each column are sorted in ascending from top to bottom.
 */

/**
 * @param {number[][]} matrix
 * @param {number} target
 * @return {boolean}
 */
var searchMatrix = function(matrix, target) {
  for (let i = 0, j = matrix[0].length - 1; i < matrix.length && j >= 0;) {
    if (matrix[i][j] === target) {
      return true;
    }
    matrix[i][j] > target ? j-- : i++;
  }
  return false;
};
```

0241 Different Ways To Add Parentheses

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 241. Different Ways to Add Parentheses
 * https://leetcode.com/problems/different-ways-to-add-parentheses/
 * Difficulty: Medium
 *
 * Given a string expression of numbers and operators, return all possible results from
 * computing all the different possible ways to group numbers and operators. You may
 * return the answer in any order.
 *
 * The test cases are generated such that the output values fit in a 32-bit integer and
 * the number of different results does not exceed 104.
 */

/**
 * @param {string} expression
 * @return {number[]}
 */
var diffWaysToCompute = function(expression) {
  const results = [];

  for (let i = 0; i < expression.length; i++) {
    if ('+-*'.includes(expression[i])) {
      const left = diffWaysToCompute(expression.slice(0, i));
      const right = diffWaysToCompute(expression.slice(i + 1));
      left.forEach(l => right.forEach(r =>
        results.push(expression[i] === '+' ? l + r : expression[i] === '-' ? l - r : l * r)));
    }
  }

  return results.length ? results : [+expression];
};
```

0242 Valid Anagram

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 242. Valid Anagram
```

```
 * https://leetcode.com/problems/valid-anagram/
 * Difficulty: Easy
 *
 * Given two strings s and t, return true if t is an anagram of s, and false otherwise.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isAnagram = function(s, t) {
  const sort = str => str.split('').sort().join('');
  return sort(s) === sort(t);
};
```

0243 Shortest Word Distance

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 243. Shortest Word Distance
 * https://leetcode.com/problems/shortest-word-distance/
 * Difficulty: Easy
 *
 * Given an array of strings wordsDict and two different strings that already exist in the
array
 * word1 and word2, return the shortest distance between these two words in the list.
 */

/**
 * @param {string[]} wordsDict
 * @param {string} word1
 * @param {string} word2
 * @return {number}
 */
var shortestDistance = function(wordsDict, word1, word2) {
  let result = Infinity;
  let index1 = -1;
  let index2 = -1;

  for (let i = 0; i < wordsDict.length; i++) {
    if (wordsDict[i] === word1) {
      index1 = i;
      if (index2 !== -1) {
        result = Math.min(result, index1 - index2);
      }
    } else if (wordsDict[i] === word2) {
      index2 = i;
      if (index1 !== -1) {
        result = Math.min(result, index2 - index1);
      }
    }
  }

  return result;
};
```

0244 Shortest Word Distance Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 244. Shortest Word Distance II
```

```
 * https://leetcode.com/problems/shortest-word-distance-ii/
 * Difficulty: Medium
 *
 * Design a data structure that will be initialized with a string array, and then it should
 * answer queries of the shortest distance between two different strings from the array.
 *
 * Implement the WordDistance class:
 * - WordDistance(String[] wordsDict) initializes the object with the strings array wordsDict.
 * - int shortest(String word1, String word2) returns the shortest distance between word1 and
 *   word2 in the array wordsDict.
 */

/**
 * @param {string[]} wordsDict
 */
var WordDistance = function(wordsDict) {
  this.wordIndices = new Map();
  for (let i = 0; i < wordsDict.length; i++) {
    if (!this.wordIndices.has(wordsDict[i])) {
      this.wordIndices.set(wordsDict[i], []);
    }
    this.wordIndices.get(wordsDict[i]).push(i);
  }
};

/**
 * @param {string} word1
 * @param {string} word2
 * @return {number}
 */
WordDistance.prototype.shortest = function(word1, word2) {
  const indices1 = this.wordIndices.get(word1);
  const indices2 = this.wordIndices.get(word2);
  let minDistance = Infinity;
  let i = 0;
  let j = 0;

  while (i < indices1.length && j < indices2.length) {
    minDistance = Math.min(minDistance, Math.abs(indices1[i] - indices2[j]));
    if (indices1[i] < indices2[j]) {
      i++;
    } else {
      j++;
    }
  }

  return minDistance;
};
```

0245 Shortest Word Distance Iii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 245. Shortest Word Distance III
 * https://leetcode.com/problems/shortest-word-distance-iii/
 * Difficulty: Medium
 *
 * Given an array of strings wordsDict and two strings that already exist in the array word1
 * and word2, return the shortest distance between the occurrence of these two words in the
list.
 *
 * Note that word1 and word2 may be the same. It is guaranteed that they represent two
individual
 * words in the list.
 */

/**
 * @param {string[]} wordsDict
 * @param {string} word1
```

```
 * @param {string} word2
 * @return {number}
 */
var shortestWordDistance = function(wordsDict, word1, word2) {
  let result = Infinity;
  let lastWord1Index = -1;
  let lastWord2Index = -1;
  const areSameWords = word1 === word2;

  for (let index = 0; index < wordsDict.length; index++) {
    const currentWord = wordsDict[index];

    if (currentWord === word1) {
      if (areSameWords) {
        if (lastWord1Index !== -1) {
          result = Math.min(result, index - lastWord1Index);
        }
      } else if (lastWord2Index !== -1) {
        result = Math.min(result, index - lastWord2Index);
      }
      lastWord1Index = index;
    } else if (!areSameWords && currentWord === word2) {
      if (lastWord1Index !== -1) {
        result = Math.min(result, index - lastWord1Index);
      }
      lastWord2Index = index;
    }
  }

  return result;
};
```

0246 Strobogrammatic Number

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 246. Strobogrammatic Number
 * https://leetcode.com/problems/strobogrammatic-number/
 * Difficulty: Easy
 *
 * Given a string num which represents an integer, return true if num is a strobogrammatic
number.
 *
 * A strobogrammatic number is a number that looks the same when rotated 180 degrees (looked
at
 * upside down).
 */

/**
 * @param {string} num
 * @return {boolean}
 */
var isStrobogrammatic = function(num) {
  const validPairs = new Map([
    ['0', '0'],
    ['1', '1'],
    ['6', '9'],
    ['8', '8'],
    ['9', '6']
  ]);
  let left = 0;
  let right = num.length - 1;

  while (left <= right) {
    const leftDigit = num[left];
    const rightDigit = num[right];

    if (!validPairs.has(leftDigit) || validPairs.get(leftDigit) !== rightDigit) {
      return false;
```

```
    }

    left++;
    right--;
  }

  return true;
};
```

0247 Strobogrammatic Number Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 247. Strobogrammatic Number II
 * https://leetcode.com/problems/strobogrammatic-number-ii/
 * Difficulty: Medium
 *
 * Given an integer n, return all the strobogrammatic numbers that are of length n. You may
 * return the answer in any order.
 *
 * A strobogrammatic number is a number that looks the same when rotated 180 degrees (looked
 * at upside down).
 */

/**
 * @param {number} n
 * @return {string[]}
 */
var findStrobogrammatic = function(n) {
  const pairs = [
    ['0', '0'],
    ['1', '1'],
    ['6', '9'],
    ['8', '8'],
    ['9', '6']
  ];

  return generateStrobogrammatic(n, n);

  function generateStrobogrammatic(length, finalLength) {
    if (length === 0) return [''];
    if (length === 1) return ['0', '1', '8'];

    const result = [];
    const subNumbers = generateStrobogrammatic(length - 2, finalLength);
    for (const [left, right] of pairs) {
      for (const sub of subNumbers) {
        if (length === finalLength && left === '0') continue;
        result.push(left + sub + right);
      }
    }

    return result;
  }
};
```

0248 Strobogrammatic Number Iii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 248. Strobogrammatic Number III
 * https://leetcode.com/problems/strobogrammatic-number-iii/
 * Difficulty: Hard
```

```
 *
 * Given two strings low and high that represent two integers low and high where low <= high,
 * return the number of strobogrammatic numbers in the range [low, high].
 *
 * A strobogrammatic number is a number that looks the same when rotated 180 degrees (looked
 * at upside down).
 */

/**
 * @param {string} low
 * @param {string} high
 * @return {number}
 */
var strobogrammaticInRange = function(low, high) {
  const pairs = [
    ['0', '0'],
    ['1', '1'],
    ['6', '9'],
    ['8', '8'],
    ['9', '6']
  ];

  function generateStrobogrammatic(length, isOuter) {
    if (length === 0) return [''];
    if (length === 1) return ['0', '1', '8'];

    const result = [];
    const subNumbers = generateStrobogrammatic(length - 2, false);

    for (const [left, right] of pairs) {
      for (const sub of subNumbers) {
        if (isOuter && left === '0') continue;
        result.push(left + sub + right);
      }
    }

    return result;
  }

  function countInRange(num, lowVal, highVal) {
    if (num.length < lowVal.length || num.length > highVal.length) return false;
    if (num.length === lowVal.length && num < lowVal) return false;
    if (num.length === highVal.length && num > highVal) return false;
    return true;
  }

  let count = 0;
  const lowLength = low.length;
  const highLength = high.length;

  for (let len = lowLength; len <= highLength; len++) {
    const numbers = generateStrobogrammatic(len, true);
    for (const num of numbers) {
      if (countInRange(num, low, high)) {
        count++;
      }
    }
  }

  return count;
};
```

0249 Group Shifted Strings

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 249. Group Shifted Strings
 * https://leetcode.com/problems/group-shifted-strings/
 * Difficulty: Medium
```

```
 *
 * Perform the following shift operations on a string:
 * - Right shift: Replace every letter with the successive letter of the English alphabet,
where
 *   'z' is replaced by 'a'. For example, "abc" can be right-shifted to "bcd" or "xyz" can be
 *   right-shifted to "yza".
 * - Left shift: Replace every letter with the preceding letter of the English alphabet, where
 *   'a' is replaced by 'z'. For example, "bcd" can be left-shifted to "abc" or "yza" can be
 *   left-shifted to "xyz".
 *
 * We can keep shifting the string in both directions to form an endless shifting sequence.
 * - For example, shift "abc" to form the sequence: ... <-> "abc" <-> "bcd" <-> ... <-> "xyz"
 *   <-> "yza" <-> .... <-> "zab" <-> "abc" <-> ...
 *
 * You are given an array of strings strings, group together all strings[i] that belong to the
 same
 * shifting sequence. You may return the answer in any order.
 */

/**
 * @param {string[]} strings
 * @return {string[][]}
 */
var groupStrings = function(strings) {
  const groups = new Map();

  for (const str of strings) {
    const key = getShiftKey(str);
    if (!groups.has(key)) {
      groups.set(key, []);
    }
    groups.get(key).push(str);
  }

  return Array.from(groups.values());

  function getShiftKey(str) {
    if (str.length === 1) return 'single';
    const key = [];
    for (let i = 1; i < str.length; i++) {
      let diff = str.charCodeAt(i) - str.charCodeAt(i - 1);
      if (diff < 0) diff += 26;
      key.push(diff);
    }
    return key.join(',');
  }
};
```

0250 Count Univalue Subtrees

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 250. Count Univalue Subtrees
 * https://leetcode.com/problems/count-univalue-subtrees/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the number of uni-value subtrees.
 *
 * A uni-value subtree means all nodes of the subtree have the same value.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
```

```
/**
 * @param {TreeNode} root
 * @return {number}
 */
var countUnivalSubtrees = function(root) {
  let count = 0;
  traverse(root);
  return count;

  function isUnival(node, value) {
    if (!node) return true;

    if (node.val !== value) return false;

    return isUnival(node.left, value) && isUnival(node.right, value);
  }

  function traverse(node) {
    if (!node) return;

    if (isUnival(node, node.val)) count++;

    traverse(node.left);
    traverse(node.right);
  }
};
```

0251 Flatten 2D Vector

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 251. Flatten 2D Vector
 * https://leetcode.com/problems/flatten-2d-vector/
 * Difficulty: Medium
 *
 * Design an iterator to flatten a 2D vector. It should support the next and hasNext
operations.
 *
 * Implement the Vector2D class:
 * - Vector2D(int[][] vec) initializes the object with the 2D vector vec.
 * - next() returns the next element from the 2D vector and moves the pointer one step
forward.
 *   You may assume that all the calls to next are valid.
 * - hasNext() returns true if there are still some elements in the vector, and false
otherwise.
 */

/**
 * @param {number[][]} vec
 */
var Vector2D = function(vec) {
  this.vector = vec;
  this.row = 0;
  this.col = 0;
};

/**
 * @return {number}
 */
Vector2D.prototype.next = function() {
  while (this.row < this.vector.length && this.col >= this.vector[this.row].length) {
    this.row++;
    this.col = 0;
  }
  return this.vector[this.row][this.col++];
};

/**
 * @return {boolean}
```

```
 */
Vector2D.prototype.hasNext = function() {
  while (this.row < this.vector.length && this.col >= this.vector[this.row].length) {
    this.row++;
    this.col = 0;
  }
  return this.row < this.vector.length;
};
```

0252 Meeting Rooms

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 252. Meeting Rooms
 * https://leetcode.com/problems/meeting-rooms/
 * Difficulty: Easy
 *
 * Given an array of meeting time intervals where intervals[i] = [starti, endi], determine
 * if a person could attend all meetings.
 */

/**
 * @param {number[][]} intervals
 * @return {boolean}
 */
var canAttendMeetings = function(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);

  for (let i = 1; i < intervals.length; i++) {
    if (intervals[i][0] < intervals[i - 1][1]) {
      return false;
    }
  }

  return true;
};
```

0253 Meeting Rooms Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 253. Meeting Rooms II
 * https://leetcode.com/problems/meeting-rooms-ii/
 * Difficulty: Medium
 *
 * Given an array of meeting time intervals intervals where intervals[i] = [starti, endi],
 * return the minimum number of conference rooms required.
 */

/**
 * @param {number[][]} intervals
 * @return {number}
 */
var minMeetingRooms = function(intervals) {
  const starts = intervals.map(([start]) => start).sort((a, b) => a - b);
  const ends = intervals.map(([, end]) => end).sort((a, b) => a - b);
  let rooms = 0;
  let startIndex = 0;
  let endIndex = 0;
  let result = 0;

  while (startIndex < intervals.length) {
    if (starts[startIndex] < ends[endIndex]) {
```

```
      rooms++;
      result = Math.max(result, rooms);
      startIndex++;
    } else {
      rooms--;
      endIndex++;
    }
  }

  return result;
};
```

0254 Factor Combinations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 254. Factor Combinations
 * https://leetcode.com/problems/factor-combinations/
 * Difficulty: Medium
 *
 * Numbers can be regarded as the product of their factors.
 *
 * For example, 8 = 2 x 2 x 2 = 2 x 4.
 *
 * Given an integer n, return all possible combinations of its factors. You may return the
 * answer in any order.
 *
 * Note that the factors should be in the range [2, n - 1].
 */

/**
 * @param {number} n
 * @return {number[][]}
 */
var getFactors = function(n) {
  const result = [];
  findFactors(n, 2, []);
  return result;

  function findFactors(currentNum, start, combination) {
    for (let i = start; i * i <= currentNum; i++) {
      if (currentNum % i === 0) {
        const nextNum = currentNum / i;
        if (nextNum >= i && nextNum < currentNum) {
          result.push([...combination, i, nextNum]);
          findFactors(nextNum, i, [...combination, i]);
        }
      }
    }
  }
};
```

0255 Verify Preorder Sequence In Binary Search Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 255. Verify Preorder Sequence in Binary Search Tree
 * https://leetcode.com/problems/verify-preorder-sequence-in-binary-search-tree/
 * Difficulty: Medium
 *
 * Given an array of unique integers preorder, return true if it is the correct preorder
 * traversal sequence of a binary search tree.
 */
```

```
/**
 * @param {number[]} preorder
 * @return {boolean}
 */
var verifyPreorder = function(preorder) {
  let minLimit = -Infinity;
  const stack = [];

  for (const value of preorder) {
    while (stack.length && stack[stack.length - 1] < value) {
      minLimit = stack.pop();
    }
    if (value <= minLimit) return false;
    stack.push(value);
  }

  return true;
};
```

0256 Paint House

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 256. Paint House
 * https://leetcode.com/problems/paint-house/
 * Difficulty: Medium
 *
 * There is a row of n houses, where each house can be painted one of three colors: red, blue,
 * or green. The cost of painting each house with a certain color is different. You have to
 * paint all the houses such that no two adjacent houses have the same color.
 *
 * The cost of painting each house with a certain color is represented by an n x 3 cost matrix
 * costs.
 * - For example, costs[0][0] is the cost of painting house 0 with the color red; costs[1][2]
 *   is the cost of painting house 1 with color green, and so on...
 *
 * Return the minimum cost to paint all houses.
 */

/**
 * @param {number[][]} costs
 * @return {number}
 */
var minCost = function(costs) {
  let prev = [...costs[0]];

  for (let i = 1; i < costs.length; i++) {
    prev = [
      costs[i][0] + Math.min(prev[1], prev[2]),
      costs[i][1] + Math.min(prev[0], prev[2]),
      costs[i][2] + Math.min(prev[0], prev[1])
    ];
  }

  return Math.min(...prev);
};
```

0257 Binary Tree Paths

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 257. Binary Tree Paths
```

```
 * https://leetcode.com/problems/binary-tree-paths/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return all root-to-leaf paths in any order.
 *
 * A leaf is a node with no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {string[]}
 */
var binaryTreePaths = function(root) {
  return tranverse(root);
};

function tranverse(node, path = [], result = []) {
  if (!node) return;
  path.push(node.val.toString());
  if (!node.left && !node.right) {
    result.push(path.join('->'));
  } else {
    tranverse(node.left, path.slice(), result);
    tranverse(node.right, path.slice(), result);
  }
  return result;
}
```

0258 Add Digits

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 258. Add Digits
 * https://leetcode.com/problems/add-digits/
 * Difficulty: Easy
 *
 * Given an integer num, repeatedly add all its digits until
 * the result has only one digit, and return it.
 */

/**
 * @param {number} num
 * @return {number}
 */
var addDigits = function(num) {
  return num < 10 ? num : num % 9 === 0 ? 9 : num % 9;
};
```

0259 3Sum Smaller

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 259. 3Sum Smaller
 * https://leetcode.com/problems/3sum-smaller/
```

```
 * Difficulty: Medium
 *
 * Given an array of n integers nums and an integer target, find the number of index triplets
 * i, j, k with 0 <= i < j < k < n that satisfy the condition nums[i] + nums[j] + nums[k] <
target.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var threeSumSmaller = function(nums, target) {
  nums.sort((a, b) => a - b);
  const n = nums.length;
  let result = 0;

  for (let i = 0; i < n - 2; i++) {
    let left = i + 1;
    let right = n - 1;

    while (left < right) {
      const sum = nums[i] + nums[left] + nums[right];
      if (sum < target) {
        result += right - left;
        left++;
      } else {
        right--;
      }
    }
  }

  return result;
};
```

0260 Single Number Iii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 260. Single Number III
 * https://leetcode.com/problems/single-number-iii/
 * Difficulty: Medium
 *
 * Given an integer array nums, in which exactly two elements appear only once and all
 * the other elements appear exactly twice. Find the two elements that appear only once.
 * You can return the answer in any order.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var singleNumber = function(nums) {
  const set = new Set();
  nums.forEach(n => set.has(n) ? set.delete(n) : set.add(n));
  return Array.from(set);
};
```

0261 Graph Valid Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 261. Graph Valid Tree
```

```
 * https://leetcode.com/problems/graph-valid-tree/
 * Difficulty: Medium
 *
 * You have a graph of n nodes labeled from 0 to n - 1. You are given an integer n and a list
of
 * edges where edges[i] = [ai, bi] indicates that there is an undirected edge between nodes ai
 * and bi in the graph.
 *
 * Return true if the edges of the given graph make up a valid tree, and false otherwise.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {boolean}
 */
var validTree = function(n, edges) {
  const parent = new Array(n).fill(-1);

  for (const [u, v] of edges) {
    if (!union(u, v)) return false;
  }

  let components = 0;
  for (let i = 0; i < n; i++) {
    if (parent[i] === -1) components++;
    if (components > 1) return false;
  }

  return edges.length === n - 1;

  function find(x) {
    if (parent[x] === -1) return x;
    return parent[x] = find(parent[x]);
  }

  function union(x, y) {
    const rootX = find(x);
    const rootY = find(y);
    if (rootX === rootY) return false;
    parent[rootX] = rootY;
    return true;
  }
};
```

0263 Ugly Number

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 263. Ugly Number
 * https://leetcode.com/problems/ugly-number/
 * Difficulty: Easy
 *
 * Write a program to check whether a given number is an ugly number.
 *
 * Ugly numbers are positive numbers whose prime factors only include 2, 3, 5.
 */

/**
 * @param {number} num
 * @return {boolean}
 */
var isUgly = function(num) {
  for (let i of [2, 3, 5]) {
    while (num && num % i === 0) num /= i;
  }
  return num === 1;
};
```

0264 Ugly Number Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 264. Ugly Number II
 * https://leetcode.com/problems/ugly-number-ii/
 * Difficulty: Medium
 *
 * Write a program to find the n-th ugly number.
 *
 * Ugly numbers are positive numbers whose prime factors only include 2, 3, 5.
 */

/**
 * @param {number} n
 * @return {number}
 */
var nthUglyNumber = function(n, factors = [2, 3, 5], offsets = [0, 0, 0]) {
  const numbers = [1];
  while (numbers.length < n) {
    const candidates = factors.map((v, i) => numbers[offsets[i]] * v);
    const target = Math.min(...candidates);
    candidates.forEach((c, i) => target === c ? offsets[i]++ : null);
    numbers.push(target)
  }
  return numbers[n - 1];
};
```

0265 Paint House Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 265. Paint House II
 * https://leetcode.com/problems/paint-house-ii/
 * Difficulty: Hard
 *
 * There are a row of n houses, each house can be painted with one of the k colors. The cost
of
 * painting each house with a certain color is different. You have to paint all the houses
such
 * that no two adjacent houses have the same color.
 *
 * The cost of painting each house with a certain color is represented by an n x k cost matrix
 * costs.
 * - For example, costs[0][0] is the cost of painting house 0 with color 0; costs[1][2] is the
 *   cost of painting house 1 with color 2, and so on...
 *
 * Return the minimum cost to paint all houses.
 */

/**
 * @param {number[][]} costs
 * @return {number}
 */
var minCostII = function(costs) {
  const n = costs.length;
  const k = costs[0].length;
  let prev = [...costs[0]];

  for (let i = 1; i < n; i++) {
    const curr = new Array(k);
    let min1 = Infinity;
    let min2 = Infinity;
```

```
    let index = -1;

    for (let j = 0; j < k; j++) {
      if (prev[j] < min1) {
        min2 = min1;
        min1 = prev[j];
        index = j;
      } else if (prev[j] < min2) {
        min2 = prev[j];
      }
    }

    for (let j = 0; j < k; j++) {
      curr[j] = costs[i][j] + (j === index ? min2 : min1);
    }

    prev = curr;
  }

  return Math.min(...prev);
};
```

0266 Palindrome Permutation

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 266. Palindrome Permutation
 * https://leetcode.com/problems/palindrome-permutation/
 * Difficulty: Easy
 *
 * Given a string s, return true if a permutation of the string could form a palindrome
 * and false otherwise.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var canPermutePalindrome = function(s) {
  const map = new Map();

  for (const char of s) {
    map.set(char, (map.get(char) || 0) + 1);
  }

  let oddCount = 0;
  for (const count of map.values()) {
    if (count % 2 !== 0) oddCount++;
    if (oddCount > 1) return false;
  }

  return true;
};
```

0267 Palindrome Permutation Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 267. Palindrome Permutation II
 * https://leetcode.com/problems/palindrome-permutation-ii/
 * Difficulty: Medium
 *
 * Given a string s, return all the palindromic permutations (without duplicates) of it.
```

```
 *
 * You may return the answer in any order. If s has no palindromic permutation, return an
 * empty list.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var generatePalindromes = function(s) {
  const map = new Map();
  for (const char of s) {
    map.set(char, (map.get(char) || 0) + 1);
  }

  let oddChar = '';
  const half = [];
  let oddCount = 0;

  for (const [char, count] of map) {
    if (count % 2) {
      oddCount++;
      oddChar = char;
    }
    for (let i = 0; i < Math.floor(count / 2); i++) {
      half.push(char);
    }
  }

  if (oddCount > 1) return [];

  const result = new Set();

  half.sort();
  permute(half, [], new Array(half.length).fill(false));
  return Array.from(result);

  function permute(chars, current, used) {
    if (current.length === chars.length) {
      const palindrome = current.join('') + oddChar + current.reverse().join('');
      result.add(palindrome);
      current.reverse();
      return;
    }

    for (let i = 0; i < chars.length; i++) {
      if (!used[i] && (i === 0 || chars[i] !== chars[i - 1] || used[i - 1])) {
        used[i] = true;
        current.push(chars[i]);
        permute(chars, current, used);
        current.pop();
        used[i] = false;
      }
    }
  }
};
```

0268 Missing Number

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 268. Missing Number
 * https://leetcode.com/problems/missing-number/
 * Difficulty: Easy
 *
 * Given an array `nums` containing `n` distinct numbers in the range `[0, n]`,
 * return the only number in the range that is missing from the array.
 *
 * Follow up: Could you implement a solution using only `O(1)` extra space
```

```
 * complexity and `O(n)` runtime complexity?
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var missingNumber = function(nums) {
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    result += i + 1 - nums[i];
  }

  return result;
};
```

0269 Alien Dictionary

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 269. Alien Dictionary
 * https://leetcode.com/problems/alien-dictionary/
 * Difficulty: Hard
 *
 * There is a new alien language that uses the English alphabet. However, the order of the
 * letters is unknown to you.
 *
 * You are given a list of strings words from the alien language's dictionary. Now it is
 * claimed that the strings in words are sorted lexicographically by the rules of this new
 * language.
 *
 * If this claim is incorrect, and the given arrangement of string in words cannot correspond
 * to any order of letters, return "".
 *
 * Otherwise, return a string of the unique letters in the new alien language sorted in
 * lexicographically increasing order by the new language's rules. If there are multiple
 * solutions, return any of them.
 */

/**
 * @param {string[]} words
 * @return {string}
 */
var alienOrder = function(words) {
  const graph = new Map();
  const inDegree = new Map();

  for (const word of words) {
    for (const char of word) {
      if (!graph.has(char)) {
        graph.set(char, new Set());
        inDegree.set(char, 0);
      }
    }
  }

  for (let i = 1; i < words.length; i++) {
    const prev = words[i - 1];
    const curr = words[i];
    const minLen = Math.min(prev.length, curr.length);

    if (prev.length > curr.length && prev.startsWith(curr)) return '';

    for (let j = 0; j < minLen; j++) {
      if (prev[j] !== curr[j]) {
        if (!graph.get(prev[j]).has(curr[j])) {
          graph.get(prev[j]).add(curr[j]);
          inDegree.set(curr[j], inDegree.get(curr[j]) + 1);
```

```
        }
        break;
      }
    }
  }

  const queue = [];
  for (const [char, degree] of inDegree) {
    if (degree === 0) queue.push(char);
  }

  let result = '';
  while (queue.length) {
    const char = queue.shift();
    result += char;

    for (const next of graph.get(char)) {
      inDegree.set(next, inDegree.get(next) - 1);
      if (inDegree.get(next) === 0) queue.push(next);
    }
  }

  return result.length === graph.size ? result : '';
};
```

0270 Closest Binary Search Tree Value

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 270. Closest Binary Search Tree Value
 * https://leetcode.com/problems/closest-binary-search-tree-value/
 * Difficulty: Easy
 *
 * Given the root of a binary search tree and a target value, return the value in the BST that
 * is closest to the target. If there are multiple answers, print the smallest.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} target
 * @return {number}
 */
var closestValue = function(root, target) {
  let closest = root.val;

  while (root) {
    closest = Math.abs(root.val - target) < Math.abs(closest - target)
      || (Math.abs(root.val - target) === Math.abs(closest - target) && root.val < closest)
      ? root.val
      : closest;

    root = target < root.val ? root.left : root.right;
  }

  return closest;
};
```

0271 Encode And Decode Strings

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 271. Encode and Decode Strings
 * https://leetcode.com/problems/encode-and-decode-strings/
 * Difficulty: Medium
 *
 * Design an algorithm to encode a list of strings to a string. The encoded string is then
sent over
 * the network and is decoded back to the original list of strings.
 *
 * Machine 1 (sender) has the function:
 * string encode(vector<string> strs) {
 *   // ... your code
 *   return encoded_string;
 * }
 * Machine 2 (receiver) has the function:
 * vector<string> decode(string s) {
 *   //... your code
 *   return strs;
 * }
 * So Machine 1 does:
 * string encoded_string = encode(strs);
 * and Machine 2 does:
 * vector<string> strs2 = decode(encoded_string);
 * strs2 in Machine 2 should be the same as strs in Machine 1.
 *
 * Implement the encode and decode methods.
 *
 * You are not allowed to solve the problem using any serialize methods (such as eval).
 */

/**
 * Encodes a list of strings to a single string.
 *
 * @param {string[]} strs
 * @return {string}
 */
var encode = function(strs) {
  return strs.map(str => `${str.length}:${str}`).join('');
};

/**
 * Decodes a single string to a list of strings.
 *
 * @param {string} s
 * @return {string[]}
 */
var decode = function(s) {
  const result = [];
  let i = 0;

  while (i < s.length) {
    const colon = s.indexOf(':', i);
    const len = parseInt(s.slice(i, colon));
    result.push(s.slice(colon + 1, colon + 1 + len));
    i = colon + 1 + len;
  }

  return result;
};
```

0272 Closest Binary Search Tree Value Ii

HARD

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 272. Closest Binary Search Tree Value II
 * https://leetcode.com/problems/closest-binary-search-tree-value-ii/
 * Difficulty: Hard
 *
 * Given the root of a binary search tree, a target value, and an integer k, return the k
values
 * in the BST that are closest to the target. You may return the answer in any order.
 *
 * You are guaranteed to have only one unique set of k values in the BST that are closest to
the
 * target.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} target
 * @param {number} k
 * @return {number[]}
 */
var closestKValues = function(root, target, k) {
  const values = [];

  inOrder(root);

  values.sort((a, b) => Math.abs(a - target) - Math.abs(b - target));
  return values.slice(0, k);

  function inOrder(node) {
    if (!node) return;
    inOrder(node.left);
    values.push(node.val);
    inOrder(node.right);
  }
};
```

0273 Integer To English Words

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 273. Integer to English Words
 * https://leetcode.com/problems/integer-to-english-words/
 * Difficulty: Hard
 *
 * Convert a non-negative integer `num` to its English words representation.
 */

/**
 * @param {number} num
 * @return {string}
 */
var numberToWords = function(num) {
```

```
  if (num === 0) {
    return 'Zero';
  }

  return convert(num).replace(/\s+/g, ' ').trim();
};

const belowTen = ['', 'One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine'];
const belowTwenty = ['Ten', 'Eleven', 'Twelve', 'Thirteen', 'Fourteen', 'Fifteen', 'Sixteen',
'Seventeen', 'Eighteen', 'Nineteen'];
const belowHundred = ['', 'Ten', 'Twenty', 'Thirty', 'Forty', 'Fifty', 'Sixty', 'Seventy',
'Eighty', 'Ninety'];

function convert(num) {
  if (num < 10) return belowTen[num];
  else if (num < 20) return belowTwenty[num - 10];
  else if (num < 100) return belowHundred[Math.floor(num / 10)] + ' ' + convert(num % 10);
  else if (num < 1000) return convert(Math.floor(num / 100)) + ' Hundred ' +  convert(num %
100);
  else if (num < 1000000) return convert(Math.floor(num / 1000)) + ' Thousand ' +  convert(num
 % 1000);
  else if (num < 1000000000) return convert(Math.floor(num / 1000000)) + ' Million ' +
convert(num % 1000000);
  else return convert(Math.floor(num / 1000000000)) + ' Billion ' +  convert(num %
1000000000);
}
```

0274 H Index

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 274. H-Index
 * https://leetcode.com/problems/h-index/
 * Difficulty: Medium
 *
 * Given an array of integers citations where citations[i] is the number of citations
 * a researcher received for their ith paper, return the researcher's h-index.
 *
 * According to the definition of h-index on Wikipedia: The h-index is defined as the
 * maximum value of h such that the given researcher has published at least h papers
 * that have each been cited at least h times.
 */

/**
 * @param {number[]} citations
 * @return {number}
 */
var hIndex = function(citations) {
  citations.sort((a, b) => a - b);
  for (let i = 0; i < citations.length; i++) {
    if (citations[i] >= citations.length - i) {
      return citations.length - i;
    }
  }
  return 0;
};
```

0275 H Index Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 275. H-Index II
 * https://leetcode.com/problems/h-index-ii/
```

```
 * Difficulty: Medium
 *
 * Given an array of integers citations where citations[i] is the number of citations a
 * researcher received for their ith paper and citations is sorted in ascending order,
 * return the researcher's h-index.
 *
 * According to the definition of h-index on Wikipedia: The h-index is defined as the
 * maximum value of h such that the given researcher has published at least h papers
 * that have each been cited at least h times.
 *
 * You must write an algorithm that runs in logarithmic time.
 */

/**
 * @param {number[]} citations
 * @return {number}
 */
var hIndex = function(citations) {
  let start = 0;

  for (let end = citations.length - 1; start <= end;) {
    const middle = Math.floor((start + end) / 2);
    if (citations.length - middle - 1 < citations[middle]) {
      end = middle - 1;
    } else {
      start = middle + 1;
    }
  }

  return citations.length - start;
};
```

0276 Paint Fence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 276. Paint Fence
 * https://leetcode.com/problems/paint-fence/
 * Difficulty: Medium
 *
 * You are painting a fence of n posts with k different colors. You must paint the posts
following
 * these rules:
 * - Every post must be painted exactly one color.
 * - There cannot be three or more consecutive posts with the same color.
 *
 * Given the two integers n and k, return the number of ways you can paint the fence.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var numWays = function(n, k) {
  if (n === 1) return k;
  if (n === 2) return k * k;

  let same = k;
  let diff = k * (k - 1);

  for (let i = 3; i <= n; i++) {
    const prevSame = same;
    same = diff;
    diff = (prevSame + diff) * (k - 1);
  }

  return same + diff;
};
```

0277 Find The Celebrity

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 277. Find the Celebrity
 * https://leetcode.com/problems/find-the-celebrity/
 * Difficulty: Medium
 *
 * Suppose you are at a party with n people labeled from 0 to n - 1 and among them, there may
 * exist one celebrity. The definition of a celebrity is that all the other n - 1 people know
 * the celebrity, but the celebrity does not know any of them.
 *
 * Now you want to find out who the celebrity is or verify that there is not one. You are only
 * allowed to ask questions like: "Hi, A. Do you know B?" to get information about whether A
 * knows B. You need to find out the celebrity (or verify there is not one) by asking as few
 * questions as possible (in the asymptotic sense).
 *
 * You are given an integer n and a helper function bool knows(a, b) that tells you whether a
 * knows b. Implement a function int findCelebrity(n). There will be exactly one celebrity if
 * they are at the party.
 *
 * Return the celebrity's label if there is a celebrity at the party. If there is no
celebrity,
 * return -1.
 *
 * Note that the n x n 2D array graph given as input is not directly available to you, and
 * instead only accessible through the helper function knows. graph[i][j] == 1 represents
 * person i knows person j, wherease graph[i][j] == 0 represents person j does not know person
 i.
 */

/**
 * @param {function} knows()
 * @return {function}
 */
var solution = function(knows) {
  /**
   * @param {integer} n Total people
   * @return {integer} The celebrity
   */
  return function(n) {
    let candidate = 0;

    for (let i = 1; i < n; i++) {
      if (knows(candidate, i)) {
        candidate = i;
      }
    }

    for (let i = 0; i < n; i++) {
      if (i !== candidate && (knows(candidate, i) || !knows(i, candidate))) {
        return -1;
      }
    }

    return candidate;
  };
};
```

0278 First Bad Version

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
```

```
 * 278. First Bad Version
 * https://leetcode.com/problems/first-bad-version/
 * Difficulty: Medium
 *
 * You are a product manager and currently leading a team to develop a new product.
 * Unfortunately, the latest version of your product fails the quality check. Since
 * each version is developed based on the previous version, all the versions after
 * a bad version are also bad.
 *
 * Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad
 * one, which causes all the following ones to be bad.
 *
 * You are given an API bool isBadVersion(version) which returns whether version is
 * bad. Implement a function to find the first bad version. You should minimize the
 * number of calls to the API.
 */

/**
 * @param {function} isBadVersion()
 * @return {function}
 */
var solution = function(isBadVersion) {
  /**
   * @param {integer} n Total versions
   * @return {integer} The first bad version
   */
  return n => {
    let left = 1;
    let right = n;

    while (left <= right) {
      const pivot = Math.floor((right + left) / 2);
      if (isBadVersion(pivot)) {
        right = pivot - 1;
      } else {
        left = pivot + 1;
      }
    }

    return left;
  };
};
```

0279 Perfect Squares

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 279. Perfect Squares
 * https://leetcode.com/problems/perfect-squares/
 * Difficulty: Medium
 *
 * Given an integer n, return the least number of perfect square numbers that sum to n.
 *
 * A perfect square is an integer that is the square of an integer; in other words, it is
 * the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares
 * while 3 and 11 are not.
 */

/**
 * @param {number} n
 * @return {number}
 */
var numSquares = function(n) {
  const dp = new Array(n + 1).fill(0).map((_, i) => !i ? 0 : Infinity);

  for (let i = 1; i <= n; i++) {
    for (let j = 1; j * j <= i; j++) {
      dp[i] = Math.min(dp[i], dp[i - j * j] + 1);
    }
```

```
  }

  return dp[n];
};
```

0280 Wiggle Sort

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 280. Wiggle Sort
 * https://leetcode.com/problems/wiggle-sort/
 * Difficulty: Medium
 *
 * Given an integer array nums, reorder it such that nums[0] <= nums[1] >= nums[2] <=
nums[3]....
 *
 * You may assume the input array always has a valid answer.
 */

/**
 * @param {number[]} nums
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var wiggleSort = function(nums) {
  const n = nums.length;

  for (let i = 0; i < n - 1; i++) {
    if (i % 2 === 0 && nums[i] > nums[i + 1]) {
      [nums[i], nums[i + 1]] = [nums[i + 1], nums[i]];
    } else if (i % 2 === 1 && nums[i] < nums[i + 1]) {
      [nums[i], nums[i + 1]] = [nums[i + 1], nums[i]];
    }
  }
};
```

0281 Zigzag Iterator

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 281. Zigzag Iterator
 * https://leetcode.com/problems/zigzag-iterator/
 * Difficulty: Medium
 *
 * Given two vectors of integers v1 and v2, implement an iterator to return their elements
 * alternately.
 *
 * Implement the ZigzagIterator class:
 * - ZigzagIterator(List<int> v1, List<int> v2) initializes the object with the two vectors
 *   v1 and v2.
 * - boolean hasNext() returns true if the iterator still has elements, and false otherwise.
 * - int next() returns the current element of the iterator and moves the iterator to the
 *   next element.
 */

/**
 * @constructor
 * @param {Integer[]} v1
 * @param {Integer[]} v2
 */
var ZigzagIterator = function ZigzagIterator(v1, v2) {
  this.queue = [];
  if (v1.length) this.queue.push([v1, 0]);
  if (v2.length) this.queue.push([v2, 0]);
```

```
};

/**
 * @this ZigzagIterator
 * @returns {boolean}
 */
ZigzagIterator.prototype.hasNext = function hasNext() {
  return this.queue.length > 0;
};

/**
 * @this ZigzagIterator
 * @returns {integer}
 */
ZigzagIterator.prototype.next = function next() {
  const [vector, index] = this.queue.shift();
  const value = vector[index];

  if (index + 1 < vector.length) {
    this.queue.push([vector, index + 1]);
  }

  return value;
};
```

0282 Expression Add Operators

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 282. Expression Add Operators
 * https://leetcode.com/problems/expression-add-operators/
 * Difficulty: Hard
 *
 * Given a string num that contains only digits and an integer target, return all
possibilities
 * to insert the binary operators '+', '-', and/or '*' between the digits of num so that the
 * resultant expression evaluates to the target value.
 *
 * Note that operands in the returned expressions should not contain leading zeros.
 */

/**
 * @param {string} num
 * @param {number} target
 * @return {string[]}
 */
var addOperators = function(num, target) {
  const result = [];
  backtrack('', 0, 0, 0);
  return result;

  function backtrack(expression, sum, prev, start) {
    if (start === num.length && sum === target) {
      result.push(expression);
      return;
    }
    for (let i = start, str = ''; i < num.length; i++) {
      str += num[i];
      const n = Number(str);
      if (!start) {
        backtrack(str, n, n, i + 1);
        if (str === '0') return;
        continue;
      }
      backtrack(expression + '*' + n, sum - prev + prev * n, prev * n, i + 1);
      backtrack(expression + '+' + n, sum + n, n, i + 1);
      backtrack(expression + '-' + n, sum - n, -n, i + 1);
      if (str === '0') return;
    }
```

```
  }
};
```

0283 Move Zeroes

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 283. Move Zeroes
 * https://leetcode.com/problems/move-zeroes/
 * Difficulty: Easy
 *
 * Given an integer array nums, move all 0's to the end of it while maintaining the
 * relative order of the non-zero elements.
 *
 * Note that you must do this in-place without making a copy of the array.
 */

/**
 * @param {number[]} nums
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var moveZeroes = function(nums) {
  for (let i = 0, j = 0; i < nums.length; i++) {
    if (nums[i] !== 0) {
      [nums[i], nums[j++]] = [nums[j], nums[i]];
    }
  }
};
```

0284 Peeking Iterator

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 284. Peeking Iterator
 * https://leetcode.com/problems/peeking-iterator/
 * Difficulty: Medium
 *
 * Design an iterator that supports the peek operation on an existing iterator in addition to
 * the hasNext and the next operations.
 *
 * Implement the PeekingIterator class:
 * - PeekingIterator(Iterator<int> nums) Initializes the object with the given integer
iterator
 *   iterator.
 * - int next() Returns the next element in the array and moves the pointer to the next
element.
 * - boolean hasNext() Returns true if there are still elements in the array.
 * - int peek() Returns the next element in the array without moving the pointer.
 *
 * Note: Each language may have a different implementation of the constructor and Iterator,
but
 * they all support the int next() and boolean hasNext() functions.
 */

/**
 * @param {Iterator} iterator
 */
var PeekingIterator = function(iterator) {
  this.iterator = iterator;
  this.peekValue = iterator.next();
};

/**
```

```
 * @return {number}
 */
PeekingIterator.prototype.peek = function() {
  return this.peekValue;
};

/**
 * @return {number}
 */
PeekingIterator.prototype.next = function() {
  const next = this.peekValue;
  this.peekValue = this.iterator.next();
  return next;
};

/**
 * @return {boolean}
 */
PeekingIterator.prototype.hasNext = function() {
  return this.peekValue > 0;
};
```

0285 Inorder Successor In Bst

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 285. Inorder Successor in BST
 * https://leetcode.com/problems/inorder-successor-in-bst/
 * Difficulty: Medium
 *
 * Given the root of a binary search tree and a node p in it, return the in-order successor of
 * that node in the BST. If the given node has no in-order successor in the tree, return null.
 *
 * The successor of a node p is the node with the smallest key greater than p.val.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @return {TreeNode}
 */
var inorderSuccessor = function(root, p) {
  let successor = null;
  while (root) {
    if (p.val < root.val) {
      successor = root;
      root = root.left;
    } else {
      root = root.right;
    }
  }
  return successor;
};
```

0286 Walls And Gates

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 286. Walls and Gates
 * https://leetcode.com/problems/walls-and-gates/
 * Difficulty: Medium
 *
 * You are given an m x n grid rooms initialized with these three possible values.
 * - -1 A wall or an obstacle.
 * - 0 A gate.
 * - INF Infinity means an empty room. We use the value 231 - 1 = 2147483647 to represent INF
 *   as you may assume that the distance to a gate is less than 2147483647.
 *
 * Fill each empty room with the distance to its nearest gate. If it is impossible to reach
 * a gate, it should be filled with INF.
 */

/**
 * @param {number[][]} rooms
 * @return {void} Do not return anything, modify rooms in-place instead.
 */
var wallsAndGates = function(rooms) {
  const rows = rooms.length;
  const cols = rooms[0].length;
  const queue = [];
  const INF = 2147483647;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (rooms[i][j] === 0) {
        queue.push([i, j]);
      }
    }
  }

  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  while (queue.length) {
    const [row, col] = queue.shift();

    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;

      if (newRow >= 0 && newRow < rows && newCol >= 0 && newCol < cols
          && rooms[newRow][newCol] === INF) {
        rooms[newRow][newCol] = rooms[row][col] + 1;
        queue.push([newRow, newCol]);
      }
    }
  }
};
```

0287 Find The Duplicate Number

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 287. Find the Duplicate Number
 * https://leetcode.com/problems/find-the-duplicate-number/
 * Difficulty: Medium
 *
 * Given an array of integers nums containing n + 1 integers where each integer is in the
 * range [1, n] inclusive.
 *
 * There is only one repeated number in nums, return this repeated number.
 *
 * You must solve the problem without modifying the array nums and using only constant extra
space.
 */

/**
```

```
 * @param {number[]} nums
 * @return {number}
 */
var findDuplicate = function(nums) {
  let slow = nums[0];
  let fast = nums[nums[0]];

  while (slow !== fast) {
    slow = nums[slow];
    fast = nums[nums[fast]];
  }
  slow = 0;
  while (slow !== fast) {
    slow = nums[slow];
    fast = nums[fast];
  }

  return slow;
};
```

0288 Unique Word Abbreviation

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 288. Unique Word Abbreviation
 * https://leetcode.com/problems/unique-word-abbreviation/
 * Difficulty: Medium
 *
 * The abbreviation of a word is a concatenation of its first letter, the number of characters
 * between the first and last letter, and its last letter. If a word has only two characters,
 * then it is an abbreviation of itself.
 *
 * For example:
 * - dog --> d1g because there is one letter between the first letter 'd' and the last letter
'g'.
 * - internationalization --> i18n because there are 18 letters between the first letter 'i'
and
 *   the last letter 'n'.
 * - it --> it because any word with only two characters is an abbreviation of itself.
 *
 * Implement the ValidWordAbbr class:
 * - ValidWordAbbr(String[] dictionary) Initializes the object with a dictionary of words.
 * - boolean isUnique(string word) Returns true if either of the following conditions are met
 *   (otherwise returns false):
 *   - There is no word in dictionary whose abbreviation is equal to word's abbreviation.
 *   - For any word in dictionary whose abbreviation is equal to word's abbreviation, that
word
 *     and word are the same.
 */

/**
 * @param {string[]} dictionary
 */
var ValidWordAbbr = function(dictionary) {
  this.abbrMap = new Map();

  for (const word of dictionary) {
    const abbr = this.getAbbreviation(word);
    if (!this.abbrMap.has(abbr)) {
      this.abbrMap.set(abbr, new Set());
    }
    this.abbrMap.get(abbr).add(word);
  }
};

/**
 * @param {string} word
 * @return {string}
 */
```

```
ValidWordAbbr.prototype.getAbbreviation = function(word) {
  if (word.length <= 2) return word;
  return word[0] + (word.length - 2) + word[word.length - 1];
};

/**
 * @param {string} word
 * @return {boolean}
 */
ValidWordAbbr.prototype.isUnique = function(word) {
  const abbr = this.getAbbreviation(word);
  const words = this.abbrMap.get(abbr);
  return !words || (words.size === 1 && words.has(word));
};
```

0289 Game Of Life

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 289. Game of Life
 * https://leetcode.com/problems/game-of-life/
 * Difficulty: Medium
 *
 * According to Wikipedia's article: "The Game of Life, also known simply as Life, is a
cellular
 * automaton devised by the British mathematician John Horton Conway in 1970."
 *
 * The board is made up of an m x n grid of cells, where each cell has an initial state: live
 * (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight
neighbors
 * (horizontal, vertical, diagonal) using the following four rules (taken from the above
Wikipedia
 * article):
 * 1. Any live cell with fewer than two live neighbors dies as if caused by under-population.
 * 2. Any live cell with two or three live neighbors lives on to the next generation.
 * 3. Any live cell with more than three live neighbors dies, as if by over-population.
 * 4. Any dead cell with exactly three live neighbors becomes a live cell, as if by
reproduction.
 *
 * The next state of the board is determined by applying the above rules simultaneously to
every
 * cell in the current state of the m x n grid board. In this process, births and deaths occur
 * simultaneously.
 *
 * Given the current state of the board, update the board to reflect its next state.
 *
 * Note that you do not need to return anything.
 */

/**
 * @param {number[][]} board
 * @return {void} Do not return anything, modify board in-place instead.
 */
var gameOfLife = function(board) {
  const directions = [[1, -1], [1, 0], [1, 1], [0, -1], [0, 1], [-1, -1], [-1, 0], [-1, 1]];
  const r = board.length;
  const c = board[0].length;

  for (let i = 0; i < r; i++) {
    for (let j = 0; j < c; j++) {
      let lives = 0;
      for (const d of directions) {
        if (d[0] + i < 0 || d[0] + i >= r || d[1] + j < 0 || d[1] + j >= c) {
          continue;
        } else if (board[d[0] + i][d[1] + j] === 1 || board[d[0] + i][d[1] + j] === 2) {
          lives++;
        }
      }
      if (board[i][j] === 0 && lives === 3) {
```

```
        board[i][j] = 3;
      } else if (board[i][j] === 1 && (lives < 2 || lives > 3)) {
        board[i][j] = 2;
      }
    }
  }

  for (let i = 0; i < r; i++) {
    for (let j = 0; j < c; j++) {
      board[i][j] %= 2;
    }
  }
};
```

0290 Word Pattern

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 290. Word Pattern
 * https://leetcode.com/problems/word-pattern/
 * Difficulty: Easy
 *
 * Given a pattern and a string s, find if s follows the same pattern.
 *
 * Here follow means a full match, such that there is a bijection between a letter in
 * pattern and a non-empty word in s. Specifically:
 * - Each letter in pattern maps to exactly one unique word in s.
 * - Each unique word in s maps to exactly one letter in pattern.
 * - No two letters map to the same word, and no two words map to the same letter.
 */

/**
 * @param {string} pattern
 * @param {string} s
 * @return {boolean}
 */
var wordPattern = function(pattern, s) {
  const words = s.split(' ');
  if (pattern.length !== words.length) return false;
  const map = new Map();
  return pattern.split('').every((char, i) =>
    map.has(char)
      ? map.get(char) === words[i]
      : !([...map.values()].includes(words[i])) && map.set(char, words[i])
  );
};
```

0291 Word Pattern Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 291. Word Pattern II
 * https://leetcode.com/problems/word-pattern-ii/
 * Difficulty: Medium
 *
 * Given a pattern and a string s, return true if s matches the pattern.
 *
 * A string s matches a pattern if there is some bijective mapping of single characters to
 * non-empty strings such that if each character in pattern is replaced by the string it
 * maps to, then the resulting string is s. A bijective mapping means that no two characters
 * map to the same string, and no character maps to two different strings.
 */
```

```
/**
 * @param {string} pattern
 * @param {string} s
 * @return {boolean}
 */
var wordPatternMatch = function(pattern, s) {
  const charToWord = new Map();
  const usedWords = new Set();

  return backtrack(0, 0);

  function backtrack(patIndex, strIndex) {
    if (patIndex === pattern.length && strIndex === s.length) return true;
    if (patIndex >= pattern.length || strIndex >= s.length) return false;

    const char = pattern[patIndex];
    if (charToWord.has(char)) {
      const word = charToWord.get(char);
      if (!s.startsWith(word, strIndex)) return false;
      return backtrack(patIndex + 1, strIndex + word.length);
    }

    for (let i = strIndex + 1; i <= s.length; i++) {
      const word = s.slice(strIndex, i);
      if (usedWords.has(word)) continue;

      charToWord.set(char, word);
      usedWords.add(word);

      if (backtrack(patIndex + 1, strIndex + word.length)) return true;

      charToWord.delete(char);
      usedWords.delete(word);
    }

    return false;
  }
};
```

0292 Nim Game

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 292. Nim Game
 * https://leetcode.com/problems/nim-game/
 * Difficulty: Easy
 *
 * You are playing the following Nim Game with your friend:
 * - Initially, there is a heap of stones on the table.
 * - You and your friend will alternate taking turns, and you go first.
 * - On each turn, the person whose turn it is will remove 1 to 3 stones from the heap.
 * - The one who removes the last stone is the winner.
 *
 * Given n, the number of stones in the heap, return true if you can win the game assuming
 * both you and your friend play optimally, otherwise return false.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var canWinNim = function(n) {
  return n % 4 !== 0;
};
```

0293 Flip Game

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 293. Flip Game
 * https://leetcode.com/problems/flip-game/
 * Difficulty: Easy
 *
 * You are playing a Flip Game with your friend.
 *
 * You are given a string currentState that contains only '+' and '-'. You and your friend
take
 * turns to flip two consecutive "++" into "--". The game ends when a person can no longer
make
 * a move, and therefore the other person will be the winner.
 *
 * Return all possible states of the string currentState after one valid move. You may return
 * the answer in any order. If there is no valid move, return an empty list [].
 */

/**
 * @param {string} currentState
 * @return {string[]}
 */
var generatePossibleNextMoves = function(currentState) {
  const results = [];

  for (let i = 0; i < currentState.length - 1; i++) {
    if (currentState[i] === '+' && currentState[i + 1] === '+') {
      results.push(
        currentState.slice(0, i) + '--' + currentState.slice(i + 2)
      );
    }
  }

  return results;
};
```

0294 Flip Game Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 294. Flip Game II
 * https://leetcode.com/problems/flip-game-ii/
 * Difficulty: Medium
 *
 * You are playing a Flip Game with your friend.
 *
 * You are given a string currentState that contains only '+' and '-'. You and your friend
take
 * turns to flip two consecutive "++" into "--". The game ends when a person can no longer
make
 * a move, and therefore the other person will be the winner.
 *
 * Return true if the starting player can guarantee a win, and false otherwise.
 */

/**
 * @param {string} currentState
 * @return {boolean}
 */
var canWin = function(currentState) {
  const map = new Map();
```

```
  return canWinFrom(currentState);

  function canWinFrom(state) {
    if (map.has(state)) return map.get(state);

    for (let i = 0; i < state.length - 1; i++) {
      if (state[i] === '+' && state[i + 1] === '+') {
        const nextState = state.slice(0, i) + '--' + state.slice(i + 2);
        if (!canWinFrom(nextState)) {
          map.set(state, true);
          return true;
        }
      }
    }

    map.set(state, false);
    return false;
  }
};
```

0295 Find Median From Data Stream

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 295. Find Median from Data Stream
 * https://leetcode.com/problems/find-median-from-data-stream/
 * Difficulty: Hard
 *
 * The median is the middle value in an ordered integer list. If the size of the list is
 * even, there is no middle value, and the median is the mean of the two middle values.
 *
 * - For example, for arr = [2,3,4], the median is 3.
 * - For example, for arr = [2,3], the median is (2 + 3) / 2 = 2.5.
 *
 * Implement the MedianFinder class:
 * - MedianFinder() initializes the MedianFinder object.
 * - void addNum(int num) adds the integer num from the data stream to the data structure.
 * - double findMedian() returns the median of all elements so far. Answers within 10-5 of
 *   the actual answer will be accepted.
 */


var MedianFinder = function() {
  this.minHeap = new MinPriorityQueue();
  this.maxHeap = new MaxPriorityQueue();
};

/**
 * @param {number} num
 * @return {void}
 */
MedianFinder.prototype.addNum = function(num) {
  this.minHeap.enqueue(num);
  this.maxHeap.enqueue(this.minHeap.dequeue().element);
  if (this.minHeap.size() < this.maxHeap.size()) {
    this.minHeap.enqueue(this.maxHeap.dequeue().element);
  }
};

/**
 * @return {number}
 */
MedianFinder.prototype.findMedian = function() {
  return this.minHeap.size() > this.maxHeap.size()
    ? this.minHeap.front().element
    : (this.minHeap.front().element + this.maxHeap.front().element) / 2;
};
```

0296 Best Meeting Point

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 296. Best Meeting Point
 * https://leetcode.com/problems/best-meeting-point/
 * Difficulty: Hard
 *
 * Given an m x n binary grid grid where each 1 marks the home of one friend, return the
minimal
 * total travel distance.
 *
 * The total travel distance is the sum of the distances between the houses of the friends and
 * the meeting point.
 *
 * The distance is calculated using Manhattan Distance, where
 * distance(p1, p2) = |p2.x - p1.x| + |p2.y - p1.y|.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minTotalDistance = function(grid) {
  const rows = [];
  const cols = [];

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[0].length; j++) {
      if (grid[i][j] === 1) {
        rows.push(i);
        cols.push(j);
      }
    }
  }

  rows.sort((a, b) => a - b);
  cols.sort((a, b) => a - b);

  let result = 0;
  for (let i = 0; i < rows.length; i++) {
    result += Math.abs(rows[i] - rows[Math.floor(rows.length / 2)]);
    result += Math.abs(cols[i] - cols[Math.floor(cols.length / 2)]);
  }
  return result;
};
```

0297 Serialize And Deserialize Binary Tree

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 297. Serialize and Deserialize Binary Tree
 * https://leetcode.com/problems/serialize-and-deserialize-binary-tree/
 * Difficulty: Hard
 *
 * Serialization is the process of converting a data structure or object into a sequence of
bits so
 * that it can be stored in a file or memory buffer, or transmitted across a network
connection link
 * to be reconstructed later in the same or another computer environment.
 *
 * Design an algorithm to serialize and deserialize a binary tree. There is no restriction on
how
```

```
 * your serialization/deserialization algorithm should work. You just need to ensure that a
binary
 * tree can be serialized to a string and this string can be deserialized to the original tree
 * structure.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */

/**
 * Encodes a tree to a single string.
 *
 * @param {TreeNode} root
 * @return {string}
 */
var serialize = function(root) {
  return JSON.stringify(root);
};

/**
 * Decodes your encoded data to tree.
 *
 * @param {string} data
 * @return {TreeNode}
 */
var deserialize = function(data) {
  return JSON.parse(data);
};
```

0298 Binary Tree Longest Consecutive Sequence

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 298. Binary Tree Longest Consecutive Sequence
 * https://leetcode.com/problems/binary-tree-longest-consecutive-sequence/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the length of the longest consecutive sequence
path.
 *
 * A consecutive sequence path is a path where the values increase by one along the path.
 *
 * Note that the path can start at any node in the tree, and you cannot go from a node to its
 * parent in the path.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var longestConsecutive = function(root) {
  let result = 0;
  traverse(root, root.val - 1, 0);
  return result;

  function traverse(node, parentValue, length) {
```

```
    if (!node) return;

    const currentLength = parentValue + 1 === node.val ? length + 1 : 1;
    result = Math.max(result, currentLength);

    traverse(node.left, node.val, currentLength);
    traverse(node.right, node.val, currentLength);
  }
};
```

0299 Bulls And Cows

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 299. Bulls and Cows
 * https://leetcode.com/problems/bulls-and-cows/
 * Difficulty: Medium
 *
 * You are playing the Bulls and Cows game with your friend.
 *
 * You write down a secret number and ask your friend to guess what the number is.
 * When your friend makes a guess, you provide a hint with the following info:
 * - The number of "bulls", which are digits in the guess that are in the correct position.
 * - The number of "cows", which are digits in the guess that are in your secret number but
 *   are located in the wrong position. Specifically, the non-bull digits in the guess that
 *   could be rearranged such that they become bulls.
 *
 * Given the secret number secret and your friend's guess guess, return the hint for your
 * friend's guess.
 *
 * The hint should be formatted as "xAyB", where x is the number of bulls and y is the number
 * of cows. Note that both secret and guess may contain duplicate digits.
 */

/**
 * @param {string} secret
 * @param {string} guess
 * @return {string}
 */
var getHint = function(secret, guess) {
  const map = Array(10).fill(0);
  let a = 0;
  let b = 0;

  for (const i in secret) {
    if (secret[i] === guess[i]) {
      a++;
    } else {
      map[secret[i]]++;
      map[guess[i]]--;
      b += map[secret[i]] <= 0;
      b += map[guess[i]] >= 0;
    }
  }

  return `${a}A${b}B`;
};
```

0300 Longest Increasing Subsequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 300. Longest Increasing Subsequence
```

```
 * https://leetcode.com/problems/longest-increasing-subsequence/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the length of the longest strictly increasing
subsequence.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var lengthOfLIS = function(nums) {
  if (!nums.length) return 0;

  const dp = new Array(nums.length).fill(1);
  for (let i = 1; i < nums.length; i++) {
    for (let j = 0; j < i; j++) {
      if (nums[i] > nums[j]) {
        dp[i] = Math.max(dp[i], dp[j] + 1);
      }
    }
  }

  return Math.max(...dp);
};
```

0301 Remove Invalid Parentheses

HARD

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 301. Remove Invalid Parentheses
 * https://leetcode.com/problems/remove-invalid-parentheses/
 * Difficulty: Hard
 *
 * Given a string s that contains parentheses and letters, remove the minimum number of
 * invalid parentheses to make the input string valid.
 *
 * Return a list of unique strings that are valid with the minimum number of removals.
 * You may return the answer in any order.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var removeInvalidParentheses = function(s) {
  const result = new Set();
  let minRemoved = Infinity;

  backtrack(s, 0, 0, 0, 0, '');

  return Array.from(result);

  function backtrack(str, index, open, close, removed, curr) {
    if (index === s.length) {
      if (open === close && removed <= minRemoved) {
        if (removed < minRemoved) {
          result.clear();
          minRemoved = removed;
        }
        result.add(curr);
      }
      return;
    }

    if (s[index] !== '(' && s[index] !== ')') {
      backtrack(str, index + 1, open, close, removed, curr + s[index]);
    } else {
      backtrack(str, index + 1, open, close, removed + 1, curr);
```

```
      if (s[index] === '(') {
        backtrack(str, index + 1, open + 1, close, removed, curr + '(');
      } else if (close < open) {
        backtrack(str, index + 1, open, close + 1, removed, curr + ')');
      }
    }
  }
};
```

0302 Smallest Rectangle Enclosing Black Pixels

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 302. Smallest Rectangle Enclosing Black Pixels
 * https://leetcode.com/problems/smallest-rectangle-enclosing-black-pixels/
 * Difficulty: Hard
 *
 * You are given an m x n binary matrix image where 0 represents a white pixel and 1
represents
 * a black pixel.
 *
 * The black pixels are connected (i.e., there is only one black region). Pixels are connected
 * horizontally and vertically.
 *
 * Given two integers x and y that represents the location of one of the black pixels, return
 * the area of the smallest (axis-aligned) rectangle that encloses all black pixels.
 *
 * You must write an algorithm with less than O(mn) runtime complexity
 */

/**
 * @param {character[][]} image
 * @param {number} x
 * @param {number} y
 * @return {number}
 */
var minArea = function(image, x, y) {
  const rows = image.length;
  const cols = image[0].length;
  const top = binarySearch(0, x, true, true);
  const bottom = binarySearch(x, rows - 1, true, false);
  const left = binarySearch(0, y, false, true);
  const right = binarySearch(y, cols - 1, false, false);

  return (bottom - top + 1) * (right - left + 1);

  function binarySearch(start, end, isRow, isMin) {
    let result = isMin ? end : start;
    while (start <= end) {
      const mid = Math.floor((start + end) / 2);
      let hasBlack = false;
      for (let i = 0; i < (isRow ? cols : rows); i++) {
        const pixel = isRow ? image[mid][i] : image[i][mid];
        if (pixel === '1') {
          hasBlack = true;
          break;
        }
      }
      if (hasBlack) {
        result = mid;
        if (isMin) {
          end = mid - 1;
        } else {
          start = mid + 1;
        }
      } else {
        if (isMin) {
          start = mid + 1;
        } else {
```

```
          end = mid - 1;
        }
      }
    }
    return result;
  }
};
```

0303 Range Sum Query Immutable

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 303. Range Sum Query - Immutable
 * https://leetcode.com/problems/range-sum-query-immutable/
 * Difficulty: Easy
 *
 * Given an integer array nums, handle multiple queries of the following type:
 * - Calculate the sum of the elements of nums between indices left and right inclusive
 *   where left <= right.
 *
 * Implement the NumArray class:
 * - NumArray(int[] nums) Initializes the object with the integer array nums.
 * - int sumRange(int left, int right) Returns the sum of the elements of nums between
 *   indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).

 */

/**
 * @param {number[]} nums
 */
var NumArray = function(nums) {
  this.nums = nums;
};

/**
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
NumArray.prototype.sumRange = function(left, right) {
  return this.nums.slice(left, right + 1).reduce((sum, n) => sum + n, 0);
};
```

0304 Range Sum Query 2D Immutable

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 304. Range Sum Query 2D - Immutable
 * https://leetcode.com/problems/range-sum-query-2d-immutable/
 * Difficulty: Medium
 *
 * Given a 2D matrix matrix, handle multiple queries of the following type:
 * - Calculate the sum of the elements of matrix inside the rectangle defined by its upper
 *   left corner (row1, col1) and lower right corner (row2, col2).
 *
 * Implement the NumMatrix class:
 * - NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.
 * - int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements
 *   of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower
 *   right corner (row2, col2).
 *
 * You must design an algorithm where sumRegion works on O(1) time complexity.
 */
```

```
/**
 * @param {number[][]} matrix
 */
var NumMatrix = function(matrix) {
  this.sums = new Array(matrix.length + 1).fill().map(() => {
    return new Array(matrix[0].length + 1).fill(0);
  });

  for (let i = 1; i <= matrix.length; i++) {
    for (let j = 1; j <= matrix[0].length; j++) {
      this.sums[i][j] = matrix[i - 1][j - 1] + this.sums[i - 1][j]
        + this.sums[i][j - 1] - this.sums[i - 1][j - 1];
    }
  }
};

/**
 * @param {number} row1
 * @param {number} col1
 * @param {number} row2
 * @param {number} col2
 * @return {number}
 */
NumMatrix.prototype.sumRegion = function(row1, col1, row2, col2) {
  return this.sums[row2 + 1][col2 + 1] - this.sums[row2 + 1][col1]
    - this.sums[row1][col2 + 1] + this.sums[row1][col1];
};
```

0305 Number Of Islands Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 305. Number of Islands II
 * https://leetcode.com/problems/number-of-islands-ii/
 * Difficulty: Hard
 *
 * You are given an empty 2D binary grid grid of size m x n. The grid represents a map where
0's
 * represent water and 1's represent land. Initially, all the cells of grid are water cells
 * (i.e., all the cells are 0's).
 *
 * We may perform an add land operation which turns the water at position into a land. You are
 * given an array positions where positions[i] = [ri, ci] is the position (ri, ci) at which we
 * should operate the ith operation.
 *
 * Return an array of integers answer where answer[i] is the number of islands after turning
 * the cell (ri, ci) into a land.
 *
 * An island is surrounded by water and is formed by connecting adjacent lands horizontally
 * or vertically. You may assume all four edges of the grid are all surrounded by water.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number[][]} positions
 * @return {number[]}
 */
var numIslands2 = function(m, n, positions) {
  const parent = new Map();
  const rank = new Map();
  const result = [];
  const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];
  let islandCount = 0;

  for (const [row, col] of positions) {
    const current = row * n + col;
    if (parent.has(current)) {
```

```
      result.push(islandCount);
      continue;
    }
    islandCount++;
    parent.set(current, current);
    rank.set(current, 0);

    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;
      if (newRow >= 0 && newRow < m && newCol >= 0 && newCol < n) {
        const neighbor = newRow * n + newCol;
        if (parent.has(neighbor)) {
          union(current, neighbor);
        }
      }
    }
    result.push(islandCount);
  }

  return result;

  function find(node) {
    if (!parent.has(node)) {
      parent.set(node, node);
      rank.set(node, 0);
    }
    if (parent.get(node) !== node) {
      parent.set(node, find(parent.get(node)));
    }
    return parent.get(node);
  }

  function union(node1, node2) {
    const root1 = find(node1);
    const root2 = find(node2);
    if (root1 === root2) return;
    const rank1 = rank.get(root1);
    const rank2 = rank.get(root2);
    if (rank1 < rank2) {
      parent.set(root1, root2);
    } else if (rank1 > rank2) {
      parent.set(root2, root1);
    } else {
      parent.set(root2, root1);
      rank.set(root1, rank1 + 1);
    }
    islandCount--;
  }
};
```

0306 Additive Number

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 306. Additive Number
 * https://leetcode.com/problems/additive-number/
 * Difficulty: Medium
 *
 * An additive number is a string whose digits can form an additive sequence.
 *
 * A valid additive sequence should contain at least three numbers. Except for the first two
 * numbers, each subsequent number in the sequence must be the sum of the preceding two.
 *
 * Given a string containing only digits, return true if it is an additive number or false
 * otherwise.
 *
 * Note: Numbers in the additive sequence cannot have leading zeros, so sequence 1, 2, 03 or
 * 1, 02, 3 is invalid.
```

```
 */

/**
 * @param {string} num
 * @param {Array} group
 * @param {number} startIndex
 * @return {boolean}
 */
var isAdditiveNumber = function(num, group = [], startIndex = 0) {
  if (startIndex === num.length && group.length >= 3) {
    return true;
  }

  for (let i = startIndex; i < num.length; i++) {
    if (num[startIndex] === '0' && i !== startIndex) {
      break;
    }
    const n = +num.slice(startIndex, i + 1);
    if (group[group.length - 1] + group[group.length - 2] !== n && group.length >= 2) {
      continue;
    }
    group.push(n);
    if (isAdditiveNumber(num, group, i + 1)) {
      return true;
    }
    group.pop();
  }

  return false;
};
```

0307 Range Sum Query Mutable

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 307. Range Sum Query - Mutable
 * https://leetcode.com/problems/range-sum-query-mutable/
 * Difficulty: Medium
 *
 * Given an integer array nums, handle multiple queries of the following types:
 * 1. Update the value of an element in nums.
 * 2. Calculate the sum of the elements of nums between indices left and right inclusive
 *    where left <= right.
 *
 * Implement the NumArray class:
 * - NumArray(int[] nums) Initializes the object with the integer array nums.
 * - void update(int index, int val) Updates the value of nums[index] to be val.
 * - int sumRange(int left, int right) Returns the sum of the elements of nums between
 *   indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).
 */

/**
 * @param {number[]} nums
 */
var NumArray = function(nums) {
  this.n = nums.length;
  this.tree = new Array(2 * this.n);

  for (let i = 0; i < this.n; i++) {
    this.tree[this.n + i] = nums[i];
  }

  for (let i = this.n - 1; i > 0; i--) {
    this.tree[i] = this.tree[i * 2] + this.tree[i * 2 + 1];
  }
};

/**
 * @param {number} index
```

```
 * @param {number} val
 * @return {void}
 */
NumArray.prototype.update = function(index, val) {
  let position = this.n + index;
  this.tree[position] = val;

  while (position) {
    let left = position;
    let right = position;
    if (position % 2 === 0) {
      right = position + 1;
    } else {
      left = position - 1;
    }
    this.tree[position >> 1] = this.tree[left] + this.tree[right];
    position >>= 1;
  }
};

/**
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
NumArray.prototype.sumRange = function(left, right) {
  let sum = 0;
  let l = this.n + left;
  let r = this.n + right + 1;

  while (l < r) {
    if (l % 2 == 1) {
      sum += this.tree[l++];
    }
    if (r % 2 == 1) {
      sum += this.tree[--r];
    }
    l >>= 1;
    r >>= 1;
  }

  return sum;
};
```

0308 Range Sum Query 2D Mutable

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 308. Range Sum Query 2D - Mutable
 * https://leetcode.com/problems/range-sum-query-2d-mutable/
 * Difficulty: Medium
 *
 * Given a 2D matrix matrix, handle multiple queries of the following types:
 * 1. Update the value of a cell in matrix.
 * 2. Calculate the sum of the elements of matrix inside the rectangle defined by its upper
left
 *    corner (row1, col1) and lower right corner (row2, col2).
 *
 * Implement the NumMatrix class:
 * - NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.
 * - void update(int row, int col, int val) Updates the value of matrix[row][col] to be val.
 * - int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of
 *   matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right
 *   corner (row2, col2).
 */

/**
 * @param {number[][]} matrix
 */
```

```
var NumMatrix = function(matrix) {
  this.rows = matrix.length;
  this.cols = matrix[0].length;
  this.matrix = matrix;
  this.tree = new Array(this.rows + 1).fill().map(() => new Array(this.cols + 1).fill(0));

  for (let i = 0; i < this.rows; i++) {
    for (let j = 0; j < this.cols; j++) {
      this.updateTree(i + 1, j + 1, matrix[i][j]);
    }
  }
};

/**
 * @param {number} row
 * @param {number} col
 * @param {number} val
 * @return {void}
 */
NumMatrix.prototype.updateTree = function(row, col, val) {
  while (row <= this.rows) {
    let c = col;
    while (c <= this.cols) {
      this.tree[row][c] += val;
      c += c & (-c);
    }
    row += row & (-row);
  }
};

/**
 * @param {number} row
 * @param {number} col
 * @return {number}
 */
NumMatrix.prototype.getSum = function(row, col) {
  let sum = 0;
  while (row > 0) {
    let c = col;
    while (c > 0) {
      sum += this.tree[row][c];
      c -= c & (-c);
    }
    row -= row & (-row);
  }
  return sum;
};

/**
 * @param {number} row
 * @param {number} col
 * @param {number} val
 * @return {void}
 */
NumMatrix.prototype.update = function(row, col, val) {
  const diff = val - this.matrix[row][col];
  this.matrix[row][col] = val;
  this.updateTree(row + 1, col + 1, diff);
};

/**
 * @param {number} row1
 * @param {number} col1
 * @param {number} row2
 * @param {number} col2
 * @return {number}
 */
NumMatrix.prototype.sumRegion = function(row1, col1, row2, col2) {
  return this.getSum(row2 + 1, col2 + 1)
    - this.getSum(row2 + 1, col1)
    - this.getSum(row1, col2 + 1)
    + this.getSum(row1, col1);
};
```

0309 Best Time To Buy And Sell Stock With Cooldown

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 309. Best Time to Buy and Sell Stock with Cooldown
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-with-cooldown/
 * Difficulty: Medium
 *
 * You are given an array prices where prices[i] is the price of a given stock on the ith day.
 *
 * Find the maximum profit you can achieve. You may complete as many transactions as you like
 * (i.e., buy one and sell one share of the stock multiple times) with the following
restrictions:
 * - After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).
 *
 * Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the
stock
 * before you buy again).
 */

/**
 * @param {number[]} prices
 * @return {number}
 */
var maxProfit = function(prices) {
  let result = 0;
  let remaining = 0;

  for (let i = 0, limit = -Infinity; i < prices.length; i++) {
    limit = Math.max(limit, remaining - prices[i]);
    remaining = Math.max(remaining, result);
    result = limit + prices[i];
  }

  return Math.max(result, remaining);
};
```

0310 Minimum Height Trees

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 310. Minimum Height Trees
 * https://leetcode.com/problems/minimum-height-trees/
 * Difficulty: Medium
 *
 * A tree is an undirected graph in which any two vertices are connected by exactly one
 * path. In other words, any connected graph without simple cycles is a tree.
 *
 * Given a tree of n nodes labelled from 0 to n - 1, and an array of n - 1 edges where
 * edges[i] = [ai, bi] indicates that there is an undirected edge between the two nodes
 * ai and bi in the tree, you can choose any node of the tree as the root. When you select
 * a node x as the root, the result tree has height h. Among all possible rooted trees,
 * those with minimum height (i.e. min(h))  are called minimum height trees (MHTs).
 *
 * Return a list of all MHTs' root labels. You can return the answer in any order.
 *
 * The height of a rooted tree is the number of edges on the longest downward path
 * between the root and a leaf.
 */

/**
```

```
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[]}
 */
var findMinHeightTrees = function(n, edges) {
  if (n === 1) return [0];

  const lookup = new Array(n).fill().map(() => new Set());
  for (const [a, b] of edges) {
    lookup[a].add(b);
    lookup[b].add(a);
  }

  let result = [];
  for (let i = 0; i < n; i++) {
    if (lookup[i].size === 1) result.push(i);
  }

  while (n > 2) {
    n -= result.length;
    const modified = [];
    for (const item of result) {
      const adj = lookup[item].values().next().value;
      lookup[adj].delete(item);
      if (lookup[adj].size === 1) {
        modified.push(adj);
      }
    }
    result = modified;
  }

  return result;
};
```

0311 Sparse Matrix Multiplication

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 311. Sparse Matrix Multiplication
 * https://leetcode.com/problems/sparse-matrix-multiplication/
 * Difficulty: Medium
 *
 * Given two sparse matrices mat1 of size m x k and mat2 of size k x n, return the result of
 * mat1 x mat2. You may assume that multiplication is always possible.
 */

/**
 * @param {number[][]} mat1
 * @param {number[][]} mat2
 * @return {number[][]}
 */
var multiply = function(mat1, mat2) {
  const m = mat1.length;
  const k = mat1[0].length;
  const n = mat2[0].length;
  const result = new Array(m).fill().map(() => new Array(n).fill(0));

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < k; j++) {
      if (mat1[i][j] !== 0) {
        for (let l = 0; l < n; l++) {
          if (mat2[j][l] !== 0) {
            result[i][l] += mat1[i][j] * mat2[j][l];
          }
        }
      }
    }
  }
```

```
  return result;
};
```

0312 Burst Balloons

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 312. Burst Balloons
 * https://leetcode.com/problems/burst-balloons/
 * Difficulty: Hard
 *
 * You are given n balloons, indexed from 0 to n - 1. Each balloon is painted with a number
 * on it represented by an array nums. You are asked to burst all the balloons.
 *
 * If you burst the ith balloon, you will get nums[i - 1] * nums[i] * nums[i + 1] coins.
 * If i - 1 or i + 1 goes out of bounds of the array, then treat it as if there is a
 * balloon with a 1 painted on it.
 *
 * Return the maximum coins you can collect by bursting the balloons wisely.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxCoins = function(nums) {
  const track = [1, ...nums, 1];
  const dp = new Array(nums.length + 2).fill().map(() => new Array(nums.length + 2).fill(0));

  for (let count = 1; count <= nums.length; count++) {
    for (let i = 1; i <= nums.length - count + 1; i++) {
      const j = i + count - 1;
      for (let k = i; k <= j; k++) {
        dp[i - 1][j + 1] = Math.max(
          dp[i - 1][j + 1],
          dp[i - 1][k] + dp[k][j + 1] + track[i - 1] * track[k] * track[j + 1]
        );
      }
    }
  }

  return dp[0][nums.length + 1];
};
```

0313 Super Ugly Number

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 313. Super Ugly Number
 * https://leetcode.com/problems/super-ugly-number/
 * Difficulty: Medium
 *
 * A super ugly number is a positive integer whose prime factors are in the array primes.
 *
 * Given an integer n and an array of integers primes, return the nth super ugly number.
 *
 * The nth super ugly number is guaranteed to fit in a 32-bit signed integer.
 */

/**
 * @param {number} n
 * @param {number[]} primes
 * @return {number}
```

```
 */
var nthSuperUglyNumber = function(n, primes) {
  const result = [1];
  const pointers = new Array(primes.length).fill(0);
  const next = [...primes];

  while (result.length < n) {
    const min = Math.min(...next);
    result.push(min);

    for (let i = 0; i < primes.length; i++) {
      if (next[i] === min) {
        pointers[i]++;
        next[i] = primes[i] * result[pointers[i]];
      }
    }
  }

  return result[n - 1];
};
```

0314 Binary Tree Vertical Order Traversal

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 314. Binary Tree Vertical Order Traversal
 * https://leetcode.com/problems/binary-tree-vertical-order-traversal/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the vertical order traversal of its nodes' values.
 * (i.e., from top to bottom, column by column).
 *
 * If two nodes are in the same row and column, the order should be from left to right.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var verticalOrder = function(root) {
  if (!root) return [];

  const columns = new Map();
  const queue = [[root, 0]];
  let minCol = 0;
  let maxCol = 0;

  while (queue.length) {
    const [node, col] = queue.shift();
    if (!columns.has(col)) columns.set(col, []);
    columns.get(col).push(node.val);

    if (node.left) {
      queue.push([node.left, col - 1]);
      minCol = Math.min(minCol, col - 1);
    }
    if (node.right) {
      queue.push([node.right, col + 1]);
      maxCol = Math.max(maxCol, col + 1);
    }
```

```
  }

  const result = [];
  for (let col = minCol; col <= maxCol; col++) {
    if (columns.has(col)) result.push(columns.get(col));
  }

  return result;
};
```

0315 Count Of Smaller Numbers After Self

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 315. Count of Smaller Numbers After Self
 * https://leetcode.com/problems/count-of-smaller-numbers-after-self/
 * Difficulty: Hard
 *
 * Given an integer array nums, return an integer array counts where counts[i] is the
 * number of smaller elements to the right of nums[i].
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var countSmaller = function(nums) {
  const result = new Array(nums.length).fill(0);
  const rank = new Map([...nums].sort((a, b) => a - b).map((n, i) => [n, i]));
  const group = new Array(nums.length + 1).fill(0);

  for (let i = nums.length - 1; i >= 0; i--) {
    const rankIndex = rank.get(nums[i]) + 1;
    for (let j = rankIndex - 1, sum = 0; j > 0; j -= j & -j) {
      sum += group[j];
      result[i] = sum;
    }
    for (let j = rankIndex; j < group.length; j += j & -j) {
      group[j]++;
    }
  }

  return result;
};
```

0316 Remove Duplicate Letters

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 316. Remove Duplicate Letters
 * https://leetcode.com/problems/remove-duplicate-letters/
 * Difficulty: Medium
 *
 * Given a string s, remove duplicate letters so that every letter appears once and only once.
 You must make sure your result is
 * the smallest in lexicographical order among all possible results.
 */

/**
 * @param {string} s
 * @return {string}
 */
```

```
var removeDuplicateLetters = function(s) {
  const stack = [];

  for (let i = 0; i < s.length; i++) {
    const letter = s[i];
    if (stack.indexOf(letter) > -1) {
      continue;
    }
    while (stack.length > 0 && stack[stack.length - 1] > letter &&
s.indexOf(stack[stack.length - 1], i) > i) {
      stack.pop();
    }
    stack.push(letter);
  }

  return stack.join('');
};
```

0317 Shortest Distance From All Buildings

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Look for breadth-first traversal and a queue.

SOURCE CODE

```
/**
 * 317. Shortest Distance from All Buildings
 * https://leetcode.com/problems/shortest-distance-from-all-buildings/
 * Difficulty: Hard
 *
 * You are given an m x n grid grid of values 0, 1, or 2, where:
 * - each 0 marks an empty land that you can pass by freely,
 * - each 1 marks a building that you cannot pass through, and
 * - each 2 marks an obstacle that you cannot pass through.
 *
 * You want to build a house on an empty land that reaches all buildings in the shortest total
 * travel distance. You can only move up, down, left, and right.
 *
 * Return the shortest travel distance for such a house. If it is not possible to build such
 * a house according to the above rules, return -1.
 *
 * The total travel distance is the sum of the distances between the houses of the friends and
 * the meeting point.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var shortestDistance = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const distances = new Array(rows).fill().map(() => new Array(cols).fill(0));
  const reachCounts = new Array(rows).fill().map(() => new Array(cols).fill(0));
  let buildingCount = 0;

  function bfs(startRow, startCol) {
    const queue = [[startRow, startCol]];
    const visited = new Array(rows).fill().map(() => new Array(cols).fill(false));
    visited[startRow][startCol] = true;
    let distance = 0;

    const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];

    while (queue.length) {
      const size = queue.length;
      distance++;
      for (let i = 0; i < size; i++) {
        const [row, col] = queue.shift();
        for (const [dr, dc] of directions) {
          const newRow = row + dr;
          const newCol = col + dc;
```

```
          if (newRow >= 0 && newRow < rows && newCol >= 0 && newCol < cols
              && !visited[newRow][newCol] && grid[newRow][newCol] !== 1
              && grid[newRow][newCol] !== 2) {
            queue.push([newRow, newCol]);
            visited[newRow][newCol] = true;
            distances[newRow][newCol] += distance;
            reachCounts[newRow][newCol]++;
          }
        }
      }
    }
  }

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 1) {
        buildingCount++;
        bfs(i, j);
      }
    }
  }

  let minDistance = Infinity;
  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 0 && reachCounts[i][j] === buildingCount) {
        minDistance = Math.min(minDistance, distances[i][j]);
      }
    }
  }

  return minDistance === Infinity ? -1 : minDistance;
};
```

0318 Maximum Product Of Word Lengths

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 318. Maximum Product of Word Lengths
 * https://leetcode.com/problems/maximum-product-of-word-lengths/
 * Difficulty: Medium
 *
 * Given a string array words, return the maximum value of length(word[i]) * length(word[j])
 * where the two words do not share common letters. If no such two words exist, return 0.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var maxProduct = function(words) {
  const letters = words.map(word => Array.from(new Set(word)));
  let result = 0;

  for (let i = 0; i < words.length - 1; i++) {
    for (let j = i + 1; j < words.length; j++) {
      if (!letters[i].some(item => letters[j].includes(item))) {
        const product = words[i].length * words[j].length;
        if (product > result) {
          result = product;
        }
      }
    }
  }

  return result;
};
```

0319 Bulb Switcher

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 319. Bulb Switcher
 * https://leetcode.com/problems/bulb-switcher/
 * Difficulty: Medium
 *
 * There are n bulbs that are initially off. You first turn on all the bulbs, then you turn
 * off every second bulb.
 *
 * On the third round, you toggle every third bulb (turning on if it's off or turning off if
 * it's on). For the ith round, you toggle every i bulb. For the nth round, you only toggle
 * the last bulb.
 *
 * Return the number of bulbs that are on after n rounds.
 */

/**
 * @param {number} n
 * @return {number}
 */
var bulbSwitch = function(n) {
  return Math.floor(n ** 0.5);
};
```

0320 Generalized Abbreviation

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 320. Generalized Abbreviation
 * https://leetcode.com/problems/generalized-abbreviation/
 * Difficulty: Medium
 *
 * A word's generalized abbreviation can be constructed by taking any number of
non-overlapping
 * and non-adjacent substrings and replacing them with their respective lengths.
 * - For example, "abcde" can be abbreviated into:
 *   - "a3e" ("bcd" turned into "3")
 *   - "1bcd1" ("a" and "e" both turned into "1")
 *   - "5" ("abcde" turned into "5")
 *   - "abcde" (no substrings replaced)
 * - However, these abbreviations are invalid:
 *   - "23" ("ab" turned into "2" and "cde" turned into "3") is invalid as the substrings
 *     chosen are adjacent.
 *   - "22de" ("ab" turned into "2" and "bc" turned into "2") is invalid as the substring
 *     chosen overlap.
 *
 * Given a string word, return a list of all the possible generalized abbreviations of word.
 * Return the answer in any order.
 */

/**
 * @param {string} word
 * @return {string[]}
 */
var generateAbbreviations = function(word) {
  const result = [];
  backtrack('', 0, 0);
  return result;

  function backtrack(current, pos, count) {
    if (pos === word.length) {
```

```
      result.push(current + (count > 0 ? count : ''));
      return;
    }

    backtrack(current + (count > 0 ? count : '') + word[pos], pos + 1, 0);
    backtrack(current, pos + 1, count + 1);
  }
};
```

0321 Create Maximum Number

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 321. Create Maximum Number
 * https://leetcode.com/problems/create-maximum-number/
 * Difficulty: Hard
 *
 * You are given two integer arrays nums1 and nums2 of lengths m and n respectively.
 * nums1 and nums2 represent the digits of two numbers. You are also given an integer k.
 *
 * Create the maximum number of length k <= m + n from digits of the two numbers. The
 * relative order of the digits from the same array must be preserved.
 *
 * Return an array of the k digits representing the answer.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @param {number} k
 * @return {number[]}
 */
var maxNumber = function(nums1, nums2, k) {
  const m = nums1.length;
  const n = nums2.length;
  let result = new Array(k).fill(0);

  for (let i = Math.max(0, k - n); i <= Math.min(k, m); i++) {
    if (i + n < k) continue;
    const candidate = merge(
      getMaxSubsequence(nums1, i),
      getMaxSubsequence(nums2, k - i),
    );
    result = candidate > result ? candidate : result;
  }

  return result;

  function getMaxSubsequence(nums, count) {
    const stack = [];
    let removeIndex = nums.length - count;

    nums.forEach(n => {
      while (stack.length && stack[stack.length - 1] < n && removeIndex) {
        stack.pop();
        removeIndex--;
      }
      stack.push(n);
    });

    return stack.slice(0, count);
  }

  function merge(a1, a2) {
    const result = [];

    while (a1.length || a2.length) {
      const compare = a1 > a2 ? a1 : a2;
      result.push(compare[0]);
```

```
      compare.shift();
    }

    return result;
  }
};
```

0322 Coin Change

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 322. Coin Change
 * https://leetcode.com/problems/coin-change/
 * Difficulty: Medium
 *
 * You are given an integer array coins representing coins of different denominations and an
 * integer amount representing a total amount of money.
 *
 * Return the fewest number of coins that you need to make up that amount. If that amount of
 * money cannot be made up by any combination of the coins, return -1.
 *
 * You may assume that you have an infinite number of each kind of coin.
 */

/**
 * @param {number[]} coins
 * @param {number} amount
 * @return {number}
 */
var coinChange = function(coins, amount) {
  const counts = new Array(amount + 1).fill(amount + 1);
  counts[0] = 0;

  for (let i = 1; i <= amount; i++) {
    for (let j = 0; j < coins.length; j++) {
      if (i - coins[j] >= 0) {
        counts[i] = Math.min(counts[i], 1 + counts[i - coins[j]]);
      }
    }
  }

  return counts[amount] !== amount + 1 ? counts[amount] : -1;
};
```

0323 Number Of Connected Components In An Undirected Graph

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 323. Number of Connected Components in an Undirected Graph
 * https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph/
 * Difficulty: Medium
 *
 * You have a graph of n nodes. You are given an integer n and an array edges where
 * edges[i] = [ai, bi] indicates that there is an edge between ai and bi in the graph.
 *
 * Return the number of connected components in the graph.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
```

```
var countComponents = function(n, edges) {
  const parent = new Array(n).fill().map((_, i) => i);
  let components = n;

  for (const [node1, node2] of edges) {
    union(node1, node2);
  }

  return components;

  function find(node) {
    if (parent[node] !== node) {
      parent[node] = find(parent[node]);
    }
    return parent[node];
  }

  function union(node1, node2) {
    const root1 = find(node1);
    const root2 = find(node2);
    if (root1 !== root2) {
      parent[root1] = root2;
      components--;
    }
  }
};
```

0324 Wiggle Sort Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 324. Wiggle Sort II
 * https://leetcode.com/problems/wiggle-sort-ii/
 * Difficulty: Medium
 *
 * Given an integer array nums, reorder it such that nums[0] < nums[1] > nums[2] < nums[3]
 *
 * You may assume the input array always has a valid answer.
 */

/**
 * @param {number[]} nums
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var wiggleSort = function(nums) {
  nums.sort((a, b) => a - b);
  const m = Math.floor((nums.length - 1) / 2);
  for (let i = 0, l = m, r = nums.length - 1, t = [...nums]; i < nums.length; i++) {
    nums[i] = i % 2 ? t[r--] : t[l--];
  }
};
```

0325 Maximum Size Subarray Sum Equals K

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 325. Maximum Size Subarray Sum Equals k
 * https://leetcode.com/problems/maximum-size-subarray-sum-equals-k/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return the maximum length of a subarray that
 * sums to k. If there is not one, return 0 instead.
 */
```

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxSubArrayLen = function(nums, k) {
  const sumIndex = new Map([[0, -1]]);
  let currentSum = 0;
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    currentSum += nums[i];
    if (sumIndex.has(currentSum - k)) {
      result = Math.max(result, i - sumIndex.get(currentSum - k));
    }
    if (!sumIndex.has(currentSum)) {
      sumIndex.set(currentSum, i);
    }
  }

  return result;
};
```

0326 Power Of Three

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 326. Power of Three
 * https://leetcode.com/problems/power-of-three/
 * Difficulty: Easy
 *
 * Given an integer n, return true if it is a power of three. Otherwise, return false.
 *
 * An integer n is a power of three, if there exists an integer x such that n == 3x.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isPowerOfThree = function(n) {
  if (n > 1) {
    while (n % 3 === 0) {
      n /= 3
    }
  }
  return n === 1;
};
```

0327 Count Of Range Sum

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 327. Count of Range Sum
 * https://leetcode.com/problems/count-of-range-sum/
 * Difficulty: Hard
 *
 * Given an integer array nums and two integers lower and upper, return the number of range
 * sums that lie in [lower, upper] inclusive.
 *
 * Range sum S(i, j) is defined as the sum of the elements in nums between indices i and j
 * inclusive, where i <= j.
```

```
 */

/**
 * @param {number[]} nums
 * @param {number} lower
 * @param {number} upper
 * @return {number}
 */
var countRangeSum = function(nums, lower, upper) {
  const result = [0];
  for (let i = 0; i < nums.length; i++) {
    result.push(result[i] + nums[i]);
  }

  function mergeSort(input, left, right) {
    if (left >= right) return 0;
    const middle = Math.floor((left + right) / 2);
    let count = mergeSort(input, left, middle) + mergeSort(input, middle + 1, right);
    let i = left;
    let j = middle + 1;
    let k = middle + 1;

    while (i <= middle) {
      while (j <= right && input[j] - input[i] < lower) j++;
      while (k <= right && input[k] - input[i] <= upper) k++;
      count += k - j;
      i++;
    }

    const sorted = [];
    i = left;
    j = middle + 1;
    while (i <= middle || j <= right) {
      if (i > middle) sorted.push(input[j++]);
      else if (j > right) sorted.push(input[i++]);
      else if (input[i] <= input[j]) sorted.push(input[i++]);
      else sorted.push(input[j++]);
    }

    for (let i = 0; i < sorted.length; i++) {
      input[left + i] = sorted[i];
    }

    return count;
  }

  return mergeSort(result, 0, result.length - 1);
};
```

0328 Odd Even Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 328. Odd Even Linked List
 * https://leetcode.com/problems/odd-even-linked-list/
 * Difficulty: Medium
 *
 * Given the head of a singly linked list, group all the nodes with odd indices together
followed
 * by the nodes with even indices, and return the reordered list.
 *
 * The first node is considered odd, and the second node is even, and so on.
 *
 * Note that the relative order inside both the even and odd groups should remain as it was in
 * the input.
 *
 * You must solve the problem in O(1) extra space complexity and O(n) time complexity.
 */
```

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var oddEvenList = function(head) {
  if (!head) return head;

  const even = head.next;
  let odd = head;

  while (odd.next && odd.next.next) {
    const pointer = odd.next;
    odd.next = odd.next.next;
    odd = odd.next;
    pointer.next = odd.next;
  }

  odd.next = even;
  return head;
};
```

0329 Longest Increasing Path In A Matrix

HARD

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 329. Longest Increasing Path in a Matrix
 * https://leetcode.com/problems/longest-increasing-path-in-a-matrix/
 * Difficulty: Hard
 *
 * Given an m x n integers matrix, return the length of the longest increasing path in matrix.
 *
 * From each cell, you can either move in four directions: left, right, up, or down. You may
 * not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).
 */

/**
 * @param {number[][]} matrix
 * @return {number}
 */
var longestIncreasingPath = function(matrix) {
  if (!matrix.length) return 0;
  const m = matrix.length;
  const n = matrix[0].length;
  const cache = new Array(m).fill().map(() => new Array(n));
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  return Math.max(...new Array(m).fill().map((_, i) =>
    new Array(n).fill().map((_, j) => dfs(i, j))).flat());

  function dfs(i, j) {
    return cache[i][j] || (cache[i][j] = 1 + Math.max(...directions.map(([di, dj]) => {
      const [a, b] = [i + di, j + dj];
      return a >= 0 && a < m && b >= 0 && b < n && matrix[a][b] > matrix[i][j]
        ? dfs(a, b) : 0;
    })));
  }
};
```

0330 Patching Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 330. Patching Array
 * https://leetcode.com/problems/patching-array/
 * Difficulty: Hard
 *
 * Given a sorted integer array nums and an integer n, add/patch elements to the array such
 * that any number in the range [1, n] inclusive can be formed by the sum of some elements
 * in the array.
 *
 * Return the minimum number of patches required.
 */

/**
 * @param {number[]} nums
 * @param {number} n
 * @return {number}
 */
var minPatches = function(nums, n) {
  let result = 0;

  for (let i = 0, k = 1; k <= n;) {
    if (i < nums.length && nums[i] <= k) {
      k += nums[i++];
    } else {
      k += k;
      result++;
    }
  }

  return result;
};
```

0331 Verify Preorder Serialization Of A Binary Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 331. Verify Preorder Serialization of a Binary Tree
 * https://leetcode.com/problems/verify-preorder-serialization-of-a-binary-tree/
 * Difficulty: Medium
 *
 * One way to serialize a binary tree is to use preorder traversal. When we encounter a
non-null
 * node, we record the node's value. If it is a null node, we record using a sentinel value
such
 * as '#'.
 *
 * For example, the above binary tree can be serialized to the string
"9,3,4,#,#,1,#,#,2,#,6,#,#",
 * where '#' represents a null node.
 *
 * Given a string of comma-separated values preorder, return true if it is a correct preorder
 * traversal serialization of a binary tree.
 *
 * It is guaranteed that each comma-separated value in the string must be either an integer or
 * a character '#' representing null pointer.
 *
 * You may assume that the input format is always valid.
 * For example, it could never contain two consecutive commas, such as "1,,3".
 * Note: You are not allowed to reconstruct the tree.
 */
```

```
/**
 * @param {string} preorder
 * @return {boolean}
 */
var isValidSerialization = function(preorder) {
  let result = 1;
  for (const node of preorder.split(',')) {
    if (result) {
      result += node === '#' ? -1 : 1;
    } else {
      return false;
    }
  }
  return result < 1;
};
```

0332 Reconstruct Itinerary

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 332. Reconstruct Itinerary
 * https://leetcode.com/problems/reconstruct-itinerary/
 * Difficulty: Hard
 *
 * You are given a list of airline tickets where tickets[i] = [fromi, toi] represent the
departure
 * and the arrival airports of one flight. Reconstruct the itinerary in order and return it.
 *
 * All of the tickets belong to a man who departs from "JFK", thus, the itinerary must begin
with
 * "JFK". If there are multiple valid itineraries, you should return the itinerary that has
the
 * smallest lexical order when read as a single string.
 *
 * For example, the itinerary ["JFK", "LGA"] has a smaller lexical order than ["JFK", "LGB"].
 *
 * You may assume all tickets form at least one valid itinerary. You must use all the tickets
once
 * and only once.
 */

/**
 * @param {string[][]} tickets
 * @return {string[]}
 */
var findItinerary = function(tickets) {
  const graph = new Map();
  const itinerary = [];

  for (const [from, to] of tickets) {
    if (!graph.has(from)) {
      graph.set(from, []);
    }
    graph.get(from).push(to);
  }

  for (const [_, destinations] of graph) {
    destinations.sort().reverse();
  }

  dfs('JFK');

  return itinerary.reverse();

  function dfs(airport) {
    while (graph.has(airport) && graph.get(airport).length) {
      dfs(graph.get(airport).pop());
```

```
    }
    itinerary.push(airport);
  }
};
```

0333 Largest Bst Subtree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 333. Largest BST Subtree
 * https://leetcode.com/problems/largest-bst-subtree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, find the largest subtree, which is also a Binary Search
 * Tree (BST), where the largest means subtree has the largest number of nodes.
 *
 * A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned
 * properties:
 * - The left subtree values are less than the value of their parent (root) node's value.
 * - The right subtree values are greater than the value of their parent (root) node's value.
 *
 * Note: A subtree must include all of its descendants.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var largestBSTSubtree = function(root) {
  let maxSize = 0;
  traverse(root);
  return maxSize;

  function traverse(node) {
    if (!node) return { isBST: true, size: 0, min: Infinity, max: -Infinity };

    const left = traverse(node.left);
    const right = traverse(node.right);
    if (left.isBST && right.isBST && node.val > left.max && node.val < right.min) {
      const size = left.size + right.size + 1;
      maxSize = Math.max(maxSize, size);
      return {
        isBST: true,
        size,
        min: Math.min(left.min, node.val),
        max: Math.max(right.max, node.val)
      };
    }

    return { isBST: false, size: 0, min: 0, max: 0 };
  }
};
```

0334 Increasing Triplet Subsequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 334. Increasing Triplet Subsequence
 * https://leetcode.com/problems/increasing-triplet-subsequence/
 * Difficulty: Medium
 *
 * Given an integer array nums, return true if there exists a triple of indices
 * (i, j, k) such that i < j < k and nums[i] < nums[j] < nums[k]. If no such
 * indices exists, return false.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var increasingTriplet = function(nums) {
  let first = Infinity;
  let second = Infinity;

  for (const current of nums) {
    if (current > second && current > first) {
      return true;
    }
    if (current > first) {
      second = current;
    } else {
      first = current;
    }
  }

  return false;
};
```

0335 Self Crossing

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 335. Self Crossing
 * https://leetcode.com/problems/self-crossing/
 * Difficulty: Hard
 *
 * You are given an array of integers distance.
 *
 * You start at the point (0, 0) on an X-Y plane, and you move distance[0] meters to the
north, then
 * distance[1] meters to the west, distance[2] meters to the south, distance[3] meters to the
east,
 * and so on. In other words, after each move, your direction changes counter-clockwise.
 *
 * Return true if your path crosses itself or false if it does not.
 */

/**
 * @param {number[]} d
 * @return {boolean}
 */
var isSelfCrossing = function(d) {
  for (let i = 3; i < d.length; i++) {
    if (d[i] >= d[i - 2] && d[i - 1] <= d[i - 3]) {
      return true;
    }
    if (i >= 4 && d[i - 1] === d[i - 3] && d[i] + d[i - 4] >= d[i - 2]) {
      return true;
    }
    if (i >= 5 && d[i - 2] >= d[i - 4] && d[i] + d[i - 4] >= d[i - 2]
        && d[i - 1] <= d[i - 3] && d[i - 1] + d[i - 5] >= d[i - 3]) {
      return true;
    }
  }
```

```
  return false;
};
```

0336 Palindrome Pairs

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 336. Palindrome Pairs
 * https://leetcode.com/problems/palindrome-pairs/
 * Difficulty: Hard
 *
 * You are given a 0-indexed array of unique strings words.
 *
 * A palindrome pair is a pair of integers (i, j) such that:
 * - 0 <= i, j < words.length,
 * - i != j, and
 * - words[i] + words[j] (the concatenation of the two strings) is a palindrome.
 *
 * Return an array of all the palindrome pairs of words.
 *
 * You must write an algorithm with O(sum of words[i].length) runtime complexity.
 */

/**
 * @param {string[]} words
 * @return {number[][]}
 */
var palindromePairs = function(words) {
  const result = [];
  const map = new Map();
  const rMap = new Map();

  words.forEach((word, i) => {
    map.set(word, i);
    rMap.set(word.split('').reverse().join(''), i);
  });

  words.forEach((word, i) => {
    for (let j = 0; j <= word.length; j++) {
      if (isPalindrome(word, 0, j - 1)) {
        const right = word.slice(j);
        if (rMap.has(right) && rMap.get(right) !== i) {
          result.push([rMap.get(right), i]);
        }
      }
      if (j < word.length && isPalindrome(word, j, word.length - 1)) {
        const left = word.slice(0, j);
        if (rMap.has(left) && rMap.get(left) !== i) {
          result.push([i, rMap.get(left)]);
        }
      }
    }
  });

  return result;

  function isPalindrome(s, start, end) {
    while (start < end) {
      if (s[start++] !== s[end--]) return false;
    }
    return true;
  }
};
```

0337 House Robber Iii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 337. House Robber III
 * https://leetcode.com/problems/house-robber-iii/
 * Difficulty: Medium
 *
 * The thief has found himself a new place for his thievery again. There is only one
 * entrance to this area, called root.
 *
 * Besides the root, each house has one and only one parent house. After a tour, the smart
 * thief realized that all houses in this place form a binary tree. It will automatically
 * contact the police if two directly-linked houses were broken into on the same night.
 *
 * Given the root of the binary tree, return the maximum amount of money the thief can rob
 * without alerting the police.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var rob = function(root) {
  return Math.max(...traverse(root));

  function traverse(node) {
    if (!node) return [0, 0];
    const [l1, l2] = traverse(node.left);
    const [r1, r2] = traverse(node.right);
    return [node.val + l2 + r2, Math.max(l1 + r1, l2 + r2, l1 + r2, l2 + r1)];
  }
};
```

0338 Counting Bits

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 338. Counting Bits
 * https://leetcode.com/problems/counting-bits/
 * Difficulty: Easy
 *
 * Given an integer n, return an array ans of length n + 1 such that for each i (0 <= i <= n),
 * ans[i] is the number of 1's in the binary representation of i.
 */

/**
 * @param {number} n
 * @return {number[]}
 */
var countBits = function(n) {
  const result = new Array(n + 1).fill(0);
  for (let i = 0; i <= n; i++) {
    result[i] = result[i >> 1] + (i & 1);
  }
```

```
  return result;
};
```

0339 Nested List Weight Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 339. Nested List Weight Sum
 * https://leetcode.com/problems/nested-list-weight-sum/
 * Difficulty: Medium
 *
 * You are given a nested list of integers nestedList. Each element is either an integer or
 * a list whose elements may also be integers or other lists.
 *
 * The depth of an integer is the number of lists that it is inside of. For example, the
nested
 * list [1,[2,2],[[3],2],1] has each integer's value set to its depth.
 *
 * Return the sum of each integer in nestedList multiplied by its depth.
 */

/**
 * @param {NestedInteger[]} nestedList
 * @return {number}
 */
var depthSum = function(nestedList) {
  return calculateDepth(nestedList, 1);

  function calculateDepth(list, depth) {
    let total = 0;
    for (const element of list) {
      if (element.isInteger()) {
        total += element.getInteger() * depth;
      } else {
        total += calculateDepth(element.getList(), depth + 1);
      }
    }
    return total;
  }
};
```

0340 Longest Substring With At Most K Distinct Characters

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 340. Longest Substring with At Most K Distinct Characters
 * https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/
 * Difficulty: Medium
 *
 * Given a string s and an integer k, return the length of the longest substring of s that
 * contains at most k distinct characters.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var lengthOfLongestSubstringKDistinct = function(s, k) {
  const map = new Map();
  let result = 0;
  let left = 0;
```

```
  for (let right = 0; right < s.length; right++) {
    map.set(s[right], (map.get(s[right]) || 0) + 1);

    while (map.size > k) {
      map.set(s[left], map.get(s[left]) - 1);
      if (map.get(s[left]) === 0) {
        map.delete(s[left]);
      }
      left++;
    }

    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

0341 Flatten Nested List Iterator

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 341. Flatten Nested List Iterator
 * https://leetcode.com/problems/flatten-nested-list-iterator/
 * Difficulty: Medium
 *
 * You are given a nested list of integers nestedList. Each element is either an integer or a
list
 * whose elements may also be integers or other lists. Implement an iterator to flatten it.
 *
 * Implement the NestedIterator class:
 * - NestedIterator(List<NestedInteger> nestedList) Initializes the iterator with the nested
list
 *   nestedList.
 * - int next() Returns the next integer in the nested list.
 * - boolean hasNext() Returns true if there are still some integers in the nested list and
false
 *   otherwise.
 *
 * Your code will be tested with the following pseudocode:
 * initialize iterator with nestedList
 * res = []
 * while iterator.hasNext()
 *   append iterator.next() to the end of res
 * return res
 *
 * If res matches the expected flattened list, then your code will be judged as correct.
 */

/**
 * @constructor
 * @param {NestedInteger[]} nestedList
 */
var NestedIterator = function(nestedList) {
  this.stack = [];
  this.flatten(nestedList);
};

/**
 * @this NestedIterator
 * @param {NestedInteger[]} nestedList
 * @returns {void}
 */
NestedIterator.prototype.flatten = function(list) {
  for (let i = list.length - 1; i >= 0; i--) {
    this.stack.push(list[i]);
  }
};
```

```
/**
 * @this NestedIterator
 * @returns {boolean}
 */
NestedIterator.prototype.hasNext = function() {
  while (this.stack.length > 0 && !this.stack[this.stack.length - 1].isInteger()) {
    const nested = this.stack.pop().getList();
    this.flatten(nested);
  }
  return this.stack.length > 0;
};

/**
 * @this NestedIterator
 * @returns {integer}
 */
NestedIterator.prototype.next = function() {
  return this.stack.pop().getInteger();
};
```

0342 Power Of Four

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 342. Power of Four
 * https://leetcode.com/problems/power-of-four/
 * Difficulty: Easy
 *
 * Given an integer n, return true if it is a power of four. Otherwise, return false.
 *
 * An integer n is a power of four, if there exists an integer x such that n == 4x.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isPowerOfFour = function(n) {
  if (n > 1) {
    while (n % 4 === 0) {
      n /= 4
    }
  }
  return n === 1;
};
```

0343 Integer Break

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 343. Integer Break
 * https://leetcode.com/problems/integer-break/
 * Difficulty: Medium
 *
 * Given an integer n, break it into the sum of k positive integers, where k >= 2,
 * and maximize the product of those integers.
 *
 * Return the maximum product you can get.
 */

/**
 * @param {number} n
 * @return {number}
```

```
 */
var integerBreak = function(n) {
  if (n <= 3) return n - 1;
  let product = 1;
  while (n > 4) {
    product *= 3;
    n -= 3;
  }
  return product * n;
};
```

0344 Reverse String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 344. Reverse String
 * https://leetcode.com/problems/reverse-string/
 * Difficulty: Easy
 *
 * Write a function that reverses a string. The input string is given as an array
 * of characters s.
 *
 * You must do this by modifying the input array in-place with O(1) extra memory.
 */

/**
 * @param {character[]} s
 * @return {void} Do not return anything, modify s in-place instead.
 */
var reverseString = function(s) {
  for (let i = 0; i < Math.floor(s.length / 2); i++) {
    [s[s.length - 1 - i], s[i]] = [s[i], s[s.length - 1 - i]];
  }
};
```

0345 Reverse Vowels Of A String

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 345. Reverse Vowels of a String
 * https://leetcode.com/problems/reverse-vowels-of-a-string/
 * Difficulty: Easy
 *
 * Write a function that takes a string as input
 * and reverse only the vowels of a string.
 */

/**
 * @param {string} s
 * @return {string}
 */
var reverseVowels = function(s) {
  const vowels = s.match(/[aeiou]/ig);
  return s.replace(/[aeiou]/ig, () => vowels.pop());
};
```

0346 Moving Average From Data Stream

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 346. Moving Average from Data Stream
 * https://leetcode.com/problems/moving-average-from-data-stream/
 * Difficulty: Easy
 *
 * Given a stream of integers and a window size, calculate the moving average of all integers
 * in the sliding window.
 *
 * Implement the MovingAverage class:
 * - MovingAverage(int size) Initializes the object with the size of the window size.
 * - double next(int val) Returns the moving average of the last size values of the stream.
 */

/**
 * @param {number} size
 */
var MovingAverage = function(size) {
  this.window = [];
  this.maxSize = size;
  this.sum = 0;
};

/**
 * @param {number} val
 * @return {number}
 */
MovingAverage.prototype.next = function(val) {
  this.window.push(val);
  this.sum += val;

  if (this.window.length > this.maxSize) {
    this.sum -= this.window.shift();
  }

  return this.sum / this.window.length;
};
```

0347 Top K Frequent Elements

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 347. Top K Frequent Elements
 * https://leetcode.com/problems/top-k-frequent-elements/
 * Difficulty: Medium
 *
 * Given an integer array `nums` and an integer `k`, return the `k` most
 * frequent elements. You may return the answer in any order.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var topKFrequent = function(nums, k) {
  const map = new Map();

  nums.forEach(value => map.set(value, (map.get(value) || 0) + 1));

  return [...map]
    .sort((a, b) => b[1] - a[1])
    .slice(0, k)
    .map(([value]) => value)
};
```

0348 Design Tic Tac Toe

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 348. Design Tic-Tac-Toe
 * https://leetcode.com/problems/design-tic-tac-toe/
 * Difficulty: Medium
 *
 * Assume the following rules are for the tic-tac-toe game on an n x n board between two
players:
 * 1. A move is guaranteed to be valid and is placed on an empty block.
 * 2. Once a winning condition is reached, no more moves are allowed.
 * 3. A player who succeeds in placing n of their marks in a horizontal, vertical, or diagonal
 *    row wins the game.
 *
 * Implement the TicTacToe class:
 * - TicTacToe(int n) Initializes the object the size of the board n.
 * - int move(int row, int col, int player) Indicates that the player with id player plays at
the
 *   cell (row, col) of the board. The move is guaranteed to be a valid move, and the two
players
 *   alternate in making moves. Return
 *   - 0 if there is no winner after the move,
 *   - 1 if player 1 is the winner after the move, or
 *   - 2 if player 2 is the winner after the move.
 */

/**
 * @param {number} n
 */
var TicTacToe = function(n) {
  this.size = n;
  this.rows = new Array(n).fill(0);
  this.cols = new Array(n).fill(0);
  this.diagonal = 0;
  this.antiDiagonal = 0;
};

/**
 * @param {number} row
 * @param {number} col
 * @param {number} player
 * @return {number}
 */
TicTacToe.prototype.move = function(row, col, player) {
  const value = player === 1 ? 1 : -1;

  this.rows[row] += value;
  this.cols[col] += value;

  if (row === col) {
    this.diagonal += value;
  }

  if (row + col === this.size - 1) {
    this.antiDiagonal += value;
  }

  if (Math.abs(this.rows[row]) === this.size || Math.abs(this.cols[col]) === this.size
    || Math.abs(this.diagonal) === this.size || Math.abs(this.antiDiagonal) === this.size) {
    return player;
  }

  return 0;
};
```

0349 Intersection Of Two Arrays

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 349. Intersection of Two Arrays
 * https://leetcode.com/problems/intersection-of-two-arrays/
 * Difficulty: Easy
 *
 * Given two integer arrays `nums1` and `nums2`, return an array of their intersection.
 *
 * Each element in the result must be unique and you may return the result in any order.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var intersection = function(nums1, nums2) {
  const set = new Set(nums1);
  return nums2.filter(value => set.delete(value));
};
```

0350 Intersection Of Two Arrays Ii

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 350. Intersection of Two Arrays II
 * https://leetcode.com/problems/intersection-of-two-arrays-ii/
 * Difficulty: Easy
 *
 * Given two integer arrays nums1 and nums2, return an array of their intersection.
 *
 * Each element in the result must appear as many times as it shows in both arrays
 * and you may return the result in any order.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var intersect = function(nums1, nums2) {
  const result = [];
  const map = {};

  nums1.forEach(value => map[value] = (map[value] || 0) + 1);
  nums2.forEach(value => {
    if (map[value]) {
      result.push(value);
      map[value]--;
    }
  });

  return result;
};
```

0351 Android Unlock Patterns

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 351. Android Unlock Patterns
 * https://leetcode.com/problems/android-unlock-patterns/
 * Difficulty: Medium
 *
 * Android devices have a special lock screen with a 3 x 3 grid of dots. Users can set an
 * "unlock pattern" by connecting the dots in a specific sequence, forming a series of joined
 * line segments where each segment's endpoints are two consecutive dots in the sequence.
 * A sequence of k dots is a valid unlock pattern if both of the following are true:
 * - All the dots in the sequence are distinct.
 * - If the line segment connecting two consecutive dots in the sequence passes through the
 *   center of any other dot, the other dot must have previously appeared in the sequence.
 *   No jumps through the center non-selected dots are allowed.
 *   - For example, connecting dots 2 and 9 without dots 5 or 6 appearing beforehand is valid
 *     because the line from dot 2 to dot 9 does not pass through the center of either dot 5
or 6.
 *   - However, connecting dots 1 and 3 without dot 2 appearing beforehand is invalid because
 *     the line from dot 1 to dot 3 passes through the center of dot 2.
 *
 * Here are some example valid and invalid unlock patterns:
 */

/**
 * @param {number} m
 * @param {number} n
 * @return {number}
 */
var numberOfPatterns = function(m, n) {
  const jumps = new Array(10).fill().map(() => new Array(10).fill(0));
  jumps[1][3] = jumps[3][1] = 2;
  jumps[1][7] = jumps[7][1] = 4;
  jumps[3][9] = jumps[9][3] = 6;
  jumps[7][9] = jumps[9][7] = 8;
  jumps[1][9] = jumps[9][1] = jumps[2][8] = jumps[8][2] = jumps[3][7]
    = jumps[7][3] = jumps[4][6] = jumps[6][4] = 5;

  function backtrack(current, visited, length) {
    if (length > n) return 0;
    let count = length >= m ? 1 : 0;
    if (length === n) return count;

    for (let next = 1; next <= 9; next++) {
      if (!visited.has(next)) {
        const jump = jumps[current][next];
        if (jump === 0 || visited.has(jump)) {
          visited.add(next);
          count += backtrack(next, visited, length + 1);
          visited.delete(next);
        }
      }
    }
    return count;
  }

  let total = 0;
  const visited = new Set();
  visited.add(1);
  total += backtrack(1, visited, 1) * 4;
  visited.delete(1);

  visited.add(2);
  total += backtrack(2, visited, 1) * 4;
  visited.delete(2);

  visited.add(5);
  total += backtrack(5, visited, 1);
```

```
  return total;
};
```

0352 Data Stream As Disjoint Intervals

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 352. Data Stream as Disjoint Intervals
 * https://leetcode.com/problems/data-stream-as-disjoint-intervals/
 * Difficulty: Hard
 *
 * Given a data stream input of non-negative integers a1, a2, ..., an, summarize the
 * numbers seen so far as a list of disjoint intervals.
 *
 * Implement the SummaryRanges class:
 * - SummaryRanges() Initializes the object with an empty stream.
 * - void addNum(int value) Adds the integer value to the stream.
 * - int[][] getIntervals() Returns a summary of the integers in the stream currently as
 *   a list of disjoint intervals [starti, endi]. The answer should be sorted by starti.
 */

var SummaryRanges = function() {
  this.list = [];
};

/**
 * @param {number} value
 * @return {void}
 */
SummaryRanges.prototype.addNum = function(value) {
  this.list[value] = true;
};

/**
 * @return {number[][]}
 */
SummaryRanges.prototype.getIntervals = function() {
  const result = [];

  for (let i = 0; i < this.list.length; i++) {
    if (this.list[i]) {
      let j = i;
      while (this.list[j]) {
        j++;
      }
      result.push([i, j - 1]);
      i = j;
    }
  }

  return result;
};
```

0353 Design Snake Game

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 353. Design Snake Game
 * https://leetcode.com/problems/design-snake-game/
 * Difficulty: Medium
 *
 * Design a Snake game that is played on a device with screen size height x width. Play the
```

```
 * game online if you are not familiar with the game.
 *
 * The snake is initially positioned at the top left corner (0, 0) with a length of 1 unit.
 *
 * You are given an array food where food[i] = (ri, ci) is the row and column position of a
 * piece of food that the snake can eat. When a snake eats a piece of food, its length and
 * the game's score both increase by 1.
 *
 * Each piece of food appears one by one on the screen, meaning the second piece of food will
 * not appear until the snake eats the first piece of food.
 *
 * When a piece of food appears on the screen, it is guaranteed that it will not appear on a
 * block occupied by the snake.
 *
 * The game is over if the snake goes out of bounds (hits a wall) or if its head occupies a
 * space that its body occupies after moving (i.e. a snake of length 4 cannot run into
itself).
 *
 * Implement the SnakeGame class:
 * - SnakeGame(int width, int height, int[][] food) Initializes the object with a screen of
size
 *   height x width and the positions of the food.
 * - int move(String direction) Returns the score of the game after applying one direction
move
 *   by the snake. If the game is over, return -1.
 */

/**
 * @param {number} width
 * @param {number} height
 * @param {number[][]} food
 */
var SnakeGame = function(width, height, food) {
  this.width = width;
  this.height = height;
  this.food = food;
  this.foodIndex = 0;
  this.score = 0;
  this.snake = [[0, 0]];
  this.bodySet = new Set(['0,0']);
};

/**
 * @param {string} direction
 * @return {number}
 */
SnakeGame.prototype.move = function(direction) {
  const head = [...this.snake[0]];

  if (direction === 'U') head[0]--;
  else if (direction === 'D') head[0]++;
  else if (direction === 'L') head[1]--;
  else if (direction === 'R') head[1]++;

  if (head[0] < 0 || head[0] >= this.height || head[1] < 0 || head[1] >= this.width) {
    return -1;
  }

  const tail = this.snake.pop();
  this.bodySet.delete(`${tail[0]},${tail[1]}`);

  const headKey = `${head[0]},${head[1]}`;
  if (this.bodySet.has(headKey)) {
    return -1;
  }

  this.snake.unshift(head);
  this.bodySet.add(headKey);

  if (this.foodIndex < this.food.length && head[0] === this.food[this.foodIndex][0]
      && head[1] === this.food[this.foodIndex][1]) {
    this.snake.push(tail);
    this.bodySet.add(`${tail[0]},${tail[1]}`);
    this.foodIndex++;
    this.score++;
```

```
  }

  return this.score;
};
```

0354 Russian Doll Envelopes

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 354. Russian Doll Envelopes
 * https://leetcode.com/problems/russian-doll-envelopes/
 * Difficulty: Hard
 *
 * You are given a 2D array of integers envelopes where envelopes[i] = [wi, hi] represents
 * the width and the height of an envelope.
 *
 * One envelope can fit into another if and only if both the width and height of one envelope
 * are greater than the other envelope's width and height.
 *
 * Return the maximum number of envelopes you can Russian doll (i.e., put one inside the
other).
 *
 * Note: You cannot rotate an envelope.
 */

/**
 * @param {number[][]} envelopes
 * @return {number}
 */
var maxEnvelopes = function(envelopes) {
  const result = [];

  envelopes.sort((a, b) => a[0] === b[0] ? b[1] - a[1] : a[0] - b[0]);
  envelopes.forEach(([_, h]) => {
    let left = 0;
    let right = result.length;
    while (left < right) {
      const middle = Math.floor((left + right) / 2);
      if (result[middle] >= h) {
        right = middle;
      } else {
        left = middle + 1;
      }
    }
    if (left === result.length) {
      result.push(h);
    } else {
      result[left] = h;
    }
  });

  return result.length;
};
```

0355 Design Twitter

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 355. Design Twitter
 * https://leetcode.com/problems/design-twitter/
 * Difficulty: Medium
 *
 * Design a simplified version of Twitter where users can post tweets, follow/unfollow another
```

```
 user,
 * and is able to see the 10 most recent tweets in the user's news feed.
 *
 * Implement the Twitter class:
 * - Twitter() Initializes your twitter object.
 * - void postTweet(int userId, int tweetId) Composes a new tweet with ID tweetId by the user
 *   userId. Each call to this function will be made with a unique tweetId.
 * - List<Integer> getNewsFeed(int userId) Retrieves the 10 most recent tweet IDs in the
user's news
 *   feed. Each item in the news feed must be posted by users who the user followed or by the
user
 *   themself. Tweets must be ordered from most recent to least recent.
 * - void follow(int followerId, int followeeId) The user with ID followerId started following
 the
 *   user with ID followeeId.
 * - void unfollow(int followerId, int followeeId) The user with ID followerId started
unfollowing
 *   the user with ID followeeId.
 */


var Twitter = function() {
  this.tweets = [];
  this.followers = new Map();
};

/**
 * @param {number} userId
 * @param {number} tweetId
 * @return {void}
 */
Twitter.prototype.postTweet = function(userId, tweetId) {
  this.tweets.unshift([userId, tweetId]);
};

/**
 * @param {number} userId
 * @return {number[]}
 */
Twitter.prototype.getNewsFeed = function(userId) {
  const result = [];
  for (let i = 0; i < this.tweets.length && result.length < 10; i++) {
    const [user, tweet] = this.tweets[i] ?? [];
    if (user === userId || (this.followers.get(userId) &&
this.followers.get(userId).has(user))) {
      result.push(tweet);
    }
  }
  return result;
};

/**
 * @param {number} followerId
 * @param {number} followeeId
 * @return {void}
 */
Twitter.prototype.follow = function(followerId, followeeId) {
  if (followerId !== followeeId) {
    if (!this.followers.has(followerId)) {
      this.followers.set(followerId, new Set());
    }
    this.followers.get(followerId).add(followeeId);
  }
};

/**
 * @param {number} followerId
 * @param {number} followeeId
 * @return {void}
 */
Twitter.prototype.unfollow = function(followerId, followeeId) {
  if (this.followers.has(followerId)) {
    this.followers.get(followerId).delete(followeeId);
  }
};
```

0356 Line Reflection

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 356. Line Reflection
 * https://leetcode.com/problems/line-reflection/
 * Difficulty: Medium
 *
 * Given n points on a 2D plane, find if there is such a line parallel to the y-axis that
reflects
 * the given points symmetrically.
 *
 * In other words, answer whether or not if there exists a line that after reflecting all
points
 * over the given line, the original points' set is the same as the reflected ones.
 *
 * Note that there can be repeated points.
 */

/**
 * @param {number[][]} points
 * @return {boolean}
 */
var isReflected = function(points) {
  const pointSet = new Set(points.map(([x, y]) => `${x},${y}`));
  let minX = Infinity;
  let maxX = -Infinity;

  for (const [x] of points) {
    minX = Math.min(minX, x);
    maxX = Math.max(maxX, x);
  }

  const sum = minX + maxX;
  for (const [x, y] of points) {
    if (!pointSet.has(`${sum - x},${y}`)) {
      return false;
    }
  }

  return true;
};
```

0357 Count Numbers With Unique Digits

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 357. Count Numbers with Unique Digits
 * https://leetcode.com/problems/count-numbers-with-unique-digits/
 * Difficulty: Medium
 *
 * Given an integer n, return the count of all numbers with unique digits, x, where 0 <= x <
10n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countNumbersWithUniqueDigits = function(n) {
  if (n === 0) return 1;
  if (n > 10) return countNumbersWithUniqueDigits(10);
  let result = 9;
```

```
  for (let i = 0; i < n - 1; i++) {
    result *= (9 - i);
  }
  return result + countNumbersWithUniqueDigits(n - 1);
};
```

0358 Rearrange String K Distance Apart

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 358. Rearrange String k Distance Apart
 * https://leetcode.com/problems/rearrange-string-k-distance-apart/
 * Difficulty: Hard
 *
 * Given a string s and an integer k, rearrange s such that the same characters are at least
 * distance k from each other. If it is not possible to rearrange the string, return an empty
 * string "".
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var rearrangeString = function(s, k) {
  if (k <= 1) return s;

  const charCount = new Array(26).fill(0);
  for (const char of s) {
    charCount[char.charCodeAt(0) - 97]++;
  }

  const maxHeap = [];
  for (let i = 0; i < 26; i++) {
    if (charCount[i] > 0) {
      maxHeap.push([charCount[i], String.fromCharCode(i + 97)]);
    }
  }
  maxHeap.sort((a, b) => b[0] - a[0]);

  const maxFreq = maxHeap[0] ? maxHeap[0][0] : 0;
  if (maxFreq > Math.ceil(s.length / k)) return '';

  const result = new Array(s.length).fill('');
  let index = 0;

  while (maxHeap.length) {
    const temp = [];
    for (let i = 0; i < k && maxHeap.length; i++) {
      const [count, char] = maxHeap.shift();
      while (index < s.length && result[index] !== '') {
        index++;
      }
      if (index >= s.length) index = 0;
      result[index] = char;
      index++;
      if (count > 1) temp.push([count - 1, char]);
    }

    temp.sort((a, b) => b[0] - a[0]);
    maxHeap.push(...temp);
    maxHeap.sort((a, b) => b[0] - a[0]);
  }

  for (let i = 0; i <= s.length - k; i++) {
    const seen = new Set();
    for (let j = i; j < i + k; j++) {
      if (seen.has(result[j])) return '';
```

```
      seen.add(result[j]);
    }
  }

  return result.join('');
};
```

0359 Logger Rate Limiter

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 359. Logger Rate Limiter
 * https://leetcode.com/problems/logger-rate-limiter/
 * Difficulty: Easy
 *
 * Design a logger system that receives a stream of messages along with their timestamps.
 * Each unique message should only be printed at most every 10 seconds (i.e. a message
 * printed at timestamp t will prevent other identical messages from being printed until
 * timestamp t + 10).
 *
 * All messages will come in chronological order. Several messages may arrive at the same
timestamp.
 *
 * Implement the Logger class:
 * - Logger() Initializes the logger object.
 * - bool shouldPrintMessage(int timestamp, string message) Returns true if the message should
 *   be printed in the given timestamp, otherwise returns false.
 */

var Logger = function() {
  this.messageTimestamps = new Map();
};

/**
 * @param {number} timestamp
 * @param {string} message
 * @return {boolean}
 */
Logger.prototype.shouldPrintMessage = function(timestamp, message) {
  const nextAllowed = this.messageTimestamps.get(message) || 0;
  if (timestamp >= nextAllowed) {
    this.messageTimestamps.set(message, timestamp + 10);
    return true;
  }
  return false;
};
```

0360 Sort Transformed Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 360. Sort Transformed Array
 * https://leetcode.com/problems/sort-transformed-array/
 * Difficulty: Medium
 *
 * Given a sorted integer array nums and three integers a, b and c, apply a quadratic function
 * of the form f(x) = ax2 + bx + c to each element nums[i] in the array, and return the array
 * in a sorted order.
 */

/**
 * @param {number[]} nums
 * @param {number} a
```

```
 * @param {number} b
 * @param {number} c
 * @return {number[]}
 */
var sortTransformedArray = function(nums, a, b, c) {
  const result = new Array(nums.length);
  let left = 0;
  let right = nums.length - 1;
  let index = a >= 0 ? nums.length - 1 : 0;

  while (left <= right) {
    const leftVal = transform(nums[left]);
    const rightVal = transform(nums[right]);

    if (a >= 0) {
      if (leftVal > rightVal) {
        result[index--] = leftVal;
        left++;
      } else {
        result[index--] = rightVal;
        right--;
      }
    } else {
      if (leftVal < rightVal) {
        result[index++] = leftVal;
        left++;
      } else {
        result[index++] = rightVal;
        right--;
      }
    }
  }

  return result;

  function transform(x) {
    return a * x * x + b * x + c;
  }
};
```

0361 Bomb Enemy

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 361. Bomb Enemy
 * https://leetcode.com/problems/bomb-enemy/
 * Difficulty: Medium
 *
 * Given an m x n matrix grid where each cell is either a wall 'W', an enemy 'E' or empty '0',
 * return the maximum enemies you can kill using one bomb. You can only place the bomb in an
 * empty cell.
 *
 * The bomb kills all the enemies in the same row and column from the planted point until it
 * hits the wall since it is too strong to be destroyed.
 */

/**
 * @param {character[][]} grid
 * @return {number}
 */
var maxKilledEnemies = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const colHits = new Array(cols).fill(0);
  let rowHits = 0;
  let result = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
```

```
      if (j === 0 || grid[i][j-1] === 'W') {
        rowHits = 0;
        for (let k = j; k < cols && grid[i][k] !== 'W'; k++) {
          if (grid[i][k] === 'E') rowHits++;
        }
      }
      if (i === 0 || grid[i-1][j] === 'W') {
        colHits[j] = 0;
        for (let k = i; k < rows && grid[k][j] !== 'W'; k++) {
          if (grid[k][j] === 'E') colHits[j]++;
        }
      }
      if (grid[i][j] === '0') {
        result = Math.max(result, rowHits + colHits[j]);
      }
    }
  }

  return result;
};
```

0362 Design Hit Counter

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 362. Design Hit Counter
 * https://leetcode.com/problems/design-hit-counter/
 * Difficulty: Medium
 *
 * Design a hit counter which counts the number of hits received in the past 5 minutes
 * (i.e., the past 300 seconds).
 *
 * Your system should accept a timestamp parameter (in seconds granularity), and you may
 * assume that calls are being made to the system in chronological order (i.e., timestamp
 * is monotonically increasing). Several hits may arrive roughly at the same time.
 *
 * Implement the HitCounter class:
 * - HitCounter() Initializes the object of the hit counter system.
 * - void hit(int timestamp) Records a hit that happened at timestamp (in seconds). Several
 *   hits may happen at the same timestamp.
 * - int getHits(int timestamp) Returns the number of hits in the past 5 minutes from
timestamp
 *   (i.e., the past 300 seconds).
 */

var HitCounter = function() {
  this.hits = [];
};

/**
 * @param {number} timestamp
 * @return {void}
 */
HitCounter.prototype.hit = function(timestamp) {
  this.hits.push(timestamp);
};

/**
 * @param {number} timestamp
 * @return {number}
 */
HitCounter.prototype.getHits = function(timestamp) {
  const threshold = timestamp - 300;
  while (this.hits.length && this.hits[0] <= threshold) {
    this.hits.shift();
  }
  return this.hits.length;
};
```

0363 Max Sum Of Rectangle No Larger Than K

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 363. Max Sum of Rectangle No Larger Than K
 * https://leetcode.com/problems/max-sum-of-rectangle-no-larger-than-k/
 * Difficulty: Hard
 *
 * Given an m x n matrix matrix and an integer k, return the max sum of a rectangle in the
 * matrix such that its sum is no larger than k.
 *
 * It is guaranteed that there will be a rectangle with a sum no larger than k.
 */

/**
 * @param {number[][]} matrix
 * @param {number} k
 * @return {number}
 */
var maxSumSubmatrix = function(matrix, k) {
  let max = -Infinity;

  for (let l = 0; l < matrix[0].length; l++) {
    const counts = new Array(matrix.length).fill(0);
    for (let r = l; r < matrix[0].length; r++) {
      for (let i = 0; i < matrix.length; i++) counts[i] += matrix[i][r];
      const set = new Set([0]);
      let sum = 0;
      let value = -Infinity;
      for (const num of counts) {
        sum += num;
        for (const previous of set) {
          if (sum - previous <= k) {
            value = Math.max(value, sum - previous);
          }
        }
        set.add(sum);
      }
      max = Math.max(max, value);
    }
  }

  return max;
};
```

0364 Nested List Weight Sum Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 364. Nested List Weight Sum II
 * https://leetcode.com/problems/nested-list-weight-sum-ii/
 * Difficulty: Medium
 *
 * You are given a nested list of integers nestedList. Each element is either an integer or
 * a list whose elements may also be integers or other lists.
 *
 * The depth of an integer is the number of lists that it is inside of. For example, the
 * nested list [1,[2,2],[[3],2],1] has each integer's value set to its depth. Let maxDepth
 * be the maximum depth of any integer.
 *
 * The weight of an integer is maxDepth - (the depth of the integer) + 1.
 *
 * Return the sum of each integer in nestedList multiplied by its weight.
```

```
 */

/**
 * @param {NestedInteger[]} nestedList
 * @return {number}
 */
var depthSumInverse = function(nestedList) {
  const maxDepth = findMaxDepth(nestedList, 1);
  return calculateSum(nestedList, 1, maxDepth);

  function findMaxDepth(list, depth) {
    let maxDepth = depth;
    for (const element of list) {
      if (!element.isInteger()) {
        maxDepth = Math.max(maxDepth, findMaxDepth(element.getList(), depth + 1));
      }
    }
    return maxDepth;
  }

  function calculateSum(list, depth, maxDepth) {
    let total = 0;
    for (const element of list) {
      if (element.isInteger()) {
        total += element.getInteger() * (maxDepth - depth + 1);
      } else {
        total += calculateSum(element.getList(), depth + 1, maxDepth);
      }
    }
    return total;
  }
};
```

0365 Water And Jug Problem

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 365. Water and Jug Problem
 * https://leetcode.com/problems/water-and-jug-problem/
 * Difficulty: Medium
 *
 * You are given two jugs with capacities x liters and y liters. You have an infinite water
 * supply. Return whether the total amount of water in both jugs may reach target using the
 * following operations:
 * - Fill either jug completely with water.
 * - Completely empty either jug.
 * - Pour water from one jug into another until the receiving jug is full, or the
 *   transferring jug is empty.
 */

/**
 * @param {number} x
 * @param {number} y
 * @param {number} target
 * @return {boolean}
 */
var canMeasureWater = function(x, y, target) {
  if (target > x + y) return false;
  if (target === 0) return true;
  if (x === 0) return target === y;
  if (y === 0) return target === x;
  return target % gcd(x, y) === 0;

  function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
  }
};
```

0366 Find Leaves Of Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 366. Find Leaves of Binary Tree
 * https://leetcode.com/problems/find-leaves-of-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, collect a tree's nodes as if you were doing this:
 * - Collect all the leaf nodes.
 * - Remove all the leaf nodes.
 * - Repeat until the tree is empty.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var findLeaves = function(root) {
  const levels = [];
  getHeight(root);
  return levels;

  function getHeight(node) {
    if (!node) return -1;

    const leftHeight = getHeight(node.left);
    const rightHeight = getHeight(node.right);
    const height = Math.max(leftHeight, rightHeight) + 1;

    if (levels.length <= height) {
      levels.push([]);
    }
    levels[height].push(node.val);

    return height;
  }
};
```

0367 Valid Perfect Square

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 367. Valid Perfect Square
 * https://leetcode.com/problems/valid-perfect-square/
 * Difficulty: Easy
 *
 * Given a positive integer num, return true if num is a perfect square or false otherwise.
 *
 * A perfect square is an integer that is the square of an integer. In other words, it is the
 * product of some integer with itself.
 *
 * You must not use any built-in library function, such as sqrt.
 */
```

```
/**
 * @param {number} num
 * @return {boolean}
 */
var isPerfectSquare = function(num) {
  let i = 1;

  while (num > 0) {
    num -= i;
    i += 2;
    if (num === 0) {
      return true;
    }
  }

  return false;
};
```

0368 Largest Divisible Subset

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 368. Largest Divisible Subset
 * https://leetcode.com/problems/largest-divisible-subset/
 * Difficulty: Medium
 *
 * Given a set of distinct positive integers nums, return the largest subset answer such that
 * every pair (answer[i], answer[j]) of elements in this subset satisfies:
 * - answer[i] % answer[j] == 0, or
 * - answer[j] % answer[i] == 0
 *
 * If there are multiple solutions, return any of them.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var largestDivisibleSubset = function(nums) {
  nums.sort((a, b) => a - b);
  const dp = nums.map(() => [1, -1]);
  let max = 0;
  for (let i = 1; i < nums.length; i++) {
    for (let j = 0; j < i; j++) {
      if (nums[i] % nums[j] === 0 && dp[j][0] + 1 > dp[i][0]) {
        dp[i] = [dp[j][0] + 1, j], dp[i][0] > dp[max][0] && (max = i);
      }
    }
  }

  const result = [];
  for (let i = max; i >= 0; i = dp[i][1]) {
    result.unshift(nums[i]);
  }

  return result;
};
```

0369 Plus One Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 369. Plus One Linked List
```

```
 * https://leetcode.com/problems/plus-one-linked-list/
 * Difficulty: Medium
 *
 * Given a non-negative integer represented as a linked list of digits, plus one to the
integer.
 *
 * The digits are stored such that the most significant digit is at the head of the list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var plusOne = function(head) {
  const list = new ListNode(0, head);
  let notNine = list;

  while (head) {
    if (head.val !== 9) notNine = head;
    head = head.next;
  }

  notNine.val++;
  notNine = notNine.next;

  while (notNine) {
    notNine.val = 0;
    notNine = notNine.next;
  }

  return list.val === 0 ? list.next : list;
};
```

0370 Range Addition

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 370. Range Addition
 * https://leetcode.com/problems/range-addition/
 * Difficulty: Medium
 *
 * You are given an integer length and an array updates where
 * updates[i] = [startIdxi, endIdxi, inci].
 *
 * You have an array arr of length length with all zeros, and you have some operation
 * to apply on arr. In the ith operation, you should increment all the elements
 * arr[startIdxi], arr[startIdxi + 1], ..., arr[endIdxi] by inci.
 *
 * Return arr after applying all the updates.
 */

/**
 * @param {number} length
 * @param {number[][]} updates
 * @return {number[]}
 */
var getModifiedArray = function(length, updates) {
  const result = new Array(length).fill(0);

  for (const [start, end, inc] of updates) {
    result[start] += inc;
    if (end + 1 < length) result[end + 1] -= inc;
```

```
  }

  for (let i = 1; i < length; i++) {
    result[i] += result[i - 1];
  }

  return result;
};
```

0371 Sum Of Two Integers

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 371. Sum of Two Integers
 * https://leetcode.com/problems/sum-of-two-integers/
 * Difficulty: Medium
 *
 * Given two integers a and b, return the sum of the two integers
 * without using the operators + and -.
 */

/**
 * @param {number} a
 * @param {number} b
 * @return {number}
 */
var getSum = function(a, b) {
  const sum = a ^ b;
  const carry = (a & b) << 1;

  if (!carry) {
    return sum;
  }

  return getSum(sum, carry);
};
```

0372 Super Pow

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 372. Super Pow
 * https://leetcode.com/problems/super-pow/
 * Difficulty: Medium
 *
 * Your task is to calculate ab mod 1337 where a is a positive integer and b
 * is an extremely large positive integer given in the form of an array.
 */

/**
 * @param {number} a
 * @param {number[]} b
 * @return {number}
 */
var superPow = function(a, b) {
  return helper(BigInt(a), BigInt(b.join('')), 1337n);
};

function helper(a, b, mod) {
  let r = 1n;

  while (b > 0n) {
    if (b % 2n == 1) {
```

```
      r = r * a % mod;
    }
    b >>= 1n;
    a = a * a % mod;
  }

  return Number(r);
};
```

0373 Find K Pairs With Smallest Sums

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 373. Find K Pairs with Smallest Sums
 * https://leetcode.com/problems/find-k-pairs-with-smallest-sums/
 * Difficulty: Medium
 *
 * You are given two integer arrays nums1 and nums2 sorted in non-decreasing order and an
integer k.
 *
 * Define a pair (u, v) which consists of one element from the first array and one element
from the
 * second array.
 *
 * Return the k pairs (u1, v1), (u2, v2), ..., (uk, vk) with the smallest sums.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @param {number} k
 * @return {number[][]}
 */
var kSmallestPairs = function(nums1, nums2, k) {
  const minHeap = new PriorityQueue((a, b) => a[0] - b[0]);
  const result = [];
  const visited = new Set();

  minHeap.enqueue([nums1[0] + nums2[0], 0, 0]);
  visited.add('0,0');

  for (let count = 0; count < k && !minHeap.isEmpty(); count++) {
    const [currentSum, index1, index2] = minHeap.dequeue();
    result.push([nums1[index1], nums2[index2]]);

    if (index1 + 1 < nums1.length && !visited.has(`${index1 + 1},${index2}`)) {
      minHeap.enqueue([nums1[index1 + 1] + nums2[index2], index1 + 1, index2]);
      visited.add(`${index1 + 1},${index2}`);
    }

    if (index2 + 1 < nums2.length && !visited.has(`${index1},${index2 + 1}`)) {
      minHeap.enqueue([nums1[index1] + nums2[index2 + 1], index1, index2 + 1]);
      visited.add(`${index1},${index2 + 1}`);
    }
  }

  return result;
};
```

0374 Guess Number Higher Or Lower

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
```

```
 * 374. Guess Number Higher or Lower
 * https://leetcode.com/problems/guess-number-higher-or-lower/
 * Difficulty: Medium
 *
 * We are playing the Guess Game. The game is as follows:
 *
 * I pick a number from 1 to n. You have to guess which number I picked.
 *
 * Every time you guess wrong, I will tell you whether the number I picked is higher
 * or lower than your guess.
 *
 * You call a pre-defined API int guess(int num), which returns 3 possible results:
 * - -1: The number I picked is lower than your guess (i.e. pick < num).
 * - 1: The number I picked is higher than your guess (i.e. pick > num).
 * - 0: The number I picked is equal to your guess (i.e. pick == num).
 *
 * Return the number that I picked.
 */

/**
 * Forward declaration of guess API.
 * @param {number} num   your guess
 * @return              -1 if num is lower than the guess number
 *                   1 if num is higher than the guess number
 *                       otherwise return 0
 * var guess = function(num) {}
 */

/**
 * @param {number} n
 * @return {number}
 */
var guessNumber = function(n) {
  let left = 1;
  let right = n;

  while (left <= right) {
    const pivot = Math.floor((right + left) / 2);
    switch (guess(pivot)) {
      case -1: right = pivot - 1; break;
      case 1: left = pivot + 1; break;
      case 0: return pivot;
    }
  }
};
```

0375 Guess Number Higher Or Lower Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 375. Guess Number Higher or Lower II
 * https://leetcode.com/problems/guess-number-higher-or-lower-ii/
 * Difficulty: Medium
 *
 * We are playing the Guessing Game. The game will work as follows:
 * 1. I pick a number between 1 and n.
 * 2. You guess a number.
 * 3. If you guess the right number, you win the game.
 * 4. If you guess the wrong number, then I will tell you whether the number I picked is
higher
 *    or lower, and you will continue guessing.
 * 5. Every time you guess a wrong number x, you will pay x dollars. If you run out of money,
 *    you lose the game.
 *
 * Given a particular n, return the minimum amount of money you need to guarantee a win
 * regardless of what number I pick.
 */

/**
```

```
 * @param {number} n
 * @return {number}
 */
var getMoneyAmount = function(n) {
  const dp = new Array(n + 1).fill().map(() => new Array(n + 1).fill(0));

  for (let i = 2; i <= n; i++) {
    for (let j = 1; j <= n - i + 1; j++) {
      const k = j + i - 1;
      let min = Infinity;
      for (let pivot = j; pivot < k; pivot++) {
        min = Math.min(min, pivot + Math.max(dp[j][pivot - 1], dp[pivot + 1][k]));
      }
      dp[j][k] = min;
    }
  }

  return dp[1][n];
};
```

0376 Wiggle Subsequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 376. Wiggle Subsequence
 * https://leetcode.com/problems/wiggle-subsequence/
 * Difficulty: Medium
 *
 * A wiggle sequence is a sequence where the differences between successive numbers strictly
 * alternate between positive and negative. The first difference (if one exists) may be either
 * positive or negative. A sequence with one element and a sequence with two non-equal
elements
 * are trivially wiggle sequences.
 * - For example, [1, 7, 4, 9, 2, 5] is a wiggle sequence because the differences (6, -3, 5,
-7, 3)
 *   alternate between positive and negative.
 * - In contrast, [1, 4, 7, 2, 5] and [1, 7, 4, 5, 5] are not wiggle sequences. The first is
not
 *   because its first two differences are positive, and the second is not because its last
 *   difference is zero.
 *
 * A subsequence is obtained by deleting some elements (possibly zero) from the original
sequence,
 * leaving the remaining elements in their original order.
 *
 * Given an integer array nums, return the length of the longest wiggle subsequence of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var wiggleMaxLength = function(nums) {
  let up = 1;
  let down = 1;

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] > nums[i - 1]) {
      up = down + 1;
    } else if (nums[i] < nums[i - 1]) {
      down = up + 1;
    }
  }

  return Math.max(up, down);
};
```

0377 Combination Sum Iv

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 377. Combination Sum IV
 * https://leetcode.com/problems/combination-sum-iv/
 * Difficulty: Medium
 *
 * Given an array of distinct integers nums and a target integer target, return the number of
 * possible combinations that add up to target.
 *
 * The test cases are generated so that the answer can fit in a 32-bit integer.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var combinationSum4 = function(nums, target) {
  const dp = new Array(target + 1).fill(0);
  dp[0] = 1;

  for (let i = 1; i <= target; i++) {
    nums.forEach(n => dp[i] += i >= n ? dp[i - n] : 0);
  }

  return dp[target];
};
```

0378 Kth Smallest Element In A Sorted Matrix

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 378. Kth Smallest Element in a Sorted Matrix
 * https://leetcode.com/problems/kth-smallest-element-in-a-sorted-matrix/
 * Difficulty: Medium
 *
 * Given an n x n matrix where each of the rows and columns is sorted in ascending order,
 * return the kth smallest element in the matrix.
 *
 * Note that it is the kth smallest element in the sorted order, not the kth distinct element.
 *
 * You must find a solution with a memory complexity better than O(n2).
 */

/**
 * @param {number[][]} matrix
 * @param {number} k
 * @return {number}
 */
var kthSmallest = function(matrix, k) {
  return matrix.flat().sort((a, b) => a - b)[k - 1];
};
```

0379 Design Phone Directory

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 379. Design Phone Directory
 * https://leetcode.com/problems/design-phone-directory/
 * Difficulty: Medium
 *
 * Design a phone directory that initially has maxNumbers empty slots that can store numbers.
 * The directory should store numbers, check if a certain slot is empty or not, and empty a
 * given slot.
 *
 * Implement the PhoneDirectory class:
 * - PhoneDirectory(int maxNumbers) Initializes the phone directory with the number of
 *   available slots maxNumbers.
 * - int get() Provides a number that is not assigned to anyone. Returns -1 if no number
 *   is available.
 * - bool check(int number) Returns true if the slot number is available and false otherwise.
 * - void release(int number) Recycles or releases the slot number.
 */

/**
 * @param {number} maxNumbers
 */
var PhoneDirectory = function(maxNumbers) {
  this.available = new Set();
  this.released = [];
  this.max = maxNumbers;
  for (let i = 0; i < maxNumbers; i++) {
    this.available.add(i);
  }
};

/**
 * @return {number}
 */
PhoneDirectory.prototype.get = function() {
  if (this.available.size === 0 && this.released.length === 0) return -1;
  let number;
  if (this.released.length > 0) {
    number = this.released.pop();
  } else {
    number = this.available.values().next().value;
    this.available.delete(number);
  }
  return number;
};

/**
 * @param {number} number
 * @return {boolean}
 */
PhoneDirectory.prototype.check = function(number) {
  return number >= 0 && number < this.max
    && (this.available.has(number) || this.released.includes(number));
};

/**
 * @param {number} number
 * @return {void}
 */
PhoneDirectory.prototype.release = function(number) {
  if (number >= 0 && number < this.max
      && !this.available.has(number) && !this.released.includes(number)) {
    this.released.push(number);
  }
};
```

0380 Insert Delete Getrandom O1

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 380. Insert Delete GetRandom O(1)
 * https://leetcode.com/problems/insert-delete-getrandom-o1/
 * Difficulty: Medium
 *
 * Implement the RandomizedSet class:
 * - RandomizedSet() Initializes the RandomizedSet object.
 * - bool insert(int val) Inserts an item val into the set if not present. Returns true if the
 *   item was not present, false otherwise.
 * - bool remove(int val) Removes an item val from the set if present. Returns true if the
item
 *   was present, false otherwise.
 * - int getRandom() Returns a random element from the current set of elements (it's
guaranteed
 *   that at least one element exists when this method is called). Each element must have the
 *   same probability of being returned.
 *
 * You must implement the functions of the class such that each function works in average O(1)
 * time complexity.
 */


var RandomizedSet = function() {
  this.set = new Set();
};

/**
 * @param {number} val
 * @return {boolean}
 */
RandomizedSet.prototype.insert = function(val) {
  const hasValue = this.set.has(val);
  this.set.add(val);
  return !hasValue;
};

/**
 * @param {number} val
 * @return {boolean}
 */
RandomizedSet.prototype.remove = function(val) {
  const hasValue = this.set.has(val);
  this.set.delete(val);
  return hasValue;
};

/**
 * @return {number}
 */
RandomizedSet.prototype.getRandom = function() {
  const item = Array.from(this.set);
  const randomIndex = Math.floor(Math.random() * item.length);
  return item[randomIndex];
};
```

0381 Insert Delete Getrandom O1 Duplicates Allowed

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 381. Insert Delete GetRandom O(1) - Duplicates allowed
 * https://leetcode.com/problems/insert-delete-getrandom-o1-duplicates-allowed/
 * Difficulty: Hard
 *
 * RandomizedCollection is a data structure that contains a collection of numbers, possibly
 * duplicates (i.e., a multiset). It should support inserting and removing specific elements
 * and also reporting a random element.
 *
 * Implement the RandomizedCollection class:
 * - RandomizedCollection() Initializes the empty RandomizedCollection object.
```

```
 * - bool insert(int val) Inserts an item val into the multiset, even if the item is already
 *   present. Returns true if the item is not present, false otherwise.
 * - bool remove(int val) Removes an item val from the multiset if present. Returns true if
 *   the item is present, false otherwise. Note that if val has multiple occurrences in the
 *   multiset, we only remove one of them.
 * - int getRandom() Returns a random element from the current multiset of elements. The
 *   probability of each element being returned is linearly related to the number of the same
 *   values the multiset contains.
 *
 * You must implement the functions of the class such that each function works on average O(1)
 * time complexity.
 *
 * Note: The test cases are generated such that getRandom will only be called if there is at
 * least one item in the RandomizedCollection.
 */

var RandomizedCollection = function() {
  this.values = [];
  this.map = new Map();
};

/**
 * @param {number} val
 * @return {boolean}
 */
RandomizedCollection.prototype.insert = function(val) {
  this.values.push(val);
  const values = this.map.get(val) || new Set();
  values.add(this.values.length - 1);
  this.map.set(val, values);
  return values.size === 1;
};

/**
 * @param {number} val
 * @return {boolean}
 */
RandomizedCollection.prototype.remove = function(val) {
  if (!this.map.has(val)) return false;
  const values = this.map.get(val);
  const index = values.values().next().value;
  values.delete(index);
  if (index < this.values.length - 1) {
    const last = this.values.pop();
    this.values[index] = last;
    const lastValues = this.map.get(last);
    lastValues.delete(this.values.length);
    lastValues.add(index);
  } else {
    this.values.pop();
  }
  if (!values.size) {
    this.map.delete(val);
  }
  return true;
};

/**
 * @return {number}
 */
RandomizedCollection.prototype.getRandom = function() {
  return this.values[Math.floor(Math.random() * this.values.length)];
};
```

0382 Linked List Random Node

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 382. Linked List Random Node
```

```
 * https://leetcode.com/problems/linked-list-random-node/
 * Difficulty: Medium
 *
 * Given a singly linked list, return a random node's value from the linked list. Each node
must
 * have the same probability of being chosen.
 *
 * Implement the Solution class:
 * - Solution(ListNode head) Initializes the object with the head of the singly-linked list
head.
 * - int getRandom() Chooses a node randomly from the list and returns its value. All the
nodes
 *   of the list should be equally likely to be chosen.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 */
var Solution = function(head) {
  this.head = head;
};

/**
 * @return {number}
 */
Solution.prototype.getRandom = function() {
  let current = this.head;
  let result = current.val;
  let count = 1;

  while (current.next) {
    current = current.next;
    count++;
    if (Math.random() < 1 / count) {
      result = current.val;
    }
  }

  return result;
};
```

0383 Ransom Note

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 383. Ransom Note
 * https://leetcode.com/problems/ransom-note/
 * Difficulty: Easy
 *
 * Given two stings ransomNote and magazine, return true if ransomNote
 * can be constructed from magazine and false otherwise.
 *
 * Each letter in magazine can only be used once in ransomNote.
 */

/**
 * @param {string} ransomNote
 * @param {string} magazine
 * @return {boolean}
 */
var canConstruct = function(ransomNote, magazine) {
  const map = new Map();
```

```
  for (let i = magazine.length - 1; i > -1; i--) {
    map.set(magazine[i], (map.get(magazine[i]) || 0) + 1);
  }

  for (let i = ransomNote.length - 1; i > -1; i--) {
    if ((map.get(ransomNote[i]) || 0) < 1) {
      return false;
    }
    map.set(ransomNote[i], map.get(ransomNote[i]) - 1);
  }

  return true;
};
```

0384 Shuffle An Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 384. Shuffle an Array
 * https://leetcode.com/problems/shuffle-an-array/
 * Difficulty: Medium
 *
 * Given an integer array nums, design an algorithm to randomly shuffle the array.
 * All permutations of the array should be equally likely as a result of the shuffling.
 *
 * Implement the Solution class:
 * - Solution(int[] nums) Initializes the object with the integer array nums.
 * - int[] reset() Resets the array to its original configuration and returns it.
 * - int[] shuffle() Returns a random shuffling of the array.
 */

/**
 * @param {number[]} nums
 */
var Solution = function(nums) {
  this.original = [...nums];
  this.result = [...nums];
};

/**
 * @return {number[]}
 */
Solution.prototype.reset = function() {
  this.result = [...this.original];
  return this.result;
};

/**
 * @return {number[]}
 */
Solution.prototype.shuffle = function() {
  for (let i = this.result.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [this.result[i], this.result[j]] = [this.result[j], this.result[i]];
  }
  return this.result;
};
```

0385 Mini Parser

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 385. Mini Parser
 * https://leetcode.com/problems/mini-parser/
 * Difficulty: Medium
 *
 * Given a string s represents the serialization of a nested list, implement a parser to
deserialize
 * it and return the deserialized NestedInteger.
 *
 * Each element is either an integer or a list whose elements may also be integers or other
lists.
 */

/**
 * @param {string} s
 * @return {NestedInteger}
 */
var deserialize = function(s) {
  return traverse(JSON.parse(s));

  function traverse(str) {
    if (Number.isInteger(str)) {
      return new NestedInteger(str);
    }
    const value = new NestedInteger();
    for (const s of str) {
      value.add(traverse(s));
    }
    return value;
  };
};
```

0386 Lexicographical Numbers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 386. Lexicographical Numbers
 * https://leetcode.com/problems/lexicographical-numbers/
 * Difficulty: Medium
 *
 * Given an integer n, return all the numbers in the range [1, n] sorted in lexicographical
order.
 *
 * You must write an algorithm that runs in O(n) time.
 */

/**
 * @param {number} n
 * @return {number[]}
 */
var lexicalOrder = function(n) {
  const result = [];
  let value = 1;

  for (let i = 0; i < n; i++) {
    result.push(value);
    if (value * 10 <= n) {
      value *= 10;
    } else {
      while (value % 10 === 9 || value >= n) {
        value = Math.floor(value / 10);
      }
      value++;
    }
  }

  return result;
};
```

0387 First Unique Character In A String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 387. First Unique Character in a String
 * https://leetcode.com/problems/first-unique-character-in-a-string/
 * Difficulty: Easy
 *
 * Given a string, find the first non-repeating character in
 * it and return it's index. If it doesn't exist, return -1.
 */

/**
 * @param {string} s
 * @return {number}
 */
var firstUniqChar = function(s) {
  for (let i = 0; i < s.length; i++) {
    if (s.indexOf(s.charAt(i)) === s.lastIndexOf(s.charAt(i))) {
      return i;
    }
  }

  return -1;
};
```

0388 Longest Absolute File Path

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 388. Longest Absolute File Path
 * https://leetcode.com/problems/longest-absolute-file-path/
 * Difficulty: Medium
 *
 * Suppose we have a file system that stores both files and directories. An example of one
system
 * is represented in the following picture:
 *
 * Here, we have dir as the only directory in the root. dir contains two subdirectories,
subdir1
 * and subdir2. subdir1 contains a file file1.ext and subdirectory subsubdir1. subdir2
contains
 * a subdirectory subsubdir2, which contains a file file2.ext.
 * In text form, it looks like this (with ■ representing the tab character):
 *
 * dir
 * ■ subdir1
 * ■ ■ file1.ext
 * ■ ■ subsubdir1
 * ■ subdir2
 * ■ ■ subsubdir2
 * ■ ■ ■ file2.ext
 *
 * If we were to write this representation in code, it will look like this:
 *
"dir\n\tsubdir1\n\t\tfile1.ext\n\t\tsubsubdir1\n\tsubdir2\n\t\tsubsubdir2\n\t\t\tfile2.ext".
 * Note that the '\n' and '\t' are the new-line and tab characters.
 *
 * Every file and directory has a unique absolute path in the file system, which is the order
of
 * directories that must be opened to reach the file/directory itself, all concatenated by
'/'s.
 * Using the above example, the absolute path to file2.ext is
```

```
"dir/subdir2/subsubdir2/file2.ext".
 * Each directory name consists of letters, digits, and/or spaces. Each file name is of the
form
 * name.extension, where name and extension consist of letters, digits, and/or spaces.
 *
 * Given a string input representing the file system in the explained format, return the
length of
 * the longest absolute path to a file in the abstracted file system. If there is no file in
the
 * system, return 0.
 *
 * Note that the testcases are generated such that the file system is valid and no file or
directory
 * name has length 0.
 */

/**
 * @param {string} input
 * @return {number}
 */
var lengthLongestPath = function(input) {
  const stack = [0];
  let result = 0;

  for (const line of input.split('\n')) {
    const level = line.lastIndexOf('\t') + 1;
    while (stack.length > level + 1) stack.pop();
    stack.push(stack.at(-1) + line.length - level + 1);
    if (line.includes('.')) {
      result = Math.max(result, stack.at(-1) - 1);
    }
  }

  return result;
};
```

0389 Find The Difference

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 389. Find the Difference
 * https://leetcode.com/problems/find-the-difference/
 * Difficulty: Easy
 *
 * You are given two strings s and t.
 *
 * String t is generated by random shuffling string s and then add one more letter at
 * a random position.
 *
 * Return the letter that was added to t.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {character}
 */
var findTheDifference = function(s, t) {
  const map = new Map();
  t.split('').forEach(c => map.set(c, (map.get(c) ?? 0) + 1));
  s.split('').forEach(c => map.set(c, map.get(c) - 1));
  return Array.from(map).find(([letter, count]) => count)[0];
};
```

0390 Elimination Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 390. Elimination Game
 * https://leetcode.com/problems/elimination-game/
 * Difficulty: Medium
 *
 * You have a list arr of all integers in the range [1, n] sorted in a strictly increasing
order.
 * Apply the following algorithm on arr:
 * - Starting from left to right, remove the first number and every other number afterward
until
 *   you reach the end of the list.
 * - Repeat the previous step again, but this time from right to left, remove the rightmost
number
 *   and every other number from the remaining numbers.
 * - Keep repeating the steps again, alternating left to right and right to left, until a
single
 *   number remains.
 *
 * Given the integer n, return the last number that remains in arr.
 */

/**
 * @param {number} n
 * @return {number}
 */
var lastRemaining = function(n) {
  let result = 1;

  for (let isRemaining = true, count = n, step = 1; count > 1;) {
    if (isRemaining || count % 2 === 1) {
      result += step;
    }
    count = Math.floor(count / 2);
    isRemaining = !isRemaining;
    step *= 2;
  }

  return result;
};
```

0391 Perfect Rectangle

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 391. Perfect Rectangle
 * https://leetcode.com/problems/perfect-rectangle/
 * Difficulty: Hard
 *
 * Given an array rectangles where rectangles[i] = [xi, yi, ai, bi] represents an axis-aligned
 * rectangle. The bottom-left point of the rectangle is (xi, yi) and the top-right point of it
 * is (ai, bi).
 *
 * Return true if all the rectangles together form an exact cover of a rectangular region.
 */

/**
 * @param {number[][]} rectangles
 * @return {boolean}
 */
var isRectangleCover = function(rectangles) {
```

```
  const result = new Set();
  let area = 0;
  let minX = Infinity;
  let maxX = -Infinity;
  let minY = Infinity;
  let maxY = -Infinity;

  for (const [x1, y1, x2, y2] of rectangles) {
    area += (x2 - x1) * (y2 - y1);
    minX = Math.min(minX, x1);
    minY = Math.min(minY, y1);
    maxX = Math.max(maxX, x2);
    maxY = Math.max(maxY, y2);
    for (const corner of [`${x1},${y1}`, `${x2},${y1}`, `${x1},${y2}`, `${x2},${y2}`]) {
      result.has(corner) ? result.delete(corner) : result.add(corner);
    }
  }

  return area === (maxX - minX) * (maxY - minY) && result.size === 4
    && result.has(`${minX},${minY}`) && result.has(`${maxX},${minY}`)
    && result.has(`${minX},${maxY}`) && result.has(`${maxX},${maxY}`);
};
```

0392 Is Subsequence

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 392. Is Subsequence
 * https://leetcode.com/problems/is-subsequence/
 * Difficulty: Easy
 *
 * Given two strings s and t, return true if s is a subsequence of t, or false otherwise.
 *
 * A subsequence of a string is a new string that is formed from the original string by
 * deleting some (can be none) of the characters without disturbing the relative positions
 * of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isSubsequence = function(s, t) {
  if (s.length > t.length) return false;
  let count = 0;
  for (let i = 0; i < t.length; i++) {
    if (s[count] === t[i]) {
      count++;
    }
  }
  return count === s.length;
};
```

0393 Utf 8 Validation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 393. UTF-8 Validation
 * https://leetcode.com/problems/utf-8-validation/
 * Difficulty: Medium
 *
 * Given an integer array data representing the data, return whether it is a valid UTF-8
```

```
encoding
 * (i.e. it translates to a sequence of valid UTF-8 encoded characters).
 *
 * A character in UTF8 can be from 1 to 4 bytes long, subjected to the following rules:
 * 1. For a 1-byte character, the first bit is a 0, followed by its Unicode code.
 * 2. For an n-bytes character, the first n bits are all one's, the n + 1 bit is 0, followed
by
 *   n - 1 bytes with the most significant 2 bits being 10.
 *
 * This is how the UTF-8 encoding would work:
 *      Number of Bytes   |        UTF-8 Octet Sequence
 *                        |              (binary)
 *    --------------------+---------------------------------------
 *             1          |   0xxxxxxx
 *             2          |   110xxxxx 10xxxxxx
 *             3          |   1110xxxx 10xxxxxx 10xxxxxx
 *             4          |   11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
 *
 * x denotes a bit in the binary form of a byte that may be either 0 or 1.
 *
 * Note: The input is an array of integers. Only the least significant 8 bits of each integer
is
 * used to store the data. This means each integer represents only 1 byte of data.
 */

/**
 * @param {number[]} data
 * @return {boolean}
 */
var validUtf8 = function(data) {
  let result = 0;

  for (const n of data) {
    if (result === 0) {
      if (n >> 7 === 0) result = 0;
      else if (n >> 5 === 0b110) result = 1;
      else if (n >> 4 === 0b1110) result = 2;
      else if (n >> 3 === 0b11110) result = 3;
      else return false;
    } else {
      if (n >> 6 !== 0b10) return false;
      result--;
    }
  }

  return result === 0;
};
```

0394 Decode String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 394. Decode String
 * https://leetcode.com/problems/decode-string/
 * Difficulty: Medium
 *
 * Given an encoded string, return its decoded string.
 *
 * The encoding rule is: k[encoded_string], where the encoded_string inside the square
 * brackets is being repeated exactly k times. Note that k is guaranteed to be a
 * positive integer.
 *
 * You may assume that the input string is always valid; there are no extra white spaces,
 * square brackets are well-formed, etc. Furthermore, you may assume that the original
 * data does not contain any digits and that digits are only for those repeat numbers, k.
 * For example, there will not be input like 3a or 2[4].
 *
 * The test cases are generated so that the length of the output will never exceed 105.
 */
```

```
/**
 * @param {string} s
 * @return {string}
 */
var decodeString = function(s) {
  const stack = [];
  let result = '';
  let decoder = 0;

  for (const c of s) {
    if (!isNaN(c) && Number(c) >= 0 && Number(c) <= 9) {
      decoder = Number(c) + decoder * 10;
    } else if (c === '[') {
      stack.push([result, decoder]);
      result = '';
      decoder = 0;
    } else if (c === ']') {
      const [previous, count] = stack.pop();
      result = previous + result.repeat(count);
    } else {
      result += c;
    }
  }

  return result;
};
```

0395 Longest Substring With At Least K Repeating Characters

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 395. Longest Substring with At Least K Repeating Characters
 * https://leetcode.com/problems/longest-substring-with-at-least-k-repeating-characters/
 * Difficulty: Medium
 *
 * Given a string s and an integer k, return the length of the longest substring of s such
 * that the frequency of each character in this substring is greater than or equal to k.
 *
 * If no such substring exists, return 0.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var longestSubstring = function(s, k) {
  for (const char of Array.from(new Set(s))) {
    if (s.match(new RegExp(char, 'g')).length < k) {
      return Math.max(...s.split(char).map(str => longestSubstring(str, k)));
    }
  }
  return s.length;
};
```

0396 Rotate Function

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 396. Rotate Function
 * https://leetcode.com/problems/rotate-function/
 * Difficulty: Medium
```

```
 *
 * You are given an integer array nums of length n.
 *
 * Assume arrk to be an array obtained by rotating nums by k positions clock-wise.
 * We define the rotation function F on nums as follow:
 * - F(k) = 0 * arrk[0] + 1 * arrk[1] + ... + (n - 1) * arrk[n - 1].
 *
 * Return the maximum value of F(0), F(1), ..., F(n-1).
 *
 * The test cases are generated so that the answer fits in a 32-bit integer.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxRotateFunction = function(nums) {
  const sum = nums.reduce((a, b) => a + b, 0);
  let current = nums.reduce((total, n, i) => total + i * n, 0);
  let result = current;

  for (let i = 1; i < nums.length; i++) {
    current = current + sum - nums.length * nums[nums.length - i];
    result = Math.max(result, current);
  }

  return result;
};
```

0397 Integer Replacement

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 397. Integer Replacement
 * https://leetcode.com/problems/integer-replacement/
 * Difficulty: Medium
 *
 * Given a positive integer n, you can apply one of the following operations:
 * 1. If n is even, replace n with n / 2.
 * 2. If n is odd, replace n with either n + 1 or n - 1.
 *
 * Return the minimum number of operations needed for n to become 1.
 */

/**
 * @param {number} n
 * @return {number}
 */
/**
 * @param {number} n
 * @param {number} count
 * @return {number}
 */
var integerReplacement = function(n, count = 0) {
  if (n === 1) return count;
  if (n % 2 === 0) {
    return integerReplacement(n / 2, count + 1);
  } else {
    return Math.min(integerReplacement(n + 1, count + 1), integerReplacement(n - 1, count +
1));
  }
};
```

0398 Random Pick Index

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 398. Random Pick Index
 * https://leetcode.com/problems/random-pick-index/
 * Difficulty: Medium
 *
 * Given an integer array nums with possible duplicates, randomly output the index of a given
 * target number. You can assume that the given target number must exist in the array.
 *
 * Implement the Solution class:
 * - Solution(int[] nums) Initializes the object with the array nums.
 * - int pick(int target) Picks a random index i from nums where nums[i] == target. If there
 *   are multiple valid i's, then each index should have an equal probability of returning.
 */

/**
 * @param {number[]} nums
 */
var Solution = function(nums) {
  this.map = new Map();
  for (let i = 0; i < nums.length; i++) {
    if (!this.map.has(nums[i])) {
      this.map.set(nums[i], []);
    }
    this.map.get(nums[i]).push(i);
  }
};

/**
 * @param {number} target
 * @return {number}
 */
Solution.prototype.pick = function(target) {
  const result = this.map.get(target);
  return result[Math.floor(Math.random() * result.length)];
};
```

0399 Evaluate Division

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 399. Evaluate Division
 * https://leetcode.com/problems/evaluate-division/
 * Difficulty: Medium
 *
 * You are given an array of variable pairs equations and an array of real numbers values,
 * where equations[i] = [Ai, Bi] and values[i] represent the equation Ai / Bi = values[i].
 * Each Ai or Bi is a string that represents a single variable.
 *
 * You are also given some queries, where queries[j] = [Cj, Dj] represents the jth query
 * where you must find the answer for Cj / Dj = ?.
 *
 * Return the answers to all queries. If a single answer cannot be determined, return -1.0.
 *
 * Note: The input is always valid. You may assume that evaluating the queries will not
 * result in division by zero and that there is no contradiction.
 *
 * Note: The variables that do not occur in the list of equations are undefined, so the
 * answer cannot be determined for them.
 */
```

```
/**
 * @param {string[][]} equations
 * @param {number[]} values
 * @param {string[][]} queries
 * @return {number[]}
 */
var calcEquation = function(equations, values, queries) {
  const map = equations.reduce((result, [a, b], index) => {
    result.set(a, [...(result.get(a) ?? []), [b, values[index]]]);
    result.set(b, [...(result.get(b) ?? []), [a, 1 / values[index]]]);
    return result;
  }, new Map());

  function traverse([a, b], seen = new Set(), current = 1) {
    if (!map.has(a) || !map.has(b)) return -1;
    if (a === b) return current;
    seen.add(a);

    for (const [key, value] of map.get(a)) {
      if (seen.has(key)) continue;
      const result = traverse([key, b], seen, current * value);
      if (result) return result;
    }
    return null;
  }

  return queries.map(item => traverse(item) ?? -1);
};
```

0400 Nth Digit

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 400. Nth Digit
 * https://leetcode.com/problems/nth-digit/
 * Difficulty: Medium
 *
 * Given an integer n, return the nth digit of the infinite integer sequence
 * [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ...].
 */

/**
 * @param {number} n
 * @return {number}
 */
var findNthDigit = function(n) {
  let total = 1;
  let start = 1;

  for (let count = 9; n > total * count; total++, count *= 10, start *= 10) {
    n -= total * count;
  }

  start += Math.floor((n - 1) / total);
  return +(start.toString()[(n - 1) % total]);
};
```

0401 Binary Watch

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 401. Binary Watch
 * https://leetcode.com/problems/binary-watch/
```

```
 * Difficulty: Easy
 *
 * A binary watch has 4 LEDs on the top to represent the hours (0-11), and 6 LEDs on the
bottom
 * to represent the minutes (0-59). Each LED represents a zero or one, with the least
significant
 * bit on the right.
 * - For example, the below binary watch reads "4:51".
 * Given an integer turnedOn which represents the number of LEDs that are currently on
(ignoring
 * the PM), return all possible times the watch could represent. You may return the answer in
 * any order.
 * The hour must not contain a leading zero.
 * - For example, "01:00" is not valid. It should be "1:00".
 * The minute must consist of two digits and may contain a leading zero.
 * - For example, "10:2" is not valid. It should be "10:02".
 */

/**
 * @param {number} turnedOn
 * @return {string[]}
 */
var readBinaryWatch = function(turnedOn) {
  const result = [];

  for (let h = 0; h < 12; h++) {
    for (let m = 0; m < 60; m++) {
      if (h.toString(2).split('1').length - 1 + m.toString(2).split('1').length - 1 ===
turnedOn) {
        result.push(`${h}:${m < 10 ? '0' + m : m}`);
      }
    }
  }

  return result;
};
```

0402 Remove K Digits

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 402. Remove K Digits
 * https://leetcode.com/problems/remove-k-digits/
 * Difficulty: Medium
 *
 * Given string num representing a non-negative integer num, and an integer k, return
 * the smallest possible integer after removing k digits from num.
 */

/**
 * @param {string} num
 * @param {number} k
 * @return {string}
 */
var removeKdigits = function(num, k) {
  const stack = [];

  for (const n of num) {
    while (k > 0 && stack.length && stack[stack.length - 1] > n) {
      stack.pop();
      k--;
    }
    stack.push(n);
  }

  while (k > 0) {
    stack.pop();
    k--;
  }
```

```
  const result = stack.join('').replace(/^0+/, '');
  return result.length ? result : '0';
};
```

0403 Frog Jump

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 403. Frog Jump
 * https://leetcode.com/problems/frog-jump/
 * Difficulty: Hard
 *
 * A frog is crossing a river. The river is divided into some number of units, and at each
 * unit, there may or may not exist a stone. The frog can jump on a stone, but it must not
 * jump into the water.
 *
 * Given a list of stones positions (in units) in sorted ascending order, determine if the
 * frog can cross the river by landing on the last stone. Initially, the frog is on the
 * first stone and assumes the first jump must be 1 unit.
 *
 * If the frog's last jump was k units, its next jump must be either k - 1, k, or k + 1
 * units. The frog can only jump in the forward direction.
 */

/**
 * @param {number[]} stones
 * @return {boolean}
 */
var canCross = function(stones) {
  const dp = new Map();
  stones.forEach(stone => dp.set(stone, new Set()));
  dp.get(0).add(0);

  for (let i = 0; i < stones.length; i++) {
    const curr = stones[i];
    for (const prevJump of dp.get(curr)) {
      for (const jump of [prevJump - 1, prevJump, prevJump + 1]) {
        if (jump > 0 && dp.has(curr + jump)) {
          dp.get(curr + jump).add(jump);
        }
      }
    }
  }

  return dp.get(stones[stones.length - 1]).size > 0;
};
```

0404 Sum Of Left Leaves

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 404. Sum of Left Leaves
 * https://leetcode.com/problems/sum-of-left-leaves/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return the sum of all left leaves.
 *
 * A leaf is a node with no children. A left leaf is a leaf that is the left child of another
node.
 */

/**
```

```
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} include
 * @return {number}
 */
var sumOfLeftLeaves = function(root, include) {
  if (!root) return 0;
  if (!root.left && !root.right && include) return root.val;
  return sumOfLeftLeaves(root.left, true) + sumOfLeftLeaves(root.right);
};
```

0405 Convert A Number To Hexadecimal

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 405. Convert a Number to Hexadecimal
 * https://leetcode.com/problems/convert-a-number-to-hexadecimal/
 * Difficulty: Easy
 *
 * Given an integer num, return a string representing its hexadecimal
 * representation. For negative integers, two’s complement method is used.
 *
 * All the letters in the answer string should be lowercase characters,
 * and there should not be any leading zeros in the answer except for
 * the zero itself.
 *
 * Note: You are not allowed to use any built-in library method to directly
 * solve this problem.
 */

/**
 * @param {number} num
 * @return {string}
 */
var toHex = function(num) {
  if (num === 0) {
    return num.toString();
  }
  const words = '0123456789abcdef';
  let result = '';
  while (num !== 0) {
    result = words[num & 0xf] + result;
    num >>>= 4;
  }
  return result;
};
```

0406 Queue Reconstruction By Height

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 406. Queue Reconstruction by Height
 * https://leetcode.com/problems/queue-reconstruction-by-height/
 * Difficulty: Medium
 *
 * You are given an array of people, people, which are the attributes of some people in a
```

```
queue
 * (not necessarily in order). Each people[i] = [hi, ki] represents the ith person of height
 * hi with exactly ki other people in front who have a height greater than or equal to hi.
 *
 * Reconstruct and return the queue that is represented by the input array people. The
returned
 * queue should be formatted as an array queue, where queue[j] = [hj, kj] is the attributes of
 * the jth person in the queue (queue[0] is the person at the front of the queue).
 */

/**
 * @param {number[][]} people
 * @return {number[][]}
 */
var reconstructQueue = function(people) {
  return people
    .sort(([h1, k1], [h2, k2]) => h2 - h1 || k1 - k2)
    .reduce((queue, person) => queue.splice(person[1], 0, person) && queue, []);
};
```

0407 Trapping Rain Water Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 407. Trapping Rain Water II
 * https://leetcode.com/problems/trapping-rain-water-ii/
 * Difficulty: Hard
 *
 * Given an m x n integer matrix heightMap representing the height of each unit cell in a 2D
 * elevation map, return the volume of water it can trap after raining.
 */

/**
 * @param {number[][]} heightMap
 * @return {number}
 */
var trapRainWater = function(heightMap) {
  const minHeap = new MinPriorityQueue({ priority: ([value]) => value });
  const history = new Array(heightMap.length).fill(0)
    .map(() => new Array(heightMap[0].length).fill(false));

  for (let i = 0; i < heightMap.length; i++) {
    for (let j = 0; j < heightMap[0].length; j++) {
      if (i === 0 || i === heightMap.length - 1 || j === 0 || j === heightMap[0].length - 1) {
        minHeap.enqueue([heightMap[i][j], i, j]);
        history[i][j] = true;
      }
    }
  }

  let result = 0;
  while (minHeap.size()) {
    const [weight, i, j] = minHeap.dequeue().element;
    [[0, 1], [1, 0], [0, -1], [-1, 0]].forEach(([x, y]) => {
      const [rowIndex, columnIndex] = [i + x, j + y];
      if (rowIndex >= 0 && rowIndex < heightMap.length && columnIndex >= 0
          && columnIndex < heightMap[0].length && !history[rowIndex][columnIndex]) {
        result += Math.max(0, weight - heightMap[rowIndex][columnIndex]);
        minHeap.enqueue([
          Math.max(weight, heightMap[rowIndex][columnIndex]), rowIndex, columnIndex
        ]);
        history[rowIndex][columnIndex] = true;
      }
    });
  }

  return result;
};
```

0408 Valid Word Abbreviation

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 408. Valid Word Abbreviation
 * https://leetcode.com/problems/valid-word-abbreviation/
 * Difficulty: Easy
 *
 * A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings
 * with their lengths. The lengths should not have leading zeros.
 *
 * For example, a string such as "substitution" could be abbreviated as (but not limited to):
 * - "s10n" ("s ubstitutio n")
 * - "sub4u4" ("sub stit u tion")
 * - "12" ("substitution")
 * - "su3i1u2on" ("su bst i t u ti on")
 * - "substitution" (no substrings replaced)
 *
 * The following are not valid abbreviations:
 * - "s55n" ("s ubsti tutio n", the replaced substrings are adjacent)
 * - "s010n" (has leading zeros)
 * - "s0ubstitution" (replaces an empty substring)
 *
 * Given a string word and an abbreviation abbr, return whether the string matches the given
 * abbreviation.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {string} word
 * @param {string} abbr
 * @return {boolean}
 */
var validWordAbbreviation = function(word, abbr) {
  let wordIndex = 0;
  let abbrIndex = 0;

  while (wordIndex < word.length && abbrIndex < abbr.length) {
    if (word[wordIndex] === abbr[abbrIndex]) {
      wordIndex++;
      abbrIndex++;
      continue;
    }

    if (abbr[abbrIndex] < '0' || abbr[abbrIndex] > '9') return false;
    if (abbr[abbrIndex] === '0') return false;

    let num = 0;
    while (abbrIndex < abbr.length && /[0-9]/.test(abbr[abbrIndex])) {
      num = num * 10 + Number(abbr[abbrIndex]);
      abbrIndex++;
    }
    wordIndex += num;
  }

  return wordIndex === word.length && abbrIndex === abbr.length;
};
```

0409 Longest Palindrome

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
```

```
 * 409. Longest Palindrome
 * https://leetcode.com/problems/longest-palindrome/
 * Difficulty: Easy
 *
 * Given a string s which consists of lowercase or uppercase letters, return the length
 * of the longest palindrome that can be built with those letters.
 *
 * Letters are case sensitive, for example, "Aa" is not considered a palindrome.
 */

/**
 * @param {string} s
 * @return {number}
 */
var longestPalindrome = function(s) {
  const set = new Set();
  return [...s].reduce((count, c) => {
    return set.delete(c) ? count + 2 : (set.add(c), count);
  }, 0) + (set.size > 0 ? 1 : 0);
};
```

0410 Split Array Largest Sum

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 410. Split Array Largest Sum
 * https://leetcode.com/problems/split-array-largest-sum/
 * Difficulty: Hard
 *
 * Given an integer array nums and an integer k, split nums into k non-empty subarrays
 * such that the largest sum of any subarray is minimized.
 *
 * Return the minimized largest sum of the split.
 *
 * A subarray is a contiguous part of the array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var splitArray = function(nums, k) {
  let left = Math.max(...nums);
  let right = nums.reduce((a, b) => a + b);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    let count = 1;
    let sum = 0;

    for (const num of nums) {
      if (sum + num <= mid) {
        sum += num;
      } else {
        count++;
        sum = num;
      }
    }

    if (count > k) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
};
```

0411 Minimum Unique Word Abbreviation

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 411. Minimum Unique Word Abbreviation
 * https://leetcode.com/problems/minimum-unique-word-abbreviation/
 * Difficulty: Hard
 *
 * A string can be abbreviated by replacing any number of non-adjacent substrings with their
 * lengths. For example, a string such as "substitution" could be abbreviated as (but not
 * limited to):
 * - "s10n" ("s ubstitutio n")
 * - "sub4u4" ("sub stit u tion")
 * - "12" ("substitution")
 * - "su3i1u2on" ("su bst i t u ti on")
 * - "substitution" (no substrings replaced)
 *
 * Note that "s55n" ("s ubsti tutio n") is not a valid abbreviation of "substitution" because
 * the replaced substrings are adjacent.
 *
 * The length of an abbreviation is the number of letters that were not replaced plus the
 * number of substrings that were replaced. For example, the abbreviation "s10n" has a length
 * of 3 (2 letters + 1 substring) and "su3i1u2on" has a length of 9 (6 letters + 3
substrings).
 *
 * Given a target string target and an array of strings dictionary, return an abbreviation
 * of target with the shortest possible length such that it is not an abbreviation of any
 * string in dictionary. If there are multiple shortest abbreviations, return any of them.
 */

/**
 * @param {string} target
 * @param {string[]} dictionary
 * @return {string}
 */
var minAbbreviation = function(target, dictionary) {
  let minLength = target.length;
  let result = target;
  const validDict = dictionary.filter(word => word.length === target.length);

  for (let mask = 0; mask < (1 << target.length); mask++) {
    const abbr = getAbbr(target, mask);
    if (abbr.length <= minLength) {
      let isValid = true;
      for (const word of validDict) {
        if (conflicts(abbr, word)) {
          isValid = false;
          break;
        }
      }
      if (isValid) {
        if (abbr.length < minLength) {
          minLength = abbr.length;
          result = abbr;
        } else if (abbr.length === minLength && abbr < result) {
          result = abbr;
        }
      }
    }
  }

  return result;

  function getAbbr(str, mask) {
    let abbr = '';
    let count = 0;
    for (let i = 0; i < str.length; i++) {
      if (mask & (1 << i)) {
        if (count) {
```

```
          abbr += count;
          count = 0;
        }
        abbr += str[i];
      } else {
        count++;
      }
    }
    if (count) abbr += count;
    return abbr;
  }

  function conflicts(abbr, word) {
    let i = 0;
    let j = 0;
    while (i < abbr.length && j < word.length) {
      if (i < abbr.length && j < word.length && abbr[i] === word[j]) {
        i++;
        j++;
      } else if (i < abbr.length && /\d/.test(abbr[i])) {
        let num = 0;
        while (i < abbr.length && /\d/.test(abbr[i])) {
          num = num * 10 + Number(abbr[i++]);
        }
        j += num;
      } else {
        return false;
      }
    }
    return i === abbr.length && j === word.length;
  }
};
```

0412 Fizz Buzz

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 412. Fizz Buzz
 * https://leetcode.com/problems/fizz-buzz/
 * Difficulty: Easy
 *
 * Given an integer n, return a string array answer (1-indexed) where:
 * - answer[i] == "FizzBuzz" if i is divisible by 3 and 5.
 * - answer[i] == "Fizz" if i is divisible by 3.
 * - answer[i] == "Buzz" if i is divisible by 5.
 * - answer[i] == i (as a string) if none of the above conditions are true.
 */

/**
 * @param {number} n
 * @return {string[]}
 */
var fizzBuzz = function(n) {
  return Array.from(new Array(n), (_, i) => i + 1).map(i =>
    i % 3 === 0 && i % 5 === 0
      ? 'FizzBuzz' : i % 3 === 0 ? 'Fizz' : i % 5 === 0 ? 'Buzz' : String(i)
  );
};
```

0413 Arithmetic Slices

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 413. Arithmetic Slices
 * https://leetcode.com/problems/arithmetic-slices/
 * Difficulty: Medium
 *
 * An integer array is called arithmetic if it consists of at least three elements and if
 * the difference between any two consecutive elements is the same.
 *
 * For example, [1,3,5,7,9], [7,7,7,7], and [3,-1,-5,-9] are arithmetic sequences.
 *
 * Given an integer array nums, return the number of arithmetic subarrays of nums.
 *
 * A subarray is a contiguous subsequence of the array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var numberOfArithmeticSlices = function(nums) {
  if (nums.length < 3) return 0;
  let result = 0;

  for (let i = 2, value = 0; i < nums.length; i++) {
    if (nums[i] - nums[i - 1] === nums[i - 1] - nums[i - 2]) {
      value++;
      result += value;
    } else {
      value = 0;
    }
  }

  return result;
};
```

0414 Third Maximum Number

EASY

APPROACH NOTES

Uses a Set for unique membership. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 414. Third Maximum Number
 * https://leetcode.com/problems/third-maximum-number/
 * Difficulty: Easy
 *
 * Given an integer array nums, return the third distinct maximum number in this array.
 * If the third maximum does not exist, return the maximum number.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var thirdMax = function(nums) {
  const sortedSet = [...new Set(nums)].sort((a, b) => a - b);
  const thirdMax = sortedSet[sortedSet.length - 3];

  return thirdMax !== undefined
    ? thirdMax
    : sortedSet[sortedSet.length - 1];
};
```

0415 Add Strings

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 415. Add Strings
 * https://leetcode.com/problems/add-strings/
 * Difficulty: Easy
 *
 * Given two non-negative integers, num1 and num2 represented as string, return the sum of
num1 and
 * num2 as a string.
 *
 * You must solve the problem without using any built-in library for handling large integers
(such
 * as BigInteger). You must also not convert the inputs to integers directly.
 */

/**
 * @param {string} num1
 * @param {string} num2
 * @return {string}
 */
var addStrings = function(num1, num2) {
  let result = '';

  for (let i = num1.length - 1, j = num2.length - 1, carry = 0; i > -1 || j > -1 || carry ===
1;) {
    if (i > -1) {
      carry += num1.charCodeAt(i--) - 48;
    }
    if (j > -1) {
      carry += num2.charCodeAt(j--) - 48;
    }
    result = (carry % 10) + result;
    carry = Math.floor(carry / 10);
  }

  return result;
};
```

0416 Partition Equal Subset Sum

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 416. Partition Equal Subset Sum
 * https://leetcode.com/problems/partition-equal-subset-sum/
 * Difficulty: Medium
 *
 * Given an integer array nums, return true if you can partition the array into two subsets
 * such that the sum of the elements in both subsets is equal or false otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var canPartition = function(nums) {
  const sum = nums.reduce((a, b) => a + b);
  if (sum % 2) {
    return false;
  }

  const target = sum / 2;
  let set = new Set([0]);
  for (const n of nums) {
    const next = new Set(set);
    for (const value of set) {
      if (value + n === target) {
        return true;
      }
      next.add(value + n);
    }
    set = next;
```

```
  }

  return false;
};
```

0417 Pacific Atlantic Water Flow

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 417. Pacific Atlantic Water Flow
 * https://leetcode.com/problems/pacific-atlantic-water-flow/
 * Difficulty: Medium
 *
 * There is an m x n rectangular island that borders both the Pacific Ocean and Atlantic
Ocean.
 * The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches
 * the island's right and bottom edges.
 *
 * The island is partitioned into a grid of square cells. You are given an m x n integer
 * matrix heights where heights[r][c] represents the height above sea level of the cell
 * at coordinate (r, c).
 *
 * The island receives a lot of rain, and the rain water can flow to neighboring cells
directly
 * north, south, east, and west if the neighboring cell's height is less than or equal to the
 * current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.
 *
 * Return a 2D list of grid coordinates result where result[i] = [ri, ci] denotes that rain
water
 * can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.
 */

/**
 * @param {number[][]} heights
 * @return {number[][]}
 */
var pacificAtlantic = function(heights) {
  const result = [];
  const pacific = new Set();
  const atlantic = new Set();

  for (let i = 0; i < heights.length; i++) {
    dfs(i, 0, pacific);
    dfs(i, heights[0].length - 1, atlantic);
  }
  for (let j = 0; j < heights[0].length; j++) {
    dfs(0, j, pacific);
    dfs(heights.length - 1, j, atlantic);
  }

  for (let i = 0; i < heights.length; i++) {
    for (let j = 0; j < heights[0].length; j++) {
      const key = `${i},${j}`;
      if (pacific.has(key) && atlantic.has(key)) {
        result.push([i, j]);
      }
    }
  }

  return result;

  function dfs(r, c, ocean) {
    const key = `${r},${c}`;
    if (ocean.has(key)) return;
    ocean.add(key);

    [[r - 1, c], [r + 1, c], [r, c - 1], [r, c + 1]].forEach(([nr, nc]) => {
      if (nr >= 0 && nr < heights.length && nc >= 0 && nc < heights[0].length
```

```
        && heights[nr][nc] >= heights[r][c]) {
        dfs(nr, nc, ocean);
      }
    });
  }
};
```

0418 Sentence Screen Fitting

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 418. Sentence Screen Fitting
 * https://leetcode.com/problems/sentence-screen-fitting/
 * Difficulty: Medium
 *
 * Given a rows x cols screen and a sentence represented as a list of strings, return the
number
 * of times the given sentence can be fitted on the screen.
 *
 * The order of words in the sentence must remain unchanged, and a word cannot be split into
two
 * lines. A single space must separate two consecutive words in a line.
 */

/**
 * @param {string[]} sentence
 * @param {number} rows
 * @param {number} cols
 * @return {number}
 */
var wordsTyping = function(sentence, rows, cols) {
  const sentenceLength = sentence.length;
  const wordLengths = sentence.map(word => word.length + 1);
  let rowIndex = 0;
  let colIndex = 0;
  let wordIndex = 0;
  let result = 0;

  while (rowIndex < rows) {
    if (colIndex + wordLengths[wordIndex] - 1 <= cols) {
      colIndex += wordLengths[wordIndex];
      wordIndex++;
      if (wordIndex === sentenceLength) {
        result++;
        wordIndex = 0;
      }
    } else {
      rowIndex++;
      colIndex = 0;
    }
  }

  return result;
};
```

0419 Battleships In A Board

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 419. Battleships in a Board
 * https://leetcode.com/problems/battleships-in-a-board/
 * Difficulty: Medium
 *
```

```
 * Given an m x n matrix board where each cell is a battleship 'X' or empty '.', return the
 * number of the battleships on board.
 *
 * Battleships can only be placed horizontally or vertically on board. In other words, they
 * can only be made of the shape 1 x k (1 row, k columns) or k x 1 (k rows, 1 column), where
 * k can be of any size. At least one horizontal or vertical cell separates between two
 * battleships (i.e., there are no adjacent battleships).
 */

/**
 * @param {character[][]} boardx
 * @return {number}
 */
var countBattleships = function(board) {
  let count = 0;

  for (let i = 0; i < board.length; i++) {
    for (let j = 0; j < board[i].length; j++) {
      if (board[i][j] === 'X' && board[i][j - 1] !== 'X' && (!board[i - 1] || board[i - 1][j]
!== 'X')) {
        count++;
      }
    }
  }

  return count;
};
```

0420 Strong Password Checker

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 420. Strong Password Checker
 * https://leetcode.com/problems/strong-password-checker/
 * Difficulty: Hard
 *
 * A password is considered strong if the below conditions are all met:
 * - It has at least 6 characters and at most 20 characters.
 * - It contains at least one lowercase letter, at least one uppercase letter, and at
 *   least one digit.
 * - It does not contain three repeating characters in a row (i.e., "Baaabb0" is weak,
 *   but "Baaba0" is strong).
 *
 * Given a string password, return the minimum number of steps required to make password
 * strong. if password is already strong, return 0.
 *
 * In one step, you can:
 * - Insert one character to password,
 * - Delete one character from password, or
 * - Replace one character of password with another character.
 */

/**
 * @param {string} password
 * @return {number}
 */
var strongPasswordChecker = function(password) {
  const types = 3 - (/[a-z]/.test(password) + /[A-Z]/.test(password) +
/[0-9]/.test(password));
  let replacements = 0;
  let modOneCount = 0;
  let modTwoCount = 0;

  if (password.length < 6) {
    return Math.max(6 - password.length, types);
  }

  for (let i = 0; i < password.length;) {
    let j = i;
```

```
    while (j < password.length && password[j] === password[i]) {
      j++;
    }
    const repeatLength = j - i;
    if (repeatLength >= 3) {
      replacements += Math.floor(repeatLength / 3);
      if (repeatLength % 3 === 0) modOneCount++;
      if (repeatLength % 3 === 1) modTwoCount++;
    }
    i = j;
  }

  if (password.length <= 20) {
    return Math.max(types, replacements);
  }

  const deletions = password.length - 20;
  replacements -= Math.min(deletions, modOneCount);
  replacements -= Math.min(Math.max(deletions - modOneCount, 0), modTwoCount * 2) / 2;
  replacements -= Math.max(deletions - modOneCount - modTwoCount * 2, 0) / 3;

  return deletions + Math.max(types, Math.max(0, Math.ceil(replacements)));
};
```

0421 Maximum Xor Of Two Numbers In An Array

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 421. Maximum XOR of Two Numbers in an Array
 * https://leetcode.com/problems/maximum-xor-of-two-numbers-in-an-array/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the maximum result of nums[i] XOR
 * nums[j], where 0 <= i <= j < n.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMaximumXOR = function(nums) {
  let result = 0;

  for (let i = 31, mask = 0; i >= 0; i--) {
    mask |= (1 << i);
    const set = new Set(nums.map(num => num & mask));
    const target = result | (1 << i);
    for (const prefix of set) {
      if (set.has(prefix ^ target)) {
        result = target;
        break;
      }
    }
  }

  return result;
};
```

0422 Valid Word Square

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 422. Valid Word Square
```

```
 * https://leetcode.com/problems/valid-word-square/
 * Difficulty: Easy
 *
 * Given an array of strings words, return true if it forms a valid word square.
 *
 * A sequence of strings forms a valid word square if the kth row and column read the same
 * string, where 0 <= k < max(numRows, numColumns).
 */

/**
 * @param {string[]} words
 * @return {boolean}
 */
var validWordSquare = function(words) {
  const rowCount = words.length;

  for (let rowIndex = 0; rowIndex < rowCount; rowIndex++) {
    const row = words[rowIndex];
    for (let colIndex = 0; colIndex < row.length; colIndex++) {
      if (colIndex >= rowCount || rowIndex >= words[colIndex].length
          || row[colIndex] !== words[colIndex][rowIndex]) {
        return false;
      }
    }
  }

  return true;
};
```

0423 Reconstruct Original Digits From English

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 423. Reconstruct Original Digits from English
 * https://leetcode.com/problems/reconstruct-original-digits-from-english/
 * Difficulty: Medium
 *
 * Given a string s containing an out-of-order English representation of digits 0-9,
 * return the digits in ascending order.
 */

/**
 * @param {string} s
 * @return {string}
 */
var originalDigits = function(s) {
  const c = new Array(26).fill(0);
  const d = new Array(10).fill(0);

  for (const char of s) c[char.charCodeAt(0) - 97]++;

  [d[0], d[2], d[4], d[6], d[8]] = [c[25], c[22], c[20], c[23], c[6]];
  d[3] = c[7] - d[8];
  d[5] = c[5] - d[4];
  d[7] = c[18] - d[6];
  d[1] = c[14] - d[0] - d[2] - d[4];
  d[9] = (c[13] - d[1] - d[7]) / 2;

  return d.reduce((str, count, i) => str + String(i).repeat(count), '');
};
```

0424 Longest Repeating Character Replacement

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 424. Longest Repeating Character Replacement
 * https://leetcode.com/problems/longest-repeating-character-replacement/
 * Difficulty: Medium
 *
 * You are given a string s and an integer k. You can choose any character of the string
 * and change it to any other uppercase English character. You can perform this operation
 * at most k times.
 *
 * Return the length of the longest substring containing the same letter you can get after
 * performing the above operations.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var characterReplacement = function(s, k) {
  const count = new Map();
  let max = 0;
  let left = 0;

  return s.split('').reduce((maxLength, char, right) => {
    count.set(char, (count.get(char) || 0) + 1);
    max = Math.max(max, count.get(char));
    if (right - left + 1 - max > k) {
      count.set(s[left], count.get(s[left++]) - 1);
    }
    return Math.max(maxLength, right - left + 1);
  }, 0);
};
```

0425 Word Squares

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 425. Word Squares
 * https://leetcode.com/problems/word-squares/
 * Difficulty: Hard
 *
 * Given an array of unique strings words, return all the word squares you can build from
 * words. The same word from words can be used multiple times. You can return the answer
 * in any order.
 *
 * A sequence of strings forms a valid word square if the kth row and column read the same
 * string, where 0 <= k < max(numRows, numColumns).
 *
 * - For example, the word sequence ["ball","area","lead","lady"] forms a word square because
 *   each word reads the same both horizontally and vertically.
 */

/**
 * @param {string[]} words
 * @return {string[][]}
 */
var wordSquares = function(words) {
  const result = [];
  const prefixMap = new Map();
  const wordLength = words[0].length;

  for (const word of words) {
    for (let i = 0; i < word.length; i++) {
      const prefix = word.slice(0, i);
      if (!prefixMap.has(prefix)) {
        prefixMap.set(prefix, []);
      }
      prefixMap.get(prefix).push(word);
```

```
    }
  }

  function buildSquare(currentSquare) {
    if (currentSquare.length === wordLength) {
      result.push([...currentSquare]);
      return;
    }

    const prefix = currentSquare
      .map(word => word[currentSquare.length])
      .join('');

    const candidates = prefixMap.get(prefix) || [];
    for (const candidate of candidates) {
      currentSquare.push(candidate);
      buildSquare(currentSquare);
      currentSquare.pop();
    }
  }

  for (const word of words) {
    buildSquare([word]);
  }

  return result;
};
```

0426 Convert Binary Search Tree To Sorted Doubly Linked List

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 426. Convert Binary Search Tree to Sorted Doubly Linked List
 * https://leetcode.com/problems/convert-binary-search-tree-to-sorted-doubly-linked-list/
 * Difficulty: Medium
 *
 * Convert a Binary Search Tree to a sorted Circular Doubly-Linked List in place.
 *
 * You can think of the left and right pointers as synonymous to the predecessor and successor
 * pointers in a doubly-linked list. For a circular doubly linked list, the predecessor of the
 * first element is the last element, and the successor of the last element is the first
element.
 *
 * We want to do the transformation in place. After the transformation, the left pointer of
the
 * tree node should point to its predecessor, and the right pointer should point to its
successor.
 * You should return the pointer to the smallest element of the linked list.
 */

/**
 * // Definition for a _Node.
 * function _Node(val, left, right) {
 *      this.val = val;
 *      this.left = left;
 *      this.right = right;
 *  };
 */

/**
 * @param {_Node} root
 * @return {_Node}
 */
var treeToDoublyList = function(root) {
  if (!root) return null;

  let first = null;
  let last = null;
```

```
  inorder(root);

  last.right = first;
  first.left = last;

  return first;

  function inorder(node) {
    if (!node) return;

    inorder(node.left);

    if (last) {
      last.right = node;
      node.left = last;
    } else {
      first = node;
    }
    last = node;

    inorder(node.right);
  }
};
```

0427 Construct Quad Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 427. Construct Quad Tree
 * https://leetcode.com/problems/construct-quad-tree/
 * Difficulty: Medium
 *
 * Given a n * n matrix grid of 0's and 1's only. We want to represent grid with a Quad-Tree.
 *
 * Return the root of the Quad-Tree representing grid.
 *
 * A Quad-Tree is a tree data structure in which each internal node has exactly four children.
 * Besides, each node has two attributes:
 * - val: True if the node represents a grid of 1's or False if the node represents a grid of
 *   0's. Notice that you can assign the val to True or False when isLeaf is False, and both
are
 *   accepted in the answer.
 * - isLeaf: True if the node is a leaf node on the tree or False if the node has four
children.
 */

/**
 * // Definition for a QuadTree node.
 * function _Node(val,isLeaf,topLeft,topRight,bottomLeft,bottomRight) {
 *    this.val = val;
 *    this.isLeaf = isLeaf;
 *    this.topLeft = topLeft;
 *    this.topRight = topRight;
 *    this.bottomLeft = bottomLeft;
 *    this.bottomRight = bottomRight;
 * };
 */

/**
 * @param {number[][]} grid
 * @return {_Node}
 */
var construct = function(grid) {
  return build(0, 0, grid.length);

  function build(x, y, n) {
    const value = grid[x][y];
    for (let i = x; i < x + n; i++) {
      for (let j = y; j < y + n; j++) {
```

```
        if (grid[i][j] !== value) {
          n /= 2;
          return new _Node(true, false,
            build(x, y, n),
            build(x, y + n, n),
            build(x + n, y, n),
            build(x + n, y + n, n)
          );
        }
      }
    }
    return new _Node(value === 1, true, null, null, null, null);
  }
};
```

0428 Serialize And Deserialize N Ary Tree

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 428. Serialize and Deserialize N-ary Tree
 * https://leetcode.com/problems/serialize-and-deserialize-n-ary-tree/
 * Difficulty: Hard
 *
 * Serialization is the process of converting a data structure or object into a sequence of
 * bits so that it can be stored in a file or memory buffer, or transmitted across a network
 * connection link to be reconstructed later in the same or another computer environment.
 *
 * Design an algorithm to serialize and deserialize an N-ary tree. An N-ary tree is a rooted
 * tree in which each node has no more than N children. There is no restriction on how your
 * serialization/deserialization algorithm should work. You just need to ensure that an N-ary
 * tree can be serialized to a string and this string can be deserialized to the original
 * tree structure.
 *
 * For example, you may serialize the following 3-ary tree as [1 [3[5 6] 2 4]]. Note that
 * this is just an example, you do not necessarily need to follow this format.
 *
 * Or you can follow LeetCode's level order traversal serialization format, where each
 * group of children is separated by the null value.
 *
 * For example, the above tree may be serialized as
 * [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14].
 *
 * You do not necessarily need to follow the above-suggested formats, there are many more
 * different formats that work so please be creative and come up with different approaches
 * yourself.
 */

/**
 * // Definition for a _Node.
 * function _Node(val,children) {
 *    this.val = val;
 *    this.children = children;
 * };
 */

class Codec {
  constructor() {}

  serialize = function(root) {
    if (!root) return '';
    const result = [];

    function serializeNode(node) {
      result.push(node.val);
      result.push(node.children.length);
      for (const child of node.children) {
        serializeNode(child);
      }
    }
```

```
    serializeNode(root);
    return result.join(',');
  };

  deserialize = function(data) {
    if (!data) return null;
    const values = data.split(',').map(Number);
    let index = 0;

    function deserializeNode() {
      const val = values[index++];
      const childCount = values[index++];
      const children = [];
      for (let i = 0; i < childCount; i++) {
        children.push(deserializeNode());
      }
      return new _Node(val, children);
    }

    return deserializeNode();
  };
}
```

0429 N Ary Tree Level Order Traversal

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 429. N-ary Tree Level Order Traversal
 * https://leetcode.com/problems/n-ary-tree-level-order-traversal/
 * Difficulty: Medium
 *
 * Given an n-ary tree, return the level order traversal of its nodes' values.
 *
 * Nary-Tree input serialization is represented in their level order traversal, each group of
 * children is separated by the null value.
 */

/**
 * // Definition for a _Node.
 * function _Node(val,children) {
 *    this.val = val;
 *    this.children = children;
 * };
 */

/**
 * @param {_Node|null} root
 * @return {number[][]}
 */
var levelOrder = function(root) {
  if (!root) return [];

  const result = [];
  const queue = [root];

  while (queue.length) {
    const [level, current] = [queue.length, []];
    for (let i = 0; i < level; i++) {
      const node = queue.shift();
      current.push(node.val);
      queue.push(...node.children);
    }
    result.push(current);
  }

  return result;
};
```

0430 Flatten A Multilevel Doubly Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 430. Flatten a Multilevel Doubly Linked List
 * https://leetcode.com/problems/flatten-a-multilevel-doubly-linked-list/
 * Difficulty: Medium
 *
 * You are given a doubly linked list, which contains nodes that have a next pointer, a
previous
 * pointer, and an additional child pointer. This child pointer may or may not point to a
separate
 * doubly linked list, also containing these special nodes. These child lists may have one or
more
 * children of their own, and so on, to produce a multilevel data structure as shown in the
example
 * below.
 *
 * Given the head of the first level of the list, flatten the list so that all the nodes
appear in
 * a single-level, doubly linked list. Let curr be a node with a child list. The nodes in the
child
 * list should appear after curr and before curr.next in the flattened list.
 *
 * Return the head of the flattened list. The nodes in the list must have all of their child
 * pointers set to null.
 */

/**
 * // Definition for a _Node.
 * function _Node(val,prev,next,child) {
 *    this.val = val;
 *    this.prev = prev;
 *    this.next = next;
 *    this.child = child;
 * };
 */

/**
 * @param {_Node} head
 * @return {_Node}
 */
var flatten = function(head) {
  if (!head) return null;

  let current = head;
  while (current) {
    if (!current.child) {
      current = current.next;
    } else {
      const { child, next } = current;
      current.child = null;
      current.next = child;
      child.prev = current;

      let tail = child;
      while (tail.next) {
        tail = tail.next;
      }

      tail.next = next;
      if (next) {
        next.prev = tail;
      }
      current = current.next;
    }
  }

  return head;
```

```
};
```

0431 Encode N Ary Tree To Binary Tree

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 431. Encode N-ary Tree to Binary Tree
 * https://leetcode.com/problems/encode-n-ary-tree-to-binary-tree/
 * Difficulty: Hard
 *
 * Design an algorithm to encode an N-ary tree into a binary tree and decode the binary tree
 * to get the original N-ary tree. An N-ary tree is a rooted tree in which each node has no
 * more than N children. Similarly, a binary tree is a rooted tree in which each node has no
 * more than 2 children. There is no restriction on how your encode/decode algorithm should
 * work. You just need to ensure that an N-ary tree can be encoded to a binary tree and this
 * binary tree can be decoded to the original N-nary tree structure.
 *
 * Nary-Tree input serialization is represented in their level order traversal, each group
 * of children is separated by the null value (See following example).
 *
 * For example, you may encode the following 3-ary tree to a binary tree in this way:
 * - Input: root = [1,null,3,2,4,null,5,6]
 *
 * Note that the above is just an example which might or might not work. You do not
necessarily
 * need to follow this format, so please be creative and come up with different approaches
yourself.
 */

/**
 * // Definition for a _Node.
 * function _Node(val,children) {
 *    this.val = val;
 *    this.children = children;
 * };
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */

class Codec {
  constructor() {}

  encode = function(root) {
    if (!root) return null;

    const binaryRoot = new TreeNode(root.val);

    if (root.children.length > 0) {
      binaryRoot.left = this.encode(root.children[0]);
    }

    let sibling = binaryRoot.left;
    for (let i = 1; i < root.children.length; i++) {
      if (sibling) {
        sibling.right = this.encode(root.children[i]);
        sibling = sibling.right;
      }
    }

    return binaryRoot;
  };

  decode = function(root) {
```

```
    if (!root) return null;

    const updated = new _Node(root.val, []);
    let sibling = root.left;
    while (sibling) {
      updated.children.push(this.decode(sibling));
      sibling = sibling.right;
    }

    return updated;
  };
}
```

0432 All Oone Data Structure

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 432. All O`one Data Structure
 * https://leetcode.com/problems/all-oone-data-structure/
 * Difficulty: Hard
 *
 * Design a data structure to store the strings' count with the ability to return the strings
 * with minimum and maximum counts.
 *
 * Implement the AllOne class:
 * - AllOne() Initializes the object of the data structure.
 * - inc(String key) Increments the count of the string key by 1. If key does not exist in the
 *   data structure, insert it with count 1.
 * - dec(String key) Decrements the count of the string key by 1. If the count of key is 0
after
 *   the decrement, remove it from the data structure. It is guaranteed that key exists in the
 *   data structure before the decrement.
 * - getMaxKey() Returns one of the keys with the maximal count. If no element exists, return
 *   an empty string "".
 * - getMinKey() Returns one of the keys with the minimum count. If no element exists, return
 *   an empty string "".
 *
 * Note that each function must run in O(1) average time complexity.
 */

var AllOne = function() {
  this.map = new Map();
  this.head = { count: 0, keys: new Set(), prev: null, next: null };
  this.tail = { count: Infinity, keys: new Set(), prev: this.head, next: null };
  this.head.next = this.tail;
};

/**
 * @param {string} key
 * @return {void}
 */
AllOne.prototype.inc = function(key) {
  const node = this.map.get(key) || this.head;
  const count = node.count + 1;
  let next = node.next;

  if (next.count !== count) {
    next = { count, keys: new Set(), prev: node, next: node.next };
    node.next.prev = next;
    node.next = next;
  }

  next.keys.add(key);
  node.keys.delete(key);
  this.map.set(key, next);

  if (node !== this.head && node.keys.size === 0) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
```

```
  }
};

/**
 * @param {string} key
 * @return {void}
 */
AllOne.prototype.dec = function(key) {
  const node = this.map.get(key);
  const count = node.count - 1;

  node.keys.delete(key);

  if (count === 0) {
    this.map.delete(key);
  } else {
    let prev = node.prev;
    if (prev.count !== count) {
      prev = { count, keys: new Set(), prev: node.prev, next: node };
      node.prev.next = prev;
      node.prev = prev;
    }
    prev.keys.add(key);
    this.map.set(key, prev);
  }

  if (node.keys.size === 0) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
  }
};

/**
 * @return {string}
 */
AllOne.prototype.getMaxKey = function() {
  const node = this.tail.prev;
  return node === this.head ? '' : node.keys.values().next().value;
};

/**
 * @return {string}
 */
AllOne.prototype.getMinKey = function() {
  const node = this.head.next;
  return node === this.tail ? '' : node.keys.values().next().value;
};
```

0433 Minimum Genetic Mutation

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 433. Minimum Genetic Mutation
 * https://leetcode.com/problems/minimum-genetic-mutation/
 * Difficulty: Medium
 *
 * A gene string can be represented by an 8-character long string, with choices from
 * 'A', 'C', 'G', and 'T'.
 *
 * Suppose we need to investigate a mutation from a gene string startGene to a gene
 * string endGene where one mutation is defined as one single character changed in the
 * gene string.
 *
 * For example, "AACCGGTT" --> "AACCGGTA" is one mutation.
 *
 * There is also a gene bank bank that records all the valid gene mutations. A gene must
 * be in bank to make it a valid gene string.
 *
```

```
 * Given the two gene strings startGene and endGene and the gene bank bank, return the
 * minimum number of mutations needed to mutate from startGene to endGene. If there is
 * no such a mutation, return -1.
 *
 * Note that the starting point is assumed to be valid, so it might not be included in
 * the bank.
 */

/**
 * @param {string} startGene
 * @param {string} endGene
 * @param {string[]} bank
 * @return {number}
 */
var minMutation = function(startGene, endGene, bank) {
  const set = new Set(bank);
  if (!set.has(endGene)) return -1;

  const queue = [[startGene, 0]];
  const seen = new Set([startGene]);

  while (queue.length) {
    const [gene, steps] = queue.shift();
    if (gene === endGene) return steps;

    for (let i = 0; i < 8; i++) {
      for (const char of ['A', 'C', 'G', 'T']) {
        if (char !== gene[i]) {
          const next = gene.slice(0, i) + char + gene.slice(i + 1);
          if (set.has(next) && !seen.has(next)) {
            queue.push([next, steps + 1]);
            seen.add(next);
          }
        }
      }
    }
  }

  return -1;
};
```

0434 Number Of Segments In A String

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 434. Number of Segments in a String
 * https://leetcode.com/problems/number-of-segments-in-a-string/
 * Difficulty: Easy
 *
 * Given a string s, return the number of segments in the string.
 *
 * A segment is defined to be a contiguous sequence of non-space characters.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countSegments = function(s) {
  return s.split(/\s+/).filter(s => s).length;
};
```

0435 Non Overlapping Intervals

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 435. Non-overlapping Intervals
 * https://leetcode.com/problems/non-overlapping-intervals/
 * Difficulty: Medium
 *
 * Given an array of intervals intervals where intervals[i] = [starti, endi], return the
 * minimum number of intervals you need to remove to make the rest of the intervals
 * non-overlapping.
 *
 * Note that intervals which only touch at a point are non-overlapping. For example,
 * [1, 2] and [2, 3] are non-overlapping.
 */

/**
 * @param {number[][]} intervals
 * @return {number}
 */
var eraseOverlapIntervals = function(intervals) {
  let count = 0;

  intervals.sort((a, b) => a[1] - b[1]);
  let prevEnd = intervals[0][1];

  for (let i = 1; i < intervals.length; i++) {
    const [start, end] = intervals[i];
    if (prevEnd > start) count++;
    else prevEnd = end;
  }

  return count;
};
```

0436 Find Right Interval

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 436. Find Right Interval
 * https://leetcode.com/problems/find-right-interval/
 * Difficulty: Medium
 *
 * You are given an array of intervals, where intervals[i] = [starti, endi] and each
 * starti is unique.
 *
 * The right interval for an interval i is an interval j such that startj >= endi and
 * startj is minimized. Note that i may equal j.
 *
 * Return an array of right interval indices for each interval i. If no right interval
 * exists for interval i, then put -1 at index i.
 */

/**
 * @param {number[][]} intervals
 * @return {number[]}
 */
var findRightInterval = function(intervals) {
  const lookup = intervals.map(([start], i) => [start, i])
    .sort((a, b) => a[0] - b[0]);
  const result = new Array(intervals.length);

  for (let i = 0; i < intervals.length; i++) {
    const target = intervals[i][1];
    let left = 0;
    let right = lookup.length - 1;

    while (left <= right) {
```

```
      const middle = Math.floor((left + right) / 2);
      if (lookup[middle][0] >= target) {
        right = middle - 1;
      } else {
        left = middle + 1;
      }
    }

    result[i] = left < lookup.length ? lookup[left][1] : -1;
  }

  return result;
};
```

0437 Path Sum Iii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 437. Path Sum III
 * https://leetcode.com/problems/path-sum-iii/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and an integer targetSum, return the number of paths where
the
 * sum of the values along the path equals targetSum.
 *
 * The path does not need to start or end at the root or a leaf, but it must go downwards
 * (i.e., traveling only from parent nodes to child nodes).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} sum
 * @return {number}
 */
var pathSum = function(root, sum) {
  if (!root) return 0;
  return (subPathSum(root, sum) + pathSum(root.left, sum) + pathSum(root.right, sum));
};

function subPathSum(node, sum) {
  if (!node) return 0;
  const self = node.val === sum ? 1 : 0;
  return self + subPathSum(node.left, sum - node.val) + subPathSum(node.right, sum -
node.val);
}
```

0438 Find All Anagrams In A String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 438. Find All Anagrams in a String
 * https://leetcode.com/problems/find-all-anagrams-in-a-string/
 * Difficulty: Medium
 *
```

```
 * Given two strings s and p, return an array of all the start indices of p's anagrams in s.
 *
 * You may return the answer in any order.
 */

/**
 * @param {string} s
 * @param {string} p
 * @return {number[]}
 */
var findAnagrams = function(s, p) {
  const group = new Array(26).fill(0);
  const result = [];

  for (let i = 0; i < p.length; i++) {
    group[p.charCodeAt(i) - 97]--;
  }

  outer: for (let i = 0; i < s.length; i++) {
    group[s.charCodeAt(i) - 97]++;
    if (i < p.length - 1) {
      continue;
    } else if (i > p.length - 1) {
      group[s.charCodeAt(i - p.length) - 97]--;
    }
    for (let j = 0; j < 26; j++) {
      if (group[j]) {
        continue outer;
      }
    }
    result.push(i + 1 - p.length);
  }

  return result;
};
```

0439 Ternary Expression Parser

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 439. Ternary Expression Parser
 * https://leetcode.com/problems/ternary-expression-parser/
 * Difficulty: Medium
 *
 * Given a string expression representing arbitrarily nested ternary expressions,
 * evaluate the expression, and return the result of it.
 *
 * You can always assume that the given expression is valid and only contains digits,
 * '?', ':', 'T', and 'F' where 'T' is true and 'F' is false. All the numbers in the
 * expression are one-digit numbers (i.e., in the range [0, 9]).
 *
 * The conditional expressions group right-to-left (as usual in most languages), and
 * the result of the expression will always evaluate to either a digit, 'T' or 'F'.
 */

/**
 * @param {string} expression
 * @return {string}
 */
var parseTernary = function(expression) {
  const stack = [];

  for (let i = expression.length - 1; i >= 0; i--) {
    const char = expression[i];
    if (char !== ':' && char !== '?') {
      stack.push(char);
    } else if (char === '?') {
      const condition = expression[i - 1];
      const trueValue = stack.pop();
```

```
      const falseValue = stack.pop();
      stack.push(condition === 'T' ? trueValue : falseValue);
      i--;
    }
  }

  return stack[0];
};
```

0440 K Th Smallest In Lexicographical Order

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 440. K-th Smallest in Lexicographical Order
 * https://leetcode.com/problems/k-th-smallest-in-lexicographical-order/
 * Difficulty: Hard
 *
 * Given two integers n and k, return the kth lexicographically smallest integer in
 * the range [1, n].
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var findKthNumber = function(n, k) {
  let result = 1;
  k--;

  while (k > 0) {
    let count = 0;
    for (let first = result, last = result + 1; first <= n; first *= 10, last *= 10) {
      count += Math.min(n + 1, last) - first;
    }

    if (count <= k) {
      result++;
      k -= count;
    } else {
      result *= 10;
      k--;
    }
  }

  return result;
};
```

0441 Arranging Coins

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 441. Arranging Coins
 * https://leetcode.com/problems/arranging-coins/
 * Difficulty: Easy
 *
 * You have n coins and you want to build a staircase with these coins. The staircase consists
 * of k rows where the ith row has exactly i coins. The last row of the staircase may be
incomplete.
 *
 * Given the integer n, return the number of complete rows of the staircase you will build.
 */
```

```
/**
 * @param {number} n
 * @return {number}
 */
var arrangeCoins = function(n) {
  let count = 0;
  for (; count <= n; count++) {
    n -= count;
  }
  return count - 1;
};
```

0442 Find All Duplicates In An Array

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 442. Find All Duplicates in an Array
 * https://leetcode.com/problems/find-all-duplicates-in-an-array/
 * Difficulty: Medium
 *
 * Given an integer array nums of length n where all the integers of nums
 * are in the range [1, n] and each integer appears once or twice, return
 * an array of all the integers that appears twice.
 *
 * You must write an algorithm that runs in O(n) time and uses only constant
 * extra space.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var findDuplicates = function(nums) {
  const result = [];

  nums.forEach((num, i) => {
    const key = Math.abs(num) - 1;
    if (nums[key] < 0) {
      result.push(Math.abs(num));
    } else {
      nums[key] *= -1;
    }
  });

  return result;
};
```

0443 String Compression

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 443. String Compression
 * https://leetcode.com/problems/string-compression/
 * Difficulty: Medium
 *
 * Given an array of characters chars, compress it using the following algorithm:
 *
 * Begin with an empty string s. For each group of consecutive repeating characters in chars:
 * - If the group's length is 1, append the character to s.
 * - Otherwise, append the character followed by the group's length.
 *
 * The compressed string s should not be returned separately, but instead, be stored in the
 * input character array chars. Note that group lengths that are 10 or longer will be split
```

```
 * into multiple characters in chars.
 *
 * After you are done modifying the input array, return the new length of the array.
 *
 * You must write an algorithm that uses only constant extra space.
 */

/**
 * @param {character[]} chars
 * @return {number}
 */
var compress = function(chars) {
  let pointer = 0;
  for (let i = 0; i < chars.length; i++) {
    const char = chars[i];
    let count = 0;
    while (i < chars.length && chars[i] === char) {
      count++;
      i++;
    }
    chars[pointer++] = char;
    if (count !== 1) {
      String(count).split('').forEach(n => chars[pointer++] = n);
    }
    i--;
  }
  chars.length = pointer;
  return pointer;
};
```

0444 Sequence Reconstruction

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 444. Sequence Reconstruction
 * https://leetcode.com/problems/sequence-reconstruction/
 * Difficulty: Medium
 *
 * You are given an integer array nums of length n where nums is a permutation of the
 * integers in the range [1, n]. You are also given a 2D integer array sequences where
 * sequences[i] is a subsequence of nums.
 *
 * Check if nums is the shortest possible and the only supersequence. The shortest
 * supersequence is a sequence with the shortest length and has all sequences[i] as
 * subsequences. There could be multiple valid supersequences for the given array sequences.
 * - For example, for sequences = [[1,2],[1,3]], there are two shortest supersequences,
 *   [1,2,3] and [1,3,2].
 * - While for sequences = [[1,2],[1,3],[1,2,3]], the only shortest supersequence possible
 *   is [1,2,3]. [1,2,3,4] is a possible supersequence but not the shortest.
 *
 * Return true if nums is the only shortest supersequence for sequences, or false otherwise.
 *
 * A subsequence is a sequence that can be derived from another sequence by deleting some or
 * no elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} sequences
 * @return {boolean}
 */
var sequenceReconstruction = function(nums, sequences) {
  const n = nums.length;
  const graph = new Map();
  const inDegree = new Array(n + 1).fill(0);

  for (let i = 1; i <= n; i++) {
    graph.set(i, []);
```

```
  }

  for (const seq of sequences) {
    for (let i = 1; i < seq.length; i++) {
      graph.get(seq[i - 1]).push(seq[i]);
      inDegree[seq[i]]++;
    }
  }

  const queue = [];
  for (let i = 1; i <= n; i++) {
    if (inDegree[i] === 0) {
      queue.push(i);
    }
  }

  if (queue.length !== 1) return false;

  const result = [];
  while (queue.length) {
    if (queue.length > 1) return false;
    const curr = queue.shift();
    result.push(curr);

    const nextNodes = graph.get(curr);
    for (const next of nextNodes) {
      inDegree[next]--;
      if (inDegree[next] === 0) {
        queue.push(next);
      }
    }
  }

  return result.length === n && result.every((val, i) => val === nums[i]);
};
```

0445 Add Two Numbers Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 445. Add Two Numbers II
 * https://leetcode.com/problems/add-two-numbers-ii/
 * Difficulty: Medium
 *
 * You are given two non-empty linked lists representing two non-negative integers. The most
 * significant digit comes first and each of their nodes contains a single digit. Add the two
 * numbers and return the sum as a linked list.
 *
 * You may assume the two numbers do not contain any leading zero, except the number 0 itself.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} l1
 * @param {ListNode} l2
 * @return {ListNode}
 */
var addTwoNumbers = function(l1, l2) {
  const s1 = [];
  const s2 = [];
  let carry = 0;
  let head = null;
```

```
  while (l1) {
    s1.push(l1.val);
    l1 = l1.next;
  }

  while (l2) {
    s2.push(l2.val);
    l2 = l2.next;
  }

  while (s1.length || s2.length || carry) {
    const sum = (s1.pop() || 0) + (s2.pop() || 0) + carry;
    carry = sum / 10 | 0;
    head = Object.assign(new ListNode(sum % 10), { next: head });
  }

  return head;
};
```

0446 Arithmetic Slices Ii Subsequence

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 446. Arithmetic Slices II - Subsequence
 * https://leetcode.com/problems/arithmetic-slices-ii-subsequence/
 * Difficulty: Hard
 *
 * Given an integer array nums, return the number of all the arithmetic subsequences of nums.
 *
 * A sequence of numbers is called arithmetic if it consists of at least three elements and if
 * the difference between any two consecutive elements is the same.
 *
 * - For example, [1, 3, 5, 7, 9], [7, 7, 7, 7], and [3, -1, -5, -9] are arithmetic sequences.
 * - For example, [1, 1, 2, 5, 7] is not an arithmetic sequence.
 *
 * A subsequence of an array is a sequence that can be formed by removing some elements
(possibly
 * none) of the array.
 *
 * - For example, [2,5,10] is a subsequence of [1,2,1,2,4,1,5,10].
 *
 * The test cases are generated so that the answer fits in 32-bit integer.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var numberOfArithmeticSlices = function(nums) {
  const dp = new Array(nums.length).fill().map(() => new Map());
  let result = 0;

  for (let i = 1; i < nums.length; i++) {
    for (let j = 0; j < i; j++) {
      const diff = nums[i] - nums[j];
      const prev = dp[j].get(diff) || 0;
      const current = (dp[i].get(diff) || 0) + prev + 1;
      dp[i].set(diff, current);
      result += prev;
    }
  }

  return result;
};
```

0447 Number Of Boomerangs

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 447. Number of Boomerangs
 * https://leetcode.com/problems/number-of-boomerangs/
 * Difficulty: Medium
 *
 * You are given n points in the plane that are all distinct, where points[i] = [xi, yi].
 * A boomerang is a tuple of points (i, j, k) such that the distance between i and j equals
 * the distance between i and k (the order of the tuple matters).
 *
 * Return the number of boomerangs.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var numberOfBoomerangs = function(points) {
  let count = 0;

  for (let i = 0; i < points.length; i++) {
    const distances = new Map();
    for (let j = 0; j < points.length; j++) {
      if (i === j) continue;
      const d = (points[i][0] - points[j][0]) ** 2 + (points[i][1] - points[j][1]) ** 2;
      distances.set(d, (distances.get(d) || 0) + 1);
    }
    for (const d of distances.values()) {
      if (d > 1) count += d * (d - 1);
    }
  }

  return count;
};
```

0448 Find All Numbers Disappeared In An Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 448. Find All Numbers Disappeared in an Array
 * https://leetcode.com/problems/find-all-numbers-disappeared-in-an-array/
 * Difficulty: Easy
 *
 * Given an array `nums` of `n` integers where `nums[i]` is in the
 * range `[1, n]`, return an array of all the integers in the
 * range `[1, n]` that do not appear in nums.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var findDisappearedNumbers = function(nums) {
  const result = [];

  for (let i = 0; i < nums.length;) {
    if (nums[nums[i] - 1] !== nums[i]) {
      [nums[nums[i] - 1], nums[i]] = [nums[i], nums[nums[i] - 1]];
    } else {
      i++;
    }
```

```
  }

  nums.forEach((v, i) => {
    if (v != i + 1) {
      result.push(i + 1);
    }
  });

  return result;
};
```

0449 Serialize And Deserialize Bst

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 449. Serialize and Deserialize BST
 * https://leetcode.com/problems/serialize-and-deserialize-bst/
 * Difficulty: Medium
 *
 * Serialization is converting a data structure or object into a sequence of bits so that it
 * can be stored in a file or memory buffer, or transmitted across a network connection link
 * to be reconstructed later in the same or another computer environment.
 *
 * Design an algorithm to serialize and deserialize a binary search tree. There is no
restriction
 * on how your serialization/deserialization algorithm should work. You need to ensure that a
 * binary search tree can be serialized to a string, and this string can be deserialized to
the
 * original tree structure.
 *
 * The encoded string should be as compact as possible.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */

/**
 * Encodes a tree to a single string.
 *
 * @param {TreeNode} root
 * @return {string}
 */
var serialize = function(root) {
  if (!root) return '';

  const result = [];
  preorder(root);
  return result.join(',');

  function preorder(node) {
    if (!node) return;
    result.push(node.val);
    preorder(node.left);
    preorder(node.right);
  }
};

/**
 * Decodes your encoded data to tree.
 *
 * @param {string} data
 * @return {TreeNode}
 */
var deserialize = function(data) {
```

```
  if (!data) return null;

  return buildBST(data.split(',').map(Number), 0, Infinity);

  function buildBST(input, min, max) {
    if (!input.length || input[0] < min || input[0] > max) return null;
    const val = input.shift();
    const root = new TreeNode(val);
    root.left = buildBST(input, min, val);
    root.right = buildBST(input, val, max);
    return root;
  }
};
```

0450 Delete Node In A Bst

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 450. Delete Node in a BST
 * https://leetcode.com/problems/delete-node-in-a-bst/
 * Difficulty: Medium
 *
 * Given a root node reference of a BST and a key, delete the node with the given key in
 * the BST. Return the root node reference (possibly updated) of the BST.
 *
 * Basically, the deletion can be divided into two stages:
 * 1. Search for a node to remove.
 * 2. If the node is found, delete the node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} key
 * @return {TreeNode}
 */
var deleteNode = function(root, key) {
  if (!root) return root;
  if (root.val < key) {
    root.right = deleteNode(root.right, key);
  } else if (root.val > key) {
    root.left = deleteNode(root.left, key);
  } else {
    if (root.left === null) {
      return root.right;
    } else if (root.right === null) {
      return root.left;
    } else {
      let node = root.right;
      while (node.left) {
        node = node.left;
      }
      node.left = root.left;
      return root.right;
    }
  }
  return root;
};
```

0451 Sort Characters By Frequency

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 451. Sort Characters By Frequency
 * https://leetcode.com/problems/sort-characters-by-frequency/
 * Difficulty: Medium
 *
 * Given a string, sort it in decreasing order
 * based on the frequency of characters.
 */

/**
 * @param {string} s
 * @return {string}
 */
var frequencySort = function(s) {
  const map = new Map();
  s.split('').forEach(char => {
    map.set(char, map.has(char) ? map.get(char) + 1 : 1);
  });

  return [...map]
    .sort((a, b) => b[1] - a[1])
    .map(entry => entry[0].repeat(entry[1]))
    .join('');
};
```

0452 Minimum Number Of Arrows To Burst Balloons

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 452. Minimum Number of Arrows to Burst Balloons
 * https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/
 * Difficulty: Medium
 *
 * There are some spherical balloons taped onto a flat wall that represents the XY-plane.
 * The balloons are represented as a 2D integer array points where points[i] = [xstart, xend]
 * denotes a balloon whose horizontal diameter stretches between xstart and xend. You do not
 * know the exact y-coordinates of the balloons.
 *
 * Arrows can be shot up directly vertically (in the positive y-direction) from different
points
 * along the x-axis. A balloon with xstart and xend is burst by an arrow shot at x if
 * x(start) <= x <= x(end). There is no limit to the number of arrows that can be shot. A shot
 * arrow keeps traveling up infinitely, bursting any balloons in its path.
 *
 * Given the array points, return the minimum number of arrows that must be shot to burst all
 * balloons.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var findMinArrowShots = function(points) {
  let result = 0;
  let i = 0;

  points.sort(([a], [b]) => a - b);

  while (i < points.length) {
    let [left, right] = points[i];
```

```
    i++;
    while (i < points.length && points[i][0] <= right && points[i][1] >= left) {
      left = Math.max(left, points[i][0]);
      right = Math.min(right, points[i][1]);
      i++;
    }
    result++;
  }

  return result;
};
```

0453 Minimum Moves To Equal Array Elements

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 453. Minimum Moves to Equal Array Elements
 * https://leetcode.com/problems/minimum-moves-to-equal-array-elements/
 * Difficulty: Medium
 *
 * Given an integer array nums of size n, return the minimum number of moves required to
 * make all array elements equal.
 *
 * In one move, you can increment n - 1 elements of the array by 1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minMoves = function(nums) {
  const min = Math.min(...nums);
  return nums.reduce((sum, num) => sum + num - min, 0);
};
```

0454 4Sum Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 454. 4Sum II
 * https://leetcode.com/problems/4sum-ii/
 * Difficulty: Medium
 *
 * Given four integer arrays nums1, nums2, nums3, and nums4 all of length n, return the number
 * of tuples (i, j, k, l) such that:
 * - 0 <= i, j, k, l < n
 * - nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @param {number[]} nums3
 * @param {number[]} nums4
 * @return {number}
 */
var fourSumCount = function(nums1, nums2, nums3, nums4) {
  const map = new Map();
  nums1.forEach(n1 => {
    nums2.forEach(n2 => map.set(n1 + n2, (map.get(n1 + n2) || 0) + 1));
  });
  return nums3.reduce((count, n3) => {
    return count + nums4.reduce((sum, n4) => sum + (map.get(-(n3 + n4)) || 0), 0);
```

```
  }, 0);
};
```

0455 Assign Cookies

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 455. Assign Cookies
 * https://leetcode.com/problems/assign-cookies/
 * Difficulty: Easy
 *
 * Assume you are an awesome parent and want to give your children some cookies. But, you
 * should give each child at most one cookie.
 *
 * Each child i has a greed factor g[i], which is the minimum size of a cookie that the
 * child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can
 * assign the cookie j to the child i, and the child i will be content. Your goal is to
 * maximize the number of your content children and output the maximum number.
 */

/**
 * @param {number[]} g
 * @param {number[]} s
 * @return {number}
 */
var findContentChildren = function(g, s) {
  g.sort((a, b) => a - b);
  s.sort((a, b) => a - b);

  let result = 0;
  for (let j = 0; result < g.length && j < s.length; j++) {
    if (s[j] >= g[result]) {
      result++;
    }
  }

  return result;
};
```

0456 132 Pattern

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 456. 132 Pattern
 * https://leetcode.com/problems/132-pattern/
 * Difficulty: Medium
 *
 * Given an array of n integers nums, a 132 pattern is a subsequence of three integers
nums[i],
 * nums[j] and nums[k] such that i < j < k and nums[i] < nums[k] < nums[j].
 *
 * Return true if there is a 132 pattern in nums, otherwise, return false.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var find132pattern = function(nums) {
  const stack = [];

  for (let i = nums.length - 1, j = -Infinity; i >= 0; i--) {
    while (nums[i] > stack[stack.length - 1]) {
```

```
      j = stack.pop();
    }
    if (nums[i] < j) {
      return true;
    }
    stack.push(nums[i]);
  }

  return false;
};
```

0457 Circular Array Loop

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 457. Circular Array Loop
 * https://leetcode.com/problems/circular-array-loop/
 * Difficulty: Medium
 *
 * You are playing a game involving a circular array of non-zero integers nums. Each nums[i]
 * denotes the number of indices forward/backward you must move if you are located at index i:
 * - If nums[i] is positive, move nums[i] steps forward, and
 * - If nums[i] is negative, move nums[i] steps backward.
 *
 * Since the array is circular, you may assume that moving forward from the last element puts
 * you on the first element, and moving backwards from the first element puts you on the last
 * element.
 *
 * A cycle in the array consists of a sequence of indices seq of length k where:
 * - Following the movement rules above results in the repeating index sequence seq[0] ->
seq[1]
 *   -> ... -> seq[k - 1] -> seq[0] -> ...
 * - Every nums[seq[j]] is either all positive or all negative.
 * - k > 1
 *
 * Return true if there is a cycle in nums, or false otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var circularArrayLoop = function(nums) {
  const n = nums.length;

  for (let i = 0; i < n; i++) {
    if (dfs(i, i, nums[i] > 0)) return true;
  }

  return false;

  function dfs(current, prev, flag) {
    if (nums[current] === 0) return current != prev;
    if (flag != (nums[current] > 0)) return false;
    const moves = nums[current];
    nums[current] = 0;
    const next = (current + (moves % n) + n) % n;
    const result = dfs(next, current, flag);
    nums[current] = moves;
    return result;
  }
};
```

0458 Poor Pigs

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 458. Poor Pigs
 * https://leetcode.com/problems/poor-pigs/
 * Difficulty: Hard
 *
 * There are buckets buckets of liquid, where exactly one of the buckets is poisonous. To
figure
 * out which one is poisonous, you feed some number of (poor) pigs the liquid to see whether
 * they will die or not. Unfortunately, you only have minutesToTest minutes to determine which
 * bucket is poisonous.
 *
 * You can feed the pigs according to these steps:
 * 1. Choose some live pigs to feed.
 * 2. For each pig, choose which buckets to feed it. The pig will consume all the chosen
buckets
 *    simultaneously and will take no time. Each pig can feed from any number of buckets, and
each
 *    bucket can be fed from by any number of pigs.
 * 3. Wait for minutesToDie minutes. You may not feed any other pigs during this time.
 * 4. After minutesToDie minutes have passed, any pigs that have been fed the poisonous bucket
 *    will die, and all others will survive.
 * 5. Repeat this process until you run out of time.
 *
 * Given buckets, minutesToDie, and minutesToTest, return the minimum number of pigs needed to
 * figure out which bucket is poisonous within the allotted time.
 */

/**
 * @param {number} buckets
 * @param {number} minutesToDie
 * @param {number} minutesToTest
 * @return {number}
 */
var poorPigs = function(buckets, minutesToDie, minutesToTest) {
  const max = Math.floor(minutesToTest / minutesToDie) + 1;
  let result = 0;

  while (Math.pow(max, result) < buckets) {
    result++;
  }

  return result;
};
```

0459 Repeated Substring Pattern

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 459. Repeated Substring Pattern
 * https://leetcode.com/problems/repeated-substring-pattern/
 * Difficulty: Easy
 *
 * Given a non-empty string check if it can be constructed by taking a
 * substring of it and appending multiple copies of the substring
 * together. You may assume the given string consists of lowercase
 * English letters only and its length will not exceed 10000.
 */

/**
 * @param {string} s
```

```
 * @return {boolean}
 */
var repeatedSubstringPattern = function(s) {
  return (s + s).slice(1, s.length * 2 - 1).indexOf(s) !== -1;
};
```

0460 Lfu Cache

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 460. LFU Cache
 * https://leetcode.com/problems/lfu-cache/
 * Difficulty: Hard
 *
 * Design and implement a data structure for a Least Frequently Used (LFU) cache.
 *
 * Implement the LFUCache class:
 * - LFUCache(int capacity) Initializes the object with the capacity of the data structure.
 * - int get(int key) Gets the value of the key if the key exists in the cache. Otherwise,
 *   returns -1.
 * - void put(int key, int value) Update the value of the key if present, or inserts the
 *   key if not already present. When the cache reaches its capacity, it should invalidate
 *   and remove the least frequently used key before inserting a new item. For this problem,
 *   when there is a tie (i.e., two or more keys with the same frequency), the least recently
 *   used key would be invalidated.
 *
 * To determine the least frequently used key, a use counter is maintained for each key in
 * the cache. The key with the smallest use counter is the least frequently used key.
 *
 * When a key is first inserted into the cache, its use counter is set to 1 (due to the
 * put operation). The use counter for a key in the cache is incremented either a get or
 * put operation is called on it.
 *
 * The functions get and put must each run in O(1) average time complexity.
 */

/**
 * @param {number} capacity
 */
var LFUCache = function(capacity) {
  this.capacity = capacity;
  this.size = 0;
  this.minFreq = 0;
  this.values = new Map();
  this.freq = new Map();
  this.keys = new Map();
};

/**
 * @param {number} key
 * @return {number}
 */
LFUCache.prototype.get = function(key) {
  if (!this.values.has(key)) return -1;

  const freq = this.freq.get(key);
  this.freq.set(key, freq + 1);

  this.keys.get(freq).delete(key);
  if (!this.keys.get(freq).size) {
    this.keys.delete(freq);
    if (this.minFreq === freq) this.minFreq++;
  }

  if (!this.keys.has(freq + 1)) this.keys.set(freq + 1, new Set());
  this.keys.get(freq + 1).add(key);

  return this.values.get(key);
};
```

```
/**
 * @param {number} key
 * @param {number} value
 * @return {void}
 */
LFUCache.prototype.put = function(key, value) {
  if (this.capacity === 0) return;

  if (this.values.has(key)) {
    this.values.set(key, value);
    this.get(key);
    return;
  }

  if (this.size === this.capacity) {
    const keyToRemove = this.keys.get(this.minFreq).values().next().value;
    this.keys.get(this.minFreq).delete(keyToRemove);
    if (!this.keys.get(this.minFreq).size) {
      this.keys.delete(this.minFreq);
    }
    this.values.delete(keyToRemove);
    this.freq.delete(keyToRemove);
    this.size--;
  }

  this.values.set(key, value);
  this.freq.set(key, 1);
  if (!this.keys.has(1)) this.keys.set(1, new Set());
  this.keys.get(1).add(key);
  this.minFreq = 1;
  this.size++;
};
```

0461 Hamming Distance

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 461. Hamming Distance
 * https://leetcode.com/problems/hamming-distance/
 * Difficulty: Easy
 *
 * The Hamming distance between two integers is the number of positions at which the
corresponding
 * bits are different.
 *
 * Given two integers x and y, return the Hamming distance between them.
 */

/**
 * @param {number} x
 * @param {number} y
 * @return {number}
 */
var hammingDistance = function(x, y) {
  return (x ^ y).toString(2).replace(/0+/g, '').length;
};
```

0462 Minimum Moves To Equal Array Elements Ii

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 462. Minimum Moves to Equal Array Elements II
```

```
 * https://leetcode.com/problems/minimum-moves-to-equal-array-elements-ii/
 * Difficulty: Medium
 *
 * Given an integer array nums of size n, return the minimum number of moves
 * required to make all array elements equal.
 *
 * In one move, you can increment or decrement an element of the array by 1.
 *
 * Test cases are designed so that the answer will fit in a 32-bit integer.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minMoves2 = function(nums) {
  nums.sort((a, b) => a - b);
  const median = nums[Math.floor(nums.length / 2)];
  return nums.reduce((sum, num) => sum + Math.abs(num - median), 0);
};
```

0463 Island Perimeter

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 463. Island Perimeter
 * https://leetcode.com/problems/island-perimeter/
 * Difficulty: Medium
 *
 * You are given row x col grid representing a map where grid[i][j] = 1 represents land and
 * grid[i][j] = 0 represents water.
 *
 * Grid cells are connected horizontally/vertically (not diagonally). The grid is completely
 * surrounded by water, and there is exactly one island (i.e., one or more connected land
cells).
 *
 * The island doesn't have "lakes", meaning the water inside isn't connected to the water
around
 * the island. One cell is a square with side length 1. The grid is rectangular, width and
height
 * don't exceed 100. Determine the perimeter of the island.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var islandPerimeter = function(grid) {
  let island = 0;
  let adjacent = 0;

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[0].length; j++) {
      if (grid[i][j] === 1) {
        island++;
        if (i + 1 < grid.length && grid[i + 1][j] === 1) adjacent++;
        if (j + 1 < grid[0].length && grid[i][j + 1] === 1) adjacent++;
      }
    }
  }

  return island * 4 - adjacent * 2;
};
```

0464 Can I Win

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 464. Can I Win
 * https://leetcode.com/problems/can-i-win/
 * Difficulty: Medium
 *
 * In the "100 game" two players take turns adding, to a running total, any integer from 1 to
10.
 * The player who first causes the running total to reach or exceed 100 wins.
 *
 * What if we change the game so that players cannot re-use integers?
 *
 * For example, two players might take turns drawing from a common pool of numbers from 1 to
15
 * without replacement until they reach a total >= 100.
 *
 * Given two integers maxChoosableInteger and desiredTotal, return true if the first player to
 * move can force a win, otherwise, return false. Assume both players play optimally.
 */

/**
 * @param {number} maxChoosableInteger
 * @param {number} desiredTotal
 * @return {boolean}
 */
var canIWin = function(maxChoosableInteger, desiredTotal) {
  const memo = new Map();
  const sum = (maxChoosableInteger * (maxChoosableInteger + 1)) / 2;

  if (desiredTotal <= 0) return true;
  if (sum < desiredTotal) return false;

  return check(0, desiredTotal);

  function check(state, remaining) {
    if (remaining <= 0) return false;

    const key = state.toString();
    if (memo.has(key)) return memo.get(key);

    for (let i = 1; i <= maxChoosableInteger; i++) {
      const mask = 1 << i;
      if (!(state & mask) && (i >= remaining || !check(state | mask, remaining - i))) {
        memo.set(key, true);
        return true;
      }
    }

    memo.set(key, false);
    return false;
  }
};
```

0465 Optimal Account Balancing

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 465. Optimal Account Balancing
 * https://leetcode.com/problems/optimal-account-balancing/
 * Difficulty: Hard
 *
```

```
 * You are given an array of transactions transactions where
 * transactions[i] = [fromi, toi, amounti] indicates that the person with ID = fromi
 * gave amounti $ to the person with ID = toi.
 *
 * Return the minimum number of transactions required to settle the debt.
 */

/**
 * @param {number[][]} transactions
 * @return {number}
 */
var minTransfers = function(transactions) {
  const balances = new Array(12).fill(0);

  for (const [from, to, amount] of transactions) {
    balances[from] -= amount;
    balances[to] += amount;
  }

  const debts = balances.filter(balance => balance !== 0);
  return helper(0, 0);

  function helper(index, count) {
    if (index === debts.length) return count;

    if (debts[index] === 0) return helper(index + 1, count);

    let minTransactions = Infinity;
    const currentDebt = debts[index];

    for (let i = index + 1; i < debts.length; i++) {
      if (debts[i] * currentDebt < 0) {
        debts[i] += currentDebt;
        minTransactions = Math.min(minTransactions, helper(index + 1, count + 1));
        debts[i] -= currentDebt;
      }
    }

    return minTransactions;
  }
};
```

0466 Count The Repetitions

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 466. Count The Repetitions
 * https://leetcode.com/problems/count-the-repetitions/
 * Difficulty: Hard
 *
 * We define str = [s, n] as the string str which consists of the string s concatenated n
times.
 *
 * - For example, str == ["abc", 3] =="abcabcabc".
 *
 * We define that string s1 can be obtained from string s2 if we can remove some characters
from
 * s2 such that it becomes s1.
 *
 * - For example, s1 = "abc" can be obtained from s2 = "abdbec" based on our definition by
 *   removing the bolded underlined characters.
 *
 * You are given two strings s1 and s2 and two integers n1 and n2. You have the two strings
 * str1 = [s1, n1] and str2 = [s2, n2].
 *
 * Return the maximum integer m such that str = [str2, m] can be obtained from str1.
 */

/**
```

```
 * @param {string} s1
 * @param {number} n1
 * @param {string} s2
 * @param {number} n2
 * @return {number}
 */
var getMaxRepetitions = function(s1, n1, s2, n2) {
  const pattern = {};
  let s2Count = 0;
  let s2Index = 0;

  for (let i = 0; i < n1; i++) {
    for (let j = 0; j < s1.length; j++) {
      if (s1[j] === s2[s2Index]) {
        s2Index++;
        if (s2Index === s2.length) {
          s2Index = 0;
          s2Count++;
        }
      }
    }

    if (pattern[s2Index]) {
      const [prevIndex, prevCount] = pattern[s2Index];
      const remainingCycles = Math.floor((n1 - i - 1) / (i - prevIndex));
      i += remainingCycles * (i - prevIndex);
      s2Count += remainingCycles * (s2Count - prevCount);
    } else {
      pattern[s2Index] = [i, s2Count];
    }
  }

  return Math.floor(s2Count / n2);
};
```

0467 Unique Substrings In Wraparound String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 467. Unique Substrings in Wraparound String
 * https://leetcode.com/problems/unique-substrings-in-wraparound-string/
 * Difficulty: Medium
 *
 * We define the string base to be the infinite wraparound string of
"abcdefghijklmnopqrstuvwxyz",
 * so base will look like this:
 * - "...zabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcd....".
 *
 *  Given a string s, return the number of unique non-empty substrings of s are present in
base.
 */

/**
 * @param {string} s
 * @return {number}
 */
var findSubstringInWraproundString = function(s) {
  const maxLength = new Array(26).fill(0);
  let count = 0;

  for (let i = 0; i < s.length; i++) {
    count = i > 0 && (s.charCodeAt(i) - s.charCodeAt(i - 1) - 1) % 26 === 0 ? count + 1 : 1;
    const index = s.charCodeAt(i) - 97;
    maxLength[index] = Math.max(maxLength[index], count);
  }

  return maxLength.reduce((sum, n) => sum + n, 0);
};
```

0468 Validate Ip Address

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 468. Validate IP Address
 * https://leetcode.com/problems/validate-ip-address/
 * Difficulty: Medium
 *
 * Given a string queryIP, return "IPv4" if IP is a valid IPv4 address, "IPv6" if IP is a
valid
 * IPv6 address or "Neither" if IP is not a correct IP of any type.
 *
 * A valid IPv4 address is an IP in the form "x1.x2.x3.x4" where 0 <= xi <= 255 and xi cannot
 * contain leading zeros. For example, "192.168.1.1" and "192.168.1.0" are valid IPv4
addresses
 * while "192.168.01.1", "192.168.1.00", and "192.168@1.1" are invalid IPv4 addresses.
 *
 * A valid IPv6 address is an IP in the form "x1:x2:x3:x4:x5:x6:x7:x8" where:
 * - 1 <= xi.length <= 4
 * - xi is a hexadecimal string which may contain digits, lowercase English letter ('a' to
'f')
 *   and upper-case English letters ('A' to 'F').
 * - Leading zeros are allowed in xi.
 *
 * For example, "2001:0db8:85a3:0000:0000:8a2e:0370:7334" and
"2001:db8:85a3:0:0:8A2E:0370:7334"
 * are valid IPv6 addresses, while "2001:0db8:85a3::8A2E:037j:7334" and
 * "02001:0db8:85a3:0000:0000:8a2e:0370:7334" are invalid IPv6 addresses.
 */

/**
 * @param {string} queryIP
 * @return {string}
 */
var validIPAddress = function(queryIP) {
  const ipv4 =
/^((\d|[1-9]\d|1\d\d|2[0-4]\d|25[0-5])\.){3}(\d|[1-9]\d|1\d\d|2[0-4]\d|25[0-5])$/;
  const ipv6 = /^([\da-fA-F]{1,4}:){7}[\da-fA-F]{1,4}$/;

  return ipv4.test(queryIP) ? 'IPv4' : ipv6.test(queryIP) ? 'IPv6' : 'Neither';
};
```

0469 Convex Polygon

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 469. Convex Polygon
 * https://leetcode.com/problems/convex-polygon/
 * Difficulty: Medium
 *
 * You are given an array of points on the X-Y plane points where points[i] = [xi, yi].
 * The points form a polygon when joined sequentially.
 *
 * Return true if this polygon is convex and false otherwise.
 *
 * You may assume the polygon formed by given points is always a simple polygon. In other
 * words, we ensure that exactly two edges intersect at each vertex and that edges otherwise
 * don't intersect each other.
 */

/**
 * @param {number[][]} points
 * @return {boolean}
```

```
 */
var isConvex = function(points) {
  const n = points.length;
  let prevSign = 0;

  for (let i = 0; i < n; i++) {
    const p1 = points[i];
    const p2 = points[(i + 1) % n];
    const p3 = points[(i + 2) % n];
    const dx1 = p2[0] - p1[0];
    const dy1 = p2[1] - p1[1];
    const dx2 = p3[0] - p2[0];
    const dy2 = p3[1] - p2[1];
    const crossProduct = dx1 * dy2 - dy1 * dx2;
    const currentSign = Math.sign(crossProduct);

    if (currentSign !== 0) {
      if (prevSign !== 0 && currentSign !== prevSign) {
        return false;
      }
      prevSign = currentSign;
    }
  }

  return true;
};
```

0470 Implement Rand10 Using Rand7

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 470. Implement Rand10() Using Rand7()
 * https://leetcode.com/problems/implement-rand10-using-rand7/
 * Difficulty: Medium
 *
 * Given the API rand7() that generates a uniform random integer in the range [1, 7],
 * write a function rand10() that generates a uniform random integer in the range [1, 10].
 * You can only call the API rand7(), and you shouldn't call any other API. Please do not
 * use a language's built-in random API.
 *
 * Each test case will have one internal argument n, the number of times that your implemented
 * function rand10() will be called while testing. Note that this is not an argument passed to
 * rand10().
 */

/**
 * The rand7() API is already defined for you.
 * var rand7 = function() {}
 * @return {number} a random integer in the range 1 to 7
 */
var rand10 = function() {
  while (true) {
    const n = (rand7() - 1) * 7 + rand7();
    if (n <= 40) {
      return (n % 10) + 1;
    }
  }
};
```

0471 Encode String With Shortest Length

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 471. Encode String with Shortest Length
 * https://leetcode.com/problems/encode-string-with-shortest-length/
 * Difficulty: Hard
 *
 * Given a string s, encode the string such that its encoded length is the shortest.
 *
 * The encoding rule is: k[encoded_string], where the encoded_string inside the square
brackets
 * is being repeated exactly k times. k should be a positive integer.
 *
 * If an encoding process does not make the string shorter, then do not encode it. If there
are
 * several solutions, return any of them.
 */

/**
 * @param {string} s
 * @return {string}
 */
var encode = function(s) {
  const n = s.length;
  const dp = Array.from({ length: n }, () => new Array(n).fill(''));

  for (let len = 1; len <= n; len++) {
    for (let start = 0; start + len <= n; start++) {
      const end = start + len - 1;
      const substr = s.slice(start, end + 1);

      dp[start][end] = substr;

      for (let k = 1; k < len; k++) {
        const left = dp[start][start + k - 1];
        const right = dp[start + k][end];
        const combined = left + right;
        if (combined.length < dp[start][end].length) {
          dp[start][end] = combined;
        }
      }

      let repeat = 1;
      const patternLen = len;
      while (repeat * patternLen <= n && s.slice(start, start + patternLen).repeat(repeat)
        === s.slice(start, start + repeat * patternLen)) {
        const encoded = `${repeat}[${dp[start][start + patternLen - 1]}]`;
        if (encoded.length < dp[start][end].length) {
          dp[start][end] = encoded;
        }
        repeat++;
      }

      for (let k = 1; k < len; k++) {
        if (len % k === 0) {
          const pattern = s.slice(start, start + k);
          if (pattern.repeat(len / k) === substr) {
            const encoded = `${len / k}[${dp[start][start + k - 1]}]`;
            if (encoded.length < dp[start][end].length) {
              dp[start][end] = encoded;
            }
          }
        }
      }
    }
  }

  return dp[0][n - 1];
};
```

0472 Concatenated Words

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 472. Concatenated Words
 * https://leetcode.com/problems/concatenated-words/
 * Difficulty: Hard
 *
 * Given an array of strings words (without duplicates), return all the concatenated words
 * in the given list of words.
 *
 * A concatenated word is defined as a string that is comprised entirely of at least two
 * shorter words (not necessarily distinct) in the given array.
 */

/**
 * @param {string[]} words
 * @return {string[]}
 */
var findAllConcatenatedWordsInADict = function(words) {
  const set = new Set(words);
  const map = new Map();

  function isValid(word) {
    if (map.has(word)) return map.get(word);
    for (let i = 1; i < word.length; i++) {
      const suffix = word.slice(i);
      if (set.has(word.slice(0, i)) && (set.has(suffix) || isValid(suffix))) {
        map.set(word, true);
        return true;
      }
    }
    map.set(word, false);
    return false;
  }

  return words.filter(word => word.length > 0 && isValid(word));
};
```

0473 Matchsticks To Square

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 473. Matchsticks to Square
 * https://leetcode.com/problems/matchsticks-to-square/
 * Difficulty: Medium
 *
 * You are given an integer array matchsticks where matchsticks[i] is the length of the ith
 * matchstick. You want to use all the matchsticks to make one square. You should not break
 * any stick, but you can link them up, and each matchstick must be used exactly one time.
 *
 * Return true if you can make this square and false otherwise.
 */

/**
 * @param {number[]} matchsticks
 * @return {boolean}
 */
var makesquare = function(matchsticks) {
  if (matchsticks.length < 4) return false;
  const sum = matchsticks.reduce((a, b) => a + b);
  if (sum % 4) return false;

  const side = sum / 4;
  matchsticks.sort((a, b) => b - a);
  if (matchsticks[0] > side) return false;

  const sides = [0, 0, 0, 0];
  return (function backtrack(index) {
    if (index === matchsticks.length) return true;
    for (let i = 0; i < 4; i++) {
```

```
      if (sides[i] + matchsticks[index] <= side) {
        sides[i] += matchsticks[index];
        if (backtrack(index + 1)) return true;
        sides[i] -= matchsticks[index];
        if (!sides[i]) break;
      }
    }
    return false;
  })(0);
};
```

0474 Ones And Zeroes

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 474. Ones and Zeroes
 * https://leetcode.com/problems/ones-and-zeroes/
 * Difficulty: Medium
 *
 * You are given an array of binary strings strs and two integers m and n.
 *
 * Return the size of the largest subset of strs such that there are at most m 0's and n 1's
 * in the subset.
 *
 * A set x is a subset of a set y if all elements of x are also elements of y.
 */

/**
 * @param {string[]} strs
 * @param {number} m
 * @param {number} n
 * @return {number}
 */
var findMaxForm = function(strs, m, n) {
  const dp = Array.from({ length: m + 1 }, () => new Uint16Array(n + 1));
  for (const str of strs) {
    const zeros = (str.match(/0/g) || []).length;
    const ones = str.length - zeros;
    for (let i = m; i >= zeros; i--) {
      for (let j = n; j >= ones; j--) {
        dp[i][j] = Math.max(dp[i][j], dp[i - zeros][j - ones] + 1);
      }
    }
  }
  return dp[m][n];
};
```

0475 Heaters

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 475. Heaters
 * https://leetcode.com/problems/heaters/
 * Difficulty: Medium
 *
 * Winter is coming! During the contest, your first job is to design a standard heater with
 * a fixed warm radius to warm all the houses.
 *
 * Every house can be warmed, as long as the house is within the heater's warm radius range.
 *
 * Given the positions of houses and heaters on a horizontal line, return the minimum radius
 * standard of heaters so that those heaters could cover all houses.
```

```
 *
 * Notice that all the heaters follow your radius standard, and the warm radius will the same.
 */

/**
 * @param {number[]} houses
 * @param {number[]} heaters
 * @return {number}
 */
var findRadius = function(houses, heaters) {
  let result = 0;

  houses.sort((a, b) => a - b);
  heaters.sort((a, b) => a - b);
  for (let i = 0, j = 0; i < houses.length; i++) {
    while (j < heaters.length - 1 && houses[i] - heaters[j] > heaters[j + 1] - houses[i]) {
      j++;
    }
    result = Math.max(result, Math.abs(heaters[j] - houses[i]));
  }

  return result;
};
```

0476 Number Complement

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 476. Number Complement
 * https://leetcode.com/problems/number-complement/
 * Difficulty: Easy
 *
 * The complement of an integer is the integer you get when you flip all the 0's to 1's
 * and all the 1's to 0's in its binary representation.
 *
 * For example, The integer 5 is "101" in binary and its complement is "010" which is
 * the integer 2.
 *
 * Given an integer num, return its complement.
 */

/**
 * @param {number} num
 * @return {number}
 */
var findComplement = function(num) {
  return parseInt(num.toString(2).split('').map(n => 1 - n).join(''), 2);
};
```

0477 Total Hamming Distance

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 477. Total Hamming Distance
 * https://leetcode.com/problems/total-hamming-distance/
 * Difficulty: Medium
 *
 * The Hamming distance between two integers is the number of positions at which the
 * corresponding bits are different.
 *
 * Given an integer array nums, return the sum of Hamming distances between all the
 * pairs of the integers in nums.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var totalHammingDistance = function(nums) {
  let result = 0;

  for (let bit = 0; bit < 32; bit++) {
    const ones = nums.reduce((count, n) => count + ((n >> bit) & 1), 0);
    result += ones * (nums.length - ones);
  }

  return result;
};
```

0478 Generate Random Point In A Circle

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 478. Generate Random Point in a Circle
 * https://leetcode.com/problems/generate-random-point-in-a-circle/
 * Difficulty: Medium
 *
 * Given the radius and the position of the center of a circle, implement the function
randPoint
 * which generates a uniform random point inside the circle.
 *
 * Implement the Solution class:
 * - Solution(double radius, double x_center, double y_center) initializes the object with the
 *   radius of the circle radius and the position of the center (x_center, y_center).
 * - randPoint() returns a random point inside the circle. A point on the circumference of the
 *   circle is considered to be in the circle. The answer is returned as an array [x, y].
 */

/**
 * @param {number} radius
 * @param {number} x
 * @param {number} y
 */
var Solution = function(radius, x, y) {
  this.r = radius;
  this.x = x;
  this.y = y;
};

/**
 * @return {number[]}
 */
Solution.prototype.randPoint = function() {
  while (true) {
    const [x, y] = [Math.random() * 2 - 1, Math.random() * 2 - 1];
    if (x * x + y * y <= 1) {
      return [this.x + x * this.r, this.y + y * this.r];
    }
  }
};
```

0479 Largest Palindrome Product

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 479. Largest Palindrome Product
```

```
 * https://leetcode.com/problems/largest-palindrome-product/
 * Difficulty: Hard
 *
 * Given an integer n, return the largest palindromic integer that can be represented as the
 * product of two n-digits integers. Since the answer can be very large, return it modulo
1337.
 */

/**
 * @param {number} n
 * @return {number}
 */
var largestPalindrome = function(n) {
  if (n === 1) return 9;
  const max = 10 ** n - 1;
  for (let a = 1; a <= 9 * 10 ** (n - 1); a++) {
    const h = max - a + 1;
    const l = BigInt(String(h).split('').reverse().join(''));
    const x = BigInt(a) * BigInt(a) - 4n * l;
    if (x < 0) continue;
    const sqrtX = BigInt(Math.floor(Math.sqrt(Number(x))));
    if (sqrtX * sqrtX !== x && (sqrtX + 1n) * (sqrtX + 1n) !== x) continue;
    return Number((l + 10n ** BigInt(n) * BigInt(h)) % 1337n);
  }
};
```

0480 Sliding Window Median

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 480. Sliding Window Median
 * https://leetcode.com/problems/sliding-window-median/
 * Difficulty: Hard
 *
 * The median is the middle value in an ordered integer list. If the size of the list is even,
 * there is no middle value. So the median is the mean of the two middle values.
 *
 * - For examples, if arr = [2,3,4], the median is 3.
 * - For examples, if arr = [1,2,3,4], the median is (2 + 3) / 2 = 2.5.
 *
 * You are given an integer array nums and an integer k. There is a sliding window of size k
 * which is moving from the very left of the array to the very right. You can only see the k
 * numbers in the window. Each time the sliding window moves right by one position.
 *
 * Return the median array for each window in the original array. Answers within 10-5 of the
 * actual value will be accepted.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var medianSlidingWindow = function(nums, k) {
  const maxHeap = new MaxPriorityQueue();
  const minHeap = new MinPriorityQueue();
  const result = [];
  const map = {};

  for (let i = 0; i < k; i++) maxHeap.enqueue(nums[i]);
  for (let i = 0; i < k >> 1; i++) minHeap.enqueue(maxHeap.dequeue());
  for (let i = k; i <= nums.length; i++) {
    result.push(k & 1 ? maxHeap.front() : (maxHeap.front() + minHeap.front()) / 2);
    if (i === nums.length) break;
    map[nums[i - k]] = (map[nums[i - k]] || 0) + 1;
    const balance = (nums[i - k] <= maxHeap.front() ? -1 : 1)
      + (nums[i] <= maxHeap.front() ? 1 : -1);
    nums[i] <= maxHeap.front() ? maxHeap.enqueue(nums[i]) : minHeap.enqueue(nums[i]);
    balance < 0 && minHeap.size() && maxHeap.enqueue(minHeap.dequeue());
```

```
    balance > 0 && maxHeap.size() && minHeap.enqueue(maxHeap.dequeue());
    while (maxHeap.size() && map[maxHeap.front()]) map[maxHeap.dequeue()]--;
    while (minHeap.size() && map[minHeap.front()]) map[minHeap.dequeue()]--;
  }

  return result;
};
```

0481 Magical String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 481. Magical String
 * https://leetcode.com/problems/magical-string/
 * Difficulty: Medium
 *
 * A magical string s consists of only '1' and '2' and obeys the following rules:
 * - The string s is magical because concatenating the number of contiguous occurrences of
 *   characters '1' and '2' generates the string s itself.
 * - The first few elements of s is s = "1221121221221121122……". If we group the consecutive
 *   1's and 2's in s, it will be "1 22 11 2 1 22 1 22 11 2 11 22 ......" and the occurrences
 *   of 1's or 2's in each group are "1 2 2 1 1 2 1 2 2 1 2 2 ......". You can see that the
 *   occurrence sequence is s itself.
 *
 * Given an integer n, return the number of 1's in the first n number in the magical string s.
 */

/**
 * @param {number} n
 * @return {number}
 */
var magicalString = function(n) {
  if (n <= 0) return 0;
  if (n <= 3) return 1;

  const stack = [1, 2, 2];
  let result = 1;

  for (let i = 2, num = 1; stack.length < n; i++) {
    for (let j = 0; j < stack[i]; j++) {
      stack.push(num);
      if (num === 1 && stack.length <= n) {
        result++;
      }
    }
    num = num === 1 ? 2 : 1;
  }

  return result;
};
```

0482 License Key Formatting

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 482. License Key Formatting
 * https://leetcode.com/problems/license-key-formatting/
 * Difficulty: Easy
 *
 * You are given a license key represented as a string s that consists of only alphanumeric
 * characters and dashes. The string is separated into n + 1 groups by n dashes. You are
 * also given an integer k.
 *
```

```
 * We want to reformat the string s such that each group contains exactly k characters,
 * except for the first group, which could be shorter than k but still must contain at
 * least one character. Furthermore, there must be a dash inserted between two groups,
 * and you should convert all lowercase letters to uppercase.
 *
 * Return the reformatted license key.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var licenseKeyFormatting = function(s, k) {
  const result = [];
  s = s.toUpperCase().replace(/\-+/g, '');
  if (s.length % k) {
    result.push(s.substring(0, s.length % k));
  }
  for (let i = s.length % k; i < s.length; i += k) {
    result.push(s.substring(i, i + k));
  }
  return result.join('-');
};
```

0483 Smallest Good Base

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 483. Smallest Good Base
 * https://leetcode.com/problems/smallest-good-base/
 * Difficulty: Hard
 *
 * Given an integer n represented as a string, return the smallest good base of n.
 *
 * We call k >= 2 a good base of n, if all digits of n base k are 1's.
 */

/**
 * @param {string} n
 * @return {string}
 */
var smallestGoodBase = function(n) {
  const num = BigInt(n);
  for (let m = Math.floor(Math.log2(Number(n))); m >= 1; m--) {
    const k = BigInt(Math.floor(Number(n) ** (1 / m)));
    if (k < 2n) continue;
    const sum = (k ** BigInt(m + 1) - 1n) / (k - 1n);
    if (sum === num) return k.toString();
  }
  return (num - 1n).toString();
};
```

0484 Find Permutation

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 484. Find Permutation
 * https://leetcode.com/problems/find-permutation/
 * Difficulty: Medium
 *
 * A permutation perm of n integers of all the integers in the range [1, n] can be
 * represented as a string s of length n - 1 where:
```

```
 * - s[i] == 'I' if perm[i] < perm[i + 1], and
 * - s[i] == 'D' if perm[i] > perm[i + 1].
 *
 * Given a string s, reconstruct the lexicographically smallest permutation perm and return
it.
 */

/**
 * @param {string} s
 * @return {number[]}
 */
var findPermutation = function(s) {
  const n = s.length + 1;
  const result = new Array(n).fill(0).map((_, i) => i + 1);

  for (let i = 0; i < s.length; i++) {
    if (s[i] === 'D') {
      let start = i;
      while (i < s.length && s[i] === 'D') {
        i++;
      }
      let end = i;
      while (start < end) {
        [result[start], result[end]] = [result[end], result[start]];
        start++;
        end--;
      }
      i--;
    }
  }

  return result;
};
```

0485 Max Consecutive Ones

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 485. Max Consecutive Ones
 * https://leetcode.com/problems/max-consecutive-ones/
 * Difficulty: Easy
 *
 * Given a binary array nums, return the maximum number of consecutive 1's in the array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMaxConsecutiveOnes = function(nums) {
  let max = 0;
  for (let i = 0, count = 0; i < nums.length; i++) {
    count = nums[i] ? count + 1 : 0;
    max = Math.max(max, count);
  }
  return max;
};
```

0486 Predict The Winner

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 486. Predict the Winner
```

```
 * https://leetcode.com/problems/predict-the-winner/
 * Difficulty: Medium
 *
 * You are given an integer array nums. Two players are playing a game with this array:
 * player 1 and player 2.
 *
 * Player 1 and player 2 take turns, with player 1 starting first. Both players start
 * the game with a score of 0. At each turn, the player takes one of the numbers from
 * either end of the array (i.e., nums[0] or nums[nums.length - 1]) which reduces the
 * size of the array by 1. The player adds the chosen number to their score. The game
 * ends when there are no more elements in the array.
 *
 * Return true if Player 1 can win the game. If the scores of both players are equal,
 * then player 1 is still the winner, and you should also return true. You may assume
 * that both players are playing optimally.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var predictTheWinner = function(nums) {
  const diff = new Array(nums.length).fill(0);

  for (let i = nums.length - 1; i >= 0; i--) {
    diff[i] = nums[i];
    for (let j = i + 1; j < nums.length; j++) {
      diff[j] = Math.max(nums[i] - diff[j], nums[j] - diff[j - 1]);
    }
  }

  return diff[nums.length - 1] >= 0;
};
```

0487 Max Consecutive Ones Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 487. Max Consecutive Ones II
 * https://leetcode.com/problems/max-consecutive-ones-ii/
 * Difficulty: Medium
 *
 * Given a binary array nums, return the maximum number of consecutive 1's in the array
 * if you can flip at most one 0.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMaxConsecutiveOnes = function(nums) {
  let result = 0;
  let currentConsecutive = 0;
  let zeroCount = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    if (nums[right] === 0) {
      zeroCount++;
    }

    while (zeroCount > 1) {
      if (nums[left] === 0) {
        zeroCount--;
      }
      currentConsecutive--;
      left++;
    }
```

```
    currentConsecutive++;
    result = Math.max(result, currentConsecutive);
  }

  return result;
};
```

0488 Zuma Game

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 488. Zuma Game
 * https://leetcode.com/problems/zuma-game/
 * Difficulty: Hard
 *
 * You are playing a variation of the game Zuma.
 *
 * In this variation of Zuma, there is a single row of colored balls on a board, where each
ball
 * can be colored red 'R', yellow 'Y', blue 'B', green 'G', or white 'W'. You also have
several
 * colored balls in your hand.
 *
 * Your goal is to clear all of the balls from the board. On each turn:
 * - Pick any ball from your hand and insert it in between two balls in the row or on either
end
 *   of the row.
 * - If there is a group of three or more consecutive balls of the same color, remove the
group of
 *   balls from the board.
 *   - If this removal causes more groups of three or more of the same color to form, then
continue
 *     removing each group until there are none left.
 * - If there are no more balls on the board, then you win the game.
 * - Repeat this process until you either win or do not have any more balls in your hand.
 *
 * Given a string board, representing the row of balls on the board, and a string hand,
representing
 * the balls in your hand, return the minimum number of balls you have to insert to clear all
the
 * balls from the board. If you cannot clear all the balls from the board using the balls in
your
 * hand, return -1.
 */

/**
 * @param {string} board
 * @param {string} hand
 * @return {number}
 */
var findMinStep = function(board, hand) {
  const handCount = {};
  for (const ball of hand) {
    handCount[ball] = (handCount[ball] || 0) + 1;
  }

  const memo = new Map();

  function dfs(board, hand) {
    if (board.length === 0) return 0;
    if (Object.values(hand).every(count => count === 0)) return -1;
    const key = board + getHandString(hand);
    if (memo.has(key)) return memo.get(key);
    let minSteps = Infinity;

    for (const color in hand) {
      if (hand[color] <= 0) continue;
      hand[color]--;
```

```
      for (let i = 0; i <= board.length; i++) {
        if (i > 0 && i < board.length) {
          const left = board[i-1];
          const right = board[i];
          if (left !== color && right !== color && left !== right) continue;
        }
        const newBoard = board.slice(0, i) + color + board.slice(i);
        const afterRemove = removeConsecutive(newBoard);
        const steps = dfs(afterRemove, hand);
        if (steps !== -1) {
          minSteps = Math.min(minSteps, steps + 1);
        }
      }
      hand[color]++;
    }
    const result = minSteps === Infinity ? -1 : minSteps;
    memo.set(key, result);
    return result;
  };

  return dfs(board, handCount);
};

function getHandString(hand) {
  return Object.entries(hand)
    .filter(([_, count]) => count > 0)
    .sort(([a], [b]) => a.localeCompare(b))
    .map(([color, count]) => color + count)
    .join('');
}

function removeConsecutive(board) {
  let changed = true;
  while (changed) {
    changed = false;
    for (let i = 0; i < board.length;) {
      let j = i;
      while (j < board.length && board[j] === board[i]) {
        j++;
      }
      if (j - i >= 3) {
        board = board.substring(0, i) + board.substring(j);
        changed = true;
        i = 0;
      } else {
        i++;
      }
    }
  }
  return board;
}
```

0489 Robot Room Cleaner

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 489. Robot Room Cleaner
 * https://leetcode.com/problems/robot-room-cleaner/
 * Difficulty: Hard
 *
 * You are controlling a robot that is located somewhere in a room. The room is modeled as an
m x n
 * binary grid where 0 represents a wall and 1 represents an empty slot.
 *
 * The robot starts at an unknown location in the room that is guaranteed to be empty, and you
 do
 * not have access to the grid, but you can move the robot using the given API Robot.
 *
 * You are tasked to use the robot to clean the entire room (i.e., clean every empty cell in
```

```
the
 * room). The robot with the four given APIs can move forward, turn left, or turn right. Each
turn
 * is 90 degrees.
 *
 * When the robot tries to move into a wall cell, its bumper sensor detects the obstacle, and
it
 * stays on the current cell.
 *
 * Design an algorithm to clean the entire room using the following APIs:
 *
 * interface Robot {
 *   // returns true if next cell is open and robot moves into the cell.
 *   // returns false if next cell is obstacle and robot stays on the current cell.
 *   boolean move();
 *
 *   // Robot will stay on the same cell after calling turnLeft/turnRight.
 *   // Each turn will be 90 degrees.
 *   void turnLeft();
 *   void turnRight();
 *
 *   // Clean the current cell.
 *   void clean();
 * }
 *
 * Note that the initial direction of the robot will be facing up. You can assume all four
edges of
 * the grid are all surrounded by a wall.
 *
 * Custom testing:
 * The input is only given to initialize the room and the robot's position internally. You
must
 * solve this problem "blindfolded". In other words, you must control the robot using only the
 * four mentioned APIs without knowing the room layout and the initial robot's position.
 */

/**
 * @param {Robot} robot
 * @return {void}
 */
var cleanRoom = function(robot) {
  const visited = new Set();
  const directions = [[-1, 0], [0, 1], [1, 0], [0, -1]];

  backtrack(0, 0, 0);

  function backtrack(row, col, dir) {
    const key = `${row},${col}`;
    if (visited.has(key)) return;

    visited.add(key);
    robot.clean();

    for (let i = 0; i < 4; i++) {
      const newDir = (dir + i) % 4;
      const [dx, dy] = directions[newDir];
      const newRow = row + dx;
      const newCol = col + dy;

      if (robot.move()) {
        backtrack(newRow, newCol, newDir);
        robot.turnRight();
        robot.turnRight();
        robot.move();
        robot.turnLeft();
        robot.turnLeft();
      }
      robot.turnRight();
    }
  }
};
```

0490 The Maze

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 490. The Maze
 * https://leetcode.com/problems/the-maze/
 * Difficulty: Medium
 *
 * There is a ball in a maze with empty spaces (represented as 0) and walls (represented as
1).
 * The ball can go through the empty spaces by rolling up, down, left or right, but it won't
 * stop rolling until hitting a wall. When the ball stops, it could choose the next direction.
 *
 * Given the m x n maze, the ball's start position and the destination, where
 * start = [startrow, startcol] and destination = [destinationrow, destinationcol],
 * return true if the ball can stop at the destination, otherwise return false.
 *
 * You may assume that the borders of the maze are all walls (see examples).
 */

/**
 * @param {number[][]} maze
 * @param {number[]} start
 * @param {number[]} destination
 * @return {boolean}
 */
var hasPath = function(maze, start, destination) {
  const rows = maze.length;
  const cols = maze[0].length;
  const visited = new Set();
  const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];

  return helper(start[0], start[1]);

  function helper(row, col) {
    if (visited.has(`${row},${col}`)) return false;
    visited.add(`${row},${col}`);

    if (row === destination[0] && col === destination[1]) return true;

    for (const [dr, dc] of directions) {
      let newRow = row;
      let newCol = col;

      while (newRow + dr >= 0 && newRow + dr < rows && newCol + dc >= 0 && newCol + dc < cols
             && maze[newRow + dr][newCol + dc] === 0) {
        newRow += dr;
        newCol += dc;
      }

      if (helper(newRow, newCol)) return true;
    }

    return false;
  }
};
```

0491 Non Decreasing Subsequences

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 491. Non-decreasing Subsequences
 * https://leetcode.com/problems/non-decreasing-subsequences/
 * Difficulty: Medium
 *
 * Given an integer array nums, return all the different possible non-decreasing
 * subsequences of the given array with at least two elements. You may return
 * the answer in any order.
 */

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var findSubsequences = function(nums) {
  const result = new Set();
  backtrack(result, nums, 0, []);
  return Array.from(result).map(s => s.split(','));
};

function backtrack(result, nums, offset, order) {
  if (order.length > 1) {
    result.add(order.join());
  }
  for (let index = offset; index < nums.length; index++) {
    if (!(order[order.length - 1] > nums[index])) {
      backtrack(result, nums, index + 1, [...order, nums[index]]);
    }
  }
}
```

0492 Construct The Rectangle

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 492. Construct the Rectangle
 * https://leetcode.com/problems/construct-the-rectangle/
 * Difficulty: Easy
 *
 * A web developer needs to know how to design a web page's size. So, given
 * a specific rectangular web page’s area, your job by now is to design a
 * rectangular web page, whose length L and width W satisfy the following
 * requirements:
 *
 * The area of the rectangular web page you designed must equal to the given
 * target area. The width W should not be larger than the length L, which
 * means L >= W. The difference between length L and width W should be as
 * small as possible.
 *
 * Return an array [L, W] where L and W are the length and width of the web
 * page you designed in sequence.
 */

/**
 * @param {number} area
 * @return {number[]}
 */
var constructRectangle = function(area) {
  let width = Math.floor(Math.sqrt(area));
  while (area % width) {
    width--;
  }
  return [area / width, width];
};
```

0493 Reverse Pairs

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 493. Reverse Pairs
 * https://leetcode.com/problems/reverse-pairs/
 * Difficulty: Hard
 *
 * Given an integer array nums, return the number of reverse pairs in the array.
 *
 * A reverse pair is a pair (i, j) where:
 * - 0 <= i < j < nums.length and
 * - nums[i] > 2 * nums[j].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var reversePairs = function(nums) {
  let count = 0;
  mergeSort(nums, 0, nums.length - 1);
  return count;

  function mergeSort(input, left, right) {
    if (left >= right) return;
    const middle = Math.floor((left + right) / 2);
    mergeSort(input, left, middle);
    mergeSort(input, middle + 1, right);
    merge(input, left, middle, right);
  }

  function merge(input, left, middle, right) {
    let j = middle + 1;
    for (let i = left; i <= middle; i++) {
      while (j <= right && input[i] > 2 * input[j]) {
        j++;
      }
      count += j - (middle + 1);
    }

    const order = [];
    let i = left;
    j = middle + 1;
    while (i <= middle && j <= right) {
      if (input[i] <= input[j]) {
        order.push(input[i++]);
      } else {
        order.push(input[j++]);
      }
    }
    while (i <= middle) {
      order.push(input[i++]);
    }
    while (j <= right) {
      order.push(input[j++]);
    }
    for (let k = 0; k < order.length; k++) {
      input[left + k] = order[k];
    }
  }
};
```

0494 Target Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 494. Target Sum
 * https://leetcode.com/problems/target-sum/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer target.
 *
 * You want to build an expression out of nums by adding one of the symbols '+' and '-' before
 * each integer in nums and then concatenate all the integers.
 *
 * For example, if nums = [2, 1], you can add a '+' before 2 and a '-' before 1 and
concatenate
 * them to build the expression "+2-1".
 *
 * Return the number of different expressions that you can build, which evaluates to target.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var findTargetSumWays = function(nums, target) {
  const map = new Map();

  function dp(index, sum) {
    if (index === nums.length) {
      return sum === target ? 1 : 0;
    }
    const key = `${index},${sum}`;
    if (map.has(key)) {
      return map.get(key);
    }
    const r = dp(index + 1, sum + nums[index]) + dp(index + 1, sum - nums[index]);
    map.set(key, r);
    return r;
  }

  return dp(0, 0);
};
```

0495 Teemo Attacking

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 495. Teemo Attacking
 * https://leetcode.com/problems/teemo-attacking/
 * Difficulty: Easy
 *
 * Our hero Teemo is attacking an enemy Ashe with poison attacks! When Teemo attacks Ashe,
Ashe gets
 * poisoned for a exactly duration seconds. More formally, an attack at second t will mean
Ashe is
 * poisoned during the inclusive time interval [t, t + duration - 1]. If Teemo attacks again
before
 * the poison effect ends, the timer for it is reset, and the poison effect will end duration
 * seconds after the new attack.
 *
 * You are given a non-decreasing integer array timeSeries, where timeSeries[i] denotes that
Teemo
```

```
 * attacks Ashe at second timeSeries[i], and an integer duration.
 *
 * Return the total number of seconds that Ashe is poisoned.
 */

/**
 * @param {number[]} timeSeries
 * @param {number} duration
 * @return {number}
 */
var findPoisonedDuration = function(timeSeries, duration) {
  return timeSeries.reduce((sum, n, i) => {
    return sum + Math.min(duration, (timeSeries[i + 1] ?? n + duration) - n);
  }, 0);
};
```

0496 Next Greater Element I

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 496. Next Greater Element I
 * https://leetcode.com/problems/next-greater-element-i/
 * Difficulty: Easy
 *
 * The next greater element of some element x in an array is the first greater element
 * that is to the right of x in the same array.
 *
 * You are given two distinct 0-indexed integer arrays nums1 and nums2, where nums1 is
 * a subset of nums2.
 *
 * For each 0 <= i < nums1.length, find the index j such that nums1[i] == nums2[j] and
 * determine the next greater element of nums2[j] in nums2. If there is no next greater
 * element, then the answer for this query is -1.
 *
 * Return an array ans of length nums1.length such that ans[i] is the next greater
 * element as described above.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var nextGreaterElement = function(nums1, nums2) {
  return nums1.map(n => {
    let index = nums2.indexOf(n);
    while (nums2[index] <= n) {
      index++;
    }
    return index >= nums2.length ? -1 : nums2[index];
  });
};
```

0497 Random Point In Non Overlapping Rectangles

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 497. Random Point in Non-overlapping Rectangles
 * https://leetcode.com/problems/random-point-in-non-overlapping-rectangles/
 * Difficulty: Medium
 *
 * You are given an array of non-overlapping axis-aligned rectangles rects where
 * rects[i] = [ai, bi, xi, yi] indicates that (ai, bi) is the bottom-left corner point of the
```

```
 * ith rectangle and (xi, yi) is the top-right corner point of the ith rectangle. Design an
 * algorithm to pick a random integer point inside the space covered by one of the given
 * rectangles. A point on the perimeter of a rectangle is included in the space covered by
 * the rectangle.
 *
 * Any integer point inside the space covered by one of the given rectangles should be equally
 * likely to be returned.
 *
 * Note that an integer point is a point that has integer coordinates.
 *
 * Implement the Solution class:
 * - Solution(int[][] rects) Initializes the object with the given rectangles rects.
 * - int[] pick() Returns a random integer point [u, v] inside the space covered by one of the
 *   given rectangles.
 */

/**
 * @param {number[][]} rects
 */
var Solution = function(rects) {
  this.rects = rects;
  this.areas = rects.map(([x1, y1, x2, y2]) => (x2 - x1 + 1) * (y2 - y1 + 1));
  this.totalPoints = this.areas.reduce((sum, area) => sum + area, 0);
};

/**
 * @return {number[]}
 */
Solution.prototype.pick = function() {
  let random = Math.floor(Math.random() * this.totalPoints);
  let i = 0;
  while (random >= this.areas[i]) {
    random -= this.areas[i];
    i++;
  }
  const [x1, y1, x2, y2] = this.rects[i];
  const x = x1 + Math.floor(Math.random() * (x2 - x1 + 1));
  const y = y1 + Math.floor(Math.random() * (y2 - y1 + 1));
  return [x, y];
};
```

0498 Diagonal Traverse

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 498. Diagonal Traverse
 * https://leetcode.com/problems/diagonal-traverse/
 * Difficulty: Medium
 *
 * Given an m x n matrix mat, return an array of all the elements of the array in a diagonal
order.
 */

/**
 * @param {number[][]} mat
 * @return {number[]}
 */
var findDiagonalOrder = function(mat) {
  const result = new Array(mat.length * mat[0].length);

  for (let r = 0, c = 0, d = 1, i = 0; i < mat.length * mat[0].length;) {
    result[i++] = mat[r][c];

    if (d === 1) {
      if (c === mat[0].length - 1) {
        r++;
        d = -1;
      } else if (r === 0) {
        c++;
```

```
        d = -1;
      } else {
        r--;
        c++;
      }
    } else {
      if (r === mat.length - 1) {
        c++;
        d = 1;
      } else if (c === 0) {
        r++;
        d = 1;
      } else {
        r++;
        c--;
      }
    }
  }

  return result;
};
```

0499 The Maze Iii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 499. The Maze III
 * https://leetcode.com/problems/the-maze-iii/
 * Difficulty: Hard
 *
 * There is a ball in a maze with empty spaces (represented as 0) and walls (represented as
1).
 * The ball can go through the empty spaces by rolling up, down, left or right, but it won't
 * stop rolling until hitting a wall. When the ball stops, it could choose the next direction.
 * There is also a hole in this maze. The ball will drop into the hole if it rolls onto the
hole.
 *
 * Given the m x n maze, the ball's position ball and the hole's position hole, where
 * ball = [ballrow, ballcol] and hole = [holerow, holecol], return a string instructions of
 * all the instructions that the ball should follow to drop in the hole with the shortest
 * distance possible. If there are multiple valid instructions, return the lexicographically
 * minimum one. If the ball can't drop in the hole, return "impossible".
 *
 * If there is a way for the ball to drop in the hole, the answer instructions should contain
 * the characters 'u' (i.e., up), 'd' (i.e., down), 'l' (i.e., left), and 'r' (i.e., right).
 *
 * The distance is the number of empty spaces traveled by the ball from the start position
 * (excluded) to the destination (included).
 *
 * You may assume that the borders of the maze are all walls (see examples).
 */

var findShortestWay = function(maze, ball, hole) {
  const rows = maze.length;
  const cols = maze[0].length;
  const directions = [[-1, 0, 'u'], [1, 0, 'd'], [0, -1, 'l'], [0, 1, 'r']];
  const distances = new Array(rows).fill().map(() => new Array(cols).fill(Infinity));
  const paths = new Array(rows).fill().map(() => new Array(cols).fill(''));

  const pq = [[0, ball[0], ball[1], '']];
  distances[ball[0]][ball[1]] = 0;

  while (pq.length > 0) {
    pq.sort((a, b) => a[0] - b[0] || a[3].localeCompare(b[3]));
    const [currentDist, row, col, path] = pq.shift();

    if (row === hole[0] && col === hole[1]) {
      return path;
```

```
    }

    if (currentDist > distances[row][col]
        || (currentDist === distances[row][col] && path > paths[row][col])) {
      continue;
    }

    for (const [dr, dc, dir] of directions) {
      let newRow = row;
      let newCol = col;
      let steps = 0;

      while (newRow + dr >= 0 && newRow + dr < rows && newCol + dc >= 0 && newCol + dc < cols
            && maze[newRow + dr][newCol + dc] === 0) {
        newRow += dr;
        newCol += dc;
        steps++;

        if (newRow === hole[0] && newCol === hole[1]) {
          break;
        }
      }

      if (steps > 0) {
        const newDist = currentDist + steps;
        const newPath = path + dir;

        if (newDist < distances[newRow][newCol]
          || (newDist === distances[newRow][newCol] && newPath < paths[newRow][newCol])) {
          distances[newRow][newCol] = newDist;
          paths[newRow][newCol] = newPath;
          pq.push([newDist, newRow, newCol, newPath]);
        }
      }
    }
  }

  return 'impossible';
};
```

0500 Keyboard Row

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 500. Keyboard Row
 * https://leetcode.com/problems/keyboard-row/
 * Difficulty: Easy
 *
 * Given an array of strings words, return the words that can be typed using letters
 * of the alphabet on only one row of American keyboard like the image below.
 *
 * In the American keyboard:
 * - the first row consists of the characters "qwertyuiop",
 * - the second row consists of the characters "asdfghjkl", and
 * - the third row consists of the characters "zxcvbnm".
 */

/**
 * @param {string[]} words
 * @return {string[]}
 */
var findWords = function(words) {
  const rows = ['qwertyuiop', 'asdfghjkl', 'zxcvbnm'];
  return words.filter(word => rows.some(row => word.toLowerCase().split('').every(s =>
row.includes(s))));
};
```

0501 Find Mode In Binary Search Tree

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 501. Find Mode in Binary Search Tree
 * https://leetcode.com/problems/find-mode-in-binary-search-tree/
 * Difficulty: Easy
 *
 * Given the root of a binary search tree (BST) with duplicates, return all
 * the mode(s) (i.e., the most frequently occurred element) in it.
 *
 * If the tree has more than one mode, return them in any order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var findMode = function(root) {
  const map = new Map();
  dfs(root);

  function dfs(root) {
    if (!root) return 0;
    map.set(root.val, (map.get(root.val) || 0) + 1);
    [root.left, root.right].forEach(dfs);
  }

  const max = Math.max(...map.values());
  return Array.from(map.keys()).filter(key => map.get(key) === max);
};
```

0502 Ipo

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 502. IPO
 * https://leetcode.com/problems/ipo/
 * Difficulty: Hard
 *
 * Suppose LeetCode will start its IPO soon. In order to sell a good price of
 * its shares to Venture Capital, LeetCode would like to work on some projects
 * to increase its capital before the IPO. Since it has limited resources, it
 * can only finish at most k distinct projects before the IPO. Help LeetCode
 * design the best way to maximize its total capital after finishing at most
 * k distinct projects.
 *
 * You are given n projects where the ith project has a pure profit profits[i]
 * and a minimum capital of capital[i] is needed to start it.
 *
 * Initially, you have w capital. When you finish a project, you will obtain
 * its pure profit and the profit will be added to your total capital.
 *
 * Pick a list of at most k distinct projects from given projects to maximize
 * your final capital, and return the final maximized capital.
```

```
 *
 * The answer is guaranteed to fit in a 32-bit signed integer.
 */

/**
 * @param {number} k
 * @param {number} w
 * @param {number[]} profits
 * @param {number[]} capital
 * @return {number}
 */
var findMaximizedCapital = function(k, w, profits, capital) {
  const queue = new MinPriorityQueue();
  const descendingQueue = new MaxPriorityQueue();

  for (let i = 0; i < capital.length; i++) {
    queue.enqueue([capital[i], profits[i]], capital[i]);
  }

  for (let i = 0; i < k; i++) {
    while (!queue.isEmpty() && queue.front().element[0] <= w) {
      const element = queue.dequeue().element;
      descendingQueue.enqueue(element, element[1]);
    }
    if (descendingQueue.isEmpty()) {
      return w;
    }
    w += descendingQueue.dequeue().element[1];
  }

  return w;
};
```

0503 Next Greater Element Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 503. Next Greater Element II
 * https://leetcode.com/problems/next-greater-element-ii/
 * Difficulty: Medium
 *
 * Given a circular integer array nums (i.e., the next element of
 * nums[nums.length - 1] is nums[0]), return the next greater number
 * for every element in nums.
 *
 * The next greater number of a number x is the first greater number
 * to its traversing-order next in the array, which means you could
 * search circularly to find its next greater number. If it doesn't
 * exist, return -1 for this number.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var nextGreaterElements = function(nums) {
  const total = nums.length;
  const result = new Array(total).fill(-1);
  const stack = [];

  for (let i = 0; i < 2 * total; i++) {
    while (stack.length && nums[i % total] > nums[stack[stack.length - 1]]) {
      result[stack.pop()] = nums[i % total];
    }
    stack.push(i % total);
  }

  return result;
};
```

0504 Base 7

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 504. Base 7
 * https://leetcode.com/problems/base-7/
 * Difficulty: Easy
 *
 * Given an integer num, return a string of its base 7 representation.
 */

/**
 * @param {number} num
 * @return {string}
 */
var convertToBase7 = function(num) {
  return num.toString(7);
};
```

0505 The Maze Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 505. The Maze II
 * https://leetcode.com/problems/the-maze-ii/
 * Difficulty: Medium
 *
 * There is a ball in a maze with empty spaces (represented as 0) and walls (represented as
1).
 * The ball can go through the empty spaces by rolling up, down, left or right, but it won't
 * stop rolling until hitting a wall. When the ball stops, it could choose the next direction.
 *
 * Given the m x n maze, the ball's start position and the destination, where
 * start = [startrow, startcol] and destination = [destinationrow, destinationcol], return
 * the shortest distance for the ball to stop at the destination. If the ball cannot stop
 * at destination, return -1.
 *
 * The distance is the number of empty spaces traveled by the ball from the start position
 * (excluded) to the destination (included).
 *
 * You may assume that the borders of the maze are all walls (see examples).
 */

/**
 * @param {number[][]} maze
 * @param {number[]} start
 * @param {number[]} destination
 * @return {number}
 */
var shortestDistance = function(maze, start, destination) {
  const rows = maze.length;
  const cols = maze[0].length;
  const distances = new Array(rows).fill().map(() => new Array(cols).fill(Infinity));
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  const queue = [[start[0], start[1], 0]];
  distances[start[0]][start[1]] = 0;

  while (queue.length) {
    const [row, col, distance] = queue.shift();
    if (distance > distances[row][col]) continue;

    for (const [dx, dy] of directions) {
      let nextRow = row;
```

```
      let nextCol = col;
      let steps = 0;

      while (nextRow + dx >= 0 && nextRow + dx < rows && nextCol + dy >= 0
            && nextCol + dy < cols && maze[nextRow + dx][nextCol + dy] === 0
      ) {
        nextRow += dx;
        nextCol += dy;
        steps++;
      }

      const newDistance = distance + steps;
      if (newDistance < distances[nextRow][nextCol]) {
        distances[nextRow][nextCol] = newDistance;
        queue.push([nextRow, nextCol, newDistance]);
      }
    }
  }

  const result = distances[destination[0]][destination[1]];
  return result === Infinity ? -1 : result;
};
```

0506 Relative Ranks

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 506. Relative Ranks
 * https://leetcode.com/problems/relative-ranks/
 * Difficulty: Easy
 *
 * You are given an integer array score of size n, where score[i] is the score of the ith
 * athlete in a competition. All the scores are guaranteed to be unique.
 *
 * The athletes are placed based on their scores, where the 1st place athlete has the highest
 * score, the 2nd place athlete has the 2nd highest score, and so on. The placement of each
 * athlete determines their rank:
 * - The 1st place athlete's rank is "Gold Medal".
 * - The 2nd place athlete's rank is "Silver Medal".
 * - The 3rd place athlete's rank is "Bronze Medal".
 * - For the 4th place to the nth place athlete, their rank is their placement number
 *   (i.e., the xth place athlete's rank is "x").
 *
 * Return an array answer of size n where answer[i] is the rank of the ith athlete.
 */

/**
 * @param {number[]} score
 * @return {string[]}
 */
var findRelativeRanks = function(score) {
  const PLACEMENTS = ['Gold Medal', 'Silver Medal', 'Bronze Medal'];
  const map = new Map();
  score.forEach((rank, index) => map.set(rank, index));

  const result = score.slice();
  const sorted = [...map].sort(([a], [b]) => b - a);

  sorted.forEach(([_, index], rank) => {
    result[index] = PLACEMENTS[rank] || `${rank + 1}`;
  });

  return result;
};
```

0507 Perfect Number

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 507. Perfect Number
 * https://leetcode.com/problems/perfect-number/
 * Difficulty: Easy
 *
 * A perfect number is a positive integer that is equal to the sum of its positive
 * divisors, excluding the number itself. A divisor of an integer x is an integer
 * that can divide x evenly.
 *
 * Given an integer n, return true if n is a perfect number, otherwise return false.
 */

/**
 * @param {number} num
 * @return {boolean}
 */
var checkPerfectNumber = function(num) {
  let result = 0;
  for (let i = 1; i <= num / 2; i++) {
    if (num % i === 0) result += i;
  }
  return result === num;
};
```

0508 Most Frequent Subtree Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 508. Most Frequent Subtree Sum
 * https://leetcode.com/problems/most-frequent-subtree-sum/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the most frequent subtree sum. If there is a tie,
return
 * all the values with the highest frequency in any order.
 *
 * The subtree sum of a node is defined as the sum of all the node values formed by the
subtree
 * rooted at that node (including the node itself).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var findFrequentTreeSum = function(root) {
  if (!root) return [];

  const map = new Map();
  let result = [];
  let max = 0;
```

```
  traverse(root);

  for (const [sum, freq] of map) {
    if (freq > max) {
      max = freq;
      result = [sum];
    } else if (freq === max) {
      result.push(sum);
    }
  }

  return result;

  function traverse(node) {
    if (!node) return 0;
    const sum = node.val + traverse(node.left) + traverse(node.right);
    map.set(sum, (map.get(sum) || 0) + 1);
    return sum;
  }
};
```

0509 Fibonacci Number

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 509. Fibonacci Number
 * https://leetcode.com/problems/fibonacci-number/
 * Difficulty: Easy
 *
 * The Fibonacci numbers, commonly denoted F(n) form a sequence, called the Fibonacci
sequence,
 * such that each number is the sum of the two preceding ones, starting from 0 and 1. That is:
 * - F(0) = 0, F(1) = 1
 * - F(n) = F(n - 1) + F(n - 2), for n > 1.
 * Given n, calculate F(n).
 */

/**
 * @param {number} n
 * @return {number}
 */
var fib = function(n) {
  const nums = [0, 1];

  for (let i = 2; i <= n; i++) {
    nums.push(nums[i - 2] + nums[i - 1]);
  }

  return nums[n];
};
```

0510 Inorder Successor In Bst Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 510. Inorder Successor in BST II
 * https://leetcode.com/problems/inorder-successor-in-bst-ii/
 * Difficulty: Medium
 *
 * Given a node in a binary search tree, return the in-order successor of that node in the
BST.
 * If that node has no in-order successor, return null.
 *
```

```
 * The successor of a node is the node with the smallest key greater than node.val.
 *
 * You will have direct access to the node but not to the root of the tree. Each node will
have
 * a reference to its parent node. Below is the definition for Node:
 *
 * class Node {
 *     public int val;
 *     public Node left;
 *     public Node right;
 *     public Node parent;
 * }
 */

/**
 * // Definition for a _Node.
 * function _Node(val) {
 *    this.val = val;
 *    this.left = null;
 *    this.right = null;
 *    this.parent = null;
 * };
 */

/**
 * @param {_Node} node
 * @return {_Node}
 */
var inorderSuccessor = function(node) {
  if (node.right) {
    let successor = node.right;
    while (successor.left) {
      successor = successor.left;
    }
    return successor;
  }

  let current = node;
  while (current.parent && current.parent.right === current) {
    current = current.parent;
  }
  return current.parent;
};
```

0513 Find Bottom Left Tree Value

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 513. Find Bottom Left Tree Value
 * https://leetcode.com/problems/find-bottom-left-tree-value/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the leftmost value in the last row of the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var findBottomLeftValue = function(root) {
  const queue = [root];
```

```
  let result = root.val;

  while (queue.length) {
    const size = queue.length;
    result = queue[0].val;
    for (let i = 0; i < size; i++) {
      const node = queue.shift();
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
  }

  return result;
};
```

0514 Freedom Trail

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 514. Freedom Trail
 * https://leetcode.com/problems/freedom-trail/
 * Difficulty: Hard
 *
 * In the video game Fallout 4, the quest "Road to Freedom" requires players to reach a metal
dial
 * called the "Freedom Trail Ring" and use the dial to spell a specific keyword to open the
door.
 *
 * Given a string ring that represents the code engraved on the outer ring and another string
key
 * that represents the keyword that needs to be spelled, return the minimum number of steps to
 spell
 * all the characters in the keyword.
 *
 * Initially, the first character of the ring is aligned at the "12:00" direction. You should
spell
 * all the characters in key one by one by rotating ring clockwise or anticlockwise to make
each
 * character of the string key aligned at the "12:00" direction and then by pressing the
center
 * button.
 *
 * At the stage of rotating the ring to spell the key character key[i]:
 * - You can rotate the ring clockwise or anticlockwise by one place, which counts as one
step.
 *   The final purpose of the rotation is to align one of ring's characters at the "12:00"
 *   direction, where this character must equal key[i].
 * - If the character key[i] has been aligned at the "12:00" direction, press the center
button
 *   to spell, which also counts as one step. After the pressing, you could begin to spell the
 *   next character in the key (next stage). Otherwise, you have finished all the spelling.
 */

/**
 * @param {string} ring
 * @param {string} key
 * @return {number}
 */
var findRotateSteps = function(ring, key) {
  const map = new Map();
  return dp(0, 0);

  function dp(ringIndex, keyIndex) {
    if (keyIndex === key.length) return 0;
    const state = `${ringIndex},${keyIndex}`;
    if (map.has(state)) return map.get(state);
    let minSteps = Infinity;
    for (let i = 0; i < ring.length; i++) {
      if (ring[i] === key[keyIndex]) {
```

```
        const distance = Math.abs(i - ringIndex);
        const steps = Math.min(distance, ring.length - distance);
        minSteps = Math.min(minSteps, steps + 1 + dp(i, keyIndex + 1));
      }
    }
    map.set(state, minSteps);
    return minSteps;
  }
};
```

0515 Find Largest Value In Each Tree Row

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 515. Find Largest Value in Each Tree Row
 * https://leetcode.com/problems/find-largest-value-in-each-tree-row/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return an array of the largest value in each row
 * of the tree (0-indexed).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var largestValues = function(root) {
  if (!root) return [];
  const result = [];
  const queue = [root];

  while (queue.length) {
    const level = queue.length;
    let max = -Infinity;
    for (let i = 0; i < level; i++) {
      const node = queue.shift();
      max = Math.max(max, node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(max);
  }

  return result;
};
```

0516 Longest Palindromic Subsequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 516. Longest Palindromic Subsequence
 * https://leetcode.com/problems/longest-palindromic-subsequence/
 * Difficulty: Medium
 *
 * Given a string s, find the longest palindromic subsequence's length in s.
```

```
 *
 * A subsequence is a sequence that can be derived from another sequence by deleting some
 * or no elements without changing the order of the remaining elements.
 */

/**
 * @param {string} s
 * @return {number}
 */
/**
 * @param {string} s
 * @return {number}
 */
var longestPalindromeSubseq = function(s) {
  const dp = new Array(s.length).fill(0);

  for (let i = s.length - 1, previous; i >= 0; i--) {
    previous = dp.slice();
    dp[i] = 1;
    for (let j = i + 1; j < s.length; j++) {
      if (s[i] === s[j]) {
        dp[j] = (previous[j - 1] || 0) + 2;
      } else {
        dp[j] = Math.max(dp[j - 1], previous[j]);
      }
    }
  }

  return dp[s.length - 1];
};
```

0517 Super Washing Machines

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 517. Super Washing Machines
 * https://leetcode.com/problems/super-washing-machines/
 * Difficulty: Hard
 *
 * You have n super washing machines on a line. Initially, each washing machine has some
 * dresses or is empty.
 *
 * For each move, you could choose any m (1 <= m <= n) washing machines, and pass one dress
 * of each washing machine to one of its adjacent washing machines at the same time.
 *
 * Given an integer array machines representing the number of dresses in each washing machine
 * from left to right on the line, return the minimum number of moves to make all the washing
 * machines have the same number of dresses. If it is not possible to do it, return -1.
 */

/**
 * @param {number[]} machines
 * @return {number}
 */
var findMinMoves = function(machines) {
  const total = machines.reduce((sum, num) => sum + num, 0);
  const target = total / machines.length;
  if (total % machines.length !== 0) return -1;

  let result = 0;
  for (let i = 0, balance = 0; i < machines.length; i++) {
    balance += machines[i] - target;
    result = Math.max(result, Math.abs(balance), machines[i] - target);
  }
  return result;
};
```

0518 Coin Change Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 518. Coin Change II
 * https://leetcode.com/problems/coin-change-ii/
 * Difficulty: Medium
 *
 * You are given an integer array coins representing coins of different denominations and
 * an integer amount representing a total amount of money.
 *
 * Return the number of combinations that make up that amount. If that amount of money
 * cannot be made up by any combination of the coins, return 0.
 *
 * You may assume that you have an infinite number of each kind of coin.
 *
 * The answer is guaranteed to fit into a signed 32-bit integer.
 */

/**
 * @param {number} amount
 * @param {number[]} coins
 * @return {number}
 */
var change = function(amount, coins) {
  const dp = new Array(amount + 1).fill(0);
  dp[0] = 1;

  for (const coin of coins) {
    for (let i = coin; i <= amount; i++) {
      dp[i] += dp[i - coin];
    }
  }

  return dp[amount];
};
```

0519 Random Flip Matrix

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 519. Random Flip Matrix
 * https://leetcode.com/problems/random-flip-matrix/
 * Difficulty: Medium
 *
 * There is an m x n binary grid matrix with all the values set 0 initially. Design an
algorithm
 * to randomly pick an index (i, j) where matrix[i][j] == 0 and flips it to 1. All the indices
 * (i, j) where matrix[i][j] == 0 should be equally likely to be returned.
 *
 * Optimize your algorithm to minimize the number of calls made to the built-in random
function
 * of your language and optimize the time and space complexity.
 *
 * Implement the Solution class:
 * - Solution(int m, int n) Initializes the object with the size of the binary matrix m and n.
 * - int[] flip() Returns a random index [i, j] of the matrix where matrix[i][j] == 0 and
flips
 *   it to 1.
 * - void reset() Resets all the values of the matrix to be 0.
 */

/**
```

```
 * @param {number} m
 * @param {number} n
 */
var Solution = function(m, n) {
  this.m = m;
  this.n = n;
  this.total = m * n;
  this.flipped = new Map();
};

/**
 * @return {number[]}
 */
Solution.prototype.flip = function() {
  const index = Math.floor(Math.random() * this.total--);
  const result = this.flipped.get(index) ?? index;
  this.flipped.set(index, this.flipped.get(this.total) ?? this.total);
  return [Math.floor(result / this.n), result % this.n];
};

/**
 * @return {void}
 */
Solution.prototype.reset = function() {
  this.total = this.m * this.n;
  this.flipped.clear();
};
```

0520 Detect Capital

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 520. Detect Capital
 * https://leetcode.com/problems/detect-capital/
 * Difficulty: Easy
 *
 * We define the usage of capitals in a word to be right when one of the following
 * cases holds:
 * - All letters in this word are capitals, like "USA".
 * - All letters in this word are not capitals, like "leetcode".
 * - Only the first letter in this word is capital, like "Google".
 *
 * Given a string word, return true if the usage of capitals in it is right.
 */

/**
 * @param {string} word
 * @return {boolean}
 */
var detectCapitalUse = function(word) {
  return /^[A-Z]?[a-z]+$|^[A-Z]+$/.test(word);
};
```

0521 Longest Uncommon Subsequence I

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 521. Longest Uncommon Subsequence I
 * https://leetcode.com/problems/longest-uncommon-subsequence-i/
 * Difficulty: Easy
 *
 * Given two strings a and b, return the length of the longest uncommon subsequence between
 * a and b. If no such uncommon subsequence exists, return -1.
```

```
 *
 * An uncommon subsequence between two strings is a string that is a subsequence of exactly
 * one of them.
 */

/**
 * @param {string} a
 * @param {string} b
 * @return {number}
 */
var findLUSlength = function(a, b) {
  return a === b ? -1 : Math.max(a.length, b.length);
};
```

0522 Longest Uncommon Subsequence Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 522. Longest Uncommon Subsequence II
 * https://leetcode.com/problems/longest-uncommon-subsequence-ii/
 * Difficulty: Medium
 *
 * Given an array of strings strs, return the length of the longest uncommon subsequence
 * between them. If the longest uncommon subsequence does not exist, return -1.
 *
 * An uncommon subsequence between an array of strings is a string that is a subsequence
 * of one string but not the others.
 *
 * A subsequence of a string s is a string that can be obtained after deleting any number
 * of characters from s.
 *
 * For example, "abc" is a subsequence of "aebdc" because you can delete the underlined
 * characters in "aebdc" to get "abc". Other subsequences of "aebdc" include "aebdc",
 * "aeb", and "" (empty string).
 */

/**
 * @param {string[]} strs
 * @return {number}
 */
var findLUSlength = function(strs) {
  strs.sort((a, b) => b.length - a.length);

  for (let i = 0; i < strs.length; i++) {
    let isUnique = true;
    for (let j = 0; j < strs.length; j++) {
      if (i !== j && isSubsequence(strs[i], strs[j])) {
        isUnique = false;
        break;
      }
    }
    if (isUnique) {
      return strs[i].length;
    }
  }

  return -1;

  function isSubsequence(s1, s2) {
    let index = 0;
    for (const char of s2) {
      if (char === s1[index]) index++;
      if (index === s1.length) return true;
    }
    return false;
  }
};
```

0523 Continuous Subarray Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 523. Continuous Subarray Sum
 * https://leetcode.com/problems/continuous-subarray-sum/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return true if nums has a good subarray
 * or false otherwise.
 *
 * A good subarray is a subarray where:
 * - its length is at least two, and
 * - the sum of the elements of the subarray is a multiple of k.
 *
 * Note that:
 * - A subarray is a contiguous part of the array.
 * - An integer x is a multiple of k if there exists an integer n such that x = n * k.
 *   0 is always a multiple of k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
var checkSubarraySum = function(nums, k) {
  const map = new Map([[0, -1]]);

  for (let i = 0, sum = 0; i < nums.length; i++) {
    sum += nums[i];
    if (k !== 0) {
      sum = sum % k;
    }

    if (map.has(sum)) {
      if (i - map.get(sum) >= 2) {
        return true;
      }
    } else {
      map.set(sum, i);
    }
  }

  return false;
};
```

0524 Longest Word In Dictionary Through Deleting

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 524. Longest Word in Dictionary through Deleting
 * https://leetcode.com/problems/longest-word-in-dictionary-through-deleting/
 * Difficulty: Medium
 *
 * Given a string s and a string array dictionary, return the longest string in the dictionary
 * that can be formed by deleting some of the given string characters. If there is more than
 * one possible result, return the longest word with the smallest lexicographical order.
 * If there is no possible result, return the empty string.
 */

/**
 * @param {string} s
```

```
 * @param {string[]} dictionary
 * @return {string}
 */
var findLongestWord = function(s, dictionary) {
  return dictionary
    .sort((a, b) => a.length === b.length ? a.localeCompare(b) : b.length - a.length)
    .find(word => {
      let i = 0;
      for (const character of s) {
        if (character === word[i]) i++;
        if (i === word.length) return true;
      }
      return false;
    }) || '';
};
```

0525 Contiguous Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 525. Contiguous Array
 * https://leetcode.com/problems/contiguous-array/
 * Difficulty: Medium
 *
 * Given a binary array nums, return the maximum length of a contiguous subarray
 * with an equal number of 0 and 1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMaxLength = function(nums) {
  const map = new Map([[0, -1]]);
  let result = 0;

  for (let i = 0, count = 0; i < nums.length; i++) {
    count += nums[i] === 1 ? 1 : -1;
    if (map.has(count)) {
      result = Math.max(result, i - map.get(count));
    } else {
      map.set(count, i);
    }
  }

  return result;
};
```

0526 Beautiful Arrangement

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 526. Beautiful Arrangement
 * https://leetcode.com/problems/beautiful-arrangement/
 * Difficulty: Medium
 *
 * Suppose you have n integers labeled 1 through n. A permutation of those n integers perm
 * (1-indexed) is considered a beautiful arrangement if for every i (1 <= i <= n), either
 * of the following is true:
 * - perm[i] is divisible by i.
 * - i is divisible by perm[i].
 *
 * Given an integer n, return the number of the beautiful arrangements that you can construct.
```

```
 */

/**
 * @param {number} n
 * @return {number}
 */
var countArrangement = function(n) {
  let count = 0;
  backtrack(1, 0);
  return count;

  function backtrack(offset, used) {
    if (offset > n) {
      count++;
      return;
    }
    for (let i = 1; i <= n; i++) {
      if (!(used & (1 << i)) && (i % offset === 0 || offset % i === 0)) {
        backtrack(offset + 1, used | (1 << i));
      }
    }
  }
};
```

0527 Word Abbreviation

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 527. Word Abbreviation
 * https://leetcode.com/problems/word-abbreviation/
 * Difficulty: Hard
 *
 * Given an array of distinct strings words, return the minimal possible abbreviations for
 * every word.
 *
 * The following are the rules for a string abbreviation:
 * 1. The initial abbreviation for each word is: the first character, then the number of
 *    characters in between, followed by the last character.
 * 2. If more than one word shares the same abbreviation, then perform the following
operation:
 *    - Increase the prefix (characters in the first part) of each of their abbreviations by
1.
 *      - For example, say you start with the words ["abcdef","abndef"] both initially
abbreviated
 *        as "a4f". Then, a sequence of operations would be
 *        ["a4f","a4f"] -> ["ab3f","ab3f"] -> ["abc2f","abn2f"].
 *   - This operation is repeated until every abbreviation is unique.
 * 3. At the end, if an abbreviation did not make a word shorter, then keep it as the original
 word.
 */

/**
 * @param {string[]} words
 * @return {string[]}
 */
var wordsAbbreviation = function(words) {
  const n = words.length;
  const result = new Array(n).fill('');
  const groups = new Map();

  function getAbbreviation(word, prefixLen) {
    const len = word.length;
    if (len <= prefixLen + 2) return word;
    return word.slice(0, prefixLen) + (len - prefixLen - 1) + word[len - 1];
  }

  for (let i = 0; i < n; i++) {
    const word = words[i];
```

```
    const prefixLen = 1;
    const abbr = getAbbreviation(word, prefixLen);

    if (!groups.has(abbr)) {
      groups.set(abbr, []);
    }
    groups.get(abbr).push([i, prefixLen]);
  }

  while (true) {
    let resolved = true;
    const newGroups = new Map();

    for (const [abbr, indices] of groups) {
      if (indices.length === 1) {
        const [index, prefixLen] = indices[0];
        const word = words[index];
        const finalAbbr = getAbbreviation(word, prefixLen);
        result[index] = finalAbbr.length < word.length ? finalAbbr : word;
        continue;
      }

      resolved = false;
      for (const [index, prefixLen] of indices) {
        const word = words[index];
        const newAbbr = getAbbreviation(word, prefixLen + 1);

        if (!newGroups.has(newAbbr)) {
          newGroups.set(newAbbr, []);
        }
        newGroups.get(newAbbr).push([index, prefixLen + 1]);
      }
    }

    if (resolved) break;
    groups.clear();
    for (const [abbr, indices] of newGroups) {
      groups.set(abbr, indices);
    }
  }

  return result;
};
```

0528 Random Pick With Weight

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 528. Random Pick with Weight
 * https://leetcode.com/problems/random-pick-with-weight/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of positive integers w where w[i] describes the
 * weight of the ith index.
 *
 * You need to implement the function pickIndex(), which randomly picks an index in
 * the range [0, w.length - 1] (inclusive) and returns it. The probability of picking
 * an index i is w[i] / sum(w).
 *
 * For example, if w = [1, 3], the probability of picking index 0 is 1 / (1 + 3) = 0.25
 * (i.e., 25%), and the probability of picking index 1 is 3 / (1 + 3) = 0.75 (i.e., 75%).
 */

/**
 * @param {number[]} w
 */
var Solution = function(w) {
  this.sums = [];
  let sum = 0;
```

```
  for (const weight of w) {
    sum += weight;
    this.sums.push(sum);
  }
  this.total = sum;
};

/**
 * @return {number}
 */
Solution.prototype.pickIndex = function() {
  const target = Math.random() * this.total;
  let left = 0;
  let right = this.sums.length - 1;
  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (this.sums[mid] <= target) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }
  return left;
};
```

0529 Minesweeper

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 529. Minesweeper
 * https://leetcode.com/problems/minesweeper/
 * Difficulty: Medium
 *
 * Let's play the minesweeper game (Wikipedia, online game)!
 *
 * You are given an m x n char matrix board representing the game board where:
 * - 'M' represents an unrevealed mine,
 * - 'E' represents an unrevealed empty square,
 * - 'B' represents a revealed blank square that has no adjacent mines (i.e., above, below,
 *   left, right, and all 4 diagonals),
 * - digit ('1' to '8') represents how many mines are adjacent to this revealed square, and
 * - 'X' represents a revealed mine.
 *
 * You are also given an integer array click where click = [clickr, clickc] represents the
 * next click position among all the unrevealed squares ('M' or 'E').
 *
 * Return the board after revealing this position according to the following rules:
 * 1. If a mine 'M' is revealed, then the game is over. You should change it to 'X'.
 * 2. If an empty square 'E' with no adjacent mines is revealed, then change it to a revealed
 *    blank 'B' and all of its adjacent unrevealed squares should be revealed recursively.
 * 3. If an empty square 'E' with at least one adjacent mine is revealed, then change it to a
 *    digit ('1' to '8') representing the number of adjacent mines.
 * 4. Return the board when no more squares will be revealed.
 */

/**
 * @param {character[][]} board
 * @param {number[]} click
 * @return {character[][]}
 */
var updateBoard = function(board, click) {
  const [row, col] = click;
  const rows = board.length;
  const cols = board[0].length;
  const directions = [
    [-1, -1], [-1, 0], [-1, 1], [0, -1], [0, 1], [1, -1], [1, 0], [1, 1]
  ];

  if (board[row][col] === 'M') {
```

```
    board[row][col] = 'X';
    return board;
  }

  dfs(row, col);
  return board;

  function dfs(r, c) {
    if (r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] !== 'E') return;

    let mines = 0;
    for (const [dr, dc] of directions) {
      const nr = r + dr;
      const nc = c + dc;
      if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && board[nr][nc] === 'M') {
        mines++;
      }
    }

    board[r][c] = mines > 0 ? mines.toString() : 'B';
    if (mines === 0) {
      for (const [dr, dc] of directions) {
        dfs(r + dr, c + dc);
      }
    }
  }
};
```

0530 Minimum Absolute Difference In Bst

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 530. Minimum Absolute Difference in BST
 * https://leetcode.com/problems/minimum-absolute-difference-in-bst/
 * Difficulty: Easy
 *
 * Given the root of a Binary Search Tree (BST), return the minimum absolute difference
between
 * the values of any two different nodes in the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var getMinimumDifference = function(root) {
  let result = Infinity;
  let previous = null;

  inorder(root);
  return result;

  function inorder(node) {
    if (!node) return;
    inorder(node.left);
    result = previous === null ? result : Math.min(result, node.val - previous);
    previous = node.val;
    inorder(node.right);
  }
};
```

0531 Lonely Pixel I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 531. Lonely Pixel I
 * https://leetcode.com/problems/lonely-pixel-i/
 * Difficulty: Medium
 *
 * Given an m x n picture consisting of black 'B' and white 'W' pixels, return the number of
 * black lonely pixels.
 *
 * A black lonely pixel is a character 'B' that located at a specific position where the same
 * row and same column don't have any other black pixels.
 */

/**
 * @param {character[][]} picture
 * @return {number}
 */
var findLonelyPixel = function(picture) {
  const rows = picture.length;
  const cols = picture[0].length;
  const rowCounts = new Array(rows).fill(0);
  const colCounts = new Array(cols).fill(0);
  let result = 0;

  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      if (picture[row][col] === 'B') {
        rowCounts[row]++;
        colCounts[col]++;
      }
    }
  }

  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      if (picture[row][col] === 'B' && rowCounts[row] === 1 && colCounts[col] === 1) {
        result++;
      }
    }
  }

  return result;
};
```

0532 K Diff Pairs In An Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 532. K-diff Pairs in an Array
 * https://leetcode.com/problems/k-diff-pairs-in-an-array/
 * Difficulty: Medium
 *
 * Given an array of integers nums and an integer k, return the number of unique k-diff
 * pairs in the array.
 *
 * A k-diff pair is an integer pair (nums[i], nums[j]), where the following are true:
 * - 0 <= i, j < nums.length
 * - i != j
 * - |nums[i] - nums[j]| == k
 *
 * Notice that |val| denotes the absolute value of val.
```

```
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var findPairs = function(nums, k) {
  const map = new Map();
  let result = 0;

  for (const num of nums) {
    if (!map.has(num)) {
      if (!k) {
        map.set(num, 1);
      } else {
        if (map.has(num - k)) result++;
        if (map.has(num + k)) result++;
        map.set(num, 1);
      }
    } else if (!k && map.get(num) === 1) {
      result++;
      map.set(num, 2);
    }
  }

  return result;
};
```

0533 Lonely Pixel Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 533. Lonely Pixel II
 * https://leetcode.com/problems/lonely-pixel-ii/
 * Difficulty: Medium
 *
 * Given an m x n picture consisting of black 'B' and white 'W' pixels and an integer target,
 * return the number of black lonely pixels.
 *
 * A black lonely pixel is a character 'B' that located at a specific position (r, c) where:
 * - Row r and column c both contain exactly target black pixels.
 * - For all rows that have a black pixel at column c, they should be exactly the same as row
r.
 */

/**
 * @param {character[][]} picture
 * @param {number} target
 * @return {number}
 */
var findBlackPixel = function(picture, target) {
  const rows = picture.length;
  const cols = picture[0].length;
  const rowCounts = new Array(rows).fill(0);
  const colCounts = new Array(cols).fill(0);
  const rowPatterns = new Map();

  for (let row = 0; row < rows; row++) {
    let blackCount = 0;
    for (let col = 0; col < cols; col++) {
      if (picture[row][col] === 'B') {
        blackCount++;
        colCounts[col]++;
      }
    }
    rowCounts[row] = blackCount;
    if (blackCount === target) {
      const pattern = picture[row].join('');
```

```
      rowPatterns.set(pattern, (rowPatterns.get(pattern) || 0) + 1);
    }
  }

  let result = 0;
  for (let col = 0; col < cols; col++) {
    if (colCounts[col] === target) {
      for (let row = 0; row < rows; row++) {
        if (picture[row][col] === 'B' && rowCounts[row] === target) {
          const pattern = picture[row].join('');
          if (rowPatterns.get(pattern) === target) {
            result += target;
            break;
          }
        }
      }
    }
  }

  return result;
};
```

0535 Encode And Decode Tinyurl

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 535. Encode and Decode TinyURL
 * https://leetcode.com/problems/encode-and-decode-tinyurl/
 * Difficulty: Medium
 *
 * TinyURL is a URL shortening service where you enter a URL such as
 * https://leetcode.com/problems/design-tinyurl and it returns a short URL such as
 * http://tinyurl.com/4e9iAk. Design a class to encode a URL and decode a tiny URL.
 *
 * There is no restriction on how your encode/decode algorithm should work. You just
 * need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be
 * decoded to the original URL.
 *
 * Implement the Solution class:
 * - Solution() Initializes the object of the system.
 * - String encode(String longUrl) Returns a tiny URL for the given longUrl.
 * - String decode(String shortUrl) Returns the original long URL for the given shortUrl.
 *   It is guaranteed that the given shortUrl was encoded by the same object.
 */

const map = new Map();
let counter = 1;

/**
 * Encodes a URL to a shortened URL.
 *
 * @param {string} longUrl
 * @return {string}
 */
var encode = function(longUrl) {
  const shortUrl = 'https://tinyurl.com/' + counter.toString();
  map.set(shortUrl, longUrl);
  counter++;
  return shortUrl;
};

/**
 * Decodes a shortened URL to its original URL.
 *
 * @param {string} shortUrl
 * @return {string}
 */
var decode = function(shortUrl) {
  return map.get(shortUrl) || null;
```

```
};
```

0536 Construct Binary Tree From String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 536. Construct Binary Tree from String
 * https://leetcode.com/problems/construct-binary-tree-from-string/
 * Difficulty: Medium
 *
 * You need to construct a binary tree from a string consisting of parenthesis and integers.
 *
 * The whole input represents a binary tree. It contains an integer followed by zero, one or
 * two pairs of parenthesis. The integer represents the root's value and a pair of parenthesis
 * contains a child binary tree with the same structure.
 *
 * You always start to construct the left child node of the parent first if it exists.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {string} s
 * @return {TreeNode}
 */
var str2tree = function(s) {
  if (!s) return null;
  let index = 0;
  return constructTree();

  function parseNumber() {
    const isNegative = s[index] === '-';
    if (isNegative) index++;

    let num = 0;
    while (index < s.length && /[0-9]/.test(s[index])) {
      num = num * 10 + parseInt(s[index]);
      index++;
    }

    return isNegative ? -num : num;
  }

  function constructTree() {
    if (index >= s.length) return null;

    const value = parseNumber();
    const node = new TreeNode(value);
    if (index < s.length && s[index] === '(') {
      index++;
      node.left = constructTree();
      index++;
    }

    if (index < s.length && s[index] === '(') {
      index++;
      node.right = constructTree();
      index++;
    }

    return node;
  }
};
```

0537 Complex Number Multiplication

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 537. Complex Number Multiplication
 * https://leetcode.com/problems/complex-number-multiplication/
 * Difficulty: Medium
 *
 * A complex number can be represented as a string on the form "real+imaginaryi" where:
 * - real is the real part and is an integer in the range [-100, 100].
 * - imaginary is the imaginary part and is an integer in the range [-100, 100].
 * - i2 == -1.
 *
 * Given two complex numbers num1 and num2 as strings, return a string of the complex number
 * that represents their multiplications.
 */

/**
 * @param {string} num1
 * @param {string} num2
 * @return {string}
 */
var complexNumberMultiply = function(num1, num2) {
  const [r1, i1] = num1.split('+').map(n => parseInt(n.replace('i', '')));
  const [r2, i2] = num2.split('+').map(n => parseInt(n.replace('i', '')));
  return `${r1 * r2 - i1 * i2}+${r1 * i2 + r2 * i1}i`;
};
```

0538 Convert Bst To Greater Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 538. Convert BST to Greater Tree
 * https://leetcode.com/problems/convert-bst-to-greater-tree/
 * Difficulty: Medium
 *
 * Given the root of a Binary Search Tree (BST), convert it to a Greater Tree such that every
 * key of the original BST is changed to the original key plus the sum of all keys greater
 * than the original key in BST.
 *
 * As a reminder, a binary search tree is a tree that satisfies these constraints:
 * - The left subtree of a node contains only nodes with keys less than the node's key.
 * - The right subtree of a node contains only nodes with keys greater than the node's key.
 * - Both the left and right subtrees must also be binary search trees.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var convertBST = function(root) {
  let sum = 0;
  traverse(root);
  return root;
```

```
  function traverse(node) {
    if (!node) return;
    traverse(node.right);
    sum += node.val;
    node.val = sum;
    traverse(node.left);
  }
};
```

0539 Minimum Time Difference

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 539. Minimum Time Difference
 * https://leetcode.com/problems/minimum-time-difference/
 * Difficulty: Medium
 *
 * Given a list of 24-hour clock time points in "HH:MM" format, return the minimum minutes
 * difference between any two time-points in the list.
 */

/**
 * @param {string[]} timePoints
 * @return {number}
 */
var findMinDifference = function(timePoints) {
  const minutes = timePoints.map(time => {
    const [h, m] = time.split(':').map(Number);
    return h * 60 + m;
  }).sort((a, b) => a - b);
  let diff = Infinity;

  for (let i = 1; i < minutes.length; i++) {
    diff = Math.min(diff, minutes[i] - minutes[i - 1]);
  }

  return Math.min(diff, 1440 - minutes[minutes.length - 1] + minutes[0]);
};
```

0540 Single Element In A Sorted Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 540. Single Element in a Sorted Array
 * https://leetcode.com/problems/single-element-in-a-sorted-array/
 * Difficulty: Medium
 *
 * You are given a sorted array consisting of only integers where every element appears
 * exactly twice, except for one element which appears exactly once.
 *
 * Return the single element that appears only once.
 *
 * Your solution must run in O(log n) time and O(1) space.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var singleNonDuplicate = function(nums) {
  let left = 0;
  let right = nums.length - 1;
```

```
  while (left < right) {
    let middle = left + Math.floor((right - left) / 2);
    middle = middle - (middle % 2);
    if (nums[middle] === nums[middle + 1]) {
      left = middle + 2;
    } else {
      right = middle;
    }
  }

  return nums[left];
};
```

0541 Reverse String Ii

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 541. Reverse String II
 * https://leetcode.com/problems/reverse-string-ii/
 * Difficulty: Easy
 *
 * Given a string and an integer k, you need to reverse the first k characters
 * for every 2k characters counting from the start of the string. If there are
 * less than k characters left, reverse all of them. If there are less than 2k
 * but greater than or equal to k characters, then reverse the first k
 * characters and left the other as original.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var reverseStr = function(s, k) {
  const split = s.split('');
  for (let i = 0; i < s.length; i += 2 * k) {
    split.splice(i, 0, ...split.splice(i, k).reverse());
  }
  return split.join('');
};
```

0542 01 Matrix

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 542. 01 Matrix
 * https://leetcode.com/problems/01-matrix/
 * Difficulty: Medium
 *
 * Given an m x n binary matrix mat, return the distance of the nearest 0 for each cell.
 *
 * The distance between two adjacent cells is 1.
 */

/**
 * @param {number[][]} matrix
 * @return {number[][]}
 */
var updateMatrix = function(matrix) {
  const queue = [];

  for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < matrix[0].length; j++) {
```

```
      if (matrix[i][j] === 0) {
        queue.push([i, j, 0]);
      } else {
        matrix[i][j] = Infinity;
      }
    }
  }


  while (queue.length) {
    const [i, j, k] = queue.shift();
    if (matrix[i][j] > k) {
      matrix[i][j] = k;
    }
    [[-1, 0], [1, 0], [0, -1], [0, 1]].forEach(p => {
      const [x, y, z] = [i + p[0], j + p[1], k + 1];
      if (x > -1 && x < matrix.length && y > -1 && y < matrix[0].length) {
        if (matrix[x][y] === Infinity) {
          queue.push([x, y, z]);
        }
      }
    });
  }

  return matrix;
};
```

0543 Diameter Of Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 543. Diameter of Binary Tree
 * https://leetcode.com/problems/diameter-of-binary-tree/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return the length of the diameter of the tree.
 *
 * The diameter of a binary tree is the length of the longest path between any two nodes in a
 * tree. This path may or may not pass through the root.
 *
 * The length of a path between two nodes is represented by the number of edges between them.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var diameterOfBinaryTree = function(root) {
  let result = 0;
  maxDepth(root);
  return result;

  function maxDepth(node) {
    if (!node) return 0;
    const left = maxDepth(node.left);
    const right = maxDepth(node.right);
    result = Math.max(result, left + right);
    return Math.max(left, right) + 1;
  }
};
```

0544 Output Contest Matches

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 544. Output Contest Matches
 * https://leetcode.com/problems/output-contest-matches/
 * Difficulty: Medium
 *
 * During the NBA playoffs, we always set the rather strong team to play with the rather weak
 * team, like making the rank 1 team play with the rank nth team, which is a good strategy to
 * make the contest more interesting.
 *
 * Given n teams, return their final contest matches in the form of a string.
 *
 * The n teams are labeled from 1 to n, which represents their initial rank (i.e., Rank 1 is
 * the strongest team and Rank n is the weakest team).
 *
 * We will use parentheses '(', and ')' and commas ',' to represent the contest team pairing.
 * We use the parentheses for pairing and the commas for partition. During the pairing process
 * in each round, you always need to follow the strategy of making the rather strong one pair
 * with the rather weak one.
 */

/**
 * @param {number} n
 * @return {string}
 */
var findContestMatch = function(n) {
  const teams = Array.from({ length: n }, (_, i) => (i + 1).toString());
  return pairTeams(teams);

  function pairTeams(arr) {
    if (arr.length === 1) return arr[0];

    const nextRound = [];
    for (let i = 0; i < arr.length / 2; i++) {
      nextRound.push(`(${arr[i]},${arr[arr.length - 1 - i]})`);
    }

    return pairTeams(nextRound);
  }
};
```

0545 Boundary Of Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 545. Boundary of Binary Tree
 * https://leetcode.com/problems/boundary-of-binary-tree/
 * Difficulty: Medium
 *
 * The boundary of a binary tree is the concatenation of the root, the left boundary, the
leaves
 * ordered from left-to-right, and the reverse order of the right boundary.
 *
 * The left boundary is the set of nodes defined by the following:
 * - The root node's left child is in the left boundary. If the root does not have a left
child,
 *   then the left boundary is empty.
 * - If a node in the left boundary and has a left child, then the left child is in the left
 *   boundary.
 * - If a node is in the left boundary, has no left child, but has a right child, then the
right
```

```
 *   child is in the left boundary.
 * - The leftmost leaf is not in the left boundary.
 *
 * The right boundary is similar to the left boundary, except it is the right side of the
root's
 * right subtree. Again, the leaf is not part of the right boundary, and the right boundary is
 * empty if the root does not have a right child.
 *
 * The leaves are nodes that do not have any children. For this problem, the root is not a
leaf.
 *
 * Given the root of a binary tree, return the values of its boundary.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var boundaryOfBinaryTree = function(root) {
  if (!root) return [];
  if (!root.left && !root.right) return [root.val];

  const result = [root.val];

  collectLeftBoundary(root.left);
  collectLeaves(root);
  collectRightBoundary(root.right);

  return result;

  function isLeaf(node) {
    return node && !node.left && !node.right;
  }

  function collectLeftBoundary(node) {
    if (!node || isLeaf(node)) return;
    result.push(node.val);
    if (node.left) {
      collectLeftBoundary(node.left);
    } else {
      collectLeftBoundary(node.right);
    }
  }

  function collectLeaves(node) {
    if (!node) return;
    if (isLeaf(node)) {
      result.push(node.val);
      return;
    }
    collectLeaves(node.left);
    collectLeaves(node.right);
  }

  function collectRightBoundary(node) {
    if (!node || isLeaf(node)) return;
    if (node.right) {
      collectRightBoundary(node.right);
    } else {
      collectRightBoundary(node.left);
    }
    result.push(node.val);
  }
};
```

0546 Remove Boxes

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 546. Remove Boxes
 * https://leetcode.com/problems/remove-boxes/
 * Difficulty: Hard
 *
 * You are given several boxes with different colors represented by different positive
numbers.
 *
 * You may experience several rounds to remove boxes until there is no box left. Each time you
 * can choose some continuous boxes with the same color (i.e., composed of k boxes, k >= 1),
 * remove them and get k * k points.
 *
 * Return the maximum points you can get.
 */

/**
 * @param {number[]} boxes
 * @return {number}
 */
var removeBoxes = function(boxes) {
  const dp = new Array(boxes.length).fill().map(() => {
    return new Array(boxes.length).fill().map(() => new Array(boxes.length + 1).fill(0));
  });
  return dfs(0, boxes.length - 1, 0);

  function dfs(l, r, k) {
    if (l > r) return 0;
    if (dp[l][r][k]) return dp[l][r][k];
    let i = l;
    let count = k + 1;
    while (i < r && boxes[i] === boxes[i + 1]) {
      i++;
      count++;
    }
    let max = count * count + dfs(i + 1, r, 0);
    for (let m = i + 1; m <= r; m++) {
      if (boxes[l] === boxes[m]) {
        max = Math.max(max, dfs(i + 1, m - 1, 0) + dfs(m, r, count));
      }
    }
    return dp[l][r][k] = max;
  }
};
```

0547 Number Of Provinces

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 547. Number of Provinces
 * https://leetcode.com/problems/number-of-provinces/
 * Difficulty: Medium
 *
 * There are n cities. Some of them are connected, while some are not. If city a is connected
 * directly with city b, and city b is connected directly with city c, then city a is
connected
 * indirectly with city c.
 *
 * A province is a group of directly or indirectly connected cities and no other cities
```

```
outside
 * of the group.
 *
 * You are given an n x n matrix isConnected where isConnected[i][j] = 1 if the ith city and
the
 * jth city are directly connected, and isConnected[i][j] = 0 otherwise.

Return the total number of provinces.
 */

/**
 * @param {number[][]} isConnected
 * @return {number}
 */
var findCircleNum = function(isConnected) {
  const seen = new Array(isConnected.length).fill(0);
  let result = 0;

  function dfs(node) {
    seen[node] = 1;
    for (let i = 0; i < isConnected.length; i++) {
      if (isConnected[node][i] === 1 && !seen[i]) {
        dfs(i);
      }
    }
  }

  for (let i = 0; i < isConnected.length; i++) {
    if (!seen[i]) {
      result++;
      dfs(i);
    }
  }

  return result;
};
```

0548 Split Array With Equal Sum

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 548. Split Array with Equal Sum
 * https://leetcode.com/problems/split-array-with-equal-sum/
 * Difficulty: Hard
 *
 * Given an integer array nums of length n, return true if there is a triplet (i, j, k) which
 * satisfies the following conditions:
 * - 0 < i, i + 1 < j, j + 1 < k < n - 1
 * - The sum of subarrays (0, i - 1), (i + 1, j - 1), (j + 1, k - 1) and (k + 1, n - 1) is
equal.
 *
 * A subarray (l, r) represents a slice of the original array starting from the element
 * indexed l to the element indexed r.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var splitArray = function(nums) {
  const length = nums.length;
  if (length < 7) return false;

  const prefixSums = new Array(length + 1).fill(0);
  for (let i = 0; i < length; i++) {
    prefixSums[i + 1] = prefixSums[i] + nums[i];
  }

  const getSum = (start, end) => prefixSums[end + 1] - prefixSums[start];
```

```
  for (let j = 3; j < length - 3; j++) {
    const seenSums = new Set();
    for (let i = 1; i < j - 1; i++) {
      const firstSum = getSum(0, i - 1);
      const secondSum = getSum(i + 1, j - 1);
      if (firstSum === secondSum) {
        seenSums.add(firstSum);
      }
    }
    for (let k = j + 2; k < length - 1; k++) {
      const thirdSum = getSum(j + 1, k - 1);
      const fourthSum = getSum(k + 1, length - 1);
      if (thirdSum === fourthSum && seenSums.has(thirdSum)) {
        return true;
      }
    }
  }
  return false;
};
```

0549 Binary Tree Longest Consecutive Sequence Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 549. Binary Tree Longest Consecutive Sequence II
 * https://leetcode.com/problems/binary-tree-longest-consecutive-sequence-ii/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the length of the longest consecutive path in the
tree.
 *
 * A consecutive path is a path where the values of the consecutive nodes in the path differ
by one.
 * This path can be either increasing or decreasing.
 * - For example, [1,2,3,4] and [4,3,2,1] are both considered valid, but the path [1,2,4,3] is
 *   not valid.
 *
 * On the other hand, the path can be in the child-Parent-child order, where not necessarily
be
 * parent-child order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var longestConsecutive = function(root) {
  let maxLength = 0;
  traverse(root);
  return maxLength;

  function traverse(node) {
    if (!node) return [0, 0];

    let inc = 1;
    let dec = 1;

    if (node.left) {
      const [leftInc, leftDec] = traverse(node.left);
      if (node.val === node.left.val + 1) {
```

```
        dec = Math.max(dec, leftDec + 1);
      } else if (node.val === node.left.val - 1) {
        inc = Math.max(inc, leftInc + 1);
      }
    }

    if (node.right) {
      const [rightInc, rightDec] = traverse(node.right);
      if (node.val === node.right.val + 1) {
        dec = Math.max(dec, rightDec + 1);
      } else if (node.val === node.right.val - 1) {
        inc = Math.max(inc, rightInc + 1);
      }
    }

    maxLength = Math.max(maxLength, inc + dec - 1);
    return [inc, dec];
  }
};
```

0551 Student Attendance Record I

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 551. Student Attendance Record I
 * https://leetcode.com/problems/student-attendance-record-i/
 * Difficulty: Easy
 *
 * You are given a string s representing an attendance record for a student
 * where each character signifies whether the student was absent, late, or
 * present on that day. The record only contains the following three characters:
 * - 'A': Absent.
 * - 'L': Late.
 * - 'P': Present.
 *
 * The student is eligible for an attendance award if they meet both of the following
criteria:
 * - The student was absent ('A') for strictly fewer than 2 days total.
 * - The student was never late ('L') for 3 or more consecutive days.
 *
 * Return true if the student is eligible for an attendance award, or false otherwise.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var checkRecord = function(s) {
  return !s.includes('LLL') && (s.indexOf('A') === s.lastIndexOf('A'));
};
```

0552 Student Attendance Record Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 552. Student Attendance Record II
 * https://leetcode.com/problems/student-attendance-record-ii/
 * Difficulty: Hard
 *
 * An attendance record for a student can be represented as a string where each character
 * signifies whether the student was absent, late, or present on that day. The record only
 * contains the following three characters:
 * - 'A': Absent.
```

```
 * - 'L': Late.
 * - 'P': Present.
 *
 * Any student is eligible for an attendance award if they meet both of the following
 * criteria:
 * - The student was absent ('A') for strictly fewer than 2 days total.
 * - The student was never late ('L') for 3 or more consecutive days.
 *
 * Given an integer n, return the number of possible attendance records of length n that make
 * a student eligible for an attendance award. The answer may be very large, so return it
 * modulo 109 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
var checkRecord = function(n) {
  const MOD = 1e9 + 7;
  const dp = [[1, 0, 0], [0, 0, 0]];

  for (let i = 0; i < n; i++) {
    const m = dp.map(row => [...row]);
    dp[0][0] = (m[0][0] + m[0][1] + m[0][2]) % MOD;
    dp[0][1] = m[0][0];
    dp[0][2] = m[0][1];
    dp[1][0] = (m[0][0] + m[0][1] + m[0][2] + m[1][0] + m[1][1] + m[1][2]) % MOD;
    dp[1][1] = m[1][0];
    dp[1][2] = m[1][1];
  }

  return (dp[0][0] + dp[0][1] + dp[0][2] + dp[1][0] + dp[1][1] + dp[1][2]) % MOD;
};
```

0553 Optimal Division

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 553. Optimal Division
 * https://leetcode.com/problems/optimal-division/
 * Difficulty: Medium
 *
 * You are given an integer array nums. The adjacent integers in nums will perform
 * the float division.
 * - For example, for nums = [2,3,4], we will evaluate the expression "2/3/4".
 *
 * However, you can add any number of parenthesis at any position to change the priority
 * of operations. You want to add these parentheses such the value of the expression after
 * the evaluation is maximum.
 *
 * Return the corresponding expression that has the maximum value in string format.
 *
 * Note: your expression should not contain redundant parenthesis.
 */

/**
 * @param {number[]} nums
 * @return {string}
 */
var optimalDivision = function(nums) {
  if (nums.length === 1) return nums[0].toString();
  if (nums.length === 2) return `${nums[0]}/${nums[1]}`;
  return `${nums[0]}/(${nums.slice(1).join('/')})`;
};
```

0554 Brick Wall

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 554. Brick Wall
 * https://leetcode.com/problems/brick-wall/
 * Difficulty: Medium
 *
 * There is a rectangular brick wall in front of you with n rows of bricks. The ith row has
 * some number of bricks each of the same height (i.e., one unit) but they can be of different
 * widths. The total width of each row is the same.
 *
 * Draw a vertical line from the top to the bottom and cross the least bricks. If your line
 * goes through the edge of a brick, then the brick is not considered as crossed. You cannot
 * draw a line just along one of the two vertical edges of the wall, in which case the line
 * will obviously cross no bricks.
 *
 * Given the 2D array wall that contains the information about the wall, return the minimum
 * number of crossed bricks after drawing such a vertical line.
 */

/**
 * @param {number[][]} wall
 * @return {number}
 */
var leastBricks = function(wall) {
  const map = new Map();
  let max = 0;

  for (const row of wall) {
    for (let i = 0, sum = 0; i < row.length - 1; i++) {
      sum += row[i];
      map.set(sum, (map.get(sum) || 0) + 1);
      max = Math.max(max, map.get(sum));
    }
  }

  return wall.length - max;
};
```

0555 Split Concatenated Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 555. Split Concatenated Strings
 * https://leetcode.com/problems/split-concatenated-strings/
 * Difficulty: Medium
 *
 * You are given an array of strings strs. You could concatenate these strings together into a
 * loop, where for each string, you could choose to reverse it or not. Among all the possible
loops
 *
 * Return the lexicographically largest string after cutting the loop, which will make the
looped
 * string into a regular one.
 *
 *
 * Specifically, to find the lexicographically largest string, you need to experience two
phases:
 * 1. Concatenate all the strings into a loop, where you can reverse some strings or not and
connect
 *    them in the same order as given.
 * 2. Cut and make one breakpoint in any place of the loop, which will make the looped string
```

```
into
 *    a regular one starting from the character at the cutpoint.
 *
 * And your job is to find the lexicographically largest one among all the possible regular
strings.
 */

/**
 * @param {string[]} strs
 * @return {string}
 */
var splitLoopedString = function(strs) {
  const normalized = strs.map(str => {
    const reversed = str.split('').reverse().join('');
    return str > reversed ? str : reversed;
  });

  let result = '';
  for (let i = 0; i < normalized.length; i++) {
    const current = normalized[i];
    const prefix = normalized.slice(i + 1).join('') + normalized.slice(0, i).join('');
    const original = current;
    const reversed = current.split('').reverse().join('');

    for (const str of [original, reversed]) {
      for (let j = 0; j <= str.length; j++) {
        const candidate = str.slice(j) + prefix + str.slice(0, j);
        if (candidate > result) {
          result = candidate;
        }
      }
    }
  }

  return result;
};
```

0556 Next Greater Element Iii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 556. Next Greater Element III
 * https://leetcode.com/problems/next-greater-element-iii/
 * Difficulty: Medium
 *
 * Given a positive integer n, find the smallest integer which has exactly the same digits
existing
 * in the integer n and is greater in value than n. If no such positive integer exists, return
 -1.
 *
 * Note that the returned integer should fit in 32-bit integer, if there is a valid answer but
 it
 * does not fit in 32-bit integer, return -1.
 */

/**
 * @param {number} n
 * @return {number}
 */
var nextGreaterElement = function(n) {
  const digits = [...String(n)];
  let i = digits.length - 2;

  while (i >= 0 && digits[i] >= digits[i + 1]) i--;
  if (i < 0) return -1;

  let j = digits.length - 1;
  while (j >= 0 && digits[j] <= digits[i]) j--;
```

```
  [digits[i], digits[j]] = [digits[j], digits[i]];
  let left = i + 1;
  let right = digits.length - 1;
  while (left < right) {
    [digits[left], digits[right]] = [digits[right], digits[left]];
    left++;
    right--;
  }

  const result = +digits.join('');
  return result > n && result <= 2**31 - 1 ? result : -1;
};
```

0557 Reverse Words In A String Iii

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 557. Reverse Words in a String III
 * https://leetcode.com/problems/reverse-words-in-a-string-iii/
 * Difficulty: Easy
 *
 * Given a string s, reverse the order of characters in each word within a sentence while
 * still preserving whitespace and initial word order.
 */

/**
 * @param {string} str
 * @return {string}
 */
var reverseWords = function(str) {
  return str.split(/\s/).map(s => s.split('').reverse().join('')).join(' ');
};
```

0558 Logical Or Of Two Binary Grids Represented As Quad Trees

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 558. Logical OR of Two Binary Grids Represented as Quad-Trees
 * https://leetcode.com/problems/logical-or-of-two-binary-grids-represented-as-quad-trees/
 * Difficulty: Medium
 *
 * A Binary Matrix is a matrix in which all the elements are either 0 or 1.
 *
 * Given quadTree1 and quadTree2. quadTree1 represents a n * n binary matrix and quadTree2
 * represents another n * n binary matrix.
 *
 * Return a Quad-Tree representing the n * n binary matrix which is the result of logical
 * bitwise OR of the two binary matrixes represented by quadTree1 and quadTree2.
 *
 * Notice that you can assign the value of a node to True or False when isLeaf is False, and
 * both are accepted in the answer.
 *
 * A Quad-Tree is a tree data structure in which each internal node has exactly four children.
 * Besides, each node has two attributes:
 * - val: True if the node represents a grid of 1's or False if the node represents a grid of
0's.
 * - isLeaf: True if the node is leaf node on the tree or False if the node has the four
children.
 * class Node {
 *     public boolean val;
 *     public boolean isLeaf;
 *     public Node topLeft;
 *     public Node topRight;
```

```
 *     public Node bottomLeft;
 *     public Node bottomRight;
 * }.
 *
 * We can construct a Quad-Tree from a two-dimensional area using the following steps:
 * 1. If the current grid has the same value (i.e all 1's or all 0's) set isLeaf True and set
val to
 *    the value of the grid and set the four children to Null and stop.
 * 2. If the current grid has different values, set isLeaf to False and set val to any value
and
 *    divide the current grid into four sub-grids as shown in the photo.
 * 3. Recurse for each of the children with the proper sub-grid.
 */

/**
 * // Definition for a QuadTree node.
 * function _Node(val,isLeaf,topLeft,topRight,bottomLeft,bottomRight) {
 *    this.val = val;
 *    this.isLeaf = isLeaf;
 *    this.topLeft = topLeft;
 *    this.topRight = topRight;
 *    this.bottomLeft = bottomLeft;
 *    this.bottomRight = bottomRight;
 * };
 */

/**
 * @param {_Node} quadTree1
 * @param {_Node} quadTree2
 * @return {_Node}
 */
var intersect = function(quadTree1, quadTree2) {
  if (quadTree1.isLeaf) return quadTree1.val ? quadTree1 : quadTree2;
  if (quadTree2.isLeaf) return quadTree2.val ? quadTree2 : quadTree1;

  const topLeft = intersect(quadTree1.topLeft, quadTree2.topLeft);
  const topRight = intersect(quadTree1.topRight, quadTree2.topRight);
  const bottomLeft = intersect(quadTree1.bottomLeft, quadTree2.bottomLeft);
  const bottomRight = intersect(quadTree1.bottomRight, quadTree2.bottomRight);

  if (topLeft.isLeaf && topRight.isLeaf && bottomLeft.isLeaf && bottomRight.isLeaf
      && topLeft.val === topRight.val && topRight.val === bottomLeft.val
      && bottomLeft.val === bottomRight.val) {
    return new _Node(topLeft.val, true, null, null, null, null);
  }

  return new _Node(false, false, topLeft, topRight, bottomLeft, bottomRight);
};
```

0559 Maximum Depth Of N Ary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 559. Maximum Depth of N-ary Tree
 * https://leetcode.com/problems/maximum-depth-of-n-ary-tree/
 * Difficulty: Easy
 *
 * Given a n-ary tree, find its maximum depth.
 *
 * The maximum depth is the number of nodes along the longest path from the root node down to
 * the farthest leaf node.
 *
 * Nary-Tree input serialization is represented in their level order traversal, each group of
 * children is separated by the null value (See examples).
 */

/**
 * // Definition for a _Node.
 * function _Node(val,children) {
```

```
 *    this.val = val === undefined ? null : val;
 *    this.children = children === undefined ? null : children;
 * };
 */

/**
 * @param {_Node|null} root
 * @return {number}
 */
var maxDepth = function(root) {
  if (!root) return 0;
  if (!root.children || !root.children.length) return 1;
  return Math.max(...root.children.map(child => maxDepth(child))) + 1;
};
```

0560 Subarray Sum Equals K

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 560. Subarray Sum Equals K
 * https://leetcode.com/problems/subarray-sum-equals-k/
 * Difficulty: Medium
 *
 * Given an array of integers nums and an integer k, return the total number of subarrays
 * whose sum equals to k.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var subarraySum = function(nums, k) {
  const map = new Map([[0, 1]]);
  let result = 0;

  for (let i = 0, count = 0; i < nums.length; i++) {
    count += nums[i];
    result += map.get(count - k) ?? 0;
    map.set(count, (map.get(count) ?? 0) + 1);
  }

  return result;
};
```

0561 Array Partition

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 561. Array Partition
 * https://leetcode.com/problems/array-partition/
 * Difficulty: Easy
 *
 * Given an integer array nums of 2n integers, group these integers into n pairs (a1, b1),
 * (a2, b2), ..., (an, bn) such that the sum of min(ai, bi) for all i is maximized.
 * Return the maximized sum.
 */

/**
 * @param {number[]} nums
 * @return {number}
```

```
 */
var arrayPairSum = function(nums) {
  return nums.sort((a, b) => a - b).reduce((sum, v, i, a) =>
    sum + (i % 2 === 0 ? Math.min(v, a[i + 1]) : 0), 0);
};
```

0562 Longest Line Of Consecutive One In Matrix

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 562. Longest Line of Consecutive One in Matrix
 * https://leetcode.com/problems/longest-line-of-consecutive-one-in-matrix/
 * Difficulty: Medium
 *
 * Given an m x n binary matrix mat, return the length of the longest line of consecutive one
 * in the matrix.
 *
 * The line could be horizontal, vertical, diagonal, or anti-diagonal.
 */

/**
 * @param {number[][]} mat
 * @return {number}
 */
var longestLine = function(mat) {
  const rows = mat.length;
  const cols = mat[0].length;
  const dp = new Array(rows).fill().map(() => new Array(cols).fill().map(() => [0, 0, 0, 0]));
  let maxLength = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (mat[i][j] === 1) {
        dp[i][j][0] = j > 0 ? dp[i][j - 1][0] + 1 : 1;
        dp[i][j][1] = i > 0 ? dp[i - 1][j][1] + 1 : 1;
        dp[i][j][2] = (i > 0 && j > 0) ? dp[i - 1][j - 1][2] + 1 : 1;
        dp[i][j][3] = (i > 0 && j < cols - 1) ? dp[i - 1][j + 1][3] + 1 : 1;
        maxLength = Math.max(maxLength, ...dp[i][j]);
      }
    }
  }

  return maxLength;
};
```

0563 Binary Tree Tilt

EASY

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 563. Binary Tree Tilt
 * https://leetcode.com/problems/binary-tree-tilt/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return the sum of every tree node's tilt.
 *
 * The tilt of a tree node is the absolute difference between the sum of all
 * left subtree node values and all right subtree node values. If a node does
 * not have a left child, then the sum of the left subtree node values is
 * treated as 0. The rule is similar if the node does not have a right child.
 */

/**
 * Definition for a binary tree node.
```

```
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var findTilt = function(root) {
  const result = { val: 0 };
  dfs(root, result);
  return result.val;
};

function dfs(root, tilt) {
  if (!root) return 0;
  const left = dfs(root.left, tilt);
  const right = dfs(root.right, tilt);
  tilt.val += Math.abs(left - right);
  return root.val + left + right;
}
```

0564 Find The Closest Palindrome

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 564. Find the Closest Palindrome
 * https://leetcode.com/problems/find-the-closest-palindrome/
 * Difficulty: Hard
 *
 * Given a string n representing an integer, return the closest integer (not including
itself),
 * which is a palindrome. If there is a tie, return the smaller one.
 *
 * The closest is defined as the absolute difference minimized between two integers.
 */

/**
 * @param {string} n
 * @return {string}
 */
var nearestPalindromic = function(n) {
  const num = BigInt(n);

  if (num <= 10n) return String(num - 1n);
  if (num === 11n) return '9';
  if (n === '1' + '0'.repeat(n.length - 1)) return String(num - 1n);
  if (n === '9'.repeat(n.length)) return String(num + 2n);

  const leftHalf = n.slice(0, Math.ceil(n.length / 2));
  const leftNum = BigInt(leftHalf);
  const candidates = [
    String(10n ** BigInt(n.length - 1) - 1n),
    createPalindrome(leftNum - 1n, n.length),
    createPalindrome(leftNum, n.length),
    createPalindrome(leftNum + 1n, n.length),
    String(10n ** BigInt(n.length) + 1n)
  ].filter(x => x !== n);

  return candidates.reduce((min, curr) => {
    const currDiff = BigInt(curr) > num ? BigInt(curr) - num : num - BigInt(curr);
    const minDiff = BigInt(min) > num ? BigInt(min) - num : num - BigInt(min);
    return currDiff < minDiff
      ? curr
      : currDiff === minDiff
        ? (curr < min ? curr : min)
        : min;
```

```
  });

  function createPalindrome(left, length) {
    const s = String(left);
    return length % 2 === 0
      ? s + s.split('').reverse().join('')
      : s + s.slice(0, -1).split('').reverse().join('');
  }
};
```

0565 Array Nesting

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 565. Array Nesting
 * https://leetcode.com/problems/array-nesting/
 * Difficulty: Medium
 *
 * A zero-indexed array A of length N contains all integers from 0 to N-1.
 * Find and return the longest length of set S, where S[i] = {A[i],
 * A[A[i]], A[A[A[i]]], ... } subjected to the rule below.
 *
 * Suppose the first element in S starts with the selection of element
 * A[i] of index = i, the next element in S should be A[A[i]], and then
 * A[A[A[i]]]… By that analogy, we stop adding right before a duplicate
 * element occurs in S.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var arrayNesting = function(nums) {
  const history = new Set();
  let max = 0;
  for (let i = 0; i < nums.length; i++) {
    let count = 0;
    let j = i;
    while (!history.has(nums[j])) {
      j = nums[j];
      history.add(j);
      count++;
    }
    max = Math.max(max, count);
  }
  return max;
};
```

0566 Reshape The Matrix

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 566. Reshape the Matrix
 * https://leetcode.com/problems/reshape-the-matrix/
 * Difficulty: Easy
 *
 * In MATLAB, there is a handy function called reshape which can reshape an m x n
 * matrix into a new one with a different size r x c keeping its original data.
 *
 * You are given an m x n matrix mat and two integers r and c representing the
 * number of rows and the number of columns of the wanted reshaped matrix.
 *
```

```
 * The reshaped matrix should be filled with all the elements of the original
 * matrix in the same row-traversing order as they were.
 *
 * If the reshape operation with given parameters is possible and legal, output
 * the new reshaped matrix; Otherwise, output the original matrix.
 */

/**
 * @param {number[][]} mat
 * @param {number} r
 * @param {number} c
 * @return {number[][]}
 */
var matrixReshape = function(mat, r, c) {
  const flat = mat.flat();
  const result = [];

  if (flat.length !== r * c) {
    return mat;
  }

  while (flat.length) {
    result.push(flat.splice(0, c));
  }

  return result;
};
```

0567 Permutation In String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 567. Permutation in String
 * https://leetcode.com/problems/permutation-in-string/
 * Difficulty: Medium
 *
 * Given two strings s1 and s2, return true if s2 contains a permutation of s1, or false
otherwise.
 *
 * In other words, return true if one of s1's permutations is the substring of s2.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var checkInclusion = function(s1, s2) {
  const getCharCode = c => c.charCodeAt() - 'a'.charCodeAt();
  const isMatch = (a1, a2) => a1.every((n, i) => a2[i] === n);

  if (s1.length > s2.length) {
    return false;
  }

  const map1 = new Array(26).fill(0);
  const map2 = new Array(26).fill(0);
  for (let i = 0; i < s1.length; i++) {
    map1[getCharCode(s1[i])]++;
    map2[getCharCode(s2[i])]++;
  }

  for (let i = 0; i < s2.length - s1.length; i++) {
    if (isMatch(map1, map2)) return true;
    map2[getCharCode(s2[i + s1.length])]++;
    map2[getCharCode(s2[i])]--;
  }

  return isMatch(map1, map2);
```

```
};
```

0568 Maximum Vacation Days

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 568. Maximum Vacation Days
 * https://leetcode.com/problems/maximum-vacation-days/
 * Difficulty: Hard
 *
 * LeetCode wants to give one of its best employees the option to travel among n cities to
collect
 * algorithm problems. But all work and no play makes Jack a dull boy, you could take
vacations in
 * some particular cities and weeks. Your job is to schedule the traveling to maximize the
number
 * of vacation days you could take, but there are certain rules and restrictions you need to
follow.
 *
 * Rules and restrictions:
 * 1. You can only travel among n cities, represented by indexes from 0 to n - 1. Initially,
you
 *    are in the city indexed 0 on Monday.
 * 2. The cities are connected by flights. The flights are represented as an n x n matrix (not
 *    necessarily symmetrical), called flights representing the airline status from the city i
 to
 *    the city j. If there is no flight from the city i to the city j, flights[i][j] == 0;
 *    Otherwise, flights[i][j] == 1. Also, flights[i][i] == 0 for all i.
 * 3. You totally have k weeks (each week has seven days) to travel. You can only take flights
 at
 *    most once per day and can only take flights on each week's Monday morning. Since flight
time
 *    is so short, we do not consider the impact of flight time.
 * 4. For each city, you can only have restricted vacation days in different weeks, given an n
 x k
 *    matrix called days representing this relationship. For the value of days[i][j], it
represents
 *    the maximum days you could take a vacation in the city i in the week j.
 * 5. You could stay in a city beyond the number of vacation days, but you should work on the
extra
 *    days, which will not be counted as vacation days.
 * 6. If you fly from city A to city B and take the vacation on that day, the deduction
towards
 *    vacation days will count towards the vacation days of city B in that week.
 * 7. We do not consider the impact of flight hours on the calculation of vacation days.
 *
 * Given the two matrices flights and days, return the maximum vacation days you could take
during
 * k weeks.
 */

/**
 * @param {number[][]} flights
 * @param {number[][]} days
 * @return {number}
 */
var maxVacationDays = function(flights, days) {
  const cities = flights.length;
  const weeks = days[0].length;
  const dp = new Array(weeks + 1).fill().map(() => new Array(cities).fill(-Infinity));

  dp[0][0] = 0;

  for (let week = 1; week <= weeks; week++) {
    for (let currCity = 0; currCity < cities; currCity++) {
      for (let prevCity = 0; prevCity < cities; prevCity++) {
        if (dp[week - 1][prevCity] !== -Infinity
            && (flights[prevCity][currCity] || prevCity === currCity)) {
          dp[week][currCity] = Math.max(
```

```
            dp[week][currCity],
            dp[week - 1][prevCity] + days[currCity][week - 1]
          );
        }
      }
    }
  }

  return Math.max(0, ...dp[weeks]);
};
```

0572 Subtree Of Another Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 572. Subtree of Another Tree
 * https://leetcode.com/problems/subtree-of-another-tree/
 * Difficulty: Easy
 *
 * Given the roots of two binary trees root and subRoot, return true if there is a subtree of
 * root with the same structure and node values of subRoot and false otherwise.
 *
 * A subtree of a binary tree tree is a tree that consists of a node in tree and all of this
 * node's descendants. The tree tree could also be considered as a subtree of itself.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} subRoot
 * @return {boolean}
 */
var isSubtree = function(root, subRoot) {
  return traverse(root).includes(traverse(subRoot));
  function traverse(node) {
    return !node ? '#' : `,${node.val},${traverse(node.left)},${traverse(node.right)}`;
  }
};
```

0573 Squirrel Simulation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 573. Squirrel Simulation
 * https://leetcode.com/problems/squirrel-simulation/
 * Difficulty: Medium
 *
 * You are given two integers height and width representing a garden of size height x width.
 * You are also given:
 * - an array tree where tree = [treer, treec] is the position of the tree in the garden,
 * - an array squirrel where squirrel = [squirrelr, squirrelc] is the position of the squirrel
 *   in the garden,
 * - and an array nuts where nuts[i] = [nutir, nutic] is the position of the ith nut in the
garden.
 *
 * The squirrel can only take at most one nut at one time and can move in four directions: up,
```

```
 down,
 * left, and right, to the adjacent cell.
 *
 * Return the minimal distance for the squirrel to collect all the nuts and put them under the
 tree
 * one by one.
 *
 * The distance is the number of moves.
 */

/**
 * @param {number} height
 * @param {number} width
 * @param {number[]} tree
 * @param {number[]} squirrel
 * @param {number[][]} nuts
 * @return {number}
 */
var minDistance = function(height, width, tree, squirrel, nuts) {
  const helper = (p1, p2) => Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] - p2[1]);

  let totalDistance = 0;
  let maxSaving = -Infinity;

  for (const nut of nuts) {
    const nutToTree = helper(nut, tree);
    totalDistance += 2 * nutToTree;
    const squirrelToNut = helper(squirrel, nut);
    maxSaving = Math.max(maxSaving, nutToTree - squirrelToNut);
  }

  return totalDistance - maxSaving;
};
```

0575 Distribute Candies

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 575. Distribute Candies
 * https://leetcode.com/problems/distribute-candies/
 * Difficulty: Easy
 *
 * Alice has n candies, where the ith candy is of type candyType[i]. Alice noticed that
 * she started to gain weight, so she visited a doctor.
 *
 * The doctor advised Alice to only eat n / 2 of the candies she has (n is always even).
 * Alice likes her candies very much, and she wants to eat the maximum number of different
 * types of candies while still following the doctor's advice.
 *
 * Given the integer array candyType of length n, return the maximum number of different
 * types of candies she can eat if she only eats n / 2 of them.
 */

/**
 * @param {number[]} candyType
 * @return {number}
 */
var distributeCandies = function(candyType) {
  return Math.min(new Set(candyType).size, candyType.length / 2);
};
```

0576 Out Of Boundary Paths

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 576. Out of Boundary Paths
 * https://leetcode.com/problems/out-of-boundary-paths/
 * Difficulty: Medium
 *
 * There is an m x n grid with a ball. The ball is initially at the position [startRow,
 * startColumn]. You are allowed to move the ball to one of the four adjacent cells in
 * the grid (possibly out of the grid crossing the grid boundary). You can apply at most
 * maxMove moves to the ball.
 *
 * Given the five integers m, n, maxMove, startRow, startColumn, return the number of
 * paths to move the ball out of the grid boundary. Since the answer can be very large,
 * return it modulo 109 + 7.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number} maxMove
 * @param {number} startRow
 * @param {number} startColumn
 * @return {number}
 */
var findPaths = function(m, n, maxMove, startRow, startColumn) {
  const MOD = 1e9 + 7;
  const directions = [[0, 1], [0, -1], [1, 0], [-1, 0]];
  const dp = new Array(maxMove + 1).fill().map(() => {
    return new Array(m).fill().map(() => new Array(n).fill(-1));
  });

  return solve(maxMove, startRow, startColumn);

  function solve(moves, row, col) {
    if (row < 0 || row >= m || col < 0 || col >= n) return 1;
    if (moves === 0) return 0;
    if (dp[moves][row][col] !== -1) return dp[moves][row][col];

    let paths = 0;
    for (const [dr, dc] of directions) {
      paths = (paths + solve(moves - 1, row + dr, col + dc)) % MOD;
    }
    return dp[moves][row][col] = paths;
  }
};
```

0581 Shortest Unsorted Continuous Subarray

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 581. Shortest Unsorted Continuous Subarray
 * https://leetcode.com/problems/shortest-unsorted-continuous-subarray/
 * Difficulty: Medium
 *
 * Given an integer array nums, you need to find one continuous subarray such that if you
 * only sort this subarray in non-decreasing order, then the whole array will be sorted
 * in non-decreasing order.
 *
 * Return the shortest such subarray and output its length.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findUnsortedSubarray = function(nums) {
  let start = -1;
  let end = -2;
```

```
  for (let i = 1, min = nums[nums.length - 1], max = nums[0]; i < nums.length; i++) {
    max = Math.max(max, nums[i]);
    min = Math.min(min, nums[nums.length - 1 - i]);
    if (nums[i] < max) {
      end = i;
    }
    if (nums[nums.length - 1 - i] > min) {
      start = nums.length - 1 - i;
    }
  }

  return end - start + 1;
};
```

0582 Kill Process

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 582. Kill Process
 * https://leetcode.com/problems/kill-process/
 * Difficulty: Medium
 *
 * You have n processes forming a rooted tree structure. You are given two integer arrays pid
 * and ppid, where pid[i] is the ID of the ith process and ppid[i] is the ID of the ith
process's
 * parent process.
 *
 * Each process has only one parent process but may have multiple children processes. Only one
 * process has ppid[i] = 0, which means this process has no parent process (the root of the
tree).
 *
 * When a process is killed, all of its children processes will also be killed.
 *
 * Given an integer kill representing the ID of a process you want to kill, return a list of
the
 * IDs of the processes that will be killed. You may return the answer in any order.
 */

/**
 * @param {number[]} pid
 * @param {number[]} ppid
 * @param {number} kill
 * @return {number[]}
 */
var killProcess = function(pid, ppid, kill) {
  const map = new Map();
  for (let i = 0; i < ppid.length; i++) {
    if (!map.has(ppid[i])) {
      map.set(ppid[i], []);
    }
    map.get(ppid[i]).push(pid[i]);
  }

  const result = [];
  traverse(kill);
  return result;

  function traverse(process) {
    result.push(process);
    if (map.has(process)) {
      for (const child of map.get(process)) {
        traverse(child);
      }
    }
  }
};
```

0583 Delete Operation For Two Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 583. Delete Operation for Two Strings
 * https://leetcode.com/problems/delete-operation-for-two-strings/
 * Difficulty: Medium
 *
 * Given two strings word1 and word2, return the minimum number of steps required to
 * make word1 and word2 the same.
 *
 * In one step, you can delete exactly one character in either string.
 */

/**
 * @param {string} word1
 * @param {string} word2
 * @return {number}
 */
var minDistance = function(word1, word2) {
  const dp = new Array(word1.length + 1).fill().map(() => {
    return new Array(word2.length + 1).fill(0);
  });

  for (let i = 1; i <= word1.length; i++) {
    dp[i][0] = i;
  }
  for (let j = 1; j <= word2.length; j++) {
    dp[0][j] = j;
  }
  for (let i = 1; i <= word1.length; i++) {
    for (let j = 1; j <= word2.length; j++) {
      dp[i][j] = word1[i - 1] === word2[j - 1]
        ? dp[i - 1][j - 1]
        : Math.min(dp[i - 1][j], dp[i][j - 1]) + 1;
    }
  }

  return dp[word1.length][word2.length];
};
```

0587 Erect The Fence

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 587. Erect the Fence
 * https://leetcode.com/problems/erect-the-fence/
 * Difficulty: Hard
 *
 * You are given an array trees where trees[i] = [xi, yi] represents the location of
 * a tree in the garden.
 *
 * Fence the entire garden using the minimum length of rope, as it is expensive. The
 * garden is well-fenced only if all the trees are enclosed.
 *
 * Return the coordinates of trees that are exactly located on the fence perimeter.
 * You may return the answer in any order.
 */

/**
 * @param {number[][]} trees
```

```
 * @return {number[][]}
 */
var outerTrees = function(trees) {
  const a = [];
  const b = [];

  trees.sort(([x1, y1], [x2, y2]) => x1 === x2 ? y1 - y2 : x1 - x2);
  for (const point of trees) {
    while (a.length >= 2 && cross(a[a.length - 2], a[a.length - 1], point) < 0) {
      a.pop();
    }
    a.push(point);
    while (b.length >= 2 && cross(b[b.length - 2], b[b.length - 1], point) > 0) {
      b.pop();
    }
    b.push(point);
  }

  return [...new Set([...a, ...b].map(p => JSON.stringify(p)))].map(p => JSON.parse(p));

  function cross(p, q, r) {
    return (q[0] - p[0]) * (r[1] - p[1]) - (q[1] - p[1]) * (r[0] - p[0]);
  }
};
```

0588 Design In Memory File System

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 588. Design In-Memory File System
 * https://leetcode.com/problems/design-in-memory-file-system/
 * Difficulty: Hard
 *
 * Design a data structure that simulates an in-memory file system.
 *
 * Implement the FileSystem class:
 * - FileSystem() Initializes the object of the system.
 * - List<String> ls(String path)
 *   - If path is a file path, returns a list that only contains this file's name.
 *   - If path is a directory path, returns the list of file and directory names in this
directory.
 *   - The answer should in lexicographic order.
 * - void mkdir(String path) Makes a new directory according to the given path. The given
directory
 *   path does not exist. If the middle directories in the path do not exist, you should
create
 *   them as well.
 * - void addContentToFile(String filePath, String content)
 *   - If filePath does not exist, creates that file containing given content.
 *   - If filePath already exists, appends the given content to original content.
 * - String readContentFromFile(String filePath) Returns the content in the file at filePath.
 */

var FileSystem = function() {
  this.root = new Map();
};

/**
 * @param {string} path
 * @return {string[]}
 */
FileSystem.prototype.ls = function(path) {
  const parts = path === '/' ? [] : path.split('/').slice(1);
  let current = this.root;

  for (const part of parts) {
    current = current.get(part);
  }
```

```
  if (typeof current === 'string') {
    return [parts[parts.length - 1]];
  }

  return Array.from(current.keys()).sort();
};

/**
 * @param {string} path
 * @return {void}
 */
FileSystem.prototype.mkdir = function(path) {
  const parts = path.split('/').slice(1);
  let current = this.root;

  for (const part of parts) {
    if (!current.has(part)) {
      current.set(part, new Map());
    }
    current = current.get(part);
  }
};

/**
 * @param {string} filePath
 * @param {string} content
 * @return {void}
 */
FileSystem.prototype.addContentToFile = function(filePath, content) {
  const parts = filePath.split('/').slice(1);
  const fileName = parts.pop();
  let current = this.root;

  for (const part of parts) {
    if (!current.has(part)) {
      current.set(part, new Map());
    }
    current = current.get(part);
  }

  const existingContent = current.get(fileName) || '';
  current.set(fileName, existingContent + content);
};

/**
 * @param {string} filePath
 * @return {string}
 */
FileSystem.prototype.readContentFromFile = function(filePath) {
  const parts = filePath.split('/').slice(1);
  const fileName = parts.pop();
  let current = this.root;

  for (const part of parts) {
    current = current.get(part);
  }

  return current.get(fileName);
};
```

0589 N Ary Tree Preorder Traversal

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 589. N-ary Tree Preorder Traversal
 * https://leetcode.com/problems/n-ary-tree-preorder-traversal/
 * Difficulty: Easy
 *
```

```
 * Given the root of an n-ary tree, return the preorder traversal of its nodes' values.
 *
 * Nary-Tree input serialization is represented in their level order traversal. Each group
 * of children is separated by the null value (See examples)
 */

/**
 * // Definition for a _Node.
 * function _Node(val, children) {
 *    this.val = val;
 *    this.children = children;
 * };
 */

/**
 * @param {_Node|null} root
 * @return {number[]}
 */
var preorder = function(root) {
  return !root ? [] : [root.val, ...root.children.flatMap(n => preorder(n))];
};
```

0590 N Ary Tree Postorder Traversal

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 590. N-ary Tree Postorder Traversal
 * https://leetcode.com/problems/n-ary-tree-postorder-traversal/
 * Difficulty: Easy
 *
 * Given the root of an n-ary tree, return the postorder traversal of its nodes' values.
 *
 * Nary-Tree input serialization is represented in their level order traversal. Each group
 * of children is separated by the null value (See examples)
 */

/**
 * // Definition for a _Node.
 * function _Node(val,children) {
 *    this.val = val;
 *    this.children = children;
 * };
 */

/**
 * @param {_Node|null} root
 * @return {number[]}
 */
var postorder = function(root) {
  const result = [];
  traverse(root);
  return result;

  function traverse(node) {
    if (!node) return;
    node.children.forEach(child => traverse(child));
    result.push(node.val);
  }
};
```

0591 Tag Validator

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 591. Tag Validator
 * https://leetcode.com/problems/tag-validator/
 * Difficulty: Hard
 *
 * Given a string representing a code snippet, implement a tag validator to parse the code and
 * return whether it is valid.
 *
 * A code snippet is valid if all the following rules hold:
 * 1. The code must be wrapped in a valid closed tag. Otherwise, the code is invalid.
 * 2. A closed tag (not necessarily valid) has exactly the following format:
 *    <TAG_NAME>TAG_CONTENT</TAG_NAME>. Among them, <TAG_NAME> is the start tag, and
</TAG_NAME>
 *    is the end tag. The TAG_NAME in start and end tags should be the same. A closed tag is
valid
 *    if and only if the TAG_NAME and TAG_CONTENT are valid.
 * 3. A valid TAG_NAME only contain upper-case letters, and has length in range [1,9].
Otherwise,
 *    the TAG_NAME is invalid.
 * 4. A valid TAG_CONTENT may contain other valid closed tags, cdata and any characters (see
 *    note1) EXCEPT unmatched <, unmatched start and end tag, and unmatched or closed tags
with
 *    invalid TAG_NAME. Otherwise, the TAG_CONTENT is invalid.
 * 5. A start tag is unmatched if no end tag exists with the same TAG_NAME, and vice versa.
 *    However, you also need to consider the issue of unbalanced when tags are nested.
 * 6. A < is unmatched if you cannot find a subsequent >. And when you find a < or </, all
 *    the subsequent characters until the next > should be parsed as TAG_NAME (not necessarily
 *    valid).
 * 7. The cdata has the following format : <![CDATA[CDATA_CONTENT]]>. The range of
CDATA_CONTENT
 *    is defined as the characters between <![CDATA[ and the first subsequent ]]>.
 * 8. CDATA_CONTENT may contain any characters. The function of cdata is to forbid the
validator
 *    to parse CDATA_CONTENT, so even it has some characters that can be parsed as tag (no
matter
 *    valid or invalid), you should treat it as regular characters.
 */

/**
 * @param {string} code
 * @return {boolean}
 */
var isValid = function(code) {
  const stack = [];

  for (let i = 0; i < code.length;) {
    if (i > 0 && !stack.length) {
      return false;
    }
    if (code.startsWith('<![CDATA[', i)) {
      const j = i + 9;
      i = code.indexOf(']]>', j);
      if (i === -1) {
        return false;
      }
      i += 3;
    } else if (code.startsWith('</', i)) {
      const j = i + 2;
      i = code.indexOf('>', j);
      if (i === -1) {
        return false;
      }
      const tag = code.slice(j, i);
      if (!stack.length || stack.pop() !== tag || !/^[A-Z]{1,9}$/.test(tag)) {
        return false;
      }
      i++;
    } else if (code[i] === '<') {
      const j = i + 1;
      i = code.indexOf('>', j);
      if (i === -1) {
        return false;
      }
      const tag = code.slice(j, i);
      if (!/^[A-Z]{1,9}$/.test(tag)) {
```

```
        return false;
      }
      stack.push(tag);
      i++;
    } else {
      i++;
    }
  }

  return !stack.length;
};
```

0592 Fraction Addition And Subtraction

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 592. Fraction Addition and Subtraction
 * https://leetcode.com/problems/fraction-addition-and-subtraction/
 * Difficulty: Medium
 *
 * Given a string expression representing an expression of fraction addition and subtraction,
 * return the calculation result in string format.
 *
 * The final result should be an irreducible fraction. If your final result is an integer,
 * change it to the format of a fraction that has a denominator 1. So in this case, 2 should
 * be converted to 2/1.
 */

/**
 * @param {string} expression
 * @return {string}
 */
var fractionAddition = function(expression) {
  const gcdCalc = (a, b) => b === 0 ? a : gcdCalc(b, a % b);
  const fractions = expression.match(/[+-]?\d+\/\d+/g);
  let n = 0;
  let d = 1;

  for (const fraction of fractions) {
    const [numerator, denominator] = fraction.split('/').map(Number);
    n = n * denominator + numerator * d;
    d = d * denominator;
    const gcd = Math.abs(gcdCalc(n, d));
    n /= gcd;
    d /= gcd;
  }

  return `${n}/${d}`;
};
```

0593 Valid Square

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 593. Valid Square
 * https://leetcode.com/problems/valid-square/
 * Difficulty: Medium
 *
 * Given the coordinates of four points in 2D space p1, p2, p3 and p4, return true if the
 * four points construct a square.
 *
 * The coordinate of a point pi is represented as [xi, yi]. The input is not given in any
order.
```

```
 *
 * A valid square has four equal sides with positive length and four equal angles (90-degree
 * angles).
 */

/**
 * @param {number[]} p1
 * @param {number[]} p2
 * @param {number[]} p3
 * @param {number[]} p4
 * @return {boolean}
 */
var validSquare = function(p1, p2, p3, p4) {
  const helper = (a, b) => (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2;
  const points = [p1, p2, p3, p4];
  const set = new Set();

  for (let i = 0; i < 4; i++) {
    for (let j = i + 1; j < 4; j++) {
      const d = helper(points[i], points[j]);
      if (!d) {
        return false;
      }
      set.add(d);
    }
  }

  return set.size === 2;
};
```

0594 Longest Harmonious Subsequence

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 594. Longest Harmonious Subsequence
 * https://leetcode.com/problems/longest-harmonious-subsequence/
 * Difficulty: Easy
 *
 * We define a harmonious array as an array where the difference between its maximum
 * value and its minimum value is exactly 1.
 *
 * Given an integer array nums, return the length of its longest harmonious subsequence
 * among all its possible subsequences.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findLHS = function(nums) {
  const map = new Map();
  nums.forEach(n => map.set(n, (map.get(n) ?? 0) + 1));

  return Array.from(map).reduce((result, [n, count]) => {
    return map.has(n + 1) ? Math.max(result, count + map.get(n + 1)) : result;
  }, 0);
};
```

0598 Range Addition Ii

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 598. Range Addition II
```

```
 * https://leetcode.com/problems/range-addition-ii/
 * Difficulty: Easy
 *
 * You are given an m x n matrix M initialized with all 0's and an array of operations ops,
 * where ops[i] = [ai, bi] means M[x][y] should be incremented by one for all 0 <= x < ai
 * and 0 <= y < bi.
 *
 * Count and return the number of maximum integers in the matrix after performing all the
 * operations.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number[][]} ops
 * @return {number}
 */
var maxCount = function(m, n, ops) {
  let a = m;
  let b = n;

  for (const op of ops) {
    a = Math.min(a, op[0]);
    b = Math.min(b, op[1]);
  }

  return a * b;
};
```

0599 Minimum Index Sum Of Two Lists

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 599. Minimum Index Sum of Two Lists
 * https://leetcode.com/problems/minimum-index-sum-of-two-lists/
 * Difficulty: Easy
 *
 * Suppose Andy and Doris want to choose a restaurant for dinner, and they both have a list
 * of favorite restaurants represented by strings.
 *
 * You need to help them find out their common interest with the least list index sum.
 * If there is a choice tie between answers, output all of them with no order requirement.
 * You could assume there always exists an answer.
 */

/**
 * @param {string[]} list1
 * @param {string[]} list2
 * @return {string[]}
 */
var findRestaurant = function(list1, list2) {
  const map = new Map(list1.map((str, index) => [str, index]));

  return list2
    .map((str, index) => map.has(str) ? [map.get(str) + index, str] : null)
    .filter(Boolean)
    .sort(([a], [b]) => a - b)
    .filter(([sum], index, [[lowest]]) => sum === lowest)
    .map(([, str]) => str);
};
```

0600 Non Negative Integers Without Consecutive Ones

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 600. Non-negative Integers without Consecutive Ones
 * https://leetcode.com/problems/non-negative-integers-without-consecutive-ones/
 * Difficulty: Hard
 *
 * Given a positive integer n, return the number of the integers in the range [0, n] whose
 * binary representations do not contain consecutive ones.
 */

/**
 * @param {number} n
 * @return {number}
 */
var findIntegers = function(n) {
  const binary = n.toString(2);
  const dp = new Array(binary.length + 1).fill(0);
  let result = 0;

  dp[0] = 1;
  dp[1] = 2;

  for (let i = 2; i <= binary.length; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
  }

  for (let i = 0, previous = 0; i < binary.length; i++) {
    if (binary[i] === '1') {
      result += dp[binary.length - i - 1];
      if (previous === 1) {
        return result;
      }
      previous = 1;
    } else {
      previous = 0;
    }
  }

  return result + 1;
};
```

0604 Design Compressed String Iterator

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 604. Design Compressed String Iterator
 * https://leetcode.com/problems/design-compressed-string-iterator/
 * Difficulty: Easy
 *
 * Design and implement a data structure for a compressed string iterator. The given
compressed
 * string will be in the form of each letter followed by a positive integer representing the
 * number of this letter existing in the original uncompressed string.
 *
 * Implement the StringIterator class:
 * - next() Returns the next character if the original string still has uncompressed
characters,
 *   otherwise returns a white space.
 * - hasNext() Returns true if there is any letter needs to be uncompressed in the original
string,
 *   otherwise returns false.
 */

/**
 * @param {string} compressedString
 */
var StringIterator = function(compressedString) {
  this.segments = [];
  this.currentIndex = 0;
```

```
  this.currentCount = 0;

  let i = 0;
  while (i < compressedString.length) {
    const character = compressedString[i];
    i++;
    let count = '';
    while (i < compressedString.length && /\d/.test(compressedString[i])) {
      count += compressedString[i];
      i++;
    }
    this.segments.push([character, parseInt(count)]);
  }
};

/**
 * @return {character}
 */
StringIterator.prototype.next = function() {
  if (!this.hasNext()) {
    return ' ';
  }

  if (this.currentCount === 0) {
    this.currentCount = this.segments[this.currentIndex][1];
  }

  const character = this.segments[this.currentIndex][0];
  this.currentCount--;

  if (this.currentCount === 0) {
    this.currentIndex++;
  }

  return character;
};

/**
 * @return {boolean}
 */
StringIterator.prototype.hasNext = function() {
  return this.currentIndex < this.segments.length;
};
```

0605 Can Place Flowers

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 605. Can Place Flowers
 * https://leetcode.com/problems/can-place-flowers/
 * Difficulty: Easy
 *
 * You have a long flowerbed in which some of the plots are planted, and some are not.
 * However, flowers cannot be planted in adjacent plots.
 *
 * Given an integer array flowerbed containing 0's and 1's, where 0 means empty and 1
 * means not empty, and an integer n, return true if n new flowers can be planted in
 * the flowerbed without violating the no-adjacent-flowers rule and false otherwise.
 */

/**
 * @param {number[]} flowerbed
 * @param {number} n
 * @return {boolean}
 */
var canPlaceFlowers = function(flowerbed, n) {
  let count = 0;
  for (let i = 0; i < flowerbed.length; i++) {
    if (flowerbed[i] === 0 && flowerbed[i - 1] !== 1 && flowerbed[i + 1] !== 1) {
```

```
      count++;
      flowerbed[i] = 1;
    }
  }
  return count >= n;
};
```

0606 Construct String From Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 606. Construct String from Binary Tree
 * https://leetcode.com/problems/construct-string-from-binary-tree/
 * Difficulty: Easy
 *
 * You need to construct a string consists of parenthesis and integers
 * from a binary tree with the preorder traversing way.
 *
 * The null node needs to be represented by empty parenthesis pair "()".
 * And you need to omit all the empty parenthesis pairs that don't
 * affect the one-to-one mapping relationship between the string
 * and the original binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} t
 * @return {string}
 */
var tree2str = function(t) {
  return !t ? '' :
`${t.val}(${tree2str(t.left)})(${tree2str(t.right)})`.replace(/(\(\)){2}|\(\)(?=$|\))/g, '');
};


// alternative, longer but faster version:
/**
 * @param {TreeNode} t
 * @return {string}
 */
var tree2str = function(t) {
  let str = t ? `${t.val}` : '';
  if (t && t.right) str += `(${tree2str(t.left)})(${tree2str(t.right)})`;
  else if (t && t.left) str += `(${tree2str(t.left)})`;
  return str;
};
```

0609 Find Duplicate File In System

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 609. Find Duplicate File in System
 * https://leetcode.com/problems/find-duplicate-file-in-system/
 * Difficulty: Medium
 *
 * Given a list paths of directory info, including the directory path, and all the files with
 * contents in this directory, return all the duplicate files in the file system in terms of
```

```
 * their paths. You may return the answer in any order.
 *
 * A group of duplicate files consists of at least two files that have the same content.
 *
 * A single directory info string in the input list has the following format:
 * - "root/d1/d2/.../dm f1.txt(f1_content) f2.txt(f2_content) ... fn.txt(fn_content)"
 *
 * It means there are n files (f1.txt, f2.txt ... fn.txt) with content (f1_content, f2_content
 * ... fn_content) respectively in the directory "root/d1/d2/.../dm". Note that n >= 1 and
 * m >= 0. If m = 0, it means the directory is just the root directory.
 *
 * The output is a list of groups of duplicate file paths. For each group, it contains all the
 * file paths of the files that have the same content. A file path is a string that has the
 * following format:
 * - "directory_path/file_name.txt"
 */

/**
 * @param {string[]} paths
 * @return {string[][]}
 */
var findDuplicate = function(paths) {
  const map = new Map();

  for (const path of paths) {
    const [directory, ...files] = path.split(' ');
    for (const file of files) {
      const [name, content] = file.split('(');
      const fullPath = `${directory}/${name}`;
      const fileContent = content.slice(0, -1);
      map.set(fileContent, (map.get(fileContent) || []).concat(fullPath));
    }
  }

  return Array.from(map.values()).filter(group => group.length > 1);
};
```

0611 Valid Triangle Number

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 611. Valid Triangle Number
 * https://leetcode.com/problems/valid-triangle-number/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the number of triplets chosen from the array that
 * can make triangles if we take them as side lengths of a triangle.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var triangleNumber = function(nums) {
  let count = 0;

  nums.sort((a, b) => a - b);

  for (let k = nums.length - 1; k >= 2; k--) {
    for (let i = 0, j = k - 1; i < j;) {
      if (nums[i] + nums[j] > nums[k]) {
        count += j - i;
        j--;
      } else {
        i++;
      }
    }
  }
```

```
  return count;
};
```

0616 Add Bold Tag In String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 616. Add Bold Tag in String
 * https://leetcode.com/problems/add-bold-tag-in-string/
 * Difficulty: Medium
 *
 * You are given a string s and an array of strings words.
 *
 * You should add a closed pair of bold tag <b> and </b> to wrap the substrings in s that
 * exist in words.
 * - If two such substrings overlap, you should wrap them together with only one pair of
 *   closed bold-tag.
 * - If two substrings wrapped by bold tags are consecutive, you should combine them.
 *
 * Return s after adding the bold tags.
 */

/**
 * @param {string} s
 * @param {string[]} words
 * @return {string}
 */
var addBoldTag = function(s, words) {
  const boldIntervals = [];

  for (const word of words) {
    let start = s.indexOf(word);
    while (start !== -1) {
      boldIntervals.push([start, start + word.length]);
      start = s.indexOf(word, start + 1);
    }
  }

  if (!boldIntervals.length) return s;

  boldIntervals.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  const mergedIntervals = [];
  let [currentStart, currentEnd] = boldIntervals[0];

  for (let i = 1; i < boldIntervals.length; i++) {
    const [nextStart, nextEnd] = boldIntervals[i];
    if (nextStart <= currentEnd) {
      currentEnd = Math.max(currentEnd, nextEnd);
    } else {
      mergedIntervals.push([currentStart, currentEnd]);
      [currentStart, currentEnd] = [nextStart, nextEnd];
    }
  }
  mergedIntervals.push([currentStart, currentEnd]);

  let result = '';
  let lastEnd = 0;
  for (const [start, end] of mergedIntervals) {
    result += s.slice(lastEnd, start) + '<b>' + s.slice(start, end) + '</b>';
    lastEnd = end;
  }

  result += s.slice(lastEnd);

  return result;
};
```

0617 Merge Two Binary Trees

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 617. Merge Two Binary Trees
 * https://leetcode.com/problems/merge-two-binary-trees/
 * Difficulty: Easy
 *
 * Given two binary trees and imagine that when you put one
 * of them to cover the other, some nodes of the two trees
 * are overlapped while the others are not.
 *
 * You need to merge them into a new binary tree.
 * The merge rule is that if two nodes overlap, then sum
 * node values up as the new value of the merged node.
 * Otherwise, the NOT null node will be used as the
 * node of new tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} t1
 * @param {TreeNode} t2
 * @return {TreeNode}
 */
var mergeTrees = function(t1, t2) {
  if (!t1) return t2;
  if (!t2) return t1;

  t1.val += t2.val;
  t1.left = mergeTrees(t1.left, t2.left);
  t1.right = mergeTrees(t1.right, t2.right);

  return t1;
};
```

0621 Task Scheduler

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 621. Task Scheduler
 * https://leetcode.com/problems/task-scheduler/
 * Difficulty: Medium
 *
 * You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number
n.
 * Each CPU interval can be idle or allow the completion of one task. Tasks can be completed
 * in any order, but there's a constraint: there has to be a gap of at least n intervals
 * between two tasks with the same label.
 *
 * Return the minimum number of CPU intervals required to complete all tasks.
 */

/**
 * @param {character[]} tasks
 * @param {number} n
 * @return {number}
```

```
 */
var leastInterval = function(tasks, n) {
  const map = new Map();
  let maxValue = 0;
  let maxCount = 0;

  tasks.forEach(key => {
    const value = map.has(key) ? map.get(key) + 1 : 1;
    map.set(key, value);

    if (value > maxValue) {
      maxValue = value;
      maxCount = 1;
    } else if (value === maxValue) {
      maxCount++;
    }
  });

  return Math.max(tasks.length, (maxValue - 1) * (n + 1) + maxCount);
};
```

0622 Design Circular Queue

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 622. Design Circular Queue
 * https://leetcode.com/problems/design-circular-queue/
 * Difficulty: Medium
 *
 * Design your implementation of the circular queue. The circular queue is a linear data
structure
 * in which the operations are performed based on FIFO (First In First Out) principle, and the
 * last position is connected back to the first position to make a circle. It is also called
 * "Ring Buffer".
 *
 * One of the benefits of the circular queue is that we can make use of the spaces in front of
 * the queue. In a normal queue, once the queue becomes full, we cannot insert the next
element
 * even if there is a space in front of the queue. But using the circular queue, we can use
the
 * space to store new values.
 *
 * Implement the MyCircularQueue class:
 * - MyCircularQueue(k) Initializes the object with the size of the queue to be k.
 * - int Front() Gets the front item from the queue. If the queue is empty, return -1.
 * - int Rear() Gets the last item from the queue. If the queue is empty, return -1.
 * - boolean enQueue(int value) Inserts an element into the circular queue. Return true if the
 *   operation is successful.
 * - boolean deQueue() Deletes an element from the circular queue. Return true if the
operation
 *   is successful.
 * - boolean isEmpty() Checks whether the circular queue is empty or not.
 * - boolean isFull() Checks whether the circular queue is full or not.
 *
 * You must solve the problem without using the built-in queue data structure in your
programming
 * language.
 */

/**
 * @param {number} k
 */
var MyCircularQueue = function(k) {
  this.queue = new Array(k);
  this.size = k;
  this.front = -1;
  this.rear = -1;
};
```

```
/**
 * @param {number} value
 * @return {boolean}
 */
MyCircularQueue.prototype.enQueue = function(value) {
  if (this.isFull()) {
    return false;
  }
  if (this.isEmpty()) {
    this.front = 0;
  }
  this.rear = (this.rear + 1) % this.size;
  this.queue[this.rear] = value;
  return true;
};

/**
 * @return {boolean}
 */
MyCircularQueue.prototype.deQueue = function() {
  if (this.isEmpty()) {
    return false;
  } else if (this.front === this.rear) {
    this.front = -1;
    this.rear = -1;
  } else {
    this.front = (this.front + 1) % this.size;
  }
  return true;
};

/**
 * @return {number}
 */
MyCircularQueue.prototype.Front = function() {
  return this.isEmpty() ? -1 : this.queue[this.front];
};

/**
 * @return {number}
 */
MyCircularQueue.prototype.Rear = function() {
  return this.isEmpty() ? -1 : this.queue[this.rear];
};

/**
 * @return {boolean}
 */
MyCircularQueue.prototype.isEmpty = function() {
  return this.front === -1;
};

/**
 * @return {boolean}
 */
MyCircularQueue.prototype.isFull = function() {
  return (this.rear + 1) % this.size === this.front;
};
```

0623 Add One Row To Tree

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 623. Add One Row to Tree
 * https://leetcode.com/problems/add-one-row-to-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and two integers val and depth, add a row of nodes with
value
```

```
 * val at the given depth depth.
 *
 * Note that the root node is at depth 1.
 *
 * The adding rule is:
 * - Given the integer depth, for each not null tree node cur at the depth depth - 1, create
two
 *   tree nodes with value val as cur's left subtree root and right subtree root.
 * - cur's original left subtree should be the left subtree of the new left subtree root.
 * - cur's original right subtree should be the right subtree of the new right subtree root.
 * - If depth == 1 that means there is no depth depth - 1 at all, then create a tree node with
 *   value val as the new root of the whole original tree, and the original tree is the new
root's
 *   left subtree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} val
 * @param {number} depth
 * @return {TreeNode}
 */
var addOneRow = function(root, val, depth) {
  if (depth === 1) {
    return new TreeNode(val, root);
  }
  dfs(root, 1);

  return root;

  function dfs(node, level) {
    if (!node) return;
    if (level === depth - 1) {
      node.left = new TreeNode(val, node.left);
      node.right = new TreeNode(val, null, node.right);
      return;
    }
    dfs(node.left, level + 1);
    dfs(node.right, level + 1);
  }
};
```

0624 Maximum Distance In Arrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 624. Maximum Distance in Arrays
 * https://leetcode.com/problems/maximum-distance-in-arrays/
 * Difficulty: Medium
 *
 * You are given m arrays, where each array is sorted in ascending order.
 *
 * You can pick up two integers from two different arrays (each array picks one) and calculate
 * the distance. We define the distance between two integers a and b to be their absolute
 * difference |a - b|.
 *
 * Return the maximum distance.
 */

/**
 * @param {number[][]} arrays
```

```
 * @return {number}
 */
var maxDistance = function(arrays) {
  let result = 0;
  let min = arrays[0][0];
  let max = arrays[0][arrays[0].length - 1];

  for (let i = 1; i < arrays.length; i++) {
    const newMin = arrays[i][0];
    const newMax = arrays[i][arrays[i].length - 1];
    result = Math.max(result, Math.abs(newMax - min), Math.abs(max - newMin));
    min = Math.min(min, newMin);
    max = Math.max(max, newMax);
  }

  return result;
};
```

0625 Minimum Factorization

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 625. Minimum Factorization
 * https://leetcode.com/problems/minimum-factorization/
 * Difficulty: Medium
 *
 * Given a positive integer num, return the smallest positive integer x whose multiplication
 * of each digit equals num. If there is no answer or the answer is not fit in 32-bit signed
 * integer, return 0.
 */

/**
 * @param {number} num
 * @return {number}
 */
var smallestFactorization = function(num) {
  if (num < 2) return num;

  const digits = [];
  for (let divisor = 9; divisor > 1; divisor--) {
    while (num % divisor === 0) {
      digits.push(divisor);
      num /= divisor;
    }
  }

  if (num > 1) return 0;

  let result = 0;
  for (const digit of digits.reverse()) {
    result = result * 10 + digit;
    if (result > 2**31 - 1) return 0;
  }

  return result;
};
```

0628 Maximum Product Of Three Numbers

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 628. Maximum Product of Three Numbers
 * https://leetcode.com/problems/maximum-product-of-three-numbers/
```

```
 * Difficulty: Easy
 *
 * Given an integer array, find three numbers whose
 * product is maximum and output the maximum product.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumProduct = function(nums) {
  const n = nums.length;
  nums.sort((a, b) => a - b);
  return Math.max(nums[0] * nums[1] * nums[n - 1], nums[n - 1] * nums[n - 2] * nums[n - 3]);
};
```

0629 K Inverse Pairs Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 629. K Inverse Pairs Array
 * https://leetcode.com/problems/k-inverse-pairs-array/
 * Difficulty: Hard
 *
 * For an integer array nums, an inverse pair is a pair of integers [i, j] where
 * 0 <= i < j < nums.length and nums[i] > nums[j].
 *
 * Given two integers n and k, return the number of different arrays consisting of numbers
from
 * 1 to n such that there are exactly k inverse pairs. Since the answer can be huge, return
 * it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var kInversePairs = function(n, k) {
  const MOD = 1e9 + 7;
  const dp = new Array(k + 1).fill(0);

  dp[0] = 1;
  for (let i = 1; i <= n; i++) {
    const previous = dp.slice();
    dp[0] = 1;
    for (let j = 1; j <= k; j++) {
      dp[j] = (previous[j] + dp[j-1]) % MOD;
      if (j >= i) dp[j] = (dp[j] - previous[j-i] + MOD) % MOD;
    }
  }

  return dp[k];
};
```

0630 Course Schedule Iii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 630. Course Schedule III
 * https://leetcode.com/problems/course-schedule-iii/
 * Difficulty: Hard
 *
```

```
 * There are n different online courses numbered from 1 to n. You are given an array courses
 * where courses[i] = [durationi, lastDayi] indicate that the ith course should be taken
 * continuously for durationi days and must be finished before or on lastDayi.
 *
 * You will start on the 1st day and you cannot take two or more courses simultaneously.
 *
 * Return the maximum number of courses that you can take.
 */

/**
 * @param {number[][]} courses
 * @return {number}
 */
var scheduleCourse = function(courses) {
  courses.sort((a, b) => a[1] - b[1]);

  const maxHeap = new PriorityQueue({ compare: (a, b) => b - a });
  let value = 0;

  for (const [duration, lastDay] of courses) {
    if (value + duration <= lastDay) {
      value += duration;
      maxHeap.enqueue(duration);
    } else {
      if (maxHeap.front() > duration) {
        var prev = maxHeap.dequeue();
        value -= prev;
        value += duration;
        maxHeap.enqueue(duration);
      }
    }
  }

  return maxHeap.size();
};
```

0631 Design Excel Sum Formula

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 631. Design Excel Sum Formula
 * https://leetcode.com/problems/design-excel-sum-formula/
 * Difficulty: Hard
 *
 * Design the basic function of Excel and implement the function of the sum formula.
 *
 * Implement the Excel class:
 * - Excel(int height, char width) Initializes the object with the height and the width
 *   of the sheet. The sheet is an integer matrix mat of size height x width with the row
 *   index in the range [1, height] and the column index in the range ['A', width]. All the
 *   values should be zero initially.
 * - void set(int row, char column, int val) Changes the value at mat[row][column] to be val.
 * - int get(int row, char column) Returns the value at mat[row][column].
 * - int sum(int row, char column, List<String> numbers) Sets the value at mat[row][column] to
 *   be the sum of cells represented by numbers and returns the value at mat[row][column].
 *   This sum formula should exist until this cell is overlapped by another value or another
 *   sum formula. numbers[i] could be on the format:
 *   - "ColRow" that represents a single cell.
 *     - For example, "F7" represents the cell mat[7]['F'].
 *   - "ColRow1:ColRow2" that represents a range of cells. The range will always be a
rectangle
 *     where "ColRow1" represent the position of the top-left cell, and "ColRow2" represents
the
 *     position of the bottom-right cell.
 *     - For example, "B3:F7" represents the cells mat[i][j] for 3 <= i <= 7 and 'B' <= j <=
'F'.
 *
 * Note: You could assume that there will not be any circular sum reference.
 *
```

```
 * For example, mat[1]['A'] == sum(1, "B") and mat[1]['B'] == sum(1, "A").
 */

/**
 * @param {number} height
 * @param {character} width
 */
var Excel = function(height, width) {
  this.height = height;
  this.width = width.charCodeAt(0) - 64;
  this.matrix = new Array(height).fill().map(() => new Array(this.width).fill(0));
  this.formulas = new Map();
  this.dependencies = new Map();
};

/**
 * @param {number} row
 * @param {character} column
 * @param {number} val
 * @return {void}
 */
Excel.prototype.set = function(row, column, val) {
  const col = column.charCodeAt(0) - 65;
  const key = `${row}:${column}`;
  this.matrix[row - 1][col] = val;
  if (this.formulas.has(key)) {
    this.formulas.delete(key);
    this.dependencies.get(key)?.delete(key);
  }
  this.updateDependents(row, column);
};

/**
 * @param {number} row
 * @param {character} column
 * @return {number}
 */
Excel.prototype.get = function(row, column) {
  const col = column.charCodeAt(0) - 65;
  return this.matrix[row - 1][col];
};

/**
 * @param {number} row
 * @param {character} column
 * @param {string[]} numbers
 * @return {number}
 */
Excel.prototype.sum = function(row, column, numbers) {
  const col = column.charCodeAt(0) - 65;
  const cellKey = `${row}:${column}`;
  const cells = [];

  for (const num of numbers) {
    if (num.includes(':')) {
      const [start, end] = num.split(':');
      const startCol = start[0].charCodeAt(0) - 65;
      const startRow = parseInt(start.slice(1)) - 1;
      const endCol = end[0].charCodeAt(0) - 65;
      const endRow = parseInt(end.slice(1)) - 1;

      for (let r = startRow; r <= endRow; r++) {
        for (let c = startCol; c <= endCol; c++) {
          cells.push([r, c]);
        }
      }
    } else {
      const c = num[0].charCodeAt(0) - 65;
      const r = parseInt(num.slice(1)) - 1;
      cells.push([r, c]);
    }
  }

  if (this.formulas.has(cellKey)) {
    for (const [r, c] of this.formulas.get(cellKey)) {
```

```
      const depKey = `${r + 1}:${String.fromCharCode(c + 65)}`;
      this.dependencies.get(depKey)?.delete(cellKey);
    }
  }

  this.formulas.set(cellKey, cells);
  for (const [r, c] of cells) {
    const depKey = `${r + 1}:${String.fromCharCode(c + 65)}`;
    if (!this.dependencies.has(depKey)) {
      this.dependencies.set(depKey, new Set());
    }
    this.dependencies.get(depKey).add(cellKey);
  }

  const total = this.calculateSum(cells);
  this.matrix[row - 1][col] = total;
  this.updateDependents(row, column);
  return total;
};

Excel.prototype.calculateSum = function(cells) {
  return cells.reduce((sum, [r, c]) => sum + this.matrix[r][c], 0);
};

Excel.prototype.updateDependents = function(row, column) {
  const key = `${row}:${column}`;
  if (!this.dependencies.has(key)) return;

  for (const depKey of this.dependencies.get(key)) {
    const [depRow, depCol] = depKey.split(':');
    const col = depCol.charCodeAt(0) - 65;
    const cells = this.formulas.get(depKey);
    if (cells) {
      const newSum = this.calculateSum(cells);
      this.matrix[parseInt(depRow) - 1][col] = newSum;
      this.updateDependents(parseInt(depRow), depCol);
    }
  }
};
```

0632 Smallest Range Covering Elements From K Lists

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 632. Smallest Range Covering Elements from K Lists
 * https://leetcode.com/problems/smallest-range-covering-elements-from-k-lists/
 * Difficulty: Hard
 *
 * You have k lists of sorted integers in non-decreasing order. Find the smallest range that
 * includes at least one number from each of the k lists.
 *
 * We define the range [a, b] is smaller than range [c, d] if b - a < d - c or a < c
 * if b - a == d - c.
 */

/**
 * @param {number[][]} nums
 * @return {number[]}
 */
var smallestRange = function(nums) {
  const k = nums.length;
  const listCountMap = new Map();
  let coveredLists = 0;
  let minDiff = Infinity;
  let minStart;
  let minEnd;

  const allElements = nums.flatMap((list, listIndex) =>
```

```
    list.map(num => ({ num, index: listIndex }))
  ).sort((a, b) => a.num - b.num);

  let windowStart = 0;
  for (let windowEnd = 0; windowEnd < allElements.length; windowEnd++) {
    const currentElement = allElements[windowEnd];
    const listIndex = currentElement.index;
    const count = listCountMap.get(listIndex) ?? 0;

    if (count === 0) coveredLists += 1;
    listCountMap.set(listIndex, count + 1);

    while (coveredLists === k) {
      const leftElement = allElements[windowStart];
      const range = currentElement.num - leftElement.num;

      if (range < minDiff) {
        minDiff = range;
        minStart = leftElement.num;
        minEnd = currentElement.num;
      }
      const leftListIndex = leftElement.index;
      const leftCount = listCountMap.get(leftListIndex);
      listCountMap.set(leftListIndex, leftCount - 1);
      if (leftCount - 1 === 0) coveredLists -= 1;
      windowStart += 1;
    }
  }

  return [minStart, minEnd];
};
```

0633 Sum Of Square Numbers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 633. Sum of Square Numbers
 * https://leetcode.com/problems/sum-of-square-numbers/
 * Difficulty: Medium
 *
 * Given a non-negative integer c, decide whether there're two integers a and b such
 * that a2 + b2 = c.
 */

/**
 * @param {number} c
 * @return {boolean}
 */
var judgeSquareSum = function(c) {
  for (let left = 0, right = Math.floor(Math.sqrt(c)); left <= right;) {
    const sum = left * left + right * right;
    if (sum === c) return true;
    if (sum < c) left++;
    else right--;
  }
  return false;
};
```

0634 Find The Derangement Of An Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 634. Find the Derangement of An Array
```

```
 * https://leetcode.com/problems/find-the-derangement-of-an-array/
 * Difficulty: Medium
 *
 * In combinatorial mathematics, a derangement is a permutation of the elements of a set,
 * such that no element appears in its original position.
 *
 * You are given an integer n. There is originally an array consisting of n integers from
 * 1 to n in ascending order, return the number of derangements it can generate. Since the
 * answer may be huge, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
var findDerangement = function(n) {
  const MOD = 1e9 + 7;
  if (n === 1) return 0;
  if (n === 2) return 1;

  let prev2 = 0;
  let prev1 = 1;
  for (let i = 3; i <= n; i++) {
    const current = ((i - 1) * (prev1 + prev2)) % MOD;
    prev2 = prev1;
    prev1 = current;
  }

  return prev1;
};
```

0635 Design Log Storage System

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 635. Design Log Storage System
 * https://leetcode.com/problems/design-log-storage-system/
 * Difficulty: Medium
 *
 * You are given several logs, where each log contains a unique ID and timestamp. Timestamp is
 a
 * string that has the following format: Year:Month:Day:Hour:Minute:Second, for example,
 * 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.
 *
 * Implement the LogSystem class:
 * - LogSystem() Initializes the LogSystem object.
 * - void put(int id, string timestamp) Stores the given log (id, timestamp) in your storage
system.
 * - int[] retrieve(string start, string end, string granularity) Returns the IDs of the logs
whose
 *   timestamps are within the range from start to end inclusive. start and end all have the
same
 *   format as timestamp, and granularity means how precise the range should be (i.e. to the
exact
 *   Day, Minute, etc.). For example, start = "2017:01:01:23:59:59", end =
"2017:01:02:23:59:59",
 *   and granularity = "Day" means that we need to find the logs within the inclusive range
from
 *   Jan. 1st 2017 to Jan. 2nd 2017, and the Hour, Minute, and Second for each log entry can
be
 *   ignored.
 */

var LogSystem = function() {
  this.logs = [];
  this.granularities = {
    Year: 4,
    Month: 7,
    Day: 10,
```

```
    Hour: 13,
    Minute: 16,
    Second: 19
  };
};

/**
 * @param {number} id
 * @param {string} timestamp
 * @return {void}
 */
LogSystem.prototype.put = function(id, timestamp) {
  this.logs.push([timestamp, id]);
};

/**
 * @param {string} start
 * @param {string} end
 * @param {string} granularity
 * @return {number[]}
 */
LogSystem.prototype.retrieve = function(start, end, granularity) {
  const precision = this.granularities[granularity];
  const startKey = start.slice(0, precision).padEnd(19, '0');
  const endKey = end.slice(0, precision).padEnd(19, '9');

  const result = [];
  for (const [timestamp, id] of this.logs) {
    const key = timestamp.slice(0, precision).padEnd(19, '0');
    if (key >= startKey && key <= endKey) {
      result.push(id);
    }
  }

  return result.sort((a, b) => a - b);
};
```

0636 Exclusive Time Of Functions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 636. Exclusive Time of Functions
 * https://leetcode.com/problems/exclusive-time-of-functions/
 * Difficulty: Medium
 *
 * On a single-threaded CPU, we execute a program containing n functions. Each function has a
 * unique ID between 0 and n-1.
 *
 * Function calls are stored in a call stack: when a function call starts, its ID is pushed
 * onto the stack, and when a function call ends, its ID is popped off the stack. The function
 * whose ID is at the top of the stack is the current function being executed. Each time a
 * function starts or ends, we write a log with the ID, whether it started or ended, and the
 * timestamp.
 *
 * You are given a list logs, where logs[i] represents the ith log message formatted as a
string
 * "{function_id}:{"start" | "end"}:{timestamp}". For example, "0:start:3" means a function
call
 * with function ID 0 started at the beginning of timestamp 3, and "1:end:2" means a function
 * call with function ID 1 ended at the end of timestamp 2. Note that a function can be called
 * multiple times, possibly recursively.
 *
 * A function's exclusive time is the sum of execution times for all function calls in the
program.
 * For example, if a function is called twice, one call executing for 2 time units and another
 * call executing for 1 time unit, the exclusive time is 2 + 1 = 3.
 *
 * Return the exclusive time of each function in an array, where the value at the ith index
 * represents the exclusive time for the function with ID i.
```

```
 */

/**
 * @param {number} n
 * @param {string[]} logs
 * @return {number[]}
 */
var exclusiveTime = function(n, logs) {
  const stack = [];
  const result = new Array(n).fill(0);
  let previousTime = 0;

  for (const log of logs) {
    const [id, action, time] = log.split(':');
    const currentTime = +time;

    if (action === 'start') {
      if (stack.length) {
        result[stack[stack.length - 1]] += currentTime - previousTime;
      }
      stack.push(+id);
      previousTime = currentTime;
    } else {
      result[stack.pop()] += currentTime - previousTime + 1;
      previousTime = currentTime + 1;
    }
  }

  return result;
};
```

0637 Average Of Levels In Binary Tree

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 637. Average of Levels in Binary Tree
 * https://leetcode.com/problems/average-of-levels-in-binary-tree/
 * Difficulty: Easy
 *
 * Given the root of a binary tree, return the average value of the nodes on each level in the
 * form of an array. Answers within 10-5 of the actual answer will be accepted.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var averageOfLevels = function(root) {
  const result = [];
  const queue = [root];

  while (queue.length) {
    const level = queue.length;
    let sum = 0;

    for (let i = 0; i < level; i++) {
      const node = queue.shift();
      sum += node.val;
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
```

```
    result.push(sum / level);
  }

  return result;
};
```

0638 Shopping Offers

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 638. Shopping Offers
 * https://leetcode.com/problems/shopping-offers/
 * Difficulty: Medium
 *
 * In LeetCode Store, there are n items to sell. Each item has a price. However, there are
 * some special offers, and a special offer consists of one or more different kinds of items
 * with a sale price.
 *
 * You are given an integer array price where price[i] is the price of the ith item, and an
 * integer array needs where needs[i] is the number of pieces of the ith item you want to buy.
 *
 * You are also given an array special where special[i] is of size n + 1 where special[i][j]
 * is the number of pieces of the jth item in the ith offer and special[i][n] (i.e., the last
 * integer in the array) is the price of the ith offer.
 *
 * Return the lowest price you have to pay for exactly certain items as given, where you could
 * make optimal use of the special offers. You are not allowed to buy more items than you
want,
 * even if that would lower the overall price. You could use any of the special offers as many
 * times as you want.
 */

/**
 * @param {number[]} price
 * @param {number[][]} special
 * @param {number[]} needs
 * @return {number}
 */
var shoppingOffers = function(price, special, needs) {
  const map = new Map();
  return dp(needs);

  function dp(input) {
    const key = input.join(',');
    if (map.has(key)) return map.get(key);
    let minCost = input.reduce((sum, need, i) => sum + need * price[i], 0);
    for (const offer of special) {
      const nextNeeds = [...input];
      let valid = true;
      for (let i = 0; i < price.length; i++) {
        if (nextNeeds[i] < offer[i]) {
          valid = false;
          break;
        }
        nextNeeds[i] -= offer[i];
      }
      if (valid) {
        minCost = Math.min(minCost, offer[price.length] + dp(nextNeeds));
      }
    }
    map.set(key, minCost);
    return minCost;
  }
};
```

0639 Decode Ways Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 639. Decode Ways II
 * https://leetcode.com/problems/decode-ways-ii/
 * Difficulty: Hard
 *
 * A message containing letters from A-Z can be encoded into numbers using the following
mapping:
 * - 'A' -> "1"
 * - 'B' -> "2"
 * - 'Z' -> "26"
 *
 * To decode an encoded message, all the digits must be grouped then mapped back into letters
using
 * the reverse of the mapping above (there may be multiple ways). For example, "11106" can be
mapped
 * into:
 * - "AAJF" with the grouping (1 1 10 6)
 * - "KJF" with the grouping (11 10 6)
 *
 * Note that the grouping (1 11 06) is invalid because "06" cannot be mapped into 'F' since
"6" is
 * different from "06".
 *
 * In addition to the mapping above, an encoded message may contain the '*' character, which
can
 * represent any digit from '1' to '9' ('0' is excluded). For example, the encoded message
"1*"
 * may represent any of the encoded messages "11", "12", "13", "14", "15", "16", "17", "18",
or
 * "19". Decoding "1*" is equivalent to decoding any of the encoded messages it can represent.
 *
 * Given a string s consisting of digits and '*' characters, return the number of ways to
decode it.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numDecodings = function(s) {
  const mod = 1e9 + 7;
  let [i0, i1, i2, i3] = [1, 0, 0, 0];

  for (const c of s) {
    if (c == '*') {
      i3 = 9 * i0 + 9 * i1 + 6 * i2;
      i1 = i0;
      i2 = i0;
    } else {
      i3 = (c > '0') * i0 + i1 + (c <= '6') * i2;
      i1 = (c == '1') * i0;
      i2 = (c == '2') * i0;
    }
    i0 = i3 % mod;
  }

  return i0;
};
```

0640 Solve The Equation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 640. Solve the Equation
 * https://leetcode.com/problems/solve-the-equation/
 * Difficulty: Medium
 *
 * Solve a given equation and return the value of 'x' in the form of a string "x=#value".
 * The equation contains only '+', '-' operation, the variable 'x' and its coefficient.
 * You should return "No solution" if there is no solution for the equation, or "Infinite
 * solutions" if there are infinite solutions for the equation.
 *
 * If there is exactly one solution for the equation, we ensure that the value of 'x' is
 * an integer.
 */

/**
 * @param {string} equation
 * @return {string}
 */
var solveEquation = function(equation) {
  const [left, right] = equation.split('=');
  const leftSide = parseSide(left);
  const rightSide = parseSide(right);
  const totalX = leftSide.xCount - rightSide.xCount;
  const totalNum = rightSide.numSum - leftSide.numSum;

  if (totalX === 0 && totalNum === 0) return 'Infinite solutions';
  if (totalX === 0) return 'No solution';

  return `x=${totalNum / totalX}`;

  function parseSide(side) {
    let xCount = 0;
    let numSum = 0;
    let currentNum = 0;
    let sign = 1;

    for (let i = 0; i < side.length; i++) {
      const char = side[i];
      if (char === 'x') {
        const v = i === 0 || side[i-1] === '+' || side[i-1] === '-';
        xCount += sign * (currentNum === 0 && (v) ? 1 : currentNum);
        currentNum = 0;
      } else if (char === '+' || char === '-') {
        numSum += sign * currentNum;
        sign = char === '+' ? 1 : -1;
        currentNum = 0;
      } else {
        currentNum = currentNum * 10 + parseInt(char);
      }
    }
    numSum += sign * currentNum;

    return { xCount, numSum };
  }
};
```

0641 Design Circular Deque

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
```

```
 * 641. Design Circular Deque
 * https://leetcode.com/problems/design-circular-deque/
 * Difficulty: Medium
 *
 * Design your implementation of the circular double-ended queue (deque).
 *
 * Implement the MyCircularDeque class:
 * - MyCircularDeque(int k) Initializes the deque with a maximum size of k.
 * - boolean insertFront() Adds an item at the front of Deque. Returns true if the operation
 *   is successful, or false otherwise.
 * - boolean insertLast() Adds an item at the rear of Deque. Returns true if the operation
 *   is successful, or false otherwise.
 * - boolean deleteFront() Deletes an item from the front of Deque. Returns true if the
 *   operation is successful, or false otherwise.
 * - boolean deleteLast() Deletes an item from the rear of Deque. Returns true if the
operation
 *   is successful, or false otherwise.
 * - int getFront() Returns the front item from the Deque. Returns -1 if the deque is empty.
 * - int getRear() Returns the last item from Deque. Returns -1 if the deque is empty.
 * - boolean isEmpty() Returns true if the deque is empty, or false otherwise.
 * - boolean isFull() Returns true if the deque is full, or false otherwise.
 */

/**
 * @param {number} k
 */
var MyCircularDeque = function(k) {
  this.queue = new Array(k);
  this.size = k;
  this.front = 0;
  this.rear = -1;
  this.count = 0;
};

/**
 * @param {number} value
 * @return {boolean}
 */
MyCircularDeque.prototype.insertFront = function(value) {
  if (this.isFull()) return false;
  this.front = (this.front - 1 + this.size) % this.size;
  this.queue[this.front] = value;
  this.count++;
  if (this.count === 1) this.rear = this.front;
  return true;
};

/**
 * @param {number} value
 * @return {boolean}
 */
MyCircularDeque.prototype.insertLast = function(value) {
  if (this.isFull()) return false;
  this.rear = (this.rear + 1) % this.size;
  this.queue[this.rear] = value;
  this.count++;
  return true;
};

/**
 * @return {boolean}
 */
MyCircularDeque.prototype.deleteFront = function() {
  if (this.isEmpty()) return false;
  this.front = (this.front + 1) % this.size;
  this.count--;
  return true;
};

/**
 * @return {boolean}
 */
MyCircularDeque.prototype.deleteLast = function() {
  if (this.isEmpty()) return false;
  this.rear = (this.rear - 1 + this.size) % this.size;
```

```
  this.count--;
  return true;
};

/**
 * @return {number}
 */
MyCircularDeque.prototype.getFront = function() {
  return this.isEmpty() ? -1 : this.queue[this.front];
};

/**
 * @return {number}
 */
MyCircularDeque.prototype.getRear = function() {
  return this.isEmpty() ? -1 : this.queue[this.rear];
};

/**
 * @return {boolean}
 */
MyCircularDeque.prototype.isEmpty = function() {
  return this.count === 0;
};

/**
 * @return {boolean}
 */
MyCircularDeque.prototype.isFull = function() {
  return this.count === this.size;
};
```

0642 Design Search Autocomplete System

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 642. Design Search Autocomplete System
 * https://leetcode.com/problems/design-search-autocomplete-system/
 * Difficulty: Hard
 *
 * Design a search autocomplete system for a search engine. Users may input a sentence (at
 * least one word and end with a special character '#').
 *
 * You are given a string array sentences and an integer array times both of length n where
 * sentences[i] is a previously typed sentence and times[i] is the corresponding number of
 * times the sentence was typed. For each input character except '#', return the top 3
 * historical hot sentences that have the same prefix as the part of the sentence already
typed.
 *
 * Here are the specific rules:
 * - The hot degree for a sentence is defined as the number of times a user typed the exactly
 *   same sentence before.
 * - The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest
 *   one). If several sentences have the same hot degree, use ASCII-code order (smaller one
 *   appears first).
 * - If less than 3 hot sentences exist, return as many as you can.
 * - When the input is a special character, it means the sentence ends, and in this case, you
 *   need to return an empty list.
 *
 * Implement the AutocompleteSystem class:
 * - AutocompleteSystem(String[] sentences, int[] times) Initializes the object with the
 *   sentences and times arrays.
 * - List<String> input(char c) This indicates that the user typed the character c.
 *   - Returns an empty array [] if c == '#' and stores the inputted sentence in the system.
 *   - Returns the top 3 historical hot sentences that have the same prefix as the part of the
 *     sentence already typed. If there are fewer than 3 matches, return them all.
 */

/**
```

```
 * @param {string[]} sentences
 * @param {number[]} times
 */
var AutocompleteSystem = function(sentences, times) {
  this.trie = {};
  this.current = '';
  this.root = this.trie;

  for (let i = 0; i < sentences.length; i++) {
    this.insert(sentences[i], times[i]);
  }
};

/**
 * @param {character} c
 * @return {string[]}
 */
AutocompleteSystem.prototype.input = function(c) {
  if (c === '#') {
    this.insert(this.current, 1);
    this.current = '';
    this.root = this.trie;
    return [];
  }

  this.current += c;
  if (!this.root[c]) {
    this.root[c] = {};
    this.root = this.root[c];
    return [];
  }

  this.root = this.root[c];
  const candidates = [];
  this.search(this.root, this.current, candidates);

  candidates.sort((a, b) => {
    if (a.count !== b.count) return b.count - a.count;
    return a.sentence < b.sentence ? -1 : 1;
  });

  return candidates.slice(0, 3).map(item => item.sentence);
};

AutocompleteSystem.prototype.insert = function(sentence, count) {
  let node = this.trie;
  for (const char of sentence) {
    if (!node[char]) node[char] = {};
    node = node[char];
  }
  node.isEnd = (node.isEnd || 0) + count;
};

AutocompleteSystem.prototype.search = function(node, prefix, candidates) {
  if (node.isEnd) {
    candidates.push({ sentence: prefix, count: node.isEnd });
  }

  for (const char in node) {
    if (char !== 'isEnd') {
      this.search(node[char], prefix + char, candidates);
    }
  }
};
```

0643 Maximum Average Subarray I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 643. Maximum Average Subarray I
 * https://leetcode.com/problems/maximum-average-subarray-i/
 * Difficulty: Easy
 *
 * You are given an integer array nums consisting of n elements, and an integer k.
 *
 * Find a contiguous subarray whose length is equal to k that has the maximum average
 * value and return this value. Any answer with a calculation error less than 10-5
 * will be accepted.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var findMaxAverage = function(nums, k) {
  let sum = 0;
  for (let i = 0; i < k; i++) {
    sum += nums[i];
  }

  let max = sum;
  for (let i = k; i < nums.length; i++) {
    sum = sum - nums[i - k] + nums[i];
    max = Math.max(max, sum);
  }

  return max / k;
};
```

0644 Maximum Average Subarray Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 644. Maximum Average Subarray II
 * https://leetcode.com/problems/maximum-average-subarray-ii/
 * Difficulty: Hard
 *
 * You are given an integer array nums consisting of n elements, and an integer k.
 *
 * Find a contiguous subarray whose length is greater than or equal to k that has the maximum
 * average value and return this value. Any answer with a calculation error less than 10-5
will
 * be accepted.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var findMaxAverage = function(nums, k) {
  const n = nums.length;
  let minVal = Math.min(...nums);
  let maxVal = Math.max(...nums);

  while (maxVal - minVal > 1e-5) {
    const mid = (minVal + maxVal) / 2;
    if (canFindLargerAverage(nums, k, mid)) {
      minVal = mid;
    } else {
      maxVal = mid;
    }
  }

  return minVal;
};
```

```
function canFindLargerAverage(nums, k, target) {
  let sum = 0;
  let prevSum = 0;
  let minPrevSum = 0;

  for (let i = 0; i < k; i++) {
    sum += nums[i] - target;
  }

  if (sum >= 0) return true;

  for (let i = k; i < nums.length; i++) {
    sum += nums[i] - target;
    prevSum += nums[i - k] - target;
    minPrevSum = Math.min(minPrevSum, prevSum);
    if (sum - minPrevSum >= 0) return true;
  }

  return false;
}
```

0645 Set Mismatch

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 645. Set Mismatch
 * https://leetcode.com/problems/set-mismatch/
 * Difficulty: Medium
 *
 * You have a set of integers s, which originally contains all the numbers from 1 to n.
 * Unfortunately, due to some error, one of the numbers in s got duplicated to another
 * number in the set, which results in repetition of one number and loss of another number.
 *
 * You are given an integer array nums representing the data status of this set after
 * the error.
 *
 * Find the number that occurs twice and the number that is missing and return them in
 * the form of an array.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var findErrorNums = function(nums) {
  const set = new Set();
  const result = [];

  for (const n of nums) {
    if (set.has(n)) {
      result.push(n);
    }
    set.add(n);
  }

  for (let i = nums.length; i > 0; i--) {
    if (!set.has(i)) {
      result.push(i);
      break;
    }
  }

  return result;
};
```

0646 Maximum Length Of Pair Chain

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 646. Maximum Length of Pair Chain
 * https://leetcode.com/problems/maximum-length-of-pair-chain/
 * Difficulty: Medium
 *
 * You are given an array of n pairs pairs where pairs[i] = [lefti, righti] and lefti <
righti.
 *
 * A pair p2 = [c, d] follows a pair p1 = [a, b] if b < c. A chain of pairs can be formed in
 * this fashion.
 *
 * Return the length longest chain which can be formed.
 *
 * You do not need to use up all the given intervals. You can select pairs in any order.
 */

/**
 * @param {number[][]} pairs
 * @return {number}
 */
var findLongestChain = function(pairs) {
  pairs.sort((a, b) => a[1] - b[1]);
  let pointer = -Infinity;
  let result = 0;

  for (const [start, end] of pairs) {
    if (start > pointer) {
      pointer = end;
      result++;
    }
  }

  return result;
};
```

0647 Palindromic Substrings

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 647. Palindromic Substrings
 * https://leetcode.com/problems/palindromic-substrings/
 * Difficulty: Medium
 *
 * Given a string s, return the number of palindromic substrings in it.
 *
 * A string is a palindrome when it reads the same backward as forward.
 *
 * A substring is a contiguous sequence of characters within the string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countSubstrings = function(s) {
  let result = 0;

  for (let i = 0; i < s.length; i++) {
    result += expand(i, i);
    result += expand(i, i + 1);
```

```
  }

  return result;

  function expand(left, right) {
    let count = 0;
    while (left >= 0 && right < s.length && s[left] === s[right]) {
      count++;
      left--;
      right++;
    }
    return count;
  }
};
```

0648 Replace Words

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 648. Replace Words
 * https://leetcode.com/problems/replace-words/
 * Difficulty: Medium
 *
 * In English, we have a concept called root, which can be followed by some other words
 * to form another longer word - let's call this word successor. For example, the root
 * an, followed by other, which can form another word another.
 *
 * Now, given a dictionary consisting of many roots and a sentence. You need to replace
 * all the successor in the sentence with the root forming it. If a successor has many
 * roots can form it, replace it with the root with the shortest length.
 *
 * You need to output the sentence after the replacement.
 */

/**
 * @param {string[]} dict
 * @param {string} sentence
 * @return {string}
 */
var replaceWords = function(dict, sentence) {
  const set = new Set(dict);
  return sentence.split(/\s+/).map(word => {
    for (let i = 1; i <= word.length; i++) {
      if (set.has(word.slice(0, i))) return word.slice(0, i);
    }
    return word;
  }).join(' ');
};
```

0649 Dota2 Senate

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 649. Dota2 Senate
 * https://leetcode.com/problems/dota2-senate/
 * Difficulty: Medium
 *
 * In the world of Dota2, there are two parties: the Radiant and the Dire.
 *
 * The Dota2 senate consists of senators coming from two parties. Now the Senate wants
 * to decide on a change in the Dota2 game. The voting for this change is a round-based
 * procedure. In each round, each senator can exercise one of the two rights:
 * - Ban one senator's right: A senator can make another senator lose all his rights in
```

```
 *   this and all the following rounds.
 * - Announce the victory: If this senator found the senators who still have rights to
 *   vote are all from the same party, he can announce the victory and decide on the
 *   change in the game.
 *
 * Given a string senate representing each senator's party belonging. The character 'R'
 * and 'D' represent the Radiant party and the Dire party. Then if there are n senators,
 * the size of the given string will be n.
 *
 * The round-based procedure starts from the first senator to the last senator in the
 * given order. This procedure will last until the end of voting. All the senators who
 * have lost their rights will be skipped during the procedure.
 *
 * Suppose every senator is smart enough and will play the best strategy for his own
 * party. Predict which party will finally announce the victory and change the Dota2
 * game. The output should be "Radiant" or "Dire".
*/

/**
 * @param {string} senate
 * @return {string}
 */
var predictPartyVictory = function(senate) {
  const rQueue = [];
  const dQueue = [];

  for (let i = 0; i < senate.length; i++) {
    if (senate[i] === 'R') rQueue.push(i);
    else dQueue.push(i);
  }

  while (rQueue.length && dQueue.length) {
    const [rIndex, dIndex] = [rQueue.shift(), dQueue.shift()];
    if (rIndex < dIndex) {
      rQueue.push(rIndex + senate.length);
    } else {
      dQueue.push(dIndex + senate.length);
    }
  }

  return rQueue.length > 0 ? 'Radiant' : 'Dire';
};
```

0650 2 Keys Keyboard

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 650. 2 Keys Keyboard
 * https://leetcode.com/problems/2-keys-keyboard/
 * Difficulty: Medium
 *
 * There is only one character 'A' on the screen of a notepad. You can perform one of two
 * operations on this notepad for each step:
 * - Copy All: You can copy all the characters present on the screen (a partial copy is
 *   not allowed).
 * - Paste: You can paste the characters which are copied last time.
 *
 * Given an integer n, return the minimum number of operations to get the character 'A'
 * exactly n times on the screen.
 */

/**
 * @param {number} n
 * @return {number}
 */
var minSteps = function(n) {
  let result = 0;

  for (let factor = 2; n > 1;) {
```

```
    while (n % factor === 0) {
      result += factor;
      n /= factor;
    }
    factor++;
  }

  return result;
};
```

0651 4 Keys Keyboard

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 651. 4 Keys Keyboard
 * https://leetcode.com/problems/4-keys-keyboard/
 * Difficulty: Medium
 *
 * Imagine you have a special keyboard with the following keys:
 * - A: Print one 'A' on the screen.
 * - Ctrl-A: Select the whole screen.
 * - Ctrl-C: Copy selection to buffer.
 * - Ctrl-V: Print buffer on screen appending it after what has already been printed.
 *
 * Given an integer n, return the maximum number of 'A' you can print on the screen with at
 * most n presses on the keys.
 */

/**
 * @param {number} n
 * @return {number}
 */
var maxA = function(n) {
  const dp = new Array(n + 1).fill(0);

  for (let i = 1; i <= n; i++) {
    dp[i] = dp[i - 1] + 1;
    for (let j = 2; j < i; j++) {
      dp[i] = Math.max(dp[i], dp[j - 2] * (i - j + 1));
    }
  }

  return dp[n];
};
```

0652 Find Duplicate Subtrees

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 652. Find Duplicate Subtrees
 * https://leetcode.com/problems/find-duplicate-subtrees/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return all duplicate subtrees.
 *
 * For each kind of duplicate subtrees, you only need to return the root node of any one of
them.
 *
 * Two trees are duplicate if they have the same structure with the same node values.
 */

/**
 * Definition for a binary tree node.
```

```
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode[]}
 */
var findDuplicateSubtrees = function(root) {
  const map = new Map();
  const result = [];

  serialize(root);

  return result;

  function serialize(node) {
    if (!node) return '#';
    const key = `${node.val},${serialize(node.left)},${serialize(node.right)}`;
    map.set(key, (map.get(key) || 0) + 1);
    if (map.get(key) === 2) {
      result.push(node);
    }
    return key;
  }
};
```

0653 Two Sum Iv Input Is A Bst

EASY

APPROACH NOTES

Uses a Set for unique membership. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 653. Two Sum IV - Input is a BST
 * https://leetcode.com/problems/two-sum-iv-input-is-a-bst/
 * Difficulty: Easy
 *
 * Given the root of a Binary Search Tree and a target number k, return true
 * if there exist two elements in the BST such that their sum is equal to the
 * given target.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} k
 * @return {boolean}
 */
var findTarget = function(root, k) {
  const set = new Set();
  return dfs(set, root, k);
};

function dfs(set, node, k) {
  if (!node) return false;
  if (set.has(k - node.val)) return true;

  set.add(node.val);

  return dfs(set, node.left, k) || dfs(set, node.right, k);
}
```

0654 Maximum Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 654. Maximum Binary Tree
 * https://leetcode.com/problems/maximum-binary-tree/
 * Difficulty: Medium
 *
 * You are given an integer array nums with no duplicates. A maximum binary tree can be built
 * recursively from nums using the following algorithm:
 * 1. Create a root node whose value is the maximum value in nums.
 * 2. Recursively build the left subtree on the subarray prefix to the left of the maximum
value.
 * 3. Recursively build the right subtree on the subarray suffix to the right of the maximum
value.
 *
 * Return the maximum binary tree built from nums.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var constructMaximumBinaryTree = function(nums) {
  if (!nums.length) return null;

  const max = Math.max(...nums);
  const index = nums.indexOf(max);
  const head = new TreeNode(max);

  head.left = constructMaximumBinaryTree(nums.slice(0, index));
  head.right = constructMaximumBinaryTree(nums.slice(index + 1));

  return head;
};
```

0655 Print Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 655. Print Binary Tree
 * https://leetcode.com/problems/print-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, construct a 0-indexed m x n string matrix res that
represents
 * a formatted layout of the tree. The formatted layout matrix should be constructed using the
 * following rules:
 * - The height of the tree is height and the number of rows m should be equal to height + 1.
 * - The number of columns n should be equal to 2height+1 - 1.
 * - Place the root node in the middle of the top row (more formally, at location
res[0][(n-1)/2]).
 * - For each node that has been placed in the matrix at position res[r][c], place its left
child at
 *   res[r+1][c-2height-r-1] and its right child at res[r+1][c+2height-r-1].
```

```
 * - Continue this process until all the nodes in the tree have been placed.
 * - Any empty cells should contain the empty string "".
 *
 * Return the constructed matrix res.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {string[][]}
 */
var printTree = function(root) {
  const height = getHeight(root);
  const columns = Math.pow(2, height + 1) - 1;
  const result = new Array(height + 1).fill().map(() => {
    return new Array(columns).fill('');
  });

  fill(root, 0, Math.floor((columns - 1) / 2), height);

  return result;

  function fill(node, r, c, h) {
    if (!node) return;
    result[r][c] = node.val.toString();
    fill(node.left, r + 1, c - Math.pow(2, h - r - 1), h);
    fill(node.right, r + 1, c + Math.pow(2, h - r - 1), h);
  }

  function getHeight(node) {
    if (!node) return -1;
    return 1 + Math.max(getHeight(node.left), getHeight(node.right));
  }
};
```

0656 Coin Path

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 656. Coin Path
 * https://leetcode.com/problems/coin-path/
 * Difficulty: Hard
 *
 * You are given an integer array coins (1-indexed) of length n and an integer maxJump. You
can
 * jump to any index i of the array coins if coins[i] != -1 and you have to pay coins[i] when
you
 * visit index i. In addition to that, if you are currently at index i, you can only jump to
any
 * index i + k where i + k <= n and k is a value in the range [1, maxJump].
 *
 * You are initially positioned at index 1 (coins[1] is not -1). You want to find the path
that
 * reaches index n with the minimum cost.
 *
 * Return an integer array of the indices that you will visit in order so that you can reach
 * index n with the minimum cost. If there are multiple paths with the same cost, return the
 * lexicographically smallest such path. If it is not possible to reach index n, return an
 * empty array.
 *
 * A path p1 = [Pa1, Pa2, ..., Pax] of length x is lexicographically smaller than
 * p2 = [Pb1, Pb2, ..., Pbx] of length y, if and only if at the first j where Paj and Pbj
```

```
 * differ, Paj < Pbj; when no such j exists, then x < y.
 */

/**
 * @param {number[]} coins
 * @param {number} maxJump
 * @return {number[]}
 */
var cheapestJump = function(coins, maxJump) {
  const n = coins.length;
  const minCost = new Array(n + 1).fill(Infinity);
  const parent = new Array(n + 1).fill(-1);

  minCost[1] = coins[0];

  for (let i = 1; i <= n; i++) {
    if (coins[i - 1] === -1 || minCost[i] === Infinity) continue;

    for (let j = i + 1; j <= Math.min(i + maxJump, n); j++) {
      if (coins[j - 1] === -1) continue;

      const newCost = minCost[i] + coins[j - 1];
      if (newCost < minCost[j]) {
        minCost[j] = newCost;
        parent[j] = i;
      } else if (newCost === minCost[j] && parent[j] !== -1) {
        const currentPath = buildPath(parent, j);
        const newPath = buildPath(parent, j, i);
        if (compare(newPath, currentPath)) {
          parent[j] = i;
        }
      }
    }
  }

  if (minCost[n] === Infinity) return [];

  const result = [];
  let current = n;
  while (current !== -1) {
    result.unshift(current);
    current = parent[current];
  }

  return result;
};

function buildPath(parent, end, newParent = null) {
  const path = [];
  let current = end;
  if (newParent !== null) {
    path.unshift(current);
    current = newParent;
  }
  while (current !== -1) {
    path.unshift(current);
    current = parent[current];
  }
  return path;
}

function compare(path1, path2) {
  for (let i = 0; i < Math.min(path1.length, path2.length); i++) {
    if (path1[i] < path2[i]) return true;
    if (path1[i] > path2[i]) return false;
  }
  return path1.length < path2.length;
}
```

0657 Robot Return To Origin

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 657. Robot Return to Origin
 * https://leetcode.com/problems/robot-return-to-origin/
 * Difficulty: Easy
 *
 * There is a robot starting at the position (0, 0), the origin, on a 2D plane. Given a
sequence
 * of its moves, judge if this robot ends up at (0, 0) after it completes its moves.
 *
 * You are given a string moves that represents the move sequence of the robot where moves[i]
 * represents its ith move. Valid moves are 'R' (right), 'L' (left), 'U' (up), and 'D' (down).
 *
 * Return true if the robot returns to the origin after it finishes all of its moves, or false
 * otherwise.
 *
 * Note: The way that the robot is "facing" is irrelevant. 'R' will always make the robot move
 * to the right once, 'L' will always make it move left, etc. Also, assume that the magnitude
 * of the robot's movement is the same for each move.
 */

/**
 * @param {string} moves
 * @return {boolean}
 */
/**
 * @param {string} moves
 * @return {boolean}
 */
var judgeCircle = function(moves) {
  let x = 0;
  let y = 0;

  for (const move of moves) {
    x += move === 'R' ? 1 : move === 'L' ? -1 : 0;
    y += move === 'U' ? 1 : move === 'D' ? -1 : 0;
  }

  return x === 0 && y === 0;
};
```

0658 Find K Closest Elements

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 658. Find K Closest Elements
 * https://leetcode.com/problems/find-k-closest-elements/
 * Difficulty: Medium
 *
 * Given a sorted integer array arr, two integers k and x, return the k closest integers to x
 * in the array. The result should also be sorted in ascending order.
 *
 * An integer a is closer to x than an integer b if:
 * - |a - x| < |b - x|, or
 * - |a - x| == |b - x| and a < b
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @param {number} x
```

```
 * @return {number[]}
 */
var findClosestElements = function(arr, k, x) {
  let left = 0;
  let right = arr.length - k;

  while (left < right) {
    const middle = Math.floor((left + right) / 2);
    const x1 = arr[middle];
    const x2 = arr[middle + k];

    if (x - x1 > x2 - x) {
      left = middle + 1;
    } else {
      right = middle;
    }
  }

  return arr.slice(left, left + k);
};
```

0659 Split Array Into Consecutive Subsequences

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 659. Split Array into Consecutive Subsequences
 * https://leetcode.com/problems/split-array-into-consecutive-subsequences/
 * Difficulty: Medium
 *
 * You are given an integer array nums that is sorted in non-decreasing order.
 *
 * Determine if it is possible to split nums into one or more subsequences such that both of
the
 * following conditions are true:
 * - Each subsequence is a consecutive increasing sequence (i.e. each integer is exactly one
more
 *   than the previous integer).
 * - All subsequences have a length of 3 or more.
 *
 * Return true if you can split nums according to the above conditions, or false otherwise.
 *
 * A subsequence of an array is a new array that is formed from the original array by deleting
 some
 * (can be none) of the elements without disturbing the relative positions of the remaining
 * elements. (i.e., [1,3,5] is a subsequence of [1,2,3,4,5] while [1,3,2] is not).
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isPossible = function(nums) {
  const map = new Map();
  const map2 = new Map();

  for (const n of nums) {
    map.set(n, (map.get(n) || 0) + 1);
  }

  for (const n of nums) {
    if (map.get(n) === 0) continue;

    if ((map2.get(n) || 0) > 0) {
      map2.set(n, map2.get(n) - 1);
      map.set(n, map.get(n) - 1);
      map2.set(n + 1, (map2.get(n + 1) || 0) + 1);
    } else if ((map.get(n) || 0) > 0 && (map.get(n + 1) || 0) > 0 && (map.get(n + 2) || 0) >
0) {
      map.set(n, map.get(n) - 1);
```

```
      map.set(n + 1, map.get(n + 1) - 1);
      map.set(n + 2, map.get(n + 2) - 1);
      map2.set(n + 3, (map2.get(n + 3) || 0) + 1);
    } else {
      return false;
    }
  }

  return true;
};
```

0660 Remove 9

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 660. Remove 9
 * https://leetcode.com/problems/remove-9/
 * Difficulty: Hard
 *
 * Start from integer 1, remove any integer that contains 9 such as 9, 19, 29...
 *
 * Now, you will have a new integer sequence [1, 2, 3, 4, 5, 6, 7, 8, 10, 11, ...].
 *
 * Given an integer n, return the nth (1-indexed) integer in the new sequence.
 */

/**
 * @param {number} n
 * @return {number}
 */
var newInteger = function(n) {
  let result = 0;
  let base = 1;

  while (n > 0) {
    result += (n % 9) * base;
    n = Math.floor(n / 9);
    base *= 10;
  }

  return result;
};
```

0661 Image Smoother

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 661. Image Smoother
 * https://leetcode.com/problems/image-smoother/
 * Difficulty: Easy
 *
 * An image smoother is a filter of the size 3 x 3 that can be applied to each cell of an
image by
 * rounding down the average of the cell and the eight surrounding cells (i.e., the average of
 the
 * nine cells in the blue smoother). If one or more of the surrounding cells of a cell is not
 * present, we do not consider it in the average (i.e., the average of the four cells in the
red
 * smoother).
 *
 * Given an m x n integer matrix img representing the grayscale of an image, return the image
after
 * applying the smoother on each cell of it.
```

```
 */

/**
 * @param {number[][]} img
 * @return {number[][]}
 */
var imageSmoother = function(img) {
  const result = new Array(img.length).fill().map(() => {
    return new Array(img[0].length).fill(0);
  });

  for (let i = 0; i < img.length; i++) {
    for (let j = 0; j < img[0].length; j++) {
      let sum = 0;
      let count = 0;
      for (let di = -1; di <= 1; di++) {
        for (let dj = -1; dj <= 1; dj++) {
          const ni = i + di;
          const nj = j + dj;
          if (ni >= 0 && ni < img.length && nj >= 0 && nj < img[0].length) {
            sum += img[ni][nj];
            count++;
          }
        }
      }
      result[i][j] = Math.floor(sum / count);
    }
  }

  return result;
};
```

0662 Maximum Width Of Binary Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 662. Maximum Width of Binary Tree
 * https://leetcode.com/problems/maximum-width-of-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the maximum width of the given tree.
 *
 * The maximum width of a tree is the maximum width among all levels.
 *
 * The width of one level is defined as the length between the end-nodes (the leftmost and
 * rightmost non-null nodes), where the null nodes between the end-nodes that would be present
 * in a complete binary tree extending down to that level are also counted into the length
 * calculation.
 *
 * It is guaranteed that the answer will in the range of a 32-bit signed integer.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var widthOfBinaryTree = function(root) {
  const queue = [[root, 0n]];
  let result = 0n;

  while (queue.length) {
```

```
    const total = queue.length;
    const levelStart = queue[0][1];
    let levelEnd;

    for (let i = 0; i < total; i++) {
      const [node, index] = queue.shift();
      levelEnd = index;
      if (node.left) queue.push([node.left, index * 2n]);
      if (node.right) queue.push([node.right, index * 2n + 1n]);
    }

    result = result > (levelEnd - levelStart + 1n)
      ? result
      : (levelEnd - levelStart + 1n);
  }

  return Number(result);
};
```

0663 Equal Tree Partition

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 663. Equal Tree Partition
 * https://leetcode.com/problems/equal-tree-partition/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return true if you can partition the tree into two trees
 * with equal sums of values after removing exactly one edge on the original tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var checkEqualTree = function(root) {
  const subtreeSums = new Set();
  const totalSum = calculateSum(root);

  return totalSum % 2 === 0 && subtreeSums.has(totalSum / 2);

  function calculateSum(node) {
    if (!node) return 0;

    const leftSum = calculateSum(node.left);
    const rightSum = calculateSum(node.right);
    const currentSum = node.val + leftSum + rightSum;

    if (node !== root) {
      subtreeSums.add(currentSum);
    }

    return currentSum;
  }
};
```

0664 Strange Printer

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 664. Strange Printer
 * https://leetcode.com/problems/strange-printer/
 * Difficulty: Hard
 *
 * There is a strange printer with the following two special properties:
 * - The printer can only print a sequence of the same character each time.
 * - At each turn, the printer can print new characters starting from and ending at any place
and
 *   will cover the original existing characters.
 *
 * Given a string s, return the minimum number of turns the printer needed to print it.
 */

/**
 * @param {string} s
 * @return {number}
 */
var strangePrinter = function(s) {
  const n = s.length;
  const dp = new Array(n).fill().map(() => new Array(n).fill(0));

  for (let i = n - 1; i >= 0; i--) {
    dp[i][i] = 1;
    for (let j = i + 1; j < n; j++) {
      dp[i][j] = dp[i][j - 1] + 1;
      for (let k = i; k < j; k++) {
        if (s[k] === s[j]) {
          dp[i][j] = Math.min(dp[i][j], dp[i][k] + (k + 1 <= j - 1 ? dp[k + 1][j - 1] : 0));
        }
      }
    }
  }

  return dp[0][n - 1];
};
```

0665 Non Decreasing Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 665. Non-decreasing Array
 * https://leetcode.com/problems/non-decreasing-array/
 * Difficulty: Medium
 *
 * Given an array nums with n integers, your task is to check if it could become
non-decreasing
 * by modifying at most one element.
 *
 * We define an array is non-decreasing if nums[i] <= nums[i + 1] holds for every i (0-based)
 * such that (0 <= i <= n - 2).
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var checkPossibility = function(nums) {
  let count = 0;
```

```
  for (let i = 0; i < nums.length - 1; i++) {
    if (nums[i] > nums[i + 1]) {
      count++;
      if (count > 1) return false;
      if (i > 0 && nums[i - 1] > nums[i + 1]) {
        nums[i + 1] = nums[i];
      } else {
        nums[i] = nums[i + 1];
      }
    }
  }

  return true;
};
```

0666 Path Sum Iv

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 666. Path Sum IV
 * https://leetcode.com/problems/path-sum-iv/
 * Difficulty: Medium
 *
 * If the depth of a tree is smaller than 5, then this tree can be represented by an array
 * of three-digit integers. You are given an ascending array nums consisting of three-digit
 * integers representing a binary tree with a depth smaller than 5, where for each integer:
 * - The hundreds digit represents the depth d of this node, where 1 <= d <= 4.
 * - The tens digit represents the position p of this node within its level, where 1 <= p <=
8,
 *   corresponding to its position in a full binary tree.
 * - The units digit represents the value v of this node, where 0 <= v <= 9.
 *
 * Return the sum of all paths from the root towards the leaves.
 *
 * It is guaranteed that the given array represents a valid connected binary tree.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var pathSum = function(nums) {
  const nodeMap = new Map();

  for (const num of nums) {
    const depth = Math.floor(num / 100);
    const position = Math.floor((num % 100) / 10);
    const value = num % 10;
    nodeMap.set(`${depth}-${position}`, value);
  }

  return dfs(1, 1, 0);

  function dfs(depth, position, currentSum) {
    const key = `${depth}-${position}`;
    if (!nodeMap.has(key)) return 0;

    const nodeValue = nodeMap.get(key);
    const pathSum = currentSum + nodeValue;

    const leftChild = `${depth + 1}-${position * 2 - 1}`;
    const rightChild = `${depth + 1}-${position * 2}`;

    if (!nodeMap.has(leftChild) && !nodeMap.has(rightChild)) {
      return pathSum;
    }

    return dfs(depth + 1, position * 2 - 1, pathSum)
      + dfs(depth + 1, position * 2, pathSum);
```

```
  }
};
```

0667 Beautiful Arrangement Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 667. Beautiful Arrangement II
 * https://leetcode.com/problems/beautiful-arrangement-ii/
 * Difficulty: Medium
 *
 * Given two integers n and k, construct a list answer that contains n different positive
integers
 * ranging from 1 to n and obeys the following requirement:
 * - Suppose this list is answer = [a1, a2, a3, ... , an], then the list [|a1 - a2|, |a2 -
a3|,
 *   |a3 - a4|, ... , |an-1 - an|] has exactly k distinct integers.
 *
 * Return the list answer. If there multiple valid answers, return any of them.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number[]}
 */
var constructArray = function(n, k) {
  const result = [];

  for (let i = 0, left = 1, right = n; i < n; i++) {
    if (k > 1) {
      result.push(k % 2 === 0 ? right-- : left++);
      k--;
    } else {
      result.push(left++);
    }
  }

  return result;
};
```

0668 Kth Smallest Number In Multiplication Table

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 668. Kth Smallest Number in Multiplication Table
 * https://leetcode.com/problems/kth-smallest-number-in-multiplication-table/
 * Difficulty: Hard
 *
 * Nearly everyone has used the Multiplication Table. The multiplication table of size m x n
is
 * an integer matrix mat where mat[i][j] == i * j (1-indexed).
 *
 * Given three integers m, n, and k, return the kth smallest element in the m x n
multiplication
 * table.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number} k
 * @return {number}
```

```
 */
var findKthNumber = function(m, n, k) {
  let low = 1;
  let high = m * n;

  while (low < high) {
    const middle = Math.floor((low + high) / 2);
    if (helper(middle) < k) {
      low = middle + 1;
    } else {
      high = middle;
    }
  }

  return low;

  function helper(x) {
    let count = 0;
    for (let i = 1; i <= m; i++) {
      count += Math.min(Math.floor(x / i), n);
    }
    return count;
  }
};
```

0669 Trim A Binary Search Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 669. Trim a Binary Search Tree
 * https://leetcode.com/problems/trim-a-binary-search-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary search tree and the lowest and highest boundaries as low and
high,
 * trim the tree so that all its elements lies in [low, high]. Trimming the tree should not
change
 * the relative structure of the elements that will remain in the tree (i.e., any node's
descendant
 * should remain a descendant). It can be proven that there is a unique answer.
 *
 * Return the root of the trimmed binary search tree. Note that the root may change depending
on
 * the given bounds.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} low
 * @param {number} high
 * @return {TreeNode}
 */
var trimBST = function(root, low, high) {
  if (!root) return null;

  if (root.val < low) return trimBST(root.right, low, high);
  if (root.val > high) return trimBST(root.left, low, high);
  root.left = trimBST(root.left, low, high);
  root.right = trimBST(root.right, low, high);

  return root;
```

```
};
```

0670 Maximum Swap

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 670. Maximum Swap
 * https://leetcode.com/problems/maximum-swap/
 * Difficulty: Medium
 *
 * You are given an integer num. You can swap two digits at most once to get the maximum
 * valued number.
 *
 * Return the maximum valued number you can get.
 */

/**
 * @param {number} num
 * @return {number}
 */
var maximumSwap = function(num) {
  const digits = [...(num).toString()];
  const last = new Array(10).fill(-1);
  digits.forEach((d, i) => last[d] = i);

  for (let i = 0; i < digits.length; i++) {
    for (let d = 9; d > digits[i]; d--) {
      if (last[d] > i) {
        [digits[i], digits[last[d]]] = [digits[last[d]], digits[i]];
        return +digits.join('');
      }
    }
  }

  return num;
};
```

0671 Second Minimum Node In A Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 671. Second Minimum Node In a Binary Tree
 * https://leetcode.com/problems/second-minimum-node-in-a-binary-tree/
 * Difficulty: Easy
 *
 * Given a non-empty special binary tree consisting of nodes with the non-negative value,
 * where each node in this tree has exactly two or zero sub-node. If the node has two
 * sub-nodes, then this node's value is the smaller value among its two sub-nodes. More
 * formally, the property root.val = min(root.left.val, root.right.val) always holds.
 *
 * Given such a binary tree, you need to output the second minimum value in the set made
 * of all the nodes' value in the whole tree.
 *
 * If no such second minimum value exists, output -1 instead.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
```

```
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var findSecondMinimumValue = function(root) {
  return root ? (n => n === Infinity ? -1 : n)(traverse(root, root.val)) : -1;
  function traverse(node, min) {
    if (!node) return Infinity;
    if (node.val > min) return node.val;
    return Math.min(traverse(node.left, min), traverse(node.right, min));
  }
};
```

0672 Bulb Switcher Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 672. Bulb Switcher II
 * https://leetcode.com/problems/bulb-switcher-ii/
 * Difficulty: Medium
 *
 * There is a room with n bulbs labeled from 1 to n that all are turned on initially, and four
 * buttons on the wall. Each of the four buttons has a different functionality where:
 * - Button 1: Flips the status of all the bulbs.
 * - Button 2: Flips the status of all the bulbs with even labels (i.e., 2, 4, ...).
 * - Button 3: Flips the status of all the bulbs with odd labels (i.e., 1, 3, ...).
 * - Button 4: Flips the status of all the bulbs with a label j = 3k + 1 where k = 0, 1, 2,
...
 *   (i.e., 1, 4, 7, 10, ...).
 * - You must make exactly presses button presses in total. For each press, you may pick any
 *   of the four buttons to press.
 *
 * Given the two integers n and presses, return the number of different possible statuses
after
 * performing all presses button presses.
 */

/**
 * @param {number} n
 * @param {number} presses
 * @return {number}
 */
var flipLights = function(n, presses) {
  if (presses === 0) return 1;
  if (n === 1) return presses >= 1 ? 2 : 1;
  if (n === 2) return presses === 1 ? 3 : 4;
  if (presses === 1) return 4;
  if (presses === 2) return 7;
  return 8;
};
```

0673 Number Of Longest Increasing Subsequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 673. Number of Longest Increasing Subsequence
 * https://leetcode.com/problems/number-of-longest-increasing-subsequence/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the number of longest increasing subsequences.
 *
 * Notice that the sequence has to be strictly increasing.
```

```
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findNumberOfLIS = function(nums) {
  const lengths = new Array(nums.length).fill(1);
  const counts = new Array(nums.length).fill(1);

  for (let i = 1; i < nums.length; i++) {
    for (let j = 0; j < i; j++) {
      if (nums[i] > nums[j]) {
        if (lengths[j] + 1 > lengths[i]) {
          lengths[i] = lengths[j] + 1;
          counts[i] = counts[j];
        } else if (lengths[j] + 1 === lengths[i]) {
          counts[i] += counts[j];
        }
      }
    }
  }

  const max = Math.max(...lengths);
  return lengths.reduce((sum, l, i) => {
    return sum + (l === max ? counts[i] : 0);
  }, 0);
};
```

0674 Longest Continuous Increasing Subsequence

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 674. Longest Continuous Increasing Subsequence
 * https://leetcode.com/problems/longest-continuous-increasing-subsequence/
 * Difficulty: Easy
 *
 * Given an unsorted array of integers nums, return the length of the longest continuous
increasing
 * subsequence (i.e. subarray). The subsequence must be strictly increasing.
 *
 * A continuous increasing subsequence is defined by two indices l and r (l < r) such that it
is
 * [nums[l], nums[l + 1], ..., nums[r - 1], nums[r]] and for each l <= i < r, nums[i] < nums[i
 + 1].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findLengthOfLCIS = function(nums) {
  let result = 1;

  for (let i = 1, v = 1; i < nums.length; i++) {
    v = nums[i] > nums[i-1] ? v + 1 : 1;
    result = Math.max(result, v);
  }

  return result;
};
```

0675 Cut Off Trees For Golf Event

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 675. Cut Off Trees for Golf Event
 * https://leetcode.com/problems/cut-off-trees-for-golf-event/
 * Difficulty: Hard
 *
 * You are asked to cut off all the trees in a forest for a golf event. The forest is
represented
 * as an m x n matrix. In this matrix:
 * - 0 means the cell cannot be walked through.
 * - 1 represents an empty cell that can be walked through.
 * - A number greater than 1 represents a tree in a cell that can be walked through, and this
number
 *   is the tree's height.
 *
 * In one step, you can walk in any of the four directions: north, east, south, and west. If
you are
 * standing in a cell with a tree, you can choose whether to cut it off.
 *
 * You must cut off the trees in order from shortest to tallest. When you cut off a tree, the
value
 * at its cell becomes 1 (an empty cell).
 *
 * Starting from the point (0, 0), return the minimum steps you need to walk to cut off all
the
 * trees. If you cannot cut off all the trees, return -1.
 *
 * Note: The input is generated such that no two trees have the same height, and there is at
least
 * one tree needs to be cut off.
 */

/**
 * @param {number[][]} forest
 * @return {number}
 */
const cutOffTree = (forest) => {
  const treeHeights = forest.flat().filter((height) => height > 1).sort((a, b) => a - b);
  let currentPosition = [0, 0];
  let totalDistance = 0;

  while (treeHeights.length) {
    const gridCopy = forest.map((row) => [...row]);
    const result = findDistance(currentPosition, treeHeights.shift(), gridCopy);
    if (result === null) return -1;
    const [nextPosition, distance] = result;
    currentPosition = nextPosition;
    totalDistance += distance;
  }
  return totalDistance;

  function findDistance(startPosition, targetHeight, grid) {
    const directions = [[1, 0], [-1, 0], [0, 1], [0, -1]];
    let queue = [startPosition];
    let distance = 0;

    while (queue.length) {
      const nextLevel = [];

      for (const [row, col] of queue) {
        if (grid[row][col] === targetHeight) return [[row, col], distance];
        if (!grid[row][col]) continue;

        for (const [deltaRow, deltaCol] of directions) {
          const newRow = row + deltaRow;
          const newCol = col + deltaCol;
          if (
            newRow >= 0 && newRow < grid.length && newCol >= 0
            && newCol < grid[0].length && grid[newRow][newCol]
          ) {
            nextLevel.push([newRow, newCol]);
          }
```

```
        }
        grid[row][col] = 0;
      }
      distance += 1;
      queue = nextLevel;
    }
    return null;
  }
};
```

0676 Implement Magic Dictionary

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 676. Implement Magic Dictionary
 * https://leetcode.com/problems/implement-magic-dictionary/
 * Difficulty: Medium
 *
 * Design a data structure that is initialized with a list of different words. Provided a
string,
 * you should determine if you can change exactly one character in this string to match any
word
 * in the data structure.
 *
 * Implement the MagicDictionary class:
 * - MagicDictionary() Initializes the object.
 * - void buildDict(String[] dictionary) Sets the data structure with an array of distinct
 *   strings dictionary.
 * - bool search(String searchWord) Returns true if you can change exactly one character in
 *   searchWord to match any string in the data structure, otherwise returns false.
 */

var MagicDictionary = function() {
  this.words = new Set();
};

/**
 * @param {string[]} dictionary
 * @return {void}
 */
MagicDictionary.prototype.buildDict = function(dictionary) {
  this.words = new Set(dictionary);
};

/**
 * @param {string} searchWord
 * @return {boolean}
 */
MagicDictionary.prototype.search = function(searchWord) {
  const words = searchWord.split('');
  return Array.from(this.words).some(word => {
    if (word.length !== searchWord.length) return false;
    let diff = 0;
    for (let i = 0; i < word.length; i++) {
      if (word[i] !== words[i]) diff++;
      if (diff > 1) return false;
    }
    return diff === 1;
  });
};
```

0677 Map Sum Pairs

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 677. Map Sum Pairs
 * https://leetcode.com/problems/map-sum-pairs/
 * Difficulty: Medium
 *
 * Design a map that allows you to do the following:
 * - Maps a string key to a given value.
 * - Returns the sum of the values that have a key with a prefix equal to a given string.
 *
 * Implement the MapSum class:
 * - MapSum() Initializes the MapSum object.
 * - void insert(String key, int val) Inserts the key-val pair into the map. If the key
 *   already existed, the original key-value pair will be overridden to the new one.
 * - int sum(string prefix) Returns the sum of all the pairs' value whose key starts
 *   with the prefix.
 */

var MapSum = function() {
  this.map = new Map();
};

/**
 * @param {string} key
 * @param {number} val
 * @return {void}
 */
MapSum.prototype.insert = function(key, val) {
  this.map.set(key, val);
};

/**
 * @param {string} prefix
 * @return {number}
 */
MapSum.prototype.sum = function(prefix) {
  return [...this.map.entries()].reduce((total, [key, val]) => {
    return key.startsWith(prefix) ? total + val : total;
  }, 0);
};
```

0678 Valid Parenthesis String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 678. Valid Parenthesis String
 * https://leetcode.com/problems/valid-parenthesis-string/
 * Difficulty: Medium
 *
 * Given a string s containing only three types of characters: '(', ')' and '*', return
 * true if s is valid.
 *
 * The following rules define a valid string:
 * - Any left parenthesis '(' must have a corresponding right parenthesis ')'.
 * - Any right parenthesis ')' must have a corresponding left parenthesis '('.
 * - Left parenthesis '(' must go before the corresponding right parenthesis ')'.
 * - '*' could be treated as a single right parenthesis ')' or a single left parenthesis
 *   '(' or an empty string "".
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var checkValidString = function(s) {
  let result = 0;
  let offset = 0;

  for (const character of s) {
```

```
    result += character === '(' ? 1 : -1;
    offset += character !== ')' ? 1 : -1;
    if (offset < 0) return false;
    result = Math.max(0, result);
  }

  return !result;
};
```

0679 24 Game

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 679. 24 Game
 * https://leetcode.com/problems/24-game/
 * Difficulty: Hard
 *
 * You are given an integer array cards of length 4. You have four cards, each containing a
number
 * in the range [1, 9]. You should arrange the numbers on these cards in a mathematical
expression
 * using the operators ['+', '-', '*', '/'] and the parentheses '(' and ')' to get the value
24.
 *
 * You are restricted with the following rules:
 * - The division operator '/' represents real division, not integer division.
 *   - For example, 4 / (1 - 2 / 3) = 4 / (1 / 3) = 12.
 * - Every operation done is between two numbers. In particular, we cannot use '-' as a
 *   unary operator.
 *   - For example, if cards = [1, 1, 1, 1], the expression "-1 - 1 - 1 - 1" is not allowed.
 * - You cannot concatenate numbers together
 *   - For example, if cards = [1, 2, 1, 2], the expression "12 + 12" is not valid.
 *
 * Return true if you can get such expression that evaluates to 24, and false otherwise.
 */

/**
 * @param {number[]} cards
 * @return {boolean}
 */
const judgePoint24 = function(cards) {
  if (cards.length === 1) return Math.abs(cards[0] - 24) < 0.1;

  for (let i = 0; i < cards.length; i++) {
    for (let j = i + 1; j < cards.length; j++) {
      const remaining = new Array(cards.length - 1);
      for (let index = 0, current = 0; current < cards.length; current++) {
        if (i === current || j === current) continue;
        remaining[index++] = cards[current];
      }
      const a = cards[i];
      const b = cards[j];
      const operations = [a + b, a - b, b - a, a * b, a / b, b / a];
      for (const result of operations) {
        if (result === 0) continue;
        remaining[cards.length - 2] = result;
        if (judgePoint24(remaining)) return true;
      }
    }
  }

  return false;
};
```

0680 Valid Palindrome Ii

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 680. Valid Palindrome II
 * https://leetcode.com/problems/valid-palindrome-ii/
 * Difficulty: Easy
 *
 * Given a string s, return true if the s can be palindrome after deleting at most one
 * character from it.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var validPalindrome = function(s) {
  for (let left = 0, right = s.length - 1; left < right; left++, right--) {
    if (s[left] !== s[right]) {
      return isPalindrome(s, left + 1, right) || isPalindrome(s, left, right - 1);
    }
  }

  return true;
};

function isPalindrome(s, left, right) {
  while (left < right) {
    if (s[left] !== s[right]) {
      return false;
    }
    right--;
    left++;
  }

  return true;
}
```

0681 Next Closest Time

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 681. Next Closest Time
 * https://leetcode.com/problems/next-closest-time/
 * Difficulty: Medium
 *
 * Given a time represented in the format "HH:MM", form the next closest time by reusing the
 * current digits. There is no limit on how many times a digit can be reused.
 *
 * You may assume the given input string is always valid. For example, "01:34", "12:09" are
 * all valid. "1:34", "12:9" are all invalid.
 */

/**
 * @param {string} time
 * @return {string}
 */
var nextClosestTime = function(time) {
  const digits = new Set(time.replace(':', ''));
  const sortedDigits = [...digits].sort();
```

```
  return generateNextTime(time);

  function isValidTime(h, m) {
    return h < 24 && m < 60;
  }

  function generateNextTime(timeStr) {
    const [hours, minutes] = timeStr.split(':');

    for (let i = 3; i >= 0; i--) {
      const pos = i < 2 ? i : i + 1;
      const currentDigit = timeStr[pos];

      for (const digit of sortedDigits) {
        if (digit > currentDigit) {
          const newTime = timeStr.substring(0, pos) + digit + timeStr.substring(pos + 1);
          const [h, m] = newTime.split(':').map(Number);
          if (isValidTime(h, m)) {
            return newTime;
          }
        }
      }

      const newTime = timeStr.substring(0, pos) + sortedDigits[0] + timeStr.substring(pos +
1);
      timeStr = newTime;
    }

    return sortedDigits[0].repeat(2) + ':' + sortedDigits[0].repeat(2);
  }
};
```

0682 Baseball Game

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 682. Baseball Game
 * https://leetcode.com/problems/baseball-game/
 * Difficulty: Easy
 *
 * You are keeping the scores for a baseball game with strange rules. At the beginning of the
 * game, you start with an empty record.
 *
 * You are given a list of strings operations, where operations[i] is the ith operation you
 * must apply to the record and is one of the following:
 * - An integer x.
 *   - Record a new score of x.
 * - '+'.
 *   - Record a new score that is the sum of the previous two scores.
 * - 'D'.
 *   - Record a new score that is the double of the previous score.
 * - 'C'.
 *   - Invalidate the previous score, removing it from the record.
 *
 * Return the sum of all the scores on the record after applying all the operations.
 *
 * The test cases are generated such that the answer and all intermediate calculations fit in
a
 * 32-bit integer and that all operations are valid.
 */

/**
 * @param {string[]} ops
 * @return {number}
 */
var calPoints = function(ops) {
  const stack = [];

  for (const op of ops) {
```

```
    if (op === 'C') stack.pop();
    else if (op === 'D') stack.push(stack.at(-1) * 2);
    else if (op === '+') stack.push(stack.at(-1) + stack.at(-2));
    else stack.push(Number(op));
  }

  return stack.reduce((sum, num) => sum + num, 0);
};
```

0683 K Empty Slots

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 683. K Empty Slots
 * https://leetcode.com/problems/k-empty-slots/
 * Difficulty: Hard
 *
 * You have n bulbs in a row numbered from 1 to n. Initially, all the bulbs are turned off.
 * We turn on exactly one bulb every day until all bulbs are on after n days.
 *
 * You are given an array bulbs of length n where bulbs[i] = x means that on the (i+1)th day,
 * we will turn on the bulb at position x where i is 0-indexed and x is 1-indexed.
 *
 * Given an integer k, return the minimum day number such that there exists two turned on
 * bulbs that have exactly k bulbs between them that are all turned off. If there isn't
 * such day, return -1.
 */

/**
 * @param {number[]} bulbs
 * @param {number} k
 * @return {number}
 */
var kEmptySlots = function(bulbs, k) {
  const days = new Array(bulbs.length + 1);

  for (let i = 0; i < bulbs.length; i++) {
    days[bulbs[i]] = i + 1;
  }

  let left = 1;
  let right = k + 2;
  let minDay = Infinity;

  while (right <= bulbs.length) {
    let mid = left + 1;
    let isValid = true;

    while (mid < right) {
      if (days[mid] < Math.max(days[left], days[right])) {
        isValid = false;
        left = mid;
        right = mid + k + 1;
        break;
      }
      mid++;
    }

    if (isValid) {
      minDay = Math.min(minDay, Math.max(days[left], days[right]));
      left++;
      right++;
    }
  }

  return minDay === Infinity ? -1 : minDay;
};
```

0684 Redundant Connection

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 684. Redundant Connection
 * https://leetcode.com/problems/redundant-connection/
 * Difficulty: Medium
 *
 * In this problem, a tree is an undirected graph that is connected and has no cycles.
 *
 * You are given a graph that started as a tree with n nodes labeled from 1 to n, with
 * one additional edge added. The added edge has two different vertices chosen from 1
 * to n, and was not an edge that already existed. The graph is represented as an array
 * edges of length n where edges[i] = [ai, bi] indicates that there is an edge between
 * nodes ai and bi in the graph.
 *
 * Return an edge that can be removed so that the resulting graph is a tree of n nodes.
 * If there are multiple answers, return the answer that occurs last in the input.
 */

/**
 * @param {number[][]} edges
 * @return {number[]}
 */
var findRedundantConnection = function(edges) {
  const adjacency = new Map();

  function traverse(node, target, prev) {
    if (node === target) {
      return true;
    }
    for (const nextNode of adjacency.get(node)) {
      if (nextNode !== prev && traverse(nextNode, target, node)) {
        return true;
      }
    }
    return false;
  }

  for (const edge of edges) {
    const [a, b] = edge;
    adjacency.set(a, !adjacency.has(a) ? [b] : [...adjacency.get(a), b]);
    adjacency.set(b, !adjacency.has(b) ? [a] : [...adjacency.get(b), a]);

    if (traverse(b, a, a)) {
      return [a, b];
    }
  }
};
```

0685 Redundant Connection Ii

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 685. Redundant Connection II
 * https://leetcode.com/problems/redundant-connection-ii/
 * Difficulty: Hard
 *
 * In this problem, a rooted tree is a directed graph such that, there is exactly one node
 * (the root) for which all other nodes are descendants of this node, plus every node has
 * exactly one parent, except for the root node which has no parents.
 *
 * The given input is a directed graph that started as a rooted tree with n nodes (with
```

```
 * distinct values from 1 to n), with one additional directed edge added. The added edge
 * has two different vertices chosen from 1 to n, and was not an edge that already existed.
 *
 * The resulting graph is given as a 2D-array of edges. Each element of edges is a pair
 * [ui, vi] that represents a directed edge connecting nodes ui and vi, where ui is a parent
 * of child vi.
 *
 * Return an edge that can be removed so that the resulting graph is a rooted tree of n
 * nodes. If there are multiple answers, return the answer that occurs last in the given
2D-array.
 */

/**
 * @param {number[][]} edges
 * @return {number[]}
 */
var findRedundantDirectedConnection = function(edges) {
  const n = edges.length;
  const parent = Array.from({ length: n + 1 }, (_, i) => i);
  const rank = new Uint32Array(n + 1);
  const parents = new Int32Array(n + 1).fill(-1);
  let conflict = -1;
  let cycle = -1;

  edges.forEach(([u, v], i) => {
    if (parents[v] !== -1) conflict = i;
    else {
      parents[v] = u;
      union(u, v) || (cycle = i);
    }
  });

  return conflict === -1 ? edges[cycle] : cycle === -1
    ? edges[conflict] : [parents[edges[conflict][1]], edges[conflict][1]];

  function find(x) {
    return parent[x] === x ? x : (parent[x] = find(parent[x]));
  }
  function union(x, y) {
    const [px, py] = [find(x), find(y)];
    if (px === py) return false;
    rank[px] < rank[py]
      ? parent[px] = py
      : rank[px] > rank[py] ? parent[py] = px : (parent[py] = px, rank[px]++);
    return true;
  }
};
```

0686 Repeated String Match

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 686. Repeated String Match
 * https://leetcode.com/problems/repeated-string-match/
 * Difficulty: Easy
 *
 * Given two strings A and B, find the minimum number of times A has to be
 * repeated such that B is a substring of it. If no such solution, return -1.
 *
 * For example, with A = "abcd" and B = "cdabcdab".
 *
 * Return 3, because by repeating A three times (“abcdabcdabcd”), B is a
 * substring of it; and B is not a substring of A repeated two times ("abcdabcd").
 */

/**
 * @param {string} A
 * @param {string} B
 * @return {number}
```

```
 */
var repeatedStringMatch = function(A, B) {
  const length = Math.ceil(B.length / A.length);
  if (A.repeat(length).includes(B)) return length;
  if ((A.repeat(length) + A).includes(B)) return length + 1;
  return -1;
};
```

0687 Longest Univalue Path

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 687. Longest Univalue Path
 * https://leetcode.com/problems/longest-univalue-path/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the length of the longest path, where each node
 * in the path has the same value. This path may or may not pass through the root.
 *
 * The length of the path between two nodes is represented by the number of edges between
them.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var longestUnivaluePath = function(root) {
  let max = 0;
  dfs(root);
  return max;

  function dfs(node) {
    if (!node) return 0;

    const left = dfs(node.left);
    const right = dfs(node.right);
    const leftPath = node.left?.val === node.val ? left + 1 : 0;
    const rightPath = node.right?.val === node.val ? right + 1 : 0;

    max = Math.max(max, leftPath + rightPath);
    return Math.max(leftPath, rightPath);
  }
};
```

0688 Knight Probability In Chessboard

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 688. Knight Probability in Chessboard
 * https://leetcode.com/problems/knight-probability-in-chessboard/
 * Difficulty: Medium
 *
 * On an n x n chessboard, a knight starts at the cell (row, column) and attempts to make
exactly
```

```
 * k moves. The rows and columns are 0-indexed, so the top-left cell is (0, 0), and the
bottom-right
 * cell is (n - 1, n - 1).
 *
 * A chess knight has eight possible moves it can make, as illustrated below. Each move is two
 cells
 * in a cardinal direction, then one cell in an orthogonal direction.
 *
 * Each time the knight is to move, it chooses one of eight possible moves uniformly at random
 * (even if the piece would go off the chessboard) and moves there.
 *
 * The knight continues moving until it has made exactly k moves or has moved off the
chessboard.
 *
 * Return the probability that the knight remains on the board after it has stopped moving.
 */

/**
 * @param {number} n
 * @param {number} k
 * @param {number} row
 * @param {number} column
 * @return {number}
 */
var knightProbability = function(n, k, row, column) {
  const moves = [[-2, -1], [-2, 1], [-1, -2], [-1, 2], [1, -2], [1, 2], [2, -1], [2, 1]];
  const dp = new Map();

  return calc(k, row, column);

  function calc(movesLeft, r, c) {
    if (r < 0 || r >= n || c < 0 || c >= n) return 0;
    if (movesLeft === 0) return 1;

    const key = `${movesLeft},${r},${c}`;
    if (dp.has(key)) return dp.get(key);

    let prob = 0;
    for (const [dr, dc] of moves) {
      prob += calc(movesLeft - 1, r + dr, c + dc) / 8;
    }

    dp.set(key, prob);
    return prob;
  }
};
```

0689 Maximum Sum Of 3 Non Overlapping Subarrays

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 689. Maximum Sum of 3 Non-Overlapping Subarrays
 * https://leetcode.com/problems/maximum-sum-of-3-non-overlapping-subarrays/
 * Difficulty: Hard
 *
 * Given an integer array nums and an integer k, find three non-overlapping subarrays of
 * length k with maximum sum and return them.
 *
 * Return the result as a list of indices representing the starting position of each
 * interval (0-indexed). If there are multiple answers, return the lexicographically
 * smallest one.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var maxSumOfThreeSubarrays = function(nums, k) {
```

```
  const n = nums.length;
  let [sumFirst, sumSecond, sumThird] = [0, 0, 0];
  let [maxFirst, maxFirstSecond, maxTotal] = [0, 0, 0];
  let [startFirst, startFirstBest, startSecondBest] = [0, 0, k];
  let result = [0, k, 2 * k];

  for (let i = 0; i < k; i++) {
    sumFirst += nums[i];
    sumSecond += nums[i + k];
    sumThird += nums[i + 2 * k];
  }
  [maxFirst, maxFirstSecond, maxTotal] = [
    sumFirst, sumFirst + sumSecond,
    sumFirst + sumSecond + sumThird
  ];

  for (let i = 1; i <= n - 3 * k; i++) {
    sumFirst += nums[i + k - 1] - nums[i - 1];
    sumSecond += nums[i + 2 * k - 1] - nums[i + k - 1];
    sumThird += nums[i + 3 * k - 1] - nums[i + 2 * k - 1];

    if (sumFirst > maxFirst) [maxFirst, startFirst] = [sumFirst, i];
    if (maxFirst + sumSecond > maxFirstSecond) {
      [maxFirstSecond, startFirstBest, startSecondBest] = [maxFirst + sumSecond, startFirst, i
 + k];
    }
    if (maxFirstSecond + sumThird > maxTotal) {
      [maxTotal, ...result] = [
        maxFirstSecond + sumThird,
        startFirstBest,
        startSecondBest, i + 2 * k
      ];
    }
  }

  return result;
};
```

0690 Employee Importance

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 690. Employee Importance
 * https://leetcode.com/problems/employee-importance/
 * Difficulty: Medium
 *
 * You have a data structure of employee information, including the employee's unique ID,
 * importance value, and direct subordinates' IDs.
 *
 * You are given an array of employees employees where:
 * - employees[i].id is the ID of the ith employee.
 * - employees[i].importance is the importance value of the ith employee.
 * - employees[i].subordinates is a list of the IDs of the direct subordinates of the ith
employee.
 *
 * Given an integer id that represents an employee's ID, return the total importance value of
this
 * employee and all their direct and indirect subordinates.
 */

/**
 * Definition for Employee.
 * function Employee(id, importance, subordinates) {
 *     this.id = id;
 *     this.importance = importance;
 *     this.subordinates = subordinates;
 * }
 */
```

```
/**
 * @param {Employee[]} employees
 * @param {number} id
 * @return {number}
 */
var GetImportance = function(employees, id) {
  const map = new Map(employees.map(e => [e.id, e]));
  return dfs(id);

  function dfs(id) {
    if (!map.has(id)) return 0;
    const e = map.get(id);
    return e.importance + e.subordinates.reduce((sum, id) => sum + dfs(id), 0);
  }
};
```

0691 Stickers To Spell Word

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 691. Stickers to Spell Word
 * https://leetcode.com/problems/stickers-to-spell-word/
 * Difficulty: Hard
 *
 * We are given n different types of stickers. Each sticker has a lowercase English word on
it.
 *
 * You would like to spell out the given string target by cutting individual letters from your
 * collection of stickers and rearranging them. You can use each sticker more than once if you
 * want, and you have infinite quantities of each sticker.
 *
 * Return the minimum number of stickers that you need to spell out target. If the task is
 * impossible, return -1.
 *
 * Note: In all test cases, all words were chosen randomly from the 1000 most common US
English
 * words, and target was chosen as a concatenation of two random words.
 */

/**
 * @param {string[]} stickers
 * @param {string} target
 * @return {number}
 */
var minStickers = function(stickers, target) {
  const dp = new Map([['', 0]]);
  return helper(target);

  function helper(str) {
    if (dp.has(str)) return dp.get(str);
    let result = Infinity;

    for (const s of stickers.filter(s => s.includes(str[0]))) {
      result = Math.min(result, 1 + helper(calcDiff(str, s)));
    }

    dp.set(str, result === Infinity || result === 0 ? -1 : result);
    return dp.get(str);
  }

  function calcDiff(str1, str2) {
    for (const c of str2) {
      if (str1.includes(c)) str1 = str1.replace(c, '');
    }
    return str1;
  }
};
```

0692 Top K Frequent Words

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 692. Top K Frequent Words
 * https://leetcode.com/problems/top-k-frequent-words/
 * Difficulty: Medium
 *
 * Given an array of strings words and an integer k, return the k most frequent strings.
 *
 * Return the answer sorted by the frequency from highest to lowest. Sort the words with
 * the same frequency by their lexicographical order.
 */

/**
 * @param {string[]} words
 * @param {number} k
 * @return {string[]}
 */
var topKFrequent = function(words, k) {
  const map = new Map();
  words.forEach(word => map.set(word, (map.get(word) || 0) + 1));
  return [...map.entries()]
    .sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]))
    .slice(0, k)
    .map(([word]) => word);
};
```

0693 Binary Number With Alternating Bits

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 693. Binary Number with Alternating Bits
 * https://leetcode.com/problems/binary-number-with-alternating-bits/
 * Difficulty: Easy
 *
 * Given a positive integer, check whether it has alternating bits: namely, if two adjacent
 * bits will always have different values.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var hasAlternatingBits = function(n) {
  for (let i = 1, b = n.toString(2); i < b.length; i++) {
    if (b[i - 1] === b[i]) return false;
  }
  return true;
};
```

0694 Number Of Distinct Islands

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
```

```
 * 694. Number of Distinct Islands
 * https://leetcode.com/problems/number-of-distinct-islands/
 * Difficulty: Medium
 *
 * You are given an m x n binary matrix grid. An island is a group of 1's (representing land)
 * connected 4-directionally (horizontal or vertical.) You may assume all four edges of the
 * grid are surrounded by water.
 *
 * An island is considered to be the same as another if and only if one island can be
translated
 * (and not rotated or reflected) to equal the other.
 *
 * Return the number of distinct islands.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var numDistinctIslands = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const visited = Array.from({ length: rows }, () => new Array(cols).fill(false));
  const islandShapes = new Set();

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 1 && !visited[i][j]) {
        const shape = [];
        dfs(i, j, i, j, shape);
        islandShapes.add(shape.join('|'));
      }
    }
  }

  return islandShapes.size;

  function dfs(row, col, baseRow, baseCol, shape) {
    if (row < 0 || row >= rows || col < 0 || col >= cols
      || visited[row][col] || grid[row][col] === 0) {
      return;
    }

    visited[row][col] = true;
    shape.push(`${row - baseRow},${col - baseCol}`);

    dfs(row + 1, col, baseRow, baseCol, shape);
    dfs(row - 1, col, baseRow, baseCol, shape);
    dfs(row, col + 1, baseRow, baseCol, shape);
    dfs(row, col - 1, baseRow, baseCol, shape);
  }
};
```

0695 Max Area Of Island

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 695. Max Area of Island
 * https://leetcode.com/problems/max-area-of-island/
 * Difficulty: Medium
 *
 * You are given an m x n binary matrix grid. An island is a group of 1's (representing land)
 * connected 4-directionally (horizontal or vertical.) You may assume all four edges of the
 * grid are surrounded by water.
 *
 * The area of an island is the number of cells with a value 1 in the island.
 *
 * Return the maximum area of an island in grid. If there is no island, return 0.
 */
```

```
/**
 * @param {number[][]} grid
 * @return {number}
 */
var maxAreaOfIsland = function(grid) {
  const cache = new Set();
  let max = 0;

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[0].length; j++) {
      max = Math.max(max, traverse(grid, cache, i, j));
    }
  }

  return max;
};

function traverse(grid, cache, x, y) {
  let count = 0;
  if (grid[x] && grid[x][y] === 1 && !cache.has(`${x},${y}`)) {
    cache.add(`${x},${y}`);
    count += [[x - 1, y], [x + 1, y], [x, y - 1], [x, y + 1]]
      .map(([x, y]) => traverse(grid, cache, x, y))
      .reduce((sum, count) => sum + count, 0) + 1;
  }
  return count;
}
```

0696 Count Binary Substrings

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 696. Count Binary Substrings
 * https://leetcode.com/problems/count-binary-substrings/
 * Difficulty: Easy
 *
 * Given a binary string s, return the number of non-empty substrings that have the same
 * number of 0's and 1's, and all the 0's and all the 1's in these substrings are grouped
 * consecutively.
 *
 * Substrings that occur multiple times are counted the number of times they occur.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countBinarySubstrings = function(s) {
  let result = 0;

  for (let i = 0, group = [], count = 1, prevCount = 0; i < s.length; i++) {
    if (s[i] === s[i + 1]) {
      count++;
    } else {
      if (prevCount) {
        result += prevCount <= count ? prevCount : count;
      }
      prevCount = count;
      count = 1;
    }
  }

  return result;
};
```

0697 Degree Of An Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 697. Degree of an Array
 * https://leetcode.com/problems/degree-of-an-array/
 * Difficulty: Easy
 *
 * Given a non-empty array of non-negative integers nums, the degree of this array is
 * defined as the maximum frequency of any one of its elements.
 *
 * Your task is to find the smallest possible length of a (contiguous) subarray of
 * nums, that has the same degree as nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findShortestSubArray = function(nums) {
  const map = new Map();

  nums.forEach((key, index) => {
    if (!map.has(key)) {
      map.set(key, { start: index, end: index, count: 1 });
    }
    const { start, end, count } = map.get(key);
    map.set(key, { start, end: index, count: count + 1 });
  });

  let max = 0;
  let result = Infinity;
  Array.from(map).forEach(([_, { start, end, count }]) => {
    const min = (end - start) + 1;
    if (count > max || (min < result && count === max)) {
      result = min;
      max = count;
    }
  });

  return result;
};
```

0698 Partition To K Equal Sum Subsets

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 698. Partition to K Equal Sum Subsets
 * https://leetcode.com/problems/partition-to-k-equal-sum-subsets/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return true if it is possible to divide this
 * array into k non-empty subsets whose sums are all equal.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
var canPartitionKSubsets = function(nums, k) {
  const sum = nums.reduce((a, b) => a + b);
  if (sum % k !== 0) {
```

```
    return false;
  }

  const target = sum / k;
  nums.sort((a, b) => b - a);
  if (nums[0] > target) {
    return false;
  }

  const used = new Array(nums.length).fill(false);
  return backtrack(0, 0, 0);

  function backtrack(index, count, updatedSum) {
    if (count === k) return true;
    if (updatedSum === target) return backtrack(0, count + 1, 0);
    if (updatedSum > target) return false;

    for (let i = index; i < nums.length; i++) {
      if (!used[i]) {
        used[i] = true;
        if (backtrack(i + 1, count, updatedSum + nums[i])) return true;
        used[i] = false;
        if (updatedSum === 0) break;
      }
    }
    return false;
  }
};
```

0699 Falling Squares

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 699. Falling Squares
 * https://leetcode.com/problems/falling-squares/
 * Difficulty: Hard
 *
 * There are several squares being dropped onto the X-axis of a 2D plane.
 *
 * You are given a 2D integer array positions where positions[i] = [lefti, sideLengthi]
 * represents the ith square with a side length of sideLengthi that is dropped with its
 * left edge aligned with X-coordinate lefti.
 *
 * Each square is dropped one at a time from a height above any landed squares. It then
 * falls downward (negative Y direction) until it either lands on the top side of another
 * square or on the X-axis. A square brushing the left/right side of another square does
 * not count as landing on it. Once it lands, it freezes in place and cannot be moved.
 *
 * After each square is dropped, you must record the height of the current tallest stack
 * of squares.
 *
 * Return an integer array ans where ans[i] represents the height described above after
 * dropping the ith square.
 */

/**
 * @param {number[][]} positions
 * @return {number[]}
 */
var fallingSquares = function(positions) {
  const map = new Map();
  const result = [];
  let max = 0;

  for (const [left, side] of positions) {
    const right = left + side;
    let height = 0;

    for (const [start, [i, n]] of map) {
```

```
      const end = start + i;
      if (right > start && left < end) {
        height = Math.max(height, n);
      }
    }

    height += side;
    map.set(left, [side, height]);
    max = Math.max(max, height);
    result.push(max);
  }

  return result;
};
```

0700 Search In A Binary Search Tree

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 700. Search in a Binary Search Tree
 * https://leetcode.com/problems/search-in-a-binary-search-tree/
 * Difficulty: Easy
 *
 * You are given the root of a binary search tree (BST) and an integer val.
 *
 * Find the node in the BST that the node's value equals val and return the
 * subtree rooted with that node. If such a node does not exist, return null.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} val
 * @return {TreeNode}
 */
var searchBST = function(root, val) {
  while (root) {
    if (root.val === val) {
      return root;
    }
    root = root.val > val ? root.left : root.right;
  }

  return null;
};
```

0701 Insert Into A Binary Search Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 701. Insert into a Binary Search Tree
 * https://leetcode.com/problems/insert-into-a-binary-search-tree/
 * Difficulty: Medium
 *
 * You are given the root node of a binary search tree (BST) and a value
 * to insert into the tree. Return the root node of the BST after the
```

```
 * insertion. It is guaranteed that the new value does not exist in the
 * original BST.
 *
 * Notice that there may exist multiple valid ways for the insertion,
 * as long as the tree remains a BST after insertion. You can return
 * any of them.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} val
 * @return {TreeNode}
 */
var insertIntoBST = function(root, val) {
  if (!root) {
    return new TreeNode(val);
  }

  if (val > root.val) {
    root.right = insertIntoBST(root.right, val);
  } else {
    root.left = insertIntoBST(root.left, val);
  }

  return root;
};
```

0702 Search In A Sorted Array Of Unknown Size

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 702. Search in a Sorted Array of Unknown Size
 * https://leetcode.com/problems/search-in-a-sorted-array-of-unknown-size/
 * Difficulty: Medium
 *
 * This is an interactive problem.
 *
 * You have a sorted array of unique elements and an unknown size. You do not have an access
 * to the array but you can use the ArrayReader interface to access it. You can call
 * ArrayReader.get(i) that:
 * - returns the value at the ith index (0-indexed) of the secret array (i.e., secret[i]), or
 * - returns 231 - 1 if the i is out of the boundary of the array.
 *
 * You are also given an integer target.
 *
 * Return the index k of the hidden array where secret[k] == target or return -1 otherwise.
 *
 * You must write an algorithm with O(log n) runtime complexity.
 */

/**
 * @param {ArrayReader} reader
 * @param {number} target
 * @return {number}
 */
var search = function(reader, target) {
  let left = 0;
  let right = 1;

  while (reader.get(right) < target) {
    left = right;
```

```
    right *= 2;
  }

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    const value = reader.get(mid);

    if (value === target) {
      return mid;
    } else if (value < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return -1;
};
```

0703 Kth Largest Element In A Stream

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 703. Kth Largest Element in a Stream
 * https://leetcode.com/problems/kth-largest-element-in-a-stream/
 * Difficulty: Easy
 *
 * You are part of a university admissions office and need to keep track of the kth highest
test
 * score from applicants in real-time. This helps to determine cut-off marks for interviews
and
 * admissions dynamically as new applicants submit their scores.
 *
 * You are tasked to implement a class which, for a given integer k, maintains a stream of
test
 * scores and continuously returns the kth highest test score after a new score has been
submitted.
 * More specifically, we are looking for the kth highest score in the sorted list of all
scores.
 *
 * Implement the KthLargest class:
 * - KthLargest(int k, int[] nums) Initializes the object with the integer k and the stream of
 *   test scores nums.
 * - int add(int val) Adds a new test score val to the stream and returns the element
representing
 *   the kth largest element in the pool of test scores so far.
 */

/**
 * @param {number} k
 * @param {number[]} nums
 */
var KthLargest = function(k, nums) {
  this.main = new MinPriorityQueue();
  this.k = k;

  nums.forEach(n => this.main.enqueue(n));

  while (this.main.size() > k) {
    this.main.dequeue().element;
  }
};

/**
 * @param {number} val
 * @return {number}
 */
KthLargest.prototype.add = function(val) {
  this.main.enqueue(val);
```

```
  if (this.main.size() > this.k) {
    this.main.dequeue().element;
  }

  return this.main.front().element;
};
```

0704 Binary Search

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 704. Binary Search
 * https://leetcode.com/problems/binary-search/
 * Difficulty: Easy
 *
 * Given an array of integers nums which is sorted in ascending order, and an
 * integer target, write a function to search target in nums. If target exists,
 * then return its index. Otherwise, return -1.
 *
 * You must write an algorithm with O(log n) runtime complexity.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var search = function(nums, target) {
  let left = 0;
  let right = nums.length - 1;

  while (left <= right) {
    const pivot = Math.floor((right + left) / 2);
    if (nums[pivot] === target) return pivot;
    if (target < nums[pivot]) right = pivot - 1;
    else left = pivot + 1;
  }

  return -1;
};
```

0705 Design Hashset

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 705. Design HashSet
 * https://leetcode.com/problems/design-hashset/
 * Difficulty: Easy
 *
 * Design a HashSet without using any built-in hash table libraries.
 *
 * Implement MyHashSet class:
 * - void add(key) Inserts the value key into the HashSet.
 * - bool contains(key) Returns whether the value key exists in the
 *   HashSet or not.
 * - void remove(key) Removes the value key in the HashSet. If key
 *   does not exist in the HashSet, do nothing.
 */

var MyHashSet = function() {
  this._keys = [];
};
```

```
/**
 * @param {number} key
 * @return {void}
 */
MyHashSet.prototype.add = function(key) {
  this._keys[key] = 1;
};

/**
 * @param {number} key
 * @return {void}
 */
MyHashSet.prototype.remove = function(key) {
  this._keys[key] = undefined;
};

/**
 * @param {number} key
 * @return {boolean}
 */
MyHashSet.prototype.contains = function(key) {
  return this._keys[key] !== undefined;
};
```

0706 Design Hashmap

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 706. Design HashMap
 * https://leetcode.com/problems/design-hashmap/
 * Difficulty: Easy
 *
 * Design a HashMap without using any built-in hash table libraries.
 * Implement the MyHashMap class:
 * - MyHashMap() initializes the object with an empty map.
 * - void put(int key, int value) inserts a (key, value) pair into the HashMap.
 *   If the key already exists in the map, update the corresponding value.
 * - int get(int key) returns the value to which the specified key is mapped,
 *   or -1 if this map contains no mapping for the key.
 * - void remove(key) removes the key and its corresponding value if the map
 *   contains the mapping for the key.
 */


var MyHashMap = function() {
  this._keys = [];
  this._values = [];
};

/**
 * @param {number} key
 * @param {number} value
 * @return {void}
 */
MyHashMap.prototype.put = function(key, value) {
  if (this._keys[key] === undefined) {
    this._values.push(value);
    this._keys[key] = this._values.length - 1;
  } else {
    this._values[this._keys[key]] = value;
  }
};

/**
 * @param {number} key
 * @return {number}
 */
MyHashMap.prototype.get = function(key) {
```

```
  const offset = this._keys[key];
  return offset === undefined ? -1 : this._values[offset];
}
/**
 * @param {number} key
 * @return {void}
 */
MyHashMap.prototype.remove = function(key) {
  if (this._keys[key] !== undefined) {
    this._values[this._keys[key]] = undefined;
    this._keys[key] = undefined;
  }
};
```

0707 Design Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 707. Design Linked List
 * https://leetcode.com/problems/design-linked-list/
 * Difficulty: Medium
 *
 * Design your implementation of the linked list. You can choose to use a singly or
 * doubly linked list.
 *
 * A node in a singly linked list should have two attributes: val and next. val is the value
 * of the current node, and next is a pointer/reference to the next node.
 *
 * If you want to use the doubly linked list, you will need one more attribute prev to
indicate
 * the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.
 *
 * Implement the MyLinkedList class:
 * - MyLinkedList() Initializes the MyLinkedList object.
 * - int get(int index) Get the value of the indexth node in the linked list. If the
 *   index is invalid, return -1.
 * - void addAtHead(int val) Add a node of value val before the first element of the
 *   linked list. After the insertion, the new node will be the first node of the linked list.
 * - void addAtTail(int val) Append a node of value val as the last element of the linked
list.
 * - void addAtIndex(int index, int val) Add a node of value val before the indexth node
 *   in the linked list. If index equals the length of the linked list, the node will be
 *   appended to the end of the linked list. If index is greater than the length, the node
 *   will not be inserted.
 * - void deleteAtIndex(int index) Delete the indexth node in the linked list, if the index
 *   is valid.
 */

var MyLinkedList = function() {
  this.head = null;
  this.size = 0;
};

/**
 * @param {number} index
 * @return {number}
 */
MyLinkedList.prototype.get = function(index) {
  if (index < 0 || index >= this.size) return -1;
  let current = this.head;
  while (index--) current = current.next;
  return current.val;
};

/**
 * @param {number} val
 * @return {void}
 */
MyLinkedList.prototype.addAtHead = function(val) {
```

```
  const node = new Node(val);
  node.next = this.head;
  this.head = node;
  this.size++;
};

/**
 * @param {number} val
 * @return {void}
 */
MyLinkedList.prototype.addAtTail = function(val) {
  const node = new Node(val);
  if (!this.head) {
    this.head = node;
  } else {
    let current = this.head;
    while (current.next) current = current.next;
    current.next = node;
  }
  this.size++;
};

/**
 * @param {number} index
 * @param {number} val
 * @return {void}
 */
MyLinkedList.prototype.addAtIndex = function(index, val) {
  if (index < 0 || index > this.size) return;
  if (index === 0) return this.addAtHead(val);
  const node = new Node(val);
  let current = this.head;
  while (--index) current = current.next;
  node.next = current.next;
  current.next = node;
  this.size++;
};

/**
 * @param {number} index
 * @return {void}
 */
MyLinkedList.prototype.deleteAtIndex = function(index) {
  if (index < 0 || index >= this.size) return;
  this.size--;
  if (index === 0) {
    this.head = this.head.next;
    return;
  }
  let current = this.head;
  while (--index) current = current.next;
  current.next = current.next.next;
};

class Node {
  constructor(val) {
    this.val = val;
    this.next = null;
  }
}
```

0708 Insert Into A Sorted Circular Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 708. Insert into a Sorted Circular Linked List
 * https://leetcode.com/problems/insert-into-a-sorted-circular-linked-list/
 * Difficulty: Medium
 *
```

```
 * Given a Circular Linked List node, which is sorted in non-descending order, write a
 * function to insert a value insertVal into the list such that it remains a sorted
 * circular list. The given node can be a reference to any single node in the list and
 * may not necessarily be the smallest value in the circular list.
 *
 * If there are multiple suitable places for insertion, you may choose any place to
 * insert the new value. After the insertion, the circular list should remain sorted.
 *
 * If the list is empty (i.e., the given node is null), you should create a new single
 * circular list and return the reference to that single node. Otherwise, you should
 * return the originally given node.
 */

/**
 * @param {_Node} head
 * @param {number} insertVal
 * @return {_Node}
 */
var insert = function(head, insertVal) {
  const newNode = new _Node(insertVal, null);

  if (!head) {
    newNode.next = newNode;
    return newNode;
  }

  if (head.next === head) {
    newNode.next = head;
    head.next = newNode;
    return head;
  }

  let prev = head;
  let curr = head.next;
  do {
    if ((prev.val <= insertVal && insertVal <= curr.val)
        || (prev.val > curr.val && (insertVal >= prev.val || insertVal <= curr.val))) {
      break;
    }
    prev = curr;
    curr = curr.next;
  } while (prev !== head);

  prev.next = newNode;
  newNode.next = curr;

  return head;
};
```

0709 To Lower Case

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 709. To Lower Case
 * https://leetcode.com/problems/to-lower-case/
 * Difficulty: Easy
 *
 * Given a string s, return the string after replacing every uppercase letter with the same
 * lowercase letter.
 */

/**
 * @param {string} s
 * @return {string}
 */
var toLowerCase = function(s) {
  return s.toLowerCase();
};
```

0710 Random Pick With Blacklist

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 710. Random Pick with Blacklist
 * https://leetcode.com/problems/random-pick-with-blacklist/
 * Difficulty: Hard
 *
 * You are given an integer n and an array of unique integers blacklist. Design an algorithm
to
 * pick a random integer in the range [0, n - 1] that is not in blacklist. Any integer that is
 * in the mentioned range and not in blacklist should be equally likely to be returned.
 *
 * Optimize your algorithm such that it minimizes the number of calls to the built-in random
 * function of your language.
 *
 * Implement the Solution class:
 * - Solution(int n, int[] blacklist) Initializes the object with the integer n and the
 *   blacklisted integers blacklist.
 * - int pick() Returns a random integer in the range [0, n - 1] and not in blacklist.
 */

/**
 * @param {number} n
 * @param {number[]} blacklist
 */
var Solution = function(n, blacklist) {
  this.size = n - blacklist.length;
  this.mapping = new Map();
  blacklist = new Set(blacklist);

  let last = n - 1;
  for (const b of blacklist) {
    if (b < this.size) {
      while (blacklist.has(last)) last--;
      this.mapping.set(b, last--);
    }
  }
};

/**
 * @return {number}
 */
Solution.prototype.pick = function() {
  const index = Math.floor(Math.random() * this.size);
  return this.mapping.get(index) ?? index;
};
```

0711 Number Of Distinct Islands Ii

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 711. Number of Distinct Islands II
 * https://leetcode.com/problems/number-of-distinct-islands-ii/
 * Difficulty: Hard
 *
 * You are given an m x n binary matrix grid. An island is a group of 1's (representing land)
 * connected 4-directionally (horizontal or vertical.) You may assume all four edges of the
 * grid are surrounded by water.
 *
 * An island is considered to be the same as another if they have the same shape, or have
```

```
 * the same shape after rotation (90, 180, or 270 degrees only) or reflection (left/right
 * direction or up/down direction).
 *
 * Return the number of distinct islands.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var numDistinctIslands2 = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const visited = Array.from({ length: rows }, () => new Array(cols).fill(false));
  const uniqueShapes = new Set();

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 1 && !visited[i][j]) {
        const cells = [];
        dfs(i, j, cells);
        const canonical = normalize(cells);
        uniqueShapes.add(canonical);
      }
    }
  }

  return uniqueShapes.size;

  function dfs(row, col, cells) {
    if (row < 0 || row >= rows || col < 0 || col >= cols
        || visited[row][col] || grid[row][col] === 0) {
      return;
    }

    visited[row][col] = true;
    cells.push([row, col]);

    dfs(row + 1, col, cells);
    dfs(row - 1, col, cells);
    dfs(row, col + 1, cells);
    dfs(row, col - 1, cells);
  }

  function normalize(cells) {
    const transformations = [
      (r, c) => [r, c],
      (r, c) => [r, -c],
      (r, c) => [-r, c],
      (r, c) => [-r, -c],
      (r, c) => [c, r],
      (r, c) => [c, -r],
      (r, c) => [-c, r],
      (r, c) => [-c, -r]
    ];

    const shapes = [];

    for (const transform of transformations) {
      const transformed = cells.map(([r, c]) => transform(r, c));
      transformed.sort((a, b) => a[0] === b[0] ? a[1] - b[1] : a[0] - b[0]);

      const minR = transformed[0][0];
      const minC = transformed[0][1];
      const normalized = transformed.map(([r, c]) => [r - minR, c - minC]);

      shapes.push(normalized.map(([r, c]) => `${r},${c}`).join('|'));
    }

    return shapes.sort()[0];
  }
};
```

0712 Minimum Ascii Delete Sum For Two Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 712. Minimum ASCII Delete Sum for Two Strings
 * https://leetcode.com/problems/minimum-ascii-delete-sum-for-two-strings/
 * Difficulty: Medium
 *
 * Given two strings s1 and s2, return the lowest ASCII sum of deleted characters to
 * make two strings equal.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {number}
 */
var minimumDeleteSum = function(s1, s2) {
  const dp = new Array(s1.length + 1).fill().map(() => {
    return new Array(s2.length + 1).fill(0);
  });

  for (let i = 1; i <= s1.length; i++) {
    dp[i][0] = dp[i - 1][0] + s1.charCodeAt(i - 1);
  }
  for (let j = 1; j <= s2.length; j++) {
    dp[0][j] = dp[0][j - 1] + s2.charCodeAt(j - 1);
  }

  for (let i = 1; i <= s1.length; i++) {
    for (let j = 1; j <= s2.length; j++) {
      if (s1[i - 1] === s2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1];
      } else {
        dp[i][j] = Math.min(
          dp[i - 1][j] + s1.charCodeAt(i - 1),
          dp[i][j - 1] + s2.charCodeAt(j - 1)
        );
      }
    }
  }

  return dp[s1.length][s2.length];
};
```

0713 Subarray Product Less Than K

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 713. Subarray Product Less Than K
 * https://leetcode.com/problems/subarray-product-less-than-k/
 * Difficulty: Medium
 *
 * Your are given an array of positive integers nums.
 *
 * Count and print the number of (contiguous) subarrays where the
 * product of all the elements in the subarray is less than k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
```

```
 */
var numSubarrayProductLessThanK = function(nums, k) {
  let result = 0;
  for (let right = 0, left = 0, product = 1; right < nums.length; right++) {
    product *= nums[right];
    while (product >= k && left < nums.length) {
      product /= nums[left];
      left++;
    }
    if (left < nums.length) {
      result = result + right - left + 1;
    }
  }
  return result;
};
```

0714 Best Time To Buy And Sell Stock With Transaction Fee

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 714. Best Time to Buy and Sell Stock with Transaction Fee
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-with-transaction-fee/
 * Difficulty: Medium
 *
 * You are given an array prices where prices[i] is the price of a given stock on the ith day,
 * and an integer fee representing a transaction fee.
 *
 * Find the maximum profit you can achieve. You may complete as many transactions as you like,
 * but you need to pay the transaction fee for each transaction.
 */

/**
 * @param {number[]} prices
 * @param {number} fee
 * @return {number}
 */
var maxProfit = function(prices, fee) {
  let result = 0;
  for (let i = 1, sell = -prices[0]; i < prices.length; i++) {
    result = Math.max(result, sell + prices[i] - fee);
    sell = Math.max(sell, result - prices[i]);
  }
  return result;
};
```

0715 Range Module

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 715. Range Module
 * https://leetcode.com/problems/range-module/
 * Difficulty: Hard
 *
 * A Range Module is a module that tracks ranges of numbers. Design a data structure to track
the
 * ranges represented as half-open intervals and query about them.
 *
 * A half-open interval [left, right) denotes all the real numbers x where left <= x < right.
 *
 * Implement the RangeModule class:
 * - RangeModule() Initializes the object of the data structure.
 * - void addRange(int left, int right) Adds the half-open interval [left, right), tracking
every
```

```
 *   real number in that interval. Adding an interval that partially overlaps with currently
tracked
 *   numbers should add any numbers in the interval [left, right) that are not already
tracked.
 * - boolean queryRange(int left, int right) Returns true if every real number in the interval
 *   [left, right) is currently being tracked, and false otherwise.
 * - void removeRange(int left, int right) Stops tracking every real number currently being
tracked
 *   in the half-open interval [left, right).
 */

var RangeModule = function() {
  this.ranges = [];
};

/**
 * @param {number} left
 * @param {number} right
 * @return {void}
 */
RangeModule.prototype.addRange = function(left, right) {
  const intervals = [];
  let placed = false;
  for (const [start, end] of this.ranges) {
    if (start > right && !placed) {
      intervals.push([left, right]);
      placed = true;
    }
    if (end < left || start > right) {
      intervals.push([start, end]);
    } else {
      left = Math.min(left, start);
      right = Math.max(right, end);
    }
  }
  if (!placed) intervals.push([left, right]);
  this.ranges = intervals;
};

/**
 * @param {number} left
 * @param {number} right
 * @return {boolean}
 */
RangeModule.prototype.queryRange = function(left, right) {
  for (const [start, end] of this.ranges) {
    if (start <= left && end >= right) return true;
    if (start > left) break;
  }
  return false;
};

/**
 * @param {number} left
 * @param {number} right
 * @return {void}
 */
RangeModule.prototype.removeRange = function(left, right) {
  const intervals = [];
  for (const [start, end] of this.ranges) {
    if (end <= left || start >= right) {
      intervals.push([start, end]);
    } else {
      if (start < left) intervals.push([start, left]);
      if (end > right) intervals.push([right, end]);
    }
  }
  this.ranges = intervals;
};
```

0716 Max Stack

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 716. Max Stack
 * https://leetcode.com/problems/max-stack/
 * Difficulty: Hard
 *
 * Design a max stack data structure that supports the stack operations and supports finding
 * the stack's maximum element.
 *
 * Implement the MaxStack class:
 * - MaxStack() Initializes the stack object.
 * - void push(int x) Pushes element x onto the stack.
 * - int pop() Removes the element on top of the stack and returns it.
 * - int top() Gets the element on the top of the stack without removing it.
 * - int peekMax() Retrieves the maximum element in the stack without removing it.
 * - int popMax() Retrieves the maximum element in the stack and removes it. If there is more
 *   than one maximum element, only remove the top-most one.
 *
 * You must come up with a solution that supports O(1) for each top call and O(logn) for each
 * other call.
 */

var MaxStack = function() {
  this.stack = [];
  this.maxHeap = new PriorityQueue((a, b) => a.val === b.val ? b.id - a.id : b.val - a.val);
  this.nodeId = 0;
  this.deleted = new Set();
};

/**
* @param {number} x
* @return {void}
*/
MaxStack.prototype.push = function(x) {
  const id = this.nodeId++;
  const node = { val: x, id };
  this.stack.push(node);
  this.maxHeap.enqueue(node);
};

/**
* @return {number}
*/
MaxStack.prototype.pop = function() {
  this.cleanStack();
  const node = this.stack.pop();
  this.deleted.add(node.id);
  return node.val;
};

/**
* @return {number}
*/
MaxStack.prototype.top = function() {
  this.cleanStack();
  return this.stack[this.stack.length - 1].val;
};

/**
* @return {number}
*/
MaxStack.prototype.peekMax = function() {
  this.cleanMaxHeap();
  return this.maxHeap.front().val;
};

/**
```

```
* @return {number}
*/
MaxStack.prototype.popMax = function() {
  this.cleanMaxHeap();
  const maxNode = this.maxHeap.dequeue();
  this.deleted.add(maxNode.id);
  return maxNode.val;
};

/**
* @return {void}
*/
MaxStack.prototype.cleanStack = function() {
  while (this.stack.length && this.deleted.has(this.stack[this.stack.length - 1].id)) {
    this.stack.pop();
  }
};

/**
* @return {void}
*/
MaxStack.prototype.cleanMaxHeap = function() {
  while (this.maxHeap.size() && this.deleted.has(this.maxHeap.front().id)) {
    this.maxHeap.dequeue();
  }
};
```

0717 1 Bit And 2 Bit Characters

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 717. 1-bit and 2-bit Characters
 * https://leetcode.com/problems/1-bit-and-2-bit-characters/
 * Difficulty: Easy
 *
 * We have two special characters:
 * - The first character can be represented by one bit 0.
 * - The second character can be represented by two bits (10 or 11).
 *
 * Given a binary array bits that ends with 0, return true if the last character
 * must be a one-bit character.
 */

/**
 * @param {number[]} bits
 * @return {boolean}
 */
var isOneBitCharacter = function(bits) {
  return bits.slice(0, bits.length - 1).join('').replace(/11|10|0/g, '') !== '1';
};
```

0718 Maximum Length Of Repeated Subarray

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 718. Maximum Length of Repeated Subarray
 * https://leetcode.com/problems/maximum-length-of-repeated-subarray/
 * Difficulty: Medium
 *
 * Given two integer arrays nums1 and nums2, return the maximum length of a subarray
 * that appears in both arrays.
 */
```

```
/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var findLength = function(nums1, nums2) {
  const dp = new Array(nums1.length + 1).fill().map(() => {
    return new Array(nums2.length + 1).fill(0);
  });
  let result = 0;

  for (let i = 1; i <= nums1.length; i++) {
    for (let j = 1; j <= nums2.length; j++) {
      if (nums1[i - 1] === nums2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
        result = Math.max(result, dp[i][j]);
      }
    }
  }

  return result;
};
```

0719 Find K Th Smallest Pair Distance

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 719. Find K-th Smallest Pair Distance
 * https://leetcode.com/problems/find-k-th-smallest-pair-distance/
 * Difficulty: Hard
 *
 * The distance of a pair of integers a and b is defined as the absolute difference between a
and b.
 *
 * Given an integer array nums and an integer k, return the kth smallest distance among all
the
 * pairs nums[i] and nums[j] where 0 <= i < j < nums.length.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var smallestDistancePair = function(nums, k) {
  nums.sort((a, b) => a - b);
  let left = 0;
  let right = nums[nums.length - 1] - nums[0];

  while (left < right) {
    const middle = Math.floor((left + right) / 2);
    let count = 0;
    let j = 0;

    for (let i = 0; i < nums.length; i++) {
      while (j < nums.length && nums[j] - nums[i] <= middle) j++;
      count += j - i - 1;
    }

    if (count < k) {
      left = middle + 1;
    } else {
      right = middle;
    }
  }

  return left;
};
```

0720 Longest Word In Dictionary

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 720. Longest Word in Dictionary
 * https://leetcode.com/problems/longest-word-in-dictionary/
 * Difficulty: Medium
 *
 * Given an array of strings words representing an English Dictionary, return the longest word
 * in words that can be built one character at a time by other words in words.
 *
 * If there is more than one possible answer, return the longest word with the smallest
 * lexicographical order. If there is no answer, return the empty string.
 *
 * Note that the word should be built from left to right with each additional character being
 * added to the end of a previous word.
 */

/**
 * @param {string[]} words
 * @return {string}
 */
var longestWord = function(words) {
  const set = new Set();
  let result = '';

  words.sort().forEach(word => {
    if (word.length === 1 || set.has(word.slice(0, -1))) {
      set.add(word);
      result = word.length > result.length ? word : result;
    }
  });

  return result;
};
```

0721 Accounts Merge

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 721. Accounts Merge
 * https://leetcode.com/problems/accounts-merge/
 * Difficulty: Medium
 *
 * Given a list of accounts where each element accounts[i] is a list of strings, where the
first
 * element accounts[i][0] is a name, and the rest of the elements are emails representing
emails
 * of the account.
 *
 * Now, we would like to merge these accounts. Two accounts definitely belong to the same
person
 * if there is some common email to both accounts. Note that even if two accounts have the
same
 * name, they may belong to different people as people could have the same name. A person can
 * have any number of accounts initially, but all of their accounts definitely have the same
name.
 *
 * After merging the accounts, return the accounts in the following format: the first element
of
 * each account is the name, and the rest of the elements are emails in sorted order. The
accounts
 * themselves can be returned in any order.
```

```
 */

/**
 * @param {string[][]} accounts
 * @return {string[][]}
 */
var accountsMerge = function(accounts) {
  const parent = new Map();
  const names = new Map();
  const groups = new Map();

  const find = node => {
    if (!parent.has(node)) parent.set(node, node);
    return parent.get(node) === node ? node : parent.set(node,
find(parent.get(node))).get(node);
  };

  for (const [name, ...emails] of accounts) {
    for (const email of emails) {
      names.set(email, name);
      find(email);
      parent.set(find(email), find(emails[0]));
    }
  }

  for (const email of parent.keys()) {
    const root = find(email);
    if (!groups.has(root)) groups.set(root, new Set());
    groups.get(root).add(email);
  }

  return Array.from(groups, ([root, emails]) => [names.get(root), ...[...emails].sort()]);
};
```

0722 Remove Comments

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 722. Remove Comments
 * https://leetcode.com/problems/remove-comments/
 * Difficulty: Medium
 *
 * Given a C++ program, remove comments from it. The program source is an array where
source[i] is the i-th
 * line of the source code. This represents the result of splitting the original source code
string by the
 * newline character \n.
 * In C++, there are two types of comments, line comments, and block comments.
 * The string // denotes a line comment, which represents that it and rest of the characters
to the right
 * of it in the same line should be ignored.
 * The string /* denotes a block comment, which represents that all characters until the next
 * (non-overlapping) occurrence of should be ignored.
 * It is guaranteed that every open block comment will eventually be closed, so /* outside of
a line or block
 * comment always starts a new comment.
 * Finally, implicit newline characters can be deleted by block comments.
 * After removing the comments from the source code, return the source code in the same
format.
 */

/**
 * @param {string[]} source
 * @return {string[]}
 */
var removeComments = function(source) {
  return source.join('\n').replace(/\/\*.*?\*\/|\/\/[^\n]+/gs,
'').split(/\n/).filter(Boolean);
};
```

0723 Candy Crush

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 723. Candy Crush
 * https://leetcode.com/problems/candy-crush/
 * Difficulty: Medium
 *
 * This question is about implementing a basic elimination algorithm for Candy Crush.
 *
 * Given an m x n integer array board representing the grid of candy where board[i][j]
represents
 * the type of candy. A value of board[i][j] == 0 represents that the cell is empty.
 *
 * The given board represents the state of the game following the player's move. Now, you need
 to
 * restore the board to a stable state by crushing candies according to the following rules:
 * - If three or more candies of the same type are adjacent vertically or horizontally, crush
them
 *   all at the same time - these positions become empty.
 * - After crushing all candies simultaneously, if an empty space on the board has candies on
top
 *   of itself, then these candies will drop until they hit a candy or bottom at the same
time.
 *   No new candies will drop outside the top boundary.
 * - After the above steps, there may exist more candies that can be crushed. If so, you need
to
 *   repeat the above steps.
 * - If there does not exist more candies that can be crushed (i.e., the board is stable),
then
 *   return the current board.
 *
 * You need to perform the above rules until the board becomes stable, then return the stable
board.
 */

/**
 * @param {number[][]} board
 * @return {number[][]}
 */
var candyCrush = function(board) {
  const rows = board.length;
  const cols = board[0].length;

  while (markCrushables()) {
    dropCandies();
  }

  return board;

  function markCrushables() {
    let hasCrushables = false;

    for (let row = 0; row < rows; row++) {
      for (let col = 0; col < cols - 2; col++) {
        const candy = Math.abs(board[row][col]);
        if (candy !== 0 && candy === Math.abs(board[row][col + 1])
            && candy === Math.abs(board[row][col + 2])) {
          hasCrushables = true;
          board[row][col] = -candy;
          board[row][col + 1] = -candy;
          board[row][col + 2] = -candy;
        }
      }
    }

    for (let col = 0; col < cols; col++) {
      for (let row = 0; row < rows - 2; row++) {
        const candy = Math.abs(board[row][col]);
```

```
        if (candy !== 0 && candy === Math.abs(board[row + 1][col])
            && candy === Math.abs(board[row + 2][col])) {
          hasCrushables = true;
          board[row][col] = -candy;
          board[row + 1][col] = -candy;
          board[row + 2][col] = -candy;
        }
      }
    }

    return hasCrushables;
  }

  function dropCandies() {
    for (let col = 0; col < cols; col++) {
      let bottom = rows - 1;
      for (let row = rows - 1; row >= 0; row--) {
        if (board[row][col] > 0) {
          board[bottom][col] = board[row][col];
          if (bottom !== row) {
            board[row][col] = 0;
          }
          bottom--;
        } else if (board[row][col] < 0) {
          board[row][col] = 0;
        }
      }
    }
  }
};
```

0724 Find Pivot Index

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 724. Find Pivot Index
 * https://leetcode.com/problems/find-pivot-index/
 * Difficulty: Easy
 *
 * Given an array of integers nums, calculate the pivot index of this array.
 *
 * The pivot index is the index where the sum of all the numbers strictly to the left
 * of the index is equal to the sum of all the numbers strictly to the index's right.
 *
 * If the index is on the left edge of the array, then the left sum is 0 because there
 * are no elements to the left. This also applies to the right edge of the array.
 *
 * Return the leftmost pivot index. If no such index exists, return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var pivotIndex = function(nums) {
  let left = 0;
  let right = nums.reduce((sum, n) => sum + n, 0);

  for (let i = 0; i < nums.length; i++) {
    left += nums[i];
    right -= nums[i];
    if (left - nums[i] === right) {
      return i;
    }
  }

  return -1;
};
```

0725 Split Linked List In Parts

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 725. Split Linked List in Parts
 * https://leetcode.com/problems/split-linked-list-in-parts/
 * Difficulty: Medium
 *
 * Given the head of a singly linked list and an integer k, split the linked list into k
 * consecutive linked list parts.
 *
 * The length of each part should be as equal as possible: no two parts should have a size
 * differing by more than one. This may lead to some parts being null.
 *
 * The parts should be in the order of occurrence in the input list, and parts occurring
 * earlier should always have a size greater than or equal to parts occurring later.
 *
 * Return an array of the k parts.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} k
 * @return {ListNode[]}
 */
var splitListToParts = function(head, k) {
  let count = 0;
  let current = head;

  while (current) {
    count++;
    current = current.next;
  }

  const base = Math.floor(count / k);
  const extra = count % k;
  const result = new Array(k).fill(null);

  current = head;
  for (let i = 0; i < k && current; i++) {
    result[i] = current;
    const partSize = base + (i < extra ? 1 : 0);
    for (let j = 1; j < partSize; j++) {
      current = current.next;
    }
    const next = current.next;
    current.next = null;
    current = next;
  }

  return result;
};
```

0726 Number Of Atoms

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 726. Number of Atoms
 * https://leetcode.com/problems/number-of-atoms/
 * Difficulty: Hard
 *
 * Given a string formula representing a chemical formula, return the count of each atom.
 *
 * The atomic element always starts with an uppercase character, then zero or more lowercase
 * letters, representing the name.
 *
 * One or more digits representing that element's count may follow if the count is greater
 * than 1. If the count is 1, no digits will follow.
 * - For example, "H2O" and "H2O2" are possible, but "H1O2" is impossible.
 *
 * Two formulas are concatenated together to produce another formula.
 * - For example, "H2O2He3Mg4" is also a formula.
 *
 * A formula placed in parentheses, and a count (optionally added) is also a formula.
 * - For example, "(H2O2)" and "(H2O2)3" are formulas.
 *
 * Return the count of all elements as a string in the following form: the first name (in
sorted
 * order), followed by its count (if that count is more than 1), followed by the second name
 * (in sorted order), followed by its count (if that count is more than 1), and so on.
 *
 * The test cases are generated so that all the values in the output fit in a 32-bit integer.
 */

/**
 * @param {string} formula
 * @return {string}
 */
var countOfAtoms = function(formula) {
  const [counts] = parseFormula(formula, 0);
  return [...counts].sort().map(([atom, count]) => atom + (count > 1 ? count : '')).join('');

  function parseFormula(str, index) {
    const atomCounts = new Map();
    while (index < str.length && str[index] !== ')') {
      if (str[index] === '(') {
        const [subCounts, nextIndex] = parseFormula(str, index + 1);
        const [multiplier, updatedIndex] = extractNumber(str, nextIndex + 1);
        for (const [atom, count] of subCounts) {
          atomCounts.set(atom, (atomCounts.get(atom) || 0) + count * (multiplier || 1));
        }
        index = updatedIndex;
      } else {
        const [atom, nextIndex] = extractAtom(str, index);
        const [count, updatedIndex] = extractNumber(str, nextIndex);
        atomCounts.set(atom, (atomCounts.get(atom) || 0) + (count || 1));
        index = updatedIndex;
      }
    }
    return [atomCounts, index];
  }

  function extractAtom(str, index) {
    let atom = str[index];
    while (++index < str.length && /[a-z]/.test(str[index])) atom += str[index];
    return [atom, index];
  }

  function extractNumber(str, index) {
    let number = 0;
    while (index < str.length && /[0-9]/.test(str[index])) {
      number = number * 10 + Number(str[index++]);
    }
    return [number || null, index];
  }
};
```

0727 Minimum Window Subsequence

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 727. Minimum Window Subsequence
 * https://leetcode.com/problems/minimum-window-subsequence/
 * Difficulty: Hard
 *
 * Given strings s1 and s2, return the minimum contiguous substring part of s1, so that
 * s2 is a subsequence of the part.
 *
 * If there is no such window in s1 that covers all characters in s2, return the empty
 * string "". If there are multiple such minimum-length windows, return the one with the
 * left-most starting index.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {string}
 */
var minWindow = function(s1, s2) {
  const n = s1.length;
  const m = s2.length;
  const dp = new Array(m + 1).fill().map(() => new Array(n + 1).fill(Infinity));
  dp[0][0] = 0;

  for (let j = 1; j <= n; j++) {
    dp[0][j] = 0;
    for (let i = 1; i <= m; i++) {
      if (s1[j - 1] === s2[i - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = dp[i][j - 1] !== Infinity ? dp[i][j - 1] + 1 : Infinity;
      }
    }
  }

  let minLength = Infinity;
  let endIndex = -1;
  for (let j = 1; j <= n; j++) {
    if (dp[m][j] < minLength) {
      minLength = dp[m][j];
      endIndex = j;
    }
  }

  return minLength === Infinity ? '' : s1.slice(endIndex - minLength, endIndex);
};
```

0728 Self Dividing Numbers

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 728. Self Dividing Numbers
 * https://leetcode.com/problems/self-dividing-numbers/
 * Difficulty: Easy
 *
 * A self-dividing number is a number that is divisible by every digit it contains.
 * - For example, 128 is a self-dividing number because 128 % 1 == 0, 128 % 2 == 0,
 *   and 128 % 8 == 0.
 *
 * A self-dividing number is not allowed to contain the digit zero.
```

```
 *
 * Given two integers left and right, return a list of all the self-dividing numbers
 * in the range [left, right] (both inclusive).
 */

/**
 * @param {number} left
 * @param {number} right
 * @return {number[]}
 */
var selfDividingNumbers = function(left, right) {
  return Array.from({ length: right - left + 1 }, (_, i) => left + i)
    .filter(n => String(n).split('').every(d => d !== '0' && n % d === 0));
};
```

0729 My Calendar I

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 729. My Calendar I
 * https://leetcode.com/problems/my-calendar-i/
 * Difficulty: Medium
 *
 * You are implementing a program to use as your calendar. We can add a new event if adding
the
 * event will not cause a double booking.
 *
 * A double booking happens when two events have some non-empty intersection (i.e., some
moment
 * is common to both events.).
 *
 * The event can be represented as a pair of integers startTime and endTime that represents a
 * booking on the half-open interval [startTime, endTime), the range of real numbers x such
 * that startTime <= x < endTime.
 *
 * Implement the MyCalendar class:
 * - MyCalendar() Initializes the calendar object.
 * - boolean book(int startTime, int endTime) Returns true if the event can be added to the
 *   calendar successfully without causing a double booking. Otherwise, return false and do
 *   not add the event to the calendar.
 */

class MyCalendar {
  constructor() {
    this.events = [];
  }

  /**
   * @param {number} startTime
   * @param {number} endTime
   * @returns {boolean}
   */
  book(startTime, endTime) {
    const event = [startTime, endTime];
    const index = this.findInsertIndex(startTime);

    if (this.overlapsPrevious(index - 1, startTime) || this.overlapsNext(index, endTime)) {
      return false;
    }

    this.events.splice(index, 0, event);
    return true;
  }

  /**
   * @param {number} startTime
   * @returns {number}
   */
  findInsertIndex(startTime) {
```

```
    let left = 0;
    let right = this.events.length;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (this.events[mid][0] > startTime) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }

    return left;
  }

  /**
   * @param {number} prevIndex
   * @param {number} startTime
   * @returns {boolean}
   */
  overlapsPrevious(prevIndex, startTime) {
    return prevIndex >= 0 && this.events[prevIndex][1] > startTime;
  }

  /**
   * @param {number} nextIndex
   * @param {number} endTime
   * @returns {boolean}
   */
  overlapsNext(nextIndex, endTime) {
    return nextIndex < this.events.length && this.events[nextIndex][0] < endTime;
  }
}
```

0730 Count Different Palindromic Subsequences

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 730. Count Different Palindromic Subsequences
 * https://leetcode.com/problems/count-different-palindromic-subsequences/
 * Difficulty: Hard
 *
 * Given a string s, return the number of different non-empty palindromic subsequences in s.
 * Since the answer may be very large, return it modulo 109 + 7.
 *
 * A subsequence of a string is obtained by deleting zero or more characters from the string.
 *
 * A sequence is palindromic if it is equal to the sequence reversed.
 *
 * Two sequences a1, a2, ... and b1, b2, ... are different if there is some i for which ai !=
bi.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countPalindromicSubsequences = function(s) {
  const MOD = 1e9 + 7;
  const dp = new Array(s.length).fill().map(() => {
    return new Array(s.length).fill(0);
  });

  for (let i = 0; i < s.length; i++) {
    dp[i][i] = 1;
  }

  for (let len = 2; len <= s.length; len++) {
    for (let start = 0; start + len <= s.length; start++) {
```

```
      const end = start + len - 1;
      const charStart = s[start];
      const charEnd = s[end];

      if (charStart !== charEnd) {
        dp[start][end] = (dp[start + 1][end] + dp[start][end - 1] - dp[start + 1][end - 1]) %
MOD;
      } else {
        let left = start + 1;
        let right = end - 1;

        while (left <= right && s[left] !== charStart) left++;
        while (left <= right && s[right] !== charStart) right--;

        if (left > right) {
          dp[start][end] = (2 * dp[start + 1][end - 1] + 2) % MOD;
        } else if (left === right) {
          dp[start][end] = (2 * dp[start + 1][end - 1] + 1) % MOD;
        } else {
          dp[start][end] = (2 * dp[start + 1][end - 1] - dp[left + 1][right - 1]) % MOD;
        }
      }

      if (dp[start][end] < 0) dp[start][end] += MOD;
    }
  }

  return dp[0][s.length - 1];
};
```

0731 My Calendar Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 731. My Calendar II
 * https://leetcode.com/problems/my-calendar-ii/
 * Difficulty: Medium
 *
 * You are implementing a program to use as your calendar. We can add a new event if adding
the
 * event will not cause a triple booking.
 *
 * A triple booking happens when three events have some non-empty intersection (i.e., some
moment
 * is common to all the three events.).
 *
 * The event can be represented as a pair of integers startTime and endTime that represents a
 * booking on the half-open interval [startTime, endTime), the range of real numbers x such
that
 * startTime <= x < endTime.
 *
 * Implement the MyCalendarTwo class:
 * - MyCalendarTwo() Initializes the calendar object.
 * - boolean book(int startTime, int endTime) Returns true if the event can be added to the
calendar
 *   successfully without causing a triple booking. Otherwise, return false and do not add the
 event
 *   to the calendar.
 */

var MyCalendarTwo = function() {
  this.bookings = [];
  this.overlaps = [];
};

/**
 * @param {number} startTime
 * @param {number} endTime
 * @return {boolean}
```

```
 */
MyCalendarTwo.prototype.book = function(startTime, endTime) {
  for (const [start, end] of this.overlaps) {
    if (startTime < end && endTime > start) return false;
  }

  for (const [start, end] of this.bookings) {
    if (startTime < end && endTime > start) {
      this.overlaps.push([Math.max(start, startTime), Math.min(end, endTime)]);
    }
  }

  this.bookings.push([startTime, endTime]);
  return true;
};
```

0732 My Calendar Iii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 732. My Calendar III
 * https://leetcode.com/problems/my-calendar-iii/
 * Difficulty: Hard
 *
 * A k-booking happens when k events have some non-empty intersection (i.e., there is some
time
 * that is common to all k events.)
 *
 * You are given some events [startTime, endTime), after each given event, return an integer k
 * representing the maximum k-booking between all the previous events.
 *
 * Implement the MyCalendarThree class:
 * - MyCalendarThree() Initializes the object.
 * - int book(int startTime, int endTime) Returns an integer k representing the largest
integer
 *   such that there exists a k-booking in the calendar.
 */

class MyCalendarThree {
  constructor() {
    this.events = [];
  }

  /**
   * @param {number} startTime
   * @param {number} endTime
   * @returns {number}
   */
  book(startTime, endTime) {
    this.insertTime(startTime, 1);
    this.insertTime(endTime, -1);
    let maxBookings = 1;
    let currentBookings = 0;
    this.events.forEach(event => maxBookings = Math.max(currentBookings += event[1],
maxBookings));
    return maxBookings;
  }

  /**
   * @param {number} targetTime
   * @param {number} value
   */
  insertTime(targetTime, value) {
    const events = this.events;
    let left = 0;
    let right = events.length - 1;
    while (left <= right) {
      const mid = left + right >> 1;
      if (events[mid][0] < targetTime) left = mid + 1;
```

```
      else if (events[mid][0] > targetTime) right = mid - 1;
      else return events[mid][1] += value;
    }
    events.splice(left, 0, [targetTime, value]);
  }
}
```

0733 Flood Fill

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 733. Flood Fill
 * https://leetcode.com/problems/flood-fill/
 * Difficulty: Easy
 *
 * An image is represented by an m x n integer grid image where image[i][j] represents
 * the pixel value of the image.
 *
 * You are also given three integers sr, sc, and newColor. You should perform a flood
 * fill on the image starting from the pixel image[sr][sc].
 *
 * To perform a flood fill, consider the starting pixel, plus any pixels connected
 * 4-directionally to the starting pixel of the same color as the starting pixel,
 * plus any pixels connected 4-directionally to those pixels (also with the same color),
 * and so on. Replace the color of all of the aforementioned pixels with newColor.
 *
 * Return the modified image after performing the flood fill.
 */

/**
 * @param {number[][]} image
 * @param {number} sr
 * @param {number} sc
 * @param {number} newColor
 * @return {number[][]}
 */
var floodFill = function(image, sr, sc, newColor) {
  fill(image, sr, sc, image[sr][sc], newColor);
  return image;
};

function fill(image, x, y, initialColor, newColor) {
  if (image[x] && image[x][y] === initialColor && initialColor !== newColor) {
    image[x][y] = newColor;
    [[x - 1, y], [x + 1, y], [x, y - 1], [x, y + 1]]
      .forEach(([x, y]) => fill(image, x, y, initialColor, newColor));
  }
}
```

0734 Sentence Similarity

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 734. Sentence Similarity
 * https://leetcode.com/problems/sentence-similarity/
 * Difficulty: Easy
 *
 * We can represent a sentence as an array of words, for example, the sentence
 * "I am happy with leetcode" can be represented as arr = ["I","am",happy","with","leetcode"].
 *
 * Given two sentences sentence1 and sentence2 each represented as a string array and given an
 * array of string pairs similarPairs where similarPairs[i] = [xi, yi] indicates that the two
 * words xi and yi are similar.
```

```
 *
 * Return true if sentence1 and sentence2 are similar, or false if they are not similar.
 *
 * Two sentences are similar if:
 * - They have the same length (i.e., the same number of words)
 * - sentence1[i] and sentence2[i] are similar.
 *
 * Notice that a word is always similar to itself, also notice that the similarity relation is
 * not transitive. For example, if the words a and b are similar, and the words b and c are
 * similar, a and c are not necessarily similar.
 */

/**
 * @param {string[]} sentence1
 * @param {string[]} sentence2
 * @param {string[][]} similarPairs
 * @return {boolean}
 */
var areSentencesSimilar = function(sentence1, sentence2, similarPairs) {
  if (sentence1.length !== sentence2.length) {
    return false;
  }

  const similarWords = new Set();
  for (const [word1, word2] of similarPairs) {
    similarWords.add(`${word1}:${word2}`);
    similarWords.add(`${word2}:${word1}`);
  }

  for (let i = 0; i < sentence1.length; i++) {
    const word1 = sentence1[i];
    const word2 = sentence2[i];
    if (word1 !== word2 && !similarWords.has(`${word1}:${word2}`)) {
      return false;
    }
  }

  return true;
};
```

0735 Asteroid Collision

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 735. Asteroid Collision
 * https://leetcode.com/problems/asteroid-collision/
 * Difficulty: Medium
 *
 * We are given an array asteroids of integers representing asteroids in a row. The indices
 * of the asteriod in the array represent their relative position in space.
 *
 * For each asteroid, the absolute value represents its size, and the sign represents its
 * direction (positive meaning right, negative meaning left). Each asteroid moves at the
 * same speed.
 *
 * Find out the state of the asteroids after all collisions. If two asteroids meet, the
 * smaller one will explode. If both are the same size, both will explode. Two asteroids
 * moving in the same direction will never meet.
 */

/**
 * @param {number[]} asteroids
 * @return {number[]}
 */
var asteroidCollision = function(asteroids) {
  const result = [];

  for (let i = 0; i < asteroids.length; i++) {
    const previous = result[result.length - 1];
```

```
    const current = asteroids[i];

    if (!result.length || previous < 0 || current > 0) {
      result.push(current);
    } else if (-current === previous) {
      result.pop();
    } else if (-current > previous) {
      result.pop();
      i--;
    }
  }

  return result;
};
```

0736 Parse Lisp Expression

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 736. Parse Lisp Expression
 * https://leetcode.com/problems/parse-lisp-expression/
 * Difficulty: Hard
 *
 * You are given a string expression representing a Lisp-like expression to return the integer
 * value of.
 *
 * The syntax for these expressions is given as follows.
 * - An expression is either an integer, let expression, add expression, mult expression, or
an
 *   assigned variable. Expressions always evaluate to a single integer.
 * - (An integer could be positive or negative.)
 * - A let expression takes the form "(let v1 e1 v2 e2 ... vn en expr)", where let is always
the
 *   string "let", then there are one or more pairs of alternating variables and expressions,
 *   meaning that the first variable v1 is assigned the value of the expression e1, the second
 *   variable v2 is assigned the value of the expression e2, and so on sequentially; and then
the
 *   value of this let expression is the value of the expression expr.
 * - An add expression takes the form "(add e1 e2)" where add is always the string "add",
there are
 *   always two expressions e1, e2 and the result is the addition of the evaluation of e1 and
the
 *   evaluation of e2.
 * - A mult expression takes the form "(mult e1 e2)" where mult is always the string "mult",
there
 *   are always two expressions e1, e2 and the result is the multiplication of the evaluation
of
 *   e1 and the evaluation of e2.
 * - For this question, we will use a smaller subset of variable names. A variable starts with
 a
 *   lowercase letter, then zero or more lowercase letters or digits. Additionally, for your
 *   convenience, the names "add", "let", and "mult" are protected and will never be used as
 *   variable names.
 * - Finally, there is the concept of scope. When an expression of a variable name is
evaluated,
 *   within the context of that evaluation, the innermost scope (in terms of parentheses) is
 *   checked first for the value of that variable, and then outer scopes are checked
sequentially.
 *   It is guaranteed that every expression is legal. Please see the examples for more details
 *   on the scope.
 */

/**
 * @param {string} expression
 * @return {number}
 */
var evaluate = function(expression) {
  return parse(expression, new Map());
```

```
  function parse(exp, scope) {
    if (!isNaN(exp)) return parseInt(exp);
    if (!exp.startsWith('(')) return scope.get(exp) || 0;

    const tokens = tokenize(exp.slice(1, -1));
    if (tokens[0] === 'add') return parse(tokens[1], scope) + parse(tokens[2], scope);
    if (tokens[0] === 'mult') return parse(tokens[1], scope) * parse(tokens[2], scope);

    const newScope = new Map(scope);
    for (let i = 1; i < tokens.length - 1; i += 2) {
      newScope.set(tokens[i], parse(tokens[i + 1], newScope));
    }
    return parse(tokens[tokens.length - 1], newScope);
  }

  function tokenize(exp) {
    const result = [];
    let current = '';
    let depth = 0;

    for (const char of exp) {
      if (char === ' ' && depth === 0) {
        result.push(current);
        current = '';
      } else {
        current += char;
        if (char === '(') depth++;
        if (char === ')') depth--;
      }
    }
    result.push(current);
    return result;
  }
};
```

0737 Sentence Similarity Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 737. Sentence Similarity II
 * https://leetcode.com/problems/sentence-similarity-ii/
 * Difficulty: Medium
 *
 * We can represent a sentence as an array of words, for example, the sentence
 * "I am happy with leetcode" can be represented as arr = ["I","am",happy","with","leetcode"].
 *
 * Given two sentences sentence1 and sentence2 each represented as a string array and given an
 * array of string pairs similarPairs where similarPairs[i] = [xi, yi] indicates that the two
 * words xi and yi are similar.
 *
 * Return true if sentence1 and sentence2 are similar, or false if they are not similar.
 *
 * Two sentences are similar if:
 * - They have the same length (i.e., the same number of words)
 * - sentence1[i] and sentence2[i] are similar.
 *
 * Notice that a word is always similar to itself, also notice that the similarity relation is
 * transitive. For example, if the words a and b are similar, and the words b and c are
similar,
 * then a and c are similar.
 */

/**
 * @param {string[]} sentence1
 * @param {string[]} sentence2
 * @param {string[][]} similarPairs
 * @return {boolean}
 */
var areSentencesSimilarTwo = function(sentence1, sentence2, similarPairs) {
```

```
  if (sentence1.length !== sentence2.length) {
    return false;
  }

  const graph = new Map();
  for (const [word1, word2] of similarPairs) {
    if (!graph.has(word1)) graph.set(word1, new Set());
    if (!graph.has(word2)) graph.set(word2, new Set());
    graph.get(word1).add(word2);
    graph.get(word2).add(word1);
  }

  for (let i = 0; i < sentence1.length; i++) {
    if (!areSimilar(sentence1[i], sentence2[i])) {
      return false;
    }
  }

  return true;

  function areSimilar(start, target) {
    if (start === target) return true;
    const visited = new Set();
    const queue = [start];
    visited.add(start);

    while (queue.length) {
      const word = queue.shift();
      for (const neighbor of graph.get(word) || []) {
        if (neighbor === target) return true;
        if (!visited.has(neighbor)) {
          visited.add(neighbor);
          queue.push(neighbor);
        }
      }
    }
    return false;
  }
};
```

0738 Monotone Increasing Digits

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 738. Monotone Increasing Digits
 * https://leetcode.com/problems/monotone-increasing-digits/
 * Difficulty: Medium
 *
 * An integer has monotone increasing digits if and only if each pair of adjacent digits
 * x and y satisfy x <= y.
 *
 * Given an integer n, return the largest number that is less than or equal to n with
 * monotone increasing digits.
 */

/**
 * @param {number} n
 * @return {number}
 */
var monotoneIncreasingDigits = function(n) {
  const digits = String(n).split('').map(Number);
  let offset = digits.length;

  for (let i = digits.length - 1; i > 0; i--) {
    if (digits[i - 1] > digits[i]) {
      offset = i;
      digits[i - 1]--;
    }
  }
```

```
  for (let i = offset; i < digits.length; i++) {
    digits[i] = 9;
  }

  return +digits.join('');
};
```

0739 Daily Temperatures

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 739. Daily Temperatures
 * https://leetcode.com/problems/daily-temperatures/
 * Difficulty: Medium
 *
 * Given a list of daily temperatures T, return a list such that, for
 * each day in the input, tells you how many days you would have to
 * wait until a warmer temperature. If there is no future day for
 * which this is possible, put 0 instead.
 *
 * For example, given the list of temperatures
 * T = [73, 74, 75, 71, 69, 72, 76, 73], your output
 * should be [1, 1, 4, 2, 1, 1, 0, 0].
 *
 * Note: The length of temperatures will be in the range [1, 30000].
 * Each temperature will be an integer in the range [30, 100].
 */

/**
 * @param {number[]} T
 * @return {number[]}
 */
var dailyTemperatures = function(T) {
  const result = new Array(T.length).fill(0);
  const stack = [];

  for (let i = 0; i < T.length; i++) {
    while (stack.length && T[stack[stack.length - 1]] < T[i]) {
      const topIndex = stack.pop();
      result[topIndex] = i - topIndex;
    }
    stack.push(i);
  }

  return result;
};
```

0740 Delete And Earn

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 740. Delete and Earn
 * https://leetcode.com/problems/delete-and-earn/
 * Difficulty: Medium
 *
 * You are given an integer array nums. You want to maximize the number of points you get by
 * performing the following operation any number of times:
 * - Pick any nums[i] and delete it to earn nums[i] points. Afterwards, you must delete every
 *   element equal to nums[i] - 1 and every element equal to nums[i] + 1.
 *
 * Return the maximum number of points you can earn by applying the above operation some
number
```

```
 * of times.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var deleteAndEarn = function(nums) {
  const points = new Array(10001).fill(0);
  let previous = 0;
  let result = 0;

  for (const num of nums) {
    points[num] += num;
  }

  for (const value of points) {
    const temp = result;
    result = Math.max(result, previous + value);
    previous = temp;
  }

  return result;
};
```

0741 Cherry Pickup

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 741. Cherry Pickup
 * https://leetcode.com/problems/cherry-pickup/
 * Difficulty: Hard
 *
 * You are given an n x n grid representing a field of cherries, each cell is one of three
 * possible integers.
 * - 0 means the cell is empty, so you can pass through,
 * - 1 means the cell contains a cherry that you can pick up and pass through, or
 * - -1 means the cell contains a thorn that blocks your way.
 *
 * Return the maximum number of cherries you can collect by following the rules below:
 * - Starting at the position (0, 0) and reaching (n - 1, n - 1) by moving right or down
through
 *   valid path cells (cells with value 0 or 1).
 * - After reaching (n - 1, n - 1), returning to (0, 0) by moving left or up through valid
path
 *   cells.
 * - When passing through a path cell containing a cherry, you pick it up, and the cell
becomes
 *   an empty cell 0.
 * - If there is no valid path between (0, 0) and (n - 1, n - 1), then no cherries can be
collected.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var cherryPickup = function(grid) {
  const n = grid.length;
  const dp = Array.from({ length: n }, () =>
    Array.from({ length: n }, () =>
      new Array(2 * n - 1).fill(-1))
  );

  if (grid[0][0] === -1) return 0;
  dp[0][0][0] = grid[0][0];

  for (let t = 1; t <= 2 * n - 2; t++) {
    for (let x1 = Math.max(0, t - n + 1); x1 <= Math.min(n - 1, t); x1++) {
```

```
      for (let x2 = Math.max(0, t - n + 1); x2 <= Math.min(n - 1, t); x2++) {
        const y1 = t - x1;
        const y2 = t - x2;

        if (grid[x1][y1] === -1 || grid[x2][y2] === -1) continue;

        const cherries = grid[x1][y1] + (x1 === x2 ? 0 : grid[x2][y2]);
        let maxPrev = -1;

        for (const px1 of [x1 - 1, x1]) {
          for (const px2 of [x2 - 1, x2]) {
            if (px1 >= 0 && px2 >= 0 && dp[px1][px2][t - 1] >= 0) {
              maxPrev = Math.max(maxPrev, dp[px1][px2][t - 1]);
            }
          }
        }

        if (maxPrev >= 0) {
          dp[x1][x2][t] = maxPrev + cherries;
        }
      }
    }
  }

  return dp[n - 1][n - 1][2 * n - 2] < 0 ? 0 : dp[n - 1][n - 1][2 * n - 2];
};
```

0742 Closest Leaf In A Binary Tree

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 742. Closest Leaf in a Binary Tree
 * https://leetcode.com/problems/closest-leaf-in-a-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree where every node has a unique value and a target integer k,
 * return the value of the nearest leaf node to the target k in the tree.
 *
 * Nearest to a leaf means the least number of edges traveled on the binary tree to reach any
 * leaf of the tree. Also, a node is called a leaf if it has no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} k
 * @return {number}
 */
var findClosestLeaf = function(root, k) {
  const graph = new Map();
  const leaves = new Set();

  buildGraph(root, null);

  const queue = [k];
  const visited = new Set([k]);

  while (queue.length) {
    const current = queue.shift();
    if (leaves.has(current)) return current;

    for (const neighbor of graph.get(current) || []) {
```

```
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
  }

  return -1;

  function buildGraph(node, parent) {
    if (!node) return;
    if (!node.left && !node.right) leaves.add(node.val);
    if (parent) {
      if (!graph.has(node.val)) graph.set(node.val, new Set());
      if (!graph.has(parent.val)) graph.set(parent.val, new Set());
      graph.get(node.val).add(parent.val);
      graph.get(parent.val).add(node.val);
    }
    buildGraph(node.left, node);
    buildGraph(node.right, node);
  }
};
```

0743 Network Delay Time

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 743. Network Delay Time
 * https://leetcode.com/problems/network-delay-time/
 * Difficulty: Medium
 *
 * You are given a network of n nodes, labeled from 1 to n. You are also given times, a
 * list of travel times as directed edges times[i] = (ui, vi, wi), where ui is the source
 * node, vi is the target node, and wi is the time it takes for a signal to travel from
 * source to target.
 *
 * We will send a signal from a given node k. Return the minimum time it takes for all
 * the n nodes to receive the signal. If it is impossible for all the n nodes to receive
 * the signal, return -1.
 */

/**
 * @param {number[][]} times
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var networkDelayTime = function(times, n, k) {
  const time = new Array(n + 1).fill(Infinity);

  time[k] = 0;

  for (let i = 0; i < n; i++) {
    for (const [u, v, w] of times) {
      if (time[u] === Infinity) continue;
      if (time[v] > time[u] + w) {
        time[v] = time[u] + w;
      }
    }
  }

  let result = 0;
  for (let i = 1; i <= n; i++) {
    result = Math.max(result, time[i]);
  }

  return result === Infinity ? -1 : result;
};
```

0744 Find Smallest Letter Greater Than Target

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 744. Find Smallest Letter Greater Than Target
 * https://leetcode.com/problems/find-smallest-letter-greater-than-target/
 * Difficulty: Easy
 *
 * You are given an array of characters letters that is sorted in non-decreasing order,
 * and a character target. There are at least two different characters in letters.
 *
 * Return the smallest character in letters that is lexicographically greater than target.
 * If such a character does not exist, return the first character in letters.
 */

/**
 * @param {character[]} letters
 * @param {character} target
 * @return {character}
 */
var nextGreatestLetter = function(letters, target) {
  return letters.find(l => l > target) || letters[0];
};
```

0745 Prefix And Suffix Search

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 745. Prefix and Suffix Search
 * https://leetcode.com/problems/prefix-and-suffix-search/
 * Difficulty: Hard
 *
 * Design a special dictionary that searches the words in it by a prefix and a suffix.
 *
 * Implement the WordFilter class:
 * - WordFilter(string[] words) Initializes the object with the words in the dictionary.
 * - f(string pref, string suff) Returns the index of the word in the dictionary, which
 *   has the prefix pref and the suffix suff. If there is more than one valid index,
 *   return the largest of them. If there is no such word in the dictionary, return -1.
 */

/**
 * @param {string[]} words
 */
var WordFilter = function(words) {
  this.map = new Map();

  for (let i = 0; i < words.length; i++) {
    let prefix = '';

    for (let j = 0; j <= words[i].length; j++) {
      let suffix = '';

      for (let k = 0; k <= words[i].length; k++) {
        suffix = words[i].slice(k);
        this.map.set(prefix + '#' + suffix, i);
      }

      prefix += words[i][j];
    }
  }
};
```

```
/**
 * @param {string} pref
 * @param {string} suff
 * @return {number}
 */
WordFilter.prototype.f = function(pref, suff) {
  const target = `${pref}#${suff}`;
  return this.map.has(target) ? this.map.get(target) : -1;
};
```

0746 Min Cost Climbing Stairs

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 746. Min Cost Climbing Stairs
 * https://leetcode.com/problems/min-cost-climbing-stairs/
 * Difficulty: Easy
 *
 * You are given an integer array cost where cost[i] is the cost of ith step on a staircase.
 * Once you pay the cost, you can either climb one or two steps.
 *
 * You can either start from the step with index 0, or the step with index 1.
 *
 * Return the minimum cost to reach the top of the floor.
 */

/**
 * @param {number[]} cost
 * @return {number}
 */
var minCostClimbingStairs = function(cost) {
  let [first, second] = cost;

  for (let index = 2; index < cost.length; index++) {
    const cursor = cost[index] + Math.min(first, second);
    first = second;
    second = cursor;
  }

  return Math.min(first, second);
};
```

0747 Largest Number At Least Twice Of Others

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 747. Largest Number At Least Twice of Others
 * https://leetcode.com/problems/largest-number-at-least-twice-of-others/
 * Difficulty: Easy
 *
 * You are given an integer array nums where the largest integer is unique.
 *
 * Determine whether the largest element in the array is at least twice as much
 * as every other number in the array. If it is, return the index of the largest
 * element, or return -1 otherwise.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var dominantIndex = function(nums) {
  const max = Math.max(...nums);
```

```
  return nums.find(n => n !== max && n > max / 2) ? -1 : nums.indexOf(max);
};
```

0748 Shortest Completing Word

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 748. Shortest Completing Word
 * https://leetcode.com/problems/shortest-completing-word/
 * Difficulty: Easy
 *
 * Given a string licensePlate and an array of strings words, find the shortest completing
 * word in words.
 *
 * A completing word is a word that contains all the letters in licensePlate. Ignore numbers
 * and spaces in licensePlate, and treat letters as case insensitive. If a letter appears more
 * than once in licensePlate, then it must appear in the word the same number of times or
more.
 *
 * For example, if licensePlate = "aBc 12c", then it contains letters 'a', 'b' (ignoring
case),
 * and 'c' twice. Possible completing words are "abccdef", "caaacab", and "cbca".
 *
 * Return the shortest completing word in words. It is guaranteed an answer exists. If there
 * are multiple shortest completing words, return the first one that occurs in words.
 */

/**
 * @param {string} licensePlate
 * @param {string[]} words
 * @return {string}
 */
var shortestCompletingWord = function(licensePlate, words) {
  const license = licensePlate.toLowerCase().replace(/[\d\s]+/g, '');
  const sortedWords = [...words].sort((a, b) => a.length - b.length);

  for (const word of sortedWords) {
    let updatedLicense = license;

    for (let i = 0; i < word.length; i++) {
      updatedLicense = updatedLicense.replace(word[i], '');
      if (!updatedLicense) {
        return word;
      }
    }
  }
};
```

0749 Contain Virus

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 749. Contain Virus
 * https://leetcode.com/problems/contain-virus/
 * Difficulty: Hard
 *
 * A virus is spreading rapidly, and your task is to quarantine the infected area by
 * installing walls.
 *
 * The world is modeled as an m x n binary grid isInfected, where isInfected[i][j] == 0
 * represents uninfected cells, and isInfected[i][j] == 1 represents cells contaminated
 * with the virus. A wall (and only one wall) can be installed between any two 4-directionally
```

```
 * adjacent cells, on the shared boundary.
 *
 * Every night, the virus spreads to all neighboring cells in all four directions unless
 * blocked by a wall. Resources are limited. Each day, you can install walls around only one
 * region (i.e., the affected area (continuous block of infected cells) that threatens the
 * most uninfected cells the following night). There will never be a tie.
 *
 * Return the number of walls used to quarantine all the infected regions. If the world will
 * become fully infected, return the number of walls used.
 */

/**
 * @param {number[][]} isInfected
 * @return {number}
 */
var containVirus = function(isInfected) {
  const verticalBarriers = new Set();
  const horizontalBarriers = new Set();
  const grid = isInfected;
  const rows = grid.length;
  const cols = grid[0].length;
  const directions = [[-1, 0, -cols, -cols], [1, 0, cols, 0], [0, -1, -1, -1], [0, 1, 1, 0]];
  let target;
  let nextState;
  let maxThreat;
  let threatenedCells;
  let nextVertical;
  let nextHorizontal;
  let verticalWalls;
  let horizontalWalls;
  let startRow;
  let startCol;

  while (true) {
    target = 1;
    nextState = 2;
    maxThreat = 0;
    grid.forEach((row, r) => row.forEach((cell, c) => cell && exploreRegion(r, c)));
    if (maxThreat === 0) return verticalBarriers.size + horizontalBarriers.size;
    target = 2;
    nextState = 0;
    traverseRegion(startRow, startCol);
    verticalWalls.forEach(pos => verticalBarriers.add(pos));
    horizontalWalls.forEach(pos => horizontalBarriers.add(pos));
    nextState = 1;
    threatenedCells = new Set();
    grid.forEach((row, r) => row.forEach((cell, c) => cell && traverseRegion(r, c)));
    threatenedCells.forEach(pos => grid[Math.floor(pos / cols)][pos % cols] = 1);
  }

  function exploreRegion(row, col) {
    threatenedCells = new Set();
    nextVertical = new Set();
    nextHorizontal = new Set();
    traverseRegion(row, col);
    if (threatenedCells.size < maxThreat) return;
    maxThreat = threatenedCells.size;
    verticalWalls = nextVertical;
    horizontalWalls = nextHorizontal;
    startRow = row;
    startCol = col;
  }

  function traverseRegion(row, col) {
    if (grid[row][col] !== target) return;
    grid[row][col] = nextState;
    const position = cols * row + col;
    for (let i = 0; i < 4; i++) {
      const [deltaRow, deltaCol, threatOffset, wallOffset] = directions[i];
      const newRow = row + deltaRow;
      const newCol = col + deltaCol;
      if (newRow < 0 || newRow === rows || newCol < 0 || newCol === cols) continue;
      traverseRegion(newRow, newCol);
      if (grid[newRow][newCol]
        || (deltaRow ? verticalBarriers : horizontalBarriers).has(position + wallOffset))
```

```
continue;
      if (nextState) threatenedCells.add(position + threatOffset);
      if (nextState > 1) (deltaRow ? nextVertical : nextHorizontal).add(position +
wallOffset);
    }
  }
};
```

0750 Number Of Corner Rectangles

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 750. Number Of Corner Rectangles
 * https://leetcode.com/problems/number-of-corner-rectangles/
 * Difficulty: Medium
 *
 * Given an m x n integer matrix grid where each entry is only 0 or 1, return the number of
 * corner rectangles.
 *
 * A corner rectangle is four distinct 1's on the grid that forms an axis-aligned rectangle.
 * Note that only the corners need to have the value 1. Also, all four 1's used must be
distinct.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var countCornerRectangles = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  let result = 0;

  for (let row1 = 0; row1 < rows; row1++) {
    for (let row2 = row1 + 1; row2 < rows; row2++) {
      let sharedOnes = 0;
      for (let col = 0; col < cols; col++) {
        if (grid[row1][col] === 1 && grid[row2][col] === 1) {
          sharedOnes++;
        }
      }
      result += sharedOnes * (sharedOnes - 1) / 2;
    }
  }

  return result;
};
```

0751 Ip To Cidr

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 751. IP to CIDR
 * https://leetcode.com/problems/ip-to-cidr/
 * Difficulty: Medium
 *
 * An IP address is a formatted 32-bit unsigned integer where each group of 8 bits is printed
as
 * a decimal number and the dot character '.' splits the groups.
 * - For example, the binary number 00001111 10001000 11111111 01101011 (spaces added for
clarity)
 *   formatted as an IP address would be "15.136.255.107".
 *
```

```
 * A CIDR block is a format used to denote a specific set of IP addresses. It is a string
consisting
 * of a base IP address, followed by a slash, followed by a prefix length k. The addresses it
covers
 * are all the IPs whose first k bits are the same as the base IP address.
 * - For example, "123.45.67.89/20" is a CIDR block with a prefix length of 20. Any IP address
 whose
 *   binary representation matches 01111011 00101101 0100xxxx xxxxxxxx, where x can be either
0 or
 *   1, is in the set covered by the CIDR block.
 *
 * You are given a start IP address ip and the number of IP addresses we need to cover n. Your
 goal
 * is to use as few CIDR blocks as possible to cover all the IP addresses in the inclusive
 * range [ip, ip + n - 1] exactly. No other IP addresses outside of the range should be
covered.
 *
 * Return the shortest list of CIDR blocks that covers the range of IP addresses. If there are
 * multiple answers, return any of them.
 */

/**
 * @param {string} ip
 * @param {number} n
 * @return {string[]}
 */
var ipToCIDR = function(ip, n) {
  let start = ipToNumber(ip);
  const result = [];

  while (n > 0) {
    let rightmostOneBit = start & (-start);
    if (rightmostOneBit === 0) rightmostOneBit = 4294967296;

    let blockSize = Math.min(rightmostOneBit, n);
    let temp = blockSize;
    while ((temp & (temp - 1)) !== 0) {
      temp = temp & (temp - 1);
    }
    blockSize = temp;

    let prefixLength = 32;
    let mask = blockSize;
    while (mask > 1) {
      mask >>>= 1;
      prefixLength--;
    }

    result.push(`${numberToIp(start)}/${prefixLength}`);

    start += blockSize;
    n -= blockSize;
  }

  return result;

  function ipToNumber(ip) {
    const parts = ip.split('.').map(Number);
    return (parts[0] << 24) + (parts[1] << 16) + (parts[2] << 8) + parts[3];
  }

  function numberToIp(num) {
    return [
      (num >>> 24) & 255,
      (num >>> 16) & 255,
      (num >>> 8) & 255,
      num & 255
    ].join('.');
  }
};
```

0752 Open The Lock

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 752. Open the Lock
 * https://leetcode.com/problems/open-the-lock/
 * Difficulty: Medium
 *
 * You have a lock in front of you with 4 circular wheels. Each wheel has 10 slots: '0', '1',
'2',
 * '3', '4', '5', '6', '7', '8', '9'. The wheels can rotate freely and wrap around: for
example
 * we can turn '9' to be '0', or '0' to be '9'. Each move consists of turning one wheel one
slot.
 *
 * The lock initially starts at '0000', a string representing the state of the 4 wheels.
 *
 * You are given a list of deadends dead ends, meaning if the lock displays any of these
codes,
 * the wheels of the lock will stop turning and you will be unable to open it.
 *
 * Given a target representing the value of the wheels that will unlock the lock, return the
 * minimum total number of turns required to open the lock, or -1 if it is impossible.
 */

/**
 * @param {string[]} deadends
 * @param {string} target
 * @return {number}
 */
var openLock = function(deadends, target) {
  const dead = new Set(deadends);
  const queue = [['0000', 0]];
  const seen = new Set(['0000']);

  if (dead.has('0000')) return -1;
  while (queue.length) {
    const [combo, turns] = queue.shift();
    if (combo === target) return turns;

    for (let i = 0; i < 4; i++) {
      const current = combo[i];
      const up = (parseInt(current) + 1) % 10;
      const down = (parseInt(current) + 9) % 10;

      const nextUp = combo.slice(0, i) + up + combo.slice(i + 1);
      if (!seen.has(nextUp) && !dead.has(nextUp)) {
        queue.push([nextUp, turns + 1]);
        seen.add(nextUp);
      }

      const nextDown = combo.slice(0, i) + down + combo.slice(i + 1);
      if (!seen.has(nextDown) && !dead.has(nextDown)) {
        queue.push([nextDown, turns + 1]);
        seen.add(nextDown);
      }
    }
  }

  return -1;
};
```

0753 Cracking The Safe

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 753. Cracking the Safe
 * https://leetcode.com/problems/cracking-the-safe/
 * Difficulty: Hard
 *
 * There is a safe protected by a password. The password is a sequence of n digits where each
 * digit can be in the range [0, k - 1].
 *
 * The safe has a peculiar way of checking the password. When you enter in a sequence, it
checks
 * the most recent n digits that were entered each time you type a digit.
 *
 * For example, the correct password is "345" and you enter in "012345":
 * - After typing 0, the most recent 3 digits is "0", which is incorrect.
 * - After typing 1, the most recent 3 digits is "01", which is incorrect.
 * - After typing 2, the most recent 3 digits is "012", which is incorrect.
 * - After typing 3, the most recent 3 digits is "123", which is incorrect.
 * - After typing 4, the most recent 3 digits is "234", which is incorrect.
 * - After typing 5, the most recent 3 digits is "345", which is correct and the safe unlocks.
 *
 * Return any string of minimum length that will unlock the safe at some point of entering it.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {string}
 */
var crackSafe = function(n, k) {
  if (n === 1) return Array.from({ length: k }, (_, i) => i).join('');
  const seen = new Set();
  const targetSize = k ** n;
  let sequence = '0'.repeat(n);

  seen.add(sequence);
  build(sequence);
  return sequence;

  function build(curr) {
    if (seen.size === targetSize) return true;

    const prefix = curr.slice(-n + 1);
    for (let digit = 0; digit < k; digit++) {
      const next = prefix + digit;
      if (!seen.has(next)) {
        seen.add(next);
        sequence += digit;
        if (build(sequence)) return true;
        sequence = sequence.slice(0, -1);
        seen.delete(next);
      }
    }
    return false;
  }
};
```

0754 Reach A Number

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
```

```
 * 754. Reach a Number
 * https://leetcode.com/problems/reach-a-number/
 * Difficulty: Medium
 *
 * You are standing at position 0 on an infinite number line. There is a destination at
 * position target.
 *
 * You can make some number of moves numMoves so that:
 * - On each move, you can either go left or right.
 * - During the ith move (starting from i == 1 to i == numMoves), you take i steps in the
 *   chosen direction.
 *
 * Given the integer target, return the minimum number of moves required (i.e., the minimum
 * numMoves) to reach the destination.
 */

/**
 * @param {number} target
 * @return {number}
 */
var reachNumber = function(target) {
  const absTarget = Math.abs(target);
  let moves = Math.floor(Math.sqrt(2 * absTarget));

  while (true) {
    const sum = moves * (moves + 1) / 2;
    if (sum >= absTarget && (sum - absTarget) % 2 === 0) return moves;
    moves++;
  }
};
```

0755 Pour Water

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 755. Pour Water
 * https://leetcode.com/problems/pour-water/
 * Difficulty: Medium
 *
 * You are given an elevation map represents as an integer array heights where heights[i]
 * representing the height of the terrain at index i. The width at each index is 1. You
 * are also given two integers volume and k. volume units of water will fall at index k.
 *
 * Water first drops at the index k and rests on top of the highest terrain or water at
 * that index. Then, it flows according to the following rules:
 * - If the droplet would eventually fall by moving left, then move left.
 * - Otherwise, if the droplet would eventually fall by moving right, then move right.
 * - Otherwise, rise to its current position.
 *
 * Here, "eventually fall" means that the droplet will eventually be at a lower level if
 * it moves in that direction. Also, level means the height of the terrain plus any water
 * in that column.
 *
 * We can assume there is infinitely high terrain on the two sides out of bounds of the
 * array. Also, there could not be partial water being spread out evenly on more than one
 * grid block, and each unit of water has to be in exactly one block.
 */

/**
 * @param {number[]} heights
 * @param {number} volume
 * @param {number} k
 * @return {number[]}
 */
var pourWater = function(heights, volume, k) {
  const water = new Array(heights.length).fill(0);

  for (let drop = 0; drop < volume; drop++) {
    if (canFallLeft(k)) {
```

```
      water[findLeftPosition(k)]++;
    } else if (canFallRight(k)) {
      water[findRightPosition(k)]++;
    } else {
      water[k]++;
    }
  }

  return heights.map((h, i) => h + water[i]);

  function canFallLeft(index) {
    let currentLevel = heights[index] + water[index];
    for (let i = index - 1; i >= 0; i--) {
      const level = heights[i] + water[i];
      if (level < currentLevel) return true;
      if (level > currentLevel) return false;
      currentLevel = level;
    }
    return false;
  }

  function canFallRight(index) {
    let currentLevel = heights[index] + water[index];
    for (let i = index + 1; i < heights.length; i++) {
      const level = heights[i] + water[i];
      if (level < currentLevel) return true;
      if (level > currentLevel) return false;
      currentLevel = level;
    }
    return false;
  }

  function findLeftPosition(index) {
    let minHeight = heights[index] + water[index];
    let position = index;
    for (let i = index - 1; i >= 0; i--) {
      const level = heights[i] + water[i];
      if (level > minHeight) break;
      if (level < minHeight) {
        minHeight = level;
        position = i;
      }
    }
    return position;
  }

  function findRightPosition(index) {
    let minHeight = heights[index] + water[index];
    let position = index;
    for (let i = index + 1; i < heights.length; i++) {
      const level = heights[i] + water[i];
      if (level > minHeight) break;
      if (level < minHeight) {
        minHeight = level;
        position = i;
      }
    }
    return position;
  }
};
```

0756 Pyramid Transition Matrix

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 756. Pyramid Transition Matrix
 * https://leetcode.com/problems/pyramid-transition-matrix/
 * Difficulty: Medium
 *
```

```
 * You are stacking blocks to form a pyramid. Each block has a color, which is represented by
 * a single letter. Each row of blocks contains one less block than the row beneath it and is
 * centered on top.
 *
 * To make the pyramid aesthetically pleasing, there are only specific triangular patterns
that
 * are allowed. A triangular pattern consists of a single block stacked on top of two blocks.
 * The patterns are given as a list of three-letter strings allowed, where the first two
characters
 * of a pattern represent the left and right bottom blocks respectively, and the third
character
 * is the top block.
 *
 * For example, "ABC" represents a triangular pattern with a 'C' block stacked on top of an
'A'
 * (left) and 'B' (right) block. Note that this is different from "BAC" where 'B' is on the
left
 * bottom and 'A' is on the right bottom.
 *
 * You start with a bottom row of blocks bottom, given as a single string, that you must use
as
 * the base of the pyramid.
 *
 * Given bottom and allowed, return true if you can build the pyramid all the way to the top
such
 * that every triangular pattern in the pyramid is in allowed, or false otherwise.
 */

/**
 * @param {string} bottom
 * @param {string[]} allowed
 * @return {boolean}
 */
var pyramidTransition = function(bottom, allowed) {
  const transitions = new Map();
  for (const [left, right, top] of allowed) {
    const key = left + right;
    transitions.set(key, (transitions.get(key) || '') + top);
  }

  function canBuild(row, nextRow = '') {
    if (row.length === 1) return true;
    if (nextRow.length === row.length - 1) return canBuild(nextRow);

    const pair = row.slice(nextRow.length, nextRow.length + 2);
    const options = transitions.get(pair) || '';

    for (const top of options) {
      if (canBuild(row, nextRow + top)) return true;
    }
    return false;
  }

  return canBuild(bottom);
};
```

0757 Set Intersection Size At Least Two

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 757. Set Intersection Size At Least Two
 * https://leetcode.com/problems/set-intersection-size-at-least-two/
 * Difficulty: Hard
 *
 * You are given a 2D integer array intervals where intervals[i] = [starti, endi] represents
 * all the integers from starti to endi inclusively.
 *
 * A containing set is an array nums where each interval from intervals has at least two
 * integers in nums.
```

```
 *
 * For example, if intervals = [[1,3], [3,7], [8,9]], then [1,2,4,7,8,9] and [2,3,4,8,9]
 * are containing sets.
 *
 * Return the minimum possible size of a containing set.
 */

/**
 * @param {number[][]} intervals
 * @return {number}
 */
var intersectionSizeTwo = function(intervals) {
  intervals.sort((a, b) => a[1] - b[1] || b[0] - a[0]);
  let size = 0;
  let smallest = -1;
  let secondSmallest = -1;

  for (const [start, end] of intervals) {
    const hasTwo = start <= smallest;
    const hasOne = start <= secondSmallest;

    if (hasTwo) continue;
    if (hasOne) {
      smallest = secondSmallest;
      secondSmallest = end;
      size++;
    } else {
      smallest = end - 1;
      secondSmallest = end;
      size += 2;
    }
  }

  return size;
};
```

0758 Bold Words In String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 758. Bold Words in String
 * https://leetcode.com/problems/bold-words-in-string/
 * Difficulty: Medium
 *
 * Given an array of keywords words and a string s, make all appearances of all keywords
words[i]
 * in s bold. Any letters between <b> and </b> tags become bold.
 *
 * Return s after adding the bold tags. The returned string should use the least number of
tags
 * possible, and the tags should form a valid combination.
 */

/**
 * @param {string[]} words
 * @param {string} s
 * @return {string}
 */
var boldWords = function(words, s) {
  const boldIntervals = [];

  for (const word of words) {
    let start = s.indexOf(word);
    while (start !== -1) {
      boldIntervals.push([start, start + word.length]);
      start = s.indexOf(word, start + 1);
    }
  }
```

```
  if (!boldIntervals.length) return s;

  boldIntervals.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  const mergedIntervals = [];
  let [currentStart, currentEnd] = boldIntervals[0];

  for (let i = 1; i < boldIntervals.length; i++) {
    const [nextStart, nextEnd] = boldIntervals[i];
    if (nextStart <= currentEnd) {
      currentEnd = Math.max(currentEnd, nextEnd);
    } else {
      mergedIntervals.push([currentStart, currentEnd]);
      [currentStart, currentEnd] = [nextStart, nextEnd];
    }
  }
  mergedIntervals.push([currentStart, currentEnd]);

  let result = '';
  let lastEnd = 0;
  for (const [start, end] of mergedIntervals) {
    result += s.slice(lastEnd, start) + '<b>' + s.slice(start, end) + '</b>';
    lastEnd = end;
  }

  result += s.slice(lastEnd);

  return result;
};
```

0759 Employee Free Time

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 759. Employee Free Time
 * https://leetcode.com/problems/employee-free-time/
 * Difficulty: Hard
 *
 * We are given a list schedule of employees, which represents the working time for each
employee.
 *
 * Each employee has a list of non-overlapping Intervals, and these intervals are in sorted
order.
 *
 * Return the list of finite intervals representing common, positive-length free time for all
 * employees, also in sorted order.
 *
 * (Even though we are representing Intervals in the form [x, y], the objects inside are
Intervals,
 * not lists or arrays. For example, schedule[0][0].start = 1, schedule[0][0].end = 2, and
 * schedule[0][0][0] is not defined).  Also, we wouldn't include intervals like [5, 5] in our
 * answer, as they have zero length.
 */

/**
 * @param {Interval[][]} schedule
 * @return {Interval[]}
 */
var employeeFreeTime = function(schedule) {
  const allIntervals = [];

  for (const employee of schedule) {
    for (const interval of employee) {
      allIntervals.push(interval);
    }
  }

  allIntervals.sort((a, b) => a.start - b.start);
```

```
  const merged = [];
  for (const interval of allIntervals) {
    if (merged.length === 0 || merged[merged.length - 1].end < interval.start) {
      merged.push(interval);
    } else {
      merged[merged.length - 1].end = Math.max(merged[merged.length - 1].end, interval.end);
    }
  }

  const result = [];
  for (let i = 0; i < merged.length - 1; i++) {
    if (merged[i].end < merged[i + 1].start) {
      result.push(new Interval(merged[i].end, merged[i + 1].start));
    }
  }

  return result;
};
```

0760 Find Anagram Mappings

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 760. Find Anagram Mappings
 * https://leetcode.com/problems/find-anagram-mappings/
 * Difficulty: Easy
 *
 * You are given two integer arrays nums1 and nums2 where nums2 is an anagram of nums1. Both
 * arrays may contain duplicates.
 *
 * Return an index mapping array mapping from nums1 to nums2 where mapping[i] = j means the
 * ith element in nums1 appears in nums2 at index j. If there are multiple answers, return
 * any of them.
 *
 * An array a is an anagram of an array b means b is made by randomizing the order of the
 * elements in a.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var anagramMappings = function(nums1, nums2) {
  const map = new Map();

  for (let i = 0; i < nums2.length; i++) {
    if (!map.has(nums2[i])) {
      map.set(nums2[i], []);
    }
    map.get(nums2[i]).push(i);
  }

  const result = [];
  for (const num of nums1) {
    result.push(map.get(num).pop());
  }

  return result;
};
```

0761 Special Binary String

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 761. Special Binary String
 * https://leetcode.com/problems/special-binary-string/
 * Difficulty: Hard
 *
 * Special binary strings are binary strings with the following two properties:
 * - The number of 0's is equal to the number of 1's.
 * - Every prefix of the binary string has at least as many 1's as 0's.
 *
 * You are given a special binary string s.
 *
 * A move consists of choosing two consecutive, non-empty, special substrings of s,
 * and swapping them. Two strings are consecutive if the last character of the first
 * string is exactly one index before the first character of the second string.
 *
 * Return the lexicographically largest resulting string possible after applying
 * the mentioned operations on the string.
 */

/**
 * @param {string} s
 * @return {string}
 */
function makeLargestSpecial(s) {
  if (s.length <= 2) return s;

  let count = 0;
  let start = 0;
  const specials = [];

  for (let i = 0; i < s.length; i++) {
    count += s[i] === '1' ? 1 : -1;

    if (count === 0) {
      specials.push('1' + makeLargestSpecial(s.slice(start + 1, i)) + '0');
      start = i + 1;
    }
  }

  return specials.sort().reverse().join('');
}
```

0762 Prime Number Of Set Bits In Binary Representation

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 762. Prime Number of Set Bits in Binary Representation
 * https://leetcode.com/problems/prime-number-of-set-bits-in-binary-representation/
 * Difficulty: Easy
 *
 * Given two integers left and right, return the count of numbers in the inclusive range
[left, right] having
 * a prime number of set bits in their binary representation.
 *
 * Recall that the number of set bits an integer has is the number of 1's present when written
 in binary.
 *
 * For example, 21 written in binary is 10101, which has 3 set bits.
 */

/**
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
var countPrimeSetBits = function(left, right) {
```

```
  const primes = new Set([2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]);
  let result = 0;

  for (let i = left; i <= right; i++) {
    if (primes.has(i.toString(2).replace(/0+/g, '').length)) {
      result++;
    }
  }

  return result;
};
```

0763 Partition Labels

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 763. Partition Labels
 * https://leetcode.com/problems/partition-labels/
 * Difficulty: Medium
 *
 * You are given a string s. We want to partition the string into as many parts as
 * possible so that each letter appears in at most one part. For example, the
 * string "ababcc" can be partitioned into ["abab", "cc"], but partitions such as
 * ["aba", "bcc"] or ["ab", "ab", "cc"] are invalid.
 *
 * Note that the partition is done so that after concatenating all the parts in
 * order, the resultant string should be s.
 *
 * Return a list of integers representing the size of these parts.
 */

/**
 * @param {string} s
 * @return {number[]}
 */
var partitionLabels = function(s) {
  const map = new Map();
  for (let i = 0; i < s.length; i++) {
    map.set(s[i], i);
  }

  const result = [];
  for (let i = 0, start = 0, end = 0; i < s.length; i++) {
    end = Math.max(end, map.get(s[i]));
    if (i === end) {
      result.push(end - start + 1);
      start = i + 1;
    }
  }

  return result;
};
```

0764 Largest Plus Sign

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 764. Largest Plus Sign
 * https://leetcode.com/problems/largest-plus-sign/
 * Difficulty: Medium
 *
 * You are given an integer n. You have an n x n binary grid grid with all values initially
 * 1's except for some indices given in the array mines. The ith element of the array mines
```

```
 * is defined as mines[i] = [xi, yi] where grid[xi][yi] == 0.
 *
 * Return the order of the largest axis-aligned plus sign of 1's contained in grid. If
 * there is none, return 0.
 *
 * An axis-aligned plus sign of 1's of order k has some center grid[r][c] == 1 along with four
 * arms of length k - 1 going up, down, left, and right, and made of 1's. Note that there
could
 * be 0's or 1's beyond the arms of the plus sign, only the relevant area of the plus sign is
 * checked for 1's.
 */

/**
 * @param {number} n
 * @param {number[][]} mines
 * @return {number}
 */
function orderOfLargestPlusSign(n, mines) {
  const grid = Array(n).fill().map(() => Array(n).fill(1));
  mines.forEach(([x, y]) => grid[x][y] = 0);

  const left = Array(n).fill().map(() => Array(n).fill(0));
  const right = Array(n).fill().map(() => Array(n).fill(0));
  const up = Array(n).fill().map(() => Array(n).fill(0));
  const down = Array(n).fill().map(() => Array(n).fill(0));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      left[i][j] = grid[i][j] ? (j > 0 ? left[i][j-1] + 1 : 1) : 0;
      up[i][j] = grid[i][j] ? (i > 0 ? up[i-1][j] + 1 : 1) : 0;
    }
  }

  for (let i = n - 1; i >= 0; i--) {
    for (let j = n - 1; j >= 0; j--) {
      right[i][j] = grid[i][j] ? (j < n-1 ? right[i][j+1] + 1 : 1) : 0;
      down[i][j] = grid[i][j] ? (i < n-1 ? down[i+1][j] + 1 : 1) : 0;
    }
  }

  let maxOrder = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      maxOrder = Math.max(
        maxOrder,
        Math.min(left[i][j], right[i][j], up[i][j], down[i][j])
      );
    }
  }

  return maxOrder;
}
```

0765 Couples Holding Hands

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 765. Couples Holding Hands
 * https://leetcode.com/problems/couples-holding-hands/
 * Difficulty: Hard
 *
 * There are n couples sitting in 2n seats arranged in a row and want to hold hands.
 *
 * The people and seats are represented by an integer array row where row[i] is the ID of the
 * person sitting in the ith seat. The couples are numbered in order, the first couple being
 * (0, 1), the second couple being (2, 3), and so on with the last couple being (2n - 2, 2n -
1).
 *
 * Return the minimum number of swaps so that every couple is sitting side by side. A swap
```

```
 * consists of choosing any two people, then they stand up and switch seats.
 */

/**
 * @param {number[]} row
 * @return {number}
 */
function minSwapsCouples(row) {
  const n = row.length / 2;
  const partner = new Array(2 * n);
  const position = new Array(2 * n);

  for (let i = 0; i < 2 * n; i++) {
    partner[i] = i % 2 === 0 ? i + 1 : i - 1;
    position[row[i]] = i;
  }

  let swaps = 0;
  for (let i = 0; i < 2 * n; i += 2) {
    const current = row[i];
    const expected = partner[current];
    const nextPerson = row[i + 1];

    if (nextPerson !== expected) {
      row[i + 1] = expected;
      row[position[expected]] = nextPerson;
      position[nextPerson] = position[expected];
      position[expected] = i + 1;
      swaps++;
    }
  }

  return swaps;
}
```

0766 Toeplitz Matrix

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 766. Toeplitz Matrix
 * https://leetcode.com/problems/toeplitz-matrix/
 * Difficulty: Easy
 *
 * Given an m x n matrix, return true if the matrix is Toeplitz. Otherwise, return false.
 *
 * A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.
 */

/**
 * @param {number[][]} matrix
 * @return {boolean}
 */
function isToeplitzMatrix(matrix) {
  const rows = matrix.length;
  const cols = matrix[0].length;

  for (let r = 0; r < rows - 1; r++) {
    for (let c = 0; c < cols - 1; c++) {
      if (matrix[r][c] !== matrix[r + 1][c + 1]) {
        return false;
      }
    }
  }

  return true;
}
```

0767 Reorganize String

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 767. Reorganize String
 * https://leetcode.com/problems/reorganize-string/
 * Difficulty: Medium
 *
 * Given a string s, rearrange the characters of s so that any two adjacent characters are
 * not the same.
 *
 * Return any possible rearrangement of s or return "" if not possible.
 */

/**
 * @param {string} s
 * @return {string}
 */
var reorganizeString = function(s) {
  const charCount = new Map();
  for (const char of s) {
    charCount.set(char, (charCount.get(char) || 0) + 1);
  }

  const maxHeap = [...charCount.entries()]
    .sort((a, b) => b[1] - a[1]);

  if (maxHeap[0][1] > Math.ceil(s.length / 2)) {
    return '';
  }

  const result = [];
  let index = 0;

  while (maxHeap.length) {
    const [char, count] = maxHeap.shift();
    for (let i = 0; i < count; i++) {
      result[index] = char;
      index += 2;
      if (index >= s.length) index = 1;
    }
  }

  return result.join('');
};
```

0768 Max Chunks To Make Sorted Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 768. Max Chunks To Make Sorted II
 * https://leetcode.com/problems/max-chunks-to-make-sorted-ii/
 * Difficulty: Hard
 *
 * You are given an integer array arr.
 *
 * We split arr into some number of chunks (i.e., partitions), and individually sort each
chunk.
 * After concatenating them, the result should equal the sorted array.
 *
 * Return the largest number of chunks we can make to sort the array.
```

```
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var maxChunksToSorted = function(arr) {
  const stack = [];

  for (const num of arr) {
    if (stack.length === 0 || stack[stack.length - 1] <= num) {
      stack.push(num);
    } else {
      const max = stack.pop();
      while (stack.length && stack[stack.length - 1] > num) {
        stack.pop();
      }
      stack.push(max);
    }
  }

  return stack.length;
};
```

0769 Max Chunks To Make Sorted

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 769. Max Chunks To Make Sorted
 * https://leetcode.com/problems/max-chunks-to-make-sorted/
 * Difficulty: Medium
 *
 * You are given an integer array arr of length n that represents a permutation of the
integers
 * in the range [0, n - 1].
 *
 * We split arr into some number of chunks (i.e., partitions), and individually sort each
chunk.
 * After concatenating them, the result should equal the sorted array.
 *
 * Return the largest number of chunks we can make to sort the array.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var maxChunksToSorted = function(arr) {
  let result = 0;

  for (let i = 0, max = 0; i < arr.length; i++) {
    max = Math.max(max, arr[i]);
    if (max === i) {
      result++;
    }
  }

  return result;
};
```

0770 Basic Calculator Iv

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 770. Basic Calculator IV
 * https://leetcode.com/problems/basic-calculator-iv/
 * Difficulty: Hard
 *
 * Given an expression such as expression = "e + 8 - a + 5" and an evaluation map such as
{"e": 1}
 * (given in terms of evalvars = ["e"] and evalints = [1]), return a list of tokens
representing
 * the simplified expression, such as ["-1*a","14"]
 * - An expression alternates chunks and symbols, with a space separating each chunk and
symbol.
 * - A chunk is either an expression in parentheses, a variable, or a non-negative integer.
 * - A variable is a string of lowercase letters (not including digits.) Note that variables
can
 *   be multiple letters, and note that variables never have a leading coefficient or unary
operator
 *   like "2x" or "-x".
 *
 * Expressions are evaluated in the usual order: brackets first, then multiplication, then
addition
 * and subtraction.
 *
 * - For example, expression = "1 + 2 * 3" has an answer of ["7"].
 *
 * The format of the output is as follows:
 * - For each term of free variables with a non-zero coefficient, we write the free variables
within
 *   a term in sorted order lexicographically.
 *   - For example, we would never write a term like "b*a*c", only "a*b*c".
 * - Terms have degrees equal to the number of free variables being multiplied, counting
 *   multiplicity. We write the largest degree terms of our answer first, breaking ties by
 *   lexicographic order ignoring the leading coefficient of the term.
 *   - For example, "a*a*b*c" has degree 4.
 * - The leading coefficient of the term is placed directly to the left with an asterisk
separating
 *   it from the variables (if they exist.) A leading coefficient of 1 is still printed.
 * - An example of a well-formatted answer is ["-2*a*a*a", "3*a*a*b", "3*b*b", "4*a", "5*c",
"-6"].
 * - Terms (including constant terms) with coefficient 0 are not included.
 *   - For example, an expression of "0" has an output of [].
 *
 * Note: You may assume that the given expression is always valid. All intermediate results
will be
 * in the range of [-231, 231 - 1].
 */

/**
 * @param {string} expression
 * @param {string[]} evalvars
 * @param {number[]} evalints
 * @return {string[]}
 */
var basicCalculatorIV = function(expression, evalvars, evalints) {
  const map = new Map();
  for (let i = 0; i < evalvars.length; i++) {
    map.set(evalvars[i], evalints[i]);
  }

  const result = parse(expression, map);
  return result.toStringArray();

  function parse(expr, map) {
    const tokens = tokenize(expr);
    return parseExpr(tokens, 0, map)[0];
  }

  function tokenize(expr) {
    const tokens = [];
    let i = 0;

    while (i < expr.length) {
      if (expr[i] === ' ') {
        i++; continue;
```

```
    }

    if ('+-*()'.includes(expr[i])) {
      tokens.push(expr[i++]);
      continue;
    }

    if (/[a-z]/.test(expr[i])) {
      let variable = '';
      while (i < expr.length && /[a-z]/.test(expr[i])) {
        variable += expr[i++];
      }
      tokens.push(variable);
      continue;
    }

    if (/\d/.test(expr[i])) {
      let num = '';
      while (i < expr.length && /\d/.test(expr[i])) {
        num += expr[i++];
      }
      tokens.push(parseInt(num));
      continue;
    }

    i++;
  }

  return tokens;
}

function parseExpr(tokens, start, map) {
  let [left, pos] = parseTerm(tokens, start, map);

  while (pos < tokens.length && (tokens[pos] === '+' || tokens[pos] === '-')) {
    const op = tokens[pos];
    const [right, nextPos] = parseTerm(tokens, pos + 1, map);

    left = op === '+' ? left.add(right) : left.subtract(right);
    pos = nextPos;
  }

  return [left, pos];
}

function parseTerm(tokens, start, map) {
  let [left, pos] = parseFactor(tokens, start, map);

  while (pos < tokens.length && tokens[pos] === '*') {
    const [right, nextPos] = parseFactor(tokens, pos + 1, map);
    left = left.multiply(right);
    pos = nextPos;
  }

  return [left, pos];
}

function parseFactor(tokens, start, map) {
  const token = tokens[start];

  if (token === '(') {
    const [expr, pos] = parseExpr(tokens, start + 1, map);
    return [expr, pos + 1];
  }

  if (typeof token === 'string' && /[a-z]/.test(token)) {
    return map.has(token)
      ? [new Expression([new Term(map.get(token))]), start + 1]
      : [new Expression([new Term(1, [token])]), start + 1];
  }

  if (typeof token === 'number') {
    return [new Expression([new Term(token)]), start + 1];
  }
```

```
    throw new Error(`Unexpected token: ${token}`);
  }
};

class Term {
  constructor(coefficient = 0, variables = []) {
    this.coefficient = coefficient;
    this.variables = [...variables].sort();
  }

  multiply(other) {
    return new Term(
      this.coefficient * other.coefficient,
      [...this.variables, ...other.variables].sort()
    );
  }

  toString() {
    if (this.coefficient === 0) return '';
    if (this.variables.length === 0) return `${this.coefficient}`;
    return `${this.coefficient}*${this.variables.join('*')}`;
  }

  get degree() {
    return this.variables.length;
  }

  compare(other) {
    if (this.degree !== other.degree) return other.degree - this.degree;

    for (let i = 0; i < this.degree; i++) {
      if (this.variables[i] !== other.variables[i]) {
        return this.variables[i].localeCompare(other.variables[i]);
      }
    }
    return 0;
  }
}

class Expression {
  constructor(terms = []) {
    this.terms = terms;
  }

  add(other, multiplier = 1) {
    const termMap = new Map();

    for (const term of this.terms) {
      const key = term.variables.join('*');
      termMap.set(key, term);
    }

    for (const term of other.terms) {
      const key = term.variables.join('*');
      if (termMap.has(key)) {
        termMap.get(key).coefficient += term.coefficient * multiplier;
      } else {
        const newTerm = new Term(term.coefficient * multiplier, term.variables);
        this.terms.push(newTerm);
        termMap.set(key, newTerm);
      }
    }

    this.terms = this.terms.filter(term => term.coefficient !== 0);
    return this;
  }

  subtract(other) {
    return this.add(other, -1);
  }

  multiply(other) {
    const result = new Expression();

    for (const term1 of this.terms) {
```

```
      for (const term2 of other.terms) {
        const product = term1.multiply(term2);
        if (product.coefficient !== 0) {
          result.add(new Expression([product]));
        }
      }
    }

    return result;
  }

  toStringArray() {
    this.terms.sort((a, b) => a.compare(b));
    return this.terms.map(term => term.toString()).filter(Boolean);
  }
}
```

0771 Jewels And Stones

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 771. Jewels and Stones
 * https://leetcode.com/problems/jewels-and-stones/
 * Difficulty: Easy
 *
 * You're given strings jewels representing the types of stones that are jewels, and stones
 * representing the stones you have. Each character in stones is a type of stone you have.
 * You want to know how many of the stones you have are also jewels.
 *
 * Letters are case sensitive, so "a" is considered a different type of stone from "A".
 */

/**
 * @param {string} jewels
 * @param {string} stones
 * @return {number}
 */
var numJewelsInStones = function(jewels, stones) {
  const set = new Set(jewels);
  let result = 0;

  for (const stone of stones) {
    if (set.has(stone)) {
      result++;
    }
  }

  return result;
};
```

0772 Basic Calculator Iii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 772. Basic Calculator III
 * https://leetcode.com/problems/basic-calculator-iii/
 * Difficulty: Hard
 *
 * Implement a basic calculator to evaluate a simple expression string.
 *
 * The expression string contains only non-negative integers, '+', '-', '*', '/' operators,
and
 * open '(' and closing parentheses ')'. The integer division should truncate toward zero.
```

```
 *
 * You may assume that the given expression is always valid. All intermediate results will be
 * in the range of [-231, 231 - 1].
 *
 * Note: You are not allowed to use any built-in function which evaluates strings as
mathematical
 * expressions, such as eval().
 */

/**
 * @param {string} s
 * @return {number}
 */
var calculate = function(s) {
  let index = 0;
  return parseExpression();

  function parseExpression() {
    let result = parseTerm();

    while (index < s.length && (s[index] === '+' || s[index] === '-')) {
      const operator = s[index++];
      const term = parseTerm();
      result = operator === '+' ? result + term : result - term;
    }

    return result;
  }

  function parseTerm() {
    let result = parseFactor();

    while (index < s.length && (s[index] === '*' || s[index] === '/')) {
      const operator = s[index++];
      const factor = parseFactor();
      if (operator === '*') {
        result *= factor;
      } else {
        result = Math.trunc(result / factor);
      }
    }

    return result;
  }

  function parseFactor() {
    skipSpaces();

    if (s[index] === '(') {
      index++;
      const result = parseExpression();
      index++;
      return result;
    }

    let result = 0;
    while (index < s.length && s[index] >= '0' && s[index] <= '9') {
      result = result * 10 + parseInt(s[index++]);
    }

    return result;
  }

  function skipSpaces() {
    while (index < s.length && s[index] === ' ') {
      index++;
    }
  }
};
```

0773 Sliding Puzzle

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 773. Sliding Puzzle
 * https://leetcode.com/problems/sliding-puzzle/
 * Difficulty: Hard
 *
 * On an 2 x 3 board, there are five tiles labeled from 1 to 5, and an empty square
represented by
 * 0. A move consists of choosing 0 and a 4-directionally adjacent number and swapping it.
 *
 * The state of the board is solved if and only if the board is [[1,2,3],[4,5,0]].
 *
 * Given the puzzle board board, return the least number of moves required so that the state
of the
 * board is solved. If it is impossible for the state of the board to be solved, return -1.
 */

/**
 * @param {number[][]} board
 * @return {number}
 */
var slidingPuzzle = function(board) {
  const target = '123450';
  const moves = [
    [1, 3], [0, 2, 4], [1, 5],
    [0, 4], [1, 3, 5], [2, 4]
  ];

  const start = board.flat().join('');
  if (start === target) return 0;

  const queue = [[start, 0]];
  const seen = new Set([start]);

  while (queue.length) {
    const [state, steps] = queue.shift();
    const index = state.indexOf('0');

    for (const next of moves[index]) {
      const chars = state.split('');
      [chars[index], chars[next]] = [chars[next], chars[index]];
      const nextState = chars.join('');

      if (nextState === target) return steps + 1;

      if (!seen.has(nextState)) {
        seen.add(nextState);
        queue.push([nextState, steps + 1]);
      }
    }
  }

  return -1;
};
```

0774 Minimize Max Distance To Gas Station

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 774. Minimize Max Distance to Gas Station
 * https://leetcode.com/problems/minimize-max-distance-to-gas-station/
 * Difficulty: Hard
 *
 * You are given an integer array stations that represents the positions of the gas stations
 * on the x-axis. You are also given an integer k.
 *
 * You should add k new gas stations. You can add the stations anywhere on the x-axis, and
 * not necessarily on an integer position.
 *
 * Let penalty() be the maximum distance between adjacent gas stations after adding the k
 * new stations.
 *
 * Return the smallest possible value of penalty(). Answers within 10-6 of the actual answer
 * will be accepted.
 */

/**
 * @param {number[]} stations
 * @param {number} k
 * @return {number}
 */
var minmaxGasDist = function(stations, k) {
  const gaps = [];
  for (let i = 1; i < stations.length; i++) {
    gaps.push(stations[i] - stations[i - 1]);
  }

  let left = 0;
  let right = Math.max(...gaps);

  while (right - left > 1e-6) {
    const mid = (left + right) / 2;

    let stationsNeeded = 0;
    for (const gap of gaps) {
      stationsNeeded += Math.floor(gap / mid);
    }

    if (stationsNeeded <= k) {
      right = mid;
    } else {
      left = mid;
    }
  }

  return right;
};
```

0775 Global And Local Inversions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 775. Global and Local Inversions
 * https://leetcode.com/problems/global-and-local-inversions/
 * Difficulty: Medium
 *
 * You are given an integer array nums of length n which represents a permutation of all the
 * integers in the range [0, n - 1].
 *
 * The number of global inversions is the number of the different pairs (i, j) where:
 * - 0 <= i < j < n
 * - nums[i] > nums[j]
 *
 * The number of local inversions is the number of indices i where:
 * - 0 <= i < n - 1
 * - nums[i] > nums[i + 1]
 *
 * Return true if the number of global inversions is equal to the number of local inversions.
```

```
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isIdealPermutation = function(nums) {
  for (let i = 0; i < nums.length; i++) {
    if (Math.abs(nums[i] - i) > 1) {
      return false;
    }
  }
  return true;
};
```

0776 Split Bst

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 776. Split BST
 * https://leetcode.com/problems/split-bst/
 * Difficulty: Medium
 *
 * Given the root of a binary search tree (BST) and an integer target, split the tree into
 * two subtrees where the first subtree has nodes that are all smaller or equal to the
 * target value, while the second subtree has all nodes that are greater than the target
 * value. It is not necessarily the case that the tree contains a node with the value target.
 *
 * Additionally, most of the structure of the original tree should remain. Formally, for
 * any child c with parent p in the original tree, if they are both in the same subtree
 * after the split, then node c should still have the parent p.
 *
 * Return an array of the two roots of the two subtrees in order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} target
 * @return {TreeNode[]}
 */
var splitBST = function(root, target) {
  if (!root) return [null, null];

  if (root.val <= target) {
    const [smaller, larger] = splitBST(root.right, target);
    root.right = smaller;
    return [root, larger];
  } else {
    const [smaller, larger] = splitBST(root.left, target);
    root.left = larger;
    return [smaller, root];
  }
};
```

0777 Swap Adjacent In Lr String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 777. Swap Adjacent in LR String
 * https://leetcode.com/problems/swap-adjacent-in-lr-string/
 * Difficulty: Medium
 *
 * In a string composed of 'L', 'R', and 'X' characters, like "RXXLRXRXL", a move consists of
either
 * replacing one occurrence of "XL" with "LX", or replacing one occurrence of "RX" with "XR".
Given
 * the starting string start and the ending string result, return True if and only if there
exists
 * a sequence of moves to transform start to result.
 */

/**
 * @param {string} start
 * @param {string} result
 * @return {boolean}
 */
var canTransform = function(start, result) {
  if (start.replace(/X/g, '') !== result.replace(/X/g, '')) {
    return false;
  }

  let i = 0;
  let j = 0;
  const n = start.length;

  while (i < n && j < n) {
    while (i < n && start[i] === 'X') i++;
    while (j < n && result[j] === 'X') j++;

    if (i === n && j === n) return true;
    if (i === n || j === n) return false;

    if (start[i] !== result[j]) return false;

    if (start[i] === 'L' && i < j) return false;
    if (start[i] === 'R' && i > j) return false;

    i++;
    j++;
  }

  return true;
};
```

0778 Swim In Rising Water

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 778. Swim in Rising Water
 * https://leetcode.com/problems/swim-in-rising-water/
 * Difficulty: Hard
 *
 * You are given an n x n integer matrix grid where each value grid[i][j] represents the
elevation
 * at that point (i, j).
 *
 * The rain starts to fall. At time t, the depth of the water everywhere is t. You can swim
from
 * a square to another 4-directionally adjacent square if and only if the elevation of both
squares
 * individually are at most t. You can swim infinite distances in zero time. Of course, you
must
 * stay within the boundaries of the grid during your swim.
```

```
 *
 * Return the least time until you can reach the bottom right square (n - 1, n - 1) if you
start
 * at the top left square (0, 0).
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var swimInWater = function(grid) {
  const n = grid.length;
  let left = grid[0][0];
  let right = n * n - 1;

  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  const canReachDestination = (time) => {
    if (grid[0][0] > time) return false;

    const visited = Array(n).fill().map(() => Array(n).fill(false));
    const queue = [[0, 0]];
    visited[0][0] = true;

    while (queue.length > 0) {
      const [row, col] = queue.shift();

      if (row === n - 1 && col === n - 1) {
        return true;
      }

      for (const [dr, dc] of directions) {
        const newRow = row + dr;
        const newCol = col + dc;

        if (
          newRow >= 0 && newRow < n && newCol >= 0 && newCol < n
          && !visited[newRow][newCol] && grid[newRow][newCol] <= time
        ) {
          queue.push([newRow, newCol]);
          visited[newRow][newCol] = true;
        }
      }
    }

    return false;
  };

  while (left < right) {
    const mid = Math.floor((left + right) / 2);

    if (canReachDestination(mid)) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;
};
```

0779 K Th Symbol In Grammar

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 779. K-th Symbol in Grammar
 * https://leetcode.com/problems/k-th-symbol-in-grammar/
 * Difficulty: Medium
 *
```

```
 * We build a table of n rows (1-indexed). We start by writing 0 in the 1st row. Now in every
 * subsequent row, we look at the previous row and replace each occurrence of 0 with 01, and
 * each occurrence of 1 with 10.
 *
 * - For example, for n = 3, the 1st row is 0, the 2nd row is 01, and the 3rd row is 0110.
 *
 * Given two integer n and k, return the kth (1-indexed) symbol in the nth row of a table of n
 rows.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var kthGrammar = function(n, k) {
  if (n === 1) return 0;

  const length = 1 << (n - 1);
  const mid = length / 2;

  if (k <= mid) {
    return kthGrammar(n - 1, k);
  } else {
    return 1 - kthGrammar(n - 1, k - mid);
  }
};
```

0780 Reaching Points

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 780. Reaching Points
 * https://leetcode.com/problems/reaching-points/
 * Difficulty: Hard
 *
 * Given four integers sx, sy, tx, and ty, return true if it is possible to convert the point
 * (sx, sy) to the point (tx, ty) through some operations, or false otherwise.
 *
 * The allowed operation on some point (x, y) is to convert it to either (x, x + y) or (x + y,
 y).
 */

/**
 * @param {number} sx
 * @param {number} sy
 * @param {number} tx
 * @param {number} ty
 * @return {boolean}
 */
var reachingPoints = function(sx, sy, tx, ty) {
  while (tx >= sx && ty >= sy) {
    if (tx === sx && ty === sy) {
      return true;
    }

    if (tx > ty) {
      // If we can directly reach sx by making proper subtractions
      if (ty === sy) {
        return (tx - sx) % ty === 0;
      }
      tx %= ty;
    } else {
      // If we can directly reach sy by making proper subtractions
      if (tx === sx) {
        return (ty - sy) % tx === 0;
      }
      ty %= tx;
    }
```

```
  }

  return false;
};
```

0781 Rabbits In Forest

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 781. Rabbits in Forest
 * https://leetcode.com/problems/rabbits-in-forest/
 * Difficulty: Medium
 *
 * There is a forest with an unknown number of rabbits. We asked n rabbits "How many rabbits
have
 * the same color as you?" and collected the answers in an integer array answers where
answers[i]
 * is the answer of the ith rabbit.
 *
 * Given the array answers, return the minimum number of rabbits that could be in the forest.
 */

/**
 * @param {number[]} answers
 * @return {number}
 */
var numRabbits = function(answers) {
  const colorGroups = {};
  let totalRabbits = 0;

  for (const answer of answers) {
    if (answer === 0) {
      totalRabbits++;
      continue;
    }

    if (!colorGroups[answer] || colorGroups[answer] === 0) {
      totalRabbits += answer + 1;
      colorGroups[answer] = answer;
    } else {
      colorGroups[answer]--;
    }
  }

  return totalRabbits;
};
```

0782 Transform To Chessboard

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 782. Transform to Chessboard
 * https://leetcode.com/problems/transform-to-chessboard/
 * Difficulty: Hard
 *
 * You are given an n x n binary grid board. In each move, you can swap any two rows with each
 * other, or any two columns with each other.
 *
 * Return the minimum number of moves to transform the board into a chessboard board. If the
 * task is impossible, return -1.
 *
 * A chessboard board is a board where no 0's and no 1's are 4-directionally adjacent.
 */
```

```
/**
 * @param {number[][]} board
 * @return {number}
 */
var movesToChessboard = function(board) {
  const n = board.length;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if ((board[0][0] ^ board[i][0] ^ board[0][j] ^ board[i][j]) === 1) {
        return -1;
      }
    }
  }

  let rowSum = 0;
  let colSum = 0;
  let rowSwaps = 0;
  let colSwaps = 0;

  for (let i = 0; i < n; i++) {
    rowSum += board[0][i];
    colSum += board[i][0];

    if (board[i][0] === i % 2) rowSwaps++;
    if (board[0][i] === i % 2) colSwaps++;
  }

  if (rowSum !== Math.floor(n / 2) && rowSum !== Math.ceil(n / 2)) return -1;
  if (colSum !== Math.floor(n / 2) && colSum !== Math.ceil(n / 2)) return -1;

  if (n % 2 === 1) {
    if (rowSwaps % 2 === 1) rowSwaps = n - rowSwaps;
    if (colSwaps % 2 === 1) colSwaps = n - colSwaps;
  } else {
    rowSwaps = Math.min(rowSwaps, n - rowSwaps);
    colSwaps = Math.min(colSwaps, n - colSwaps);
  }

  return Math.floor((rowSwaps + colSwaps) / 2);
};
```

0783 Minimum Distance Between Bst Nodes

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 783. Minimum Distance Between BST Nodes
 * https://leetcode.com/problems/minimum-distance-between-bst-nodes/
 * Difficulty: Easy
 *
 * Given the root of a Binary Search Tree (BST), return the minimum difference between
 * the values of any two different nodes in the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var minDiffInBST = function(root) {
  let result = Infinity;
```

```
  let previous = null;

  inorder(root);
  return result;

  function inorder(node) {
    if (!node) return;
    inorder(node.left);
    result = previous === null ? result : Math.min(result, node.val - previous);
    previous = node.val;
    inorder(node.right);
  }
};
```

0784 Letter Case Permutation

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 784. Letter Case Permutation
 * https://leetcode.com/problems/letter-case-permutation/
 * Difficulty: Medium
 *
 * Given a string s, you can transform every letter individually to be lowercase or uppercase
 * to create another string.
 *
 * Return a list of all possible strings we could create. Return the output in any order.
 */

/**
 * @param {string} str
 * @return {string[]}
 */
var letterCasePermutation = function(str) {
  const result = [];
  backtrack(result, str);
  return result;
};

function backtrack(result, input, permutation = '', offset = 0) {
  if (input.length === permutation.length) {
    result.push(permutation);
  } else {
    const target = input[offset];
    if (isNaN(target)) {
      [target.toLowerCase(), target.toUpperCase()].forEach(s => {
        backtrack(result, input, permutation + s, offset + 1);
      });
    } else {
      backtrack(result, input, permutation + target, offset + 1);
    }
  }
}
```

0785 Is Graph Bipartite

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 785. Is Graph Bipartite?
 * https://leetcode.com/problems/is-graph-bipartite/
 * Difficulty: Medium
 *
 * There is an undirected graph with n nodes, where each node is numbered between 0 and n - 1.
 * You are given a 2D array graph, where graph[u] is an array of nodes that node u is adjacent
```

```
 * to. More formally, for each v in graph[u], there is an undirected edge between node u and
 * node v. The graph has the following properties:
 * - There are no self-edges (graph[u] does not contain u).
 * - There are no parallel edges (graph[u] does not contain duplicate values).
 * - If v is in graph[u], then u is in graph[v] (the graph is undirected).
 * - The graph may not be connected, meaning there may be two nodes u and v such that there is
 *   no path between them.
 *
 * A graph is bipartite if the nodes can be partitioned into two independent sets A and B such
 * that every edge in the graph connects a node in set A and a node in set B.
 *
 * Return true if and only if it is bipartite.
 */

/**
 * @param {number[][]} graph
 * @return {boolean}
 */
var isBipartite = function(graph) {
  const n = graph.length;
  const colors = new Array(n).fill(-1);

  for (let i = 0; i < n; i++) {
    if (colors[i] === -1 && !colorGraph(i, 0, graph, colors)) {
      return false;
    }
  }

  return true;
};

function colorGraph(start, color, graph, colors) {
  const queue = [start];
  colors[start] = color;

  while (queue.length > 0) {
    const node = queue.shift();
    const nextColor = 1 - colors[node];

    for (const neighbor of graph[node]) {
      if (colors[neighbor] === -1) {
        colors[neighbor] = nextColor;
        queue.push(neighbor);
      } else if (colors[neighbor] !== nextColor) {
        return false;
      }
    }
  }

  return true;
}
```

0786 K Th Smallest Prime Fraction

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 786. K-th Smallest Prime Fraction
 * https://leetcode.com/problems/k-th-smallest-prime-fraction/
 * Difficulty: Medium
 *
 * You are given a sorted integer array arr containing 1 and prime numbers, where all the
integers
 * of arr are unique. You are also given an integer k.
 *
 * For every i and j where 0 <= i < j < arr.length, we consider the fraction arr[i] / arr[j].
 *
 * Return the kth smallest fraction considered. Return your answer as an array of integers of
size
 * 2, where answer[0] == arr[i] and answer[1] == arr[j].
```

```
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number[]}
 */
var kthSmallestPrimeFraction = function(arr, k) {
  const n = arr.length;
  let left = 0;
  let right = 1;

  while (left < right) {
    const mid = (left + right) / 2;
    let count = 0;
    let maxFraction = [0, 1];
    let j = 1;

    for (let i = 0; i < n - 1; i++) {
      while (j < n && arr[i] > mid * arr[j]) {
        j++;
      }

      count += n - j;

      if (j < n && arr[i] * maxFraction[1] > maxFraction[0] * arr[j]) {
        maxFraction = [arr[i], arr[j]];
      }
    }

    if (count === k) {
      return maxFraction;
    } else if (count < k) {
      left = mid;
    } else {
      right = mid;
    }
  }

  return [];
};
```

0787 Cheapest Flights Within K Stops

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 787. Cheapest Flights Within K Stops
 * https://leetcode.com/problems/cheapest-flights-within-k-stops/
 * Difficulty: Medium
 *
 * There are n cities connected by some number of flights. You are given an array flights
where
 * flights[i] = [fromi, toi, pricei] indicates that there is a flight from city fromi to city
 * toi with cost pricei.
 *
 * You are also given three integers src, dst, and k, return the cheapest price from src to
dst
 * with at most k stops. If there is no such route, return -1.
 */

/**
 * @param {number} n
 * @param {number[][]} flights
 * @param {number} src
 * @param {number} dst
 * @param {number} k
 * @return {number}
 */
var findCheapestPrice = function(n, flights, src, dst, k) {
```

```
  const MAX_INT = Number.MAX_SAFE_INTEGER;
  let prices = new Array(n).fill(MAX_INT);
  prices[src] = 0;

  for (let i = 0; i <= k; i++) {
    const tempPrices = [...prices];

    for (const [from, to, price] of flights) {
      if (prices[from] === MAX_INT) continue;

      const newPrice = prices[from] + price;
      if (newPrice < tempPrices[to]) {
        tempPrices[to] = newPrice;
      }
    }

    prices = tempPrices;
  }

  return prices[dst] === MAX_INT ? -1 : prices[dst];
};
```

0788 Rotated Digits

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 788. Rotated Digits
 * https://leetcode.com/problems/rotated-digits/
 * Difficulty: Medium
 *
 * An integer x is a good if after rotating each digit individually by 180 degrees, we get a
valid
 * number that is different from x. Each digit must be rotated - we cannot choose to leave it
alone.
 *
 * A number is valid if each digit remains a digit after rotation. For example:
 * - 0, 1, and 8 rotate to themselves,
 * - 2 and 5 rotate to each other (in this case they are rotated in a different direction, in
other
 *   words, 2 or 5 gets mirrored),
 * - 6 and 9 rotate to each other, and
 * - the rest of the numbers do not rotate to any other number and become invalid.
 *
 * Given an integer n, return the number of good integers in the range [1, n].
 */

/**
 * @param {number} n
 * @return {number}
 */
var rotatedDigits = function(n) {
  let result = 0;

  for (let i = 1; i <= n; i++) {
    if (verify(i)) {
      result++;
    }
  }

  return result;

  function verify(num) {
    const digits = num.toString().split('');
    let result = false;

    for (const digit of digits) {
      if (digit === '3' || digit === '4' || digit === '7') {
        return false;
      }
```

```
      if (digit === '2' || digit === '5' || digit === '6' || digit === '9') {
        result = true;
      }
    }

    return result;
  }
};
```

0789 Escape The Ghosts

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 789. Escape The Ghosts
 * https://leetcode.com/problems/escape-the-ghosts/
 * Difficulty: Medium
 *
 * You are playing a simplified PAC-MAN game on an infinite 2-D grid. You start at the point
 * [0, 0], and you are given a destination point target = [xtarget, ytarget] that you are
trying
 * to get to. There are several ghosts on the map with their starting positions given as a 2D
 * array ghosts, where ghosts[i] = [xi, yi] represents the starting position of the ith ghost.
 * All inputs are integral coordinates.
 *
 * Each turn, you and all the ghosts may independently choose to either move 1 unit in any of
 * the four cardinal directions: north, east, south, or west, or stay still. All actions
happen
 * simultaneously.
 *
 * You escape if and only if you can reach the target before any ghost reaches you. If you
reach
 * any square (including the target) at the same time as a ghost, it does not count as an
escape.
 *
 * Return true if it is possible to escape regardless of how the ghosts move, otherwise return
 * false.
 */

/**
 * @param {number[][]} ghosts
 * @param {number[]} target
 * @return {boolean}
 */
var escapeGhosts = function(ghosts, target) {
  const distanceToTarget = calcDistance([0, 0], target);

  for (const ghost of ghosts) {
    const ghostDistance = calcDistance(ghost, target);
    if (ghostDistance <= distanceToTarget) {
      return false;
    }
  }

  return true;
};

function calcDistance(point1, point2) {
  return Math.abs(point1[0] - point2[0]) + Math.abs(point1[1] - point2[1]);
}
```

0790 Domino And Tromino Tiling

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 790. Domino and Tromino Tiling
 * https://leetcode.com/problems/domino-and-tromino-tiling/
 * Difficulty: Medium
 *
 * You have two types of tiles: a 2 x 1 domino shape and a tromino shape. You may rotate
 * these shapes.
 *
 * Given an integer n, return the number of ways to tile an 2 x n board. Since the answer
 * may be very large, return it modulo 109 + 7.
 *
 * In a tiling, every square must be covered by a tile. Two tilings are different if and
 * only if there are two 4-directionally adjacent cells on the board such that exactly
 * one of the tilings has both squares occupied by a tile.
 */

/**
 * @param {number} n
 * @return {number}
 */
var numTilings = function(n) {
  const MOD_VALUE = Math.pow(10, 9) + 7;
  const dp = { 1: 1, 2: 2, 3: 5 };
  for (let i = 4; i <= n; i++) {
    dp[i] = (2 * dp[i - 1] + dp[i - 3]) % MOD_VALUE;
  }
  return dp[n];
};
```

0791 Custom Sort String

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 791. Custom Sort String
 * https://leetcode.com/problems/custom-sort-string/
 * Difficulty: Medium
 *
 * S and T are strings composed of lowercase letters. In S,
 * no letter occurs more than once.
 *
 * S was sorted in some custom order previously. We want to
 * permute the characters of T so that they match the order
 * that S was sorted. More specifically, if x occurs before
 * y in S, then x should occur before y in the returned string.
 *
 * Return any permutation of T (as a string) that satisfies
 * this property.
 */

/**
 * @param {string} S
 * @param {string} T
 * @return {string}
 */
var customSortString = function(S, T) {
  const order = S.split('').reduce((o, k, i) => ({...o, [k]: i}), {});
  return T.split('').sort((a, b) => (order[a] || 0) - (order[b] || 0)).join('');
};
```

0792 Number Of Matching Subsequences

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 792. Number of Matching Subsequences
 * https://leetcode.com/problems/number-of-matching-subsequences/
 * Difficulty: Medium
 *
 * Given a string s and an array of strings words, return the number of words[i] that is a
 * subsequence of s.
 *
 * A subsequence of a string is a new string generated from the original string with some
 * characters (can be none) deleted without changing the relative order of the remaining
 * characters.
 *
 * For example, "ace" is a subsequence of "abcde".
 */

/**
 * @param {string} s
 * @param {string[]} words
 * @return {number}
 */
var numMatchingSubseq = function(s, words) {
  const charMap = new Map();

  for (let i = 0; i < 26; i++) {
    const char = String.fromCharCode(97 + i);
    charMap.set(char, []);
  }

  for (const word of words) {
    const firstChar = word[0];
    const wordInfo = { word, index: 0 };
    charMap.get(firstChar).push(wordInfo);
  }

  let count = 0;

  for (const char of s) {
    const bucket = charMap.get(char);
    charMap.set(char, []);

    for (const wordInfo of bucket) {
      wordInfo.index++;

      if (wordInfo.index === wordInfo.word.length) {
        count++;
      } else {
        const nextChar = wordInfo.word[wordInfo.index];
        charMap.get(nextChar).push(wordInfo);
      }
    }
  }

  return count;
};
```

0793 Preimage Size Of Factorial Zeroes Function

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 793. Preimage Size of Factorial Zeroes Function
 * https://leetcode.com/problems/preimage-size-of-factorial-zeroes-function/
 * Difficulty: Hard
 *
 * Let f(x) be the number of zeroes at the end of x!. Recall that x! = 1 * 2 * 3 * ... * x
 * and by convention, 0! = 1.
 *
 * For example, f(3) = 0 because 3! = 6 has no zeroes at the end, while f(11) = 2 because
 * 11! = 39916800 has two zeroes at the end.
```

```
 *
 * Given an integer k, return the number of non-negative integers x have the property that
f(x) = k.
 */

/**
 * @param {number} k
 * @return {number}
 */
var preimageSizeFZF = function(k) {
  return findUpperBound(k) - findLowerBound(k);
};

function findLowerBound(k) {
  let left = 0;
  let right = 5 * (10 ** 10);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    const zeroes = countTrailingZeroes(mid);

    if (zeroes < k) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
}

function findUpperBound(k) {
  let left = 0;
  let right = 5 * (10 ** 10);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    const zeroes = countTrailingZeroes(mid);

    if (zeroes <= k) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
}

function countTrailingZeroes(n) {
  let count = 0;
  let powerOfFive = 5;

  while (n >= powerOfFive) {
    count += Math.floor(n / powerOfFive);
    powerOfFive *= 5;
  }

  return count;
}
```

0794 Valid Tic Tac Toe State

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 794. Valid Tic-Tac-Toe State
 * https://leetcode.com/problems/valid-tic-tac-toe-state/
 * Difficulty: Medium
 *
```

```
 * Given a Tic-Tac-Toe board as a string array board, return true if and only if it is
possible
 * to reach this board position during the course of a valid tic-tac-toe game.
 *
 * The board is a 3 x 3 array that consists of characters ' ', 'X', and 'O'. The ' ' character
 * represents an empty square.
 *
 * Here are the rules of Tic-Tac-Toe:
 * - Players take turns placing characters into empty squares ' '.
 * - The first player always places 'X' characters, while the second player always places 'O'
 *   characters.
 * - 'X' and 'O' characters are always placed into empty squares, never filled ones.
 * - The game ends when there are three of the same (non-empty) character filling any row,
 *   column, or diagonal.
 * - The game also ends if all squares are non-empty.
 * - No more moves can be played if the game is over.
 */

/**
 * @param {string[]} board
 * @return {boolean}
 */
var validTicTacToe = function(board) {
  let xCount = 0;
  let oCount = 0;

  for (const row of board) {
    for (const cell of row) {
      if (cell === 'X') xCount++;
      if (cell === 'O') oCount++;
    }
  }

  if (xCount !== oCount && xCount !== oCount + 1) return false;

  const hasWon = player => {
    for (let i = 0; i < 3; i++) {
      if (board[i] === player.repeat(3)) return true;
      if (board[0][i] === player && board[1][i] === player && board[2][i] === player) return
true;
    }

    return (board[0][0] === player && board[1][1] === player && board[2][2] === player)
      || (board[0][2] === player && board[1][1] === player && board[2][0] === player);
  };

  const xWin = hasWon('X');
  const oWin = hasWon('O');

  if (xWin && xCount !== oCount + 1) return false;
  if (oWin && xCount !== oCount) return false;
  if (xWin && oWin) return false;

  return true;
};
```

0795 Number Of Subarrays With Bounded Maximum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 795. Number of Subarrays with Bounded Maximum
 * https://leetcode.com/problems/number-of-subarrays-with-bounded-maximum/
 * Difficulty: Medium
 *
 * Given an integer array nums and two integers left and right, return the number of
contiguous
 * non-empty subarrays such that the value of the maximum array element in that subarray is in
 * the range [left, right].
 *
```

```
 * The test cases are generated so that the answer will fit in a 32-bit integer.
 */

/**
 * @param {number[]} nums
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
var numSubarrayBoundedMax = function(nums, left, right) {
  let result = 0;
  let validCount = 0;
  let prevInvalidGreater = -1;

  for (let i = 0; i < nums.length; i++) {
    if (nums[i] > right) {
      validCount = 0;
      prevInvalidGreater = i;
    } else if (nums[i] >= left) {
      validCount = i - prevInvalidGreater;
    }

    result += validCount;
  }

  return result;
};
```

0796 Rotate String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 796. Rotate String
 * https://leetcode.com/problems/rotate-string/
 * Difficulty: Easy
 *
 * Given two strings s and goal, return true if and only if s can become goal
 * after some number of shifts on s.
 *
 * A shift on s consists of moving the leftmost character of s to the rightmost
 * position.
 *
 * For example, if s = "abcde", then it will be "bcdea" after one shift.
 */

/**
 * @param {string} s
 * @param {string} goal
 * @return {boolean}
 */
var rotateString = function(s, goal) {
  for (let i = 0; i < s.length; i++) {
    if (s.slice(i) + s.slice(0, i) === goal) return true;
  }
  return false;
};
```

0797 All Paths From Source To Target

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 797. All Paths From Source to Target
 * https://leetcode.com/problems/all-paths-from-source-to-target/
```

```
 * Difficulty: Medium
 *
 * Given a directed acyclic graph (DAG) of n nodes labeled from 0 to n - 1, find all possible
paths
 * from node 0 to node n - 1 and return them in any order.
 *
 * The graph is given as follows: graph[i] is a list of all nodes you can visit from node i
(i.e.,
 * there is a directed edge from node i to node graph[i][j]).
 */

/**
 * @param {number[][]} graph
 * @return {number[][]}
 */
var allPathsSourceTarget = function(graph) {
  const target = graph.length - 1;
  const paths = [];

  const dfs = (node, path) => {
    if (node === target) {
      paths.push([...path]);
      return;
    }

    for (const neighbor of graph[node]) {
      path.push(neighbor);
      dfs(neighbor, path);
      path.pop();
    }
  };

  dfs(0, [0]);
  return paths;
};
```

0798 Smallest Rotation With Highest Score

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 798. Smallest Rotation with Highest Score
 * https://leetcode.com/problems/smallest-rotation-with-highest-score/
 * Difficulty: Hard
 *
 * You are given an array nums. You can rotate it by a non-negative integer k so that the
array
 * becomes [nums[k], nums[k + 1], ... nums[nums.length - 1], nums[0], nums[1], ...,
nums[k-1]].
 * Afterward, any entries that are less than or equal to their index are worth one point.
 *
 * For example, if we have nums = [2,4,1,3,0], and we rotate by k = 2, it becomes [1,3,0,2,4].
 * This is worth 3 points because 1 > 0 [no points], 3 > 1 [no points], 0 <= 2 [one point],
 * 2 <= 3 [one point], 4 <= 4 [one point].
 *
 * Return the rotation index k that corresponds to the highest score we can achieve if we
rotated
 * nums by it. If there are multiple answers, return the smallest such index k.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var bestRotation = function(nums) {
  const n = nums.length;
  const changes = new Array(n).fill(0);

  for (let i = 0; i < n; i++) {
    const lowIndex = (i + 1) % n;
```

```
    const highIndex = (i - nums[i] + 1 + n) % n;

    changes[lowIndex]++;
    changes[highIndex]--;

    if (lowIndex > highIndex) {
      changes[0]++;
    }
  }

  let maxScore = 0;
  let maxIndex = 0;
  let currentScore = 0;

  for (let i = 0; i < n; i++) {
    currentScore += changes[i];
    if (currentScore > maxScore) {
      maxScore = currentScore;
      maxIndex = i;
    }
  }

  return maxIndex;
};
```

0799 Champagne Tower

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 799. Champagne Tower
 * https://leetcode.com/problems/champagne-tower/
 * Difficulty: Medium
 *
 * We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2
glasses, and
 * so on until the 100th row. Each glass holds one cup of champagne.
 *
 * Then, some champagne is poured into the first glass at the top. When the topmost glass is
full,
 * any excess liquid poured will fall equally to the glass immediately to the left and right
of it.
 * When those glasses become full, any excess champagne will fall equally to the left and
right of
 * those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the
 * floor.)
 *
 * For example, after one cup of champagne is poured, the top most glass is full. After two
cups
 * of champagne are poured, the two glasses on the second row are half full. After three cups
of
 * champagne are poured, those two cups become full - there are 3 full glasses total now.
After
 * four cups of champagne are poured, the third row has the middle glass half full, and the
two
 * outside glasses are a quarter full, as pictured below.
 */

/**
 * @param {number} poured
 * @param {number} queryRow
 * @param {number} queryGlass
 * @return {number}
 */
var champagneTower = function(poured, queryRow, queryGlass) {
  const tower = new Array(101).fill().map(() => new Array(101).fill(0));
  tower[0][0] = poured;

  for (let row = 0; row <= queryRow; row++) {
    for (let glass = 0; glass <= row; glass++) {
```

```
      const excess = (tower[row][glass] - 1) / 2;

      if (excess > 0) {
        tower[row + 1][glass] += excess;
        tower[row + 1][glass + 1] += excess;
      }
    }
  }

  return Math.min(1, tower[queryRow][queryGlass]);
};
```

0800 Similar Rgb Color

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 800. Similar RGB Color
 * https://leetcode.com/problems/similar-rgb-color/
 * Difficulty: Easy
 *
 * The red-green-blue color "#AABBCC" can be written as "#ABC" in shorthand.
 * - For example, "#15c" is shorthand for the color "#1155cc".
 *
 * The similarity between the two colors "#ABCDEF" and "#UVWXYZ"
 * is -(AB - UV)2 - (CD - WX)2 - (EF - YZ)2.
 *
 * Given a string color that follows the format "#ABCDEF", return a string represents
 * the color that is most similar to the given color and has a shorthand (i.e., it can
 * be represented as some "#XYZ").
 *
 * Any answer which has the same highest similarity as the best answer will be accepted.
 */

/**
 * @param {string} color
 * @return {string}
 */
var similarRGB = function(color) {
  const shorthandValues = [
    '00', '11', '22', '33', '44', '55', '66', '77',
    '88', '99', 'aa', 'bb', 'cc', 'dd', 'ee', 'ff'
  ];

  const red = helper(color.slice(1, 3));
  const green = helper(color.slice(3, 5));
  const blue = helper(color.slice(5, 7));

  return `#${red}${green}${blue}`;

  function helper(hex) {
    const value = parseInt(hex, 16);
    let minDiff = Infinity;
    let closest = '';

    for (const shorthand of shorthandValues) {
      const shorthandValue = parseInt(shorthand, 16);
      const diff = Math.abs(value - shorthandValue);
      if (diff < minDiff) {
        minDiff = diff;
        closest = shorthand;
      }
    }

    return closest;
  }
};
```

0801 Minimum Swaps To Make Sequences Increasing

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 801. Minimum Swaps To Make Sequences Increasing
 * https://leetcode.com/problems/minimum-swaps-to-make-sequences-increasing/
 * Difficulty: Hard
 *
 * You are given two integer arrays of the same length nums1 and nums2. In one operation, you
are
 * allowed to swap nums1[i] with nums2[i].
 *
 * For example, if nums1 = [1,2,3,8], and nums2 = [5,6,7,4], you can swap the element at i = 3
 to
 * obtain nums1 = [1,2,3,4] and nums2 = [5,6,7,8].
 *
 * Return the minimum number of needed operations to make nums1 and nums2 strictly increasing.
 The
 * test cases are generated so that the given input always makes it possible.
 *
 * An array arr is strictly increasing if and only if
 * arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1].
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minSwap = function(nums1, nums2) {
  let noSwap = 0;
  let swap = 1;

  for (let i = 1; i < nums1.length; i++) {
    let nextNoSwap = Infinity;
    let nextSwap = Infinity;

    const canKeepBoth = nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1];
    const canSwapBoth = nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1];

    if (canKeepBoth) {
      nextNoSwap = Math.min(nextNoSwap, noSwap);
      nextSwap = Math.min(nextSwap, swap + 1);
    }

    if (canSwapBoth) {
      nextNoSwap = Math.min(nextNoSwap, swap);
      nextSwap = Math.min(nextSwap, noSwap + 1);
    }

    noSwap = nextNoSwap;
    swap = nextSwap;
  }

  return Math.min(noSwap, swap);
};
```

0802 Find Eventual Safe States

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 802. Find Eventual Safe States
 * https://leetcode.com/problems/find-eventual-safe-states/
```

```
 * Difficulty: Medium
 *
 * There is a directed graph of n nodes with each node labeled from 0 to n - 1. The graph is
 * represented by a 0-indexed 2D integer array graph where graph[i] is an integer array of
 * nodes adjacent to node i, meaning there is an edge from node i to each node in graph[i].
 *
 * A node is a terminal node if there are no outgoing edges. A node is a safe node if every
 * possible path starting from that node leads to a terminal node (or another safe node).
 *
 * Return an array containing all the safe nodes of the graph. The answer should be sorted
 * in ascending order.
 */

/**
 * @param {number[][]} graph
 * @return {number[]}
 */
var eventualSafeNodes = function(graph) {
  const nodes = graph.map(() => []);
  const totals = graph.map(n => n.length);
  const queue = totals.map((v, i) => v ? -1 : i).filter(v => v >= 0);
  const result = [];

  graph.forEach((n, i) => n.forEach(v => nodes[v].push(i)));

  while (queue.length) {
    const node = queue.shift();
    result.push(node);
    nodes[node].forEach(v => !--totals[v] && queue.push(v));
  }

  return result.sort((a, b) => a - b);
};
```

0803 Bricks Falling When Hit

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 803. Bricks Falling When Hit
 * https://leetcode.com/problems/bricks-falling-when-hit/
 * Difficulty: Hard
 *
 * You are given an m x n binary grid, where each 1 represents a brick and 0 represents an
empty
 * space. A brick is stable if:
 * - It is directly connected to the top of the grid, or
 * - At least one other brick in its four adjacent cells is stable.
 *
 * You are also given an array hits, which is a sequence of erasures we want to apply. Each
time
 * we want to erase the brick at the location hits[i] = (rowi, coli). The brick on that
location
 * (if it exists) will disappear. Some other bricks may no longer be stable because of that
erasure
 * and will fall. Once a brick falls, it is immediately erased from the grid (i.e., it does
not
 * land on other stable bricks).
 *
 * Return an array result, where each result[i] is the number of bricks that will fall after
the
 * ith erasure is applied.
 *
 * Note that an erasure may refer to a location with no brick, and if it does, no bricks drop.
 */

/**
 * @param {number[][]} grid
 * @param {number[][]} hits
 * @return {number[]}
```

```
 */
var hitBricks = function(grid, hits) {
  const rows = grid.length;
  const cols = grid[0].length;
  const directions = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  const fallenBricks = new Array(hits.length).fill(0);

  hits.forEach(([x, y]) => {
    if (grid[x][y] === 1) grid[x][y] = 2;
  });

  for (let col = 0; col < cols; col++) markStableBricks(0, col, grid);

  for (let i = hits.length - 1; i >= 0; i--) {
    const [x, y] = hits[i];
    if (grid[x][y] !== 2) continue;

    grid[x][y] = 1;
    if (hasStableNeighbor(x, y, grid)) {
      fallenBricks[i] = markStableBricks(x, y, grid) - 1;
    }
  }

  return fallenBricks;

  function markStableBricks(x, y, grid) {
    if (x < 0 || x >= rows || y < 0 || y >= cols || grid[x][y] !== 1) return 0;

    grid[x][y] = 3;
    return 1 + directions.reduce((count, [dx, dy]) => {
      return count + markStableBricks(x + dx, y + dy, grid);
    }, 0);
  }

  function hasStableNeighbor(x, y, grid) {
    if (x === 0) return true;

    return directions.some(([dx, dy]) => {
      const newX = x + dx;
      const newY = y + dy;
      return newX >= 0 && newX < rows && newY >= 0 && newY < cols && grid[newX][newY] === 3;
    });
  }
};
```

0804 Unique Morse Code Words

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 804. Unique Morse Code Words
 * https://leetcode.com/problems/unique-morse-code-words/
 * Difficulty: Easy
 *
 * International Morse Code defines a standard encoding where
 * each letter is mapped to a series of dots and dashes, as
 * follows: "a" maps to ".-", "b" maps to "-...",
 * "c" maps to "-.-.", and so on.
 *
 * For convenience, the full table for the 26 letters of the
 * English alphabet is given below:
 *
 * [".-","-...","-.-.","-..",".","..-.","--.","....","..",
 * ".---","-.-",".-..","--","-.","---",".--.","--.-",".-.",
 * "...","-","..-","...-",".--","-..-","-.--","--.."]
 *
 * Now, given a list of words, each word can be written as a
 * concatenation of the Morse code of each letter.
 * For example, "cba" can be written as "-.-..--...",
 * (which is the concatenation "-.-." + "-..." + ".-").
```

```
 * We'll call such a concatenation, the transformation of a word.
 *
 * Return the number of different transformations
 * among all words we have.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var uniqueMorseRepresentations = function(words) {
  const codes = [".-","-...","-.-.","-..",".","..-.","--.","....","..",".---",
                 "-.-",".-..","--","-.","---",".--.","--.-",".-.","...","-",
                 "..-","...-",".--","-..-","-.--","--.."];
  const map = new Map(codes.map((v, i) => [String.fromCharCode(97 + i), v]));
  const transformed = words.map(word => word.split('').map(c => map.get(c)).join(''));
  return new Set(transformed).size;
};
```

0805 Split Array With Same Average

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 805. Split Array With Same Average
 * https://leetcode.com/problems/split-array-with-same-average/
 * Difficulty: Hard
 *
 * You are given an integer array nums.
 *
 * You should move each element of nums into one of the two arrays A and B such that A and B
are
 * non-empty, and average(A) == average(B).
 *
 * Return true if it is possible to achieve that and false otherwise.
 *
 * Note that for an array arr, average(arr) is the sum of all the elements of arr over the
length
 * of arr.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var splitArraySameAverage = function(nums) {
  if (nums.length <= 1) return false;
  const sum = nums.reduce((acc, num) => acc + num, 0);
  const dp = Array(Math.floor(nums.length / 2) + 1).fill().map(() => new Set());
  dp[0].add(0);

  for (const num of nums) {
    for (let size = Math.min(Math.floor(nums.length / 2), dp.length - 1); size > 0; size--) {
      for (const prevSum of dp[size - 1]) {
        dp[size].add(prevSum + num);
      }
    }
  }

  for (let size = 1; size <= Math.floor(nums.length / 2); size++) {
    const targetSum = sum * size / nums.length;

    if (targetSum % 1 === 0 && dp[size].has(targetSum)) {
      return true;
    }
  }

  return false;
};
```

0806 Number Of Lines To Write String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 806. Number of Lines To Write String
 * https://leetcode.com/problems/number-of-lines-to-write-string/
 * Difficulty: Easy
 *
 * You are given a string s of lowercase English letters and an array widths denoting how many
 * pixels wide each lowercase English letter is. Specifically, widths[0] is the width of 'a',
 * widths[1] is the width of 'b', and so on.
 *
 * You are trying to write s across several lines, where each line is no longer than 100
pixels.
 * Starting at the beginning of s, write as many letters on the first line such that the total
 * width does not exceed 100 pixels. Then, from where you stopped in s, continue writing as
many
 * letters as you can on the second line. Continue this process until you have written all of
s.
 *
 * Return an array result of length 2 where:
 * - result[0] is the total number of lines.
 * - result[1] is the width of the last line in pixels.
 */

/**
 * @param {number[]} widths
 * @param {string} s
 * @return {number[]}
 */
var numberOfLines = function(widths, s) {
  let lines = 1;
  let width = 0;

  for (const char of s) {
    const charWidth = widths[char.charCodeAt(0) - 97];

    if (width + charWidth > 100) {
      lines++;
      width = charWidth;
    } else {
      width += charWidth;
    }
  }

  return [lines, width];
};
```

0807 Max Increase To Keep City Skyline

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 807. Max Increase to Keep City Skyline
 * https://leetcode.com/problems/max-increase-to-keep-city-skyline/
 * Difficulty: Medium
 *
 * There is a city composed of n x n blocks, where each block contains a single building
shaped
 * like a vertical square prism. You are given a 0-indexed n x n integer matrix grid where
 * grid[r][c] represents the height of the building located in the block at row r and column
c.
 *
 * A city's skyline is the outer contour formed by all the building when viewing the side of
```

```
the
 * city from a distance. The skyline from each cardinal direction north, east, south, and west
 * may be different.
 *
 * We are allowed to increase the height of any number of buildings by any amount (the amount
can
 * be different per building). The height of a 0-height building can also be increased.
However,
 * increasing the height of a building should not affect the city's skyline from any cardinal
 * direction.
 *
 * Return the maximum total sum that the height of the buildings can be increased by without
 * changing the city's skyline from any cardinal direction.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maxIncreaseKeepingSkyline = function(grid) {
  let result = 0;
  const rowMaxes = grid.map(row => Math.max(...row));
  const columnMaxes = grid.map((row, index) => grid.map(row => row[index]))
    .map(row => Math.max(...row));

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[i].length; j++) {
      const currentHeight = grid[i][j];
      const minHeight = Math.min(
        Math.max(rowMaxes[i], currentHeight),
        Math.max(columnMaxes[j], currentHeight)
      );

      result += (minHeight - currentHeight);
    }
  }

  return result;
};
```

0808 Soup Servings

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 808. Soup Servings
 * https://leetcode.com/problems/soup-servings/
 * Difficulty: Medium
 *
 * There are two types of soup: type A and type B. Initially, we have n ml of each type of
soup.
 * There are four kinds of operations:
 * 1. Serve 100 ml of soup A and 0 ml of soup B,
 * 2. Serve 75 ml of soup A and 25 ml of soup B,
 * 3. Serve 50 ml of soup A and 50 ml of soup B, and
 * 4. Serve 25 ml of soup A and 75 ml of soup B.
 *
 * When we serve some soup, we give it to someone, and we no longer have it. Each turn, we
will
 * choose from the four operations with an equal probability 0.25. If the remaining volume of
 * soup is not enough to complete the operation, we will serve as much as possible. We stop
once
 * we no longer have some quantity of both types of soup.
 *
 * Note that we do not have an operation where all 100 ml's of soup B are used first.
 *
 * Return the probability that soup A will be empty first, plus half the probability that A
and B
 * become empty at the same time. Answers within 10-5 of the actual answer will be accepted.
 */
```

```
/**
 * @param {number} n
 * @return {number}
 */
var soupServings = function(n) {
  if (n >= 4800) return 1.0;

  n = Math.ceil(n / 25);
  const memo = new Map();

  function calculateProbability(a, b) {
    if (a <= 0 && b <= 0) return 0.5;
    if (a <= 0) return 1.0;
    if (b <= 0) return 0.0;

    const key = a * 10000 + b;
    if (memo.has(key)) return memo.get(key);

    const probability = 0.25 * (
      calculateProbability(a - 4, b) + calculateProbability(a - 3, b - 1)
      + calculateProbability(a - 2, b - 2) + calculateProbability(a - 1, b - 3)
    );

    memo.set(key, probability);
    return probability;
  }

  return calculateProbability(n, n);
};
```

0809 Expressive Words

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 809. Expressive Words
 * https://leetcode.com/problems/expressive-words/
 * Difficulty: Medium
 *
 * Sometimes people repeat letters to represent extra feeling. For example:
 * - "hello" -> "heeellooo"
 * - "hi" -> "hiiii"
 *
 * In these strings like "heeellooo", we have groups of adjacent letters that are all the
 * same: "h", "eee", "ll", "ooo".
 *
 * You are given a string s and an array of query strings words. A query word is stretchy
 * if it can be made to be equal to s by any number of applications of the following extension
 * operation: choose a group consisting of characters c, and add some number of characters
 * c to the group so that the size of the group is three or more.
 *
 * For example, starting with "hello", we could do an extension on the group "o" to get
 * "hellooo", but we cannot get "helloo" since the group "oo" has a size less than three.
 * Also, we could do another extension like "ll" -> "lllll" to get "helllllooo". If
 * s = "helllllooo", then the query word "hello" would be stretchy because of these two
 * extension operations: query = "hello" -> "hellooo" -> "helllllooo" = s.
 *
 * Return the number of query strings that are stretchy.
 */

/**
 * @param {string} s
 * @param {string[]} words
 * @return {number}
 */
var expressiveWords = function(s, words) {
  let result = 0;

  for (const word of words) {
```

```
    if (verify(s, word)) {
      result++;
    }
  }

  return result;
};

function verify(s, word) {
  let i = 0;
  let j = 0;

  while (i < s.length && j < word.length) {
    if (s[i] !== word[j]) return false;

    const char = s[i];
    let sCount = 0;

    while (i < s.length && s[i] === char) {
      sCount++;
      i++;
    }

    let wordCount = 0;

    while (j < word.length && word[j] === char) {
      wordCount++;
      j++;
    }

    if (wordCount > sCount) return false;
    if (sCount > wordCount && sCount < 3) return false;
  }

  return i === s.length && j === word.length;
}
```

0810 Chalkboard Xor Game

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 810. Chalkboard XOR Game
 * https://leetcode.com/problems/chalkboard-xor-game/
 * Difficulty: Hard
 *
 * You are given an array of integers nums represents the numbers written on a chalkboard.
 *
 * Alice and Bob take turns erasing exactly one number from the chalkboard, with Alice
starting
 * first. If erasing a number causes the bitwise XOR of all the elements of the chalkboard to
 * become 0, then that player loses. The bitwise XOR of one element is that element itself,
and
 * the bitwise XOR of no elements is 0.
 *
 * Also, if any player starts their turn with the bitwise XOR of all the elements of the
chalkboard
 * equal to 0, then that player wins.
 *
 * Return true if and only if Alice wins the game, assuming both players play optimally.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var xorGame = function(nums) {
  const total = nums.reduce((xor, num) => xor ^ num, 0);
  if (total === 0) return true;
  return nums.length % 2 === 0;
```

```
};
```

0811 Subdomain Visit Count

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 811. Subdomain Visit Count
 * https://leetcode.com/problems/subdomain-visit-count/
 * Difficulty: Medium
 *
 * A website domain "discuss.leetcode.com" consists of various subdomains. At the top level,
we have
 * "com", at the next level, we have "leetcode.com" and at the lowest level,
"discuss.leetcode.com".
 * When we visit a domain like "discuss.leetcode.com", we will also visit the parent domains
 * "leetcode.com" and "com" implicitly.
 *
 * A count-paired domain is a domain that has one of the two formats "rep d1.d2.d3" or "rep
d1.d2"
 * where rep is the number of visits to the domain and d1.d2.d3 is the domain itself.
 *
 * For example, "9001 discuss.leetcode.com" is a count-paired domain that indicates that
 * discuss.leetcode.com was visited 9001 times.
 *
 * Given an array of count-paired domains cpdomains, return an array of the count-paired
domains of
 * each subdomain in the input. You may return the answer in any order.
 */

/**
 * @param {string[]} cpdomains
 * @return {string[]}
 */
var subdomainVisits = function(cpdomains) {
  const domainCount = new Map();

  for (const cpdomain of cpdomains) {
    const [countStr, domain] = cpdomain.split(' ');
    const count = parseInt(countStr);
    const subdomains = getSubdomains(domain);

    for (const subdomain of subdomains) {
      domainCount.set(subdomain, (domainCount.get(subdomain) || 0) + count);
    }
  }

  const result = [];
  for (const [domain, count] of domainCount) {
    result.push(`${count} ${domain}`);
  }

  return result;
};

function getSubdomains(domain) {
  const parts = domain.split('.');
  const subdomains = [];

  for (let i = 0; i < parts.length; i++) {
    subdomains.push(parts.slice(i).join('.'));
  }

  return subdomains;
}
```

0812 Largest Triangle Area

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 812. Largest Triangle Area
 * https://leetcode.com/problems/largest-triangle-area/
 * Difficulty: Easy
 *
 * Given an array of points on the X-Y plane points where points[i] = [xi, yi], return the
area of
 * the largest triangle that can be formed by any three different points. Answers within 10-5
of
 * the actual answer will be accepted.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var largestTriangleArea = function(points) {
  let maxArea = 0;

  for (let i = 0; i < points.length; i++) {
    for (let j = i + 1; j < points.length; j++) {
      for (let k = j + 1; k < points.length; k++) {
        const area = calculateTriangleArea(points[i], points[j], points[k]);
        maxArea = Math.max(maxArea, area);
      }
    }
  }

  return maxArea;
};

function calculateTriangleArea(p1, p2, p3) {
  const [[x1, y1], [x2, y2], [x3, y3]] = [p1, p2, p3];
  return Math.abs(
    (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2)) / 2
  );
}
```

0813 Largest Sum Of Averages

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 813. Largest Sum of Averages
 * https://leetcode.com/problems/largest-sum-of-averages/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. You can partition the array into at
most
 * k non-empty adjacent subarrays. The score of a partition is the sum of the averages of each
 * subarray.
 *
 * Note that the partition must use every integer in nums, and that the score is not
necessarily
 * an integer.
 *
 * Return the maximum score you can achieve of all the possible partitions. Answers within
10-6
 * of the actual answer will be accepted.
 */
```

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var largestSumOfAverages = function(nums, k) {
  const prefixSum = new Array(nums.length + 1).fill(0);
  for (let i = 0; i < nums.length; i++) {
    prefixSum[i + 1] = prefixSum[i] + nums[i];
  }

  const dp = new Array(nums.length).fill(0).map(() => new Array(k + 1).fill(0));

  for (let i = 0; i < nums.length; i++) {
    dp[i][1] = calculateAverage(i, nums.length - 1);
  }

  for (let partitions = 2; partitions <= k; partitions++) {
    for (let start = 0; start < nums.length - partitions + 1; start++) {
      for (let end = start; end < nums.length - partitions + 1; end++) {
        dp[start][partitions] = Math.max(
          dp[start][partitions],
          calculateAverage(start, end) + dp[end + 1][partitions - 1]
        );
      }
    }
  }

  return dp[0][k];

  function calculateAverage(i, j) {
    return (prefixSum[j + 1] - prefixSum[i]) / (j - i + 1);
  }
};
```

0814 Binary Tree Pruning

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 814. Binary Tree Pruning
 * https://leetcode.com/problems/binary-tree-pruning/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the same tree where every subtree (of the given
tree)
 * not containing a 1 has been removed.
 *
 * A subtree of a node node is node plus every node that is a descendant of node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var pruneTree = function(root) {
  if (!root) return null;
  root.left = pruneTree(root.left);
  root.right = pruneTree(root.right);
  if (!root.left && !root.right && root.val === 0) {
    return null;
  }
```

```
  return root;
};
```

0815 Bus Routes

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 815. Bus Routes
 * https://leetcode.com/problems/bus-routes/
 * Difficulty: Hard
 *
 * You are given an array routes representing bus routes where routes[i] is a bus route that
 * the ith bus repeats forever.
 *
 * For example, if routes[0] = [1, 5, 7], this means that the 0th bus travels in the sequence
 * 1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ... forever.
 *
 * You will start at the bus stop source (You are not on any bus initially), and you want to
 * go to the bus stop target. You can travel between bus stops by buses only.
 *
 * Return the least number of buses you must take to travel from source to target.
 *
 * Return -1 if it is not possible.
 */


/**
 * @param {number[][]} routes
 * @param {number} source
 * @param {number} target
 * @return {number}
 */
var numBusesToDestination = function(routes, source, target) {
  if (source === target) return 0;

  const stopToBuses = new Map();

  for (let busId = 0; busId < routes.length; busId++) {
    for (const stop of routes[busId]) {
      if (!stopToBuses.has(stop)) {
        stopToBuses.set(stop, []);
      }
      stopToBuses.get(stop).push(busId);
    }
  }
  if (!stopToBuses.has(source) || !stopToBuses.has(target)) return -1;

  const visitedBuses = new Set();
  const visitedStops = new Set([source]);
  const queue = [[source, 0]];

  while (queue.length > 0) {
    const [currentStop, busCount] = queue.shift();
    if (currentStop === target) return busCount;
    for (const busId of stopToBuses.get(currentStop)) {
      if (visitedBuses.has(busId)) continue;
      visitedBuses.add(busId);
      for (const nextStop of routes[busId]) {
        if (visitedStops.has(nextStop)) continue;
        visitedStops.add(nextStop);
        queue.push([nextStop, busCount + 1]);
        if (nextStop === target) return busCount + 1;
      }
    }
  }

  return -1;
};
```

0816 Ambiguous Coordinates

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 816. Ambiguous Coordinates
 * https://leetcode.com/problems/ambiguous-coordinates/
 * Difficulty: Medium
 *
 * We had some 2-dimensional coordinates, like "(1, 3)" or "(2, 0.5)". Then, we removed all
commas,
 * decimal points, and spaces and ended up with the string s.
 * - For example, "(1, 3)" becomes s = "(13)" and "(2, 0.5)" becomes s = "(205)".
 *
 * Return a list of strings representing all possibilities for what our original coordinates
could
 * have been.
 *
 * Our original representation never had extraneous zeroes, so we never started with numbers
like
 * "00", "0.0", "0.00", "1.0", "001", "00.01", or any other number that can be represented
with
 * fewer digits. Also, a decimal point within a number never occurs without at least one digit
 * occurring before it, so we never started with numbers like ".1".
 *
 * The final answer list can be returned in any order. All coordinates in the final answer
have
 * exactly one space between them (occurring after the comma.)
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var ambiguousCoordinates = function(s) {
  const digits = s.slice(1, s.length - 1);
  const result = [];

  for (let i = 1; i < digits.length; i++) {
    const leftPart = digits.substring(0, i);
    const rightPart = digits.substring(i);

    const validLeftCoords = getValidCoordinates(leftPart);
    const validRightCoords = getValidCoordinates(rightPart);

    for (const left of validLeftCoords) {
      for (const right of validRightCoords) {
        result.push(`(${left}, ${right})`);
      }
    }
  }

  return result;
};

function getValidCoordinates(s) {
  const coords = [];

  if (isValidInteger(s)) {
    coords.push(s);
  }

  for (let i = 1; i < s.length; i++) {
    const integerPart = s.substring(0, i);
    const decimalPart = s.substring(i);

    if (isValidInteger(integerPart) && isValidDecimal(decimalPart)) {
      coords.push(`${integerPart}.${decimalPart}`);
    }
  }
```

```
  return coords;
}

function isValidInteger(s) {
  return s === '0' || s[0] !== '0';
}

function isValidDecimal(s) {
  return s[s.length - 1] !== '0';
}
```

0817 Linked List Components

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 817. Linked List Components
 * https://leetcode.com/problems/linked-list-components/
 * Difficulty: Medium
 *
 * You are given the head of a linked list containing unique integer values and an integer
array
 * nums that is a subset of the linked list values.
 *
 * Return the number of connected components in nums where two values are connected if they
appear
 * consecutively in the linked list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number[]} nums
 * @return {number}
 */
var numComponents = function(head, nums) {
  const numSet = new Set(nums);
  let componentCount = 0;
  let inComponent = false;

  let current = head;
  while (current) {
    if (numSet.has(current.val)) {
      if (!inComponent) {
        componentCount++;
        inComponent = true;
      }
    } else {
      inComponent = false;
    }
    current = current.next;
  }

  return componentCount;
};
```

0818 Race Car

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 818. Race Car
 * https://leetcode.com/problems/race-car/
 * Difficulty: Hard
 *
 * Your car starts at position 0 and speed +1 on an infinite number line. Your car can go into
 * negative positions. Your car drives automatically according to a sequence of instructions
 * 'A' (accelerate) and 'R' (reverse):
 * - When you get an instruction 'A', your car does the following:
 *   - position += speed
 *   - speed *= 2
 * - When you get an instruction 'R', your car does the following:
 *   - If your speed is positive then speed = -1
 *   - otherwise speed = 1
 *
 * Your position stays the same.
 *
 * For example, after commands "AAR", your car goes to positions 0 --> 1 --> 3 --> 3, and your
 * speed goes to 1 --> 2 --> 4 --> -1.
 *
 * Given a target position target, return the length of the shortest sequence of instructions
 * to get there.
 */

/**
 * @param {number} target
 * @return {number}
 */
var racecar = function(target) {
  const dp = new Array(target + 1).fill(Infinity);
  dp[0] = 0;

  for (let i = 1; i <= target; i++) {
    const k = Math.ceil(Math.log2(i + 1));

    if ((1 << k) - 1 === i) {
      dp[i] = k;
      continue;
    }

    dp[i] = k + 1 + dp[(1 << k) - 1 - i];

    for (let j = 0; j < k - 1; j++) {
      dp[i] = Math.min(
        dp[i],
        (k - 1) + 1 + j + 1 + dp[i - ((1 << (k - 1)) - 1) + (1 << j) - 1]
      );
    }
  }

  return dp[target];
};
```

0819 Most Common Word

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 819. Most Common Word
 * https://leetcode.com/problems/most-common-word/
 * Difficulty: Easy
 *
 * Given a paragraph and a list of banned words, return the most frequent word that
 * is not in the list of banned words.  It is guaranteed there is at least one word
 * that isn't banned, and that the answer is unique.
 *
 * Words in the list of banned words are given in lowercase, and free of punctuation.
```

```
 * Words in the paragraph are not case sensitive.  The answer is in lowercase.
 */

/**
 * @param {string} paragraph
 * @param {string[]} banned
 * @return {string}
 */
var mostCommonWord = function(paragraph, banned) {
  const occurrences = new Map();
  paragraph.toLowerCase().replace(/[^\w\s]/g, ' ').split(/\s+/).forEach(word => {
    if (!banned.includes(word)) {
      occurrences.set(word, occurrences.has(word) ? occurrences.get(word) + 1 : 1);
    }
  });
  return Array.from(occurrences).sort((a, b) => b[1] - a[1])[0][0];
};
```

0820 Short Encoding Of Words

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 820. Short Encoding of Words
 * https://leetcode.com/problems/short-encoding-of-words/
 * Difficulty: Medium
 *
 * A valid encoding of an array of words is any reference string s and array of indices
 * such that:
 * - words.length == indices.length
 * - The reference string s ends with the '#' character.
 * - For each index indices[i], the substring of s starting from indices[i] and up to
 *   (but not including) the next '#' character is equal to words[i].
 *
 * Given an array of words, return the length of the shortest reference string s possible
 * of any valid encoding of words.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var minimumLengthEncoding = function(words) {
  const uniqueWords = [...new Set(words)];
  const wordSet = new Set(uniqueWords);

  for (const word of uniqueWords) {
    for (let i = 1; i < word.length; i++) {
      const suffix = word.slice(i);
      wordSet.delete(suffix);
    }
  }

  let result = 0;
  for (const word of wordSet) {
    result += word.length + 1;
  }

  return result;
};
```

0821 Shortest Distance To A Character

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 821. Shortest Distance to a Character
 * https://leetcode.com/problems/shortest-distance-to-a-character/
 * Difficulty: Easy
 *
 * Given a string s and a character c that occurs in s, return an array of
 * integers answer where answer.length == s.length and answer[i] is the
 * distance from index i to the closest occurrence of character c in s.
 *
 * The distance between two indices i and j is abs(i - j), where abs is the
 * absolute value function.
 */

/**
 * @param {string} s
 * @param {character} c
 * @return {number[]}
 */
var shortestToChar = function(s, c) {
  const result = [];
  const pointers = [];

  for (let i = 0; i < s.length; i++) {
    if (s[i] === c) pointers.push(i);
  }

  for (let i = 0, j = 0; i < s.length; i++) {
    if (i > pointers[j + 1] && pointers.length - 2 > j) {
      j++;
    }

    const offset = pointers.length > 1
      ? Math.min(Math.abs(pointers[j] - i), Math.abs(pointers[j + 1] - i))
      : Math.abs(pointers[j] - i);

    result.push(offset);
  }

  return result;
};
```

0822 Card Flipping Game

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 822. Card Flipping Game
 * https://leetcode.com/problems/card-flipping-game/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer arrays fronts and backs of length n, where the ith card
 has
 * the positive integer fronts[i] printed on the front and backs[i] printed on the back.
Initially,
 * each card is placed on a table such that the front number is facing up and the other is
facing
 * down. You may flip over any number of cards (possibly zero).
 *
 * After flipping the cards, an integer is considered good if it is facing down on some card
and
 * not facing up on any card.
 *
 * Return the minimum possible good integer after flipping the cards. If there are no good
integers,
 * return 0.
 */

/**
 * @param {number[]} fronts
 * @param {number[]} backs
```

```
 * @return {number}
 */
var flipgame = function(fronts, backs) {
  const impossibleValues = new Set();
  const n = fronts.length;

  for (let i = 0; i < n; i++) {
    if (fronts[i] === backs[i]) {
      impossibleValues.add(fronts[i]);
    }
  }

  let minGoodValue = Infinity;

  for (let i = 0; i < n; i++) {
    if (!impossibleValues.has(fronts[i])) {
      minGoodValue = Math.min(minGoodValue, fronts[i]);
    }

    if (!impossibleValues.has(backs[i])) {
      minGoodValue = Math.min(minGoodValue, backs[i]);
    }
  }

  return minGoodValue === Infinity ? 0 : minGoodValue;
};
```

0823 Binary Trees With Factors

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 823. Binary Trees With Factors
 * https://leetcode.com/problems/binary-trees-with-factors/
 * Difficulty: Medium
 *
 * Given an array of unique integers, arr, where each integer arr[i] is strictly greater than
1.
 *
 * We make a binary tree using these integers, and each number may be used for any number of
times.
 * Each non-leaf node's value should be equal to the product of the values of its children.
 *
 * Return the number of binary trees we can make. The answer may be too large so return the
answer
 * modulo 109 + 7.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var numFactoredBinaryTrees = function(arr) {
  const MOD = 1e9 + 7;
  const dp = {};
  const numMap = new Map();

  arr.sort((a, b) => a - b);
  for (let i = 0; i < arr.length; i++) {
    numMap.set(arr[i], i);
  }

  let result = 0;
  for (let i = 0; i < arr.length; i++) {
    dp[arr[i]] = 1;
    for (let j = 0; j < i; j++) {
      if (arr[i] % arr[j] === 0) {
        const complement = arr[i] / arr[j];
        if (numMap.has(complement)) {
```

```
          dp[arr[i]] = (dp[arr[i]] + dp[arr[j]] * dp[complement]) % MOD;
        }
      }
    }
    result = (result + dp[arr[i]]) % MOD;
  }

  return result;
};
```

0824 Goat Latin

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 824. Goat Latin
 * https://leetcode.com/problems/goat-latin/
 * Difficulty: Easy
 *
 * A sentence S is given, composed of words separated by spaces.
 * Each word consists of lowercase and uppercase letters only.
 *
 * We would like to convert the sentence to "Goat Latin"
 * (a made-up language similar to Pig Latin.)
 *
 * The rules of Goat Latin are as follows:
 * - If a word begins with a vowel (a, e, i, o, or u), append "ma" to the
 *   end of the word. For example, the word 'apple' becomes 'applema'.
 * - If a word begins with a consonant (i.e. not a vowel), remove the
 *   first letter and append it to the end, then add "ma".
 *   For example, the word "goat" becomes "oatgma".
 * - Add one letter 'a' to the end of each word per its word index in
 *   the sentence, starting with 1.
 *   For example, the first word gets "a" added to the end, the second word gets
 *   "aa" added to the end and so on.
 *
 * Return the final sentence representing the conversion from S to Goat Latin.
 */

/**
 * @param {string} S
 * @return {string}
 */
var toGoatLatin = function(S) {
  return S.split(' ').map((word, index) => {
    return `${word.replace(/^([^aeiou])(.*)/ig, '$2$1')}ma${'a'.repeat(index + 1)}`;
  }).join(' ');
};
```

0825 Friends Of Appropriate Ages

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 825. Friends Of Appropriate Ages
 * https://leetcode.com/problems/friends-of-appropriate-ages/
 * Difficulty: Medium
 *
 * There are n persons on a social media website. You are given an integer array ages where
ages[i]
 * is the age of the ith person.
 *
 * A Person x will not send a friend request to a person y (x != y) if any of the following
 * conditions is true:
 * - age[y] <= 0.5 * age[x] + 7
```

```
 * - age[y] > age[x]
 * - age[y] > 100 && age[x] < 100
 *
 * Otherwise, x will send a friend request to y.
 *
 * Note that if x sends a request to y, y will not necessarily send a request to x. Also, a
person
 * will not send a friend request to themself.
 *
 * Return the total number of friend requests made.
 */

/**
 * @param {number[]} ages
 * @return {number}
 */
var numFriendRequests = function(ages) {
  const ageCount = new Array(121).fill(0);

  for (const age of ages) {
    ageCount[age]++;
  }

  let result = 0;
  for (let ageX = 1; ageX <= 120; ageX++) {
    if (ageCount[ageX] === 0) continue;
    for (let ageY = 1; ageY <= 120; ageY++) {
      if (ageCount[ageY] === 0) continue;
      if (ageY <= 0.5 * ageX + 7) continue;
      if (ageY > ageX) continue;
      if (ageY > 100 && ageX < 100) continue;
      if (ageX === ageY) {
        result += ageCount[ageX] * (ageCount[ageY] - 1);
      } else {
        result += ageCount[ageX] * ageCount[ageY];
      }
    }
  }

  return result;
};
```

0826 Most Profit Assigning Work

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 826. Most Profit Assigning Work
 * https://leetcode.com/problems/most-profit-assigning-work/
 * Difficulty: Medium
 *
 * You have n jobs and m workers. You are given three arrays: difficulty, profit, and worker
where:
 * - difficulty[i] and profit[i] are the difficulty and the profit of the ith job, and
 * - worker[j] is the ability of jth worker (i.e., the jth worker can only complete a job with
 *   difficulty at most worker[j]).
 *
 * Every worker can be assigned at most one job, but one job can be completed multiple times.
 * - For example, if three workers attempt the same job that pays $1, then the total profit
will
 *   be $3. If a worker cannot complete any job, their profit is $0.
 *
 * Return the maximum profit we can achieve after assigning the workers to the jobs.
 */

/**
 * @param {number[]} difficulty
 * @param {number[]} profit
 * @param {number[]} worker
```

```
 * @return {number}
 */
var maxProfitAssignment = function(difficulty, profit, worker) {
  const jobs = difficulty.map((d, i) => [d, profit[i]]);
  jobs.sort((a, b) => a[0] - b[0]);

  const n = jobs.length;
  const bestProfit = new Array(n);
  let maxProfit = 0;

  for (let i = 0; i < n; i++) {
    maxProfit = Math.max(maxProfit, jobs[i][1]);
    bestProfit[i] = [jobs[i][0], maxProfit];
  }

  return worker.reduce((total, ability) => {
    let left = 0;
    let right = n - 1;

    while (left <= right) {
      const mid = (left + right) >> 1;
      bestProfit[mid][0] <= ability ? left = mid + 1 : right = mid - 1;
    }

    return total + (right >= 0 ? bestProfit[right][1] : 0);
  }, 0);
};
```

0827 Making A Large Island

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 827. Making A Large Island
 * https://leetcode.com/problems/making-a-large-island/
 * Difficulty: Hard
 *
 * You are given an n x n binary matrix grid. You are allowed to change at most one 0 to be 1.
 *
 * Return the size of the largest island in grid after applying this operation.
 *
 * An island is a 4-directionally connected group of 1s.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var largestIsland = function(grid) {
  function traverse(tile, grid, i, j) {
    if (i < 0 || j < 0 || i >= grid.length || j >= grid[0].length
        || grid[i][j] === 0 || grid[i][j] === tile) {
      return 0;
    }
    grid[i][j] = tile;
    return 1 + (
      traverse(tile, grid, i + 1, j)
      + traverse(tile, grid, i - 1, j)
      + traverse(tile, grid, i, j + 1)
      + traverse(tile, grid, i, j - 1)
    );
  };

  const map = new Map();
  let tile = 2;
  let result = -1;

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid.length; j++) {
      if (grid[i][j] === 1) {
```

```
        const value = traverse(tile, grid, i, j);
        map.set(tile, value);
        tile += 1;
        result = Math.max(result, value);
      }
    }
  }

  map.set(0, 0);

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid.length; j++) {
      if (!grid[i][j]) {
        const seen = new Set();
        let sum = 0;
        if (i > 0) seen.add(grid[i - 1][j]);
        if (j > 0) seen.add(grid[i][j - 1]);
        if (i < grid.length - 1) seen.add(grid[i + 1][j]);
        if (j < grid.length - 1) seen.add(grid[i][j + 1]);
        seen.forEach(val => sum += map.get(val));
        result = Math.max(result, sum + 1);
      }
    }
  }

  return result;
};
```

0828 Count Unique Characters Of All Substrings Of A Given String

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 828. Count Unique Characters of All Substrings of a Given String
 * https://leetcode.com/problems/count-unique-characters-of-all-substrings-of-a-given-string/
 * Difficulty: Hard
 *
 * Let's define a function countUniqueChars(s) that returns the number of unique characters in
 s.
 * - For example, calling countUniqueChars(s) if s = "LEETCODE" then "L", "T", "C", "O", "D"
are
 *   the unique characters since they appear only once in s, therefore countUniqueChars(s) =
5.
 *
 * Given a string s, return the sum of countUniqueChars(t) where t is a substring of s. The
test
 * cases are generated such that the answer fits in a 32-bit integer.
 *
 * Notice that some substrings can be repeated so in this case you have to count the repeated
 * ones too.
 */

/**
 * @param {string} s
 * @return {number}
 */
var uniqueLetterString = function(s) {
  const n = s.length;
  const lastPositions = {};
  let result = 0;

  for (let i = 0; i < n; i++) {
    const char = s[i];
    if (!lastPositions[char]) lastPositions[char] = [-1, -1];

    const [prevPrev, prev] = lastPositions[char];
    result += (i - prev) * (prev - prevPrev);
    lastPositions[char] = [prev, i];
  }
```

```
  for (const char of Object.keys(lastPositions)) {
    const [prev, pos] = lastPositions[char];
    if (pos >= 0) result += (n - pos) * (pos - prev);
  }

  return result;
};
```

0829 Consecutive Numbers Sum

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 829. Consecutive Numbers Sum
 * https://leetcode.com/problems/consecutive-numbers-sum/
 * Difficulty: Hard
 *
 * Given an integer n, return the number of ways you can write n as the sum of consecutive
 * positive integers.
 */

/**
 * @param {number} n
 * @return {number}
 */
var consecutiveNumbersSum = function(n) {
  let result = 0;

  for (let k = 1; k < Math.sqrt(2 * n); k++) {
    const numerator = n - (k * (k - 1)) / 2;
    if (numerator % k === 0) {
      const startingNum = numerator / k;
      if (startingNum > 0) {
        result++;
      }
    }
  }

  return result;
};
```

0830 Positions Of Large Groups

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 830. Positions of Large Groups
 * https://leetcode.com/problems/positions-of-large-groups/
 * Difficulty: Easy
 *
 * In a string s of lowercase letters, these letters form consecutive groups of the same
character.
 *
 * For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and
"yy".
 *
 * A group is identified by an interval [start, end], where start and end denote the start and
 end
 * indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6].
 *
 * A group is considered large if it has 3 or more characters.
 *
 * Return the intervals of every large group sorted in increasing order by start index.
 */
```

```
/**
 * @param {string} s
 * @return {number[][]}
 */
var largeGroupPositions = function(s) {
  const result = [];
  let start = 0;

  for (let i = 1; i <= s.length; i++) {
    if (s.length === i || s[i] !== s[start]) {
      if (i - start >= 3) {
        result.push([start, i - 1]);
      }
      start = i;
    }
  }

  return result;
};
```

0831 Masking Personal Information

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 831. Masking Personal Information
 * https://leetcode.com/problems/masking-personal-information/
 * Difficulty: Medium
 *
 * We are given a personal information string S, which may represent either an email address
or a phone number.
 * We would like to mask this personal information according to the following rules:
 * 1. Email address:
 * We define a name to be a string of length ≥ 2 consisting of only lowercase letters a-z or
uppercase letters A-Z.
 * An email address starts with a name, followed by the symbol '@', followed by a name,
followed by the dot '.' and
 * followed by a name. All email addresses are guaranteed to be valid and in the format of
"name1@name2.name3".
 * To mask an email, all names must be converted to lowercase and all letters between the
first and last letter of
 * the first name must be replaced by 5 asterisks '*'.
 *
 * 2. Phone number:
 * A phone number is a string consisting of only the digits 0-9 or the characters from the set
 * {'+', '-', '(', ')', ' '}. You may assume a phone number contains 10 to 13 digits. The last
 10 digits make up the
 * local number, while the digits before those make up the country code. Note that the country
 code is optional. We
 * want to expose only the last 4 digits and mask all other digits. The local number should be
 formatted and masked
 * as "***-***-1111", where 1 represents the exposed digits. To mask a phone number with
country code like
 * "+111 111 111 1111", we write it in the form "+***-***-***-1111".  The '+' sign and the
first '-' sign before the
 * local number should only exist if there is a country code.  For example, a 12 digit phone
number mask should start
 * with "+**-". Note that extraneous characters like "(", ")", " ", as well as extra dashes or
 plus signs not part of
 * the above formatting scheme should be removed.
 *
 * Return the correct "mask" of the information provided.
 */

// The one-line solution:
/**
 * @param {string} S
 * @return {string}
 */
var maskPII = function(S) {
```

```
  return S.toLowerCase()
    .replace(/^([a-z])[a-z]*([a-z])/, '$1*****$2')
    .replace(/[)(\-\s+]+/g, '')
    .replace(/^(\d*)(\d{3})(\d{3})(\d{4})$/, '+$1-$2-$3-$4')
    .replace(/\d(?!\d{0,3}$)/g, '*')
    .replace(/^\+\-/, '');
};

// More readable alternative:
/**
 * @param {string} S
 * @return {string}
 */
var maskPII = function(S) {
  if (S.includes('@')) {
    return S.toLowerCase().replace(/^(\w)\w*(\w)/, '$1*****$2');
  } else {
    return S.replace(/\D+/g, '')
      .replace(/^(\d*)(\d{3})(\d{3})(\d{4})$/, '+$1-$2-$3-$4')
      .replace(/\d(?!\d{0,3}$)/g, '*')
      .replace(/^\+\-/, '');
  }
};
```

0832 Flipping An Image

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 832. Flipping an Image
 * https://leetcode.com/problems/flipping-an-image/
 * Difficulty: Easy
 *
 * Given an n x n binary matrix image, flip the image horizontally, then invert it, and
 * return the resulting image.
 *
 * To flip an image horizontally means that each row of the image is reversed.
 * - For example, flipping [1,1,0] horizontally results in [0,1,1].
 *
 * To invert an image means that each 0 is replaced by 1, and each 1 is replaced by 0.
 * - For example, inverting [0,1,1] results in [1,0,0].
 */

/**
 * @param {number[][]} image
 * @return {number[][]}
 */
var flipAndInvertImage = function(image) {
  return image.map(row => row.reverse().map(num => 1 ^ num));
};
```

0833 Find And Replace In String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 833. Find And Replace in String
 * https://leetcode.com/problems/find-and-replace-in-string/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s that you must perform k replacement operations on.
 * The replacement operations are given as three 0-indexed parallel arrays, indices, sources,
 * and targets, all of length k.
 *
 * To complete the ith replacement operation:
```

```
 * 1. Check if the substring sources[i] occurs at index indices[i] in the original string s.
 * 2. If it does not occur, do nothing.
 * 3. Otherwise if it does occur, replace that substring with targets[i].
 *
 * For example, if s = "abcd", indices[i] = 0, sources[i] = "ab", and targets[i] = "eee", then
 * the result of this replacement will be "eeecd".
 *
 * All replacement operations must occur simultaneously, meaning the replacement operations
should
 * not affect the indexing of each other. The testcases will be generated such that the
replacements
 * will not overlap.
 *
 * - For example, a testcase with s = "abc", indices = [0, 1], and sources = ["ab","bc"] will
not be
 *   generated because the "ab" and "bc" replacements overlap.
 *
 * Return the resulting string after performing all replacement operations on s.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} s
 * @param {number[]} indices
 * @param {string[]} sources
 * @param {string[]} targets
 * @return {string}
 */
var findReplaceString = function(s, indices, sources, targets) {
  const replacements = [];

  for (let i = 0; i < indices.length; i++) {
    const index = indices[i];
    const source = sources[i];
    const target = targets[i];

    if (s.substring(index, index + source.length) === source) {
      replacements.push({ index, source, target });
    }
  }

  replacements.sort((a, b) => b.index - a.index);

  let result = s;
  for (const { index, source, target } of replacements) {
    result = result.substring(0, index) + target + result.substring(index + source.length);
  }

  return result;
};
```

0834 Sum Of Distances In Tree

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 834. Sum of Distances in Tree
 * https://leetcode.com/problems/sum-of-distances-in-tree/
 * Difficulty: Hard
 *
 * There is an undirected connected tree with n nodes labeled from 0 to n - 1 and n - 1 edges.
 *
 * You are given the integer n and the array edges where edges[i] = [ai, bi] indicates that
there
 * is an edge between nodes ai and bi in the tree.
 *
 * Return an array answer of length n where answer[i] is the sum of the distances between the
ith
 * node in the tree and all other nodes.
```

```
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[]}
 */
var sumOfDistancesInTree = function(n, edges) {
  const graph = Array.from({ length: n }, () => []);
  const count = new Array(n).fill(1);
  const result = new Array(n).fill(0);

  for (const [a, b] of edges) {
    graph[a].push(b);
    graph[b].push(a);
  }

  function dfs1(node, parent) {
    for (const child of graph[node]) {
      if (child !== parent) {
        dfs1(child, node);
        count[node] += count[child];
        result[node] += result[child] + count[child];
      }
    }
  }

  function dfs2(node, parent) {
    for (const child of graph[node]) {
      if (child !== parent) {
        result[child] = result[node] - count[child] + (n - count[child]);
        dfs2(child, node);
      }
    }
  }

  dfs1(0, -1);
  dfs2(0, -1);

  return result;
};
```

0835 Image Overlap

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 835. Image Overlap
 * https://leetcode.com/problems/image-overlap/
 * Difficulty: Medium
 *
 * You are given two images, img1 and img2, represented as binary, square matrices of size n x
 n.
 * A binary matrix has only 0s and 1s as values.
 *
 * We translate one image however we choose by sliding all the 1 bits left, right, up, and/or
down
 * any number of units. We then place it on top of the other image. We can then calculate the
 * overlap by counting the number of positions that have a 1 in both images.
 *
 * Note also that a translation does not include any kind of rotation. Any 1 bits that are
 * translated outside of the matrix borders are erased.
 *
 * Return the largest possible overlap.
 */

/**
 * @param {number[][]} img1
 * @param {number[][]} img2
 * @return {number}
```

```
 */
var largestOverlap = function(img1, img2) {
  const positions1 = [];
  const positions2 = [];

  for (let i = 0; i < img1.length; i++) {
    for (let j = 0; j < img1.length; j++) {
      if (img1[i][j] === 1) {
        positions1.push([i, j]);
      }
      if (img2[i][j] === 1) {
        positions2.push([i, j]);
      }
    }
  }

  if (positions1.length === 0 || positions2.length === 0) {
    return 0;
  }

  const translations = new Map();
  let maxOverlap = 0;

  for (const [r1, c1] of positions1) {
    for (const [r2, c2] of positions2) {
      const translation = `${r2 - r1},${c2 - c1}`;

      if (!translations.has(translation)) {
        translations.set(translation, 0);
      }

      translations.set(translation, translations.get(translation) + 1);
      maxOverlap = Math.max(maxOverlap, translations.get(translation));
    }
  }

  return maxOverlap;
};
```

0836 Rectangle Overlap

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 836. Rectangle Overlap
 * https://leetcode.com/problems/rectangle-overlap/
 * Difficulty: Easy
 *
 * An axis-aligned rectangle is represented as a list [x1, y1, x2, y2], where (x1, y1) is the
 * coordinate of its bottom-left corner, and (x2, y2) is the coordinate of its top-right
corner.
 * Its top and bottom edges are parallel to the X-axis, and its left and right edges are
parallel
 * to the Y-axis.
 *
 * Two rectangles overlap if the area of their intersection is positive. To be clear, two
rectangles
 * that only touch at the corner or edges do not overlap.
 *
 * Given two axis-aligned rectangles rec1 and rec2, return true if they overlap, otherwise
return
 * false.
 */

/**
 * @param {number[]} rec1
 * @param {number[]} rec2
 * @return {boolean}
 */
var isRectangleOverlap = function(rec1, rec2) {
```

```
  const [x1, y1, x2, y2] = rec1;
  const [a1, b1, a2, b2] = rec2;
  return x1 < a2 && a1 < x2 && y1 < b2 && b1 < y2;
};
```

0837 New 21 Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 837. New 21 Game
 * https://leetcode.com/problems/new-21-game/
 * Difficulty: Medium
 *
 * Alice plays the following game, loosely based on the card game "21".
 *
 * Alice starts with 0 points and draws numbers while she has less than k points. During each
draw,
 * she gains an integer number of points randomly from the range [1, maxPts], where maxPts is
an
 * integer. Each draw is independent and the outcomes have equal probabilities.
 *
 * Alice stops drawing numbers when she gets k or more points.
 *
 * Return the probability that Alice has n or fewer points.
 *
 * Answers within 10-5 of the actual answer are considered accepted.
 */

/**
 * @param {number} n
 * @param {number} k
 * @param {number} maxPts
 * @return {number}
 */
var new21Game = function(n, k, maxPts) {
  if (k === 0 || n >= k + maxPts - 1) return 1.0;

  const dp = new Array(n + 1).fill(0);
  dp[0] = 1.0;

  let sum = 1.0;
  for (let i = 1; i <= n; i++) {
    dp[i] = sum / maxPts;
    if (i < k) {
      sum += dp[i];
    }
    if (i >= maxPts) {
      sum -= dp[i - maxPts];
    }
  }

  let result = 0;
  for (let i = k; i <= n; i++) {
    result += dp[i];
  }

  return result;
};
```

0838 Push Dominoes

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 838. Push Dominoes
 * https://leetcode.com/problems/push-dominoes/
 * Difficulty: Medium
 *
 * There are n dominoes in a line, and we place each domino vertically upright. In the
beginning,
 * we simultaneously push some of the dominoes either to the left or to the right.
 *
 * After each second, each domino that is falling to the left pushes the adjacent domino on
the
 * left. Similarly, the dominoes falling to the right push their adjacent dominoes standing on
 * the right.
 *
 * When a vertical domino has dominoes falling on it from both sides, it stays still due to
the
 * balance of the forces.
 *
 * For the purposes of this question, we will consider that a falling domino expends no
additional
 * force to a falling or already fallen domino.
 *
 * You are given a string dominoes representing the initial state where:
 * - dominoes[i] = 'L', if the ith domino has been pushed to the left,
 * - dominoes[i] = 'R', if the ith domino has been pushed to the right, and
 * - dominoes[i] = '.', if the ith domino has not been pushed.
 *
 * Return a string representing the final state.
 */

/**
 * @param {string} dominoes
 * @return {string}
 */
var pushDominoes = function(dominoes) {
  const forces = new Array(dominoes.length).fill(0);

  let force = 0;
  for (let i = 0; i < dominoes.length; i++) {
    if (dominoes[i] === 'R') {
      force = dominoes.length;
    } else if (dominoes[i] === 'L') {
      force = 0;
    } else {
      force = Math.max(force - 1, 0);
    }
    forces[i] += force;
  }

  force = 0;
  for (let i = dominoes.length - 1; i >= 0; i--) {
    if (dominoes[i] === 'L') {
      force = dominoes.length;
    } else if (dominoes[i] === 'R') {
      force = 0;
    } else {
      force = Math.max(force - 1, 0);
    }
    forces[i] -= force;
  }

  return forces.map(f => {
    if (f > 0) return 'R';
    if (f < 0) return 'L';
    return '.';
  }).join('');
};
```

0839 Similar String Groups

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 839. Similar String Groups
 * https://leetcode.com/problems/similar-string-groups/
 * Difficulty: Hard
 *
 * Two strings, X and Y, are considered similar if either they are identical or we can make
them
 * equivalent by swapping at most two letters (in distinct positions) within the string X.
 *
 * For example, "tars" and "rats" are similar (swapping at positions 0 and 2), and "rats" and
 * "arts" are similar, but "star" is not similar to "tars", "rats", or "arts".
 *
 * Together, these form two connected groups by similarity: {"tars", "rats", "arts"} and
{"star"}.
 * Notice that "tars" and "arts" are in the same group even though they are not similar.
Formally,
 * each group is such that a word is in the group if and only if it is similar to at least one
 * other word in the group.
 *
 * We are given a list strs of strings where every string in strs is an anagram of every other
 * string in strs. How many groups are there?
 */

/**
 * @param {string[]} strs
 * @return {number}
 */
var numSimilarGroups = function(strs) {
  const parent = Array.from({ length: strs.length }, (_, i) => i);
  const rank = new Array(strs.length).fill(0);

  for (let i = 0; i < strs.length; i++) {
    for (let j = i + 1; j < strs.length; j++) {
      if (areSimilar(strs[i], strs[j])) union(i, j);
    }
  }

  const groups = new Set();
  for (let i = 0; i < strs.length; i++) groups.add(find(i));

  return groups.size;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    const rootX = find(x);
    const rootY = find(y);

    if (rootX === rootY) return;

    if (rank[rootX] < rank[rootY]) {
      parent[rootX] = rootY;
    } else {
      parent[rootY] = rootX;
      if (rank[rootX] === rank[rootY]) rank[rootX]++;
    }
  }

  function areSimilar(a, b) {
    if (a === b) return true;

    let diff = 0;
    for (let i = 0; i < a.length; i++) {
      if (a[i] !== b[i] && ++diff > 2) return false;
    }

    return diff === 2;
  }
};
```

0840 Magic Squares In Grid

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 840. Magic Squares In Grid
 * https://leetcode.com/problems/magic-squares-in-grid/
 * Difficulty: Medium
 *
 * A 3 x 3 magic square is a 3 x 3 grid filled with distinct numbers from 1 to 9 such that
each row,
 * column, and both diagonals all have the same sum.
 *
 * Given a row x col grid of integers, how many 3 x 3 magic square subgrids are there?
 *
 * Note: while a magic square can only contain numbers from 1 to 9, grid may contain numbers
up
 * to 15.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var numMagicSquaresInside = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;

  if (rows < 3 || cols < 3) return 0;

  let result = 0;
  for (let r = 0; r <= rows - 3; r++) {
    for (let c = 0; c <= cols - 3; c++) {
      if (isMagicSquare(grid, r, c)) result++;
    }
  }

  return result;
};

function isMagicSquare(grid, r, c) {
  if (grid[r + 1][c + 1] !== 5) return false;

  const cells = [
    grid[r][c], grid[r][c + 1], grid[r][c + 2],
    grid[r + 1][c], grid[r + 1][c + 1], grid[r + 1][c + 2],
    grid[r + 2][c], grid[r + 2][c + 1], grid[r + 2][c + 2]
  ];

  if (!cells.every(val => val >= 1 && val <= 9) || new Set(cells).size !== 9) return false;

  return (
    grid[r][c] + grid[r][c + 1] + grid[r][c + 2] === 15
    && grid[r + 1][c] + grid[r + 1][c + 1] + grid[r + 1][c + 2] === 15
    && grid[r + 2][c] + grid[r + 2][c + 1] + grid[r + 2][c + 2] === 15
    && grid[r][c] + grid[r + 1][c] + grid[r + 2][c] === 15
    && grid[r][c + 1] + grid[r + 1][c + 1] + grid[r + 2][c + 1] === 15
    && grid[r][c + 2] + grid[r + 1][c + 2] + grid[r + 2][c + 2] === 15
    && grid[r][c] + grid[r + 1][c + 1] + grid[r + 2][c + 2] === 15
    && grid[r][c + 2] + grid[r + 1][c + 1] + grid[r + 2][c] === 15
  );
}
```

0841 Keys And Rooms

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 841. Keys and Rooms
 * https://leetcode.com/problems/keys-and-rooms/
 * Difficulty: Medium
 *
 * There are n rooms labeled from 0 to n - 1 and all the rooms are locked except for room 0.
 * Your goal is to visit all the rooms. However, you cannot enter a locked room without
 * having its key.
 *
 * When you visit a room, you may find a set of distinct keys in it. Each key has a number
 * on it, denoting which room it unlocks, and you can take all of them with you to unlock
 * the other rooms.
 *
 * Given an array rooms where rooms[i] is the set of keys that you can obtain if you visited
 * room i, return true if you can visit all the rooms, or false otherwise.
 */

/**
 * @param {number[][]} rooms
 * @return {boolean}
 */
var canVisitAllRooms = function(rooms) {
  const set = new Set([0]);
  for (const key of set) {
    rooms[key].forEach(value => set.add(value));
  }
  return set.size === rooms.length;
};
```

0842 Split Array Into Fibonacci Sequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 842. Split Array into Fibonacci Sequence
 * https://leetcode.com/problems/split-array-into-fibonacci-sequence/
 * Difficulty: Medium
 *
 * You are given a string of digits num, such as "123456579". We can split it into a
Fibonacci-like
 * sequence [123, 456, 579].
 *
 * Formally, a Fibonacci-like sequence is a list f of non-negative integers such that:
 * - 0 <= f[i] < 231, (that is, each integer fits in a 32-bit signed integer type),
 * - f.length >= 3, and
 * - f[i] + f[i + 1] == f[i + 2] for all 0 <= i < f.length - 2.
 *
 * Note that when splitting the string into pieces, each piece must not have extra leading
zeroes,
 * except if the piece is the number 0 itself.
 *
 * Return any Fibonacci-like sequence split from num, or return [] if it cannot be done.
 */

/**
 * @param {string} num
 * @return {number[]}
 */
var splitIntoFibonacci = function(num) {
  const result = [];
  const MAX_INT = 2 ** 31 - 1;
  backtrack(0);
  return result;

  function backtrack(index) {
    if (index === num.length && result.length >= 3) return true;
    for (let length = 1; length <= num.length - index; length++) {
```

```
      if (num[index] === '0' && length > 1) break;
      const current = parseInt(num.substring(index, index + length));
      if (current > MAX_INT) break;
      if (result.length >= 2) {
        const sum = result[result.length - 1] + result[result.length - 2];
        if (current > sum) break;
        if (current < sum) continue;
      }
      result.push(current);
      if (backtrack(index + length)) return true;
      result.pop();
    }
    return false;
  }
};
```

0843 Guess The Word

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 843. Guess the Word
 * https://leetcode.com/problems/guess-the-word/
 * Difficulty: Hard
 *
 * You are given an array of unique strings words where words[i] is six letters long. One word
 of
 * words was chosen as a secret word.
 *
 * You are also given the helper object Master. You may call Master.guess(word) where word is
a
 * six-letter-long string, and it must be from words. Master.guess(word) returns:
 * - -1 if word is not from words, or
 * - an integer representing the number of exact matches (value and position) of your guess to
 *   the secret word.
 *
 * There is a parameter allowedGuesses for each test case where allowedGuesses is the maximum
 * number of times you can call Master.guess(word).
 *
 * For each test case, you should call Master.guess with the secret word without exceeding the
 * maximum number of allowed guesses. You will get:
 * - "Either you took too many guesses, or you did not find the secret word." if you called
 *   Master.guess more than allowedGuesses times or if you did not call Master.guess with the
 *   secret word, or
 * - "You guessed the secret word correctly." if you called Master.guess with the secret word
 *   with the number of calls to Master.guess less than or equal to allowedGuesses.
 *
 * The test cases are generated such that you can guess the secret word with a reasonable
 * strategy (other than using the bruteforce method).
 */

/**
 * @param {string[]} words
 * @param {Master} master
 * @return {void}
 */
var findSecretWord = function(words, master) {
  for (let attempt = 0; attempt < 30; attempt++) {
    const randomWord = words[Math.floor(Math.random() * words.length)];
    const matchCount = master.guess(randomWord);

    if (matchCount === 6) return;

    words = words.filter(word =>
      matchCount === -1
        ? countMatches(word, randomWord) === 0
        : countMatches(word, randomWord) === matchCount
    );
  }
```

```
  function countMatches(word1, word2) {
    let matches = 0;
    for (let i = 0; i < word1.length; i++) {
      if (word1[i] === word2[i]) matches++;
    }
    return matches;
  }
};
```

0844 Backspace String Compare

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 844. Backspace String Compare
 * https://leetcode.com/problems/backspace-string-compare/
 * Difficulty: Easy
 *
 * Given two strings `s` and `t`, return true if they are equal when both
 * are typed into empty text editors. '#' means a backspace character.
 *
 * Note that after backspacing an empty text, the text will continue empty.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var backspaceCompare = function(s, t) {
  return handleBackspaces(s) === handleBackspaces(t);
};

function handleBackspaces(input) {
  return input.split('').reduce((result, char) => {
    if (char === '#') {
      result.pop();
    } else {
      result.push(char);
    }
    return result;
  }, []).join('');
}
```

0845 Longest Mountain In Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 845. Longest Mountain in Array
 * https://leetcode.com/problems/longest-mountain-in-array/
 * Difficulty: Medium
 *
 * You may recall that an array arr is a mountain array if and only if:
 * - arr.length >= 3
 * - There exists some index i (0-indexed) with 0 < i < arr.length - 1 such that:
 *   - arr[0] < arr[1] < ... < arr[i - 1] < arr[i]
 *   - arr[i] > arr[i + 1] > ... > arr[arr.length - 1]
 *
 * Given an integer array arr, return the length of the longest subarray, which is a
 * mountain. Return 0 if there is no mountain subarray.
 */

/**
 * @param {number[]} arr
```

```
 * @return {number}
 */
var longestMountain = function(arr) {
  const n = arr.length;
  if (n < 3) return 0;

  let maxLength = 0;
  let i = 1;

  while (i < n - 1) {
    const isPeak = arr[i] > arr[i - 1] && arr[i] > arr[i + 1];
    if (!isPeak) {
      i++;
      continue;
    }
    let leftBound = i - 1;
    while (leftBound > 0 && arr[leftBound - 1] < arr[leftBound]) {
      leftBound--;
    }
    let rightBound = i + 1;
    while (rightBound < n - 1 && arr[rightBound] > arr[rightBound + 1]) {
      rightBound++;
    }
    const mountainLength = rightBound - leftBound + 1;
    maxLength = Math.max(maxLength, mountainLength);
    i = rightBound + 1;
  }

  return maxLength;
};
```

0846 Hand Of Straights

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 846. Hand of Straights
 * https://leetcode.com/problems/hand-of-straights/
 * Difficulty: Medium
 *
 * Alice has some number of cards and she wants to rearrange
 * the cards into groups so that each group is of size groupSize,
 * and consists of groupSize consecutive cards.
 *
 * Given an integer array hand where hand[i] is the value written
 * on the ith card and an integer groupSize, return true if she
 * can rearrange the cards, or false otherwise.
 */

/**
 * @param {number[]} hand
 * @param {number} groupSize
 * @return {boolean}
 */
var isNStraightHand = function(hand, groupSize) {
  if (hand.length % groupSize) {
    return false;
  }

  const map = {};
  const set = new Set(hand);
  hand.forEach(x => map[x] ? map[x]++ : map[x] = 1);

  let count = hand.length / groupSize;
  while (count--) {
    let min = Math.min(...set);
    for (let i = min; i < min + groupSize; i++) {
      if (!map[i]) {
        return false;
```

```
      }
      if (!--map[i]) {
        set.delete(i);
      }
    }
  }

  return true;
};
```

0847 Shortest Path Visiting All Nodes

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 847. Shortest Path Visiting All Nodes
 * https://leetcode.com/problems/shortest-path-visiting-all-nodes/
 * Difficulty: Hard
 *
 * You have an undirected, connected graph of n nodes labeled from 0 to n - 1. You are given
an
 * array graph where graph[i] is a list of all the nodes connected with node i by an edge.
 *
 * Return the length of the shortest path that visits every node. You may start and stop at
any
 * node, you may revisit nodes multiple times, and you may reuse edges.
 */

/**
 * @param {number[][]} graph
 * @return {number}
 */
var shortestPathLength = function(graph) {
  const n = graph.length;

  if (n === 1) return 0;

  const allVisited = (1 << n) - 1;
  const queue = [];
  const visited = new Set();

  for (let i = 0; i < n; i++) {
    const initialState = (1 << i);
    queue.push([i, initialState, 0]);
    visited.add(`${i},${initialState}`);
  }

  while (queue.length > 0) {
    const [node, state, distance] = queue.shift();
    if (state === allVisited) {
      return distance;
    }
    for (const neighbor of graph[node]) {
      const newState = state | (1 << neighbor);
      const key = `${neighbor},${newState}`;
      if (!visited.has(key)) {
        visited.add(key);
        queue.push([neighbor, newState, distance + 1]);
      }
    }
  }

  return -1;
};
```

0848 Shifting Letters

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 848. Shifting Letters
 * https://leetcode.com/problems/shifting-letters/
 * Difficulty: Medium
 *
 * You are given a string s of lowercase English letters and an integer array shifts of the
 * same length.
 *
 * Call the shift() of a letter, the next letter in the alphabet, (wrapping around so that
 * 'z' becomes 'a').
 *
 * For example, shift('a') = 'b', shift('t') = 'u', and shift('z') = 'a'.
 *
 * Now for each shifts[i] = x, we want to shift the first i + 1 letters of s, x times.
 *
 * Return the final string after all such shifts to s are applied.
 */

/**
 * @param {string} s
 * @param {number[]} shifts
 * @return {string}
 */
var shiftingLetters = function(s, shifts) {
  const cumulativeShifts = new Array(s.length).fill(0);

  cumulativeShifts[s.length - 1] = shifts[s.length - 1] % 26;
  for (let i = s.length - 2; i >= 0; i--) {
    cumulativeShifts[i] = (cumulativeShifts[i + 1] + shifts[i]) % 26;
  }

  let result = '';
  for (let i = 0; i < s.length; i++) {
    const charCode = s.charCodeAt(i);
    const shiftedCode = ((charCode - 97 + cumulativeShifts[i]) % 26) + 97;
    result += String.fromCharCode(shiftedCode);
  }

  return result;
};
```

0849 Maximize Distance To Closest Person

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 849. Maximize Distance to Closest Person
 * https://leetcode.com/problems/maximize-distance-to-closest-person/
 * Difficulty: Medium
 *
 * You are given an array representing a row of seats where seats[i] = 1 represents a person
 * sitting in the ith seat, and seats[i] = 0 represents that the ith seat is empty
(0-indexed).
 *
 * There is at least one empty seat, and at least one person sitting.
 *
 * Alex wants to sit in the seat such that the distance between him and the closest person to
 * him is maximized.
 *
 * Return that maximum distance to the closest person.
 */
```

```
/**
 * @param {number[]} seats
 * @return {number}
 */
var maxDistToClosest = function(seats) {
  let lastOccupied = -1;
  let result = 0;

  for (let i = 0; i < seats.length; i++) {
    if (seats[i] === 1) {
      if (lastOccupied === -1) {
        result = i;
      } else {
        result = Math.max(result, Math.floor((i - lastOccupied) / 2));
      }
      lastOccupied = i;
    }
  }

  if (lastOccupied !== seats.length - 1) {
    result = Math.max(result, seats.length - 1 - lastOccupied);
  }

  return result;
};
```

0850 Rectangle Area Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 850. Rectangle Area II
 * https://leetcode.com/problems/rectangle-area-ii/
 * Difficulty: Hard
 *
 * You are given a 2D array of axis-aligned rectangles. Each rectangle[i] = [xi1, yi1, xi2,
yi2]
 * denotes the ith rectangle where (xi1, yi1) are the coordinates of the bottom-left corner,
and
 * (xi2, yi2) are the coordinates of the top-right corner.
 *
 * Calculate the total area covered by all rectangles in the plane. Any area covered by two or
 more
 * rectangles should only be counted once.
 *
 * Return the total area. Since the answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {number[][]} rectangles
 * @return {number}
 */
var rectangleArea = function(rectangles) {
  const MOD = 1e9 + 7;
  const events = [];
  const xCoords = new Set();
  const yCoords = new Set();

  for (const [x1, y1, x2, y2] of rectangles) {
    xCoords.add(x1);
    xCoords.add(x2);
    yCoords.add(y1);
    yCoords.add(y2);
    events.push([x1, 1, y1, y2]);
    events.push([x2, -1, y1, y2]);
  }

  const xArray = [...xCoords].sort((a, b) => a - b);
  const yArray = [...yCoords].sort((a, b) => a - b);
```

```
  const xMap = new Map();
  const yMap = new Map();

  for (let i = 0; i < xArray.length; i++) {
    xMap.set(xArray[i], i);
  }

  for (let i = 0; i < yArray.length; i++) {
    yMap.set(yArray[i], i);
  }

  const grid = Array(xArray.length).fill(0).map(() => Array(yArray.length).fill(0));

  for (const [x1, y1, x2, y2] of rectangles) {
    for (let x = xMap.get(x1); x < xMap.get(x2); x++) {
      for (let y = yMap.get(y1); y < yMap.get(y2); y++) {
        grid[x][y] = 1;
      }
    }
  }

  let totalArea = 0;

  for (let x = 0; x < xArray.length - 1; x++) {
    for (let y = 0; y < yArray.length - 1; y++) {
      if (grid[x][y]) {
        const width = xArray[x + 1] - xArray[x];
        const height = yArray[y + 1] - yArray[y];
        totalArea = (totalArea + width * height) % MOD;
      }
    }
  }

  return totalArea;
};
```

0851 Loud And Rich

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 851. Loud and Rich
 * https://leetcode.com/problems/loud-and-rich/
 * Difficulty: Medium
 *
 * There is a group of n people labeled from 0 to n - 1 where each person has a different
amount
 * of money and a different level of quietness.
 *
 * You are given an array richer where richer[i] = [ai, bi] indicates that ai has more money
than
 * bi and an integer array quiet where quiet[i] is the quietness of the ith person. All the
given
 * data in richer are logically correct (i.e., the data will not lead you to a situation where
 x
 * is richer than y and y is richer than x at the same time).
 *
 * Return an integer array answer where answer[x] = y if y is the least quiet person (that is,
 * the person y with the smallest value of quiet[y]) among all people who definitely have
equal
 * to or more money than the person x.
 */

/**
 * @param {number[][]} richer
 * @param {number[]} quiet
 * @return {number[]}
 */
var loudAndRich = function(richer, quiet) {
```

```
  const graph = new Array(quiet.length).fill().map(() => []);
  const result = new Array(quiet.length).fill(-1);

  for (const [a, b] of richer) {
    graph[b].push(a);
  }

  function dfs(person) {
    if (result[person] !== -1) return result[person];

    let quietestPerson = person;

    for (const richerPerson of graph[person]) {
      const candidate = dfs(richerPerson);
      if (quiet[candidate] < quiet[quietestPerson]) {
        quietestPerson = candidate;
      }
    }

    result[person] = quietestPerson;
    return quietestPerson;
  }

  for (let i = 0; i < quiet.length; i++) {
    dfs(i);
  }

  return result;
};
```

0852 Peak Index In A Mountain Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 852. Peak Index in a Mountain Array
 * https://leetcode.com/problems/peak-index-in-a-mountain-array/
 * Difficulty: Medium
 *
 * You are given an integer mountain array arr of length n where the values increase to a peak
 * element and then decrease.
 *
 * Return the index of the peak element.
 *
 * Your task is to solve it in O(log(n)) time complexity.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var peakIndexInMountainArray = function(arr) {
  let left = 0;
  let right = arr.length - 1;

  while (left < right) {
    const middle = Math.floor((left + right) / 2);

    if (arr[middle] > arr[middle + 1]) {
      right = middle;
    } else {
      left = middle + 1;
    }
  }

  return left;
};
```

0853 Car Fleet

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 853. Car Fleet
 * https://leetcode.com/problems/car-fleet/
 * Difficulty: Medium
 *
 * There are n cars at given miles away from the starting mile 0, traveling to reach the mile
 * target.
 *
 * You are given two integer array position and speed, both of length n, where position[i]
 * is the starting mile of the ith car and speed[i] is the speed of the ith car in miles per
hour.
 *
 * A car cannot pass another car, but it can catch up and then travel next to it at the speed
of
 * the slower car.
 *
 * A car fleet is a car or cars driving next to each other. The speed of the car fleet is the
 * minimum speed of any car in the fleet.
 *
 * If a car catches up to a car fleet at the mile target, it will still be considered as part
 * of the car fleet.
 *
 * Return the number of car fleets that will arrive at the destination.
 */

/**
 * @param {number} target
 * @param {number[]} position
 * @param {number[]} speed
 * @return {number}
 */
var carFleet = function(target, position, speed) {
  const cars = position.map((pos, i) => ({
    position: pos,
    timeToTarget: (target - pos) / speed[i]
  }));
  cars.sort((a, b) => b.position - a.position);

  let slowestTime = 0;
  let result = 0;
  for (const car of cars) {
    if (car.timeToTarget > slowestTime) {
      result++;
      slowestTime = car.timeToTarget;
    }
  }

  return result;
};
```

0854 K Similar Strings

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 854. K-Similar Strings
 * https://leetcode.com/problems/k-similar-strings/
 * Difficulty: Hard
 *
```

```
 * Strings s1 and s2 are k-similar (for some non-negative integer k) if we can swap the
positions
 * of two letters in s1 exactly k times so that the resulting string equals s2.
 *
 * Given two anagrams s1 and s2, return the smallest k for which s1 and s2 are k-similar.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {number}
 */
var kSimilarity = function(s1, s2) {
  if (s1 === s2) return 0;
  const queue = [s1];
  const visited = new Set([s1]);
  let swaps = 0;

  while (queue.length) {
    const size = queue.length;
    for (let i = 0; i < size; i++) {
      const current = queue.shift();
      if (current === s2) return swaps;
      let j = 0;
      while (j < current.length && current[j] === s2[j]) j++;
      for (let k = j + 1; k < current.length; k++) {
        if (current[k] === s2[j] && current[k] !== s2[k]) {
          const nextStr = swap(current, j, k);

          if (!visited.has(nextStr)) {
            if (nextStr === s2) return swaps + 1;
            visited.add(nextStr);
            queue.push(nextStr);
          }
        }
      }
    }
    swaps++;
  }

  return swaps;
};

function swap(str, i, j) {
  const chars = str.split('');
  [chars[i], chars[j]] = [chars[j], chars[i]];
  return chars.join('');
}
```

0855 Exam Room

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 855. Exam Room
 * https://leetcode.com/problems/exam-room/
 * Difficulty: Medium
 *
 * There is an exam room with n seats in a single row labeled from 0 to n - 1.
 *
 * When a student enters the room, they must sit in the seat that maximizes the distance to
the
 * closest person. If there are multiple such seats, they sit in the seat with the lowest
number.
 * If no one is in the room, then the student sits at seat number 0.
 *
 * Design a class that simulates the mentioned exam room.
 *
 * Implement the ExamRoom class:
 * - ExamRoom(int n) Initializes the object of the exam room with the number of the seats n.
```

```
 * - int seat() Returns the label of the seat at which the next student will set.
 * - void leave(int p) Indicates that the student sitting at seat p will leave the room. It is
 *   guaranteed that there will be a student sitting at seat p.
 */

/**
 * @param {number} n - Number of seats in the exam room
 */
var ExamRoom = function(n) {
  this.n = n;
  this.seats = [];
};

/**
 * @return {number}
 */
ExamRoom.prototype.seat = function() {
  if (this.seats.length === 0) {
    this.seats.push(0);
    return 0;
  }

  let maxDistance = this.seats[0];
  let chosenSeat = 0;

  for (let i = 1; i < this.seats.length; i++) {
    const distance = Math.floor((this.seats[i] - this.seats[i-1]) / 2);
    if (distance > maxDistance) {
      maxDistance = distance;
      chosenSeat = Math.floor((this.seats[i] + this.seats[i-1]) / 2);
    }
  }

  const endDistance = this.n - 1 - this.seats[this.seats.length - 1];
  if (endDistance > maxDistance) {
    chosenSeat = this.n - 1;
    maxDistance = endDistance;
  }

  const insertPosition = this.findInsertPosition(chosenSeat);
  this.seats.splice(insertPosition, 0, chosenSeat);

  return chosenSeat;
};

/**
 * @param {number} p
 * @return {void}
 */
ExamRoom.prototype.leave = function(p) {
  const index = this.seats.indexOf(p);
  this.seats.splice(index, 1);
};

/**
 * @param {number} seat
 * @return {number}
 */
ExamRoom.prototype.findInsertPosition = function(seat) {
  let left = 0;
  let right = this.seats.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (this.seats[mid] < seat) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return left;
};
```

0856 Score Of Parentheses

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 856. Score of Parentheses
 * https://leetcode.com/problems/score-of-parentheses/
 * Difficulty: Medium
 *
 * Given a balanced parentheses string s, return the score of the string.
 *
 * The score of a balanced parentheses string is based on the following rule:
 * - "()" has score 1.
 * - AB has score A + B, where A and B are balanced parentheses strings.
 * - (A) has score 2 * A, where A is a balanced parentheses string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var scoreOfParentheses = function(s) {
  const stack = [0];

  for (const char of s) {
    if (char === '(') {
      stack.push(0);
    } else {
      const value = stack.pop();
      stack[stack.length - 1] += Math.max(2 * value, 1);
    }
  }

  return stack[0];
};
```

0857 Minimum Cost To Hire K Workers

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 857. Minimum Cost to Hire K Workers
 * https://leetcode.com/problems/minimum-cost-to-hire-k-workers/
 * Difficulty: Hard
 *
 * There are n workers. You are given two integer arrays quality and wage where quality[i] is
the
 * quality of the ith worker and wage[i] is the minimum wage expectation for the ith worker.
 *
 * We want to hire exactly k workers to form a paid group. To hire a group of k workers, we
must
 * pay them according to the following rules:
 * 1. Every worker in the paid group must be paid at least their minimum wage expectation.
 * 2. In the group, each worker's pay must be directly proportional to their quality. This
means
 *    if a worker’s quality is double that of another worker in the group, then they must be
paid
 *    twice as much as the other worker.
 *
 * Given the integer k, return the least amount of money needed to form a paid group
satisfying
 * the above conditions. Answers within 10-5 of the actual answer will be accepted.
 */

/**
```

```
 * @param {number[]} quality
 * @param {number[]} wage
 * @param {number} k
 * @return {number}
 */
var mincostToHireWorkers = function(quality, wage, k) {
  const workers = quality.map((q, i) => ({ ratio: wage[i] / q, quality: q }))
    .sort((a, b) => a.ratio - b.ratio);

  let result = Infinity;
  let qualitySum = 0;
  const maxHeap = new MaxPriorityQueue();

  for (const worker of workers) {
    maxHeap.enqueue(worker.quality);
    qualitySum += worker.quality;

    if (maxHeap.size() > k) {
      qualitySum -= maxHeap.dequeue();
    }

    if (maxHeap.size() === k) {
      result = Math.min(result, qualitySum * worker.ratio);
    }
  }

  return result;
};
```

0858 Mirror Reflection

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 858. Mirror Reflection
 * https://leetcode.com/problems/mirror-reflection/
 * Difficulty: Medium
 *
 * There is a special square room with mirrors on each of the four walls. Except for the
southwest
 * corner, there are receptors on each of the remaining corners, numbered 0, 1, and 2.
 *
 * The square room has walls of length p and a laser ray from the southwest corner first meets
 the
 * east wall at a distance q from the 0th receptor.
 *
 * Given the two integers p and q, return the number of the receptor that the ray meets first.
 *
 * The test cases are guaranteed so that the ray will meet a receptor eventually.
 */

/**
 * @param {number} p
 * @param {number} q
 * @return {number}
 */
var mirrorReflection = function(p, q) {
  const gcd = findGCD(p, q);
  const m = q / gcd;
  const n = p / gcd;

  if (m % 2 === 0) return 0;
  return n % 2 === 1 ? 1 : 2;
};

function findGCD(a, b) {
  return b === 0 ? a : findGCD(b, a % b);
}
```

0859 Buddy Strings

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 859. Buddy Strings
 * https://leetcode.com/problems/buddy-strings/
 * Difficulty: Easy
 *
 * Given two strings s and goal, return true if you can swap two letters in s so the result is
 * equal to goal, otherwise, return false.
 *
 * Swapping letters is defined as taking two indices i and j (0-indexed) such that i != j and
 * swapping the characters at s[i] and s[j].
 *
 * For example, swapping at indices 0 and 2 in "abcd" results in "cbad".
 */

/**
 * @param {string} s
 * @param {string} goal
 * @return {boolean}
 */
var buddyStrings = function(s, goal) {
  if (s.length !== goal.length) return false;

  const differences = [];
  const charCount = new Map();
  let hasDuplicate = false;

  for (let i = 0; i < s.length; i++) {
    if (s[i] !== goal[i]) {
      differences.push(i);
      if (differences.length > 2) return false;
    }
    charCount.set(s[i], (charCount.get(s[i]) || 0) + 1);
    if (charCount.get(s[i]) > 1) hasDuplicate = true;
  }

  if (differences.length === 0) return hasDuplicate;
  if (differences.length !== 2) return false;

  const [first, second] = differences;
  return s[first] === goal[second] && s[second] === goal[first];
};
```

0860 Lemonade Change

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 860. Lemonade Change
 * https://leetcode.com/problems/lemonade-change/
 * Difficulty: Easy
 *
 * At a lemonade stand, each lemonade costs $5. Customers are standing in a queue to buy from
you
 * and order one at a time (in the order specified by bills). Each customer will only buy one
 * lemonade and pay with either a $5, $10, or $20 bill. You must provide the correct change to
 * each customer so that the net transaction is that the customer pays $5.
 *
 * Note that you do not have any change in hand at first.
 *
 * Given an integer array bills where bills[i] is the bill the ith customer pays, return true
if
```

```
 * you can provide every customer with the correct change, or false otherwise.
 */

/**
 * @param {number[]} bills
 * @return {boolean}
 */
var lemonadeChange = function(bills) {
  let fives = 0;
  let tens = 0;

  for (const bill of bills) {
    if (bill === 5) {
      fives++;
    } else if (bill === 10) {
      if (fives === 0) return false;
      fives--;
      tens++;
    } else {
      if (tens > 0 && fives > 0) {
        tens--;
        fives--;
      } else if (fives >= 3) {
        fives -= 3;
      } else {
        return false;
      }
    }
  }

  return true;
};
```

0861 Score After Flipping Matrix

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 861. Score After Flipping Matrix
 * https://leetcode.com/problems/score-after-flipping-matrix/
 * Difficulty: Medium
 *
 * You are given an m x n binary matrix grid.
 *
 * A move consists of choosing any row or column and toggling each value in that row or
 * column (i.e., changing all 0's to 1's, and all 1's to 0's).
 *
 * Every row of the matrix is interpreted as a binary number, and the score of the matrix
 * is the sum of these numbers.
 *
 * Return the highest possible score after making any number of moves (including zero moves).
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var matrixScore = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;

  for (let row = 0; row < rows; row++) {
    if (grid[row][0] === 0) {
      for (let col = 0; col < cols; col++) {
        grid[row][col] ^= 1;
      }
    }
  }

  let result = rows * (1 << (cols - 1));
```

```
  for (let col = 1; col < cols; col++) {
    let onesCount = 0;
    for (let row = 0; row < rows; row++) {
      onesCount += grid[row][col];
    }
    const maxOnes = Math.max(onesCount, rows - onesCount);
    result += maxOnes * (1 << (cols - 1 - col));
  }

  return result;
};
```

0862 Shortest Subarray With Sum At Least K

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 862. Shortest Subarray with Sum at Least K
 * https://leetcode.com/problems/shortest-subarray-with-sum-at-least-k/
 * Difficulty: Hard
 *
 * Given an integer array nums and an integer k, return the length of the shortest non-empty
 * subarray of nums with a sum of at least k. If there is no such subarray, return -1.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var shortestSubarray = function(nums, k) {
  const prefixSums = [0];
  for (let i = 0; i < nums.length; i++) {
    prefixSums.push(prefixSums[i] + nums[i]);
  }

  const deque = [];
  let shortestLength = Infinity;

  for (let i = 0; i < prefixSums.length; i++) {
    while (deque.length && prefixSums[i] - prefixSums[deque[0]] >= k) {
      shortestLength = Math.min(shortestLength, i - deque.shift());
    }
    while (deque.length && prefixSums[i] <= prefixSums[deque[deque.length - 1]]) {
      deque.pop();
    }
    deque.push(i);
  }

  return shortestLength === Infinity ? -1 : shortestLength;
};
```

0863 All Nodes Distance K In Binary Tree

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 863. All Nodes Distance K in Binary Tree
 * https://leetcode.com/problems/all-nodes-distance-k-in-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, the value of a target node target, and an integer k,
```

```
return an
 * array of the values of all nodes that have a distance k from the target node.
 *
 * You can return the answer in any order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} target
 * @param {number} k
 * @return {number[]}
 */
var distanceK = function(root, target, k) {
  const graph = new Map();
  const result = [];

  buildGraph(root, null);
  findNodesAtDistance(target.val, 0, new Set());

  return result;

  function buildGraph(node, parent) {
    if (!node) return;
    if (parent) {
      graph.set(node.val, graph.get(node.val) || new Set());
      graph.get(node.val).add(parent.val);
      graph.set(parent.val, graph.get(parent.val) || new Set());
      graph.get(parent.val).add(node.val);
    }
    buildGraph(node.left, node);
    buildGraph(node.right, node);
  }

  function findNodesAtDistance(currentVal, distance, visited) {
    if (distance === k) {
      result.push(currentVal);
      return;
    }
    visited.add(currentVal);
    const neighbors = graph.get(currentVal) || new Set();
    for (const neighbor of neighbors) {
      if (!visited.has(neighbor)) {
        findNodesAtDistance(neighbor, distance + 1, visited);
      }
    }
  }
};
```

0864 Shortest Path To Get All Keys

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 864. Shortest Path to Get All Keys
 * https://leetcode.com/problems/shortest-path-to-get-all-keys/
 * Difficulty: Hard
 *
 * You are given an m x n grid grid where:
 * - '.' is an empty cell.
 * - '#' is a wall.
 * - '@' is the starting point.
 * - Lowercase letters represent keys.
```

```
 * - Uppercase letters represent locks.
 *
 * You start at the starting point and one move consists of walking one space in one of the
 * four cardinal directions. You cannot walk outside the grid, or walk into a wall.
 *
 * If you walk over a key, you can pick it up and you cannot walk over a lock unless you have
 * its corresponding key.
 *
 * For some 1 <= k <= 6, there is exactly one lowercase and one uppercase letter of the first
 * k letters of the English alphabet in the grid. This means that there is exactly one key for
 * each lock, and one lock for each key; and also that the letters used to represent the keys
 * and locks were chosen in the same order as the English alphabet.
 *
 * Return the lowest number of moves to acquire all keys. If it is impossible, return -1.
 */

/**
 * @param {string[]} grid
 * @return {number}
 */
var shortestPathAllKeys = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  let startRow;
  let startCol;
  let totalKeys = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === '@') {
        startRow = i;
        startCol = j;
      } else if (/[a-f]/.test(grid[i][j])) {
        totalKeys++;
      }
    }
  }

  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  const queue = [[startRow, startCol, 0, 0]];
  const visited = new Set([`${startRow},${startCol},0`]);

  while (queue.length) {
    const [row, col, keys, steps] = queue.shift();
    if (keys === (1 << totalKeys) - 1) return steps;
    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;

      if (newRow >= 0 && newRow < rows && newCol >= 0
          && newCol < cols && grid[newRow][newCol] !== '#') {
        const cell = grid[newRow][newCol];
        const newKeys = /[a-f]/.test(cell)
          ? keys | (1 << (cell.charCodeAt(0) - 97))
          : keys;

        if (/[A-F]/.test(cell) && !(keys & (1 << (cell.charCodeAt(0) - 65)))) continue;
        const state = `${newRow},${newCol},${newKeys}`;
        if (!visited.has(state)) {
          visited.add(state);
          queue.push([newRow, newCol, newKeys, steps + 1]);
        }
      }
    }
  }

  return -1;
};
```

0865 Smallest Subtree With All The Deepest Nodes

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 865. Smallest Subtree with all the Deepest Nodes
 * https://leetcode.com/problems/smallest-subtree-with-all-the-deepest-nodes/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, the depth of each node is the shortest distance to the
root.
 *
 * Return the smallest subtree such that it contains all the deepest nodes in the original
tree.
 *
 * A node is called the deepest if it has the largest depth possible among any node in the
entire
 * tree.
 *
 * The subtree of a node is a tree consisting of that node, plus the set of all descendants of
 * that node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var subtreeWithAllDeepest = function(root) {
  return findDepthAndNode(root).node;

  function findDepthAndNode(node) {
    if (!node) return { depth: 0, node: null };

    const left = findDepthAndNode(node.left);
    const right = findDepthAndNode(node.right);

    if (left.depth === right.depth) {
      return { depth: left.depth + 1, node: node };
    }
    const deeper = left.depth > right.depth ? left : right;
    return { depth: deeper.depth + 1, node: deeper.node };
  }
};
```

0866 Prime Palindrome

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 866. Prime Palindrome
 * https://leetcode.com/problems/prime-palindrome/
 * Difficulty: Medium
 *
 * Given an integer n, return the smallest prime palindrome greater than or equal to n.
 *
 * An integer is prime if it has exactly two divisors: 1 and itself. Note that 1 is not a
prime
```

```
 * number.
 * - For example, 2, 3, 5, 7, 11, and 13 are all primes.
 *
 * An integer is a palindrome if it reads the same from left to right as it does from right to
 left.
 * - For example, 101 and 12321 are palindromes.
 *
 * The test cases are generated so that the answer always exists and is in the range [2, 2 *
108].
 */

/**
 * @param {number} n
 * @return {number}
 */
var primePalindrome = function(start) {
  while (true) {
    const numStr = start.toString();
    if (numStr.length % 2 === 0 && start > 11) {
      start = 10 ** Math.ceil(Math.log10(start + 1));
      continue;
    }
    if (!isPalindrome(numStr)) {
      start++;
      continue;
    }
    if (isPrime(start)) return start;
    start++;
  }
};

function isPrime(num) {
  if (num <= 1) return false;
  if (num <= 3) return true;
  if (num % 2 === 0 || num % 3 === 0) return false;
  for (let i = 3; i <= Math.sqrt(num) + 1; i += 2) {
    if (num % i === 0) return false;
  }
  return true;
}

function isPalindrome(str) {
  let left = 0;
  let right = str.length - 1;
  while (left < right) {
    if (str[left++] !== str[right--]) return false;
  }
  return true;
}
```

0867 Transpose Matrix

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 867. Transpose Matrix
 * https://leetcode.com/problems/transpose-matrix/
 * Difficulty: Easy
 *
 * Given a matrix A, return the transpose of A.
 *
 * The transpose of a matrix is the matrix flipped over it's main
 * diagonal, switching the row and column indices of the matrix.
 */

/**
 * @param {number[][]} A
 * @return {number[][]}
 */
var transpose = function(A) {
```

```
  return A[0].map((_,  i) => A.map(j => j[i]));
};
```

0868 Binary Gap

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 868. Binary Gap
 * https://leetcode.com/problems/binary-gap/
 * Difficulty: Easy
 *
 * Given a positive integer n, find and return the longest distance between any two adjacent
1's
 * in the binary representation of n. If there are no two adjacent 1's, return 0.
 *
 * Two 1's are adjacent if there are only 0's separating them (possibly no 0's). The distance
 * between two 1's is the absolute difference between their bit positions. For example, the
two
 * 1's in "1001" have a distance of 3.
 */

/**
 * @param {number} n
 * @return {number}
 */
var binaryGap = function(n) {
  return Math.max(0, ...n.toString(2).split('1').slice(1, -1).map(s => s.length + 1));
};
```

0869 Reordered Power Of 2

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 869. Reordered Power of 2
 * https://leetcode.com/problems/reordered-power-of-2/
 * Difficulty: Medium
 *
 * You are given an integer n. We reorder the digits in any order (including the original
order)
 * such that the leading digit is not zero.
 *
 * Return true if and only if we can do this so that the resulting number is a power of two.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var reorderedPowerOf2 = function(n) {
  const numSignature = getDigitCount(n);
  const maxPower = Math.floor(Math.log2(10 ** 9));

  for (let i = 0; i <= maxPower; i++) {
    if (getDigitCount(1 << i) === numSignature) return true;
  }

  return false;

  function getDigitCount(num) {
    const count = Array(10).fill(0);
    while (num > 0) {
      count[num % 10]++;
      num = Math.floor(num / 10);
```

```
    }
    return count.join('');
  }
};
```

0870 Advantage Shuffle

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 870. Advantage Shuffle
 * https://leetcode.com/problems/advantage-shuffle/
 * Difficulty: Medium
 *
 * You are given two integer arrays nums1 and nums2 both of the same length. The advantage of
nums1
 * with respect to nums2 is the number of indices i for which nums1[i] > nums2[i].
 *
 * Return any permutation of nums1 that maximizes its advantage with respect to nums2.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var advantageCount = function(nums1, nums2) {
  const sortedNums1 = [...nums1].sort((a, b) => a - b);
  const indexedNums2 = nums2.map((val, idx) => ({ val, idx }))
    .sort((a, b) => a.val - b.val);

  const result = new Array(nums1.length);
  let left = 0;
  let right = nums1.length - 1;

  for (let i = nums1.length - 1; i >= 0; i--) {
    const current = indexedNums2[i];
    if (sortedNums1[right] > current.val) {
      result[current.idx] = sortedNums1[right--];
    } else {
      result[current.idx] = sortedNums1[left++];
    }
  }

  return result;
};
```

0871 Minimum Number Of Refueling Stops

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 871. Minimum Number of Refueling Stops
 * https://leetcode.com/problems/minimum-number-of-refueling-stops/
 * Difficulty: Hard
 *
 * A car travels from a starting position to a destination which is target miles east of the
 * starting position.
 *
 * There are gas stations along the way. The gas stations are represented as an array stations
 * where stations[i] = [positioni, fueli] indicates that the ith gas station is positioni
miles
 * east of the starting position and has fueli liters of gas.
 *
 * The car starts with an infinite tank of gas, which initially has startFuel liters of fuel
```

```
in
 * it. It uses one liter of gas per one mile that it drives. When the car reaches a gas
station,
 * it may stop and refuel, transferring all the gas from the station into the car.
 *
 * Return the minimum number of refueling stops the car must make in order to reach its
destination.
 * If it cannot reach the destination, return -1.
 *
 * Note that if the car reaches a gas station with 0 fuel left, the car can still refuel
there.
 * If the car reaches the destination with 0 fuel left, it is still considered to have
arrived.
 */

/**
 * @param {number} target
 * @param {number} startFuel
 * @param {number[][]} stations
 * @return {number}
 */
var minRefuelStops = function(target, startFuel, stations) {
  const dp = [startFuel];
  const stationCount = stations.length;

  for (let i = 0; i < stationCount; i++) {
    dp.push(0);
    for (let stops = i; stops >= 0; stops--) {
      if (dp[stops] >= stations[i][0]) {
        dp[stops + 1] = Math.max(dp[stops + 1], dp[stops] + stations[i][1]);
      }
    }
  }

  for (let stops = 0; stops <= stationCount; stops++) {
    if (dp[stops] >= target) return stops;
  }

  return -1;
};
```

0872 Leaf Similar Trees

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 872. Leaf-Similar Trees
 * https://leetcode.com/problems/leaf-similar-trees/
 * Difficulty: Easy
 *
 * Consider all the leaves of a binary tree, from left to right order, the values of
 * those leaves form a leaf value sequence.
 *
 * For example, in the given tree above, the leaf value sequence is (6, 7, 4, 9, 8).
 *
 * Two binary trees are considered leaf-similar if their leaf value sequence is the same.
 *
 * Return true if and only if the two given trees with head nodes root1 and root2 are
 * leaf-similar.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
```

```
 * @param {TreeNode} root1
 * @param {TreeNode} root2
 * @return {boolean}
 */
var leafSimilar = function(root1, root2) {
  return traverse(root1, []).toString() === traverse(root2, []).toString();
};

function traverse(root, result) {
  if (!root.left && !root.right) {
    result.push(root.val);
  }
  if (root.left) {
    traverse(root.left, result);
  }
  if (root.right) {
    traverse(root.right, result);
  }
  return result;
}
```

0873 Length Of Longest Fibonacci Subsequence

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 873. Length of Longest Fibonacci Subsequence
 * https://leetcode.com/problems/length-of-longest-fibonacci-subsequence/
 * Difficulty: Medium
 *
 * A sequence x1, x2, ..., xn is Fibonacci-like if:
 * - n >= 3
 * - xi + xi+1 == xi+2 for all i + 2 <= n
 *
 * Given a strictly increasing array arr of positive integers forming a sequence, return the
length
 * of the longest Fibonacci-like subsequence of arr. If one does not exist, return 0.
 *
 * A subsequence is derived from another sequence arr by deleting any number of elements
(including
 * none) from arr, without changing the order of the remaining elements. For example, [3, 5,
8] is
 * a subsequence of [3, 4, 5, 6, 7, 8].
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var lenLongestFibSubseq = function(arr) {
  const set = new Set(arr);
  let max = 0;

  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      let [x, y, count] = [arr[i], arr[j], 2];

      while (set.has(x + y)) {
        const next = x + y;
        x = y;
        y = next;
        count++;
      }

      max = Math.max(max, count);
    }
  }

  return max >= 3 ? max : 0;
```

```
};
```

0874 Walking Robot Simulation

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 874. Walking Robot Simulation
 * https://leetcode.com/problems/walking-robot-simulation/
 * Difficulty: Medium
 *
 * A robot on an infinite XY-plane starts at point (0, 0) facing north. The robot receives an
array
 * of integers commands, which represents a sequence of moves that it needs to execute. There
are
 * only three possible types of instructions the robot can receive:
 * - -2: Turn left 90 degrees.
 * - -1: Turn right 90 degrees.
 * - 1 <= k <= 9: Move forward k units, one unit at a time.
 *
 * Some of the grid squares are obstacles. The ith obstacle is at grid point
 * obstacles[i] = (xi, yi). If the robot runs into an obstacle, it will stay in its current
location
 * (on the block adjacent to the obstacle) and move onto the next command.
 *
 * Return the maximum squared Euclidean distance that the robot reaches at any point in its
path
 * (i.e. if the distance is 5, return 25).
 *
 * Note:
 * - There can be an obstacle at (0, 0). If this happens, the robot will ignore the obstacle
until
 *   it has moved off the origin. However, it will be unable to return to (0, 0) due to the
 *   obstacle.
 * - North means +Y direction.
 * - East means +X direction.
 * - South means -Y direction.
 * - West means -X direction.
 */

/**
 * @param {number[]} commands
 * @param {number[][]} obstacles
 * @return {number}
 */
var robotSim = function(commands, obstacles) {
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  const obstacleSet = new Set(obstacles.map(([x, y]) => `${x},${y}`));

  let x = 0;
  let y = 0;
  let directionIndex = 0;
  let result = 0;

  for (const command of commands) {
    if (command === -1) {
      directionIndex = (directionIndex + 1) % 4;
    } else if (command === -2) {
      directionIndex = (directionIndex + 3) % 4;
    } else {
      const [dx, dy] = directions[directionIndex];
      for (let step = 0; step < command; step++) {
        const nextX = x + dx;
        const nextY = y + dy;
        if (obstacleSet.has(`${nextX},${nextY}`)) break;
        x = nextX;
        y = nextY;
        result = Math.max(result, x * x + y * y);
      }
    }
```

```
  }

  return result;
};
```

0875 Koko Eating Bananas

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 875. Koko Eating Bananas
 * https://leetcode.com/problems/koko-eating-bananas/
 * Difficulty: Medium
 *
 * Koko loves to eat bananas. There are n piles of bananas, the ith pile has piles[i] bananas.
 * The guards have gone and will come back in h hours.
 *
 * Koko can decide her bananas-per-hour eating speed of k. Each hour, she chooses some pile of
 * bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats
 * all of them instead and will not eat any more bananas during this hour.
 *
 * Koko likes to eat slowly but still wants to finish eating all the bananas before the guards
 * return.
 *
 * Return the minimum integer k such that she can eat all the bananas within h hours.
 */

/**
 * @param {number[]} piles
 * @param {number} h
 * @return {number}
 */
var minEatingSpeed = function(piles, h) {
  const fn = speed => piles.reduce((sum, pile) => sum + Math.ceil(pile / speed), 0);
  let min = 1;
  let max = Math.max(...piles);
  let result = max;

  while (min <= max) {
    const middle = Math.floor((min + max) / 2);

    if (fn(middle) <= h) {
      result = middle;
      max = middle - 1;
    } else {
      min = middle + 1;
    }
  }

  return result;
};
```

0876 Middle Of The Linked List

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 876. Middle of the Linked List
 * https://leetcode.com/problems/middle-of-the-linked-list/
 * Difficulty: Easy
 *
 * Given the head of a singly linked list, return the middle node of the linked list.
 *
 * If there are two middle nodes, return the second middle node.
 */
```

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var middleNode = function(head) {
  let slow = head;
  let fast = head;

  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
  }

  return slow;
};
```

0877 Stone Game

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 877. Stone Game
 * https://leetcode.com/problems/stone-game/
 * Difficulty: Medium
 *
 * Alice and Bob play a game with piles of stones. There are an even number of piles arranged
in a
 * row, and each pile has a positive integer number of stones piles[i].
 *
 * The objective of the game is to end with the most stones. The total number of stones across
 all
 * the piles is odd, so there are no ties.
 *
 * Alice and Bob take turns, with Alice starting first. Each turn, a player takes the entire
pile
 * of stones either from the beginning or from the end of the row. This continues until there
are
 * no more piles left, at which point the person with the most stones wins.
 *
 * Assuming Alice and Bob play optimally, return true if Alice wins the game, or false if Bob
wins.
 */

/**
 * @param {number[]} piles
 * @return {boolean}
 */
var stoneGame = function(piles) {
  return true;
};
```

0878 Nth Magical Number

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 878. Nth Magical Number
```

```
 * https://leetcode.com/problems/nth-magical-number/
 * Difficulty: Hard
 *
 * A positive integer is magical if it is divisible by either a or b.
 *
 * Given the three integers n, a, and b, return the nth magical number. Since the answer may
be
 * very large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} a
 * @param {number} b
 */
var nthMagicalNumber = function(n, a, b) {
  const MOD = 1e9 + 7;
  const lcm = (a * b) / gcd(a, b);
  const cycleNumbers = lcm / a + lcm / b - 1;
  const cycles = Math.floor(n / cycleNumbers);
  const remainder = n % cycleNumbers;

  const result = (cycles * lcm) % MOD;
  if (remainder === 0) return result;

  let left = 1;
  let right = lcm;
  while (left < right) {
    const mid = left + Math.floor((right - left) / 2);
    const count = Math.floor(mid / a) + Math.floor(mid / b) - Math.floor(mid / lcm);
    if (count < remainder) left = mid + 1;
    else right = mid;
  }

  return (result + left) % MOD;

  function gcd(x, y) {
    return y === 0 ? x : gcd(y, x % y);
  }
};
```

0879 Profitable Schemes

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 879. Profitable Schemes
 * https://leetcode.com/problems/profitable-schemes/
 * Difficulty: Hard
 *
 * There is a group of n members, and a list of various crimes they could commit. The ith
crime
 * generates a profit[i] and requires group[i] members to participate in it. If a member
 * participates in one crime, that member can't participate in another crime.
 *
 * Let's call a profitable scheme any subset of these crimes that generates at least minProfit
 * profit, and the total number of members participating in that subset of crimes is at most
n.
 *
 * Return the number of schemes that can be chosen. Since the answer may be very large, return
 * it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} minProfit
 * @param {number[]} group
 * @param {number[]} profit
 * @return {number}
 */
```

```
var profitableSchemes = function(n, minProfit, group, profit) {
  const MOD = 1e9 + 7;
  const dp = Array.from({ length: group.length + 1 }, () =>
    Array.from({ length: n + 1 }, () => new Array(minProfit + 1).fill(0))
  );

  dp[0][0][0] = 1;

  for (let crime = 1; crime <= group.length; crime++) {
    const membersNeeded = group[crime - 1];
    const profitGained = profit[crime - 1];

    for (let members = 0; members <= n; members++) {
      for (let currentProfit = 0; currentProfit <= minProfit; currentProfit++) {
        dp[crime][members][currentProfit] = dp[crime - 1][members][currentProfit];

        if (members >= membersNeeded) {
          const prevProfit = Math.max(0, currentProfit - profitGained);
          dp[crime][members][currentProfit] = (dp[crime][members][currentProfit]
            + dp[crime - 1][members - membersNeeded][prevProfit]) % MOD;
        }
      }
    }
  }

  let result = 0;
  for (let members = 0; members <= n; members++) {
    result = (result + dp[group.length][members][minProfit]) % MOD;
  }

  return result;
};
```

0880 Decoded String At Index

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 880. Decoded String at Index
 * https://leetcode.com/problems/decoded-string-at-index/
 * Difficulty: Medium
 *
 * You are given an encoded string s. To decode the string to a tape, the encoded string is
read
 * one character at a time and the following steps are taken:
 * - If the character read is a letter, that letter is written onto the tape.
 * - If the character read is a digit d, the entire current tape is repeatedly written d - 1
 *   more times in total.
 *
 * Given an integer k, return the kth letter (1-indexed) in the decoded string.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var decodeAtIndex = function(s, k) {
  let tapeLength = 0;
  let i = 0;

  while (tapeLength < k) {
    if (isLetter(s[i])) {
      tapeLength++;
    } else {
      tapeLength *= parseInt(s[i]);
    }
    i++;
  }
```

```
  while (i--) {
    if (isLetter(s[i])) {
      if (tapeLength === k) return s[i];
      tapeLength--;
    } else {
      tapeLength = Math.floor(tapeLength / parseInt(s[i]));
      k = k % tapeLength || tapeLength;
    }
  }
};

function isLetter(char) {
  return /[a-z]/.test(char);
}
```

0881 Boats To Save People

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 881. Boats to Save People
 * https://leetcode.com/problems/boats-to-save-people/
 * Difficulty: Medium
 *
 * You are given an array people where people[i] is the weight of the ith person, and an
infinite
 * number of boats where each boat can carry a maximum weight of limit. Each boat carries at
 * most two people at the same time, provided the sum of the weight of those people is at most
 * limit.
 *
 * Return the minimum number of boats to carry every given person.
 */

/**
 * @param {number[]} people
 * @param {number} limit
 * @return {number}
 */
var numRescueBoats = function(people, limit) {
  people.sort((a, b) => a - b);
  let boats = 0;
  let left = 0;
  let right = people.length - 1;

  while (left <= right) {
    if (left < right && people[left] + people[right] <= limit) {
      left++;
      right--;
    } else {
      right--;
    }
    boats++;
  }

  return boats;
};
```

0882 Reachable Nodes In Subdivided Graph

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 882. Reachable Nodes In Subdivided Graph
 * https://leetcode.com/problems/reachable-nodes-in-subdivided-graph/
```

```
 * Difficulty: Hard
 *
 * You are given an undirected graph (the "original graph") with n nodes labeled from 0 to n -
 1.
 * You decide to subdivide each edge in the graph into a chain of nodes, with the number of
new
 * nodes varying between each edge.
 *
 * The graph is given as a 2D array of edges where edges[i] = [ui, vi, cnti] indicates that
there
 * is an edge between nodes ui and vi in the original graph, and cnti is the total number of
new
 * nodes that you will subdivide the edge into. Note that cnti == 0 means you will not
subdivide
 * the edge.
 *
 * To subdivide the edge [ui, vi], replace it with (cnti + 1) new edges and cnti new nodes.
The
 * new nodes are x1, x2, ..., xcnti, and the new edges are [ui, x1], [x1, x2], [x2, x3], ...,
 * [xcnti-1, xcnti], [xcnti, vi].
 *
 * In this new graph, you want to know how many nodes are reachable from the node 0, where a
node
 * is reachable if the distance is maxMoves or less.
 *
 * Given the original graph and maxMoves, return the number of nodes that are reachable from
node
 * 0 in the new graph.
 */

/**
 * @param {number[][]} edges
 * @param {number} maxMoves
 * @param {number} n
 * @return {number}
 */
var reachableNodes = function(edges, maxMoves, n) {
  const graph = {};
  for (let i = 0; i < n; i++) {
    graph[i] = [];
  }

  for (const [u, v, w] of edges) {
    graph[u].push([v, w]);
    graph[v].push([u, w]);
  }

  const dist = {};
  for (let i = 0; i < n; i++) {
    dist[i] = Infinity;
  }
  dist[0] = 0;

  const used = {};
  const pq = [[0, 0]];
  while (pq.length) {
    pq.sort((a, b) => a[0] - b[0]);
    const [d, node] = pq.shift();

    if (d > dist[node]) continue;

    for (const [nei, weight] of graph[node]) {
      const edgeId = `${Math.min(node, nei)}-${Math.max(node, nei)}`;
      const dirId = node < nei ? 0 : 1;

      if (!used[edgeId]) {
        used[edgeId] = [0, 0];
      }
      used[edgeId][dirId] = Math.min(maxMoves - d, weight);
      if (d + weight + 1 <= maxMoves && d + weight + 1 < dist[nei]) {
        dist[nei] = d + weight + 1;
        pq.push([dist[nei], nei]);
      }
    }
  }
```

```
  let result = 0;
  for (let i = 0; i < n; i++) {
    if (dist[i] <= maxMoves) result++;
  }

  for (const [u, v, cnt] of edges) {
    const edgeId = `${Math.min(u, v)}-${Math.max(u, v)}`;
    if (used[edgeId]) {
      const totalUsed = used[edgeId][0] + used[edgeId][1];
      result += Math.min(cnt, totalUsed);
    }
  }

  return result;
};
```

0883 Projection Area Of 3D Shapes

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 883. Projection Area of 3D Shapes
 * https://leetcode.com/problems/projection-area-of-3d-shapes/
 * Difficulty: Easy
 *
 * You are given an n x n grid where we place some 1 x 1 x 1 cubes that are axis-aligned
 * with the x, y, and z axes.
 *
 * Each value v = grid[i][j] represents a tower of v cubes placed on top of the cell (i, j).
 *
 * We view the projection of these cubes onto the xy, yz, and zx planes.
 *
 * A projection is like a shadow, that maps our 3-dimensional figure to a 2-dimensional plane.
 * We are viewing the "shadow" when looking at the cubes from the top, the front, and the
side.
 *
 * Return the total area of all three projections.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var projectionArea = function(grid) {
  let topView = 0;
  let frontView = 0;
  let sideView = 0;

  for (let i = 0; i < grid.length; i++) {
    let maxRow = 0;
    let maxCol = 0;

    for (let j = 0; j < grid.length; j++) {
      if (grid[i][j] > 0) topView++;
      maxRow = Math.max(maxRow, grid[i][j]);
      maxCol = Math.max(maxCol, grid[j][i]);
    }

    frontView += maxRow;
    sideView += maxCol;
  }

  return topView + frontView + sideView;
};
```

0884 Uncommon Words From Two Sentences

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 884. Uncommon Words from Two Sentences
 * https://leetcode.com/problems/uncommon-words-from-two-sentences/
 * Difficulty: Easy
 *
 * A sentence is a string of single-space separated words where each word
 * consists only of lowercase letters.
 *
 * A word is uncommon if it appears exactly once in one of the sentences,
 * and does not appear in the other sentence.
 *
 * Given two sentences s1 and s2, return a list of all the uncommon words.
 * You may return the answer in any order.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {string[]}
 */
var uncommonFromSentences = function(s1, s2) {
  const map = new Map();
  (s1 + ' ' + s2).split(/\s+/).forEach(s => map.set(s, (map.get(s) || 0) + 1));
  return [...map].reduce((a, [k, c]) => c === 1 ? [...a, k] : a, []);
};
```

0885 Spiral Matrix Iii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 885. Spiral Matrix III
 * https://leetcode.com/problems/spiral-matrix-iii/
 * Difficulty: Medium
 *
 * You start at the cell (rStart, cStart) of an rows x cols grid facing east. The northwest
corner
 * is at the first row and column in the grid, and the southeast corner is at the last row and
 * column.
 *
 * You will walk in a clockwise spiral shape to visit every position in this grid. Whenever
you
 * move outside the grid's boundary, we continue our walk outside the grid (but may return to
the
 * grid boundary later.). Eventually, we reach all rows * cols spaces of the grid.
 *
 * Return an array of coordinates representing the positions of the grid in the order you
visited
 * them.
 */

/**
 * @param {number} rows
 * @param {number} cols
 * @param {number} rStart
 * @param {number} cStart
 * @return {number[][]}
 */
var spiralMatrixIII = function(rows, cols, rStart, cStart) {
  const result = [];
  const total = rows * cols;
```

```
  let row = rStart;
  let col = cStart;
  let step = 1;
  let direction = 0;
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  result.push([row, col]);

  while (result.length < total) {
    for (let i = 0; i < 2; i++) {
      const [dr, dc] = directions[direction];
      for (let j = 0; j < step; j++) {
        row += dr;
        col += dc;
        if (row >= 0 && row < rows && col >= 0 && col < cols) {
          result.push([row, col]);
        }
      }
      direction = (direction + 1) % 4;
    }
    step++;
  }

  return result;
};
```

0886 Possible Bipartition

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 886. Possible Bipartition
 * https://leetcode.com/problems/possible-bipartition/
 * Difficulty: Medium
 *
 * We want to split a group of n people (labeled from 1 to n) into two groups of any size.
 * Each person may dislike some other people, and they should not go into the same group.
 *
 * Given the integer n and the array dislikes where dislikes[i] = [ai, bi] indicates that
 * the person labeled ai does not like the person labeled bi, return true if it is possible
 * to split everyone into two groups in this way.
 */

/**
 * @param {number} n
 * @param {number[][]} dislikes
 * @return {boolean}
 */
var possibleBipartition = function(n, dislikes) {
  const graph = Array.from({ length: n + 1 }, () => []);
  dislikes.forEach(([a, b]) => {
    graph[a].push(b);
    graph[b].push(a);
  });

  const colors = new Array(n + 1).fill(0);

  function colorGraph(person, color) {
    colors[person] = color;
    for (const neighbor of graph[person]) {
      if (colors[neighbor] === color) return false;
      if (colors[neighbor] === 0 && !colorGraph(neighbor, -color)) return false;
    }
    return true;
  }

  for (let person = 1; person <= n; person++) {
    if (colors[person] === 0 && !colorGraph(person, 1)) return false;
  }
```

```
  return true;
};
```

0887 Super Egg Drop

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 887. Super Egg Drop
 * https://leetcode.com/problems/super-egg-drop/
 * Difficulty: Hard
 *
 * You are given k identical eggs and you have access to a building with n floors labeled
 * from 1 to n.
 *
 * You know that there exists a floor f where 0 <= f <= n such that any egg dropped at a
 * floor higher than f will break, and any egg dropped at or below floor f will not break.
 *
 * Each move, you may take an unbroken egg and drop it from any floor x (where 1 <= x <= n).
 * If the egg breaks, you can no longer use it. However, if the egg does not break, you may
 * reuse it in future moves.
 *
 * Return the minimum number of moves that you need to determine with certainty what the
 * value of f is.
 */

/**
 * @param {number} k
 * @param {number} n
 * @return {number}
 */
var superEggDrop = function(k, n) {
  const dp = new Array(k + 1).fill().map(() => new Array(n + 1).fill(0));

  function solve(eggs, floors) {
    if (floors === 0 || floors === 1) return floors;
    if (eggs === 1) return floors;
    if (dp[eggs][floors]) return dp[eggs][floors];

    let minMoves = floors;
    let low = 1;
    let high = floors;

    while (low <= high) {
      const middle = Math.floor((low + high) / 2);
      const breakCase = solve(eggs - 1, middle - 1);
      const noBreakCase = solve(eggs, floors - middle);
      const moves = 1 + Math.max(breakCase, noBreakCase);

      minMoves = Math.min(minMoves, moves);
      if (breakCase < noBreakCase) low = middle + 1;
      else high = middle - 1;
    }

    dp[eggs][floors] = minMoves;
    return minMoves;
  }

  return solve(k, n);
};
```

0888 Fair Candy Swap

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 888. Fair Candy Swap
 * https://leetcode.com/problems/fair-candy-swap/
 * Difficulty: Easy
 *
 * Alice and Bob have a different total number of candies. You are given two integer arrays
 * aliceSizes and bobSizes where aliceSizes[i] is the number of candies of the ith box of
 * candy that Alice has and bobSizes[j] is the number of candies of the jth box of candy
 * that Bob has.
 *
 * Since they are friends, they would like to exchange one candy box each so that after the
 * exchange, they both have the same total amount of candy. The total amount of candy a
 * person has is the sum of the number of candies in each box they have.
 *
 * Return an integer array answer where answer[0] is the number of candies in the box that
 * Alice must exchange, and answer[1] is the number of candies in the box that Bob must
 * exchange. If there are multiple answers, you may return any one of them. It is guaranteed
 * that at least one answer exists.
 */

/**
 * @param {number[]} aliceSizes
 * @param {number[]} bobSizes
 * @return {number[]}
 */
var fairCandySwap = function(aliceSizes, bobSizes) {
  const aliceTotal = aliceSizes.reduce((sum, size) => sum + size, 0);
  const bobTotal = bobSizes.reduce((sum, size) => sum + size, 0);
  const diff = (aliceTotal - bobTotal) / 2;
  const bobSet = new Set(bobSizes);

  for (const aliceBox of aliceSizes) {
    const targetBobBox = aliceBox - diff;
    if (bobSet.has(targetBobBox)) {
      return [aliceBox, targetBobBox];
    }
  }
};
```

0889 Construct Binary Tree From Preorder And Postorder Traversal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 889. Construct Binary Tree from Preorder and Postorder Traversal
 * https://leetcode.com/problems/construct-binary-tree-from-preorder-and-postorder-traversal/
 * Difficulty: Medium
 *
 * Given two integer arrays, preorder and postorder where preorder is the preorder traversal
of a
 * binary tree of distinct values and postorder is the postorder traversal of the same tree,
 * reconstruct and return the binary tree.
 *
 * If there exist multiple answers, you can return any of them.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} preorder
 * @param {number[]} postorder
 * @return {TreeNode}
 */
var constructFromPrePost = function(preorder, postorder) {
```

```
  if (!preorder.length) return null;

  const root = new TreeNode(preorder[0]);
  if (preorder.length === 1) return root;

  const leftSize = postorder.indexOf(preorder[1]) + 1;
  root.left = constructFromPrePost(
    preorder.slice(1, leftSize + 1),
    postorder.slice(0, leftSize)
  );
  root.right = constructFromPrePost(
    preorder.slice(leftSize + 1),
    postorder.slice(leftSize, -1)
  );

  return root;
};
```

0890 Find And Replace Pattern

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 890. Find and Replace Pattern
 * https://leetcode.com/problems/find-and-replace-pattern/
 * Difficulty: Medium
 *
 * You have a list of words and a pattern, and you want to know which words in words matches
the pattern.
 *
 * A word matches the pattern if there exists a permutation of letters p so that after
replacing every
 * letter x in the pattern with p(x), we get the desired word.
 *
 * (Recall that a permutation of letters is a bijection from letters to letters: every letter
maps to
 * another letter, and no two letters map to the same letter.)
 *
 * Return a list of the words in words that match the given pattern.
 *
 * You may return the answer in any order.
 */

/**
 * @param {string[]} words
 * @param {string} pattern
 * @return {string[]}
 */
var findAndReplacePattern = function(words, pattern) {
  const map = (p, o = {}, count = 0) => p.split('').map(c => o[c] = o[c] ||
String(count++)).join('');
  return words.filter(word => map(word) === map(pattern));
};
```

0891 Sum Of Subsequence Widths

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 891. Sum of Subsequence Widths
 * https://leetcode.com/problems/sum-of-subsequence-widths/
 * Difficulty: Hard
 *
 * The width of a sequence is the difference between the maximum and minimum elements
 * in the sequence.
```

```
 *
 * Given an array of integers nums, return the sum of the widths of all the non-empty
 * subsequences of nums. Since the answer may be very large, return it modulo 109 + 7.
 *
 * A subsequence is a sequence that can be derived from an array by deleting some or
 * no elements without changing the order of the remaining elements. For example,
 * [3,6,2,7] is a subsequence of the array [0,3,1,6,2,2,7].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumSubseqWidths = function(nums) {
  const MOD = 1e9 + 7;
  const n = nums.length;
  const powers = [1];

  nums.sort((a, b) => a - b);
  for (let i = 1; i < n; i++) {
    powers[i] = (powers[i - 1] * 2) % MOD;
  }

  let result = 0;
  for (let i = 0; i < n; i++) {
    const contribution = (nums[i] * (powers[i] - powers[n - 1 - i] + MOD)) % MOD;
    result = (result + contribution) % MOD;
  }

  return result;
};
```

0892 Surface Area Of 3D Shapes

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 892. Surface Area of 3D Shapes
 * https://leetcode.com/problems/surface-area-of-3d-shapes/
 * Difficulty: Easy
 *
 * You are given an n x n grid where you have placed some 1 x 1 x 1 cubes. Each value
 * v = grid[i][j] represents a tower of v cubes placed on top of cell (i, j).
 *
 * After placing these cubes, you have decided to glue any directly adjacent cubes to
 * each other, forming several irregular 3D shapes.
 *
 * Return the total surface area of the resulting shapes.
 *
 * Note: The bottom face of each shape counts toward its surface area.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var surfaceArea = function(grid) {
  let result = 0;

  for (let row = 0; row < grid.length; row++) {
    for (let col = 0; col < grid.length; col++) {
      if (grid[row][col]) {
        result += 2 + 4 * grid[row][col];

        result -= row > 0 ? Math.min(grid[row][col], grid[row - 1][col]) * 2 : 0;
        result -= col > 0 ? Math.min(grid[row][col], grid[row][col - 1]) * 2 : 0;
      }
    }
  }
```

```
  return result;
};
```

0893 Groups Of Special Equivalent Strings

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 893. Groups of Special-Equivalent Strings
 * https://leetcode.com/problems/groups-of-special-equivalent-strings/
 * Difficulty: Medium
 *
 * You are given an array of strings of the same length words.
 *
 * In one move, you can swap any two even indexed characters or any two odd indexed characters
 of
 * a string words[i].
 *
 * Two strings words[i] and words[j] are special-equivalent if after any number of moves,
 * words[i] == words[j].
 * - For example, words[i] = "zzxy" and words[j] = "xyzz" are special-equivalent because we
may
 *   make the moves "zzxy" -> "xzzy" -> "xyzz".
 *
 * A group of special-equivalent strings from words is a non-empty subset of words such that:
 * - Every pair of strings in the group are special equivalent, and
 * - The group is the largest size possible (i.e., there is not a string words[i] not in the
 *   group such that words[i] is special-equivalent to every string in the group).
 *
 * Return the number of groups of special-equivalent strings from words.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var numSpecialEquivGroups = function(words) {
  return new Set(words.map(normalizeWord)).size;

  function normalizeWord(word) {
    const evenChars = [];
    const oddChars = [];
    for (let i = 0; i < word.length; i++) {
      if (i % 2 === 0) evenChars.push(word[i]);
      else oddChars.push(word[i]);
    }
    return `${evenChars.sort().join('')}-${oddChars.sort().join('')}`;
  }
};
```

0894 All Possible Full Binary Trees

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 894. All Possible Full Binary Trees
 * https://leetcode.com/problems/all-possible-full-binary-trees/
 * Difficulty: Medium
 *
 * Given an integer n, return a list of all possible full binary trees with n nodes. Each node
 * of each tree in the answer must have Node.val == 0.
 *
 * Each element of the answer is the root node of one possible tree. You may return the final
 * list of trees in any order.
```

```
 *
 * A full binary tree is a binary tree where each node has exactly 0 or 2 children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number} n
 * @return {TreeNode[]}
 */
var allPossibleFBT = function(n) {
  const memo = new Map();
  return generateTrees(n);

  function generateTrees(nodes) {
    if (nodes % 2 === 0) return [];
    if (nodes === 1) return [new TreeNode(0)];
    if (memo.has(nodes)) return memo.get(nodes);

    const result = [];
    for (let leftNodes = 1; leftNodes < nodes - 1; leftNodes += 2) {
      const rightNodes = nodes - 1 - leftNodes;
      const leftTrees = generateTrees(leftNodes);
      const rightTrees = generateTrees(rightNodes);

      for (const left of leftTrees) {
        for (const right of rightTrees) {
          result.push(new TreeNode(0, left, right));
        }
      }
    }

    memo.set(nodes, result);
    return result;
  }
};
```

0895 Maximum Frequency Stack

HARD

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 895. Maximum Frequency Stack
 * https://leetcode.com/problems/maximum-frequency-stack/
 * Difficulty: Hard
 *
 * Design a stack-like data structure to push elements to the stack and pop the most frequent
 * element from the stack.
 *
 * Implement the FreqStack class:
 * - FreqStack() constructs an empty frequency stack.
 * - void push(int val) pushes an integer val onto the top of the stack.
 * - int pop() removes and returns the most frequent element in the stack.
 *   - If there is a tie for the most frequent element, the element closest to the stack's
 *     top is removed and returned.
 */

var FreqStack = function() {
  this.frequencyMap = new Map();
  this.groupMap = new Map();
  this.maxFrequency = 0;
};

/**
```

```
 * @param {number} val
 * @return {void}
 */
FreqStack.prototype.push = function(val) {
  const frequency = (this.frequencyMap.get(val) || 0) + 1;
  this.frequencyMap.set(val, frequency);
  this.maxFrequency = Math.max(this.maxFrequency, frequency);

  if (!this.groupMap.has(frequency)) {
    this.groupMap.set(frequency, []);
  }
  this.groupMap.get(frequency).push(val);
};

/**
 * @return {number}
 */
FreqStack.prototype.pop = function() {
  const topGroup = this.groupMap.get(this.maxFrequency);
  const val = topGroup.pop();

  this.frequencyMap.set(val, this.maxFrequency - 1);
  if (topGroup.length === 0) {
    this.groupMap.delete(this.maxFrequency);
    this.maxFrequency--;
  }

  return val;
};
```

0896 Monotonic Array

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 896. Monotonic Array
 * https://leetcode.com/problems/monotonic-array/
 * Difficulty: Easy
 *
 * An array is monotonic if it is either monotone increasing or monotone decreasing.
 *
 * An array nums is monotone increasing if for all i <= j, nums[i] <= nums[j]. An array nums
 * is monotone decreasing if for all i <= j, nums[i] >= nums[j].
 *
 * Given an integer array nums, return true if the given array is monotonic, or false
otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isMonotonic = function(nums) {
  const isIncreasing = nums.every((num, i) => i === 0 || num >= nums[i - 1]);
  const isDecreasing = nums.every((num, i) => i === 0 || num <= nums[i - 1]);
  return isIncreasing || isDecreasing;
};
```

0897 Increasing Order Search Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 897. Increasing Order Search Tree
 * https://leetcode.com/problems/increasing-order-search-tree/
```

```
 * Difficulty: Easy
 *
 * Given the root of a binary search tree, rearrange the tree in in-order so that the leftmost
 * node in the tree is now the root of the tree, and every node has no left child and only one
 * right child.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var increasingBST = function(root) {
  const result = new TreeNode(0);
  let current = result;

  inorderTraversal(root);
  return result.right;

  function inorderTraversal(node) {
    if (!node) return;

    inorderTraversal(node.left);
    current.right = new TreeNode(node.val);
    current = current.right;
    inorderTraversal(node.right);
  }
};
```

0898 Bitwise Ors Of Subarrays

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 898. Bitwise ORs of Subarrays
 * https://leetcode.com/problems/bitwise-ors-of-subarrays/
 * Difficulty: Medium
 *
 * Given an integer array arr, return the number of distinct bitwise ORs of all the non-empty
 * subarrays of arr.
 *
 * The bitwise OR of a subarray is the bitwise OR of each integer in the subarray. The bitwise
 * OR of a subarray of one integer is that integer.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var subarrayBitwiseORs = function(arr) {
  const results = new Set();
  let previous = new Set();

  for (const num of arr) {
    const current = new Set([num]);
    for (const prev of previous) {
      current.add(prev | num);
    }
    previous = current;
    previous.forEach(val => results.add(val));
  }
```

```
  return results.size;
};
```

0899 Orderly Queue

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 899. Orderly Queue
 * https://leetcode.com/problems/orderly-queue/
 * Difficulty: Hard
 *
 * You are given a string s and an integer k. You can choose one of the first k letters of s
 * and append it at the end of the string.
 *
 * Return the lexicographically smallest string you could have after applying the mentioned
 * step any number of moves.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var orderlyQueue = function(s, k) {
  if (k === 1) {
    let smallest = s;
    for (let i = 0; i < s.length; i++) {
      const rotated = s.slice(i) + s.slice(0, i);
      if (rotated < smallest) smallest = rotated;
    }
    return smallest;
  }
  return [...s].sort().join('');
};
```

0900 Rle Iterator

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 900. RLE Iterator
 * https://leetcode.com/problems/rle-iterator/
 * Difficulty: Medium
 *
 * We can use run-length encoding (i.e., RLE) to encode a sequence of integers. In a
run-length
 * encoded array of even length encoding (0-indexed), for all even i, encoding[i] tells us the
 * number of times that the non-negative integer value encoding[i + 1] is repeated in the
sequence.
 * - For example, the sequence arr = [8,8,8,5,5] can be encoded to be encoding = [3,8,2,5].
 *   encoding = [3,8,0,9,2,5] and encoding = [2,8,1,8,2,5] are also valid RLE of arr.
 *
 * Given a run-length encoded array, design an iterator that iterates through it.
 *
 * Implement the RLEIterator class:
 * - RLEIterator(int[] encoded) Initializes the object with the encoded array encoded.
 * - int next(int n) Exhausts the next n elements and returns the last element exhausted in
this
 *   way. If there is no element left to exhaust, return -1 instead.
 */

/**
 * @param {number[]} encoding
```

```
 */
var RLEIterator = function(encoding) {
  this.pairs = [];
  this.index = 0;
  this.count = 0;

  for (let i = 0; i < encoding.length; i += 2) {
    if (encoding[i] > 0) {
      this.pairs.push([encoding[i], encoding[i + 1]]);
    }
  }
};

/**
 * @param {number} n
 * @return {number}
 */
RLEIterator.prototype.next = function(n) {
  while (n > 0 && this.index < this.pairs.length) {
    const available = this.pairs[this.index][0] - this.count;

    if (n <= available) {
      this.count += n;
      return this.pairs[this.index][1];
    }

    n -= available;
    this.count = 0;
    this.index++;
  }

  return -1;
};
```

0901 Online Stock Span

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 901. Online Stock Span
 * https://leetcode.com/problems/online-stock-span/
 * Difficulty: Medium
 *
 * Design an algorithm that collects daily price quotes for some stock and returns the
 * span of that stock's price for the current day.
 *
 * The span of the stock's price in one day is the maximum number of consecutive days
 * (starting from that day and going backward) for which the stock price was less than
 * or equal to the price of that day.
 *
 * - For example, if the prices of the stock in the last four days is [7,2,1,2] and the
 *   price of the stock today is 2, then the span of today is 4 because starting from
 *   today, the price of the stock was less than or equal 2 for 4 consecutive days.
 *
 * - Also, if the prices of the stock in the last four days is [7,34,1,2] and the price
 *   of the stock today is 8, then the span of today is 3 because starting from today,
 *   the price of the stock was less than or equal 8 for 3 consecutive days.
 *
 * Implement the StockSpanner class:
 * - StockSpanner() Initializes the object of the class.
 * - int next(int price) Returns the span of the stock's price given that today's
 *   price is price.
 */

var StockSpanner = function() {
  this.stack = [];
};

/**
 * @param {number} price
```

```
 * @return {number}
 */
StockSpanner.prototype.next = function(price) {
  let count = 1;

  while (this.stack.length && price >= this.stack[this.stack.length - 1][0]) {
    count += this.stack[this.stack.length - 1][1];
    this.stack.pop();
  }

  this.stack.push([price, count]);

  return count;
};
```

0902 Numbers At Most N Given Digit Set

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 902. Numbers At Most N Given Digit Set
 * https://leetcode.com/problems/numbers-at-most-n-given-digit-set/
 * Difficulty: Hard
 *
 * Given an array of digits which is sorted in non-decreasing order. You can write numbers
using
 * each digits[i] as many times as we want. For example, if digits = ['1','3','5'], we may
write
 * numbers such as '13', '551', and '1351315'.
 *
 * Return the number of positive integers that can be generated that are less than or equal to
 * a given integer n.
 */

/**
 * @param {string[]} digits
 * @param {number} n
 * @return {number}
 */
var atMostNGivenDigitSet = function(digits, n) {
  const str = n.toString();
  const digitCount = digits.length;
  let result = 0;

  for (let i = 1; i < str.length; i++) {
    result += Math.pow(digitCount, i);
  }

  function countValid(prefixLen) {
    if (prefixLen === str.length) return 1;

    const currentDigit = str[prefixLen];
    let valid = 0;

    for (const digit of digits) {
      if (digit < currentDigit) {
        valid += Math.pow(digitCount, str.length - prefixLen - 1);
      } else if (digit === currentDigit) {
        valid += countValid(prefixLen + 1);
      } else {
        break;
      }
    }

    return valid;
  }

  return result + countValid(0);
};
```

0903 Valid Permutations For Di Sequence

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 903. Valid Permutations for DI Sequence
 * https://leetcode.com/problems/valid-permutations-for-di-sequence/
 * Difficulty: Hard
 *
 * You are given a string s of length n where s[i] is either:
 * - 'D' means decreasing, or
 * - 'I' means increasing.
 *
 * A permutation perm of n + 1 integers of all the integers in the range [0, n] is called a
 * valid permutation if for all valid i:
 * - If s[i] == 'D', then perm[i] > perm[i + 1], and
 * - If s[i] == 'I', then perm[i] < perm[i + 1].
 *
 * Return the number of valid permutations perm. Since the answer may be large, return it
 * modulo 109 + 7.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numPermsDISequence = function(s) {
  const MOD = 1e9 + 7;
  const n = s.length;
  const dp = Array.from({ length: n + 1 }, () => new Array(n + 1).fill(0));

  dp[0][0] = 1;

  for (let len = 1; len <= n; len++) {
    for (let curr = 0; curr <= len; curr++) {
      if (s[len - 1] === 'D') {
        for (let prev = curr; prev < len; prev++) {
          dp[len][curr] = (dp[len][curr] + dp[len - 1][prev]) % MOD;
        }
      } else {
        for (let prev = 0; prev < curr; prev++) {
          dp[len][curr] = (dp[len][curr] + dp[len - 1][prev]) % MOD;
        }
      }
    }
  }

  let result = 0;
  for (let num = 0; num <= n; num++) {
    result = (result + dp[n][num]) % MOD;
  }

  return result;
};
```

0904 Fruit Into Baskets

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 904. Fruit Into Baskets
 * https://leetcode.com/problems/fruit-into-baskets/
 * Difficulty: Medium
```

```
 *
 * You are visiting a farm that has a single row of fruit trees arranged from left to right.
 * The trees are represented by an integer array fruits where fruits[i] is the type of fruit
 * the ith tree produces.
 *
 * You want to collect as much fruit as possible. However, the owner has some strict rules
 * that you must follow:
 * - You only have two baskets, and each basket can only hold a single type of fruit. There
 *   is no limit on the amount of fruit each basket can hold.
 * - Starting from any tree of your choice, you must pick exactly one fruit from every tree
 *   (including the start tree) while moving to the right. The picked fruits must fit in one
 *   of your baskets.
 * - Once you reach a tree with fruit that cannot fit in your baskets, you must stop.
 *
 * Given the integer array fruits, return the maximum number of fruits you can pick.
 */

/**
 * @param {number[]} fruits
 * @return {number}
 */
var totalFruit = function(fruits) {
  const map = new Map();
  let result = 0;
  let start = 0;

  for (let end = 0; end < fruits.length; end++) {
    map.set(fruits[end], (map.get(fruits[end]) || 0) + 1);

    while (map.size > 2) {
      map.set(fruits[start], map.get(fruits[start]) - 1);
      if (map.get(fruits[start]) === 0) {
        map.delete(fruits[start]);
      }
      start++;
    }

    result = Math.max(result, end - start + 1);
  }

  return result;
};
```

0905 Sort Array By Parity

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 905. Sort Array By Parity
 * https://leetcode.com/problems/sort-array-by-parity/
 * Difficulty: Easy
 *
 * Given an integer array nums, move all the even integers at the beginning of
 * the array followed by all the odd integers.
 *
 * Return any array that satisfies this condition.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var sortArrayByParity = function(nums) {
  return nums.sort((a, b) => a % 2 === 0 ? -1 : 1);
};
```

0906 Super Palindromes

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 906. Super Palindromes
 * https://leetcode.com/problems/super-palindromes/
 * Difficulty: Hard
 *
 * Let's say a positive integer is a super-palindrome if it is a palindrome, and it is also
 * the square of a palindrome.
 *
 * Given two positive integers left and right represented as strings, return the number of
 * super-palindromes integers in the inclusive range [left, right].
 */

/**
 * @param {string} left
 * @param {string} right
 * @return {number}
 */
var superpalindromesInRange = function(lowerBound, upperBound) {
  let result = 9n >= lowerBound && 9n <= upperBound ? 1 : 0;

  const isPalindrome = sequence => {
    for (let start = 0, end = sequence.length - 1; start < end; start++, end--) {
      if (sequence[start] !== sequence[end]) return false;
    }
    return true;
  };

  for (let base = 1; base < 19684; base++) {
    const ternary = base.toString(3);
    if (isPalindrome(ternary)) {
      const square = BigInt(ternary) * BigInt(ternary);
      if (square > upperBound) return result;
      if (square >= lowerBound && isPalindrome(square.toString())) {
        result++;
      }
    }
  }

  return result;
};
```

0907 Sum Of Subarray Minimums

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 907. Sum of Subarray Minimums
 * https://leetcode.com/problems/sum-of-subarray-minimums/
 * Difficulty: Medium
 *
 * Given an array of integers arr, find the sum of min(b), where b ranges over every
(contiguous)
 * subarray of arr. Since the answer may be large, return the answer modulo 109 + 7.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var sumSubarrayMins = function(arr) {
  const MOD = 1e9 + 7;
```

```
  const left = new Array(arr.length);
  const right = new Array(arr.length);
  const stack = [];

  for (let i = 0; i < arr.length; i++) {
    while (stack.length && arr[stack[stack.length - 1]] > arr[i]) {
      stack.pop();
    }
    left[i] = stack.length ? i - stack[stack.length - 1] : i + 1;
    stack.push(i);
  }

  stack.length = 0;
  for (let i = arr.length - 1; i >= 0; i--) {
    while (stack.length && arr[stack[stack.length - 1]] >= arr[i]) {
      stack.pop();
    }
    right[i] = stack.length ? stack[stack.length - 1] - i : arr.length - i;
    stack.push(i);
  }

  let result = 0;
  for (let i = 0; i < arr.length; i++) {
    result = (result + arr[i] * left[i] * right[i]) % MOD;
  }

  return result;
};
```

0908 Smallest Range I

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 908. Smallest Range I
 * https://leetcode.com/problems/smallest-range-i/
 * Difficulty: Easy
 *
 * You are given an integer array nums and an integer k.
 *
 * In one operation, you can choose any index i where 0 <= i < nums.length and change nums[i]
to
 * nums[i] + x where x is an integer from the range [-k, k]. You can apply this operation at
most
 * once for each index i.
 *
 * The score of nums is the difference between the maximum and minimum elements in nums.
 *
 * Return the minimum score of nums after applying the mentioned operation at most once for
each
 * index in it.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var smallestRangeI = function(nums, k) {
  return Math.max(Math.max(...nums) - Math.min(...nums) - 2 * k, 0);
};
```

0909 Snakes And Ladders

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the

loop invariant.

SOURCE CODE

```
/**
 * 909. Snakes and Ladders
 * https://leetcode.com/problems/snakes-and-ladders/
 * Difficulty: Medium
 *
 * You are given an n x n integer matrix board where the cells are labeled from 1 to n2 in a
 * Boustrophedon style starting from the bottom left of the board (i.e. board[n - 1][0]) and
 * alternating direction each row.
 *
 * You start on square 1 of the board. In each move, starting from square curr, do the
following:
 * - Choose a destination square next with a label in the range [curr + 1, min(curr + 6, n2)].
 *   - This choice simulates the result of a standard 6-sided die roll: i.e., there are always
 at
 *     most 6 destinations, regardless of the size of the board.
 * - If next has a snake or ladder, you must move to the destination of that snake or ladder.
 *   Otherwise, you move to next.
 * - The game ends when you reach the square n2.
 *
 * A board square on row r and column c has a snake or ladder if board[r][c] != -1. The
destination
 * of that snake or ladder is board[r][c]. Squares 1 and n2 are not the starting points of any
 * snake or ladder.
 *
 * Note that you only take a snake or ladder at most once per dice roll. If the destination to
 a
 * snake or ladder is the start of another snake or ladder, you do not follow the subsequent
snake
 * or ladder.
 *
 * - For example, suppose the board is [[-1,4],[-1,3]], and on the first move, your
destination
 *   square is 2. You follow the ladder to square 3, but do not follow the subsequent ladder
to 4.
 *
 * Return the least number of dice rolls required to reach the square n2. If it is not
possible to
 * reach the square, return -1.
 */

/**
 * @param {number[][]} board
 * @return {number}
 */
var snakesAndLadders = function(board) {
  const target = board.length * board.length;
  const queue = [1];
  const seen = new Set([1]);
  let moves = 0;

  while (queue.length) {
    const size = queue.length;
    for (let i = 0; i < size; i++) {
      const current = queue.shift();
      if (current === target) return moves;
      for (let dice = 1; dice <= 6 && current + dice <= target; dice++) {
        const next = current + dice;
        const [r, c] = getCoordinates(next);
        const destination = board[r][c] === -1 ? next : board[r][c];
        if (!seen.has(destination)) {
          seen.add(destination);
          queue.push(destination);
        }
      }
    }
    moves++;
  }

  return -1;

  function getCoordinates(square) {
    const r = board.length - 1 - Math.floor((square - 1) / board.length);
    const c = (board.length - 1 - r) % 2 === 0
```

```
      ? (square - 1) % board.length
      : board.length - 1 - (square - 1) % board.length;
    return [r, c];
  }
};
```

0910 Smallest Range Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 910. Smallest Range II
 * https://leetcode.com/problems/smallest-range-ii/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k.
 *
 * For each index i where 0 <= i < nums.length, change nums[i] to be either
 * nums[i] + k or nums[i] - k.
 *
 * The score of nums is the difference between the maximum and minimum elements
 * in nums.
 *
 * Return the minimum score of nums after changing the values at each index.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var smallestRangeII = function(nums, k) {
  nums.sort((a, b) => a - b);

  let result = nums[nums.length - 1] - nums[0];
  for (let i = 0; i < nums.length - 1; i++) {
    const high = Math.max(nums[nums.length - 1] - k, nums[i] + k);
    const low = Math.min(nums[0] + k, nums[i + 1] - k);
    result = Math.min(result, high - low);
  }

  return result;
};
```

0911 Online Election

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 911. Online Election
 * https://leetcode.com/problems/online-election/
 * Difficulty: Medium
 *
 * You are given two integer arrays persons and times. In an election, the ith vote was cast
for
 * persons[i] at time times[i].
 *
 * For each query at a time t, find the person that was leading the election at time t. Votes
 * cast at time t will count towards our query. In the case of a tie, the most recent vote
(among
 * tied candidates) wins.
 *
 * Implement the TopVotedCandidate class:
 * - TopVotedCandidate(int[] persons, int[] times) Initializes the object with the persons and
```

```
 *   times arrays.
 * - int q(int t) Returns the number of the person that was leading the election at time t
according
 *   to the mentioned rules.
 */

/**
 * @param {number[]} persons
 * @param {number[]} times
 */
var TopVotedCandidate = function(persons, times) {
  this.times = times;
  this.leaders = [];
  const voteCount = new Map();
  let maxVotes = 0;
  let currentLeader = 0;

  for (let i = 0; i < persons.length; i++) {
    const votes = (voteCount.get(persons[i]) || 0) + 1;
    voteCount.set(persons[i], votes);

    if (votes >= maxVotes) {
      maxVotes = votes;
      currentLeader = persons[i];
    }
    this.leaders[i] = currentLeader;
  }
};

/**
 * @param {number} t
 * @return {number}
 */
TopVotedCandidate.prototype.q = function(t) {
  let left = 0;
  let right = this.times.length - 1;

  while (left <= right) {
    const middle = Math.floor((left + right) / 2);
    if (this.times[middle] <= t) {
      left = middle + 1;
    } else {
      right = middle - 1;
    }
  }

  return this.leaders[right];
};
```

0912 Sort An Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 912. Sort an Array
 * https://leetcode.com/problems/sort-an-array/
 * Difficulty: Medium
 *
 * Given an array of integers nums, sort the array in ascending order and return it.
 *
 * You must solve the problem without using any built-in functions in O(nlog(n)) time
 * complexity and with the smallest space complexity possible.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var sortArray = function(nums) {
  if (nums.length <= 1) {
```

```
    return nums;
  }

  const min = Math.min(...nums);
  const max = Math.max(...nums);
  const count = new Array(max - min + 1).fill(0);

  nums.forEach(n => count[n - min]++);

  for (let i = 0, j = 0; i < count.length; i++) {
    while (count[i]-- > 0) {
      nums[j++] = i + min;
    }
  }

  return nums;
};
```

0913 Cat And Mouse

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 913. Cat and Mouse
 * https://leetcode.com/problems/cat-and-mouse/
 * Difficulty: Hard
 *
 * A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns.
 *
 * The graph is given as follows: graph[a] is a list of all nodes b such that ab is an edge of
 * the graph.
 *
 * The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and
 * there is a hole at node 0.
 *
 * During each player's turn, they must travel along one edge of the graph that meets where
they
 * are. For example, if the Mouse is at node 1, it must travel to any node in graph[1].
 *
 * Additionally, it is not allowed for the Cat to travel to the Hole (node 0).
 *
 * Then, the game can end in three ways:
 * - If ever the Cat occupies the same node as the Mouse, the Cat wins.
 * - If ever the Mouse reaches the Hole, the Mouse wins.
 * - If ever a position is repeated (i.e., the players are in the same position as a previous
 *   turn, and it is the same player's turn to move), the game is a draw.
 *
 * Given a graph, and assuming both players play optimally, return
 * - 1 if the mouse wins the game,
 * - 2 if the cat wins the game, or
 * - 0 if the game is a draw.
 */

/**
 * @param {number[][]} graph
 * @return {number}
 */
var catMouseGame = function(graph) {
  const MOUSE_TURN = 0;
  const CAT_TURN = 1;
  const MOUSE_WIN = 1;
  const CAT_WIN = 2;
  const n = graph.length;

  const color = new Array(n).fill().map(() => {
    return new Array(n).fill().map(() => new Array(2).fill(0));
  });

  const degree = new Array(n).fill().map(() => {
    return new Array(n).fill().map(() => new Array(2).fill(0));
```

```
  });

  const queue = [];

  for (let i = 0; i < n; i++) {
    for (let turn = 0; turn < 2; turn++) {
      color[0][i][turn] = MOUSE_WIN;
      queue.push([0, i, turn, MOUSE_WIN]);

      if (i > 0) {
        color[i][i][turn] = CAT_WIN;
        queue.push([i, i, turn, CAT_WIN]);
      }
    }
  }

  for (let m = 0; m < n; m++) {
    for (let c = 0; c < n; c++) {
      degree[m][c][MOUSE_TURN] = graph[m].length;
      degree[m][c][CAT_TURN] = graph[c].length;

      for (let x = 0; x < graph[c].length; x++) {
        if (graph[c][x] === 0) {
          degree[m][c][CAT_TURN]--;
          break;
        }
      }
    }
  }

  while (queue.length > 0) {
    const [mouse, cat, turn, result] = queue.shift();

    const prevTurn = 1 - turn;
    const prevPositions = [];

    if (prevTurn === MOUSE_TURN) {
      for (const prevMouse of graph[mouse]) {
        prevPositions.push([prevMouse, cat]);
      }
    } else {
      for (const prevCat of graph[cat]) {
        if (prevCat !== 0) {
          prevPositions.push([mouse, prevCat]);
        }
      }
    }

    for (const [prevMouse, prevCat] of prevPositions) {
      if (color[prevMouse][prevCat][prevTurn] !== 0) continue;

      if ((prevTurn === MOUSE_TURN && result === MOUSE_WIN)
          || (prevTurn === CAT_TURN && result === CAT_WIN)) {
        color[prevMouse][prevCat][prevTurn] = result;
        queue.push([prevMouse, prevCat, prevTurn, result]);
      } else {
        degree[prevMouse][prevCat][prevTurn]--;
        if (degree[prevMouse][prevCat][prevTurn] === 0) {
          color[prevMouse][prevCat][prevTurn] = result;
          queue.push([prevMouse, prevCat, prevTurn, result]);
        }
      }
    }
  }

  return color[1][2][MOUSE_TURN];
};
```

0914 X Of A Kind In A Deck Of Cards

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 914. X of a Kind in a Deck of Cards
 * https://leetcode.com/problems/x-of-a-kind-in-a-deck-of-cards/
 * Difficulty: Medium
 *
 * You are given an integer array deck where deck[i] represents the number written on the ith
card.
 *
 * Partition the cards into one or more groups such that:
 * - Each group has exactly x cards where x > 1, and
 * - All the cards in one group have the same integer written on them.
 *
 * Return true if such partition is possible, or false otherwise.
 */

/**
 * @param {number[]} deck
 * @return {boolean}
 */
var hasGroupsSizeX = function(deck) {
  const gcd = (a, b) => !b ? a : gcd(b, a % b);
  const map = new Map();
  deck.forEach(n => map.set(n, (map.get(n) || 0) + 1));
  return [...map.values()].reduce(gcd) > 1;
};
```

0915 Partition Array Into Disjoint Intervals

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 915. Partition Array into Disjoint Intervals
 * https://leetcode.com/problems/partition-array-into-disjoint-intervals/
 * Difficulty: Medium
 *
 * Given an integer array nums, partition it into two (contiguous) subarrays left and right so
 that:
 * - Every element in left is less than or equal to every element in right.
 * - left and right are non-empty.
 * - left has the smallest possible size.
 *
 * Return the length of left after such a partitioning.
 *
 * Test cases are generated such that partitioning exists.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var partitionDisjoint = function(nums) {
  let leftMax = nums[0];
  let currentMax = nums[0];
  let partitionIndex = 0;

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] < leftMax) {
      partitionIndex = i;
      leftMax = currentMax;
    }
    currentMax = Math.max(currentMax, nums[i]);
  }

  return partitionIndex + 1;
};
```

0916 Word Subsets

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 916. Word Subsets
 * https://leetcode.com/problems/word-subsets/
 * Difficulty: Medium
 *
 * You are given two string arrays words1 and words2.
 *
 * A string b is a subset of string a if every letter in b occurs in a including multiplicity.
 *
 * For example, "wrr" is a subset of "warrior" but is not a subset of "world".
 *
 * A string a from words1 is universal if for every string b in words2, b is a subset of a.
 *
 * Return an array of all the universal strings in words1. You may return the answer in any
order.
 */

/**
 * @param {string[]} words1
 * @param {string[]} words2
 * @return {string[]}
 */
var wordSubsets = function(words1, words2) {
  const count = (string, char) => string.split(char).length - 1;
  const subset = Array.from(words2.reduce((map, b) => {
    b.split('').forEach(char => {
      map.set(char, (map.get(char) || 0) > count(b, char) ? map.get(char) : count(b, char));
    });
    return map;
  }, new Map()));
  return words1.filter(a => subset.every(match => count(a, match[0]) >= match[1]));
};
```

0917 Reverse Only Letters

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 917. Reverse Only Letters
 * https://leetcode.com/problems/reverse-only-letters/
 * Difficulty: Easy
 *
 * Given a string s, reverse the string according to the following rules:
 * - All the characters that are not English letters remain in the same position.
 * - All the English letters (lowercase or uppercase) should be reversed.
 *
 * Return s after reversing it.
 */

/**
 * @param {string} s
 * @return {string}
 */
var reverseOnlyLetters = function(s) {
  const chars = s.split('');
  let left = 0;
  let right = s.length - 1;

  while (left < right) {
    while (left < right && !/[a-zA-Z]/.test(chars[left])) {
      left++;
```

```
    }
    while (left < right && !/[a-zA-Z]/.test(chars[right])) {
      right--;
    }
    [chars[left], chars[right]] = [chars[right], chars[left]];
    left++;
    right--;
  }

  return chars.join('');
};
```

0918 Maximum Sum Circular Subarray

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 918. Maximum Sum Circular Subarray
 * https://leetcode.com/problems/maximum-sum-circular-subarray/
 * Difficulty: Medium
 *
 * Given a circular integer array nums of length n, return the maximum possible sum of
 * a non-empty subarray of nums.
 *
 * A circular array means the end of the array connects to the beginning of the array.
 * Formally, the next element of nums[i] is nums[(i + 1) % n] and the previous element
 * of nums[i] is nums[(i - 1 + n) % n].
 *
 * A subarray may only include each element of the fixed buffer nums at most once.
 * Formally, for a subarray nums[i], nums[i + 1], ..., nums[j], there does not exist
 * i <= k1, k2 <= j with k1 % n == k2 % n.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubarraySumCircular = function(nums) {
  let [min, max, sum] = [nums[0], nums[0], nums[0]];

  for (let i = 1, prevMin = nums[0], prevMax = nums[0]; i < nums.length; i++) {
    prevMax = Math.max(nums[i], prevMax + nums[i]);
    max = Math.max(max, prevMax);
    prevMin = Math.min(nums[i], prevMin + nums[i]);
    min = Math.min(min, prevMin);
    sum += nums[i];
  }

  return max > 0 ? Math.max(max, sum - min) : max;
};
```

0919 Complete Binary Tree Inserter

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 919. Complete Binary Tree Inserter
 * https://leetcode.com/problems/complete-binary-tree-inserter/
 * Difficulty: Medium
 *
 * A complete binary tree is a binary tree in which every level, except possibly the last,
 * is completely filled, and all nodes are as far left as possible.
 *
 * Design an algorithm to insert a new node to a complete binary tree keeping it complete
 * after the insertion.
```

```
 *
 * Implement the CBTInserter class:
 * - CBTInserter(TreeNode root) Initializes the data structure with the root of the complete
 *   binary tree.
 * - int insert(int v) Inserts a TreeNode into the tree with value Node.val == val so that
 *   the tree remains complete, and returns the value of the parent of the inserted TreeNode.
 * - TreeNode get_root() Returns the root node of the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 */
var CBTInserter = function(root) {
  this.root = root;
  this.deque = [];

  const queue = [root];
  while (queue.length) {
    const node = queue.shift();
    if (!node.left || !node.right) this.deque.push(node);
    if (node.left) queue.push(node.left);
    if (node.right) queue.push(node.right);
  }
};

/**
 * @param {number} val
 * @return {number}
 */
CBTInserter.prototype.insert = function(val) {
  const node = new TreeNode(val);
  const parent = this.deque[0];
  if (!parent.left) {
    parent.left = node;
  } else {
    parent.right = node;
    this.deque.shift();
  }
  this.deque.push(node);
  return parent.val;
};

/**
 * @return {TreeNode}
 */
CBTInserter.prototype.get_root = function() {
  return this.root;
};
```

0920 Number Of Music Playlists

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 920. Number of Music Playlists
 * https://leetcode.com/problems/number-of-music-playlists/
 * Difficulty: Hard
 *
 * Your music player contains n different songs. You want to listen to goal songs (not
necessarily
 * different) during your trip. To avoid boredom, you will create a playlist so that:
 * - Every song is played at least once.
```

```
 * - A song can only be played again only if k other songs have been played.
 *
 * Given n, goal, and k, return the number of possible playlists that you can create. Since
the
 * answer can be very large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} goal
 * @param {number} k
 * @return {number}
 */
var numMusicPlaylists = function(n, goal, k) {
  const MOD = 1e9 + 7;
  const dp = new Array(goal + 1).fill().map(() => new Array(n + 1).fill(0));
  dp[0][0] = 1;

  for (let songs = 1; songs <= goal; songs++) {
    for (let uniques = 1; uniques <= Math.min(songs, n); uniques++) {
      dp[songs][uniques] = dp[songs - 1][uniques - 1] * (n - (uniques - 1));
      if (uniques > k) {
        dp[songs][uniques] += dp[songs - 1][uniques] * (uniques - k);
      }
      dp[songs][uniques] %= MOD;
    }
  }

  return dp[goal][n];
};
```

0921 Minimum Add To Make Parentheses Valid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 921. Minimum Add to Make Parentheses Valid
 * https://leetcode.com/problems/minimum-add-to-make-parentheses-valid/
 * Difficulty: Medium
 *
 * A parentheses string is valid if and only if:
 * - It is the empty string,
 * - It can be written as AB (A concatenated with B), where A and B are valid strings, or
 * - It can be written as (A), where A is a valid string.
 *
 * You are given a parentheses string s. In one move, you can insert a parenthesis at any
position
 * of the string.
 * - For example, if s = "())", you can insert an opening parenthesis to be "(())" or a
closing
 *   parenthesis to be "()))".
 *
 * Return the minimum number of moves required to make s valid.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minAddToMakeValid = function(s) {
  let openCount = 0;
  let unmatchedClose = 0;

  for (const character of s) {
    if (character === '(') {
      openCount++;
    } else if (openCount > 0) {
      openCount--;
    } else {
      unmatchedClose++;
```

```
    }
  }

  return openCount + unmatchedClose;
};
```

0922 Sort Array By Parity Ii

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 922. Sort Array By Parity II
 * https://leetcode.com/problems/sort-array-by-parity-ii/
 * Difficulty: Easy
 *
 * Given an array of integers nums, half of the integers in nums are odd,
 * and the other half are even.
 *
 * Sort the array so that whenever nums[i] is odd, i is odd, and whenever
 * nums[i] is even, i is even.
 *
 * Return any answer array that satisfies this condition.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var sortArrayByParityII = function(nums) {
  let i = 0;
  let j = 1;

  while (i < nums.length && j < nums.length) {
    if (nums[i] % 2 === 0) {
      i += 2;
    } else if (nums[j] % 2 === 1) {
      j += 2;
    } else {
      [nums[i], nums[j]] = [nums[j], nums[i]];
      i += 2;
      j += 2;
    }
  }

  return nums;
};
```

0923 3Sum With Multiplicity

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 923. 3Sum With Multiplicity
 * https://leetcode.com/problems/3sum-with-multiplicity/
 * Difficulty: Medium
 *
 * Given an integer array arr, and an integer target, return the number of tuples i, j, k such
 * that i < j < k and arr[i] + arr[j] + arr[k] == target.
 *
 * As the answer can be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} arr
```

```
 * @param {number} target
 * @return {number}
 */
var threeSumMulti = function(arr, target) {
  const MOD = 1e9 + 7;
  const counts = new Map();
  let result = 0;

  for (const num of arr) {
    counts.set(num, (counts.get(num) || 0) + 1);
  }

  const uniqueNums = [...counts.keys()].sort((a, b) => a - b);

  for (let i = 0; i < uniqueNums.length; i++) {
    const num1 = uniqueNums[i];
    for (let j = i; j < uniqueNums.length; j++) {
      const num2 = uniqueNums[j];
      const num3 = target - num1 - num2;

      if (num3 < num2) continue;

      const count1 = counts.get(num1);
      const count2 = counts.get(num2);
      const count3 = counts.get(num3) || 0;

      if (num1 === num2 && num2 === num3) {
        result = (result + (count1 * (count1 - 1) * (count1 - 2)) / 6) % MOD;
      } else if (num1 === num2) {
        result = (result + (count1 * (count1 - 1) * count3) / 2) % MOD;
      } else if (num2 === num3) {
        result = (result + (count1 * count2 * (count2 - 1)) / 2) % MOD;
      } else {
        result = (result + count1 * count2 * count3) % MOD;
      }
    }
  }

  return result;
};
```

0924 Minimize Malware Spread

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 924. Minimize Malware Spread
 * https://leetcode.com/problems/minimize-malware-spread/
 * Difficulty: Hard
 *
 * You are given a network of n nodes represented as an n x n adjacency matrix graph, where
the ith
 * node is directly connected to the jth node if graph[i][j] == 1.
 *
 * Some nodes initial are initially infected by malware. Whenever two nodes are directly
connected,
 * and at least one of those two nodes is infected by malware, both nodes will be infected b
 * malware. This spread of malware will continue until no more nodes can be infected in this
manner.
 *
 * Suppose M(initial) is the final number of nodes infected with malware in the entire network
 after
 * the spread of malware stops. We will remove exactly one node from initial.
 *
 * Return the node that, if removed, would minimize M(initial). If multiple nodes could be
removed
 * to minimize M(initial), return such a node with the smallest index.
 *
 * Note that if a node was removed from the initial list of infected nodes, it might still be
 * infected later due to the malware spread.
```

```
 */

/**
 * @param {number[][]} graph
 * @param {number[]} initial
 * @return {number}
 */
var minMalwareSpread = function(graph, initial) {
  const n = graph.length;
  const parent = new Array(n).fill().map((_, i) => i);
  const size = new Array(n).fill(1);

  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      if (graph[i][j] === 1) {
        union(i, j);
      }
    }
  }

  let maxSize = 0;
  let result = Math.min(...initial);

  for (const node of initial) {
    const root = find(node);
    let componentInfected = 0;

    for (const infectedNode of initial) {
      if (find(infectedNode) === root) {
        componentInfected++;
      }
    }

    if (componentInfected === 1 && size[root] > maxSize) {
      maxSize = size[root];
      result = node;
    } else if (componentInfected === 1 && size[root] === maxSize) {
      result = Math.min(result, node);
    }
  }

  return result;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    const rootX = find(x);
    const rootY = find(y);
    if (rootX !== rootY) {
      parent[rootX] = rootY;
      size[rootY] += size[rootX];
    }
  }
};
```

0925 Long Pressed Name

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 925. Long Pressed Name
 * https://leetcode.com/problems/long-pressed-name/
 * Difficulty: Easy
 *
 * Your friend is typing his name into a keyboard. Sometimes, when typing a character c,
```

```
 * the key might get long pressed, and the character will be typed 1 or more times.
 *
 * You examine the typed characters of the keyboard.  Return True if it is possible that
 * it was your friends name, with some characters (possibly none) being long pressed.
 */

/**
 * @param {string} name
 * @param {string} typed
 * @return {boolean}
 */
var isLongPressedName = function(name, typed) {
  let i = 0;
  return typed.split('').filter(l => name[i] === l ? ++i : false).length === name.length;
};
```

0926 Flip String To Monotone Increasing

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 926. Flip String to Monotone Increasing
 * https://leetcode.com/problems/flip-string-to-monotone-increasing/
 * Difficulty: Medium
 *
 * A binary string is monotone increasing if it consists of some number of 0's
 * (possibly none), followed by some number of 1's (also possibly none).
 *
 * You are given a binary string s. You can flip s[i] changing it from 0 to 1
 * or from 1 to 0.
 *
 * Return the minimum number of flips to make s monotone increasing.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minFlipsMonoIncr = function(s) {
  let result = 0;
  let count = 0;

  for (const str of s) {
    if (str == '1') {
      count++;
    } else if (str =='0' && count > 0) {
      result++;
      count--;
    }
  }

  return result;
};
```

0927 Three Equal Parts

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 927. Three Equal Parts
 * https://leetcode.com/problems/three-equal-parts/
 * Difficulty: Hard
 *
 * You are given an array arr which consists of only zeros and ones, divide the array into
three
```

```
 * non-empty parts such that all of these parts represent the same binary value.
 *
 * If it is possible, return any [i, j] with i + 1 < j, such that:
 * - arr[0], arr[1], ..., arr[i] is the first part,
 * - arr[i + 1], arr[i + 2], ..., arr[j - 1] is the second part, and
 * - arr[j], arr[j + 1], ..., arr[arr.length - 1] is the third part.
 * - All three parts have equal binary values.
 *
 * If it is not possible, return [-1, -1].
 *
 * Note that the entire part is used when considering what binary value it represents. For
example,
 * [1,1,0] represents 6 in decimal, not 3. Also, leading zeros are allowed, so [0,1,1] and
[1,1]
 * represent the same value.
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var threeEqualParts = function(arr) {
  const totalOnes = arr.reduce((sum, num) => sum + num, 0);
  if (totalOnes % 3 !== 0) {
    return [-1, -1];
  }
  if (totalOnes === 0) {
    return [0, arr.length - 1];
  }

  const onesPerPart = totalOnes / 3;
  let firstStart = -1;
  let secondStart = -1;
  let thirdStart = -1;
  let onesCount = 0;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === 1) {
      onesCount++;
      if (onesCount === 1) firstStart = i;
      if (onesCount === onesPerPart + 1) secondStart = i;
      if (onesCount === 2 * onesPerPart + 1) thirdStart = i;
    }
  }

  const patternLength = arr.length - thirdStart;
  if (secondStart - firstStart < patternLength || thirdStart - secondStart < patternLength) {
    return [-1, -1];
  }
  for (let i = 0; i < patternLength; i++) {
    if (arr[firstStart + i] !== arr[secondStart + i]
        || arr[firstStart + i] !== arr[thirdStart + i]) {
      return [-1, -1];
    }
  }

  const i = firstStart + patternLength - 1;
  const j = secondStart + patternLength;

  return [i, j];
};
```

0928 Minimize Malware Spread Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 928. Minimize Malware Spread II
 * https://leetcode.com/problems/minimize-malware-spread-ii/
 * Difficulty: Hard
 *
```

```
 * You are given a network of n nodes represented as an n x n adjacency matrix graph, where
the
 * ith node is directly connected to the jth node if graph[i][j] == 1.
 *
 * Some nodes initial are initially infected by malware. Whenever two nodes are directly
connected,
 * and at least one of those two nodes is infected by malware, both nodes will be infected by
 * malware. This spread of malware will continue until no more nodes can be infected in this
manner.
 *
 * Suppose M(initial) is the final number of nodes infected with malware in the entire network
 after
 * the spread of malware stops.
 *
 * We will remove exactly one node from initial, completely removing it and any connections
from
 * this node to any other node.
 *
 * Return the node that, if removed, would minimize M(initial). If multiple nodes could be
removed
 * to minimize M(initial), return such a node with the smallest index.
 */

/**
 * @param {number[][]} graph
 * @param {number[]} initial
 * @return {number}
 */
var minMalwareSpread = function(graph, initial) {
  const n = graph.length;
  const initialSet = new Set(initial);

  initial.sort((a, b) => a - b);

  const infected = new Set(initial);
  const queue = [...initial];

  for (let i = 0; i < queue.length; i++) {
    const node = queue[i];
    for (let neighbor = 0; neighbor < n; neighbor++) {
      if (graph[node][neighbor] === 1 && !infected.has(neighbor)) {
        infected.add(neighbor);
        queue.push(neighbor);
      }
    }
  }

  const sourcesMap = new Array(n).fill().map(() => []);

  for (const initialNode of initial) {
    const reachable = new Set();
    const visited = new Set(initial);
    visited.delete(initialNode);

    const q = [initialNode];
    while (q.length > 0) {
      const node = q.shift();
      reachable.add(node);

      for (let neighbor = 0; neighbor < n; neighbor++) {
        if (graph[node][neighbor] === 1 && !visited.has(neighbor)) {
          visited.add(neighbor);
          q.push(neighbor);
        }
      }
    }

    for (let node = 0; node < n; node++) {
      if (reachable.has(node) && !initialSet.has(node)) {
        sourcesMap[node].push(initialNode);
      }
    }
  }

  const savedCounts = new Map();
```

```
  for (let node = 0; node < n; node++) {
    if (sourcesMap[node].length === 1) {
      const source = sourcesMap[node][0];
      savedCounts.set(source, (savedCounts.get(source) || 0) + 1);
    }
  }

  let maxSaved = 0;
  let result = initial[0];

  for (const node of initial) {
    const saved = savedCounts.get(node) || 0;
    if (saved > maxSaved) {
      maxSaved = saved;
      result = node;
    } else if (saved === maxSaved && node < result) {
      result = node;
    }
  }

  return result;
};
```

0929 Unique Email Addresses

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 929. Unique Email Addresses
 * https://leetcode.com/problems/unique-email-addresses/
 * Difficulty: Easy
 *
 * Every email consists of a local name and a domain name, separated by the @ sign.
 *
 * For example, in alice@leetcode.com, alice is the local name, and leetcode.com is the domain
 name.
 *
 * Besides lowercase letters, these emails may contain '.'s or '+'s.
 *
 * If you add periods ('.') between some characters in the local name part of an email
address, mail
 * sent there will be forwarded to the same address without dots in the local name.
 * For example, "alice.z@leetcode.com" and "alicez@leetcode.com" forward to the same email
address.
 * (Note that this rule does not apply for domain names.)
 *
 * If you add a plus ('+') in the local name, everything after the first plus sign will be
ignored.
 * This allows certain emails to be filtered, for example m.y+name@email.com will be forwarded
 to
 * my@email.com. (Again, this rule does not apply for domain names.)
 *
 * It is possible to use both of these rules at the same time.
 *
 * Given a list of emails, we send one email to each address in the list.
 * How many different addresses actually receive mails?
 */

/**
 * @param {string[]} emails
 * @return {number}
 */
var numUniqueEmails = function(emails) {
  return new Set(emails.map(e => e.replace(/\.(?=[\w\.+]+@)|\+[\w.+]+(?=@)/g, ''))).size;
};
```

0930 Binary Subarrays With Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 930. Binary Subarrays With Sum
 * https://leetcode.com/problems/binary-subarrays-with-sum/
 * Difficulty: Medium
 *
 * Given a binary array nums and an integer goal, return the number of non-empty subarrays
 * with a sum goal.
 *
 * A subarray is a contiguous part of the array.
 */

/**
 * @param {number[]} nums
 * @param {number} goal
 * @return {number}
 */
var numSubarraysWithSum = function(nums, goal) {
  const map = new Map([[0, 1]]);
  let result = 0;
  let sum = 0;

  for (const num of nums) {
    sum += num;
    result += map.get(sum - goal) || 0;
    map.set(sum, (map.get(sum) || 0) + 1);
  }

  return result;
};
```

0931 Minimum Falling Path Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 931. Minimum Falling Path Sum
 * https://leetcode.com/problems/minimum-falling-path-sum/
 * Difficulty: Medium
 *
 * Given an n x n array of integers matrix, return the minimum sum of any falling path
 * through matrix.
 *
 * A falling path starts at any element in the first row and chooses the element in the
 * next row that is either directly below or diagonally left/right. Specifically, the
 * next element from position (row, col) will be (row + 1, col - 1), (row + 1, col),
 * or (row + 1, col + 1).
 */

/**
 * @param {number[][]} matrix
 * @return {number}
 */
var minFallingPathSum = function(matrix) {
  let previousRow = [...matrix[0]];

  for (let row = 1; row < matrix.length; row++) {
    const currentRow = new Array(matrix.length);
    for (let col = 0; col < matrix.length; col++) {
      const leftDiagonal = col > 0 ? previousRow[col - 1] : Infinity;
      const directlyAbove = previousRow[col];
      const rightDiagonal = col < matrix.length - 1 ? previousRow[col + 1] : Infinity;
```

```
      currentRow[col] = matrix[row][col] + Math.min(leftDiagonal, directlyAbove,
rightDiagonal);
    }
    previousRow = currentRow;
  }

  return Math.min(...previousRow);
};
```

0932 Beautiful Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 932. Beautiful Array
 * https://leetcode.com/problems/beautiful-array/
 * Difficulty: Medium
 *
 * An array nums of length n is beautiful if:
 * - nums is a permutation of the integers in the range [1, n].
 * - For every 0 <= i < j < n, there is no index k with i < k < j where
 * 2 * nums[k] == nums[i] + nums[j].
 *
 * Given the integer n, return any beautiful array nums of length n.
 * There will be at least one valid answer for the given n.
 */

/**
 * @param {number} n
 * @return {number[]}
 */
var beautifulArray = function(n) {
  const memo = new Map();
  return helper(n);

  function helper(size) {
    if (memo.has(size)) return memo.get(size);
    if (size === 1) return [1];

    const odds = helper(Math.ceil(size / 2));
    const evens = helper(Math.floor(size / 2));
    const result = [
      ...odds.map(x => x * 2 - 1),
      ...evens.map(x => x * 2)
    ];

    memo.set(size, result);
    return result;
  }
};
```

0933 Number Of Recent Calls

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 933. Number of Recent Calls
 * https://leetcode.com/problems/number-of-recent-calls/
 * Difficulty: Easy
 *
 * You have a RecentCounter class which counts the number of recent requests within
 * a certain time frame.
 *
 * Implement the RecentCounter class:
 * - RecentCounter() Initializes the counter with zero recent requests.
```

```
 * - int ping(int t) Adds a new request at time t, where t represents some time in
 *   milliseconds, and returns the number of requests that has happened in the past
 *   3000 milliseconds (including the new request). Specifically, return the number
 *   of requests that have happened in the inclusive range [t - 3000, t].
 *
 * It is guaranteed that every call to ping uses a strictly larger value of t than
 * the previous call.
 */


var RecentCounter = function() {
  this.queue = [];
};

/**
 * @param {number} t
 * @return {number}
 */
RecentCounter.prototype.ping = function(t) {
  this.queue.push(t);
  while (this.queue[0] < t - 3000) {
    this.queue.shift();
  }
  return this.queue.length;
};
```

0934 Shortest Bridge

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 934. Shortest Bridge
 * https://leetcode.com/problems/shortest-bridge/
 * Difficulty: Medium
 *
 * You are given an n x n binary matrix grid where 1 represents land and 0 represents water.
 *
 * An island is a 4-directionally connected group of 1's not connected to any other 1's.
 * There are exactly two islands in grid.
 *
 * You may change 0's to 1's to connect the two islands to form one island.
 *
 * Return the smallest number of 0's you must flip to connect the two islands.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var shortestBridge = function(grid) {
  const n = grid.length;
  const queue = [];
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  outer: for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if (grid[i][j] === 1) {
        findFirstIsland(i, j);
        break outer;
      }
    }
  }

  let distance = 0;
  while (queue.length) {
    const size = queue.length;
    for (let i = 0; i < size; i++) {
      const [row, col] = queue.shift();
      for (const [dx, dy] of directions) {
        const newRow = row + dx;
```

```
        const newCol = col + dy;
        if (newRow < 0 || newRow >= n || newCol < 0 || newCol >= n
            || grid[newRow][newCol] === 2) continue;
        if (grid[newRow][newCol] === 1) return distance;
        grid[newRow][newCol] = 2;
        queue.push([newRow, newCol]);
      }
    }
    distance++;
  }

  return distance;

  function findFirstIsland(row, col) {
    if (row < 0 || row >= n || col < 0 || col >= n || grid[row][col] !== 1) return;
    grid[row][col] = 2;
    queue.push([row, col]);
    for (const [dx, dy] of directions) {
      findFirstIsland(row + dx, col + dy);
    }
  }
};
```

0935 Knight Dialer

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 935. Knight Dialer
 * https://leetcode.com/problems/knight-dialer/
 * Difficulty: Medium
 *
 * The chess knight has a unique movement, it may move two squares vertically and one square
 * horizontally, or two squares horizontally and one square vertically (with both forming
 * the shape of an L). The possible movements of chess knight are shown in this diagram:
 *
 * A chess knight can move as indicated in the chess diagram below.
 *
 * We have a chess knight and a phone pad as shown below, the knight can only stand on a
 * numeric cell (i.e. blue cell).
 *
 * Given an integer n, return how many distinct phone numbers of length n we can dial.
 *
 * You are allowed to place the knight on any numeric cell initially and then you should
 * perform n - 1 jumps to dial a number of length n. All jumps should be valid knight jumps.
 *
 * As the answer may be very large, return the answer modulo 109 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
var knightDialer = function(n) {
  const MOD = 1e9 + 7;
  const moves = [
    [4, 6], [6, 8], [7, 9], [4, 8],
    [0, 3, 9], [], [0, 1, 7], [2, 6],
    [1, 3], [2, 4]
  ];
  let prevCounts = new Array(10).fill(1);

  for (let jump = 1; jump < n; jump++) {
    const currCounts = new Array(10).fill(0);
    for (let digit = 0; digit < 10; digit++) {
      for (const nextDigit of moves[digit]) {
        currCounts[nextDigit] = (currCounts[nextDigit] + prevCounts[digit]) % MOD;
      }
    }
    prevCounts = currCounts;
```

```
  }

  return prevCounts.reduce((sum, count) => (sum + count) % MOD, 0);
};
```

0936 Stamping The Sequence

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 936. Stamping The Sequence
 * https://leetcode.com/problems/stamping-the-sequence/
 * Difficulty: Hard
 *
 * You are given two strings stamp and target. Initially, there is a string s of length
 * target.length with all s[i] == '?'.
 *
 * In one turn, you can place stamp over s and replace every letter in the s with the
 * corresponding letter from stamp.
 *
 * For example, if stamp = "abc" and target = "abcba", then s is "?????" initially.
 * In one turn you can:
 * - place stamp at index 0 of s to obtain "abc??",
 * - place stamp at index 1 of s to obtain "?abc?", or
 * - place stamp at index 2 of s to obtain "??abc".
 *
 * Note that stamp must be fully contained in the boundaries of s in order to stamp
 * (i.e., you cannot place stamp at index 3 of s).
 *
 * We want to convert s to target using at most 10 * target.length turns.
 *
 * Return an array of the index of the left-most letter being stamped at each turn. If we
 * cannot obtain target from s within 10 * target.length turns, return an empty array.
 */

/**
 * @param {string} stamp
 * @param {string} target
 * @return {number[]}
 */
var movesToStamp = function(stamp, target) {
  const stampLength = stamp.length;
  const targetLength = target.length;
  const moves = [];
  const targetArray = target.split('');
  let totalReplaced = 0;

  function tryStampAt(position) {
    let canStamp = false;
    let hasUnstamped = false;

    for (let i = 0; i < stampLength; i++) {
      const currentChar = targetArray[position + i];
      if (currentChar === '?') continue;
      if (currentChar !== stamp[i]) return false;
      hasUnstamped = true;
    }

    if (hasUnstamped) {
      for (let i = 0; i < stampLength; i++) {
        if (targetArray[position + i] !== '?') {
          targetArray[position + i] = '?';
          totalReplaced++;
        }
      }
      canStamp = true;
    }

    return canStamp;
  }
```

```
  const maxMoves = 10 * targetLength;
  while (moves.length <= maxMoves) {
    let madeChange = false;
    for (let i = 0; i <= targetLength - stampLength; i++) {
      if (tryStampAt(i)) {
        moves.push(i);
        madeChange = true;
        break;
      }
    }
    if (!madeChange) break;
    if (totalReplaced === targetLength) return moves.reverse();
  }

  return [];
};
```

0937 Reorder Data In Log Files

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 937. Reorder Data in Log Files
 * https://leetcode.com/problems/reorder-data-in-log-files/
 * Difficulty: Medium
 *
 * You are given an array of logs. Each log is a space-delimited string of words,
 * where the first word is the identifier.
 *
 * There are two types of logs:
 * - Letter-logs: All words (except the identifier) consist of lowercase English
 *   letters.
 * - Digit-logs: All words (except the identifier) consist of digits.
 *
 * Reorder these logs so that:
 * 1. The letter-logs come before all digit-logs.
 * 2. The letter-logs are sorted lexicographically by their contents. If their
 *   contents are the same, then sort them lexicographically by their identifiers.
 * 3. The digit-logs maintain their relative ordering.
 *
 * Return the final order of the logs.
 */

/**
 * @param {string[]} logs
 * @return {string[]}
 */
var reorderLogFiles = function(logs) {
  const split = logs.map(l => l.split(' '));
  const sortedA = split.filter(l => isNaN(parseInt(l[1], 10))).sort((a, b) => {
    return a[1] !== b[1]
      ? a[1].localeCompare(b[1]) : a[2] !== b[2]
        ? a[2].localeCompare(b[2]) : a.length === b.length
          ? a[0].localeCompare(b[0]) : a.length - b.length;
  }).map(l => l.join(' '));
  const sortedB = split.filter(l => !isNaN(parseInt(l[1], 10)))
    .sort((a, b) => b - a)
    .map(l => l.join(' '));

  return [...sortedA, ...sortedB];
};
```

0938 Range Sum Of Bst

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 938. Range Sum of BST
 * https://leetcode.com/problems/range-sum-of-bst/
 * Difficulty: Easy
 *
 * Given the root node of a binary search tree and two integers low and high, return the sum
 * of values of all nodes with a value in the inclusive range [low, high].
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} low
 * @param {number} high
 * @return {number}
 */
var rangeSumBST = function(root, low, high) {
  return traverse(root);

  function traverse(node) {
    if (!node) return 0;
    const value = node.val;
    if (value < low) return traverse(node.right);
    if (value > high) return traverse(node.left);
    return value + traverse(node.left) + traverse(node.right);
  }
};
```

0939 Minimum Area Rectangle

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 939. Minimum Area Rectangle
 * https://leetcode.com/problems/minimum-area-rectangle/
 * Difficulty: Medium
 *
 * You are given an array of points in the X-Y plane points where points[i] = [xi, yi].
 *
 * Return the minimum area of a rectangle formed from these points, with sides parallel to the
 * X and Y axes. If there is not any such rectangle, return 0.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var minAreaRect = function(points) {
  const pointSet = new Set(points.map(([x, y]) => `${x},${y}`));
  let result = 0;

  for (let i = 0; i < points.length; i++) {
    const [x1, y1] = points[i];
    for (let j = i + 1; j < points.length; j++) {
      const [x2, y2] = points[j];
      if (x1 !== x2 && y1 !== y2) {
        if (pointSet.has(`${x1},${y2}`) && pointSet.has(`${x2},${y1}`)) {
          const area = Math.abs(x1 - x2) * Math.abs(y1 - y2);
          result = result === 0 ? area : Math.min(result, area);
        }
```

```
      }
    }
  }

  return result;
};
```

0940 Distinct Subsequences Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 940. Distinct Subsequences II
 * https://leetcode.com/problems/distinct-subsequences-ii/
 * Difficulty: Hard
 *
 * Given a string s, return the number of distinct non-empty subsequences of s. Since the
answer
 * may be very large, return it modulo 109 + 7.
 *
 * A subsequence of a string is a new string that is formed from the original string by
deleting
 * some (can be none) of the characters without disturbing the relative positions of the
remaining
 * characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not.
 */

/**
 * @param {string} s
 * @return {number}
 */
var distinctSubseqII = function(s) {
  const MOD = 1e9 + 7;
  const lastOccurrence = new Array(26).fill(-1);
  const dp = new Array(s.length + 1).fill(0);
  dp[0] = 1;

  for (let i = 0; i < s.length; i++) {
    const charIndex = s.charCodeAt(i) - 97;
    dp[i + 1] = (dp[i] * 2) % MOD;
    if (lastOccurrence[charIndex] !== -1) {
      dp[i + 1] = (dp[i + 1] - dp[lastOccurrence[charIndex]] + MOD) % MOD;
    }
    lastOccurrence[charIndex] = i;
  }

  return (dp[s.length] - 1 + MOD) % MOD;
};
```

0941 Valid Mountain Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 941. Valid Mountain Array
 * https://leetcode.com/problems/valid-mountain-array/
 * Difficulty: Easy
 *
 * Given an array of integers arr, return true if and only if it is a valid mountain array.
 *
 * Recall that arr is a mountain array if and only if:
 * - arr.length >= 3
 * - There exists some i with 0 < i < arr.length - 1 such that:
 *   - arr[0] < arr[1] < ... < arr[i - 1] < arr[i]
 *   - arr[i] > arr[i + 1] > ... > arr[arr.length - 1]
```

```
 */

/**
 * @param {number[]} arr
 * @return {boolean}
 */
var validMountainArray = function(arr) {
  if (arr.length < 3) return false;

  const peakIndex = arr.indexOf(Math.max(...arr));
  if (peakIndex === 0 || peakIndex === arr.length - 1) return false;

  for (let i = 1; i < peakIndex; i++) {
    if (arr[i] <= arr[i - 1]) return false;
  }

  for (let i = peakIndex + 1; i < arr.length; i++) {
    if (arr[i] >= arr[i - 1]) return false;
  }

  return true;
};
```

0942 Di String Match

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 942. DI String Match
 * https://leetcode.com/problems/di-string-match/
 * Difficulty: Easy
 *
 * A permutation perm of n + 1 integers of all the integers in the range [0, n] can be
represented
 * as a string s of length n where:
 * - s[i] == 'I' if perm[i] < perm[i + 1], and
 * - s[i] == 'D' if perm[i] > perm[i + 1].
 *
 * Given a string s, reconstruct the permutation perm and return it. If there are multiple
valid
 * permutations perm, return any of them.
 */

/**
 * @param {string} s
 * @return {number[]}
 */
var diStringMatch = function(s) {
  const result = [];
  let low = 0;
  let high = s.length;

  for (const char of s) {
    if (char === 'I') {
      result.push(low);
      low++;
    } else {
      result.push(high);
      high--;
    }
  }

  result.push(low);
  return result;
};
```

0943 Find The Shortest Superstring

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 943. Find the Shortest Superstring
 * https://leetcode.com/problems/find-the-shortest-superstring/
 * Difficulty: Hard
 *
 * Given an array of strings words, return the smallest string that contains each string in
words
 * as a substring. If there are multiple valid strings of the smallest length, return any of
them.
 *
 * You may assume that no string in words is a substring of another string in words.
 */

/**
 * @param {string[]} words
 * @return {string}
 */
var shortestSuperstring = function(words) {
  const n = words.length;
  const overlap = Array(n).fill().map(() => Array(n).fill(0));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if (i !== j) {
        for (let k = Math.min(words[i].length, words[j].length); k > 0; k--) {
          if (words[i].slice(-k) === words[j].slice(0, k)) {
            overlap[i][j] = k;
            break;
          }
        }
      }
    }
  }

  const dp = new Array(1 << n).fill().map(() => new Array(n).fill(''));
  for (let i = 0; i < n; i++) {
    dp[1 << i][i] = words[i];
  }

  for (let mask = 1; mask < (1 << n); mask++) {
    for (let i = 0; i < n; i++) {
      if (!(mask & (1 << i))) continue;
      for (let j = 0; j < n; j++) {
        if (mask & (1 << j)) continue;
        const nextMask = mask | (1 << j);
        const candidate = dp[mask][i] + words[j].slice(overlap[i][j]);
        if (dp[nextMask][j] === '' || candidate.length < dp[nextMask][j].length) {
          dp[nextMask][j] = candidate;
        }
      }
    }
  }

  let result = dp[(1 << n) - 1][0];
  for (let i = 1; i < n; i++) {
    if (dp[(1 << n) - 1][i].length < result.length) {
      result = dp[(1 << n) - 1][i];
    }
  }

  return result;
};
```

0944 Delete Columns To Make Sorted

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 944. Delete Columns to Make Sorted
 * https://leetcode.com/problems/delete-columns-to-make-sorted/
 * Difficulty: Easy
 *
 * You are given an array of n strings strs, all of the same length.
 *
 * The strings can be arranged such that there is one on each line, making a grid.
 *
 * For example, strs = ["abc", "bce", "cae"] can be arranged as follows:
 * - abc
 * - bce
 * - cae
 *
 * You want to delete the columns that are not sorted lexicographically. In the above
 * example (0-indexed), columns 0 ('a', 'b', 'c') and 2 ('c', 'e', 'e') are sorted,
 * while column 1 ('b', 'c', 'a') is not, so you would delete column 1.
 *
 * Return the number of columns that you will delete.
 */

/**
 * @param {string[]} strs
 * @return {number}
 */
var minDeletionSize = function(strs) {
  let result = 0;

  for (let col = 0; col < strs[0].length; col++) {
    for (let row = 1; row < strs.length; row++) {
      if (strs[row][col] < strs[row - 1][col]) {
        result++;
        break;
      }
    }
  }

  return result;
};
```

0945 Minimum Increment To Make Array Unique

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 945. Minimum Increment to Make Array Unique
 * https://leetcode.com/problems/minimum-increment-to-make-array-unique/
 * Difficulty: Medium
 *
 * You are given an integer array nums. In one move, you can pick an index i where
 * 0 <= i < nums.length and increment nums[i] by 1.
 *
 * Return the minimum number of moves to make every value in nums unique.
 *
 * The test cases are generated so that the answer fits in a 32-bit integer.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
```

```
var minIncrementForUnique = function(nums) {
  nums.sort((a, b) => a - b);
  let result = 0;

  for (let i = 1, previous = nums[0]; i < nums.length; i++) {
    if (nums[i] <= previous) {
      previous++;
      result += previous - nums[i];
    } else {
      previous = nums[i];
    }
  }

  return result;
};
```

0946 Validate Stack Sequences

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 946. Validate Stack Sequences
 * https://leetcode.com/problems/validate-stack-sequences/
 * Difficulty: Medium
 *
 * Given two integer arrays pushed and popped each with distinct values, return true if this
could
 * have been the result of a sequence of push and pop operations on an initially empty stack,
or
 * false otherwise.
 */

/**
 * @param {number[]} pushed
 * @param {number[]} popped
 * @return {boolean}
 */
var validateStackSequences = function(pushed, popped) {
  const stack = [];
  let popIndex = 0;

  for (const number of pushed) {
    stack.push(number);

    while (stack.length && stack[stack.length - 1] === popped[popIndex]) {
      stack.pop();
      popIndex++;
    }
  }

  return stack.length === 0;
};
```

0947 Most Stones Removed With Same Row Or Column

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 947. Most Stones Removed with Same Row or Column
 * https://leetcode.com/problems/most-stones-removed-with-same-row-or-column/
 * Difficulty: Medium
 *
 * On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point
may
 * have at most one stone.
```

```
 *
 * A stone can be removed if it shares either the same row or the same column as another stone
 * that has not been removed.
 *
 * Given an array stones of length n where stones[i] = [xi, yi] represents the location of the
 * ith stone, return the largest possible number of stones that can be removed.
 */

/**
 * @param {number[][]} stones
 * @return {number}
 */
var removeStones = function(stones) {
  const parent = new Map();
  const find = (stoneIndex) => {
    if (!parent.has(stoneIndex)) {
      parent.set(stoneIndex, stoneIndex);
    }
    if (parent.get(stoneIndex) !== stoneIndex) {
      parent.set(stoneIndex, find(parent.get(stoneIndex)));
    }
    return parent.get(stoneIndex);
  };

  const union = (stone1, stone2) => {
    parent.set(find(stone1), find(stone2));
  };

  const stoneMap = new Map();
  for (let i = 0; i < stones.length; i++) {
    const [row, col] = stones[i];
    const rowKey = `r${row}`;
    const colKey = `c${col}`;

    if (stoneMap.has(rowKey)) {
      union(i, stoneMap.get(rowKey));
    } else {
      stoneMap.set(rowKey, i);
    }

    if (stoneMap.has(colKey)) {
      union(i, stoneMap.get(colKey));
    } else {
      stoneMap.set(colKey, i);
    }
  }

  const uniqueGroups = new Set();
  for (let i = 0; i < stones.length; i++) {
    uniqueGroups.add(find(i));
  }

  return stones.length - uniqueGroups.size;
};
```

0948 Bag Of Tokens

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 948. Bag of Tokens
 * https://leetcode.com/problems/bag-of-tokens/
 * Difficulty: Medium
 *
 * You start with an initial power of power, an initial score of 0, and a bag of tokens given
as
 * an integer array tokens, where each tokens[i] denotes the value of tokeni.
 *
 * Your goal is to maximize the total score by strategically playing these tokens. In one
move,
```

```
 * you can play an unplayed token in one of the two ways (but not both for the same token):
 * - Face-up: If your current power is at least tokens[i], you may play tokeni, losing
tokens[i]
 *   power and gaining 1 score.
 * - Face-down: If your current score is at least 1, you may play tokeni, gaining tokens[i]
power
 *   and losing 1 score.
 *
 * Return the maximum possible score you can achieve after playing any number of tokens.
 */

/**
 * @param {number[]} tokens
 * @param {number} power
 * @return {number}
 */
var bagOfTokensScore = function(tokens, power) {
  let score = 0;
  let result = 0;
  let left = 0;
  let right = tokens.length - 1;

  tokens.sort((a, b) => a - b);

  while (left <= right && (power >= tokens[left] || score > 0)) {
    while (left <= right && power >= tokens[left]) {
      power -= tokens[left];
      score++;
      left++;
    }
    result = Math.max(result, score);

    if (left <= right && score > 0) {
      power += tokens[right];
      score--;
      right--;
    }
  }

  return result;
};
```

0949 Largest Time For Given Digits

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 949. Largest Time for Given Digits
 * https://leetcode.com/problems/largest-time-for-given-digits/
 * Difficulty: Medium
 *
 * Given an array arr of 4 digits, find the latest 24-hour time that can be made using
 * each digit exactly once.
 *
 * 24-hour times are formatted as "HH:MM", where HH is between 00 and 23, and MM is between
 * 00 and 59. The earliest 24-hour time is 00:00, and the latest is 23:59.
 *
 * Return the latest 24-hour time in "HH:MM" format. If no valid time can be made, return
 * an empty string.
 */

/**
 * @param {number[]} arr
 * @return {string}
 */
var largestTimeFromDigits = function(arr) {
  let result = '';
  helper(arr, new Array(4).fill(false), []);
  return result;
```

```
  function helper(digits, used, current) {
    if (current.length === 4) {
      const hours = parseInt(current.slice(0, 2).join(''));
      const minutes = parseInt(current.slice(2).join(''));
      if (hours <= 23 && minutes <= 59) {
        const time = `${current[0]}${current[1]}:${current[2]}${current[3]}`;
        if (time > result) result = time;
      }
      return;
    }

    for (let i = 0; i < digits.length; i++) {
      if (!used[i]) {
        used[i] = true;
        current.push(digits[i]);
        helper(digits, used, current);
        current.pop();
        used[i] = false;
      }
    }
  }
};
```

0950 Reveal Cards In Increasing Order

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 950. Reveal Cards In Increasing Order
 * https://leetcode.com/problems/reveal-cards-in-increasing-order/
 * Difficulty: Medium
 *
 * You are given an integer array deck. There is a deck of cards where every card has a
 * unique integer. The integer on the ith card is deck[i].
 *
 * You can order the deck in any order you want. Initially, all the cards start face down
 * (unrevealed) in one deck.
 *
 * You will do the following steps repeatedly until all cards are revealed:
 * 1. Take the top card of the deck, reveal it, and take it out of the deck.
 * 2. If there are still cards in the deck then put the next top card of the deck at the
 *    bottom of the deck.
 * 3. If there are still unrevealed cards, go back to step 1. Otherwise, stop.
 *
 * Return an ordering of the deck that would reveal the cards in increasing order.
 *
 * Note that the first entry in the answer is considered to be the top of the deck.
 */

/**
 * @param {number[]} deck
 * @return {number[]}
 */
var deckRevealedIncreasing = function(deck) {
  deck.sort((a, b) => b - a);
  const result = [];

  for (const card of deck) {
    if (result.length) {
      result.unshift(result.pop());
    }
    result.unshift(card);
  }

  return result;
};
```

0951 Flip Equivalent Binary Trees

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 951. Flip Equivalent Binary Trees
 * https://leetcode.com/problems/flip-equivalent-binary-trees/
 * Difficulty: Medium
 *
 * For a binary tree T, we can define a flip operation as follows: choose any node, and swap
the
 * left and right child subtrees.
 *
 * A binary tree X is flip equivalent to a binary tree Y if and only if we can make X equal to
 Y
 * after some number of flip operations.
 *
 * Given the roots of two binary trees root1 and root2, return true if the two trees are flip
 * equivalent or false otherwise.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root1
 * @param {TreeNode} root2
 * @return {boolean}
 */
var flipEquiv = function(root1, root2) {
  return helper(root1, root2);

  function helper(node1, node2) {
    if (!node1 && !node2) return true;
    if (!node1 || !node2 || node1.val !== node2.val) return false;
    return (helper(node1.left, node2.left) && helper(node1.right, node2.right))
      || (helper(node1.left, node2.right) && helper(node1.right, node2.left));
  }
};
```

0952 Largest Component Size By Common Factor

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 952. Largest Component Size by Common Factor
 * https://leetcode.com/problems/largest-component-size-by-common-factor/
 * Difficulty: Hard
 *
 * You are given an integer array of unique positive integers nums. Consider the following
graph:
 * - There are nums.length nodes, labeled nums[0] to nums[nums.length - 1],
 * - There is an undirected edge between nums[i] and nums[j] if nums[i] and nums[j] share a
common
 *   factor greater than 1.
 *
 * Return the size of the largest connected component in the graph.
 */

/**
```

```
 * @param {number[]} nums
 * @return {number}
 */
var largestComponentSize = function(nums) {
  const max = Math.max(...nums);
  const parent = new Array(max + 1).fill().map((_, i) => i);
  const rank = new Array(max + 1).fill(0);

  for (const num of nums) {
    for (let factor = 2; factor * factor <= num; factor++) {
      if (num % factor === 0) {
        union(parent, rank, num, factor);
        union(parent, rank, num, num / factor);
      }
    }
  }

  const map = new Map();
  let result = 0;

  for (const num of nums) {
    const root = find(parent, num);
    map.set(root, (map.get(root) || 0) + 1);
    result = Math.max(result, map.get(root));
  }

  return result;

  function find(parent, x) {
    if (parent[x] !== x) {
      parent[x] = find(parent, parent[x]);
    }
    return parent[x];
  }

  function union(parent, rank, x, y) {
    const rootX = find(parent, x);
    const rootY = find(parent, y);
    if (rootX !== rootY) {
      if (rank[rootX] < rank[rootY]) {
        parent[rootX] = rootY;
      } else if (rank[rootX] > rank[rootY]) {
        parent[rootY] = rootX;
      } else {
        parent[rootY] = rootX;
        rank[rootX]++;
      }
    }
  }
};
```

0953 Verifying An Alien Dictionary

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 953. Verifying an Alien Dictionary
 * https://leetcode.com/problems/verifying-an-alien-dictionary/
 * Difficulty: Easy
 *
 * In an alien language, surprisingly, they also use English lowercase letters, but possibly
 * in a different order. The order of the alphabet is some permutation of lowercase letters.
 *
 * Given a sequence of words written in the alien language, and the order of the alphabet,
 * return true if and only if the given words are sorted lexicographically in this alien
language.
 */

/**
 * @param {string[]} words
```

```
 * @param {string} order
 * @return {boolean}
 */
var isAlienSorted = function(words, order) {
  const map = new Map(order.split('').map((char, index) => [char, index]));

  for (let i = 0; i < words.length - 1; i++) {
    if (!helper(words[i], words[i + 1])) return false;
  }

  return true;

  function helper(word1, word2) {
    for (let i = 0; i < word1.length; i++) {
      if (i >= word2.length) return false;
      const char1 = map.get(word1[i]);
      const char2 = map.get(word2[i]);
      if (char1 < char2) return true;
      if (char1 > char2) return false;
    }
    return true;
  }
};
```

0954 Array Of Doubled Pairs

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 954. Array of Doubled Pairs
 * https://leetcode.com/problems/array-of-doubled-pairs/
 * Difficulty: Medium
 *
 * Given an integer array of even length arr, return true if it is possible to reorder arr
such
 * that arr[2 * i + 1] = 2 * arr[2 * i] for every 0 <= i < len(arr) / 2, or false otherwise.
 */

/**
 * @param {number[]} arr
 * @return {boolean}
 */
var canReorderDoubled = function(arr) {
  const map = new Map();
  arr.sort((a, b) => Math.abs(a) - Math.abs(b));

  for (const num of arr) {
    map.set(num, (map.get(num) || 0) + 1);
  }
  for (const num of arr) {
    if (map.get(num) === 0) continue;
    if (!map.has(2 * num) || map.get(2 * num) === 0) return false;

    map.set(num, map.get(num) - 1);
    map.set(2 * num, map.get(2 * num) - 1);
  }

  return true;
};
```

0955 Delete Columns To Make Sorted Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 955. Delete Columns to Make Sorted II
 * https://leetcode.com/problems/delete-columns-to-make-sorted-ii/
 * Difficulty: Medium
 *
 * You are given an array of n strings strs, all of the same length.
 *
 * We may choose any deletion indices, and we delete all the characters in those indices
 * for each string.
 *
 * For example, if we have strs = ["abcdef","uvwxyz"] and deletion indices {0, 2, 3}, then
 * the final array after deletions is ["bef", "vyz"].
 *
 * Suppose we chose a set of deletion indices answer such that after deletions, the final
 * array has its elements in lexicographic order (i.e., strs[0] <= strs[1] <= strs[2]
 * <= ... <= strs[n - 1]). Return the minimum possible value of answer.length.
 */

/**
 * @param {string[]} strs
 * @return {number}
 */
var minDeletionSize = function(strs) {
  let previousSorted = new Array(strs.length).fill('');
  let result = 0;

  for (let col = 0; col < strs[0].length; col++) {
    const currentSorted = previousSorted.slice();
    let isValid = true;

    for (let row = 0; row < strs.length; row++) {
      currentSorted[row] += strs[row][col];
      if (row > 0 && currentSorted[row] < currentSorted[row - 1]) {
        isValid = false;
        break;
      }
    }

    if (isValid) {
      previousSorted = currentSorted;
    } else {
      result++;
    }
  }

  return result;
};
```

0956 Tallest Billboard

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 956. Tallest Billboard
 * https://leetcode.com/problems/tallest-billboard/
 * Difficulty: Hard
 *
 * You are installing a billboard and want it to have the largest height. The billboard will
have
 * two steel supports, one on each side. Each steel support must be an equal height.
 *
 * You are given a collection of rods that can be welded together. For example, if you have
rods
 * of lengths 1, 2, and 3, you can weld them together to make a support of length 6.
 *
 * Return the largest possible height of your billboard installation. If you cannot support
the
 * billboard, return 0.
 */
```

```
/**
 * @param {number[]} rods
 * @return {number}
 */
var tallestBillboard = function(rods) {
  const differences = new Map([[0, 0]]);

  for (const rod of rods) {
    const currentDiffs = new Map(differences);

    for (const [diff, taller] of currentDiffs) {
      const shorter = taller - diff;
      const newDiff = diff + rod;
      const newTaller = taller + rod;
      differences.set(newDiff, Math.max(differences.get(newDiff) || 0, newTaller));

      const diffAbs = Math.abs(diff - rod);
      const diffMax = Math.max(shorter + rod, taller);
      differences.set(diffAbs, Math.max(differences.get(diffAbs) || 0, diffMax));
    }
  }

  return differences.get(0) || 0;
};
```

0957 Prison Cells After N Days

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 957. Prison Cells After N Days
 * https://leetcode.com/problems/prison-cells-after-n-days/
 * Difficulty: Medium
 *
 * There are 8 prison cells in a row and each cell is either occupied or vacant.
 *
 * Each day, whether the cell is occupied or vacant changes according to the following rules:
 * - If a cell has two adjacent neighbors that are both occupied or both vacant, then the cell
 *   becomes occupied.
 * - Otherwise, it becomes vacant.
 *
 * Note that because the prison is a row, the first and the last cells in the row can't have
two
 * adjacent neighbors.
 *
 * You are given an integer array cells where cells[i] == 1 if the ith cell is occupied and
 * cells[i] == 0 if the ith cell is vacant, and you are given an integer n.
 *
 * Return the state of the prison after n days (i.e., n such changes described above).
 */

/**
 * @param {number[]} cells
 * @param {number} n
 * @return {number[]}
 */
var prisonAfterNDays = function(cells, n) {
  const nextState = cells => [
    0,
    ...Array.from({ length: 6 }, (_, i) => cells[i] === cells[i + 2] ? 1 : 0),
    0
  ];

  let current = [...cells];
  const seen = new Map();
  let cycleLength = 0;

  while (n > 0) {
    const stateKey = current.join('');
    if (seen.has(stateKey)) {
```

```
      const cycleStart = seen.get(stateKey);
      cycleLength = cycleStart - n;
      n %= cycleLength;
    }
    seen.set(stateKey, n);
    if (n > 0) {
      current = nextState(current);
      n--;
    }
  }

  return current;
};
```

0958 Check Completeness Of A Binary Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 958. Check Completeness of a Binary Tree
 * https://leetcode.com/problems/check-completeness-of-a-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, determine if it is a complete binary tree.
 *
 * In a complete binary tree, every level, except possibly the last, is completely filled, and
 * all nodes in the last level are as far left as possible. It can have between 1 and 2h nodes
 * inclusive at the last level h.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isCompleteTree = function(root) {
  const queue = [root];
  let foundNull = false;

  while (queue.length) {
    const node = queue.shift();
    if (!node) {
      foundNull = true;
      continue;
    }
    if (foundNull) return false;
    queue.push(node.left);
    queue.push(node.right);
  }

  return true;
};
```

0959 Regions Cut By Slashes

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 959. Regions Cut By Slashes
 * https://leetcode.com/problems/regions-cut-by-slashes/
 * Difficulty: Medium
 *
 * An n x n grid is composed of 1 x 1 squares where each 1 x 1 square consists of
 * a '/', '\', or blank space ' '. These characters divide the square into contiguous
 * regions.
 *
 * Given the grid grid represented as a string array, return the number of regions.
 *
 * Note that backslash characters are escaped, so a '\' is represented as '\\'.
 */

/**
 * @param {string[]} grid
 * @return {number}
 */
var regionsBySlashes = function(grid) {
  const size = grid.length * 3;
  const matrix = new Array(size).fill().map(() => new Array(size).fill(0));

  for (let row = 0; row < grid.length; row++) {
    for (let col = 0; col < grid.length; col++) {
      const baseRow = row * 3;
      const baseCol = col * 3;
      if (grid[row][col] === '/') {
        matrix[baseRow][baseCol + 2] = 1;
        matrix[baseRow + 1][baseCol + 1] = 1;
        matrix[baseRow + 2][baseCol] = 1;
      } else if (grid[row][col] === '\\') {
        matrix[baseRow][baseCol] = 1;
        matrix[baseRow + 1][baseCol + 1] = 1;
        matrix[baseRow + 2][baseCol + 2] = 1;
      }
    }
  }

  let result = 0;
  for (let row = 0; row < size; row++) {
    for (let col = 0; col < size; col++) {
      if (matrix[row][col] === 0) {
        helper(row, col);
        result++;
      }
    }
  }

  return result;

  function helper(x, y) {
    if (x < 0 || x >= size || y < 0 || y >= size || matrix[x][y] !== 0) return;
    matrix[x][y] = 1;
    helper(x + 1, y);
    helper(x - 1, y);
    helper(x, y + 1);
    helper(x, y - 1);
  }
};
```

0960 Delete Columns To Make Sorted Iii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 960. Delete Columns to Make Sorted III
 * https://leetcode.com/problems/delete-columns-to-make-sorted-iii/
 * Difficulty: Hard
 *
 * You are given an array of n strings strs, all of the same length.
 *
```

```
 * We may choose any deletion indices, and we delete all the characters in those indices
 * for each string.
 *
 * For example, if we have strs = ["abcdef","uvwxyz"] and deletion indices {0, 2, 3}, then
 * the final array after deletions is ["bef", "vyz"].
 *
 * Suppose we chose a set of deletion indices answer such that after deletions, the final
 * array has every string (row) in lexicographic order. (i.e.,
 * (strs[0][0] <= strs[0][1] <= ... <= strs[0][strs[0].length - 1]),
 * and (strs[1][0] <= strs[1][1] <= ... <= strs[1][strs[1].length - 1]),
 * and so on). Return the minimum possible value of answer.length.
 */

/**
 * @param {string[]} strs
 * @return {number}
 */
var minDeletionSize = function(strs) {
  const isValid = (input, a, b) => input.every(row => row[a] <= row[b]);
  const cols = strs[0].length;
  const dp = new Array(cols).fill(1);

  for (let i = 1; i < cols; i++) {
    for (let j = 0; j < i; j++) {
      if (isValid(strs, j, i)) {
        dp[i] = Math.max(dp[i], dp[j] + 1);
      }
    }
  }

  return cols - Math.max(...dp);
};
```

0961 N Repeated Element In Size 2N Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 961. N-Repeated Element in Size 2N Array
 * https://leetcode.com/problems/n-repeated-element-in-size-2n-array/
 * Difficulty: Easy
 *
 * You are given an integer array nums with the following properties:
 * - nums.length == 2 * n.
 * - nums contains n + 1 unique elements.
 * - Exactly one element of nums is repeated n times.
 *
 * Return the element that is repeated n times.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var repeatedNTimes = function(nums) {
  const map = {};
  for (let i = 0; i < nums.length; i++) {
    if (!map[nums[i]]) map[nums[i]] = 1;
    else return nums[i];
  }
};
```

0962 Maximum Width Ramp

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 962. Maximum Width Ramp
 * https://leetcode.com/problems/maximum-width-ramp/
 * Difficulty: Medium
 *
 * A ramp in an integer array nums is a pair (i, j) for which i < j and nums[i] <= nums[j].
 * The width of such a ramp is j - i.
 *
 * Given an integer array nums, return the maximum width of a ramp in nums. If there is no
 * ramp in nums, return 0.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxWidthRamp = function(nums) {
  const stack = [];
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    if (!stack.length || nums[stack.at(-1)] > nums[i]) {
      stack.push(i);
    }
  }

  for (let j = nums.length - 1; j >= 0; j--) {
    while (stack.length && nums[stack.at(-1)] <= nums[j]) {
      result = Math.max(result, j - stack.pop());
    }
  }

  return result;
};
```

0963 Minimum Area Rectangle Ii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 963. Minimum Area Rectangle II
 * https://leetcode.com/problems/minimum-area-rectangle-ii/
 * Difficulty: Medium
 *
 * You are given an array of points in the X-Y plane points where points[i] = [xi, yi].
 *
 * Return the minimum area of any rectangle formed from these points, with sides not
necessarily
 * parallel to the X and Y axes. If there is not any such rectangle, return 0.
 *
 * Answers within 10^-5 of the actual answer will be accepted.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var minAreaFreeRect = function(points) {
  const n = points.length;
  const set = new Set(points.map(p => `${p[0]},${p[1]}`));
  let result = 0;

  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      const [x1, y1] = points[i];
      const [x2, y2] = points[j];
      const dx1 = x2 - x1;
      const dy1 = y2 - y1;
```

```
      for (let k = 0; k < n; k++) {
        if (k === i || k === j) continue;
        const [x3, y3] = points[k];
        const dx2 = x3 - x1;
        const dy2 = y3 - y1;

        if (dx1 * dx2 + dy1 * dy2 !== 0) continue;

        const x4 = x2 + dx2;
        const y4 = y2 + dy2;

        if (set.has(`${x4},${y4}`)) {
          const area = Math.sqrt(dx1 * dx1 + dy1 * dy1) * Math.sqrt(dx2 * dx2 + dy2 * dy2);
          result = result === 0 ? area : Math.min(result, area);
        }
      }
    }
  }

  return result;
};
```

0964 Least Operators To Express Number

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 964. Least Operators to Express Number
 * https://leetcode.com/problems/least-operators-to-express-number/
 * Difficulty: Hard
 *
 * Given a single positive integer x, we will write an expression of the form
 * x (op1) x (op2) x (op3) x ... where each operator op1, op2, etc. is either addition,
 * subtraction, multiplication, or division (+, -, *, or /). For example, with x = 3,
 * we might write 3 * 3 / 3 + 3 - 3 which is a value of 3.
 *
 * When writing such an expression, we adhere to the following conventions:
 * - The division operator (/) returns rational numbers.
 * - There are no parentheses placed anywhere.
 * - We use the usual order of operations: multiplication and division happen before
 *   addition and subtraction.
 * - It is not allowed to use the unary negation operator (-). For example, "x - x" is
 *   a valid expression as it only uses subtraction, but "-x + x" is not because it uses
 *   negation.
 *
 * We would like to write an expression with the least number of operators such that the
 * expression equals the given target. Return the least number of operators used.
 */

/**
 * @param {number} x
 * @param {number} target
 * @return {number}
 */
var leastOpsExpressTarget = function(x, target) {
  let costP = 0;
  let costN = 0;
  let power = 0;
  let minP;
  let minN;

  while (target > 0) {
    const digit = target % x;
    target = Math.floor(target / x);

    if (power > 0) {
      const nextPositive = Math.min(digit * power + costP, (digit + 1) * power + costN);
      const nextNegative = Math.min((x - digit) * power + costP, (x - digit - 1) * power +
costN);
      costP = nextPositive;
```

```
      costN = nextNegative;
    } else {
      costP = digit * 2;
      costN = (x - digit) * 2;
    }

    minP = costP;
    minN = costN;
    power++;
  }

  return Math.min(minP, power + minN) - 1;
};
```

0965 Univalued Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 965. Univalued Binary Tree
 * https://leetcode.com/problems/univalued-binary-tree/
 * Difficulty: Easy
 *
 * A binary tree is uni-valued if every node in the tree has the same value.
 *
 * Given the root of a binary tree, return true if the given tree is uni-valued, or false
otherwise.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isUnivalTree = function(root) {
  const traverse = n => !n ? true : n.val === root.val && isUnivalTree(n);
  return traverse(root.left) && traverse(root.right);
};
```

0966 Vowel Spellchecker

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 966. Vowel Spellchecker
 * https://leetcode.com/problems/vowel-spellchecker/
 * Difficulty: Medium
 *
 * Given a wordlist, we want to implement a spellchecker that converts a query word into a
correct word.
 * For a given query word, the spell checker handles two categories of spelling mistakes:
 *
 * - Capitalization: If the query matches a word in the wordlist (case-insensitive), then the
query word
 *   is returned with the same case as the case in the wordlist.
 *   Example: wordlist = ["yellow"], query = "YellOw": correct = "yellow"
 *   Example: wordlist = ["Yellow"], query = "yellow": correct = "Yellow"
 *   Example: wordlist = ["yellow"], query = "yellow": correct = "yellow"
```

```
 * - Vowel Errors: If after replacing the vowels ('a', 'e', 'i', 'o', 'u') of the query word
with any vowel
 *   individually, it matches a word in the wordlist (case-insensitive), then the query word
is returned
 *   with the same case as the match in the wordlist.
 *   Example: wordlist = ["YellOw"], query = "yollow": correct = "YellOw"
 *   Example: wordlist = ["YellOw"], query = "yeellow": correct = "" (no match)
 *   Example: wordlist = ["YellOw"], query = "yllw": correct = "" (no match)
 *
 * In addition, the spell checker operates under the following precedence rules:
 * - When the query exactly matches a word in the wordlist (case-sensitive), you should return
 the same word back.
 * - When the query matches a word up to capitlization, you should return the first such match
 in the wordlist.
 * - When the query matches a word up to vowel errors, you should return the first such match
in the wordlist.
 * - If the query has no matches in the wordlist, you should return the empty string.
 *
 * Given some queries, return a list of words answer, where answer[i] is the correct word for
query = queries[i].
 */

/**
 * @param {string[]} words
 * @param {string[]} queries
 * @return {string[]}
 */
var spellchecker = function(words, queries) {
  const set = new Set(words);
  const map = new Map();
  const format = s => s.toLowerCase().replace(/[aeiou]/g, '_');
  words.forEach(word => {
    if (!map.has(word.toLowerCase())) map.set(word.toLowerCase(), word);
    if (!map.has(format(word))) map.set(format(word), word);
  });
  return queries.map(q => set.has(q) ? q : map.get(q.toLowerCase()) || map.get(format(q)) ||
'');
};
```

0967 Numbers With Same Consecutive Differences

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 967. Numbers With Same Consecutive Differences
 * https://leetcode.com/problems/numbers-with-same-consecutive-differences/
 * Difficulty: Medium
 *
 * Given two integers n and k, return an array of all the integers of length n where the
difference
 * between every two consecutive digits is k. You may return the answer in any order.
 *
 * Note that the integers should not have leading zeros. Integers as 02 and 043 are not
allowed.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number[]}
 */
var numsSameConsecDiff = function(n, k) {
  const result = [];

  for (let digit = 1; digit <= 9; digit++) {
    buildNumber(digit, n - 1);
  }

  return result;
```

```
  function buildNumber(current, digitsLeft) {
    if (digitsLeft === 0) {
      result.push(current);
      return;
    }

    const lastDigit = current % 10;

    if (lastDigit + k <= 9) {
      buildNumber(current * 10 + lastDigit + k, digitsLeft - 1);
    }

    if (k !== 0 && lastDigit - k >= 0) {
      buildNumber(current * 10 + lastDigit - k, digitsLeft - 1);
    }
  }
};
```

0968 Binary Tree Cameras

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 968. Binary Tree Cameras
 * https://leetcode.com/problems/binary-tree-cameras/
 * Difficulty: Hard
 *
 * You are given the root of a binary tree. We install cameras on the tree nodes where each
camera
 * at a node can monitor its parent, itself, and its immediate children.
 *
 * Return the minimum number of cameras needed to monitor all nodes of the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var minCameraCover = function(root) {
  let cameras = 0;
  const rootState = traverse(root);
  return rootState === 0 ? cameras + 1 : cameras;

  function traverse(node) {
    if (!node) return 1;

    const left = traverse(node.left);
    const right = traverse(node.right);

    if (left === 0 || right === 0) {
      cameras++;
      return 2;
    }

    if (left === 2 || right === 2) {
      return 1;
    }

    return 0;
  }
};
```

0969 Pancake Sorting

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 969. Pancake Sorting
 * https://leetcode.com/problems/pancake-sorting/
 * Difficulty: Medium
 *
 * Given an array of integers arr, sort the array by performing a series of pancake flips.
 *
 * In one pancake flip we do the following steps:
 * - Choose an integer k where 1 <= k <= arr.length.
 * - Reverse the sub-array arr[0...k-1] (0-indexed).
 *
 * For example, if arr = [3,2,1,4] and we performed a pancake flip choosing k = 3, we reverse
 * the sub-array [3,2,1], so arr = [1,2,3,4] after the pancake flip at k = 3.
 *
 * Return an array of the k-values corresponding to a sequence of pancake flips that sort arr.
 * Any valid answer that sorts the array within 10 * arr.length flips will be judged as
correct.
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var pancakeSort = function(arr) {
  const result = [];
  let end = arr.length;

  while (end > 1) {
    const maxIndex = arr.indexOf(end);

    if (maxIndex !== end - 1) {
      if (maxIndex !== 0) {
        result.push(maxIndex + 1);
        reverse(arr, maxIndex);
      }
      result.push(end);
      reverse(arr, end - 1);
    }
    end--;
  }

  return result;
};

function reverse(arr, k) {
  let left = 0;
  while (left < k) {
    [arr[left], arr[k]] = [arr[k], arr[left]];
    left++;
    k--;
  }
}
```

0970 Powerful Integers

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 970. Powerful Integers
 * https://leetcode.com/problems/powerful-integers/
 * Difficulty: Easy
```

```
 *
 * Given two positive integers x and y, an integer
 * is powerful if it is equal to x^i + y^j for some
 * integers i >= 0 and j >= 0.
 *
 * Return a list of all powerful integers that have
 * value less than or equal to bound.
 *
 * You may return the answer in any order.  In your
 * answer, each value should occur at most once.
 */

/**
 * @param {number} x
 * @param {number} y
 * @param {number} bound
 * @return {number[]}
 */
var powerfulIntegers = function(x, y, bound) {
  const result = new Set();

  for (let i = 1; i < bound; i *= x) {
    for (let j = 1; i + j <= bound; j *= y) {
      result.add(i + j);

      if (y === 1) { break };
    }

    if (x === 1) { break };
  }

  return Array.from(result);
};
```

0971 Flip Binary Tree To Match Preorder Traversal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 971. Flip Binary Tree To Match Preorder Traversal
 * https://leetcode.com/problems/flip-binary-tree-to-match-preorder-traversal/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree with n nodes, where each node is uniquely assigned
a
 * value from 1 to n. You are also given a sequence of n values voyage, which is the desired
 * pre-order traversal of the binary tree.
 *
 * Any node in the binary tree can be flipped by swapping its left and right subtrees. For
 * example, flipping node 1 will have the following effect.
 *
 * Flip the smallest number of nodes so that the pre-order traversal of the tree matches
voyage.
 *
 * Return a list of the values of all flipped nodes. You may return the answer in any order.
 * If it is impossible to flip the nodes in the tree to make the pre-order traversal match
 * voyage, return the list [-1].
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number[]} voyage
```

```
 * @return {number[]}
 */
var flipMatchVoyage = function(root, voyage) {
  const flips = [];
  let index = 0;
  return traverse(root) ? flips : [-1];

  function traverse(node) {
    if (!node) return true;
    if (node.val !== voyage[index++]) return false;

    if (!node.left) return traverse(node.right);
    if (node.left.val === voyage[index]) return traverse(node.left) && traverse(node.right);

    flips.push(node.val);
    [node.left, node.right] = [node.right, node.left];
    return traverse(node.left) && traverse(node.right);
  }
};
```

0972 Equal Rational Numbers

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 972. Equal Rational Numbers
 * https://leetcode.com/problems/equal-rational-numbers/
 * Difficulty: Hard
 *
 * Given two strings s and t, each of which represents a non-negative rational number, return
true
 * if and only if they represent the same number. The strings may use parentheses to denote
the
 * repeating part of the rational number.
 *
 * A rational number can be represented using up to three parts: <IntegerPart>,
<NonRepeatingPart>,
 * and a <RepeatingPart>. The number will be represented in one of the following three ways:
 * - <IntegerPart>
 *   - For example, 12, 0, and 123.
 * - <IntegerPart><.><NonRepeatingPart>
 *   - For example, 0.5, 1., 2.12, and 123.0001.
 * - <IntegerPart><.><NonRepeatingPart><(><RepeatingPart><)>
 *   - For example, 0.1(6), 1.(9), 123.00(1212).
 *
 * The repeating portion of a decimal expansion is conventionally denoted within a pair of
round
 * brackets. For example:
 * - 1/6 = 0.16666666... = 0.1(6) = 0.1666(6) = 0.166(66).
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isRationalEqual = function(s, t) {
  const valueS = parseRational(s);
  const valueT = parseRational(t);
  return Math.abs(valueS - valueT) < 1e-10;

  function parseRational(str) {
    const [integer, decimal] = str.split('.');
    if (!decimal) return parseInt(integer, 10);

    const openParen = decimal.indexOf('(');
    if (openParen === -1) return parseInt(integer, 10) + parseFloat(`0.${decimal}`);

    const nonRepeating = decimal.slice(0, openParen);
    const repeating = decimal.slice(openParen + 1, -1);
```

```
    const nonRepeatValue = nonRepeating ? parseFloat(`0.${nonRepeating}`) : 0;
    const repeatValue = parseInt(repeating, 10) / (10 ** repeating.length - 1)
      / (10 ** nonRepeating.length);

    return parseInt(integer, 10) + nonRepeatValue + repeatValue;
  }
};
```

0973 K Closest Points To Origin

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 973. K Closest Points to Origin
 * https://leetcode.com/problems/k-closest-points-to-origin/
 * Difficulty: Medium
 *
 * Given an array of points where points[i] = [xi, yi] represents a point on the X-Y
 * plane and an integer k, return the k closest points to the origin (0, 0).
 *
 * The distance between two points on the X-Y plane is the Euclidean distance
 * (i.e., √(x1 - x2)2 + (y1 - y2)2).
 *
 * You may return the answer in any order. The answer is guaranteed to be unique
 * (except for the order that it is in).
 */

/**
 * @param {number[][]} points
 * @param {number} k
 * @return {number[][]}
 */
var kClosest = function(points, k) {
  const getDistance = (x, y) => x * x + y * y;

  return points
    .map(point => ({ point, dist: getDistance(point[0], point[1]) }))
    .sort((a, b) => a.dist - b.dist)
    .slice(0, k)
    .map(item => item.point);
};
```

0974 Subarray Sums Divisible By K

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 974. Subarray Sums Divisible by K
 * https://leetcode.com/problems/subarray-sums-divisible-by-k/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return the number of non-empty subarrays
 * that have a sum divisible by k.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var subarraysDivByK = function(nums, k) {
  const remainderCount = new Map([[0, 1]]);
  let sum = 0;
```

```
  let result = 0;

  for (const num of nums) {
    sum += num;
    const remainder = ((sum % k) + k) % k;
    result += remainderCount.get(remainder) || 0;
    remainderCount.set(remainder, (remainderCount.get(remainder) || 0) + 1);
  }

  return result;
};
```

0975 Odd Even Jump

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 975. Odd Even Jump
 * https://leetcode.com/problems/odd-even-jump/
 * Difficulty: Hard
 *
 * You are given an integer array arr. From some starting index, you can make a series of
jumps.
 * The (1st, 3rd, 5th, ...) jumps in the series are called odd-numbered jumps, and the (2nd,
 * 4th, 6th, ...) jumps in the series are called even-numbered jumps. Note that the jumps are
 * numbered, not the indices.
 *
 * You may jump forward from index i to index j (with i < j) in the following way:
 * - During odd-numbered jumps (i.e., jumps 1, 3, 5, ...), you jump to the index j such that
 *   arr[i] <= arr[j] and arr[j] is the smallest possible value. If there are multiple such
 *   indices j, you can only jump to the smallest such index j.
 * - During even-numbered jumps (i.e., jumps 2, 4, 6, ...), you jump to the index j such that
 *   arr[i] >= arr[j] and arr[j] is the largest possible value. If there are multiple such
 *   indices j, you can only jump to the smallest such index j.
 * - It may be the case that for some index i, there are no legal jumps.
 *
 * A starting index is good if, starting from that index, you can reach the end of the array
(index
 * arr.length - 1) by jumping some number of times (possibly 0 or more than once).
 *
 * Return the number of good starting indices.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var oddEvenJumps = function(arr) {
  const oddCanReach = new Array(arr.length).fill(false);
  const evenCanReach = new Array(arr.length).fill(false);
  oddCanReach[arr.length - 1] = true;
  evenCanReach[arr.length - 1] = true;
  let result = 1;

  const sortedIndices = Array.from({ length: arr.length }, (_, i) => i);
  sortedIndices.sort((a, b) => arr[a] - arr[b] || a - b);

  const oddNext = new Array(arr.length);
  const evenNext = new Array(arr.length);
  const stack = [];

  for (const i of sortedIndices) {
    while (stack.length && stack[stack.length - 1] < i) {
      oddNext[stack.pop()] = i;
    }
    stack.push(i);
  }

  sortedIndices.sort((a, b) => arr[b] - arr[a] || a - b);
```

```
  stack.length = 0;

  for (const i of sortedIndices) {
    while (stack.length && stack[stack.length - 1] < i) {
      evenNext[stack.pop()] = i;
    }
    stack.push(i);
  }

  for (let i = arr.length - 2; i >= 0; i--) {
    if (oddNext[i] !== undefined) {
      oddCanReach[i] = evenCanReach[oddNext[i]];
      if (oddCanReach[i]) result++;
    }
    if (evenNext[i] !== undefined) {
      evenCanReach[i] = oddCanReach[evenNext[i]];
    }
  }

  return result;
};
```

0976 Largest Perimeter Triangle

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 976. Largest Perimeter Triangle
 * https://leetcode.com/problems/largest-perimeter-triangle/
 * Difficulty: Easy
 *
 * Given an array A of positive lengths, return the largest
 * perimeter of a triangle with non-zero area, formed from 3
 * of these lengths.
 *
 * If it is impossible to form any triangle of non-zero
 * area, return 0.
 */

/**
 * @param {number[]} A
 * @return {number}
 */
var largestPerimeter = function(A) {
  A.sort((a, b) => a - b);

  for (let i = A.length - 1; i > 1; --i) {
    if (A[i] < A[i - 1] + A[i - 2]) {
      return A[i] + A[i - 1] + A[i - 2];
    }
  }

  return 0;
};
```

0977 Squares Of A Sorted Array

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 977. Squares of a Sorted Array
 * https://leetcode.com/problems/squares-of-a-sorted-array/
 * Difficulty: Easy
 *
 * Given an array of integers A sorted in non-decreasing order,
```

```
 * return an array of the squares of each number,
 * also in sorted non-decreasing order.
 */

/**
 * @param {number[]} A
 * @return {number[]}
 */
var sortedSquares = function(A) {
  return A.map(n => Math.pow(n, 2)).sort((a, b) => a - b);
};
```

0978 Longest Turbulent Subarray

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 978. Longest Turbulent Subarray
 * https://leetcode.com/problems/longest-turbulent-subarray/
 * Difficulty: Medium
 *
 * Given an integer array arr, return the length of a maximum size turbulent subarray of arr.
 *
 * A subarray is turbulent if the comparison sign flips between each adjacent pair of elements
 * in the subarray.
 *
 * More formally, a subarray [arr[i], arr[i + 1], ..., arr[j]] of arr is said to be turbulent
 * if and only if:
 * - For i <= k < j:
 *   - arr[k] > arr[k + 1] when k is odd, and
 *   - arr[k] < arr[k + 1] when k is even.
 * - Or, for i <= k < j:
 *   - arr[k] > arr[k + 1] when k is even, and
 *   - arr[k] < arr[k + 1] when k is odd.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var maxTurbulenceSize = function(arr) {
  let result = 1;
  let inc = 1;
  let dec = 1;

  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > arr[i - 1]) {
      inc = dec + 1;
      dec = 1;
    } else if (arr[i] < arr[i - 1]) {
      dec = inc + 1;
      inc = 1;
    } else {
      inc = 1;
      dec = 1;
    }
    result = Math.max(result, inc, dec);
  }

  return result;
};
```

0979 Distribute Coins In Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 979. Distribute Coins in Binary Tree
 * https://leetcode.com/problems/distribute-coins-in-binary-tree/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree with n nodes where each node in the tree has
node.val
 * coins. There are n coins in total throughout the whole tree.
 *
 * In one move, we may choose two adjacent nodes and move one coin from one node to another.
 * A move may be from parent to child, or from child to parent.
 *
 * Return the minimum number of moves required to make every node have exactly one coin.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var distributeCoins = function(root) {
  let result = 0;
  traverse(root);
  return result;

  function traverse(node) {
    if (!node) return 0;

    const leftExcess = traverse(node.left);
    const rightExcess = traverse(node.right);
    const totalExcess = node.val + leftExcess + rightExcess - 1;

    result += Math.abs(leftExcess) + Math.abs(rightExcess);

    return totalExcess;
  }
};
```

0980 Unique Paths Iii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 980. Unique Paths III
 * https://leetcode.com/problems/unique-paths-iii/
 * Difficulty: Hard
 *
 * You are given an m x n integer array grid where grid[i][j] could be:
 * - 1 representing the starting square. There is exactly one starting square.
 * - 2 representing the ending square. There is exactly one ending square.
 * - 0 representing empty squares we can walk over.
 * - -1 representing obstacles that we cannot walk over.
 *
 * Return the number of 4-directional walks from the starting square to the ending
 * square, that walk over every non-obstacle square exactly once.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var uniquePathsIII = function(grid) {
```

```
  const rows = grid.length;
  const cols = grid[0].length;
  let emptySquares = 0;
  let startRow;
  let startCol;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 0) emptySquares++;
      if (grid[i][j] === 1) [startRow, startCol] = [i, j];
    }
  }

  function explorePaths(row, col, remaining) {
    if (row < 0 || row >= rows || col < 0 || col >= cols || grid[row][col] < 0) {
      return 0;
    }
    if (grid[row][col] === 2) {
      return remaining === 0 ? 1 : 0;
    }

    const current = grid[row][col];
    grid[row][col] = -1;
    const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
    let pathCount = 0;

    for (const [dr, dc] of directions) {
      pathCount += explorePaths(row + dr, col + dc, remaining - 1);
    }

    grid[row][col] = current;
    return pathCount;
  }

  return explorePaths(startRow, startCol, emptySquares + 1);
};
```

0981 Time Based Key Value Store

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 981. Time Based Key-Value Store
 * https://leetcode.com/problems/time-based-key-value-store/
 * Difficulty: Medium
 *
 * Design a time-based key-value data structure that can store multiple values for the same
key
 * at different time stamps and retrieve the key's value at a certain timestamp.
 *
 * Implement the TimeMap class:
 * - TimeMap() Initializes the object of the data structure.
 * - void set(String key, String value, int timestamp) Stores the key key with the value value
 *   at the given time timestamp.
 * - String get(String key, int timestamp) Returns a value such that set was called
previously,
 *   with timestamp_prev <= timestamp. If there are multiple such values, it returns the value
 *   associated with the largest timestamp_prev. If there are no values, it returns "".
 */

var TimeMap = function() {
  this.store = new Map();
};

/**
 * @param {string} key
 * @param {string} value
 * @param {number} timestamp
 * @return {void}
 */
```

```
TimeMap.prototype.set = function(key, value, timestamp) {
  if (!this.store.has(key)) this.store.set(key, []);
  this.store.get(key).push([timestamp, value]);
};

/**
 * @param {string} key
 * @param {number} timestamp
 * @return {string}
 */
TimeMap.prototype.get = function(key, timestamp) {
  if (!this.store.has(key)) return '';
  const entries = this.store.get(key);
  let left = 0;
  let right = entries.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    const [midTime, midValue] = entries[mid];

    if (midTime === timestamp) return midValue;
    if (midTime < timestamp) left = mid + 1;
    else right = mid - 1;
  }

  return right >= 0 ? entries[right][1] : '';
};
```

0982 Triples With Bitwise And Equal To Zero

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 982. Triples with Bitwise AND Equal To Zero
 * https://leetcode.com/problems/triples-with-bitwise-and-equal-to-zero/
 * Difficulty: Hard
 *
 * Given an integer array nums, return the number of AND triples.
 *
 * An AND triple is a triple of indices (i, j, k) such that:
 * - 0 <= i < nums.length
 * - 0 <= j < nums.length
 * - 0 <= k < nums.length
 * - nums[i] & nums[j] & nums[k] == 0, where & represents the bitwise-AND operator.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countTriplets = function(nums) {
  const map = new Map();

  for (const first of nums) {
    for (const second of nums) {
      const pairAnd = first & second;
      map.set(pairAnd, (map.get(pairAnd) || 0) + 1);
    }
  }

  let result = 0;
  for (const third of nums) {
    for (const [pair, count] of map) {
      if ((pair & third) === 0) {
        result += count;
      }
    }
  }

  return result;
```

```
};
```

0983 Minimum Cost For Tickets

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 983. Minimum Cost For Tickets
 * https://leetcode.com/problems/minimum-cost-for-tickets/
 * Difficulty: Medium
 *
 * You have planned some train traveling one year in advance. The days of the year in which
you will
 * travel are given as an integer array days. Each day is an integer from 1 to 365.
 *
 * Train tickets are sold in three different ways:
 * - a 1-day pass is sold for costs[0] dollars,
 * - a 7-day pass is sold for costs[1] dollars, and
 * - a 30-day pass is sold for costs[2] dollars.
 *
 * The passes allow that many days of consecutive travel.
 *
 * For example, if we get a 7-day pass on day 2, then we can travel for 7 days: 2, 3, 4, 5, 6,
 7,
 * and 8.
 *
 * Return the minimum number of dollars you need to travel every day in the given list of
days.
 */

/**
 * @param {number[]} days
 * @param {number[]} costs
 * @return {number}
 */
var mincostTickets = function(days, costs) {
  const lastDay = days[days.length - 1];
  const travelDays = new Set(days);
  const minCost = new Array(lastDay + 1).fill(0);

  for (let day = 1; day <= lastDay; day++) {
    if (!travelDays.has(day)) {
      minCost[day] = minCost[day - 1];
      continue;
    }

    const oneDayCost = minCost[day - 1] + costs[0];
    const sevenDayCost = minCost[Math.max(0, day - 7)] + costs[1];
    const thirtyDayCost = minCost[Math.max(0, day - 30)] + costs[2];

    minCost[day] = Math.min(oneDayCost, sevenDayCost, thirtyDayCost);
  }

  return minCost[lastDay];
};
```

0984 String Without Aaa Or Bbb

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 984. String Without AAA or BBB
 * https://leetcode.com/problems/string-without-aaa-or-bbb/
 * Difficulty: Medium
 *
```

```
 * Given two integers a and b, return any string s such that:
 * - s has length a + b and contains exactly a 'a' letters, and exactly b 'b' letters,
 * - The substring 'aaa' does not occur in s, and
 * - The substring 'bbb' does not occur in s.
 */

/**
 * @param {number} a
 * @param {number} b
 * @return {string}
 */
var strWithout3a3b = function(a, b) {
  let result = '';

  while (a > 0 || b > 0) {
    if (a > b && a > 0 && !result.endsWith('aa')) {
      result += 'a';
      a--;
    } else if (b > 0 && !result.endsWith('bb')) {
      result += 'b';
      b--;
    } else if (a > 0) {
      result += 'a';
      a--;
    } else {
      result += 'b';
      b--;
    }
  }

  return result;
};
```

0985 Sum Of Even Numbers After Queries

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 985. Sum of Even Numbers After Queries
 * https://leetcode.com/problems/sum-of-even-numbers-after-queries/
 * Difficulty: Easy
 *
 * We have an array A of integers, and an array queries of queries.
 *
 * For the i-th query val = queries[i][0], index = queries[i][1],
 * we add val to A[index].  Then, the answer to the i-th query
 * is the sum of the even values of A.
 *
 * (Here, the given index = queries[i][1] is a 0-based index, and
 * each query permanently modifies the array A.)
 *
 * Return the answer to all queries. Your answer array should
 * have answer[i] as the answer to the i-th query.
 */

/**
 * @param {number[]} A
 * @param {number[][]} queries
 * @return {number[]}
 */
var sumEvenAfterQueries = function(A, queries) {
  return queries.reduce((sums, query) => {
    A[query[1]] += query[0];
    return [...sums, A.reduce((sum, n) => sum + (n % 2 === 0 ? n : 0), 0)];
  }, []);
};

// Significantly faster, less elegant approach:
/**
 * @param {number[]} A
```

```
 * @param {number[][]} queries
 * @return {number[]}
 */
var sumEvenAfterQueries = function(A, queries) {
  return queries.reduce((sums, query) => {
    const [v, i] = query;
    const prev = A[i];
    const lastSum = sums.length ? sums[sums.length - 1] : A.reduce((sum, n) => sum + (n % 2
=== 0 ? n : 0), 0);
    A[i] += v;
    sums.push(lastSum - (prev % 2 === 0 ? prev : 0) + (A[i] % 2 === 0 ? A[i] : 0));
    return sums;
  }, []);
};
```

0986 Interval List Intersections

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 986. Interval List Intersections
 * https://leetcode.com/problems/interval-list-intersections/
 * Difficulty: Medium
 *
 * You are given two lists of closed intervals, firstList and secondList, where
 * firstList[i] = [starti, endi] and secondList[j] = [startj, endj]. Each list of
 * intervals is pairwise disjoint and in sorted order.
 *
 * Return the intersection of these two interval lists.
 *
 * A closed interval [a, b] (with a <= b) denotes the set of real numbers x with a <= x <= b.
 *
 * The intersection of two closed intervals is a set of real numbers that are either empty
 * or represented as a closed interval. For example, the intersection of [1, 3] and
 * [2, 4] is [2, 3].
 */

/**
 * @param {number[][]} firstList
 * @param {number[][]} secondList
 * @return {number[][]}
 */
var intervalIntersection = function(firstList, secondList) {
  const result = [];
  let i = 0;
  let j = 0;

  while (i < firstList.length && j < secondList.length) {
    const start = Math.max(firstList[i][0], secondList[j][0]);
    const end = Math.min(firstList[i][1], secondList[j][1]);

    if (start <= end) {
      result.push([start, end]);
    }

    if (firstList[i][1] < secondList[j][1]) {
      i++;
    } else {
      j++;
    }
  }

  return result;
};
```

0987 Vertical Order Traversal Of A Binary Tree

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 987. Vertical Order Traversal of a Binary Tree
 * https://leetcode.com/problems/vertical-order-traversal-of-a-binary-tree/
 * Difficulty: Hard
 *
 * Given the root of a binary tree, calculate the vertical order traversal of the binary tree.
 *
 * For each node at position (row, col), its left and right children will be at positions
 * (row + 1, col - 1) and (row + 1, col + 1) respectively. The root of the tree is at (0, 0).
 *
 * The vertical order traversal of a binary tree is a list of top-to-bottom orderings for each
 * column index starting from the leftmost column and ending on the rightmost column. There
may
 * be multiple nodes in the same row and same column. In such a case, sort these nodes by
their
 * values.
 *
 * Return the vertical order traversal of the binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var verticalTraversal = function(root) {
  const nodes = [];

  function traverse(node, row, col) {
    if (!node) return;
    nodes.push([col, row, node.val]);
    traverse(node.left, row + 1, col - 1);
    traverse(node.right, row + 1, col + 1);
  }

  traverse(root, 0, 0);
  nodes.sort((a, b) => a[0] - b[0] || a[1] - b[1] || a[2] - b[2]);

  const result = [];
  let currentCol = nodes[0][0];
  let currentGroup = [];

  for (const [col, _, val] of nodes) {
    if (col !== currentCol) {
      result.push(currentGroup);
      currentGroup = [val];
      currentCol = col;
    } else {
      currentGroup.push(val);
    }
  }

  result.push(currentGroup);
  return result;
};
```

0988 Smallest String Starting From Leaf

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 988. Smallest String Starting From Leaf
 * https://leetcode.com/problems/smallest-string-starting-from-leaf/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree where each node has a value in the range [0, 25]
 * representing the letters 'a' to 'z'.
 *
 * Return the lexicographically smallest string that starts at a leaf of this tree and ends
 * at the root.
 *
 * As a reminder, any shorter prefix of a string is lexicographically smaller.
 *
 * For example, "ab" is lexicographically smaller than "aba".
 *
 * A leaf of a node is a node that has no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {string}
 */
var smallestFromLeaf = function(root) {
  let result = null;
  traverse(root, []);
  return result || '';

  function traverse(node, path) {
    if (!node) return;

    path.unshift(String.fromCharCode(node.val + 97));

    if (!node.left && !node.right) {
      const current = path.join('');
      if (!result || current < result) {
        result = current;
      }
    }

    traverse(node.left, path);
    traverse(node.right, path);
    path.shift();
  }
};
```

0989 Add To Array Form Of Integer

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 989. Add to Array-Form of Integer
 * https://leetcode.com/problems/add-to-array-form-of-integer/
 * Difficulty: Easy
```

```
 *
 * For a non-negative integer X, the array-form of X is
 * an array of its digits in left to right order.
 * For example, if X = 1231, then the array form is [1,2,3,1].
 *
 * Given the array-form A of a non-negative integer X,
 * return the array-form of the integer X+K.
 */

/**
 * @param {number[]} A
 * @param {number} K
 * @return {number[]}
 */
var addToArrayForm = function(A, K) {
  return String(BigInt(A.join('')) + BigInt(K)).split('');
};
```

0990 Satisfiability Of Equality Equations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 990. Satisfiability of Equality Equations
 * https://leetcode.com/problems/satisfiability-of-equality-equations/
 * Difficulty: Medium
 *
 * You are given an array of strings equations that represent relationships between variables
where
 * each string equations[i] is of length 4 and takes one of two different forms: "xi==yi" or
 * "xi!=yi".Here, xi and yi are lowercase letters (not necessarily different) that represent
 * one-letter variable names.
 *
 * Return true if it is possible to assign integers to variable names so as to satisfy all the
 given
 * equations, or false otherwise.
 */

/**
 * @param {string[]} equations
 * @return {boolean}
 */
var equationsPossible = function(equations) {
  const parent = new Array(26).fill(0).map((_, i) => i);

  for (const equation of equations) {
    if (equation[1] === '=') {
      const x = equation[0].charCodeAt(0) - 97;
      const y = equation[3].charCodeAt(0) - 97;
      union(x, y);
    }
  }

  for (const equation of equations) {
    if (equation[1] === '!') {
      const x = equation[0].charCodeAt(0) - 97;
      const y = equation[3].charCodeAt(0) - 97;
      if (find(x) === find(y)) {
        return false;
      }
    }
  }

  return true;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
```

```
  }

  function union(x, y) {
    parent[find(x)] = find(y);
  }
};
```

0991 Broken Calculator

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 991. Broken Calculator
 * https://leetcode.com/problems/broken-calculator/
 * Difficulty: Medium
 *
 * There is a broken calculator that has the integer startValue on its display initially.
 * In one operation, you can:
 * - multiply the number on display by 2, or
 * - subtract 1 from the number on display.
 *
 * Given two integers startValue and target, return the minimum number of operations needed
 * to display target on the calculator.
 */

/**
 * @param {number} startValue
 * @param {number} target
 * @return {number}
 */
var brokenCalc = function(startValue, target) {
  let operations = 0;
  let current = target;

  while (current > startValue) {
    if (current % 2 === 0) {
      current = current / 2;
    } else {
      current++;
    }
    operations++;
  }

  return operations + startValue - current;
};
```

0992 Subarrays With K Different Integers

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 992. Subarrays with K Different Integers
 * https://leetcode.com/problems/subarrays-with-k-different-integers/
 * Difficulty: Hard
 *
 * Given an integer array nums and an integer k, return the number of good subarrays of nums.
 *
 * A good array is an array where the number of different integers in that array is exactly k.
 *
 * For example, [1,2,3,1,2] has 3 different integers: 1, 2, and 3.
 *
 * A subarray is a contiguous part of an array.
 */
```

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var subarraysWithKDistinct = function(nums, k) {
  return countAtMostK(k) - countAtMostK(k - 1);

  function countAtMostK(distinct) {
    const map = new Map();
    let left = 0;
    let result = 0;

    for (let right = 0; right < nums.length; right++) {
      map.set(nums[right], (map.get(nums[right]) || 0) + 1);

      while (map.size > distinct) {
        map.set(nums[left], map.get(nums[left]) - 1);
        if (map.get(nums[left]) === 0) {
          map.delete(nums[left]);
        }
        left++;
      }

      result += right - left + 1;
    }

    return result;
  }
};
```

0993 Cousins In Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 993. Cousins in Binary Tree
 * https://leetcode.com/problems/cousins-in-binary-tree/
 * Difficulty: Easy
 *
 * Given the root of a binary tree with unique values and the values of two different nodes of
 the
 * tree x and y, return true if the nodes corresponding to the values x and y in the tree are
 * cousins, or false otherwise.
 *
 * Two nodes of a binary tree are cousins if they have the same depth with different parents.
 *
 * Note that in a binary tree, the root node is at the depth 0, and children of each depth k
node
 * are at the depth k + 1.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} x
 * @param {number} y
 * @return {boolean}
 */
var isCousins = function(root, x, y) {
  let xDepth = 0;
  let yDepth = 0;
  let xParent = null;
```

```
  traverse(root, 0, null);

  return xDepth === yDepth && xParent !== yParent;

  function traverse(node, depth, parent) {
    if (!node) return;

    if (node.val === x) {
      xDepth = depth;
      xParent = parent;
    }
    if (node.val === y) {
      yDepth = depth;
      yParent = parent;
    }

    traverse(node.left, depth + 1, node);
    traverse(node.right, depth + 1, node);
  }
};
```

0994 Rotting Oranges

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 994. Rotting Oranges
 * https://leetcode.com/problems/rotting-oranges/
 * Difficulty: Medium
 *
 * You are given an m x n grid where each cell can have one of three values:
 * - 0 representing an empty cell,
 * - 1 representing a fresh orange, or
 * - 2 representing a rotten orange.
 *
 * Every minute, any fresh orange that is 4-directionally adjacent to a rotten
 * orange becomes rotten.
 *
 * Return the minimum number of minutes that must elapse until no cell has a
 * fresh orange. If this is impossible, return -1.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var orangesRotting = function(grid) {
  const queue = [];
  let count = 0;

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[0].length; j++) {
      if (grid[i][j] === 2) {
        queue.push([i, j, 0]);
      }
    }
  }

  while (queue.length) {
    const [i, j, k] = queue.shift();
    [[-1, 0], [1, 0], [0, -1], [0, 1]].forEach(p => {
      const [x, y, z] = [i + p[0], j + p[1], k + 1];
      if (x > -1 && x < grid.length && y > -1 && y < grid[0].length) {
        if (grid[x][y] === 1) {
          grid[x][y] = 2;
          count = Math.max(count, z);
          queue.push([x, y, z]);
        }
      }
```

```
    });
  }

  const isIncomplete = grid.some(row => row.some(n => n === 1));
  return isIncomplete ? -1 : count;
};
```

0995 Minimum Number Of K Consecutive Bit Flips

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 995. Minimum Number of K Consecutive Bit Flips
 * https://leetcode.com/problems/minimum-number-of-k-consecutive-bit-flips/
 * Difficulty: Hard
 *
 * You are given a binary array nums and an integer k.
 *
 * A k-bit flip is choosing a subarray of length k from nums and simultaneously changing every
 0 in
 * the subarray to 1, and every 1 in the subarray to 0.
 *
 * Return the minimum number of k-bit flips required so that there is no 0 in the array. If it
 is
 * not possible, return -1.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minKBitFlips = function(nums, k) {
  const queue = [];
  let flipCount = 0;
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    if (queue.length && queue[0] === i) {
      queue.shift();
      flipCount--;
    }

    if ((nums[i] + flipCount) % 2 === 0) {
      if (i + k > nums.length) return -1;
      result++;
      flipCount++;
      queue.push(i + k);
    }
  }

  return result;
};
```

0996 Number Of Squareful Arrays

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 996. Number of Squareful Arrays
 * https://leetcode.com/problems/number-of-squareful-arrays/
 * Difficulty: Hard
 *
```

```
 * An array is squareful if the sum of every pair of adjacent elements is a perfect square.
 *
 * Given an integer array nums, return the number of permutations of nums that are squareful.
 *
 * Two permutations perm1 and perm2 are different if there is some index i such that
 * perm1[i] != perm2[i].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var numSquarefulPerms = function(nums) {
  nums.sort((a, b) => a - b);
  const length = nums.length;
  const used = new Array(length).fill(false);
  let result = 0;

  generatePermutations(0, length);
  return result;

  function isPerfectSquare(num) {
    const sqrt = Math.sqrt(num);
    return Number.isInteger(sqrt);
  }

  function generatePermutations(prevNum, remaining) {
    if (remaining === 0) {
      result++;
      return;
    }

    for (let i = 0; i < length; i++) {
      if (!used[i] && (i === 0 || nums[i] !== nums[i-1] || used[i-1])
          && (remaining === length || isPerfectSquare(prevNum + nums[i]))) {
        used[i] = true;
        generatePermutations(nums[i], remaining - 1);
        used[i] = false;
      }
    }
  }
};
```

0997 Find The Town Judge

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 997. Find the Town Judge
 * https://leetcode.com/problems/find-the-town-judge/
 * Difficulty: Easy
 *
 * In a town, there are n people labeled from 1 to n. There is a rumor that one of these
 * people is secretly the town judge.
 *
 * If the town judge exists, then:
 * - The town judge trusts nobody.
 * - Everybody (except for the town judge) trusts the town judge.
 * - There is exactly one person that satisfies properties 1 and 2.
 *
 * You are given an array trust where trust[i] = [ai, bi] representing that the person labeled
 * ai trusts the person labeled bi.
 *
 * Return the label of the town judge if the town judge exists and can be identified, or
return
 * -1 otherwise.
 */

/**
 * @param {number} n
```

```
 * @param {number[][]} trust
 * @return {number}
 */
var findJudge = function(n, trust) {
  if (!trust.length) {
    return n === 1 ? n : -1;
  }

  const map = new Map();
  trust.forEach(([_, value]) => map.set(value, (map.get(value) || 0 ) + 1));

  const [judge, count] = [...map].sort(([,a], [,b]) => b - a)[0];
  return count === n - 1  && !trust.find(([key]) => key === judge) ? judge : -1;
};
```

0998 Maximum Binary Tree Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 998. Maximum Binary Tree II
 * https://leetcode.com/problems/maximum-binary-tree-ii/
 * Difficulty: Medium
 *
 * A maximum tree is a tree where every node has a value greater than any other value
 * in its subtree.
 *
 * You are given the root of a maximum binary tree and an integer val.
 *
 * Just as in the previous problem, the given tree was constructed from a list a
 * (root = Construct(a)) recursively with the following Construct(a) routine:
 * - If a is empty, return null.
 * - Otherwise, let a[i] be the largest element of a. Create a root node with the value a[i].
 * - The left child of root will be Construct([a[0], a[1], ..., a[i - 1]]).
 * - The right child of root will be Construct([a[i + 1], a[i + 2], ..., a[a.length - 1]]).
 * - Return root.
 *
 * Note that we were not given a directly, only a root node root = Construct(a).
 *
 * Suppose b is a copy of a with the value val appended to it. It is guaranteed that b has
 * unique values.
 *
 * Return Construct(b).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} val
 * @return {TreeNode}
 */
var insertIntoMaxTree = function(root, val) {
  if (!root || val > root.val) {
    const newRoot = createNode(val);
    newRoot.left = root;
    return newRoot;
  }

  root.right = insertIntoMaxTree(root.right, val);
  return root;

  function createNode(value) {
    return new TreeNode(value);
```

```
  }
};
```

0999 Available Captures For Rook

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 999. Available Captures for Rook
 * https://leetcode.com/problems/available-captures-for-rook/
 * Difficulty: Easy
 *
 * You are given an 8 x 8 matrix representing a chessboard. There is exactly one white rook
 * represented by 'R', some number of white bishops 'B', and some number of black pawns 'p'.
 * Empty squares are represented by '.'.
 *
 * A rook can move any number of squares horizontally or vertically (up, down, left, right)
 * until it reaches another piece or the edge of the board. A rook is attacking a pawn if it
 * can move to the pawn's square in one move.
 *
 * Note: A rook cannot move through other pieces, such as bishops or pawns. This means a rook
 * cannot attack a pawn if there is another piece blocking the path.
 *
 * Return the number of pawns the white rook is attacking.
 */

/**
 * @param {character[][]} board
 * @return {number}
 */
var numRookCaptures = function(board) {
  let rookRow;
  let rookCol;

  for (let i = 0; i < 8; i++) {
    for (let j = 0; j < 8; j++) {
      if (board[i][j] === 'R') {
        rookRow = i;
        rookCol = j;
        break;
      }
    }
  }

  const result = countPawns(-1, 0) + countPawns(1, 0)
    + countPawns(0, -1) + countPawns(0, 1);

  return result;

  function countPawns(rowStep, colStep) {
    let row = rookRow + rowStep;
    let col = rookCol + colStep;

    while (row >= 0 && row < 8 && col >= 0 && col < 8) {
      if (board[row][col] === 'p') return 1;
      if (board[row][col] === 'B') return 0;
      row += rowStep;
      col += colStep;
    }
    return 0;
  }
};
```

1000 Minimum Cost To Merge Stones

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1000. Minimum Cost to Merge Stones
 * https://leetcode.com/problems/minimum-cost-to-merge-stones/
 * Difficulty: Hard
 *
 * There are n piles of stones arranged in a row. The ith pile has stones[i] stones.
 *
 * A move consists of merging exactly k consecutive piles into one pile, and the cost of
 * this move is equal to the total number of stones in these k piles.
 *
 * Return the minimum cost to merge all piles of stones into one pile. If it is impossible,
 * return -1.
 */

/**
 * @param {number[]} stones
 * @param {number} k
 * @return {number}
 */
var mergeStones = function(stones, k) {
  const n = stones.length;
  if ((n - 1) % (k - 1) !== 0) return -1;

  const prefixSums = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) {
    prefixSums[i + 1] = prefixSums[i] + stones[i];
  }

  const dp = new Array(n).fill().map(() => new Array(n).fill(0));

  for (let len = k; len <= n; len++) {
    for (let start = 0; start + len <= n; start++) {
      const end = start + len - 1;
      dp[start][end] = Infinity;

      for (let mid = start; mid < end; mid += k - 1) {
        dp[start][end] = Math.min(
          dp[start][end],
          dp[start][mid] + dp[mid + 1][end]
        );
      }

      if ((len - 1) % (k - 1) === 0) {
        dp[start][end] += prefixSums[end + 1] - prefixSums[start];
      }
    }
  }

  return dp[0][n - 1];
};
```

1001 Grid Illumination

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1001. Grid Illumination
 * https://leetcode.com/problems/grid-illumination/
 * Difficulty: Hard
 *
 * There is a 2D grid of size n x n where each cell of this grid has a lamp that is initially
 * turned off.
 *
 * You are given a 2D array of lamp positions lamps, where lamps[i] = [rowi, coli] indicates
that
 * the lamp at grid[rowi][coli] is turned on. Even if the same lamp is listed more than once,
it
```

```
 * is turned on.
 *
 * When a lamp is turned on, it illuminates its cell and all other cells in the same row,
column,
 * or diagonal.
 *
 * You are also given another 2D array queries, where queries[j] = [rowj, colj]. For the jth
query,
 * determine whether grid[rowj][colj] is illuminated or not. After answering the jth query,
turn off
 * the lamp at grid[rowj][colj] and its 8 adjacent lamps if they exist. A lamp is adjacent if
its
 * cell shares either a side or corner with grid[rowj][colj].
 *
 * Return an array of integers ans, where ans[j] should be 1 if the cell in the jth query was
 * illuminated, or 0 if the lamp was not.
 */

/**
 * @param {number} n
 * @param {number[][]} lamps
 * @param {number[][]} queries
 * @return {number[]}
 */
var gridIllumination = function(n, lamps, queries) {
  const rowCount = new Map();
  const colCount = new Map();
  const diag1Count = new Map();
  const diag2Count = new Map();
  const lampSet = new Set();

  for (const [r, c] of lamps) {
    if (!lampSet.has(`${r},${c}`)) {
      lampSet.add(`${r},${c}`);
      rowCount.set(r, (rowCount.get(r) || 0) + 1);
      colCount.set(c, (colCount.get(c) || 0) + 1);
      diag1Count.set(r - c, (diag1Count.get(r - c) || 0) + 1);
      diag2Count.set(r + c, (diag2Count.get(r + c) || 0) + 1);
    }
  }

  const directions = [[0, 0], [0, 1], [0, -1], [1, 0], [1, 1], [1, -1], [-1, 0], [-1, 1], [-1,
 -1]];
  const result = new Array(queries.length);

  for (let i = 0; i < queries.length; i++) {
    const [r, c] = queries[i];
    result[i] = (rowCount.get(r) > 0 || colCount.get(c) > 0
      || diag1Count.get(r - c) > 0 || diag2Count.get(r + c) > 0) ? 1 : 0;

    for (const [dr, dc] of directions) {
      const newR = r + dr;
      const newC = c + dc;
      const key = `${newR},${newC}`;

      if (newR >= 0 && newR < n && newC >= 0 && newC < n && lampSet.has(key)) {
        lampSet.delete(key);
        rowCount.set(newR, rowCount.get(newR) - 1);
        colCount.set(newC, colCount.get(newC) - 1);
        diag1Count.set(newR - newC, diag1Count.get(newR - newC) - 1);
        diag2Count.set(newR + newC, diag2Count.get(newR + newC) - 1);
      }
    }
  }

  return result;
};
```

1002 Find Common Characters

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1002. Find Common Characters
 * https://leetcode.com/problems/find-common-characters/
 * Difficulty: Easy
 *
 * Given a string array words, return an array of all characters that show up in all strings
within
 * the words (including duplicates). You may return the answer in any order.
 */

/**
 * @param {string[]} words
 * @return {string[]}
 */
var commonChars = function(words) {
  return words
    .map(word => [...word])
    .reduce((common, word) => common.filter(s => {
      const index = word.indexOf(s);
      if (index !== -1) {
        word.splice(index, 1);
      }
      return index !== -1;
    }), [...words[0]]);
};
```

1003 Check If Word Is Valid After Substitutions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1003. Check If Word Is Valid After Substitutions
 * https://leetcode.com/problems/check-if-word-is-valid-after-substitutions/
 * Difficulty: Medium
 *
 * Given a string s, determine if it is valid.
 *
 * A string s is valid if, starting with an empty string t = "", you can transform t into s
after
 * performing the following operation any number of times:
 * - Insert string "abc" into any position in t. More formally, t becomes tleft + "abc" +
tright,
 *   where t == tleft + tright. Note that tleft and tright may be empty.
 *
 * Return true if s is a valid string, otherwise, return false.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var isValid = function(s) {
  const stack = [];

  for (const char of s) {
    if (char === 'c') {
      if (stack.length < 2 || stack.pop() !== 'b' || stack.pop() !== 'a') {
        return false;
      }
    } else {
      stack.push(char);
    }
  }

  return stack.length === 0;
};
```

1004 Max Consecutive Ones Iii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1004. Max Consecutive Ones III
 * https://leetcode.com/problems/max-consecutive-ones-iii/
 * Difficulty: Medium
 *
 * Given a binary array nums and an integer k, return the maximum number of consecutive 1's
 * in the array if you can flip at most k 0's.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var longestOnes = function(nums, k) {
  let left = 0;
  let right = 0;

  while (right < nums.length) {
    if (!nums[right]) k--;
    if (k < 0) {
      if (!nums[left]) k++;
      left++;
    }
    right++;
  }

  return right - left;
};
```

1005 Maximize Sum Of Array After K Negations

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1005. Maximize Sum Of Array After K Negations
 * https://leetcode.com/problems/maximize-sum-of-array-after-k-negations/
 * Difficulty: Easy
 *
 * Given an integer array nums and an integer k, modify the array in the following way:
 * - choose an index i and replace nums[i] with -nums[i].
 *
 * You should apply this process exactly k times. You may choose the same index i
 * multiple times.
 *
 * Return the largest possible sum of the array after modifying it in this way.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var largestSumAfterKNegations = function(nums, k) {
  while (k) {
    const i = nums.indexOf(Math.min(...nums));
    nums[i] *= -1;
    k--;
  }
  return nums.reduce((sum, n) => sum + n, 0);
};
```

1006 Clumsy Factorial

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1006. Clumsy Factorial
 * https://leetcode.com/problems/clumsy-factorial/
 * Difficulty: Medium
 *
 * The factorial of a positive integer n is the product of all positive integers less
 * than or equal to n.
 * - For example, factorial(10) = 10 * 9 * 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1.
 *
 * We make a clumsy factorial using the integers in decreasing order by swapping out the
 * multiply operations for a fixed rotation of operations with multiply '*', divide '/',
 * add '+', and subtract '-' in this order.
 * - For example, clumsy(10) = 10 * 9 / 8 + 7 - 6 * 5 / 4 + 3 - 2 * 1.
 *
 * However, these operations are still applied using the usual order of operations of
 * arithmetic. We do all multiplication and division steps before any addition or
 * subtraction steps, and multiplication and division steps are processed left to right.
 *
 * Additionally, the division that we use is floor division such that 10 * 9 / 8 = 90 / 8 =
11.
 *
 * Given an integer n, return the clumsy factorial of n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var clumsy = function(n) {
  if (n <= 2) return n;
  if (n === 3) return 6;

  let result = Math.floor(n * (n - 1) / (n - 2)) + (n - 3);
  n -= 4;

  while (n >= 4) {
    result -= Math.floor(n * (n - 1) / (n - 2)) - (n - 3);
    n -= 4;
  }

  if (n === 3) result -= 6;
  else if (n === 2) result -= 2;
  else if (n === 1) result -= 1;

  return result;
};
```

1007 Minimum Domino Rotations For Equal Row

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1007. Minimum Domino Rotations For Equal Row
 * https://leetcode.com/problems/minimum-domino-rotations-for-equal-row/
 * Difficulty: Medium
 *
 * In a row of dominoes, tops[i] and bottoms[i] represent the top and bottom halves of the
 * ith domino. (A domino is a tile with two numbers from 1 to 6 - one on each half of the
tile.)
 *
 * We may rotate the ith domino, so that tops[i] and bottoms[i] swap values.
```

```
 *
 * Return the minimum number of rotations so that all the values in tops are the same, or all
 * the values in bottoms are the same.
 *
 * If it cannot be done, return -1.
 */

/**
 * @param {number[]} tops
 * @param {number[]} bottoms
 * @return {number}
 */
var minDominoRotations = function(tops, bottoms) {
  const topResult = countRotations(tops[0]);
  if (topResult !== -1) return topResult;

  return countRotations(bottoms[0]);

  function countRotations(target) {
    let topCount = 0;
    let bottomCount = 0;

    for (let i = 0; i < tops.length; i++) {
      if (tops[i] !== target && bottoms[i] !== target) return -1;
      topCount += tops[i] !== target ? 1 : 0;
      bottomCount += bottoms[i] !== target ? 1 : 0;
    }

    return Math.min(topCount, bottomCount);
  }
};
```

1008 Construct Binary Search Tree From Preorder Traversal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1008. Construct Binary Search Tree from Preorder Traversal
 * https://leetcode.com/problems/construct-binary-search-tree-from-preorder-traversal/
 * Difficulty: Medium
 *
 * Given an array of integers preorder, which represents the preorder traversal of a BST
(i.e.,
 * binary search tree), construct the tree and return its root.
 *
 * It is guaranteed that there is always possible to find a binary search tree with the given
 * requirements for the given test cases.
 *
 * A binary search tree is a binary tree where for every node, any descendant of Node.left has
 * a value strictly less than Node.val, and any descendant of Node.right has a value strictly
 * greater than Node.val.
 *
 * A preorder traversal of a binary tree displays the value of the node first, then traverses
 * Node.left, then traverses Node.right.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} preorder
 * @return {TreeNode}
 */
var bstFromPreorder = function(preorder) {
  let index = 0;
```

```
  return buildBST(Infinity);

  function buildBST(bound) {
    if (index >= preorder.length || preorder[index] > bound) return null;

    const node = new TreeNode(preorder[index++]);
    node.left = buildBST(node.val);
    node.right = buildBST(bound);

    return node;
  }
};
```

1009 Complement Of Base 10 Integer

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1009. Complement of Base 10 Integer
 * https://leetcode.com/problems/complement-of-base-10-integer/
 * Difficulty: Easy
 *
 * Every non-negative integer N has a binary representation.
 * For example, 5 can be represented as "101" in binary,
 * 11 as "1011" in binary, and so on.  Note that except for
 * N = 0, there are no leading zeroes in any binary representation.
 *
 * The complement of a binary representation is the number in binary
 * you get when changing every 1 to a 0 and 0 to a 1. For example,
 * the complement of "101" in binary is "010" in binary.
 *
 * For a given number N in base-10, return the complement of it's
 * binary representation as a base-10 integer.
 */

/**
 * @param {number} N
 * @return {number}
 */
var bitwiseComplement = function(N) {
  return parseInt(N.toString(2).split('').map(n => n ^ 1).join(''), 2);
};
```

1010 Pairs Of Songs With Total Durations Divisible By 60

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1010. Pairs of Songs With Total Durations Divisible by 60
 * https://leetcode.com/problems/pairs-of-songs-with-total-durations-divisible-by-60/
 * Difficulty: Medium
 *
 * You are given a list of songs where the ith song has a duration of time[i] seconds.
 *
 * Return the number of pairs of songs for which their total duration in seconds is
 * divisible by 60. Formally, we want the number of indices i, j such that i < j with
 * (time[i] + time[j]) % 60 == 0.
 */

/**
 * @param {number[]} time
 * @return {number}
 */
var numPairsDivisibleBy60 = function(time) {
  const mods = new Array(60).fill(0);
```

```
  return time.reduce((sum, t) => {
    sum += mods[(60 - t % 60) % 60];
    mods[t % 60]++;
    return sum;
  }, 0);
};
```

1011 Capacity To Ship Packages Within D Days

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1011. Capacity To Ship Packages Within D Days
 * https://leetcode.com/problems/capacity-to-ship-packages-within-d-days/
 * Difficulty: Medium
 *
 * A conveyor belt has packages that must be shipped from one port to another within days
days.
 *
 * The ith package on the conveyor belt has a weight of weights[i]. Each day, we load the ship
 * with packages on the conveyor belt (in the order given by weights). We may not load more
weight
 * than the maximum weight capacity of the ship.
 *
 * Return the least weight capacity of the ship that will result in all the packages on the
conveyor
 * belt being shipped within days days.
 */

/**
 * @param {number[]} weights
 * @param {number} days
 * @return {number}
 */
var shipWithinDays = function(weights, days) {
  let left = Math.max(...weights);
  let right = weights.reduce((sum, weight) => sum + weight, 0);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    let currentDays = 1;
    let currentWeight = 0;

    for (const weight of weights) {
      if (currentWeight + weight > mid) {
        currentDays++;
        currentWeight = weight;
      } else {
        currentWeight += weight;
      }
    }

    if (currentDays > days) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
};
```

1012 Numbers With Repeated Digits

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1012. Numbers With Repeated Digits
 * https://leetcode.com/problems/numbers-with-repeated-digits/
 * Difficulty: Hard
 *
 * Given an integer n, return the number of positive integers in the range [1, n] that have at
 * least one repeated digit.
 */

/**
 * @param {number} n
 * @return {number}
 */
var numDupDigitsAtMostN = function(n) {
  const digits = String(n).split('').map(Number);
  const length = digits.length;
  let result = 0;

  function calculatePermutations(size, available) {
    let total = 1;
    for (let i = 0; i < size; i++) {
      total *= available - i;
    }
    return total;
  }

  for (let i = 1; i < length; i++) {
    result += 9 * calculatePermutations(i - 1, 9);
  }

  const used = new Set();
  for (let i = 0; i < length; i++) {
    const start = i === 0 ? 1 : 0;
    const limit = digits[i];

    for (let d = start; d < limit; d++) {
      if (!used.has(d)) {
        result += calculatePermutations(length - i - 1, 10 - used.size - 1);
      }
    }

    if (used.has(digits[i])) break;
    used.add(digits[i]);
    if (i === length - 1) result++;
  }

  return n - result;
};
```

1013 Partition Array Into Three Parts With Equal Sum

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1013. Partition Array Into Three Parts With Equal Sum
 * https://leetcode.com/problems/partition-array-into-three-parts-with-equal-sum/
 * Difficulty: Easy
 *
 * Given an array of integers arr, return true if we can partition the array into three
 * non-empty parts with equal sums.
 *
 * Formally, we can partition the array if we can find indexes i + 1 < j with (arr[0] + arr[1]
 * + ... + arr[i] == arr[i + 1] + arr[i + 2] + ... + arr[j - 1] == arr[j] + arr[j + 1]
 * + ... + arr[arr.length - 1])
 */

/**
 * @param {number[]} arr
 * @return {boolean}
```

```
 */
var canThreePartsEqualSum = function(arr) {
  const totalSum = arr.reduce((sum, num) => sum + num, 0);
  if (totalSum % 3 !== 0) return false;

  const targetSum = totalSum / 3;
  let partsFound = 0;
  let currentSum = 0;

  for (const num of arr) {
    currentSum += num;
    if (currentSum === targetSum) {
      partsFound++;
      currentSum = 0;
    }
  }

  return partsFound >= 3;
};
```

1014 Best Sightseeing Pair

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1014. Best Sightseeing Pair
 * https://leetcode.com/problems/best-sightseeing-pair/
 * Difficulty: Medium
 *
 * You are given an integer array values where values[i] represents the value of the ith
sightseeing
 * spot. Two sightseeing spots i and j have a distance j - i between them.
 *
 * The score of a pair (i < j) of sightseeing spots is values[i] + values[j] + i - j: the sum
of the
 * values of the sightseeing spots, minus the distance between them.
 *
 * Return the maximum score of a pair of sightseeing spots.
 */

/**
 * @param {number[]} values
 * @return {number}
 */
var maxScoreSightseeingPair = function(values) {
  let maxLeft = values[0];
  let result = 0;

  for (let j = 1; j < values.length; j++) {
    result = Math.max(result, maxLeft + values[j] - j);
    maxLeft = Math.max(maxLeft, values[j] + j);
  }

  return result;
};
```

1015 Smallest Integer Divisible By K

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1015. Smallest Integer Divisible by K
 * https://leetcode.com/problems/smallest-integer-divisible-by-k/
 * Difficulty: Medium
 *
```

```
 * Given a positive integer k, you need to find the length of the smallest positive integer n
such
 * that n is divisible by k, and n only contains the digit 1.
 *
 * Return the length of n. If there is no such n, return -1.
 *
 * Note: n may not fit in a 64-bit signed integer.
 */

/**
 * @param {number} k
 * @return {number}
 */
var smallestRepunitDivByK = function(k) {
  if (k % 2 === 0 || k % 5 === 0) return -1;
  let remainder = 0;
  let length = 1;

  while (length <= k) {
    remainder = (remainder * 10 + 1) % k;
    if (remainder === 0) return length;
    length++;
  }

  return -1;
};
```

1016 Binary String With Substrings Representing 1 To N

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1016. Binary String With Substrings Representing 1 To N
 * https://leetcode.com/problems/binary-string-with-substrings-representing-1-to-n/
 * Difficulty: Medium
 *
 * Given a binary string s and a positive integer n, return true if the binary representation
of
 * all the integers in the range [1, n] are substrings of s, or false otherwise.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {number} n
 * @return {boolean}
 */
var queryString = function(binaryString, maxNumber) {
  return hasAllSubstrings(binaryString, maxNumber);

  function hasAllSubstrings(str, num) {
    for (let i = 1; i <= num; i++) {
      if (!str.includes(i.toString(2))) return false;
    }
    return true;
  }
};
```

1017 Convert To Base 2

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1017. Convert to Base -2
```

```
 * https://leetcode.com/problems/convert-to-base-2/
 * Difficulty: Medium
 *
 * Given an integer n, return a binary string representing its representation in base -2.
 *
 * Note that the returned string should not have leading zeros unless the string is "0".
 */

/**
 * @param {number} n
 * @return {string}
 */
var baseNeg2 = function(number) {
  if (number === 0) return '0';

  let result = '';
  let current = number;

  while (current !== 0) {
    const remainder = current & 1;
    result = remainder + result;
    current = (current - remainder) / -2;
  }

  return result;
};
```

1018 Binary Prefix Divisible By 5

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1018. Binary Prefix Divisible By 5
 * https://leetcode.com/problems/binary-prefix-divisible-by-5/
 * Difficulty: Easy
 *
 * You are given a binary array nums (0-indexed).
 *
 * We define xi as the number whose binary representation is the subarray nums[0..i]
 * (from most-significant-bit to least-significant-bit).
 *
 * For example, if nums = [1,0,1], then x0 = 1, x1 = 2, and x2 = 5.
 *
 * Return an array of booleans answer where answer[i] is true if xi is divisible by 5.
 */

/**
 * @param {number[]} nums
 * @return {boolean[]}
 */
var prefixesDivBy5 = function(nums) {
  const result = new Array(nums.length);

  for (let i = 0, current = 0; i < nums.length; i++) {
    current = (current * 2 + nums[i]) % 5;
    result[i] = current === 0;
  }

  return result;
};
```

1019 Next Greater Node In Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1019. Next Greater Node In Linked List
 * https://leetcode.com/problems/next-greater-node-in-linked-list/
 * Difficulty: Medium
 *
 * You are given the head of a linked list with n nodes.
 *
 * For each node in the list, find the value of the next greater node. That is, for each node,
 * find the value of the first node that is next to it and has a strictly larger value than
it.
 *
 * Return an integer array answer where answer[i] is the value of the next greater node of the
 * ith node (1-indexed). If the ith node does not have a next greater node, set answer[i] = 0.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {number[]}
 */
var nextLargerNodes = function(head) {
  const values = [];
  let current = head;

  while (current) {
    values.push(current.val);
    current = current.next;
  }

  const result = new Array(values.length).fill(0);
  const stack = [];

  for (let i = 0; i < values.length; i++) {
    while (stack.length && values[stack[stack.length - 1]] < values[i]) {
      result[stack.pop()] = values[i];
    }
    stack.push(i);
  }

  return result;
};
```

1020 Number Of Enclaves

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1020. Number of Enclaves
 * https://leetcode.com/problems/number-of-enclaves/
 * Difficulty: Medium
 *
 * You are given an m x n binary matrix grid, where 0 represents a sea cell and 1 represents
 * a land cell.
 *
 * A move consists of walking from one land cell to another adjacent (4-directionally) land
cell
 * or walking off the boundary of the grid.
 *
 * Return the number of land cells in grid for which we cannot walk off the boundary of the
grid
 * in any number of moves.
 */

/**
```

```
 * @param {number[][]} grid
 * @return {number}
 */
var numEnclaves = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;

  for (let i = 0; i < rows; i++) {
    exploreBoundary(i, 0);
    exploreBoundary(i, cols - 1);
  }

  for (let j = 0; j < cols; j++) {
    exploreBoundary(0, j);
    exploreBoundary(rows - 1, j);
  }

  let result = 0;
  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      result += grid[i][j];
    }
  }

  return result;

  function exploreBoundary(row, col) {
    if (row < 0 || row >= rows || col < 0 || col >= cols || grid[row][col] !== 1) return;
    grid[row][col] = 0;
    exploreBoundary(row + 1, col);
    exploreBoundary(row - 1, col);
    exploreBoundary(row, col + 1);
    exploreBoundary(row, col - 1);
  }
};
```

1021 Remove Outermost Parentheses

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1021. Remove Outermost Parentheses
 * https://leetcode.com/problems/remove-outermost-parentheses/
 * Difficulty: Easy
 *
 * A valid parentheses string is either empty "", "(" + A + ")", or A + B, where A and B are
valid
 * parentheses strings, and + represents string concatenation.
 * - For example, "", "()", "(())()", and "(()(()))" are all valid parentheses strings.
 *
 * A valid parentheses string s is primitive if it is nonempty, and there does not exist a way
 to
 * split it into s = A + B, with A and B nonempty valid parentheses strings.
 *
 * Given a valid parentheses string s, consider its primitive decomposition: s = P1
 * + P2 + ... + Pk, where Pi are primitive valid parentheses strings.
 *
 * Return s after removing the outermost parentheses of every primitive string in the
primitive
 * decomposition of s.
 */

/**
 * @param {string} s
 * @return {string}
 */
var removeOuterParentheses = function(s) {
  let result = '';
  let depth = 0;
```

```
  for (const char of s) {
    if (char === '(') {
      if (depth > 0) result += char;
      depth++;
    } else {
      depth--;
      if (depth > 0) result += char;
    }
  }

  return result;
};
```

1022 Sum Of Root To Leaf Binary Numbers

EASY

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1022. Sum of Root To Leaf Binary Numbers
 * https://leetcode.com/problems/sum-of-root-to-leaf-binary-numbers/
 * Difficulty: Easy
 *
 * You are given the root of a binary tree where each node has a value 0 or 1. Each
root-to-leaf
 * path represents a binary number starting with the most significant bit.
 *
 * For example, if the path is 0 -> 1 -> 1 -> 0 -> 1, then this could represent 01101 in
binary,
 * which is 13.
 *
 * For all leaves in the tree, consider the numbers represented by the path from the root to
that
 * leaf. Return the sum of these numbers.
 *
 * The test cases are generated so that the answer fits in a 32-bits integer.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var sumRootToLeaf = function(root) {
  return dfs(root);
};

function dfs(node, str = '') {
  if (!node) return 0;
  str += node.val;
  if (!node.left && !node.right) {
    return parseInt(str, 2)
  }
  return dfs(node.left, str) + dfs(node.right, str);
}
```

1023 Camelcase Matching

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1023. Camelcase Matching
 * https://leetcode.com/problems/camelcase-matching/
 * Difficulty: Medium
 *
 * Given an array of strings queries and a string pattern, return a boolean array answer
 * where answer[i] is true if queries[i] matches pattern, and false otherwise.
 *
 * A query word queries[i] matches pattern if you can insert lowercase English letters
 * pattern so that it equals the query. You may insert each character at any position
 * and you may not insert any characters.
 */

/**
 * @param {string[]} queries
 * @param {string} pattern
 * @return {boolean[]}
 */
var camelMatch = function(queries, pattern) {
  return queries.map(s => s.match(pattern.split(/(.)/).join('[a-z]*'))?.at(0) === s);
};
```

1024 Video Stitching

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1024. Video Stitching
 * https://leetcode.com/problems/video-stitching/
 * Difficulty: Medium
 *
 * You are given a series of video clips from a sporting event that lasted time seconds.
 * These video clips can be overlapping with each other and have varying lengths.
 *
 * Each video clip is described by an array clips where clips[i] = [starti, endi] indicates
 * that the ith clip started at starti and ended at endi.
 *
 * We can cut these clips into segments freely.
 * - For example, a clip [0, 7] can be cut into segments [0, 1] + [1, 3] + [3, 7].
 *
 * Return the minimum number of clips needed so that we can cut the clips into segments that
 * cover the entire sporting event [0, time]. If the task is impossible, return -1.
 */

/**
 * @param {number[][]} clips
 * @param {number} time
 * @return {number}
 */
var videoStitching = function(clips, time) {
  let count = 0;
  let currentEnd = 0;
  let nextEnd = 0;

  clips.sort((a, b) => a[0] - b[0]);

  for (let i = 0; i < clips.length && currentEnd < time;) {
    count++;
    while (i < clips.length && clips[i][0] <= currentEnd) {
      nextEnd = Math.max(nextEnd, clips[i][1]);
      i++;
    }
    if (nextEnd === currentEnd) return -1;
    currentEnd = nextEnd;
  }

  return currentEnd >= time ? count : -1;
```

```
};
```

1025 Divisor Game

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1025. Divisor Game
 * https://leetcode.com/problems/divisor-game/
 * Difficulty: Easy
 *
 * Alice and Bob take turns playing a game, with Alice starting first.
 *
 * Initially, there is a number n on the chalkboard. On each player's turn, that player makes
a
 * move consisting of:
 * - Choosing any x with 0 < x < n and n % x == 0.
 * - Replacing the number n on the chalkboard with n - x.
 *
 * Also, if a player cannot make a move, they lose the game.
 *
 * Return true if and only if Alice wins the game, assuming both players play optimally.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var divisorGame = function(n) {
  return n % 2 === 0;
};
```

1026 Maximum Difference Between Node And Ancestor

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1026. Maximum Difference Between Node and Ancestor
 * https://leetcode.com/problems/maximum-difference-between-node-and-ancestor/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, find the maximum value v for which there exist different
nodes
 * a and b where v = |a.val - b.val| and a is an ancestor of b.
 *
 * A node a is an ancestor of b if either: any child of a is equal to b or any child of a is
an
 * ancestor of b.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxAncestorDiff = function(root) {
  return traverse(root, root.val, root.val);
```

```
  function traverse(node, maxVal, minVal) {
    if (!node) return maxVal - minVal;

    maxVal = Math.max(maxVal, node.val);
    minVal = Math.min(minVal, node.val);

    return Math.max(
      traverse(node.left, maxVal, minVal),
      traverse(node.right, maxVal, minVal)
    );
  }
};
```

1027 Longest Arithmetic Subsequence

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1027. Longest Arithmetic Subsequence
 * https://leetcode.com/problems/longest-arithmetic-subsequence/
 * Difficulty: Medium
 *
 * Given an array nums of integers, return the length of the longest arithmetic
 * subsequence in nums.
 *
 * Note that:
 * - A subsequence is an array that can be derived from another array by deleting some
 *   or no elements without changing the order of the remaining elements.
 * - A sequence seq is arithmetic if seq[i + 1] - seq[i] are all the same value
 *   (for 0 <= i < seq.length - 1).
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var longestArithSeqLength = function(nums) {
  const dp = new Array(nums.length).fill().map(() => new Map());
  let result = 2;

  for (let i = 1; i < nums.length; i++) {
    for (let j = 0; j < i; j++) {
      const diff = nums[i] - nums[j];
      const prevLength = dp[j].get(diff) || 1;
      dp[i].set(diff, prevLength + 1);
      result = Math.max(result, prevLength + 1);
    }
  }

  return result;
};
```

1028 Recover A Tree From Preorder Traversal

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1028. Recover a Tree From Preorder Traversal
 * https://leetcode.com/problems/recover-a-tree-from-preorder-traversal/
 * Difficulty: Hard
 *
 * We run a preorder depth-first search (DFS) on the root of a binary tree.
 *
 * At each node in this traversal, we output D dashes (where D is the depth of this node),
 * then we output the value of this node.  If the depth of a node is D, the depth of its
```

```
 * immediate child is D + 1.  The depth of the root node is 0.
 *
 * If a node has only one child, that child is guaranteed to be the left child.
 *
 * Given the output traversal of this traversal, recover the tree and return its root.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {string} traversal
 * @return {TreeNode}
 */
var recoverFromPreorder = function(traversal) {
  const stack = [];

  for (let i = 0; i < traversal.length;) {
    let depth = 0;
    while (traversal[i] === '-') {
      depth++;
      i++;
    }

    let value = 0;
    while (i < traversal.length && traversal[i] !== '-') {
      value = value * 10 + parseInt(traversal[i]);
      i++;
    }

    const node = new TreeNode(value);
    while (stack.length > depth) stack.pop();
    if (stack.length) {
      if (!stack[stack.length - 1].left) stack[stack.length - 1].left = node;
      else stack[stack.length - 1].right = node;
    }
    stack.push(node);
  }

  return stack[0];
};
```

1029 Two City Scheduling

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1029. Two City Scheduling
 * https://leetcode.com/problems/two-city-scheduling/
 * Difficulty: Medium
 *
 * A company is planning to interview 2n people. Given the array costs where
 * costs[i] = [aCosti, bCosti], the cost of flying the ith person to city a is aCosti,
 * and the cost of flying the ith person to city b is bCosti.
 *
 * Return the minimum cost to fly every person to a city such that exactly n people
 * arrive in each city.
 */

/**
 * @param {number[][]} costs
 * @return {number}
 */
/**
 * @param {number[][]} costs
```

```
 * @return {number}
 */
var twoCitySchedCost = function(costs) {
  return costs
    .sort((a, b) => (a[0] - a[1]) - (b[0] - b[1]))
    .reduce((sum, [a, b], index) => sum + (index < costs.length / 2 ? a : b), 0);
};
```

1030 Matrix Cells In Distance Order

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1030. Matrix Cells in Distance Order
 * https://leetcode.com/problems/matrix-cells-in-distance-order/
 * Difficulty: Easy
 *
 * You are given four integers row, cols, rCenter, and cCenter. There is a rows x cols matrix
and
 * you are on the cell with the coordinates (rCenter, cCenter).
 *
 * Return the coordinates of all cells in the matrix, sorted by their distance from (rCenter,
 * cCenter) from the smallest distance to the largest distance. You may return the answer in
any
 * order that satisfies this condition.
 *
 * The distance between two cells (r1, c1) and (r2, c2) is |r1 - r2| + |c1 - c2|.
 */

/**
 * @param {number} rows
 * @param {number} cols
 * @param {number} rCenter
 * @param {number} cCenter
 * @return {number[][]}
 */
var allCellsDistOrder = function(rows, cols, rCenter, cCenter) {
  const result = [];
  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      result.push([row, col]);
    }
  }
  return result.sort((a, b) => {
    return Math.abs(a[0] - rCenter) + Math.abs(a[1] - cCenter)
      - Math.abs(b[0] - rCenter) - Math.abs(b[1] - cCenter);
  });
};
```

1031 Maximum Sum Of Two Non Overlapping Subarrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1031. Maximum Sum of Two Non-Overlapping Subarrays
 * https://leetcode.com/problems/maximum-sum-of-two-non-overlapping-subarrays/
 * Difficulty: Medium
 *
 * Given an integer array nums and two integers firstLen and secondLen, return the maximum sum
 * of elements in two non-overlapping subarrays with lengths firstLen and secondLen.
 *
 * The array with length firstLen could occur before or after the array with length secondLen,
 * but they have to be non-overlapping.
 *
 * A subarray is a contiguous part of an array.
```

```
 */

/**
 * @param {number[]} nums
 * @param {number} firstLen
 * @param {number} secondLen
 * @return {number}
 */
var maxSumTwoNoOverlap = function(nums, firstLen, secondLen) {
  const prefixSums = getPrefixSums(nums);
  let maxFirst = 0;
  let maxSecond = 0;
  let result = 0;

  for (let i = firstLen; i <= nums.length - secondLen; i++) {
    maxFirst = Math.max(maxFirst, prefixSums[i] - prefixSums[i - firstLen]);
    const secondSum = prefixSums[i + secondLen] - prefixSums[i];
    result = Math.max(result, maxFirst + secondSum);
  }

  for (let i = secondLen; i <= nums.length - firstLen; i++) {
    maxSecond = Math.max(maxSecond, prefixSums[i] - prefixSums[i - secondLen]);
    const firstSum = prefixSums[i + firstLen] - prefixSums[i];
    result = Math.max(result, maxSecond + firstSum);
  }

  return result;

  function getPrefixSums(arr) {
    const sums = [0];
    for (let i = 0; i < arr.length; i++) {
      sums.push(sums[i] + arr[i]);
    }
    return sums;
  }
};
```

1032 Stream Of Characters

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1032. Stream of Characters
 * https://leetcode.com/problems/stream-of-characters/
 * Difficulty: Hard
 *
 * Design an algorithm that accepts a stream of characters and checks if a suffix of these
 * characters is a string of a given array of strings words.
 *
 * For example, if words = ["abc", "xyz"] and the stream added the four characters (one by
one)
 * 'a', 'x', 'y', and 'z', your algorithm should detect that the suffix "xyz" of the
characters
 * "axyz" matches "xyz" from words.
 *
 * Implement the StreamChecker class:
 * - StreamChecker(String[] words) Initializes the object with the strings array words.
 * - boolean query(char letter) Accepts a new character from the stream and returns true if
any
 *   non-empty suffix from the stream forms a word that is in words.
 */

/**
 * @param {string[]} words
 */
var StreamChecker = function(words) {
  this.trie = {};
  this.stream = [];

  words.forEach(word => buildTrie(word, this.trie));
```

```
  function buildTrie(word, trie) {
    let node = trie;
    for (let i = word.length - 1; i >= 0; i--) {
      const char = word[i];
      node[char] = node[char] || {};
      node = node[char];
    }
    node.isEnd = true;
  }
};

/**
 * @param {character} letter
 * @return {boolean}
 */
StreamChecker.prototype.query = function(letter) {
  this.stream.push(letter);
  let node = this.trie;

  for (let i = this.stream.length - 1; i >= 0; i--) {
    const char = this.stream[i];
    if (!node[char]) return false;
    node = node[char];
    if (node.isEnd) return true;
  }

  return false;
};
```

1033 Moving Stones Until Consecutive

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1033. Moving Stones Until Consecutive
 * https://leetcode.com/problems/moving-stones-until-consecutive/
 * Difficulty: Medium
 *
 * There are three stones in different positions on the X-axis. You are given three integers
a, b,
 * and c, the positions of the stones.
 *
 * In one move, you pick up a stone at an endpoint (i.e., either the lowest or highest
position
 * stone), and move it to an unoccupied position between those endpoints. Formally, let's say
the
 * stones are currently at positions x, y, and z with x < y < z. You pick up the stone at
either
 * position x or position z, and move that stone to an integer position k, with x < k < z and
 * k != y.
 *
 * The game ends when you cannot make any more moves (i.e., the stones are in three
consecutive
 * positions).
 *
 * Return an integer array answer of length 2 where:
 * - answer[0] is the minimum number of moves you can play, and
 * - answer[1] is the maximum number of moves you can play.
 */

/**
 * @param {number} a
 * @param {number} b
 * @param {number} c
 * @return {number[]}
 */
var numMovesStones = function(a, b, c) {
  const positions = [a, b, c].sort((x, y) => x - y);
  const minGap = Math.min(positions[1] - positions[0], positions[2] - positions[1]);
```

```
  const totalGap = positions[2] - positions[0] - 2;

  const minMoves = positions[1] - positions[0] === 1 && positions[2] - positions[1] === 1
    ? 0 : minGap <= 2 ? 1 : 2;
  const maxMoves = totalGap;

  return [minMoves, maxMoves];
};
```

1034 Coloring A Border

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1034. Coloring A Border
 * https://leetcode.com/problems/coloring-a-border/
 * Difficulty: Medium
 *
 * You are given an m x n integer matrix grid, and three integers row, col, and color. Each
value
 * in the grid represents the color of the grid square at that location.
 *
 * Two squares are called adjacent if they are next to each other in any of the 4 directions.
 *
 * Two squares belong to the same connected component if they have the same color and they are
 * adjacent.
 *
 * The border of a connected component is all the squares in the connected component that are
 * either adjacent to (at least) a square not in the component, or on the boundary of the grid
 * (the first or last row or column).
 *
 * You should color the border of the connected component that contains the square
grid[row][col]
 * with color.
 *
 * Return the final grid.
 */

/**
 * @param {number[][]} grid
 * @param {number} row
 * @param {number} col
 * @param {number} color
 * @return {number[][]}
 */
var colorBorder = function(grid, row, col, color) {
  const rows = grid.length;
  const cols = grid[0].length;
  const visited = new Set();
  const originalColor = grid[row][col];
  const borders = new Set();

  function findBorders(r, c) {
    if (r < 0 || r >= rows || c < 0 || c >= cols
        || visited.has(`${r},${c}`) || grid[r][c] !== originalColor) {
      return;
    }

    visited.add(`${r},${c}`);

    if (r === 0 || r === rows - 1 || c === 0 || c === cols - 1
        || (r > 0 && grid[r - 1][c] !== originalColor)
        || (r < rows - 1 && grid[r + 1][c] !== originalColor)
        || (c > 0 && grid[r][c - 1] !== originalColor)
        || (c < cols - 1 && grid[r][c + 1] !== originalColor)) {
      borders.add(`${r},${c}`);
    }

    findBorders(r - 1, c);
    findBorders(r + 1, c);
```

```
    findBorders(r, c - 1);
    findBorders(r, c + 1);
  }

  findBorders(row, col);
  borders.forEach(pos => {
    const [r, c] = pos.split(',').map(Number);
    grid[r][c] = color;
  });

  return grid;
};
```

1035 Uncrossed Lines

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1035. Uncrossed Lines
 * https://leetcode.com/problems/uncrossed-lines/
 * Difficulty: Medium
 *
 * You are given two integer arrays nums1 and nums2. We write the integers of nums1 and nums2
 * (in the order they are given) on two separate horizontal lines.
 *
 * We may draw connecting lines: a straight line connecting two numbers nums1[i] and nums2[j]
 * such that:
 * - nums1[i] == nums2[j], and
 * - the line we draw does not intersect any other connecting (non-horizontal) line.
 *
 * Note that a connecting line cannot intersect even at the endpoints (i.e., each number can
only
 * belong to one connecting line).
 *
 * Return the maximum number of connecting lines we can draw in this way.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var maxUncrossedLines = function(nums1, nums2) {
  const m = nums1.length;
  const n = nums2.length;

  const dp = new Array(m + 1).fill().map(() => new Array(n + 1).fill(0));

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (nums1[i - 1] === nums2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }

  return dp[m][n];
};
```

1036 Escape A Large Maze

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1036. Escape a Large Maze
 * https://leetcode.com/problems/escape-a-large-maze/
 * Difficulty: Hard
 *
 * There is a 1 million by 1 million grid on an XY-plane, and the coordinates of each grid
 * square are (x, y).
 *
 * We start at the source = [sx, sy] square and want to reach the target = [tx, ty] square.
 * There is also an array of blocked squares, where each blocked[i] = [xi, yi] represents
 * a blocked square with coordinates (xi, yi).
 *
 * Each move, we can walk one square north, east, south, or west if the square is not in
 * the array of blocked squares. We are also not allowed to walk outside of the grid.
 *
 * Return true if and only if it is possible to reach the target square from the source
 * square through a sequence of valid moves.
 */

/**
 * @param {number[][]} blocked
 * @param {number[]} source
 * @param {number[]} target
 * @return {boolean}
 */
var isEscapePossible = function(blocked, source, target) {
  const MAX = 1e6;
  const LIMIT = 19900;
  const blockedSet = new Set(blocked.map(([x, y]) => `${x},${y}`));

  if (!helper(source, target)) {
    return false;
  }

  return helper(target, source);

  function helper(start, goal) {
    const queue = [start];
    const visited = new Set([`${start[0]},${start[1]}`]);
    const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

    while (queue.length && visited.size <= LIMIT) {
      const [x, y] = queue.shift();

      for (const [dx, dy] of directions) {
        const newX = x + dx;
        const newY = y + dy;
        const key = `${newX},${newY}`;

        if (newX < 0 || newX >= MAX || newY < 0 || newY >= MAX
            || blockedSet.has(key) || visited.has(key)) {
          continue;
        }

        if (newX === goal[0] && newY === goal[1]) {
          return true;
        }

        queue.push([newX, newY]);
        visited.add(key);
      }
    }

    return visited.size > LIMIT;
  }
};
```

1037 Valid Boomerang

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1037. Valid Boomerang
 * https://leetcode.com/problems/valid-boomerang/
 * Difficulty: Easy
 *
 * A boomerang is a set of 3 points that are all distinct and not in a straight line.
 *
 * Given a list of three points in the plane, return whether these points are a boomerang.
 */

/**
 * @param {number[][]} points
 * @return {boolean}
 */
var isBoomerang = function(points) {
  return (points[0][1] - points[1][1]) * (points[1][0] - points[2][0])
    !== (points[0][0] - points[1][0]) * (points[1][1] - points[2][1]);
};
```

1038 Binary Search Tree To Greater Sum Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1038. Binary Search Tree to Greater Sum Tree
 * https://leetcode.com/problems/binary-search-tree-to-greater-sum-tree/
 * Difficulty: Medium
 *
 * Given the root of a Binary Search Tree (BST), convert it to a Greater Tree such that every
 * key of the original BST is changed to the original key plus the sum of all keys greater
 * than the original key in BST.
 *
 * As a reminder, a binary search tree is a tree that satisfies these constraints:
 * - The left subtree of a node contains only nodes with keys less than the node's key.
 * - The right subtree of a node contains only nodes with keys greater than the node's key.
 * - Both the left and right subtrees must also be binary search trees.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var bstToGst = function(root) {
  let sum = 0;
  traverse(root);
  return root;

  function traverse(node) {
    if (!node) return;
    traverse(node.right);
    sum += node.val;
    node.val = sum;
    traverse(node.left);
  }
};
```

1039 Minimum Score Triangulation Of Polygon

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1039. Minimum Score Triangulation of Polygon
 * https://leetcode.com/problems/minimum-score-triangulation-of-polygon/
 * Difficulty: Medium
 *
 * You have a convex n-sided polygon where each vertex has an integer value. You are given an
 * integer array values where values[i] is the value of the ith vertex in clockwise order.
 *
 * Polygon triangulation is a process where you divide a polygon into a set of triangles and
 * the vertices of each triangle must also be vertices of the original polygon. Note that no
 * other shapes other than triangles are allowed in the division. This process will result
 * in n - 2 triangles.
 *
 * You will triangulate the polygon. For each triangle, the weight of that triangle is the
 * product of the values at its vertices. The total score of the triangulation is the sum of
 * these weights over all n - 2 triangles.
 *
 * Return the minimum possible score that you can achieve with some triangulation of the
polygon.
 */

/**
 * @param {number[]} values
 * @return {number}
 */
var minScoreTriangulation = function(values) {
  const n = values.length;
  const dp = new Array(n).fill().map(() => new Array(n).fill(0));

  for (let len = 2; len < n; len++) {
    for (let start = 0; start + len < n; start++) {
      const end = start + len;
      dp[start][end] = Infinity;

      for (let mid = start + 1; mid < end; mid++) {
        const score = dp[start][mid] + dp[mid][end] + values[start] * values[mid] *
values[end];
        dp[start][end] = Math.min(dp[start][end], score);
      }
    }
  }

  return dp[0][n - 1];
};
```

1040 Moving Stones Until Consecutive Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1040. Moving Stones Until Consecutive II
 * https://leetcode.com/problems/moving-stones-until-consecutive-ii/
 * Difficulty: Medium
 *
 * There are some stones in different positions on the X-axis. You are given an integer array
 * stones, the positions of the stones.
 *
 * Call a stone an endpoint stone if it has the smallest or largest position. In one move, you
 * pick up an endpoint stone and move it to an unoccupied position so that it is no longer an
```

```
 * endpoint stone.
 * - In particular, if the stones are at say, stones = [1,2,5], you cannot move the endpoint
 *   stone at position 5, since moving it to any position (such as 0, or 3) will still keep
that
 *   stone as an endpoint stone.
 *
 * The game ends when you cannot make any more moves (i.e., the stones are in three
consecutive
 * positions).
 *
 * Return an integer array answer of length 2 where:
 * - answer[0] is the minimum number of moves you can play, and
 * - answer[1] is the maximum number of moves you can play.
 */

/**
 * @param {number[]} stones
 * @return {number[]}
 */
var numMovesStonesII = function(stones) {
  stones.sort((a, b) => a - b);
  const n = stones.length;
  const maxMoves = Math.max(stones[n - 1] - stones[1] - n + 2, stones[n - 2] - stones[0] - n +
 2);
  let minMoves = n;

  let left = 0;
  for (let right = 0; right < n; right++) {
    while (stones[right] - stones[left] + 1 > n) {
      left++;
    }
    const windowSize = right - left + 1;
    if (windowSize === n - 1 && stones[right] - stones[left] + 1 === n - 1) {
      minMoves = Math.min(minMoves, 2);
    } else {
      minMoves = Math.min(minMoves, n - windowSize);
    }
  }

  return [minMoves, maxMoves];
};
```

1041 Robot Bounded In Circle

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1041. Robot Bounded In Circle
 * https://leetcode.com/problems/robot-bounded-in-circle/
 * Difficulty: Medium
 *
 * On an infinite plane, a robot initially stands at (0, 0) and faces north.
 * The robot can receive one of three instructions:
 * - "G": go straight 1 unit;
 * - "L": turn 90 degrees to the left;
 * - "R": turn 90 degrees to the right.
 *
 * The robot performs the instructions given in order, and repeats them forever.
 * Return true if and only if there exists a circle in the plane such that the
 * robot never leaves the circle.
 */

/**
 * @param {string} instructions
 * @return {boolean}
 */
var isRobotBounded = function(instructions) {
  let x = 0, y = 0;
  let dx = 0, dy = 1;
```

```
  for (const direction of instructions) {
    switch (direction) {
      case 'R': [dx, dy] = [dy, -dx]; break;
      case 'L': [dy, dx] = [dx, -dy]; break;
      case 'G': [x, y] = [x + dx, y + dy]; break;
    }
  }

  return (x === 0 && y === 0) || dy !== 1;
}
```

1042 Flower Planting With No Adjacent

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1042. Flower Planting With No Adjacent
 * https://leetcode.com/problems/flower-planting-with-no-adjacent/
 * Difficulty: Medium
 *
 * You have n gardens, labeled from 1 to n, and an array paths where paths[i] = [xi, yi]
describes
 * a bidirectional path between garden xi to garden yi. In each garden, you want to plant one
of
 * 4 types of flowers.
 *
 * All gardens have at most 3 paths coming into or leaving it.
 *
 * Your task is to choose a flower type for each garden such that, for any two gardens
connected
 * by a path, they have different types of flowers.
 *
 * Return any such a choice as an array answer, where answer[i] is the type of flower planted
in
 * the (i+1)th garden. The flower types are denoted 1, 2, 3, or 4. It is guaranteed an answer
 * exists.
 */

/**
 * @param {number} n
 * @param {number[][]} paths
 * @return {number[]}
 */
var gardenNoAdj = function(n, paths) {
  const graph = Array.from({ length: n }, () => new Set());
  const result = new Array(n).fill(0);

  for (const [x, y] of paths) {
    graph[x - 1].add(y - 1);
    graph[y - 1].add(x - 1);
  }

  for (let garden = 0; garden < n; garden++) {
    const usedColors = new Set();
    for (const neighbor of graph[garden]) {
      usedColors.add(result[neighbor]);
    }

    for (let color = 1; color <= 4; color++) {
      if (!usedColors.has(color)) {
        result[garden] = color;
        break;
      }
    }
  }

  return result;
};
```

1043 Partition Array For Maximum Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1043. Partition Array for Maximum Sum
 * https://leetcode.com/problems/partition-array-for-maximum-sum/
 * Difficulty: Medium
 *
 * Given an integer array arr, partition the array into (contiguous) subarrays of length at
most k.
 * After partitioning, each subarray has their values changed to become the maximum value of
that
 * subarray.
 *
 * Return the largest sum of the given array after partitioning. Test cases are generated so
that
 * the answer fits in a 32-bit integer.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number}
 */
var maxSumAfterPartitioning = function(arr, k) {
  const dp = new Array(arr.length + 1).fill(0);

  for (let i = 1; i <= arr.length; i++) {
    let maxVal = 0;
    for (let j = 1; j <= Math.min(k, i); j++) {
      maxVal = Math.max(maxVal, arr[i - j]);
      dp[i] = Math.max(dp[i], dp[i - j] + maxVal * j);
    }
  }

  return dp[arr.length];
};
```

1044 Longest Duplicate Substring

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1044. Longest Duplicate Substring
 * https://leetcode.com/problems/longest-duplicate-substring/
 * Difficulty: Hard
 *
 * Given a string s, consider all duplicated substrings: (contiguous) substrings of s that
occur 2
 * or more times. The occurrences may overlap.
 *
 * Return any duplicated substring that has the longest possible length. If s does not have a
 * duplicated substring, the answer is "".
 */

/**
 * @param {string} s
 * @return {string}
 */
var longestDupSubstring = function(s) {
  const MOD = 1e9 + 7;
  const BASE = 26;
```

```
  let left = 1;
  let right = s.length - 1;
  let result = '';

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    const candidate = checkLength(mid);
    if (candidate) {
      result = candidate;
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;

  function checkLength(len) {
    let hash = 0;
    const seen = new Map();
    let basePow = 1;
    for (let i = 0; i < len - 1; i++) {
      basePow = (basePow * BASE) % MOD;
    }

    for (let i = 0; i < len; i++) {
      hash = (hash * BASE + (s.charCodeAt(i) - 97)) % MOD;
    }

    seen.set(hash, [0]);

    for (let i = 1; i <= s.length - len; i++) {
      hash = (hash - ((s.charCodeAt(i - 1) - 97) * basePow) % MOD + MOD) % MOD;
      hash = (hash * BASE + (s.charCodeAt(i + len - 1) - 97)) % MOD;
      if (seen.has(hash)) {
        const prevIndices = seen.get(hash);
        const currSubstr = s.slice(i, i + len);
        for (const prev of prevIndices) {
          if (s.slice(prev, prev + len) === currSubstr) {
            return currSubstr;
          }
        }
        prevIndices.push(i);
      } else {
        seen.set(hash, [i]);
      }
    }
    return '';
  }
};
```

1046 Last Stone Weight

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1046. Last Stone Weight
 * https://leetcode.com/problems/last-stone-weight/
 * Difficulty: Easy
 *
 * You are given an array of integers stones where stones[i] is the weight of the ith stone.
 *
 * We are playing a game with the stones. On each turn, we choose the heaviest two stones and
 * smash them together. Suppose the heaviest two stones have weights x and y with x <= y. The
 * result of this smash is:
 * - If x == y, both stones are destroyed, and
 * - If x != y, the stone of weight x is destroyed, and the stone of weight y has new weight y
 - x.
 *
 * At the end of the game, there is at most one stone left.
```

```
 *
 * Return the weight of the last remaining stone. If there are no stones left, return 0.
 */

/**
 * @param {number[]} stones
 * @return {number}
 */
var lastStoneWeight = function(stones) {
  const heap = stones.sort((a, b) => b - a);

  while (heap.length > 1) {
    const heaviest = heap.shift();
    const secondHeaviest = heap.shift();
    if (heaviest !== secondHeaviest) {
      heap.push(heaviest - secondHeaviest);
      heap.sort((a, b) => b - a);
    }
  }

  return heap.length ? heap[0] : 0;
};
```

1047 Remove All Adjacent Duplicates In String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1047. Remove All Adjacent Duplicates In String
 * https://leetcode.com/problems/remove-all-adjacent-duplicates-in-string/
 * Difficulty: Easy
 *
 * You are given a string s consisting of lowercase English letters. A duplicate removal
 * consists of choosing two adjacent and equal letters and removing them.
 *
 * We repeatedly make duplicate removals on s until we no longer can.
 *
 * Return the final string after all such duplicate removals have been made. It can be
 * proven that the answer is unique.
 */

/**
 * @param {string} s
 * @return {string}
 */
var removeDuplicates = function(s) {
  const stack = [];

  for (const char of s) {
    stack[stack.length - 1] === char ? stack.pop() : stack.push(char);
  }

  return stack.join('');
};
```

1048 Longest String Chain

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1048. Longest String Chain
 * https://leetcode.com/problems/longest-string-chain/
 * Difficulty: Medium
 *
```

```
 * You are given an array of words where each word consists of lowercase English letters.
 *
 * wordA is a predecessor of wordB if and only if we can insert exactly one letter anywhere in
 * wordA without changing the order of the other characters to make it equal to wordB.
 *
 * For example, "abc" is a predecessor of "abac", while "cba" is not a predecessor of "bcad".
 *
 * A word chain is a sequence of words [word1, word2, ..., wordk] with k >= 1, where word1 is
 * a predecessor of word2, word2 is a predecessor of word3, and so on. A single word is
 * trivially a word chain with k == 1.
 *
 * Return the length of the longest possible word chain with words chosen from the given list
 * of words.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var longestStrChain = function(words) {
  const map = new Map();
  let result = 1;

  words.sort((a, b) => a.length - b.length);

  for (const word of words) {
    let currentLength = 1;
    for (let i = 0; i < word.length; i++) {
      const prev = word.slice(0, i) + word.slice(i + 1);
      if (map.has(prev)) {
        currentLength = Math.max(currentLength, map.get(prev) + 1);
      }
    }
    map.set(word, currentLength);
    result = Math.max(result, currentLength);
  }

  return result;
};
```

1049 Last Stone Weight Ii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1049. Last Stone Weight II
 * https://leetcode.com/problems/last-stone-weight-ii/
 * Difficulty: Medium
 *
 * You are given an array of integers stones where stones[i] is the weight of the ith stone.
 *
 * We are playing a game with the stones. On each turn, we choose any two stones and smash
 * them together. Suppose the stones have weights x and y with x <= y. The result of this
smash is:
 * - If x == y, both stones are destroyed, and
 * - If x != y, the stone of weight x is destroyed, and the stone of weight y has new weight y
 - x.
 *
 * At the end of the game, there is at most one stone left.
 *
 * Return the smallest possible weight of the left stone. If there are no stones left, return
0.
 */

/**
 * @param {number[]} stones
 * @return {number}
 */
var lastStoneWeightII = function(stones) {
  const totalSum = stones.reduce((sum, weight) => sum + weight, 0);
```

```
  const target = Math.floor(totalSum / 2);
  const set = new Set([0]);

  for (const stone of stones) {
    const prev = new Set(set);
    for (const sum of prev) {
      if (sum + stone <= target) {
        set.add(sum + stone);
      }
    }
  }

  return totalSum - 2 * Math.max(...set);
};
```

1051 Height Checker

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1051. Height Checker
 * https://leetcode.com/problems/height-checker/
 * Difficulty: Easy
 *
 * A school is trying to take an annual photo of all the students. The students are asked
 * to stand in a single file line in non-decreasing order by height. Let this ordering be
 * represented by the integer array expected where expected[i] is the expected height of
 * the ith student in line.
 *
 * You are given an integer array heights representing the current order that the students
 * are standing in. Each heights[i] is the height of the ith student in line (0-indexed).
 *
 * Return the number of indices where heights[i] != expected[i].
 */

/**
 * @param {number[]} heights
 * @return {number}
 */
var heightChecker = function(heights) {
  return [...heights]
    .sort((a, b) => a - b)
    .filter((expected, i) => heights[i] !== expected).length;
};
```

1052 Grumpy Bookstore Owner

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1052. Grumpy Bookstore Owner
 * https://leetcode.com/problems/grumpy-bookstore-owner/
 * Difficulty: Medium
 *
 * There is a bookstore owner that has a store open for n minutes. You are given an integer
array
 * customers of length n where customers[i] is the number of the customers that enter the
store
 * at the start of the ith minute and all those customers leave after the end of that minute.
 *
 * During certain minutes, the bookstore owner is grumpy. You are given a binary array grumpy
 * where grumpy[i] is 1 if the bookstore owner is grumpy during the ith minute, and is 0
otherwise.
 *
 * When the bookstore owner is grumpy, the customers entering during that minute are not
```

```
satisfied.
 * Otherwise, they are satisfied.
 *
 * The bookstore owner knows a secret technique to remain not grumpy for minutes consecutive
 * minutes, but this technique can only be used once.
 *
 * Return the maximum number of customers that can be satisfied throughout the day.
 */

/**
 * @param {number[]} customers
 * @param {number[]} grumpy
 * @param {number} minutes
 * @return {number}
 */
var maxSatisfied = function(customers, grumpy, minutes) {
  let baseSatisfied = 0;
  let windowGain = 0;
  let maxGain = 0;

  for (let i = 0; i < customers.length; i++) {
    if (grumpy[i] === 0) {
      baseSatisfied += customers[i];
    }
    if (i < minutes) {
      windowGain += grumpy[i] * customers[i];
    } else {
      windowGain += grumpy[i] * customers[i] - grumpy[i - minutes] * customers[i - minutes];
    }
    maxGain = Math.max(maxGain, windowGain);
  }

  return baseSatisfied + maxGain;
};
```

1053 Previous Permutation With One Swap

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1053. Previous Permutation With One Swap
 * https://leetcode.com/problems/previous-permutation-with-one-swap/
 * Difficulty: Medium
 *
 * Given an array of positive integers arr (not necessarily distinct), return the
lexicographically
 * largest permutation that is smaller than arr, that can be made with exactly one swap. If it
 * cannot be done, then return the same array.
 *
 * Note that a swap exchanges the positions of two numbers arr[i] and arr[j]
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var prevPermOpt1 = function(arr) {
  const result = [...arr];
  let swapIndex = -1;

  for (let i = arr.length - 2; i >= 0; i--) {
    if (arr[i] > arr[i + 1]) {
      swapIndex = i;
      break;
    }
  }

  if (swapIndex === -1) return arr;

  let maxLessIndex = swapIndex + 1;
```

```
  for (let j = swapIndex + 2; j < arr.length; j++) {
    if (arr[j] < arr[swapIndex] && arr[j] > arr[maxLessIndex]) {
      maxLessIndex = j;
    }
  }

  [result[swapIndex], result[maxLessIndex]] = [result[maxLessIndex], result[swapIndex]];

  return result;
};
```

1054 Distant Barcodes

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1054. Distant Barcodes
 * https://leetcode.com/problems/distant-barcodes/
 * Difficulty: Medium
 *
 * In a warehouse, there is a row of barcodes, where the ith barcode is barcodes[i].
 *
 * Rearrange the barcodes so that no two adjacent barcodes are equal. You may return any
answer,
 * and it is guaranteed an answer exists.
 */

/**
 * @param {number[]} barcodes
 * @return {number[]}
 */
var rearrangeBarcodes = function(barcodes) {
  const frequencyMap = new Map();
  for (const code of barcodes) {
    frequencyMap.set(code, (frequencyMap.get(code) || 0) + 1);
  }

  const sortedCodes = [...frequencyMap.entries()]
    .sort((a, b) => b[1] - a[1]);

  const result = new Array(barcodes.length);
  let index = 0;

  for (const [code, count] of sortedCodes) {
    for (let i = 0; i < count; i++) {
      if (index >= barcodes.length) index = 1;
      result[index] = code;
      index += 2;
    }
  }

  return result;
};
```

1055 Shortest Way To Form String

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1055. Shortest Way to Form String
 * https://leetcode.com/problems/shortest-way-to-form-string/
 * Difficulty: Medium
```

```
 *
 * A subsequence of a string is a new string that is formed from the original string by
deleting
 * some (can be none) of the characters without disturbing the relative positions of the
remaining
 * characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).
 *
 * Given two strings source and target, return the minimum number of subsequences of source
such
 * that their concatenation equals target. If the task is impossible, return -1.
 */

/**
 * @param {string} source
 * @param {string} target
 * @return {number}
 */
var shortestWay = function(source, target) {
  const sourceSet = new Set(source);

  for (const char of target) {
    if (!sourceSet.has(char)) {
      return -1;
    }
  }

  let subsequences = 0;
  let targetIndex = 0;

  while (targetIndex < target.length) {
    let sourceIndex = 0;
    const startTargetIndex = targetIndex;

    while (sourceIndex < source.length && targetIndex < target.length) {
      if (source[sourceIndex] === target[targetIndex]) {
        targetIndex++;
      }
      sourceIndex++;
    }

    if (targetIndex === startTargetIndex) {
      return -1;
    }

    subsequences++;
  }

  return subsequences;
};
```

1056 Confusing Number

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1056. Confusing Number
 * https://leetcode.com/problems/confusing-number/
 * Difficulty: Easy
 *
 * A confusing number is a number that when rotated 180 degrees becomes a different number
with
 * each digit valid.
 *
 * We can rotate digits of a number by 180 degrees to form new digits.
 * - When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6
respectively.
 * - When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.
 *
 * Note that after rotating a number, we can ignore leading zeros.
 * - For example, after rotating 8000, we have 0008 which is considered as just 8.
```

```
 *
 * Given an integer n, return true if it is a confusing number, or false otherwise.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var confusingNumber = function(n) {
  const rotationMap = { '0': '0', '1': '1', '6': '9', '8': '8', '9': '6' };
  const digits = n.toString();

  let rotated = '';
  for (let i = digits.length - 1; i >= 0; i--) {
    if (!(digits[i] in rotationMap)) {
      return false;
    }
    rotated += rotationMap[digits[i]];
  }

  return parseInt(rotated, 10) !== n;
};
```

1057 Campus Bikes

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1057. Campus Bikes
 * https://leetcode.com/problems/campus-bikes/
 * Difficulty: Medium
 *
 * On a campus represented on the X-Y plane, there are n workers and m bikes, with n <= m.
 *
 * You are given an array workers of length n where workers[i] = [xi, yi] is the position of
 * the ith worker. You are also given an array bikes of length m where bikes[j] = [xj, yj]
 * is the position of the jth bike. All the given positions are unique.
 *
 * Assign a bike to each worker. Among the available bikes and workers, we choose the
 * (workeri, bikej) pair with the shortest Manhattan distance between each other and assign
 * the bike to that worker.
 *
 * If there are multiple (workeri, bikej) pairs with the same shortest Manhattan distance,
 * we choose the pair with the smallest worker index. If there are multiple ways to do that,
 * we choose the pair with the smallest bike index. Repeat this process until there are no
 * available workers.
 *
 * Return an array answer of length n, where answer[i] is the index (0-indexed) of the bike
 * that the ith worker is assigned to.
 *
 * The Manhattan distance between two points p1 and p2 is
 * Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|.
 */

/**
 * @param {number[][]} workers
 * @param {number[][]} bikes
 * @return {number[]}
 */
var assignBikes = function(workers, bikes) {
  const distances = [];

  for (let i = 0; i < workers.length; i++) {
    for (let j = 0; j < bikes.length; j++) {
      const distance = Math.abs(
        workers[i][0] - bikes[j][0]) + Math.abs(workers[i][1] - bikes[j][1]
      );
      distances.push([distance, i, j]);
    }
```

```
  }

  distances.sort((a, b) => {
    if (a[0] !== b[0]) return a[0] - b[0];
    if (a[1] !== b[1]) return a[1] - b[1];
    return a[2] - b[2];
  });

  const result = new Array(workers.length);
  const usedBikes = new Set();
  const assignedWorkers = new Set();

  for (const [distance, workerIndex, bikeIndex] of distances) {
    if (!assignedWorkers.has(workerIndex) && !usedBikes.has(bikeIndex)) {
      result[workerIndex] = bikeIndex;
      assignedWorkers.add(workerIndex);
      usedBikes.add(bikeIndex);
    }
  }

  return result;
};
```

1058 Minimize Rounding Error To Meet Target

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1058. Minimize Rounding Error to Meet Target
 * https://leetcode.com/problems/minimize-rounding-error-to-meet-target/
 * Difficulty: Medium
 *
 * Given an array of prices [p1,p2...,pn] and a target, round each price pi to Roundi(pi) so
 * that the rounded array [Round1(p1),Round2(p2)...,Roundn(pn)] sums to the given target.
 * Each operation Roundi(pi) could be either Floor(pi) or Ceil(pi).
 *
 * Return the string "-1" if the rounded array is impossible to sum to target. Otherwise,
 * return the smallest rounding error, which is defined as Σ |Roundi(pi) - (pi)| for i from
 * 1 to n, as a string with three places after the decimal.
 */

/**
 * @param {string[]} prices
 * @param {number} target
 * @return {string}
 */
var minimizeError = function(prices, target) {
  const nums = prices.map(p => parseFloat(p));
  const floors = nums.map(n => Math.floor(n));
  const ceils = nums.map(n => Math.ceil(n));

  const minSum = floors.reduce((sum, f) => sum + f, 0);
  const maxSum = ceils.reduce((sum, c) => sum + c, 0);

  if (target < minSum || target > maxSum) {
    return '-1';
  }

  const ceilsNeeded = target - minSum;

  const items = nums.map((n, i) => ({
    floorError: n - floors[i],
    ceilError: ceils[i] - n,
    diff: (ceils[i] - n) - (n - floors[i])
  }));

  items.sort((a, b) => a.diff - b.diff);

  let totalError = 0;
```

```
  for (let i = 0; i < nums.length; i++) {
    if (i < ceilsNeeded) {
      totalError += items[i].ceilError;
    } else {
      totalError += items[i].floorError;
    }
  }

  return totalError.toFixed(3);
};
```

1059 All Paths From Source Lead To Destination

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1059. All Paths from Source Lead to Destination
 * https://leetcode.com/problems/all-paths-from-source-lead-to-destination/
 * Difficulty: Medium
 *
 * Given the edges of a directed graph where edges[i] = [ai, bi] indicates there is an edge
 * between nodes ai and bi, and two nodes source and destination of this graph, determine
 * whether or not all paths starting from source eventually, end at destination, that is:
 * - At least one path exists from the source node to the destination node
 * - If a path exists from the source node to a node with no outgoing edges, then that node
 *   is equal to destination.
 * - The number of possible paths from source to destination is a finite number.
 *
 * Return true if and only if all roads from source lead to destination.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number} source
 * @param {number} destination
 * @return {boolean}
 */
var leadsToDestination = function(n, edges, source, destination) {
  const graph = new Map();
  const states = new Array(n).fill(0);

  for (const [from, to] of edges) {
    if (!graph.has(from)) {
      graph.set(from, []);
    }
    graph.get(from).push(to);
  }

  return dfs(source);

  function dfs(node) {
    if (states[node] === 1) return false;
    if (states[node] === 2) return true;

    states[node] = 1;

    if (!graph.has(node)) {
      states[node] = 2;
      return node === destination;
    }

    for (const neighbor of graph.get(node)) {
      if (!dfs(neighbor)) {
        return false;
      }
    }

    states[node] = 2;
    return true;
```

```
  }
};
```

1060 Missing Element In Sorted Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1060. Missing Element in Sorted Array
 * https://leetcode.com/problems/missing-element-in-sorted-array/
 * Difficulty: Medium
 *
 * Given an integer array nums which is sorted in ascending order and all of its elements are
 * unique and given also an integer k, return the kth missing number starting from the
leftmost
 * number of the array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var missingElement = function(nums, k) {
  const n = nums.length;

  if (k > getMissingCount(n - 1)) {
    return nums[n - 1] + k - getMissingCount(n - 1);
  }

  let left = 0;
  let right = n - 1;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);

    if (getMissingCount(mid) < k) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return nums[left - 1] + k - getMissingCount(left - 1);

  function getMissingCount(index) {
    return nums[index] - nums[0] - index;
  }
};
```

1061 Lexicographically Smallest Equivalent String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1061. Lexicographically Smallest Equivalent String
 * https://leetcode.com/problems/lexicographically-smallest-equivalent-string/
 * Difficulty: Medium
 *
 * You are given two strings of the same length s1 and s2 and a string baseStr.
 *
 * We say s1[i] and s2[i] are equivalent characters.
 * - For example, if s1 = "abc" and s2 = "cde", then we have 'a' == 'c', 'b' == 'd', and 'c'
== 'e'.
 *
```

```
 * Equivalent characters follow the usual rules of any equivalence relation:
 * - Reflexivity: 'a' == 'a'.
 * - Symmetry: 'a' == 'b' implies 'b' == 'a'.
 * - Transitivity: 'a' == 'b' and 'b' == 'c' implies 'a' == 'c'.
 *
 * For example, given the equivalency information from s1 = "abc" and s2 = "cde", "acd" and
"aab"
 * are equivalent strings of baseStr = "eed", and "aab" is the lexicographically smallest
equivalent
 * string of baseStr.
 *
 * Return the lexicographically smallest equivalent string of baseStr by using the equivalency
 * information from s1 and s2.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @param {string} baseStr
 * @return {string}
 */
var smallestEquivalentString = function(s1, s2, baseStr) {
  const parent = Array(26).fill().map((_, i) => i);

  for (let i = 0; i < s1.length; i++) {
    union(s1.charCodeAt(i) - 97, s2.charCodeAt(i) - 97);
  }

  return baseStr.split('')
    .map(char => String.fromCharCode(97 + find(char.charCodeAt(0) - 97)))
    .join('');

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    const rootX = find(x);
    const rootY = find(y);
    if (rootX < rootY) {
      parent[rootY] = rootX;
    } else if (rootX > rootY) {
      parent[rootX] = rootY;
    }
  }
};
```

1062 Longest Repeating Substring

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1062. Longest Repeating Substring
 * https://leetcode.com/problems/longest-repeating-substring/
 * Difficulty: Medium
 *
 * Given a string s, return the length of the longest repeating substrings. If no repeating
 * substring exists, return 0.
 */

/**
 * @param {string} s
 * @return {number}
 */
var longestRepeatingSubstring = function(s) {
  const n = s.length;
  const dp = new Array(n + 1).fill(null).map(() => new Array(n + 1).fill(0));
```

```
  let result = 0;

  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= n; j++) {
      if (i !== j && s[i - 1] === s[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
        result = Math.max(result, dp[i][j]);
      }
    }
  }

  return result;
};
```

1063 Number Of Valid Subarrays

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1063. Number of Valid Subarrays
 * https://leetcode.com/problems/number-of-valid-subarrays/
 * Difficulty: Hard
 *
 * Given an integer array nums, return the number of non-empty subarrays with the leftmost
 * element of the subarray not larger than other elements in the subarray.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var validSubarrays = function(nums) {
  const n = nums.length;
  const stack = [];
  let result = 0;

  for (let i = 0; i < n; i++) {
    while (stack.length && nums[stack[stack.length - 1]] > nums[i]) {
      const index = stack.pop();
      result += i - index;
    }
    stack.push(i);
  }

  while (stack.length) {
    const index = stack.pop();
    result += n - index;
  }

  return result;
};
```

1064 Fixed Point

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1064. Fixed Point
 * https://leetcode.com/problems/fixed-point/
 * Difficulty: Easy
 *
 * Given an array of distinct integers arr, where arr is sorted in ascending order, return
 * the smallest index i that satisfies arr[i] == i. If there is no such index, return -1.
```

```
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var fixedPoint = function(arr) {
  let left = 0;
  let right = arr.length - 1;
  let result = -1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);

    if (arr[mid] === mid) {
      result = mid;
      right = mid - 1;
    } else if (arr[mid] < mid) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;
};
```

1065 Index Pairs Of A String

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1065. Index Pairs of a String
 * https://leetcode.com/problems/index-pairs-of-a-string/
 * Difficulty: Easy
 *
 * Given a string text and an array of strings words, return an array of all index
 * pairs [i, j] so that the substring text[i...j] is in words.
 *
 * Return the pairs [i, j] in sorted order (i.e., sort them by their first coordinate,
 * and in case of ties sort them by their second coordinate).
 */

/**
 * @param {string} text
 * @param {string[]} words
 * @return {number[][]}
 */
var indexPairs = function(text, words) {
  const result = [];
  const wordSet = new Set(words);

  for (let i = 0; i < text.length; i++) {
    for (let j = i; j < text.length; j++) {
      const substring = text.slice(i, j + 1);
      if (wordSet.has(substring)) {
        result.push([i, j]);
      }
    }
  }

  return result;
};
```

1066 Campus Bikes Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1066. Campus Bikes II
 * https://leetcode.com/problems/campus-bikes-ii/
 * Difficulty: Medium
 *
 * On a campus represented as a 2D grid, there are n workers and m bikes, with n <= m.
 * Each worker and bike is a 2D coordinate on this grid.
 *
 * We assign one unique bike to each worker so that the sum of the Manhattan distances
 * between each worker and their assigned bike is minimized.
 *
 * Return the minimum possible sum of Manhattan distances between each worker and their
 * assigned bike.
 *
 * The Manhattan distance between two points p1 and p2 is
 * Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|.
 */

/**
 * @param {number[][]} workers
 * @param {number[][]} bikes
 * @return {number}
 */
var assignBikes = function(workers, bikes) {
  const n = workers.length;
  const m = bikes.length;
  const memo = new Map();

  return dfs(0, 0);

  function getDistance(worker, bike) {
    return Math.abs(workers[worker][0] - bikes[bike][0])
      + Math.abs(workers[worker][1] - bikes[bike][1]);
  }

  function dfs(workerIndex, usedBikes) {
    if (workerIndex === n) return 0;

    const key = `${workerIndex}-${usedBikes}`;
    if (memo.has(key)) return memo.get(key);

    let minDistance = Infinity;

    for (let bikeIndex = 0; bikeIndex < m; bikeIndex++) {
      if ((usedBikes & (1 << bikeIndex)) === 0) {
        const distance = getDistance(workerIndex, bikeIndex)
          + dfs(workerIndex + 1, usedBikes | (1 << bikeIndex));
        minDistance = Math.min(minDistance, distance);
      }
    }

    memo.set(key, minDistance);
    return minDistance;
  }
};
```

1067 Digit Count In Range

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 1067. Digit Count in Range
 * https://leetcode.com/problems/digit-count-in-range/
 * Difficulty: Hard
 *
 * Given a single-digit integer d and two integers low and high, return the number of times
 * that d occurs as a digit in all integers in the inclusive range [low, high].
 */

/**
 * @param {number} d
 * @param {number} low
 * @param {number} high
 * @return {number}
 */
var digitsCount = function(d, low, high) {
  return countDigits(high) - countDigits(low - 1);

  function countDigits(num) {
    if (num < 0) return 0;

    const str = num.toString();
    const n = str.length;
    let count = 0;

    for (let pos = 0; pos < n; pos++) {
      const left = pos > 0 ? parseInt(str.substring(0, pos)) : 0;
      const current = parseInt(str[pos]);
      const right = pos < n - 1 ? parseInt(str.substring(pos + 1)) : 0;
      const rightLength = n - pos - 1;
      const rightPower = Math.pow(10, rightLength);

      if (d === 0) {
        if (pos === 0) {
          continue;
        }

        if (current > d) {
          count += (left - 1) * rightPower + rightPower;
        } else if (current === d) {
          count += (left - 1) * rightPower + right + 1;
        } else {
          if (left > 0) {
            count += (left - 1) * rightPower + rightPower;
          }
        }
      } else {
        if (current > d) {
          count += (left + 1) * rightPower;
        } else if (current === d) {
          count += left * rightPower + right + 1;
        } else {
          count += left * rightPower;
        }
      }
    }

    return count;
  }
};
```

1071 Greatest Common Divisor Of Strings

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1071. Greatest Common Divisor of Strings
 * https://leetcode.com/problems/greatest-common-divisor-of-strings/
 * Difficulty: Easy
 *
 * For two strings s and t, we say "t divides s" if and only if s = t + t + t + ... + t + t
```

```
 * (i.e., t is concatenated with itself one or more times).
 *
 * Given two strings str1 and str2, return the largest string x such that x divides both str1
 * and str2.
 */

/**
 * @param {string} str1
 * @param {string} str2
 * @return {string}
 */
var gcdOfStrings = function(str1, str2) {
  if (str1 + str2 !== str2 + str1) return '';

  for (let i = str2.length; i >= 0; i--) {
    if (str1.length % i === 0 && str2.length % i === 0) {
      return str1.substring(0, i);
    }
  }
};
```

1072 Flip Columns For Maximum Number Of Equal Rows

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1072. Flip Columns For Maximum Number of Equal Rows
 * https://leetcode.com/problems/flip-columns-for-maximum-number-of-equal-rows/
 * Difficulty: Medium
 *
 * You are given an m x n binary matrix matrix.
 *
 * You can choose any number of columns in the matrix and flip every cell in that column
 * (i.e., Change the value of the cell from 0 to 1 or vice versa).
 *
 * Return the maximum number of rows that have all values equal after some number of flips.
 */

/**
 * @param {number[][]} matrix
 * @return {number}
 */
var maxEqualRowsAfterFlips = function(matrix) {
  const rowPatterns = new Map();
  const rows = matrix.length;
  const cols = matrix[0].length;

  for (let i = 0; i < rows; i++) {
    let pattern = '';
    let flippedPattern = '';
    for (let j = 0; j < cols; j++) {
      pattern += matrix[i][j];
      flippedPattern += 1 - matrix[i][j];
    }
    rowPatterns.set(pattern, (rowPatterns.get(pattern) || 0) + 1);
    rowPatterns.set(flippedPattern, (rowPatterns.get(flippedPattern) || 0) + 1);
  }

  let result = 0;
  for (const count of rowPatterns.values()) {
    result = Math.max(result, count);
  }

  return result;
};
```

1073 Adding Two Negabinary Numbers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1073. Adding Two Negabinary Numbers
 * https://leetcode.com/problems/adding-two-negabinary-numbers/
 * Difficulty: Medium
 *
 * Given two numbers arr1 and arr2 in base -2, return the result of adding them together.
 *
 * Each number is given in array format:  as an array of 0s and 1s, from most significant
 * bit to least significant bit.  For example, arr = [1,1,0,1] represents the number
 * (-2)^3 + (-2)^2 + (-2)^0 = -3.  A number arr in array, format is also guaranteed to
 * have no leading zeros: either arr == [0] or arr[0] == 1.
 *
 * Return the result of adding arr1 and arr2 in the same format: as an array of 0s and 1s
 * with no leading zeros.
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
 * @return {number[]}
 */
var addNegabinary = function(arr1, arr2) {
  const result = [];
  let i = arr1.length - 1;
  let j = arr2.length - 1;
  let carry = 0;

  while (i >= 0 || j >= 0 || carry !== 0) {
    const x = i >= 0 ? arr1[i--] : 0;
    const y = j >= 0 ? arr2[j--] : 0;
    const sum = x + y + carry;

    result.unshift(sum & 1);
    carry = sum >= 2 ? -1 : sum < 0 ? 1 : 0;
  }

  while (result.length > 1 && result[0] === 0) {
    result.shift();
  }

  return result;
};
```

1074 Number Of Submatrices That Sum To Target

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1074. Number of Submatrices That Sum to Target
 * https://leetcode.com/problems/number-of-submatrices-that-sum-to-target/
 * Difficulty: Hard
 *
 * Given a matrix and a target, return the number of non-empty submatrices that sum to target.
 *
 * A submatrix x1, y1, x2, y2 is the set of all cells matrix[x][y] with x1 <= x <= x2 and
 * y1 <= y <= y2.
 *
 * Two submatrices (x1, y1, x2, y2) and (x1', y1', x2', y2') are different if they have some
 * coordinate that is different: for example, if x1 != x1'.
 */
```

```
/**
 * @param {number[][]} matrix
 * @param {number} target
 * @return {number}
 */
var numSubmatrixSumTarget = function(matrix, target) {
  const rows = matrix.length;
  const cols = matrix[0].length;
  let result = 0;

  for (let i = 0; i < rows; i++) {
    const prefixSums = new Array(cols).fill(0);
    for (let j = i; j < rows; j++) {
      const sumFreq = new Map([[0, 1]]);
      let currentSum = 0;

      for (let k = 0; k < cols; k++) {
        prefixSums[k] += matrix[j][k];
        currentSum += prefixSums[k];
        result += sumFreq.get(currentSum - target) || 0;
        sumFreq.set(currentSum, (sumFreq.get(currentSum) || 0) + 1);
      }
    }
  }

  return result;
};
```

1078 Occurrences After Bigram

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1078. Occurrences After Bigram
 * https://leetcode.com/problems/occurrences-after-bigram/
 * Difficulty: Easy
 *
 * Given two strings first and second, consider occurrences in some text of the form "first
second
 * third", where second comes immediately after first, and third comes immediately after
second.
 *
 * Return an array of all the words third for each occurrence of "first second third".
 */

/**
 * @param {string} text
 * @param {string} first
 * @param {string} second
 * @return {string[]}
 */
var findOcurrences = function(text, first, second) {
  const words = text.split(' ');
  const result = [];

  for (let i = 0; i < words.length - 2; i++) {
    if (words[i] === first && words[i + 1] === second) {
      result.push(words[i + 2]);
    }
  }

  return result;
};
```

1079 Letter Tile Possibilities

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1079. Letter Tile Possibilities
 * https://leetcode.com/problems/letter-tile-possibilities/
 * Difficulty: Medium
 *
 * You have n  tiles, where each tile has one letter tiles[i] printed on it.
 *
 * Return the number of possible non-empty sequences of letters you can make using the letters
 * printed on those tiles.
 */

/**
 * @param {string} tiles
 * @return {number}
 */
var numTilePossibilities = function(tiles) {
  const map = {};
  tiles.split('').forEach(t => map[t] = (map[t] ?? 0) + 1);
  return backtrack();

  function backtrack() {
    let count = 0;
    for (const key of Object.keys(map)) {
      if (!map[key]) continue;
      count += 1;
      map[key] -= 1;
      count += backtrack();
      map[key] += 1;
    }
    return count;
  }
};
```

1080 Insufficient Nodes In Root To Leaf Paths

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1080. Insufficient Nodes in Root to Leaf Paths
 * https://leetcode.com/problems/insufficient-nodes-in-root-to-leaf-paths/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and an integer limit, delete all insufficient nodes in the
 * tree simultaneously, and return the root of the resulting binary tree.
 *
 * A node is insufficient if every root to leaf path intersecting this node has a sum strictly
 * less than limit.
 *
 * A leaf is a node with no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
```

```
 * @param {number} limit
 * @return {TreeNode}
 */
var sufficientSubset = function(root, limit) {
  return checkPath(root, 0) ? root : null;

  function checkPath(node, sum) {
    if (!node) return false;
    if (!node.left && !node.right) return sum + node.val >= limit;

    const leftValid = checkPath(node.left, sum + node.val);
    const rightValid = checkPath(node.right, sum + node.val);

    if (!leftValid) node.left = null;
    if (!rightValid) node.right = null;

    return leftValid || rightValid;
  }
};
```

1081 Smallest Subsequence Of Distinct Characters

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1081. Smallest Subsequence of Distinct Characters
 * https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/
 * Difficulty: Medium
 *
 * Given a string s, return the lexicographically smallest subsequence
 * of s that contains all the distinct characters of s exactly once.
 */

/**
 * @param {string} s
 * @return {string}
 */
var smallestSubsequence = function(s) {
  const stack = [];

  for (let i = 0; i < s.length; i++) {
    const letter = s[i];
    if (stack.indexOf(letter) > -1) {
      continue;
    }
    while (stack.length > 0 && stack[stack.length - 1] > letter &&
s.indexOf(stack[stack.length - 1], i) > i) {
      stack.pop();
    }
    stack.push(letter);
  }

  return stack.join('');
};
```

1085 Sum Of Digits In The Minimum Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1085. Sum of Digits in the Minimum Number
 * https://leetcode.com/problems/sum-of-digits-in-the-minimum-number/
 * Difficulty: Easy
 *
 * Given an integer array nums, return 0 if the sum of the digits of the minimum
```

```
 * integer in nums is odd, or 1 otherwise.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumOfDigits = function(nums) {
  const digitSum = Math.min(...nums)
    .toString()
    .split('')
    .reduce((sum, digit) => sum + parseInt(digit), 0);

  return digitSum % 2 === 0 ? 1 : 0;
};
```

1086 High Five

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1086. High Five
 * https://leetcode.com/problems/high-five/
 * Difficulty: Easy
 *
 * Given a list of the scores of different students, items, where items[i] = [IDi, scorei]
 * represents one score from a student with IDi, calculate each student's top five average.
 *
 * Return the answer as an array of pairs result, where result[j] = [IDj, topFiveAveragej]
 * represents the student with IDj and their top five average. Sort result by IDj in
 * increasing order.
 *
 * A student's top five average is calculated by taking the sum of their top five scores
 * and dividing it by 5 using integer division.
 */

/**
 * @param {number[][]} items
 * @return {number[][]}
 */
var highFive = function(items) {
  const studentScores = new Map();

  for (const [id, score] of items) {
    if (!studentScores.has(id)) {
      studentScores.set(id, []);
    }
    studentScores.get(id).push(score);
  }

  const result = [];
  for (const [id, scores] of studentScores) {
    scores.sort((a, b) => b - a);
    const topFiveSum = scores.slice(0, 5).reduce((sum, score) => sum + score, 0);
    const average = Math.floor(topFiveSum / 5);
    result.push([id, average]);
  }

  result.sort((a, b) => a[0] - b[0]);

  return result;
};
```

1087 Brace Expansion

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1087. Brace Expansion
 * https://leetcode.com/problems/brace-expansion/
 * Difficulty: Medium
 *
 * You are given a string s representing a list of words. Each letter in the word has
 * one or more options.
 * - If there is one option, the letter is represented as is.
 * - If there is more than one option, then curly braces delimit the options.
 *   For example, "{a,b,c}" represents options ["a", "b", "c"].
 *
 * For example, if s = "a{b,c}", the first character is always 'a', but the second
 * character can be 'b' or 'c'. The original list is ["ab", "ac"].
 *
 * Return all words that can be formed in this manner, sorted in lexicographical order.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var expand = function(s) {
  const groups = [];
  let i = 0;

  while (i < s.length) {
    if (s[i] === '{') {
      const start = i + 1;
      while (s[i] !== '}') i++;
      const options = s.substring(start, i).split(',');
      groups.push(options);
      i++;
    } else {
      groups.push([s[i]]);
      i++;
    }
  }

  const result = [];

  backtrack(0, '');

  return result.sort();

  function backtrack(index, current) {
    if (index === groups.length) {
      result.push(current);
      return;
    }

    for (const option of groups[index]) {
      backtrack(index + 1, current + option);
    }
  }
};
```

1088 Confusing Number Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1088. Confusing Number II
 * https://leetcode.com/problems/confusing-number-ii/
 * Difficulty: Hard
 *
 * A confusing number is a number that when rotated 180 degrees becomes a different number
 * with each digit valid.
 *
 * We can rotate digits of a number by 180 degrees to form new digits.
 * - When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6
respectively.
 * - When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.
 *
 * Note that after rotating a number, we can ignore leading zeros.
 * - For example, after rotating 8000, we have 0008 which is considered as just 8.
 *
 * Given an integer n, return the number of confusing numbers in the inclusive range [1, n].
 */

/**
 * @param {number} n
 * @return {number}
 */
var confusingNumberII = function(n) {
  const validDigits = [0, 1, 6, 8, 9];
  const rotationMap = { 0: 0, 1: 1, 6: 9, 8: 8, 9: 6 };
  let count = 0;

  const maxLength = n.toString().length;

  for (let len = 1; len <= maxLength; len++) {
    dfs(0, 0, len);
  }

  return count;

  function isConfusing(num) {
    const original = num;
    let rotated = 0;

    while (num > 0) {
      const digit = num % 10;
      rotated = rotated * 10 + rotationMap[digit];
      num = Math.floor(num / 10);
    }

    return original !== rotated;
  }

  function dfs(current, length, targetLength) {
    if (length === targetLength) {
      if (current <= n && current > 0 && isConfusing(current)) {
        count++;
      }
      return;
    }

    for (const digit of validDigits) {
      if (current === 0 && digit === 0) continue;
      const next = current * 10 + digit;
      if (next > n) break;
      dfs(next, length + 1, targetLength);
    }
  }
};
```

1089 Duplicate Zeros

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1089. Duplicate Zeros
 * https://leetcode.com/problems/duplicate-zeros/
 * Difficulty: Easy
 *
 * Given a fixed-length integer array arr, duplicate each occurrence of zero, shifting the
 * remaining elements to the right.
 *
 * Note that elements beyond the length of the original array are not written. Do the above
 * modifications to the input array in place and do not return anything.
 */

/**
 * @param {number[]} arr
 * @return {void} Do not return anything, modify arr in-place instead.
 */
var duplicateZeros = function(arr) {
  let zerosToAdd = 0;

  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === 0) zerosToAdd++;
  }

  for (let i = arr.length - 1; i >= 0; i--) {
    if (arr[i] === 0) {
      zerosToAdd--;
      if (i + zerosToAdd + 1 < arr.length) {
        arr[i + zerosToAdd + 1] = 0;
      }
    }
    if (i + zerosToAdd < arr.length) {
      arr[i + zerosToAdd] = arr[i];
    }
  }
};
```

1090 Largest Values From Labels

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1090. Largest Values From Labels
 * https://leetcode.com/problems/largest-values-from-labels/
 * Difficulty: Medium
 *
 * You are given n item's value and label as two integer arrays values and labels. You are
also
 * given two integers numWanted and useLimit.
 *
 * Your task is to find a subset of items with the maximum sum of their values such that:
 * - The number of items is at most numWanted.
 * - The number of items with the same label is at most useLimit.
 *
 * Return the maximum sum.
 */

/**
 * @param {number[]} values
 * @param {number[]} labels
 * @param {number} numWanted
 * @param {number} useLimit
 * @return {number}
 */
var largestValsFromLabels = function(values, labels, numWanted, useLimit) {
  const items = values.map((value, index) => ({ value, label: labels[index] }));
  items.sort((a, b) => b.value - a.value);
```

```
  const labelCount = new Map();
  let result = 0;
  let itemsUsed = 0;

  for (const { value, label } of items) {
    const currentCount = labelCount.get(label) || 0;
    if (itemsUsed < numWanted && currentCount < useLimit) {
      result += value;
      labelCount.set(label, currentCount + 1);
      itemsUsed++;
    }
  }

  return result;
};
```

1091 Shortest Path In Binary Matrix

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1091. Shortest Path in Binary Matrix
 * https://leetcode.com/problems/shortest-path-in-binary-matrix/
 * Difficulty: Medium
 *
 * Given an n x n binary matrix grid, return the length of the shortest clear path in the
matrix.
 * If there is no clear path, return -1.
 *
 * A clear path in a binary matrix is a path from the top-left cell (i.e., (0, 0)) to the
 * bottom-right cell (i.e., (n - 1, n - 1)) such that:
 * - All the visited cells of the path are 0.
 * - All the adjacent cells of the path are 8-directionally connected (i.e., they are
different
 *   and they share an edge or a corner).
 *
 * The length of a clear path is the number of visited cells of this path.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var shortestPathBinaryMatrix = function(grid) {
  const size = grid.length;
  if (grid[0][0] === 1 || grid[size - 1][size - 1] === 1) return -1;

  const queue = [[0, 0, 1]];
  const directions = [[-1, -1], [-1, 0], [-1, 1], [0, -1], [0, 1], [1, -1], [1, 0], [1, 1]];
  grid[0][0] = 1;

  while (queue.length) {
    const [row, col, distance] = queue.shift();
    if (row === size - 1 && col === size - 1) return distance;

    for (const [deltaRow, deltaCol] of directions) {
      const newRow = row + deltaRow;
      const newCol = col + deltaCol;

      if (newRow >= 0 && newRow < size && newCol >= 0
          && newCol < size && grid[newRow][newCol] === 0) {
        queue.push([newRow, newCol, distance + 1]);
        grid[newRow][newCol] = 1;
      }
    }
  }

  return -1;
};
```

1092 Shortest Common Supersequence

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1092. Shortest Common Supersequence
 * https://leetcode.com/problems/shortest-common-supersequence/
 * Difficulty: Hard
 *
 * Given two strings str1 and str2, return the shortest string that has both str1 and str2 as
 * subsequences. If there are multiple valid strings, return any of them.
 *
 * A string s is a subsequence of string t if deleting some number of characters from t
(possibly
 * 0) results in the string s.
 */

/**
 * @param {string} str1
 * @param {string} str2
 * @return {string}
 */
var shortestCommonSupersequence = function(str1, str2) {
  const dp = new Array(str1.length + 1).fill().map(() => new Array(str2.length + 1).fill(0));
  let result = '';

  for (let i = 0; i <= str1.length; i++) {
    dp[i][0] = i;
  }
  for (let j = 0; j <= str2.length; j++) {
    dp[0][j] = j;
  }

  for (let i = 1; i <= str1.length; i++) {
    for (let j = 1; j <= str2.length; j++) {
      dp[i][j] = str1[i-1] === str2[j-1] ? dp[i-1][j-1] + 1 : Math.min(dp[i-1][j], dp[i][j-1])
 + 1;
    }
  }

  for (let i = str1.length, j = str2.length; i > 0 || j > 0;) {
    if (!i) {
      result = str2[--j] + result;
    } else if (!j) {
      result = str1[--i] + result;
    } else if (str1[i-1] === str2[j-1]) {
      result = str1[--i] + (str2[--j], result);
    } else {
      dp[i][j] === dp[i-1][j] + 1
        ? result = str1[--i] + result
        : result = str2[--j] + result;
    }
  }

  return result;
};
```

1093 Statistics From A Large Sample

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1093. Statistics from a Large Sample
 * https://leetcode.com/problems/statistics-from-a-large-sample/
 * Difficulty: Medium
```

```
 *
 * You are given a large sample of integers in the range [0, 255]. Since the sample is so
large,
 * it is represented by an array count where count[k] is the number of times that k appears in
 * the sample.
 *
 * Calculate the following statistics:
 * - minimum: The minimum element in the sample.
 * - maximum: The maximum element in the sample.
 * - mean: The average of the sample, calculated as the total sum of all elements divided by
the
 *   total number of elements.
 * - median:
 *   - If the sample has an odd number of elements, then the median is the middle element once
 *     the sample is sorted.
 *   - If the sample has an even number of elements, then the median is the average of the two
 *     middle elements once the sample is sorted.
 * - mode: The number that appears the most in the sample. It is guaranteed to be unique.
 *
 * Return the statistics of the sample as an array of floating-point numbers [minimum,
maximum,
 * mean, median, mode]. Answers within 10-5 of the actual answer will be accepted.
 */

/**
 * @param {number[]} count
 * @return {number[]}
 */
var sampleStats = function(count) {
  let minimum = 256;
  let maximum = -1;
  let sum = 0;
  let totalCount = 0;
  let mode = 0;
  let maxFrequency = 0;

  for (let value = 0; value < 256; value++) {
    const frequency = count[value];
    if (frequency > 0) {
      minimum = Math.min(minimum, value);
      maximum = Math.max(maximum, value);
      sum += value * frequency;
      totalCount += frequency;
      if (frequency > maxFrequency) {
        maxFrequency = frequency;
        mode = value;
      }
    }
  }

  const mean = sum / totalCount;
  const median = findMedian(count, totalCount);

  return [minimum, maximum, mean, median, mode];
};

function findMedian(count, totalCount) {
  const isOdd = totalCount % 2 === 1;
  const target = Math.floor(totalCount / 2);
  let currentCount = 0;
  let firstMedian = -1;

  for (let value = 0; value < 256; value++) {
    currentCount += count[value];
    if (isOdd) {
      if (currentCount > target) return value;
    } else {
      if (firstMedian === -1 && currentCount >= target) {
        firstMedian = value;
      }
      if (currentCount > target) {
        return (firstMedian + value) / 2;
      }
    }
  }
```

```
}
```

1094 Car Pooling

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1094. Car Pooling
 * https://leetcode.com/problems/car-pooling/
 * Difficulty: Medium
 *
 * There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot
turn
 * around and drive west).
 *
 * You are given the integer capacity and an array trips where
 * trips[i] = [numPassengersi, fromi, toi] indicates that the ith trip has numPassengersi
 * passengers and the locations to pick them up and drop them off are fromi and toi
respectively.
 * The locations are given as the number of kilometers due east from the car's initial
location.
 *
 * Return true if it is possible to pick up and drop off all passengers for all the given
trips,
 * or false otherwise.
 */

/**
 * @param {number[][]} trips
 * @param {number} capacity
 * @return {boolean}
 */
var carPooling = function(trips, capacity) {
  const timeline = new Array(1001).fill(0);

  for (const [passengers, start, end] of trips) {
    timeline[start] += passengers;
    timeline[end] -= passengers;
  }

  let currentPassengers = 0;
  for (const change of timeline) {
    currentPassengers += change;
    if (currentPassengers > capacity) return false;
  }

  return true;
};
```

1095 Find In Mountain Array

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1095. Find in Mountain Array
 * https://leetcode.com/problems/find-in-mountain-array/
 * Difficulty: Hard
 *
 * You may recall that an array arr is a mountain array if and only if:
 * - arr.length >= 3
 * - There exists some i with 0 < i < arr.length - 1 such that:
 *   - arr[0] < arr[1] < ... < arr[i - 1] < arr[i]
 *   - arr[i] > arr[i + 1] > ... > arr[arr.length - 1]
 *
 * Given a mountain array mountainArr, return the minimum index such that
```

```
 * mountainArr.get(index) == target. If such an index does not exist, return -1.
 *
 * You cannot access the mountain array directly. You may only access the array
 * using a MountainArray interface:
 * - MountainArray.get(k) returns the element of the array at index k (0-indexed).
 * - MountainArray.length() returns the length of the array.
 *
 * Submissions making more than 100 calls to MountainArray.get will be judged Wrong Answer.
 * Also, any solutions that attempt to circumvent the judge will result in disqualification.
 */

/**
 * @param {number} target
 * @param {MountainArray} mountainArr
 * @return {number}
 */
var findInMountainArray = function(target, mountainArr) {
  const length = mountainArr.length();

  let left = 0;
  let right = length - 1;
  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (mountainArr.get(mid) < mountainArr.get(mid + 1)) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }
  const peak = left;

  left = 0;
  right = peak;
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    const value = mountainArr.get(mid);
    if (value === target) return mid;
    if (value < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  left = peak;
  right = length - 1;
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    const value = mountainArr.get(mid);
    if (value === target) return mid;
    if (value > target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return -1;
};
```

1096 Brace Expansion Ii

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1096. Brace Expansion II
 * https://leetcode.com/problems/brace-expansion-ii/
 * Difficulty: Hard
 *
```

```
 * Under the grammar given below, strings can represent a set of lowercase words. Let R(expr)
 * denote the set of words the expression represents.
 *
 * The grammar can best be understood through simple examples:
 * - Single letters represent a singleton set containing that word.
 *   - R("a") = {"a"}
 *   - R("w") = {"w"}
 * - When we take a comma-delimited list of two or more expressions, we take the union of
 *   possibilities.
 *   - R("{a,b,c}") = {"a","b","c"}
 *   - R("{{a,b},{b,c}}") = {"a","b","c"} (notice the final set only contains each word
 *     at most once)
 * - When we concatenate two expressions, we take the set of possible concatenations between
two
 *   words where the first word comes from the first expression and the second word comes from
 *   the second expression.
 *   - R("{a,b}{c,d}") = {"ac","ad","bc","bd"}
 *   - R("a{b,c}{d,e}f{g,h}") = {"abdfg", "abdfh", "abefg", "abefh", "acdfg", "acdfh",
 *     "acefg", "acefh"}
 *
 * Formally, the three rules for our grammar:
 * - For every lowercase letter x, we have R(x) = {x}.
 * - For expressions e1, e2, ... , ek with k >= 2, we have R({e1, e2, ...}) = R(e1) ∪ R(e2) ∪
...
 * - For expressions e1 and e2, we have R(e1 + e2) = {a + b for (a, b) in R(e1) × R(e2)},
 *   where + denotes concatenation, and × denotes the cartesian product.
 *
 * Given an expression representing a set of words under the given grammar, return the sorted
list
 * of words that the expression represents.
 */

/**
 * @param {string} expression
 * @return {string[]}
 */
var braceExpansionII = function(expression) {
  function cartesian(set1, set2) {
    const result = new Set();
    for (const a of set1) {
      for (const b of set2) {
        result.add(a + b);
      }
    }
    return result;
  }

  function evaluate(exp) {
    let i = 0;

    function parseUnit() {
      let result = new Set();

      if (exp[i] === '{') {
        i++;
        result = parseExprList();
        i++;
      } else {
        result.add(exp[i]);
        i++;
      }

      return result;
    }

    function parseExprList() {
      const result = new Set();

      const firstExpr = parseExpr();
      for (const word of firstExpr) {
        result.add(word);
      }

      while (i < exp.length && exp[i] === ',') {
        i++;
```

```
        const nextExpr = parseExpr();
        for (const word of nextExpr) {
          result.add(word);
        }
      }

      return result;
    }

    function parseExpr() {
      let result = new Set(['']);

      while (i < exp.length && exp[i] !== ',' && exp[i] !== '}') {
        const unit = parseUnit();
        result = cartesian(result, unit);
      }

      return result;
    }

    return parseExprList();
  }

  return [...evaluate(expression)].sort();
};
```

1099 Two Sum Less Than K

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1099. Two Sum Less Than K
 * https://leetcode.com/problems/two-sum-less-than-k/
 * Difficulty: Easy
 *
 * Given an array nums of integers and integer k, return the maximum sum such that there
 * exists i < j with nums[i] + nums[j] = sum and sum < k. If no i, j exist satisfying
 * this equation, return -1.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var twoSumLessThanK = function(nums, k) {
  nums.sort((a, b) => a - b);
  let left = 0;
  let right = nums.length - 1;
  let result = -1;

  while (left < right) {
    const sum = nums[left] + nums[right];

    if (sum < k) {
      result = Math.max(result, sum);
      left++;
    } else {
      right--;
    }
  }

  return result;
};
```

1100 Find K Length Substrings With No Repeated Characters

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1100. Find K-Length Substrings With No Repeated Characters
 * https://leetcode.com/problems/find-k-length-substrings-with-no-repeated-characters/
 * Difficulty: Medium
 *
 * Given a string s and an integer k, return the number of substrings in s of length k
 * with no repeated characters.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var numKLenSubstrNoRepeats = function(s, k) {
  if (k > s.length) return 0;

  const charCount = new Map();
  let count = 0;

  for (let i = 0; i < k; i++) {
    charCount.set(s[i], (charCount.get(s[i]) || 0) + 1);
  }

  if (charCount.size === k) count++;

  for (let i = k; i < s.length; i++) {
    const leftChar = s[i - k];
    const rightChar = s[i];

    charCount.set(leftChar, charCount.get(leftChar) - 1);
    if (charCount.get(leftChar) === 0) {
      charCount.delete(leftChar);
    }

    charCount.set(rightChar, (charCount.get(rightChar) || 0) + 1);

    if (charCount.size === k) count++;
  }

  return count;
};
```

1101 The Earliest Moment When Everyone Become Friends

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1101. The Earliest Moment When Everyone Become Friends
 * https://leetcode.com/problems/the-earliest-moment-when-everyone-become-friends/
 * Difficulty: Medium
 *
 * There are n people in a social group labeled from 0 to n - 1. You are given an array logs
 * where logs[i] = [timestampi, xi, yi] indicates that xi and yi will be friends at the time
 * timestampi.
 *
 * Friendship is symmetric. That means if a is friends with b, then b is friends with a.
 * Also, person a is acquainted with a person b if a is friends with b, or a is a friend
 * of someone acquainted with b.
 *
 * Return the earliest time for which every person became acquainted with every other person.
```

```
 * If there is no such earliest time, return -1.
 */

/**
 * @param {number[][]} logs
 * @param {number} n
 * @return {number}
 */
var earliestAcq = function(logs, n) {
  logs.sort((a, b) => a[0] - b[0]);

  const parent = Array.from({ length: n }, (_, i) => i);
  const rank = new Array(n).fill(0);
  let components = n;
  for (const [timestamp, x, y] of logs) {
    if (union(x, y) && components === 1) {
      return timestamp;
    }
  }

  return -1;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    const rootX = find(x);
    const rootY = find(y);

    if (rootX === rootY) return false;

    if (rank[rootX] < rank[rootY]) {
      parent[rootX] = rootY;
    } else if (rank[rootX] > rank[rootY]) {
      parent[rootY] = rootX;
    } else {
      parent[rootY] = rootX;
      rank[rootX]++;
    }

    components--;
    return true;
  }
};
```

1102 Path With Maximum Minimum Value

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1102. Path With Maximum Minimum Value
 * https://leetcode.com/problems/path-with-maximum-minimum-value/
 * Difficulty: Medium
 *
 * Given an m x n integer matrix grid, return the maximum score of a path starting
 * at (0, 0) and ending at (m - 1, n - 1) moving in the 4 cardinal directions.
 *
 * The score of a path is the minimum value in that path.
 * - For example, the score of the path 8 → 4 → 5 → 9 is 4.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maximumMinimumPath = function(grid) {
```

```
  const m = grid.length;
  const n = grid[0].length;
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  const visited = new Array(m).fill(null).map(() => new Array(n).fill(false));

  const maxHeap = new PriorityQueue((a, b) => b[0] - a[0]);
  maxHeap.enqueue([grid[0][0], 0, 0]);
  visited[0][0] = true;

  while (!maxHeap.isEmpty()) {
    const [currentScore, row, col] = maxHeap.dequeue();

    if (row === m - 1 && col === n - 1) {
      return currentScore;
    }

    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;

      if (newRow >= 0 && newRow < m && newCol >= 0 && newCol < n && !visited[newRow][newCol])
{
        visited[newRow][newCol] = true;
        const newScore = Math.min(currentScore, grid[newRow][newCol]);
        maxHeap.enqueue([newScore, newRow, newCol]);
      }
    }
  }

  return -1;
};
```

1103 Distribute Candies To People

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1103. Distribute Candies to People
 * https://leetcode.com/problems/distribute-candies-to-people/
 * Difficulty: Easy
 *
 * We distribute some number of `candies`, to a row of `n = num_people`
 * people in the following way:
 *
 * We then give 1 candy to the first person, 2 candies to the second
 * person, and so on until we give `n` candies to the last person.
 *
 * Then, we go back to the start of the row, giving `n + 1` candies to
 * the first person, `n + 2` candies to the second person, and so on
 * until we give `2 * n` candies to the last person.
 *
 * This process repeats (with us giving one more candy each time,
 * and moving to the start of the row after we reach the end) until
 * we run out of candies. The last person will receive all of our
 * remaining candies (not necessarily one more than the previous gift).
 *
 * Return an array (of length `num_people` and sum `candies`) that
 * represents the final distribution of candies.
 */

/**
 * @param {number} candies
 * @param {number} num_people
 * @return {number[]}
 */
var distributeCandies = function(candies, num_people) {
  const answer = new Array(num_people).fill(0);

  for (let amount = 0; candies > 0; candies -= amount) {
    answer[amount % num_people] += Math.min(candies, ++amount);
```

```
  }

  return answer;
};
```

1104 Path In Zigzag Labelled Binary Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1104. Path In Zigzag Labelled Binary Tree
 * https://leetcode.com/problems/path-in-zigzag-labelled-binary-tree/
 * Difficulty: Medium
 *
 * In an infinite binary tree where every node has two children, the nodes are labelled
 * in row order.
 *
 * In the odd numbered rows (ie., the first, third, fifth,...), the labelling is left to
right,
 * while in the even numbered rows (second, fourth, sixth,...), the labelling is right to
left.
 *
 * Given the label of a node in this tree, return the labels in the path from the root of the
 * tree to the node with that label.
 */

/**
 * @param {number} label
 * @return {number[]}
 */
var pathInZigZagTree = function(label) {
  const result = [];
  let current = label;

  while (current >= 1) {
    result.unshift(current);
    const level = Math.floor(Math.log2(current));
    current = Math.floor(((2 ** (level + 1)) - 1 + (2 ** level) - current) / 2);
  }

  return result;
};
```

1105 Filling Bookcase Shelves

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1105. Filling Bookcase Shelves
 * https://leetcode.com/problems/filling-bookcase-shelves/
 * Difficulty: Medium
 *
 * You are given an array books where books[i] = [thicknessi, heighti] indicates the thickness
 * and height of the ith book. You are also given an integer shelfWidth.
 *
 * We want to place these books in order onto bookcase shelves that have a total width
shelfWidth.
 *
 * We choose some of the books to place on this shelf such that the sum of their thickness is
less
 * than or equal to shelfWidth, then build another level of the shelf of the bookcase so that
the
 * total height of the bookcase has increased by the maximum height of the books we just put
down.
 * We repeat this process until there are no more books to place.
```

```
 *
 * Note that at each step of the above process, the order of the books we place is the same
order
 * as the given sequence of books.
 *
 * For example, if we have an ordered list of 5 books, we might place the first and second
book
 * onto the first shelf, the third book on the second shelf, and the fourth and fifth book on
the
 * last shelf.
 *
 * Return the minimum possible height that the total bookshelf can be after placing shelves in
 * this manner.
 */

/**
 * @param {number[][]} books
 * @param {number} shelfWidth
 * @return {number}
 */
var minHeightShelves = function(books, shelfWidth) {
  const dp = new Array(books.length + 1).fill(0);

  for (let i = 1; i <= books.length; i++) {
    let width = books[i - 1][0];
    let height = books[i - 1][1];
    dp[i] = dp[i - 1] + height;

    let j = i - 2;
    while (j >= 0 && width + books[j][0] <= shelfWidth) {
      width += books[j][0];
      height = Math.max(height, books[j][1]);
      dp[i] = Math.min(dp[i], dp[j] + height);
      j--;
    }
  }

  return dp[books.length];
};
```

1106 Parsing A Boolean Expression

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1106. Parsing A Boolean Expression
 * https://leetcode.com/problems/parsing-a-boolean-expression/
 * Difficulty: Hard
 *
 * A boolean expression is an expression that evaluates to either true or false. It can be in
 * one of the following shapes:
 * - 't' that evaluates to true.
 * - 'f' that evaluates to false.
 * - '!(subExpr)' that evaluates to the logical NOT of the inner expression subExpr.
 * - '&(subExpr1, subExpr2, ..., subExprn)' that evaluates to the logical AND of the inner
 *   expressions subExpr1, subExpr2, ..., subExprn where n >= 1.
 * - '|(subExpr1, subExpr2, ..., subExprn)' that evaluates to the logical OR of the inner
 *   expressions subExpr1, subExpr2, ..., subExprn where n >= 1.
 *
 * Given a string expression that represents a boolean expression, return the evaluation of
 * that expression.
 *
 * It is guaranteed that the given expression is valid and follows the given rules.
 */

/**
 * @param {string} expression
 * @return {boolean}
 */
var parseBoolExpr = function(expression) {
```

```
  const stack = [];

  for (const char of expression) {
    if (char === ')') {
      const operands = [];
      while (stack[stack.length - 1] !== '(') {
        operands.push(stack.pop());
      }
      stack.pop();
      const operator = stack.pop();

      if (operator === '!') {
        stack.push(!operands[0]);
      } else if (operator === '&') {
        stack.push(operands.every(val => val));
      } else if (operator === '|') {
        stack.push(operands.some(val => val));
      }
    } else if (char !== ',') {
      stack.push(char === 't' ? true : char === 'f' ? false : char);
    }
  }

  return stack[0];
};
```

1108 Defanging An Ip Address

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1108. Defanging an IP Address
 * https://leetcode.com/problems/defanging-an-ip-address/
 * Difficulty: Easy
 *
 * Given a valid (IPv4) IP address, return a defanged version of that IP address.
 *
 * A defanged IP address replaces every period "." with "[.]".
 */

/**
 * @param {string} address
 * @return {string}
 */
var defangIPaddr = function(address) {
  return address.replace(/\./g, '[.]');
};

// or...

/**
 * @param {string} address
 * @return {string}
 */
var defangIPaddr = function(address) {
  return address.split('.').join('[.]');
};
```

1109 Corporate Flight Bookings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1109. Corporate Flight Bookings
 * https://leetcode.com/problems/corporate-flight-bookings/
```

```
 * Difficulty: Medium
 *
 * There are n flights that are labeled from 1 to n.
 *
 * You are given an array of flight bookings bookings, where bookings[i] = [firsti, lasti,
seatsi]
 * represents a booking for flights firsti through lasti (inclusive) with seatsi seats
reserved
 * for each flight in the range.
 *
 * Return an array answer of length n, where answer[i] is the total number of seats reserved
for
 * flight i.
 */

/**
 * @param {number[][]} bookings
 * @param {number} n
 * @return {number[]}
 */
var corpFlightBookings = function(bookings, n) {
  const result = new Array(n).fill(0);

  for (const [start, end, count] of bookings) {
    result[start - 1] += count;
    if (end < n) result[end] -= count;
  }

  for (let i = 1; i < n; i++) {
    result[i] += result[i - 1];
  }

  return result;
};
```

1110 Delete Nodes And Return Forest

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1110. Delete Nodes And Return Forest
 * https://leetcode.com/problems/delete-nodes-and-return-forest/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, each node in the tree has a distinct value.
 *
 * After deleting all nodes with a value in toDelete, we are left with a forest (a disjoint
 * union of trees).
 *
 * Return the roots of the trees in the remaining forest. You may return the result in any
order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number[]} toDelete
 * @return {TreeNode[]}
 */
var delNodes = function(root, toDelete) {
  const deleteSet = new Set(toDelete);
  const forest = [];
```

```
  traverse(root, true);
  return forest;

  function traverse(node, isRoot) {
    if (!node) return null;

    const shouldDelete = deleteSet.has(node.val);
    if (isRoot && !shouldDelete) forest.push(node);

    node.left = traverse(node.left, shouldDelete);
    node.right = traverse(node.right, shouldDelete);

    return shouldDelete ? null : node;
  }
};
```

1111 Maximum Nesting Depth Of Two Valid Parentheses Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1111. Maximum Nesting Depth of Two Valid Parentheses Strings
 * https://leetcode.com/problems/maximum-nesting-depth-of-two-valid-parentheses-strings/
 * Difficulty: Medium
 *
 * A string is a valid parentheses string (denoted VPS) if and only if it consists of "(" and
")"
 * characters only, and:
 * - It is the empty string, or
 * - It can be written as AB (A concatenated with B), where A and B are VPS's, or
 * - It can be written as (A), where A is a VPS.
 *
 * We can similarly define the nesting depth depth(S) of any VPS S as follows:
 * - depth("") = 0
 * - depth(A + B) = max(depth(A), depth(B)), where A and B are VPS's
 * - depth("(" + A + ")") = 1 + depth(A), where A is a VPS.
 *
 * For example,  "", "()()", and "()(()())" are VPS's (with nesting depths 0, 1, and 2),
 * and ")(" and "(()" are not VPS's.
 *
 * Given a VPS seq, split it into two disjoint subsequences A and B, such that A and B are
VPS's
 * (and A.length + B.length = seq.length).
 *
 * Now choose any such A and B such that max(depth(A), depth(B)) is the minimum possible
value.
 *
 * Return an answer array (of length seq.length) that encodes such a choice of A and B:
 * answer[i] = 0 if seq[i] is part of A, else answer[i] = 1.  Note that even though multiple
 * answers may exist, you may return any of them.
 */

/**
 * @param {string} seq
 * @return {number[]}
 */
var maxDepthAfterSplit = function(seq) {
  const result = new Array(seq.length);
  let depth = 0;

  for (let i = 0; i < seq.length; i++) {
    if (seq[i] === '(') {
      result[i] = depth++ % 2;
    } else {
      result[i] = --depth % 2;
    }
  }

  return result;
};
```

1118 Number Of Days In A Month

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1118. Number of Days in a Month
 * https://leetcode.com/problems/number-of-days-in-a-month/
 * Difficulty: Easy
 *
 * Given a year year and a month month, return the number of days of that month.
 */

/**
 * @param {number} year
 * @param {number} month
 * @return {number}
 */
var numberOfDays = function(year, month) {
  return new Date(year, month, 0).getDate();
};
```

1119 Remove Vowels From A String

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1119. Remove Vowels from a String
 * https://leetcode.com/problems/remove-vowels-from-a-string/
 * Difficulty: Easy
 *
 * Given a string s, remove the vowels 'a', 'e', 'i', 'o', and 'u' from it, and
 * return the new string.
 */

/**
 * @param {string} s
 * @return {string}
 */
var removeVowels = function(s) {
  return s.replace(/[aeiou]/g, '');
};
```

1120 Maximum Average Subtree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1120. Maximum Average Subtree
 * https://leetcode.com/problems/maximum-average-subtree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the maximum average value of a subtree of that
tree.
 * Answers within 10-5 of the actual answer will be accepted.
 *
 * A subtree of a tree is any node of that tree plus all its descendants.
 *
 * The average value of a tree is the sum of its values, divided by the number of nodes.
 */
```

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maximumAverageSubtree = function(root) {
  let maxAverage = 0;
  calculateSubtree(root);
  return maxAverage;

  function calculateSubtree(node) {
    if (!node) return [0, 0];

    const [leftSum, leftCount] = calculateSubtree(node.left);
    const [rightSum, rightCount] = calculateSubtree(node.right);
    const totalSum = leftSum + rightSum + node.val;
    const totalCount = leftCount + rightCount + 1;
    const currentAverage = totalSum / totalCount;

    maxAverage = Math.max(maxAverage, currentAverage);

    return [totalSum, totalCount];
  }
};
```

1121 Divide Array Into Increasing Sequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1121. Divide Array Into Increasing Sequences
 * https://leetcode.com/problems/divide-array-into-increasing-sequences/
 * Difficulty: Hard
 *
 * Given an integer array nums sorted in non-decreasing order and an integer k, return true
 * if this array can be divided into one or more disjoint increasing subsequences of length
 * at least k, or false otherwise.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
var canDivideIntoSubsequences = function(nums, k) {
  let maxFrequency = 0;
  let currentFrequency = 1;

  for (let i = 1; i < nums.length; i++) {
    currentFrequency = nums[i] === nums[i - 1] ? currentFrequency + 1 : 1;
    maxFrequency = Math.max(maxFrequency, currentFrequency);
  }

  return k * maxFrequency <= nums.length;
};
```

1122 Relative Sort Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1122. Relative Sort Array
 * https://leetcode.com/problems/relative-sort-array/
 * Difficulty: Easy
 *
 * Given two arrays arr1 and arr2, the elements of arr2 are distinct, and all
 * elements in arr2 are also in arr1.
 *
 * Sort the elements of arr1 such that the relative ordering of items in arr1
 * are the same as in arr2. Elements that do not appear in arr2 should be
 * placed at the end of arr1 in ascending order.
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
 * @return {number[]}
 */
var relativeSortArray = function(arr1, arr2) {
  const map = new Map(arr2.map((v, i) => [v, i]));
  return arr1.sort((a, b) => (map.get(a) ?? arr2.length + a) - (map.get(b) ?? arr2.length +
b));
};
```

1123 Lowest Common Ancestor Of Deepest Leaves

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1123. Lowest Common Ancestor of Deepest Leaves
 * https://leetcode.com/problems/lowest-common-ancestor-of-deepest-leaves/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the lowest common ancestor of its deepest leaves.
 *
 * Recall that:
 * - The node of a binary tree is a leaf if and only if it has no children
 * - The depth of the root of the tree is 0. if the depth of a node is d, the depth of each of
 *   its children is d + 1.
 * - The lowest common ancestor of a set S of nodes, is the node A with the largest depth such
 *   that every node in S is in the subtree with root A.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var lcaDeepestLeaves = function(root) {
  return findDepthAndNode(root).node;

  function findDepthAndNode(node) {
    if (!node) return { depth: 0, node: null };
```

```
    const left = findDepthAndNode(node.left);
    const right = findDepthAndNode(node.right);

    if (left.depth === right.depth) {
      return { depth: left.depth + 1, node: node };
    }

    const deeper = left.depth > right.depth ? left : right;
    return { depth: deeper.depth + 1, node: deeper.node };
  }
};
```

1124 Longest Well Performing Interval

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1124. Longest Well-Performing Interval
 * https://leetcode.com/problems/longest-well-performing-interval/
 * Difficulty: Medium
 *
 * We are given hours, a list of the number of hours worked per day for a given employee.
 *
 * A day is considered to be a tiring day if and only if the number of hours worked is
 * (strictly) greater than 8.
 *
 * A well-performing interval is an interval of days for which the number of tiring days
 * is strictly larger than the number of non-tiring days.
 *
 * Return the length of the longest well-performing interval.
 */

/**
 * @param {number[]} hours
 * @return {number}
 */
var longestWPI = function(hours) {
  let result = 0;
  let score = 0;
  const seen = new Map();

  for (let i = 0; i < hours.length; i++) {
    score += hours[i] > 8 ? 1 : -1;

    if (score > 0) {
      result = i + 1;
    } else {
      if (!seen.has(score)) {
        seen.set(score, i);
      }
      if (seen.has(score - 1)) {
        result = Math.max(result, i - seen.get(score - 1));
      }
    }
  }

  return result;
};
```

1125 Smallest Sufficient Team

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1125. Smallest Sufficient Team
```

```
 * https://leetcode.com/problems/smallest-sufficient-team/
 * Difficulty: Hard
 *
 * In a project, you have a list of required skills reqSkills, and a list of people. The ith
 * person people[i] contains a list of skills that the person has.
 *
 * Consider a sufficient team: a set of people such that for every required skill in
reqSkills,
 * there is at least one person in the team who has that skill. We can represent these teams
by
 * the index of each person.
 *
 * For example, team = [0, 1, 3] represents the people with skills people[0], people[1], and
 * people[3].
 *
 * Return any sufficient team of the smallest possible size, represented by the index of each
 * person. You may return the answer in any order.
 *
 * It is guaranteed an answer exists.
 */

/**
 * @param {string[]} reqSkills
 * @param {string[][]} people
 * @return {number[]}
 */
var smallestSufficientTeam = function(reqSkills, people) {
  const skillCount = reqSkills.length;
  const skillMap = new Map(reqSkills.map((skill, index) => [skill, 1 << index]));
  const peopleSkills = people.map(skills =>
    skills.reduce((mask, skill) => mask | skillMap.get(skill), 0)
  );
  const target = (1 << skillCount) - 1;
  const dp = new Map([[0, []]]);

  for (let i = 0; i < people.length; i++) {
    const currentSkill = peopleSkills[i];
    for (const [prevMask, team] of dp) {
      const newMask = prevMask | currentSkill;
      if (!dp.has(newMask) || dp.get(newMask).length > team.length + 1) {
        dp.set(newMask, [...team, i]);
      }
    }
  }

  return dp.get(target);
};
```

1128 Number Of Equivalent Domino Pairs

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1128. Number of Equivalent Domino Pairs
 * https://leetcode.com/problems/number-of-equivalent-domino-pairs/
 * Difficulty: Easy
 *
 * Given a list of dominoes, dominoes[i] = [a, b] is equivalent to dominoes[j] = [c, d]
 * if and only if either (a == c and b == d), or (a == d and b == c) - that is, one domino
 * can be rotated to be equal to another domino.
 *
 * Return the number of pairs (i, j) for which 0 <= i < j < dominoes.length, and
 * dominoes[i] is equivalent to dominoes[j].
 */

/**
 * @param {number[][]} dominoes
 * @return {number}
 */
var numEquivDominoPairs = function(dominoes) {
```

```
  const map = new Map();
  let result = 0;

  for (const [a, b] of dominoes) {
    const key = Math.min(a, b) * 10 + Math.max(a, b);
    const count = map.get(key) || 0;
    result += count;
    map.set(key, count + 1);
  }

  return result;
};
```

1129 Shortest Path With Alternating Colors

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1129. Shortest Path with Alternating Colors
 * https://leetcode.com/problems/shortest-path-with-alternating-colors/
 * Difficulty: Medium
 *
 * You are given an integer n, the number of nodes in a directed graph where the nodes are
 * labeled from 0 to n - 1. Each edge is red or blue in this graph, and there could be
 * self-edges and parallel edges.
 *
 * You are given two arrays redEdges and blueEdges where:
 * - redEdges[i] = [ai, bi] indicates that there is a directed red edge from node ai
 *   to node bi in the graph, and
 * - blueEdges[j] = [uj, vj] indicates that there is a directed blue edge from node uj
 *   to node vj in the graph.
 *
 * Return an array answer of length n, where each answer[x] is the length of the shortest
 * path from node 0 to node x such that the edge colors alternate along the path, or -1
 * if such a path does not exist.
 */

/**
 * @param {number} n
 * @param {number[][]} redEdges
 * @param {number[][]} blueEdges
 * @return {number[]}
 */
var shortestAlternatingPaths = function(n, redEdges, blueEdges) {
  const redGraph = Array(n).fill().map(() => new Set());
  const blueGraph = Array(n).fill().map(() => new Set());

  for (const [from, to] of redEdges) redGraph[from].add(to);
  for (const [from, to] of blueEdges) blueGraph[from].add(to);

  const distances = Array(n).fill().map(() => [Infinity, Infinity]);
  const queue = [[0, 0], [0, 1]];
  distances[0] = [0, 0];

  while (queue.length) {
    const [node, color] = queue.shift();
    const nextGraph = color ? redGraph : blueGraph;
    const nextColor = 1 - color;

    for (const nextNode of nextGraph[node]) {
      if (distances[nextNode][nextColor] === Infinity) {
        distances[nextNode][nextColor] = distances[node][color] + 1;
        queue.push([nextNode, nextColor]);
      }
    }
  }

  return distances.map(([red, blue]) => {
    const min = Math.min(red, blue);
```

```
    return min === Infinity ? -1 : min;
  });
};
```

1130 Minimum Cost Tree From Leaf Values

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1130. Minimum Cost Tree From Leaf Values
 * https://leetcode.com/problems/minimum-cost-tree-from-leaf-values/
 * Difficulty: Medium
 *
 * Given an array arr of positive integers, consider all binary trees such that:
 * - Each node has either 0 or 2 children;
 * - The values of arr correspond to the values of each leaf in an in-order traversal of the
tree.
 * - The value of each non-leaf node is equal to the product of the largest leaf value in its
left
 *   and right subtree, respectively.
 *
 * Among all possible binary trees considered, return the smallest possible sum of the values
of
 * each non-leaf node. It is guaranteed this sum fits into a 32-bit integer.
 *
 * A node is a leaf if and only if it has zero children.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var mctFromLeafValues = function(arr) {
  const stack = [];
  let result = 0;

  for (const value of arr) {
    while (stack.length && stack[stack.length - 1] <= value) {
      const small = stack.pop();
      result += small * Math.min(stack[stack.length - 1] || Infinity, value);
    }
    stack.push(value);
  }

  while (stack.length > 1) {
    const small = stack.pop();
    result += small * stack[stack.length - 1];
  }

  return result;
};
```

1131 Maximum Of Absolute Value Expression

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1131. Maximum of Absolute Value Expression
 * https://leetcode.com/problems/maximum-of-absolute-value-expression/
 * Difficulty: Medium
 *
 * Given two arrays of integers with equal lengths, return the maximum value of:
 * - |arr1[i] - arr1[j]| + |arr2[i] - arr2[j]| + |i - j|
 *
 * where the maximum is taken over all 0 <= i, j < arr1.length.
```

```
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
 * @return {number}
 */
var maxAbsValExpr = function(arr1, arr2) {
  const directions = [[1, 1], [1, -1], [-1, 1], [-1, -1]];
  let result = 0;

  for (const [dx, dy] of directions) {
    let minSeen = Infinity;
    let maxSeen = -Infinity;

    for (let i = 0; i < arr1.length; i++) {
      const value = dx * arr1[i] + dy * arr2[i] + i;
      minSeen = Math.min(minSeen, value);
      maxSeen = Math.max(maxSeen, value);
    }

    result = Math.max(result, maxSeen - minSeen);
  }

  return result;
};
```

1133 Largest Unique Number

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1133. Largest Unique Number
 * https://leetcode.com/problems/largest-unique-number/
 * Difficulty: Easy
 *
 * Given an integer array nums, return the largest integer that only occurs once.
 * If no integer occurs once, return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var largestUniqueNumber = function(nums) {
  const map = new Map();

  for (const num of nums) {
    map.set(num, (map.get(num) || 0) + 1);
  }

  let result = -1;
  for (const [num, frequency] of map) {
    if (frequency === 1) {
      result = Math.max(result, num);
    }
  }

  return result;
};
```

1134 Armstrong Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```javascript
/**
 * 1134. Armstrong Number
 * https://leetcode.com/problems/armstrong-number/
 * Difficulty: Easy
 *
 * Given an integer n, return true if and only if it is an Armstrong number.
 *
 * The k-digit number n is an Armstrong number if and only if the kth power of each digit sums
 to n.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isArmstrong = function(n) {
  const digits = n.toString();
  const digitCount = digits.length;
  const digitSum = digits
    .split('')
    .reduce((sum, digit) => sum + Math.pow(parseInt(digit, 10), digitCount), 0);

  return digitSum === n;
};
```

1135 Connecting Cities With Minimum Cost

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```javascript
/**
 * 1135. Connecting Cities With Minimum Cost
 * https://leetcode.com/problems/connecting-cities-with-minimum-cost/
 * Difficulty: Medium
 *
 * There are n cities labeled from 1 to n. You are given the integer n and an array
 * connections where connections[i] = [xi, yi, costi] indicates that the cost of connecting
 * city xi and city yi (bidirectional connection) is costi.
 *
 * Return the minimum cost to connect all the n cities such that there is at least one path
 * between each pair of cities. If it is impossible to connect all the n cities, return -1,
 *
 * The cost is the sum of the connections' costs used.
 */

/**
 * @param {number} n
 * @param {number[][]} connections
 * @return {number}
 */
var minimumCost = function(n, connections) {
  const parent = Array.from({ length: n + 1 }, (_, i) => i);

  connections.sort((a, b) => a[2] - b[2]);

  let totalCost = 0;
  let edgesUsed = 0;

  for (const [city1, city2, cost] of connections) {
    if (union(city1, city2)) {
      totalCost += cost;
      edgesUsed++;

      if (edgesUsed === n - 1) {
        return totalCost;
      }
    }
  }

  return -1;
```

```
  function findParent(x) {
    if (parent[x] !== x) {
      parent[x] = findParent(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    const rootX = findParent(x);
    const rootY = findParent(y);

    if (rootX !== rootY) {
      parent[rootX] = rootY;
      return true;
    }
    return false;
  }
};
```

1136 Parallel Courses

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1136. Parallel Courses
 * https://leetcode.com/problems/parallel-courses/
 * Difficulty: Medium
 *
 * You are given an integer n, which indicates that there are n courses labeled from 1 to n.
 * You are also given an array relations where relations[i] = [prevCoursei, nextCoursei],
 * representing a prerequisite relationship between course prevCoursei and course
 * nextCoursei: course prevCoursei has to be taken before course nextCoursei.
 *
 * In one semester, you can take any number of courses as long as you have taken all the
 * prerequisites in the previous semester for the courses you are taking.
 *
 * Return the minimum number of semesters needed to take all courses. If there is no way to
 * take all the courses, return -1.
 */

/**
 * @param {number} n
 * @param {number[][]} relations
 * @return {number}
 */
var minimumSemesters = function(n, relations) {
  const graph = new Map();
  const indegree = new Array(n + 1).fill(0);

  for (let i = 1; i <= n; i++) {
    graph.set(i, []);
  }

  for (const [prev, next] of relations) {
    graph.get(prev).push(next);
    indegree[next]++;
  }

  const queue = [];
  for (let i = 1; i <= n; i++) {
    if (indegree[i] === 0) {
      queue.push(i);
    }
  }

  let semesters = 0;
  let coursesCompleted = 0;

  while (queue.length > 0) {
```

```
    const currentSemesterSize = queue.length;
    semesters++;

    for (let i = 0; i < currentSemesterSize; i++) {
      const course = queue.shift();
      coursesCompleted++;

      for (const nextCourse of graph.get(course)) {
        indegree[nextCourse]--;
        if (indegree[nextCourse] === 0) {
          queue.push(nextCourse);
        }
      }
    }
  }

  return coursesCompleted === n ? semesters : -1;
};
```

1137 N Th Tribonacci Number

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1137. N-th Tribonacci Number
 * https://leetcode.com/problems/n-th-tribonacci-number/
 * Difficulty: Easy
 *
 * The Tribonacci sequence Tn is defined as follows:
 *
 * T0 = 0, T1 = 1, T2 = 1, and Tn+3 = Tn + Tn+1 + Tn+2 for n >= 0.
 *
 * Given n, return the value of Tn.
 */

/**
 * @param {number} n
 * @return {number}
 */
var tribonacci = function(n) {
  const nums = [0, 1, 1];

  for (let i = 3; i <= n; i++) {
    nums.push(nums[i - 3] + nums[i - 2] + nums[i - 1]);
  }

  return nums[n];
};
```

1138 Alphabet Board Path

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1138. Alphabet Board Path
 * https://leetcode.com/problems/alphabet-board-path/
 * Difficulty: Medium
 *
 * On an alphabet board, we start at position (0, 0), corresponding to character board[0][0].
 *
 * Here, board = ["abcde", "fghij", "klmno", "pqrst", "uvwxy", "z"], as shown in the diagram
below.
 *
 * We may make the following moves:
 * - 'U' moves our position up one row, if the position exists on the board;
```

```
 * - 'D' moves our position down one row, if the position exists on the board;
 * - 'L' moves our position left one column, if the position exists on the board;
 * - 'R' moves our position right one column, if the position exists on the board;
 * - '!' adds the character board[r][c] at our current position (r, c) to the answer.
 *
 * (Here, the only positions that exist on the board are positions with letters on them.)
 *
 * Return a sequence of moves that makes our answer equal to target in the minimum number of
 * moves. You may return any path that does so.
 */

/**
 * @param {string} target
 * @return {string}
 */
var alphabetBoardPath = function(target) {
  let path = '';
  let row = 0;
  let col = 0;

  for (const char of target) {
    const code = char.charCodeAt(0) - 97;
    const targetRow = Math.floor(code / 5);
    const targetCol = code % 5;

    while (row > targetRow) path += 'U', row--;
    while (col > targetCol) path += 'L', col--;
    while (row < targetRow) path += 'D', row++;
    while (col < targetCol) path += 'R', col++;

    path += '!';
  }

  return path;
};
```

1139 Largest 1 Bordered Square

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1139. Largest 1-Bordered Square
 * https://leetcode.com/problems/largest-1-bordered-square/
 * Difficulty: Medium
 *
 * Given a 2D grid of 0s and 1s, return the number of elements in the largest square subgrid
 * that has all 1s on its border, or 0 if such a subgrid doesn't exist in the grid.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var largest1BorderedSquare = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const left = Array(rows).fill().map(() => Array(cols).fill(0));
  const top = Array(rows).fill().map(() => Array(cols).fill(0));
  let maxSide = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j]) {
        left[i][j] = j > 0 ? left[i][j - 1] + 1 : 1;
        top[i][j] = i > 0 ? top[i - 1][j] + 1 : 1;

        let side = Math.min(left[i][j], top[i][j]);
        while (side > maxSide) {
          if (left[i - side + 1][j] >= side && top[i][j - side + 1] >= side) {
            maxSide = side;
```

```
            break;
          }
          side--;
        }
      }
    }
  }

  return maxSide * maxSide;
};
```

1140 Stone Game Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1140. Stone Game II
 * https://leetcode.com/problems/stone-game-ii/
 * Difficulty: Medium
 *
 * Alice and Bob continue their games with piles of stones. There are a number of piles
arranged
 * in a row, and each pile has a positive integer number of stones piles[i]. The objective of
 * the game is to end with the most stones.
 *
 * Alice and Bob take turns, with Alice starting first.
 *
 * On each player's turn, that player can take all the stones in the first X remaining piles,
 * where 1 <= X <= 2M. Then, we set M = max(M, X). Initially, M = 1.
 *
 * The game continues until all the stones have been taken.
 *
 * Assuming Alice and Bob play optimally, return the maximum number of stones Alice can get.
 */

/**
 * @param {number[]} piles
 * @return {number}
 */
var stoneGameII = function(piles) {
  const n = piles.length;
  const cache = new Map();
  const suffixSums = new Array(n + 1).fill(0);

  for (let i = n - 1; i >= 0; i--) {
    suffixSums[i] = suffixSums[i + 1] + piles[i];
  }

  return maxStones(0, 1);

  function maxStones(index, m) {
    if (index >= n) return 0;
    if (2 * m >= n - index) return suffixSums[index];

    const key = `${index},${m}`;
    if (cache.has(key)) return cache.get(key);

    let opponentMin = Infinity;
    for (let x = 1; x <= 2 * m; x++) {
      opponentMin = Math.min(opponentMin, maxStones(index + x, Math.max(m, x)));
    }

    const result = suffixSums[index] - opponentMin;
    cache.set(key, result);
    return result;
  }
};
```

1143 Longest Common Subsequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1143. Longest Common Subsequence
 * https://leetcode.com/problems/longest-common-subsequence/
 * Difficulty: Medium
 *
 * Given two strings text1 and text2, return the length of their longest common subsequence.
 * If there is no common subsequence, return 0.
 *
 * A subsequence of a string is a new string generated from the original string with some
 * characters (can be none) deleted without changing the relative order of the remaining
 * characters.
 *
 * - For example, "ace" is a subsequence of "abcde".
 *
 * A common subsequence of two strings is a subsequence that is common to both strings.
 */

/**
 * @param {string} text1
 * @param {string} text2
 * @return {number}
 */
var longestCommonSubsequence = function(text1, text2) {
  const dp = new Array(text1.length + 1).fill(0).map(() => new Array(text2.length +
1).fill(0));

  for (let i = 1; i <= text1.length; i++) {
    for (let j = 1; j <= text2.length; j++) {
      if (text1.charAt(i - 1) === text2.charAt(j - 1)) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }

  return dp[text1.length][text2.length];
};
```

1144 Decrease Elements To Make Array Zigzag

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1144. Decrease Elements To Make Array Zigzag
 * https://leetcode.com/problems/decrease-elements-to-make-array-zigzag/
 * Difficulty: Medium
 *
 * Given an array nums of integers, a move consists of choosing any element and decreasing it
by 1.
 *
 * An array A is a zigzag array if either:
 * - Every even-indexed element is greater than adjacent elements, ie.
 *   A[0] > A[1] < A[2] > A[3] < A[4] > ...
 * - OR, every odd-indexed element is greater than adjacent elements, ie.
 *   A[0] < A[1] > A[2] < A[3] > A[4] < ...
 *
 * Return the minimum number of moves to transform the given array nums into a zigzag array.
 */

/**
```

```
 * @param {number[]} nums
 * @return {number}
 */
var movesToMakeZigzag = function(nums) {
  function calculateMoves(peakEven) {
    let moves = 0;
    const adjusted = [...nums];

    for (let i = 0; i < adjusted.length; i++) {
      if ((i % 2 === 0) === peakEven) {
        const left = i > 0 ? adjusted[i - 1] : Infinity;
        const right = i < adjusted.length - 1 ? adjusted[i + 1] : Infinity;
        const target = Math.min(left, right) - 1;
        if (adjusted[i] >= target) {
          moves += adjusted[i] - target;
          adjusted[i] = target;
        }
      }
    }

    return moves;
  }

  const evenPeaks = calculateMoves(true);
  const oddPeaks = calculateMoves(false);

  return Math.min(evenPeaks, oddPeaks);
};
```

1145 Binary Tree Coloring Game

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1145. Binary Tree Coloring Game
 * https://leetcode.com/problems/binary-tree-coloring-game/
 * Difficulty: Medium
 *
 * Two players play a turn based game on a binary tree. We are given the root of this binary
tree,
 * and the number of nodes n in the tree. n is odd, and each node has a distinct value from 1
to n.
 *
 * Initially, the first player names a value x with 1 <= x <= n, and the second player names a
 value
 * y with 1 <= y <= n and y != x. The first player colors the node with value x red, and the
second
 * player colors the node with value y blue.
 *
 * Then, the players take turns starting with the first player. In each turn, that player
chooses
 * a node of their color (red if player 1, blue if player 2) and colors an uncolored neighbor
of
 * the chosen node (either the left child, right child, or parent of the chosen node.)
 *
 * If (and only if) a player cannot choose such a node in this way, they must pass their turn.
 If
 * both players pass their turn, the game ends, and the winner is the player that colored more
 * nodes.
 *
 * You are the second player. If it is possible to choose such a y to ensure you win the game,
 * return true. If it is not possible, return false.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
```

```
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} n
 * @param {number} x
 * @return {boolean}
 */
var btreeGameWinningMove = function(root, n, x) {
  let leftCount = 0;
  let rightCount = 0;

  countNodes(root);

  const parentCount = n - leftCount - rightCount - 1;
  const maxRegion = Math.max(parentCount, leftCount, rightCount);

  return maxRegion > n / 2;

  function countNodes(node) {
    if (!node) return 0;
    const left = countNodes(node.left);
    const right = countNodes(node.right);
    if (node.val === x) {
      leftCount = left;
      rightCount = right;
    }
    return left + right + 1;
  }
};
```

1146 Snapshot Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1146. Snapshot Array
 * https://leetcode.com/problems/snapshot-array/
 * Difficulty: Medium
 *
 * Implement a SnapshotArray that supports the following interface:
 * - SnapshotArray(int length) initializes an array-like data structure with the given length.
 *   Initially, each element equals 0.
 * - void set(index, val) sets the element at the given index to be equal to val.
 * - int snap() takes a snapshot of the array and returns the snapId: the total number of
times
 *   we called snap() minus 1.
 * - int get(index, snapId) returns the value at the given index, at the time we took the
 *   snapshot with the given snapId
 */

/**
 * @param {number} length
 */
var SnapshotArray = function(length) {
  this.history = new Map();
  this.snapshots = 0;
  this.length = length;
};

/**
 * @param {number} index
 * @param {number} val
 * @return {void}
 */
SnapshotArray.prototype.set = function(index, val) {
  const current = this.history.get(this.snapshots) || new Map();
  current.set(index, val);
  this.history.set(this.snapshots, current);
};
```

```
/**
 * @return {number}
 */
SnapshotArray.prototype.snap = function() {
  return this.snapshots++;
};

/**
 * @param {number} index
 * @param {number} snapId
 * @return {number}
 */
SnapshotArray.prototype.get = function(index, snapId) {
  for (let id = snapId; id >= 0; id--) {
    const snapshot = this.history.get(id);
    if (snapshot && snapshot.has(index)) {
      return snapshot.get(index);
    }
  }
  return 0;
};
```

1147 Longest Chunked Palindrome Decomposition

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1147. Longest Chunked Palindrome Decomposition
 * https://leetcode.com/problems/longest-chunked-palindrome-decomposition/
 * Difficulty: Hard
 *
 * You are given a string text. You should split it to k substrings (subtext1,
 * subtext2, ..., subtextk) such that:
 * - subtexti is a non-empty string.
 * - The concatenation of all the substrings is equal to text (i.e., subtext1 + subtext2
 *   + ... + subtextk == text).
 * - subtexti == subtextk - i + 1 for all valid values of i (i.e., 1 <= i <= k).
 *
 * Return the largest possible value of k.
 */

/**
 * @param {string} text
 * @return {number}
 */
var longestDecomposition = function(text) {
  return decompose(0, text.length - 1);

  function decompose(start, end) {
    if (start > end) return 0;
    if (start === end) return 1;

    const left = start;
    const right = end;
    let chunkSize = 0;

    while (left + chunkSize < right - chunkSize) {
      chunkSize++;
      if (text.slice(left, left + chunkSize) === text.slice(right - chunkSize + 1, right + 1))
 {
        return 2 + decompose(left + chunkSize, right - chunkSize);
      }
    }

    return 1;
  }
};
```

1150 Check If A Number Is Majority Element In A Sorted Array

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1150. Check If a Number Is Majority Element in a Sorted Array
 * https://leetcode.com/problems/check-if-a-number-is-majority-element-in-a-sorted-array/
 * Difficulty: Easy
 *
 * Given an integer array nums sorted in non-decreasing order and an integer target, return
true
 * if target is a majority element, or false otherwise.
 *
 * A majority element in an array nums is an element that appears more than nums.length / 2
times
 * in the array.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {boolean}
 */
var isMajorityElement = function(nums, target) {
  const firstIndex = findFirstIndex(target);
  if (firstIndex >= nums.length || nums[firstIndex] !== target) {
    return false;
  }

  const lastIndex = findLastIndex(target);
  const frequency = lastIndex - firstIndex + 1;

  return frequency > nums.length / 2;

  function findFirstIndex(target) {
    let left = 0;
    let right = nums.length;
    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (nums[mid] < target) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }
    return left;
  }

  function findLastIndex(target) {
    let left = 0;
    let right = nums.length;
    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (nums[mid] <= target) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }
    return left - 1;
  }
};
```

1151 Minimum Swaps To Group All 1S Together

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1151. Minimum Swaps to Group All 1's Together
 * https://leetcode.com/problems/minimum-swaps-to-group-all-1s-together/
 * Difficulty: Medium
 *
 * Given a binary array data, return the minimum number of swaps required to group all
 * 1’s present in the array together in any place in the array.
 */

/**
 * @param {number[]} data
 * @return {number}
 */
var minSwaps = function(data) {
  const totalOnes = data.reduce((sum, val) => sum + val, 0);

  if (totalOnes <= 1) return 0;

  let currentOnes = 0;
  for (let i = 0; i < totalOnes; i++) {
    currentOnes += data[i];
  }

  let maxOnes = currentOnes;

  for (let i = totalOnes; i < data.length; i++) {
    currentOnes = currentOnes - data[i - totalOnes] + data[i];
    maxOnes = Math.max(maxOnes, currentOnes);
  }

  return totalOnes - maxOnes;
};
```

1152 Analyze User Website Visit Pattern

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1152. Analyze User Website Visit Pattern
 * https://leetcode.com/problems/analyze-user-website-visit-pattern/
 * Difficulty: Medium
 *
 * You are given two string arrays username and website and an integer array timestamp.
 * All the given arrays are of the same length and the tuple [username[i], website[i],
 * timestamp[i]] indicates that the user username[i] visited the website website[i] at
 * time timestamp[i].
 *
 * A pattern is a list of three websites (not necessarily distinct).
 * - For example, ["home", "away", "love"], ["leetcode", "love", "leetcode"],
 *   and ["luffy", "luffy", "luffy"] are all patterns.
 *
 * The score of a pattern is the number of users that visited all the websites in the
 * pattern in the same order they appeared in the pattern.
 * - For example, if the pattern is ["home", "away", "love"], the score is the number of
 *   users x such that x visited "home" then visited "away" and visited "love" after that.
 * - Similarly, if the pattern is ["leetcode", "love", "leetcode"], the score is the number
 *   of users x such that x visited "leetcode" then visited "love" and visited "leetcode"
 *   one more time after that.
 * - Also, if the pattern is ["luffy", "luffy", "luffy"], the score is the number of users
 *   x such that x visited "luffy" three different times at different timestamps.
 *
 * Return the pattern with the largest score. If there is more than one pattern with the
 * same largest score, return the lexicographically smallest such pattern.
 *
 * Note that the websites in a pattern do not need to be visited contiguously, they only
 * need to be visited in the order they appeared in the pattern.
 */
```

```
/**
 * @param {string[]} username
 * @param {number[]} timestamp
 * @param {string[]} website
 * @return {string[]}
 */
var mostVisitedPattern = function(username, timestamp, website) {
  const visitData = username.map((user, i) => ({
    user,
    time: timestamp[i],
    site: website[i]
  }));

  visitData.sort((a, b) => a.time - b.time);

  const userVisits = new Map();

  for (const visit of visitData) {
    if (!userVisits.has(visit.user)) {
      userVisits.set(visit.user, []);
    }
    userVisits.get(visit.user).push(visit.site);
  }

  const patternCounts = new Map();

  for (const [user, sites] of userVisits) {
    if (sites.length < 3) continue;

    const userPatterns = new Set();

    for (let i = 0; i < sites.length - 2; i++) {
      for (let j = i + 1; j < sites.length - 1; j++) {
        for (let k = j + 1; k < sites.length; k++) {
          const pattern = [sites[i], sites[j], sites[k]].join(',');
          userPatterns.add(pattern);
        }
      }
    }

    for (const pattern of userPatterns) {
      patternCounts.set(pattern, (patternCounts.get(pattern) || 0) + 1);
    }
  }

  let maxCount = 0;
  let bestPattern = '';

  for (const [pattern, count] of patternCounts) {
    if (count > maxCount || (count === maxCount && pattern < bestPattern)) {
      maxCount = count;
      bestPattern = pattern;
    }
  }

  return bestPattern.split(',');
};
```

1153 String Transforms Into Another String

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1153. String Transforms Into Another String
 * https://leetcode.com/problems/string-transforms-into-another-string/
 * Difficulty: Hard
 *
 * Given two strings str1 and str2 of the same length, determine whether you can transform
 * str1 into str2 by doing zero or more conversions.
```

```
 *
 * In one conversion you can convert all occurrences of one character in str1 to any other
 * lowercase English character.
 *
 * Return true if and only if you can transform str1 into str2.
 */

/**
 * @param {string} str1
 * @param {string} str2
 * @return {boolean}
 */
var canConvert = function(str1, str2) {
  if (str1 === str2) return true;

  const mapping = new Map();
  const uniqueCharsInStr2 = new Set();
  for (let i = 0; i < str1.length; i++) {
    const char1 = str1[i];
    const char2 = str2[i];

    if (mapping.has(char1)) {
      if (mapping.get(char1) !== char2) {
        return false;
      }
    } else {
      mapping.set(char1, char2);
    }

    uniqueCharsInStr2.add(char2);
  }

  return uniqueCharsInStr2.size < 26;
};
```

1154 Day Of The Year

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1154. Day of the Year
 * https://leetcode.com/problems/day-of-the-year/
 * Difficulty: Easy
 *
 * Given a string date representing a Gregorian calendar date formatted as YYYY-MM-DD,
 * return the day number of the year.
 */

/**
 * @param {string} date
 * @return {number}
 */
var dayOfYear = function(date) {
  const daysInMonth = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];
  const [year, month, day] = date.split('-').map(Number);
  const isLeapYear = year % 4 === 0 && (year % 100 !== 0 || year % 400 === 0);
  if (isLeapYear) daysInMonth[1] = 29;

  let result = day;
  for (let i = 0; i < month - 1; i++) {
    result += daysInMonth[i];
  }

  return result;
};
```

1155 Number Of Dice Rolls With Target Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1155. Number of Dice Rolls With Target Sum
 * https://leetcode.com/problems/number-of-dice-rolls-with-target-sum/
 * Difficulty: Medium
 *
 * You have n dice, and each dice has k faces numbered from 1 to k.
 *
 * Given three integers n, k, and target, return the number of possible ways (out of the kn
total
 * ways) to roll the dice, so the sum of the face-up numbers equals target. Since the answer
may
 * be too large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} k
 * @param {number} target
 * @return {number}
 */
var numRollsToTarget = function(diceCount, faceCount, targetSum) {
  const MOD = 1e9 + 7;
  let dp = new Array(targetSum + 1).fill(0);
  dp[0] = 1;

  for (let dice = 1; dice <= diceCount; dice++) {
    const nextDp = new Array(targetSum + 1).fill(0);
    for (let sum = 1; sum <= targetSum; sum++) {
      for (let face = 1; face <= faceCount && face <= sum; face++) {
        nextDp[sum] = (nextDp[sum] + dp[sum - face]) % MOD;
      }
    }
    dp = nextDp;
  }

  return dp[targetSum];
};
```

1156 Swap For Longest Repeated Character Substring

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1156. Swap For Longest Repeated Character Substring
 * https://leetcode.com/problems/swap-for-longest-repeated-character-substring/
 * Difficulty: Medium
 *
 * You are given a string text. You can swap two of the characters in the text.
 *
 * Return the length of the longest substring with repeated characters.
 */

/**
 * @param {string} text
 * @return {number}
 */
var maxRepOpt1 = function(text) {
  const map = new Map();
  for (const char of text) {
```

```
    map.set(char, (map.get(char) || 0) + 1);
  }

  let result = 0;
  for (let i = 0; i < text.length;) {
    const currentChar = text[i];
    const segmentStart = i;

    while (i < text.length && text[i] === currentChar) i++;
    const segmentLength = i - segmentStart;

    const totalAvailable = map.get(currentChar);
    if (i < text.length && segmentLength < totalAvailable) {
      const nextStart = i + 1;
      let nextLength = 0;
      while (nextStart + nextLength < text.length && text[nextStart + nextLength] ===
currentChar) {
        nextLength++;
      }
      result = Math.max(result, Math.min(segmentLength + nextLength + 1, totalAvailable));
    }
    result = Math.max(result, Math.min(
      segmentLength + (totalAvailable > segmentLength ? 1 : 0), totalAvailable
    ));
  }

  return result;
};
```

1157 Online Majority Element In Subarray

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1157. Online Majority Element In Subarray
 * https://leetcode.com/problems/online-majority-element-in-subarray/
 * Difficulty: Hard
 *
 * Design a data structure that efficiently finds the majority element of a given subarray.
 *
 * The majority element of a subarray is an element that occurs threshold times or more in
 * the subarray.
 *
 * Implementing the MajorityChecker class:
 * - MajorityChecker(int[] arr) Initializes the instance of the class with the given array
arr.
 * - int query(int left, int right, int threshold) returns the element in the subarray
 *   arr[left...right] that occurs at least threshold times, or -1 if no such element exists.
 */

/**
 * @param {number[]} arr
 */
var MajorityChecker = function(arr) {
  this.positionMap = new Map();
  this.array = arr;
  for (let i = 0; i < arr.length; i++) {
    const positions = this.positionMap.get(arr[i]) || [];
    positions.push(i);
    this.positionMap.set(arr[i], positions);
  }
};

/**
 * @param {number} left
 * @param {number} right
 * @param {number} threshold
 * @return {number}
 */
```

```
MajorityChecker.prototype.query = function(left, right, threshold) {
  const maxAttempts = 20;
  const rangeLength = right - left + 1;

  for (let attempt = 0; attempt < maxAttempts; attempt++) {
    const candidate = this.array[left + Math.floor(Math.random() * rangeLength)];
    const positions = this.positionMap.get(candidate);
    const count = countInRange(positions, left, right);

    if (count >= threshold) return candidate;
    if (count + rangeLength - count < threshold) return -1;
  }
  return -1;
};

function countInRange(positions, left, right) {
  const leftIndex = lowerBound(positions, left);
  const rightIndex = upperBound(positions, right);
  return rightIndex - leftIndex;
}

function lowerBound(arr, target) {
  let start = 0;
  let end = arr.length;
  while (start < end) {
    const mid = Math.floor((start + end) / 2);
    if (arr[mid] < target) start = mid + 1;
    else end = mid;
  }
  return start;
}

function upperBound(arr, target) {
  let start = 0;
  let end = arr.length;
  while (start < end) {
    const mid = Math.floor((start + end) / 2);
    if (arr[mid] <= target) start = mid + 1;
    else end = mid;
  }
  return start;
}
```

1160 Find Words That Can Be Formed By Characters

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1160. Find Words That Can Be Formed by Characters
 * https://leetcode.com/problems/find-words-that-can-be-formed-by-characters/
 * Difficulty: Easy
 *
 * You are given an array of strings words and a string chars.
 *
 * A string is good if it can be formed by characters from chars (each character can only be
 * used once).
 *
 * Return the sum of lengths of all good strings in words.
 */

/**
 * @param {string[]} words
 * @param {string} chars
 * @return {number}
 */
var countCharacters = function(words, availableChars) {
  const charFrequency = new Map();
  for (const char of availableChars) {
    charFrequency.set(char, (charFrequency.get(char) || 0) + 1);
  }
```

```
  let result = 0;
  for (const word of words) {
    const tempFrequency = new Map(charFrequency);
    let isValid = true;

    for (const char of word) {
      if (!tempFrequency.has(char) || tempFrequency.get(char) === 0) {
        isValid = false;
        break;
      }
      tempFrequency.set(char, tempFrequency.get(char) - 1);
    }

    if (isValid) result += word.length;
  }

  return result;
};
```

1161 Maximum Level Sum Of A Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1161. Maximum Level Sum of a Binary Tree
 * https://leetcode.com/problems/maximum-level-sum-of-a-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, the level of its root is 1, the level of its
 * children is 2, and so on.
 *
 * Return the smallest level x such that the sum of all the values of nodes at
 * level x is maximal.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxLevelSum = function(root) {
  const sums = [-Infinity];

  traverse(root, 1);

  return sums.indexOf(Math.max(...sums));

  function traverse(node, depth) {
    if (!node) return;
    if (sums[depth] === undefined) {
      sums.push(node.val);
    } else {
      sums[depth] += node.val;
    }
    traverse(node.left, depth + 1);
    traverse(node.right, depth + 1);
  }
};
```

1162 As Far From Land As Possible

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1162. As Far from Land as Possible
 * https://leetcode.com/problems/as-far-from-land-as-possible/
 * Difficulty: Medium
 *
 * Given an n x n grid containing only values 0 and 1, where 0 represents water and 1
represents
 * land, find a water cell such that its distance to the nearest land cell is maximized, and
return
 * the distance. If no land or water exists in the grid, return -1.
 *
 * The distance used in this problem is the Manhattan distance: the distance between two cells
 * (x0, y0) and (x1, y1) is |x0 - x1| + |y0 - y1|.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maxDistance = function(grid) {
  const size = grid.length;
  const queue = [];
  let waterCount = 0;

  for (let row = 0; row < size; row++) {
    for (let col = 0; col < size; col++) {
      if (grid[row][col] === 1) {
        queue.push([row, col]);
      } else {
        waterCount++;
      }
    }
  }

  if (waterCount === 0 || queue.length === 0) return -1;

  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  let distance = -1;

  while (queue.length > 0) {
    distance++;
    const levelSize = queue.length;
    for (let i = 0; i < levelSize; i++) {
      const [currentRow, currentCol] = queue.shift();
      for (const [deltaRow, deltaCol] of directions) {
        const newRow = currentRow + deltaRow;
        const newCol = currentCol + deltaCol;
        if (newRow >= 0 && newRow < size && newCol >= 0 && newCol < size
            && grid[newRow][newCol] === 0) {
          grid[newRow][newCol] = 1;
          queue.push([newRow, newCol]);
        }
      }
    }
  }

  return distance === 0 ? -1 : distance;
};
```

1163 Last Substring In Lexicographical Order

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1163. Last Substring in Lexicographical Order
 * https://leetcode.com/problems/last-substring-in-lexicographical-order/
 * Difficulty: Hard
 *
 * Given a string s, return the last substring of s in lexicographical order.
 */

/**
 * @param {string} s
 * @return {string}
 */
var lastSubstring = function(s) {
  let i = 0;
  let j = 1;
  let k = 0;
  const n = s.length;

  while (j + k < n) {
    if (s[i + k] === s[j + k]) {
      k++;
    } else if (s[i + k] < s[j + k]) {
      i = Math.max(i + k + 1, j);
      j = i + 1;
      k = 0;
    } else {
      j += k + 1;
      k = 0;
    }
  }

  return s.substring(i);
};
```

1165 Single Row Keyboard

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1165. Single-Row Keyboard
 * https://leetcode.com/problems/single-row-keyboard/
 * Difficulty: Easy
 *
 * There is a special keyboard with all keys in a single row.
 *
 * Given a string keyboard of length 26 indicating the layout of the keyboard (indexed
 * from 0 to 25). Initially, your finger is at index 0. To type a character, you have
 * to move your finger to the index of the desired character. The time taken to move
 * your finger from index i to index j is |i - j|.
 *
 * You want to type a string word. Write a function to calculate how much time it takes
 * to type it with one finger.
 */

/**
 * @param {string} keyboard
 * @param {string} word
 * @return {number}
 */
var calculateTime = function(keyboard, word) {
  const map = new Map();

  for (let i = 0; i < keyboard.length; i++) {
    map.set(keyboard[i], i);
  }

  let currentPosition = 0;
```

```
  let result = 0;

  for (const char of word) {
    const targetPosition = map.get(char);
    result += Math.abs(currentPosition - targetPosition);
    currentPosition = targetPosition;
  }

  return result;
};
```

1166 Design File System

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1166. Design File System
 * https://leetcode.com/problems/design-file-system/
 * Difficulty: Medium
 *
 * You are asked to design a file system that allows you to create new paths and associate
 * them with different values.
 *
 * The format of a path is one or more concatenated strings of the form: / followed by one
 * or more lowercase English letters. For example, "/leetcode" and "/leetcode/problems" are
 * valid paths while an empty string "" and "/" are not.
 *
 * Implement the FileSystem class:
 * - bool createPath(string path, int value) Creates a new path and associates a value to it
 *   if possible and returns true. Returns false if the path already exists or its parent path
 *   doesn't exist.
 * - int get(string path) Returns the value associated with path or returns -1 if the path
 *   doesn't exist.
 */

var FileSystem = function() {
  this.pathMap = new Map();
};

/**
 * @param {string} path
 * @param {number} value
 * @return {boolean}
 */
FileSystem.prototype.createPath = function(path, value) {
  if (this.pathMap.has(path)) {
    return false;
  }

  const lastSlashIndex = path.lastIndexOf('/');
  const parentPath = path.substring(0, lastSlashIndex);

  if (parentPath.length > 0 && !this.pathMap.has(parentPath)) {
    return false;
  }

  this.pathMap.set(path, value);
  return true;
};

/**
 * @param {string} path
 * @return {number}
 */
FileSystem.prototype.get = function(path) {
  return this.pathMap.get(path) ?? -1;
};
```

1167 Minimum Cost To Connect Sticks

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1167. Minimum Cost to Connect Sticks
 * https://leetcode.com/problems/minimum-cost-to-connect-sticks/
 * Difficulty: Medium
 *
 * You have some number of sticks with positive integer lengths. These lengths are given as
 * an array sticks, where sticks[i] is the length of the ith stick.
 *
 * You can connect any two sticks of lengths x and y into one stick by paying a cost of x + y.
 * You must connect all the sticks until there is only one stick remaining.
 *
 * Return the minimum cost of connecting all the given sticks into one stick in this way.
 */

/**
 * @param {number[]} sticks
 * @return {number}
 */
var connectSticks = function(sticks) {
  const minHeap = [...sticks].sort((a, b) => a - b);
  let result = 0;

  while (minHeap.length > 1) {
    const first = minHeap.shift();
    const second = minHeap.shift();
    const combinedCost = first + second;

    result += combinedCost;

    let insertIndex = 0;
    while (insertIndex < minHeap.length && minHeap[insertIndex] < combinedCost) {
      insertIndex++;
    }
    minHeap.splice(insertIndex, 0, combinedCost);
  }

  return result;
};
```

1168 Optimize Water Distribution In A Village

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1168. Optimize Water Distribution in a Village
 * https://leetcode.com/problems/optimize-water-distribution-in-a-village/
 * Difficulty: Hard
 *
 * There are n houses in a village. We want to supply water for all the houses by building
 * wells and laying pipes.
 *
 * For each house i, we can either build a well inside it directly with cost wells[i - 1]
 * (note the -1 due to 0-indexing), or pipe in water from another well to it. The costs
 * to lay pipes between houses are given by the array pipes where each
 * pipes[j] = [house1j, house2j, costj] represents the cost to connect house1j and house2j
 * together using a pipe. Connections are bidirectional, and there could be multiple valid
 * connections between the same two houses with different costs.
 *
 * Return the minimum total cost to supply water to all houses.
 */
```

```
/**
 * @param {number} n
 * @param {number[]} wells
 * @param {number[][]} pipes
 * @return {number}
 */
var minCostToSupplyWater = function(n, wells, pipes) {
  const edges = [];

  for (let i = 0; i < n; i++) {
    edges.push([0, i + 1, wells[i]]);
  }

  for (const [house1, house2, cost] of pipes) {
    edges.push([house1, house2, cost]);
  }

  edges.sort((a, b) => a[2] - b[2]);

  const parent = Array.from({ length: n + 1 }, (_, i) => i);
  let result = 0;
  let edgesUsed = 0;

  for (const [from, to, cost] of edges) {
    if (union(from, to)) {
      result += cost;
      edgesUsed++;
      if (edgesUsed === n) break;
    }
  }

  return result;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    const rootX = find(x);
    const rootY = find(y);
    if (rootX !== rootY) {
      parent[rootX] = rootY;
      return true;
    }
    return false;
  }
};
```

1169 Invalid Transactions

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1169. Invalid Transactions
 * https://leetcode.com/problems/invalid-transactions/
 * Difficulty: Medium
 *
 * A transaction is possibly invalid if:
 * - the amount exceeds $1000, or;
 * - if it occurs within (and including) 60 minutes of another transaction with the same name
in
 *   a different city.
 *
 * You are given an array of strings transaction where transactions[i] consists of
comma-separated
 * values representing the name, time (in minutes), amount, and city of the transaction.
 *
```

```
 * Return a list of transactions that are possibly invalid. You may return the answer in any
order.
 */

/**
 * @param {string[]} transactions
 * @return {string[]}
 */
var invalidTransactions = function(transactions) {
  const parsed = transactions.map(t => {
    const [name, time, amount, city] = t.split(',');
    return { name, time: Number(time), amount: Number(amount), city };
  });

  const invalid = new Set();

  for (let i = 0; i < parsed.length; i++) {
    const current = parsed[i];
    if (current.amount > 1000) {
      invalid.add(i);
    }

    for (let j = 0; j < parsed.length; j++) {
      const other = parsed[j];
      if (i !== j && current.name === other.name
          && Math.abs(current.time - other.time) <= 60 && current.city !== other.city) {
        invalid.add(i);
        invalid.add(j);
      }
    }
  }

  return [...invalid].map(index => transactions[index]);
};
```

1170 Compare Strings By Frequency Of The Smallest Character

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1170. Compare Strings by Frequency of the Smallest Character
 * https://leetcode.com/problems/compare-strings-by-frequency-of-the-smallest-character/
 * Difficulty: Medium
 *
 * Let the function f(s) be the frequency of the lexicographically smallest character in a
non-empty
 * string s. For example, if s = "dcce" then f(s) = 2 because the lexicographically smallest
 * character is 'c', which has a frequency of 2.
 *
 * You are given an array of strings words and another array of query strings queries. For
each
 * query queries[i], count the number of words in words such that f(queries[i]) < f(W) for
each
 * W in words.
 *
 * Return an integer array answer, where each answer[i] is the answer to the ith query.
 */

/**
 * @param {string[]} queries
 * @param {string[]} words
 * @return {number[]}
 */
var numSmallerByFrequency = function(queries, words) {
  const wordFrequencies = words.map(getMinCharFreq).sort((a, b) => a - b);

  return queries.map(query => {
    const queryFreq = getMinCharFreq(query);
    let left = 0;
```

```
    let right = wordFrequencies.length;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (wordFrequencies[mid] <= queryFreq) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }

    return wordFrequencies.length - left;
  });

  function getMinCharFreq(str) {
    let minChar = 'z';
    let freq = 0;
    for (const char of str) {
      if (char < minChar) {
        minChar = char;
        freq = 1;
      } else if (char === minChar) {
        freq++;
      }
    }
    return freq;
  }
};
```

1171 Remove Zero Sum Consecutive Nodes From Linked List

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1171. Remove Zero Sum Consecutive Nodes from Linked List
 * https://leetcode.com/problems/remove-zero-sum-consecutive-nodes-from-linked-list/
 * Difficulty: Medium
 *
 * Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that
sum to
 * 0 until there are no such sequences.
 *
 * After doing so, return the head of the final linked list.  You may return any such answer.
 *
 * (Note that in the examples below, all sequences are serializations of ListNode objects.)
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var removeZeroSumSublists = function(head) {
  const node = new ListNode(0, head);
  const map = new Map();
  let prefixSum = 0;

  let current = node;
  while (current) {
    prefixSum += current.val;
    map.set(prefixSum, current);
    current = current.next;
  }
```

```
  prefixSum = 0;
  current = node;
  while (current) {
    prefixSum += current.val;
    if (map.has(prefixSum) && map.get(prefixSum) !== current) {
      current.next = map.get(prefixSum).next;
    }
    current = current.next;
  }

  return node.next;
};
```

1172 Dinner Plate Stacks

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1172. Dinner Plate Stacks
 * https://leetcode.com/problems/dinner-plate-stacks/
 * Difficulty: Hard
 *
 * You have an infinite number of stacks arranged in a row and numbered (left to right) from
0,
 * each of the stacks has the same maximum capacity.
 *
 * Implement the DinnerPlates class:
 * - DinnerPlates(int capacity) Initializes the object with the maximum capacity of the stacks
 *   capacity.
 * - void push(int val) Pushes the given integer val into the leftmost stack with a size less
 *   than capacity.
 * - int pop() Returns the value at the top of the rightmost non-empty stack and removes it
 *   from that stack, and returns -1 if all the stacks are empty.
 * - int popAtStack(int index) Returns the value at the top of the stack with the given index
 *   index and removes it from that stack or returns -1 if the stack with that given index is
empty.
 */

/**
 * @param {number} capacity
 */
var DinnerPlates = function(capacity) {
  this.maxCapacity = capacity;
  this.plateStacks = [];
  this.availableIndices = [];
};

/**
 * @param {number} val
 * @return {void}
 */
DinnerPlates.prototype.push = function(val) {
  const targetIndex = this.availableIndices.length
    ? this.availableIndices.pop()
    : this.plateStacks.length - 1;

  if (!this.plateStacks[targetIndex] || this.plateStacks[targetIndex].length ===
this.maxCapacity) {
    this.plateStacks.push([val]);
  } else {
    this.plateStacks[targetIndex].push(val);
  }
};

/**
 * @return {number}
 */
DinnerPlates.prototype.pop = function() {
  while (this.plateStacks.length && !this.plateStacks.at(-1).length) {
    while (this.plateStacks.length - 1 === this.availableIndices.at(-1)) {
```

```
      this.availableIndices.pop();
    }
    this.plateStacks.pop();
  }

  if (!this.plateStacks.length) {
    return -1;
  }

  const result = this.plateStacks.at(-1).pop() ?? -1;

  while (this.plateStacks.length && !this.plateStacks.at(-1).length) {
    while (this.plateStacks.length - 1 === this.availableIndices.at(-1)) {
      this.availableIndices.pop();
    }
    this.plateStacks.pop();
  }

  return result;
};

/**
 * @param {number} index
 * @return {number}
 */
DinnerPlates.prototype.popAtStack = function(index) {
  if (index >= this.plateStacks.length || !this.plateStacks[index].length) {
    return -1;
  }

  const result = this.plateStacks[index].pop();
  const temp = [];

  while (this.availableIndices.length && this.availableIndices.at(-1) < index) {
    temp.push(this.availableIndices.pop());
  }

  this.availableIndices.push(index);
  while (temp.length) {
    this.availableIndices.push(temp.shift());
  }

  return result;
};
```

1175 Prime Arrangements

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1175. Prime Arrangements
 * https://leetcode.com/problems/prime-arrangements/
 * Difficulty: Easy
 *
 * Return the number of permutations of 1 to n so that prime numbers are at prime
 * indices (1-indexed.)
 *
 * (Recall that an integer is prime if and only if it is greater than 1, and cannot be written
 * as a product of two positive integers both smaller than it.)
 *
 * Since the answer may be large, return the answer modulo 10^9 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
var numPrimeArrangements = function(n) {
  let primeCount = 0;
  for (let i = 1; i <= n; i++) {
```

```
    if (isPrime(i)) primeCount++;
  }

  const MOD = 1000000007n;
  const nonPrimeCount = n - primeCount;
  const primePermutations = factorial(primeCount);
  const nonPrimePermutations = factorial(nonPrimeCount);

  return Number((primePermutations * nonPrimePermutations) % MOD);

  function isPrime(n) {
    if (n < 2) return false;
    for (let i = 2; i * i <= n; i++) {
      if (n % i === 0) return false;
    }
    return true;
  }

  function factorial(n) {
    let product = 1n;
    for (let i = 2; i <= n; i++) {
      product *= BigInt(i);
    }
    return product;
  }
};
```

1176 Diet Plan Performance

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1176. Diet Plan Performance
 * https://leetcode.com/problems/diet-plan-performance/
 * Difficulty: Easy
 *
 * A dieter consumes calories[i] calories on the i-th day.
 *
 * Given an integer k, for every consecutive sequence of k days
 * (calories[i], calories[i+1], ..., calories[i+k-1] for all 0 <= i <= n-k),
 * they look at T, the total calories consumed during that sequence of k days
 * (calories[i] + calories[i+1] + ... + calories[i+k-1]):
 * - If T < lower, they performed poorly on their diet and lose 1 point;
 * - If T > upper, they performed well on their diet and gain 1 point;
 * - Otherwise, they performed normally and there is no change in points.
 *
 * Initially, the dieter has zero points. Return the total number of points the dieter
 * has after dieting for calories.length days.
 *
 * Note that the total points can be negative.
 */

/**
 * @param {number[]} calories
 * @param {number} k
 * @param {number} lower
 * @param {number} upper
 * @return {number}
 */
var dietPlanPerformance = function(calories, k, lower, upper) {
  let windowSum = 0;
  let points = 0;

  for (let i = 0; i < k; i++) {
    windowSum += calories[i];
  }

  if (windowSum < lower) points--;
  else if (windowSum > upper) points++;
```

```
  for (let i = k; i < calories.length; i++) {
    windowSum = windowSum - calories[i - k] + calories[i];

    if (windowSum < lower) points--;
    else if (windowSum > upper) points++;
  }

  return points;
};
```

1177 Can Make Palindrome From Substring

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1177. Can Make Palindrome from Substring
 * https://leetcode.com/problems/can-make-palindrome-from-substring/
 * Difficulty: Medium
 *
 * You are given a string s and array queries where queries[i] = [lefti, righti, ki]. We may
 * rearrange the substring s[lefti...righti] for each query and then choose up to ki of them
 * to replace with any lowercase English letter.
 *
 * If the substring is possible to be a palindrome string after the operations above, the
 * result of the query is true. Otherwise, the result is false.
 *
 * Return a boolean array answer where answer[i] is the result of the ith query queries[i].
 *
 * Note that each letter is counted individually for replacement, so if, for example
 * s[lefti...righti] = "aaa", and ki = 2, we can only replace two of the letters. Also, note
 * that no query modifies the initial string s.
 */

/**
 * @param {string} s
 * @param {number[][]} queries
 * @return {boolean[]}
 */
var canMakePaliQueries = function(s, queries) {
  const prefixCounts = new Array(s.length + 1).fill().map(() => new Array(26).fill(0));

  for (let i = 0; i < s.length; i++) {
    prefixCounts[i + 1] = [...prefixCounts[i]];
    prefixCounts[i + 1][s.charCodeAt(i) - 97]++;
  }

  const getOddCount = (left, right) => {
    let odds = 0;
    for (let j = 0; j < 26; j++) {
      const count = prefixCounts[right + 1][j] - prefixCounts[left][j];
      odds += count % 2;
    }
    return odds;
  };

  const result = queries.map(([left, right, k]) => {
    const oddCount = getOddCount(left, right);
    return oddCount <= 2 * k + 1;
  });

  return result;
};
```

1178 Number Of Valid Words For Each Puzzle

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1178. Number of Valid Words for Each Puzzle
 * https://leetcode.com/problems/number-of-valid-words-for-each-puzzle/
 * Difficulty: Hard
 *
 * With respect to a given puzzle string, a word is valid if both the following conditions
 * are satisfied:
 * - word contains the first letter of puzzle.
 * - For each letter in word, that letter is in puzzle.
 *   - For example, if the puzzle is "abcdefg", then valid words are "faced", "cabbage",
 *     and "baggage", while
 *   - invalid words are "beefed" (does not include 'a') and "based" (includes 's' which
 *     is not in the puzzle).
 *
 * Return an array answer, where answer[i] is the number of words in the given word list words
 * that is valid with respect to the puzzle puzzles[i].
 */

/**
 * @param {string[]} words
 * @param {string[]} puzzles
 * @return {number[]}
 */
var findNumOfValidWords = function(words, puzzles) {
  const wordMasks = new Map();
  for (const word of words) {
    const mask = getBitmask(word);
    wordMasks.set(mask, (wordMasks.get(mask) || 0) + 1);
  }

  const result = [];
  for (const puzzle of puzzles) {
    const puzzleMask = getBitmask(puzzle);
    const firstCharBit = 1 << (puzzle.charCodeAt(0) - 97);
    let count = 0;

    let submask = puzzleMask;
    do {
      if (submask & firstCharBit && wordMasks.has(submask)) {
        count += wordMasks.get(submask);
      }
      submask = (submask - 1) & puzzleMask;
    } while (submask);

    result.push(count);
  }

  return result;

  function getBitmask(str) {
    let mask = 0;
    for (const char of str) {
      mask |= 1 << (char.charCodeAt(0) - 97);
    }
    return mask;
  }
};
```

1180 Count Substrings With Only One Distinct Letter

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1180. Count Substrings with Only One Distinct Letter
 * https://leetcode.com/problems/count-substrings-with-only-one-distinct-letter/
 * Difficulty: Easy
```

```
 *
 * Given a string s, return the number of substrings that have only one distinct letter.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countLetters = function(s) {
  let result = 0;
  let consecutiveCount = 1;

  for (let i = 1; i < s.length; i++) {
    if (s[i] === s[i - 1]) {
      consecutiveCount++;
    } else {
      result += (consecutiveCount * (consecutiveCount + 1)) / 2;
      consecutiveCount = 1;
    }
  }

  result += (consecutiveCount * (consecutiveCount + 1)) / 2;

  return result;
};
```

1181 Before And After Puzzle

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1181. Before and After Puzzle
 * https://leetcode.com/problems/before-and-after-puzzle/
 * Difficulty: Medium
 *
 * Given a list of phrases, generate a list of Before and After puzzles.
 *
 * A phrase is a string that consists of lowercase English letters and spaces only. No
 * space appears in the start or the end of a phrase. There are no consecutive spaces
 * in a phrase.
 *
 * Before and After puzzles are phrases that are formed by merging two phrases where the
 * last word of the first phrase is the same as the first word of the second phrase.
 *
 * Return the Before and After puzzles that can be formed by every two phrases phrases[i]
 * and phrases[j] where i != j. Note that the order of matching two phrases matters, we
 * want to consider both orders.
 *
 * You should return a list of distinct strings sorted lexicographically.
 */

/**
 * @param {string[]} phrases
 * @return {string[]}
 */
var beforeAndAfterPuzzles = function(phrases) {
  const set = new Set();

  for (let i = 0; i < phrases.length; i++) {
    for (let j = 0; j < phrases.length; j++) {
      if (i !== j) {
        const firstWords = phrases[i].split(' ');
        const secondWords = phrases[j].split(' ');

        const lastWordFirst = firstWords[firstWords.length - 1];
        const firstWordSecond = secondWords[0];
        if (lastWordFirst === firstWordSecond) {
          const merged = firstWords.concat(secondWords.slice(1)).join(' ');
          set.add(merged);
```

```
        }
      }
    }
  }

  return Array.from(set).sort();
};
```

1182 Shortest Distance To Target Color

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1182. Shortest Distance to Target Color
 * https://leetcode.com/problems/shortest-distance-to-target-color/
 * Difficulty: Medium
 *
 * You are given an array colors, in which there are three colors: 1, 2 and 3.
 *
 * You are also given some queries. Each query consists of two integers i and c,
 * return the shortest distance between the given index i and the target color c.
 * If there is no solution return -1.
 */

/**
 * @param {number[]} colors
 * @param {number[][]} queries
 * @return {number[]}
 */
var shortestDistanceColor = function(colors, queries) {
  const colorIndices = { 1: [], 2: [], 3: [] };

  for (let i = 0; i < colors.length; i++) {
    colorIndices[colors[i]].push(i);
  }

  return queries.map(([index, color]) =>
    findClosestDistance(index, colorIndices[color])
  );

  function findClosestDistance(targetIndex, colorArray) {
    if (colorArray.length === 0) return -1;

    let left = 0;
    let right = colorArray.length - 1;
    let closestDistance = Infinity;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      const distance = Math.abs(colorArray[mid] - targetIndex);
      closestDistance = Math.min(closestDistance, distance);

      if (colorArray[mid] < targetIndex) {
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }

    return closestDistance;
  }
};
```

1183 Maximum Number Of Ones

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1183. Maximum Number of Ones
 * https://leetcode.com/problems/maximum-number-of-ones/
 * Difficulty: Hard
 *
 * Consider a matrix M with dimensions width * height, such that every cell has value 0 or 1,
 * and any square sub-matrix of M of size sideLength * sideLength has at most maxOnes ones.
 *
 * Return the maximum possible number of ones that the matrix M can have.
 */

/**
 * @param {number} width
 * @param {number} height
 * @param {number} sideLength
 * @param {number} maxOnes
 * @return {number}
 */
var maximumNumberOfOnes = function(width, height, sideLength, maxOnes) {
  const frequencies = [];

  for (let i = 0; i < sideLength; i++) {
    for (let j = 0; j < sideLength; j++) {
      const rowCount = Math.ceil((height - i) / sideLength);
      const colCount = Math.ceil((width - j) / sideLength);
      frequencies.push(rowCount * colCount);
    }
  }

  frequencies.sort((a, b) => b - a);

  let result = 0;
  for (let i = 0; i < maxOnes; i++) {
    result += frequencies[i];
  }

  return result;
};
```

1184 Distance Between Bus Stops

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1184. Distance Between Bus Stops
 * https://leetcode.com/problems/distance-between-bus-stops/
 * Difficulty: Easy
 *
 * A bus has n stops numbered from 0 to n - 1 that form a circle. We know the distance between
 * all pairs of neighboring stops where distance[i] is the distance between the stops number
 * i and (i + 1) % n.
 *
 * The bus goes along both directions i.e. clockwise and counterclockwise.
 *
 * Return the shortest distance between the given start and destination stops.
 */

/**
 * @param {number[]} distance
 * @param {number} start
 * @param {number} destination
 * @return {number}
 */
var distanceBetweenBusStops = function(distance, start, destination) {
  const normalizeIndices = (from, to) => from < to ? [from, to] : [to, from];

  const [left, right] = normalizeIndices(start, destination);
```

```
  const clockwiseDistance = distance.slice(left, right).reduce((sum, dist) => sum + dist, 0);
  const totalDistance = distance.reduce((sum, dist) => sum + dist, 0);
  const counterclockwiseDistance = totalDistance - clockwiseDistance;

  return Math.min(clockwiseDistance, counterclockwiseDistance);
};
```

1185 Day Of The Week

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1185. Day of the Week
 * https://leetcode.com/problems/day-of-the-week/
 * Difficulty: Easy
 *
 * Given a date, return the corresponding day of the week for that date.
 *
 * The input is given as three integers representing the day, month and year respectively.
 *
 * Return the answer as one of the following values {"Sunday", "Monday", "Tuesday",
"Wednesday",
 * "Thursday", "Friday", "Saturday"}.
 */

/**
 * @param {number} day
 * @param {number} month
 * @param {number} year
 * @return {string}
 */
var dayOfTheWeek = function(day, month, year) {
  return ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'][
    new Date(year, month - 1, day).getDay()
  ];
};
```

1186 Maximum Subarray Sum With One Deletion

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1186. Maximum Subarray Sum with One Deletion
 * https://leetcode.com/problems/maximum-subarray-sum-with-one-deletion/
 * Difficulty: Medium
 *
 * Given an array of integers, return the maximum sum for a non-empty subarray (contiguous
elements)
 * with at most one element deletion. In other words, you want to choose a subarray and
optionally
 * delete one element from it so that there is still at least one element left and the sum of
the
 * remaining elements is maximum possible.
 *
 * Note that the subarray needs to be non-empty after deleting one element.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var maximumSum = function(arr) {
  let maxNoDelete = arr[0];
  let maxOneDelete = 0;
  let result = arr[0];
```

```
  for (let i = 1; i < arr.length; i++) {
    const prevMaxNoDelete = maxNoDelete;
    maxNoDelete = Math.max(arr[i], maxNoDelete + arr[i]);
    maxOneDelete = Math.max(prevMaxNoDelete, maxOneDelete + arr[i]);
    result = Math.max(result, maxNoDelete, maxOneDelete);
  }

  return result;
};
```

1187 Make Array Strictly Increasing

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1187. Make Array Strictly Increasing
 * https://leetcode.com/problems/make-array-strictly-increasing/
 * Difficulty: Hard
 *
 * Given two integer arrays arr1 and arr2, return the minimum number of operations (possibly
zero)
 * needed to make arr1 strictly increasing.
 *
 * In one operation, you can choose two indices 0 <= i < arr1.length and 0 <= j < arr2.length
and
 * do the assignment arr1[i] = arr2[j].
 *
 * If there is no way to make arr1 strictly increasing, return -1.
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
 * @return {number}
 */
var makeArrayIncreasing = function(arr1, arr2) {
  arr2 = [...new Set(arr2)].sort((a, b) => a - b);
  let dp = new Map([[-Infinity, 0]]);

  for (let i = 0; i < arr1.length; i++) {
    const nextDp = new Map();
    let minSteps = Infinity;

    for (const [prev, steps] of dp) {
      if (steps >= minSteps) continue;

      if (arr1[i] > prev) {
        updateMin(nextDp, arr1[i], steps);
        minSteps = Math.min(minSteps, steps);
      }

      const start = binarySearch(arr2, prev);
      for (let j = start; j < arr2.length; j++) {
        if (arr2[j] <= prev) continue;
        updateMin(nextDp, arr2[j], steps + 1);
        minSteps = Math.min(minSteps, steps + 1);
        if (arr2[j] >= arr1[i]) break;
      }
    }

    if (nextDp.size === 0) return -1;
    dp = pruneStates(nextDp);
  }

  return Math.min(...dp.values());
};

function updateMin(map, key, value) {
  map.set(key, Math.min(value, map.get(key) ?? Infinity));
```

```
}

function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length;
  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (arr[mid] <= target) left = mid + 1;
    else right = mid;
  }
  return left;
}

function pruneStates(input) {
  const map = new Map();
  let minSteps = Infinity;
  for (const [val, steps] of [...input].sort((a, b) => a[0] - b[0])) {
    if (steps < minSteps) {
      map.set(val, steps);
      minSteps = steps;
    }
  }
  return map;
}
```

1189 Maximum Number Of Balloons

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1189. Maximum Number of Balloons
 * https://leetcode.com/problems/maximum-number-of-balloons/
 * Difficulty: Easy
 *
 * Given a string text, you want to use the characters of text
 * to form as many instances of the word "balloon" as possible.
 *
 * You can use each character in text at most once. Return the
 * maximum number of instances that can be formed.
 */

/**
 * @param {string} text
 * @return {number}
 */
var maxNumberOfBalloons = function(text, word = 'balloon', i = 0, count = 0) {
  const map = text.split('').reduce((o, k) => ({...o, [k]: o[k] ? ++o[k] : 1}), {});
  while (true) {
    const char = word[i++ % word.length];
    if (!map[char] || map[char]-- === 0) break;
    if (i % word.length === 0) count++;
  }
  return count;
};
```

1190 Reverse Substrings Between Each Pair Of Parentheses

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1190. Reverse Substrings Between Each Pair of Parentheses
 * https://leetcode.com/problems/reverse-substrings-between-each-pair-of-parentheses/
 * Difficulty: Medium
 *
 * You are given a string s that consists of lower case English letters and brackets.
```

```
 *
 * Reverse the strings in each pair of matching parentheses, starting from the innermost one.
 *
 * Your result should not contain any brackets.
 */

/**
 * @param {string} s
 * @return {string}
 */
var reverseParentheses = function(s) {
  const stack = [[]];

  for (const char of s) {
    if (char === '(') {
      stack.push([]);
    } else if (char === ')') {
      const reversed = stack.pop().reverse();
      stack[stack.length - 1].push(...reversed);
    } else {
      stack[stack.length - 1].push(char);
    }
  }

  return stack[0].join('');
};
```

1191 K Concatenation Maximum Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1191. K-Concatenation Maximum Sum
 * https://leetcode.com/problems/k-concatenation-maximum-sum/
 * Difficulty: Medium
 *
 * Given an integer array arr and an integer k, modify the array by repeating it k times.
 *
 * For example, if arr = [1, 2] and k = 3 then the modified array will be [1, 2, 1, 2, 1, 2].
 *
 * Return the maximum sub-array sum in the modified array. Note that the length of the
sub-array
 * can be 0 and its sum in that case is 0.
 *
 * As the answer can be very large, return the answer modulo 109 + 7.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number}
 */
var kConcatenationMaxSum = function(arr, k) {
  const MOD = 1e9 + 7;
  const arraySum = arr.reduce((sum, num) => sum + num, 0);
  const maxSingle = helper(arr);

  if (k === 1) return maxSingle;

  const doubleArray = [...arr, ...arr];
  const maxDouble = helper(doubleArray);

  if (arraySum > 0) {
    return Number((BigInt(maxDouble) + BigInt(arraySum) * BigInt(k - 2)) % BigInt(MOD));
  }

  return maxDouble;
};

function helper(arr) {
```

```
  let maxSoFar = 0;
  let maxEndingHere = 0;

  for (const num of arr) {
    maxEndingHere = Math.max(0, maxEndingHere + num);
    maxSoFar = Math.max(maxSoFar, maxEndingHere);
  }

  return maxSoFar;
}
```

1192 Critical Connections In A Network

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1192. Critical Connections in a Network
 * https://leetcode.com/problems/critical-connections-in-a-network/
 * Difficulty: Hard
 *
 * There are n servers numbered from 0 to n - 1 connected by undirected server-to-server
connections
 * forming a network where connections[i] = [ai, bi] represents a connection between servers
ai and
 * bi. Any server can reach other servers directly or indirectly through the network.
 *
 * A critical connection is a connection that, if removed, will make some servers unable to
reach
 * some other server.
 *
 * Return all critical connections in the network in any order.
 */

/**
 * @param {number} n
 * @param {number[][]} connections
 * @return {number[][]}
 */
var criticalConnections = function(n, connections) {
  const graph = Array.from({ length: n }, () => []);
  const discoveryTimes = new Array(n).fill(-1);
  const lowestReachableTimes = new Array(n).fill(-1);
  const criticalEdges = [];
  let time = 0;

  connections.forEach(([from, to]) => {
    graph[from].push(to);
    graph[to].push(from);
  });

  exploreNode(0, -1);

  return criticalEdges;

  function exploreNode(current, parent) {
    discoveryTimes[current] = lowestReachableTimes[current] = time++;

    for (const neighbor of graph[current]) {
      if (neighbor === parent) continue;
      if (discoveryTimes[neighbor] === -1) {
        exploreNode(neighbor, current);
        lowestReachableTimes[current] = Math.min(
          lowestReachableTimes[current],
          lowestReachableTimes[neighbor]
        );
        if (lowestReachableTimes[neighbor] > discoveryTimes[current]) {
          criticalEdges.push([current, neighbor]);
        }
      } else {
        lowestReachableTimes[current] = Math.min(
```

```
          lowestReachableTimes[current],
          discoveryTimes[neighbor]
        );
      }
    }
  }
};
```

1196 How Many Apples Can You Put Into The Basket

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1196. How Many Apples Can You Put into the Basket
 * https://leetcode.com/problems/how-many-apples-can-you-put-into-the-basket/
 * Difficulty: Easy
 *
 * You have some apples and a basket that can carry up to 5000 units of weight.
 *
 * Given an integer array weight where weight[i] is the weight of the ith apple,
 * return the maximum number of apples you can put in the basket.
 */

/**
 * @param {number[]} weight
 * @return {number}
 */
var maxNumberOfApples = function(weight) {
  weight.sort((a, b) => a - b);

  let totalWeight = 0;
  let result = 0;

  for (const appleWeight of weight) {
    if (totalWeight + appleWeight <= 5000) {
      totalWeight += appleWeight;
      result++;
    } else {
      break;
    }
  }

  return result;
};
```

1197 Minimum Knight Moves

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1197. Minimum Knight Moves
 * https://leetcode.com/problems/minimum-knight-moves/
 * Difficulty: Medium
 *
 * In an infinite chess board with coordinates from -infinity to +infinity, you have
 * a knight at square [0, 0].
 *
 * A knight has 8 possible moves it can make, as illustrated below. Each move is two
 * squares in a cardinal direction, then one square in an orthogonal direction.
 *
 * Return the minimum number of steps needed to move the knight to the square [x, y].
 * It is guaranteed the answer exists.
 */

/**
```

```
 * @param {number} x
 * @param {number} y
 * @return {number}
 */
var minKnightMoves = function(x, y) {
  x = Math.abs(x);
  y = Math.abs(y);

  const memo = new Map();
  return dfs(x, y);

  function dfs(currX, currY) {
    if (currX + currY === 0) return 0;
    if (currX + currY === 2) return 2;

    const key = `${currX},${currY}`;
    if (memo.has(key)) return memo.get(key);

    const result = Math.min(
      dfs(Math.abs(currX - 1), Math.abs(currY - 2)),
      dfs(Math.abs(currX - 2), Math.abs(currY - 1))
    ) + 1;

    memo.set(key, result);
    return result;
  }
};
```

1198 Find Smallest Common Element In All Rows

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1198. Find Smallest Common Element in All Rows
 * https://leetcode.com/problems/find-smallest-common-element-in-all-rows/
 * Difficulty: Medium
 *
 * Given an m x n matrix mat where every row is sorted in strictly increasing order,
 * return the smallest common element in all rows.
 *
 * If there is no common element, return -1.
 */

/**
 * @param {number[][]} mat
 * @return {number}
 */
var smallestCommonElement = function(mat) {
  const map = new Map();
  const rows = mat.length;

  for (let i = 0; i < rows; i++) {
    for (const element of mat[i]) {
      map.set(element, (map.get(element) || 0) + 1);

      if (map.get(element) === rows) {
        return element;
      }
    }
  }

  return -1;
};
```

1199 Minimum Time To Build Blocks

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1199. Minimum Time to Build Blocks
 * https://leetcode.com/problems/minimum-time-to-build-blocks/
 * Difficulty: Hard
 *
 * You are given a list of blocks, where blocks[i] = t means that the i-th block needs
 * t units of time to be built. A block can only be built by exactly one worker.
 *
 * A worker can either split into two workers (number of workers increases by one) or
 * build a block then go home. Both decisions cost some time.
 *
 * The time cost of spliting one worker into two workers is given as an integer split.
 * Note that if two workers split at the same time, they split in parallel so the cost
 * would be split.
 *
 * Output the minimum time needed to build all blocks.
 *
 * Initially, there is only one worker.
 */

/**
 * @param {number[]} blocks
 * @param {number} split
 * @return {number}
 */
var minBuildTime = function(blocks, split) {
  const minHeap = new PriorityQueue((a, b) => a - b);

  for (const block of blocks) {
    minHeap.enqueue(block);
  }

  while (minHeap.size() > 1) {
    const first = minHeap.dequeue();
    const second = minHeap.dequeue();

    const mergedTime = split + Math.max(first, second);
    minHeap.enqueue(mergedTime);
  }

  return minHeap.dequeue();
};
```

1200 Minimum Absolute Difference

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1200. Minimum Absolute Difference
 * https://leetcode.com/problems/minimum-absolute-difference/
 * Difficulty: Easy
 *
 * Given an array of distinct integers arr, find all pairs of elements with the minimum
absolute
 * difference of any two elements.
 *
 * Return a list of pairs in ascending order(with respect to pairs), each pair [a, b] follows:
 * - a, b are from arr
 * - a < b
 * - b - a equals to the minimum absolute difference of any two elements in arr
 */
```

```
/**
 * @param {number[]} arr
 * @return {number[][]}
 */
var minimumAbsDifference = function(arr) {
  arr.sort((a, b) => a - b);

  const min = arr.reduce((m, n, i) => Math.min(m, n - (arr[i - 1] ?? -Infinity)), Infinity);
  return arr.reduce((result, n, i) => {
    if (min === n - arr[i - 1]) {
      result.push([arr[i - 1], n]);
    }
    return result;
  }, []);
};
```

1201 Ugly Number Iii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1201. Ugly Number III
 * https://leetcode.com/problems/ugly-number-iii/
 * Difficulty: Medium
 *
 * An ugly number is a positive integer that is divisible by a, b, or c.
 *
 * Given four integers n, a, b, and c, return the nth ugly number.
 */

/**
 * @param {number} n
 * @param {number} a
 * @param {number} b
 * @param {number} c
 * @return {number}
 */
var nthUglyNumber = function(n, a, b, c) {
  const ab = lcm(a, b);
  const bc = lcm(b, c);
  const ac = lcm(a, c);
  const abc = lcm(ab, c);
  let left = 1;
  let right = 2 * 10**9;

  while (left < right) {
    const mid = left + Math.floor((right - left) / 2);
    if (countUgly(mid) < n) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;

  function lcm(x, y) {
    return (x * y) / gcd(x, y);
  }

  function gcd(x, y) {
    return y === 0 ? x : gcd(y, x % y);
  }

  function countUgly(num) {
    return Math.floor(num / a) + Math.floor(num / b) + Math.floor(num / c)
      - Math.floor(num / ab) - Math.floor(num / bc) - Math.floor(num / ac)
      + Math.floor(num / abc);
  }
```

```
};
```

1202 Smallest String With Swaps

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1202. Smallest String With Swaps
 * https://leetcode.com/problems/smallest-string-with-swaps/
 * Difficulty: Medium
 *
 * You are given a string s, and an array of pairs of indices in the string pairs where
 * pairs[i] = [a, b] indicates 2 indices(0-indexed) of the string.
 *
 * You can swap the characters at any pair of indices in the given pairs any number of times.
 *
 * Return the lexicographically smallest string that s can be changed to after using the
swaps.
 */

/**
 * @param {string} s
 * @param {number[][]} pairs
 * @return {string}
 */
var smallestStringWithSwaps = function(s, pairs) {
  const parent = Array.from({ length: s.length }, (_, i) => i);
  const rank = new Array(s.length).fill(0);

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  pairs.forEach(([x, y]) => union(x, y));

  const components = new Map();
  for (let i = 0; i < s.length; i++) {
    const root = find(i);
    if (!components.has(root)) {
      components.set(root, { indices: [], chars: [] });
    }
    components.get(root).indices.push(i);
    components.get(root).chars.push(s[i]);
  }

  const result = new Array(s.length);
  for (const { indices, chars } of components.values()) {
    chars.sort((a, b) => b.localeCompare(a));
    indices.sort((a, b) => a - b);
    indices.forEach((idx, i) => result[idx] = chars.pop());
  }

  return result.join('');

  function union(x, y) {
    const rootX = find(x);
    const rootY = find(y);
    if (rootX !== rootY) {
      if (rank[rootX] < rank[rootY]) {
        parent[rootX] = rootY;
      } else if (rank[rootX] > rank[rootY]) {
        parent[rootY] = rootX;
      } else {
        parent[rootY] = rootX;
        rank[rootX]++;
      }
```

```
    }
  }
};
```

1203 Sort Items By Groups Respecting Dependencies

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1203. Sort Items by Groups Respecting Dependencies
 * https://leetcode.com/problems/sort-items-by-groups-respecting-dependencies/
 * Difficulty: Hard
 *
 * There are n items each belonging to zero or one of m groups where group[i] is the group
that
 * the i-th item belongs to and it's equal to -1 if the i-th item belongs to no group. The
items
 * and the groups are zero indexed. A group can have no item belonging to it.
 *
 * Return a sorted list of the items such that:
 * - The items that belong to the same group are next to each other in the sorted list.
 * - There are some relations between these items where beforeItems[i] is a list containing
all
 *   the items that should come before the i-th item in the sorted array (to the left of the
i-th
 *   item).
 *
 * Return any solution if there is more than one solution and return an empty list if there is
 * no solution.
 */

/**
 * @param {number} n
 * @param {number} m
 * @param {number[]} group
 * @param {number[][]} beforeItems
 * @return {number[]}
 */
var sortItems = function(n, m, group, beforeItems) {
  const groupAdj = Array.from({ length: n + m }, () => []);
  const itemAdj = Array.from({ length: n }, () => []);
  const groupInDegree = new Array(n + m).fill(0);
  const itemInDegree = new Array(n).fill(0);
  let nextGroupId = m;

  assignGroupIds();
  buildGraphs();

  const groupOrder = topologicalSort(groupAdj, groupInDegree, nextGroupId);
  if (!groupOrder.length) return [];

  const itemOrder = topologicalSort(itemAdj, itemInDegree, n);
  if (!itemOrder.length) return [];

  const groupToItems = new Map();
  itemOrder.forEach(item => {
    if (!groupToItems.has(group[item])) {
      groupToItems.set(group[item], []);
    }
    groupToItems.get(group[item]).push(item);
  });

  const result = [];
  groupOrder.forEach(groupId => {
    if (groupToItems.has(groupId)) {
      result.push(...groupToItems.get(groupId));
    }
  });
```

```
  return result;

  function assignGroupIds() {
    for (let i = 0; i < n; i++) {
      if (group[i] === -1) {
        group[i] = nextGroupId++;
      }
    }
  }

  function buildGraphs() {
    for (let i = 0; i < n; i++) {
      for (const before of beforeItems[i]) {
        if (group[before] !== group[i]) {
          groupAdj[group[before]].push(group[i]);
          groupInDegree[group[i]]++;
        } else {
          itemAdj[before].push(i);
          itemInDegree[i]++;
        }
      }
    }
  }

  function topologicalSort(adjList, inDegree, size) {
    const queue = [];
    const order = [];

    for (let i = 0; i < size; i++) {
      if (inDegree[i] === 0) queue.push(i);
    }

    while (queue.length) {
      const current = queue.shift();
      order.push(current);
      for (const next of adjList[current]) {
        inDegree[next]--;
        if (inDegree[next] === 0) queue.push(next);
      }
    }

    return order.length === size ? order : [];
  }
};
```

1206 Design Skiplist

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1206. Design Skiplist
 * https://leetcode.com/problems/design-skiplist/
 * Difficulty: Hard
 *
 * Design a Skiplist without using any built-in libraries.
 *
 * A skiplist is a data structure that takes O(log(n)) time to add, erase and search.
Comparing with
 * treap and red-black tree which has the same function and performance, the code length of
Skiplist
 * can be comparatively short and the idea behind Skiplists is just simple linked lists.
 *
 * For example, we have a Skiplist containing [30,40,50,60,70,90] and we want to add 80 and 45
 into
 * it. The Skiplist works this way:
 *
 * You can see there are many layers in the Skiplist. Each layer is a sorted linked list. With
 the
 * help of the top layers, add, erase and search can be faster than O(n). It can be proven
that the
```

```
 * average time complexity for each operation is O(log(n)) and space complexity is O(n).
 *
 * See more about Skiplist: https://en.wikipedia.org/wiki/Skip_list
 * Implement the Skiplist class:
 * - Skiplist() Initializes the object of the skiplist.
 * - bool search(int target) Returns true if the integer target exists in the Skiplist or
false
 *   otherwise.
 * - void add(int num) Inserts the value num into the SkipList.
 * - bool erase(int num) Removes the value num from the Skiplist and returns true. If num does
 not
 *   exist in the Skiplist, do nothing and return false. If there exist multiple num values,
 *   removing any one of them is fine.
 */

var Skiplist = function() {
  this.values = {};
};

/**
 * @param {number} target
 * @return {boolean}
 */
Skiplist.prototype.search = function(target) {
  return target in this.values;
};

/**
 * @param {number} num
 * @return {void}
 */
Skiplist.prototype.add = function(num) {
  this.values[num] = (this.values[num] ?? 0) + 1;
};

/**
 * @param {number} num
 * @return {boolean}
 */
Skiplist.prototype.erase = function(num) {
  if (!(num in this.values)) {
    return false;
  }
  if (!--this.values[num]) {
    delete this.values[num];
  }
  return true;
};
```

1207 Unique Number Of Occurrences

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1207. Unique Number of Occurrences
 * https://leetcode.com/problems/unique-number-of-occurrences/
 * Difficulty: Easy
 *
 * Given an array of integers arr, return true if the number of occurrences of each
 * value in the array is unique or false otherwise.
 */

/**
 * @param {number[]} arr
 * @return {boolean}
 */
var uniqueOccurrences = function(arr) {
  const map = new Map();
  arr.forEach(n => map.set(n, map.has(n) ? map.get(n) + 1 : 1));
  const occurrences = Array.from(map.values());
```

```
  return new Set(occurrences).size === occurrences.length;
};
```

1208 Get Equal Substrings Within Budget

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1208. Get Equal Substrings Within Budget
 * https://leetcode.com/problems/get-equal-substrings-within-budget/
 * Difficulty: Medium
 *
 * You are given two strings s and t of the same length and an integer maxCost.
 *
 * You want to change s to t. Changing the ith character of s to ith character of t costs
 * |s[i] - t[i]| (i.e., the absolute difference between the ASCII values of the characters).
 *
 * Return the maximum length of a substring of s that can be changed to be the same as the
 * corresponding substring of t with a cost less than or equal to maxCost. If there is no
 * substring from s that can be changed to its corresponding substring from t, return 0.
 */

/**
 * @param {string} s
 * @param {string} t
 * @param {number} maxCost
 * @return {number}
 */
var equalSubstring = function(s, t, maxCost) {
  let left = -1;

  for (let right = 0; right < s.length; right++) {
    maxCost -= Math.abs(s.charCodeAt(right) - t.charCodeAt(right));

    if (maxCost < 0) {
      left++;
      maxCost += Math.abs(s.charCodeAt(left) - t.charCodeAt(left));
    }
  }

  return s.length - left - 1;
};
```

1209 Remove All Adjacent Duplicates In String Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1209. Remove All Adjacent Duplicates in String II
 * https://leetcode.com/problems/remove-all-adjacent-duplicates-in-string-ii/
 * Difficulty: Medium
 *
 * You are given a string s and an integer k, a k duplicate removal consists of choosing k
 * adjacent and equal letters from s and removing them, causing the left and the right side
 * of the deleted substring to concatenate together.
 *
 * We repeatedly make k duplicate removals on s until we no longer can.
 *
 * Return the final string after all such duplicate removals have been made. It is guaranteed
 * that the answer is unique.
 */

/**
 * @param {string} s
 * @param {number} k
```

```
 * @return {string}
 */
var removeDuplicates = function(s, k) {
  const stack = [];

  for (const char of s) {
    if (stack.length && stack[stack.length - 1][0] === char) {
      stack[stack.length - 1][1]++;
      if (stack[stack.length - 1][1] === k) {
        stack.pop();
      }
    } else {
      stack.push([char, 1]);
    }
  }

  return stack.reduce((result, [char, count]) => result + char.repeat(count), '');
};
```

1210 Minimum Moves To Reach Target With Rotations

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1210. Minimum Moves to Reach Target with Rotations
 * https://leetcode.com/problems/minimum-moves-to-reach-target-with-rotations/
 * Difficulty: Hard
 *
 * In an n*n grid, there is a snake that spans 2 cells and starts moving from the top left
corner
 * at (0, 0) and (0, 1). The grid has empty cells represented by zeros and blocked cells
represented
 * by ones. The snake wants to reach the lower right corner at (n-1, n-2) and (n-1, n-1).
 *
 * In one move the snake can:
 * - Move one cell to the right if there are no blocked cells there. This move keeps the
 *   horizontal/vertical position of the snake as it is.
 * - Move down one cell if there are no blocked cells there. This move keeps the
horizontal/vertical
 *   position of the snake as it is.
 * - Rotate clockwise if it's in a horizontal position and the two cells under it are both
empty.
 *   In that case the snake moves from (r, c) and (r, c+1) to (r, c) and (r+1, c).
 * - Rotate counterclockwise if it's in a vertical position and the two cells to its right are
 both
 *   empty. In that case the snake moves from (r, c) and (r+1, c) to (r, c) and (r, c+1).
 *
 * Return the minimum number of moves to reach the target.
 *
 * If there is no way to reach the target, return -1.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minimumMoves = function(grid) {
  const n = grid.length;
  const queue = [[0, 0, 0, 1, 0]];
  const visited = new Set(['0,0,0,1']);

  while (queue.length) {
    const [tailX, tailY, headX, headY, moves] = queue.shift();

    if (tailX === n-1 && tailY === n-2 && headX === n-1 && headY === n-1) {
      return moves;
    }

    const isHorizontal = tailX === headX;
```

```
    if (isHorizontal) {
      if (headY + 1 < n && isValid(tailX, tailY + 1) && isValid(headX, headY + 1)) {
        const rightState = `${tailX},${tailY + 1},${headX},${headY + 1}`;
        if (!visited.has(rightState)) {
          visited.add(rightState);
          queue.push([tailX, tailY + 1, headX, headY + 1, moves + 1]);
        }
      }
      if (tailX + 1 < n && isValid(tailX + 1, tailY) && isValid(headX + 1, headY)) {
        const downState = `${tailX + 1},${tailY},${headX + 1},${headY}`;
        if (!visited.has(downState)) {
          visited.add(downState);
          queue.push([tailX + 1, tailY, headX + 1, headY, moves + 1]);
        }
      }
      if (tailX + 1 < n && isValid(tailX + 1, tailY) && isValid(tailX + 1, tailY + 1)) {
        const rotateState = `${tailX},${tailY},${tailX + 1},${tailY}`;
        if (!visited.has(rotateState)) {
          visited.add(rotateState);
          queue.push([tailX, tailY, tailX + 1, tailY, moves + 1]);
        }
      }
    } else {
      if (headX + 1 < n && isValid(tailX + 1, tailY) && isValid(headX + 1, headY)) {
        const downState = `${tailX + 1},${tailY},${headX + 1},${headY}`;
        if (!visited.has(downState)) {
          visited.add(downState);
          queue.push([tailX + 1, tailY, headX + 1, headY, moves + 1]);
        }
      }
      if (tailY + 1 < n && isValid(tailX, tailY + 1) && isValid(headX, tailY + 1)) {
        const rightState = `${tailX},${tailY + 1},${headX},${tailY + 1}`;
        if (!visited.has(rightState)) {
          visited.add(rightState);
          queue.push([tailX, tailY + 1, headX, tailY + 1, moves + 1]);
        }
      }
      if (tailY + 1 < n && isValid(tailX, tailY + 1) && isValid(tailX + 1, tailY + 1)) {
        const rotateState = `${tailX},${tailY},${tailX},${tailY + 1}`;
        if (!visited.has(rotateState)) {
          visited.add(rotateState);
          queue.push([tailX, tailY, tailX, tailY + 1, moves + 1]);
        }
      }
    }
  }

  return -1;

  function isValid(x, y) {
    return x >= 0 && x < n && y >= 0 && y < n && grid[x][y] === 0;
  }
};
```

1213 Intersection Of Three Sorted Arrays

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1213. Intersection of Three Sorted Arrays
 * https://leetcode.com/problems/intersection-of-three-sorted-arrays/
 * Difficulty: Easy
 *
 * Given three integer arrays arr1, arr2 and arr3 sorted in strictly increasing order,
 * return a sorted array of only the integers that appeared in all three arrays.
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
```

```
 * @param {number[]} arr3
 * @return {number[]}
 */
var arraysIntersection = function(arr1, arr2, arr3) {
  let i = 0;
  let j = 0;
  let k = 0;
  const result = [];

  while (i < arr1.length && j < arr2.length && k < arr3.length) {
    if (arr1[i] === arr2[j] && arr2[j] === arr3[k]) {
      result.push(arr1[i]);
      i++;
      j++;
      k++;
    } else {
      const minValue = Math.min(arr1[i], arr2[j], arr3[k]);
      if (arr1[i] === minValue) i++;
      if (arr2[j] === minValue) j++;
      if (arr3[k] === minValue) k++;
    }
  }

  return result;
};
```

1214 Two Sum Bsts

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1214. Two Sum BSTs
 * https://leetcode.com/problems/two-sum-bsts/
 * Difficulty: Medium
 *
 * Given the roots of two binary search trees, root1 and root2, return true if and only if
 * there is a node in the first tree and a node in the second tree whose values sum up to
 * a given integer target.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root1
 * @param {TreeNode} root2
 * @param {number} target
 * @return {boolean}
 */
var twoSumBSTs = function(root1, root2, target) {
  const valuesSet = new Set();
  collectValues(root1);
  return searchForComplement(root2);

  function collectValues(node) {
    if (!node) return;
    valuesSet.add(node.val);
    collectValues(node.left);
    collectValues(node.right);
  }

  function searchForComplement(node) {
    if (!node) return false;

    const complement = target - node.val;
```

```
    if (valuesSet.has(complement)) return true;

    return searchForComplement(node.left) || searchForComplement(node.right);
  }
};
```

1215 Stepping Numbers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1215. Stepping Numbers
 * https://leetcode.com/problems/stepping-numbers/
 * Difficulty: Medium
 *
 * A stepping number is an integer such that all of its adjacent digits have an absolute
 * difference of exactly 1.
 * - For example, 321 is a stepping number while 421 is not.
 *
 * Given two integers low and high, return a sorted list of all the stepping numbers in the
 * inclusive range [low, high].
 */

/**
 * @param {number} low
 * @param {number} high
 * @return {number[]}
 */
var countSteppingNumbers = function(low, high) {
  const result = [];

  if (low === 0) result.push(0);

  const minLength = low.toString().length;
  const maxLength = high.toString().length;

  for (let length = minLength; length <= maxLength; length++) {
    for (let start = 1; start <= 9; start++) {
      dfs(start.toString(), length);
    }
  }

  return result.sort((a, b) => a - b);

  function dfs(current, targetLength) {
    if (current.length === targetLength) {
      const num = parseInt(current);
      if (num >= low && num <= high) {
        result.push(num);
      }
      return;
    }

    const lastDigit = parseInt(current[current.length - 1]);

    if (lastDigit > 0) {
      dfs(current + (lastDigit - 1), targetLength);
    }
    if (lastDigit < 9) {
      dfs(current + (lastDigit + 1), targetLength);
    }
  }
};
```

1216 Valid Palindrome Iii

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1216. Valid Palindrome III
 * https://leetcode.com/problems/valid-palindrome-iii/
 * Difficulty: Hard
 *
 * Given a string s and an integer k, return true if s is a k-palindrome.
 *
 * A string is k-palindrome if it can be transformed into a palindrome by removing
 * at most k characters from it.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {boolean}
 */
var isValidPalindrome = function(s, k) {
  const n = s.length;
  const memo = new Array(n).fill().map(() => new Array(n).fill(-1));
  const longestPalindrome = longestPalindromeSubseq(0, n - 1);
  const deletions = n - longestPalindrome;

  return deletions <= k;

  function longestPalindromeSubseq(left, right) {
    if (left > right) return 0;
    if (left === right) return 1;

    if (memo[left][right] !== -1) return memo[left][right];

    if (s[left] === s[right]) {
      memo[left][right] = 2 + longestPalindromeSubseq(left + 1, right - 1);
    } else {
      memo[left][right] = Math.max(
        longestPalindromeSubseq(left + 1, right),
        longestPalindromeSubseq(left, right - 1)
      );
    }

    return memo[left][right];
  }
};
```

1217 Minimum Cost To Move Chips To The Same Position

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1217. Minimum Cost to Move Chips to The Same Position
 * https://leetcode.com/problems/minimum-cost-to-move-chips-to-the-same-position/
 * Difficulty: Easy
 *
 * We have n chips, where the position of the ith chip is position[i].
 *
 * We need to move all the chips to the same position. In one step, we can change the position
 * of the ith chip from position[i] to:
 * - position[i] + 2 or position[i] - 2 with cost = 0.
 * - position[i] + 1 or position[i] - 1 with cost = 1.
 *
 * Return the minimum cost needed to move all the chips to the same position.
 */
```

```
/**
 * @param {number[]} position
 * @return {number}
 */
var minCostToMoveChips = function(position) {
  const count = position.reduce((result, n) => n % 2 ? ++result : result, 0);
  return count >= position.length / 2 ? position.length - count : count;
};
```

1218 Longest Arithmetic Subsequence Of Given Difference

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1218. Longest Arithmetic Subsequence of Given Difference
 * https://leetcode.com/problems/longest-arithmetic-subsequence-of-given-difference/
 * Difficulty: Medium
 *
 * Given an integer array arr and an integer difference, return the length of the longest
 * subsequence in arr which is an arithmetic sequence such that the difference between
adjacent
 * elements in the subsequence equals difference.
 *
 * A subsequence is a sequence that can be derived from arr by deleting some or no elements
without
 * changing the order of the remaining elements.
 */

/**
 * @param {number[]} arr
 * @param {number} difference
 * @return {number}
 */
var longestSubsequence = function(arr, difference) {
  const sequenceLengths = new Map();
  let result = 0;

  for (const num of arr) {
    const prevNum = num - difference;
    const currentLength = (sequenceLengths.get(prevNum) || 0) + 1;
    sequenceLengths.set(num, currentLength);
    result = Math.max(result, currentLength);
  }

  return result;
};
```

1219 Path With Maximum Gold

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1219. Path with Maximum Gold
 * https://leetcode.com/problems/path-with-maximum-gold/
 * Difficulty: Medium
 *
 * In a gold mine grid of size m x n, each cell in this mine has an integer representing the
 * amount of gold in that cell, 0 if it is empty.
 *
 * Return the maximum amount of gold you can collect under the conditions:
 * - Every time you are located in a cell you will collect all the gold in that cell.
 * - From your position, you can walk one step to the left, right, up, or down.
 * - You can't visit the same cell more than once.
 * - Never visit a cell with 0 gold.
```

```
 * - You can start and stop collecting gold from any position in the grid that has some gold.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var getMaximumGold = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  let maxGold = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] !== 0) {
        exploreGold(i, j, 0);
      }
    }
  }

  return maxGold;

  function exploreGold(row, col, currentGold) {
    if (row < 0 || row >= rows || col < 0 || col >= cols || grid[row][col] === 0) {
      maxGold = Math.max(maxGold, currentGold);
      return;
    }

    const goldHere = grid[row][col];
    grid[row][col] = 0;

    exploreGold(row - 1, col, currentGold + goldHere);
    exploreGold(row + 1, col, currentGold + goldHere);
    exploreGold(row, col - 1, currentGold + goldHere);
    exploreGold(row, col + 1, currentGold + goldHere);

    grid[row][col] = goldHere;
  }
};
```

1220 Count Vowels Permutation

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1220. Count Vowels Permutation
 * https://leetcode.com/problems/count-vowels-permutation/
 * Difficulty: Hard
 *
 * Given an integer n, your task is to count how many strings of length n can be formed under
 * the following rules:
 * - Each character is a lower case vowel ('a', 'e', 'i', 'o', 'u')
 * - Each vowel 'a' may only be followed by an 'e'.
 * - Each vowel 'e' may only be followed by an 'a' or an 'i'.
 * - Each vowel 'i' may not be followed by another 'i'.
 * - Each vowel 'o' may only be followed by an 'i' or a 'u'.
 * - Each vowel 'u' may only be followed by an 'a'.
 *
 * Since the answer may be too large, return it modulo 10^9 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countVowelPermutation = function(n) {
  const MOD = 1000000007n;
  let aCount = 1n;
  let eCount = 1n;
  let iCount = 1n;
```

```
  let oCount = 1n;
  let uCount = 1n;

  for (let length = 2; length <= n; length++) {
    const prevACount = aCount;
    const prevECount = eCount;
    const prevICount = iCount;
    const prevOCount = oCount;
    const prevUCount = uCount;

    aCount = (prevECount + prevICount + prevUCount) % MOD;
    eCount = (prevACount + prevICount) % MOD;
    iCount = (prevECount + prevOCount) % MOD;
    oCount = (prevICount) % MOD;
    uCount = (prevICount + prevOCount) % MOD;
  }

  const total = aCount + eCount + iCount + oCount + uCount;

  return Number(total % MOD);
};
```

1221 Split A String In Balanced Strings

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1221. Split a String in Balanced Strings
 * https://leetcode.com/problems/split-a-string-in-balanced-strings/
 * Difficulty: Easy
 *
 * Balanced strings are those that have an equal quantity of 'L' and 'R' characters.
 *
 * Given a balanced string s, split it into some number of substrings such that:
 * - Each substring is balanced.
 *
 * Return the maximum number of balanced strings you can obtain.
 */

/**
 * @param {string} s
 * @return {number}
 */
var balancedStringSplit = function(s) {
  let result = 0;
  let balance = 0;

  for (const char of s) {
    if (char === 'R') balance++;
    else balance--;

    if (balance === 0) result++;
  }

  return result;
};
```

1222 Queens That Can Attack The King

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1222. Queens That Can Attack the King
 * https://leetcode.com/problems/queens-that-can-attack-the-king/
```

```
 * Difficulty: Medium
 *
 * On a 0-indexed 8 x 8 chessboard, there can be multiple black queens and one white king.
 *
 * You are given a 2D integer array queens where queens[i] = [xQueeni, yQueeni] represents the
 * position of the ith black queen on the chessboard. You are also given an integer array king
 * of length 2 where king = [xKing, yKing] represents the position of the white king.
 *
 * Return the coordinates of the black queens that can directly attack the king. You may
return
 * the answer in any order.
 */

/**
 * @param {number[][]} queens
 * @param {number[]} king
 * @return {number[][]}
 */
var queensAttacktheKing = function(queens, king) {
  const [kingX, kingY] = king;
  const queenSet = new Set(queens.map(([x, y]) => `${x},${y}`));
  const attackers = [];
  const directions = [
    [-1, 0], [1, 0], [0, -1], [0, 1],
    [-1, -1], [-1, 1], [1, -1], [1, 1]
  ];

  for (const [dx, dy] of directions) {
    let x = kingX + dx;
    let y = kingY + dy;

    while (x >= 0 && x < 8 && y >= 0 && y < 8) {
      const pos = `${x},${y}`;
      if (queenSet.has(pos)) {
        attackers.push([x, y]);
        break;
      }
      x += dx;
      y += dy;
    }
  }

  return attackers;
};
```

1223 Dice Roll Simulation

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1223. Dice Roll Simulation
 * https://leetcode.com/problems/dice-roll-simulation/
 * Difficulty: Hard
 *
 * A die simulator generates a random number from 1 to 6 for each roll. You introduced a
constraint
 * to the generator such that it cannot roll the number i more than rollMax[i] (1-indexed)
 * consecutive times.
 *
 * Given an array of integers rollMax and an integer n, return the number of distinct
sequences
 * that can be obtained with exact n rolls. Since the answer may be too large, return it
modulo
 * 1e9 + 7.
 *
 * Two sequences are considered different if at least one element differs from each other.
 */

/**
 * @param {number} n
```

```
 * @param {number[]} rollMax
 * @return {number}
 */
var dieSimulator = function(n, rollMax) {
  const MOD = 1e9 + 7;
  const faces = 6;

  const dp = new Array(n + 1).fill().map(() => new Array(faces).fill(0));
  for (let face = 0; face < faces; face++) {
    dp[1][face] = 1;
  }

  for (let rolls = 2; rolls <= n; rolls++) {
    for (let face = 0; face < faces; face++) {
      const maxConsecutive = Math.min(rolls, rollMax[face]);
      let sequences = 0;

      for (let streak = 1; streak <= maxConsecutive; streak++) {
        if (streak === rolls) {
          sequences = (sequences + 1) % MOD;
        } else {
          let prevCombinations = 0;
          for (let prevFace = 0; prevFace < faces; prevFace++) {
            if (prevFace !== face) {
              prevCombinations = (prevCombinations + dp[rolls - streak][prevFace]) % MOD;
            }
          }
          sequences = (sequences + prevCombinations) % MOD;
        }
      }
      dp[rolls][face] = sequences;
    }
  }

  let result = 0;
  for (let face = 0; face < faces; face++) {
    result = (result + dp[n][face]) % MOD;
  }

  return result;
};
```

1224 Maximum Equal Frequency

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1224. Maximum Equal Frequency
 * https://leetcode.com/problems/maximum-equal-frequency/
 * Difficulty: Hard
 *
 * Given an array nums of positive integers, return the longest possible length of an array
prefix
 * of nums, such that it is possible to remove exactly one element from this prefix so that
every
 * number that has appeared in it will have the same number of occurrences.
 *
 * If after removing one element there are no remaining elements, it's still considered that
every
 * appeared number has the same number of ocurrences (0).
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxEqualFreq = function(nums) {
  const countMap = new Map();
  const freqMap = new Map();
```

```
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    const num = nums[i];
    const prevCount = countMap.get(num) || 0;
    const newCount = prevCount + 1;

    countMap.set(num, newCount);

    if (prevCount > 0) {
      freqMap.set(prevCount, (freqMap.get(prevCount) || 0) - 1);
      if (freqMap.get(prevCount) === 0) freqMap.delete(prevCount);
    }
    freqMap.set(newCount, (freqMap.get(newCount) || 0) + 1);

    if (freqMap.size === 1) {
      const [freq, occurrences] = [...freqMap.entries()][0];
      if (freq === 1 || occurrences === 1) result = i + 1;
    } else if (freqMap.size === 2) {
      const freqs = [...freqMap.keys()].sort((a, b) => a - b);
      if (freqs[0] === 1 && freqMap.get(freqs[0]) === 1) result = i + 1;
      if (freqs[1] === freqs[0] + 1 && freqMap.get(freqs[1]) === 1) result = i + 1;
    }
  }

  return result;
};
```

1227 Airplane Seat Assignment Probability

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1227. Airplane Seat Assignment Probability
 * https://leetcode.com/problems/airplane-seat-assignment-probability/
 * Difficulty: Medium
 *
 * n passengers board an airplane with exactly n seats. The first passenger has lost the
ticket and
 * picks a seat randomly. But after that, the rest of the passengers will:
 * - Take their own seat if it is still available, and
 * - Pick other seats randomly when they find their seat occupied
 *
 * Return the probability that the nth person gets his own seat.
 */

/**
 * @param {number} n
 * @return {number}
 */
var nthPersonGetsNthSeat = function(n) {
  return n === 1 ? 1 : 0.5;
};
```

1228 Missing Number In Arithmetic Progression

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1228. Missing Number In Arithmetic Progression
 * https://leetcode.com/problems/missing-number-in-arithmetic-progression/
 * Difficulty: Easy
 *
 * In some array arr, the values were in arithmetic progression: the values
 * arr[i + 1] - arr[i] are all equal for every 0 <= i < arr.length - 1.
```

```
 *
 * A value from arr was removed that was not the first or last value in the array.
 *
 * Given arr, return the removed value.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var missingNumber = function(arr) {
  const n = arr.length;
  const totalDiff = arr[n - 1] - arr[0];
  const expectedDiff = totalDiff / n;

  for (let i = 1; i < n; i++) {
    if (arr[i] - arr[i - 1] !== expectedDiff) {
      return arr[i - 1] + expectedDiff;
    }
  }

  return arr[0] + expectedDiff;
};
```

1229 Meeting Scheduler

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1229. Meeting Scheduler
 * https://leetcode.com/problems/meeting-scheduler/
 * Difficulty: Medium
 *
 * Given the availability time slots arrays slots1 and slots2 of two people and a meeting
 * duration duration, return the earliest time slot that works for both of them and is of
 * duration duration.
 *
 * If there is no common time slot that satisfies the requirements, return an empty array.
 *
 * The format of a time slot is an array of two elements [start, end] representing an
 * inclusive time range from start to end.
 *
 * It is guaranteed that no two availability slots of the same person intersect with each
 * other. That is, for any two time slots [start1, end1] and [start2, end2] of the same
 * person, either start1 > end2 or start2 > end1.
 */

/**
 * @param {number[][]} slots1
 * @param {number[][]} slots2
 * @param {number} duration
 * @return {number[]}
 */
var minAvailableDuration = function(slots1, slots2, duration) {
  slots1.sort((a, b) => a[0] - b[0]);
  slots2.sort((a, b) => a[0] - b[0]);

  let i = 0;
  let j = 0;

  while (i < slots1.length && j < slots2.length) {
    const overlapStart = Math.max(slots1[i][0], slots2[j][0]);
    const overlapEnd = Math.min(slots1[i][1], slots2[j][1]);

    if (overlapEnd - overlapStart >= duration) {
      return [overlapStart, overlapStart + duration];
    }

    if (slots1[i][1] < slots2[j][1]) {
      i++;
```

```
    } else {
      j++;
    }
  }

  return [];
};
```

1230 Toss Strange Coins

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1230. Toss Strange Coins
 * https://leetcode.com/problems/toss-strange-coins/
 * Difficulty: Medium
 *
 * You have some coins.  The i-th coin has a probability prob[i] of facing heads when tossed.
 *
 * Return the probability that the number of coins facing heads equals target if you toss
every
 * coin exactly once.
 */

/**
 * @param {number[]} prob
 * @param {number} target
 * @return {number}
 */
var probabilityOfHeads = function(prob, target) {
  const n = prob.length;
  const dp = new Array(target + 1).fill(0);
  dp[0] = 1;

  for (let i = 0; i < n; i++) {
    for (let j = Math.min(i + 1, target); j >= 1; j--) {
      dp[j] = dp[j] * (1 - prob[i]) + dp[j - 1] * prob[i];
    }
    dp[0] *= (1 - prob[i]);
  }

  return dp[target];
};
```

1231 Divide Chocolate

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1231. Divide Chocolate
 * https://leetcode.com/problems/divide-chocolate/
 * Difficulty: Hard
 *
 * You have one chocolate bar that consists of some chunks. Each chunk has its own sweetness
 * given by the array sweetness.
 *
 * You want to share the chocolate with your k friends so you start cutting the chocolate bar
 * into k + 1 pieces using k cuts, each piece consists of some consecutive chunks.
 *
 * Being generous, you will eat the piece with the minimum total sweetness and give the other
 * pieces to your friends.
 *
 * Find the maximum total sweetness of the piece you can get by cutting the chocolate bar
optimally.
 */
```

```
/**
 * @param {number[]} sweetness
 * @param {number} k
 * @return {number}
 */
var maximizeSweetness = function(sweetness, k) {
  let left = Math.min(...sweetness);
  let right = sweetness.reduce((sum, val) => sum + val, 0);
  let result = left;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);

    if (canDivide(mid)) {
      result = mid;
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;

  function canDivide(minSweetness) {
    let pieces = 0;
    let currentSum = 0;

    for (const chunk of sweetness) {
      currentSum += chunk;
      if (currentSum >= minSweetness) {
        pieces++;
        currentSum = 0;
      }
    }

    return pieces >= k + 1;
  }
};
```

1232 Check If It Is A Straight Line

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1232. Check If It Is a Straight Line
 * https://leetcode.com/problems/check-if-it-is-a-straight-line/
 * Difficulty: Easy
 *
 * You are given an array `coordinates`, `coordinates[i] = [x, y]`,
 * where `[x, y] represents the coordinate of a point.
 *
 * Check if these points make a straight line in the XY plane.
 */

/**
 * @param {number[][]} coordinates
 * @return {boolean}
 */
var checkStraightLine = function(coords) {
  const m = (coords[0][1] - coords[1][1]) / (coords[0][0] - coords[1][0]);
  const b = coords[0][1] - m * coords[0][0];

  return coords.every(coord => coord[1] === m * coord[0] + b);
};
```

1233 Remove Sub Folders From The Filesystem

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1233. Remove Sub-Folders from the Filesystem
 * https://leetcode.com/problems/remove-sub-folders-from-the-filesystem/
 * Difficulty: Medium
 *
 * Given a list of folders, remove all sub-folders in those folders
 * and return in any order the folders after removing.
 *
 * If a folder[i] is located within another folder[j], it is called
 * a sub-folder of it. The format of a path is one or more concatenated
 * strings of the form: / followed by one or more lowercase English
 * letters. For example, /leetcode and /leetcode/problems are valid
 * paths while an empty string and / are not.
 */

/**
 * @param {string[]} folder
 * @return {string[]}
 */
var removeSubfolders = function(folder) {
  const set = new Set(folder);
  return folder.filter(path => !getPathSegments(path).some(p => set.has(p)));
};

function getPathSegments(path) {
  const split = path.split('/');
  const segments = [split.shift()];
  while (split.length > 1) {
    segments.push(segments[segments.length - 1] + '/' + split.shift());
  }
  return segments;
}
```

1234 Replace The Substring For Balanced String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1234. Replace the Substring for Balanced String
 * https://leetcode.com/problems/replace-the-substring-for-balanced-string/
 * Difficulty: Medium
 *
 * You are given a string s of length n containing only four kinds of characters:
 * 'Q', 'W', 'E', and 'R'.
 *
 * A string is said to be balanced if each of its characters appears n / 4 times where n is
 * the length of the string.
 *
 * Return the minimum length of the substring that can be replaced with any other string of
 * the same length to make s balanced. If s is already balanced, return 0.
 */

/**
 * @param {string} s
 * @return {number}
 */
var balancedString = function(s) {
  const target = s.length / 4;
  const charCount = { Q: 0, W: 0, E: 0, R: 0 };

  for (const char of s) {
```

```
    charCount[char]++;
  }

  if (Object.values(charCount).every(count => count === target)) return 0;

  let result = s.length;
  let left = 0;

  for (let right = 0; right < s.length; right++) {
    charCount[s[right]]--;

    while (left <= right && Object.values(charCount).every(count => count <= target)) {
      result = Math.min(result, right - left + 1);
      charCount[s[left]]++;
      left++;
    }
  }

  return result;
};
```

1235 Maximum Profit In Job Scheduling

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1235. Maximum Profit in Job Scheduling
 * https://leetcode.com/problems/maximum-profit-in-job-scheduling/
 * Difficulty: Hard
 *
 * We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i],
 * obtaining a profit of profit[i].
 *
 * You're given the startTime, endTime and profit arrays, return the maximum profit you can
 * take such that there are no two jobs in the subset with overlapping time range.
 *
 * If you choose a job that ends at time X you will be able to start another job that starts
 * at time X.
 */

/**
 * @param {number[]} startTime
 * @param {number[]} endTime
 * @param {number[]} profit
 * @return {number}
 */
var jobScheduling = function(startTime, endTime, profit) {
  const jobCount = startTime.length;
  const indices = Array.from({ length: jobCount }, (_, i) => i)
    .sort((a, b) => startTime[a] - startTime[b]);

  const maxProfit = new Array(jobCount + 1).fill(0);

  for (let i = jobCount - 1; i >= 0; i--) {
    let left = i + 1;
    let right = jobCount - 1;
    const currentEnd = endTime[indices[i]];

    while (left <= right) {
      const mid = left + right >> 1;
      if (startTime[indices[mid]] < currentEnd) left = mid + 1;
      else right = mid - 1;
    }

    maxProfit[i] = Math.max(maxProfit[i + 1], profit[indices[i]] + maxProfit[left]);
  }

  return maxProfit[0];
};
```

1236 Web Crawler

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1236. Web Crawler
 * https://leetcode.com/problems/web-crawler/
 * Difficulty: Medium
 *
 * Given a url startUrl and an interface HtmlParser, implement a web crawler to crawl all
 * links that are under the same hostname as startUrl.
 *
 * Return all urls obtained by your web crawler in any order.
 *
 * Your crawler should:
 * - Start from the page: startUrl
 * - Call HtmlParser.getUrls(url) to get all urls from a webpage of given url.
 * - Do not crawl the same link twice.
 * - Explore only the links that are under the same hostname as startUrl.
 *
 * As shown in the example url above, the hostname is example.org. For simplicity sake, you
may
 * assume all urls use http protocol without any port specified. For example, the urls
 * http://leetcode.com/problems and http://leetcode.com/contest are under the same hostname,
 * while urls http://example.org/test and http://example.com/abc are not under the same
hostname.
 *
 * The HtmlParser interface is defined as such:
 * interface HtmlParser {
 *   // Return a list of all urls from a webpage of given url.
 *   public List<String> getUrls(String url);
 * }
 *
 * Below are two examples explaining the functionality of the problem, for custom testing
purposes
 * you'll have three variables urls, edges and startUrl. Notice that you will only have access
 to
 * startUrl in your code, while urls and edges are not directly accessible to you in code.
 *
 * Note: Consider the same URL with the trailing slash "/" as a different URL. For example,
 * "http://news.yahoo.com", and "http://news.yahoo.com/" are different urls.
 */

/**
 * @param {string} startUrl
 * @param {HtmlParser} htmlParser
 * @return {string[]}
*/
var crawl = function(startUrl, htmlParser) {
  const targetHostname = getHostname(startUrl);
  const visited = new Set();
  const queue = [startUrl];
  const result = [];

  while (queue.length > 0) {
    const currentUrl = queue.shift();

    if (visited.has(currentUrl)) continue;

    visited.add(currentUrl);
    result.push(currentUrl);

    const urls = htmlParser.getUrls(currentUrl);

    for (const url of urls) {
      if (!visited.has(url) && getHostname(url) === targetHostname) {
        queue.push(url);
      }
    }
```

```
  }

  return result;

  function getHostname(url) {
    return url.split('/')[2];
  }
};
```

1237 Find Positive Integer Solution For A Given Equation

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1237. Find Positive Integer Solution for a Given Equation
 * https://leetcode.com/problems/find-positive-integer-solution-for-a-given-equation/
 * Difficulty: Medium
 *
 * Given a callable function f(x, y) with a hidden formula and a value z, reverse engineer
 * the formula and return all positive integer pairs x and y where f(x,y) == z. You may
 * return the pairs in any order.
 *
 * While the exact formula is hidden, the function is monotonically increasing, i.e.:
 * - f(x, y) < f(x + 1, y)
 * - f(x, y) < f(x, y + 1)
 */

/**
 * // This is the CustomFunction's API interface.
 * // You should not implement it, or speculate about its implementation
 * function CustomFunction() {
 *     @param {integer, integer} x, y
 *     @return {integer}
 *     this.f = function(x, y) {
 *         ...
 *     };
 * };
 */

/**
 * @param {CustomFunction} customfunction
 * @param {integer} z
 * @return {integer[][]}
 */
function findSolution(customfunction, z) {
  const pairs = [];
  let left = 1;
  let right = 1000;

  while (left <= 1000 && right >= 1) {
    const value = customfunction.f(left, right);
    if (value === z) {
      pairs.push([left, right]);
      left++;
      right--;
    } else if (value < z) {
      left++;
    } else {
      right--;
    }
  }

  return pairs;
}
```

1238 Circular Permutation In Binary Representation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1238. Circular Permutation in Binary Representation
 * https://leetcode.com/problems/circular-permutation-in-binary-representation/
 * Difficulty: Medium
 *
 * Given 2 integers n and start. Your task is return any permutation p of (0,1,2.....,2^n -1)
 * such that:
 * - p[0] = start
 * - p[i] and p[i+1] differ by only one bit in their binary representation.
 * - p[0] and p[2^n -1] must also differ by only one bit in their binary representation.
 */

/**
 * @param {number} n
 * @param {number} start
 * @return {number[]}
 */
var circularPermutation = function(n, start) {
  return helper(n).map(num => num ^ start);
};

function helper(n) {
  const sequence = [0];
  for (let i = 0; i < n; i++) {
    const mirror = sequence.slice().reverse();
    const power = 1 << i;
    for (let j = 0; j < mirror.length; j++) {
      sequence.push(mirror[j] + power);
    }
  }
  return sequence;
}
```

1239 Maximum Length Of A Concatenated String With Unique Characters

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1239. Maximum Length of a Concatenated String with Unique Characters
 *
https://leetcode.com/problems/maximum-length-of-a-concatenated-string-with-unique-characters/
 * Difficulty: Medium
 *
 * You are given an array of strings arr. A string s is formed by the concatenation of a
subsequence
 * of arr that has unique characters.
 *
 * Return the maximum possible length of s.
 *
 * A subsequence is an array that can be derived from another array by deleting some or no
elements
 * without changing the order of the remaining elements.
 */

/**
 * @param {string[]} arr
 * @return {number}
 */
function maxLength(arr) {
  let result = 0;
  explore('', 0);
```

```
  return result;

  function explore(current, index) {
    result = Math.max(result, current.length);
    for (let i = index; i < arr.length; i++) {
      if (isUnique(current + arr[i])) {
        explore(current + arr[i], i + 1);
      }
    }
  }

  function isUnique(str) {
    return new Set(str).size === str.length;
  }
}
```

1240 Tiling A Rectangle With The Fewest Squares

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1240. Tiling a Rectangle with the Fewest Squares
 * https://leetcode.com/problems/tiling-a-rectangle-with-the-fewest-squares/
 * Difficulty: Hard
 *
 * Given a rectangle of size n x m, return the minimum number of integer-sided squares that
 * tile the rectangle.
 */

/**
 * @param {number} n
 * @param {number} m
 * @return {number}
 */
function tilingRectangle(n, m) {
  let result = n * m;
  const grid = new Array(n).fill().map(() => new Array(m).fill(false));
  explore(0);
  return result;

  function canPlaceSquare(row, col, size) {
    if (row + size > n || col + size > m) return false;
    for (let i = row; i < row + size; i++) {
      for (let j = col; j < col + size; j++) {
        if (grid[i][j]) return false;
      }
    }
    return true;
  }

  function fillSquare(row, col, size, value) {
    for (let i = row; i < row + size; i++) {
      for (let j = col; j < col + size; j++) {
        grid[i][j] = value;
      }
    }
  }

  function explore(count) {
    if (count >= result) return;
    let row = -1;
    let col = -1;
    for (let i = 0; i < n && row === -1; i++) {
      for (let j = 0; j < m; j++) {
        if (!grid[i][j]) {
          row = i;
          col = j;
          break;
        }
      }
```

```
    }
    if (row === -1) {
      result = Math.min(result, count);
      return;
    }
    for (let size = Math.min(n - row, m - col); size >= 1; size--) {
      if (canPlaceSquare(row, col, size)) {
        fillSquare(row, col, size, true);
        explore(count + 1);
        fillSquare(row, col, size, false);
      }
    }
  }
}
```

1243 Array Transformation

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1243. Array Transformation
 * https://leetcode.com/problems/array-transformation/
 * Difficulty: Easy
 *
 * Given an initial array arr, every day you produce a new array using the array of the
 * previous day.
 *
 * On the i-th day, you do the following operations on the array of day i-1 to produce
 * the array of day i:
 * - If an element is smaller than both its left neighbor and its right neighbor, then this
 *   element is incremented.
 * - If an element is bigger than both its left neighbor and its right neighbor, then this
 *   element is decremented.
 *
 * The first and last elements never change.
 *
 * After some days, the array does not change. Return that final array.
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var transformArray = function(arr) {
  let current = [...arr];

  while (true) {
    const next = [...current];
    let changed = false;

    for (let i = 1; i < current.length - 1; i++) {
      if (current[i] < current[i - 1] && current[i] < current[i + 1]) {
        next[i]++;
        changed = true;
      } else if (current[i] > current[i - 1] && current[i] > current[i + 1]) {
        next[i]--;
        changed = true;
      }
    }

    if (!changed) break;
    current = next;
  }

  return current;
};
```

1244 Design A Leaderboard

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1244. Design A Leaderboard
 * https://leetcode.com/problems/design-a-leaderboard/
 * Difficulty: Medium
 *
 * Design a Leaderboard class, which has 3 functions:
 * - addScore(playerId, score): Update the leaderboard by adding score to the given player's
 *   score. If there is no player with such id in the leaderboard, add him to the leaderboard
 *   with the given score.
 * - top(K): Return the score sum of the top K players.
 * - reset(playerId): Reset the score of the player with the given id to 0 (in other words
 *   erase it from the leaderboard). It is guaranteed that the player was added to the
leaderboard
 *   before calling this function.
 *
 * Initially, the leaderboard is empty.
 */


var Leaderboard = function() {
  this.scores = new Map();
};

/**
 * @param {number} playerId
 * @param {number} score
 * @return {void}
 */
Leaderboard.prototype.addScore = function(playerId, score) {
  this.scores.set(playerId, (this.scores.get(playerId) || 0) + score);
};

/**
 * @param {number} K
 * @return {number}
 */
Leaderboard.prototype.top = function(K) {
  const sortedScores = Array.from(this.scores.values()).sort((a, b) => b - a);
  return sortedScores.slice(0, K).reduce((sum, score) => sum + score, 0);
};

/**
 * @param {number} playerId
 * @return {void}
 */
Leaderboard.prototype.reset = function(playerId) {
  this.scores.delete(playerId);
};
```

1245 Tree Diameter

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1245. Tree Diameter
 * https://leetcode.com/problems/tree-diameter/
 * Difficulty: Medium
 *
 * The diameter of a tree is the number of edges in the longest path in that tree.
 *
 * There is an undirected tree of n nodes labeled from 0 to n - 1. You are given a 2D array
```

```
 * edges where edges.length == n - 1 and edges[i] = [ai, bi] indicates that there is an
 * undirected edge between nodes ai and bi in the tree.
 *
 * Return the diameter of the tree.
 */

/**
 * @param {number[][]} edges
 * @return {number}
 */
var treeDiameter = function(edges) {
  if (edges.length === 0) return 0;

  const graph = new Map();
  for (const [a, b] of edges) {
    if (!graph.has(a)) graph.set(a, []);
    if (!graph.has(b)) graph.set(b, []);
    graph.get(a).push(b);
    graph.get(b).push(a);
  }

  const [farthestFromStart] = bfs(0);
  const [, diameter] = bfs(farthestFromStart);

  return diameter;

  function bfs(start) {
    const visited = new Set();
    const queue = [[start, 0]];
    visited.add(start);
    let farthestNode = start;
    let maxDistance = 0;

    while (queue.length > 0) {
      const [node, distance] = queue.shift();

      if (distance > maxDistance) {
        maxDistance = distance;
        farthestNode = node;
      }

      for (const neighbor of graph.get(node) || []) {
        if (!visited.has(neighbor)) {
          visited.add(neighbor);
          queue.push([neighbor, distance + 1]);
        }
      }
    }

    return [farthestNode, maxDistance];
  }
};
```

1246 Palindrome Removal

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1246. Palindrome Removal
 * https://leetcode.com/problems/palindrome-removal/
 * Difficulty: Hard
 *
 * You are given an integer array arr.
 *
 * In one move, you can select a palindromic subarray arr[i], arr[i + 1], ..., arr[j]
 * where i <= j, and remove that subarray from the given array. Note that after removing
 * a subarray, the elements on the left and on the right of that subarray move to fill
 * the gap left by the removal.
 *
 * Return the minimum number of moves needed to remove all numbers from the array.
```

```
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var minimumMoves = function(arr) {
  const n = arr.length;
  const dp = new Array(n).fill().map(() => new Array(n).fill(Infinity));

  for (let i = 0; i < n; i++) {
    dp[i][i] = 1;
  }

  for (let length = 2; length <= n; length++) {
    for (let i = 0; i <= n - length; i++) {
      const j = i + length - 1;

      if (arr[i] === arr[j]) {
        if (length === 2) {
          dp[i][j] = 1;
        } else {
          dp[i][j] = dp[i + 1][j - 1];
        }
      }

      for (let k = i; k < j; k++) {
        dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k + 1][j]);
      }
    }
  }

  return dp[0][n - 1];
};
```

1247 Minimum Swaps To Make Strings Equal

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1247. Minimum Swaps to Make Strings Equal
 * https://leetcode.com/problems/minimum-swaps-to-make-strings-equal/
 * Difficulty: Medium
 *
 * You are given two strings s1 and s2 of equal length consisting of letters "x" and "y" only.
 * Your task is to make these two strings equal to each other. You can swap any two characters
 * that belong to different strings, which means: swap s1[i] and s2[j].
 *
 * Return the minimum number of swaps required to make s1 and s2 equal, or return -1 if it is
 * impossible to do so.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {number}
 */
function minimumSwap(s1, s2) {
  let xyCount = 0;
  let yxCount = 0;

  for (let i = 0; i < s1.length; i++) {
    if (s1[i] === 'x' && s2[i] === 'y') xyCount++;
    if (s1[i] === 'y' && s2[i] === 'x') yxCount++;
  }

  if ((xyCount + yxCount) % 2 !== 0) return -1;

  return Math.floor(xyCount / 2) + Math.floor(yxCount / 2) + (xyCount % 2) * 2;
}
```

1248 Count Number Of Nice Subarrays

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1248. Count Number of Nice Subarrays
 * https://leetcode.com/problems/count-number-of-nice-subarrays/
 * Difficulty: Medium
 *
 * Given an array of integers nums and an integer k. A continuous subarray is called nice if
there
 * are k odd numbers on it.
 *
 * Return the number of nice sub-arrays.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var numberOfSubarrays = function(nums, k) {
  const prefix = new Map([[0, 1]]);
  let oddCount = 0;
  let result = 0;

  for (const num of nums) {
    oddCount += num % 2;
    result += prefix.get(oddCount - k) || 0;
    prefix.set(oddCount, (prefix.get(oddCount) || 0) + 1);
  }

  return result;
};
```

1249 Minimum Remove To Make Valid Parentheses

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1249. Minimum Remove to Make Valid Parentheses
 * https://leetcode.com/problems/minimum-remove-to-make-valid-parentheses/
 * Difficulty: Medium
 *
 * Given a string s of '(' , ')' and lowercase English characters.
 *
 * Your task is to remove the minimum number of parentheses ( '(' or ')',
 * in any positions ) so that the resulting parentheses string is valid
 * and return any valid string.
 *
 * Formally, a parentheses string is valid if and only if:
 * - It is the empty string, contains only lowercase characters, or
 * - It can be written as AB (A concatenated with B), where A and B are valid strings, or
 * - It can be written as (A), where A is a valid string.
 */

/**
 * @param {string} s
 * @return {string}
 */
var minRemoveToMakeValid = function(s) {
  const inputArray = s.split('');
  const stack = [];

  inputArray.forEach((letter, index) => {
```

```
    if (letter === '(') {
      stack.push(index);
    } else if (letter === ')') {
      if (stack.length) {
        stack.pop();
      } else {
        inputArray[index] = '';
      }
    }
  });

  stack.forEach(index => inputArray[index] = '');
  return inputArray.join('');
};
```

1250 Check If It Is A Good Array

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1250. Check If It Is a Good Array
 * https://leetcode.com/problems/check-if-it-is-a-good-array/
 * Difficulty: Hard
 *
 * Given an array nums of positive integers. Your task is to select some subset of nums,
multiply
 * each element by an integer and add all these numbers. The array is said to be good if you
can
 * obtain a sum of 1 from the array by any possible subset and multiplicand.
 *
 * Return True if the array is good otherwise return False.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isGoodArray = function(nums) {
  let gcd = nums[0];

  for (let num of nums) {
    while (num !== 0) {
      const temp = gcd % num;
      gcd = num;
      num = temp;
    }
    if (gcd === 1) return true;
  }

  return gcd === 1;
};
```

1252 Cells With Odd Values In A Matrix

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1252. Cells with Odd Values in a Matrix
 * https://leetcode.com/problems/cells-with-odd-values-in-a-matrix/
 * Difficulty: Easy
 *
 * Given `n` and `m` which are the dimensions of a matrix
 * initialized by zeros and given an array `indices`
 * where `indices[i] = [ri, ci]`.
 * For each pair of `[ri, ci]` you have to increment all
```

```
 * cells in row `ri` and column `ci` by 1.
 *
 * Return the number of cells with odd values in the
 * matrix after applying the increment to all `indices`.
 */

/**
 * @param {number} n
 * @param {number} m
 * @param {number[][]} indices
 * @return {number}
 */
var oddCells = function(n, m, indices) {
  const matrix = new Array(n).fill().map(_ => new Array(m).fill(0));

  indices.forEach(indice => {
    const [row, column] = indice;
    matrix[row].forEach((value, index) => matrix[row][index] = value + 1);
    matrix.forEach(row => row[column] = row[column] + 1);
  });

  return matrix.reduce((count, row) => {
    return count + row.filter(value => value % 2 !== 0).length;
  }, 0);
};
```

1253 Reconstruct A 2 Row Binary Matrix

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1253. Reconstruct a 2-Row Binary Matrix
 * https://leetcode.com/problems/reconstruct-a-2-row-binary-matrix/
 * Difficulty: Medium
 *
 * Given the following details of a matrix with n columns and 2 rows:
 * - The matrix is a binary matrix, which means each element in the matrix can be 0 or 1.
 * - The sum of elements of the 0-th(upper) row is given as upper.
 * - The sum of elements of the 1-st(lower) row is given as lower.
 * - The sum of elements in the i-th column(0-indexed) is colsum[i], where colsum is given as
an
 *   integer array with length n.
 *
 * Your task is to reconstruct the matrix with upper, lower and colsum.
 * Return it as a 2-D integer array.
 * If there are more than one valid solution, any of them will be accepted.
 * If no valid solution exists, return an empty 2-D array.
 */

/**
 * @param {number} upper
 * @param {number} lower
 * @param {number[]} colsum
 * @return {number[][]}
 */
var reconstructMatrix = function(upper, lower, colsum) {
  const n = colsum.length;
  const matrix = [[], []];
  let upperLeft = upper;
  let lowerLeft = lower;

  for (let i = 0; i < n; i++) {
    if (colsum[i] === 2) {
      matrix[0][i] = 1;
      matrix[1][i] = 1;
      upperLeft--;
      lowerLeft--;
    } else {
      matrix[0][i] = 0;
      matrix[1][i] = 0;
```

```
    }
  }

  if (upperLeft < 0 || lowerLeft < 0) return [];

  for (let i = 0; i < n; i++) {
    if (colsum[i] === 1) {
      if (upperLeft > 0) {
        matrix[0][i] = 1;
        matrix[1][i] = 0;
        upperLeft--;
      } else if (lowerLeft > 0) {
        matrix[0][i] = 0;
        matrix[1][i] = 1;
        lowerLeft--;
      } else {
        return [];
      }
    }
  }

  return upperLeft === 0 && lowerLeft === 0 ? matrix : [];
};
```

1254 Number Of Closed Islands

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1254. Number of Closed Islands
 * https://leetcode.com/problems/number-of-closed-islands/
 * Difficulty: Medium
 *
 * Given a 2D grid consists of 0s (land) and 1s (water).  An island is a maximal
4-directionally
 * connected group of 0s and a closed island is an island totally (all left, top, right,
bottom)
 * surrounded by 1s.
 *
 * Return the number of closed islands.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var closedIsland = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;

  for (let i = 0; i < rows; i++) {
    markBorderConnected(i, 0);
    markBorderConnected(i, cols - 1);
  }
  for (let j = 0; j < cols; j++) {
    markBorderConnected(0, j);
    markBorderConnected(rows - 1, j);
  }

  let result = 0;
  for (let i = 1; i < rows - 1; i++) {
    for (let j = 1; j < cols - 1; j++) {
      result += countClosed(i, j);
    }
  }

  return result;

  function markBorderConnected(row, col) {
    if (row < 0 || row >= rows || col < 0 || col >= cols || grid[row][col] !== 0) return;
```

```
    grid[row][col] = 1;
    markBorderConnected(row - 1, col);
    markBorderConnected(row + 1, col);
    markBorderConnected(row, col - 1);
    markBorderConnected(row, col + 1);
  }

  function countClosed(row, col) {
    if (row < 0 || row >= rows || col < 0 || col >= cols || grid[row][col] !== 0) return 0;
    grid[row][col] = 1;
    countClosed(row - 1, col);
    countClosed(row + 1, col);
    countClosed(row, col - 1);
    countClosed(row, col + 1);
    return 1;
  }
};
```

1255 Maximum Score Words Formed By Letters

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1255. Maximum Score Words Formed by Letters
 * https://leetcode.com/problems/maximum-score-words-formed-by-letters/
 * Difficulty: Hard
 *
 * Given a list of words, list of  single letters (might be repeating) and score of every
character.
 *
 * Return the maximum score of any valid set of words formed by using the given letters
(words[i]
 * cannot be used two or more times).
 *
 * It is not necessary to use all characters in letters and each letter can only be used once.
 * Score of letters 'a', 'b', 'c', ... ,'z' is given by score[0], score[1], ... , score[25]
 * respectively.
 */

/**
 * @param {string[]} words
 * @param {character[]} letters
 * @param {number[]} score
 * @return {number}
 */
var maxScoreWords = function(words, letters, score) {
  const letterCount = new Array(26).fill(0);
  for (const letter of letters) {
    letterCount[letter.charCodeAt(0) - 97]++;
  }
  return calculateMaxScore(0, letterCount);

  function calculateMaxScore(index, available) {
    if (index === words.length) return 0;

    const skipScore = calculateMaxScore(index + 1, available);
    const word = words[index];
    const tempCount = [...available];
    let canUse = true;
    let wordScore = 0;

    for (const char of word) {
      const charIndex = char.charCodeAt(0) - 97;
      if (tempCount[charIndex] === 0) {
        canUse = false;
        break;
      }
      tempCount[charIndex]--;
      wordScore += score[charIndex];
    }
```

```
    const useScore = canUse ? wordScore + calculateMaxScore(index + 1, tempCount) : 0;
    return Math.max(skipScore, useScore);
  }
};
```

1256 Encode Number

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1256. Encode Number
 * https://leetcode.com/problems/encode-number/
 * Difficulty: Medium
 *
 * Given a non-negative integer num, Return its encoding string.
 *
 * The encoding is done by converting the integer to a string using a secret function that
 * you should deduce from the following table:
 */

/**
 * @param {number} num
 * @return {string}
 */
var encode = function(num) {
  return (num + 1).toString(2).slice(1);
};
```

1257 Smallest Common Region

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1257. Smallest Common Region
 * https://leetcode.com/problems/smallest-common-region/
 * Difficulty: Medium
 *
 * You are given some lists of regions where the first region of each list directly contains
 * all other regions in that list.
 *
 * If a region x contains a region y directly, and region y contains region z directly, then
 * region x is said to contain region z indirectly. Note that region x also indirectly
contains
 * all regions indirectly containd in y.
 *
 * Naturally, if a region x contains (either directly or indirectly) another region y, then x
 * is bigger than or equal to y in size. Also, by definition, a region x contains itself.
 *
 * Given two regions: region1 and region2, return the smallest region that contains both of
them.
 *
 * It is guaranteed the smallest region exists.
 */

/**
 * @param {string[][]} regions
 * @param {string} region1
 * @param {string} region2
 * @return {string}
 */
var findSmallestRegion = function(regions, region1, region2) {
  const map = new Map();

  for (const region of regions) {
```

```
    const parent = region[0];
    for (let i = 1; i < region.length; i++) {
      map.set(region[i], parent);
    }
  }

  const path1 = getPath(region1);
  const path2 = getPath(region2);
  const set = new Set(path1);

  for (const ancestor of path2) {
    if (set.has(ancestor)) {
      return ancestor;
    }
  }

  function getPath(region) {
    const path = [];
    let current = region;
    while (current) {
      path.push(current);
      current = map.get(current);
    }
    return path;
  }
};
```

1258 Synonymous Sentences

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1258. Synonymous Sentences
 * https://leetcode.com/problems/synonymous-sentences/
 * Difficulty: Medium
 *
 * You are given a list of equivalent string pairs synonyms where synonyms[i] = [si, ti]
 * indicates that si and ti are equivalent strings. You are also given a sentence text.
 *
 * Return all possible synonymous sentences sorted lexicographically.
 */

/**
 * @param {string[][]} synonyms
 * @param {string} text
 * @return {string[]}
 */
var generateSentences = function(synonyms, text) {
  const graph = new Map();

  for (const [word1, word2] of synonyms) {
    if (!graph.has(word1)) graph.set(word1, []);
    if (!graph.has(word2)) graph.set(word2, []);
    graph.get(word1).push(word2);
    graph.get(word2).push(word1);
  }

  const words = text.split(' ');
  const allCombinations = [];
  backtrack(0, []);
  return allCombinations.sort();

  function findSynonyms(word) {
    if (!graph.has(word)) return [word];

    const visited = new Set();
    const queue = [word];
    const synonymGroup = [];

    while (queue.length > 0) {
```

```
      const current = queue.shift();
      if (visited.has(current)) continue;

      visited.add(current);
      synonymGroup.push(current);

      for (const neighbor of graph.get(current)) {
        if (!visited.has(neighbor)) {
          queue.push(neighbor);
        }
      }
    }

    return synonymGroup.sort();
  }

  function backtrack(index, currentSentence) {
    if (index === words.length) {
      allCombinations.push(currentSentence.join(' '));
      return;
    }

    const synonyms = findSynonyms(words[index]);
    for (const synonym of synonyms) {
      currentSentence.push(synonym);
      backtrack(index + 1, currentSentence);
      currentSentence.pop();
    }
  }
};
```

1259 Handshakes That Dont Cross

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1259. Handshakes That Don't Cross
 * https://leetcode.com/problems/handshakes-that-dont-cross/
 * Difficulty: Hard
 *
 * You are given an even number of people numPeople that stand around a circle and each
 * person shakes hands with someone else so that there are numPeople / 2 handshakes total.
 *
 * Return the number of ways these handshakes could occur such that none of the handshakes
cross.
 *
 * Since the answer could be very large, return it modulo 109 + 7.
 */

/**
 * @param {number} numPeople
 * @return {number}
 */
var numberOfWays = function(numPeople) {
  const MOD = 1e9 + 7;
  const n = numPeople / 2;
  const dp = new Array(n + 1).fill(0n);
  dp[0] = 1n;

  for (let i = 1; i <= n; i++) {
    for (let j = 0; j < i; j++) {
      dp[i] = (dp[i] + dp[j] * dp[i - 1 - j]) % BigInt(MOD);
    }
  }

  return Number(dp[n]);
};
```

1260 Shift 2D Grid

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1260. Shift 2D Grid
 * https://leetcode.com/problems/shift-2d-grid/
 * Difficulty: Easy
 *
 * Given a 2D grid of size m x n and an integer k. You need to shift the grid k times.
 *
 * In one shift operation:
 * - Element at grid[i][j] moves to grid[i][j + 1].
 * - Element at grid[i][n - 1] moves to grid[i + 1][0].
 * - Element at grid[m - 1][n - 1] moves to grid[0][0].
 *
 * Return the 2D grid after applying shift operation k times.
 */

/**
 * @param {number[][]} grid
 * @param {number} k
 * @return {number[][]}
 */
var shiftGrid = function(grid, k) {
  const rows = grid.length;
  const cols = grid[0].length;
  const total = rows * cols;

  if (k === 0 || k % total === 0) return grid;

  const effectiveShifts = k % total;
  const flat = grid.flat();
  const shifted = [...flat.slice(-effectiveShifts), ...flat.slice(0, -effectiveShifts)];
  const result = [];
  for (let i = 0; i < rows; i++) {
    result.push(shifted.slice(i * cols, (i + 1) * cols));
  }

  return result;
};
```

1261 Find Elements In A Contaminated Binary Tree

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1261. Find Elements in a Contaminated Binary Tree
 * https://leetcode.com/problems/find-elements-in-a-contaminated-binary-tree/
 * Difficulty: Medium
 *
 * Given a binary tree with the following rules:
 * 1. root.val == 0
 * 2. For any treeNode:
 *   1. If treeNode.val has a value x and treeNode.left != null,
 *      then treeNode.left.val == 2 * x + 1
 *   2. If treeNode.val has a value x and treeNode.right != null,
 *      then treeNode.right.val == 2 * x + 2
 *
 * Now the binary tree is contaminated, which means all treeNode.val have
 * been changed to -1.
 *
 * Implement the FindElements class:
 * - FindElements(TreeNode* root) Initializes the object with a contaminated binary
 *   tree and recovers it.
```

```
 * - bool find(int target) Returns true if the target value exists in the recovered
 *   binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 */
var FindElements = function(root) {
  this.values = new Set();
  root.val = 0;

  const traverse = node => {
    if (!node) return;
    this.values.add(node.val);
    if (node.left) node.left.val = 2 * node.val + 1;
    if (node.right) node.right.val = 2 * node.val + 2;
    traverse(node.left);
    traverse(node.right);
  };

  traverse(root);
};

/**
 * @param {number} target
 * @return {boolean}
 */
FindElements.prototype.find = function(target) {
  return this.values.has(target);
};
```

1262 Greatest Sum Divisible By Three

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1262. Greatest Sum Divisible by Three
 * https://leetcode.com/problems/greatest-sum-divisible-by-three/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the maximum possible sum of elements of the array such
 * that it is divisible by three.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSumDivThree = function(nums) {
  const dp = [0, -Infinity, -Infinity];

  for (const num of nums) {
    const prev = [...dp];
    for (let remainder = 0; remainder < 3; remainder++) {
      dp[(remainder + num) % 3] = Math.max(dp[(remainder + num) % 3], prev[remainder] + num);
    }
  }

  return dp[0];
};
```

1263 Minimum Moves To Move A Box To Their Target Location

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1263. Minimum Moves to Move a Box to Their Target Location
 * https://leetcode.com/problems/minimum-moves-to-move-a-box-to-their-target-location/
 * Difficulty: Hard
 *
 * A storekeeper is a game in which the player pushes boxes around in a warehouse trying to
get
 * them to target locations.
 *
 * The game is represented by an m x n grid of characters grid where each element is a wall,
floor,
 * or box.
 *
 * Your task is to move the box 'B' to the target position 'T' under the following rules:
 * - The character 'S' represents the player. The player can move up, down, left, right in
grid if
 *   it is a floor (empty cell).
 * - The character '.' represents the floor which means a free cell to walk.
 * - The character '#' represents the wall which means an obstacle (impossible to walk there).
 * - There is only one box 'B' and one target cell 'T' in the grid.
 * - The box can be moved to an adjacent free cell by standing next to the box and then moving
 in
 *   the direction of the box. This is a push.
 * - The player cannot walk through the box.
 *
 * Return the minimum number of pushes to move the box to the target. If there is no way to
reach
 * the target, return -1.
 */

/**
 * @param {character[][]} grid
 * @return {number}
 */
var minPushBox = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  let player;
  let box;
  let target;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 'S') player = [i, j];
      if (grid[i][j] === 'B') box = [i, j];
      if (grid[i][j] === 'T') target = [i, j];
    }
  }

  const queue = [[...box, ...player, 0]];
  const seen = new Set([`${box}-${player}`]);
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  function canReach(src, dst, fixedBox) {
    const visited = new Set();
    const stack = [src];

    while (stack.length) {
      const [r, c] = stack.pop();
      if (r === dst[0] && c === dst[1]) return true;
      if (visited.has(`${r}-${c}`)) continue;
      visited.add(`${r}-${c}`);

      for (const [dr, dc] of directions) {
        const nr = r + dr;
```

```
        const nc = c + dc;
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || grid[nr][nc] === '#'
            || (nr === fixedBox[0] && nc === fixedBox[1])) continue;
        stack.push([nr, nc]);
      }
    }
    return false;
  }

  while (queue.length) {
    const [boxRow, boxCol, playerRow, playerCol, pushes] = queue.shift();
    if (boxRow === target[0] && boxCol === target[1]) return pushes;

    for (const [dr, dc] of directions) {
      const newBoxRow = boxRow + dr;
      const newBoxCol = boxCol + dc;
      const newPlayerRow = boxRow - dr;
      const newPlayerCol = boxCol - dc;

      if (newBoxRow < 0 || newBoxRow >= rows || newBoxCol < 0 || newBoxCol >= cols
          || grid[newBoxRow][newBoxCol] === '#') continue;
      if (!canReach([playerRow, playerCol], [newPlayerRow, newPlayerCol], [boxRow, boxCol])) {
        continue;
      }

      const state = `${newBoxRow}-${newBoxCol}-${newPlayerRow}-${newPlayerCol}`;
      if (seen.has(state)) continue;

      seen.add(state);
      queue.push([newBoxRow, newBoxCol, newPlayerRow, newPlayerCol, pushes + 1]);
    }
  }

  return -1;
};
```

1265 Print Immutable Linked List In Reverse

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1265. Print Immutable Linked List in Reverse
 * https://leetcode.com/problems/print-immutable-linked-list-in-reverse/
 * Difficulty: Medium
 *
 * You are given an immutable linked list, print out all values of each node in reverse
 * with the help of the following interface:
 * - ImmutableListNode: An interface of immutable linked list, you are given the head of the
list.
 *
 * You need to use the following functions to access the linked list (you can't access the
 * ImmutableListNode directly):
 * - ImmutableListNode.printValue(): Print value of the current node.
 * - ImmutableListNode.getNext(): Return the next node.
 *
 * The input is only given to initialize the linked list internally. You must solve this
problem
 * without modifying the linked list. In other words, you must operate the linked list using
 * only the mentioned APIs.
 */

/**
 * @param {ImmutableListNode} head
 * @return {void}
 */
var printLinkedListInReverse = function(head) {
  if (!head) return;
  printLinkedListInReverse(head.getNext());
  head.printValue();
};
```

1266 Minimum Time Visiting All Points

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1266. Minimum Time Visiting All Points
 * https://leetcode.com/problems/minimum-time-visiting-all-points/
 * Difficulty: Easy
 *
 * On a 2D plane, there are n points with integer coordinates points[i] = [xi, yi]. Return the
 * minimum time in seconds to visit all the points in the order given by points.
 *
 * You can move according to these rules:
 * - In 1 second, you can either:
 *   - move vertically by one unit,
 *   - move horizontally by one unit, or
 *   - move diagonally sqrt(2) units (in other words, move one unit vertically then one unit
 *     horizontally in 1 second).
 * - You have to visit the points in the same order as they appear in the array.
 * - You are allowed to pass through points that appear later in the order, but these do not
 *   count as visits.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var minTimeToVisitAllPoints = function(points) {
  return points.reduce((steps, point, i) => steps + (!points[i + 1] ? 0 : Math.max(
    Math.abs(point[0] - points[i + 1][0]), Math.abs(point[1] - points[i + 1][1])
  )), 0);
};
```

1267 Count Servers That Communicate

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1267. Count Servers that Communicate
 * https://leetcode.com/problems/count-servers-that-communicate/
 * Difficulty: Medium
 *
 * You are given a map of a server center, represented as a m * n integer matrix grid,
 * where 1 means that on that cell there is a server and 0 means that it is no server.
 * Two servers are said to communicate if they are on the same row or on the same column.
 *
 * Return the number of servers that communicate with any other server.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var countServers = function(grid) {
  const rows = new Array(grid.length).fill(0);
  const columns = new Array(grid[0].length).fill(0);

  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[i].length; j++) {
      if (grid[i][j]) {
        rows[i]++;
        columns[j]++;
      }
    }
  }
```

```
  let result = 0;
  for (let i = 0; i < grid.length; i++) {
    for (let j = 0; j < grid[i].length; j++) {
      if (!grid[i][j] || (rows[i] < 2 && columns[j] < 2)) {
        continue;
      }
      result++;
    }
  }

  return result;
};
```

1268 Search Suggestions System

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1268. Search Suggestions System
 * https://leetcode.com/problems/search-suggestions-system/
 * Difficulty: Medium
 *
 * You are given an array of strings products and a string searchWord.
 *
 * Design a system that suggests at most three product names from products after each
 * character of searchWord is typed. Suggested products should have common prefix with
 * searchWord. If there are more than three products with a common prefix return the
 * three lexicographically minimums products.
 *
 * Return a list of lists of the suggested products after each character of searchWord
 * is typed.
 */

/**
 * @param {string[]} products
 * @param {string} searchWord
 * @return {string[][]}
 */
var suggestedProducts = function(products, searchWord) {
  products.sort();
  const result = new Array(searchWord.length);
  for (let i = 0; i < searchWord.length; i++) {
    products = products.filter((word) => word[i] === searchWord[i]);
    result[i] = products.slice(0, 3);
  }
  return result;
};
```

1269 Number Of Ways To Stay In The Same Place After Some Steps

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1269. Number of Ways to Stay in the Same Place After Some Steps
 * https://leetcode.com/problems/number-of-ways-to-stay-in-the-same-place-after-some-steps/
 * Difficulty: Hard
 *
 * You have a pointer at index 0 in an array of size arrLen. At each step, you can move 1
position
 * to the left, 1 position to the right in the array, or stay in the same place (The pointer
should
 * not be placed outside the array at any time).
 *
 * Given two integers steps and arrLen, return the number of ways such that your pointer is
```

```
still
 * at index 0 after exactly steps steps. Since the answer may be too large, return it modulo
 * 109 + 7.
 */

/**
 * @param {number} steps
 * @param {number} arrLen
 * @return {number}
 */
var numWays = function(steps, arrLen) {
  const MOD = 1e9 + 7;
  const maxPos = Math.min(steps, arrLen - 1);
  let dp = new Array(maxPos + 1).fill(0);
  dp[0] = 1;

  for (let step = 1; step <= steps; step++) {
    const prev = dp;
    dp = new Array(maxPos + 1).fill(0);
    for (let pos = 0; pos <= maxPos; pos++) {
      dp[pos] = prev[pos];
      if (pos > 0) dp[pos] = (dp[pos] + prev[pos - 1]) % MOD;
      if (pos < maxPos) dp[pos] = (dp[pos] + prev[pos + 1]) % MOD;
    }
  }

  return dp[0];
};
```

1271 Hexspeak

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1271. Hexspeak
 * https://leetcode.com/problems/hexspeak/
 * Difficulty: Easy
 *
 * A decimal number can be converted to its Hexspeak representation by first converting it to
 * an uppercase hexadecimal string, then replacing all occurrences of the digit '0' with the
 * letter 'O', and the digit '1' with the letter 'I'. Such a representation is valid if and
 * only if it consists only of the letters in the set {'A', 'B', 'C', 'D', 'E', 'F', 'I',
'O'}.
 *
 * Given a string num representing a decimal integer n, return the Hexspeak representation of
 * n if it is valid, otherwise return "ERROR".
 */

/**
 * @param {string} num
 * @return {string}
 */
var toHexspeak = function(num) {
  const hex = BigInt(num).toString(16).toUpperCase();
  const validChars = new Set(['0', '1', 'A', 'B', 'C', 'D', 'E', 'F']);

  for (const char of hex) {
    if (!validChars.has(char)) {
      return 'ERROR';
    }
  }

  return hex.replace(/0/g, 'O').replace(/1/g, 'I');
};
```

1272 Remove Interval

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1272. Remove Interval
 * https://leetcode.com/problems/remove-interval/
 * Difficulty: Medium
 *
 * A set of real numbers can be represented as the union of several disjoint intervals,
 * where each interval is in the form [a, b). A real number x is in the set if one of
 * its intervals [a, b) contains x (i.e. a <= x < b).
 *
 * You are given a sorted list of disjoint intervals intervals representing a set of
 * real numbers as described above, where intervals[i] = [ai, bi] represents the interval
 * [ai, bi). You are also given another interval toBeRemoved.
 *
 * Return the set of real numbers with the interval toBeRemoved removed from intervals.
 * In other words, return the set of real numbers such that every x in the set is in
 * intervals but not in toBeRemoved. Your answer should be a sorted list of disjoint
 * intervals as described above.
 */

/**
 * @param {number[][]} intervals
 * @param {number[]} toBeRemoved
 * @return {number[][]}
 */
var removeInterval = function(intervals, toBeRemoved) {
  const result = [];
  const [removeStart, removeEnd] = toBeRemoved;

  for (const [start, end] of intervals) {
    if (end <= removeStart || start >= removeEnd) {
      result.push([start, end]);
    } else {
      if (start < removeStart) {
        result.push([start, removeStart]);
      }
      if (end > removeEnd) {
        result.push([removeEnd, end]);
      }
    }
  }

  return result;
};
```

1273 Delete Tree Nodes

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1273. Delete Tree Nodes
 * https://leetcode.com/problems/delete-tree-nodes/
 * Difficulty: Medium
 *
 * A tree rooted at node 0 is given as follows:
 * - The number of nodes is nodes;
 * - The value of the ith node is value[i];
 * - The parent of the ith node is parent[i].
 *
 * Remove every subtree whose sum of values of nodes is zero.
 *
 * Return the number of the remaining nodes in the tree.
```

```
 */

/**
 * @param {number} nodes
 * @param {number[]} parent
 * @param {number[]} value
 * @return {number}
 */
var deleteTreeNodes = function(nodes, parent, value) {
  const children = new Array(nodes).fill().map(() => []);

  for (let i = 1; i < nodes; i++) {
    children[parent[i]].push(i);
  }

  return dfs(0)[1];

  function dfs(node) {
    let subtreeSum = value[node];
    let subtreeCount = 1;

    for (const child of children[node]) {
      const [childSum, childCount] = dfs(child);
      subtreeSum += childSum;
      subtreeCount += childCount;
    }

    if (subtreeSum === 0) {
      return [0, 0];
    }

    return [subtreeSum, subtreeCount];
  }
};
```

1274 Number Of Ships In A Rectangle

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1274. Number of Ships in a Rectangle
 * https://leetcode.com/problems/number-of-ships-in-a-rectangle/
 * Difficulty: Hard
 *
 * Each ship is located at an integer point on the sea represented by a cartesian plane, and
 * each integer point may contain at most 1 ship.
 *
 * You have a function Sea.hasShips(topRight, bottomLeft) which takes two points as arguments
 * and returns true If there is at least one ship in the rectangle represented by the two
 * points, including on the boundary.
 *
 * Given two points: the top right and bottom left corners of a rectangle, return the number
 * of ships present in that rectangle. It is guaranteed that there are at most 10 ships in
 * that rectangle.
 *
 * Submissions making more than 400 calls to hasShips will be judged Wrong Answer. Also, any
 * solutions that attempt to circumvent the judge will result in disqualification.
 */

/**
 * @param {Sea} sea
 * @param {integer[]} topRight
 * @param {integer[]} bottomLeft
 * @return {integer}
 */
var countShips = function(sea, topRight, bottomLeft) {
  const [x2, y2] = topRight;
  const [x1, y1] = bottomLeft;

  if (x1 > x2 || y1 > y2 || !sea.hasShips(topRight, bottomLeft)) {
```

```
    return 0;
  }

  if (x1 === x2 && y1 === y2) {
    return 1;
  }

  const midX = Math.floor((x1 + x2) / 2);
  const midY = Math.floor((y1 + y2) / 2);

  return countShips(sea, [midX, midY], [x1, y1])
    + countShips(sea, [x2, midY], [midX + 1, y1])
    + countShips(sea, [midX, y2], [x1, midY + 1])
    + countShips(sea, [x2, y2], [midX + 1, midY + 1]);
};
```

1275 Find Winner On A Tic Tac Toe Game

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1275. Find Winner on a Tic Tac Toe Game
 * https://leetcode.com/problems/find-winner-on-a-tic-tac-toe-game/
 * Difficulty: Easy
 *
 * Tic-tac-toe is played by two players A and B on a 3 x 3 grid. The rules of Tic-Tac-Toe are:
 * - Players take turns placing characters into empty squares ' '.
 * - The first player A always places 'X' characters, while the second player B always places
 *   'O' characters.
 * - 'X' and 'O' characters are always placed into empty squares, never on filled ones.
 * - The game ends when there are three of the same (non-empty) character filling any row,
 *   column, or diagonal.
 * - The game also ends if all squares are non-empty.
 * - No more moves can be played if the game is over.
 *
 * Given a 2D integer array moves where moves[i] = [rowi, coli] indicates that the ith move
 * will be played on grid[rowi][coli]. return the winner of the game if it exists (A or B).
 * In case the game ends in a draw return "Draw". If there are still movements to play return
 * "Pending".
 *
 * You can assume that moves is valid (i.e., it follows the rules of Tic-Tac-Toe), the grid is
 * initially empty, and A will play first.
 */

/**
 * @param {number[][]} moves
 * @return {string}
 */
var tictactoe = function(moves) {
  const rows = new Array(3).fill(0);
  const cols = new Array(3).fill(0);
  let diagonal = 0;
  let antiDiagonal = 0;

  for (let i = 0; i < moves.length; i++) {
    const value = i % 2 === 0 ? 1 : -1;
    const [row, col] = moves[i];

    rows[row] += value;
    cols[col] += value;
    if (row === col) diagonal += value;
    if (row + col === 2) antiDiagonal += value;

    if ([rows[row], cols[col], diagonal, antiDiagonal].includes(3)) return 'A';
    if ([rows[row], cols[col], diagonal, antiDiagonal].includes(-3)) return 'B';
  }

  return moves.length === 9 ? 'Draw' : 'Pending';
};
```

1276 Number Of Burgers With No Waste Of Ingredients

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1276. Number of Burgers with No Waste of Ingredients
 * https://leetcode.com/problems/number-of-burgers-with-no-waste-of-ingredients/
 * Difficulty: Medium
 *
 * Given two integers tomatoSlices and cheeseSlices. The ingredients of different burgers are
 * as follows:
 * - Jumbo Burger: 4 tomato slices and 1 cheese slice.
 * - Small Burger: 2 Tomato slices and 1 cheese slice.
 *
 * Return [total_jumbo, total_small] so that the number of remaining tomatoSlices equal to 0
and
 * the number of remaining cheeseSlices equal to 0. If it is not possible to make the
remaining
 * tomatoSlices and cheeseSlices equal to 0 return [].
 */

/**
 * @param {number} tomatoSlices
 * @param {number} cheeseSlices
 * @return {number[]}
 */
var numOfBurgers = function(tomatoSlices, cheeseSlices) {
  const jumbo = Math.floor((tomatoSlices - 2 * cheeseSlices) / (4 - 2));
  const small = cheeseSlices - jumbo;

  if (jumbo < 0 || small < 0 || jumbo * 4 + small * 2 !== tomatoSlices) {
    return [];
  }

  return [jumbo, small];
};
```

1277 Count Square Submatrices With All Ones

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1277. Count Square Submatrices with All Ones
 * https://leetcode.com/problems/count-square-submatrices-with-all-ones/
 * Difficulty: Medium
 *
 * Given a m * n matrix of ones and zeros, return how many square submatrices have all ones.
 */

/**
 * @param {number[][]} matrix
 * @return {number}
 */
var countSquares = function(matrix) {
  const rows = matrix.length;
  const cols = matrix[0].length;
  const dp = Array.from({ length: rows }, () => new Array(cols).fill(0));
  let result = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (matrix[i][j] === 1) {
        if (i === 0 || j === 0) {
          dp[i][j] = 1;
        } else {
```

```
          dp[i][j] = Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1;
        }
        result += dp[i][j];
      }
    }
  }

  return result;
};
```

1278 Palindrome Partitioning Iii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1278. Palindrome Partitioning III
 * https://leetcode.com/problems/palindrome-partitioning-iii/
 * Difficulty: Hard
 *
 * You are given a string s containing lowercase letters and an integer k. You need to:
 * - First, change some characters of s to other lowercase English letters.
 * - Then divide s into k non-empty disjoint substrings such that each substring is a
palindrome.
 *
 * Return the minimal number of characters that you need to change to divide the string.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var palindromePartition = function(s, k) {
  const n = s.length;
  const cost = Array.from({ length: n }, () => new Array(n).fill(0));

  for (let len = 2; len <= n; len++) {
    for (let i = 0; i + len <= n; i++) {
      const j = i + len - 1;
      cost[i][j] = cost[i + 1][j - 1] + (s[i] === s[j] ? 0 : 1);
    }
  }

  const dp = Array.from({ length: n + 1 }, () => new Array(k + 1).fill(Infinity));
  dp[0][0] = 0;

  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= Math.min(i, k); j++) {
      for (let m = 0; m < i; m++) {
        dp[i][j] = Math.min(dp[i][j], dp[m][j - 1] + cost[m][i - 1]);
      }
    }
  }

  return dp[n][k];
};
```

1281 Subtract The Product And Sum Of Digits Of An Integer

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1281. Subtract the Product and Sum of Digits of an Integer
 * https://leetcode.com/problems/subtract-the-product-and-sum-of-digits-of-an-integer/
 * Difficulty: Easy
```

```
 *
 * Given an integer number n, return the difference between the product of its digits and
 * the sum of its digits.
 */

/**
 * @param {number} n
 * @return {number}
 */
var subtractProductAndSum = function(n) {
  let product = 1;
  let sum = 0;

  while (n > 0) {
    const digit = n % 10;
    product *= digit;
    sum += digit;
    n = Math.floor(n / 10);
  }

  return product - sum;
};
```

1282 Group The People Given The Group Size They Belong To

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1282. Group the People Given the Group Size They Belong To
 * https://leetcode.com/problems/group-the-people-given-the-group-size-they-belong-to/
 * Difficulty: Medium
 *
 * There are n people that are split into some unknown number of groups. Each person is
labeled
 * with a unique ID from 0 to n - 1.
 *
 * You are given an integer array groupSizes, where groupSizes[i] is the size of the group
that
 * person i is in. For example, if groupSizes[1] = 3, then person 1 must be in a group of size
 3.
 *
 * Return a list of groups such that each person i is in a group of size groupSizes[i].
 *
 * Each person should appear in exactly one group, and every person must be in a group. If
there
 * are multiple answers, return any of them. It is guaranteed that there will be at least one
 * valid solution for the given input.
 */

/**
 * @param {number[]} groupSizes
 * @return {number[][]}
 */
var groupThePeople = function(groupSizes) {
  const map = new Map();
  const grouped = [];

  for (let i = 0; i < groupSizes.length; i++) {
    const size = groupSizes[i];

    map.set(size, map.has(size) ? [...map.get(size), i] : [i]);

    if (map.get(size).length === size) {
      grouped.push(map.get(size));
      map.delete(size);
    }
  }

  return grouped;
};
```

1283 Find The Smallest Divisor Given A Threshold

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1283. Find the Smallest Divisor Given a Threshold
 * https://leetcode.com/problems/find-the-smallest-divisor-given-a-threshold/
 * Difficulty: Medium
 *
 * Given an array of integers nums and an integer threshold, we will choose a positive integer
 * divisor, divide all the array by it, and sum the division's result. Find the smallest
divisor
 * such that the result mentioned above is less than or equal to threshold.
 *
 * Each result of the division is rounded to the nearest integer greater than or equal to that
 * element. (For example: 7/3 = 3 and 10/2 = 5).
 *
 * The test cases are generated so that there will be an answer.
 */

/**
 * @param {number[]} nums
 * @param {number} threshold
 * @return {number}
 */
var smallestDivisor = function(nums, threshold) {
  let left = 1;
  let right = Math.max(...nums);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    const sum = nums.reduce((acc, num) => acc + Math.ceil(num / mid), 0);

    if (sum <= threshold) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;
};
```

1284 Minimum Number Of Flips To Convert Binary Matrix To Zero Matrix

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1284. Minimum Number of Flips to Convert Binary Matrix to Zero Matrix
 *
https://leetcode.com/problems/minimum-number-of-flips-to-convert-binary-matrix-to-zero-matrix/
 * Difficulty: Hard
 *
 * Given a m x n binary matrix mat. In one step, you can choose one cell and flip it and all
the
 * four neighbors of it if they exist (Flip is changing 1 to 0 and 0 to 1). A pair of cells
are
 * called neighbors if they share one edge.
 *
 * Return the minimum number of steps required to convert mat to a zero matrix or -1 if you
cannot.
 *
 * A binary matrix is a matrix with all cells equal to 0 or 1 only.
```

```
 *
 * A zero matrix is a matrix with all cells equal to 0.
 */

/**
 * @param {number[][]} mat
 * @return {number}
 */
var minFlips = function(mat) {
  const rows = mat.length;
  const cols = mat[0].length;
  const target = 0;
  const start = mat.flat().reduce((acc, val, idx) => acc | (val << idx), 0);
  const queue = [[start, 0]];
  const seen = new Set([start]);
  const directions = [[0, 0], [0, 1], [0, -1], [1, 0], [-1, 0]];

  while (queue.length) {
    const [state, steps] = queue.shift();

    if (state === target) return steps;

    for (let i = 0; i < rows; i++) {
      for (let j = 0; j < cols; j++) {
        let nextState = state;
        for (const [di, dj] of directions) {
          const ni = i + di;
          const nj = j + dj;
          if (ni >= 0 && ni < rows && nj >= 0 && nj < cols) {
            const pos = ni * cols + nj;
            nextState ^= (1 << pos);
          }
        }
        if (!seen.has(nextState)) {
          seen.add(nextState);
          queue.push([nextState, steps + 1]);
        }
      }
    }
  }

  return -1;
};
```

1286 Iterator For Combination

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1286. Iterator for Combination
 * https://leetcode.com/problems/iterator-for-combination/
 * Difficulty: Medium
 *
 * Design the CombinationIterator class:
 * - CombinationIterator(string characters, int combinationLength) Initializes the object with
 a
 *   string characters of sorted distinct lowercase English letters and a number
combinationLength
 *   as arguments.
 * - next() Returns the next combination of length combinationLength in lexicographical order.
 * - hasNext() Returns true if and only if there exists a next combination.
 */

/**
 * @param {string} characters
 * @param {number} combinationLength
 */
var CombinationIterator = function(characters, combinationLength) {
  this.chars = characters;
  this.length = combinationLength;
```

```
  this.indices = new Array(combinationLength).fill(0).map((_, i) => i);
  this.hasMore = true;
};

/**
 * @return {string}
 */
CombinationIterator.prototype.next = function() {
  const result = this.indices.map(i => this.chars[i]).join('');

  this.hasMore = false;
  for (let i = this.length - 1; i >= 0; i--) {
    if (this.indices[i] < this.chars.length - (this.length - i)) {
      this.indices[i]++;
      for (let j = i + 1; j < this.length; j++) {
        this.indices[j] = this.indices[j - 1] + 1;
      }
      this.hasMore = true;
      break;
    }
  }

  return result;
};

/**
 * @return {boolean}
 */
CombinationIterator.prototype.hasNext = function() {
  return this.hasMore;
};
```

1287 Element Appearing More Than 25 In Sorted Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1287. Element Appearing More Than 25% In Sorted Array
 * https://leetcode.com/problems/element-appearing-more-than-25-in-sorted-array/
 * Difficulty: Easy
 *
 * Given an integer array sorted in non-decreasing order,
 * there is exactly one integer in the array that occurs more than 25% of the time.
 *
 * Return that integer.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var findSpecialInteger = function(arr) {
  const limit = arr.length * 0.25;
  const offset = limit % 1 === 0 ? limit : Math.ceil(limit) - 1;

  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === arr[i + offset]) {
      return arr[i];
    }
  }
};
```

1288 Remove Covered Intervals

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1288. Remove Covered Intervals
 * https://leetcode.com/problems/remove-covered-intervals/
 * Difficulty: Medium
 *
 * Given an array intervals where intervals[i] = [li, ri] represent the interval [li, ri),
remove
 * all intervals that are covered by another interval in the list.
 *
 * The interval [a, b) is covered by the interval [c, d) if and only if c <= a and b <= d.
 *
 * Return the number of remaining intervals.
 */

/**
 * @param {number[][]} intervals
 * @return {number}
 */
var removeCoveredIntervals = function(intervals) {
  intervals.sort((a, b) => a[0] - b[0] || b[1] - a[1]);

  let result = 1;
  let currentEnd = intervals[0][1];

  for (let i = 1; i < intervals.length; i++) {
    if (intervals[i][1] > currentEnd) {
      result++;
      currentEnd = intervals[i][1];
    }
  }

  return result;
};
```

1289 Minimum Falling Path Sum Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1289. Minimum Falling Path Sum II
 * https://leetcode.com/problems/minimum-falling-path-sum-ii/
 * Difficulty: Hard
 *
 * Given an n x n integer matrix grid, return the minimum sum of a falling path with non-zero
 * shifts.
 *
 * A falling path with non-zero shifts is a choice of exactly one element from each row of
grid
 * such that no two elements chosen in adjacent rows are in the same column.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minFallingPathSum = function(grid) {
  const n = grid.length;
  let prevRow = [...grid[0]];

  for (let row = 1; row < n; row++) {
    const currRow = new Array(n);
    for (let col = 0; col < n; col++) {
      let minPrev = Infinity;
      for (let prevCol = 0; prevCol < n; prevCol++) {
        if (prevCol !== col) {
          minPrev = Math.min(minPrev, prevRow[prevCol]);
        }
      }
      currRow[col] = grid[row][col] + minPrev;
```

```
    }
    prevRow = currRow;
  }

  return Math.min(...prevRow);
};
```

1290 Convert Binary Number In A Linked List To Integer

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1290. Convert Binary Number in a Linked List to Integer
 * https://leetcode.com/problems/convert-binary-number-in-a-linked-list-to-integer/
 * Difficulty: Easy
 *
 * Given `head` which is a reference node to a singly-linked list.
 * The value of each node in the linked list is either 0 or 1.
 * The linked list holds the binary representation of a number.
 * Return the decimal value of the number in the linked list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val) {
 *     this.val = val;
 *     this.next = null;
 * }
 */
/**
 * @param {ListNode} head
 * @return {number}
 */
var getDecimalValue = function(head) {
  let binary = String(head.val);

  while (head.next !== null) {
    head = head.next;
    binary += head.val;
  }

  return parseInt(binary, 2);
};
```

1291 Sequential Digits

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1291. Sequential Digits
 * https://leetcode.com/problems/sequential-digits/
 * Difficulty: Medium
 *
 * An integer has sequential digits if and only if each digit in the number is one more than
the previous digit.
 * Return a sorted list of all the integers in the range [low, high] inclusive that have
sequential digits.
 */

/**
 * @param {number} low
 * @param {number} high
 * @return {number[]}
 */
```

```
var sequentialDigits = function(low, high) {
  return getSequentialNumbers().filter(n => n >= low && n <= high);
};

function getSequentialNumbers() {
  const numbers = new Set([]);

  for (let i = 0; i < 10; i++) {
    for (let j = 9; j > 0 && j > i + 1; j--) {
      numbers.add(+'123456789'.slice(i, j));
    }
  }

  return [...numbers].sort((a, b) => a - b);
}
```

1292 Maximum Side Length Of A Square With Sum Less Than Or Equal To Thr

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1292. Maximum Side Length of a Square with Sum Less than or Equal to Threshold
 * https://leetcode.com/problems/maximum-side-length-of-a-square-with-sum-less-than-or-equal-t
o-threshold/
 * Difficulty: Medium
 *
 * Given a `m x n` matrix `mat` and an integer `threshold`.
 * Return the maximum side-length of a square with a sum less than or equal to `threshold` or
return 0 if there is no such square.
 */

/**
 * @param {number[][]} mat
 * @param {number} threshold
 * @return {number}
 */
var maxSideLength = function(mat, threshold) {
  let max = 0;

  loop1: for (let i = 0; i < mat[0].length; i++) {
    loop2: for (let j = 0; j < mat.length; j++) {
      if (!mat[j + 1] || !mat[j][i + 1]) { break loop2; }
      loop3: for (let s = 1; s < 1000; s++) {
        if (!mat[j + s] || !mat[j][i + s] || getSquareSum(mat, i, j, s + 1) > threshold) {
break loop3; }
        max = Math.max(s + 1, max);
      }
    }
  }

  return max;
};

function getSquareSum(mat, i, j, size) {
  let sum = 0;
  for (let s = 0; s < size; s++) {
    mat[j + s].slice(i, i + size).forEach(v => sum += v);
  }
  return sum;
}
```

1293 Shortest Path In A Grid With Obstacles Elimination

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1293. Shortest Path in a Grid with Obstacles Elimination
 * https://leetcode.com/problems/shortest-path-in-a-grid-with-obstacles-elimination/
 * Difficulty: Hard
 *
 * You are given an m x n integer matrix grid where each cell is either 0 (empty) or 1
(obstacle).
 * You can move up, down, left, or right from and to an empty cell in one step.
 *
 * Return the minimum number of steps to walk from the upper left corner (0, 0) to the lower
right
 * corner (m - 1, n - 1) given that you can eliminate at most k obstacles. If it is not
possible to
 * find such walk return -1.
 */

function shortestPath(grid, k) {
  const rows = grid.length;
  const cols = grid[0].length;
  if (rows === 1 && cols === 1) return 0;

  let steps = 1;
  let queue = [[0, 0, k]];
  const visited = Array.from({ length: rows }, () => new Int16Array(cols).fill(-1));
  const directions = [0, -1, 0, 1, 0];

  while (queue.length) {
    const nextLevel = [];
    for (const [row, col, obstaclesLeft] of queue) {
      for (let i = 0; i < 4; i++) {
        const newRow = row + directions[i];
        const newCol = col + directions[i + 1];

        if (newRow < 0 || newRow === rows || newCol < 0
            || newCol === cols || visited[newRow][newCol] >= obstaclesLeft) continue;

        if (newRow === rows - 1 && newCol === cols - 1) return steps;

        const remainingObstacles = grid[newRow][newCol] ? obstaclesLeft - 1 : obstaclesLeft;
        visited[newRow][newCol] = remainingObstacles;
        nextLevel.push([newRow, newCol, remainingObstacles]);
      }
    }
    steps++;
    queue = nextLevel;
  }

  return -1;
}
```

1295 Find Numbers With Even Number Of Digits

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1295. Find Numbers with Even Number of Digits
 * https://leetcode.com/problems/find-numbers-with-even-number-of-digits/
 * Difficulty: Easy
 *
 * Given an array nums of integers, return how many of them contain an even
 * number of digits.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findNumbers = function(nums) {
  return nums.filter(num => String(num).length % 2 === 0).length;
};
```

1296 Divide Array In Sets Of K Consecutive Numbers

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1296. Divide Array in Sets of K Consecutive Numbers
 * https://leetcode.com/problems/divide-array-in-sets-of-k-consecutive-numbers/
 * Difficulty: Medium
 *
 * Given an array of integers nums and a positive integer k, find whether it is
 * possible to divide this array into sets of k consecutive numbers.
 *
 * Return true if it is possible. Otherwise, return false.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
var isPossibleDivide = function(nums, k) {
  if (nums.length % k) {
    return false;
  }

  const map = {};
  const set = new Set(nums);
  nums.forEach(x => map[x] ? map[x]++ : map[x] = 1);

  let count = nums.length / k;
  while (count--) {
    let min = Math.min(...set);
    for (let i = min; i < min + k; i++) {
      if (!map[i]) {
        return false;
      }
      if (!--map[i]) {
        set.delete(i);
      }
    }
  }

  return true;
};
```

1297 Maximum Number Of Occurrences Of A Substring

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1297. Maximum Number of Occurrences of a Substring
 * https://leetcode.com/problems/maximum-number-of-occurrences-of-a-substring/
 * Difficulty: Medium
 *
 * Given a string s, return the maximum number of ocurrences of
 * any substring under the following rules:
 *
 * - The number of unique characters in the substring must be less
 *   than or equal to maxLetters.
 * - The substring size must be between minSize and maxSize inclusive.
 */

/**
```

```
 * @param {string} s
 * @param {number} maxLetters
 * @param {number} minSize
 * @param {number} maxSize
 * @return {number}
 */
var maxFreq = function(s, maxLetters, minSize, maxSize) {
  const occurrences = new Map();

  for (let i = 0; i <= s.length - minSize; i++) {
    const string = s.substr(i, minSize);
    if (new Set(string.split('')).size <= maxLetters) {
      occurrences.set(string, occurrences.has(string) ? occurrences.get(string) + 1 : 1);
    }
  }

  return (Array.from(occurrences).sort((a, b) => b[1] - a[1])[0] || [])[1] || 0;
};
```

1298 Maximum Candies You Can Get From Boxes

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1298. Maximum Candies You Can Get from Boxes
 * https://leetcode.com/problems/maximum-candies-you-can-get-from-boxes/
 * Difficulty: Hard
 *
 * You have n boxes labeled from 0 to n - 1. You are given four arrays: status, candies, keys,
 and
 * containedBoxes where:
 * - status[i] is 1 if the ith box is open and 0 if the ith box is closed,
 * - candies[i] is the number of candies in the ith box,
 * - keys[i] is a list of the labels of the boxes you can open after opening the ith box.
 * - containedBoxes[i] is a list of the boxes you found inside the ith box.
 *
 * You are given an integer array initialBoxes that contains the labels of the boxes you
initially
 * have. You can take all the candies in any open box and you can use the keys in it to open
new
 * boxes and you also can use the boxes you find in it.
 *
 * Return the maximum number of candies you can get following the rules above.
 */

/**
 * @param {number[]} status
 * @param {number[]} candies
 * @param {number[][]} keys
 * @param {number[][]} containedBoxes
 * @param {number[]} initialBoxes
 * @return {number}
 */
var maxCandies = function(status, candies, keys, containedBoxes, initialBoxes) {
  let totalCandies = 0;
  const queue = [...initialBoxes];
  const availableKeys = new Set();
  const unopenedBoxes = new Set();

  while (queue.length) {
    const currentBox = queue.shift();

    if (status[currentBox]) {
      totalCandies += candies[currentBox];
      for (const key of keys[currentBox]) availableKeys.add(key);
      for (const box of containedBoxes[currentBox]) queue.push(box);
    } else {
      unopenedBoxes.add(currentBox);
    }
```

```
    for (const box of unopenedBoxes) {
      if (availableKeys.has(box)) {
        status[box] = 1;
        unopenedBoxes.delete(box);
        queue.push(box);
      }
    }
  }

  return totalCandies;
};
```

1299 Replace Elements With Greatest Element On Right Side

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1299. Replace Elements with Greatest Element on Right Side
 * https://leetcode.com/problems/replace-elements-with-greatest-element-on-right-side/
 * Difficulty: Easy
 *
 * Given an array arr, replace every element in that array with the greatest element among the
 * elements to its right, and replace the last element with -1.
 *
 * After doing so, return the array.
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var replaceElements = function(arr) {
  let maxRight = -1;
  for (let i = arr.length - 1; i >= 0; i--) {
    const current = arr[i];
    arr[i] = maxRight;
    maxRight = Math.max(maxRight, current);
  }
  return arr;
};
```

1300 Sum Of Mutated Array Closest To Target

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1300. Sum of Mutated Array Closest to Target
 * https://leetcode.com/problems/sum-of-mutated-array-closest-to-target/
 * Difficulty: Medium
 *
 * Given an integer array arr and a target value target, return the integer value such that
when
 * we change all the integers larger than value in the given array to be equal to value, the
 * sum of the array gets as close as possible (in absolute difference) to target.
 *
 * In case of a tie, return the minimum such integer.
 *
 * Notice that the answer is not neccesarilly a number from arr.
 */

/**
 * @param {number[]} arr
 * @param {number} target
 * @return {number}
```

```
 */
var findBestValue = function(arr, target) {
  arr.sort((a, b) => a - b);
  let prefixSum = 0;
  const length = arr.length;

  for (let i = 0; i < length; i++) {
    const remaining = length - i;
    const avg = (target - prefixSum) / remaining;

    if (avg <= arr[i]) {
      const floorVal = Math.floor(avg);
      const ceilVal = Math.ceil(avg);
      const floorSum = prefixSum + floorVal * remaining;
      const ceilSum = prefixSum + ceilVal * remaining;

      return Math.abs(floorSum - target) <= Math.abs(ceilSum - target) ? floorVal : ceilVal;
    }

    prefixSum += arr[i];
  }

  return arr[length - 1];
};
```

1301 Number Of Paths With Max Score

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1301. Number of Paths with Max Score
 * https://leetcode.com/problems/number-of-paths-with-max-score/
 * Difficulty: Hard
 *
 * You are given a square board of characters. You can move on the board starting at the
bottom
 * right square marked with the character 'S'.
 *
 * You need to reach the top left square marked with the character 'E'. The rest of the
squares
 * are labeled either with a numeric character 1, 2, ..., 9 or with an obstacle 'X'. In one
move
 * you can go up, left or up-left (diagonally) only if there is no obstacle there.
 *
 * Return a list of two integers: the first integer is the maximum sum of numeric characters
you
 * can collect, and the second is the number of such paths that you can take to get that
maximum
 * sum, taken modulo 10^9 + 7.
 *
 * In case there is no path, return [0, 0].
 */

/**
 * @param {string[]} board
 * @return {number[]}
 */
var pathsWithMaxScore = function(board) {
  const n = board.length;
  const mod = 1000000007;
  const dp = Array.from({ length: n }, () =>
    new Array(n).fill([-Infinity, 0])
  );

  dp[n-1][n-1] = [0, 1];

  for (let row = n-1; row >= 0; row--) {
    for (let col = n-1; col >= 0; col--) {
      if (board[row][col] === 'X') {
        dp[row][col] = [-Infinity, 0];
```

```
        continue;
      }

      if (row === n-1 && col === n-1) continue;

      const value = board[row][col] === 'E' ? 0 : Number(board[row][col]);
      const directions = [[0, 1], [1, 0], [1, 1]];

      for (const [dr, dc] of directions) {
        const nextRow = row + dr;
        const nextCol = col + dc;

        if (nextRow >= n || nextCol >= n) continue;
        if (dp[nextRow][nextCol][1] === 0) continue;

        const score = dp[nextRow][nextCol][0] + value;
        if (score > dp[row][col][0]) {
          dp[row][col] = [score, dp[nextRow][nextCol][1]];
        } else if (score === dp[row][col][0]) {
          dp[row][col][1] = (dp[row][col][1] + dp[nextRow][nextCol][1]) % mod;
        }
      }
    }
  }

  return dp[0][0][1] === 0 ? [0, 0] : dp[0][0];
};
```

1302 Deepest Leaves Sum

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1302. Deepest Leaves Sum
 * https://leetcode.com/problems/deepest-leaves-sum/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the sum of values of its deepest leaves.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var deepestLeavesSum = function(root) {
  let maxDepth = 0;
  let sumAtMaxDepth = 0;

  function traverse(node, depth) {
    if (!node) return;

    if (depth > maxDepth) {
      maxDepth = depth;
      sumAtMaxDepth = node.val;
    } else if (depth === maxDepth) {
      sumAtMaxDepth += node.val;
    }

    traverse(node.left, depth + 1);
    traverse(node.right, depth + 1);
  }
```

```
  traverse(root, 0);
  return sumAtMaxDepth;
};
```

1304 Find N Unique Integers Sum Up To Zero

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1304. Find N Unique Integers Sum up to Zero
 * https://leetcode.com/problems/find-n-unique-integers-sum-up-to-zero/
 * Difficulty: Easy
 *
 * Given an integer n, return any array containing
 * n unique integers such that they add up to 0.
 */

/**
 * @param {number} n
 * @return {number[]}
 */
var sumZero = function(n) {
  const result = n % 2 !== 0 ? [0] : [];
  while (result.length < n) {
    result.push(result.length + 1, (result.length + 1) * -1);
  }
  return result;
};
```

1305 All Elements In Two Binary Search Trees

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1305. All Elements in Two Binary Search Trees
 * https://leetcode.com/problems/all-elements-in-two-binary-search-trees/
 * Difficulty: Medium
 *
 * Given two binary search trees root1 and root2, return a list containing all the integers
from
 * both trees sorted in ascending order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root1
 * @param {TreeNode} root2
 * @return {number[]}
 */
var getAllElements = function(root1, root2) {
  const values = [];

  inorder(root1);
  inorder(root2);

  return values.sort((a, b) => a - b);

  function inorder(node) {
```

```
    if (!node) return;
    inorder(node.left);
    values.push(node.val);
    inorder(node.right);
  }
};
```

1306 Jump Game Iii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1306. Jump Game III
 * https://leetcode.com/problems/jump-game-iii/
 * Difficulty: Medium
 *
 * Given an array of non-negative integers arr, you are initially positioned at start index of
 * the array. When you are at index i, you can jump to i + arr[i] or i - arr[i], check if you
 * can reach any index with value 0.
 *
 * Notice that you can not jump outside of the array at any time.
 */

/**
 * @param {number[]} arr
 * @param {number} start
 * @return {boolean}
 */
var canReach = function(arr, start) {
  const visited = new Set();
  const queue = [start];

  while (queue.length) {
    const current = queue.shift();

    if (arr[current] === 0) return true;
    if (visited.has(current)) continue;

    visited.add(current);

    const jump = arr[current];
    const nextRight = current + jump;
    const nextLeft = current - jump;

    if (nextRight < arr.length) queue.push(nextRight);
    if (nextLeft >= 0) queue.push(nextLeft);
  }

  return false;
};
```

1307 Verbal Arithmetic Puzzle

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1307. Verbal Arithmetic Puzzle
 * https://leetcode.com/problems/verbal-arithmetic-puzzle/
 * Difficulty: Hard
 *
 * Given an equation, represented by words on the left side and the result on the right side.
 *
 * You need to check if the equation is solvable under the following rules:
 * - Each character is decoded as one digit (0 - 9).
 * - No two characters can map to the same digit.
```

```
 * - Each words[i] and result are decoded as one number without leading zeros.
 * - Sum of numbers on the left side (words) will equal to the number on the right side
(result).
 *
 * Return true if the equation is solvable, otherwise return false.
 */

/**
 * @param {string[]} words
 * @param {string} result
 * @return {boolean}
 */
var isSolvable = function(words, result) {
  const charToDigit = new Map();
  const usedDigits = new Set();
  const leadingChars = new Set();

  if (words.some(word => word.length > result.length)) return false;

  words.forEach(word => {
    if (word.length > 1) leadingChars.add(word[0]);
  });
  if (result.length > 1) leadingChars.add(result[0]);

  return solve(0, 0, 0);

  function solve(pos, wordIdx, carry) {
    if (pos >= result.length) return carry === 0;

    if (wordIdx <= words.length - 1) {
      const word = words[wordIdx];
      if (pos >= word.length) return solve(pos, wordIdx + 1, carry);

      const char = word[word.length - 1 - pos];
      if (charToDigit.has(char)) return solve(pos, wordIdx + 1, carry +
charToDigit.get(char));

      for (let digit = leadingChars.has(char) ? 1 : 0; digit <= 9; digit++) {
        if (usedDigits.has(digit)) continue;
        usedDigits.add(digit);
        charToDigit.set(char, digit);
        if (solve(pos, wordIdx + 1, carry + digit)) return true;
        usedDigits.delete(digit);
        charToDigit.delete(char);
      }
      return false;
    }

    const sumDigit = carry % 10;
    const resultChar = result[result.length - 1 - pos];

    if (!charToDigit.has(resultChar)) {
      if (usedDigits.has(sumDigit) || (pos === result.length - 1 && sumDigit === 0)) return
false;
      charToDigit.set(resultChar, sumDigit);
      usedDigits.add(sumDigit);
      const success = solve(pos + 1, 0, (carry - sumDigit) / 10);
      charToDigit.delete(resultChar);
      usedDigits.delete(sumDigit);
      return success;
    }

    return charToDigit.get(resultChar) === sumDigit && solve(pos + 1, 0, (carry - sumDigit) /
10);
  }
};
```

1309 Decrypt String From Alphabet To Integer Mapping

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1309. Decrypt String from Alphabet to Integer Mapping
 * https://leetcode.com/problems/decrypt-string-from-alphabet-to-integer-mapping/
 * Difficulty: Easy
 *
 * Given a string s formed by digits ('0' - '9') and '#' .
 * We want to map s to English lowercase characters as follows:
 *
 * - Characters ('a' to 'i') are represented by ('1' to '9') respectively.
 * - Characters ('j' to 'z') are represented by ('10#' to '26#') respectively.
 *
 * Return the string formed after mapping.
 *
 * It's guaranteed that a unique mapping will always exist.
 */

/**
 * @param {string} s
 * @return {string}
 */
var freqAlphabets = function(s) {
  return s.replace(/\d{2}#|\d/g, match => {
    return String.fromCharCode('a'.charCodeAt() + parseInt(match, 10) - 1);
  });
};
```

1310 Xor Queries Of A Subarray

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1310. XOR Queries of a Subarray
 * https://leetcode.com/problems/xor-queries-of-a-subarray/
 * Difficulty: Medium
 *
 * You are given an array arr of positive integers. You are also given the array queries where
 * queries[i] = [lefti, righti].
 *
 * For each query i compute the XOR of elements from lefti to righti (that is, arr[lefti] XOR
 * arr[lefti + 1] XOR ... XOR arr[righti]).
 *
 * Return an array answer where answer[i] is the answer to the ith query.
 */

/**
 * @param {number[]} arr
 * @param {number[][]} queries
 * @return {number[]}
 */
var xorQueries = function(arr, queries) {
  const options = [0];
  arr.forEach((num, i) => options.push(options[i] ^ num));
  return queries.map(([left, right]) => options[right + 1] ^ options[left]);
};
```

1311 Get Watched Videos By Your Friends

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1311. Get Watched Videos by Your Friends
 * https://leetcode.com/problems/get-watched-videos-by-your-friends/
 * Difficulty: Medium
```

```
 *
 * There are n people, each person has a unique id between 0 and n-1. Given the arrays
watchedVideos
 * and friends, where watchedVideos[i] and friends[i] contain the list of watched videos and
the
 * list of friends respectively for the person with id = i.
 *
 * Level 1 of videos are all watched videos by your friends, level 2 of videos are all watched
 * videos by the friends of your friends and so on. In general, the level k of videos are all
 * watched videos by people with the shortest path exactly equal to k with you. Given your id
 * and the level of videos, return the list of videos ordered by their frequencies
(increasing).
 * For videos with the same frequency order them alphabetically from least to greatest.
 */

/**
 * @param {string[][]} watchedVideos
 * @param {number[][]} friends
 * @param {number} id
 * @param {number} level
 * @return {string[]}
 */
var watchedVideosByFriends = function(watchedVideos, friends, id, level) {
  const visited = new Set([id]);
  let queue = [id];
  let currentLevel = 0;

  while (queue.length && currentLevel < level) {
    const nextQueue = [];
    queue.forEach(person => friends[person].forEach(friend => {
      if (!visited.has(friend)) {
        visited.add(friend);
        nextQueue.push(friend);
      }
    }));
    queue = nextQueue;
    currentLevel++;
  }

  const videoFrequency = new Map();
  queue.forEach(person => watchedVideos[person].forEach(video => {
    return videoFrequency.set(video, (videoFrequency.get(video) || 0) + 1);
  }));

  return [...videoFrequency.entries()]
    .sort((a, b) => a[1] === b[1] ? a[0].localeCompare(b[0]) : a[1] - b[1])
    .map(([video]) => video);
};
```

1312 Minimum Insertion Steps To Make A String Palindrome

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1312. Minimum Insertion Steps to Make a String Palindrome
 * https://leetcode.com/problems/minimum-insertion-steps-to-make-a-string-palindrome/
 * Difficulty: Hard
 *
 * Given a string s. In one step you can insert any character at any index of the string.
 *
 * Return the minimum number of steps to make s palindrome.
 *
 * A Palindrome String is one that reads the same backward as well as forward.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minInsertions = function(s) {
```

```
  const n = s.length;
  const dp = new Array(n).fill().map(() => new Array(n).fill(0));

  for (let len = 2; len <= n; len++) {
    for (let start = 0; start + len <= n; start++) {
      const end = start + len - 1;
      dp[start][end] = s[start] === s[end]
        ? dp[start + 1][end - 1]
        : Math.min(dp[start + 1][end], dp[start][end - 1]) + 1;
    }
  }

  return dp[0][n - 1];
};
```

1313 Decompress Run Length Encoded List

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1313. Decompress Run-Length Encoded List
 * https://leetcode.com/problems/decompress-run-length-encoded-list/
 * Difficulty: Easy
 *
 * We are given a list nums of integers representing a list compressed with run-length
encoding.
 *
 * Consider each adjacent pair of elements [a, b] = [nums[2*i], nums[2*i+1]] (with i >= 0).
 * For each such pair, there are a elements with value b in the decompressed list.
 *
 * Return the decompressed list.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var decompressRLElist = function(nums, result = []) {
  for (let i = 0; i < nums.length; i += 2) {
    result.push(...new Array(nums[i]).fill(nums[i + 1]));
  }
  return result;
};
```

1314 Matrix Block Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1314. Matrix Block Sum
 * https://leetcode.com/problems/matrix-block-sum/
 * Difficulty: Medium
 *
 * Given a m x n matrix mat and an integer k, return a matrix answer where each answer[i][j]
 * is the sum of all elements mat[r][c] for:
 * - i - k <= r <= i + k,
 * - j - k <= c <= j + k, and
 * - (r, c) is a valid position in the matrix.
 */

/**
 * @param {number[][]} mat
 * @param {number} k
 * @return {number[][]}
 */
```

```
var matrixBlockSum = function(mat, k) {
  const rows = mat.length;
  const cols = mat[0].length;
  const prefixSums = new Array(rows + 1).fill().map(() => new Array(cols + 1).fill(0));

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      prefixSums[i + 1][j + 1] = prefixSums[i + 1][j]
        + prefixSums[i][j + 1] - prefixSums[i][j] + mat[i][j];
    }
  }

  const result = new Array(rows).fill().map(() => new Array(cols).fill(0));

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      const top = Math.max(0, i - k);
      const bottom = Math.min(rows - 1, i + k);
      const left = Math.max(0, j - k);
      const right = Math.min(cols - 1, j + k);

      result[i][j] = prefixSums[bottom + 1][right + 1]
        - prefixSums[bottom + 1][left] - prefixSums[top][right + 1] + prefixSums[top][left];
    }
  }

  return result;
};
```

1315 Sum Of Nodes With Even Valued Grandparent

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1315. Sum of Nodes with Even-Valued Grandparent
 * https://leetcode.com/problems/sum-of-nodes-with-even-valued-grandparent/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the sum of values of nodes with an even-valued
 * grandparent. If there are no nodes with an even-valued grandparent, return 0.
 *
 * A grandparent of a node is the parent of its parent if it exists.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var sumEvenGrandparent = function(root) {
  let result = 0;
  exploreTree(root, null, null);
  return result;

  function exploreTree(current, parent, grandparent) {
    if (!current) return;

    if (grandparent?.val % 2 === 0) {
      result += current.val;
    }

    exploreTree(current.left, current, parent);
    exploreTree(current.right, current, parent);
```

```
  }
};
```

1316 Distinct Echo Substrings

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1316. Distinct Echo Substrings
 * https://leetcode.com/problems/distinct-echo-substrings/
 * Difficulty: Hard
 *
 * Return the number of distinct non-empty substrings of text that can be written as the
 * concatenation of some string with itself (i.e. it can be written as a + a where a is
 * some string).
 */

/**
 * @param {string} text
 * @return {number}
 */
var distinctEchoSubstrings = function(text) {
  const set = new Set();

  for (let size = 1; size <= text.length / 2; size++) {
    let count = 0;

    for (let left = 0, right = size; right < text.length; left++, right++) {
      if (text[left] === text[right]) {
        count++;
      } else {
        count = 0;
      }

      if (count === size) {
        set.add(text.slice(left - size + 1, right + 1));
        count--;
      }
    }
  }

  return set.size;
};
```

1317 Convert Integer To The Sum Of Two No Zero Integers

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1317. Convert Integer to the Sum of Two No-Zero Integers
 * https://leetcode.com/problems/convert-integer-to-the-sum-of-two-no-zero-integers/
 * Difficulty: Easy
 *
 * Given an integer n. No-Zero integer is a positive integer which
 * doesn't contain any 0 in its decimal representation.
 *
 * Return a list of two integers [A, B] where:
 *
 * - A and B are No-Zero integers.
 * - A + B = n
 *
 * It's guarateed that there is at least one valid solution.
 * If there are many valid solutions you can return any of them.
 */
```

```
/**
 * @param {number} n
 * @return {number[]}
 */
var getNoZeroIntegers = function(n) {
  const hasNoZero = n => String(n).indexOf('0') === -1;
  for (let i = n; i; --i) {
    if (hasNoZero(i) && hasNoZero(n - i)) {
      return [i, n - i];
    }
  }
};
```

1318 Minimum Flips To Make A Or B Equal To C

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1318. Minimum Flips to Make a OR b Equal to c
 * https://leetcode.com/problems/minimum-flips-to-make-a-or-b-equal-to-c/
 * Difficulty: Medium
 *
 * Given 3 positives numbers a, b and c.
 * Return the minimum flips required in some bits of a and b
 * to make ( a OR b == c ). (bitwise OR operation).
 *
 * Flip operation consists of change any single bit 1 to 0
 * or change the bit 0 to 1 in their binary representation.
 */

/**
 * @param {number} a
 * @param {number} b
 * @param {number} c
 * @return {number}
 */
var minFlips = function(a, b, c, flips = 0) {
  [a, b, c] = [a, b, c].map(x => x.toString(2).padStart(32, 0));

  for (let i = 0; i < c.length; i++) {
    if ((+a[i] | +b[i]) !== +c[i]) {
      if (((+a[i] ^ 1) | +b[i]) === +c[i] || (+a[i] | (+b[i] ^ 1)) === +c[i]) flips++;
      else if (((+a[i] ^ 1) | (+b[i] ^ 1)) === +c[i]) flips += 2;
    }
  }

  return flips;
};
```

1319 Number Of Operations To Make Network Connected

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1319. Number of Operations to Make Network Connected
 * https://leetcode.com/problems/number-of-operations-to-make-network-connected/
 * Difficulty: Medium
 *
 * There are n computers numbered from 0 to n - 1 connected by ethernet cables connections
forming
 * a network where connections[i] = [ai, bi] represents a connection between computers ai and
bi.
 * Any computer can reach any other computer directly or indirectly through the network.
 *
 * You are given an initial computer network connections. You can extract certain cables
```

```
between
 * two directly connected computers, and place them between any pair of disconnected computers
 to
 * make them directly connected.
 *
 * Return the minimum number of times you need to do this in order to make all the computers
 * connected. If it is not possible, return -1.
 */

/**
 * @param {number} n
 * @param {number[][]} connections
 * @return {number}
 */
var makeConnected = function(n, connections) {
  const parent = Array(n).fill(-1);
  let isNotConnected = n - 1;
  let count = 0;

  connections.forEach(([node, connection]) => {
    if (search(node) !== search(connection)) {
      const p1 = search(node);
      const p2 = search(connection);
      if (p1 !== p2) {
        parent[p2] = p1;
      }
      isNotConnected--;
    } else {
      count++;
    }
  });

  return isNotConnected <= count ? isNotConnected : -1;

  function search(node) {
    if (parent[node] === -1) {
      return node;
    }
    parent[node] = search(parent[node]);
    return parent[node];
  }
};
```

1320 Minimum Distance To Type A Word Using Two Fingers

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1320. Minimum Distance to Type a Word Using Two Fingers
 * https://leetcode.com/problems/minimum-distance-to-type-a-word-using-two-fingers/
 * Difficulty: Hard
 *
 * You have a keyboard layout as shown above in the X-Y plane, where each English uppercase
letter
 * is located at some coordinate.
 *
 * For example, the letter 'A' is located at coordinate (0, 0), the letter 'B' is located at
 * coordinate (0, 1), the letter 'P' is located at coordinate (2, 3) and the letter 'Z' is
located
 * at coordinate (4, 1).
 *
 * Given the string word, return the minimum total distance to type such string using only two
 * fingers.
 *
 * The distance between coordinates (x1, y1) and (x2, y2) is |x1 - x2| + |y1 - y2|.
 *
 * Note that the initial positions of your two fingers are considered free so do not count
towards
 * your total distance, also your two fingers do not have to start at the first letter or the
first
```

```
 * two letters.
 */

/**
 * @param {string} word
 * @return {number}
 */
var minimumDistance = function(word) {
  const memo = {};
  return dp(0, null, null);

  function dp(index, finger1, finger2) {
    if (index === word.length) return 0;

    const key = `${index},${finger1},${finger2}`;
    if (memo[key] !== undefined) return memo[key];

    const nextChar = word[index];
    const moveFinger1 = distance(finger1, nextChar) + dp(index + 1, nextChar, finger2);
    const moveFinger2 = distance(finger2, nextChar) + dp(index + 1, finger1, nextChar);

    memo[key] = Math.min(moveFinger1, moveFinger2);
    return memo[key];
  }

  function getPosition(char) {
    const code = char.charCodeAt(0) - 65;
    return [Math.floor(code / 6), code % 6];
  }

  function distance(from, to) {
    if (from === null) return 0;
    const [x1, y1] = getPosition(from);
    const [x2, y2] = getPosition(to);
    return Math.abs(x1 - x2) + Math.abs(y1 - y2);
  }
};
```

1323 Maximum 69 Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1323. Maximum 69 Number
 * https://leetcode.com/problems/maximum-69-number/
 * Difficulty: Easy
 *
 * Given a positive integer num consisting only of digits 6 and 9.
 *
 * Return the maximum number you can get by changing at most
 * one digit (6 becomes 9, and 9 becomes 6).
 */

/**
 * @param {number} num
 * @return {number}
 */
var maximum69Number = function(num) {
  return +String(num).replace(6, 9);
};
```

1324 Print Words Vertically

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1324. Print Words Vertically
 * https://leetcode.com/problems/print-words-vertically/
 * Difficulty: Medium
 *
 * Given a string s. Return all the words vertically in the same
 * order in which they appear in s.
 * Words are returned as a list of strings, complete with spaces
 * when is necessary. (Trailing spaces are not allowed).
 * Each word would be put on only one column and that in one
 * column there will be only one word.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var printVertically = function(s) {
  const result = [];
  s.split(/\s+/).forEach((word, i) => word.split('').forEach((letter, j) => {
    result[j] = result[j] || '';
    result[j] += letter.padStart(i - result[j].length + 1, ' ');
  }));
  return result;
};
```

1325 Delete Leaves With A Given Value

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1325. Delete Leaves With a Given Value
 * https://leetcode.com/problems/delete-leaves-with-a-given-value/
 * Difficulty: Medium
 *
 * Given a binary tree root and an integer target, delete all the leaf nodes with value
target.
 *
 * Note that once you delete a leaf node with value target, if its parent node becomes a leaf
 * node and has the value target, it should also be deleted (you need to continue doing that
 * until you cannot).
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} target
 * @return {TreeNode}
 */
var removeLeafNodes = function(root, target) {
  return pruneLeaves(root);

  function pruneLeaves(node) {
    if (!node) return null;

    node.left = pruneLeaves(node.left);
    node.right = pruneLeaves(node.right);

    if (!node.left && !node.right && node.val === target) {
      return null;
    }

    return node;
```

```
  }
};
```

1326 Minimum Number Of Taps To Open To Water A Garden

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1326. Minimum Number of Taps to Open to Water a Garden
 * https://leetcode.com/problems/minimum-number-of-taps-to-open-to-water-a-garden/
 * Difficulty: Hard
 *
 * There is a one-dimensional garden on the x-axis. The garden starts at the point 0 and ends
at
 * the point n. (i.e., the length of the garden is n).
 *
 * There are n + 1 taps located at points [0, 1, ..., n] in the garden.
 *
 * Given an integer n and an integer array ranges of length n + 1 where ranges[i] (0-indexed)
means
 * the i-th tap can water the area [i - ranges[i], i + ranges[i]] if it was open.
 *
 * Return the minimum number of taps that should be open to water the whole garden, If the
garden
 * cannot be watered return -1.
 */

/**
 * @param {number} n
 * @param {number[]} ranges
 * @return {number}
 */
var minTaps = function(n, ranges) {
  const intervals = ranges.map((range, index) => [
    Math.max(0, index - range),
    Math.min(n, index + range)
  ]);

  intervals.sort((a, b) => a[0] - b[0] || b[1] - a[1]);

  let taps = 0;
  let currentEnd = 0;
  let nextEnd = 0;
  let i = 0;

  while (i < intervals.length && currentEnd < n) {
    taps++;

    while (i < intervals.length && intervals[i][0] <= currentEnd) {
      nextEnd = Math.max(nextEnd, intervals[i][1]);
      i++;
    }

    if (currentEnd === nextEnd) return -1;
    currentEnd = nextEnd;
  }

  return currentEnd >= n ? taps : -1;
};
```

1328 Break A Palindrome

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 1328. Break a Palindrome
 * https://leetcode.com/problems/break-a-palindrome/
 * Difficulty: Medium
 *
 * Given a palindromic string of lowercase English letters palindrome, replace exactly one
 * character with any lowercase English letter so that the resulting string is not a
palindrome
 * and that it is the lexicographically smallest one possible.
 *
 * Return the resulting string. If there is no way to replace a character to make it not a
 * palindrome, return an empty string.
 *
 * A string a is lexicographically smaller than a string b (of the same length) if in the
first
 * position where a and b differ, a has a character strictly smaller than the corresponding
 * character in b. For example, "abcc" is lexicographically smaller than "abcd" because the
 * first position they differ is at the fourth character, and 'c' is smaller than 'd'.
 */

/**
 * @param {string} palindrome
 * @return {string}
 */
var breakPalindrome = function(palindrome) {
  if (palindrome.length === 1) return '';

  const chars = palindrome.split('');

  for (let i = 0; i < Math.floor(chars.length / 2); i++) {
    if (chars[i] !== 'a') {
      chars[i] = 'a';
      return chars.join('');
    }
  }

  chars[chars.length - 1] = 'b';
  return chars.join('');
};
```

1329 Sort The Matrix Diagonally

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1329. Sort the Matrix Diagonally
 * https://leetcode.com/problems/sort-the-matrix-diagonally/
 * Difficulty: Medium
 *
 * A matrix diagonal is a diagonal line of cells starting from some cell in either the topmost
 * row or leftmost column and going in the bottom-right direction until reaching the matrix's
 * end. For example, the matrix diagonal starting from mat[2][0], where mat is a 6 x 3 matrix,
 * includes cells mat[2][0], mat[3][1], and mat[4][2].
 *
 * Given an m x n matrix mat of integers, sort each matrix diagonal in ascending order and
return
 * the resulting matrix.
 */

/**
 * @param {number[][]} mat
 * @return {number[][]}
 */
var diagonalSort = function(mat) {
  const rows = mat.length;
  const cols = mat[0].length;
  const diagonals = new Map();

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
```

```
      const diagonalKey = i - j;
      if (!diagonals.has(diagonalKey)) {
        diagonals.set(diagonalKey, []);
      }
      diagonals.get(diagonalKey).push(mat[i][j]);
    }
  }

  diagonals.forEach(values => values.sort((a, b) => a - b));

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      const diagonalKey = i - j;
      mat[i][j] = diagonals.get(diagonalKey).shift();
    }
  }

  return mat;
};
```

1330 Reverse Subarray To Maximize Array Value

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1330. Reverse Subarray To Maximize Array Value
 * https://leetcode.com/problems/reverse-subarray-to-maximize-array-value/
 * Difficulty: Hard
 *
 * You are given an integer array nums. The value of this array is defined as the sum
 * of |nums[i] - nums[i + 1]| for all 0 <= i < nums.length - 1.
 *
 * You are allowed to select any subarray of the given array and reverse it. You can
 * perform this operation only once.
 *
 * Find maximum possible value of the final array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxValueAfterReverse = function(nums) {
  let baseSum = 0;
  let maxGain = 0;
  let minPair = Infinity;
  let maxPair = -Infinity;
  const n = nums.length;

  for (let i = 0; i < n - 1; i++) {
    baseSum += Math.abs(nums[i] - nums[i + 1]);
    minPair = Math.min(minPair, Math.max(nums[i], nums[i + 1]));
    maxPair = Math.max(maxPair, Math.min(nums[i], nums[i + 1]));
  }

  maxGain = Math.max(0, 2 * (maxPair - minPair));

  for (let i = 0; i < n - 1; i++) {
    const left = i > 0 ? Math.abs(nums[0] - nums[i + 1]) - Math.abs(nums[i] - nums[i + 1]) :
0;
    const right = i < n - 2 ? Math.abs(nums[n - 1] - nums[i]) - Math.abs(nums[i] - nums[i +
1]) : 0;
    maxGain = Math.max(maxGain, left, right);
  }

  return baseSum + maxGain;
};
```

1331 Rank Transform Of An Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1331. Rank Transform of an Array
 * https://leetcode.com/problems/rank-transform-of-an-array/
 * Difficulty: Easy
 *
 * Given an array of integers arr, replace each element with its rank.
 *
 * The rank represents how large the element is. The rank has the following rules:
 * - Rank is an integer starting from 1.
 * - The larger the element, the larger the rank. If two elements are equal, their
 *   rank must be the same.
 * - Rank should be as small as possible.
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var arrayRankTransform = function(arr) {
  const ranks = [...new Set(arr)]
    .sort((a, b) => a - b)
    .reduce((map, value, index) => map.set(value, index + 1), new Map());

  return arr.map(rank => ranks.get(rank));
};
```

1332 Remove Palindromic Subsequences

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1332. Remove Palindromic Subsequences
 * https://leetcode.com/problems/remove-palindromic-subsequences/
 * Difficulty: Easy
 *
 * Given a string s consisting only of letters 'a' and 'b'.
 * In a single step you can remove one palindromic subsequence from s.
 *
 * Return the minimum number of steps to make the given string empty.
 * A string is a subsequence of a given string, if it is generated by
 * deleting some characters of a given string without changing its order.
 *
 * A string is called palindrome if is one that reads the same backward
 * as well as forward.
 */

/**
 * @param {string} s
 * @return {number}
 */
var removePalindromeSub = function(s) {
  return s.length === 0 ? 0 : (s.split('').reverse().join('') === s ? 1 : 2);
};
```

1333 Filter Restaurants By Vegan Friendly Price And Distance

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1333. Filter Restaurants by Vegan-Friendly, Price and Distance
 * https://leetcode.com/problems/filter-restaurants-by-vegan-friendly-price-and-distance/
 * Difficulty: Medium
 *
 * Given the array restaurants where  restaurants[i] = [idi, ratingi, veganFriendlyi, pricei,
distancei].
 * You have to filter the restaurants using three filters.
 *
 * The veganFriendly filter will be either true (meaning you should only include restaurants
with
 * veganFriendlyi set to true) or false (meaning you can include any restaurant). In addition,
 you
 * have the filters maxPrice and maxDistance which are the maximum value for price and
distance of
 * restaurants you should consider respectively.
 *
 * Return the array of restaurant IDs after filtering, ordered by rating from highest to
lowest.
 * For restaurants with the same rating, order them by id from highest to lowest. For
simplicity
 * veganFriendlyi and veganFriendly take value 1 when it is true, and 0 when it is false.
 */

/**
 * @param {number[][]} restaurants
 * @param {number} veganFriendly
 * @param {number} maxPrice
 * @param {number} maxDistance
 * @return {number[]}
 */
var filterRestaurants = function(restaurants, veganFriendly, maxPrice, maxDistance) {
  return restaurants.filter(([,, v, p, d]) => v >= veganFriendly && p <= maxPrice && d <=
maxDistance)
    .sort((a, b) => a[1] === b[1] ? b[0] - a[0] : b[1] - a[1])
    .map(r => r[0]);
};
```

1334 Find The City With The Smallest Number Of Neighbors At A Threshold

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance
 * https://leetcode.com/problems/find-the-city-with-the-smallest-number-of-neighbors-at-a-thre
shold-distance/
 * Difficulty: Medium
 *
 * There are n cities numbered from 0 to n-1. Given the array edges where
 * edges[i] = [fromi, toi, weighti] represents a bidirectional and weighted edge between
cities
 * fromi and toi, and given the integer distanceThreshold.
 *
 * Return the city with the smallest number of cities that are reachable through some path and
 whose
 * distance is at most distanceThreshold, If there are multiple such cities, return the city
with
 * the greatest number.
 *
 * Notice that the distance of a path connecting cities i and j is equal to the sum of the
edges'
 * weights along that path.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
```

```
 * @param {number} distanceThreshold
 * @return {number}
 */
var findTheCity = function(n, edges, distanceThreshold) {
  const distances = Array.from({ length: n }, () => Array(n).fill(Infinity));

  for (let i = 0; i < n; i++) {
    distances[i][i] = 0;
  }

  for (const [from, to, weight] of edges) {
    distances[from][to] = weight;
    distances[to][from] = weight;
  }

  for (let k = 0; k < n; k++) {
    for (let i = 0; i < n; i++) {
      for (let j = 0; j < n; j++) {
        distances[i][j] = Math.min(distances[i][j], distances[i][k] + distances[k][j]);
      }
    }
  }

  let minNeighbors = n;
  let result = 0;

  for (let i = 0; i < n; i++) {
    const neighbors = distances[i].filter(dist => dist <= distanceThreshold).length - 1;
    if (neighbors <= minNeighbors) {
      minNeighbors = neighbors;
      result = i;
    }
  }

  return result;
};
```

1335 Minimum Difficulty Of A Job Schedule

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1335. Minimum Difficulty of a Job Schedule
 * https://leetcode.com/problems/minimum-difficulty-of-a-job-schedule/
 * Difficulty: Hard
 *
 * You want to schedule a list of jobs in d days. Jobs are dependent (i.e To work on the ith
job,
 * you have to finish all the jobs j where 0 <= j < i).
 *
 * You have to finish at least one task every day. The difficulty of a job schedule is the sum
 of
 * difficulties of each day of the d days. The difficulty of a day is the maximum difficulty
of a
 * job done on that day.
 *
 * You are given an integer array jobDifficulty and an integer d. The difficulty of the ith
job
 * is jobDifficulty[i].
 *
 * Return the minimum difficulty of a job schedule. If you cannot find a schedule for the jobs
 * return -1.
 */

/**
 * @param {number[]} jobDifficulty
 * @param {number} d
 * @return {number}
 */
var minDifficulty = function(jobDifficulty, d) {
```

```
  const n = jobDifficulty.length;
  if (n < d) return -1;

  const dp = new Array(d + 1).fill().map(() => new Array(n + 1).fill(Infinity));
  dp[0][0] = 0;

  for (let days = 1; days <= d; days++) {
    for (let i = days; i <= n; i++) {
      let maxDifficulty = 0;
      for (let j = i - 1; j >= days - 1; j--) {
        maxDifficulty = Math.max(maxDifficulty, jobDifficulty[j]);
        dp[days][i] = Math.min(dp[days][i], dp[days - 1][j] + maxDifficulty);
      }
    }
  }

  return dp[d][n] === Infinity ? -1 : dp[d][n];
};
```

1337 The K Weakest Rows In A Matrix

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1337. The K Weakest Rows in a Matrix
 * https://leetcode.com/problems/the-k-weakest-rows-in-a-matrix/
 * Difficulty: Easy
 *
 * You are given an m x n binary matrix mat of 1's (representing soldiers) and 0's
(representing
 * civilians). The soldiers are positioned in front of the civilians. That is, all the 1's
will
 * appear to the left of all the 0's in each row.
 *
 * A row i is weaker than a row j if one of the following is true:
 * - The number of soldiers in row i is less than the number of soldiers in row j.
 * - Both rows have the same number of soldiers and i < j.
 *
 * Return the indices of the k weakest rows in the matrix ordered from weakest to strongest.
 */

/**
 * @param {number[][]} mat
 * @param {number} k
 * @return {number[]}
 */
var kWeakestRows = function(mat, k) {
  const soldierCounts = mat
    .map((row, index) => [row.reduce((sum, val) => sum + val, 0), index])
    .sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  return soldierCounts.slice(0, k).map(pair => pair[1]);
};
```

1338 Reduce Array Size To The Half

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1338. Reduce Array Size to The Half
 * https://leetcode.com/problems/reduce-array-size-to-the-half/
 * Difficulty: Medium
 *
 * You are given an integer array arr. You can choose a set of integers and remove all the
```

```
 * occurrences of these integers in the array.
 *
 * Return the minimum size of the set so that at least half of the integers of the array
 * are removed.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var minSetSize = function(arr) {
  const map = new Map();
  const halfLength = arr.length / 2;

  arr.forEach(num => map.set(num, (map.get(num) || 0) + 1));

  const frequencies = Array.from(map.values()).sort((a, b) => b - a);

  let removedCount = 0;
  let result = 0;

  for (const freq of frequencies) {
    removedCount += freq;
    result++;
    if (removedCount >= halfLength) break;
  }

  return result;
};
```

1339 Maximum Product Of Splitted Binary Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1339. Maximum Product of Splitted Binary Tree
 * https://leetcode.com/problems/maximum-product-of-splitted-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, split the binary tree into two subtrees by removing one
edge
 * such that the product of the sums of the subtrees is maximized.
 *
 * Return the maximum product of the sums of the two subtrees. Since the answer may be too
large,
 * return it modulo 109 + 7.
 *
 * Note that you need to maximize the answer before taking the mod and not after taking it.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxProduct = function(root) {
  const sums = [];
  const MOD = 1e9 + 7;
  const total = calculateSum(root);
  let maxProd = 0;

  for (const sum of sums) {
    maxProd = Math.max(maxProd, sum * (total - sum));
```

```
  }

  return maxProd % MOD;

  function calculateSum(node) {
    if (!node) return 0;
    const leftSum = calculateSum(node.left);
    const rightSum = calculateSum(node.right);
    const totalSum = node.val + leftSum + rightSum;
    sums.push(totalSum);
    return totalSum;
  }
};
```

1340 Jump Game V

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1340. Jump Game V
 * https://leetcode.com/problems/jump-game-v/
 * Difficulty: Hard
 *
 * Given an array of integers arr and an integer d. In one step you can jump from index i to
index:
 * - i + x where: i + x < arr.length and  0 < x <= d.
 * - i - x where: i - x >= 0 and  0 < x <= d.
 *
 * In addition, you can only jump from index i to index j if arr[i] > arr[j] and arr[i] >
arr[k] for
 * all indices k between i and j (More formally min(i, j) < k < max(i, j)).
 *
 * You can choose any index of the array and start jumping. Return the maximum number of
indices you
 * can visit.
 *
 * Notice that you can not jump outside of the array at any time.
 */

/**
 * @param {number[]} arr
 * @param {number} d
 * @return {number}
 */
var maxJumps = function(arr, d) {
  const n = arr.length;
  const memo = new Array(n).fill(0);
  let result = 1;

  for (let i = 0; i < n; i++) {
    result = Math.max(result, exploreJumps(i));
  }

  return result;

  function exploreJumps(index) {
    if (memo[index]) return memo[index];

    let maxJumpsFromHere = 1;

    for (let offset = 1; offset <= d; offset++) {
      const right = index + offset;
      if (right < n && arr[index] > arr[right]) {
        maxJumpsFromHere = Math.max(maxJumpsFromHere, 1 + exploreJumps(right));
      } else {
        break;
      }
    }

    for (let offset = 1; offset <= d; offset++) {
```

```
      const left = index - offset;
      if (left >= 0 && arr[index] > arr[left]) {
        maxJumpsFromHere = Math.max(maxJumpsFromHere, 1 + exploreJumps(left));
      } else {
        break;
      }
    }

    memo[index] = maxJumpsFromHere;
    return maxJumpsFromHere;
  }
};
```

1342 Number Of Steps To Reduce A Number To Zero

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1342. Number of Steps to Reduce a Number to Zero
 * https://leetcode.com/problems/number-of-steps-to-reduce-a-number-to-zero/
 * Difficulty: Easy
 *
 * Given an integer num, return the number of steps to reduce it to zero.
 *
 * In one step, if the current number is even, you have to divide it by 2,
 * otherwise, you have to subtract 1 from it.
 */

/**
 * @param {number} num
 * @return {number}
 */
var numberOfSteps = function(num) {
  let steps = 0;
  while (num > 0 && ++steps) {
    num = num % 2 === 0 ? num / 2 : num -1;
  }
  return steps;
};
```

1343 Number Of Sub Arrays Of Size K And Average Greater Than Or Equal T

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1343. Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold
 * https://leetcode.com/problems/number-of-sub-arrays-of-size-k-and-average-greater-than-or-eq
ual-to-threshold/
 * Difficulty: Medium
 *
 * Given an array of integers arr and two integers k and threshold, return the number of
sub-arrays
 * of size k and average greater than or equal to threshold.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @param {number} threshold
 * @return {number}
 */
var numOfSubarrays = function(arr, k, threshold) {
  let result = 0;
  let windowSum = 0;
  const minSum = k * threshold;
```

```
  for (let i = 0; i < k; i++) {
    windowSum += arr[i];
  }

  if (windowSum >= minSum) result++;

  for (let i = k; i < arr.length; i++) {
    windowSum += arr[i] - arr[i - k];
    if (windowSum >= minSum) result++;
  }

  return result;
};
```

1344 Angle Between Hands Of A Clock

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1344. Angle Between Hands of a Clock
 * https://leetcode.com/problems/angle-between-hands-of-a-clock/
 * Difficulty: Medium
 *
 * Given two numbers, hour and minutes, return the smaller angle (in degrees) formed between
the
 * hour and the minute hand.
 *
 * Answers within 10-5 of the actual value will be accepted as correct.
 */

/**
 * @param {number} hour
 * @param {number} minutes
 * @return {number}
 */
var angleClock = function(hour, minutes) {
  const hourAngle = (hour % 12 + minutes / 60) * 30;
  const minuteAngle = minutes * 6;
  const angle = Math.abs(hourAngle - minuteAngle);
  return Math.min(angle, 360 - angle);
};
```

1345 Jump Game Iv

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1345. Jump Game IV
 * https://leetcode.com/problems/jump-game-iv/
 * Difficulty: Hard
 *
 * Given an array of integers arr, you are initially positioned at the first index of the
array.
 *
 * In one step you can jump from index i to index:
 * - i + 1 where: i + 1 < arr.length.
 * - i - 1 where: i - 1 >= 0.
 * - j where: arr[i] == arr[j] and i != j.
 *
 * Return the minimum number of steps to reach the last index of the array.
 *
 * Notice that you can not jump outside of the array at any time.
 */
```

```
/**
 * @param {number[]} arr
 * @return {number}
 */
var minJumps = function(arr) {
  const n = arr.length;
  if (n <= 1) return 0;

  const valueToIndices = new Map();
  for (let i = 0; i < n; i++) {
    if (!valueToIndices.has(arr[i])) {
      valueToIndices.set(arr[i], []);
    }
    valueToIndices.get(arr[i]).push(i);
  }

  const visited = new Set([0]);
  let queue = [0];
  let result = 0;

  while (queue.length) {
    const nextQueue = [];

    for (const current of queue) {
      if (current === n - 1) return result;

      if (current + 1 < n && !visited.has(current + 1)) {
        visited.add(current + 1);
        nextQueue.push(current + 1);
      }

      if (current - 1 >= 0 && !visited.has(current - 1)) {
        visited.add(current - 1);
        nextQueue.push(current - 1);
      }

      for (const next of valueToIndices.get(arr[current]) || []) {
        if (!visited.has(next)) {
          visited.add(next);
          nextQueue.push(next);
        }
      }

      valueToIndices.delete(arr[current]);
    }

    queue = nextQueue;
    result++;
  }

  return result;
};
```

1346 Check If N And Its Double Exist

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1346. Check If N and Its Double Exist
 * https://leetcode.com/problems/check-if-n-and-its-double-exist/
 * Difficulty: Easy
 *
 * Given an array arr of integers, check if there exist two indices i and j such that:
 * - i != j
 * - 0 <= i, j < arr.length
 * - arr[i] == 2 * arr[j]
 */

/**
 * @param {number[]} arr
```

```
 * @return {boolean}
 */
var checkIfExist = function(arr) {
  const set = new Set();

  for (const num of arr) {
    if (set.has(num * 2) || set.has(num / 2)) {
      return true;
    }
    set.add(num);
  }

  return false;
};
```

1347 Minimum Number Of Steps To Make Two Strings Anagram

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1347. Minimum Number of Steps to Make Two Strings Anagram
 * https://leetcode.com/problems/minimum-number-of-steps-to-make-two-strings-anagram/
 * Difficulty: Medium
 *
 * You are given two strings of the same length s and t. In one step you can choose any
character
 * of t and replace it with another character.
 *
 * Return the minimum number of steps to make t an anagram of s.
 *
 * An Anagram of a string is a string that contains the same characters with a different (or
the
 * same) ordering.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {number}
 */
var minSteps = function(s, t) {
  const charCount = new Array(26).fill(0);
  let result = 0;

  for (let i = 0; i < s.length; i++) {
    charCount[s.charCodeAt(i) - 97]++;
    charCount[t.charCodeAt(i) - 97]--;
  }

  for (const count of charCount) {
    if (count > 0) result += count;
  }

  return result;
};
```

1348 Tweet Counts Per Frequency

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1348. Tweet Counts Per Frequency
 * https://leetcode.com/problems/tweet-counts-per-frequency/
 * Difficulty: Medium
 *
```

```
 * A social media company is trying to monitor activity on their site by analyzing the number
 * of tweets that occur in select periods of time. These periods can be partitioned into
 * smaller time chunks based on a certain frequency (every minute, hour, or day).
 *
 * For example, the period [10, 10000] (in seconds) would be partitioned into the following
 * time chunks with these frequencies:
 * - Every minute (60-second chunks): [10,69], [70,129], [130,189], ..., [9970,10000]
 * - Every hour (3600-second chunks): [10,3609], [3610,7209], [7210,10000]
 * - Every day (86400-second chunks): [10,10000]
 *
 * Notice that the last chunk may be shorter than the specified frequency's chunk size and
will
 * always end with the end time of the period (10000 in the above example).
 *
 * Design and implement an API to help the company with their analysis.
 *
 * Implement the TweetCounts class:
 * - TweetCounts() Initializes the TweetCounts object.
 * - void recordTweet(String tweetName, int time) Stores the tweetName at the recorded time
 *   (in seconds).
 * - List<Integer> getTweetCountsPerFrequency(String freq, String tweetName, int startTime,
 *   int endTime) Returns a list of integers representing the number of tweets with tweetName
 *   in each time chunk for the given period of time [startTime, endTime] (in seconds) and
 *   frequency freq.
 * - freq is one of "minute", "hour", or "day" representing a frequency of every minute, hour,
 *   or day respectively.
 */

var TweetCounts = function() {
  this.tweets = new Map();
};

/**
 * @param {string} tweetName
 * @param {number} time
 * @return {void}
 */
TweetCounts.prototype.recordTweet = function(tweetName, time) {
  if (!this.tweets.has(tweetName)) {
    this.tweets.set(tweetName, []);
  }
  this.tweets.get(tweetName).push(time);
};

/**
 * @param {string} freq
 * @param {string} tweetName
 * @param {number} startTime
 * @param {number} endTime
 * @return {number[]}
 */
TweetCounts.prototype.getTweetCountsPerFrequency = function(freq, tweetName, startTime,
endTime) {
  let interval;
  if (freq === 'minute') interval = 60;
  else if (freq === 'hour') interval = 3600;
  else interval = 86400;

  const count = Math.floor((endTime - startTime) / interval) + 1;
  const result = new Array(count).fill(0);

  if (this.tweets.has(tweetName)) {
    for (const time of this.tweets.get(tweetName)) {
      if (time >= startTime && time <= endTime) {
        const index = Math.floor((time - startTime) / interval);
        result[index]++;
      }
    }
  }

  return result;
};
```

1349 Maximum Students Taking Exam

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1349. Maximum Students Taking Exam
 * https://leetcode.com/problems/maximum-students-taking-exam/
 * Difficulty: Hard
 *
 * Given a m * n matrix seats  that represent seats distributions in a classroom. If a seat is
 * broken, it is denoted by '#' character otherwise it is denoted by a '.' character.
 *
 * Students can see the answers of those sitting next to the left, right, upper left and upper
 * right, but he cannot see the answers of the student sitting directly in front or behind
him.
 * Return the maximum number of students that can take the exam together without any cheating
 * being possible.
 *
 * Students must be placed in seats in good condition.
 */

/**
 * @param {character[][]} seats
 * @return {number}
 */
var maxStudents = function(seats) {
  const rows = seats.length;
  const cols = seats[0].length;
  const cache = new Map();

  return countStudents(0, 0);

  function isValid(row, currMask, prevMask) {
    for (let j = 0; j < cols; j++) {
      if (!(currMask & (1 << j))) continue;
      if (seats[row][j] === '#') return false;
      if (j > 0 && (currMask & (1 << (j - 1)))) return false;
      if (j < cols - 1 && (currMask & (1 << (j + 1)))) return false;
      if (row > 0) {
        if (j > 0 && (prevMask & (1 << (j - 1)))) return false;
        if (j < cols - 1 && (prevMask & (1 << (j + 1)))) return false;
      }
    }
    return true;
  }

  function countStudents(row, prevMask) {
    if (row === rows) return 0;
    const key = `${row},${prevMask}`;
    if (cache.has(key)) return cache.get(key);

    let maxCount = 0;
    for (let currMask = 0; currMask < (1 << cols); currMask++) {
      if (!isValid(row, currMask, prevMask)) continue;
      const students = (currMask.toString(2).match(/1/g) || []).length;
      maxCount = Math.max(maxCount, students + countStudents(row + 1, currMask));
    }

    cache.set(key, maxCount);
    return maxCount;
  }
};
```

1351 Count Negative Numbers In A Sorted Matrix

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1351. Count Negative Numbers in a Sorted Matrix
 * https://leetcode.com/problems/count-negative-numbers-in-a-sorted-matrix/
 * Difficulty: Easy
 *
 * Given a m x n matrix grid which is sorted in non-increasing order both
 * row-wise and column-wise, return the number of negative numbers in grid.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var countNegatives = function(grid) {
  return grid.reduce((count, row) => {
    while (row.pop() < 0) {
      count++;
    }
    return count;
  }, 0);
};
```

1352 Product Of The Last K Numbers

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1352. Product of the Last K Numbers
 * https://leetcode.com/problems/product-of-the-last-k-numbers/
 * Difficulty: Medium
 *
 * Design an algorithm that accepts a stream of integers and retrieves the product of the
 * last k integers of the stream.
 *
 * Implement the ProductOfNumbers class:
 * - ProductOfNumbers() Initializes the object with an empty stream.
 * - void add(int num) Appends the integer num to the stream.
 * - int getProduct(int k) Returns the product of the last k numbers in the current list.
 *   You can assume that always the current list has at least k numbers.
 * - The test cases are generated so that, at any time, the product of any contiguous sequence
 *   of numbers will fit into a single 32-bit integer without overflowing.
 */

var ProductOfNumbers = function() {
  this.nums = [];
};

/**
 * @param {number} num
 * @return {void}
 */
ProductOfNumbers.prototype.add = function(num) {
  this.nums.push(num);
};

/**
 * @param {number} k
 * @return {number}
 */
ProductOfNumbers.prototype.getProduct = function(k) {
  return this.nums.slice(-k).reduce((product, n) => product * n, 1);
};
```

1353 Maximum Number Of Events That Can Be Attended

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1353. Maximum Number of Events That Can Be Attended
 * https://leetcode.com/problems/maximum-number-of-events-that-can-be-attended/
 * Difficulty: Medium
 *
 * You are given an array of events where events[i] = [startDayi, endDayi]. Every event i
starts
 * at startDayi and ends at endDayi.
 *
 * You can attend an event i at any day d where startTimei <= d <= endTimei. You can only
attend
 * one event at any time d.
 *
 * Return the maximum number of events you can attend.
 */

/**
 * @param {number[][]} events
 * @return {number}
 */
var maxEvents = function(events) {
  events.sort((a, b) => a[0] - b[0]);
  const heap = [];
  let result = 0;
  let eventIndex = 0;
  let currentDay = 1;

  while (eventIndex < events.length || heap.length) {
    while (eventIndex < events.length && events[eventIndex][0] <= currentDay) {
      heapPush(heap, events[eventIndex][1]);
      eventIndex++;
    }

    while (heap.length && heap[0] < currentDay) {
      heapPop(heap);
    }

    if (heap.length) {
      heapPop(heap);
      result++;
    }

    currentDay++;
  }

  function heapPush(arr, val) {
    arr.push(val);
    let i = arr.length - 1;
    while (i > 0) {
      const parent = Math.floor((i - 1) / 2);
      if (arr[parent] <= arr[i]) break;
      [arr[parent], arr[i]] = [arr[i], arr[parent]];
      i = parent;
    }
  }

  function heapPop(arr) {
    const result = arr[0];
    arr[0] = arr[arr.length - 1];
    arr.pop();
    let i = 0;
    while (true) {
      const left = 2 * i + 1;
      const right = 2 * i + 2;
      let smallest = i;
      if (left < arr.length && arr[left] < arr[smallest]) smallest = left;
```

```
      if (right < arr.length && arr[right] < arr[smallest]) smallest = right;
      if (smallest === i) break;
      [arr[i], arr[smallest]] = [arr[smallest], arr[i]];
      i = smallest;
    }
    return result;
  }

  return result;
};
```

1354 Construct Target Array With Multiple Sums

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1354. Construct Target Array With Multiple Sums
 * https://leetcode.com/problems/construct-target-array-with-multiple-sums/
 * Difficulty: Hard
 *
 * You are given an array target of n integers. From a starting array arr consisting of n 1's,
 * you may perform the following procedure:
 * - let x be the sum of all elements currently in your array.
 * - choose index i, such that 0 <= i < n and set the value of arr at index i to x.
 * - You may repeat this procedure as many times as needed.
 *
 * Return true if it is possible to construct the target array from arr, otherwise, return
false.
 */

/**
 * @param {number[]} target
 * @return {boolean}
 */
function isPossible(target) {
  let totalSum = target.reduce((sum, num) => sum + num, 0);
  const heap = [...target];

  buildHeap();

  while (heap[0] > 1) {
    const largest = heap[0];
    totalSum -= largest;
    if (totalSum < 1) return false;
    const newValue = largest - Math.floor((largest - 1) / totalSum) * totalSum;
    if (newValue === largest) return false;
    totalSum += newValue;
    heap[0] = newValue;
    heapifyDown(0);
  }

  return true;

  function heapifyDown(index) {
    let largest = index;
    const left = 2 * index + 1;
    const right = 2 * index + 2;
    if (left < heap.length && heap[left] > heap[largest]) largest = left;
    if (right < heap.length && heap[right] > heap[largest]) largest = right;
    if (largest !== index) {
      [heap[index], heap[largest]] = [heap[largest], heap[index]];
      heapifyDown(largest);
    }
  }

  function buildHeap() {
    for (let i = Math.floor(heap.length / 2) - 1; i >= 0; i--) {
      heapifyDown(i);
    }
  }
```

```
}
```

1356 Sort Integers By The Number Of 1 Bits

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1356. Sort Integers by The Number of 1 Bits
 * https://leetcode.com/problems/sort-integers-by-the-number-of-1-bits/
 * Difficulty: Easy
 *
 * Given an integer array arr. You have to sort the integers in the array in
 * ascending order by the number of 1's in their binary representation and
 * in case of two or more integers have the same number of 1's you have to
 * sort them in ascending order.
 *
 * Return the sorted array.
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var sortByBits = function(arr) {
  const getCount = n => n.toString(2).replace(/0/g, '').length;

  return arr.sort((a, b) => getCount(a) - getCount(b) || a - b);
};
```

1357 Apply Discount Every N Orders

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1357. Apply Discount Every n Orders
 * https://leetcode.com/problems/apply-discount-every-n-orders/
 * Difficulty: Medium
 *
 * There is a supermarket that is frequented by many customers. The products sold at the
supermarket
 * are represented as two parallel integer arrays products and prices, where the ith product
has an
 * ID of products[i] and a price of prices[i].
 *
 * When a customer is paying, their bill is represented as two parallel integer arrays product
 and
 * amount, where the jth product they purchased has an ID of product[j], and amount[j] is how
much
 * of the product they bought. Their subtotal is calculated as the sum of each amount[j] *
(price
 * of the jth product).
 *
 * The supermarket decided to have a sale. Every nth customer paying for their groceries will
be
 * given a percentage discount. The discount amount is given by discount, where they will be
given
 * discount percent off their subtotal. More formally, if their subtotal is bill, then they
would
 * actually pay bill * ((100 - discount) / 100).
 *
 * Implement the Cashier class:
 * - Cashier(int n, int discount, int[] products, int[] prices) Initializes the object with n,
 *   the discount, and the products and their prices.
 * - double getBill(int[] product, int[] amount) Returns the final total of the bill with the
 *   discount applied (if any). Answers within 10-5 of the actual value will be accepted.
```

```
 */

/**
 * @param {number} n
 * @param {number} discount
 * @param {number[]} products
 * @param {number[]} prices
 */
var Cashier = function(n, discount, products, prices) {
  this.customerCount = 0;
  this.discountPercent = discount;
  this.nthCustomer = n;
  this.priceMap = new Map();
  for (let i = 0; i < products.length; i++) {
    this.priceMap.set(products[i], prices[i]);
  }
};

/**
 * @param {number[]} product
 * @param {number[]} amount
 * @return {number}
 */
Cashier.prototype.getBill = function(product, amount) {
  this.customerCount++;
  let total = 0;
  for (let i = 0; i < product.length; i++) {
    total += this.priceMap.get(product[i]) * amount[i];
  }
  if (this.customerCount % this.nthCustomer === 0) {
    return total * (100 - this.discountPercent) / 100;
  }
  return total;
};
```

1358 Number Of Substrings Containing All Three Characters

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1358. Number of Substrings Containing All Three Characters
 * https://leetcode.com/problems/number-of-substrings-containing-all-three-characters/
 * Difficulty: Medium
 *
 * Given a string s consisting only of characters a, b and c.
 *
 * Return the number of substrings containing at least one occurrence of all these
 * characters a, b and c.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numberOfSubstrings = function(s) {
  const count = [0, 0, 0];
  let result = 0;
  let left = 0;

  for (let right = 0; right < s.length; right++) {
    count[s[right].charCodeAt(0) - 97]++;

    while (count[0] > 0 && count[1] > 0 && count[2] > 0) {
      result += s.length - right;
      count[s[left].charCodeAt(0) - 97]--;
      left++;
    }
  }

  return result;
```

```
};
```

1359 Count All Valid Pickup And Delivery Options

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1359. Count All Valid Pickup and Delivery Options
 * https://leetcode.com/problems/count-all-valid-pickup-and-delivery-options/
 * Difficulty: Hard
 *
 * Given n orders, each order consists of a pickup and a delivery service.
 *
 * Count all valid pickup/delivery possible sequences such that delivery(i) is always after
 * of pickup(i).
 *
 * Since the answer may be too large, return it modulo 10^9 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
function countOrders(n) {
  const MOD = 1e9 + 7;
  let result = 1;

  for (let i = 1; i <= n; i++) {
    result = (result * (2 * i - 1) * i) % MOD;
  }

  return result;
}
```

1360 Number Of Days Between Two Dates

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1360. Number of Days Between Two Dates
 * https://leetcode.com/problems/number-of-days-between-two-dates/
 * Difficulty: Easy
 *
 * Write a program to count the number of days between two dates.
 *
 * The two dates are given as strings, their format is `YYYY-MM-DD`
 * as shown in the examples.
 */

/**
 * @param {string} date1
 * @param {string} date2
 * @return {number}
 */
var daysBetweenDates = function(date1, date2) {
  return Math.abs(new Date(date1) - new Date(date2)) / (24 * 60 * 60 * 1000);
};
```

1361 Validate Binary Tree Nodes

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1361. Validate Binary Tree Nodes
 * https://leetcode.com/problems/validate-binary-tree-nodes/
 * Difficulty: Medium
 *
 * You have n binary tree nodes numbered from 0 to n - 1 where node i has two children
 * leftChild[i] and rightChild[i], return true if and only if all the given nodes form
 * exactly one valid binary tree.
 *
 * If node i has no left child then leftChild[i] will equal -1, similarly for the right child.
 *
 * Note that the nodes have no values and that we only use the node numbers in this problem.
 */

/**
 * @param {number} n
 * @param {number[]} leftChild
 * @param {number[]} rightChild
 * @return {boolean}
 */
var validateBinaryTreeNodes = function(n, leftChild, rightChild) {
  const parentCount = new Array(n).fill(0);
  let rootCandidate = -1;

  for (let i = 0; i < n; i++) {
    if (leftChild[i] !== -1) parentCount[leftChild[i]]++;
    if (rightChild[i] !== -1) parentCount[rightChild[i]]++;
  }

  for (let i = 0; i < n; i++) {
    if (parentCount[i] === 0) {
      if (rootCandidate !== -1) return false;
      rootCandidate = i;
    }
    if (parentCount[i] > 1) return false;
  }

  if (rootCandidate === -1) return false;

  return countNodes(rootCandidate, new Set()) === n;

  function countNodes(node, visited) {
    if (node === -1) return 0;
    if (visited.has(node)) return -1;
    visited.add(node);
    const leftNodes = countNodes(leftChild[node], visited);
    const rightNodes = countNodes(rightChild[node], visited);
    if (leftNodes === -1 || rightNodes === -1) return -1;
    return 1 + leftNodes + rightNodes;
  }
};
```

1362 Closest Divisors

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1362. Closest Divisors
 * https://leetcode.com/problems/closest-divisors/
 * Difficulty: Medium
 *
 * Given an integer num, find the closest two integers in absolute difference whose product
 * equals num + 1 or num + 2.
 *
 * Return the two integers in any order.
 */
```

```
/**
 * @param {number} num
 * @return {number[]}
 */
function closestDivisors(num) {
  const pair1 = findClosestPair(num + 1);
  const pair2 = findClosestPair(num + 2);

  return Math.abs(pair1[1] - pair1[0]) <= Math.abs(pair2[1] - pair2[0]) ? pair1 : pair2;

  function findClosestPair(target) {
    let minDifference = Infinity;
    let pair = [];

    for (let divisor = 1; divisor <= Math.sqrt(target); divisor++) {
      if (target % divisor === 0) {
        const complement = target / divisor;
        const difference = Math.abs(complement - divisor);
        if (difference < minDifference) {
          minDifference = difference;
          pair = [divisor, complement];
        }
      }
    }

    return pair;
  }
}
```

1363 Largest Multiple Of Three

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1363. Largest Multiple of Three
 * https://leetcode.com/problems/largest-multiple-of-three/
 * Difficulty: Hard
 *
 * Given an array of digits digits, return the largest multiple of three that can be formed by
 * concatenating some of the given digits in any order. If there is no answer return an empty
 * string.
 *
 * Since the answer may not fit in an integer data type, return the answer as a string. Note
 * that the returning answer must not contain unnecessary leading zeros.
 */

/**
 * @param {number[]} digits
 * @return {string}
 */
var largestMultipleOfThree = function(digits) {
  const counts = new Array(10).fill(0);
  let sum = 0;
  for (const digit of digits) {
    counts[digit]++;
    sum += digit;
  }

  const remainder = sum % 3;
  if (remainder === 0) {
    return buildLargestNumber(counts, sum, 0);
  }

  const sortedDigits = digits.slice().sort((a, b) => a - b);
  for (const digit of sortedDigits) {
    if (digit % 3 === remainder) {
      counts[digit]--;
      return buildLargestNumber(counts, sum - digit, 0);
    }
```

```
  }

  for (let i = 0; i < sortedDigits.length; i++) {
    for (let j = i + 1; j < sortedDigits.length; j++) {
      if ((sortedDigits[i] + sortedDigits[j]) % 3 === remainder) {
        counts[sortedDigits[i]]--;
        counts[sortedDigits[j]]--;
        return buildLargestNumber(counts, sum - sortedDigits[i] - sortedDigits[j], 0);
      }
    }
  }

  return '';

  function buildLargestNumber(counts, total, modulo) {
    let result = '';
    for (let digit = 9; digit >= 0; digit--) {
      let count = counts[digit];
      while (count > 0 && total >= modulo) {
        result += digit;
        total -= digit;
        count--;
      }
    }
    return result.length > 0 && result[0] === '0' ? '0' : result;
  }
};
```

1365 How Many Numbers Are Smaller Than The Current Number

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1365. How Many Numbers Are Smaller Than the Current Number
 * https://leetcode.com/problems/how-many-numbers-are-smaller-than-the-current-number/
 * Difficulty: Easy
 *
 * Given the array nums, for each nums[i] find out how many numbers in the array are smaller
 * than it. That is, for each nums[i] you have to count the number of valid j's such that
 * j != i and nums[j] < nums[i].
 *
 * Return the answer in an array.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var smallerNumbersThanCurrent = function(nums) {
  const sorted = nums.slice().sort((a, b) => a - b);
  return nums.map(num => sorted.indexOf(num));
};
```

1366 Rank Teams By Votes

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1366. Rank Teams by Votes
 * https://leetcode.com/problems/rank-teams-by-votes/
 * Difficulty: Medium
 *
 * In a special ranking system, each voter gives a rank from highest to lowest to all
 * teams participating in the competition.
```

```
 *
 * The ordering of teams is decided by who received the most position-one votes. If two
 * or more teams tie in the first position, we consider the second position to resolve
 * the conflict, if they tie again, we continue this process until the ties are resolved.
 * If two or more teams are still tied after considering all positions, we rank them
 * alphabetically based on their team letter.
 *
 * You are given an array of strings votes which is the votes of all voters in the ranking
 * systems. Sort all teams according to the ranking system described above.
 *
 * Return a string of all teams sorted by the ranking system.
 */

/**
 * @param {string[]} votes
 * @return {string}
 */
var rankTeams = function(votes) {
  if (votes.length === 1) {
    return votes[0];
  }

  const map = new Map(votes[0].split('').map(s => {
    return [s, new Array(votes[0].length).fill(0)];
  }));

  for (vote of votes) {
    for (let i = 0; i < vote.length; i++) {
      map.get(vote[i])[i]++;
    }
  }

  const result = votes[0].split('').sort((a, b) => {
    for (let i = 0; i < votes[0].length; i++) {
      if (map.get(a)[i] > map.get(b)[i]) return -1;
      if (map.get(a)[i] < map.get(b)[i]) return 1;
    }
    return a < b ? -1 : 1;
  });

  return result.join('');
};
```

1367 Linked List In Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1367. Linked List in Binary Tree
 * https://leetcode.com/problems/linked-list-in-binary-tree/
 * Difficulty: Medium
 *
 * Given a binary tree root and a linked list with head as the first node.
 *
 * Return True if all the elements in the linked list starting from the head correspond to
some
 * downward path connected in the binary tree otherwise return False.
 *
 * In this context downward path means a path that starts at some node and goes downwards.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * Definition for a binary tree node.
```

```
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {ListNode} head
 * @param {TreeNode} root
 * @return {boolean}
 */
var isSubPath = function(head, root) {
  if (!head) return true;
  if (!root) return false;
  return checkPath(head, root) || isSubPath(head, root.left) || isSubPath(head, root.right);

  function checkPath(listNode, treeNode) {
    if (!listNode) return true;
    if (!treeNode) return false;
    if (listNode.val !== treeNode.val) return false;
    return checkPath(listNode.next, treeNode.left) || checkPath(listNode.next,
treeNode.right);
  }
};
```

1368 Minimum Cost To Make At Least One Valid Path In A Grid

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1368. Minimum Cost to Make at Least One Valid Path in a Grid
 * https://leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid/
 * Difficulty: Hard
 *
 * Given an m x n grid. Each cell of the grid has a sign pointing to the next cell you should
 * visit if you are currently in this cell. The sign of grid[i][j] can be:
 * - 1 which means go to the cell to the right. (i.e go from grid[i][j] to grid[i][j + 1])
 * - 2 which means go to the cell to the left. (i.e go from grid[i][j] to grid[i][j - 1])
 * - 3 which means go to the lower cell. (i.e go from grid[i][j] to grid[i + 1][j])
 * - 4 which means go to the upper cell. (i.e go from grid[i][j] to grid[i - 1][j])
 *
 * Notice that there could be some signs on the cells of the grid that point outside the grid.
 *
 * You will initially start at the upper left cell (0, 0). A valid path in the grid is a path
 * that starts from the upper left cell (0, 0) and ends at the bottom-right cell (m - 1, n -
1)
 * following the signs on the grid. The valid path does not have to be the shortest.
 *
 * You can modify the sign on a cell with cost = 1. You can modify the sign on a cell one time
 * only.
 *
 * Return the minimum cost to make the grid have at least one valid path.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minCost = function(grid) {
  const queue = [[0, 0]];
  const directions = [[0, 1, 1], [0, -1, 2], [1, 0, 3], [-1, 0, 4]];
  const bfs = grid.map(r => r.map(_ => Infinity));
  bfs[0][0] = 0;

  while (queue.length > 0) {
    const [x, y] = queue.shift();
    for (const [dx, dy, value] of directions) {
      const [cX, cY] = [x + dx, y + dy];
      if (grid[cX]?.[cY]) {
        const updatedValue = bfs[x][y] + (grid[x][y] !== value);
```

```
        if (updatedValue < bfs[cX][cY]) {
          bfs[cX][cY] = updatedValue;
          queue.push([cX, cY]);
        }
      }
    }
  }

  return bfs.at(-1).at(-1);
};
```

1370 Increasing Decreasing String

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1370. Increasing Decreasing String
 * https://leetcode.com/problems/increasing-decreasing-string/
 * Difficulty: Easy
 *
 * You are given a string s. Reorder the string using the following algorithm:
 * 1. Remove the smallest character from s and append it to the result.
 * 2. Remove the smallest character from s that is greater than the last appended character,
 *   and append it to the result.
 * 3. Repeat step 2 until no more characters can be removed.
 * 4. Remove the largest character from s and append it to the result.
 * 5. Remove the largest character from s that is smaller than the last appended character,
 *   and append it to the result.
 * 6. Repeat step 5 until no more characters can be removed.
 * 7. Repeat steps 1 through 6 until all characters from s have been removed.
 *
 * If the smallest or largest character appears more than once, you may choose any occurrence
to
 * append to the result.
 *
 * Return the resulting string after reordering s using this algorithm.
 */

/**
 * @param {string} s
 * @return {string}
 */
var sortString = function(s) {
  const charCounts = new Array(26).fill(0);
  for (const char of s) {
    charCounts[char.charCodeAt(0) - 97]++;
  }

  let result = '';
  while (result.length < s.length) {
    for (let i = 0; i < 26; i++) {
      if (charCounts[i] > 0) {
        result += String.fromCharCode(i + 97);
        charCounts[i]--;
      }
    }
    for (let i = 25; i >= 0; i--) {
      if (charCounts[i] > 0) {
        result += String.fromCharCode(i + 97);
        charCounts[i]--;
      }
    }
  }

  return result;
};
```

1371 Find The Longest Substring Containing Vowels In Even Counts

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1371. Find the Longest Substring Containing Vowels in Even Counts
 * https://leetcode.com/problems/find-the-longest-substring-containing-vowels-in-even-counts/
 * Difficulty: Medium
 *
 * Given the string s, return the size of the longest substring containing each vowel an even
number
 * of times. That is, 'a', 'e', 'i', 'o', and 'u' must appear an even number of times.
 */

/**
 * @param {string} s
 * @return {number}
 */
var findTheLongestSubstring = function(s) {
  const vowelMap = new Map([['a', 0], ['e', 1], ['i', 2], ['o', 3], ['u', 4]]);
  const stateToIndex = new Map([[0, -1]]);
  let state = 0;
  let result = 0;

  for (let i = 0; i < s.length; i++) {
    if (vowelMap.has(s[i])) {
      state ^= 1 << vowelMap.get(s[i]);
    }
    if (stateToIndex.has(state)) {
      result = Math.max(result, i - stateToIndex.get(state));
    } else {
      stateToIndex.set(state, i);
    }
  }

  return result;
};
```

1372 Longest Zigzag Path In A Binary Tree

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1372. Longest ZigZag Path in a Binary Tree
 * https://leetcode.com/problems/longest-zigzag-path-in-a-binary-tree/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree.
 *
 * A ZigZag path for a binary tree is defined as follow:
 * - Choose any node in the binary tree and a direction (right or left).
 * - If the current direction is right, move to the right child of the current node;
 *   otherwise, move to the left child.
 * - Change the direction from right to left or from left to right.
 * - Repeat the second and third steps until you can't move in the tree.
 *
 * Zigzag length is defined as the number of nodes visited - 1. (A single node has a
 * length of 0).
 *
 * Return the longest ZigZag path contained in that tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
```

```
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var longestZigZag = function(root) {
  let result = 0;

  dfs(root, true, 0);
  dfs(root, false, 0);

  return result;

  function dfs(node, isLeft, total) {
    if (!node) return;
    result = Math.max(result, total);
    dfs(node.left, true, isLeft ? 1 : total + 1);
    dfs(node.right, false, isLeft ? total + 1 : 1);
  }
};
```

1373 Maximum Sum Bst In Binary Tree

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1373. Maximum Sum BST in Binary Tree
 * https://leetcode.com/problems/maximum-sum-bst-in-binary-tree/
 * Difficulty: Hard
 *
 * Given a binary tree root, return the maximum sum of all keys of any sub-tree which is also
a
 * Binary Search Tree (BST).
 *
 * Assume a BST is defined as follows:
 * - The left subtree of a node contains only nodes with keys less than the node's key.
 * - The right subtree of a node contains only nodes with keys greater than the node's key.
 * - Both the left and right subtrees must also be binary search trees.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxSumBST = function(root) {
  let result = 0;
  traverse(root);
  return result;

  function traverse(node) {
    if (!node) return { isBST: true, min: Infinity, max: -Infinity, sum: 0 };

    const left = traverse(node.left);
    const right = traverse(node.right);

    if (left.isBST && right.isBST && node.val > left.max && node.val < right.min) {
      const currentSum = node.val + left.sum + right.sum;
      result = Math.max(result, currentSum);
```

```
      return {
        isBST: true,
        min: Math.min(node.val, left.min),
        max: Math.max(node.val, right.max),
        sum: currentSum
      };
    }

    return { isBST: false, min: 0, max: 0, sum: 0 };
  }
};
```

1374 Generate A String With Characters That Have Odd Counts

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1374. Generate a String With Characters That Have Odd Counts
 * https://leetcode.com/problems/generate-a-string-with-characters-that-have-odd-counts/
 * Difficulty: Easy
 *
 * Given an integer n, return a string with n characters such that each character in such
 * string occurs an odd number of times.
 *
 * The returned string must contain only lowercase English letters. If there are multiples
 * valid strings, return any of them.
 */

/**
 * @param {number} n
 * @return {string}
 */
var generateTheString = function(n) {
  return 'a'.repeat(n % 2 ? n : n - 1) + (n % 2 ? '' : 'z');
};
```

1375 Number Of Times Binary String Is Prefix Aligned

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1375. Number of Times Binary String Is Prefix-Aligned
 * https://leetcode.com/problems/number-of-times-binary-string-is-prefix-aligned/
 * Difficulty: Medium
 *
 * You have a 1-indexed binary string of length n where all the bits are 0 initially. We will
 * flip all the bits of this binary string (i.e., change them from 0 to 1) one by one. You are
 * given a 1-indexed integer array flips where flips[i] indicates that the bit at index i will
 * be flipped in the ith step.
 *
 * A binary string is prefix-aligned if, after the ith step, all the bits in the inclusive
range
 * [1, i] are ones and all the other bits are zeros.
 *
 * Return the number of times the binary string is prefix-aligned during the flipping process.
 */

/**
 * @param {number[]} light
 * @return {number}
 */
var numTimesAllBlue = function(flips) {
  let maxFlipped = 0;
  let result = 0;
```

```
  flips.forEach((position, step) => {
    maxFlipped = Math.max(maxFlipped, position);
    if (maxFlipped === step + 1) result++;
  });

  return result;
};
```

1376 Time Needed To Inform All Employees

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1376. Time Needed to Inform All Employees
 * https://leetcode.com/problems/time-needed-to-inform-all-employees/
 * Difficulty: Medium
 *
 * A company has n employees with a unique ID for each employee from 0 to n - 1. The head
 * of the company is the one with headID.
 *
 * Each employee has one direct manager given in the manager array where manager[i] is the
 * direct manager of the i-th employee, manager[headID] = -1. Also, it is guaranteed that
 * the subordination relationships have a tree structure.
 *
 * The head of the company wants to inform all the company employees of an urgent piece of
 * news. He will inform his direct subordinates, and they will inform their subordinates,
 * and so on until all employees know about the urgent news.
 *
 * The i-th employee needs informTime[i] minutes to inform all of his direct subordinates
 * (i.e., After informTime[i] minutes, all his direct subordinates can start spreading the
news).
 *
 * Return the number of minutes needed to inform all the employees about the urgent news.
 */

/**
 * @param {number} n
 * @param {number} headID
 * @param {number[]} manager
 * @param {number[]} informTime
 * @return {number}
 */
var numOfMinutes = function(n, headID, manager, informTime) {
  const adjacencyList = Array.from({ length: n }, () => []);

  for (let i = 0; i < n; i++) {
    if (manager[i] !== -1) {
      adjacencyList[manager[i]].push(i);
    }
  }

  return calculateTime(headID);

  function calculateTime(employee) {
    let maxSubordinateTime = 0;

    for (const subordinate of adjacencyList[employee]) {
      maxSubordinateTime = Math.max(maxSubordinateTime, calculateTime(subordinate));
    }

    return informTime[employee] + maxSubordinateTime;
  }
};
```

1377 Frog Position After T Seconds

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1377. Frog Position After T Seconds
 * https://leetcode.com/problems/frog-position-after-t-seconds/
 * Difficulty: Hard
 *
 * Given an undirected tree consisting of n vertices numbered from 1 to n. A frog starts
 * jumping from vertex 1. In one second, the frog jumps from its current vertex to another
 * unvisited vertex if they are directly connected. The frog can not jump back to a visited
 * vertex. In case the frog can jump to several vertices, it jumps randomly to one of them
 * with the same probability. Otherwise, when the frog can not jump to any unvisited vertex,
 * it jumps forever on the same vertex.
 *
 * The edges of the undirected tree are given in the array edges, where edges[i] = [ai, bi]
 * means that exists an edge connecting the vertices ai and bi.
 *
 * Return the probability that after t seconds the frog is on the vertex target. Answers
 * within 10-5 of the actual answer will be accepted.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number} t
 * @param {number} target
 * @return {number}
 */
var frogPosition = function(n, edges, t, target) {
  const adjacencyList = Array.from({ length: n + 1 }, () => []);
  for (const [a, b] of edges) {
    adjacencyList[a].push(b);
    adjacencyList[b].push(a);
  }

  return explore(1, 0, 1, new Set());

  function explore(vertex, time, probability, visited) {
    if (time > t) return 0;
    if (vertex === target && time === t) return probability;
    if (vertex === target && adjacencyList[vertex].every(v => visited.has(v))) return
probability;

    const unvisitedNeighbors = adjacencyList[vertex].filter(v => !visited.has(v));
    if (unvisitedNeighbors.length === 0) return 0;

    const nextProbability = probability / unvisitedNeighbors.length;
    visited.add(vertex);

    for (const neighbor of unvisitedNeighbors) {
      const result = explore(neighbor, time + 1, nextProbability, visited);
      if (result > 0) return result;
    }

    return 0;
  }
};
```

1379 Find A Corresponding Node Of A Binary Tree In A Clone Of That Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
```

```
 * 1379. Find a Corresponding Node of a Binary Tree in a Clone of That Tree
 * https://leetcode.com/problems/find-a-corresponding-node-of-a-binary-tree-in-a-clone-of-that
-tree/
 * Difficulty: Easy
 *
 * Given two binary trees original and cloned and given a reference to a node target in the
original
 * tree.
 *
 * The cloned tree is a copy of the original tree.
 *
 * Return a reference to the same node in the cloned tree.
 *
 * Note that you are not allowed to change any of the two trees or the target node and the
answer
 * must be a reference to a node in the cloned tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} original
 * @param {TreeNode} cloned
 * @param {TreeNode} target
 * @return {TreeNode}
 */
var getTargetCopy = function(original, cloned, target) {
  return traverse(original, cloned);

  function traverse(originalNode, clonedNode) {
    if (!originalNode) return null;
    if (originalNode === target) return clonedNode;

    const leftResult = traverse(originalNode.left, clonedNode.left);
    if (leftResult) return leftResult;

    return traverse(originalNode.right, clonedNode.right);
  }
};
```

1380 Lucky Numbers In A Matrix

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1380. Lucky Numbers in a Matrix
 * https://leetcode.com/problems/lucky-numbers-in-a-matrix/
 * Difficulty: Easy
 *
 * Given an m x n matrix of distinct numbers, return all lucky numbers
 * in the matrix in any order.
 *
 * A lucky number is an element of the matrix such that it is the minimum
 * element in its row and maximum in its column.
 */

/**
 * @param {number[][]} matrix
 * @return {number[]}
 */
var luckyNumbers  = function(matrix) {
  const min = matrix.map(row => Math.min(...row));
  const max = matrix[0].map((_, i) => Math.max(...matrix.map(row => row[i])));

  return min.filter(value => max.includes(value));
```

```
};
```

1381 Design A Stack With Increment Operation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1381. Design a Stack With Increment Operation
 * https://leetcode.com/problems/design-a-stack-with-increment-operation/
 * Difficulty: Medium
 *
 * Design a stack that supports increment operations on its elements.
 *
 * Implement the CustomStack class:
 * - CustomStack(int maxSize) Initializes the object with maxSize which is the maximum number
 *   of elements in the stack.
 * - void push(int x) Adds x to the top of the stack if the stack has not reached the maxSize.
 * - int pop() Pops and returns the top of the stack or -1 if the stack is empty.
 * - void inc(int k, int val) Increments the bottom k elements of the stack by val. If there
are
 *   less than k elements in the stack, increment all the elements in the stack.
 */

/**
 * @param {number} maxSize
 */
var CustomStack = function(maxSize) {
  this.elements = [];
  this.capacity = maxSize;
};

/**
 * @param {number} x
 * @return {void}
 */
CustomStack.prototype.push = function(x) {
  if (this.elements.length < this.capacity) {
    this.elements.push(x);
  }
};

/**
 * @return {number}
 */
CustomStack.prototype.pop = function() {
  return this.elements.length ? this.elements.pop() : -1;
};

/**
 * @param {number} k
 * @param {number} val
 * @return {void}
 */
CustomStack.prototype.increment = function(k, val) {
  const limit = Math.min(k, this.elements.length);
  for (let i = 0; i < limit; i++) {
    this.elements[i] += val;
  }
};
```

1382 Balance A Binary Search Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
```

```
 * 1382. Balance a Binary Search Tree
 * https://leetcode.com/problems/balance-a-binary-search-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary search tree, return a balanced binary search tree with the same
 * node values. If there is more than one answer, return any of them.
 *
 * A binary search tree is balanced if the depth of the two subtrees of every node never
differs
 * by more than 1.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var balanceBST = function(root) {
  const nodes = [];
  collectInOrder(root);
  return buildBalancedTree(0, nodes.length - 1);

  function collectInOrder(node) {
    if (!node) return;
    collectInOrder(node.left);
    nodes.push(node.val);
    collectInOrder(node.right);
  }

  function buildBalancedTree(start, end) {
    if (start > end) return null;
    const mid = Math.floor((start + end) / 2);
    const node = new TreeNode(nodes[mid]);
    node.left = buildBalancedTree(start, mid - 1);
    node.right = buildBalancedTree(mid + 1, end);
    return node;
  }
};
```

1383 Maximum Performance Of A Team

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1383. Maximum Performance of a Team
 * https://leetcode.com/problems/maximum-performance-of-a-team/
 * Difficulty: Hard
 *
 * You are given two integers n and k and two integer arrays speed and efficiency both of
length n.
 * There are n engineers numbered from 1 to n. speed[i] and efficiency[i] represent the speed
and
 * efficiency of the ith engineer respectively.
 *
 * Choose at most k different engineers out of the n engineers to form a team with the maximum
 * performance.
 *
 * The performance of a team is the sum of its engineers' speeds multiplied by the minimum
 * efficiency among its engineers.
 *
 * Return the maximum performance of this team. Since the answer can be a huge number, return
it
```

```
 * modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number[]} speed
 * @param {number[]} efficiency
 * @param {number} k
 * @return {number}
 */
var maxPerformance = function(n, speed, efficiency, k) {
  const engineers = Array.from({ length: n }, (_, i) => [efficiency[i], speed[i]])
    .sort((a, b) => b[0] - a[0]);
  const speedHeap = [];
  let totalSpeed = 0n;
  let maxPerf = 0n;
  const MOD = 1000000007n;

  function insertSpeed(val) {
    speedHeap.push(val);
    let i = speedHeap.length - 1;
    while (i > 0) {
      const parent = Math.floor((i - 1) / 2);
      if (speedHeap[parent] <= speedHeap[i]) break;
      [speedHeap[i], speedHeap[parent]] = [speedHeap[parent], speedHeap[i]];
      i = parent;
    }
  }

  function removeMinSpeed() {
    const min = speedHeap[0];
    speedHeap[0] = speedHeap.pop();
    let i = 0;
    while (true) {
      const left = 2 * i + 1;
      const right = 2 * i + 2;
      let smallest = i;
      if (left < speedHeap.length && speedHeap[left] < speedHeap[smallest]) {
        smallest = left;
      }
      if (right < speedHeap.length && speedHeap[right] < speedHeap[smallest]) {
        smallest = right;
      }
      if (smallest === i) break;
      [speedHeap[i], speedHeap[smallest]] = [speedHeap[smallest], speedHeap[i]];
      i = smallest;
    }
    return min;
  }

  for (const [eff, spd] of engineers) {
    insertSpeed(spd);
    totalSpeed += BigInt(spd);

    if (speedHeap.length > k) {
      totalSpeed -= BigInt(removeMinSpeed());
    }

    maxPerf = maxPerf > totalSpeed * BigInt(eff) ? maxPerf : totalSpeed * BigInt(eff);
  }

  return Number(maxPerf % MOD);
};
```

1385 Find The Distance Value Between Two Arrays

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1385. Find the Distance Value Between Two Arrays
```

```
 * https://leetcode.com/problems/find-the-distance-value-between-two-arrays/
 * Difficulty: Easy
 *
 * Given two integer arrays arr1 and arr2, and the integer d, return the distance value
between
 * the two arrays.
 *
 * The distance value is defined as the number of elements arr1[i] such that there is not any
 * element arr2[j] where |arr1[i]-arr2[j]| <= d.
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
 * @param {number} d
 * @return {number}
 */
var findTheDistanceValue = function(arr1, arr2, d) {
  let result = 0;

  for (const num1 of arr1) {
    let isValid = true;
    for (const num2 of arr2) {
      if (Math.abs(num1 - num2) <= d) {
        isValid = false;
        break;
      }
    }
    if (isValid) result++;
  }

  return result;
};
```

1386 Cinema Seat Allocation

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1386. Cinema Seat Allocation
 * https://leetcode.com/problems/cinema-seat-allocation/
 * Difficulty: Medium
 *
 * A cinema has n rows of seats, numbered from 1 to n and there are ten seats in each row,
labelled
 * from 1 to 10 as shown in the figure above.
 *
 * Given the array reservedSeats containing the numbers of seats already reserved, for
example,
 * reservedSeats[i] = [3,8] means the seat located in row 3 and labelled with 8 is already
reserved.
 *
 * Return the maximum number of four-person groups you can assign on the cinema seats. A
four-person
 * group occupies four adjacent seats in one single row. Seats across an aisle (such as [3,3]
and
 * [3,4]) are not considered to be adjacent, but there is an exceptional case on which an
aisle
 * split a four-person group, in that case, the aisle split a four-person group in the middle,
 which
 * means to have two people on each side.
 */

/**
 * @param {number} n
 * @param {number[][]} reservedSeats
 * @return {number}
 */
var maxNumberOfFamilies = function(n, reservedSeats) {
  const rowReservations = new Map();
```

```
  for (const [row, seat] of reservedSeats) {
    if (!rowReservations.has(row)) {
      rowReservations.set(row, new Set());
    }
    rowReservations.get(row).add(seat);
  }

  let result = 2 * (n - rowReservations.size);

  for (const seats of rowReservations.values()) {
    let canPlaceGroup = false;

    if (!seats.has(2) && !seats.has(3) && !seats.has(4) && !seats.has(5)) {
      result++;
      canPlaceGroup = true;
    }
    if (!seats.has(6) && !seats.has(7) && !seats.has(8) && !seats.has(9)) {
      result++;
      canPlaceGroup = true;
    }
    if (!canPlaceGroup && !seats.has(4) && !seats.has(5) && !seats.has(6) && !seats.has(7)) {
      result++;
    }
  }

  return result;
};
```

1387 Sort Integers By The Power Value

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1387. Sort Integers by The Power Value
 * https://leetcode.com/problems/sort-integers-by-the-power-value/
 * Difficulty: Medium
 *
 * The power of an integer x is defined as the number of steps needed to transform x into
 * 1 using the following steps:
 * - if x is even then x = x / 2
 * - if x is odd then x = 3 * x + 1
 *
 * For example, the power of x = 3 is 7 because 3 needs 7 steps to become 1
 * (3 --> 10 --> 5 --> 16 --> 8 --> 4 --> 2 --> 1).
 *
 * Given three integers lo, hi and k. The task is to sort all integers in the interval
 * [lo, hi] by the power value in ascending order, if two or more integers have the same
 * power value sort them by ascending order.
 *
 * Return the kth integer in the range [lo, hi] sorted by the power value.
 *
 * Notice that for any integer x (lo <= x <= hi) it is guaranteed that x will transform
 * into 1 using these steps and that the power of x is will fit in a 32-bit signed integer.
 */

/**
 * @param {number} lo
 * @param {number} hi
 * @param {number} k
 * @return {number}
 */
var getKth = function(lo, hi, k) {
  const powerValues = new Map();
  const numbers = Array.from({ length: hi - lo + 1 }, (_, i) => lo + i);

  numbers.sort((a, b) => {
    const powerA = getPower(a);
    const powerB = getPower(b);
```

```
    return powerA === powerB ? a - b : powerA - powerB;
  });

  return numbers[k - 1];

  function calculatePower(num) {
    let steps = 0;
    while (num !== 1) {
      if (num % 2 === 0) {
        num = num / 2;
      } else {
        num = 3 * num + 1;
      }
      steps++;
    }
    return steps;
  }

  function getPower(num) {
    if (!powerValues.has(num)) {
      powerValues.set(num, calculatePower(num));
    }
    return powerValues.get(num);
  }
};
```

1388 Pizza With 3N Slices

HARD

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1388. Pizza With 3n Slices
 * https://leetcode.com/problems/pizza-with-3n-slices/
 * Difficulty: Hard
 *
 * There is a pizza with 3n slices of varying size, you and your friends will take slices
 * of pizza as follows:
 * - You will pick any pizza slice.
 * - Your friend Alice will pick the next slice in the anti-clockwise direction of your pick.
 * - Your friend Bob will pick the next slice in the clockwise direction of your pick.
 * - Repeat until there are no more slices of pizzas.
 *
 * Given an integer array slices that represent the sizes of the pizza slices in a clockwise
 * direction, return the maximum possible sum of slice sizes that you can pick.
 */

/**
 * @param {number[]} slices
 * @return {number}
 */
var maxSizeSlices = function(slices) {
  const n = slices.length;
  const count = n / 3;
  const case1 = selectMaxSum(slices, count - 1, 2, n - 2);
  const case2 = selectMaxSum(slices, count, 1, n - 1);

  return Math.max(slices[0] + case1, case2);

  function selectMaxSum(arr, count, start, end, memo = new Map()) {
    const key = `${count},${start},${end}`;
    if (count === 0 || start > end) return 0;
    if (memo.has(key)) return memo.get(key);

    const includeFirst = arr[start] + selectMaxSum(arr, count - 1, start + 2, end, memo);
    const excludeFirst = selectMaxSum(arr, count, start + 1, end, memo);
    const result = Math.max(includeFirst, excludeFirst);

    memo.set(key, result);
    return result;
  }
```

```
};
```

1389 Create Target Array In The Given Order

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1389. Create Target Array in the Given Order
 * https://leetcode.com/problems/create-target-array-in-the-given-order/
 * Difficulty: Easy
 *
 * Given two arrays of integers nums and index. Your task is to create target array
 * under the following rules:
 *
 * - Initially target array is empty.
 * - From left to right read nums[i] and index[i], insert at index index[i] the value
 * nums[i] in target array.
 * - Repeat the previous step until there are no elements to read in nums and index.
 * - Return the target array.
 *
 * It is guaranteed that the insertion operations will be valid.
 */

/**
 * @param {number[]} nums
 * @param {number[]} index
 * @return {number[]}
 */
var createTargetArray = function(nums, index) {
  const result = [];
  index.forEach((i, j) => result.splice(i, 0, nums[j]));
  return result;
};
```

1390 Four Divisors

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1390. Four Divisors
 * https://leetcode.com/problems/four-divisors/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the sum of divisors of the integers in that array
 * that have exactly four divisors. If there is no such integer in the array, return 0.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumFourDivisors = function(nums) {
  function countDivisors(num) {
    const divisors = new Set();
    for (let i = 1; i * i <= num; i++) {
      if (num % i === 0) {
        divisors.add(i);
        divisors.add(num / i);
      }
    }
    return divisors;
  }

  let result = 0;
  for (const num of nums) {
```

```
    const divisors = countDivisors(num);
    if (divisors.size === 4) {
      const sum = [...divisors].reduce((acc, val) => acc + val, 0);
      result += sum;
    }
  }

  return result;
};
```

1391 Check If There Is A Valid Path In A Grid

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1391. Check if There is a Valid Path in a Grid
 * https://leetcode.com/problems/check-if-there-is-a-valid-path-in-a-grid/
 * Difficulty: Medium
 *
 * You are given an m x n grid. Each cell of grid represents a street. The street of
grid[i][j]
 * can be:
 * - 1 which means a street connecting the left cell and the right cell.
 * - 2 which means a street connecting the upper cell and the lower cell.
 * - 3 which means a street connecting the left cell and the lower cell.
 * - 4 which means a street connecting the right cell and the lower cell.
 * - 5 which means a street connecting the left cell and the upper cell.
 * - 6 which means a street connecting the right cell and the upper cell.
 *
 * You will initially start at the street of the upper-left cell (0, 0). A valid path in the
 * grid is a path that starts from the upper left cell (0, 0) and ends at the bottom-right
 * cell (m - 1, n - 1). The path should only follow the streets.
 *
 * Notice that you are not allowed to change any street.
 *
 * Return true if there is a valid path in the grid or false otherwise.
 */

/**
 * @param {number[][]} grid
 * @return {boolean}
 */
var hasValidPath = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const directions = {
    1: [[0, -1, [1, 4, 6]], [0, 1, [1, 3, 5]]],
    2: [[-1, 0, [2, 3, 4]], [1, 0, [2, 5, 6]]],
    3: [[0, -1, [1, 4, 6]], [1, 0, [2, 5, 6]]],
    4: [[0, 1, [1, 3, 5]], [1, 0, [2, 5, 6]]],
    5: [[0, -1, [1, 4, 6]], [-1, 0, [2, 3, 4]]],
    6: [[0, 1, [1, 3, 5]], [-1, 0, [2, 3, 4]]]
  };

  return explorePath(0, 0);

  function isValid(x, y) {
    return x >= 0 && x < rows && y >= 0 && y < cols;
  }

  function explorePath(x, y, visited = new Set()) {
    if (!isValid(x, y) || visited.has(`${x},${y}`)) return false;
    if (x === rows - 1 && y === cols - 1) return true;

    visited.add(`${x},${y}`);
    const street = grid[x][y];

    for (const [dx, dy, validStreets] of directions[street]) {
      const newX = x + dx;
      const newY = y + dy;
```

```
      if (isValid(newX, newY) && validStreets.includes(grid[newX][newY])) {
        if (explorePath(newX, newY, visited)) return true;
      }
    }

    return false;
  }
};
```

1392 Longest Happy Prefix

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1392. Longest Happy Prefix
 * https://leetcode.com/problems/longest-happy-prefix/
 * Difficulty: Hard
 *
 * A string is called a happy prefix if is a non-empty prefix which is also a suffix
 * (excluding itself).
 *
 * Given a string s, return the longest happy prefix of s. Return an empty string ""
 * if no such prefix exists.
 */

/**
 * @param {string} s
 * @return {string}
 */
var longestPrefix = function(s) {
  let prefixLength = 0;
  const prefixHash = new Array(s.length).fill(0);

  for (let i = 1; i < s.length; i++) {
    while (prefixLength > 0 && s[i] !== s[prefixLength]) {
      prefixLength = prefixHash[prefixLength - 1];
    }
    if (s[i] === s[prefixLength]) {
      prefixLength++;
    }
    prefixHash[i] = prefixLength;
  }

  return s.slice(0, prefixHash[s.length - 1]);
};
```

1394 Find Lucky Integer In An Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1394. Find Lucky Integer in an Array
 * https://leetcode.com/problems/find-lucky-integer-in-an-array/
 * Difficulty: Easy
 *
 * Given an array of integers arr, a lucky integer is an integer that has a frequency
 * in the array equal to its value.
 *
 * Return the largest lucky integer in the array. If there is no lucky integer return -1.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
```

```
var findLucky = function(arr) {
  const frequencyMap = new Map();
  let result = -1;

  for (const num of arr) {
    frequencyMap.set(num, (frequencyMap.get(num) || 0) + 1);
  }
  for (const [num, count] of frequencyMap) {
    if (num === count && num > result) {
      result = num;
    }
  }

  return result;
};
```

1395 Count Number Of Teams

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1395. Count Number of Teams
 * https://leetcode.com/problems/count-number-of-teams/
 * Difficulty: Medium
 *
 * There are n soldiers standing in a line. Each soldier is assigned a unique rating value.
 *
 * You have to form a team of 3 soldiers amongst them under the following rules:
 * - Choose 3 soldiers with index (i, j, k) with rating (rating[i], rating[j], rating[k]).
 * - A team is valid if: (rating[i] < rating[j] < rating[k]) or (rating[i] > rating[j] >
rating[k])
 *   where (0 <= i < j < k < n).
 *
 * Return the number of teams you can form given the conditions. (soldiers can be part of
 * multiple teams).
 */

/**
 * @param {number[]} rating
 * @return {number}
 */
var numTeams = function(rating) {
  let result = 0;
  const n = rating.length;

  for (let j = 1; j < n - 1; j++) {
    let leftLess = 0;
    let leftGreater = 0;
    let rightLess = 0;
    let rightGreater = 0;

    for (let i = 0; i < j; i++) {
      if (rating[i] < rating[j]) leftLess++;
      else leftGreater++;
    }

    for (let k = j + 1; k < n; k++) {
      if (rating[k] < rating[j]) rightLess++;
      else rightGreater++;
    }

    result += leftLess * rightGreater + leftGreater * rightLess;
  }

  return result;
};
```

1396 Design Underground System

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1396. Design Underground System
 * https://leetcode.com/problems/design-underground-system/
 * Difficulty: Medium
 *
 * An underground railway system is keeping track of customer travel times between different
 * stations. They are using this data to calculate the average time it takes to travel from
 * one station to another.
 *
 * Implement the UndergroundSystem class:
 * - void checkIn(int id, string stationName, int t)
 *   - A customer with a card ID equal to id, checks in at the station stationName at time t.
 *   - A customer can only be checked into one place at a time.
 * - void checkOut(int id, string stationName, int t)
 *   - A customer with a card ID equal to id, checks out from the station stationName at time
t.
 * - double getAverageTime(string startStation, string endStation)
 *   - Returns the average time it takes to travel from startStation to endStation.
 *   - The average time is computed from all the previous traveling times from startStation to
 *     endStation that happened directly, meaning a check in at startStation followed by a
check
 *     out from endStation.
 *   - The time it takes to travel from startStation to endStation may be different from the
 *     time it takes to travel from endStation to startStation.
 *
 * There will be at least one customer that has traveled from startStation to endStation
before
 * getAverageTime is called.
 * You may assume all calls to the checkIn and checkOut methods are consistent. If a customer
 * checks in at time t1 then checks out at time t2, then t1 < t2. All events happen in
 * chronological order.
 */


var UndergroundSystem = function() {
  this.travels = new Map();
  this.averages = new Map();
};

/**
 * @param {number} id
 * @param {string} stationName
 * @param {number} t
 * @return {void}
 */
UndergroundSystem.prototype.checkIn = function(id, stationName, t) {
  this.travels.set(id, { startStation: stationName, startTime: t });
};

/**
 * @param {number} id
 * @param {string} stationName
 * @param {number} t
 * @return {void}
 */
UndergroundSystem.prototype.checkOut = function(id, stationName, t) {
  const { startStation, startTime } = this.travels.get(id);
  const route = `${startStation}-${stationName}`;
  const duration = t - startTime;

  if (!this.averages.has(route)) {
    this.averages.set(route, { totalTime: 0, count: 0 });
  }

  const record = this.averages.get(route);
  record.totalTime += duration;
```

```
  record.count += 1;

  this.travels.delete(id);
};

/**
 * @param {string} startStation
 * @param {string} endStation
 * @return {number}
 */
UndergroundSystem.prototype.getAverageTime = function(startStation, endStation) {
  const route = `${startStation}-${endStation}`;
  const { totalTime, count } = this.averages.get(route);
  return totalTime / count;
};
```

1397 Find All Good Strings

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1397. Find All Good Strings
 * https://leetcode.com/problems/find-all-good-strings/
 * Difficulty: Hard
 *
 * Given the strings s1 and s2 of size n and the string evil, return the number of good
strings.
 *
 * A good string has size n, it is alphabetically greater than or equal to s1, it is
alphabetically
 * smaller than or equal to s2, and it does not contain the string evil as a substring. Since
the
 * answer can be a huge number, return this modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {string} s1
 * @param {string} s2
 * @param {string} evil
 * @return {number}
 */
var findGoodStrings = function(n, s1, s2, evil) {
  const MOD = 1e9 + 7;
  const memo = new Map();
  const evilLPS = computeLPS(evil);

  function computeLPS(pattern) {
    const lps = new Array(pattern.length).fill(0);
    let length = 0;
    let i = 1;
    while (i < pattern.length) {
      if (pattern[i] === pattern[length]) {
        length++;
        lps[i] = length;
        i++;
      } else {
        if (length !== 0) {
          length = lps[length - 1];
        } else {
          lps[i] = 0;
          i++;
        }
      }
    }
    return lps;
  }

  function countValid(pos, evilMatched, bound1, bound2, str1, str2, evilStr, lps) {
```

```
    if (evilMatched === evilStr.length) return 0;
    if (pos === n) return 1;

    const key = `${pos}:${evilMatched}:${bound1}:${bound2}`;
    if (memo.has(key)) return memo.get(key);

    let result = 0;
    const start = bound1 ? str1[pos] : 'a';
    const end = bound2 ? str2[pos] : 'z';

    for (let c = start; c <= end; c = String.fromCharCode(c.charCodeAt(0) + 1)) {
      let newEvilMatched = evilMatched;
      while (newEvilMatched > 0 && c !== evilStr[newEvilMatched]) {
        newEvilMatched = lps[newEvilMatched - 1];
      }
      if (c === evilStr[newEvilMatched]) {
        newEvilMatched++;
      }

      const newBound1 = bound1 && c === str1[pos];
      const newBound2 = bound2 && c === str2[pos];

      result = (result + countValid(
        pos + 1, newEvilMatched, newBound1, newBound2, str1, str2, evilStr, lps
      )) % MOD;
    }

    memo.set(key, result);
    return result;
  }

  return countValid(0, 0, true, true, s1, s2, evil, evilLPS);
};
```

1399 Count Largest Group

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1399. Count Largest Group
 * https://leetcode.com/problems/count-largest-group/
 * Difficulty: Easy
 *
 * You are given an integer n.
 *
 * Each number from 1 to n is grouped according to the sum of its digits.
 *
 * Return the number of groups that have the largest size.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countLargestGroup = function(n) {
  const map = new Map();
  let maxSize = 0;

  for (let num = 1; num <= n; num++) {
    const digitSum = calculateDigitSum(num);
    const size = (map.get(digitSum) || 0) + 1;
    map.set(digitSum, size);
    maxSize = Math.max(maxSize, size);
  }

  let result = 0;
  for (const size of map.values()) {
    if (size === maxSize) result++;
  }
```

```
  return result;
};

function calculateDigitSum(num) {
  let sum = 0;
  while (num > 0) {
    sum += num % 10;
    num = Math.floor(num / 10);
  }
  return sum;
}
```

1400 Construct K Palindrome Strings

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1400. Construct K Palindrome Strings
 * https://leetcode.com/problems/construct-k-palindrome-strings/
 * Difficulty: Medium
 *
 * Given a string s and an integer k, return true if you can use all the characters in s to
 * construct k palindrome strings or false otherwise.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {boolean}
 */
var canConstruct = function(s, k) {
  const occurrences = new Array(26).fill(0);
  s.split('').forEach(c => occurrences[c.charCodeAt(0) - 'a'.charCodeAt(0)]++);

  const oddCount = occurrences.filter(n => n % 2 !== 0).length;
  return k <= s.length && oddCount <= k;
};
```

1401 Circle And Rectangle Overlapping

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1401. Circle and Rectangle Overlapping
 * https://leetcode.com/problems/circle-and-rectangle-overlapping/
 * Difficulty: Medium
 *
 * You are given a circle represented as (radius, xCenter, yCenter) and an axis-aligned
rectangle
 * represented as (x1, y1, x2, y2), where (x1, y1) are the coordinates of the bottom-left
corner,
 * and (x2, y2) are the coordinates of the top-right corner of the rectangle.
 *
 * Return true if the circle and rectangle are overlapped otherwise return false. In other
words,
 * check if there is any point (xi, yi) that belongs to the circle and the rectangle at the
same
 * time.
 */

/**
 * @param {number} radius
 * @param {number} xCenter
 * @param {number} yCenter
```

```
 * @param {number} x1
 * @param {number} y1
 * @param {number} x2
 * @param {number} y2
 * @return {boolean}
 */
var checkOverlap = function(radius, xCenter, yCenter, x1, y1, x2, y2) {
  const closestX = Math.max(x1, Math.min(x2, xCenter));
  const closestY = Math.max(y1, Math.min(y2, yCenter));

  return ((xCenter - closestX) ** 2 + (yCenter - closestY) ** 2) <= radius ** 2;
};
```

1402 Reducing Dishes

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1402. Reducing Dishes
 * https://leetcode.com/problems/reducing-dishes/
 * Difficulty: Hard
 *
 * A chef has collected data on the satisfaction level of his n dishes.
 * Chef can cook any dish in 1 unit of time.
 *
 * Like-time coefficient of a dish is defined as the time taken to cook
 * that dish including previous dishes multiplied by its satisfaction
 * level i.e. time[i] * satisfaction[i].
 *
 * Return the maximum sum of like-time coefficient that the chef can
 * obtain after dishes preparation.
 *
 * Dishes can be prepared in any order and the chef can discard some
 * dishes to get this maximum value.
 */

/**
 * @param {number[]} satisfaction
 * @return {number}
 */
var maxSatisfaction = function(satisfaction) {
  let max = 0;

  satisfaction.sort((a, b) => a - b);

  while (satisfaction.length) {
    max = Math.max(
      max,
      satisfaction.reduce((total, n, i) => total + n * (i + 1), 0)
    );
    satisfaction.shift();
  }

  return max;
};
```

1403 Minimum Subsequence In Non Increasing Order

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1403. Minimum Subsequence in Non-Increasing Order
 * https://leetcode.com/problems/minimum-subsequence-in-non-increasing-order/
 * Difficulty: Easy
 *
```

```
 * Given the array nums, obtain a subsequence of the array whose sum of elements is strictly
 * greater than the sum of the non included elements in such subsequence.
 *
 * If there are multiple solutions, return the subsequence with minimum size and if there
still
 * exist multiple solutions, return the subsequence with the maximum total sum of all its
elements.
 * A subsequence of an array can be obtained by erasing some (possibly zero) elements from the
 * array.
 *
 * Note that the solution with the given constraints is guaranteed to be unique. Also return
the
 * answer sorted in non-increasing order.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var minSubsequence = function(nums) {
  const sorted = nums.sort((a, b) => b - a);
  const totalSum = sorted.reduce((sum, num) => sum + num, 0);
  const result = [];
  let currentSum = 0;

  for (const num of sorted) {
    currentSum += num;
    result.push(num);
    if (currentSum > totalSum - currentSum) break;
  }

  return result;
};
```

1404 Number Of Steps To Reduce A Number In Binary Representation To One

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1404. Number of Steps to Reduce a Number in Binary Representation to One
 * https://leetcode.com/problems/number-of-steps-to-reduce-a-number-in-binary-representation-t
o-one/
 * Difficulty: Medium
 *
 * Given the binary representation of an integer as a string s, return the number of steps to
reduce
 * it to 1 under the following rules:
 * - If the current number is even, you have to divide it by 2.
 * - If the current number is odd, you have to add 1 to it.
 *
 * It is guaranteed that you can always reach one for all test cases.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numSteps = function(s) {
  let num = BigInt(`0b${s}`);
  let steps = 0;

  while (num !== 1n) {
    if (num % 2n === 0n) {
      num /= 2n;
    } else {
      num += 1n;
    }
    steps++;
  }
```

```
  return steps;
};
```

1405 Longest Happy String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1405. Longest Happy String
 * https://leetcode.com/problems/longest-happy-string/
 * Difficulty: Medium
 *
 * A string s is called happy if it satisfies the following conditions:
 * - s only contains the letters 'a', 'b', and 'c'.
 * - s does not contain any of "aaa", "bbb", or "ccc" as a substring.
 * - s contains at most a occurrences of the letter 'a'.
 * - s contains at most b occurrences of the letter 'b'.
 * - s contains at most c occurrences of the letter 'c'.
 *
 * Given three integers a, b, and c, return the longest possible happy string. If there are
 * multiple longest happy strings, return any of them. If there is no such string, return
 * the empty string "".
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {number} a
 * @param {number} b
 * @param {number} c
 * @return {string}
 */
var longestDiverseString = function(a, b, c) {
  const counts = [
    { char: 'a', count: a },
    { char: 'b', count: b },
    { char: 'c', count: c }
  ];
  const result = [];

  while (true) {
    counts.sort((x, y) => y.count - x.count);

    let added = false;
    for (let i = 0; i < 3; i++) {
      const { char, count } = counts[i];
      if (count === 0) continue;

      const total = result.length;
      if (total >= 2 && result[total - 1] === char && result[total - 2] === char) {
        continue;
      }

      result.push(char);
      counts[i].count--;
      added = true;
      break;
    }

    if (!added) break;
  }

  return result.join('');
};
```

1406 Stone Game Iii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1406. Stone Game III
 * https://leetcode.com/problems/stone-game-iii/
 * Difficulty: Hard
 *
 * Alice and Bob continue their games with piles of stones. There are several stones arranged
 * in a row, and each stone has an associated value which is an integer given in the array
 * stoneValue.
 *
 * Alice and Bob take turns, with Alice starting first. On each player's turn, that player can
 * take 1, 2, or 3 stones from the first remaining stones in the row.
 *
 * The score of each player is the sum of the values of the stones taken. The score of each
 * player is 0 initially.
 *
 * The objective of the game is to end with the highest score, and the winner is the player
 * with the highest score and there could be a tie. The game continues until all the stones
 * have been taken.
 *
 * Assume Alice and Bob play optimally.
 *
 * Return "Alice" if Alice will win, "Bob" if Bob will win, or "Tie" if they will end the game
 * with the same score.
 */

/**
 * @param {number[]} stoneValue
 * @return {string}
 */
var stoneGameIII = function(stoneValue) {
  const n = stoneValue.length;
  const cache = new Array(n + 1).fill(null);

  function findOptimalScore(index) {
    if (index >= n) return 0;
    if (cache[index] !== null) return cache[index];

    let maxScore = -Infinity;
    let currentSum = 0;

    for (let stones = 1; stones <= 3 && index + stones - 1 < n; stones++) {
      currentSum += stoneValue[index + stones - 1];
      const opponentScore = findOptimalScore(index + stones);
      maxScore = Math.max(maxScore, currentSum + (index + stones < n ? -opponentScore : 0));
    }

    cache[index] = maxScore;
    return maxScore;
  }

  const aliceScore = findOptimalScore(0);
  if (aliceScore > 0) return 'Alice';
  if (aliceScore < 0) return 'Bob';
  return 'Tie';
};
```

1408 String Matching In An Array

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
```

```
 * 1408. String Matching in an Array
 * https://leetcode.com/problems/string-matching-in-an-array/
 * Difficulty: Easy
 *
 * Given an array of string words. Return all strings in words which is
 * substring of another word in any order.
 *
 * String words[i] is substring of words[j], if can be obtained removing
 * some characters to left and/or right side of words[j].
 */

/**
 * @param {string[]} words
 * @return {string[]}
 */
var stringMatching = function(words) {
  return words.filter(s => words.some(word => word !== s && word.includes(s)));
};
```

1409 Queries On A Permutation With Key

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1409. Queries on a Permutation With Key
 * https://leetcode.com/problems/queries-on-a-permutation-with-key/
 * Difficulty: Medium
 *
 * Given the array queries of positive integers between 1 and m, you have to process all
 * queries[i] (from i=0 to i=queries.length-1) according to the following rules:
 * - In the beginning, you have the permutation P=[1,2,3,...,m].
 * - For the current i, find the position of queries[i] in the permutation P (indexing from 0)
 *   and then move this at the beginning of the permutation P. Notice that the position of
 *   queries[i] in P is the result for queries[i].
 *
 * Return an array containing the result for the given queries.
 */

/**
 * @param {number[]} queries
 * @param {number} m
 * @return {number[]}
 */
var processQueries = function(queries, m) {
  const permutation = Array.from({ length: m }, (_, i) => i + 1);
  const result = [];

  for (const query of queries) {
    const position = permutation.indexOf(query);
    result.push(position);
    permutation.splice(position, 1);
    permutation.unshift(query);
  }

  return result;
};
```

1410 Html Entity Parser

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1410. HTML Entity Parser
 * https://leetcode.com/problems/html-entity-parser/
 * Difficulty: Medium
```

```
 *
 * HTML entity parser is the parser that takes HTML code as input and replace all the
 * entities of the special characters by the characters itself.
 *
 * The special characters and their entities for HTML are:
 * - Quotation Mark: the entity is &quot; and symbol character is ".
 * - Single Quote Mark: the entity is &apos; and symbol character is '.
 * - Ampersand: the entity is &amp; and symbol character is &.
 * - Greater Than Sign: the entity is &gt; and symbol character is >.
 * - Less Than Sign: the entity is &lt; and symbol character is <.
 * - Slash: the entity is &frasl; and symbol character is /.
 *
 * Given the input text string to the HTML parser, you have to implement the entity parser.
 *
 * Return the text after replacing the entities by the special characters.
 */

/**
 * @param {string} text
 * @return {string}
 */
var entityParser = function(text) {
  const map = {
    '&quot;': '"',
    '&apos;': '\'',
    '&amp;': '&',
    '&gt;': '>',
    '&lt;': '<',
    '&frasl;': '/'
  };

  return text.replace(new RegExp(Object.keys(map).join('|'), 'g'), m => map[m]);
};
```

1411 Number Of Ways To Paint N 3 Grid

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1411. Number of Ways to Paint N × 3 Grid
 * https://leetcode.com/problems/number-of-ways-to-paint-n-3-grid/
 * Difficulty: Hard
 *
 * You have a grid of size n x 3 and you want to paint each cell of the grid with exactly one
of
 * the three colors: Red, Yellow, or Green while making sure that no two adjacent cells have
the
 * same color (i.e., no two cells that share vertical or horizontal sides have the same
color).
 *
 * Given n the number of rows of the grid, return the number of ways you can paint this grid.
 * As the answer may grow large, the answer must be computed modulo 109 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
var numOfWays = function(n) {
  const MOD = 1000000007n;
  let pattern121 = 6n;
  let pattern123 = 6n;

  for (let row = 2; row <= n; row++) {
    const nextPattern121 = (3n * pattern121 + 2n * pattern123) % MOD;
    const nextPattern123 = (2n * pattern121 + 2n * pattern123) % MOD;
    pattern121 = nextPattern121;
    pattern123 = nextPattern123;
  }
```

```
  return Number((pattern121 + pattern123) % MOD);
};
```

1413 Minimum Value To Get Positive Step By Step Sum

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1413. Minimum Value to Get Positive Step by Step Sum
 * https://leetcode.com/problems/minimum-value-to-get-positive-step-by-step-sum/
 * Difficulty: Easy
 *
 * Given an array of integers nums, you start with an initial positive value startValue.
 *
 * In each iteration, you calculate the step by step sum of startValue plus elements in
 * nums (from left to right).
 *
 * Return the minimum positive value of startValue such that the step by step sum is never
 * less than 1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minStartValue = function(nums) {
  let minSum = 0;
  let currentSum = 0;

  for (const num of nums) {
    currentSum += num;
    minSum = Math.min(minSum, currentSum);
  }

  return Math.max(1, 1 - minSum);
};
```

1414 Find The Minimum Number Of Fibonacci Numbers Whose Sum Is K

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1414. Find the Minimum Number of Fibonacci Numbers Whose Sum Is K
 * https://leetcode.com/problems/find-the-minimum-number-of-fibonacci-numbers-whose-sum-is-k/
 * Difficulty: Medium
 *
 * Given an integer k, return the minimum number of Fibonacci numbers whose sum is equal to k.
 * The same Fibonacci number can be used multiple times.
 *
 * The Fibonacci numbers are defined as:
 * - F1 = 1
 * - F2 = 1
 * - Fn = Fn-1 + Fn-2 for n > 2.
 *
 * It is guaranteed that for the given constraints we can always find such Fibonacci numbers
 * that sum up to k.
 */

/**
 * @param {number} k
 * @return {number}
 */
var findMinFibonacciNumbers = function(k) {
  const fibonacci = [1, 1];
  while (fibonacci[fibonacci.length - 1] <= k) {
```

```
    fibonacci.push(fibonacci[fibonacci.length - 1] + fibonacci[fibonacci.length - 2]);
  }

  let count = 0;
  let remaining = k;

  for (let i = fibonacci.length - 1; i >= 0 && remaining > 0; i--) {
    while (fibonacci[i] <= remaining) {
      remaining -= fibonacci[i];
      count++;
    }
  }

  return count;
};
```

1415 The K Th Lexicographical String Of All Happy Strings Of Length N

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1415. The k-th Lexicographical String of All Happy Strings of Length n
 * https://leetcode.com/problems/the-k-th-lexicographical-string-of-all-happy-strings-of-lengt
h-n/
 * Difficulty: Medium
 *
 * A happy string is a string that:
 * - consists only of letters of the set ['a', 'b', 'c'].
 * - s[i] != s[i + 1] for all values of i from 1 to s.length - 1 (string is 1-indexed).
 *
 * For example, strings "abc", "ac", "b" and "abcbabcbcb" are all happy strings and strings
"aa",
 * "baa" and "ababbc" are not happy strings.
 *
 * Given two integers n and k, consider a list of all happy strings of length n sorted in
 * lexicographical order.
 *
 * Return the kth string of this list or return an empty string if there are less than k happy
 * strings of length n.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {string}
 */
var getHappyString = function(n, k) {
  return backtrack('') || '';

  function backtrack(str) {
    if (str.length === n) {
      return --k ? false : str;
    }
    for (const character of 'abc') {
      if (character === str[str.length - 1]) {
        continue;
      }
      const value = backtrack(str + character);
      if (value) {
        return value;
      }
    }

    return false;
  }
};
```

1416 Restore The Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1416. Restore The Array
 * https://leetcode.com/problems/restore-the-array/
 * Difficulty: Hard
 *
 * A program was supposed to print an array of integers. The program forgot to print
whitespaces
 * and the array is printed as a string of digits s and all we know is that all integers in
the
 * array were in the range [1, k] and there are no leading zeros in the array.
 *
 * Given the string s and the integer k, return the number of the possible arrays that can be
 * printed as s using the mentioned program. Since the answer may be very large, return it
modulo
 * 109 + 7.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
function numberOfArrays(s, k) {
  const MOD = 1e9 + 7;
  const dp = new Array(s.length + 1).fill(0);
  dp[0] = 1;

  for (let i = 1; i <= s.length; i++) {
    let num = 0;
    for (let j = i - 1; j >= Math.max(0, i - String(k).length); j--) {
      if (j < 0) break;
      num = Number(s[j]) * Math.pow(10, i - j - 1) + num;
      if (num <= k && (s[j] !== '0')) {
        dp[i] = (dp[i] + dp[j]) % MOD;
      }
    }
  }

  return dp[s.length];
}
```

1417 Reformat The String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1417. Reformat The String
 * https://leetcode.com/problems/reformat-the-string/
 * Difficulty: Easy
 *
 * You are given an alphanumeric string s. (Alphanumeric string is a string consisting of
lowercase
 * English letters and digits).
 *
 * You have to find a permutation of the string where no letter is followed by another letter
and
 * no digit is followed by another digit. That is, no two adjacent characters have the same
type.
 *
 * Return the reformatted string or return an empty string if it is impossible to reformat the
 * string.
```

```
 */

/**
 * @param {string} s
 * @return {string}
 */
var reformat = function(s) {
  const letters = [];
  const digits = [];
  for (const char of s) {
    if (isNaN(char)) {
      letters.push(char);
    } else {
      digits.push(char);
    }
  }
  if (Math.abs(letters.length - digits.length) > 1) {
    return '';
  }
  const result = [];
  const longer = letters.length >= digits.length ? letters : digits;
  const shorter = longer === letters ? digits : letters;
  for (let i = 0; i < longer.length; i++) {
    result.push(longer[i]);
    if (i < shorter.length) {
      result.push(shorter[i]);
    }
  }
  return result.join('');
};
```

1418 Display Table Of Food Orders In A Restaurant

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1418. Display Table of Food Orders in a Restaurant
 * https://leetcode.com/problems/display-table-of-food-orders-in-a-restaurant/
 * Difficulty: Medium
 *
 * Given the array orders, which represents the orders that customers have done in a
restaurant.
 * More specifically orders[i]=[customerNamei,tableNumberi,foodItemi] where customerNamei is
the
 * name of the customer, tableNumberi is the table customer sit at, and foodItemi is the item
 * customer orders.
 *
 * Return the restaurant's “display table”. The “display table” is a table whose row entries
 * denote how many of each food item each table ordered. The first column is the table number
 * and the remaining columns correspond to each food item in alphabetical order. The first row
 * should be a header whose first column is “Table”, followed by the names of the food items.
 * Note that the customer names are not part of the table. Additionally, the rows should be
 * sorted in numerically increasing order.
 */

/**
 * @param {string[][]} orders
 * @return {string[][]}
 */
var displayTable = function(orders) {
  const foodItems = new Set();
  const tableOrders = new Map();

  for (const [, table, item] of orders) {
    foodItems.add(item);
    const tableMap = tableOrders.get(table) || new Map();
    tableMap.set(item, (tableMap.get(item) || 0) + 1);
    tableOrders.set(table, tableMap);
  }
```

```
  const sortedItems = [...foodItems].sort();
  const header = ['Table', ...sortedItems];
  const result = [header];

  for (const table of [...tableOrders.keys()].sort((a, b) => a - b)) {
    const row = [table];
    const items = tableOrders.get(table);
    for (const item of sortedItems) {
      row.push(String(items.get(item) || 0));
    }
    result.push(row);
  }

  return result;
};
```

1419 Minimum Number Of Frogs Croaking

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1419. Minimum Number of Frogs Croaking
 * https://leetcode.com/problems/minimum-number-of-frogs-croaking/
 * Difficulty: Medium
 *
 * You are given the string croakOfFrogs, which represents a combination of the string "croak"
 * from different frogs, that is, multiple frogs can croak at the same time, so multiple
"croak"
 * are mixed.
 *
 * Return the minimum number of different frogs to finish all the croaks in the given string.
 *
 * A valid "croak" means a frog is printing five letters 'c', 'r', 'o', 'a', and 'k'
sequentially.
 * The frogs have to print all five letters to finish a croak. If the given string is not a
 * combination of a valid "croak" return -1.
 */

/**
 * @param {string} croakOfFrogs
 * @return {number}
 */
var minNumberOfFrogs = function(croakOfFrogs) {
  let activeFrogs = 0;
  let maxFrogs = 0;
  const counts = { c: 0, r: 0, o: 0, a: 0, k: 0 };
  const order = 'croak';

  for (const char of croakOfFrogs) {
    if (!order.includes(char)) return -1;
    counts[char]++;

    if (char === 'c') {
      activeFrogs++;
      maxFrogs = Math.max(maxFrogs, activeFrogs);
    } else if (char === 'k') {
      activeFrogs--;
    }

    for (let i = 1; i < order.length; i++) {
      if (counts[order[i]] > counts[order[i - 1]]) return -1;
    }
  }

  return activeFrogs === 0 && counts.c === counts.k ? maxFrogs : -1;
};
```

1420 Build Array Where You Can Find The Maximum Exactly K Comparisons

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1420. Build Array Where You Can Find The Maximum Exactly K Comparisons
 * https://leetcode.com/problems/build-array-where-you-can-find-the-maximum-exactly-k-comparis
ons/
 * Difficulty: Hard
 *
 * You are given three integers n, m and k. Consider the following algorithm to find the
maximum
 * element of an array of positive integers.
 *
 * You should build the array arr which has the following properties:
 * - arr has exactly n integers.
 * - 1 <= arr[i] <= m where (0 <= i < n).
 * - After applying the mentioned algorithm to arr, the value search_cost is equal to k.
 *
 * Return the number of ways to build the array arr under the mentioned conditions. As the
answer
 * may grow large, the answer must be computed modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} m
 * @param {number} k
 * @return {number}
 */
function numOfArrays(length, maxValue, searchCost) {
  const MOD = 1e9 + 7;
  const cache = Array.from({ length: length + 1 }, () =>
    Array.from({ length: maxValue + 1 }, () =>
      Array(searchCost + 1).fill(-1)
    )
  );

  function computeArrays(pos, currentMax, remainingCost) {
    if (pos === length) return remainingCost === 0 ? 1 : 0;
    if (remainingCost < 0) return 0;
    if (cache[pos][currentMax][remainingCost] !== -1) {
      return cache[pos][currentMax][remainingCost];
    }

    let total = 0;
    for (let value = 1; value <= currentMax; value++) {
      total = (total + computeArrays(pos + 1, currentMax, remainingCost)) % MOD;
    }

    if (remainingCost > 0) {
      for (let value = currentMax + 1; value <= maxValue; value++) {
        total = (total + computeArrays(pos + 1, value, remainingCost - 1)) % MOD;
      }
    }

    return cache[pos][currentMax][remainingCost] = total;
  }

  let result = 0;
  for (let value = 1; value <= maxValue; value++) {
    result = (result + computeArrays(1, value, searchCost - 1)) % MOD;
  }

  return result;
}
```

1422 Maximum Score After Splitting A String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1422. Maximum Score After Splitting a String
 * https://leetcode.com/problems/maximum-score-after-splitting-a-string/
 * Difficulty: Easy
 *
 * Given a string s of zeros and ones, return the maximum score after splitting the string
into
 * two non-empty substrings (i.e. left substring and right substring).
 *
 * The score after splitting a string is the number of zeros in the left substring plus the
number
 * of ones in the right substring.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxScore = function(s) {
  let rightOnes = s.split('').reduce((count, char) => count + (char === '1'), 0);
  let leftZeros = 0;
  let result = 0;

  for (let i = 0; i < s.length - 1; i++) {
    leftZeros += s[i] === '0';
    rightOnes -= s[i] === '1';
    result = Math.max(result, leftZeros + rightOnes);
  }

  return result;
};
```

1423 Maximum Points You Can Obtain From Cards

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1423. Maximum Points You Can Obtain from Cards
 * https://leetcode.com/problems/maximum-points-you-can-obtain-from-cards/
 * Difficulty: Medium
 *
 * There are several cards arranged in a row, and each card has an associated number of
 * points. The points are given in the integer array cardPoints.
 *
 * In one step, you can take one card from the beginning or from the end of the row.
 * You have to take exactly k cards.
 *
 * Your score is the sum of the points of the cards you have taken.
 *
 * Given the integer array cardPoints and the integer k, return the maximum score you can
obtain.
 */

/**
 * @param {number[]} cardPoints
 * @param {number} k
 * @return {number}
 */
var maxScore = function(cardPoints, k) {
  const totalLength = cardPoints.length;
  const windowSize = totalLength - k;
```

```
  let windowSum = cardPoints.slice(0, windowSize).reduce((sum, num) => sum + num, 0);
  let minWindowSum = windowSum;
  const totalSum = cardPoints.reduce((sum, num) => sum + num, 0);

  for (let i = windowSize; i < totalLength; i++) {
    windowSum = windowSum + cardPoints[i] - cardPoints[i - windowSize];
    minWindowSum = Math.min(minWindowSum, windowSum);
  }

  return totalSum - minWindowSum;
};
```

1424 Diagonal Traverse Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1424. Diagonal Traverse II
 * https://leetcode.com/problems/diagonal-traverse-ii/
 * Difficulty: Medium
 *
 * Given a 2D integer array nums, return all elements of nums in diagonal order as shown
 * in the below images.
 */

/**
 * @param {number[][]} nums
 * @return {number[]}
 */
var findDiagonalOrder = function(nums) {
  const diagonalGroups = [];

  for (let row = 0; row < nums.length; row++) {
    for (let col = 0; col < nums[row].length; col++) {
      const sum = row + col;
      if (!diagonalGroups[sum]) {
        diagonalGroups[sum] = [];
      }
      diagonalGroups[sum].push(nums[row][col]);
    }
  }

  const result = [];
  for (const group of diagonalGroups) {
    if (group) {
      result.push(...group.reverse());
    }
  }

  return result;
};
```

1425 Constrained Subsequence Sum

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1425. Constrained Subsequence Sum
 * https://leetcode.com/problems/constrained-subsequence-sum/
 * Difficulty: Hard
 *
 * Given an integer array nums and an integer k, return the maximum sum of a non-empty
subsequence
 * of that array such that for every two consecutive integers in the subsequence, nums[i] and
 * nums[j], where i < j, the condition j - i <= k is satisfied.
```

```
 *
 * A subsequence of an array is obtained by deleting some number of elements (can be zero)
from the
 * array, leaving the remaining elements in their original order.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var constrainedSubsetSum = function(nums, k) {
  const maxSums = [...nums];
  const deque = [];
  let result = nums[0];

  for (let i = 0; i < nums.length; i++) {
    while (deque.length && deque[0] < i - k) {
      deque.shift();
    }

    if (deque.length) {
      maxSums[i] = Math.max(maxSums[i], maxSums[deque[0]] + nums[i]);
    }

    while (deque.length && maxSums[deque[deque.length - 1]] <= maxSums[i]) {
      deque.pop();
    }

    deque.push(i);
    result = Math.max(result, maxSums[i]);
  }

  return result;
};
```

1426 Counting Elements

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1426. Counting Elements
 * https://leetcode.com/problems/counting-elements/
 * Difficulty: Easy
 *
 * Given an integer array arr, count how many elements x there are, such that x + 1 is also
 * in arr. If there are duplicates in arr, count them separately.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var countElements = function(arr) {
  const set = new Set(arr);
  return arr.filter(num => set.has(num + 1)).length;
};
```

1427 Perform String Shifts

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1427. Perform String Shifts
 * https://leetcode.com/problems/perform-string-shifts/
```

```
 * Difficulty: Easy
 *
 * You are given a string s containing lowercase English letters, and a matrix shift,
 * where shift[i] = [directioni, amounti]:
 * - directioni can be 0 (for left shift) or 1 (for right shift).
 * - amounti is the amount by which string s is to be shifted.
 * - A left shift by 1 means remove the first character of s and append it to the end.
 * - Similarly, a right shift by 1 means remove the last character of s and add it to the
beginning.
 *
 * Return the final string after all operations.
 */

/**
 * @param {string} s
 * @param {number[][]} shift
 * @return {string}
 */
var stringShift = function(s, shift) {
  let netShift = 0;

  for (const [direction, amount] of shift) {
    netShift += direction === 0 ? -amount : amount;
  }

  const n = s.length;
  netShift = ((netShift % n) + n) % n;

  return s.slice(-netShift) + s.slice(0, -netShift);
};
```

1428 Leftmost Column With At Least A One

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1428. Leftmost Column with at Least a One
 * https://leetcode.com/problems/leftmost-column-with-at-least-a-one/
 * Difficulty: Medium
 *
 * A row-sorted binary matrix means that all elements are 0 or 1 and each row of the matrix
 * is sorted in non-decreasing order.
 *
 * Given a row-sorted binary matrix binaryMatrix, return the index (0-indexed) of the leftmost
 * column with a 1 in it. If such an index does not exist, return -1.
 *
 * You can't access the Binary Matrix directly. You may only access the matrix using a
 * BinaryMatrix interface:
 * - BinaryMatrix.get(row, col) returns the element of the matrix at index (row, col)
(0-indexed).
 * - BinaryMatrix.dimensions() returns the dimensions of the matrix as a list of 2
 *   elements [rows, cols], which means the matrix is rows x cols.
 *
 * Submissions making more than 1000 calls to BinaryMatrix.get will be judged Wrong Answer.
 * Also, any solutions that attempt to circumvent the judge will result in disqualification.
 *
 * For custom testing purposes, the input will be the entire binary matrix mat. You will not
 * have access to the binary matrix directly.
 */

/**
 * @param {BinaryMatrix} binaryMatrix
 * @return {number}
 */
var leftMostColumnWithOne = function(binaryMatrix) {
  const [rows, cols] = binaryMatrix.dimensions();
  let result = -1;
  let row = 0;
  let col = cols - 1;
```

```
  while (row < rows && col >= 0) {
    if (binaryMatrix.get(row, col) === 1) {
      result = col;
      col--;
    } else {
      row++;
    }
  }

  return result;
};
```

1429 First Unique Number

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1429. First Unique Number
 * https://leetcode.com/problems/first-unique-number/
 * Difficulty: Medium
 *
 * You have a queue of integers, you need to retrieve the first unique integer in the queue.
 *
 * Implement the FirstUnique class:
 * - FirstUnique(int[] nums) Initializes the object with the numbers in the queue.
 * - int showFirstUnique() returns the value of the first unique integer of the queue, and
 *   returns -1 if there is no such integer.
 * - void add(int value) insert value to the queue.
 */

/**
* @param {number[]} nums
*/
var FirstUnique = function(nums) {
  this.queue = [];
  this.counts = new Map();
  this.uniqueSet = new Set();

  for (const num of nums) {
    this.add(num);
  }
};

/**
* @return {number}
*/
FirstUnique.prototype.showFirstUnique = function() {
  while (this.queue.length > 0 && !this.uniqueSet.has(this.queue[0])) {
    this.queue.shift();
  }
  return this.queue.length > 0 ? this.queue[0] : -1;
};

/**
* @param {number} value
* @return {void}
*/
FirstUnique.prototype.add = function(value) {
  const count = this.counts.get(value) || 0;
  this.counts.set(value, count + 1);

  if (count === 0) {
    this.queue.push(value);
    this.uniqueSet.add(value);
  } else if (count === 1) {
    this.uniqueSet.delete(value);
  }
};
```

1430 Check If A String Is A Valid Sequence From Root To Leaves Path In

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1430. Check If a String Is a Valid Sequence from Root to Leaves Path in a Binary Tree
 * https://leetcode.com/problems/check-if-a-string-is-a-valid-sequence-from-root-to-leaves-pat
h-in-a-binary-tree/
 * Difficulty: Medium
 *
 * Given a binary tree where each path going from the root to any leaf form a valid
 * sequence, check if a given string is a valid sequence in such binary tree.
 *
 * We get the given string from the concatenation of an array of integers arr and the
 * concatenation of all values of the nodes along a path results in a sequence in the
 * given binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number[]} arr
 * @return {boolean}
 */
var isValidSequence = function(root, arr) {
  return dfs(root, 0);

  function dfs(node, index) {
    if (!node || index >= arr.length || node.val !== arr[index]) {
      return false;
    }

    if (index === arr.length - 1) {
      return !node.left && !node.right;
    }

    return dfs(node.left, index + 1) || dfs(node.right, index + 1);
  }
};
```

1431 Kids With The Greatest Number Of Candies

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1431. Kids With the Greatest Number of Candies
 * https://leetcode.com/problems/kids-with-the-greatest-number-of-candies/
 * Difficulty: Easy
 *
 * There are n kids with candies. You are given an integer array candies, where each
 * candies[i] represents the number of candies the ith kid has, and an integer
 * extraCandies, denoting the number of extra candies that you have.
 *
 * Return a boolean array result of length n, where result[i] is true if, after giving
 * the ith kid all the extraCandies, they will have the greatest number of candies
 * among all the kids, or false otherwise.
 *
 * Note that multiple kids can have the greatest number of candies.
```

```
 */

/**
 * @param {number[]} candies
 * @param {number} extraCandies
 * @return {boolean[]}
 */
var kidsWithCandies = function(candies, extraCandies) {
  const max = Math.max(...candies);
  return candies.map(count => count + extraCandies >= max);
};
```

1432 Max Difference You Can Get From Changing An Integer

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1432. Max Difference You Can Get From Changing an Integer
 * https://leetcode.com/problems/max-difference-you-can-get-from-changing-an-integer/
 * Difficulty: Medium
 *
 * You are given an integer num. You will apply the following steps exactly two times:
 * - Pick a digit x (0 <= x <= 9).
 * - Pick another digit y (0 <= y <= 9). The digit y can be equal to x.
 * - Replace all the occurrences of x in the decimal representation of num by y.
 * - The new integer cannot have any leading zeros, also the new integer cannot be 0.
 *
 * Let a and b be the results of applying the operations to num the first and second times,
 * respectively.
 *
 * Return the max difference between a and b.
 */

/**
 * @param {number} num
 * @return {number}
 */
var maxDiff = function(num) {
  const digits = num.toString().split('');

  const maxNum = digits.slice();
  for (let i = 0; i < maxNum.length; i++) {
    if (maxNum[i] !== '9') {
      const target = maxNum[i];
      for (let j = i; j < maxNum.length; j++) {
        if (maxNum[j] === target) {
          maxNum[j] = '9';
        }
      }
      break;
    }
  }

  const minNum = digits.slice();
  if (minNum[0] !== '1') {
    const target = minNum[0];
    for (let j = 0; j < minNum.length; j++) {
      if (minNum[j] === target) {
        minNum[j] = '1';
      }
    }
  } else {
    for (let i = 1; i < minNum.length; i++) {
      if (minNum[i] !== '0' && minNum[i] !== '1') {
        const target = minNum[i];
        for (let j = i; j < minNum.length; j++) {
          if (minNum[j] === target) {
            minNum[j] = '0';
          }
        }
```

```
        break;
      }
    }
  }

  return parseInt(maxNum.join('')) - parseInt(minNum.join(''));
};
```

1433 Check If A String Can Break Another String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1433. Check If a String Can Break Another String
 * https://leetcode.com/problems/check-if-a-string-can-break-another-string/
 * Difficulty: Medium
 *
 * Given two strings: s1 and s2 with the same size, check if some permutation of string s1 can
 * break some permutation of string s2 or vice-versa. In other words s2 can break s1 or
vice-versa.
 *
 * A string x can break string y (both of size n) if x[i] >= y[i] (in alphabetical order) for
all
 * i between 0 and n-1.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var checkIfCanBreak = function(s1, s2) {
  const sorted1 = s1.split('').sort();
  const sorted2 = s2.split('').sort();

  let canBreak1 = true;
  let canBreak2 = true;

  for (let i = 0; i < s1.length; i++) {
    if (sorted1[i] < sorted2[i]) {
      canBreak1 = false;
    }
    if (sorted2[i] < sorted1[i]) {
      canBreak2 = false;
    }
  }

  return canBreak1 || canBreak2;
};
```

1434 Number Of Ways To Wear Different Hats To Each Other

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1434. Number of Ways to Wear Different Hats to Each Other
 * https://leetcode.com/problems/number-of-ways-to-wear-different-hats-to-each-other/
 * Difficulty: Hard
 *
 * There are n people and 40 types of hats labeled from 1 to 40.
 *
 * Given a 2D integer array hats, where hats[i] is a list of all hats preferred by the ith
person.
 *
 * Return the number of ways that n people can wear different hats from each other.
```

```
 *
 * Since the answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {number[][]} hats
 * @return {number}
 */
var numberWays = function(hats) {
  const MOD = 1e9 + 7;
  const hatToPeople = Array(41).fill().map(() => []);
  const cache = new Map();

  for (let person = 0; person < hats.length; person++) {
    for (const hat of hats[person]) {
      hatToPeople[hat].push(person);
    }
  }

  function assignHats(hat, usedMask) {
    if (hat > 40) {
      return usedMask === (1 << hats.length) - 1 ? 1 : 0;
    }

    const key = `${hat}:${usedMask}`;
    if (cache.has(key)) {
      return cache.get(key);
    }

    let ways = assignHats(hat + 1, usedMask);

    for (const person of hatToPeople[hat]) {
      if (!(usedMask & (1 << person))) {
        ways = (ways + assignHats(hat + 1, usedMask | (1 << person))) % MOD;
      }
    }

    cache.set(key, ways);
    return ways;
  }

  return assignHats(1, 0);
};
```

1436 Destination City

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1436. Destination City
 * https://leetcode.com/problems/destination-city/
 * Difficulty: Easy
 *
 * You are given the array paths, where paths[i] = [cityAi, cityBi] means there
 * exists a direct path going from cityAi to cityBi. Return the destination city,
 * that is, the city without any path outgoing to another city.
 *
 * It is guaranteed that the graph of paths forms a line without any loop, therefore,
 * there will be exactly one destination city.
 */

/**
 * @param {string[][]} paths
 * @return {string}
 */
var destCity = function(paths) {
  const set = new Set(paths.map(([a]) => a));
  return paths.find(([_, b]) => !set.has(b))[1];
};
```

1437 Check If All 1S Are At Least Length K Places Away

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1437. Check If All 1's Are at Least Length K Places Away
 * https://leetcode.com/problems/check-if-all-1s-are-at-least-length-k-places-away/
 * Difficulty: Easy
 *
 * Given an array nums of 0s and 1s and an integer k, return True if all 1's are at
 * least k places away from each other, otherwise return False.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
var kLengthApart = function(nums, k) {
  let offset = k;
  for (const num of nums) {
    if (num === 0) { offset++; continue; }
    if (offset < k) return false;
    offset = 0;
  }
  return true;
};
```

1438 Longest Continuous Subarray With Absolute Diff Less Than Or Equal

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1438. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit
 * https://leetcode.com/problems/longest-continuous-subarray-with-absolute-diff-less-than-or-e
qual-to-limit/
 * Difficulty: Medium
 *
 * Given an array of integers nums and an integer limit, return the size of the longest
non-empty
 * subarray such that the absolute difference between any two elements of this subarray is
less than
 * or equal to limit.
 */

/**
 * @param {number[]} nums
 * @param {number} limit
 * @return {number}
 */
var longestSubarray = function(nums, limit) {
  const maxDeque = [];
  const minDeque = [];
  let result = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    while (maxDeque.length && nums[maxDeque[maxDeque.length - 1]] <= nums[right]) {
      maxDeque.pop();
    }
    maxDeque.push(right);

    while (minDeque.length && nums[minDeque[minDeque.length - 1]] >= nums[right]) {
      minDeque.pop();
    }
```

```
    minDeque.push(right);

    while (nums[maxDeque[0]] - nums[minDeque[0]] > limit) {
      if (maxDeque[0] < minDeque[0]) {
        left = maxDeque.shift() + 1;
      } else {
        left = minDeque.shift() + 1;
      }
    }

    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

1439 Find The Kth Smallest Sum Of A Matrix With Sorted Rows

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1439. Find the Kth Smallest Sum of a Matrix With Sorted Rows
 * https://leetcode.com/problems/find-the-kth-smallest-sum-of-a-matrix-with-sorted-rows/
 * Difficulty: Hard
 *
 * You are given an m x n matrix mat that has its rows sorted in non-decreasing order and
 * an integer k.
 *
 * You are allowed to choose exactly one element from each row to form an array.
 *
 * Return the kth smallest array sum among all possible arrays.
 */

/**
 * @param {number[][]} mat
 * @param {number} k
 * @return {number}
 */
var kthSmallest = function(mat, k) {
  let sums = [0];
  const rows = mat.length;

  for (let row = 0; row < rows; row++) {
    const nextSums = [];
    for (const prevSum of sums) {
      for (const num of mat[row]) {
        nextSums.push(prevSum + num);
      }
    }
    sums = nextSums.sort((a, b) => a - b).slice(0, Math.min(k, nextSums.length));
  }

  return sums[k - 1];
};
```

1441 Build An Array With Stack Operations

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1441. Build an Array With Stack Operations
 * https://leetcode.com/problems/build-an-array-with-stack-operations/
 * Difficulty: Medium
 *
 * You are given an integer array target and an integer n.
```

```
 *
 * You have an empty stack with the two following operations:
 * - "Push": pushes an integer to the top of the stack.
 * - "Pop": removes the integer on the top of the stack.
 *
 * You also have a stream of the integers in the range [1, n].
 *
 * Use the two stack operations to make the numbers in the stack (from the bottom to the top)
equal
 * to target. You should follow the following rules:
 * - If the stream of the integers is not empty, pick the next integer from the stream and
push it
 *   to the top of the stack.
 * - If the stack is not empty, pop the integer at the top of the stack.
 * - If, at any moment, the elements in the stack (from the bottom to the top) are equal to
target,
 *   do not read new integers from the stream and do not do more operations on the stack.
 *
 * Return the stack operations needed to build target following the mentioned rules. If there
are
 * multiple valid answers, return any of them.
 */

/**
 * @param {number[]} target
 * @param {number} n
 * @return {string[]}
 */
var buildArray = function(target, n) {
  const result = [];
  let current = 1;

  for (const num of target) {
    while (current < num) {
      result.push('Push');
      result.push('Pop');
      current++;
    }
    result.push('Push');
    current++;
  }

  return result;
};
```

1442 Count Triplets That Can Form Two Arrays Of Equal Xor

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1442. Count Triplets That Can Form Two Arrays of Equal XOR
 * https://leetcode.com/problems/count-triplets-that-can-form-two-arrays-of-equal-xor/
 * Difficulty: Medium
 *
 * Given an array of integers arr.
 *
 * We want to select three indices i, j and k where (0 <= i < j <= k < arr.length).
 *
 * Let's define a and b as follows:
 * - a = arr[i] ^ arr[i + 1] ^ ... ^ arr[j - 1]
 * - b = arr[j] ^ arr[j + 1] ^ ... ^ arr[k]
 *
 * Note that ^ denotes the bitwise-xor operation.
 *
 * Return the number of triplets (i, j and k) Where a == b.
 */

/**
 * @param {number[]} arr
 * @return {number}
```

```
 */
var countTriplets = function(arr) {
  const prefixXor = [0];
  let result = 0;

  for (const num of arr) {
    prefixXor.push(prefixXor[prefixXor.length - 1] ^ num);
  }

  for (let i = 0; i < arr.length; i++) {
    for (let k = i + 1; k < arr.length; k++) {
      if ((prefixXor[k + 1] ^ prefixXor[i]) === 0) {
        result += k - i;
      }
    }
  }

  return result;
};
```

1443 Minimum Time To Collect All Apples In A Tree

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1443. Minimum Time to Collect All Apples in a Tree
 * https://leetcode.com/problems/minimum-time-to-collect-all-apples-in-a-tree/
 * Difficulty: Medium
 *
 * Given an undirected tree consisting of n vertices numbered from 0 to n-1, which has some
apples
 * in their vertices. You spend 1 second to walk over one edge of the tree. Return the minimum
 time
 * in seconds you have to spend to collect all apples in the tree, starting at vertex 0 and
coming
 * back to this vertex.
 *
 * The edges of the undirected tree are given in the array edges, where edges[i] = [ai, bi]
means
 * that exists an edge connecting the vertices ai and bi. Additionally, there is a boolean
array
 * hasApple, where hasApple[i] = true means that vertex i has an apple; otherwise, it does not
 * have any apple.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {boolean[]} hasApple
 * @return {number}
 */
var minTime = function(n, edges, hasApple) {
  const map = new Map();
  const seen = new Set();
  let time = 0;

  edges.forEach(([value, key]) => map.set(key, value));
  for (let i = n - 1; i >= 0; i--) {
    if (hasApple[i]) {
      dfs(i);
    }
  }

  function dfs(key) {
    if (key === 0 || seen.has(key)) return;
    seen.add(key);
    time += 2;
    dfs(map.get(key));
  }
```

```
  return time;
};
```

1444 Number Of Ways Of Cutting A Pizza

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1444. Number of Ways of Cutting a Pizza
 * https://leetcode.com/problems/number-of-ways-of-cutting-a-pizza/
 * Difficulty: Hard
 *
 * Given a rectangular pizza represented as a rows x cols matrix containing the following
 * characters: 'A' (an apple) and '.' (empty cell) and given the integer k. You have to cut
 * the pizza into k pieces using k-1 cuts.
 *
 * For each cut you choose the direction: vertical or horizontal, then you choose a cut
 * position at the cell boundary and cut the pizza into two pieces. If you cut the pizza
 * vertically, give the left part of the pizza to a person. If you cut the pizza horizontally,
 * give the upper part of the pizza to a person. Give the last piece of pizza to the last
person.
 *
 * Return the number of ways of cutting the pizza such that each piece contains at least one
apple.
 * Since the answer can be a huge number, return this modulo 10^9 + 7.
 */

/**
 * @param {string[]} pizza
 * @param {number} k
 * @return {number}
 */
var ways = function(pizza, k) {
  const MOD = 1e9 + 7;
  const rows = pizza.length;
  const cols = pizza[0].length;
  const appleCount = new Array(rows + 1).fill().map(() => new Array(cols + 1).fill(0));
  const dpCache = new Array(rows + 1).fill().map(() =>
    new Array(cols + 1).fill().map(() => new Array(k + 1).fill(-1))
  );

  for (let i = rows - 1; i >= 0; i--) {
    for (let j = cols - 1; j >= 0; j--) {
      appleCount[i][j] = (pizza[i][j] === 'A' ? 1 : 0)
        + appleCount[i + 1][j] + appleCount[i][j + 1] - appleCount[i + 1][j + 1];
    }
  }

  return countWays(0, 0, k - 1);

  function countWays(row, col, cutsLeft) {
    if (appleCount[row][col] === 0) return 0;
    if (cutsLeft === 0) return 1;
    if (dpCache[row][col][cutsLeft] !== -1) return dpCache[row][col][cutsLeft];

    let result = 0;

    for (let i = row + 1; i < rows; i++) {
      if (appleCount[row][col] - appleCount[i][col] > 0) {
        result = (result + countWays(i, col, cutsLeft - 1)) % MOD;
      }
    }

    for (let j = col + 1; j < cols; j++) {
      if (appleCount[row][col] - appleCount[row][j] > 0) {
        result = (result + countWays(row, j, cutsLeft - 1)) % MOD;
      }
    }

    dpCache[row][col][cutsLeft] = result;
```

```
    return result;
  }
};
```

1446 Consecutive Characters

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1446. Consecutive Characters
 * https://leetcode.com/problems/consecutive-characters/
 * Difficulty: Easy
 *
 * The power of the string is the maximum length of a non-empty substring
 * that contains only one unique character.
 *
 * Given a string s, return the power of s.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxPower = function(s) {
  let max = 1, count = 0;
  for (let i = 0; i < s.length; i++) {
    const prev = s[i - 1];
    count = prev === s[i] ? count + 1 : 0;
    max = Math.max(max, count + 1);
  }
  return max;
};
```

1447 Simplified Fractions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1447. Simplified Fractions
 * https://leetcode.com/problems/simplified-fractions/
 * Difficulty: Medium
 *
 * Given an integer n, return a list of all simplified fractions between 0 and 1
 * (exclusive) such that the denominator is less-than-or-equal-to n. The
 * fractions can be in any order.
 */

/**
 * @param {number} n
 * @return {string[]}
 */
var simplifiedFractions = function(n) {
  const gcd = (a, b) => !b ? a : gcd(b, a % b);
  const result = [];

  for (let i = 2; i <= n; i++) {
    for (let j = 1; j < i; j++) {
      if (gcd(i, j) === 1) {
        result.push(`${j}/${i}`);
      }
    }
  }

  return result;
};
```

1448 Count Good Nodes In Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1448. Count Good Nodes in Binary Tree
 * https://leetcode.com/problems/count-good-nodes-in-binary-tree/
 * Difficulty: Medium
 *
 * Given a binary tree root, a node X in the tree is named good if in the path from
 * root to X there are no nodes with a value greater than X.
 *
 * Return the number of good nodes in the binary tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var goodNodes = function(root, max = -Infinity) {
  if (!root) return 0;
  const n = root.val >= max ? 1 : 0;
  return n + goodNodes(root.left, n ? root.val : max) + goodNodes(root.right, n ? root.val :
max);
};
```

1449 Form Largest Integer With Digits That Add Up To Target

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1449. Form Largest Integer With Digits That Add up to Target
 * https://leetcode.com/problems/form-largest-integer-with-digits-that-add-up-to-target/
 * Difficulty: Hard
 *
 * Given an array of integers cost and an integer target, return the maximum integer you can
 * paint under the following rules:
 * - The cost of painting a digit (i + 1) is given by cost[i] (0-indexed).
 * - The total cost used must be equal to target.
 * - The integer does not have 0 digits.
 *
 * Since the answer may be very large, return it as a string. If there is no way to paint any
 * integer given the condition, return "0".
 */

/**
 * @param {number[]} cost
 * @param {number} target
 * @return {string}
 */
var largestNumber = function(cost, target) {
  const maxDigits = new Array(target + 1).fill('0');
  maxDigits[0] = '';

  for (let current = 1; current <= target; current++) {
    for (let digit = 1; digit <= 9; digit++) {
      const prev = current - cost[digit - 1];
```

```
      if (prev >= 0 && maxDigits[prev] !== '0') {
        const candidate = digit + maxDigits[prev];
        if (candidate.length > maxDigits[current].length
            || (candidate.length === maxDigits[current].length && candidate >
maxDigits[current])) {
          maxDigits[current] = candidate;
        }
      }
    }
  }

  return maxDigits[target];
};
```

1450 Number Of Students Doing Homework At A Given Time

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1450. Number of Students Doing Homework at a Given Time
 * https://leetcode.com/problems/number-of-students-doing-homework-at-a-given-time/
 * Difficulty: Easy
 *
 * Given two integer arrays startTime and endTime and given an integer queryTime.
 *
 * The ith student started doing their homework at the time startTime[i] and finished it
 * at time endTime[i].
 *
 * Return the number of students doing their homework at time queryTime. More formally,
 * return the number of students where queryTime lays in the interval [startTime[i],
 * endTime[i]] inclusive.
 */

/**
 * @param {number[]} startTime
 * @param {number[]} endTime
 * @param {number} queryTime
 * @return {number}
 */
var busyStudent = function(startTime, endTime, queryTime) {
  return startTime.filter((s, i) => queryTime >= s && queryTime <= endTime[i]).length;
};
```

1451 Rearrange Words In A Sentence

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1451. Rearrange Words in a Sentence
 * https://leetcode.com/problems/rearrange-words-in-a-sentence/
 * Difficulty: Medium
 *
 * Given a sentence text (A sentence is a string of space-separated words) in the following
format:
 * - First letter is in upper case.
 * - Each word in text are separated by a single space.
 *
 * Your task is to rearrange the words in text such that all words are rearranged in an
increasing
 * order of their lengths. If two words have the same length, arrange them in their original
order.
 *
 * Return the new text following the format shown above.
 */
```

```
/**
 * @param {string} text
 * @return {string}
 */
var arrangeWords = function(text) {
  return text
    .split(/\s+/)
    .sort((a, b) => a.length - b.length)
    .join(' ')
    .toLowerCase()
    .replace(/^./, m => m.toUpperCase());
};
```

1452 People Whose List Of Favorite Companies Is Not A Subset Of Another

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1452. People Whose List of Favorite Companies Is Not a Subset of Another List
 * https://leetcode.com/problems/people-whose-list-of-favorite-companies-is-not-a-subset-of-an
other-list/
 * Difficulty: Medium
 *
 * Given the array favoriteCompanies where favoriteCompanies[i] is the list of favorites
companies
 * for the ith person (indexed from 0).
 *
 * Return the indices of people whose list of favorite companies is not a subset of any other
list
 * of favorites companies. You must return the indices in increasing order.
 */

/**
 * @param {string[][]} favoriteCompanies
 * @return {number[]}
 */
var peopleIndexes = function(favoriteCompanies) {
  const companySets = favoriteCompanies.map(companies => new Set(companies));
  const result = [];

  for (let i = 0; i < favoriteCompanies.length; i++) {
    let isSubset = false;
    for (let j = 0; j < favoriteCompanies.length; j++) {
      if (i !== j && isSubsetOf(companySets[i], companySets[j])) {
        isSubset = true;
        break;
      }
    }
    if (!isSubset) {
      result.push(i);
    }
  }

  return result;

  function isSubsetOf(setA, setB) {
    if (setA.size > setB.size) return false;
    for (const item of setA) {
      if (!setB.has(item)) return false;
    }
    return true;
  }
};
```

1453 Maximum Number Of Darts Inside Of A Circular Dartboard

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1453. Maximum Number of Darts Inside of a Circular Dartboard
 * https://leetcode.com/problems/maximum-number-of-darts-inside-of-a-circular-dartboard/
 * Difficulty: Hard
 *
 * Alice is throwing n darts on a very large wall. You are given an array darts where
 * darts[i] = [xi, yi] is the position of the ith dart that Alice threw on the wall.
 *
 * Bob knows the positions of the n darts on the wall. He wants to place a dartboard of
 * radius r on the wall so that the maximum number of darts that Alice throws lie on the
 * dartboard.
 *
 * Given the integer r, return the maximum number of darts that can lie on the dartboard.
 */

/**
 * @param {number[][]} darts
 * @param {number} r
 * @return {number}
 */
var numPoints = function(darts, r) {
  let maxDarts = 1;
  const n = darts.length;
  const radiusSquared = r * r;

  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      const dx = darts[j][0] - darts[i][0];
      const dy = darts[j][1] - darts[i][1];
      const distSquared = dx * dx + dy * dy;

      if (distSquared > 4 * radiusSquared + 1e-8) continue;

      const midX = (darts[i][0] + darts[j][0]) / 2;
      const midY = (darts[i][1] + darts[j][1]) / 2;

      if (distSquared < 1e-8) continue;

      const dist = Math.sqrt(distSquared) / 2;
      const height = Math.sqrt(radiusSquared - dist * dist);

      let perpX = -dy;
      let perpY = dx;
      const norm = Math.sqrt(perpX * perpX + perpY * perpY);
      perpX /= norm;
      perpY /= norm;

      const count1 = countDartsInCircle(darts, midX + height * perpX, midY + height * perpY,
r);
      const count2 = countDartsInCircle(darts, midX - height * perpX, midY - height * perpY,
r);

      maxDarts = Math.max(maxDarts, count1, count2);
    }
  }

  return maxDarts;

  function countDartsInCircle(darts, centerX, centerY, r) {
    let count = 0;
    const radiusSquared = r * r;

    for (const dart of darts) {
      const dx = dart[0] - centerX;
      const dy = dart[1] - centerY;
      if (dx * dx + dy * dy <= radiusSquared + 1e-8) {
```

```
        count++;
      }
    }

    return count;
  }
};
```

1455 Check If A Word Occurs As A Prefix Of Any Word In A Sentence

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1455. Check If a Word Occurs As a Prefix of Any Word in a Sentence
 * https://leetcode.com/problems/check-if-a-word-occurs-as-a-prefix-of-any-word-in-a-sentence/
 * Difficulty: Easy
 *
 * Given a sentence that consists of some words separated by a single space, and a searchWord,
 * check if searchWord is a prefix of any word in sentence.
 *
 * Return the index of the word in sentence (1-indexed) where searchWord is a prefix of this
word.
 * If searchWord is a prefix of more than one word, return the index of the first word
(minimum index).
 * If there is no such word return -1.
 *
 * A prefix of a string s is any leading contiguous substring of s.
 */

/**
 * @param {string} sentence
 * @param {string} searchWord
 * @return {number}
 */
var isPrefixOfWord = function(sentence, searchWord) {
  return sentence.split(/\s+/).findIndex(s => s.startsWith(searchWord)) + 1 || -1;
};
```

1456 Maximum Number Of Vowels In A Substring Of Given Length

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1456. Maximum Number of Vowels in a Substring of Given Length
 * https://leetcode.com/problems/maximum-number-of-vowels-in-a-substring-of-given-length/
 * Difficulty: Medium
 *
 * Given a string s and an integer k.
 *
 * Return the maximum number of vowel letters in any substring of s with length k.
 *
 * Vowel letters in English are (a, e, i, o, u).
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var maxVowels = function(s, k) {
  const vowels = 'aeiou';
  let result = 0;

  for (let i = 0, count = 0; i < s.length; i++) {
    if (vowels.includes(s[i])) {
```

```
      count++;
    }
    if (i >= k && vowels.includes(s[i - k])) {
      count--;
    }
    result = Math.max(result, count);
  }

  return result;
};
```

1457 Pseudo Palindromic Paths In A Binary Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1457. Pseudo-Palindromic Paths in a Binary Tree
 * https://leetcode.com/problems/pseudo-palindromic-paths-in-a-binary-tree/
 * Difficulty: Medium
 *
 * Given a binary tree where node values are digits from 1 to 9. A path in the binary tree
 * is said to be pseudo-palindromic if at least one permutation of the node values in the
 * path is a palindrome.
 *
 * Return the number of pseudo-palindromic paths going from the root node to leaf nodes.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var pseudoPalindromicPaths = function(root) {
  const frequency = new Array(10).fill(0);
  let count = 0;

  traverse(root);

  return count;

  function traverse(node) {
    if (!node) return;

    frequency[node.val]++;

    if (!node.left && !node.right) {
      let oddCount = 0;
      for (let i = 1; i <= 9; i++) {
        if (frequency[i] % 2 !== 0) oddCount++;
      }
      if (oddCount <= 1) count++;
    }

    traverse(node.left);
    traverse(node.right);

    frequency[node.val]--;
  }
};
```

1458 Max Dot Product Of Two Subsequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1458. Max Dot Product of Two Subsequences
 * https://leetcode.com/problems/max-dot-product-of-two-subsequences/
 * Difficulty: Hard
 *
 * Given two arrays nums1 and nums2.
 *
 * Return the maximum dot product between non-empty subsequences of nums1 and nums2 with the
 * same length.
 *
 * A subsequence of a array is a new array which is formed from the original array by deleting
 * some (can be none) of the characters without disturbing the relative positions of the
remaining
 * characters. (ie, [2,3,5] is a subsequence of [1,2,3,4,5] while [1,5,3] is not).
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var maxDotProduct = function(nums1, nums2) {
  const m = nums1.length;
  const n = nums2.length;
  const dp = new Array(m + 1).fill().map(() => new Array(n + 1).fill(-Infinity));

  let result = -Infinity;
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      result = Math.max(result, computeMax(i, j));
    }
  }

  return result;

  function computeMax(i, j) {
    if (i === 0 || j === 0) return -Infinity;
    if (dp[i][j] !== -Infinity) return dp[i][j];

    dp[i][j] = nums1[i - 1] * nums2[j - 1];
    dp[i][j] = Math.max(
      dp[i][j],
      dp[i][j] + computeMax(i - 1, j - 1),
      computeMax(i - 1, j),
      computeMax(i, j - 1)
    );

    return dp[i][j];
  }
};
```

1460 Make Two Arrays Equal By Reversing Sub Arrays

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1460. Make Two Arrays Equal by Reversing Sub-arrays
 * https://leetcode.com/problems/make-two-arrays-equal-by-reversing-sub-arrays/
 * Difficulty: Easy
 *
 * Given two integer arrays of equal length target and arr.
```

```
 *
 * In one step, you can select any non-empty sub-array of arr and reverse it.
 * You are allowed to make any number of steps.
 *
 * Return True if you can make arr equal to target, or False otherwise.
 */

/**
 * @param {number[]} target
 * @param {number[]} arr
 * @return {boolean}
 */
var canBeEqual = function(target, arr) {
  return target.sort().join('') === arr.sort().join('');
};
```

1461 Check If A String Contains All Binary Codes Of Size K

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1461. Check If a String Contains All Binary Codes of Size K
 * https://leetcode.com/problems/check-if-a-string-contains-all-binary-codes-of-size-k/
 * Difficulty: Medium
 *
 * Given a binary string s and an integer k, return true if every binary code of length k is a
 * substring of s. Otherwise, return false.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {boolean}
 */
var hasAllCodes = function(s, k) {
  const requiredCount = 1 << k;
  const seenCodes = new Set();

  for (let i = 0; i <= s.length - k; i++) {
    seenCodes.add(s.slice(i, i + k));
    if (seenCodes.size === requiredCount) return true;
  }

  return false;
};
```

1462 Course Schedule Iv

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1462. Course Schedule IV
 * https://leetcode.com/problems/course-schedule-iv/
 * Difficulty: Medium
 *
 * There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1.
 * You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you
 * must take course ai first if you want to take course bi.
 *
 * - For example, the pair [0, 1] indicates that you have to take course 0 before you can take
 *   course 1.
 *
 * Prerequisites can also be indirect. If course a is a prerequisite of course b, and course b
 * is a prerequisite of course c, then course a is a prerequisite of course c.
 *
```

```
 * You are also given an array queries where queries[j] = [uj, vj]. For the jth query, you
 * should answer whether course uj is a prerequisite of course vj or not.
 *
 * Return a boolean array answer, where answer[j] is the answer to the jth query.
 */

/**
 * @param {number} numCourses
 * @param {number[][]} prerequisites
 * @param {number[][]} queries
 * @return {boolean[]}
 */
var checkIfPrerequisite = function(numCourses, prerequisites, queries) {
  const lookup = new Array(numCourses).fill().map(() => new Array(numCourses).fill(false));
  prerequisites.forEach(([u, v]) => lookup[u][v] = true);

  for (let i = 0; i < numCourses; i++) {
    for (let j = 0; j < numCourses; j++) {
      for (let k = 0; k < numCourses; k++) {
        if (lookup[j][i] && lookup[i][k]) {
          lookup[j][k] = true;
        }
      }
    }
  }

  return queries.map(([u, v]) => lookup[u][v]);
};
```

1463 Cherry Pickup Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1463. Cherry Pickup II
 * https://leetcode.com/problems/cherry-pickup-ii/
 * Difficulty: Hard
 *
 * You are given a rows x cols matrix grid representing a field of cherries where grid[i][j]
 * represents the number of cherries that you can collect from the (i, j) cell.
 *
 * You have two robots that can collect cherries for you:
 * - Robot #1 is located at the top-left corner (0, 0), and
 * - Robot #2 is located at the top-right corner (0, cols - 1).
 *
 * Return the maximum number of cherries collection using both robots by following the rules
below:
 * - From a cell (i, j), robots can move to cell (i + 1, j - 1), (i + 1, j), or (i + 1, j +
1).
 * - When any robot passes through a cell, It picks up all cherries, and the cell becomes an
empty
 *   cell.
 * - When both robots stay in the same cell, only one takes the cherries.
 * - Both robots cannot move outside of the grid at any moment.
 * - Both robots should reach the bottom row in grid.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var cherryPickup = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const cache = new Array(rows).fill().map(() => {
    return new Array(cols).fill().map(() => new Array(cols).fill(-1));
  });

  return Math.max(0, findMaxCherries(0, 0, cols - 1));
```

```
  function findMaxCherries(row, col1, col2) {
    if (row === rows || col1 < 0 || col1 >= cols || col2 < 0 || col2 >= cols) {
      return -Infinity;
    }
    if (cache[row][col1][col2] !== -1) {
      return cache[row][col1][col2];
    }

    const cherries = col1 === col2 ? grid[row][col1] : grid[row][col1] + grid[row][col2];

    if (row === rows - 1) {
      return cherries;
    }

    let maxCherries = -Infinity;
    for (const nextCol1 of [col1 - 1, col1, col1 + 1]) {
      for (const nextCol2 of [col2 - 1, col2, col2 + 1]) {
        maxCherries = Math.max(maxCherries, findMaxCherries(row + 1, nextCol1, nextCol2));
      }
    }

    cache[row][col1][col2] = cherries + maxCherries;
    return cache[row][col1][col2];
  }
};
```

1464 Maximum Product Of Two Elements In An Array

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1464. Maximum Product of Two Elements in an Array
 * https://leetcode.com/problems/maximum-product-of-two-elements-in-an-array/
 * Difficulty: Easy
 *
 * Given the array of integers nums, you will choose two different indices i and j
 * of that array. Return the maximum value of (nums[i]-1)*(nums[j]-1).
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxProduct = function(nums) {
  const [a, b] = nums.sort((a, b) => b - a);
  return (a - 1) * (b - 1);
};
```

1465 Maximum Area Of A Piece Of Cake After Horizontal And Vertical Cuts

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1465. Maximum Area of a Piece of Cake After Horizontal and Vertical Cuts
 * https://leetcode.com/problems/maximum-area-of-a-piece-of-cake-after-horizontal-and-vertical
-cuts/
 * Difficulty: Medium
 *
 * You are given a rectangular cake of size h x w and two arrays of integers horizontalCuts
and
 * verticalCuts where:
 * - horizontalCuts[i] is the distance from the top of the rectangular cake to the ith
horizontal
 *   cut and similarly, and
 * - verticalCuts[j] is the distance from the left of the rectangular cake to the jth vertical
```

```
 cut.
 *
 * Return the maximum area of a piece of cake after you cut at each horizontal and vertical
position
 * provided in the arrays horizontalCuts and verticalCuts. Since the answer can be a large
number,
 * return this modulo 109 + 7.
 */

/**
 * @param {number} h
 * @param {number} w
 * @param {number[]} horizontalCuts
 * @param {number[]} verticalCuts
 * @return {number}
 */
var maxArea = function(h, w, horizontalCuts, verticalCuts) {
  horizontalCuts.sort((a, b) => a - b);
  verticalCuts.sort((a, b) => a - b);

  let maxHeight = Math.max(horizontalCuts[0], h - horizontalCuts[horizontalCuts.length - 1]);
  let maxWidth = Math.max(verticalCuts[0], w - verticalCuts[verticalCuts.length - 1]);

  for (let i = 1; i < horizontalCuts.length; i++) {
    maxHeight = Math.max(maxHeight, horizontalCuts[i] - horizontalCuts[i - 1]);
  }
  for (let i = 1; i < verticalCuts.length; i++) {
    maxWidth = Math.max(maxWidth, verticalCuts[i] - verticalCuts[i - 1]);
  }

  return Number((BigInt(maxHeight) * BigInt(maxWidth)) % BigInt(1000000007));
};
```

1466 Reorder Routes To Make All Paths Lead To The City Zero

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1466. Reorder Routes to Make All Paths Lead to the City Zero
 * https://leetcode.com/problems/reorder-routes-to-make-all-paths-lead-to-the-city-zero/
 * Difficulty: Medium
 *
 * There are n cities numbered from 0 to n - 1 and n - 1 roads such that there is only one way
 * to travel between two different cities (this network form a tree). Last year, The ministry
 * of transport decided to orient the roads in one direction because they are too narrow.
 *
 * Roads are represented by connections where connections[i] = [ai, bi] represents a road from
 * city ai to city bi.
 *
 * This year, there will be a big event in the capital (city 0), and many people want to
travel
 * to this city.
 *
 * Your task consists of reorienting some roads such that each city can visit the city 0.
Return
 * the minimum number of edges changed.

It's guaranteed that each city can reach city 0 after reorder.
 */

/**
 * @param {number} n
 * @param {number[][]} connections
 * @return {number}
 */
var minReorder = function(n, connections) {
  const graph = new Array(n).fill(null).map(() => []);

  connections.forEach(([k, v]) => {
    graph[k].push([v, 1]);
```

```
    graph[v].push([k, 0]);
  });

  function dfs(node, parent) {
    let count = 0;

    graph[node].forEach(([k, v]) => {
      if (k !== parent) {
        count += v;
        count += dfs(k, node);
      }
    });

    return count;
  }

  return dfs(0, -1);
};
```

1467 Probability Of A Two Boxes Having The Same Number Of Distinct Ball

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1467. Probability of a Two Boxes Having The Same Number of Distinct Balls
 * https://leetcode.com/problems/probability-of-a-two-boxes-having-the-same-number-of-distinct
-balls/
 * Difficulty: Hard
 *
 * Given 2n balls of k distinct colors. You will be given an integer array balls of size k
where
 * balls[i] is the number of balls of color i.
 *
 * All the balls will be shuffled uniformly at random, then we will distribute the first n
balls to
 * the first box and the remaining n balls to the other box (Please read the explanation of
the
 * second example carefully).
 *
 * Please note that the two boxes are considered different. For example, if we have two balls
of
 * colors a and b, and two boxes [] and (), then the distribution [a] (b) is considered
different
 * than the distribution [b] (a) (Please read the explanation of the first example carefully).
 *
 * Return the probability that the two boxes have the same number of distinct balls. Answers
 * within 10-5 of the actual value will be accepted as correct.
 */

/**
 * @param {number[]} balls
 * @return {number}
 */
var getProbability = function(balls) {
  const totalBalls = balls.reduce((sum, count) => sum + count, 0);
  const halfBalls = totalBalls / 2;
  let validCombinations = 0;
  let totalCombinations = 0;

  calculateCombinations(0, new Array(balls.length).fill(0), halfBalls, 0);
  return validCombinations / totalCombinations;

  function countDistinct(config) {
    return config.filter(count => count > 0).length;
  }

  function calculateCombinations(index, config, remaining, firstBoxCount) {
    if (index === balls.length) {
      if (firstBoxCount === halfBalls) {
        const secondBox = balls.map((count, i) => count - config[i]);
```

```
        if (countDistinct(config) === countDistinct(secondBox)) {
          validCombinations += combinatorialProduct(config) * combinatorialProduct(secondBox);
        }
        totalCombinations += combinatorialProduct(config) * combinatorialProduct(secondBox);
      }
      return;
    }

    for (let i = 0; i <= balls[index] && firstBoxCount + i <= halfBalls; i++) {
      if (remaining >= i) {
        config[index] = i;
        calculateCombinations(index + 1, config, remaining - i, firstBoxCount + i);
        config[index] = 0;
      }
    }
  }

  function combinatorialProduct(config) {
    const numerator = factorial(halfBalls);
    let denominator = 1;
    for (const count of config) {
      denominator *= factorial(count);
    }
    return numerator / denominator;
  }

  function factorial(n) {
    let result = 1;
    for (let i = 2; i <= n; i++) {
      result *= i;
    }
    return result;
  }
};
```

1469 Find All The Lonely Nodes

EASY

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1469. Find All The Lonely Nodes
 * https://leetcode.com/problems/find-all-the-lonely-nodes/
 * Difficulty: Easy
 *
 * In a binary tree, a lonely node is a node that is the only child of its parent node.
 * The root of the tree is not lonely because it does not have a parent node.
 *
 * Given the root of a binary tree, return an array containing the values of all lonely
 * nodes in the tree. Return the list in any order.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var getLonelyNodes = function(root) {
  const result = [];
  dfs(root);
  return result;

  function dfs(node) {
    if (!node) return;
```

```
    if (node.left && !node.right) {
      result.push(node.left.val);
    }

    if (!node.left && node.right) {
      result.push(node.right.val);
    }

    dfs(node.left);
    dfs(node.right);
  }
};
```

1470 Shuffle The Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1470. Shuffle the Array
 * https://leetcode.com/problems/shuffle-the-array/
 * Difficulty: Easy
 *
 * Given the array nums consisting of 2n elements in the form [x1,x2,...,xn,y1,y2,...,yn].
 *
 * Return the array in the form [x1,y1,x2,y2,...,xn,yn].
 */

/**
 * @param {number[]} nums
 * @param {number} n
 * @return {number[]}
 */
var shuffle = function(nums, n) {
  const shuffled = [];
  for (let i = 0; i < nums.length / 2; i++) {
    shuffled.push(nums[i], nums[i + nums.length / 2]);
  }
  return shuffled;
};
```

1471 The K Strongest Values In An Array

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1471. The k Strongest Values in an Array
 * https://leetcode.com/problems/the-k-strongest-values-in-an-array/
 * Difficulty: Medium
 *
 * Given an array of integers arr and an integer k.
 *
 * A value arr[i] is said to be stronger than a value arr[j] if |arr[i] - m| > |arr[j] - m|
 * where m is the centre of the array.
 *
 * If |arr[i] - m| == |arr[j] - m|, then arr[i] is said to be stronger than arr[j]
 * if arr[i] > arr[j].
 *
 * Return a list of the strongest k values in the array. return the answer in any arbitrary
order.
 *
 * The centre is the middle value in an ordered integer list. More formally, if the length of
the
 * list is n, the centre is the element in position ((n - 1) / 2) in the sorted list
(0-indexed).
```

```
 * - For arr = [6, -3, 7, 2, 11], n = 5 and the centre is obtained by sorting the array
 *   arr = [-3, 2, 6, 7, 11] and the centre is arr[m] where m = ((5 - 1) / 2) = 2. The centre
is 6.
 * - For arr = [-7, 22, 17, 3], n = 4 and the centre is obtained by sorting the array
 *   arr = [-7, 3, 17, 22] and the centre is arr[m] where m = ((4 - 1) / 2) = 1. The centre is
 3.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number[]}
 */
var getStrongest = function(arr, k) {
  const sorted = arr.slice().sort((a, b) => a - b);
  const centre = sorted[Math.floor((sorted.length - 1) / 2)];

  return arr.sort((a, b) => {
    const diffA = Math.abs(a - centre);
    const diffB = Math.abs(b - centre);
    return diffA === diffB ? b - a : diffB - diffA;
  }).slice(0, k);
};
```

1472 Design Browser History

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1472. Design Browser History
 * https://leetcode.com/problems/design-browser-history/
 * Difficulty: Medium
 *
 * You have a browser of one tab where you start on the homepage and you
 * can visit another url, get back in the history number of steps or move
 * forward in the history number of steps.
 *
 * Implement the BrowserHistory class:
 *
 * - `BrowserHistory(string homepage)` Initializes the object with the
 *   homepage of the browser.
 * - `void visit(string url)` Visits url from the current page. It clears
 *   up all the forward history.
 * - `string back(int steps)` Move steps back in history. If you can only
 *   return x steps in the history and steps > x, you will return only x
 *   steps. Return the current url after moving back in history at most steps.
 * - `string forward(int steps)` Move steps forward in history. If you can
 *   only forward x steps in the history and steps > x, you will forward only
 *   x steps. Return the current url after forwarding in history at most steps.
 */

/**
 * @param {string} homepage
 */
var BrowserHistory = function(homepage) {
  this.history = [];
  this.cursor = -1;
  this.visit(homepage);
};

/**
 * @param {string} url
 * @return {void}
 */
BrowserHistory.prototype.visit = function(url) {
  this.history.splice(this.cursor + 1, this.history.length);
  this.history.push(url);
  this.cursor++;
};
```

```
/**
 * @param {number} steps
 * @return {string}
 */
BrowserHistory.prototype.back = function(steps) {
  this.cursor = Math.max(0, this.cursor - steps);
  return this.history[this.cursor];
};

/**
 * @param {number} steps
 * @return {string}
 */
BrowserHistory.prototype.forward = function(steps) {
  this.cursor = Math.min(this.cursor + steps, this.history.length - 1);
  return this.history[this.cursor];
};
```

1473 Paint House Iii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1473. Paint House III
 * https://leetcode.com/problems/paint-house-iii/
 * Difficulty: Hard
 *
 * There is a row of m houses in a small city, each house must be painted with one of the n
 * colors (labeled from 1 to n), some houses that have been painted last summer should not
 * be painted again.
 *
 * A neighborhood is a maximal group of continuous houses that are painted with the same
color.
 *
 * For example: houses = [1,2,2,3,3,2,1,1] contains 5 neighborhoods [{1}, {2,2}, {3,3}, {2},
{1,1}].
 *
 * Given an array houses, an m x n matrix cost and an integer target where:
 * - houses[i]: is the color of the house i, and 0 if the house is not painted yet.
 * - cost[i][j]: is the cost of paint the house i with the color j + 1.
 *
 * Return the minimum cost of painting all the remaining houses in such a way that there are
exactly
 * target neighborhoods. If it is not possible, return -1.
 */

/**
 * @param {number[]} houses
 * @param {number[][]} cost
 * @param {number} m
 * @param {number} n
 * @param {number} target
 * @return {number}
 */
var minCost = function(houses, cost, m, n, target) {
  const cache = new Map();
  const result = findMinCost(0, 0, 0);
  return result === Infinity ? -1 : result;

  function findMinCost(index, prevColor, neighborhoods) {
    if (index === m) return neighborhoods === target ? 0 : Infinity;
    if (neighborhoods > target) return Infinity;

    const key = `${index}:${prevColor}:${neighborhoods}`;
    if (cache.has(key)) return cache.get(key);

    let minCost = Infinity;
    const currentColor = houses[index];

    if (currentColor !== 0) {
```

```
      const newNeighborhoods = prevColor === currentColor ? neighborhoods : neighborhoods + 1;
      minCost = findMinCost(index + 1, currentColor, newNeighborhoods);
    } else {
      for (let color = 1; color <= n; color++) {
        const newNeighborhoods = prevColor === color ? neighborhoods : neighborhoods + 1;
        const currentCost = cost[index][color - 1]
          + findMinCost(index + 1, color, newNeighborhoods);
        minCost = Math.min(minCost, currentCost);
      }
    }

    cache.set(key, minCost);
    return minCost;
  }
};
```

1474 Delete N Nodes After M Nodes Of A Linked List

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1474. Delete N Nodes After M Nodes of a Linked List
 * https://leetcode.com/problems/delete-n-nodes-after-m-nodes-of-a-linked-list/
 * Difficulty: Easy
 *
 * You are given the head of a linked list and two integers m and n.
 *
 * Traverse the linked list and remove some nodes in the following way:
 * - Start with the head as the current node.
 * - Keep the first m nodes starting with the current node.
 * - Remove the next n nodes
 * - Keep repeating steps 2 and 3 until you reach the end of the list.
 *
 * Return the head of the modified list after removing the mentioned nodes.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} m
 * @param {number} n
 * @return {ListNode}
 */
var deleteNodes = function(head, m, n) {
  let current = head;

  while (current) {
    for (let i = 1; i < m && current; i++) {
      current = current.next;
    }

    if (!current) break;

    let nodeToDelete = current.next;
    for (let i = 0; i < n && nodeToDelete; i++) {
      nodeToDelete = nodeToDelete.next;
    }

    current.next = nodeToDelete;
    current = nodeToDelete;
  }

  return head;
};
```

1475 Final Prices With A Special Discount In A Shop

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1475. Final Prices With a Special Discount in a Shop
 * https://leetcode.com/problems/final-prices-with-a-special-discount-in-a-shop/
 * Difficulty: Easy
 *
 * Given the array prices where prices[i] is the price of the ith item in a shop.
 * There is a special discount for items in the shop, if you buy the ith item,
 * then you will receive a discount equivalent to prices[j] where j is the minimum
 * index such that j > i and prices[j] <= prices[i], otherwise, you will not receive
 * any discount at all.
 *
 * Return an array where the ith element is the final price you will pay for the ith
 * item of the shop considering the special discount.
 */

/**
 * @param {number[]} prices
 * @return {number[]}
 */
var finalPrices = function(prices) {
  return prices.map((n, i) => n - (prices.slice(i + 1).find(v => v <= n) || 0));
};
```

1476 Subrectangle Queries

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1476. Subrectangle Queries
 * https://leetcode.com/problems/subrectangle-queries/
 * Difficulty: Medium
 *
 * Implement the class SubrectangleQueries which receives a rows x cols rectangle as a matrix
 * of integers in the constructor and supports two methods:
 * 1. updateSubrectangle(int row1, int col1, int row2, int col2, int newValue)
 *    - Updates all values with newValue in the subrectangle whose upper left coordinate is
 *      (row1,col1) and bottom right coordinate is (row2,col2).
 * 2. getValue(int row, int col)
 *
 * Returns the current value of the coordinate (row,col) from the rectangle.
 */

/**
 * @param {number[][]} rectangle
 */
var SubrectangleQueries = function(rectangle) {
  this.grid = rectangle.map(row => [...row]);
};

/**
 * @param {number} row1
 * @param {number} col1
 * @param {number} row2
 * @param {number} col2
 * @param {number} newValue
 * @return {void}
 */
SubrectangleQueries.prototype.updateSubrectangle = function(row1, col1, row2, col2, newValue)
{
  for (let i = row1; i <= row2; i++) {
    for (let j = col1; j <= col2; j++) {
```

```
      this.grid[i][j] = newValue;
    }
  }
};

/**
 * @param {number} row
 * @param {number} col
 * @return {number}
 */
SubrectangleQueries.prototype.getValue = function(row, col) {
  return this.grid[row][col];
};
```

1477 Find Two Non Overlapping Sub Arrays Each With Target Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1477. Find Two Non-overlapping Sub-arrays Each With Target Sum
 * https://leetcode.com/problems/find-two-non-overlapping-sub-arrays-each-with-target-sum/
 * Difficulty: Medium
 *
 * You are given an array of integers arr and an integer target.
 *
 * You have to find two non-overlapping sub-arrays of arr each with a sum equal target. There
 * can be multiple answers so you have to find an answer where the sum of the lengths of the
 * two sub-arrays is minimum.
 *
 * Return the minimum sum of the lengths of the two required sub-arrays, or return -1 if you
 * cannot find such two sub-arrays.
 */

/**
 * @param {number[]} arr
 * @param {number} target
 * @return {number}
 */
var minSumOfLengths = function(arr, target) {
  const prefixSums = new Map([[0, -1]]);
  let currentSum = 0;
  let minLength = Infinity;
  const minLengths = new Array(arr.length).fill(Infinity);
  let result = Infinity;

  for (let i = 0; i < arr.length; i++) {
    currentSum += arr[i];
    if (prefixSums.has(currentSum - target)) {
      const start = prefixSums.get(currentSum - target);
      const length = i - start;
      minLength = Math.min(minLength, length);
      if (start >= 0 && minLengths[start] !== Infinity) {
        result = Math.min(result, length + minLengths[start]);
      }
    }
    minLengths[i] = minLength;
    prefixSums.set(currentSum, i);
  }

  return result === Infinity ? -1 : result;
};
```

1478 Allocate Mailboxes

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and

sorting cost.

SOURCE CODE

```
/**
 * 1478. Allocate Mailboxes
 * https://leetcode.com/problems/allocate-mailboxes/
 * Difficulty: Hard
 *
 * Given the array houses where houses[i] is the location of the ith house along a street and
 * an integer k, allocate k mailboxes in the street.
 *
 * Return the minimum total distance between each house and its nearest mailbox.
 *
 * The test cases are generated so that the answer fits in a 32-bit integer.
 */

/**
 * @param {number[]} houses
 * @param {number} k
 * @return {number}
 */
var minDistance = function(houses, k) {
  houses.sort((a, b) => a - b);
  const n = houses.length;
  const cache = new Map();

  return findMinDistance(0, k);

  function medianDistance(start, end) {
    let sum = 0;
    const mid = Math.floor((start + end) / 2);
    for (let i = start; i <= end; i++) {
      sum += Math.abs(houses[i] - houses[mid]);
    }
    return sum;
  }

  function findMinDistance(index, mailboxes) {
    if (index === n) return mailboxes === 0 ? 0 : Infinity;
    if (mailboxes === 0) return Infinity;

    const key = `${index}:${mailboxes}`;
    if (cache.has(key)) return cache.get(key);

    let minDist = Infinity;
    for (let j = index; j < n && n - j >= mailboxes; j++) {
      const distance = medianDistance(index, j) + findMinDistance(j + 1, mailboxes - 1);
      minDist = Math.min(minDist, distance);
    }

    cache.set(key, minDist);
    return minDist;
  }
};
```

1480 Running Sum Of 1D Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1480. Running Sum of 1d Array
 * https://leetcode.com/problems/running-sum-of-1d-array/
 * Difficulty: Easy
 *
 * Given an array nums. We define a running sum of an array as
 * runningSum[i] = sum(nums[0]…nums[i]).
 *
 * Return the running sum of nums.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number[]}
 */
var runningSum = function(nums) {
  for (let i = 1; i < nums.length; i++) {
    nums[i] = nums[i - 1] + nums[i];
  }
  return nums;
};
```

1481 Least Number Of Unique Integers After K Removals

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1481. Least Number of Unique Integers after K Removals
 * https://leetcode.com/problems/least-number-of-unique-integers-after-k-removals/
 * Difficulty: Medium
 *
 * Given an array of integers arr and an integer k. Find the least number of
 * unique integers after removing exactly k elements.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number}
 */
var findLeastNumOfUniqueInts = function(arr, k) {
  const map = new Map();
  arr.forEach(n => map.set(n, map.has(n) ? map.get(n) + 1 : 1));

  const sorted = [...map.values()].sort((a, b) => a - b);
  while (k && sorted.length && sorted[0] <= k) {
    k -= sorted[0];
    sorted.shift();
  }

  return sorted.length;
};
```

1482 Minimum Number Of Days To Make M Bouquets

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1482. Minimum Number of Days to Make m Bouquets
 * https://leetcode.com/problems/minimum-number-of-days-to-make-m-bouquets/
 * Difficulty: Medium
 *
 * You are given an integer array bloomDay, an integer m and an integer k.
 *
 * You want to make m bouquets. To make a bouquet, you need to use k adjacent flowers from the
 * garden.
 *
 * The garden consists of n flowers, the ith flower will bloom in the bloomDay[i] and then can
be
 * used in exactly one bouquet.
 *
 * Return the minimum number of days you need to wait to be able to make m bouquets from the
garden.
 * If it is impossible to make m bouquets return -1.
 */
```

```
/**
 * @param {number[]} bloomDay
 * @param {number} m
 * @param {number} k
 * @return {number}
 */
var minDays = function(bloomDay, m, k) {
  if (m * k > bloomDay.length) return -1;

  let left = 1;
  let right = Math.max(...bloomDay);
  let result = -1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (canMakeBouquets(mid)) {
      result = mid;
      right = mid - 1;
    } else {
      left = mid + 1;
    }
  }

  return result;

  function canMakeBouquets(days) {
    let bouquets = 0;
    let flowers = 0;
    for (const bloom of bloomDay) {
      if (bloom <= days) {
        flowers++;
        if (flowers === k) {
          bouquets++;
          flowers = 0;
        }
      } else {
        flowers = 0;
      }
    }
    return bouquets >= m;
  }
};
```

1483 Kth Ancestor Of A Tree Node

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1483. Kth Ancestor of a Tree Node
 * https://leetcode.com/problems/kth-ancestor-of-a-tree-node/
 * Difficulty: Hard
 *
 * You are given a tree with n nodes numbered from 0 to n - 1 in the form of a parent array
 * parent where parent[i] is the parent of ith node. The root of the tree is node 0. Find the
 * kth ancestor of a given node.
 *
 * The kth ancestor of a tree node is the kth node in the path from that node to the root
node.
 *
 * Implement the TreeAncestor class:
 * - TreeAncestor(int n, int[] parent) Initializes the object with the number of nodes in the
tree
 *   and the parent array.
 * - int getKthAncestor(int node, int k) return the kth ancestor of the given node node. If
there
 *   is no such ancestor, return -1.
 */

/**
```

```
 * @param {number} n
 * @param {number[]} parent
 */
var TreeAncestor = function(n, parent) {
  const maxDepth = Math.ceil(Math.log2(n));
  this.ancestors = Array.from({ length: n }, () => new Array(maxDepth + 1).fill(-1));

  for (let i = 0; i < n; i++) {
    this.ancestors[i][0] = parent[i];
  }

  for (let j = 1; j <= maxDepth; j++) {
    for (let i = 0; i < n; i++) {
      if (this.ancestors[i][j - 1] !== -1) {
        this.ancestors[i][j] = this.ancestors[this.ancestors[i][j - 1]][j - 1];
      }
    }
  }
};

/**
 * @param {number} node
 * @param {number} k
 * @return {number}
 */
TreeAncestor.prototype.getKthAncestor = function(node, k) {
  let current = node;
  for (let j = 0; k > 0 && current !== -1; j++, k >>= 1) {
    if (k & 1) {
      current = this.ancestors[current][j];
    }
  }
  return current;
};
```

1485 Clone Binary Tree With Random Pointer

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1485. Clone Binary Tree With Random Pointer
 * https://leetcode.com/problems/clone-binary-tree-with-random-pointer/
 * Difficulty: Medium
 *
 * A binary tree is given such that each node contains an additional random pointer which
 * could point to any node in the tree or null.
 *
 * Return a deep copy of the tree.
 *
 * The tree is represented in the same input/output way as normal binary trees where each
 * node is represented as a pair of [val, random_index] where:
 * - val: an integer representing Node.val
 * - random_index: the index of the node (in the input) where the random pointer points to,
 *   or null if it does not point to any node.
 *
 * You will be given the tree in class Node and you should return the cloned tree in class
 * NodeCopy. NodeCopy class is just a clone of Node class with the same attributes and
constructors.
 */

/**
 * // Definition for a _Node.
 * function _Node(val, left, right, random) {
 *    this.val = val === undefined ? null : val;
 *    this.left = left === undefined ? null : left;
 *    this.right = right === undefined ? null : right;
 *    this.random = random === undefined ? null : random;
 * };
 */
```

```
/**
 * @param {_Node} root
 * @return {NodeCopy}
 */
var copyRandomBinaryTree = function(root) {
  if (!root) return null;
  const nodeMap = new Map();
  return createCopy(root);

  function createCopy(node) {
    if (!node) return null;
    if (nodeMap.has(node)) return nodeMap.get(node);

    const copy = new NodeCopy(node.val);
    nodeMap.set(node, copy);

    copy.left = createCopy(node.left);
    copy.right = createCopy(node.right);
    copy.random = createCopy(node.random);

    return copy;
  }
};
```

1486 Xor Operation In An Array

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1486. XOR Operation in an Array
 * https://leetcode.com/problems/xor-operation-in-an-array/
 * Difficulty: Easy
 *
 * Given an integer n and an integer start.
 *
 * Define an array nums where nums[i] = start + 2*i (0-indexed) and n == nums.length.
 *
 * Return the bitwise XOR of all elements of nums.
 */

/**
 * @param {number} n
 * @param {number} start
 * @return {number}
 */
var xorOperation = function(n, start) {
  return new Array(n)
    .fill(0)
    .map((_, i) => start + 2 * i)
    .reduce((result, n) => result ^ n, 0);
};
```

1487 Making File Names Unique

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1487. Making File Names Unique
 * https://leetcode.com/problems/making-file-names-unique/
 * Difficulty: Medium
 *
 * Given an array of strings names of size n. You will create n folders in your file system
such
 * that, at the ith minute, you will create a folder with the name names[i].
```

```
 *
 * Since two files cannot have the same name, if you enter a folder name that was previously
used,
 * the system will have a suffix addition to its name in the form of (k), where, k is the
smallest
 * positive integer such that the obtained name remains unique.
 *
 * Return an array of strings of length n where ans[i] is the actual name the system will
assign
 * to the ith folder when you create it.
 */

/**
 * @param {string[]} names
 * @return {string[]}
 */
var getFolderNames = function(names) {
  const map = new Map();
  const result = [];

  for (const name of names) {
    if (!map.has(name)) {
      result.push(name);
      map.set(name, 1);
    } else {
      let suffix = map.get(name);
      let newName = `${name}(${suffix})`;
      while (map.has(newName)) {
        suffix++;
        newName = `${name}(${suffix})`;
      }
      result.push(newName);
      map.set(name, suffix + 1);
      map.set(newName, 1);
    }
  }

  return result;
};
```

1488 Avoid Flood In The City

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1488. Avoid Flood in The City
 * https://leetcode.com/problems/avoid-flood-in-the-city/
 * Difficulty: Medium
 *
 * Your country has an infinite number of lakes. Initially, all the lakes are empty, but when
it
 * rains over the nth lake, the nth lake becomes full of water. If it rains over a lake that
is
 * full of water, there will be a flood. Your goal is to avoid floods in any lake.
 *
 * Given an integer array rains where:
 * - rains[i] > 0 means there will be rains over the rains[i] lake.
 * - rains[i] == 0 means there are no rains this day and you can choose one lake this day and
 *   dry it.
 *
 * Return an array ans where:
 * - ans.length == rains.length
 * - ans[i] == -1 if rains[i] > 0.
 * - ans[i] is the lake you choose to dry in the ith day if rains[i] == 0.
 *
 * If there are multiple valid answers return any of them. If it is impossible to avoid flood
return
 * an empty array.
 *
 * Notice that if you chose to dry a full lake, it becomes empty, but if you chose to dry an
```

```
empty
 * lake, nothing changes.
 */

/**
 * @param {number[]} rains
 * @return {number[]}
 */
var avoidFlood = function(rains) {
  const fullLakes = new Map();
  const dryDays = [];
  const result = new Array(rains.length).fill(-1);

  for (let i = 0; i < rains.length; i++) {
    if (rains[i] === 0) {
      dryDays.push(i);
    } else {
      if (fullLakes.has(rains[i])) {
        const lastRain = fullLakes.get(rains[i]);
        let found = false;
        for (let j = 0; j < dryDays.length; j++) {
          if (dryDays[j] > lastRain) {
            result[dryDays[j]] = rains[i];
            dryDays.splice(j, 1);
            found = true;
            break;
          }
        }
        if (!found) return [];
      }
      fullLakes.set(rains[i], i);
    }
  }

  for (const dryDay of dryDays) {
    result[dryDay] = 1;
  }

  return result;
};
```

1489 Find Critical And Pseudo Critical Edges In Minimum Spanning Tree

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1489. Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree
 * https://leetcode.com/problems/find-critical-and-pseudo-critical-edges-in-minimum-spanning-t
ree/
 * Difficulty: Hard
 *
 * Given a weighted undirected connected graph with n vertices numbered from 0 to n - 1, and
an
 * array edges where edges[i] = [ai, bi, weighti] represents a bidirectional and weighted edge
 * between nodes ai and bi. A minimum spanning tree (MST) is a subset of the graph's edges
that
 * connects all vertices without cycles and with the minimum possible total edge weight.
 *
 * Find all the critical and pseudo-critical edges in the given graph's minimum spanning tree
 * (MST). An MST edge whose deletion from the graph would cause the MST weight to increase is
 * called a critical edge. On the other hand, a pseudo-critical edge is that which can appear
 * in some MSTs but not all.
 *
 * Note that you can return the indices of the edges in any order.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[][]}
```

```
 */
var findCriticalAndPseudoCriticalEdges = function(n, edges) {
  const indexedEdges = edges.map((edge, i) => [...edge, i]);
  indexedEdges.sort((a, b) => a[2] - b[2]);

  const minWeight = getMSTWeight();
  const critical = [];
  const pseudoCritical = [];

  for (let i = 0; i < edges.length; i++) {
    if (getMSTWeight(i) > minWeight) {
      critical.push(indexedEdges[i][3]);
    } else if (getMSTWeight(-1, i) === minWeight) {
      pseudoCritical.push(indexedEdges[i][3]);
    }
  }

  return [critical, pseudoCritical];

  function findParent(parents, x) {
    if (parents[x] !== x) parents[x] = findParent(parents, parents[x]);
    return parents[x];
  }

  function union(parents, ranks, x, y) {
    const px = findParent(parents, x);
    const py = findParent(parents, y);
    if (px === py) return false;
    if (ranks[px] < ranks[py]) {
      parents[px] = py;
    } else if (ranks[px] > ranks[py]) {
      parents[py] = px;
    } else {
      parents[py] = px;
      ranks[px]++;
    }
    return true;
  }

  function getMSTWeight(exclude = -1, include = -1) {
    const parents = Array.from({ length: n }, (_, i) => i);
    const ranks = new Array(n).fill(0);
    let weight = 0;
    let edgesUsed = 0;

    if (include !== -1) {
      const [u, v, w] = indexedEdges[include];
      if (union(parents, ranks, u, v)) {
        weight += w;
        edgesUsed++;
      }
    }

    for (let i = 0; i < indexedEdges.length; i++) {
      if (i === exclude || i === include) continue;
      const [u, v, w] = indexedEdges[i];
      if (union(parents, ranks, u, v)) {
        weight += w;
        edgesUsed++;
      }
    }

    return edgesUsed === n - 1 ? weight : Infinity;
  }
};
```

1490 Clone N Ary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1490. Clone N-ary Tree
 * https://leetcode.com/problems/clone-n-ary-tree/
 * Difficulty: Medium
 *
 * Given a root of an N-ary tree, return a deep copy (clone) of the tree.
 *
 * Each node in the n-ary tree contains a val (int) and a list (List[Node]) of its children.
 *
 * class Node {
 *     public int val;
 *     public List<Node> children;
 * }
 *
 * Nary-Tree input serialization is represented in their level order traversal, each group
 * of children is separated by the null value (See examples).
 */

/**
 * // Definition for a _Node.
 * function _Node(val, children) {
 *    this.val = val === undefined ? 0 : val;
 *    this.children = children === undefined ? [] : children;
 * };
 */

/**
 * @param {_Node|null} node
 * @return {_Node|null}
 */
var cloneTree = function(root) {
  if (!root) return null;

  const clonedNode = new _Node(root.val);
  clonedNode.children = root.children.map(child => cloneTree(child));

  return clonedNode;
};
```

1491 Average Salary Excluding The Minimum And Maximum Salary

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1491. Average Salary Excluding the Minimum and Maximum Salary
 * https://leetcode.com/problems/average-salary-excluding-the-minimum-and-maximum-salary/
 * Difficulty: Easy
 *
 * Given an array of unique integers salary where salary[i] is the salary of the employee i.
 *
 * Return the average salary of employees excluding the minimum and maximum salary.
 */

/**
 * @param {number[]} salary
 * @return {number}
 */
var average = function(salary) {
  salary.sort((a, b) => a - b);
  salary.shift();
  salary.pop();

  return salary.reduce((total, n) => total + n, 0) / salary.length;
};
```

1492 The Kth Factor Of N

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1492. The kth Factor of n
 * https://leetcode.com/problems/the-kth-factor-of-n/
 * Difficulty: Medium
 *
 * Given two positive integers n and k.
 *
 * A factor of an integer n is defined as an integer i where n % i == 0.
 *
 * Consider a list of all factors of n sorted in ascending order, return
 * the kth factor in this list or return -1 if n has less than k factors.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var kthFactor = function(n, k) {
  for (let i = 1; i <= n; i++) {
    if (n % i === 0) {
      k--;
    }
    if (!k) return i;
  }
  return -1;
};
```

1493 Longest Subarray Of 1S After Deleting One Element

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1493. Longest Subarray of 1's After Deleting One Element
 * https://leetcode.com/problems/longest-subarray-of-1s-after-deleting-one-element/
 * Difficulty: Medium
 *
 * Given a binary array nums, you should delete one element from it.
 *
 * Return the size of the longest non-empty subarray containing only 1's in the
 * resulting array. Return 0 if there is no such subarray.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var longestSubarray = function(nums) {
  const grouped = nums.join('').split('0').map(s => s.length);
  let max = 0;

  for (let i = 0; i < grouped.length; i++) {
    max = Math.max(
      max,
      (grouped[i - 1] ?? 0) + grouped[i],
      grouped[i] + (grouped[i + 1] ?? 0)
    );
  }

  return max - (nums.includes(0) ? 0 : 1);
};
```

1494 Parallel Courses Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1494. Parallel Courses II
 * https://leetcode.com/problems/parallel-courses-ii/
 * Difficulty: Hard
 *
 * You are given an integer n, which indicates that there are n courses labeled from 1 to n.
You are
 * also given an array relations where relations[i] = [prevCoursei, nextCoursei], representing
 a
 * prerequisite relationship between course prevCoursei and course nextCoursei: course
prevCoursei
 * has to be taken before course nextCoursei. Also, you are given the integer k.
 *
 * In one semester, you can take at most k courses as long as you have taken all the
prerequisites
 * in the previous semesters for the courses you are taking.
 *
 * Return the minimum number of semesters needed to take all courses. The testcases will be
 * generated such that it is possible to take every course.
 */

/**
 * @param {number} n
 * @param {number[][]} relations
 * @param {number} k
 * @return {number}
 */
var minNumberOfSemesters = function(n, relations, k) {
  const prerequisites = new Array(n).fill(0);
  for (const [prev, next] of relations) {
    prerequisites[next - 1] |= 1 << (prev - 1);
  }

  const cache = new Array(1 << n).fill(-1);

  return findMinSemesters(0);

  function findMinSemesters(courses) {
    if (courses === (1 << n) - 1) return 0;
    if (cache[courses] !== -1) return cache[courses];

    let available = 0;
    for (let i = 0; i < n; i++) {
      if (!(courses & (1 << i)) && (courses & prerequisites[i]) === prerequisites[i]) {
        available |= 1 << i;
      }
    }

    let minSemesters = n + 1;
    const combinations = function(pos, count, selected) {
      if (count === 0 || pos === n) {
        if (selected) {
          minSemesters = Math.min(
            minSemesters,
            1 + findMinSemesters(courses | selected)
          );
        }
        return;
      }
      if (!(available & (1 << pos))) return combinations(pos + 1, count, selected);
      combinations(pos + 1, count - 1, selected | (1 << pos));
      combinations(pos + 1, count, selected);
    };

    combinations(0, Math.min(k, bitCount(available)), 0);
    cache[courses] = minSemesters;
```

```
    return minSemesters;
  }

  function bitCount(num) {
    let count = 0;
    while (num) {
      count += num & 1;
      num >>= 1;
    }
    return count;
  }
};
```

1496 Path Crossing

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1496. Path Crossing
 * https://leetcode.com/problems/path-crossing/
 * Difficulty: Easy
 *
 * Given a string path, where path[i] = 'N', 'S', 'E' or 'W', each representing
 * moving one unit north, south, east, or west, respectively. You start at the
 * origin (0, 0) on a 2D plane and walk on the path specified by path.
 *
 * Return true if the path crosses itself at any point, that is, if at any time
 * you are on a location you have previously visited. Return false otherwise.
 */

/**
 * @param {string} path
 * @return {boolean}
 */
var isPathCrossing = function(path) {
  let x = 0, y = 0;
  const toKey = (a, b) => `${a},${b}`;
  const history = new Set([toKey(x, y)]);

  for (let direction of path) {
    switch (direction) {
      case 'N': x++; break;
      case 'W': y--; break;
      case 'S': x--; break;
      case 'E': y++; break;
    }
    if (history.has(toKey(x, y))) {
      return true;
    }
    history.add(toKey(x, y));
  }

  return false;
};
```

1497 Check If Array Pairs Are Divisible By K

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1497. Check If Array Pairs Are Divisible by k
 * https://leetcode.com/problems/check-if-array-pairs-are-divisible-by-k/
 * Difficulty: Medium
 *
 * Given an array of integers arr of even length n and an integer k.
```

```
 *
 * We want to divide the array into exactly n / 2 pairs such that the sum of each pair is
 * divisible by k.
 *
 * Return true If you can find a way to do that or false otherwise.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {boolean}
 */
var canArrange = function(arr, k) {
  const remainders = new Array(k).fill(0);

  for (const num of arr) {
    const remainder = ((num % k) + k) % k;
    remainders[remainder]++;
  }

  for (let i = 0; i <= Math.floor(k / 2); i++) {
    if (i === 0 || i * 2 === k) {
      if (remainders[i] % 2 !== 0) return false;
    } else if (remainders[i] !== remainders[k - i]) {
      return false;
    }
  }

  return true;
};
```

1498 Number Of Subsequences That Satisfy The Given Sum Condition

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1498. Number of Subsequences That Satisfy the Given Sum Condition
 * https://leetcode.com/problems/number-of-subsequences-that-satisfy-the-given-sum-condition/
 * Difficulty: Medium
 *
 * You are given an array of integers nums and an integer target.
 *
 * Return the number of non-empty subsequences of nums such that the sum of the minimum and
 * maximum element on it is less or equal to target. Since the answer may be too large,
 * return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var numSubseq = function(nums, target) {
  const MOD = 1e9 + 7;
  const sortedNums = nums.sort((a, b) => a - b);
  const length = nums.length;
  let result = 0;
  const powers = new Array(length).fill(1);

  for (let i = 1; i < length; i++) {
    powers[i] = (powers[i - 1] * 2) % MOD;
  }

  let left = 0;
  let right = length - 1;

  while (left <= right) {
    if (sortedNums[left] + sortedNums[right] <= target) {
      result = (result + powers[right - left]) % MOD;
```

```
      left++;
    } else {
      right--;
    }
  }

  return result;
};
```

1499 Max Value Of Equation

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1499. Max Value of Equation
 * https://leetcode.com/problems/max-value-of-equation/
 * Difficulty: Hard
 *
 * You are given an array points containing the coordinates of points on a 2D plane,
 * sorted by the x-values, where points[i] = [xi, yi] such that xi < xj for all
 * 1 <= i < j <= points.length. You are also given an integer k.
 *
 * Return the maximum value of the equation yi + yj + |xi - xj| where |xi - xj| <= k
 * and 1 <= i < j <= points.length.
 *
 * It is guaranteed that there exists at least one pair of points that satisfy the
 * constraint |xi - xj| <= k.
 */

/**
 * @param {number[][]} points
 * @param {number} k
 * @return {number}
 */
var findMaxValueOfEquation = function(points, k) {
  let result = -Infinity;
  const q = [];

  for (let j = 0; j < points.length; j++) {
    const [xj, yj] = points[j];

    while (q.length && xj - points[q[0]][0] > k) {
      q.shift();
    }

    if (q.length) {
      const [xi, yi] = points[q[0]];
      result = Math.max(result, yi + yj + xj - xi);
    }

    while (q.length && yj - xj >= points[q[q.length - 1]][1] - points[q[q.length - 1]][0]) {
      q.pop();
    }

    q.push(j);
  }

  return result;
};
```

1500 Design A File Sharing System

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 1500. Design a File Sharing System
 * https://leetcode.com/problems/design-a-file-sharing-system/
 * Difficulty: Medium
 *
 * We will use a file-sharing system to share a very large file which consists of m small
 * chunks with IDs from 1 to m.
 *
 * When users join the system, the system should assign a unique ID to them. The unique ID
 * should be used once for each user, but when a user leaves the system, the ID can be reused
again.
 *
 * Users can request a certain chunk of the file, the system should return a list of IDs of
 * all the users who own this chunk. If the user receives a non-empty list of IDs, they
receive
 * the requested chunk successfully.
 *
 * Implement the FileSharing class:
 * - FileSharing(int m) Initializes the object with a file of m chunks.
 * - int join(int[] ownedChunks): A new user joined the system owning some chunks of the file,
 *   the system should assign an id to the user which is the smallest positive integer not
 *   taken by any other user. Return the assigned id.
 * - void leave(int userID): The user with userID will leave the system, you cannot take file
 *   chunks from them anymore.
 * - int[] request(int userID, int chunkID): The user userID requested the file chunk with
chunkID.
 *   Return a list of the IDs of all users that own this chunk sorted in ascending order.
 */

/**
* @param {number} m
*/
var FileSharing = function(m) {
  this.chunkToUsers = new Array(m + 1).fill().map(() => new Set());
  this.userToChunks = new Map();
  this.availableIds = [];
  this.nextId = 1;
};

/**
* @param {number[]} ownedChunks
* @return {number}
*/
FileSharing.prototype.join = function(ownedChunks) {
  let userId;

  if (this.availableIds.length > 0) {
    userId = this.availableIds.sort((a, b) => a - b).shift();
  } else {
    userId = this.nextId++;
  }

  this.userToChunks.set(userId, new Set(ownedChunks));

  for (const chunk of ownedChunks) {
    this.chunkToUsers[chunk].add(userId);
  }

  return userId;
};

/**
* @param {number} userID
* @return {void}
*/
FileSharing.prototype.leave = function(userID) {
  const userChunks = this.userToChunks.get(userID);

  if (userChunks) {
    for (const chunk of userChunks) {
      this.chunkToUsers[chunk].delete(userID);
    }
    this.userToChunks.delete(userID);
    this.availableIds.push(userID);
  }
};
```

```
/**
* @param {number} userID
* @param {number} chunkID
* @return {number[]}
*/
FileSharing.prototype.request = function(userID, chunkID) {
  const usersWithChunk = Array.from(this.chunkToUsers[chunkID]).sort((a, b) => a - b);

  if (usersWithChunk.length > 0) {
    this.userToChunks.get(userID).add(chunkID);
    this.chunkToUsers[chunkID].add(userID);
  }

  return usersWithChunk;
};
```

1502 Can Make Arithmetic Progression From Sequence

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1502. Can Make Arithmetic Progression From Sequence
 * https://leetcode.com/problems/can-make-arithmetic-progression-from-sequence/
 * Difficulty: Easy
 *
 * A sequence of numbers is called an arithmetic progression if the difference
 * between any two consecutive elements is the same.
 *
 * Given an array of numbers arr, return true if the array can be rearranged to
 * form an arithmetic progression. Otherwise, return false.
 */

/**
 * @param {number[]} arr
 * @return {boolean}
 */
var canMakeArithmeticProgression = function(arr) {
  arr.sort((a, b) => a - b);

  for (let i = 2, diff = arr[1] - arr[0]; i < arr.length; i++) {
    if (arr[i] - arr[i - 1] !== diff) {
      return false;
    }
  }

  return true;
};
```

1503 Last Moment Before All Ants Fall Out Of A Plank

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1503. Last Moment Before All Ants Fall Out of a Plank
 * https://leetcode.com/problems/last-moment-before-all-ants-fall-out-of-a-plank/
 * Difficulty: Medium
 *
 * We have a wooden plank of the length n units. Some ants are walking on the plank, each ant
moves
 * with a speed of 1 unit per second. Some of the ants move to the left, the other move to the
 * right.
 *
 * When two ants moving in two different directions meet at some point, they change their
directions
```

```
 * and continue moving again. Assume changing directions does not take any additional time.
 *
 * When an ant reaches one end of the plank at a time t, it falls out of the plank
immediately.
 *
 * Given an integer n and two integer arrays left and right, the positions of the ants moving
to the
 * left and the right, return the moment when the last ant(s) fall out of the plank.
 */

/**
 * @param {number} n
 * @param {number[]} left
 * @param {number[]} right
 * @return {number}
 */
var getLastMoment = function(n, left, right) {
  let result = 0;

  for (const pos of left) {
    result = Math.max(result, pos);
  }
  for (const pos of right) {
    result = Math.max(result, n - pos);
  }

  return result;
};
```

1504 Count Submatrices With All Ones

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1504. Count Submatrices With All Ones
 * https://leetcode.com/problems/count-submatrices-with-all-ones/
 * Difficulty: Medium
 *
 * Given an m x n binary matrix mat, return the number of submatrices that have all ones.
 */

/**
 * @param {number[][]} mat
 * @return {number}
 */
var numSubmat = function(mat) {
  const rows = mat.length;
  const cols = mat[0].length;
  const heights = new Array(cols).fill(0);
  let result = 0;

  for (let i = 0; i < rows; i++) {
    const stack = [];
    for (let j = 0; j < cols; j++) {
      heights[j] = mat[i][j] === 1 ? heights[j] + 1 : 0;

      while (stack.length && heights[j] <= heights[stack[stack.length - 1]]) {
        stack.pop();
      }

      stack.push(j);

      for (let k = 0; k < stack.length; k++) {
        const height = heights[stack[k]];
        const width = k === 0 ? stack[k] + 1 : stack[k] - stack[k - 1];
        result += height * width;
      }
    }
  }
```

```
  return result;
};
```

1505 Minimum Possible Integer After At Most K Adjacent Swaps On Digits

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1505. Minimum Possible Integer After at Most K Adjacent Swaps On Digits
 * https://leetcode.com/problems/minimum-possible-integer-after-at-most-k-adjacent-swaps-on-di
gits/
 * Difficulty: Hard
 *
 * You are given a string num representing the digits of a very large integer and an integer
k. You
 * are allowed to swap any two adjacent digits of the integer at most k times.
 *
 * Return the minimum integer you can obtain also as a string.
 */

/**
 * @param {string} num
 * @param {number} k
 * @return {string}
 */
var minInteger = function(num, k) {
  const digits = num.split('');
  const n = digits.length;
  const fenwick = new Array(n + 1).fill(0);
  const digitQueues = Array.from({ length: 10 }, () => []);
  const result = [];

  for (let i = 0; i < n; i++) {
    digitQueues[digits[i]].push(i);
  }

  for (let i = 0; i < n; i++) {
    for (let digit = 0; digit <= 9; digit++) {
      if (digitQueues[digit].length === 0) continue;
      const pos = digitQueues[digit][0];
      const swapsNeeded = pos - fetchValue(pos);
      if (swapsNeeded <= k) {
        k -= swapsNeeded;
        result.push(digit);
        update(pos);
        digitQueues[digit].shift();
        break;
      }
    }
  }

  return result.join('');

  function update(index) {
    for (let i = index + 1; i <= n; i += i & -i) {
      fenwick[i]++;
    }
  }

  function fetchValue(index) {
    let sum = 0;
    for (let i = index + 1; i > 0; i -= i & -i) {
      sum += fenwick[i];
    }
    return sum;
  }
};
```

1506 Find Root Of N Ary Tree

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1506. Find Root of N-Ary Tree
 * https://leetcode.com/problems/find-root-of-n-ary-tree/
 * Difficulty: Medium
 *
 * You are given all the nodes of an N-ary tree as an array of Node objects, where each
 * node has a unique value.
 *
 * Return the root of the N-ary tree.
 *
 * Custom testing:
 * - An N-ary tree can be serialized as represented in its level order traversal where each
 *   group of children is separated by the null value (see examples).
 */

/**
 * // Definition for a _Node.
 * function _Node(val, children) {
 *    this.val = val === undefined ? 0 : val;
 *    this.children = children === undefined ? [] : children;
 * };
 */

/**
 * @param {_Node[]} tree
 * @return {_Node}
 */
var findRoot = function(tree) {
  const set = new Set();

  for (const node of tree) {
    for (const child of node.children) {
      set.add(child.val);
    }
  }

  for (const node of tree) {
    if (!set.has(node.val)) {
      return node;
    }
  }
};
```

1507 Reformat Date

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1507. Reformat Date
 * https://leetcode.com/problems/reformat-date/
 * Difficulty: Easy
 *
 * Given a date string in the form Day Month Year, where:
 * - Day is in the set {"1st", "2nd", "3rd", "4th", ..., "30th", "31st"}.
 * - Month is in the set {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep",
"Oct", "Nov", "Dec"}.
 * - Year is in the range [1900, 2100].
 *
 * Convert the date string to the format YYYY-MM-DD, where:
 * - YYYY denotes the 4 digit year.
 * - MM denotes the 2 digit month.
```

```
 * - DD denotes the 2 digit day.
 */

/**
 * @param {string} date
 * @return {string}
 */
var reformatDate = function(date) {
  const [_, day, month, year] = date.match(/(\d+)\w+\s+(\w+)\s+(\d+)/);
  return new Date(`${day} ${month} ${year}`).toISOString().split('T')[0];
};
```

1508 Range Sum Of Sorted Subarray Sums

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1508. Range Sum of Sorted Subarray Sums
 * https://leetcode.com/problems/range-sum-of-sorted-subarray-sums/
 * Difficulty: Medium
 *
 * You are given the array nums consisting of n positive integers. You computed the sum of all
 * non-empty continuous subarrays from the array and then sorted them in non-decreasing order,
 * creating a new array of n * (n + 1) / 2 numbers.
 *
 * Return the sum of the numbers from index left to index right (indexed from 1), inclusive,
 * in the new array. Since the answer can be a huge number return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @param {number} n
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
var rangeSum = function(nums, n, left, right) {
  const MOD = 1e9 + 7;
  const sums = [];

  for (let start = 0; start < n; start++) {
    let currentSum = 0;
    for (let end = start; end < n; end++) {
      currentSum += nums[end];
      sums.push(currentSum);
    }
  }

  sums.sort((a, b) => a - b);

  let result = 0;
  for (let i = left - 1; i < right; i++) {
    result = (result + sums[i]) % MOD;
  }

  return result;
};
```

1509 Minimum Difference Between Largest And Smallest Value In Three Mov

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1509. Minimum Difference Between Largest and Smallest Value in Three Moves
 * https://leetcode.com/problems/minimum-difference-between-largest-and-smallest-value-in-thre
```

```
e-moves/
 * Difficulty: Medium
 *
 * You are given an integer array nums.
 *
 * In one move, you can choose one element of nums and change it to any value.
 *
 * Return the minimum difference between the largest and smallest value of nums after
performing at
 * most three moves.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minDifference = function(nums) {
  if (nums.length <= 4) return 0;

  nums.sort((a, b) => a - b);

  const n = nums.length;
  let result = Infinity;

  result = Math.min(result, nums[n - 1] - nums[3]);
  result = Math.min(result, nums[n - 2] - nums[2]);
  result = Math.min(result, nums[n - 3] - nums[1]);
  result = Math.min(result, nums[n - 4] - nums[0]);

  return result;
};
```

1510 Stone Game Iv

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1510. Stone Game IV
 * https://leetcode.com/problems/stone-game-iv/
 * Difficulty: Hard
 *
 * Alice and Bob take turns playing a game, with Alice starting first.
 *
 * Initially, there are n stones in a pile. On each player's turn, that player makes a move
 * consisting of removing any non-zero square number of stones in the pile.
 *
 * Also, if a player cannot make a move, he/she loses the game.
 *
 * Given a positive integer n, return true if and only if Alice wins the game otherwise return
 * false, assuming both players play optimally.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var winnerSquareGame = function(n) {
  const dp = new Array(n + 1).fill(false);

  for (let stones = 1; stones <= n; stones++) {
    for (let square = 1; square * square <= stones; square++) {
      if (!dp[stones - square * square]) {
        dp[stones] = true;
        break;
      }
    }
  }

  return dp[n];
};
```

1512 Number Of Good Pairs

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1512. Number of Good Pairs
 * https://leetcode.com/problems/number-of-good-pairs/
 * Difficulty: Easy
 *
 * Given an array of integers nums, return the number of good pairs.
 *
 * A pair (i, j) is called good if nums[i] == nums[j] and i < j.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var numIdenticalPairs = function(nums) {
  const map = new Array(101).fill(0);
  return nums.reduce((sum, n) => sum += map[n]++, 0);
};
```

1513 Number Of Substrings With Only 1S

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1513. Number of Substrings With Only 1s
 * https://leetcode.com/problems/number-of-substrings-with-only-1s/
 * Difficulty: Medium
 *
 * Given a binary string s, return the number of substrings with all characters 1's. Since the
 * answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numSub = function(s) {
  const MOD = 1e9 + 7;
  let result = 0;
  let consecutiveOnes = 0;

  for (const char of s) {
    if (char === '1') {
      consecutiveOnes++;
      result = (result + consecutiveOnes) % MOD;
    } else {
      consecutiveOnes = 0;
    }
  }

  return result;
};
```

1514 Path With Maximum Probability

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1514. Path with Maximum Probability
 * https://leetcode.com/problems/path-with-maximum-probability/
 * Difficulty: Medium
 *
 * You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge
list
 * where edges[i] = [a, b] is an undirected edge connecting the nodes a and b with a
probability
 * of success of traversing that edge succProb[i].
 *
 * Given two nodes start and end, find the path with the maximum probability of success to go
from
 * start to end and return its success probability.
 *
 * If there is no path from start to end, return 0. Your answer will be accepted if it differs
 from
 * the correct answer by at most 1e-5.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[]} succProb
 * @param {number} startNode
 * @param {number} endNode
 * @return {number}
 */
var maxProbability = function(n, edges, succProb, startNode, endNode) {
  const maxProbs = new Array(n).fill(0);
  maxProbs[startNode] = 1;

  const graph = Array.from({ length: n }, () => []);
  edges.forEach(([a, b], i) => {
    graph[a].push([b, succProb[i]]);
    graph[b].push([a, succProb[i]]);
  });

  const queue = [startNode];
  while (queue.length) {
    const current = queue.shift();
    for (const [next, prob] of graph[current]) {
      const newProb = maxProbs[current] * prob;
      if (newProb > maxProbs[next]) {
        maxProbs[next] = newProb;
        queue.push(next);
      }
    }
  }

  return maxProbs[endNode];
};
```

1515 Best Position For A Service Centre

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1515. Best Position for a Service Centre
 * https://leetcode.com/problems/best-position-for-a-service-centre/
 * Difficulty: Hard
 *
 * A delivery company wants to build a new service center in a new city. The company knows the
 * positions of all the customers in this city on a 2D-Map and wants to build the new center
in
 * a position such that the sum of the euclidean distances to all customers is minimum.
 *
 * Given an array positions where positions[i] = [xi, yi] is the position of the ith customer
on
```

```
 * the map, return the minimum sum of the euclidean distances to all customers.
 *
 * In other words, you need to choose the position of the service center [xcentre, ycentre]
such
 * that the following formula is minimized:
 * - Answers within 10-5 of the actual value will be accepted.
 */

/**
 * @param {number[][]} positions
 * @return {number}
 */
var getMinDistSum = function(positions) {
  const n = positions.length;
  let xSum = 0;
  let ySum = 0;

  for (const [x, y] of positions) {
    xSum += x;
    ySum += y;
  }

  let xCenter = xSum / n;
  let yCenter = ySum / n;
  const epsilon = 1e-7;
  const maxIterations = 1000;

  for (let iter = 0; iter < maxIterations; iter++) {
    let xNumerator = 0;
    let yNumerator = 0;
    let denominator = 0;

    for (const [x, y] of positions) {
      const dx = xCenter - x;
      const dy = yCenter - y;
      const dist = Math.sqrt(dx * dx + dy * dy);
      if (dist < epsilon) continue;
      const weight = 1 / dist;
      xNumerator += x * weight;
      yNumerator += y * weight;
      denominator += weight;
    }

    if (denominator < epsilon) break;

    const newX = xNumerator / denominator;
    const newY = yNumerator / denominator;

    if (Math.abs(newX - xCenter) < epsilon && Math.abs(newY - yCenter) < epsilon) {
      break;
    }

    xCenter = newX;
    yCenter = newY;
  }

  let sumDist = 0;
  for (const [x, y] of positions) {
    sumDist += Math.sqrt((xCenter - x) ** 2 + (yCenter - y) ** 2);
  }

  return sumDist;
};
```

1516 Move Sub Tree Of N Ary Tree

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1516. Move Sub-Tree of N-Ary Tree
```

```
 * https://leetcode.com/problems/move-sub-tree-of-n-ary-tree/
 * Difficulty: Hard
 *
 * Given the root of an N-ary tree of unique values, and two nodes of the tree p and q.
 *
 * You should move the subtree of the node p to become a direct child of node q. If p is
 * already a direct child of q, do not change anything. Node p must be the last child in
 * the children list of node q.
 *
 * Return the root of the tree after adjusting it.
 *
 * There are 3 cases for nodes p and q:
 * 1. Node q is in the sub-tree of node p.
 * 2. Node p is in the sub-tree of node q.
 * 3. Neither node p is in the sub-tree of node q nor node q is in the sub-tree of node p.
 *
 * In cases 2 and 3, you just need to move p (with its sub-tree) to be a child of q, but in
 * case 1 the tree may be disconnected, thus you need to reconnect the tree again. Please
 * read the examples carefully before solving this problem.
 *
 * Nary-Tree input serialization is represented in their level order traversal, each group
 * of children is separated by the null value (See examples).
 */

/**
 * // Definition for a Node.
 * function Node(val, children) {
 *    this.val = val === undefined ? 0 : val;
 *    this.children = children === undefined ? [] : children;
 * };
 */

/**
 * @param {Node} root
 * @param {Node} p
 * @param {Node} q
 * @return {Node}
 */
var moveSubTree = function(root, p, q) {
  if (q.children.includes(p)) return root;

  const pParent = findParent(root, p);
  const qParent = findParent(root, q);

  if (isDescendant(q, p)) {
    qParent.children = qParent.children.filter(child => child !== q);
    q.children.push(p);

    if (!pParent) return q;

    pParent.children[pParent.children.indexOf(p)] = q;
  } else {
    q.children.push(p);
    pParent.children = pParent.children.filter(child => child !== p);
  }

  return root;

  function findParent(node, target) {
    if (!node) return null;
    for (const child of node.children) {
      if (child === target) return node;
      const parent = findParent(child, target);
      if (parent) return parent;
    }
    return null;
  }

  function isDescendant(target, node) {
    if (!node) return false;
    if (node === target) return true;
    return node.children.some(child => isDescendant(target, child));
  }
};
```

1518 Water Bottles

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1518. Water Bottles
 * https://leetcode.com/problems/water-bottles/
 * Difficulty: Easy
 *
 * There are numBottles water bottles that are initially full of water. You can exchange
numExchange
 * empty water bottles from the market with one full water bottle.
 *
 * The operation of drinking a full water bottle turns it into an empty bottle.
 *
 * Given the two integers numBottles and numExchange, return the maximum number of water
bottles you
 * can drink.
 */

/**
 * @param {number} numBottles
 * @param {number} numExchange
 * @return {number}
 */
var numWaterBottles = function(numBottles, numExchange) {
  let result = numBottles;
  let emptyBottles = numBottles;

  while (emptyBottles >= numExchange) {
    const newBottles = Math.floor(emptyBottles / numExchange);
    result += newBottles;
    emptyBottles = newBottles + (emptyBottles % numExchange);
  }

  return result;
};
```

1519 Number Of Nodes In The Sub Tree With The Same Label

MEDIUM

APPROACH NOTES

Look for breadth-first traversal and a queue.

SOURCE CODE

```
/**
 * 1519. Number of Nodes in the Sub-Tree With the Same Label
 * https://leetcode.com/problems/number-of-nodes-in-the-sub-tree-with-the-same-label/
 * Difficulty: Medium
 *
 * You are given a tree (i.e. a connected, undirected graph that has no cycles)
 * consisting of n nodes numbered from 0 to n - 1 and exactly n - 1 edges. The
 * root of the tree is the node 0, and each node of the tree has a label which
 * is a lower-case character given in the string labels (i.e. The node with the
 * number i has the label labels[i]).
 *
 * The edges array is given on the form edges[i] = [ai, bi], which means there
 * is an edge between nodes ai and bi in the tree.
 *
 * Return an array of size n where ans[i] is the number of nodes in the subtree
 * of the ith node which have the same label as node i.
 *
 * A subtree of a tree T is the tree consisting of a node in T and all of its
 * descendant nodes.
 */

/**
 * @param {number} n
```

```
 * @param {number[][]} edges
 * @param {string} labels
 * @return {number[]}
 */
var countSubTrees = function(n, edges, labels) {
  const lookup = Array.from(Array(n), () => []);
  const result = new Array(n).fill(0);

  edges.forEach(([x, y]) => {
    lookup[x].push(y);
    lookup[y].push(x);
  });

  function bfs(index, previous, chars = new Array(26).fill(0)) {
    const key = labels.charCodeAt(index) - 97;
    const count = chars[key];
    chars[key]++;
    lookup[index].forEach(i => {
      if (i !== previous) {
        bfs(i, index, chars);
      }
    });
    result[index] = chars[key] - count;
  }

  bfs(0, -1);

  return result;
};
```

1520 Maximum Number Of Non Overlapping Substrings

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1520. Maximum Number of Non-Overlapping Substrings
 * https://leetcode.com/problems/maximum-number-of-non-overlapping-substrings/
 * Difficulty: Hard
 *
 * Given a string s of lowercase letters, you need to find the maximum number of non-empty
 * substrings of s that meet the following conditions:
 * 1. The substrings do not overlap, that is for any two substrings s[i..j] and s[x..y],
 *    either j < x or i > y is true.
 * 2. A substring that contains a certain character c must also contain all occurrences of c.
 *
 * Find the maximum number of substrings that meet the above conditions. If there are multiple
 * solutions with the same number of substrings, return the one with minimum total length. It
 * can be shown that there exists a unique solution of minimum total length.
 *
 * Notice that you can return the substrings in any order.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var maxNumOfSubstrings = function(s) {
  const charRange = new Array(26).fill().map(() => [Infinity, -Infinity]);

  for (let i = 0; i < s.length; i++) {
    const charIndex = s.charCodeAt(i) - 97;
    charRange[charIndex][0] = Math.min(charRange[charIndex][0], i);
    charRange[charIndex][1] = Math.max(charRange[charIndex][1], i);
  }

  const intervals = [];
  for (let c = 0; c < 26; c++) {
    if (charRange[c][0] === Infinity) continue;
    const start = charRange[c][0];
    let end = charRange[c][1];
```

```
    let valid = true;

    for (let i = start; i <= end; i++) {
      const charIndex = s.charCodeAt(i) - 97;
      if (charRange[charIndex][0] < start) {
        valid = false;
        break;
      }
      end = Math.max(end, charRange[charIndex][1]);
    }

    if (valid) {
      intervals.push([start, end]);
    }
  }

  intervals.sort((a, b) => a[1] - b[1] || a[0] - b[0]);

  const result = [];
  let lastEnd = -1;

  for (const [start, end] of intervals) {
    if (start > lastEnd) {
      result.push(s.slice(start, end + 1));
      lastEnd = end;
    }
  }

  return result;
};
```

1521 Find A Value Of A Mysterious Function Closest To Target

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1521. Find a Value of a Mysterious Function Closest to Target
 * https://leetcode.com/problems/find-a-value-of-a-mysterious-function-closest-to-target/
 * Difficulty: Hard
 *
 * Winston was given the above mysterious function func. He has an integer array arr and an
integer
 * target and he wants to find the values l and r that make the value |func(arr, l, r) -
target|
 * minimum possible.
 *
 * Return the minimum possible value of |func(arr, l, r) - target|.
 *
 * Notice that func should be called with the values l and r where 0 <= l, r < arr.length.
 */

/**
 * @param {number[]} arr
 * @param {number} target
 * @return {number}
 */
var closestToTarget = function(arr, target) {
  let result = Infinity;
  let seen = new Set();

  for (let right = 0; right < arr.length; right++) {
    const current = new Set([arr[right]]);
    result = Math.min(result, Math.abs(arr[right] - target));

    for (const previous of seen) {
      const currentAnd = previous & arr[right];
      current.add(currentAnd);
      result = Math.min(result, Math.abs(currentAnd - target));
    }
```

```
    seen = current;
  }

  return result;
};
```

1522 Diameter Of N Ary Tree

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1522. Diameter of N-Ary Tree
 * https://leetcode.com/problems/diameter-of-n-ary-tree/
 * Difficulty: Medium
 *
 * Given a root of an N-ary tree, you need to compute the length of the diameter of the tree.
 *
 * The diameter of an N-ary tree is the length of the longest path between any two nodes in
 * the tree. This path may or may not pass through the root.
 *
 * (Nary-Tree input serialization is represented in their level order traversal, each group
 * of children is separated by the null value.)
 */

/**
 * // Definition for a _Node.
 * function _Node(val, children) {
 *    this.val = val === undefined ? 0 : val;
 *    this.children = children === undefined ? [] : children;
 * };
 */

/**
 * @param {_Node} root
 * @return {number}
 */
var diameter = function(root) {
  let maxDiameter = 0;
  dfs(root);
  return maxDiameter;

  function dfs(node) {
    if (!node) return 0;

    const depths = node.children.map(child => dfs(child)).sort((a, b) => b - a);

    const currentDiameter = depths.length >= 2 ? depths[0] + depths[1] : depths[0] || 0;
    maxDiameter = Math.max(maxDiameter, currentDiameter);

    return (depths[0] || 0) + 1;
  }
};
```

1523 Count Odd Numbers In An Interval Range

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1523. Count Odd Numbers in an Interval Range
 * https://leetcode.com/problems/count-odd-numbers-in-an-interval-range/
 * Difficulty: Easy
 *
 * Given two non-negative integers low and high. Return the count of odd numbers between low
 * and high (inclusive).
 */
```

```
/**
 * @param {number} low
 * @param {number} high
 * @return {number}
 */
var countOdds = function(low, high) {
  const range = high - low + 1;

  if (low % 2 !== 0 && high % 2 !== 0) {
    return Math.floor(range / 2) + 1;
  }

  return Math.floor(range / 2);
};
```

1524 Number Of Sub Arrays With Odd Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1524. Number of Sub-arrays With Odd Sum
 * https://leetcode.com/problems/number-of-sub-arrays-with-odd-sum/
 * Difficulty: Medium
 *
 * Given an array of integers arr, return the number of subarrays with an odd sum.
 *
 * Since the answer can be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var numOfSubarrays = function(arr) {
  let even = 0;
  let odd = 0;

  for (let i = 0, total = 0; i < arr.length; i++) {
    total += arr[i];
    odd += +(total % 2 === 1);
    even += +(total % 2 === 0);
  }

  return (odd * even + odd) % (1e9 + 7);
};
```

1525 Number Of Good Ways To Split A String

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1525. Number of Good Ways to Split a String
 * https://leetcode.com/problems/number-of-good-ways-to-split-a-string/
 * Difficulty: Medium
 *
 * You are given a string s.
 *
 * A split is called good if you can split s into two non-empty strings sleft and sright where
 * their concatenation is equal to s (i.e., sleft + sright = s) and the number of distinct
letters
 * in sleft and sright is the same.
 *
 * Return the number of good splits you can make in s.
 */
```

```
/**
 * @param {string} s
 * @return {number}
 */
var numSplits = function(s) {
  const leftDistinct = new Map();
  const rightDistinct = new Map();
  let result = 0;

  for (const char of s) {
    rightDistinct.set(char, (rightDistinct.get(char) || 0) + 1);
  }

  for (let i = 0; i < s.length - 1; i++) {
    const char = s[i];
    leftDistinct.set(char, (leftDistinct.get(char) || 0) + 1);

    if (rightDistinct.get(char) === 1) {
      rightDistinct.delete(char);
    } else {
      rightDistinct.set(char, rightDistinct.get(char) - 1);
    }

    if (leftDistinct.size === rightDistinct.size) {
      result++;
    }
  }

  return result;
};
```

1526 Minimum Number Of Increments On Subarrays To Form A Target Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1526. Minimum Number of Increments on Subarrays to Form a Target Array
 * https://leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-ar
ray/
 * Difficulty: Hard
 *
 * You are given an integer array target. You have an integer array initial of the same size
as
 * target with all elements initially zeros.
 *
 * In one operation you can choose any subarray from initial and increment each value by one.
 *
 * Return the minimum number of operations to form a target array from initial.
 *
 * The test cases are generated so that the answer fits in a 32-bit integer.
 */

/**
 * @param {number[]} target
 * @return {number}
 */
var minNumberOperations = function(target) {
  let operations = target[0];
  for (let i = 1; i < target.length; i++) {
    if (target[i] > target[i - 1]) {
      operations += target[i] - target[i - 1];
    }
  }
  return operations;
};
```

1528 Shuffle String

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1528. Shuffle String
 * https://leetcode.com/problems/shuffle-string/
 * Difficulty: Easy
 *
 * You are given a string s and an integer array indices of the same length.
 * The string s will be shuffled such that the character at the ith position
 * moves to indices[i] in the shuffled string.
 *
 * Return the shuffled string.
 */

/**
 * @param {string} s
 * @param {number[]} indices
 * @return {string}
 */
var restoreString = function(s, indices) {
  return indices.reduce((result, index, offset) => {
    result[index] = s[offset];
    return result;
  }, new Array(indices.length)).join('');
};
```

1529 Minimum Suffix Flips

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1529. Minimum Suffix Flips
 * https://leetcode.com/problems/minimum-suffix-flips/
 * Difficulty: Medium
 *
 * You are given a 0-indexed binary string target of length n. You have another binary string
s
 * of length n that is initially set to all zeros. You want to make s equal to target.
 *
 * In one operation, you can pick an index i where 0 <= i < n and flip all bits in the
inclusive
 * range [i, n - 1]. Flip means changing '0' to '1' and '1' to '0'.
 *
 * Return the minimum number of operations needed to make s equal to target.
 */

/**
 * @param {string} target
 * @return {number}
 */
var minFlips = function(target) {
  let current = '0';
  let result = 0;

  for (const bit of target) {
    if (bit !== current) {
      result++;
      current = bit;
    }
  }

  return result;
};
```

1530 Number Of Good Leaf Nodes Pairs

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1530. Number of Good Leaf Nodes Pairs
 * https://leetcode.com/problems/number-of-good-leaf-nodes-pairs/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree and an integer distance. A pair of two different
leaf
 * nodes of a binary tree is said to be good if the length of the shortest path between them
is
 * less than or equal to distance.
 *
 * Return the number of good leaf node pairs in the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} distance
 * @return {number}
 */
var countPairs = function(root, distance) {
  let result = 0;
  traverseTree(root);
  return result;

  function traverseTree(node) {
    if (!node) return [];
    if (!node.left && !node.right) return [1];

    const leftDistances = traverseTree(node.left);
    const rightDistances = traverseTree(node.right);

    for (const left of leftDistances) {
      for (const right of rightDistances) {
        if (left + right <= distance) result++;
      }
    }

    const allDistances = [];
    for (const dist of leftDistances.concat(rightDistances)) {
      if (dist + 1 <= distance) allDistances.push(dist + 1);
    }

    return allDistances;
  }
};
```

1531 String Compression Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1531. String Compression II
 * https://leetcode.com/problems/string-compression-ii/
```

```
 * Difficulty: Hard
 *
 * Run-length encoding is a string compression method that works by replacing consecutive
 * identical characters (repeated 2 or more times) with the concatenation of the character
 * and the number marking the count of the characters (length of the run). For example,
 * to compress the string "aabccc" we replace "aa" by "a2" and replace "ccc" by "c3". Thus
 * the compressed string becomes "a2bc3".
 *
 * Notice that in this problem, we are not adding '1' after single characters.
 *
 * Given a string s and an integer k. You need to delete at most k characters from s such that
 * the run-length encoded version of s has minimum length.
 *
 * Find the minimum length of the run-length encoded version of s after deleting at most
 * k characters.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var getLengthOfOptimalCompression = function(s, k) {
  const n = s.length;
  const dp = new Array(n + 1).fill().map(() => new Array(k + 1).fill(9999));
  dp[0][0] = 0;

  for (let i = 1; i <= n; i++) {
    for (let j = 0; j <= k; j++) {
      let count = 0;
      let deletions = 0;

      for (let m = i; m >= 1; m--) {
        if (s[m - 1] === s[i - 1]) count++;
        else deletions++;

        if (j - deletions >= 0) {
          dp[i][j] = Math.min(
            dp[i][j],
            dp[m - 1][j - deletions] + calculateEncodedLength(count)
          );
        }
      }

      if (j > 0) {
        dp[i][j] = Math.min(dp[i][j], dp[i - 1][j - 1]);
      }
    }
  }

  return dp[n][k];

  function calculateEncodedLength(count) {
    if (count === 0) return 0;
    if (count === 1) return 1;
    if (count < 10) return 2;
    if (count < 100) return 3;
    return 4;
  }
};
```

1533 Find The Index Of The Large Integer

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1533. Find the Index of the Large Integer
 * https://leetcode.com/problems/find-the-index-of-the-large-integer/
 * Difficulty: Medium
 *
```

```
 * We have an integer array arr, where all the integers in arr are equal except for one
integer
 * which is larger than the rest of the integers. You will not be given direct access to the
 * array, instead, you will have an API ArrayReader which have the following functions:
 * - int compareSub(int l, int r, int x, int y): where 0 <= l, r, x, y < ArrayReader.length(),
 *   l <= r and x <= y. The function compares the sum of sub-array arr[l..r] with the sum of
 *   the sub-array arr[x..y] and returns:
 *   - 1 if arr[l]+arr[l+1]+...+arr[r] > arr[x]+arr[x+1]+...+arr[y].
 *   - 0 if arr[l]+arr[l+1]+...+arr[r] == arr[x]+arr[x+1]+...+arr[y].
 *   - -1 if arr[l]+arr[l+1]+...+arr[r] < arr[x]+arr[x+1]+...+arr[y].
 * - int length(): Returns the size of the array.
 *
 * You are allowed to call compareSub() 20 times at most. You can assume both functions work
 * in O(1) time.
 *
 * Return the index of the array arr which has the largest integer.
 */

/**
 * @param {ArrayReader} reader
 * @return {number}
 */
var getIndex = function(reader) {
  const n = reader.length();
  let left = 0;
  let right = n - 1;

  while (left < right) {
    const length = right - left + 1;
    const mid = Math.floor((left + right) / 2);

    if (length % 2 === 1) {
      const comparison = reader.compareSub(left, mid - 1, mid + 1, right);
      if (comparison === 0) {
        return mid;
      } else if (comparison === 1) {
        right = mid - 1;
      } else {
        left = mid + 1;
      }
    } else {
      const comparison = reader.compareSub(left, mid, mid + 1, right);
      if (comparison === 1) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
  }

  return left;
};
```

1534 Count Good Triplets

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1534. Count Good Triplets
 * https://leetcode.com/problems/count-good-triplets/
 * Difficulty: Easy
 *
 * Given an array of integers arr, and three integers a, b and c. You need to find the number
 * of good triplets.
 *
 * A triplet (arr[i], arr[j], arr[k]) is good if the following conditions are true:
 * - 0 <= i < j < k < arr.length
 * - |arr[i] - arr[j]| <= a
 * - |arr[j] - arr[k]| <= b
 * - |arr[i] - arr[k]| <= c
```

```
 *
 * Where |x| denotes the absolute value of x.
 *
 * Return the number of good triplets.
 */

function countGoodTriplets(arr, a, b, c) {
  const length = arr.length;
  let result = 0;

  for (let i = 0; i < length - 2; i++) {
    for (let j = i + 1; j < length - 1; j++) {
      if (Math.abs(arr[i] - arr[j]) <= a) {
        for (let k = j + 1; k < length; k++) {
          if (Math.abs(arr[j] - arr[k]) <= b && Math.abs(arr[i] - arr[k]) <= c) {
            result++;
          }
        }
      }
    }
  }

  return result;
}
```

1535 Find The Winner Of An Array Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1535. Find the Winner of an Array Game
 * https://leetcode.com/problems/find-the-winner-of-an-array-game/
 * Difficulty: Medium
 *
 * Given an integer array arr of distinct integers and an integer k.
 *
 * A game will be played between the first two elements of the array (i.e. arr[0] and arr[1]).
 * In each round of the game, we compare arr[0] with arr[1], the larger integer wins and
 * remains at position 0, and the smaller integer moves to the end of the array. The game
 * ends when an integer wins k consecutive rounds.
 *
 * Return the integer which will win the game.
 *
 * It is guaranteed that there will be a winner of the game.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number}
 */
var getWinner = function(arr, k) {
  let result = arr[0];

  for (let i = 1, count = 0; i < arr.length && count < k; i++) {
    if (result < arr[i]) {
      result = arr[i];
      count = 0;
    }
    count++;
  }

  return result;
};
```

1536 Minimum Swaps To Arrange A Binary Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1536. Minimum Swaps to Arrange a Binary Grid
 * https://leetcode.com/problems/minimum-swaps-to-arrange-a-binary-grid/
 * Difficulty: Medium
 *
 * Given an n x n binary grid, in one step you can choose two adjacent rows of the grid and
 * swap them.
 *
 * A grid is said to be valid if all the cells above the main diagonal are zeros.
 *
 * Return the minimum number of steps needed to make the grid valid, or -1 if the grid cannot
 * be valid.
 *
 * The main diagonal of a grid is the diagonal that starts at cell (1, 1) and ends at cell (n,
 n).
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minSwaps = function(grid) {
  const n = grid.length;
  const trailingZeros = new Array(n).fill(0);

  for (let i = 0; i < n; i++) {
    trailingZeros[i] = countTrailingZeros(i);
  }

  let swaps = 0;
  for (let i = 0; i < n - 1; i++) {
    const requiredZeros = n - i - 1;
    let found = false;

    for (let j = i; j < n; j++) {
      if (trailingZeros[j] >= requiredZeros) {
        found = true;
        for (let k = j; k > i; k--) {
          [trailingZeros[k], trailingZeros[k - 1]] = [trailingZeros[k - 1], trailingZeros[k]];
          swaps++;
        }
        break;
      }
    }

    if (!found) return -1;
  }

  return swaps;

  function countTrailingZeros(row) {
    let count = 0;
    for (let j = n - 1; j >= 0; j--) {
      if (grid[row][j] === 0) count++;
      else break;
    }
    return count;
  }
};
```

1537 Get The Maximum Score

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1537. Get the Maximum Score
 * https://leetcode.com/problems/get-the-maximum-score/
 * Difficulty: Hard
 *
 * You are given two sorted arrays of distinct integers nums1 and nums2.
 *
 * A valid path is defined as follows:
 * - Choose array nums1 or nums2 to traverse (from index-0).
 * - Traverse the current array from left to right.
 * - If you are reading any value that is present in nums1 and nums2 you are allowed to change
 *   your path to the other array. (Only one repeated value is considered in the valid path).
 *
 * The score is defined as the sum of unique values in a valid path.
 *
 * Return the maximum score you can obtain of all possible valid paths. Since the answer may
be
 * too large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var maxSum = function(nums1, nums2) {
  const MOD = 1e9 + 7;
  const m = nums1.length;
  const n = nums2.length;
  let i = 0;
  let j = 0;
  let sum1 = 0;
  let sum2 = 0;

  while (i < m && j < n) {
    if (nums1[i] < nums2[j]) {
      sum1 += nums1[i++];
    } else if (nums1[i] > nums2[j]) {
      sum2 += nums2[j++];
    } else {
      sum1 = sum2 = Math.max(sum1, sum2) + nums1[i];
      i++;
      j++;
    }
  }

  while (i < m) {
    sum1 += nums1[i++];
  }

  while (j < n) {
    sum2 += nums2[j++];
  }

  return Math.max(sum1, sum2) % MOD;
};
```

1538 Guess The Majority In A Hidden Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1538. Guess the Majority in a Hidden Array
 * https://leetcode.com/problems/guess-the-majority-in-a-hidden-array/
 * Difficulty: Medium
 *
 * We have an integer array nums, where all the integers in nums are 0 or 1. You will not
 * be given direct access to the array, instead, you will have an API ArrayReader which
 * have the following functions:
 * - int query(int a, int b, int c, int d): where 0 <= a < b < c < d < ArrayReader.length().
 *   The function returns the distribution of the value of the 4 elements and returns:
 *   - 4 : if the values of the 4 elements are the same (0 or 1).
 *   - 2 : if three elements have a value equal to 0 and one element has value equal to 1 or
 *     vice versa.
 *   - 0 : if two element have a value equal to 0 and two elements have a value equal to 1.
 * - int length(): Returns the size of the array.
 *
 * You are allowed to call query() 2 * n times at most where n is equal to
ArrayReader.length().
 *
 * Return any index of the most frequent value in nums, in case of tie, return -1.
 */

/**
 * // This is the ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * function ArrayReader() {
 *     // Compares 4 different elements in the array
 *     // return 4 if the values of the 4 elements are the same (0 or 1).
 *     // return 2 if three elements have a value equal to 0 and one element has value equal
 *     // to 1 or vice versa.
 *     // return 0 : if two element have a value equal to 0 and two elements have a value
 *     // equal to 1.
 *     @param {number} a, b, c, d
 *     @return {number}
 *     this.query = function(a, b, c, d) {
 *         ...
 *     };
 *
 *     // Returns the length of the array
 *     @return {number}
 *     this.length = function() {
 *         ...
 *     };
 * };
 */

/**
 * @param {ArrayReader} reader
 * @return {number}
 */
var guessMajority = function(reader) {
  const baseQuery = reader.query(0, 1, 2, 3);
  const n = reader.length();
  let sameAsZeroCount = 1;
  let differentIndex = -1;

  for (let i = 4; i < n; i++) {
    const currentQuery = reader.query(1, 2, 3, i);
    if (baseQuery === currentQuery) {
      sameAsZeroCount++;
    } else {
      differentIndex = i;
    }
  }

  const referenceQuery = reader.query(1, 2, 3, 4);
  checkElement([0, 2, 3, 4], 1);
  checkElement([0, 1, 3, 4], 2);
  checkElement([0, 1, 2, 4], 3);

  const differentCount = n - sameAsZeroCount;
  if (sameAsZeroCount > differentCount) {
    return 0;
  } else if (sameAsZeroCount === differentCount) {
    return -1;
```

```
  } else {
    return differentIndex;
  }

  function checkElement(indices, elementIndex) {
    if (referenceQuery === reader.query(...indices)) {
      sameAsZeroCount++;
    } else {
      differentIndex = elementIndex;
    }
  }
};
```

1539 Kth Missing Positive Number

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1539. Kth Missing Positive Number
 * https://leetcode.com/problems/kth-missing-positive-number/
 * Difficulty: Easy
 *
 * Given an array arr of positive integers sorted in a strictly increasing order, and an
integer k.
 *
 * Return the kth positive integer that is missing from this array.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number}
 */
var findKthPositive = function(arr, k) {
  let left = 0;
  let right = arr.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    const count = arr[mid] - mid - 1;

    if (count < k) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return left + k;
};
```

1540 Can Convert String In K Moves

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1540. Can Convert String in K Moves
 * https://leetcode.com/problems/can-convert-string-in-k-moves/
 * Difficulty: Medium
 *
 * Given two strings s and t, your goal is to convert s into t in k moves or less.
 *
 * During the ith (1 <= i <= k) move you can:
 * - Choose any index j (1-indexed) from s, such that 1 <= j <= s.length and j has not been
chosen
```

```
 *   in any previous move, and shift the character at that index i times.
 * - Do nothing.
 *
 * Shifting a character means replacing it by the next letter in the alphabet (wrapping around
 so
 * that 'z' becomes 'a'). Shifting a character by i means applying the shift operations i
times.
 *
 * Remember that any index j can be picked at most once.
 *
 * Return true if it's possible to convert s into t in no more than k moves, otherwise return
false.
 */

/**
 * @param {string} s
 * @param {string} t
 * @param {number} k
 * @return {boolean}
 */
var canConvertString = function(source, target, maxMoves) {
  if (source.length !== target.length) return false;
  const shiftCounts = new Array(26).fill(0);

  for (let i = 0; i < source.length; i++) {
    const shiftNeeded = (target.charCodeAt(i) - source.charCodeAt(i) + 26) % 26;
    if (shiftNeeded > 0) {
      shiftCounts[shiftNeeded]++;
    }
  }

  for (let shift = 1; shift < 26; shift++) {
    if (shiftCounts[shift] === 0) continue;
    const totalMoves = shift + 26 * (shiftCounts[shift] - 1);
    if (totalMoves > maxMoves) return false;
  }

  return true;
};
```

1541 Minimum Insertions To Balance A Parentheses String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1541. Minimum Insertions to Balance a Parentheses String
 * https://leetcode.com/problems/minimum-insertions-to-balance-a-parentheses-string/
 * Difficulty: Medium
 *
 * Given a parentheses string s containing only the characters '(' and ')'. A parentheses
string
 * is balanced if:
 * - Any left parenthesis '(' must have a corresponding two consecutive right parenthesis
'))'.
 * - Left parenthesis '(' must go before the corresponding two consecutive right parenthesis
'))'.
 *
 * In other words, we treat '(' as an opening parenthesis and '))' as a closing parenthesis.
 * - For example, "())", "())(())))" and "(())())))" are balanced, ")()", "()))" and "((()))"
are
 *   not balanced.
 *
 * You can insert the characters '(' and ')' at any position of the string to balance it if
needed.
 *
 * Return the minimum number of insertions needed to make s balanced.
 */

/**
 * @param {string} s
```

```
 * @return {number}
 */
var minInsertions = function(s) {
  let result = 0;
  let openCount = 0;

  for (let i = 0; i < s.length; i++) {
    if (s[i] === '(') {
      openCount++;
    } else {
      if (i + 1 < s.length && s[i + 1] === ')') {
        i++;
      } else {
        result++;
      }
      if (openCount > 0) {
        openCount--;
      } else {
        result++;
      }
    }
  }

  result += openCount * 2;

  return result;
};
```

1542 Find Longest Awesome Substring

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1542. Find Longest Awesome Substring
 * https://leetcode.com/problems/find-longest-awesome-substring/
 * Difficulty: Hard
 *
 * You are given a string s. An awesome substring is a non-empty substring of s such that we
 * can make any number of swaps in order to make it a palindrome.
 *
 * Return the length of the maximum length awesome substring of s.
 */

/**
 * @param {string} s
 * @return {number}
 */
var longestAwesome = function(s) {
  let result = 0;
  let prefixMask = 0;
  const digitMasks = new Map();
  digitMasks.set(0, -1);

  for (let i = 0; i < s.length; i++) {
    const digit = s.charCodeAt(i) - '0'.charCodeAt(0);
    prefixMask ^= (1 << digit);

    if (digitMasks.has(prefixMask)) {
      result = Math.max(result, i - digitMasks.get(prefixMask));
    } else {
      digitMasks.set(prefixMask, i);
    }

    for (let j = 0; j < 10; j++) {
      const oddMask = prefixMask ^ (1 << j);
      if (digitMasks.has(oddMask)) {
        result = Math.max(result, i - digitMasks.get(oddMask));
      }
    }
  }
```

```
  return result;
};
```

1544 Make The String Great

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1544. Make The String Great
 * https://leetcode.com/problems/make-the-string-great/
 * Difficulty: Easy
 *
 * Given a string s of lower and upper case English letters.
 *
 * A good string is a string which doesn't have two adjacent characters s[i] and s[i + 1]
where:
 * - 0 <= i <= s.length - 2
 * - s[i] is a lower-case letter and s[i + 1] is the same letter but in upper-case or
vice-versa.
 *
 * To make the string good, you can choose two adjacent characters that make the string bad
and
 * remove them. You can keep doing this until the string becomes good.
 *
 * Return the string after making it good. The answer is guaranteed to be unique under the
given
 * constraints.
 *
 * Notice that an empty string is also good.
 */

/**
 * @param {string} s
 * @return {string}
 */
var makeGood = function(s) {
  const stack = [];

  for (const char of s) {
    const lastChar = stack[stack.length - 1];
    if (lastChar && ((char.toLowerCase() === lastChar.toLowerCase()) && (char !== lastChar)))
{
      stack.pop();
    } else {
      stack.push(char);
    }
  }

  return stack.join('');
};
```

1545 Find Kth Bit In Nth Binary String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1545. Find Kth Bit in Nth Binary String
 * https://leetcode.com/problems/find-kth-bit-in-nth-binary-string/
 * Difficulty: Medium
 *
 * Given two positive integers n and k, the binary string Sn is formed as follows:
 * - S1 = "0"
 * - Si = Si - 1 + "1" + reverse(invert(Si - 1)) for i > 1
 *
```

```
 * Where + denotes the concatenation operation, reverse(x) returns the reversed string x, and
 * invert(x) inverts all the bits in x (0 changes to 1 and 1 changes to 0).
 *
 * For example, the first four strings in the above sequence are:
 * - S1 = "0"
 * - S2 = "011"
 * - S3 = "0111001"
 * - S4 = "011100110110001"
 *
 * Return the kth bit in Sn. It is guaranteed that k is valid for the given n.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {character}
 */
var findKthBit = function(n, k) {
  let position = k - 1;
  let invertCount = 0;
  let length = (1 << n) - 1;

  while (position !== 0) {
    const mid = length >> 1;

    if (position === mid) {
      return invertCount % 2 === 0 ? '1' : '0';
    }

    if (position > mid) {
      position = length - position - 1;
      invertCount++;
    } else {
      length = mid;
    }
  }

  return invertCount % 2 === 0 ? '0' : '1';
};
```

1546 Maximum Number Of Non Overlapping Subarrays With Sum Equals Target

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1546. Maximum Number of Non-Overlapping Subarrays With Sum Equals Target
 * https://leetcode.com/problems/maximum-number-of-non-overlapping-subarrays-with-sum-equals-t
arget/
 * Difficulty: Medium
 *
 * Given an array nums and an integer target, return the maximum number of non-empty
non-overlapping
 * subarrays such that the sum of values in each subarray is equal to target.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var maxNonOverlapping = function(nums, target) {
  const prefixSums = new Map();
  let currentSum = 0;
  let result = 0;
  prefixSums.set(0, 0);

  for (let i = 0; i < nums.length; i++) {
    currentSum += nums[i];

    if (prefixSums.has(currentSum - target)) {
```

```
      result++;
      prefixSums.clear();
      prefixSums.set(0, i + 1);
      currentSum = 0;
    } else {
      prefixSums.set(currentSum, i + 1);
    }
  }

  return result;
};
```

1547 Minimum Cost To Cut A Stick

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1547. Minimum Cost to Cut a Stick
 * https://leetcode.com/problems/minimum-cost-to-cut-a-stick/
 * Difficulty: Hard
 *
 * Given a wooden stick of length n units. The stick is labelled from 0 to n. For example, a
 * stick of length 6 is labelled as follows.
 *
 * Given an integer array cuts where cuts[i] denotes a position you should perform a cut at.
 *
 * You should perform the cuts in order, you can change the order of the cuts as you wish.
 *
 * The cost of one cut is the length of the stick to be cut, the total cost is the sum of
costs
 * of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum
of
 * their lengths is the length of the stick before the cut). Please refer to the first example
 * for a better explanation.
 *
 * Return the minimum total cost of the cuts.
 */

/**
 * @param {number} n
 * @param {number[]} cuts
 * @return {number}
 */
var minCost = function(n, cuts) {
  cuts.push(0, n);
  cuts.sort((a, b) => a - b);

  const memo = new Map();

  return computeMinCost(0, cuts.length - 1);

  function computeMinCost(left, right) {
    if (right - left <= 1) return 0;

    const key = `${left},${right}`;
    if (memo.has(key)) return memo.get(key);

    let minCost = Infinity;
    for (let i = left + 1; i < right; i++) {
      const cost = (cuts[right] - cuts[left]) + computeMinCost(left, i) + computeMinCost(i,
right);
      minCost = Math.min(minCost, cost);
    }

    memo.set(key, minCost);
    return minCost;
  }
};
```

1548 The Most Similar Path In A Graph

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1548. The Most Similar Path in a Graph
 * https://leetcode.com/problems/the-most-similar-path-in-a-graph/
 * Difficulty: Hard
 *
 * We have n cities and m bi-directional roads where roads[i] = [ai, bi] connects city ai
 * with city bi. Each city has a name consisting of exactly three upper-case English letters
 * given in the string array names. Starting at any city x, you can reach any city y where
 * y != x (i.e., the cities and the roads are forming an undirected connected graph).
 *
 * You will be given a string array targetPath. You should find a path in the graph of the
 * same length and with the minimum edit distance to targetPath.
 *
 * You need to return the order of the nodes in the path with the minimum edit distance.
 * The path should be of the same length of targetPath and should be valid (i.e., there
 * should be a direct road between ans[i] and ans[i + 1]). If there are multiple answers
 * return any one of them.
 */

/**
 * @param {number} n
 * @param {number[][]} roads
 * @param {string[]} names
 * @param {string[]} targetPath
 * @return {number[]}
 */
var mostSimilar = function(n, roads, names, targetPath) {
  const graph = new Array(n).fill().map(() => []);
  for (const [a, b] of roads) {
    graph[a].push(b);
    graph[b].push(a);
  }

  const m = targetPath.length;
  const dp = new Array(m).fill().map(() => new Array(n).fill(Infinity));
  const parent = new Array(m).fill().map(() => new Array(n).fill(-1));

  for (let city = 0; city < n; city++) {
    dp[0][city] = names[city] === targetPath[0] ? 0 : 1;
  }

  for (let step = 1; step < m; step++) {
    for (let city = 0; city < n; city++) {
      for (const neighbor of graph[city]) {
        const cost = dp[step - 1][neighbor] + (names[city] === targetPath[step] ? 0 : 1);
        if (cost < dp[step][city]) {
          dp[step][city] = cost;
          parent[step][city] = neighbor;
        }
      }
    }
  }

  let minCost = Infinity;
  let lastCity = -1;
  for (let city = 0; city < n; city++) {
    if (dp[m - 1][city] < minCost) {
      minCost = dp[m - 1][city];
      lastCity = city;
    }
  }

  const result = [];
  let currentCity = lastCity;
  for (let step = m - 1; step >= 0; step--) {
    result.unshift(currentCity);
```

```
    currentCity = parent[step][currentCity];
  }

  return result;
};
```

1550 Three Consecutive Odds

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1550. Three Consecutive Odds
 * https://leetcode.com/problems/three-consecutive-odds/
 * Difficulty: Easy
 *
 * Given an integer array arr, return true if there are three consecutive odd
 * numbers in the array. Otherwise, return false.
 */

/**
 * @param {number[]} arr
 * @return {boolean}
 */
var threeConsecutiveOdds = function(arr) {
  for (let i = 0, count = 0; i < arr.length; i++) {
    count = arr[i] % 2 !== 0 ? count + 1 : 0;
    if (count === 3) return true;
  }
  return false;
};
```

1551 Minimum Operations To Make Array Equal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1551. Minimum Operations to Make Array Equal
 * https://leetcode.com/problems/minimum-operations-to-make-array-equal/
 * Difficulty: Medium
 *
 * You have an array arr of length n where arr[i] = (2 * i) + 1 for all valid values
 * of i (i.e., 0 <= i < n).
 *
 * In one operation, you can select two indices x and y where 0 <= x, y < n and subtract
 * 1 from arr[x] and add 1 to arr[y] (i.e., perform arr[x] -=1 and arr[y] += 1). The goal
 * is to make all the elements of the array equal. It is guaranteed that all the elements
 * of the array can be made equal using some operations.
 *
 * Given an integer n, the length of the array, return the minimum number of operations
 * needed to make all the elements of arr equal.
 */

/**
 * @param {number} n
 * @return {number}
 */
var minOperations = function(n) {
  return n * n / 4;
};
```

1552 Magnetic Force Between Two Balls

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1552. Magnetic Force Between Two Balls
 * https://leetcode.com/problems/magnetic-force-between-two-balls/
 * Difficulty: Medium
 *
 * In the universe Earth C-137, Rick discovered a special form of magnetic force between two
balls
 * if they are put in his new invented basket. Rick has n empty baskets, the ith basket is at
 * position[i], Morty has m balls and needs to distribute the balls into the baskets such that
 * the minimum magnetic force between any two balls is maximum.
 *
 * Rick stated that magnetic force between two different balls at positions x and y is |x -
y|.
 *
 * Given the integer array position and the integer m. Return the required force.
 */

/**
 * @param {number[]} position
 * @param {number} m
 * @return {number}
 */
var maxDistance = function(position, m) {
  position.sort((a, b) => a - b);

  let left = 1;
  let right = position[position.length - 1] - position[0];
  let result = 0;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (canPlaceBalls(mid)) {
      result = mid;
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;

  function canPlaceBalls(minForce) {
    let count = 1;
    let lastPos = position[0];

    for (let i = 1; i < position.length; i++) {
      if (position[i] - lastPos >= minForce) {
        count++;
        lastPos = position[i];
      }
    }

    return count >= m;
  }
};
```

1553 Minimum Number Of Days To Eat N Oranges

HARD

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1553. Minimum Number of Days to Eat N Oranges
 * https://leetcode.com/problems/minimum-number-of-days-to-eat-n-oranges/
 * Difficulty: Hard
 *
 * There are n oranges in the kitchen and you decided to eat some of these oranges every day
as
 * follows:
 * - Eat one orange.
 * - If the number of remaining oranges n is divisible by 2 then you can eat n / 2 oranges.
 * - If the number of remaining oranges n is divisible by 3 then you can eat 2 * (n / 3)
oranges.
 *
 * You can only choose one of the actions per day.
 *
 * Given the integer n, return the minimum number of days to eat n oranges.
 */

/**
 * @param {number} n
 * @return {number}
 */
var minDays = function(n) {
  const memo = new Map();
  return computeMinDays(n);

  function computeMinDays(oranges) {
    if (oranges <= 1) return oranges;
    if (memo.has(oranges)) return memo.get(oranges);

    const eatHalf = computeMinDays(Math.floor(oranges / 2)) + (oranges % 2) + 1;
    const eatTwoThirds = computeMinDays(Math.floor(oranges / 3)) + (oranges % 3) + 1;
    const minDays = Math.min(eatHalf, eatTwoThirds);
    memo.set(oranges, minDays);
    return minDays;
  }
};
```

1554 Strings Differ By One Character

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1554. Strings Differ by One Character
 * https://leetcode.com/problems/strings-differ-by-one-character/
 * Difficulty: Medium
 *
 * Given a list of strings dict where all the strings are of the same length.
 *
 * Return true if there are 2 strings that only differ by 1 character in the same index,
 * otherwise return false.
 */

/**
 * @param {string[]} dict
 * @return {boolean}
 */
var differByOne = function(dict) {
  const m = dict[0].length;

  for (let pos = 0; pos < m; pos++) {
    const patterns = new Set();

    for (const word of dict) {
      const pattern = word.slice(0, pos) + '*' + word.slice(pos + 1);

      if (patterns.has(pattern)) {
        return true;
      }
```

```
      patterns.add(pattern);
    }
  }

  return false;
};
```

1556 Thousand Separator

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1556. Thousand Separator
 * https://leetcode.com/problems/thousand-separator/
 * Difficulty: Easy
 *
 * Given an integer n, add a dot (".") as the thousands separator and return it in string
format.
 */

/**
 * @param {number} n
 * @return {string}
 */
var thousandSeparator = function(n) {
  const digits = n.toString().split('');
  const result = [];

  while (digits.length > 0) {
    const group = digits.splice(-3).join('');
    result.unshift(group);
    if (digits.length > 0) {
      result.unshift('.');
    }
  }

  return result.join('');
};
```

1557 Minimum Number Of Vertices To Reach All Nodes

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1557. Minimum Number of Vertices to Reach All Nodes
 * https://leetcode.com/problems/minimum-number-of-vertices-to-reach-all-nodes/
 * Difficulty: Medium
 *
 * Given a directed acyclic graph, with n vertices numbered from 0 to n-1, and an array edges
 * where edges[i] = [fromi, toi] represents a directed edge from node fromi to node toi.
 *
 * Find the smallest set of vertices from which all nodes in the graph are reachable. It's
 * guaranteed that a unique solution exists.
 *
 * Notice that you can return the vertices in any order.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[]}
 */
var findSmallestSetOfVertices = function(n, edges) {
  const hasIncomingEdge = new Array(n).fill(false);
```

```
  for (const [_, to] of edges) {
    hasIncomingEdge[to] = true;
  }

  const result = [];
  for (let i = 0; i < n; i++) {
    if (!hasIncomingEdge[i]) {
      result.push(i);
    }
  }

  return result;
};
```

1558 Minimum Numbers Of Function Calls To Make Target Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1558. Minimum Numbers of Function Calls to Make Target Array
 * https://leetcode.com/problems/minimum-numbers-of-function-calls-to-make-target-array/
 * Difficulty: Medium
 *
 * You are given an integer array nums. You have an integer array arr of the same length with
all
 * values set to 0 initially. You also have the following modify function.
 *
 * You want to use the modify function to convert arr to nums using the minimum number of
calls.
 *
 * Return the minimum number of function calls to make nums from arr.
 *
 * The test cases are generated so that the answer fits in a 32-bit signed integer.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minOperations = function(nums) {
  let totalOperations = 0;
  let maxDivisions = 0;

  for (let num of nums) {
    let currentDivisions = 0;
    let increments = 0;

    while (num > 0) {
      if (num % 2 === 1) {
        increments++;
        num--;
      } else {
        currentDivisions++;
        num /= 2;
      }
    }

    totalOperations += increments;
    maxDivisions = Math.max(maxDivisions, currentDivisions);
  }

  return totalOperations + maxDivisions;
};
```

1559 Detect Cycles In 2D Grid

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1559. Detect Cycles in 2D Grid
 * https://leetcode.com/problems/detect-cycles-in-2d-grid/
 * Difficulty: Medium
 *
 * Given a 2D array of characters grid of size m x n, you need to find if there exists any
cycle
 * consisting of the same value in grid.
 *
 * A cycle is a path of length 4 or more in the grid that starts and ends at the same cell.
From
 * a given cell, you can move to one of the cells adjacent to it - in one of the four
directions
 * (up, down, left, or right), if it has the same value of the current cell.
 *
 * Also, you cannot move to the cell that you visited in your last move. For example, the
cycle
 * (1, 1) -> (1, 2) -> (1, 1) is invalid because from (1, 2) we visited (1, 1) which was the
last
 * visited cell.
 *
 * Return true if any cycle of the same value exists in grid, otherwise, return false.
 */

/**
 * @param {character[][]} grid
 * @return {boolean}
 */
var containsCycle = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const visited = new Set();

  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      if (!visited.has(`${row},${col}`)) {
        if (explorePath(row, col, -1, -1, grid[row][col])) return true;
      }
    }
  }

  return false;

  function explorePath(row, col, prevRow, prevCol, char) {
    const key = `${row},${col}`;
    if (visited.has(key)) return true;
    visited.add(key);

    const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];
    for (const [dx, dy] of directions) {
      const newRow = row + dx;
      const newCol = col + dy;

      if (newRow === prevRow && newCol === prevCol) continue;
      if (newRow < 0 || newRow >= rows || newCol < 0 || newCol >= cols) continue;
      if (grid[newRow][newCol] !== char) continue;

      if (explorePath(newRow, newCol, row, col, char)) return true;
    }

    return false;
  }
};
```

1560 Most Visited Sector In A Circular Track

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1560. Most Visited Sector in a Circular Track
 * https://leetcode.com/problems/most-visited-sector-in-a-circular-track/
 * Difficulty: Easy
 *
 * Given an integer n and an integer array rounds. We have a circular track which consists of
n
 * sectors labeled from 1 to n. A marathon will be held on this track, the marathon consists
of
 * m rounds. The ith round starts at sector rounds[i - 1] and ends at sector rounds[i]. For
 * example, round 1 starts at sector rounds[0] and ends at sector rounds[1]
 *
 * Return an array of the most visited sectors sorted in ascending order.
 *
 * Notice that you circulate the track in ascending order of sector numbers in the
counter-clockwise
 * direction (See the first example).
 */

/**
 * @param {number} n
 * @param {number[]} rounds
 * @return {number[]}
 */
var mostVisited = function(n, rounds) {
  const sectorVisits = new Array(n + 1).fill(0);
  let maxVisits = 0;

  for (let i = 1; i < rounds.length; i++) {
    let start = rounds[i - 1];
    const end = rounds[i];

    while (start !== end) {
      sectorVisits[start]++;
      maxVisits = Math.max(maxVisits, sectorVisits[start]);
      start = start === n ? 1 : start + 1;
    }
  }

  sectorVisits[rounds[rounds.length - 1]]++;
  maxVisits = Math.max(maxVisits, sectorVisits[rounds[rounds.length - 1]]);

  const result = [];
  for (let i = 1; i <= n; i++) {
    if (sectorVisits[i] === maxVisits) {
      result.push(i);
    }
  }

  return result;
};
```

1561 Maximum Number Of Coins You Can Get

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1561. Maximum Number of Coins You Can Get
 * https://leetcode.com/problems/maximum-number-of-coins-you-can-get/
 * Difficulty: Medium
 *
```

```
 * There are 3n piles of coins of varying size, you and your friends will take piles of coins
as
 * follows:
 * - In each step, you will choose any 3 piles of coins (not necessarily consecutive).
 * - Of your choice, Alice will pick the pile with the maximum number of coins.
 * - You will pick the next pile with the maximum number of coins.
 * - Your friend Bob will pick the last pile.
 * - Repeat until there are no more piles of coins.
 *
 * Given an array of integers piles where piles[i] is the number of coins in the ith pile.
 *
 * Return the maximum number of coins that you can have.
 */

/**
 * @param {number[]} piles
 * @return {number}
 */
var maxCoins = function(piles) {
  piles.sort((a, b) => b - a);
  let result = 0;
  const rounds = piles.length / 3;

  for (let i = 1; i < piles.length - rounds; i += 2) {
    result += piles[i];
  }

  return result;
};
```

1562 Find Latest Group Of Size M

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1562. Find Latest Group of Size M
 * https://leetcode.com/problems/find-latest-group-of-size-m/
 * Difficulty: Medium
 *
 * Given an array arr that represents a permutation of numbers from 1 to n.
 *
 * You have a binary string of size n that initially has all its bits set to zero. At each
step
 * i (assuming both the binary string and arr are 1-indexed) from 1 to n, the bit at position
 * arr[i] is set to 1.
 *
 * You are also given an integer m. Find the latest step at which there exists a group of ones
 * of length m. A group of ones is a contiguous substring of 1's such that it cannot be
extended
 * in either direction.
 *
 * Return the latest step at which there exists a group of ones of length exactly m. If no
such
 * group exists, return -1.
 */

/**
 * @param {number[]} arr
 * @param {number} m
 * @return {number}
 */
var findLatestStep = function(arr, m) {
  const lengths = new Array(arr.length + 2).fill(0);
  const count = new Map();
  let result = -1;

  for (let step = 0; step < arr.length; step++) {
    const pos = arr[step];
    const left = lengths[pos - 1];
    const right = lengths[pos + 1];
```

```
    const newLength = left + right + 1;

    lengths[pos - left] = newLength;
    lengths[pos + right] = newLength;
    lengths[pos] = newLength;

    count.set(left, (count.get(left) || 0) - 1);
    count.set(right, (count.get(right) || 0) - 1);
    count.set(newLength, (count.get(newLength) || 0) + 1);

    if (count.get(m) > 0) result = step + 1;
  }

  return result;
};
```

1563 Stone Game V

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1563. Stone Game V
 * https://leetcode.com/problems/stone-game-v/
 * Difficulty: Hard
 *
 * There are several stones arranged in a row, and each stone has an associated value which
 * is an integer given in the array stoneValue.
 *
 * In each round of the game, Alice divides the row into two non-empty rows (i.e. left row and
 * right row), then Bob calculates the value of each row which is the sum of the values of all
 * the stones in this row. Bob throws away the row which has the maximum value, and Alice's
 * score increases by the value of the remaining row. If the value of the two rows are equal,
 * Bob lets Alice decide which row will be thrown away. The next round starts with the
remaining
 * row.
 *
 * The game ends when there is only one stone remaining. Alice's is initially zero.
 *
 * Return the maximum score that Alice can obtain.
 */

/**
 * @param {number[]} stoneValue
 * @return {number}
 */
var stoneGameV = function(stoneValue) {
  const n = stoneValue.length;
  const prefixScore = Array.from({ length: n + 1 }, () => 0);
  const dp = Array.from({ length: n }, () => new Array(n).fill(-1));

  for (let index = 1; index <= n; index++) {
    prefixScore[index] = prefixScore[index - 1] + stoneValue[index - 1];
  }

  return getMaxScore(0, n - 1);

  function getMaxScore(left, right) {
    if (left >= right) return 0;
    if (dp[left][right] !== -1) return dp[left][right];
    let result = 0;

    for (let index = left; index < right; index++) {
      const heap1 = prefixScore[index + 1] - prefixScore[left];
      const heap2 = prefixScore[right + 1] - prefixScore[index + 1];

      if (heap1 > heap2) {
        const score = heap2 + getMaxScore(index + 1, right);

        result = Math.max(score, result);
      } else if (heap1 < heap2) {
```

```
        const score = heap1 + getMaxScore(left, index);

        result = Math.max(score, result);
      } else {
        const score1 = heap2 + getMaxScore(index + 1, right);
        const score2 = heap1 + getMaxScore(left, index);

        result = Math.max(result, score1, score2);
      }
    }

    dp[left][right] = result;

    return result;
  }
};
```

1564 Put Boxes Into The Warehouse I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1564. Put Boxes Into the Warehouse I
 * https://leetcode.com/problems/put-boxes-into-the-warehouse-i/
 * Difficulty: Medium
 *
 * You are given two arrays of positive integers, boxes and warehouse, representing the
 * heights of some boxes of unit width and the heights of n rooms in a warehouse respectively.
 * The warehouse's rooms are labelled from 0 to n - 1 from left to right where warehouse[i]
 * (0-indexed) is the height of the ith room.
 *
 * Boxes are put into the warehouse by the following rules:
 * - Boxes cannot be stacked.
 * - You can rearrange the insertion order of the boxes.
 * - Boxes can only be pushed into the warehouse from left to right only.
 * - If the height of some room in the warehouse is less than the height of a box, then that
 *   box and all other boxes behind it will be stopped before that room.
 *
 * Return the maximum number of boxes you can put into the warehouse.
 */

/**
 * @param {number[]} boxes
 * @param {number[]} warehouse
 * @return {number}
 */
var maxBoxesInWarehouse = function(boxes, warehouse) {
  const effectiveHeights = new Array(warehouse.length);
  effectiveHeights[0] = warehouse[0];

  for (let i = 1; i < warehouse.length; i++) {
    effectiveHeights[i] = Math.min(effectiveHeights[i - 1], warehouse[i]);
  }

  boxes.sort((a, b) => a - b);

  let boxIndex = 0;
  let count = 0;
  for (let i = effectiveHeights.length - 1; i >= 0; i--) {
    if (boxIndex < boxes.length && boxes[boxIndex] <= effectiveHeights[i]) {
      count++;
      boxIndex++;
    }
  }

  return count;
};
```

1566 Detect Pattern Of Length M Repeated K Or More Times

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1566. Detect Pattern of Length M Repeated K or More Times
 * https://leetcode.com/problems/detect-pattern-of-length-m-repeated-k-or-more-times/
 * Difficulty: Easy
 *
 * Given an array of positive integers arr, find a pattern of length m that is repeated k or
more times.
 *
 * A pattern is a subarray (consecutive sub-sequence) that consists of one or more values,
repeated multiple
 * times consecutively without overlapping. A pattern is defined by its length and the number
of repetitions.
 *
 * Return true if there exists a pattern of length m that is repeated k or more times,
otherwise return false.
 */

/**
 * @param {number[]} arr
 * @param {number} m
 * @param {number} k
 * @return {boolean}
 */
var containsPattern = function(arr, m, k) {
  for (let i = 0; i < arr.length - m + 1; i++) {
    const pattern = new Array(k).fill(arr.slice(i, i + m).join()).join();
    if (arr.join().includes(pattern)) {
      return true;
    }
  }
  return false;
};
```

1567 Maximum Length Of Subarray With Positive Product

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1567. Maximum Length of Subarray With Positive Product
 * https://leetcode.com/problems/maximum-length-of-subarray-with-positive-product/
 * Difficulty: Medium
 *
 * Given an array of integers nums, find the maximum length of a subarray where the product of
 * all its elements is positive.
 *
 * A subarray of an array is a consecutive sequence of zero or more values taken out of that
array.
 *
 * Return the maximum length of a subarray with positive product.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var getMaxLen = function(nums) {
  let result = 0;
  let positiveCount = 0;
  let negativeCount = 0;
  let firstNegativeIndex = -1;
```

```
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] === 0) {
      positiveCount = 0;
      negativeCount = 0;
      firstNegativeIndex = -1;
      continue;
    }

    if (nums[i] > 0) {
      positiveCount++;
    } else {
      negativeCount++;
      if (firstNegativeIndex === -1) {
        firstNegativeIndex = i;
      }
    }

    if (negativeCount % 2 === 0) {
      result = Math.max(result, positiveCount + negativeCount);
    } else {
      result = Math.max(result, i - firstNegativeIndex);
    }
  }

  return result;
};
```

1568 Minimum Number Of Days To Disconnect Island

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1568. Minimum Number of Days to Disconnect Island
 * https://leetcode.com/problems/minimum-number-of-days-to-disconnect-island/
 * Difficulty: Hard
 *
 * You are given an m x n binary grid grid where 1 represents land and 0 represents water. An
 * island is a maximal 4-directionally (horizontal or vertical) connected group of 1's.
 *
 * The grid is said to be connected if we have exactly one island, otherwise is said
disconnected.
 *
 * In one day, we are allowed to change any single land cell (1) into a water cell (0).
 *
 * Return the minimum number of days to disconnect the grid.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minDays = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;

  if (countIslands() !== 1) return 0;

  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      if (grid[r][c] === 1) {
        grid[r][c] = 0;
        if (countIslands() !== 1) return 1;
        grid[r][c] = 1;
      }
    }
  }

  return 2;

  function countIslands() {
```

```
    const visited = Array(rows).fill().map(() => Array(cols).fill(false));
    let islands = 0;

    function dfs(row, col) {
      if (row < 0 || row >= rows || col < 0 || col >= cols
          || visited[row][col] || grid[row][col] === 0) {
        return;
      }
      visited[row][col] = true;
      dfs(row + 1, col);
      dfs(row - 1, col);
      dfs(row, col + 1);
      dfs(row, col - 1);
    }

    for (let r = 0; r < rows; r++) {
      for (let c = 0; c < cols; c++) {
        if (grid[r][c] === 1 && !visited[r][c]) {
          islands++;
          dfs(r, c);
        }
      }
    }
    return islands;
  }
};
```

1569 Number Of Ways To Reorder Array To Get Same Bst

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1569. Number of Ways to Reorder Array to Get Same BST
 * https://leetcode.com/problems/number-of-ways-to-reorder-array-to-get-same-bst/
 * Difficulty: Hard
 *
 * Given an array nums that represents a permutation of integers from 1 to n. We are going to
 * construct a binary search tree (BST) by inserting the elements of nums in order into an
 * initially empty BST. Find the number of different ways to reorder nums so that the
constructed
 * BST is identical to that formed from the original array nums.
 *
 * For example, given nums = [2,1,3], we will have 2 as the root, 1 as a left child, and 3 as
a
 * right child. The array [2,3,1] also yields the same BST but [3,2,1] yields a different BST.
 *
 * Return the number of ways to reorder nums such that the BST formed is identical to the
original
 * BST formed from nums.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var numOfWays = function(nums) {
  const MOD = BigInt(10 ** 9 + 7);
  const factorialCache = Array(nums.length).fill(null);
  factorialCache[0] = 1n;

  function calculatePermutations(arr) {
    if (arr.length < 3) return 1n;

    const root = arr[0];
    const leftSubtree = [];
    const rightSubtree = [];

    for (let i = 1; i < arr.length; i++) {
```

```
      if (arr[i] < root) {
        leftSubtree.push(arr[i]);
      } else {
        rightSubtree.push(arr[i]);
      }
    }

    const leftPermutations = calculatePermutations(leftSubtree);
    const rightPermutations = calculatePermutations(rightSubtree);
    const totalNodes = BigInt(arr.length - 1);
    const leftNodes = BigInt(leftSubtree.length);

    return (helper(totalNodes, leftNodes) * leftPermutations * rightPermutations) % MOD;
  }

  function helper(n, k) {
    factorialCache[n] = computeFactorial(n);
    factorialCache[n - k] = computeFactorial(n - k);
    factorialCache[k] = computeFactorial(k);
    return factorialCache[n] / (factorialCache[k] * factorialCache[n - k]);
  }

  function computeFactorial(n) {
    if (factorialCache[n]) return factorialCache[n];
    return n * computeFactorial(n - 1n);
  }

  return Number((calculatePermutations(nums) - 1n) % MOD);
};
```

1570 Dot Product Of Two Sparse Vectors

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1570. Dot Product of Two Sparse Vectors
 * https://leetcode.com/problems/dot-product-of-two-sparse-vectors/
 * Difficulty: Medium
 *
 * Given two sparse vectors, compute their dot product.
 *
 * Implement class SparseVector:
 * - SparseVector(nums) Initializes the object with the vector nums
 * - dotProduct(vec) Compute the dot product between the instance of SparseVector and vec
 *
 * A sparse vector is a vector that has mostly zero values, you should store the sparse vector
 * efficiently and compute the dot product between two SparseVector.
 *
 * Follow up: What if only one of the vectors is sparse?
 */

/**
 * @param {number[]} nums
 * @return {SparseVector}
 */
var SparseVector = function(nums) {
  this.nonZeroElements = new Map();
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] !== 0) {
      this.nonZeroElements.set(i, nums[i]);
    }
  }
};

/**
 * @param {SparseVector} vec
 * @return {number}
 */
SparseVector.prototype.dotProduct = function(vec) {
  let result = 0;
```

```
  for (const [index, value] of this.nonZeroElements) {
    if (vec.nonZeroElements.has(index)) {
      result += value * vec.nonZeroElements.get(index);
    }
  }

  return result;
};
```

1572 Matrix Diagonal Sum

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1572. Matrix Diagonal Sum
 * https://leetcode.com/problems/matrix-diagonal-sum/
 * Difficulty: Easy
 *
 * Given a square matrix mat, return the sum of the matrix diagonals.
 *
 * Only include the sum of all the elements on the primary diagonal and all the elements on
the
 * secondary diagonal that are not part of the primary diagonal.
 */

/**
 * @param {number[][]} mat
 * @return {number}
 */
var diagonalSum = function(mat) {
  const n = mat.length;
  let sum = 0;

  for (let i = 0; i < n; i++) {
    sum += mat[i][i];
    sum += mat[i][n - 1 - i];
  }

  if (n % 2 === 1) {
    sum -= mat[Math.floor(n / 2)][Math.floor(n / 2)];
  }

  return sum;
};
```

1573 Number Of Ways To Split A String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1573. Number of Ways to Split a String
 * https://leetcode.com/problems/number-of-ways-to-split-a-string/
 * Difficulty: Medium
 *
 * Given a binary string s, you can split s into 3 non-empty strings s1, s2, and s3
 * where s1 + s2 + s3 = s.
 *
 * Return the number of ways s can be split such that the number of ones is the same in s1,
 * s2, and s3. Since the answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {string} s
 * @return {number}
```

```
 */
var numWays = function(s) {
  const MOD = 1e9 + 7;
  let ones = 0;

  for (const char of s) {
    if (char === '1') ones++;
  }

  if (ones % 3 !== 0) return 0;
  if (ones === 0) {
    return Number((BigInt(s.length - 1) * BigInt(s.length - 2) / 2n) % BigInt(MOD));
  }

  const targetOnes = ones / 3;
  let firstCount = 0;
  let secondCount = 0;
  let currentOnes = 0;

  for (let i = 0; i < s.length; i++) {
    if (s[i] === '1') currentOnes++;
    if (currentOnes === targetOnes) firstCount++;
    else if (currentOnes === 2 * targetOnes) secondCount++;
  }

  return Number((BigInt(firstCount) * BigInt(secondCount)) % BigInt(MOD));
};
```

1574 Shortest Subarray To Be Removed To Make Array Sorted

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1574. Shortest Subarray to be Removed to Make Array Sorted
 * https://leetcode.com/problems/shortest-subarray-to-be-removed-to-make-array-sorted/
 * Difficulty: Medium
 *
 * Given an integer array arr, remove a subarray (can be empty) from arr such that the
remaining
 * elements in arr are non-decreasing.
 *
 * Return the length of the shortest subarray to remove.
 *
 * A subarray is a contiguous subsequence of the array.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var findLengthOfShortestSubarray = function(arr) {
  const n = arr.length;
  let left = 0;

  while (left < n - 1 && arr[left] <= arr[left + 1]) {
    left++;
  }

  if (left === n - 1) return 0;

  let right = n - 1;
  while (right > 0 && arr[right - 1] <= arr[right]) {
    right--;
  }

  let minLength = Math.min(n - left - 1, right);

  for (let i = 0, j = right; i <= left && j < n; i++) {
    while (j < n && arr[i] > arr[j]) {
      j++;
```

```
    }
    minLength = Math.min(minLength, j - i - 1);
  }

  return minLength;
};
```

1575 Count All Possible Routes

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1575. Count All Possible Routes
 * https://leetcode.com/problems/count-all-possible-routes/
 * Difficulty: Hard
 *
 * You are given an array of distinct positive integers locations where locations[i]
represents
 * the position of city i. You are also given integers start, finish and fuel representing the
 * starting city, ending city, and the initial amount of fuel you have, respectively.
 *
 * At each step, if you are at city i, you can pick any city j such that j != i and
 * 0 <= j < locations.length and move to city j. Moving from city i to city j reduces the
 * amount of fuel you have by |locations[i] - locations[j]|. Please notice that |x| denotes
 * the absolute value of x.
 *
 * Notice that fuel cannot become negative at any point in time, and that you are allowed to
visit
 * any city more than once (including start and finish).
 *
 * Return the count of all possible routes from start to finish. Since the answer may be too
large,
 * return it modulo 109 + 7.
 */

/**
 * @param {number[]} locations
 * @param {number} start
 * @param {number} finish
 * @param {number} fuel
 * @return {number}
 */
var countRoutes = function(locations, start, finish, fuel) {
  const MOD = 1e9 + 7;
  const memo = new Array(locations.length).fill().map(() => new Array(fuel + 1).fill(-1));

  function calculateRoutes(currentCity, remainingFuel) {
    if (remainingFuel < 0) return 0;
    if (memo[currentCity][remainingFuel] !== -1) return memo[currentCity][remainingFuel];

    let routes = currentCity === finish ? 1 : 0;

    for (let nextCity = 0; nextCity < locations.length; nextCity++) {
      if (nextCity !== currentCity) {
        const fuelCost = Math.abs(locations[currentCity] - locations[nextCity]);
        routes = (routes + calculateRoutes(nextCity, remainingFuel - fuelCost)) % MOD;
      }
    }

    memo[currentCity][remainingFuel] = routes;
    return routes;
  }

  return calculateRoutes(start, fuel);
};
```

1576 Replace All S To Avoid Consecutive Repeating Characters

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1576. Replace All ?'s to Avoid Consecutive Repeating Characters
 * https://leetcode.com/problems/replace-all-s-to-avoid-consecutive-repeating-characters/
 * Difficulty: Medium
 *
 * Given a string s containing only lowercase English letters and the '?' character, convert
all
 * the '?' characters into lowercase letters such that the final string does not contain any
 * consecutive repeating characters. You cannot modify the non '?' characters.
 *
 * It is guaranteed that there are no consecutive repeating characters in the given string
except
 * for '?'.
 *
 * Return the final string after all the conversions (possibly zero) have been made. If there
is
 * more than one solution, return any of them. It can be shown that an answer is always
possible
 * with the given constraints.
 */

/**
 * @param {string} s
 * @return {string}
 */
var modifyString = function(s) {
  const substitute = takenCharacteres => {
    for (let charCode = 97; ; charCode++) {
      const attempt = String.fromCharCode(charCode);

      if (!takenCharacteres.includes(attempt)) {
        return attempt;
      }
    }
  };

  const result = [...s];
  for (let index = 0; index < s.length; index++) {
    const takenCharacteres = [result[index - 1], result[index + 1]];
    result[index] = s[index] === '?' ? substitute(takenCharacteres) : s[index];
  }
  return result.join('');
};
```

1577 Number Of Ways Where Square Of Number Is Equal To Product Of Two N

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1577. Number of Ways Where Square of Number Is Equal to Product of Two Numbers
 * https://leetcode.com/problems/number-of-ways-where-square-of-number-is-equal-to-product-of-
two-numbers/
 * Difficulty: Medium
 *
 * Given two arrays of integers nums1 and nums2, return the number of triplets formed (type 1
and
 * type 2) under the following rules:
 * - Type 1: Triplet (i, j, k) if nums1[i]2 == nums2[j] * nums2[k] where 0 <= i < nums1.length
 *   and 0 <= j < k < nums2.length.
 * - Type 2: Triplet (i, j, k) if nums2[i]2 == nums1[j] * nums1[k] where 0 <= i < nums2.length
 *   and 0 <= j < k < nums1.length.
```

```
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var numTriplets = function(nums1, nums2) {
  return countTriplets(nums1, nums2) + countTriplets(nums2, nums1);

  function countTriplets(arr1, arr2) {
    const productMap = new Map();
    let count = 0;

    for (let i = 0; i < arr2.length; i++) {
      for (let j = i + 1; j < arr2.length; j++) {
        const product = BigInt(arr2[i]) * BigInt(arr2[j]);
        productMap.set(product, (productMap.get(product) || 0) + 1);
      }
    }

    for (const num of arr1) {
      const square = BigInt(num) * BigInt(num);
      if (productMap.has(square)) {
        count += productMap.get(square);
      }
    }

    return count;
  }
};
```

1578 Minimum Time To Make Rope Colorful

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1578. Minimum Time to Make Rope Colorful
 * https://leetcode.com/problems/minimum-time-to-make-rope-colorful/
 * Difficulty: Medium
 *
 * Alice has n balloons arranged on a rope. You are given a 0-indexed string colors where
colors[i]
 * is the color of the ith balloon.
 *
 * Alice wants the rope to be colorful. She does not want two consecutive balloons to be of
the same
 * color, so she asks Bob for help. Bob can remove some balloons from the rope to make it
colorful.
 * You are given a 0-indexed integer array neededTime where neededTime[i] is the time (in
seconds)
 * that Bob needs to remove the ith balloon from the rope.
 *
 * Return the minimum time Bob needs to make the rope colorful.
 */

/**
 * @param {string} colors
 * @param {number[]} neededTime
 * @return {number}
 */
var minCost = function(colors, neededTime) {
  let result = 0;
  let i = 0;

  while (i < colors.length - 1) {
    if (colors[i] === colors[i + 1]) {
      let maxTime = neededTime[i];
      let sumTime = neededTime[i];
      let j = i + 1;
```

```
      while (j < colors.length && colors[j] === colors[i]) {
        maxTime = Math.max(maxTime, neededTime[j]);
        sumTime += neededTime[j];
        j++;
      }

      result += sumTime - maxTime;
      i = j;
    } else {
      i++;
    }
  }

  return result;
};
```

1579 Remove Max Number Of Edges To Keep Graph Fully Traversable

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1579. Remove Max Number of Edges to Keep Graph Fully Traversable
 * https://leetcode.com/problems/remove-max-number-of-edges-to-keep-graph-fully-traversable/
 * Difficulty: Hard
 *
 * Alice and Bob have an undirected graph of n nodes and three types of edges:
 * - Type 1: Can be traversed by Alice only.
 * - Type 2: Can be traversed by Bob only.
 * - Type 3: Can be traversed by both Alice and Bob.
 *
 * Given an array edges where edges[i] = [typei, ui, vi] represents a bidirectional edge of
type
 * typei between nodes ui and vi, find the maximum number of edges you can remove so that
after
 * removing the edges, the graph can still be fully traversed by both Alice and Bob. The graph
 * is fully traversed by Alice and Bob if starting from any node, they can reach all other
nodes.
 *
 * Return the maximum number of edges you can remove, or return -1 if Alice and Bob cannot
fully
 * traverse the graph.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var maxNumEdgesToRemove = function(n, edges) {
  const alice = new UnionFind(n + 1);
  const bob = new UnionFind(n + 1);
  let removableEdges = 0;

  for (const [type, u, v] of edges) {
    if (type === 3) {
      const usedAlice = alice.union(u, v);
      const usedBob = bob.union(u, v);
      if (!usedAlice && !usedBob) {
        removableEdges++;
      }
    }
  }

  for (const [type, u, v] of edges) {
    if (type === 1) {
      if (!alice.union(u, v)) {
        removableEdges++;
      }
    } else if (type === 2) {
```

```
      if (!bob.union(u, v)) {
        removableEdges++;
      }
    }
  }

  return alice.components === 2 && bob.components === 2 ? removableEdges : -1;
};

class UnionFind {
  constructor(size) {
    this.parent = Array(size).fill().map((_, i) => i);
    this.rank = Array(size).fill(0);
    this.components = size;
  }

  find(x) {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]);
    }
    return this.parent[x];
  }

  union(x, y) {
    const rootX = this.find(x);
    const rootY = this.find(y);
    if (rootX === rootY) return false;
    if (this.rank[rootX] < this.rank[rootY]) {
      this.parent[rootX] = rootY;
    } else if (this.rank[rootX] > this.rank[rootY]) {
      this.parent[rootY] = rootX;
    } else {
      this.parent[rootY] = rootX;
      this.rank[rootX]++;
    }
    this.components--;
    return true;
  }
}
```

1580 Put Boxes Into The Warehouse Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1580. Put Boxes Into the Warehouse II
 * https://leetcode.com/problems/put-boxes-into-the-warehouse-ii/
 * Difficulty: Medium
 *
 * You are given two arrays of positive integers, boxes and warehouse, representing the
heights
 * of some boxes of unit width and the heights of n rooms in a warehouse respectively. The
 * warehouse's rooms are labeled from 0 to n - 1 from left to right where warehouse[i]
(0-indexed)
 * is the height of the ith room.
 *
 * Boxes are put into the warehouse by the following rules:
 * - Boxes cannot be stacked.
 * - You can rearrange the insertion order of the boxes.
 * - Boxes can be pushed into the warehouse from either side (left or right)
 * - If the height of some room in the warehouse is less than the height of a box, then that
box
 *   and all other boxes behind it will be stopped before that room.
 *
 * Return the maximum number of boxes you can put into the warehouse.
 */

/**
 * @param {number[]} boxes
 * @param {number[]} warehouse
```

```
 * @return {number}
 */
var maxBoxesInWarehouse = function(boxes, warehouse) {
  boxes.sort((a, b) => a - b);

  let leftPointer = 0;
  let rightPointer = warehouse.length - 1;
  let leftMaxHeight = warehouse[0];
  let rightMaxHeight = warehouse[warehouse.length - 1];
  let result = 0;

  while (result < warehouse.length && leftPointer <= rightPointer && boxes.length > 0) {
    while (boxes.length > 0 && boxes[boxes.length - 1] > leftMaxHeight
          && boxes[boxes.length - 1] > rightMaxHeight) {
      boxes.pop();
    }

    if (boxes.length === 0) break;

    const canFitLeft = boxes[boxes.length - 1] <= leftMaxHeight;
    const canFitRight = boxes[boxes.length - 1] <= rightMaxHeight;

    if (canFitLeft && (!canFitRight || leftMaxHeight <= rightMaxHeight)) {
      boxes.pop();
      leftPointer++;
      result++;
      if (leftPointer < warehouse.length) {
        leftMaxHeight = Math.min(warehouse[leftPointer], leftMaxHeight);
      }
    } else if (canFitRight) {
      boxes.pop();
      rightPointer--;
      result++;
      if (rightPointer >= 0) {
        rightMaxHeight = Math.min(warehouse[rightPointer], rightMaxHeight);
      }
    } else {
      break;
    }
  }

  return result;
};
```

1582 Special Positions In A Binary Matrix

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1582. Special Positions in a Binary Matrix
 * https://leetcode.com/problems/special-positions-in-a-binary-matrix/
 * Difficulty: Easy
 *
 * Given an m x n binary matrix mat, return the number of special positions in mat.
 *
 * A position (i, j) is called special if mat[i][j] == 1 and all other elements in row i and
 * column j are 0 (rows and columns are 0-indexed).
 */

/**
 * @param {number[][]} mat
 * @return {number}
 */
var numSpecial = function(mat) {
  const rows = mat.length;
  const cols = mat[0].length;
  const rowSums = new Array(rows).fill(0);
  const colSums = new Array(cols).fill(0);
  let result = 0;
```

```
  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (mat[i][j] === 1) {
        rowSums[i]++;
        colSums[j]++;
      }
    }
  }

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (mat[i][j] === 1 && rowSums[i] === 1 && colSums[j] === 1) {
        result++;
      }
    }
  }

  return result;
};
```

1583 Count Unhappy Friends

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1583. Count Unhappy Friends
 * https://leetcode.com/problems/count-unhappy-friends/
 * Difficulty: Medium
 *
 * You are given a list of preferences for n friends, where n is always even.
 *
 * For each person i, preferences[i] contains a list of friends sorted in the order of
preference.
 * In other words, a friend earlier in the list is more preferred than a friend later in the
list.
 * Friends in each list are denoted by integers from 0 to n-1.
 *
 * All the friends are divided into pairs. The pairings are given in a list pairs, where
 * pairs[i] = [xi, yi] denotes xi is paired with yi and yi is paired with xi.
 *
 * However, this pairing may cause some of the friends to be unhappy. A friend x is unhappy if
 x
 * is paired with y and there exists a friend u who is paired with v but:
 * - x prefers u over y, and
 * - u prefers x over v.
 *
 * Return the number of unhappy friends.
 */

/**
 * @param {number} n
 * @param {number[][]} preferences
 * @param {number[][]} pairs
 * @return {number}
 */
var unhappyFriends = function(n, preferences, pairs) {
  const rank = Array.from({ length: n }, () => new Array(n).fill(0));
  const pairMap = new Array(n).fill(0);
  let result = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n - 1; j++) {
      rank[i][preferences[i][j]] = j;
    }
  }

  for (const [x, y] of pairs) {
    pairMap[x] = y;
    pairMap[y] = x;
  }
```

```
  for (let x = 0; x < n; x++) {
    const y = pairMap[x];
    for (const u of preferences[x]) {
      if (u === y) break;
      const v = pairMap[u];
      if (rank[u][x] < rank[u][v]) {
        result++;
        break;
      }
    }
  }

  return result;
};
```

1584 Min Cost To Connect All Points

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1584. Min Cost to Connect All Points
 * https://leetcode.com/problems/min-cost-to-connect-all-points/
 * Difficulty: Medium
 *
 * You are given an array points representing integer coordinates of some points on a
2D-plane,
 * where points[i] = [xi, yi].
 *
 * The cost of connecting two points [xi, yi] and [xj, yj] is the manhattan distance between
 * them: |xi - xj| + |yi - yj|, where |val| denotes the absolute value of val.
 *
 * Return the minimum cost to make all points connected. All points are connected if there is
 * exactly one simple path between any two points.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var minCostConnectPoints = function(points) {
  const n = points.length;
  const minCost = Array(n).fill(Infinity);
  const visited = new Set();
  let result = 0;

  minCost[0] = 0;

  for (let i = 0; i < n; i++) {
    let minIdx = -1;
    let minVal = Infinity;

    for (let j = 0; j < n; j++) {
      if (!visited.has(j) && minCost[j] < minVal) {
        minVal = minCost[j];
        minIdx = j;
      }
    }

    if (minIdx === -1) break;

    visited.add(minIdx);
    result += minVal;

    for (let j = 0; j < n; j++) {
      if (!visited.has(j)) {
        const cost = Math.abs(points[minIdx][0] - points[j][0])
          + Math.abs(points[minIdx][1] - points[j][1]);
        minCost[j] = Math.min(minCost[j], cost);
      }
```

```
    }
  }

  return result;
};
```

1585 Check If String Is Transformable With Substring Sort Operations

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1585. Check If String Is Transformable With Substring Sort Operations
 *
https://leetcode.com/problems/check-if-string-is-transformable-with-substring-sort-operations/
 * Difficulty: Hard
 *
 * Given two strings s and t, transform string s into string t using the following operation
any
 * number of times:
 * - Choose a non-empty substring in s and sort it in place so the characters are in ascending
 *   order.
 *   - For example, applying the operation on the underlined substring in "14234" results
 *     in "12344".
 *
 * Return true if it is possible to transform s into t. Otherwise, return false.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isTransformable = function(source, target) {
  const digitPositions = Array.from({ length: 10 }, () => []);
  for (let index = 0; index < source.length; index++) {
    digitPositions[source[index]].push(index);
  }

  const currentIndices = new Array(10).fill(0);

  for (const digit of target) {
    const digitValue = parseInt(digit, 10);
    if (currentIndices[digitValue] >= digitPositions[digitValue].length) {
      return false;
    }

    const position = digitPositions[digitValue][currentIndices[digitValue]];
    for (let smallerDigit = 0; smallerDigit < digitValue; smallerDigit++) {
      if (currentIndices[smallerDigit] < digitPositions[smallerDigit].length
          && digitPositions[smallerDigit][currentIndices[smallerDigit]] < position) {
        return false;
      }
    }

    currentIndices[digitValue]++;
  }

  return true;
};
```

1586 Binary Search Tree Iterator Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1586. Binary Search Tree Iterator II
 * https://leetcode.com/problems/binary-search-tree-iterator-ii/
 * Difficulty: Medium
 *
 * Implement the BSTIterator class that represents an iterator over the in-order traversal
 * of a binary search tree (BST):
 * - BSTIterator(TreeNode root) Initializes an object of the BSTIterator class. The root of
the
 *   BST is given as part of the constructor. The pointer should be initialized to a
non-existent
 *   number smaller than any element in the BST.
 * - boolean hasNext() Returns true if there exists a number in the traversal to the right of
the
 *   pointer, otherwise returns false.
 * - int next() Moves the pointer to the right, then returns the number at the pointer.
 * - boolean hasPrev() Returns true if there exists a number in the traversal to the left of
the
 *   pointer, otherwise returns false.
 * - int prev() Moves the pointer to the left, then returns the number at the pointer.
 *
 * Notice that by initializing the pointer to a non-existent smallest number, the first call
to
 * next() will return the smallest element in the BST.
 *
 * You may assume that next() and prev() calls will always be valid. That is, there will be at
 * least a next/previous number in the in-order traversal when next()/prev() is called.
 */

/**
 * @param {TreeNode} root
 */
var BSTIterator = function(root) {
  this.inorderValues = [];
  this.currentIndex = -1;
  this.inorderTraversal(root);
};

/**
 * @param {TreeNode} node
 */
BSTIterator.prototype.inorderTraversal = function(node) {
  if (!node) return;
  this.inorderTraversal(node.left);
  this.inorderValues.push(node.val);
  this.inorderTraversal(node.right);
};

/**
 * @return {boolean}
 */
BSTIterator.prototype.hasNext = function() {
  return this.currentIndex + 1 < this.inorderValues.length;
};

/**
 * @return {number}
 */
BSTIterator.prototype.next = function() {
  this.currentIndex++;
  return this.inorderValues[this.currentIndex];
};

/**
 * @return {boolean}
 */
BSTIterator.prototype.hasPrev = function() {
  return this.currentIndex > 0;
};

/**
 * @return {number}
 */
BSTIterator.prototype.prev = function() {
```

```
  this.currentIndex--;
  return this.inorderValues[this.currentIndex];
};
```

1588 Sum Of All Odd Length Subarrays

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1588. Sum of All Odd Length Subarrays
 * https://leetcode.com/problems/sum-of-all-odd-length-subarrays/
 * Difficulty: Easy
 *
 * Given an array of positive integers arr, return the sum of all possible odd-length
 * subarrays of arr.
 *
 * A subarray is a contiguous subsequence of the array.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var sumOddLengthSubarrays = function(arr) {
  let result = 0;

  for (let index = 0; index < arr.length; index++) {
    result += arr[index] * Math.ceil(((index + 1) * (arr.length - index)) / 2);
  }

  return result;
};
```

1589 Maximum Sum Obtained Of Any Permutation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1589. Maximum Sum Obtained of Any Permutation
 * https://leetcode.com/problems/maximum-sum-obtained-of-any-permutation/
 * Difficulty: Medium
 *
 * We have an array of integers, nums, and an array of requests where requests[i] = [starti,
endi].
 * The ith request asks for the sum of nums[starti] + nums[starti + 1] + ... + nums[endi - 1]
 * + nums[endi]. Both starti and endi are 0-indexed.
 *
 * Return the maximum total sum of all requests among all permutations of nums.
 *
 * Since the answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} requests
 * @return {number}
 */
var maxSumRangeQuery = function(nums, requests) {
  const MOD = 1e9 + 7;
  const n = nums.length;
  const freq = new Array(n + 1).fill(0);

  for (const [start, end] of requests) {
    freq[start]++;
    freq[end + 1]--;
```

```
  }

  const count = new Array(n).fill(0);
  let current = 0;
  for (let i = 0; i < n; i++) {
    current += freq[i];
    count[i] = current;
  }

  count.sort((a, b) => b - a);
  nums.sort((a, b) => b - a);

  let total = 0;
  for (let i = 0; i < n; i++) {
    total = (total + nums[i] * count[i]) % MOD;
  }

  return total;
};
```

1590 Make Sum Divisible By P

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1590. Make Sum Divisible by P
 * https://leetcode.com/problems/make-sum-divisible-by-p/
 * Difficulty: Medium
 *
 * Given an array of positive integers nums, remove the smallest subarray (possibly empty)
such that
 * the sum of the remaining elements is divisible by p. It is not allowed to remove the whole
array.
 *
 * Return the length of the smallest subarray that you need to remove, or -1 if it's
impossible.
 *
 * A subarray is defined as a contiguous block of elements in the array.
 */

/**
 * @param {number[]} nums
 * @param {number} p
 * @return {number}
 */
var minSubarray = function(nums, p) {
  const totalSum = nums.reduce((sum, num) => sum + num, 0);
  const remainder = totalSum % p;

  if (remainder === 0) return 0;

  const prefixSums = new Map([[0, -1]]);
  let currentSum = 0;
  let minLength = nums.length;

  for (let i = 0; i < nums.length; i++) {
    currentSum = (currentSum + nums[i]) % p;
    const target = (currentSum - remainder + p) % p;

    if (prefixSums.has(target)) {
      minLength = Math.min(minLength, i - prefixSums.get(target));
    }

    prefixSums.set(currentSum, i);
  }

  return minLength < nums.length ? minLength : -1;
};
```

1591 Strange Printer Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1591. Strange Printer II
 * https://leetcode.com/problems/strange-printer-ii/
 * Difficulty: Hard
 *
 * There is a strange printer with the following two special requirements:
 * - On each turn, the printer will print a solid rectangular pattern of a single color on the
 *   grid. This will cover up the existing colors in the rectangle.
 * - Once the printer has used a color for the above operation, the same color cannot be used
again.
 *
 * You are given a m x n matrix targetGrid, where targetGrid[row][col] is the color in the
position
 * (row, col) of the grid.
 *
 * Return true if it is possible to print the matrix targetGrid, otherwise, return false.
 */

/**
 * @param {number[][]} targetGrid
 * @return {boolean}
 */
var isPrintable = function(targetGrid) {
  const rows = targetGrid.length;
  const cols = targetGrid[0].length;
  const colorBounds = new Map();

  for (let color = 1; color <= 60; color++) {
    let minRow = rows;
    let maxRow = -1;
    let minCol = cols;
    let maxCol = -1;

    for (let r = 0; r < rows; r++) {
      for (let c = 0; c < cols; c++) {
        if (targetGrid[r][c] === color) {
          minRow = Math.min(minRow, r);
          maxRow = Math.max(maxRow, r);
          minCol = Math.min(minCol, c);
          maxCol = Math.max(maxCol, c);
        }
      }
    }
    if (maxRow >= 0) {
      colorBounds.set(color, [minRow, maxRow, minCol, maxCol]);
    }
  }

  const dependencies = new Map();
  for (const [color, [minRow, maxRow, minCol, maxCol]] of colorBounds) {
    const deps = new Set();
    for (let r = minRow; r <= maxRow; r++) {
      for (let c = minCol; c <= maxCol; c++) {
        if (targetGrid[r][c] !== color) {
          deps.add(targetGrid[r][c]);
        }
      }
    }
    dependencies.set(color, deps);
  }

  const visited = new Set();
  const recStack = new Set();

  function hasCycle(color) {
    if (recStack.has(color)) return true;
```

```
    if (visited.has(color)) return false;

    visited.add(color);
    recStack.add(color);

    for (const dep of dependencies.get(color) || []) {
      if (hasCycle(dep)) return true;
    }

    recStack.delete(color);
    return false;
  }

  for (const color of colorBounds.keys()) {
    if (hasCycle(color)) return false;
  }

  return true;
};
```

1592 Rearrange Spaces Between Words

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1592. Rearrange Spaces Between Words
 * https://leetcode.com/problems/rearrange-spaces-between-words/
 * Difficulty: Easy
 *
 * You are given a string text of words that are placed among some number of spaces. Each word
 * consists of one or more lowercase English letters and are separated by at least one space.
 * It's guaranteed that text contains at least one word.
 *
 * Rearrange the spaces so that there is an equal number of spaces between every pair of
adjacent
 * words and that number is maximized. If you cannot redistribute all the spaces equally,
place
 * the extra spaces at the end, meaning the returned string should be the same length as text.
 *
 * Return the string after rearranging the spaces.
 */

/**
 * @param {string} text
 * @return {string}
 */
var reorderSpaces = function(text) {
  const words = text.trim().split(/\s+/);
  const spaceCount = text.length - words.join('').length;
  const wordCount = words.length;

  if (wordCount === 1) return words[0] + ' '.repeat(spaceCount);

  const spacesBetween = Math.floor(spaceCount / (wordCount - 1));
  const extraSpaces = spaceCount % (wordCount - 1);

  return words.join(' '.repeat(spacesBetween)) + ' '.repeat(extraSpaces);
};
```

1593 Split A String Into The Max Number Of Unique Substrings

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1593. Split a String Into the Max Number of Unique Substrings
```

```
 * https://leetcode.com/problems/split-a-string-into-the-max-number-of-unique-substrings/
 * Difficulty: Medium
 *
 * Given a string s, return the maximum number of unique substrings that the given string can
 * be split into.
 *
 * You can split string s into any list of non-empty substrings, where the concatenation of
 * the substrings forms the original string. However, you must split the substrings such that
 * all of them are unique.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxUniqueSplit = function(s) {
  return backtrack(0, new Set());

  function backtrack(start, seen) {
    if (start === s.length) return seen.size;

    let maxSplits = 0;
    for (let end = start + 1; end <= s.length; end++) {
      const substring = s.slice(start, end);
      if (!seen.has(substring)) {
        seen.add(substring);
        maxSplits = Math.max(maxSplits, backtrack(end, seen));
        seen.delete(substring);
      }
    }

    return maxSplits;
  }
};
```

1594 Maximum Non Negative Product In A Matrix

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1594. Maximum Non Negative Product in a Matrix
 * https://leetcode.com/problems/maximum-non-negative-product-in-a-matrix/
 * Difficulty: Medium
 *
 * You are given a m x n matrix grid. Initially, you are located at the top-left corner (0,
0),
 * and in each step, you can only move right or down in the matrix.
 *
 * Among all possible paths starting from the top-left corner (0, 0) and ending in the
bottom-right
 * corner (m - 1, n - 1), find the path with the maximum non-negative product. The product of
a
 * path is the product of all integers in the grid cells visited along the path.
 *
 * Return the maximum non-negative product modulo 109 + 7. If the maximum product is negative,
 * return -1.
 *
 * Notice that the modulo is performed after getting the maximum product.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maxProductPath = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const dp = Array.from({ length: rows }, () =>
```

```
    Array.from({ length: cols }, () => [1, 1])
  );

  dp[0][0] = [grid[0][0], grid[0][0]];

  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      if (row === 0 && col === 0) continue;

      let minProduct = Infinity;
      let maxProduct = -Infinity;

      if (row > 0) {
        minProduct = Math.min(
          minProduct,
          dp[row - 1][col][0] * grid[row][col],
          dp[row - 1][col][1] * grid[row][col]
        );
        maxProduct = Math.max(
          maxProduct,
          dp[row - 1][col][0] * grid[row][col],
          dp[row - 1][col][1] * grid[row][col]
        );
      }

      if (col > 0) {
        minProduct = Math.min(
          minProduct,
          dp[row][col - 1][0] * grid[row][col],
          dp[row][col - 1][1] * grid[row][col]
        );
        maxProduct = Math.max(
          maxProduct,
          dp[row][col - 1][0] * grid[row][col],
          dp[row][col - 1][1] * grid[row][col]
        );
      }

      dp[row][col] = [minProduct, maxProduct];
    }
  }

  const maxResult = dp[rows - 1][cols - 1][1];
  return maxResult < 0 ? -1 : maxResult % (10 ** 9 + 7);
};
```

1595 Minimum Cost To Connect Two Groups Of Points

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1595. Minimum Cost to Connect Two Groups of Points
 * https://leetcode.com/problems/minimum-cost-to-connect-two-groups-of-points/
 * Difficulty: Hard
 *
 * You are given two groups of points where the first group has size1 points, the second group
 * has size2 points, and size1 >= size2.
 *
 * The cost of the connection between any two points are given in an size1 x size2 matrix
where
 * cost[i][j] is the cost of connecting point i of the first group and point j of the second
group.
 * The groups are connected if each point in both groups is connected to one or more points in
 the
 * opposite group. In other words, each point in the first group must be connected to at least
 one
 * point in the second group, and each point in the second group must be connected to at least
 one
 * point in the first group.
 *
```

```
 * Return the minimum cost it takes to connect the two groups.
 */

/**
* @param {number[][]} cost
* @return {number}
*/
var connectTwoGroups = function(cost) {
  const size1 = cost.length;
  const size2 = cost[0].length;

  const minCostGroup2 = new Array(size2).fill(Infinity);
  for (let j = 0; j < size2; j++) {
    for (let i = 0; i < size1; i++) {
      minCostGroup2[j] = Math.min(minCostGroup2[j], cost[i][j]);
    }
  }

  const memo = new Array(size1).fill(0).map(() => new Array(1 << size2).fill(-1));

  return dfs(0, 0);

  function dfs(i, mask) {
    if (i === size1) {
      let remainingCost = 0;
      for (let j = 0; j < size2; j++) {
        if ((mask & (1 << j)) === 0) {
          remainingCost += minCostGroup2[j];
        }
      }
      return remainingCost;
    }

    if (memo[i][mask] !== -1) return memo[i][mask];

    let minCost = Infinity;

    for (let j = 0; j < size2; j++) {
      minCost = Math.min(
        minCost,
        cost[i][j] + dfs(i + 1, mask | (1 << j))
      );
    }

    memo[i][mask] = minCost;
    return minCost;
  }
};
```

1597 Build Binary Expression Tree From Infix Expression

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1597. Build Binary Expression Tree From Infix Expression
 * https://leetcode.com/problems/build-binary-expression-tree-from-infix-expression/
 * Difficulty: Hard
 *
 * A binary expression tree is a kind of binary tree used to represent arithmetic expressions.
 * Each node of a binary expression tree has either zero or two children. Leaf nodes (nodes
 * with 0 children) correspond to operands (numbers), and internal nodes (nodes with 2
children)
 * correspond to the operators '+' (addition), '-' (subtraction), '*' (multiplication), and
 * '/' (division).
 *
 * For each internal node with operator o, the infix expression it represents is (A o B),
 * where A is the expression the left subtree represents and B is the expression the right
 * subtree represents.
 *
```

```
 * You are given a string s, an infix expression containing operands, the operators described
 * above, and parentheses '(' and ')'.
 *
 * Return any valid binary expression tree, whose in-order traversal reproduces s after
omitting
 * the parenthesis from it.
 *
 * Please note that order of operations applies in s. That is, expressions in parentheses are
 * evaluated first, and multiplication and division happen before addition and subtraction.
 *
 * Operands must also appear in the same order in both s and the in-order traversal of the
tree.
 */

/**
 * Definition for a binary tree node.
 * function Node(val, left, right) {
 *     this.val = (val===undefined ? " " : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {string} s
 * @return {Node}
 */
var expTree = function(s) {
  const operators = new Map([['+', 1], ['-', 1], ['*', 2], ['/', 2]]);
  const operatorStack = [];
  const operandStack = [];

  for (let i = 0; i < s.length; i++) {
    const char = s[i];

    if (char >= '0' && char <= '9') {
      operandStack.push(createNode(char));
    } else if (char === '(') {
      operatorStack.push(char);
    } else if (char === ')') {
      while (operatorStack.length > 0 && operatorStack[operatorStack.length - 1] !== '(') {
        processOperator();
      }
      operatorStack.pop();
    } else if (operators.has(char)) {
      while (operatorStack.length > 0 && operatorStack[operatorStack.length - 1] !== '('
        && operators.get(operatorStack[operatorStack.length - 1]) >= operators.get(char)) {
        processOperator();
      }
      operatorStack.push(char);
    }
  }

  while (operatorStack.length > 0) {
    processOperator();
  }

  return operandStack[0];

  function processOperator() {
    const operator = operatorStack.pop();
    const right = operandStack.pop();
    const left = operandStack.pop();
    const node = createNode(operator);
    node.left = left;
    node.right = right;
    operandStack.push(node);
  }

  function createNode(val) {
    return new Node(val);
  }
};
```

1598 Crawler Log Folder

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1598. Crawler Log Folder
 * https://leetcode.com/problems/crawler-log-folder/
 * Difficulty: Easy
 *
 * The Leetcode file system keeps a log each time some user performs
 * a change folder operation.
 *
 * The operations are described below:
 *
 * - `"../"` : Move to the parent folder of the current folder. (If
 *   you are already in the main folder, remain in the same folder).
 * - `"./"` : Remain in the same folder.
 * - `"x/"` : Move to the child folder named x (This folder is
 *   guaranteed to always exist).
 *
 * You are given a list of strings logs where logs[i] is the operation
 * performed by the user at the ith step.
 *
 * The file system starts in the main folder, then the operations in
 * logs are performed.
 *
 * Return the minimum number of operations needed to go back to the
 * main folder after the change folder operations.
 */

/**
 * @param {string[]} logs
 * @return {number}
 */
var minOperations = function(logs) {
  return logs.reduce((depth, log) =>
    Math.max(0, depth + (log === '../' ? -1 : log === './' ? 0 : 1)), 0
  );
};
```

1599 Maximum Profit Of Operating A Centennial Wheel

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1599. Maximum Profit of Operating a Centennial Wheel
 * https://leetcode.com/problems/maximum-profit-of-operating-a-centennial-wheel/
 * Difficulty: Medium
 *
 * You are the operator of a Centennial Wheel that has four gondolas, and each gondola has
room for
 * up to four people. You have the ability to rotate the gondolas counterclockwise, which
costs you
 * runningCost dollars.
 *
 * You are given an array customers of length n where customers[i] is the number of new
customers
 * arriving just before the ith rotation (0-indexed). This means you must rotate the wheel i
times
 * before the customers[i] customers arrive. You cannot make customers wait if there is room
in the
 * gondola. Each customer pays boardingCost dollars when they board on the gondola closest to
the
 * ground and will exit once that gondola reaches the ground again.
 *
```

```
 * You can stop the wheel at any time, including before serving all customers. If you decide
to stop
 * serving customers, all subsequent rotations are free in order to get all the customers down
 * safely. Note that if there are currently more than four customers waiting at the wheel,
only
 * four will board the gondola, and the rest will wait for the next rotation.
 *
 * Return the minimum number of rotations you need to perform to maximize your profit. If
there is
 * no scenario where the profit is positive, return -1.
 */

/**
 * @param {number[]} customers
 * @param {number} boardingCost
 * @param {number} runningCost
 * @return {number}
 */
var minOperationsMaxProfit = function(customers, boardingCost, runningCost) {
  let waitingCustomers = 0;
  let totalBoarded = 0;
  let maxProfit = -Infinity;
  let rotationAtMaxProfit = -1;
  let currentProfit = 0;
  let rotations = 0;

  const gondolaCapacity = 4;

  for (let i = 0; i < customers.length || waitingCustomers > 0; i++) {
    rotations++;

    if (i < customers.length) {
      waitingCustomers += customers[i];
    }

    const boarding = Math.min(waitingCustomers, gondolaCapacity);
    waitingCustomers -= boarding;
    totalBoarded += boarding;

    currentProfit = totalBoarded * boardingCost - rotations * runningCost;

    if (currentProfit > maxProfit) {
      maxProfit = currentProfit;
      rotationAtMaxProfit = rotations;
    }
  }

  return maxProfit > 0 ? rotationAtMaxProfit : -1;
};
```

1600 Throne Inheritance

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1600. Throne Inheritance
 * https://leetcode.com/problems/throne-inheritance/
 * Difficulty: Medium
 *
 * A kingdom consists of a king, his children, his grandchildren, and so on. Every once in a
while,
 * someone in the family dies or a child is born.
 *
 * The kingdom has a well-defined order of inheritance that consists of the king as the first
 * member. Let's define the recursive function Successor(x, curOrder), which given a person x
and
 * the inheritance order so far, returns who should be the next person after x in the order of
 * inheritance.
 *
 * For example, assume we have a kingdom that consists of the king, his children Alice and Bob
```

```
 * (Alice is older than Bob), and finally Alice's son Jack.
 *
 * 1. In the beginning, curOrder will be ["king"].
 * 2. Calling Successor(king, curOrder) will return Alice, so we append to curOrder to
 *    get ["king", "Alice"].
 * 3. Calling Successor(Alice, curOrder) will return Jack, so we append to curOrder to
 *    get ["king", "Alice", "Jack"].
 * 4. Calling Successor(Jack, curOrder) will return Bob, so we append to curOrder to
 *    get ["king", "Alice", "Jack", "Bob"].
 * 5. Calling Successor(Bob, curOrder) will return null. Thus the order of inheritance
 *    will be ["king", "Alice", "Jack", "Bob"].
 *
 * Using the above function, we can always obtain a unique order of inheritance.
 *
 * Implement the ThroneInheritance class:
 * - ThroneInheritance(string kingName) Initializes an object of the ThroneInheritance class.
 *   The name of the king is given as part of the constructor.
 * - void birth(string parentName, string childName) Indicates that parentName gave birth
 *   to childName.
 * - void death(string name) Indicates the death of name. The death of the person doesn't
 *   affect the Successor function nor the current inheritance order. You can treat it as
 *   just marking the person as dead.
 * - string[] getInheritanceOrder() Returns a list representing the current order of
inheritance
 *   excluding dead people.
 */

/**
 * @param {string} kingName
 */
var ThroneInheritance = function(kingName) {
  this.king = kingName;
  this.familyTree = new Map();
  this.deceased = new Set();
};

/**
 * @param {string} parentName
 * @param {string} childName
 * @return {void}
 */
ThroneInheritance.prototype.birth = function(parentName, childName) {
  if (!this.familyTree.has(parentName)) {
    this.familyTree.set(parentName, []);
  }
  this.familyTree.get(parentName).push(childName);
};

/**
 * @param {string} name
 * @return {void}
 */
ThroneInheritance.prototype.death = function(name) {
  this.deceased.add(name);
};

/**
 * @return {string[]}
 */
ThroneInheritance.prototype.getInheritanceOrder = function() {
  const result = [];
  const traverse = (person) => {
    if (!this.deceased.has(person)) {
      result.push(person);
    }
    const children = this.familyTree.get(person) || [];
    for (const child of children) {
      traverse(child);
    }
  };
  traverse(this.king);
  return result;
};
```

1601 Maximum Number Of Achievable Transfer Requests

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1601. Maximum Number of Achievable Transfer Requests
 * https://leetcode.com/problems/maximum-number-of-achievable-transfer-requests/
 * Difficulty: Hard
 *
 * We have n buildings numbered from 0 to n - 1. Each building has a number of employees.
 * It's transfer season, and some employees want to change the building they reside in.
 *
 * You are given an array requests where requests[i] = [fromi, toi] represents an employee's
 * request to transfer from building fromi to building toi.
 *
 * All buildings are full, so a list of requests is achievable only if for each building, the
 * net change in employee transfers is zero. This means the number of employees leaving is
 * equal to the number of employees moving in. For example if n = 3 and two employees are
 * leaving building 0, one is leaving building 1, and one is leaving building 2, there should
 * be two employees moving to building 0, one employee moving to building 1, and one employee
 * moving to building 2.
 *
 * Return the maximum number of achievable requests.
 */

/**
 * @param {number} n
 * @param {number[][]} requests
 * @return {number}
 */
var maximumRequests = function(n, requests) {
  let maxAchievable = 0;
  tryCombination(0, 0, new Array(n).fill(0));
  return maxAchievable;

  function tryCombination(index, count, balance) {
    if (index === requests.length) {
      if (balance.every(val => val === 0)) {
        maxAchievable = Math.max(maxAchievable, count);
      }
      return;
    }

    const [from, to] = requests[index];
    balance[from]--;
    balance[to]++;
    tryCombination(index + 1, count + 1, balance);
    balance[from]++;
    balance[to]--;

    tryCombination(index + 1, count, balance);
  }
};
```

1602 Find Nearest Right Node In Binary Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1602. Find Nearest Right Node in Binary Tree
 * https://leetcode.com/problems/find-nearest-right-node-in-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and a node u in the tree, return the nearest node on the
 * same level that is to the right of u, or return null if u is the rightmost node in its
```

```
level.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} u
 * @return {TreeNode}
 */
var findNearestRightNode = function(root, u) {
  const queue = [root];

  while (queue.length > 0) {
    const levelSize = queue.length;

    for (let i = 0; i < levelSize; i++) {
      const currentNode = queue.shift();

      if (currentNode === u) {
        return i === levelSize - 1 ? null : queue.shift();
      }

      if (currentNode.left) queue.push(currentNode.left);
      if (currentNode.right) queue.push(currentNode.right);
    }
  }

  return null;
};
```

1603 Design Parking System

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1603. Design Parking System
 * https://leetcode.com/problems/design-parking-system/
 * Difficulty: Easy
 *
 * Design a parking system for a parking lot. The parking lot has three kinds of parking
 * spaces: big, medium, and small, with a fixed number of slots for each size.
 *
 * Implement the ParkingSystem class:
 * - ParkingSystem(int big, int medium, int small) Initializes object of the ParkingSystem
class.
 *   The number of slots for each parking space are given as part of the constructor.
 * - bool addCar(int carType) Checks whether there is a parking space of carType for the car
that
 *   wants to get into the parking lot. carType can be of three kinds: big, medium, or small,
which
 *   are represented by 1, 2, and 3 respectively. A car can only park in a parking space of
its
 *   carType. If there is no space available, return false, else park the car in that size
space
 *   and return true.
 */

/**
 * @param {number} big
 * @param {number} medium
 * @param {number} small
 */
var ParkingSystem = function(big, medium, small) {
```

```
  this.spaces = { 1: big, 2: medium, 3: small };
};

/**
 * @param {number} carType
 * @return {boolean}
 */
ParkingSystem.prototype.addCar = function(carType) {
  if (this.spaces[carType] > 0) {
    this.spaces[carType]--;
    return true;
  }
  return false;
};
```

1604 Alert Using Same Key Card Three Or More Times In A One Hour Period

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1604. Alert Using Same Key-Card Three or More Times in a One Hour Period
 * https://leetcode.com/problems/alert-using-same-key-card-three-or-more-times-in-a-one-hour-p
eriod/
 * Difficulty: Medium
 *
 * LeetCode company workers use key-cards to unlock office doors. Each time a worker uses
their
 * key-card, the security system saves the worker's name and the time when it was used. The
system
 * emits an alert if any worker uses the key-card three or more times in a one-hour period.
 *
 * You are given a list of strings keyName and keyTime where [keyName[i], keyTime[i]]
corresponds
 * to a person's name and the time when their key-card was used in a single day.
 *
 * Access times are given in the 24-hour time format "HH:MM", such as "23:51" and "09:49".
 *
 * Return a list of unique worker names who received an alert for frequent keycard use. Sort
the
 * names in ascending order alphabetically.
 *
 * Notice that "10:00" - "11:00" is considered to be within a one-hour period, while
 * "22:51" - "23:52" is not considered to be within a one-hour period.
 */

/**
 * @param {string[]} keyName
 * @param {string[]} keyTime
 * @return {string[]}
 */
var alertNames = function(keyName, keyTime) {
  const usageMap = new Map();
  for (let i = 0; i < keyName.length; i++) {
    if (!usageMap.has(keyName[i])) {
      usageMap.set(keyName[i], []);
    }
    usageMap.get(keyName[i]).push(timeToMinutes(keyTime[i]));
  }

  const result = [];
  for (const [name, times] of usageMap) {
    times.sort((a, b) => a - b);
    for (let i = 2; i < times.length; i++) {
      if (times[i] - times[i - 2] <= 60) {
        result.push(name);
        break;
      }
    }
  }
```

```
  return result.sort();

  function timeToMinutes(time) {
    const [hours, minutes] = time.split(':').map(Number);
    return hours * 60 + minutes;
  }
};
```

1605 Find Valid Matrix Given Row And Column Sums

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1605. Find Valid Matrix Given Row and Column Sums
 * https://leetcode.com/problems/find-valid-matrix-given-row-and-column-sums/
 * Difficulty: Medium
 *
 * You are given two arrays rowSum and colSum of non-negative integers where rowSum[i] is the
sum
 * of the elements in the ith row and colSum[j] is the sum of the elements of the jth column
of a
 * 2D matrix. In other words, you do not know the elements of the matrix, but you do know the
sums
 * of each row and column.
 *
 * Find any matrix of non-negative integers of size rowSum.length x colSum.length that
satisfies
 * the rowSum and colSum requirements.
 *
 * Return a 2D array representing any matrix that fulfills the requirements. It's guaranteed
that
 * at least one matrix that fulfills the requirements exists.
 */

/**
 * @param {number[]} rowSum
 * @param {number[]} colSum
 * @return {number[][]}
 */
var restoreMatrix = function(rowSum, colSum) {
  const rows = rowSum.length;
  const cols = colSum.length;
  const matrix = Array.from({ length: rows }, () => Array(cols).fill(0));

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      const value = Math.min(rowSum[i], colSum[j]);
      matrix[i][j] = value;
      rowSum[i] -= value;
      colSum[j] -= value;
    }
  }

  return matrix;
};
```

1606 Find Servers That Handled Most Number Of Requests

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1606. Find Servers That Handled Most Number of Requests
 * https://leetcode.com/problems/find-servers-that-handled-most-number-of-requests/
```

```
 * Difficulty: Hard
 *
 * You have k servers numbered from 0 to k-1 that are being used to handle multiple requests
 * simultaneously. Each server has infinite computational capacity but cannot handle more than
 * one request at a time. The requests are assigned to servers according to a specific
algorithm:
 * - The ith (0-indexed) request arrives.
 * - If all servers are busy, the request is dropped (not handled at all).
 * - If the (i % k)th server is available, assign the request to that server.
 * - Otherwise, assign the request to the next available server (wrapping around the list of
servers
 *   and starting from 0 if necessary). For example, if the ith server is busy, try to assign
the
 *   request to the (i+1)th server, then the (i+2)th server, and so on.
 *
 * You are given a strictly increasing array arrival of positive integers, where arrival[i]
 * represents the arrival time of the ith request, and another array load, where load[i]
represents
 * the load of the ith request (the time it takes to complete). Your goal is to find the
busiest
 * server(s). A server is considered busiest if it handled the most number of requests
successfully
 * among all the servers.
 *
 * Return a list containing the IDs (0-indexed) of the busiest server(s). You may return the
IDs
 * in any order.
 */

/**
 * @param {number} k
 * @param {number[]} arrival
 * @param {number[]} load
 * @return {number[]}
 */
var busiestServers = function(k, arrival, load) {
  const requests = new Array(k).fill(0);
  const busy = new PriorityQueue((a, b) => a[0] - b[0]);
  const right = Array.from({length: k}, (_, i) => i);
  let left = [];

  for (let i = 0; i < arrival.length; i++) {
    const time = arrival[i];
    const target = i % k;

    while (!busy.isEmpty() && busy.front()[0] <= time) {
      const serverId = busy.dequeue()[1];
      (serverId >= target) ? binaryInsert(right, serverId) : binaryInsert(left, serverId);
    }

    let assigned = -1;

    if (right.length > 0) {
      const index = binarySearch(right, target);
      if (index < right.length) {
        assigned = right[index];
        right.splice(index, 1);
      }
    }

    if (assigned === -1 && left.length > 0) {
      assigned = left[0];
      left.shift();
    }

    if (assigned !== -1) {
      requests[assigned]++;
      busy.enqueue([time + load[i], assigned]);
    }

    if ((i + 1) % k === 0) {
      right.push(...left);
      right.sort((a, b) => a - b);
      left = [];
    }
```

```
  }

  const maxRequests = Math.max(...requests);
  return requests.reduce((result, count, index) => {
    if (count === maxRequests) result.push(index);
    return result;
  }, []);
};

function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length - 1;
  let result = arr.length;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (arr[mid] >= target) {
      result = mid;
      right = mid - 1;
    } else {
      left = mid + 1;
    }
  }

  return result;
}

function binaryInsert(arr, val) {
  arr.splice(binarySearch(arr, val), 0, val);
}
```

1608 Special Array With X Elements Greater Than Or Equal X

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1608. Special Array With X Elements Greater Than or Equal X
 * https://leetcode.com/problems/special-array-with-x-elements-greater-than-or-equal-x/
 * Difficulty: Easy
 *
 * You are given an array nums of non-negative integers. nums is considered special if there
exists
 * a number x such that there are exactly x numbers in nums that are greater than or equal to
x.
 *
 * Notice that x does not have to be an element in nums.
 *
 * Return x if the array is special, otherwise, return -1. It can be proven that if nums is
special,
 * the value for x is unique.
 */

/**
 * @param {number[]} nums
 * @param {number} start
 * @param {number} end
 * @return {number}
 */
function specialArray(numbers) {
  numbers.sort((a, b) => b - a);
  let left = 0;
  let right = numbers.length;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (mid === 0 || numbers[mid - 1] >= mid) {
      if (mid === numbers.length || numbers[mid] < mid) {
        return mid;
      }
      left = mid + 1;
```

```
    } else {
      right = mid - 1;
    }
  }

  return -1;
}
```

1609 Even Odd Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1609. Even Odd Tree
 * https://leetcode.com/problems/even-odd-tree/
 * Difficulty: Medium
 *
 * A binary tree is named Even-Odd if it meets the following conditions:
 * - The root of the binary tree is at level index 0, its children are at level index 1, their
 *   children are at level index 2, etc.
 * - For every even-indexed level, all nodes at the level have odd integer values in strictly
 *   increasing order (from left to right).
 * - For every odd-indexed level, all nodes at the level have even integer values in strictly
 *   decreasing order (from left to right).
 *
 * Given the root of a binary tree, return true if the binary tree is Even-Odd, otherwise
 * return false.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isEvenOddTree = function(root) {
  let queue = [root];
  let level = 0;

  while (queue.length) {
    const levelSize = queue.length;
    let prevValue = level % 2 === 0 ? -Infinity : Infinity;
    const newQueue = [];

    for (let i = 0; i < levelSize; i++) {
      const node = queue[i];
      const isEvenLevel = level % 2 === 0;
      const isOddValue = node.val % 2 === 1;

      if (isEvenLevel && (!isOddValue || node.val <= prevValue)) return false;
      if (!isEvenLevel && (isOddValue || node.val >= prevValue)) return false;

      prevValue = node.val;
      if (node.left) newQueue.push(node.left);
      if (node.right) newQueue.push(node.right);
    }

    queue = newQueue;
    level++;
  }

  return true;
};
```

1610 Maximum Number Of Visible Points

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1610. Maximum Number of Visible Points
 * https://leetcode.com/problems/maximum-number-of-visible-points/
 * Difficulty: Hard
 *
 * You are given an array points, an integer angle, and your location, where location = [posx,
 posy]
 * and points[i] = [xi, yi] both denote integral coordinates on the X-Y plane.
 *
 * Initially, you are facing directly east from your position. You cannot move from your
position,
 * but you can rotate. In other words, posx and posy cannot be changed. Your field of view in
 * degrees is represented by angle, determining how wide you can see from any given view
direction.
 * Let d be the amount in degrees that you rotate counterclockwise. Then, your field of view
is the
 * inclusive range of angles [d - angle/2, d + angle/2].
 *
 * You can see some set of points if, for each point, the angle formed by the point, your
position,
 * and the immediate east direction from your position is in your field of view.
 *
 * There can be multiple points at one coordinate. There may be points at your location, and
you
 * can always see these points regardless of your rotation. Points do not obstruct your vision
 to
 * other points.
 *
 * Return the maximum number of points you can see.
 */

/**
 * @param {number[][]} points
 * @param {number} angle
 * @param {number[]} location
 * @return {number}
 */
var visiblePoints = function(points, angle, location) {
  const angles = [];
  let originPoints = 0;
  const [x0, y0] = location;

  for (const [x, y] of points) {
    if (x === x0 && y === y0) {
      originPoints++;
      continue;
    }
    const radian = Math.atan2(y - y0, x - x0);
    const degree = (radian * 180) / Math.PI;
    angles.push(degree);
    angles.push(degree + 360);
  }

  angles.sort((a, b) => a - b);
  const threshold = angle;
  let maxVisible = 0;
  let left = 0;

  for (let right = 0; right < angles.length; right++) {
    while (angles[right] - angles[left] > threshold) {
      left++;
    }
    maxVisible = Math.max(maxVisible, right - left + 1);
  }
```

```
  return maxVisible + originPoints;
};
```

1611 Minimum One Bit Operations To Make Integers Zero

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1611. Minimum One Bit Operations to Make Integers Zero
 * https://leetcode.com/problems/minimum-one-bit-operations-to-make-integers-zero/
 * Difficulty: Hard
 *
 * Given an integer n, you must transform it into 0 using the following operations any number
 * of times:
 * - Change the rightmost (0th) bit in the binary representation of n.
 * - Change the ith bit in the binary representation of n if the (i-1)th bit is set to 1 and
the
 *   (i-2)th through 0th bits are set to 0.
 *
 * Return the minimum number of operations to transform n into 0.
 */

/**
 * @param {number} n
 * @return {number}
 */
var minimumOneBitOperations = function(n) {
  if (n === 0) return 0;

  let result = 0;
  let bit = 1;
  while (bit <= n) {
    if (n & bit) {
      result = (1 << (bit.toString(2).length)) - 1 - result;
    }
    bit <<= 1;
  }

  return result;
};
```

1612 Check If Two Expression Trees Are Equivalent

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1612. Check If Two Expression Trees are Equivalent
 * https://leetcode.com/problems/check-if-two-expression-trees-are-equivalent/
 * Difficulty: Medium
 *
 * A binary expression tree is a kind of binary tree used to represent arithmetic expressions.
 * Each node of a binary expression tree has either zero or two children. Leaf nodes (nodes
 * with 0 children) correspond to operands (variables), and internal nodes (nodes with two
 * children) correspond to the operators. In this problem, we only consider the '+' operator
 * (i.e. addition).
 *
 * You are given the roots of two binary expression trees, root1 and root2. Return true if
 * the two binary expression trees are equivalent. Otherwise, return false.
 *
 * Two binary expression trees are equivalent if they evaluate to the same value regardless
 * of what the variables are set to.
 */

/**
 * Definition for a binary tree node.
```

```
 * function Node(val, left, right) {
 *     this.val = (val===undefined ? " " : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {Node} root1
 * @param {Node} root2
 * @return {boolean}
 */
var checkEquivalence = function(root1, root2) {
  const count1 = getVariableCount(root1);
  const count2 = getVariableCount(root2);
  const variables1 = Object.keys(count1);
  const variables2 = Object.keys(count2);

  if (variables1.length !== variables2.length) return false;

  for (const variable of variables1) {
    if (count1[variable] !== count2[variable]) return false;
  }

  return true;

  function getVariableCount(node) {
    if (!node) return {};

    if (node.val !== '+') {
      return { [node.val]: 1 };
    }

    const leftCount = getVariableCount(node.left);
    const rightCount = getVariableCount(node.right);
    const mergedCount = { ...leftCount };
    for (const variable in rightCount) {
      if (rightCount.hasOwnProperty(variable)) {
        mergedCount[variable] = (mergedCount[variable] || 0) + rightCount[variable];
      }
    }

    return mergedCount;
  }
};
```

1614 Maximum Nesting Depth Of The Parentheses

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1614. Maximum Nesting Depth of the Parentheses
 * https://leetcode.com/problems/maximum-nesting-depth-of-the-parentheses/
 * Difficulty: Easy
 *
 * Given a valid parentheses string s, return the nesting depth of s. The nesting depth is the
 * maximum number of nested parentheses.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxDepth = function(s) {
  let currentDepth = 0;
  let maxDepth = 0;

  for (const char of s) {
    if (char === '(') {
      currentDepth++;
      maxDepth = Math.max(maxDepth, currentDepth);
```

```
    } else if (char === ')') {
      currentDepth--;
    }
  }

  return maxDepth;
};
```

1615 Maximal Network Rank

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1615. Maximal Network Rank
 * https://leetcode.com/problems/maximal-network-rank/
 * Difficulty: Medium
 *
 * There is an infrastructure of n cities with some number of roads connecting these cities.
 * Each roads[i] = [ai, bi] indicates that there is a bidirectional road between cities ai and
 bi.
 *
 * The network rank of two different cities is defined as the total number of directly
connected
 * roads to either city. If a road is directly connected to both cities, it is only counted
once.
 *
 * The maximal network rank of the infrastructure is the maximum network rank of all pairs of
 * different cities.
 *
 * Given the integer n and the array roads, return the maximal network rank of the entire
 * infrastructure.
 */

/**
 * @param {number} n
 * @param {number[][]} roads
 * @return {number}
 */
var maximalNetworkRank = function(n, citiesConnections) {
  const cityRoadCounts = new Array(n).fill(0);
  const directConnections = new Set();

  for (const [cityA, cityB] of citiesConnections) {
    cityRoadCounts[cityA]++;
    cityRoadCounts[cityB]++;
    directConnections.add(`${Math.min(cityA, cityB)}-${Math.max(cityA, cityB)}`);
  }

  let maxNetworkRank = 0;

  for (let cityA = 0; cityA < n; cityA++) {
    for (let cityB = cityA + 1; cityB < n; cityB++) {
      let networkRank = cityRoadCounts[cityA] + cityRoadCounts[cityB];
      if (directConnections.has(`${cityA}-${cityB}`)
          || directConnections.has(`${cityB}-${cityA}`)) {
        networkRank--;
      }
      maxNetworkRank = Math.max(maxNetworkRank, networkRank);
    }
  }

  return maxNetworkRank;
};
```

1616 Split Two Strings To Make Palindrome

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1616. Split Two Strings to Make Palindrome
 * https://leetcode.com/problems/split-two-strings-to-make-palindrome/
 * Difficulty: Medium
 *
 * You are given two strings a and b of the same length. Choose an index and split both
strings at
 * the same index, splitting a into two strings: aprefix and asuffix where a = aprefix +
asuffix,
 * and splitting b into two strings: bprefix and bsuffix where b = bprefix + bsuffix. Check if
 * aprefix + bsuffix or bprefix + asuffix forms a palindrome.
 *
 * When you split a string s into sprefix and ssuffix, either ssuffix or sprefix is allowed to
 be
 * empty. For example, if s = "abc", then "" + "abc", "a" + "bc", "ab" + "c" , and "abc" + ""
are
 * valid splits.
 *
 * Return true if it is possible to form a palindrome string, otherwise return false.
 *
 * Notice that x + y denotes the concatenation of strings x and y.
 */

/**
 * @param {string} a
 * @param {string} b
 * @return {boolean}
 */
var checkPalindromeFormation = function(a, b) {
  return check(a, b) || check(b, a);

  function isPalindrome(str, left, right) {
    while (left < right) {
      if (str[left++] !== str[right--]) return false;
    }
    return true;
  }

  function check(str1, str2) {
    let left = 0;
    let right = str1.length - 1;

    while (left < right && str1[left] === str2[right]) {
      left++;
      right--;
    }

    return isPalindrome(str1, left, right) || isPalindrome(str2, left, right);
  }
};
```

1617 Count Subtrees With Max Distance Between Cities

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1617. Count Subtrees With Max Distance Between Cities
 * https://leetcode.com/problems/count-subtrees-with-max-distance-between-cities/
 * Difficulty: Hard
 *
 * There are n cities numbered from 1 to n. You are given an array edges of size n-1, where
```

```
 * edges[i] = [ui, vi] represents a bidirectional edge between cities ui and vi. There exists
 * a unique path between each pair of cities. In other words, the cities form a tree.
 *
 * A subtree is a subset of cities where every city is reachable from every other city in the
 * subset, where the path between each pair passes through only the cities from the subset.
 * Two subtrees are different if there is a city in one subtree that is not present in the
other.
 *
 * For each d from 1 to n-1, find the number of subtrees in which the maximum distance between
 * any two cities in the subtree is equal to d.
 *
 * Return an array of size n-1 where the dth element (1-indexed) is the number of subtrees in
 * which the maximum distance between any two cities is equal to d.
 *
 * Notice that the distance between the two cities is the number of edges in the path between
them.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[]}
 */
var countSubgraphsForEachDiameter = function(n, connections) {
  const adjacencyList = Array.from({ length: n }, () => []);
  for (const [u, v] of connections) {
    adjacencyList[u - 1].push(v - 1);
    adjacencyList[v - 1].push(u - 1);
  }

  const diameterCounts = new Array(n - 1).fill(0);

  for (let mask = 1; mask < 1 << n; mask++) {
    const selectedNodes = Array(n).fill(0);
    let nodeCount = 0;
    for (let i = 0; i < n; i++) {
      if (mask & (1 << i)) {
        selectedNodes[i] = 1;
        nodeCount++;
      }
    }

    if (nodeCount < 2) continue;

    const start = selectedNodes.findIndex(bit => bit);
    const { maxDist: dist1, farthestNode, distances } = findMaxDistance(selectedNodes, start);

    if (distances.some((d, i) => selectedNodes[i] && d === -1)) continue;

    const { maxDist: dist2 } = findMaxDistance(selectedNodes, farthestNode);

    if (dist2 > 0) {
      diameterCounts[dist2 - 1]++;
    }
  }

  return diameterCounts;

  function findMaxDistance(nodes, start) {
    const distances = new Array(n).fill(-1);
    const queue = [start];
    distances[start] = 0;
    let maxDist = 0;
    let farthestNode = start;

    while (queue.length) {
      const current = queue.shift();
      for (const neighbor of adjacencyList[current]) {
        if (nodes[neighbor] && distances[neighbor] === -1) {
          distances[neighbor] = distances[current] + 1;
          if (distances[neighbor] > maxDist) {
            maxDist = distances[neighbor];
            farthestNode = neighbor;
          }
          queue.push(neighbor);
```

```
        }
      }
    }

    return { maxDist, farthestNode, distances };
  }
};
```

1618 Maximum Font To Fit A Sentence In A Screen

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1618. Maximum Font to Fit a Sentence in a Screen
 * https://leetcode.com/problems/maximum-font-to-fit-a-sentence-in-a-screen/
 * Difficulty: Medium
 *
 * You are given a string text. We want to display text on a screen of width w and height h.
 * You can choose any font size from array fonts, which contains the available font sizes in
 * ascending order.
 *
 * You can use the FontInfo interface to get the width and height of any character at any
 * available font size.
 *
 * The FontInfo interface is defined as such:
 * interface FontInfo {
 *   // Returns the width of character ch on the screen using font size fontSize.
 *   // O(1) per call
 *   public int getWidth(int fontSize, char ch);
 *
 *   // Returns the height of any character on the screen using font size fontSize.
 *   // O(1) per call
 *   public int getHeight(int fontSize);
 * }
 *
 * The calculated width of text for some fontSize is the sum of every
 * getWidth(fontSize, text[i]) call for each 0 <= i < text.length (0-indexed).
 * The calculated height of text for some fontSize is getHeight(fontSize). Note
 * that text is displayed on a single line.
 *
 * It is guaranteed that FontInfo will return the same value if you call getHeight
 * or getWidth with the same parameters.
 *
 * It is also guaranteed that for any font size fontSize and any character ch:
 * - getHeight(fontSize) <= getHeight(fontSize+1)
 * - getWidth(fontSize, ch) <= getWidth(fontSize+1, ch)
 *
 * Return the maximum font size you can use to display text on the screen. If text cannot
 * fit on the display with any font size, return -1.
 */

/**
 * // This is the FontInfo's API interface.
 * // You should not implement it, or speculate about its implementation
 * function FontInfo() {
 *
 *  @param {number} fontSize
 *  @param {char} ch
 *      @return {number}
 *      this.getWidth = function(fontSize, ch) {
 *       ...
 *      };
 *
 *  @param {number} fontSize
 *      @return {number}
 *      this.getHeight = function(fontSize) {
 *       ...
 *      };
 * };
 */
```

```
/**
 * @param {string} text
 * @param {number} w
 * @param {number} h
 * @param {number[]} fonts
 * @param {FontInfo} fontInfo
 * @return {number}
 */
var maxFont = function(text, w, h, fonts, fontInfo) {
  let left = 0;
  let right = fonts.length - 1;
  let maxFontSize = -1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    const fontSize = fonts[mid];

    if (canFit(fontSize)) {
      maxFontSize = fontSize;
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return maxFontSize;

  function canFit(fontSize) {
    if (fontInfo.getHeight(fontSize) > h) return false;

    let totalWidth = 0;
    for (const char of text) {
      totalWidth += fontInfo.getWidth(fontSize, char);
      if (totalWidth > w) return false;
    }
    return true;
  }
};
```

1619 Mean Of Array After Removing Some Elements

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1619. Mean of Array After Removing Some Elements
 * https://leetcode.com/problems/mean-of-array-after-removing-some-elements/
 * Difficulty: Easy
 *
 * Given an integer array arr, return the mean of the remaining integers after removing the
 * smallest 5% and the largest 5% of the elements.
 *
 * Answers within 10-5 of the actual answer will be considered accepted.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var trimMean = function(numbers) {
  const sortedNumbers = numbers.sort((a, b) => a - b);
  const trimSize = numbers.length * 0.05;
  const trimmedNumbers = sortedNumbers.slice(trimSize, -trimSize);
  const sum = trimmedNumbers.reduce((acc, num) => acc + num, 0);

  return sum / trimmedNumbers.length;
};
```

1620 Coordinate With Maximum Network Quality

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1620. Coordinate With Maximum Network Quality
 * https://leetcode.com/problems/coordinate-with-maximum-network-quality/
 * Difficulty: Medium
 *
 * You are given an array of network towers towers, where towers[i] = [xi, yi, qi] denotes the
 * ith network tower with location (xi, yi) and quality factor qi. All the coordinates are
 * integral coordinates on the X-Y plane, and the distance between the two coordinates is the
 * Euclidean distance.
 *
 * You are also given an integer radius where a tower is reachable if the distance is less
than
 * or equal to radius. Outside that distance, the signal becomes garbled, and the tower is not
 * reachable.
 *
 * The signal quality of the ith tower at a coordinate (x, y) is calculated with the formula
 * ■qi / (1 + d)■, where d is the distance between the tower and the coordinate. The network
 * quality at a coordinate is the sum of the signal qualities from all the reachable towers.
 *
 * Return the array [cx, cy] representing the integral coordinate (cx, cy) where the network
 * quality is maximum. If there are multiple coordinates with the same network quality, return
 * the lexicographically minimum non-negative coordinate.
 *
 * Note:
 * - A coordinate (x1, y1) is lexicographically smaller than (x2, y2) if either:
 *   - x1 < x2, or
 *   - x1 == x2 and y1 < y2.
 * - ■val■ is the greatest integer less than or equal to val (the floor function).
 */

/**
 * @param {number[][]} towers
 * @param {number} radius
 * @return {number[]}
 */
var bestCoordinate = function(towers, radius) {
  let maxQuality = 0;
  let optimalCoord = [0, 0];

  for (let x = 0; x <= 50; x++) {
    for (let y = 0; y <= 50; y++) {
      let currentQuality = 0;

      for (const [towerX, towerY, quality] of towers) {
        const distance = calculateDistance(x, y, towerX, towerY);
        if (distance <= radius) {
          currentQuality += Math.floor(quality / (1 + distance));
        }
      }

      if (currentQuality > maxQuality) {
        maxQuality = currentQuality;
        optimalCoord = [x, y];
      } else if (currentQuality === maxQuality && currentQuality > 0) {
        if (x < optimalCoord[0] || (x === optimalCoord[0] && y < optimalCoord[1])) {
          optimalCoord = [x, y];
        }
      }
    }
  }

  return optimalCoord;

  function calculateDistance(x1, y1, x2, y2) {
    return Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2);
  }
```

```
};
```

1621 Number Of Sets Of K Non Overlapping Line Segments

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1621. Number of Sets of K Non-Overlapping Line Segments
 * https://leetcode.com/problems/number-of-sets-of-k-non-overlapping-line-segments/
 * Difficulty: Medium
 *
 * Given n points on a 1-D plane, where the ith point (from 0 to n-1) is at x = i, find the
number
 * of ways we can draw exactly k non-overlapping line segments such that each segment covers
two
 * or more points. The endpoints of each segment must have integral coordinates. The k line
segments
 * do not have to cover all n points, and they are allowed to share endpoints.
 *
 * Return the number of ways we can draw k non-overlapping line segments. Since this number
can be
 * huge, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var numberOfSets = function(n, k) {
  const MOD = 1e9 + 7;
  const comb = Array.from({ length: n + k }, () => Array(n + k).fill(0));

  for (let i = 0; i < n + k; i++) {
    comb[i][0] = 1;
    for (let j = 1; j <= i; j++) {
      comb[i][j] = (comb[i - 1][j - 1] + comb[i - 1][j]) % MOD;
    }
  }

  return comb[n + k - 1][2 * k];
};
```

1622 Fancy Sequence

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1622. Fancy Sequence
 * https://leetcode.com/problems/fancy-sequence/
 * Difficulty: Hard
 *
 * Write an API that generates fancy sequences using the append, addAll, and multAll
operations.
 *
 * Implement the Fancy class:
 * - Fancy() Initializes the object with an empty sequence.
 * - void append(val) Appends an integer val to the end of the sequence.
 * - void addAll(inc) Increments all existing values in the sequence by an integer inc.
 * - void multAll(m) Multiplies all existing values in the sequence by an integer m.
 * - int getIndex(idx) Gets the current value at index idx (0-indexed) of the sequence modulo
 *   109 + 7. If the index is greater or equal than the length of the sequence, return -1.
 */

var Fancy = function() {
```

```
  this.sequence = [];
  this.additive = 0n;
  this.multiplicative = 1n;
  this.MOD = 1000000007n;
};

/**
 * @param {number} val
 * @return {void}
 */
Fancy.prototype.append = function(val) {
  const valBig = BigInt(val);
  const inverse = this.modInverse(this.multiplicative);
  const adjustedVal = ((valBig - this.additive + this.MOD) * inverse) % this.MOD;
  this.sequence.push(adjustedVal);
};

/**
 * @param {number} inc
 * @return {void}
 */
Fancy.prototype.addAll = function(inc) {
  this.additive = (this.additive + BigInt(inc)) % this.MOD;
};

/**
 * @param {number} m
 * @return {void}
 */
Fancy.prototype.multAll = function(m) {
  const mBig = BigInt(m);
  this.additive = (this.additive * mBig) % this.MOD;
  this.multiplicative = (this.multiplicative * mBig) % this.MOD;
};

/**
 * @param {number} idx
 * @return {number}
 */
Fancy.prototype.getIndex = function(idx) {
  if (idx >= this.sequence.length) return -1;
  return Number((this.sequence[idx] * this.multiplicative + this.additive) % this.MOD);
};

/**
 * @param {number} a
 * @return {number}
 */
Fancy.prototype.modInverse = function(a) {
  let m = this.MOD;
  const m0 = m;
  let x = 1n;
  let y = 0n;

  a = a % m;
  while (a > 1n) {
    const q = a / m;
    [a, m] = [m, a % m];
    [x, y] = [y, x - q * y];
  }

  return x < 0n ? x + m0 : x;
};
```

1624 Largest Substring Between Two Equal Characters

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1624. Largest Substring Between Two Equal Characters
```

```
 * https://leetcode.com/problems/largest-substring-between-two-equal-characters/
 * Difficulty: Easy
 *
 * Given a string s, return the length of the longest substring between two equal characters,
 * excluding the two characters. If there is no such substring return -1.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxLengthBetweenEqualCharacters = function(s) {
  const charFirstIndex = new Map();
  let maxLength = -1;

  for (let i = 0; i < s.length; i++) {
    const char = s[i];
    if (charFirstIndex.has(char)) {
      maxLength = Math.max(maxLength, i - charFirstIndex.get(char) - 1);
    } else {
      charFirstIndex.set(char, i);
    }
  }

  return maxLength;
};
```

1625 Lexicographically Smallest String After Applying Operations

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1625. Lexicographically Smallest String After Applying Operations
 * https://leetcode.com/problems/lexicographically-smallest-string-after-applying-operations/
 * Difficulty: Medium
 *
 * You are given a string s of even length consisting of digits from 0 to 9, and two integers
a
 * and b.
 *
 * You can apply either of the following two operations any number of times and in any order
on s:
 * - Add a to all odd indices of s (0-indexed). Digits post 9 are cycled back to 0. For
example,
 *   if s = "3456" and a = 5, s becomes "3951".
 * - Rotate s to the right by b positions. For example, if s = "3456" and b = 1, s becomes
"6345".
 *
 * Return the lexicographically smallest string you can obtain by applying the above
operations any
 * number of times on s.
 *
 * A string a is lexicographically smaller than a string b (of the same length) if in the
first
 * position where a and b differ, string a has a letter that appears earlier in the alphabet
than
 * the corresponding letter in b. For example, "0158" is lexicographically smaller than "0190"
 * because the first position they differ is at the third letter, and '5' comes before '9'.
 */

/**
 * @param {string} s
 * @param {number} a
 * @param {number} b
 * @return {string}
 */
var findLexSmallestString = function(s, a, b) {
```

```
  const visited = new Set();
  let smallestString = s;
  const queue = [s];

  const addOperation = str => {
    const chars = str.split('');
    for (let i = 1; i < str.length; i += 2) {
      chars[i] = String((parseInt(chars[i]) + a) % 10);
    }
    return chars.join('');
  };

  const rotateOperation = str => {
    return str.slice(-b) + str.slice(0, -b);
  };

  while (queue.length) {
    const current = queue.shift();
    if (visited.has(current)) continue;
    visited.add(current);
    if (current < smallestString) smallestString = current;

    const added = addOperation(current);
    const rotated = rotateOperation(current);

    queue.push(added);
    queue.push(rotated);
  }

  return smallestString;
};
```

1626 Best Team With No Conflicts

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1626. Best Team With No Conflicts
 * https://leetcode.com/problems/best-team-with-no-conflicts/
 * Difficulty: Medium
 *
 * You are the manager of a basketball team. For the upcoming tournament, you want to choose
the
 * team with the highest overall score. The score of the team is the sum of scores of all the
 * players in the team.
 *
 * However, the basketball team is not allowed to have conflicts. A conflict exists if a
younger
 * player has a strictly higher score than an older player. A conflict does not occur between
 * players of the same age.
 *
 * Given two lists, scores and ages, where each scores[i] and ages[i] represents the score and
 * age of the ith player, respectively, return the highest overall score of all possible
basketball
 * teams.
 */

/**
 * @param {number[]} scores
 * @param {number[]} ages
 * @return {number}
 */
var bestTeamScore = function(scores, ages) {
  const players = ages.map((age, index) => ({ age, score: scores[index] }));
  players.sort((a, b) => a.age - b.age || a.score - b.score);

  const maxScores = new Array(players.length).fill(0);
  let highestScore = 0;

  for (let current = 0; current < players.length; current++) {
```

```
    maxScores[current] = players[current].score;
    for (let previous = 0; previous < current; previous++) {
      if (players[previous].score <= players[current].score) {
        maxScores[current] = Math.max(
          maxScores[current],
          maxScores[previous] + players[current].score
        );
      }
    }
    highestScore = Math.max(highestScore, maxScores[current]);
  }

  return highestScore;
};
```

1627 Graph Connectivity With Threshold

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1627. Graph Connectivity With Threshold
 * https://leetcode.com/problems/graph-connectivity-with-threshold/
 * Difficulty: Hard
 *
 * We have n cities labeled from 1 to n. Two different cities with labels x and y are directly
 * connected by a bidirectional road if and only if x and y share a common divisor strictly
 * greater than some threshold. More formally, cities with labels x and y have a road between
 * them if there exists an integer z such that all of the following are true:
 * - x % z == 0,
 * - y % z == 0, and
 * - z > threshold.
 *
 * Given the two integers, n and threshold, and an array of queries, you must determine for
each
 * queries[i] = [ai, bi] if cities ai and bi are connected directly or indirectly. (i.e. there
 * is some path between them).
 *
 * Return an array answer, where answer.length == queries.length and answer[i] is true if for
 * the ith query, there is a path between ai and bi, or answer[i] is false if there is no
path.
 */

/**
 * @param {number} n
 * @param {number} threshold
 * @param {number[][]} queries
 * @return {boolean[]}
 */
var areConnected = function(n, threshold, queries) {
  const parent = new Array(n + 1).fill().map((_, i) => i);

  for (let z = threshold + 1; z <= n; z++) {
    for (let x = z; x <= n; x += z) {
      union(x, z);
    }
  }

  return queries.map(([a, b]) => find(a) === find(b));

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    parent[find(x)] = find(y);
  }
};
```

1628 Design An Expression Tree With Evaluate Function

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1628. Design an Expression Tree With Evaluate Function
 * https://leetcode.com/problems/design-an-expression-tree-with-evaluate-function/
 * Difficulty: Medium
 *
 * Given the postfix tokens of an arithmetic expression, build and return the binary
expression
 * tree that represents this expression.
 *
 * Postfix notation is a notation for writing arithmetic expressions in which the operands
 * (numbers) appear before their operators. For example, the postfix tokens of the expression
 * 4*(5-(7+2)) are represented in the array postfix = ["4","5","7","2","+","-","*"].
 *
 * The class Node is an interface you should use to implement the binary expression tree.
 * The returned tree will be tested using the evaluate function, which is supposed to evaluate
 * the tree's value. You should not remove the Node class; however, you can modify it as you
 * wish, and you can define other classes to implement it if needed.
 *
 * A binary expression tree is a kind of binary tree used to represent arithmetic expressions.
 * Each node of a binary expression tree has either zero or two children. Leaf nodes (nodes
 * with 0 children) correspond to operands (numbers), and internal nodes (nodes with two
 * children) correspond to the operators '+' (addition), '-' (subtraction), '*'
 * (multiplication), and '/' (division).
 *
 * It's guaranteed that no subtree will yield a value that exceeds 109 in absolute value,
 * and all the operations are valid (i.e., no division by zero).
 *
 * Follow up: Could you design the expression tree such that it is more modular? For example,
 * is your design able to support additional operators without making changes to your existing
 * evaluate implementation?
 */

/**
 * This is the interface for the expression tree Node.
 * You should not remove it, and you can define some classes to implement it.
 */
var Node = function() {
  if (this.constructor === Node) {
    throw new Error('Cannot instanciate abstract class');
  }
};

Node.prototype.evaluate = function() {
  throw new Error('Cannot call abstract method');
};

class NumberNode extends Node {
  constructor(value) {
    super();
    this.value = parseInt(value);
  }

  evaluate() {
    return this.value;
  }
}

class OperatorNode extends Node {
  constructor(operator, left, right) {
    super();
    this.operator = operator;
    this.left = left;
    this.right = right;
  }

  evaluate() {
```

```
    const leftValue = this.left.evaluate();
    const rightValue = this.right.evaluate();

    switch (this.operator) {
      case '+':
        return leftValue + rightValue;
      case '-':
        return leftValue - rightValue;
      case '*':
        return leftValue * rightValue;
      case '/':
        return Math.floor(leftValue / rightValue);
      default:
        throw new Error('Unknown operator');
    }
  }
}

/**
 * This is the TreeBuilder class.
 * You can treat it as the driver code that takes the postinfix input
 * and returns the expression tree representing it as a Node.
 */
class TreeBuilder {
  buildTree(postfix) {
    const stack = [];
    const operators = new Set(['+', '-', '*', '/']);

    for (const token of postfix) {
      if (operators.has(token)) {
        const right = stack.pop();
        const left = stack.pop();
        stack.push(new OperatorNode(token, left, right));
      } else {
        stack.push(new NumberNode(token));
      }
    }

    return stack[0];
  }
}
```

1629 Slowest Key

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1629. Slowest Key
 * https://leetcode.com/problems/slowest-key/
 * Difficulty: Easy
 *
 * A newly designed keypad was tested, where a tester pressed a sequence of n keys, one at a
time.
 *
 * You are given a string keysPressed of length n, where keysPressed[i] was the ith key
pressed in
 * the testing sequence, and a sorted list releaseTimes, where releaseTimes[i] was the time
the
 * ith key was released. Both arrays are 0-indexed. The 0th key was pressed at the time 0, and
 * every subsequent key was pressed at the exact time the previous key was released.
 *
 * The tester wants to know the key of the keypress that had the longest duration. The ith
keypress
 * had a duration of releaseTimes[i] - releaseTimes[i - 1], and the 0th keypress had a
duration of
 * releaseTimes[0].
 *
 * Note that the same key could have been pressed multiple times during the test, and these
multiple
 * presses of the same key may not have had the same duration.
```

```
 *
 * Return the key of the keypress that had the longest duration. If there are multiple such
 * keypresses, return the lexicographically largest key of the keypresses.
 */

/**
 * @param {number[]} releaseTimes
 * @param {string} keysPressed
 * @return {character}
 */
var slowestKey = function(releaseTimes, keysPressed) {
  let maxDuration = releaseTimes[0];
  let result = keysPressed[0];

  for (let i = 1; i < releaseTimes.length; i++) {
    const duration = releaseTimes[i] - releaseTimes[i - 1];
    if (duration > maxDuration || (duration === maxDuration && keysPressed[i] > result)) {
      maxDuration = duration;
      result = keysPressed[i];
    }
  }

  return result;
};
```

1630 Arithmetic Subarrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1630. Arithmetic Subarrays
 * https://leetcode.com/problems/arithmetic-subarrays/
 * Difficulty: Medium
 *
 * A sequence of numbers is called arithmetic if it consists of at least two elements, and the
 * difference between every two consecutive elements is the same. More formally, a sequence s
 * is arithmetic if and only if s[i+1] - s[i] == s[1] - s[0] for all valid i.
 *
 * For example, these are arithmetic sequences:
 * - 1, 3, 5, 7, 9
 * - 7, 7, 7, 7
 * - 3, -1, -5, -9
 *
 * The following sequence is not arithmetic:
 * - 1, 1, 2, 5, 7
 *
 * You are given an array of n integers, nums, and two arrays of m integers each, l and r,
 * representing the m range queries, where the ith query is the range [l[i], r[i]]. All the
 * arrays are 0-indexed.
 *
 * Return a list of boolean elements answer, where answer[i] is true if the subarray
 * nums[l[i]], nums[l[i]+1], ... , nums[r[i]] can be rearranged to form an arithmetic
 * sequence, and false otherwise.
 */

/**
 * @param {number[]} nums
 * @param {number[]} l
 * @param {number[]} r
 * @return {boolean[]}
 */
var checkArithmeticSubarrays = function(nums, l, r) {
  return l.map((start, index) => {
    const end = r[index];
    const subarray = nums.slice(start, end + 1);
    return isArithmetic(subarray);
  });

  function isArithmetic(subarray) {
    if (subarray.length < 2) return false;
```

```
    subarray.sort((a, b) => a - b);
    const diff = subarray[1] - subarray[0];
    for (let i = 2; i < subarray.length; i++) {
      if (subarray[i] - subarray[i - 1] !== diff) return false;
    }
    return true;
  }
};
```

1631 Path With Minimum Effort

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1631. Path With Minimum Effort
 * https://leetcode.com/problems/path-with-minimum-effort/
 * Difficulty: Medium
 *
 * You are a hiker preparing for an upcoming hike. You are given heights, a 2D array of size
 * rows x columns, where heights[row][col] represents the height of cell (row, col). You are
 * situated in the top-left cell, (0, 0), and you hope to travel to the bottom-right cell,
 * (rows-1, columns-1) (i.e., 0-indexed). You can move up, down, left, or right, and you wish
 * to find a route that requires the minimum effort.
 *
 * A route's effort is the maximum absolute difference in heights between two consecutive
 * cells of the route.
 *
 * Return the minimum effort required to travel from the top-left cell to the bottom-right
cell.
 */

/**
 * @param {number[][]} heights
 * @return {number}
 */
var minimumEffortPath = function(heights) {
  const rows = heights.length;
  const cols = heights[0].length;
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  function canReach(maxEffort) {
    const visited = Array.from({ length: rows }, () => Array(cols).fill(false));
    const queue = [[0, 0]];
    visited[0][0] = true;

    while (queue.length) {
      const [row, col] = queue.shift();
      if (row === rows - 1 && col === cols - 1) return true;

      for (const [dr, dc] of directions) {
        const newRow = row + dr;
        const newCol = col + dc;

        if (newRow >= 0 && newRow < rows && newCol >= 0
            && newCol < cols && !visited[newRow][newCol]) {
          const effort = Math.abs(heights[newRow][newCol] - heights[row][col]);
          if (effort <= maxEffort) {
            visited[newRow][newCol] = true;
            queue.push([newRow, newCol]);
          }
        }
      }
    }
    return false;
  }

  let left = 0;
  let right = 1000000;
  let result = right;
```

```
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (canReach(mid)) {
      result = mid;
      right = mid - 1;
    } else {
      left = mid + 1;
    }
  }

  return result;
};
```

1632 Rank Transform Of A Matrix

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1632. Rank Transform of a Matrix
 * https://leetcode.com/problems/rank-transform-of-a-matrix/
 * Difficulty: Hard
 *
 * Given an m x n matrix, return a new matrix answer where answer[row][col] is the rank of
 * matrix[row][col].
 *
 * The rank is an integer that represents how large an element is compared to other elements.
 * It is calculated using the following rules:
 * - The rank is an integer starting from 1.
 * - If two elements p and q are in the same row or column, then:
 *   - If p < q then rank(p) < rank(q)
 *   - If p == q then rank(p) == rank(q)
 *   - If p > q then rank(p) > rank(q)
 * - The rank should be as small as possible.
 *
 * The test cases are generated so that answer is unique under the given rules.
 */

/**
 * @param {number[][]} matrix
 * @return {number[][]}
 */
var matrixRankTransform = function(matrix) {
  const rows = matrix.length;
  const cols = matrix[0].length;
  const result = Array.from({ length: rows }, () => new Array(cols).fill(0));
  const parent = new Array(rows * cols).fill(-1);
  const elements = [];

  for (let row = 0; row < rows; row++) {
    const valueToCoords = new Map();
    for (let col = 0; col < cols; col++) {
      const value = matrix[row][col];
      if (!valueToCoords.has(value)) valueToCoords.set(value, []);
      valueToCoords.get(value).push([row, col]);
    }
    for (const coords of valueToCoords.values()) {
      for (let i = 1; i < coords.length; i++) {
        const [r1, c1] = coords[0];
        const [r2, c2] = coords[i];
        union(r1 * cols + c1, r2 * cols + c2);
      }
    }
  }

  for (let col = 0; col < cols; col++) {
    const valueToCoords = new Map();
    for (let row = 0; row < rows; row++) {
      const value = matrix[row][col];
      if (!valueToCoords.has(value)) valueToCoords.set(value, []);
```

```
    valueToCoords.get(value).push([row, col]);
  }
  for (const coords of valueToCoords.values()) {
    for (let i = 1; i < coords.length; i++) {
      const [r1, c1] = coords[0];
      const [r2, c2] = coords[i];
      union(r1 * cols + c1, r2 * cols + c2);
    }
  }
}

for (let row = 0; row < rows; row++) {
  for (let col = 0; col < cols; col++) {
    elements.push([matrix[row][col], row, col]);
  }
}
elements.sort((a, b) => a[0] - b[0]);

const rowMaxRank = new Array(rows).fill(0);
const colMaxRank = new Array(cols).fill(0);
const groups = new Map();

for (let i = 0; i < elements.length; i++) {
  const [value, row, col] = elements[i];
  const root = find(row * cols + col);
  if (!groups.has(root)) groups.set(root, []);
  groups.get(root).push([row, col]);
}

let i = 0;
while (i < elements.length) {
  const value = elements[i][0];
  const currentGroups = new Map();
  while (i < elements.length && elements[i][0] === value) {
    const [, row, col] = elements[i];
    const root = find(row * cols + col);
    if (!currentGroups.has(root)) currentGroups.set(root, []);
    currentGroups.get(root).push([row, col]);
    i++;
  }

  for (const [root, coords] of currentGroups) {
    let maxRank = 0;
    for (const [row, col] of coords) {
      maxRank = Math.max(maxRank, rowMaxRank[row], colMaxRank[col]);
    }
    const rank = maxRank + 1;
    for (const [row, col] of coords) {
      result[row][col] = rank;
      rowMaxRank[row] = rank;
      colMaxRank[col] = rank;
    }
  }
}

return result;

function find(x) {
  if (parent[x] < 0) return x;
  return parent[x] = find(parent[x]);
}

function union(x, y) {
  const px = find(x);
  const py = find(y);

  if (px !== py) {
    if (parent[px] < parent[py]) {
      parent[px] += parent[py];
      parent[py] = px;
    } else {
      parent[py] += parent[px];
      parent[px] = py;
    }
  }
```

```
  }
};
```

1634 Add Two Polynomials Represented As Linked Lists

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1634. Add Two Polynomials Represented as Linked Lists
 * https://leetcode.com/problems/add-two-polynomials-represented-as-linked-lists/
 * Difficulty: Medium
 *
 * A polynomial linked list is a special type of linked list where every node represents a
 * term in a polynomial expression.
 *
 * Each node has three attributes:
 * - coefficient: an integer representing the number multiplier of the term. The coefficient
 *   of the term 9x4 is 9.
 * - power: an integer representing the exponent. The power of the term 9x4 is 4.
 * - next: a pointer to the next node in the list, or null if it is the last node of the list.
 *
 * For example, the polynomial 5x3 + 4x - 7 is represented by the polynomial linked list
 * illustrated below:
 *
 * The polynomial linked list must be in its standard form: the polynomial must be in strictly
 * descending order by its power value. Also, terms with a coefficient of 0 are omitted.
 *
 * Given two polynomial linked list heads, poly1 and poly2, add the polynomials together
 * and return the head of the sum of the polynomials.
 *
 * PolyNode format:
 * The input/output format is as a list of n nodes, where each node is represented as
 * its [coefficient, power]. For example, the polynomial 5x3 + 4x - 7 would be represented
 * as: [[5,3],[4,1],[-7,0]].
 */

/**
 * Definition for polynomial singly-linked list.
 * function PolyNode(x=0, y=0, next=null) {
 *     this.coefficient = x;
 *     this.power = y;
 *     this.next = next;
 * }
 */

/**
 * @param {PolyNode} poly1
 * @param {PolyNode} poly2
 * @return {PolyNode}
 */
var addPoly = function(poly1, poly2) {
  const dummyHead = new PolyNode();
  let current = dummyHead;
  let pointer1 = poly1;
  let pointer2 = poly2;

  while (pointer1 || pointer2) {
    let coefficient = 0;
    let power = 0;

    if (!pointer1) {
      coefficient = pointer2.coefficient;
      power = pointer2.power;
      pointer2 = pointer2.next;
    } else if (!pointer2) {
      coefficient = pointer1.coefficient;
      power = pointer1.power;
      pointer1 = pointer1.next;
    } else if (pointer1.power > pointer2.power) {
      coefficient = pointer1.coefficient;
```

```
      power = pointer1.power;
      pointer1 = pointer1.next;
    } else if (pointer2.power > pointer1.power) {
      coefficient = pointer2.coefficient;
      power = pointer2.power;
      pointer2 = pointer2.next;
    } else {
      coefficient = pointer1.coefficient + pointer2.coefficient;
      power = pointer1.power;
      pointer1 = pointer1.next;
      pointer2 = pointer2.next;
    }

    if (coefficient !== 0) {
      current.next = new PolyNode(coefficient, power);
      current = current.next;
    }
  }

  return dummyHead.next;
};
```

1636 Sort Array By Increasing Frequency

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1636. Sort Array by Increasing Frequency
 * https://leetcode.com/problems/sort-array-by-increasing-frequency/
 * Difficulty: Easy
 *
 * Given an array of integers nums, sort the array in increasing order based on the frequency
 * of the values. If multiple values have the same frequency, sort them in decreasing order.
 *
 * Return the sorted array.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var frequencySort = function(nums) {
  const map = new Map();
  nums.forEach(num => map.set(num, (map.get(num) || 0) + 1));

  return nums.sort((a, b) => {
    const freqA = map.get(a);
    const freqB = map.get(b);
    return freqA === freqB ? b - a : freqA - freqB;
  });
};
```

1637 Widest Vertical Area Between Two Points Containing No Points

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1637. Widest Vertical Area Between Two Points Containing No Points
 * https://leetcode.com/problems/widest-vertical-area-between-two-points-containing-no-points/
 * Difficulty: Easy
 *
 * Given n points on a 2D plane where points[i] = [xi, yi], Return the widest vertical area
between
 * two points such that no points are inside the area.
 *
```

```
 * A vertical area is an area of fixed-width extending infinitely along the y-axis (i.e.,
infinite
 * height). The widest vertical area is the one with the maximum width.
 *
 * Note that points on the edge of a vertical area are not considered included in the area.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var maxWidthOfVerticalArea = function(points) {
  points.sort((a, b) => a[0] - b[0]);
  let result = 0;
  for (let i = 1; i < points.length; i++) {
    result = Math.max(result, points[i][0] - points[i - 1][0]);
  }
  return result;
};
```

1638 Count Substrings That Differ By One Character

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1638. Count Substrings That Differ by One Character
 * https://leetcode.com/problems/count-substrings-that-differ-by-one-character/
 * Difficulty: Medium
 *
 * Given two strings s and t, find the number of ways you can choose a non-empty substring of
s
 * and replace a single character by a different character such that the resulting substring
is
 * a substring of t. In other words, find the number of substrings in s that differ from some
 * substring in t by exactly one character.
 *
 * For example, the underlined substrings in "computer" and "computation" only differ by the
 * 'e'/'a', so this is a valid way.
 *
 * Return the number of substrings that satisfy the condition above.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {number}
 */
var countSubstrings = function(s, t) {
  let result = 0;

  for (let i = 0; i < s.length; i++) {
    for (let j = 0; j < t.length; j++) {
      let diffCount = 0;
      for (let k = 0; i + k < s.length && j + k < t.length; k++) {
        if (s[i + k] !== t[j + k]) diffCount++;
        if (diffCount === 1) result++;
        if (diffCount > 1) break;
      }
    }
  }

  return result;
};
```

1639 Number Of Ways To Form A Target String Given A Dictionary

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1639. Number of Ways to Form a Target String Given a Dictionary
 * https://leetcode.com/problems/number-of-ways-to-form-a-target-string-given-a-dictionary/
 * Difficulty: Hard
 *
 * You are given a list of strings of the same length words and a string target.
 *
 * Your task is to form target using the given words under the following rules:
 * - target should be formed from left to right.
 * - To form the ith character (0-indexed) of target, you can choose the kth character of the
jth
 *   string in words if target[i] = words[j][k].
 * - Once you use the kth character of the jth string of words, you can no longer use the xth
 *   character of any string in words where x <= k. In other words, all characters to the left
 of
 *   or at index k become unusuable for every string.
 * - Repeat the process until you form the string target.
 *
 * Notice that you can use multiple characters from the same string in words provided the
 * conditions above are met.
 *
 * Return the number of ways to form target from words. Since the answer may be too large,
return
 * it modulo 109 + 7.
 */

/**
 * @param {string[]} words
 * @param {string} target
 * @return {number}
 */
var numWays = function(words, target) {
  const MOD = 1e9 + 7;
  const wordLength = words[0].length;
  const targetLength = target.length;
  const charCounts = Array(wordLength).fill().map(() => Array(26).fill(0));

  for (const word of words) {
    for (let i = 0; i < wordLength; i++) {
      charCounts[i][word.charCodeAt(i) - 97]++;
    }
  }

  const dp = Array(targetLength + 1).fill().map(() => Array(wordLength + 1).fill(0));
  dp[0][0] = 1;

  for (let i = 0; i <= targetLength; i++) {
    for (let j = 0; j < wordLength; j++) {
      if (i < targetLength) {
        const charIndex = target.charCodeAt(i) - 97;
        dp[i + 1][j + 1] = (dp[i + 1][j + 1] + dp[i][j] * charCounts[j][charIndex]) % MOD;
      }
      dp[i][j + 1] = (dp[i][j + 1] + dp[i][j]) % MOD;
    }
  }

  return dp[targetLength][wordLength];
};
```

1640 Check Array Formation Through Concatenation

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1640. Check Array Formation Through Concatenation
 * https://leetcode.com/problems/check-array-formation-through-concatenation/
 * Difficulty: Easy
 *
 * You are given an array of distinct integers arr and an array of integer arrays pieces,
 * where the integers in pieces are distinct. Your goal is to form arr by concatenating
 * the arrays in pieces in any order. However, you are not allowed to reorder the integers
 * in each array pieces[i].
 *
 * Return true if it is possible to form the array arr from pieces. Otherwise, return false.
 */

/**
 * @param {number[]} arr
 * @param {number[][]} pieces
 * @return {boolean}
 */
var canFormArray = function(arr, pieces) {
  const numberToPiece = new Map();
  for (const piece of pieces) {
    numberToPiece.set(piece[0], piece);
  }

  let index = 0;
  while (index < arr.length) {
    const currentNumber = arr[index];
    if (!numberToPiece.has(currentNumber)) {
      return false;
    }
    const piece = numberToPiece.get(currentNumber);
    for (const num of piece) {
      if (arr[index] !== num) {
        return false;
      }
      index++;
    }
  }

  return true;
};
```

1641 Count Sorted Vowel Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1641. Count Sorted Vowel Strings
 * https://leetcode.com/problems/count-sorted-vowel-strings/
 * Difficulty: Medium
 *
 * Given an integer n, return the number of strings of length n that consist only of vowels
 * (a, e, i, o, u) and are lexicographically sorted.
 *
 * A string s is lexicographically sorted if for all valid i, s[i] is the same as or comes
 * before s[i+1] in the alphabet.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countVowelStrings = function(n) {
  const vowelCounts = [1, 1, 1, 1, 1];

  for (let i = 1; i < n; i++) {
```

```
    for (let j = 3; j >= 0; j--) {
      vowelCounts[j] += vowelCounts[j + 1];
    }
  }

  return vowelCounts.reduce((sum, count) => sum + count, 0);
};
```

1642 Furthest Building You Can Reach

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1642. Furthest Building You Can Reach
 * https://leetcode.com/problems/furthest-building-you-can-reach/
 * Difficulty: Medium
 *
 * You are given an integer array heights representing the heights of buildings, some bricks,
 * and some ladders.
 *
 * You start your journey from building 0 and move to the next building by possibly using
 * bricks or ladders.
 *
 * While moving from building i to building i+1 (0-indexed):
 * - If the current building's height is greater than or equal to the next building's height,
 *   you do not need a ladder or bricks.
 * - If the current building's height is less than the next building's height, you can either
 *   use one ladder or (h[i+1] - h[i]) bricks.
 *
 * Return the furthest building index (0-indexed) you can reach if you use the given ladders
 * and bricks optimally.
 */

/**
 * @param {number[]} heights
 * @param {number} bricks
 * @param {number} ladders
 * @return {number}
 */
var furthestBuilding = function(heights, bricks, ladders) {
  const minHeap = new MinPriorityQueue();

  for (let i = 0; i < heights.length - 1; i++) {
    const climb = heights[i + 1] - heights[i];
    if (climb > 0) {
      minHeap.push(climb);
      if (minHeap.size() > ladders) {
        bricks -= minHeap.pop();
        if (bricks < 0) {
          return i;
        }
      }
    }
  }

  return heights.length - 1;
};
```

1643 Kth Smallest Instructions

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1643. Kth Smallest Instructions
 * https://leetcode.com/problems/kth-smallest-instructions/
```

```
 * Difficulty: Hard
 *
 * Bob is standing at cell (0, 0), and he wants to reach destination: (row, column). He can
only
 * travel right and down. You are going to help Bob by providing instructions for him to reach
 * destination.
 *
 * The instructions are represented as a string, where each character is either:
 * - 'H', meaning move horizontally (go right), or
 * - 'V', meaning move vertically (go down).
 *
 * Multiple instructions will lead Bob to destination. For example, if destination is (2, 3),
 * both "HHHVV" and "HVHVH" are valid instructions.
 *
 * However, Bob is very picky. Bob has a lucky number k, and he wants the kth
lexicographically
 * smallest instructions that will lead him to destination. k is 1-indexed.
 *
 * Given an integer array destination and an integer k, return the kth lexicographically
 * smallest instructions that will take Bob to destination.
 */

/**
 * @param {number[]} destination
 * @param {number} k
 * @return {string}
 */
var kthSmallestPath = function(destination, k) {
  let verticalMoves = destination[0];
  let horizontalMoves = destination[1];
  const totalMoves = verticalMoves + horizontalMoves;
  let path = '';

  for (let i = 0; i < totalMoves; i++) {
    if (horizontalMoves === 0) {
      path += 'V';
      verticalMoves--;
      continue;
    }
    const combinations = calculateCombinations(
      verticalMoves + horizontalMoves - 1,
      horizontalMoves - 1
    );
    if (k <= combinations) {
      path += 'H';
      horizontalMoves--;
    } else {
      path += 'V';
      verticalMoves--;
      k -= combinations;
    }
  }

  return path;
};

function calculateCombinations(n, r) {
  if (r < 0 || r > n) return 0;
  let result = 1;
  for (let i = 1; i <= r; i++) {
    result = result * (n - i + 1) / i;
  }
  return Math.floor(result);
}
```

1644 Lowest Common Ancestor Of A Binary Tree Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
```

```
 * 1644. Lowest Common Ancestor of a Binary Tree II
 * https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree-ii/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the lowest common ancestor (LCA) of two given
 * nodes, p and q. If either node p or q does not exist in the tree, return null. All
 * values of the nodes in the tree are unique.
 *
 * According to the definition of LCA on Wikipedia: "The lowest common ancestor of two
 * nodes p and q in a binary tree T is the lowest node that has both p and q as descendants
 * (where we allow a node to be a descendant of itself)". A descendant of a node x is a
 * node y that is on the path from node x to some leaf node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, p, q) {
  let foundP = false;
  let foundQ = false;

  const lca = findLCA(root);
  return foundP && foundQ ? lca : null;

  function findLCA(node) {
    if (!node) return null;

    const leftResult = findLCA(node.left);
    const rightResult = findLCA(node.right);

    let currentMatch = false;
    if (node === p) {
      foundP = true;
      currentMatch = true;
    }

    if (node === q) {
      foundQ = true;
      currentMatch = true;
    }

    if (leftResult && rightResult) {
      return node;
    }

    if (currentMatch && (leftResult || rightResult)) {
      return node;
    }

    return leftResult || rightResult || (currentMatch ? node : null);
  }
};
```

1646 Get Maximum In Generated Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1646. Get Maximum in Generated Array
 * https://leetcode.com/problems/get-maximum-in-generated-array/
```

```
 * Difficulty: Easy
 *
 * You are given an integer n. A 0-indexed integer array nums of length n + 1 is generated
 * in the following way:
 * - nums[0] = 0
 * - nums[1] = 1
 * - nums[2 * i] = nums[i] when 2 <= 2 * i <= n
 * - nums[2 * i + 1] = nums[i] + nums[i + 1] when 2 <= 2 * i + 1 <= n
 *
 * Return the maximum integer in the array nums.
 */

/**
 * @param {number} n
 * @return {number}
 */
var getMaximumGenerated = function(n) {
  if (n === 0) return 0;

  const generatedArray = new Array(n + 1).fill(0);
  generatedArray[1] = 1;
  let maximumValue = 1;

  for (let i = 1; i <= Math.floor(n / 2); i++) {
    if (2 * i <= n) {
      generatedArray[2 * i] = generatedArray[i];
      maximumValue = Math.max(maximumValue, generatedArray[2 * i]);
    }
    if (2 * i + 1 <= n) {
      generatedArray[2 * i + 1] = generatedArray[i] + generatedArray[i + 1];
      maximumValue = Math.max(maximumValue, generatedArray[2 * i + 1]);
    }
  }

  return maximumValue;
};
```

1647 Minimum Deletions To Make Character Frequencies Unique

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1647. Minimum Deletions to Make Character Frequencies Unique
 * https://leetcode.com/problems/minimum-deletions-to-make-character-frequencies-unique/
 * Difficulty: Medium
 *
 * A string s is called good if there are no two different characters in s that have the
 * same frequency.
 *
 * Given a string s, return the minimum number of characters you need to delete to make s
good.
 *
 * The frequency of a character in a string is the number of times it appears in the string.
 * For example, in the string "aab", the frequency of 'a' is 2, while the frequency of 'b' is
1.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minDeletions = function(s) {
  const charFrequencies = new Array(26).fill(0);
  for (const char of s) {
    charFrequencies[char.charCodeAt(0) - 97]++;
  }

  charFrequencies.sort((a, b) => b - a);
  let result = 0;
```

```
  const usedFrequencies = new Set();

  for (const freq of charFrequencies) {
    if (freq === 0) break;
    let currentFreq = freq;
    while (usedFrequencies.has(currentFreq) && currentFreq > 0) {
      currentFreq--;
      result++;
    }
    usedFrequencies.add(currentFreq);
  }

  return result;
};
```

1648 Sell Diminishing Valued Colored Balls

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1648. Sell Diminishing-Valued Colored Balls
 * https://leetcode.com/problems/sell-diminishing-valued-colored-balls/
 * Difficulty: Medium
 *
 * You have an inventory of different colored balls, and there is a customer that wants orders
 * balls of any color.
 *
 * The customer weirdly values the colored balls. Each colored ball's value is the number of
balls
 * of that color you currently have in your inventory. For example, if you own 6 yellow balls,
 * the customer would pay 6 for the first yellow ball. After the transaction, there are only 5
 * yellow balls left, so the next yellow ball is then valued at 5 (i.e., the value of the
balls
 * decreases as you sell more to the customer).
 *
 * You are given an integer array, inventory, where inventory[i] represents the number of
balls of
 * the ith color that you initially own. You are also given an integer orders, which
represents
 * the total number of balls that the customer wants. You can sell the balls in any order.
 *
 * Return the maximum total value that you can attain after selling orders colored balls. As
the
 * answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} inventory
 * @param {number} orders
 * @return {number}
 */
var maxProfit = function(inventory, orders) {
  const MOD = 1e9 + 7;
  inventory.sort((a, b) => b - a);
  inventory.push(0);

  let totalProfit = 0;
  let currentColor = 0;
  let ballsSold = 0;

  while (ballsSold < orders) {
    const currentCount = inventory[currentColor];
    const nextCount = inventory[currentColor + 1];
    const colors = currentColor + 1;
    const ballsToSell = Math.min(
      orders - ballsSold,
      colors * (currentCount - nextCount)
    );

    const fullSets = Math.floor(ballsToSell / colors);
```

```
    const remainder = ballsToSell % colors;

    if (fullSets > 0) {
      const endValue = currentCount - fullSets + 1;
      let sequenceSum = (BigInt(colors) * BigInt(fullSets) % BigInt(MOD))
        * (BigInt(currentCount) + BigInt(endValue)) % BigInt(MOD);
      sequenceSum = (sequenceSum * BigInt(500000004)) % BigInt(MOD);
      totalProfit = (totalProfit + Number(sequenceSum)) % MOD;
    }

    if (remainder > 0) {
      totalProfit = (totalProfit + (remainder * (currentCount - fullSets)) % MOD) % MOD;
    }

    ballsSold += ballsToSell;
    currentColor++;
  }

  return totalProfit;
};
```

1649 Create Sorted Array Through Instructions

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1649. Create Sorted Array through Instructions
 * https://leetcode.com/problems/create-sorted-array-through-instructions/
 * Difficulty: Hard
 *
 * Given an integer array instructions, you are asked to create a sorted array from the
elements in
 * instructions. You start with an empty container nums. For each element from left to right
in
 * instructions, insert it into nums. The cost of each insertion is the minimum of the
following:
 * - The number of elements currently in nums that are strictly less than instructions[i].
 * - The number of elements currently in nums that are strictly greater than instructions[i].
 *
 * For example, if inserting element 3 into nums = [1,2,3,5], the cost of insertion is min(2,
1)
 * (elements 1 and 2 are less than 3, element 5 is greater than 3) and nums will become
[1,2,3,3,5].
 *
 * Return the total cost to insert all elements from instructions into nums. Since the answer
may
 * be large, return it modulo 109 + 7
 */

/**
 * @param {number[]} instructions
 * @return {number}
 */
var createSortedArray = function(instructions) {
  const MOD = 1e9 + 7;
  const maxValue = Math.max(...instructions);
  const fenwickTree = new Array(maxValue + 1).fill(0);
  let totalCost = 0;

  for (let i = 0; i < instructions.length; i++) {
    const value = instructions[i];
    const lessCount = query(value - 1);
    const greaterCount = i - query(value);
    totalCost = (totalCost + Math.min(lessCount, greaterCount)) % MOD;
    update(value);
  }

  return totalCost;

  function update(index) {
```

```
    for (let i = index; i <= maxValue; i += i & -i) {
      fenwickTree[i]++;
    }
  }

  function query(index) {
    let sum = 0;
    for (let i = index; i > 0; i -= i & -i) {
      sum += fenwickTree[i];
    }
    return sum;
  }
};
```

1650 Lowest Common Ancestor Of A Binary Tree Iii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1650. Lowest Common Ancestor of a Binary Tree III
 * https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree-iii/
 * Difficulty: Medium
 *
 * Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).
 *
 * Each node will have a reference to its parent node. The definition for Node is below:
 * class Node {
 *     public int val;
 *     public Node left;
 *     public Node right;
 *     public Node parent;
 * }
 *
 * According to the definition of LCA on Wikipedia: "The lowest common ancestor of two
 * nodes p and q in a tree T is the lowest node that has both p and q as descendants
 * (where we allow a node to be a descendant of itself)."
 */

/**
 * // Definition for a _Node.
 * function _Node(val) {
 *    this.val = val;
 *    this.left = null;
 *    this.right = null;
 *    this.parent = null;
 * };
 */

/**
 * @param {_Node} p
 * @param {_Node} q
 * @return {_Node}
 */
var lowestCommonAncestor = function(p, q) {
  const set = new Set();

  let current = p;
  while (current) {
    set.add(current);
    current = current.parent;
  }

  current = q;
  while (current) {
    if (set.has(current)) {
      return current;
    }
    current = current.parent;
  }
```

```
  return null;
};
```

1652 Defuse The Bomb

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1652. Defuse the Bomb
 * https://leetcode.com/problems/defuse-the-bomb/
 * Difficulty: Easy
 *
 * You have a bomb to defuse, and your time is running out! Your informer will provide you
with
 * a circular array code of length of n and a key k.
 *
 * To decrypt the code, you must replace every number. All the numbers are replaced
simultaneously.
 * - If k > 0, replace the ith number with the sum of the next k numbers.
 * - If k < 0, replace the ith number with the sum of the previous k numbers.
 * - If k == 0, replace the ith number with 0.
 *
 * As code is circular, the next element of code[n-1] is code[0], and the previous element of
 * code[0] is code[n-1].
 *
 * Given the circular array code and an integer key k, return the decrypted code to defuse the
 bomb!
 */

/**
 * @param {number[]} code
 * @param {number} k
 * @return {number[]}
 */
var decrypt = function(code, k) {
  const n = code.length;
  const result = new Array(n).fill(0);

  if (k === 0) return result;

  const isPositive = k > 0;
  const steps = Math.abs(k);

  for (let i = 0; i < n; i++) {
    let sum = 0;
    for (let j = 1; j <= steps; j++) {
      const index = isPositive ? (i + j) % n : (i - j + n) % n;
      sum += code[index];
    }
    result[i] = sum;
  }

  return result;
};
```

1653 Minimum Deletions To Make String Balanced

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1653. Minimum Deletions to Make String Balanced
 * https://leetcode.com/problems/minimum-deletions-to-make-string-balanced/
 * Difficulty: Medium
 *
 * You are given a string s consisting only of characters 'a' and 'b'.
```

```
 *
 * You can delete any number of characters in s to make s balanced. s is balanced if there is
 * no pair of indices (i,j) such that i < j and s[i] = 'b' and s[j]= 'a'.
 *
 * Return the minimum number of deletions needed to make s balanced.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumDeletions = function(s) {
  let aCountRight = 0;
  for (const char of s) {
    if (char === 'a') aCountRight++;
  }

  let bCountLeft = 0;
  let result = aCountRight;

  for (const char of s) {
    if (char === 'a') aCountRight--;
    else bCountLeft++;
    result = Math.min(result, aCountRight + bCountLeft);
  }

  return result;
};
```

1654 Minimum Jumps To Reach Home

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1654. Minimum Jumps to Reach Home
 * https://leetcode.com/problems/minimum-jumps-to-reach-home/
 * Difficulty: Medium
 *
 * A certain bug's home is on the x-axis at position x. Help them get there from position 0.
 *
 * The bug jumps according to the following rules:
 * - It can jump exactly a positions forward (to the right).
 * - It can jump exactly b positions backward (to the left).
 * - It cannot jump backward twice in a row.
 * - It cannot jump to any forbidden positions.
 *
 * The bug may jump forward beyond its home, but it cannot jump to positions numbered with
 * negative integers.
 *
 * Given an array of integers forbidden, where forbidden[i] means that the bug cannot jump to
 * the position forbidden[i], and integers a, b, and x, return the minimum number of jumps
 * needed for the bug to reach its home. If there is no possible sequence of jumps that lands
 * the bug on position x, return -1.
 */

/**
 * @param {number[]} forbidden
 * @param {number} a
 * @param {number} b
 * @param {number} x
 * @return {number}
 */
var minimumJumps = function(forbidden, a, b, x) {
  const forbiddenSet = new Set(forbidden);
  const maxPosition = Math.max(x, Math.max(...forbidden)) + a + b;
  const queue = [[0, 0, false]];
  const visited = new Set([`0,false`]);

  while (queue.length) {
    const [position, jumps, isBackward] = queue.shift();
```

```
    if (position === x) return jumps;

    const forwardPosition = position + a;
    if (forwardPosition <= maxPosition && !forbiddenSet.has(forwardPosition)
        && !visited.has(`${forwardPosition},false`)) {
      queue.push([forwardPosition, jumps + 1, false]);
      visited.add(`${forwardPosition},false`);
    }

    if (!isBackward && b > 0) {
      const backwardPosition = position - b;
      if (backwardPosition >= 0 && !forbiddenSet.has(backwardPosition)
          && !visited.has(`${backwardPosition},true`)) {
        queue.push([backwardPosition, jumps + 1, true]);
        visited.add(`${backwardPosition},true`);
      }
    }
  }

  return -1;
};
```

1655 Distribute Repeating Integers

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1655. Distribute Repeating Integers
 * https://leetcode.com/problems/distribute-repeating-integers/
 * Difficulty: Hard
 *
 * You are given an array of n integers, nums, where there are at most 50 unique values in
 * the array. You are also given an array of m customer order quantities, quantity, where
 * quantity[i] is the amount of integers the ith customer ordered. Determine if it is possible
 * to distribute nums such that:
 * - The ith customer gets exactly quantity[i] integers,
 * - The integers the ith customer gets are all equal, and
 * - Every customer is satisfied.
 *
 * Return true if it is possible to distribute nums according to the above conditions.
 */

/**
 * @param {number[]} nums
 * @param {number[]} quantity
 * @return {boolean}
 */
var canDistribute = function(nums, quantity) {
  const frequencyMap = new Map();
  for (const num of nums) {
    frequencyMap.set(num, (frequencyMap.get(num) || 0) + 1);
  }

  const frequencies = Array.from(frequencyMap.values()).sort((a, b) => b - a);
  quantity.sort((a, b) => b - a);

  return canSatisfy(0, frequencies);

  function canSatisfy(index, counts) {
    if (index === quantity.length) return true;

    for (let i = 0; i < counts.length; i++) {
      if (counts[i] >= quantity[index]) {
        counts[i] -= quantity[index];
        if (canSatisfy(index + 1, counts)) return true;
        counts[i] += quantity[index];
      }
    }
```

```
    return false;
  }
};
```

1656 Design An Ordered Stream

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1656. Design an Ordered Stream
 * https://leetcode.com/problems/design-an-ordered-stream/
 * Difficulty: Easy
 *
 * There is a stream of n (idKey, value) pairs arriving in an arbitrary order, where idKey is
 * an integer between 1 and n and value is a string. No two pairs have the same id.
 *
 * Design a stream that returns the values in increasing order of their IDs by returning a
chunk
 * (list) of values after each insertion. The concatenation of all the chunks should result in
 a
 * list of the sorted values.
 *
 * Implement the OrderedStream class:
 * - OrderedStream(int n) Constructs the stream to take n values.
 * - String[] insert(int idKey, String value) Inserts the pair (idKey, value) into the stream,
 *   then returns the largest possible chunk of currently inserted values that appear next in
 *   the order.
 */

/**
 * @param {number} n
 */
var OrderedStream = function(n) {
  this.stream = new Array(n).fill(null);
  this.nextIndex = 0;
};

/**
 * @param {number} idKey
 * @param {string} value
 * @return {string[]}
 */
OrderedStream.prototype.insert = function(idKey, value) {
  const index = idKey - 1;
  this.stream[index] = value;

  if (index !== this.nextIndex) return [];

  const result = [];
  while (this.nextIndex < this.stream.length && this.stream[this.nextIndex] !== null) {
    result.push(this.stream[this.nextIndex]);
    this.nextIndex++;
  }

  return result;
};
```

1657 Determine If Two Strings Are Close

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1657. Determine if Two Strings Are Close
 * https://leetcode.com/problems/determine-if-two-strings-are-close/
```

```
 * Difficulty: Medium
 *
 * Two strings are considered close if you can attain one from the other using the following
 * operations:
 *
 * - Operation 1: Swap any two existing characters.
 *   For example, abcde -> aecdb
 * - Operation 2: Transform every occurrence of one existing character into another existing
 *   character, and do the same with the other character.
 *   For example, aacabb -> bbcbaa (all a's turn into b's, and all b's turn into a's)
 *
 * You can use the operations on either string as many times as necessary.
 *
 * Given two strings, word1 and word2, return true if word1 and word2 are close, and false
 * otherwise.
 */

/**
 * @param {string} word1
 * @param {string} word2
 * @return {boolean}
 */
var closeStrings = function(word1, word2) {
  const map1 = helper(word1);
  const map2 = helper(word2);

  for (let i = 0; i < map1.length; i++) {
    if ((map1[i] === 0 || map2[i] === 0) && map1[i] !== map2[i]) {
      return false;
    }
  }

  [map1, map2].forEach(m => m.sort((a, b) => b - a));

  return map1.join('') === map2.join('');
};

function helper(input) {
  const map = Array(26).fill(0);
  for (let i = 0; i < input.length; i++) {
    map[input.charCodeAt(i) - 'a'.charCodeAt(0)]++;
  }
  return map;
}
```

1658 Minimum Operations To Reduce X To Zero

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1658. Minimum Operations to Reduce X to Zero
 * https://leetcode.com/problems/minimum-operations-to-reduce-x-to-zero/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer x. In one operation, you can either
remove
 * the leftmost or the rightmost element from the array nums and subtract its value from x.
Note
 * that this modifies the array for future operations.
 *
 * Return the minimum number of operations to reduce x to exactly 0 if it is possible,
otherwise,
 * return -1.
 */

/**
 * @param {number[]} nums
 * @param {number} x
 * @return {number}
 */
```

```
var minOperations = function(nums, x) {
  const targetSum = nums.reduce((sum, num) => sum + num, 0) - x;
  if (targetSum < 0) return -1;
  if (targetSum === 0) return nums.length;

  let currentSum = 0;
  let maxLength = -1;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    currentSum += nums[right];

    while (currentSum > targetSum && left <= right) {
      currentSum -= nums[left];
      left++;
    }

    if (currentSum === targetSum) {
      maxLength = Math.max(maxLength, right - left + 1);
    }
  }

  return maxLength === -1 ? -1 : nums.length - maxLength;
};
```

1659 Maximize Grid Happiness

HARD

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1659. Maximize Grid Happiness
 * https://leetcode.com/problems/maximize-grid-happiness/
 * Difficulty: Hard
 *
 * You are given four integers, m, n, introvertsCount, and extrovertsCount. You have an m x n
grid,
 * and there are two types of people: introverts and extroverts. There are introvertsCount
 * introverts and extrovertsCount extroverts.
 *
 * You should decide how many people you want to live in the grid and assign each of them one
grid
 * cell. Note that you do not have to have all the people living in the grid.
 *
 * The happiness of each person is calculated as follows:
 * - Introverts start with 120 happiness and lose 30 happiness for each neighbor (introvert or
 *   extrovert).
 * - Extroverts start with 40 happiness and gain 20 happiness for each neighbor (introvert or
 *   extrovert).
 *
 * Neighbors live in the directly adjacent cells north, east, south, and west of a person's
cell.
 *
 * The grid happiness is the sum of each person's happiness. Return the maximum possible grid
 * happiness.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number} introvertsCount
 * @param {number} extrovertsCount
 * @return {number}
 */
var getMaxGridHappiness = function(m, n, introvertsCount, extrovertsCount) {
  const memo = new Map();
  const maxState = Math.pow(3, n);

  return calculateHappiness(0, 0, introvertsCount, extrovertsCount, 0);

  function calculateHappiness(row, col, introverts, extroverts, prevState) {
```

```
    if (row === m || (introverts === 0 && extroverts === 0)) return 0;
    if (col === n) return calculateHappiness(row + 1, 0, introverts, extroverts, prevState);

    const key = `${row},${col},${introverts},${extroverts},${prevState}`;
    if (memo.has(key)) return memo.get(key);

    const nextState = (prevState * 3) % maxState;
    let maxHappiness = calculateHappiness(row, col + 1, introverts, extroverts, nextState);

    if (introverts > 0) {
      let happiness = 120;
      const leftNeighbor = col > 0 ? prevState % 3 : 0;
      const upNeighbor = row > 0 ? Math.floor(prevState / Math.pow(3, n - 1)) % 3 : 0;

      if (leftNeighbor) happiness -= 30;
      if (upNeighbor) happiness -= 30;

      let neighborHappiness = 0;
      if (leftNeighbor === 1) neighborHappiness -= 30;
      if (leftNeighbor === 2) neighborHappiness += 20;
      if (upNeighbor === 1) neighborHappiness -= 30;
      if (upNeighbor === 2) neighborHappiness += 20;

      maxHappiness = Math.max(
        maxHappiness,
        happiness + neighborHappiness
        + calculateHappiness(row, col + 1, introverts - 1, extroverts, nextState + 1)
      );
    }

    if (extroverts > 0) {
      let happiness = 40;
      const leftNeighbor = col > 0 ? prevState % 3 : 0;
      const upNeighbor = row > 0 ? Math.floor(prevState / Math.pow(3, n - 1)) % 3 : 0;

      if (leftNeighbor) happiness += 20;
      if (upNeighbor) happiness += 20;

      let neighborHappiness = 0;
      if (leftNeighbor === 1) neighborHappiness -= 30;
      if (leftNeighbor === 2) neighborHappiness += 20;
      if (upNeighbor === 1) neighborHappiness -= 30;
      if (upNeighbor === 2) neighborHappiness += 20;

      maxHappiness = Math.max(
        maxHappiness,
        happiness + neighborHappiness
        + calculateHappiness(row, col + 1, introverts, extroverts - 1, nextState + 2)
      );
    }

    memo.set(key, maxHappiness);
    return maxHappiness;
  }
};
```

1660 Correct A Binary Tree

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1660. Correct a Binary Tree
 * https://leetcode.com/problems/correct-a-binary-tree/
 * Difficulty: Medium
 *
 * You have a binary tree with a small defect. There is exactly one invalid node where its
right
 * child incorrectly points to another node at the same depth but to the invalid node's right.
 *
 * Given the root of the binary tree with this defect, root, return the root of the binary
```

```
tree
 * after removing this invalid node and every node underneath it (minus the node it
incorrectly
 * points to).
 *
 * Custom testing:
 * The test input is read as 3 lines:
 * - TreeNode root
 * - int fromNode (not available to correctBinaryTree)
 * - int toNode (not available to correctBinaryTree)
 *
 * After the binary tree rooted at root is parsed, the TreeNode with value of fromNode will
have
 * its right child pointer pointing to the TreeNode with a value of toNode. Then, root is
passed
 * to correctBinaryTree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} from
 * @param {number} to
 * @return {TreeNode}
 */
var correctBinaryTree = function(root) {
  const set = new Set();
  return traverse(root, null, false);

  function traverse(node, parent, isLeft) {
    if (!node) return null;

    if (node.right && set.has(node.right)) {
      if (parent) {
        if (isLeft) {
          parent.left = null;
        } else {
          parent.right = null;
        }
      }
      return null;
    }

    set.add(node);

    node.right = traverse(node.right, node, false);
    node.left = traverse(node.left, node, true);

    return node;
  }
};
```

1662 Check If Two String Arrays Are Equivalent

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1662. Check If Two String Arrays are Equivalent
 * https://leetcode.com/problems/check-if-two-string-arrays-are-equivalent/
 * Difficulty: Easy
 *
 * Given two string arrays word1 and word2, return true if the two arrays represent the same
 * string, and false otherwise.
```

```
 *
 * A string is represented by an array if the array elements concatenated in order forms the
string.
 */

/**
 * @param {string[]} word1
 * @param {string[]} word2
 * @return {boolean}
 */
var arrayStringsAreEqual = function(word1, word2) {
  return word1.join('') === word2.join('');
};
```

1663 Smallest String With A Given Numeric Value

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1663. Smallest String With A Given Numeric Value
 * https://leetcode.com/problems/smallest-string-with-a-given-numeric-value/
 * Difficulty: Medium
 *
 * The numeric value of a lowercase character is defined as its position (1-indexed) in the
 * alphabet, so the numeric value of a is 1, the numeric value of b is 2, the numeric value
 * of c is 3, and so on.
 *
 * The numeric value of a string consisting of lowercase characters is defined as the sum of
 * its characters' numeric values. For example, the numeric value of the string "abe" is equal
 * to 1 + 2 + 5 = 8.
 *
 * You are given two integers n and k. Return the lexicographically smallest string with
length
 * equal to n and numeric value equal to k.
 *
 * Note that a string x is lexicographically smaller than string y if x comes before y in
 * dictionary order, that is, either x is a prefix of y, or if i is the first position
 * such that x[i] != y[i], then x[i] comes before y[i] in alphabetic order.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {string}
 */
var getSmallestString = function(n, k) {
  const result = new Array(n).fill('a');
  let remainingValue = k - n;

  for (let i = n - 1; i >= 0 && remainingValue > 0; i--) {
    const addValue = Math.min(25, remainingValue);
    result[i] = String.fromCharCode(97 + addValue);
    remainingValue -= addValue;
  }

  return result.join('');
};
```

1664 Ways To Make A Fair Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1664. Ways to Make a Fair Array
 * https://leetcode.com/problems/ways-to-make-a-fair-array/
```

```
 * Difficulty: Medium
 *
 * You are given an integer array nums. You can choose exactly one index (0-indexed) and
remove
 * the element. Notice that the index of the elements may change after the removal.
 *
 * For example, if nums = [6,1,7,4,1]:
 * - Choosing to remove index 1 results in nums = [6,7,4,1].
 * - Choosing to remove index 2 results in nums = [6,1,4,1].
 * - Choosing to remove index 4 results in nums = [6,1,7,4].
 *
 * An array is fair if the sum of the odd-indexed values equals the sum of the even-indexed
values.
 *
 * Return the number of indices that you could choose such that after the removal, nums is
fair.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var waysToMakeFair = function(nums) {
  let evenSum = 0;
  let oddSum = 0;
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    if (i % 2 === 0) evenSum += nums[i];
    else oddSum += nums[i];
  }

  let currentEven = 0;
  let currentOdd = 0;

  for (let i = 0; i < nums.length; i++) {
    if (i % 2 === 0) {
      evenSum -= nums[i];
      if (currentEven + oddSum === currentOdd + evenSum) result++;
      currentEven += nums[i];
    } else {
      oddSum -= nums[i];
      if (currentEven + oddSum === currentOdd + evenSum) result++;
      currentOdd += nums[i];
    }
  }

  return result;
};
```

1665 Minimum Initial Energy To Finish Tasks

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1665. Minimum Initial Energy to Finish Tasks
 * https://leetcode.com/problems/minimum-initial-energy-to-finish-tasks/
 * Difficulty: Hard
 *
 * You are given an array tasks where tasks[i] = [actuali, minimumi]:
 * - actuali is the actual amount of energy you spend to finish the ith task.
 * - minimumi is the minimum amount of energy you require to begin the ith task.
 *
 * For example, if the task is [10, 12] and your current energy is 11, you cannot start
 * this task. However, if your current energy is 13, you can complete this task, and your
 * energy will be 3 after finishing it.
 *
 * You can finish the tasks in any order you like.
 *
 * Return the minimum initial amount of energy you will need to finish all the tasks.
```

```
 */

/**
 * @param {number[][]} tasks
 * @return {number}
 */
var minimumEffort = function(tasks) {
  tasks.sort((a, b) => (b[1] - b[0]) - (a[1] - a[0]));
  let result = 0;
  let currentEnergy = 0;

  for (const [actual, minimum] of tasks) {
    if (currentEnergy < minimum) {
      result += minimum - currentEnergy;
      currentEnergy = minimum;
    }
    currentEnergy -= actual;
  }

  return result;
};
```

1666 Change The Root Of A Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1666. Change the Root of a Binary Tree
 * https://leetcode.com/problems/change-the-root-of-a-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and a leaf node, reroot the tree so that the leaf is the
 * new root.
 *
 * You can reroot the tree with the following steps for each node cur on the path starting
 * from the leaf up to the root excluding the root:
 * 1. If cur has a left child, then that child becomes cur's right child.
 * 2. cur's original parent becomes cur's left child. Note that in this process the original
 *    parent's pointer to cur becomes null, making it have at most one child.
 *
 * Return the new root of the rerooted tree.
 *
 * Note: Ensure that your solution sets the Node.parent pointers correctly after rerooting or
 * you will receive "Wrong Answer".
 */

/**
 * // Definition for a Node.
 * function Node(val) {
 *    this.val = val;
 *    this.left = null;
 *    this.right = null;
 *    this.parent = null;
 * };
 */

/**
 * @param {Node} node
 * @return {Node}
 */
var flipBinaryTree = function(root, leaf) {
  helper(leaf, null);
  return leaf;

  function helper(node, newParent) {
    const originalParent = node.parent;
    node.parent = newParent;
    if (!originalParent) return;

    if (node.left) node.right = node.left;
```

```
    node.left = originalParent;

    if (originalParent.left === node) {
      originalParent.left = null;
    } else {
      originalParent.right = null;
    }

    helper(originalParent, node);
  }
};
```

1668 Maximum Repeating Substring

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1668. Maximum Repeating Substring
 * https://leetcode.com/problems/maximum-repeating-substring/
 * Difficulty: Easy
 *
 * For a string sequence, a string word is k-repeating if word concatenated k times is a
substring
 * of sequence. The word's maximum k-repeating value is the highest value k where word is
k-repeating
 * in sequence. If word is not a substring of sequence, word's maximum k-repeating value is 0.
 *
 * Given strings sequence and word, return the maximum k-repeating value of word in sequence.
 */

/**
 * @param {string} sequence
 * @param {string} word
 * @return {number}
 */
var maxRepeating = function(sequence, word) {
  let count = Math.floor(sequence.length / word.length) + 1;
  while (--count) {
    if (sequence.includes(word.repeat(count))) return count;
  }
  return count;
};
```

1669 Merge In Between Linked Lists

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1669. Merge In Between Linked Lists
 * https://leetcode.com/problems/merge-in-between-linked-lists/
 * Difficulty: Medium
 *
 * You are given two linked lists: list1 and list2 of sizes n and m respectively.
 *
 * Remove list1's nodes from the ath node to the bth node, and put list2 in their place.
 *
 * The blue edges and nodes in the following figure indicate the result:
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
```

```
 */
/**
 * @param {ListNode} list1
 * @param {number} a
 * @param {number} b
 * @param {ListNode} list2
 * @return {ListNode}
 */
var mergeInBetween = function(list1, a, b, list2) {
  const result = new ListNode(0, list1);

  for (let i = 1; i < a; i++) {
    list1 = list1.next;
  }
  let tail = list1.next;
  list1.next = list2;
  for (let i = a; i < b; i++) {
    tail = tail.next;
  }
  while (list1 && list1.next) {
    list1 = list1.next;
  }
  list1.next = tail.next;

  return result.next;
};
```

1670 Design Front Middle Back Queue

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1670. Design Front Middle Back Queue
 * https://leetcode.com/problems/design-front-middle-back-queue/
 * Difficulty: Medium
 *
 * Design a queue that supports push and pop operations in the front, middle, and back.
 *
 * Implement the FrontMiddleBack class:
 * - FrontMiddleBack() Initializes the queue.
 * - void pushFront(int val) Adds val to the front of the queue.
 * - void pushMiddle(int val) Adds val to the middle of the queue.
 * - void pushBack(int val) Adds val to the back of the queue.
 * - int popFront() Removes the front element of the queue and returns it. If the queue is
 *   empty, return -1.
 * - int popMiddle() Removes the middle element of the queue and returns it. If the queue is
 *   empty, return -1.
 * - int popBack() Removes the back element of the queue and returns it. If the queue is
 *   empty, return -1.
 *
 * Notice that when there are two middle position choices, the operation is performed on the
 * frontmost middle position choice. For example:
 * - Pushing 6 into the middle of [1, 2, 3, 4, 5] results in [1, 2, 6, 3, 4, 5].
 * - Popping the middle from [1, 2, 3, 4, 5, 6] returns 3 and results in [1, 2, 4, 5, 6].
 */

var FrontMiddleBackQueue = function() {
  this.elements = [];
};

/**
 * @param {number} val
 * @return {void}
 */
FrontMiddleBackQueue.prototype.pushFront = function(val) {
  this.elements.unshift(val);
};

/**
 * @param {number} val
```

```
 * @return {void}
 */
FrontMiddleBackQueue.prototype.pushMiddle = function(val) {
  const mid = Math.floor(this.elements.length / 2);
  this.elements.splice(mid, 0, val);
};

/**
 * @param {number} val
 * @return {void}
 */
FrontMiddleBackQueue.prototype.pushBack = function(val) {
  this.elements.push(val);
};

/**
 * @return {number}
 */
FrontMiddleBackQueue.prototype.popFront = function() {
  return this.elements.length ? this.elements.shift() : -1;
};

/**
 * @return {number}
 */
FrontMiddleBackQueue.prototype.popMiddle = function() {
  if (!this.elements.length) return -1;
  const mid = Math.floor((this.elements.length - 1) / 2);
  return this.elements.splice(mid, 1)[0];
};

/**
 * @return {number}
 */
FrontMiddleBackQueue.prototype.popBack = function() {
  return this.elements.length ? this.elements.pop() : -1;
};
```

1671 Minimum Number Of Removals To Make Mountain Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1671. Minimum Number of Removals to Make Mountain Array
 * https://leetcode.com/problems/minimum-number-of-removals-to-make-mountain-array/
 * Difficulty: Hard
 *
 * You may recall that an array arr is a mountain array if and only if:
 * - arr.length >= 3
 * - There exists some index i (0-indexed) with 0 < i < arr.length - 1 such that:
 *   - arr[0] < arr[1] < ... < arr[i - 1] < arr[i]
 *   - arr[i] > arr[i + 1] > ... > arr[arr.length - 1]
 *
 * Given an integer array nums, return the minimum number of elements to remove to make
 * nums a mountain array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumMountainRemovals = function(nums) {
  const length = nums.length;
  const leftLIS = new Array(length).fill(1);
  const rightLIS = new Array(length).fill(1);

  for (let i = 1; i < length; i++) {
    for (let j = 0; j < i; j++) {
      if (nums[i] > nums[j]) {
        leftLIS[i] = Math.max(leftLIS[i], leftLIS[j] + 1);
```

```
      }
    }
  }

  for (let i = length - 2; i >= 0; i--) {
    for (let j = length - 1; j > i; j--) {
      if (nums[i] > nums[j]) {
        rightLIS[i] = Math.max(rightLIS[i], rightLIS[j] + 1);
      }
    }
  }

  let maxMountainLength = 0;
  for (let i = 1; i < length - 1; i++) {
    if (leftLIS[i] > 1 && rightLIS[i] > 1) {
      maxMountainLength = Math.max(maxMountainLength, leftLIS[i] + rightLIS[i] - 1);
    }
  }

  return length - maxMountainLength;
};
```

1672 Richest Customer Wealth

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1672. Richest Customer Wealth
 * https://leetcode.com/problems/richest-customer-wealth/
 * Difficulty: Easy
 *
 * You are given an m x n integer grid accounts where accounts[i][j] is the
 * amount of money the i■■■■■■■■■■■■■th■■■■■ customer has in the j■■■■■■■■■■■■■th■■■■■ bank. Return
the wealth
 * that the richest customer has.
 *
 * A customer's wealth is the amount of money they have in all their bank
 * accounts. The richest customer is the customer that has the maximum wealth.
 */

/**
 * @param {number[][]} accounts
 * @return {number}
 */
var maximumWealth = function(accounts) {
  return Math.max(...accounts.map(account => account.reduce((sum, n) => sum + n), 0));
};
```

1673 Find The Most Competitive Subsequence

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1673. Find the Most Competitive Subsequence
 * https://leetcode.com/problems/find-the-most-competitive-subsequence/
 * Difficulty: Medium
 *
 * Given an integer array nums and a positive integer k, return the most competitive
subsequence
 * of nums of size k.
 *
 * An array's subsequence is a resulting sequence obtained by erasing some (possibly zero)
elements
 * from the array.
 *
```

```
 * We define that a subsequence a is more competitive than a subsequence b (of the same
length) if
 * in the first position where a and b differ, subsequence a has a number less than the
 * corresponding number in b. For example, [1,3,4] is more competitive than [1,3,5] because
the
 * first position they differ is at the final number, and 4 is less than 5.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var mostCompetitive = function(nums, k) {
  const result = [];
  const n = nums.length;

  for (let i = 0; i < n; i++) {
    while (result.length && result[result.length - 1] > nums[i] && result.length + n - i > k)
{
      result.pop();
    }
    if (result.length < k) {
      result.push(nums[i]);
    }
  }

  return result;
};
```

1674 Minimum Moves To Make Array Complementary

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1674. Minimum Moves to Make Array Complementary
 * https://leetcode.com/problems/minimum-moves-to-make-array-complementary/
 * Difficulty: Medium
 *
 * You are given an integer array nums of even length n and an integer limit. In one move, you
 * can replace any integer from nums with another integer between 1 and limit, inclusive.
 *
 * The array nums is complementary if for all indices i (0-indexed), nums[i] + nums[n - 1 - i]
 * equals the same number. For example, the array [1,2,3,4] is complementary because for all
 * indices i, nums[i] + nums[n - 1 - i] = 5.
 *
 * Return the minimum number of moves required to make nums complementary.
 */

/**
 * @param {number[]} nums
 * @param {number} limit
 * @return {number}
 */
var minMoves = function(nums, limit) {
  const n = nums.length;
  const delta = new Array(2 * limit + 2).fill(0);
  let result = n;

  for (let i = 0; i < n / 2; i++) {
    const left = nums[i];
    const right = nums[n - 1 - i];
    const minSum = Math.min(left, right) + 1;
    const maxSum = Math.max(left, right) + limit;
    delta[2] += 2;
    delta[minSum] -= 1;
    delta[left + right] -= 1;
    delta[left + right + 1] += 1;
    delta[maxSum + 1] += 1;
  }
```

```
  let moves = 0;
  for (let i = 2; i <= 2 * limit; i++) {
    moves += delta[i];
    result = Math.min(result, moves);
  }

  return result;
};
```

1675 Minimize Deviation In Array

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1675. Minimize Deviation in Array
 * https://leetcode.com/problems/minimize-deviation-in-array/
 * Difficulty: Hard
 *
 * You are given an array nums of n positive integers.
 *
 * You can perform two types of operations on any element of the array any number of times:
 * - If the element is even, divide it by 2.
 *   - For example, if the array is [1,2,3,4], then you can do this operation on the last
element,
 *     and the array will be [1,2,3,2].
 * - If the element is odd, multiply it by 2.
 *   - For example, if the array is [1,2,3,4], then you can do this operation on the first
element,
 *     and the array will be [2,2,3,4].
 *
 * The deviation of the array is the maximum difference between any two elements in the array.
 *
 * Return the minimum deviation the array can have after performing some number of operations.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumDeviation = function(nums) {
  const heap = [];
  let minElement = Infinity;

  for (const num of nums) {
    const value = num % 2 ? num * 2 : num;
    heap.push(value);
    minElement = Math.min(minElement, value);
  }

  for (let i = Math.floor(heap.length / 2) - 1; i >= 0; i--) {
    siftDown(i);
  }

  let minDeviation = heap[0] - minElement;

  while (heap[0] % 2 === 0) {
    const maxElement = heap[0];
    heap[0] = maxElement / 2;
    minElement = Math.min(minElement, heap[0]);
    siftDown(0);
    minDeviation = Math.min(minDeviation, heap[0] - minElement);
  }

  return minDeviation;

  function siftDown(index) {
    while (2 * index + 1 < heap.length) {
      let maxChild = 2 * index + 1;
      if (maxChild + 1 < heap.length && heap[maxChild + 1] > heap[maxChild]) {
```

```
        maxChild++;
      }
      if (heap[index] < heap[maxChild]) {
        [heap[index], heap[maxChild]] = [heap[maxChild], heap[index]];
        index = maxChild;
      } else {
        break;
      }
    }
  }
};
```

1676 Lowest Common Ancestor Of A Binary Tree Iv

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1676. Lowest Common Ancestor of a Binary Tree IV
 * https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree-iv/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and an array of TreeNode objects nodes, return the lowest
common
 * ancestor (LCA) of all the nodes in nodes. All the nodes will exist in the tree, and all
values
 * of the tree's nodes are unique.
 *
 * Extending the definition of LCA on Wikipedia: "The lowest common ancestor of n nodes p1,
 * p2, ..., pn in a binary tree T is the lowest node that has every pi as a descendant (where
we
 * allow a node to be a descendant of itself) for every valid i". A descendant of a node x is
a
 * node y that is on the path from node x to some leaf node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode[]} nodes
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, nodes) {
  const targetSet = new Set(nodes);
  return helper(root);

  function helper(node) {
    if (!node) return null;

    if (targetSet.has(node)) return node;

    const leftResult = helper(node.left);
    const rightResult = helper(node.right);

    if (leftResult && rightResult) return node;

    return leftResult || rightResult;
  }
};
```

1678 Goal Parser Interpretation

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1678. Goal Parser Interpretation
 * https://leetcode.com/problems/goal-parser-interpretation/
 * Difficulty: Easy
 *
 * You own a Goal Parser that can interpret a string command. The command consists of an
alphabet
 * of "G", "()" and/or "(al)" in some order. The Goal Parser will interpret "G" as the string
 * "G", "()" as the string "o", and "(al)" as the string "al". The interpreted strings are
then
 * concatenated in the original order.
 *
 * Given the string command, return the Goal Parser's interpretation of command.
 */

/**
 * @param {string} command
 * @return {string}
 */
var interpret = function(command) {
  let result = '';

  for (let i = 0; i < command.length; i++) {
    if (command[i] === 'G') {
      result += 'G';
    } else if (command[i] === '(' && command[i + 1] === ')') {
      result += 'o';
      i++;
    } else {
      result += 'al';
      i += 3;
    }
  }

  return result;
};
```

1679 Max Number Of K Sum Pairs

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1679. Max Number of K-Sum Pairs
 * https://leetcode.com/problems/max-number-of-k-sum-pairs/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k.
 *
 * In one operation, you can pick two numbers from the array whose sum equals k and
 * remove them from the array.
 *
 * Return the maximum number of operations you can perform on the array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxOperations = function(nums, k) {
  const map = new Map();
```

```
  let result = 0;

  nums.forEach(n => {
    const diff = k - n;

    if (map.get(diff)) {
      result++;
      map.set(diff, map.get(diff) - 1);
    } else {
      map.set(n, (map.get(n) ?? 0) + 1);
    }
  });

  return result;
};
```

1680 Concatenation Of Consecutive Binary Numbers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1680. Concatenation of Consecutive Binary Numbers
 * https://leetcode.com/problems/concatenation-of-consecutive-binary-numbers/
 * Difficulty: Medium
 *
 * Given an integer n, return the decimal value of the binary string formed by concatenating
the
 * binary representations of 1 to n in order, modulo 109 + 7.
 */

/**
 * @param {number} n
 * @return {number}
 */
var concatenatedBinary = function(n) {
  const MOD = 1e9 + 7;
  let result = 0;

  for (let i = 1; i <= n; i++) {
    const bitLength = Math.floor(Math.log2(i)) + 1;
    result = ((result * (1 << bitLength)) % MOD + i) % MOD;
  }

  return result;
};
```

1681 Minimum Incompatibility

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1681. Minimum Incompatibility
 * https://leetcode.com/problems/minimum-incompatibility/
 * Difficulty: Hard
 *
 * You are given an integer array nums and an integer k. You are asked to distribute this
array
 * into k subsets of equal size such that there are no two equal elements in the same subset.
 *
 * A subset's incompatibility is the difference between the maximum and minimum elements in
that
 * array.
 *
 * Return the minimum possible sum of incompatibilities of the k subsets after distributing
```

```
the
 * array optimally, or return -1 if it is not possible.
 *
 * A subset is a group integers that appear in the array with no particular order.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minimumIncompatibility = function(nums, k) {
  const n = nums.length;
  const subsetSize = n / k;
  const freq = new Array(n + 1).fill(0);
  for (const num of nums) {
    freq[num]++;
    if (freq[num] > k) return -1;
  }

  nums.sort((a, b) => a - b);
  const subsetValues = new Map();

  computeSubsets(0, 0, 0, 0, 0, []);

  const dp = new Array(1 << n).fill(Infinity);
  dp[0] = 0;

  for (let mask = 0; mask < (1 << n); mask++) {
    if (dp[mask] === Infinity) continue;
    for (const [subsetMask, value] of subsetValues) {
      if ((mask & subsetMask) === 0) {
        const newMask = mask | subsetMask;
        dp[newMask] = Math.min(dp[newMask], dp[mask] + value);
      }
    }
  }

  return dp[(1 << n) - 1] === Infinity ? -1 : dp[(1 << n) - 1];

  function computeSubsets(mask, index, count, minVal, maxVal, selected) {
    if (count === subsetSize) {
      subsetValues.set(mask, maxVal - minVal);
      return;
    }
    if (index >= n || n - index < subsetSize - count) return;

    computeSubsets(mask, index + 1, count, minVal, maxVal, selected);
    if (!selected.includes(nums[index])) {
      computeSubsets(mask | (1 << index), index + 1, count + 1,
        count === 0 ? nums[index] : minVal, nums[index], [...selected, nums[index]]);
    }
  }
};
```

1682 Longest Palindromic Subsequence Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1682. Longest Palindromic Subsequence II
 * https://leetcode.com/problems/longest-palindromic-subsequence-ii/
 * Difficulty: Medium
 *
 * A subsequence of a string s is considered a good palindromic subsequence if:
 * - It is a subsequence of s.
 * - It is a palindrome (has the same value if reversed).
 * - It has an even length.
 * - No two consecutive characters are equal, except the two middle ones.
 *
```

```
 * For example, if s = "abcabcabb", then "abba" is considered a good palindromic subsequence,
 * while "bcb" (not even length) and "bbbb" (has equal consecutive characters) are not.
 *
 * Given a string s, return the length of the longest good palindromic subsequence in s.
 */

/**
 * @param {string} s
 * @return {number}
 */
var longestPalindromeSubseq = function(s) {
  const stringLength = s.length;
  const alphabetSize = 26;
  const baseCharCode = 97;

  const memoTable = new Array(alphabetSize + 1);
  for (let i = 0; i <= alphabetSize; i++) {
    const matrix = memoTable[i] = new Array(stringLength);
    for (let j = 0; j < stringLength; j++) {
      matrix[j] = new Array(stringLength);
    }
  }

  return findLongestPalindrome(alphabetSize, 0, stringLength - 1);

  function findLongestPalindrome(previousChar, leftIndex, rightIndex) {
    if (leftIndex >= rightIndex) return 0;

    const cachedResult = memoTable[previousChar][leftIndex][rightIndex];
    if (cachedResult !== undefined) return cachedResult;

    let maxLength = 0;
    const leftChar = s.charCodeAt(leftIndex) - baseCharCode;
    const rightChar = s.charCodeAt(rightIndex) - baseCharCode;

    if (leftChar === rightChar) {
      if (leftChar !== previousChar) {
        maxLength += 2;
      }
      maxLength += findLongestPalindrome(leftChar, leftIndex + 1, rightIndex - 1);
    } else {
      const skipLeftResult = findLongestPalindrome(previousChar, leftIndex + 1, rightIndex);
      const skipRightResult = findLongestPalindrome(previousChar, leftIndex, rightIndex - 1);
      maxLength += Math.max(skipLeftResult, skipRightResult);
    }

    return memoTable[previousChar][leftIndex][rightIndex] = maxLength;
  }
};
```

1684 Count The Number Of Consistent Strings

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1684. Count the Number of Consistent Strings
 * https://leetcode.com/problems/count-the-number-of-consistent-strings/
 * Difficulty: Easy
 *
 * You are given a string allowed consisting of distinct characters and an array of strings
words.
 * A string is consistent if all characters in the string appear in the string allowed.
 *
 * Return the number of consistent strings in the array words.
 */

/**
 * @param {string} allowed
 * @param {string[]} words
 * @return {number}
```

```
 */
var countConsistentStrings = function(allowed, words) {
  const allowedSet = new Set(allowed);
  let result = 0;

  for (const word of words) {
    let isConsistent = true;
    for (const char of word) {
      if (!allowedSet.has(char)) {
        isConsistent = false;
        break;
      }
    }
    if (isConsistent) result++;
  }

  return result;
};
```

1685 Sum Of Absolute Differences In A Sorted Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1685. Sum of Absolute Differences in a Sorted Array
 * https://leetcode.com/problems/sum-of-absolute-differences-in-a-sorted-array/
 * Difficulty: Medium
 *
 * You are given an integer array nums sorted in non-decreasing order.
 *
 * Build and return an integer array result with the same length as nums such that result[i]
is
 * equal to the summation of absolute differences between nums[i] and all the other elements
 * in the array.
 *
 * In other words, result[i] is equal to sum(|nums[i]-nums[j]|) where 0 <= j < nums.length
 * and j != i (0-indexed).
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var getSumAbsoluteDifferences = function(nums) {
  const n = nums.length;
  const result = new Array(n);
  let prefixSum = 0;
  let suffixSum = nums.reduce((sum, num) => sum + num, 0);

  for (let i = 0; i < n; i++) {
    const current = nums[i];
    suffixSum -= current;
    result[i] = (current * i - prefixSum) + (suffixSum - current * (n - 1 - i));
    prefixSum += current;
  }

  return result;
};
```

1686 Stone Game Vi

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1686. Stone Game VI
```

```
 * https://leetcode.com/problems/stone-game-vi/
 * Difficulty: Medium
 *
 * Alice and Bob take turns playing a game, with Alice starting first.
 *
 * There are n stones in a pile. On each player's turn, they can remove a stone from the pile
and
 * receive points based on the stone's value. Alice and Bob may value the stones differently.
 *
 * You are given two integer arrays of length n, aliceValues and bobValues. Each
aliceValues[i]
 * and bobValues[i] represents how Alice and Bob, respectively, value the ith stone.
 *
 * The winner is the person with the most points after all the stones are chosen. If both
players
 * have the same amount of points, the game results in a draw. Both players will play
optimally.
 * Both players know the other's values.
 *
 * Determine the result of the game, and:
 * - If Alice wins, return 1.
 * - If Bob wins, return -1.
 * - If the game results in a draw, return 0.
 */

/**
 * @param {number[]} aliceValues
 * @param {number[]} bobValues
 * @return {number}
 */
var stoneGameVI = function(aliceValues, bobValues) {
  const n = aliceValues.length;
  const stones = aliceValues.map((alice, i) => ({
    sum: alice + bobValues[i],
    alice: alice,
    bob: bobValues[i]
  }));

  stones.sort((a, b) => b.sum - a.sum);

  let aliceScore = 0;
  let bobScore = 0;

  for (let i = 0; i < n; i++) {
    if (i % 2 === 0) {
      aliceScore += stones[i].alice;
    } else {
      bobScore += stones[i].bob;
    }
  }

  return aliceScore > bobScore ? 1 : aliceScore < bobScore ? -1 : 0;
};
```

1687 Delivering Boxes From Storage To Ports

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1687. Delivering Boxes from Storage to Ports
 * https://leetcode.com/problems/delivering-boxes-from-storage-to-ports/
 * Difficulty: Hard
 *
 * You have the task of delivering some boxes from storage to their ports using only one ship.
 * However, this ship has a limit on the number of boxes and the total weight that it can
carry.
 *
 * You are given an array boxes, where boxes[i] = [portsi, weighti], and three integers
 * portsCount, maxBoxes, and maxWeight.
 * - portsi is the port where you need to deliver the ith box and weightsi is the weight of
```

```
 *   the ith box.
 * - portsCount is the number of ports.
 * - maxBoxes and maxWeight are the respective box and weight limits of the ship.
 *
 * The boxes need to be delivered in the order they are given. The ship will follow these
steps:
 * - The ship will take some number of boxes from the boxes queue, not violating the maxBoxes
and
 *   maxWeight constraints.
 * - For each loaded box in order, the ship will make a trip to the port the box needs to be
 *   delivered to and deliver it. If the ship is already at the correct port, no trip is
needed,
 *   and the box can immediately be delivered.
 * - The ship then makes a return trip to storage to take more boxes from the queue.
 *
 * The ship must end at storage after all the boxes have been delivered.
 *
 * Return the minimum number of trips the ship needs to make to deliver all boxes to their
 * respective ports.
 */

/**
 * @param {number[][]} boxes
 * @param {number} portsCount
 * @param {number} maxBoxes
 * @param {number} maxWeight
 * @return {number}
 */
var boxDelivering = function(boxes, portsCount, maxBoxes, maxWeight) {
  const n = boxes.length;
  const minTrips = new Array(n + 1).fill(0);
  let start = 0;
  let portChanges = 0;

  for (let end = 0; end < n; end++) {
    maxBoxes--;
    maxWeight -= boxes[end][1];
    if (end > 0 && boxes[end][0] !== boxes[end - 1][0]) portChanges++;

    while (maxBoxes < 0 || maxWeight < 0
          || (start < end && minTrips[start + 1] === minTrips[start])) {
      maxBoxes++;
      maxWeight += boxes[start++][1];
      if (start > 0 && boxes[start][0] !== boxes[start - 1][0]) portChanges--;
    }

    minTrips[end + 1] = portChanges + 2 + minTrips[start];
  }

  return minTrips[n];
};
```

1688 Count Of Matches In Tournament

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1688. Count of Matches in Tournament
 * https://leetcode.com/problems/count-of-matches-in-tournament/
 * Difficulty: Easy
 *
 * You are given an integer n, the number of teams in a tournament that has strange rules:
 * - If the current number of teams is even, each team gets paired with another team. A total
 *   of n / 2 matches are played, and n / 2 teams advance to the next round.
 * - If the current number of teams is odd, one team randomly advances in the tournament, and
 *   the rest gets paired. A total of (n - 1) / 2 matches are played, and (n - 1) / 2 + 1
teams
 *   advance to the next round.
 *
 * Return the number of matches played in the tournament until a winner is decided.
```

```
 */

/**
 * @param {number} n
 * @return {number}
 */
var numberOfMatches = function(n) {
  return n - 1;
};
```

1689 Partitioning Into Minimum Number Of Deci Binary Numbers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1689. Partitioning Into Minimum Number Of Deci-Binary Numbers
 * https://leetcode.com/problems/partitioning-into-minimum-number-of-deci-binary-numbers/
 * Difficulty: Medium
 *
 * A decimal number is called deci-binary if each of its digits is either 0 or 1 without any
 * leading zeros. For example, 101 and 1100 are deci-binary, while 112 and 3001 are not.
 *
 * Given a string n that represents a positive decimal integer, return the minimum number of
 * positive deci-binary numbers needed so that they sum up to n.
 */

/**
 * @param {string} n
 * @return {number}
 */
var minPartitions = function(n) {
  let maxDigit = 0;
  for (const digit of n) {
    maxDigit = Math.max(maxDigit, parseInt(digit));
  }
  return maxDigit;
};
```

1690 Stone Game Vii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1690. Stone Game VII
 * https://leetcode.com/problems/stone-game-vii/
 * Difficulty: Medium
 *
 * Alice and Bob take turns playing a game, with Alice starting first.
 *
 * There are n stones arranged in a row. On each player's turn, they can remove either the
leftmost
 * stone or the rightmost stone from the row and receive points equal to the sum of the
remaining
 * stones' values in the row. The winner is the one with the higher score when there are no
stones
 * left to remove.
 *
 * Bob found that he will always lose this game (poor Bob, he always loses), so he decided to
 * minimize the score's difference. Alice's goal is to maximize the difference in the score.
 *
 * Given an array of integers stones where stones[i] represents the value of the ith stone
from
 * the left, return the difference in Alice and Bob's score if they both play optimally.
 */
```

```
/**
 * @param {number[]} stones
 * @return {number}
 */
var stoneGameVII = function(stones) {
  const n = stones.length;
  const prefixSum = new Array(n + 1).fill(0);
  const dp = new Array(n).fill().map(() => new Array(n).fill(0));

  for (let i = 0; i < n; i++) {
    prefixSum[i + 1] = prefixSum[i] + stones[i];
  }

  for (let len = 2; len <= n; len++) {
    for (let start = 0; start <= n - len; start++) {
      const end = start + len - 1;
      const leftScore = prefixSum[end + 1] - prefixSum[start + 1] - dp[start + 1][end];
      const rightScore = prefixSum[end] - prefixSum[start] - dp[start][end - 1];
      dp[start][end] = Math.max(leftScore, rightScore);
    }
  }

  return dp[0][n - 1];
};
```

1691 Maximum Height By Stacking Cuboids

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1691. Maximum Height by Stacking Cuboids
 * https://leetcode.com/problems/maximum-height-by-stacking-cuboids/
 * Difficulty: Hard
 *
 * Given n cuboids where the dimensions of the ith cuboid is cuboids[i] = [widthi, lengthi,
heighti]
 * (0-indexed). Choose a subset of cuboids and place them on each other.
 *
 * You can place cuboid i on cuboid j if widthi <= widthj and lengthi <= lengthj and
 * heighti <= heightj. You can rearrange any cuboid's dimensions by rotating it to put it on
 * another cuboid.
 *
 * Return the maximum height of the stacked cuboids.
 */

/**
 * @param {number[][]} cuboids
 * @return {number}
 */
var maxHeight = function(cuboids) {
  const sortedCuboids = cuboids.map(dim => dim.sort((a, b) => a - b))
    .sort((a, b) => a[0] - b[0] || a[1] - b[1] || a[2] - b[2]);
  const n = sortedCuboids.length;
  const maxHeights = new Array(n).fill(0);
  let result = 0;

  for (let i = 0; i < n; i++) {
    maxHeights[i] = sortedCuboids[i][2];
    for (let j = 0; j < i; j++) {
      if (sortedCuboids[j][0] <= sortedCuboids[i][0]
          && sortedCuboids[j][1] <= sortedCuboids[i][1]
          && sortedCuboids[j][2] <= sortedCuboids[i][2]) {
        maxHeights[i] = Math.max(maxHeights[i], maxHeights[j] + sortedCuboids[i][2]);
      }
    }
    result = Math.max(result, maxHeights[i]);
  }

  return result;
};
```

1692 Count Ways To Distribute Candies

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1692. Count Ways to Distribute Candies
 * https://leetcode.com/problems/count-ways-to-distribute-candies/
 * Difficulty: Hard
 *
 * There are n unique candies (labeled 1 through n) and k bags. You are asked to distribute
all
 * the candies into the bags such that every bag has at least one candy.
 *
 * There can be multiple ways to distribute the candies. Two ways are considered different if
 * the candies in one bag in the first way are not all in the same bag in the second way. The
 * order of the bags and the order of the candies within each bag do not matter.
 *
 * For example, (1), (2,3) and (2), (1,3) are considered different because candies 2 and 3 in
 * the bag (2,3) in the first way are not in the same bag in the second way (they are split
 * between the bags (2) and (1,3)). However, (1), (2,3) and (3,2), (1) are considered the same
 * because the candies in each bag are all in the same bags in both ways.
 *
 * Given two integers, n and k, return the number of different ways to distribute the candies.
 * As the answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var waysToDistribute = function(n, k) {
  const MOD = 1e9 + 7;
  const dp = new Array(n + 1).fill(0).map(() => new Array(k + 1).fill(0));

  for (let i = 1; i <= n; i++) {
    dp[i][1] = 1;
  }

  for (let candies = 2; candies <= n; candies++) {
    for (let bags = 2; bags <= Math.min(candies, k); bags++) {
      dp[candies][bags] = (dp[candies - 1][bags - 1] + (bags * dp[candies - 1][bags]) % MOD) %
 MOD;
    }
  }

  return dp[n][k];
};
```

1694 Reformat Phone Number

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1694. Reformat Phone Number
 * https://leetcode.com/problems/reformat-phone-number/
 * Difficulty: Easy
 *
 * You are given a phone number as a string number. number consists of digits, spaces ' ',
 * and/or dashes '-'.
 *
 * You would like to reformat the phone number in a certain manner. Firstly, remove all spaces
 * and dashes. Then, group the digits from left to right into blocks of length 3 until there
 * are 4 or fewer digits. The final digits are then grouped as follows:
 * - 2 digits: A single block of length 2.
```

```
 * - 3 digits: A single block of length 3.
 * - 4 digits: Two blocks of length 2 each.
 *
 * The blocks are then joined by dashes. Notice that the reformatting process should never
produce
 * any blocks of length 1 and produce at most two blocks of length 2.
 *
 * Return the phone number after formatting.
 */

/**
 * @param {string} number
 * @return {string}
 */
var reformatNumber = function(number) {
  const digits = number.replace(/[^0-9]/g, '');
  const blocks = [];
  let i = 0;

  while (i < digits.length) {
    const remaining = digits.length - i;
    if (remaining > 4) {
      blocks.push(digits.slice(i, i + 3));
      i += 3;
    } else if (remaining === 4) {
      blocks.push(digits.slice(i, i + 2), digits.slice(i + 2));
      break;
    } else if (remaining === 2 || remaining === 3) {
      blocks.push(digits.slice(i));
      break;
    }
  }

  return blocks.join('-');
};
```

1695 Maximum Erasure Value

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1695. Maximum Erasure Value
 * https://leetcode.com/problems/maximum-erasure-value/
 * Difficulty: Medium
 *
 * You are given an array of positive integers nums and want to erase a subarray containing
unique
 * elements. The score you get by erasing the subarray is equal to the sum of its elements.
 *
 * Return the maximum score you can get by erasing exactly one subarray.
 *
 * An array b is called to be a subarray of a if it forms a contiguous subsequence of a, that
is,
 * if it is equal to a[l],a[l+1],...,a[r] for some (l,r).
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumUniqueSubarray = function(nums) {
  const seen = new Set();
  let result = 0;
  let currentSum = 0;
  let start = 0;

  for (let end = 0; end < nums.length; end++) {
    while (seen.has(nums[end])) {
      seen.delete(nums[start]);
```

```
      currentSum -= nums[start];
      start++;
    }
    seen.add(nums[end]);
    currentSum += nums[end];
    result = Math.max(result, currentSum);
  }

  return result;
};
```

1696 Jump Game Vi

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1696. Jump Game VI
 * https://leetcode.com/problems/jump-game-vi/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums and an integer k.
 *
 * You are initially standing at index 0. In one move, you can jump at most k steps forward
without
 * going outside the boundaries of the array. That is, you can jump from index i to any index
in
 * the range [i + 1, min(n - 1, i + k)] inclusive.
 *
 * You want to reach the last index of the array (index n - 1). Your score is the sum of all
 * nums[j] for each index j you visited in the array.
 *
 * Return the maximum score you can get.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxResult = function(nums, k) {
  const n = nums.length;
  const maxScores = new Array(n).fill(-Infinity);
  const deque = [];

  maxScores[0] = nums[0];
  deque.push(0);

  for (let i = 1; i < n; i++) {
    while (deque.length && deque[0] < i - k) {
      deque.shift();
    }

    maxScores[i] = nums[i] + maxScores[deque[0]];

    while (deque.length && maxScores[i] >= maxScores[deque[deque.length - 1]]) {
      deque.pop();
    }

    deque.push(i);
  }

  return maxScores[n - 1];
};
```

1697 Checking Existence Of Edge Length Limited Paths

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1697. Checking Existence of Edge Length Limited Paths
 * https://leetcode.com/problems/checking-existence-of-edge-length-limited-paths/
 * Difficulty: Hard
 *
 * An undirected graph of n nodes is defined by edgeList, where edgeList[i] = [ui, vi, disi]
denotes
 * an edge between nodes ui and vi with distance disi. Note that there may be multiple edges
between
 * two nodes.
 *
 * Given an array queries, where queries[j] = [pj, qj, limitj], your task is to determine for
each
 * queries[j] whether there is a path between pj and qj such that each edge on the path has a
 * distance strictly less than limitj.
 *
 * Return a boolean array answer, where answer.length == queries.length and the jth value of
answer
 * is true if there is a path for queries[j] is true, and false otherwise.
 */

/**
 * @param {number} n
 * @param {number[][]} edgeList
 * @param {number[][]} queries
 * @return {boolean[]}
 */
var distanceLimitedPathsExist = function(n, edgeList, queries) {
  const parent = new Array(n).fill().map((_, i) => i);

  edgeList.sort((a, b) => a[2] - b[2]);
  const sortedQueries = queries.map((q, i) => [...q, i]).sort((a, b) => a[2] - b[2]);
  const result = new Array(queries.length).fill(false);

  let edgeIndex = 0;
  for (const [p, q, limit, index] of sortedQueries) {
    while (edgeIndex < edgeList.length && edgeList[edgeIndex][2] < limit) {
      union(edgeList[edgeIndex][0], edgeList[edgeIndex][1]);
      edgeIndex++;
    }
    result[index] = find(p) === find(q);
  }

  return result;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    parent[find(x)] = find(y);
  }
};
```

1698 Number Of Distinct Substrings In A String

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1698. Number of Distinct Substrings in a String
 * https://leetcode.com/problems/number-of-distinct-substrings-in-a-string/
 * Difficulty: Medium
 *
 * Given a string s, return the number of distinct substrings of s.
 *
 * A substring of a string is obtained by deleting any number of characters (possibly zero)
 * from the front of the string and any number (possibly zero) from the back of the string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countDistinct = function(s) {
  const set = new Set();
  const length = s.length;

  for (let start = 0; start < length; start++) {
    for (let end = start; end < length; end++) {
      set.add(s.substring(start, end + 1));
    }
  }

  return set.size;
};
```

1700 Number Of Students Unable To Eat Lunch

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1700. Number of Students Unable to Eat Lunch
 * https://leetcode.com/problems/number-of-students-unable-to-eat-lunch/
 * Difficulty: Easy
 *
 * The school cafeteria offers circular and square sandwiches at lunch break, referred to by
 * numbers 0 and 1 respectively. All students stand in a queue. Each student either prefers
 * square or circular sandwiches.
 *
 * The number of sandwiches in the cafeteria is equal to the number of students. The
sandwiches
 * are placed in a stack. At each step:
 * - If the student at the front of the queue prefers the sandwich on the top of the stack,
they
 *   will take it and leave the queue.
 * - Otherwise, they will leave it and go to the queue's end.
 *
 * This continues until none of the queue students want to take the top sandwich and are thus
 * unable to eat.
 *
 * You are given two integer arrays students and sandwiches where sandwiches[i] is the type of
 * the ith sandwich in the stack (i = 0 is the top of the stack) and students[j] is the
 * preference of the jth student in the initial queue (j = 0 is the front of the queue).
 * Return the number of students that are unable to eat.
 */

/**
 * @param {number[]} students
 * @param {number[]} sandwiches
 * @return {number}
 */
var countStudents = function(students, sandwiches) {
  const preferenceCount = [0, 0];
  for (const pref of students) {
    preferenceCount[pref]++;
  }
```

```
  for (const sandwich of sandwiches) {
    if (preferenceCount[sandwich] === 0) {
      return preferenceCount[0] + preferenceCount[1];
    }
    preferenceCount[sandwich]--;
  }

  return 0;
};
```

1701 Average Waiting Time

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1701. Average Waiting Time
 * https://leetcode.com/problems/average-waiting-time/
 * Difficulty: Medium
 *
 * There is a restaurant with a single chef. You are given an array customers, where
 * customers[i] = [arrivali, timei]:
 * - arrivali is the arrival time of the ith customer. The arrival times are sorted in
 *   non-decreasing order.
 * - timei is the time needed to prepare the order of the ith customer.
 *
 * When a customer arrives, he gives the chef his order, and the chef starts preparing it once
 * he is idle. The customer waits till the chef finishes preparing his order. The chef does
not
 * prepare food for more than one customer at a time. The chef prepares food for customers in
 * the order they were given in the input.
 *
 * Return the average waiting time of all customers. Solutions within 10-5 from the actual
 * answer are considered accepted.
 */

/**
 * @param {number[][]} customers
 * @return {number}
 */
var averageWaitingTime = function(customers) {
  let totalWait = 0;
  let currentTime = 0;

  for (const [arrival, prepTime] of customers) {
    const startTime = Math.max(arrival, currentTime);
    const finishTime = startTime + prepTime;
    totalWait += finishTime - arrival;
    currentTime = finishTime;
  }

  return totalWait / customers.length;
};
```

1702 Maximum Binary String After Change

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1702. Maximum Binary String After Change
 * https://leetcode.com/problems/maximum-binary-string-after-change/
 * Difficulty: Medium
 *
 * You are given a binary string binary consisting of only 0's or 1's. You can apply each of
 * the following operations any number of times:
 * - Operation 1: If the number contains the substring "00", you can replace it with "10".
```

```
 *   - For example, "00010" -> "10010"
 * - Operation 2: If the number contains the substring "10", you can replace it with "01".
 *   - For example, "00010" -> "00001"
 *
 * Return the maximum binary string you can obtain after any number of operations. Binary
 * string x is greater than binary string y if x's decimal representation is greater than
 * y's decimal representation.
 */

/**
 * @param {string} binary
 * @return {string}
 */
var maximumBinaryString = function(binary) {
  const n = binary.length;
  let zeroCount = 0;
  let firstZero = -1;

  for (let i = 0; i < n; i++) {
    if (binary[i] === '0') {
      if (firstZero === -1) firstZero = i;
      zeroCount++;
    }
  }

  if (zeroCount <= 1) return binary;

  const result = new Array(n).fill('1');
  result[firstZero + zeroCount - 1] = '0';

  return result.join('');
};
```

1703 Minimum Adjacent Swaps For K Consecutive Ones

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1703. Minimum Adjacent Swaps for K Consecutive Ones
 * https://leetcode.com/problems/minimum-adjacent-swaps-for-k-consecutive-ones/
 * Difficulty: Hard
 *
 * You are given an integer array, nums, and an integer k. nums comprises of only 0's and 1's.
 In
 * one move, you can choose two adjacent indices and swap their values.
 *
 * Return the minimum number of moves required so that nums has k consecutive 1's.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minMoves = function(nums, k) {
  const ones = [];
  for (let i = 0; i < nums.length; i++) {
    if (nums[i] === 1) ones.push(i);
  }

  const prefixSum = [0];
  for (let i = 0; i < ones.length; i++) {
    prefixSum.push(prefixSum[i] + ones[i]);
  }

  let minSwaps = Infinity;
  for (let i = 0; i <= ones.length - k; i++) {
    const j = i + k - 1;
    const mid = i + Math.floor(k / 2);
    const target = ones[mid];
```

```
    const leftCount = mid - i;
    const rightCount = j - mid;
    const leftSum = prefixSum[mid] - prefixSum[i];
    const rightSum = prefixSum[j + 1] - prefixSum[mid + 1];
    const swaps = (target * leftCount - leftSum) + (rightSum - target * rightCount);
    minSwaps = Math.min(minSwaps, swaps);
  }

  const medianAdjust = Math.floor(k / 2) * Math.ceil(k / 2);
  return minSwaps - medianAdjust;
};
```

1704 Determine If String Halves Are Alike

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1704. Determine if String Halves Are Alike
 * https://leetcode.com/problems/determine-if-string-halves-are-alike/
 * Difficulty: Easy
 *
 * You are given a string s of even length. Split this string into two halves of equal
lengths,
 * and let a be the first half and b be the second half.
 *
 * Two strings are alike if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A',
 * 'E', 'I', 'O', 'U'). Notice that s contains uppercase and lowercase letters.
 *
 * Return true if a and b are alike. Otherwise, return false.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var halvesAreAlike = function(s) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']);
  let firstHalfVowels = 0;
  let secondHalfVowels = 0;
  const mid = s.length / 2;

  for (let i = 0; i < mid; i++) {
    if (vowels.has(s[i])) firstHalfVowels++;
    if (vowels.has(s[i + mid])) secondHalfVowels++;
  }

  return firstHalfVowels === secondHalfVowels;
};
```

1705 Maximum Number Of Eaten Apples

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1705. Maximum Number of Eaten Apples
 * https://leetcode.com/problems/maximum-number-of-eaten-apples/
 * Difficulty: Medium
 *
 * There is a special kind of apple tree that grows apples every day for n days. On the ith
day,
 * the tree grows apples[i] apples that will rot after days[i] days, that is on day i +
days[i]
 * the apples will be rotten and cannot be eaten. On some days, the apple tree does not grow
any
 * apples, which are denoted by apples[i] == 0 and days[i] == 0.
```

```
 *
 * You decided to eat at most one apple a day (to keep the doctors away). Note that you can
keep
 * eating after the first n days.
 *
 * Given two integer arrays days and apples of length n, return the maximum number of apples
you
 * can eat.
 */

/**
 * @param {number[]} apples
 * @param {number[]} days
 * @return {number}
 */
var eatenApples = function(apples, days) {
  const expiryCounts = new Array(40001).fill(0);
  let result = 0;
  let earliestExpiry = Infinity;
  let maxExpiry = apples.length;

  for (let day = 0; day <= maxExpiry; day++) {
    if (earliestExpiry < day) earliestExpiry = day;

    if (day < apples.length && apples[day]) {
      const expiry = day + days[day] - 1;
      expiryCounts[expiry] += apples[day];
      earliestExpiry = Math.min(expiry, earliestExpiry);
      maxExpiry = Math.max(expiry, maxExpiry);
    }

    while (!expiryCounts[earliestExpiry] && earliestExpiry < maxExpiry) {
      earliestExpiry++;
    }

    if (expiryCounts[earliestExpiry]) {
      result++;
      expiryCounts[earliestExpiry]--;
    }
  }

  return result;
};
```

1706 Where Will The Ball Fall

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1706. Where Will the Ball Fall
 * https://leetcode.com/problems/where-will-the-ball-fall/
 * Difficulty: Medium
 *
 * You have a 2-D grid of size m x n representing a box, and you have n balls. The box is open
 on
 * the top and bottom sides.
 *
 * Each cell in the box has a diagonal board spanning two corners of the cell that can
redirect a
 * ball to the right or to the left.
 * - A board that redirects the ball to the right spans the top-left corner to the
bottom-right
 *   corner and is represented in the grid as 1.
 * - A board that redirects the ball to the left spans the top-right corner to the bottom-left
 *   corner and is represented in the grid as -1.
 *
 * We drop one ball at the top of each column of the box. Each ball can get stuck in the box
or fall
 * out of the bottom. A ball gets stuck if it hits a "V" shaped pattern between two boards or
if a
```

```
 * board redirects the ball into either wall of the box.
 *
 * Return an array answer of size n where answer[i] is the column that the ball falls out of
at the
 * bottom after dropping the ball from the ith column at the top, or -1 if the ball gets stuck
 in
 * the box.
 */

/**
 * @param {number[][]} grid
 * @return {number[]}
 */
var findBall = function(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const result = new Array(n);

  for (let col = 0; col < n; col++) {
    let currentCol = col;
    let row = 0;

    while (row < m) {
      const nextCol = currentCol + grid[row][currentCol];

      if (nextCol < 0 || nextCol >= n || grid[row][currentCol] !== grid[row][nextCol]) {
        currentCol = -1;
        break;
      }

      currentCol = nextCol;
      row++;
    }

    result[col] = currentCol;
  }

  return result;
};
```

1707 Maximum Xor With An Element From Array

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1707. Maximum XOR With an Element From Array
 * https://leetcode.com/problems/maximum-xor-with-an-element-from-array/
 * Difficulty: Hard
 *
 * You are given an array nums consisting of non-negative integers. You are also given a
queries
 * array, where queries[i] = [xi, mi].
 *
 * The answer to the ith query is the maximum bitwise XOR value of xi and any element of nums
that
 * does not exceed mi. In other words, the answer is max(nums[j] XOR xi) for all j such that
 * nums[j] <= mi. If all elements in nums are larger than mi, then the answer is -1.
 *
 * Return an integer array answer where answer.length == queries.length and answer[i] is the
 * answer to the ith query.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} queries
 * @return {number[]}
 */
var maximizeXor = function(nums, queries) {
  nums.sort((a, b) => a - b);
```

```
  const sortedQueries = queries.map((q, i) => [q[0], q[1], i]).sort((a, b) => a[1] - b[1]);
  const result = new Array(queries.length).fill(-1);

  const trie = {};
  let numIndex = 0;

  for (const [x, m, queryIndex] of sortedQueries) {
    while (numIndex < nums.length && nums[numIndex] <= m) {
      let node = trie;
      for (let bit = 30; bit >= 0; bit--) {
        const bitValue = (nums[numIndex] >> bit) & 1;
        if (!node[bitValue]) node[bitValue] = {};
        node = node[bitValue];
      }
      numIndex++;
    }

    if (numIndex === 0) continue;

    let maxXor = 0;
    let node = trie;
    for (let bit = 30; bit >= 0; bit--) {
      const bitValue = (x >> bit) & 1;
      const oppositeBit = bitValue ^ 1;
      if (node[oppositeBit]) {
        maxXor |= (1 << bit);
        node = node[oppositeBit];
      } else {
        node = node[bitValue];
      }
    }
    result[queryIndex] = maxXor;
  }

  return result;
};
```

1708 Largest Subarray Length K

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1708. Largest Subarray Length K
 * https://leetcode.com/problems/largest-subarray-length-k/
 * Difficulty: Easy
 *
 * An array A is larger than some array B if for the first index i where A[i] != B[i], A[i] >
B[i].
 *
 * For example, consider 0-indexing:
 * - [1,3,2,4] > [1,2,2,4], since at index 1, 3 > 2.
 * - [1,4,4,4] < [2,1,1,1], since at index 0, 1 < 2.
 *
 * A subarray is a contiguous subsequence of the array.
 *
 * Given an integer array nums of distinct integers, return the largest subarray of nums of
 * length k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var largestSubarray = function(nums, k) {
  let maxStartIndex = 0;

  for (let i = 1; i <= nums.length - k; i++) {
    if (nums[i] > nums[maxStartIndex]) {
      maxStartIndex = i;
```

```
    }
  }

  return nums.slice(maxStartIndex, maxStartIndex + k);
};
```

1710 Maximum Units On A Truck

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1710. Maximum Units on a Truck
 * https://leetcode.com/problems/maximum-units-on-a-truck/
 * Difficulty: Easy
 *
 * You are assigned to put some amount of boxes onto one truck. You are given a 2D array
 * boxTypes, where boxTypes[i] = [numberOfBoxesi, numberOfUnitsPerBoxi]:
 * - numberOfBoxesi is the number of boxes of type i.
 * - numberOfUnitsPerBoxi is the number of units in each box of the type i.
 *
 * You are also given an integer truckSize, which is the maximum number of boxes that can be
 * put on the truck. You can choose any boxes to put on the truck as long as the number of
 * boxes does not exceed truckSize.
 *
 * Return the maximum total number of units that can be put on the truck.
 */

/**
 * @param {number[][]} boxTypes
 * @param {number} truckSize
 * @return {number}
 */
var maximumUnits = function(boxTypes, truckSize) {
  boxTypes.sort((a, b) => b[1] - a[1]);
  let result = 0;
  let remainingBoxes = truckSize;

  for (const [count, units] of boxTypes) {
    const boxesToTake = Math.min(count, remainingBoxes);
    result += boxesToTake * units;
    remainingBoxes -= boxesToTake;
    if (remainingBoxes === 0) break;
  }

  return result;
};
```

1711 Count Good Meals

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1711. Count Good Meals
 * https://leetcode.com/problems/count-good-meals/
 * Difficulty: Medium
 *
 * A good meal is a meal that contains exactly two different food items with a sum of
deliciousness
 * equal to a power of two.
 *
 * You can pick any two different foods to make a good meal.
 *
 * Given an array of integers deliciousness where deliciousness[i] is the deliciousness of the
 * ith item of food, return the number of different good meals you can make from this list
 * modulo 109 + 7.
```

```
 *
 * Note that items with different indices are considered different even if they have the same
 * deliciousness value.
 */

/**
 * @param {number[]} deliciousness
 * @return {number}
 */
var countPairs = function(deliciousness) {
  const MOD = 1e9 + 7;
  const frequency = new Map();
  let result = 0;

  for (const value of deliciousness) {
    for (let power = 1; power <= 1 << 21; power <<= 1) {
      const complement = power - value;
      if (frequency.has(complement)) {
        result = (result + frequency.get(complement)) % MOD;
      }
    }
    frequency.set(value, (frequency.get(value) || 0) + 1);
  }

  return result;
};
```

1712 Ways To Split Array Into Three Subarrays

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1712. Ways to Split Array Into Three Subarrays
 * https://leetcode.com/problems/ways-to-split-array-into-three-subarrays/
 * Difficulty: Medium
 *
 * A split of an integer array is good if:
 * - The array is split into three non-empty contiguous subarrays - named left, mid, right
 *   respectively from left to right.
 * - The sum of the elements in left is less than or equal to the sum of the elements in mid,
 *   and the sum of the elements in mid is less than or equal to the sum of the elements in
right.
 *
 * Given nums, an array of non-negative integers, return the number of good ways to split
nums.
 * As the number may be too large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var waysToSplit = function(nums) {
  const MOD = 1e9 + 7;
  const n = nums.length;
  const prefixSum = new Array(n).fill(0);

  prefixSum[0] = nums[0];
  for (let i = 1; i < n; i++) {
    prefixSum[i] = prefixSum[i - 1] + nums[i];
  }

  let splitCount = 0;
  for (let i = 0; i < n - 2; i++) {
    let left = i + 1;
    let right = n - 2;
    let validStart = -1;
    let validEnd = -1;

    while (left <= right) {
```

```
      const mid = Math.floor((left + right) / 2);
      const leftSum = prefixSum[i];
      const midSum = prefixSum[mid] - prefixSum[i];
      const rightSum = prefixSum[n - 1] - prefixSum[mid];

      if (leftSum <= midSum && midSum <= rightSum) {
        validStart = mid;
        right = mid - 1;
      } else if (leftSum > midSum) {
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }

    left = validStart === -1 ? n - 1 : validStart;
    right = n - 2;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      const leftSum = prefixSum[i];
      const midSum = prefixSum[mid] - prefixSum[i];
      const rightSum = prefixSum[n - 1] - prefixSum[mid];

      if (leftSum <= midSum && midSum <= rightSum) {
        validEnd = mid;
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }

    if (validStart !== -1 && validEnd !== -1 && validStart <= validEnd) {
      splitCount = (splitCount + validEnd - validStart + 1) % MOD;
    }
  }

  return splitCount;
};
```

1713 Minimum Operations To Make A Subsequence

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1713. Minimum Operations to Make a Subsequence
 * https://leetcode.com/problems/minimum-operations-to-make-a-subsequence/
 * Difficulty: Hard
 *
 * You are given an array target that consists of distinct integers and another integer array
 * arr that can have duplicates.
 *
 * In one operation, you can insert any integer at any position in arr. For example, if
 * arr = [1,4,1,2], you can add 3 in the middle and make it [1,4,3,1,2]. Note that you can
 * insert the integer at the very beginning or end of the array.
 *
 * Return the minimum number of operations needed to make target a subsequence of arr.
 *
 * A subsequence of an array is a new array generated from the original array by deleting some
 * elements (possibly none) without changing the remaining elements' relative order. For
 * example, [2,7,4] is a subsequence of [4,2,3,7,2,1,4] (the underlined elements),
 * while [2,4,2] is not.
 */

/**
 * @param {number[]} target
 * @param {number[]} arr
 * @return {number}
 */
```

```
var minOperations = function(target, arr) {
  const valueToIndex = new Map(target.map((val, i) => [val, i]));
  const sequence = arr.filter(val => valueToIndex.has(val)).map(val => valueToIndex.get(val));

  const lis = [];
  for (const index of sequence) {
    const pos = binarySearch(lis, index);
    if (pos === lis.length) {
      lis.push(index);
    } else {
      lis[pos] = index;
    }
  }

  return target.length - lis.length;
};

function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (arr[mid] < target) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
}
```

1714 Sum Of Special Evenly Spaced Elements In Array

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1714. Sum Of Special Evenly-Spaced Elements In Array
 * https://leetcode.com/problems/sum-of-special-evenly-spaced-elements-in-array/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums consisting of n non-negative integers.
 *
 * You are also given an array queries, where queries[i] = [xi, yi]. The answer to the ith
 * query is the sum of all nums[j] where xi <= j < n and (j - xi) is divisible by yi.
 *
 * Return an array answer where answer.length == queries.length and answer[i] is the answer
 * to the ith query modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} queries
 * @return {number[]}
 */
var solve = function(nums, queries) {
  const MOD = 1e9 + 7;
  const n = nums.length;
  const threshold = Math.sqrt(n);
  const precomputed = new Map();

  for (let step = 1; step <= threshold; step++) {
    const suffixSums = new Array(n).fill(0);
    for (let start = n - 1; start >= 0; start--) {
      if (start + step < n) {
        suffixSums[start] = (nums[start] + suffixSums[start + step]) % MOD;
      } else {
        suffixSums[start] = nums[start];
      }
```

```
    }
    precomputed.set(step, suffixSums);
  }

  const result = [];

  for (const [startIndex, step] of queries) {
    if (step <= threshold) {
      result.push(precomputed.get(step)[startIndex] || 0);
    } else {
      let sum = 0;
      for (let i = startIndex; i < n; i += step) {
        sum = (sum + nums[i]) % MOD;
      }
      result.push(sum);
    }
  }

  return result;
};
```

1716 Calculate Money In Leetcode Bank

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1716. Calculate Money in Leetcode Bank
 * https://leetcode.com/problems/calculate-money-in-leetcode-bank/
 * Difficulty: Easy
 *
 * Hercy wants to save money for his first car. He puts money in the Leetcode bank every day.
 *
 * He starts by putting in $1 on Monday, the first day. Every day from Tuesday to Sunday, he
will
 * put in $1 more than the day before. On every subsequent Monday, he will put in $1 more than
 the
 * previous Monday. Given n, return the total amount of money he will have in the Leetcode
bank at
 * the end of the nth day.
 */

/**
 * @param {number} n
 * @return {number}
 */
var totalMoney = function(n) {
  let mondayCount = Math.floor(n / 7);
  let aggregate = (mondayCount ** 2 + 7 * mondayCount) / 2;
  let result = aggregate * 7;
  for (let i = 0; i < n - mondayCount * 7; i++) {
    result += mondayCount + i + 1;
  }
  return result;
};
```

1717 Maximum Score From Removing Substrings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1717. Maximum Score From Removing Substrings
 * https://leetcode.com/problems/maximum-score-from-removing-substrings/
 * Difficulty: Medium
 *
 * You are given a string s and two integers x and y. You can perform two types of operations
```

```
 * any number of times.
 *
 * - Remove substring "ab" and gain x points.
 *   - For example, when removing "ab" from "cabxbae" it becomes "cxbae".
 * - Remove substring "ba" and gain y points.
 *   - For example, when removing "ba" from "cabxbae" it becomes "cabxe".
 *
 * Return the maximum points you can gain after applying the above operations on s.
 */

/**
 * @param {string} s
 * @param {number} x
 * @param {number} y
 * @return {number}
 */
var maximumGain = function(s, x, y) {
  let result = 0;
  const stack = [];

  const [primary, secondary, primaryScore, secondaryScore] = x >= y
    ? ['ab', 'ba', x, y]
    : ['ba', 'ab', y, x];

  for (const char of s) {
    if (stack.length && stack[stack.length - 1] === primary[0] && char === primary[1]) {
      stack.pop();
      result += primaryScore;
    } else {
      stack.push(char);
    }
  }

  const remaining = [];
  for (const char of stack) {
    if (remaining.length && remaining[remaining.length - 1] === secondary[0]
        && char === secondary[1]) {
      remaining.pop();
      result += secondaryScore;
    } else {
      remaining.push(char);
    }
  }

  return result;
};
```

1718 Construct The Lexicographically Largest Valid Sequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1718. Construct the Lexicographically Largest Valid Sequence
 * https://leetcode.com/problems/construct-the-lexicographically-largest-valid-sequence/
 * Difficulty: Medium
 *
 * Given an integer n, find a sequence that satisfies all of the following:
 * - The integer 1 occurs once in the sequence.
 * - Each integer between 2 and n occurs twice in the sequence.
 * - For every integer i between 2 and n, the distance between the two occurrences of i is
 *   exactly i.
 *
 * The distance between two numbers on the sequence, a[i] and a[j], is the absolute difference
 * of their indices, |j - i|.
 *
 * Return the lexicographically largest sequence. It is guaranteed that under the given
 * constraints, there is always a solution.
 *
 * A sequence a is lexicographically larger than a sequence b (of the same length) if in the
first
```

```
 * position where a and b differ, sequence a has a number greater than the corresponding
number in
 * b. For example, [0,1,9,0] is lexicographically larger than [0,1,5,6] because the first
position
 * they differ is at the third number, and 9 is greater than 5.
 */

/**
 * @param {number} n
 * @return {number[]}
 */
var constructDistancedSequence = function(n) {
  const result = new Array(2 * n - 1).fill(0);
  const group = new Array(n + 1).fill(false);

  backtrack(0);

  function backtrack(index) {
    if (index === 2 * n - 1) {
      return true;
    } else if (result[index] !== 0) {
      return backtrack(index + 1);
    }
    for (let num = n; num >= 1; num--) {
      if (group[num]) {
        continue;
      }
      group[num] = true;
      result[index] = num;
      if (num === 1 || (index + num < 2 * n - 1 && result[index + num] === 0)) {
        if (num > 1) {
          result[index + num] = num;
        }
        if (backtrack(index + 1)) {
          return true;
        }
        if (num > 1) {
          result[index + num] = 0;
        }
      }
      result[index] = 0;
      group[num] = false;
    }
    return false;
  }

  return result;
};
```

1719 Number Of Ways To Reconstruct A Tree

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1719. Number Of Ways To Reconstruct A Tree
 * https://leetcode.com/problems/number-of-ways-to-reconstruct-a-tree/
 * Difficulty: Hard
 *
 * You are given an array pairs, where pairs[i] = [xi, yi], and:
 * - There are no duplicates.
 * - xi < yi
 *
 * Let ways be the number of rooted trees that satisfy the following conditions:
 * - The tree consists of nodes whose values appeared in pairs.
 * - A pair [xi, yi] exists in pairs if and only if xi is an ancestor of yi or yi is
 *   an ancestor of xi.
 * - Note: the tree does not have to be a binary tree.
 *
 * Two ways are considered to be different if there is at least one node that has different
 * parents in both ways.
```

```
 *
 * Return:
 * - 0 if ways == 0
 * - 1 if ways == 1
 * - 2 if ways > 1
 *
 * A rooted tree is a tree that has a single root node, and all edges are oriented to be
 * outgoing from the root.
 *
 * An ancestor of a node is any node on the path from the root to that node (excluding
 * the node itself). The root has no ancestors.
 */

/**
 * @param {number[][]} pairs
 * @return {number}
 */
var checkWays = function(pairs) {
  const graph = new Map();
  for (const [x, y] of pairs) {
    if (!graph.has(x)) graph.set(x, new Set());
    if (!graph.has(y)) graph.set(y, new Set());
    graph.get(x).add(y);
    graph.get(y).add(x);
  }

  const nodes = [...graph.keys()].sort((a, b) => graph.get(b).size - graph.get(a).size);
  const n = nodes.length;
  if (n === 1) return 1;

  const root = nodes[0];
  if (graph.get(root).size !== n - 1) return 0;

  let equalDegreeCount = 0;
  for (let i = 1; i < n; i++) {
    const node = nodes[i];
    let found = false;

    for (let j = i - 1; j >= 0; j--) {
      const ancestor = nodes[j];
      if (graph.get(node).has(ancestor)) {
        for (const neighbor of graph.get(node)) {
          if (neighbor !== ancestor && !graph.get(ancestor).has(neighbor)) {
            return 0;
          }
        }
        if (graph.get(node).size === graph.get(ancestor).size) {
          equalDegreeCount++;
        }
        found = true;
        break;
      }
    }

    if (!found) return 0;
  }

  return equalDegreeCount > 0 ? 2 : 1;
};
```

1720 Decode Xored Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1720. Decode XORed Array
 * https://leetcode.com/problems/decode-xored-array/
 * Difficulty: Easy
 *
 * There is a hidden integer array arr that consists of n non-negative integers.
```

```
 *
 * It was encoded into another integer array encoded of length n - 1, such that
 * encoded[i] = arr[i] XOR arr[i + 1]. For example, if arr = [1,0,2,1], then encoded =
[1,2,3].
 *
 * You are given the encoded array. You are also given an integer first, that is the first
 * element of arr, i.e. arr[0].
 *
 * Return the original array arr. It can be proved that the answer exists and is unique.
 */

/**
 * @param {number[]} encoded
 * @param {number} first
 * @return {number[]}
 */
var decode = function(encoded, first) {
  const result = [first];

  for (let i = 0; i < encoded.length; i++) {
    result.push(result[i] ^ encoded[i]);
  }

  return result;
};
```

1721 Swapping Nodes In A Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1721. Swapping Nodes in a Linked List
 * https://leetcode.com/problems/swapping-nodes-in-a-linked-list/
 * Difficulty: Medium
 *
 * You are given the head of a linked list, and an integer k.
 *
 * Return the head of the linked list after swapping the values of the kth node from the
beginning
 * and the kth node from the end (the list is 1-indexed).
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @param {number} k
 * @return {ListNode}
 */
var swapNodes = function(head, k) {
  let firstNode = head;
  for (let i = 1; i < k; i++) {
    firstNode = firstNode.next;
  }

  let slow = head;
  let secondNode = firstNode.next;
  while (secondNode) {
    slow = slow.next;
    secondNode = secondNode.next;
  }

  const temp = firstNode.val;
  firstNode.val = slow.val;
  slow.val = temp;
```

```
  return head;
};
```

1722 Minimize Hamming Distance After Swap Operations

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1722. Minimize Hamming Distance After Swap Operations
 * https://leetcode.com/problems/minimize-hamming-distance-after-swap-operations/
 * Difficulty: Medium
 *
 * You are given two integer arrays, source and target, both of length n. You are also given
an
 * array allowedSwaps where each allowedSwaps[i] = [ai, bi] indicates that you are allowed to
 * swap the elements at index ai and index bi (0-indexed) of array source. Note that you can
 * swap elements at a specific pair of indices multiple times and in any order.
 *
 * The Hamming distance of two arrays of the same length, source and target, is the number of
 * positions where the elements are different. Formally, it is the number of indices i for
 * 0 <= i <= n-1 where source[i] != target[i] (0-indexed).
 *
 * Return the minimum Hamming distance of source and target after performing any amount of
swap
 * operations on array source.
 */

/**
 * @param {number[]} source
 * @param {number[]} target
 * @param {number[][]} allowedSwaps
 * @return {number}
 */
var minimumHammingDistance = function(source, target, allowedSwaps) {
  const n = source.length;
  const parent = new Array(n).fill().map((_, i) => i);

  for (const [a, b] of allowedSwaps) {
    union(a, b);
  }

  const groups = new Map();
  for (let i = 0; i < n; i++) {
    const root = find(i);
    if (!groups.has(root)) {
      groups.set(root, { source: [], target: [] });
    }
    groups.get(root).source.push(source[i]);
    groups.get(root).target.push(target[i]);
  }

  let result = 0;
  for (const { source, target } of groups.values()) {
    const sourceCount = new Map();
    for (const num of source) {
      sourceCount.set(num, (sourceCount.get(num) || 0) + 1);
    }
    for (const num of target) {
      const count = sourceCount.get(num) || 0;
      if (count === 0) {
        result++;
      } else {
        sourceCount.set(num, count - 1);
      }
    }
  }

  return result;
```

```
  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    parent[find(x)] = find(y);
  }
};
```

1723 Find Minimum Time To Finish All Jobs

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1723. Find Minimum Time to Finish All Jobs
 * https://leetcode.com/problems/find-minimum-time-to-finish-all-jobs/
 * Difficulty: Hard
 *
 * You are given an integer array jobs, where jobs[i] is the amount of time it takes to
complete
 * the ith job.
 *
 * There are k workers that you can assign jobs to. Each job should be assigned to exactly one
 * worker. The working time of a worker is the sum of the time it takes to complete all jobs
 * assigned to them. Your goal is to devise an optimal assignment such that the maximum
working
 * time of any worker is minimized.
 *
 * Return the minimum possible maximum working time of any assignment.
 */

/**
 * @param {number[]} jobs
 * @param {number} k
 * @return {number}
 */
var minimumTimeRequired = function(jobs, k) {
  const n = jobs.length;
  const workerTimes = new Array(k).fill(0);
  let result = Infinity;

  jobs.sort((a, b) => b - a);
  backtrack(0, 0);

  return result;

  function backtrack(jobIndex, maxTime) {
    if (jobIndex === n) {
      result = Math.min(result, maxTime);
      return;
    }

    if (maxTime >= result) return;

    for (let i = 0; i < k; i++) {
      if (workerTimes[i] + jobs[jobIndex] >= result) continue;

      workerTimes[i] += jobs[jobIndex];
      backtrack(jobIndex + 1, Math.max(maxTime, workerTimes[i]));
      workerTimes[i] -= jobs[jobIndex];

      if (workerTimes[i] === 0) break;
    }
  }
};
```

1724 Checking Existence Of Edge Length Limited Paths Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1724. Checking Existence of Edge Length Limited Paths II
 * https://leetcode.com/problems/checking-existence-of-edge-length-limited-paths-ii/
 * Difficulty: Hard
 *
 * An undirected graph of n nodes is defined by edgeList, where edgeList[i] = [ui, vi, disi]
 * denotes an edge between nodes ui and vi with distance disi. Note that there may be multiple
 * edges between two nodes, and the graph may not be connected.
 *
 * Implement the DistanceLimitedPathsExist class:
 * - DistanceLimitedPathsExist(int n, int[][] edgeList) Initializes the class with an
 *   undirected graph.
 * - boolean query(int p, int q, int limit) Returns true if there exists a path from p to q
 *   such that each edge on the path has a distance strictly less than limit, and otherwise
 *   false.
 */

/**
 * @param {number} n
 * @param {number[][]} edgeList
 */
var DistanceLimitedPathsExist = function(n, edgeList) {
  this.nodeCount = n;
  this.sortedEdges = edgeList.slice().sort((a, b) => a[2] - b[2]);
  this.unionFindByLimit = new Map();
  this.processedLimits = new Set();
};

/**
 * @param {number} p
 * @param {number} q
 * @param {number} limit
 * @return {boolean}
 */
DistanceLimitedPathsExist.prototype.query = function(p, q, limit) {
  if (!this.processedLimits.has(limit)) {
    const unionFind = new UnionFind(this.nodeCount);

    for (const [u, v, weight] of this.sortedEdges) {
      if (weight >= limit) break;
      unionFind.union(u, v);
    }

    this.unionFindByLimit.set(limit, unionFind);
    this.processedLimits.add(limit);
  }

  return this.unionFindByLimit.get(limit).connected(p, q);
};

class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);
    this.rank = new Array(n).fill(0);
  }

  find(x) {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]);
    }
    return this.parent[x];
  }

  union(x, y) {
    const rootX = this.find(x);
    const rootY = this.find(y);
```

```
    if (rootX !== rootY) {
      if (this.rank[rootX] < this.rank[rootY]) {
        this.parent[rootX] = rootY;
      } else if (this.rank[rootX] > this.rank[rootY]) {
        this.parent[rootY] = rootX;
      } else {
        this.parent[rootY] = rootX;
        this.rank[rootX]++;
      }
    }
  }

  connected(x, y) {
    return this.find(x) === this.find(y);
  }
}
```

1725 Number Of Rectangles That Can Form The Largest Square

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1725. Number Of Rectangles That Can Form The Largest Square
 * https://leetcode.com/problems/number-of-rectangles-that-can-form-the-largest-square/
 * Difficulty: Easy
 *
 * You are given an array rectangles where rectangles[i] = [li, wi] represents the ith
rectangle
 * of length li and width wi.
 *
 * You can cut the ith rectangle to form a square with a side length of k if both k <= li and
 * k <= wi. For example, if you have a rectangle [4,6], you can cut it to get a square with a
 * side length of at most 4.
 *
 * Let maxLen be the side length of the largest square you can obtain from any of the given
 * rectangles.
 *
 * Return the number of rectangles that can make a square with a side length of maxLen.
 */

/**
 * @param {number[][]} rectangles
 * @return {number}
 */
var countGoodRectangles = function(rectangles) {
  let maxSide = 0;
  let result = 0;

  for (const [length, width] of rectangles) {
    const side = Math.min(length, width);
    if (side > maxSide) {
      maxSide = side;
      result = 1;
    } else if (side === maxSide) {
      result++;
    }
  }

  return result;
};
```

1726 Tuple With Same Product

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1726. Tuple with Same Product
 * https://leetcode.com/problems/tuple-with-same-product/
 * Difficulty: Medium
 *
 * Given an array nums of distinct positive integers, return the number of tuples (a, b, c, d)
 * such that a * b = c * d where a, b, c, and d are elements of nums, and a != b != c != d.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var tupleSameProduct = function(nums) {
  const map = new Map();
  let count = 0;

  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j < nums.length; j++) {
      map.set(nums[i] * nums[j], map.get(nums[i] * nums[j]) + 1 || 1);
    }
  }

  Array.from(map).forEach(([key, value]) => {
    count += value > 1 ? value * (value - 1) / 2 : 0;
  });

  return count * 8;
};
```

1727 Largest Submatrix With Rearrangements

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1727. Largest Submatrix With Rearrangements
 * https://leetcode.com/problems/largest-submatrix-with-rearrangements/
 * Difficulty: Medium
 *
 * You are given a binary matrix matrix of size m x n, and you are allowed to rearrange the
 * columns of the matrix in any order.
 *
 * Return the area of the largest submatrix within matrix where every element of the submatrix
 * is 1 after reordering the columns optimally.
 */

/**
 * @param {number[][]} matrix
 * @return {number}
 */
var largestSubmatrix = function(matrix) {
  const rows = matrix.length;
  const cols = matrix[0].length;
  let result = 0;

  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      if (row > 0 && matrix[row][col] === 1) {
        matrix[row][col] += matrix[row - 1][col];
      }
    }

    const heights = matrix[row].slice().sort((a, b) => b - a);
    for (let i = 0; i < cols; i++) {
      if (heights[i] === 0) break;
      result = Math.max(result, heights[i] * (i + 1));
    }
  }
```

```
  return result;
};
```

1728 Cat And Mouse Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1728. Cat and Mouse II
 * https://leetcode.com/problems/cat-and-mouse-ii/
 * Difficulty: Hard
 *
 * A game is played by a cat and a mouse named Cat and Mouse.
 *
 * The environment is represented by a grid of size rows x cols, where each element is a wall,
 * floor, player (Cat, Mouse), or food.
 * - Players are represented by the characters 'C'(Cat),'M'(Mouse).
 * - Floors are represented by the character '.' and can be walked on.
 * - Walls are represented by the character '#' and cannot be walked on.
 * - Food is represented by the character 'F' and can be walked on.
 * - There is only one of each character 'C', 'M', and 'F' in grid.
 *
 * Mouse and Cat play according to the following rules:
 * - Mouse moves first, then they take turns to move.
 * - During each turn, Cat and Mouse can jump in one of the four directions (left, right, up,
down).
 *   They cannot jump over the wall nor outside of the grid.
 * - catJump, mouseJump are the maximum lengths Cat and Mouse can jump at a time,
respectively.
 *   Cat and Mouse can jump less than the maximum length.
 * - Staying in the same position is allowed.
 * - Mouse can jump over Cat.
 *
 * The game can end in 4 ways:
 * - If Cat occupies the same position as Mouse, Cat wins.
 * - If Cat reaches the food first, Cat wins.
 * - If Mouse reaches the food first, Mouse wins.
 * - If Mouse cannot get to the food within 1000 turns, Cat wins.
 *
 * Given a rows x cols matrix grid and two integers catJump and mouseJump, return true if
Mouse
 * can win the game if both Cat and Mouse play optimally, otherwise return false.
 */

/**
 * @param {string[]} grid
 * @param {number} catJump
 * @param {number} mouseJump
 * @return {boolean}
 */
var canMouseWin = function(grid, catJump, mouseJump) {
  if (typeof grid === 'string') grid = [grid];
  const rows = grid.length;
  const cols = grid[0].length;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  let mouse;
  let cat;
  let food;
  let available = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      const cell = grid[i][j];
      if (cell !== '#') available++;
      if (cell === 'M') mouse = [i, j];
      else if (cell === 'C') cat = [i, j];
      else if (cell === 'F') food = [i, j];
    }
  }
```

```
  const memo = new Map();

  return canWin(0, mouse, cat);

  function getKey(turn, [mr, mc], [cr, cc]) {
    return `${turn},${mr},${mc},${cr},${cc}`;
  }

  function isValid(r, c) {
    return r >= 0 && r < rows && c >= 0 && c < cols && grid[r][c] !== '#';
  }

  function canWin(turn, mousePos, catPos) {
    if (turn >= available * 2) return false;

    const key = getKey(turn, mousePos, catPos);
    if (memo.has(key)) return memo.get(key);

    const isMouseTurn = turn % 2 === 0;
    const [r, c] = isMouseTurn ? mousePos : catPos;
    const maxJump = isMouseTurn ? mouseJump : catJump;

    for (const [dr, dc] of dirs) {
      for (let jump = 0; jump <= maxJump; jump++) {
        const nr = r + dr * jump;
        const nc = c + dc * jump;
        if (!isValid(nr, nc)) break;

        if (isMouseTurn) {
          if (grid[nr][nc] === 'F' || canWin(turn + 1, [nr, nc], catPos)) {
            memo.set(key, true);
            return true;
          }
        } else {
          if (grid[nr][nc] === 'F' || (nr === mousePos[0] && nc === mousePos[1])
              || !canWin(turn + 1, mousePos, [nr, nc])) {
            memo.set(key, false);
            return false;
          }
        }
      }
    }

    memo.set(key, !isMouseTurn);
    return !isMouseTurn;
  }
};
```

1730 Shortest Path To Get Food

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1730. Shortest Path to Get Food
 * https://leetcode.com/problems/shortest-path-to-get-food/
 * Difficulty: Medium
 *
 * You are starving and you want to eat food as quickly as possible. You want to find the
 * shortest path to arrive at any food cell.
 *
 * You are given an m x n character matrix, grid, of these different types of cells:
 * - '*' is your location. There is exactly one '*' cell.
 * - '#' is a food cell. There may be multiple food cells.
 * - 'O' is free space, and you can travel through these cells.
 * - 'X' is an obstacle, and you cannot travel through these cells.
 *
 * You can travel to any adjacent cell north, east, south, or west of your current location
 * if there is not an obstacle.
 *
```

```
 * Return the length of the shortest path for you to reach any food cell. If there is no
 * path for you to reach food, return -1.
 */

/**
 * @param {character[][]} grid
 * @return {number}
 */
var getFood = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  let startRow = 0;
  let startCol = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === '*') {
        startRow = i;
        startCol = j;
        break;
      }
    }
  }

  const queue = [[startRow, startCol, 0]];
  const visited = new Set();
  visited.add(`${startRow},${startCol}`);

  while (queue.length > 0) {
    const [currentRow, currentCol, steps] = queue.shift();

    if (grid[currentRow][currentCol] === '#') {
      return steps;
    }

    for (const [deltaRow, deltaCol] of directions) {
      const newRow = currentRow + deltaRow;
      const newCol = currentCol + deltaCol;
      const key = `${newRow},${newCol}`;

      if (newRow >= 0 && newRow < rows && newCol >= 0 && newCol < cols && !visited.has(key)
         && grid[newRow][newCol] !== 'X') {
        visited.add(key);
        queue.push([newRow, newCol, steps + 1]);
      }
    }
  }

  return -1;
};
```

1732 Find The Highest Altitude

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1732. Find the Highest Altitude
 * https://leetcode.com/problems/find-the-highest-altitude/
 * Difficulty: Easy
 *
 * There is a biker going on a road trip. The road trip consists of n + 1 points at different
 * altitudes. The biker starts his trip on point 0 with altitude equal 0.
 *
 * You are given an integer array gain of length n where gain[i] is the net gain in altitude
 * between points i and i + 1 for all (0 <= i < n). Return the highest altitude of a point.
 */

/**
 * @param {number[]} gain
```

```
 * @return {number}
 */
var largestAltitude = function(gain) {
  return Math.max(...gain.reduce((all, n, i) => all.push(all[i] - n * -1) && all, [0]));
};
```

1733 Minimum Number Of People To Teach

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1733. Minimum Number of People to Teach
 * https://leetcode.com/problems/minimum-number-of-people-to-teach/
 * Difficulty: Medium
 *
 * On a social network consisting of m users and some friendships between users, two users can
 * communicate with each other if they know a common language.
 *
 * You are given an integer n, an array languages, and an array friendships where:
 * - There are n languages numbered 1 through n,
 * - languages[i] is the set of languages the ith user knows, and
 * - friendships[i] = [ui, vi] denotes a friendship between the users ui and vi.
 *
 * You can choose one language and teach it to some users so that all friends can communicate
 * with each other. Return the minimum number of users you need to teach.
 *
 * Note that friendships are not transitive, meaning if x is a friend of y and y is a friend
of
 * z, this doesn't guarantee that x is a friend of z.
 */

/**
 * @param {number} n
 * @param {number[][]} languages
 * @param {number[][]} friendships
 * @return {number}
 */
var minimumTeachings = function(n, languages, friendships) {
  const languageUsers = Array.from({ length: n + 1 }, () => new Set());
  const nonCommunicating = new Set();

  for (let i = 0; i < languages.length; i++) {
    for (const lang of languages[i]) {
      languageUsers[lang].add(i + 1);
    }
  }

  for (const [u, v] of friendships) {
    let canCommunicate = false;
    for (const lang of languages[u - 1]) {
      if (languages[v - 1].includes(lang)) {
        canCommunicate = true;
        break;
      }
    }
    if (!canCommunicate) {
      nonCommunicating.add(u);
      nonCommunicating.add(v);
    }
  }

  let result = Infinity;
  for (let lang = 1; lang <= n; lang++) {
    let usersToTeach = 0;
    for (const user of nonCommunicating) {
      if (!languages[user - 1].includes(lang)) {
        usersToTeach++;
      }
    }
    result = Math.min(result, usersToTeach);
```

```
  }

  return result;
};
```

1734 Decode Xored Permutation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1734. Decode XORed Permutation
 * https://leetcode.com/problems/decode-xored-permutation/
 * Difficulty: Medium
 *
 * There is an integer array perm that is a permutation of the first n positive integers,
 * where n is always odd.
 *
 * It was encoded into another integer array encoded of length n - 1, such that
 * encoded[i] = perm[i] XOR perm[i + 1]. For example, if perm = [1,3,2], then encoded = [2,1].
 *
 * Given the encoded array, return the original array perm. It is guaranteed that the answer
 * exists and is unique.
 */

/**
 * @param {number[]} encoded
 * @return {number[]}
 */
var decode = function(encoded) {
  const n = encoded.length + 1;
  let totalXor = 0;
  for (let i = 1; i <= n; i++) {
    totalXor ^= i;
  }

  let oddXor = 0;
  for (let i = 1; i < encoded.length; i += 2) {
    oddXor ^= encoded[i];
  }

  const result = new Array(n);
  result[0] = totalXor ^ oddXor;

  for (let i = 0; i < n - 1; i++) {
    result[i + 1] = result[i] ^ encoded[i];
  }

  return result;
};
```

1735 Count Ways To Make Array With Product

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1735. Count Ways to Make Array With Product
 * https://leetcode.com/problems/count-ways-to-make-array-with-product/
 * Difficulty: Hard
 *
 * You are given a 2D integer array, queries. For each queries[i], where queries[i] = [ni,
ki],
 * find the number of different ways you can place positive integers into an array of size ni
 * such that the product of the integers is ki. As the number of ways may be too large, the
 * answer to the ith query is the number of ways modulo 109 + 7.
```

```
 *
 * Return an integer array answer where answer.length == queries.length, and answer[i] is the
 * answer to the ith query.
 */

/**
 * @param {number[][]} queries
 * @return {number[]}
 */
var waysToFillArray = function(queries) {
  const MOD = 1000000007n;
  const MAX_N = 20000;
  const factorial = new Array(MAX_N + 1).fill(1n);
  const inverse = new Array(MAX_N + 1).fill(1n);

  for (let i = 1; i <= MAX_N; i++) {
    factorial[i] = (factorial[i - 1] * BigInt(i)) % MOD;
  }
  inverse[MAX_N] = BigInt(modInverse(Number(factorial[MAX_N] % MOD), Number(MOD)));
  for (let i = MAX_N - 1; i >= 0; i--) {
    inverse[i] = (inverse[i + 1] * BigInt(i + 1)) % MOD;
  }

  const result = [];
  for (const [size, product] of queries) {
    if (product === 1) {
      result.push(1);
      continue;
    }
    const factors = getPrimeFactors(product);
    let ways = 1n;
    for (const count of factors.values()) {
      const n = size + count - 1;
      ways = (ways * BigInt(combinations(n, count))) % MOD;
    }
    result.push(Number(ways));
  }

  return result;

  function modInverse(a, m) {
    const m0 = m;
    let t;
    let q;
    let x0 = 0;
    let x1 = 1;
    while (a > 1) {
      q = Math.floor(a / m);
      t = m;
      m = a % m;
      a = t;
      t = x0;
      x0 = x1 - q * x0;
      x1 = t;
    }
    return x1 < 0 ? x1 + m0 : x1;
  }

  function combinations(n, k) {
    if (k < 0 || k > n || n < 0 || n > MAX_N || n - k < 0) return 0;
    const result = (factorial[n] * inverse[k] * inverse[n - k]) % MOD;
    return Number(result);
  }

  function getPrimeFactors(num) {
    const factors = new Map();
    for (let i = 2; i * i <= num; i++) {
      while (num % i === 0) {
        factors.set(i, (factors.get(i) || 0) + 1);
        num /= i;
      }
    }
    if (num > 1) {
      factors.set(num, (factors.get(num) || 0) + 1);
    }
```

```
    return factors;
  }
};
```

1736 Latest Time By Replacing Hidden Digits

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1736. Latest Time by Replacing Hidden Digits
 * https://leetcode.com/problems/latest-time-by-replacing-hidden-digits/
 * Difficulty: Easy
 *
 * You are given a string time in the form of  hh:mm, where some of the digits in the string
 * are hidden (represented by ?).
 *
 * The valid times are those inclusively between 00:00 and 23:59.
 *
 * Return the latest valid time you can get from time by replacing the hidden digits.
 */

/**
 * @param {string} time
 * @return {string}
 */
var maximumTime = function(time) {
  const result = time.split('');

  if (result[0] === '?') {
    result[0] = result[1] === '?' || result[1] <= '3' ? '2' : '1';
  }

  if (result[1] === '?') {
    result[1] = result[0] === '2' ? '3' : '9';
  }

  if (result[3] === '?') {
    result[3] = '5';
  }

  if (result[4] === '?') {
    result[4] = '9';
  }

  return result.join('');
};
```

1737 Change Minimum Characters To Satisfy One Of Three Conditions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1737. Change Minimum Characters to Satisfy One of Three Conditions
 * https://leetcode.com/problems/change-minimum-characters-to-satisfy-one-of-three-conditions/
 * Difficulty: Medium
 *
 * You are given two strings a and b that consist of lowercase letters. In one operation, you
can
 * change any character in a or b to any lowercase letter.
 *
 * Your goal is to satisfy one of the following three conditions:
 * - Every letter in a is strictly less than every letter in b in the alphabet.
 * - Every letter in b is strictly less than every letter in a in the alphabet.
 * - Both a and b consist of only one distinct letter.
 *
```

```
 * Return the minimum number of operations needed to achieve your goal.
 */

/**
 * @param {string} a
 * @param {string} b
 * @return {number}
 */
var minCharacters = function(a, b) {
  const aFreq = getFrequency(a);
  const bFreq = getFrequency(b);

  const condition1 = operationsForCondition1(aFreq, bFreq);
  const condition2 = operationsForCondition1(bFreq, aFreq);
  const condition3 = operationsForCondition3(aFreq, bFreq);

  return Math.min(condition1, condition2, condition3);

  function getFrequency(str) {
    const freq = Array(26).fill(0);
    for (const char of str) {
      freq[char.charCodeAt(0) - 97]++;
    }
    return freq;
  }

  function operationsForCondition1(aFreq, bFreq) {
    let minOps = Infinity;
    for (let i = 0; i < 25; i++) {
      let ops = 0;
      for (let j = 0; j <= i; j++) ops += aFreq[j];
      for (let j = i + 1; j < 26; j++) ops += bFreq[j];
      minOps = Math.min(minOps, ops);
    }
    return minOps;
  }

  function operationsForCondition3(aFreq, bFreq) {
    let minOps = Infinity;
    for (let i = 0; i < 26; i++) {
      const ops = a.length + b.length - aFreq[i] - bFreq[i];
      minOps = Math.min(minOps, ops);
    }
    return minOps;
  }
};
```

1738 Find Kth Largest Xor Coordinate Value

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1738. Find Kth Largest XOR Coordinate Value
 * https://leetcode.com/problems/find-kth-largest-xor-coordinate-value/
 * Difficulty: Medium
 *
 * You are given a 2D matrix of size m x n, consisting of non-negative integers. You are also
 * given an integer k.
 *
 * The value of coordinate (a, b) of the matrix is the XOR of all matrix[i][j] where
 * 0 <= i <= a < m and 0 <= j <= b < n (0-indexed).
 *
 * Find the kth largest value (1-indexed) of all the coordinates of matrix.
 */

/**
 * @param {number[][]} matrix
 * @param {number} k
 * @return {number}
 */
```

```
var kthLargestValue = function(matrix, k) {
  const rows = matrix.length;
  const cols = matrix[0].length;
  const xorValues = new Array(rows * cols);
  const prefixXor = Array.from({ length: rows + 1 }, () => Array(cols + 1).fill(0));

  let index = 0;
  for (let i = 1; i <= rows; i++) {
    for (let j = 1; j <= cols; j++) {
      prefixXor[i][j] = prefixXor[i-1][j] ^ prefixXor[i][j-1]
        ^ prefixXor[i-1][j-1] ^ matrix[i-1][j-1];
      xorValues[index++] = prefixXor[i][j];
    }
  }

  xorValues.sort((a, b) => b - a);
  return xorValues[k-1];
};
```

1739 Building Boxes

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1739. Building Boxes
 * https://leetcode.com/problems/building-boxes/
 * Difficulty: Hard
 *
 * You have a cubic storeroom where the width, length, and height of the room are all equal to
 n
 * units. You are asked to place n boxes in this room where each box is a cube of unit side
 * length. There are however some rules to placing the boxes:
 * - You can place the boxes anywhere on the floor.
 * - If box x is placed on top of the box y, then each side of the four vertical sides of the
box
 *   y must either be adjacent to another box or to a wall.
 *
 * Given an integer n, return the minimum possible number of boxes touching the floor.
 */

/**
 * @param {number} n
 * @return {number}
 */
var minimumBoxes = function(n) {
  if (n < 4) return n;

  let baseSize = 0;
  let totalBoxes = 0;
  const triangular = k => k * (k + 1) / 2;

  while (totalBoxes + triangular(baseSize + 1) <= n) {
    baseSize++;
    totalBoxes += triangular(baseSize);
  }

  const remaining = n - totalBoxes;
  let extraFloor = 0;

  while (triangular(extraFloor) < remaining) extraFloor++;

  return triangular(baseSize) + extraFloor;
};
```

1740 Find Distance In A Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1740. Find Distance in a Binary Tree
 * https://leetcode.com/problems/find-distance-in-a-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree and two integers p and q, return the distance between the
 * nodes of value p and value q in the tree.
 *
 * The distance between two nodes is the number of edges on the path from one to the other.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} p
 * @param {number} q
 * @return {number}
 */
var findDistance = function(root, p, q) {
  if (p === q) return 0;

  const lca = findLCA(root);
  const depthP = findDepth(lca, p, 0);
  const depthQ = findDepth(lca, q, 0);

  return depthP + depthQ;

  function findLCA(node) {
    if (!node || node.val === p || node.val === q) return node;

    const leftResult = findLCA(node.left);
    const rightResult = findLCA(node.right);

    if (leftResult && rightResult) return node;
    return leftResult || rightResult;
  }

  function findDepth(node, target, depth) {
    if (!node) return -1;
    if (node.val === target) return depth;

    const leftDepth = findDepth(node.left, target, depth + 1);
    if (leftDepth !== -1) return leftDepth;

    return findDepth(node.right, target, depth + 1);
  }
};
```

1742 Maximum Number Of Balls In A Box

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1742. Maximum Number of Balls in a Box
 * https://leetcode.com/problems/maximum-number-of-balls-in-a-box/
 * Difficulty: Easy
 *
 * You are working in a ball factory where you have n balls numbered from lowLimit up to
 * highLimit inclusive (i.e., n == highLimit - lowLimit + 1), and an infinite number of
 * boxes numbered from 1 to infinity.
 *
 * Your job at this factory is to put each ball in the box with a number equal to the sum
 * of digits of the ball's number. For example, the ball number 321 will be put in the
 * box number 3 + 2 + 1 = 6 and the ball number 10 will be put in the box number 1 + 0 = 1.
 *
 * Given two integers lowLimit and highLimit, return the number of balls in the box with
 * the most balls.
 */

/**
 * @param {number} lowLimit
 * @param {number} highLimit
 * @return {number}
 */
var countBalls = function(lowLimit, highLimit) {
  const boxCounts = new Map();
  let result = 0;

  for (let ball = lowLimit; ball <= highLimit; ball++) {
    const box = sumDigits(ball);
    const count = (boxCounts.get(box) || 0) + 1;
    boxCounts.set(box, count);
    result = Math.max(result, count);
  }

  return result;

  function sumDigits(num) {
    let sum = 0;
    while (num > 0) {
      sum += num % 10;
      num = Math.floor(num / 10);
    }
    return sum;
  }
};
```

1743 Restore The Array From Adjacent Pairs

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1743. Restore the Array From Adjacent Pairs
 * https://leetcode.com/problems/restore-the-array-from-adjacent-pairs/
 * Difficulty: Medium
 *
 * There is an integer array nums that consists of n unique elements, but you have
 * forgotten it. However, you do remember every pair of adjacent elements in nums.
 *
 * You are given a 2D integer array adjacentPairs of size n - 1 where each
 * adjacentPairs[i] = [ui, vi] indicates that the elements ui and vi are adjacent in nums.
 *
 * It is guaranteed that every adjacent pair of elements nums[i] and nums[i+1] will exist in
 * adjacentPairs, either as [nums[i], nums[i+1]] or [nums[i+1], nums[i]]. The pairs can appear
 * in any order.
 *
 * Return the original array nums. If there are multiple solutions, return any of them.
 */

/**
 * @param {number[][]} adjacentPairs
```

```
 * @return {number[]}
 */
var restoreArray = function(adjacentPairs) {
  const graph = new Map();
  for (const [u, v] of adjacentPairs) {
    graph.set(u, (graph.get(u) || []).concat(v));
    graph.set(v, (graph.get(v) || []).concat(u));
  }

  let start;
  for (const [node, neighbors] of graph) {
    if (neighbors.length === 1) {
      start = node;
      break;
    }
  }

  const result = [start];
  let prev = start;
  let curr = graph.get(start)[0];

  while (graph.get(curr).length > 1) {
    result.push(curr);
    const next = graph.get(curr).find(n => n !== prev);
    prev = curr;
    curr = next;
  }
  result.push(curr);

  return result;
};
```

1744 Can You Eat Your Favorite Candy On Your Favorite Day

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1744. Can You Eat Your Favorite Candy on Your Favorite Day?
 * https://leetcode.com/problems/can-you-eat-your-favorite-candy-on-your-favorite-day/
 * Difficulty: Medium
 *
 * You are given a (0-indexed) array of positive integers candiesCount where candiesCount[i]
 * represents the number of candies of the ith type you have. You are also given a 2D array
 * queries where queries[i] = [favoriteTypei, favoriteDayi, dailyCapi].
 *
 * You play a game with the following rules:
 * - You start eating candies on day 0.
 * - You cannot eat any candy of type i unless you have eaten all candies of type i - 1.
 * - You must eat at least one candy per day until you have eaten all the candies.
 *
 * Construct a boolean array answer such that answer.length == queries.length and answer[i] is
 * true if you can eat a candy of type favoriteTypei on day favoriteDayi without eating more
 * than dailyCapi candies on any day, and false otherwise. Note that you can eat different
 * types of candy on the same day, provided that you follow rule 2.
 *
 * Return the constructed array answer.
 */

/**
 * @param {number[]} candiesCount
 * @param {number[][]} queries
 * @return {boolean[]}
 */
var canEat = function(candiesCount, queries) {
  const prefixSums = [0];
  for (const count of candiesCount) {
    prefixSums.push(prefixSums.at(-1) + count);
  }

  const result = new Array(queries.length);
```

```
  for (let i = 0; i < queries.length; i++) {
    const [type, day, cap] = queries[i];
    const minCandies = day;
    const maxCandies = (day + 1) * cap;
    const typeStart = prefixSums[type];
    const typeEnd = prefixSums[type + 1] - 1;
    result[i] = maxCandies > typeStart && minCandies <= typeEnd;
  }

  return result;
};
```

1745 Palindrome Partitioning Iv

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1745. Palindrome Partitioning IV
 * https://leetcode.com/problems/palindrome-partitioning-iv/
 * Difficulty: Hard
 *
 * Given a string s, return true if it is possible to split the string s into three non-empty
 * palindromic substrings. Otherwise, return false.
 *
 * A string is said to be palindrome if it the same string when reversed.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var checkPartitioning = function(s) {
  const n = s.length;
  const isPalindrome = Array.from({ length: n }, () => Array(n).fill(false));

  for (let i = n - 1; i >= 0; i--) {
    for (let j = i; j < n; j++) {
      if (s[i] === s[j] && (j - i <= 2 || isPalindrome[i + 1][j - 1])) {
        isPalindrome[i][j] = true;
      }
    }
  }

  for (let i = 1; i < n - 1; i++) {
    for (let j = i; j < n - 1; j++) {
      if (isPalindrome[0][i - 1] && isPalindrome[i][j] && isPalindrome[j + 1][n - 1]) {
        return true;
      }
    }
  }

  return false;
};
```

1746 Maximum Subarray Sum After One Operation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1746. Maximum Subarray Sum After One Operation
 * https://leetcode.com/problems/maximum-subarray-sum-after-one-operation/
 * Difficulty: Medium
 *
 * You are given an integer array nums. You must perform exactly one operation where you can
 * replace one element nums[i] with nums[i] * nums[i].
```

```
 *
 * Return the maximum possible subarray sum after exactly one operation. The subarray must
 * be non-empty.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSumAfterOperation = function(nums) {
  const n = nums.length;
  let maxWithoutSquare = nums[0];
  let maxWithSquare = nums[0] * nums[0];
  let result = maxWithSquare;

  for (let i = 1; i < n; i++) {
    const currentValue = nums[i];
    const squaredValue = currentValue * currentValue;

    const newMaxWithSquare = Math.max(
      squaredValue,
      maxWithoutSquare + squaredValue,
      maxWithSquare + currentValue
    );

    const newMaxWithoutSquare = Math.max(
      currentValue,
      maxWithoutSquare + currentValue
    );

    maxWithSquare = newMaxWithSquare;
    maxWithoutSquare = newMaxWithoutSquare;
    result = Math.max(result, maxWithSquare);
  }

  return result;
};
```

1748 Sum Of Unique Elements

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 1748. Sum of Unique Elements
 * https://leetcode.com/problems/sum-of-unique-elements/
 * Difficulty: Easy
 *
 * You are given an integer array nums. The unique elements of an array are the
 * elements that appear exactly once in the array.
 *
 * Return the sum of all the unique elements of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumOfUnique = function(nums) {
  const map = new Map();
  nums.forEach(n => map.set(n, (map.get(n) || 0) + 1));

  return [...map].reduce((sum, [key, value]) => {
    return sum + (value === 1 ? key : 0);
  }, 0);
};
```

1749 Maximum Absolute Sum Of Any Subarray

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1749. Maximum Absolute Sum of Any Subarray
 * https://leetcode.com/problems/maximum-absolute-sum-of-any-subarray/
 * Difficulty: Medium
 *
 * You are given an integer array nums. The absolute sum of a subarray
 * [numsl, numsl+1, ..., numsr-1, numsr] is abs(numsl + numsl+1 + ... + numsr-1 + numsr).
 *
 * Return the maximum absolute sum of any (possibly empty) subarray of nums.
 *
 * Note that abs(x) is defined as follows:
 * - If x is a negative integer, then abs(x) = -x.
 * - If x is a non-negative integer, then abs(x) = x.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxAbsoluteSum = function(nums) {
  let result = 0;

  for (let i = 0, min = 0, max = 0; i < nums.length; i++) {
    max = Math.max(nums[i], max + nums[i]);
    min = Math.min(nums[i], min + nums[i]);
    result = Math.max(result, Math.abs(max), Math.abs(min));
  }

  return result;
};
```

1750 Minimum Length Of String After Deleting Similar Ends

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1750. Minimum Length of String After Deleting Similar Ends
 * https://leetcode.com/problems/minimum-length-of-string-after-deleting-similar-ends/
 * Difficulty: Medium
 *
 * Given a string s consisting only of characters 'a', 'b', and 'c'. You are asked to apply
the
 * following algorithm on the string any number of times:
 * 1. Pick a non-empty prefix from the string s where all the characters in the prefix are
equal.
 * 2. Pick a non-empty suffix from the string s where all the characters in this suffix are
equal.
 * 3. The prefix and the suffix should not intersect at any index.
 * 4. The characters from the prefix and suffix must be the same.
 * 5. Delete both the prefix and the suffix.
 *
 * Return the minimum length of s after performing the above operation any number of times
 * (possibly zero times).
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumLength = function(s) {
  let left = 0;
```

```
  let right = s.length - 1;

  while (left < right && s[left] === s[right]) {
    const char = s[left];
    while (left <= right && s[left] === char) left++;
    while (left <= right && s[right] === char) right--;
  }

  return Math.max(0, right - left + 1);
};
```

1751 Maximum Number Of Events That Can Be Attended Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1751. Maximum Number of Events That Can Be Attended II
 * https://leetcode.com/problems/maximum-number-of-events-that-can-be-attended-ii/
 * Difficulty: Hard
 *
 * You are given an array of events where events[i] = [startDayi, endDayi, valuei]. The ith
event
 * starts at startDayi and ends at endDayi, and if you attend this event, you will receive a
value
 * of valuei. You are also given an integer k which represents the maximum number of events
you can
 * attend.
 *
 * You can only attend one event at a time. If you choose to attend an event, you must attend
the
 * entire event. Note that the end day is inclusive: that is, you cannot attend two events
where
 * one of them starts and the other ends on the same day.
 *
 * Return the maximum sum of values that you can receive by attending events.
 */

/**
 * @param {number[][]} events
 * @param {number} k
 * @return {number}
 */
var maxValue = function(events, k) {
  events.sort((a, b) => a[0] - b[0]);
  const n = events.length;
  const dp = Array.from({ length: k + 1 }, () => new Array(n + 1).fill(-1));

  return maximize(0, k);

  function findNext(index, end) {
    let left = index;
    let right = n;
    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (events[mid][0] > end) {
        right = mid;
      } else {
        left = mid + 1;
      }
    }
    return left;
  }

  function maximize(index, remaining) {
    if (index >= n || remaining === 0) return 0;
    if (dp[remaining][index] !== -1) return dp[remaining][index];

    const nextIndex = findNext(index + 1, events[index][1]);
    const take = events[index][2] + maximize(nextIndex, remaining - 1);
    const skip = maximize(index + 1, remaining);
```

```
    return dp[remaining][index] = Math.max(take, skip);
  }
};
```

1752 Check If Array Is Sorted And Rotated

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1752. Check if Array Is Sorted and Rotated
 * https://leetcode.com/problems/check-if-array-is-sorted-and-rotated/
 * Difficulty: Easy
 *
 * Given an array nums, return true if the array was originally sorted in non-decreasing
 * order, then rotated some number of positions (including zero). Otherwise, return false.
 *
 * There may be duplicates in the original array.
 *
 * Note: An array A rotated by x positions results in an array B of the same length such
 * that A[i] == B[(i+x) % A.length], where % is the modulo operation.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var check = function(nums) {
  let count = 0;

  for (let i = 0; i < nums.length - 1; i++) {
    if (nums[i] > nums[i + 1]) {
      count++;
    }
  }

  if (nums[nums.length - 1] > nums[0]) {
    count++;
  }

  return count < 2 ? true : false;
};
```

1753 Maximum Score From Removing Stones

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1753. Maximum Score From Removing Stones
 * https://leetcode.com/problems/maximum-score-from-removing-stones/
 * Difficulty: Medium
 *
 * You are playing a solitaire game with three piles of stones of sizes a, b, and c
respectively.
 * Each turn you choose two different non-empty piles, take one stone from each, and add 1
point
 * to your score. The game stops when there are fewer than two non-empty piles (meaning there
are
 * no more available moves).
 *
 * Given three integers a, b, and c return the maximum score you can get.
 */

/**
 * @param {number} a
```

```
 * @param {number} b
 * @param {number} c
 * @return {number}
 */
var maximumScore = function(a, b, c) {
  const piles = [a, b, c].sort((x, y) => y - x);
  let score = 0;

  while (piles[0] > 0 && piles[1] > 0) {
    piles[0]--;
    piles[1]--;
    score++;
    piles.sort((x, y) => y - x);
  }

  return score;
};
```

1754 Largest Merge Of Two Strings

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1754. Largest Merge Of Two Strings
 * https://leetcode.com/problems/largest-merge-of-two-strings/
 * Difficulty: Medium
 *
 * You are given two strings word1 and word2. You want to construct a string merge in the
 * following way: while either word1 or word2 are non-empty, choose one of the following
 * options:
 * - If word1 is non-empty, append the first character in word1 to merge and delete it from
word1.
 *   - For example, if word1 = "abc" and merge = "dv", then after choosing this operation,
 *     word1 = "bc" and merge = "dva".
 * - If word2 is non-empty, append the first character in word2 to merge and delete it from
word2.
 *   - For example, if word2 = "abc" and merge = "", then after choosing this operation,
 *     word2 = "bc" and merge = "a".
 *
 * Return the lexicographically largest merge you can construct.
 *
 * A string a is lexicographically larger than a string b (of the same length) if in the first
 * position where a and b differ, a has a character strictly larger than the corresponding
 * character in b. For example, "abcd" is lexicographically larger than "abcc" because the
 * first position they differ is at the fourth character, and d is greater than c.
 */

/**
 * @param {string} word1
 * @param {string} word2
 * @return {string}
 */
var largestMerge = function(word1, word2) {
  let merge = '';
  let i = 0;
  let j = 0;

  while (i < word1.length && j < word2.length) {
    if (word1.slice(i) >= word2.slice(j)) {
      merge += word1[i++];
    } else {
      merge += word2[j++];
    }
  }

  merge += word1.slice(i) + word2.slice(j);
  return merge;
};
```

1755 Closest Subsequence Sum

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1755. Closest Subsequence Sum
 * https://leetcode.com/problems/closest-subsequence-sum/
 * Difficulty: Hard
 *
 * You are given an integer array nums and an integer goal.
 *
 * You want to choose a subsequence of nums such that the sum of its elements is the closest
 * possible to goal. That is, if the sum of the subsequence's elements is sum, then you want
 * to minimize the absolute difference abs(sum - goal).
 *
 * Return the minimum possible value of abs(sum - goal).
 *
 * Note that a subsequence of an array is an array formed by removing some elements (possibly
 * all or none) of the original array.
 */

/**
 * @param {number[]} nums
 * @param {number} goal
 * @return {number}
 */
var minAbsDifference = function(nums, goal) {
  const n = nums.length;
  const half = Math.floor(n / 2);
  const leftSums = new Set();
  const rightSums = new Set();

  generateSums(0, half, leftSums);
  generateSums(half, n, rightSums);

  const rightArray = [...rightSums].sort((a, b) => a - b);
  let minDiff = Infinity;

  for (const leftSum of leftSums) {
    const target = goal - leftSum;
    let left = 0;
    let right = rightArray.length - 1;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      const sum = leftSum + rightArray[mid];
      minDiff = Math.min(minDiff, Math.abs(sum - goal));

      if (sum < goal) {
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }
  }

  return minDiff;

  function generateSums(start, end, sums, current = 0) {
    if (start === end) {
      sums.add(current);
      return;
    }
    generateSums(start + 1, end, sums, current);
    generateSums(start + 1, end, sums, current + nums[start]);
  }
};
```

1756 Design Most Recently Used Queue

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1756. Design Most Recently Used Queue
 * https://leetcode.com/problems/design-most-recently-used-queue/
 * Difficulty: Medium
 *
 * Design a queue-like data structure that moves the most recently used element to the end
 * of the queue.
 *
 * Implement the MRUQueue class:
 * - MRUQueue(int n) constructs the MRUQueue with n elements: [1,2,3,...,n].
 * - int fetch(int k) moves the kth element (1-indexed) to the end of the queue and returns
it.
 */

/**
 * @param {number} n
 */
var MRUQueue = function(n) {
  this.elements = Array.from({ length: n }, (_, i) => i + 1);
};

/**
 * @param {number} k
 * @return {number}
 */
MRUQueue.prototype.fetch = function(k) {
  const element = this.elements[k - 1];
  this.elements.splice(k - 1, 1);
  this.elements.push(element);
  return element;
};
```

1758 Minimum Changes To Make Alternating Binary String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1758. Minimum Changes To Make Alternating Binary String
 * https://leetcode.com/problems/minimum-changes-to-make-alternating-binary-string/
 * Difficulty: Easy
 *
 * You are given a string s consisting only of the characters '0' and '1'. In one operation,
you can
 * change any '0' to '1' or vice versa.
 *
 * The string is called alternating if no two adjacent characters are equal. For example, the
string
 * "010" is alternating, while the string "0100" is not.
 *
 * Return the minimum number of operations needed to make s alternating.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minOperations = function(s) {
  let changesToZeroStart = 0;
  let changesToOneStart = 0;

  for (let i = 0; i < s.length; i++) {
```

```
    const expectedZeroStart = i % 2 === 0 ? '0' : '1';
    const expectedOneStart = i % 2 === 0 ? '1' : '0';
    if (s[i] !== expectedZeroStart) changesToZeroStart++;
    if (s[i] !== expectedOneStart) changesToOneStart++;
  }

  return Math.min(changesToZeroStart, changesToOneStart);
};
```

1759 Count Number Of Homogenous Substrings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1759. Count Number of Homogenous Substrings
 * https://leetcode.com/problems/count-number-of-homogenous-substrings/
 * Difficulty: Medium
 *
 * Given a string s, return the number of homogenous substrings of s. Since the answer may be
 * too large, return it modulo 109 + 7.
 *
 * A string is homogenous if all the characters of the string are the same.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countHomogenous = function(s) {
  const mod = 1e9 + 7;
  let result = 0;
  let streak = 1;

  for (let i = 1; i < s.length; i++) {
    if (s[i] === s[i - 1]) {
      streak++;
    } else {
      result = (result + (streak * (streak + 1) / 2)) % mod;
      streak = 1;
    }
  }

  result = (result + (streak * (streak + 1) / 2)) % mod;

  return result;
};
```

1760 Minimum Limit Of Balls In A Bag

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1760. Minimum Limit of Balls in a Bag
 * https://leetcode.com/problems/minimum-limit-of-balls-in-a-bag/
 * Difficulty: Medium
 *
 * You are given an integer array nums where the ith bag contains nums[i] balls. You are also
 * given an integer maxOperations.
 *
 * You can perform the following operation at most maxOperations times:
 * - Take any bag of balls and divide it into two new bags with a positive number of balls.
 *   - For example, a bag of 5 balls can become two new bags of 1 and 4 balls, or two new bags
 *     of 2 and 3 balls.
```

```
 *
 * Your penalty is the maximum number of balls in a bag. You want to minimize your penalty
after
 * the operations.
 *
 * Return the minimum possible penalty after performing the operations.
 */

/**
 * @param {number[]} nums
 * @param {number} maxOperations
 * @return {number}
 */
var minimumSize = function(nums, maxOperations) {
  let left = 1;
  let right = Math.max(...nums);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    let operations = 0;

    for (const num of nums) {
      operations += Math.ceil(num / mid) - 1;
    }

    if (operations <= maxOperations) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;
};
```

1761 Minimum Degree Of A Connected Trio In A Graph

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1761. Minimum Degree of a Connected Trio in a Graph
 * https://leetcode.com/problems/minimum-degree-of-a-connected-trio-in-a-graph/
 * Difficulty: Hard
 *
 * You are given an undirected graph. You are given an integer n which is the number of nodes
in
 * the graph and an array edges, where each edges[i] = [ui, vi] indicates that there is an
 * undirected edge between ui and vi.
 *
 * A connected trio is a set of three nodes where there is an edge between every pair of them.
 *
 * The degree of a connected trio is the number of edges where one endpoint is in the trio,
and
 * the other is not.
 *
 * Return the minimum degree of a connected trio in the graph, or -1 if the graph has no
 * connected trios.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var minTrioDegree = function(n, edges) {
  const graph = Array.from({ length: n + 1 }, () => new Set());
  const degrees = Array(n + 1).fill(0);

  for (const [u, v] of edges) {
    graph[u].add(v);
```

```
    graph[v].add(u);
    degrees[u]++;
    degrees[v]++;
  }

  let minDegree = Infinity;

  for (let i = 1; i <= n; i++) {
    for (const j of graph[i]) {
      for (const k of graph[j]) {
        if (graph[k].has(i)) {
          const trioDegree = degrees[i] + degrees[j] + degrees[k] - 6;
          minDegree = Math.min(minDegree, trioDegree);
        }
      }
    }
  }

  return minDegree === Infinity ? -1 : minDegree;
};
```

1762 Buildings With An Ocean View

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1762. Buildings With an Ocean View
 * https://leetcode.com/problems/buildings-with-an-ocean-view/
 * Difficulty: Medium
 *
 * There are n buildings in a line. You are given an integer array heights of size n that
 * represents the heights of the buildings in the line.
 *
 * The ocean is to the right of the buildings. A building has an ocean view if the building
 * can see the ocean without obstructions. Formally, a building has an ocean view if all the
 * buildings to its right have a smaller height.
 *
 * Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in
 * increasing order.
 */

/**
 * @param {number[]} heights
 * @return {number[]}
 */
var findBuildings = function(heights) {
  const result = [];
  let maxHeightFromRight = 0;

  for (let i = heights.length - 1; i >= 0; i--) {
    if (heights[i] > maxHeightFromRight) {
      result.push(i);
      maxHeightFromRight = heights[i];
    }
  }

  return result.reverse();
};
```

1763 Longest Nice Substring

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1763. Longest Nice Substring
```

```
 * https://leetcode.com/problems/longest-nice-substring/
 * Difficulty: Easy
 *
 * A string s is nice if, for every letter of the alphabet that s contains, it appears both
 * in uppercase and lowercase. For example, "abABB" is nice because 'A' and 'a' appear,
 * and 'B' and 'b' appear. However, "abA" is not because 'b' appears, but 'B' does not.
 *
 * Given a string s, return the longest substring of s that is nice. If there are multiple,
 * return the substring of the earliest occurrence. If there are none, return an empty string.
 */

/**
 * @param {string} s
 * @return {string}
 */
var longestNiceSubstring = function(s) {
  let longest = '';

  for (let i = 0; i < s.length; i++) {
    for (let j = i; j < s.length; j++) {
      const substr = s.slice(i, j + 1);
      if (substr.length <= longest.length) continue;
      const chars = new Set(substr.toLowerCase().split(''));
      let isNice = true;
      for (const char of chars) {
        if (!substr.includes(char.toLowerCase()) || !substr.includes(char.toUpperCase())) {
          isNice = false;
          break;
        }
      }
      if (isNice) longest = substr;
    }
  }

  return longest;
};
```

1764 Form Array By Concatenating Subarrays Of Another Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1764. Form Array by Concatenating Subarrays of Another Array
 * https://leetcode.com/problems/form-array-by-concatenating-subarrays-of-another-array/
 * Difficulty: Medium
 *
 * You are given a 2D integer array groups of length n. You are also given an integer array
nums.
 *
 * You are asked if you can choose n disjoint subarrays from the array nums such that the ith
 * subarray is equal to groups[i] (0-indexed), and if i > 0, the (i-1)th subarray appears
before
 * the ith subarray in nums (i.e. the subarrays must be in the same order as groups).
 *
 * Return true if you can do this task, and false otherwise.
 *
 * Note that the subarrays are disjoint if and only if there is no index k such that nums[k]
 * belongs to more than one subarray. A subarray is a contiguous sequence of elements within
 * an array.
 */

/**
 * @param {number[][]} groups
 * @param {number[]} nums
 * @return {boolean}
 */
var canChoose = function(groups, nums) {
  const rows = groups.map(row => ',' + row.join(',') + ',');
  const joined = `,${nums.join(',')},`;
```

```
  for (let i = 0, offset = 0; i < rows.length; i++) {
    offset = joined.indexOf(rows[i], offset);
    if (offset === -1) {
      return false;
    }
    offset += rows[i].length - 1;
  }

  return true;
};
```

1765 Map Of Highest Peak

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1765. Map of Highest Peak
 * https://leetcode.com/problems/map-of-highest-peak/
 * Difficulty: Medium
 *
 * You are given an integer matrix isWater of size m x n that represents a map of land and
 * water cells:
 * - If isWater[i][j] == 0, cell (i, j) is a land cell.
 * - If isWater[i][j] == 1, cell (i, j) is a water cell.
 *
 * You must assign each cell a height in a way that follows these rules:
 * - The height of each cell must be non-negative.
 * - If the cell is a water cell, its height must be 0.
 * - Any two adjacent cells must have an absolute height difference of at most 1. A cell is
 * adjacent to another cell if the former is directly north, east, south, or west of the
 * latter (i.e., their sides are touching).
 *
 * Find an assignment of heights such that the maximum height in the matrix is maximized.
 *
 * Return an integer matrix height of size m x n where height[i][j] is cell (i, j)'s height.
 * If there are multiple solutions, return any of them.
 */

/**
 * @param {number[][]} isWater
 * @return {number[][]}
 */
var highestPeak = function(isWater) {
  const map = isWater.map(row => row.map(() => 0));
  const values = isWater.map((row, i) => {
    return row.map((value, j) => value ? [i, j] : 0);
  }).flat().filter(Boolean);

  for (let value = 0; values.length > value;) {
    const [i, j] = values[value++];
    const level = map[i][j] + 1;
    [[1, 0], [-1, 0], [0, -1], [0, 1]]
      .map(direction => [i + direction[0], j + direction[1]])
      .filter(([x, y]) => 0 === isWater[x]?.[y] && !map[x][y])
      .forEach(([x, y]) => (map[x][y] = level, values.push([x, y])));
  }

  return map;
};
```

1766 Tree Of Coprimes

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1766. Tree of Coprimes
 * https://leetcode.com/problems/tree-of-coprimes/
 * Difficulty: Hard
 *
 * There is a tree (i.e., a connected, undirected graph that has no cycles) consisting of n
nodes
 * numbered from 0 to n - 1 and exactly n - 1 edges. Each node has a value associated with it,
 * and the root of the tree is node 0.
 *
 * To represent this tree, you are given an integer array nums and a 2D array edges. Each
nums[i]
 * represents the ith node's value, and each edges[j] = [uj, vj] represents an edge between
nodes
 * uj and vj in the tree.
 *
 * Two values x and y are coprime if gcd(x, y) == 1 where gcd(x, y) is the greatest common
divisor
 * of x and y.
 *
 * An ancestor of a node i is any other node on the shortest path from node i to the root. A
node
 * is not considered an ancestor of itself.
 *
 * Return an array ans of size n, where ans[i] is the closest ancestor to node i such that
 * nums[i] and nums[ans[i]] are coprime, or -1 if there is no such ancestor.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} edges
 * @return {number[]}
 */
var getCoprimes = function(nums, edges) {
  const n = nums.length;
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  const result = Array(n).fill(-1);
  const valueDepth = Array(51).fill().map(() => []);
  const visited = new Set();

  dfs(0, 0, -1);
  return result;

  function dfs(node, depth, parent) {
    const val = nums[node];
    let maxDepth = -1;
    let closestAncestor = -1;

    for (let i = 1; i <= 50; i++) {
      if (gcd(val, i) === 1 && valueDepth[i].length) {
        const [ancestor, d] = valueDepth[i].at(-1);
        if (d > maxDepth) {
          maxDepth = d;
          closestAncestor = ancestor;
        }
      }
    }

    result[node] = closestAncestor;
    valueDepth[val].push([node, depth]);
    visited.add(node);

    for (const neighbor of graph[node]) {
      if (!visited.has(neighbor)) {
        dfs(neighbor, depth + 1, node);
      }
    }

    valueDepth[val].pop();
  }
```

```
  function gcd(a, b) {
    while (b) [a, b] = [b, a % b];
    return a;
  }
};
```

1768 Merge Strings Alternately

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1768. Merge Strings Alternately
 * https://leetcode.com/problems/merge-strings-alternately/
 * Difficulty: Easy
 *
 * You are given two strings word1 and word2. Merge the strings by adding letters in
alternating
 * order, starting with word1. If a string is longer than the other, append the additional
 * letters onto the end of the merged string.
 *
 * Return the merged string.
 */

/**
 * @param {string} word1
 * @param {string} word2
 * @return {string}
 */
var mergeAlternately = function(word1, word2) {
  let result = '';
  for (let i = 0; i < Math.max(word1.length, word2.length); i++) {
    if (i < word1.length) result += word1[i];
    if (i < word2.length) result += word2[i];
  }
  return result;
};
```

1769 Minimum Number Of Operations To Move All Balls To Each Box

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1769. Minimum Number of Operations to Move All Balls to Each Box
 * https://leetcode.com/problems/minimum-number-of-operations-to-move-all-balls-to-each-box/
 * Difficulty: Medium
 *
 * You have n boxes. You are given a binary string boxes of length n, where boxes[i] is '0' if
 * the ith box is empty, and '1' if it contains one ball.
 *
 * In one operation, you can move one ball from a box to an adjacent box. Box i is adjacent to
 * box j if abs(i - j) == 1. Note that after doing so, there may be more than one ball in some
 * boxes.
 *
 * Return an array answer of size n, where answer[i] is the minimum number of operations
needed
 * to move all the balls to the ith box.
 *
 * Each answer[i] is calculated considering the initial state of the boxes.
 */

/**
 * @param {string} boxes
 * @return {number[]}
 */
```

```
var minOperations = function(boxes) {
  const result = new Array(boxes.length).fill(0);

  for (let i = 0; i < boxes.length; i++) {
    let total = 0;
    for (let j = 0; j < boxes.length; j++) {
      if (boxes[j] === '1') {
        total += Math.abs(j - i);
      }
    }
    result[i] = total;
  }

  return result;
};
```

1770 Maximum Score From Performing Multiplication Operations

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1770. Maximum Score from Performing Multiplication Operations
 * https://leetcode.com/problems/maximum-score-from-performing-multiplication-operations/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays nums and multipliers of size n and m
respectively,
 * where n >= m.
 *
 * You begin with a score of 0. You want to perform exactly m operations. On the ith operation
 * (0-indexed) you will:
 * - Choose one integer x from either the start or the end of the array nums.
 * - Add multipliers[i] * x to your score.
 *   - Note that multipliers[0] corresponds to the first operation, multipliers[1] to the
second
 *     operation, and so on.
 * - Remove x from nums.
 *
 * Return the maximum score after performing m operations.
 */

/**
 * @param {number[]} nums
 * @param {number[]} multipliers
 * @return {number}
 */
var maximumScore = function(nums, multipliers) {
  const m = multipliers.length;
  const dp = Array.from({ length: m + 1 }, () => Array(m + 1).fill(0));

  for (let op = m - 1; op >= 0; op--) {
    for (let left = op; left >= 0; left--) {
      const right = nums.length - 1 - (op - left);
      const takeLeft = multipliers[op] * nums[left] + dp[op + 1][left + 1];
      const takeRight = multipliers[op] * nums[right] + dp[op + 1][left];
      dp[op][left] = Math.max(takeLeft, takeRight);
    }
  }

  return dp[0][0];
};
```

1771 Maximize Palindrome Length From Subsequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1771. Maximize Palindrome Length From Subsequences
 * https://leetcode.com/problems/maximize-palindrome-length-from-subsequences/
 * Difficulty: Hard
 *
 * You are given two strings, word1 and word2. You want to construct a string in the following
 * manner:
 * - Choose some non-empty subsequence subsequence1 from word1.
 * - Choose some non-empty subsequence subsequence2 from word2.
 * - Concatenate the subsequences: subsequence1 + subsequence2, to make the string.
 *
 * Return the length of the longest palindrome that can be constructed in the described
manner.
 * If no palindromes can be constructed, return 0.
 *
 * A subsequence of a string s is a string that can be made by deleting some (possibly none)
 * characters from s without changing the order of the remaining characters.
 *
 * A palindrome is a string that reads the same forward as well as backward.
 */

/**
 * @param {string} word1
 * @param {string} word2
 * @return {number}
 */
var longestPalindrome = function(word1, word2) {
  const s = word1 + word2;
  const n = s.length;
  const dp = Array.from({ length: n }, () => Array(n).fill(0));
  let result = 0;

  for (let i = n - 1; i >= 0; i--) {
    dp[i][i] = 1;
    for (let j = i + 1; j < n; j++) {
      if (s[i] === s[j]) {
        dp[i][j] = dp[i + 1][j - 1] + 2;
        if (i < word1.length && j >= word1.length) {
          result = Math.max(result, dp[i][j]);
        }
      } else {
        dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
      }
    }
  }

  return result;
};
```

1772 Sort Features By Popularity

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1772. Sort Features by Popularity
 * https://leetcode.com/problems/sort-features-by-popularity/
 * Difficulty: Medium
 *
 * You are given a string array features where features[i] is a single word that represents
 * the name of a feature of the latest product you are working on. You have made a survey
 * where users have reported which features they like. You are given a string array responses,
 * where each responses[i] is a string containing space-separated words.
 *
 * The popularity of a feature is the number of responses[i] that contain the feature. You
 * want to sort the features in non-increasing order by their popularity. If two features
 * have the same popularity, order them by their original index in features. Notice that
 * one response could contain the same feature multiple times; this feature is only counted
 * once in its popularity.
 *
```

```
 * Return the features in sorted order.
 */

/**
 * @param {string[]} features
 * @param {string[]} responses
 * @return {string[]}
 */
var sortFeatures = function(features, responses) {
  const map = new Map();

  for (const feature of features) {
    map.set(feature, 0);
  }

  for (const response of responses) {
    const uniqueWords = new Set(response.split(' '));
    for (const word of uniqueWords) {
      if (map.has(word)) {
        map.set(word, map.get(word) + 1);
      }
    }
  }

  return features
    .map((feature, index) => ({ feature, popularity: map.get(feature), index }))
    .sort((a, b) => {
      if (a.popularity !== b.popularity) {
        return b.popularity - a.popularity;
      }
      return a.index - b.index;
    })
    .map(item => item.feature);
};
```

1773 Count Items Matching A Rule

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1773. Count Items Matching a Rule
 * https://leetcode.com/problems/count-items-matching-a-rule/
 * Difficulty: Easy
 *
 * You are given an array items, where each items[i] = [typei, colori, namei] describes the
type,
 * color, and name of the ith item. You are also given a rule represented by two strings,
 * ruleKey and ruleValue.
 *
 * The ith item is said to match the rule if one of the following is true:
 * - ruleKey == "type" and ruleValue == typei.
 * - ruleKey == "color" and ruleValue == colori.
 * - ruleKey == "name" and ruleValue == namei.
 *
 * Return the number of items that match the given rule.
 */

/**
 * @param {string[][]} items
 * @param {string} ruleKey
 * @param {string} ruleValue
 * @return {number}
 */
var countMatches = function(items, ruleKey, ruleValue) {
  const keyIndex = { type: 0, color: 1, name: 2 };
  let result = 0;

  for (const item of items) {
    if (item[keyIndex[ruleKey]] === ruleValue) {
      result++;
```

```
    }
  }

  return result;
};
```

1774 Closest Dessert Cost

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1774. Closest Dessert Cost
 * https://leetcode.com/problems/closest-dessert-cost/
 * Difficulty: Medium
 *
 * You would like to make dessert and are preparing to buy the ingredients. You have n ice
 * cream base flavors and m types of toppings to choose from. You must follow these rules
 * when making your dessert:
 * - There must be exactly one ice cream base.
 * - You can add one or more types of topping or have no toppings at all.
 * - There are at most two of each type of topping.
 *
 * You are given three inputs:
 * - baseCosts, an integer array of length n, where each baseCosts[i] represents the price of
 *   the ith ice cream base flavor.
 * - toppingCosts, an integer array of length m, where each toppingCosts[i] is the price of
 *   one of the ith topping.
 * - target, an integer representing your target price for dessert.
 *
 * You want to make a dessert with a total cost as close to target as possible.
 *
 * Return the closest possible cost of the dessert to target. If there are multiple, return
 * the lower one.
 */

/**
 * @param {number[]} baseCosts
 * @param {number[]} toppingCosts
 * @param {number} target
 * @return {number}
 */
var closestCost = function(baseCosts, toppingCosts, target) {
  let result = Infinity;

  for (const base of baseCosts) {
    exploreToppings(0, base);
  }

  return result;

  function exploreToppings(index, currentCost) {
    const diff = Math.abs(currentCost - target);
    const prevDiff = Math.abs(result - target);

    if (diff < prevDiff || (diff === prevDiff && currentCost < result)) {
      result = currentCost;
    }

    if (index >= toppingCosts.length || currentCost > target) return;

    exploreToppings(index + 1, currentCost);
    exploreToppings(index + 1, currentCost + toppingCosts[index]);
    exploreToppings(index + 1, currentCost + 2 * toppingCosts[index]);
  }
};
```

1775 Equal Sum Arrays With Minimum Number Of Operations

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1775. Equal Sum Arrays With Minimum Number of Operations
 * https://leetcode.com/problems/equal-sum-arrays-with-minimum-number-of-operations/
 * Difficulty: Medium
 *
 * You are given two arrays of integers nums1 and nums2, possibly of different lengths. The
values
 * in the arrays are between 1 and 6, inclusive.
 *
 * In one operation, you can change any integer's value in any of the arrays to any value
between
 * 1 and 6, inclusive.
 *
 * Return the minimum number of operations required to make the sum of values in nums1 equal
to
 * the sum of values in nums2. Return -1 if it is not possible to make the sum of the two
arrays
 * equal.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minOperations = function(nums1, nums2) {
  let sum1 = nums1.reduce((a, b) => a + b, 0);
  let sum2 = nums2.reduce((a, b) => a + b, 0);

  if (sum1 < sum2) {
    [nums1, nums2, sum1, sum2] = [nums2, nums1, sum2, sum1];
  }

  const maxSum1 = nums1.length * 6;
  const minSum1 = nums1.length;
  const maxSum2 = nums2.length * 6;
  const minSum2 = nums2.length;

  if (maxSum1 < minSum2 || maxSum2 < minSum1) return -1;

  let result = 0;
  const gains = [];
  const losses = [];

  for (const num of nums1) {
    losses.push(num - 1);
  }
  for (const num of nums2) {
    gains.push(6 - num);
  }

  losses.sort((a, b) => b - a);
  gains.sort((a, b) => b - a);

  let i = 0;
  let j = 0;
  let diff = sum1 - sum2;

  while (diff > 0 && (i < losses.length || j < gains.length)) {
    const change = i < losses.length && (j >= gains.length || losses[i] >= gains[j])
      ? losses[i++]
      : gains[j++];
    const take = Math.min(diff, change);
    diff -= take;
    result++;
```

```
  }

  return result;
};
```

1776 Car Fleet Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1776. Car Fleet II
 * https://leetcode.com/problems/car-fleet-ii/
 * Difficulty: Hard
 *
 * There are n cars traveling at different speeds in the same direction along a one-lane road.
 * You are given an array cars of length n, where cars[i] = [positioni, speedi] represents:
 * - positioni is the distance between the ith car and the beginning of the road in meters.
 *   It is guaranteed that positioni < positioni+1.
 * - speedi is the initial speed of the ith car in meters per second.
 *
 * For simplicity, cars can be considered as points moving along the number line. Two cars
 * collide when they occupy the same position. Once a car collides with another car, they
 * unite and form a single car fleet. The cars in the formed fleet will have the same position
 * and the same speed, which is the initial speed of the slowest car in the fleet.
 *
 * Return an array answer, where answer[i] is the time, in seconds, at which the ith car
 * collides with the next car, or -1 if the car does not collide with the next car. Answers
 * within 10-5 of the actual answers are accepted.
 */

/**
 * @param {number[][]} cars
 * @return {number[]}
 */
var getCollisionTimes = function(cars) {
  const n = cars.length;
  const stack = [];
  const collisionTimes = Array(n).fill(-1);

  for (let i = n - 1; i >= 0; i--) {
    const [pos, speed] = cars[i];

    while (stack.length > 0) {
      const j = stack[stack.length - 1];
      const [nextPos, nextSpeed] = cars[j];

      if (speed <= nextSpeed || (collisionTimes[j] > 0
          && (nextPos - pos) / (speed - nextSpeed) >= collisionTimes[j])) {
        stack.pop();
      } else {
        break;
      }
    }

    if (stack.length > 0) {
      const j = stack[stack.length - 1];
      const [nextPos, nextSpeed] = cars[j];
      collisionTimes[i] = (nextPos - pos) / (speed - nextSpeed);
    }

    stack.push(i);
  }

  return collisionTimes;
};
```

1778 Shortest Path In A Hidden Grid

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1778. Shortest Path in a Hidden Grid
 * https://leetcode.com/problems/shortest-path-in-a-hidden-grid/
 * Difficulty: Medium
 *
 * This is an interactive problem.
 *
 * There is a robot in a hidden grid, and you are trying to get it from its starting cell to
the
 * target cell in this grid. The grid is of size m x n, and each cell in the grid is either
empty
 * or blocked. It is guaranteed that the starting cell and the target cell are different, and
 * neither of them is blocked.
 *
 * You want to find the minimum distance to the target cell. However, you do not know the
grid's
 * dimensions, the starting cell, nor the target cell. You are only allowed to ask queries to
the
 * GridMaster object.
 *
 * The GridMaster class has the following functions:
 * - boolean canMove(char direction) Returns true if the robot can move in that direction.
 *   Otherwise, it returns false.
 * - void move(char direction) Moves the robot in that direction. If this move would move the
 *   robot to a blocked cell or off the grid, the move will be ignored, and the robot will
 *   remain in the same position.
 * - boolean isTarget() Returns true if the robot is currently on the target cell. Otherwise,
 *   it returns false.
 *
 * Note that direction in the above functions should be a character from {'U','D','L','R'},
 * representing the directions up, down, left, and right, respectively.
 *
 * Return the minimum distance between the robot's initial starting cell and the target cell.
 * If there is no valid path between the cells, return -1.
 *
 * Custom testing:
 *
 * The test input is read as a 2D matrix grid of size m x n where:
 * - grid[i][j] == -1 indicates that the robot is in cell (i, j) (the starting cell).
 * - grid[i][j] == 0 indicates that the cell (i, j) is blocked.
 * - grid[i][j] == 1 indicates that the cell (i, j) is empty.
 * - grid[i][j] == 2 indicates that the cell (i, j) is the target cell.
 *
 * There is exactly one -1 and 2 in grid. Remember that you will not have this information
 * in your code.
 */

/**
 * // This is the GridMaster's API interface.
 * // You should not implement it, or speculate about its implementation
 * function GridMaster() {
 *
 *     @param {character} direction
 *     @return {boolean}
 *     this.canMove = function(direction) {
 *         ...
 *     };
 *     @param {character} direction
 *     @return {void}
 *     this.move = function(direction) {
 *         ...
 *     };
 *     @return {boolean}
 *     this.isTarget = function() {
 *         ...
```

```
 *     };
 * };
 */

/**
 * @param {GridMaster} master
 * @return {integer}
 */
var findShortestPath = function(master) {
  const directions = [
    ['U', 'D', 0, -1],
    ['D', 'U', 0, 1],
    ['L', 'R', -1, 0],
    ['R', 'L', 1, 0]
  ];

  const visited = new Set();
  let targetPosition = null;

  dfs(0, 0);

  if (!targetPosition) return -1;

  const queue = [[0, 0, 0]];
  const bfsVisited = new Set(['0,0']);

  while (queue.length > 0) {
    const [x, y, distance] = queue.shift();

    if (x === targetPosition[0] && y === targetPosition[1]) {
      return distance;
    }

    for (const [, , dx, dy] of directions) {
      const newX = x + dx;
      const newY = y + dy;
      const key = `${newX},${newY}`;

      if (!bfsVisited.has(key) && visited.has(key)) {
        bfsVisited.add(key);
        queue.push([newX, newY, distance + 1]);
      }
    }
  }

  return -1;

  function dfs(x, y) {
    const key = `${x},${y}`;
    if (visited.has(key)) return;
    visited.add(key);

    if (master.isTarget()) {
      targetPosition = [x, y];
      return;
    }

    for (const [moveDir, backDir, dx, dy] of directions) {
      if (master.canMove(moveDir)) {
        master.move(moveDir);
        dfs(x + dx, y + dy);
        master.move(backDir);
      }
    }
  }
};
```

1779 Find Nearest Point That Has The Same X Or Y Coordinate

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1779. Find Nearest Point That Has the Same X or Y Coordinate
 * https://leetcode.com/problems/find-nearest-point-that-has-the-same-x-or-y-coordinate/
 * Difficulty: Easy
 *
 * You are given two integers, x and y, which represent your current location on a Cartesian
 * grid: (x, y). You are also given an array points where each points[i] = [ai, bi] represents
 * that a point exists at (ai, bi). A point is valid if it shares the same x-coordinate or the
 * same y-coordinate as your location.
 *
 * Return the index (0-indexed) of the valid point with the smallest Manhattan distance from
 * your current location. If there are multiple, return the valid point with the smallest
index.
 * If there are no valid points, return -1.
 *
 * The Manhattan distance between two points (x1, y1) and (x2, y2) is abs(x1 - x2) + abs(y1 -
y2).
 */

/**
 * @param {number} x
 * @param {number} y
 * @param {number[][]} points
 * @return {number}
 */
var nearestValidPoint = function(x, y, points) {
  let minDistance = Infinity;
  let result = -1;

  for (let i = 0; i < points.length; i++) {
    const [px, py] = points[i];
    if (px === x || py === y) {
      const distance = Math.abs(px - x) + Math.abs(py - y);
      if (distance < minDistance) {
        minDistance = distance;
        result = i;
      }
    }
  }

  return result;
};
```

1780 Check If Number Is A Sum Of Powers Of Three

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1780. Check if Number is a Sum of Powers of Three
 * https://leetcode.com/problems/check-if-number-is-a-sum-of-powers-of-three/
 * Difficulty: Medium
 *
 * Given an integer n, return true if it is possible to represent n as the sum
 * of distinct powers of three. Otherwise, return false.
 *
 * An integer y is a power of three if there exists an integer x such that y == 3x.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var checkPowersOfThree = function(n) {
  while (n) {
    if (n % 3 === 2) {
      return 0;
    }
    n = n / 3 >> 0;
  }
```

```
  return 1;
};
```

1781 Sum Of Beauty Of All Substrings

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1781. Sum of Beauty of All Substrings
 * https://leetcode.com/problems/sum-of-beauty-of-all-substrings/
 * Difficulty: Medium
 *
 * The beauty of a string is the difference in frequencies between the most frequent and
 * least frequent characters.
 * - For example, the beauty of "abaacc" is 3 - 1 = 2.
 *
 * Given a string s, return the sum of beauty of all of its substrings.
 */

/**
 * @param {string} s
 * @return {number}
 */
var beautySum = function(s) {
  let result = 0;

  for (let i = 0; i < s.length; i++) {
    const freq = new Map();
    for (let j = i; j < s.length; j++) {
      freq.set(s[j], (freq.get(s[j]) || 0) + 1);
      let maxFreq = 0;
      let minFreq = Infinity;
      for (const count of freq.values()) {
        maxFreq = Math.max(maxFreq, count);
        minFreq = Math.min(minFreq, count);
      }
      result += maxFreq - minFreq;
    }
  }

  return result;
};
```

1782 Count Pairs Of Nodes

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1782. Count Pairs Of Nodes
 * https://leetcode.com/problems/count-pairs-of-nodes/
 * Difficulty: Hard
 *
 * You are given an undirected graph defined by an integer n, the number of nodes, and a 2D
 * integer array edges, the edges in the graph, where edges[i] = [ui, vi] indicates that there
 * is an undirected edge between ui and vi. You are also given an integer array queries.
 *
 * Let incident(a, b) be defined as the number of edges that are connected to either node a or
 b.
 *
 * The answer to the jth query is the number of pairs of nodes (a, b) that satisfy both of the
 * following conditions:
 * - a < b
 * - incident(a, b) > queries[j]
 *
```

```
 * Return an array answers such that answers.length == queries.length and answers[j] is the
 * answer of the jth query.
 *
 * Note that there can be multiple edges between the same two nodes.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[]} queries
 * @return {number[]}
 */
var countPairs = function(n, edges, queries) {
  const degrees = Array(n + 1).fill(0);
  const edgeCounts = new Map();

  for (const [u, v] of edges) {
    degrees[u]++;
    degrees[v]++;
    const key = u < v ? `${u},${v}` : `${v},${u}`;
    edgeCounts.set(key, (edgeCounts.get(key) || 0) + 1);
  }

  const sortedDegrees = [...degrees].slice(1).sort((a, b) => a - b);
  const result = Array(queries.length).fill(0);

  for (let q = 0; q < queries.length; q++) {
    let total = 0;
    let left = 0;
    let right = n - 1;

    while (left < right) {
      if (sortedDegrees[left] + sortedDegrees[right] > queries[q]) {
        total += right - left;
        right--;
      } else {
        left++;
      }
    }

    for (const [key, count] of edgeCounts) {
      const [u, v] = key.split(',').map(Number);
      if (degrees[u] + degrees[v] > queries[q] && degrees[u] + degrees[v] - count <=
queries[q]) {
        total--;
      }
    }

    result[q] = total;
  }

  return result;
};
```

1784 Check If Binary String Has At Most One Segment Of Ones

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1784. Check if Binary String Has at Most One Segment of Ones
 * https://leetcode.com/problems/check-if-binary-string-has-at-most-one-segment-of-ones/
 * Difficulty: Easy
 *
 * Given a binary string s without leading zeros, return true if s contains at most one
 * contiguous segment of ones. Otherwise, return false.
 */

/**
 * @param {string} s
 * @return {boolean}
```

```
 */
var checkOnesSegment = function(s) {
  let seenZero = false;

  for (let i = 1; i < s.length; i++) {
    if (s[i] === '0') {
      seenZero = true;
    } else if (seenZero) {
      return false;
    }
  }

  return true;
};
```

1785 Minimum Elements To Add To Form A Given Sum

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1785. Minimum Elements to Add to Form a Given Sum
 * https://leetcode.com/problems/minimum-elements-to-add-to-form-a-given-sum/
 * Difficulty: Medium
 *
 * You are given an integer array nums and two integers limit and goal. The array nums has
 * an interesting property that abs(nums[i]) <= limit.
 *
 * Return the minimum number of elements you need to add to make the sum of the array equal
 * to goal. The array must maintain its property that abs(nums[i]) <= limit.
 *
 * Note that abs(x) equals x if x >= 0, and -x otherwise.
 */

/**
 * @param {number[]} nums
 * @param {number} limit
 * @param {number} goal
 * @return {number}
 */
var minElements = function(nums, limit, goal) {
  const currentSum = nums.reduce((sum, num) => sum + num, 0);
  const difference = Math.abs(goal - currentSum);
  return Math.ceil(difference / limit);
};
```

1786 Number Of Restricted Paths From First To Last Node

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1786. Number of Restricted Paths From First to Last Node
 * https://leetcode.com/problems/number-of-restricted-paths-from-first-to-last-node/
 * Difficulty: Medium
 *
 * There is an undirected weighted connected graph. You are given a positive integer n which
 * denotes that the graph has n nodes labeled from 1 to n, and an array edges where each
 * edges[i] = [ui, vi, weighti] denotes that there is an edge between nodes ui and vi with
 * weight equal to weighti.
 *
 * A path from node start to node end is a sequence of nodes [z0, z1, z2, ..., zk] such that
 * z0 = start and zk = end and there is an edge between zi and zi+1 where 0 <= i <= k-1.
 *
 * The distance of a path is the sum of the weights on the edges of the path. Let
 * distanceToLastNode(x) denote the shortest distance of a path between node n and node x.
```

```
 * A restricted path is a path that also satisfies that
 * distanceToLastNode(zi) > distanceToLastNode(zi+1) where 0 <= i <= k-1.
 *
 * Return the number of restricted paths from node 1 to node n. Since that number may be too
 * large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var countRestrictedPaths = function(n, edges) {
  const MODULO = 1e9 + 7;
  const graph = {};
  for (const [u, v, weight] of edges) {
    (graph[u] ??= []).push({ edge: v, weight });
    (graph[v] ??= []).push({ edge: u, weight });
  }

  const distances = new Array(n + 1).fill(Number.MAX_SAFE_INTEGER);
  distances[n] = 0;

  const NOT_VISITED = 0;
  const VISITED = 1;
  const IN_QUEUE = 2;
  const states = new Array(n + 1).fill(NOT_VISITED);
  const queue = [n];

  while (queue.length) {
    const node = queue.shift();
    states[node] = IN_QUEUE;

    for (const { edge, weight } of graph[node]) {
      if (distances[node] + weight >= distances[edge]) continue;

      distances[edge] = distances[node] + weight;

      if (states[edge] === NOT_VISITED) {
        queue.push(edge);
        states[edge] = VISITED;
      } else if (states[edge] === IN_QUEUE) {
        queue.unshift(edge);
      }
    }
  }

  const memo = new Map([[n, 1]]);

  return countPaths(1);

  function countPaths(node) {
    if (memo.has(node)) return memo.get(node);

    let count = 0;
    for (const { edge } of graph[node]) {
      if (distances[edge] < distances[node]) {
        count = (count + countPaths(edge)) % MODULO;
      }
    }

    memo.set(node, count);
    return count;
  }
};
```

1787 Make The Xor Of All Segments Equal To Zero

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1787. Make the XOR of All Segments Equal to Zero
 * https://leetcode.com/problems/make-the-xor-of-all-segments-equal-to-zero/
 * Difficulty: Hard
 *
 * You are given an array nums and an integer k. The XOR of a segment [left, right] where
 * left <= right is the XOR of all the elements with indices between left and right,
 * inclusive: nums[left] XOR nums[left+1] XOR ... XOR nums[right].
 *
 * Return the minimum number of elements to change in the array such that the XOR of all
 * segments of size k is equal to zero.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minChanges = function(nums, k) {
  const n = nums.length;
  const maxVal = 1 << 10;
  const groups = new Array(k).fill().map(() => new Map());

  for (let i = 0; i < n; i++) {
    groups[i % k].set(nums[i], (groups[i % k].get(nums[i]) || 0) + 1);
  }

  const dp = new Array(maxVal).fill(n);
  dp[0] = 0;

  for (let pos = 0; pos < k; pos++) {
    const prevDp = [...dp];
    dp.fill(n);
    const groupSize = Math.floor(n / k) + (pos < n % k ? 1 : 0);
    const minPrev = Math.min(...prevDp);

    for (let xor = 0; xor < maxVal; xor++) {
      if (prevDp[xor] === n) continue;
      for (const [val, count] of groups[pos]) {
        const newXor = xor ^ val;
        dp[newXor] = Math.min(dp[newXor], prevDp[xor] + groupSize - count);
      }
    }

    for (let xor = 0; xor < maxVal; xor++) {
      dp[xor] = Math.min(dp[xor], minPrev + groupSize);
    }
  }

  return dp[0];
};
```

1788 Maximize The Beauty Of The Garden

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1788. Maximize the Beauty of the Garden
 * https://leetcode.com/problems/maximize-the-beauty-of-the-garden/
 * Difficulty: Hard
 *
 * There is a garden of n flowers, and each flower has an integer beauty value. The flowers
 * are arranged in a line. You are given an integer array flowers of size n and each
flowers[i]
 * represents the beauty of the ith flower.
 *
 * A garden is valid if it meets these conditions:
 * - The garden has at least two flowers.
 * - The first and the last flower of the garden have the same beauty value.
 *
```

```
 * As the appointed gardener, you have the ability to remove any (possibly none) flowers from
 * the garden. You want to remove flowers in a way that makes the remaining garden valid. The
 * beauty of the garden is the sum of the beauty of all the remaining flowers.
 *
 * Return the maximum possible beauty of some valid garden after you have removed any
(possibly
 * none) flowers.
 */

/**
 * @param {number[]} flowers
 * @return {number}
 */
var maximumBeauty = function(flowers) {
  const map = new Map();

  for (let i = 0; i < flowers.length; i++) {
    if (!map.has(flowers[i])) {
      map.set(flowers[i], []);
    }
    map.get(flowers[i]).push(i);
  }

  const prefixSum = new Array(flowers.length + 1).fill(0);
  for (let i = 0; i < flowers.length; i++) {
    prefixSum[i + 1] = prefixSum[i] + Math.max(0, flowers[i]);
  }

  let result = -Infinity;

  for (const [value, indices] of map) {
    if (indices.length >= 2) {
      const firstIndex = indices[0];
      const lastIndex = indices[indices.length - 1];
      const positiveSum = prefixSum[lastIndex] - prefixSum[firstIndex + 1];
      const beauty = 2 * value + positiveSum;
      result = Math.max(result, beauty);
    }
  }

  return result;
};
```

1790 Check If One String Swap Can Make Strings Equal

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1790. Check if One String Swap Can Make Strings Equal
 * https://leetcode.com/problems/check-if-one-string-swap-can-make-strings-equal/
 * Difficulty: Easy
 *
 * You are given two strings s1 and s2 of equal length. A string swap is an operation where
 * you choose two indices in a string (not necessarily different) and swap the characters
 * at these indices.
 *
 * Return true if it is possible to make both strings equal by performing at most one string
 * swap on exactly one of the strings. Otherwise, return false.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var areAlmostEqual = function(s1, s2) {
  const indices = [];
  for (const i in s1) {
    if (s1[i] !== s2[i] && indices.push(i) > 2) return false;
  }
```

```
  return s1[indices[0]] === s2[indices[1]] && s1[indices[1]] === s2[indices[0]];
};
```

1791 Find Center Of Star Graph

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1791. Find Center of Star Graph
 * https://leetcode.com/problems/find-center-of-star-graph/
 * Difficulty: Easy
 *
 * There is an undirected star graph consisting of n nodes labeled from 1 to n. A star graph
is a
 * graph where there is one center node and exactly n - 1 edges that connect the center node
with
 * every other node.
 *
 * You are given a 2D integer array edges where each edges[i] = [ui, vi] indicates that there
is an
 * edge between the nodes ui and vi. Return the center of the given star graph.
 */

/**
 * @param {number[][]} edges
 * @return {number}
 */
var findCenter = function(edges) {
  const set = new Set();
  for (const edge of edges) {
    for (const node of edge) {
      if (set.has(node)) {
        return node;
      }
      set.add(node);
    }
  }
};
```

1792 Maximum Average Pass Ratio

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1792. Maximum Average Pass Ratio
 * https://leetcode.com/problems/maximum-average-pass-ratio/
 * Difficulty: Medium
 *
 * There is a school that has classes of students and each class will be having a final exam.
 * You are given a 2D integer array classes, where classes[i] = [passi, totali]. You know
 * beforehand that in the ith class, there are totali total students, but only passi number
 * of students will pass the exam.
 *
 * You are also given an integer extraStudents. There are another extraStudents brilliant
 * students that are guaranteed to pass the exam of any class they are assigned to. You want
 * to assign each of the extraStudents students to a class in a way that maximizes the average
 * pass ratio across all the classes.
 *
 * The pass ratio of a class is equal to the number of students of the class that will pass
 * the exam divided by the total number of students of the class. The average pass ratio is
 * the sum of pass ratios of all the classes divided by the number of the classes.
 *
 * Return the maximum possible average pass ratio after assigning the extraStudents students.
 * Answers within 10-5 of the actual answer will be accepted.
 */
```

```
/**
* @param {number[][]} classes
* @param {number} extraStudents
* @return {number}
*/
var maxAverageRatio = function(classes, extraStudents) {
  const maxHeap = [];

  for (const [pass, total] of classes) {
    const profit = getProfit(pass, total);
    maxHeap.push([profit, pass, total]);
  }

  heapify();

  for (let i = 0; i < extraStudents; i++) {
    const [_, pass, total] = extractMax();
    const newPass = pass + 1;
    const newTotal = total + 1;
    const newProfit = getProfit(newPass, newTotal);
    insert([newProfit, newPass, newTotal]);
  }

  let sumRatio = 0;
  for (const [_, pass, total] of maxHeap) {
    sumRatio += pass / total;
  }

  return sumRatio / classes.length;

  function getProfit(pass, total) {
    return (pass + 1) / (total + 1) - pass / total;
  }

  function heapify() {
    for (let i = Math.floor(maxHeap.length / 2) - 1; i >= 0; i--) {
      siftDown(i);
    }
  }

  function siftDown(i) {
    const left = 2 * i + 1;
    const right = 2 * i + 2;
    let largest = i;

    if (left < maxHeap.length && maxHeap[left][0] > maxHeap[largest][0]) {
      largest = left;
    }

    if (right < maxHeap.length && maxHeap[right][0] > maxHeap[largest][0]) {
      largest = right;
    }

    if (largest !== i) {
      [maxHeap[i], maxHeap[largest]] = [maxHeap[largest], maxHeap[i]];
      siftDown(largest);
    }
  }

  function extractMax() {
    const max = maxHeap[0];
    const last = maxHeap.pop();

    if (maxHeap.length > 0) {
      maxHeap[0] = last;
      siftDown(0);
    }

    return max;
  }

  function insert(item) {
    maxHeap.push(item);
```

```
    let i = maxHeap.length - 1;
    let parent = Math.floor((i - 1) / 2);

    while (i > 0 && maxHeap[i][0] > maxHeap[parent][0]) {
      [maxHeap[i], maxHeap[parent]] = [maxHeap[parent], maxHeap[i]];
      i = parent;
      parent = Math.floor((i - 1) / 2);
    }
  }
};
```

1793 Maximum Score Of A Good Subarray

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1793. Maximum Score of a Good Subarray
 * https://leetcode.com/problems/maximum-score-of-a-good-subarray/
 * Difficulty: Hard
 *
 * You are given an array of integers nums (0-indexed) and an integer k.
 *
 * The score of a subarray (i, j) is defined as min(nums[i], nums[i+1], ..., nums[j]) *
 * (j - i + 1). A good subarray is a subarray where i <= k <= j.
 *
 * Return the maximum possible score of a good subarray.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumScore = function(nums, k) {
  let result = nums[k];
  let minimum = nums[k];
  let left = k;
  let right = k;
  const length = nums.length;

  while (left > 0 || right < length - 1) {
    const leftValue = left > 0 ? nums[left - 1] : 0;
    const rightValue = right < length - 1 ? nums[right + 1] : 0;

    if (leftValue >= rightValue) {
      left--;
      minimum = Math.min(minimum, leftValue);
    } else {
      right++;
      minimum = Math.min(minimum, rightValue);
    }

    result = Math.max(result, minimum * (right - left + 1));
  }

  return result;
};
```

1794 Count Pairs Of Equal Substrings With Minimum Difference

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1794. Count Pairs of Equal Substrings With Minimum Difference
 * https://leetcode.com/problems/count-pairs-of-equal-substrings-with-minimum-difference/
```

```
 * Difficulty: Medium
 *
 * You are given two strings firstString and secondString that are 0-indexed and consist only
 * of lowercase English letters. Count the number of index quadruples (i,j,a,b) that satisfy
 * the following conditions:
 * - 0 <= i <= j < firstString.length
 * - 0 <= a <= b < secondString.length
 * - The substring of firstString that starts at the ith character and ends at the jth
character
 *   (inclusive) is equal to the substring of secondString that starts at the ath character
and
 *   ends at the bth character (inclusive).
 * - j - a is the minimum possible value among all quadruples that satisfy the previous
conditions.
 *
 * Return the number of such quadruples.
 */

/**
 * @param {string} firstString
 * @param {string} secondString
 * @return {number}
 */
var countQuadruples = function(firstString, secondString) {
  const firstPositions = new Array(26).fill(0);
  const secondPositions = new Array(26).fill(0);
  let minDifference = Infinity;
  let quadrupleCount = 0;

  for (let j = firstString.length - 1; j >= 0; j--) {
    firstPositions[firstString[j].charCodeAt(0) - 97] = j + 1;
  }

  for (let a = 0; a < secondString.length; a++) {
    secondPositions[secondString[a].charCodeAt(0) - 97] = a + 1;
  }

  for (let i = 0; i < 26; i++) {
    if (firstPositions[i] && secondPositions[i]) {
      const difference = firstPositions[i] - secondPositions[i];
      if (difference < minDifference) {
        minDifference = difference;
        quadrupleCount = 0;
      }
      if (difference === minDifference) {
        quadrupleCount++;
      }
    }
  }

  return quadrupleCount;
};
```

1796 Second Largest Digit In A String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1796. Second Largest Digit in a String
 * https://leetcode.com/problems/second-largest-digit-in-a-string/
 * Difficulty: Easy
 *
 * Given an alphanumeric string s, return the second largest numerical digit that appears in
 * s, or -1 if it does not exist.
 *
 * An alphanumeric string is a string consisting of lowercase English letters and digits.
 */

/**
 * @param {string} s
```

```
 * @return {number}
 */
var secondHighest = function(s) {
  let largest = -1;
  let result = -1;

  for (const char of s) {
    if (/\d/.test(char)) {
      const digit = parseInt(char);
      if (digit > largest) {
        result = largest;
        largest = digit;
      } else if (digit < largest && digit > result) {
        result = digit;
      }
    }
  }

  return result;
};
```

1797 Design Authentication Manager

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1797. Design Authentication Manager
 * https://leetcode.com/problems/design-authentication-manager/
 * Difficulty: Medium
 *
 * There is an authentication system that works with authentication tokens. For each session,
 * the user will receive a new authentication token that will expire timeToLive seconds after
 * the currentTime. If the token is renewed, the expiry time will be extended to expire
 * timeToLive seconds after the (potentially different) currentTime.
 *
 * Implement the AuthenticationManager class:
 * - AuthenticationManager(int timeToLive) constructs the AuthenticationManager and sets the
 *   timeToLive.
 * - generate(string tokenId, int currentTime) generates a new token with the given tokenId
 *   at the given currentTime in seconds.
 * - renew(string tokenId, int currentTime) renews the unexpired token with the given tokenId
 *   at the given currentTime in seconds. If there are no unexpired tokens with the given
 *   tokenId, the request is ignored, and nothing happens.
 * - countUnexpiredTokens(int currentTime) returns the number of unexpired tokens at the
 *   given currentTime.
 *
 * Note that if a token expires at time t, and another action happens on time t (renew or
 * countUnexpiredTokens), the expiration takes place before the other actions.
 */

/**
 * @param {number} timeToLive
 */
var AuthenticationManager = function(timeToLive) {
  this.timeToLive = timeToLive;
  this.tokens = new Map();
};

/**
 * @param {string} tokenId
 * @param {number} currentTime
 * @return {void}
 */
AuthenticationManager.prototype.generate = function(tokenId, currentTime) {
  this.tokens.set(tokenId, currentTime + this.timeToLive);
};

/**
 * @param {string} tokenId
 * @param {number} currentTime
```

```
 * @return {void}
 */
AuthenticationManager.prototype.renew = function(tokenId, currentTime) {
  if (this.tokens.has(tokenId) && this.tokens.get(tokenId) > currentTime) {
    this.tokens.set(tokenId, currentTime + this.timeToLive);
  }
};

/**
 * @param {number} currentTime
 * @return {number}
 */
AuthenticationManager.prototype.countUnexpiredTokens = function(currentTime) {
  let count = 0;
  for (const [tokenId, expiry] of this.tokens) {
    if (expiry > currentTime) {
      count++;
    } else {
      this.tokens.delete(tokenId);
    }
  }
  return count;
};
```

1798 Maximum Number Of Consecutive Values You Can Make

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1798. Maximum Number of Consecutive Values You Can Make
 * https://leetcode.com/problems/maximum-number-of-consecutive-values-you-can-make/
 * Difficulty: Medium
 *
 * You are given an integer array coins of length n which represents the n coins that you
 * own. The value of the ith coin is coins[i]. You can make some value x if you can choose
 * some of your n coins such that their values sum up to x.
 *
 * Return the maximum number of consecutive integer values that you can make with your coins
 * starting from and including 0.
 *
 * Note that you may have multiple coins of the same value.
 */

/**
 * @param {number[]} coins
 * @return {number}
 */
var getMaximumConsecutive = function(coins) {
  let maxReachable = 0;

  coins.sort((a, b) => a - b);
  for (const coin of coins) {
    if (coin > maxReachable + 1) break;
    maxReachable += coin;
  }

  return maxReachable + 1;
};
```

1799 Maximize Score After N Operations

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1799. Maximize Score After N Operations
```

```
 * https://leetcode.com/problems/maximize-score-after-n-operations/
 * Difficulty: Hard
 *
 * You are given nums, an array of positive integers of size 2 * n. You must perform n
operations
 * on this array.
 *
 * In the ith operation (1-indexed), you will:
 * - Choose two elements, x and y.
 * - Receive a score of i * gcd(x, y).
 * - Remove x and y from nums.
 *
 * Return the maximum score you can receive after performing n operations.
 *
 * The function gcd(x, y) is the greatest common divisor of x and y.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxScore = function(nums) {
  const n = nums.length;
  const pairsGcd = Array(n).fill().map(() => Array(n).fill(0));
  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      pairsGcd[i][j] = gcd(nums[i], nums[j]);
    }
  }

  return solve(1, 0, {}, pairsGcd);

  function gcd(a, b) {
    while (b) {
      a %= b;
      [a, b] = [b, a];
    }
    return a;
  }

  function solve(operations, mask, memo, pairsGcd) {
    if (operations > nums.length / 2) return 0;
    if (memo[mask] !== undefined) return memo[mask];

    let maxScore = 0;
    for (let i = 0; i < nums.length; i++) {
      if (mask & (1 << i)) continue;
      for (let j = i + 1; j < nums.length; j++) {
        if (mask & (1 << j)) continue;
        const newMask = mask | (1 << i) | (1 << j);
        const score = operations * pairsGcd[i][j] + solve(operations + 1, newMask, memo,
pairsGcd);
        maxScore = Math.max(maxScore, score);
      }
    }

    return memo[mask] = maxScore;
  }
};
```

1800 Maximum Ascending Subarray Sum

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1800. Maximum Ascending Subarray Sum
 * https://leetcode.com/problems/maximum-ascending-subarray-sum/
 * Difficulty: Easy
 *
 * Given an array of positive integers nums, return the maximum possible sum of an
```

```
 * ascending subarray in nums.
 *
 * A subarray is defined as a contiguous sequence of numbers in an array.
 *
 * A subarray [numsl, numsl+1, ..., numsr-1, numsr] is ascending if for all i where
 * l <= i < r, numsi  < numsi+1. Note that a subarray of size 1 is ascending.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxAscendingSum = function(nums) {
  let result = nums[0];
  for (let i = 1, max = nums[0]; i < nums.length; i++) {
    max = nums[i] > nums[i - 1] ? max + nums[i] : nums[i];
    result = Math.max(result, max);
  }
  return result;
};
```

1801 Number Of Orders In The Backlog

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1801. Number of Orders in the Backlog
 * https://leetcode.com/problems/number-of-orders-in-the-backlog/
 * Difficulty: Medium
 *
 * You are given a 2D integer array orders, where each orders[i] = [pricei, amounti,
orderTypei]
 * denotes that amounti orders have been placed of type orderTypei at the price pricei.
 * The orderTypei is:
 * - 0 if it is a batch of buy orders, or
 * - 1 if it is a batch of sell orders.
 *
 * Note that orders[i] represents a batch of amounti independent orders with the same price
and
 * order type. All orders represented by orders[i] will be placed before all orders
represented
 * by orders[i+1] for all valid i.
 *
 * There is a backlog that consists of orders that have not been executed. The backlog is
initially
 * empty. When an order is placed, the following happens:
 * - If the order is a buy order, you look at the sell order with the smallest price in the
backlog.
 *   If that sell order's price is smaller than or equal to the current buy order's price,
they will
 *   match and be executed, and that sell order will be removed from the backlog. Else, the
buy
 *   order is added to the backlog.
 * - Vice versa, if the order is a sell order, you look at the buy order with the largest
price in
 *   the backlog. If that buy order's price is larger than or equal to the current sell
order's
 *   price, they will match and be executed, and that buy order will be removed from the
backlog.
 *   Else, the sell order is added to the backlog.
 *
 * Return the total amount of orders in the backlog after placing all the orders from the
input.
 * Since this number can be large, return it modulo 109 + 7.
 */

/**
 * @param {number[][]} orders
 * @return {number}
 */
```

```
var getNumberOfBacklogOrders = function(orders) {
  const MOD = 1e9 + 7;
  const buyOrders = new MinPriorityQueue();
  const sellOrders = new MinPriorityQueue();

  for (let [price, amount, type] of orders) {
    if (type === 0) {
      const order = { price: -price, amount, originalPrice: price, valueOf: () => -price };
      while (amount > 0 && !sellOrders.isEmpty() && sellOrders.front().price <= price) {
        const sellOrder = sellOrders.dequeue();
        const matchAmount = Math.min(amount, sellOrder.amount);
        amount -= matchAmount;
        sellOrder.amount -= matchAmount;
        if (sellOrder.amount > 0) sellOrders.enqueue(sellOrder);
      }
      if (amount > 0) buyOrders.enqueue({ ...order, amount });
    } else {
      const order = { price, amount, valueOf: () => price };
      while (amount > 0 && !buyOrders.isEmpty() && buyOrders.front().originalPrice >= price) {
        const buyOrder = buyOrders.dequeue();
        const matchAmount = Math.min(amount, buyOrder.amount);
        amount -= matchAmount;
        buyOrder.amount -= matchAmount;
        if (buyOrder.amount > 0) buyOrders.enqueue(buyOrder);
      }
      if (amount > 0) sellOrders.enqueue({ ...order, amount });
    }
  }

  let total = 0;
  while (!buyOrders.isEmpty()) {
    total = (total + buyOrders.dequeue().amount) % MOD;
  }
  while (!sellOrders.isEmpty()) {
    total = (total + sellOrders.dequeue().amount) % MOD;
  }

  return total;
};
```

1802 Maximum Value At A Given Index In A Bounded Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1802. Maximum Value at a Given Index in a Bounded Array
 * https://leetcode.com/problems/maximum-value-at-a-given-index-in-a-bounded-array/
 * Difficulty: Medium
 *
 * You are given three positive integers: n, index, and maxSum. You want to construct an array
 * nums (0-indexed) that satisfies the following conditions:
 * - nums.length == n
 * - nums[i] is a positive integer where 0 <= i < n.
 * - abs(nums[i] - nums[i+1]) <= 1 where 0 <= i < n-1.
 * - The sum of all the elements of nums does not exceed maxSum.
 * - nums[index] is maximized.
 *
 * Return nums[index] of the constructed array.
 *
 * Note that abs(x) equals x if x >= 0, and -x otherwise.
 */

/**
 * @param {number} n
 * @param {number} index
 * @param {number} maxSum
 * @return {number}
 */
var maxValue = function(n, index, maxSum) {
  function minSumRequired(peak) {
```

```
    const leftCount = Math.min(index, peak - 1);
    const rightCount = Math.min(n - index - 1, peak - 1);
    let sum = peak;
    sum += (peak - 1 + peak - leftCount) * leftCount / 2;
    sum += (peak - 1 + peak - rightCount) * rightCount / 2;
    sum += (index - leftCount) + (n - index - 1 - rightCount);
    return sum;
  }

  let low = 1;
  let high = maxSum;
  let result = 1;

  while (low <= high) {
    const mid = Math.floor((low + high) / 2);
    if (minSumRequired(mid) <= maxSum) {
      result = mid;
      low = mid + 1;
    } else {
      high = mid - 1;
    }
  }

  return result;
};
```

1803 Count Pairs With Xor In A Range

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1803. Count Pairs With XOR in a Range
 * https://leetcode.com/problems/count-pairs-with-xor-in-a-range/
 * Difficulty: Hard
 *
 * Given a (0-indexed) integer array nums and two integers low and high, return the number
 * of nice pairs.
 *
 * A nice pair is a pair (i, j) where 0 <= i < j < nums.length and low <= (nums[i] XOR
 * nums[j]) <= high.
 */

/**
 * @param {number[]} nums
 * @param {number} low
 * @param {number} high
 * @return {number}
 */
var countPairs = function(nums, low, high) {
  const root = new TrieNode();
  let total = 0;

  for (const num of nums) {
    total += countPairsInRange(num, high, root) - countPairsInRange(num, low - 1, root);
    insert(num, root);
  }

  return total;
};

class TrieNode {
  constructor() {
    this.children = [null, null];
    this.count = 0;
  }
}

function countPairsInRange(num, limit, root, depth = 14) {
  let count = 0;
  let node = root;
```

```
  for (let i = depth; i >= 0 && node; i--) {
    const numBit = (num >> i) & 1;
    const limitBit = (limit >> i) & 1;
    if (limitBit === 0) {
      node = node.children[numBit];
    } else {
      if (node.children[numBit]) {
        count += node.children[numBit].count;
      }
      node = node.children[numBit ^ 1];
    }
  }
  return count + (node ? node.count : 0);
}

function insert(num, root) {
  let node = root;
  for (let i = 14; i >= 0; i--) {
    const bit = (num >> i) & 1;
    if (!node.children[bit]) {
      node.children[bit] = new TrieNode();
    }
    node = node.children[bit];
    node.count++;
  }
}
```

1804 Implement Trie Ii Prefix Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1804. Implement Trie II (Prefix Tree)
 * https://leetcode.com/problems/implement-trie-ii-prefix-tree/
 * Difficulty: Medium
 *
 * A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently
store
 * and retrieve keys in a dataset of strings. There are various applications of this data
structure,
 * such as autocomplete and spellchecker.
 *
 * Implement the Trie class:
 * - Trie() Initializes the trie object.
 * - void insert(String word) Inserts the string word into the trie.
 * - int countWordsEqualTo(String word) Returns the number of instances of the string word in
the
 *   trie.
 * - int countWordsStartingWith(String prefix) Returns the number of strings in the trie that
have
 *   the string prefix as a prefix.
 * - void erase(String word) Erases the string word from the trie.
 */

var Trie = function() {
  this.root = { children: {}, wordCount: 0, prefixCount: 0 };
};

/**
 * @param {string} word
 * @return {void}
 */
Trie.prototype.insert = function(word) {
  let currentNode = this.root;
  for (const char of word) {
    if (!currentNode.children[char]) {
      currentNode.children[char] = { children: {}, wordCount: 0, prefixCount: 0 };
    }
    currentNode = currentNode.children[char];
    currentNode.prefixCount++;
```

```
  }
  currentNode.wordCount++;
};

/**
 * @param {string} word
 * @return {number}
 */
Trie.prototype.countWordsEqualTo = function(word) {
  let currentNode = this.root;
  for (const char of word) {
    if (!currentNode.children[char]) {
      return 0;
    }
    currentNode = currentNode.children[char];
  }
  return currentNode.wordCount;
};

/**
 * @param {string} prefix
 * @return {number}
 */
Trie.prototype.countWordsStartingWith = function(prefix) {
  let currentNode = this.root;
  for (const char of prefix) {
    if (!currentNode.children[char]) {
      return 0;
    }
    currentNode = currentNode.children[char];
  }
  return currentNode.prefixCount;
};

/**
 * @param {string} word
 * @return {void}
 */
Trie.prototype.erase = function(word) {
  let currentNode = this.root;
  for (const char of word) {
    currentNode = currentNode.children[char];
    currentNode.prefixCount--;
  }
  currentNode.wordCount--;
};
```

1805 Number Of Different Integers In A String

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1805. Number of Different Integers in a String
 * https://leetcode.com/problems/number-of-different-integers-in-a-string/
 * Difficulty: Easy
 *
 * You are given a string word that consists of digits and lowercase English letters.
 *
 * You will replace every non-digit character with a space. For example, "a123bc34d8ef34"
 * will become " 123  34 8  34". Notice that you are left with some integers that are
 * separated by at least one space: "123", "34", "8", and "34".
 *
 * Return the number of different integers after performing the replacement operations on
word.
 *
 * Two integers are considered different if their decimal representations without any leading
 * zeros are different.
 */

/**
```

```
 * @param {string} word
 * @return {number}
 */
var numDifferentIntegers = function(word) {
  const uniqueIntegers = new Set();
  let currentNum = '';

  for (const char of word) {
    if (/\d/.test(char)) {
      currentNum += char;
    } else if (currentNum) {
      uniqueIntegers.add(String(BigInt(currentNum)));
      currentNum = '';
    }
  }

  if (currentNum) {
    uniqueIntegers.add(String(BigInt(currentNum)));
  }

  return uniqueIntegers.size;
};
```

1806 Minimum Number Of Operations To Reinitialize A Permutation

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1806. Minimum Number of Operations to Reinitialize a Permutation
 * https://leetcode.com/problems/minimum-number-of-operations-to-reinitialize-a-permutation/
 * Difficulty: Medium
 *
 * You are given an even integer n. You initially have a permutation perm of size n where
 * perm[i] == i (0-indexed).
 *
 * In one operation, you will create a new array arr, and for each i:
 * - If i % 2 == 0, then arr[i] = perm[i / 2].
 * - If i % 2 == 1, then arr[i] = perm[n / 2 + (i - 1) / 2].
 *
 * You will then assign arr to perm.
 *
 * Return the minimum non-zero number of operations you need to perform on perm to return the
 * permutation to its initial value.
 */

/**
 * @param {number} n
 * @return {number}
 */
var reinitializePermutation = function(n) {
  let currentIndex = 1;
  let result = 0;

  do {
    if (currentIndex % 2 === 0) {
      currentIndex = currentIndex / 2;
    } else {
      currentIndex = n / 2 + (currentIndex - 1) / 2;
    }
    result++;
  } while (currentIndex !== 1);

  return result;
};
```

1807 Evaluate The Bracket Pairs Of A String

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1807. Evaluate the Bracket Pairs of a String
 * https://leetcode.com/problems/evaluate-the-bracket-pairs-of-a-string/
 * Difficulty: Medium
 *
 * You are given a string s that contains some bracket pairs, with each pair containing
 * a non-empty key.
 *
 * - For example, in the string "(name)is(age)yearsold", there are two bracket pairs that
contain
 *   the keys "name" and "age".
 *
 * You know the values of a wide range of keys. This is represented by a 2D string array
knowledge
 * where each knowledge[i] = [keyi, valuei] indicates that key keyi has a value of valuei.
 *
 * You are tasked to evaluate all of the bracket pairs. When you evaluate a bracket pair that
 * contains some key keyi, you will:
 * - Replace keyi and the bracket pair with the key's corresponding valuei.
 * - If you do not know the value of the key, you will replace keyi and the bracket pair with
a
 *   question mark "?" (without the quotation marks).
 *
 * Each key will appear at most once in your knowledge. There will not be any nested brackets
in s.
 *
 * Return the resulting string after evaluating all of the bracket pairs.
 */

/**
 * @param {string} s
 * @param {string[][]} knowledge
 * @return {string}
 */
var evaluate = function(s, knowledge) {
  const keyValueMap = new Map(knowledge);
  let result = '';
  let i = 0;

  while (i < s.length) {
    if (s[i] === '(') {
      i++;
      let key = '';
      while (i < s.length && s[i] !== ')') {
        key += s[i++];
      }
      result += keyValueMap.get(key) || '?';
      i++;
    } else {
      result += s[i++];
    }
  }

  return result;
};
```

1808 Maximize Number Of Nice Divisors

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
```

```
 * 1808. Maximize Number of Nice Divisors
 * https://leetcode.com/problems/maximize-number-of-nice-divisors/
 * Difficulty: Hard
 *
 * You are given a positive integer primeFactors. You are asked to construct a positive
 * integer n that satisfies the following conditions:
 * - The number of prime factors of n (not necessarily distinct) is at most primeFactors.
 * - The number of nice divisors of n is maximized. Note that a divisor of n is nice if it
 *   is divisible by every prime factor of n. For example, if n = 12, then its prime factors
 *   are [2,2,3], then 6 and 12 are nice divisors, while 3 and 4 are not.
 *
 * Return the number of nice divisors of n. Since that number can be too large, return it
 * modulo 109 + 7.
 *
 * Note that a prime number is a natural number greater than 1 that is not a product of two
 * smaller natural numbers. The prime factors of a number n is a list of prime numbers such
 * that their product equals n.
 */

/**
 * @param {number} primeFactors
 * @return {number}
 */
var maxNiceDivisors = function(primeFactors) {
  const MOD = 1e9 + 7;

  if (primeFactors <= 3) return primeFactors;

  const quotient = Math.floor(primeFactors / 3);
  const remainder = primeFactors % 3;

  if (remainder === 0) return power(3, quotient);
  if (remainder === 1) return (power(3, quotient - 1) * 4) % MOD;

  return (power(3, quotient) * 2) % MOD;

  function power(base, exponent) {
    let result = BigInt(1);
    base = BigInt(base);
    while (exponent > 0) {
      if (exponent & 1) result = (result * base) % BigInt(MOD);
      base = (base * base) % BigInt(MOD);
      exponent >>= 1;
    }
    return Number(result);
  }
};
```

1810 Minimum Path Cost In A Hidden Grid

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1810. Minimum Path Cost in a Hidden Grid
 * https://leetcode.com/problems/minimum-path-cost-in-a-hidden-grid/
 * Difficulty: Medium
 *
 * This is an interactive problem.
 *
 * There is a robot in a hidden grid, and you are trying to get it from its starting cell to
the
 * target cell in this grid. The grid is of size m x n, and each cell in the grid is either
empty
 * or blocked. It is guaranteed that the starting cell and the target cell are different, and
 * neither of them is blocked.
 *
 * Each cell has a cost that you need to pay each time you move to the cell. The starting
cell's
 * cost is not applied before the robot moves.
 *
```

```
 * You want to find the minimum total cost to move the robot to the target cell. However, you
do
 * not know the grid's dimensions, the starting cell, nor the target cell. You are only
allowed
 * to ask queries to the GridMaster object.
 *
 * The GridMaster class has the following functions:
 * - boolean canMove(char direction) Returns true if the robot can move in that direction.
 *   Otherwise, it returns false.
 * - int move(char direction) Moves the robot in that direction and returns the cost of moving
 *   to that cell. If this move would move the robot to a blocked cell or off the grid, the
move
 *   will be ignored, the robot will remain in the same position, and the function will return
 -1.
 * - boolean isTarget() Returns true if the robot is currently on the target cell. Otherwise,
it
 *   returns false.
 *
 * Note that direction in the above functions should be a character from {'U','D','L','R'},
 * representing the directions up, down, left, and right, respectively.
 *
 * Return the minimum total cost to get the robot from its initial starting cell to the target
 * cell. If there is no valid path between the cells, return -1.
 *
 * Custom testing:
 *
 * The test input is read as a 2D matrix grid of size m x n and four integers r1, c1, r2, and
 * c2 where:
 * - grid[i][j] == 0 indicates that the cell (i, j) is blocked.
 * - grid[i][j] >= 1 indicates that the cell (i, j) is empty and grid[i][j] is the cost to
move
 *   to that cell.
 * - (r1, c1) is the starting cell of the robot.
 * - (r2, c2) is the target cell of the robot.
 *
 * Remember that you will not have this information in your code.
 */

/**
 * // This is the GridMaster's API interface.
 * // You should not implement it, or speculate about its implementation
 * function GridMaster() {
 *
 *     @param {character} direction
 *     @return {boolean}
 *     this.canMove = function(direction) {
 *         ...
 *     };
 *     @param {character} direction
 *     @return {integer}
 *     this.move = function(direction) {
 *         ...
 *     };
 *     @return {boolean}
 *     this.isTarget = function() {
 *         ...
 *     };
 * };
 */

/**
 * @param {GridMaster} master
 * @return {integer}
 */
var findShortestPath = function(master) {
  const directions = ['U', 'D', 'L', 'R'];
  const opposites = { 'U': 'D', 'D': 'U', 'L': 'R', 'R': 'L' };
  const deltas = { 'U': [-1, 0], 'D': [1, 0], 'L': [0, -1], 'R': [0, 1] };
  const visited = new Set();
  const cellCosts = new Map();
  let targetPosition = null;

  cellCosts.set('0,0', 0);
  dfs(0, 0);
```

```
  if (!targetPosition) return -1;

  const dijkstraVisited = new Set();
  const priorityQueue = new PriorityQueue((a, b) => a[2] - b[2]);
  priorityQueue.enqueue([0, 0, 0]);

  while (!priorityQueue.isEmpty()) {
    const [currentRow, currentCol, currentCost] = priorityQueue.dequeue();
    const currentKey = `${currentRow},${currentCol}`;

    if (dijkstraVisited.has(currentKey)) continue;
    dijkstraVisited.add(currentKey);

    if (currentRow === targetPosition[0] && currentCol === targetPosition[1]) {
      return currentCost;
    }

    for (const direction of directions) {
      const [deltaRow, deltaCol] = deltas[direction];
      const nextRow = currentRow + deltaRow;
      const nextCol = currentCol + deltaCol;
      const nextKey = `${nextRow},${nextCol}`;

      if (cellCosts.has(nextKey) && !dijkstraVisited.has(nextKey)) {
        const nextCost = currentCost + cellCosts.get(nextKey);
        priorityQueue.enqueue([nextRow, nextCol, nextCost]);
      }
    }
  }

  return -1;

  function dfs(row, col) {
    const key = `${row},${col}`;
    if (visited.has(key)) return;

    visited.add(key);

    if (master.isTarget()) {
      targetPosition = [row, col];
    }

    for (const direction of directions) {
      if (master.canMove(direction)) {
        const cost = master.move(direction);
        if (cost !== -1) {
          const [deltaRow, deltaCol] = deltas[direction];
          const newRow = row + deltaRow;
          const newCol = col + deltaCol;
          const newKey = `${newRow},${newCol}`;

          if (!cellCosts.has(newKey)) {
            cellCosts.set(newKey, cost);
          }

          dfs(newRow, newCol);

          master.move(opposites[direction]);
        }
      }
    }
  }
};
```

1812 Determine Color Of A Chessboard Square

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1812. Determine Color of a Chessboard Square
```

```
 * https://leetcode.com/problems/determine-color-of-a-chessboard-square/
 * Difficulty: Easy
 *
 * You are given coordinates, a string that represents the coordinates of a square of
 * the chessboard. Below is a chessboard for your reference.
 *
 * Return true if the square is white, and false if the square is black.
 *
 * The coordinate will always represent a valid chessboard square. The coordinate will
 * always have the letter first, and the number second.
 */

/**
 * @param {string} coordinates
 * @return {boolean}
 */
var squareIsWhite = function(coordinates) {
  const [x, y] = coordinates;
  return (x.charCodeAt() + +y) % 2 !== 0;
};
```

1813 Sentence Similarity Iii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1813. Sentence Similarity III
 * https://leetcode.com/problems/sentence-similarity-iii/
 * Difficulty: Medium
 *
 * You are given two strings sentence1 and sentence2, each representing a sentence composed of
 * words. A sentence is a list of words that are separated by a single space with no leading
 * or trailing spaces. Each word consists of only uppercase and lowercase English characters.
 *
 * Two sentences s1 and s2 are considered similar if it is possible to insert an arbitrary
 * sentence (possibly empty) inside one of these sentences such that the two sentences become
 * equal. Note that the inserted sentence must be separated from existing words by spaces.
 *
 * For example:
 * - s1 = "Hello Jane" and s2 = "Hello my name is Jane" can be made equal by inserting "my
name is"
 *   between "Hello" and "Jane" in s1.
 * - s1 = "Frog cool" and s2 = "Frogs are cool" are not similar, since although there is a
sentence
 *   "s are" inserted into s1, it is not separated from "Frog" by a space.
 *
 * Given two sentences sentence1 and sentence2, return true if sentence1 and sentence2 are
 * similar. Otherwise, return false.
 */

/**
 * @param {string} sentence1
 * @param {string} sentence2
 * @return {boolean}
 */
var areSentencesSimilar = function(sentence1, sentence2) {
  let words1 = sentence1.split(' ');
  let words2 = sentence2.split(' ');
  if (words1.length < words2.length) [words1, words2] = [words2, words1];

  let prefixLength = 0;
  while (prefixLength < words2.length && words1[prefixLength] === words2[prefixLength]) {
    prefixLength++;
  }

  let suffixLength = 0;
  while (suffixLength < words2.length - prefixLength
         && words1[words1.length - 1 - suffixLength] === words2[words2.length - 1 -
suffixLength]) {
    suffixLength++;
```

```
  }

  return prefixLength + suffixLength >= words2.length;
};
```

1814 Count Nice Pairs In An Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1814. Count Nice Pairs in an Array
 * https://leetcode.com/problems/count-nice-pairs-in-an-array/
 * Difficulty: Medium
 *
 * You are given an array nums that consists of non-negative integers. Let us define rev(x)
 * as the reverse of the non-negative integer x. For example, rev(123) = 321, and rev(120) =
21.
 * A pair of indices (i, j) is nice if it satisfies all of the following conditions:
 * - 0 <= i < j < nums.length
 * - nums[i] + rev(nums[j]) == nums[j] + rev(nums[i])
 *
 * Return the number of nice pairs of indices. Since that number can be too large, return it
 * modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countNicePairs = function(nums) {
  const MOD = 1e9 + 7;
  const map = new Map();
  let result = 0;

  for (const num of nums) {
    const diff = num - reverseNumber(num);
    const count = map.get(diff) || 0;
    result = (result + count) % MOD;
    map.set(diff, count + 1);
  }

  return result;

  function reverseNumber(num) {
    return Number(String(num).split('').reverse().join(''));
  }
};
```

1815 Maximum Number Of Groups Getting Fresh Donuts

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1815. Maximum Number of Groups Getting Fresh Donuts
 * https://leetcode.com/problems/maximum-number-of-groups-getting-fresh-donuts/
 * Difficulty: Hard
 *
 * There is a donuts shop that bakes donuts in batches of batchSize. They have a rule where
they
 * must serve all of the donuts of a batch before serving any donuts of the next batch. You
are
 * given an integer batchSize and an integer array groups, where groups[i] denotes that there
is
 * a group of groups[i] customers that will visit the shop. Each customer will get exactly one
 * donut.
```

```
 *
 * When a group visits the shop, all customers of the group must be served before serving any
 * of the following groups. A group will be happy if they all get fresh donuts. That is, the
 * first customer of the group does not receive a donut that was left over from the previous
group.
 *
 * You can freely rearrange the ordering of the groups. Return the maximum possible number of
 * happy groups after rearranging the groups.
 */

/**
 * @param {number} batchSize
 * @param {number[]} groups
 * @return {number}
 */
var maxHappyGroups = function(batchSize, groups) {
  const remainders = new Array(batchSize).fill(0);
  let result = 0;

  for (const size of groups) {
    remainders[size % batchSize]++;
  }

  result += remainders[0];
  remainders[0] = 0;

  return result + findMaxHappy(remainders, 0);

  function findMaxHappy(remainders, leftover, memo = new Map()) {
    const key = remainders.join(',') + ',' + leftover;
    if (memo.has(key)) return memo.get(key);

    let maxHappy = 0;
    for (let i = 1; i < batchSize; i++) {
      if (remainders[i] === 0) continue;
      remainders[i]--;

      let current = 0;
      if (leftover === 0) current = 1;
      maxHappy = Math.max(
        maxHappy, current + findMaxHappy(remainders, (leftover + i) % batchSize, memo)
      );

      remainders[i]++;
    }

    memo.set(key, maxHappy);
    return maxHappy;
  }
};
```

1816 Truncate Sentence

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1816. Truncate Sentence
 * https://leetcode.com/problems/truncate-sentence/
 * Difficulty: Easy
 *
 * A sentence is a list of words that are separated by a single space with no leading or
 * trailing spaces. Each of the words consists of only uppercase and lowercase English
 * letters (no punctuation).
 * - For example, "Hello World", "HELLO", and "hello world hello world" are all sentences.
 *
 * You are given a sentence s and an integer k. You want to truncate s such that it contains
 * only the first k words. Return s after truncating it.
 */

/**
```

```
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var truncateSentence = function(s, k) {
  return s.split(' ').slice(0, k).join(' ');
};
```

1817 Finding The Users Active Minutes

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1817. Finding the Users Active Minutes
 * https://leetcode.com/problems/finding-the-users-active-minutes/
 * Difficulty: Medium
 *
 * You are given the logs for users' actions on LeetCode, and an integer k. The logs are
represented
 * by a 2D integer array logs where each logs[i] = [IDi, timei] indicates that the user with
IDi
 * performed an action at the minute timei.
 *
 * Multiple users can perform actions simultaneously, and a single user can perform multiple
actions
 * in the same minute.
 *
 * The user active minutes (UAM) for a given user is defined as the number of unique minutes
in which
 * the user performed an action on LeetCode. A minute can only be counted once, even if
multiple actions
 * occur during it.
 *
 * You are to calculate a 1-indexed array answer of size k such that, for each j (1 <= j <=
k), answer[j]
 * is the number of users whose UAM equals j.
 *
 * Return the array answer as described above.
 */

/**
 * @param {number[][]} logs
 * @param {number} k
 * @return {number[]}
 */
var findingUsersActiveMinutes = function(logs, k) {
  const map = new Map();
  logs.forEach(([id, time]) => {
    map.set(id, map.get(id) || new Set());
    map.get(id).add(time);
  });

  const result = new Array(k).fill(0);
  [...map.values()].forEach(({ size }) => result[size - 1]++);

  return result;
};
```

1818 Minimum Absolute Sum Difference

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1818. Minimum Absolute Sum Difference
```

```
 * https://leetcode.com/problems/minimum-absolute-sum-difference/
 * Difficulty: Medium
 *
 * You are given two positive integer arrays nums1 and nums2, both of length n.
 *
 * The absolute sum difference of arrays nums1 and nums2 is defined as the sum of
 * |nums1[i] - nums2[i]| for each 0 <= i < n (0-indexed).
 *
 * You can replace at most one element of nums1 with any other element in nums1 to minimize
 * the absolute sum difference.
 *
 * Return the minimum absolute sum difference after replacing at most one element in the
 * array nums1. Since the answer may be large, return it modulo 109 + 7.
 *
 * |x| is defined as:
 * - x if x >= 0, or
 * - -x if x < 0.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minAbsoluteSumDiff = function(nums1, nums2) {
  const MOD = 1e9 + 7;
  const sortedNums1 = [...nums1].sort((a, b) => a - b);
  let totalDiff = 0;
  let maxReduction = 0;

  for (let i = 0; i < nums1.length; i++) {
    const currentDiff = Math.abs(nums1[i] - nums2[i]);
    totalDiff = (totalDiff + currentDiff) % MOD;

    let left = 0;
    let right = nums1.length - 1;
    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      const newDiff = Math.abs(sortedNums1[mid] - nums2[i]);
      const reduction = currentDiff - newDiff;
      maxReduction = Math.max(maxReduction, reduction);

      if (sortedNums1[mid] < nums2[i]) left = mid + 1;
      else right = mid - 1;
    }
  }

  return (totalDiff - maxReduction + MOD) % MOD;
};
```

1819 Number Of Different Subsequences Gcds

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1819. Number of Different Subsequences GCDs
 * https://leetcode.com/problems/number-of-different-subsequences-gcds/
 * Difficulty: Hard
 *
 * You are given an array nums that consists of positive integers.
 *
 * The GCD of a sequence of numbers is defined as the greatest integer that divides all the
 * numbers in the sequence evenly.
 * - For example, the GCD of the sequence [4,6,16] is 2.
 *
 * A subsequence of an array is a sequence that can be formed by removing some elements
(possibly
 * none) of the array.
 * - For example, [2,5,10] is a subsequence of [1,2,1,2,4,1,5,10].
 *
```

```
 * Return the number of different GCDs among all non-empty subsequences of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countDifferentSubsequenceGCDs = function(nums) {
  const maxNum = Math.max(...nums);
  const exists = new Array(maxNum + 1).fill(false);
  let result = 0;

  function gcd(a, b) {
    while (b) [a, b] = [b, a % b];
    return a;
  }

  for (const num of nums) {
    exists[num] = true;
  }

  for (let i = 1; i <= maxNum; i++) {
    let currentGcd = 0;
    for (let j = i; j <= maxNum; j += i) {
      if (exists[j]) {
        currentGcd = currentGcd ? gcd(currentGcd, j) : j;
      }
    }
    if (currentGcd === i) result++;
  }

  return result;
};
```

1820 Maximum Number Of Accepted Invitations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1820. Maximum Number of Accepted Invitations
 * https://leetcode.com/problems/maximum-number-of-accepted-invitations/
 * Difficulty: Medium
 *
 * There are m boys and n girls in a class attending an upcoming party.
 *
 * You are given an m x n integer matrix grid, where grid[i][j] equals 0 or 1. If grid[i][j]
== 1,
 * then that means the ith boy can invite the jth girl to the party. A boy can invite at most
one
 * girl, and a girl can accept at most one invitation from a boy.
 *
 * Return the maximum possible number of accepted invitations.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maximumInvitations = function(grid) {
  const boyCount = grid.length;
  const girlCount = grid[0].length;
  const girlMatched = new Array(girlCount).fill(-1);

  let result = 0;

  for (let boy = 0; boy < boyCount; boy++) {
    const visited = new Array(girlCount).fill(false);
    if (findMatch(boy, visited)) {
      result++;
    }
```

```
  }

  return result;

  function findMatch(boy, visited) {
    for (let girl = 0; girl < girlCount; girl++) {
      if (grid[boy][girl] && !visited[girl]) {
        visited[girl] = true;

        if (girlMatched[girl] === -1 || findMatch(girlMatched[girl], visited)) {
          girlMatched[girl] = boy;
          return true;
        }
      }
    }
    return false;
  }
};
```

1822 Sign Of The Product Of An Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1822. Sign of the Product of an Array
 * https://leetcode.com/problems/sign-of-the-product-of-an-array/
 * Difficulty: Easy
 *
 * Implement a function signFunc(x) that returns:
 * - 1 if x is positive.
 * - -1 if x is negative.
 * - 0 if x is equal to 0.
 *
 * You are given an integer array nums. Let product be the product of all values in the array
nums.
 *
 * Return signFunc(product).
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var arraySign = function(nums) {
  let negativeCount = 0;

  for (const num of nums) {
    if (num === 0) return 0;
    if (num < 0) negativeCount++;
  }

  return negativeCount % 2 === 0 ? 1 : -1;
};
```

1823 Find The Winner Of The Circular Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1823. Find the Winner of the Circular Game
 * https://leetcode.com/problems/find-the-winner-of-the-circular-game/
 * Difficulty: Medium
 *
 * There are n friends that are playing a game. The friends are sitting in a circle and are
 * numbered from 1 to n in clockwise order. More formally, moving clockwise from the ith
```

```
 * friend brings you to the (i+1)th friend for 1 <= i < n, and moving clockwise from the
 * nth friend brings you to the 1st friend.
 *
 * The rules of the game are as follows:
 * 1. Start at the 1st friend.
 * 2. Count the next k friends in the clockwise direction including the friend you started at.
 *    The counting wraps around the circle and may count some friends more than once.
 * 3. The last friend you counted leaves the circle and loses the game.
 * 4. If there is still more than one friend in the circle, go back to step 2 starting from
the
 *    friend immediately clockwise of the friend who just lost and repeat.
 * 5. Else, the last friend in the circle wins the game.
 *
 * Given the number of friends, n, and an integer k, return the winner of the game.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var findTheWinner = function(n, k) {
  let winner = 0;

  for (let i = 1; i <= n; i++) {
    winner = (winner + k) % i;
  }

  return winner + 1;
};
```

1824 Minimum Sideway Jumps

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1824. Minimum Sideway Jumps
 * https://leetcode.com/problems/minimum-sideway-jumps/
 * Difficulty: Medium
 *
 * There is a 3 lane road of length n that consists of n + 1 points labeled from 0 to n. A
frog
 * starts at point 0 in the second lane and wants to jump to point n. However, there could be
 * obstacles along the way.
 *
 * You are given an array obstacles of length n + 1 where each obstacles[i] (ranging from 0 to
 3)
 * describes an obstacle on the lane obstacles[i] at point i. If obstacles[i] == 0, there are
no
 * obstacles at point i. There will be at most one obstacle in the 3 lanes at each point.
 * - For example, if obstacles[2] == 1, then there is an obstacle on lane 1 at point 2.
 *
 * The frog can only travel from point i to point i + 1 on the same lane if there is not an
obstacle
 * on the lane at point i + 1. To avoid obstacles, the frog can also perform a side jump to
jump
 * to another lane (even if they are not adjacent) at the same point if there is no obstacle
on
 * the new lane.
 * - For example, the frog can jump from lane 3 at point 3 to lane 1 at point 3.
 *
 * Return the minimum number of side jumps the frog needs to reach any lane at point n
starting
 * from lane 2 at point 0.
 *
 * Note: There will be no obstacles on points 0 and n.
 */

/**
 * @param {number[]} obstacles
```

```
 * @return {number}
 */
var minSideJumps = function(obstacles) {
  const n = obstacles.length;
  let dp = [1, 0, 1];

  for (let i = 1; i < n; i++) {
    const currentObstacle = obstacles[i];
    const nextDp = [Infinity, Infinity, Infinity];

    for (let lane = 1; lane <= 3; lane++) {
      if (lane === currentObstacle) continue;

      nextDp[lane - 1] = dp[lane - 1];

      for (let fromLane = 1; fromLane <= 3; fromLane++) {
        if (fromLane !== lane && fromLane !== currentObstacle) {
          nextDp[lane - 1] = Math.min(nextDp[lane - 1], dp[fromLane - 1] + 1);
        }
      }
    }

    dp = nextDp;
  }

  return Math.min(...dp);
};
```

1825 Finding Mk Average

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1825. Finding MK Average
 * https://leetcode.com/problems/finding-mk-average/
 * Difficulty: Hard
 *
 * You are given two integers, m and k, and a stream of integers. You are tasked to implement
a data
 * structure that calculates the MKAverage for the stream.
 *
 * The MKAverage can be calculated using these steps:
 * - If the number of the elements in the stream is less than m you should consider the
MKAverage to
 *   be -1. Otherwise, copy the last m elements of the stream to a separate container.
 * - Remove the smallest k elements and the largest k elements from the container.
 * - Calculate the average value for the rest of the elements rounded down to the nearest
integer.
 *
 * Implement the MKAverage class:
 * - MKAverage(int m, int k) Initializes the MKAverage object with an empty stream and the two
 *   integers m and k.
 * - void addElement(int num) Inserts a new element num into the stream.
 * - int calculateMKAverage() Calculates and returns the MKAverage for the current stream
rounded
 *   down to the nearest integer.
 */

/**
 * @param {number} m
 * @param {number} k
 */
var MKAverage = function(m, k) {
  this.m = m;
  this.k = k;
  this.stream = [];
  this.sortedStream = [];
  this.sum = 0;
};
```

```
/**
 * @param {number} num
 * @return {void}
 */
MKAverage.prototype.addElement = function(num) {
  this.stream.push(num);

  const insertIndex = this.findInsertPosition(num);
  this.sortedStream.splice(insertIndex, 0, num);
  this.sum += num;

  if (this.stream.length > this.m) {
    const oldestElement = this.stream.shift();
    const oldestIndex = this.sortedStream.indexOf(oldestElement);
    this.sortedStream.splice(oldestIndex, 1);
    this.sum -= oldestElement;
  }
};

/**
 * @return {number}
 */
MKAverage.prototype.calculateMKAverage = function() {
  if (this.stream.length < this.m) {
    return -1;
  }

  let adjustedSum = this.sum;

  for (let i = 0; i < this.k; i++) {
    adjustedSum -= this.sortedStream[i];
  }
  for (let i = this.sortedStream.length - this.k; i < this.sortedStream.length; i++) {
    adjustedSum -= this.sortedStream[i];
  }

  return Math.floor(adjustedSum / (this.m - 2 * this.k));
};

/**
 * Helper method to find insert position using binary search
 * @param {number} num
 * @return {number}
 */
MKAverage.prototype.findInsertPosition = function(num) {
  let left = 0;
  let right = this.sortedStream.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (this.sortedStream[mid] < num) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return left;
};
```

1826 Faulty Sensor

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1826. Faulty Sensor
 * https://leetcode.com/problems/faulty-sensor/
 * Difficulty: Easy
 *
 * An experiment is being conducted in a lab. To ensure accuracy, there are two sensors
```

```
 * collecting data simultaneously. You are given two arrays sensor1 and sensor2, where
 * sensor1[i] and sensor2[i] are the ith data points collected by the two sensors.
 *
 * However, this type of sensor has a chance of being defective, which causes exactly one data
 * point to be dropped. After the data is dropped, all the data points to the right of the
 * dropped data are shifted one place to the left, and the last data point is replaced with
 * some random value. It is guaranteed that this random value will not be equal to the dropped
 * value.
 * - For example, if the correct data is [1,2,3,4,5] and 3 is dropped, the sensor could
 *   return [1,2,4,5,7] (the last position can be any value, not just 7).
 *
 * We know that there is a defect in at most one of the sensors. Return the sensor number
 * (1 or 2) with the defect. If there is no defect in either sensor or if it is impossible
 * to determine the defective sensor, return -1.
 */

/**
 * @param {number[]} sensor1
 * @param {number[]} sensor2
 * @return {number}
 */
var badSensor = function(sensor1, sensor2) {
  const length = sensor1.length;
  let firstDifference = -1;

  for (let i = 0; i < length; i++) {
    if (sensor1[i] !== sensor2[i]) {
      firstDifference = i;
      break;
    }
  }

  if (firstDifference === -1) {
    return -1;
  }

  let sensor1Defective = true;
  let sensor2Defective = true;

  for (let i = firstDifference; i < length - 1; i++) {
    if (sensor1[i] !== sensor2[i + 1]) {
      sensor1Defective = false;
      break;
    }
  }

  for (let i = firstDifference; i < length - 1; i++) {
    if (sensor2[i] !== sensor1[i + 1]) {
      sensor2Defective = false;
      break;
    }
  }

  if (sensor1Defective && !sensor2Defective) {
    return 1;
  }

  if (sensor2Defective && !sensor1Defective) {
    return 2;
  }

  return -1;
};
```

1827 Minimum Operations To Make The Array Increasing

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1827. Minimum Operations to Make the Array Increasing
```

```
 * https://leetcode.com/problems/minimum-operations-to-make-the-array-increasing/
 * Difficulty: Easy
 *
 * You are given an integer array nums (0-indexed). In one operation, you can choose an
 * element of the array and increment it by 1.
 * - For example, if nums = [1,2,3], you can choose to increment nums[1] to make nums =
[1,3,3].
 *
 * Return the minimum number of operations needed to make nums strictly increasing.
 *
 * An array nums is strictly increasing if nums[i] < nums[i+1] for all 0 <= i < nums.length -
1.
 * An array of length 1 is trivially strictly increasing.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minOperations = function(nums) {
  let result = 0;
  let previousValue = nums[0];

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] <= previousValue) {
      const requiredValue = previousValue + 1;
      result += requiredValue - nums[i];
      previousValue = requiredValue;
    } else {
      previousValue = nums[i];
    }
  }

  return result;
};
```

1828 Queries On Number Of Points Inside A Circle

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1828. Queries on Number of Points Inside a Circle
 * https://leetcode.com/problems/queries-on-number-of-points-inside-a-circle/
 * Difficulty: Medium
 *
 * You are given an array points where points[i] = [xi, yi] is the coordinates of the ith
 * point on a 2D plane. Multiple points can have the same coordinates.
 *
 * You are also given an array queries where queries[j] = [xj, yj, rj] describes a circle
 * centered at (xj, yj) with a radius of rj.
 *
 * For each query queries[j], compute the number of points inside the jth circle. Points on
 * the border of the circle are considered inside.
 *
 * Return an array answer, where answer[j] is the answer to the jth query.
 */

/**
 * @param {number[][]} points
 * @param {number[][]} queries
 * @return {number[]}
 */
var countPoints = function(points, queries) {
  const result = [];

  for (const [centerX, centerY, radius] of queries) {
    let pointsInside = 0;
    for (const [pointX, pointY] of points) {
      const distance = Math.sqrt((centerX - pointX) ** 2 + (centerY - pointY) ** 2);
      if (distance <= radius) {
```

```
        pointsInside++;
      }
    }
    result.push(pointsInside);
  }

  return result;
};
```

1829 Maximum Xor For Each Query

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1829. Maximum XOR for Each Query
 * https://leetcode.com/problems/maximum-xor-for-each-query/
 * Difficulty: Medium
 *
 * You are given a sorted array nums of n non-negative integers and an integer maximumBit.
 * You want to perform the following query n times:
 * - Find a non-negative integer k < 2maximumBit such that nums[0] XOR nums[1] XOR ... XOR
 *   nums[nums.length-1] XOR k is maximized. k is the answer to the ith query.
 * - Remove the last element from the current array nums.
 *
 * Return an array answer, where answer[i] is the answer to the ith query.
 */

/**
 * @param {number[]} nums
 * @param {number} maximumBit
 * @return {number[]}
 */
var getMaximumXor = function(nums, maximumBit) {
  const result = [];
  let currentXor = 0;
  const maxValue = (1 << maximumBit) - 1;

  for (const num of nums) {
    currentXor ^= num;
  }

  for (let i = nums.length - 1; i >= 0; i--) {
    const optimalK = currentXor ^ maxValue;
    result.push(optimalK);
    currentXor ^= nums[i];
  }

  return result;
};
```

1830 Minimum Number Of Operations To Make String Sorted

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1830. Minimum Number of Operations to Make String Sorted
 * https://leetcode.com/problems/minimum-number-of-operations-to-make-string-sorted/
 * Difficulty: Hard
 *
 * You are given a string s (0-indexed). You are asked to perform the following operation on
 * s until you get a sorted string:
 * 1. Find the largest index i such that 1 <= i < s.length and s[i] < s[i - 1].
 * 2. Find the largest index j such that i <= j < s.length and s[k] < s[i - 1] for all the
 *    possible values of k in the range [i, j] inclusive.
 * 3. Swap the two characters at indices i - 1 and j.
```

```
 * 4. Reverse the suffix starting at index i.
 *
 * Return the number of operations needed to make the string sorted. Since the answer can be
too
 * large, return it modulo 109 + 7.
 */

/**
 * @param {string} s
 * @return {number}
 */
var makeStringSorted = function(s) {
  const MOD = 1e9 + 7;
  const length = s.length;
  const factorials = new Array(length + 1).fill(1n);
  const inverses = new Array(length + 1).fill(1n);

  for (let i = 2; i <= length; i++) {
    factorials[i] = (factorials[i - 1] * BigInt(i)) % BigInt(MOD);
  }

  for (let i = 1; i <= length; i++) {
    inverses[i] = modInverse(factorials[i], BigInt(MOD));
  }

  let result = 0n;
  const charCounts = new Array(26).fill(0);
  for (let i = length - 1; i >= 0; i--) {
    const charIndex = s.charCodeAt(i) - 97;
    charCounts[charIndex]++;
    let smallerChars = 0;
    for (let j = 0; j < charIndex; j++) {
      smallerChars += charCounts[j];
    }
    let totalPermutations = factorials[length - i - 1];
    for (const count of charCounts) {
      totalPermutations = (totalPermutations * inverses[count]) % BigInt(MOD);
    }
    result = (result + BigInt(smallerChars) * totalPermutations) % BigInt(MOD);
  }

  return Number(result);

  function modInverse(a, m) {
    const m0 = m;
    let q;
    let x0 = 0n;
    let x1 = 1n;
    while (a > 1n) {
      q = a / m;
      [a, m] = [m, a % m];
      [x0, x1] = [x1 - q * x0, x0];
    }
    return x1 < 0n ? x1 + m0 : x1;
  }

  function combinations(n, k) {
    if (k < 0 || k > n) return 0n;
    return (factorials[n] * inverses[k] % BigInt(MOD)) * inverses[n - k] % BigInt(MOD);
  }
};
```

1832 Check If The Sentence Is Pangram

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1832. Check if the Sentence Is Pangram
 * https://leetcode.com/problems/check-if-the-sentence-is-pangram/
 * Difficulty: Easy
```

```
 *
 * A pangram is a sentence where every letter of the English alphabet appears
 * at least once.
 *
 * Given a string sentence containing only lowercase English letters, return
 * true if sentence is a pangram, or false otherwise.
 */

/**
 * @param {string} sentence
 * @return {boolean}
 */
var checkIfPangram = function(sentence) {
  return new Set([...sentence]).size === 26;
};
```

1833 Maximum Ice Cream Bars

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1833. Maximum Ice Cream Bars
 * https://leetcode.com/problems/maximum-ice-cream-bars/
 * Difficulty: Medium
 *
 * It is a sweltering summer day, and a boy wants to buy some ice cream bars.
 *
 * At the store, there are n ice cream bars. You are given an array costs of
 * length n, where costs[i] is the price of the ith ice cream bar in coins.
 * The boy initially has coins coins to spend, and he wants to buy as many
 * ice cream bars as possible.
 *
 * Note: The boy can buy the ice cream bars in any order.
 *
 * Return the maximum number of ice cream bars the boy can buy with coins.
 */

/**
 * @param {number[]} costs
 * @param {number} coins
 * @return {number}
 */
var maxIceCream = function(costs, coins) {
  let count = 0;
  costs.sort((a, b) => a - b);
  for (let i = 0; i < costs.length; i++) {
    if (costs[i] <= coins) {
      count++;
      coins -= costs[i];
    }
  }
  return count;
};
```

1834 Single Threaded Cpu

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1834. Single-Threaded CPU
 * https://leetcode.com/problems/single-threaded-cpu/
 * Difficulty: Medium
 *
 * You are given n tasks labeled from 0 to n - 1 represented by a 2D integer array tasks,
 * where tasks[i] = [enqueueTimei, processingTimei] means that the ith task will be available
```

```
 * to process at enqueueTimei and will take processingTimei to finish processing.
 *
 * You have a single-threaded CPU that can process at most one task at a time and will act in
 * the following way:
 * - If the CPU is idle and there are no available tasks to process, the CPU remains idle.
 * - If the CPU is idle and there are available tasks, the CPU will choose the one with the
shortest
 *   processing time. If multiple tasks have the same shortest processing time, it will choose
 the
 *   task with the smallest index.
 * - Once a task is started, the CPU will process the entire task without stopping.
 * - The CPU can finish a task then start a new one instantly.
 *
 * Return the order in which the CPU will process the tasks.
 */

/**
 * @param {number[][]} tasks
 * @return {number[]}
 */
var getOrder = function(tasks) {
  const indexedTasks = tasks.map(([enqueue, process], index) => ({ enqueue, process, index }))
    .sort((a, b) => a.enqueue - b.enqueue);

  const result = [];
  const availableTasks = new PriorityQueue((a, b) => a.process - b.process || a.index -
b.index);
  let currentTime = indexedTasks[0].enqueue;
  let taskIndex = 0;

  while (result.length < tasks.length) {
    while (taskIndex < tasks.length && indexedTasks[taskIndex].enqueue <= currentTime) {
      availableTasks.push(indexedTasks[taskIndex]);
      taskIndex++;
    }

    if (availableTasks.size() > 0) {
      const task = availableTasks.pop();
      result.push(task.index);
      currentTime += task.process;
    } else {
      currentTime = indexedTasks[taskIndex].enqueue;
    }
  }

  return result;
};
```

1835 Find Xor Sum Of All Pairs Bitwise And

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1835. Find XOR Sum of All Pairs Bitwise AND
 * https://leetcode.com/problems/find-xor-sum-of-all-pairs-bitwise-and/
 * Difficulty: Hard
 *
 * The XOR sum of a list is the bitwise XOR of all its elements. If the list only contains one
 * element, then its XOR sum will be equal to this element.
 * - For example, the XOR sum of [1,2,3,4] is equal to 1 XOR 2 XOR 3 XOR 4 = 4, and the XOR
 *   sum of [3] is equal to 3.
 *
 * You are given two 0-indexed arrays arr1 and arr2 that consist only of non-negative
integers.
 *
 * Consider the list containing the result of arr1[i] AND arr2[j] (bitwise AND) for every (i,
j)
 * pair where 0 <= i < arr1.length and 0 <= j < arr2.length.
 *
 * Return the XOR sum of the aforementioned list.
```

```
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
 * @return {number}
 */
var getXORSum = function(arr1, arr2) {
  let xor1 = 0;
  let xor2 = 0;

  for (const num of arr1) {
    xor1 ^= num;
  }

  for (const num of arr2) {
    xor2 ^= num;
  }

  return xor1 & xor2;
};
```

1836 Remove Duplicates From An Unsorted Linked List

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1836. Remove Duplicates From an Unsorted Linked List
 * https://leetcode.com/problems/remove-duplicates-from-an-unsorted-linked-list/
 * Difficulty: Medium
 *
 * Given the head of a linked list, find all the values that appear more than once in the list
 * and delete the nodes that have any of those values.
 *
 * Return the linked list after the deletions.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var deleteDuplicatesUnsorted = function(head) {
  const map = new Map();
  let current = head;

  while (current) {
    map.set(current.val, (map.get(current.val) || 0) + 1);
    current = current.next;
  }

  const dummy = new ListNode(0);
  dummy.next = head;
  let previous = dummy;
  current = head;

  while (current) {
    if (map.get(current.val) > 1) {
      previous.next = current.next;
    } else {
      previous = current;
    }
    current = current.next;
  }
```

```
  return dummy.next;
};
```

1837 Sum Of Digits In Base K

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1837. Sum of Digits in Base K
 * https://leetcode.com/problems/sum-of-digits-in-base-k/
 * Difficulty: Easy
 *
 * Given an integer n (in base 10) and a base k, return the sum of the digits of n after
 * converting n from base 10 to base k.
 *
 * After converting, each digit should be interpreted as a base 10 number, and the sum should
 * be returned in base 10.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var sumBase = function(n, k) {
  let digitSum = 0;

  while (n > 0) {
    digitSum += n % k;
    n = Math.floor(n / k);
  }

  return digitSum;
};
```

1838 Frequency Of The Most Frequent Element

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1838. Frequency of the Most Frequent Element
 * https://leetcode.com/problems/frequency-of-the-most-frequent-element/
 * Difficulty: Medium
 *
 * The frequency of an element is the number of times it occurs in an array.
 *
 * You are given an integer array nums and an integer k. In one operation, you can choose
 * an index of nums and increment the element at that index by 1.
 *
 * Return the maximum possible frequency of an element after performing at most k operations.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxFrequency = function(nums, k) {
  nums.sort((a, b) => a - b);

  let result = 0;
  let currentSum = 0;
  let left = 0;
```

```
  for (let right = 0; right < nums.length; right++) {
    currentSum += nums[right];
    while (currentSum + k < nums[right] * (right - left + 1)) {
      currentSum -= nums[left];
      left++;
    }
    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

1839 Longest Substring Of All Vowels In Order

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1839. Longest Substring Of All Vowels in Order
 * https://leetcode.com/problems/longest-substring-of-all-vowels-in-order/
 * Difficulty: Medium
 *
 * A string is considered beautiful if it satisfies the following conditions:
 * - Each of the 5 English vowels ('a', 'e', 'i', 'o', 'u') must appear at least once in it.
 * - The letters must be sorted in alphabetical order (i.e. all 'a's before 'e's, all 'e's
 *   before 'i's, etc.).
 *
 * For example, strings "aeiou" and "aaaaaaeiiiioou" are considered beautiful, but "uaeio",
"aeoiu",
 * and "aaaeeeooo" are not beautiful.
 *
 * Given a string word consisting of English vowels, return the length of the longest
beautiful
 * substring of word. If no such substring exists, return 0.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} word
 * @return {number}
 */
var longestBeautifulSubstring = function(word) {
  let result = 0;
  let start = 0;
  let vowelCount = 1;

  for (let i = 1; i < word.length; i++) {
    if (word[i] < word[i - 1]) {
      start = i;
      vowelCount = 1;
    } else if (word[i] > word[i - 1]) {
      vowelCount++;
    }

    if (vowelCount === 5) {
      result = Math.max(result, i - start + 1);
    }
  }

  return result;
};
```

1840 Maximum Building Height

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1840. Maximum Building Height
 * https://leetcode.com/problems/maximum-building-height/
 * Difficulty: Hard
 *
 * You want to build n new buildings in a city. The new buildings will be built in a line
 * and are labeled from 1 to n.
 *
 * However, there are city restrictions on the heights of the new buildings:
 * - The height of each building must be a non-negative integer.
 * - The height of the first building must be 0.
 * - The height difference between any two adjacent buildings cannot exceed 1.
 *
 * Additionally, there are city restrictions on the maximum height of specific buildings.
 * These restrictions are given as a 2D integer array restrictions where
 * restrictions[i] = [idi, maxHeighti] indicates that building idi must have a height
 * less than or equal to maxHeighti.
 *
 * It is guaranteed that each building will appear at most once in restrictions, and building
 * 1 will not be in restrictions.
 *
 * Return the maximum possible height of the tallest building.
 */

/**
 * @param {number} n
 * @param {number[][]} restrictions
 * @return {number}
 */
var maxBuilding = function(n, restrictions) {
  restrictions.push([1, 0], [n, n - 1]);
  restrictions.sort((a, b) => a[0] - b[0]);

  const length = restrictions.length;

  for (let i = 1; i < length; i++) {
    const [currId, currHeight] = restrictions[i];
    const [prevId, prevHeight] = restrictions[i - 1];
    restrictions[i][1] = Math.min(currHeight, prevHeight + (currId - prevId));
  }

  for (let i = length - 2; i >= 0; i--) {
    const [currId, currHeight] = restrictions[i];
    const [nextId, nextHeight] = restrictions[i + 1];
    restrictions[i][1] = Math.min(currHeight, nextHeight + (nextId - currId));
  }

  let maxHeight = 0;

  for (let i = 1; i < length; i++) {
    const [leftId, leftHeight] = restrictions[i - 1];
    const [rightId, rightHeight] = restrictions[i];
    const distance = rightId - leftId;
    const heightDiff = Math.abs(rightHeight - leftHeight);
    const peakHeight = Math.max(leftHeight, rightHeight) + Math.floor((distance - heightDiff)
/ 2);
    maxHeight = Math.max(maxHeight, peakHeight);
  }

  return maxHeight;
};
```

1842 Next Palindrome Using Same Digits

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 1842. Next Palindrome Using Same Digits
 * https://leetcode.com/problems/next-palindrome-using-same-digits/
 * Difficulty: Hard
 *
 * You are given a numeric string num, representing a very large palindrome.
 *
 * Return the smallest palindrome larger than num that can be created by rearranging its
 * digits. If no such palindrome exists, return an empty string "".
 *
 * A palindrome is a number that reads the same backward as forward.
 */

/**
 * @param {string} num
 * @return {string}
 */
var nextPalindrome = function(num) {
  const length = num.length;
  const halfLength = Math.floor(length / 2);
  const leftHalf = num.slice(0, halfLength).split('');

  if (!nextPermutation(leftHalf)) {
    return '';
  }

  const rightHalf = leftHalf.slice().reverse();
  const result = leftHalf.join('') + (length % 2 === 1 ? num[halfLength] : '') +
rightHalf.join('');

  return result;

  function nextPermutation(digits) {
    let pivot = -1;

    for (let i = digits.length - 2; i >= 0; i--) {
      if (digits[i] < digits[i + 1]) {
        pivot = i;
        break;
      }
    }

    if (pivot === -1) return false;

    for (let i = digits.length - 1; i > pivot; i--) {
      if (digits[i] > digits[pivot]) {
        [digits[pivot], digits[i]] = [digits[i], digits[pivot]];
        break;
      }
    }

    const leftPart = digits.slice(0, pivot + 1);
    const rightPart = digits.slice(pivot + 1).reverse();

    for (let i = 0; i < digits.length; i++) {
      digits[i] = i < leftPart.length ? leftPart[i] : rightPart[i - leftPart.length];
    }

    return true;
  }
};
```

1844 Replace All Digits With Characters

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1844. Replace All Digits with Characters
 * https://leetcode.com/problems/replace-all-digits-with-characters/
 * Difficulty: Easy
 *
```

```
 * You are given a 0-indexed string s that has lowercase English letters in its even indices
and
 * digits in its odd indices.
 *
 * You must perform an operation shift(c, x), where c is a character and x is a digit, that
returns
 * the xth character after c.
 * - For example, shift('a', 5) = 'f' and shift('x', 0) = 'x'.
 *
 * For every odd index i, you want to replace the digit s[i] with the result of the
 * shift(s[i-1], s[i]) operation.
 *
 * Return s after replacing all digits. It is guaranteed that shift(s[i-1], s[i]) will never
 * exceed 'z'.
 *
 * Note that shift(c, x) is not a preloaded function, but an operation to be implemented as
part
 * of the solution.
 */

/**
 * @param {string} s
 * @return {string}
 */
var replaceDigits = function(s) {
  const result = s.split('');

  for (let i = 1; i < s.length; i += 2) {
    result[i] = shiftChar(s[i - 1], s[i]);
  }

  return result.join('');

  function shiftChar(char, shift) {
    return String.fromCharCode(char.charCodeAt(0) + parseInt(shift));
  }
};
```

1845 Seat Reservation Manager

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 1845. Seat Reservation Manager
 * https://leetcode.com/problems/seat-reservation-manager/
 * Difficulty: Medium
 *
 * Design a system that manages the reservation state of n seats that are numbered from 1 to
n.
 *
 * Implement the SeatManager class:
 * - SeatManager(int n) Initializes a SeatManager object that will manage n seats numbered
from
 *   1 to n. All seats are initially available.
 * - int reserve() Fetches the smallest-numbered unreserved seat, reserves it, and returns its
 *   number.
 * - void unreserve(int seatNumber) Unreserves the seat with the given seatNumber.
 */

/**
 * @param {number} n
 */
var SeatManager = function(n) {
  this.nextAvailable = 1;
  this.unreservedSeats = new Set();
};

/**
 * @return {number}
 */
```

```
SeatManager.prototype.reserve = function() {
  if (this.unreservedSeats.size > 0) {
    const minSeat = Math.min(...this.unreservedSeats);
    if (minSeat < this.nextAvailable) {
      this.unreservedSeats.delete(minSeat);
      return minSeat;
    }
  }
  return this.nextAvailable++;
};

/**
 * @param {number} seatNumber
 * @return {void}
 */
SeatManager.prototype.unreserve = function(seatNumber) {
  this.unreservedSeats.add(seatNumber);
};
```

1846 Maximum Element After Decreasing And Rearranging

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1846. Maximum Element After Decreasing and Rearranging
 * https://leetcode.com/problems/maximum-element-after-decreasing-and-rearranging/
 * Difficulty: Medium
 *
 * You are given an array of positive integers arr. Perform some operations (possibly none) on
 * arr so that it satisfies these conditions:
 * - The value of the first element in arr must be 1.
 * - The absolute difference between any 2 adjacent elements must be less than or equal to 1.
 *   In other words, abs(arr[i] - arr[i - 1]) <= 1 for each i where 1 <= i < arr.length
 *   (0-indexed). abs(x) is the absolute value of x.
 *
 * There are 2 types of operations that you can perform any number of times:
 * - Decrease the value of any element of arr to a smaller positive integer.
 * - Rearrange the elements of arr to be in any order.
 *
 * Return the maximum possible value of an element in arr after performing the operations to
 * satisfy the conditions.
 */

/**
 * @param {number[]} arr
 * @return {number}
 */
var maximumElementAfterDecrementingAndRearranging = function(arr) {
  arr.sort((a, b) => a - b);
  let maxPossible = 1;

  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > maxPossible) {
      maxPossible++;
    }
  }

  return maxPossible;
};
```

1847 Closest Room

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1847. Closest Room
 * https://leetcode.com/problems/closest-room/
 * Difficulty: Hard
 *
 * There is a hotel with n rooms. The rooms are represented by a 2D integer array rooms where
 * rooms[i] = [roomIdi, sizei] denotes that there is a room with room number roomIdi and size
 * equal to sizei. Each roomIdi is guaranteed to be unique.
 *
 * You are also given k queries in a 2D array queries where queries[j] = [preferredj,
minSizej].
 * The answer to the jth query is the room number id of a room such that:
 * - The room has a size of at least minSizej, and
 * - abs(id - preferredj) is minimized, where abs(x) is the absolute value of x.
 *
 * If there is a tie in the absolute difference, then use the room with the smallest such id.
 * If there is no such room, the answer is -1.
 *
 * Return an array answer of length k where answer[j] contains the answer to the jth query.
 */

/**
 * @param {number[][]} rooms
 * @param {number[][]} queries
 * @return {number[]}
 */
var closestRoom = function(rooms, queries) {
  rooms.sort((a, b) => b[1] - a[1]);
  const sortedQueries = queries
    .map((q, i) => [q[0], q[1], i])
    .sort((a, b) => b[1] - a[1]);

  const result = new Array(queries.length).fill(-1);
  const roomIds = [];
  let roomIndex = 0;

  for (const [preferred, minSize, queryIndex] of sortedQueries) {
    while (roomIndex < rooms.length && rooms[roomIndex][1] >= minSize) {
      insertSorted(rooms[roomIndex][0]);
      roomIndex++;
    }

    result[queryIndex] = findClosestId(preferred);
  }

  return result;

  function insertSorted(id) {
    let left = 0;
    let right = roomIds.length;
    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (roomIds[mid] < id) left = mid + 1;
      else right = mid;
    }
    roomIds.splice(left, 0, id);
  }

  function findClosestId(target) {
    if (!roomIds.length) return -1;

    let left = 0;
    let right = roomIds.length - 1;
    let closest = roomIds[0];
    let minDiff = Math.abs(closest - target);

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      const id = roomIds[mid];
      const diff = Math.abs(id - target);

      if (diff < minDiff || (diff === minDiff && id < closest)) {
        minDiff = diff;
        closest = id;
      }
```

```
      if (id < target) left = mid + 1;
      else if (id > target) right = mid - 1;
      else break;
    }

    return closest;
  }
};
```

1848 Minimum Distance To The Target Element

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1848. Minimum Distance to the Target Element
 * https://leetcode.com/problems/minimum-distance-to-the-target-element/
 * Difficulty: Easy
 *
 * Given an integer array nums (0-indexed) and two integers target and start, find an index i
 * such that nums[i] == target and abs(i - start) is minimized. Note that abs(x) is the
absolute
 * value of x.
 *
 * Return abs(i - start).
 *
 * It is guaranteed that target exists in nums.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @param {number} start
 * @return {number}
 */
var getMinDistance = function(nums, target, start) {
  let result = Infinity;

  for (let i = 0; i < nums.length; i++) {
    if (nums[i] === target) {
      const distance = Math.abs(i - start);
      result = Math.min(result, distance);
    }
  }

  return result;
};
```

1849 Splitting A String Into Descending Consecutive Values

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1849. Splitting a String Into Descending Consecutive Values
 * https://leetcode.com/problems/splitting-a-string-into-descending-consecutive-values/
 * Difficulty: Medium
 *
 * You are given a string s that consists of only digits.
 *
 * Check if we can split s into two or more non-empty substrings such that the numerical
values of
 * the substrings are in descending order and the difference between numerical values of every
 two
 * adjacent substrings is equal to 1.
 * - For example, the string s = "0090089" can be split into ["0090", "089"] with numerical
```

```
values
 *   [90,89]. The values are in descending order and adjacent values differ by 1, so this way
is
 *   valid.
 * - Another example, the string s = "001" can be split into ["0", "01"], ["00", "1"], or
 *   ["0", "0", "1"]. However all the ways are invalid because they have numerical values
 *   [0,1], [0,1], and [0,0,1] respectively, all of which are not in descending order.
 *
 * Return true if it is possible to split s as described above, or false otherwise.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var splitString = function(s) {
  let firstValue = 0;
  for (let i = 0; i < s.length - 1; i++) {
    firstValue = firstValue * 10 + parseInt(s[i]);
    if (trySplit(i + 1, firstValue)) return true;
  }

  return false;

  function trySplit(index, prevValue) {
    if (index === s.length) return true;

    let currentValue = 0;
    for (let i = index; i < s.length; i++) {
      currentValue = currentValue * 10 + parseInt(s[i]);
      if (currentValue >= prevValue) break;
      if (prevValue - currentValue === 1 && trySplit(i + 1, currentValue)) {
        return true;
      }
    }

    return false;
  }
};
```

1850 Minimum Adjacent Swaps To Reach The Kth Smallest Number

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1850. Minimum Adjacent Swaps to Reach the Kth Smallest Number
 * https://leetcode.com/problems/minimum-adjacent-swaps-to-reach-the-kth-smallest-number/
 * Difficulty: Medium
 *
 * You are given a string num, representing a large integer, and an integer k.
 *
 * We call some integer wonderful if it is a permutation of the digits in num and is greater
in
 * value than num. There can be many wonderful integers. However, we only care about the
 * smallest-valued ones.
 * - For example, when num = "5489355142":
 *   - The 1st smallest wonderful integer is "5489355214".
 *   - The 2nd smallest wonderful integer is "5489355241".
 *   - The 3rd smallest wonderful integer is "5489355412".
 *   - The 4th smallest wonderful integer is "5489355421".
 *
 * Return the minimum number of adjacent digit swaps that needs to be applied to num to reach
 * the kth smallest wonderful integer.
 *
 * The tests are generated in such a way that kth smallest wonderful integer exists.
 */

/**
```

```
 * @param {string} num
 * @param {number} k
 * @return {number}
 */
var getMinSwaps = function(num, k) {
  const original = num.split('');
  const target = num.split('');

  for (let i = 0; i < k; i++) {
    nextPermutation(target);
  }

  let result = 0;
  for (let i = 0; i < original.length; i++) {
    if (original[i] !== target[i]) {
      let j = i + 1;
      while (j < original.length && original[j] !== target[i]) j++;
      while (j > i) {
        [original[j], original[j - 1]] = [original[j - 1], original[j]];
        result++;
        j--;
      }
    }
  }

  return result;

  function nextPermutation(arr) {
    let i = arr.length - 2;
    while (i >= 0 && arr[i] >= arr[i + 1]) i--;
    if (i < 0) return false;

    let j = arr.length - 1;
    while (arr[j] <= arr[i]) j--;

    [arr[i], arr[j]] = [arr[j], arr[i]];

    let left = i + 1;
    let right = arr.length - 1;
    while (left < right) {
      [arr[left], arr[right]] = [arr[right], arr[left]];
      left++;
      right--;
    }
    return true;
  }
};
```

1851 Minimum Interval To Include Each Query

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1851. Minimum Interval to Include Each Query
 * https://leetcode.com/problems/minimum-interval-to-include-each-query/
 * Difficulty: Hard
 *
 * You are given a 2D integer array intervals, where intervals[i] = [lefti, righti] describes
 * the ith interval starting at lefti and ending at righti (inclusive). The size of an
interval
 * is defined as the number of integers it contains, or more formally righti - lefti + 1.
 *
 * You are also given an integer array queries. The answer to the jth query is the size of the
 * smallest interval i such that lefti <= queries[j] <= righti. If no such interval exists,
 * the answer is -1.
 *
 * Return an array containing the answers to the queries.
 */
```

```
/**
 * @param {number[][]} intervals
 * @param {number[]} queries
 * @return {number[]}
 */
var minInterval = function(intervals, queries) {
  intervals.sort((a, b) => a[0] - b[0]);
  const sortedQueries = queries
    .map((q, i) => [q, i])
    .sort((a, b) => a[0] - b[0]);

  const activeIntervals = new PriorityQueue((a, b) => a[0] - b[0]);
  const result = new Array(queries.length).fill(-1);
  let intervalIndex = 0;

  for (const [queryValue, queryIndex] of sortedQueries) {
    while (intervalIndex < intervals.length && intervals[intervalIndex][0] <= queryValue) {
      const [start, end] = intervals[intervalIndex];
      activeIntervals.enqueue([end - start + 1, end]);
      intervalIndex++;
    }

    while (!activeIntervals.isEmpty() && activeIntervals.front()?.[1] < queryValue) {
      activeIntervals.dequeue();
    }

    if (!activeIntervals.isEmpty()) {
      result[queryIndex] = activeIntervals.front()[0];
    }
  }

  return result;
};
```

1852 Distinct Numbers In Each Subarray

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1852. Distinct Numbers in Each Subarray
 * https://leetcode.com/problems/distinct-numbers-in-each-subarray/
 * Difficulty: Medium
 *
 * You are given an integer array nums of length n and an integer k. Your task is to find
 * the number of distinct elements in every subarray of size k within nums.
 *
 * Return an array ans such that ans[i] is the count of distinct elements in
 * nums[i..(i + k - 1)] for each index 0 <= i < n - k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var distinctNumbers = function(nums, k) {
  const result = [];
  const map = new Map();

  for (let i = 0; i < k; i++) {
    map.set(nums[i], (map.get(nums[i]) || 0) + 1);
  }

  result.push(map.size);

  for (let i = k; i < nums.length; i++) {
    const leftElement = nums[i - k];
    const rightElement = nums[i];

    map.set(leftElement, map.get(leftElement) - 1);
```

```
    if (map.get(leftElement) === 0) {
      map.delete(leftElement);
    }

    map.set(rightElement, (map.get(rightElement) || 0) + 1);

    result.push(map.size);
  }

  return result;
};
```

1854 Maximum Population Year

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1854. Maximum Population Year
 * https://leetcode.com/problems/maximum-population-year/
 * Difficulty: Easy
 *
 * You are given a 2D integer array logs where each logs[i] = [birthi, deathi] indicates the
 * birth and death years of the ith person.
 *
 * The population of some year x is the number of people alive during that year. The ith
person
 * is counted in year x's population if x is in the inclusive range [birthi, deathi - 1]. Note
 * that the person is not counted in the year that they die.
 *
 * Return the earliest year with the maximum population.
 */

/**
 * @param {number[][]} logs
 * @return {number}
 */
var maximumPopulation = function(logs) {
  const populationChanges = new Array(101).fill(0);
  const baseYear = 1950;

  for (const [birth, death] of logs) {
    populationChanges[birth - baseYear]++;
    populationChanges[death - baseYear]--;
  }

  let maxPopulation = 0;
  let currentPopulation = 0;
  let result = baseYear;

  for (let i = 0; i < populationChanges.length; i++) {
    currentPopulation += populationChanges[i];
    if (currentPopulation > maxPopulation) {
      maxPopulation = currentPopulation;
      result = baseYear + i;
    }
  }

  return result;
};
```

1855 Maximum Distance Between A Pair Of Values

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 1855. Maximum Distance Between a Pair of Values
 * https://leetcode.com/problems/maximum-distance-between-a-pair-of-values/
 * Difficulty: Medium
 *
 * You are given two non-increasing 0-indexed integer arrays nums2 and nums2.
 *
 * A pair of indices (i, j), where 0 <= i < nums1.length and 0 <= j < nums2.length,
 * is valid if both i <= j and nums1[i] <= nums2[j]. The distance of the pair is j - i.
 *
 * Return the maximum distance of any valid pair (i, j). If there are no valid pairs, return
0.
 *
 * An array arr is non-increasing if arr[i-1] >= arr[i] for every 1 <= i < arr.length.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var maxDistance = function(nums1, nums2) {
  let result = 0;
  let left = 0;

  for (let right = 0; right < nums2.length; right++) {
    while (left < nums1.length && nums1[left] > nums2[right]) {
      left++;
    }
    if (left <= right && left < nums1.length) {
      result = Math.max(result, right - left);
    }
  }

  return result;
};
```

1856 Maximum Subarray Min Product

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1856. Maximum Subarray Min-Product
 * https://leetcode.com/problems/maximum-subarray-min-product/
 * Difficulty: Medium
 *
 * The min-product of an array is equal to the minimum value in the array multiplied by
 * the array's sum.
 * - For example, the array [3,2,5] (minimum value is 2) has a min-product of
 *   2 * (3+2+5) = 2 * 10 = 20.
 *
 * Given an array of integers nums, return the maximum min-product of any non-empty
 * subarray of nums. Since the answer may be large, return it modulo 109 + 7.
 *
 * Note that the min-product should be maximized before performing the modulo operation.
 * Testcases are generated such that the maximum min-product without modulo will fit in
 * a 64-bit signed integer.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSumMinProduct = function(nums) {
  const modulo = 1000000007n;
  const stack = [];
  const prefixSums = [0n];
  let maxMinProduct = 0n;
```

```
  for (let i = 0; i < nums.length; i++) {
    prefixSums.push(prefixSums[i] + BigInt(nums[i]));
  }

  for (let right = 0; right <= nums.length; right++) {
    const current = right < nums.length ? BigInt(nums[right]) : 0n;
    while (stack.length && stack[stack.length - 1].value > current) {
      const { index, value } = stack.pop();
      const left = stack.length ? stack[stack.length - 1].index + 1 : 0;
      const subarraySum = prefixSums[right] - prefixSums[left];
      maxMinProduct = maxMinProduct > value * subarraySum ? maxMinProduct : value *
subarraySum;
    }
    stack.push({ index: right, value: current });
  }

  return Number(maxMinProduct % modulo);
};
```

1857 Largest Color Value In A Directed Graph

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1857. Largest Color Value in a Directed Graph
 * https://leetcode.com/problems/largest-color-value-in-a-directed-graph/
 * Difficulty: Hard
 *
 * There is a directed graph of n colored nodes and m edges. The nodes are numbered from 0 to
n - 1.
 *
 * You are given a string colors where colors[i] is a lowercase English letter representing
the
 * color of the ith node in this graph (0-indexed). You are also given a 2D array edges where
 * edges[j] = [aj, bj] indicates that there is a directed edge from node aj to node bj.
 *
 * A valid path in the graph is a sequence of nodes x1 -> x2 -> x3 -> ... -> xk such that
there
 * is a directed edge from xi to xi+1 for every 1 <= i < k. The color value of the path is the
 * number of nodes that are colored the most frequently occurring color along that path.
 *
 * Return the largest color value of any valid path in the given graph, or -1 if the graph
 * contains a cycle.
 */

/**
 * @param {string} colors
 * @param {number[][]} edges
 * @return {number}
 */
var largestPathValue = function(colors, edges) {
  const nodeCount = colors.length;
  const adjacencyList = Array.from({ length: nodeCount }, () => []);
  const inDegree = new Array(nodeCount).fill(0);
  const colorCounts = Array.from({ length: nodeCount }, () => new Array(26).fill(0));
  const queue = [];
  let maxColorValue = 0;
  let processedNodes = 0;

  for (const [from, to] of edges) {
    adjacencyList[from].push(to);
    inDegree[to]++;
  }

  for (let i = 0; i < nodeCount; i++) {
    if (inDegree[i] === 0) queue.push(i);
  }

  while (queue.length) {
    const current = queue.shift();
```

```
    processedNodes++;
    const colorIndex = colors.charCodeAt(current) - 97;
    colorCounts[current][colorIndex]++;

    for (const next of adjacencyList[current]) {
      for (let i = 0; i < 26; i++) {
        colorCounts[next][i] = Math.max(colorCounts[next][i], colorCounts[current][i]);
      }
      if (--inDegree[next] === 0) queue.push(next);
    }

    maxColorValue = Math.max(maxColorValue, ...colorCounts[current]);
  }

  return processedNodes === nodeCount ? maxColorValue : -1;
};
```

1858 Longest Word With All Prefixes

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1858. Longest Word With All Prefixes
 * https://leetcode.com/problems/longest-word-with-all-prefixes/
 * Difficulty: Medium
 *
 * Given an array of strings words, find the longest string in words such that every
 * prefix of it is also in words.
 * - For example, let words = ["a", "app", "ap"]. The string "app" has prefixes "ap" and "a",
 *   all of which are in words.
 *
 * Return the string described above. If there is more than one string with the same length,
 * return the lexicographically smallest one, and if no string exists, return "".
 */

/**
 * @param {string[]} words
 * @return {string}
 */
var longestWord = function(words) {
  const set = new Set(words);
  let result = '';

  for (const word of words) {
    let hasAllPrefixes = true;

    for (let i = 1; i < word.length; i++) {
      if (!set.has(word.slice(0, i))) {
        hasAllPrefixes = false;
        break;
      }
    }

    if (hasAllPrefixes) {
      if (word.length > result.length || (word.length === result.length && word < result)) {
        result = word;
      }
    }
  }

  return result;
};
```

1859 Sorting The Sentence

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1859. Sorting the Sentence
 * https://leetcode.com/problems/sorting-the-sentence/
 * Difficulty: Easy
 *
 * A sentence is a list of words that are separated by a single space with no leading or
trailing
 * spaces. Each word consists of lowercase and uppercase English letters.
 *
 * A sentence can be shuffled by appending the 1-indexed word position to each word then
rearranging
 * the words in the sentence.
 * - For example, the sentence "This is a sentence" can be shuffled as "sentence4 a3 is2
This1" or
 *   "is2 sentence4 This1 a3".
 *
 * Given a shuffled sentence s containing no more than 9 words, reconstruct and return the
original
 * sentence.
 */

/**
 * @param {string} s
 * @return {string}
 */
var sortSentence = function(s) {
  const words = s.split(' ');
  const sortedWords = new Array(words.length);

  for (const word of words) {
    const position = parseInt(word.slice(-1)) - 1;
    sortedWords[position] = word.slice(0, -1);
  }

  return sortedWords.join(' ');
};
```

1860 Incremental Memory Leak

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1860. Incremental Memory Leak
 * https://leetcode.com/problems/incremental-memory-leak/
 * Difficulty: Medium
 *
 * You are given two integers memory1 and memory2 representing the available memory in bits on
 * two memory sticks. There is currently a faulty program running that consumes an increasing
 * amount of memory every second.
 *
 * At the ith second (starting from 1), i bits of memory are allocated to the stick with more
 * available memory (or from the first memory stick if both have the same available memory).
 * If neither stick has at least i bits of available memory, the program crashes.
 *
 * Return an array containing [crashTime, memory1crash, memory2crash], where crashTime is the
 * time (in seconds) when the program crashed and memory1crash and memory2crash are the
 * available bits of memory in the first and second sticks respectively.
 */

/**
 * @param {number} memory1
 * @param {number} memory2
 * @return {number[]}
 */
var memLeak = function(memory1, memory2) {
  let time = 1;
  let stick1 = memory1;
```

```
  let stick2 = memory2;

  while (time <= stick1 || time <= stick2) {
    if (stick1 >= stick2) {
      if (stick1 < time) break;
      stick1 -= time;
    } else {
      if (stick2 < time) break;
      stick2 -= time;
    }
    time++;
  }

  return [time, stick1, stick2];
};
```

1861 Rotating The Box

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1861. Rotating the Box
 * https://leetcode.com/problems/rotating-the-box/
 * Difficulty: Medium
 *
 * You are given an m x n matrix of characters boxGrid representing a side-view of a box.
 * Each cell of the box is one of the following:
 * - A stone '#'
 * - A stationary obstacle '*'
 * - Empty '.'
 *
 * The box is rotated 90 degrees clockwise, causing some of the stones to fall due to gravity.
 * Each stone falls down until it lands on an obstacle, another stone, or the bottom of the
box.
 * Gravity does not affect the obstacles' positions, and the inertia from the box's rotation
 * does not affect the stones' horizontal positions.
 *
 * It is guaranteed that each stone in boxGrid rests on an obstacle, another stone, or the
 * bottom of the box.
 *
 * Return an n x m matrix representing the box after the rotation described above.
 */

/**
 * @param {character[][]} boxGrid
 * @return {character[][]}
 */
var rotateTheBox = function(boxGrid) {
  const rows = boxGrid.length;
  const cols = boxGrid[0].length;
  const result = Array.from({ length: cols }, () => new Array(rows).fill('.'));

  for (let row = 0; row < rows; row++) {
    let freePosition = cols - 1;
    for (let col = cols - 1; col >= 0; col--) {
      if (boxGrid[row][col] === '*') {
        result[col][rows - 1 - row] = '*';
        freePosition = col - 1;
      } else if (boxGrid[row][col] === '#') {
        result[freePosition][rows - 1 - row] = '#';
        freePosition--;
      }
    }
  }

  return result;
};
```

1862 Sum Of Floored Pairs

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1862. Sum of Floored Pairs
 * https://leetcode.com/problems/sum-of-floored-pairs/
 * Difficulty: Hard
 *
 * Given an integer array nums, return the sum of floor(nums[i] / nums[j]) for all pairs of
 * indices 0 <= i, j < nums.length in the array. Since the answer may be too large, return
 * it modulo 109 + 7.
 *
 * The floor() function returns the integer part of the division.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumOfFlooredPairs = function(nums) {
  const modulo = 1e9 + 7;
  const maxValue = Math.max(...nums);
  const frequency = new Array(maxValue + 1).fill(0);
  const prefixSum = new Array(maxValue + 1).fill(0);
  let result = 0;

  for (const num of nums) {
    frequency[num]++;
  }

  for (let i = 1; i <= maxValue; i++) {
    prefixSum[i] = prefixSum[i - 1] + frequency[i];
  }

  for (let i = 1; i <= maxValue; i++) {
    if (frequency[i] === 0) continue;
    for (let divisor = i; divisor <= maxValue; divisor += i) {
      const quotient = Math.floor(divisor / i);
      const count = (
        prefixSum[Math.min(maxValue, divisor + i - 1)] - prefixSum[divisor - 1]
      ) % modulo;
      result = (result + quotient * frequency[i] * count) % modulo;
    }
  }

  return result;
};
```

1863 Sum Of All Subset Xor Totals

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1863. Sum of All Subset XOR Totals
 * https://leetcode.com/problems/sum-of-all-subset-xor-totals/
 * Difficulty: Easy
 *
 * The XOR total of an array is defined as the bitwise XOR of all its elements, or 0 if
 * the array is empty.
 *
 * For example, the XOR total of the array [2,5,6] is 2 XOR 5 XOR 6 = 1.
 * Given an array nums, return the sum of all XOR totals for every subset of nums.
 * Note: Subsets with the same elements should be counted multiple times.
 *
```

```
 * An array a is a subset of an array b if a can be obtained from b by deleting some (possibly
 * zero) elements of b.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var subsetXORSum = function(nums) {
  const totalSubsets = 1 << nums.length;
  let result = 0;

  for (let mask = 0; mask < totalSubsets; mask++) {
    let currentXOR = 0;
    for (let i = 0; i < nums.length; i++) {
      if (mask & (1 << i)) {
        currentXOR ^= nums[i];
      }
    }
    result += currentXOR;
  }

  return result;
};
```

1864 Minimum Number Of Swaps To Make The Binary String Alternating

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1864. Minimum Number of Swaps to Make the Binary String Alternating
 *
https://leetcode.com/problems/minimum-number-of-swaps-to-make-the-binary-string-alternating/
 * Difficulty: Medium
 *
 * Given a binary string s, return the minimum number of character swaps to make it
alternating,
 * or -1 if it is impossible.
 *
 * The string is called alternating if no two adjacent characters are equal. For example, the
 * strings "010" and "1010" are alternating, while the string "0100" is not.
 *
 * Any two characters may be swapped, even if they are not adjacent.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minSwaps = function(s) {
  const length = s.length;
  let ones = 0;
  let zeros = 0;

  for (const char of s) {
    if (char === '1') ones++;
    else zeros++;
  }

  if (Math.abs(ones - zeros) > 1) return -1;

  let mismatchesStartWithZero = 0;
  let mismatchesStartWithOne = 0;

  for (let i = 0; i < length; i++) {
    if (i % 2 === 0) {
      if (s[i] !== '0') mismatchesStartWithZero++;
      if (s[i] !== '1') mismatchesStartWithOne++;
    } else {
      if (s[i] !== '1') mismatchesStartWithZero++;
```

```
      if (s[i] !== '0') mismatchesStartWithOne++;
    }
  }

  if (ones === zeros) {
    return Math.min(mismatchesStartWithZero, mismatchesStartWithOne) >> 1;
  }

  return (ones > zeros ? mismatchesStartWithOne : mismatchesStartWithZero) >> 1;
};
```

1865 Finding Pairs With A Certain Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1865. Finding Pairs With a Certain Sum
 * https://leetcode.com/problems/finding-pairs-with-a-certain-sum/
 * Difficulty: Medium
 *
 * You are given two integer arrays nums1 and nums2. You are tasked to implement a data
structure
 * that supports queries of two types:
 * 1. Add a positive integer to an element of a given index in the array nums2.
 * 2. Count the number of pairs (i, j) such that nums1[i] + nums2[j] equals a given
 *    value (0 <= i < nums1.length and 0 <= j < nums2.length).
 *
 * Implement the FindSumPairs class:
 * - FindSumPairs(int[] nums1, int[] nums2) Initializes the FindSumPairs object with two
integer
 *   arrays nums1 and nums2.
 * - void add(int index, int val) Adds val to nums2[index], i.e., apply nums2[index] += val.
 * - int count(int tot) Returns the number of pairs (i, j) such that nums1[i] + nums2[j] ==
tot.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 */
var FindSumPairs = function(nums1, nums2) {
  this.firstArray = nums1;
  this.secondArray = nums2;
  this.frequency = new Map();

  for (const num of nums2) {
    this.frequency.set(num, (this.frequency.get(num) || 0) + 1);
  }
};

/**
 * @param {number} index
 * @param {number} val
 * @return {void}
 */
FindSumPairs.prototype.add = function(index, val) {
  const oldValue = this.secondArray[index];
  this.secondArray[index] += val;
  const newValue = this.secondArray[index];

  this.frequency.set(oldValue, this.frequency.get(oldValue) - 1);
  this.frequency.set(newValue, (this.frequency.get(newValue) || 0) + 1);
};

/**
 * @param {number} tot
 * @return {number}
 */
FindSumPairs.prototype.count = function(tot) {
  let result = 0;
```

```
  for (const num of this.firstArray) {
    const complement = tot - num;
    result += this.frequency.get(complement) || 0;
  }

  return result;
};
```

1866 Number Of Ways To Rearrange Sticks With K Sticks Visible

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1866. Number of Ways to Rearrange Sticks With K Sticks Visible
 * https://leetcode.com/problems/number-of-ways-to-rearrange-sticks-with-k-sticks-visible/
 * Difficulty: Hard
 *
 * There are n uniquely-sized sticks whose lengths are integers from 1 to n. You want to
arrange
 * the sticks such that exactly k sticks are visible from the left. A stick is visible from
the
 * left if there are no longer sticks to the left of it.
 * - For example, if the sticks are arranged [1,3,2,5,4], then the sticks with lengths 1, 3,
 *   and 5 are visible from the left.
 *
 * Given n and k, return the number of such arrangements. Since the answer may be large,
return
 * it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var rearrangeSticks = function(n, k) {
  const modulo = 1e9 + 7;
  const dp = Array.from({ length: n + 1 }, () => new Array(k + 1).fill(0));
  dp[0][0] = 1;

  for (let sticks = 1; sticks <= n; sticks++) {
    for (let visible = 0; visible <= k; visible++) {
      if (visible > sticks) continue;
      if (visible > 0) {
        dp[sticks][visible] = (dp[sticks][visible] + dp[sticks - 1][visible - 1]) % modulo;
      }
      dp[sticks][visible] = (dp[sticks][visible] + dp[sticks - 1][visible] * (sticks - 1)) %
modulo;
    }
  }

  return dp[n][k];
};
```

1868 Product Of Two Run Length Encoded Arrays

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1868. Product of Two Run-Length Encoded Arrays
 * https://leetcode.com/problems/product-of-two-run-length-encoded-arrays/
 * Difficulty: Medium
 *
 * Run-length encoding is a compression algorithm that allows for an integer array nums with
```

```
 * many segments of consecutive repeated numbers to be represented by a (generally smaller)
 * 2D array encoded. Each encoded[i] = [vali, freqi] describes the ith segment of repeated
 * numbers in nums where vali is the value that is repeated freqi times.
 * - For example, nums = [1,1,1,2,2,2,2,2] is represented by the run-length encoded array
 *   encoded = [[1,3],[2,5]]. Another way to read this is "three 1's followed by five 2's".
 *
 * The product of two run-length encoded arrays encoded1 and encoded2 can be calculated using
 * the following steps:
 * 1. Expand both encoded1 and encoded2 into the full arrays nums1 and nums2 respectively.
 * 2. Create a new array prodNums of length nums1.length and set prodNums[i] = nums1[i] *
nums2[i].
 * 3. Compress prodNums into a run-length encoded array and return it.
 *
 * You are given two run-length encoded arrays encoded1 and encoded2 representing full arrays
 * nums1 and nums2 respectively. Both nums1 and nums2 have the same length. Each
 * encoded1[i] = [vali, freqi] describes the ith segment of nums1, and each
 * encoded2[j] = [valj, freqj] describes the jth segment of nums2.
 *
 * Return the product of encoded1 and encoded2.
 *
 * Note: Compression should be done such that the run-length encoded array has the minimum
 * possible length.
 */

/**
 * @param {number[][]} encoded1
 * @param {number[][]} encoded2
 * @return {number[][]}
 */
var findRLEArray = function(encoded1, encoded2) {
  const result = [];
  let i = 0;
  let j = 0;
  let freq1 = 0;
  let freq2 = 0;

  while (i < encoded1.length && j < encoded2.length) {
    if (freq1 === 0) {
      freq1 = encoded1[i][1];
    }
    if (freq2 === 0) {
      freq2 = encoded2[j][1];
    }

    const product = encoded1[i][0] * encoded2[j][0];
    const minFreq = Math.min(freq1, freq2);
    if (result.length > 0 && result[result.length - 1][0] === product) {
      result[result.length - 1][1] += minFreq;
    } else {
      result.push([product, minFreq]);
    }

    freq1 -= minFreq;
    freq2 -= minFreq;

    if (freq1 === 0) i++;
    if (freq2 === 0) j++;
  }

  return result;
};
```

1869 Longer Contiguous Segments Of Ones Than Zeros

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1869. Longer Contiguous Segments of Ones than Zeros
 * https://leetcode.com/problems/longer-contiguous-segments-of-ones-than-zeros/
 * Difficulty: Easy
```

```
 *
 * Given a binary string s, return true if the longest contiguous segment of 1's is strictly
 * longer than the longest contiguous segment of 0's in s, or return false otherwise.
 * - For example, in s = "110100010" the longest continuous segment of 1s has length 2, and
the
 *   longest continuous segment of 0s has length 3.
 *
 * Note that if there are no 0's, then the longest continuous segment of 0's is considered to
 * have a length 0. The same applies if there is no 1's.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var checkZeroOnes = function(s) {
  let maxOnes = 0;
  let maxZeros = 0;
  let currentOnes = 0;
  let currentZeros = 0;

  for (const char of s) {
    if (char === '1') {
      currentOnes++;
      currentZeros = 0;
      maxOnes = Math.max(maxOnes, currentOnes);
    } else {
      currentZeros++;
      currentOnes = 0;
      maxZeros = Math.max(maxZeros, currentZeros);
    }
  }

  return maxOnes > maxZeros;
};
```

1870 Minimum Speed To Arrive On Time

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1870. Minimum Speed to Arrive on Time
 * https://leetcode.com/problems/minimum-speed-to-arrive-on-time/
 * Difficulty: Medium
 *
 * You are given a floating-point number hour, representing the amount of time you have to
reach
 * the office. To commute to the office, you must take n trains in sequential order. You are
also
 * given an integer array dist of length n, where dist[i] describes the distance (in
kilometers)
 * of the ith train ride.
 *
 * Each train can only depart at an integer hour, so you may need to wait in between each
train
 * ride.
 * - For example, if the 1st train ride takes 1.5 hours, you must wait for an additional 0.5
hours
 *   before you can depart on the 2nd train ride at the 2 hour mark.
 *
 * Return the minimum positive integer speed (in kilometers per hour) that all the trains must
 * travel at for you to reach the office on time, or -1 if it is impossible to be on time.
 *
 * Tests are generated such that the answer will not exceed 107 and hour will have at most two
 * digits after the decimal point.
 */

/**
 * @param {number[]} dist
 * @param {number} hour
```

```
 * @return {number}
 */
var minSpeedOnTime = function(dist, hour) {
  const trainCount = dist.length;
  if (Math.ceil(trainCount - 1) > hour) return -1;

  let minSpeed = 1;
  let maxSpeed = 10000000;
  let result = -1;

  while (minSpeed <= maxSpeed) {
    const midSpeed = (minSpeed + maxSpeed) >> 1;
    let totalTime = 0;

    for (let i = 0; i < trainCount - 1; i++) {
      totalTime += Math.ceil(dist[i] / midSpeed);
    }
    totalTime += dist[trainCount - 1] / midSpeed;

    if (totalTime <= hour) {
      result = midSpeed;
      maxSpeed = midSpeed - 1;
    } else {
      minSpeed = midSpeed + 1;
    }
  }

  return result;
};
```

1871 Jump Game Vii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1871. Jump Game VII
 * https://leetcode.com/problems/jump-game-vii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed binary string s and two integers minJump and maxJump. In the
 * beginning, you are standing at index 0, which is equal to '0'. You can move from index
 * i to index j if the following conditions are fulfilled:
 * - i + minJump <= j <= min(i + maxJump, s.length - 1), and
 * - s[j] == '0'.
 *
 * Return true if you can reach index s.length - 1 in s, or false otherwise.
 */

/**
 * @param {string} s
 * @param {number} minJump
 * @param {number} maxJump
 * @return {boolean}
 */
var canReach = function(s, minJump, maxJump) {
  const length = s.length;
  const reachable = new Array(length).fill(false);
  reachable[0] = true;
  const queue = [0];
  let farthest = 0;

  while (queue.length) {
    const current = queue.shift();
    const start = current + minJump;
    const end = Math.min(current + maxJump, length - 1);

    for (let next = Math.max(start, farthest + 1); next <= end; next++) {
      if (s[next] === '0' && !reachable[next]) {
        reachable[next] = true;
        queue.push(next);
```

```
        if (next === length - 1) return true;
      }
    }
    farthest = Math.max(farthest, end);
  }

  return false;
};
```

1872 Stone Game Viii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1872. Stone Game VIII
 * https://leetcode.com/problems/stone-game-viii/
 * Difficulty: Hard
 *
 * Alice and Bob take turns playing a game, with Alice starting first.
 *
 * There are n stones arranged in a row. On each player's turn, while the number of stones is
more
 * than one, they will do the following:
 * 1. Choose an integer x > 1, and remove the leftmost x stones from the row.
 * 2. Add the sum of the removed stones' values to the player's score.
 * 3. Place a new stone, whose value is equal to that sum, on the left side of the row.
 *
 * The game stops when only one stone is left in the row.
 *
 * The score difference between Alice and Bob is (Alice's score - Bob's score). Alice's goal
is to
 * maximize the score difference, and Bob's goal is the minimize the score difference.
 *
 * Given an integer array stones of length n where stones[i] represents the value of the ith
stone
 * from the left, return the score difference between Alice and Bob if they both play
optimally.
 */

/**
 * @param {number[]} stones
 * @return {number}
 */
var stoneGameVIII = function(stones) {
  const length = stones.length;
  const prefixSums = new Array(length).fill(0);
  prefixSums[0] = stones[0];

  for (let i = 1; i < length; i++) {
    prefixSums[i] = prefixSums[i - 1] + stones[i];
  }

  let result = prefixSums[length - 1];
  for (let i = length - 2; i >= 1; i--) {
    result = Math.max(result, prefixSums[i] - result);
  }

  return result;
};
```

1874 Minimize Product Sum Of Two Arrays

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
```

```
 * 1874. Minimize Product Sum of Two Arrays
 * https://leetcode.com/problems/minimize-product-sum-of-two-arrays/
 * Difficulty: Medium
 *
 * The product sum of two equal-length arrays a and b is equal to the sum of a[i] * b[i]
 * for all 0 <= i < a.length (0-indexed).
 * - For example, if a = [1,2,3,4] and b = [5,2,3,1], the product sum would be
 *   1*5 + 2*2 + 3*3 + 4*1 = 22.
 *
 * Given two arrays nums1 and nums2 of length n, return the minimum product sum if you are
 * allowed to rearrange the order of the elements in nums1.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minProductSum = function(nums1, nums2) {
  nums1.sort((a, b) => a - b);
  nums2.sort((a, b) => b - a);

  return nums1.reduce((sum, val, i) => sum + val * nums2[i], 0);
};
```

1876 Substrings Of Size Three With Distinct Characters

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1876. Substrings of Size Three with Distinct Characters
 * https://leetcode.com/problems/substrings-of-size-three-with-distinct-characters/
 * Difficulty: Easy
 *
 * A string is good if there are no repeated characters.
 *
 * Given a string s, return the number of good substrings of length three in s.
 *
 * Note that if there are multiple occurrences of the same substring, every occurrence should
 * be counted.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countGoodSubstrings = function(s) {
  if (s.length < 3) return 0;

  let result = 0;
  const charCount = new Map();

  for (let i = 0; i < 3; i++) {
    charCount.set(s[i], (charCount.get(s[i]) || 0) + 1);
  }
  if (charCount.size === 3) result++;

  for (let i = 3; i < s.length; i++) {
    const oldChar = s[i - 3];
    charCount.set(oldChar, charCount.get(oldChar) - 1);
    if (charCount.get(oldChar) === 0) charCount.delete(oldChar);

    charCount.set(s[i], (charCount.get(s[i]) || 0) + 1);
    if (charCount.size === 3) result++;
  }

  return result;
};
```

1877 Minimize Maximum Pair Sum In Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1877. Minimize Maximum Pair Sum in Array
 * https://leetcode.com/problems/minimize-maximum-pair-sum-in-array/
 * Difficulty: Medium
 *
 * The pair sum of a pair (a,b) is equal to a + b. The maximum pair sum is the largest pair
sum
 * in a list of pairs.
 * - For example, if we have pairs (1,5), (2,3), and (4,4), the maximum pair sum would be
 *   max(1+5, 2+3, 4+4) = max(6, 5, 8) = 8.
 *
 * Given an array nums of even length n, pair up the elements of nums into n / 2 pairs such
that:
 * - Each element of nums is in exactly one pair, and
 * - The maximum pair sum is minimized.
 *
 * Return the minimized maximum pair sum after optimally pairing up the elements.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minPairSum = function(nums) {
  nums.sort((a, b) => a - b);
  let result = 0;
  let left = 0;
  let right = nums.length - 1;

  while (left < right) {
    result = Math.max(result, nums[left] + nums[right]);
    left++;
    right--;
  }

  return result;
};
```

1878 Get Biggest Three Rhombus Sums In A Grid

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1878. Get Biggest Three Rhombus Sums in a Grid
 * https://leetcode.com/problems/get-biggest-three-rhombus-sums-in-a-grid/
 * Difficulty: Medium
 *
 * You are given an m x n integer matrix grid.
 *
 * A rhombus sum is the sum of the elements that form the border of a regular rhombus shape in
 * grid. The rhombus must have the shape of a square rotated 45 degrees with each of the
corners
 * centered in a grid cell. Below is an image of four valid rhombus shapes with the
corresponding
 * colored cells that should be included in each rhombus sum.
 *
 * Note that the rhombus can have an area of 0, which is depicted by the purple rhombus in the
 * bottom right corner.
 *
```

```
 * Return the biggest three distinct rhombus sums in the grid in descending order. If there
are
 * less than three distinct values, return all of them.
 */

/**
 * @param {number[][]} grid
 * @return {number[]}
 */
var getBiggestThree = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const sums = new Set();

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      sums.add(grid[i][j]);
      for (let size = 1; size <= Math.min(i, rows - 1 - i, j, cols - 1 - j); size++) {
        let rhombusSum = 0;
        rhombusSum += grid[i - size][j] + grid[i + size][j] + grid[i][j - size] + grid[i][j +
size];
        for (let offset = 1; offset < size; offset++) {
          rhombusSum += grid[i - size + offset][j + offset] + grid[i + size - offset][j +
offset];
          rhombusSum += grid[i - size + offset][j - offset] + grid[i + size - offset][j -
offset];
        }
        sums.add(rhombusSum);
      }
    }
  }

  const sortedSums = [...sums].sort((a, b) => b - a);
  return sortedSums.slice(0, Math.min(3, sortedSums.length));
};
```

1879 Minimum Xor Sum Of Two Arrays

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1879. Minimum XOR Sum of Two Arrays
 * https://leetcode.com/problems/minimum-xor-sum-of-two-arrays/
 * Difficulty: Hard
 *
 * You are given two integer arrays nums1 and nums2 of length n.
 *
 * The XOR sum of the two integer arrays is (nums1[0] XOR nums2[0]) + (nums1[1] XOR nums2[1])
 * + ... + (nums1[n - 1] XOR nums2[n - 1]) (0-indexed).
 * - For example, the XOR sum of [1,2,3] and [3,2,1] is equal to (1 XOR 3) + (2 XOR 2) +
 *   (3 XOR 1) = 2 + 0 + 2 = 4.
 *
 * Rearrange the elements of nums2 such that the resulting XOR sum is minimized.
 *
 * Return the XOR sum after the rearrangement.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minimumXORSum = function(nums1, nums2) {
  const n = nums1.length;
  const dp = new Array(1 << n).fill(Infinity);
  dp[0] = 0;

  for (let mask = 0; mask < (1 << n); mask++) {
    const usedCount = mask.toString(2).split('1').length - 1;
    for (let j = 0; j < n; j++) {
```

```
      if (!(mask & (1 << j))) {
        const newMask = mask | (1 << j);
        dp[newMask] = Math.min(dp[newMask], dp[mask] + (nums1[usedCount] ^ nums2[j]));
      }
    }
  }

  return dp[(1 << n) - 1];
};
```

1880 Check If Word Equals Summation Of Two Words

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1880. Check if Word Equals Summation of Two Words
 * https://leetcode.com/problems/check-if-word-equals-summation-of-two-words/
 * Difficulty: Easy
 *
 * The letter value of a letter is its position in the alphabet starting from
 * 0 (i.e. 'a' -> 0, 'b' -> 1, 'c' -> 2, etc.).
 *
 * The numerical value of some string of lowercase English letters s is the
 * concatenation of the letter values of each letter in s, which is then
 * converted into an integer.
 *
 * For example, if s = "acb", we concatenate each letter's letter value,
 * resulting in "021". After converting it, we get 21.
 *
 * You are given three strings firstWord, secondWord, and targetWord, each
 * consisting of lowercase English letters 'a' through 'j' inclusive.
 *
 * Return true if the summation of the numerical values of firstWord and
 * secondWord equals the numerical value of targetWord, or false otherwise.
 */

/**
 * @param {string} firstWord
 * @param {string} secondWord
 * @param {string} targetWord
 * @return {boolean}
 */
var isSumEqual = function(firstWord, secondWord, targetWord) {
  const [a, b, c] = [firstWord, secondWord, targetWord]
    .map(str => +str.split('').map(v => v.charCodeAt() - 97).join(''));

  return a + b === c;
};
```

1881 Maximum Value After Insertion

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1881. Maximum Value after Insertion
 * https://leetcode.com/problems/maximum-value-after-insertion/
 * Difficulty: Medium
 *
 * You are given a very large integer n, represented as a string, and an integer digit x.
 * The digits in n and the digit x are in the inclusive range [1, 9], and n may represent
 * a negative number.
 *
 * You want to maximize n's numerical value by inserting x anywhere in the decimal
representation
 * of n. You cannot insert x to the left of the negative sign.
```

```
 * - For example, if n = 73 and x = 6, it would be best to insert it between 7 and 3, making
 *   n = 763.
 * - If n = -55 and x = 2, it would be best to insert it before the first 5, making n = -255.
 *
 * Return a string representing the maximum value of n after the insertion.
 */

/**
 * @param {string} n
 * @param {number} x
 * @return {string}
 */
var maxValue = function(n, x) {
  const isNegative = n[0] === '-';
  const digits = n.slice(isNegative ? 1 : 0);
  let result = '';

  if (isNegative) {
    for (let i = 0; i <= digits.length; i++) {
      if (i === digits.length || x < parseInt(digits[i])) {
        result = '-' + digits.slice(0, i) + x + digits.slice(i);
        break;
      }
    }
  } else {
    for (let i = 0; i <= digits.length; i++) {
      if (i === digits.length || x > parseInt(digits[i])) {
        result = digits.slice(0, i) + x + digits.slice(i);
        break;
      }
    }
  }

  return result;
};
```

1882 Process Tasks Using Servers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1882. Process Tasks Using Servers
 * https://leetcode.com/problems/process-tasks-using-servers/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer arrays servers and tasks of lengths n and m
respectively.
 * servers[i] is the weight of the ith server, and tasks[j] is the time needed to process the
 * jth task in seconds.
 *
 * Tasks are assigned to the servers using a task queue. Initially, all servers are free, and
 * the queue is empty.
 *
 * At second j, the jth task is inserted into the queue (starting with the 0th task being
inserted
 * at second 0). As long as there are free servers and the queue is not empty, the task in the
 front
 * of the queue will be assigned to a free server with the smallest weight, and in case of a
tie,
 * it is assigned to a free server with the smallest index.
 *
 * If there are no free servers and the queue is not empty, we wait until a server becomes
free
 * and immediately assign the next task. If multiple servers become free at the same time,
then
 * multiple tasks from the queue will be assigned in order of insertion following the weight
and
 * index priorities above.
 *
 * A server that is assigned task j at second t will be free again at second t + tasks[j].
```

```
 *
 * Build an array ans of length m, where ans[j] is the index of the server the jth task will
 * be assigned to.
 *
 * Return the array ans.
 */

/**
 * @param {number[]} servers
 * @param {number[]} tasks
 * @return {number[]}
 */
var assignTasks = function(servers, tasks) {
  const serverHeap = new PriorityQueue((a, b) => a[0] * 1000000 + a[1] - (b[0] * 1000000 +
b[1]));
  const busyHeap = new PriorityQueue((a, b) => a[0] - b[0]);
  const result = new Array(tasks.length);

  for (let i = 0; i < servers.length; i++) {
    serverHeap.enqueue([servers[i], i]);
  }

  let time = 0;
  let taskIndex = 0;

  while (taskIndex < tasks.length) {
    time = Math.max(time, taskIndex);

    while (!busyHeap.isEmpty() && busyHeap.front()[0] <= time) {
      const [, weight, index] = busyHeap.dequeue();
      serverHeap.enqueue([weight, index]);
    }

    while (serverHeap.size() && taskIndex < tasks.length && taskIndex <= time) {
      const [weight, index] = serverHeap.dequeue();
      result[taskIndex] = index;
      busyHeap.enqueue([time + tasks[taskIndex], weight, index]);
      taskIndex++;
    }

    if (serverHeap.isEmpty() && taskIndex < tasks.length) {
      const [freeTime, weight, index] = busyHeap.dequeue();
      time = freeTime;
      serverHeap.enqueue([weight, index]);
    }
  }

  return result;
};
```

1883 Minimum Skips To Arrive At Meeting On Time

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1883. Minimum Skips to Arrive at Meeting On Time
 * https://leetcode.com/problems/minimum-skips-to-arrive-at-meeting-on-time/
 * Difficulty: Hard
 *
 * You are given an integer hoursBefore, the number of hours you have to travel to your
meeting.
 * To arrive at your meeting, you have to travel through n roads. The road lengths are given
as
 * an integer array dist of length n, where dist[i] describes the length of the ith road in
 * kilometers. In addition, you are given an integer speed, which is the speed (in km/h) you
 * will travel at.
 *
 * After you travel road i, you must rest and wait for the next integer hour before you can
begin
 * traveling on the next road. Note that you do not have to rest after traveling the last road
```

```
 * because you are already at the meeting.
 * - For example, if traveling a road takes 1.4 hours, you must wait until the 2 hour mark
before
 *   traveling the next road. If traveling a road takes exactly 2 hours, you do not need to
wait.
 *
 * However, you are allowed to skip some rests to be able to arrive on time, meaning you do
not
 * need to wait for the next integer hour. Note that this means you may finish traveling
future
 * roads at different hour marks.
 * - For example, suppose traveling the first road takes 1.4 hours and traveling the second
road
 *   takes 0.6 hours. Skipping the rest after the first road will mean you finish traveling
the
 *   second road right at the 2 hour mark, letting you start traveling the third road
immediately.
 *
 * Return the minimum number of skips required to arrive at the meeting on time, or -1 if it
is
 * impossible.
 */

/**
 * @param {number[]} dist
 * @param {number} speed
 * @param {number} hoursBefore
 * @return {number}
 */
var minSkips = function(dist, speed, hoursBefore) {
  const n = dist.length;
  const dp = Array.from({ length: n + 1 }, () => new Array(n + 1).fill(Infinity));
  dp[0][0] = 0;

  for (let i = 1; i <= n; i++) {
    for (let skips = 0; skips <= i; skips++) {
      if (skips < i) {
        const prevTime = dp[i - 1][skips];
        if (prevTime !== Infinity) {
          const time = prevTime + dist[i - 1];
          dp[i][skips] = Math.min(dp[i][skips], Math.ceil(time / speed) * speed);
        }
      }
      if (skips > 0) {
        const prevTime = dp[i - 1][skips - 1];
        if (prevTime !== Infinity) {
          dp[i][skips] = Math.min(dp[i][skips], prevTime + dist[i - 1]);
        }
      }
    }
  }

  const targetTime = hoursBefore * speed;
  for (let skips = 0; skips <= n; skips++) {
    if (dp[n][skips] <= targetTime) {
      return skips;
    }
  }

  return -1;
};
```

1884 Egg Drop With 2 Eggs And N Floors

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1884. Egg Drop With 2 Eggs and N Floors
 * https://leetcode.com/problems/egg-drop-with-2-eggs-and-n-floors/
 * Difficulty: Medium
```

```
 *
 * You are given two identical eggs and you have access to a building with n floors labeled
 * from 1 to n.
 *
 * You know that there exists a floor f where 0 <= f <= n such that any egg dropped at a floor
 * higher than f will break, and any egg dropped at or below floor f will not break.
 *
 * In each move, you may take an unbroken egg and drop it from any floor x (where 1 <= x <=
n).
 * If the egg breaks, you can no longer use it. However, if the egg does not break, you may
 * reuse it in future moves.
 *
 * Return the minimum number of moves that you need to determine with certainty what the value
 * of f is.
 */

/**
 * @param {number} n
 * @return {number}
 */
var twoEggDrop = function(n) {
  const dp = Array.from({ length: 3 }, () => new Array(n + 1).fill(Infinity));
  dp[0][0] = 0;
  dp[1][0] = 0;
  dp[2][0] = 0;

  for (let floors = 1; floors <= n; floors++) {
    dp[1][floors] = floors;
    for (let eggs = 2; eggs <= 2; eggs++) {
      for (let k = 1; k <= floors; k++) {
        const breaks = dp[eggs - 1][k - 1];
        const survives = dp[eggs][floors - k];
        dp[eggs][floors] = Math.min(dp[eggs][floors], 1 + Math.max(breaks, survives));
      }
    }
  }

  return dp[2][n];
};
```

1885 Count Pairs In Two Arrays

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1885. Count Pairs in Two Arrays
 * https://leetcode.com/problems/count-pairs-in-two-arrays/
 * Difficulty: Medium
 *
 * Given two integer arrays nums1 and nums2 of length n, count the pairs of indices (i, j)
 * such that i < j and nums1[i] + nums1[j] > nums2[i] + nums2[j].
 *
 * Return the number of pairs satisfying the condition.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var countPairs = function(nums1, nums2) {
  const differences = nums1.map((val, i) => val - nums2[i]);
  differences.sort((a, b) => a - b);

  let count = 0;
  let left = 0;
  let right = differences.length - 1;

  while (left < right) {
    if (differences[left] + differences[right] > 0) {
```

```
      count += right - left;
      right--;
    } else {
      left++;
    }
  }

  return count;
};
```

1886 Determine Whether Matrix Can Be Obtained By Rotation

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1886. Determine Whether Matrix Can Be Obtained By Rotation
 * https://leetcode.com/problems/determine-whether-matrix-can-be-obtained-by-rotation/
 * Difficulty: Easy
 *
 * Given two n x n binary matrices mat and target, return true if it is possible to make mat
 * equal to target by rotating mat in 90-degree increments, or false otherwise.
 */

/**
 * @param {number[][]} mat
 * @param {number[][]} target
 * @return {boolean}
 */
var findRotation = function(mat, target) {
  const goal = JSON.stringify(target);
  for (let i = 0; i < 4; i++) {
    if (JSON.stringify(rotate(mat)) === goal) {
      return true;
    }
  }
  return false;
};

function rotate(matrix) {
  matrix = matrix.reverse();

  for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < i; j++) {
      [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];
    }
  }

  return matrix;
}
```

1887 Reduction Operations To Make The Array Elements Equal

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1887. Reduction Operations to Make the Array Elements Equal
 * https://leetcode.com/problems/reduction-operations-to-make-the-array-elements-equal/
 * Difficulty: Medium
 *
 * Given an integer array nums, your goal is to make all elements in nums equal. To complete
one
 * operation, follow these steps:
 * 1. Find the largest value in nums. Let its index be i (0-indexed) and its value be largest.
 *    If there are multiple elements with the largest value, pick the smallest i.
 * 2. Find the next largest value in nums strictly smaller than largest. Let its value be
```

```
 *    nextLargest.
 * 3. Reduce nums[i] to nextLargest.
 *
 * Return the number of operations to make all elements in nums equal.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var reductionOperations = function(nums) {
  nums.sort((a, b) => a - b);
  let result = 0;
  let count = 0;

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] !== nums[i - 1]) {
      count++;
    }
    result += count;
  }

  return result;
};
```

1888 Minimum Number Of Flips To Make The Binary String Alternating

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1888. Minimum Number of Flips to Make the Binary String Alternating
 *
https://leetcode.com/problems/minimum-number-of-flips-to-make-the-binary-string-alternating/
 * Difficulty: Medium
 *
 * You are given a binary string s. You are allowed to perform two types of operations on the
 * string in any sequence:
 * - Type-1: Remove the character at the start of the string s and append it to the end of
 *   the string.
 * - Type-2: Pick any character in s and flip its value, i.e., if its value is '0' it becomes
 *   '1' and vice-versa.
 *
 * Return the minimum number of type-2 operations you need to perform such that s becomes
 * alternating.
 *
 * The string is called alternating if no two adjacent characters are equal.
 *
 * For example, the strings "010" and "1010" are alternating, while the string "0100" is not.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minFlips = function(s) {
  const n = s.length;
  let result = n;
  let flipsZeroStart = 0;
  let flipsOneStart = 0;

  for (let i = 0; i < n; i++) {
    if (s[i] !== (i % 2 === 0 ? '0' : '1')) flipsZeroStart++;
    if (s[i] !== (i % 2 === 0 ? '1' : '0')) flipsOneStart++;
  }

  result = Math.min(flipsZeroStart, flipsOneStart);

  for (let i = 0; i < n - 1; i++) {
    if (s[i] !== (i % 2 === 0 ? '0' : '1')) flipsZeroStart--;
    if (s[i] !== (i % 2 === 0 ? '1' : '0')) flipsOneStart--;
```

```
    if (s[i] !== ((n + i) % 2 === 0 ? '0' : '1')) flipsZeroStart++;
    if (s[i] !== ((n + i) % 2 === 0 ? '1' : '0')) flipsOneStart++;
    result = Math.min(result, flipsZeroStart, flipsOneStart);
  }

  return result;
};
```

1889 Minimum Space Wasted From Packaging

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1889. Minimum Space Wasted From Packaging
 * https://leetcode.com/problems/minimum-space-wasted-from-packaging/
 * Difficulty: Hard
 *
 * You have n packages that you are trying to place in boxes, one package in each box. There
are
 * m suppliers that each produce boxes of different sizes (with infinite supply). A package
can
 * be placed in a box if the size of the package is less than or equal to the size of the box.
 *
 * The package sizes are given as an integer array packages, where packages[i] is the size of
the
 * ith package. The suppliers are given as a 2D integer array boxes, where boxes[j] is an
array
 * of box sizes that the jth supplier produces.
 *
 * You want to choose a single supplier and use boxes from them such that the total wasted
space
 * is minimized. For each package in a box, we define the space wasted to be size of the box -
 size
 * of the package. The total wasted space is the sum of the space wasted in all the boxes.
 * - For example, if you have to fit packages with sizes [2,3,5] and the supplier offers boxes
 of
 *   sizes [4,8], you can fit the packages of size-2 and size-3 into two boxes of size-4 and
the
 *   package with size-5 into a box of size-8. This would result in a waste
 *   of (4-2) + (4-3) + (8-5) = 6.
 *
 * Return the minimum total wasted space by choosing the box supplier optimally, or -1 if it
is
 * impossible to fit all the packages inside boxes. Since the answer may be large, return it
 * modulo 109 + 7.
 */

/**
 * @param {number[]} packages
 * @param {number[][]} boxes
 * @return {number}
 */
var minWastedSpace = function(packages, boxes) {
  const modulo = 1e9 + 7;
  packages.sort((a, b) => a - b);
  let minWaste = BigInt(2) ** BigInt(60);
  const n = packages.length;

  const prefixSums = new Array(n + 1).fill(BigInt(0));
  for (let i = 0; i < n; i++) {
    prefixSums[i + 1] = prefixSums[i] + BigInt(packages[i]);
  }

  for (const supplier of boxes) {
    supplier.sort((a, b) => a - b);
    if (supplier[supplier.length - 1] < packages[n - 1]) continue;

    let waste = BigInt(0);
    let packageIndex = 0;
```

```
    for (const box of supplier) {
      if (packageIndex >= n) break;
      const upperBound = binarySearch(packages, box, packageIndex);
      if (upperBound > packageIndex) {
        const count = BigInt(upperBound - packageIndex);
        waste += count * BigInt(box) - (prefixSums[upperBound] - prefixSums[packageIndex]);
        packageIndex = upperBound;
      }
    }

    if (packageIndex === n) {
      minWaste = minWaste < waste ? minWaste : waste;
    }
  }

  return minWaste === BigInt(2) ** BigInt(60) ? -1 : Number(minWaste % BigInt(modulo));
};

function binarySearch(arr, target, start) {
  let left = start;
  let right = arr.length;

  while (left < right) {
    const mid = (left + right) >> 1;
    if (arr[mid] <= target) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
}
```

1891 Cutting Ribbons

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1891. Cutting Ribbons
 * https://leetcode.com/problems/cutting-ribbons/
 * Difficulty: Medium
 *
 * You are given an integer array ribbons, where ribbons[i] represents the length of the ith
 * ribbon, and an integer k. You may cut any of the ribbons into any number of segments of
 * positive integer lengths, or perform no cuts at all.
 *
 * - For example, if you have a ribbon of length 4, you can:
 *   - Keep the ribbon of length 4,
 *   - Cut it into one ribbon of length 3 and one ribbon of length 1,
 *   - Cut it into two ribbons of length 2,
 *   - Cut it into one ribbon of length 2 and two ribbons of length 1, or
 *   - Cut it into four ribbons of length 1.
 *
 * Your task is to determine the maximum length of ribbon, x, that allows you to cut at least
 * k ribbons, each of length x. You can discard any leftover ribbon from the cuts. If it is
 * impossible to cut k ribbons of the same length, return 0.
 */

/**
 * @param {number[]} ribbons
 * @param {number} k
 * @return {number}
 */
var maxLength = function(ribbons, k) {
  let left = 1;
  let right = Math.max(...ribbons);
  let result = 0;
```

```
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);

    if (helper(mid)) {
      result = mid;
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;

  function helper(length) {
    let count = 0;
    for (const ribbon of ribbons) {
      count += Math.floor(ribbon / length);
      if (count >= k) return true;
    }
    return false;
  }
};
```

1893 Check If All The Integers In A Range Are Covered

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1893. Check if All the Integers in a Range Are Covered
 * https://leetcode.com/problems/check-if-all-the-integers-in-a-range-are-covered/
 * Difficulty: Easy
 *
 * You are given a 2D integer array ranges and two integers left and right. Each
 * ranges[i] = [starti, endi] represents an inclusive interval between starti and endi.
 *
 * Return true if each integer in the inclusive range [left, right] is covered by at least
 * one interval in ranges. Return false otherwise.
 *
 * An integer x is covered by an interval ranges[i] = [starti, endi] if starti <= x <= endi.
 */

/**
 * @param {number[][]} ranges
 * @param {number} left
 * @param {number} right
 * @return {boolean}
 */
var isCovered = function(ranges, left, right) {
  const covered = new Array(51).fill(false);

  for (const [start, end] of ranges) {
    for (let i = start; i <= end; i++) {
      covered[i] = true;
    }
  }

  for (let i = left; i <= right; i++) {
    if (!covered[i]) return false;
  }

  return true;
};
```

1894 Find The Student That Will Replace The Chalk

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1894. Find the Student that Will Replace the Chalk
 * https://leetcode.com/problems/find-the-student-that-will-replace-the-chalk/
 * Difficulty: Medium
 *
 * There are n students in a class numbered from 0 to n - 1. The teacher will give each
student
 * a problem starting with the student number 0, then the student number 1, and so on until
the
 * teacher reaches the student number n - 1. After that, the teacher will restart the process,
 * starting with the student number 0 again.
 *
 * You are given a 0-indexed integer array chalk and an integer k. There are initially k
pieces
 * of chalk. When the student number i is given a problem to solve, they will use chalk[i]
 * pieces of chalk to solve that problem. However, if the current number of chalk pieces is
 * strictly less than chalk[i], then the student number i will be asked to replace the chalk.
 *
 * Return the index of the student that will replace the chalk pieces.
 */

/**
 * @param {number[]} chalk
 * @param {number} k
 * @return {number}
 */
var chalkReplacer = function(chalk, k) {
  let totalChalk = 0;
  for (const amount of chalk) {
    totalChalk += amount;
  }

  k = k % totalChalk;

  for (let i = 0; i < chalk.length; i++) {
    if (k < chalk[i]) return i;
    k -= chalk[i];
  }

  return 0;
};
```

1895 Largest Magic Square

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1895. Largest Magic Square
 * https://leetcode.com/problems/largest-magic-square/
 * Difficulty: Medium
 *
 * A k x k magic square is a k x k grid filled with integers such that every row sum, every
 * column sum, and both diagonal sums are all equal. The integers in the magic square do not
 * have to be distinct. Every 1 x 1 grid is trivially a magic square.
 *
 * Given an m x n integer grid, return the size (i.e., the side length k) of the largest magic
 * square that can be found within this grid.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var largestMagicSquare = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const rowSums = Array.from({ length: rows + 1 }, () => new Array(cols + 1).fill(0));
```

```
  const colSums = Array.from({ length: rows + 1 }, () => new Array(cols + 1).fill(0));

  for (let i = 1; i <= rows; i++) {
    for (let j = 1; j <= cols; j++) {
      rowSums[i][j] = rowSums[i][j - 1] + grid[i - 1][j - 1];
      colSums[i][j] = colSums[i - 1][j] + grid[i - 1][j - 1];
    }
  }

  const maxSize = 1;
  for (let size = Math.min(rows, cols); size >= 2; size--) {
    for (let i = size; i <= rows; i++) {
      for (let j = size; j <= cols; j++) {
        const rowSum = rowSums[i][j] - rowSums[i][j - size];
        let isMagic = true;

        for (let k = 1; k < size; k++) {
          if (rowSums[i - k][j] - rowSums[i - k][j - size] !== rowSum) {
            isMagic = false;
            break;
          }
        }

        if (!isMagic) continue;

        for (let k = 0; k < size; k++) {
          if (colSums[i][j - k] - colSums[i - size][j - k] !== rowSum) {
            isMagic = false;
            break;
          }
        }

        if (!isMagic) continue;

        let diagSum1 = 0;
        let diagSum2 = 0;
        for (let k = 0; k < size; k++) {
          diagSum1 += grid[i - size + k][j - size + k];
          diagSum2 += grid[i - size + k][j - 1 - k];
        }

        if (diagSum1 === rowSum && diagSum2 === rowSum) {
          return size;
        }
      }
    }
  }

  return maxSize;
};
```

1896 Minimum Cost To Change The Final Value Of Expression

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1896. Minimum Cost to Change the Final Value of Expression
 * https://leetcode.com/problems/minimum-cost-to-change-the-final-value-of-expression/
 * Difficulty: Hard
 *
 * You are given a valid boolean expression as a string expression consisting of the
 * characters '1','0','&' (bitwise AND operator),'|' (bitwise OR operator),'(', and ')'.
 * - For example, "()1|1" and "(1)&()" are not valid while "1", "(((1))|(0))", and "1|(0&(1))"
 *   are valid expressions.
 *
 * Return the minimum cost to change the final value of the expression.
 * - For example, if expression = "1|1|(0&0)&1", its value is
 *   1|1|(0&0)&1 = 1|1|0&1 = 1|0&1 = 1&1 = 1. We want to apply operations so that the new
 *   expression evaluates to 0.
 *
```

```
 * The cost of changing the final value of an expression is the number of operations performed
 * on the expression. The types of operations are described as follows:
 * - Turn a '1' into a '0'.
 * - Turn a '0' into a '1'.
 * - Turn a '&' into a '|'.
 * - Turn a '|' into a '&'.
 *
 * Note: '&' does not take precedence over '|' in the order of calculation. Evaluate
parentheses
 * first, then in left-to-right order.
 */

/**
 * @param {string} expression
 * @return {number}
 */
var minOperationsToFlip = function(expression) {
  const stack = [];
  for (const char of expression) {
    if (char === ')') {
      const values = [];
      while (stack.length && stack[stack.length - 1][0] !== '(') {
        values.push(stack.pop());
      }
      stack.pop();
      let [val1, cost1] = values.pop();
      while (values.length) {
        const [op] = values.pop();
        const [val2, cost2] = values.pop();
        if (op === '&') {
          if (val1 === 1 && val2 === 1) {
            val1 = 1;
            cost1 = Math.min(cost1, cost2);
          } else if (val1 === 0 && val2 === 0) {
            val1 = 0;
            cost1 = Math.min(cost1 + 1, cost2 + 1);
          } else {
            val1 = 0;
            cost1 = Math.min(cost1 + cost2, 1);
          }
        } else {
          if (val1 === 1 && val2 === 1) {
            val1 = 1;
            cost1 = Math.min(cost1 + 1, cost2 + 1);
          } else if (val1 === 0 && val2 === 0) {
            val1 = 0;
            cost1 = Math.min(cost1, cost2);
          } else {
            val1 = 1;
            cost1 = Math.min(cost1 + cost2, 1);
          }
        }
      }
      stack.push([val1, cost1]);
    } else if (char === '1') {
      stack.push([1, 1]);
    } else if (char === '0') {
      stack.push([0, 1]);
    } else {
      stack.push([char]);
    }
  }

  const values = [];
  while (stack.length) {
    values.push(stack.pop());
  }

  let [val1, cost1] = values.pop();
  while (values.length) {
    const [op] = values.pop();
    const [val2, cost2] = values.pop();
    if (op === '&') {
      if (val1 === 1 && val2 === 1) {
        val1 = 1;
```

```
        cost1 = Math.min(cost1, cost2);
      } else if (val1 === 0 && val2 === 0) {
        val1 = 0;
        cost1 = Math.min(cost1 + 1, cost2 + 1);
      } else {
        val1 = 0;
        cost1 = Math.min(cost1 + cost2, 1);
      }
    } else {
      if (val1 === 1 && val2 === 1) {
        val1 = 1;
        cost1 = Math.min(cost1 + 1, cost2 + 1);
      } else if (val1 === 0 && val2 === 0) {
        val1 = 0;
        cost1 = Math.min(cost1, cost2);
      } else {
        val1 = 1;
        cost1 = Math.min(cost1 + cost2, 1);
      }
    }
  }

  return cost1;
};
```

1897 Redistribute Characters To Make All Strings Equal

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1897. Redistribute Characters to Make All Strings Equal
 * https://leetcode.com/problems/redistribute-characters-to-make-all-strings-equal/
 * Difficulty: Easy
 *
 * You are given an array of strings words (0-indexed).
 *
 * In one operation, pick two distinct indices i and j, where words[i] is a non-empty string,
 * and move any character from words[i] to any position in words[j].
 *
 * Return true if you can make every string in words equal using any number of operations, and
 * false otherwise.
 */

/**
 * @param {string[]} words
 * @return {boolean}
 */
var makeEqual = function(words) {
  const charCount = new Array(26).fill(0);
  const wordCount = words.length;

  for (const word of words) {
    for (const char of word) {
      charCount[char.charCodeAt(0) - 97]++;
    }
  }

  for (const count of charCount) {
    if (count % wordCount !== 0) return false;
  }

  return true;
};
```

1898 Maximum Number Of Removable Characters

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1898. Maximum Number of Removable Characters
 * https://leetcode.com/problems/maximum-number-of-removable-characters/
 * Difficulty: Medium
 *
 * You are given two strings s and p where p is a subsequence of s. You are also given a
distinct
 * 0-indexed integer array removable containing a subset of indices of s (s is also
0-indexed).
 *
 * You want to choose an integer k (0 <= k <= removable.length) such that, after removing k
 * characters from s using the first k indices in removable, p is still a subsequence of s.
 * More formally, you will mark the character at s[removable[i]] for each 0 <= i < k, then
 * remove all marked characters and check if p is still a subsequence.
 *
 * Return the maximum k you can choose such that p is still a subsequence of s after the
removals.
 *
 * A subsequence of a string is a new string generated from the original string with some
 * characters (can be none) deleted without changing the relative order of the remaining
characters.
 */

/**
 * @param {string} s
 * @param {string} p
 * @param {number[]} removable
 * @return {number}
 */
var maximumRemovals = function(s, p, removable) {
  let left = 0;
  let right = removable.length;

  while (left <= right) {
    const mid = (left + right) >> 1;
    const removed = new Set(removable.slice(0, mid));
    let i = 0;
    let j = 0;

    while (i < s.length && j < p.length) {
      if (!removed.has(i) && s[i] === p[j]) {
        j++;
      }
      i++;
    }

    if (j === p.length) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return right;
};
```

1899 Merge Triplets To Form Target Triplet

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 1899. Merge Triplets to Form Target Triplet
 * https://leetcode.com/problems/merge-triplets-to-form-target-triplet/
 * Difficulty: Medium
 *
 * A triplet is an array of three integers. You are given a 2D integer array triplets, where
 * triplets[i] = [ai, bi, ci] describes the ith triplet. You are also given an integer array
 * target = [x, y, z] that describes the triplet you want to obtain.
 *
 * To obtain target, you may apply the following operation on triplets any number of times
 * (possibly zero):
 * - Choose two indices (0-indexed) i and j (i != j) and update triplets[j] to become
 *   [max(ai, aj), max(bi, bj), max(ci, cj)].
 *   - For example, if triplets[i] = [2, 5, 3] and triplets[j] = [1, 7, 5], triplets[j] will
 *     be updated to [max(2, 1), max(5, 7), max(3, 5)] = [2, 7, 5].
 *
 * Return true if it is possible to obtain the target triplet [x, y, z] as an element of
triplets,
 * or false otherwise.
 */

/**
 * @param {number[][]} triplets
 * @param {number[]} target
 * @return {boolean}
 */
var mergeTriplets = function(triplets, target) {
  let canFormX = false;
  let canFormY = false;
  let canFormZ = false;

  for (const [a, b, c] of triplets) {
    if (a <= target[0] && b <= target[1] && c <= target[2]) {
      if (a === target[0]) canFormX = true;
      if (b === target[1]) canFormY = true;
      if (c === target[2]) canFormZ = true;
    }
  }

  return canFormX && canFormY && canFormZ;
};
```

1900 The Earliest And Latest Rounds Where Players Compete

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1900. The Earliest and Latest Rounds Where Players Compete
 * https://leetcode.com/problems/the-earliest-and-latest-rounds-where-players-compete/
 * Difficulty: Hard
 *
 * There is a tournament where n players are participating. The players are standing in a
single row
 * and are numbered from 1 to n based on their initial standing position (player 1 is the
first
 * player in the row, player 2 is the second player in the row, etc.).
 *
 * The tournament consists of multiple rounds (starting from round number 1). In each round,
the ith
 * player from the front of the row competes against the ith player from the end of the row,
and the
 * winner advances to the next round. When the number of players is odd for the current round,
 the
 * player in the middle automatically advances to the next round.
 * - For example, if the row consists of players 1, 2, 4, 6, 7
 *   - Player 1 competes against player 7.
 *   - Player 2 competes against player 6.
 *   - Player 4 automatically advances to the next round.
 *
 * After each round is over, the winners are lined back up in the row based on the original
```

```
ordering
 * assigned to them initially (ascending order).
 *
 * The players numbered firstPlayer and secondPlayer are the best in the tournament. They can
win
 * against any other player before they compete against each other. If any two other players
compete
 * against each other, either of them might win, and thus you may choose the outcome of this
round.
 *
 * Given the integers n, firstPlayer, and secondPlayer, return an integer array containing two
 * values, the earliest possible round number and the latest possible round number in which
these
 * two players will compete against each other, respectively.
 */

/**
 * @param {number} n
 * @param {number} firstPlayer
 * @param {number} secondPlayer
 * @return {number[]}
 */
var earliestAndLatest = function(numPlayers, firstPlayer, secondPlayer) {
  let minRounds = Infinity;
  let maxRounds = 0;

  dfs(0, 1);

  return [minRounds, maxRounds];

  function dfs(playersEliminated, numRounds) {
    let roundResults = [playersEliminated];
    let i = 1;
    let j = numPlayers;

    while (true) {
      while (playersEliminated & (1 << i)) i++;
      while (playersEliminated & (1 << j)) j--;

      if (i >= j) break;

      if (i === firstPlayer && j === secondPlayer) {
        minRounds = Math.min(minRounds, numRounds);
        maxRounds = Math.max(maxRounds, numRounds);
        return;
      }

      const newRoundResults = [];

      if (j !== firstPlayer && j !== secondPlayer) {
        for (const roundResult of roundResults) {
          newRoundResults.push(roundResult | (1 << j));
        }
      }

      if (i !== firstPlayer && i !== secondPlayer) {
        for (const roundResult of roundResults) {
          newRoundResults.push(roundResult | (1 << i));
        }
      }

      i++;
      j--;
      roundResults = newRoundResults;
    }

    if (!roundResults.length) return;

    numRounds++;
    roundResults.forEach(roundResult => dfs(roundResult, numRounds));
  }
};
```

1901 Find A Peak Element Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1901. Find a Peak Element II
 * https://leetcode.com/problems/find-a-peak-element-ii/
 * Difficulty: Medium
 *
 * A peak element in a 2D grid is an element that is strictly greater than all of its adjacent
 * neighbors to the left, right, top, and bottom.
 *
 * Given a 0-indexed m x n matrix mat where no two adjacent cells are equal, find any peak
element
 * mat[i][j] and return the length 2 array [i,j].
 *
 * You may assume that the entire matrix is surrounded by an outer perimeter with the value -1
 in
 * each cell.
 *
 * You must write an algorithm that runs in O(m log(n)) or O(n log(m)) time.
 */

/**
 * @param {number[][]} mat
 * @return {number[]}
 */
var findPeakGrid = function(mat) {
  const rows = mat.length;
  const cols = mat[0].length;

  return binarySearch(0, rows - 1);

  function getMaxIndexInRow(row, left, right) {
    let maxIndex = left;
    for (let i = left; i <= right; i++) {
      if (mat[row][i] > mat[row][maxIndex]) {
        maxIndex = i;
      }
    }
    return maxIndex;
  }

  function binarySearch(startRow, endRow) {
    if (startRow > endRow) return null;

    const midRow = Math.floor((startRow + endRow) / 2);
    const maxCol = getMaxIndexInRow(midRow, 0, cols - 1);
    const maxValue = mat[midRow][maxCol];

    const topValue = midRow > 0 ? mat[midRow - 1][maxCol] : -1;
    const bottomValue = midRow < rows - 1 ? mat[midRow + 1][maxCol] : -1;
    const leftValue = maxCol > 0 ? mat[midRow][maxCol - 1] : -1;
    const rightValue = maxCol < cols - 1 ? mat[midRow][maxCol + 1] : -1;

    if (maxValue > topValue && maxValue > bottomValue
        && maxValue > leftValue && maxValue > rightValue) {
      return [midRow, maxCol];
    }

    if (midRow > 0 && topValue > maxValue) {
      return binarySearch(startRow, midRow - 1);
    }

    return binarySearch(midRow + 1, endRow);
  }
};
```

1902 Depth Of Bst Given Insertion Order

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1902. Depth of BST Given Insertion Order
 * https://leetcode.com/problems/depth-of-bst-given-insertion-order/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array order of length n, a permutation of integers
 * from 1 to n representing the order of insertion into a binary search tree.
 *
 * A binary search tree is defined as follows:
 * - The left subtree of a node contains only nodes with keys less than the node's key.
 * - The right subtree of a node contains only nodes with keys greater than the node's key.
 * - Both the left and right subtrees must also be binary search trees.
 *
 * The binary search tree is constructed as follows:
 * - order[0] will be the root of the binary search tree.
 * - All subsequent elements are inserted as the child of any existing node such that the
 *   binary search tree properties hold.
 *
 * Return the depth of the binary search tree.
 *
 * A binary tree's depth is the number of nodes along the longest path from the root node
 * down to the farthest leaf node.
 */

/**
 * @param {number[]} order
 * @return {number}
 */
var maxDepthBST = function(order) {
  const n = order.length;
  const parents = new Array(n + 1).fill(0);
  const insertOrders = new Array(n + 1).fill(0);

  for (let i = 0; i < n; i++) {
    insertOrders[order[i]] = i + 1;
  }

  const stack = [];

  for (let node = 0; node <= n; node++) {
    const insertOrder = insertOrders[node];

    while (stack.length > 0 && insertOrders[stack[stack.length - 1]] > insertOrder) {
      const prevNode = stack.pop();
      if (insertOrders[parents[prevNode]] < insertOrder) {
        parents[prevNode] = node;
      }
    }

    if (stack.length > 0) {
      parents[node] = stack[stack.length - 1];
    }

    stack.push(node);
  }

  const depths = new Array(n + 1).fill(0);
  for (const num of order) {
    depths[num] = depths[parents[num]] + 1;
  }

  return Math.max(...depths);
};
```

1903 Largest Odd Number In String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1903. Largest Odd Number in String
 * https://leetcode.com/problems/largest-odd-number-in-string/
 * Difficulty: Easy
 *
 * You are given a string num, representing a large integer. Return the largest-valued odd
 * integer (as a string) that is a non-empty substring of num, or an empty string "" if no
 * odd integer exists.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} num
 * @return {string}
 */
var largestOddNumber = function(num) {
  for (let i = num.length - 1; i >= 0; i--) {
    if (parseInt(num[i]) % 2 === 1) {
      return num.slice(0, i + 1);
    }
  }
  return '';
};
```

1904 The Number Of Full Rounds You Have Played

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1904. The Number of Full Rounds You Have Played
 * https://leetcode.com/problems/the-number-of-full-rounds-you-have-played/
 * Difficulty: Medium
 *
 * You are participating in an online chess tournament. There is a chess round that starts
 * every 15 minutes. The first round of the day starts at 00:00, and after every 15 minutes,
 * a new round starts.
 * - For example, the second round starts at 00:15, the fourth round starts at 00:45, and the
 *   seventh round starts at 01:30.
 *
 * You are given two strings loginTime and logoutTime where:
 * - loginTime is the time you will login to the game, and
 * - logoutTime is the time you will logout from the game.
 *
 * If logoutTime is earlier than loginTime, this means you have played from loginTime to
midnight
 * and from midnight to logoutTime.
 *
 * Return the number of full chess rounds you have played in the tournament.
 *
 * Note: All the given times follow the 24-hour clock. That means the first round of the day
starts
 * at 00:00 and the last round of the day starts at 23:45.
 */

/**
 * @param {string} loginTime
 * @param {string} logoutTime
 * @return {number}
 */
var numberOfRounds = function(loginTime, logoutTime) {
```

```
  let start = toMinutes(loginTime);
  let end = toMinutes(logoutTime);

  if (end < start) {
    end += 24 * 60;
  }

  start = Math.ceil(start / 15);
  end = Math.floor(end / 15);

  return Math.max(0, end - start);

  function toMinutes(time) {
    const [hours, minutes] = time.split(':').map(Number);
    return hours * 60 + minutes;
  }
};
```

1905 Count Sub Islands

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1905. Count Sub Islands
 * https://leetcode.com/problems/count-sub-islands/
 * Difficulty: Medium
 *
 * You are given two m x n binary matrices grid1 and grid2 containing only 0's (representing
water)
 * and 1's (representing land). An island is a group of 1's connected 4-directionally
(horizontal
 * or vertical). Any cells outside of the grid are considered water cells.
 *
 * An island in grid2 is considered a sub-island if there is an island in grid1 that contains
all
 * the cells that make up this island in grid2.
 *
 * Return the number of islands in grid2 that are considered sub-islands.
 */

/**
 * @param {number[][]} grid1
 * @param {number[][]} grid2
 * @return {number}
 */
var countSubIslands = function(grid1, grid2) {
  const rows = grid1.length;
  const cols = grid1[0].length;
  let result = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid2[i][j] === 1 && isValidSubIsland(i, j)) {
        result++;
      }
    }
  }

  return result;

  function isValidSubIsland(row, col) {
    if (row < 0 || row >= rows || col < 0 || col >= cols || grid2[row][col] === 0) {
      return true;
    }

    grid2[row][col] = 0;
    let isSubIsland = grid1[row][col] === 1;

    isSubIsland &= isValidSubIsland(row - 1, col);
    isSubIsland &= isValidSubIsland(row + 1, col);
```

```
    isSubIsland &= isValidSubIsland(row, col - 1);
    isSubIsland &= isValidSubIsland(row, col + 1);

    return isSubIsland;
  }
};
```

1906 Minimum Absolute Difference Queries

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1906. Minimum Absolute Difference Queries
 * https://leetcode.com/problems/minimum-absolute-difference-queries/
 * Difficulty: Medium
 *
 * The minimum absolute difference of an array a is defined as the minimum value of |a[i] -
a[j]|,
 * where 0 <= i < j < a.length and a[i] != a[j]. If all elements of a are the same, the
minimum
 * absolute difference is -1.
 * - For example, the minimum absolute difference of the array [5,2,3,7,2] is |2 - 3| = 1.
Note
 *   that it is not 0 because a[i] and a[j] must be different.
 *
 * You are given an integer array nums and the array queries where queries[i] = [li, ri]. For
each
 * query i, compute the minimum absolute difference of the subarray nums[li...ri] containing
the
 * elements of nums between the 0-based indices li and ri (inclusive).
 *
 * Return an array ans where ans[i] is the answer to the ith query.
 *
 * A subarray is a contiguous sequence of elements in an array.
 *
 * The value of |x| is defined as:
 * - x if x >= 0.
 * - -x if x < 0.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} queries
 * @return {number[]}
 */
var minDifference = function(nums, queries) {
  const maxValue = 100;
  const prefixCounts = new Array(nums.length + 1).fill().map(() => new Array(maxValue +
1).fill(0));

  for (let i = 0; i < nums.length; i++) {
    for (let j = 1; j <= maxValue; j++) {
      prefixCounts[i + 1][j] = prefixCounts[i][j] + (nums[i] === j ? 1 : 0);
    }
  }

  return queries.map(([start, end]) => getMinDiff(start, end));

  function getMinDiff(start, end) {
    let minDiff = Infinity;
    let prevValue = -1;

    for (let j = 1; j <= maxValue; j++) {
      if (prefixCounts[end + 1][j] - prefixCounts[start][j] > 0) {
        if (prevValue !== -1) {
          minDiff = Math.min(minDiff, j - prevValue);
        }
        prevValue = j;
      }
    }
```

```
    return minDiff === Infinity ? -1 : minDiff;
  }
};
```

1908 Game Of Nim

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1908. Game of Nim
 * https://leetcode.com/problems/game-of-nim/
 * Difficulty: Medium
 *
 * Alice and Bob take turns playing a game with Alice starting first.
 *
 * In this game, there are n piles of stones. On each player's turn, the player should remove
 * any positive number of stones from a non-empty pile of his or her choice. The first player
 * who cannot make a move loses, and the other player wins.
 *
 * Given an integer array piles, where piles[i] is the number of stones in the ith pile,
return
 * true if Alice wins, or false if Bob wins.
 *
 * Both Alice and Bob play optimally.
 */

/**
 * @param {number[]} piles
 * @return {boolean}
 */
var nimGame = function(piles) {
  const map = new Map();
  return helper(piles);

  function helper(currentPiles) {
    const key = currentPiles.join(',');
    if (map.has(key)) return map.get(key);

    if (currentPiles.every(pile => pile === 0)) {
      map.set(key, false);
      return false;
    }

    for (let i = 0; i < currentPiles.length; i++) {
      for (let take = 1; take <= currentPiles[i]; take++) {
        const nextPiles = [...currentPiles];
        nextPiles[i] -= take;

        if (!helper(nextPiles)) {
          map.set(key, true);
          return true;
        }
      }
    }

    map.set(key, false);
    return false;
  }
};
```

1909 Remove One Element To Make The Array Strictly Increasing

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1909. Remove One Element to Make the Array Strictly Increasing
 * https://leetcode.com/problems/remove-one-element-to-make-the-array-strictly-increasing/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums, return true if it can be made strictly increasing
after
 * removing exactly one element, or false otherwise. If the array is already strictly
increasing,
 * return true.
 *
 * The array nums is strictly increasing if nums[i - 1] < nums[i] for each index
 * (1 <= i < nums.length).
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var canBeIncreasing = function(nums) {
  for (let i = 0; i < nums.length; i++) {
    if (isStrictlyIncreasing(nums, i)) return true;
  }

  return false;

  function isStrictlyIncreasing(arr, skipIndex) {
    for (let i = 1; i < arr.length; i++) {
      if (i === skipIndex) continue;
      const prev = i - 1 === skipIndex ? i - 2 : i - 1;
      if (prev >= 0 && arr[i] <= arr[prev]) return false;
    }
    return true;
  }
};
```

1910 Remove All Occurrences Of A Substring

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1910. Remove All Occurrences of a Substring
 * https://leetcode.com/problems/remove-all-occurrences-of-a-substring/
 * Difficulty: Medium
 *
 * Given two strings s and part, perform the following operation on s until all occurrences of
 * the substring part are removed:
 * - Find the leftmost occurrence of the substring part and remove it from s.
 *
 * Return s after removing all occurrences of part.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} s
 * @param {string} part
 * @return {string}
 */
var removeOccurrences = function(s, part) {
  while (s.includes(part)) {
    s = s.replace(part, '');
  }
  return s;
};
```

1911 Maximum Alternating Subsequence Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1911. Maximum Alternating Subsequence Sum
 * https://leetcode.com/problems/maximum-alternating-subsequence-sum/
 * Difficulty: Medium
 *
 * The alternating sum of a 0-indexed array is defined as the sum of the elements at even
 * indices minus the sum of the elements at odd indices.
 * - For example, the alternating sum of [4,2,5,3] is (4 + 5) - (2 + 3) = 4.
 *
 * Given an array nums, return the maximum alternating sum of any subsequence of nums (after
 * reindexing the elements of the subsequence).
 *
 * A subsequence of an array is a new array generated from the original array by deleting some
 * elements (possibly none) without changing the remaining elements' relative order. For
 * example, [2,7,4] is a subsequence of [4,2,3,7,2,1,4] (the underlined elements),
 * while [2,4,2] is not.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxAlternatingSum = function(nums) {
  let result = 0;
  let oddSum = 0;

  for (const num of nums) {
    const prevEven = result;
    result = Math.max(result, oddSum + num);
    oddSum = Math.max(oddSum, prevEven - num);
  }

  return result;
};
```

1912 Design Movie Rental System

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1912. Design Movie Rental System
 * https://leetcode.com/problems/design-movie-rental-system/
 * Difficulty: Hard
 *
 * You have a movie renting company consisting of n shops. You want to implement a renting
system
 * that supports searching for, booking, and returning movies. The system should also support
 * generating a report of the currently rented movies.
 *
 * Each movie is given as a 2D integer array entries where entries[i] = [shopi, moviei,
pricei]
 * indicates that there is a copy of movie moviei at shop shopi with a rental price of pricei.
 * Each shop carries at most one copy of a movie moviei.
 *
 * The system should support the following functions:
 * - Search: Finds the cheapest 5 shops that have an unrented copy of a given movie. The shops
 *   should be sorted by price in ascending order, and in case of a tie, the one with the
smaller
 *   shopi should appear first. If there are less than 5 matching shops, then all of them
should be
 *   returned. If no shop has an unrented copy, then an empty list should be returned.
```

```
 * - Rent: Rents an unrented copy of a given movie from a given shop.
 * - Drop: Drops off a previously rented copy of a given movie at a given shop.
 * - Report: Returns the cheapest 5 rented movies (possibly of the same movie ID) as a 2D list
 res
 *   where res[j] = [shopj, moviej] describes that the jth cheapest rented movie moviej was
rented
 *   from the shop shopj. The movies in res should be sorted by price in ascending order, and
in
 *   case of a tie, the one with the smaller shopj should appear first, and if there is still
tie,
 *   the one with the smaller moviej should appear first. If there are fewer than 5 rented
movies,
 *   then all of them should be returned. If no movies are currently being rented, then an
empty
 *   list should be returned.
 *
 * Implement the MovieRentingSystem class:
 * - MovieRentingSystem(int n, int[][] entries) Initializes the MovieRentingSystem object with
 n
 *   shops and the movies in entries.
 * - List<Integer> search(int movie) Returns a list of shops that have an unrented copy of the
 *   given movie as described above.
 * - void rent(int shop, int movie) Rents the given movie from the given shop.
 * - void drop(int shop, int movie) Drops off a previously rented movie at the given shop.
 * - List<List<Integer>> report() Returns a list of cheapest rented movies as described above.
 *
 * Note: The test cases will be generated such that rent will only be called if the shop has
an
 * unrented copy of the movie, and drop will only be called if the shop had previously rented
out
 * the movie.
 */

/**
 * @param {number} n
 * @param {number[][]} entries
 */
var MovieRentingSystem = function(n, entries) {
  this.shopMovies = new Map();
  this.movieShops = new Map();
  this.rented = new PriorityQueue((a, b) => {
    if (a[2] !== b[2]) return a[2] - b[2];
    if (a[0] !== b[0]) return a[0] - b[0];
    return a[1] - b[1];
  });
  this.rentedLookup = new Map();

  for (const [shop, movie, price] of entries) {
    if (!this.shopMovies.has(shop)) {
      this.shopMovies.set(shop, new Map());
    }
    this.shopMovies.get(shop).set(movie, price);

    if (!this.movieShops.has(movie)) {
      this.movieShops.set(movie, new PriorityQueue((a, b) => {
        if (a[1] !== b[1]) return a[1] - b[1];
        return a[0] - b[0];
      }));
    }
    this.movieShops.get(movie).enqueue([shop, price]);
  }
};

/**
 * @param {number} movie
 * @return {number[]}
 */
MovieRentingSystem.prototype.search = function(movie) {
  if (!this.movieShops.has(movie)) return [];

  const queue = this.movieShops.get(movie);
  const result = [];
  const seen = new Set();

  while (!queue.isEmpty() && result.length < 5) {
```

```
    const [shop, price] = queue.dequeue();
    if (this.shopMovies.get(shop)?.has(movie) && !seen.has(shop)) {
      result.push(shop);
      seen.add(shop);
    }
  }

  for (const shop of result) {
    queue.enqueue([shop, this.shopMovies.get(shop).get(movie)]);
  }

  return result;
};

/**
 * @param {number} shop
 * @param {number} movie
 * @return {void}
 */
MovieRentingSystem.prototype.rent = function(shop, movie) {
  const price = this.shopMovies.get(shop).get(movie);
  this.shopMovies.get(shop).delete(movie);
  this.rented.enqueue([shop, movie, price]);
  this.rentedLookup.set(`${shop}:${movie}`, price);
};

/**
 * @param {number} shop
 * @param {number} movie
 * @return {void}
 */
MovieRentingSystem.prototype.drop = function(shop, movie) {
  const price = this.rentedLookup.get(`${shop}:${movie}`);
  this.rentedLookup.delete(`${shop}:${movie}`);
  if (!this.shopMovies.get(shop).has(movie)) {
    this.shopMovies.get(shop).set(movie, price);
    this.movieShops.get(movie).enqueue([shop, price]);
  }
};

/**
 * @return {number[][]}
 */
MovieRentingSystem.prototype.report = function() {
  const result = [];
  const seen = new Set();

  while (!this.rented.isEmpty() && result.length < 5) {
    const [shop, movie, price] = this.rented.dequeue();
    const key = `${shop}:${movie}`;
    if (this.rentedLookup.has(key) && !seen.has(key)) {
      result.push([shop, movie]);
      seen.add(key);
    }
  }

  for (const [shop, movie] of result) {
    const price = this.rentedLookup.get(`${shop}:${movie}`);
    this.rented.enqueue([shop, movie, price]);
  }

  return result;
};
```

1913 Maximum Product Difference Between Two Pairs

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1913. Maximum Product Difference Between Two Pairs
```

```
 * https://leetcode.com/problems/maximum-product-difference-between-two-pairs/
 * Difficulty: Easy
 *
 * The product difference between two pairs (a, b) and (c, d) is defined as (a * b) - (c * d).
 * - For example, the product difference between (5, 6) and (2, 7) is (5 * 6) - (2 * 7) = 16.
 *
 * Given an integer array nums, choose four distinct indices w, x, y, and z such that the
product
 * difference between pairs (nums[w], nums[x]) and (nums[y], nums[z]) is maximized.
 *
 * Return the maximum such product difference.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxProductDifference = function(nums) {
  let max1 = 0;
  let max2 = 0;
  let min1 = Infinity;
  let min2 = Infinity;

  for (const num of nums) {
    if (num > max1) {
      max2 = max1;
      max1 = num;
    } else if (num > max2) {
      max2 = num;
    }

    if (num < min1) {
      min2 = min1;
      min1 = num;
    } else if (num < min2) {
      min2 = num;
    }
  }

  return max1 * max2 - min1 * min2;
};
```

1914 Cyclically Rotating A Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1914. Cyclically Rotating a Grid
 * https://leetcode.com/problems/cyclically-rotating-a-grid/
 * Difficulty: Medium
 *
 * You are given an m x n integer matrix grid, where m and n are both even integers, and
 * an integer k.
 *
 * The matrix is composed of several layers, which is shown in the below image, where each
 * color is its own layer.
 *
 * A cyclic rotation of the matrix is done by cyclically rotating each layer in the matrix.
 * To cyclically rotate a layer once, each element in the layer will take the place of the
 * adjacent element in the counter-clockwise direction. An example rotation is shown below.
 */

/**
 * @param {number[][]} grid
 * @param {number} k
 * @return {number[][]}
 */
var rotateGrid = function(grid, k) {
  const rows = grid.length;
  const cols = grid[0].length;
```

```
  const result = grid.map(row => [...row]);

  for (let layer = 0; layer < Math.min(rows, cols) / 2; layer++) {
    rotateLayer(layer, layer, rows - 1 - layer, cols - 1 - layer, k);
  }

  return result;

  function rotateLayer(top, left, bottom, right, rotations) {
    if (top >= bottom || left >= right) return;

    const perimeter = 2 * (bottom - top + right - left);
    const effectiveRotations = rotations % perimeter;
    if (effectiveRotations === 0) return;

    const elements = [];
    for (let i = top; i <= bottom; i++) elements.push(grid[i][left]);
    for (let j = left + 1; j <= right; j++) elements.push(grid[bottom][j]);
    for (let i = bottom - 1; i >= top; i--) elements.push(grid[i][right]);
    for (let j = right - 1; j > left; j--) elements.push(grid[top][j]);

    const offset = (perimeter - effectiveRotations) % perimeter;
    let index = offset;

    for (let i = top; i <= bottom; i++) {
      result[i][left] = elements[index % perimeter];
      index++;
    }
    for (let j = left + 1; j <= right; j++) {
      result[bottom][j] = elements[index % perimeter];
      index++;
    }
    for (let i = bottom - 1; i >= top; i--) {
      result[i][right] = elements[index % perimeter];
      index++;
    }
    for (let j = right - 1; j > left; j--) {
      result[top][j] = elements[index % perimeter];
      index++;
    }
  }
};
```

1915 Number Of Wonderful Substrings

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1915. Number of Wonderful Substrings
 * https://leetcode.com/problems/number-of-wonderful-substrings/
 * Difficulty: Medium
 *
 * A wonderful string is a string where at most one letter appears an odd number of times.
 * - For example, "ccjjc" and "abab" are wonderful, but "ab" is not.
 *
 * Given a string word that consists of the first ten lowercase English letters ('a' through
 * 'j'), return the number of wonderful non-empty substrings in word. If the same substring
 * appears multiple times in word, then count each occurrence separately.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} word
 * @return {number}
 */
var wonderfulSubstrings = function(word) {
  let count = 0;
  const freq = new Map([[0, 1]]);
  let state = 0;
```

```
  for (const char of word) {
    const bit = 1 << (char.charCodeAt(0) - 97);
    state ^= bit;

    count += freq.get(state) || 0;
    freq.set(state, (freq.get(state) || 0) + 1);

    for (let i = 0; i < 10; i++) {
      const oddBit = state ^ (1 << i);
      count += freq.get(oddBit) || 0;
    }
  }

  return count;
};
```

1916 Count Ways To Build Rooms In An Ant Colony

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1916. Count Ways to Build Rooms in an Ant Colony
 * https://leetcode.com/problems/count-ways-to-build-rooms-in-an-ant-colony/
 * Difficulty: Hard
 *
 * You are an ant tasked with adding n new rooms numbered 0 to n-1 to your colony. You are
given
 * the expansion plan as a 0-indexed integer array of length n, prevRoom, where prevRoom[i]
 * indicates that you must build room prevRoom[i] before building room i, and these two rooms
 * must be connected directly. Room 0 is already built, so prevRoom[0] = -1. The expansion
plan
 * is given such that once all the rooms are built, every room will be reachable from room 0.
 *
 * You can only build one room at a time, and you can travel freely between rooms you have
 * already built only if they are connected. You can choose to build any room as long as its
 * previous room is already built.
 *
 * Return the number of different orders you can build all the rooms in. Since the answer may
 * be large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} prevRoom
 * @return {number}
 */
var waysToBuildRooms = function(prevRoom) {
  const n = prevRoom.length;
  const mod = 1e9 + 7;
  const graph = Array(n).fill().map(() => []);
  const factorial = Array(n + 1).fill(1n);
  const invFactorial = Array(n + 1).fill(1n);

  for (let i = 1; i <= n; i++) {
    factorial[i] = (factorial[i - 1] * BigInt(i)) % BigInt(mod);
  }

  for (let i = 1; i <= n; i++) {
    invFactorial[i] = modInverse(factorial[i], BigInt(mod));
  }

  for (let i = 1; i < n; i++) {
    graph[prevRoom[i]].push(i);
  }

  return Number(countSubtreeSizes(0)[1]);

  function countSubtreeSizes(node) {
    let size = 1;
    let result = 1n;
```

```
    for (const child of graph[node]) {
      const [childSize, childResult] = countSubtreeSizes(child);
      size += childSize;
      result = (result * childResult * invFactorial[childSize]) % BigInt(mod);
    }

    return [size, (result * factorial[size - 1]) % BigInt(mod)];
  }

  function modInverse(a, m) {
    const m0 = m;
    let q;
    let x0 = 0n;
    let x1 = 1n;

    while (a > 1n) {
      q = a / m;
      [a, m] = [m, a % m];
      [x0, x1] = [x1 - q * x0, x0];
    }
    return x1 < 0n ? x1 + m0 : x1;
  }
};
```

1918 Kth Smallest Subarray Sum

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1918. Kth Smallest Subarray Sum
 * https://leetcode.com/problems/kth-smallest-subarray-sum/
 * Difficulty: Medium
 *
 * Given an integer array nums of length n and an integer k, return the kth smallest subarray
sum.
 *
 * A subarray is defined as a non-empty contiguous sequence of elements in an array. A
subarray
 * sum is the sum of all elements in the subarray.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var kthSmallestSubarraySum = function(nums, k) {
  let result = Math.min(...nums);
  let maxSum = nums.reduce((sum, num) => sum + num, 0);

  while (result < maxSum) {
    const midSum = Math.floor((result + maxSum) / 2);

    if (countSubarraysWithSumLessOrEqual(midSum) < k) {
      result = midSum + 1;
    } else {
      maxSum = midSum;
    }
  }

  return result;

  function countSubarraysWithSumLessOrEqual(target) {
    let count = 0;
    let left = 0;
    let currentSum = 0;

    for (let right = 0; right < nums.length; right++) {
      currentSum += nums[right];
```

```
      while (currentSum > target) {
        currentSum -= nums[left];
        left++;
      }

      count += right - left + 1;
    }

    return count;
  }
};
```

1920 Build Array From Permutation

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1920. Build Array from Permutation
 * https://leetcode.com/problems/build-array-from-permutation/
 * Difficulty: Easy
 *
 * Given a zero-based permutation nums (0-indexed), build an array ans of the same length
 * where ans[i] = nums[nums[i]] for each 0 <= i < nums.length and return it.
 *
 * A zero-based permutation nums is an array of distinct integers from 0 to nums.length - 1
 * (inclusive).
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var buildArray = function(nums) {
  return nums.map(n => nums[n]);
};
```

1921 Eliminate Maximum Number Of Monsters

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1921. Eliminate Maximum Number of Monsters
 * https://leetcode.com/problems/eliminate-maximum-number-of-monsters/
 * Difficulty: Medium
 *
 * You are playing a video game where you are defending your city from a group of n monsters.
You
 * are given a 0-indexed integer array dist of size n, where dist[i] is the initial distance
in
 * kilometers of the ith monster from the city.
 *
 * The monsters walk toward the city at a constant speed. The speed of each monster is given
to
 * you in an integer array speed of size n, where speed[i] is the speed of the ith monster in
 * kilometers per minute.
 *
 * You have a weapon that, once fully charged, can eliminate a single monster. However, the
weapon
 * takes one minute to charge. The weapon is fully charged at the very start.
 *
 * You lose when any monster reaches your city. If a monster reaches the city at the exact
moment
 * the weapon is fully charged, it counts as a loss, and the game ends before you can use your
 * weapon.
```

```
 *
 * Return the maximum number of monsters that you can eliminate before you lose, or n if you
can
 * eliminate all the monsters before they reach the city.
 */

/**
 * @param {number[]} dist
 * @param {number[]} speed
 * @return {number}
 */
var eliminateMaximum = function(dist, speed) {
  const n = dist.length;
  const arrivalTimes = new Array(n);

  for (let i = 0; i < n; i++) {
    arrivalTimes[i] = dist[i] / speed[i];
  }

  arrivalTimes.sort((a, b) => a - b);

  for (let i = 0; i < n; i++) {
    if (arrivalTimes[i] <= i) {
      return i;
    }
  }

  return n;
};
```

1922 Count Good Numbers

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1922. Count Good Numbers
 * https://leetcode.com/problems/count-good-numbers/
 * Difficulty: Medium
 *
 * A digit string is good if the digits (0-indexed) at even indices are even and the digits
 * at odd indices are prime (2, 3, 5, or 7).
 *
 * For example, "2582" is good because the digits (2 and 8) at even positions are even and
 * the digits (5 and 2) at odd positions are prime. However, "3245" is not good because 3
 * is at an even index but is not even.
 *
 * Given an integer n, return the total number of good digit strings of length n. Since the
 * answer may be large, return it modulo 109 + 7.
 *
 * A digit string is a string consisting of digits 0 through 9 that may contain leading zeros.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countGoodNumbers = function(n) {
  const MOD = 1e9 + 7;
  const evenCount = 5;
  const primeCount = 4;
  const evenPositions = Math.ceil(n / 2);
  const oddPositions = Math.floor(n / 2);
  const evenResult = power(evenCount, evenPositions);
  const oddResult = power(primeCount, oddPositions);

  return Number(BigInt(evenResult) * BigInt(oddResult) % BigInt(MOD));

  function power(base, exponent) {
    if (exponent === 0) return 1;
    let half = power(base, Math.floor(exponent / 2));
```

```
    half = BigInt(half) * BigInt(half) % BigInt(MOD);
    if (exponent % 2) half = half * BigInt(base) % BigInt(MOD);
    return Number(half);
  }
};
```

1923 Longest Common Subpath

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1923. Longest Common Subpath
 * https://leetcode.com/problems/longest-common-subpath/
 * Difficulty: Hard
 *
 * There is a country of n cities numbered from 0 to n - 1. In this country, there is a road
 * connecting every pair of cities.
 *
 * There are m friends numbered from 0 to m - 1 who are traveling through the country. Each
one
 * of them will take a path consisting of some cities. Each path is represented by an integer
 * array that contains the visited cities in order. The path may contain a city more than
once,
 * but the same city will not be listed consecutively.
 *
 * Given an integer n and a 2D integer array paths where paths[i] is an integer array
representing
 * the path of the ith friend, return the length of the longest common subpath that is shared
 * by every friend's path, or 0 if there is no common subpath at all.
 *
 * A subpath of a path is a contiguous sequence of cities within that path.
 */

/**
 * @param {number} n
 * @param {number[][]} paths
 * @return {number}
 */
var longestCommonSubpath = function(n, paths) {
  if (paths.length === 0) return 0;

  paths.sort((a, b) => a.length - b.length);

  let left = 0;
  let right = paths[0].length;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);

    if (checkCommonSubpathExists(paths, mid)) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return right;
};

function checkCommonSubpathExists(paths, length) {
  if (length === 0) return true;

  const firstPath = paths[0];

  if (firstPath.length < length) return false;

  const prime = 1000000007;
  const base = 100003;
```

```
let highestPower = 1;
for (let i = 0; i < length - 1; i++) {
  highestPower = (highestPower * base) % prime;
}

let hashToPositions = new Map();

let hash = 0;
for (let i = 0; i < length; i++) {
  hash = (hash * base + firstPath[i]) % prime;
}

if (!hashToPositions.has(hash)) {
  hashToPositions.set(hash, []);
}
hashToPositions.get(hash).push(0);

for (let i = 1; i <= firstPath.length - length; i++) {
  hash = ((hash - firstPath[i - 1] * highestPower % prime + prime)
         % prime * base + firstPath[i + length - 1]) % prime;

  if (!hashToPositions.has(hash)) {
    hashToPositions.set(hash, []);
  }
  hashToPositions.get(hash).push(i);
}

for (let pathIdx = 1; pathIdx < paths.length; pathIdx++) {
  const path = paths[pathIdx];

  if (path.length < length) return false;

  const newHashToPositions = new Map();
  hash = 0;
  for (let i = 0; i < length; i++) {
    hash = (hash * base + path[i]) % prime;
  }

  if (hashToPositions.has(hash)) {
    const positions = hashToPositions.get(hash);
    for (const pos of positions) {
      let isMatch = true;
      for (let j = 0; j < length; j++) {
        if (firstPath[pos + j] !== path[j]) {
          isMatch = false;
          break;
        }
      }

      if (isMatch) {
        if (!newHashToPositions.has(hash)) {
          newHashToPositions.set(hash, []);
        }
        newHashToPositions.get(hash).push(pos);
        break;
      }
    }
  }

  for (let i = 1; i <= path.length - length; i++) {
    hash = ((hash - path[i - 1] * highestPower % prime + prime)
           % prime * base + path[i + length - 1]) % prime;

    if (hashToPositions.has(hash)) {
      const positions = hashToPositions.get(hash);
      for (const pos of positions) {
        let isMatch = true;
        for (let j = 0; j < length; j++) {
          if (firstPath[pos + j] !== path[i + j]) {
            isMatch = false;
            break;
          }
        }

        if (isMatch) {
```

```
            if (!newHashToPositions.has(hash)) {
              newHashToPositions.set(hash, []);
            }
            newHashToPositions.get(hash).push(pos);
            break;
          }
        }
      }
    }

    if (newHashToPositions.size === 0) return false;

    hashToPositions = newHashToPositions;
  }

  return hashToPositions.size > 0;
}
```

1924 Erect The Fence Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1924. Erect the Fence II
 * https://leetcode.com/problems/erect-the-fence-ii/
 * Difficulty: Hard
 *
 * You are given a 2D integer array trees where trees[i] = [xi, yi] represents the location
 * of the ith tree in the garden.
 *
 * You are asked to fence the entire garden using the minimum length of rope possible. The
garden
 * is well-fenced only if all the trees are enclosed and the rope used forms a perfect circle.
 * A tree is considered enclosed if it is inside or on the border of the circle.
 *
 * More formally, you must form a circle using the rope with a center (x, y) and radius r
where
 * all trees lie inside or on the circle and r is minimum.
 *
 * Return the center and radius of the circle as a length 3 array [x, y, r]. Answers within
10-5
 * of the actual answer will be accepted.
 */

/**
 * @param {number[][]} trees
 * @return {number[]}
 */
var outerTrees = function(trees) {
  const n = trees.length;

  if (n === 1) {
    return [trees[0][0], trees[0][1], 0];
  }

  const shuffled = [...trees];
  for (let i = shuffled.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]];
  }

  return getMinimumEnclosingCircle(shuffled, []);

  function distance(p1, p2) {
    return Math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2);
  }

  function getCircleFromTwoPoints(p1, p2) {
    const x = (p1[0] + p2[0]) / 2;
    const y = (p1[1] + p2[1]) / 2;
```

```
    const r = distance(p1, p2) / 2;
    return [x, y, r];
  }

  function getCircleFromThreePoints(p1, p2, p3) {
    const [x1, y1] = p1;
    const [x2, y2] = p2;
    const [x3, y3] = p3;

    const d = 2 * (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2));

    if (Math.abs(d) < 1e-10) {
      return null;
    }

    const ux = ((x1 ** 2 + y1 ** 2) * (y2 - y3) + (x2 ** 2 + y2 ** 2) * (y3 - y1)
      + (x3 ** 2 + y3 ** 2) * (y1 - y2)) / d;
    const uy = ((x1 ** 2 + y1 ** 2) * (x3 - x2) + (x2 ** 2 + y2 ** 2) * (x1 - x3)
      + (x3 ** 2 + y3 ** 2) * (x2 - x1)) / d;
    const r = distance([ux, uy], p1);

    return [ux, uy, r];
  }

  function isInsideCircle(point, circle) {
    const [x, y, r] = circle;
    return distance(point, [x, y]) <= r + 1e-7;
  }

  function getMinimumEnclosingCircle(points, boundary) {
    if (boundary.length === 3 || points.length === 0) {
      if (boundary.length === 0) return [0, 0, 0];
      if (boundary.length === 1) return [boundary[0][0], boundary[0][1], 0];
      if (boundary.length === 2) return getCircleFromTwoPoints(boundary[0], boundary[1]);
      return getCircleFromThreePoints(boundary[0], boundary[1], boundary[2]);
    }

    const point = points[0];
    const circle = getMinimumEnclosingCircle(points.slice(1), boundary);
    if (isInsideCircle(point, circle)) {
      return circle;
    }

    return getMinimumEnclosingCircle(points.slice(1), boundary.concat([point]));
  }
};
```

1925 Count Square Sum Triples

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1925. Count Square Sum Triples
 * https://leetcode.com/problems/count-square-sum-triples/
 * Difficulty: Easy
 *
 * A square triple (a,b,c) is a triple where a, b, and c are integers and a2 + b2 = c2.
 *
 * Given an integer n, return the number of square triples such that 1 <= a, b, c <= n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countTriples = function(n) {
  let result = 0;

  for (let a = 1; a <= n; a++) {
    for (let b = 1; b <= n; b++) {
```

```
      const sum = a * a + b * b;
      const c = Math.sqrt(sum);
      if (c <= n && Number.isInteger(c)) {
        result++;
      }
    }
  }

  return result;
};
```

1926 Nearest Exit From Entrance In Maze

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1926. Nearest Exit from Entrance in Maze
 * https://leetcode.com/problems/nearest-exit-from-entrance-in-maze/
 * Difficulty: Medium
 *
 * You are given an m x n matrix maze (0-indexed) with empty cells (represented as '.') and
 * walls (represented as '+'). You are also given the entrance of the maze, where entrance
 * = [entrancerow, entrancecol] denotes the row and column of the cell you are initially
 * standing at.
 *
 * In one step, you can move one cell up, down, left, or right. You cannot step into a cell
 * with a wall, and you cannot step outside the maze. Your goal is to find the nearest exit
 * from the entrance. An exit is defined as an empty cell that is at the border of the maze.
 * The entrance does not count as an exit.
 *
 * Return the number of steps in the shortest path from the entrance to the nearest exit,
 * or -1 if no such path exists.
 */

/**
 * @param {character[][]} maze
 * @param {number[]} entrance
 * @return {number}
 */
var nearestExit = function(maze, entrance) {
  const queue = [[entrance, 0]];

  while (queue.length) {
    const [cell, steps] = queue.shift();
    const [i, j] = cell;
    if (i === maze.length || i === -1 || j === maze[0].length || j === -1 || maze[i][j] !==
'.') {
      continue;
    }
    if ((i === maze.length - 1 || i === 0 || j === maze[0].length - 1 || j === 0) && steps !==
 0) {
      return steps;
    }
    maze[i][j] = '*';
    queue.push([[i, j + 1], steps + 1]);
    queue.push([[i, j - 1], steps + 1]);
    queue.push([[i + 1, j], steps + 1]);
    queue.push([[i - 1, j], steps + 1]);
  }

  return -1;
};
```

1927 Sum Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1927. Sum Game
 * https://leetcode.com/problems/sum-game/
 * Difficulty: Medium
 *
 * Alice and Bob take turns playing a game, with Alice starting first.
 *
 * You are given a string num of even length consisting of digits and '?' characters. On each
 * turn, a player will do the following if there is still at least one '?' in num:
 * 1. Choose an index i where num[i] == '?'.
 * 2. Replace num[i] with any digit between '0' and '9'.
 *
 * The game ends when there are no more '?' characters in num.
 *
 * For Bob to win, the sum of the digits in the first half of num must be equal to the sum of
 * the digits in the second half. For Alice to win, the sums must not be equal.
 * - For example, if the game ended with num = "243801", then Bob wins because 2+4+3 = 8+0+1.
 *   If the game ended with num = "243803", then Alice wins because 2+4+3 != 8+0+3.
 *
 * Assuming Alice and Bob play optimally, return true if Alice will win and false if Bob will
win.
 */

/**
 * @param {string} num
 * @return {boolean}
 */
var sumGame = function(num) {
  const n = num.length;
  const half = n / 2;

  let leftSum = 0;
  let rightSum = 0;
  let leftQuestionMarks = 0;
  let rightQuestionMarks = 0;

  for (let i = 0; i < n; i++) {
    if (i < half) {
      if (num[i] === '?') {
        leftQuestionMarks++;
      } else {
        leftSum += parseInt(num[i]);
      }
    } else {
      if (num[i] === '?') {
        rightQuestionMarks++;
      } else {
        rightSum += parseInt(num[i]);
      }
    }
  }

  const sumDiff = leftSum - rightSum;
  const diff = leftQuestionMarks - rightQuestionMarks;

  return (diff % 2 !== 0) || (sumDiff + diff * 4.5 !== 0);
};
```

1928 Minimum Cost To Reach Destination In Time

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1928. Minimum Cost to Reach Destination in Time
 * https://leetcode.com/problems/minimum-cost-to-reach-destination-in-time/
 * Difficulty: Hard
 *
```

```
 * There is a country of n cities numbered from 0 to n - 1 where all the cities are connected
by
 * bi-directional roads. The roads are represented as a 2D integer array edges where
 * edges[i] = [xi, yi, timei] denotes a road between cities xi and yi that takes timei minutes
 * to travel. There may be multiple roads of differing travel times connecting the same two
 * cities, but no road connects a city to itself.
 *
 * Each time you pass through a city, you must pay a passing fee. This is represented as a
 * 0-indexed integer array passingFees of length n where passingFees[j] is the amount of
dollars
 * you must pay when you pass through city j.
 *
 * In the beginning, you are at city 0 and want to reach city n - 1 in maxTime minutes or
less.
 * The cost of your journey is the summation of passing fees for each city that you passed
 * through at some moment of your journey (including the source and destination cities).
 *
 * Given maxTime, edges, and passingFees, return the minimum cost to complete your journey,
 * or -1 if you cannot complete it within maxTime minutes.
 */

/**
 * @param {number} maxTime
 * @param {number[][]} edges
 * @param {number[]} passingFees
 * @return {number}
 */
var minCost = function(maxTime, edges, passingFees) {
  const n = passingFees.length;
  const dp = new Array(n).fill().map(() => new Array(maxTime + 1).fill(Infinity));
  dp[0][0] = passingFees[0];

  const graph = new Array(n).fill().map(() => []);
  for (const [u, v, time] of edges) {
    graph[u].push([v, time]);
    graph[v].push([u, time]);
  }

  const pq = new PriorityQueue((a, b) => a.cost - b.cost);
  pq.enqueue({ cost: passingFees[0], city: 0, time: 0 });

  while (!pq.isEmpty()) {
    const { cost, city, time } = pq.dequeue();
    if (cost > dp[city][time]) continue;

    for (const [nextCity, travelTime] of graph[city]) {
      const newTime = time + travelTime;
      if (newTime > maxTime) continue;

      const newCost = cost + passingFees[nextCity];
      if (newCost < dp[nextCity][newTime]) {
        dp[nextCity][newTime] = newCost;
        pq.enqueue({ cost: newCost, city: nextCity, time: newTime });
      }
    }
  }

  let minCost = Infinity;
  for (let t = 0; t <= maxTime; t++) {
    minCost = Math.min(minCost, dp[n - 1][t]);
  }

  return minCost === Infinity ? -1 : minCost;
};
```

1929 Concatenation Of Array

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
```

```
 * 1929. Concatenation of Array
 * https://leetcode.com/problems/concatenation-of-array/
 * Difficulty: Easy
 *
 * Given an integer array `nums` of length `n`, you want to create an
 * array `ans` of length `2n` where `ans[i] == nums[i`] and
 * `ans[i + n] == nums[i]` for `0 <= i < n` (0-indexed).
 *
 * Specifically, `ans` is the concatenation of two `nums` arrays.
 *
 * Return the array `ans`.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var getConcatenation = function(nums) {
  return [...nums, ...nums];
};
```

1930 Unique Length 3 Palindromic Subsequences

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1930. Unique Length-3 Palindromic Subsequences
 * https://leetcode.com/problems/unique-length-3-palindromic-subsequences/
 * Difficulty: Medium
 *
 * Given a string s, return the number of unique palindromes of length three that are a
 * subsequence of s.
 *
 * Note that even if there are multiple ways to obtain the same subsequence, it is still
 * only counted once.
 *
 * A palindrome is a string that reads the same forwards and backwards.
 *
 * A subsequence of a string is a new string generated from the original string with some
 * characters (can be none) deleted without changing the relative order of the remaining
 * characters.
 *
 * For example, "ace" is a subsequence of "abcde".
 */

/**
 * @param {string} s
 * @return {number}
 */
var countPalindromicSubsequence = function(s) {
  let result = 0;

  for (let i = 0; i < 26; ++i) {
    const char = String.fromCharCode(i + 97);
    const left = s.indexOf(char);
    const right = s.lastIndexOf(char);
    if (left !== -1 && right !== -1 && left < right) {
      result += new Set(s.substring(left + 1, right)).size;
    }
  }

  return result;
};
```

1931 Painting A Grid With Three Different Colors

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1931. Painting a Grid With Three Different Colors
 * https://leetcode.com/problems/painting-a-grid-with-three-different-colors/
 * Difficulty: Hard
 *
 * You are given two integers m and n. Consider an m x n grid where each cell is initially
white.
 * You can paint each cell red, green, or blue. All cells must be painted.
 *
 * Return the number of ways to color the grid with no two adjacent cells having the same
color.
 * Since the answer can be very large, return it modulo 109 + 7.
 */

/**
 * @param {number} m
 * @param {number} n
 * @return {number}
 */
var colorTheGrid = function(m, n) {
  const MOD = 1e9 + 7;
  const colors = 3;
  const validStates = generateValidStates(m, colors);
  const stateCount = validStates.length;
  let dp = new Array(stateCount).fill(1);

  for (let col = 1; col < n; col++) {
    const nextDp = new Array(stateCount).fill(0);
    for (let i = 0; i < stateCount; i++) {
      for (let j = 0; j < stateCount; j++) {
        if (isCompatible(validStates[i], validStates[j])) {
          nextDp[j] = (nextDp[j] + dp[i]) % MOD;
        }
      }
    }
    dp = nextDp;
  }

  let totalWays = 0;
  for (const count of dp) {
    totalWays = (totalWays + count) % MOD;
  }

  return totalWays;

  function generateValidStates(rows, colors) {
    const states = [];
    generateStates([], rows, colors, states);
    return states;
  }

  function generateStates(current, rows, colors, states) {
    if (current.length === rows) {
      states.push([...current]);
      return;
    }
    for (let c = 0; c < colors; c++) {
      if (current.length === 0 || current[current.length - 1] !== c) {
        current.push(c);
        generateStates(current, rows, colors, states);
        current.pop();
      }
    }
  }

  function isCompatible(prevState, currState) {
```

```
    for (let i = 0; i < prevState.length; i++) {
      if (prevState[i] === currState[i]) {
        return false;
      }
    }
    return true;
  }
};
```

1932 Merge Bsts To Create Single Bst

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1932. Merge BSTs to Create Single BST
 * https://leetcode.com/problems/merge-bsts-to-create-single-bst/
 * Difficulty: Hard
 *
 * You are given n BST (binary search tree) root nodes for n separate BSTs stored in an array
 * trees (0-indexed). Each BST in trees has at most 3 nodes, and no two roots have the same
 * value. In one operation, you can:
 * - Select two distinct indices i and j such that the value stored at one of the leaves of
 *   trees[i] is equal to the root value of trees[j].
 * - Replace the leaf node in trees[i] with trees[j].
 * - Remove trees[j] from trees.
 *
 * Return the root of the resulting BST if it is possible to form a valid BST after
 * performing n - 1 operations, or null if it is impossible to create a valid BST.
 *
 * A BST (binary search tree) is a binary tree where each node satisfies the following
property:
 * - Every node in the node's left subtree has a value strictly less than the node's value.
 * - Every node in the node's right subtree has a value strictly greater than the node's
value.
 *
 * A leaf is a node that has no children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode[]} trees
 * @return {TreeNode}
 */
var canMerge = function(trees) {
  const valueToTree = new Map();
  const leafValues = new Set();
  const inDegree = new Map();

  for (const tree of trees) {
    valueToTree.set(tree.val, tree);
    if (tree.left) leafValues.add(tree.left.val);
    if (tree.right) leafValues.add(tree.right.val);
  }

  let root = null;
  for (const tree of trees) {
    if (!leafValues.has(tree.val)) {
      if (root) return null;
      root = tree;
    }
    if (tree.left) inDegree.set(tree.left.val, (inDegree.get(tree.left.val) || 0) + 1);
    if (tree.right) inDegree.set(tree.right.val, (inDegree.get(tree.right.val) || 0) + 1);
  }
```

```
  if (!root) return null;

  const merged = mergeTrees(root, valueToTree, inDegree);
  if (!merged || valueToTree.size > 1 || !isValidBST(merged, -Infinity, Infinity)) {
    return null;
  }

  return merged;

  function mergeTrees(node, valueToTree, inDegree) {
    if (!node) return node;

    if (!node.left && !node.right && valueToTree.has(node.val) && inDegree.get(node.val) ===
1) {
      const tree = valueToTree.get(node.val);
      if (tree === node) return node;
      valueToTree.delete(node.val);
      return mergeTrees(tree, valueToTree, inDegree);
    }

    node.left = mergeTrees(node.left, valueToTree, inDegree);
    node.right = mergeTrees(node.right, valueToTree, inDegree);
    return node;
  }

  function isValidBST(node, min, max) {
    if (!node) return true;
    if (node.val <= min || node.val >= max) return false;
    return isValidBST(node.left, min, node.val) && isValidBST(node.right, node.val, max);
  }
};
```

1933 Check If String Is Decomposable Into Value Equal Substrings

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1933. Check if String Is Decomposable Into Value-Equal Substrings
 * https://leetcode.com/problems/check-if-string-is-decomposable-into-value-equal-substrings/
 * Difficulty: Easy
 *
 * A value-equal string is a string where all characters are the same.
 * - For example, "1111" and "33" are value-equal strings.
 * - In contrast, "123" is not a value-equal string.
 *
 * Given a digit string s, decompose the string into some number of consecutive value-equal
 * substrings where exactly one substring has a length of 2 and the remaining substrings
 * have a length of 3.
 *
 * Return true if you can decompose s according to the above rules. Otherwise, return false.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var isDecomposable = function(s) {
  const groups = [];
  let i = 0;

  while (i < s.length) {
    let j = i;
    while (j < s.length && s[j] === s[i]) {
      j++;
    }
    groups.push(j - i);
    i = j;
```

```
  }

  let result = false;

  for (const length of groups) {
    if (length % 3 === 1) {
      return false;
    } else if (length % 3 === 2) {
      if (result) {
        return false;
      }
      result = true;
    }
  }

  return result;
};
```

1935 Maximum Number Of Words You Can Type

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1935. Maximum Number of Words You Can Type
 * https://leetcode.com/problems/maximum-number-of-words-you-can-type/
 * Difficulty: Easy
 *
 * There is a malfunctioning keyboard where some letter keys do not work.
 * All other keys on the keyboard work properly.
 *
 * Given a string text of words separated by a single space (no leading or
 * trailing spaces) and a string brokenLetters of all distinct letter keys
 * that are broken, return the number of words in text you can fully type
 * using this keyboard.
 */

/**
 * @param {string} text
 * @param {string} brokenLetters
 * @return {number}
 */
var canBeTypedWords = function(text, brokenLetters) {
  return text
    .split(/\s+/)
    .filter(word => !brokenLetters.split('').some(s => word.includes(s)))
    .length;
};
```

1936 Add Minimum Number Of Rungs

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1936. Add Minimum Number of Rungs
 * https://leetcode.com/problems/add-minimum-number-of-rungs/
 * Difficulty: Medium
 *
 * You are given a strictly increasing integer array rungs that represents the height of rungs
 * on a ladder. You are currently on the floor at height 0, and you want to reach the last
rung.
 *
 * You are also given an integer dist. You can only climb to the next highest rung if the
 * distance between where you are currently at (the floor or on a rung) and the next rung is
 * at most dist. You are able to insert rungs at any positive integer height if a rung is not
 * already there.
```

```
 *
 * Return the minimum number of rungs that must be added to the ladder in order for you to
climb
 * to the last rung.
 */

/**
 * @param {number[]} rungs
 * @param {number} dist
 * @return {number}
 */
var addRungs = function(rungs, dist) {
  let currentHeight = 0;
  let result = 0;

  for (const rung of rungs) {
    const gap = rung - currentHeight;
    if (gap > dist) {
      result += Math.ceil(gap / dist) - 1;
    }
    currentHeight = rung;
  }

  return result;
};
```

1937 Maximum Number Of Points With Cost

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1937. Maximum Number of Points with Cost
 * https://leetcode.com/problems/maximum-number-of-points-with-cost/
 * Difficulty: Medium
 *
 * You are given an m x n integer matrix points (0-indexed). Starting with 0 points, you want
 * to maximize the number of points you can get from the matrix.
 *
 * To gain points, you must pick one cell in each row. Picking the cell at coordinates (r, c)
 * will add points[r][c] to your score.
 *
 * However, you will lose points if you pick a cell too far from the cell that you picked in
 * the previous row. For every two adjacent rows r and r + 1 (where 0 <= r < m - 1), picking
 * cells at coordinates (r, c1) and (r + 1, c2) will subtract abs(c1 - c2) from your score.
 *
 * Return the maximum number of points you can achieve.
 *
 * abs(x) is defined as:
 * - x for x >= 0.
 * - -x for x < 0.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var maxPoints = function(points) {
  const rows = points.length;
  const cols = points[0].length;
  let prevRow = points[0].slice();

  for (let r = 1; r < rows; r++) {
    const currRow = new Array(cols).fill(0);
    const leftMax = new Array(cols).fill(0);
    const rightMax = new Array(cols).fill(0);

    leftMax[0] = prevRow[0];
    for (let c = 1; c < cols; c++) {
      leftMax[c] = Math.max(leftMax[c - 1] - 1, prevRow[c]);
    }
```

```
    rightMax[cols - 1] = prevRow[cols - 1];
    for (let c = cols - 2; c >= 0; c--) {
      rightMax[c] = Math.max(rightMax[c + 1] - 1, prevRow[c]);
    }

    for (let c = 0; c < cols; c++) {
      currRow[c] = points[r][c] + Math.max(leftMax[c], rightMax[c]);
    }

    prevRow = currRow;
  }

  return Math.max(...prevRow);
};
```

1938 Maximum Genetic Difference Query

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1938. Maximum Genetic Difference Query
 * https://leetcode.com/problems/maximum-genetic-difference-query/
 * Difficulty: Hard
 *
 * There is a rooted tree consisting of n nodes numbered 0 to n - 1. Each node's number
denotes
 * its unique genetic value (i.e. the genetic value of node x is x). The genetic difference
 * between two genetic values is defined as the bitwise-XOR of their values. You are given
 * the integer array parents, where parents[i] is the parent for node i. If node x is the root
 * of the tree, then parents[x] == -1.
 *
 * You are also given the array queries where queries[i] = [nodei, vali]. For each query i,
 * find the maximum genetic difference between vali and pi, where pi is the genetic value of
 * any node that is on the path between nodei and the root (including nodei and the root).
 * More formally, you want to maximize vali XOR pi.
 *
 * Return an array ans where ans[i] is the answer to the ith query.
 */

/**
 * @param {number[]} parents
 * @param {number[][]} queries
 * @return {number[]}
 */
var maxGeneticDifference = function(parents, queries) {
  const n = parents.length;
  const graph = Array(n).fill().map(() => []);
  let root = -1;

  for (let i = 0; i < n; i++) {
    if (parents[i] === -1) {
      root = i;
    } else {
      graph[parents[i]].push(i);
    }
  }

  const trie = { count: 0, children: {} };
  const queryMap = Array(n).fill().map(() => []);
  const result = new Array(queries.length).fill(0);

  for (let i = 0; i < queries.length; i++) {
    queryMap[queries[i][0]].push([queries[i][1], i]);
  }

  dfs(root);
  return result;

  function insertTrie(val) {
```

```
    let node = trie;
    for (let bit = 17; bit >= 0; bit--) {
      const b = (val >> bit) & 1;
      if (!node.children[b]) {
        node.children[b] = { count: 0, children: {} };
      }
      node = node.children[b];
      node.count++;
    }
  }

  function removeTrie(val) {
    let node = trie;
    for (let bit = 17; bit >= 0; bit--) {
      const b = (val >> bit) & 1;
      node = node.children[b];
      node.count--;
    }
  }

  function queryTrie(val) {
    let node = trie;
    let maxXor = 0;
    for (let bit = 17; bit >= 0; bit--) {
      const b = (val >> bit) & 1;
      if (node.children[1 - b]?.count > 0) {
        maxXor |= (1 << bit);
        node = node.children[1 - b];
      } else if (node.children[b]?.count > 0) {
        node = node.children[b];
      } else {
        return maxXor;
      }
    }
    return maxXor;
  }

  function dfs(node) {
    insertTrie(node);
    for (const [val, idx] of queryMap[node]) {
      result[idx] = queryTrie(val);
    }
    for (const child of graph[node]) {
      dfs(child);
    }
    removeTrie(node);
  }
};
```

1940 Longest Common Subsequence Between Sorted Arrays

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1940. Longest Common Subsequence Between Sorted Arrays
 * https://leetcode.com/problems/longest-common-subsequence-between-sorted-arrays/
 * Difficulty: Medium
 *
 * Given an array of integer arrays arrays where each arrays[i] is sorted in strictly
increasing
 * order, return an integer array representing the longest common subsequence among all the
arrays.
 *
 * A subsequence is a sequence that can be derived from another sequence by deleting some
elements
 * (possibly none) without changing the order of the remaining elements.
 */

/**
```

```
 * @param {number[][]} arrays
 * @return {number[]}
 */
var longestCommonSubsequence = function(arrays) {
  const map = new Map();
  const totalArrays = arrays.length;

  for (const array of arrays) {
    for (const num of array) {
      map.set(num, (map.get(num) || 0) + 1);
    }
  }

  const result = [];
  for (const [num, count] of map) {
    if (count === totalArrays) {
      result.push(num);
    }
  }

  return result.sort((a, b) => a - b);
};
```

1941 Check If All Characters Have Equal Number Of Occurrences

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1941. Check if All Characters Have Equal Number of Occurrences
 * https://leetcode.com/problems/check-if-all-characters-have-equal-number-of-occurrences/
 * Difficulty: Easy
 *
 * Given a string s, return true if s is a good string, or false otherwise.
 *
 * A string s is good if all the characters that appear in s have the same number of
occurrences
 * (i.e., the same frequency).
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var areOccurrencesEqual = function(s) {
  const map = new Map();
  for (const char of s) {
    map.set(char, (map.get(char) || 0) + 1);
  }

  const frequencies = new Set(map.values());
  return frequencies.size === 1;
};
```

1942 The Number Of The Smallest Unoccupied Chair

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1942. The Number of the Smallest Unoccupied Chair
 * https://leetcode.com/problems/the-number-of-the-smallest-unoccupied-chair/
 * Difficulty: Medium
 *
 * There is a party where n friends numbered from 0 to n - 1 are attending. There is an
infinite
```

```
 * number of chairs in this party that are numbered from 0 to infinity. When a friend arrives
at
 * the party, they sit on the unoccupied chair with the smallest number.
 * - For example, if chairs 0, 1, and 5 are occupied when a friend comes, they will sit on
chair
 *   number 2.
 *
 * When a friend leaves the party, their chair becomes unoccupied at the moment they leave.
 * If another friend arrives at that same moment, they can sit in that chair.
 *
 * You are given a 0-indexed 2D integer array times where times[i] = [arrivali, leavingi],
 * indicating the arrival and leaving times of the ith friend respectively, and an integer
 * targetFriend. All arrival times are distinct.
 *
 * Return the chair number that the friend numbered targetFriend will sit on.
 */

/**
 * @param {number[][]} times
 * @param {number} targetFriend
 * @return {number}
 */
var smallestChair = function(times, targetFriend) {
  const events = [];

  for (let i = 0; i < times.length; i++) {
    events.push([times[i][0], 'arrive', i]);
    events.push([times[i][1], 'leave', i]);
  }

  events.sort((a, b) => a[0] - b[0] || (a[1] === 'leave' ? -1 : 1));

  const availableChairs = [];
  const occupiedChairs = new Map();
  let nextChair = 0;

  for (const [time, eventType, friendId] of events) {
    if (eventType === 'leave') {
      const chairNumber = occupiedChairs.get(friendId);
      occupiedChairs.delete(friendId);
      availableChairs.push(chairNumber);
      availableChairs.sort((a, b) => a - b);
    } else {
      let assignedChair;
      if (availableChairs.length > 0) {
        assignedChair = availableChairs.shift();
      } else {
        assignedChair = nextChair++;
      }

      occupiedChairs.set(friendId, assignedChair);

      if (friendId === targetFriend) {
        return assignedChair;
      }
    }
  }
};
```

1943 Describe The Painting

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1943. Describe the Painting
 * https://leetcode.com/problems/describe-the-painting/
 * Difficulty: Medium
 *
 * There is a long and thin painting that can be represented by a number line. The painting
was
```

```
 * painted with multiple overlapping segments where each segment was painted with a unique
color.
 * You are given a 2D integer array segments, where segments[i] = [starti, endi, colori]
 * represents the half-closed segment [starti, endi) with colori as the color.
 *
 * The colors in the overlapping segments of the painting were mixed when it was painted. When
 * two or more colors mix, they form a new color that can be represented as a set of mixed
colors.
 * - For example, if colors 2, 4, and 6 are mixed, then the resulting mixed color is {2,4,6}.
 *
 * For the sake of simplicity, you should only output the sum of the elements in the set
rather
 * than the full set.
 *
 * You want to describe the painting with the minimum number of non-overlapping half-closed
 * segments of these mixed colors. These segments can be represented by the 2D array painting
 * where painting[j] = [leftj, rightj, mixj] describes a half-closed segment [leftj, rightj)
 * with the mixed color sum of mixj.
 * - For example, the painting created with segments = [[1,4,5],[1,7,7]] can be described by
 *   painting = [[1,4,12],[4,7,7]] because:
 *   - [1,4) is colored {5,7} (with a sum of 12) from both the first and second segments.
 *   - [4,7) is colored {7} from only the second segment.
 *
 * Return the 2D array painting describing the finished painting (excluding any parts that are
 * not painted). You may return the segments in any order.
 *
 * A half-closed segment [a, b) is the section of the number line between points a and b
including
 * point a and not including point b.
 */

/**
 * @param {number[][]} segments
 * @return {number[][]}
 */
var splitPainting = function(segments) {
  const events = [];
  for (const [start, end, color] of segments) {
    events.push([start, color]);
    events.push([end, -color]);
  }

  events.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  const result = [];
  let currentSum = 0;
  let prevPoint = events[0][0];

  for (const [point, color] of events) {
    if (currentSum > 0 && point > prevPoint) {
      result.push([prevPoint, point, currentSum]);
    }
    currentSum += color;
    prevPoint = point;
  }

  return result;
};
```

1944 Number Of Visible People In A Queue

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1944. Number of Visible People in a Queue
 * https://leetcode.com/problems/number-of-visible-people-in-a-queue/
 * Difficulty: Hard
 *
 * There are n people standing in a queue, and they numbered from 0 to n - 1 in left to right
 * order. You are given an array heights of distinct integers where heights[i] represents the
```

```
 * height of the ith person.
 *
 * A person can see another person to their right in the queue if everybody in between is
 * shorter than both of them. More formally, the ith person can see the jth person
 * if i < j and min(heights[i], heights[j]) > max(heights[i+1], heights[i+2], ...,
heights[j-1]).
 *
 * Return an array answer of length n where answer[i] is the number of people the ith person
 * can see to their right in the queue.
 */

/**
 * @param {number[]} heights
 * @return {number[]}
 */
var canSeePersonsCount = function(heights) {
  const n = heights.length;
  const result = new Array(n).fill(0);
  const stack = [];

  for (let i = n - 1; i >= 0; i--) {
    while (stack.length && heights[i] > stack[stack.length - 1]) {
      stack.pop();
      result[i]++;
    }
    if (stack.length) {
      result[i]++;
    }
    stack.push(heights[i]);
  }

  return result;
};
```

1945 Sum Of Digits Of String After Convert

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1945. Sum of Digits of String After Convert
 * https://leetcode.com/problems/sum-of-digits-of-string-after-convert/
 * Difficulty: Easy
 *
 * You are given a string s consisting of lowercase English letters, and an integer k. Your
task
 * is to convert the string into an integer by a special process, and then transform it by
 * summing its digits repeatedly k times. More specifically, perform the following steps:
 * 1. Convert s into an integer by replacing each letter with its position in the alphabet
(i.e.
 *    replace 'a' with 1, 'b' with 2, ..., 'z' with 26).
 * 2. Transform the integer by replacing it with the sum of its digits.
 * 3. Repeat the transform operation (step 2) k times in total.
 *
 * For example, if s = "zbax" and k = 2, then the resulting integer would be 8 by the
following
 * operations:
 * 1. Convert: "zbax" → "(26)(2)(1)(24)" → "262124" → 262124
 * 2. Transform #1: 262124 → 2 + 6 + 2 + 1 + 2 + 4 → 17
 * 3. Transform #2: 17 → 1 + 7 → 8
 *
 * Return the resulting integer after performing the operations described above.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var getLucky = function(s, k) {
  let number = '';
```

```
  for (const char of s) {
    number += (char.charCodeAt(0) - 96).toString();
  }

  let result = 0;
  for (let i = 0; i < k; i++) {
    result = 0;
    for (const digit of number) {
      result += parseInt(digit);
    }
    number = result.toString();
  }

  return result;
};
```

1946 Largest Number After Mutating Substring

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1946. Largest Number After Mutating Substring
 * https://leetcode.com/problems/largest-number-after-mutating-substring/
 * Difficulty: Medium
 *
 * You are given a string num, which represents a large integer. You are also given a
0-indexed
 * integer array change of length 10 that maps each digit 0-9 to another digit. More formally,
 * digit d maps to digit change[d].
 *
 * You may choose to mutate a single substring of num. To mutate a substring, replace each
digit
 * num[i] with the digit it maps to in change (i.e. replace num[i] with change[num[i]]).
 *
 * Return a string representing the largest possible integer after mutating (or choosing not
to)
 * a single substring of num.
 *
 * A substring is a contiguous sequence of characters within the string.
 */

/**
 * @param {string} num
 * @param {number[]} change
 * @return {string}
 */
var maximumNumber = function(num, change) {
  const digits = num.split('');
  let mutated = false;

  for (let i = 0; i < digits.length; i++) {
    const currentDigit = parseInt(digits[i]);
    const mappedDigit = change[currentDigit];

    if (mappedDigit > currentDigit) {
      digits[i] = mappedDigit.toString();
      mutated = true;
    } else if (mappedDigit < currentDigit && mutated) {
      break;
    }
  }

  return digits.join('');
};
```

1947 Maximum Compatibility Score Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1947. Maximum Compatibility Score Sum
 * https://leetcode.com/problems/maximum-compatibility-score-sum/
 * Difficulty: Medium
 *
 * There is a survey that consists of n questions where each question's answer is either 0
 * (no) or 1 (yes).
 *
 * The survey was given to m students numbered from 0 to m - 1 and m mentors numbered from
 * 0 to m - 1. The answers of the students are represented by a 2D integer array students
 * where students[i] is an integer array that contains the answers of the ith student
 * (0-indexed). The answers of the mentors are represented by a 2D integer array mentors
 * where mentors[j] is an integer array that contains the answers of the jth mentor
(0-indexed).
 *
 * Each student will be assigned to one mentor, and each mentor will have one student assigned
 * to them. The compatibility score of a student-mentor pair is the number of answers that
 * are the same for both the student and the mentor.
 * - For example, if the student's answers were [1, 0, 1] and the mentor's answers were
 *   [0, 0, 1], then their compatibility score is 2 because only the second and the third
 *   answers are the same.
 *
 * You are tasked with finding the optimal student-mentor pairings to maximize the sum of the
 * compatibility scores.
 *
 * Given students and mentors, return the maximum compatibility score sum that can be
achieved.
 */

/**
 * @param {number[][]} students
 * @param {number[][]} mentors
 * @return {number}
 */
var maxCompatibilitySum = function(students, mentors) {
  const m = students.length;
  let maxScore = 0;

  backtrack(0, new Array(m).fill(false), 0);
  return maxScore;

  function calculateScore(student, mentor) {
    let score = 0;
    for (let i = 0; i < student.length; i++) {
      if (student[i] === mentor[i]) score++;
    }
    return score;
  }

  function backtrack(studentIndex, usedMentors, currentScore) {
    if (studentIndex === m) {
      maxScore = Math.max(maxScore, currentScore);
      return;
    }

    for (let mentorIndex = 0; mentorIndex < m; mentorIndex++) {
      if (!usedMentors[mentorIndex]) {
        usedMentors[mentorIndex] = true;
        backtrack(studentIndex + 1, usedMentors,
          currentScore + calculateScore(students[studentIndex], mentors[mentorIndex]));
        usedMentors[mentorIndex] = false;
      }
    }
  }
};
```

1948 Delete Duplicate Folders In System

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1948. Delete Duplicate Folders in System
 * https://leetcode.com/problems/delete-duplicate-folders-in-system/
 * Difficulty: Hard
 *
 * Due to a bug, there are many duplicate folders in a file system. You are given a 2D array
 * paths, where paths[i] is an array representing an absolute path to the ith folder in the
 * file system.
 * - For example, ["one", "two", "three"] represents the path "/one/two/three".
 *
 * Two folders (not necessarily on the same level) are identical if they contain the same
 * non-empty set of identical subfolders and underlying subfolder structure. The folders do
 * not need to be at the root level to be identical. If two or more folders are identical,
 * then mark the folders as well as all their subfolders.
 * - For example, folders "/a" and "/b" in the file structure below are identical. They (as
 *   well as their subfolders) should all be marked:
 *   - /a
 *   - /a/x
 *   - /a/x/y
 *   - /a/z
 *   - /b
 *   - /b/x
 *   - /b/x/y
 *   - /b/z
 * - However, if the file structure also included the path "/b/w", then the folders "/a" and
"/b"
 *   would not be identical. Note that "/a/x" and "/b/x" would still be considered identical
 *   even with the added folder.
 *
 * Once all the identical folders and their subfolders have been marked, the file system will
delete
 * all of them. The file system only runs the deletion once, so any folders that become
identical
 * after the initial deletion are not deleted.
 *
 * Return the 2D array ans containing the paths of the remaining folders after deleting all
the
 * marked folders. The paths may be returned in any order.
 */

/**
 * @param {string[][]} paths
 * @return {string[][]}
 */
var deleteDuplicateFolder = function(paths) {
  const root = {};

  for (const path of paths) {
    let node = root;
    for (const folder of path) {
      if (!node[folder]) node[folder] = {};
      node = node[folder];
    }
  }

  const structures = new Map();

  const serialize = (node, path) => {
    if (Object.keys(node).length === 0) return '';

    const folders = [];
    const keys = Object.keys(node);
    for (const folder of keys) {
      const serialized = serialize(node[folder], [...path, folder]);
      if (serialized !== null) {
        folders.push(`${folder}(${serialized})`);
```

```
      } else {
        delete node[folder];
      }
    }

    folders.sort();
    const key = folders.join('');

    if (key.length > 0) {
      if (!structures.has(key)) {
        structures.set(key, []);
      }
      structures.get(key).push(path);
    }

    return key;
  };

  serialize(root, []);

  const toDelete = new Set();

  for (const [structure, paths] of structures.entries()) {
    if (paths.length > 1) {
      for (const path of paths) {
        toDelete.add(path.join('/'));
      }
    }
  }

  const result = [];

  const collectPaths = (node, path) => {
    const currentPath = path.join('/');
    if (toDelete.has(currentPath)) return;

    if (path.length > 0) {
      result.push([...path]);
    }

    const keys = Object.keys(node);
    for (const folder of keys) {
      collectPaths(node[folder], [...path, folder]);
    }
  };

  const rootFolders = Object.keys(root);
  for (const folder of rootFolders) {
    collectPaths(root[folder], [folder]);
  }

  return result;
};
```

1950 Maximum Of Minimum Values In All Subarrays

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1950. Maximum of Minimum Values in All Subarrays
 * https://leetcode.com/problems/maximum-of-minimum-values-in-all-subarrays/
 * Difficulty: Medium
 *
 * You are given an integer array nums of size n. You are asked to solve n queries for each
 * integer i in the range 0 <= i < n.
 *
 * To solve the ith query:
 * 1. Find the minimum value in each possible subarray of size i + 1 of the array nums.
 * 2. Find the maximum of those minimum values. This maximum is the answer to the query.
 *
```

```
 * Return a 0-indexed integer array ans of size n such that ans[i] is the answer to the ith
query.
 *
 * A subarray is a contiguous sequence of elements in an array.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var findMaximums = function(nums) {
  const n = nums.length;
  const result = new Array(n);
  const leftBound = new Array(n);
  const rightBound = new Array(n);
  const stack = [];

  for (let i = 0; i < n; i++) {
    while (stack.length > 0 && nums[stack[stack.length - 1]] >= nums[i]) {
      stack.pop();
    }
    leftBound[i] = stack.length > 0 ? stack[stack.length - 1] : -1;
    stack.push(i);
  }

  stack.length = 0;

  for (let i = n - 1; i >= 0; i--) {
    while (stack.length > 0 && nums[stack[stack.length - 1]] >= nums[i]) {
      stack.pop();
    }
    rightBound[i] = stack.length > 0 ? stack[stack.length - 1] : n;
    stack.push(i);
  }

  result.fill(0);

  for (let i = 0; i < n; i++) {
    const maxSubarrayLength = rightBound[i] - leftBound[i] - 1;
    result[maxSubarrayLength - 1] = Math.max(result[maxSubarrayLength - 1], nums[i]);
  }

  for (let i = n - 2; i >= 0; i--) {
    result[i] = Math.max(result[i], result[i + 1]);
  }

  return result;
};
```

1952 Three Divisors

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1952. Three Divisors
 * https://leetcode.com/problems/three-divisors/
 * Difficulty: Easy
 *
 * Given an integer n, return true if n has exactly three positive divisors. Otherwise,
 * return false.
 *
 * An integer m is a divisor of n if there exists an integer k such that n = k * m.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isThree = function(n) {
  let divisorCount = 0;
```

```
  for (let i = 1; i <= n; i++) {
    if (n % i === 0) {
      divisorCount++;
      if (divisorCount > 3) return false;
    }
  }

  return divisorCount === 3;
};
```

1953 Maximum Number Of Weeks For Which You Can Work

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1953. Maximum Number of Weeks for Which You Can Work
 * https://leetcode.com/problems/maximum-number-of-weeks-for-which-you-can-work/
 * Difficulty: Medium
 *
 * There are n projects numbered from 0 to n - 1. You are given an integer array milestones
where
 * each milestones[i] denotes the number of milestones the ith project has.
 *
 * You can work on the projects following these two rules:
 * - Every week, you will finish exactly one milestone of one project. You must work every
week.
 * - You cannot work on two milestones from the same project for two consecutive weeks.
 * - Once all the milestones of all the projects are finished, or if the only milestones that
you
 *   can work on will cause you to violate the above rules, you will stop working. Note that
you
 *   may not be able to finish every project's milestones due to these constraints.
 *
 * Return the maximum number of weeks you would be able to work on the projects without
violating
 * the rules mentioned above.
 */

/**
 * @param {number[]} milestones
 * @return {number}
 */
var numberOfWeeks = function(milestones) {
  let total = 0;
  let maxMilestones = 0;

  for (const count of milestones) {
    total += count;
    maxMilestones = Math.max(maxMilestones, count);
  }

  return Math.min(total, 2 * (total - maxMilestones) + 1);
};
```

1954 Minimum Garden Perimeter To Collect Enough Apples

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1954. Minimum Garden Perimeter to Collect Enough Apples
 * https://leetcode.com/problems/minimum-garden-perimeter-to-collect-enough-apples/
 * Difficulty: Medium
 *
 * In a garden represented as an infinite 2D grid, there is an apple tree planted at every
```

```
 * integer coordinate. The apple tree planted at an integer coordinate (i, j) has |i| + |j|
 * apples growing on it.
 *
 * You will buy an axis-aligned square plot of land that is centered at (0, 0).
 *
 * Given an integer neededApples, return the minimum perimeter of a plot such that at least
 * neededApples apples are inside or on the perimeter of that plot.
 *
 * The value of |x| is defined as:
 * - x if x >= 0
 * - -x if x < 0
 */

/**
 * @param {number} neededApples
 * @return {number}
 */
var minimumPerimeter = function(neededApples) {
  let n = 0;
  let totalApples = 0;

  while (totalApples < neededApples) {
    n++;
    totalApples = 2 * n * (n + 1) * (2 * n + 1);
  }

  return 8 * n;
};
```

1955 Count Number Of Special Subsequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1955. Count Number of Special Subsequences
 * https://leetcode.com/problems/count-number-of-special-subsequences/
 * Difficulty: Hard
 *
 * A sequence is special if it consists of a positive number of 0s, followed by a positive
number
 * of 1s, then a positive number of 2s.
 * - For example, [0,1,2] and [0,0,1,1,1,2] are special.
 * - In contrast, [2,1,0], [1], and [0,1,2,0] are not special.
 *
 * Given an array nums (consisting of only integers 0, 1, and 2), return the number of
different
 * subsequences that are special. Since the answer may be very large, return it modulo 109 +
7.
 *
 * A subsequence of an array is a sequence that can be derived from the array by deleting some
 * or no elements without changing the order of the remaining elements. Two subsequences are
 * different if the set of indices chosen are different.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countSpecialSubsequences = function(nums) {
  const MOD = 1e9 + 7;
  const dp = [1, 0, 0, 0];

  for (const num of nums) {
    dp[num + 1] = (dp[num + 1] * 2 % MOD + dp[num]) % MOD;
  }

  return dp[3];
};
```

1956 Minimum Time For K Virus Variants To Spread

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1956. Minimum Time For K Virus Variants to Spread
 * https://leetcode.com/problems/minimum-time-for-k-virus-variants-to-spread/
 * Difficulty: Hard
 *
 * There are n unique virus variants in an infinite 2D grid. You are given a 2D array points,
 * where points[i] = [xi, yi] represents a virus originating at (xi, yi) on day 0. Note that
 * it is possible for multiple virus variants to originate at the same point.
 *
 * Every day, each cell infected with a virus variant will spread the virus to all neighboring
 * points in the four cardinal directions (i.e. up, down, left, and right). If a cell has
multiple
 * variants, all the variants will spread without interfering with each other.
 *
 * Given an integer k, return the minimum integer number of days for any point to contain at
least
 * k of the unique virus variants.
 */

/**
 * @param {number[][]} points
 * @param {number} k
 * @return {number}
 */
var minDayskVariants = function(points, k) {
  let left = 0;
  let right = 200;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);

    if (check(mid)) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;

  function check(day) {
    for (let x = 1; x <= 100; x++) {
      for (let y = 1; y <= 100; y++) {
        let count = 0;
        for (const [px, py] of points) {
          if (Math.abs(x - px) + Math.abs(y - py) <= day) {
            count++;
          }
        }
        if (count >= k) {
          return true;
        }
      }
    }
    return false;
  }
};
```

1957 Delete Characters To Make Fancy String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1957. Delete Characters to Make Fancy String
 * https://leetcode.com/problems/delete-characters-to-make-fancy-string/
 * Difficulty: Easy
 *
 * A fancy string is a string where no three consecutive characters are equal.
 *
 * Given a string s, delete the minimum possible number of characters from s to make
 * it fancy.
 *
 * Return the final string after the deletion. It can be shown that the answer will
 * always be unique.
 */

/**
 * @param {string} s
 * @return {string}
 */
var makeFancyString = function(s) {
  let result = s[0];
  let count = 1;

  for (let i = 1; i < s.length; i++) {
    if (s[i] === s[i - 1]) {
      if (count < 2) {
        result += s[i];
        count++;
      }
    } else {
      result += s[i];
      count = 1;
    }
  }

  return result;
};
```

1958 Check If Move Is Legal

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1958. Check if Move is Legal
 * https://leetcode.com/problems/check-if-move-is-legal/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 8 x 8 grid board, where board[r][c] represents the cell (r, c) on
 a
 * game board. On the board, free cells are represented by '.', white cells are represented by
 * 'W', and black cells are represented by 'B'.
 *
 * Each move in this game consists of choosing a free cell and changing it to the color you
are
 * playing as (either white or black). However, a move is only legal if, after changing it,
the
 * cell becomes the endpoint of a good line (horizontal, vertical, or diagonal).
 *
 * A good line is a line of three or more cells (including the endpoints) where the endpoints
of
 * the line are one color, and the remaining cells in the middle are the opposite color (no
cells
 * in the line are free). You can find examples for good lines in the figure below.
 *
 * Given two integers rMove and cMove and a character color representing the color you are
playing
 * as (white or black), return true if changing cell (rMove, cMove) to color color is a legal
move,
 * or false if it is not legal.
 */
```

```
/**
 * @param {character[][]} board
 * @param {number} rMove
 * @param {number} cMove
 * @param {character} color
 * @return {boolean}
 */
var checkMove = function(board, rMove, cMove, color) {
  const opposite = color === 'B' ? 'W' : 'B';
  const directions = [
    [0, 1], [0, -1], [1, 0], [-1, 0],
    [1, 1], [1, -1], [-1, 1], [-1, -1]
  ];

  for (const [dr, dc] of directions) {
    let row = rMove + dr;
    let col = cMove + dc;
    let count = 1;

    while (row >= 0 && row < 8 && col >= 0 && col < 8 && board[row][col] === opposite) {
      row += dr;
      col += dc;
      count++;
    }

    if (count >= 2 && row >= 0 && row < 8 && col >= 0 && col < 8 && board[row][col] === color)
 {
      return true;
    }
  }

  return false;
};
```

1959 Minimum Total Space Wasted With K Resizing Operations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1959. Minimum Total Space Wasted With K Resizing Operations
 * https://leetcode.com/problems/minimum-total-space-wasted-with-k-resizing-operations/
 * Difficulty: Medium
 *
 * You are currently designing a dynamic array. You are given a 0-indexed integer array nums,
 * where nums[i] is the number of elements that will be in the array at time i. In addition,
 * you are given an integer k, the maximum number of times you can resize the array (to any
size).
 *
 * The size of the array at time t, sizet, must be at least nums[t] because there needs to be
 * enough space in the array to hold all the elements. The space wasted at time t is defined
 * as sizet - nums[t], and the total space wasted is the sum of the space wasted across every
 * time t where 0 <= t < nums.length.
 *
 * Return the minimum total space wasted if you can resize the array at most k times.
 *
 * Note: The array can have any size at the start and does not count towards the number of
 * resizing operations.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minSpaceWastedKResizing = function(nums, k) {
  const n = nums.length;
  const dp = new Array(n + 1).fill().map(() => new Array(k + 2).fill(Infinity));
  dp[0][0] = 0;
```

```
  for (let i = 1; i <= n; i++) {
    let maxVal = 0;
    let sum = 0;

    for (let j = i; j > 0; j--) {
      maxVal = Math.max(maxVal, nums[j - 1]);
      sum += nums[j - 1];
      const waste = maxVal * (i - j + 1) - sum;

      for (let resizes = 0; resizes <= k; resizes++) {
        if (dp[j - 1][resizes] !== Infinity) {
          dp[i][resizes + 1] = Math.min(dp[i][resizes + 1], dp[j - 1][resizes] + waste);
        }
      }
    }
  }

  let result = Infinity;
  for (let resizes = 1; resizes <= k + 1; resizes++) {
    result = Math.min(result, dp[n][resizes]);
  }

  return result;
};
```

1960 Maximum Product Of The Length Of Two Palindromic Substrings

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1960. Maximum Product of the Length of Two Palindromic Substrings
 * https://leetcode.com/problems/maximum-product-of-the-length-of-two-palindromic-substrings/
 * Difficulty: Hard
 *
 * You are given a 0-indexed string s and are tasked with finding two non-intersecting
palindromic
 * substrings of odd length such that the product of their lengths is maximized.
 *
 * More formally, you want to choose four integers i, j, k, l such that
 * 0 <= i <= j < k <= l < s.length and both the substrings s[i...j] and s[k...l] are
palindromes
 * and have odd lengths. s[i...j] denotes a substring from index i to index j inclusive.
 *
 * Return the maximum possible product of the lengths of the two non-intersecting palindromic
 * substrings.
 *
 * A palindrome is a string that is the same forward and backward. A substring is a contiguous
 * sequence of characters in a string.
 */

/**
* @param {string} s
* @return {number}
*/
var maxProduct = function(s) {
  const n = s.length;
  const before = new Array(n).fill(0);
  const after = new Array(n).fill(0);

  let center = -1;
  let right = -1;
  const dp = new Array(n).fill(0);

  for (let i = 0; i < n; i++) {
    const radius = i <= right ? Math.min(dp[2 * center - i], right - i) : 0;
    let left = i - radius;
    let rt = i + radius;

    while (left >= 0 && rt < n && s[left] === s[rt]) {
      before[rt] = Math.max(before[rt], rt - left + 1);
```

```
      after[left] = Math.max(after[left], rt - left + 1);
      left--;
      rt++;
    }

    dp[i] = rt - i - 1;

    if (rt - 1 > right) {
      center = i;
      right = rt - 1;
    }
  }

  for (let i = 1; i < n; i++) {
    before[i] = Math.max(before[i - 1], before[i]);
  }

  for (let i = n - 2; i >= 0; i--) {
    after[i] = Math.max(after[i + 1], after[i]);
  }

  let result = 0;
  for (let i = 1; i < n; i++) {
    result = Math.max(result, before[i - 1] * after[i]);
  }

  return result;
};
```

1961 Check If String Is A Prefix Of Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1961. Check If String Is a Prefix of Array
 * https://leetcode.com/problems/check-if-string-is-a-prefix-of-array/
 * Difficulty: Easy
 *
 * Given a string s and an array of strings words, determine whether s is a prefix string of
words.
 *
 * A string s is a prefix string of words if s can be made by concatenating the first k
strings in
 * words for some positive k no larger than words.length.
 *
 * Return true if s is a prefix string of words, or false otherwise.
 */

/**
 * @param {string} s
 * @param {string[]} words
 * @return {boolean}
 */
var isPrefixString = function(s, words) {
  let currentIndex = 0;

  for (const word of words) {
    if (currentIndex + word.length > s.length) return false;
    if (s.slice(currentIndex, currentIndex + word.length) !== word) return false;
    currentIndex += word.length;
    if (currentIndex === s.length) return true;
  }

  return false;
};
```

1962 Remove Stones To Minimize The Total

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1962. Remove Stones to Minimize the Total
 * https://leetcode.com/problems/remove-stones-to-minimize-the-total/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array piles, where piles[i] represents the number
 * of stones in the ith pile, and an integer k. You should apply the following operation
 * exactly k times:
 * - Choose any piles[i] and remove ceil(piles[i] / 2) stones from it.
 *
 * Notice that you can apply the operation on the same pile more than once.
 *
 * Return the minimum possible total number of stones remaining after applying the k
operations.
 *
 * ceil(x) is the smallest integer that is greater than or equal to x (i.e., rounds x up).
 */

/**
 * @param {number[]} piles
 * @param {number} k
 * @return {number}
 */
var minStoneSum = function(piles, k) {
  const maxHeap = new PriorityQueue((a, b) => b - a);
  let result = 0;

  for (const pile of piles) {
    maxHeap.enqueue(pile);
    result += pile;
  }

  while (k-- > 0) {
    const largest = maxHeap.dequeue();
    maxHeap.enqueue(largest - Math.floor(largest / 2));
    result -= Math.floor(largest / 2);
  }

  return result;
};
```

1963 Minimum Number Of Swaps To Make The String Balanced

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1963. Minimum Number of Swaps to Make the String Balanced
 * https://leetcode.com/problems/minimum-number-of-swaps-to-make-the-string-balanced/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s of even length n. The string consists of exactly n / 2
opening
 * brackets '[' and n / 2 closing brackets ']'.
 *
 * A string is called balanced if and only if:
 * - It is the empty string, or
 * - It can be written as AB, where both A and B are balanced strings, or
 * - It can be written as [C], where C is a balanced string.
 *
 * You may swap the brackets at any two indices any number of times.
 *
```

```
 * Return the minimum number of swaps to make s balanced.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minSwaps = function(s) {
  let imbalance = 0;
  let maxImbalance = 0;

  for (const bracket of s) {
    if (bracket === '[') {
      imbalance--;
    } else {
      imbalance++;
      maxImbalance = Math.max(maxImbalance, imbalance);
    }
  }

  return Math.ceil(maxImbalance / 2);
};
```

1964 Find The Longest Valid Obstacle Course At Each Position

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1964. Find the Longest Valid Obstacle Course at Each Position
 * https://leetcode.com/problems/find-the-longest-valid-obstacle-course-at-each-position/
 * Difficulty: Hard
 *
 * You want to build some obstacle courses. You are given a 0-indexed integer array obstacles
of
 * length n, where obstacles[i] describes the height of the ith obstacle.
 *
 * For every index i between 0 and n - 1 (inclusive), find the length of the longest obstacle
 * course in obstacles such that:
 * - You choose any number of obstacles between 0 and i inclusive.
 * - You must include the ith obstacle in the course.
 * - You must put the chosen obstacles in the same order as they appear in obstacles.
 * - Every obstacle (except the first) is taller than or the same height as the obstacle
immediately
 *   before it.
 *
 * Return an array ans of length n, where ans[i] is the length of the longest obstacle course
for
 * index i as described above.
 */

/**
 * @param {number[]} obstacles
 * @return {number[]}
 */
var longestObstacleCourseAtEachPosition = function(obstacles) {
  const n = obstacles.length;
  const result = new Array(n).fill(1);
  const stack = [];

  for (let i = 0; i < n; i++) {
    const height = obstacles[i];
    let left = 0;
    let right = stack.length;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (stack[mid] <= height) {
        left = mid + 1;
      } else {
        right = mid;
```

```
      }
    }

    if (left < stack.length) {
      stack[left] = height;
    } else {
      stack.push(height);
    }

    result[i] = left + 1;
  }

  return result;
};
```

1966 Binary Searchable Numbers In An Unsorted Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1966. Binary Searchable Numbers in an Unsorted Array
 * https://leetcode.com/problems/binary-searchable-numbers-in-an-unsorted-array/
 * Difficulty: Medium
 *
 * Consider a function that implements an algorithm similar to Binary Search. The function has
 * two input parameters: sequence is a sequence of integers, and target is an integer value.
 * The purpose of the function is to find if the target exists in the sequence.
 *
 * The pseudocode of the function is as follows:
 *
 * func(sequence, target)
 *   while sequence is not empty
 *     randomly choose an element from sequence as the pivot
 *     if pivot = target, return true
 *     else if pivot < target, remove pivot and all elements to its left from the sequence
 *     else, remove pivot and all elements to its right from the sequence
 *   end while
 *   return false
 *
 * When the sequence is sorted, the function works correctly for all values. When the sequence
 * is not sorted, the function does not work for all values, but may still work for some
values.
 *
 * Given an integer array nums, representing the sequence, that contains unique numbers and
may
 * or may not be sorted, return the number of values that are guaranteed to be found using the
 * function, for every possible pivot selection.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var binarySearchableNumbers = function(nums) {
  const n = nums.length;
  const leftMax = new Array(n);
  const rightMin = new Array(n);

  leftMax[0] = nums[0];
  for (let i = 1; i < n; i++) {
    leftMax[i] = Math.max(leftMax[i - 1], nums[i]);
  }

  rightMin[n - 1] = nums[n - 1];
  for (let i = n - 2; i >= 0; i--) {
    rightMin[i] = Math.min(rightMin[i + 1], nums[i]);
  }

  let count = 0;
```

```
  for (let i = 0; i < n; i++) {
    const leftMaxVal = i > 0 ? leftMax[i - 1] : -Infinity;
    const rightMinVal = i < n - 1 ? rightMin[i + 1] : Infinity;

    if (leftMaxVal < nums[i] && nums[i] < rightMinVal) {
      count++;
    }
  }

  return count;
};
```

1967 Number Of Strings That Appear As Substrings In Word

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1967. Number of Strings That Appear as Substrings in Word
 * https://leetcode.com/problems/number-of-strings-that-appear-as-substrings-in-word/
 * Difficulty: Easy
 *
 * Given an array of strings patterns and a string word, return the number of strings in
patterns
 * that exist as a substring in word.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string[]} patterns
 * @param {string} word
 * @return {number}
 */
var numOfStrings = function(patterns, word) {
  return patterns.reduce((count, pattern) => word.includes(pattern) ? count + 1 : count, 0);
};
```

1968 Array With Elements Not Equal To Average Of Neighbors

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1968. Array With Elements Not Equal to Average of Neighbors
 * https://leetcode.com/problems/array-with-elements-not-equal-to-average-of-neighbors/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums of distinct integers. You want to rearrange the
elements
 * in the array such that every element in the rearranged array is not equal to the average of
 * its neighbors.
 *
 * More formally, the rearranged array should have the property such that for every i in the
 * range 1 <= i < nums.length - 1, (nums[i-1] + nums[i+1]) / 2 is not equal to nums[i].
 *
 * Return any rearrangement of nums that meets the requirements.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var rearrangeArray = function(nums) {
  const result = new Array(nums.length);
  nums.sort((a, b) => a - b);
```

```
  for (let i = 0, left = 0, right = nums.length - 1; i < nums.length; i++) {
    result[i] = i % 2 === 0 ? nums[left++] : nums[right--];
  }

  return result;
};
```

1969 Minimum Non Zero Product Of The Array Elements

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1969. Minimum Non-Zero Product of the Array Elements
 * https://leetcode.com/problems/minimum-non-zero-product-of-the-array-elements/
 * Difficulty: Medium
 *
 * You are given a positive integer p. Consider an array nums (1-indexed) that consists of the
 * integers in the inclusive range [1, 2p - 1] in their binary representations. You are
allowed
 * to do the following operation any number of times:
 * - Choose two elements x and y from nums.
 * - Choose a bit in x and swap it with its corresponding bit in y. Corresponding bit refers
to
 *   the bit that is in the same position in the other integer.
 *
 * For example, if x = 1101 and y = 0011, after swapping the 2nd bit from the right, we have
 * x = 1111 and y = 0001.
 *
 * Find the minimum non-zero product of nums after performing the above operation any number
 * of times. Return this product modulo 109 + 7.
 *
 * Note: The answer should be the minimum product before the modulo operation is done.
 */

/**
 * @param {number} p
 * @return {number}
 */
var minNonZeroProduct = function(p) {
  const MOD = 1000000007n;
  const maxVal = (1n << BigInt(p)) - 1n;
  const halfCount = (maxVal - 1n) / 2n;
  const base = maxVal - 1n;

  function pow(base, exp, mod) {
    let result = 1n;
    base = base % mod;
    while (exp > 0n) {
      if (exp & 1n) result = (result * base) % mod;
      base = (base * base) % mod;
      exp >>= 1n;
    }
    return result;
  }

  return Number((maxVal * pow(base, halfCount, MOD)) % MOD);
};
```

1970 Last Day Where You Can Still Cross

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1970. Last Day Where You Can Still Cross
 * https://leetcode.com/problems/last-day-where-you-can-still-cross/
```

```
 * Difficulty: Hard
 *
 * There is a 1-based binary matrix where 0 represents land and 1 represents water. You are
given
 * integers row and col representing the number of rows and columns in the matrix,
respectively.
 *
 * Initially on day 0, the entire matrix is land. However, each day a new cell becomes flooded
 * with water. You are given a 1-based 2D array cells, where cells[i] = [ri, ci] represents
that
 * on the ith day, the cell on the rith row and cith column (1-based coordinates) will be
covered
 * with water (i.e., changed to 1).
 *
 * You want to find the last day that it is possible to walk from the top to the bottom by
only
 * walking on land cells. You can start from any cell in the top row and end at any cell in
the
 * bottom row. You can only travel in the four cardinal directions (left, right, up, and
down).
 *
 * Return the last day where it is possible to walk from the top to the bottom by only walking
 * on land cells.
 */

/**
 * @param {number} row
 * @param {number} col
 * @param {number[][]} cells
 * @return {number}
 */
var latestDayToCross = function(row, col, cells) {
  const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];
  let left = 1;
  let right = cells.length;
  let result = 0;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (canCross(mid)) {
      result = mid;
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;

  function canCross(day) {
    const grid = new Array(row).fill().map(() => new Array(col).fill(0));
    for (let i = 0; i < day; i++) {
      const [r, c] = cells[i];
      grid[r - 1][c - 1] = 1;
    }

    const queue = [];
    const visited = new Array(row).fill().map(() => new Array(col).fill(false));

    for (let c = 0; c < col; c++) {
      if (grid[0][c] === 0) {
        queue.push([0, c]);
        visited[0][c] = true;
      }
    }

    while (queue.length) {
      const [r, c] = queue.shift();
      if (r === row - 1) return true;

      for (const [dr, dc] of directions) {
        const nr = r + dr;
        const nc = c + dc;
        if (nr >= 0 && nr < row && nc >= 0 && nc < col && !visited[nr][nc] && grid[nr][nc] ===
0) {
```

```
          queue.push([nr, nc]);
          visited[nr][nc] = true;
        }
      }
    }
    return false;
  }
};
```

1971 Find If Path Exists In Graph

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1971. Find if Path Exists in Graph
 * https://leetcode.com/problems/find-if-path-exists-in-graph/
 * Difficulty: Easy
 *
 * There is a bi-directional graph with n vertices, where each vertex is labeled from 0 to
 * n - 1 (inclusive). The edges in the graph are represented as a 2D integer array edges,
 * where each edges[i] = [ui, vi] denotes a bi-directional edge between vertex ui and
 * vertex vi. Every vertex pair is connected by at most one edge, and no vertex has an
 * edge to itself.
 *
 * You want to determine if there is a valid path that exists from vertex source to vertex
 * destination.
 *
 * Given edges and the integers n, source, and destination, return true if there is a valid
 * path from source to destination, or false otherwise.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number} source
 * @param {number} destination
 * @return {boolean}
 */
var validPath = function(n, edges, source, destination) {
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  const visited = new Set();
  const queue = [source];

  while (queue.length) {
    const vertex = queue.shift();
    if (vertex === destination) return true;
    if (visited.has(vertex)) continue;
    visited.add(vertex);

    for (const neighbor of graph[vertex]) {
      if (!visited.has(neighbor)) {
        queue.push(neighbor);
      }
    }
  }

  return false;
};
```

1973 Count Nodes Equal To Sum Of Descendants

MEDIUM

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 1973. Count Nodes Equal to Sum of Descendants
 * https://leetcode.com/problems/count-nodes-equal-to-sum-of-descendants/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the number of nodes where the value of the node is
 * equal to the sum of the values of its descendants.
 *
 * A descendant of a node x is any node that is on the path from node x to some leaf node. The
 * sum is considered to be 0 if the node has no descendants.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var equalToDescendants = function(root) {
  let count = 0;
  dfs(root);
  return count;

  function dfs(node) {
    if (!node) return 0;

    const leftSum = dfs(node.left);
    const rightSum = dfs(node.right);
    const descendantSum = leftSum + rightSum;

    if (node.val === descendantSum) {
      count++;
    }

    return node.val + descendantSum;
  }
};
```

1974 Minimum Time To Type Word Using Special Typewriter

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1974. Minimum Time to Type Word Using Special Typewriter
 * https://leetcode.com/problems/minimum-time-to-type-word-using-special-typewriter/
 * Difficulty: Easy
 *
 * There is a special typewriter with lowercase English letters 'a' to 'z' arranged in a
circle
 * with a pointer. A character can only be typed if the pointer is pointing to that character.
 * The pointer is initially pointing to the character 'a'.
 *
 * Each second, you may perform one of the following operations:
 * - Move the pointer one character counterclockwise or clockwise.
 * - Type the character the pointer is currently on.
```

```
 *
 * Given a string word, return the minimum number of seconds to type out the characters in
word.
 */

/**
 * @param {string} word
 * @return {number}
 */
var minTimeToType = function(word) {
  let position = 'a';
  let result = 0;

  for (const char of word) {
    const clockwise = Math.abs(char.charCodeAt(0) - position.charCodeAt(0));
    const counterclockwise = 26 - clockwise;
    const moves = Math.min(clockwise, counterclockwise);
    result += moves + 1;
    position = char;
  }

  return result;
};
```

1975 Maximum Matrix Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1975. Maximum Matrix Sum
 * https://leetcode.com/problems/maximum-matrix-sum/
 * Difficulty: Medium
 *
 * You are given an n x n integer matrix. You can do the following operation any number of
times:
 * - Choose any two adjacent elements of matrix and multiply each of them by -1.
 *
 * Two elements are considered adjacent if and only if they share a border.
 *
 * Your goal is to maximize the summation of the matrix's elements. Return the maximum sum of
the
 * matrix's elements using the operation mentioned above.
 */

/**
 * @param {number[][]} matrix
 * @return {number}
 */
var maxMatrixSum = function(matrix) {
  let totalSum = 0;
  let negativeCount = 0;
  let minAbs = Infinity;

  for (const row of matrix) {
    for (const num of row) {
      totalSum += Math.abs(num);
      if (num < 0) negativeCount++;
      minAbs = Math.min(minAbs, Math.abs(num));
    }
  }

  return negativeCount % 2 === 0 ? totalSum : totalSum - 2 * minAbs;
};
```

1976 Number Of Ways To Arrive At Destination

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1976. Number of Ways to Arrive at Destination
 * https://leetcode.com/problems/number-of-ways-to-arrive-at-destination/
 * Difficulty: Medium
 *
 * You are in a city that consists of n intersections numbered from 0 to n - 1 with
bi-directional
 * roads between some intersections. The inputs are generated such that you can reach any
 * intersection from any other intersection and that there is at most one road between any two
 * intersections.
 *
 * You are given an integer n and a 2D integer array roads where roads[i] = [ui, vi, timei]
means
 * that there is a road between intersections ui and vi that takes timei minutes to travel.
You
 * want to know in how many ways you can travel from intersection 0 to intersection n - 1 in
the
 * shortest amount of time.
 *
 * Return the number of ways you can arrive at your destination in the shortest amount of
time.
 * Since the answer may be large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number[][]} roads
 * @return {number}
 */
var countPaths = function(n, roads) {
  const MOD = 1e9 + 7;
  const graph = Array.from({ length: n }, () => []);
  const distances = new Array(n).fill(Infinity);
  const ways = new Array(n).fill(0);

  for (const [u, v, time] of roads) {
    graph[u].push([v, time]);
    graph[v].push([u, time]);
  }

  const queue = [[0, 0]];
  distances[0] = 0;
  ways[0] = 1;

  while (queue.length) {
    const [dist, node] = queue.shift();

    if (dist > distances[node]) continue;

    for (const [next, time] of graph[node]) {
      const newDist = dist + time;

      if (newDist < distances[next]) {
        distances[next] = newDist;
        ways[next] = ways[node];
        queue.push([newDist, next]);
        queue.sort((a, b) => a[0] - b[0]);
      } else if (newDist === distances[next]) {
        ways[next] = (ways[next] + ways[node]) % MOD;
      }
    }
  }

  return ways[n - 1];
};
```

1977 Number Of Ways To Separate Numbers

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1977. Number of Ways to Separate Numbers
 * https://leetcode.com/problems/number-of-ways-to-separate-numbers/
 * Difficulty: Hard
 *
 * You wrote down many positive integers in a string called num. However, you realized that
you
 * forgot to add commas to seperate the different numbers. You remember that the list of
integers
 * was non-decreasing and that no integer had leading zeros.
 *
 * Return the number of possible lists of integers that you could have written down to get the
 * string num. Since the answer may be large, return it modulo 109 + 7.
 */

/**
 * @param {string} num
 * @return {number}
 */
var numberOfCombinations = function(num) {
  const n = num.length;
  if (num[0] === '0') return 0;

  const MOD = 1e9 + 7;
  const lcp = new Array(n + 1).fill().map(() => new Array(n + 1).fill(0));

  for (let i = n - 1; i >= 0; i--) {
    for (let j = n - 1; j >= 0; j--) {
      if (num[i] === num[j]) {
        lcp[i][j] = lcp[i + 1][j + 1] + 1;
      }
    }
  }

  const dp = new Array(n).fill().map(() => new Array(n + 1).fill(0));
  const prefixSum = new Array(n).fill().map(() => new Array(n + 1).fill(0));

  dp[0][1] = 1;
  prefixSum[0][1] = 1;

  for (let i = 1; i < n; i++) {
    for (let j = 1; j <= i; j++) {
      prefixSum[i - 1][j] = (prefixSum[i - 1][j - 1] + dp[i - 1][j]) % MOD;
    }

    for (let len = 1; len <= i + 1; len++) {
      if (num[i - len + 1] === '0') continue;

      if (i - len < 0) {
        dp[i][len] = 1;
        continue;
      }

      dp[i][len] = prefixSum[i - len][Math.min(i - len + 1, len)];

      if (len <= i - len + 1) {
        const start1 = i - 2 * len + 1;
        const start2 = i - len + 1;

        if (!compare(start1, start2, len)) {
          dp[i][len] = (dp[i][len] - dp[i - len][len] + MOD) % MOD;
        }
      }
    }

    prefixSum[i][0] = 0;
```

```
    for (let j = 1; j <= i + 1; j++) {
      prefixSum[i][j] = (prefixSum[i][j - 1] + dp[i][j]) % MOD;
    }
  }

  return prefixSum[n - 1][n];

  function compare(i1, i2, len) {
    const commonLen = lcp[i1][i2];
    if (commonLen >= len) return true;
    return i1 + commonLen < n && i2 + commonLen < n && num[i1 + commonLen] <= num[i2 +
commonLen];
  }
};
```

1979 Find Greatest Common Divisor Of Array

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1979. Find Greatest Common Divisor of Array
 * https://leetcode.com/problems/find-greatest-common-divisor-of-array/
 * Difficulty: Easy
 *
 * Given an integer array nums, return the greatest common divisor of the smallest number and
 * largest number in nums.
 *
 * The greatest common divisor of two numbers is the largest positive integer that evenly
 * divides both numbers.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findGCD = function(nums) {
  return gcd(Math.min(...nums), Math.max(...nums));

  function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
  }
};
```

1980 Find Unique Binary String

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 1980. Find Unique Binary String
 * https://leetcode.com/problems/find-unique-binary-string/
 * Difficulty: Medium
 *
 * Given an array of strings nums containing n unique binary strings each of length n,
 * return a binary string of length n that does not appear in nums. If there are multiple
 * answers, you may return any of them.
 */

/**
 * @param {string[]} nums
 * @return {string}
 */
var findDifferentBinaryString = function(nums) {
  return nums.map((n, i) => n[i] === '0' ? '1' : '0').join('');
};
```

1981 Minimize The Difference Between Target And Chosen Elements

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1981. Minimize the Difference Between Target and Chosen Elements
 * https://leetcode.com/problems/minimize-the-difference-between-target-and-chosen-elements/
 * Difficulty: Medium
 *
 * You are given an m x n integer matrix mat and an integer target.
 *
 * Choose one integer from each row in the matrix such that the absolute difference between
 * target and the sum of the chosen elements is minimized.
 *
 * Return the minimum absolute difference.
 *
 * The absolute difference between two numbers a and b is the absolute value of a - b.
 */

/**
 * @param {number[][]} mat
 * @param {number} target
 * @return {number}
 */
var minimizeTheDifference = function(mat, target) {
  const m = mat.length;
  const maxSum = 70 * m;
  let possibleSums = new Set([0]);

  for (const row of mat) {
    const nextSums = new Set();
    for (const num of row) {
      for (const sum of possibleSums) {
        const newSum = sum + num;
        if (newSum <= target + maxSum) {
          nextSums.add(newSum);
        }
      }
    }
    possibleSums = nextSums;
  }

  let result = Infinity;
  for (const sum of possibleSums) {
    result = Math.min(result, Math.abs(sum - target));
  }

  return result;
};
```

1982 Find Array Given Subset Sums

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1982. Find Array Given Subset Sums
 * https://leetcode.com/problems/find-array-given-subset-sums/
 * Difficulty: Hard
 *
 * You are given an integer n representing the length of an unknown array that you are trying
 * to recover. You are also given an array sums containing the values of all 2n subset sums of
 * the unknown array (in no particular order).
 *
```

```
 * Return the array ans of length n representing the unknown array. If multiple answers exist,
 * return any of them.
 *
 * An array sub is a subset of an array arr if sub can be obtained from arr by deleting some
 * (possibly zero or all) elements of arr. The sum of the elements in sub is one possible
 * subset sum of arr. The sum of an empty array is considered to be 0.
 *
 * Note: Test cases are generated such that there will always be at least one correct answer.
 */

/**
* @param {number} n
* @param {number[]} sums
* @return {number[]}
*/
var recoverArray = function(n, sums) {
  sums.sort((a, b) => a - b);

  const result = [];
  while (result.length < n) {
    const diff = sums[1] - sums[0];

    const withNum = [];
    const withoutNum = [];
    const freq = new Map();
    for (const sum of sums) {
      freq.set(sum, (freq.get(sum) || 0) + 1);
    }

    for (const sum of sums) {
      if (freq.get(sum) > 0) {
        freq.set(sum, freq.get(sum) - 1);

        if (freq.get(sum + diff) > 0) {
          freq.set(sum + diff, freq.get(sum + diff) - 1);
          withoutNum.push(sum);
          withNum.push(sum + diff);
        } else {
          return [];
        }
      }
    }

    if (withoutNum.includes(0)) {
      result.push(diff);
      sums = withoutNum;
    } else {
      result.push(-diff);
      sums = withNum;
    }
  }

  return result;
};
```

1983 Widest Pair Of Indices With Equal Range Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1983. Widest Pair of Indices With Equal Range Sum
 * https://leetcode.com/problems/widest-pair-of-indices-with-equal-range-sum/
 * Difficulty: Medium
 *
 * You are given two 0-indexed binary arrays nums1 and nums2. Find the widest pair of indices
(i, j)
 * such that i <= j and nums1[i] + nums1[i+1] + ... + nums1[j] == nums2[i] + nums2[i+1] + ...
+
 * nums2[j].
 *
```

```
 * The widest pair of indices is the pair with the largest distance between i and j. The
distance
 * between a pair of indices is defined as j - i + 1.
 *
 * Return the distance of the widest pair of indices. If no pair of indices meets the
conditions,
 * return 0.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var widestPairOfIndices = function(nums1, nums2) {
  const n = nums1.length;
  const map = new Map();
  map.set(0, -1);

  let prefixDiff = 0;
  let result = 0;

  for (let i = 0; i < n; i++) {
    prefixDiff += nums1[i] - nums2[i];

    if (map.has(prefixDiff)) {
      const distance = i - map.get(prefixDiff);
      result = Math.max(result, distance);
    } else {
      map.set(prefixDiff, i);
    }
  }

  return result;
};
```

1984 Minimum Difference Between Highest And Lowest Of K Scores

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1984. Minimum Difference Between Highest and Lowest of K Scores
 * https://leetcode.com/problems/minimum-difference-between-highest-and-lowest-of-k-scores/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums, where nums[i] represents the score of the
 * ith student. You are also given an integer k.
 *
 * Pick the scores of any k students from the array so that the difference between the highest
 * and the lowest of the k scores is minimized.
 *
 * Return the minimum possible difference.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minimumDifference = function(nums, k) {
  nums.sort((a, b) => a - b);
  let result = Infinity;

  for (let i = 0; i <= nums.length - k; i++) {
    result = Math.min(result, nums[i + k - 1] - nums[i]);
  }

  return result;
};
```

1985 Find The Kth Largest Integer In The Array

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1985. Find the Kth Largest Integer in the Array
 * https://leetcode.com/problems/find-the-kth-largest-integer-in-the-array/
 * Difficulty: Medium
 *
 * You are given an array of strings nums and an integer k. Each string in nums
 * represents an integer without leading zeros.
 *
 * Return the string that represents the kth largest integer in nums.
 *
 * Note: Duplicate numbers should be counted distinctly. For example, if nums is
 * ["1","2","2"], "2" is the first largest integer, "2" is the second-largest
 * integer, and "1" is the third-largest integer.
 */

/**
 * @param {string[]} nums
 * @param {number} k
 * @return {string}
 */
var kthLargestNumber = function(nums, k) {
  return String(nums.map(BigInt).sort((a, b) => b - a >= 0 ? 1 : -1)[k - 1]);
};
```

1986 Minimum Number Of Work Sessions To Finish The Tasks

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1986. Minimum Number of Work Sessions to Finish the Tasks
 * https://leetcode.com/problems/minimum-number-of-work-sessions-to-finish-the-tasks/
 * Difficulty: Medium
 *
 * There are n tasks assigned to you. The task times are represented as an integer array tasks
 of
 * length n, where the ith task takes tasks[i] hours to finish. A work session is when you
work
 * for at most sessionTime consecutive hours and then take a break.
 *
 * You should finish the given tasks in a way that satisfies the following conditions:
 * - If you start a task in a work session, you must complete it in the same work session.
 * - You can start a new task immediately after finishing the previous one.
 * - You may complete the tasks in any order.
 *
 * Given tasks and sessionTime, return the minimum number of work sessions needed to finish
all the
 * tasks following the conditions above.
 *
 * The tests are generated such that sessionTime is greater than or equal to the maximum
element
 * in tasks[i].
 */

/**
 * @param {number[]} tasks
 * @param {number} sessionTime
 * @return {number}
 */
var minSessions = function(tasks, sessionTime) {
  const n = tasks.length;
  const dp = new Array(1 << n).fill(n + 1);
```

```
  dp[0] = 0;

  for (let mask = 1; mask < 1 << n; mask++) {
    let time = 0;
    for (let i = 0; i < n; i++) {
      if (mask & (1 << i)) {
        time += tasks[i];
      }
    }

    for (let subset = mask; subset; subset = (subset - 1) & mask) {
      if (subset === mask) continue;
      let subsetTime = 0;
      for (let i = 0; i < n; i++) {
        if (subset & (1 << i)) {
          subsetTime += tasks[i];
        }
      }
      if (subsetTime <= sessionTime) {
        dp[mask] = Math.min(dp[mask], dp[mask ^ subset] + 1);
      }
    }

    if (time <= sessionTime) {
      dp[mask] = 1;
    }
  }

  return dp[(1 << n) - 1];
};
```

1987 Number Of Unique Good Subsequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1987. Number of Unique Good Subsequences
 * https://leetcode.com/problems/number-of-unique-good-subsequences/
 * Difficulty: Hard
 *
 * You are given a binary string binary. A subsequence of binary is considered good if it is
not
 * empty and has no leading zeros (with the exception of "0").
 *
 * Find the number of unique good subsequences of binary.
 * - For example, if binary = "001", then all the good subsequences are ["0", "0", "1"], so
the
 *   unique good subsequences are "0" and "1". Note that subsequences "00", "01", and "001"
are
 *   not good because they have leading zeros.
 *
 * Return the number of unique good subsequences of binary. Since the answer may be very
large,
 * return it modulo 109 + 7.
 *
 * A subsequence is a sequence that can be derived from another sequence by deleting some or
no
 * elements without changing the order of the remaining elements.
 */

/**
 * @param {string} binary
 * @return {number}
 */
var numberOfUniqueGoodSubsequences = function(binary) {
  const MOD = 1e9 + 7;
  let endsWithZero = 0;
  let endsWithOne = 0;
  let hasZero = false;
```

```
  for (const digit of binary) {
    if (digit === '0') {
      hasZero = true;
      endsWithZero = (endsWithZero + endsWithOne) % MOD;
    } else {
      endsWithOne = (endsWithOne + endsWithZero + 1) % MOD;
    }
  }

  return (endsWithZero + endsWithOne + (hasZero ? 1 : 0)) % MOD;
};
```

1989 Maximum Number Of People That Can Be Caught In Tag

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1989. Maximum Number of People That Can Be Caught in Tag
 * https://leetcode.com/problems/maximum-number-of-people-that-can-be-caught-in-tag/
 * Difficulty: Medium
 *
 * You are playing a game of tag with your friends. In tag, people are divided into two
 * teams: people who are "it", and people who are not "it". The people who are "it" want
 * to catch as many people as possible who are not "it".
 *
 * You are given a 0-indexed integer array team containing only zeros (denoting people who
 * are not "it") and ones (denoting people who are "it"), and an integer dist. A person who
 * is "it" at index i can catch any one person whose index is in the range [i - dist, i +
dist]
 * (inclusive) and is not "it".
 *
 * Return the maximum number of people that the people who are "it" can catch.
 */

/**
 * @param {number[]} team
 * @param {number} dist
 * @return {number}
 */
var catchMaximumAmountofPeople = function(team, dist) {
  const catchers = [];
  const targets = [];

  for (let i = 0; i < team.length; i++) {
    if (team[i] === 1) {
      catchers.push(i);
    } else {
      targets.push(i);
    }
  }

  let result = 0;
  let targetIndex = 0;
  for (const catcherPos of catchers) {
    while (targetIndex < targets.length && targets[targetIndex] < catcherPos - dist) {
      targetIndex++;
    }

    if (targetIndex < targets.length && targets[targetIndex] <= catcherPos + dist) {
      result++;
      targetIndex++;
    }
  }

  return result;
};
```

1991 Find The Middle Index In Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1991. Find the Middle Index in Array
 * https://leetcode.com/problems/find-the-middle-index-in-array/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums, find the leftmost middleIndex (i.e., the smallest
amongst
 * all the possible ones).
 *
 * A middleIndex is an index where nums[0] + nums[1] + ... +
 * nums[middleIndex-1] == nums[middleIndex+1] + nums[middleIndex+2] + ... +
nums[nums.length-1].
 *
 * If middleIndex == 0, the left side sum is considered to be 0. Similarly, if
 * middleIndex == nums.length - 1, the right side sum is considered to be 0.
 *
 * Return the leftmost middleIndex that satisfies the condition, or -1 if there is no such
index.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMiddleIndex = function(nums) {
  let left = 0;
  let right = nums.reduce((sum, n) => sum + n, 0);

  for (let i = 0; i < nums.length; i++) {
    left += nums[i];
    right -= nums[i];
    if (left - nums[i] === right) {
      return i;
    }
  }

  return -1;
};
```

1992 Find All Groups Of Farmland

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1992. Find All Groups of Farmland
 * https://leetcode.com/problems/find-all-groups-of-farmland/
 * Difficulty: Medium
 *
 * You are given a 0-indexed m x n binary matrix land where a 0 represents a hectare of
forested
 * land and a 1 represents a hectare of farmland.
 *
 * To keep the land organized, there are designated rectangular areas of hectares that consist
 * entirely of farmland. These rectangular areas are called groups. No two groups are
adjacent,
 * meaning farmland in one group is not four-directionally adjacent to another farmland in a
 * different group.
 *
 * land can be represented by a coordinate system where the top left corner of land is (0, 0)
 * and the bottom right corner of land is (m-1, n-1). Find the coordinates of the top left and
 * bottom right corner of each group of farmland. A group of farmland with a top left corner
```

```
 * at (r1, c1) and a bottom right corner at (r2, c2) is represented by the 4-length
 * array [r1, c1, r2, c2].
 *
 * Return a 2D array containing the 4-length arrays described above for each group of farmland
 * in land. If there are no groups of farmland, return an empty array. You may return the
answer
 * in any order.
 */

/**
 * @param {number[][]} land
 * @return {number[][]}
 */
var findFarmland = function(land) {
  const m = land.length;
  const n = land[0].length;
  const result = [];

  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (land[r][c] === 1) {
        let r2 = r;
        let c2 = c;

        while (r2 + 1 < m && land[r2 + 1][c] === 1) r2++;
        while (c2 + 1 < n && land[r][c2 + 1] === 1) c2++;

        result.push([r, c, r2, c2]);

        for (let i = r; i <= r2; i++) {
          for (let j = c; j <= c2; j++) {
            land[i][j] = 0;
          }
        }
      }
    }
  }

  return result;
};
```

1993 Operations On Tree

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1993. Operations on Tree
 * https://leetcode.com/problems/operations-on-tree/
 * Difficulty: Medium
 *
 * You are given a tree with n nodes numbered from 0 to n - 1 in the form of a parent array
 * parent where parent[i] is the parent of the ith node. The root of the tree is node 0,
 * so parent[0] = -1 since it has no parent. You want to design a data structure that
 * allows users to lock, unlock, and upgrade nodes in the tree.
 *
 * The data structure should support the following functions:
 * - Lock: Locks the given node for the given user and prevents other users from locking
 *   the same node. You may only lock a node using this function if the node is unlocked.
 * - Unlock: Unlocks the given node for the given user. You may only unlock a node using
 *   this function if it is currently locked by the same user.
 * - Upgrade: Locks the given node for the given user and unlocks all of its descendants
 *   regardless of who locked it. You may only upgrade a node if all 3 conditions are true:
 *   - The node is unlocked,
 *   - It has at least one locked descendant (by any user), and
 *   - It does not have any locked ancestors.
 *
 * Implement the LockingTree class:
 * - LockingTree(int[] parent) initializes the data structure with the parent array.
 * - lock(int num, int user) returns true if it is possible for the user with id user to lock
```

```
 *   the node num, or false otherwise. If it is possible, the node num will become locked by
 *   the user with id user.
 * - unlock(int num, int user) returns true if it is possible for the user with id user to
 *   unlock the node num, or false otherwise. If it is possible, the node num will become
unlocked.
 * - upgrade(int num, int user) returns true if it is possible for the user with id user to
 *   upgrade the node num, or false otherwise. If it is possible, the node num will be
upgraded.
 */

/**
 * @param {number[]} parent
 */
var LockingTree = function(parent) {
  this.parent = parent;
  this.locked = new Map();
  this.children = new Array(parent.length).fill().map(() => []);
  for (let i = 1; i < parent.length; i++) {
    this.children[parent[i]].push(i);
  }
};

/**
 * @param {number} num
 * @param {number} user
 * @return {boolean}
 */
LockingTree.prototype.lock = function(num, user) {
  if (this.locked.has(num)) return false;
  this.locked.set(num, user);
  return true;
};

/**
 * @param {number} num
 * @param {number} user
 * @return {boolean}
 */
LockingTree.prototype.unlock = function(num, user) {
  if (!this.locked.has(num) || this.locked.get(num) !== user) return false;
  this.locked.delete(num);
  return true;
};

/**
 * @param {number} num
 * @param {number} user
 * @return {boolean}
 */
LockingTree.prototype.upgrade = function(num, user) {
  if (this.locked.has(num)) return false;

  let node = num;
  while (node !== -1) {
    if (this.locked.has(node)) return false;
    node = this.parent[node];
  }

  const descendants = [];
  const queue = [num];
  let hasLockedDescendant = false;

  while (queue.length) {
    const curr = queue.shift();
    if (this.locked.has(curr)) {
      hasLockedDescendant = true;
      descendants.push(curr);
    }
    queue.push(...this.children[curr]);
  }

  if (!hasLockedDescendant) return false;

  for (const descendant of descendants) {
    this.locked.delete(descendant);
```

```
  }
  this.locked.set(num, user);
  return true;
};
```

1994 The Number Of Good Subsets

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1994. The Number of Good Subsets
 * https://leetcode.com/problems/the-number-of-good-subsets/
 * Difficulty: Hard
 *
 * You are given an integer array nums. We call a subset of nums good if its product can be
 * represented as a product of one or more distinct prime numbers.
 *
 * - For example, if nums = [1, 2, 3, 4]:
 *   - [2, 3], [1, 2, 3], and [1, 3] are good subsets with products 6 = 2*3, 6 = 2*3, and 3 =
3
 *     respectively.
 *   - [1, 4] and [4] are not good subsets with products 4 = 2*2 and 4 = 2*2 respectively.
 *
 * Return the number of different good subsets in nums modulo 109 + 7.
 *
 * A subset of nums is any array that can be obtained by deleting some (possibly none or all)
 * elements from nums. Two subsets are different if and only if the chosen indices to delete
 * are different.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var numberOfGoodSubsets = function(nums) {
  const MOD = 1e9 + 7;
  const primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29];
  const freq = new Array(31).fill(0);
  for (const num of nums) {
    freq[num]++;
  }

  const dp = new Array(1 << primes.length).fill(0);
  dp[0] = 1;
  for (let i = 0; i < freq[1]; i++) {
    dp[0] = (dp[0] * 2) % MOD;
  }

  for (let num = 2; num <= 30; num++) {
    if (freq[num] === 0) continue;
    let mask = 0;
    let valid = true;
    for (let i = 0; i < primes.length; i++) {
      let count = 0;
      let temp = num;
      while (temp % primes[i] === 0) {
        count++;
        temp /= primes[i];
      }
      if (count > 1) {
        valid = false;
        break;
      }
      if (count === 1) {
        mask |= 1 << i;
      }
    }

    if (!valid) continue;
```

```
    const prev = dp.slice();
    for (let j = 0; j < 1 << primes.length; j++) {
      if ((j & mask) === 0) {
        dp[j | mask] = (dp[j | mask] + prev[j] * freq[num]) % MOD;
      }
    }
  }

  let result = 0;
  for (let i = 1; i < 1 << primes.length; i++) {
    result = (result + dp[i]) % MOD;
  }

  return result;
};
```

1995 Count Special Quadruplets

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1995. Count Special Quadruplets
 * https://leetcode.com/problems/count-special-quadruplets/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums, return the number of distinct quadruplets
 * (a, b, c, d) such that:
 * - nums[a] + nums[b] + nums[c] == nums[d], and
 * - a < b < c < d
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countQuadruplets = function(nums) {
  const map = new Map();
  let result = 0;

  for (let c = nums.length - 2; c >= 2; c--) {
    map.set(nums[c + 1], (map.get(nums[c + 1]) || 0) + 1);
    for (let a = 0; a < c - 1; a++) {
      for (let b = a + 1; b < c; b++) {
        const sum = nums[a] + nums[b] + nums[c];
        result += map.get(sum) || 0;
      }
    }
  }

  return result;
};
```

1996 The Number Of Weak Characters In The Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 1996. The Number of Weak Characters in the Game
 * https://leetcode.com/problems/the-number-of-weak-characters-in-the-game/
 * Difficulty: Medium
 *
 * You are playing a game that contains multiple characters, and each of the
 * characters has two main properties: attack and defense. You are given a 2D
 * integer array properties where properties[i] = [attacki, defensei] represents
 * the properties of the ith character in the game.
```

```
 *
 * A character is said to be weak if any other character has both attack and
 * defense levels strictly greater than this character's attack and defense levels.
 * More formally, a character i is said to be weak if there exists another character
 * j where attackj > attacki and defensej > defensei.
 *
 * Return the number of weak characters.
 */

/**
 * @param {number[][]} properties
 * @return {number}
 */
var numberOfWeakCharacters = function(properties) {
  let result = 0;
  let max = 0;

  properties.sort((a, b) => (b[0] === a[0]) ? a[1] - b[1] : b[0] - a[0]);

  for (let i = 0; i < properties.length; i++) {
    if (properties[i][1] < max) result++;
    max = Math.max(max, properties[i][1]);
  }

  return result;
};
```

1997 First Day Where You Have Been In All The Rooms

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 1997. First Day Where You Have Been in All the Rooms
 * https://leetcode.com/problems/first-day-where-you-have-been-in-all-the-rooms/
 * Difficulty: Medium
 *
 * There are n rooms you need to visit, labeled from 0 to n - 1. Each day is labeled, starting
 * from 0. You will go in and visit one room a day.
 *
 * Initially on day 0, you visit room 0. The order you visit the rooms for the coming days is
 * determined by the following rules and a given 0-indexed array nextVisit of length n:
 * - Assuming that on a day, you visit room i,
 * - if you have been in room i an odd number of times (including the current visit), on the
next
 *   day you will visit a room with a lower or equal room number specified by nextVisit[i]
 *   where 0 <= nextVisit[i] <= i;
 * - if you have been in room i an even number of times (including the current visit), on the
 *   next day you will visit room (i + 1) mod n.
 *
 * Return the label of the first day where you have been in all the rooms. It can be shown
 * that such a day exists. Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nextVisit
 * @return {number}
 */
var firstDayBeenInAllRooms = function(nextVisit) {
  const MOD = 1e9 + 7;
  const n = nextVisit.length;
  const dp = new Array(n).fill(0);

  for (let i = 1; i < n; i++) {
    dp[i] = (2 * dp[i - 1] + 2 - dp[nextVisit[i - 1]] + MOD) % MOD;
  }

  return dp[n - 1];
};
```

1998 Gcd Sort Of An Array

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1998. GCD Sort of an Array
 * https://leetcode.com/problems/gcd-sort-of-an-array/
 * Difficulty: Hard
 *
 * You are given an integer array nums, and you can perform the following operation any
 * number of times on nums:
 * - Swap the positions of two elements nums[i] and nums[j] if gcd(nums[i], nums[j]) > 1
 *   where gcd(nums[i], nums[j]) is the greatest common divisor of nums[i] and nums[j].
 *
 * Return true if it is possible to sort nums in non-decreasing order using the above swap
 * method, or false otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var gcdSort = function(nums) {
  const maxNum = Math.max(...nums);
  const parent = new Array(maxNum + 1).fill().map((_, i) => i);
  const minPrime = new Array(maxNum + 1).fill(0);
  for (let i = 2; i <= maxNum; i++) {
    if (minPrime[i] === 0) {
      for (let j = i; j <= maxNum; j += i) {
        minPrime[j] = i;
      }
    }
  }

  const groups = new Map();
  for (const num of nums) {
    let curr = num;
    const primes = new Set();
    while (curr > 1) {
      const prime = minPrime[curr];
      primes.add(prime);
      curr /= prime;
    }
    for (const prime of primes) {
      if (!groups.has(prime)) groups.set(prime, []);
      groups.get(prime).push(num);
    }
  }

  for (const numbers of groups.values()) {
    for (let i = 1; i < numbers.length; i++) {
      union(numbers[i - 1], numbers[i], parent);
    }
  }

  const sorted = [...nums].sort((a, b) => a - b);
  for (let i = 0; i < nums.length; i++) {
    if (find(nums[i], parent) !== find(sorted[i], parent)) {
      return false;
    }
  }

  return true;

  function find(x, parent) {
    if (parent[x] !== x) parent[x] = find(parent[x], parent);
    return parent[x];
  }

  function union(x, y, parent) {
```

```
    parent[find(x, parent)] = find(y, parent);
  }
};
```

1999 Smallest Greater Multiple Made Of Two Digits

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 1999. Smallest Greater Multiple Made of Two Digits
 * https://leetcode.com/problems/smallest-greater-multiple-made-of-two-digits/
 * Difficulty: Medium
 *
 * Given three integers, k, digit1, and digit2, you want to find the smallest integer that is:
 * - Larger than k,
 * - A multiple of k, and
 * - Comprised of only the digits digit1 and/or digit2.
 *
 * Return the smallest such integer. If no such integer exists or the integer exceeds the
limit
 * of a signed 32-bit integer (231 - 1), return -1.
 */

/**
 * @param {number} k
 * @param {number} digit1
 * @param {number} digit2
 * @return {number}
 */
var findInteger = function(k, digit1, digit2) {
  const minDigit = Math.min(digit1, digit2);
  const maxDigit = Math.max(digit1, digit2);
  const queue = minDigit !== maxDigit ? [minDigit, maxDigit] : [minDigit];

  while (queue.length > 0) {
    const current = queue.shift();

    if (current === 0 || current > 2147483647) {
      continue;
    }

    if (current > k && current % k === 0) {
      return current;
    }

    queue.push(current * 10 + minDigit);
    if (minDigit !== maxDigit) {
      queue.push(current * 10 + maxDigit);
    }
  }

  return -1;
};
```

2000 Reverse Prefix Of Word

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2000. Reverse Prefix of Word
 * https://leetcode.com/problems/reverse-prefix-of-word/
 * Difficulty: Easy
 *
 * Given a 0-indexed string word and a character ch, reverse the segment of word that
 * starts at index 0 and ends at the index of the first occurrence of ch (inclusive).
```

```
 * If the character ch does not exist in word, do nothing.
 *
 * For example, if word = "abcdefd" and ch = "d", then you should reverse the segment
 * that starts at 0 and ends at 3 (inclusive). The resulting string will be "dcbaefd".
 */

/**
 * @param {string} word
 * @param {character} ch
 * @return {string}
 */
var reversePrefix = function(word, ch) {
  const index = word.indexOf(ch) + 1;
  return word.slice(0, index).split('').reverse().join('') + word.slice(index);
};
```

2001 Number Of Pairs Of Interchangeable Rectangles

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2001. Number of Pairs of Interchangeable Rectangles
 * https://leetcode.com/problems/number-of-pairs-of-interchangeable-rectangles/
 * Difficulty: Medium
 *
 * You are given n rectangles represented by a 0-indexed 2D integer array rectangles,
 * where rectangles[i] = [widthi, heighti] denotes the width and height of the ith rectangle.
 *
 * Two rectangles i and j (i < j) are considered interchangeable if they have the same
 * width-to-height ratio. More formally, two rectangles are interchangeable if
 * widthi/heighti == widthj/heightj (using decimal division, not integer division).
 *
 * Return the number of pairs of interchangeable rectangles in rectangles.
 */

/**
 * @param {number[][]} rectangles
 * @return {number}
 */
var interchangeableRectangles = function(rectangles) {
  const map = new Map();
  let result = 0;

  for (const [width, height] of rectangles) {
    const gcd = (a, b) => b === 0 ? a : gcd(b, a % b);
    const divisor = gcd(width, height);
    const ratio = `${width / divisor}/${height / divisor}`;

    if (map.has(ratio)) {
      result += map.get(ratio);
      map.set(ratio, map.get(ratio) + 1);
    } else {
      map.set(ratio, 1);
    }
  }

  return result;
};
```

2002 Maximum Product Of The Length Of Two Palindromic Subsequences

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2002. Maximum Product of the Length of Two Palindromic Subsequences
```

```
 *
https://leetcode.com/problems/maximum-product-of-the-length-of-two-palindromic-subsequences/
 * Difficulty: Medium
 *
 * Given a string s, find two disjoint palindromic subsequences of s such that the product of
their
 * lengths is maximized. The two subsequences are disjoint if they do not both pick a
character at
 * the same index.
 *
 * Return the maximum possible product of the lengths of the two palindromic subsequences.
 *
 * A subsequence is a string that can be derived from another string by deleting some or no
 * characters without changing the order of the remaining characters. A string is palindromic
 * if it reads the same forward and backward.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxProduct = function(s) {
  const n = s.length;
  let result = 0;

  for (let mask1 = 1; mask1 < (1 << n); mask1++) {
    for (let mask2 = 1; mask2 < (1 << n); mask2++) {
      if (mask1 & mask2) continue;
      const len1 = isPalindrome(s, mask1);
      if (len1 === 0) continue;
      const len2 = isPalindrome(s, mask2);
      if (len2 === 0) continue;
      result = Math.max(result, len1 * len2);
    }
  }

  return result;

  function isPalindrome(str, mask) {
    const chars = [];
    for (let i = 0; i < n; i++) {
      if (mask & (1 << i)) chars.push(str[i]);
    }
    let left = 0;
    let right = chars.length - 1;
    while (left < right) {
      if (chars[left++] !== chars[right--]) return 0;
    }
    return chars.length;
  }
};
```

2003 Smallest Missing Genetic Value In Each Subtree

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2003. Smallest Missing Genetic Value in Each Subtree
 * https://leetcode.com/problems/smallest-missing-genetic-value-in-each-subtree/
 * Difficulty: Hard
 *
 * There is a family tree rooted at 0 consisting of n nodes numbered 0 to n - 1. You are given
 a
 * 0-indexed integer array parents, where parents[i] is the parent for node i. Since node 0 is
 * the root, parents[0] == -1.
 *
 * There are 105 genetic values, each represented by an integer in the inclusive range [1,
105].
 * You are given a 0-indexed integer array nums, where nums[i] is a distinct genetic value for
```

```
 * node i.
 *
 * Return an array ans of length n where ans[i] is the smallest genetic value that is missing
 * from the subtree rooted at node i.
 *
 * The subtree rooted at a node x contains node x and all of its descendant nodes.
 */

/**
 * @param {number[]} parents
 * @param {number[]} nums
 * @return {number[]}
 */
var smallestMissingValueSubtree = function(parents, nums) {
  const n = parents.length;
  const result = new Array(n).fill(1);
  const children = Array.from({ length: n }, () => []);
  const seen = new Set();
  let maxMissing = 1;

  for (let i = 1; i < n; i++) {
    children[parents[i]].push(i);
  }

  const nodeWithOne = nums.indexOf(1);
  if (nodeWithOne === -1) return result;

  let current = nodeWithOne;
  while (current !== -1) {
    const stack = [current];
    while (stack.length) {
      const node = stack.pop();
      seen.add(nums[node]);
      for (const child of children[node]) {
        if (!seen.has(nums[child])) stack.push(child);
      }
    }
    while (seen.has(maxMissing)) maxMissing++;
    result[current] = maxMissing;
    current = parents[current];
  }

  return result;
};
```

2005 Subtree Removal Game With Fibonacci Tree

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2005. Subtree Removal Game with Fibonacci Tree
 * https://leetcode.com/problems/subtree-removal-game-with-fibonacci-tree/
 * Difficulty: Hard
 *
 * A Fibonacci tree is a binary tree created using the order function order(n):
 * - order(0) is the empty tree.
 * - order(1) is a binary tree with only one node.
 * - order(n) is a binary tree that consists of a root node with the left subtree as order(n -
 2)
 *   and the right subtree as order(n - 1).
 *
 * Alice and Bob are playing a game with a Fibonacci tree with Alice staring first. On each
turn,
 * a player selects a node and removes that node and its subtree. The player that is forced to
 * delete root loses.
 *
 * Given the integer n, return true if Alice wins the game or false if Bob wins, assuming both
 * players play optimally.
 *
 * A subtree of a binary tree tree is a tree that consists of a node in tree and all of this
```

```
 * node's descendants. The tree tree could also be considered as a subtree of itself.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var findGameWinner = function(n) {
  return n % 6 !== 1;
};
```

2006 Count Number Of Pairs With Absolute Difference K

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2006. Count Number of Pairs With Absolute Difference K
 * https://leetcode.com/problems/count-number-of-pairs-with-absolute-difference-k/
 * Difficulty: Easy
 *
 * Given an integer array nums and an integer k, return the number of pairs (i, j) where
 * i < j such that |nums[i] - nums[j]| == k.
 *
 * The value of |x| is defined as:
 * - x if x >= 0.
 * - -x if x < 0.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countKDifference = function(nums, k) {
  const map = new Map();
  let result = 0;

  for (const num of nums) {
    result += (map.get(num - k) || 0) + (map.get(num + k) || 0);
    map.set(num, (map.get(num) || 0) + 1);
  }

  return result;
};
```

2007 Find Original Array From Doubled Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2007. Find Original Array From Doubled Array
 * https://leetcode.com/problems/find-original-array-from-doubled-array/
 * Difficulty: Medium
 *
 * An integer array original is transformed into a doubled array changed by appending twice
the
 * value of every element in original, and then randomly shuffling the resulting array.
 *
 * Given an array changed, return original if changed is a doubled array. If changed is not a
 * doubled array, return an empty array. The elements in original may be returned in any
order.
 */

/**
```

```
 * @param {number[]} changed
 * @return {number[]}
 */
var findOriginalArray = function(changed) {
  if (changed.length % 2 !== 0) return [];

  const frequency = new Map();
  const result = [];

  changed.sort((a, b) => a - b);

  for (const num of changed) {
    frequency.set(num, (frequency.get(num) || 0) + 1);
  }

  for (const num of changed) {
    if (frequency.get(num) === 0) continue;

    frequency.set(num, frequency.get(num) - 1);

    const doubled = num * 2;
    if (!frequency.has(doubled) || frequency.get(doubled) === 0) return [];

    frequency.set(doubled, frequency.get(doubled) - 1);
    result.push(num);
  }

  return result;
};
```

2008 Maximum Earnings From Taxi

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2008. Maximum Earnings From Taxi
 * https://leetcode.com/problems/maximum-earnings-from-taxi/
 * Difficulty: Medium
 *
 * There are n points on a road you are driving your taxi on. The n points on the road are
 * labeled from 1 to n in the direction you are going, and you want to drive from point 1
 * to point n to make money by picking up passengers. You cannot change the direction of the
taxi.
 *
 * The passengers are represented by a 0-indexed 2D integer array rides, where
 * rides[i] = [starti, endi, tipi] denotes the ith passenger requesting a ride from point
 * starti to point endi who is willing to give a tipi dollar tip.
 *
 * For each passenger i you pick up, you earn endi - starti + tipi dollars. You may only drive
 * at most one passenger at a time.
 *
 * Given n and rides, return the maximum number of dollars you can earn by picking up the
 * passengers optimally.
 *
 * Note: You may drop off a passenger and pick up a different passenger at the same point.
 */

/**
 * @param {number} n
 * @param {number[][]} rides
 * @return {number}
 */
var maxTaxiEarnings = function(n, rides) {
  rides.sort((a, b) => a[1] - b[1]);
  const dp = new Array(n + 1).fill(0);
  let rideIndex = 0;

  for (let point = 1; point <= n; point++) {
    dp[point] = dp[point - 1];
```

```
    while (rideIndex < rides.length && rides[rideIndex][1] === point) {
      const [start, end, tip] = rides[rideIndex];
      dp[point] = Math.max(dp[point], dp[start] + (end - start + tip));
      rideIndex++;
    }
  }

  return dp[n];
};
```

2009 Minimum Number Of Operations To Make Array Continuous

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2009. Minimum Number of Operations to Make Array Continuous
 * https://leetcode.com/problems/minimum-number-of-operations-to-make-array-continuous/
 * Difficulty: Hard
 *
 * You are given an integer array nums. In one operation, you can replace any element in nums
 * with any integer.
 *
 * nums is considered continuous if both of the following conditions are fulfilled:
 * - All elements in nums are unique.
 * - The difference between the maximum element and the minimum element in nums equals
 *   nums.length - 1.
 *
 * For example, nums = [4, 2, 5, 3] is continuous, but nums = [1, 2, 3, 5, 6] is not
continuous.
 *
 * Return the minimum number of operations to make nums continuous.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minOperations = function(nums) {
  const n = nums.length;
  const uniqueNums = [...new Set(nums)].sort((a, b) => a - b);
  let result = n;

  for (let i = 0, j = 0; i < uniqueNums.length; i++) {
    while (j < uniqueNums.length && uniqueNums[j] - uniqueNums[i] <= n - 1) {
      j++;
    }
    result = Math.min(result, n - (j - i));
  }

  return result;
};
```

2011 Final Value Of Variable After Performing Operations

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2011. Final Value of Variable After Performing Operations
 * https://leetcode.com/problems/final-value-of-variable-after-performing-operations/
 * Difficulty: Easy
 *
 * There is a programming language with only four operations and one variable X:
 * - ++X and X++ increments the value of the variable X by 1.
 * - --X and X-- decrements the value of the variable X by 1.
```

```
 *
 * Initially, the value of X is 0.
 *
 * Given an array of strings operations containing a list of operations, return the
 * final value of X after performing all the operations.
 */

/**
 * @param {string[]} operations
 * @return {number}
 */
var finalValueAfterOperations = function(operations) {
  return operations
    .map(n => n.includes('++') ? 1 : -1)
    .reduce((sum, n) => sum + n, 0);
};
```

2012 Sum Of Beauty In The Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2012. Sum of Beauty in the Array
 * https://leetcode.com/problems/sum-of-beauty-in-the-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. For each index i (1 <= i <= nums.length - 2)
 * the beauty of nums[i] equals:
 * - 2, if nums[j] < nums[i] < nums[k], for all 0 <= j < i and for all i < k <= nums.length -
1.
 * - 1, if nums[i - 1] < nums[i] < nums[i + 1], and the previous condition is not satisfied.
 * - 0, if none of the previous conditions holds.
 *
 * Return the sum of beauty of all nums[i] where 1 <= i <= nums.length - 2.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumOfBeauties = function(nums) {
  const n = nums.length;
  let result = 0;
  const leftMax = new Array(n).fill(nums[0]);
  const rightMin = new Array(n).fill(nums[n - 1]);

  for (let i = 1; i < n; i++) {
    leftMax[i] = Math.max(leftMax[i - 1], nums[i - 1]);
  }

  for (let i = n - 2; i >= 0; i--) {
    rightMin[i] = Math.min(rightMin[i + 1], nums[i + 1]);
  }

  for (let i = 1; i < n - 1; i++) {
    if (leftMax[i] < nums[i] && nums[i] < rightMin[i]) {
      result += 2;
    } else if (nums[i - 1] < nums[i] && nums[i] < nums[i + 1]) {
      result += 1;
    }
  }

  return result;
};
```

2013 Detect Squares

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2013. Detect Squares
 * https://leetcode.com/problems/detect-squares/
 * Difficulty: Medium
 *
 * You are given a stream of points on the X-Y plane. Design an algorithm that:
 * - Adds new points from the stream into a data structure. Duplicate points are allowed and
should
 *   be treated as different points.
 * - Given a query point, counts the number of ways to choose three points from the data
structure
 *   such that the three points and the query point form an axis-aligned square with positive
area.
 *
 * An axis-aligned square is a square whose edges are all the same length and are either
parallel
 * or perpendicular to the x-axis and y-axis.
 *
 * Implement the DetectSquares class:
 * - DetectSquares() Initializes the object with an empty data structure.
 * - void add(int[] point) Adds a new point point = [x, y] to the data structure.
 * - int count(int[] point) Counts the number of ways to form axis-aligned squares with point
 *   point = [x, y] as described above.
 */

var DetectSquares = function() {
  this.points = new Map();
};

/**
 * @param {number[]} point
 * @return {void}
 */
DetectSquares.prototype.add = function(point) {
  const [x, y] = point;
  const key = `${x},${y}`;
  this.points.set(key, (this.points.get(key) || 0) + 1);
};

/**
 * @param {number[]} point
 * @return {number}
 */
DetectSquares.prototype.count = function(point) {
  const [x, y] = point;
  let result = 0;

  for (const [key, count] of this.points) {
    const [px, py] = key.split(',').map(Number);
    if (px === x || py === y || Math.abs(px - x) !== Math.abs(py - y)) continue;

    const point1 = `${x},${py}`;
    const point2 = `${px},${y}`;
    result += count * (this.points.get(point1) || 0) * (this.points.get(point2) || 0);
  }

  return result;
};
```

2014 Longest Subsequence Repeated K Times

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2014. Longest Subsequence Repeated k Times
 * https://leetcode.com/problems/longest-subsequence-repeated-k-times/
 * Difficulty: Hard
 *
 * You are given a string s of length n, and an integer k. You are tasked to find the longest
 * subsequence repeated k times in string s.
 *
 * A subsequence is a string that can be derived from another string by deleting some or no
 * characters without changing the order of the remaining characters.
 *
 * A subsequence seq is repeated k times in the string s if seq * k is a subsequence of s,
 * where seq * k represents a string constructed by concatenating seq k times.
 * - For example, "bba" is repeated 2 times in the string "bababcba", because the string
 *   "bbabba", constructed by concatenating "bba" 2 times, is a subsequence of the
 *   string "bababcba".
 *
 * Return the longest subsequence repeated k times in string s. If multiple such subsequences
 * are found, return the lexicographically largest one. If there is no such subsequence,
 * return an empty string.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var longestSubsequenceRepeatedK = function(s, k) {
  const n = s.length;
  const freq = new Array(26).fill(0);
  for (const char of s) {
    freq[char.charCodeAt(0) - 97]++;
  }

  let candidates = '';
  for (let i = 0; i < 26; i++) {
    const count = Math.floor(freq[i] / k);
    candidates += String.fromCharCode(97 + i).repeat(count);
  }

  function canFormSubsequence(seq) {
    let j = 0;
    for (let i = 0; i < n && j < seq.length * k; i++) {
      if (s[i] === seq[j % seq.length]) j++;
    }
    return j >= seq.length * k;
  }

  let result = '';
  backtrack('', candidates);
  return result;

  function backtrack(curr, remaining) {
    if (curr.length > result.length || (curr.length === result.length && curr > result)) {
      if (canFormSubsequence(curr)) result = curr;
    }
    if (!remaining) return;

    for (let i = remaining.length - 1; i >= 0; i--) {
      const nextChar = remaining[i];
      backtrack(curr + nextChar, remaining.slice(0, i) + remaining.slice(i + 1));
    }
  }
};
```

2015 Average Height Of Buildings In Each Segment

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2015. Average Height of Buildings in Each Segment
 * https://leetcode.com/problems/average-height-of-buildings-in-each-segment/
 * Difficulty: Medium
 *
 * A perfectly straight street is represented by a number line. The street has building(s) on
 * it and is represented by a 2D integer array buildings, where buildings[i] = [starti, endi,
 * heighti]. This means that there is a building with heighti in the half-closed
 * segment [starti, endi).
 *
 * You want to describe the heights of the buildings on the street with the minimum number of
 * non-overlapping segments. The street can be represented by the 2D integer array street
 * where street[j] = [leftj, rightj, averagej] describes a half-closed segment [leftj, rightj)
 * of the road where the average heights of the buildings in the segment is averagej.
 *
 * - For example, if buildings = [[1,5,2],[3,10,4]], the street could be represented by
 *   street = [[1,3,2],[3,5,3],[5,10,4]] because:
 *   - From 1 to 3, there is only the first building with an average height of 2 / 1 = 2.
 *   - From 3 to 5, both the first and the second building are there with an average height
 *     of (2+4) / 2 = 3.
 *   - From 5 to 10, there is only the second building with an average height of 4 / 1 = 4.
 *
 * Given buildings, return the 2D integer array street as described above (excluding any areas
 * of the street where there are no buldings). You may return the array in any order.
 *
 * The average of n elements is the sum of the n elements divided (integer division) by n.
 *
 * A half-closed segment [a, b) is the section of the number line between points a and b
including
 * point a and not including point b.
 */

/**
 * @param {number[][]} buildings
 * @return {number[][]}
 */
var averageHeightOfBuildings = function(buildings) {
  const events = [];

  for (const [start, end, height] of buildings) {
    events.push([start, height, 1]);
    events.push([end, -height, -1]);
  }

  events.sort((a, b) => a[0] - b[0] || a[2] - b[2]);

  const result = [];
  let currentSum = 0;
  let currentCount = 0;
  let prevPos = -1;

  for (const [pos, height, delta] of events) {
    if (currentCount > 0 && pos > prevPos) {
      const avgHeight = Math.floor(currentSum / currentCount);

      if (result.length > 0 && result[result.length - 1][1] === prevPos
          && result[result.length - 1][2] === avgHeight) {
        result[result.length - 1][1] = pos;
      } else {
        result.push([prevPos, pos, avgHeight]);
      }
    }

    currentSum += height;
    currentCount += delta;
    prevPos = pos;
  }

  return result;
};
```

2016 Maximum Difference Between Increasing Elements

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2016. Maximum Difference Between Increasing Elements
 * https://leetcode.com/problems/maximum-difference-between-increasing-elements/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums of size n, find the maximum difference
 * between nums[i] and nums[j] (i.e., nums[j] - nums[i]), such that
 * 0 <= i < j < n and nums[i] < nums[j].
 *
 * Return the maximum difference. If no such i and j exists, return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumDifference = function(nums) {
  let max = 0;

  for (let i = 0, min = nums[0]; i < nums.length; i++) {
    min = Math.min(min, nums[i]);
    max = Math.max(max, nums[i] - min);
  }

  return max === 0 ? -1 : max;
};
```

2017 Grid Game

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2017. Grid Game
 * https://leetcode.com/problems/grid-game/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D array grid of size 2 x n, where grid[r][c] represents the
number
 * of points at position (r, c) on the matrix. Two robots are playing a game on this matrix.
 *
 * Both robots initially start at (0, 0) and want to reach (1, n-1). Each robot may only move
 * to the right ((r, c) to (r, c + 1)) or down ((r, c) to (r + 1, c)).
 *
 * At the start of the game, the first robot moves from (0, 0) to (1, n-1), collecting all the
 * points from the cells on its path. For all cells (r, c) traversed on the path, grid[r][c]
is
 * set to 0. Then, the second robot moves from (0, 0) to (1, n-1), collecting the points on
its
 * path. Note that their paths may intersect with one another.
 *
 * The first robot wants to minimize the number of points collected by the second robot. In
 * contrast, the second robot wants to maximize the number of points it collects. If both
 * robots play optimally, return the number of points collected by the second robot.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var gridGame = function(grid) {
  let result = Infinity;
```

```
  let sum1 = grid[0].reduce((a, b) => a + b, 0);
  let sum2 = 0;

  for (let i = 0; i < grid[0].length; i++) {
    sum1 -= grid[0][i];
    result = Math.min(result, Math.max(sum1, sum2));
    sum2 += grid[1][i];
  }

  return result;
};
```

2018 Check If Word Can Be Placed In Crossword

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2018. Check if Word Can Be Placed In Crossword
 * https://leetcode.com/problems/check-if-word-can-be-placed-in-crossword/
 * Difficulty: Medium
 *
 * You are given an m x n matrix board, representing the current state of a crossword puzzle.
 * The crossword contains lowercase English letters (from solved words), ' ' to represent any
 * empty cells, and '#' to represent any blocked cells.
 *
 * A word can be placed horizontally (left to right or right to left) or vertically (top to
 * bottom or bottom to top) in the board if:
 * - It does not occupy a cell containing the character '#'.
 * - The cell each letter is placed in must either be ' ' (empty) or match the letter already
 *   on the board.
 * - There must not be any empty cells ' ' or other lowercase letters directly left or right
 *   of the word if the word was placed horizontally.
 * - There must not be any empty cells ' ' or other lowercase letters directly above or below
 *   the word if the word was placed vertically.
 *
 * Given a string word, return true if word can be placed in board, or false otherwise.
 */

/**
 * @param {character[][]} board
 * @param {string} word
 * @return {boolean}
 */
var placeWordInCrossword = function(board, word) {
  const rows = board.length;
  const cols = board[0].length;
  const wordLen = word.length;

  function canPlace(row, col, dr, dc) {
    for (let i = 0; i < wordLen; i++) {
      const r = row + i * dr;
      const c = col + i * dc;
      if (r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] === '#') return false;
      if (board[r][c] !== ' ' && board[r][c] !== word[i]) return false;
    }

    const beforeR = row - dr;
    const beforeC = col - dc;
    const afterR = row + wordLen * dr;
    const afterC = col + wordLen * dc;

    if ((beforeR >= 0 && beforeR < rows && beforeC >= 0
        && beforeC < cols && board[beforeR][beforeC] !== '#')
        || (afterR >= 0 && afterR < rows && afterC >= 0
           && afterC < cols && board[afterR][afterC] !== '#')) {
      return false;
    }

    return true;
  }
```

```
  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      if (canPlace(r, c, 0, 1) || canPlace(r, c, 0, -1)
          || canPlace(r, c, 1, 0) || canPlace(r, c, -1, 0)) {
        return true;
      }
    }
  }

  return false;
};
```

2019 The Score Of Students Solving Math Expression

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2019. The Score of Students Solving Math Expression
 * https://leetcode.com/problems/the-score-of-students-solving-math-expression/
 * Difficulty: Hard
 *
 * You are given a string s that contains digits 0-9, addition symbols '+', and multiplication
 * symbols '*' only, representing a valid math expression of single digit numbers (e.g.,
3+5*2).
 * This expression was given to n elementary school students. The students were instructed to
 * get the answer of the expression by following this order of operations:
 * 1. Compute multiplication, reading from left to right; Then,
 * 2. Compute addition, reading from left to right.
 *
 * You are given an integer array answers of length n, which are the submitted answers of the
 * students in no particular order. You are asked to grade the answers, by following these
rules:
 * - If an answer equals the correct answer of the expression, this student will be rewarded
 *   5 points;
 * - Otherwise, if the answer could be interpreted as if the student applied the operators in
 *   the wrong order but had correct arithmetic, this student will be rewarded 2 points;
 * - Otherwise, this student will be rewarded 0 points.
 *
 * Return the sum of the points of the students.
 */

/**
 * @param {string} s
 * @param {number[]} answers
 * @return {number}
 */
var scoreOfStudents = function(s, answers) {
  const correct = eval(s.replace(/(\d)([*+])/g, '$1 $2 '));

  const n = s.length;
  const dp = new Map();

  function compute(start, end) {
    const key = `${start},${end}`;
    if (dp.has(key)) return dp.get(key);

    const results = new Set();
    if (start === end) {
      results.add(Number(s[start]));
      dp.set(key, results);
      return results;
    }

    for (let i = start + 1; i < end; i += 2) {
      const leftResults = compute(start, i - 1);
      const rightResults = compute(i + 1, end);
      const op = s[i];

      for (const left of leftResults) {
```

```
        for (const right of rightResults) {
          const val = op === '+' ? left + right : left * right;
          if (val <= 1000) results.add(val);
        }
      }
    }

    dp.set(key, results);
    return results;
  }

  const wrongAnswers = compute(0, n - 1);
  let result = 0;

  for (const answer of answers) {
    if (answer === correct) result += 5;
    else if (wrongAnswers.has(answer)) result += 2;
  }

  return result;
};
```

2021 Brightest Position On Street

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2021. Brightest Position on Street
 * https://leetcode.com/problems/brightest-position-on-street/
 * Difficulty: Medium
 *
 * A perfectly straight street is represented by a number line. The street has street lamp(s)
 * on it and is represented by a 2D integer array lights. Each lights[i] = [positioni, rangei]
 * indicates that there is a street lamp at position positioni that lights up the area from
 * [positioni - rangei, positioni + rangei] (inclusive).
 *
 * The brightness of a position p is defined as the number of street lamp that light up the
 * position p.
 *
 * Given lights, return the brightest position on the street. If there are multiple brightest
 * positions, return the smallest one.
 */

/**
 * @param {number[][]} lights
 * @return {number}
 */
var brightestPosition = function(lights) {
  const events = [];

  for (const [position, range] of lights) {
    events.push([position - range, 1]);
    events.push([position + range + 1, -1]);
  }

  events.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  let maxBrightness = 0;
  let result = 0;
  let currentBrightness = 0;

  for (const [position, delta] of events) {
    currentBrightness += delta;

    if (currentBrightness > maxBrightness) {
      maxBrightness = currentBrightness;
      result = position;
    }
  }
```

```
  return result;
};
```

2022 Convert 1D Array Into 2D Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2022. Convert 1D Array Into 2D Array
 * https://leetcode.com/problems/convert-1d-array-into-2d-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed 1-dimensional (1D) integer array original, and two integers,
 * m and n. You are tasked with creating a 2-dimensional (2D) array with  m rows and n
 * columns using all the elements from original.
 *
 * The elements from indices 0 to n - 1 (inclusive) of original should form the first row
 * of the constructed 2D array, the elements from indices n to 2 * n - 1 (inclusive) should
 * form the second row of the constructed 2D array, and so on.
 *
 * Return an m x n 2D array constructed according to the above procedure, or an empty 2D array
 * if it is impossible.
 */

/**
 * @param {number[]} original
 * @param {number} m
 * @param {number} n
 * @return {number[][]}
 */
var construct2DArray = function(original, m, n) {
  if (original.length !== m * n) return [];

  const result = [];
  for (let i = 0; i < m; i++) {
    result.push(original.slice(i * n, (i + 1) * n));
  }

  return result;
};
```

2023 Number Of Pairs Of Strings With Concatenation Equal To Target

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2023. Number of Pairs of Strings With Concatenation Equal to Target
 *
https://leetcode.com/problems/number-of-pairs-of-strings-with-concatenation-equal-to-target/
 * Difficulty: Medium
 *
 * Given an array of digit strings nums and a digit string target, return the number of pairs
of
 * indices (i, j) (where i != j) such that the concatenation of nums[i] + nums[j] equals
target.
 */

/**
 * @param {string[]} nums
 * @param {string} target
 * @return {number}
 */
var numOfPairs = function(nums, target) {
  const map = new Map();
  let result = 0;
```

```
  for (const num of nums) {
    if (target.startsWith(num)) {
      const suffix = target.slice(num.length);
      result += map.get(suffix) || 0;
    }
    if (target.endsWith(num)) {
      const prefix = target.slice(0, target.length - num.length);
      result += map.get(prefix) || 0;
    }
    map.set(num, (map.get(num) || 0) + 1);
  }

  return result;
};
```

2024 Maximize The Confusion Of An Exam

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2024. Maximize the Confusion of an Exam
 * https://leetcode.com/problems/maximize-the-confusion-of-an-exam/
 * Difficulty: Medium
 *
 * A teacher is writing a test with n true/false questions, with 'T' denoting true and 'F'
denoting
 * false. He wants to confuse the students by maximizing the number of consecutive questions
with
 * the same answer (multiple trues or multiple falses in a row).
 *
 * You are given a string answerKey, where answerKey[i] is the original answer to the ith
question.
 * In addition, you are given an integer k, the maximum number of times you may perform the
 * following operation:
 * - Change the answer key for any question to 'T' or 'F' (i.e., set answerKey[i] to 'T' or
'F').
 *
 * Return the maximum number of consecutive 'T's or 'F's in the answer key after performing
the
 * operation at most k times.
 */

/**
 * @param {string} answerKey
 * @param {number} k
 * @return {number}
 */
var maxConsecutiveAnswers = function(answerKey, k) {
  const flipsLeft = k;
  const count = { T: 0, F: 0 };
  let result = 0;
  let left = 0;

  for (let right = 0; right < answerKey.length; right++) {
    count[answerKey[right]]++;
    const maxCount = Math.max(count.T, count.F);

    while (right - left + 1 - maxCount > flipsLeft) {
      count[answerKey[left]]--;
      left++;
    }

    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

2025 Maximum Number Of Ways To Partition An Array

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2025. Maximum Number of Ways to Partition an Array
 * https://leetcode.com/problems/maximum-number-of-ways-to-partition-an-array/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums of length n. The number of ways to partition
 * nums is the number of pivot indices that satisfy both conditions:
 * - 1 <= pivot < n
 * - nums[0] + nums[1] + ... + nums[pivot - 1] == nums[pivot] + nums[pivot + 1] + ... + nums[n
 - 1]
 *
 * You are also given an integer k. You can choose to change the value of one element of nums
to
 * k, or to leave the array unchanged.
 *
 * Return the maximum possible number of ways to partition nums to satisfy both conditions
after
 * changing at most one element.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var waysToPartition = function(nums, k) {
  const n = nums.length;
  const prefixSums = [nums[0]];
  const leftDiffs = new Map();
  const rightDiffs = new Map();

  for (let i = 1; i < n; i++) {
    prefixSums[i] = prefixSums[i - 1] + nums[i];
    const diff = prefixSums[i - 1];
    rightDiffs.set(diff, (rightDiffs.get(diff) || 0) + 1);
  }

  const totalSum = prefixSums[n - 1];
  let result = totalSum % 2 === 0 ? (rightDiffs.get(totalSum / 2) || 0) : 0;

  for (let i = 0; i < n; i++) {
    const delta = k - nums[i];
    const newTotalSum = totalSum + delta;

    if (newTotalSum % 2 === 0) {
      const targetSum = newTotalSum / 2;
      const waysFromLeft = leftDiffs.get(targetSum) || 0;
      const waysFromRight = rightDiffs.get(targetSum - delta) || 0;
      result = Math.max(result, waysFromLeft + waysFromRight);
    }

    if (i < n - 1) {
      const currentDiff = prefixSums[i];
      leftDiffs.set(currentDiff, (leftDiffs.get(currentDiff) || 0) + 1);
      rightDiffs.set(currentDiff, rightDiffs.get(currentDiff) - 1);
    }
  }

  return result;
};
```

2027 Minimum Moves To Convert String

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2027. Minimum Moves to Convert String
 * https://leetcode.com/problems/minimum-moves-to-convert-string/
 * Difficulty: Easy
 *
 * You are given a string s consisting of n characters which are either 'X' or 'O'.
 *
 * A move is defined as selecting three consecutive characters of s and converting them
 * to 'O'. Note that if a move is applied to the character 'O', it will stay the same.
 *
 * Return the minimum number of moves required so that all the characters of s are
 * converted ]to 'O'.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumMoves = function(s) {
  return s.match(/X.{0,2}/g)?.length ?? 0;
};
```

2028 Find Missing Observations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2028. Find Missing Observations
 * https://leetcode.com/problems/find-missing-observations/
 * Difficulty: Medium
 *
 * You have observations of n + m 6-sided dice rolls with each face numbered from 1 to 6. n
 * of the observations went missing, and you only have the observations of m rolls.
 * Fortunately, you have also calculated the average value of the n + m rolls.
 *
 * You are given an integer array rolls of length m where rolls[i] is the value of the ith
 * observation. You are also given the two integers mean and n.
 *
 * Return an array of length n containing the missing observations such that the average
 * value of the n + m rolls is exactly mean. If there are multiple valid answers, return
 * any of them. If no such array exists, return an empty array.
 *
 * The average value of a set of k numbers is the sum of the numbers divided by k.
 *
 * Note that mean is an integer, so the sum of the n + m rolls should be divisible by n + m.
 */

/**
 * @param {number[]} rolls
 * @param {number} mean
 * @param {number} n
 * @return {number[]}
 */
var missingRolls = function(rolls, mean, n) {
  const totalRolls = rolls.length + n;
  const targetSum = mean * totalRolls;
  const currentSum = rolls.reduce((sum, roll) => sum + roll, 0);
  const missingSum = targetSum - currentSum;

  if (missingSum < n || missingSum > 6 * n) return [];
```

```
  const baseValue = Math.floor(missingSum / n);
  const remainder = missingSum % n;
  const result = [];

  for (let i = 0; i < n; i++) {
    if (i < remainder) {
      result.push(baseValue + 1);
    } else {
      result.push(baseValue);
    }
  }

  return result;
};
```

2029 Stone Game Ix

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2029. Stone Game IX
 * https://leetcode.com/problems/stone-game-ix/
 * Difficulty: Medium
 *
 * Alice and Bob continue their games with stones. There is a row of n stones, and each stone
has
 * an associated value. You are given an integer array stones, where stones[i] is the value of
 the
 * ith stone.
 *
 * Alice and Bob take turns, with Alice starting first. On each turn, the player may remove
any
 * stone from stones. The player who removes a stone loses if the sum of the values of all
 * removed stones is divisible by 3. Bob will win automatically if there are no remaining
stones
 * (even if it is Alice's turn).
 *
 * Assuming both players play optimally, return true if Alice wins and false if Bob wins.
 */

/**
 * @param {number[]} stones
 * @return {boolean}
 */
var stoneGameIX = function(stones) {
  const remainderCount = [0, 0, 0];
  for (const stone of stones) {
    remainderCount[stone % 3]++;
  }

  if (remainderCount[0] % 2 === 0) {
    return remainderCount[1] >= 1 && remainderCount[2] >= 1;
  }

  return Math.abs(remainderCount[1] - remainderCount[2]) > 2;
};
```

2030 Smallest K Length Subsequence With Occurrences Of A Letter

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2030. Smallest K-Length Subsequence With Occurrences of a Letter
 * https://leetcode.com/problems/smallest-k-length-subsequence-with-occurrences-of-a-letter/
 * Difficulty: Hard
```

```
 *
 * You are given a string s, an integer k, a letter letter, and an integer repetition.
 *
 * Return the lexicographically smallest subsequence of s of length k that has the letter
 * letter appear at least repetition times. The test cases are generated so that the letter
 * appears in s at least repetition times.
 *
 * A subsequence is a string that can be derived from another string by deleting some or no
 * characters without changing the order of the remaining characters.
 *
 * A string a is lexicographically smaller than a string b if in the first position where a
 * and b differ, string a has a letter that appears earlier in the alphabet than the
 * corresponding letter in b.
 */

/**
 * @param {string} s
 * @param {number} k
 * @param {character} letter
 * @param {number} repetition
 * @return {string}
 */
var smallestSubsequence = function(s, k, letter, repetition) {
  const n = s.length;
  let letterTotal = 0;
  for (let i = 0; i < n; i++) {
    if (s[i] === letter) letterTotal++;
  }

  const stack = [];
  let stackLetterCount = 0;
  let remainingCount = letterTotal;

  for (let i = 0; i < n; i++) {
    const char = s[i];

    if (char === letter) {
      remainingCount--;
    }

    while (
      stack.length > 0 && stack[stack.length - 1] > char && stack.length - 1 + (n - i) >= k
      && (stack[stack.length - 1] !== letter || stackLetterCount + remainingCount >
repetition)
    ) {
      if (stack[stack.length - 1] === letter) {
        stackLetterCount--;
      }
      stack.pop();
    }

    if (stack.length < k) {
      if (char === letter) {
        stackLetterCount++;
      }

      const neededLetters = repetition - stackLetterCount;
      const remainingPositions = k - stack.length - 1;
      if (char === letter || remainingPositions >= neededLetters) {
        stack.push(char);
      }
    }
  }

  if (stackLetterCount < repetition) {
    let missingLetters = repetition - stackLetterCount;
    const result = [...stack];

    for (let i = k - 1; i >= 0 && missingLetters > 0; i--) {
      if (result[i] !== letter) {
        result[i] = letter;
        missingLetters--;
      }
    }
```

```
    return result.join('');
  }

  return stack.join('');
};
```

2031 Count Subarrays With More Ones Than Zeros

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2031. Count Subarrays With More Ones Than Zeros
 * https://leetcode.com/problems/count-subarrays-with-more-ones-than-zeros/
 * Difficulty: Medium
 *
 * You are given a binary array nums containing only the integers 0 and 1. Return the number
 * of subarrays in nums that have more 1's than 0's. Since the answer may be very large,
 * return it modulo 109 + 7.
 *
 * A subarray is a contiguous sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var subarraysWithMoreZerosThanOnes = function(nums) {
  const MOD = 1e9 + 7;
  const dp = [0, 0];
  const mp = new Map();
  let sum = 0;
  let answer = 0;

  mp.set(0, 1);

  for (const num of nums) {
    const dpPrev = [...dp];

    if (num === 1) {
      sum++;
    } else {
      sum--;
    }

    dp[0] = mp.get(sum) || 0;

    if (num === 1) {
      dp[1] = (dpPrev[0] + dpPrev[1] + 1) % MOD;
    } else {
      dp[1] = (dpPrev[1] - dp[0] + MOD) % MOD;
    }

    mp.set(sum, (mp.get(sum) || 0) + 1);
    answer = (answer + dp[1]) % MOD;
  }

  return answer;
};
```

2032 Two Out Of Three

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2032. Two Out of Three
```

```
 * https://leetcode.com/problems/two-out-of-three/
 * Difficulty: Easy
 *
 * Given three integer arrays nums1, nums2, and nums3, return a distinct array containing all
 * the values that are present in at least two out of the three arrays. You may return the
 * values in any order.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @param {number[]} nums3
 * @return {number[]}
 */
var twoOutOfThree = function(nums1, nums2, nums3) {
  const set1 = new Set(nums1);
  const set2 = new Set(nums2);
  const set3 = new Set(nums3);
  const count = new Map();

  for (const num of set1) {
    count.set(num, (count.get(num) || 0) + 1);
  }

  for (const num of set2) {
    count.set(num, (count.get(num) || 0) + 1);
  }

  for (const num of set3) {
    count.set(num, (count.get(num) || 0) + 1);
  }

  const result = [];
  for (const [num, freq] of count) {
    if (freq >= 2) result.push(num);
  }

  return result;
};
```

2033 Minimum Operations To Make A Uni Value Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2033. Minimum Operations to Make a Uni-Value Grid
 * https://leetcode.com/problems/minimum-operations-to-make-a-uni-value-grid/
 * Difficulty: Medium
 *
 * You are given a 2D integer grid of size m x n and an integer x. In one operation, you can
add x
 * to or subtract x from any element in the grid.
 *
 * A uni-value grid is a grid where all the elements of it are equal.
 *
 * Return the minimum number of operations to make the grid uni-value. If it is not possible,
 * return -1.
 */

/**
 * @param {number[][]} grid
 * @param {number} x
 * @return {number}
 */
var minOperations = function(grid, x) {
  if (values.length === 1) return 0;
  const values = grid.flat();
  values.sort((a, b) => a - b);
  const median = values[Math.floor(values.length / 2)];
  let operations = 0;
```

```
  for (const value of values) {
    const diff = Math.abs(value - median);
    if (diff % x !== 0) return -1;
    operations += diff / x;
  }

  return operations;
};
```

2034 Stock Price Fluctuation

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2034. Stock Price Fluctuation
 * https://leetcode.com/problems/stock-price-fluctuation/
 * Difficulty: Medium
 *
 * You are given a stream of records about a particular stock. Each record contains a
timestamp
 * and the corresponding price of the stock at that timestamp.
 *
 * Unfortunately due to the volatile nature of the stock market, the records do not come in
order.
 * Even worse, some records may be incorrect. Another record with the same timestamp may
appear
 * later in the stream correcting the price of the previous wrong record.
 *
 * Design an algorithm that:
 * - Updates the price of the stock at a particular timestamp, correcting the price from any
 *   previous records at the timestamp.
 * - Finds the latest price of the stock based on the current records. The latest price is the
 *   price at the latest timestamp recorded.
 * - Finds the maximum price the stock has been based on the current records.
 * - Finds the minimum price the stock has been based on the current records.
 *
 * Implement the StockPrice class:
 * - StockPrice() Initializes the object with no price records.
 * - void update(int timestamp, int price) Updates the price of the stock at the given
timestamp.
 * - int current() Returns the latest price of the stock.
 * - int maximum() Returns the maximum price of the stock.
 * - int minimum() Returns the minimum price of the stock.
 */

var StockPrice = function() {
  this.timestamps = new Map();
  this.highestTimestamp = 0;
  this.minHeap = new PriorityQueue((a, b) => a[0] - b[0]);
  this.maxHeap = new PriorityQueue((a, b) => b[0] - a[0]);
};

/**
 * @param {number} timestamp
 * @param {number} price
 * @return {void}
 */
StockPrice.prototype.update = function(timestamp, price) {
  this.timestamps.set(timestamp, price);
  this.highestTimestamp = Math.max(this.highestTimestamp, timestamp);

  this.minHeap.enqueue([price, timestamp]);
  this.maxHeap.enqueue([price, timestamp]);
};

/**
 * @return {number}
 */
StockPrice.prototype.current = function() {
```

```
  return this.timestamps.get(this.highestTimestamp);
};

/**
 * @return {number}
 */
StockPrice.prototype.maximum = function() {
  let [currPrice, timestamp] = this.maxHeap.dequeue();

  while (currPrice !== this.timestamps.get(timestamp)) {
    [currPrice, timestamp] = this.maxHeap.dequeue();
  }

  this.maxHeap.enqueue([currPrice, timestamp]);
  return currPrice;
};

/**
 * @return {number}
 */
StockPrice.prototype.minimum = function() {
  let [currPrice, timestamp] = this.minHeap.dequeue();

  while (currPrice !== this.timestamps.get(timestamp)) {
    [currPrice, timestamp] = this.minHeap.dequeue();
  }

  this.minHeap.enqueue([currPrice, timestamp]);
  return currPrice;
};
```

2035 Partition Array Into Two Arrays To Minimize Sum Difference

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2035. Partition Array Into Two Arrays to Minimize Sum Difference
 * https://leetcode.com/problems/partition-array-into-two-arrays-to-minimize-sum-difference/
 * Difficulty: Hard
 *
 * You are given an integer array nums of 2 * n integers. You need to partition nums into two
arrays
 * of length n to minimize the absolute difference of the sums of the arrays. To partition
nums,
 * put each element of nums into one of the two arrays.
 *
 * Return the minimum possible absolute difference.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumDifference = function(nums) {
  const n = nums.length / 2;
  let result = Infinity;
  const totalSum = nums.reduce((sum, num) => sum + num, 0);
  const firstHalfSubsets = new Map();

  for (let mask = 0; mask < (1 << n); mask++) {
    let size = 0;
    let subsetSum = 0;

    for (let i = 0; i < n; i++) {
      if ((mask & (1 << i)) !== 0) {
        size++;
        subsetSum += nums[i];
      }
    }
```

```
    if (!firstHalfSubsets.has(size)) {
      firstHalfSubsets.set(size, []);
    }
    firstHalfSubsets.get(size).push(subsetSum);
  }

  for (const [size, sums] of firstHalfSubsets) {
    sums.sort((a, b) => a - b);
  }

  for (let mask = 0; mask < (1 << n); mask++) {
    let size = 0;
    let secondHalfSum = 0;

    for (let i = 0; i < n; i++) {
      if ((mask & (1 << i)) !== 0) {
        size++;
        secondHalfSum += nums[n + i];
      }
    }

    const complementSize = n - size;
    const firstHalfSums = firstHalfSubsets.get(complementSize);
    const target = (totalSum - 2 * secondHalfSum) / 2;
    const closestIndex = binarySearch(firstHalfSums, target);

    if (closestIndex < firstHalfSums.length) {
      result = Math.min(
        result, Math.abs(totalSum - 2 * (secondHalfSum + firstHalfSums[closestIndex]))
      );
    }

    if (closestIndex > 0) {
      result = Math.min(
        result, Math.abs(totalSum - 2 * (secondHalfSum + firstHalfSums[closestIndex - 1]))
      );
    }
  }

  return result;
};

function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length - 1;

  if (right < 0) return 0;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);

    if (arr[mid] < target) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
}
```

2036 Maximum Alternating Subarray Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2036. Maximum Alternating Subarray Sum
 * https://leetcode.com/problems/maximum-alternating-subarray-sum/
 * Difficulty: Medium
```

```
 *
 * A subarray of a 0-indexed integer array is a contiguous non-empty sequence of elements
 * within an array.
 *
 * The alternating subarray sum of a subarray that ranges from index i to j (inclusive,
 * 0 <= i <= j < nums.length) is nums[i] - nums[i+1] + nums[i+2] - ... +/- nums[j].
 *
 * Given a 0-indexed integer array nums, return the maximum alternating subarray sum of
 * any subarray of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumAlternatingSubarraySum = function(nums) {
  let result = -Infinity;
  let positive = -Infinity;
  let negative = -Infinity;

  for (const num of nums) {
    const newPositive = Math.max(negative + num, num);
    const newNegative = positive - num;

    positive = newPositive;
    negative = newNegative;
    result = Math.max(result, positive, negative);
  }

  return result;
};
```

2037 Minimum Number Of Moves To Seat Everyone

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2037. Minimum Number of Moves to Seat Everyone
 * https://leetcode.com/problems/minimum-number-of-moves-to-seat-everyone/
 * Difficulty: Easy
 *
 * There are n availabe seats and n students standing in a room. You are given an array
 * seats of length n, where seats[i] is the position of the ith seat. You are also given
 * the array students of length n, where students[j] is the position of the jth student.
 *
 * You may perform the following move any number of times:
 * - Increase or decrease the position of the ith student by 1 (i.e., moving the ith
 * student from position x to x + 1 or x - 1)
 *
 * Return the minimum number of moves required to move each student to a seat such that
 * no two students are in the same seat.
 *
 * Note that there may be multiple seats or students in the same position at the beginning.
 */

/**
 * @param {number[]} seats
 * @param {number[]} students
 * @return {number}
 */
var minMovesToSeat = function(seats, students) {
  seats.sort((a, b) => a - b);
  students.sort((a, b) => a - b);
  return seats.reduce((a, b, i) => a += Math.abs(seats[i] - students[i]), 0);
};
```

2038 Remove Colored Pieces If Both Neighbors Are The Same Color

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2038. Remove Colored Pieces if Both Neighbors are the Same Color
 * https://leetcode.com/problems/remove-colored-pieces-if-both-neighbors-are-the-same-color/
 * Difficulty: Medium
 *
 * There are n pieces arranged in a line, and each piece is colored either by 'A' or by 'B'.
 * You are given a string colors of length n where colors[i] is the color of the ith piece.
 *
 * Alice and Bob are playing a game where they take alternating turns removing pieces from the
 * line. In this game, Alice moves first.
 * - Alice is only allowed to remove a piece colored 'A' if both its neighbors are also
colored
 *   'A'. She is not allowed to remove pieces that are colored 'B'.
 * - Bob is only allowed to remove a piece colored 'B' if both its neighbors are also colored
 *   'B'. He is not allowed to remove pieces that are colored 'A'.
 * - Alice and Bob cannot remove pieces from the edge of the line.
 * - If a player cannot make a move on their turn, that player loses and the other player
wins.
 *
 * Assuming Alice and Bob play optimally, return true if Alice wins, or return false if Bob
wins.
 */

/**
 * @param {string} colors
 * @return {boolean}
 */
var winnerOfGame = function(colors) {
  let aliceMoves = 0;
  let bobMoves = 0;

  for (let i = 1; i < colors.length - 1; i++) {
    if (colors[i] === 'A' && colors[i - 1] === 'A' && colors[i + 1] === 'A') {
      aliceMoves++;
    } else if (colors[i] === 'B' && colors[i - 1] === 'B' && colors[i + 1] === 'B') {
      bobMoves++;
    }
  }

  return aliceMoves > bobMoves;
};
```

2039 The Time When The Network Becomes Idle

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2039. The Time When the Network Becomes Idle
 * https://leetcode.com/problems/the-time-when-the-network-becomes-idle/
 * Difficulty: Medium
 *
 * There is a network of n servers, labeled from 0 to n - 1. You are given a 2D integer array
 * edges, where edges[i] = [ui, vi] indicates there is a message channel between servers ui
 * and vi, and they can pass any number of messages to each other directly in one second.
 * You are also given a 0-indexed integer array patience of length n.
 *
 * All servers are connected, i.e., a message can be passed from one server to any other
 * server(s) directly or indirectly through the message channels.
 *
 * The server labeled 0 is the master server. The rest are data servers. Each data server
 * needs to send its message to the master server for processing and wait for a reply.
```

```
 * Messages move between servers optimally, so every message takes the least amount of
 * time to arrive at the master server. The master server will process all newly arrived
 * messages instantly and send a reply to the originating server via the reversed path
 * the message had gone through.
 *
 * At the beginning of second 0, each data server sends its message to be processed. Starting
 * from second 1, at the beginning of every second, each data server will check if it has
 * received a reply to the message it sent (including any newly arrived replies) from the
 * master server:
 * - If it has not, it will resend the message periodically. The data server i will resend
 *   the message every patience[i] second(s), i.e., the data server i will resend the message
 *   if patience[i] second(s) have elapsed since the last time the message was sent from this
 *   server.
 * - Otherwise, no more resending will occur from this server.
 *
 * The network becomes idle when there are no messages passing between servers or arriving
 * at servers.
 *
 * Return the earliest second starting from which the network becomes idle.
 */

/**
 * @param {number[][]} edges
 * @param {number[]} patience
 * @return {number}
 */
var networkBecomesIdle = function(edges, patience) {
  const n = patience.length;
  const adjList = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    adjList[u].push(v);
    adjList[v].push(u);
  }

  const distances = new Array(n).fill(Infinity);
  distances[0] = 0;
  const queue = [0];
  let maxTime = 0;

  while (queue.length) {
    const curr = queue.shift();
    for (const next of adjList[curr]) {
      if (distances[next] === Infinity) {
        distances[next] = distances[curr] + 1;
        queue.push(next);
        if (next !== 0) {
          const roundTrip = 2 * distances[next];
          const lastSent = Math.floor((roundTrip - 1) / patience[next]) * patience[next];
          maxTime = Math.max(maxTime, lastSent + roundTrip);
        }
      }
    }
  }

  return maxTime + 1;
};
```

2040 Kth Smallest Product Of Two Sorted Arrays

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2040. Kth Smallest Product of Two Sorted Arrays
 * https://leetcode.com/problems/kth-smallest-product-of-two-sorted-arrays/
 * Difficulty: Hard
 *
 * Given two sorted 0-indexed integer arrays nums1 and nums2 as well as an integer k, return
 * the kth (1-based) smallest product of nums1[i] * nums2[j] where 0 <= i < nums1.length
 * and 0 <= j < nums2.length.
 */
```

```
/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @param {number} k
 * @return {number}
 */
var kthSmallestProduct = function(nums1, nums2, k) {
  function countProducts(maxProduct) {
    let count = 0;
    for (const num1 of nums1) {
      let left = 0;
      let right = nums2.length;

      if (num1 >= 0) {
        while (left < right) {
          const mid = Math.floor((left + right) / 2);
          if (num1 * nums2[mid] <= maxProduct) left = mid + 1;
          else right = mid;
        }
        count += left;
      } else {
        while (left < right) {
          const mid = Math.floor((left + right) / 2);
          if (num1 * nums2[mid] <= maxProduct) right = mid;
          else left = mid + 1;
        }
        count += nums2.length - left;
      }
    }
    return count;
  }

  let low = -(10 ** 10);
  let high = 10 ** 10;

  while (low < high) {
    const mid = Math.floor((low + high) / 2);
    if (countProducts(mid) >= k) high = mid;
    else low = mid + 1;
  }

  return low;
};
```

2042 Check If Numbers Are Ascending In A Sentence

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2042. Check if Numbers Are Ascending in a Sentence
 * https://leetcode.com/problems/check-if-numbers-are-ascending-in-a-sentence/
 * Difficulty: Easy
 *
 * A sentence is a list of tokens separated by a single space with no leading or trailing
spaces.
 * Every token is either a positive number consisting of digits 0-9 with no leading zeros, or
a
 * word consisting of lowercase English letters.
 * - For example, "a puppy has 2 eyes 4 legs" is a sentence with seven tokens: "2" and "4" are
 *   numbers and the other tokens such as "puppy" are words.
 *
 * Given a string s representing a sentence, you need to check if all the numbers in s are
strictly
 * increasing from left to right (i.e., other than the last number, each number is strictly
smaller
 * than the number on its right in s).
 *
 * Return true if so, or false otherwise.
 */
```

```
/**
 * @param {string} s
 * @return {boolean}
 */
var areNumbersAscending = function(s) {
  const numbers = s.split(' ').filter(token => !isNaN(token)).map(Number);
  for (let i = 1; i < numbers.length; i++) {
    if (numbers[i] <= numbers[i - 1]) return false;
  }
  return true;
};
```

2043 Simple Bank System

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2043. Simple Bank System
 * https://leetcode.com/problems/simple-bank-system/
 * Difficulty: Medium
 *
 * You have been tasked with writing a program for a popular bank that will automate all its
 * incoming transactions (transfer, deposit, and withdraw). The bank has n accounts numbered
 * from 1 to n. The initial balance of each account is stored in a 0-indexed integer array
 * balance, with the (i + 1)th account having an initial balance of balance[i].
 *
 * Execute all the valid transactions. A transaction is valid if:
 * - The given account number(s) are between 1 and n, and
 * - The amount of money withdrawn or transferred from is less than or equal to the balance
 *   of the account.
 *
 * Implement the Bank class:
 * - Bank(long[] balance) Initializes the object with the 0-indexed integer array balance.
 * - boolean transfer(int account1, int account2, long money) Transfers money dollars from the
 *   account numbered account1 to the account numbered account2. Return true if the
transaction
 *   was successful, false otherwise.
 * - boolean deposit(int account, long money) Deposit money dollars into the account numbered
 *   account. Return true if the transaction was successful, false otherwise.
 * - boolean withdraw(int account, long money) Withdraw money dollars from the account
numbered
 *   account. Return true if the transaction was successful, false otherwise.
 */

/**
 * @param {number[]} balance
 */
var Bank = function(balance) {
  this.accounts = balance;
};

/**
 * @param {number} account1
 * @param {number} account2
 * @param {number} money
 * @return {boolean}
 */
Bank.prototype.transfer = function(account1, account2, money) {
  if (account1 > this.accounts.length || account2 > this.accounts.length) return false;
  if (this.accounts[account1 - 1] < money) return false;

  this.accounts[account1 - 1] -= money;
  this.accounts[account2 - 1] += money;
  return true;
};

/**
 * @param {number} account
 * @param {number} money
```

```
 * @return {boolean}
 */
Bank.prototype.deposit = function(account, money) {
  if (account > this.accounts.length) return false;

  this.accounts[account - 1] += money;
  return true;
};

/**
 * @param {number} account
 * @param {number} money
 * @return {boolean}
 */
Bank.prototype.withdraw = function(account, money) {
  if (account > this.accounts.length || this.accounts[account - 1] < money) return false;

  this.accounts[account - 1] -= money;
  return true;
};
```

2044 Count Number Of Maximum Bitwise Or Subsets

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2044. Count Number of Maximum Bitwise-OR Subsets
 * https://leetcode.com/problems/count-number-of-maximum-bitwise-or-subsets/
 * Difficulty: Medium
 *
 * Given an integer array nums, find the maximum possible bitwise OR of a subset of nums and
return
 * the number of different non-empty subsets with the maximum bitwise OR.
 *
 * An array a is a subset of an array b if a can be obtained from b by deleting some (possibly
 zero)
 * elements of b. Two subsets are considered different if the indices of the elements chosen
are
 * different.
 *
 * The bitwise OR of an array a is equal to a[0] OR a[1] OR ... OR a[a.length - 1]
(0-indexed).
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countMaxOrSubsets = function(nums) {
  const maxOr = nums.reduce((or, num) => or | num, 0);
  let count = 0;

  backtrack(0, 0);

  return count;

  function backtrack(index, currentOr) {
    if (currentOr === maxOr) count++;
    for (let i = index; i < nums.length; i++) {
      backtrack(i + 1, currentOr | nums[i]);
    }
  }
};
```

2045 Second Minimum Time To Reach Destination

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2045. Second Minimum Time to Reach Destination
 * https://leetcode.com/problems/second-minimum-time-to-reach-destination/
 * Difficulty: Hard
 *
 * A city is represented as a bi-directional connected graph with n vertices where each vertex
 * is labeled from 1 to n (inclusive). The edges in the graph are represented as a 2D integer
 * array edges, where each edges[i] = [ui, vi] denotes a bi-directional edge between vertex
 * ui and vertex vi. Every vertex pair is connected by at most one edge, and no vertex has
 * an edge to itself. The time taken to traverse any edge is time minutes.
 *
 * Each vertex has a traffic signal which changes its color from green to red and vice versa
 * every change minutes. All signals change at the same time. You can enter a vertex at any
 * time, but can leave a vertex only when the signal is green. You cannot wait at a vertex
 * if the signal is green.
 *
 * The second minimum value is defined as the smallest value strictly larger than the minimum
value.
 * - For example the second minimum value of [2, 3, 4] is 3, and the second minimum value of
 *   [2, 2, 4] is 4.
 *
 * Given n, edges, time, and change, return the second minimum time it will take to go from
 * vertex 1 to vertex n.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number} time
 * @param {number} change
 * @return {number}
 */
var secondMinimum = function(n, edges, time, change) {
  const adjList = Array.from({ length: n + 1 }, () => []);
  for (const [u, v] of edges) {
    adjList[u].push(v);
    adjList[v].push(u);
  }

  const distances = Array.from({ length: n + 1 }, () => [Infinity, Infinity]);
  const queue = [[1, 0]];
  distances[1][0] = 0;

  while (queue.length) {
    const [node, currTime] = queue.shift();

    for (const next of adjList[node]) {
      const signalCycle = Math.floor(currTime / change);
      const isGreen = signalCycle % 2 === 0;
      let nextTime = currTime + time;

      if (!isGreen) {
        nextTime = (signalCycle + 1) * change + time;
      }

      if (nextTime < distances[next][0]) {
        distances[next][1] = distances[next][0];
        distances[next][0] = nextTime;
        queue.push([next, nextTime]);
      } else if (nextTime > distances[next][0] && nextTime < distances[next][1]) {
        distances[next][1] = nextTime;
        queue.push([next, nextTime]);
      }
    }
  }
```

```
  return distances[n][1];
};
```

2046 Sort Linked List Already Sorted Using Absolute Values

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2046. Sort Linked List Already Sorted Using Absolute Values
 * https://leetcode.com/problems/sort-linked-list-already-sorted-using-absolute-values/
 * Difficulty: Medium
 *
 * Given the head of a singly linked list that is sorted in non-decreasing order using the
 * absolute values of its nodes, return the list sorted in non-decreasing order using the
 * actual values of its nodes.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var sortLinkedList = function(head) {
  if (!head || !head.next) return head;

  let current = head.next;
  let prev = head;

  while (current) {
    if (current.val < 0) {
      const next = current.next;
      prev.next = next;
      current.next = head;
      head = current;
      current = next;
    } else {
      prev = current;
      current = current.next;
    }
  }

  return head;
};
```

2047 Number Of Valid Words In A Sentence

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2047. Number of Valid Words in a Sentence
 * https://leetcode.com/problems/number-of-valid-words-in-a-sentence/
 * Difficulty: Easy
 *
 * A sentence consists of lowercase letters ('a' to 'z'), digits ('0' to '9'),
 * hyphens ('-'), punctuation marks ('!', '.', and ','), and spaces (' ') only.
 * Each sentence can be broken down into one or more tokens separated by one or
 * more spaces ' '.
 *
 * A token is a valid word if all three of the following are true:
```

```
 * - It only contains lowercase letters, hyphens, and/or punctuation (no digits).
 * - There is at most one hyphen '-'. If present, it must be surrounded by lowercase
 *   characters ("a-b" is valid, but "-ab" and "ab-" are not valid).
 * - There is at most one punctuation mark. If present, it must be at the end of
 *   the token ("ab,", "cd!", and "." are valid, but "a!b" and "c.," are not valid).
 *
 * Examples of valid words include "a-b.", "afad", "ba-c", "a!", and "!".
 *
 * Given a string sentence, return the number of valid words in sentence.
 */

/**
 * @param {string} sentence
 * @return {number}
 */
var countValidWords = function(sentence) {
  return sentence.trim().split(/\s+/).reduce((total, str) => {
    return total + (/^(?:([a-z]+\-?[a-z]+)|([a-z+]))?[!.,]?$/.test(str) ? 1 : 0);
  }, 0);
};
```

2048 Next Greater Numerically Balanced Number

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2048. Next Greater Numerically Balanced Number
 * https://leetcode.com/problems/next-greater-numerically-balanced-number/
 * Difficulty: Medium
 *
 * An integer x is numerically balanced if for every digit d in the number x, there are
exactly
 * d occurrences of that digit in x.
 *
 * Given an integer n, return the smallest numerically balanced number strictly greater than
n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var nextBeautifulNumber = function(n) {
  let candidate = n + 1;
  while (candidate <= 10000000) {
    if (isBalanced(candidate)) return candidate;
    candidate++;
  }

  return -1;

  function isBalanced(num) {
    const freq = new Array(10).fill(0);
    const str = num.toString();

    for (const digit of str) {
      freq[digit]++;
    }

    for (const digit of str) {
      if (freq[digit] !== parseInt(digit)) return false;
    }

    return true;
  }
};
```

2049 Count Nodes With The Highest Score

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2049. Count Nodes With the Highest Score
 * https://leetcode.com/problems/count-nodes-with-the-highest-score/
 * Difficulty: Medium
 *
 * There is a binary tree rooted at 0 consisting of n nodes. The nodes are labeled from 0 to n
 - 1.
 * You are given a 0-indexed integer array parents representing the tree, where parents[i] is
the
 * parent of node i. Since node 0 is the root, parents[0] == -1.
 *
 * Each node has a score. To find the score of a node, consider if the node and the edges
connected
 * to it were removed. The tree would become one or more non-empty subtrees. The size of a
subtree
 * is the number of the nodes in it. The score of the node is the product of the sizes of all
 * those subtrees.
 *
 * Return the number of nodes that have the highest score.
 */

/**
 * @param {number[]} parents
 * @return {number}
 */
var countHighestScoreNodes = function(parents) {
  const n = parents.length;
  const children = Array.from({ length: n }, () => []);
  for (let i = 1; i < n; i++) {
    children[parents[i]].push(i);
  }

  const sizes = new Array(n).fill(0);
  let maxScore = 0n;
  let maxScoreCount = 0;

  calculateSize(0);

  return maxScoreCount;

  function calculateSize(node) {
    let leftSize = 0;
    let rightSize = 0;

    if (children[node].length > 0) leftSize = calculateSize(children[node][0]);
    if (children[node].length > 1) rightSize = calculateSize(children[node][1]);

    sizes[node] = leftSize + rightSize + 1;

    let score = 1n;
    if (leftSize > 0) score *= BigInt(leftSize);
    if (rightSize > 0) score *= BigInt(rightSize);
    const parentSize = n - sizes[node];
    if (parentSize > 0) score *= BigInt(parentSize);

    if (score > maxScore) {
      maxScore = score;
      maxScoreCount = 1;
    } else if (score === maxScore) {
      maxScoreCount++;
    }

    return sizes[node];
  }
};
```

2050 Parallel Courses Iii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2050. Parallel Courses III
 * https://leetcode.com/problems/parallel-courses-iii/
 * Difficulty: Hard
 *
 * You are given an integer n, which indicates that there are n courses labeled from 1 to n.
 * You are also given a 2D integer array relations where relations[j] = [prevCoursej,
nextCoursej]
 * denotes that course prevCoursej has to be completed before course nextCoursej (prerequisite
 * relationship). Furthermore, you are given a 0-indexed integer array time where time[i]
denotes
 * how many months it takes to complete the (i+1)th course.
 *
 * You must find the minimum number of months needed to complete all the courses following
these
 * rules:
 * - You may start taking a course at any time if the prerequisites are met.
 * - Any number of courses can be taken at the same time.
 *
 * Return the minimum number of months needed to complete all the courses.
 *
 * Note: The test cases are generated such that it is possible to complete every course (i.e.,
 the
 * graph is a directed acyclic graph).
 */

/**
 * @param {number} n
 * @param {number[][]} relations
 * @param {number[]} time
 * @return {number}
 */
var minimumTime = function(n, relations, time) {
  const adjacencyList = Array.from({ length: n + 1 }, () => []);
  const inDegree = new Array(n + 1).fill(0);
  const completionTime = new Array(n + 1).fill(0);

  for (const [prev, next] of relations) {
    adjacencyList[prev].push(next);
    inDegree[next]++;
  }

  const queue = [];
  for (let i = 1; i <= n; i++) {
    if (inDegree[i] === 0) {
      queue.push(i);
      completionTime[i] = time[i - 1];
    }
  }

  while (queue.length) {
    const current = queue.shift();
    for (const next of adjacencyList[current]) {
      completionTime[next] = Math.max(
        completionTime[next],
        completionTime[current] + time[next - 1]
      );
      if (--inDegree[next] === 0) {
        queue.push(next);
      }
    }
  }

  return Math.max(...completionTime);
};
```

2052 Minimum Cost To Separate Sentence Into Rows

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2052. Minimum Cost to Separate Sentence Into Rows
 * https://leetcode.com/problems/minimum-cost-to-separate-sentence-into-rows/
 * Difficulty: Medium
 *
 * You are given a string sentence containing words separated by spaces, and an integer k.
 * Your task is to separate sentence into rows where the number of characters in each row
 * is at most k. You may assume that sentence does not begin or end with a space, and the
 * words in sentence are separated by a single space.
 *
 * You can split sentence into rows by inserting line breaks between words in sentence.
 * A word cannot be split between two rows. Each word must be used exactly once, and the
 * word order cannot be rearranged. Adjacent words in a row should be separated by a single
 * space, and rows should not begin or end with spaces.
 *
 * The cost of a row with length n is (k - n)2, and the total cost is the sum of the costs
 * for all rows except the last one.
 * - For example if sentence = "i love leetcode" and k = 12:
 *   - Separating sentence into "i", "love", and "leetcode" has a cost
 *     of (12 - 1)2 + (12 - 4)2 = 185.
 *   - Separating sentence into "i love", and "leetcode" has a cost of (12 - 6)2 = 36.
 *   - Separating sentence into "i", and "love leetcode" is not possible because the
 *     length of "love leetcode" is greater than k.
 *
 * Return the minimum possible total cost of separating sentence into rows.
 */

/**
 * @param {string} sentence
 * @param {number} k
 * @return {number}
 */
var minimumCost = function(sentence, k) {
  const words = sentence.split(' ');
  const n = words.length;
  const map = new Map();

  return dp(0);

  function dp(wordIndex) {
    if (wordIndex === n) return 0;
    if (map.has(wordIndex)) return map.get(wordIndex);

    let currentLength = 0;
    let minCost = Infinity;

    for (let i = wordIndex; i < n; i++) {
      if (i > wordIndex) currentLength += 1;
      currentLength += words[i].length;

      if (currentLength > k) break;

      const remainingCost = dp(i + 1);
      const currentCost = i === n - 1 ? 0 : (k - currentLength) ** 2;
      minCost = Math.min(minCost, currentCost + remainingCost);
    }

    map.set(wordIndex, minCost);
    return minCost;
  }
};
```

2053 Kth Distinct String In An Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2053. Kth Distinct String in an Array
 * https://leetcode.com/problems/kth-distinct-string-in-an-array/
 * Difficulty: Medium
 *
 * A distinct string is a string that is present only once in an array.
 *
 * Given an array of strings arr, and an integer k, return the kth distinct
 * string present in arr. If there are fewer than k distinct strings, return
 * an empty string "".
 *
 * Note that the strings are considered in the order in which they appear in
 * the array.
 */

/**
 * @param {string[]} arr
 * @param {number} k
 * @return {string}
 */
var kthDistinct = function(arr, k) {
  const map = new Map();
  arr.forEach(n => map.set(n, (map.get(n) || 0) + 1));

  const sorted = [...map].sort(([,a], [,b]) => a - b);
  const [kth, count] = sorted[k - 1] || [];

  return count === 1 ? kth : '';
};
```

2054 Two Best Non Overlapping Events

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2054. Two Best Non-Overlapping Events
 * https://leetcode.com/problems/two-best-non-overlapping-events/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D integer array of events where
 * events[i] = [startTimei, endTimei, valuei]. The ith event starts at
 * startTimei and ends at endTimei, and if you attend this event, you will
 * receive a value of valuei. You can choose at most two non-overlapping
 * events to attend such that the sum of their values is maximized.
 *
 * Return this maximum sum.
 *
 * Note that the start time and end time is inclusive: that is, you cannot
 * attend two events where one of them starts and the other ends at the
 * same time. More specifically, if you attend an event with end time t,
 * the next event must start at or after t + 1.
 */

/**
 * @param {number[][]} events
 * @return {number}
 */
var maxTwoEvents = function(events) {
  events.sort((a, b) => a[1] - b[1]);
```

```
  const maxValueUpTo = new Array(events.length);
  maxValueUpTo[0] = events[0][2];

  for (let i = 1; i < events.length; i++) {
    maxValueUpTo[i] = Math.max(maxValueUpTo[i - 1], events[i][2]);
  }

  let result = 0;
  for (let i = 0; i < events.length; i++) {
    const [start, end, value] = events[i];
    result = Math.max(result, value);

    let left = 0;
    let right = i - 1;
    let bestIndex = -1;
    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      if (events[mid][1] < start) {
        bestIndex = mid;
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }

    if (bestIndex !== -1) {
      result = Math.max(result, value + maxValueUpTo[bestIndex]);
    }
  }

  return result;
};
```

2055 Plates Between Candles

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2055. Plates Between Candles
 * https://leetcode.com/problems/plates-between-candles/
 * Difficulty: Medium
 *
 * There is a long table with a line of plates and candles arranged on top of it. You are
given
 * a 0-indexed string s consisting of characters '*' and '|' only, where a '*' represents a
plate
 * and a '|' represents a candle.
 *
 * You are also given a 0-indexed 2D integer array queries where queries[i] = [lefti, righti]
 * denotes the substring s[lefti...righti] (inclusive). For each query, you need to find the
 * number of plates between candles that are in the substring. A plate is considered between
 * candles if there is at least one candle to its left and at least one candle to its right
 * in the substring.
 * - For example, s = "||**||**|*", and a query [3, 8] denotes the substring "*||**|". The
number
 *   of plates between candles in this substring is 2, as each of the two plates has at least
one
 *   candle in the substring to its left and right.
 *
 * Return an integer array answer where answer[i] is the answer to the ith query.
 */

/**
 * @param {string} s
 * @param {number[][]} queries
 * @return {number[]}
 */
var platesBetweenCandles = function(s, queries) {
  const n = s.length;
```

```
  const prefixPlates = new Array(n + 1).fill(0);
  const leftCandle = new Array(n).fill(-1);
  const rightCandle = new Array(n).fill(-1);

  for (let i = 0, candle = -1; i < n; i++) {
    prefixPlates[i + 1] = prefixPlates[i] + (s[i] === '*' ? 1 : 0);
    if (s[i] === '|') candle = i;
    leftCandle[i] = candle;
  }

  for (let i = n - 1, candle = -1; i >= 0; i--) {
    if (s[i] === '|') candle = i;
    rightCandle[i] = candle;
  }

  const result = new Array(queries.length).fill(0);
  for (let i = 0; i < queries.length; i++) {
    const [start, end] = queries[i];
    const left = rightCandle[start];
    const right = leftCandle[end];
    if (left !== -1 && right !== -1 && left < right) {
      result[i] = prefixPlates[right] - prefixPlates[left];
    }
  }

  return result;
};
```

2056 Number Of Valid Move Combinations On Chessboard

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2056. Number of Valid Move Combinations On Chessboard
 * https://leetcode.com/problems/number-of-valid-move-combinations-on-chessboard/
 * Difficulty: Hard
 *
 * There is an 8 x 8 chessboard containing n pieces (rooks, queens, or bishops). You are given
 a
 * string array pieces of length n, where pieces[i] describes the type (rook, queen, or
bishop)
 * of the ith piece. In addition, you are given a 2D integer array positions also of length n,
 * where positions[i] = [ri, ci] indicates that the ith piece is currently at the 1-based
 * coordinate (ri, ci) on the chessboard.
 *
 * When making a move for a piece, you choose a destination square that the piece will travel
 * toward and stop on.
 * - A rook can only travel horizontally or vertically from (r, c) to the direction of (r+1,
c),
 *   (r-1, c), (r, c+1), or (r, c-1).
 * - A queen can only travel horizontally, vertically, or diagonally from (r, c) to the
direction
 *   of (r+1, c), (r-1, c), (r, c+1), (r, c-1), (r+1, c+1), (r+1, c-1), (r-1, c+1), (r-1,
c-1).
 * - A bishop can only travel diagonally from (r, c) to the direction of (r+1, c+1), (r+1,
c-1),
 *   (r-1, c+1), (r-1, c-1).
 *
 * You must make a move for every piece on the board simultaneously. A move combination
consists of
 * all the moves performed on all the given pieces. Every second, each piece will
instantaneously
 * travel one square towards their destination if they are not already at it. All pieces start
 * traveling at the 0th second. A move combination is invalid if, at a given time, two or more
 * pieces occupy the same square.
 *
 * Return the number of valid move combinations.
 *
 * Notes:
```

```
 * - No two pieces will start in the same square.
 * - You may choose the square a piece is already on as its destination.
 * - If two pieces are directly adjacent to each other, it is valid for them to move past each
 other
 *   and swap positions in one second.
 */

/**
 * @param {string[]} pieces
 * @param {number[][]} positions
 * @return {number}
 */
var countCombinations = function(pieces, positions) {
  const n = pieces.length;
  const directions = {
    rook: [[0, 1], [0, -1], [1, 0], [-1, 0]],
    bishop: [[1, 1], [1, -1], [-1, 1], [-1, -1]],
    queen: [[0, 1], [0, -1], [1, 0], [-1, 0], [1, 1], [1, -1], [-1, 1], [-1, -1]]
  };

  return generateCombinations(0, [], []);

  function isValidPosition(row, col) {
    return row >= 1 && row <= 8 && col >= 1 && col <= 8;
  }

  function simulateMoves(moves, steps) {
    const current = positions.map(([r, c]) => [r, c]);
    const maxSteps = Math.max(...steps);

    for (let t = 0; t <= maxSteps; t++) {
      const positionsAtTime = new Set();
      for (let i = 0; i < n; i++) {
        const [dr, dc] = moves[i];
        const step = Math.min(t, steps[i]);
        const newRow = current[i][0] + dr * step;
        const newCol = current[i][1] + dc * step;

        if (t > steps[i] && steps[i] > 0 && !isValidPosition(newRow, newCol)) continue;

        const posKey = `${newRow},${newCol}`;
        if (positionsAtTime.has(posKey)) return false;
        positionsAtTime.add(posKey);
      }
    }

    return true;
  }

  function generateCombinations(index, selectedMoves, selectedSteps) {
    if (index === n) {
      return simulateMoves(selectedMoves, selectedSteps) ? 1 : 0;
    }

    let total = 0;
    const [startRow, startCol] = positions[index];
    const pieceDirections = directions[pieces[index]];

    total += generateCombinations(index + 1, [...selectedMoves, [0, 0]], [...selectedSteps,
0]);

    for (const [dr, dc] of pieceDirections) {
      let row = startRow;
      let col = startCol;
      let step = 0;

      while (isValidPosition(row + dr, col + dc)) {
        row += dr;
        col += dc;
        step++;
        total += generateCombinations(
          index + 1,
          [...selectedMoves, [dr, dc]],
          [...selectedSteps, step]
        );
```

```
      }
    }

    return total;
  }
};
```

2057 Smallest Index With Equal Value

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2057. Smallest Index With Equal Value
 * https://leetcode.com/problems/smallest-index-with-equal-value/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums, return the smallest index i of nums such that i mod
 * 10 == nums[i], or -1 if such index does not exist.
 *
 * x mod y denotes the remainder when x is divided by y.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var smallestEqual = function(nums) {
  for (let i = 0; i < nums.length; i++) {
    if (i % 10 === nums[i]) {
      return i;
    }
  }
  return -1;
};
```

2058 Find The Minimum And Maximum Number Of Nodes Between Critical Poin

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2058. Find the Minimum and Maximum Number of Nodes Between Critical Points
 * https://leetcode.com/problems/find-the-minimum-and-maximum-number-of-nodes-between-critical
-points/
 * Difficulty: Medium
 *
 * A critical point in a linked list is defined as either a local maxima or a local minima.
 *
 * A node is a local maxima if the current node has a value strictly greater than the previous
 node
 * and the next node.
 *
 * A node is a local minima if the current node has a value strictly smaller than the previous
 node
 * and the next node.
 *
 * Note that a node can only be a local maxima/minima if there exists both a previous node and
 a
 * next node.
 *
 * Given a linked list head, return an array of length 2 containing [minDistance, maxDistance]
 * where minDistance is the minimum distance between any two distinct critical points and
 * maxDistance is the maximum distance between any two distinct critical points. If there are
 * fewer than two critical points, return [-1, -1].
 */
```

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {number[]}
 */
var nodesBetweenCriticalPoints = function(head) {
  const criticalPositions = [];
  let position = 1;
  let prev = head;
  let current = head.next;

  while (current.next) {
    const next = current.next;
    if ((current.val > prev.val && current.val > next.val) ||
        (current.val < prev.val && current.val < next.val)) {
      criticalPositions.push(position);
    }
    prev = current;
    current = next;
    position++;
  }

  if (criticalPositions.length < 2) {
    return [-1, -1];
  }

  let minDistance = Infinity;
  for (let i = 1; i < criticalPositions.length; i++) {
    minDistance = Math.min(minDistance, criticalPositions[i] - criticalPositions[i - 1]);
  }

  const maxDistance = criticalPositions[criticalPositions.length - 1] - criticalPositions[0];

  return [minDistance, maxDistance];
};
```

2059 Minimum Operations To Convert Number

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2059. Minimum Operations to Convert Number
 * https://leetcode.com/problems/minimum-operations-to-convert-number/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums containing distinct numbers, an integer start,
 * and an integer goal. There is an integer x that is initially set to start, and you want to
 * perform operations on x such that it is converted to goal. You can perform the following
 * operation repeatedly on the number x:
 *
 * If 0 <= x <= 1000, then for any index i in the array (0 <= i < nums.length), you can set x
 * to any of the following:
 * - x + nums[i]
 * - x - nums[i]
 * - x ^ nums[i] (bitwise-XOR)
 *
 * Note that you can use each nums[i] any number of times in any order. Operations that set x
 * to be out of the range 0 <= x <= 1000 are valid, but no more operations can be done
afterward.
 *
 * Return the minimum number of operations needed to convert x = start into goal, and -1 if it
 * is not possible.
 */
```

```
/**
 * @param {number[]} nums
 * @param {number} start
 * @param {number} goal
 * @return {number}
 */
var minimumOperations = function(nums, start, goal) {
  const queue = [[start, 0]];
  const visited = new Set([start]);

  while (queue.length) {
    const [current, steps] = queue.shift();

    for (const num of nums) {
      const candidates = [
        current + num,
        current - num,
        current ^ num
      ];

      for (const next of candidates) {
        if (next === goal) {
          return steps + 1;
        }
        if (next >= 0 && next <= 1000 && !visited.has(next)) {
          visited.add(next);
          queue.push([next, steps + 1]);
        }
      }
    }
  }

  return -1;
};
```

2060 Check If An Original String Exists Given Two Encoded Strings

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2060. Check if an Original String Exists Given Two Encoded Strings
 * https://leetcode.com/problems/check-if-an-original-string-exists-given-two-encoded-strings/
 * Difficulty: Hard
 *
 * An original string, consisting of lowercase English letters, can be encoded by the
following
 * steps:
 * - Arbitrarily split it into a sequence of some number of non-empty substrings.
 * - Arbitrarily choose some elements (possibly none) of the sequence, and replace each with
its
 *   length (as a numeric string).
 * - Concatenate the sequence as the encoded string.
 *
 * For example, one way to encode an original string "abcdefghijklmnop" might be:
 * - Split it as a sequence: ["ab", "cdefghijklmn", "o", "p"].
 * - Choose the second and third elements to be replaced by their lengths, respectively. The
 *   sequence becomes ["ab", "12", "1", "p"].
 * - Concatenate the elements of the sequence to get the encoded string: "ab121p".
 *
 * Given two encoded strings s1 and s2, consisting of lowercase English letters and digits 1-9
 * (inclusive), return true if there exists an original string that could be encoded as both
s1
 * and s2. Otherwise, return false.
 *
 * Note: The test cases are generated such that the number of consecutive digits in s1 and s2
 * does not exceed 3.
 */

/**
```

```
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var possiblyEquals = function(s1, s2) {
  const memo = new Array(s1.length + 1).fill().map(() => new Array(s2.length + 1).fill()
    .map(() => ({})));

  function match(pos1, pos2, diff) {
    if (pos1 === s1.length && pos2 === s2.length) return diff === 0;
    if (memo[pos1][pos2][diff] !== undefined) return memo[pos1][pos2][diff];

    const char1 = s1[pos1];
    const char2 = s2[pos2];

    if (pos1 < s1.length && pos2 < s2.length && char1 === char2 && diff === 0) {
      if (match(pos1 + 1, pos2 + 1, 0)) return true;
    }

    if (pos1 < s1.length && isNaN(char1) && diff < 0) {
      if (match(pos1 + 1, pos2, diff + 1)) return true;
    }

    if (pos2 < s2.length && isNaN(char2) && diff > 0) {
      if (match(pos1, pos2 + 1, diff - 1)) return true;
    }

    let num = 0;
    for (let i = 0; i < 3 && pos1 + i < s1.length; i++) {
      if (isNaN(s1[pos1 + i])) break;
      num = num * 10 + parseInt(s1[pos1 + i]);
      if (match(pos1 + i + 1, pos2, diff + num)) return true;
    }

    num = 0;
    for (let i = 0; i < 3 && pos2 + i < s2.length; i++) {
      if (isNaN(s2[pos2 + i])) break;
      num = num * 10 + parseInt(s2[pos2 + i]);
      if (match(pos1, pos2 + i + 1, diff - num)) return true;
    }

    return memo[pos1][pos2][diff] = false;
  }

  return match(0, 0, 0);
};
```

2061 Number Of Spaces Cleaning Robot Cleaned

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2061. Number of Spaces Cleaning Robot Cleaned
 * https://leetcode.com/problems/number-of-spaces-cleaning-robot-cleaned/
 * Difficulty: Medium
 *
 * A room is represented by a 0-indexed 2D binary matrix room where a 0 represents an empty
 * space and a 1 represents a space with an object. The top left corner of the room will be
 * empty in all test cases.
 *
 * A cleaning robot starts at the top left corner of the room and is facing right. The robot
 * will continue heading straight until it reaches the edge of the room or it hits an object,
 * after which it will turn 90 degrees clockwise and repeat this process. The starting space
 * and all spaces that the robot visits are cleaned by it.
 *
 * Return the number of clean spaces in the room if the robot runs indefinitely.
 */

/**
 * @param {number[][]} room
```

```
 * @return {number}
 */
var numberOfCleanRooms = function(room) {
  const rows = room.length;
  const cols = room[0].length;
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  const visited = new Set();
  const states = new Set();
  let row = 0;
  let col = 0;
  let direction = 0;

  while (true) {
    const state = `${row},${col},${direction}`;
    if (states.has(state)) break;

    states.add(state);
    visited.add(`${row},${col}`);

    const nextRow = row + directions[direction][0];
    const nextCol = col + directions[direction][1];

    if (nextRow < 0 || nextRow >= rows || nextCol < 0
        || nextCol >= cols || room[nextRow][nextCol] === 1) {
      direction = (direction + 1) % 4;
    } else {
      row = nextRow;
      col = nextCol;
    }
  }

  return visited.size;
};
```

2062 Count Vowel Substrings Of A String

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2062. Count Vowel Substrings of a String
 * https://leetcode.com/problems/count-vowel-substrings-of-a-string/
 * Difficulty: Easy
 *
 * A substring is a contiguous (non-empty) sequence of characters within a string.
 *
 * A vowel substring is a substring that only consists of vowels ('a', 'e', 'i', 'o', and 'u')
 * and has all five vowels present in it.
 *
 * Given a string word, return the number of vowel substrings in word.
 */

/**
 * @param {string} word
 * @return {number}
 */
var countVowelSubstrings = function(word) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u']);
  let total = 0;

  for (let start = 0; start < word.length; start++) {
    const vowelCount = new Map();
    for (let end = start; end < word.length && vowels.has(word[end]); end++) {
      vowelCount.set(word[end], (vowelCount.get(word[end]) || 0) + 1);
      if (vowelCount.size === 5) {
        total++;
      }
    }
  }

  return total;
```

```
};
```

2063 Vowels Of All Substrings

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2063. Vowels of All Substrings
 * https://leetcode.com/problems/vowels-of-all-substrings/
 * Difficulty: Medium
 *
 * Given a string word, return the sum of the number of vowels ('a', 'e', 'i', 'o', and 'u')
in
 * every substring of word.
 *
 * A substring is a contiguous (non-empty) sequence of characters within a string.
 *
 * Note: Due to the large constraints, the answer may not fit in a signed 32-bit integer.
Please
 * be careful during the calculations.
 */

/**
 * @param {string} word
 * @return {number}
 */
var countVowels = function(word) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u']);
  let total = 0;

  for (let i = 0; i < word.length; i++) {
    if (vowels.has(word[i])) {
      total += (i + 1) * (word.length - i);
    }
  }

  return total;
};
```

2064 Minimized Maximum Of Products Distributed To Any Store

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2064. Minimized Maximum of Products Distributed to Any Store
 * https://leetcode.com/problems/minimized-maximum-of-products-distributed-to-any-store/
 * Difficulty: Medium
 *
 * You are given an integer n indicating there are n specialty retail stores. There are m
product
 * types of varying amounts, which are given as a 0-indexed integer array quantities, where
 * quantities[i] represents the number of products of the ith product type.
 *
 * You need to distribute all products to the retail stores following these rules:
 * - A store can only be given at most one product type but can be given any amount of it.
 * - After distribution, each store will have been given some number of products (possibly 0).
 *   Let x represent the maximum number of products given to any store. You want x to be as
small
 *   as possible, i.e., you want to minimize the maximum number of products that are given to
any
 *   store.
 *
 * Return the minimum possible x.
 */
```

```
/**
 * @param {number} n
 * @param {number[]} quantities
 * @return {number}
 */
var minimizedMaximum = function(n, quantities) {
  let left = 1;
  let right = Math.max(...quantities);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    let storesNeeded = 0;

    for (const quantity of quantities) {
      storesNeeded += Math.ceil(quantity / mid);
    }

    if (storesNeeded <= n) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;
};
```

2065 Maximum Path Quality Of A Graph

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2065. Maximum Path Quality of a Graph
 * https://leetcode.com/problems/maximum-path-quality-of-a-graph/
 * Difficulty: Hard
 *
 * There is an undirected graph with n nodes numbered from 0 to n - 1 (inclusive). You are
given a
 * 0-indexed integer array values where values[i] is the value of the ith node. You are also
given
 * a 0-indexed 2D integer array edges, where each edges[j] = [uj, vj, timej] indicates that
there
 * is an undirected edge between the nodes uj and vj, and it takes timej seconds to travel
between
 * the two nodes. Finally, you are given an integer maxTime.
 *
 * A valid path in the graph is any path that starts at node 0, ends at node 0, and takes at
most
 * maxTime seconds to complete. You may visit the same node multiple times. The quality of a
valid
 * path is the sum of the values of the unique nodes visited in the path (each node's value is
 * added at most once to the sum).
 *
 * Return the maximum quality of a valid path.
 *
 * Note: There are at most four edges connected to each node.
 */

/**
 * @param {number[]} values
 * @param {number[][]} edges
 * @param {number} maxTime
 * @return {number}
 */
var maximalPathQuality = function(values, edges, maxTime) {
  const n = values.length;
  const adjList = Array.from({ length: n }, () => []);

  for (const [u, v, time] of edges) {
```

```
    adjList[u].push([v, time]);
    adjList[v].push([u, time]);
  }

  let result = 0;
  const initialVisited = new Set([0]);

  dfs(0, maxTime, values[0], initialVisited);

  return result;

  function dfs(node, timeLeft, quality, visited) {
    if (timeLeft < 0) return 0;
    if (node === 0) {
      result = Math.max(result, quality);
    }

    for (const [nextNode, travelTime] of adjList[node]) {
      const newVisited = new Set(visited);
      const newQuality = newVisited.has(nextNode) ? quality : quality + values[nextNode];
      newVisited.add(nextNode);
      dfs(nextNode, timeLeft - travelTime, newQuality, newVisited);
    }

    return result;
  }
};
```

2067 Number Of Equal Count Substrings

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2067. Number of Equal Count Substrings
 * https://leetcode.com/problems/number-of-equal-count-substrings/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s consisting of only lowercase English letters, and an
 * integer count. A substring of s is said to be an equal count substring if, for each
 * unique letter in the substring, it appears exactly count times in the substring.
 *
 * Return the number of equal count substrings in s.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {number} count
 * @return {number}
 */
var equalCountSubstrings = function(s, count) {
  const n = s.length;
  let result = 0;

  for (let uniqueChars = 1; uniqueChars <= 26; uniqueChars++) {
    const targetLength = uniqueChars * count;
    if (targetLength > n) break;

    const charCount = new Map();
    let validChars = 0;

    for (let right = 0; right < n; right++) {
      const rightChar = s[right];
      charCount.set(rightChar, (charCount.get(rightChar) || 0) + 1);

      if (charCount.get(rightChar) === count) {
        validChars++;
      } else if (charCount.get(rightChar) === count + 1) {
        validChars--;
```

```
      }

      if (right >= targetLength) {
        const leftChar = s[right - targetLength];
        if (charCount.get(leftChar) === count) {
          validChars--;
        } else if (charCount.get(leftChar) === count + 1) {
          validChars++;
        }

        charCount.set(leftChar, charCount.get(leftChar) - 1);
        if (charCount.get(leftChar) === 0) {
          charCount.delete(leftChar);
        }
      }

      if (right >= targetLength - 1 && validChars === uniqueChars
          && charCount.size === uniqueChars) {
        result++;
      }
    }
  }

  return result;
};
```

2068 Check Whether Two Strings Are Almost Equivalent

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2068. Check Whether Two Strings are Almost Equivalent
 * https://leetcode.com/problems/check-whether-two-strings-are-almost-equivalent/
 * Difficulty: Easy
 *
 * Two strings word1 and word2 are considered almost equivalent if the differences between the
 * frequencies of each letter from 'a' to 'z' between word1 and word2 is at most 3.
 *
 * Given two strings word1 and word2, each of length n, return true if word1 and word2 are
almost
 * equivalent, or false otherwise.
 *
 * The frequency of a letter x is the number of times it occurs in the string.
 */

/**
 * @param {string} word1
 * @param {string} word2
 * @return {boolean}
 */
var checkAlmostEquivalent = function(word1, word2) {
  const map = new Array(26).fill(0);

  for (let i = 0; i < word1.length; i++) {
    map[word1.charCodeAt(i) - 97]++;
    map[word2.charCodeAt(i) - 97]--;
  }

  return map.every(diff => Math.abs(diff) <= 3);
};
```

2069 Walking Robot Simulation Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```javascript
/**
 * 2069. Walking Robot Simulation II
 * https://leetcode.com/problems/walking-robot-simulation-ii/
 * Difficulty: Medium
 *
 * A width x height grid is on an XY-plane with the bottom-left cell at (0, 0) and the
 * top-right cell at (width - 1, height - 1). The grid is aligned with the four cardinal
 * directions ("North", "East", "South", and "West"). A robot is initially at cell (0, 0)
 * facing direction "East".
 *
 * The robot can be instructed to move for a specific number of steps. For each step, it does
 * the following.
 * 1. Attempts to move forward one cell in the direction it is facing.
 * 2. If the cell the robot is moving to is out of bounds, the robot instead turns 90 degrees
 *    counterclockwise and retries the step.
 *
 * After the robot finishes moving the number of steps required, it stops and awaits the next
 * instruction.
 *
 * Implement the Robot class:
 * - Robot(int width, int height) Initializes the width x height grid with the robot at (0, 0)
 *   facing "East".
 * - void step(int num) Instructs the robot to move forward num steps.
 * - int[] getPos() Returns the current cell the robot is at, as an array of length 2, [x, y].
 * - String getDir() Returns the current direction of the robot, "North", "East", "South", or
 *   "West".
 */

/**
 * @param {number} width
 * @param {number} height
 */
var Robot = function(width, height) {
  this.width = width;
  this.height = height;
  this.position = [0, 0];
  this.directionIndex = 0;
  this.directions = ['East', 'North', 'West', 'South'];
  this.moves = [[1, 0], [0, 1], [-1, 0], [0, -1]];
  this.perimeter = 2 * (width + height - 2);
};

/**
 * @param {number} num
 * @return {void}
 */
Robot.prototype.step = function(num) {
  num %= this.perimeter;
  if (num === 0) num = this.perimeter;

  while (num > 0) {
    const [dx, dy] = this.moves[this.directionIndex];
    const [x, y] = this.position;
    let steps;

    if (this.directionIndex === 0) steps = this.width - 1 - x;
    else if (this.directionIndex === 1) steps = this.height - 1 - y;
    else if (this.directionIndex === 2) steps = x;
    else steps = y;

    if (steps >= num) {
      this.position = [x + dx * num, y + dy * num];
      num = 0;
    } else {
      this.position = [x + dx * steps, y + dy * steps];
      this.directionIndex = (this.directionIndex + 1) % 4;
      num -= steps;
    }
  }
};

/**
 * @return {number[]}
 */
Robot.prototype.getPos = function() {
```

```
  return this.position;
};

/**
 * @return {string}
 */
Robot.prototype.getDir = function() {
  return this.directions[this.directionIndex];
};
```

2070 Most Beautiful Item For Each Query

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2070. Most Beautiful Item for Each Query
 * https://leetcode.com/problems/most-beautiful-item-for-each-query/
 * Difficulty: Medium
 *
 * You are given a 2D integer array items where items[i] = [pricei, beautyi] denotes the price
 and
 * beauty of an item respectively.
 *
 * You are also given a 0-indexed integer array queries. For each queries[j], you want to
determine
 * the maximum beauty of an item whose price is less than or equal to queries[j]. If no such
item
 * exists, then the answer to this query is 0.
 *
 * Return an array answer of the same length as queries where answer[j] is the answer to the
jth
 * query.
 */

/**
 * @param {number[][]} items
 * @param {number[]} queries
 * @return {number[]}
 */
var maximumBeauty = function(items, queries) {
  items.sort((a, b) => a[0] - b[0]);
  const maxBeauty = [];
  let currentMax = 0;

  for (const [, beauty] of items) {
    currentMax = Math.max(currentMax, beauty);
    maxBeauty.push(currentMax);
  }

  const result = new Array(queries.length).fill(0);

  for (let i = 0; i < queries.length; i++) {
    const price = queries[i];
    let left = 0;
    let right = items.length - 1;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      if (items[mid][0] <= price) {
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }

    if (right >= 0) {
      result[i] = maxBeauty[right];
    }
  }
```

```
  return result;
};
```

2071 Maximum Number Of Tasks You Can Assign

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2071. Maximum Number of Tasks You Can Assign
 * https://leetcode.com/problems/maximum-number-of-tasks-you-can-assign/
 * Difficulty: Hard
 *
 * You have n tasks and m workers. Each task has a strength requirement stored in a 0-indexed
 * integer array tasks, with the ith task requiring tasks[i] strength to complete. The
strength
 * of each worker is stored in a 0-indexed integer array workers, with the jth worker having
 * workers[j] strength. Each worker can only be assigned to a single task and must have a
 * strength greater than or equal to the task's strength requirement (i.e., workers[j] >=
tasks[i]).
 *
 * Additionally, you have pills magical pills that will increase a worker's strength by
strength.
 * You can decide which workers receive the magical pills, however, you may only give each
worker
 * at most one magical pill.
 *
 * Given the 0-indexed integer arrays tasks and workers and the integers pills and strength,
 * return the maximum number of tasks that can be completed.
 */

/**
 * @param {number[]} tasks
 * @param {number[]} workers
 * @param {number} pills
 * @param {number} strength
 * @return {number}
 */
var maxTaskAssign = function(tasks, workers, pills, strength) {
  tasks.sort((a, b) => a - b);
  workers.sort((a, b) => a - b);

  const n = tasks.length;
  const m = workers.length;

  let left = 0;
  let right = Math.min(n, m);

  while (left < right) {
    const mid = Math.floor((left + right + 1) / 2);

    if (canAssign(mid)) {
      left = mid;
    } else {
      right = mid - 1;
    }
  }

  return left;

  function canAssign(k) {
    const availableTasks = tasks.slice(0, k);
    const availableWorkers = workers.slice(m - k);
    let remainingPills = pills;

    for (let i = k - 1; i >= 0; i--) {
      const task = availableTasks[i];

      if (availableWorkers[availableWorkers.length - 1] >= task) {
```

```
        availableWorkers.pop();
        continue;
      }

      if (remainingPills === 0) return false;

      let found = false;
      for (let j = 0; j < availableWorkers.length; j++) {
        if (availableWorkers[j] + strength >= task) {
          availableWorkers.splice(j, 1);
          remainingPills--;
          found = true;
          break;
        }
      }

      if (!found) return false;
    }

    return true;
  }
};
```

2073 Time Needed To Buy Tickets

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2073. Time Needed to Buy Tickets
 * https://leetcode.com/problems/time-needed-to-buy-tickets/
 * Difficulty: Easy
 *
 * There are n people in a line queuing to buy tickets, where the 0th person is at the front
 * of the line and the (n - 1)th person is at the back of the line.
 *
 * You are given a 0-indexed integer array tickets of length n where the number of tickets
 * that the ith person would like to buy is tickets[i].
 *
 * Each person takes exactly 1 second to buy a ticket. A person can only buy 1 ticket at a
 * time and has to go back to the end of the line (which happens instantaneously) in order
 * to buy more tickets. If a person does not have any tickets left to buy, the person will
 * leave the line.
 *
 * Return the time taken for the person initially at position k (0-indexed) to finish buying
 * tickets.
 */

/**
 * @param {number[]} tickets
 * @param {number} k
 * @return {number}
 */
var timeRequiredToBuy = function(tickets, k) {
  const target = tickets[k];
  let result = 0;

  for (let i = 0; i < tickets.length; i++) {
    if (i <= k) {
      result += Math.min(tickets[i], target);
    } else {
      result += Math.min(tickets[i], target - 1);
    }
  }

  return result;
};
```

2074 Reverse Nodes In Even Length Groups

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2074. Reverse Nodes in Even Length Groups
 * https://leetcode.com/problems/reverse-nodes-in-even-length-groups/
 * Difficulty: Medium
 *
 * You are given the head of a linked list.
 *
 * The nodes in the linked list are sequentially assigned to non-empty groups whose lengths
form
 * the sequence of the natural numbers (1, 2, 3, 4, ...). The length of a group is the number
 * of nodes assigned to it. In other words:
 * - The 1st node is assigned to the first group.
 * - The 2nd and the 3rd nodes are assigned to the second group.
 * - The 4th, 5th, and 6th nodes are assigned to the third group, and so on.
 *
 * Note that the length of the last group may be less than or equal to 1 + the length of the
second
 * to last group.
 *
 * Reverse the nodes in each group with an even length, and return the head of the modified
linked
 * list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var reverseEvenLengthGroups = function(head) {
  let prevGroup = head;
  let groupLength = 2;

  while (prevGroup.next) {
    let count = 0;
    let current = prevGroup.next;
    const groupStart = current;

    while (current && count < groupLength) {
      current = current.next;
      count++;
    }

    if (count % 2 === 0) {
      prevGroup.next = reverseGroup(groupStart, count);
    }

    for (let i = 0; i < count; i++) {
      prevGroup = prevGroup.next;
    }

    groupLength++;
  }

  return head;

  function reverseGroup(start, length) {
    let prev = null;
    let current = start;
    for (let i = 0; i < length; i++) {
```

```
      const next = current.next;
      current.next = prev;
      prev = current;
      current = next;
    }
    start.next = current;
    return prev;
  }
};
```

2075 Decode The Slanted Ciphertext

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2075. Decode the Slanted Ciphertext
 * https://leetcode.com/problems/decode-the-slanted-ciphertext/
 * Difficulty: Medium
 *
 * A string originalText is encoded using a slanted transposition cipher to a string
encodedText
 * with the help of a matrix having a fixed number of rows rows.
 *
 * originalText is placed first in a top-left to bottom-right manner.
 *
 * The blue cells are filled first, followed by the red cells, then the yellow cells, and so
on,
 * until we reach the end of originalText. The arrow indicates the order in which the cells
are
 * filled. All empty cells are filled with ' '. The number of columns is chosen such that the
 * rightmost column will not be empty after filling in originalText.
 *
 * encodedText is then formed by appending all characters of the matrix in a row-wise fashion.
 *
 * The characters in the blue cells are appended first to encodedText, then the red cells, and
 so
 * on, and finally the yellow cells. The arrow indicates the order in which the cells are
accessed.
 *
 * For example, if originalText = "cipher" and rows = 3, then we encode it in the following
manner.
 *
 * The blue arrows depict how originalText is placed in the matrix, and the red arrows denote
the
 * order in which encodedText is formed. In the above example, encodedText = "ch ie pr".
 *
 * Given the encoded string encodedText and number of rows rows, return the original string
 * originalText.
 *
 * Note: originalText does not have any trailing spaces ' '. The test cases are generated such
 that
 * there is only one possible originalText.
 */

/**
 * @param {string} encodedText
 * @param {number} rows
 * @return {string}
 */
var decodeCiphertext = function(encodedText, rows) {
  if (rows === 1) return encodedText;

  const count = encodedText.length / rows;
  let result = '';

  for (let i = 0; i < count; i++) {
    for (let row = 0; row < rows && row + i < count; row++) {
      result += encodedText[row * count + i + row];
    }
  }
```

```
  return result.trimEnd();
};
```

2076 Process Restricted Friend Requests

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2076. Process Restricted Friend Requests
 * https://leetcode.com/problems/process-restricted-friend-requests/
 * Difficulty: Hard
 *
 * You are given an integer n indicating the number of people in a network. Each person is
 * labeled from 0 to n - 1.
 *
 * You are also given a 0-indexed 2D integer array restrictions, where restrictions[i] = [xi,
yi]
 * means that person xi and person yi cannot become friends, either directly or indirectly
through
 * other people.
 *
 * Initially, no one is friends with each other. You are given a list of friend requests as a
 * 0-indexed 2D integer array requests, where requests[j] = [uj, vj] is a friend request
between
 * person uj and person vj.
 *
 * A friend request is successful if uj and vj can be friends. Each friend request is
processed
 * in the given order (i.e., requests[j] occurs before requests[j + 1]), and upon a successful
 * request, uj and vj become direct friends for all future friend requests.
 *
 * Return a boolean array result, where each result[j] is true if the jth friend request is
 * successful or false if it is not.
 *
 * Note: If uj and vj are already direct friends, the request is still successful.
 */

/**
 * @param {number} n
 * @param {number[][]} restrictions
 * @param {number[][]} requests
 * @return {boolean[]}
 */
var friendRequests = function(n, restrictions, requests) {
  const parent = Array.from({ length: n }, (_, i) => i);
  const result = new Array(requests.length).fill(true);

  for (let j = 0; j < requests.length; j++) {
    const [u, v] = requests[j];
    const pu = find(u);
    const pv = find(v);

    let canBeFriends = true;
    for (const [x, y] of restrictions) {
      const px = find(x);
      const py = find(y);
      if ((pu === px && pv === py) || (pu === py && pv === px)) {
        canBeFriends = false;
        break;
      }
    }

    result[j] = canBeFriends;
    if (canBeFriends) {
      union(u, v);
    }
  }

  return result;
```

```
  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    parent[find(x)] = find(y);
  }
};
```

2077 Paths In Maze That Lead To Same Room

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2077. Paths in Maze That Lead to Same Room
 * https://leetcode.com/problems/paths-in-maze-that-lead-to-same-room/
 * Difficulty: Medium
 *
 * A maze consists of n rooms numbered from 1 to n, and some rooms are connected by corridors.
 * You are given a 2D integer array corridors where corridors[i] = [room1i, room2i] indicates
 * that there is a corridor connecting room1i and room2i, allowing a person in the maze to go
 * from room1i to room2i and vice versa.
 *
 * The designer of the maze wants to know how confusing the maze is. The confusion score of
 * the maze is the number of different cycles of length 3.
 * - For example, 1 → 2 → 3 → 1 is a cycle of length 3, but 1 → 2 → 3 → 4 and
 *   1 → 2 → 3 → 2 → 1 are not.
 *
 * Two cycles are considered to be different if one or more of the rooms visited in the first
 * cycle is not in the second cycle.
 *
 * Return the confusion score of the maze.
 */

/**
 * @param {number} n
 * @param {number[][]} corridors
 * @return {number}
 */
var numberOfPaths = function(n, corridors) {
  const graph = Array.from({ length: n + 1 }, () => new Set());

  for (const [room1, room2] of corridors) {
    graph[room1].add(room2);
    graph[room2].add(room1);
  }

  let cycles = 0;

  for (let i = 1; i <= n; i++) {
    const neighbors = Array.from(graph[i]);

    for (let j = 0; j < neighbors.length; j++) {
      for (let k = j + 1; k < neighbors.length; k++) {
        const neighbor1 = neighbors[j];
        const neighbor2 = neighbors[k];

        if (neighbor1 > i && neighbor2 > i && graph[neighbor1].has(neighbor2)) {
          cycles++;
        }
      }
    }
  }

  return cycles;
};
```

2078 Two Furthest Houses With Different Colors

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2078. Two Furthest Houses With Different Colors
 * https://leetcode.com/problems/two-furthest-houses-with-different-colors/
 * Difficulty: Easy
 *
 * There are n houses evenly lined up on the street, and each house is beautifully painted.
 * You are given a 0-indexed integer array colors of length n, where colors[i] represents
 * the color of the ith house.
 *
 * Return the maximum distance between two houses with different colors.
 *
 * The distance between the ith and jth houses is abs(i - j), where abs(x) is the absolute
 * value of x.
 */

/**
 * @param {number[]} colors
 * @return {number}
 */
var maxDistance = function(colors) {
  let result = 0;

  for (let i = 0; i < colors.length; i++) {
    for (let j = i + 1; j < colors.length; j++) {
      if (colors[i] !== colors[j]) {
        result = Math.max(result, j - i);
      }
    }
  }

  return result;
};
```

2079 Watering Plants

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2079. Watering Plants
 * https://leetcode.com/problems/watering-plants/
 * Difficulty: Medium
 *
 * You want to water n plants in your garden with a watering can. The plants are arranged in
 * a row and are labeled from 0 to n - 1 from left to right where the ith plant is located at
 * x = i. There is a river at x = -1 that you can refill your watering can at.
 *
 * Each plant needs a specific amount of water. You will water the plants in the following
way:
 * - Water the plants in order from left to right.
 * - After watering the current plant, if you do not have enough water to completely water the
 *   next plant, return to the river to fully refill the watering can.
 * - You cannot refill the watering can early.
 *
 * You are initially at the river (i.e., x = -1). It takes one step to move one unit on the
x-axis.
 *
 * Given a 0-indexed integer array plants of n integers, where plants[i] is the amount of
water
 * the ith plant needs, and an integer capacity representing the watering can capacity, return
 * the number of steps needed to water all the plants.
 */
```

```
/**
 * @param {number[]} plants
 * @param {number} capacity
 * @return {number}
 */
var wateringPlants = function(plants, capacity) {
  let water = capacity;
  let result = 0;

  for (let i = 0; i < plants.length; i++) {
    if (water < plants[i]) {
      result += 2 * i;
      water = capacity;
    }
    water -= plants[i];
    result++;
  }

  return result;
};
```

2080 Range Frequency Queries

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2080. Range Frequency Queries
 * https://leetcode.com/problems/range-frequency-queries/
 * Difficulty: Medium
 *
 * Design a data structure to find the frequency of a given value in a given subarray.
 *
 * The frequency of a value in a subarray is the number of occurrences of that value in
 * the subarray.
 *
 * Implement the RangeFreqQuery class:
 * - RangeFreqQuery(int[] arr) Constructs an instance of the class with the given 0-indexed
 *   integer array arr.
 * - int query(int left, int right, int value) Returns the frequency of value in the subarray
 *   arr[left...right].
 * - A subarray is a contiguous sequence of elements within an array. arr[left...right]
denotes
 *   the subarray that contains the elements of nums between indices left and right
(inclusive).
 */

/**
 * @param {number[]} arr
 */
var RangeFreqQuery = function(arr) {
  this.frequencyMap = new Map();

  for (let i = 0; i < arr.length; i++) {
    if (!this.frequencyMap.has(arr[i])) {
      this.frequencyMap.set(arr[i], []);
    }
    this.frequencyMap.get(arr[i]).push(i);
  }
};

/**
 * @param {number} left
 * @param {number} right
 * @param {number} value
 * @return {number}
 */
RangeFreqQuery.prototype.query = function(left, right, value) {
  if (!this.frequencyMap.has(value)) return 0;
```

```
  const indices = this.frequencyMap.get(value);
  let start = 0;
  let end = indices.length - 1;
  let leftBound = -1;
  let rightBound = -1;

  while (start <= end) {
    const mid = Math.floor((start + end) / 2);
    if (indices[mid] >= left) {
      leftBound = mid;
      end = mid - 1;
    } else {
      start = mid + 1;
    }
  }

  start = 0;
  end = indices.length - 1;
  while (start <= end) {
    const mid = Math.floor((start + end) / 2);
    if (indices[mid] <= right) {
      rightBound = mid;
      start = mid + 1;
    } else {
      end = mid - 1;
    }
  }

  return leftBound === -1 || rightBound === -1 ? 0 : rightBound - leftBound + 1;
};
```

2081 Sum Of K Mirror Numbers

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2081. Sum of k-Mirror Numbers
 * https://leetcode.com/problems/sum-of-k-mirror-numbers/
 * Difficulty: Hard
 *
 * A k-mirror number is a positive integer without leading zeros that reads the same both
forward
 * and backward in base-10 as well as in base-k.
 * - For example, 9 is a 2-mirror number. The representation of 9 in base-10 and base-2 are 9
and
 *   1001 respectively, which read the same both forward and backward.
 * - On the contrary, 4 is not a 2-mirror number. The representation of 4 in base-2 is 100,
which
 *   does not read the same both forward and backward.
 *
 * Given the base k and the number n, return the sum of the n smallest k-mirror numbers.
 */

/**
 * @param {number} k
 * @param {number} n
 * @return {number}
 */
var kMirror = function(k, n) {
  const kMirrorNumbers = [];
  let sum = 0;
  let length = 1;

  while (kMirrorNumbers.length < n) {
    generatePalindromes(length).forEach(num => {
      if (kMirrorNumbers.length < n && isKMirror(num, k)) {
        kMirrorNumbers.push(num);
        sum += num;
      }
```

```
    });
    length++;
  }

  return sum;
};

function toBaseK(num, k) {
  if (num === 0) return '0';

  let result = '';
  while (num > 0) {
    result = (num % k) + result;
    num = Math.floor(num / k);
  }

  return result;
}

function isKMirror(num, k) {
  const baseKRepresentation = toBaseK(num, k);
  return isPalindrome(baseKRepresentation);
}

function isPalindrome(str) {
  let left = 0;
  let right = str.length - 1;

  while (left < right) {
    if (str[left] !== str[right]) return false;
    left++;
    right--;
  }

  return true;
}

function generatePalindromes(length) {
  const palindromes = [];

  if (length === 1) {
    for (let i = 1; i <= 9; i++) {
      palindromes.push(i);
    }
    return palindromes;
  }

  if (length % 2 === 0) {
    const half = length / 2;
    const start = Math.pow(10, half - 1);
    const end = Math.pow(10, half) - 1;

    for (let i = start; i <= end; i++) {
      const firstHalf = i.toString();
      const secondHalf = firstHalf.split('').reverse().join('');
      palindromes.push(parseInt(firstHalf + secondHalf));
    }
  } else {
    const half = Math.floor(length / 2);
    const start = Math.pow(10, half);
    const end = Math.pow(10, half + 1) - 1;

    for (let i = start; i <= end; i++) {
      const withoutMiddle = Math.floor(i / 10);
      const middle = i % 10;
      const firstHalf = withoutMiddle.toString();
      const secondHalf = firstHalf.split('').reverse().join('');
      palindromes.push(parseInt(firstHalf + middle + secondHalf));
    }
  }

  return palindromes;
}
```

2083 Substrings That Begin And End With The Same Letter

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2083. Substrings That Begin and End With the Same Letter
 * https://leetcode.com/problems/substrings-that-begin-and-end-with-the-same-letter/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s consisting of only lowercase English letters.
 * Return the number of substrings in s that begin and end with the same character.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numberOfSubstrings = function(s) {
  const map = new Map();

  for (const char of s) {
    map.set(char, (map.get(char) || 0) + 1);
  }

  let result = 0;

  for (const count of map.values()) {
    result += (count * (count + 1)) / 2;
  }

  return result;
};
```

2085 Count Common Words With One Occurrence

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2085. Count Common Words With One Occurrence
 * https://leetcode.com/problems/count-common-words-with-one-occurrence/
 * Difficulty: Easy
 *
 * Given two string arrays words1 and words2, return the number of strings
 * that appear exactly once in each of the two arrays.
 */

/**
 * @param {string[]} words1
 * @param {string[]} words2
 * @return {number}
 */
var countWords = function(words1, words2) {
  const map = new Map();
  words1.forEach(n => map.set(n, (map.get(n) || 0) + 1));
  [...map].forEach(([key, count]) => count !== 1 ? map.delete(key) : null);
  words2.forEach(n => map.has(n) ? map.set(n, (map.get(n) || 0) + 1) : null);

  return [...map].filter(([, count]) => count === 2).length;
};
```

2086 Minimum Number Of Food Buckets To Feed The Hamsters

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2086. Minimum Number of Food Buckets to Feed the Hamsters
 * https://leetcode.com/problems/minimum-number-of-food-buckets-to-feed-the-hamsters/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string hamsters where hamsters[i] is either:
 * - 'H' indicating that there is a hamster at index i, or
 * - '.' indicating that index i is empty.
 *
 * You will add some number of food buckets at the empty indices in order to feed the
hamsters.
 * A hamster can be fed if there is at least one food bucket to its left or to its right. More
 * formally, a hamster at index i can be fed if you place a food bucket at index i - 1 and/or
 * at index i + 1.
 *
 * Return the minimum number of food buckets you should place at empty indices to feed all the
 * hamsters or -1 if it is impossible to feed all of them.
 */

/**
 * @param {string} hamsters
 * @return {number}
 */
var minimumBuckets = function(hamsters) {
  const n = hamsters.length;
  let result = 0;

  for (let i = 0; i < n; i++) {
    if (hamsters[i] === 'H') {
      if (i > 0 && hamsters[i - 1] === 'B') continue;
      if (i < n - 1 && hamsters[i + 1] === '.') {
        result++;
        hamsters = hamsters.slice(0, i + 1) + 'B' + hamsters.slice(i + 2);
      } else if (i > 0 && hamsters[i - 1] === '.') {
        result++;
        hamsters = hamsters.slice(0, i - 1) + 'B' + hamsters.slice(i);
      } else {
        return -1;
      }
    }
  }

  return result;
};
```

2087 Minimum Cost Homecoming Of A Robot In A Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2087. Minimum Cost Homecoming of a Robot in a Grid
 * https://leetcode.com/problems/minimum-cost-homecoming-of-a-robot-in-a-grid/
 * Difficulty: Medium
 *
 * There is an m x n grid, where (0, 0) is the top-left cell and (m - 1, n - 1) is the
bottom-right
 * cell. You are given an integer array startPos where startPos = [startrow, startcol]
indicates
 * that initially, a robot is at the cell (startrow, startcol). You are also given an integer
array
 * homePos where homePos = [homerow, homecol] indicates that its home is at the cell (homerow,
```

```
 * homecol).
 *
 * The robot needs to go to its home. It can move one cell in four directions: left, right,
up,
 * or down, and it can not move outside the boundary. Every move incurs some cost. You are
further
 * given two 0-indexed integer arrays: rowCosts of length m and colCosts of length n.
 * - If the robot moves up or down into a cell whose row is r, then this move costs
rowCosts[r].
 * - If the robot moves left or right into a cell whose column is c, then this move costs
 *   colCosts[c].
 *
 * Return the minimum total cost for this robot to return home.
 */

/**
 * @param {number[]} startPos
 * @param {number[]} homePos
 * @param {number[]} rowCosts
 * @param {number[]} colCosts
 * @return {number}
 */
var minCost = function(startPos, homePos, rowCosts, colCosts) {
  let result = 0;
  const [startRow, startCol] = startPos;
  const [homeRow, homeCol] = homePos;

  const rowStep = startRow < homeRow ? 1 : -1;
  for (let row = startRow + rowStep; row !== homeRow + rowStep; row += rowStep) {
    result += rowCosts[row];
  }

  const colStep = startCol < homeCol ? 1 : -1;
  for (let col = startCol + colStep; col !== homeCol + colStep; col += colStep) {
    result += colCosts[col];
  }

  return result;
};
```

2088 Count Fertile Pyramids In A Land

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2088. Count Fertile Pyramids in a Land
 * https://leetcode.com/problems/count-fertile-pyramids-in-a-land/
 * Difficulty: Hard
 *
 * A farmer has a rectangular grid of land with m rows and n columns that can be divided into
 * unit cells. Each cell is either fertile (represented by a 1) or barren (represented by a
0).
 * All cells outside the grid are considered barren.
 *
 * A pyramidal plot of land can be defined as a set of cells with the following criteria:
 * 1. The number of cells in the set has to be greater than 1 and all cells must be fertile.
 * 2. The apex of a pyramid is the topmost cell of the pyramid. The height of a pyramid is the
 *    number of rows it covers. Let (r, c) be the apex of the pyramid, and its height be h.
 *    Then, the plot comprises of cells (i, j) where r <= i <= r + h - 1 and
 *    c - (i - r) <= j <= c + (i - r).
 *
 * An inverse pyramidal plot of land can be defined as a set of cells with similar criteria:
 * 1. The number of cells in the set has to be greater than 1 and all cells must be fertile.
 * 2. The apex of an inverse pyramid is the bottommost cell of the inverse pyramid. The height
 *    of an inverse pyramid is the number of rows it covers. Let (r, c) be the apex of the
 *    pyramid, and its height be h. Then, the plot comprises of cells (i, j) where
 *    r - h + 1 <= i <= r and c - (r - i) <= j <= c + (r - i).
 *
 * Some examples of valid and invalid pyramidal (and inverse pyramidal) plots are shown below.
 * Black cells indicate fertile cells.
```

```
 *
 * Given a 0-indexed m x n binary matrix grid representing the farmland, return the total
number
 * of pyramidal and inverse pyramidal plots that can be found in grid.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var countPyramids = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  let result = 0;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 1) {
        result += countPyramidAt(i, j, false);
        result += countPyramidAt(i, j, true);
      }
    }
  }

  return result;

  function countPyramidAt(row, col, isInverse) {
    let height = 0;
    for (let h = 1; ; h++) {
      const r = isInverse ? row - h : row + h;
      if (r < 0 || r >= rows) break;
      const left = col - h;
      const right = col + h;
      if (left < 0 || right >= cols) break;
      let valid = true;
      for (let j = left; j <= right; j++) {
        if (grid[r][j] === 0) {
          valid = false;
          break;
        }
      }
      if (!valid) break;
      height++;
    }
    return height;
  }
};
```

2089 Find Target Indices After Sorting Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2089. Find Target Indices After Sorting Array
 * https://leetcode.com/problems/find-target-indices-after-sorting-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums and a target element target.
 *
 * A target index is an index i such that nums[i] == target.
 *
 * Return a list of the target indices of nums after sorting nums in non-decreasing order.
 * If there are no target indices, return an empty list. The returned list must be sorted
 * in increasing order.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
```

```
 */
var targetIndices = function(nums, target) {
  const result = [];

  nums.sort((a, b) => a - b);

  for (let i = 0; i < nums.length; i++) {
    if (nums[i] === target) {
      result.push(i);
    }
  }

  return result;
};
```

2090 K Radius Subarray Averages

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2090. K Radius Subarray Averages
 * https://leetcode.com/problems/k-radius-subarray-averages/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums of n integers, and an integer k.
 *
 * The k-radius average for a subarray of nums centered at some index i with the radius k is
 * the average of all elements in nums between the indices i - k and i + k (inclusive). If
 * there are less than k elements before or after the index i, then the k-radius average is
-1.
 *
 * Build and return an array avgs of length n where avgs[i] is the k-radius average for the
 * subarray centered at index i.
 *
 * The average of x elements is the sum of the x elements divided by x, using integer
division.
 * The integer division truncates toward zero, which means losing its fractional part.
 * - For example, the average of four elements 2, 3, 1, and
 *   5 is (2 + 3 + 1 + 5) / 4 = 11 / 4 = 2.75, which truncates to 2.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var getAverages = function(nums, k) {
  const n = nums.length;
  const windowSize = 2 * k + 1;
  const result = new Array(n).fill(-1);

  if (windowSize > n) return result;

  let sum = 0;
  for (let i = 0; i < windowSize; i++) {
    sum += nums[i];
  }

  result[k] = Math.floor(sum / windowSize);

  for (let i = k + 1; i < n - k; i++) {
    sum = sum - nums[i - k - 1] + nums[i + k];
    result[i] = Math.floor(sum / windowSize);
  }

  return result;
};
```

2091 Removing Minimum And Maximum From Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2091. Removing Minimum and Maximum From Array
 * https://leetcode.com/problems/removing-minimum-and-maximum-from-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of distinct integers nums.
 *
 * There is an element in nums that has the lowest value and an element that has the highest
 * value. We call them the minimum and maximum respectively. Your goal is to remove both these
 * elements from the array.
 *
 * A deletion is defined as either removing an element from the front of the array or removing
 * an element from the back of the array.
 *
 * Return the minimum number of deletions it would take to remove both the minimum and maximum
 * element from the array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumDeletions = function(nums) {
  const n = nums.length;
  let minIndex = 0;
  let maxIndex = 0;

  for (let i = 1; i < n; i++) {
    if (nums[i] < nums[minIndex]) minIndex = i;
    if (nums[i] > nums[maxIndex]) maxIndex = i;
  }

  const left = Math.min(minIndex, maxIndex);
  const right = Math.max(minIndex, maxIndex);

  return Math.min(
    right + 1,
    n - left,
    left + 1 + n - right
  );
};
```

2092 Find All People With Secret

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2092. Find All People With Secret
 * https://leetcode.com/problems/find-all-people-with-secret/
 * Difficulty: Hard
 *
 * You are given an integer n indicating there are n people numbered from 0 to n - 1. You are
also
 * given a 0-indexed 2D integer array meetings where meetings[i] = [xi, yi, timei] indicates
that
 * person xi and person yi have a meeting at timei. A person may attend multiple meetings at
the
 * same time. Finally, you are given an integer firstPerson.
 *
```

```
 * Person 0 has a secret and initially shares the secret with a person firstPerson at time 0.
This
 * secret is then shared every time a meeting takes place with a person that has the secret.
More
 * formally, for every meeting, if a person xi has the secret at timei, then they will share
the
 * secret with person yi, and vice versa.
 *
 * The secrets are shared instantaneously. That is, a person may receive the secret and share
it
 * with people in other meetings within the same time frame.
 *
 * Return a list of all the people that have the secret after all the meetings have taken
place.
 * You may return the answer in any order.
 */

/**
 * @param {number} n
 * @param {number[][]} meetings
 * @param {number} firstPerson
 * @return {number[]}
 */
var findAllPeople = function(n, meetings, firstPerson) {
  const parent = Array.from({ length: n }, (_, i) => i);

  union(0, firstPerson);
  meetings.sort((a, b) => a[2] - b[2]);

  let i = 0;
  while (i < meetings.length) {
    const currentTime = meetings[i][2];
    const group = [];

    while (i < meetings.length && meetings[i][2] === currentTime) {
      const [x, y] = meetings[i];
      group.push(x, y);
      union(x, y);
      i++;
    }

    for (const person of group) {
      if (find(person) !== find(0)) {
        parent[person] = person;
      }
    }
  }

  const result = [];
  for (let j = 0; j < n; j++) {
    if (find(j) === find(0)) {
      result.push(j);
    }
  }

  return result;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    parent[find(x)] = find(y);
  }
};
```

2093 Minimum Cost To Reach City With Discounts

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2093. Minimum Cost to Reach City With Discounts
 * https://leetcode.com/problems/minimum-cost-to-reach-city-with-discounts/
 * Difficulty: Medium
 *
 * A series of highways connect n cities numbered from 0 to n - 1. You are given a 2D
 * integer array highways where highways[i] = [city1i, city2i, tolli] indicates that
 * there is a highway that connects city1i and city2i, allowing a car to go from city1i
 * to city2i and vice versa for a cost of tolli.
 *
 * You are also given an integer discounts which represents the number of discounts you have.
 * You can use a discount to travel across the ith highway for a cost of tolli / 2 (integer
 * division). Each discount may only be used once, and you can only use at most one discount
 * per highway.
 *
 * Return the minimum total cost to go from city 0 to city n - 1, or -1 if it is not possible
 * to go from city 0 to city n - 1.
 */

/**
 * @param {number} n
 * @param {number[][]} highways
 * @param {number} discounts
 * @return {number}
 */
var minimumCost = function(n, highways, discounts) {
  const graph = Array.from({ length: n }, () => []);

  for (const [a, b, c] of highways) {
    graph[a].push([b, c]);
    graph[b].push([a, c]);
  }

  const pq = new PriorityQueue((a, b) => a[0] - b[0]);
  pq.enqueue([0, 0, 0]);
  const visited = Array.from({ length: n }, () => new Array(discounts + 1).fill(Infinity));
  visited[0][0] = 0;

  while (!pq.isEmpty()) {
    const [cost, city, discountUsed] = pq.dequeue();

    if (city === n - 1) return cost;

    for (const [nextCity, weight] of graph[city]) {
      if (cost + weight < visited[nextCity][discountUsed]) {
        pq.enqueue([cost + weight, nextCity, discountUsed]);
        visited[nextCity][discountUsed] = cost + weight;
      }

      if (discountUsed < discounts
          && cost + Math.floor(weight / 2) < visited[nextCity][discountUsed + 1]) {
        pq.enqueue([cost + Math.floor(weight / 2), nextCity, discountUsed + 1]);
        visited[nextCity][discountUsed + 1] = cost + Math.floor(weight / 2);
      }
    }
  }

  return -1;
};
```

2094 Finding 3 Digit Even Numbers

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2094. Finding 3-Digit Even Numbers
 * https://leetcode.com/problems/finding-3-digit-even-numbers/
 * Difficulty: Easy
 *
 * You are given an integer array digits, where each element is a digit. The array may contain
 * duplicates.
 *
 * You need to find all the unique integers that follow the given requirements:
 * - The integer consists of the concatenation of three elements from digits in any arbitrary
order.
 * - The integer does not have leading zeros.
 * - The integer is even.
 *
 * For example, if the given digits were [1, 2, 3], integers 132 and 312 follow the
requirements.
 *
 * Return a sorted array of the unique integers.
 */

/**
 * @param {number[]} digits
 * @return {number[]}
 */
var findEvenNumbers = function(digits) {
  const frequency = new Array(10).fill(0);
  const uniqueNumbers = new Set();

  for (const digit of digits) {
    frequency[digit]++;
  }

  for (let hundreds = 1; hundreds <= 9; hundreds++) {
    if (frequency[hundreds] === 0) continue;
    frequency[hundreds]--;

    for (let tens = 0; tens <= 9; tens++) {
      if (frequency[tens] === 0) continue;
      frequency[tens]--;

      for (let ones = 0; ones <= 8; ones += 2) {
        if (frequency[ones] === 0) continue;
        uniqueNumbers.add(hundreds * 100 + tens * 10 + ones);
      }

      frequency[tens]++;
    }

    frequency[hundreds]++;
  }

  return Array.from(uniqueNumbers).sort((a, b) => a - b);
};
```

2095 Delete The Middle Node Of A Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
```

```
 * 2095. Delete the Middle Node of a Linked List
 * https://leetcode.com/problems/delete-the-middle-node-of-a-linked-list/
 * Difficulty: Medium
 *
 * You are given the head of a linked list. Delete the middle node, and return the head
 * of the modified linked list.
 *
 * The middle node of a linked list of size n is the ■n / 2■th node from the start using
 * 0-based indexing, where ■x■ denotes the largest integer less than or equal to x.
 *
 * For n = 1, 2, 3, 4, and 5, the middle nodes are 0, 1, 1, 2, and 2, respectively.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var deleteMiddle = function(head) {
  let slow = new ListNode(0, head);
  let fast = head;

  if (!head.next) {
    return null;
  }

  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
  }

  slow.next = slow.next.next;

  return head;
};
```

2096 Step By Step Directions From A Binary Tree Node To Another

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2096. Step-By-Step Directions From a Binary Tree Node to Another
 * https://leetcode.com/problems/step-by-step-directions-from-a-binary-tree-node-to-another/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree with n nodes. Each node is uniquely assigned a
value
 * from 1 to n. You are also given an integer startValue representing the value of the start
node
 * s, and a different integer destValue representing the value of the destination node t.
 *
 * Find the shortest path starting from node s and ending at node t. Generate step-by-step
 * directions of such path as a string consisting of only the uppercase letters 'L', 'R', and
 * 'U'. Each letter indicates a specific direction:
 * - 'L' means to go from a node to its left child node.
 * - 'R' means to go from a node to its right child node.
 * - 'U' means to go from a node to its parent node.
 *
 * Return the step-by-step directions of the shortest path from node s to node t.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
```

```
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} startValue
 * @param {number} destValue
 * @return {string}
 */
var getDirections = function(root, startValue, destValue) {
  const startPath = [];
  const destPath = [];

  findPath(root, startValue, startPath);
  findPath(root, destValue, destPath);

  let i = 0;
  while (i < startPath.length && i < destPath.length && startPath[i] === destPath[i]) {
    i++;
  }

  const upMoves = 'U'.repeat(startPath.length - i);
  const downMoves = destPath.slice(i).join('');

  return upMoves + downMoves;

  function findPath(node, value, path) {
    if (!node) return false;
    if (node.val === value) return true;

    path.push('L');
    if (findPath(node.left, value, path)) return true;
    path.pop();

    path.push('R');
    if (findPath(node.right, value, path)) return true;
    path.pop();

    return false;
  }
};
```

2097 Valid Arrangement Of Pairs

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2097. Valid Arrangement of Pairs
 * https://leetcode.com/problems/valid-arrangement-of-pairs/
 * Difficulty: Hard
 *
 * You are given a 0-indexed 2D integer array pairs where pairs[i] = [starti, endi]. An
arrangement
 * of pairs is valid if for every index i where 1 <= i < pairs.length, we have endi-1 ==
starti.
 *
 * Return any valid arrangement of pairs.
 *
 * Note: The inputs will be generated such that there exists a valid arrangement of pairs.
 */

/**
 * @param {number[][]} pairs
 * @return {number[][]}
 */
var validArrangement = function(pairs) {
  const graph = new Map();
```

```
  const degree = new Map();

  for (const [start, end] of pairs) {
    if (!graph.has(start)) graph.set(start, []);
    graph.get(start).push(end);

    degree.set(start, (degree.get(start) || 0) + 1);
    degree.set(end, (degree.get(end) || 0) - 1);
  }

  let startNode = pairs[0][0];
  for (const [node, deg] of degree) {
    if (deg > 0) {
      startNode = node;
      break;
    }
  }

  const result = [];
  helper(startNode);

  return result.reverse();

  function helper(node) {
    while (graph.get(node)?.length) {
      const next = graph.get(node).pop();
      helper(next);
      result.push([node, next]);
    }
  }
};
```

2098 Subsequence Of Size K With The Largest Even Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2098. Subsequence of Size K With the Largest Even Sum
 * https://leetcode.com/problems/subsequence-of-size-k-with-the-largest-even-sum/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. Find the largest even sum of any
 * subsequence of nums that has a length of k.
 *
 * Return this sum, or -1 if such a sum does not exist.
 *
 * A subsequence is an array that can be derived from another array by deleting some or no
 * elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var largestEvenSum = function(nums, k) {
  const evenNums = nums.filter(num => num % 2 === 0).sort((a, b) => b - a);
  const oddNums = nums.filter(num => num % 2 === 1).sort((a, b) => b - a);
  let result = -1;

  for (let evenCount = 0; evenCount <= Math.min(k, evenNums.length); evenCount++) {
    const oddCount = k - evenCount;
    if (oddCount > oddNums.length || oddCount % 2 === 1) continue;
    let currentSum = 0;
    for (let i = 0; i < evenCount; i++) {
      currentSum += evenNums[i];
    }
    for (let i = 0; i < oddCount; i++) {
      currentSum += oddNums[i];
    }
```

```
    result = Math.max(result, currentSum);
  }

  return result;
};
```

2099 Find Subsequence Of Length K With The Largest Sum

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2099. Find Subsequence of Length K With the Largest Sum
 * https://leetcode.com/problems/find-subsequence-of-length-k-with-the-largest-sum/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. You want to find a subsequence
 * of nums of length k that has the largest sum.
 *
 * Return any such subsequence as an integer array of length k.
 *
 * A subsequence is an array that can be derived from another array by deleting some or
 * no elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var maxSubsequence = function(nums, k) {
  const map = new Map();
  nums.slice()
      .sort((a, b) => a - b)
      .slice(-k)
      .forEach(n => map.set(n, (map.get(n) || 0) + 1));

  return nums.filter(n => {
    const isInSubsequence = map.get(n);

    if (isInSubsequence) {
      map.set(n, map.get(n) - 1);
    }

    return isInSubsequence;
  });
};
```

2100 Find Good Days To Rob The Bank

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2100. Find Good Days to Rob the Bank
 * https://leetcode.com/problems/find-good-days-to-rob-the-bank/
 * Difficulty: Medium
 *
 * You and a gang of thieves are planning on robbing a bank. You are given a 0-indexed integer
 * array security, where security[i] is the number of guards on duty on the ith day. The days
 * are numbered starting from 0. You are also given an integer time.
 *
 * The ith day is a good day to rob the bank if:
 * - There are at least time days before and after the ith day,
 * - The number of guards at the bank for the time days before i are non-increasing, and
 * - The number of guards at the bank for the time days after i are non-decreasing.
 *
```

```
 * More formally, this means day i is a good day to rob the bank if and only if
 * security[i - time] >= security[i - time + 1] >= ... >= security[i]
 * <= ... <= security[i + time - 1] <= security[i + time].
 *
 * Return a list of all days (0-indexed) that are good days to rob the bank. The order that
the
 * days are returned in does not matter.
 */

/**
 * @param {number[]} security
 * @param {number} time
 * @return {number[]}
 */
var goodDaysToRobBank = function(security, time) {
  const n = security.length;
  const nonIncreasing = new Array(n).fill(0);
  const nonDecreasing = new Array(n).fill(0);
  const result = [];

  for (let i = 1; i < n; i++) {
    if (security[i] <= security[i - 1]) {
      nonIncreasing[i] = nonIncreasing[i - 1] + 1;
    }
    if (security[n - i - 1] <= security[n - i]) {
      nonDecreasing[n - i - 1] = nonDecreasing[n - i] + 1;
    }
  }

  for (let i = time; i < n - time; i++) {
    if (nonIncreasing[i] >= time && nonDecreasing[i] >= time) {
      result.push(i);
    }
  }

  return result;
};
```

2101 Detonate The Maximum Bombs

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2101. Detonate the Maximum Bombs
 * https://leetcode.com/problems/detonate-the-maximum-bombs/
 * Difficulty: Medium
 *
 * You are given a list of bombs. The range of a bomb is defined as the area where its effect
 * can be felt. This area is in the shape of a circle with the center as the location of the
bomb.
 *
 * The bombs are represented by a 0-indexed 2D integer array bombs where bombs[i] = [xi, yi,
ri].
 * xi and yi denote the X-coordinate and Y-coordinate of the location of the ith bomb, whereas
 ri
 * denotes the radius of its range.
 *
 * You may choose to detonate a single bomb. When a bomb is detonated, it will detonate all
bombs
 * that lie in its range. These bombs will further detonate the bombs that lie in their
ranges.
 *
 * Given the list of bombs, return the maximum number of bombs that can be detonated if you
are
 * allowed to detonate only one bomb.
 */

/**
 * @param {number[][]} bombs
```

```
 * @return {number}
 */
var maximumDetonation = function(bombs) {
  const n = bombs.length;
  const graph = Array.from({ length: n }, () => []);

  for (let i = 0; i < n; i++) {
    const [x1, y1, r1] = bombs[i];
    for (let j = 0; j < n; j++) {
      if (i === j) continue;
      const [x2, y2] = bombs[j];
      const distance = Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2);
      if (distance <= r1) {
        graph[i].push(j);
      }
    }
  }

  let result = 0;
  for (let i = 0; i < n; i++) {
    result = Math.max(result, dfs(i, new Set()));
  }

  return result;

  function dfs(node, visited) {
    visited.add(node);
    let count = 1;
    for (const neighbor of graph[node]) {
      if (!visited.has(neighbor)) {
        count += dfs(neighbor, visited);
      }
    }
    return count;
  }
};
```

2102 Sequentially Ordinal Rank Tracker

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2102. Sequentially Ordinal Rank Tracker
 * https://leetcode.com/problems/sequentially-ordinal-rank-tracker/
 * Difficulty: Hard
 *
 * A scenic location is represented by its name and attractiveness score, where name is a
unique
 * string among all locations and score is an integer. Locations can be ranked from the best
to
 * the worst. The higher the score, the better the location. If the scores of two locations
are
 * equal, then the location with the lexicographically smaller name is better.
 *
 * You are building a system that tracks the ranking of locations with the system initially
 * starting with no locations. It supports:
 * - Adding scenic locations, one at a time.
 * - Querying the ith best location of all locations already added, where i is the number of
 *   times the system has been queried (including the current query).
 *   - For example, when the system is queried for the 4th time, it returns the 4th best
location
 *     of all locations already added.
 *
 * Note that the test data are generated so that at any time, the number of queries does not
 * exceed the number of locations added to the system.
 *
 * Implement the SORTracker class:
 * - SORTracker() Initializes the tracker system.
 * - void add(string name, int score) Adds a scenic location with name and score to the
system.
```

```
 * - string get() Queries and returns the ith best location, where i is the number of times
this
 *   method has been invoked (including this invocation).
 */

var SORTracker = function() {
  this.count = 0;
  this.minHeap = new PriorityQueue((a, b) => a[0] - b[0] || b[1].localeCompare(a[1]));
  this.maxHeap = new PriorityQueue((a, b) => b[0] - a[0] || a[1].localeCompare(b[1]));
};

/**
 * @param {string} name
 * @param {number} score
 * @return {void}
 */
SORTracker.prototype.add = function(name, score) {
  this.minHeap.enqueue([score, name]);
  this.maxHeap.enqueue(this.minHeap.dequeue());
};

/**
 * @return {string}
 */
SORTracker.prototype.get = function() {
  const maxElement = this.maxHeap.dequeue();
  this.minHeap.enqueue(maxElement);

  return maxElement[1];
};
```

2103 Rings And Rods

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2103. Rings and Rods
 * https://leetcode.com/problems/rings-and-rods/
 * Difficulty: Easy
 *
 * There are n rings and each ring is either red, green, or blue. The rings are distributed
across
 * ten rods labeled from 0 to 9.
 *
 * You are given a string rings of length 2n that describes the n rings that are placed onto
the
 * rods. Every two characters in rings forms a color-position pair that is used to describe
each
 * ring where:
 * - The first character of the ith pair denotes the ith ring's color ('R', 'G', 'B').
 * - The second character of the ith pair denotes the rod that the ith ring is placed on
 *   ('0' to '9').
 *
 * For example, "R3G2B1" describes n == 3 rings: a red ring placed onto the rod labeled 3, a
green
 * ring placed onto the rod labeled 2, and a blue ring placed onto the rod labeled 1.
 *
 * Return the number of rods that have all three colors of rings on them.
 */

/**
 * @param {string} rings
 * @return {number}
 */
var countPoints = function(rings) {
  const map = new Map();

  for (let i = 0; i < rings.length; i += 2) {
    const color = rings[i];
    const rod = rings[i + 1];
```

```
    if (!map.has(rod)) map.set(rod, new Set());
    map.get(rod).add(color);
  }

  let result = 0;
  for (const colors of map.values()) {
    if (colors.size === 3) result++;
  }

  return result;
};
```

2104 Sum Of Subarray Ranges

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2104. Sum of Subarray Ranges
 * https://leetcode.com/problems/sum-of-subarray-ranges/
 * Difficulty: Medium
 *
 * You are given an integer array nums. The range of a subarray of nums is the difference
between
 * the largest and smallest element in the subarray.
 *
 * Return the sum of all subarray ranges of nums.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var subArrayRanges = function(nums) {
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    let min = nums[i];
    let max = nums[i];

    for (let j = i; j < nums.length; j++) {
      min = Math.min(min, nums[j]);
      max = Math.max(max, nums[j]);
      result += max - min;
    }
  }

  return result;
};
```

2105 Watering Plants Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2105. Watering Plants II
 * https://leetcode.com/problems/watering-plants-ii/
 * Difficulty: Medium
 *
 * Alice and Bob want to water n plants in their garden. The plants are arranged in a row and
are
 * labeled from 0 to n - 1 from left to right where the ith plant is located at x = i.
 *
 * Each plant needs a specific amount of water. Alice and Bob have a watering can each,
```

```
initially
 * full. They water the plants in the following way:
 * - Alice waters the plants in order from left to right, starting from the 0th plant. Bob
waters
 *   the plants in order from right to left, starting from the (n - 1)th plant. They begin
watering
 *   the plants simultaneously.
 * - It takes the same amount of time to water each plant regardless of how much water it
needs.
 * - Alice/Bob must water the plant if they have enough in their can to fully water it.
Otherwise,
 *   they first refill their can (instantaneously) then water the plant.
 * - In case both Alice and Bob reach the same plant, the one with more water currently in
his/her
 *   watering can should water this plant. If they have the same amount of water, then Alice
should
 *   water this plant.
 *
 * Given a 0-indexed integer array plants of n integers, where plants[i] is the amount of
water the
 * ith plant needs, and two integers capacityA and capacityB representing the capacities of
Alice's
 * and Bob's watering cans respectively, return the number of times they have to refill to
water all
 * the plants.
 */

/**
 * @param {number[]} plants
 * @param {number} capacityA
 * @param {number} capacityB
 * @return {number}
 */
var minimumRefill = function(plants, capacityA, capacityB) {
  let result = 0;
  let left = 0;
  let right = plants.length - 1;
  let waterA = capacityA;
  let waterB = capacityB;

  while (left <= right) {
    if (left === right) {
      if (waterA >= waterB && waterA < plants[left]) result++;
      if (waterB > waterA && waterB < plants[right]) result++;
      break;
    }

    if (waterA < plants[left]) {
      waterA = capacityA;
      result++;
    }
    waterA -= plants[left++];

    if (waterB < plants[right]) {
      waterB = capacityB;
      result++;
    }
    waterB -= plants[right--];
  }

  return result;
};
```

2106 Maximum Fruits Harvested After At Most K Steps

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2106. Maximum Fruits Harvested After at Most K Steps
 * https://leetcode.com/problems/maximum-fruits-harvested-after-at-most-k-steps/
```

```
 * Difficulty: Hard
 *
 * Fruits are available at some positions on an infinite x-axis. You are given a 2D integer
array
 * fruits where fruits[i] = [positioni, amounti] depicts amounti fruits at the position
positioni.
 * fruits is already sorted by positioni in ascending order, and each positioni is unique.
 *
 * You are also given an integer startPos and an integer k. Initially, you are at the position
 * startPos. From any position, you can either walk to the left or right. It takes one step to
 * move one unit on the x-axis, and you can walk at most k steps in total. For every position
 * you reach, you harvest all the fruits at that position, and the fruits will disappear from
 * that position.
 *
 * Return the maximum total number of fruits you can harvest.
 */

/**
 * @param {number[][]} fruits
 * @param {number} startPos
 * @param {number} k
 * @return {number}
 */
var maxTotalFruits = function(fruits, startPos, k) {
  let result = 0;
  let left = 0;
  let currentSum = 0;

  for (let right = 0; right < fruits.length; right++) {
    currentSum += fruits[right][1];

    while (left <= right) {
      const minPos = fruits[left][0];
      const maxPos = fruits[right][0];
      const steps = Math.min(
        Math.abs(startPos - minPos) + (maxPos - minPos),
        Math.abs(startPos - maxPos) + (maxPos - minPos)
      );

      if (steps <= k) break;
      currentSum -= fruits[left][1];
      left++;
    }

    if (left <= right) {
      result = Math.max(result, currentSum);
    }
  }

  return result;
};
```

2107 Number Of Unique Flavors After Sharing K Candies

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2107. Number of Unique Flavors After Sharing K Candies
 * https://leetcode.com/problems/number-of-unique-flavors-after-sharing-k-candies/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array candies, where candies[i] represents the flavor
 * of the ith candy. Your mom wants you to share these candies with your little sister by
 * giving her k consecutive candies, but you want to keep as many flavors of candies as
possible.
 *
 * Return the maximum number of unique flavors of candy you can keep after sharing with your
sister.
 */
```

```
/**
 * @param {number[]} candies
 * @param {number} k
 * @return {number}
 */
var shareCandies = function(candies, k) {
  if (k === 0) return new Set(candies).size;
  if (k === candies.length) return 0;

  const totalFlavorCount = new Map();
  for (const candy of candies) {
    totalFlavorCount.set(candy, (totalFlavorCount.get(candy) || 0) + 1);
  }

  const windowFlavorCount = new Map();
  let uniqueRemainingFlavors = totalFlavorCount.size;

  for (let i = 0; i < k; i++) {
    const candy = candies[i];
    const prevWindowCount = windowFlavorCount.get(candy) || 0;
    windowFlavorCount.set(candy, prevWindowCount + 1);

    if (prevWindowCount + 1 === totalFlavorCount.get(candy)) {
      uniqueRemainingFlavors--;
    }
  }

  let result = uniqueRemainingFlavors;

  for (let i = k; i < candies.length; i++) {
    const addCandy = candies[i];
    const removeCandy = candies[i - k];

    const prevAddCount = windowFlavorCount.get(addCandy) || 0;
    windowFlavorCount.set(addCandy, prevAddCount + 1);
    if (prevAddCount + 1 === totalFlavorCount.get(addCandy)) {
      uniqueRemainingFlavors--;
    }

    const prevRemoveCount = windowFlavorCount.get(removeCandy);
    windowFlavorCount.set(removeCandy, prevRemoveCount - 1);
    if (prevRemoveCount === totalFlavorCount.get(removeCandy)) {
      uniqueRemainingFlavors++;
    }

    result = Math.max(result, uniqueRemainingFlavors);
  }

  return result;
};
```

2108 Find First Palindromic String In The Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2108. Find First Palindromic String in the Array
 * https://leetcode.com/problems/find-first-palindromic-string-in-the-array/
 * Difficulty: Easy
 *
 * Given an array of strings words, return the first palindromic string in the array. If there
 is
 * no such string, return an empty string "".
 *
 * A string is palindromic if it reads the same forward and backward.
 */

/**
 * @param {string[]} words
 * @return {string}
```

```
 */
var firstPalindrome = function(words) {
  for (const word of words) {
    if (word === word.split('').reverse().join('')) {
      return word;
    }
  }
  return '';
};
```

2109 Adding Spaces To A String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2109. Adding Spaces to a String
 * https://leetcode.com/problems/adding-spaces-to-a-string/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s and a 0-indexed integer array spaces that describes the
 * indices in the original string where spaces will be added. Each space should be inserted
 * before the character at the given index.
 * - For example, given s = "EnjoyYourCoffee" and spaces = [5, 9], we place spaces before 'Y'
 *   and 'C', which are at indices 5 and 9 respectively. Thus, we obtain "Enjoy Your Coffee".
 *
 * Return the modified string after the spaces have been added.
 */

/**
 * @param {string} s
 * @param {number[]} spaces
 * @return {string}
 */
var addSpaces = function(s, spaces) {
  let result = '';
  let spaceIndex = 0;

  for (let i = 0; i < s.length; i++) {
    if (spaceIndex < spaces.length && i === spaces[spaceIndex]) {
      result += ' ';
      spaceIndex++;
    }
    result += s[i];
  }

  return result;
};
```

2110 Number Of Smooth Descent Periods Of A Stock

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2110. Number of Smooth Descent Periods of a Stock
 * https://leetcode.com/problems/number-of-smooth-descent-periods-of-a-stock/
 * Difficulty: Medium
 *
 * You are given an integer array prices representing the daily price history of a stock,
 * where prices[i] is the stock price on the ith day.
 *
 * A smooth descent period of a stock consists of one or more contiguous days such that the
 * price on each day is lower than the price on the preceding day by exactly 1. The first
 * day of the period is exempted from this rule.
 *
 * Return the number of smooth descent periods.
```

```
 */

/**
 * @param {number[]} prices
 * @return {number}
 */
var getDescentPeriods = function(prices) {
  let result = 1;
  let currentLength = 1;

  for (let i = 1; i < prices.length; i++) {
    if (prices[i] === prices[i - 1] - 1) {
      currentLength++;
    } else {
      currentLength = 1;
    }
    result += currentLength;
  }

  return result;
};
```

2111 Minimum Operations To Make The Array K Increasing

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2111. Minimum Operations to Make the Array K-Increasing
 * https://leetcode.com/problems/minimum-operations-to-make-the-array-k-increasing/
 * Difficulty: Hard
 *
 * You are given a 0-indexed array arr consisting of n positive integers, and a positive
integer k.
 *
 * The array arr is called K-increasing if arr[i-k] <= arr[i] holds for every index i, where
 * k <= i <= n-1.
 *
 * - For example, arr = [4, 1, 5, 2, 6, 2] is K-increasing for k = 2 because:
 *   - arr[0] <= arr[2] (4 <= 5)
 *   - arr[1] <= arr[3] (1 <= 2)
 *   - arr[2] <= arr[4] (5 <= 6)
 *   - arr[3] <= arr[5] (2 <= 2)
 * - However, the same arr is not K-increasing for k = 1 (because arr[0] > arr[1]) or k = 3
(because
 *   arr[0] > arr[3]).
 *
 * In one operation, you can choose an index i and change arr[i] into any positive integer.
 *
 * Return the minimum number of operations required to make the array K-increasing for the
given k.
 */

/**
 * @param {number[]} arr
 * @param {number} k
 * @return {number}
 */
var kIncreasing = function(arr, k) {
  let result = 0;
  for (let i = 0; i < k; i++) {
    const subsequence = [];
    for (let j = i; j < arr.length; j += k) {
      subsequence.push(arr[j]);
    }
    result += longestNonDecreasingSubsequence(subsequence);
  }

  return result;

  function longestNonDecreasingSubsequence(nums) {
```

```
    const tails = [];
    for (const num of nums) {
      let left = 0;
      let right = tails.length;
      while (left < right) {
        const mid = Math.floor((left + right) / 2);
        if (tails[mid] <= num) {
          left = mid + 1;
        } else {
          right = mid;
        }
      }
      tails[left] = num;
    }
    return nums.length - tails.length;
  }
};
```

2113 Elements In Array After Removing And Replacing Elements

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2113. Elements in Array After Removing and Replacing Elements
 * https://leetcode.com/problems/elements-in-array-after-removing-and-replacing-elements/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. Initially on minute 0, the array is
unchanged.
 * Every minute, the leftmost element in nums is removed until no elements remain. Then, every
 * minute, one element is appended to the end of nums, in the order they were removed in,
until
 * the original array is restored. This process repeats indefinitely.
 * - For example, the array [0,1,2] would change as follows: [0,1,2] → [1,2] → [2] → [] → [0]
→
 *   [0,1] → [0,1,2] → [1,2] → [2] → [] → [0] → [0,1] → [0,1,2] → ...
 *
 * You are also given a 2D integer array queries of size n where queries[j] = [timej, indexj].
 * The answer to the jth query is:
 * - nums[indexj] if indexj < nums.length at minute timej
 * - -1 if indexj >= nums.length at minute timej
 *
 * Return an integer array ans of size n where ans[j] is the answer to the jth query.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} queries
 * @return {number[]}
 */
var elementInNums = function(nums, queries) {
  const n = nums.length;
  const cycleLength = 2 * n;

  return queries.map(([time, index]) => {
    const position = time % cycleLength;

    if (position < n) {
      const currentLength = n - position;
      return index < currentLength ? nums[position + index] : -1;
    } else {
      const currentLength = position - n;
      return index < currentLength ? nums[index] : -1;
    }
  });
};
```

2114 Maximum Number Of Words Found In Sentences

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2114. Maximum Number of Words Found in Sentences
 * https://leetcode.com/problems/maximum-number-of-words-found-in-sentences/
 * Difficulty: Easy
 *
 * A sentence is a list of words that are separated by a single space with no leading or
trailing spaces.
 *
 * You are given an array of strings sentences, where each sentences[i] represents a single
sentence.
 *
 * Return the maximum number of words that appear in a single sentence.
 */

/**
 * @param {string[]} sentences
 * @return {number}
 */
var mostWordsFound = function(sentences) {
  return sentences.reduce((max, str) => Math.max(max, str.split(/\s+/).length), 0);
};
```

2115 Find All Possible Recipes From Given Supplies

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2115. Find All Possible Recipes from Given Supplies
 * https://leetcode.com/problems/find-all-possible-recipes-from-given-supplies/
 * Difficulty: Medium
 *
 * You have information about n different recipes. You are given a string array recipes and a
2D
 * string array ingredients. The ith recipe has the name recipes[i], and you can create it if
you
 * have all the needed ingredients from ingredients[i]. A recipe can also be an ingredient for
 * other recipes, i.e., ingredients[i] may contain a string that is in recipes.
 *
 * You are also given a string array supplies containing all the ingredients that you
initially
 * have, and you have an infinite supply of all of them.
 *
 * Return a list of all the recipes that you can create. You may return the answer in any
order.
 *
 * Note that two recipes may contain each other in their ingredients.
 */

/**
 * @param {string[]} recipes
 * @param {string[][]} ingredients
 * @param {string[]} supplies
 * @return {string[]}
 */
var findAllRecipes = function(recipes, ingredients, supplies) {
  const available = new Set(supplies);
  const recipeGraph = new Map();
  const inDegree = new Map();
  const queue = [];
  const result = [];
```

```
  recipes.forEach((recipe, index) => {
    inDegree.set(recipe, ingredients[index].length);
    ingredients[index].forEach(ing => {
      if (!recipeGraph.has(ing)) recipeGraph.set(ing, []);
      recipeGraph.get(ing).push(recipe);
    });
  });

  available.forEach(supply => {
    if (recipeGraph.has(supply)) {
      recipeGraph.get(supply).forEach(recipe => {
        const count = inDegree.get(recipe) - 1;
        inDegree.set(recipe, count);
        if (count === 0) queue.push(recipe);
      });
    }
  });

  while (queue.length) {
    const currentRecipe = queue.shift();
    result.push(currentRecipe);
    if (recipeGraph.has(currentRecipe)) {
      recipeGraph.get(currentRecipe).forEach(nextRecipe => {
        const count = inDegree.get(nextRecipe) - 1;
        inDegree.set(nextRecipe, count);
        if (count === 0) queue.push(nextRecipe);
      });
    }
  }

  return result;
};
```

2116 Check If A Parentheses String Can Be Valid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2116. Check if a Parentheses String Can Be Valid
 * https://leetcode.com/problems/check-if-a-parentheses-string-can-be-valid/
 * Difficulty: Medium
 *
 * A parentheses string is a non-empty string consisting only of '(' and ')'. It is valid
 * if any of the following conditions is true:
 * - It is ().
 * - It can be written as AB (A concatenated with B), where A and B are valid parentheses
 *   strings.
 * - It can be written as (A), where A is a valid parentheses string.
 *
 * You are given a parentheses string s and a string locked, both of length n. locked is a
 * binary string consisting only of '0's and '1's. For each index i of locked,
 * - If locked[i] is '1', you cannot change s[i].
 * - But if locked[i] is '0', you can change s[i] to either '(' or ')'.
 *
 * Return true if you can make s a valid parentheses string. Otherwise, return false.
 */

/**
 * @param {string} s
 * @param {string} locked
 * @return {boolean}
 */
var canBeValid = function(s, locked) {
  if (s.length % 2) return false;

  let symmetrical = 0;
  for (let i = 0; i < s.length; i++) {
    if (locked[i] === '0' || s[i] === '(') {
      symmetrical++;
    } else {
```

```
      symmetrical--;
    }
    if (symmetrical < 0) {
      return false;
    }
  }

  symmetrical = 0;
  for (let i = s.length - 1; i >= 0; i--) {
    if (locked[i] === '0' || s[i] === ')') {
      symmetrical++;
    } else {
      symmetrical--;
    }
    if (symmetrical < 0) {
      return false;
    }
  }

  return true;
};
```

2117 Abbreviating The Product Of A Range

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2117. Abbreviating the Product of a Range
 * https://leetcode.com/problems/abbreviating-the-product-of-a-range/
 * Difficulty: Hard
 *
 * You are given two positive integers left and right with left <= right. Calculate the
product of
 * all integers in the inclusive range [left, right].
 *
 * Since the product may be very large, you will abbreviate it following these steps:
 * 1. Count all trailing zeros in the product and remove them. Let us denote this count as C.
 *    - For example, there are 3 trailing zeros in 1000, and there are 0 trailing zeros in
546.
 * 2. Denote the remaining number of digits in the product as d. If d > 10, then express the
product
 *    as <pre>...<suf> where <pre> denotes the first 5 digits of the product, and <suf>
denotes the
 *    last 5 digits of the product after removing all trailing zeros. If d <= 10, we keep it
 *    unchanged.
 *    - For example, we express 1234567654321 as 12345...54321, but 1234567 is represented as
 *      1234567.
 * 3. Finally, represent the product as a string "<pre>...<suf>eC".
 *    - For example, 12345678987600000 will be represented as "12345...89876e5".
 *
 * Return a string denoting the abbreviated product of all integers in the inclusive range
[left,
 * right].
 */

/**
 * @param {number} left
 * @param {number} right
 * @return {string}
 */
var abbreviateProduct = function(left, right) {
  let zeros = 0;
  let count2 = 0;
  let count5 = 0;

  for (let i = left; i <= right; i++) {
    let n = i;
    while (n % 2 === 0) {
      count2++;
      n = Math.floor(n / 2);
```

```
    }
    n = i;
    while (n % 5 === 0) {
      count5++;
      n = Math.floor(n / 5);
    }
  }
  zeros = Math.min(count2, count5);

  let digits = 0;
  for (let i = left; i <= right; i++) {
    digits += Math.log10(i);
  }
  digits = Math.floor(digits) + 1;

  if (digits - zeros <= 10) {
    let product = 1n;
    for (let i = left; i <= right; i++) {
      product *= BigInt(i);
    }
    for (let i = 0; i < zeros; i++) {
      product /= 10n;
    }
    return product.toString() + 'e' + zeros;
  }

  let prefix = 1;
  for (let i = left; i <= right; i++) {
    prefix *= i;
    while (prefix >= 1e10) {
      prefix /= 10;
    }
  }
  prefix = prefix.toString().slice(0, 5);

  let suffix = 1n;
  for (let i = right; i >= left; i--) {
    suffix = (suffix * BigInt(i));
    while (suffix % 10n === 0n) {
      suffix /= 10n;
    }
    suffix = suffix % (10n ** 15n);
  }

  suffix = suffix.toString();
  while (suffix.length < 5) {
    suffix = '0' + suffix;
  }
  suffix = suffix.slice(-5);

  return prefix + '...' + suffix + 'e' + zeros;
};
```

2119 A Number After A Double Reversal

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2119. A Number After a Double Reversal
 * https://leetcode.com/problems/a-number-after-a-double-reversal/
 * Difficulty: Easy
 *
 * Reversing an integer means to reverse all its digits.
 * - For example, reversing 2021 gives 1202. Reversing 12300 gives 321 as the leading zeros
 *   are not retained.
 *
 * Given an integer num, reverse num to get reversed1, then reverse reversed1 to get
reversed2.
 * Return true if reversed2 equals num. Otherwise return false.
 */
```

```
/**
 * @param {number} num
 * @return {boolean}
 */
var isSameAfterReversals = function(num) {
  return num === 0 || num % 10 !== 0;
};
```

2120 Execution Of All Suffix Instructions Staying In A Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2120. Execution of All Suffix Instructions Staying in a Grid
 * https://leetcode.com/problems/execution-of-all-suffix-instructions-staying-in-a-grid/
 * Difficulty: Medium
 *
 * There is an n x n grid, with the top-left cell at (0, 0) and the bottom-right cell at
 * (n - 1, n - 1). You are given the integer n and an integer array startPos where
 * startPos = [startrow, startcol] indicates that a robot is initially at cell (startrow,
startcol).
 *
 * You are also given a 0-indexed string s of length m where s[i] is the ith instruction for
the
 * robot: 'L' (move left), 'R' (move right), 'U' (move up), and 'D' (move down).
 *
 * The robot can begin executing from any ith instruction in s. It executes the instructions
one
 * by one towards the end of s but it stops if either of these conditions is met:
 * - The next instruction will move the robot off the grid.
 * - There are no more instructions left to execute.
 *
 * Return an array answer of length m where answer[i] is the number of instructions the robot
can
 * execute if the robot begins executing from the ith instruction in s.
 */

/**
 * @param {number} n
 * @param {number[]} startPos
 * @param {string} s
 * @return {number[]}
 */
var executeInstructions = function(n, startPos, s) {
  const m = s.length;
  const result = new Array(m).fill(0);

  for (let i = 0; i < m; i++) {
    let row = startPos[0];
    let col = startPos[1];
    let steps = 0;

    for (let j = i; j < m; j++) {
      if (s[j] === 'L') col--;
      else if (s[j] === 'R') col++;
      else if (s[j] === 'U') row--;
      else row++;

      if (row < 0 || row >= n || col < 0 || col >= n) break;
      steps++;
    }

    result[i] = steps;
  }

  return result;
};
```

2121 Intervals Between Identical Elements

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2121. Intervals Between Identical Elements
 * https://leetcode.com/problems/intervals-between-identical-elements/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of n integers arr.
 *
 * The interval between two elements in arr is defined as the absolute difference between
their
 * indices. More formally, the interval between arr[i] and arr[j] is |i - j|.
 *
 * Return an array intervals of length n where intervals[i] is the sum of intervals between
arr[i]
 * and each element in arr with the same value as arr[i].
 *
 * Note: |x| is the absolute value of x.
 */

/**
 * @param {number[]} arr
 * @return {number[]}
 */
var getDistances = function(arr) {
  const valueIndices = new Map();
  const result = new Array(arr.length).fill(0);

  for (let i = 0; i < arr.length; i++) {
    if (!valueIndices.has(arr[i])) {
      valueIndices.set(arr[i], []);
    }
    valueIndices.get(arr[i]).push(i);
  }

  for (const indices of valueIndices.values()) {
    let prefixSum = 0;
    for (let i = 1; i < indices.length; i++) {
      prefixSum += indices[i] - indices[0];
    }

    result[indices[0]] = prefixSum;

    for (let i = 1; i < indices.length; i++) {
      const diff = indices[i] - indices[i - 1];
      prefixSum += diff * (i - (indices.length - i));
      result[indices[i]] = prefixSum;
    }
  }

  return result;
};
```

2122 Recover The Original Array

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2122. Recover the Original Array
 * https://leetcode.com/problems/recover-the-original-array/
 * Difficulty: Hard
```

```
 *
 * Alice had a 0-indexed array arr consisting of n positive integers. She chose an arbitrary
 * positive integer k and created two new 0-indexed integer arrays lower and higher in the
 * following manner:
 * 1. lower[i] = arr[i] - k, for every index i where 0 <= i < n
 * 2. higher[i] = arr[i] + k, for every index i where 0 <= i < n
 *
 * Unfortunately, Alice lost all three arrays. However, she remembers the integers that were
 * present in the arrays lower and higher, but not the array each integer belonged to. Help
 * Alice and recover the original array.
 *
 * Given an array nums consisting of 2n integers, where exactly n of the integers were present
 * in lower and the remaining in higher, return the original array arr. In case the answer is
 * not unique, return any valid array.
 *
 * Note: The test cases are generated such that there exists at least one valid array arr.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var recoverArray = function(nums) {
  const n = nums.length / 2;
  nums.sort((a, b) => a - b);

  for (let i = 1; i < 2 * n; i++) {
    const k = nums[i] - nums[0];
    if (k <= 0 || k % 2 !== 0) continue;

    const original = [];
    const used = new Array(2 * n).fill(false);
    let count = 0;

    for (let left = 0, right = i; right < 2 * n && count < n; right++) {
      while (left < right && used[left]) left++;
      if (left >= right) continue;

      if (nums[right] - nums[left] === k) {
        original.push(nums[left] + k / 2);
        used[left] = used[right] = true;
        count++;
        left++;
      }
    }

    if (count === n) return original;
  }

  return [];
};
```

2123 Minimum Operations To Remove Adjacent Ones In Matrix

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2123. Minimum Operations to Remove Adjacent Ones in Matrix
 * https://leetcode.com/problems/minimum-operations-to-remove-adjacent-ones-in-matrix/
 * Difficulty: Hard
 *
 * You are given a 0-indexed binary matrix grid. In one operation, you can flip any 1 in
 * grid to be 0.
 *
 * A binary matrix is well-isolated if there is no 1 in the matrix that is 4-directionally
 * connected (i.e., horizontal and vertical) to another 1.
 *
 * Return the minimum number of operations to make grid well-isolated.
 */
```

```
/**
 * @param {number[][]} grid
 * @return {number}
 */
var minimumOperations = function(grid) {
  const m = grid.length;
  const n = grid[0].length;
  let count = 0;
  const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];
  const match = Array.from({ length: m }, () => new Array(n).fill(-1));
  const visited = Array.from({ length: m }, () => new Array(n).fill(-1));

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      if (grid[i][j] && match[i][j] === -1) {
        visited[i][j] = i * n + j;
        count += dfs(i, j, visited[i][j]);
      }
    }
  }

  return count;

  function dfs(i, j, v) {
    for (const [di, dj] of directions) {
      const x = i + di;
      const y = j + dj;
      if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] && visited[x][y] !== v) {
        visited[x][y] = v;
        if (match[x][y] === -1 || dfs(Math.floor(match[x][y] / n), match[x][y] % n, v)) {
          match[x][y] = i * n + j;
          match[i][j] = x * n + y;
          return 1;
        }
      }
    }
    return 0;
  }
};
```

2124 Check If All As Appears Before All Bs

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2124. Check if All A's Appears Before All B's
 * https://leetcode.com/problems/check-if-all-as-appears-before-all-bs/
 * Difficulty: Easy
 *
 * Given a string s consisting of only the characters 'a' and 'b', return true if every 'a'
appears
 * before every 'b' in the string. Otherwise, return false.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var checkString = function(s) {
  let seen = false;

  for (const char of s) {
    if (char === 'b') seen = true;
    else if (seen) return false;
  }

  return true;
};
```

2125 Number Of Laser Beams In A Bank

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2125. Number of Laser Beams in a Bank
 * https://leetcode.com/problems/number-of-laser-beams-in-a-bank/
 * Difficulty: Medium
 *
 * Anti-theft security devices are activated inside a bank. You are given a 0-indexed binary
string
 * array bank representing the floor plan of the bank, which is an m x n 2D matrix. bank[i]
 * represents the ith row, consisting of '0's and '1's. '0' means the cell is empty, while'1'
 * means the cell has a security device.
 *
 * There is one laser beam between any two security devices if both conditions are met:
 * - The two devices are located on two different rows: r1 and r2, where r1 < r2.
 * - For each row i where r1 < i < r2, there are no security devices in the ith row.
 *
 * Laser beams are independent, i.e., one beam does not interfere nor join with another.
 *
 * Return the total number of laser beams in the bank.
 */

/**
 * @param {string[]} bank
 * @return {number}
 */
var numberOfBeams = function(bank) {
  let result = 0;
  let prevDevices = 0;

  for (const row of bank) {
    let currentDevices = 0;
    for (const cell of row) {
      if (cell === '1') currentDevices++;
    }
    if (currentDevices > 0) {
      result += prevDevices * currentDevices;
      prevDevices = currentDevices;
    }
  }

  return result;
};
```

2126 Destroying Asteroids

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2126. Destroying Asteroids
 * https://leetcode.com/problems/destroying-asteroids/
 * Difficulty: Medium
 *
 * You are given an integer mass, which represents the original mass of a planet. You are
further
 * given an integer array asteroids, where asteroids[i] is the mass of the ith asteroid.
 *
 * You can arrange for the planet to collide with the asteroids in any arbitrary order. If the
 mass
 * of the planet is greater than or equal to the mass of the asteroid, the asteroid is
destroyed
 * and the planet gains the mass of the asteroid. Otherwise, the planet is destroyed.
 *
```

```
 * Return true if all asteroids can be destroyed. Otherwise, return false.
 */

/**
 * @param {number} mass
 * @param {number[]} asteroids
 * @return {boolean}
 */
var asteroidsDestroyed = function(mass, asteroids) {
  asteroids.sort((a, b) => a - b);

  let planetMass = BigInt(mass);
  for (const asteroid of asteroids) {
    if (planetMass < BigInt(asteroid)) return false;
    planetMass += BigInt(asteroid);
  }

  return true;
};
```

2127 Maximum Employees To Be Invited To A Meeting

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2127. Maximum Employees to Be Invited to a Meeting
 * https://leetcode.com/problems/maximum-employees-to-be-invited-to-a-meeting/
 * Difficulty: Hard
 *
 * A company is organizing a meeting and has a list of n employees, waiting to be invited.
 * They have arranged for a large circular table, capable of seating any number of employees.
 *
 * The employees are numbered from 0 to n - 1. Each employee has a favorite person and they
 * will attend the meeting only if they can sit next to their favorite person at the table.
 * The favorite person of an employee is not themself.
 *
 * Given a 0-indexed integer array favorite, where favorite[i] denotes the favorite person of
 * the ith employee, return the maximum number of employees that can be invited to the
meeting.
 */

/**
 * @param {number[]} fav
 * @return {number}
 */
var maximumInvitations = function(fav) {
  const n = fav.length;
  const graph = new Array(n).fill().map(() => []);
  const seen = new Set();
  let result = 0;

  for (let i = 0; i < n; i++) {
    graph[fav[i]].push(i);
  }

  for (let i = 0; i < n; i++) {
    result += i < fav[i] || fav[fav[i]] != i ? 0 : dfs(i, fav[i]) + dfs(fav[i], i);
  }

  for (let i = 0; i < n; i++) {
    let j = i;

    if (!seen.has(i)) {
      const m = new Map();

      for (let k = 0; !seen.has(j); k++) {
        seen.add(j);
        m.set(j, k);
        j = fav[j];
      }
```

```
      result = Math.max(result, m.size - m.get(j) || 0);
    }
  }

  function dfs(i, j) {
    return graph[i].reduce((max, n) => Math.max(max, n == j ? 0 : dfs(n, j)), 0) + 1;
  };

  return result;
};
```

2128 Remove All Ones With Row And Column Flips

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2128. Remove All Ones With Row and Column Flips
 * https://leetcode.com/problems/remove-all-ones-with-row-and-column-flips/
 * Difficulty: Medium
 *
 * You are given an m x n binary matrix grid.
 *
 * In one operation, you can choose any row or column and flip each value in that row or
 * column (i.e., changing all 0's to 1's, and all 1's to 0's).
 *
 * Return true if it is possible to remove all 1's from grid using any number of operations
 * or false otherwise.
 */

/**
 * @param {number[][]} grid
 * @return {boolean}
 */
var removeOnes = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const firstRow = grid[0];

  for (let i = 1; i < rows; i++) {
    let sameAsFirst = true;
    let oppositeToFirst = true;

    for (let j = 0; j < cols; j++) {
      if (grid[i][j] !== firstRow[j]) {
        sameAsFirst = false;
      }
      if (grid[i][j] === firstRow[j]) {
        oppositeToFirst = false;
      }
    }

    if (!sameAsFirst && !oppositeToFirst) {
      return false;
    }
  }

  return true;
};
```

2129 Capitalize The Title

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
```

```
 * 2129. Capitalize the Title
 * https://leetcode.com/problems/capitalize-the-title/
 * Difficulty: Easy
 *
 * You are given a string title consisting of one or more words separated by a single space,
 * where each word consists of English letters. Capitalize the string by changing the
 * capitalization of each word such that:
 * - If the length of the word is 1 or 2 letters, change all letters to lowercase.
 * - Otherwise, change the first letter to uppercase and the remaining letters to lowercase.
 * Return the capitalized title.
 */

/**
 * @param {string} title
 * @return {string}
 */
var capitalizeTitle = function(title) {
  return title.split(/\s+/).map(word => {
    return word.toLowerCase().replace(/^\w/, l => word.length > 2 ? l.toUpperCase() : l);
  }).join(' ');
};
```

2130 Maximum Twin Sum Of A Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2130. Maximum Twin Sum of a Linked List
 * https://leetcode.com/problems/maximum-twin-sum-of-a-linked-list/
 * Difficulty: Medium
 *
 * In a linked list of size n, where n is even, the ith node (0-indexed) of the linked list is
 known
 * as the twin of the (n-1-i)th node, if 0 <= i <= (n / 2) - 1.
 *
 * For example, if n = 4, then node 0 is the twin of node 3, and node 1 is the twin of node 2.
 These
 * are the only nodes with twins for n = 4.
 *
 * The twin sum is defined as the sum of a node and its twin.
 *
 * Given the head of a linked list with even length, return the maximum twin sum of the linked
 list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {number}
 */
var pairSum = function(head) {
  const stack = [];
  while (head) {
    stack.push(head.val);
    head = head.next;
  }

  let max = 0;
  for (let i = 0; i < stack.length; i++) {
    max = Math.max(max, stack[i] + stack[stack.length - 1 - i]);
  }

  return max;
};
```

2131 Longest Palindrome By Concatenating Two Letter Words

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2131. Longest Palindrome by Concatenating Two Letter Words
 * https://leetcode.com/problems/longest-palindrome-by-concatenating-two-letter-words/
 * Difficulty: Medium
 *
 * You are given an array of strings words. Each element of words consists of two lowercase
 * English letters.
 *
 * Create the longest possible palindrome by selecting some elements from words and
concatenating
 * them in any order. Each element can be selected at most once.
 *
 * Return the length of the longest palindrome that you can create. If it is impossible to
create
 * any palindrome, return 0.
 *
 * A palindrome is a string that reads the same forward and backward.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var longestPalindrome = function(words) {
  const map = new Map();
  let length = 0;
  let hasCenter = false;

  for (const word of words) {
    map.set(word, (map.get(word) || 0) + 1);
  }

  for (const word of map.keys()) {
    const reverse = word[1] + word[0];

    if (word === reverse) {
      const count = map.get(word);
      length += Math.floor(count / 2) * 4;
      if (count % 2 === 1) hasCenter = true;
    } else if (map.has(reverse)) {
      const pairs = Math.min(map.get(word), map.get(reverse));
      length += pairs * 4;
      map.set(word, 0);
      map.set(reverse, 0);
    }
  }

  return hasCenter ? length + 2 : length;
};
```

2132 Stamping The Grid

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2132. Stamping the Grid
 * https://leetcode.com/problems/stamping-the-grid/
 * Difficulty: Hard
 *
 * You are given an m x n binary matrix grid where each cell is either 0 (empty) or 1
(occupied).
```

```
 *
 * You are then given stamps of size stampHeight x stampWidth. We want to fit the stamps such
that
 * they follow the given restrictions and requirements:
 * 1. Cover all the empty cells.
 * 2. Do not cover any of the occupied cells.
 * 3. We can put as many stamps as we want.
 * 4. Stamps can overlap with each other.
 * 5. Stamps are not allowed to be rotated.
 * 6. Stamps must stay completely inside the grid.
 *
 * Return true if it is possible to fit the stamps while following the given restrictions and
 * requirements. Otherwise, return false.
 */

/**
 * @param {number[][]} grid
 * @param {number} stampHeight
 * @param {number} stampWidth
 * @return {boolean}
 */
var possibleToStamp = function(grid, stampHeight, stampWidth) {
  const rows = grid.length;
  const cols = grid[0].length;
  const prefixSum = new Array(rows + 1).fill().map(() => new Array(cols + 1).fill(0));
  const diff = new Array(rows + 1).fill().map(() => new Array(cols + 1).fill(0));

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      prefixSum[i + 1][j + 1] = prefixSum[i + 1][j] + prefixSum[i][j + 1]
        - prefixSum[i][j] + grid[i][j];
    }
  }

  for (let i = 0; i <= rows - stampHeight; i++) {
    for (let j = 0; j <= cols - stampWidth; j++) {
      const x = i + stampHeight;
      const y = j + stampWidth;
      if (prefixSum[x][y] - prefixSum[x][j] - prefixSum[i][y] + prefixSum[i][j] === 0) {
        diff[i][j]++;
        diff[i][y]--;
        diff[x][j]--;
        diff[x][y]++;
      }
    }
  }

  const covered = new Array(rows).fill().map(() => new Array(cols).fill(0));
  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      covered[i][j] = (i > 0 ? covered[i - 1][j] : 0)
        + (j > 0 ? covered[i][j - 1] : 0)
        - (i > 0 && j > 0 ? covered[i - 1][j - 1] : 0) + diff[i][j];
      if (grid[i][j] === 0 && covered[i][j] === 0) return false;
    }
  }

  return true;
};
```

2133 Check If Every Row And Column Contains All Numbers

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2133. Check if Every Row and Column Contains All Numbers
 * https://leetcode.com/problems/check-if-every-row-and-column-contains-all-numbers/
 * Difficulty: Easy
 *
 * An n x n matrix is valid if every row and every column contains all the integers from 1
```

```
 * to n (inclusive).
 *
 * Given an n x n integer matrix matrix, return true if the matrix is valid. Otherwise,
 * return false.
 */

/**
 * @param {number[][]} matrix
 * @return {boolean}
 */
var checkValid = function(matrix) {
  const n = matrix.length;

  for (let i = 0; i < n; i++) {
    const rowSet = new Set();
    const colSet = new Set();

    for (let j = 0; j < n; j++) {
      rowSet.add(matrix[i][j]);
      colSet.add(matrix[j][i]);
    }

    if (rowSet.size !== n || colSet.size !== n) return false;
  }

  return true;
};
```

2134 Minimum Swaps To Group All 1S Together Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2134. Minimum Swaps to Group All 1's Together II
 * https://leetcode.com/problems/minimum-swaps-to-group-all-1s-together-ii/
 * Difficulty: Medium
 *
 * A swap is defined as taking two distinct positions in an array and swapping the values in
them.
 *
 * A circular array is defined as an array where we consider the first element and the last
element
 * to be adjacent.
 *
 * Given a binary circular array nums, return the minimum number of swaps required to group
all
 * 1's present in the array together at any location.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minSwaps = function(nums) {
  const onesCount = nums.reduce((sum, num) => sum + num, 0);
  const n = nums.length;
  let result = Infinity;
  let currentZeros = 0;

  for (let i = 0; i < onesCount; i++) {
    if (nums[i] === 0) currentZeros++;
  }

  result = currentZeros;

  for (let i = 1; i < n; i++) {
    if (nums[(i - 1) % n] === 0) currentZeros--;
    if (nums[(i + onesCount - 1) % n] === 0) currentZeros++;
    result = Math.min(result, currentZeros);
  }
```

```
  return result;
};
```

2135 Count Words Obtained After Adding A Letter

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2135. Count Words Obtained After Adding a Letter
 * https://leetcode.com/problems/count-words-obtained-after-adding-a-letter/
 * Difficulty: Medium
 *
 * You are given two 0-indexed arrays of strings startWords and targetWords. Each string
consists
 * of lowercase English letters only.
 *
 * For each string in targetWords, check if it is possible to choose a string from startWords
and
 * perform a conversion operation on it to be equal to that from targetWords.
 *
 * The conversion operation is described in the following two steps:
 * 1. Append any lowercase letter that is not present in the string to its end.
 *    - For example, if the string is "abc", the letters 'd', 'e', or 'y' can be added to it,
but
 *      not 'a'. If 'd' is added, the resulting string will be "abcd".
 * 2. Rearrange the letters of the new string in any arbitrary order.
 *    - For example, "abcd" can be rearranged to "acbd", "bacd", "cbda", and so on. Note that
it
 *      can also be rearranged to "abcd" itself.
 *
 * Return the number of strings in targetWords that can be obtained by performing the
operations
 * on any string of startWords.
 *
 * Note that you will only be verifying if the string in targetWords can be obtained from a
string
 * in startWords by performing the operations. The strings in startWords do not actually
change
 * during this process.
 */

/**
 * @param {string[]} startWords
 * @param {string[]} targetWords
 * @return {number}
 */
var wordCount = function(startWords, targetWords) {
  const sortedStartWords = new Set(startWords.map(word => [...word].sort().join('')));
  let result = 0;

  for (const target of targetWords) {
    const sortedTarget = [...target].sort().join('');
    for (let i = 0; i < target.length; i++) {
      const candidate = sortedTarget.slice(0, i) + sortedTarget.slice(i + 1);
      if (sortedStartWords.has(candidate)) {
        result++;
        break;
      }
    }
  }

  return result;
};
```

2136 Earliest Possible Day Of Full Bloom

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2136. Earliest Possible Day of Full Bloom
 * https://leetcode.com/problems/earliest-possible-day-of-full-bloom/
 * Difficulty: Hard
 *
 * You have n flower seeds. Every seed must be planted first before it can begin to grow, then
 * bloom. Planting a seed takes time and so does the growth of a seed. You are given two
0-indexed
 * integer arrays plantTime and growTime, of length n each:
 * - plantTime[i] is the number of full days it takes you to plant the ith seed. Every day,
you can
 *   work on planting exactly one seed. You do not have to work on planting the same seed on
 *   consecutive days, but the planting of a seed is not complete until you have worked
plantTime[i]
 *   days on planting it in total.
 * - growTime[i] is the number of full days it takes the ith seed to grow after being
completely
 *   planted. After the last day of its growth, the flower blooms and stays bloomed forever.
 *
 * From the beginning of day 0, you can plant the seeds in any order.
 *
 * Return the earliest possible day where all seeds are blooming.
 */

/**
 * @param {number[]} plantTime
 * @param {number[]} growTime
 * @return {number}
 */
var earliestFullBloom = function(plantTime, growTime) {
  const seeds = plantTime.map((plant, index) => ({ plant, grow: growTime[index] }));
  seeds.sort((a, b) => b.grow - a.grow);

  let plantingDays = 0;
  let result = 0;

  for (const { plant, grow } of seeds) {
    plantingDays += plant;
    result = Math.max(result, plantingDays + grow);
  }

  return result;
};
```

2137 Pour Water Between Buckets To Make Water Levels Equal

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2137. Pour Water Between Buckets to Make Water Levels Equal
 * https://leetcode.com/problems/pour-water-between-buckets-to-make-water-levels-equal/
 * Difficulty: Medium
 *
 * You have n buckets each containing some gallons of water in it, represented by a 0-indexed
 * integer array buckets, where the ith bucket contains buckets[i] gallons of water. You are
 * also given an integer loss.
 *
 * You want to make the amount of water in each bucket equal. You can pour any amount of water
 * from one bucket to another bucket (not necessarily an integer). However, every time you
 * pour k gallons of water, you spill loss percent of k.
 *
```

```
 * Return the maximum amount of water in each bucket after making the amount of water equal.
 * Answers within 10-5 of the actual answer will be accepted.
 */

/**
 * @param {number[]} buckets
 * @param {number} loss
 * @return {number}
 */
var equalizeWater = function(buckets, loss) {
  const n = buckets.length;
  const totalWater = buckets.reduce((sum, water) => sum + water, 0);
  const retentionRate = (100 - loss) / 100;

  let left = 0;
  let right = Math.max(...buckets);

  while (right - left > 1e-7) {
    const mid = (left + right) / 2;

    if (canAchieveLevel(mid)) {
      left = mid;
    } else {
      right = mid;
    }
  }

  return left;

  function canAchieveLevel(targetLevel) {
    let surplus = 0;
    let deficit = 0;

    for (const water of buckets) {
      if (water > targetLevel) {
        surplus += water - targetLevel;
      } else {
        deficit += targetLevel - water;
      }
    }

    return surplus * retentionRate >= deficit;
  }
};
```

2138 Divide A String Into Groups Of Size K

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2138. Divide a String Into Groups of Size k
 * https://leetcode.com/problems/divide-a-string-into-groups-of-size-k/
 * Difficulty: Easy
 *
 * A string s can be partitioned into groups of size k using the following procedure:
 * - The first group consists of the first k characters of the string, the second group
consists
 *   of the next k characters of the string, and so on. Each element can be a part of exactly
one
 *   group.
 * - For the last group, if the string does not have k characters remaining, a character fill
is
 *   used to complete the group.
 *
 * Note that the partition is done so that after removing the fill character from the last
group
 * (if it exists) and concatenating all the groups in order, the resultant string should be s.
 *
 * Given the string s, the size of each group k and the character fill, return a string array
 * denoting the composition of every group s has been divided into, using the above procedure.
```

```
 */

/**
 * @param {string} s
 * @param {number} k
 * @param {character} fill
 * @return {string[]}
 */
var divideString = function(s, k, fill) {
  const paddedString = s + fill.repeat((k - s.length % k) % k);
  const result = [];

  for (let i = 0; i < paddedString.length; i += k) {
    result.push(paddedString.slice(i, i + k));
  }

  return result;
};
```

2139 Minimum Moves To Reach Target Score

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2139. Minimum Moves to Reach Target Score
 * https://leetcode.com/problems/minimum-moves-to-reach-target-score/
 * Difficulty: Medium
 *
 * You are playing a game with integers. You start with the integer 1 and you want to reach
 * the integer target.
 *
 * In one move, you can either:
 * - Increment the current integer by one (i.e., x = x + 1).
 * - Double the current integer (i.e., x = 2 * x).
 *
 * You can use the increment operation any number of times, however, you can only use the
double
 * operation at most maxDoubles times.
 *
 * Given the two integers target and maxDoubles, return the minimum number of moves needed to
 * reach target starting with 1.
 */

/**
 * @param {number} target
 * @param {number} maxDoubles
 * @return {number}
 */
var minMoves = function(target, maxDoubles) {
  let moves = 0;
  let current = target;

  while (current > 1 && maxDoubles > 0) {
    if (current % 2 === 0) {
      current /= 2;
      maxDoubles--;
    } else {
      current--;
    }
    moves++;
  }

  return moves + (current - 1);
};
```

2140 Solving Questions With Brainpower

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2140. Solving Questions With Brainpower
 * https://leetcode.com/problems/solving-questions-with-brainpower/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D integer array questions where questions[i] = [pointsi,
brainpoweri].
 *
 * The array describes the questions of an exam, where you have to process the questions in
order
 * (i.e., starting from question 0) and make a decision whether to solve or skip each
question.
 * Solving question i will earn you pointsi points but you will be unable to solve each of the
 next
 * brainpoweri questions. If you skip question i, you get to make the decision on the next
question.
 *
 * For example, given questions = [[3, 2], [4, 3], [4, 4], [2, 5]]:
 * - If question 0 is solved, you will earn 3 points but you will be unable to solve questions
 *   1 and 2.
 * - If instead, question 0 is skipped and question 1 is solved, you will earn 4 points but
you
 *   will be unable to solve questions 2 and 3.
 *
 * Return the maximum points you can earn for the exam.
 */

/**
 * @param {number[][]} questions
 * @return {number}
 */
var mostPoints = function(questions) {
  const dp = new Array(questions.length + 1).fill(0);

  for (let i = questions.length - 1; i >= 0; i--) {
    const [points, brainpower] = questions[i];
    const nextAvailable = Math.min(questions.length, i + brainpower + 1);
    dp[i] = Math.max(points + dp[nextAvailable], dp[i + 1]);
  }

  return dp[0];
};
```

2141 Maximum Running Time Of N Computers

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2141. Maximum Running Time of N Computers
 * https://leetcode.com/problems/maximum-running-time-of-n-computers/
 * Difficulty: Hard
 *
 * You have n computers. You are given the integer n and a 0-indexed integer array batteries
 * where the ith battery can run a computer for batteries[i] minutes. You are interested in
 * running all n computers simultaneously using the given batteries.
 *
 * Initially, you can insert at most one battery into each computer. After that and at any
 * integer time moment, you can remove a battery from a computer and insert another battery
 * any number of times. The inserted battery can be a totally new battery or a battery from
 * another computer. You may assume that the removing and inserting processes take no time.
 *
```

```
 * Note that the batteries cannot be recharged.
 *
 * Return the maximum number of minutes you can run all the n computers simultaneously.
 */

/**
 * @param {number} n
 * @param {number[]} batteries
 * @return {number}
 */
var maxRunTime = function(n, batteries) {
  let left = 1;
  let right = Math.floor(batteries.reduce((sum, battery) => sum + battery, 0) / n);

  while (left < right) {
    const mid = Math.floor((left + right + 1) / 2);
    const total = batteries.reduce((sum, battery) => sum + Math.min(battery, mid), 0);

    if (total >= n * mid) {
      left = mid;
    } else {
      right = mid - 1;
    }
  }

  return left;
};
```

2143 Choose Numbers From Two Arrays In Range

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2143. Choose Numbers From Two Arrays in Range
 * https://leetcode.com/problems/choose-numbers-from-two-arrays-in-range/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays nums1 and nums2 of length n.
 *
 * A range [l, r] (inclusive) where 0 <= l <= r < n is balanced if:
 * - For every i in the range [l, r], you pick either nums1[i] or nums2[i].
 * - The sum of the numbers you pick from nums1 equals to the sum of the numbers you pick from
 *   nums2 (the sum is considered to be 0 if you pick no numbers from an array).
 *
 * Two balanced ranges from [l1, r1] and [l2, r2] are considered to be different if at least
 * one of the following is true:
 * - l1 != l2
 * - r1 != r2
 * - nums1[i] is picked in the first range, and nums2[i] is picked in the second range or vice
 *   versa for at least one i.
 *
 * Return the number of different ranges that are balanced. Since the answer may be very
large,
 * return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var countSubranges = function(nums1, nums2) {
  const MOD = 1e9 + 7;
  const n = nums1.length;
  let dp = new Map();
  let result = 0;

  for (let i = 0; i < n; i++) {
    const n1 = nums1[i];
    const n2 = nums2[i];
```

```
    const newDp = new Map();

    newDp.set(n1, (newDp.get(n1) || 0) + 1);
    newDp.set(-n2, (newDp.get(-n2) || 0) + 1);

    for (const [key, value] of dp) {
      newDp.set(key + n1, (newDp.get(key + n1) || 0) + value);
      newDp.set(key - n2, (newDp.get(key - n2) || 0) + value);
    }

    for (const [key, value] of newDp) {
      newDp.set(key, value % MOD);
    }

    result = (result + (newDp.get(0) || 0)) % MOD;
    dp = newDp;
  }

  return result;
};
```

2144 Minimum Cost Of Buying Candies With Discount

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2144. Minimum Cost of Buying Candies With Discount
 * https://leetcode.com/problems/minimum-cost-of-buying-candies-with-discount/
 * Difficulty: Easy
 *
 * A shop is selling candies at a discount. For every two candies sold, the shop gives a
 * third candy for free.
 *
 * The customer can choose any candy to take away for free as long as the cost of the
 * chosen candy is less than or equal to the minimum cost of the two candies bought.
 *
 * - For example, if there are 4 candies with costs 1, 2, 3, and 4, and the customer buys
 *   candies with costs 2 and 3, they can take the candy with cost 1 for free, but not the
 *   candy with cost 4.
 *
 * Given a 0-indexed integer array cost, where cost[i] denotes the cost of the ith candy,
 * return the minimum cost of buying all the candies.
 */

/**
 * @param {number[]} cost
 * @return {number}
 */
var minimumCost = function(cost) {
  cost.sort((a, b) => b - a);
  let result = 0;

  for (let i = 0; i < cost.length; i += 3) {
    result += cost[i] + (cost[i + 1] || 0);
  }

  return result;
};
```

2145 Count The Hidden Sequences

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2145. Count the Hidden Sequences
```

```
 * https://leetcode.com/problems/count-the-hidden-sequences/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of n integers differences, which describes the differences
 * between each pair of consecutive integers of a hidden sequence of length (n + 1). More
formally,
 * call the hidden sequence hidden, then we have that differences[i] = hidden[i + 1] -
hidden[i].
 *
 * You are further given two integers lower and upper that describe the inclusive range of
values
 * [lower, upper] that the hidden sequence can contain.
 *
 * - For example, given differences = [1, -3, 4], lower = 1, upper = 6, the hidden sequence is
 a
 *   sequence of length 4 whose elements are in between 1 and 6 (inclusive).
 *   - [3, 4, 1, 5] and [4, 5, 2, 6] are possible hidden sequences.
 *   - [5, 6, 3, 7] is not possible since it contains an element greater than 6.
 *   - [1, 2, 3, 4] is not possible since the differences are not correct.
 *
 * Return the number of possible hidden sequences there are. If there are no possible
sequences,
 * return 0.
 */

/**
 * @param {number[]} differences
 * @param {number} lower
 * @param {number} upper
 * @return {number}
 */
var numberOfArrays = function(differences, lower, upper) {
  let minValue = 0;
  let maxValue = 0;
  let current = 0;

  for (const diff of differences) {
    current += diff;
    minValue = Math.min(minValue, current);
    maxValue = Math.max(maxValue, current);
  }

  const range = upper - lower;
  const validRange = range - (maxValue - minValue);

  return validRange >= 0 ? validRange + 1 : 0;
};
```

2146 K Highest Ranked Items Within A Price Range

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2146. K Highest Ranked Items Within a Price Range
 * https://leetcode.com/problems/k-highest-ranked-items-within-a-price-range/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D integer array grid of size m x n that represents a map of the
 * items in a shop. The integers in the grid represent the following:
 * - 0 represents a wall that you cannot pass through.
 * - 1 represents an empty cell that you can freely move to and from.
 *
 * All other positive integers represent the price of an item in that cell. You may also
freely
 * move to and from these item cells.
 *
 * It takes 1 step to travel between adjacent grid cells.
 *
 * You are also given integer arrays pricing and start where pricing = [low, high] and
```

```
 * start = [row, col] indicates that you start at the position (row, col) and are interested
 * only in items with a price in the range of [low, high] (inclusive). You are further given
 * an integer k.
 *
 * You are interested in the positions of the k highest-ranked items whose prices are within
 * the given price range. The rank is determined by the first of these criteria that is
different:
 * 1. Distance, defined as the length of the shortest path from the start (shorter distance
has
 *    a higher rank).
 * 2. Price (lower price has a higher rank, but it must be in the price range).
 * 3. The row number (smaller row number has a higher rank).
 * 4. The column number (smaller column number has a higher rank).
 *
 * Return the k highest-ranked items within the price range sorted by their rank (highest to
 * lowest). If there are fewer than k reachable items within the price range, return all of
them.
 */

/**
 * @param {number[][]} grid
 * @param {number[]} pricing
 * @param {number[]} start
 * @param {number} k
 * @return {number[][]}
 */
var highestRankedKItems = function(grid, pricing, start, k) {
  const [rows, cols] = [grid.length, grid[0].length];
  const [low, high] = pricing;
  const [startRow, startCol] = start;
  const directions = [[0, 1], [0, -1], [1, 0], [-1, 0]];
  const queue = [[startRow, startCol, 0]];
  const visited = new Set();
  visited.add(`${startRow},${startCol}`);
  const validItems = [];

  while (queue.length > 0) {
    const [row, col, distance] = queue.shift();
    const price = grid[row][col];

    if (price >= low && price <= high) {
      validItems.push([distance, price, row, col]);
    }

    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;
      const key = `${newRow},${newCol}`;

      if (newRow >= 0 && newRow < rows && newCol >= 0 && newCol < cols
          && grid[newRow][newCol] > 0 && !visited.has(key)) {
        visited.add(key);
        queue.push([newRow, newCol, distance + 1]);
      }
    }
  }

  validItems.sort((a, b) => {
    if (a[0] !== b[0]) return a[0] - b[0];
    if (a[1] !== b[1]) return a[1] - b[1];
    if (a[2] !== b[2]) return a[2] - b[2];
    return a[3] - b[3];
  });

  return validItems.slice(0, k).map(item => [item[2], item[3]]);
};
```

2147 Number Of Ways To Divide A Long Corridor

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2147. Number of Ways to Divide a Long Corridor
 * https://leetcode.com/problems/number-of-ways-to-divide-a-long-corridor/
 * Difficulty: Hard
 *
 * Along a long library corridor, there is a line of seats and decorative plants. You are
given
 * a 0-indexed string corridor of length n consisting of letters 'S' and 'P' where each 'S'
 * represents a seat and each 'P' represents a plant.
 *
 * One room divider has already been installed to the left of index 0, and another to the
right
 * of index n - 1. Additional room dividers can be installed. For each position between
indices
 * i - 1 and i (1 <= i <= n - 1), at most one divider can be installed.
 *
 * Divide the corridor into non-overlapping sections, where each section has exactly two seats
 * with any number of plants. There may be multiple ways to perform the division. Two ways are
 * different if there is a position with a room divider installed in the first way but not in
 * the second way.
 *
 * Return the number of ways to divide the corridor. Since the answer may be very large,
return
 * it modulo 109 + 7. If there is no way, return 0.
 */

/**
 * @param {string} corridor
 * @return {number}
 */
var numberOfWays = function(corridor) {
  const MOD = 1e9 + 7;
  let seatCount = 0;
  let result = 1;
  let lastPairEnd = -1;

  for (let i = 0; i < corridor.length; i++) {
    if (corridor[i] === 'S') {
      seatCount++;
    }
  }
  if (seatCount === 0 || seatCount % 2 !== 0) {
    return 0;
  }

  seatCount = 0;

  for (let i = 0; i < corridor.length; i++) {
    if (corridor[i] === 'S') {
      seatCount++;

      if (seatCount % 2 === 0) {
        lastPairEnd = i;
      } else if (seatCount > 1) {
        const plantsCount = i - lastPairEnd - 1;
        result = (result * (plantsCount + 1)) % MOD;
      }
    }
  }

  return result;
};
```

2148 Count Elements With Strictly Smaller And Greater Elements

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2148. Count Elements With Strictly Smaller and Greater Elements
```

```
 * https://leetcode.com/problems/count-elements-with-strictly-smaller-and-greater-elements/
 * Difficulty: Easy
 *
 * Given an integer array nums, return the number of elements that have both a strictly
 * smaller and a strictly greater element appear in nums.
 */

/**
* @param {number[]} nums
* @return {number}
*/
var countElements = function(nums) {
  let result = 0;

  if (nums.length <= 2) return 0;
  for (const num of nums) {
    if (num > Math.min(...nums) && num < Math.max(...nums)) {
      result++;
    }
  }

  return result;
};
```

2149 Rearrange Array Elements By Sign

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2149. Rearrange Array Elements by Sign
 * https://leetcode.com/problems/rearrange-array-elements-by-sign/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of even length consisting of an equal number
 * of positive and negative integers.
 *
 * You should return the array of nums such that the the array follows the given conditions:
 * 1. Every consecutive pair of integers have opposite signs.
 * 2. For all integers with the same sign, the order in which they were present in nums is
 *    preserved.
 * 3. The rearranged array begins with a positive integer.
 *
 * Return the modified array after rearranging the elements to satisfy the aforementioned
 * conditions.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var rearrangeArray = function(nums) {
  const positives = nums.filter(num => num > 0);
  const negatives = nums.filter(num => num < 0);
  const result = [];

  for (let i = 0; i < positives.length; i++) {
    result.push(positives[i], negatives[i]);
  }

  return result;
};
```

2150 Find All Lonely Numbers In The Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2150. Find All Lonely Numbers in the Array
 * https://leetcode.com/problems/find-all-lonely-numbers-in-the-array/
 * Difficulty: Medium
 *
 * You are given an integer array nums. A number x is lonely when it appears only once, and
 * no adjacent numbers (i.e. x + 1 and x - 1) appear in the array.
 *
 * Return all lonely numbers in nums. You may return the answer in any order.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var findLonely = function(nums) {
  const map = new Map();

  for (const num of nums) {
    map.set(num, (map.get(num) || 0) + 1);
  }

  const result = [];
  for (const [num, count] of map) {
    if (count === 1 && !map.has(num - 1) && !map.has(num + 1)) {
      result.push(num);
    }
  }

  return result;
};
```

2151 Maximum Good People Based On Statements

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2151. Maximum Good People Based on Statements
 * https://leetcode.com/problems/maximum-good-people-based-on-statements/
 * Difficulty: Hard
 *
 * There are two types of persons:
 * - The good person: The person who always tells the truth.
 * - The bad person: The person who might tell the truth and might lie.
 *
 * You are given a 0-indexed 2D integer array statements of size n x n that represents the
 * statements made by n people about each other. More specifically, statements[i][j] could
 * be one of the following:
 * - 0 which represents a statement made by person i that person j is a bad person.
 * - 1 which represents a statement made by person i that person j is a good person.
 * - 2 represents that no statement is made by person i about person j.
 *
 * Additionally, no person ever makes a statement about themselves. Formally, we have that
 * statements[i][i] = 2 for all 0 <= i < n.
 *
 * Return the maximum number of people who can be good based on the statements made by the
 * n people.
 */

/**
 * @param {number[][]} statements
 * @return {number}
 */
var maximumGood = function(statements) {
  const n = statements.length;
  let result = 0;

  backtrack(0, new Array(n).fill(false), 0);
  return result;
```

```
  function isValid(goodPeople) {
    for (let i = 0; i < n; i++) {
      if (!goodPeople[i]) continue;
      for (let j = 0; j < n; j++) {
        if (statements[i][j] === 0 && goodPeople[j]) return false;
        if (statements[i][j] === 1 && !goodPeople[j]) return false;
      }
    }
    return true;
  }

  function backtrack(index, goodPeople, goodCount) {
    if (index === n) {
      if (isValid(goodPeople)) {
        result = Math.max(result, goodCount);
      }
      return;
    }

    goodPeople[index] = true;
    backtrack(index + 1, goodPeople, goodCount + 1);
    goodPeople[index] = false;
    backtrack(index + 1, goodPeople, goodCount);
  }
};
```

2152 Minimum Number Of Lines To Cover Points

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2152. Minimum Number of Lines to Cover Points
 * https://leetcode.com/problems/minimum-number-of-lines-to-cover-points/
 * Difficulty: Medium
 *
 * You are given an array points where points[i] = [xi, yi] represents a point on an X-Y
plane.
 *
 * Straight lines are going to be added to the X-Y plane, such that every point is covered by
at
 * least one line.
 *
 * Return the minimum number of straight lines needed to cover all the points.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var minimumLines = function(points) {
  const n = points.length;
  const linePoints = new Map();

  for (let i = 0; i < n; i++) {
    const [x1, y1] = points[i];
    for (let j = i + 1; j < n; j++) {
      const [x2, y2] = points[j];
      let a;
      let b;

      if (x1 === x2) {
        a = x1;
        b = Number.MAX_SAFE_INTEGER;
      } else {
        a = (y2 - y1) / (x2 - x1);
        b = y1 - a * x1;
      }

      const key = `${a},${b}`;
```

```
      if (!linePoints.has(key)) {
        linePoints.set(key, new Set());
      }
      linePoints.get(key).add(`${x1},${y1}`);
      linePoints.get(key).add(`${x2},${y2}`);
    }
  }

  const lines = [];
  for (const [line, pointSet] of linePoints) {
    if (pointSet.size > 2) {
      lines.push(line);
    }
  }

  let answer = Math.ceil(n / 2);
  const m = lines.length;

  for (let mask = 1; mask < (1 << m); mask++) {
    let j = 0;
    const lineCount = mask.toString(2).split('1').length - 1;
    const currentPoints = new Set();
    let tempMask = mask;

    while (tempMask > 0) {
      if (tempMask % 2) {
        const line = lines[m - 1 - j];
        for (const point of linePoints.get(line)) {
          currentPoints.add(point);
        }
      }
      tempMask >>= 1;
      j++;
    }

    answer = Math.min(answer, lineCount + Math.ceil((n - currentPoints.size) / 2));
  }

  return answer;
};
```

2154 Keep Multiplying Found Values By Two

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2154. Keep Multiplying Found Values by Two
 * https://leetcode.com/problems/keep-multiplying-found-values-by-two/
 * Difficulty: Easy
 *
 * You are given an array of integers nums. You are also given an integer original which
 * is the first number that needs to be searched for in nums.
 *
 * You then do the following steps:
 * If original is found in nums, multiply it by two (i.e., set original = 2 * original).
 * Otherwise, stop the process.
 * Repeat this process with the new number as long as you keep finding the number.
 * Return the final value of original.
 */

/**
 * @param {number[]} nums
 * @param {number} original
 * @return {number}
 */
var findFinalValue = function(nums, original) {
  return nums
    .sort((a, b) => a - b)
    .reduce((result, n) => result *= n === result ? 2 : 1, original);
};
```

2155 All Divisions With The Highest Score Of A Binary Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2155. All Divisions With the Highest Score of a Binary Array
 * https://leetcode.com/problems/all-divisions-with-the-highest-score-of-a-binary-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed binary array nums of length n. nums can be divided at index i
 * (where 0 <= i <= n) into two arrays (possibly empty) numsleft and numsright:
 * - numsleft has all the elements of nums between index 0 and i - 1 (inclusive), while
numsright
 *   has all the elements of nums between index i and n - 1 (inclusive).
 * - If i == 0, numsleft is empty, while numsright has all the elements of nums.
 * - If i == n, numsleft has all the elements of nums, while numsright is empty.
 *
 * The division score of an index i is the sum of the number of 0's in numsleft and the number
 * of 1's in numsright.
 *
 * Return all distinct indices that have the highest possible division score. You may return
 * the answer in any order.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var maxScoreIndices = function(nums) {
  const result = [0];
  let leftZeros = 0;
  let rightOnes = nums.reduce((sum, num) => sum + num, 0);
  let maxScore = rightOnes;

  for (let i = 0; i < nums.length; i++) {
    leftZeros += nums[i] === 0 ? 1 : 0;
    rightOnes -= nums[i];
    const score = leftZeros + rightOnes;

    if (score > maxScore) {
      maxScore = score;
      result.length = 0;
      result.push(i + 1);
    } else if (score === maxScore) {
      result.push(i + 1);
    }
  }

  return result;
};
```

2156 Find Substring With Given Hash Value

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2156. Find Substring With Given Hash Value
 * https://leetcode.com/problems/find-substring-with-given-hash-value/
 * Difficulty: Hard
 *
 * The hash of a 0-indexed string s of length k, given integers p and m, is computed using
 * the following function:
 * - hash(s, p, m) = (val(s[0]) * p0 + val(s[1]) * p1 + ... + val(s[k-1]) * pk-1) mod m.
 *
 * Where val(s[i]) represents the index of s[i] in the alphabet from val('a') = 1 to val('z')
```

```
= 26.
 *
 * You are given a string s and the integers power, modulo, k, and hashValue. Return sub, the
 * first substring of s of length k such that hash(sub, power, modulo) == hashValue.
 *
 * The test cases will be generated such that an answer always exists.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
* @param {string} s
* @param {number} power
* @param {number} modulo
* @param {number} k
* @param {number} hashValue
* @return {string}
*/
var subStrHash = function(s, power, modulo, k, hashValue) {
  power = BigInt(power);
  modulo = BigInt(modulo);
  hashValue = BigInt(hashValue);

  const powers = new Array(k);
  powers[0] = 1n;
  for (let i = 1; i < k; i++) {
    powers[i] = (powers[i - 1] * power) % modulo;
  }

  for (let start = 0; start <= s.length - k; start++) {
    let hash = 0n;
    for (let i = 0; i < k; i++) {
      const charVal = BigInt(s.charCodeAt(start + i) - 'a'.charCodeAt(0) + 1);
      hash = (hash + charVal * powers[i]) % modulo;
    }
    if (hash === hashValue) {
      return s.substring(start, start + k);
    }
  }

  return '';
};
```

2157 Groups Of Strings

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2157. Groups of Strings
 * https://leetcode.com/problems/groups-of-strings/
 * Difficulty: Hard
 *
 * You are given a 0-indexed array of strings words. Each string consists of lowercase English
 * letters only. No letter occurs more than once in any string of words.
 *
 * Two strings s1 and s2 are said to be connected if the set of letters of s2 can be obtained
 * from the set of letters of s1 by any one of the following operations:
 * - Adding exactly one letter to the set of the letters of s1.
 * - Deleting exactly one letter from the set of the letters of s1.
 * - Replacing exactly one letter from the set of the letters of s1 with any letter, including
 *   itself.
 *
 * The array words can be divided into one or more non-intersecting groups. A string belongs
 * to a group if any one of the following is true:
 * - It is connected to at least one other string of the group.
 * - It is the only string present in the group.
 *
 * Note that the strings in words should be grouped in such a manner that a string belonging
 * to a group cannot be connected to a string present in any other group. It can be proved
 * that such an arrangement is always unique.
```

```
 *
 * Return an array ans of size 2 where:
 * - ans[0] is the maximum number of groups words can be divided into, and
 * - ans[1] is the size of the largest group.
 */

/**
 * @param {string[]} words
 * @return {number[]}
 */
var groupStrings = function(words) {
  const n = words.length;
  const masks = words.map(word => {
    let mask = 0;
    for (let i = 0; i < word.length; i++) {
      mask |= (1 << (word.charCodeAt(i) - 'a'.charCodeAt(0)));
    }
    return mask;
  });
  const uf = new UnionFind(n);
  const maskToIndex = new Map();
  for (let i = 0; i < n; i++) {
    if (maskToIndex.has(masks[i])) {
      uf.union(maskToIndex.get(masks[i]), i);
    } else {
      maskToIndex.set(masks[i], i);
    }
  }
  const processed = new Set();

  for (let i = 0; i < n; i++) {
    const mask = masks[i];
    if (processed.has(mask)) continue;
    processed.add(mask);

    for (let bit = 0; bit < 26; bit++) {
      if ((mask & (1 << bit)) === 0) {
        const newMask = mask | (1 << bit);
        if (maskToIndex.has(newMask)) {
          uf.union(i, maskToIndex.get(newMask));
        }
      }
    }

    for (let bit = 0; bit < 26; bit++) {
      if ((mask & (1 << bit)) !== 0) {
        const newMask = mask & ~(1 << bit);
        if (maskToIndex.has(newMask)) {
          uf.union(i, maskToIndex.get(newMask));
        }
      }
    }

    for (let remove = 0; remove < 26; remove++) {
      if ((mask & (1 << remove)) !== 0) {
        for (let add = 0; add < 26; add++) {
          if ((mask & (1 << add)) === 0) {
            const newMask = (mask & ~(1 << remove)) | (1 << add);
            if (maskToIndex.has(newMask)) {
              uf.union(i, maskToIndex.get(newMask));
            }
          }
        }
      }
    }
  }

  return [uf.count, uf.getMaxSize()];
};

class UnionFind {
  constructor(n) {
    this.parent = Array(n).fill().map((_, i) => i);
    this.rank = Array(n).fill(0);
    this.count = n;
```

```
    this.sizes = Array(n).fill(1);
  }

  find(x) {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]);
    }
    return this.parent[x];
  }

  union(x, y) {
    const rootX = this.find(x);
    const rootY = this.find(y);

    if (rootX === rootY) return false;

    if (this.rank[rootX] < this.rank[rootY]) {
      this.parent[rootX] = rootY;
      this.sizes[rootY] += this.sizes[rootX];
    } else if (this.rank[rootX] > this.rank[rootY]) {
      this.parent[rootY] = rootX;
      this.sizes[rootX] += this.sizes[rootY];
    } else {
      this.parent[rootY] = rootX;
      this.rank[rootX]++;
      this.sizes[rootX] += this.sizes[rootY];
    }

    this.count--;
    return true;
  }

  getMaxSize() {
    let maxSize = 0;
    for (let i = 0; i < this.parent.length; i++) {
      if (this.parent[i] === i) {
        maxSize = Math.max(maxSize, this.sizes[i]);
      }
    }
    return maxSize;
  }
}
```

2158 Amount Of New Area Painted Each Day

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2158. Amount of New Area Painted Each Day
 * https://leetcode.com/problems/amount-of-new-area-painted-each-day/
 * Difficulty: Hard
 *
 * There is a long and thin painting that can be represented by a number line. You are given
 * a 0-indexed 2D integer array paint of length n, where paint[i] = [starti, endi]. This means
 * that on the ith day you need to paint the area between starti and endi.
 *
 * Painting the same area multiple times will create an uneven painting so you only want to
 * paint each area of the painting at most once.
 *
 * Return an integer array worklog of length n, where worklog[i] is the amount of new area
that
 * you painted on the ith day.
 */

/**
 * @param {number[][]} paint
 * @return {number[]}
 */
var amountPainted = function(paint) {
```

```
  const intervals = new Map();
  const result = [];

  for (const [start, end] of paint) {
    let left = start;
    let right = end;
    let paintAmount = right - left;

    const keys = Array.from(intervals.keys()).sort((a, b) => a - b);
    const toRemove = [];
    let merged = false;

    for (const key of keys) {
      const value = intervals.get(key);

      if (key > right || value < left) continue;

      if (!merged && key <= left && value >= left) {
        left = Math.min(left, key);
        right = Math.max(right, value);
        paintAmount = Math.max(0, end - Math.max(start, value));
        toRemove.push(key);
        merged = true;
      } else if (key >= left && key < right) {
        paintAmount -= Math.min(right, value) - key;
        right = Math.max(right, value);
        toRemove.push(key);
      } else if (value > left && value <= right) {
        paintAmount -= value - Math.max(left, key);
        left = Math.min(left, key);
        toRemove.push(key);
      }
    }

    for (const key of toRemove) {
      intervals.delete(key);
    }

    intervals.set(left, right);
    result.push(Math.max(0, paintAmount));
  }

  return result;
};
```

2160 Minimum Sum Of Four Digit Number After Splitting Digits

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2160. Minimum Sum of Four Digit Number After Splitting Digits
 * https://leetcode.com/problems/minimum-sum-of-four-digit-number-after-splitting-digits/
 * Difficulty: Easy
 *
 * You are given a positive integer num consisting of exactly four digits. Split num into two
 * new integers new1 and new2 by using the digits found in num. Leading zeros are allowed in
 * new1 and new2, and all the digits found in num must be used.
 * - For example, given num = 2932, you have the following digits: two 2's, one 9 and one 3.
 *   Some of the possible pairs [new1, new2] are [22, 93], [23, 92], [223, 9] and [2, 329].
 *
 * Return the minimum possible sum of new1 and new2.
 */

/**
 * @param {number} num
 * @return {number}
 */
var minimumSum = function(num) {
  const digits = String(num).split('').map(Number).sort((a, b) => a - b);
  return (digits[0] * 10 + digits[2]) + (digits[1] * 10 + digits[3]);
```

```
};
```

2161 Partition Array According To Given Pivot

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2161. Partition Array According to Given Pivot
 * https://leetcode.com/problems/partition-array-according-to-given-pivot/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums and an integer pivot. Rearrange nums such that
 * the following conditions are satisfied:
 * - Every element less than pivot appears before every element greater than pivot.
 * - Every element equal to pivot appears in between the elements less than and greater than
 *   pivot.
 * - The relative order of the elements less than pivot and the elements greater than pivot
 *   is maintained.
 *   - More formally, consider every pi, pj where pi is the new position of the ith element
 *     and pj is the new position of the jth element. If i < j and both elements are smaller
 *     (or larger) than pivot, then pi < pj.
 *
 * Return nums after the rearrangement.
 */

/**
 * @param {number[]} nums
 * @param {number} pivot
 * @return {number[]}
 */
var pivotArray = function(nums, pivot) {
  return [
    ...nums.filter(x => x < pivot),
    ...nums.filter(x => x === pivot),
    ...nums.filter(x => x > pivot),
  ];
};
```

2162 Minimum Cost To Set Cooking Time

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2162. Minimum Cost to Set Cooking Time
 * https://leetcode.com/problems/minimum-cost-to-set-cooking-time/
 * Difficulty: Medium
 *
 * A generic microwave supports cooking times for:
 * - at least 1 second.
 * - at most 99 minutes and 99 seconds.
 *
 * To set the cooking time, you push at most four digits. The microwave normalizes what you
push
 * as four digits by prepending zeroes. It interprets the first two digits as the minutes and
 * the last two digits as the seconds. It then adds them up as the cooking time. For example,
 * - You push 9 5 4 (three digits). It is normalized as 0954 and interpreted as 9 minutes and
 *   54 seconds.
 * - You push 0 0 0 8 (four digits). It is interpreted as 0 minutes and 8 seconds.
 * - You push 8 0 9 0. It is interpreted as 80 minutes and 90 seconds.
 * - You push 8 1 3 0. It is interpreted as 81 minutes and 30 seconds.
 *
 * You are given integers startAt, moveCost, pushCost, and targetSeconds. Initially, your
finger
 * is on the digit startAt. Moving the finger above any specific digit costs moveCost units of
 * fatigue. Pushing the digit below the finger once costs pushCost units of fatigue.
```

```
 *
 * There can be multiple ways to set the microwave to cook for targetSeconds seconds but you
are
 * interested in the way with the minimum cost.
 *
 * Return the minimum cost to set targetSeconds seconds of cooking time.
 *
 * Remember that one minute consists of 60 seconds.
 */

/**
 * @param {number} startAt
 * @param {number} moveCost
 * @param {number} pushCost
 * @param {number} targetSeconds
 * @return {number}
 */
var minCostSetTime = function(startAt, moveCost, pushCost, targetSeconds) {
  const minutes1 = Math.floor(targetSeconds / 60);
  const seconds1 = targetSeconds % 60;
  let result = Infinity;

  if (minutes1 <= 99) {
    const digits1 = getDigits(minutes1, seconds1);
    result = Math.min(result, calculateCost(digits1));
  }

  const minutes2 = Math.floor(targetSeconds / 60) - 1;
  const seconds2 = targetSeconds % 60 + 60;
  if (minutes2 >= 0 && minutes2 <= 99 && seconds2 <= 99) {
    const digits2 = getDigits(minutes2, seconds2);
    result = Math.min(result, calculateCost(digits2));
  }

  return result;

  function calculateCost(digits) {
    let totalCost = 0;
    let currentDigit = startAt;

    for (const digit of digits) {
      if (digit !== currentDigit) {
        totalCost += moveCost;
        currentDigit = digit;
      }
      totalCost += pushCost;
    }

    return totalCost;
  }

  function getDigits(minutes, seconds) {
    const result = [];

    if (minutes > 0) {
      if (minutes >= 10) {
        result.push(Math.floor(minutes / 10));
      }
      result.push(minutes % 10);
    }

    if (minutes > 0 || seconds >= 10) {
      result.push(Math.floor(seconds / 10));
    }

    result.push(seconds % 10);

    return result;
  }
};
```

2163 Minimum Difference In Sums After Removal Of Elements

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2163. Minimum Difference in Sums After Removal of Elements
 * https://leetcode.com/problems/minimum-difference-in-sums-after-removal-of-elements/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums consisting of 3 * n elements.
 *
 * You are allowed to remove any subsequence of elements of size exactly n from nums.
 * The remaining 2 * n elements will be divided into two equal parts:
 * - The first n elements belonging to the first part and their sum is sumfirst.
 * - The next n elements belonging to the second part and their sum is sumsecond.
 *
 * The difference in sums of the two parts is denoted as sumfirst - sumsecond.
 * - For example, if sumfirst = 3 and sumsecond = 2, their difference is 1.
 * - Similarly, if sumfirst = 2 and sumsecond = 3, their difference is -1.
 *
 * Return the minimum difference possible between the sums of the two parts after the removal
 * of n elements.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumDifference = function(nums) {
  const n = nums.length / 3;
  const minFirstPart = new Array(nums.length).fill(0);
  const maxSecondPart = new Array(nums.length).fill(0);
  const maxHeap = new PriorityQueue((a, b) => b - a);
  const minHeap = new PriorityQueue((a, b) => a - b);

  let currentSum = 0;
  for (let i = 0; i < n; i++) {
    maxHeap.enqueue(nums[i]);
    currentSum += nums[i];
  }
  minFirstPart[n - 1] = currentSum;

  for (let i = n; i < 2 * n; i++) {
    maxHeap.enqueue(nums[i]);
    currentSum += nums[i];
    const removed = maxHeap.dequeue();
    currentSum -= removed;
    minFirstPart[i] = currentSum;
  }

  currentSum = 0;
  minHeap.clear();
  for (let i = 2 * n; i < 3 * n; i++) {
    minHeap.enqueue(nums[i]);
    currentSum += nums[i];
  }
  maxSecondPart[2 * n] = currentSum;

  for (let i = 2 * n - 1; i >= n; i--) {
    minHeap.enqueue(nums[i]);
    currentSum += nums[i];
    const removed = minHeap.dequeue();
    currentSum -= removed;
    maxSecondPart[i] = currentSum;
  }

  let result = Infinity;
  for (let i = n - 1; i < 2 * n; i++) {
    const difference = minFirstPart[i] - maxSecondPart[i + 1];
    result = Math.min(result, difference);
```

```
  }

  return result;
};
```

2164 Sort Even And Odd Indices Independently

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2164. Sort Even and Odd Indices Independently
 * https://leetcode.com/problems/sort-even-and-odd-indices-independently/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums. Rearrange the values of nums according to the
 * following rules:
 * 1. Sort the values at odd indices of nums in non-increasing order.
 *    - For example, if nums = [4,1,2,3] before this step, it becomes [4,3,2,1] after. The
values
 *      at odd indices 1 and 3 are sorted in non-increasing order.
 * 2. Sort the values at even indices of nums in non-decreasing order.
 *    - For example, if nums = [4,1,2,3] before this step, it becomes [2,1,4,3] after. The
values
 *      at even indices 0 and 2 are sorted in non-decreasing order.
 *
 * Return the array formed after rearranging the values of nums.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var sortEvenOdd = function(nums) {
  const evens = [];
  const odds = [];

  for (let i = 0; i < nums.length; i++) {
    if (i % 2 === 0) {
      evens.push(nums[i]);
    } else {
      odds.push(nums[i]);
    }
  }

  evens.sort((a, b) => a - b);
  odds.sort((a, b) => b - a);

  const result = [];
  let evenIndex = 0;
  let oddIndex = 0;

  for (let i = 0; i < nums.length; i++) {
    if (i % 2 === 0) {
      result.push(evens[evenIndex++]);
    } else {
      result.push(odds[oddIndex++]);
    }
  }

  return result;
};
```

2165 Smallest Value Of The Rearranged Number

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2165. Smallest Value of the Rearranged Number
 * https://leetcode.com/problems/smallest-value-of-the-rearranged-number/
 * Difficulty: Medium
 *
 * You are given an integer num. Rearrange the digits of num such that its value is minimized
 * and it does not contain any leading zeros.
 *
 * Return the rearranged number with minimal value.
 *
 * Note that the sign of the number does not change after rearranging the digits.
 */

/**
 * @param {number} num
 * @return {number}
 */
var smallestNumber = function(num) {
  const isNegative = num < 0;
  const digits = Math.abs(num).toString().split('').map(Number);

  if (isNegative) {
    digits.sort((a, b) => b - a);
    return -parseInt(digits.join(''), 10);
  }

  digits.sort((a, b) => a - b);
  const firstNonZero = digits.findIndex(d => d !== 0);

  if (firstNonZero === -1) return 0;

  [digits[0], digits[firstNonZero]] = [digits[firstNonZero], digits[0]];
  return parseInt(digits.join(''), 10);
};
```

2166 Design Bitset

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2166. Design Bitset
 * https://leetcode.com/problems/design-bitset/
 * Difficulty: Medium
 *
 * A Bitset is a data structure that compactly stores bits.
 *
 * Implement the Bitset class:
 * - Bitset(int size) Initializes the Bitset with size bits, all of which are 0.
 * - void fix(int idx) Updates the value of the bit at the index idx to 1. If the value was
 *   already 1, no change occurs.
 * - void unfix(int idx) Updates the value of the bit at the index idx to 0. If the value
 *   was already 0, no change occurs.
 * - void flip() Flips the values of each bit in the Bitset. In other words, all bits with
 *   value 0 will now have value 1 and vice versa.
 * - boolean all() Checks if the value of each bit in the Bitset is 1. Returns true if it
 *   satisfies the condition, false otherwise.
 * - boolean one() Checks if there is at least one bit in the Bitset with value 1. Returns
 *   true if it satisfies the condition, false otherwise.
 * - int count() Returns the total number of bits in the Bitset which have value 1.
 * - String toString() Returns the current composition of the Bitset. Note that in the
 *   resultant string, the character at the ith index should coincide with the value at
 *   the ith bit of the Bitset.
 */

/**
 * @param {number} size
 */
var Bitset = function(size) {
  this.bits = new Uint8Array(size);
```

```
  this.ones = 0;
  this.flipped = false;
};

/**
 * @param {number} idx
 * @return {void}
 */
Bitset.prototype.fix = function(idx) {
  if (this.flipped) {
    if (this.bits[idx] === 1) {
      this.bits[idx] = 0;
      this.ones++;
    }
  } else {
    if (this.bits[idx] === 0) {
      this.bits[idx] = 1;
      this.ones++;
    }
  }
};

/**
 * @param {number} idx
 * @return {void}
 */
Bitset.prototype.unfix = function(idx) {
  if (this.flipped) {
    if (this.bits[idx] === 0) {
      this.bits[idx] = 1;
      this.ones--;
    }
  } else {
    if (this.bits[idx] === 1) {
      this.bits[idx] = 0;
      this.ones--;
    }
  }
};

/**
 * @return {void}
 */
Bitset.prototype.flip = function() {
  this.flipped = !this.flipped;
  this.ones = this.bits.length - this.ones;
};

/**
 * @return {boolean}
 */
Bitset.prototype.all = function() {
  return this.ones === this.bits.length;
};

/**
 * @return {boolean}
 */
Bitset.prototype.one = function() {
  return this.ones > 0;
};

/**
 * @return {number}
 */
Bitset.prototype.count = function() {
  return this.ones;
};

/**
 * @return {string}
 */
Bitset.prototype.toString = function() {
  let result = '';
  for (let i = 0; i < this.bits.length; i++) {
```

```
    result += this.flipped ? 1 - this.bits[i] : this.bits[i];
  }
  return result;
};
```

2167 Minimum Time To Remove All Cars Containing Illegal Goods

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2167. Minimum Time to Remove All Cars Containing Illegal Goods
 * https://leetcode.com/problems/minimum-time-to-remove-all-cars-containing-illegal-goods/
 * Difficulty: Hard
 *
 * You are given a 0-indexed binary string s which represents a sequence of train cars. s[i] =
'0'
 * denotes that the ith car does not contain illegal goods and s[i] = '1' denotes that the ith
car
 * does contain illegal goods.
 *
 * As the train conductor, you would like to get rid of all the cars containing illegal goods.
 * You can do any of the following three operations any number of times:
 * 1. Remove a train car from the left end (i.e., remove s[0]) which takes 1 unit of time.
 * 2. Remove a train car from the right end (i.e., remove s[s.length - 1]) which takes 1 unit
 *    of time.
 * 3. Remove a train car from anywhere in the sequence which takes 2 units of time.
 *
 * Return the minimum time to remove all the cars containing illegal goods.
 *
 * Note that an empty sequence of cars is considered to have no cars containing illegal goods.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumTime = function(s) {
  let leftCost = 0;
  let result = s.length;

  for (let i = 0; i < s.length; i++) {
    leftCost = Math.min(leftCost + (s[i] === '1' ? 2 : 0), i + 1);
    const rightCost = s.length - i - 1;
    result = Math.min(result, leftCost + rightCost);
  }

  return result;
};
```

2168 Unique Substrings With Equal Digit Frequency

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2168. Unique Substrings With Equal Digit Frequency
 * https://leetcode.com/problems/unique-substrings-with-equal-digit-frequency/
 * Difficulty: Medium
 *
 * Given a digit string s, return the number of unique substrings of s where every digit
appears
 * the same number of times.
 */

/**
 * @param {string} s
```

```
 * @return {number}
 */
var equalDigitFrequency = function(s) {
  const uniqueSubstrings = new Set();
  const n = s.length;

  for (let i = 0; i < n; i++) {
    const digitCount = new Map();

    for (let j = i; j < n; j++) {
      const digit = s[j];
      digitCount.set(digit, (digitCount.get(digit) || 0) + 1);

      if (hasEqualFrequency(digitCount)) {
        uniqueSubstrings.add(s.substring(i, j + 1));
      }
    }
  }

  return uniqueSubstrings.size;

  function hasEqualFrequency(counts) {
    const frequencies = Array.from(counts.values());
    return frequencies.every(freq => freq === frequencies[0]);
  }
};
```

2169 Count Operations To Obtain Zero

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2169. Count Operations to Obtain Zero
 * https://leetcode.com/problems/count-operations-to-obtain-zero/
 * Difficulty: Easy
 *
 * You are given two non-negative integers num1 and num2.
 *
 * In one operation, if num1 >= num2, you must subtract num2 from num1, otherwise subtract
num1
 * from num2.
 * - For example, if num1 = 5 and num2 = 4, subtract num2 from num1, thus obtaining num1 = 1
and
 *   num2 = 4. However, if num1 = 4 and num2 = 5, after one operation, num1 = 4 and num2 = 1.
 *
 * Return the number of operations required to make either num1 = 0 or num2 = 0.
 */

/**
 * @param {number} num1
 * @param {number} num2
 * @return {number}
 */
var countOperations = function(num1, num2) {
  let operations = 0;

  while (num1 !== 0 && num2 !== 0) {
    if (num1 >= num2) {
      num1 -= num2;
    } else {
      num2 -= num1;
    }
    operations++;
  }

  return operations;
};
```

2170 Minimum Operations To Make The Array Alternating

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2170. Minimum Operations to Make the Array Alternating
 * https://leetcode.com/problems/minimum-operations-to-make-the-array-alternating/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums consisting of n positive integers.
 *
 * The array nums is called alternating if:
 * - nums[i - 2] == nums[i], where 2 <= i <= n - 1.
 * - nums[i - 1] != nums[i], where 1 <= i <= n - 1.
 *
 * In one operation, you can choose an index i and change nums[i] into any positive integer.
 *
 * Return the minimum number of operations required to make the array alternating.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumOperations = function(nums) {
  const evenFreq = new Map();
  const oddFreq = new Map();

  if (nums.length <= 1) return 0;
  for (let i = 0; i < nums.length; i++) {
    if (i % 2 === 0) {
      evenFreq.set(nums[i], (evenFreq.get(nums[i]) || 0) + 1);
    } else {
      oddFreq.set(nums[i], (oddFreq.get(nums[i]) || 0) + 1);
    }
  }

  const evenTop = [...evenFreq.entries()].sort((a, b) => b[1] - a[1]).slice(0, 2);
  const oddTop = [...oddFreq.entries()].sort((a, b) => b[1] - a[1]).slice(0, 2);
  const evenTotal = Math.ceil(nums.length / 2);
  const oddTotal = Math.floor(nums.length / 2);
  let minOps = nums.length;

  for (const [evenNum, evenCount] of evenTop.length ? evenTop : [[0, 0]]) {
    for (const [oddNum, oddCount] of oddTop.length ? oddTop : [[0, 0]]) {
      if (evenNum !== oddNum || evenNum === 0) {
        minOps = Math.min(minOps, (evenTotal - evenCount) + (oddTotal - oddCount));
      }
    }
  }

  return minOps === nums.length ? Math.min(evenTotal, oddTotal) : minOps;
};
```

2171 Removing Minimum Number Of Magic Beans

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2171. Removing Minimum Number of Magic Beans
 * https://leetcode.com/problems/removing-minimum-number-of-magic-beans/
 * Difficulty: Medium
 *
```

```
 * You are given an array of positive integers beans, where each integer represents the number
 * of magic beans found in a particular magic bag.
 *
 * Remove any number of beans (possibly none) from each bag such that the number of beans in
 * each remaining non-empty bag (still containing at least one bean) is equal. Once a bean
 * has been removed from a bag, you are not allowed to return it to any of the bags.
 *
 * Return the minimum number of magic beans that you have to remove.
 */

/**
 * @param {number[]} beans
 * @return {number}
 */
var minimumRemoval = function(beans) {
  const sortedBeans = beans.sort((a, b) => a - b);
  const totalBeans = sortedBeans.reduce((sum, bean) => sum + bean, 0);
  let result = totalBeans;
  let remaining = totalBeans;

  for (let i = 0; i < sortedBeans.length; i++) {
    remaining -= sortedBeans[i];
    const equalBags = sortedBeans.length - i;
    result = Math.min(result, totalBeans - sortedBeans[i] * equalBags);
  }

  return result;
};
```

2172 Maximum And Sum Of Array

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2172. Maximum AND Sum of Array
 * https://leetcode.com/problems/maximum-and-sum-of-array/
 * Difficulty: Hard
 *
 * You are given an integer array nums of length n and an integer numSlots such that
 * 2 * numSlots >= n. There are numSlots slots numbered from 1 to numSlots.
 *
 * You have to place all n integers into the slots such that each slot contains at most
 * two numbers. The AND sum of a given placement is the sum of the bitwise AND of every
 * number with its respective slot number.
 * - For example, the AND sum of placing the numbers [1, 3] into slot 1 and [4, 6] into
 *   slot 2 is equal to (1 AND 1) + (3 AND 1) + (4 AND 2) + (6 AND 2) = 1 + 1 + 0 + 2 = 4.
 *
 * Return the maximum possible AND sum of nums given numSlots slots.
 */

/**
 * @param {number[]} nums
 * @param {number} numSlots
 * @return {number}
 */
var maximumANDSum = function(nums, numSlots) {
  const map = new Map();

  return dp(0, new Array(numSlots).fill(0));

  function dp(index, slots) {
    if (index >= nums.length) return 0;

    const key = `${index},${slots.join(',')}`;
    if (map.has(key)) return map.get(key);

    let result = 0;
    for (let i = 0; i < numSlots; i++) {
      if (slots[i] < 2) {
        slots[i]++;
```

```
        result = Math.max(result, (nums[index] & (i + 1)) + dp(index + 1, slots));
        slots[i]--;
      }
    }

    map.set(key, result);
    return result;
  }
};
```

2174 Remove All Ones With Row And Column Flips Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2174. Remove All Ones With Row and Column Flips II
 * https://leetcode.com/problems/remove-all-ones-with-row-and-column-flips-ii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed m x n binary matrix grid.
 *
 * In one operation, you can choose any i and j that meet the following conditions:
 * - 0 <= i < m
 * - 0 <= j < n
 * - grid[i][j] == 1
 *
 * and change the values of all cells in row i and column j to zero.
 *
 * Return the minimum number of operations needed to remove all 1's from grid.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var removeOnes = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const map = new Map();

  return solve(grid);

  function solve(currentGrid) {
    const key = currentGrid.map(row => row.join('')).join('|');
    if (map.has(key)) return map.get(key);

    if (hasNoOnes(currentGrid)) return 0;

    let minOps = Infinity;

    for (let i = 0; i < rows; i++) {
      for (let j = 0; j < cols; j++) {
        if (currentGrid[i][j] === 1) {
          const newGrid = performOperation(currentGrid, i, j);
          minOps = Math.min(minOps, 1 + solve(newGrid));
        }
      }
    }

    map.set(key, minOps);
    return minOps;
  }

  function hasNoOnes(grid) {
    return grid.every(row => row.every(cell => cell === 0));
  }

  function performOperation(grid, targetRow, targetCol) {
    const newGrid = grid.map(row => [...row]);
```

```
    for (let j = 0; j < cols; j++) {
      newGrid[targetRow][j] = 0;
    }

    for (let i = 0; i < rows; i++) {
      newGrid[i][targetCol] = 0;
    }

    return newGrid;
  }
};
```

2176 Count Equal And Divisible Pairs In An Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2176. Count Equal and Divisible Pairs in an Array
 * https://leetcode.com/problems/count-equal-and-divisible-pairs-in-an-array/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums of length n and an integer k, return the number of
 * pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by
 k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countPairs = function(nums, k) {
  const frequencyMap = new Map();
  let result = 0;

  nums.forEach((num, index) => {
    if (frequencyMap.has(num)) {
      const values = frequencyMap.get(num);
      values.forEach(prevIndex => {
        if ((prevIndex * index) % k === 0) {
          result++;
        }
      });
      values.push(index);
    } else {
      frequencyMap.set(num, [index]);
    }
  });

  return result;
};
```

2177 Find Three Consecutive Integers That Sum To A Given Number

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2177. Find Three Consecutive Integers That Sum to a Given Number
 * https://leetcode.com/problems/find-three-consecutive-integers-that-sum-to-a-given-number/
 * Difficulty: Medium
 *
 * Given an integer num, return three consecutive integers (as a sorted array) that sum to
num.
 * If num cannot be expressed as the sum of three consecutive integers, return an empty array.
 */
```

```
/**
 * @param {number} num
 * @return {number[]}
 */
var sumOfThree = function(num) {
  if (num % 3 !== 0) return [];

  const middle = num / 3;
  return [middle - 1, middle, middle + 1];
};
```

2178 Maximum Split Of Positive Even Integers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2178. Maximum Split of Positive Even Integers
 * https://leetcode.com/problems/maximum-split-of-positive-even-integers/
 * Difficulty: Medium
 *
 * You are given an integer finalSum. Split it into a sum of a maximum number of unique
 * positive even integers.
 * - For example, given finalSum = 12, the following splits are valid (unique positive even
 *   integers summing up to finalSum): (12), (2 + 10), (2 + 4 + 6), and (4 + 8). Among them,
 *   (2 + 4 + 6) contains the maximum number of integers. Note that finalSum cannot be split
 *   into (2 + 2 + 4 + 4) as all the numbers should be unique.
 *
 * Return a list of integers that represent a valid split containing a maximum number of
 * integers. If no valid split exists for finalSum, return an empty list. You may return
 * the integers in any order.
 */

/**
 * @param {number} finalSum
 * @return {number[]}
 */
var maximumEvenSplit = function(finalSum) {
  if (finalSum % 2 !== 0) return [];

  const result = [];
  let current = 2;
  let remaining = finalSum;

  while (remaining >= current) {
    if (remaining - current <= current) {
      result.push(remaining);
      return result;
    }
    result.push(current);
    remaining -= current;
    current += 2;
  }

  return result;
};
```

2179 Count Good Triplets In An Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2179. Count Good Triplets in an Array
 * https://leetcode.com/problems/count-good-triplets-in-an-array/
 * Difficulty: Hard
```

```
 *
 * You are given two 0-indexed arrays nums1 and nums2 of length n, both of which are
permutations
 * of [0, 1, ..., n - 1].
 *
 * A good triplet is a set of 3 distinct values which are present in increasing order by
position
 * both in nums1 and nums2. In other words, if we consider pos1v as the index of the value v
in
 * nums1 and pos2v as the index of the value v in nums2, then a good triplet will be a set (x,
 y, z)
 * where 0 <= x, y, z <= n - 1, such that pos1x < pos1y < pos1z and pos2x < pos2y < pos2z.
 *
 * Return the total number of good triplets.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var goodTriplets = function(nums1, nums2) {
  const length = nums1.length;
  const pos1 = new Array(length);
  const pos2 = new Array(length);

  for (let i = 0; i < length; i++) {
    pos1[nums1[i]] = i;
    pos2[nums2[i]] = i;
  }

  const indices = new Array(length);
  for (let i = 0; i < length; i++) {
    indices[pos1[i]] = pos2[i];
  }

  const leftTree = new Array(length + 1).fill(0);
  const rightTree = new Array(length + 1).fill(0);

  let result = 0;
  for (let i = length - 1; i >= 0; i--) {
    const position = indices[i];
    update(rightTree, position, 1);
  }

  for (let i = 0; i < length; i++) {
    const position = indices[i];
    update(rightTree, position, -1);
    const smaller = query(leftTree, position);
    const larger = query(rightTree, length - 1) - query(rightTree, position);
    result += smaller * larger;
    update(leftTree, position, 1);
  }

  return result;

  function update(tree, index, delta) {
    for (let i = index + 1; i <= length; i += i & -i) {
      tree[i] += delta;
    }
  }

  function query(tree, index) {
    let sum = 0;
    for (let i = index + 1; i > 0; i -= i & -i) {
      sum += tree[i];
    }
    return sum;
  }
};
```

2180 Count Integers With Even Digit Sum

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2180. Count Integers With Even Digit Sum
 * https://leetcode.com/problems/count-integers-with-even-digit-sum/
 * Difficulty: Easy
 *
 * Given a positive integer num, return the number of positive integers less than or equal to
num
 * whose digit sums are even.
 *
 * The digit sum of a positive integer is the sum of all its digits.
 */

/**
 * @param {number} num
 * @return {number}
 */
var countEven = function(num) {
  let result = 0;

  for (let i = 1; i <= num; i++) {
    let sum = 0;
    let current = i;
    while (current > 0) {
      sum += current % 10;
      current = Math.floor(current / 10);
    }
    if (sum % 2 === 0) {
      result++;
    }
  }

  return result;
};
```

2181 Merge Nodes In Between Zeros

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2181. Merge Nodes in Between Zeros
 * https://leetcode.com/problems/merge-nodes-in-between-zeros/
 * Difficulty: Medium
 *
 * You are given the head of a linked list, which contains a series of integers separated
 * by 0's. The beginning and end of the linked list will have Node.val == 0.
 *
 * For every two consecutive 0's, merge all the nodes lying in between them into a single
 * node whose value is the sum of all the merged nodes. The modified list should not contain
 * any 0's.
 *
 * Return the head of the modified linked list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
```

```
 * @param {ListNode} head
 * @return {ListNode}
 */
var mergeNodes = function(head) {
  const result = new ListNode(0);
  let current = result;
  let sum = 0;

  head = head.next;

  while (head) {
    if (head.val === 0) {
      current.next = new ListNode(sum);
      current = current.next;
      sum = 0;
    } else {
      sum += head.val;
    }
    head = head.next;
  }

  return result.next;
};
```

2182 Construct String With Repeat Limit

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2182. Construct String With Repeat Limit
 * https://leetcode.com/problems/construct-string-with-repeat-limit/
 * Difficulty: Medium
 *
 * You are given a string s and an integer repeatLimit. Construct a new string
 * repeatLimitedString using the characters of s such that no letter appears more than
 * repeatLimit times in a row. You do not have to use all characters from s.
 *
 * Return the lexicographically largest repeatLimitedString possible.
 *
 * A string a is lexicographically larger than a string b if in the first position where
 * a and b differ, string a has a letter that appears later in the alphabet than the
 * corresponding letter in b. If the first min(a.length, b.length) characters do not differ,
 * then the longer string is the lexicographically larger one.
 */

/**
 * @param {string} s
 * @param {number} repeatLimit
 * @return {string}
 */
var repeatLimitedString = function(s, repeatLimit) {
  const frequency = new Array(26).fill(0);
  for (const char of s) {
    frequency[char.charCodeAt(0) - 97]++;
  }

  const result = [];
  let currentChar = 25;

  while (currentChar >= 0) {
    if (frequency[currentChar] === 0) {
      currentChar--;
      continue;
    }

    const useCount = Math.min(frequency[currentChar], repeatLimit);
    for (let i = 0; i < useCount; i++) {
      result.push(String.fromCharCode(currentChar + 97));
    }
    frequency[currentChar] -= useCount;
```

```
    if (frequency[currentChar] > 0) {
      let nextChar = currentChar - 1;
      while (nextChar >= 0 && frequency[nextChar] === 0) {
        nextChar--;
      }

      if (nextChar < 0) break;

      result.push(String.fromCharCode(nextChar + 97));
      frequency[nextChar]--;
    } else {
      currentChar--;
    }
  }

  return result.join('');
};
```

2183 Count Array Pairs Divisible By K

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2183. Count Array Pairs Divisible by K
 * https://leetcode.com/problems/count-array-pairs-divisible-by-k/
 * Difficulty: Hard
 *
 * Given a 0-indexed integer array nums of length n and an integer k, return the number
 * of pairs (i, j) such that:
 * - 0 <= i < j <= n - 1 and
 * - nums[i] * nums[j] is divisible by k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countPairs = function(nums, k) {
  const map = new Map();
  let pairs = 0;

  for (const num of nums) {
    const gcd1 = gcd(num, k);
    for (const [gcd2, count] of map) {
      if ((gcd1 * gcd2) % k === 0) {
        pairs += count;
      }
    }
    map.set(gcd1, (map.get(gcd1) || 0) + 1);
  }

  return pairs;
};

function gcd(a, b) {
  while (b) {
    [a, b] = [b, a % b];
  }
  return a;
}
```

2184 Number Of Ways To Build Sturdy Brick Wall

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2184. Number of Ways to Build Sturdy Brick Wall
 * https://leetcode.com/problems/number-of-ways-to-build-sturdy-brick-wall/
 * Difficulty: Medium
 *
 * You are given integers height and width which specify the dimensions of a brick wall you
are
 * building. You are also given a 0-indexed array of unique integers bricks, where the ith
brick
 * has a height of 1 and a width of bricks[i]. You have an infinite supply of each type of
brick
 * and bricks may not be rotated.
 *
 * Each row in the wall must be exactly width units long. For the wall to be sturdy, adjacent
rows
 * in the wall should not join bricks at the same location, except at the ends of the wall.
 *
 * Return the number of ways to build a sturdy wall. Since the answer may be very large,
return
 * it modulo 109 + 7.
 */

/**
 * @param {number} height
 * @param {number} width
 * @param {number[]} bricks
 * @return {number}
 */
var buildWall = function(height, width, bricks) {
  const MOD = 1e9 + 7;
  const patterns = [];

  generatePatterns(0, []);

  if (patterns.length === 0) return 0;

  const graph = buildCompatibilityGraph();
  let dp = new Array(patterns.length).fill(1);
  for (let row = 1; row < height; row++) {
    const newDp = new Array(patterns.length).fill(0);

    for (let i = 0; i < patterns.length; i++) {
      for (const j of graph[i]) {
        newDp[i] = (newDp[i] + dp[j]) % MOD;
      }
    }

    dp = newDp;
  }

  return dp.reduce((sum, count) => (sum + count) % MOD, 0);

  function generatePatterns(currentWidth, currentPattern) {
    if (currentWidth === width) {
      patterns.push([...currentPattern]);
      return;
    }

    for (const brick of bricks) {
      if (currentWidth + brick <= width) {
        currentPattern.push(brick);
        generatePatterns(currentWidth + brick, currentPattern);
        currentPattern.pop();
      }
    }
  }
```

```
  function buildCompatibilityGraph() {
    const graph = Array.from({ length: patterns.length }, () => []);

    for (let i = 0; i < patterns.length; i++) {
      for (let j = 0; j < patterns.length; j++) {
        if (areCompatible(patterns[i], patterns[j])) {
          graph[i].push(j);
        }
      }
    }

    return graph;
  }

  function areCompatible(pattern1, pattern2) {
    const cuts1 = new Set();
    const cuts2 = new Set();

    let pos = 0;
    for (const brick of pattern1) {
      pos += brick;
      if (pos < width) cuts1.add(pos);
    }

    pos = 0;
    for (const brick of pattern2) {
      pos += brick;
      if (pos < width) cuts2.add(pos);
    }

    for (const cut of cuts1) {
      if (cuts2.has(cut)) return false;
    }

    return true;
  }
};
```

2185 Counting Words With A Given Prefix

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2185. Counting Words With a Given Prefix
 * https://leetcode.com/problems/counting-words-with-a-given-prefix/
 * Difficulty: Easy
 *
 * You are given an array of strings words and a string pref.
 *
 * Return the number of strings in words that contain pref as a prefix.
 *
 * A prefix of a string s is any leading contiguous substring of s.
 */

/**
 * @param {string[]} words
 * @param {string} pref
 * @return {number}
 */
var prefixCount = function(words, pref) {
  return words.filter(word => word.startsWith(pref)).length;
};
```

2186 Minimum Number Of Steps To Make Two Strings Anagram Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2186. Minimum Number of Steps to Make Two Strings Anagram II
 * https://leetcode.com/problems/minimum-number-of-steps-to-make-two-strings-anagram-ii/
 * Difficulty: Medium
 *
 * You are given two strings s and t. In one step, you can append any character to either s or
 t.
 *
 * Return the minimum number of steps to make s and t anagrams of each other.
 *
 * An anagram of a string is a string that contains the same characters with a different (or
the
 * same) ordering.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {number}
 */
var minSteps = function(s, t) {
  const charCount = new Array(26).fill(0);

  for (const char of s) {
    charCount[char.charCodeAt(0) - 97]++;
  }

  for (const char of t) {
    charCount[char.charCodeAt(0) - 97]--;
  }

  let result = 0;
  for (const count of charCount) {
    result += Math.abs(count);
  }

  return result;
};
```

2187 Minimum Time To Complete Trips

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2187. Minimum Time to Complete Trips
 * https://leetcode.com/problems/minimum-time-to-complete-trips/
 * Difficulty: Medium
 *
 * You are given an array time where time[i] denotes the time taken by the ith bus to complete
 * one trip.
 *
 * Each bus can make multiple trips successively; that is, the next trip can start immediately
 * after completing the current trip. Also, each bus operates independently; that is, the
trips
 * of one bus do not influence the trips of any other bus.
 *
 * You are also given an integer totalTrips, which denotes the number of trips all buses
should
 * make in total. Return the minimum time required for all buses to complete at least
totalTrips
 * trips.
```

```
 */

/**
 * @param {number[]} time
 * @param {number} totalTrips
 * @return {number}
 */
var minimumTime = function(time, totalTrips) {
  let left = 1;
  let right = Math.min(...time) * totalTrips;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    const trips = time.reduce((sum, t) => sum + Math.floor(mid / t), 0);

    if (trips >= totalTrips) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;
};
```

2188 Minimum Time To Finish The Race

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2188. Minimum Time to Finish the Race
 * https://leetcode.com/problems/minimum-time-to-finish-the-race/
 * Difficulty: Hard
 *
 * You are given a 0-indexed 2D integer array tires where tires[i] = [fi, ri] indicates that
 * the ith tire can finish its xth successive lap in fi * ri(x-1) seconds.
 * - For example, if fi = 3 and ri = 2, then the tire would finish its 1st lap in 3 seconds,
 *   its 2nd lap in 3 * 2 = 6 seconds, its 3rd lap in 3 * 22 = 12 seconds, etc.
 *
 * You are also given an integer changeTime and an integer numLaps.
 *
 * The race consists of numLaps laps and you may start the race with any tire. You have an
 * unlimited supply of each tire and after every lap, you may change to any given tire
(including
 * the current tire type) if you wait changeTime seconds.
 *
 * Return the minimum time to finish the race.
 */

/**
 * @param {number[][]} tires
 * @param {number} changeTime
 * @param {number} numLaps
 * @return {number}
 */
var minimumFinishTime = function(tires, changeTime, numLaps) {
  const minTimes = new Array(18).fill(Infinity);
  const bestTime = new Array(numLaps + 1).fill(Infinity);

  for (const [baseTime, multiplier] of tires) {
    let currentTime = baseTime;
    let totalTime = baseTime;

    for (let lap = 1; lap <= Math.min(numLaps, 17); lap++) {
      if (currentTime > changeTime + baseTime) break;
      minTimes[lap] = Math.min(minTimes[lap], totalTime);
      currentTime *= multiplier;
      totalTime += currentTime;
    }
  }
```

```
  bestTime[0] = 0;

  for (let lap = 1; lap <= numLaps; lap++) {
    for (let prev = 1; prev <= Math.min(lap, 17); prev++) {
      if (minTimes[prev] !== Infinity) {
        bestTime[lap] = Math.min(
          bestTime[lap],
          bestTime[lap - prev] + minTimes[prev] + (lap === prev ? 0 : changeTime)
        );
      }
    }
  }

  return bestTime[numLaps];
};
```

2189 Number Of Ways To Build House Of Cards

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2189. Number of Ways to Build House of Cards
 * https://leetcode.com/problems/number-of-ways-to-build-house-of-cards/
 * Difficulty: Medium
 *
 * You are given an integer n representing the number of playing cards you have. A house of
 * cards meets the following conditions:
 * - A house of cards consists of one or more rows of triangles and horizontal cards.
 * - Triangles are created by leaning two cards against each other.
 * - One card must be placed horizontally between all adjacent triangles in a row.
 * - Any triangle on a row higher than the first must be placed on a horizontal card from the
 *   previous row.
 * - Each triangle is placed in the leftmost available spot in the row.
 *
 * Return the number of distinct house of cards you can build using all n cards. Two houses of
 * cards are considered distinct if there exists a row where the two houses contain a
different
 * number of cards.
 */

/**
 * @param {number} n
 * @return {number}
 */
var houseOfCards = function(n) {
  const dp = new Array(n + 1).fill(0);
  dp[0] = 1;

  for (let i = 2; i <= n; i += 3) {
    for (let j = n; j >= i; j--) {
      dp[j] += dp[j - i];
    }
  }

  return dp[n];
};
```

2190 Most Frequent Number Following Key In An Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2190. Most Frequent Number Following Key In an Array
 * https://leetcode.com/problems/most-frequent-number-following-key-in-an-array/
```

```
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums. You are also given an integer key, which is
 * present in nums.
 *
 * For every unique integer target in nums, count the number of times target immediately
follows
 * an occurrence of key in nums. In other words, count the number of indices i such that:
 * - 0 <= i <= nums.length - 2,
 * - nums[i] == key and,
 * - nums[i + 1] == target.
 *
 * Return the target with the maximum count. The test cases will be generated such that the
target
 * with maximum count is unique.
 */

/**
 * @param {number[]} nums
 * @param {number} key
 * @return {number}
 */
var mostFrequent = function(nums, key) {
  const frequency = new Map();
  let maxCount = 0;
  let result = 0;

  for (let i = 0; i < nums.length - 1; i++) {
    if (nums[i] === key) {
      const target = nums[i + 1];
      const count = (frequency.get(target) || 0) + 1;
      frequency.set(target, count);

      if (count > maxCount) {
        maxCount = count;
        result = target;
      }
    }
  }

  return result;
};
```

2191 Sort The Jumbled Numbers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2191. Sort the Jumbled Numbers
 * https://leetcode.com/problems/sort-the-jumbled-numbers/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array mapping which represents the mapping rule of a
shuffled
 * decimal system. mapping[i] = j means digit i should be mapped to digit j in this system.
 *
 * The mapped value of an integer is the new integer obtained by replacing each occurrence of
digit
 * i in the integer with mapping[i] for all 0 <= i <= 9.
 *
 * You are also given another integer array nums. Return the array nums sorted in
non-decreasing
 * order based on the mapped values of its elements.
 *
 * Notes:
 * - Elements with the same mapped values should appear in the same relative order as in the
input.
 * - The elements of nums should only be sorted based on their mapped values and not be
replaced by
 *   them.
```

```
 */

/**
 * @param {number[]} mapping
 * @param {number[]} nums
 * @return {number[]}
 */
var sortJumbled = function(mapping, nums) {
  const mapped = nums.map((num, index) => {
    let mappedNum = 0;
    let temp = num;

    if (temp === 0) {
      mappedNum = mapping[0];
    } else {
      const digits = [];
      while (temp > 0) {
        digits.push(mapping[temp % 10]);
        temp = Math.floor(temp / 10);
      }
      while (digits.length > 0) {
        mappedNum = mappedNum * 10 + digits.pop();
      }
    }

    return { original: num, mapped: mappedNum, index };
  });

  mapped.sort((a, b) => {
    if (a.mapped === b.mapped) return a.index - b.index;
    return a.mapped - b.mapped;
  });

  return mapped.map(item => item.original);
};
```

2192 All Ancestors Of A Node In A Directed Acyclic Graph

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2192. All Ancestors of a Node in a Directed Acyclic Graph
 * https://leetcode.com/problems/all-ancestors-of-a-node-in-a-directed-acyclic-graph/
 * Difficulty: Medium
 *
 * You are given a positive integer n representing the number of nodes of a Directed Acyclic
 * Graph (DAG). The nodes are numbered from 0 to n - 1 (inclusive).
 *
 * You are also given a 2D integer array edges, where edges[i] = [fromi, toi] denotes that
there
 * is a unidirectional edge from fromi to toi in the graph.
 *
 * Return a list answer, where answer[i] is the list of ancestors of the ith node, sorted in
 * ascending order.
 *
 * A node u is an ancestor of another node v if u can reach v via a set of edges.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[][]}
 */
var getAncestors = function(n, edges) {
  const graph = Array.from({ length: n }, () => []);
  const ancestors = Array.from({ length: n }, () => new Set());

  for (const [from, to] of edges) {
    graph[to].push(from);
```

```
  }

  for (let i = 0; i < n; i++) {
    findAncestors(i);
  }

  return ancestors.map(set => [...set].sort((a, b) => a - b));

  function findAncestors(node) {
    if (ancestors[node].size > 0) return;

    for (const parent of graph[node]) {
      ancestors[node].add(parent);
      findAncestors(parent);
      for (const ancestor of ancestors[parent]) {
        ancestors[node].add(ancestor);
      }
    }
  }
};
```

2193 Minimum Number Of Moves To Make Palindrome

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2193. Minimum Number of Moves to Make Palindrome
 * https://leetcode.com/problems/minimum-number-of-moves-to-make-palindrome/
 * Difficulty: Hard
 *
 * You are given a string s consisting only of lowercase English letters.
 *
 * In one move, you can select any two adjacent characters of s and swap them.
 *
 * Return the minimum number of moves needed to make s a palindrome.
 *
 * Note that the input will be generated such that s can always be converted to a palindrome.
 */

/**
* @param {string} s
* @return {number}
*/
var minMovesToMakePalindrome = function(s) {
  const chars = s.split('');
  let moves = 0;

  while (chars.length > 1) {
    const matchIndex = chars.lastIndexOf(chars[0]);

    if (matchIndex === 0) {
      const middlePos = Math.floor(chars.length / 2);
      moves += middlePos;
      chars.splice(0, 1);
    } else {
      for (let i = matchIndex; i < chars.length - 1; i++) {
        [chars[i], chars[i + 1]] = [chars[i + 1], chars[i]];
        moves++;
      }

      chars.pop();
      chars.shift();
    }
  }

  return moves;
};
```

2194 Cells In A Range On An Excel Sheet

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2194. Cells in a Range on an Excel Sheet
 * https://leetcode.com/problems/cells-in-a-range-on-an-excel-sheet/
 * Difficulty: Easy
 *
 * A cell (r, c) of an excel sheet is represented as a string "<col><row>" where:
 * - <col> denotes the column number c of the cell. It is represented by alphabetical letters.
 *   - For example, the 1st column is denoted by 'A', the 2nd by 'B', the 3rd by 'C', and so
on.
 * - <row> is the row number r of the cell. The rth row is represented by the integer r.
 *   - You are given a string s in the format "<col1><row1>:<col2><row2>", where <col1>
represents
 *     the column c1, <row1> represents the row r1, <col2> represents the column c2, and
<row2>
 *     represents the row r2, such that r1 <= r2 and c1 <= c2.
 *
 * Return the list of cells (x, y) such that r1 <= x <= r2 and c1 <= y <= c2. The cells should
 * be represented as strings in the format mentioned above and be sorted in non-decreasing
order
 * first by columns and then by rows.
 */

/**
 * @param {string} s
 * @return {string[]}
 */
var cellsInRange = function(s) {
  const result = [];
  const startCol = s.charCodeAt(0);
  const endCol = s.charCodeAt(3);
  const startRow = parseInt(s[1], 10);
  const endRow = parseInt(s[4], 10);

  for (let col = startCol; col <= endCol; col++) {
    for (let row = startRow; row <= endRow; row++) {
      result.push(String.fromCharCode(col) + row);
    }
  }

  return result;
};
```

2195 Append K Integers With Minimal Sum

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2195. Append K Integers With Minimal Sum
 * https://leetcode.com/problems/append-k-integers-with-minimal-sum/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. Append k unique positive integers
that do
 * not appear in nums to nums such that the resulting total sum is minimum.
 *
 * Return the sum of the k integers appended to nums.
 */

/**
```

```
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minimalKSum = function(nums, k) {
  const sortedUnique = [...new Set(nums)].sort((a, b) => a - b);
  let sum = BigInt(0);
  let current = 1;
  let i = 0;

  while (k > 0 && i < sortedUnique.length) {
    if (current < sortedUnique[i]) {
      const count = Math.min(k, sortedUnique[i] - current);
      sum += (BigInt(current) + BigInt(current + count - 1)) * BigInt(count) / BigInt(2);
      k -= count;
      current += count;
    } else {
      current = sortedUnique[i] + 1;
      i++;
    }
  }

  if (k > 0) {
    sum += (BigInt(current) + BigInt(current + k - 1)) * BigInt(k) / BigInt(2);
  }

  return Number(sum);
};
```

2196 Create Binary Tree From Descriptions

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2196. Create Binary Tree From Descriptions
 * https://leetcode.com/problems/create-binary-tree-from-descriptions/
 * Difficulty: Medium
 *
 * You are given a 2D integer array descriptions where descriptions[i] = [parenti, childi,
isLefti]
 * indicates that parenti is the parent of childi in a binary tree of unique values.
Furthermore,
 * - If isLefti == 1, then childi is the left child of parenti.
 * - If isLefti == 0, then childi is the right child of parenti.
 *
 * Construct the binary tree described by descriptions and return its root.
 *
 * The test cases will be generated such that the binary tree is valid.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[][]} descriptions
 * @return {TreeNode}
 */
var createBinaryTree = function(descriptions) {
  const nodes = new Map();
  const children = new Set();

  for (const [parent, child, isLeft] of descriptions) {
    if (!nodes.has(parent)) {
      nodes.set(parent, new TreeNode(parent));
    }
```

```
    if (!nodes.has(child)) {
      nodes.set(child, new TreeNode(child));
    }
    children.add(child);
    if (isLeft) {
      nodes.get(parent).left = nodes.get(child);
    } else {
      nodes.get(parent).right = nodes.get(child);
    }
  }

  for (const [val, node] of nodes) {
    if (!children.has(val)) {
      return node;
    }
  }

  return null;
};
```

2197 Replace Non Coprime Numbers In Array

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2197. Replace Non-Coprime Numbers in Array
 * https://leetcode.com/problems/replace-non-coprime-numbers-in-array/
 * Difficulty: Hard
 *
 * You are given an array of integers nums. Perform the following steps:
 * 1. Find any two adjacent numbers in nums that are non-coprime.
 * 2. If no such numbers are found, stop the process.
 * 3. Otherwise, delete the two numbers and replace them with their LCM (Least Common
Multiple).
 * 4. Repeat this process as long as you keep finding two adjacent non-coprime numbers.
 *
 * Return the final modified array. It can be shown that replacing adjacent non-coprime
numbers in
 * any arbitrary order will lead to the same result.
 *
 * The test cases are generated such that the values in the final array are less than or equal
 to
 * 108.
 *
 * Two values x and y are non-coprime if GCD(x, y) > 1 where GCD(x, y) is the Greatest Common
 * Divisor of x and y.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var replaceNonCoprimes = function(nums) {
  const result = [];

  for (let num of nums) {
    while (result.length > 0) {
      const last = result[result.length - 1];
      const gcdValue = gcd(last, num);
      if (gcdValue === 1) break;
      result.pop();
      num = (last / gcdValue) * num;
    }
    result.push(num);
  }

  return result;
};

function gcd(a, b) {
```

```
  while (b) {
    [a, b] = [b, a % b];
  }
  return a;
}
```

2198 Number Of Single Divisor Triplets

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2198. Number of Single Divisor Triplets
 * https://leetcode.com/problems/number-of-single-divisor-triplets/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of positive integers nums. A triplet of three distinct
 * indices (i, j, k) is called a single divisor triplet of nums if nums[i] + nums[j] + nums[k]
 * is divisible by exactly one of nums[i], nums[j], or nums[k].
 *
 * Return the number of single divisor triplets of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var singleDivisorTriplet = function(nums) {
  let result = 0;
  const count = new Array(101).fill(0);

  for (const num of nums) {
    count[num]++;
  }

  for (let i = 1; i <= 100; i++) {
    if (!count[i]) continue;
    for (let j = i; j <= 100; j++) {
      if (!count[j]) continue;
      for (let k = j + (i === j ? 1 : 0); k <= 100; k++) {
        if (!count[k]) continue;

        const sum = i + j + k;
        const divisors = (sum % i === 0 ? 1 : 0)
          + (sum % j === 0 ? 1 : 0) + (sum % k === 0 ? 1 : 0);

        if (divisors === 1) {
          if (i === j) {
            result += count[i] * (count[i] - 1) / 2 * count[k];
          } else if (j === k) {
            result += count[i] * count[j] * (count[j] - 1) / 2;
          } else {
            result += count[i] * count[j] * count[k];
          }
        }
      }
    }
  }

  return result * 6;
};
```

2200 Find All K Distant Indices In An Array

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2200. Find All K-Distant Indices in an Array
 * https://leetcode.com/problems/find-all-k-distant-indices-in-an-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums and two integers key and k. A k-distant index
 * is an index i of nums for which there exists at least one index j such that |i - j| <= k
 * and nums[j] == key.
 *
 * Return a list of all k-distant indices sorted in increasing order.
 */

/**
 * @param {number[]} nums
 * @param {number} key
 * @param {number} k
 * @return {number[]}
 */
var findKDistantIndices = function(nums, key, k) {
  const result = new Set();

  for (let j = 0; j < nums.length; j++) {
    if (nums[j] === key) {
      for (let i = Math.max(0, j - k); i <= Math.min(nums.length - 1, j + k); i++) {
        result.add(i);
      }
    }
  }

  return [...result].sort((a, b) => a - b);
};
```

2201 Count Artifacts That Can Be Extracted

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2201. Count Artifacts That Can Be Extracted
 * https://leetcode.com/problems/count-artifacts-that-can-be-extracted/
 * Difficulty: Medium
 *
 * There is an n x n 0-indexed grid with some artifacts buried in it. You are given the
integer
 * n and a 0-indexed 2D integer array artifacts describing the positions of the rectangular
 * artifacts where artifacts[i] = [r1i, c1i, r2i, c2i] denotes that the ith artifact is buried
 * in the subgrid where:
 * - (r1i, c1i) is the coordinate of the top-left cell of the ith artifact and
 * - (r2i, c2i) is the coordinate of the bottom-right cell of the ith artifact.
 *
 * You will excavate some cells of the grid and remove all the mud from them. If the cell has
a
 * part of an artifact buried underneath, it will be uncovered. If all the parts of an
artifact
 * are uncovered, you can extract it.
 *
 * Given a 0-indexed 2D integer array dig where dig[i] = [ri, ci] indicates that you will
excavate
 * the cell (ri, ci), return the number of artifacts that you can extract.
 *
 * The test cases are generated such that:
 * - No two artifacts overlap.
 * - Each artifact only covers at most 4 cells.
 * - The entries of dig are unique.
 */

/**
 * @param {number} n
 * @param {number[][]} artifacts
 * @param {number[][]} dig
```

```
 * @return {number}
 */
var digArtifacts = function(n, artifacts, dig) {
  const excavated = new Set();
  let result = 0;

  for (const [row, col] of dig) {
    excavated.add(`${row},${col}`);
  }

  for (const [r1, c1, r2, c2] of artifacts) {
    let allUncovered = true;

    for (let r = r1; r <= r2; r++) {
      for (let c = c1; c <= c2; c++) {
        if (!excavated.has(`${r},${c}`)) {
          allUncovered = false;
          break;
        }
      }
      if (!allUncovered) break;
    }

    if (allUncovered) result++;
  }

  return result;
};
```

2202 Maximize The Topmost Element After K Moves

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2202. Maximize the Topmost Element After K Moves
 * https://leetcode.com/problems/maximize-the-topmost-element-after-k-moves/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums representing the contents of a pile, where
nums[0]
 * is the topmost element of the pile.
 *
 * In one move, you can perform either of the following:
 * - If the pile is not empty, remove the topmost element of the pile.
 * - If there are one or more removed elements, add any one of them back onto the pile. This
element
 *   becomes the new topmost element.
 *
 * You are also given an integer k, which denotes the total number of moves to be made.
 *
 * Return the maximum value of the topmost element of the pile possible after exactly k moves.
 In
 * case it is not possible to obtain a non-empty pile after k moves, return -1.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumTop = function(nums, k) {
  const n = nums.length;

  if (n === 1 && k % 2 === 1) return -1;
  if (k === 0) return nums[0];
  if (k === 1) return n > 1 ? nums[1] : -1;

  let result = 0;
  for (let i = 0; i < Math.min(k - 1, n); i++) {
    result = Math.max(result, nums[i]);
```

```
  }

  if (k < n) {
    result = Math.max(result, nums[k]);
  }

  return result;
};
```

2203 Minimum Weighted Subgraph With The Required Paths

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2203. Minimum Weighted Subgraph With the Required Paths
 * https://leetcode.com/problems/minimum-weighted-subgraph-with-the-required-paths/
 * Difficulty: Hard
 *
 * You are given an integer n denoting the number of nodes of a weighted directed graph. The
nodes
 * are numbered from 0 to n - 1.
 *
 * You are also given a 2D integer array edges where edges[i] = [fromi, toi, weighti] denotes
that
 * there exists a directed edge from fromi to toi with weight weighti.
 *
 * Lastly, you are given three distinct integers src1, src2, and dest denoting three distinct
nodes
 * of the graph.
 *
 * Return the minimum weight of a subgraph of the graph such that it is possible to reach dest
 from
 * both src1 and src2 via a set of edges of this subgraph. In case such a subgraph does not
exist,
 * return -1.
 *
 * A subgraph is a graph whose vertices and edges are subsets of the original graph. The
weight of
 * a subgraph is the sum of weights of its constituent edges.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number} src1
 * @param {number} src2
 * @param {number} dest
 * @return {number}
 */
var minimumWeight = function(n, edges, src1, src2, dest) {
  const forwardGraph = Array.from({ length: n }, () => []);
  const reverseGraph = Array.from({ length: n }, () => []);

  for (const [from, to, weight] of edges) {
    forwardGraph[from].push([to, weight]);
    reverseGraph[to].push([from, weight]);
  }

  const distFromSrc1 = dijkstra(forwardGraph, src1);
  const distFromSrc2 = dijkstra(forwardGraph, src2);
  const distToDest = dijkstra(reverseGraph, dest);
  let minWeight = Infinity;
  for (let i = 0; i < n; i++) {
    if (distFromSrc1[i] !== Infinity && distFromSrc2[i] !== Infinity
        && distToDest[i] !== Infinity) {
      minWeight = Math.min(minWeight, distFromSrc1[i] + distFromSrc2[i] + distToDest[i]);
    }
  }

  return minWeight === Infinity ? -1 : minWeight;
```

```
  function dijkstra(graph, start) {
    const distances = new Array(n).fill(Infinity);
    distances[start] = 0;
    const pq = new PriorityQueue((a, b) => a[0] - b[0]);
    pq.enqueue([0, start]);

    while (!pq.isEmpty()) {
      const [dist, node] = pq.dequeue();
      if (dist > distances[node]) continue;

      for (const [next, weight] of graph[node]) {
        if (distances[next] > dist + weight) {
          distances[next] = dist + weight;
          pq.enqueue([distances[next], next]);
        }
      }
    }

    return distances;
  }
};
```

2204 Distance To A Cycle In Undirected Graph

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2204. Distance to a Cycle in Undirected Graph
 * https://leetcode.com/problems/distance-to-a-cycle-in-undirected-graph/
 * Difficulty: Hard
 *
 * You are given a positive integer n representing the number of nodes in a connected
undirected
 * graph containing exactly one cycle. The nodes are numbered from 0 to n - 1 (inclusive).
 *
 * You are also given a 2D integer array edges, where edges[i] = [node1i, node2i] denotes that
 * there is a bidirectional edge connecting node1i and node2i in the graph.
 *
 * The distance between two nodes a and b is defined to be the minimum number of edges that
are
 * needed to go from a to b.
 *
 * Return an integer array answer of size n, where answer[i] is the minimum distance between
the
 * ith node and any node in the cycle.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[]}
 */
var distanceToCycle = function(n, edges) {
  const graph = Array.from({ length: n }, () => []);
  const result = new Array(n).fill(0);
  const degree = new Array(n).fill(0);
  const queue = [];

  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  for (let i = 0; i < n; i++) {
    degree[i] = graph[i].length;
    if (degree[i] === 1) {
      queue.push(i);
    }
  }
```

```
  while (queue.length > 0) {
    const node = queue.shift();
    result[node] = Infinity;

    for (const neighbor of graph[node]) {
      if (degree[neighbor] > 1 && --degree[neighbor] === 1) {
        queue.push(neighbor);
      }
    }
  }

  for (let i = 0; i < n; i++) {
    if (degree[i] > 1) {
      queue.push(i);
    }
  }

  while (queue.length > 0) {
    const node = queue.shift();

    for (const neighbor of graph[node]) {
      if (result[neighbor] > result[node] + 1) {
        result[neighbor] = result[node] + 1;
        queue.push(neighbor);
      }
    }
  }

  return result;
};
```

2206 Divide Array Into Equal Pairs

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2206. Divide Array Into Equal Pairs
 * https://leetcode.com/problems/divide-array-into-equal-pairs/
 * Difficulty: Easy
 *
 * You are given an integer array nums consisting of 2 * n integers.
 *
 * You need to divide nums into n pairs such that:
 * - Each element belongs to exactly one pair.
 * - The elements present in a pair are equal.
 *
 * Return true if nums can be divided into n pairs, otherwise return false.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
/**
 * @param {number[]} nums
 * @return {boolean}
 */
var divideArray = function(nums) {
  const map = new Map();

  for (const num of nums) {
    map.set(num, (map.get(num) || 0) + 1);
  }
  for (const count of map.values()) {
    if (count % 2 !== 0) {
      return false;
    }
  }
```

```
  return true;
};
```

2207 Maximize Number Of Subsequences In A String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2207. Maximize Number of Subsequences in a String
 * https://leetcode.com/problems/maximize-number-of-subsequences-in-a-string/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string text and another 0-indexed string pattern of length 2,
both
 * of which consist of only lowercase English letters.
 *
 * You can add either pattern[0] or pattern[1] anywhere in text exactly once. Note that the
 * character can be added even at the beginning or at the end of text.
 *
 * Return the maximum number of times pattern can occur as a subsequence of the modified text.
 *
 * A subsequence is a string that can be derived from another string by deleting some or no
 * characters without changing the order of the remaining characters.
 */

/**
 * @param {string} text
 * @param {string} pattern
 * @return {number}
 */
var maximumSubsequenceCount = function(text, pattern) {
  const firstChar = pattern[0];
  const secondChar = pattern[1];
  let firstCount = 0;
  let secondCount = 0;
  let subsequences = 0;

  for (const char of text) {
    if (char === secondChar) {
      subsequences += firstCount;
      secondCount++;
    }
    if (char === firstChar) {
      firstCount++;
    }
  }

  return subsequences + Math.max(firstCount, secondCount);
};
```

2208 Minimum Operations To Halve Array Sum

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2208. Minimum Operations to Halve Array Sum
 * https://leetcode.com/problems/minimum-operations-to-halve-array-sum/
 * Difficulty: Medium
 *
 * You are given an array nums of positive integers. In one operation, you can choose any
 * number from nums and reduce it to exactly half the number. (Note that you may choose
 * this reduced number in future operations.)
 *
 * Return the minimum number of operations to reduce the sum of nums by at least half.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var halveArray = function(nums) {
  const pq = new PriorityQueue((a, b) => b - a);
  let totalSum = 0;

  for (const num of nums) {
    pq.enqueue(num);
    totalSum += num;
  }

  const target = totalSum / 2;
  let reducedSum = 0;
  let result = 0;

  while (reducedSum < target) {
    const largest = pq.dequeue();
    const halved = largest / 2;
    reducedSum += halved;
    pq.enqueue(halved);
    result++;
  }

  return result;
};
```

2209 Minimum White Tiles After Covering With Carpets

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2209. Minimum White Tiles After Covering With Carpets
 * https://leetcode.com/problems/minimum-white-tiles-after-covering-with-carpets/
 * Difficulty: Hard
 *
 * You are given a 0-indexed binary string floor, which represents the colors of tiles on a
floor:
 * - floor[i] = '0' denotes that the ith tile of the floor is colored black.
 * - On the other hand, floor[i] = '1' denotes that the ith tile of the floor is colored
white.
 *
 * You are also given numCarpets and carpetLen. You have numCarpets black carpets, each of
length
 * carpetLen tiles. Cover the tiles with the given carpets such that the number of white tiles
 * still visible is minimum. Carpets may overlap one another.
 *
 * Return the minimum number of white tiles still visible.
 */

/**
 * @param {string} floor
 * @param {number} numCarpets
 * @param {number} carpetLen
 * @return {number}
 */
var minimumWhiteTiles = function(floor, numCarpets, carpetLen) {
  const n = floor.length;
  const dp = Array.from({ length: n + 1 }, () => Array(numCarpets + 1).fill(0));

  for (let i = 1; i <= n; i++) {
    for (let j = 0; j <= numCarpets; j++) {
      const skip = dp[i - 1][j] + (floor[i - 1] === '1' ? 1 : 0);
      const cover = j > 0 ? dp[Math.max(0, i - carpetLen)][j - 1] : Infinity;
      dp[i][j] = Math.min(skip, cover);
    }
  }
```

```
  return dp[n][numCarpets];
};
```

2210 Count Hills And Valleys In An Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2210. Count Hills and Valleys in an Array
 * https://leetcode.com/problems/count-hills-and-valleys-in-an-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums. An index i is part of a hill in nums if the
 * closest non-equal neighbors of i are smaller than nums[i]. Similarly, an index i is part
 * of a valley in nums if the closest non-equal neighbors of i are larger than nums[i].
 * Adjacent indices i and j are part of the same hill or valley if nums[i] == nums[j].
 *
 * Note that for an index to be part of a hill or valley, it must have a non-equal neighbor
 * on both the left and right of the index.
 *
 * Return the number of hills and valleys in nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countHillValley = function(nums) {
  let count = 0;
  let prev = nums[0];

  for (let i = 1; i < nums.length - 1; i++) {
    if (nums[i] === nums[i + 1]) continue;
    const left = prev;
    const right = nums[i + 1];

    if ((nums[i] > left && nums[i] > right) || (nums[i] < left && nums[i] < right)) {
      count++;
    }

    prev = nums[i];
  }

  return count;
};
```

2211 Count Collisions On A Road

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2211. Count Collisions on a Road
 * https://leetcode.com/problems/count-collisions-on-a-road/
 * Difficulty: Medium
 *
 * There are n cars on an infinitely long road. The cars are numbered from 0 to n - 1 from
left
 * to right and each car is present at a unique point.
 *
 * You are given a 0-indexed string directions of length n. directions[i] can be either 'L',
'R',
 * or 'S' denoting whether the ith car is moving towards the left, towards the right, or
staying
 * at its current point respectively. Each moving car has the same speed.
 *
```

```
 * The number of collisions can be calculated as follows:
 * - When two cars moving in opposite directions collide with each other, the number of
collisions
 *   increases by 2.
 * - When a moving car collides with a stationary car, the number of collisions increases by
1.
 *
 * After a collision, the cars involved can no longer move and will stay at the point where
they
 * collided. Other than that, cars cannot change their state or direction of motion.
 *
 * Return the total number of collisions that will happen on the road.
 */

/**
 * @param {string} directions
 * @return {number}
 */
var countCollisions = function(directions) {
  let result = 0;
  let left = 0;
  let right = directions.length - 1;

  while (left < directions.length && directions[left] === 'L') left++;
  while (right >= 0 && directions[right] === 'R') right--;

  for (let i = left; i <= right; i++) {
    if (directions[i] !== 'S') result++;
  }

  return result;
};
```

2212 Maximum Points In An Archery Competition

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2212. Maximum Points in an Archery Competition
 * https://leetcode.com/problems/maximum-points-in-an-archery-competition/
 * Difficulty: Medium
 *
 * Alice and Bob are opponents in an archery competition. The competition has set the
 * following rules:
 * 1. Alice first shoots numArrows arrows and then Bob shoots numArrows arrows.
 * 2. The points are then calculated as follows:
 *    1. The target has integer scoring sections ranging from 0 to 11 inclusive.
 *    2. For each section of the target with score k (in between 0 to 11), say Alice and Bob
have
 *       shot ak and bk arrows on that section respectively. If ak >= bk, then Alice takes k
points.
 *       If ak < bk, then Bob takes k points.
 *    3. However, if ak == bk == 0, then nobody takes k points.
 * - For example, if Alice and Bob both shot 2 arrows on the section with score 11, then Alice
 takes
 *   11 points. On the other hand, if Alice shot 0 arrows on the section with score 11 and Bob
 shot
 *   2 arrows on that same section, then Bob takes 11 points.
 *
 * You are given the integer numArrows and an integer array aliceArrows of size 12, which
represents
 * the number of arrows Alice shot on each scoring section from 0 to 11. Now, Bob wants to
maximize
 * the total number of points he can obtain.
 *
 * Return the array bobArrows which represents the number of arrows Bob shot on each scoring
section
 * from 0 to 11. The sum of the values in bobArrows should equal numArrows.
 *
 * If there are multiple ways for Bob to earn the maximum total points, return any one of
```

```
them.
 */

/**
 * @param {number} numArrows
 * @param {number[]} aliceArrows
 * @return {number[]}
 */
var maximumBobPoints = function(numArrows, aliceArrows) {
  let maxScore = 0;
  let bestConfig = new Array(12).fill(0);

  backtrack(1, numArrows, 0, new Array(12).fill(0));
  return bestConfig;

  function backtrack(index, arrowsLeft, score, config) {
    if (index === 12 || arrowsLeft === 0) {
      if (score > maxScore) {
        maxScore = score;
        bestConfig = [...config];
        bestConfig[0] += arrowsLeft;
      }
      return;
    }

    const needed = aliceArrows[index] + 1;
    if (arrowsLeft >= needed) {
      config[index] = needed;
      backtrack(index + 1, arrowsLeft - needed, score + index, config);
      config[index] = 0;
    }

    backtrack(index + 1, arrowsLeft, score, config);
  }
};
```

2213 Longest Substring Of One Repeating Character

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2213. Longest Substring of One Repeating Character
 * https://leetcode.com/problems/longest-substring-of-one-repeating-character/
 * Difficulty: Hard
 *
 * You are given a 0-indexed string s. You are also given a 0-indexed string queryCharacters
of
 * length k and a 0-indexed array of integer indices queryIndices of length k, both of which
are
 * used to describe k queries.
 *
 * The ith query updates the character in s at index queryIndices[i] to the character
 * queryCharacters[i].
 *
 * Return an array lengths of length k where lengths[i] is the length of the longest substring
 of
 * s consisting of only one repeating character after the ith query is performed.
 */

/**
* @param {string} s
* @param {string} queryCharacters
* @param {number[]} queryIndices
* @return {number[]}
*/
var longestRepeating = function(s, queryCharacters, queryIndices) {
  const chars = s.split('');
  const n = chars.length;
  const k = queryIndices.length;
  const result = [];
```

```
class Node {
  constructor() {
    this.left = 0;
    this.right = 0;
    this.max = 0;
    this.total = 0;
  }
}

const tree = new Array(4 * n);
for (let i = 0; i < 4 * n; i++) {
  tree[i] = new Node();
}

function buildTree(node, start, end) {
  if (start === end) {
    tree[node].left = 1;
    tree[node].right = 1;
    tree[node].max = 1;
    tree[node].total = 1;
    return;
  }

  const mid = Math.floor((start + end) / 2);
  buildTree(2 * node, start, mid);
  buildTree(2 * node + 1, mid + 1, end);

  updateNode(node, start, end);
}

function updateNode(node, start, end) {
  const leftChild = 2 * node;
  const rightChild = 2 * node + 1;
  const mid = Math.floor((start + end) / 2);

  tree[node].total = tree[leftChild].total + tree[rightChild].total;

  if (tree[leftChild].total === tree[leftChild].left
      && mid + 1 <= end && chars[mid] === chars[mid + 1]) {
    tree[node].left = tree[leftChild].total + tree[rightChild].left;
  } else {
    tree[node].left = tree[leftChild].left;
  }

  if (tree[rightChild].total === tree[rightChild].right
      && mid >= start && chars[mid] === chars[mid + 1]) {
    tree[node].right = tree[rightChild].total + tree[leftChild].right;
  } else {
    tree[node].right = tree[rightChild].right;
  }

  tree[node].max = Math.max(tree[leftChild].max, tree[rightChild].max);

  if (mid >= start && mid + 1 <= end && chars[mid] === chars[mid + 1]) {
    tree[node].max = Math.max(tree[node].max,
      tree[leftChild].right + tree[rightChild].left);
  }
}

function update(node, start, end, index) {
  if (index < start || index > end) {
    return;
  }

  if (start === end) {
    return;
  }

  const mid = Math.floor((start + end) / 2);
  update(2 * node, start, mid, index);
  update(2 * node + 1, mid + 1, end, index);

  updateNode(node, start, end);
}
```

```
  buildTree(1, 0, n - 1);

  for (let q = 0; q < k; q++) {
    const index = queryIndices[q];
    const newChar = queryCharacters[q];

    chars[index] = newChar;
    update(1, 0, n - 1, index);

    result.push(tree[1].max);
  }

  return result;
};
```

2214 Minimum Health To Beat Game

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2214. Minimum Health to Beat Game
 * https://leetcode.com/problems/minimum-health-to-beat-game/
 * Difficulty: Medium
 *
 * You are playing a game that has n levels numbered from 0 to n - 1. You are given a
0-indexed
 * integer array damage where damage[i] is the amount of health you will lose to complete the
 * ith level.
 *
 * You are also given an integer armor. You may use your armor ability at most once during the
 * game on any level which will protect you from at most armor damage.
 *
 * You must complete the levels in order and your health must be greater than 0 at all times
to
 * beat the game.
 *
 * Return the minimum health you need to start with to beat the game.
 */

/**
 * @param {number[]} damage
 * @param {number} armor
 * @return {number}
 */
var minimumHealth = function(damage, armor) {
  const totalDamage = damage.reduce((sum, dmg) => sum + dmg, 0);
  const maxDamage = Math.max(...damage);
  const armorReduction = Math.min(armor, maxDamage);

  return totalDamage - armorReduction + 1;
};
```

2215 Find The Difference Of Two Arrays

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 2215. Find the Difference of Two Arrays
 * https://leetcode.com/problems/find-the-difference-of-two-arrays/
 * Difficulty: Easy
 *
 * Given two 0-indexed integer arrays nums1 and nums2, return a list answer of size 2 where:
 * - answer[0] is a list of all distinct integers in nums1 which are not present in nums2.
 * - answer[1] is a list of all distinct integers in nums2 which are not present in nums1.
```

```
 *
 * Note that the integers in the lists may be returned in any order.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[][]}
 */
var findDifference = function(nums1, nums2) {
  const set1 = new Set(nums1);
  const set2 = new Set(nums2);
  return [
    [...set1].filter(n => !set2.has(n)),
    [...set2].filter(n => !set1.has(n))
  ];
};
```

2216 Minimum Deletions To Make Array Beautiful

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2216. Minimum Deletions to Make Array Beautiful
 * https://leetcode.com/problems/minimum-deletions-to-make-array-beautiful/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. The array nums is beautiful if:
 * - nums.length is even.
 * - nums[i] != nums[i + 1] for all i % 2 == 0.
 *
 * Note that an empty array is considered beautiful.
 *
 * You can delete any number of elements from nums. When you delete an element, all the
elements
 * to the right of the deleted element will be shifted one unit to the left to fill the gap
 * created and all the elements to the left of the deleted element will remain unchanged.
 *
 * Return the minimum number of elements to delete from nums to make it beautiful.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minDeletion = function(nums) {
  let result = 0;
  let i = 0;

  while (i < nums.length - 1) {
    if (nums[i] === nums[i + 1]) {
      result++;
      i++;
    } else {
      i += 2;
    }
  }

  if ((nums.length - result) % 2 !== 0) {
    result++;
  }

  return result;
};
```

2217 Find Palindrome With Fixed Length

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2217. Find Palindrome With Fixed Length
 * https://leetcode.com/problems/find-palindrome-with-fixed-length/
 * Difficulty: Medium
 *
 * Given an integer array queries and a positive integer intLength, return an array answer
where
 * answer[i] is either the queries[i]th smallest positive palindrome of length intLength or -1
 * if no such palindrome exists.
 *
 * A palindrome is a number that reads the same backwards and forwards. Palindromes cannot
have
 * leading zeros.
 */

/**
 * @param {number[]} queries
 * @param {number} intLength
 * @return {number[]}
 */
var kthPalindrome = function(queries, intLength) {
  const halfLength = Math.ceil(intLength / 2);
  const maxPalindromes = Math.pow(10, halfLength - 1) * 9;

  return queries.map(generatePalindrome);

  function generatePalindrome(query) {
    if (query > maxPalindromes) return -1;

    let firstHalf = Math.pow(10, halfLength - 1) + query - 1;
    let result = firstHalf;

    if (intLength % 2 === 1) firstHalf = Math.floor(firstHalf / 10);

    while (firstHalf > 0) {
      result = result * 10 + (firstHalf % 10);
      firstHalf = Math.floor(firstHalf / 10);
    }

    return result;
  }
};
```

2218 Maximum Value Of K Coins From Piles

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2218. Maximum Value of K Coins From Piles
 * https://leetcode.com/problems/maximum-value-of-k-coins-from-piles/
 * Difficulty: Hard
 *
 * There are n piles of coins on a table. Each pile consists of a positive number of coins of
 * assorted denominations.
 *
 * In one move, you can choose any coin on top of any pile, remove it, and add it to your
wallet.
 *
 * Given a list piles, where piles[i] is a list of integers denoting the composition of the
ith
 * pile from top to bottom, and a positive integer k, return the maximum total value of coins
```

```
 * you can have in your wallet if you choose exactly k coins optimally.
 */

/**
 * @param {number[][]} piles
 * @param {number} k
 * @return {number}
 */
var maxValueOfCoins = function(piles, k) {
  const n = piles.length;
  const dp = Array.from({ length: n + 1 }, () => new Array(k + 1).fill(-1));

  return maximize(0, k);

  function maximize(pileIndex, remainingCoins) {
    if (pileIndex === n || remainingCoins === 0) return 0;
    if (dp[pileIndex][remainingCoins] !== -1) return dp[pileIndex][remainingCoins];

    let maxValue = maximize(pileIndex + 1, remainingCoins);
    let currentSum = 0;

    for (let i = 0; i < Math.min(piles[pileIndex].length, remainingCoins); i++) {
      currentSum += piles[pileIndex][i];
      maxValue = Math.max(
        maxValue,
        currentSum + maximize(pileIndex + 1, remainingCoins - (i + 1))
      );
    }

    return dp[pileIndex][remainingCoins] = maxValue;
  }
};
```

2219 Maximum Sum Score Of Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2219. Maximum Sum Score of Array
 * https://leetcode.com/problems/maximum-sum-score-of-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of length n.
 *
 * The sum score of nums at an index i where 0 <= i < n is the maximum of:
 * - The sum of the first i + 1 elements of nums.
 * - The sum of the last n - i elements of nums.
 *
 * Return the maximum sum score of nums at any index.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumSumScore = function(nums) {
  const n = nums.length;
  const totalSum = nums.reduce((sum, num) => sum + num, 0);
  let prefixSum = 0;
  let maxScore = -Infinity;

  for (let i = 0; i < n; i++) {
    prefixSum += nums[i];
    const suffixSum = totalSum - prefixSum + nums[i];
    const score = Math.max(prefixSum, suffixSum);
    maxScore = Math.max(maxScore, score);
  }

  return maxScore;
};
```

2220 Minimum Bit Flips To Convert Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2220. Minimum Bit Flips to Convert Number
 * https://leetcode.com/problems/minimum-bit-flips-to-convert-number/
 * Difficulty: Easy
 *
 * A bit flip of a number x is choosing a bit in the binary representation of x and flipping
it
 * from either 0 to 1 or 1 to 0.
 * - For example, for x = 7, the binary representation is 111 and we may choose any bit
(including
 *   any leading zeros not shown) and flip it. We can flip the first bit from the right to get
 110,
 *   flip the second bit from the right to get 101, flip the fifth bit from the right (a
leading
 *   zero) to get 10111, etc.
 *
 * Given two integers start and goal, return the minimum number of bit flips to convert start
to
 * goal.
 */

/**
 * @param {number} start
 * @param {number} goal
 * @return {number}
 */
var minBitFlips = function(start, goal) {
  return (start ^ goal).toString(2).replace(/0+/g, '').length;
};
```

2221 Find Triangular Sum Of An Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2221. Find Triangular Sum of an Array
 * https://leetcode.com/problems/find-triangular-sum-of-an-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums, where nums[i] is a digit between 0 and
 * 9 (inclusive).
 *
 * The triangular sum of nums is the value of the only element present in nums after the
 * following process terminates:
 * 1. Let nums comprise of n elements. If n == 1, end the process. Otherwise, create a new
 *    0-indexed integer array newNums of length n - 1.
 * 2. For each index i, where 0 <= i < n - 1, assign the value of newNums[i] as
 *    (nums[i] + nums[i+1]) % 10, where % denotes modulo operator.
 * 3. Replace the array nums with newNums.
 * 4. Repeat the entire process starting from step 1.
 *
 * Return the triangular sum of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var triangularSum = function(nums) {
  while (nums.length > 1) {
    const updated = [];
```

```
    for (let i = 0; i < nums.length - 1; i++) {
      updated.push((nums[i] + nums[i + 1]) % 10);
    }
    nums = updated;
  }
  return nums[0];
};
```

2222 Number Of Ways To Select Buildings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2222. Number of Ways to Select Buildings
 * https://leetcode.com/problems/number-of-ways-to-select-buildings/
 * Difficulty: Medium
 *
 * You are given a 0-indexed binary string s which represents the types of buildings along
 * a street where:
 * - s[i] = '0' denotes that the ith building is an office and
 * - s[i] = '1' denotes that the ith building is a restaurant.
 *
 * As a city official, you would like to select 3 buildings for random inspection. However, to
 * ensure variety, no two consecutive buildings out of the selected buildings can be of the
same
 * type.
 * - For example, given s = "001101", we cannot select the 1st, 3rd, and 5th buildings as that
 *   would form "011" which is not allowed due to having two consecutive buildings of the same
 type.
 *
 * Return the number of valid ways to select 3 buildings.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numberOfWays = function(s) {
  const prefixCounts = [[0, 0]];
  let zeros = 0;
  let ones = 0;

  for (const char of s) {
    if (char === '0') zeros++;
    else ones++;
    prefixCounts.push([zeros, ones]);
  }

  let result = 0;
  for (let i = 1; i < s.length - 1; i++) {
    if (s[i] === '0') {
      const leftOnes = prefixCounts[i][1];
      const rightOnes = prefixCounts[s.length][1] - prefixCounts[i + 1][1];
      result += leftOnes * rightOnes;
    } else {
      const leftZeros = prefixCounts[i][0];
      const rightZeros = prefixCounts[s.length][0] - prefixCounts[i + 1][0];
      result += leftZeros * rightZeros;
    }
  }

  return result;
};
```

2223 Sum Of Scores Of Built Strings

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2223. Sum of Scores of Built Strings
 * https://leetcode.com/problems/sum-of-scores-of-built-strings/
 * Difficulty: Hard
 *
 * You are building a string s of length n one character at a time, prepending each new
character
 * to the front of the string. The strings are labeled from 1 to n, where the string with
length
 * i is labeled si.
 * - For example, for s = "abaca", s1 == "a", s2 == "ca", s3 == "aca", etc.
 *
 * The score of si is the length of the longest common prefix between si and sn (Note that s
== sn).
 *
 * Given the final string s, return the sum of the score of every si.
 */

/**
 * @param {string} s
 * @return {number}
 */
var sumScores = function(s) {
  const n = s.length;
  const z = new Array(n).fill(0);
  let left = 0;
  let right = 0;

  for (let i = 1; i < n; i++) {
    if (i <= right) {
      z[i] = Math.min(right - i + 1, z[i - left]);
    }
    while (i + z[i] < n && s[z[i]] === s[i + z[i]]) {
      z[i]++;
    }
    if (i + z[i] - 1 > right) {
      left = i;
      right = i + z[i] - 1;
    }
  }

  return z.reduce((sum, val) => sum + val, n);
};
```

2224 Minimum Number Of Operations To Convert Time

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2224. Minimum Number of Operations to Convert Time
 * https://leetcode.com/problems/minimum-number-of-operations-to-convert-time/
 * Difficulty: Easy
 *
 * You are given two strings current and correct representing two 24-hour times.
 *
 * 24-hour times are formatted as "HH:MM", where HH is between 00 and 23, and MM is between 00
 * and 59. The earliest 24-hour time is 00:00, and the latest is 23:59.
 *
 * In one operation you can increase the time current by 1, 5, 15, or 60 minutes. You can
perform
 * this operation any number of times.
```

```
 *
 * Return the minimum number of operations needed to convert current to correct.
 */

/**
 * @param {string} current
 * @param {string} correct
 * @return {number}
 */
var convertTime = function(current, correct) {
  const toMinutes = time => {
    const [hours, minutes] = time.split(':').map(Number);
    return hours * 60 + minutes;
  };

  let diff = toMinutes(correct) - toMinutes(current);
  const increments = [60, 15, 5, 1];
  let result = 0;

  for (const increment of increments) {
    result += Math.floor(diff / increment);
    diff %= increment;
  }

  return result;
};
```

2225 Find Players With Zero Or One Losses

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2225. Find Players With Zero or One Losses
 * https://leetcode.com/problems/find-players-with-zero-or-one-losses/
 * Difficulty: Medium
 *
 * You are given an integer array matches where matches[i] = [winneri, loseri] indicates that
 * the player winneri defeated player loseri in a match.
 *
 * Return a list answer of size 2 where:
 * - answer[0] is a list of all players that have not lost any matches.
 * - answer[1] is a list of all players that have lost exactly one match.
 *
 * The values in the two lists should be returned in increasing order.
 *
 * Note:
 * - You should only consider the players that have played at least one match.
 * - The testcases will be generated such that no two matches will have the same outcome.
 */

/**
 * @param {number[][]} matches
 * @return {number[][]}
 */
var findWinners = function(matches) {
  const lossCount = new Map();

  for (const [winner, loser] of matches) {
    lossCount.set(winner, lossCount.get(winner) || 0);
    lossCount.set(loser, (lossCount.get(loser) || 0) + 1);
  }

  const noLosses = [];
  const oneLoss = [];

  for (const [player, losses] of lossCount) {
    if (losses === 0) noLosses.push(player);
    else if (losses === 1) oneLoss.push(player);
  }
```

```
  return [
    noLosses.sort((a, b) => a - b),
    oneLoss.sort((a, b) => a - b)
  ];
};
```

2226 Maximum Candies Allocated To K Children

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2226. Maximum Candies Allocated to K Children
 * https://leetcode.com/problems/maximum-candies-allocated-to-k-children/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array candies. Each element in the array denotes a pile
of
 * candies of size candies[i]. You can divide each pile into any number of sub piles, but you
 * cannot merge two piles together.
 *
 * You are also given an integer k. You should allocate piles of candies to k children such
that
 * each child gets the same number of candies. Each child can be allocated candies from only
one
 * pile of candies and some piles of candies may go unused.
 *
 * Return the maximum number of candies each child can get.
 */

/**
 * @param {number[]} candies
 * @param {number} k
 * @return {number}
 */
var maximumCandies = function(candies, k) {
  let left = 0;
  let right = 1e7 + 1;

  while (left + 1 !== right) {
    const middle = Math.floor((left + right) / 2);
    let piles = 0;
    for (const candy of candies) {
      piles += Math.floor(candy / middle);
    }
    if (piles >= k) {
      left = middle;
    } else {
      right = middle;
    }
  }

  return left;
};
```

2227 Encrypt And Decrypt Strings

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2227. Encrypt and Decrypt Strings
 * https://leetcode.com/problems/encrypt-and-decrypt-strings/
 * Difficulty: Hard
 *
 * You are given a character array keys containing unique characters and a string array values
```

```
 * containing strings of length 2. You are also given another string array dictionary that
 * contains all permitted original strings after decryption. You should implement a data
structure
 * that can encrypt or decrypt a 0-indexed string.
 *
 * A string is encrypted with the following process:
 * 1. For each character c in the string, we find the index i satisfying keys[i] == c in keys.
 * 2. Replace c with values[i] in the string.
 *
 * Note that in case a character of the string is not present in keys, the encryption process
cannot
 * be carried out, and an empty string "" is returned.
 *
 * A string is decrypted with the following process:
 * 1. For each substring s of length 2 occurring at an even index in the string, we find an i
such
 *    that values[i] == s. If there are multiple valid i, we choose any one of them. This
means a
 *    string could have multiple possible strings it can decrypt to.
 * 2. Replace s with keys[i] in the string.
 *
 * Implement the Encrypter class:
 * - Encrypter(char[] keys, String[] values, String[] dictionary) Initializes the Encrypter
class
 *   with keys, values, and dictionary.
 * - String encrypt(String word1) Encrypts word1 with the encryption process described above
and
 *   returns the encrypted string.
 * - int decrypt(String word2) Returns the number of possible strings word2 could decrypt to
that
 *   also appear in dictionary.
 */

/**
 * @param {character[]} keys
 * @param {string[]} values
 * @param {string[]} dictionary
 */
var Encrypter = function(keys, values, dictionary) {
  this.encryptMap = new Map();
  this.validEncryptions = new Map();

  for (let i = 0; i < keys.length; i++) {
    this.encryptMap.set(keys[i], values[i]);
  }

  for (const word of dictionary) {
    const encrypted = this.encrypt(word);
    if (encrypted !== '') {
      this.validEncryptions.set(encrypted, (this.validEncryptions.get(encrypted) || 0) + 1);
    }
  }
};

/**
 * @param {string} word1
 * @return {string}
 */
Encrypter.prototype.encrypt = function(word1) {
  let result = '';

  for (const char of word1) {
    if (!this.encryptMap.has(char)) {
      return '';
    }
    result += this.encryptMap.get(char);
  }

  return result;
};

/**
 * @param {string} word2
 * @return {number}
 */
```

```
Encrypter.prototype.decrypt = function(word2) {
  return this.validEncryptions.get(word2) || 0;
};
```

2229 Check If An Array Is Consecutive

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2229. Check if an Array Is Consecutive
 * https://leetcode.com/problems/check-if-an-array-is-consecutive/
 * Difficulty: Easy
 *
 * Given an integer array nums, return true if nums is consecutive, otherwise return false.
 *
 * An array is consecutive if it contains every number in the range [x, x + n - 1]
(inclusive),
 * where x is the minimum number in the array and n is the length of the array.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isConsecutive = function(nums) {
  const set = new Set(nums);
  if (set.size !== nums.length) return false;

  const minNum = Math.min(...nums);
  for (let i = 0; i < nums.length; i++) {
    if (!set.has(minNum + i)) {
      return false;
    }
  }

  return true;
};
```

2231 Largest Number After Digit Swaps By Parity

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2231. Largest Number After Digit Swaps by Parity
 * https://leetcode.com/problems/largest-number-after-digit-swaps-by-parity/
 * Difficulty: Easy
 *
 * You are given a positive integer num. You may swap any two digits of num that have the
 * same parity (i.e. both odd digits or both even digits).
 *
 * Return the largest possible value of num after any number of swaps.
 */

/**
 * @param {number} num
 * @return {number}
 */
var largestInteger = function(num) {
  const digits = num.toString().split('').map(Number);
  const evenDigits = [];
  const oddDigits = [];

  for (const digit of digits) {
    if (digit % 2 === 0) {
      evenDigits.push(digit);
```

```
    } else {
      oddDigits.push(digit);
    }
  }

  evenDigits.sort((a, b) => b - a);
  oddDigits.sort((a, b) => b - a);

  let evenIndex = 0;
  let oddIndex = 0;
  const result = [];

  for (const digit of digits) {
    if (digit % 2 === 0) {
      result.push(evenDigits[evenIndex++]);
    } else {
      result.push(oddDigits[oddIndex++]);
    }
  }

  return parseInt(result.join(''), 10);
};
```

2232 Minimize Result By Adding Parentheses To Expression

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2232. Minimize Result by Adding Parentheses to Expression
 * https://leetcode.com/problems/minimize-result-by-adding-parentheses-to-expression/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string expression of the form "<num1>+<num2>" where <num1>
 * and <num2> represent positive integers.
 *
 * Add a pair of parentheses to expression such that after the addition of parentheses,
 * expression is a valid mathematical expression and evaluates to the smallest possible
 * value. The left parenthesis must be added to the left of '+' and the right parenthesis
 * must be added to the right of '+'.
 *
 * Return expression after adding a pair of parentheses such that expression evaluates to the
 * smallest possible value. If there are multiple answers that yield the same result, return
 * any of them.
 *
 * The input has been generated such that the original value of expression, and the value of
 * expression after adding any pair of parentheses that meets the requirements fits within
 * a signed 32-bit integer.
 */

/**
 * @param {string} expression
 * @return {string}
 */
var minimizeResult = function(expression) {
  const plusIndex = expression.indexOf('+');
  const left = expression.slice(0, plusIndex);
  const right = expression.slice(plusIndex + 1);
  let minValue = Infinity;
  let result = '';

  for (let i = 0; i < left.length; i++) {
    for (let j = 1; j <= right.length; j++) {
      const leftPrefix = i === 0 ? 1 : parseInt(left.slice(0, i));
      const leftNum = parseInt(left.slice(i), 10);
      const rightNum = parseInt(right.slice(0, j), 10);
      const rightSuffix = j === right.length ? 1 : parseInt(right.slice(j), 10);
      const value = leftPrefix * (leftNum + rightNum) * rightSuffix;

      if (value < minValue) {
        minValue = value;
```

```
        result = `${left.slice(0, i)}(${left.slice(i)}+${right.slice(0,
j)})${right.slice(j)}`;
      }
    }
  }

  return result;
};
```

2233 Maximum Product After K Increments

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2233. Maximum Product After K Increments
 * https://leetcode.com/problems/maximum-product-after-k-increments/
 * Difficulty: Medium
 *
 * You are given an array of non-negative integers nums and an integer k. In one operation,
 * you may choose any element from nums and increment it by 1.
 *
 * Return the maximum product of nums after at most k operations. Since the answer may be
 * very large, return it modulo 109 + 7. Note that you should maximize the product before
 * taking the modulo.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumProduct = function(nums, k) {
  const MOD = 1e9 + 7;
  const minHeap = new PriorityQueue((a, b) => a - b);

  for (const num of nums) {
    minHeap.enqueue(num);
  }

  for (let i = 0; i < k; i++) {
    const smallest = minHeap.dequeue();
    minHeap.enqueue(smallest + 1);
  }

  let result = 1;
  while (!minHeap.isEmpty()) {
    result = (result * minHeap.dequeue()) % MOD;
  }

  return result;
};
```

2234 Maximum Total Beauty Of The Gardens

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2234. Maximum Total Beauty of the Gardens
 * https://leetcode.com/problems/maximum-total-beauty-of-the-gardens/
 * Difficulty: Hard
 *
 * Alice is a caretaker of n gardens and she wants to plant flowers to maximize the total
 * beauty of all her gardens.
 *
```

```
 * You are given a 0-indexed integer array flowers of size n, where flowers[i] is the number
 * of flowers already planted in the ith garden. Flowers that are already planted cannot be
 * removed. You are then given another integer newFlowers, which is the maximum number of
 * flowers that Alice can additionally plant. You are also given the integers target, full,
 * and partial.
 *
 * A garden is considered complete if it has at least target flowers. The total beauty of the
 * gardens is then determined as the sum of the following:
 * - The number of complete gardens multiplied by full.
 * - The minimum number of flowers in any of the incomplete gardens multiplied by partial.
 *   If there are no incomplete gardens, then this value will be 0.
 *
 * Return the maximum total beauty that Alice can obtain after planting at most newFlowers
flowers.
 */

/**
 * @param {number[]} flowers
 * @param {number} newFlowers
 * @param {number} target
 * @param {number} full
 * @param {number} partial
 * @return {number}
 */
var maximumBeauty = function(flowers, newFlowers, target, full, partial) {
  const n = flowers.length;
  flowers.sort((a, b) => b - a);

  const suffixCost = Array(n + 1).fill(0);
  let lastUniqueIndex = n - 1;
  let minIndex = n - 2;

  for (; minIndex >= 0; --minIndex) {
    if (flowers[minIndex] >= target) break;

    const flowerDiff = flowers[minIndex] - flowers[minIndex + 1];
    const gardenCount = n - lastUniqueIndex;
    suffixCost[minIndex] = suffixCost[minIndex + 1] + flowerDiff * gardenCount;

    if (suffixCost[minIndex] > newFlowers) break;

    if (flowers[minIndex] !== flowers[minIndex - 1]) {
      lastUniqueIndex = minIndex;
    }
  }

  ++minIndex;

  const remainingFlowersForMin = newFlowers - suffixCost[minIndex];
  const gardenCountForMin = n - minIndex;
  let minFlowerValue = Math.min(
    target - 1,
    flowers[minIndex] + Math.floor(remainingFlowersForMin / gardenCountForMin)
  );

  let result = 0;

  for (let i = 0; i < n; ++i) {
    if (flowers[i] >= target) {
      continue;
    }

    const currentBeauty = i * full + minFlowerValue * partial;
    result = Math.max(result, currentBeauty);

    const flowersNeeded = target - flowers[i];
    if (flowersNeeded > newFlowers) break;

    newFlowers -= flowersNeeded;
    flowers[i] = target;

    while (
      minIndex <= i || newFlowers < suffixCost[minIndex]
      || (minIndex > 0 && flowers[minIndex] === flowers[minIndex - 1])
    ) {
```

```
      ++minIndex;
    }

    const updatedRemaining = newFlowers - suffixCost[minIndex];
    const updatedGardenCount = n - minIndex;
    minFlowerValue = Math.min(
      target - 1,
      flowers[minIndex] + Math.floor(updatedRemaining / updatedGardenCount)
    );
  }

  if (flowers[n - 1] >= target) {
    result = Math.max(result, n * full);
  }

  return result;
};
```

2235 Add Two Integers

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2235. Add Two Integers
 * https://leetcode.com/problems/add-two-integers/
 * Difficulty: Easy
 *
 * Given two integers num1 and num2, return the sum of the two integers.
 */

/**
 * @param {number} num1
 * @param {number} num2
 * @return {number}
 */
var sum = function(num1, num2) {
  return num1 + num2;
};
```

2236 Root Equals Sum Of Children

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2236. Root Equals Sum of Children
 * https://leetcode.com/problems/root-equals-sum-of-children/
 * Difficulty: Easy
 *
 * You are given the root of a binary tree that consists of exactly 3 nodes: the root, its
 * left child, and its right child.
 *
 * Return true if the value of the root is equal to the sum of the values of its two children,
 * or false otherwise.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
```

```
 * @return {boolean}
 */
var checkTree = function(root) {
  return root.val === root.left.val + root.right.val;
};
```

2237 Count Positions On Street With Required Brightness

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2237. Count Positions on Street With Required Brightness
 * https://leetcode.com/problems/count-positions-on-street-with-required-brightness/
 * Difficulty: Medium
 *
 * You are given an integer n. A perfectly straight street is represented by a number line
 * ranging from 0 to n - 1. You are given a 2D integer array lights representing the street
 * lamp(s) on the street. Each lights[i] = [positioni, rangei] indicates that there is a
 * street lamp at position positioni that lights up the area from
 * [max(0, positioni - rangei), min(n - 1, positioni + rangei)] (inclusive).
 *
 * The brightness of a position p is defined as the number of street lamps that light up the
 * position p. You are given a 0-indexed integer array requirement of size n where
 * requirement[i] is the minimum brightness of the ith position on the street.
 *
 * Return the number of positions i on the street between 0 and n - 1 that have a brightness
 * of at least requirement[i].
 */

/**
 * @param {number} n
 * @param {number[][]} lights
 * @param {number[]} requirement
 * @return {number}
 */
var meetRequirement = function(n, lights, requirement) {
  const brightness = new Array(n).fill(0);
  for (const [position, range] of lights) {
    const start = Math.max(0, position - range);
    const end = Math.min(n - 1, position + range);

    brightness[start]++;
    if (end + 1 < n) {
      brightness[end + 1]--;
    }
  }

  let currentBrightness = 0;
  let result = 0;
  for (let i = 0; i < n; i++) {
    currentBrightness += brightness[i];
    if (currentBrightness >= requirement[i]) {
      result++;
    }
  }

  return result;
};
```

2239 Find Closest Number To Zero

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2239. Find Closest Number to Zero
```

```
 * https://leetcode.com/problems/find-closest-number-to-zero/
 * Difficulty: Easy
 *
 * Given an integer array nums of size n, return the number with the value closest to 0 in
nums.
 * If there are multiple answers, return the number with the largest value.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findClosestNumber = function(nums) {
  let result = nums[0];
  let minDistance = Math.abs(nums[0]);

  for (const num of nums) {
    const distance = Math.abs(num);
    if (distance < minDistance || (distance === minDistance && num > result)) {
      minDistance = distance;
      result = num;
    }
  }

  return result;
};
```

2240 Number Of Ways To Buy Pens And Pencils

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2240. Number of Ways to Buy Pens and Pencils
 * https://leetcode.com/problems/number-of-ways-to-buy-pens-and-pencils/
 * Difficulty: Medium
 *
 * You are given an integer total indicating the amount of money you have. You are also given
 * two integers cost1 and cost2 indicating the price of a pen and pencil respectively. You can
 * spend part or all of your money to buy multiple quantities (or none) of each kind of
writing
 * utensil.
 *
 * Return the number of distinct ways you can buy some number of pens and pencils.
 */

/**
 * @param {number} total
 * @param {number} cost1
 * @param {number} cost2
 * @return {number}
 */
var waysToBuyPensPencils = function(total, cost1, cost2) {
  let result = 0;
  const maxPens = Math.floor(total / cost1);

  for (let pens = 0; pens <= maxPens; pens++) {
    const remaining = total - pens * cost1;
    const pencils = Math.floor(remaining / cost2);
    result += pencils + 1;
  }

  return result;
};
```

2241 Design An Atm Machine

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2241. Design an ATM Machine
 * https://leetcode.com/problems/design-an-atm-machine/
 * Difficulty: Medium
 *
 * There is an ATM machine that stores banknotes of 5 denominations: 20, 50, 100, 200, and
 * 500 dollars. Initially the ATM is empty. The user can use the machine to deposit or
 * withdraw any amount of money.
 *
 * When withdrawing, the machine prioritizes using banknotes of larger values.
 * - For example, if you want to withdraw $300 and there are 2 $50 banknotes, 1 $100 banknote,
 *   and 1 $200 banknote, then the machine will use the $100 and $200 banknotes.
 * - However, if you try to withdraw $600 and there are 3 $200 banknotes and 1 $500 banknote,
 *   then the withdraw request will be rejected because the machine will first try to use the
 *   $500 banknote and then be unable to use banknotes to complete the remaining $100. Note
 *   that the machine is not allowed to use the $200 banknotes instead of the $500 banknote.
 *
 * Implement the ATM class:
 * - ATM() Initializes the ATM object.
 * - void deposit(int[] banknotesCount) Deposits new banknotes in the order $20, $50, $100,
 *   $200, and $500.
 * - int[] withdraw(int amount) Returns an array of length 5 of the number of banknotes that
 *   will be handed to the user in the order $20, $50, $100, $200, and $500, and update the
 *   number of banknotes in the ATM after withdrawing. Returns [-1] if it is not possible (do
 *   not withdraw any banknotes in this case).
 */

var ATM = function() {
  this.denominations = [20, 50, 100, 200, 500];
  this.notes = [0, 0, 0, 0, 0];
};

/**
 * @param {number[]} banknotesCount
 * @return {void}
 */
ATM.prototype.deposit = function(banknotesCount) {
  for (let i = 0; i < 5; i++) {
    this.notes[i] += banknotesCount[i];
  }
};

/**
 * @param {number} amount
 * @return {number[]}
 */
ATM.prototype.withdraw = function(amount) {
  const result = [0, 0, 0, 0, 0];
  let remaining = amount;

  for (let i = 4; i >= 0; i--) {
    const count = Math.min(Math.floor(remaining / this.denominations[i]), this.notes[i]);
    result[i] = count;
    remaining -= count * this.denominations[i];
  }

  if (remaining !== 0) return [-1];

  for (let i = 0; i < 5; i++) {
    this.notes[i] -= result[i];
  }

  return result;
};
```

2242 Maximum Score Of A Node Sequence

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2242. Maximum Score of a Node Sequence
 * https://leetcode.com/problems/maximum-score-of-a-node-sequence/
 * Difficulty: Hard
 *
 * There is an undirected graph with n nodes, numbered from 0 to n - 1.
 *
 * You are given a 0-indexed integer array scores of length n where scores[i] denotes the
score
 * of node i. You are also given a 2D integer array edges where edges[i] = [ai, bi] denotes
that
 * there exists an undirected edge connecting nodes ai and bi.
 *
 * A node sequence is valid if it meets the following conditions:
 * - There is an edge connecting every pair of adjacent nodes in the sequence.
 * - No node appears more than once in the sequence.
 *
 * The score of a node sequence is defined as the sum of the scores of the nodes in the
sequence.
 *
 * Return the maximum score of a valid node sequence with a length of 4. If no such sequence
exists,
 * return -1.
 */

/**
 * @param {number[]} scores
 * @param {number[][]} edges
 * @return {number}
 */
var maximumScore = function(scores, edges) {
  const n = scores.length;
  const graph = new Array(n).fill().map(() => []);

  for (const [a, b] of edges) {
    graph[a].push(b);
    graph[b].push(a);
  }

  for (let i = 0; i < n; i++) {
    graph[i].sort((a, b) => scores[b] - scores[a]);
    if (graph[i].length > 3) {
      graph[i] = graph[i].slice(0, 3);
    }
  }

  let result = -1;
  for (const [a, b] of edges) {
    for (const c of graph[a]) {
      if (c === b) continue;
      for (const d of graph[b]) {
        if (d === a || d === c) continue;
        result = Math.max(result, scores[a] + scores[b] + scores[c] + scores[d]);
      }
    }
  }

  return result;
};
```

2243 Calculate Digit Sum Of A String

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2243. Calculate Digit Sum of a String
 * https://leetcode.com/problems/calculate-digit-sum-of-a-string/
 * Difficulty: Easy
 *
 * You are given a string s consisting of digits and an integer k.
 *
 * A round can be completed if the length of s is greater than k. In one round, do the
following:
 * 1. Divide s into consecutive groups of size k such that the first k characters are in the
first
 *    group, the next k characters are in the second group, and so on. Note that the size of
the
 *    last group can be smaller than k.
 * 2. Replace each group of s with a string representing the sum of all its digits. For
example,
 *    "346" is replaced with "13" because 3 + 4 + 6 = 13.
 * 3. Merge consecutive groups together to form a new string. If the length of the string is
greater
 *    than k, repeat from step 1.
 *
 * Return s after all rounds have been completed.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var digitSum = function(s, k) {
  while (s.length > k) {
    let next = '';
    for (let i = 0; i < s.length; i += k) {
      const group = s.slice(i, i + k);
      const sum = group.split('').reduce((acc, digit) => acc + Number(digit), 0);
      next += sum;
    }
    s = next;
  }
  return s;
};
```

2244 Minimum Rounds To Complete All Tasks

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2244. Minimum Rounds to Complete All Tasks
 * https://leetcode.com/problems/minimum-rounds-to-complete-all-tasks/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array tasks, where tasks[i] represents the difficulty
level
 * of a task. In each round, you can complete either 2 or 3 tasks of the same difficulty
level.
 *
 * Return the minimum rounds required to complete all the tasks, or -1 if it is not possible
to
 * complete all the tasks.
 */
```

```
/**
 * @param {number[]} tasks
 * @return {number}
 */
var minimumRounds = function(tasks) {
  const map = new Map();
  let result = 0;

  tasks.forEach(task => map.set(task, map.has(task) ? map.get(task) + 1 : 1));

  for (const count of map.values()) {
    if (count < 2) {
      return -1;
    }
    result += Math.floor(count / 3) + (count % 3 === 0 ? 0 : 1);
  }

  return result;
};
```

2245 Maximum Trailing Zeros In A Cornered Path

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2245. Maximum Trailing Zeros in a Cornered Path
 * https://leetcode.com/problems/maximum-trailing-zeros-in-a-cornered-path/
 * Difficulty: Medium
 *
 * You are given a 2D integer array grid of size m x n, where each cell contains a positive
integer.
 *
 * A cornered path is defined as a set of adjacent cells with at most one turn. More
specifically,
 * the path should exclusively move either horizontally or vertically up to the turn (if there
 is
 * one), without returning to a previously visited cell. After the turn, the path will then
move
 * exclusively in the alternate direction: move vertically if it moved horizontally, and vice
versa,
 * also without returning to a previously visited cell.
 *
 * The product of a path is defined as the product of all the values in the path.
 *
 * Return the maximum number of trailing zeros in the product of a cornered path found in
grid.
 *
 * Note:
 * - Horizontal movement means moving in either the left or right direction.
 * - Vertical movement means moving in either the up or down direction.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maxTrailingZeros = function(grid) {
  const m = grid.length;
  const n = grid[0].length;

  function countFactors(num) {
    let count2 = 0;
    let count5 = 0;

    while (num % 2 === 0) {
      count2++;
      num = Math.floor(num / 2);
    }

    while (num % 5 === 0) {
```

```
      count5++;
      num = Math.floor(num / 5);
    }

    return [count2, count5];
  }

  const factors = new Array(m).fill().map(() => new Array(n).fill().map(() => [0, 0]));
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      factors[i][j] = countFactors(grid[i][j]);
    }
  }

  const rowPrefix = new Array(m).fill().map(() => new Array(n + 1).fill().map(() => [0, 0]));
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      rowPrefix[i][j + 1][0] = rowPrefix[i][j][0] + factors[i][j][0];
      rowPrefix[i][j + 1][1] = rowPrefix[i][j][1] + factors[i][j][1];
    }
  }

  const colPrefix = new Array(m + 1).fill().map(() => new Array(n).fill().map(() => [0, 0]));
  for (let j = 0; j < n; j++) {
    for (let i = 0; i < m; i++) {
      colPrefix[i + 1][j][0] = colPrefix[i][j][0] + factors[i][j][0];
      colPrefix[i + 1][j][1] = colPrefix[i][j][1] + factors[i][j][1];
    }
  }

  let maxZeros = 0;
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      const [count2, count5] = factors[i][j];

      const leftUp = [
        rowPrefix[i][j][0] + colPrefix[i][j][0],
        rowPrefix[i][j][1] + colPrefix[i][j][1]
      ];

      const leftDown = [
        rowPrefix[i][j][0] + (colPrefix[m][j][0] - colPrefix[i + 1][j][0]),
        rowPrefix[i][j][1] + (colPrefix[m][j][1] - colPrefix[i + 1][j][1])
      ];

      const rightUp = [
        (rowPrefix[i][n][0] - rowPrefix[i][j + 1][0]) + colPrefix[i][j][0],
        (rowPrefix[i][n][1] - rowPrefix[i][j + 1][1]) + colPrefix[i][j][1]
      ];

      const rightDown = [
        (rowPrefix[i][n][0] - rowPrefix[i][j + 1][0])
          + (colPrefix[m][j][0] - colPrefix[i + 1][j][0]),
        (rowPrefix[i][n][1] - rowPrefix[i][j + 1][1])
          + (colPrefix[m][j][1] - colPrefix[i + 1][j][1])
      ];

      const paths = [
        [leftUp[0] + count2, leftUp[1] + count5],
        [leftDown[0] + count2, leftDown[1] + count5],
        [rightUp[0] + count2, rightUp[1] + count5],
        [rightDown[0] + count2, rightDown[1] + count5]
      ];

      for (const [path2, path5] of paths) {
        maxZeros = Math.max(maxZeros, Math.min(path2, path5));
      }
    }
  }

  return maxZeros;
};
```

2246 Longest Path With Different Adjacent Characters

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2246. Longest Path With Different Adjacent Characters
 * https://leetcode.com/problems/longest-path-with-different-adjacent-characters/
 * Difficulty: Hard
 *
 * You are given a tree (i.e. a connected, undirected graph that has no cycles) rooted at node
 * 0 consisting of n nodes numbered from 0 to n - 1. The tree is represented by a 0-indexed
array
 * parent of size n, where parent[i] is the parent of node i. Since node 0 is the root,
 * parent[0] == -1.
 *
 * You are also given a string s of length n, where s[i] is the character assigned to node i.
 *
 * Return the length of the longest path in the tree such that no pair of adjacent nodes on
the path
 * have the same character assigned to them.
 */

/**
 * @param {number[]} parent
 * @param {string} s
 * @return {number}
 */
var longestPath = function(parent, s) {
  const n = parent.length;
  const children = Array.from({ length: n }, () => []);
  let result = 1;

  for (let i = 1; i < n; i++) {
    children[parent[i]].push(i);
  }

  findLongest(0);

  return result;

  function findLongest(node) {
    let longest = 0;
    let secondLongest = 0;

    for (const child of children[node]) {
      const length = findLongest(child);
      if (s[child] !== s[node]) {
        if (length > longest) {
          secondLongest = longest;
          longest = length;
        } else if (length > secondLongest) {
          secondLongest = length;
        }
      }
    }

    result = Math.max(result, longest + secondLongest + 1);
    return longest + 1;
  }
};
```

2247 Maximum Cost Of Trip With K Highways

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2247. Maximum Cost of Trip With K Highways
 * https://leetcode.com/problems/maximum-cost-of-trip-with-k-highways/
 * Difficulty: Hard
 *
 * A series of highways connect n cities numbered from 0 to n - 1. You are given a 2D integer
array
 * highways where highways[i] = [city1i, city2i, tolli] indicates that there is a highway that
 * connects city1i and city2i, allowing a car to go from city1i to city2i and vice versa for a
 cost
 * of tolli.
 *
 * You are also given an integer k. You are going on a trip that crosses exactly k highways.
You
 * may start at any city, but you may only visit each city at most once during your trip.
 *
 * Return the maximum cost of your trip. If there is no trip that meets the requirements,
return -1.
 */

/**
 * @param {number} n
 * @param {number[][]} highways
 * @param {number} k
 * @return {number}
 */
var maximumCost = function(n, highways, k) {
  if (k + 1 > n) return -1;

  const graph = Array(n).fill().map(() => []);
  for (const [cityA, cityB, cost] of highways) {
    graph[cityA].push([cityB, cost]);
    graph[cityB].push([cityA, cost]);
  }

  const memo = new Map();
  let result = -1;

  for (let city = 0; city < n; city++) {
    result = Math.max(result, dp(city, 1 << city));
  }

  return result;

  function dp(city, bitmask) {
    const citiesVisited = bitmask.toString(2).split('1').length - 1;
    if (citiesVisited === k + 1) return 0;

    const key = `${city}_${bitmask}`;
    if (memo.has(key)) return memo.get(key);

    let answer = -1;
    for (const [neighborCity, highwayCost] of graph[city]) {
      if (!((bitmask >> neighborCity) & 1)) {
        const neighborAnswer = dp(neighborCity, bitmask | (1 << neighborCity));
        if (neighborAnswer !== -1) {
          answer = Math.max(answer, highwayCost + neighborAnswer);
        }
      }
    }

    memo.set(key, answer);
    return answer;
  }
};
```

2248 Intersection Of Multiple Arrays

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2248. Intersection of Multiple Arrays
 * https://leetcode.com/problems/intersection-of-multiple-arrays/
 * Difficulty: Easy
 *
 * Given a 2D integer array nums where nums[i] is a non-empty array of distinct positive
integers,
 * return the list of integers that are present in each array of nums sorted in ascending
order.
 */

/**
 * @param {number[][]} nums
 * @return {number[]}
 */
var intersection = function(nums) {
  const count = new Map();

  for (const array of nums) {
    for (const num of array) {
      count.set(num, (count.get(num) || 0) + 1);
    }
  }

  const result = [];
  for (const [num, freq] of count) {
    if (freq === nums.length) {
      result.push(num);
    }
  }

  return result.sort((a, b) => a - b);
};
```

2249 Count Lattice Points Inside A Circle

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2249. Count Lattice Points Inside a Circle
 * https://leetcode.com/problems/count-lattice-points-inside-a-circle/
 * Difficulty: Medium
 *
 * Given a 2D integer array circles where circles[i] = [xi, yi, ri] represents the center (xi,
 yi)
 * and radius ri of the ith circle drawn on a grid, return the number of lattice points that
are
 * present inside at least one circle.
 *
 * Note:
 * - A lattice point is a point with integer coordinates.
 * - Points that lie on the circumference of a circle are also considered to be inside it.
 */

/**
 * @param {number[][]} circles
 * @return {number}
 */
var countLatticePoints = function(circles) {
  const set = new Set();

  for (const [x, y, r] of circles) {
    for (let i = x - r; i <= x + r; i++) {
      for (let j = y - r; j <= y + r; j++) {
        if ((x - i) ** 2 + (y - j) ** 2 <= r ** 2) {
          set.add(`${i},${j}`);
        }
      }
    }
```

```
  }

  return set.size;
};
```

2250 Count Number Of Rectangles Containing Each Point

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2250. Count Number of Rectangles Containing Each Point
 * https://leetcode.com/problems/count-number-of-rectangles-containing-each-point/
 * Difficulty: Medium
 *
 * You are given a 2D integer array rectangles where rectangles[i] = [li, hi] indicates that
ith
 * rectangle has a length of li and a height of hi. You are also given a 2D integer array
points
 * where points[j] = [xj, yj] is a point with coordinates (xj, yj).
 *
 * The ith rectangle has its bottom-left corner point at the coordinates (0, 0) and its
top-right
 * corner point at (li, hi).
 *
 * Return an integer array count of length points.length where count[j] is the number of
rectangles
 * that contain the jth point.
 *
 * The ith rectangle contains the jth point if 0 <= xj <= li and 0 <= yj <= hi. Note that
points
 * that lie on the edges of a rectangle are also considered to be contained by that rectangle.
 */

/**
 * @param {number[][]} rectangles
 * @param {number[][]} points
 * @return {number[]}
 */
var countRectangles = function(rectangles, points) {
  const rectByHeight = new Array(101).fill().map(() => []);

  for (const [length, height] of rectangles) {
    rectByHeight[height].push(length);
  }

  for (let h = 0; h <= 100; h++) {
    rectByHeight[h].sort((a, b) => a - b);
  }

  const result = new Array(points.length).fill(0);
  for (let i = 0; i < points.length; i++) {
    const [x, y] = points[i];

    for (let h = y; h <= 100; h++) {
      const lengths = rectByHeight[h];

      let left = 0;
      let right = lengths.length - 1;
      let insertPos = lengths.length;

      while (left <= right) {
        const mid = Math.floor((left + right) / 2);
        if (lengths[mid] >= x) {
          insertPos = mid;
          right = mid - 1;
        } else {
          left = mid + 1;
        }
      }
```

```
      result[i] += lengths.length - insertPos;
    }
  }

  return result;
};
```

2251 Number Of Flowers In Full Bloom

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2251. Number of Flowers in Full Bloom
 * https://leetcode.com/problems/number-of-flowers-in-full-bloom/
 * Difficulty: Hard
 *
 * You are given a 0-indexed 2D integer array flowers, where flowers[i] = [starti, endi] means
 * the ith flower will be in full bloom from starti to endi (inclusive). You are also given a
 * 0-indexed integer array people of size n, where people[i] is the time that the ith person
 * will arrive to see the flowers.
 *
 * Return an integer array answer of size n, where answer[i] is the number of flowers that are
 * in full bloom when the ith person arrives.
 */

/**
 * @param {number[][]} flowers
 * @param {number[]} people
 * @return {number[]}
 */
var fullBloomFlowers = function(flowers, people) {
  const events = [];
  for (const [start, end] of flowers) {
    events.push([start, 1]);
    events.push([end + 1, -1]);
  }
  events.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  const result = new Array(people.length).fill(0);
  const peopleWithIndex = people.map((time, index) => [time, index]);
  peopleWithIndex.sort((a, b) => a[0] - b[0]);

  let bloomCount = 0;
  let eventIndex = 0;
  for (const [time, personIndex] of peopleWithIndex) {
    while (eventIndex < events.length && events[eventIndex][0] <= time) {
      bloomCount += events[eventIndex][1];
      eventIndex++;
    }
    result[personIndex] = bloomCount;
  }

  return result;
};
```

2254 Design Video Sharing Platform

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2254. Design Video Sharing Platform
 * https://leetcode.com/problems/design-video-sharing-platform/
 * Difficulty: Hard
```

```
 *
 * You have a video sharing platform where users can upload and delete videos. Each video is a
 * string of digits, where the ith digit of the string represents the content of the video at
 * minute i. For example, the first digit represents the content at minute 0 in the video, the
 * second digit represents the content at minute 1 in the video, and so on. Viewers of videos
 * can also like and dislike videos. Internally, the platform keeps track of the number of
views,
 * likes, and dislikes on each video.
 *
 * When a video is uploaded, it is associated with the smallest available integer videoId
starting
 * from 0. Once a video is deleted, the videoId associated with that video can be reused for
 * another video.
 *
 * Implement the VideoSharingPlatform class:
 * - VideoSharingPlatform() Initializes the object.
 * - int upload(String video) The user uploads a video. Return the videoId associated with
 *   the video.
 * - void remove(int videoId) If there is a video associated with videoId, remove the video.
 * - String watch(int videoId, int startMinute, int endMinute) If there is a video associated
with
 *   videoId, increase the number of views on the video by 1 and return the substring of the
video
 *   string starting at startMinute and ending at min(endMinute, video.length - 1)
(inclusive).
 *   Otherwise, return "-1".
 * - void like(int videoId) Increases the number of likes on the video associated with videoId
 by 1
 *   if there is a video associated with videoId.
 * - void dislike(int videoId) Increases the number of dislikes on the video associated with
videoId
 *   by 1 if there is a video associated with videoId.
 * - int[] getLikesAndDislikes(int videoId) Return a 0-indexed integer array values of length
2
 *   where values[0] is the number of likes and values[1] is the number of dislikes on the
video
 *   associated with videoId. If there is no video associated with videoId, return [-1].
 * - int getViews(int videoId) Return the number of views on the video associated with
videoId, if
 *   there is no video associated with videoId, return -1.
 */

var VideoSharingPlatform = function() {
  this.videos = new Map();
  this.availableIds = [];
  this.nextId = 0;
};

/**
 * @param {string} video
 * @return {number}
 */
VideoSharingPlatform.prototype.upload = function(video) {
  let videoId;
  if (this.availableIds.length > 0) {
    videoId = this.availableIds.shift();
  } else {
    videoId = this.nextId++;
  }

  this.videos.set(videoId, {
    content: video,
    views: 0,
    likes: 0,
    dislikes: 0
  });

  return videoId;
};

/**
 * @param {number} videoId
 * @return {void}
 */
VideoSharingPlatform.prototype.remove = function(videoId) {
```

```
  if (this.videos.has(videoId)) {
    this.videos.delete(videoId);
    this.availableIds.push(videoId);
    this.availableIds.sort((a, b) => a - b);
  }
};

/**
 * @param {number} videoId
 * @param {number} startMinute
 * @param {number} endMinute
 * @return {string}
 */
VideoSharingPlatform.prototype.watch = function(videoId, startMinute, endMinute) {
  if (!this.videos.has(videoId)) {
    return '-1';
  }

  const videoData = this.videos.get(videoId);
  videoData.views++;

  const actualEndMinute = Math.min(endMinute, videoData.content.length - 1);
  return videoData.content.substring(startMinute, actualEndMinute + 1);
};

/**
 * @param {number} videoId
 * @return {void}
 */
VideoSharingPlatform.prototype.like = function(videoId) {
  if (this.videos.has(videoId)) {
    this.videos.get(videoId).likes++;
  }
};

/**
 * @param {number} videoId
 * @return {void}
 */
VideoSharingPlatform.prototype.dislike = function(videoId) {
  if (this.videos.has(videoId)) {
    this.videos.get(videoId).dislikes++;
  }
};

/**
 * @param {number} videoId
 * @return {number[]}
 */
VideoSharingPlatform.prototype.getLikesAndDislikes = function(videoId) {
  if (!this.videos.has(videoId)) {
    return [-1];
  }

  const videoData = this.videos.get(videoId);
  return [videoData.likes, videoData.dislikes];
};

/**
 * @param {number} videoId
 * @return {number}
 */
VideoSharingPlatform.prototype.getViews = function(videoId) {
  if (!this.videos.has(videoId)) {
    return -1;
  }

  return this.videos.get(videoId).views;
};
```

2255 Count Prefixes Of A Given String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2255. Count Prefixes of a Given String
 * https://leetcode.com/problems/count-prefixes-of-a-given-string/
 * Difficulty: Easy
 *
 * You are given a string array words and a string s, where words[i] and s comprise only of
 * lowercase English letters.
 *
 * Return the number of strings in words that are a prefix of s.
 *
 * A prefix of a string is a substring that occurs at the beginning of the string. A substring
 * is a contiguous sequence of characters within a string.
 */

/**
 * @param {string[]} words
 * @param {string} s
 * @return {number}
 */
var countPrefixes = function(words, s) {
  let result = 0;

  for (const word of words) {
    if (word.length <= s.length && s.startsWith(word)) {
      result++;
    }
  }

  return result;
};
```

2256 Minimum Average Difference

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2256. Minimum Average Difference
 * https://leetcode.com/problems/minimum-average-difference/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of length n.
 *
 * The average difference of the index i is the absolute difference between the average of the
 * first i + 1 elements of nums and the average of the last n - i - 1 elements. Both averages
 * should be rounded down to the nearest integer.
 *
 * Return the index with the minimum average difference. If there are multiple such indices,
 * return the smallest one.
 *
 * Note:
 * - The absolute difference of two numbers is the absolute value of their difference.
 * - The average of n elements is the sum of the n elements divided (integer division) by n.
 * - The average of 0 elements is considered to be 0.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumAverageDifference = function(nums) {
  const n = nums.length;
```

```
  let leftSum = 0;
  let rightSum = nums.reduce((sum, num) => sum + num, 0);
  let minDiff = Infinity;
  let result = 0;

  for (let i = 0; i < n; i++) {
    leftSum += nums[i];
    rightSum -= nums[i];
    const leftAvg = Math.floor(leftSum / (i + 1));
    const rightAvg = i === n - 1 ? 0 : Math.floor(rightSum / (n - i - 1));
    const diff = Math.abs(leftAvg - rightAvg);

    if (diff < minDiff) {
      minDiff = diff;
      result = i;
    }
  }

  return result;
};
```

2257 Count Unguarded Cells In The Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2257. Count Unguarded Cells in the Grid
 * https://leetcode.com/problems/count-unguarded-cells-in-the-grid/
 * Difficulty: Medium
 *
 * You are given two integers m and n representing a 0-indexed m x n grid. You are also given
two
 * 2D integer arrays guards and walls where guards[i] = [rowi, coli] and walls[j] = [rowj,
colj]
 * represent the positions of the ith guard and jth wall respectively.
 *
 * A guard can see every cell in the four cardinal directions (north, east, south, or west)
starting
 * from their position unless obstructed by a wall or another guard. A cell is guarded if
there is
 * at least one guard that can see it.
 *
 * Return the number of unoccupied cells that are not guarded.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number[][]} guards
 * @param {number[][]} walls
 * @return {number}
 */
var countUnguarded = function(m, n, guards, walls) {
  const grid = Array.from({ length: m }, () => Array(n).fill(0));

  for (const [row, col] of walls) {
    grid[row][col] = 1;
  }
  for (const [row, col] of guards) {
    grid[row][col] = 2;
  }

  for (const [row, col] of guards) {
    for (let i = row - 1; i >= 0 && grid[i][col] !== 1 && grid[i][col] !== 2; i--) {
      grid[i][col] = 3;
    }
    for (let i = row + 1; i < m && grid[i][col] !== 1 && grid[i][col] !== 2; i++) {
      grid[i][col] = 3;
    }
    for (let j = col - 1; j >= 0 && grid[row][j] !== 1 && grid[row][j] !== 2; j--) {
```

```
      grid[row][j] = 3;
    }
    for (let j = col + 1; j < n && grid[row][j] !== 1 && grid[row][j] !== 2; j++) {
      grid[row][j] = 3;
    }
  }

  let result = 0;
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      if (grid[i][j] === 0) {
        result++;
      }
    }
  }

  return result;
};
```

2258 Escape The Spreading Fire

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2258. Escape the Spreading Fire
 * https://leetcode.com/problems/escape-the-spreading-fire/
 * Difficulty: Hard
 *
 * You are given a 0-indexed 2D integer array grid of size m x n which represents a field.
 * Each cell has one of three values:
 * - 0 represents grass,
 * - 1 represents fire,
 * - 2 represents a wall that you and fire cannot pass through.
 *
 * You are situated in the top-left cell, (0, 0), and you want to travel to the safehouse at
the
 * bottom-right cell, (m - 1, n - 1). Every minute, you may move to an adjacent grass cell.
After
 * your move, every fire cell will spread to all adjacent cells that are not walls.
 *
 * Return the maximum number of minutes that you can stay in your initial position before
moving
 * while still safely reaching the safehouse. If this is impossible, return -1. If you can
always
 * reach the safehouse regardless of the minutes stayed, return 109.
 *
 * Note that even if the fire spreads to the safehouse immediately after you have reached it,
it
 * will be counted as safely reaching the safehouse.
 *
 * A cell is adjacent to another cell if the former is directly north, east, south, or west of
 the
 * latter (i.e., their sides are touching).
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maximumMinutes = function(grid) {
  const m = grid.length;
  const n = grid[0].length;
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];
  const MAX_ANSWER = 1000000000;

  function isValid(x, y) {
    return x >= 0 && x < m && y >= 0 && y < n && grid[x][y] === 0;
  }

  function calculateFireTime() {
```

```
  const fireTime = new Array(m).fill().map(() => new Array(n).fill(Infinity));
  const queue = [];

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      if (grid[i][j] === 1) {
        queue.push([i, j, 0]);
        fireTime[i][j] = 0;
      }
    }
  }

  while (queue.length > 0) {
    const [x, y, time] = queue.shift();

    for (const [dx, dy] of directions) {
      const nx = x + dx;
      const ny = y + dy;

      if (isValid(nx, ny) && fireTime[nx][ny] === Infinity) {
        fireTime[nx][ny] = time + 1;
        queue.push([nx, ny, time + 1]);
      }
    }
  }

  return fireTime;
}

function canReachSafehouse(delay) {
  const fireTime = calculateFireTime();
  const visited = new Array(m).fill().map(() => new Array(n).fill(false));
  const queue = [[0, 0, delay]];
  visited[0][0] = true;

  while (queue.length > 0) {
    const [x, y, time] = queue.shift();

    for (const [dx, dy] of directions) {
      const nx = x + dx;
      const ny = y + dy;

      if (!isValid(nx, ny) || visited[nx][ny]) continue;

      if (nx === m - 1 && ny === n - 1) {
        if (time + 1 <= fireTime[nx][ny] || fireTime[nx][ny] === Infinity) {
          return true;
        }
      }

      if (time + 1 < fireTime[nx][ny]) {
        visited[nx][ny] = true;
        queue.push([nx, ny, time + 1]);
      }
    }
  }

  return false;
}

let left = 0;
let right = MAX_ANSWER;
let result = -1;

while (left <= right) {
  const mid = Math.floor((left + right) / 2);

  if (canReachSafehouse(mid)) {
    result = mid;
    left = mid + 1;
  } else {
    right = mid - 1;
  }
}
```

```
  return result;
};
```

2259 Remove Digit From Number To Maximize Result

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2259. Remove Digit From Number to Maximize Result
 * https://leetcode.com/problems/remove-digit-from-number-to-maximize-result/
 * Difficulty: Easy
 *
 * You are given a string number representing a positive integer and a character digit.
 *
 * Return the resulting string after removing exactly one occurrence of digit from number such
 that
 * the value of the resulting string in decimal form is maximized. The test cases are
generated such
 * that digit occurs at least once in number.
 */

/**
 * @param {string} number
 * @param {character} digit
 * @return {string}
 */
var removeDigit = function(number, digit) {
  let result = '';

  for (let i = 0; i < number.length; i++) {
    if (number[i] === digit) {
      const candidate = number.slice(0, i) + number.slice(i + 1);
      if (candidate > result) {
        result = candidate;
      }
    }
  }

  return result;
};
```

2260 Minimum Consecutive Cards To Pick Up

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2260. Minimum Consecutive Cards to Pick Up
 * https://leetcode.com/problems/minimum-consecutive-cards-to-pick-up/
 * Difficulty: Medium
 *
 * You are given an integer array cards where cards[i] represents the value of the ith card. A
 pair
 * of cards are matching if the cards have the same value.
 *
 * Return the minimum number of consecutive cards you have to pick up to have a pair of
matching
 * cards among the picked cards. If it is impossible to have matching cards, return -1.
 */

/**
 * @param {number[]} cards
 * @return {number}
 */
var minimumCardPickup = function(cards) {
  const map = new Map();
```

```
  let minLength = Infinity;

  for (let i = 0; i < cards.length; i++) {
    if (map.has(cards[i])) {
      minLength = Math.min(minLength, i - map.get(cards[i]) + 1);
    }
    map.set(cards[i], i);
  }

  return minLength === Infinity ? -1 : minLength;
};
```

2261 K Divisible Elements Subarrays

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2261. K Divisible Elements Subarrays
 * https://leetcode.com/problems/k-divisible-elements-subarrays/
 * Difficulty: Medium
 *
 * Given an integer array nums and two integers k and p, return the number of distinct
subarrays,
 * which have at most k elements that are divisible by p.
 *
 * Two arrays nums1 and nums2 are said to be distinct if:
 * - They are of different lengths, or
 * - There exists at least one index i where nums1[i] != nums2[i].
 *
 * A subarray is defined as a non-empty contiguous sequence of elements in an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @param {number} p
 * @return {number}
 */
var countDistinct = function(nums, k, p) {
  const n = nums.length;
  const seen = new Set();

  for (let start = 0; start < n; start++) {
    const subarray = [];
    let divisibleCount = 0;

    for (let end = start; end < n; end++) {
      subarray.push(nums[end]);
      if (nums[end] % p === 0) divisibleCount++;

      if (divisibleCount <= k) {
        seen.add(subarray.join(','));
      }
    }
  }

  return seen.size;
};
```

2262 Total Appeal Of A String

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2262. Total Appeal of A String
```

```
 * https://leetcode.com/problems/total-appeal-of-a-string/
 * Difficulty: Hard
 *
 * The appeal of a string is the number of distinct characters found in the string.
 * - For example, the appeal of "abbca" is 3 because it has 3 distinct characters:
 *   'a', 'b', and 'c'.
 *
 * Given a string s, return the total appeal of all of its substrings.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var appealSum = function(s) {
  const n = s.length;
  const lastSeen = new Array(26).fill(-1);
  let result = 0;

  for (let i = 0; i < n; i++) {
    const charIndex = s.charCodeAt(i) - 97;
    result += (i - lastSeen[charIndex]) * (n - i);
    lastSeen[charIndex] = i;
  }

  return result;
};
```

2263 Make Array Non Decreasing Or Non Increasing

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2263. Make Array Non-decreasing or Non-increasing
 * https://leetcode.com/problems/make-array-non-decreasing-or-non-increasing/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums. In one operation, you can:
 * - Choose an index i in the range 0 <= i < nums.length
 * - Set nums[i] to nums[i] + 1 or nums[i] - 1
 *
 * Return the minimum number of operations to make nums non-decreasing or non-increasing.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var convertArray = function(nums) {
  const levels = [...new Set(nums)].sort((a, b) => a - b);
  const reversedNums = [...nums].reverse();

  return Math.min(helper(nums, levels), helper(reversedNums, levels));

  function helper(arr, levels) {
    const dp = new Map();
    for (const level of levels) {
      dp.set(level, 0);
    }

    for (const num of arr) {
      let currentResult = Infinity;
      const newDp = new Map();

      for (const level of levels) {
        currentResult = Math.min(currentResult, dp.get(level) + Math.abs(num - level));
        newDp.set(level, currentResult);
      }
```

```
      for (const [level, value] of newDp) {
        dp.set(level, value);
      }
    }

    return dp.get(Math.max(...levels));
  }
};
```

2264 Largest 3 Same Digit Number In String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2264. Largest 3-Same-Digit Number in String
 * https://leetcode.com/problems/largest-3-same-digit-number-in-string/
 * Difficulty: Easy
 *
 * You are given a string num representing a large integer. An integer is good if it meets
 * the following conditions:
 * - It is a substring of num with length 3.
 * - It consists of only one unique digit.
 *
 * Return the maximum good integer as a string or an empty string "" if no such integer
exists.
 *
 * Note:
 * - A substring is a contiguous sequence of characters within a string.
 * - There may be leading zeroes in num or a good integer.
 */

/**
 * @param {string} num
 * @return {string}
 */
var largestGoodInteger = function(num) {
  let result = '';

  for (let i = 0; i <= num.length - 3; i++) {
    const substring = num.slice(i, i + 3);
    if (substring[0] === substring[1] && substring[1] === substring[2]) {
      if (substring > result) {
        result = substring;
      }
    }
  }

  return result;
};
```

2265 Count Nodes Equal To Average Of Subtree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2265. Count Nodes Equal to Average of Subtree
 * https://leetcode.com/problems/count-nodes-equal-to-average-of-subtree/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, return the number of nodes where the value of the node is
 * equal to the average of the values in its subtree.
 *
 * Note:
 * - The average of n elements is the sum of the n elements divided by n and rounded down to
```

```
 *   the nearest integer.
 * - A subtree of root is a tree consisting of root and all of its descendants.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var averageOfSubtree = function(root) {
  let count = 0;
  traverse(root);
  return count;

  function traverse(node) {
    if (!node) return [0, 0];

    const [leftSum, leftCount] = traverse(node.left);
    const [rightSum, rightCount] = traverse(node.right);
    const totalSum = leftSum + rightSum + node.val;
    const totalCount = leftCount + rightCount + 1;

    if (Math.floor(totalSum / totalCount) === node.val) {
      count++;
    }

    return [totalSum, totalCount];
  }
};
```

2266 Count Number Of Texts

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2266. Count Number of Texts
 * https://leetcode.com/problems/count-number-of-texts/
 * Difficulty: Medium
 *
 * Alice is texting Bob using her phone. The mapping of digits to letters is shown in the
 * figure below.
 *
 * In order to add a letter, Alice has to press the key of the corresponding digit i times,
 * where i is the position of the letter in the key.
 * - For example, to add the letter 's', Alice has to press '7' four times. Similarly, to
 *   add the letter 'k', Alice has to press '5' twice.
 * - Note that the digits '0' and '1' do not map to any letters, so Alice does not use them.
 *
 * However, due to an error in transmission, Bob did not receive Alice's text message but
 * received a string of pressed keys instead.
 * - For example, when Alice sent the message "bob", Bob received the string "2266622".
 *
 * Given a string pressedKeys representing the string received by Bob, return the total
 * number of possible text messages Alice could have sent.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {string} pressedKeys
 * @return {number}
 */
var countTexts = function(pressedKeys) {
```

```
  const mod = 1e9 + 7;
  const maxGroup = pressedKeys.length;
  const dp = new Array(maxGroup + 1).fill(0);
  dp[0] = 1;

  const map = new Map([
    ['2', 3], ['3', 3], ['4', 3], ['5', 3],
    ['6', 3], ['7', 4], ['8', 3], ['9', 4]
  ]);

  for (let i = 1; i <= maxGroup; i++) {
    dp[i] = dp[i - 1];
    if (i >= 2 && pressedKeys[i - 1] === pressedKeys[i - 2]) {
      dp[i] = (dp[i] + dp[i - 2]) % mod;
    }
    if (i >= 3 && pressedKeys[i - 1] === pressedKeys[i - 2]
        && pressedKeys[i - 2] === pressedKeys[i - 3]) {
      dp[i] = (dp[i] + dp[i - 3]) % mod;
    }
    if (i >= 4 && pressedKeys[i - 1] === pressedKeys[i - 2]
        && pressedKeys[i - 2] === pressedKeys[i - 3]
        && pressedKeys[i - 3] === pressedKeys[i - 4] && map.get(pressedKeys[i - 1]) === 4) {
      dp[i] = (dp[i] + dp[i - 4]) % mod;
    }
  }

  return dp[maxGroup];
};
```

2267 Check If There Is A Valid Parentheses String Path

HARD

APPROACH NOTES

Uses a Set for unique membership. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2267. Check if There Is a Valid Parentheses String Path
 * https://leetcode.com/problems/check-if-there-is-a-valid-parentheses-string-path/
 * Difficulty: Hard
 *
 * A parentheses string is a non-empty string consisting only of '(' and ')'. It is valid if
any of
 * the following conditions is true:
 * - It is ().
 * - It can be written as AB (A concatenated with B), where A and B are valid parentheses
strings.
 * - It can be written as (A), where A is a valid parentheses string.
 *
 * You are given an m x n matrix of parentheses grid. A valid parentheses string path in the
grid
 * is a path satisfying all of the following conditions:
 * - The path starts from the upper left cell (0, 0).
 * - The path ends at the bottom-right cell (m - 1, n - 1).
 * - The path only ever moves down or right.
 * - The resulting parentheses string formed by the path is valid.
 *
 * Return true if there exists a valid parentheses string path in the grid. Otherwise, return
false.
 */

/**
* @param {character[][]} grid
* @return {boolean}
*/
var hasValidPath = function(grid) {
  const m = grid.length;
  const n = grid[0].length;

  if ((m + n - 1) % 2 !== 0) {
    return false;
  }
```

```
  if (grid[0][0] === ')' || grid[m - 1][n - 1] === '(') {
    return false;
  }

  const visited = new Set();
  return dfs(0, 0, 0);

  function dfs(i, j, openCount) {
    if (i >= m || j >= n || openCount < 0) {
      return false;
    }
    openCount += grid[i][j] === '(' ? 1 : -1;
    if (i === m - 1 && j === n - 1) {
      return openCount === 0;
    }
    const key = `${i},${j},${openCount}`;
    if (visited.has(key)) {
      return false;
    }
    visited.add(key);

    return dfs(i + 1, j, openCount) || dfs(i, j + 1, openCount);
  }
};
```

2268 Minimum Number Of Keypresses

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2268. Minimum Number of Keypresses
 * https://leetcode.com/problems/minimum-number-of-keypresses/
 * Difficulty: Medium
 *
 * You have a keypad with 9 buttons, numbered from 1 to 9, each mapped to lowercase English
 * letters. You can choose which characters each button is matched to as long as:
 * - All 26 lowercase English letters are mapped to.
 * - Each character is mapped to by exactly 1 button.
 * - Each button maps to at most 3 characters.
 *
 * To type the first character matched to a button, you press the button once. To type the
 * second character, you press the button twice, and so on.
 *
 * Given a string s, return the minimum number of keypresses needed to type s using your
keypad.
 *
 * Note that the characters mapped to by each button, and the order they are mapped in cannot
 * be changed.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumKeypresses = function(s) {
  const frequency = new Map();
  for (const char of s) {
    frequency.set(char, (frequency.get(char) || 0) + 1);
  }

  const frequencies = Array.from(frequency.values()).sort((a, b) => b - a);

  let keypresses = 0;
  for (let i = 0; i < frequencies.length; i++) {
    const pressesPerChar = Math.floor(i / 9) + 1;
    keypresses += frequencies[i] * pressesPerChar;
  }

  return keypresses;
```

```
};
```

2269 Find The K Beauty Of A Number

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2269. Find the K-Beauty of a Number
 * https://leetcode.com/problems/find-the-k-beauty-of-a-number/
 * Difficulty: Easy
 *
 * The k-beauty of an integer num is defined as the number of substrings of num when it is
 * read as a string that meet the following conditions:
 * - It has a length of k.
 * - It is a divisor of num.
 *
 * Given integers num and k, return the k-beauty of num.
 *
 * Note:
 * - Leading zeros are allowed.
 * - 0 is not a divisor of any value.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {number} num
 * @param {number} k
 * @return {number}
 */
var divisorSubstrings = function(num, k) {
  const numStr = num.toString();
  let result = 0;

  for (let i = 0; i <= numStr.length - k; i++) {
    const substring = parseInt(numStr.slice(i, i + k));
    if (substring !== 0 && num % substring === 0) {
      result++;
    }
  }

  return result;
};
```

2270 Number Of Ways To Split Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2270. Number of Ways to Split Array
 * https://leetcode.com/problems/number-of-ways-to-split-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of length n.
 *
 * nums contains a valid split at index i if the following are true:
 * - The sum of the first i + 1 elements is greater than or equal to the sum of the
 *   last n - i - 1 elements.
 * - There is at least one element to the right of i. That is, 0 <= i < n - 1.
 *
 * Return the number of valid splits in nums.
 */

/**
 * @param {number[]} nums
```

```
 * @return {number}
 */
var waysToSplitArray = function(nums) {
  const total = nums.reduce((a, b) => a + b);
  let result = 0;

  for (let i = 0, sum = 0; i < nums.length - 1; i++) {
    sum += nums[i];
    if (sum >= total - sum) {
      result++;
    }
  }

  return result;
};
```

2271 Maximum White Tiles Covered By A Carpet

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2271. Maximum White Tiles Covered by a Carpet
 * https://leetcode.com/problems/maximum-white-tiles-covered-by-a-carpet/
 * Difficulty: Medium
 *
 * You are given a 2D integer array tiles where tiles[i] = [li, ri] represents that every tile
 * j in the range li <= j <= ri is colored white.
 *
 * You are also given an integer carpetLen, the length of a single carpet that can be placed
 * anywhere.
 *
 * Return the maximum number of white tiles that can be covered by the carpet.
 */

/**
 * @param {number[][]} tiles
 * @param {number} carpetLen
 * @return {number}
 */
var maximumWhiteTiles = function(tiles, carpetLen) {
  tiles.sort((a, b) => a[0] - b[0]);

  let result = 0;
  let covered = 0;
  let j = 0;

  for (let i = 0; i < tiles.length; i++) {
    const carpetEnd = tiles[i][0] + carpetLen - 1;

    while (j < tiles.length && tiles[j][0] <= carpetEnd) {
      if (tiles[j][1] <= carpetEnd) {
        covered += tiles[j][1] - tiles[j][0] + 1;
        j++;
      } else {
        covered += carpetEnd - tiles[j][0] + 1;
        result = Math.max(result, covered);
        covered -= carpetEnd - tiles[j][0] + 1;
        break;
      }
    }

    result = Math.max(result, covered);

    if (j === tiles.length) break;

    covered -= tiles[i][1] - tiles[i][0] + 1;
  }

  return result;
```

```
};
```

2272 Substring With Largest Variance

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2272. Substring With Largest Variance
 * https://leetcode.com/problems/substring-with-largest-variance/
 * Difficulty: Hard
 *
 * The variance of a string is defined as the largest difference between the number of
occurrences
 * of any 2 characters present in the string. Note the two characters may or may not be the
same.
 *
 * Given a string s consisting of lowercase English letters only, return the largest variance
 * possible among all substrings of s.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var largestVariance = function(s) {
  const map = new Set(s);
  let result = 0;

  for (const charA of map) {
    for (const charB of map) {
      if (charA === charB) continue;

      let countA = 0;
      let countB = 0;
      let hasA = false;
      let hasB = false;

      for (const char of s) {
        if (char === charA) {
          countA++;
          hasA = true;
        }
        if (char === charB) {
          countB++;
          hasB = true;
        }

        if (hasA && hasB) {
          result = Math.max(result, Math.abs(countA - countB));
        }

        if (countA < countB) {
          countA = 0;
          countB = 0;
          hasA = false;
          hasB = false;
        }
      }

      countA = 0;
      countB = 0;
      hasA = false;
      hasB = false;

      for (let i = s.length - 1; i >= 0; i--) {
        if (s[i] === charA) {
          countA++;
          hasA = true;
```

```
        }
        if (s[i] === charB) {
          countB++;
          hasB = true;
        }

        if (hasA && hasB) {
          result = Math.max(result, Math.abs(countA - countB));
        }

        if (countA < countB) {
          countA = 0;
          countB = 0;
          hasA = false;
          hasB = false;
        }
      }
    }
  }

  return result;
};
```

2273 Find Resultant Array After Removing Anagrams

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2273. Find Resultant Array After Removing Anagrams
 * https://leetcode.com/problems/find-resultant-array-after-removing-anagrams/
 * Difficulty: Easy
 *
 * You are given a 0-indexed string array words, where words[i] consists of lowercase
 * English letters.
 *
 * In one operation, select any index i such that 0 < i < words.length and words[i - 1] and
 * words[i] are anagrams, and delete words[i] from words. Keep performing this operation as
 * long as you can select an index that satisfies the conditions.
 *
 * Return words after performing all operations. It can be shown that selecting the indices
 * for each operation in any arbitrary order will lead to the same result.
 *
 * An Anagram is a word or phrase formed by rearranging the letters of a different word or
 * phrase using all the original letters exactly once. For example, "dacb" is an anagram of
 * "abdc".
 */

/**
 * @param {string[]} words
 * @return {string[]}
 */
var removeAnagrams = function(words) {
  const result = [words[0]];

  for (let i = 1; i < words.length; i++) {
    const prevSorted = result[result.length - 1].split('').sort().join('');
    const currSorted = words[i].split('').sort().join('');

    if (prevSorted !== currSorted) {
      result.push(words[i]);
    }
  }

  return result;
};
```

2274 Maximum Consecutive Floors Without Special Floors

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2274. Maximum Consecutive Floors Without Special Floors
 * https://leetcode.com/problems/maximum-consecutive-floors-without-special-floors/
 * Difficulty: Medium
 *
 * Alice manages a company and has rented some floors of a building as office space. Alice has
 * decided some of these floors should be special floors, used for relaxation only.
 *
 * You are given two integers bottom and top, which denote that Alice has rented all the
floors
 * from bottom to top (inclusive). You are also given the integer array special, where
special[i]
 * denotes a special floor that Alice has designated for relaxation.
 *
 * Return the maximum number of consecutive floors without a special floor.
 */

/**
 * @param {number} bottom
 * @param {number} top
 * @param {number[]} special
 * @return {number}
 */
var maxConsecutive = function(bottom, top, special) {
  special.sort((a, b) => a - b);
  let result = Math.max(special[0] - bottom, top - special[special.length - 1]);

  for (let i = 1; i < special.length; i++) {
    result = Math.max(result, special[i] - special[i - 1] - 1);
  }

  return result;
};
```

2275 Largest Combination With Bitwise And Greater Than Zero

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2275. Largest Combination With Bitwise AND Greater Than Zero
 * https://leetcode.com/problems/largest-combination-with-bitwise-and-greater-than-zero/
 * Difficulty: Medium
 *
 * The bitwise AND of an array nums is the bitwise AND of all integers in nums.
 * - For example, for nums = [1, 5, 3], the bitwise AND is equal to 1 & 5 & 3 = 1.
 * - Also, for nums = [7], the bitwise AND is 7.
 *
 * You are given an array of positive integers candidates. Compute the bitwise AND for all
 * possible combinations of elements in the candidates array.
 *
 * Return the size of the largest combination of candidates with a bitwise AND greater than 0.
 */

/**
 * @param {number[]} candidates
 * @return {number}
 */
var largestCombination = function(candidates) {
  const bitCounts = new Array(32).fill(0);
  let result = 0;
```

```
  for (const num of candidates) {
    for (let i = 0; i < 32; i++) {
      if (num & (1 << i)) {
        bitCounts[i]++;
      }
    }
  }

  for (const count of bitCounts) {
    result = Math.max(result, count);
  }

  return result;
};
```

2276 Count Integers In Intervals

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2276. Count Integers in Intervals
 * https://leetcode.com/problems/count-integers-in-intervals/
 * Difficulty: Hard
 *
 * Given an empty set of intervals, implement a data structure that can:
 * - Add an interval to the set of intervals.
 * - Count the number of integers that are present in at least one interval.
 *
 * Implement the CountIntervals class:
 * - CountIntervals() Initializes the object with an empty set of intervals.
 * - void add(int left, int right) Adds the interval [left, right] to the set of intervals.
 * - int count() Returns the number of integers that are present in at least one interval.
 *
 * Note that an interval [left, right] denotes all the integers x where left <= x <= right.
 */

var CountIntervals = function() {
  this.coverage = 0;
  this.intervals = [[-Infinity, -Infinity], [Infinity, Infinity]];
};

/**
 * @param {number} left
 * @param {number} right
 * @return {void}
 */
CountIntervals.prototype.add = function(left, right) {
  function bisectLeft(arr, target) {
    let low = 0;
    let high = arr.length;
    while (low < high) {
      const mid = Math.floor((low + high) / 2);
      if (arr[mid][0] < target) {
        low = mid + 1;
      } else {
        high = mid;
      }
    }
    return low;
  }

  function bisectRight(arr, target) {
    let low = 0;
    let high = arr.length;
    while (low < high) {
      const mid = Math.floor((low + high) / 2);
      if (arr[mid][0] <= target) {
        low = mid + 1;
      } else {
        high = mid;
```

```
      }
    }
    return low;
  }

  let li = bisectLeft(this.intervals, left - 1);
  if (this.intervals[li - 1][1] >= left - 1) {
    li -= 1;
  }

  const lval = Math.min(this.intervals[li][0], left);
  const ri = bisectRight(this.intervals, right + 1);
  const rval = Math.max(this.intervals[ri - 1][1], right);

  let toDelete = 0;
  for (let i = li; i < ri; i++) {
    toDelete += this.intervals[i][1] - this.intervals[i][0] + 1;
  }

  this.coverage += rval - lval + 1 - toDelete;
  this.intervals.splice(li, ri - li, [lval, rval]);
};

/**
 * @return {number}
 */
CountIntervals.prototype.count = function() {
  return this.coverage;
};
```

2277 Closest Node To Path In Tree

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2277. Closest Node to Path in Tree
 * https://leetcode.com/problems/closest-node-to-path-in-tree/
 * Difficulty: Hard
 *
 * You are given a positive integer n representing the number of nodes in a tree, numbered
 * from 0 to n - 1 (inclusive). You are also given a 2D integer array edges of length n - 1,
 * where edges[i] = [node1i, node2i] denotes that there is a bidirectional edge connecting
 * node1i and node2i in the tree.
 *
 * You are given a 0-indexed integer array query of length m where query[i] = [starti, endi,
nodei]
 * means that for the ith query, you are tasked with finding the node on the path from starti
to
 * endi that is closest to nodei.
 *
 * Return an integer array answer of length m, where answer[i] is the answer to the ith query.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[][]} query
 * @return {number[]}
 */
var closestNode = function(n, edges, query) {
  const graph = new Array(n).fill().map(() => []);
  for (const [a, b] of edges) {
    graph[a].push(b);
    graph[b].push(a);
  }

  const depths = new Array(n).fill(0);
  const binaryParents = new Array(n).fill().map(() => new Array(16).fill(0));
```

```
  dfs(0, [], 0);

  return query.map(([a, b, q]) => {
    const candidates = [lca(a, b), lca(a, q), lca(b, q)];
    return candidates.reduce((deepest, current) =>
      depths[current] > depths[deepest] ? current : deepest
    );
  });

  function dfs(node, parents, depth) {
    depths[node] = depth;
    for (let bit = 0; bit < 16; bit++) {
      if ((1 << bit) <= parents.length) {
        binaryParents[node][bit] = parents[parents.length - (1 << bit)];
      }
    }
    parents.push(node);
    for (const next of graph[node]) {
      if (parents.length >= 2 && next === parents[parents.length - 2]) {
        continue;
      }
      dfs(next, parents, depth + 1);
    }
    parents.pop();
  }

  function getKthParent(node, k) {
    if (!k) return node;
    let bit = 0;
    while ((1 << (bit + 1)) <= k) bit++;
    return getKthParent(binaryParents[node][bit], k & (~(1 << bit)));
  }

  function lca(a, b) {
    if (depths[a] > depths[b]) return lca(b, a);
    b = getKthParent(b, depths[b] - depths[a]);
    if (a === b) return a;
    for (let i = binaryParents[a].length - 1; i >= 0; i--) {
      if (binaryParents[a][i] !== binaryParents[b][i]) {
        a = binaryParents[a][i];
        b = binaryParents[b][i];
      }
    }
    return binaryParents[a][0];
  }
};
```

2278 Percentage Of Letter In String

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2278. Percentage of Letter in String
 * https://leetcode.com/problems/percentage-of-letter-in-string/
 * Difficulty: Easy
 *
 * Given a string s and a character letter, return the percentage of characters in s that
equal
 * letter rounded down to the nearest whole percent.
 */

/**
 * @param {string} s
 * @param {character} letter
 * @return {number}
 */
var percentageLetter = function(s, letter) {
  return Math.floor((s.split('').filter(char => char === letter).length / s.length) * 100);
};
```

2279 Maximum Bags With Full Capacity Of Rocks

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2279. Maximum Bags With Full Capacity of Rocks
 * https://leetcode.com/problems/maximum-bags-with-full-capacity-of-rocks/
 * Difficulty: Medium
 *
 * You have n bags numbered from 0 to n - 1. You are given two 0-indexed integer arrays
capacity
 * and rocks. The ith bag can hold a maximum of capacity[i] rocks and currently contains
rocks[i]
 * rocks. You are also given an integer additionalRocks, the number of additional rocks you
can
 * place in any of the bags.
 *
 * Return the maximum number of bags that could have full capacity after placing the
additional
 * rocks in some bags.
 */

/**
 * @param {number[]} capacity
 * @param {number[]} rocks
 * @param {number} additionalRocks
 * @return {number}
 */
var maximumBags = function(capacity, rocks, additionalRocks) {
  const remaining = capacity.map((cap, i) => cap - rocks[i]);
  remaining.sort((a, b) => a - b);

  let result = 0;
  let rocksLeft = additionalRocks;

  for (const needed of remaining) {
    if (needed <= rocksLeft) {
      result++;
      rocksLeft -= needed;
    } else {
      break;
    }
  }

  return result;
};
```

2280 Minimum Lines To Represent A Line Chart

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2280. Minimum Lines to Represent a Line Chart
 * https://leetcode.com/problems/minimum-lines-to-represent-a-line-chart/
 * Difficulty: Medium
 *
 * You are given a 2D integer array stockPrices where stockPrices[i] = [dayi, pricei]
indicates
 * the price of the stock on day dayi is pricei. A line chart is created from the array by
 * plotting the points on an XY plane with the X-axis representing the day and the Y-axis
 * representing the price and connecting adjacent points. One such example is shown below.
 *
 * Return the minimum number of lines needed to represent the line chart.
 */
```

```
/**
 * @param {number[][]} stockPrices
 * @return {number}
 */
var minimumLines = function(stockPrices) {
  if (stockPrices.length <= 2) return stockPrices.length - 1;

  stockPrices.sort((a, b) => a[0] - b[0]);

  let lines = 1;
  for (let i = 2; i < stockPrices.length; i++) {
    const [x0, y0] = stockPrices[i - 2];
    const [x1, y1] = stockPrices[i - 1];
    const [x2, y2] = stockPrices[i];

    const dx1 = BigInt(x1 - x0);
    const dy1 = BigInt(y1 - y0);
    const dx2 = BigInt(x2 - x1);
    const dy2 = BigInt(y2 - y1);

    if (dy1 * dx2 !== dy2 * dx1) lines++;
  }

  return lines;
};
```

2281 Sum Of Total Strength Of Wizards

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2281. Sum of Total Strength of Wizards
 * https://leetcode.com/problems/sum-of-total-strength-of-wizards/
 * Difficulty: Hard
 *
 * As the ruler of a kingdom, you have an army of wizards at your command.
 *
 * You are given a 0-indexed integer array strength, where strength[i] denotes the strength
 * of the ith wizard. For a contiguous group of wizards (i.e. the wizards' strengths form
 * a subarray of strength), the total strength is defined as the product of the following
 * two values:
 * - The strength of the weakest wizard in the group.
 * - The total of all the individual strengths of the wizards in the group.
 *
 * Return the sum of the total strengths of all contiguous groups of wizards. Since the answer
 * may be very large, return it modulo 109 + 7.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} strength
 * @return {number}
 */
var totalStrength = function(strength) {
  const n = strength.length;
  const mod = BigInt(1e9 + 7);
  const left = new Array(n).fill(-1);
  const right = new Array(n).fill(n);

  const stack = [];

  for (let i = 0; i < n; ++i) {
    while (stack.length > 0 && strength[stack[stack.length - 1]] >= strength[i]) {
      stack.pop();
    }
    if (stack.length > 0) {
      left[i] = stack[stack.length - 1];
    }
    stack.push(i);
```

```
  }

  stack.length = 0;

  for (let i = n - 1; i >= 0; --i) {
    while (stack.length > 0 && strength[stack[stack.length - 1]] > strength[i]) {
      stack.pop();
    }
    if (stack.length > 0) {
      right[i] = stack[stack.length - 1];
    }
    stack.push(i);
  }

  const prefixSum = new Array(n + 1).fill(0n);
  for (let i = 0; i < n; ++i) {
    prefixSum[i + 1] = (prefixSum[i] + BigInt(strength[i])) % mod;
  }

  const prefixSumOfPrefixSum = new Array(n + 2).fill(0n);
  for (let i = 0; i < n + 1; ++i) {
    prefixSumOfPrefixSum[i + 1] = (prefixSumOfPrefixSum[i] + prefixSum[i]) % mod;
  }

  let result = 0n;
  for (let i = 0; i < n; ++i) {
    const value = BigInt(strength[i]);
    const l = left[i] + 1;
    const r = right[i] - 1;
    const leftSum = (prefixSumOfPrefixSum[i + 1] - prefixSumOfPrefixSum[l] + mod) % mod;
    const rightSum = (prefixSumOfPrefixSum[r + 2] - prefixSumOfPrefixSum[i + 1] + mod) % mod;
    const leftCount = BigInt(i - l + 1);
    const rightCount = BigInt(r - i + 1);
    const leftPart = (rightSum * leftCount) % mod;
    const rightPart = (leftSum * rightCount) % mod;
    const contribution = (value * ((leftPart - rightPart + mod) % mod)) % mod;

    result = (result + contribution) % mod;
  }

  return Number(result);
};
```

2282 Number Of People That Can Be Seen In A Grid

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2282. Number of People That Can Be Seen in a Grid
 * https://leetcode.com/problems/number-of-people-that-can-be-seen-in-a-grid/
 * Difficulty: Medium
 *
 * You are given an m x n 0-indexed 2D array of positive integers heights where heights[i][j]
 * is the height of the person standing at position (i, j).
 *
 * A person standing at position (row1, col1) can see a person standing at position (row2,
col2) if:
 * - The person at (row2, col2) is to the right or below the person at (row1, col1). More
formally,
 *   this means that either row1 == row2 and col1 < col2 or row1 < row2 and col1 == col2.
 * - Everyone in between them is shorter than both of them.
 *
 * Return an m x n 2D array of integers answer where answer[i][j] is the number of people that
 the
 * person at position (i, j) can see.
 */

/**
 * @param {number[][]} heights
 * @return {number[][]}
```

```
 */
var seePeople = function(heights) {
  const rows = heights.length;
  const cols = heights[0].length;
  const result = new Array(rows).fill().map(() => new Array(cols).fill(0));

  for (let i = 0; i < rows; i++) {
    const stack = [];
    for (let j = cols - 1; j >= 0; j--) {
      let equal = false;
      while (stack.length > 0 && stack[stack.length - 1] <= heights[i][j]) {
        if (stack[stack.length - 1] === heights[i][j]) {
          equal = true;
        }
        stack.pop();
        result[i][j]++;
      }
      if (stack.length > 0 && !equal) {
        result[i][j]++;
      }
      stack.push(heights[i][j]);
    }
  }

  for (let j = 0; j < cols; j++) {
    const stack = [];
    for (let i = rows - 1; i >= 0; i--) {
      let equal = false;
      while (stack.length > 0 && stack[stack.length - 1] <= heights[i][j]) {
        if (stack[stack.length - 1] === heights[i][j]) {
          equal = true;
        }
        stack.pop();
        result[i][j]++;
      }
      if (stack.length > 0 && !equal) {
        result[i][j]++;
      }
      stack.push(heights[i][j]);
    }
  }

  return result;
};
```

2283 Check If Number Has Equal Digit Count And Digit Value

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2283. Check if Number Has Equal Digit Count and Digit Value
 * https://leetcode.com/problems/check-if-number-has-equal-digit-count-and-digit-value/
 * Difficulty: Easy
 *
 * You are given a 0-indexed string num of length n consisting of digits.
 *
 * Return true if for every index i in the range 0 <= i < n, the digit i occurs num[i] times
in num,
 * otherwise return false.
 */

/**
 * @param {string} num
 * @return {boolean}
 */
var digitCount = function(num) {
  const frequency = new Array(10).fill(0);

  for (const digit of num) {
    frequency[digit]++;
```

```
  }

  for (let i = 0; i < num.length; i++) {
    if (frequency[i] !== Number(num[i])) {
      return false;
    }
  }

  return true;
};
```

2284 Sender With Largest Word Count

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2284. Sender With Largest Word Count
 * https://leetcode.com/problems/sender-with-largest-word-count/
 * Difficulty: Medium
 *
 * You have a chat log of n messages. You are given two string arrays messages and senders
 * where messages[i] is a message sent by senders[i].
 *
 * A message is list of words that are separated by a single space with no leading or trailing
 * spaces. The word count of a sender is the total number of words sent by the sender. Note
 * that a sender may send more than one message.
 *
 * Return the sender with the largest word count. If there is more than one sender with the
 * largest word count, return the one with the lexicographically largest name.
 *
 * Note:
 * - Uppercase letters come before lowercase letters in lexicographical order.
 * - "Alice" and "alice" are distinct.
 */

/**
 * @param {string[]} messages
 * @param {string[]} senders
 * @return {string}
 */
var largestWordCount = function(messages, senders) {
  const map = new Map();
  let maxWords = 0;
  let result = '';

  for (let i = 0; i < messages.length; i++) {
    const wordCount = messages[i].split(' ').length;
    const sender = senders[i];
    const totalWords = (map.get(sender) || 0) + wordCount;
    map.set(sender, totalWords);

    if (totalWords > maxWords || (totalWords === maxWords && sender > result)) {
      maxWords = totalWords;
      result = sender;
    }
  }

  return result;
};
```

2285 Maximum Total Importance Of Roads

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
```

```
 * 2285. Maximum Total Importance of Roads
 * https://leetcode.com/problems/maximum-total-importance-of-roads/
 * Difficulty: Medium
 *
 * You are given an integer n denoting the number of cities in a country. The cities are
 * numbered from 0 to n - 1.
 *
 * You are also given a 2D integer array roads where roads[i] = [ai, bi] denotes that there
 * exists a bidirectional road connecting cities ai and bi.
 *
 * You need to assign each city with an integer value from 1 to n, where each value can
 * only be used once. The importance of a road is then defined as the sum of the values
 * of the two cities it connects.
 *
 * Return the maximum total importance of all roads possible after assigning the values
 * optimally.
 */

/**
 * @param {number} n
 * @param {number[][]} roads
 * @return {number}
 */
var maximumImportance = function(n, roads) {
  const degrees = new Array(n).fill(0);

  for (const [a, b] of roads) {
    degrees[a]++;
    degrees[b]++;
  }

  const cities = Array.from({ length: n }, (_, i) => i);
  cities.sort((a, b) => degrees[b] - degrees[a]);

  let result = 0;
  for (let i = 0; i < n; i++) {
    result += degrees[cities[i]] * (n - i);
  }

  return result;
};
```

2286 Booking Concert Tickets In Groups

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2286. Booking Concert Tickets in Groups
 * https://leetcode.com/problems/booking-concert-tickets-in-groups/
 * Difficulty: Hard
 *
 * A concert hall has n rows numbered from 0 to n - 1, each with m seats, numbered from
 * 0 to m - 1. You need to design a ticketing system that can allocate seats in the following
cases:
 * - If a group of k spectators can sit together in a row.
 * - If every member of a group of k spectators can get a seat. They may or may not sit
together.
 *
 * Note that the spectators are very picky. Hence:
 * - They will book seats only if each member of their group can get a seat with row number
less
 *   than or equal to maxRow. maxRow can vary from group to group.
 * - In case there are multiple rows to choose from, the row with the smallest number is
chosen.
 *   If there are multiple seats to choose in the same row, the seat with the smallest number
is
 *   chosen.
 *
 * Implement the BookMyShow class:
 * - BookMyShow(int n, int m) Initializes the object with n as number of rows and m as number
```

```
of
 *   seats per row.
 * - int[] gather(int k, int maxRow) Returns an array of length 2 denoting the row and seat
number
 *   (respectively) of the first seat being allocated to the k members of the group, who must
sit
 *   together. In other words, it returns the smallest possible r and c such that all [c, c +
k - 1]
 *   seats are valid and empty in row r, and r <= maxRow. Returns [] in case it is not
possible to
 *   allocate seats to the group.
 * - boolean scatter(int k, int maxRow) Returns true if all k members of the group can be
allocated
 *   seats in rows 0 to maxRow, who may or may not sit together. If the seats can be
allocated, it
 *   allocates k seats to the group with the smallest row numbers, and the smallest possible
seat
 *   numbers in each row. Otherwise, returns false.
 */

/**
 * @param {number} n
 * @param {number} m
 */
var BookMyShow = function(n, m) {
  this.n = n;
  this.m = m;

  let size = 1;
  while (size < n * 2) {
    size <<= 1;
  }
  this.segmentTree = new Array(size).fill(0).map(() => [0, 0]);

  this.build(0, 0, n - 1);
};

/**
 * @param {number} k
 * @param {number} maxRow
 * @return {number[]}
 */
BookMyShow.prototype.gather = function(k, maxRow) {
  const result = this.queryMax(0, 0, this.n - 1, k, maxRow);
  if (result.length) {
    this.updateMax(0, 0, this.n - 1, result[0], k);
  }
  return result;
};

/**
 * @param {number} k
 * @param {number} maxRow
 * @return {boolean}
 */
BookMyShow.prototype.scatter = function(k, maxRow) {
  const totalAvailable = this.querySum(0, 0, this.n - 1, maxRow);
  const canScatter = totalAvailable >= k;
  if (canScatter) {
    this.updateSum(0, 0, this.n - 1, k, maxRow);
  }
  return canScatter;
};

/**
 * @param {number} index
 * @param {number} left
 * @param {number} right
 */
BookMyShow.prototype.build = function(index, left, right) {
  if (left === right) {
    this.segmentTree[index] = [this.m, this.m];
    return;
  }
  const mid = Math.floor((left + right) / 2);
```

```
  this.segmentTree[index] = [this.m, (right - left + 1) * this.m];
  this.build(2 * index + 1, left, mid);
  this.build(2 * index + 2, mid + 1, right);
};

/**
 * @param {number} index
 * @param {number} left
 * @param {number} right
 * @param {number} k
 * @param {number} maxRow
 * @return {number[]}
 */
BookMyShow.prototype.queryMax = function(index, left, right, k, maxRow) {
  if (left > maxRow) return [];
  if (this.segmentTree[index][0] < k) return [];
  if (left === right) {
    return [left, this.m - this.segmentTree[index][0]];
  }

  const mid = Math.floor((left + right) / 2);
  const leftResult = this.queryMax(2 * index + 1, left, mid, k, maxRow);
  if (leftResult.length) return leftResult;
  return this.queryMax(2 * index + 2, mid + 1, right, k, maxRow);
};

/**
 * @param {number} index
 * @param {number} left
 * @param {number} right
 * @param {number} row
 * @param {number} k
 */
BookMyShow.prototype.updateMax = function(index, left, right, row, k) {
  if (left > row || right < row) return;
  if (left === right) {
    this.segmentTree[index][0] -= k;
    this.segmentTree[index][1] -= k;
    return;
  }

  const mid = Math.floor((left + right) / 2);
  this.segmentTree[index][1] -= k;
  this.updateMax(2 * index + 1, left, mid, row, k);
  this.updateMax(2 * index + 2, mid + 1, right, row, k);
  this.segmentTree[index][0] = Math.max(
    this.segmentTree[2 * index + 1][0],
    this.segmentTree[2 * index + 2][0]
  );
};

/**
 * @param {number} index
 * @param {number} left
 * @param {number} right
 * @param {number} maxRow
 * @return {number}
 */
BookMyShow.prototype.querySum = function(index, left, right, maxRow) {
  if (left > maxRow) return 0;
  if (right <= maxRow) return this.segmentTree[index][1];

  const mid = Math.floor((left + right) / 2);
  return this.querySum(2 * index + 1, left, mid, maxRow)
    + this.querySum(2 * index + 2, mid + 1, right, maxRow);
};

/**
 * @param {number} index
 * @param {number} left
 * @param {number} right
 * @param {number} k
 * @param {number} maxRow
 */
BookMyShow.prototype.updateSum = function(index, left, right, k, maxRow) {
```

```
  if (left > maxRow) return;
  if (left === right) {
    this.segmentTree[index][0] -= k;
    this.segmentTree[index][1] -= k;
    return;
  }

  const mid = Math.floor((left + right) / 2);
  this.segmentTree[index][1] -= k;

  if (mid + 1 > maxRow || this.segmentTree[2 * index + 1][1] >= k) {
    this.updateSum(2 * index + 1, left, mid, k, maxRow);
  } else {
    const leftSum = this.segmentTree[2 * index + 1][1];
    this.updateSum(2 * index + 1, left, mid, leftSum, maxRow);
    this.updateSum(2 * index + 2, mid + 1, right, k - leftSum, maxRow);
  }

  this.segmentTree[index][0] = Math.max(
    this.segmentTree[2 * index + 1][0],
    this.segmentTree[2 * index + 2][0]
  );
};
```

2287 Rearrange Characters To Make Target String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2287. Rearrange Characters to Make Target String
 * https://leetcode.com/problems/rearrange-characters-to-make-target-string/
 * Difficulty: Easy
 *
 * You are given two 0-indexed strings s and target. You can take some letters from s and
 * rearrange them to form new strings.
 *
 * Return the maximum number of copies of target that can be formed by taking letters from
 * s and rearranging them.
 */

/**
 * @param {string} s
 * @param {string} target
 * @return {number}
 */
var rearrangeCharacters = function(s, target) {
  const sourceFreq = new Array(26).fill(0);
  const targetFreq = new Array(26).fill(0);

  for (const char of s) {
    sourceFreq[char.charCodeAt(0) - 97]++;
  }

  for (const char of target) {
    targetFreq[char.charCodeAt(0) - 97]++;
  }

  let result = Infinity;
  for (let i = 0; i < 26; i++) {
    if (targetFreq[i] > 0) {
      result = Math.min(result, Math.floor(sourceFreq[i] / targetFreq[i]));
    }
  }

  return result;
};
```

2288 Apply Discount To Prices

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2288. Apply Discount to Prices
 * https://leetcode.com/problems/apply-discount-to-prices/
 * Difficulty: Medium
 *
 * A sentence is a string of single-space separated words where each word can contain digits,
 * lowercase letters, and the dollar sign '$'. A word represents a price if it is a sequence
 * of digits preceded by a dollar sign.
 * - For example, "$100", "$23", and "$6" represent prices while "100", "$", and "$1e5" do
not.
 *
 * You are given a string sentence representing a sentence and an integer discount. For each
 * word representing a price, apply a discount of discount% on the price and update the word
 * in the sentence. All updated prices should be represented with exactly two decimal places.
 *
 * Return a string representing the modified sentence.
 *
 * Note that all prices will contain at most 10 digits.
 */

/**
 * @param {string} sentence
 * @param {number} discount
 * @return {string}
 */
var discountPrices = function(sentence, discount) {
  const words = sentence.split(' ');
  const discountFactor = 1 - discount / 100;

  for (let i = 0; i < words.length; i++) {
    if (words[i].startsWith('$') && /^[0-9]+$/.test(words[i].slice(1))) {
      const price = parseInt(words[i].slice(1)) * discountFactor;
      words[i] = `$${price.toFixed(2)}`;
    }
  }

  return words.join(' ');
};
```

2289 Steps To Make Array Non Decreasing

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2289. Steps to Make Array Non-decreasing
 * https://leetcode.com/problems/steps-to-make-array-non-decreasing/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. In one step, remove all elements nums[i]
 * where nums[i - 1] > nums[i] for all 0 < i < nums.length.
 *
 * Return the number of steps performed until nums becomes a non-decreasing array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var totalSteps = function(nums) {
  const n = nums.length;
  const stack = [];
```

```
  const steps = new Array(n).fill(0);
  let result = 0;

  for (let i = n - 1; i >= 0; i--) {
    while (stack.length && nums[i] > nums[stack[stack.length - 1]]) {
      const j = stack.pop();
      steps[i] = Math.max(steps[i] + 1, steps[j]);
    }
    result = Math.max(result, steps[i]);
    stack.push(i);
  }

  return result;
};
```

2290 Minimum Obstacle Removal To Reach Corner

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2290. Minimum Obstacle Removal to Reach Corner
 * https://leetcode.com/problems/minimum-obstacle-removal-to-reach-corner/
 * Difficulty: Hard
 *
 * You are given a 0-indexed 2D integer array grid of size m x n. Each cell has one of two
values:
 * - 0 represents an empty cell,
 * - 1 represents an obstacle that may be removed.
 *
 * You can move up, down, left, or right from and to an empty cell.
 *
 * Return the minimum number of obstacles to remove so you can move from the upper left corner
 * (0, 0) to the lower right corner (m - 1, n - 1).
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minimumObstacles = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const directions = [[0, 1], [0, -1], [1, 0], [-1, 0]];
  const dp = new Array(rows).fill().map(() => new Array(cols).fill(Number.MAX_SAFE_INTEGER));
  const queue = [[0, 0, 0]];
  let result = rows * cols + 1;

  while (queue.length > 0) {
    const levelSize = queue.length;
    for (let l = 0; l < levelSize; l++) {
      const [i, j, cost] = queue.shift();

      if (i === rows - 1 && j === cols - 1) {
        result = Math.min(result, cost);
        continue;
      }

      if (cost > dp[i][j]) {
        continue;
      }

      for (const [dx, dy] of directions) {
        const newI = i + dx;
        const newJ = j + dy;

        if (isValid(newI, newJ, cost)) {
          queue.push([newI, newJ, dp[newI][newJ]]);
        }
      }
    }
```

```
  }

  return result;

  function isValid(i, j, cost) {
    const valid = i >= 0 && j >= 0 && i < rows && j < cols && dp[i][j] > cost + grid[i][j];
    if (valid) dp[i][j] = cost + grid[i][j];
    return valid;
  }
};
```

2291 Maximum Profit From Trading Stocks

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2291. Maximum Profit From Trading Stocks
 * https://leetcode.com/problems/maximum-profit-from-trading-stocks/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer arrays of the same length present and future where
 * present[i] is the current price of the ith stock and future[i] is the price of the ith
 * stock a year in the future. You may buy each stock at most once. You are also given an
 * integer budget representing the amount of money you currently have.
 *
 * Return the maximum amount of profit you can make.
 */

/**
 * @param {number[]} present
 * @param {number[]} future
 * @param {number} budget
 * @return {number}
 */
var maximumProfit = function(present, future, budget) {
  const stocks = present.map((price, i) => ({
    cost: price,
    profit: future[i] - price
  })).filter(stock => stock.profit > 0)
    .sort((a, b) => b.profit - a.profit);

  const dp = new Array(budget + 1).fill(0);
  for (const stock of stocks) {
    for (let money = budget; money >= stock.cost; money--) {
      dp[money] = Math.max(dp[money], dp[money - stock.cost] + stock.profit);
    }
  }

  return dp[budget];
};
```

2293 Min Max Game

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2293. Min Max Game
 * https://leetcode.com/problems/min-max-game/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums whose length is a power of 2.
 *
 * Apply the following algorithm on nums:
 * 1. Let n be the length of nums. If n == 1, end the process. Otherwise, create a new
0-indexed
```

```
 *    integer array newNums of length n / 2.
 * 2. For every even index i where 0 <= i < n / 2, assign the value of newNums[i] as
 *    min(nums[2 * i], nums[2 * i + 1]).
 * 3. For every odd index i where 0 <= i < n / 2, assign the value of newNums[i] as
 *    max(nums[2 * i], nums[2 * i + 1]).
 * 4. Replace the array nums with newNums.
 * 5. Repeat the entire process starting from step 1.
 *
 * Return the last number that remains in nums after applying the algorithm.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minMaxGame = function(nums) {
  let array = nums.slice();

  while (array.length > 1) {
    const newArray = new Array(array.length / 2);

    for (let i = 0; i < newArray.length; i++) {
      if (i % 2 === 0) {
        newArray[i] = Math.min(array[2 * i], array[2 * i + 1]);
      } else {
        newArray[i] = Math.max(array[2 * i], array[2 * i + 1]);
      }
    }

    array = newArray;
  }

  return array[0];
};
```

2294 Partition Array Such That Maximum Difference Is K

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2294. Partition Array Such That Maximum Difference Is K
 * https://leetcode.com/problems/partition-array-such-that-maximum-difference-is-k/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. You may partition nums into one or
more
 * subsequences such that each element in nums appears in exactly one of the subsequences.
 *
 * Return the minimum number of subsequences needed such that the difference between the
maximum
 * and minimum values in each subsequence is at most k.
 *
 * A subsequence is a sequence that can be derived from another sequence by deleting some or
no
 * elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var partitionArray = function(nums, k) {
  nums.sort((a, b) => a - b);
  let result = 1;
  let minValue = nums[0];

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] - minValue > k) {
      result++;
```

```
      minValue = nums[i];
    }
  }

  return result;
};
```

2295 Replace Elements In An Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2295. Replace Elements in an Array
 * https://leetcode.com/problems/replace-elements-in-an-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums that consists of n distinct positive integers.
 * Apply m operations to this array, where in the ith operation you replace the number
 * operations[i][0] with operations[i][1].
 *
 * It is guaranteed that in the ith operation:
 * - operations[i][0] exists in nums.
 * - operations[i][1] does not exist in nums.
 *
 * Return the array obtained after applying all the operations.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} operations
 * @return {number[]}
 */
var arrayChange = function(nums, operations) {
  const map = new Map();
  for (let i = 0; i < nums.length; i++) {
    map.set(nums[i], i);
  }

  for (const [oldVal, newVal] of operations) {
    const index = map.get(oldVal);
    nums[index] = newVal;
    map.set(newVal, index);
    map.delete(oldVal);
  }

  return nums;
};
```

2296 Design A Text Editor

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2296. Design a Text Editor
 * https://leetcode.com/problems/design-a-text-editor/
 * Difficulty: Hard
 *
 * Design a text editor with a cursor that can do the following:
 * - Add text to where the cursor is.
 * - Delete text from where the cursor is (simulating the backspace key).
 * - Move the cursor either left or right.
 *
 * When deleting text, only characters to the left of the cursor will be deleted. The cursor
 * will also remain within the actual text and cannot be moved beyond it. More formally,
 * we have that 0 <= cursor.position <= currentText.length always holds.
```

```
 *
 * Implement the TextEditor class:
 * - TextEditor() Initializes the object with empty text.
 * - void addText(string text) Appends text to where the cursor is. The cursor ends to the
 *   right of text.
 * - int deleteText(int k) Deletes k characters to the left of the cursor. Returns the number
 *   of characters actually deleted.
 * - string cursorLeft(int k) Moves the cursor to the left k times. Returns the last
 *   min(10, len) characters to the left of the cursor, where len is the number of characters
 *   to the left of the cursor.
 * - string cursorRight(int k) Moves the cursor to the right k times. Returns the last
 *   min(10, len) characters to the left of the cursor, where len is the number of characters
 *   to the left of the cursor.
 */

var TextEditor = function() {
  this.left = [];
  this.right = [];
};

/**
 * @param {string} text
 * @return {void}
 */
TextEditor.prototype.addText = function(text) {
  for (const char of text) {
    this.left.push(char);
  }
};

/**
 * @param {number} k
 * @return {number}
 */
TextEditor.prototype.deleteText = function(k) {
  const deleteCount = Math.min(k, this.left.length);
  for (let i = 0; i < deleteCount; i++) {
    this.left.pop();
  }
  return deleteCount;
};

/**
 * @param {number} k
 * @return {string}
 */
TextEditor.prototype.cursorLeft = function(k) {
  for (let i = 0; i < k && this.left.length; i++) {
    this.right.push(this.left.pop());
  }
  const start = Math.max(0, this.left.length - 10);
  return this.left.slice(start).join('');
};

/**
 * @param {number} k
 * @return {string}
 */
TextEditor.prototype.cursorRight = function(k) {
  for (let i = 0; i < k && this.right.length; i++) {
    this.left.push(this.right.pop());
  }
  const start = Math.max(0, this.left.length - 10);
  return this.left.slice(start).join('');
};
```

2297 Jump Game Viii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2297. Jump Game VIII
 * https://leetcode.com/problems/jump-game-viii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of length n. You are initially standing at
index 0.
 * You can jump from index i to index j where i < j if:
 * - nums[i] <= nums[j] and nums[k] < nums[i] for all indexes k in the range i < k < j, or
 * - nums[i] > nums[j] and nums[k] >= nums[i] for all indexes k in the range i < k < j.
 *
 * You are also given an integer array costs of length n where costs[i] denotes the cost of
jumping
 * to index i.
 *
 * Return the minimum cost to jump to the index n - 1.
 */

/**
 * @param {number[]} nums
 * @param {number[]} costs
 * @return {number}
 */
var minCost = function(nums, costs) {
  const n = nums.length;
  const dp = new Array(n).fill(Infinity);
  dp[0] = 0;

  const increasingStack = [];
  const decreasingStack = [];
  for (let i = 0; i < n; i++) {
    while (increasingStack.length > 0
           && nums[increasingStack[increasingStack.length - 1]] > nums[i]) {
      const j = increasingStack.pop();
      dp[i] = Math.min(dp[i], dp[j] + costs[i]);
    }

    while (decreasingStack.length > 0
           && nums[decreasingStack[decreasingStack.length - 1]] <= nums[i]) {
      const j = decreasingStack.pop();
      dp[i] = Math.min(dp[i], dp[j] + costs[i]);
    }

    increasingStack.push(i);
    decreasingStack.push(i);
  }

  return dp[n - 1];
};
```

2299 Strong Password Checker Ii

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2299. Strong Password Checker II
 * https://leetcode.com/problems/strong-password-checker-ii/
 * Difficulty: Easy
 *
 * A password is said to be strong if it satisfies all the following criteria:
 * - It has at least 8 characters.
 * - It contains at least one lowercase letter.
 * - It contains at least one uppercase letter.
 * - It contains at least one digit.
 * - It contains at least one special character. The special characters are the characters
 *   in the following string: "!@#$%^&*()-+".
 * - It does not contain 2 of the same character in adjacent positions (i.e., "aab" violates
 *   this condition, but "aba" does not).
 *
 * Given a string password, return true if it is a strong password. Otherwise, return false.
```

```
 */

/**
 * @param {string} password
 * @return {boolean}
 */
var strongPasswordCheckerII = function(password) {
  if (password.length < 8) return false;

  let hasLower = false;
  let hasUpper = false;
  let hasDigit = false;
  let hasSpecial = false;
  const set = new Set(['!', '@', '#', '$', '%', '^', '&', '*', '(', ')', '-', '+']);

  for (let i = 0; i < password.length; i++) {
    if (i > 0 && password[i] === password[i - 1]) return false;

    const char = password[i];
    if (/[a-z]/.test(char)) hasLower = true;
    else if (/[A-Z]/.test(char)) hasUpper = true;
    else if (/\d/.test(char)) hasDigit = true;
    else if (set.has(char)) hasSpecial = true;
  }

  return hasLower && hasUpper && hasDigit && hasSpecial;
};
```

2300 Successful Pairs Of Spells And Potions

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2300. Successful Pairs of Spells and Potions
 * https://leetcode.com/problems/successful-pairs-of-spells-and-potions/
 * Difficulty: Medium
 *
 * You are given two positive integer arrays spells and potions, of length n and m
respectively,
 * where spells[i] represents the strength of the ith spell and potions[j] represents the
strength
 * of the jth potion.
 *
 * You are also given an integer success. A spell and potion pair is considered successful if
the
 * product of their strengths is at least success.
 *
 * Return an integer array pairs of length n where pairs[i] is the number of potions that will
 form
 * a successful pair with the ith spell.
 */

/**
 * @param {number[]} spells
 * @param {number[]} potions
 * @param {number} success
 * @return {number[]}
 */
var successfulPairs = function(spells, potions, success) {
  const result = [];
  potions.sort((a, b) => a - b);

  for (let i = 0; i < spells.length; i++) {
    let left = 0;
    let right = potions.length - 1;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      if (spells[i] * potions[mid] < success) {
```

```
        left = mid + 1;
      } else {
        right = mid - 1;
      }
    }

    result[i] = potions.length - left;
  }

  return result;
};
```

2301 Match Substring After Replacement

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2301. Match Substring After Replacement
 * https://leetcode.com/problems/match-substring-after-replacement/
 * Difficulty: Hard
 *
 * You are given two strings s and sub. You are also given a 2D character array mappings where
 * mappings[i] = [oldi, newi] indicates that you may perform the following operation any
number
 * of times:
 * - Replace a character oldi of sub with newi.
 *
 * Each character in sub cannot be replaced more than once.
 *
 * Return true if it is possible to make sub a substring of s by replacing zero or more
characters
 * according to mappings. Otherwise, return false.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {string} sub
 * @param {character[][]} mappings
 * @return {boolean}
 */
var matchReplacement = function(s, sub, mappings) {
  const map = new Map();

  for (const [oldChar, newChar] of mappings) {
    if (!map.has(oldChar)) {
      map.set(oldChar, new Set());
    }
    map.get(oldChar).add(newChar);
  }

  const subLen = sub.length;
  for (let i = 0; i <= s.length - subLen; i++) {
    let isValid = true;
    for (let j = 0; j < subLen; j++) {
      const sChar = s[i + j];
      const subChar = sub[j];
      if (sChar !== subChar && (!map.has(subChar) || !map.get(subChar).has(sChar))) {
        isValid = false;
        break;
      }
    }
    if (isValid) return true;
  }

  return false;
};
```

2302 Count Subarrays With Score Less Than K

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2302. Count Subarrays With Score Less Than K
 * https://leetcode.com/problems/count-subarrays-with-score-less-than-k/
 * Difficulty: Hard
 *
 * The score of an array is defined as the product of its sum and its length.
 * - For example, the score of [1, 2, 3, 4, 5] is (1 + 2 + 3 + 4 + 5) * 5 = 75.
 *
 * Given a positive integer array nums and an integer k, return the number of non-empty
subarrays
 * of nums whose score is strictly less than k.
 *
 * A subarray is a contiguous sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countSubarrays = function(nums, k) {
  let result = 0;
  let currentSum = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    currentSum += nums[right];
    while (currentSum * (right - left + 1) >= k) {
      currentSum -= nums[left];
      left++;
    }
    result += right - left + 1;
  }

  return result;
};
```

2303 Calculate Amount Paid In Taxes

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2303. Calculate Amount Paid in Taxes
 * https://leetcode.com/problems/calculate-amount-paid-in-taxes/
 * Difficulty: Easy
 *
 * You are given a 0-indexed 2D integer array brackets where brackets[i] = [upperi, percenti]
 * means that the ith tax bracket has an upper bound of upperi and is taxed at a rate of
percenti.
 * The brackets are sorted by upper bound (i.e. upperi-1 < upperi for 0 < i <
brackets.length).
 *
 * Tax is calculated as follows:
 * - The first upper0 dollars earned are taxed at a rate of percent0.
 * - The next upper1 - upper0 dollars earned are taxed at a rate of percent1.
 * - The next upper2 - upper1 dollars earned are taxed at a rate of percent2.
 * - And so on.
 *
 * You are given an integer income representing the amount of money you earned. Return the
 * amount of money that you have to pay in taxes. Answers within 10-5 of the actual answer
 * will be accepted.
```

```
 */

/**
 * @param {number[][]} brackets
 * @param {number} income
 * @return {number}
 */
var calculateTax = function(brackets, income) {
  let result = 0;
  let previousUpper = 0;

  for (const [upper, percent] of brackets) {
    if (income > previousUpper) {
      const taxable = Math.min(income, upper) - previousUpper;
      result += taxable * (percent / 100);
      previousUpper = upper;
    } else {
      break;
    }
  }

  return result;
};
```

2304 Minimum Path Cost In A Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2304. Minimum Path Cost in a Grid
 * https://leetcode.com/problems/minimum-path-cost-in-a-grid/
 * Difficulty: Medium
 *
 * You are given a 0-indexed m x n integer matrix grid consisting of distinct integers from
 * 0 to m * n - 1. You can move in this matrix from a cell to any other cell in the next row.
 * That is, if you are in cell (x, y) such that x < m - 1, you can move to any of the cells
 * (x + 1, 0), (x + 1, 1), ..., (x + 1, n - 1). Note that it is not possible to move from
 * cells in the last row.
 *
 * Each possible move has a cost given by a 0-indexed 2D array moveCost of size (m * n) x n,
 * where moveCost[i][j] is the cost of moving from a cell with value i to a cell in column j
 * of the next row. The cost of moving from cells in the last row of grid can be ignored.
 *
 * The cost of a path in grid is the sum of all values of cells visited plus the sum of costs
 * of all the moves made. Return the minimum cost of a path that starts from any cell in the
 * first row and ends at any cell in the last row.
 */

/**
 * @param {number[][]} grid
 * @param {number[][]} moveCost
 * @return {number}
 */
var minPathCost = function(grid, moveCost) {
  const m = grid.length;
  const n = grid[0].length;
  const dp = Array.from({ length: m }, () => new Array(n).fill(Infinity));

  for (let j = 0; j < n; j++) {
    dp[0][j] = grid[0][j];
  }

  for (let i = 0; i < m - 1; i++) {
    for (let j = 0; j < n; j++) {
      const value = grid[i][j];
      for (let k = 0; k < n; k++) {
        dp[i + 1][k] = Math.min(
          dp[i + 1][k],
          dp[i][j] + grid[i + 1][k] + moveCost[value][k]
        );
```

```
      }
    }
  }

  return Math.min(...dp[m - 1]);
};
```

2305 Fair Distribution Of Cookies

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2305. Fair Distribution of Cookies
 * https://leetcode.com/problems/fair-distribution-of-cookies/
 * Difficulty: Medium
 *
 * You are given an integer array cookies, where cookies[i] denotes the number of cookies in
the
 * ith bag. You are also given an integer k that denotes the number of children to distribute
 * all the bags of cookies to. All the cookies in the same bag must go to the same child and
 * cannot be split up.
 *
 * The unfairness of a distribution is defined as the maximum total cookies obtained by a
single
 * child in the distribution.
 *
 * Return the minimum unfairness of all distributions.
 */

/**
 * @param {number[]} cookies
 * @param {number} k
 * @return {number}
 */
var distributeCookies = function(cookies, k) {
  const distribution = new Array(k).fill(0);
  let result = Infinity;

  distribute(0);

  return result;

  function distribute(index) {
    if (index === cookies.length) {
      result = Math.min(result, Math.max(...distribution));
      return;
    }

    for (let i = 0; i < k; i++) {
      distribution[i] += cookies[index];
      if (distribution[i] < result) {
        distribute(index + 1);
      }
      distribution[i] -= cookies[index];

      if (distribution[i] === 0) break;
    }
  }
};
```

2306 Naming A Company

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 2306. Naming a Company
 * https://leetcode.com/problems/naming-a-company/
 * Difficulty: Hard
 *
 * You are given an array of strings ideas that represents a list of names to be used in the
 * process of naming a company. The process of naming a company is as follows:
 * 1. Choose 2 distinct names from ideas, call them ideaA and ideaB.
 * 2. Swap the first letters of ideaA and ideaB with each other.
 * 3. If both of the new names are not found in the original ideas, then the name ideaA ideaB
 *    (the concatenation of ideaA and ideaB, separated by a space) is a valid company name.
 * 4. Otherwise, it is not a valid name.
 *
 * Return the number of distinct valid names for the company.
 */

/**
 * @param {string[]} ideas
 * @return {number}
 */
var distinctNames = function(ideas) {
  const groups = Array.from({ length: 26 }, () => new Set());
  let result = 0;

  for (const idea of ideas) {
    groups[idea.charCodeAt(0) - 97].add(idea.slice(1));
  }

  for (let i = 0; i < 25; i++) {
    for (let j = i + 1; j < 26; j++) {
      const mutualCount = [...groups[i]].filter(suffix => groups[j].has(suffix)).length;
      result += 2 * (groups[i].size - mutualCount) * (groups[j].size - mutualCount);
    }
  }

  return result;
};
```

2307 Check For Contradictions In Equations

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2307. Check for Contradictions in Equations
 * https://leetcode.com/problems/check-for-contradictions-in-equations/
 * Difficulty: Hard
 *
 * You are given a 2D array of strings equations and an array of real numbers values, where
 * equations[i] = [Ai, Bi] and values[i] means that Ai / Bi = values[i].
 *
 * Determine if there exists a contradiction in the equations. Return true if there is a
 * contradiction, or false otherwise.
 *
 * Note:
 * - When checking if two numbers are equal, check that their absolute difference is less than
 10-5.
 * - The testcases are generated such that there are no cases targeting precision, i.e. using
double
 *   is enough to solve the problem.
 */

/**
 * @param {string[][]} equations
 * @param {number[]} values
 * @return {boolean}
 */
var checkContradictions = function(equations, values) {
  const graph = new Map();

  for (let i = 0; i < equations.length; i++) {
    const [a, b] = equations[i];
```

```
    const value = values[i];

    const existingValue = dfs(a, b, new Set(), 1);

    if (existingValue !== null && Math.abs(existingValue - value) >= 1e-5) {
      return true;
    }

    addEdge(a, b, value);
  }

  return false;

  function addEdge(a, b, value) {
    if (!graph.has(a)) graph.set(a, []);
    if (!graph.has(b)) graph.set(b, []);
    graph.get(a).push([b, value]);
    graph.get(b).push([a, 1 / value]);
  }

  function dfs(start, target, visited, currentValue) {
    if (start === target) return currentValue;
    if (visited.has(start)) return null;

    visited.add(start);

    for (const [neighbor, value] of graph.get(start) || []) {
      const result = dfs(neighbor, target, visited, currentValue * value);
      if (result !== null) return result;
    }

    visited.delete(start);
    return null;
  }
};
```

2309 Greatest English Letter In Upper And Lower Case

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2309. Greatest English Letter in Upper and Lower Case
 * https://leetcode.com/problems/greatest-english-letter-in-upper-and-lower-case/
 * Difficulty: Easy
 *
 * Given a string of English letters s, return the greatest English letter which occurs as
both
 * a lowercase and uppercase letter in s. The returned letter should be in uppercase. If no
such
 * letter exists, return an empty string.
 *
 * An English letter b is greater than another letter a if b appears after a in the English
 * alphabet.
 */

/**
 * @param {string} s
 * @return {string}
 */
var greatestLetter = function(s) {
  const lowerSet = new Set();
  const upperSet = new Set();

  for (const char of s) {
    if (char >= 'a' && char <= 'z') {
      lowerSet.add(char);
    } else {
      upperSet.add(char.toLowerCase());
    }
  }
```

```
  let maxLetter = '';
  for (const char of lowerSet) {
    if (upperSet.has(char) && char > maxLetter) {
      maxLetter = char;
    }
  }

  return maxLetter.toUpperCase();
};
```

2310 Sum Of Numbers With Units Digit K

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2310. Sum of Numbers With Units Digit K
 * https://leetcode.com/problems/sum-of-numbers-with-units-digit-k/
 * Difficulty: Medium
 *
 * Given two integers num and k, consider a set of positive integers with the following
properties:
 * - The units digit of each integer is k.
 * - The sum of the integers is num.
 *
 * Return the minimum possible size of such a set, or -1 if no such set exists.
 *
 * Note:
 * - The set can contain multiple instances of the same integer, and the sum of an empty set
is
 *   considered 0.
 * - The units digit of a number is the rightmost digit of the number.
 */

/**
 * @param {number} num
 * @param {number} k
 * @return {number}
 */
var minimumNumbers = function(num, k) {
  if (num === 0) return 0;

  for (let count = 1; count <= 10; count++) {
    if ((count * k) % 10 === num % 10 && count * k <= num) {
      return count;
    }
  }

  return -1;
};
```

2311 Longest Binary Subsequence Less Than Or Equal To K

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2311. Longest Binary Subsequence Less Than or Equal to K
 * https://leetcode.com/problems/longest-binary-subsequence-less-than-or-equal-to-k/
 * Difficulty: Medium
 *
 * You are given a binary string s and a positive integer k.
 *
 * Return the length of the longest subsequence of s that makes up a binary number less than
 * or equal to k.
 *
```

```
 * Note:
 * - The subsequence can contain leading zeroes.
 * - The empty string is considered to be equal to 0.
 * - A subsequence is a string that can be derived from another string by deleting some or no
 *   characters without changing the order of the remaining characters.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var longestSubsequence = function(s, k) {
  let zeros = 0;
  let ones = 0;
  let value = 0;
  let power = 1;

  for (let i = s.length - 1; i >= 0; i--) {
    if (s[i] === '0') {
      zeros++;
    } else {
      if (power <= k && value + power <= k) {
        value += power;
        ones++;
      }
    }
    if (power <= k) {
      power *= 2;
    }
  }

  return zeros + ones;
};
```

2312 Selling Pieces Of Wood

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2312. Selling Pieces of Wood
 * https://leetcode.com/problems/selling-pieces-of-wood/
 * Difficulty: Hard
 *
 * You are given two integers m and n that represent the height and width of a rectangular
piece
 * of wood. You are also given a 2D integer array prices, where prices[i] = [hi, wi, pricei]
 * indicates you can sell a rectangular piece of wood of height hi and width wi for pricei
dollars.
 *
 * To cut a piece of wood, you must make a vertical or horizontal cut across the entire height
 * or width of the piece to split it into two smaller pieces. After cutting a piece of wood
into
 * some number of smaller pieces, you can sell pieces according to prices. You may sell
multiple
 * pieces of the same shape, and you do not have to sell all the shapes. The grain of the wood
 * makes a difference, so you cannot rotate a piece to swap its height and width.
 *
 * Return the maximum money you can earn after cutting an m x n piece of wood.
 *
 * Note that you can cut the piece of wood as many times as you want.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number[][]} prices
 * @return {number}
 */
var sellingWood = function(m, n, prices) {
```

```
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));

  for (const [h, w, price] of prices) {
    dp[h][w] = price;
  }

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      for (let k = 1; k < i; k++) {
        dp[i][j] = Math.max(dp[i][j], dp[k][j] + dp[i - k][j]);
      }
      for (let k = 1; k < j; k++) {
        dp[i][j] = Math.max(dp[i][j], dp[i][k] + dp[i][j - k]);
      }
    }
  }

  return dp[m][n];
};
```

2313 Minimum Flips In Binary Tree To Get Result

HARD

APPROACH NOTES

Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2313. Minimum Flips in Binary Tree to Get Result
 * https://leetcode.com/problems/minimum-flips-in-binary-tree-to-get-result/
 * Difficulty: Hard
 *
 * You are given the root of a binary tree with the following properties:
 * - Leaf nodes have either the value 0 or 1, representing false and true respectively.
 * - Non-leaf nodes have either the value 2, 3, 4, or 5, representing the boolean operations
 *   OR, AND, XOR, and NOT, respectively.
 *
 * You are also given a boolean result, which is the desired result of the evaluation of the
 * root node.
 *
 * The evaluation of a node is as follows:
 * - If the node is a leaf node, the evaluation is the value of the node, i.e. true or false.
 * - Otherwise, evaluate the node's children and apply the boolean operation of its value with
 *   the children's evaluations.
 *
 * In one operation, you can flip a leaf node, which causes a false node to become true, and
 * a true node to become false.
 *
 * Return the minimum number of operations that need to be performed such that the evaluation
 * of root yields result. It can be shown that there is always a way to achieve result.
 *
 * A leaf node is a node that has zero children.
 *
 * Note: NOT nodes have either a left child or a right child, but other non-leaf nodes have
both
 * a left child and a right child.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {boolean} result
 * @return {number}
 */
var minimumFlips = function(root, result) {
  const [trueFlips, falseFlips] = dfs(root);
```

```
  return result ? trueFlips : falseFlips;

  function dfs(node) {
    if (node.val === 0) {
      return [1, 0];
    }
    if (node.val === 1) {
      return [0, 1];
    }

    if (node.val === 5) {
      const [t, f] = dfs(node.left || node.right);
      return [f, t];
    }

    const [leftTrue, leftFalse] = dfs(node.left);
    const [rightTrue, rightFalse] = dfs(node.right);

    if (node.val === 2) {
      return [
        Math.min(leftTrue + rightTrue, leftFalse + rightTrue, leftTrue + rightFalse),
        leftFalse + rightFalse
      ];
    }

    if (node.val === 3) {
      return [
        leftTrue + rightTrue,
        Math.min(leftTrue + rightFalse, leftFalse + rightTrue, leftFalse + rightFalse)
      ];
    }

    if (node.val === 4) {
      return [
        Math.min(leftFalse + rightTrue, leftTrue + rightFalse),
        Math.min(leftTrue + rightTrue, leftFalse + rightFalse)
      ];
    }
  }
};
```

2315 Count Asterisks

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2315. Count Asterisks
 * https://leetcode.com/problems/count-asterisks/
 * Difficulty: Easy
 *
 * You are given a string s, where every two consecutive vertical bars '|' are grouped into
 * a pair. In other words, the 1st and 2nd '|' make a pair, the 3rd and 4th '|' make a pair,
 * and so forth.
 *
 * Return the number of '*' in s, excluding the '*' between each pair of '|'.
 *
 * Note that each '|' will belong to exactly one pair.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countAsterisks = function(s) {
  let isInsidePair = false;
  let result = 0;

  for (const char of s) {
    if (char === '|') {
      isInsidePair = !isInsidePair;
```

```
    } else if (char === '*' && !isInsidePair) {
      result++;
    }
  }

  return result;
};
```

2316 Count Unreachable Pairs Of Nodes In An Undirected Graph

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2316. Count Unreachable Pairs of Nodes in an Undirected Graph
 * https://leetcode.com/problems/count-unreachable-pairs-of-nodes-in-an-undirected-graph/
 * Difficulty: Medium
 *
 * You are given an integer n. There is an undirected graph with n nodes, numbered from 0
 * to n - 1. You are given a 2D integer array edges where edges[i] = [ai, bi] denotes that
 * there exists an undirected edge connecting nodes ai and bi.
 *
 * Return the number of pairs of different nodes that are unreachable from each other.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var countPairs = function(n, edges) {
  const graph = Array.from({ length: n }, () => []);
  for (const [a, b] of edges) {
    graph[a].push(b);
    graph[b].push(a);
  }

  const visited = new Array(n).fill(false);
  let result = 0;

  let totalNodes = 0;
  for (let node = 0; node < n; node++) {
    if (!visited[node]) {
      const componentSize = exploreComponent(node);
      result += totalNodes * componentSize;
      totalNodes += componentSize;
    }
  }

  return result;

  function exploreComponent(node) {
    if (visited[node]) return 0;
    visited[node] = true;
    let size = 1;
    for (const neighbor of graph[node]) {
      size += exploreComponent(neighbor);
    }
    return size;
  }
};
```

2317 Maximum Xor After Operations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2317. Maximum XOR After Operations
 * https://leetcode.com/problems/maximum-xor-after-operations/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. In one operation, select any non-negative
 * integer x and an index i, then update nums[i] to be equal to nums[i] AND (nums[i] XOR x).
 *
 * Note that AND is the bitwise AND operation and XOR is the bitwise XOR operation.
 *
 * Return the maximum possible bitwise XOR of all elements of nums after applying the
operation
 * any number of times.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumXOR = function(nums) {
  let maxXor = 0;

  for (const num of nums) {
    maxXor |= num;
  }

  return maxXor;
};
```

2318 Number Of Distinct Roll Sequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2318. Number of Distinct Roll Sequences
 * https://leetcode.com/problems/number-of-distinct-roll-sequences/
 * Difficulty: Hard
 *
 * You are given an integer n. You roll a fair 6-sided dice n times. Determine the total
number
 * of distinct sequences of rolls possible such that the following conditions are satisfied:
 * 1. The greatest common divisor of any adjacent values in the sequence is equal to 1.
 * 2. There is at least a gap of 2 rolls between equal valued rolls. More formally, if the
 *    value of the ith roll is equal to the value of the jth roll, then abs(i - j) > 2.
 *
 * Return the total number of distinct sequences possible. Since the answer may be very large,
 * return it modulo 109 + 7.
 *
 * Two sequences are considered distinct if at least one element is different.
 */

/**
 * @param {number} n
 * @return {number}
 */
var distinctSequences = function(n) {
  const MOD = 1e9 + 7;
  const gcd = (a, b) => b === 0 ? a : gcd(b, a % b);

  const dp = new Array(n + 1)
    .fill()
    .map(() => new Array(7).fill().map(() => new Array(7).fill(0)));

  for (let i = 1; i <= 6; i++) {
    dp[1][i][0] = 1;
  }

  for (let len = 2; len <= n; len++) {
    for (let curr = 1; curr <= 6; curr++) {
      for (let prev = 0; prev <= 6; prev++) {
```

```
        for (let prevPrev = 0; prevPrev <= 6; prevPrev++) {
          if (dp[len - 1][prev][prevPrev] === 0) continue;
          if (curr === prev || curr === prevPrev) continue;
          if (prev !== 0 && gcd(curr, prev) !== 1) continue;
          dp[len][curr][prev] = (dp[len][curr][prev] + dp[len - 1][prev][prevPrev]) % MOD;
        }
      }
    }
  }

  let result = 0;
  for (let curr = 1; curr <= 6; curr++) {
    for (let prev = 0; prev <= 6; prev++) {
      result = (result + dp[n][curr][prev]) % MOD;
    }
  }

  return result;
};
```

2319 Check If Matrix Is X Matrix

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2319. Check if Matrix Is X-Matrix
 * https://leetcode.com/problems/check-if-matrix-is-x-matrix/
 * Difficulty: Easy
 *
 * A square matrix is said to be an X-Matrix if both of the following conditions hold:
 * 1. All the elements in the diagonals of the matrix are non-zero.
 * 2. All other elements are 0.
 *
 * Given a 2D integer array grid of size n x n representing a square matrix, return true if
grid
 * is an X-Matrix. Otherwise, return false.
 */

/**
 * @param {number[][]} grid
 * @return {boolean}
 */
var checkXMatrix = function(grid) {
  const size = grid.length;

  for (let row = 0; row < size; row++) {
    for (let col = 0; col < size; col++) {
      const isDiagonal = row === col || row + col === size - 1;
      const isNonZero = grid[row][col] !== 0;

      if (isDiagonal && !isNonZero) return false;
      if (!isDiagonal && isNonZero) return false;
    }
  }

  return true;
};
```

2320 Count Number Of Ways To Place Houses

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2320. Count Number of Ways to Place Houses
 * https://leetcode.com/problems/count-number-of-ways-to-place-houses/
```

```
 * Difficulty: Medium
 *
 * There is a street with n * 2 plots, where there are n plots on each side of the street. The
 plots
 * on each side are numbered from 1 to n. On each plot, a house can be placed.
 *
 * Return the number of ways houses can be placed such that no two houses are adjacent to each
 other
 * on the same side of the street. Since the answer may be very large, return it modulo 109 +
7.
 *
 * Note that if a house is placed on the ith plot on one side of the street, a house can also
be
 * placed on the ith plot on the other side of the street.
 */

/**
 * @param {number} n
 * @return {number}
 */
var countHousePlacements = function(n) {
  const MOD = 1e9 + 7;
  let empty = 1n;
  let house = 1n;

  for (let i = 2; i <= n; i++) {
    const nextEmpty = (empty + house) % BigInt(MOD);
    const nextHouse = empty;
    empty = nextEmpty;
    house = nextHouse;
  }

  const sideWays = (empty + house) % BigInt(MOD);
  return Number((sideWays * sideWays) % BigInt(MOD));
};
```

2321 Maximum Score Of Spliced Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2321. Maximum Score Of Spliced Array
 * https://leetcode.com/problems/maximum-score-of-spliced-array/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays nums1 and nums2, both of length n.
 *
 * You can choose two integers left and right where 0 <= left <= right < n and swap the
subarray
 * nums1[left...right] with the subarray nums2[left...right].
 * - For example, if nums1 = [1,2,3,4,5] and nums2 = [11,12,13,14,15] and you choose left = 1
and
 *   right = 2, nums1 becomes [1,12,13,4,5] and nums2 becomes [11,2,3,14,15].
 *
 * You may choose to apply the mentioned operation once or not do anything.
 *
 * The score of the arrays is the maximum of sum(nums1) and sum(nums2), where sum(arr) is the
sum
 * of all the elements in the array arr.
 *
 * Return the maximum possible score.
 *
 * A subarray is a contiguous sequence of elements within an array. arr[left...right] denotes
the
 * subarray that contains the elements of nums between indices left and right (inclusive).
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
```

```
 * @return {number}
 */
var maximumsSplicedArray = function(nums1, nums2) {
  const length = nums1.length;
  let sum1 = 0;
  let sum2 = 0;

  for (let i = 0; i < length; i++) {
    sum1 += nums1[i];
    sum2 += nums2[i];
  }

  let maxGain1 = 0;
  let maxGain2 = 0;
  let currGain1 = 0;
  let currGain2 = 0;

  for (let i = 0; i < length; i++) {
    currGain1 = Math.max(0, currGain1 + nums2[i] - nums1[i]);
    currGain2 = Math.max(0, currGain2 + nums1[i] - nums2[i]);
    maxGain1 = Math.max(maxGain1, currGain1);
    maxGain2 = Math.max(maxGain2, currGain2);
  }

  return Math.max(sum1 + maxGain1, sum2 + maxGain2);
};
```

2322 Minimum Score After Removals On A Tree

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2322. Minimum Score After Removals on a Tree
 * https://leetcode.com/problems/minimum-score-after-removals-on-a-tree/
 * Difficulty: Hard
 *
 * There is an undirected connected tree with n nodes labeled from 0 to n - 1 and n - 1 edges.
 *
 * You are given a 0-indexed integer array nums of length n where nums[i] represents the value
 * of the ith node. You are also given a 2D integer array edges of length n - 1 where
 * edges[i] = [ai, bi] indicates that there is an edge between nodes ai and bi in the tree.
 *
 * Remove two distinct edges of the tree to form three connected components. For a pair of
 * removed edges, the following steps are defined:
 * 1. Get the XOR of all the values of the nodes for each of the three components
respectively.
 * 2. The difference between the largest XOR value and the smallest XOR value is the score of
 *    the pair.
 * 3. For example, say the three components have the node values: [4,5,7], [1,9], and [3,3,3].
 *    The three XOR values are 4 ^ 5 ^ 7 = 6, 1 ^ 9 = 8, and 3 ^ 3 ^ 3 = 3. The largest XOR
 *    value is 8 and the smallest XOR value is 3. The score is then 8 - 3 = 5.
 *
 * Return the minimum score of any possible pair of edge removals on the given tree.
 */

/**
* @param {number[]} nums
* @param {number[][]} edges
* @return {number}
*/
var minimumScore = function(nums, edges) {
  const n = nums.length;
  const graph = Array.from({ length: n }, () => []);

  for (const [a, b] of edges) {
    graph[a].push(b);
    graph[b].push(a);
  }

  const totalXor = nums.reduce((acc, num) => acc ^ num, 0);
```

```
  const subtreeXor = new Array(n).fill(0);

  const tin = new Array(n);
  const tout = new Array(n);
  let timer = 0;

  function dfs(node, parent) {
    tin[node] = timer++;
    subtreeXor[node] = nums[node];

    for (const neighbor of graph[node]) {
      if (neighbor !== parent) {
        dfs(neighbor, node);
        subtreeXor[node] ^= subtreeXor[neighbor];
      }
    }

    tout[node] = timer++;
  }

  dfs(0, -1);

  function isAncestor(a, b) {
    return tin[a] <= tin[b] && tout[a] >= tout[b];
  }

  let result = Infinity;
  for (let i = 0; i < n - 1; i++) {
    for (let j = i + 1; j < n - 1; j++) {
      const edge1 = edges[i];
      const edge2 = edges[j];

      let node1;
      if (isAncestor(edge1[0], edge1[1])) {
        node1 = edge1[1];
      } else {
        node1 = edge1[0];
      }
      const subtree1 = subtreeXor[node1];

      let node2;
      if (isAncestor(edge2[0], edge2[1])) {
        node2 = edge2[1];
      } else {
        node2 = edge2[0];
      }
      const subtree2 = subtreeXor[node2];

      let component1;
      let component2;
      let component3;
      if (isAncestor(node1, node2)) {
        component1 = subtree2;
        component2 = subtree1 ^ component1;
        component3 = totalXor ^ subtree1;
      } else if (isAncestor(node2, node1)) {
        component1 = subtree1;
        component2 = subtree2 ^ component1;
        component3 = totalXor ^ subtree2;
      } else {
        component1 = subtree1;
        component2 = subtree2;
        component3 = totalXor ^ component1 ^ component2;
      }

      const maxXor = Math.max(component1, component2, component3);
      const minXor = Math.min(component1, component2, component3);
      result = Math.min(result, maxXor - minXor);
    }
  }

  return result;
};
```

2323 Find Minimum Time To Finish All Jobs Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2323. Find Minimum Time to Finish All Jobs II
 * https://leetcode.com/problems/find-minimum-time-to-finish-all-jobs-ii/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer arrays jobs and workers of equal length, where jobs[i]
 * is the amount of time needed to complete the ith job, and workers[j] is the amount of time
 * the jth worker can work each day.
 *
 * Each job should be assigned to exactly one worker, such that each worker completes exactly
 * one job.
 *
 * Return the minimum number of days needed to complete all the jobs after assignment.
 */

/**
 * @param {number[]} jobs
 * @param {number[]} workers
 * @return {number}
 */
var minimumTime = function(jobs, workers) {
  jobs.sort((a, b) => b - a);
  workers.sort((a, b) => b - a);

  let result = 0;
  for (let i = 0; i < jobs.length; i++) {
    const daysNeeded = Math.ceil(jobs[i] / workers[i]);
    result = Math.max(result, daysNeeded);
  }

  return result;
};
```

2325 Decode The Message

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2325. Decode the Message
 * https://leetcode.com/problems/decode-the-message/
 * Difficulty: Easy
 *
 * You are given the strings key and message, which represent a cipher key and a secret
message,
 * respectively. The steps to decode message are as follows:
 * 1. Use the first appearance of all 26 lowercase English letters in key as the order of the
 *    substitution table.
 * 2. Align the substitution table with the regular English alphabet.
 * 3. Each letter in message is then substituted using the table.
 * 4. Spaces ' ' are transformed to themselves.
 * 5. For example, given key = "happy boy" (actual key would have at least one instance of
each
 *    letter in the alphabet), we have the partial substitution table of ('h' -> 'a', 'a' ->
'b',
 *    'p' -> 'c', 'y' -> 'd', 'b' -> 'e', 'o' -> 'f').
 *
 * Return the decoded message.
 */

/**
 * @param {string} key
```

```
 * @param {string} message
 * @return {string}
 */
var decodeMessage = function(key, message) {
  const substitution = new Map();
  let alphabetIndex = 0;

  for (const char of key) {
    if (char !== ' ' && !substitution.has(char)) {
      substitution.set(char, String.fromCharCode(97 + alphabetIndex++));
    }
  }

  let result = '';
  for (const char of message) {
    result += char === ' ' ? ' ' : substitution.get(char);
  }

  return result;
};
```

2326 Spiral Matrix Iv

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2326. Spiral Matrix IV
 * https://leetcode.com/problems/spiral-matrix-iv/
 * Difficulty: Medium
 *
 * You are given two integers m and n, which represent the dimensions of a matrix.
 *
 * You are also given the head of a linked list of integers.
 *
 * Generate an m x n matrix that contains the integers in the linked list presented in spiral
 * order (clockwise), starting from the top-left of the matrix. If there are remaining empty
 * spaces, fill them with -1.
 *
 * Return the generated matrix.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {number} m
 * @param {number} n
 * @param {ListNode} head
 * @return {number[][]}
 */
var spiralMatrix = function(m, n, head) {
  const matrix = new Array(m).fill().map(() => new Array(n).fill(-1));
  let top = 0;
  let bottom = m - 1;
  let left = 0;
  let right = n - 1;
  let current = head;

  while (top <= bottom && left <= right && current) {
    for (let col = left; col <= right && current; col++) {
      matrix[top][col] = current.val;
      current = current.next;
    }
    top++;

    for (let row = top; row <= bottom && current; row++) {
```

```
      matrix[row][right] = current.val;
      current = current.next;
    }
    right--;

    for (let col = right; col >= left && current; col--) {
      matrix[bottom][col] = current.val;
      current = current.next;
    }
    bottom--;

    for (let row = bottom; row >= top && current; row--) {
      matrix[row][left] = current.val;
      current = current.next;
    }
    left++;
  }

  return matrix;
};
```

2327 Number Of People Aware Of A Secret

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2327. Number of People Aware of a Secret
 * https://leetcode.com/problems/number-of-people-aware-of-a-secret/
 * Difficulty: Medium
 *
 * On day 1, one person discovers a secret.
 *
 * You are given an integer delay, which means that each person will share the secret with a
new
 * person every day, starting from delay days after discovering the secret. You are also given
 * an integer forget, which means that each person will forget the secret forget days after
 * discovering it. A person cannot share the secret on the same day they forgot it, or on any
 * day afterwards.
 *
 * Given an integer n, return the number of people who know the secret at the end of day n.
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} delay
 * @param {number} forget
 * @return {number}
 */
var peopleAwareOfSecret = function(n, delay, forget) {
  const MOD = 1e9 + 7;
  const dp = new Array(n + 1).fill(0);
  dp[1] = 1;

  for (let day = 1; day <= n; day++) {
    if (dp[day] === 0) continue;

    for (let shareDay = day + delay; shareDay < day + forget && shareDay <= n; shareDay++) {
      dp[shareDay] = (dp[shareDay] + dp[day]) % MOD;
    }
  }

  let result = 0;
  for (let day = Math.max(1, n - forget + 1); day <= n; day++) {
    result = (result + dp[day]) % MOD;
  }

  return result;
};
```

2328 Number Of Increasing Paths In A Grid

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2328. Number of Increasing Paths in a Grid
 * https://leetcode.com/problems/number-of-increasing-paths-in-a-grid/
 * Difficulty: Hard
 *
 * You are given an m x n integer matrix grid, where you can move from a cell to any adjacent
 * cell in all 4 directions.
 *
 * Return the number of strictly increasing paths in the grid such that you can start from any
 * cell and end at any cell. Since the answer may be very large, return it modulo 109 + 7.
 *
 * Two paths are considered different if they do not have exactly the same sequence of visited
 * cells.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var countPaths = function(grid) {
  const MOD = 1e9 + 7;
  const rows = grid.length;
  const cols = grid[0].length;
  const cache = new Array(rows).fill().map(() => new Array(cols).fill(0));
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  function explore(row, col) {
    if (cache[row][col]) return cache[row][col];

    let paths = 1;
    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;
      if (
        newRow >= 0 && newRow < rows && newCol >= 0
        && newCol < cols && grid[newRow][newCol] > grid[row][col]
      ) {
        paths = (paths + explore(newRow, newCol)) % MOD;
      }
    }

    return cache[row][col] = paths;
  }

  let total = 0;
  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      total = (total + explore(i, j)) % MOD;
    }
  }

  return total;
};
```

2330 Valid Palindrome Iv

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2330. Valid Palindrome IV
 * https://leetcode.com/problems/valid-palindrome-iv/
```

```
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s consisting of only lowercase English letters.
 * In one operation, you can change any character of s to any other character.
 *
 * Return true if you can make s a palindrome after performing exactly one or two
 * operations, or return false otherwise.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var makePalindrome = function(s) {
  const n = s.length;
  let mismatches = 0;

  for (let i = 0; i < Math.floor(n / 2); i++) {
    if (s[i] !== s[n - 1 - i]) {
      mismatches++;
    }
  }

  return mismatches <= 2;
};
```

2331 Evaluate Boolean Binary Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2331. Evaluate Boolean Binary Tree
 * https://leetcode.com/problems/evaluate-boolean-binary-tree/
 * Difficulty: Easy
 *
 * You are given the root of a full binary tree with the following properties:
 * - Leaf nodes have either the value 0 or 1, where 0 represents False and 1 represents True.
 * - Non-leaf nodes have either the value 2 or 3, where 2 represents the boolean OR and 3
represents
 *   the boolean AND.
 *
 * The evaluation of a node is as follows:
 * - If the node is a leaf node, the evaluation is the value of the node, i.e. True or False.
 * - Otherwise, evaluate the node's two children and apply the boolean operation of its value
with
 *   the children's evaluations.
 *
 * Return the boolean result of evaluating the root node.
 *
 * A full binary tree is a binary tree where each node has either 0 or 2 children.
 *
 * A leaf node is a node that has zero children.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var evaluateTree = function(root) {
  if (!root.left && !root.right) return root.val === 1;

  const leftResult = evaluateTree(root.left);
```

```
  const rightResult = evaluateTree(root.right);

  return root.val === 2 ? leftResult || rightResult : leftResult && rightResult;
};
```

2332 The Latest Time To Catch A Bus

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2332. The Latest Time to Catch a Bus
 * https://leetcode.com/problems/the-latest-time-to-catch-a-bus/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array buses of length n, where buses[i] represents the
 * departure time of the ith bus. You are also given a 0-indexed integer array passengers of
 * length m, where passengers[j] represents the arrival time of the jth passenger. All bus
 * departure times are unique. All passenger arrival times are unique.
 *
 * You are given an integer capacity, which represents the maximum number of passengers that
 * can get on each bus.
 *
 * When a passenger arrives, they will wait in line for the next available bus. You can get
 * on a bus that departs at x minutes if you arrive at y minutes where y <= x, and the bus
 * is not full. Passengers with the earliest arrival times get on the bus first.
 *
 * More formally when a bus arrives, either:
 * - If capacity or fewer passengers are waiting for a bus, they will all get on the bus, or
 * - The capacity passengers with the earliest arrival times will get on the bus.
 *
 * Return the latest time you may arrive at the bus station to catch a bus. You cannot arrive
 * at the same time as another passenger.
 *
 * Note: The arrays buses and passengers are not necessarily sorted.
 */

/**
 * @param {number[]} buses
 * @param {number[]} passengers
 * @param {number} capacity
 * @return {number}
 */
var latestTimeCatchTheBus = function(buses, passengers, capacity) {
  buses.sort((a, b) => a - b);
  passengers.sort((a, b) => a - b);

  let passengerIndex = 0;
  let lastBusPassengers = [];
  for (const busTime of buses) {
    const currentBusPassengers = [];

    while (passengerIndex < passengers.length && passengers[passengerIndex] <= busTime
          && currentBusPassengers.length < capacity) {
      currentBusPassengers.push(passengers[passengerIndex]);
      passengerIndex++;
    }

    lastBusPassengers = currentBusPassengers;
  }

  const lastBusTime = buses[buses.length - 1];
  let latestTime = lastBusTime;
  if (lastBusPassengers.length === capacity) {
    latestTime = lastBusPassengers[lastBusPassengers.length - 1] - 1;
  }

  const passengerSet = new Set(passengers);
  while (passengerSet.has(latestTime)) {
    latestTime--;
```

```
  }

  return latestTime;
};
```

2333 Minimum Sum Of Squared Difference

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2333. Minimum Sum of Squared Difference
 * https://leetcode.com/problems/minimum-sum-of-squared-difference/
 * Difficulty: Medium
 *
 * You are given two positive 0-indexed integer arrays nums1 and nums2, both of length n.
 *
 * The sum of squared difference of arrays nums1 and nums2 is defined as the sum of
 * (nums1[i] - nums2[i])2 for each 0 <= i < n.
 *
 * You are also given two positive integers k1 and k2. You can modify any of the elements
 * of nums1 by +1 or -1 at most k1 times. Similarly, you can modify any of the elements
 * of nums2 by +1 or -1 at most k2 times.
 *
 * Return the minimum sum of squared difference after modifying array nums1 at most k1
 * times and modifying array nums2 at most k2 times.
 *
 * Note: You are allowed to modify the array elements to become negative integers.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @param {number} k1
 * @param {number} k2
 * @return {number}
 */
var minSumSquareDiff = function(nums1, nums2, k1, k2) {
  const n = nums1.length;
  const frequencyMap = new Map();

  for (let i = 0; i < n; i++) {
    const diff = Math.abs(nums1[i] - nums2[i]);
    frequencyMap.set(diff, (frequencyMap.get(diff) || 0) + 1);
  }

  let totalMoves = k1 + k2;
  const maxHeap = new PriorityQueue((a, b) => b[0] - a[0]);

  for (const [value, count] of frequencyMap) {
    if (value === 0) continue;
    maxHeap.enqueue([value, count]);
  }

  while (!maxHeap.isEmpty() && totalMoves > 0) {
    const [value, count] = maxHeap.dequeue();

    if (maxHeap.isEmpty()) {
      const moves = Math.min(totalMoves, count);
      totalMoves -= moves;
      const remainingCount = count - moves;

      if (value - 1 > 0) {
        maxHeap.enqueue([value - 1, moves]);
      }
      if (remainingCount > 0) {
        maxHeap.enqueue([value, remainingCount]);
      }
    } else {
      const moves = Math.min(totalMoves, count);
```

```
      totalMoves -= moves;
      const remainingCount = count - moves;

      if (remainingCount > 0) {
        maxHeap.enqueue([value, remainingCount]);
      }

      if (!maxHeap.isEmpty() && maxHeap.front()[0] === value - 1) {
        const [nextValue, nextCount] = maxHeap.dequeue();
        maxHeap.enqueue([nextValue, nextCount + moves]);
      } else if (value - 1 > 0) {
        maxHeap.enqueue([value - 1, moves]);
      }
    }
  }

  let result = 0;
  while (!maxHeap.isEmpty()) {
    const [value, count] = maxHeap.dequeue();
    result += value * value * count;
  }

  return result;
};
```

2334 Subarray With Elements Greater Than Varying Threshold

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2334. Subarray With Elements Greater Than Varying Threshold
 * https://leetcode.com/problems/subarray-with-elements-greater-than-varying-threshold/
 * Difficulty: Hard
 *
 * You are given an integer array nums and an integer threshold.
 *
 * Find any subarray of nums of length k such that every element in the subarray is greater
 * than threshold / k.
 *
 * Return the size of any such subarray. If there is no such subarray, return -1.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} threshold
 * @return {number}
 */
var validSubarraySize = function(nums, threshold) {
  const n = nums.length;
  const stack = [];
  const nextSmaller = new Array(n).fill(n);
  const prevSmaller = new Array(n).fill(-1);

  for (let i = 0; i < n; i++) {
    while (stack.length && nums[stack[stack.length - 1]] >= nums[i]) {
      nextSmaller[stack.pop()] = i;
    }
    if (stack.length) prevSmaller[i] = stack[stack.length - 1];
    stack.push(i);
  }

  for (let i = 0; i < n; i++) {
    const k = nextSmaller[i] - prevSmaller[i] - 1;
    if (k > 0 && nums[i] > threshold / k) {
      return k;
    }
  }
```

```
  return -1;
};
```

2335 Minimum Amount Of Time To Fill Cups

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2335. Minimum Amount of Time to Fill Cups
 * https://leetcode.com/problems/minimum-amount-of-time-to-fill-cups/
 * Difficulty: Easy
 *
 * You have a water dispenser that can dispense cold, warm, and hot water. Every second, you
 * can either fill up 2 cups with different types of water, or 1 cup of any type of water.
 *
 * You are given a 0-indexed integer array amount of length 3 where amount[0], amount[1], and
 * amount[2] denote the number of cold, warm, and hot water cups you need to fill
respectively.
 * Return the minimum number of seconds needed to fill up all the cups.
 */

/**
 * @param {number[]} amount
 * @return {number}
 */
var fillCups = function(amount) {
  const [a, b, c] = amount.sort((x, y) => y - x);
  return Math.max(a, Math.ceil((a + b + c) / 2));
};
```

2336 Smallest Number In Infinite Set

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2336. Smallest Number in Infinite Set
 * https://leetcode.com/problems/smallest-number-in-infinite-set/
 * Difficulty: Medium
 *
 * You have a set which contains all positive integers [1, 2, 3, 4, 5, ...].
 *
 * Implement the SmallestInfiniteSet class:
 * - SmallestInfiniteSet() Initializes the SmallestInfiniteSet object to contain all
 *   positive integers.
 * - int popSmallest() Removes and returns the smallest integer contained in the
 *   infinite set.
 * - void addBack(int num) Adds a positive integer num back into the infinite set,
 *   if it is not already in the infinite set.
 */

var SmallestInfiniteSet = function() {
  this.set = new Array(1000).fill(1);
};

/**
 * @return {number}
 */
SmallestInfiniteSet.prototype.popSmallest = function() {
  const num = this.set.findIndex(n => n);
  this.set[num] = 0;
  return num + 1;
};

/**
 * @param {number} num
```

```
 * @return {void}
 */
SmallestInfiniteSet.prototype.addBack = function(num) {
  this.set[num - 1] = 1;
};
```

2337 Move Pieces To Obtain A String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2337. Move Pieces to Obtain a String
 * https://leetcode.com/problems/move-pieces-to-obtain-a-string/
 * Difficulty: Medium
 *
 * You are given two strings start and target, both of length n. Each string consists only of
 * the characters 'L', 'R', and '_' where:
 * - The characters 'L' and 'R' represent pieces, where a piece 'L' can move to the left only
if
 *   there is a blank space directly to its left, and a piece 'R' can move to the right only
if
 *   there is a blank space directly to its right.
 * - The character '_' represents a blank space that can be occupied by any of the 'L' or 'R'
 *   pieces.
 *
 * Return true if it is possible to obtain the string target by moving the pieces of the
string
 * start any number of times. Otherwise, return false.
 */

/**
 * @param {string} start
 * @param {string} target
 * @return {boolean}
 */
var canChange = function(start, target) {
  const pieces = [];
  const targetPieces = [];

  for (let i = 0; i < start.length; i++) {
    if (start[i] !== '_') pieces.push([start[i], i]);
    if (target[i] !== '_') targetPieces.push([target[i], i]);
  }

  if (pieces.length !== targetPieces.length) return false;

  for (let i = 0; i < pieces.length; i++) {
    if (pieces[i][0] !== targetPieces[i][0]) return false;
    if (pieces[i][0] === 'L' && pieces[i][1] < targetPieces[i][1]) return false;
    if (pieces[i][0] === 'R' && pieces[i][1] > targetPieces[i][1]) return false;
  }

  return true;
};
```

2338 Count The Number Of Ideal Arrays

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2338. Count the Number of Ideal Arrays
 * https://leetcode.com/problems/count-the-number-of-ideal-arrays/
 * Difficulty: Hard
 *
 * You are given two integers n and maxValue, which are used to describe an ideal array.
```

```
 *
 * A 0-indexed integer array arr of length n is considered ideal if the following conditions
hold:
 * - Every arr[i] is a value from 1 to maxValue, for 0 <= i < n.
 * - Every arr[i] is divisible by arr[i - 1], for 0 < i < n.
 *
 * Return the number of distinct ideal arrays of length n. Since the answer may be very large,
 * return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} maxValue
 * @return {number}
 */
var idealArrays = function(n, maxValue) {
  const MOD = 1e9 + 7;
  const MAX_N = 10010;
  const MAX_P = 15;

  const c = Array.from({ length: MAX_N + MAX_P }, () =>
    new Array(MAX_P + 1).fill(0)
  );
  const sieve = new Array(MAX_N).fill(0);
  const ps = Array.from({ length: MAX_N }, () => []);

  for (let i = 2; i < MAX_N; i++) {
    if (sieve[i] === 0) {
      for (let j = i; j < MAX_N; j += i) {
        if (sieve[j] === 0) {
          sieve[j] = i;
        }
      }
    }
  }

  for (let i = 2; i < MAX_N; i++) {
    let x = i;
    while (x > 1) {
      const p = sieve[x];
      let cnt = 0;
      while (x % p === 0) {
        x = Math.floor(x / p);
        cnt++;
      }
      ps[i].push(cnt);
    }
  }

  c[0][0] = 1;
  for (let i = 1; i < MAX_N + MAX_P; i++) {
    c[i][0] = 1;
    for (let j = 1; j <= Math.min(i, MAX_P); j++) {
      c[i][j] = (c[i - 1][j] + c[i - 1][j - 1]) % MOD;
    }
  }

  let ans = 0n;
  for (let x = 1; x <= maxValue; x++) {
    let mul = 1n;
    for (const p of ps[x]) {
      mul = (mul * BigInt(c[n + p - 1][p])) % BigInt(MOD);
    }
    ans = (ans + mul) % BigInt(MOD);
  }

  return Number(ans);
};
```

2340 Minimum Adjacent Swaps To Make A Valid Array

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2340. Minimum Adjacent Swaps to Make a Valid Array
 * https://leetcode.com/problems/minimum-adjacent-swaps-to-make-a-valid-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums.
 *
 * Swaps of adjacent elements are able to be performed on nums.
 *
 * A valid array meets the following conditions:
 * - The largest element (any of the largest elements if there are multiple) is at the
rightmost
 *   position in the array.
 * - The smallest element (any of the smallest elements if there are multiple) is at the
leftmost
 *   position in the array.
 *
 * Return the minimum swaps required to make nums a valid array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumSwaps = function(nums) {
  const n = nums.length;
  if (n === 1) return 0;

  const minValue = Math.min(...nums);
  const maxValue = Math.max(...nums);
  const minIndex = nums.indexOf(minValue);
  const maxIndex = nums.lastIndexOf(maxValue);
  let result = minIndex + (n - 1 - maxIndex);

  if (minIndex > maxIndex) {
    result -= 1;
  }

  return result;
};
```

2341 Maximum Number Of Pairs In Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2341. Maximum Number of Pairs in Array
 * https://leetcode.com/problems/maximum-number-of-pairs-in-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums. In one operation, you may do the following:
 * - Choose two integers in nums that are equal.
 * - Remove both integers from nums, forming a pair.
 *
 * The operation is done on nums as many times as possible.
 *
 * Return a 0-indexed integer array answer of size 2 where answer[0] is the number of pairs
that
 * are formed and answer[1] is the number of leftover integers in nums after doing the
operation
 * as many times as possible.
```

```
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var numberOfPairs = function(nums) {
  const frequency = new Map();
  let pairs = 0;

  for (const num of nums) {
    frequency.set(num, (frequency.get(num) || 0) + 1);
    if (frequency.get(num) === 2) {
      pairs++;
      frequency.set(num, 0);
    }
  }

  const leftovers = nums.length - pairs * 2;
  return [pairs, leftovers];
};
```

2342 Max Sum Of A Pair With Equal Sum Of Digits

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2342. Max Sum of a Pair With Equal Sum of Digits
 * https://leetcode.com/problems/max-sum-of-a-pair-with-equal-sum-of-digits/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums consisting of positive integers. You can choose two
 * indices i and j, such that i != j, and the sum of digits of the number nums[i] is equal
 * to that of nums[j].
 *
 * Return the maximum value of nums[i] + nums[j] that you can obtain over all possible indices
 * i and j that satisfy the conditions.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumSum = function(nums) {
  const map = new Map();

  return nums.reduce((result, n) => {
    const key = n.toString().split('').reduce((sum, n) => +n + sum, 0);
    result = !map.has(key) ? result : Math.max(result, n + map.get(key));
    map.set(key, Math.max(map.get(key) ?? 0, n));

    return result;
  }, -1);
};
```

2343 Query Kth Smallest Trimmed Number

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2343. Query Kth Smallest Trimmed Number
 * https://leetcode.com/problems/query-kth-smallest-trimmed-number/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of strings nums, where each string is of equal length
```

```
 * and consists of only digits.
 *
 * You are also given a 0-indexed 2D integer array queries where queries[i] = [ki, trimi].
 * For each queries[i], you need to:
 * - Trim each number in nums to its rightmost trimi digits.
 * - Determine the index of the kith smallest trimmed number in nums. If two trimmed numbers
 *   are equal, the number with the lower index is considered to be smaller.
 * - Reset each number in nums to its original length.
 *
 * Return an array answer of the same length as queries, where answer[i] is the answer to
 * the ith query.
 *
 * Note:
 * - To trim to the rightmost x digits means to keep removing the leftmost digit, until only
 *   x digits remain.
 * - Strings in nums may contain leading zeros.
 */

/**
 * @param {string[]} nums
 * @param {number[][]} queries
 * @return {number[]}
 */
var smallestTrimmedNumbers = function(nums, queries) {
  if (nums.length === 0) return [];

  const result = [];
  const stringLength = nums[0].length;
  let startIndex = 0;

  const maxHeap = new PriorityQueue((a, b) => {
    for (let i = startIndex; i < stringLength; i++) {
      if (nums[a].charAt(i) !== nums[b].charAt(i)) {
        return nums[b].charAt(i).charCodeAt(0) - nums[a].charAt(i).charCodeAt(0);
      }
    }
    return b - a;
  });

  for (let i = 0; i < queries.length; i++) {
    startIndex = stringLength - queries[i][1];
    maxHeap.clear();

    for (let j = 0; j < nums.length; j++) {
      maxHeap.enqueue(j);
      if (maxHeap.size() > queries[i][0]) {
        maxHeap.dequeue();
      }
    }

    result[i] = maxHeap.dequeue();
  }

  return result;
};
```

2344 Minimum Deletions To Make Array Divisible

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2344. Minimum Deletions to Make Array Divisible
 * https://leetcode.com/problems/minimum-deletions-to-make-array-divisible/
 * Difficulty: Hard
 *
 * You are given two positive integer arrays nums and numsDivide. You can delete any number
 * of elements from nums.
 *
 * Return the minimum number of deletions such that the smallest element in nums divides
 * all the elements of numsDivide. If this is not possible, return -1.
```

```
 *
 * Note that an integer x divides y if y % x == 0.
 */

/**
 * @param {number[]} nums
 * @param {number[]} numsDivide
 * @return {number}
 */
var minOperations = function(nums, numsDivide) {
  const targetGcd = numsDivide.reduce(gcd);
  nums.sort((a, b) => a - b);

  for (let i = 0; i < nums.length; i++) {
    if (targetGcd % nums[i] === 0) {
      return i;
    }
  }

  return -1;

  function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
  }
};
```

2345 Finding The Number Of Visible Mountains

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2345. Finding the Number of Visible Mountains
 * https://leetcode.com/problems/finding-the-number-of-visible-mountains/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D integer array peaks where peaks[i] = [xi, yi] states that
mountain
 * i has a peak at coordinates (xi, yi). A mountain can be described as a right-angled
isosceles
 * triangle, with its base along the x-axis and a right angle at its peak. More formally, the
 * gradients of ascending and descending the mountain are 1 and -1 respectively.
 *
 * A mountain is considered visible if its peak does not lie within another mountain
(including
 * the border of other mountains).
 *
 * Return the number of visible mountains.
 */

/**
 * @param {number[][]} peaks
 * @return {number}
 */
var visibleMountains = function(peaks) {
  const n = peaks.length;

  peaks.sort((a, b) => {
    const leftA = a[0] - a[1];
    const leftB = b[0] - b[1];
    if (leftA === leftB) {
      return (b[0] + b[1]) - (a[0] + a[1]);
    }
    return leftA - leftB;
  });

  let result = 0;
  let maxEnd = -Infinity;

  for (let i = 0; i < n; i++) {
    const [x, y] = peaks[i];
```

```
    const rightIntercept = x + y;

    if (rightIntercept > maxEnd) {
      maxEnd = rightIntercept;
      if (i < n - 1 && peaks[i][0] === peaks[i + 1][0] && peaks[i][1] === peaks[i + 1][1]) {
        continue;
      }
      result++;
    }
  }

  return result;
};
```

2347 Best Poker Hand

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2347. Best Poker Hand
 * https://leetcode.com/problems/best-poker-hand/
 * Difficulty: Easy
 *
 * You are given an integer array ranks and a character array suits. You have 5 cards where
the
 * ith card has a rank of ranks[i] and a suit of suits[i].
 *
 * The following are the types of poker hands you can make from best to worst:
 * 1. "Flush": Five cards of the same suit.
 * 2. "Three of a Kind": Three cards of the same rank.
 * 3. "Pair": Two cards of the same rank.
 * 4. "High Card": Any single card.
 *
 * Return a string representing the best type of poker hand you can make with the given cards.
 *
 * Note that the return values are case-sensitive.
 */

/**
 * @param {number[]} ranks
 * @param {character[]} suits
 * @return {string}
 */
var bestHand = function(ranks, suits) {
  const suitCount = new Map();
  const rankCount = new Map();

  for (let i = 0; i < 5; i++) {
    suitCount.set(suits[i], (suitCount.get(suits[i]) || 0) + 1);
    rankCount.set(ranks[i], (rankCount.get(ranks[i]) || 0) + 1);
  }

  if (suitCount.size === 1) return 'Flush';

  const maxRankCount = Math.max(...rankCount.values());
  if (maxRankCount >= 3) return 'Three of a Kind';
  if (maxRankCount === 2) return 'Pair';

  return 'High Card';
};
```

2348 Number Of Zero Filled Subarrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2348. Number of Zero-Filled Subarrays
 * https://leetcode.com/problems/number-of-zero-filled-subarrays/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the number of subarrays filled with 0.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var zeroFilledSubarray = function(nums) {
  let result = 0;
  let zeroCount = 0;

  for (const num of nums) {
    if (num === 0) {
      zeroCount++;
      result += zeroCount;
    } else {
      zeroCount = 0;
    }
  }

  return result;
};
```

2349 Design A Number Container System

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2349. Design a Number Container System
 * https://leetcode.com/problems/design-a-number-container-system/
 * Difficulty: Medium
 *
 * Design a number container system that can do the following:
 * - Insert or Replace a number at the given index in the system.
 * - Return the smallest index for the given number in the system.
 *
 * Implement the NumberContainers class:
 * - NumberContainers() Initializes the number container system.
 * - void change(int index, int number) Fills the container at index with the number.
 *   If there is already a number at that index, replace it.
 * - int find(int number) Returns the smallest index for the given number, or -1 if
 *   there is no index that is filled by number in the system.
 */

var NumberContainers = function() {
  this.indexMap = new Map();
  this.lookup = new Map();
};

/**
 * @param {number} index
 * @param {number} number
 * @return {void}
 */
NumberContainers.prototype.change = function(index, number) {
  this.indexMap.set(index, number);
  this.getQueue(number).enqueue(index);
};

/**
 * @param {number} number
 * @return {number}
 */
```

```
NumberContainers.prototype.find = function(number) {
  const queue = this.getQueue(number);
  while (queue.size() && this.indexMap.get(queue.front()) !== number) {
    queue.dequeue();
  }
  return queue.size() ? queue.front() : -1;
};

NumberContainers.prototype.getQueue = function(number) {
  if (!this.lookup.has(number)) {
    this.lookup.set(number, new PriorityQueue({ compare: (a, b) => a - b }));
  }
  return this.lookup.get(number);
};
```

2350 Shortest Impossible Sequence Of Rolls

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2350. Shortest Impossible Sequence of Rolls
 * https://leetcode.com/problems/shortest-impossible-sequence-of-rolls/
 * Difficulty: Hard
 *
 * You are given an integer array rolls of length n and an integer k. You roll a k sided dice
 * numbered from 1 to k, n times, where the result of the ith roll is rolls[i].
 *
 * Return the length of the shortest sequence of rolls so that there's no such subsequence in
rolls.
 *
 * A sequence of rolls of length len is the result of rolling a k sided dice len times.
 */

/**
 * @param {number[]} rolls
 * @param {number} k
 * @return {number}
 */
var shortestSequence = function(rolls, k) {
  const set = new Set();
  let sequences = 0;

  for (const roll of rolls) {
    set.add(roll);
    if (set.size === k) {
      sequences++;
      set.clear();
    }
  }

  return sequences + 1;
};
```

2351 First Letter To Appear Twice

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2351. First Letter to Appear Twice
 * https://leetcode.com/problems/first-letter-to-appear-twice/
 * Difficulty: Easy
 *
 * Given a string s consisting of lowercase English letters, return the first letter
 * to appear twice.
 *
```

```
 * Note:
 * - A letter a appears twice before another letter b if the second occurrence of a is before
 *   the second occurrence of b.
 * - s will contain at least one letter that appears twice.
 */

/**
 * @param {string} s
 * @return {character}
 */
var repeatedCharacter = function(s) {
  const set = new Set();

  for (const char of s) {
    if (set.has(char)) return char;
    set.add(char);
  }
};
```

2352 Equal Row And Column Pairs

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2352. Equal Row and Column Pairs
 * https://leetcode.com/problems/equal-row-and-column-pairs/
 * Difficulty: Medium
 *
 * Given a 0-indexed n x n integer matrix grid, return the number of pairs (ri, cj)
 * such that row ri and column cj are equal.
 *
 * A row and column pair is considered equal if they contain the same elements in the
 * same order (i.e., an equal array).
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var equalPairs = function(grid) {
  const columns = new Map();
  let count = 0;

  for (let i = 0; i < grid.length; i++) {
    const column = [];
    for (let j = 0; j < grid[i].length; j++) {
      column.push(grid[j][i]);
    }
    columns.set(column.join(), (columns.get(column.join()) ?? 0) + 1);
  }

  for (let i = 0; i < grid.length; i++) {
    count += columns.get(grid[i].join()) ?? 0;
  }

  return count;
};
```

2353 Design A Food Rating System

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2353. Design a Food Rating System
```

```
 * https://leetcode.com/problems/design-a-food-rating-system/
 * Difficulty: Medium
 *
 * Design a food rating system that can do the following:
 * - Modify the rating of a food item listed in the system.
 * - Return the highest-rated food item for a type of cuisine in the system.
 *
 * Implement the FoodRatings class:
 * - FoodRatings(String[] foods, String[] cuisines, int[] ratings) Initializes the system.
 *   The food items are described by foods, cuisines and ratings, all of which have a length
 *   of n.
 * - foods[i] is the name of the ith food,
 * - cuisines[i] is the type of cuisine of the ith food, and
 * - ratings[i] is the initial rating of the ith food.
 * - void changeRating(String food, int newRating) Changes the rating of the food item with
 *   the name food.
 * - String highestRated(String cuisine) Returns the name of the food item that has the
 *   highest rating for the given type of cuisine. If there is a tie, return the item with
 *   the lexicographically smaller name.
 *
 * Note that a string x is lexicographically smaller than string y if x comes before y in
 * dictionary order, that is, either x is a prefix of y, or if i is the first position such
 * that x[i] != y[i], then x[i] comes before y[i] in alphabetic order.
 */

/**
 * @param {string[]} foods
 * @param {string[]} cuisines
 * @param {number[]} ratings
 */
var FoodRatings = function(foods, cuisines, ratings) {
  this.foodToCuisine = new Map();
  this.foodToRating = new Map();
  this.cuisineToFoods = new Map();

  for (let i = 0; i < foods.length; i++) {
    const food = foods[i];
    const cuisine = cuisines[i];
    const rating = ratings[i];

    this.foodToCuisine.set(food, cuisine);
    this.foodToRating.set(food, rating);

    if (!this.cuisineToFoods.has(cuisine)) {
      this.cuisineToFoods.set(cuisine, new PriorityQueue((a, b) => {
        if (a.rating !== b.rating) return b.rating - a.rating;
        return a.food.localeCompare(b.food);
      }));
    }

    this.cuisineToFoods.get(cuisine).enqueue({ food, rating });
  }
};

/**
 * @param {string} food
 * @param {number} newRating
 * @return {void}
 */
FoodRatings.prototype.changeRating = function(food, newRating) {
  const cuisine = this.foodToCuisine.get(food);
  this.foodToRating.set(food, newRating);
  this.cuisineToFoods.get(cuisine).enqueue({ food, rating: newRating });
};

/**
 * @param {string} cuisine
 * @return {string}
 */
FoodRatings.prototype.highestRated = function(cuisine) {
  const heap = this.cuisineToFoods.get(cuisine);

  while (!heap.isEmpty()) {
    const top = heap.front();
    if (this.foodToRating.get(top.food) === top.rating) {
```

```
      return top.food;
    }
    heap.dequeue();
  }
};
```

2354 Number Of Excellent Pairs

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2354. Number of Excellent Pairs
 * https://leetcode.com/problems/number-of-excellent-pairs/
 * Difficulty: Hard
 *
 * You are given a 0-indexed positive integer array nums and a positive integer k.
 *
 * A pair of numbers (num1, num2) is called excellent if the following conditions are
satisfied:
 * - Both the numbers num1 and num2 exist in the array nums.
 * - The sum of the number of set bits in num1 OR num2 and num1 AND num2 is greater than or
equal
 *   to k, where OR is the bitwise OR operation and AND is the bitwise AND operation.
 *
 * Return the number of distinct excellent pairs.
 *
 * Two pairs (a, b) and (c, d) are considered distinct if either a != c or b != d. For
example,
 * (1, 2) and (2, 1) are distinct.
 *
 * Note that a pair (num1, num2) such that num1 == num2 can also be excellent if you have at
least
 * one occurrence of num1 in the array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countExcellentPairs = function(nums, k) {
  const map = new Map();
  const set = new Set(nums);

  for (const num of set) {
    const bits = num.toString(2).split('0').join('').length;
    map.set(bits, (map.get(bits) || 0) + 1);
  }

  let result = 0;
  for (const [i, countI] of map) {
    for (const [j, countJ] of map) {
      if (i + j >= k) {
        result += countI * countJ;
      }
    }
  }

  return result;
};
```

2355 Maximum Number Of Books You Can Take

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2355. Maximum Number of Books You Can Take
 * https://leetcode.com/problems/maximum-number-of-books-you-can-take/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array books of length n where books[i] denotes the number
 of
 * books on the ith shelf of a bookshelf.
 *
 * You are going to take books from a contiguous section of the bookshelf spanning from l to r
 * where 0 <= l <= r < n. For each index i in the range l <= i < r, you must take strictly
fewer
 * books from shelf i than shelf i + 1.
 *
 * Return the maximum number of books you can take from the bookshelf.
 */

/**
 * @param {number[]} books
 * @return {number}
 */
var maximumBooks = function(books) {
  const n = books.length;
  const dp = new Array(n).fill(0);
  const stack = [];

  for (let i = 0; i < n; i++) {
    while (stack.length > 0
           && books[stack[stack.length - 1]] >= books[i] - (i - stack[stack.length - 1])) {
      stack.pop();
    }

    if (stack.length === 0) {
      const length = Math.min(i + 1, books[i]);
      dp[i] = (length * (2 * books[i] - length + 1)) / 2;
    } else {
      const prevIndex = stack[stack.length - 1];
      const length = i - prevIndex;
      dp[i] = dp[prevIndex] + (length * (2 * books[i] - length + 1)) / 2;
    }

    stack.push(i);
  }

  return Math.max(...dp);
};
```

2357 Make Array Zero By Subtracting Equal Amounts

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 2357. Make Array Zero by Subtracting Equal Amounts
 * https://leetcode.com/problems/make-array-zero-by-subtracting-equal-amounts/
 * Difficulty: Easy
 *
 * You are given a non-negative integer array nums. In one operation, you must:
 * - Choose a positive integer x such that x is less than or equal to the smallest
 *   non-zero element in nums.
 * - Subtract x from every positive element in nums.
 *
 * Return the minimum number of operations to make every element in nums equal to 0.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumOperations = function(nums) {
  return new Set(nums.filter(num => num > 0)).size;
```

```
};
```

2358 Maximum Number Of Groups Entering A Competition

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2358. Maximum Number of Groups Entering a Competition
 * https://leetcode.com/problems/maximum-number-of-groups-entering-a-competition/
 * Difficulty: Medium
 *
 * You are given a positive integer array grades which represents the grades of students in
 * a university. You would like to enter all these students into a competition in ordered
 * non-empty groups, such that the ordering meets the following conditions:
 * - The sum of the grades of students in the ith group is less than the sum of the grades of
 *   students in the (i + 1)th group, for all groups (except the last).
 * - The total number of students in the ith group is less than the total number of students
 *   in the (i + 1)th group, for all groups (except the last).
 *
 * Return the maximum number of groups that can be formed.
 */

/**
 * @param {number[]} grades
 * @return {number}
 */
var maximumGroups = function(grades) {
  let result = 0;
  let studentsUsed = 0;

  while (studentsUsed + result + 1 <= grades.length) {
    result++;
    studentsUsed += result;
  }

  return result;
};
```

2359 Find Closest Node To Given Two Nodes

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2359. Find Closest Node to Given Two Nodes
 * https://leetcode.com/problems/find-closest-node-to-given-two-nodes/
 * Difficulty: Medium
 *
 * You are given a directed graph of n nodes numbered from 0 to n - 1, where each node has at
 * most one outgoing edge.
 *
 * The graph is represented with a given 0-indexed array edges of size n, indicating that
there
 * is a directed edge from node i to node edges[i]. If there is no outgoing edge from i, then
 * edges[i] == -1.
 *
 * You are also given two integers node1 and node2.
 *
 * Return the index of the node that can be reached from both node1 and node2, such that the
maximum
 * between the distance from node1 to that node, and from node2 to that node is minimized. If
there
 * are multiple answers, return the node with the smallest index, and if no possible answer
exists,
 * return -1.
```

```
 *
 * Note that edges may contain cycles.
 */

/**
 * @param {number[]} edges
 * @param {number} node1
 * @param {number} node2
 * @return {number}
 */
var closestMeetingNode = function(edges, node1, node2) {
  const n = edges.length;
  const dist1 = new Array(n).fill(Infinity);
  const dist2 = new Array(n).fill(Infinity);

  computeDistances(node1, dist1);
  computeDistances(node2, dist2);

  let minDist = Infinity;
  let result = -1;

  for (let i = 0; i < n; i++) {
    if (dist1[i] !== Infinity && dist2[i] !== Infinity) {
      const maxDist = Math.max(dist1[i], dist2[i]);
      if (maxDist < minDist) {
        minDist = maxDist;
        result = i;
      }
    }
  }

  return result;

  function computeDistances(start, distances) {
    let current = start;
    let steps = 0;
    const visited = new Set();

    while (current !== -1 && !visited.has(current)) {
      distances[current] = steps++;
      visited.add(current);
      current = edges[current];
    }
  }
};
```

2360 Longest Cycle In A Graph

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2360. Longest Cycle in a Graph
 * https://leetcode.com/problems/longest-cycle-in-a-graph/
 * Difficulty: Hard
 *
 * You are given a directed graph of n nodes numbered from 0 to n - 1, where each node has at
 * most one outgoing edge.
 *
 * The graph is represented with a given 0-indexed array edges of size n, indicating that
 * there is a directed edge from node i to node edges[i]. If there is no outgoing edge from
 * node i, then edges[i] == -1.
 *
 * Return the length of the longest cycle in the graph. If no cycle exists, return -1.
 *
 * A cycle is a path that starts and ends at the same node.
 */

/**
 * @param {number[]} edges
 * @return {number}
```

```
 */
var longestCycle = function(edges) {
  const n = edges.length;
  const visited = new Array(n).fill(false);
  let result = -1;

  for (let i = 0; i < n; i++) {
    if (!visited[i]) {
      findCycle(i, 0, []);
    }
  }

  return result;

  function findCycle(node, steps, path) {
    if (visited[node]) {
      const cycleStart = path.indexOf(node);
      if (cycleStart !== -1) {
        result = Math.max(result, steps - cycleStart);
      }
      return;
    }

    visited[node] = true;
    path.push(node);

    if (edges[node] !== -1) {
      findCycle(edges[node], steps + 1, path);
    }

    path.pop();
  }
};
```

2361 Minimum Costs Using The Train Line

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2361. Minimum Costs Using the Train Line
 * https://leetcode.com/problems/minimum-costs-using-the-train-line/
 * Difficulty: Hard
 *
 * A train line going through a city has two routes, the regular route and the express route.
 * Both routes go through the same n + 1 stops labeled from 0 to n. Initially, you start on
 * the regular route at stop 0.
 *
 * You are given two 1-indexed integer arrays regular and express, both of length n.
 * regular[i] describes the cost it takes to go from stop i - 1 to stop i using the regular
 * route, and express[i] describes the cost it takes to go from stop i - 1 to stop i using
 * the express route.
 *
 * You are also given an integer expressCost which represents the cost to transfer from the
 * regular route to the express route.
 *
 * Note that:
 * - There is no cost to transfer from the express route back to the regular route.
 * - You pay expressCost every time you transfer from the regular route to the express route.
 * - There is no extra cost to stay on the express route.
 *
 * Return a 1-indexed array costs of length n, where costs[i] is the minimum cost to reach
 * stop i from stop 0.
 *
 * Note that a stop can be counted as reached from either route.
 */

/**
 * @param {number[]} regular
 * @param {number[]} express
 * @param {number} expressCost
```

```
 * @return {number[]}
 */
var minimumCosts = function(regular, express, expressCost) {
  const n = regular.length;
  const result = new Array(n);
  let regularDp = 0;
  let expressDp = expressCost;

  for (let i = 1; i <= n; i++) {
    const newRegularDp = Math.min(regularDp, expressDp) + regular[i - 1];
    const newExpressDp = Math.min(regularDp + expressCost, expressDp) + express[i - 1];

    regularDp = newRegularDp;
    expressDp = newExpressDp;
    result[i - 1] = Math.min(regularDp, expressDp);
  }

  return result;
};
```

2363 Merge Similar Items

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2363. Merge Similar Items
 * https://leetcode.com/problems/merge-similar-items/
 * Difficulty: Easy
 *
 * You are given two 2D integer arrays, items1 and items2, representing two sets of items.
 * Each array items has the following properties:
 * - items[i] = [valuei, weighti] where valuei represents the value and weighti represents
 *   the weight of the ith item.
 * - The value of each item in items is unique.
 *
 * Return a 2D integer array ret where ret[i] = [valuei, weighti], with weighti being the sum
of
 * weights of all items with value valuei.
 *
 * Note: ret should be returned in ascending order by value.
 */

/**
 * @param {number[][]} items1
 * @param {number[][]} items2
 * @return {number[][]}
 */
var mergeSimilarItems = function(items1, items2) {
  const map = new Map();

  for (const [value, weight] of items1) {
    map.set(value, (map.get(value) || 0) + weight);
  }

  for (const [value, weight] of items2) {
    map.set(value, (map.get(value) || 0) + weight);
  }

  const merged = [];
  for (const [value, weight] of map) {
    merged.push([value, weight]);
  }

  return merged.sort((a, b) => a[0] - b[0]);
};
```

2364 Count Number Of Bad Pairs

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2364. Count Number of Bad Pairs
 * https://leetcode.com/problems/count-number-of-bad-pairs/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. A pair of indices (i, j) is a bad pair
 * if i < j and j - i != nums[j] - nums[i].
 *
 * Return the total number of bad pairs in nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countBadPairs = function(nums) {
  const map = new Map();
  let result = nums.length * (nums.length - 1) / 2;

  nums.forEach((n, i) => {
    result -= map.get(n - i) ?? 0;
    map.set(n - i, (map.get(n - i) ?? 0) + 1);
  });

  return result;
};
```

2365 Task Scheduler Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2365. Task Scheduler II
 * https://leetcode.com/problems/task-scheduler-ii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of positive integers tasks, representing tasks that need
 * to be completed in order, where tasks[i] represents the type of the ith task.
 *
 * You are also given a positive integer space, which represents the minimum number of days
 * that must pass after the completion of a task before another task of the same type can be
 * performed.
 *
 * Each day, until all tasks have been completed, you must either:
 * - Complete the next task from tasks, or
 * - Take a break.
 *
 * Return the minimum number of days needed to complete all tasks.
 */

/**
 * @param {number[]} tasks
 * @param {number} space
 * @return {number}
 */
var taskSchedulerII = function(tasks, space) {
  const map = new Map();
  let result = 0;

  for (const task of tasks) {
    result++;
```

```
    if (map.has(task)) {
      const lastDay = map.get(task);
      if (result - lastDay <= space) {
        result = lastDay + space + 1;
      }
    }
    map.set(task, result);
  }

  return result;
};
```

2366 Minimum Replacements To Sort The Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2366. Minimum Replacements to Sort the Array
 * https://leetcode.com/problems/minimum-replacements-to-sort-the-array/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums. In one operation you can replace any element
 * of the array with any two elements that sum to it.
 * - For example, consider nums = [5,6,7]. In one operation, we can replace nums[1] with 2 and
 4
 *   and convert nums to [5,2,4,7].
 *
 * Return the minimum number of operations to make an array that is sorted in non-decreasing
order.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumReplacement = function(nums) {
  let result = 0;
  let prev = nums[nums.length - 1];

  for (let i = nums.length - 2; i >= 0; i--) {
    if (nums[i] > prev) {
      const splits = Math.ceil(nums[i] / prev);
      result += splits - 1;
      prev = Math.floor(nums[i] / splits);
    } else {
      prev = nums[i];
    }
  }

  return result;
};
```

2367 Number Of Arithmetic Triplets

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2367. Number of Arithmetic Triplets
 * https://leetcode.com/problems/number-of-arithmetic-triplets/
 * Difficulty: Easy
 *
 * You are given a 0-indexed, strictly increasing integer array nums and a positive integer
diff.
 * A triplet (i, j, k) is an arithmetic triplet if the following conditions are met:
 * - i < j < k,
```

```
 * - nums[j] - nums[i] == diff, and
 * - nums[k] - nums[j] == diff.
 *
 * Return the number of unique arithmetic triplets.
 */

/**
 * @param {number[]} nums
 * @param {number} diff
 * @return {number}
 */
var arithmeticTriplets = function(nums, diff) {
  let result = 0;

  for (let j = 1; j < nums.length - 1; j++) {
    let i = j - 1;
    let k = j + 1;

    while (i >= 0 && nums[j] - nums[i] <= diff) {
      if (nums[j] - nums[i] === diff) {
        while (k < nums.length && nums[k] - nums[j] <= diff) {
          if (nums[k] - nums[j] === diff) {
            result++;
          }
          k++;
        }
      }
      i--;
    }
  }

  return result;
};
```

2368 Reachable Nodes With Restrictions

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2368. Reachable Nodes With Restrictions
 * https://leetcode.com/problems/reachable-nodes-with-restrictions/
 * Difficulty: Medium
 *
 * There is an undirected tree with n nodes labeled from 0 to n - 1 and n - 1 edges.
 *
 * You are given a 2D integer array edges of length n - 1 where edges[i] = [ai, bi] indicates
 * that there is an edge between nodes ai and bi in the tree. You are also given an integer
 * array restricted which represents restricted nodes.
 *
 * Return the maximum number of nodes you can reach from node 0 without visiting a restricted
node.
 *
 * Note that node 0 will not be a restricted node.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[]} restricted
 * @return {number}
 */
var reachableNodes = function(n, edges, restricted) {
  const graph = Array(n).fill().map(() => []);
  const restrictedSet = new Set(restricted);
  const visited = new Set();

  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }
```

```
  explore(0);
  return visited.size;

  function explore(node) {
    if (visited.has(node) || restrictedSet.has(node)) return;
    visited.add(node);
    for (const neighbor of graph[node]) {
      explore(neighbor);
    }
  }
};
```

2369 Check If There Is A Valid Partition For The Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2369. Check if There is a Valid Partition For The Array
 * https://leetcode.com/problems/check-if-there-is-a-valid-partition-for-the-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. You have to partition the array into one or
 * more contiguous subarrays.
 *
 * We call a partition of the array valid if each of the obtained subarrays satisfies one of
 * the following conditions:
 * 1. The subarray consists of exactly 2, equal elements. For example, the subarray [2,2] is
good.
 * 2. The subarray consists of exactly 3, equal elements. For example, the subarray [4,4,4] is
 good.
 * 3. The subarray consists of exactly 3 consecutive increasing elements, that is, the
difference
 *    between adjacent elements is 1. For example, the subarray [3,4,5] is good, but the
subarray
 *    [1,3,5] is not.
 *
 * Return true if the array has at least one valid partition. Otherwise, return false.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var validPartition = function(nums) {
  const n = nums.length;
  const dp = new Array(n + 1).fill(false);
  dp[0] = true;

  for (let i = 2; i <= n; i++) {
    if (nums[i - 1] === nums[i - 2]) {
      dp[i] = dp[i] || dp[i - 2];
    }
    if (i >= 3) {
      if (nums[i - 1] === nums[i - 2] && nums[i - 2] === nums[i - 3]) {
        dp[i] = dp[i] || dp[i - 3];
      }
      if (nums[i - 1] === nums[i - 2] + 1 && nums[i - 2] === nums[i - 3] + 1) {
        dp[i] = dp[i] || dp[i - 3];
      }
    }
  }

  return dp[n];
};
```

2370 Longest Ideal Subsequence

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2370. Longest Ideal Subsequence
 * https://leetcode.com/problems/longest-ideal-subsequence/
 * Difficulty: Medium
 *
 * You are given a string s consisting of lowercase letters and an integer k. We call a string
 * t ideal if the following conditions are satisfied:
 * - t is a subsequence of the string s.
 * - The absolute difference in the alphabet order of every two adjacent letters in t is less
 *   than or equal to k.
 *
 * Return the length of the longest ideal string.
 *
 * A subsequence is a string that can be derived from another string by deleting some or no
 * characters without changing the order of the remaining characters.
 *
 * Note that the alphabet order is not cyclic. For example, the absolute difference in the
 * alphabet order of 'a' and 'z' is 25, not 1.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var longestIdealString = function(s, k) {
  const dp = new Array(26).fill(0);
  let result = 0;

  for (const char of s) {
    const idx = char.charCodeAt(0) - 97;
    let currentMax = 0;

    for (let i = Math.max(0, idx - k); i <= Math.min(25, idx + k); i++) {
      currentMax = Math.max(currentMax, dp[i]);
    }

    dp[idx] = currentMax + 1;
    result = Math.max(result, dp[idx]);
  }

  return result;
};
```

2371 Minimize Maximum Value In A Grid

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2371. Minimize Maximum Value in a Grid
 * https://leetcode.com/problems/minimize-maximum-value-in-a-grid/
 * Difficulty: Hard
 *
 * You are given an m x n integer matrix grid containing distinct positive integers.
 *
 * You have to replace each integer in the matrix with a positive integer satisfying the
 * following conditions:
 * - The relative order of every two elements that are in the same row or column should stay
 *   the same after the replacements.
 * - The maximum number in the matrix after the replacements should be as small as possible.
 *
```

```
 * The relative order stays the same if for all pairs of elements in the original matrix such
 * that grid[r1][c1] > grid[r2][c2] where either r1 == r2 or c1 == c2, then it must be true
 * that grid[r1][c1] > grid[r2][c2] after the replacements.
 *
 * For example, if grid = [[2, 4, 5], [7, 3, 9]] then a good replacement could be either
 * grid = [[1, 2, 3], [2, 1, 4]] or grid = [[1, 2, 3], [3, 1, 4]].
 *
 * Return the resulting matrix. If there are multiple answers, return any of them.
 */

/**
 * @param {number[][]} grid
 * @return {number[][]}
 */
var minScore = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;

  const positions = [];
  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      positions.push([grid[i][j], i, j]);
    }
  }

  positions.sort((a, b) => a[0] - b[0]);

  const result = new Array(rows).fill().map(() => new Array(cols).fill(0));
  const rowMax = new Array(rows).fill(0);
  const colMax = new Array(cols).fill(0);
  for (const [value, row, col] of positions) {
    const newValue = Math.max(rowMax[row], colMax[col]) + 1;
    result[row][col] = newValue;
    rowMax[row] = newValue;
    colMax[col] = newValue;
  }

  return result;
};
```

2373 Largest Local Values In A Matrix

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2373. Largest Local Values in a Matrix
 * https://leetcode.com/problems/largest-local-values-in-a-matrix/
 * Difficulty: Easy
 *
 * You are given an n x n integer matrix grid.
 *
 * Generate an integer matrix maxLocal of size (n - 2) x (n - 2) such that:
 * - maxLocal[i][j] is equal to the largest value of the 3 x 3 matrix in grid centered around
 *   row i + 1 and column j + 1.
 *
 * In other words, we want to find the largest value in every contiguous 3 x 3 matrix in grid.
 *
 * Return the generated matrix.
 */

/**
 * @param {number[][]} grid
 * @return {number[][]}
 */
var largestLocal = function(grid) {
  const n = grid.length;
  const result = new Array(n - 2).fill().map(() => new Array(n - 2).fill(0));

  for (let i = 0; i < n - 2; i++) {
    for (let j = 0; j < n - 2; j++) {
```

```
      let maxVal = 0;
      for (let r = i; r < i + 3; r++) {
        for (let c = j; c < j + 3; c++) {
          maxVal = Math.max(maxVal, grid[r][c]);
        }
      }
      result[i][j] = maxVal;
    }
  }

  return result;
};
```

2374 Node With Highest Edge Score

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2374. Node With Highest Edge Score
 * https://leetcode.com/problems/node-with-highest-edge-score/
 * Difficulty: Medium
 *
 * You are given a directed graph with n nodes labeled from 0 to n - 1, where each node has
 * exactly one outgoing edge.
 *
 * The graph is represented by a given 0-indexed integer array edges of length n, where
edges[i]
 * indicates that there is a directed edge from node i to node edges[i].
 *
 * The edge score of a node i is defined as the sum of the labels of all the nodes that have
an
 * edge pointing to i.
 *
 * Return the node with the highest edge score. If multiple nodes have the same edge score,
return
 * the node with the smallest index.
 */

/**
 * @param {number[]} edges
 * @return {number}
 */
var edgeScore = function(edges) {
  const scores = new Array(edges.length).fill(0);

  for (let i = 0; i < edges.length; i++) {
    scores[edges[i]] += i;
  }
  let maxScore = 0;
  let result = 0;
  for (let i = 0; i < scores.length; i++) {
    if (scores[i] > maxScore) {
      maxScore = scores[i];
      result = i;
    }
  }

  return result;
};
```

2375 Construct Smallest Number From Di String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 2375. Construct Smallest Number From DI String
 * https://leetcode.com/problems/construct-smallest-number-from-di-string/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string pattern of length n consisting of the characters
 * 'I' meaning increasing and 'D' meaning decreasing.
 *
 * A 0-indexed string num of length n + 1 is created using the following conditions:
 * - num consists of the digits '1' to '9', where each digit is used at most once.
 * - If pattern[i] == 'I', then num[i] < num[i + 1].
 * - If pattern[i] == 'D', then num[i] > num[i + 1].
 *
 * Return the lexicographically smallest possible string num that meets the conditions.
 */

/**
 * @param {string} pattern
 * @return {string}
 */
var smallestNumber = function(pattern) {
  let result = '';

  for (let i = 0, stack = []; i <= pattern.length; i++) {
    stack.push(i + 1);
    if (i === pattern.length || pattern[i] === 'I') {
      while (stack.length > 0) {
        result += stack.pop();
      }
    }
  }

  return result;
};
```

2376 Count Special Integers

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2376. Count Special Integers
 * https://leetcode.com/problems/count-special-integers/
 * Difficulty: Hard
 *
 * We call a positive integer special if all of its digits are distinct.
 *
 * Given a positive integer n, return the number of special integers that belong to the
 * interval [1, n].
 */

/**
 * @param {number} n
 * @return {number}
 */
var countSpecialNumbers = function(n) {
  const digits = String(n).split('').map(Number);
  const len = digits.length;
  let total = 0;

  for (let i = 1; i < len; i++) {
    let count = 9;
    for (let j = 0; j < i - 1; j++) {
      count *= (10 - j - 1);
    }
    total += count;
  }

  const used = new Set();
  for (let i = 0; i < len; i++) {
    for (let d = (i === 0 ? 1 : 0); d < digits[i]; d++) {
      if (!used.has(d)) {
```

```
        let count = 1;
        for (let j = i + 1; j < len; j++) {
          count *= (10 - used.size - (j - i));
        }
        total += count;
      }
    }
    if (used.has(digits[i]) || digits[i] === 0 && i === 0) break;
    used.add(digits[i]);
  }

  if (used.size === len) total++;

  return total;
};
```

2378 Choose Edges To Maximize Score In A Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2378. Choose Edges to Maximize Score in a Tree
 * https://leetcode.com/problems/choose-edges-to-maximize-score-in-a-tree/
 * Difficulty: Medium
 *
 * You are given a weighted tree consisting of n nodes numbered from 0 to n - 1.
 *
 * The tree is rooted at node 0 and represented with a 2D array edges of size n where
 * edges[i] = [pari, weighti] indicates that node pari is the parent of node i, and the
 * edge between them has a weight equal to weighti. Since the root does not have a parent,
 * you have edges[0] = [-1, -1].
 *
 * Choose some edges from the tree such that no two chosen edges are adjacent and the sum
 * of the weights of the chosen edges is maximized.
 *
 * Return the maximum sum of the chosen edges.
 *
 * Note:
 * - You are allowed to not choose any edges in the tree, the sum of weights in this case will
 be 0.
 * - Two edges Edge1 and Edge2 in the tree are adjacent if they have a common node.
 *   - In other words, they are adjacent if Edge1 connects nodes a and b and Edge2 connects
nodes
 *     b and c.
 */

/**
 * @param {number[][]} edges
 * @return {number}
 */
var maxScore = function(edges) {
  const n = edges.length;
  const children = new Array(n).fill().map(() => []);
  let root = 0;

  for (let i = 0; i < n; i++) {
    if (edges[i][0] >= 0) {
      children[edges[i][0]].push([i, edges[i][1]]);
    } else {
      root = i;
    }
  }

  const [skipRoot, takeRoot] = dfs(root);
  return Math.max(skipRoot, takeRoot);

  function dfs(node) {
    let skip = 0;
    let take = 0;
```

```
    for (const [child, weight] of children[node]) {
      const [childSkip, childTake] = dfs(child);
      skip += childTake;
      take = Math.max(take, childSkip - childTake + weight);
    }

    return [skip, take + skip];
  }
};
```

2379 Minimum Recolors To Get K Consecutive Black Blocks

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2379. Minimum Recolors to Get K Consecutive Black Blocks
 * https://leetcode.com/problems/minimum-recolors-to-get-k-consecutive-black-blocks/
 * Difficulty: Easy
 *
 * You are given a 0-indexed string blocks of length n, where blocks[i] is either 'W' or 'B',
 * representing the color of the ith block. The characters 'W' and 'B' denote the colors white
 * and black, respectively.
 *
 * You are also given an integer k, which is the desired number of consecutive black blocks.
 *
 * In one operation, you can recolor a white block such that it becomes a black block.
 *
 * Return the minimum number of operations needed such that there is at least one occurrence
 * of k consecutive black blocks.
 */

/**
 * @param {string} blocks
 * @param {number} k
 * @return {number}
 */
var minimumRecolors = function(blocks, k) {
  let count = 0;

  for (let i = 0; i < k; i++) {
    if (blocks[i] === 'W') count++;
  }

  let result = count;

  for (let i = k; i < blocks.length; i++) {
    if (blocks[i] === 'W') count++;
    if (blocks[i - k] === 'W') count--;
    result = Math.min(result, count);
  }

  return result;
};
```

2380 Time Needed To Rearrange A Binary String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2380. Time Needed to Rearrange a Binary String
 * https://leetcode.com/problems/time-needed-to-rearrange-a-binary-string/
 * Difficulty: Medium
 *
 * You are given a binary string s. In one second, all occurrences of "01" are simultaneously
 * replaced with "10". This process repeats until no occurrences of "01" exist.
```

```
 *
 * Return the number of seconds needed to complete this process.
 */

/**
 * @param {string} s
 * @return {number}
 */
var secondsToRemoveOccurrences = function(s) {
  const binary = s.split('');
  let result = 0;

  while (binary.join('').includes('01')) {
    for (let i = 0; i < binary.length - 1; i++) {
      if (binary[i] === '0' && binary[i + 1] === '1') {
        binary[i] = '1';
        binary[i + 1] = '0';
        i++;
      }
    }
    result++;
  }

  return result;
};
```

2381 Shifting Letters Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2381. Shifting Letters II
 * https://leetcode.com/problems/shifting-letters-ii/
 * Difficulty: Medium
 *
 * You are given a string s of lowercase English letters and a 2D integer array shifts where
 * shifts[i] = [starti, endi, directioni]. For every i, shift the characters in s from the
 * index starti to the index endi (inclusive) forward if directioni = 1, or shift the
characters
 * backward if directioni = 0.
 *
 * Shifting a character forward means replacing it with the next letter in the alphabet
(wrapping
 * around so that 'z' becomes 'a'). Similarly, shifting a character backward means replacing
it
 * with the previous letter in the alphabet (wrapping around so that 'a' becomes 'z').
 *
 * Return the final string after all such shifts to s are applied.
 */

/**
 * @param {string} s
 * @param {number[][]} shifts
 * @return {string}
 */
var shiftingLetters = function(s, shifts) {
  const compare = new Array(s.length).fill(0);
  for (const [start, end, direction] of shifts) {
    compare[start] += direction === 1 ? 1 : -1;
    if (end + 1 < s.length) {
      compare[end + 1] += direction === 1 ? -1 : 1;
    }
  }
  let result = '';
  for (let i = 0, count = 0; i < s.length; i++) {
    count = (count + compare[i]) % 26;
    count = count < 0 ? count + 26 : count;
    result += String.fromCharCode((s.charCodeAt(i) - 97 + count) % 26 + 97);
  }
  return result;
```

```
};
```

2382 Maximum Segment Sum After Removals

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2382. Maximum Segment Sum After Removals
 * https://leetcode.com/problems/maximum-segment-sum-after-removals/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays nums and removeQueries, both of length n. For
 * the ith query, the element in nums at the index removeQueries[i] is removed, splitting
 * nums into different segments.
 *
 * A segment is a contiguous sequence of positive integers in nums. A segment sum is the sum
 * of every element in a segment.
 *
 * Return an integer array answer, of length n, where answer[i] is the maximum segment sum
after
 * applying the ith removal.
 *
 * Note: The same index will not be removed more than once.
 */

/**
 * @param {number[]} nums
 * @param {number[]} removeQueries
 * @return {number[]}
 */
var maximumSegmentSum = function(nums, removeQueries) {
  const n = nums.length;
  const result = new Array(n);
  const prefixSum = new Array(n + 1).fill(0);
  const parents = new Array(n).fill(-1);
  const sizes = new Array(n).fill(0);
  const sums = new Array(n).fill(0);
  let maxSum = 0;

  for (let i = 0; i < n; i++) {
    prefixSum[i + 1] = prefixSum[i] + nums[i];
  }

  for (let i = n - 1; i >= 0; i--) {
    result[i] = maxSum;
    const index = removeQueries[i];

    parents[index] = index;
    sizes[index] = 1;
    sums[index] = nums[index];

    if (index > 0 && parents[index - 1] !== -1) {
      const leftRoot = findRoot(parents, index - 1);
      const segmentSum = sums[leftRoot] + sums[index];

      parents[index] = leftRoot;
      sizes[leftRoot] += sizes[index];
      sums[leftRoot] = segmentSum;
    }

    if (index < n - 1 && parents[index + 1] !== -1) {
      const root = findRoot(parents, index);
      const rightRoot = findRoot(parents, index + 1);

      if (root !== rightRoot) {
        const segmentSum = sums[root] + sums[rightRoot];

        if (sizes[root] < sizes[rightRoot]) {
          parents[root] = rightRoot;
          sizes[rightRoot] += sizes[root];
```

```
          sums[rightRoot] = segmentSum;
        } else {
          parents[rightRoot] = root;
          sizes[root] += sizes[rightRoot];
          sums[root] = segmentSum;
        }
      }
    }

    maxSum = Math.max(maxSum, sums[findRoot(parents, index)]);
  }

  return result;
};

function findRoot(parents, x) {
  if (parents[x] !== x) {
    parents[x] = findRoot(parents, parents[x]);
  }
  return parents[x];
}
```

2383 Minimum Hours Of Training To Win A Competition

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2383. Minimum Hours of Training to Win a Competition
 * https://leetcode.com/problems/minimum-hours-of-training-to-win-a-competition/
 * Difficulty: Easy
 *
 * You are entering a competition, and are given two positive integers initialEnergy and
 * initialExperience denoting your initial energy and initial experience respectively.
 *
 * You are also given two 0-indexed integer arrays energy and experience, both of length n.
 *
 * You will face n opponents in order. The energy and experience of the ith opponent is
 * denoted by energy[i] and experience[i] respectively. When you face an opponent, you need
 * to have both strictly greater experience and energy to defeat them and move to the next
 * opponent if available.
 *
 * Defeating the ith opponent increases your experience by experience[i], but decreases your
 * energy by energy[i].
 *
 * Before starting the competition, you can train for some number of hours. After each hour
 * of training, you can either choose to increase your initial experience by one, or increase
 * your initial energy by one.
 *
 * Return the minimum number of training hours required to defeat all n opponents.
 */

/**
 * @param {number} initialEnergy
 * @param {number} initialExperience
 * @param {number[]} energy
 * @param {number[]} experience
 * @return {number}
 */
var minNumberOfHours = function(initialEnergy, initialExperience, energy, experience) {
  let energyNeeded = 0;
  let experienceNeeded = 0;
  let currentEnergy = initialEnergy;
  let currentExperience = initialExperience;

  for (let i = 0; i < energy.length; i++) {
    if (currentEnergy <= energy[i]) {
      const deficit = energy[i] - currentEnergy + 1;
      energyNeeded += deficit;
      currentEnergy += deficit;
    }
```

```
    if (currentExperience <= experience[i]) {
      const deficit = experience[i] - currentExperience + 1;
      experienceNeeded += deficit;
      currentExperience += deficit;
    }
    currentEnergy -= energy[i];
    currentExperience += experience[i];
  }

  return energyNeeded + experienceNeeded;
};
```

2384 Largest Palindromic Number

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2384. Largest Palindromic Number
 * https://leetcode.com/problems/largest-palindromic-number/
 * Difficulty: Medium
 *
 * You are given a string num consisting of digits only.
 *
 * Return the largest palindromic integer (in the form of a string) that can be formed
 * using digits taken from num. It should not contain leading zeroes.
 *
 * Notes:
 * - You do not need to use all the digits of num, but you must use at least one digit.
 * - The digits can be reordered.
 */

/**
 * @param {string} num
 * @return {string}
 */
var largestPalindromic = function(num) {
  const digitCount = new Array(10).fill(0);

  for (const digit of num) {
    digitCount[parseInt(digit, 10)]++;
  }

  let leftHalf = '';
  let middle = '';
  for (let digit = 9; digit >= 0; digit--) {
    const count = digitCount[digit];
    const pairs = Math.floor(count / 2);

    if (digit === 0 && leftHalf === '') {
      continue;
    }

    leftHalf += digit.toString().repeat(pairs);

    if (count % 2 === 1 && middle === '') {
      middle = digit.toString();
    }
  }

  if (leftHalf === '' && middle === '') {
    return '0';
  }

  const rightHalf = leftHalf.split('').reverse().join('');
  return leftHalf + middle + rightHalf;
};
```

2385 Amount Of Time For Binary Tree To Be Infected

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2385. Amount of Time for Binary Tree to Be Infected
 * https://leetcode.com/problems/amount-of-time-for-binary-tree-to-be-infected/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree with unique values, and an integer start.
 * At minute 0, an infection starts from the node with value start.
 *
 * Each minute, a node becomes infected if:
 * - The node is currently uninfected.
 * - The node is adjacent to an infected node.
 *
 * Return the number of minutes needed for the entire tree to be infected.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} start
 * @return {number}
 */
var amountOfTime = function(root, start) {
  const graph = new Map();
  const queue = [start];
  const visited = new Set([start]);
  let result = -1;

  buildGraph(root, null);

  while (queue.length) {
    result++;
    const levelSize = queue.length;
    for (let i = 0; i < levelSize; i++) {
      const current = queue.shift();
      for (const neighbor of graph.get(current) || []) {
        if (!visited.has(neighbor)) {
          visited.add(neighbor);
          queue.push(neighbor);
        }
      }
    }
  }

  return result;

  function buildGraph(node, parent) {
    if (!node) return;
    if (!graph.has(node.val)) graph.set(node.val, []);
    if (parent) graph.get(node.val).push(parent.val);
    if (node.left) {
      graph.get(node.val).push(node.left.val);
      graph.set(node.left.val, graph.get(node.left.val) || []);
      graph.get(node.left.val).push(node.val);
    }
    if (node.right) {
      graph.get(node.val).push(node.right.val);
      graph.set(node.right.val, graph.get(node.right.val) || []);
      graph.get(node.right.val).push(node.val);
    }
```

```
    buildGraph(node.left, node);
    buildGraph(node.right, node);
  }
};
```

2386 Find The K Sum Of An Array

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2386. Find the K-Sum of an Array
 * https://leetcode.com/problems/find-the-k-sum-of-an-array/
 * Difficulty: Hard
 *
 * You are given an integer array nums and a positive integer k. You can choose any
subsequence
 * of the array and sum all of its elements together.
 *
 * We define the K-Sum of the array as the kth largest subsequence sum that can be obtained
 * (not necessarily distinct).
 *
 * Return the K-Sum of the array.
 *
 * A subsequence is an array that can be derived from another array by deleting some or no
elements
 * without changing the order of the remaining elements.
 *
 * Note that the empty subsequence is considered to have a sum of 0.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var kSum = function(nums, k) {
  const maxSum = nums.reduce((sum, num) => sum + Math.max(0, num), 0);
  const absoluteNums = nums.map(num => Math.abs(num)).sort((a, b) => a - b);
  const maxHeap = new PriorityQueue((a, b) => a[0] - b[0]);
  maxHeap.enqueue([-maxSum + absoluteNums[0], 0]);
  let nextSum = -maxSum;

  for (let iteration = 0; iteration < k - 1; iteration++) {
    [nextSum, i] = maxHeap.dequeue();

    if (i + 1 < absoluteNums.length) {
      maxHeap.enqueue([nextSum - absoluteNums[i] + absoluteNums[i + 1], i + 1]);
      maxHeap.enqueue([nextSum + absoluteNums[i + 1], i + 1]);
    }
  }

  return -nextSum;
};
```

2387 Median Of A Row Wise Sorted Matrix

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2387. Median of a Row Wise Sorted Matrix
 * https://leetcode.com/problems/median-of-a-row-wise-sorted-matrix/
 * Difficulty: Medium
 *
 * Given an m x n matrix grid containing an odd number of integers where each row is sorted
 * in non-decreasing order, return the median of the matrix.
```

```
 *
 * You must solve the problem in less than O(m * n) time complexity.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var matrixMedian = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const totalElements = rows * cols;
  const targetPosition = Math.floor(totalElements / 2) + 1;
  let low = 0;
  let high = 1000001;

  while (low < high) {
    const mid = low + Math.floor((high - low) / 2);
    if (helper(mid)) {
      high = mid;
    } else {
      low = mid + 1;
    }
  }

  return high;

  function helper(target) {
    let count = 0;
    for (const row of grid) {
      let left = 0;
      let right = cols;
      while (left < right) {
        const mid = Math.floor((left + right) / 2);
        if (row[mid] <= target) {
          left = mid + 1;
        } else {
          right = mid;
        }
      }
      count += left;
    }
    return count >= targetPosition;
  }
};
```

2389 Longest Subsequence With Limited Sum

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2389. Longest Subsequence With Limited Sum
 * https://leetcode.com/problems/longest-subsequence-with-limited-sum/
 * Difficulty: Easy
 *
 * You are given an integer array nums of length n, and an integer array queries of length m.
 *
 * Return an array answer of length m where answer[i] is the maximum size of a subsequence
that
 * you can take from nums such that the sum of its elements is less than or equal to
queries[i].
 *
 * A subsequence is an array that can be derived from another array by deleting some or no
elements
 * without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
```

```
 * @param {number[]} queries
 * @return {number[]}
 */
var answerQueries = function(nums, queries) {
  nums.sort((a, b) => a - b);
  const prefixSums = [0];
  for (const num of nums) {
    prefixSums.push(prefixSums.at(-1) + num);
  }

  const result = new Array(queries.length);
  for (let i = 0; i < queries.length; i++) {
    let left = 0;
    let right = nums.length;
    while (left < right) {
      const mid = Math.floor((left + right + 1) / 2);
      if (prefixSums[mid] <= queries[i]) {
        left = mid;
      } else {
        right = mid - 1;
      }
    }
    result[i] = left;
  }

  return result;
};
```

2390 Removing Stars From A String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2390. Removing Stars From a String
 * https://leetcode.com/problems/removing-stars-from-a-string/
 * Difficulty: Medium
 *
 * You are given a string s, which contains stars *.
 *
 * In one operation, you can:
 * - Choose a star in s.
 * - Remove the closest non-star character to its left, as well as remove the star itself.
 *
 * Return the string after all stars have been removed.
 *
 * Note:
 * - The input will be generated such that the operation is always possible.
 * - It can be shown that the resulting string will always be unique.
 */

/**
 * @param {string} s
 * @return {string}
 */
var removeStars = function(s) {
  const result = [];
  for (let i = 0; i < s.length; i++) {
    if (s[i] !== '*') {
      result.push(s[i]);
    } else {
      result.pop();
    }
  }
  return result.join('');
};
```

2391 Minimum Amount Of Time To Collect Garbage

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2391. Minimum Amount of Time to Collect Garbage
 * https://leetcode.com/problems/minimum-amount-of-time-to-collect-garbage/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of strings garbage where garbage[i] represents the
assortment
 * of garbage at the ith house. garbage[i] consists only of the characters 'M', 'P' and 'G'
 * representing one unit of metal, paper and glass garbage respectively. Picking up one unit
of
 * any type of garbage takes 1 minute.
 *
 * You are also given a 0-indexed integer array travel where travel[i] is the number of
minutes
 * needed to go from house i to house i + 1.
 *
 * There are three garbage trucks in the city, each responsible for picking up one type of
garbage.
 * Each garbage truck starts at house 0 and must visit each house in order; however, they do
not
 * need to visit every house.
 *
 * Only one garbage truck may be used at any given moment. While one truck is driving or
picking up
 * garbage, the other two trucks cannot do anything.
 *
 * Return the minimum number of minutes needed to pick up all the garbage.
 */

/**
 * @param {string[]} garbage
 * @param {number[]} travel
 * @return {number}
 */
var garbageCollection = function(garbage, travel) {
  let result = 0;
  const lastHouse = {'M': 0, 'P': 0, 'G': 0};

  for (let i = 0; i < garbage.length; i++) {
    result += garbage[i].length;
    for (const type of garbage[i]) {
      lastHouse[type] = i;
    }
  }

  const prefixTravel = [0];
  for (const time of travel) {
    prefixTravel.push(prefixTravel.at(-1) + time);
  }

  Object.keys(lastHouse).forEach(type => {
    result += prefixTravel[lastHouse[type]];
  });

  return result;
};
```

2392 Build A Matrix With Conditions

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2392. Build a Matrix With Conditions
 * https://leetcode.com/problems/build-a-matrix-with-conditions/
 * Difficulty: Hard
 *
 * You are given a positive integer k. You are also given:
 * - a 2D integer array rowConditions of size n where rowConditions[i] = [abovei, belowi], and
 * - a 2D integer array colConditions of size m where colConditions[i] = [lefti, righti].
 *
 * The two arrays contain integers from 1 to k.
 *
 * You have to build a k x k matrix that contains each of the numbers from 1 to k exactly
once.
 * The remaining cells should have the value 0.
 *
 * The matrix should also satisfy the following conditions:
 * - The number abovei should appear in a row that is strictly above the row at which the
number
 *   belowi appears for all i from 0 to n - 1.
 * - The number lefti should appear in a column that is strictly left of the column at which
the
 *   number righti appears for all i from 0 to m - 1.
 *
 * Return any matrix that satisfies the conditions. If no answer exists, return an empty
matrix.
 */

/**
 * @param {number} k
 * @param {number[][]} rowConditions
 * @param {number[][]} colConditions
 * @return {number[][]}
 */
var buildMatrix = function(k, rowConditions, colConditions) {
  const rowOrder = helper(rowConditions, k);
  if (!rowOrder.length) return [];

  const colOrder = helper(colConditions, k);
  if (!colOrder.length) return [];

  const matrix = Array.from({ length: k }, () => new Array(k).fill(0));
  const rowIndex = new Array(k + 1).fill(0);
  const colIndex = new Array(k + 1).fill(0);

  for (let i = 0; i < k; i++) {
    rowIndex[rowOrder[i]] = i;
    colIndex[colOrder[i]] = i;
  }

  for (let num = 1; num <= k; num++) {
    matrix[rowIndex[num]][colIndex[num]] = num;
  }

  return matrix;

  function helper(edges, size) {
    const graph = Array.from({ length: size + 1 }, () => []);
    const inDegree = new Array(size + 1).fill(0);
    for (const [u, v] of edges) {
      graph[u].push(v);
      inDegree[v]++;
    }

    const queue = [];
    for (let i = 1; i <= size; i++) {
      if (inDegree[i] === 0) queue.push(i);
    }

    const order = [];
    while (queue.length) {
      const node = queue.shift();
      order.push(node);
      for (const next of graph[node]) {
        if (--inDegree[next] === 0) queue.push(next);
      }
```

```
    }

    return order.length === size ? order : [];
  }
};
```

2393 Count Strictly Increasing Subarrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2393. Count Strictly Increasing Subarrays
 * https://leetcode.com/problems/count-strictly-increasing-subarrays/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of positive integers.
 *
 * Return the number of subarrays of nums that are in strictly increasing order.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countSubarrays = function(nums) {
  let result = 0;
  let currentLength = 1;

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] > nums[i - 1]) {
      currentLength++;
    } else {
      result += (currentLength * (currentLength + 1)) / 2;
      currentLength = 1;
    }
  }

  result += (currentLength * (currentLength + 1)) / 2;

  return result;
};
```

2395 Find Subarrays With Equal Sum

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2395. Find Subarrays With Equal Sum
 * https://leetcode.com/problems/find-subarrays-with-equal-sum/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums, determine whether there exist two subarrays of length
 * 2 with equal sum. Note that the two subarrays must begin at different indices.
 *
 * Return true if these subarrays exist, and false otherwise.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
```

```
var findSubarrays = function(nums) {
  const set = new Set();

  for (let i = 0; i < nums.length - 1; i++) {
    const sum = nums[i] + nums[i + 1];
    if (set.has(sum)) return true;
    set.add(sum);
  }

  return false;
};
```

2396 Strictly Palindromic Number

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2396. Strictly Palindromic Number
 * https://leetcode.com/problems/strictly-palindromic-number/
 * Difficulty: Medium
 *
 * An integer n is strictly palindromic if, for every base b between 2 and n - 2 (inclusive),
 * the string representation of the integer n in base b is palindromic.
 *
 * Given an integer n, return true if n is strictly palindromic and false otherwise.
 *
 * A string is palindromic if it reads the same forward and backward.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isStrictlyPalindromic = function(n) {
  return false;
};
```

2397 Maximum Rows Covered By Columns

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2397. Maximum Rows Covered by Columns
 * https://leetcode.com/problems/maximum-rows-covered-by-columns/
 * Difficulty: Medium
 *
 * You are given an m x n binary matrix matrix and an integer numSelect.
 *
 * Your goal is to select exactly numSelect distinct columns from matrix such that you cover
 * as many rows as possible.
 *
 * A row is considered covered if all the 1's in that row are also part of a column that you
 * have selected. If a row does not have any 1s, it is also considered covered.
 *
 * More formally, let us consider selected = {c1, c2, ...., cnumSelect} as the set of columns
 * selected by you. A row i is covered by selected if:
 * - For each cell where matrix[i][j] == 1, the column j is in selected.
 * - Or, no cell in row i has a value of 1.
 *
 * Return the maximum number of rows that can be covered by a set of numSelect columns.
 */

/**
 * @param {number[][]} matrix
 * @param {number} numSelect
```

```
 * @return {number}
 */
var maximumRows = function(matrix, numSelect) {
  const rows = matrix.length;
  const cols = matrix[0].length;
  let result = 0;

  backtrack(0, 0, 0);

  return result;

  function countCovered(selected) {
    let covered = 0;
    for (let i = 0; i < rows; i++) {
      let isCovered = true;
      for (let j = 0; j < cols; j++) {
        if (matrix[i][j] === 1 && !(selected & (1 << j))) {
          isCovered = false;
          break;
        }
      }
      if (isCovered) covered++;
    }
    return covered;
  }

  function backtrack(index, selected, count) {
    if (count === numSelect) {
      result = Math.max(result, countCovered(selected));
      return;
    }
    if (index >= cols || cols - index + count < numSelect) return;

    backtrack(index + 1, selected | (1 << index), count + 1);
    backtrack(index + 1, selected, count);
  }
};
```

2398 Maximum Number Of Robots Within Budget

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2398. Maximum Number of Robots Within Budget
 * https://leetcode.com/problems/maximum-number-of-robots-within-budget/
 * Difficulty: Hard
 *
 * You have n robots. You are given two 0-indexed integer arrays, chargeTimes and
runningCosts,
 * both of length n. The ith robot costs chargeTimes[i] units to charge and costs
runningCosts[i]
 * units to run. You are also given an integer budget.
 *
 * The total cost of running k chosen robots is equal to max(chargeTimes) + k *
sum(runningCosts),
 * where max(chargeTimes) is the largest charge cost among the k robots and sum(runningCosts)
is
 * the sum of running costs among the k robots.
 *
 * Return the maximum number of consecutive robots you can run such that the total cost does
not
 * exceed budget.
 */

/**
 * @param {number[]} chargeTimes
 * @param {number[]} runningCosts
 * @param {number} budget
 * @return {number}
 */
```

```
var maximumRobots = function(chargeTimes, runningCosts, budget) {
  const n = chargeTimes.length;
  let left = 0;
  let result = 0;
  let runningSum = 0;
  const queue = [];

  for (let right = 0; right < n; right++) {
    runningSum += runningCosts[right];

    while (queue.length > 0 && chargeTimes[queue[queue.length - 1]] <= chargeTimes[right]) {
      queue.pop();
    }
    queue.push(right);

    while (left <= right) {
      const maxCharge = chargeTimes[queue[0]];
      const totalCost = maxCharge + (right - left + 1) * runningSum;

      if (totalCost <= budget) {
        result = Math.max(result, right - left + 1);
        break;
      }

      if (queue[0] === left) {
        queue.shift();
      }
      runningSum -= runningCosts[left];
      left++;
    }
  }

  return result;
};
```

2399 Check Distances Between Same Letters

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2399. Check Distances Between Same Letters
 * https://leetcode.com/problems/check-distances-between-same-letters/
 * Difficulty: Easy
 *
 * You are given a 0-indexed string s consisting of only lowercase English letters, where each
 * letter in s appears exactly twice. You are also given a 0-indexed integer array distance of
 * length 26.
 *
 * Each letter in the alphabet is numbered from 0 to 25 (i.e. 'a' -> 0, 'b' -> 1,
 * 'c' -> 2, ... , 'z' -> 25).
 *
 * In a well-spaced string, the number of letters between the two occurrences of the ith
letter
 * is distance[i]. If the ith letter does not appear in s, then distance[i] can be ignored.
 *
 * Return true if s is a well-spaced string, otherwise return false.
 */

/**
 * @param {string} s
 * @param {number[]} distance
 * @return {boolean}
 */
var checkDistances = function(s, distance) {
  const map = new Map();

  for (let i = 0; i < s.length; i++) {
    const char = s[i];
    if (map.has(char)) {
      map.get(char).push(i);
```

```
    } else {
      map.set(char, [i]);
    }
  }

  for (const [char, positions] of map) {
    const letterIndex = char.charCodeAt(0) - 'a'.charCodeAt(0);
    if (positions[1] - positions[0] - 1 !== distance[letterIndex]) {
      return false;
    }
  }

  return true;
};
```

2400 Number Of Ways To Reach A Position After Exactly K Steps

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2400. Number of Ways to Reach a Position After Exactly k Steps
 * https://leetcode.com/problems/number-of-ways-to-reach-a-position-after-exactly-k-steps/
 * Difficulty: Medium
 *
 * You are given two positive integers startPos and endPos. Initially, you are standing at
 * position startPos on an infinite number line. With one step, you can move either one
position
 * to the left, or one position to the right.
 *
 * Given a positive integer k, return the number of different ways to reach the position
endPos
 * starting from startPos, such that you perform exactly k steps. Since the answer may be very
 * large, return it modulo 109 + 7.
 *
 * Two ways are considered different if the order of the steps made is not exactly the same.
 *
 * Note that the number line includes negative integers.
 */

/**
 * @param {number} startPos
 * @param {number} endPos
 * @param {number} k
 * @return {number}
 */
var numberOfWays = function(startPos, endPos, k) {
  const MOD = 1e9 + 7;
  const dp = new Array(3000).fill().map(() => new Array(k + 1).fill(-1));

  return helper(startPos, endPos, k);

  function helper(currentPos, targetPos, remainingSteps) {
    if (currentPos === targetPos && remainingSteps === 0) {
      return 1;
    }
    if (remainingSteps === 0) {
      return 0;
    }

    const dpIndex = currentPos + 1000;
    if (dp[dpIndex][remainingSteps] !== -1) {
      return dp[dpIndex][remainingSteps];
    }
    const leftMove = helper(currentPos - 1, targetPos, remainingSteps - 1);
    const rightMove = helper(currentPos + 1, targetPos, remainingSteps - 1);

    dp[dpIndex][remainingSteps] = (leftMove + rightMove) % MOD;
    return dp[dpIndex][remainingSteps];
  }
};
```

2401 Longest Nice Subarray

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2401. Longest Nice Subarray
 * https://leetcode.com/problems/longest-nice-subarray/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of positive integers.
 *
 * We call a subarray of nums nice if the bitwise AND of every pair of elements that are in
 * different positions in the subarray is equal to 0.
 *
 * Return the length of the longest nice subarray.
 *
 * A subarray is a contiguous part of an array.
 *
 * Note that subarrays of length 1 are always considered nice.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var longestNiceSubarray = function(nums) {
  let result = 1;
  let left = 0;
  let usedBits = 0;

  for (let right = 0; right < nums.length; right++) {
    while ((usedBits & nums[right]) !== 0) {
      usedBits ^= nums[left];
      left++;
    }
    usedBits |= nums[right];
    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

2402 Meeting Rooms Iii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2402. Meeting Rooms III
 * https://leetcode.com/problems/meeting-rooms-iii/
 * Difficulty: Hard
 *
 * You are given an integer n. There are n rooms numbered from 0 to n - 1.
 *
 * You are given a 2D integer array meetings where meetings[i] = [starti, endi] means that
 * a meeting will be held during the half-closed time interval [starti, endi). All the values
 * of starti are unique.
 *
 * Meetings are allocated to rooms in the following manner:
 * 1. Each meeting will take place in the unused room with the lowest number.
 * 2. If there are no available rooms, the meeting will be delayed until a room becomes free.
 *    The delayed meeting should have the same duration as the original meeting.
 * 3. When a room becomes unused, meetings that have an earlier original start time should be
 *    given the room.
```

```
 *
 * Return the number of the room that held the most meetings. If there are multiple rooms,
 * return the room with the lowest number.
 *
 * A half-closed interval [a, b) is the interval between a and b including a and not including
 b.
 */

/**
 * @param {number} n
 * @param {number[][]} meetings
 * @return {number}
 */
var mostBooked = function(n, meetings) {
  meetings.sort((a, b) => a[0] - b[0]);

  const roomCount = new Array(n).fill(0);
  const availableRooms = [];
  const occupiedRooms = [];
  for (let i = 0; i < n; i++) {
    availableRooms.push(i);
  }

  for (const [start, end] of meetings) {
    while (occupiedRooms.length > 0 && occupiedRooms[0][0] <= start) {
      const [endTime, roomNumber] = occupiedRooms.shift();
      availableRooms.push(roomNumber);
      availableRooms.sort((a, b) => a - b);
    }

    let assignedRoom;
    let meetingEnd;

    if (availableRooms.length > 0) {
      assignedRoom = availableRooms.shift();
      meetingEnd = end;
    } else {
      const [earliestEndTime, roomNumber] = occupiedRooms.shift();
      assignedRoom = roomNumber;
      meetingEnd = earliestEndTime + (end - start);
    }

    roomCount[assignedRoom]++;
    occupiedRooms.push([meetingEnd, assignedRoom]);
    occupiedRooms.sort((a, b) => a[0] === b[0] ? a[1] - b[1] : a[0] - b[0]);
  }

  const maxMeetings = Math.max(...roomCount);
  return roomCount.indexOf(maxMeetings);
};
```

2403 Minimum Time To Kill All Monsters

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2403. Minimum Time to Kill All Monsters
 * https://leetcode.com/problems/minimum-time-to-kill-all-monsters/
 * Difficulty: Hard
 *
 * You are given an integer array power where power[i] is the power of the ith monster.
 *
 * You start with 0 mana points, and each day you increase your mana points by gain where gain
 * initially is equal to 1.
 *
 * Each day, after gaining gain mana, you can defeat a monster if your mana points are greater
 * than or equal to the power of that monster. When you defeat a monster:
 * - your mana points will be reset to 0, and
 * - the value of gain increases by 1.
 *
```

```
 * Return the minimum number of days needed to defeat all the monsters.
 */

/**
 * @param {number[]} power
 * @return {number}
 */
var minimumTime = function(power) {
  const n = power.length;
  const map = new Map();

  return dp(0, 1);

  function dp(mask, gain) {
    if (mask === (1 << n) - 1) return 0;

    const key = `${mask}_${gain}`;
    if (map.has(key)) return map.get(key);

    let minDays = Infinity;

    for (let i = 0; i < n; i++) {
      if (!(mask & (1 << i))) {
        const daysNeeded = Math.ceil(power[i] / gain);
        const totalDays = daysNeeded + dp(mask | (1 << i), gain + 1);
        minDays = Math.min(minDays, totalDays);
      }
    }

    map.set(key, minDays);
    return minDays;
  }
};
```

2404 Most Frequent Even Element

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2404. Most Frequent Even Element
 * https://leetcode.com/problems/most-frequent-even-element/
 * Difficulty: Easy
 *
 * Given an integer array nums, return the most frequent even element.
 *
 * If there is a tie, return the smallest one. If there is no such element, return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var mostFrequentEven = function(nums) {
  const map = new Map();
  let maxCount = 0;
  let result = -1;

  for (const num of nums) {
    if (num % 2 === 0) {
      const count = (map.get(num) || 0) + 1;
      map.set(num, count);
      if (count > maxCount || (count === maxCount && num < result)) {
        maxCount = count;
        result = num;
      }
    }
  }

  return result;
};
```

2405 Optimal Partition Of String

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2405. Optimal Partition of String
 * https://leetcode.com/problems/optimal-partition-of-string/
 * Difficulty: Medium
 *
 * Given a string s, partition the string into one or more substrings such that the characters
 * in each substring are unique. That is, no letter appears in a single substring more than
once.
 *
 * Return the minimum number of substrings in such a partition.
 *
 * Note that each character should belong to exactly one substring in a partition.
 */

/**
 * @param {string} s
 * @return {number}
 */
var partitionString = function(s) {
  const set = new Set();
  let result = 1;

  for (const char of s) {
    if (set.has(char)) {
      set.clear();
      result++;
    }
    set.add(char);
  }

  return result;
};
```

2406 Divide Intervals Into Minimum Number Of Groups

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2406. Divide Intervals Into Minimum Number of Groups
 * https://leetcode.com/problems/divide-intervals-into-minimum-number-of-groups/
 * Difficulty: Medium
 *
 * You are given a 2D integer array intervals where intervals[i] = [lefti, righti] represents
 * the inclusive interval [lefti, righti].
 *
 * You have to divide the intervals into one or more groups such that each interval is in
 * exactly one group, and no two intervals that are in the same group intersect each other.
 *
 * Return the minimum number of groups you need to make.
 *
 * Two intervals intersect if there is at least one common number between them. For example,
 * the intervals [1, 5] and [5, 8] intersect.
 */

/**
 * @param {number[][]} intervals
 * @return {number}
 */
var minGroups = function(intervals) {
  const events = [];
  for (const [start, end] of intervals) {
```

```
    events.push([start, 1]);
    events.push([end + 1, -1]);
  }

  events.sort((a, b) => a[0] === b[0] ? a[1] - b[1] : a[0] - b[0]);

  let activeIntervals = 0;
  let result = 0;
  for (const [time, change] of events) {
    activeIntervals += change;
    result = Math.max(result, activeIntervals);
  }

  return result;
};
```

2407 Longest Increasing Subsequence Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2407. Longest Increasing Subsequence II
 * https://leetcode.com/problems/longest-increasing-subsequence-ii/
 * Difficulty: Hard
 *
 * You are given an integer array nums and an integer k.
 *
 * Find the longest subsequence of nums that meets the following requirements:
 * - The subsequence is strictly increasing and
 * - The difference between adjacent elements in the subsequence is at most k.
 *
 * Return the length of the longest subsequence that meets the requirements.
 *
 * A subsequence is an array that can be derived from another array by deleting some or
 * no elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var lengthOfLIS = function(nums, k) {
  const maxValue = Math.max(...nums);
  const tree = new Array(4 * maxValue).fill(0);

  for (const currentNum of nums) {
    const rangeStart = Math.max(1, currentNum - k);
    const rangeEnd = currentNum - 1;

    const bestPreviousLength = rangeEnd >= rangeStart
      ? queryRange(1, 1, maxValue, rangeStart, rangeEnd) : 0;
    updatePosition(1, 1, maxValue, currentNum, bestPreviousLength + 1);
  }

  return queryRange(1, 1, maxValue, 1, maxValue);

  function updatePosition(nodeIndex, treeStart, treeEnd, position, newLength) {
    if (treeStart === treeEnd) {
      tree[nodeIndex] = Math.max(tree[nodeIndex], newLength);
    } else {
      const treeMid = Math.floor((treeStart + treeEnd) / 2);
      if (position <= treeMid) {
        updatePosition(2 * nodeIndex, treeStart, treeMid, position, newLength);
      } else {
        updatePosition(2 * nodeIndex + 1, treeMid + 1, treeEnd, position, newLength);
      }
      tree[nodeIndex] = Math.max(tree[2 * nodeIndex], tree[2 * nodeIndex + 1]);
    }
  }
```

```
  function queryRange(nodeIndex, treeStart, treeEnd, queryStart, queryEnd) {
    if (queryEnd < treeStart || treeEnd < queryStart) {
      return 0;
    }
    if (queryStart <= treeStart && treeEnd <= queryEnd) {
      return tree[nodeIndex];
    }
    const mid = Math.floor((treeStart + treeEnd) / 2);
    return Math.max(
      queryRange(2 * nodeIndex, treeStart, mid, queryStart, queryEnd),
      queryRange(2 * nodeIndex + 1, mid + 1, treeEnd, queryStart, queryEnd)
    );
  }
};
```

2408 Design Sql

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2408. Design SQL
 * https://leetcode.com/problems/design-sql/
 * Difficulty: Medium
 *
 * You are given two string arrays, names and columns, both of size n. The ith table is
represented
 * by the name names[i] and contains columns[i] number of columns.
 *
 * You need to implement a class that supports the following operations:
 * - Insert a row in a specific table with an id assigned using an auto-increment method,
where the
 *   id of the first inserted row is 1, and the id of each new row inserted into the same
table is
 *   one greater than the id of the last inserted row, even if the last row was removed.
 * - Remove a row from a specific table. Removing a row does not affect the id of the next
inserted
 *   row.
 * - Select a specific cell from any table and return its value.
 * - Export all rows from any table in csv format.
 *
 * Implement the SQL class:
 * - SQL(String[] names, int[] columns)
 *   - Creates the n tables.
 * - bool ins(String name, String[] row)
 *   - Inserts row into the table name and returns true.
 *   - If row.length does not match the expected number of columns, or name is not a valid
table,
 *     returns false without any insertion.
 * - void rmv(String name, int rowId)
 *   - Removes the row rowId from the table name.
 *   - If name is not a valid table or there is no row with id rowId, no removal is performed.
 * - String sel(String name, int rowId, int columnId)
 *   - Returns the value of the cell at the specified rowId and columnId in the table name.
 *   - If name is not a valid table, or the cell (rowId, columnId) is invalid, returns
"<null>".
 * - String[] exp(String name)
 *   - Returns the rows present in the table name.
 *   - If name is not a valid table, returns an empty array. Each row is represented as a
string,
 *     with each cell value (including the row's id) separated by a ",".
 */

/**
 * @param {string[]} names
 * @param {number[]} columns
 */
var SQL = function(names, columns) {
  this.tables = new Map();
  this.nextRowIds = new Map();
```

```
  for (let i = 0; i < names.length; i++) {
    this.tables.set(names[i], {
      columns: columns[i],
      rows: new Map()
    });
    this.nextRowIds.set(names[i], 1);
  }
};

/**
 * @param {string} name
 * @param {string[]} row
 * @return {boolean}
 */
SQL.prototype.ins = function(name, row) {
  if (!this.tables.has(name)) return false;

  const table = this.tables.get(name);
  if (row.length !== table.columns) return false;

  const rowId = this.nextRowIds.get(name);
  table.rows.set(rowId, row);
  this.nextRowIds.set(name, rowId + 1);

  return true;
};

/**
 * @param {string} name
 * @param {number} rowId
 * @return {void}
 */
SQL.prototype.rmv = function(name, rowId) {
  if (!this.tables.has(name)) return;

  const table = this.tables.get(name);
  table.rows.delete(rowId);
};

/**
 * @param {string} name
 * @param {number} rowId
 * @param {number} columnId
 * @return {string}
 */
SQL.prototype.sel = function(name, rowId, columnId) {
  if (!this.tables.has(name)) return '<null>';

  const table = this.tables.get(name);
  if (!table.rows.has(rowId) || columnId < 1 || columnId > table.columns) {
    return '<null>';
  }

  return table.rows.get(rowId)[columnId - 1];
};

/**
 * @param {string} name
 * @return {string[]}
 */
SQL.prototype.exp = function(name) {
  if (!this.tables.has(name)) return [];

  const table = this.tables.get(name);
  const result = [];

  for (const [rowId, row] of table.rows) {
    result.push(`${rowId},${row.join(',')}`);
  }

  return result;
};
```

2409 Count Days Spent Together

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2409. Count Days Spent Together
 * https://leetcode.com/problems/count-days-spent-together/
 * Difficulty: Easy
 *
 * Alice and Bob are traveling to Rome for separate business meetings.
 *
 * You are given 4 strings arriveAlice, leaveAlice, arriveBob, and leaveBob. Alice will be in
 * the city from the dates arriveAlice to leaveAlice (inclusive), while Bob will be in the
city
 * from the dates arriveBob to leaveBob (inclusive). Each will be a 5-character string in the
 * format "MM-DD", corresponding to the month and day of the date.
 *
 * Return the total number of days that Alice and Bob are in Rome together.
 *
 * You can assume that all dates occur in the same calendar year, which is not a leap year.
Note
 * that the number of days per month can be represented as: [31, 28, 31, 30, 31, 30, 31, 31,
30,
 * 31, 30, 31].
 */

/**
 * @param {string} arriveAlice
 * @param {string} leaveAlice
 * @param {string} arriveBob
 * @param {string} leaveBob
 * @return {number}
 */
var countDaysTogether = function(arriveAlice, leaveAlice, arriveBob, leaveBob) {
  const daysInMonth = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];

  function toDays(date) {
    const [month, day] = date.split('-').map(Number);
    let totalDays = 0;
    for (let i = 0; i < month - 1; i++) {
      totalDays += daysInMonth[i];
    }
    return totalDays + day;
  }

  const aliceStart = toDays(arriveAlice);
  const aliceEnd = toDays(leaveAlice);
  const bobStart = toDays(arriveBob);
  const bobEnd = toDays(leaveBob);

  const overlapStart = Math.max(aliceStart, bobStart);
  const overlapEnd = Math.min(aliceEnd, bobEnd);

  return Math.max(0, overlapEnd - overlapStart + 1);
};
```

2410 Maximum Matching Of Players With Trainers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2410. Maximum Matching of Players With Trainers
 * https://leetcode.com/problems/maximum-matching-of-players-with-trainers/
 * Difficulty: Medium
 *
```

```
 * You are given a 0-indexed integer array players, where players[i] represents the ability of
 * the ith player. You are also given a 0-indexed integer array trainers, where trainers[j]
 * represents the training capacity of the jth trainer.
 *
 * The ith player can match with the jth trainer if the player's ability is less than or equal
 * to the trainer's training capacity. Additionally, the ith player can be matched with at
most
 * one trainer, and the jth trainer can be matched with at most one player.
 *
 * Return the maximum number of matchings between players and trainers that satisfy these
 * conditions.
 */

/**
 * @param {number[]} players
 * @param {number[]} trainers
 * @return {number}
 */
var matchPlayersAndTrainers = function(players, trainers) {
  players.sort((a, b) => a - b);
  trainers.sort((a, b) => a - b);

  let result = 0;
  for (let j = 0; result < players.length && j < trainers.length; j++) {
    if (trainers[j] >= players[result]) {
      result++;
    }
  }

  return result;
};
```

2411 Smallest Subarrays With Maximum Bitwise Or

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2411. Smallest Subarrays With Maximum Bitwise OR
 * https://leetcode.com/problems/smallest-subarrays-with-maximum-bitwise-or/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums of length n, consisting of non-negative integers. For
 * each index i from 0 to n - 1, you must determine the size of the minimum sized non-empty
 * subarray of nums starting at i (inclusive) that has the maximum possible bitwise OR.
 * - In other words, let Bij be the bitwise OR of the subarray nums[i...j]. You need to find
 *   the smallest subarray starting at i, such that bitwise OR of this subarray is equal to
 *   max(Bik) where i <= k <= n - 1.
 *
 * The bitwise OR of an array is the bitwise OR of all the numbers in it.
 *
 * Return an integer array answer of size n where answer[i] is the length of the minimum sized
 * subarray starting at i with maximum bitwise OR.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var smallestSubarrays = function(nums) {
  const n = nums.length;
  const result = new Array(n).fill(1);
  const bitPositions = new Array(32).fill(0);

  for (let i = n - 1; i >= 0; i--) {
    for (let bit = 0; bit < 32; bit++) {
      if (nums[i] & (1 << bit)) {
        bitPositions[bit] = i;
      }
```

```
    }

    let maxIndex = i;
    for (let bit = 0; bit < 32; bit++) {
      maxIndex = Math.max(maxIndex, bitPositions[bit]);
    }

    result[i] = maxIndex - i + 1;
  }

  return result;
};
```

2412 Minimum Money Required Before Transactions

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2412. Minimum Money Required Before Transactions
 * https://leetcode.com/problems/minimum-money-required-before-transactions/
 * Difficulty: Hard
 *
 * You are given a 0-indexed 2D integer array transactions, where
 * transactions[i] = [costi, cashbacki].
 *
 * The array describes transactions, where each transaction must be completed exactly once
 * in some order. At any given moment, you have a certain amount of money. In order to
 * complete transaction i, money >= costi must hold true. After performing a transaction,
 * money becomes money - costi + cashbacki.
 *
 * Return the minimum amount of money required before any transaction so that all of the
 * transactions can be completed regardless of the order of the transactions.
 */

/**
 * @param {number[][]} transactions
 * @return {number}
 */
var minimumMoney = function(transactions) {
  let totalLoss = 0;
  let maxCost = 0;

  for (const [cost, cashback] of transactions) {
    if (cost > cashback) {
      totalLoss += cost - cashback;
      maxCost = Math.max(maxCost, cashback);
    } else {
      maxCost = Math.max(maxCost, cost);
    }
  }

  return totalLoss + maxCost;
};
```

2413 Smallest Even Multiple

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2413. Smallest Even Multiple
 * https://leetcode.com/problems/smallest-even-multiple/
 * Difficulty: Easy
 *
 * Given a positive integer n, return the smallest positive integer that is a multiple of both
 2 and n.
```

```
 */

/**
 * @param {number} n
 * @return {number}
 */
var smallestEvenMultiple = function(n) {
  return n * (n % 2 + 1);
};
```

2414 Length Of The Longest Alphabetical Continuous Substring

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2414. Length of the Longest Alphabetical Continuous Substring
 * https://leetcode.com/problems/length-of-the-longest-alphabetical-continuous-substring/
 * Difficulty: Medium
 *
 * An alphabetical continuous string is a string consisting of consecutive letters in the
alphabet.
 * In other words, it is any substring of the string "abcdefghijklmnopqrstuvwxyz".
 * - For example, "abc" is an alphabetical continuous string, while "acb" and "za" are not.
 *
 * Given a string s consisting of lowercase letters only, return the length of the longest
 * alphabetical continuous substring.
 */

/**
 * @param {string} s
 * @return {number}
 */
var longestContinuousSubstring = function(s) {
  let result = 1;
  let currentLength = 1;

  for (let i = 1; i < s.length; i++) {
    if (s.charCodeAt(i) - s.charCodeAt(i - 1) === 1) {
      currentLength++;
      result = Math.max(result, currentLength);
    } else {
      currentLength = 1;
    }
  }

  return result;
};
```

2415 Reverse Odd Levels Of Binary Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2415. Reverse Odd Levels of Binary Tree
 * https://leetcode.com/problems/reverse-odd-levels-of-binary-tree/
 * Difficulty: Medium
 *
 * Given the root of a perfect binary tree, reverse the node values at each odd level of the
tree.
 * - For example, suppose the node values at level 3 are [2,1,3,4,7,11,29,18], then it should
 *   become [18,29,11,7,4,3,1,2].
 *
 * Return the root of the reversed tree.
 *
 * A binary tree is perfect if all parent nodes have two children and all leaves are on the
```

```
same
 * level.
 *
 * The level of a node is the number of edges along the path between it and the root node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var reverseOddLevels = function(root) {
  reverseLevel([root], 0);
  return root;

  function reverseLevel(nodes, level) {
    if (!nodes.length) return;

    if (level % 2 === 1) {
      for (let i = 0, j = nodes.length - 1; i < j; i++, j--) {
        [nodes[i].val, nodes[j].val] = [nodes[j].val, nodes[i].val];
      }
    }

    const nextLevel = [];
    for (const node of nodes) {
      if (node.left) nextLevel.push(node.left);
      if (node.right) nextLevel.push(node.right);
    }

    reverseLevel(nextLevel, level + 1);
  }
};
```

2416 Sum Of Prefix Scores Of Strings

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2416. Sum of Prefix Scores of Strings
 * https://leetcode.com/problems/sum-of-prefix-scores-of-strings/
 * Difficulty: Hard
 *
 * You are given an array words of size n consisting of non-empty strings.
 *
 * We define the score of a string term as the number of strings words[i] such that term is
 * a prefix of words[i].
 * - For example, if words = ["a", "ab", "abc", "cab"], then the score of "ab" is 2, since
 *   "ab" is a prefix of both "ab" and "abc".
 *
 * Return an array answer of size n where answer[i] is the sum of scores of every non-empty
 * prefix of words[i].
 *
 * Note that a string is considered as a prefix of itself.
 */

/**
 * @param {string[]} words
 * @return {number[]}
 */
var sumPrefixScores = function(words) {
  class TrieNode {
    constructor() {
```

```
      this.children = new Map();
      this.count = 0;
    }
  }

  const root = new TrieNode();

  for (const word of words) {
    let node = root;
    for (const char of word) {
      if (!node.children.has(char)) {
        node.children.set(char, new TrieNode());
      }
      node = node.children.get(char);
      node.count++;
    }
  }

  const result = [];
  for (const word of words) {
    let node = root;
    let score = 0;
    for (const char of word) {
      node = node.children.get(char);
      score += node.count;
    }
    result.push(score);
  }

  return result;
};
```

2417 Closest Fair Integer

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2417. Closest Fair Integer
 * https://leetcode.com/problems/closest-fair-integer/
 * Difficulty: Medium
 *
 * You are given a positive integer n.
 *
 * We call an integer k fair if the number of even digits in k is equal to the number of
 * odd digits in it.
 *
 * Return the smallest fair integer that is greater than or equal to n.
 */

/**
 * @param {number} n
 * @return {number}
 */
var closestFair = function(n) {
  const digitCount = n.toString().length;
  if (digitCount % 2 === 1) {
    return smallestFairNumberWithLength(digitCount + 1);
  }

  return hasEqualEvenOddDigits(n) ? n : closestFair(n + 1);

  function hasEqualEvenOddDigits(num) {
    const counts = [0, 0];
    while (num > 0) {
      counts[(num % 10) % 2]++;
      num = Math.floor(num / 10);
    }
    return counts[0] === counts[1];
  }
```

```
  function smallestFairNumberWithLength(length) {
    let result = '1';
    for (let i = 1; i <= length / 2; i++) {
      result += '0';
    }
    for (let i = length / 2 + 1; i < length; i++) {
      result += '1';
    }
    return parseInt(result);
  }
};
```

2418 Sort The People

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2418. Sort the People
 * https://leetcode.com/problems/sort-the-people/
 * Difficulty: Easy
 *
 * You are given an array of strings names, and an array heights that consists of distinct
 * positive integers. Both arrays are of length n.
 *
 * For each index i, names[i] and heights[i] denote the name and height of the ith person.
 *
 * Return names sorted in descending order by the people's heights.
 */

/**
 * @param {string[]} names
 * @param {number[]} heights
 * @return {string[]}
 */
var sortPeople = function(names, heights) {
  const people = names.map((name, index) => ({name, height: heights[index]}));
  people.sort((a, b) => b.height - a.height);
  return people.map(person => person.name);
};
```

2419 Longest Subarray With Maximum Bitwise And

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2419. Longest Subarray With Maximum Bitwise AND
 * https://leetcode.com/problems/longest-subarray-with-maximum-bitwise-and/
 * Difficulty: Medium
 *
 * You are given an integer array nums of size n.
 *
 * Consider a non-empty subarray from nums that has the maximum possible bitwise AND.
 * - In other words, let k be the maximum value of the bitwise AND of any subarray of nums.
 *   Then, only subarrays with a bitwise AND equal to k should be considered.
 *
 * Return the length of the longest such subarray.
 *
 * The bitwise AND of an array is the bitwise AND of all the numbers in it.
 *
 * A subarray is a contiguous sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
```

```
 */
var longestSubarray = function(nums) {
  const maxValue = Math.max(...nums);
  let result = 0;
  let currentLength = 0;

  for (const num of nums) {
    if (num === maxValue) {
      currentLength++;
      result = Math.max(result, currentLength);
    } else {
      currentLength = 0;
    }
  }

  return result;
};
```

2420 Find All Good Indices

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2420. Find All Good Indices
 * https://leetcode.com/problems/find-all-good-indices/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of size n and a positive integer k.
 *
 * We call an index i in the range k <= i < n - k good if the following conditions are
satisfied:
 * - The k elements that are just before the index i are in non-increasing order.
 * - The k elements that are just after the index i are in non-decreasing order.
 *
 * Return an array of all good indices sorted in increasing order.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var goodIndices = function(nums, k) {
  const n = nums.length;
  const nonIncreasing = new Array(n).fill(1);
  const nonDecreasing = new Array(n).fill(1);
  const result = [];

  for (let i = 1; i < n; i++) {
    if (nums[i - 1] >= nums[i]) {
      nonIncreasing[i] = nonIncreasing[i - 1] + 1;
    }
    if (nums[n - i] >= nums[n - i - 1]) {
      nonDecreasing[n - i - 1] = nonDecreasing[n - i] + 1;
    }
  }

  for (let i = k; i < n - k; i++) {
    if (nonIncreasing[i - 1] >= k && nonDecreasing[i + 1] >= k) {
      result.push(i);
    }
  }

  return result;
};
```

2421 Number Of Good Paths

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2421. Number of Good Paths
 * https://leetcode.com/problems/number-of-good-paths/
 * Difficulty: Hard
 *
 * There is a tree (i.e. a connected, undirected graph with no cycles) consisting of n nodes
 * numbered from 0 to n - 1 and exactly n - 1 edges.
 *
 * You are given a 0-indexed integer array vals of length n where vals[i] denotes the value
 * of the ith node. You are also given a 2D integer array edges where edges[i] = [ai, bi]
 * denotes that there exists an undirected edge connecting nodes ai and bi.
 *
 * A good path is a simple path that satisfies the following conditions:
 * 1. The starting node and the ending node have the same value.
 * 2. All nodes between the starting node and the ending node have values less than or equal
 *    to the starting node (i.e. the starting node's value should be the maximum value along
 *    the path).
 *
 * Return the number of distinct good paths.
 *
 * Note that a path and its reverse are counted as the same path. For example, 0 -> 1 is
 * considered to be the same as 1 -> 0. A single node is also considered as a valid path.
 */

/**
 * @param {number[]} vals
 * @param {number[][]} edges
 * @return {number}
 */
var numberOfGoodPaths = function(vals, edges) {
  const n = vals.length;
  const graph = Array.from({length: n}, () => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  const parent = new Array(n).fill(-1);
  const rank = new Array(n).fill(0);
  function find(x) {
    if (parent[x] === -1) return x;
    return parent[x] = find(parent[x]);
  }

  function union(x, y) {
    let px = find(x);
    let py = find(y);
    if (px === py) return;
    if (rank[px] < rank[py]) [px, py] = [py, px];
    parent[py] = px;
    if (rank[px] === rank[py]) rank[px]++;
  }

  const valueGroups = new Map();
  for (let i = 0; i < n; i++) {
    if (!valueGroups.has(vals[i])) {
      valueGroups.set(vals[i], []);
    }
    valueGroups.get(vals[i]).push(i);
  }

  let result = 0;
  for (const value of [...valueGroups.keys()].sort((a, b) => a - b)) {
    const nodes = valueGroups.get(value);
```

```
    for (const node of nodes) {
      for (const neighbor of graph[node]) {
        if (vals[neighbor] <= value) {
          union(node, neighbor);
        }
      }
    }

    const groupCount = new Map();
    for (const node of nodes) {
      const root = find(node);
      groupCount.set(root, (groupCount.get(root) || 0) + 1);
    }

    for (const count of groupCount.values()) {
      result += (count * (count + 1)) / 2;
    }
  }

  return result;
};
```

2422 Merge Operations To Turn Array Into A Palindrome

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2422. Merge Operations to Turn Array Into a Palindrome
 * https://leetcode.com/problems/merge-operations-to-turn-array-into-a-palindrome/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of positive integers.
 *
 * You can perform the following operation on the array any number of times:
 * - Choose any two adjacent elements and replace them with their sum.
 *   - For example, if nums = [1,2,3,1], you can apply one operation to make it [1,5,1].
 *
 * Return the minimum number of operations needed to turn the array into a palindrome.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumOperations = function(nums) {
  let left = 0;
  let right = nums.length - 1;
  let leftSum = nums[left];
  let rightSum = nums[right];
  let result = 0;

  while (left < right) {
    if (leftSum === rightSum) {
      left++;
      right--;
      if (left <= right) {
        leftSum = nums[left];
        rightSum = nums[right];
      }
    } else if (leftSum < rightSum) {
      left++;
      leftSum += nums[left];
      result++;
    } else {
      right--;
      rightSum += nums[right];
      result++;
    }
  }
```

```
  return result;
};
```

2425 Bitwise Xor Of All Pairings

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2425. Bitwise XOR of All Pairings
 * https://leetcode.com/problems/bitwise-xor-of-all-pairings/
 * Difficulty: Medium
 *
 * You are given two 0-indexed arrays, nums1 and nums2, consisting of non-negative integers.
 * There exists another array, nums3, which contains the bitwise XOR of all pairings of
 * integers between nums1 and nums2 (every integer in nums1 is paired with every integer
 * in nums2 exactly once).
 *
 * Return the bitwise XOR of all integers in nums3.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var xorAllNums = function(nums1, nums2) {
  const xor = (a, b) => a.length % 2 ? b.reduce((x, y) => x ^ y): 0;
  return xor(nums1, nums2) ^ xor(nums2, nums1);
};
```

2426 Number Of Pairs Satisfying Inequality

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2426. Number of Pairs Satisfying Inequality
 * https://leetcode.com/problems/number-of-pairs-satisfying-inequality/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays nums1 and nums2, each of size n, and an
 * integer diff. Find the number of pairs (i, j) such that:
 * - 0 <= i < j <= n - 1 and
 * - nums1[i] - nums1[j] <= nums2[i] - nums2[j] + diff.
 *
 * Return the number of pairs that satisfy the conditions.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @param {number} diff
 * @return {number}
 */
var numberOfPairs = function(nums1, nums2, diff) {
  const n = nums1.length;
  const differences = new Array(n);
  for (let i = 0; i < n; i++) {
    differences[i] = nums1[i] - nums2[i];
  }
  let result = 0;
  mergeSort(0, n);
  return result;

  function mergeSort(left, right) {
    if (right - left <= 1) return;
```

```
    const mid = Math.floor((left + right) / 2);
    mergeSort(left, mid);
    mergeSort(mid, right);

    let i = left;
    let j = mid;
    while (i < mid && j < right) {
      if (differences[i] <= differences[j] + diff) {
        result += right - j;
        i++;
      } else {
        j++;
      }
    }

    const sorted = new Array(right - left);
    let k = 0;
    i = left;
    j = mid;
    while (i < mid && j < right) {
      if (differences[i] <= differences[j]) {
        sorted[k++] = differences[i++];
      } else {
        sorted[k++] = differences[j++];
      }
    }
    while (i < mid) sorted[k++] = differences[i++];
    while (j < right) sorted[k++] = differences[j++];

    for (let p = 0; p < sorted.length; p++) {
      differences[left + p] = sorted[p];
    }
  }
};
```

2427 Number Of Common Factors

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2427. Number of Common Factors
 * https://leetcode.com/problems/number-of-common-factors/
 * Difficulty: Easy
 *
 * Given two positive integers a and b, return the number of common factors of a and b.
 *
 * An integer x is a common factor of a and b if x divides both a and b.
 */

/**
 * @param {number} a
 * @param {number} b
 * @return {number}
 */
var commonFactors = function(a, b) {
  let count = 0;
  for (let i = Math.min(a, b); i > 0; i--) {
    if (a % i === 0 && b % i === 0) count++;
  }
  return count;
};
```

2428 Maximum Sum Of An Hourglass

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2428. Maximum Sum of an Hourglass
 * https://leetcode.com/problems/maximum-sum-of-an-hourglass/
 * Difficulty: Medium
 *
 * You are given an m x n integer matrix grid.
 *
 * We define an hourglass as a part of the matrix with the following form.
 *
 * Return the maximum sum of the elements of an hourglass.
 *
 * Note that an hourglass cannot be rotated and must be entirely contained within the matrix.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maxSum = function(grid) {
  let result = 0;
  const rows = grid.length;
  const cols = grid[0].length;

  for (let i = 0; i <= rows - 3; i++) {
    for (let j = 0; j <= cols - 3; j++) {
      const hourglassSum = grid[i][j] + grid[i][j+1] + grid[i][j+2] + grid[i+1][j+1]
        + grid[i+2][j] + grid[i+2][j+1] + grid[i+2][j+2];
      result = Math.max(result, hourglassSum);
    }
  }

  return result;
};
```

2429 Minimize Xor

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2429. Minimize XOR
 * https://leetcode.com/problems/minimize-xor/
 * Difficulty: Medium
 *
 * Given two positive integers num1 and num2, find the positive integer x such that:
 * - x has the same number of set bits as num2, and
 * - The value x XOR num1 is minimal.
 *
 * Note that XOR is the bitwise XOR operation.
 *
 * Return the integer x. The test cases are generated such that x is uniquely determined.
 *
 * The number of set bits of an integer is the number of 1's in its binary representation.
 */

/**
 * @param {number} num1
 * @param {number} num2
 * @return {number}
 */
var minimizeXor = function(num1, num2) {
  let count1 = num1.toString(2).split('1').length - 1;
  const count2 = num2.toString(2).split('1').length - 1;

  while (count1 > count2) {
    num1 &= (num1 - 1);
    count1--;
  }
```

```
  while (count1 < count2) {
    num1 |= (num1 + 1);
    count1++;
  }

  return num1;
};
```

2431 Maximize Total Tastiness Of Purchased Fruits

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2431. Maximize Total Tastiness of Purchased Fruits
 * https://leetcode.com/problems/maximize-total-tastiness-of-purchased-fruits/
 * Difficulty: Medium
 *
 * You are given two non-negative integer arrays price and tastiness, both arrays have the
same
 * length n. You are also given two non-negative integers maxAmount and maxCoupons.
 *
 * For every integer i in range [0, n - 1]:
 * - price[i] describes the price of ith fruit.
 * - tastiness[i] describes the tastiness of ith fruit.
 *
 * You want to purchase some fruits such that total tastiness is maximized and the total price
 * does not exceed maxAmount.
 *
 * Additionally, you can use a coupon to purchase fruit for half of its price (rounded down to
 * the closest integer). You can use at most maxCoupons of such coupons.
 *
 * Return the maximum total tastiness that can be purchased.
 *
 * Note that:
 * - You can purchase each fruit at most once.
 * - You can use coupons on some fruit at most once.
 */

/**
 * @param {number[]} price
 * @param {number[]} tastiness
 * @param {number} maxAmount
 * @param {number} maxCoupons
 * @return {number}
 */
var maxTastiness = function(price, tastiness, maxAmount, maxCoupons) {
  const n = price.length;
  const map = new Map();

  return dp(0, maxAmount, maxCoupons);

  function dp(index, remainingAmount, remainingCoupons) {
    if (index === n) return 0;

    const key = `${index}_${remainingAmount}_${remainingCoupons}`;
    if (map.has(key)) return map.get(key);

    let maxValue = dp(index + 1, remainingAmount, remainingCoupons);

    if (price[index] <= remainingAmount) {
      maxValue = Math.max(
        maxValue,
        tastiness[index] + dp(index + 1, remainingAmount - price[index], remainingCoupons)
      );
    }

    if (remainingCoupons > 0) {
      const discountedPrice = Math.floor(price[index] / 2);
      if (discountedPrice <= remainingAmount) {
```

```
        maxValue = Math.max(
          maxValue,
          tastiness[index] + dp(index + 1, remainingAmount - discountedPrice, remainingCoupons
 - 1)
        );
      }
    }

    map.set(key, maxValue);
    return maxValue;
  }
};
```

2432 The Employee That Worked On The Longest Task

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2432. The Employee That Worked on the Longest Task
 * https://leetcode.com/problems/the-employee-that-worked-on-the-longest-task/
 * Difficulty: Easy
 *
 * There are n employees, each with a unique id from 0 to n - 1.
 *
 * You are given a 2D integer array logs where logs[i] = [idi, leaveTimei] where:
 * - idi is the id of the employee that worked on the ith task, and
 * - leaveTimei is the time at which the employee finished the ith task. All the values
 *   leaveTimei are unique.
 *
 * Note that the ith task starts the moment right after the (i - 1)th task ends, and the 0th
 * task starts at time 0.
 *
 * Return the id of the employee that worked the task with the longest time. If there is a tie
 * between two or more employees, return the smallest id among them.
 */

/**
 * @param {number} n
 * @param {number[][]} logs
 * @return {number}
 */
var hardestWorker = function(n, logs) {
  let maxDuration = 0;
  let result = n;
  let startTime = 0;

  for (const [employeeId, endTime] of logs) {
    const duration = endTime - startTime;
    if (duration > maxDuration || (duration === maxDuration && employeeId < result)) {
      maxDuration = duration;
      result = employeeId;
    }
    startTime = endTime;
  }

  return result;
};
```

2433 Find The Original Array Of Prefix Xor

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2433. Find The Original Array of Prefix Xor
 * https://leetcode.com/problems/find-the-original-array-of-prefix-xor/
```

```
 * Difficulty: Medium
 *
 * You are given an integer array pref of size n. Find and return the array arr of size
 * n that satisfies:
 * - pref[i] = arr[0] ^ arr[1] ^ ... ^ arr[i].
 *
 * Note that ^ denotes the bitwise-xor operation.
 *
 * It can be proven that the answer is unique.
 */

/**
 * @param {number[]} pref
 * @return {number[]}
 */
var findArray = function(pref) {
  const result = new Array(pref.length);
  result[0] = pref[0];

  for (let i = 1; i < pref.length; i++) {
    result[i] = pref[i] ^ pref[i - 1];
  }

  return result;
};
```

2434 Using A Robot To Print The Lexicographically Smallest String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2434. Using a Robot to Print the Lexicographically Smallest String
 * https://leetcode.com/problems/using-a-robot-to-print-the-lexicographically-smallest-string/
 * Difficulty: Medium
 *
 * You are given a string s and a robot that currently holds an empty string t. Apply one of
the
 * following operations until s and t are both empty:
 * - Remove the first character of a string s and give it to the robot. The robot will append
 *   this character to the string t.
 * - Remove the last character of a string t and give it to the robot. The robot will write
 *   this character on paper.
 *
 * Return the lexicographically smallest string that can be written on the paper.
 */

/**
 * @param {string} s
 * @return {string}
 */
var robotWithString = function(s) {
  const charCount = Array(26).fill(0);
  for (const char of s) {
    charCount[char.charCodeAt(0) - 97]++;
  }

  const stack = [];
  let minCharIndex = 0;
  let result = '';

  for (const char of s) {
    stack.push(char);
    charCount[char.charCodeAt(0) - 97]--;

    while (minCharIndex < 26 && charCount[minCharIndex] === 0) {
      minCharIndex++;
    }

    while (stack.length
      && (minCharIndex === 26 || stack[stack.length - 1].charCodeAt(0) - 97 <= minCharIndex))
```

```
{
      result += stack.pop();
    }
  }

  while (stack.length) {
    result += stack.pop();
  }

  return result;
};
```

2435 Paths In Matrix Whose Sum Is Divisible By K

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2435. Paths in Matrix Whose Sum Is Divisible by K
 * https://leetcode.com/problems/paths-in-matrix-whose-sum-is-divisible-by-k/
 * Difficulty: Hard
 *
 * You are given a 0-indexed m x n integer matrix grid and an integer k. You are currently at
 * position (0, 0) and you want to reach position (m - 1, n - 1) moving only down or right.
 *
 * Return the number of paths where the sum of the elements on the path is divisible by k.
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[][]} grid
 * @param {number} k
 * @return {number}
 */
var numberOfPaths = function(grid, k) {
  const rows = grid.length;
  const cols = grid[0].length;
  const modulo = 1e9 + 7;
  const dp = Array.from({ length: rows }, () => {
    return Array.from({ length: cols }, () => Array(k).fill(0));
  });

  dp[0][0][grid[0][0] % k] = 1;

  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      const currentValue = grid[row][col] % k;
      for (let sum = 0; sum < k; sum++) {
        if (row > 0) {
          const prevSum = (sum - currentValue + k) % k;
          dp[row][col][sum] = (dp[row][col][sum] + dp[row - 1][col][prevSum]) % modulo;
        }
        if (col > 0) {
          const prevSum = (sum - currentValue + k) % k;
          dp[row][col][sum] = (dp[row][col][sum] + dp[row][col - 1][prevSum]) % modulo;
        }
      }
    }
  }

  return dp[rows - 1][cols - 1][0];
};
```

2436 Minimum Split Into Subarrays With Gcd Greater Than One

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2436. Minimum Split Into Subarrays With GCD Greater Than One
 * https://leetcode.com/problems/minimum-split-into-subarrays-with-gcd-greater-than-one/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of positive integers.
 *
 * Split the array into one or more disjoint subarrays such that:
 * - Each element of the array belongs to exactly one subarray, and
 * - The GCD of the elements of each subarray is strictly greater than 1.
 *
 * Return the minimum number of subarrays that can be obtained after the split.
 *
 * Note that:
 * - The GCD of a subarray is the largest positive integer that evenly divides all the
elements
 *   of the subarray.
 * - A subarray is a contiguous part of the array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumSplits = function(nums) {
  const n = nums.length;
  let result = 1;
  let currentGcd = nums[0];

  for (let i = 1; i < n; i++) {
    const newGcd = gcd(currentGcd, nums[i]);
    if (newGcd === 1) {
      result++;
      currentGcd = nums[i];
    } else {
      currentGcd = newGcd;
    }
  }

  return result;

  function gcd(a, b) {
    while (b !== 0) {
      const temp = b;
      b = a % b;
      a = temp;
    }
    return a;
  }
};
```

2437 Number Of Valid Clock Times

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2437. Number of Valid Clock Times
 * https://leetcode.com/problems/number-of-valid-clock-times/
 * Difficulty: Easy
 *
 * You are given a string of length 5 called time, representing the current time on a digital
 * clock in the format "hh:mm". The earliest possible time is "00:00" and the latest possible
 * time is "23:59".
 *
 * In the string time, the digits represented by the ? symbol are unknown, and must be
replaced
 * with a digit from 0 to 9.
 *
 * Return an integer answer, the number of valid clock times that can be created by replacing
```

```
 * every ? with a digit from 0 to 9.
 */

/**
 * @param {string} time
 * @return {number}
 */
var countTime = function(time) {
  let hourChoices = 1;
  let minuteChoices = 1;

  if (time[0] === '?' && time[1] === '?') {
    hourChoices = 24;
  } else if (time[0] === '?') {
    hourChoices = time[1] <= '3' ? 3 : 2;
  } else if (time[1] === '?') {
    hourChoices = time[0] === '2' ? 4 : 10;
  }

  if (time[3] === '?' && time[4] === '?') {
    minuteChoices = 60;
  } else if (time[3] === '?') {
    minuteChoices = 6;
  } else if (time[4] === '?') {
    minuteChoices = 10;
  }

  return hourChoices * minuteChoices;
};
```

2438 Range Product Queries Of Powers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2438. Range Product Queries of Powers
 * https://leetcode.com/problems/range-product-queries-of-powers/
 * Difficulty: Medium
 *
 * Given a positive integer n, there exists a 0-indexed array called powers, composed of the
 * minimum number of powers of 2 that sum to n. The array is sorted in non-decreasing order,
 * and there is only one way to form the array.
 *
 * You are also given a 0-indexed 2D integer array queries, where queries[i] = [lefti,
righti].
 * Each queries[i] represents a query where you have to find the product of all powers[j] with
 * lefti <= j <= righti.
 *
 * Return an array answers, equal in length to queries, where answers[i] is the answer to the
 * ith query. Since the answer to the ith query may be too large, each answers[i] should be
 * returned modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number[][]} queries
 * @return {number[]}
 */
var productQueries = function(n, queries) {
  const modulo = 1e9 + 7;
  const powers = [];
  while (n > 0) {
    const power = Math.floor(Math.log2(n));
    powers.push(1 << power);
    n -= 1 << power;
  }
  powers.reverse();

  const result = new Array(queries.length);
  for (let i = 0; i < queries.length; i++) {
```

```
    const [start, end] = queries[i];
    let product = 1;
    for (let j = start; j <= end; j++) {
      product = (product * powers[j]) % modulo;
    }
    result[i] = product;
  }

  return result;
};
```

2439 Minimize Maximum Of Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2439. Minimize Maximum of Array
 * https://leetcode.com/problems/minimize-maximum-of-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums comprising of n non-negative integers.
 *
 * In one operation, you must:
 * - Choose an integer i such that 1 <= i < n and nums[i] > 0.
 * - Decrease nums[i] by 1.
 * - Increase nums[i - 1] by 1.
 *
 * Return the minimum possible value of the maximum integer of nums after performing
 * any number of operations.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimizeArrayValue = function(nums) {
  let left = 0;
  let right = Math.max(...nums);
  let result = right;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    let carry = 0;
    let valid = true;

    for (let i = nums.length - 1; i >= 0; i--) {
      const total = nums[i] + carry;
      if (total > mid) {
        if (i === 0) {
          valid = false;
          break;
        }
        carry = total - mid;
      } else {
        carry = 0;
      }
    }

    if (valid) {
      result = mid;
      right = mid - 1;
    } else {
      left = mid + 1;
    }
  }

  return result;
};
```

2440 Create Components With Same Value

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2440. Create Components With Same Value
 * https://leetcode.com/problems/create-components-with-same-value/
 * Difficulty: Hard
 *
 * There is an undirected tree with n nodes labeled from 0 to n - 1.
 *
 * You are given a 0-indexed integer array nums of length n where nums[i] represents the value
 * of the ith node. You are also given a 2D integer array edges of length n - 1 where
 * edges[i] = [ai, bi] indicates that there is an edge between nodes ai and bi in the tree.
 *
 * You are allowed to delete some edges, splitting the tree into multiple connected
components.
 * Let the value of a component be the sum of all nums[i] for which node i is in the
component.
 *
 * Return the maximum number of edges you can delete, such that every connected component in
 * the tree has the same value.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} edges
 * @return {number}
 */
var componentValue = function(nums, edges) {
  const n = nums.length;
  const graph = Array.from({ length: n }, () => []);
  const totalSum = nums.reduce((sum, val) => sum + val, 0);

  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  for (let i = n; i >= 1; i--) {
    if (totalSum % i === 0) {
      const target = totalSum / i;
      const [, components] = countNodes(0, -1, target);
      if (components === i) {
        return i - 1;
      }
    }
  }

  return 0;

  function countNodes(node, parent, target) {
    let sum = nums[node];
    let components = 0;

    for (const neighbor of graph[node]) {
      if (neighbor !== parent) {
        const [childSum, childComponents] = countNodes(neighbor, node, target);
        sum += childSum;
        components += childComponents;
      }
    }

    if (sum === target) {
      return [0, components + 1];
    }

    return [sum, components];
  }
};
```

2441 Largest Positive Integer That Exists With Its Negative

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2441. Largest Positive Integer That Exists With Its Negative
 * https://leetcode.com/problems/largest-positive-integer-that-exists-with-its-negative/
 * Difficulty: Easy
 *
 * Given an integer array nums that does not contain any zeros, find the largest positive
 * integer k such that -k also exists in the array.
 *
 * Return the positive integer k. If there is no such integer, return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findMaxK = function(nums) {
  const set = new Set();
  let result = -1;

  for (const num of nums) {
    if (set.has(-num)) {
      result = Math.max(result, Math.abs(num));
    }
    set.add(num);
  }

  return result;
};
```

2442 Count Number Of Distinct Integers After Reverse Operations

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2442. Count Number of Distinct Integers After Reverse Operations
 * https://leetcode.com/problems/count-number-of-distinct-integers-after-reverse-operations/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of positive integers.
 *
 * You have to take each integer in the array, reverse its digits, and add it to the end of
the
 * array. You should apply this operation to the original integers in nums.
 *
 * Return the number of distinct integers in the final array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countDistinctIntegers = function(nums) {
  const set = new Set(nums);

  for (const num of nums) {
    let reversed = 0;
    let temp = num;
    while (temp > 0) {
      reversed = reversed * 10 + temp % 10;
```

```
      temp = Math.floor(temp / 10);
    }
    set.add(reversed);
  }

  return set.size;
};
```

2443 Sum Of Number And Its Reverse

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2443. Sum of Number and Its Reverse
 * https://leetcode.com/problems/sum-of-number-and-its-reverse/
 * Difficulty: Medium
 *
 * Given a non-negative integer num, return true if num can be expressed as the sum of any
 * non-negative integer and its reverse, or false otherwise.
 */

/**
 * @param {number} num
 * @return {boolean}
 */
var sumOfNumberAndReverse = function(num) {
  for (let i = 0; i <= num; i++) {
    let reversed = 0;
    let temp = i;
    while (temp > 0) {
      reversed = reversed * 10 + temp % 10;
      temp = Math.floor(temp / 10);
    }
    if (i + reversed === num) {
      return true;
    }
  }

  return false;
};
```

2444 Count Subarrays With Fixed Bounds

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2444. Count Subarrays With Fixed Bounds
 * https://leetcode.com/problems/count-subarrays-with-fixed-bounds/
 * Difficulty: Hard
 *
 * You are given an integer array nums and two integers minK and maxK.
 *
 * A fixed-bound subarray of nums is a subarray that satisfies the following conditions:
 * - The minimum value in the subarray is equal to minK.
 * - The maximum value in the subarray is equal to maxK.
 * - Return the number of fixed-bound subarrays.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @param {number} minK
 * @param {number} maxK
 * @return {number}
```

```
 */
var countSubarrays = function(nums, minK, maxK) {
  let result = 0;

  for (let i = 0, minIndex = -1, maxIndex = -1, invalidIndex = -1; i < nums.length; i++) {
    if (nums[i] < minK || nums[i] > maxK) {
      invalidIndex = i;
    }
    if (nums[i] === minK) {
      minIndex = i;
    }
    if (nums[i] === maxK) {
      maxIndex = i;
    }
    result += Math.max(0, Math.min(minIndex, maxIndex) - invalidIndex);
  }

  return result;
};
```

2445 Number Of Nodes With Value One

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2445. Number of Nodes With Value One
 * https://leetcode.com/problems/number-of-nodes-with-value-one/
 * Difficulty: Medium
 *
 * There is an undirected connected tree with n nodes labeled from 1 to n and n - 1 edges.
 * You are given the integer n. The parent node of a node with a label v is the node with
 * the label floor (v / 2). The root of the tree is the node with the label 1.
 *
 * - For example, if n = 7, then the node with the label 3 has the node with the label
 *   floor(3 / 2) = 1 as its parent, and the node with the label 7 has the node with the
 *   label floor(7 / 2) = 3 as its parent.
 *
 * You are also given an integer array queries. Initially, every node has a value 0 on it.
 * For each query queries[i], you should flip all values in the subtree of the node with
 * the label queries[i].
 *
 * Return the total number of nodes with the value 1 after processing all the queries.
 *
 * Note that:
 * - Flipping the value of a node means that the node with the value 0 becomes 1 and vice
versa.
 * - floor(x) is equivalent to rounding x down to the nearest integer.
 */

/**
 * @param {number} n
 * @param {number[]} queries
 * @return {number}
 */
var numberOfNodes = function(n, queries) {
  const flipCount = new Array(n + 1).fill(0);

  for (const query of queries) {
    flipCount[query]++;
  }

  let result = 0;
  for (let i = 1; i <= n; i++) {
    let totalFlips = 0;
    let current = i;

    while (current >= 1) {
      totalFlips += flipCount[current];
      current = Math.floor(current / 2);
    }
```

```
    if (totalFlips % 2 === 1) {
      result++;
    }
  }

  return result;
};
```

2446 Determine If Two Events Have Conflict

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2446. Determine if Two Events Have Conflict
 * https://leetcode.com/problems/determine-if-two-events-have-conflict/
 * Difficulty: Easy
 *
 * You are given two arrays of strings that represent two inclusive events that happened on
the
 * same day, event1 and event2, where:
 * - event1 = [startTime1, endTime1] and
 * - event2 = [startTime2, endTime2].
 *
 * Event times are valid 24 hours format in the form of HH:MM.
 *
 * A conflict happens when two events have some non-empty intersection (i.e., some moment is
common
 * to both events).
 *
 * Return true if there is a conflict between two events. Otherwise, return false.
 */

/**
 * @param {string[]} event1
 * @param {string[]} event2
 * @return {boolean}
 */
var haveConflict = function(event1, event2) {
  const [start1, end1] = event1;
  const [start2, end2] = event2;
  return start1 <= end2 && start2 <= end1;
};
```

2447 Number Of Subarrays With Gcd Equal To K

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2447. Number of Subarrays With GCD Equal to K
 * https://leetcode.com/problems/number-of-subarrays-with-gcd-equal-to-k/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return the number of subarrays of nums where
 * the greatest common divisor of the subarray's elements is k.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 *
 * The greatest common divisor of an array is the largest integer that evenly divides all the
 * array elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
```

```
 * @return {number}
 */
var subarrayGCD = function(nums, k) {
  let result = 0;
  for (let start = 0; start < nums.length; start++) {
    let currentGCD = nums[start];
    for (let end = start; end < nums.length; end++) {
      currentGCD = gcd(currentGCD, nums[end]);
      if (currentGCD === k) {
        result++;
      }
    }
  }

  return result;

  function gcd(a, b) {
    while (b) {
      a %= b;
      [a, b] = [b, a];
    }
    return a;
  }
};
```

2448 Minimum Cost To Make Array Equal

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2448. Minimum Cost to Make Array Equal
 * https://leetcode.com/problems/minimum-cost-to-make-array-equal/
 * Difficulty: Hard
 *
 * You are given two 0-indexed arrays nums and cost consisting each of n positive integers.
 *
 * You can do the following operation any number of times:
 * - Increase or decrease any element of the array nums by 1.
 *
 * The cost of doing one operation on the ith element is cost[i].
 *
 * Return the minimum total cost such that all the elements of the array nums become equal.
 */

/**
 * @param {number[]} nums
 * @param {number[]} cost
 * @return {number}
 */
var minCost = function(nums, cost) {
  const pairs = nums.map((num, i) => [num, cost[i]]).sort((a, b) => a[0] - b[0]);
  const n = nums.length;
  const prefixSum = Array(n).fill(0);
  const prefixCost = Array(n).fill(0);

  prefixSum[0] = pairs[0][0] * pairs[0][1];
  prefixCost[0] = pairs[0][1];

  for (let i = 1; i < n; i++) {
    prefixSum[i] = prefixSum[i - 1] + pairs[i][0] * pairs[i][1];
    prefixCost[i] = prefixCost[i - 1] + pairs[i][1];
  }

  let result = Infinity;
  for (let i = 0; i < n; i++) {
    const target = pairs[i][0];
    const leftCost = i > 0 ? target * prefixCost[i - 1] - prefixSum[i - 1] : 0;
    const rightCost = i < n - 1 ? prefixSum[n - 1] - prefixSum[i]
      - target * (prefixCost[n - 1] - prefixCost[i]) : 0;
    result = Math.min(result, leftCost + rightCost);
```

```
  }

  return result;
};
```

2449 Minimum Number Of Operations To Make Arrays Similar

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2449. Minimum Number of Operations to Make Arrays Similar
 * https://leetcode.com/problems/minimum-number-of-operations-to-make-arrays-similar/
 * Difficulty: Hard
 *
 * You are given two positive integer arrays nums and target, of the same length.
 *
 * In one operation, you can choose any two distinct indices i and j where 0 <= i,
 * j < nums.length and:
 * - set nums[i] = nums[i] + 2 and
 * - set nums[j] = nums[j] - 2.
 *
 * Two arrays are considered to be similar if the frequency of each element is the same.
 *
 * Return the minimum number of operations required to make nums similar to target. The
 * test cases are generated such that nums can always be similar to target.
 */

/**
 * @param {number[]} nums
 * @param {number[]} target
 * @return {number}
 */
var makeSimilar = function(nums, target) {
  const evenNums = nums.filter(num => num % 2 === 0).sort((a, b) => a - b);
  const oddNums = nums.filter(num => num % 2 === 1).sort((a, b) => a - b);
  const evenTarget = target.filter(num => num % 2 === 0).sort((a, b) => a - b);
  const oddTarget = target.filter(num => num % 2 === 1).sort((a, b) => a - b);

  let result = 0;
  for (let i = 0; i < evenNums.length; i++) {
    if (evenNums[i] > evenTarget[i]) {
      result += (evenNums[i] - evenTarget[i]) / 2;
    }
  }

  for (let i = 0; i < oddNums.length; i++) {
    if (oddNums[i] > oddTarget[i]) {
      result += (oddNums[i] - oddTarget[i]) / 2;
    }
  }

  return result;
};
```

2450 Number Of Distinct Binary Strings After Applying Operations

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2450. Number of Distinct Binary Strings After Applying Operations
 * https://leetcode.com/problems/number-of-distinct-binary-strings-after-applying-operations/
 * Difficulty: Medium
 *
 * You are given a binary string s and a positive integer k.
 *
```

```
 * You can apply the following operation on the string any number of times:
 * - Choose any substring of size k from s and flip all its characters, that is, turn all 1's
 *   into 0's, and all 0's into 1's.
 * - Return the number of distinct strings you can obtain. Since the answer may be too large,
 *   return it modulo 109 + 7.
 *
 * Note that:
 * - A binary string is a string that consists only of the characters 0 and 1.
 * - A substring is a contiguous part of a string.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var countDistinctStrings = function(s, k) {
  if (s.length <= 0) {
    return -1;
  }

  const MOD = 1e9 + 7;
  let n = s.length - k + 1;
  let result = 1;

  while (n > 0) {
    result = (result << 1) % MOD;
    n--;
  }

  return result;
};
```

2451 Odd String Difference

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2451. Odd String Difference
 * https://leetcode.com/problems/odd-string-difference/
 * Difficulty: Easy
 *
 * You are given an array of equal-length strings words. Assume that the length of each
 * string is n.
 *
 * Each string words[i] can be converted into a difference integer array difference[i] of
 * length n - 1 where difference[i][j] = words[i][j+1] - words[i][j] where 0 <= j <= n - 2.
 * Note that the difference between two letters is the difference between their positions in
 * the alphabet i.e. the position of 'a' is 0, 'b' is 1, and 'z' is 25.
 * - For example, for the string "acb", the difference integer array is [2 - 0, 1 - 2] = [2,
-1].
 *
 * All the strings in words have the same difference integer array, except one. You should
find
 * that string.
 *
 * Return the string in words that has different difference integer array.
 */

/**
 * @param {string[]} words
 * @return {string}
 */
var oddString = function(words) {
  const map = new Map();
  for (const word of words) {
    const diff = getDiff(word);
    map.set(diff, (map.get(diff) || 0) + 1);
  }
```

```
  let oddDiff;
  for (const [diff, count] of map) {
    if (count === 1) {
      oddDiff = diff;
      break;
    }
  }

  for (const word of words) {
    if (getDiff(word) === oddDiff) {
      return word;
    }
  }

  function getDiff(word) {
    const diff = [];
    for (let i = 1; i < word.length; i++) {
      diff.push(word.charCodeAt(i) - word.charCodeAt(i - 1));
    }
    return diff.join(',');
  }
};
```

2452 Words Within Two Edits Of Dictionary

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2452. Words Within Two Edits of Dictionary
 * https://leetcode.com/problems/words-within-two-edits-of-dictionary/
 * Difficulty: Medium
 *
 * You are given two string arrays, queries and dictionary. All words in each array comprise
of
 * lowercase English letters and have the same length.
 *
 * In one edit you can take a word from queries, and change any letter in it to any other
letter.
 * Find all words from queries that, after a maximum of two edits, equal some word from
dictionary.
 *
 * Return a list of all words from queries, that match with some word from dictionary after a
 * maximum of two edits. Return the words in the same order they appear in queries.
 */

/**
 * @param {string[]} queries
 * @param {string[]} dictionary
 * @return {string[]}
 */
var twoEditWords = function(queries, dictionary) {
  const result = [];

  for (const query of queries) {
    for (const word of dictionary) {
      let edits = 0;
      for (let i = 0; i < query.length; i++) {
        if (query[i] !== word[i]) {
          edits++;
          if (edits > 2) break;
        }
      }
      if (edits <= 2) {
        result.push(query);
        break;
      }
    }
  }

  return result;
```

```
};
```

2453 Destroy Sequential Targets

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2453. Destroy Sequential Targets
 * https://leetcode.com/problems/destroy-sequential-targets/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums consisting of positive integers, representing targets
 * on a number line. You are also given an integer space.
 *
 * You have a machine which can destroy targets. Seeding the machine with some nums[i] allows
 * it to destroy all targets with values that can be represented as nums[i] + c * space, where
 * c is any non-negative integer. You want to destroy the maximum number of targets in nums.
 *
 * Return the minimum value of nums[i] you can seed the machine with to destroy the maximum
 * number of targets.
 */

/**
 * @param {number[]} nums
 * @param {number} space
 * @return {number}
 */
var destroyTargets = function(nums, space) {
  const map = new Map();
  let maxCount = 0;

  for (const num of nums) {
    const remainder = num % space;
    map.set(remainder, (map.get(remainder) || 0) + 1);
    maxCount = Math.max(maxCount, map.get(remainder));
  }

  let result = Infinity;
  for (const num of nums) {
    if (map.get(num % space) === maxCount) {
      result = Math.min(result, num);
    }
  }

  return result;
};
```

2455 Average Value Of Even Numbers That Are Divisible By Three

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2455. Average Value of Even Numbers That Are Divisible by Three
 * https://leetcode.com/problems/average-value-of-even-numbers-that-are-divisible-by-three/
 * Difficulty: Easy
 *
 * Given an integer array nums of positive integers, return the average value of all even
integers
 * that are divisible by 3.
 *
 * Note that the average of n elements is the sum of the n elements divided by n and rounded
down
 * to the nearest integer.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var averageValue = function(nums) {
  let sum = 0;
  let count = 0;

  for (const num of nums) {
    if (num % 2 === 0 && num % 3 === 0) {
      sum += num;
      count++;
    }
  }

  return count > 0 ? Math.floor(sum / count) : 0;
};
```

2456 Most Popular Video Creator

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2456. Most Popular Video Creator
 * https://leetcode.com/problems/most-popular-video-creator/
 * Difficulty: Medium
 *
 * You are given two string arrays creators and ids, and an integer array views, all of length
 n.
 * The ith video on a platform was created by creators[i], has an id of ids[i], and has
views[i]
 * views.
 *
 * The popularity of a creator is the sum of the number of views on all of the creator's
videos.
 * Find the creator with the highest popularity and the id of their most viewed video.
 * - If multiple creators have the highest popularity, find all of them.
 * - If multiple videos have the highest view count for a creator, find the lexicographically
 *   smallest id.
 *
 * Note: It is possible for different videos to have the same id, meaning that ids do not
uniquely
 * identify a video. For example, two videos with the same ID are considered as distinct
videos
 * with their own viewcount.
 *
 * Return a 2D array of strings answer where answer[i] = [creatorsi, idi] means that creatorsi
 * has the highest popularity and idi is the id of their most popular video. The answer can be
 * returned in any order.
 */

/**
 * @param {string[]} creators
 * @param {string[]} ids
 * @param {number[]} views
 * @return {string[][]}
 */
var mostPopularCreator = function(creators, ids, views) {
  const map = new Map();

  for (let i = 0; i < creators.length; i++) {
    const creator = creators[i];
    const videoId = ids[i];
    const videoViews = views[i];

    if (!map.has(creator)) {
      map.set(creator, {
        totalViews: 0,
        mostViewedCount: -1,
        mostViewedId: ''
```

```
      });
    }

    const stats = map.get(creator);
    stats.totalViews += videoViews;

    if (videoViews > stats.mostViewedCount || (videoViews === stats.mostViewedCount
        && videoId < stats.mostViewedId)) {
      stats.mostViewedCount = videoViews;
      stats.mostViewedId = videoId;
    }
  }

  const maxViews = Math.max(...Array.from(map.values()).map(s => s.totalViews));

  const result = [];
  for (const [creator, stats] of map) {
    if (stats.totalViews === maxViews) {
      result.push([creator, stats.mostViewedId]);
    }
  }

  return result;
};
```

2458 Height Of Binary Tree After Subtree Removal Queries

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2458. Height of Binary Tree After Subtree Removal Queries
 * https://leetcode.com/problems/height-of-binary-tree-after-subtree-removal-queries/
 * Difficulty: Hard
 *
 * You are given the root of a binary tree with n nodes. Each node is assigned a unique value
 * from 1 to n. You are also given an array queries of size m.
 *
 * You have to perform m independent queries on the tree where in the ith query you do the
 * following:
 * - Remove the subtree rooted at the node with the value queries[i] from the tree. It is
 *   guaranteed that queries[i] will not be equal to the value of the root.
 *
 * Return an array answer of size m where answer[i] is the height of the tree after performing
 * the ith query.
 *
 * Note:
 * - The queries are independent, so the tree returns to its initial state after each query.
 * - The height of a tree is the number of edges in the longest simple path from the root to
 *   some node in the tree.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number[]} queries
 * @return {number[]}
 */
var treeQueries = function(root, queries) {
  const heights = new Map();
  const maxHeightWithout = new Map();

  computeHeight(root);
  computeMaxHeight(root, 0, -1);
```

```
  const result = new Array(queries.length);
  for (let i = 0; i < queries.length; i++) {
    result[i] = maxHeightWithout.get(queries[i]);
  }

  return result;

  function computeHeight(node) {
    if (!node) return -1;
    const leftHeight = computeHeight(node.left);
    const rightHeight = computeHeight(node.right);
    heights.set(node.val, Math.max(leftHeight, rightHeight) + 1);
    return heights.get(node.val);
  }

  function computeMaxHeight(node, depth, cousinHeight) {
    if (!node) return;
    const leftHeight = node.left ? heights.get(node.left.val) : -1;
    const rightHeight = node.right ? heights.get(node.right.val) : -1;
    const maxAlternative = Math.max(cousinHeight, depth - 1);
    maxHeightWithout.set(node.val, maxAlternative);

    computeMaxHeight(node.left, depth + 1, Math.max(cousinHeight, rightHeight + depth + 1));
    computeMaxHeight(node.right, depth + 1, Math.max(cousinHeight, leftHeight + depth + 1));
  }
};
```

2459 Sort Array By Moving Items To Empty Space

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2459. Sort Array by Moving Items to Empty Space
 * https://leetcode.com/problems/sort-array-by-moving-items-to-empty-space/
 * Difficulty: Hard
 *
 * You are given an integer array nums of size n containing each element from 0 to n - 1
 * (inclusive). Each of the elements from 1 to n - 1 represents an item, and the element
 * 0 represents an empty space.
 *
 * In one operation, you can move any item to the empty space. nums is considered to be sorted
 * if the numbers of all the items are in ascending order and the empty space is either at the
 * beginning or at the end of the array.
 *
 * For example, if n = 4, nums is sorted if:
 * - nums = [0,1,2,3] or
 * - nums = [1,2,3,0]
 *
 * ...and considered to be unsorted otherwise.
 *
 * Return the minimum number of operations needed to sort nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sortArray = function(nums) {
  const n = nums.length;
  const positions = new Array(n);

  for (let i = 0; i < n; i++) {
    positions[nums[i]] = i;
  }

  return Math.min(
    permute([...positions], 0),
    permute([...positions], 1)
  );
```

```
  function permute(pos, shift) {
    let count = 0;
    let next = 1;

    while (next < n) {
      if (pos[0] === shift * (n - 1)) {
        while (pos[next] === next - shift) {
          next++;
          if (next === n) return count;
        }
        var targetIndex = next;
      } else {
        var targetIndex = pos[0] + shift;
      }

      [pos[0], pos[targetIndex]] = [pos[targetIndex], pos[0]];
      count++;
    }

    return count;
  }
};
```

2460 Apply Operations To An Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2460. Apply Operations to an Array
 * https://leetcode.com/problems/apply-operations-to-an-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array nums of size n consisting of non-negative integers.
 *
 * You need to apply n - 1 operations to this array where, in the ith operation (0-indexed),
 * you will apply the following on the ith element of nums:
 * - If nums[i] == nums[i + 1], then multiply nums[i] by 2 and set nums[i + 1] to 0.
 *   Otherwise, you skip this operation.
 *
 * After performing all the operations, shift all the 0's to the end of the array.
 * - For example, the array [1,0,2,0,0,1] after shifting all its 0's to the end, is
[1,2,1,0,0,0].
 *
 * Return the resulting array.
 *
 * Note that the operations are applied sequentially, not all at once.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var applyOperations = function(nums) {
  for (let i = 0; i < nums.length - 1; i++) {
    if (nums[i] === nums[i + 1]) {
      nums[i] *= 2;
      nums[i + 1] = 0;
    }
  }
  return [...nums.filter(n => n), ...nums.filter(n => !n)];
};
```

2461 Maximum Sum Of Distinct Subarrays With Length K

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2461. Maximum Sum of Distinct Subarrays With Length K
 * https://leetcode.com/problems/maximum-sum-of-distinct-subarrays-with-length-k/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. Find the maximum subarray sum of all
the
 * subarrays of nums that meet the following conditions:
 * - The length of the subarray is k, and
 * - All the elements of the subarray are distinct.
 *
 * Return the maximum subarray sum of all the subarrays that meet the conditions. If no
subarray
 * meets the conditions, return 0.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumSubarraySum = function(nums, k) {
  let result = 0;
  const seen = new Map();
  let windowSum = 0;

  for (let i = 0; i < nums.length; i++) {
    seen.set(nums[i], (seen.get(nums[i]) || 0) + 1);
    windowSum += nums[i];

    if (i >= k - 1) {
      if (seen.size === k) {
        result = Math.max(result, windowSum);
      }
      const leftNum = nums[i - k + 1];
      windowSum -= leftNum;
      seen.set(leftNum, seen.get(leftNum) - 1);
      if (seen.get(leftNum) === 0) {
        seen.delete(leftNum);
      }
    }
  }

  return result;
};
```

2462 Total Cost To Hire K Workers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2462. Total Cost to Hire K Workers
 * https://leetcode.com/problems/total-cost-to-hire-k-workers/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array costs where costs[i] is the cost of hiring
 * the ith worker.
 *
 * You are also given two integers k and candidates. We want to hire exactly k workers
 * according to the following rules:
 * - You will run k sessions and hire exactly one worker in each session.
 * - In each hiring session, choose the worker with the lowest cost from either the first
 *   candidates workers or the last candidates workers. Break the tie by the smallest index.
 *   - For example, if costs = [3,2,7,7,1,2] and candidates = 2, then in the first hiring
 *     session, we will choose the 4th worker because they have the lowest cost [3,2,7,7,1,2].
```

```
 *   - In the second hiring session, we will choose 1st worker because they have the same
 *     lowest cost as 4th worker but they have the smallest index [3,2,7,7,2]. Please note
 *     that the indexing may be changed in the process.
 * - If there are fewer than candidates workers remaining, choose the worker with the lowest
 *   cost among them. Break the tie by the smallest index.
 * - A worker can only be chosen once.
 *
 * Return the total cost to hire exactly k workers.
 */

/**
 * @param {number[]} costs
 * @param {number} k
 * @param {number} candidates
 * @return {number}
 */
var totalCost = function(costs, k, candidates) {
  const queue = new PriorityQueue({ compare: (a, b) => {
    return a[0] === b[0] ? a[1] - b[1] : a[0] - b[0];
  }});

  costs.forEach((cost, index) => {
    if (index < candidates || index >= costs.length - candidates) {
      queue.enqueue([cost, index]);
    }
  });

  let result = 0;
  for (let i = 0, count = candidates, diff = costs.length - candidates - 1; i < k; i++) {
    const worker = queue.dequeue();
    result += worker[0];

    if (count <= diff) {
      let status = null;
      if (worker[1] < count) {
        status = [costs[count], count];
        count++;
      } else {
        status = [costs[diff], diff];
        diff--;
      }
      queue.enqueue(status);
    }
  }

  return result;
};
```

2463 Minimum Total Distance Traveled

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2463. Minimum Total Distance Traveled
 * https://leetcode.com/problems/minimum-total-distance-traveled/
 * Difficulty: Hard
 *
 * There are some robots and factories on the X-axis. You are given an integer array robot
where
 * robot[i] is the position of the ith robot. You are also given a 2D integer array factory
where
 * factory[j] = [positionj, limitj] indicates that positionj is the position of the jth
factory
 * and that the jth factory can repair at most limitj robots.
 *
 * The positions of each robot are unique. The positions of each factory are also unique. Note
 * that a robot can be in the same position as a factory initially.
 *
 * All the robots are initially broken; they keep moving in one direction. The direction could
 be
```

```
 * the negative or the positive direction of the X-axis. When a robot reaches a factory that
did
 * not reach its limit, the factory repairs the robot, and it stops moving.
 *
 * At any moment, you can set the initial direction of moving for some robot. Your target is
to
 * minimize the total distance traveled by all the robots.
 *
 * Return the minimum total distance traveled by all the robots. The test cases are generated
 * such that all the robots can be repaired.
 *
 * Note that:
 * - All robots move at the same speed.
 * - If two robots move in the same direction, they will never collide.
 * - If two robots move in opposite directions and they meet at some point, they do not
collide.
 *   They cross each other.
 * - If a robot passes by a factory that reached its limits, it crosses it as if it does not
exist.
 * - If the robot moved from a position x to a position y, the distance it moved is |y - x|.
 */

/**
 * @param {number[]} robot
 * @param {number[][]} factory
 * @return {number}
 */
var minimumTotalDistance = function(robot, factory) {
  robot.sort((a, b) => a - b);
  factory.sort((a, b) => a[0] - b[0]);

  const robotCount = robot.length;
  const factoryCount = factory.length;
  const memo = new Array(robotCount + 1).fill(null).map(() => new Array(factoryCount +
1).fill(-1));

  return calculateMinDistance(0, 0);

  function calculateMinDistance(robotIndex, factoryIndex) {
    if (robotIndex === robotCount) return 0;
    if (factoryIndex === factoryCount) return 1e18;

    if (memo[robotIndex][factoryIndex] !== -1) {
      return memo[robotIndex][factoryIndex];
    }

    let result = calculateMinDistance(robotIndex, factoryIndex + 1);

    let totalDistance = 0;
    for (let robotsTaken = 0; robotsTaken < factory[factoryIndex][1]
         && robotIndex + robotsTaken < robotCount; robotsTaken++) {
      totalDistance += Math.abs(robot[robotIndex + robotsTaken] - factory[factoryIndex][0]);
      result = Math.min(
        result,
        totalDistance + calculateMinDistance(robotIndex + robotsTaken + 1, factoryIndex + 1)
      );
    }

    memo[robotIndex][factoryIndex] = result;
    return result;
  }
};
```

2464 Minimum Subarrays In A Valid Split

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2464. Minimum Subarrays in a Valid Split
 * https://leetcode.com/problems/minimum-subarrays-in-a-valid-split/
```

```
 * Difficulty: Medium
 *
 * You are given an integer array nums.
 *
 * Splitting of an integer array nums into subarrays is valid if:
 * - the greatest common divisor of the first and last elements of each subarray is
 *   greater than 1, and
 * - each element of nums belongs to exactly one subarray.
 *
 * Return the minimum number of subarrays in a valid subarray splitting of nums.
 * If a valid subarray splitting is not possible, return -1.
 *
 * Note that:
 * - The greatest common divisor of two numbers is the largest positive integer that
 *   evenly divides both numbers.
 * - A subarray is a contiguous non-empty part of an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var validSubarraySplit = function(nums) {
  const n = nums.length;
  const dp = new Array(n + 1).fill(Infinity);
  dp[0] = 0;

  for (let i = 0; i < n; i++) {
    if (dp[i] === Infinity) continue;

    for (let j = i; j < n; j++) {
      if (gcd(nums[i], nums[j]) > 1) {
        dp[j + 1] = Math.min(dp[j + 1], dp[i] + 1);
      }
    }
  }

  return dp[n] === Infinity ? -1 : dp[n];

  function gcd(a, b) {
    while (b !== 0) {
      const temp = b;
      b = a % b;
      a = temp;
    }
    return a;
  }
};
```

2465 Number Of Distinct Averages

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2465. Number of Distinct Averages
 * https://leetcode.com/problems/number-of-distinct-averages/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums of even length.
 *
 * As long as nums is not empty, you must repetitively:
 * - Find the minimum number in nums and remove it.
 * - Find the maximum number in nums and remove it.
 * - Calculate the average of the two removed numbers.
 *
 * The average of two numbers a and b is (a + b) / 2.
 * - For example, the average of 2 and 3 is (2 + 3) / 2 = 2.5.
 *
 * Return the number of distinct averages calculated using the above process.
```

```
 *
 * Note that when there is a tie for a minimum or maximum number, any can be removed.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var distinctAverages = function(nums) {
  nums.sort((a, b) => a - b);
  const averages = new Set();

  for (let i = 0, j = nums.length - 1; i < j; i++, j--) {
    averages.add((nums[i] + nums[j]) / 2);
  }

  return averages.size;
};
```

2466 Count Ways To Build Good Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2466. Count Ways To Build Good Strings
 * https://leetcode.com/problems/count-ways-to-build-good-strings/
 * Difficulty: Medium
 *
 * Given the integers zero, one, low, and high, we can construct a string by starting with an
 * empty string, and then at each step perform either of the following:
 * - Append the character '0' zero times.
 * - Append the character '1' one times.
 *
 * This can be performed any number of times.
 *
 * A good string is a string constructed by the above process having a length between low and
 * high (inclusive).
 *
 * Return the number of different good strings that can be constructed satisfying these
properties.
 * Since the answer can be large, return it modulo 109 + 7.
 */

/**
 * @param {number} low
 * @param {number} high
 * @param {number} zero
 * @param {number} one
 * @return {number}
 */
var countGoodStrings = function(low, high, zero, one) {
  const modulo = 1e9 + 7;
  const dp = new Array(high + 1).fill(0);
  dp[0] = 1;

  for (let length = 1; length <= high; length++) {
    if (length >= zero) {
      dp[length] = (dp[length] + dp[length - zero]) % modulo;
    }
    if (length >= one) {
      dp[length] = (dp[length] + dp[length - one]) % modulo;
    }
  }

  let result = 0;
  for (let length = low; length <= high; length++) {
    result = (result + dp[length]) % modulo;
  }

  return result;
```

```
};
```

2467 Most Profitable Path In A Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2467. Most Profitable Path in a Tree
 * https://leetcode.com/problems/most-profitable-path-in-a-tree/
 * Difficulty: Medium
 *
 * There is an undirected tree with n nodes labeled from 0 to n - 1, rooted at node 0. You are
 * given a 2D integer array edges of length n - 1 where edges[i] = [ai, bi] indicates that
 * there is an edge between nodes ai and bi in the tree.
 *
 * At every node i, there is a gate. You are also given an array of even integers amount,
where
 * amount[i] represents:
 * - the price needed to open the gate at node i, if amount[i] is negative, or,
 * - the cash reward obtained on opening the gate at node i, otherwise.
 *
 * The game goes on as follows:
 * - Initially, Alice is at node 0 and Bob is at node bob.
 * - At every second, Alice and Bob each move to an adjacent node. Alice moves towards some
leaf
 *   node, while Bob moves towards node 0.
 * - For every node along their path, Alice and Bob either spend money to open the gate at
that
 *   node, or accept the reward. Note that:
 *   - If the gate is already open, no price will be required, nor will there be any cash
reward.
 *   - If Alice and Bob reach the node simultaneously, they share the price/reward for opening
 *     the gate there. In other words, if the price to open the gate is c, then both Alice and
 *     Bob pay c / 2 each. Similarly, if the reward at the gate is c, both of them receive
 *     c / 2 each.
 * - If Alice reaches a leaf node, she stops moving. Similarly, if Bob reaches node 0, he
stops
 *   moving. Note that these events are independent of each other.
 *
 * Return the maximum net income Alice can have if she travels towards the optimal leaf node.
 */

/**
 * @param {number[][]} edges
 * @param {number} bob
 * @param {number[]} amount
 * @return {number}
 */
var mostProfitablePath = function(edges, bob, amount) {
  const graph = new Array(edges.length + 1).fill().map(() => []);
  const path = new Array(edges.length + 1).fill(-1);

  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  function traceInitialPath(node, time) {
    path[node] = time;
    if (node === 0) {
      return true;
    }
    for (const next of graph[node]) {
      if (path[next] === -1 && traceInitialPath(next, time + 1)) {
        return true;
      }
    }
    path[node] = -1;
    return false;
  }
```

```
  traceInitialPath(bob, 0);

  function dfs(node, parent, time, income) {
    const current = (time < path[node] || path[node] === -1)
      ? amount[node]
      : time === path[node] ? amount[node] / 2 : 0;
    let maxIncome = graph[node].length === 1 && node !== 0 ? income + current : -Infinity;

    for (const next of graph[node]) {
      if (next !== parent) {
        maxIncome = Math.max(maxIncome, dfs(next, node, time + 1, income + current));
      }
    }

    return maxIncome === -Infinity ? income + current : maxIncome;
  }

  return dfs(0, -1, 0, 0);
};
```

2468 Split Message Based On Limit

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2468. Split Message Based on Limit
 * https://leetcode.com/problems/split-message-based-on-limit/
 * Difficulty: Hard
 *
 * You are given a string, message, and a positive integer, limit.
 *
 * You must split message into one or more parts based on limit. Each resulting part should
have
 * the suffix "<a/b>", where "b" is to be replaced with the total number of parts and "a" is
to
 * be replaced with the index of the part, starting from 1 and going up to b. Additionally,
the
 * length of each resulting part (including its suffix) should be equal to limit, except for
the
 * last part whose length can be at most limit.
 *
 * The resulting parts should be formed such that when their suffixes are removed and they are
 * all concatenated in order, they should be equal to message. Also, the result should contain
 * as few parts as possible.
 *
 * Return the parts message would be split into as an array of strings. If it is impossible to
 * split message as required, return an empty array.
 */

/**
 * @param {string} message
 * @param {number} limit
 * @return {string[]}
 */
var splitMessage = function(message, limit) {
  const n = message.length;
  let digitSum = 0;

  for (let parts = 1; parts <= n; parts++) {
    digitSum += String(parts).length;
    const indexLength = String(parts).length * parts;
    const formatLength = 3 * parts;

    if (limit * parts - (digitSum + indexLength + formatLength) >= n) {
      const result = [];
      let index = 0;

      for (let i = 1; i <= parts; i++) {
        const suffix = `<${i}/${parts}>`;
```

```
        const chars = limit - suffix.length;
        result.push(message.slice(index, index + chars) + suffix);
        index += chars;
      }

      return result;
    }
  }

  return [];
};
```

2469 Convert The Temperature

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2469. Convert the Temperature
 * https://leetcode.com/problems/convert-the-temperature/
 * Difficulty: Easy
 *
 * You are given a non-negative floating point number rounded to two decimal places celsius,
 * that denotes the temperature in Celsius.
 *
 * You should convert Celsius into Kelvin and Fahrenheit and return it as an array
 * ans = [kelvin, fahrenheit].
 *
 * Return the array ans. Answers within 10-5 of the actual answer will be accepted.
 */

/**
 * @param {number} celsius
 * @return {number[]}
 */
var convertTemperature = function(celsius) {
  return [celsius + 273.15, celsius * 1.80 + 32];
};
```

2470 Number Of Subarrays With Lcm Equal To K

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2470. Number of Subarrays With LCM Equal to K
 * https://leetcode.com/problems/number-of-subarrays-with-lcm-equal-to-k/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return the number of subarrays of nums
 * where the least common multiple of the subarray's elements is k.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 *
 * The least common multiple of an array is the smallest positive integer that is divisible
 * by all the array elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var subarrayLCM = function(nums, k) {
  let count = 0;

  for (let start = 0; start < nums.length; start++) {
```

```
    let currentLCM = nums[start];
    for (let end = start; end < nums.length; end++) {
      currentLCM = lcm(currentLCM, nums[end]);
      if (currentLCM > k) break;
      if (currentLCM === k) count++;
    }
  }

  return count;

  function gcd(a, b) {
    while (b) {
      a %= b;
      [a, b] = [b, a];
    }
    return a;
  }

  function lcm(a, b) {
    return (a * b) / gcd(a, b);
  }
};
```

2471 Minimum Number Of Operations To Sort A Binary Tree By Level

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2471. Minimum Number of Operations to Sort a Binary Tree by Level
 * https://leetcode.com/problems/minimum-number-of-operations-to-sort-a-binary-tree-by-level/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree with unique values.
 *
 * In one operation, you can choose any two nodes at the same level and swap their values.
 *
 * Return the minimum number of operations needed to make the values at each level sorted in a
 * strictly increasing order.
 *
 * The level of a node is the number of edges along the path between it and the root node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var minimumOperations = function(root) {
  let result = 0;
  const queue = [root];
  const levelValues = [];

  while (queue.length) {
    const levelSize = queue.length;
    const values = [];

    for (let i = 0; i < levelSize; i++) {
      const node = queue.shift();
      values.push(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
```

```
    levelValues.push(values);
  }

  for (const values of levelValues) {
    const sorted = [...values].sort((a, b) => a - b);
    const indexMap = new Map(values.map((val, i) => [val, i]));
    let swaps = 0;

    for (let i = 0; i < values.length; i++) {
      if (values[i] !== sorted[i]) {
        const targetIndex = indexMap.get(sorted[i]);
        [values[i], values[targetIndex]] = [values[targetIndex], values[i]];
        indexMap.set(values[targetIndex], targetIndex);
        indexMap.set(values[i], i);
        swaps++;
      }
    }

    result += swaps;
  }

  return result;
};
```

2472 Maximum Number Of Non Overlapping Palindrome Substrings

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2472. Maximum Number of Non-overlapping Palindrome Substrings
 * https://leetcode.com/problems/maximum-number-of-non-overlapping-palindrome-substrings/
 * Difficulty: Hard
 *
 * You are given a string s and a positive integer k.
 *
 * Select a set of non-overlapping substrings from the string s that satisfy the following
 * conditions:
 * - The length of each substring is at least k.
 * - Each substring is a palindrome.
 *
 * Return the maximum number of substrings in an optimal selection.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var maxPalindromes = function(s, k) {
  const n = s.length;
  const isPal = new Array(n).fill().map(() => new Array(n).fill(false));
  const dp = new Array(n + 1).fill(0);

  for (let len = 1; len <= n; len++) {
    for (let start = 0; start + len <= n; start++) {
      const end = start + len - 1;
      if (len === 1) {
        isPal[start][end] = true;
      } else if (len === 2) {
        isPal[start][end] = s[start] === s[end];
      } else {
        isPal[start][end] = s[start] === s[end] && isPal[start + 1][end - 1];
      }
    }
  }

  for (let end = k; end <= n; end++) {
```

```
    dp[end] = dp[end - 1];
    for (let start = end - k; start >= 0; start--) {
      if (isPal[start][end - 1]) {
        dp[end] = Math.max(dp[end], dp[start] + 1);
      }
    }
  }

  return dp[n];
};
```

2473 Minimum Cost To Buy Apples

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2473. Minimum Cost to Buy Apples
 * https://leetcode.com/problems/minimum-cost-to-buy-apples/
 * Difficulty: Medium
 *
 * You are given a positive integer n representing n cities numbered from 1 to n. You are also
 * given a 2D array roads, where roads[i] = [ai, bi, costi] indicates that there is a
bidirectional
 * road between cities ai and bi with a cost of traveling equal to costi.
 *
 * You can buy apples in any city you want, but some cities have different costs to buy
apples.
 * You are given the 1-based array appleCost where appleCost[i] is the cost of buying one
apple
 * from city i.
 *
 * You start at some city, traverse through various roads, and eventually buy exactly one
apple
 * from any city. After you buy that apple, you have to return back to the city you started
at,
 * but now the cost of all the roads will be multiplied by a given factor k.
 *
 * Given the integer k, return a 1-based array answer of size n where answer[i] is the minimum
 * total cost to buy an apple if you start at city i.
 */

/**
 * @param {number} n
 * @param {number[][]} roads
 * @param {number[]} appleCost
 * @param {number} k
 * @return {number[]}
 */
var minCost = function(n, roads, appleCost, k) {
  const graph = new Array(n).fill().map(() => []);
  for (const [cityA, cityB, cost] of roads) {
    graph[cityA - 1].push([cityB - 1, cost]);
    graph[cityB - 1].push([cityA - 1, cost]);
  }

  const result = new Array(n);
  for (let startCity = 0; startCity < n; startCity++) {
    result[startCity] = appleCost[startCity];
  }

  const heap = [];
  for (let startCity = 0; startCity < n; startCity++) {
    heap.push([appleCost[startCity], startCity]);
  }
  heap.sort((a, b) => a[0] - b[0]);

  while (heap.length > 0) {
    const [totalCost, currCity] = heap.shift();
```

```
    if (result[currCity] < totalCost) continue;

    for (const [neighbor, cost] of graph[currCity]) {
      const newCost = result[currCity] + (k + 1) * cost;
      if (result[neighbor] > newCost) {
        result[neighbor] = newCost;

        let insertIndex = 0;
        while (insertIndex < heap.length && heap[insertIndex][0] <= newCost) {
          insertIndex++;
        }
        heap.splice(insertIndex, 0, [newCost, neighbor]);
      }
    }
  }

  return result;
};
```

2475 Number Of Unequal Triplets In Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2475. Number of Unequal Triplets in Array
 * https://leetcode.com/problems/number-of-unequal-triplets-in-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array of positive integers nums. Find the number of
 * triplets (i, j, k) that meet the following conditions:
 * - 0 <= i < j < k < nums.length
 * - nums[i], nums[j], and nums[k] are pairwise distinct.
 *   - In other words, nums[i] != nums[j], nums[i] != nums[k], and nums[j] != nums[k].
 *
 * Return the number of triplets that meet the conditions.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var unequalTriplets = function(nums) {
  const map = new Map();
  for (const num of nums) {
    map.set(num, (map.get(num) || 0) + 1);
  }

  let result = 0;
  let left = 0;
  const total = nums.length;

  for (const count of map.values()) {
    const right = total - count - left;
    result += left * count * right;
    left += count;
  }

  return result;
};
```

2476 Closest Nodes Queries In A Binary Search Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
```

```
 * 2476. Closest Nodes Queries in a Binary Search Tree
 * https://leetcode.com/problems/closest-nodes-queries-in-a-binary-search-tree/
 * Difficulty: Medium
 *
 * You are given the root of a binary search tree and an array queries of size n consisting of
 * positive integers.
 *
 * Find a 2D array answer of size n where answer[i] = [mini, maxi]:
 * - mini is the largest value in the tree that is smaller than or equal to queries[i]. If a
such
 *   value does not exist, add -1 instead.
 * - maxi is the smallest value in the tree that is greater than or equal to queries[i]. If a
such
 *   value does not exist, add -1 instead.
 *
 * Return the array answer.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number[]} queries
 * @return {number[][]}
 */
var closestNodes = function(root, queries) {
  const values = [];

  inorder(root);

  const result = queries.map(query => {
    let minVal = -1;
    let maxVal = -1;

    let left = 0;
    let right = values.length - 1;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      if (values[mid] === query) {
        return [query, query];
      } else if (values[mid] < query) {
        minVal = values[mid];
        left = mid + 1;
      } else {
        maxVal = values[mid];
        right = mid - 1;
      }
    }

    return [minVal, maxVal];
  });

  return result;

  function inorder(node) {
    if (!node) return;
    inorder(node.left);
    values.push(node.val);
    inorder(node.right);
  }
};
```

2477 Minimum Fuel Cost To Report To The Capital

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2477. Minimum Fuel Cost to Report to the Capital
 * https://leetcode.com/problems/minimum-fuel-cost-to-report-to-the-capital/
 * Difficulty: Medium
 *
 * There is a tree (i.e., a connected, undirected graph with no cycles) structure country
network
 * consisting of n cities numbered from 0 to n - 1 and exactly n - 1 roads. The capital city
is
 * city 0. You are given a 2D integer array roads where roads[i] = [ai, bi] denotes that there
 * exists a bidirectional road connecting cities ai and bi.
 *
 * There is a meeting for the representatives of each city. The meeting is in the capital
city.
 *
 * There is a car in each city. You are given an integer seats that indicates the number of
seats
 * in each car.
 *
 * A representative can use the car in their city to travel or change the car and ride with
another
 * representative. The cost of traveling between two cities is one liter of fuel.
 *
 * Return the minimum number of liters of fuel to reach the capital city.
 */

/**
 * @param {number[][]} roads
 * @param {number} seats
 * @return {number}
 */
var minimumFuelCost = function(roads, seats) {
  const n = roads.length + 1;
  const graph = Array.from({ length: n }, () => []);
  for (const [a, b] of roads) {
    graph[a].push(b);
    graph[b].push(a);
  }

  let fuel = 0;
  dfs(0, -1);
  return fuel;

  function dfs(node, parent) {
    let representatives = 1;
    for (const neighbor of graph[node]) {
      if (neighbor !== parent) {
        representatives += dfs(neighbor, node);
      }
    }
    if (node !== 0) {
      fuel += Math.ceil(representatives / seats);
    }
    return representatives;
  }
};
```

2479 Maximum Xor Of Two Non Overlapping Subtrees

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2479. Maximum XOR of Two Non-Overlapping Subtrees
 * https://leetcode.com/problems/maximum-xor-of-two-non-overlapping-subtrees/
 * Difficulty: Hard
 *
 * There is an undirected tree with n nodes labeled from 0 to n - 1. You are given the integer
 * n and a 2D integer array edges of length n - 1, where edges[i] = [ai, bi] indicates that
 * there is an edge between nodes ai and bi in the tree. The root of the tree is the node
labeled 0.
 *
 * Each node has an associated value. You are given an array values of length n, where
values[i]
 * is the value of the ith node.
 *
 * Select any two non-overlapping subtrees. Your score is the bitwise XOR of the sum of the
 * values within those subtrees.
 *
 * Return the maximum possible score you can achieve. If it is impossible to find two
 * nonoverlapping subtrees, return 0.
 *
 * Note that:
 * - The subtree of a node is the tree consisting of that node and all of its descendants.
 * - Two subtrees are non-overlapping if they do not share any common node.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[]} values
 * @return {number}
 */
var maxXor = function(n, edges, values) {
  class TrieNode {
    constructor() {
      this.children = [null, null];
    }
  }

  const graph = Array(n).fill().map(() => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  const subtreeSums = new Array(n);

  calculateSubtreeSum(0, -1);
  const root = new TrieNode();

  return Number(findMaxXor(0, -1, root));

  function calculateSubtreeSum(node, parent) {
    subtreeSums[node] = BigInt(values[node]);
    for (const child of graph[node]) {
      if (child !== parent) {
        subtreeSums[node] += calculateSubtreeSum(child, node);
      }
    }
    return subtreeSums[node];
  }

  function insert(root, value) {
    let current = root;
    for (let bit = 44; bit >= 0; bit--) {
      const bitValue = (value >> BigInt(bit)) & 1n;
      const index = Number(bitValue);
      if (!current.children[index]) {
        current.children[index] = new TrieNode();
      }
      current = current.children[index];
    }
  }

  function getMaxValue(root, currentValue) {
    if (!root.children[0] && !root.children[1]) {
```

```
      return 0n;
    }

    let result = 0n;
    let current = root;

    for (let bit = 44; bit >= 0; bit--) {
      const currentBit = (currentValue >> BigInt(bit)) & 1n;
      const wantedBit = 1n - currentBit;
      const wantedIndex = Number(wantedBit);
      const currentIndex = Number(currentBit);

      if (current.children[wantedIndex]) {
        result += 1n << BigInt(bit);
        current = current.children[wantedIndex];
      } else {
        current = current.children[currentIndex];
      }
    }

    return result;
  }

  function findMaxXor(node, parent, root) {
    let result = getMaxValue(root, subtreeSums[node]);

    for (const child of graph[node]) {
      if (child !== parent) {
        const childResult = findMaxXor(child, node, root);
        if (childResult > result) {
          result = childResult;
        }
      }
    }

    insert(root, subtreeSums[node]);
    return result;
  }
};
```

2481 Minimum Cuts To Divide A Circle

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2481. Minimum Cuts to Divide a Circle
 * https://leetcode.com/problems/minimum-cuts-to-divide-a-circle/
 * Difficulty: Easy
 *
 * A valid cut in a circle can be:
 * - A cut that is represented by a straight line that touches two points on the edge of the
 *   circle and passes through its center, or
 * - A cut that is represented by a straight line that touches one point on the edge of the
 *   circle and its center.
 *
 * Some valid and invalid cuts are shown in the figures below.
 *
 * Given the integer n, return the minimum number of cuts needed to divide a circle into n
 * equal slices.
 */

/**
 * @param {number} n
 * @return {number}
 */
var numberOfCuts = function(n) {
  return n === 1 ? 0 : n % 2 === 0 ? n / 2 : n;
};
```

2482 Difference Between Ones And Zeros In Row And Column

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2482. Difference Between Ones and Zeros in Row and Column
 * https://leetcode.com/problems/difference-between-ones-and-zeros-in-row-and-column/
 * Difficulty: Medium
 *
 * You are given a 0-indexed m x n binary matrix grid.
 *
 * A 0-indexed m x n difference matrix diff is created with the following procedure:
 * - Let the number of ones in the ith row be onesRowi.
 * - Let the number of ones in the jth column be onesColj.
 * - Let the number of zeros in the ith row be zerosRowi.
 * - Let the number of zeros in the jth column be zerosColj.
 * - diff[i][j] = onesRowi + onesColj - zerosRowi - zerosColj
 *
 * Return the difference matrix diff.
 */

/**
 * @param {number[][]} grid
 * @return {number[][]}
 */
var onesMinusZeros = function(grid) {
  const rows = new Array(grid.length).fill(0);
  const columns = new Array(grid[0].length).fill(0);

  grid.forEach((row, r) => row.forEach((value, c) => {
    rows[r] += value;
    columns[c] += value;
  }));

  grid.forEach((row, r) => row.forEach((value, c) => {
    grid[r][c] = 2 * rows[r] - grid.length + 2 * columns[c] - row.length;
  }));

  return grid;
};
```

2483 Minimum Penalty For A Shop

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2483. Minimum Penalty for a Shop
 * https://leetcode.com/problems/minimum-penalty-for-a-shop/
 * Difficulty: Medium
 *
 * You are given the customer visit log of a shop represented by a 0-indexed string customers
 * consisting only of characters 'N' and 'Y':
 * - if the ith character is 'Y', it means that customers come at the ith hour
 * - whereas 'N' indicates that no customers come at the ith hour.
 *
 * If the shop closes at the jth hour (0 <= j <= n), the penalty is calculated as follows:
 * - For every hour when the shop is open and no customers come, the penalty increases by 1.
 * - For every hour when the shop is closed and customers come, the penalty increases by 1.
 *
 * Return the earliest hour at which the shop must be closed to incur a minimum penalty.
 *
 * Note that if a shop closes at the jth hour, it means the shop is closed at the hour j.
 */

/**
```

```
 * @param {string} customers
 * @return {number}
 */
var bestClosingTime = function(customers) {
  let minPenalty = 0;
  let currentPenalty = 0;
  let result = 0;

  for (let i = 0; i < customers.length; i++) {
    currentPenalty += customers[i] === 'Y' ? -1 : 1;
    if (currentPenalty < minPenalty) {
      minPenalty = currentPenalty;
      result = i + 1;
    }
  }

  return result;
};
```

2484 Count Palindromic Subsequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2484. Count Palindromic Subsequences
 * https://leetcode.com/problems/count-palindromic-subsequences/
 * Difficulty: Hard
 *
 * Given a string of digits s, return the number of palindromic subsequences of s having
length 5.
 * Since the answer may be very large, return it modulo 109 + 7.
 *
 * Note:
 * - A string is palindromic if it reads the same forward and backward.
 * - A subsequence is a string that can be derived from another string by deleting some or no
 *   characters without changing the order of the remaining characters.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countPalindromes = function(s) {
  const MOD = 1e9 + 7;
  const n = s.length;

  const prefixPairs = new Array(n).fill(0).map(() => new Array(100).fill(0));
  const suffixPairs = new Array(n).fill(0).map(() => new Array(100).fill(0));

  for (let i = 1; i < n; i++) {
    for (let pair = 0; pair < 100; pair++) {
      prefixPairs[i][pair] = prefixPairs[i - 1][pair];
    }
    for (let j = 0; j < i; j++) {
      const pair = parseInt(s[j]) * 10 + parseInt(s[i]);
      prefixPairs[i][pair]++;
    }
  }

  for (let i = n - 2; i >= 0; i--) {
    for (let pair = 0; pair < 100; pair++) {
      suffixPairs[i][pair] = suffixPairs[i + 1][pair];
    }
    for (let j = i + 1; j < n; j++) {
      const pair = parseInt(s[i]) * 10 + parseInt(s[j]);
      suffixPairs[i][pair]++;
    }
  }

  let result = 0;
```

```
  for (let i = 2; i < n - 2; i++) {
    for (let first = 0; first < 10; first++) {
      for (let second = 0; second < 10; second++) {
        const leftPair = first * 10 + second;
        const rightPair = second * 10 + first;
        result = (result + (prefixPairs[i - 1][leftPair]
          * suffixPairs[i + 1][rightPair]) % MOD) % MOD;
      }
    }
  }

  return result;
};
```

2485 Find The Pivot Integer

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2485. Find the Pivot Integer
 * https://leetcode.com/problems/find-the-pivot-integer/
 * Difficulty: Easy
 *
 * Given a positive integer n, find the pivot integer x such that:
 * - The sum of all elements between 1 and x inclusively equals the sum of all elements
between
 *   x and n inclusively.
 *
 * Return the pivot integer x. If no such integer exists, return -1. It is guaranteed that
there
 * will be at most one pivot index for the given input.
 */

/**
 * @param {number} n
 * @return {number}
 */
var pivotInteger = function(n) {
  const totalSum = n * (n + 1) / 2;
  const pivot = Math.sqrt(totalSum);
  return Number.isInteger(pivot) && pivot <= n ? pivot : -1;
};
```

2486 Append Characters To String To Make Subsequence

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2486. Append Characters to String to Make Subsequence
 * https://leetcode.com/problems/append-characters-to-string-to-make-subsequence/
 * Difficulty: Medium
 *
 * You are given two strings s and t consisting of only lowercase English letters.
 *
 * Return the minimum number of characters that need to be appended to the end of s so that
 * t becomes a subsequence of s.
 *
 * A subsequence is a string that can be derived from another string by deleting some or no
 * characters without changing the order of the remaining characters.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {number}
```

```
 */
var appendCharacters = function(s, t) {
  let sIndex = 0;
  let tIndex = 0;

  while (sIndex < s.length && tIndex < t.length) {
    if (s[sIndex] === t[tIndex]) {
      tIndex++;
    }
    sIndex++;
  }

  return t.length - tIndex;
};
```

2487 Remove Nodes From Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2487. Remove Nodes From Linked List
 * https://leetcode.com/problems/remove-nodes-from-linked-list/
 * Difficulty: Medium
 *
 * You are given the head of a linked list.
 *
 * Remove every node which has a node with a greater value anywhere to the right side of it.
 *
 * Return the head of the modified linked list.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var removeNodes = function(head) {
  const stack = [];
  let current = head;

  while (current) {
    while (stack.length && stack[stack.length - 1].val < current.val) {
      stack.pop();
    }
    stack.push(current);
    current = current.next;
  }

  let result = null;
  while (stack.length) {
    current = stack.pop();
    current.next = result;
    result = current;
  }

  return result;
};
```

2488 Count Subarrays With Median K

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2488. Count Subarrays With Median K
 * https://leetcode.com/problems/count-subarrays-with-median-k/
 * Difficulty: Hard
 *
 * You are given an array nums of size n consisting of distinct integers from 1 to n and
 * a positive integer k.
 *
 * Return the number of non-empty subarrays in nums that have a median equal to k.
 *
 * Note:
 * - The median of an array is the middle element after sorting the array in ascending order.
 *   If the array is of even length, the median is the left middle element.
 *   - For example, the median of [2,3,1,4] is 2, and the median of [8,4,3,5,1] is 4.
 * - A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countSubarrays = function(nums, k) {
  const n = nums.length;
  const kIndex = nums.indexOf(k);
  const balanceMap = new Map([[0, 1]]);
  let balance = 0;
  let result = 0;

  for (let i = kIndex + 1; i < n; i++) {
    balance += nums[i] > k ? 1 : -1;
    balanceMap.set(balance, (balanceMap.get(balance) || 0) + 1);
  }

  balance = 0;
  for (let i = kIndex; i >= 0; i--) {
    balance += nums[i] > k ? -1 : nums[i] < k ? 1 : 0;
    result += balanceMap.get(balance) || 0;
    result += balanceMap.get(balance + 1) || 0;
  }

  return result;
};
```

2489 Number Of Substrings With Fixed Ratio

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2489. Number of Substrings With Fixed Ratio
 * https://leetcode.com/problems/number-of-substrings-with-fixed-ratio/
 * Difficulty: Medium
 *
 * You are given a binary string s, and two integers num1 and num2. num1 and num2 are
 * coprime numbers.
 *
 * A ratio substring is a substring of s where the ratio between the number of 0's and
 * the number of 1's in the substring is exactly num1 : num2.
 * - For example, if num1 = 2 and num2 = 3, then "01011" and "1110000111" are ratio
substrings,
 *   while "11000" is not.
```

```
 *
 * Return the number of non-empty ratio substrings of s.
 *
 * Note that:
 * - A substring is a contiguous sequence of characters within a string.
 * - Two values x and y are coprime if gcd(x, y) == 1 where gcd(x, y) is the greatest common
 *   divisor of x and y.
 */

/**
 * @param {string} s
 * @param {number} num1
 * @param {number} num2
 * @return {number}
 */
var fixedRatio = function(s, num1, num2) {
  const n = s.length;
  const map = new Map();
  map.set(0, 1);

  let zeros = 0;
  let ones = 0;
  let result = 0;
  for (let i = 0; i < n; i++) {
    if (s[i] === '0') {
      zeros++;
    } else {
      ones++;
    }

    const difference = ones * num1 - zeros * num2;

    if (map.has(difference)) {
      result += map.get(difference);
    }

    map.set(difference, (map.get(difference) || 0) + 1);
  }

  return result;
};
```

2490 Circular Sentence

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2490. Circular Sentence
 * https://leetcode.com/problems/circular-sentence/
 * Difficulty: Easy
 *
 * A sentence is a list of words that are separated by a single space with no leading or
trailing spaces.
 *
 * For example, "Hello World", "HELLO", "hello world hello world" are all sentences.
 * Words consist of only uppercase and lowercase English letters. Uppercase and lowercase
English letters
 * are considered different.
 *
 * A sentence is circular if:
 * - The last character of a word is equal to the first character of the next word.
 * - The last character of the last word is equal to the first character of the first word.
 *
 * For example, "leetcode exercises sound delightful", "eetcode", "leetcode eats soul" are all
 circular
 * sentences. However, "Leetcode is cool", "happy Leetcode", "Leetcode" and "I like Leetcode"
are not
 * circular sentences.
 *
 * Given a string sentence, return true if it is circular. Otherwise, return false.
```

```
 */

/**
 * @param {string} sentence
 * @return {boolean}
 */
var isCircularSentence = function(sentence) {
  const match = sentence.match(/^\w$|^(\w).*\1$/) !== null;
  const count = sentence.match(/(\w)(?=\s\1)/g)?.length || 0;

  return match && count === sentence.split(/\s/).length - 1;
};
```

2491 Divide Players Into Teams Of Equal Skill

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2491. Divide Players Into Teams of Equal Skill
 * https://leetcode.com/problems/divide-players-into-teams-of-equal-skill/
 * Difficulty: Medium
 *
 * You are given a positive integer array skill of even length n where skill[i] denotes the
 * skill of the ith player. Divide the players into n / 2 teams of size 2 such that the total
 * skill of each team is equal.
 *
 * The chemistry of a team is equal to the product of the skills of the players on that team.
 *
 * Return the sum of the chemistry of all the teams, or return -1 if there is no way to divide
 * the players into teams such that the total skill of each team is equal.
 */

/**
 * @param {number[]} skill
 * @return {number}
 */
var dividePlayers = function(skill) {
  skill.sort((a, b) => a - b);
  const n = skill.length;
  const targetSum = skill[0] + skill[n - 1];
  let result = 0;

  for (let i = 0, j = n - 1; i < j; i++, j--) {
    if (skill[i] + skill[j] !== targetSum) return -1;
    result += skill[i] * skill[j];
  }

  return result;
};
```

2492 Minimum Score Of A Path Between Two Cities

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2492. Minimum Score of a Path Between Two Cities
 * https://leetcode.com/problems/minimum-score-of-a-path-between-two-cities/
 * Difficulty: Medium
 *
 * You are given a positive integer n representing n cities numbered from 1 to n. You are also
 * given a 2D array roads where roads[i] = [ai, bi, distancei] indicates that there is a
 * bidirectional road between cities ai and bi with a distance equal to distancei. The cities
 * graph is not necessarily connected.
 *
```

```
 * The score of a path between two cities is defined as the minimum distance of a road in this
 * path.
 *
 * Return the minimum possible score of a path between cities 1 and n.
 *
 * Note:
 * - A path is a sequence of roads between two cities.
 * - It is allowed for a path to contain the same road multiple times, and you can visit
cities 1
 *   and n multiple times along the path.
 * - The test cases are generated such that there is at least one path between 1 and n.
 */

/**
 * @param {number} n
 * @param {number[][]} roads
 * @return {number}
 */
var minScore = function(n, roads) {
  const graph = Array.from({ length: n + 1 }, () => []);
  for (const [a, b, dist] of roads) {
    graph[a].push([b, dist]);
    graph[b].push([a, dist]);
  }

  const visited = new Set();
  const queue = [1];
  let result = Infinity;

  while (queue.length) {
    const city = queue.shift();
    if (visited.has(city)) continue;
    visited.add(city);

    for (const [nextCity, dist] of graph[city]) {
      result = Math.min(result, dist);
      if (!visited.has(nextCity)) {
        queue.push(nextCity);
      }
    }
  }

  return result;
};
```

2493 Divide Nodes Into The Maximum Number Of Groups

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2493. Divide Nodes Into the Maximum Number of Groups
 * https://leetcode.com/problems/divide-nodes-into-the-maximum-number-of-groups/
 * Difficulty: Hard
 *
 * You are given a positive integer n representing the number of nodes in an undirected graph.
 * The nodes are labeled from 1 to n.
 *
 * You are also given a 2D integer array edges, where edges[i] = [ai, bi] indicates that there
 * is a bidirectional edge between nodes ai and bi. Notice that the given graph may be
disconnected.
 *
 * Divide the nodes of the graph into m groups (1-indexed) such that:
 * - Each node in the graph belongs to exactly one group.
 * - For every pair of nodes in the graph that are connected by an edge [ai, bi], if ai
belongs to
 *   the group with index x, and bi belongs to the group with index y, then |y - x| = 1.
 *
 * Return the maximum number of groups (i.e., maximum m) into which you can divide the nodes.
 * Return -1 if it is impossible to group the nodes with the given conditions.
 */
```

```
/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var magnificentSets = function(n, edges) {
  const graph = new Array(n).fill().map(() => []);
  for (const [i, j] of edges) {
    graph[i - 1].push(j - 1);
    graph[j - 1].push(i - 1);
  }

  const result = new Array(n).fill(0);
  for (let i = 0; i < n; i++) {
    const groups = new Array(n).fill(0);
    const queue = [i];
    let max = 1;
    let root = i;

    groups[i] = 1;

    while (queue.length) {
      const key = queue.shift();
      root = Math.min(root, key);

      for (const node of graph[key]) {
        if (groups[node] === 0) {
          groups[node] = groups[key] + 1;
          max = Math.max(max, groups[node]);
          queue.push(node);
        } else if (Math.abs(groups[node] - groups[key]) !== 1) {
          return -1;
        }
      }
    }

    result[root] = Math.max(result[root], max);
  }

  return result.reduce((a, b) => a + b);
};
```

2495 Number Of Subarrays Having Even Product

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2495. Number of Subarrays Having Even Product
 * https://leetcode.com/problems/number-of-subarrays-having-even-product/
 * Difficulty: Medium
 *
 * Given a 0-indexed integer array nums, return the number of subarrays of nums having an
 * even product.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var evenProduct = function(nums) {
  const n = nums.length;
  const totalSubarrays = (n * (n + 1)) / 2;
  let oddProductSubarrays = 0;
  let currentOddLength = 0;

  for (let i = 0; i < n; i++) {
    if (nums[i] % 2 === 1) {
      currentOddLength++;
    } else {
```

```
      oddProductSubarrays += (currentOddLength * (currentOddLength + 1)) / 2;
      currentOddLength = 0;
    }
  }

  oddProductSubarrays += (currentOddLength * (currentOddLength + 1)) / 2;

  return totalSubarrays - oddProductSubarrays;
};
```

2496 Maximum Value Of A String In An Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2496. Maximum Value of a String in an Array
 * https://leetcode.com/problems/maximum-value-of-a-string-in-an-array/
 * Difficulty: Easy
 *
 * The value of an alphanumeric string can be defined as:
 * - The numeric representation of the string in base 10, if it comprises of digits only.
 * - The length of the string, otherwise.
 *
 * Given an array strs of alphanumeric strings, return the maximum value of any string in
strs.
 */

/**
 * @param {string[]} strs
 * @return {number}
 */
var maximumValue = function(strs) {
  let result = 0;

  for (const str of strs) {
    const isNumeric = /^[0-9]+$/.test(str);
    const value = isNumeric ? parseInt(str) : str.length;
    result = Math.max(result, value);
  }

  return result;
};
```

2498 Frog Jump Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2498. Frog Jump II
 * https://leetcode.com/problems/frog-jump-ii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array stones sorted in strictly increasing order
representing
 * the positions of stones in a river.
 *
 * A frog, initially on the first stone, wants to travel to the last stone and then return to
the
 * first stone. However, it can jump to any stone at most once.
 *
 * The length of a jump is the absolute difference between the position of the stone the frog
is
 * currently on and the position of the stone to which the frog jumps.
 *
 * - More formally, if the frog is at stones[i] and is jumping to stones[j], the length of the
```

```
 jump
 *   is |stones[i] - stones[j]|.
 *
 * The cost of a path is the maximum length of a jump among all jumps in the path.
 *
 * Return the minimum cost of a path for the frog.
 */

/**
 * @param {number[]} stones
 * @return {number}
 */
var maxJump = function(stones) {
  let result = stones[1];
  for (let i = 2; i < stones.length; i++) {
    result = Math.max(result, stones[i] - stones[i - 2]);
  }
  return result;
};
```

2499 Minimum Total Cost To Make Arrays Unequal

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2499. Minimum Total Cost to Make Arrays Unequal
 * https://leetcode.com/problems/minimum-total-cost-to-make-arrays-unequal/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays nums1 and nums2, of equal length n.
 *
 * In one operation, you can swap the values of any two indices of nums1. The cost of this
 * operation is the sum of the indices.
 *
 * Find the minimum total cost of performing the given operation any number of times such
 * that nums1[i] != nums2[i] for all 0 <= i <= n - 1 after performing all the operations.
 *
 * Return the minimum total cost such that nums1 and nums2 satisfy the above condition.
 * In case it is not possible, return -1.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minimumTotalCost = function(nums1, nums2) {
  const n = nums1.length;
  const conflictIndices = [];
  const valueCount = new Map();

  for (let i = 0; i < n; i++) {
    if (nums1[i] === nums2[i]) {
      conflictIndices.push(i);
      valueCount.set(nums1[i], (valueCount.get(nums1[i]) || 0) + 1);
    }
  }

  if (conflictIndices.length === 0) return 0;

  let dominantValue = -1;
  let maxCount = 0;

  for (const [value, count] of valueCount) {
    if (count > maxCount) {
      maxCount = count;
      dominantValue = value;
    }
  }
```

```
  const swapsNeeded = conflictIndices.length;
  let helpersNeeded = Math.max(0, 2 * maxCount - swapsNeeded);

  const helperIndices = [];
  for (let i = 0; i < n && helpersNeeded > 0; i++) {
    if (nums1[i] !== nums2[i] && nums1[i] !== dominantValue && nums2[i] !== dominantValue) {
      helperIndices.push(i);
      helpersNeeded--;
    }
  }

  if (helpersNeeded > 0) return -1;

  const allIndices = [...conflictIndices, ...helperIndices];
  allIndices.sort((a, b) => a - b);

  let result = 0;
  for (let i = 0; i < allIndices.length; i++) {
    result += allIndices[i];
  }

  return result;
};
```

2500 Delete Greatest Value In Each Row

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2500. Delete Greatest Value in Each Row
 * https://leetcode.com/problems/delete-greatest-value-in-each-row/
 * Difficulty: Easy
 *
 * You are given an m x n matrix grid consisting of positive integers.
 *
 * Perform the following operation until grid becomes empty:
 * - Delete the element with the greatest value from each row. If multiple such elements
exist,
 *   delete any of them.
 * - Add the maximum of deleted elements to the answer.
 *
 * Note that the number of columns decreases by one after each operation.
 *
 * Return the answer after performing the operations described above.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var deleteGreatestValue = function(grid) {
  for (const row of grid) {
    row.sort((a, b) => a - b);
  }

  let result = 0;
  const cols = grid[0].length;
  for (let col = cols - 1; col >= 0; col--) {
    let maxInColumn = 0;
    for (let row = 0; row < grid.length; row++) {
      maxInColumn = Math.max(maxInColumn, grid[row][col]);
    }
    result += maxInColumn;
  }

  return result;
};
```

2501 Longest Square Streak In An Array

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2501. Longest Square Streak in an Array
 * https://leetcode.com/problems/longest-square-streak-in-an-array/
 * Difficulty: Medium
 *
 * You are given an integer array nums. A subsequence of nums is called a square streak if:
 * - The length of the subsequence is at least 2, and
 * - after sorting the subsequence, each element (except the first element) is the square of
 *   the previous number.
 *
 * Return the length of the longest square streak in nums, or return -1 if there is no square
 * streak.
 *
 * A subsequence is an array that can be derived from another array by deleting some or no
 * elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var longestSquareStreak = function(nums) {
  const set = new Set(nums);
  let result = -1;

  for (const num of nums) {
    let current = num;
    let length = 1;

    while (current <= 100000 && set.has(current * current)) {
      current *= current;
      length++;
    }

    if (length >= 2) {
      result = Math.max(result, length);
    }
  }

  return result;
};
```

2502 Design Memory Allocator

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2502. Design Memory Allocator
 * https://leetcode.com/problems/design-memory-allocator/
 * Difficulty: Medium
 *
 * You are given an integer n representing the size of a 0-indexed memory array. All memory
 * units are initially free.
 *
 * You have a memory allocator with the following functionalities:
 * 1. Allocate a block of size consecutive free memory units and assign it the id mID.
 * 2. Free all memory units with the given id mID.
 *
 * Note that:
```

```
 * - Multiple blocks can be allocated to the same mID.
 * - You should free all the memory units with mID, even if they were allocated in different
blocks.
 *
 * Implement the Allocator class:
 * - Allocator(int n) Initializes an Allocator object with a memory array of size n.
 * - int allocate(int size, int mID) Find the leftmost block of size consecutive free memory
units
 *   and allocate it with the id mID. Return the block's first index. If such a block does not
 *   exist, return -1.
 * - int freeMemory(int mID) Free all memory units with the id mID. Return the number of
memory
 *   units you have freed.
 */

/**
 * @param {number} n
 */
var Allocator = function(n) {
  this.memory = Array(n).fill(0);
};

/**
 * @param {number} size
 * @param {number} mID
 * @return {number}
 */
Allocator.prototype.allocate = function(size, mID) {
  let start = -1;
  let count = 0;

  for (let i = 0; i < this.memory.length; i++) {
    if (this.memory[i] === 0) {
      if (start === -1) start = i;
      count++;
      if (count === size) {
        for (let j = start; j < start + size; j++) {
          this.memory[j] = mID;
        }
        return start;
      }
    } else {
      start = -1;
      count = 0;
    }
  }

  return -1;
};

/**
 * @param {number} mID
 * @return {number}
 */
Allocator.prototype.freeMemory = function(mID) {
  let result = 0;

  for (let i = 0; i < this.memory.length; i++) {
    if (this.memory[i] === mID) {
      this.memory[i] = 0;
      result++;
    }
  }

  return result;
};
```

2503 Maximum Number Of Points From Grid Queries

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the

loop invariant.

SOURCE CODE

```
/**
 * 2503. Maximum Number of Points From Grid Queries
 * https://leetcode.com/problems/maximum-number-of-points-from-grid-queries/
 * Difficulty: Hard
 *
 * You are given an m x n integer matrix grid and an array queries of size k.
 *
 * Find an array answer of size k such that for each integer queries[i] you start in the top
left
 * cell of the matrix and repeat the following process:
 * - If queries[i] is strictly greater than the value of the current cell that you are in,
then
 *   you get one point if it is your first time visiting this cell, and you can move to any
adjacent
 *   cell in all 4 directions: up, down, left, and right.
 * - Otherwise, you do not get any points, and you end this process.
 *
 * After the process, answer[i] is the maximum number of points you can get. Note that for
each
 * query you are allowed to visit the same cell multiple times.
 *
 * Return the resulting array answer.
 */

/**
 * @param {number[][]} grid
 * @param {number[]} queries
 * @return {number[]}
 */
var maxPoints = function(grid, queries) {
  const rows = grid.length;
  const cols = grid[0].length;
  const result = new Array(queries.length);
  const sortedQueries = queries
    .map((value, index) => ({ value, index }))
    .sort((a, b) => a.value - b.value);
  const directions = [[1, 0], [0, 1], [-1, 0], [0, -1]];
  const queue = new MinPriorityQueue(([row, col]) => grid[row][col]);
  const visited = new Set();

  queue.enqueue([0, 0]);
  visited.add('0,0');

  let queryIndex = 0;
  let points = 0;
  while (queue.size()) {
    const [row, col] = queue.dequeue();
    const currentValue = grid[row][col];
    while (queryIndex < sortedQueries.length && currentValue >=
sortedQueries[queryIndex].value) {
      result[sortedQueries[queryIndex].index] = points;
      queryIndex++;
    }
    if (queryIndex === sortedQueries.length) break;
    points++;
    for (const [rowOffset, colOffset] of directions) {
      const nextRow = row + rowOffset;
      const nextCol = col + colOffset;
      const positionKey = `${nextRow},${nextCol}`;
      if (nextRow >= 0 && nextRow < rows && nextCol >= 0 && nextCol < cols
          && !visited.has(positionKey)) {
        visited.add(positionKey);
        queue.enqueue([nextRow, nextCol]);
      }
    }
  }
  while (queryIndex < sortedQueries.length) {
    result[sortedQueries[queryIndex].index] = points;
    queryIndex++;
  }

  return result;
};
```

2505 Bitwise Or Of All Subsequence Sums

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2505. Bitwise OR of All Subsequence Sums
 * https://leetcode.com/problems/bitwise-or-of-all-subsequence-sums/
 * Difficulty: Medium
 *
 * Given an integer array nums, return the value of the bitwise OR of the sum of all possible
 * subsequences in the array.
 *
 * A subsequence is a sequence that can be derived from another sequence by removing zero or
 * more elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var subsequenceSumOr = function(nums) {
  const bitCounts = new Array(64).fill(0n);

  for (const num of nums) {
    for (let bit = 0; bit < 31; bit++) {
      if (num & (1 << bit)) {
        bitCounts[bit]++;
      }
    }
  }

  for (let bit = 0; bit < 63; bit++) {
    bitCounts[bit + 1] += bitCounts[bit] / 2n;
  }

  let result = 0n;
  for (let bit = 63; bit >= 0; bit--) {
    result = (result << 1n) | (bitCounts[bit] > 0n ? 1n : 0n);
  }

  return Number(result);
};
```

2506 Count Pairs Of Similar Strings

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2506. Count Pairs Of Similar Strings
 * https://leetcode.com/problems/count-pairs-of-similar-strings/
 * Difficulty: Easy
 *
 * You are given a 0-indexed string array words.
 *
 * Two strings are similar if they consist of the same characters.
 * - For example, "abca" and "cba" are similar since both consist of characters 'a', 'b', and
'c'.
 * - However, "abacba" and "bcfd" are not similar since they do not consist of the same
characters.
 *
 * Return the number of pairs (i, j) such that 0 <= i < j <= word.length - 1 and the two
strings
 * words[i] and words[j] are similar.
 */
```

```
/**
 * @param {string[]} words
 * @return {number}
 */
var similarPairs = function(words) {
  const map = new Map();
  let result = 0;

  for (const word of words) {
    const charSet = [...new Set(word.split(''))].sort().join('');
    map.set(charSet, (map.get(charSet) || 0) + 1);
  }

  for (const count of map.values()) {
    if (count > 1) {
      result += (count * (count - 1)) / 2;
    }
  }

  return result;
};
```

2507 Smallest Value After Replacing With Sum Of Prime Factors

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2507. Smallest Value After Replacing With Sum of Prime Factors
 * https://leetcode.com/problems/smallest-value-after-replacing-with-sum-of-prime-factors/
 * Difficulty: Medium
 *
 * You are given a positive integer n.
 *
 * Continuously replace n with the sum of its prime factors.
 * - Note that if a prime factor divides n multiple times, it should be included in the sum as
 many
 *   times as it divides n.
 *
 * Return the smallest value n will take on.
 */

/**
 * @param {number} n
 * @return {number}
 */
var smallestValue = function(n) {
  while (true) {
    const next = sumPrimeFactors(n);
    if (next === n) return n;
    n = next;
  }

  function sumPrimeFactors(num) {
    let sum = 0;
    for (let i = 2; i * i <= num; i++) {
      while (num % i === 0) {
        sum += i;
        num /= i;
      }
    }
    if (num > 1) sum += num;
    return sum;
  }
};
```

2509 Cycle Length Queries In A Tree

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2509. Cycle Length Queries in a Tree
 * https://leetcode.com/problems/cycle-length-queries-in-a-tree/
 * Difficulty: Hard
 *
 * You are given an integer n. There is a complete binary tree with 2n - 1 nodes. The root of
 * that tree is the node with the value 1, and every node with a value val in the range
 * [1, 2n - 1 - 1] has two children where:
 * - The left node has the value 2 * val, and
 * - The right node has the value 2 * val + 1.
 *
 * You are also given a 2D integer array queries of length m, where queries[i] = [ai, bi]. For
 * each query, solve the following problem:
 * 1. Add an edge between the nodes with values ai and bi.
 * 2. Find the length of the cycle in the graph.
 * 3. Remove the added edge between nodes with values ai and bi.
 *
 * Note that:
 * - A cycle is a path that starts and ends at the same node, and each edge in the path is
 *   visited only once.
 * - The length of a cycle is the number of edges visited in the cycle.
 * - There could be multiple edges between two nodes in the tree after adding the edge of
 *   the query.
 *
 * Return an array answer of length m where answer[i] is the answer to the ith query.
 */

/**
 * @param {number} n
 * @param {number[][]} queries
 * @return {number[]}
 */
var cycleLengthQueries = function(n, queries) {
  const result = [];

  for (const [a, b] of queries) {
    let x = a;
    let y = b;
    let pathLength = 1;

    while (x !== y) {
      if (x > y) {
        x = Math.floor(x / 2);
      } else {
        y = Math.floor(y / 2);
      }
      pathLength++;
    }

    result.push(pathLength);
  }

  return result;
};
```

2510 Check If There Is A Path With Equal Number Of 0S And 1S

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2510. Check if There is a Path With Equal Number of 0's And 1's
```

```
 * https://leetcode.com/problems/check-if-there-is-a-path-with-equal-number-of-0s-and-1s/
 * Difficulty: Medium
 *
 * You are given a 0-indexed m x n binary matrix grid. You can move from a cell (row, col) to
 * any of the cells (row + 1, col) or (row, col + 1).
 *
 * Return true if there is a path from (0, 0) to (m - 1, n - 1) that visits an equal number
 * of 0's and 1's. Otherwise return false.
 */

/**
 * @param {number[][]} grid
 * @return {boolean}
 */
var isThereAPath = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const pathLength = rows + cols - 1;

  if (pathLength % 2 !== 0) return false;

  const visited = new Set();
  const startBalance = grid[0][0] === 1 ? 1 : -1;

  return dfs(0, 0, startBalance);

  function dfs(row, col, balance) {
    if (row === rows - 1 && col === cols - 1) {
      return balance === 0;
    }

    const key = `${row},${col},${balance}`;
    if (visited.has(key)) return false;
    visited.add(key);

    if (Math.abs(balance) > (rows - 1 - row) + (cols - 1 - col)) {
      return false;
    }

    let canReach = false;

    if (row + 1 < rows) {
      const nextBalance = balance + (grid[row + 1][col] === 1 ? 1 : -1);
      canReach = canReach || dfs(row + 1, col, nextBalance);
    }

    if (col + 1 < cols) {
      const nextBalance = balance + (grid[row][col + 1] === 1 ? 1 : -1);
      canReach = canReach || dfs(row, col + 1, nextBalance);
    }

    return canReach;
  }
};
```

2511 Maximum Enemy Forts That Can Be Captured

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2511. Maximum Enemy Forts That Can Be Captured
 * https://leetcode.com/problems/maximum-enemy-forts-that-can-be-captured/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array forts of length n representing the positions of
 * several forts. forts[i] can be -1, 0, or 1 where:
 * - -1 represents there is no fort at the ith position.
 * - 0 indicates there is an enemy fort at the ith position.
 * - 1 indicates the fort at the ith the position is under your command.
 *
```

```
 * Now you have decided to move your army from one of your forts at position i to an empty
 * position j such that:
 * - 0 <= i, j <= n - 1
 * - The army travels over enemy forts only. Formally, for all k where min(i,j) < k <
max(i,j),
 *   forts[k] == 0.
 *
 * While moving the army, all the enemy forts that come in the way are captured.
 *
 * Return the maximum number of enemy forts that can be captured. In case it is impossible to
 * move your army, or you do not have any fort under your command, return 0.
 */

/**
 * @param {number[]} forts
 * @return {number}
 */
var captureForts = function(forts) {
  let result = 0;
  let start = -1;

  for (let i = 0; i < forts.length; i++) {
    if (forts[i] === 1 || forts[i] === -1) {
      if (start !== -1 && forts[start] !== forts[i]) {
        result = Math.max(result, i - start - 1);
      }
      start = i;
    }
  }

  return result;
};
```

2512 Reward Top K Students

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2512. Reward Top K Students
 * https://leetcode.com/problems/reward-top-k-students/
 * Difficulty: Medium
 *
 * You are given two string arrays positive_feedback and negative_feedback, containing the
words
 * denoting positive and negative feedback, respectively. Note that no word is both positive
and
 * negative.
 *
 * Initially every student has 0 points. Each positive word in a feedback report increases the
 * points of a student by 3, whereas each negative word decreases the points by 1.
 *
 * You are given n feedback reports, represented by a 0-indexed string array report and a
 * 0-indexed integer array student_id, where student_id[i] represents the ID of the student
 * who has received the feedback report report[i]. The ID of each student is unique.
 *
 * Given an integer k, return the top k students after ranking them in non-increasing order
 * by their points. In case more than one student has the same points, the one with the lower
 * ID ranks higher.
 */

/**
 * @param {string[]} positive_feedback
 * @param {string[]} negative_feedback
 * @param {string[]} report
 * @param {number[]} student_id
 * @param {number} k
 * @return {number[]}
 */
var topStudents = function(positive_feedback, negative_feedback, report, student_id, k) {
```

```
  const positiveSet = new Set(positive_feedback);
  const negativeSet = new Set(negative_feedback);
  const studentScores = [];

  for (let i = 0; i < report.length; i++) {
    const words = report[i].split(' ');
    let score = 0;

    for (const word of words) {
      if (positiveSet.has(word)) {
        score += 3;
      } else if (negativeSet.has(word)) {
        score -= 1;
      }
    }

    studentScores.push([student_id[i], score]);
  }

  studentScores.sort((a, b) => {
    if (a[1] !== b[1]) return b[1] - a[1];
    return a[0] - b[0];
  });

  return studentScores.slice(0, k).map(student => student[0]);
};
```

2515 Shortest Distance To Target String In A Circular Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2515. Shortest Distance to Target String in a Circular Array
 * https://leetcode.com/problems/shortest-distance-to-target-string-in-a-circular-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed circular string array words and a string target. A circular array
 * means that the array's end connects to the array's beginning.
 * - Formally, the next element of words[i] is words[(i + 1) % n] and the previous element of
 *   words[i] is words[(i - 1 + n) % n], where n is the length of words.
 *
 * Starting from startIndex, you can move to either the next word or the previous word with 1
 * step at a time.
 *
 * Return the shortest distance needed to reach the string target. If the string target does
not
 * exist in words, return -1.
 */

/**
 * @param {string[]} words
 * @param {string} target
 * @param {number} startIndex
 * @return {number}
 */
var closestTarget = function(words, target, startIndex) {
  const n = words.length;
  let minDistance = Infinity;

  for (let i = 0; i < n; i++) {
    if (words[i] === target) {
      const forward = Math.abs(i - startIndex);
      const backward = n - forward;
      minDistance = Math.min(minDistance, forward, backward);
    }
  }

  return minDistance === Infinity ? -1 : minDistance;
};
```

2516 Take K Of Each Character From Left And Right

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2516. Take K of Each Character From Left and Right
 * https://leetcode.com/problems/take-k-of-each-character-from-left-and-right/
 * Difficulty: Medium
 *
 * You are given a string s consisting of the characters 'a', 'b', and 'c' and a non-negative
 * integer k. Each minute, you may take either the leftmost character of s, or the rightmost
 * character of s.
 *
 * Return the minimum number of minutes needed for you to take at least k of each character,
 * or return -1 if it is not possible to take k of each character.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var takeCharacters = function(s, k) {
  const n = s.length;
  const counts = { a: 0, b: 0, c: 0 };

  for (const char of s) {
    counts[char]++;
  }

  if (counts.a < k || counts.b < k || counts.c < k) return -1;

  let maxWindow = 0;
  let left = 0;
  const windowCounts = { a: 0, b: 0, c: 0 };

  for (let right = 0; right < n; right++) {
    windowCounts[s[right]]++;

    while (windowCounts.a > counts.a - k || windowCounts.b > counts.b - k
          || windowCounts.c > counts.c - k) {
      windowCounts[s[left]]--;
      left++;
    }

    maxWindow = Math.max(maxWindow, right - left + 1);
  }

  return n - maxWindow;
};
```

2517 Maximum Tastiness Of Candy Basket

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2517. Maximum Tastiness of Candy Basket
 * https://leetcode.com/problems/maximum-tastiness-of-candy-basket/
 * Difficulty: Medium
 *
 * You are given an array of positive integers price where price[i] denotes the price of the
ith
 * candy and a positive integer k.
```

```
 *
 * The store sells baskets of k distinct candies. The tastiness of a candy basket is the
smallest
 * absolute difference of the prices of any two candies in the basket.
 *
 * Return the maximum tastiness of a candy basket.
 */

/**
 * @param {number[]} price
 * @param {number} k
 * @return {number}
 */
var maximumTastiness = function(price, k) {
  price.sort((a, b) => a - b);
  let left = 0;
  let right = price[price.length - 1] - price[0];
  let result = 0;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    let count = 1;
    let prev = price[0];

    for (let i = 1; i < price.length; i++) {
      if (price[i] - prev >= mid) {
        count++;
        prev = price[i];
      }
    }

    if (count >= k) {
      result = mid;
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;
};
```

2519 Count The Number Of K Big Indices

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2519. Count the Number of K-Big Indices
 * https://leetcode.com/problems/count-the-number-of-k-big-indices/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums and a positive integer k.
 *
 * We call an index i k-big if the following conditions are satisfied:
 * - There exist at least k different indices idx1 such that idx1 < i and nums[idx1] <
nums[i].
 * - There exist at least k different indices idx2 such that idx2 > i and nums[idx2] <
nums[i].
 *
 * Return the number of k-big indices.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var kBigIndices = function(nums, k) {
  const n = nums.length;
  const hasKSmallerLeft = new Array(n).fill(false);
```

```
  const leftMaxHeap = new PriorityQueue((a, b) => b - a);
  for (let i = 0; i < n; i++) {
    if (leftMaxHeap.size() === k && leftMaxHeap.front() < nums[i]) {
      hasKSmallerLeft[i] = true;
    }
    leftMaxHeap.enqueue(nums[i]);
    if (leftMaxHeap.size() > k) {
      leftMaxHeap.dequeue();
    }
  }

  let result = 0;
  const rightMaxHeap = new PriorityQueue((a, b) => b - a);
  for (let i = n - 1; i >= 0; i--) {
    if (rightMaxHeap.size() === k && rightMaxHeap.front() < nums[i] && hasKSmallerLeft[i]) {
      result++;
    }
    rightMaxHeap.enqueue(nums[i]);
    if (rightMaxHeap.size() > k) {
      rightMaxHeap.dequeue();
    }
  }

  return result;
};
```

2520 Count The Digits That Divide A Number

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2520. Count the Digits That Divide a Number
 * https://leetcode.com/problems/count-the-digits-that-divide-a-number/
 * Difficulty: Easy
 *
 * Given an integer num, return the number of digits in num that divide num.
 *
 * An integer val divides nums if nums % val == 0.
 */

/**
 * @param {number} num
 * @return {number}
 */
var countDigits = function(num) {
  let result = 0;
  let n = num;

  while (n > 0) {
    if (num % (n % 10) === 0) result++;
    n = Math.floor(n / 10);
  }

  return result;
};
```

2521 Distinct Prime Factors Of Product Of Array

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2521. Distinct Prime Factors of Product of Array
 * https://leetcode.com/problems/distinct-prime-factors-of-product-of-array/
```

```
 * Difficulty: Medium
 *
 * Given an array of positive integers nums, return the number of distinct prime factors in
the
 * product of the elements of nums.
 *
 * Note that:
 * - A number greater than 1 is called prime if it is divisible by only 1 and itself.
 * - An integer val1 is a factor of another integer val2 if val2 / val1 is an integer.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var distinctPrimeFactors = function(nums) {
  const set = new Set();

  for (const num of nums) {
    factorize(num);
  }

  return set.size;

  function factorize(num) {
    for (let i = 2; i * i <= num; i++) {
      while (num % i === 0) {
        set.add(i);
        num /= i;
      }
    }
    if (num > 1) set.add(num);
  }
};
```

2522 Partition String Into Substrings With Values At Most K

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2522. Partition String Into Substrings With Values at Most K
 * https://leetcode.com/problems/partition-string-into-substrings-with-values-at-most-k/
 * Difficulty: Medium
 *
 * You are given a string s consisting of digits from 1 to 9 and an integer k.
 *
 * A partition of a string s is called good if:
 * - Each digit of s is part of exactly one substring.
 * - The value of each substring is less than or equal to k.
 *
 * Return the minimum number of substrings in a good partition of s. If no good partition of
 * s exists, return -1.
 *
 * Note that:
 * - The value of a string is its result when interpreted as an integer. For example, the
value
 *   of "123" is 123 and the value of "1" is 1.
 * - A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var minimumPartition = function(s, k) {
  let result = 1;
  let current = 0;

  for (const digit of s) {
```

```
    const value = current * 10 + Number(digit);
    if (value <= k) {
      current = value;
    } else {
      if (Number(digit) > k) return -1;
      current = Number(digit);
      result++;
    }
  }

  return result;
};
```

2523 Closest Prime Numbers In Range

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2523. Closest Prime Numbers in Range
 * https://leetcode.com/problems/closest-prime-numbers-in-range/
 * Difficulty: Medium
 *
 * Given two positive integers left and right, find the two integers num1 and num2 such that:
 * - left <= num1 < num2 <= right
 * - Both num1 and num2 are prime numbers.
 * - num2 - num1 is the minimum amongst all other pairs satisfying the above conditions.
 *
 * Return the positive integer array ans = [num1, num2]. If there are multiple pairs
satisfying
 * these conditions, return the one with the smallest num1 value. If no such numbers exist,
 * return [-1, -1].
 */

/**
 * @param {number} left
 * @param {number} right
 * @return {number[]}
 */
var closestPrimes = function(left, right) {
  const group = new Uint8Array(right + 1).fill(1);
  let result = [-1, -1];

  group[0] = group[1] = 0;
  for (let i = 2; i * i <= right; i++) {
    if (group[i]) {
      for (let j = i * i; j <= right; j += i) {
        group[j] = 0;
      }
    }
  }
  for (let i = Math.max(2, left), prev = -1, min = Infinity; i <= right; i++) {
    if (group[i]) {
      if (prev !== -1 && i - prev < min) {
        min = i - prev;
        result = [prev, i];
      }
      prev = i;
    }
  }

  return result;
};
```

2524 Maximum Frequency Score Of A Subarray

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2524. Maximum Frequency Score of a Subarray
 * https://leetcode.com/problems/maximum-frequency-score-of-a-subarray/
 * Difficulty: Hard
 *
 * You are given an integer array nums and a positive integer k.
 *
 * The frequency score of an array is the sum of the distinct values in the array raised to
 * the power of their frequencies, taking the sum modulo 109 + 7.
 * - For example, the frequency score of the array [5,4,5,7,4,4] is (43 + 52 + 71) modulo
 *   (109 + 7) = 96.
 *
 * Return the maximum frequency score of a subarray of size k in nums. You should maximize
 * the value under the modulo and not the actual value.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxFrequencyScore = function(nums, k) {
  const MOD = 1e9 + 7;

  let sum = 0;
  let result = 0;
  const frequency = new Map();
  for (let i = 0; i < nums.length; i++) {
    if (!frequency.has(nums[i])) {
      frequency.set(nums[i], []);
    }
    sum = adjustSum(sum, nums[i], frequency.get(nums[i]));

    if (i >= k - 1) {
      if (i >= k) {
        sum = adjustSum(sum, nums[i - k], frequency.get(nums[i - k]), false);
      }
      result = Math.max(result, sum);
    }
  }

  return result;

  function adjustSum(sum, value, powers, add = true) {
    sum = (MOD + sum - (powers.length === 0 ? 0 : powers[powers.length - 1])) % MOD;
    if (add) {
      const newPower = (value * (powers.length === 0 ? 1 : powers[powers.length - 1])) % MOD;
      powers.push(newPower);
    } else {
      powers.pop();
    }
    return (sum + (powers.length === 0 ? 0 : powers[powers.length - 1])) % MOD;
  }
};
```

2527 Find Xor Beauty Of Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2527. Find Xor-Beauty of Array
 * https://leetcode.com/problems/find-xor-beauty-of-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums.
```

```
 *
 * The effective value of three indices i, j, and k is defined as ((nums[i] | nums[j]) &
nums[k]).
 *
 * The xor-beauty of the array is the XORing of the effective values of all the possible
triplets
 * of indices (i, j, k) where 0 <= i, j, k < n.
 *
 * Return the xor-beauty of nums.
 *
 * Note that:
 * - val1 | val2 is bitwise OR of val1 and val2.
 * - val1 & val2 is bitwise AND of val1 and val2.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var xorBeauty = function(nums) {
  let result = 0;
  for (const num of nums) {
    result ^= num;
  }
  return result;
};
```

2529 Maximum Count Of Positive Integer And Negative Integer

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2529. Maximum Count of Positive Integer and Negative Integer
 * https://leetcode.com/problems/maximum-count-of-positive-integer-and-negative-integer/
 * Difficulty: Easy
 *
 * Given an array nums sorted in non-decreasing order, return the maximum
 * between the number of positive integers and the number of negative integers.
 *
 * In other words, if the number of positive integers in nums is pos and the
 * number of negative integers is neg, then return the maximum of pos and neg.
 *
 * Note that 0 is neither positive nor negative.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumCount = function(nums) {
  const result = nums.reduce(([neg = 0, pos = 0], n) => {
    return [neg + (n < 0 ? 1 : 0), pos + (n > 0 ? 1 : 0)];
  }, []);
  return Math.max(...result);
};
```

2533 Number Of Good Binary Strings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2533. Number of Good Binary Strings
 * https://leetcode.com/problems/number-of-good-binary-strings/
 * Difficulty: Medium
 *
```

```
 * You are given four integers minLength, maxLength, oneGroup and zeroGroup.
 *
 * A binary string is good if it satisfies the following conditions:
 * - The length of the string is in the range [minLength, maxLength].
 * - The size of each block of consecutive 1's is a multiple of oneGroup.
 *   - For example in a binary string 00110111100 sizes of each block of consecutive ones
 *     are [2,4].
 * - The size of each block of consecutive 0's is a multiple of zeroGroup.
 *   - For example, in a binary string 00110111100 sizes of each block of consecutive zeros
 *     are [2,1,2].
 *
 * Return the number of good binary strings. Since the answer may be too large, return it
 * modulo 109 + 7.
 *
 * Note that 0 is considered a multiple of all the numbers.
 */

/**
 * @param {number} minLength
 * @param {number} maxLength
 * @param {number} oneGroup
 * @param {number} zeroGroup
 * @return {number}
 */
var goodBinaryStrings = function(minLength, maxLength, oneGroup, zeroGroup) {
  const MOD = 1e9 + 7;
  const dp = new Array(maxLength + 1).fill(0);
  dp[0] = 1;

  for (let i = 1; i <= maxLength; i++) {
    if (oneGroup <= i) {
      dp[i] = (dp[i] + dp[i - oneGroup]) % MOD;
    }
    if (zeroGroup <= i) {
      dp[i] = (dp[i] + dp[i - zeroGroup]) % MOD;
    }
  }

  let result = 0;
  for (let i = minLength; i <= maxLength; i++) {
    result = (result + dp[i]) % MOD;
  }

  return result;
};
```

2534 Time Taken To Cross The Door

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2534. Time Taken to Cross the Door
 * https://leetcode.com/problems/time-taken-to-cross-the-door/
 * Difficulty: Hard
 *
 * There are n persons numbered from 0 to n - 1 and a door. Each person can enter or exit
 * through the door once, taking one second.
 *
 * You are given a non-decreasing integer array arrival of size n, where arrival[i] is the
 * arrival time of the ith person at the door. You are also given an array state of size n,
 * where state[i] is 0 if person i wants to enter through the door or 1 if they want to exit
 * through the door.
 *
 * If two or more persons want to use the door at the same time, they follow the following
 * rules:
 * - If the door was not used in the previous second, then the person who wants to exit
 *   goes first.
 * - If the door was used in the previous second for entering, the person who wants to
 *   enter goes first.
 * - If the door was used in the previous second for exiting, the person who wants to
```

```
 *   exit goes first.
 * - If multiple persons want to go in the same direction, the person with the smallest
 *   index goes first.
 *
 * Return an array answer of size n where answer[i] is the second at which the ith person
 * crosses the door.
 *
 * Note that:
 * - Only one person can cross the door at each second.
 * - A person may arrive at the door and wait without entering or exiting to follow the
 *   mentioned rules.
 */

/**
 * @param {number[]} arrival
 * @param {number[]} state
 * @return {number[]}
 */
var timeTaken = function(arrival, state) {
  const enterPool = [];
  const exitPool = [];
  let currentTime = 0;
  let previousState = 1;
  let i = 0;
  const result = new Array(arrival.length);

  while (i < arrival.length || enterPool.length > 0 || exitPool.length > 0) {
    while (i < arrival.length && arrival[i] <= currentTime) {
      if (state[i] === 0) {
        enterPool.push(i);
      } else {
        exitPool.push(i);
      }
      i++;
    }

    if (previousState === 1) {
      if (exitPool.length > 0) {
        result[exitPool.shift()] = currentTime;
      } else if (enterPool.length > 0) {
        result[enterPool.shift()] = currentTime;
        previousState = 0;
      }
    } else {
      if (enterPool.length > 0) {
        result[enterPool.shift()] = currentTime;
      } else if (exitPool.length > 0) {
        result[exitPool.shift()] = currentTime;
        previousState = 1;
      } else {
        previousState = 1;
      }
    }

    currentTime++;
  }

  return result;
};
```

2535 Difference Between Element Sum And Digit Sum Of An Array

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2535. Difference Between Element Sum and Digit Sum of an Array
 * https://leetcode.com/problems/difference-between-element-sum-and-digit-sum-of-an-array/
 * Difficulty: Easy
 *
 * You are given a positive integer array nums:
```

```
 * - The element sum is the sum of all the elements in nums.
 * - The digit sum is the sum of all the digits (not necessarily distinct)
 *   that appear in nums.
 *
 * Return the absolute difference between the element sum and digit sum of nums.
 * Note that the absolute difference between two integers x and y is defined
 * as |x - y|.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var differenceOfSum = function(nums) {
  const sum = input => input.reduce((sum, n) => sum + +n, 0);
  return Math.abs(sum(nums) - sum(nums.join('').split('')));
};
```

2536 Increment Submatrices By One

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2536. Increment Submatrices by One
 * https://leetcode.com/problems/increment-submatrices-by-one/
 * Difficulty: Medium
 *
 * You are given a positive integer n, indicating that we initially have an n x n 0-indexed
integer
 * matrix mat filled with zeroes.
 *
 * You are also given a 2D integer array query. For each query[i] = [row1i, col1i, row2i,
col2i],
 * you should do the following operation:
 * - Add 1 to every element in the submatrix with the top left corner (row1i, col1i) and the
bottom
 *   right corner (row2i, col2i). That is, add 1 to mat[x][y] for all row1i <= x <= row2i and
 *   col1i <= y <= col2i.
 *
 * Return the matrix mat after performing every query.
 */

/**
 * @param {number} n
 * @param {number[][]} queries
 * @return {number[][]}
 */
var rangeAddQueries = function(n, queries) {
  const matrix = new Array(n).fill().map(() => new Array(n).fill(0));

  for (const [row1, col1, row2, col2] of queries) {
    matrix[row1][col1]++;
    if (row2 + 1 < n) matrix[row2 + 1][col1]--;
    if (col2 + 1 < n) matrix[row1][col2 + 1]--;
    if (row2 + 1 < n && col2 + 1 < n) matrix[row2 + 1][col2 + 1]++;
  }

  for (let i = 0; i < n; i++) {
    for (let j = 1; j < n; j++) {
      matrix[i][j] += matrix[i][j - 1];
    }
  }

  for (let i = 1; i < n; i++) {
    for (let j = 0; j < n; j++) {
      matrix[i][j] += matrix[i - 1][j];
    }
  }

  return matrix;
```

```
};
```

2537 Count The Number Of Good Subarrays

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2537. Count the Number of Good Subarrays
 * https://leetcode.com/problems/count-the-number-of-good-subarrays/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, return the number of good subarrays of nums.
 *
 * A subarray arr is good if there are at least k pairs of indices (i, j) such that i < j
 * and arr[i] == arr[j].
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countGood = function(nums, k) {
  const frequency = new Map();
  let result = 0;
  let currentPairs = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    const num = nums[right];
    const prevCount = frequency.get(num) || 0;
    currentPairs += prevCount;
    frequency.set(num, prevCount + 1);

    while (currentPairs >= k) {
      result += nums.length - right;
      const leftNum = nums[left];
      const leftCount = frequency.get(leftNum);
      currentPairs -= leftCount - 1;
      frequency.set(leftNum, leftCount - 1);
      left++;
    }
  }

  return result;
};
```

2539 Count The Number Of Good Subsequences

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2539. Count the Number of Good Subsequences
 * https://leetcode.com/problems/count-the-number-of-good-subsequences/
 * Difficulty: Medium
 *
 * A subsequence of a string is good if it is not empty and the frequency of each one of its
 * characters is the same.
 *
 * Given a string s, return the number of good subsequences of s. Since the answer may be too
 * large, return it modulo 109 + 7.
 *
```

```
 * A subsequence is a string that can be derived from another string by deleting some or no
 * characters without changing the order of the remaining characters.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countGoodSubsequences = function(s) {
  const MOD = 1e9 + 7;
  const n = s.length;
  const factorial = new Array(n + 1);

  const frequency = new Array(26).fill(0);
  let maxFrequency = 0;

  for (let i = 0; i < n; i++) {
    frequency[s.charCodeAt(i) - 97]++;
  }

  for (let i = 0; i < 26; i++) {
    maxFrequency = Math.max(maxFrequency, frequency[i]);
  }

  let result = 0n;

  for (let k = 1; k <= maxFrequency; k++) {
    let current = 1n;
    for (let i = 0; i < 26; i++) {
      if (frequency[i] > 0) {
        current = (current * (1n + combination(frequency[i], k))) % BigInt(MOD);
      }
    }
    result = (result + current - 1n + BigInt(MOD)) % BigInt(MOD);
  }

  return Number(result);

  function computeFactorial(num) {
    if (factorial[num] !== undefined) return factorial[num];
    if (num === 0) {
      factorial[num] = 1n;
    } else {
      factorial[num] = (computeFactorial(num - 1) * BigInt(num)) % BigInt(MOD);
    }
    return factorial[num];
  }

  function modInverse(a) {
    let result = 1n;
    let base = a % BigInt(MOD);
    let exp = BigInt(MOD - 2);
    while (exp > 0n) {
      if (exp & 1n) result = (result * base) % BigInt(MOD);
      base = (base * base) % BigInt(MOD);
      exp >>= 1n;
    }
    return result;
  }

  function combination(n, k) {
    if (n < k) return 0n;
    if (n === k) return 1n;
    let result = computeFactorial(n);
    result = (result * modInverse(computeFactorial(k))) % BigInt(MOD);
    result = (result * modInverse(computeFactorial(n - k))) % BigInt(MOD);
    return result;
  }
};
```

2540 Minimum Common Value

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2540. Minimum Common Value
 * https://leetcode.com/problems/minimum-common-value/
 * Difficulty: Easy
 *
 * Given two integer arrays nums1 and nums2, sorted in non-decreasing order, return the
minimum
 * integer common to both arrays. If there is no common integer amongst nums1 and nums2,
return -1.
 *
 * Note that an integer is said to be common to nums1 and nums2 if both arrays have at least
one
 * occurrence of that integer.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var getCommon = function(nums1, nums2) {
  let i = 0;
  let j = 0;

  while (i < nums1.length && j < nums2.length) {
    if (nums1[i] === nums2[j]) return nums1[i];
    if (nums1[i] < nums2[j]) i++;
    else j++;
  }

  return -1;
};
```

2542 Maximum Subsequence Score

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2542. Maximum Subsequence Score
 * https://leetcode.com/problems/maximum-subsequence-score/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer arrays nums1 and nums2 of equal length n and a positive
 * integer k. You must choose a subsequence of indices from nums1 of length k.
 *
 * For chosen indices i0, i1, ..., ik - 1, your score is defined as:
 * - The sum of the selected elements from nums1 multiplied with the minimum of the selected
 *   elements from nums2.
 * - It can defined simply as: (nums1[i0] + nums1[i1] +...+ nums1[ik - 1]) * min(nums2[i0],
 *   nums2[i1], ... ,nums2[ik - 1]).
 *
 * Return the maximum possible score.
 *
 * A subsequence of indices of an array is a set that can be derived from the set {0, 1, ...,
n-1}
 * by deleting some or no elements.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
```

```
 * @param {number} k
 * @return {number}
 */
var maxScore = function(nums1, nums2, k) {
  const zipped = nums1.map((num1, i) => [num1, nums2[i]]).sort((a, b) => b[1] - a[1]);
  const heap = new MinPriorityQueue();
  let result = 0;
  let sum = 0;

  for (const [num, min] of zipped) {
    heap.enqueue(num);
    sum += num;

    if (heap.size() == k) {
      result = Math.max(result, sum * min);
      sum -= heap.dequeue().element;
    }
  }

  return result;
};
```

2543 Check If Point Is Reachable

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2543. Check if Point Is Reachable
 * https://leetcode.com/problems/check-if-point-is-reachable/
 * Difficulty: Hard
 *
 * There exists an infinitely large grid. You are currently at point (1, 1), and you need
 * to reach the point (targetX, targetY) using a finite number of steps.
 *
 * In one step, you can move from point (x, y) to any one of the following points:
 * - (x, y - x)
 * - (x - y, y)
 * - (2 * x, y)
 * - (x, 2 * y)
 *
 * Given two integers targetX and targetY representing the X-coordinate and Y-coordinate of
 * your final position, return true if you can reach the point from (1, 1) using some number
 * of steps, and false otherwise.
 */

/**
 * @param {number} targetX
 * @param {number} targetY
 * @return {boolean}
 */
var isReachable = function(targetX, targetY) {
  let g = gcd(targetX, targetY);
  while (g % 2 === 0) {
    g /= 2;
  }

  return g === 1;
};

function gcd(a, b) {
  while (b) {
    a %= b;
    [a, b] = [b, a];
  }
  return a;
}
```

2544 Alternating Digit Sum

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2544. Alternating Digit Sum
 * https://leetcode.com/problems/alternating-digit-sum/
 * Difficulty: Easy
 *
 * You are given a positive integer n. Each digit of n has a sign according to the following
rules:
 * - The most significant digit is assigned a positive sign.
 * - Each other digit has an opposite sign to its adjacent digits.
 *
 * Return the sum of all digits with their corresponding sign.
 */

/**
 * @param {number} n
 * @return {number}
 */
var alternateDigitSum = function(n) {
  let sum = 0;
  let isPositive = true;

  while (n > 0) {
    const digit = n % 10;
    sum += isPositive ? digit : -digit;
    isPositive = !isPositive;
    n = Math.floor(n / 10);
  }

  return isPositive ? -sum : sum;
};
```

2545 Sort The Students By Their Kth Score

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2545. Sort the Students by Their Kth Score
 * https://leetcode.com/problems/sort-the-students-by-their-kth-score/
 * Difficulty: Medium
 *
 * There is a class with m students and n exams. You are given a 0-indexed m x n integer
matrix
 * score, where each row represents one student and score[i][j] denotes the score the ith
student
 * got in the jth exam. The matrix score contains distinct integers only.
 *
 * You are also given an integer k. Sort the students (i.e., the rows of the matrix) by their
 * scores in the kth (0-indexed) exam from the highest to the lowest.
 *
 * Return the matrix after sorting it.
 */

/**
 * @param {number[][]} score
 * @param {number} k
 * @return {number[][]}
 */
var sortTheStudents = function(score, k) {
  return score.sort((studentA, studentB) => studentB[k] - studentA[k]);
};
```

2546 Apply Bitwise Operations To Make Strings Equal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2546. Apply Bitwise Operations to Make Strings Equal
 * https://leetcode.com/problems/apply-bitwise-operations-to-make-strings-equal/
 * Difficulty: Medium
 *
 * You are given two 0-indexed binary strings s and target of the same length n. You can do
the
 * following operation on s any number of times:
 * - Choose two different indices i and j where 0 <= i, j < n.
 * - Simultaneously, replace s[i] with (s[i] OR s[j]) and s[j] with (s[i] XOR s[j]).
 *
 * For example, if s = "0110", you can choose i = 0 and j = 2, then simultaneously replace
s[0]
 * with (s[0] OR s[2] = 0 OR 1 = 1), and s[2] with (s[0] XOR s[2] = 0 XOR 1 = 1), so we will
 * have s = "1110".
 *
 * Return true if you can make the string s equal to target, or false otherwise.
 */

/**
 * @param {string} s
 * @param {string} target
 * @return {boolean}
 */
var makeStringsEqual = function(s, target) {
  return (s.includes('1') === target.includes('1'));
};
```

2547 Minimum Cost To Split An Array

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2547. Minimum Cost to Split an Array
 * https://leetcode.com/problems/minimum-cost-to-split-an-array/
 * Difficulty: Hard
 *
 * You are given an integer array nums and an integer k.
 *
 * Split the array into some number of non-empty subarrays. The cost of a split is the sum of
 * the importance value of each subarray in the split.
 *
 * Let trimmed(subarray) be the version of the subarray where all numbers which appear only
 * once are removed.
 * - For example, trimmed([3,1,2,4,3,4]) = [3,4,3,4].
 *
 * The importance value of a subarray is k + trimmed(subarray).length.
 * - For example, if a subarray is [1,2,3,3,3,4,4], then trimmed([1,2,3,3,3,4,4]) =
[3,3,3,4,4].
 *   The importance value of this subarray will be k + 5.
 *
 * Return the minimum possible cost of a split of nums.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
```

```
var minCost = function(nums, k) {
  const n = nums.length;
  const dp = new Array(n + 1).fill(Infinity);
  dp[0] = 0;

  for (let end = 1; end <= n; end++) {
    const freq = new Map();
    let trimLen = 0;
    for (let start = end - 1; start >= 0; start--) {
      const num = nums[start];
      const count = (freq.get(num) || 0) + 1;
      freq.set(num, count);
      if (count === 1) trimLen++;
      if (count === 2) trimLen--;
      dp[end] = Math.min(dp[end], dp[start] + k + (end - start - trimLen));
    }
  }

  return dp[n];
};
```

2548 Maximum Price To Fill A Bag

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2548. Maximum Price to Fill a Bag
 * https://leetcode.com/problems/maximum-price-to-fill-a-bag/
 * Difficulty: Medium
 *
 * You are given a 2D integer array items where items[i] = [pricei, weighti] denotes the
 * price and weight of the ith item, respectively.
 *
 * You are also given a positive integer capacity.
 *
 * Each item can be divided into two items with ratios part1 and part2, where part1 + part2 ==
 1.
 * - The weight of the first item is weighti * part1 and the price of the first item is
 *   pricei * part1.
 * - Similarly, the weight of the second item is weighti * part2 and the price of the second
 *   item is pricei * part2.
 *
 * Return the maximum total price to fill a bag of capacity capacity with given items. If it
 * is impossible to fill a bag return -1. Answers within 10-5 of the actual answer will be
 * considered accepted.
 */

/**
 * @param {number[][]} items
 * @param {number} capacity
 * @return {number}
 */
var maxPrice = function(items, capacity) {
  const sortedItems = items.sort((a, b) => (b[0] / b[1]) - (a[0] / a[1]));
  let score = 0;

  for (const [price, weight] of sortedItems) {
    const take = Math.min(weight, capacity);
    score += price * take / weight;
    capacity -= take;
  }

  return capacity === 0 ? score : -1;
};
```

2549 Count Distinct Numbers On Board

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2549. Count Distinct Numbers on Board
 * https://leetcode.com/problems/count-distinct-numbers-on-board/
 * Difficulty: Easy
 *
 * You are given a positive integer n, that is initially placed on a board. Every day, for
 * 109 days, you perform the following procedure:
 * - For each number x present on the board, find all numbers 1 <= i <= n such that x % i ==
1.
 * - Then, place those numbers on the board.
 *
 * Return the number of distinct integers present on the board after 109 days have elapsed.
 *
 * Note:
 * - Once a number is placed on the board, it will remain on it until the end.
 * - % stands for the modulo operation. For example, 14 % 3 is 2.
 */

/**
 * @param {number} n
 * @return {number}
 */
var distinctIntegers = function(n) {
  return n === 1 ? 1 : n - 1;
};
```

2551 Put Marbles In Bags

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2551. Put Marbles in Bags
 * https://leetcode.com/problems/put-marbles-in-bags/
 * Difficulty: Hard
 *
 * You have k bags. You are given a 0-indexed integer array weights where weights[i] is the
weight
 * of the ith marble. You are also given the integer k.
 *
 * Divide the marbles into the k bags according to the following rules:
 * - No bag is empty.
 * - If the ith marble and jth marble are in a bag, then all marbles with an index between the
 ith
 *   and jth indices should also be in that same bag.
 * - If a bag consists of all the marbles with an index from i to j inclusively, then the cost
 of
 *   the bag is weights[i] + weights[j].
 *
 * The score after distributing the marbles is the sum of the costs of all the k bags.
 *
 * Return the difference between the maximum and minimum scores among marble distributions.
 */

/**
 * @param {number[]} weights
 * @param {number} k
 * @return {number}
 */
var putMarbles = function(weights, k) {
  const n = weights.length;
  if (k === n) return 0;
```

```
  const pairSums = [];
  for (let i = 0; i < n - 1; i++) {
    pairSums.push(weights[i] + weights[i + 1]);
  }

  pairSums.sort((a, b) => a - b);

  let minScore = 0;
  let maxScore = 0;

  for (let i = 0; i < k - 1; i++) {
    minScore += pairSums[i];
    maxScore += pairSums[n - 2 - i];
  }

  return maxScore - minScore;
};
```

2553 Separate The Digits In An Array

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2553. Separate the Digits in an Array
 * https://leetcode.com/problems/separate-the-digits-in-an-array/
 * Difficulty: Easy
 *
 * Given an array of positive integers nums, return an array answer that consists of the
digits
 * of each integer in nums after separating them in the same order they appear in nums.
 *
 * To separate the digits of an integer is to get all the digits it has in the same order.
 * - For example, for the integer 10921, the separation of its digits is [1,0,9,2,1].
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var separateDigits = function(nums) {
  return nums.flatMap(num => String(num).split('').map(Number));
};
```

2554 Maximum Number Of Integers To Choose From A Range I

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2554. Maximum Number of Integers to Choose From a Range I
 * https://leetcode.com/problems/maximum-number-of-integers-to-choose-from-a-range-i/
 * Difficulty: Medium
 *
 * You are given an integer array banned and two integers n and maxSum. You are choosing some
 * number of integers following the below rules:
 * - The chosen integers have to be in the range [1, n].
 * - Each integer can be chosen at most once.
 * - The chosen integers should not be in the array banned.
 * - The sum of the chosen integers should not exceed maxSum.
 *
 * Return the maximum number of integers you can choose following the mentioned rules.
 */

/**
 * @param {number[]} banned
```

```
 * @param {number} n
 * @param {number} maxSum
 * @return {number}
 */
var maxCount = function(banned, n, maxSum) {
  const set = new Set(banned);
  let result = 0;
  let currentSum = 0;

  for (let num = 1; num <= n; num++) {
    if (!set.has(num) && currentSum + num <= maxSum) {
      result++;
      currentSum += num;
    }
  }

  return result;
};
```

2557 Maximum Number Of Integers To Choose From A Range Ii

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2557. Maximum Number of Integers to Choose From a Range II
 * https://leetcode.com/problems/maximum-number-of-integers-to-choose-from-a-range-ii/
 * Difficulty: Medium
 *
 * You are given an integer array banned and two integers n and maxSum. You are choosing some
 * number of integers following the below rules:
 * - The chosen integers have to be in the range [1, n].
 * - Each integer can be chosen at most once.
 * - The chosen integers should not be in the array banned.
 * - The sum of the chosen integers should not exceed maxSum.
 *
 * Return the maximum number of integers you can choose following the mentioned rules.
 */

/**
 * @param {number[]} banned
 * @param {number} n
 * @param {number} maxSum
 * @return {number}
 */
var maxCount = function(banned, n, maxSum) {
  const initialLimit = Math.min(n, Math.floor((-1 + Math.sqrt(1 + 8 * maxSum)) / 2));
  const bannedNumbers = new Set(banned);
  let totalSum = Math.floor(initialLimit * (initialLimit + 1) / 2);
  let result = initialLimit;

  for (const bannedNumber of bannedNumbers) {
    if (bannedNumber <= initialLimit) {
      totalSum -= bannedNumber;
      result -= 1;
    }
  }

  for (let candidate = initialLimit + 1; candidate <= n; candidate++) {
    if (totalSum + candidate > maxSum) {
      return result;
    }

    if (!bannedNumbers.has(candidate)) {
      totalSum += candidate;
      result += 1;
    }
  }

  return result;
};
```

2559 Count Vowel Strings In Ranges

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2559. Count Vowel Strings in Ranges
 * https://leetcode.com/problems/count-vowel-strings-in-ranges/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of strings words and a 2D array of integers queries.
 *
 * Each query queries[i] = [li, ri] asks us to find the number of strings present at the
 * indices ranging from li to ri (both inclusive) of words that start and end with a vowel.
 *
 * Return an array ans of size queries.length, where ans[i] is the answer to the ith query.
 *
 * Note that the vowel letters are 'a', 'e', 'i', 'o', and 'u'.
 */

/**
 * @param {string[]} words
 * @param {number[][]} queries
 * @return {number[]}
 */
var vowelStrings = function(words, queries) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u']);
  const prefix = [0];
  for (let i = 0; i < words.length; i++) {
    prefix[i + 1] = prefix[i] + (vowels.has(words[i][0]) && vowels.has(words[i].at(-1)));
  }
  return queries.map(([l, r]) => prefix[r + 1] - prefix[l]);
};
```

2560 House Robber Iv

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2560. House Robber IV
 * https://leetcode.com/problems/house-robber-iv/
 * Difficulty: Medium
 *
 * There are several consecutive houses along a street, each of which has some money inside.
 * There is also a robber, who wants to steal money from the homes, but he refuses to steal
 * from adjacent homes.
 *
 * The capability of the robber is the maximum amount of money he steals from one house of
 * all the houses he robbed.
 *
 * You are given an integer array nums representing how much money is stashed in each house.
 * More formally, the ith house from the left has nums[i] dollars.
 *
 * You are also given an integer k, representing the minimum number of houses the robber will
 * steal from. It is always possible to steal at least k houses.
 *
 * Return the minimum capability of the robber out of all the possible ways to steal at least
 * k houses.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minCapability = function(nums, k) {
```

```
  let left = Math.min(...nums);
  let right = Math.max(...nums);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);

    if (canRob(nums, mid, k)) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;
};

function canRob(nums, capability, k) {
  let count = 0;
  let i = 0;

  while (i < nums.length) {
    if (nums[i] <= capability) {
      count++;
      i += 2;
    } else {
      i++;
    }
  }

  return count >= k;
}
```

2561 Rearranging Fruits

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2561. Rearranging Fruits
 * https://leetcode.com/problems/rearranging-fruits/
 * Difficulty: Hard
 *
 * You have two fruit baskets containing n fruits each. You are given two 0-indexed integer
 * arrays basket1 and basket2 representing the cost of fruit in each basket. You want to
 * make both baskets equal. To do so, you can use the following operation as many times as you
 want:
 * - Chose two indices i and j, and swap the ith fruit of basket1 with the jth fruit of
basket2.
 * - The cost of the swap is min(basket1[i],basket2[j]).
 *
 * Two baskets are considered equal if sorting them according to the fruit cost makes them
 * exactly the same baskets.
 *
 * Return the minimum cost to make both the baskets equal or -1 if impossible.
 */

/**
 * @param {number[]} basket1
 * @param {number[]} basket2
 * @return {number}
 */
var minCost = function(basket1, basket2) {
  const count1 = new Map();
  const count2 = new Map();

  for (const fruit of basket1) {
    count1.set(fruit, (count1.get(fruit) || 0) + 1);
  }

  for (const fruit of basket2) {
    count2.set(fruit, (count2.get(fruit) || 0) + 1);
```

```
  }

  const allFruits = new Set([...basket1, ...basket2]);
  const excess1 = [];
  const excess2 = [];
  let minCost = Infinity;

  for (const fruit of allFruits) {
    const freq1 = count1.get(fruit) || 0;
    const freq2 = count2.get(fruit) || 0;
    const total = freq1 + freq2;

    if (total % 2 !== 0) return -1;

    minCost = Math.min(minCost, fruit);
    const target = total / 2;

    if (freq1 > target) {
      for (let i = 0; i < freq1 - target; i++) {
        excess1.push(fruit);
      }
    } else if (freq2 > target) {
      for (let i = 0; i < freq2 - target; i++) {
        excess2.push(fruit);
      }
    }
  }

  excess1.sort((a, b) => a - b);
  excess2.sort((a, b) => b - a);

  let result = 0;
  for (let i = 0; i < excess1.length; i++) {
    const directSwap = Math.min(excess1[i], excess2[i]);
    const doubleSwap = 2 * minCost;
    result += Math.min(directSwap, doubleSwap);
  }

  return result;
};
```

2562 Find The Array Concatenation Value

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2562. Find the Array Concatenation Value
 * https://leetcode.com/problems/find-the-array-concatenation-value/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums.
 *
 * The concatenation of two numbers is the number formed by concatenating their numerals.
 * - For example, the concatenation of 15, 49 is 1549.
 *
 * The concatenation value of nums is initially equal to 0. Perform this operation until nums
 * becomes empty:
 * - If nums has a size greater than one, add the value of the concatenation of the first and
 *   the last element to the concatenation value of nums, and remove those two elements from
 *   nums. For example, if the nums was [1, 2, 4, 5, 6], add 16 to the concatenation value.
 * - If only one element exists in nums, add its value to the concatenation value of nums,
 *   then remove it.
 *
 * Return the concatenation value of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
```

```
var findTheArrayConcVal = function(nums) {
  let result = 0;
  let left = 0;
  let right = nums.length - 1;

  while (left <= right) {
    if (left === right) {
      result += nums[left];
    } else {
      result += Number(`${nums[left]}${nums[right]}`);
    }
    left++;
    right--;
  }

  return result;
};
```

2563 Count The Number Of Fair Pairs

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2563. Count the Number of Fair Pairs
 * https://leetcode.com/problems/count-the-number-of-fair-pairs/
 * Difficulty: Medium
 *
 * Given a 0-indexed integer array nums of size n and two integers lower and upper, return the
 * number of fair pairs.
 *
 * A pair (i, j) is fair if:
 * - 0 <= i < j < n, and
 * - lower <= nums[i] + nums[j] <= upper
 */

/**
 * @param {number[]} nums
 * @param {number} lower
 * @param {number} upper
 * @return {number}
 */
var countFairPairs = function(nums, lower, upper) {
  let result = 0;

  nums.sort((a, b) => a - b);

  for (let i = 0; i < nums.length - 1; i++) {
    const minTarget = lower - nums[i];
    const maxTarget = upper - nums[i];

    const first = findFirstValidIndex(i + 1, minTarget);
    const last = findFirstValidIndex(i + 1, maxTarget + 1) - 1;

    if (first <= last) {
      result += last - first + 1;
    }
  }

  return result;

  function findFirstValidIndex(start, target) {
    let left = start;
    let right = nums.length - 1;
    let index = nums.length;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      if (nums[mid] >= target) {
        index = mid;
```

```
        right = mid - 1;
      } else {
        left = mid + 1;
      }
    }

    return index;
  }
};
```

2566 Maximum Difference By Remapping A Digit

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2566. Maximum Difference by Remapping a Digit
 * https://leetcode.com/problems/maximum-difference-by-remapping-a-digit/
 * Difficulty: Easy
 *
 * You are given an integer num. You know that Bob will sneakily remap one of the 10 possible
 * digits (0 to 9) to another digit.
 *
 * Return the difference between the maximum and minimum values Bob can make by remapping
 * exactly one digit in num.
 *
 * Notes:
 * - When Bob remaps a digit d1 to another digit d2, Bob replaces all occurrences of d1 in
 *   num with d2.
 * - Bob can remap a digit to itself, in which case num does not change.
 * - Bob can remap different digits for obtaining minimum and maximum values respectively.
 * - The resulting number after remapping can contain leading zeroes.
 */

/**
 * @param {number} num
 * @return {number}
 */
var minMaxDifference = function(num) {
  const digits = String(num).split('');
  let maxNum = num;
  let minNum = num;

  for (let i = 0; i < 10; i++) {
    const digit = String(i);
    if (digits.includes(digit)) {
      const maxCandidate = Number(digits.join('').replaceAll(digit, '9'));
      const minCandidate = Number(digits.join('').replaceAll(digit, '0'));
      maxNum = Math.max(maxNum, maxCandidate);
      minNum = Math.min(minNum, minCandidate);
    }
  }

  return maxNum - minNum;
};
```

2567 Minimum Score By Changing Two Elements

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2567. Minimum Score by Changing Two Elements
 * https://leetcode.com/problems/minimum-score-by-changing-two-elements/
 * Difficulty: Medium
 *
 * You are given an integer array nums.
```

```
 * - The low score of nums is the minimum absolute difference between any two integers.
 * - The high score of nums is the maximum absolute difference between any two integers.
 * - The score of nums is the sum of the high and low scores.
 *
 * Return the minimum score after changing two elements of nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimizeSum = function(nums) {
  const sorted = nums.sort((a, b) => a - b);

  return Math.min(
    sorted[sorted.length - 2] - sorted[1],
    sorted[sorted.length - 1] - sorted[2],
    sorted[sorted.length - 3] - sorted[0]
  );
};
```

2568 Minimum Impossible Or

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2568. Minimum Impossible OR
 * https://leetcode.com/problems/minimum-impossible-or/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums.
 *
 * We say that an integer x is expressible from nums if there exist some integers
 * 0 <= index1 < index2 < ... < indexk < nums.length for which
 * nums[index1] | nums[index2] | ... | nums[indexk] = x. In other words, an
 * integer is expressible if it can be written as the bitwise OR of some subsequence
 * of nums.
 *
 * Return the minimum positive non-zero integer that is not expressible from nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minImpossibleOR = function(nums) {
  const set = new Set(nums);
  let result = 1;

  while (set.has(result)) {
    result <<= 1;
  }

  return result;
};
```

2570 Merge Two 2D Arrays By Summing Values

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2570. Merge Two 2D Arrays by Summing Values
 * https://leetcode.com/problems/merge-two-2d-arrays-by-summing-values/
 * Difficulty: Easy
```

```
 *
 * You are given two 2D integer arrays nums1 and nums2.
 * - nums1[i] = [idi, vali] indicate that the number with the id idi has a value equal to
vali.
 * - nums2[i] = [idi, vali] indicate that the number with the id idi has a value equal to
vali.
 *
 * Each array contains unique ids and is sorted in ascending order by id.
 *
 * Merge the two arrays into one array that is sorted in ascending order by id, respecting the
 * following conditions:
 * - Only ids that appear in at least one of the two arrays should be included in the
resulting
 *   array.
 * - Each id should be included only once and its value should be the sum of the values of
this
 *   id in the two arrays. If the id does not exist in one of the two arrays, then assume its
 *   value in that array to be 0.
 *
 * Return the resulting array. The returned array must be sorted in ascending order by id.
 */

/**
 * @param {number[][]} nums1
 * @param {number[][]} nums2
 * @return {number[][]}
 */
var mergeArrays = function(nums1, nums2) {
  const map = new Map(nums1);
  for (const [key, value] of nums2) {
    map.set(key, (map.get(key) ?? 0) + value);
  }
  return [...map].sort((a, b) => a[0] - b[0]);
};
```

2571 Minimum Operations To Reduce An Integer To 0

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2571. Minimum Operations to Reduce an Integer to 0
 * https://leetcode.com/problems/minimum-operations-to-reduce-an-integer-to-0/
 * Difficulty: Medium
 *
 * You are given a positive integer n, you can do the following operation any number of times:
 * - Add or subtract a power of 2 from n.
 *
 * Return the minimum number of operations to make n equal to 0.
 *
 * A number x is power of 2 if x == 2i where i >= 0.
 */

/**
 * @param {number} n
 * @return {number}
 */
var minOperations = function(n) {
  let result = 0;

  while (n > 0) {
    if ((n & 1) === 0) {
      n >>= 1;
    } else if ((n & 3) === 3) {
      n += 1;
      result++;
    } else {
      n -= 1;
      result++;
    }
  }
```

```
  return result;
};
```

2574 Left And Right Sum Differences

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2574. Left and Right Sum Differences
 * https://leetcode.com/problems/left-and-right-sum-differences/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums of size n.
 *
 * Define two arrays leftSum and rightSum where:
 * - leftSum[i] is the sum of elements to the left of the index i in the array nums.
 *   If there is no such element, leftSum[i] = 0.
 * - rightSum[i] is the sum of elements to the right of the index i in the array nums.
 *   If there is no such element, rightSum[i] = 0.
 *
 * Return an integer array answer of size n where answer[i] = |leftSum[i] - rightSum[i]|.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var leftRightDifference = function(nums) {
  const n = nums.length;
  const result = new Array(n).fill(0);
  let leftSum = 0;
  let rightSum = nums.reduce((sum, num) => sum + num, 0);

  for (let i = 0; i < n; i++) {
    rightSum -= nums[i];
    result[i] = Math.abs(leftSum - rightSum);
    leftSum += nums[i];
  }

  return result;
};
```

2576 Find The Maximum Number Of Marked Indices

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2576. Find the Maximum Number of Marked Indices
 * https://leetcode.com/problems/find-the-maximum-number-of-marked-indices/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums.
 *
 * Initially, all of the indices are unmarked. You are allowed to make this operation any
 * number of times:
 * - Pick two different unmarked indices i and j such that 2 * nums[i] <= nums[j], then
 *   mark i and j.
 *
 * Return the maximum possible number of marked indices in nums using the above operation
 * any number of times.
 */

/**
 * @param {number[]} nums
```

```
 * @return {number}
 */
var maxNumOfMarkedIndices = function(nums) {
  nums.sort((a, b) => a - b);
  let result = 0;
  let left = 0;
  let right = Math.floor(nums.length / 2);

  while (left < Math.floor(nums.length / 2) && right < nums.length) {
    if (2 * nums[left] <= nums[right]) {
      result += 2;
      left++;
      right++;
    } else {
      right++;
    }
  }

  return result;
};
```

2578 Split With Minimum Sum

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2578. Split With Minimum Sum
 * https://leetcode.com/problems/split-with-minimum-sum/
 * Difficulty: Easy
 *
 * Given a positive integer num, split it into two non-negative integers num1 and num2 such
that:
 * - The concatenation of num1 and num2 is a permutation of num.
 *   - In other words, the sum of the number of occurrences of each digit in num1 and num2 is
equal
 *     to the number of occurrences of that digit in num.
 * - num1 and num2 can contain leading zeros.
 *
 * Return the minimum possible sum of num1 and num2.
 *
 * Notes:
 * - It is guaranteed that num does not contain any leading zeros.
 * - The order of occurrence of the digits in num1 and num2 may differ from the order of
occurrence
 *   of num.
 */

/**
 * @param {number} num
 * @return {number}
 */
var splitNum = function(num) {
  const digits = String(num).split('').sort((a, b) => a - b);
  let num1 = '';
  let num2 = '';

  for (let i = 0; i < digits.length; i++) {
    if (i % 2 === 0) {
      num1 += digits[i];
    } else {
      num2 += digits[i];
    }
  }

  return Number(num1) + Number(num2);
};
```

2579 Count Total Number Of Colored Cells

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2579. Count Total Number of Colored Cells
 * https://leetcode.com/problems/count-total-number-of-colored-cells/
 * Difficulty: Medium
 *
 * There exists an infinitely large two-dimensional grid of uncolored unit cells.
 * You are given a positive integer n, indicating that you must do the following
 * routine for n minutes:
 * - At the first minute, color any arbitrary unit cell blue.
 * - Every minute thereafter, color blue every uncolored cell that touches a blue cell.
 *
 * Below is a pictorial representation of the state of the grid after minutes 1, 2, and 3.
 */

/**
 * @param {number} n
 * @return {number}
 */
var coloredCells = function(n) {
  return 1 + 2 * n * (n - 1);
};
```

2581 Count Number Of Possible Root Nodes

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2581. Count Number of Possible Root Nodes
 * https://leetcode.com/problems/count-number-of-possible-root-nodes/
 * Difficulty: Hard
 *
 * Alice has an undirected tree with n nodes labeled from 0 to n - 1. The tree is represented
as a
 * 2D integer array edges of length n - 1 where edges[i] = [ai, bi] indicates that there is an
 edge
 * between nodes ai and bi in the tree.
 *
 * Alice wants Bob to find the root of the tree. She allows Bob to make several guesses about
her
 * tree. In one guess, he does the following:
 * - Chooses two distinct integers u and v such that there exists an edge [u, v] in the tree.
 * - He tells Alice that u is the parent of v in the tree.
 *
 * Bob's guesses are represented by a 2D integer array guesses where guesses[j] = [uj, vj]
indicates
 * Bob guessed uj to be the parent of vj.
 *
 * Alice being lazy, does not reply to each of Bob's guesses, but just says that at least k of
 his
 * guesses are true.
 *
 * Given the 2D integer arrays edges, guesses and the integer k, return the number of possible
 * nodes that can be the root of Alice's tree. If there is no such tree, return 0.
 */

/**
 * @param {number[][]} edges
 * @param {number[][]} guesses
 * @param {number} k
```

```
 * @return {number}
 */
var rootCount = function(edges, guesses, k) {
  const n = edges.length + 1;
  const adj = Array.from({ length: n }, () => new Set());
  const set = new Set();

  for (const [u, v] of edges) {
    adj[u].add(v);
    adj[v].add(u);
  }

  for (const [u, v] of guesses) {
    set.add(u * n + v);
  }

  const rootCorrect = dfs(0, -1);
  let result = rootCorrect >= k ? 1 : 0;

  for (const neighbor of adj[0]) {
    helper(neighbor, 0, rootCorrect);
  }

  return result;

  function helper(node, parent, parentCorrect) {
    let count = parentCorrect;
    if (set.has(parent * n + node)) count--;
    if (set.has(node * n + parent)) count++;

    if (count >= k) result++;

    for (const neighbor of adj[node]) {
      if (neighbor !== parent) {
        helper(neighbor, node, count);
      }
    }
  }

  function dfs(node, parent) {
    let count = 0;
    for (const neighbor of adj[node]) {
      if (neighbor !== parent) {
        if (set.has(node * n + neighbor)) count++;
        count += dfs(neighbor, node);
      }
    }
    return count;
  }
};
```

2582 Pass The Pillow

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2582. Pass the Pillow
 * https://leetcode.com/problems/pass-the-pillow/
 * Difficulty: Easy
 *
 * There are n people standing in a line labeled from 1 to n. The first person in the line is
 * holding a pillow initially. Every second, the person holding the pillow passes it to the
 * next person standing in the line. Once the pillow reaches the end of the line, the
direction
 * changes, and people continue passing the pillow in the opposite direction.
 * - For example, once the pillow reaches the nth person they pass it to the n - 1th person,
 *   then to the n - 2th person and so on.
 *
 * Given the two positive integers n and time, return the index of the person holding the
pillow
```

```
 * after time seconds.
 */

/**
 * @param {number} n
 * @param {number} time
 * @return {number}
 */
var passThePillow = function(n, time) {
  const position = time % (2 * (n - 1));
  return position < n ? position + 1 : 2 * n - position - 1;
};
```

2583 Kth Largest Sum In A Binary Tree

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2583. Kth Largest Sum in a Binary Tree
 * https://leetcode.com/problems/kth-largest-sum-in-a-binary-tree/
 * Difficulty: Medium
 *
 * You are given the root of a binary tree and a positive integer k.
 *
 * The level sum in the tree is the sum of the values of the nodes that are on the same level.
 *
 * Return the kth largest level sum in the tree (not necessarily distinct). If there are fewer
 * than k levels in the tree, return -1.
 *
 * Note that two nodes are on the same level if they have the same distance from the root.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} k
 * @return {number}
 */
var kthLargestLevelSum = function(root, k) {
  const levelSums = [];
  const queue = [root];

  while (queue.length) {
    let levelSize = queue.length;
    let levelSum = 0;

    while (levelSize--) {
      const node = queue.shift();
      levelSum += node.val;
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }

    levelSums.push(levelSum);
  }

  if (levelSums.length < k) return -1;

  return levelSums.sort((a, b) => b - a)[k - 1];
};
```

2585 Number Of Ways To Earn Points

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2585. Number of Ways to Earn Points
 * https://leetcode.com/problems/number-of-ways-to-earn-points/
 * Difficulty: Hard
 *
 * There is a test that has n types of questions. You are given an integer target and a
0-indexed
 * 2D integer array types where types[i] = [counti, marksi] indicates that there are counti
 * questions of the ith type, and each one of them is worth marksi points.
 *
 * Return the number of ways you can earn exactly target points in the exam. Since the answer
may
 * be too large, return it modulo 109 + 7.
 *
 * Note that questions of the same type are indistinguishable.
 * - For example, if there are 3 questions of the same type, then solving the 1st and 2nd
questions
 *   is the same as solving the 1st and 3rd questions, or the 2nd and 3rd questions.
 */

/**
 * @param {number} target
 * @param {number[][]} types
 * @return {number}
 */
var waysToReachTarget = function(target, types) {
  const MOD = 1e9 + 7;
  const dp = new Array(target + 1).fill(0);
  dp[0] = 1;

  for (const [count, marks] of types) {
    for (let points = target; points >= 0; points--) {
      for (let k = 1; k <= count && points >= k * marks; k++) {
        dp[points] = (dp[points] + dp[points - k * marks]) % MOD;
      }
    }
  }

  return dp[target];
};
```

2586 Count The Number Of Vowel Strings In Range

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2586. Count the Number of Vowel Strings in Range
 * https://leetcode.com/problems/count-the-number-of-vowel-strings-in-range/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array of string words and two integers left and right.
 *
 * A string is called a vowel string if it starts with a vowel character and ends with
 * a vowel character where vowel characters are 'a', 'e', 'i', 'o', and 'u'.
 *
 * Return the number of vowel strings words[i] where i belongs to the inclusive range [left,
right].
 */

/**
 * @param {string[]} words
```

```
 * @param {number} left
 * @param {number} right
 * @return {number}
 */
var vowelStrings = function(words, left, right) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u']);
  let result = 0;

  for (let i = left; i <= right; i++) {
    const word = words[i];
    if (vowels.has(word[0]) && vowels.has(word[word.length - 1])) {
      result++;
    }
  }

  return result;
};
```

2587 Rearrange Array To Maximize Prefix Score

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2587. Rearrange Array to Maximize Prefix Score
 * https://leetcode.com/problems/rearrange-array-to-maximize-prefix-score/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. You can rearrange the elements of nums to any
 * order (including the given order).
 *
 * Let prefix be the array containing the prefix sums of nums after rearranging it. In other
words,
 * prefix[i] is the sum of the elements from 0 to i in nums after rearranging it. The score of
 nums
 * is the number of positive integers in the array prefix.
 *
 * Return the maximum score you can achieve.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxScore = function(nums) {
  const sorted = nums.sort((a, b) => b - a);
  let prefixSum = 0;
  let result = 0;

  for (const num of sorted) {
    prefixSum += num;
    if (prefixSum > 0) result++;
    else break;
  }

  return result;
};
```

2588 Count The Number Of Beautiful Subarrays

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2588. Count the Number of Beautiful Subarrays
 * https://leetcode.com/problems/count-the-number-of-beautiful-subarrays/
 * Difficulty: Medium
```

```
 *
 * You are given a 0-indexed integer array nums. In one operation, you can:
 * - Choose two different indices i and j such that 0 <= i, j < nums.length.
 * - Choose a non-negative integer k such that the kth bit (0-indexed) in the binary
representation
 *   of nums[i] and nums[j] is 1.
 * - Subtract 2k from nums[i] and nums[j].
 *
 * A subarray is beautiful if it is possible to make all of its elements equal to 0 after
applying
 * the above operation any number of times.
 *
 * Return the number of beautiful subarrays in the array nums.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var beautifulSubarrays = function(nums) {
  let result = 0;
  const prefixXOR = new Map([[0, 1]]);
  let currentXOR = 0;

  for (const num of nums) {
    currentXOR ^= num;
    result += prefixXOR.get(currentXOR) || 0;
    prefixXOR.set(currentXOR, (prefixXOR.get(currentXOR) || 0) + 1);
  }

  return result;
};
```

2590 Design A Todo List

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2590. Design a Todo List
 * https://leetcode.com/problems/design-a-todo-list/
 * Difficulty: Medium
 *
 * Design a Todo List Where users can add tasks, mark them as complete, or get a list of
pending
 * tasks. Users can also add tags to tasks and can filter the tasks by certain tags.
 *
 * Implement the TodoList class:
 * - TodoList() Initializes the object.
 * - int addTask(int userId, String taskDescription, int dueDate, List<String> tags) Adds a
task for
 *   the user with the ID userId with a due date equal to dueDate and a list of tags attached
to the
 *   task. The return value is the ID of the task. This ID starts at 1 and is sequentially
 *   increasing. That is, the first task's id should be 1, the second task's id should be 2,
and so
 *   on.
 * - List<String> getAllTasks(int userId) Returns a list of all the tasks not marked as
complete for
 *   the user with ID userId, ordered by the due date. You should return an empty list if the
user
 *   has no uncompleted tasks.
 * - List<String> getTasksForTag(int userId, String tag) Returns a list of all the tasks that
are
 *   not marked as complete for the user with the ID userId and have tag as one of their tags,
 *   ordered by their due date. Return an empty list if no such task exists.
 * - void completeTask(int userId, int taskId) Marks the task with the ID taskId as completed
only
 *   if the task exists and the user with the ID userId has this task, and it is uncompleted.
```

```
 */

var TodoList = function() {
  this.nextTaskId = 1;
  this.userTasks = new Map();
  this.allTasks = new Map();
};

/**
 * @param {number} userId
 * @param {string} taskDescription
 * @param {number} dueDate
 * @param {string[]} tags
 * @return {number}
 */
TodoList.prototype.addTask = function(userId, taskDescription, dueDate, tags) {
  const taskId = this.nextTaskId++;
  const task = {
    id: taskId,
    description: taskDescription,
    dueDate: dueDate,
    tags: new Set(tags),
    completed: false
  };

  if (!this.userTasks.has(userId)) {
    this.userTasks.set(userId, []);
  }
  this.userTasks.get(userId).push(task);
  this.allTasks.set(taskId, { userId, task });

  return taskId;
};

/**
 * @param {number} userId
 * @return {string[]}
 */
TodoList.prototype.getAllTasks = function(userId) {
  if (!this.userTasks.has(userId)) {
    return [];
  }

  return this.userTasks.get(userId)
    .filter(task => !task.completed)
    .sort((a, b) => a.dueDate - b.dueDate)
    .map(task => task.description);
};

/**
 * @param {number} userId
 * @param {string} tag
 * @return {string[]}
 */
TodoList.prototype.getTasksForTag = function(userId, tag) {
  if (!this.userTasks.has(userId)) {
    return [];
  }

  return this.userTasks.get(userId)
    .filter(task => !task.completed && task.tags.has(tag))
    .sort((a, b) => a.dueDate - b.dueDate)
    .map(task => task.description);
};

/**
 * @param {number} userId
 * @param {number} taskId
 * @return {void}
 */
TodoList.prototype.completeTask = function(userId, taskId) {
  const taskInfo = this.allTasks.get(taskId);
  if (taskInfo && taskInfo.userId === userId && !taskInfo.task.completed) {
    taskInfo.task.completed = true;
  }
```

```
};
```

2592 Maximize Greatness Of An Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2592. Maximize Greatness of an Array
 * https://leetcode.com/problems/maximize-greatness-of-an-array/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. You are allowed to permute nums into a new
 * array perm of your choosing.
 *
 * We define the greatness of nums be the number of indices 0 <= i < nums.length for which
 * perm[i] > nums[i].
 *
 * Return the maximum possible greatness you can achieve after permuting nums.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximizeGreatness = function(nums) {
  nums.sort((a, b) => a - b);
  let result = 0;
  let i = 0;
  let j = 1;

  while (j < nums.length) {
    if (nums[j] > nums[i]) {
      result++;
      i++;
      j++;
    } else {
      j++;
    }
  }

  return result;
};
```

2594 Minimum Time To Repair Cars

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2594. Minimum Time to Repair Cars
 * https://leetcode.com/problems/minimum-time-to-repair-cars/
 * Difficulty: Medium
 *
 * You are given an integer array ranks representing the ranks of some mechanics. ranksi is
the rank
 * of the ith mechanic. A mechanic with a rank r can repair n cars in r * n2 minutes.
 *
 * You are also given an integer cars representing the total number of cars waiting in the
garage
 * to be repaired.
 *
 * Return the minimum time taken to repair all the cars.
 *
 * Note: All the mechanics can repair the cars simultaneously.
 */
```

```
/**
 * @param {number[]} ranks
 * @param {number} cars
 * @return {number}
 */
var repairCars = function(ranks, cars) {
  let left = 1;
  let right = Math.min(...ranks) * cars * cars;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (verify(mid)) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;

  function verify(time) {
    let totalCars = 0;
    for (const rank of ranks) {
      totalCars += Math.floor(Math.sqrt(time / rank));
      if (totalCars >= cars) return true;
    }
    return false;
  }
};
```

2595 Number Of Even And Odd Bits

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2595. Number of Even and Odd Bits
 * https://leetcode.com/problems/number-of-even-and-odd-bits/
 * Difficulty: Easy
 *
 * You are given a positive integer n.
 *
 * Let even denote the number of even indices in the binary representation of n with value 1.
 *
 * Let odd denote the number of odd indices in the binary representation of n with value 1.
 *
 * Note that bits are indexed from right to left in the binary representation of a number.
 *
 * Return the array [even, odd].
 */

/**
 * @param {number} n
 * @return {number[]}
 */
var evenOddBit = function(n) {
  let evenCount = 0;
  let oddCount = 0;
  let index = 0;

  while (n > 0) {
    if (n & 1) {
      if (index % 2 === 0) evenCount++;
      else oddCount++;
    }
    n >>= 1;
    index++;
  }

  return [evenCount, oddCount];
};
```

2596 Check Knight Tour Configuration

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2596. Check Knight Tour Configuration
 * https://leetcode.com/problems/check-knight-tour-configuration/
 * Difficulty: Medium
 *
 * There is a knight on an n x n chessboard. In a valid configuration, the knight starts at
the
 * top-left cell of the board and visits every cell on the board exactly once.
 *
 * You are given an n x n integer matrix grid consisting of distinct integers from the range
 * [0, n * n - 1] where grid[row][col] indicates that the cell (row, col) is the
grid[row][col]th
 * cell that the knight visited. The moves are 0-indexed.
 *
 * Return true if grid represents a valid configuration of the knight's movements or false
 * otherwise.
 *
 * Note that a valid knight move consists of moving two squares vertically and one square
 * horizontally, or two squares horizontally and one square vertically. The figure below
 * illustrates all the possible eight moves of a knight from some cell.
 */

/**
 * @param {number[][]} grid
 * @return {boolean}
 */
var checkValidGrid = function(grid) {
  const n = grid.length;
  const moves = [[-2, -1], [-2, 1], [-1, -2], [-1, 2], [1, -2], [1, 2], [2, -1], [2, 1]];

  if (grid[0][0] !== 0) return false;

  const isValidMove = (fromRow, fromCol, toRow, toCol) => {
    for (const [dr, dc] of moves) {
      if (fromRow + dr === toRow && fromCol + dc === toCol) return true;
    }
    return false;
  };

  const positions = new Array(n * n).fill(0).map(() => [0, 0]);
  for (let row = 0; row < n; row++) {
    for (let col = 0; col < n; col++) {
      positions[grid[row][col]] = [row, col];
    }
  }

  for (let i = 1; i < n * n; i++) {
    const [prevRow, prevCol] = positions[i - 1];
    const [currRow, currCol] = positions[i];
    if (!isValidMove(prevRow, prevCol, currRow, currCol)) return false;
  }

  return true;
};
```

2597 The Number Of Beautiful Subsets

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2597. The Number of Beautiful Subsets
```

```
 * https://leetcode.com/problems/the-number-of-beautiful-subsets/
 * Difficulty: Medium
 *
 * You are given an array nums of positive integers and a positive integer k.
 *
 * A subset of nums is beautiful if it does not contain two integers with an absolute
difference
 * equal to k.
 *
 * Return the number of non-empty beautiful subsets of the array nums.
 *
 * A subset of nums is an array that can be obtained by deleting some (possibly none) elements
 * from nums. Two subsets are different if and only if the chosen indices to delete are
different.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var beautifulSubsets = function(nums, k) {
  const n = nums.length;
  let count = 0;
  backtrack(0, []);
  return count;

  function backtrack(index, selected) {
    if (index === n) {
      if (selected.length > 0) count++;
      return;
    }

    const isValid = selected.every(num => Math.abs(num - nums[index]) !== k);

    backtrack(index + 1, selected);

    if (isValid) {
      backtrack(index + 1, [...selected, nums[index]]);
    }
  }
};
```

2598 Smallest Missing Non Negative Integer After Operations

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2598. Smallest Missing Non-negative Integer After Operations
 * https://leetcode.com/problems/smallest-missing-non-negative-integer-after-operations/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums and an integer value.
 *
 * In one operation, you can add or subtract value from any element of nums.
 * - For example, if nums = [1,2,3] and value = 2, you can choose to subtract value from
nums[0]
 *   to make nums = [-1,2,3].
 *
 * The MEX (minimum excluded) of an array is the smallest missing non-negative integer in it.
 * - For example, the MEX of [-1,2,3] is 0 while the MEX of [1,0,3] is 2.
 *
 * Return the maximum MEX of nums after applying the mentioned operation any number of times.
 */

/**
 * @param {number[]} nums
 * @param {number} value
 * @return {number}
 */
```

```
var findSmallestInteger = function(nums, value) {
  const remainderCount = new Map();

  for (const num of nums) {
    const remainder = ((num % value) + value) % value;
    remainderCount.set(remainder, (remainderCount.get(remainder) || 0) + 1);
  }

  for (let mex = 0; mex <= nums.length; mex++) {
    const remainder = mex % value;
    if (!remainderCount.has(remainder) || remainderCount.get(remainder) === 0) {
      return mex;
    }
    remainderCount.set(remainder, remainderCount.get(remainder) - 1);
  }

  return nums.length;
};
```

2599 Make The Prefix Sum Non Negative

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2599. Make the Prefix Sum Non-negative
 * https://leetcode.com/problems/make-the-prefix-sum-non-negative/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. You can apply the following operation any
 * number of times:
 * - Pick any element from nums and put it at the end of nums.
 *
 * The prefix sum array of nums is an array prefix of the same length as nums such that
prefix[i]
 * is the sum of all the integers nums[j] where j is in the inclusive range [0, i].
 *
 * Return the minimum number of operations such that the prefix sum array does not contain
negative
 * integers. The test cases are generated such that it is always possible to make the prefix
sum
 * array non-negative.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var makePrefSumNonNegative = function(nums) {
  const minHeap = new PriorityQueue((a, b) => a - b);
  let prefixSum = 0;
  let result = 0;

  for (const num of nums) {
    prefixSum += num;

    if (num < 0) {
      minHeap.enqueue(num);
    }

    if (prefixSum < 0) {
      const minNegative = minHeap.dequeue();
      prefixSum -= minNegative;
      result++;
    }
  }

  return result;
};
```

2600 K Items With The Maximum Sum

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2600. K Items With the Maximum Sum
 * https://leetcode.com/problems/k-items-with-the-maximum-sum/
 * Difficulty: Easy
 *
 * There is a bag that consists of items, each item has a number 1, 0, or -1 written on it.
 *
 * You are given four non-negative integers numOnes, numZeros, numNegOnes, and k.
 *
 * The bag initially contains:
 * - numOnes items with 1s written on them.
 * - numZeroes items with 0s written on them.
 * - numNegOnes items with -1s written on them.
 *
 * We want to pick exactly k items among the available items. Return the maximum possible sum
 * of numbers written on the items.
 */

/**
 * @param {number} numOnes
 * @param {number} numZeros
 * @param {number} numNegOnes
 * @param {number} k
 * @return {number}
 */
var kItemsWithMaximumSum = function(numOnes, numZeros, numNegOnes, k) {
  let result = 0;

  if (k <= numOnes) {
    result = k;
  } else if (k <= numOnes + numZeros) {
    result = numOnes;
  } else {
    result = numOnes - (k - numOnes - numZeros);
  }

  return result;
};
```

2601 Prime Subtraction Operation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2601. Prime Subtraction Operation
 * https://leetcode.com/problems/prime-subtraction-operation/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of length n.
 *
 * You can perform the following operation as many times as you want:
 * - Pick an index i that you haven’t picked before, and pick a prime p strictly less than
 *   nums[i], then subtract p from nums[i].
 *
 * Return true if you can make nums a strictly increasing array using the above operation
 * and false otherwise.
 *
 * A strictly increasing array is an array whose each element is strictly greater than its
 * preceding element.
 */
```

```
/**
 * @param {number[]} nums
 * @return {boolean}
 */
var primeSubOperation = function(nums) {
  const primes = generatePrimes(1000);
  const adjusted = [...nums];
  let prev = 0;

  for (let i = 0; i < adjusted.length; i++) {
    const curr = adjusted[i];
    let maxPrime = 0;

    for (const prime of primes) {
      if (prime >= curr) break;
      if (curr - prime > prev && (i === adjusted.length - 1 || curr - prime < adjusted[i +
1])) {
        maxPrime = prime;
      }
    }

    adjusted[i] -= maxPrime;
    if (adjusted[i] <= prev) return false;
    prev = adjusted[i];
  }

  return true;

  function generatePrimes(limit) {
    const group = new Array(limit + 1).fill(true);
    const primes = [];
    group[0] = group[1] = false;

    for (let i = 2; i <= limit; i++) {
      if (group[i]) {
        primes.push(i);
        for (let j = i * i; j <= limit; j += i) {
          group[j] = false;
        }
      }
    }

    return primes;
  }
};
```

2604 Minimum Time To Eat All Grains

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2604. Minimum Time to Eat All Grains
 * https://leetcode.com/problems/minimum-time-to-eat-all-grains/
 * Difficulty: Hard
 *
 * There are n hens and m grains on a line. You are given the initial positions of the hens
 * and the grains in two integer arrays hens and grains of size n and m respectively.
 *
 * Any hen can eat a grain if they are on the same position. The time taken for this is
 * negligible. One hen can also eat multiple grains.
 *
 * In 1 second, a hen can move right or left by 1 unit. The hens can move simultaneously
 * and independently of each other.
 *
 * Return the minimum time to eat all grains if the hens act optimally.
 */

/**
 * @param {number[]} hens
```

```
 * @param {number[]} grains
 * @return {number}
 */
var minimumTime = function(hens, grains) {
  hens.sort((a, b) => a - b);
  grains.sort((a, b) => a - b);

  let minTime = 0;
  let maxTime = 2 * (Math.max(
    grains[grains.length - 1],
    hens[hens.length - 1]) - Math.min(grains[0], hens[0]
  ));

  while (minTime < maxTime) {
    const midTime = Math.floor((minTime + maxTime) / 2);
    if (isTimeReachable(midTime)) {
      maxTime = midTime;
    } else {
      minTime = midTime + 1;
    }
  }

  return minTime;

  function isTimeReachable(time) {
    let currentGrainIndex = 0;

    for (const currentHenPosition of hens) {
      let maximumRightReach = time;

      if (grains[currentGrainIndex] < currentHenPosition) {
        const costToReachLeftmost = currentHenPosition - grains[currentGrainIndex];
        if (costToReachLeftmost > time) return false;

        maximumRightReach = Math.max(
          0,
          time - 2 * costToReachLeftmost, Math.floor((time - costToReachLeftmost) / 2)
        );
      }

      if (currentHenPosition + maximumRightReach >= grains[currentGrainIndex]) {
        const farthestReachablePosition = currentHenPosition + maximumRightReach;
        while (currentGrainIndex < grains.length
              && grains[currentGrainIndex] <= farthestReachablePosition) {
          currentGrainIndex++;
        }
        if (currentGrainIndex === grains.length) return true;
      }
    }

    return false;
  }
};
```

2605 Form Smallest Number From Two Digit Arrays

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2605. Form Smallest Number From Two Digit Arrays
 * https://leetcode.com/problems/form-smallest-number-from-two-digit-arrays/
 * Difficulty: Easy
 *
 * Given two arrays of unique digits nums1 and nums2, return the smallest number that contains
 * at least one digit from each array.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
```

```
 * @return {number}
 */
var minNumber = function(nums1, nums2) {
  const set1 = new Set(nums1);
  const set2 = new Set(nums2);
  const min1 = Math.min(...nums1);
  const min2 = Math.min(...nums2);
  let smallestCommon = 10;

  for (const digit of set1) {
    if (set2.has(digit) && digit < smallestCommon) {
      smallestCommon = digit;
    }
  }

  if (smallestCommon !== 10) return smallestCommon;

  return min1 < min2 ? min1 * 10 + min2 : min2 * 10 + min1;
};
```

2606 Find The Substring With Maximum Cost

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2606. Find the Substring With Maximum Cost
 * https://leetcode.com/problems/find-the-substring-with-maximum-cost/
 * Difficulty: Medium
 *
 * You are given a string s, a string chars of distinct characters and an integer array vals
of
 * the same length as chars.
 *
 * The cost of the substring is the sum of the values of each character in the substring. The
 * cost of an empty string is considered 0.
 *
 * The value of the character is defined in the following way:
 * - If the character is not in the string chars, then its value is its corresponding position
 *   (1-indexed) in the alphabet.
 *   - For example, the value of 'a' is 1, the value of 'b' is 2, and so on. The value of 'z'
is 26.
 * - Otherwise, assuming i is the index where the character occurs in the string chars, then
its
 *   value is vals[i].
 *
 * Return the maximum cost among all substrings of the string s.
 */

/**
 * @param {string} s
 * @param {string} chars
 * @param {number[]} vals
 * @return {number}
 */
var maximumCostSubstring = function(s, chars, vals) {
  const charValues = new Array(26).fill().map((_, i) => i + 1);
  for (let i = 0; i < chars.length; i++) {
    charValues[chars.charCodeAt(i) - 97] = vals[i];
  }

  let result = 0;
  let currentCost = 0;
  for (const char of s) {
    currentCost = Math.max(0, currentCost + charValues[char.charCodeAt(0) - 97]);
    result = Math.max(result, currentCost);
  }

  return result;
};
```

2609 Find The Longest Balanced Substring Of A Binary String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2609. Find the Longest Balanced Substring of a Binary String
 * https://leetcode.com/problems/find-the-longest-balanced-substring-of-a-binary-string/
 * Difficulty: Easy
 *
 * You are given a binary string s consisting only of zeroes and ones.
 *
 * A substring of s is considered balanced if all zeroes are before ones and the number of
zeroes
 * is equal to the number of ones inside the substring. Notice that the empty substring is
 * considered a balanced substring.
 *
 * Return the length of the longest balanced substring of s.
 *
 * A substring is a contiguous sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var findTheLongestBalancedSubstring = function(s) {
  let result = 0;
  let zeros = 0;
  let ones = 0;

  for (let i = 0; i < s.length; i++) {
    if (s[i] === '0') {
      if (ones > 0) {
        zeros = 0;
        ones = 0;
      }
      zeros++;
    } else {
      if (zeros >= ones + 1) {
        ones++;
        result = Math.max(result, 2 * ones);
      } else {
        zeros = 0;
        ones = 0;
      }
    }
  }

  return result;
};
```

2610 Convert An Array Into A 2D Array With Conditions

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2610. Convert an Array Into a 2D Array With Conditions
 * https://leetcode.com/problems/convert-an-array-into-a-2d-array-with-conditions/
 * Difficulty: Medium
 *
 * You are given an integer array nums. You need to create a 2D array from nums satisfying
 * the following conditions:
 * - The 2D array should contain only the elements of the array nums.
```

```
 * - Each row in the 2D array contains distinct integers.
 * - The number of rows in the 2D array should be minimal.
 *
 * Return the resulting array. If there are multiple answers, return any of them.
 *
 * Note that the 2D array can have a different number of elements on each row.
 */

/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var findMatrix = function(nums) {
  const frequency = new Map();
  for (const num of nums) {
    frequency.set(num, (frequency.get(num) || 0) + 1);
  }

  const result = [];
  while (frequency.size) {
    const row = [];
    for (const [num, count] of frequency) {
      row.push(num);
      if (count === 1) {
        frequency.delete(num);
      } else {
        frequency.set(num, count - 1);
      }
    }
    result.push(row);
  }

  return result;
};
```

2613 Beautiful Pairs

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2613. Beautiful Pairs
 * https://leetcode.com/problems/beautiful-pairs/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays nums1 and nums2 of the same length. A pair
 * of indices (i,j) is called beautiful if|nums1[i] - nums1[j]| + |nums2[i] - nums2[j]|
 * is the smallest amongst all possible indices pairs where i < j.
 *
 * Return the beautiful pair. In the case that there are multiple beautiful pairs, return
 * the lexicographically smallest pair.
 *
 * Note that
 * - |x| denotes the absolute value of x.
 * - A pair of indices (i1, j1) is lexicographically smaller than (i2, j2) if i1 < i2
 *   or i1 == i2 and j1 < j2.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var beautifulPair = function(nums1, nums2) {
  const n = nums1.length;
  const max = Math.max(...nums1, ...nums2);
  const size = max + 1;

  const indices = Array.from({ length: n }, (_, i) => i)
    .sort((a, b) => nums1[a] === nums1[b] ? a - b : nums1[a] - nums1[b]);
```

```
nums1.push(-100000001);
nums2.push(100000000);

const tree1 = new Array(2 * size).fill(-1);
const tree2 = new Array(2 * size).fill(-1);
let best = null;

update(nums2[indices[0]], indices[0], 1);
update(nums2[indices[0]], indices[0], -1);

for (let i = 1; i < n; i++) {
  const index = indices[i];
  const y = nums2[index];

  const left = query(0, y, -1);
  const right = query(y, max, 1);

  best = select(index, left, right);

  update(y, index, -1);
  update(y, index, 1);
}

return best;

function compare(i, j, dir) {
  if (j === -1) return i;
  if (i === -1) return j;
  const vi = -nums1[i] + dir * nums2[i];
  const vj = -nums1[j] + dir * nums2[j];
  return vi < vj || (vi === vj && i < j) ? i : j;
}

function update(position, index, dir) {
  position += size;
  const tree = dir === 1 ? tree2 : tree1;

  while (position > 0) {
    const cur = tree[position];
    const better = compare(index, cur, dir);
    if (better !== index) return;
    tree[position] = better;
    position = Math.floor(position / 2);
  }
}

function query(l, r, dir) {
  l += size;
  r += size;
  const tree = dir === 1 ? tree2 : tree1;
  let result = -1;

  while (l <= r) {
    if (l & 1) {
      result = compare(result, tree[l], dir);
      l++;
    }
    if (!(r & 1)) {
      result = compare(result, tree[r], dir);
      r--;
    }
    l = Math.floor(l / 2);
    r = Math.floor(r / 2);
  }

  return result;
}

function select(x, y, z) {
  const pairs = [];

  if (y !== -1) {
    const d = nums1[x] + nums2[x] - nums1[y] - nums2[y];
    pairs.push([d, [Math.min(x, y), Math.max(x, y)]]);
```

```
    }

    if (z !== -1) {
      const d = nums1[x] - nums2[x] - nums1[z] + nums2[z];
      pairs.push([d, [Math.min(x, z), Math.max(x, z)]]);
    }

    if (best !== null) {
      const d = Math.abs(
        nums1[best[0]] - nums1[best[1]]) + Math.abs(nums2[best[0]] - nums2[best[1]]
      );
      pairs.push([d, best]);
    } else {
      pairs.push([Infinity, [-1, -1]]);
    }

    return pairs.sort((a, b) => {
      if (a[0] !== b[0]) return a[0] - b[0];
      if (a[1][0] !== b[1][0]) return a[1][0] - b[1][0];
      return a[1][1] - b[1][1];
    })[0][1];
  }
};
```

2615 Sum Of Distances

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2615. Sum of Distances
 * https://leetcode.com/problems/sum-of-distances/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums. There exists an array arr of length
nums.length,
 * where arr[i] is the sum of |i - j| over all j such that nums[j] == nums[i] and j != i. If
there
 * is no such j, set arr[i] to be 0.
 *
 * Return the array arr.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var distance = function(nums) {
  const valueIndices = new Map();
  const result = new Array(nums.length).fill(0);

  for (let i = 0; i < nums.length; i++) {
    if (!valueIndices.has(nums[i])) {
      valueIndices.set(nums[i], []);
    }
    valueIndices.get(nums[i]).push(i);
  }

  for (const indices of valueIndices.values()) {
    let prefixSum = 0;
    for (let i = 1; i < indices.length; i++) {
      prefixSum += indices[i] - indices[0];
    }

    result[indices[0]] = prefixSum;

    for (let i = 1; i < indices.length; i++) {
      const diff = indices[i] - indices[i - 1];
      prefixSum += diff * (i - (indices.length - i));
      result[indices[i]] = prefixSum;
    }
```

```
  }

  return result;
};
```

2616 Minimize The Maximum Difference Of Pairs

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2616. Minimize the Maximum Difference of Pairs
 * https://leetcode.com/problems/minimize-the-maximum-difference-of-pairs/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums and an integer p. Find p pairs of indices
 * of nums such that the maximum difference amongst all the pairs is minimized. Also, ensure
 * no index appears more than once amongst the p pairs.
 *
 * Note that for a pair of elements at the index i and j, the difference of this pair is
 * |nums[i] - nums[j]|, where |x| represents the absolute value of x.
 *
 * Return the minimum maximum difference among all p pairs. We define the maximum of an empty
 * set to be zero.
 */

/**
 * @param {number[]} nums
 * @param {number} p
 * @return {number}
 */
var minimizeMax = function(nums, p) {
  nums.sort((a, b) => a - b);
  const n = nums.length;

  let left = 0;
  let right = nums[n - 1] - nums[0];
  let result = 0;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (canFormPairs(mid)) {
      result = mid;
      right = mid - 1;
    } else {
      left = mid + 1;
    }
  }

  return result;

  function canFormPairs(maxDiff) {
    let pairs = 0;
    for (let i = 0; i < n - 1 && pairs < p; i += 2) {
      if (nums[i + 1] - nums[i] <= maxDiff) {
        pairs++;
      } else {
        i--;
      }
    }
    return pairs >= p;
  }
};
```

2618 Check If Object Instance Of Class

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2618. Check if Object Instance of Class
 * https://leetcode.com/problems/check-if-object-instance-of-class/
 * Difficulty: Medium
 *
 * Write a function that checks if a given value is an instance of a given class or
superclass.
 * For this problem, an object is considered an instance of a given class if that object has
 * access to that class's methods.
 *
 * There are no constraints on the data types that can be passed to the function. For example,
 * the value or the class could be undefined.
 */

/**
 * @param {*} obj
 * @param {*} classFunction
 * @return {boolean}
 */
var checkIfInstanceOf = function(obj, classFunction) {
  if (obj === null || obj === undefined || typeof classFunction !== 'function') {
    return false;
  }
  return Object(obj) instanceof classFunction;
};
```

2619 Array Prototype Last

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2619. Array Prototype Last
 * https://leetcode.com/problems/array-prototype-last/
 * Difficulty: Easy
 *
 * Write code that enhances all arrays such that you can call the array.last() method on
 * any array and it will return the last element. If there are no elements in the array,
 * it should return -1.
 *
 * You may assume the array is the output of JSON.parse.
 */

/**
 * @return {null|boolean|number|string|Array|Object}
 */
Array.prototype.last = function() {
  return !this.length ? -1 : this[this.length - 1];
};
```

2620 Counter

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2620. Counter
```

```
 * https://leetcode.com/problems/counter/
 * Difficulty: Easy
 *
 * Given an integer n, return a counter function. This counter function initially
 * returns n and then returns 1 more than the previous value every subsequent
 * time it is called (n, n + 1, n + 2, etc).
 */

/**
 * @param {number} n
 * @return {Function} counter
 */
var createCounter = function(n) {
  let count = n;
  return () => count++;
};
```

2621 Sleep

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2621. Sleep
 * https://leetcode.com/problems/sleep/
 * Difficulty: Easy
 *
 * Given a positive integer `ms`, write an asynchronous function that
 * sleeps for `ms` milliseconds. It can resolve any value.
 */

/**
 * @param {number} ms
 * @return {Promise}
 */
async function sleep(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}
```

2622 Cache With Time Limit

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2622. Cache With Time Limit
 * https://leetcode.com/problems/cache-with-time-limit/
 * Difficulty: Medium
 *
 * Write a class that allows getting and setting key-value pairs, however a time until
 * expiration is associated with each key.
 *
 * The class has three public methods:
 * - set(key, value, duration): accepts an integer key, an integer value, and a duration
 *   in milliseconds. Once the duration has elapsed, the key should be inaccessible.
 *   The method should return true if the same un-expired key already exists and false
 *   otherwise. Both the value and duration should be overwritten if the key already exists.
 *
 * - get(key): if an un-expired key exists, it should return the associated value.
 *   Otherwise it should return -1.
 *
 * - count(): returns the count of un-expired keys.
 */

var TimeLimitedCache = function() {
  this.cache = new Map();
};
```

```
/**
 * @param {number} key
 * @param {number} value
 * @param {number} duration time until expiration in ms
 * @return {boolean} if un-expired key already existed
 */
TimeLimitedCache.prototype.set = function(key, value, duration) {
  const hit = this.cache.has(key) && this.cache.get(key)[1] > Date.now();
  this.cache.set(key, [value, Date.now() + duration]);
  return hit;
};

/**
 * @param {number} key
 * @return {number} value associated with key
 */
TimeLimitedCache.prototype.get = function(key) {
  if (!this.cache.has(key)) return -1;
  const [value, expiredAt] = this.cache.get(key);
  return expiredAt > Date.now() ? value : -1;
};

/**
 * @return {number} count of non-expired keys
 */
TimeLimitedCache.prototype.count = function() {
  return [...this.cache.keys()].reduce((count, key) => {
    return this.cache.get(key)[1] > Date.now() ? ++count : count;
  }, 0);
};
```

2623 Memoize

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2623. Memoize
 * https://leetcode.com/problems/memoize/
 * Difficulty: Medium
 *
 * Given a function fn, return a memoized version of that function.
 *
 * A memoized function is a function that will never be called twice
 * with the same inputs. Instead it will return a cached value.
 *
 * You can assume there are 3 possible input functions: sum, fib,
 * and factorial.
 *
 * - `sum` accepts two integers a and b and returns a + b. Assume that
 *   if a value has already been cached for the arguments (b, a) where
 *   a != b, it cannot be used for the arguments (a, b). For example,
 *   if the arguments are (3, 2) and (2, 3), two separate calls should
 *   be made.
 * - `fib` accepts a single integer n and returns 1 if n <= 1 or
 *   fib(n - 1) + fib(n - 2) otherwise.
 * - `factorial` accepts a single integer n and returns 1 if n <= 1 or
 *   factorial(n - 1) * n otherwise.
 */

/**
 * @param {Function} fn
 * @return {Function}
 */
function memoize(fn) {
  const cache = new Map();

  return (...args) => {
    const key = String(args);
    if (cache.has(key)) {
```

```
      return cache.get(key);
    }
    const value = fn(...args);
    cache.set(key, value);
    return value;
  };
}
```

2624 Snail Traversal

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2624. Snail Traversal
 * https://leetcode.com/problems/snail-traversal/
 * Difficulty: Medium
 *
 * Write code that enhances all arrays such that you can call the snail(rowsCount,
 * colsCount) method that transforms the 1D array into a 2D array organised in the
 * pattern known as snail traversal order. Invalid input values should output an
 * empty array. If rowsCount * colsCount !== nums.length, the input is considered invalid.
 *
 * Snail traversal order starts at the top left cell with the first value of the
 * current array. It then moves through the entire first column from top to bottom,
 * followed by moving to the next column on the right and traversing it from bottom
 * to top. This pattern continues, alternating the direction of traversal with each
 * column, until the entire current array is covered. For example, when given the
 * input array [19, 10, 3, 7, 9, 8, 5, 2, 1, 17, 16, 14, 12, 18, 6, 13, 11, 20, 4, 15]
 * with rowsCount = 5 and colsCount = 4, the desired output matrix is shown below.
 * Note that iterating the matrix following the arrows corresponds to the order
 * of numbers in the original array.
 */

/**
 * @param {number} rowsCount
 * @param {number} colsCount
 * @return {Array<Array<number>>}
 */
Array.prototype.snail = function(rowsCount, colsCount) {
  if (rowsCount * colsCount !== this.length) return [];

  const result = Array.from({ length: rowsCount }, () => []);
  let index = 0;

  for (let col = 0; col < colsCount; col++) {
    if (col % 2 === 0) {
      for (let row = 0; row < rowsCount; row++) {
        result[row][col] = this[index++];
      }
    } else {
      for (let row = rowsCount - 1; row >= 0; row--) {
        result[row][col] = this[index++];
      }
    }
  }

  return result;
}
```

2625 Flatten Deeply Nested Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2625. Flatten Deeply Nested Array
```

```
 * https://leetcode.com/problems/flatten-deeply-nested-array/
 * Difficulty: Medium
 *
 * Given a multi-dimensional array arr and a depth n, return a flattened version of that
array.
 *
 * A multi-dimensional array is a recursive data structure that contains integers or other
 * multi-dimensional arrays.
 *
 * A flattened array is a version of that array with some or all of the sub-arrays removed and
 * replaced with the actual elements in that sub-array. This flattening operation should only
 * be done if the current depth of nesting is less than n. The depth of the elements in the
 * first array are considered to be 0.
 *
 * Please solve it without the built-in Array.flat method.
 */

/**
 * @param {Array} arr
 * @param {number} depth
 * @return {Array}
 */
var flat = function(arr, n) {
  if (n === 0) return arr;
  const result = [];
  for (let i = 0; i < arr.length; i++) {
    result.push(...(Array.isArray(arr[i]) ? flat(arr[i], n - 1) : [arr[i]]));
  }
  return result;
};
```

2626 Array Reduce Transformation

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2626. Array Reduce Transformation
 * https://leetcode.com/problems/array-reduce-transformation/
 * Difficulty: Easy
 *
 * Given an integer array nums, a reducer function fn, and an initial value init, return the
final
 * result obtained by executing the fn function on each element of the array, sequentially,
 * passing in the return value from the calculation on the preceding element.
 *
 * This result is achieved through the following operations: val = fn(init, nums[0]),
 * val = fn(val, nums[1]), val = fn(val, nums[2]), ... until every element in the array has
been
 * processed. The ultimate value of val is then returned.
 *
 * If the length of the array is 0, the function should return init.
 *
 * Please solve it without using the built-in Array.reduce method.
 */

/**
 * @param {number[]} nums
 * @param {Function} fn
 * @param {number} init
 * @return {number}
 */
var reduce = function(nums, fn, init) {
  let result = init;

  for (let i = 0; i < nums.length; i++) {
    result = fn(result, nums[i]);
  }

  return result;
};
```

2627 Debounce

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2627. Debounce
 * https://leetcode.com/problems/debounce/
 * Difficulty: Medium
 *
 * Given a function fn and a time in milliseconds t, return a debounced version of that
function.
 *
 * A debounced function is a function whose execution is delayed by t milliseconds and whose
 * execution is cancelled if it is called again within that window of time. The debounced
 * function should also receive the passed parameters.
 *
 * For example, let's say t = 50ms, and the function was called at 30ms, 60ms, and 100ms.
 *
 * The first 2 function calls would be cancelled, and the 3rd function call would be executed
 * at 150ms.
 *
 * If instead t = 35ms, The 1st call would be cancelled, the 2nd would be executed at 95ms,
 * and the 3rd would be executed at 135ms.
 */

/**
 * @param {Function} fn
 * @param {number} t milliseconds
 * @return {Function}
 */
var debounce = function(fn, t) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), t);
  };
};
```

2628 Json Deep Equal

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2628. JSON Deep Equal
 * https://leetcode.com/problems/json-deep-equal/
 * Difficulty: Medium
 *
 * Given two values o1 and o2, return a boolean value indicating whether two values, o1 and
o2,
 * are deeply equal.
 *
 * For two values to be deeply equal, the following conditions must be met:
 * - If both values are primitive types, they are deeply equal if they pass the === equality
check.
 * - If both values are arrays, they are deeply equal if they have the same elements in the
same
 *   order, and each element is also deeply equal according to these conditions.
 * - If both values are objects, they are deeply equal if they have the same keys, and the
 *   associated values for each key are also deeply equal according to these conditions.
 *
 * You may assume both values are the output of JSON.parse. In other words, they are valid
JSON.
 *
 * Please solve it without using lodash's _.isEqual() function
 */
```

```
/**
 * @param {null|boolean|number|string|Array|Object} o1
 * @param {null|boolean|number|string|Array|Object} o2
 * @return {boolean}
 */
var areDeeplyEqual = function(o1, o2) {
  if (o1 === o2) return true;
  if (o1 == null || o2 == null) return false;
  if (typeof o1 !== 'object' || typeof o2 !== 'object') return false;

  if (Array.isArray(o1) !== Array.isArray(o2)) return false;

  if (Array.isArray(o1)) {
    if (o1.length !== o2.length) return false;
    return o1.every((item, index) => areDeeplyEqual(item, o2[index]));
  }

  const keys1 = Object.keys(o1);
  const keys2 = Object.keys(o2);
  if (keys1.length !== keys2.length) return false;

  return keys1.every(key => keys2.includes(key) && areDeeplyEqual(o1[key], o2[key]));
};
```

2629 Function Composition

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2629. Function Composition
 * https://leetcode.com/problems/function-composition/
 * Difficulty: Easy
 *
 * Given an array of functions [f1, f2, f3, ..., fn], return a new function fn that is the
function
 * composition of the array of functions.
 *
 * The function composition of [f(x), g(x), h(x)] is fn(x) = f(g(h(x))).
 *
 * The function composition of an empty list of functions is the identity function f(x) = x.
 *
 * You may assume each function in the array accepts one integer as input and returns one
integer
 * as output.
 */

/**
 * @param {Function[]} functions
 * @return {Function}
 */
var compose = function(functions) {
  return x => functions.reduceRight((result, fn) => fn(result), x);
};
```

2630 Memoize Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2630. Memoize II
 * https://leetcode.com/problems/memoize-ii/
 * Difficulty: Hard
 *
 * Given a function fn, return a memoized version of that function.
```

```
 *
 * A memoized function is a function that will never be called twice
 * with the same inputs. Instead it will return a cached value.
 *
 * fn can be any function and there are no constraints on what type
 * of values it accepts. Inputs are considered identical if they
 * are === to each other.
 */

/**
 * @param {Function} fn
 * @return {Function}
 */
function memoize(fn) {
  const cache = {};
  const idLookup = new Map();
  function generateId(item) {
    return idLookup.get(item) ?? idLookup.set(item, idLookup.size + 1).get(item);
  }
  return (...args) => {
    const key = args.map(generateId).join('-');
    return !cache.hasOwnProperty(key) ? (cache[key] = fn(...args)) : cache[key];
  };
}
```

2631 Group By

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2631. Group By
 * https://leetcode.com/problems/group-by/
 * Difficulty: Medium
 *
 * Write code that enhances all arrays such that you can call the array.groupBy(fn) method on
any
 * array and it will return a grouped version of the array.
 *
 * A grouped array is an object where each key is the output of fn(arr[i]) and each value is
an
 * array containing all items in the original array which generate that key.
 *
 * The provided callback fn will accept an item in the array and return a string key.
 *
 * The order of each value list should be the order the items appear in the array. Any order
of
 * keys is acceptable.
 *
 * Please solve it without lodash's _.groupBy function.
 */

/**
 * @param {Function} fn
 * @return {Object}
 */
Array.prototype.groupBy = function(fn) {
  return this.reduce((grouped, item) => {
    const key = fn(item);
    grouped[key] = grouped[key] || [];
    grouped[key].push(item);
    return grouped;
  }, {});
};

/**
 * [1,2,3].groupBy(String) // {"1":[1],"2":[2],"3":[3]}
 */
```

2632 Curry

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2632. Curry
 * https://leetcode.com/problems/curry/
 * Difficulty: Medium
 *
 * Given a function fn, return a curried version of that function.
 *
 * A curried function is a function that accepts fewer or an equal number of parameters as the
 * original function and returns either another curried function or the same value the
original
 * function would have returned.
 *
 * In practical terms, if you called the original function like sum(1,2,3), you would call the
 * curried version like csum(1)(2)(3), csum(1)(2,3), csum(1,2)(3), or csum(1,2,3). All these
 * methods of calling the curried function should return the same value as the original.
 */

/**
 * @param {Function} fn
 * @return {Function}
 */
var curry = function(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...nextArgs) => curried(...args, ...nextArgs);
  };
};
```

2633 Convert Object To Json String

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2633. Convert Object to JSON String
 * https://leetcode.com/problems/convert-object-to-json-string/
 * Difficulty: Medium
 *
 * Given a value, return a valid JSON string of that value. The value can be a string, number,
 * array, object, boolean, or null. The returned string should not include extra spaces. The
 * order of keys should be the same as the order returned by Object.keys().
 *
 * Please solve it without using the built-in JSON.stringify method.
 */

/**
 * @param {null|boolean|number|string|Array|Object} object
 * @return {string}
 */
var jsonStringify = function(object) {
  if (object === null) return 'null';
  if (typeof object === 'boolean' || typeof object === 'number') return String(object);
  if (typeof object === 'string') return `"${object}"`;

  if (Array.isArray(object)) {
    const elements = object.map(item => jsonStringify(item));
    return `[${elements.join(',')}]`;
  }

  const pairs = Object.keys(object).map(key => `"${key}":${jsonStringify(object[key])}`);
```

```
  return `{${pairs.join(',')}}`;
};
```

2634 Filter Elements From Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2634. Filter Elements from Array
 * https://leetcode.com/problems/filter-elements-from-array/
 * Difficulty: Easy
 *
 * Given an integer array arr and a filtering function fn, return a filtered array
filteredArr.
 *
 * The fn function takes one or two arguments:
 * arr[i] - number from the arr
 * i - index of arr[i]
 *
 * filteredArr should only contain the elements from the arr for which the expression
 * fn(arr[i], i) evaluates to a truthy value. A truthy value is a value where
 * Boolean(value) returns true.
 *
 * Please solve it without the built-in Array.filter method.
 */

/**
 * @param {number[]} arr
 * @param {Function} fn
 * @return {number[]}
 */
var filter = function(arr, fn) {
  const result = [];

  for (let i = 0; i < arr.length; i++) {
    if (fn(arr[i], i)) {
      result.push(arr[i]);
    }
  }

  return result;
};
```

2635 Apply Transform Over Each Element In Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2635. Apply Transform Over Each Element in Array
 * https://leetcode.com/problems/apply-transform-over-each-element-in-array/
 * Difficulty: Easy
 *
 * Given an integer array arr and a mapping function fn, return a new array with a
transformation
 * applied to each element.
 *
 * The returned array should be created such that returnedArray[i] = fn(arr[i], i).
 *
 * Please solve it without the built-in Array.map method.
 */

/**
 * @param {number[]} arr
 * @param {Function} fn
 * @return {number[]}
```

```
 */
var map = function(arr, fn) {
  const result = [];

  for (let i = 0; i < arr.length; i++) {
    result.push(fn(arr[i], i));
  }

  return result;
};
```

2636 Promise Pool

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2636. Promise Pool
 * https://leetcode.com/problems/promise-pool/
 * Difficulty: Medium
 *
 * Given an array of asynchronous functions functions and a pool limit n, return an
asynchronous
 * function promisePool. It should return a promise that resolves when all the input functions
 * resolve.
 *
 * Pool limit is defined as the maximum number promises that can be pending at once.
promisePool
 * should begin execution of as many functions as possible and continue executing new
functions
 * when old promises resolve. promisePool should execute functions[i] then functions[i + 1]
then
 * functions[i + 2], etc. When the last promise resolves, promisePool should also resolve.
 *
 * For example, if n = 1, promisePool will execute one function at a time in series. However,
 * if n = 2, it first executes two functions. When either of the two functions resolve, a 3rd
 * function should be executed (if available), and so on until there are no functions left to
 * execute.
 *
 * You can assume all functions never reject. It is acceptable for promisePool to return a
promise
 * that resolves any value.
 */

/**
 * @param {Function[]} functions
 * @param {number} n
 * @return {Promise<any>}
 */
var promisePool = async function(functions, n) {
  const next = () => functions[n++]?.().then(next);
  return Promise.all(functions.slice(0, n).map(f => f().then(next)));
};
```

2637 Promise Time Limit

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2637. Promise Time Limit
 * https://leetcode.com/problems/promise-time-limit/
 * Difficulty: Medium
 *
 * Given an asynchronous function fn and a time t in milliseconds, return a new time limited
 * version of the input function. fn takes arguments provided to the time limited function.
 *
```

```
 * The time limited function should follow these rules:
 * - If the fn completes within the time limit of t milliseconds, the time limited function
 *   should resolve with the result.
 * - If the execution of the fn exceeds the time limit, the time limited function should
 *   reject with the string "Time Limit Exceeded".
 */

/**
 * @param {Function} fn
 * @param {number} t
 * @return {Function}
 */
var timeLimit = function(fn, t) {
  return async function(...args) {
    return Promise.race([
      fn(...args),
      new Promise((_, reject) => setTimeout(() => reject('Time Limit Exceeded'), t)),
    ]);
  };
};
```

2638 Count The Number Of K Free Subsets

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2638. Count the Number of K-Free Subsets
 * https://leetcode.com/problems/count-the-number-of-k-free-subsets/
 * Difficulty: Medium
 *
 * You are given an integer array nums, which contains distinct elements and an integer k.
 *
 * A subset is called a k-Free subset if it contains no two elements with an absolute
 * difference equal to k. Notice that the empty set is a k-Free subset.
 *
 * Return the number of k-Free subsets of nums.
 *
 * A subset of an array is a selection of elements (possibly none) of the array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countTheNumOfKFreeSubsets = function(nums, k) {
  nums.sort((a, b) => a - b);
  const visited = new Set();
  let result = 1;

  for (const num of nums) {
    if (visited.has(num)) continue;

    const chain = [];
    let current = num;

    while (nums.includes(current) && !visited.has(current)) {
      chain.push(current);
      visited.add(current);
      current += k;
    }

    result *= calculateChainSubsets(chain.length);
  }

  return result;

  function calculateChainSubsets(length) {
    if (length === 0) return 1;
```

```
    if (length === 1) return 2;

    let prev2 = 1;
    let prev1 = 2;

    for (let i = 2; i <= length; i++) {
      const current = prev1 + prev2;
      prev2 = prev1;
      prev1 = current;
    }

    return prev1;
  }
};
```

2639 Find The Width Of Columns Of A Grid

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2639. Find the Width of Columns of a Grid
 * https://leetcode.com/problems/find-the-width-of-columns-of-a-grid/
 * Difficulty: Easy
 *
 * You are given a 0-indexed m x n integer matrix grid. The width of a column is the
 * maximum length of its integers.
 * - For example, if grid = [[-10], [3], [12]], the width of the only column is 3
 *   since -10 is of length 3.
 *
 * Return an integer array ans of size n where ans[i] is the width of the ith column.
 *
 * The length of an integer x with len digits is equal to len if x is non-negative, and
 * len + 1 otherwise.
 */

/**
 * @param {number[][]} grid
 * @return {number[]}
 */
var findColumnWidth = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const result = new Array(cols).fill(0);

  for (let col = 0; col < cols; col++) {
    for (let row = 0; row < rows; row++) {
      const num = grid[row][col];
      result[col] = Math.max(result[col], String(num).length);
    }
  }

  return result;
};
```

2640 Find The Score Of All Prefixes Of An Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2640. Find the Score of All Prefixes of an Array
 * https://leetcode.com/problems/find-the-score-of-all-prefixes-of-an-array/
 * Difficulty: Medium
 *
 * We define the conversion array conver of an array arr as follows:
 * - conver[i] = arr[i] + max(arr[0..i]) where max(arr[0..i]) is the maximum value of
```

```
 *   arr[j] over 0 <= j <= i.
 *
 * We also define the score of an array arr as the sum of the values of the conversion array
of arr.
 *
 * Given a 0-indexed integer array nums of length n, return an array ans of length n where
ans[i]
 * is the score of the prefix nums[0..i].
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var findPrefixScore = function(nums) {
  const result = new Array(nums.length);
  let maxSoFar = 0;
  let total = 0;

  for (let i = 0; i < nums.length; i++) {
    maxSoFar = Math.max(maxSoFar, nums[i]);
    total += nums[i] + maxSoFar;
    result[i] = total;
  }

  return result;
};
```

2641 Cousins In Binary Tree Ii

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2641. Cousins in Binary Tree II
 * https://leetcode.com/problems/cousins-in-binary-tree-ii/
 * Difficulty: Medium
 *
 * Given the root of a binary tree, replace the value of each node in the tree with the
 * sum of all its cousins' values.
 *
 * Two nodes of a binary tree are cousins if they have the same depth with different parents.
 *
 * Return the root of the modified tree.
 *
 * Note that the depth of a node is the number of edges in the path from the root node to it.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {TreeNode}
 */
var replaceValueInTree = function(root) {
  const levelSums = [];

  collectSums(root, 0);
  replaceValues(root, 0, 0);

  return root;

  function collectSums(node, depth) {
    if (!node) return;
    if (levelSums.length <= depth) levelSums.push(0);
```

```
    levelSums[depth] += node.val;
    collectSums(node.left, depth + 1);
    collectSums(node.right, depth + 1);
  }

  function replaceValues(node, depth, siblingSum) {
    if (!node) return;
    node.val = levelSums[depth] - node.val - siblingSum;

    const leftVal = node.left ? node.left.val : 0;
    const rightVal = node.right ? node.right.val : 0;
    replaceValues(node.left, depth + 1, rightVal);
    replaceValues(node.right, depth + 1, leftVal);
  }
};
```

2643 Row With Maximum Ones

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2643. Row With Maximum Ones
 * https://leetcode.com/problems/row-with-maximum-ones/
 * Difficulty: Easy
 *
 * Given a m x n binary matrix mat, find the 0-indexed position of the row that contains
 * the maximum count of ones, and the number of ones in that row.
 *
 * In case there are multiple rows that have the maximum count of ones, the row with the
 * smallest row number should be selected.
 *
 * Return an array containing the index of the row, and the number of ones in it.
 */

/**
 * @param {number[][]} mat
 * @return {number[]}
 */
var rowAndMaximumOnes = function(mat) {
  let maxOnes = 0;
  let maxRow = 0;

  for (let row = 0; row < mat.length; row++) {
    const ones = mat[row].reduce((sum, val) => sum + val, 0);
    if (ones > maxOnes) {
      maxOnes = ones;
      maxRow = row;
    }
  }

  return [maxRow, maxOnes];
};
```

2644 Find The Maximum Divisibility Score

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2644. Find the Maximum Divisibility Score
 * https://leetcode.com/problems/find-the-maximum-divisibility-score/
 * Difficulty: Easy
 *
 * You are given two integer arrays nums and divisors.
 *
 * The divisibility score of divisors[i] is the number of indices j such that nums[j] is
```

```
 * divisible by divisors[i].
 *
 * Return the integer divisors[i] with the maximum divisibility score. If multiple integers
 * have the maximum score, return the smallest one.
 */

/**
 * @param {number[]} nums
 * @param {number[]} divisors
 * @return {number}
 */
var maxDivScore = function(nums, divisors) {
  let maxScore = 0;
  let result = divisors[0];

  for (const divisor of divisors) {
    let score = 0;
    for (const num of nums) {
      if (num % divisor === 0) {
        score++;
      }
    }
    if (score > maxScore || (score === maxScore && divisor < result)) {
      maxScore = score;
      result = divisor;
    }
  }

  return result;
};
```

2645 Minimum Additions To Make Valid String

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2645. Minimum Additions to Make Valid String
 * https://leetcode.com/problems/minimum-additions-to-make-valid-string/
 * Difficulty: Medium
 *
 * Given a string word to which you can insert letters "a", "b" or "c" anywhere and any number
 * of times, return the minimum number of letters that must be inserted so that word becomes
valid.
 *
 * A string is called valid if it can be formed by concatenating the string "abc" several
times.
 */

/**
 * @param {string} word
 * @return {number}
 */
var addMinimum = function(word) {
  let result = 0;
  let index = 0;

  while (index < word.length) {
    if (word[index] === 'a') {
      if (word[index + 1] === 'b' && word[index + 2] === 'c') {
        index += 3;
      } else if (word[index + 1] === 'b') {
        result += 1;
        index += 2;
      } else if (word[index + 1] === 'c') {
        result += 1;
        index += 2;
      } else {
        result += 2;
        index += 1;
      }
```

```
    } else if (word[index] === 'b') {
      if (word[index + 1] === 'c') {
        result += 1;
        index += 2;
      } else {
        result += 2;
        index += 1;
      }
    } else {
      result += 2;
      index += 1;
    }
  }

  return result;
};
```

2646 Minimize The Total Price Of The Trips

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2646. Minimize the Total Price of the Trips
 * https://leetcode.com/problems/minimize-the-total-price-of-the-trips/
 * Difficulty: Hard
 *
 * There exists an undirected and unrooted tree with n nodes indexed from 0 to n - 1. You are
 * given the integer n and a 2D integer array edges of length n - 1, where edges[i] = [ai, bi]
 * indicates that there is an edge between nodes ai and bi in the tree.
 *
 * Each node has an associated price. You are given an integer array price, where price[i] is
 * the price of the ith node.
 *
 * The price sum of a given path is the sum of the prices of all nodes lying on that path.
 *
 * Additionally, you are given a 2D integer array trips, where trips[i] = [starti, endi]
 * indicates that you start the ith trip from the node starti and travel to the node endi
 * by any path you like.
 *
 * Before performing your first trip, you can choose some non-adjacent nodes and halve the
 * prices.
 *
 * Return the minimum total price sum to perform all the given trips.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[]} price
 * @param {number[][]} trips
 * @return {number}
 */
var minimumTotalPrice = function(n, edges, price, trips) {
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  const count = new Array(n).fill(0);

  for (const [start, end] of trips) {
    dfsPath(start, end, -1, []);
  }

  return dp(0, -1, true)[1];

  function dp(node, parent, canHalve) {
    const full = count[node] * price[node];
    const half = full / 2;
```

```
    let childrenFull = 0;
    let childrenHalf = 0;

    for (const child of graph[node]) {
      if (child !== parent) {
        const [childFull, childHalf] = dp(child, node, canHalve && node !== -1);
        childrenFull += childFull;
        childrenHalf += childHalf;
      }
    }

    if (canHalve) {
      return [full + childrenHalf, Math.min(full + childrenHalf, half + childrenFull)];
    }
    return [full + childrenHalf, full + childrenHalf];
  }

  function dfsPath(start, end, parent, path) {
    path.push(start);
    if (start === end) {
      for (const node of path) count[node]++;
      return true;
    }
    for (const next of graph[start]) {
      if (next !== parent && dfsPath(next, end, start, path)) {
        return true;
      }
    }
    path.pop();
    return false;
  }
};
```

2647 Color The Triangle Red

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2647. Color the Triangle Red
 * https://leetcode.com/problems/color-the-triangle-red/
 * Difficulty: Hard
 *
 * You are given an integer n. Consider an equilateral triangle of side length n, broken up
into n2
 * unit equilateral triangles. The triangle has n 1-indexed rows where the ith row has 2i - 1
unit
 * equilateral triangles.
 *
 * The triangles in the ith row are also 1-indexed with coordinates from (i, 1) to (i, 2i -
1).
 * The following image shows a triangle of side length 4 with the indexing of its triangle.
 */

/**
 * @param {number} n
 * @return {number[][]}
 */
var colorRed = function(n) {
  const result = [[1, 1]];
  const startCycle = [1, 2, 3, 1];
  const shortCycle = [0, 1];

  for (let i = n; i > 1; i--) {
    const start = startCycle[(n - i) % 4];
    const short = shortCycle[(n - i) % 2];

    for (let j = start; j < i * 2; j += 2) {
      result.push([i, j]);
      if (short) break;
    }
```

```
  }

  return result;
};
```

2648 Generate Fibonacci Sequence

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2648. Generate Fibonacci Sequence
 * https://leetcode.com/problems/generate-fibonacci-sequence/
 * Difficulty: Easy
 *
 * Write a generator function that returns a generator object which yields the fibonacci
sequence.
 *
 * The fibonacci sequence is defined by the relation Xn = Xn-1 + Xn-2.
 *
 * The first few numbers of the series are 0, 1, 1, 2, 3, 5, 8, 13.
 */

/**
 * @return {Generator<number>}
 */
var fibGenerator = function* () {
  let current = 0;
  let next = 1;

  while (next) {
    yield current;
    [current, next] = [next, current + next];
  }
};
```

2649 Nested Array Generator

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2649. Nested Array Generator
 * https://leetcode.com/problems/nested-array-generator/
 * Difficulty: Medium
 *
 * Given a multi-dimensional array of integers, return a generator object which yields
 * integers in the same order as inorder traversal.
 *
 * A multi-dimensional array is a recursive data structure that contains both integers
 * and other multi-dimensional arrays.
 *
 * inorder traversal iterates over each array from left to right, yielding any integers
 * it encounters or applying inorder traversal to any arrays it encounters.
 */

/**
 * @param {Array} input
 * @return {Generator}
 */
var inorderTraversal = function* (input) {
  for (const item of input) {
    if (!Array.isArray(item)) yield item;
    else yield* inorderTraversal(item);
  }
};
```

2650 Design Cancellable Function

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2650. Design Cancellable Function
 * https://leetcode.com/problems/design-cancellable-function/
 * Difficulty: Hard
 *
 * Sometimes you have a long running task, and you may wish to cancel it before it completes.
 * To help with this goal, write a function cancellable that accepts a generator object and
 * returns an array of two values: a cancel function and a promise.
 *
 * You may assume the generator function will only yield promises. It is your function's
 * responsibility to pass the values resolved by the promise back to the generator. If the
 * promise rejects, your function should throw that error back to the generator.
 *
 * If the cancel callback is called before the generator is done, your function should throw
 * an error back to the generator. That error should be the string "Cancelled" (Not an Error
 * object). If the error was caught, the returned promise should resolve with the next value
 * that was yielded or returned. Otherwise, the promise should reject with the thrown error.
 * No more code should be executed.
 *
 * When the generator is done, the promise your function returned should resolve the value
 * the generator returned. If, however, the generator throws an error, the returned promise
 * should reject with the error.
 */

/**
 * @param {Generator} generator
 * @return {[Function, Promise]}
 */
var cancellable = function(generator) {
  let cancel;
  const cancelPromise = new Promise((_, reject) => {
    cancel = () => reject("Cancelled");
  });
  cancelPromise.catch(() => {});
  const promise = (async () => {
    let next = generator.next();
    while (!next.done) {
      try {
        next = generator.next(await Promise.race([next.value, cancelPromise]));
      } catch (e) {
        next = generator.throw(e);
      }
    }
    return next.value;
  })();
  return [cancel, promise];
};
```

2651 Calculate Delayed Arrival Time

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2651. Calculate Delayed Arrival Time
 * https://leetcode.com/problems/calculate-delayed-arrival-time/
 * Difficulty: Easy
 *
 * You are given a positive integer arrivalTime denoting the arrival time of a train in
 * hours, and another positive integer delayedTime denoting the amount of delay in hours.
 *
 * Return the time when the train will arrive at the station.
```

```
 *
 * Note that the time in this problem is in 24-hours format.
 */

/**
 * @param {number} arrivalTime
 * @param {number} delayedTime
 * @return {number}
 */
var findDelayedArrivalTime = function(arrivalTime, delayedTime) {
  return (arrivalTime + delayedTime) % 24;
};
```

2652 Sum Multiples

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2652. Sum Multiples
 * https://leetcode.com/problems/sum-multiples/
 * Difficulty: Easy
 *
 * Given a positive integer n, find the sum of all integers in the range [1, n] inclusive
 * that are divisible by 3, 5, or 7.
 *
 * Return an integer denoting the sum of all numbers in the given range satisfying the
constraint.
 */

/**
 * @param {number} n
 * @return {number}
 */
var sumOfMultiples = function(n) {
  return sumDivisibleBy(3) + sumDivisibleBy(5) + sumDivisibleBy(7)
    - sumDivisibleBy(15) - sumDivisibleBy(21) - sumDivisibleBy(35)
    + sumDivisibleBy(105);

  function sumDivisibleBy(divisor) {
    const count = Math.floor(n / divisor);
    return divisor * count * (count + 1) / 2;
  }
};
```

2655 Find Maximal Uncovered Ranges

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2655. Find Maximal Uncovered Ranges
 * https://leetcode.com/problems/find-maximal-uncovered-ranges/
 * Difficulty: Medium
 *
 * You are given an integer n which is the length of a 0-indexed array nums, and a 0-indexed
 * 2D-array ranges, which is a list of sub-ranges of nums (sub-ranges may overlap).
 *
 * Each row ranges[i] has exactly 2 cells:
 * - ranges[i][0], which shows the start of the ith range (inclusive)
 * - ranges[i][1], which shows the end of the ith range (inclusive)
 *
 * These ranges cover some cells of nums and leave some cells uncovered. Your task is to find
 * all of the uncovered ranges with maximal length.
 *
 * Return a 2D-array answer of the uncovered ranges, sorted by the starting point in ascending
 * order.
```

```
 *
 * By all of the uncovered ranges with maximal length, we mean satisfying two conditions:
 * - Each uncovered cell should belong to exactly one sub-range
 * - There should not exist two ranges (l1, r1) and (l2, r2) such that r1 + 1 = l2
 */

/**
 * @param {number} n
 * @param {number[][]} ranges
 * @return {number[][]}
 */
var findMaximalUncoveredRanges = function(n, ranges) {
  if (ranges.length === 0) return [[0, n - 1]];

  ranges.sort((a, b) => a[0] - b[0]);

  const mergedRanges = [];
  let currentStart = ranges[0][0];
  let currentEnd = ranges[0][1];
  for (let i = 1; i < ranges.length; i++) {
    const [start, end] = ranges[i];
    if (start <= currentEnd + 1) {
      currentEnd = Math.max(currentEnd, end);
    } else {
      mergedRanges.push([currentStart, currentEnd]);
      currentStart = start;
      currentEnd = end;
    }
  }
  mergedRanges.push([currentStart, currentEnd]);

  const result = [];

  if (mergedRanges[0][0] > 0) {
    result.push([0, mergedRanges[0][0] - 1]);
  }

  for (let i = 0; i < mergedRanges.length - 1; i++) {
    const gapStart = mergedRanges[i][1] + 1;
    const gapEnd = mergedRanges[i + 1][0] - 1;
    if (gapStart <= gapEnd) {
      result.push([gapStart, gapEnd]);
    }
  }

  if (mergedRanges[mergedRanges.length - 1][1] < n - 1) {
    result.push([mergedRanges[mergedRanges.length - 1][1] + 1, n - 1]);
  }

  return result;
};
```

2656 Maximum Sum With Exactly K Elements

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2656. Maximum Sum With Exactly K Elements
 * https://leetcode.com/problems/maximum-sum-with-exactly-k-elements/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums and an integer k. Your task is to perform
 * the following operation exactly k times in order to maximize your score:
 * 1. Select an element m from nums.
 * 2. Remove the selected element m from the array.
 * 3. Add a new element with a value of m + 1 to the array.
 * 4. Increase your score by m.
 *
 * Return the maximum score you can achieve after performing the operation exactly k times.
 */
```

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximizeSum = function(nums, k) {
  return k * Math.max(...nums) + k * (k - 1) / 2;
};
```

2657 Find The Prefix Common Array Of Two Arrays

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2657. Find the Prefix Common Array of Two Arrays
 * https://leetcode.com/problems/find-the-prefix-common-array-of-two-arrays/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer permutations A and B of length n.
 *
 * A prefix common array of A and B is an array C such that C[i] is equal to the count of
 * numbers that are present at or before the index i in both A and B.
 *
 * Return the prefix common array of A and B.
 *
 * A sequence of n integers is called a permutation if it contains all integers from 1 to
 * n exactly once.
 */

/**
 * @param {number[]} A
 * @param {number[]} B
 * @return {number[]}
 */
var findThePrefixCommonArray = function(A, B) {
  const result = [];

  for (let i = 0, count = 0, set = new Set(); i < A.length; i++) {
    if (set.has(A[i])) count++;
    if (set.has(B[i])) count++;
    if (A[i] === B[i]) count++;
    [A[i], B[i]].forEach(n => set.add(n));
    result.push(count);
  }

  return result;
};
```

2658 Maximum Number Of Fish In A Grid

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2658. Maximum Number of Fish in a Grid
 * https://leetcode.com/problems/maximum-number-of-fish-in-a-grid/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D matrix grid of size m x n, where (r, c) represents:
 * - A land cell if grid[r][c] = 0, or
 * - A water cell containing grid[r][c] fish, if grid[r][c] > 0.
 *
 * A fisher can start at any water cell (r, c) and can do the following operations
 * any number of times:
 * - Catch all the fish at cell (r, c), or
```

```
 * - Move to any adjacent water cell.
 *
 * Return the maximum number of fish the fisher can catch if he chooses his starting
 * cell optimally, or 0 if no water cell exists.
 *
 * An adjacent cell of the cell (r, c), is one of the cells (r, c + 1), (r, c - 1),
 * (r + 1, c) or (r - 1, c) if it exists.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var findMaxFish = function(grid) {
  function traverse(i, j) {
    if (i >= grid.length || i < 0 || j >= grid[i].length || j < 0 || grid[i][j] === 0) {
      return 0;
    }
    let count = grid[i][j];
    grid[i][j] = 0;

    count += traverse(i + 1, j);
    count += traverse(i, j + 1);
    count += traverse(i - 1, j);
    count += traverse(i, j - 1);

    return count;
  }

  return grid.reduce((result, r, i) => {
    return r.reduce((result, c, j) => r[j] > 0 ? Math.max(result, traverse(i, j)) : result,
result);
  }, 0);
};
```

2661 First Completely Painted Row Or Column

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2661. First Completely Painted Row or Column
 * https://leetcode.com/problems/first-completely-painted-row-or-column/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array arr, and an m x n integer matrix mat. arr and mat
 * both contain all the integers in the range [1, m * n].
 *
 * Go through each index i in arr starting from index 0 and paint the cell in mat containing
 * the integer arr[i].
 *
 * Return the smallest index i at which either a row or a column will be completely painted
 * in mat.
 */

/**
 * @param {number[]} arr
 * @param {number[][]} mat
 * @return {number}
 */
var firstCompleteIndex = function(arr, mat) {
  const rows = new Array(mat.length).fill(0);
  const columns = new Array(mat[0].length).fill(0);
  const map = new Map();

  for (let i = 0; i < mat.length; i++) {
    for (let j = 0; j < mat[i].length; j++) {
      map.set(mat[i][j], [i, j]);
    }
  }
```

```
  for (let i = 0; i < arr.length; i++) {
    const [rowIndex, columnIndex] = map.get(arr[i]);
    if (++rows[rowIndex] === mat[0].length || ++columns[columnIndex] === mat.length) {
      return i;
    }
  }
};
```

2664 The Knights Tour

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2664. The Knight’s Tour
 * https://leetcode.com/problems/the-knights-tour/
 * Difficulty: Medium
 *
 * Given two positive integers m and n which are the height and width of a 0-indexed 2D-array
 * board, a pair of positive integers (r, c) which is the starting position of the knight on
 * the board.
 *
 * Your task is to find an order of movements for the knight, in a manner that every cell of
 * the board gets visited exactly once (the starting cell is considered visited and you
 * shouldn't visit it again).
 *
 * Return the array board in which the cells' values show the order of visiting the cell
 * starting from 0 (the initial place of the knight).
 *
 * Note that a knight can move from cell (r1, c1) to cell (r2, c2) if 0 <= r2 <= m - 1 and
 * 0 <= c2 <= n - 1 and min(abs(r1 - r2), abs(c1 - c2)) = 1 and max(abs(r1 - r2), abs(c1 -
c2)) = 2.
 */

/**
 * @param {number} m
 * @param {number} n
 * @param {number} r
 * @param {number} c
 * @return {number[][]}
 */
var tourOfKnight = function(m, n, r, c) {
  const board = new Array(m).fill().map(() => new Array(n).fill(-1));
  const moves = [[-2, -1], [-2, 1], [-1, -2], [-1, 2], [1, -2], [1, 2], [2, -1], [2, 1]];

  backtrack(r, c, 0);
  return board;

  function isValid(row, col) {
    return row >= 0 && row < m && col >= 0 && col < n && board[row][col] === -1;
  }

  function getDegree(row, col) {
    let count = 0;
    for (const [dr, dc] of moves) {
      if (isValid(row + dr, col + dc)) count++;
    }
    return count;
  }

  function backtrack(row, col, moveCount) {
    board[row][col] = moveCount;

    if (moveCount === m * n - 1) return true;

    const nextMoves = [];
    for (const [dr, dc] of moves) {
      const newRow = row + dr;
      const newCol = col + dc;
      if (isValid(newRow, newCol)) {
        nextMoves.push([newRow, newCol, getDegree(newRow, newCol)]);
```

```
      }
    }

    nextMoves.sort((a, b) => a[2] - b[2]);

    for (const [newRow, newCol] of nextMoves) {
      if (backtrack(newRow, newCol, moveCount + 1)) return true;
    }

    board[row][col] = -1;
    return false;
  }
};
```

2665 Counter Ii

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2665. Counter II
 * https://leetcode.com/problems/counter-ii/
 * Difficulty: Easy
 *
 * Write a function createCounter. It should accept an initial integer init.
 * It should return an object with three functions.
 *
 * The three functions are:
 * - increment() increases the current value by 1 and then returns it.
 * - decrement() reduces the current value by 1 and then returns it.
 * - reset() sets the current value to init and then returns it.
 */

/**
 * @param {integer} init
 * @return { increment: Function, decrement: Function, reset: Function }
 */
var createCounter = function(init) {
  let current = init;
  return {
    increment: () => ++current,
    decrement: () => --current,
    reset: () => current = init,
  };
};
```

2666 Allow One Function Call

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2666. Allow One Function Call
 * https://leetcode.com/problems/allow-one-function-call/
 * Difficulty: Easy
 *
 * Given a function fn, return a new function that is identical to the original function
except
 * that it ensures fn is called at most once.
 *
 * - The first time the returned function is called, it should return the same result as fn.
 * - Every subsequent time it is called, it should return undefined.
 */

/**
 * @param {Function} fn
 * @return {Function}
```

```
 */
var once = function(fn) {
  return (...args) => fn && [fn(...args), fn = undefined][0];
};
```

2667 Create Hello World Function

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2667. Create Hello World Function
 * https://leetcode.com/problems/create-hello-world-function/
 * Difficulty: Easy
 *
 * Write a function createHelloWorld. It should return a new function that
 * always returns "Hello World".
 */

/**
 * @return {Function}
 */
var createHelloWorld = function() {
  return () => 'Hello World';
};
```

2670 Find The Distinct Difference Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2670. Find the Distinct Difference Array
 * https://leetcode.com/problems/find-the-distinct-difference-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array nums of length n.
 *
 * The distinct difference array of nums is an array diff of length n such that diff[i] is
 * equal to the number of distinct elements in the suffix nums[i + 1, ..., n - 1] subtracted
 * from the number of distinct elements in the prefix nums[0, ..., i].
 *
 * Return the distinct difference array of nums.
 *
 * Note that nums[i, ..., j] denotes the subarray of nums starting at index i and ending at
 * index j inclusive. Particularly, if i > j then nums[i, ..., j] denotes an empty subarray.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var distinctDifferenceArray = function(nums) {
  const n = nums.length;
  const result = new Array(n);
  const prefixSet = new Set();
  const suffixCount = new Map();

  for (const num of nums) {
    suffixCount.set(num, (suffixCount.get(num) || 0) + 1);
  }

  let suffixDistinct = suffixCount.size;

  for (let i = 0; i < n; i++) {
    prefixSet.add(nums[i]);
    const count = suffixCount.get(nums[i]);
```

```
    suffixCount.set(nums[i], count - 1);
    if (count === 1) {
      suffixCount.delete(nums[i]);
      suffixDistinct--;
    }
    result[i] = prefixSet.size - suffixDistinct;
  }

  return result;
};
```

2672 Number Of Adjacent Elements With The Same Color

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2672. Number of Adjacent Elements With the Same Color
 * https://leetcode.com/problems/number-of-adjacent-elements-with-the-same-color/
 * Difficulty: Medium
 *
 * You are given an integer n representing an array colors of length n where all elements
 * are set to 0's meaning uncolored. You are also given a 2D integer array queries where
 * queries[i] = [indexi, colori]. For the ith query:
 * - Set colors[indexi] to colori.
 * - Count the number of adjacent pairs in colors which have the same color (regardless of
colori).
 *
 * Return an array answer of the same length as queries where answer[i] is the answer to the
ith
 * query.
 */

/**
 * @param {number} n
 * @param {number[][]} queries
 * @return {number[]}
 */
var colorTheArray = function(n, queries) {
  const colors = new Array(n).fill(0);
  const result = new Array(queries.length).fill(0);
  let sameColorPairs = 0;

  for (let i = 0; i < queries.length; i++) {
    const [index, color] = queries[i];
    const prevColor = colors[index];

    if (prevColor !== 0) {
      if (index > 0 && colors[index - 1] === prevColor) sameColorPairs--;
      if (index < n - 1 && colors[index + 1] === prevColor) sameColorPairs--;
    }

    colors[index] = color;

    if (index > 0 && colors[index - 1] === color) sameColorPairs++;
    if (index < n - 1 && colors[index + 1] === color) sameColorPairs++;

    result[i] = sameColorPairs;
  }

  return result;
};
```

2673 Make Costs Of Paths Equal In A Binary Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2673. Make Costs of Paths Equal in a Binary Tree
 * https://leetcode.com/problems/make-costs-of-paths-equal-in-a-binary-tree/
 * Difficulty: Medium
 *
 * You are given an integer n representing the number of nodes in a perfect binary tree
consisting
 * of nodes numbered from 1 to n. The root of the tree is node 1 and each node i in the tree
has
 * two children where the left child is the node 2 * i and the right child is 2 * i + 1.
 *
 * Each node in the tree also has a cost represented by a given 0-indexed integer array cost
of
 * size n where cost[i] is the cost of node i + 1. You are allowed to increment the cost of
any
 * node by 1 any number of times.
 *
 * Return the minimum number of increments you need to make the cost of paths from the root to
 * each leaf node equal.
 *
 * Note:
 * - A perfect binary tree is a tree where each node, except the leaf nodes, has exactly 2
children.
 * - The cost of a path is the sum of costs of nodes in the path.
 */

/**
 * @param {number} n
 * @param {number[]} cost
 * @return {number}
 */
var minIncrements = function(n, cost) {
  let result = 0;

  for (let i = Math.floor(n / 2) - 1; i >= 0; i--) {
    const left = 2 * i + 1;
    const right = 2 * i + 2;
    if (left < n && right < n) {
      const maxChildCost = Math.max(cost[left], cost[right]);
      result += maxChildCost - cost[left] + maxChildCost - cost[right];
      cost[i] += maxChildCost;
    }
  }

  return result;
};
```

2674 Split A Circular Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2674. Split a Circular Linked List
 * https://leetcode.com/problems/split-a-circular-linked-list/
 * Difficulty: Medium
 *
 * Given a circular linked list list of positive integers, your task is to split it into 2
circular
 * linked lists so that the first one contains the first half of the nodes in list (exactly
 * ceil(list.length / 2) nodes) in the same order they appeared in list, and the second one
contains
 * the rest of the nodes in list in the same order they appeared in list.
 *
 * Return an array answer of length 2 in which the first element is a circular linked list
 * representing the first half and the second element is a circular linked list representing
the
 * second half.
 *
 * A circular linked list is a normal linked list with the only difference being that the last
```

```
 * node's next node, is the first node.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} list
 * @return {ListNode[]}
 */
var splitCircularLinkedList = function(list) {
  let length = 0;
  let current = list;

  do {
    length++;
    current = current.next;
  } while (current !== list);

  const firstHalfLength = Math.ceil(length / 2);

  let firstHalfEnd = list;
  for (let i = 0; i < firstHalfLength - 1; i++) {
    firstHalfEnd = firstHalfEnd.next;
  }

  const secondHalfStart = firstHalfEnd.next;

  let secondHalfEnd = secondHalfStart;
  for (let i = 0; i < length - firstHalfLength - 1; i++) {
    secondHalfEnd = secondHalfEnd.next;
  }

  firstHalfEnd.next = list;
  secondHalfEnd.next = secondHalfStart;

  return [list, secondHalfStart];
};
```

2675 Array Of Objects To Matrix

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2675. Array of Objects to Matrix
 * https://leetcode.com/problems/array-of-objects-to-matrix/
 * Difficulty: Hard
 *
 * Write a function that converts an array of objects arr into a matrix m.
 *
 * arr is an array of objects or arrays. Each item in the array can be deeply nested with
child
 * arrays and child objects. It can also contain numbers, strings, booleans, and null values.
 *
 * The first row m should be the column names. If there is no nesting, the column names are
the
 * unique keys within the objects. If there is nesting, the column names are the respective
paths
 * in the object separated by ".".
 *
 * Each of the remaining rows corresponds to an object in arr. Each value in the matrix
corresponds
 * to a value in an object. If a given object doesn't contain a value for a given column, the
cell
 * should contain an empty string "".
```

```
 *
 * The columns in the matrix should be in lexographically ascending order.
 */

/**
 * @param {Array} arr
 * @return {(string | number | boolean | null)[][]}
 */
var jsonToMatrix = function(arr) {
  const allKeys = Array.from(
    arr.reduce((keySet, item) => {
      extractKeys(item).forEach(key => keySet.add(key));
      return keySet;
    }, new Set())
  ).sort();

  return [allKeys, ...arr.map(item => allKeys.map(key => getValue(item, key)))];

  function isObject(value) {
    return value !== null && typeof value === 'object';
  }

  function extractKeys(object) {
    if (!isObject(object)) return [''];
    const keys = [];
    for (const key of Object.keys(object)) {
      const childKeys = extractKeys(object[key]);
      for (const childKey of childKeys) {
        keys.push(childKey ? `${key}.${childKey}` : key);
      }
    }
    return keys;
  }

  function getValue(obj, path) {
    const segments = path.split('.');
    let current = obj;
    let index = 0;

    while (index < segments.length && isObject(current)) {
      current = current[segments[index++]];
    }

    return index < segments.length || isObject(current) || current === undefined ? '' :
current;
  }
};
```

2676 Throttle

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2676. Throttle
 * https://leetcode.com/problems/throttle/
 * Difficulty: Medium
 *
 * Given a function fn and a time in milliseconds t, return a throttled version of that
 * function.
 *
 * A throttled function is first called without delay and then, for a time interval of
 * t milliseconds, can't be executed but should store the latest function arguments provided
 * to call fn with them after the end of the delay.
 *
 * For instance, t = 50ms, and the function was called at 30ms, 40ms, and 60ms.
 *
 * At 30ms, without delay, the throttled function fn should be called with the arguments,
 * and calling the throttled function fn should be blocked for the following t milliseconds.
 *
 * At 40ms, the function should just save arguments.
```

```
 *
 * At 60ms, arguments should overwrite currently stored arguments from the second call because
 * the second and third calls are made before 80ms. Once the delay has passed, the throttled
 * function fn should be called with the latest arguments provided during the delay period,
 * and it should also create another delay period of 80ms + t.
 */

/**
 * @param {Function} fn
 * @param {number} t
 * @return {Function}
 */
var throttle = function(fn, t) {
  let isThrottled = false;
  let nextArgs = null;

  function invoke() {
    if (nextArgs) {
      fn(...nextArgs);
      nextArgs = null;
      setTimeout(invoke, t);
    } else {
      isThrottled = false;
    }
  }

  return function(...args) {
    if (isThrottled) {
      nextArgs = args;
      return;
    }

    fn(...args);
    isThrottled = true;
    setTimeout(invoke, t);
  };
};
```

2677 Chunk Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2677. Chunk Array
 * https://leetcode.com/problems/chunk-array/
 * Difficulty: Easy
 *
 * Given an array arr and a chunk size size, return a chunked array.
 *
 * A chunked array contains the original elements in arr, but consists of subarrays each of
 * length size. The length of the last subarray may be less than size if arr.length is not
 * evenly divisible by size.
 *
 * You may assume the array is the output of JSON.parse. In other words, it is valid JSON.
 *
 * Please solve it without using lodash's _.chunk function.
 */

/**
 * @param {Array} arr
 * @param {number} size
 * @return {Array}
 */
var chunk = function(arr, size) {
  var result = [];

  for (let index = 0; index < arr.length;) {
    result.push(arr.slice(index, index + size));
    index += size;
  }
```

```
  return result;
};
```

2678 Number Of Senior Citizens

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2678. Number of Senior Citizens
 * https://leetcode.com/problems/number-of-senior-citizens/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array of strings details. Each element of details provides
 * information about a given passenger compressed into a string of length 15. The system
 * is such that:
 * - The first ten characters consist of the phone number of passengers.
 * - The next character denotes the gender of the person.
 * - The following two characters are used to indicate the age of the person.
 * - The last two characters determine the seat allotted to that person.
 *
 * Return the number of passengers who are strictly more than 60 years old.
 */

/**
 * @param {string[]} details
 * @return {number}
 */
var countSeniors = function(details) {
  let result = 0;

  for (const detail of details) {
    const age = parseInt(detail.slice(11, 13));
    if (age > 60) result++;
  }

  return result;
};
```

2680 Maximum Or

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2680. Maximum OR
 * https://leetcode.com/problems/maximum-or/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of length n and an integer k. In an operation,
 * you can choose an element and multiply it by 2.
 *
 * Return the maximum possible value of nums[0] | nums[1] | ... | nums[n - 1] that can be
obtained
 * after applying the operation on nums at most k times.
 *
 * Note that a | b denotes the bitwise or between two integers a and b.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumOr = function(nums, k) {
  const n = nums.length;
```

```
  const prefixOr = new Array(n + 1).fill(0n);
  const suffixOr = new Array(n + 1).fill(0n);

  for (let i = 0; i < n; i++) {
    prefixOr[i + 1] = prefixOr[i] | BigInt(nums[i]);
  }

  for (let i = n - 1; i >= 0; i--) {
    suffixOr[i] = suffixOr[i + 1] | BigInt(nums[i]);
  }

  let maxOr = 0n;
  for (let i = 0; i < n; i++) {
    maxOr = maxOr > (prefixOr[i] | (BigInt(nums[i]) << BigInt(k)) | suffixOr[i + 1])
      ? maxOr
      : (prefixOr[i] | (BigInt(nums[i]) << BigInt(k)) | suffixOr[i + 1]);
  }

  return Number(maxOr);
};
```

2682 Find The Losers Of The Circular Game

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2682. Find the Losers of the Circular Game
 * https://leetcode.com/problems/find-the-losers-of-the-circular-game/
 * Difficulty: Easy
 *
 * There are n friends that are playing a game. The friends are sitting in a circle and are
numbered
 * from 1 to n in clockwise order. More formally, moving clockwise from the ith friend brings
you to
 * the (i+1)th friend for 1 <= i < n, and moving clockwise from the nth friend brings you to
the 1st
 * friend.
 *
 * The rules of the game are as follows:
 *
 * 1st friend receives the ball.
 * - After that, 1st friend passes it to the friend who is k steps away from them in the
clockwise
 *   direction.
 * - After that, the friend who receives the ball should pass it to the friend who is 2 * k
steps
 *   away from them in the clockwise direction.
 * - After that, the friend who receives the ball should pass it to the friend who is 3 * k
steps
 *   away from them in the clockwise direction, and so on and so forth.
 *
 * In other words, on the ith turn, the friend holding the ball should pass it to the friend
who
 * is i * k steps away from them in the clockwise direction.
 *
 * The game is finished when some friend receives the ball for the second time.
 *
 * The losers of the game are friends who did not receive the ball in the entire game.
 *
 * Given the number of friends, n, and an integer k, return the array answer, which contains
the
 * losers of the game in the ascending order.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number[]}
 */
```

```
var circularGameLosers = function(n, k) {
  const visited = new Set();
  let current = 0;
  let step = 1;

  while (!visited.has(current)) {
    visited.add(current);
    current = (current + step * k) % n;
    step++;
  }

  const result = [];
  for (let i = 0; i < n; i++) {
    if (!visited.has(i)) {
      result.push(i + 1);
    }
  }

  return result;
};
```

2683 Neighboring Bitwise Xor

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2683. Neighboring Bitwise XOR
 * https://leetcode.com/problems/neighboring-bitwise-xor/
 * Difficulty: Medium
 *
 * A 0-indexed array derived with length n is derived by computing the bitwise XOR (⊕)
 * of adjacent values in a binary array original of length n.
 *
 * Specifically, for each index i in the range [0, n - 1]:
 * - If i = n - 1, then derived[i] = original[i] ⊕ original[0].
 * - Otherwise, derived[i] = original[i] ⊕ original[i + 1].
 *
 * Given an array derived, your task is to determine whether there exists a valid binary
 * array original that could have formed derived.
 *
 * Return true if such an array exists or false otherwise.
 * - A binary array is an array containing only 0's and 1's
 */

/**
 * @param {number[]} derived
 * @return {boolean}
 */
var doesValidArrayExist = function(derived) {
  return derived.reduce((xor, n) => xor ^ n, 0) === 0;
};
```

2684 Maximum Number Of Moves In A Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2684. Maximum Number of Moves in a Grid
 * https://leetcode.com/problems/maximum-number-of-moves-in-a-grid/
 * Difficulty: Medium
 *
 * You are given a 0-indexed m x n matrix grid consisting of positive integers.
 *
 * You can start at any cell in the first column of the matrix, and traverse the grid in the
 * following way:
```

```
 * - From a cell (row, col), you can move to any of the cells: (row - 1, col + 1), (row, col +
 1)
 *   and (row + 1, col + 1) such that the value of the cell you move to, should be strictly
bigger
 *   than the value of the current cell.
 *
 * Return the maximum number of moves that you can perform.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var maxMoves = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const memo = Array.from({ length: rows }, () => new Array(cols).fill(-1));

  let result = 0;
  for (let row = 0; row < rows; row++) {
    result = Math.max(result, explore(row, 0));
  }

  return result;

  function explore(row, col) {
    if (col === cols - 1) return 0;
    if (memo[row][col] !== -1) return memo[row][col];

    let maxSteps = 0;
    const current = grid[row][col];

    if (row > 0 && col + 1 < cols && grid[row - 1][col + 1] > current) {
      maxSteps = Math.max(maxSteps, 1 + explore(row - 1, col + 1));
    }
    if (col + 1 < cols && grid[row][col + 1] > current) {
      maxSteps = Math.max(maxSteps, 1 + explore(row, col + 1));
    }
    if (row + 1 < rows && col + 1 < cols && grid[row + 1][col + 1] > current) {
      maxSteps = Math.max(maxSteps, 1 + explore(row + 1, col + 1));
    }

    memo[row][col] = maxSteps;
    return maxSteps;
  }
};
```

2685 Count The Number Of Complete Components

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2685. Count the Number of Complete Components
 * https://leetcode.com/problems/count-the-number-of-complete-components/
 * Difficulty: Medium
 *
 * You are given an integer n. There is an undirected graph with n vertices, numbered from 0
 * to n - 1. You are given a 2D integer array edges where edges[i] = [ai, bi] denotes that
 * there exists an undirected edge connecting vertices ai and bi.
 *
 * Return the number of complete connected components of the graph.
 *
 * A connected component is a subgraph of a graph in which there exists a path between any
 * two vertices, and no vertex of the subgraph shares an edge with a vertex outside of the
 * subgraph.
 *
 * A connected component is said to be complete if there exists an edge between every pair of
 * its vertices.
 */
```

```
/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var countCompleteComponents = function(n, edges) {
  const adjacencyList = Array.from({ length: n }, () => new Set());
  edges.forEach(([a, b]) => {
    adjacencyList[a].add(b);
    adjacencyList[b].add(a);
  });

  const visited = new Set();
  let result = 0;

  function exploreComponent(vertex) {
    const component = new Set([vertex]);
    const queue = [vertex];
    visited.add(vertex);

    while (queue.length) {
      const current = queue.shift();
      adjacencyList[current].forEach(neighbor => {
        if (!visited.has(neighbor)) {
          component.add(neighbor);
          queue.push(neighbor);
          visited.add(neighbor);
        }
      });
    }
    return component;
  }

  function isComplete(component) {
    const size = component.size;
    return [...component].every(vertex =>
      adjacencyList[vertex].size === size - 1
        && [...adjacencyList[vertex]].every(neighbor => component.has(neighbor))
    );
  }

  for (let vertex = 0; vertex < n; vertex++) {
    if (!visited.has(vertex)) {
      const component = exploreComponent(vertex);
      if (isComplete(component)) result++;
    }
  }

  return result;
};
```

2689 Extract Kth Character From The Rope Tree

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2689. Extract Kth Character From The Rope Tree
 * https://leetcode.com/problems/extract-kth-character-from-the-rope-tree/
 * Difficulty: Easy
 *
 * You are given the root of a binary tree and an integer k. Besides the left and right
children,
 * every node of this tree has two other properties, a string node.val containing only
lowercase
 * English letters (possibly empty) and a non-negative integer node.len. There are two types
of
 * nodes in this tree:
 * - Leaf: These nodes have no children, node.len = 0, and node.val is some non-empty string.
 * - Internal: These nodes have at least one child (also at most two children), node.len > 0,
```

```
 *   and node.val is an empty string.
 *
 * The tree described above is called a Rope binary tree. Now we define S[node] recursively
 * as follows:
 * - If node is some leaf node, S[node] = node.val,
 * - Otherwise if node is some internal node, S[node] = concat(S[node.left], S[node.right])
 *   and S[node].length = node.len.
 *
 * Return k-th character of the string S[root].
 *
 * Note: If s and p are two strings, concat(s, p) is a string obtained by concatenating p to
s.
 * For example, concat("ab", "zz") = "abzz".
 */

/**
 * Definition for a rope tree node.
 * class RopeTreeNode {
 *     constructor(len, val, left, right) {
 *         this.len = (len===undefined ? 0 : len);
 *         this.val = (val===undefined ? "" : val);
 *         this.left = (left===undefined ? null : left);
 *         this.right = (right===undefined ? null : right);
 *     }
 * }
 */
/**
 * @param {RopeTreeNode} root
 * @param {number} k
 * @return {character}
 */
var getKthCharacter = function(root, k) {
  if (!root.left && !root.right) {
    return root.val[k - 1];
  }

  const leftLength = getSubtreeLength(root.left);

  if (k <= leftLength) {
    return getKthCharacter(root.left, k);
  } else {
    return getKthCharacter(root.right, k - leftLength);
  }

  function getSubtreeLength(node) {
    if (!node) return 0;
    if (!node.left && !node.right) {
      return node.val.length;
    }
    return node.len;
  }
};
```

2690 Infinite Method Object

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2690. Infinite Method Object
 * https://leetcode.com/problems/infinite-method-object/
 * Difficulty: Easy
 *
 * Write a function that returns an infinite-method object.
 *
 * An infinite-method object is defined as an object that allows you to call any method and
 * it will always return the name of the method.
 *
 * For example, if you execute obj.abc123(), it will return "abc123".
 */
```

```
/**
 * @return {Object}
 */
var createInfiniteObject = function() {
  return new Proxy({}, {
    get(target, property) {
      return () => property;
    }
  });
};
```

2691 Immutability Helper

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2691. Immutability Helper
 * https://leetcode.com/problems/immutability-helper/
 * Difficulty: Hard
 *
 * Creating clones of immutable objects with minor alterations can be a tedious process.
 * Write a class ImmutableHelper that serves as a tool to help with this requirement.
 * The constructor accepts an immutable object obj which will be a JSON object or array.
 *
 * The class has a single method produce which accepts a function mutator. The function
 * returns a new object which is similar to the original except it has those mutations
applied.
 *
 * mutator accepts a proxied version of obj. A user of this function can (appear to) mutate
 * this object, but the original object obj should not actually be effected.
 *
 * For example, a user could write code like this:
 * const originalObj = {"x": 5};
 * const helper = new ImmutableHelper(originalObj);
 * const newObj = helper.produce((proxy) => {
 *   proxy.x = proxy.x + 1;
 * });
 * console.log(originalObj); // {"x": 5}
 * console.log(newObj); // {"x": 6}
 *
 * Properties of the mutator function:
 * - It will always return undefined.
 * - It will never access keys that don't exist.
 * - It will never delete keys (delete obj.key)
 * - It will never call methods on a proxied object (push, shift, etc).
 * - It will never set keys to objects (proxy.x = {})
 *
 * Note on how the solution will be tested: the solution validator will only analyze
differences
 * between what was returned and the original obj. Doing a full comparison would be too
 * computationally expensive. Also, any mutations to the original object will result in a
 * wrong answer.
 */

var ImmutableHelper = function(obj) {
  this.original = obj;
};

/**
 * @param {Function} mutator
 * @return {JSON} clone of obj
 */
ImmutableHelper.prototype.produce = function(mutator) {
  const mutated = { _: this.original };

  mutator(proxify(mutated, { _: this.original }, (field, value) => {
    mutated[field] = value;
    return mutated;
  })._);
```

```
  return mutated._;

  function proxify(mutableObj, originalObj, setter) {
    return new Proxy(mutableObj, {
      set(_, property, value) {
        mutableObj = setter(property, value);
      },

      get(_, property) {
        let value = mutableObj[property];

        if (!value || typeof value !== 'object') {
          return value;
        } else {
          return proxify(value, originalObj[property], (field, newValue) => {
            if (value === originalObj[property]) {
              mutableObj = setter(
                property,
                value = Array.isArray(value) ? [...value] : { ...value }
              );
            }
            value[field] = newValue;
            return value;
          });
        }
      }
    });
  }
};
```

2692 Make Object Immutable

MEDIUM

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 2692. Make Object Immutable
 * https://leetcode.com/problems/make-object-immutable/
 * Difficulty: Medium
 *
 * Write a function that takes an object obj and returns a new immutable version of this
object.
 *
 * An immutable object is an object that can't be altered and will throw an error if any
attempt
 * is made to alter it.
 *
 * There are three types of error messages that can be produced from this new object.
 * - Attempting to modify a key on the object will result in this error message:
 *   - `Error Modifying: ${key}`.
 * - Attempting to modify an index on an array will result in this error message:
 *   - `Error Modifying Index: ${index}`.
 * - Attempting to call a method that mutates an array will result in this error message:
 *   - `Error Calling Method: ${methodName}`.
 *   - You may assume the only methods that can mutate an array
 *     are ['pop', 'push', 'shift', 'unshift', 'splice', 'sort', 'reverse'].
 *
 * obj is a valid JSON object or array, meaning it is the output of JSON.parse().
 *
 * Note that a string literal should be thrown, not an Error.
 */

/**
 * @param {Object|Array} obj
 * @return {Object|Array} immutable obj
 */
var makeImmutable = function(obj) {
  const mutatingMethods = new Set(['pop', 'push', 'shift', 'unshift', 'splice', 'sort',
'reverse']);

  return new Proxy(obj, {
```

```
    set(target, key) {
      if (Array.isArray(target)) {
        throw `Error Modifying Index: ${key}`;
      }
      throw `Error Modifying: ${key}`;
    },

    get(target, key) {
      const value = target[key];

      if (typeof value === 'object' && value !== null) {
        return makeImmutable(value);
      }

      if (typeof value === 'function') {
        return new Proxy(value, {
          apply(...args) {
            if (mutatingMethods.has(key)) {
              throw `Error Calling Method: ${key}`;
            }
            return Reflect.apply(...args);
          }
        });
      }

      return value;
    }
  });
};
```

2693 Call Function With Custom Context

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2693. Call Function with Custom Context
 * https://leetcode.com/problems/call-function-with-custom-context/
 * Difficulty: Medium
 *
 * Enhance all functions to have the callPolyfill method. The method accepts an object obj as
 * its first parameter and any number of additional arguments. The obj becomes the this
context
 * for the function. The additional arguments are passed to the function (that the
callPolyfill
 * method belongs on).
 *
 * For example if you had the function:
 *
 * function tax(price, taxRate) {
 *   const totalCost = price * (1 + taxRate);
 *   console.log(`The cost of ${this.item} is ${totalCost}`);
 * }
 *
 * Calling this function like tax(10, 0.1) will log "The cost of undefined is 11". This is
 * because the this context was not defined.
 *
 * However, calling the function like tax.callPolyfill({item: "salad"}, 10, 0.1) will log "The
 * cost of salad is 11". The this context was appropriately set, and the function logged an
 * appropriate output.
 *
 * Please solve this without using the built-in Function.call method.
 */

/**
 * @param {Object} context
 * @param {Array} args
 * @return {null|boolean|number|string|Array|Object}
 */
Function.prototype.callPolyfill = function(context, ...args) {
  return this.bind(context)(...args);
```

```
}
```

2694 Event Emitter

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2694. Event Emitter
 * https://leetcode.com/problems/event-emitter/
 * Difficulty: Medium
 *
 * Design an EventEmitter class. This interface is similar (but with some differences) to the
one
 * found in Node.js or the Event Target interface of the DOM. The EventEmitter should allow
for
 * subscribing to events and emitting them.
 *
 * Your EventEmitter class should have the following two methods:
 * - subscribe - This method takes in two arguments: the name of an event as a string and a
callback
 *   function. This callback function will later be called when the event is emitted.
 *   An event should be able to have multiple listeners for the same event. When emitting an
event
 *   with multiple callbacks, each should be called in the order in which they were
subscribed.
 *   An array of results should be returned. You can assume no callbacks passed to subscribe
are
 *   referentially identical.
 *   The subscribe method should also return an object with an unsubscribe method that enables
 the
 *   user to unsubscribe. When it is called, the callback should be removed from the list of
 *   subscriptions and undefined should be returned.
 * - emit - This method takes in two arguments: the name of an event as a string and an
optional
 *   array of arguments that will be passed to the callback(s). If there are no callbacks
subscribed
 *   to the given event, return an empty array. Otherwise, return an array of the results of
all
 *   callback calls in the order they were subscribed.
 */

class EventEmitter {
  constructor() {
    this.events = new Map();
  }

  /**
   * @param {string} eventName
   * @param {Function} callback
   * @return {Object}
   */
  subscribe(eventName, callback) {
    if (!this.events.has(eventName)) {
      this.events.set(eventName, []);
    }

    const listeners = this.events.get(eventName);
    listeners.push(callback);

    return {
      unsubscribe: () => {
        const index = listeners.indexOf(callback);
        if (index !== -1) {
          listeners.splice(index, 1);
        }
      }
    };
  }

  /**
```

```
   * @param {string} eventName
   * @param {Array} args
   * @return {Array}
   */
  emit(eventName, args = []) {
    if (!this.events.has(eventName)) {
      return [];
    }

    return this.events.get(eventName).map(listener => listener(...args));
  }
}
```

2695 Array Wrapper

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2695. Array Wrapper
 * https://leetcode.com/problems/array-wrapper/
 * Difficulty: Easy
 *
 * Create a class ArrayWrapper that accepts an array of integers in its constructor.
 *
 * This class should have two features:
 * - When two instances of this class are added together with the + operator, the resulting
 *   value is the sum of all the elements in both arrays.
 * - When the String() function is called on the instance, it will return a comma separated
 *   string surrounded by brackets. For example, [1,2,3].
 */

/**
 * @param {number[]} nums
 * @return {void}
 */
var ArrayWrapper = function(nums) {
  this.nums = nums;
};

/**
 * @return {number}
 */
ArrayWrapper.prototype.valueOf = function() {
  return this.nums.reduce((sum, n) => sum + n, 0);
};

/**
 * @return {string}
 */
ArrayWrapper.prototype.toString = function() {
  return JSON.stringify(this.nums);
};
```

2696 Minimum String Length After Removing Substrings

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2696. Minimum String Length After Removing Substrings
 * https://leetcode.com/problems/minimum-string-length-after-removing-substrings/
 * Difficulty: Easy
 *
 * You are given a string s consisting only of uppercase English letters.
 *
 * You can apply some operations to this string where, in one operation, you can remove any
```

```
 * occurrence of one of the substrings "AB" or "CD" from s.
 *
 * Return the minimum possible length of the resulting string that you can obtain.
 *
 * Note that the string concatenates after removing the substring and could produce new "AB"
 * or "CD" substrings.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minLength = function(s) {
  const stack = [];

  for (const char of s) {
    if (stack.length && ((stack[stack.length - 1] === 'A' && char === 'B')
        || (stack[stack.length - 1] === 'C' && char === 'D'))) {
      stack.pop();
    } else {
      stack.push(char);
    }
  }

  return stack.length;
};
```

2697 Lexicographically Smallest Palindrome

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2697. Lexicographically Smallest Palindrome
 * https://leetcode.com/problems/lexicographically-smallest-palindrome/
 * Difficulty: Easy
 *
 * You are given a string s consisting of lowercase English letters, and you are allowed to
 * perform operations on it. In one operation, you can replace a character in s with another
 * lowercase English letter.
 *
 * Your task is to make s a palindrome with the minimum number of operations possible. If
there
 * are multiple palindromes that can be made using the minimum number of operations, make the
 * lexicographically smallest one.
 *
 * A string a is lexicographically smaller than a string b (of the same length) if in the
first
 * position where a and b differ, string a has a letter that appears earlier in the alphabet
 * than the corresponding letter in b.
 *
 * Return the resulting palindrome string.
 */

/**
 * @param {string} s
 * @return {string}
 */
var makeSmallestPalindrome = function(s) {
  const chars = s.split('');
  let left = 0;
  let right = s.length - 1;

  while (left < right) {
    if (chars[left] !== chars[right]) {
      chars[left] = chars[right] = chars[left] < chars[right] ? chars[left] : chars[right];
    }
    left++;
    right--;
  }
```

```
  return chars.join('');
};
```

2698 Find The Punishment Number Of An Integer

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2698. Find the Punishment Number of an Integer
 * https://leetcode.com/problems/find-the-punishment-number-of-an-integer/
 * Difficulty: Medium
 *
 * Given a positive integer n, return the punishment number of n.
 *
 * The punishment number of n is defined as the sum of the squares of all integers i such
that:
 * - 1 <= i <= n
 * - The decimal representation of i * i can be partitioned into contiguous substrings such
that
 *   the sum of the integer values of these substrings equals i.
 */

/**
 * @param {number} n
 * @return {number}
 */
var punishmentNumber = function(n) {
  function canPartition(num, target, start) {
    if (!target && start === num.length) {
      return true;
    } else if (start >= num.length) {
      return false;
    }
    for (let i = start, sum = 0; i < num.length; i++) {
      sum = sum * 10 + +num[i];
      if (sum > target) {
        break;
      } else if (canPartition(num, target - sum, i + 1)) {
        return true;
      }
    }

    return false;
  }

  let result = 0;
  for (let i = 1; i <= n; i++) {
    if (canPartition((i * i).toString(), i, 0)) {
      result += i * i;
    }
  }

  return result;
};
```

2700 Differences Between Two Objects

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2700. Differences Between Two Objects
 * https://leetcode.com/problems/differences-between-two-objects/
 * Difficulty: Medium
 *
 * Write a function that accepts two deeply nested objects or arrays obj1 and obj2 and returns
```

```
 * a new object representing their differences.
 *
 * The function should compare the properties of the two objects and identify any changes.
 * The returned object should only contains keys where the value is different from obj1 to
obj2.
 *
 * For each changed key, the value should be represented as an array [obj1 value, obj2 value].
 * Keys that exist in one object but not in the other should not be included in the returned
 * object. The end result should be a deeply nested object where each leaf value is a
 * difference array.
 *
 * When comparing two arrays, the indices of the arrays are considered to be their keys.
 *
 * You may assume that both objects are the output of JSON.parse.
 */

/**
 * @param {Object|Array} obj1
 * @param {Object|Array} obj2
 * @return {Object|Array}
 */
function objDiff(obj1, obj2) {
  if (obj1 === obj2) return {};

  if (obj1 === null || obj2 === null || typeof obj1 !== typeof obj2
    || typeof obj1 !== 'object') {
    return [obj1, obj2];
  }

  if (Array.isArray(obj1) !== Array.isArray(obj2)) {
    return [obj1, obj2];
  }

  const result = {};
  for (const key in obj1) {
    if (key in obj2) {
      const difference = objDiff(obj1[key], obj2[key]);
      if (Object.keys(difference).length > 0 || Array.isArray(difference)) {
        result[key] = difference;
      }
    }
  }

  return result;
}
```

2702 Minimum Operations To Make Numbers Non Positive

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2702. Minimum Operations to Make Numbers Non-positive
 * https://leetcode.com/problems/minimum-operations-to-make-numbers-non-positive/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums and two integers x and y. In one operation,
 * you must choose an index i such that 0 <= i < nums.length and perform the following:
 * - Decrement nums[i] by x.
 * - Decrement values by y at all indices except the ith one.
 *
 * Return the minimum number of operations to make all the integers in nums less than or equal
 * to zero.
 */

/**
 * @param {number[]} nums
 * @param {number} x
 * @param {number} y
 * @return {number}
 */
```

```
var minOperations = function(nums, x, y) {
  let left = 0;
  let right = Math.max(...nums);

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (canMakeNonPositive(nums, x, y, mid)) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;

  function canMakeNonPositive(nums, x, y, operations) {
    const n = nums.length;
    const remaining = nums.map(num => num - operations * y);

    let totalExtraNeeded = 0;
    for (let i = 0; i < n; i++) {
      if (remaining[i] > 0) {
        const extraNeeded = Math.ceil(remaining[i] / (x - y));
        totalExtraNeeded += extraNeeded;
      }
    }

    return totalExtraNeeded <= operations;
  }
};
```

2703 Return Length Of Arguments Passed

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2703. Return Length of Arguments Passed
 * https://leetcode.com/problems/return-length-of-arguments-passed/
 * Difficulty: Easy
 *
 * Write a function argumentsLength that returns the count of arguments passed to it.
 */

/**
 * @param {...(null|boolean|number|string|Array|Object)} args
 * @return {number}
 */
var argumentsLength = function(...args) {
  return args.length;
};
```

2704 To Be Or Not To Be

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2704. To Be Or Not To Be
 * https://leetcode.com/problems/to-be-or-not-to-be/
 * Difficulty: Easy
 *
 * Write a function expect that helps developers test their code. It should take in any
 * value val and return an object with the following two functions.
 *
 * - toBe(val) accepts another value and returns true if the two values === each other.
 *   If they are not equal, it should throw an error "Not Equal".
```

```
 * - notToBe(val) accepts another value and returns true if the two values !== each
 *   other. If they are equal, it should throw an error "Equal".
 */

/**
 * @param {string} val
 * @return {Object}
 */
var expect = function(val) {
  const compareHelper = (error, fn) => {
    if (fn()) throw new Error(error);
    return true;
  };
  return {
    toBe: input => compareHelper('Not Equal', () => input !== val),
    notToBe: input => compareHelper('Equal', () => input === val),
  };
};
```

2705 Compact Object

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2705. Compact Object
 * https://leetcode.com/problems/compact-object/
 * Difficulty: Medium
 *
 * Given an object or array obj, return a compact object.
 *
 * A compact object is the same as the original object, except with keys containing falsy
 * values removed. This operation applies to the object and any nested objects. Arrays are
 * considered objects where the indices are keys. A value is considered falsy when
 * Boolean(value) returns false.
 *
 * You may assume the obj is the output of JSON.parse. In other words, it is valid JSON.
 */

/**
 * @param {Object|Array} obj
 * @return {Object|Array}
 */
var compactObject = function(obj) {
  if (obj === null) return null;
  if (Array.isArray(obj)) return obj.filter(Boolean).map(compactObject);
  if (typeof obj !== 'object') return obj;

  return Object.keys(obj).reduce((result, key) => {
    const value = compactObject(obj[key]);
    if (value) {
      result[key] = value;
    }
    return result;
  }, {});
};
```

2706 Buy Two Chocolates

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2706. Buy Two Chocolates
 * https://leetcode.com/problems/buy-two-chocolates/
 * Difficulty: Easy
 *
```

```
 * You are given an integer array prices representing the prices of various chocolates in
 * a store. You are also given a single integer money, which represents your initial amount
 * of money.
 *
 * You must buy exactly two chocolates in such a way that you still have some non-negative
 * leftover money. You would like to minimize the sum of the prices of the two chocolates
 * you buy.
 *
 * Return the amount of money you will have leftover after buying the two chocolates. If
 * there is no way for you to buy two chocolates without ending up in debt, return money.
 * Note that the leftover must be non-negative.
 */

/**
 * @param {number[]} prices
 * @param {number} money
 * @return {number}
 */
var buyChoco = function(prices, money) {
  prices.sort((a, b) => a - b);
  const minCost = prices[0] + prices[1];
  return minCost <= money ? money - minCost : money;
};
```

2707 Extra Characters In A String

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2707. Extra Characters in a String
 * https://leetcode.com/problems/extra-characters-in-a-string/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s and a dictionary of words dictionary. You have to break
 * s into one or more non-overlapping substrings such that each substring is present in
 * dictionary. There may be some extra characters in s which are not present in any of the
 * substrings.
 *
 * Return the minimum number of extra characters left over if you break up s optimally.
 */

/**
 * @param {string} s
 * @param {string[]} dictionary
 * @return {number}
 */
var minExtraChar = function(s, dictionary) {
  const n = s.length;
  const dp = new Array(n + 1).fill(Infinity);
  dp[0] = 0;
  const dictSet = new Set(dictionary);

  for (let i = 1; i <= n; i++) {
    dp[i] = dp[i - 1] + 1;
    for (let j = 0; j < i; j++) {
      if (dictSet.has(s.slice(j, i))) {
        dp[i] = Math.min(dp[i], dp[j]);
      }
    }
  }

  return dp[n];
};
```

2709 Greatest Common Divisor Traversal

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2709. Greatest Common Divisor Traversal
 * https://leetcode.com/problems/greatest-common-divisor-traversal/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums, and you are allowed to traverse between its
 * indices. You can traverse between index i and index j, i != j, if and only if gcd(nums[i],
 * nums[j]) > 1, where gcd is the greatest common divisor.
 *
 * Your task is to determine if for every pair of indices i and j in nums, where i < j, there
 * exists a sequence of traversals that can take us from i to j.
 *
 * Return true if it is possible to traverse between all such pairs of indices, or false
otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var canTraverseAllPairs = function(nums) {
  if (nums.length === 1) return true;
  const n = nums.length;
  const maxNum = Math.max(...nums);
  const parent = new Array(maxNum + 1).fill().map((_, i) => i);
  const rank = new Array(maxNum + 1).fill(0);

  const primeToIndex = new Map();
  for (let i = 0; i < n; i++) {
    if (nums[i] === 1) return false;
    const factors = getPrimeFactors(nums[i]);
    for (const prime of factors) {
      if (primeToIndex.has(prime)) {
        union(primeToIndex.get(prime), i);
      } else {
        primeToIndex.set(prime, i);
      }
    }
  }

  const root = find(0);
  for (let i = 1; i < n; i++) {
    if (find(i) !== root) return false;
  }

  return true;

  function find(x) {
    if (parent[x] !== x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  function union(x, y) {
    const px = find(x);
    const py = find(y);
    if (px === py) return;
    if (rank[px] < rank[py]) {
      parent[px] = py;
    } else if (rank[px] > rank[py]) {
      parent[py] = px;
    } else {
      parent[py] = px;
      rank[px]++;
```

```
    }
  }

  function getPrimeFactors(num) {
    const factors = [];
    for (let i = 2; i * i <= num; i++) {
      if (num % i === 0) {
        factors.push(i);
        while (num % i === 0) num /= i;
      }
    }
    if (num > 1) factors.push(num);
    return factors;
  }
};
```

2710 Remove Trailing Zeros From A String

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2710. Remove Trailing Zeros From a String
 * https://leetcode.com/problems/remove-trailing-zeros-from-a-string/
 * Difficulty: Easy
 *
 * Given a positive integer num represented as a string, return the integer num without
 * trailing zeros as a string.
 */

/**
 * @param {string} num
 * @return {string}
 */
var removeTrailingZeros = function(num) {
  return num.replace(/0+$/g, '');
};
```

2711 Difference Of Number Of Distinct Values On Diagonals

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2711. Difference of Number of Distinct Values on Diagonals
 * https://leetcode.com/problems/difference-of-number-of-distinct-values-on-diagonals/
 * Difficulty: Medium
 *
 * Given a 2D grid of size m x n, you should find the matrix answer of size m x n.
 *
 * The cell answer[r][c] is calculated by looking at the diagonal values of the cell
grid[r][c]:
 * - Let leftAbove[r][c] be the number of distinct values on the diagonal to the left and
above
 *   the cell grid[r][c] not including the cell grid[r][c] itself.
 * - Let rightBelow[r][c] be the number of distinct values on the diagonal to the right and
below
 *   the cell grid[r][c], not including the cell grid[r][c] itself.
 * - Then answer[r][c] = |leftAbove[r][c] - rightBelow[r][c]|.
 *
 * A matrix diagonal is a diagonal line of cells starting from some cell in either the topmost
 * row or leftmost column and going in the bottom-right direction until the end of the matrix
 * is reached.
 *
 * - For example, in the below diagram the diagonal is highlighted using the cell with indices
 *   (2, 3) colored gray:
```

```
 *   - Red-colored cells are left and above the cell.
 *   - Blue-colored cells are right and below the cell.
 */

/**
 * @param {number[][]} grid
 * @return {number[][]}
 */
var differenceOfDistinctValues = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const result = Array.from({ length: rows }, () => new Array(cols).fill(0));

  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      const leftAboveSet = new Set();
      let i = r - 1;
      let j = c - 1;
      while (i >= 0 && j >= 0) {
        leftAboveSet.add(grid[i--][j--]);
      }

      const rightBelowSet = new Set();
      i = r + 1;
      j = c + 1;
      while (i < rows && j < cols) {
        rightBelowSet.add(grid[i++][j++]);
      }

      result[r][c] = Math.abs(leftAboveSet.size - rightBelowSet.size);
    }
  }

  return result;
};
```

2712 Minimum Cost To Make All Characters Equal

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2712. Minimum Cost to Make All Characters Equal
 * https://leetcode.com/problems/minimum-cost-to-make-all-characters-equal/
 * Difficulty: Medium
 *
 * You are given a 0-indexed binary string s of length n on which you can apply two types
 * of operations:
 * - Choose an index i and invert all characters from index 0 to index i (both inclusive),
 *   with a cost of i + 1
 * - Choose an index i and invert all characters from index i to index n - 1 (both inclusive),
 *   with a cost of n - i
 *
 * Return the minimum cost to make all characters of the string equal.
 *
 * Invert a character means if its value is '0' it becomes '1' and vice-versa.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumCost = function(s) {
  let result = 0;

  for (let i = 1; i < s.length; i++) {
    if (s[i] !== s[i - 1]) {
      result += Math.min(i, s.length - i);
    }
  }
```

```
  return result;
};
```

2714 Find Shortest Path With K Hops

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2714. Find Shortest Path with K Hops
 * https://leetcode.com/problems/find-shortest-path-with-k-hops/
 * Difficulty: Hard
 *
 * You are given a positive integer n which is the number of nodes of a 0-indexed undirected
 * weighted connected graph and a 0-indexed 2D array edges where edges[i] = [ui, vi, wi]
 * indicates that there is an edge between nodes ui and vi with weight wi.
 *
 * You are also given two nodes s and d, and a positive integer k, your task is to find the
 * shortest path from s to d, but you can hop over at most k edges. In other words, make the
 * weight of at most k edges 0 and then find the shortest path from s to d.
 *
 * Return the length of the shortest path from s to d with the given condition.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number} s
 * @param {number} d
 * @param {number} k
 * @return {number}
 */
var shortestPathWithHops = function(n, edges, s, d, k) {
  const graph = new Array(n).fill().map(() => []);

  for (const [u, v, w] of edges) {
    graph[u].push([v, w]);
    graph[v].push([u, w]);
  }

  const dist = new Array(n).fill().map(() => new Array(k + 1).fill(Infinity));
  const visited = new Array(n).fill().map(() => new Array(k + 1).fill(false));
  const pq = new PriorityQueue((a, b) => a[0] - b[0]);

  pq.enqueue([0, k, s]);

  while (!pq.isEmpty()) {
    const [currentDist, hopsLeft, node] = pq.dequeue();

    if (visited[node][hopsLeft]) continue;
    visited[node][hopsLeft] = true;

    if (node === d) return currentDist;

    for (const [neighbor, weight] of graph[node]) {
      const newDist = currentDist + weight;
      if (newDist < dist[neighbor][hopsLeft]) {
        dist[neighbor][hopsLeft] = newDist;
        pq.enqueue([newDist, hopsLeft, neighbor]);
      }

      if (hopsLeft > 0 && currentDist < dist[neighbor][hopsLeft - 1]) {
        dist[neighbor][hopsLeft - 1] = currentDist;
        pq.enqueue([currentDist, hopsLeft - 1, neighbor]);
      }
    }
  }

  return -1;
};
```

2715 Timeout Cancellation

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2715. Timeout Cancellation
 * https://leetcode.com/problems/timeout-cancellation/
 * Difficulty: Easy
 *
 * Given a function fn, an array of arguments args, and a timeout t in milliseconds,
 * return a cancel function cancelFn.
 *
 * After a delay of cancelTimeMs, the returned cancel function cancelFn will be invoked.
 * > setTimeout(cancelFn, cancelTimeMs)
 *
 * Initially, the execution of the function fn should be delayed by t milliseconds.
 *
 * If, before the delay of t milliseconds, the function cancelFn is invoked, it should
 * cancel the delayed execution of fn. Otherwise, if cancelFn is not invoked within the
 * specified delay t, fn should be executed with the provided args as arguments.
 */

/**
 * @param {Function} fn
 * @param {Array} args
 * @param {number} t
 * @return {Function}
 */
var cancellable = function(fn, args, t) {
  const timeoutId = setTimeout(() => fn(...args), t);
  return () => clearTimeout(timeoutId);
};
```

2716 Minimize String Length

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 2716. Minimize String Length
 * https://leetcode.com/problems/minimize-string-length/
 * Difficulty: Easy
 *
 * Given a string s, you have two types of operation:
 * 1. Choose an index i in the string, and let c be the character in position i. Delete the
 *    closest occurrence of c to the left of i (if exists).
 * 2. Choose an index i in the string, and let c be the character in position i. Delete the
 *    closest occurrence of c to the right of i (if exists).
 *
 * Your task is to minimize the length of s by performing the above operations zero or more
times.
 *
 * Return an integer denoting the length of the minimized string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimizedStringLength = function(s) {
  return new Set(s).size;
};
```

2717 Semi Ordered Permutation

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2717. Semi-Ordered Permutation
 * https://leetcode.com/problems/semi-ordered-permutation/
 * Difficulty: Easy
 *
 * You are given a 0-indexed permutation of n integers nums.
 *
 * A permutation is called semi-ordered if the first number equals 1 and the last number
equals n.
 * You can perform the below operation as many times as you want until you make nums a
semi-ordered
 * permutation:
 * - Pick two adjacent elements in nums, then swap them.
 *
 * Return the minimum number of operations to make nums a semi-ordered permutation.
 *
 * A permutation is a sequence of integers from 1 to n of length n containing each number
exactly
 * once.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var semiOrderedPermutation = function(nums) {
  let oneIndex = 0;
  let nIndex = 0;

  for (let i = 0; i < nums.length; i++) {
    if (nums[i] === 1) oneIndex = i;
    if (nums[i] === nums.length) nIndex = i;
  }

  return oneIndex + (nums.length - 1 - nIndex) - (oneIndex > nIndex ? 1 : 0);
};
```

2721 Execute Asynchronous Functions In Parallel

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2721. Execute Asynchronous Functions in Parallel
 * https://leetcode.com/problems/execute-asynchronous-functions-in-parallel/
 * Difficulty: Medium
 *
 * Given an array of asynchronous functions functions, return a new promise promise.
 * Each function in the array accepts no arguments and returns a promise. All the
 * promises should be executed in parallel.
 *
 * promise resolves:
 * - When all the promises returned from functions were resolved successfully in
 * parallel. The resolved value of promise should be an array of all the resolved
 * values of promises in the same order as they were in the functions. The promise
 * should resolve when all the asynchronous functions in the array have completed
 * execution in parallel.
 *
 * promise rejects:
 * When any of the promises returned from functions were rejected. promise should
 * also reject with the reason of the first rejection.
 *
```

```
 * Please solve it without using the built-in Promise.all function.
 */

/**
 * @param {Array<Function>} functions
 * @return {Promise<any>}
 */
var promiseAll = function(functions) {
  return new Promise((resolve, reject) => {
    const promises = new Array(functions.length);
    let count = 0;
    functions.forEach((fn, i) => {
      fn().then(result => {
        promises[i] = result;
        if (++count === promises.length) {
          resolve(promises);
        }
      }).catch(reject);
    });
  });
};
```

2722 Join Two Arrays By Id

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2722. Join Two Arrays by ID
 * https://leetcode.com/problems/join-two-arrays-by-id/
 * Difficulty: Medium
 *
 * Given two arrays arr1 and arr2, return a new array joinedArray. All the objects in each
 * of the two inputs arrays will contain an id field that has an integer value.
 *
 * joinedArray is an array formed by merging arr1 and arr2 based on their id key. The length
 * of joinedArray should be the length of unique values of id. The returned array should be
 * sorted in ascending order based on the id key.
 *
 * If a given id exists in one array but not the other, the single object with that id should
 * be included in the result array without modification.
 *
 * If two objects share an id, their properties should be merged into a single object:
 * - If a key only exists in one object, that single key-value pair should be included in
 *   the object.
 * - If a key is included in both objects, the value in the object from arr2 should override
 *   the value from arr1.
 */

/**
 * @param {Array} arr1
 * @param {Array} arr2
 * @return {Array}
 */
var join = function(arr1, arr2) {
  const map = new Map();
  [...arr1, ...arr2].forEach(obj => map.set(obj.id, { ...map.get(obj.id), ...obj }));
  return Array.from(map.values()).sort((a, b) => a.id - b.id);
};
```

2723 Add Two Promises

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2723. Add Two Promises
```

```
 * https://leetcode.com/problems/add-two-promises/
 * Difficulty: Easy
 *
 * Given two promises promise1 and promise2, return a new promise. promise1 and promise2
 * will both resolve with a number. The returned promise should resolve with the sum of
 * the two numbers.
 */

/**
 * @param {Promise} promise1
 * @param {Promise} promise2
 * @return {Promise}
 */
var addTwoPromises = async function(promise1, promise2) {
  return Promise.all([promise1, promise2]).then(([a, b]) => a + b);
};
```

2724 Sort By

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2724. Sort By
 * https://leetcode.com/problems/sort-by/
 * Difficulty: Easy
 *
 * Given an array arr and a function fn, return a sorted array sortedArr.
 * You can assume fn only returns numbers and those numbers determine the
 * sort order of sortedArr. sortedArr must be sorted in ascending order
 * by fn output.
 *
 * You may assume that fn will never duplicate numbers for a given array.
 */

/**
 * @param {Array} arr
 * @param {Function} fn
 * @return {Array}
 */
var sortBy = function(arr, fn) {
  return arr.sort((a, b) => fn(a) - fn(b));
};
```

2725 Interval Cancellation

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2725. Interval Cancellation
 * https://leetcode.com/problems/interval-cancellation/
 * Difficulty: Easy
 *
 * Given a function fn, an array of arguments args, and an interval time t, return a cancel
 * function cancelFn.
 *
 * After a delay of cancelTimeMs, the returned cancel function cancelFn will be invoked.
 * > setTimeout(cancelFn, cancelTimeMs)
 *
 * The function fn should be called with args immediately and then called again every t
 * milliseconds until cancelFn is called at cancelTimeMs ms.
 */

/**
 * @param {Function} fn
 * @param {Array} args
```

```
 * @param {number} t
 * @return {Function}
 */
var cancellable = function(fn, args, t) {
  fn(...args);
  const timer = setInterval(() => fn(...args), t);
  return () => clearInterval(timer);
};
```

2726 Calculator With Method Chaining

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2726. Calculator with Method Chaining
 * https://leetcode.com/problems/calculator-with-method-chaining/
 * Difficulty: Easy
 *
 * Design a Calculator class. The class should provide the mathematical operations of
addition,
 * subtraction, multiplication, division, and exponentiation. It should also allow consecutive
 * operations to be performed using method chaining. The Calculator class constructor should
 * accept a number which serves as the initial value of result.
 *
 * Your Calculator class should have the following methods:
 * - add - This method adds the given number value to the result and returns the updated
Calculator.
 * - subtract - This method subtracts the given number value from the result and returns the
 *   updated Calculator.
 * - multiply - This method multiplies the result  by the given number value and returns the
 *   updated Calculator.
 * - divide - This method divides the result by the given number value and returns the updated
 *   Calculator. If the passed value is 0, an error "Division by zero is not allowed" should
 *   be thrown.
 * - power - This method raises the result to the power of the given number value and returns
 *   the updated Calculator.
 * - getResult - This method returns the result.
 *
 * Solutions within 10-5 of the actual result are considered correct.
 */

class Calculator {
  /**
   * @param {number} value
   */
  constructor(value) {
    this.value = value;
  }

  /**
   * @param {number} value
   * @return {Calculator}
   */
  add(value) {
    this.value += value;
    return this;
  }

  /**
   * @param {number} value
   * @return {Calculator}
   */
  subtract(value) {
    this.value -= value;
    return this;
  }

  /**
   * @param {number} value
   * @return {Calculator}
```

```
   */
  multiply(value) {
    this.value *= value;
    return this;
  }

  /**
   * @param {number} value
   * @return {Calculator}
   */
  divide(value) {
    if (value === 0) {
      throw new Error('Division by zero is not allowed');
    }
    this.value /= value;
    return this;
  }

  /**
   * @param {number} value
   * @return {Calculator}
   */
  power(value) {
    this.value **= value;
    return this;
  }

  /**
   * @return {number}
   */
  getResult() {
    return this.value;
  }
}
```

2727 Is Object Empty

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2727. Is Object Empty
 * https://leetcode.com/problems/is-object-empty/
 * Difficulty: Easy
 *
 * Given an object or an array, return if it is empty.
 *
 * - An empty object contains no key-value pairs.
 * - An empty array contains no elements.
 *
 * You may assume the object or array is the output of JSON.parse.
 */

/**
 * @param {Object|Array} obj
 * @return {boolean}
 */
var isEmpty = function(obj) {
  return !Object.keys(obj).length;
};
```

2728 Count Houses In A Circular Street

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 2728. Count Houses in a Circular Street
 * https://leetcode.com/problems/count-houses-in-a-circular-street/
 * Difficulty: Easy
 *
 * You are given an object street of class Street that represents a circular street and a
positive
 * integer k which represents a maximum bound for the number of houses in that street (in
other
 * words, the number of houses is less than or equal to k). Houses' doors could be open or
closed
 * initially.
 *
 * Initially, you are standing in front of a door to a house on this street. Your task is to
count
 * the number of houses in the street.
 *
 * The class Street contains the following functions which may help you:
 * - void openDoor(): Open the door of the house you are in front of.
 * - void closeDoor(): Close the door of the house you are in front of.
 * - boolean isDoorOpen(): Returns true if the door of the current house is open and false
 *   otherwise.
 * - void moveRight(): Move to the right house.
 * - void moveLeft(): Move to the left house.
 *
 * Return ans which represents the number of houses on this street.
 */

/**
 * Definition for a street.
 * class Street {
 *     @param {number[]} doors
 *     constructor(doors);
 *
 *     @return {void}
 *     openDoor();
 *
 *     @return {void}
 *     closeDoor();
 *
 *     @return {boolean}
 *     isDoorOpen();
 *
 *     @return {void}
 *     moveRight();
 *
 *     @return {void}
 *     moveLeft();
 * }
 */
/**
 * @param {Street} street
 * @param {number} k
 * @return {number}
 */
var houseCount = function(street, k) {
  for (let i = 0; i < k; i++) {
    street.closeDoor();
    street.moveLeft();
  }

  street.openDoor();
  let count = 1;

  street.moveLeft();
  while (!street.isDoorOpen()) {
    street.moveLeft();
    count++;
  }

  return count;
};
```

2729 Check If The Number Is Fascinating

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 2729. Check if The Number is Fascinating
 * https://leetcode.com/problems/check-if-the-number-is-fascinating/
 * Difficulty: Easy
 *
 * You are given an integer n that consists of exactly 3 digits.
 *
 * We call the number n fascinating if, after the following modification, the resulting number
 * contains all the digits from 1 to 9 exactly once and does not contain any 0's:
 * - Concatenate n with the numbers 2 * n and 3 * n.
 *
 * Return true if n is fascinating, or false otherwise.
 *
 * Concatenating two numbers means joining them together. For example, the concatenation of
121
 * and 371 is 121371.
 */

/**
 * @param {number} n
 * @return {boolean}
 */
var isFascinating = function(n) {
  const concatenated = `${n}${2 * n}${3 * n}`;
  const digitSet = new Set(concatenated);
  return concatenated.length === 9 && digitSet.size === 9 && !digitSet.has('0');
};
```

2732 Find A Good Subset Of The Matrix

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2732. Find a Good Subset of the Matrix
 * https://leetcode.com/problems/find-a-good-subset-of-the-matrix/
 * Difficulty: Hard
 *
 * You are given a 0-indexed m x n binary matrix grid.
 *
 * Let us call a non-empty subset of rows good if the sum of each column of the subset is at
 * most half of the length of the subset.
 *
 * More formally, if the length of the chosen subset of rows is k, then the sum of each column
 * should be at most floor(k / 2).
 *
 * Return an integer array that contains row indices of a good subset sorted in ascending
order.
 *
 * If there are multiple good subsets, you can return any of them. If there are no good
subsets,
 * return an empty array.
 *
 * A subset of rows of the matrix grid is any matrix that can be obtained by deleting some
(possibly
 * none or all) rows from grid.
 */

/**
 * @param {number[][]} grid
 * @return {number[]}
 */
```

```
var goodSubsetofBinaryMatrix = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const rowMasks = new Map();

  for (let i = 0; i < rows; i++) {
    let mask = 0;
    for (let j = 0; j < cols; j++) {
      mask |= grid[i][j] << j;
    }
    if (mask === 0) return [i];
    rowMasks.set(mask, i);
  }

  for (const [mask1, i] of rowMasks) {
    for (const [mask2, j] of rowMasks) {
      if ((mask1 & mask2) === 0) return [Math.min(i, j), Math.max(i, j)];
    }
  }

  return [];
};
```

2733 Neither Minimum Nor Maximum

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2733. Neither Minimum nor Maximum
 * https://leetcode.com/problems/neither-minimum-nor-maximum/
 * Difficulty: Easy
 *
 * Given an integer array nums containing distinct positive integers, find and return any
 * number from the array that is neither the minimum nor the maximum value in the array,
 * or -1 if there is no such number.
 *
 * Return the selected integer.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findNonMinOrMax = function(nums) {
  if (nums.length < 3) return -1;
  const minVal = Math.min(nums[0], nums[1], nums[2]);
  const maxVal = Math.max(nums[0], nums[1], nums[2]);

  for (const num of [nums[0], nums[1], nums[2]]) {
    if (num !== minVal && num !== maxVal) return num;
  }

  return -1;
};
```

2737 Find The Closest Marked Node

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2737. Find the Closest Marked Node
 * https://leetcode.com/problems/find-the-closest-marked-node/
 * Difficulty: Medium
 *
```

```
 * You are given a positive integer n which is the number of nodes of a 0-indexed directed
 * weighted graph and a 0-indexed 2D array edges where edges[i] = [ui, vi, wi] indicates
 * that there is an edge from node ui to node vi with weight wi.
 *
 * You are also given a node s and a node array marked; your task is to find the minimum
 * distance from s to any of the nodes in marked.
 *
 * Return an integer denoting the minimum distance from s to any node in marked or -1 if
 * there are no paths from s to any of the marked nodes.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number} s
 * @param {number[]} marked
 * @return {number}
 */
var minimumDistance = function(n, edges, s, marked) {
  const graph = new Array(n).fill().map(() => []);
  const markedSet = new Set(marked);

  for (const [u, v, w] of edges) {
    graph[u].push([v, w]);
  }

  const distances = new Array(n).fill(Infinity);
  const pq = new PriorityQueue((a, b) => a[0] - b[0]);
  distances[s] = 0;
  pq.enqueue([0, s]);

  while (!pq.isEmpty()) {
    const [currentDist, node] = pq.dequeue();

    if (currentDist > distances[node]) continue;

    if (markedSet.has(node)) {
      return currentDist;
    }

    for (const [neighbor, weight] of graph[node]) {
      const newDist = currentDist + weight;
      if (newDist < distances[neighbor]) {
        distances[neighbor] = newDist;
        pq.enqueue([newDist, neighbor]);
      }
    }
  }

  return -1;
};
```

2739 Total Distance Traveled

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2739. Total Distance Traveled
 * https://leetcode.com/problems/total-distance-traveled/
 * Difficulty: Easy
 *
 * A truck has two fuel tanks. You are given two integers, mainTank representing the fuel
present
 * in the main tank in liters and additionalTank representing the fuel present in the
additional
 * tank in liters.
 *
 * The truck has a mileage of 10 km per liter. Whenever 5 liters of fuel get used up in the
main
 * tank, if the additional tank has at least 1 liters of fuel, 1 liters of fuel will be
```

```
transferred
 * from the additional tank to the main tank.
 *
 * Return the maximum distance which can be traveled.
 *
 * Note: Injection from the additional tank is not continuous. It happens suddenly and
immediately
 * for every 5 liters consumed.
 */

/**
 * @param {number} mainTank
 * @param {number} additionalTank
 * @return {number}
 */
var distanceTraveled = function(mainTank, additionalTank) {
  let distance = 0;
  let fuel = mainTank;

  while (fuel >= 5 && additionalTank > 0) {
    distance += 50;
    fuel -= 5;
    fuel += 1;
    additionalTank -= 1;
  }

  distance += fuel * 10;
  return distance;
};
```

2740 Find The Value Of The Partition

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2740. Find the Value of the Partition
 * https://leetcode.com/problems/find-the-value-of-the-partition/
 * Difficulty: Medium
 *
 * You are given a positive integer array nums.
 *
 * Partition nums into two arrays, nums1 and nums2, such that:
 * - Each element of the array nums belongs to either the array nums1 or the array nums2.
 * - Both arrays are non-empty.
 * - The value of the partition is minimized.
 *
 * The value of the partition is |max(nums1) - min(nums2)|.
 *
 * Here, max(nums1) denotes the maximum element of the array nums1, and min(nums2) denotes the
 * minimum element of the array nums2.
 *
 * Return the integer denoting the value of such partition.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var findValueOfPartition = function(nums) {
  nums.sort((a, b) => a - b);
  let result = Infinity;

  for (let i = 1; i < nums.length; i++) {
    result = Math.min(result, nums[i] - nums[i - 1]);
  }

  return result;
};
```

2742 Painting The Walls

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2742. Painting the Walls
 * https://leetcode.com/problems/painting-the-walls/
 * Difficulty: Hard
 *
 * You are given two 0-indexed integer arrays, cost and time, of size n representing the costs
 * and the time taken to paint n different walls respectively. There are two painters
available:
 * - A paid painter that paints the ith wall in time[i] units of time and takes cost[i] units
of
 *   money.
 * - A free painter that paints any wall in 1 unit of time at a cost of 0. But the free
painter
 *   can only be used if the paid painter is already occupied.
 *
 * Return the minimum amount of money required to paint the n walls.
 */

/**
 * @param {number[]} cost
 * @param {number[]} time
 * @return {number}
 */
var paintWalls = function(cost, time) {
  const n = cost.length;
  const dp = new Array(n + 1).fill().map(() => new Array(n + 1).fill(Infinity));
  dp[0][0] = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j <= n; j++) {
      if (dp[i][j] === Infinity) continue;

      dp[i + 1][j] = Math.min(dp[i + 1][j], dp[i][j]);

      const walls = Math.min(n, j + time[i] + 1);
      dp[i + 1][walls] = Math.min(dp[i + 1][walls], dp[i][j] + cost[i]);
    }
  }

  return dp[n][n];
};
```

2743 Count Substrings Without Repeating Character

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2743. Count Substrings Without Repeating Character
 * https://leetcode.com/problems/count-substrings-without-repeating-character/
 * Difficulty: Medium
 *
 * You are given a string s consisting only of lowercase English letters. We call a substring
 * special if it contains no character which has occurred at least twice (in other words, it
 * does not contain a repeating character). Your task is to count the number of special
 * substrings. For example, in the string "pop", the substring "po" is a special substring,
 * however, "pop" is not special (since 'p' has occurred twice).
 *
 * Return the number of special substrings.
 *
```

```
 * A substring is a contiguous sequence of characters within a string. For example, "abc" is
 * a substring of "abcd", but "acd" is not.
 */

/**
 * @param {string} s
 * @return {number}
 */
var numberOfSpecialSubstrings = function(s) {
  const n = s.length;
  const map = new Map();
  let result = 0;
  let left = 0;

  for (let right = 0; right < n; right++) {
    const char = s[right];
    map.set(char, (map.get(char) || 0) + 1);

    while (map.get(char) > 1) {
      const leftChar = s[left];
      map.set(leftChar, map.get(leftChar) - 1);
      if (map.get(leftChar) === 0) {
        map.delete(leftChar);
      }
      left++;
    }

    result += right - left + 1;
  }

  return result;
};
```

2744 Find Maximum Number Of String Pairs

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2744. Find Maximum Number of String Pairs
 * https://leetcode.com/problems/find-maximum-number-of-string-pairs/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array words consisting of distinct strings.
 *
 * The string words[i] can be paired with the string words[j] if:
 * - The string words[i] is equal to the reversed string of words[j].
 * - 0 <= i < j < words.length.
 *
 * Return the maximum number of pairs that can be formed from the array words.
 *
 * Note that each string can belong in at most one pair.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var maximumNumberOfStringPairs = function(words) {
  const set = new Set();
  let result = 0;

  for (const word of words) {
    const reversed = word[1] + word[0];
    if (set.has(reversed)) {
      result++;
      set.delete(reversed);
    } else {
      set.add(word);
    }
  }
```

```
  return result;
};
```

2745 Construct The Longest New String

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2745. Construct the Longest New String
 * https://leetcode.com/problems/construct-the-longest-new-string/
 * Difficulty: Medium
 *
 * You are given three integers x, y, and z.
 *
 * You have x strings equal to "AA", y strings equal to "BB", and z strings equal to "AB". You
 * want to choose some (possibly all or none) of these strings and concatenate them in some
order
 * to form a new string. This new string must not contain "AAA" or "BBB" as a substring.
 *
 * Return the maximum possible length of the new string.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {number} x
 * @param {number} y
 * @param {number} z
 * @return {number}
 */
var longestString = function(x, y, z) {
  const minPairs = Math.min(x, y);
  return 2 * (minPairs * 2 + (x > minPairs ? 1 : 0) + (y > minPairs ? 1 : 0) + z);
};
```

2748 Number Of Beautiful Pairs

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2748. Number of Beautiful Pairs
 * https://leetcode.com/problems/number-of-beautiful-pairs/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums. A pair of indices i, j where
 * 0 <= i < j < nums.length is called beautiful if the first digit of nums[i] and
 * the last digit of nums[j] are coprime.
 *
 * Return the total number of beautiful pairs in nums.
 *
 * Two integers x and y are coprime if there is no integer greater than 1 that divides
 * both of them. In other words, x and y are coprime if gcd(x, y) == 1, where gcd(x, y)
 * is the greatest common divisor of x and y.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countBeautifulPairs = function(nums) {
  let result = 0;
  for (let i = 0; i < nums.length - 1; i++) {
    for (let j = i + 1; j < nums.length; j++) {
      const first = getFirstDigit(nums[i]);
```

```
      const last = getLastDigit(nums[j]);
      if (gcd(first, last) === 1) {
        result++;
      }
    }
  }

  return result;

  function gcd(a, b) {
    while (b) {
      a %= b;
      [a, b] = [b, a];
    }
    return a;
  }

  function getFirstDigit(num) {
    while (num >= 10) {
      num = Math.floor(num / 10);
    }
    return num;
  }

  function getLastDigit(num) {
    return num % 10;
  }
};
```

2749 Minimum Operations To Make The Integer Zero

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2749. Minimum Operations to Make the Integer Zero
 * https://leetcode.com/problems/minimum-operations-to-make-the-integer-zero/
 * Difficulty: Medium
 *
 * You are given two integers num1 and num2.
 *
 * In one operation, you can choose integer i in the range [0, 60] and subtract 2i + num2 from
 num1.
 *
 * Return the integer denoting the minimum number of operations needed to make num1 equal to
0.
 *
 * If it is impossible to make num1 equal to 0, return -1.
 */

/**
 * @param {number} num1
 * @param {number} num2
 * @return {number}
 */
var makeTheIntegerZero = function(num1, num2) {
  for (let operations = 1; operations <= 60; operations++) {
    const remaining = num1 - num2 * operations;

    if (remaining < operations) {
      return -1;
    }

    if (operations >= countBits(remaining)) {
      return operations;
    }
  }

  return -1;

  function countBits(n) {
```

```
    let count = 0;
    while (n > 0) {
      count += n & 1;
      n = Math.floor(n / 2);
    }
    return count;
  }
};
```

2751 Robot Collisions

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2751. Robot Collisions
 * https://leetcode.com/problems/robot-collisions/
 * Difficulty: Hard
 *
 * There are n 1-indexed robots, each having a position on a line, health, and movement
direction.
 *
 * You are given 0-indexed integer arrays positions, healths, and a string directions
(directions[i]
 * is either 'L' for left or 'R' for right). All integers in positions are unique.
 *
 * All robots start moving on the line simultaneously at the same speed in their given
directions.
 * If two robots ever share the same position while moving, they will collide.
 *
 * If two robots collide, the robot with lower health is removed from the line, and the health
 of
 * the other robot decreases by one. The surviving robot continues in the same direction it
was
 * going. If both robots have the same health, they are both removed from the line.
 *
 * Your task is to determine the health of the robots that survive the collisions, in the same
 * order that the robots were given, i.e. final health of robot 1 (if survived), final health
 * of robot 2 (if survived), and so on. If there are no survivors, return an empty array.
 *
 * Return an array containing the health of the remaining robots (in the order they were given
 * in the input), after no further collisions can occur.
 *
 * Note: The positions may be unsorted.
 */

/**
 * @param {number[]} positions
 * @param {number[]} healths
 * @param {string} directions
 * @return {number[]}
 */
var survivedRobotsHealths = function(positions, healths, directions) {
  const n = positions.length;
  const robots = Array.from({ length: n }, (_, i) => ({
    index: i,
    position: positions[i],
    health: healths[i],
    direction: directions[i]
  }));

  robots.sort((a, b) => a.position - b.position);

  const stack = [];
  for (const robot of robots) {
    if (robot.direction === 'R') {
      stack.push(robot);
      continue;
    }
```

```
    while (stack.length && stack[stack.length - 1].direction === 'R' && robot.health > 0) {
      const last = stack[stack.length - 1];
      if (last.health === robot.health) {
        stack.pop();
        robot.health = 0;
      } else if (last.health > robot.health) {
        last.health--;
        robot.health = 0;
      } else {
        stack.pop();
        robot.health--;
      }
    }
    if (robot.health > 0) {
      stack.push(robot);
    }
  }

  const result = new Array(n).fill(0);
  for (const robot of stack) {
    result[robot.index] = robot.health;
  }

  return result.filter(health => health > 0);
};
```

2753 Count Houses In A Circular Street Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2753. Count Houses in a Circular Street II
 * https://leetcode.com/problems/count-houses-in-a-circular-street-ii/
 * Difficulty: Hard
 *
 * You are given an object street of class Street that represents a circular street and a
positive
 * integer k which represents a maximum bound for the number of houses in that street (in
other
 * words, the number of houses is less than or equal to k). Houses' doors could be open or
closed
 * initially (at least one is open).
 *
 * Initially, you are standing in front of a door to a house on this street. Your task is to
count
 * the number of houses in the street.
 *
 * The class Street contains the following functions which may help you:
 * - void closeDoor(): Close the door of the house you are in front of.
 * - boolean isDoorOpen(): Returns true if the door of the current house is open and false
 *   otherwise.
 * - void moveRight(): Move to the right house.
 *
 * Note that by circular street, we mean if you number the houses from 1 to n, then the right
house
 * of housei is housei+1 for i < n, and the right house of housen is house1.
 *
 * Return ans which represents the number of houses on this street.
 */

/**
 * Definition for a street.
 * class Street {
 *     @param {number[]} doors
 *     constructor(doors);
 *
 *     @return {void}
 *     closeDoor();
 *
 *     @return {boolean}
```

```
 *     isDoorOpen();
 *
 *     @return {void}
 *     moveRight();
 * }
 */
/**
 * @param {Street} street
 * @param {number} k
 * @return {number}
 */
var houseCount = function(street, k) {
  let result = 0;

  while (true) {
    tryMoveToOpen(street, k, true);
    const distance = tryMoveToOpen(street, k, false);

    if (distance > k) {
      return result;
    }

    result = distance;
    street.closeDoor();
  }

  function tryMoveToOpen(street, k, considerCurrent) {
    if (street.isDoorOpen() && considerCurrent) {
      return 0;
    }

    let moves = 1;
    street.moveRight();

    while (!street.isDoorOpen() && moves <= k) {
      street.moveRight();
      moves++;
    }

    return moves;
  }
};
```

2754 Bind Function To Context

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2754. Bind Function to Context
 * https://leetcode.com/problems/bind-function-to-context/
 * Difficulty: Medium
 *
 * Enhance all functions to have the bindPolyfill method. When bindPolyfill is called with
 * a passed object obj, that object becomes the this context for the function.
 *
 * For example, if you had the code:
 * function f() {
 *   console.log('My context is ' + this.ctx);
 * }
 * f();
 *
 * The output would be "My context is undefined". However, if you bound the function:
 * function f() {
 *   console.log('My context is ' + this.ctx);
 * }
 * const boundFunc = f.boundPolyfill({ "ctx": "My Object" })
 * boundFunc();
 *
 * The output should be "My context is My Object".
 *
```

```
 * You may assume that a single non-null object will be passed to the bindPolyfill method.
 *
 * Please solve it without the built-in Function.bind method.
 */

/**
 * @param {Object} obj
 * @return {Function}
 */
Function.prototype.bindPolyfill = function(obj) {
  var fn = this;
  return function() {
    return fn.apply(obj, arguments);
  };
};
```

2755 Deep Merge Of Two Objects

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2755. Deep Merge of Two Objects
 * https://leetcode.com/problems/deep-merge-of-two-objects/
 * Difficulty: Medium
 *
 * Given two values obj1 and obj2, return a deepmerged value.
 *
 * Values should be deepmerged according to these rules:
 * - If the two values are objects, the resulting object should have all the keys that exist
 *   on either object. If a key belongs to both objects, deepmerge the two associated values.
 *   Otherwise, add the key-value pair to the resulting object.
 * - If the two values are arrays, the resulting array should be the same length as the longer
 *   array. Apply the same logic as you would with objects, but treat the indices as keys.
 * - Otherwise the resulting value is obj2.
 *
 * You can assume obj1 and obj2 are the output of JSON.parse().
 */

/**
 * @param {null|boolean|number|string|Array|Object} obj1
 * @param {null|boolean|number|string|Array|Object} obj2
 * @return {null|boolean|number|string|Array|Object}
 */
var deepMerge = function(obj1, obj2) {
  if (obj2 === null || typeof obj2 !== 'object') {
    return obj2;
  }

  if (obj1 === null || typeof obj1 !== 'object') {
    return obj2;
  }

  if (Array.isArray(obj1) !== Array.isArray(obj2)) {
    return obj2;
  }

  if (Array.isArray(obj1)) {
    const maxLength = Math.max(obj1.length, obj2.length);
    const result = [];

    for (let i = 0; i < maxLength; i++) {
      if (i < obj2.length) {
        result[i] = deepMerge(obj1[i], obj2[i]);
      } else {
        result[i] = obj1[i];
      }
    }

    return result;
  }
```

```
  const result = { ...obj1 };

  Object.keys(obj2).forEach(key => {
    result[key] = deepMerge(obj1[key], obj2[key]);
  });

  return result;
};
```

2756 Query Batching

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2756. Query Batching
 * https://leetcode.com/problems/query-batching/
 * Difficulty: Hard
 *
 * Batching multiple small queries into a single large query can be a useful optimization.
 * Write a class QueryBatcher that implements this functionality.
 *
 * The constructor should accept two parameters:
 * - An asynchronous function queryMultiple which accepts an array of string keys input. It
will
 *   resolve with an array of values that is the same length as the input array. Each index
 *   corresponds to the value associated with input[i]. You can assume the promise will never
 *   reject.
 * - A throttle time in milliseconds t.
 *
 * The class has a single method.
 * - async getValue(key). Accepts a single string key and resolves with a single string value.
 *   The keys passed to this function should eventually get passed to the queryMultiple
function.
 *   queryMultiple should never be called consecutively within t milliseconds. The first time
 *   getValue is called, queryMultiple should immediately be called with that single key.
 *   If after t milliseconds, getValue had been called again, all the passed keys should be
 *   passed to queryMultiple and ultimately returned. You can assume every key passed to this
 *   method is unique.
 *
 * The following diagram illustrates how the throttling algorithm works. Each rectangle
represents
 * 100ms. The throttle time is 400ms.
 */

/**
 * @param {Function} queryMultiple
 * @param {number} t
 * @return {void}
 */
var QueryBatcher = function(queryMultiple, t) {
  this.queryMultiple = queryMultiple;
  this.throttleTime = t;
  this.pendingRequests = [];
  this.batchTimer = null;
  this.lastExecutionTime = 0;
};

/**
 * @param {string} key
 * @return {Promise<string>}
 */
QueryBatcher.prototype.getValue = function(key) {
  return new Promise((resolve) => {
    const currentTime = Date.now();
    const timeSinceLastExecution = currentTime - this.lastExecutionTime;
    const remainingDelay = Math.max(0, this.throttleTime - timeSinceLastExecution);

    this.lastExecutionTime = currentTime + remainingDelay;
```

```
    this.pendingRequests.push({ key, resolve });

    if (!this.batchTimer) {
      clearTimeout(this.batchTimer);
      this.batchTimer = setTimeout(() => this.executeBatch(), remainingDelay);
    }
  });
};

/**
 * @return {Promise<void>}
 */
QueryBatcher.prototype.executeBatch = async function() {
  const currentBatch = this.pendingRequests.slice();
  this.pendingRequests = [];
  this.batchTimer = null;

  const keys = currentBatch.map((request) => request.key);
  const resolvers = currentBatch.map((request) => request.resolve);

  this.lastExecutionTime = Date.now();
  const results = await this.queryMultiple(keys);

  for (let i = 0; i < keys.length; i++) {
    resolvers[i](results[i]);
  }
};
```

2757 Generate Circular Array Values

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2757. Generate Circular Array Values
 * https://leetcode.com/problems/generate-circular-array-values/
 * Difficulty: Medium
 *
 * Given a circular array arr and an integer startIndex, return a generator object gen that
 * yields values from arr.
 *
 * The first time gen.next() is called on the generator, it should should yield
arr[startIndex].
 *
 * Each subsequent time gen.next() is called, an integer jump will be passed into the function
 * (Ex: gen.next(-3)).
 * - If jump is positive, the index should increase by that value, however if the current
index
 *   is the last index, it should instead jump to the first index.
 * - If jump is negative, the index should decrease by the magnitude of that value, however if
 *   the current index is the first index, it should instead jump to the last index.
 */

/**
 * @param {Array<number>} arr
 * @param {number} startIndex
 * @yields {number}
 */
var cycleGenerator = function* (arr, startIndex) {
  let currentIndex = startIndex;

  while (true) {
    const jump = yield arr[currentIndex];

    if (jump !== undefined) {
      currentIndex = ((currentIndex + jump) % arr.length + arr.length) % arr.length;
    }
  }
};
```

2758 Next Day

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2758. Next Day
 * https://leetcode.com/problems/next-day/
 * Difficulty: Easy
 *
 * Write code that enhances all date objects such that you can call the date.nextDay() method
 * on any date object and it will return the next day in the format YYYY-MM-DD as a string.
 */

/**
 * @return {string}
 */
Date.prototype.nextDay = function() {
  const nextDate = new Date(this);
  nextDate.setDate(nextDate.getDate() + 1);
  return nextDate.toISOString().split('T')[0];
}
```

2759 Convert Json String To Object

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2759. Convert JSON String to Object
 * https://leetcode.com/problems/convert-json-string-to-object/
 * Difficulty: Hard
 *
 * Given a string str, return parsed JSON parsedStr. You may assume the str is a valid JSON
 * string hence it only includes strings, numbers, arrays, objects, booleans, and null. str
 * will not include invisible characters and escape characters.
 *
 * Please solve it without using the built-in JSON.parse method.
 */

/**
 * @param {string} str
 * @return {null|boolean|number|string|Array|Object}
 */
var jsonParse = function(str) {
  const length = str.length;
  const stack = [];
  let currentValue = null;
  let pendingKey = null;

  for (let index = 0; index < length; index++) {
    if (str[index] === ',') continue;

    if (str[index] === '[' || str[index] === '{') {
      stack.push(currentValue);
      let newContainer = null;
      if (str[index] === '[') newContainer = [];
      else newContainer = {};

      if (Array.isArray(currentValue)) currentValue.push(newContainer);
      else if (pendingKey !== null) {
        currentValue[pendingKey] = newContainer;
        pendingKey = null;
      }
      currentValue = newContainer;
    } else if (str[index] === ']' || str[index] === '}') {
      const previousValue = stack.pop();
```

```
      if (index !== length - 1) currentValue = previousValue;
    } else {
      let parsedValue = null;

      if (str[index] === '"') {
        const startIndex = index + 1;
        while (index + 1 < length && str[index + 1] !== '"') index++;
        parsedValue = str.substring(startIndex, index + 1);
        index++;
      } else if (str[index] === '-' || ('0' <= str[index] && str[index] <= '9')) {
        const startIndex = index;
        while (index + 1 < length && (str[index + 1] === '-'
              || ('0' <= str[index + 1] && str[index + 1] <= '9') || str[index + 1] === '.'))
{
          index++;
        }
        parsedValue = Number(str.substring(startIndex, index + 1));
      } else {
        if (index + 4 <= length && str.substring(index, index + 4) === 'true') {
          parsedValue = true;
        } else if (index + 5 <= length && str.substring(index, index + 5) === 'false') {
          parsedValue = false;
        } else {
          parsedValue = null;
        }
        index += parsedValue || parsedValue === null ? 3 : 4;
      }

      if (str[index + 1] === ':') {
        pendingKey = parsedValue;
        index++;
      } else if (Array.isArray(currentValue)) {
        currentValue.push(parsedValue);
      } else if (pendingKey !== null) {
        currentValue[pendingKey] = parsedValue;
        pendingKey = null;
      } else {
        currentValue = parsedValue;
      }
    }
  }

  return currentValue;
};
```

2763 Sum Of Imbalance Numbers Of All Subarrays

HARD

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2763. Sum of Imbalance Numbers of All Subarrays
 * https://leetcode.com/problems/sum-of-imbalance-numbers-of-all-subarrays/
 * Difficulty: Hard
 *
 * The imbalance number of a 0-indexed integer array arr of length n is defined as the number
 * of indices in sarr = sorted(arr) such that:
 * - 0 <= i < n - 1, and
 * - sarr[i+1] - sarr[i] > 1
 *
 * Here, sorted(arr) is the function that returns the sorted version of arr.
 *
 * Given a 0-indexed integer array nums, return the sum of imbalance numbers of all its
subarrays.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
```

```
 */
var sumImbalanceNumbers = function(nums) {
  const n = nums.length;
  let result = 0;

  for (let i = 0; i < n; i++) {
    const seen = new Set();
    let imbalance = 0;

    for (let j = i; j < n; j++) {
      const current = nums[j];

      if (!seen.has(current)) {
        const hasNext = seen.has(current + 1);
        const hasPrev = seen.has(current - 1);

        if (hasNext && hasPrev) {
          imbalance--;
        } else if (!hasNext && !hasPrev && seen.size > 0) {
          imbalance++;
        }

        seen.add(current);
      }

      result += imbalance;
    }
  }

  return result;
};
```

2764 Is Array A Preorder Of Some Binary Tree

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2764. Is Array a Preorder of Some ■Binary Tree
 * https://leetcode.com/problems/is-array-a-preorder-of-some-binary-tree/
 * Difficulty: Medium
 *
 * Given a 0-indexed integer 2D array nodes, your task is to determine if the given array
represents
 * the preorder traversal of some binary tree.
 *
 * For each index i, nodes[i] = [id, parentId], where id is the id of the node at the index i
and
 * parentId is the id of its parent in the tree (if the node has no parent, then parentId ==
-1).
 *
 * Return true if the given array represents the preorder traversal of some tree, and false
 * otherwise.
 *
 * Note: the preorder traversal of a tree is a recursive way to traverse a tree in which we
first
 * visit the current node, then we do the preorder traversal for the left child, and finally,
we
 * do it for the right child.
 */

/**
 * @param {number[][]} nodes
 * @return {boolean}
 */
var isPreorder = function(nodes) {
  const children = new Map();
  let root = null;

  for (const [id, parentId] of nodes) {
    if (parentId === -1) {
```

```
      root = id;
    } else {
      if (!children.has(parentId)) {
        children.set(parentId, []);
      }
      children.get(parentId).push(id);
    }
  }

  const expectedOrder = [];

  traverse(root);

  for (let i = 0; i < nodes.length; i++) {
    if (nodes[i][0] !== expectedOrder[i]) {
      return false;
    }
  }

  return true;

  function traverse(nodeId) {
    if (nodeId === null) return;

    expectedOrder.push(nodeId);

    const nodeChildren = children.get(nodeId) || [];
    if (nodeChildren.length > 0) {
      traverse(nodeChildren[0]);
    }
    if (nodeChildren.length > 1) {
      traverse(nodeChildren[1]);
    }
  }
};
```

2766 Relocate Marbles

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2766. Relocate Marbles
 * https://leetcode.com/problems/relocate-marbles/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums representing the initial positions of some
 * marbles. You are also given two 0-indexed integer arrays moveFrom and moveTo of equal
length.
 *
 * Throughout moveFrom.length steps, you will change the positions of the marbles. On the ith
 * step, you will move all marbles at position moveFrom[i] to position moveTo[i].
 *
 * After completing all the steps, return the sorted list of occupied positions.
 *
 * Notes:
 * - We call a position occupied if there is at least one marble in that position.
 * - There may be multiple marbles in a single position.
 */

/**
 * @param {number[]} nums
 * @param {number[]} moveFrom
 * @param {number[]} moveTo
 * @return {number[]}
 */
var relocateMarbles = function(nums, moveFrom, moveTo) {
  const set = new Set(nums);

  for (let i = 0; i < moveFrom.length; i++) {
```

```
    set.delete(moveFrom[i]);
    set.add(moveTo[i]);
  }

  return [...set].sort((a, b) => a - b);
};
```

2767 Partition String Into Minimum Beautiful Substrings

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2767. Partition String Into Minimum Beautiful Substrings
 * https://leetcode.com/problems/partition-string-into-minimum-beautiful-substrings/
 * Difficulty: Medium
 *
 * Given a binary string s, partition the string into one or more substrings such that each
 * substring is beautiful.
 *
 * A string is beautiful if:
 * - It doesn't contain leading zeros.
 * - It's the binary representation of a number that is a power of 5.
 *
 * Return the minimum number of substrings in such partition. If it is impossible to partition
 * the string s into beautiful substrings, return -1.
 *
 * A substring is a contiguous sequence of characters in a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumBeautifulSubstrings = function(s) {
  const n = s.length;
  const powersOfFive = new Set();
  let power = 1;
  while (power.toString(2).length <= n) {
    powersOfFive.add(power.toString(2));
    power *= 5;
  }

  const result = findMinPartitions(0);
  return result === Infinity ? -1 : result;

  function findMinPartitions(index) {
    if (index === n) return 0;
    if (s[index] === '0') return Infinity;

    let minPartitions = Infinity;
    for (let end = index + 1; end <= n; end++) {
      const substring = s.slice(index, end);
      if (powersOfFive.has(substring)) {
        const next = findMinPartitions(end);
        if (next !== Infinity) {
          minPartitions = Math.min(minPartitions, 1 + next);
        }
      }
    }

    return minPartitions;
  }
};
```

2769 Find The Maximum Achievable Number

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2769. Find the Maximum Achievable Number
 * https://leetcode.com/problems/find-the-maximum-achievable-number/
 * Difficulty: Easy
 *
 * Given two integers, num and t. A number x is achievable if it can become equal to num
 * after applying the following operation at most t times:
 * - Increase or decrease x by 1, and simultaneously increase or decrease num by 1.
 *
 * Return the maximum possible value of x.
 */

/**
 * @param {number} num
 * @param {number} t
 * @return {number}
 */
var theMaximumAchievableX = function(num, t) {
  return num + 2 * t;
};
```

2773 Height Of Special Binary Tree

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2773. Height of Special Binary Tree
 * https://leetcode.com/problems/height-of-special-binary-tree/
 * Difficulty: Medium
 *
 * You are given a root, which is the root of a special binary tree with n nodes. The nodes of
 * the special binary tree are numbered from 1 to n. Suppose the tree has k leaves in the
 * following order: b1 < b2 < ... < bk.
 *
 * The leaves of this tree have a special property! That is, for every leaf bi, the following
 * conditions hold:
 * - The right child of bi is bi + 1 if i < k, and b1 otherwise.
 * - The left child of bi is bi - 1 if i > 1, and bk otherwise.
 *
 * Return the height of the given tree.
 *
 * Note: The height of a binary tree is the length of the longest path from the root to any
 * other node.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var heightOfTree = function(root) {
  if (!root || (root.left && root.left.right === root)) {
    return 0;
  }
```

```
  return 1 + Math.max(heightOfTree(root.left), heightOfTree(root.right));
};
```

2774 Array Upper Bound

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2774. Array Upper Bound
 * https://leetcode.com/problems/array-upper-bound/
 * Difficulty: Easy
 *
 * Write code that enhances all arrays such that you can call the upperBound() method on any
 * array and it will return the last index of a given target number. nums is a sorted
ascending
 * array of numbers that may contain duplicates. If the target number is not found in the
array,
 * return -1.
 */

/**
 * @param {number} target
 * @return {number}
 */
Array.prototype.upperBound = function(target) {
  let left = 0;
  let right = this.length - 1;
  let result = -1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);

    if (this[mid] === target) {
      result = mid;
      left = mid + 1;
    } else if (this[mid] < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }

  return result;
};
```

2775 Undefined To Null

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2775. Undefined to Null
 * https://leetcode.com/problems/undefined-to-null/
 * Difficulty: Medium
 *
 * Given a deeply nested object or array obj, return the object obj with any undefined
 * values replaced by null.
 *
 * undefined values are handled differently than null values when objects are converted
 * to a JSON string using JSON.stringify(). This function helps ensure serialized data
 * is free of unexpected errors.
 */

/**
 * @param {Object|Array} obj
```

```
 * @return {Object|Array}
 */
var undefinedToNull = function(obj) {
  if (obj === undefined) {
    return null;
  }

  if (obj === null || typeof obj !== 'object') {
    return obj;
  }

  if (Array.isArray(obj)) {
    return obj.map(item => undefinedToNull(item));
  }

  const result = {};
  Object.keys(obj).forEach(key => {
    result[key] = undefinedToNull(obj[key]);
  });

  return result;
};
```

2776 Convert Callback Based Function To Promise Based Function

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2776. Convert Callback Based Function to Promise Based Function
 * https://leetcode.com/problems/convert-callback-based-function-to-promise-based-function/
 * Difficulty: Medium
 *
 * Write a function that accepts another function fn and converts the callback-based function
into
 * a promise-based function.
 *
 * The function fn takes a callback as its first argument, along with any additional arguments
 * args passed as separate inputs.
 *
 * The promisify function returns a new function that should return a promise. The promise
should
 * resolve with the argument passed as the first parameter of the callback when the callback
is
 * invoked without error, and reject with the error when the callback is called with an error
as
 * the second argument.
 *
 * The following is an example of a function that could be passed into promisify.
 * function sum(callback, a, b) {
 *   if (a < 0 || b < 0) {
 *     const err = Error('a and b must be positive');
 *     callback(undefined, err);
 *   } else {
 *     callback(a + b);
 *   }
 * }
 *
 * This is the equivalent code based on promises:
 *
 * async function sum(a, b) {
 *   if (a < 0 || b < 0) {
 *     throw Error('a and b must be positive');
 *   } else {
 *     return a + b;
 *   }
 * }
 * */

/**
 * @param {Function} fn
```

```
 * @return {Function<Promise<number>>}
 */
var promisify = function(fn) {
  return async function(...args) {
    return new Promise((resolve, reject) => {
      fn((result, error) => {
        if (error) {
          reject(error);
        } else {
          resolve(result);
        }
      }, ...args);
    });
  };
};
```

2777 Date Range Generator

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2777. Date Range Generator
 * https://leetcode.com/problems/date-range-generator/
 * Difficulty: Medium
 *
 * Given a start date start, an end date end, and a positive integer step, return a generator
 * object that yields dates in the range from start to end inclusive.
 *
 * The value of step indicates the number of days between consecutive yielded values.
 *
 * All yielded dates must be in the string format YYYY-MM-DD.
 */

/**
 * @param {string} start
 * @param {string} end
 * @param {number} step
 * @yields {string}
 */
var dateRangeGenerator = function* (start, end, step) {
  const startDate = new Date(start);
  const endDate = new Date(end);
  const currentDate = new Date(startDate);

  while (currentDate <= endDate) {
    yield currentDate.toISOString().split('T')[0];
    currentDate.setDate(currentDate.getDate() + step);
  }
};
```

2778 Sum Of Squares Of Special Elements

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2778. Sum of Squares of Special Elements
 * https://leetcode.com/problems/sum-of-squares-of-special-elements/
 * Difficulty: Easy
 *
 * You are given a 1-indexed integer array nums of length n.
 *
 * An element nums[i] of nums is called special if i divides n, i.e. n % i == 0.
 *
 * Return the sum of the squares of all special elements of nums.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var sumOfSquares = function(nums) {
  const n = nums.length;
  let total = 0;

  for (let i = 1; i <= n; i++) {
    if (n % i === 0) {
      total += nums[i - 1] * nums[i - 1];
    }
  }

  return total;
};
```

2779 Maximum Beauty Of An Array After Applying Operation

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2779. Maximum Beauty of an Array After Applying Operation
 * https://leetcode.com/problems/maximum-beauty-of-an-array-after-applying-operation/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums and a non-negative integer k.
 *
 * In one operation, you can do the following:
 * - Choose an index i that hasn't been chosen before from the range [0, nums.length - 1].
 * - Replace nums[i] with any integer from the range [nums[i] - k, nums[i] + k].
 *
 * The beauty of the array is the length of the longest subsequence consisting of equal
elements.
 *
 * Return the maximum possible beauty of the array nums after applying the operation any
number
 * of times.
 *
 * Note that you can apply the operation to each index only once.
 *
 * A subsequence of an array is a new array generated from the original array by deleting some
 * elements (possibly none) without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumBeauty = function(nums, k) {
  nums.sort((a, b) => a - b);
  let result = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    while (nums[right] - nums[left] > 2 * k) {
      left++;
    }
    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

2780 Minimum Index Of A Valid Split

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2780. Minimum Index of a Valid Split
 * https://leetcode.com/problems/minimum-index-of-a-valid-split/
 * Difficulty: Medium
 *
 * An element x of an integer array arr of length m is dominant if more than half the elements
 * of arr have a value of x.
 *
 * You are given a 0-indexed integer array nums of length n with one dominant element.
 *
 * You can split nums at an index i into two arrays nums[0, ..., i] and nums[i + 1, ..., n -
1],
 * but the split is only valid if:
 * - 0 <= i < n - 1
 * - nums[0, ..., i], and nums[i + 1, ..., n - 1] have the same dominant element.
 *
 * Here, nums[i, ..., j] denotes the subarray of nums starting at index i and ending at index
j,
 * both ends being inclusive. Particularly, if j < i then nums[i, ..., j] denotes an empty
subarray.
 *
 * Return the minimum index of a valid split. If no valid split exists, return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumIndex = function(nums) {
  const map = new Map();

  for (const num of nums) {
    map.set(num, (map.get(num) || 0) + 1);
  }

  let dominantElement;
  for (const [num, count] of map) {
    if (count * 2 > nums.length) {
      dominantElement = num;
      break;
    }
  }

  let leftCount = 0;
  for (let i = 0; i < nums.length - 1; i++) {
    leftCount += nums[i] === dominantElement ? 1 : 0;
    const rightCount = map.get(dominantElement) - leftCount;

    if (leftCount * 2 > i + 1 && rightCount * 2 > nums.length - i - 1) {
      return i;
    }
  }

  return -1;
};
```

2782 Number Of Unique Categories

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 2782. Number of Unique Categories
 * https://leetcode.com/problems/number-of-unique-categories/
 * Difficulty: Medium
 *
 * You are given an integer n and an object categoryHandler of class CategoryHandler.
 *
 * There are n elements, numbered from 0 to n - 1. Each element has a category, and your task
 * is to find the number of unique categories.
 *
 * The class CategoryHandler contains the following function, which may help you:
 * - boolean haveSameCategory(integer a, integer b): Returns true if a and b are in the same
 *   category and false otherwise. Also, if either a or b is not a valid number (i.e. it's
greater
 *   than or equal to nor less than 0), it returns false.
 *
 * Return the number of unique categories.
 */

/**
 * Definition for a category handler.
 * class CategoryHandler {
 *     @param {number[]} categories
 *     constructor(categories);
 *
 *     @param {number} a
 *     @param {number} b
 *     @return {boolean}
 *     haveSameCategory(a, b);
 * }
 */
/**
 * @param {number} n
 * @param {CategoryHandler} categoryHandler
 * @return {number}
 */
var numberOfCategories = function(n, categoryHandler) {
  let result = 0;

  for (let j = 0; j < n; j++) {
    let isNewCategory = true;

    for (let i = 0; i < j; i++) {
      if (categoryHandler.haveSameCategory(i, j)) {
        isNewCategory = false;
        break;
      }
    }

    if (isNewCategory) {
      result++;
    }
  }

  return result;
};
```

2784 Check If Array Is Good

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2784. Check if Array is Good
 * https://leetcode.com/problems/check-if-array-is-good/
 * Difficulty: Easy
 *
 * You are given an integer array nums. We consider an array good if it is a permutation of
 * an array base[n].
 *
 * base[n] = [1, 2, ..., n - 1, n, n] (in other words, it is an array of length n + 1 which
 * contains 1 to n - 1 exactly once, plus two occurrences of n). For example, base[1] = [1, 1]
```

```
 * and base[3] = [1, 2, 3, 3].
 *
 * Return true if the given array is good, otherwise return false.
 *
 * Note: A permutation of integers represents an arrangement of these numbers.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isGood = function(nums) {
  const n = Math.max(...nums);
  if (nums.length !== n + 1) return false;

  const frequency = new Array(n + 1).fill(0);
  for (const num of nums) {
    if (num > n) return false;
    frequency[num]++;
  }

  for (let i = 1; i < n; i++) {
    if (frequency[i] !== 1) return false;
  }
  return frequency[n] === 2;
};
```

2785 Sort Vowels In A String

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2785. Sort Vowels in a String
 * https://leetcode.com/problems/sort-vowels-in-a-string/
 * Difficulty: Medium
 *
 * Given a 0-indexed string s, permute s to get a new string t such that:
 * - All consonants remain in their original places. More formally, if there is an index i
 *   with 0 <= i < s.length such that s[i] is a consonant, then t[i] = s[i].
 * - The vowels must be sorted in the nondecreasing order of their ASCII values. More
formally,
 *   for pairs of indices i, j with 0 <= i < j < s.length such that s[i] and s[j] are vowels,
 *   then t[i] must not have a higher ASCII value than t[j].
 *
 * Return the resulting string.
 *
 * The vowels are 'a', 'e', 'i', 'o', and 'u', and they can appear in lowercase or uppercase.
 * Consonants comprise all letters that are not vowels.
 */

/**
 * @param {string} s
 * @return {string}
 */
var sortVowels = function(s) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']);
  const vowelChars = [];

  for (const char of s) {
    if (vowels.has(char)) {
      vowelChars.push(char);
    }
  }

  vowelChars.sort();

  const result = [...s];
  let vowelIndex = 0;
```

```
  for (let i = 0; i < s.length; i++) {
    if (vowels.has(s[i])) {
      result[i] = vowelChars[vowelIndex++];
    }
  }

  return result.join('');
};
```

2787 Ways To Express An Integer As Sum Of Powers

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2787. Ways to Express an Integer as Sum of Powers
 * https://leetcode.com/problems/ways-to-express-an-integer-as-sum-of-powers/
 * Difficulty: Medium
 *
 * Given two positive integers n and x.
 *
 * Return the number of ways n can be expressed as the sum of the xth power of unique positive
 * integers, in other words, the number of sets of unique integers [n1, n2, ..., nk] where
 * n = n1x + n2x + ... + nkx.
 *
 * Since the result can be very large, return it modulo 109 + 7.
 *
 * For example, if n = 160 and x = 3, one way to express n is n = 23 + 33 + 53.
 */

/**
 * @param {number} n
 * @param {number} x
 * @return {number}
 */
var numberOfWays = function(n, x) {
  const MOD = 1e9 + 7;
  const dp = new Array(n + 1).fill(0);
  dp[0] = 1;

  let power = 1;
  while (Math.pow(power, x) <= n) {
    const currentPower = Math.pow(power, x);
    for (let sum = n; sum >= currentPower; sum--) {
      dp[sum] = (dp[sum] + dp[sum - currentPower]) % MOD;
    }
    power++;
  }

  return dp[n];
};
```

2788 Split Strings By Separator

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2788. Split Strings by Separator
 * https://leetcode.com/problems/split-strings-by-separator/
 * Difficulty: Easy
 *
 * Given an array of strings words and a character separator, split each string in words by
 * separator.
 *
 * Return an array of strings containing the new strings formed after the splits, excluding
 * empty strings.
```

```
 *
 * Notes
 * - separator is used to determine where the split should occur, but it is not included as
 *   part of the resulting strings.
 * - A split may result in more than two strings.
 * - The resulting strings must maintain the same order as they were initially given.
 */

/**
 * @param {string[]} words
 * @param {character} separator
 * @return {string[]}
 */
var splitWordsBySeparator = function(words, separator) {
  const result = [];
  for (const word of words) {
    const splitWords = word.split(separator).filter(part => part !== '');
    result.push(...splitWords);
  }
  return result;
};
```

2789 Largest Element In An Array After Merge Operations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2789. Largest Element in an Array after Merge Operations
 * https://leetcode.com/problems/largest-element-in-an-array-after-merge-operations/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums consisting of positive integers.
 *
 * You can do the following operation on the array any number of times:
 * - Choose an integer i such that 0 <= i < nums.length - 1 and nums[i] <= nums[i + 1].
 *   Replace the element nums[i + 1] with nums[i] + nums[i + 1] and delete the element
 *   nums[i] from the array.
 *
 * Return the value of the largest element that you can possibly obtain in the final array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxArrayValue = function(nums) {
  let result = nums[nums.length - 1];
  let currentSum = result;

  for (let i = nums.length - 2; i >= 0; i--) {
    if (nums[i] <= currentSum) {
      currentSum += nums[i];
    } else {
      currentSum = nums[i];
    }
    result = Math.max(result, currentSum);
  }

  return result;
};
```

2791 Count Paths That Can Form A Palindrome In A Tree

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2791. Count Paths That Can Form a Palindrome in a Tree
 * https://leetcode.com/problems/count-paths-that-can-form-a-palindrome-in-a-tree/
 * Difficulty: Hard
 *
 * You are given a tree (i.e. a connected, undirected graph that has no cycles) rooted at node
 * 0 consisting of n nodes numbered from 0 to n - 1. The tree is represented by a 0-indexed
 * array parent of size n, where parent[i] is the parent of node i. Since node 0 is the root,
 * parent[0] == -1.
 *
 * You are also given a string s of length n, where s[i] is the character assigned to the edge
 * between i and parent[i]. s[0] can be ignored.
 *
 * Return the number of pairs of nodes (u, v) such that u < v and the characters assigned to
edges
 * on the path from u to v can be rearranged to form a palindrome.
 *
 * A string is a palindrome when it reads the same backwards as forwards.
 */

/**
 * @param {number[]} parent
 * @param {string} s
 * @return {number}
 */
var countPalindromePaths = function(parent, s) {
  const n = parent.length;
  const graph = Array.from({ length: n }, () => []);
  for (let i = 1; i < n; i++) {
    graph[parent[i]].push(i);
  }

  const freq = new Map([[0, 1]]);
  let result = 0;

  dfs(0, 0);
  return result;

  function dfs(node, mask) {
    for (const child of graph[node]) {
      const newMask = mask ^ (1 << (s[child].charCodeAt(0) - 97));
      result += freq.get(newMask) || 0;
      for (let i = 0; i < 26; i++) {
        result += freq.get(newMask ^ (1 << i)) || 0;
      }
      freq.set(newMask, (freq.get(newMask) || 0) + 1);
      dfs(child, newMask);
    }
  }
};
```

2792 Count Nodes That Are Great Enough

HARD

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2792. Count Nodes That Are Great Enough
 * https://leetcode.com/problems/count-nodes-that-are-great-enough/
 * Difficulty: Hard
 *
 * You are given a root to a binary tree and an integer k. A node of this tree is called great
 * enough if the followings hold:
 * - Its subtree has at least k nodes.
 * - Its value is greater than the value of at least k nodes in its subtree.
 *
 * Return the number of nodes in this tree that are great enough.
 *
 * The node u is in the subtree of the node v, if u == v or v is an ancestor of u.
 */
```

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} k
 * @return {number}
 */
var countGreatEnoughNodes = function(root, k) {
  let result = 0;

  dfs(root);

  return result;

  function dfs(node) {
    if (!node) return [];

    const leftValues = dfs(node.left);
    const rightValues = dfs(node.right);
    const combined = [...leftValues, ...rightValues].sort((a, b) => a - b).slice(0, k);

    if (combined.length >= k && combined[k - 1] < node.val) {
      result++;
    } else {
      combined.push(node.val);
    }

    return combined;
  }
};
```

2794 Create Object From Two Arrays

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2794. Create Object from Two Arrays
 * https://leetcode.com/problems/create-object-from-two-arrays/
 * Difficulty: Easy
 *
 * Given two arrays keysArr and valuesArr, return a new object obj. Each key-value pair in obj
 * should come from keysArr[i] and valuesArr[i].
 *
 * If a duplicate key exists at a previous index, that key-value should be excluded. In other
 * words, only the first key should be added to the object.
 *
 * If the key is not a string, it should be converted into a string by calling String() on it.
 */

/**
 * @param {Array} keysArr
 * @param {Array} valuesArr
 * @return {Object}
 */
var createObject = function(keysArr, valuesArr) {
  const result = {};

  for (let i = 0; i < keysArr.length; i++) {
    const stringKey = String(keysArr[i]);
    if (!(stringKey in result)) {
      result[stringKey] = valuesArr[i];
    }
  }
```

```
  return result;
};
```

2795 Parallel Execution Of Promises For Individual Results Retrieval

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2795. Parallel Execution of Promises for Individual Results Retrieval
 *
https://leetcode.com/problems/parallel-execution-of-promises-for-individual-results-retrieval/
 * Difficulty: Medium
 *
 * Given an array functions, return a promise promise. functions is an array of functions that
 * return promises fnPromise. Each fnPromise can be resolved or rejected.
 *
 * If fnPromise is resolved:
 *
 *     obj = { status: "fulfilled", value: resolved value}
 *
 * If fnPromise is rejected:
 *
 *     obj = { status: "rejected", reason: reason of rejection (catched error message)}
 *
 * The promise should resolve with an array of these objects obj. Each obj in the array should
 * correspond to the promises in the original array function, maintaining the same order.
 *
 * Try to implement it without using the built-in method Promise.allSettled().
 */

/**
 * @param {Array<Function>} functions
 * @return {Promise<Array>}
 */
var promiseAllSettled = function(functions) {
  return new Promise(resolve => {
    const results = new Array(functions.length);
    let completedCount = 0;

    functions.forEach((fn, index) => {
      fn().then(value => results[index] = { status: 'fulfilled', value })
        .catch(reason => results[index] = { status: 'rejected', reason })
        .finally(() => {
          completedCount++;
          if (completedCount === functions.length) {
            resolve(results);
          }
        });
    });
  });
};
```

2796 Repeat String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2796. Repeat String
 * https://leetcode.com/problems/repeat-string/
 * Difficulty: Easy
 *
 * Write code that enhances all strings such that you can call the string.replicate(x) method
 * on any string and it will return repeated string x times.
 *
 * Try to implement it without using the built-in method string.repeat.
```

```
 */

/**
 * @param {number} times
 * @return {string}
 */
String.prototype.replicate = function(times) {
  let result = '';
  for (let i = 0; i < times; i++) {
    result += this;
  }
  return result;
};
```

2797 Partial Function With Placeholders

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2797. Partial Function with Placeholders
 * https://leetcode.com/problems/partial-function-with-placeholders/
 * Difficulty: Easy
 *
 * Given a function fn and an array args, return a function partialFn.
 *
 * Placeholders "_" in the args should be replaced with values from restArgs starting from
 * index 0. Any remaining values in the restArgs should be added at the end of the args.
 *
 * partialFn should return a result of fn. fn should be called with the elements of the
 * modified args passed as separate arguments.
 */

/**
 * @param {Function} fn
 * @param {Array} args
 * @return {Function}
 */
var partial = function(fn, args) {
  return function(...restArgs) {
    const finalArgs = [...args];
    let restIndex = 0;

    for (let i = 0; i < finalArgs.length; i++) {
      if (finalArgs[i] === '_' && restIndex < restArgs.length) {
        finalArgs[i] = restArgs[restIndex];
        restIndex++;
      }
    }

    while (restIndex < restArgs.length) {
      finalArgs.push(restArgs[restIndex]);
      restIndex++;
    }

    return fn(...finalArgs);
  };
};
```

2798 Number Of Employees Who Met The Target

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2798. Number of Employees Who Met the Target
 * https://leetcode.com/problems/number-of-employees-who-met-the-target/
```

```
 * Difficulty: Easy
 *
 * There are n employees in a company, numbered from 0 to n - 1. Each employee i has worked
 * for hours[i] hours in the company.
 *
 * The company requires each employee to work for at least target hours.
 *
 * You are given a 0-indexed array of non-negative integers hours of length n and a
non-negative
 * integer target.
 *
 * Return the integer denoting the number of employees who worked at least target hours.
 */

/**
 * @param {number[]} hours
 * @param {number} target
 * @return {number}
 */
var numberOfEmployeesWhoMetTarget = function(hours, target) {
  let count = 0;

  for (const hoursWorked of hours) {
    if (hoursWorked >= target) count++;
  }

  return count;
};
```

2799 Count Complete Subarrays In An Array

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2799. Count Complete Subarrays in an Array
 * https://leetcode.com/problems/count-complete-subarrays-in-an-array/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of positive integers.
 *
 * We call a subarray of an array complete if the following condition is satisfied:
 * - The number of distinct elements in the subarray is equal to the number of distinct
 *   elements in the whole array.
 *
 * Return the number of complete subarrays.
 *
 * A subarray is a contiguous non-empty part of an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countCompleteSubarrays = function(nums) {
  const totalDistinct = new Set(nums).size;
  const frequency = new Map();
  let completeCount = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    frequency.set(nums[right], (frequency.get(nums[right]) || 0) + 1);

    while (frequency.size === totalDistinct) {
      completeCount += nums.length - right;
      const leftNum = nums[left];
      frequency.set(leftNum, frequency.get(leftNum) - 1);
      if (frequency.get(leftNum) === 0) {
        frequency.delete(leftNum);
      }
      left++;
```

```
    }
  }

  return completeCount;
};
```

2802 Find The K Th Lucky Number

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2802. Find The K-th Lucky Number
 * https://leetcode.com/problems/find-the-k-th-lucky-number/
 * Difficulty: Medium
 *
 * We know that 4 and 7 are lucky digits. Also, a number is called lucky if it contains only
 * lucky digits.
 *
 * You are given an integer k, return the kth lucky number represented as a string.
 */

/**
 * @param {number} k
 * @return {string}
 */
var kthLuckyNumber = function(k) {
  let length = 1;
  let totalNumbers = 0;

  while (totalNumbers + Math.pow(2, length) < k) {
    totalNumbers += Math.pow(2, length);
    length++;
  }

  const position = k - totalNumbers;
  const binaryString = (position - 1).toString(2).padStart(length, '0');

  return binaryString.replace(/0/g, '4').replace(/1/g, '7');
};
```

2803 Factorial Generator

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2803. Factorial Generator
 * https://leetcode.com/problems/factorial-generator/
 * Difficulty: Easy
 *
 * Write a generator function that takes an integer n as an argument and returns a generator
 * object which yields the factorial sequence.
 *
 * The factorial sequence is defined by the relation n! = n * (n-1) * (n-2) * ... * 2 * 1.
 *
 * The factorial of 0 is defined as 1.
 */

/**
 * @param {number} n
 * @yields {number}
 */
function* factorial(n) {
  let currentFactorial = 1;

  for (let i = 1; i <= n; i++) {
```

```
    currentFactorial *= i;
    yield currentFactorial;
  }

  if (n === 0) {
    yield 1;
  }
}
```

2804 Array Prototype Foreach

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2804. Array Prototype ForEach
 * https://leetcode.com/problems/array-prototype-foreach/
 * Difficulty: Easy
 *
 * Write your version of method forEach that enhances all arrays such that you can call the
 * array.forEach(callback, context) method on any array and it will execute callback on each
 * element of the array. Method forEach should not return anything.
 *
 * callback accepts the following arguments:
 * - currentValue - represents the current element being processed in the array. It is the
 *   value of the element in the current iteration.
 * - index - represents the index of the current element being processed in the array.
 * - array - represents the array itself, allowing access to the entire array within the
 *   callback function.
 *
 * The context is the object that should be passed as the function context parameter to the
 * callback function, ensuring that the this keyword within the callback function refers to
 * this context object.
 *
 * Try to implement it without using the built-in array methods.
 */

/**
 * @param {Function} callback
 * @param {Object} context
 * @return {void}
 */
Array.prototype.forEach = function(callback, context) {
  for (let i = 0; i < this.length; i++) {
    callback.call(context, this[i], i, this);
  }
};
```

2805 Custom Interval

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 2805. Custom Interval
 * https://leetcode.com/problems/custom-interval/
 * Difficulty: Medium
 *
 * Function customInterval
 *
 * Given a function fn, a number delay and a number period, return a number id.
 *
 * customInterval is a function that should execute the provided function fn at intervals
based
 * on a linear pattern defined by the formula delay + period * count.
 *
 * The count in the formula represents the number of times the interval has been executed
```

```
 * starting from an initial value of 0.
 *
 * Function customClearInterval
 *
 * Given the id. id is the returned value from the function customInterval.
 *
 * customClearInterval should stop executing provided function fn at intervals.
 *
 * Note: The setTimeout and setInterval functions in Node.js return an object, not a number.
 */

const intervalMap = new Map();
let nextId = 1;

/**
 * @param {Function} fn
 * @param {number} delay
 * @param {number} period
 * @return {number} id
 */
function customInterval(fn, delay, period) {
  let count = 0;
  const id = nextId++;

  function scheduleNext() {
    const nextDelay = delay + period * count;
    const timeoutId = setTimeout(() => {
      fn();
      count++;
      scheduleNext();
    }, nextDelay);
    intervalMap.set(id, timeoutId);
  }

  scheduleNext();
  return id;
}

/**
 * @param {number} id
 * @return {void}
 */
function customClearInterval(id) {
  const timeoutId = intervalMap.get(id);
  if (timeoutId) {
    clearTimeout(timeoutId);
    intervalMap.delete(id);
  }
}
```

2806 Account Balance After Rounded Purchase

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2806. Account Balance After Rounded Purchase
 * https://leetcode.com/problems/account-balance-after-rounded-purchase/
 * Difficulty: Easy
 *
 * Initially, you have a bank account balance of 100 dollars.
 *
 * You are given an integer purchaseAmount representing the amount you will spend on a
purchase
 * in dollars, in other words, its price.
 *
 * When making the purchase, first the purchaseAmount is rounded to the nearest multiple of
10.
 * Let us call this value roundedAmount. Then, roundedAmount dollars are removed from your
bank
 * account.
```

```
 *
 * Return an integer denoting your final bank account balance after this purchase.
 *
 * Notes:
 * - 0 is considered to be a multiple of 10 in this problem.
 * - When rounding, 5 is rounded upward (5 is rounded to 10, 15 is rounded to 20, 25 to 30,
and
 *   so on).
 */

/**
 * @param {number} purchaseAmount
 * @return {number}
 */
var accountBalanceAfterPurchase = function(purchaseAmount) {
  return 100 - (Math.round(purchaseAmount / 10) * 10);
};
```

2807 Insert Greatest Common Divisors In Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2807. Insert Greatest Common Divisors in Linked List
 * https://leetcode.com/problems/insert-greatest-common-divisors-in-linked-list/
 * Difficulty: Medium
 *
 * Given the head of a linked list head, in which each node contains an integer value.
 *
 * Between every pair of adjacent nodes, insert a new node with a value equal to the greatest
 * common divisor of them.
 *
 * Return the linked list after insertion.
 *
 * The greatest common divisor of two numbers is the largest positive integer that evenly
 * divides both numbers.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var insertGreatestCommonDivisors = function(head) {
  let current = head;
  while (current && current.next) {
    const nextNode = current.next;
    const divisor = gcd(current.val, nextNode.val);
    current.next = new ListNode(divisor, nextNode);
    current = nextNode;
  }
  return head;

  function gcd(a, b) {
    while (b) {
      a %= b;
      [a, b] = [b, a];
    }
    return a;
  }
};
```

2810 Faulty Keyboard

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2810. Faulty Keyboard
 * https://leetcode.com/problems/faulty-keyboard/
 * Difficulty: Easy
 *
 * Your laptop keyboard is faulty, and whenever you type a character 'i' on it, it reverses
the
 * string that you have written. Typing other characters works as expected.
 *
 * You are given a 0-indexed string s, and you type each character of s using your faulty
keyboard.
 *
 * Return the final string that will be present on your laptop screen.
 */

/**
 * @param {string} s
 * @return {string}
 */
var finalString = function(s) {
  const result = [];

  for (const char of s) {
    if (char === 'i') {
      result.reverse();
    } else {
      result.push(char);
    }
  }

  return result.join('');
};
```

2814 Minimum Time Takes To Reach Destination Without Drowning

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2814. Minimum Time Takes to Reach Destination Without Drowning
 * https://leetcode.com/problems/minimum-time-takes-to-reach-destination-without-drowning/
 * Difficulty: Hard
 *
 * You are given an n * m 0-indexed grid of string land. Right now, you are standing at the
cell
 * that contains "S", and you want to get to the cell containing "D". There are three other
types
 * of cells in this land:
 * - ".": These cells are empty.
 * - "X": These cells are stone.
 * - "*": These cells are flooded.
 *
 * At each second, you can move to a cell that shares a side with your current cell (if it
exists).
 * Also, at each second, every empty cell that shares a side with a flooded cell becomes
flooded
 * as well.
 *
 * There are two problems ahead of your journey:
 * - You can't step on stone cells.
 * - You can't step on flooded cells since you will drown (also, you can't step on a cell that
 will
```

```
 *   be flooded at the same time as you step on it).
 *
 * Return the minimum time it takes you to reach the destination in seconds, or -1 if it is
 * impossible.
 *
 * Note that the destination will never be flooded.
 */

/**
 * @param {string[][]} land
 * @return {number}
 */
var minimumSeconds = function(land) {
  const rows = land.length;
  const cols = land[0].length;
  const directions = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  const floodQueue = [];
  const startQueue = [];

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (land[i][j] === 'S') {
        startQueue.push([i, j]);
      } else if (land[i][j] === '*') {
        floodQueue.push([i, j]);
      }
    }
  }

  let time = 0;

  while (startQueue.length > 0) {
    time++;

    const floodSize = floodQueue.length;
    for (let f = 0; f < floodSize; f++) {
      const [i, j] = floodQueue.shift();
      for (const [dx, dy] of directions) {
        const x = i + dx;
        const y = j + dy;
        if (isValidPosition(x, y) && land[x][y] === '.') {
          land[x][y] = '*';
          floodQueue.push([x, y]);
        }
      }
    }

    const startSize = startQueue.length;
    for (let s = 0; s < startSize; s++) {
      const [i, j] = startQueue.shift();
      for (const [dx, dy] of directions) {
        const x = i + dx;
        const y = j + dy;
        if (isValidPosition(x, y)) {
          if (land[x][y] === 'D') {
            return time;
          } else if (land[x][y] === '.') {
            land[x][y] = 'S';
            startQueue.push([x, y]);
          }
        }
      }
    }
  }

  return -1;

  function isValidPosition(x, y) {
    return x >= 0 && x < rows && y >= 0 && y < cols;
  }
};
```

2815 Max Pair Sum In An Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2815. Max Pair Sum in an Array
 * https://leetcode.com/problems/max-pair-sum-in-an-array/
 * Difficulty: Easy
 *
 * You are given an integer array nums. You have to find the maximum sum of a pair of numbers
from
 * nums such that the largest digit in both numbers is equal.
 *
 * For example, 2373 is made up of three distinct digits: 2, 3, and 7, where 7 is the largest
among
 * them.
 *
 * Return the maximum sum or -1 if no such pair exists.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSum = function(nums) {
  let result = -1;
  const map = new Map();

  for (const num of nums) {
    const maxDigit = getMaxDigit(num);
    if (!map.has(maxDigit)) {
      map.set(maxDigit, []);
    }
    map.get(maxDigit).push(num);
  }

  for (const numbers of map.values()) {
    if (numbers.length >= 2) {
      numbers.sort((a, b) => b - a);
      result = Math.max(result, numbers[0] + numbers[1]);
    }
  }

  return result;

  function getMaxDigit(num) {
    let max = 0;
    while (num > 0) {
      max = Math.max(max, num % 10);
      num = Math.floor(num / 10);
    }
    return max;
  }
};
```

2816 Double A Number Represented As A Linked List

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2816. Double a Number Represented as a Linked List
 * https://leetcode.com/problems/double-a-number-represented-as-a-linked-list/
 * Difficulty: Medium
```

```
 *
 * You are given the head of a non-empty linked list representing a non-negative integer
without
 * leading zeroes.
 *
 * Return the head of the linked list after doubling it.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var doubleIt = function(head) {
  const reversed = reverseList(head);
  let carry = 0;
  let current = reversed;
  let prev = null;

  while (current) {
    const doubled = current.val * 2 + carry;
    current.val = doubled % 10;
    carry = Math.floor(doubled / 10);
    prev = current;
    current = current.next;
  }

  if (carry) {
    prev.next = new ListNode(carry);
  }

  return reverseList(reversed);

  function reverseList(node) {
    let prev = null;
    let current = node;
    while (current) {
      const next = current.next;
      current.next = prev;
      prev = current;
      current = next;
    }
    return prev;
  }
};
```

2818 Apply Operations To Maximize Score

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2818. Apply Operations to Maximize Score
 * https://leetcode.com/problems/apply-operations-to-maximize-score/
 * Difficulty: Hard
 *
 * You are given an array nums of n positive integers and an integer k.
 *
 * Initially, you start with a score of 1. You have to maximize your score by applying the
following
 * operation at most k times:
 * - Choose any non-empty subarray nums[l, ..., r] that you haven't chosen previously.
 * - Choose an element x of nums[l, ..., r] with the highest prime score. If multiple such
elements
```

```
 *   exist, choose the one with the smallest index.
 * - Multiply your score by x.
 *
 * Here, nums[l, ..., r] denotes the subarray of nums starting at index l and ending at the
index
 * r, both ends being inclusive.
 *
 * The prime score of an integer x is equal to the number of distinct prime factors of x. For
 * example, the prime score of 300 is 3 since 300 = 2 * 2 * 3 * 5 * 5.
 *
 * Return the maximum possible score after applying at most k operations.
 *
 * Since the answer may be large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumScore = function(nums, k) {
  const MOD = 1000000007n;
  const MAX = 100000;
  const powerCache = {};

  const primeScores = new Array(MAX + 1).fill(0);
  for (let prime = 2; prime <= MAX; prime++) {
    if (primeScores[prime] > 0) continue;
    for (let multiple = prime; multiple <= MAX; multiple += prime) {
      primeScores[multiple]++;
    }
  }

  const elements = nums.map(num => [num, primeScores[num], 1]);

  const stack = [-1];
  const n = elements.length;

  for (let i = 0; i <= n; i++) {
    while (
      stack.length > 1 && (i === n || elements[stack[stack.length - 1]][1] < elements[i][1])
    ) {
      const current = stack.pop();
      elements[current][2] = (i - current) * (current - stack[stack.length - 1]);
    }

    stack.push(i);
  }

  elements.sort((a, b) => b[0] - a[0]);

  let result = 1n;
  let remainingOps = k;

  for (let i = 0; remainingOps > 0; i++) {
    const usedOps = Math.min(remainingOps, elements[i][2]);
    remainingOps -= usedOps;

    const cacheKey = `${elements[i][0]},${usedOps}`;
    let power;

    if (cacheKey in powerCache) {
      power = powerCache[cacheKey];
    } else {
      let base = BigInt(elements[i][0]);
      let exp = usedOps;
      power = 1n;

      while (exp > 0) {
        if (exp & 1) {
          power = (power * base) % MOD;
        }
        exp >>= 1;
        base = (base * base) % MOD;
      }
```

```
      powerCache[cacheKey] = power;
    }

    result = (result * power) % MOD;
  }

  return Number(result);
};
```

2819 Minimum Relative Loss After Buying Chocolates

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2819. Minimum Relative Loss After Buying Chocolates
 * https://leetcode.com/problems/minimum-relative-loss-after-buying-chocolates/
 * Difficulty: Hard
 *
 * You are given an integer array prices, which shows the chocolate prices and a 2D integer
 * array queries, where queries[i] = [ki, mi].
 *
 * Alice and Bob went to buy some chocolates, and Alice suggested a way to pay for them, and
 * Bob agreed.
 *
 * The terms for each query are as follows:
 * - If the price of a chocolate is less than or equal to ki, Bob pays for it.
 * - Otherwise, Bob pays ki of it, and Alice pays the rest.
 *
 * Bob wants to select exactly mi chocolates such that his relative loss is minimized, more
 * formally, if, in total, Alice has paid ai and Bob has paid bi, Bob wants to minimize bi -
ai.
 *
 * Return an integer array ans where ans[i] is Bob's minimum relative loss possible for
queries[i].
 */

/**
 * @param {number[]} prices
 * @param {number[][]} queries
 * @return {number[]}
 */
var minimumRelativeLosses = function(prices, queries) {
  prices.sort((a, b) => a - b);
  const n = prices.length;
  const result = [];

  const prefixSum = [0];
  for (let i = 0; i < n; i++) {
    prefixSum[i + 1] = prefixSum[i] + prices[i];
  }

  for (const [k, m] of queries) {
    const split = Math.min(bisectLeft(prices, k), m);
    const cut = bisectLeftWithKey(split, 2 * k, x => prices[x] + prices[n - m + x]);
    const relativeLoss = (m - cut) * 2 * k + prefixSum[n - m + cut] + prefixSum[cut] -
prefixSum[n];
    result.push(relativeLoss);
  }

  return result;

  function bisectLeft(arr, target) {
    let left = 0;
    let right = arr.length;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (arr[mid] < target) {
```

```
        left = mid + 1;
      } else {
        right = mid;
      }
    }

    return left;
  }

  function bisectLeftWithKey(length, target, keyFunc) {
    let left = 0;
    let right = length;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (keyFunc(mid) < target) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }

    return left;
  }
};
```

2821 Delay The Resolution Of Each Promise

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2821. Delay the Resolution of Each Promise
 * https://leetcode.com/problems/delay-the-resolution-of-each-promise/
 * Difficulty: Medium
 *
 * Given an array functions and a number ms, return a new array of functions.
 * - functions is an array of functions that return promises.
 * - ms represents the delay duration in milliseconds. It determines the amount of time to
wait
 *   before resolving or rejecting each promise in the new array.
 *
 * Each function in the new array should return a promise that resolves or rejects after an
 * additional delay of ms milliseconds, preserving the order of the original functions array.
 *
 * The delayAll function should ensure that each promise from functions is executed with a
 * delay, forming the new array of functions returning delayed promises.
 */

/**
 * @param {Array<Function>} functions
 * @param {number} ms
 * @return {Array<Function>}
 */
var delayAll = function(functions, ms) {
  return functions.map(fn => {
    return async function() {
      try {
        const result = await fn();
        await new Promise(resolve => setTimeout(resolve, ms));
        return result;
      } catch (error) {
        await new Promise(resolve => setTimeout(resolve, ms));
        throw error;
      }
    };
  });
};
```

2822 Inversion Of Object

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2822. Inversion of Object
 * https://leetcode.com/problems/inversion-of-object/
 * Difficulty: Easy
 *
 * Given an object or an array obj, return an inverted object or array invertedObj.
 *
 * The invertedObj should have the keys of obj as values and the values of obj as keys.
 * The indices of array should be treated as keys.
 *
 * The function should handle duplicates, meaning that if there are multiple keys in obj
 * with the same value, the invertedObj should map the value to an array containing all
 * corresponding keys.
 *
 * It is guaranteed that the values in obj are only strings.
 */

/**
 * @param {Object|Array} obj
 * @return {Object}
 */
var invertObject = function(obj) {
  const result = {};

  Object.keys(obj).forEach(key => {
    const value = obj[key];

    if (result[value] === undefined) {
      result[value] = key;
    } else if (Array.isArray(result[value])) {
      result[value].push(key);
    } else {
      result[value] = [result[value], key];
    }
  });

  return result;
};
```

2823 Deep Object Filter

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2823. Deep Object Filter
 * https://leetcode.com/problems/deep-object-filter/
 * Difficulty: Medium
 *
 * Given an object or an array obj and a function fn, return a filtered object or array
 * filteredObject.
 *
 * Function deepFilter should perform a deep filter operation on the obj. The deep filter
 * operation should remove properties for which the output of the filter function fn is
 * false, as well as any empty objects or arrays that remain after the keys have been removed.
 *
 * If the deep filter operation results in an empty object or array, with no remaining
 * properties, deepFilter should return undefined to indicate that there is no valid data
 * left in the filteredObject.
 */

/**
```

```
 * @param {Object|Array} obj
 * @param {Function} fn
 * @return {Object|Array|undefined}
 */
var deepFilter = function(obj, fn) {
  if (Array.isArray(obj)) {
    const filteredArray = obj
      .map(item => deepFilter(item, fn))
      .filter(item => item !== undefined);

    return filteredArray.length > 0 ? filteredArray : undefined;
  }

  if (typeof obj === 'object' && obj !== null) {
    const filteredObject = {};
    let hasValidProperties = false;

    Object.keys(obj).forEach(key => {
      const filteredValue = deepFilter(obj[key], fn);
      if (filteredValue !== undefined) {
        filteredObject[key] = filteredValue;
        hasValidProperties = true;
      }
    });

    return hasValidProperties ? filteredObject : undefined;
  }

  return fn(obj) ? obj : undefined;
};
```

2824 Count Pairs Whose Sum Is Less Than Target

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2824. Count Pairs Whose Sum is Less than Target
 * https://leetcode.com/problems/count-pairs-whose-sum-is-less-than-target/
 * Difficulty: Easy
 *
 * Given a 0-indexed integer array nums of length n and an integer target, return the number
 * of pairs (i, j) where 0 <= i < j < n and nums[i] + nums[j] < target.
 */

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number}
 */
var countPairs = function(nums, target) {
  let result = 0;
  let left = 0;
  let right = nums.length - 1;

  nums.sort((a, b) => a - b);

  while (left < right) {
    if (nums[left] + nums[right] < target) {
      result += right - left;
      left++;
    } else {
      right--;
    }
  }

  return result;
};
```

2825 Make String A Subsequence Using Cyclic Increments

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2825. Make String a Subsequence Using Cyclic Increments
 * https://leetcode.com/problems/make-string-a-subsequence-using-cyclic-increments/
 * Difficulty: Medium
 *
 * You are given two 0-indexed strings str1 and str2.
 *
 * In an operation, you select a set of indices in str1, and for each index i in the set,
 * increment str1[i] to the next character cyclically. That is 'a' becomes 'b', 'b' becomes
 * 'c', and so on, and 'z' becomes 'a'.
 *
 * Return true if it is possible to make str2 a subsequence of str1 by performing the
operation
 * at most once, and false otherwise.
 *
 * Note: A subsequence of a string is a new string that is formed from the original string by
 * deleting some (possibly none) of the characters without disturbing the relative positions
 * of the remaining characters.
 */

/**
 * @param {string} str1
 * @param {string} str2
 * @return {boolean}
 */
var canMakeSubsequence = function(str1, str2) {
  let i = 0;
  let j = 0;

  while (i < str1.length && j < str2.length) {
    const curr = str1[i].charCodeAt(0);
    const next = curr === 122 ? 97 : curr + 1;
    if (str1[i] === str2[j] || String.fromCharCode(next) === str2[j]) {
      j++;
    }
    i++;
  }

  return j === str2.length;
};
```

2826 Sorting Three Groups

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2826. Sorting Three Groups
 * https://leetcode.com/problems/sorting-three-groups/
 * Difficulty: Medium
 *
 * You are given an integer array nums. Each element in nums is 1, 2 or 3. In each operation,
 * you can remove an element from nums. Return the minimum number of operations to make nums
 * non-decreasing.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumOperations = function(nums) {
  const n = nums.length;
```

```
  const dp = new Array(n + 1).fill(0).map(() => new Array(4).fill(Infinity));
  dp[0][0] = 0;

  for (let i = 0; i < n; i++) {
    for (let prev = 0; prev <= 3; prev++) {
      for (let curr = 1; curr <= 3; curr++) {
        if (curr >= prev) {
          dp[i + 1][curr] = Math.min(
            dp[i + 1][curr],
            dp[i][prev] + (nums[i] === curr ? 0 : 1)
          );
        }
      }
    }
  }

  return Math.min(...dp[n].slice(1));
};
```

2828 Check If A String Is An Acronym Of Words

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2828. Check if a String Is an Acronym of Words
 * https://leetcode.com/problems/check-if-a-string-is-an-acronym-of-words/
 * Difficulty: Easy
 *
 * Given an array of strings words and a string s, determine if s is an acronym of words.
 *
 * The string s is considered an acronym of words if it can be formed by concatenating the
 * first character of each string in words in order. For example, "ab" can be formed from
 * ["apple", "banana"], but it can't be formed from ["bear", "aardvark"].
 *
 * Return true if s is an acronym of words, and false otherwise.
 */

/**
 * @param {string[]} words
 * @param {string} s
 * @return {boolean}
 */
var isAcronym = function(words, s) {
  if (words.length !== s.length) return false;
  return words.every((word, index) => word[0] === s[index]);
};
```

2829 Determine The Minimum Sum Of A K Avoiding Array

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2829. Determine the Minimum Sum of a k-avoiding Array
 * https://leetcode.com/problems/determine-the-minimum-sum-of-a-k-avoiding-array/
 * Difficulty: Medium
 *
 * You are given two integers, n and k.
 *
 * An array of distinct positive integers is called a k-avoiding array if there does not exist
 * any pair of distinct elements that sum to k.
 *
 * Return the minimum possible sum of a k-avoiding array of length n.
 */
```

```javascript
/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var minimumSum = function(n, k) {
  const set = new Set();
  let result = 0;
  let num = 1;

  for (let i = 0; i < n; i++) {
    while (set.has(k - num)) {
      num++;
    }
    result += num;
    set.add(num);
    num++;
  }

  return result;
};
```

2832 Maximal Range That Each Element Is Maximum In It

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```javascript
/**
 * 2832. Maximal Range That Each Element Is Maximum in It
 * https://leetcode.com/problems/maximal-range-that-each-element-is-maximum-in-it/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums of distinct integers.
 *
 * Let us define a 0-indexed array ans of the same length as nums in the following way:
 * - ans[i] is the maximum length of a subarray nums[l..r], such that the maximum element
 *   in that subarray is equal to nums[i].
 *
 * Return the array ans.
 *
 * Note that a subarray is a contiguous part of the array.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var maximumLengthOfRanges = function(nums) {
  const n = nums.length;
  const leftBounds = new Array(n);
  const rightBounds = new Array(n);
  const stack = [];

  for (let i = 0; i < n; i++) {
    while (stack.length > 0 && nums[stack[stack.length - 1]] < nums[i]) {
      stack.pop();
    }
    leftBounds[i] = stack.length > 0 ? stack[stack.length - 1] + 1 : 0;
    stack.push(i);
  }

  stack.length = 0;

  for (let i = n - 1; i >= 0; i--) {
    while (stack.length > 0 && nums[stack[stack.length - 1]] < nums[i]) {
      stack.pop();
    }
    rightBounds[i] = stack.length > 0 ? stack[stack.length - 1] - 1 : n - 1;
    stack.push(i);
  }
```

```
  const result = new Array(n);
  for (let i = 0; i < n; i++) {
    result[i] = rightBounds[i] - leftBounds[i] + 1;
  }

  return result;
};
```

2833 Furthest Point From Origin

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2833. Furthest Point From Origin
 * https://leetcode.com/problems/furthest-point-from-origin/
 * Difficulty: Easy
 *
 * You are given a string moves of length n consisting only of characters 'L', 'R', and '_'.
 * The string represents your movement on a number line starting from the origin 0.
 *
 * In the ith move, you can choose one of the following directions:
 * - move to the left if moves[i] = 'L' or moves[i] = '_'
 * - move to the right if moves[i] = 'R' or moves[i] = '_'
 *
 * Return the distance from the origin of the furthest point you can get to after n moves.
 */

/**
 * @param {string} moves
 * @return {number}
 */
var furthestDistanceFromOrigin = function(moves) {
  let leftCount = 0;
  let rightCount = 0;
  let wildCount = 0;

  for (const move of moves) {
    if (move === 'L') leftCount++;
    else if (move === 'R') rightCount++;
    else wildCount++;
  }

  return Math.abs(leftCount - rightCount) + wildCount;
};
```

2838 Maximum Coins Heroes Can Collect

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2838. Maximum Coins Heroes Can Collect
 * https://leetcode.com/problems/maximum-coins-heroes-can-collect/
 * Difficulty: Medium
 *
 * There is a battle and n heroes are trying to defeat m monsters. You are given two 1-indexed
 * arrays of positive integers heroes and monsters of length n and m, respectively. heroes[i]
 * is the power of ith hero, and monsters[i] is the power of ith monster.
 *
 * The ith hero can defeat the jth monster if monsters[j] <= heroes[i].
 *
 * You are also given a 1-indexed array coins of length m consisting of positive integers.
 * coins[i] is the number of coins that each hero earns after defeating the ith monster.
 *
 * Return an array ans of length n where ans[i] is the maximum number of coins that the ith
```

```
 * hero can collect from this battle.
 *
 * Notes
 * - The health of a hero doesn't get reduced after defeating a monster.
 * - Multiple heroes can defeat a monster, but each monster can be defeated by a given hero
 *   only once.
 */

/**
 * @param {number[]} heroes
 * @param {number[]} monsters
 * @param {number[]} coins
 * @return {number[]}
 */
var maximumCoins = function(heroes, monsters, coins) {
  const monsterCoinPairs = monsters.map((monster, i) => [monster, coins[i]]);
  monsterCoinPairs.sort((a, b) => a[0] - b[0]);
  const sortedMonsters = monsterCoinPairs.map(pair => pair[0]);
  const sortedCoins = monsterCoinPairs.map(pair => pair[1]);

  const prefixSum = [0];
  for (let i = 0; i < sortedCoins.length; i++) {
    prefixSum[i + 1] = prefixSum[i] + sortedCoins[i];
  }

  const result = [];
  for (const heroPower of heroes) {
    const defeatedCount = bisectRight(sortedMonsters, heroPower);
    result.push(prefixSum[defeatedCount]);
  }

  return result;

  function bisectRight(arr, target) {
    let left = 0;
    let right = arr.length;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (arr[mid] <= target) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }

    return left;
  }
};
```

2839 Check If Strings Can Be Made Equal With Operations I

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2839. Check if Strings Can be Made Equal With Operations I
 * https://leetcode.com/problems/check-if-strings-can-be-made-equal-with-operations-i/
 * Difficulty: Easy
 *
 * You are given two strings s1 and s2, both of length 4, consisting of lowercase English
 * letters.
 *
 * You can apply the following operation on any of the two strings any number of times:
 * - Choose any two indices i and j such that j - i = 2, then swap the two characters at those
 *   indices in the string.
 *
 * Return true if you can make the strings s1 and s2 equal, and false otherwise.
 */

/**
```

```
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var canBeEqual = function(s1, s2) {
  const firstPair = [s1[0], s1[2]].sort().join() === [s2[0], s2[2]].sort().join();
  const secondPair = [s1[1], s1[3]].sort().join() === [s2[1], s2[3]].sort().join();
  return firstPair && secondPair;
};
```

2840 Check If Strings Can Be Made Equal With Operations Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2840. Check if Strings Can be Made Equal With Operations II
 * https://leetcode.com/problems/check-if-strings-can-be-made-equal-with-operations-ii/
 * Difficulty: Medium
 *
 * You are given two strings s1 and s2, both of length n, consisting of lowercase English
letters.
 *
 * You can apply the following operation on any of the two strings any number of times:
 * - Choose any two indices i and j such that i < j and the difference j - i is even, then
swap the
 *   two characters at those indices in the string.
 *
 * Return true if you can make the strings s1 and s2 equal, and false otherwise.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var checkStrings = function(s1, s2) {
  const evenChars1 = [];
  const oddChars1 = [];
  const evenChars2 = [];
  const oddChars2 = [];

  for (let i = 0; i < s1.length; i++) {
    if (i % 2 === 0) {
      evenChars1.push(s1[i]);
      evenChars2.push(s2[i]);
    } else {
      oddChars1.push(s1[i]);
      oddChars2.push(s2[i]);
    }
  }

  return evenChars1.sort().join('') === evenChars2.sort().join('')
    && oddChars1.sort().join('') === oddChars2.sort().join('');
};
```

2841 Maximum Sum Of Almost Unique Subarray

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2841. Maximum Sum of Almost Unique Subarray
 * https://leetcode.com/problems/maximum-sum-of-almost-unique-subarray/
 * Difficulty: Medium
 *
 * You are given an integer array nums and two positive integers m and k.
```

```
 *
 * Return the maximum sum out of all almost unique subarrays of length k of nums. If no such
 * subarray exists, return 0.
 *
 * A subarray of nums is almost unique if it contains at least m distinct elements.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} m
 * @param {number} k
 * @return {number}
 */
var maxSum = function(nums, m, k) {
  const map = new Map();
  let subarraySum = 0;
  let result = 0;

  for (let i = 0; i < k; i++) {
    map.set(nums[i], (map.get(nums[i]) || 0) + 1);
    subarraySum += nums[i];
  }

  if (map.size >= m) {
    result = subarraySum;
  }

  for (let i = k; i < nums.length; i++) {
    map.set(nums[i - k], map.get(nums[i - k]) - 1);
    if (map.get(nums[i - k]) === 0) {
      map.delete(nums[i - k]);
    }

    map.set(nums[i], (map.get(nums[i]) || 0) + 1);
    subarraySum = subarraySum - nums[i - k] + nums[i];

    if (map.size >= m) {
      result = Math.max(result, subarraySum);
    }
  }

  return result;
};
```

2843 Count Symmetric Integers

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2843. Count Symmetric Integers
 * https://leetcode.com/problems/count-symmetric-integers/
 * Difficulty: Easy
 *
 * You are given two positive integers low and high.
 *
 * An integer x consisting of 2 * n digits is symmetric if the sum of the first n digits of x
is
 * equal to the sum of the last n digits of x. Numbers with an odd number of digits are never
 * symmetric.
 *
 * Return the number of symmetric integers in the range [low, high].
 */

/**
 * @param {number} low
 * @param {number} high
 * @return {number}
 */
```

```
var countSymmetricIntegers = function(low, high) {
  let result = 0;

  for (let number = low; number <= high; number++) {
    if (isSymmetric(number)) result++;
  }

  return result;

  function isSymmetric(number) {
    const digits = number.toString().split('');
    const length = digits.length;
    if (length % 2 !== 0) return false;
    const mid = length / 2;
    const firstHalfSum = digits.slice(0, mid).reduce((sum, digit) => sum + Number(digit), 0);
    const secondHalfSum = digits.slice(mid).reduce((sum, digit) => sum + Number(digit), 0);

    return firstHalfSum === secondHalfSum;
  }
};
```

2845 Count Of Interesting Subarrays

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2845. Count of Interesting Subarrays
 * https://leetcode.com/problems/count-of-interesting-subarrays/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums, an integer modulo, and an integer k.
 *
 * Your task is to find the count of subarrays that are interesting.
 *
 * A subarray nums[l..r] is interesting if the following condition holds:
 * - Let cnt be the number of indices i in the range [l, r] such that nums[i] % modulo == k.
 *   Then, cnt % modulo == k.
 *
 * Return an integer denoting the count of interesting subarrays.
 *
 * Note: A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} modulo
 * @param {number} k
 * @return {number}
 */
var countInterestingSubarrays = function(nums, modulo, k) {
  const prefixCounts = new Map([[0, 1]]);
  let currentSum = 0;
  let result = 0;

  for (const num of nums) {
    currentSum = (currentSum + (num % modulo === k ? 1 : 0)) % modulo;
    result += prefixCounts.get((currentSum - k + modulo) % modulo) || 0;
    prefixCounts.set(currentSum, (prefixCounts.get(currentSum) || 0) + 1);
  }

  return result;
};
```

2846 Minimum Edge Weight Equilibrium Queries In A Tree

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2846. Minimum Edge Weight Equilibrium Queries in a Tree
 * https://leetcode.com/problems/minimum-edge-weight-equilibrium-queries-in-a-tree/
 * Difficulty: Hard
 *
 * There is an undirected tree with n nodes labeled from 0 to n - 1. You are given the integer
 n and
 * a 2D integer array edges of length n - 1, where edges[i] = [ui, vi, wi] indicates that
there is
 * an edge between nodes ui and vi with weight wi in the tree.
 *
 * You are also given a 2D integer array queries of length m, where queries[i] = [ai, bi]. For
 each
 * query, find the minimum number of operations required to make the weight of every edge on
the
 * path from ai to bi equal. In one operation, you can choose any edge of the tree and change
its
 * weight to any value.
 *
 * Note that:
 * - Queries are independent of each other, meaning that the tree returns to its initial state
 on
 *   each new query.
 * - The path from ai to bi is a sequence of distinct nodes starting with node ai and ending
with
 *   node bi such that every two adjacent nodes in the sequence share an edge in the tree.
 *
 * Return an array answer of length m where answer[i] is the answer to the ith query.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[][]} queries
 * @return {number[]}
 */
var minOperationsQueries = function(n, edges, queries) {
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v, w] of edges) {
    graph[u].push([v, w]);
    graph[v].push([u, w]);
  }

  const parent = new Array(n).fill(-1);
  const weightCount = new Array(n).fill().map(() => new Array(27).fill(0));
  const depth = new Array(n).fill(0);

  dfs(0, -1, 0);

  const result = [];
  for (const [u, v] of queries) {
    const ancestor = lca(u, v);
    const counts = Array(27).fill(0);
    for (let i = 1; i <= 26; i++) {
      counts[i] = weightCount[u][i] + weightCount[v][i] - 2 * weightCount[ancestor][i];
    }
    const total = depth[u] + depth[v] - 2 * depth[ancestor];
    const maxCount = Math.max(...counts);
    result.push(total - maxCount);
  }

  return result;

  function dfs(node, par, dep) {
    parent[node] = par;
    depth[node] = dep;
    for (const [next, w] of graph[node]) {
      if (next !== par) {
        weightCount[next] = [...weightCount[node]];
        weightCount[next][w]++;
        dfs(next, node, dep + 1);
```

```
      }
    }
  }
  function lca(u, v) {
    if (depth[u] < depth[v]) [u, v] = [v, u];
    while (depth[u] > depth[v]) u = parent[u];
    while (u !== v) {
      u = parent[u];
      v = parent[v];
    }
    return u;
  }
};
```

2847 Smallest Number With Given Digit Product

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2847. Smallest Number With Given Digit Product
 * https://leetcode.com/problems/smallest-number-with-given-digit-product/
 * Difficulty: Medium
 *
 * Given a positive integer n, return a string representing the smallest positive integer
 * such that the product of its digits is equal to n, or "-1" if no such number exists.
 */

/**
 * @param {number} n
 * @return {string}
 */
var smallestNumber = function(n) {
  if (n < 10) return n.toString();

  const digits = [];
  let remaining = BigInt(n);
  for (let digit = 9; digit > 1 && remaining > 1n; digit--) {
    const bigDigit = BigInt(digit);
    while (remaining % bigDigit === 0n) {
      digits.push(digit);
      remaining /= bigDigit;
    }
  }

  if (remaining !== 1n) return '-1';

  return digits.reverse().join('');
};
```

2848 Points That Intersect With Cars

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2848. Points That Intersect With Cars
 * https://leetcode.com/problems/points-that-intersect-with-cars/
 * Difficulty: Easy
 *
 * You are given a 0-indexed 2D integer array nums representing the coordinates of the cars
 * parking on a number line. For any index i, nums[i] = [starti, endi] where starti is the
 * starting point of the ith car and endi is the ending point of the ith car.
 *
 * Return the number of integer points on the line that are covered with any part of a car.
 */
```

```
/**
 * @param {number[][]} nums
 * @return {number}
 */
var numberOfPoints = function(nums) {
  const set = new Set();

  for (const [start, end] of nums) {
    for (let i = start; i <= end; i++) {
      set.add(i);
    }
  }

  return set.size;
};
```

2850 Minimum Moves To Spread Stones Over Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2850. Minimum Moves to Spread Stones Over Grid
 * https://leetcode.com/problems/minimum-moves-to-spread-stones-over-grid/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D integer matrix grid of size 3 * 3, representing the number of
 * stones in each cell. The grid contains exactly 9 stones, and there can be multiple stones
 * in a single cell.
 *
 * In one move, you can move a single stone from its current cell to any other cell if the two
 * cells share a side.
 *
 * Return the minimum number of moves required to place one stone in each cell.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minimumMoves = function(grid) {
  const sources = [];
  const targets = [];

  for (let i = 0; i < 3; i++) {
    for (let j = 0; j < 3; j++) {
      if (grid[i][j] > 1) {
        for (let k = 1; k < grid[i][j]; k++) {
          sources.push([i, j]);
        }
      } else if (grid[i][j] === 0) {
        targets.push([i, j]);
      }
    }
  }

  let minMoves = Infinity;

  permute(0, 0);
  return minMoves;

  function permute(index, moves) {
    if (index === sources.length) {
      minMoves = Math.min(minMoves, moves);
      return;
    }

    for (let i = 0; i < targets.length; i++) {
      if (targets[i]) {
        const [si, sj] = sources[index];
        const [ti, tj] = targets[i];
```

```
        const dist = Math.abs(si - ti) + Math.abs(sj - tj);
        const temp = targets[i];
        targets[i] = null;
        permute(index + 1, moves + dist);
        targets[i] = temp;
      }
    }
  }
};
```

2852 Sum Of Remoteness Of All Cells

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2852. Sum of Remoteness of All Cells
 * https://leetcode.com/problems/sum-of-remoteness-of-all-cells/
 * Difficulty: Medium
 *
 * You are given a 0-indexed matrix grid of order n * n. Each cell in this matrix has a
 * value grid[i][j], which is either a positive integer or -1 representing a blocked cell.
 *
 * You can move from a non-blocked cell to any non-blocked cell that shares an edge.
 *
 * For any cell (i, j), we represent its remoteness as R[i][j] which is defined as the
following:
 * - If the cell (i, j) is a non-blocked cell, R[i][j] is the sum of the values grid[x][y]
such
 *   that there is no path from the non-blocked cell (x, y) to the cell (i, j).
 * - For blocked cells, R[i][j] == 0.
 *
 * Return the sum of R[i][j] over all cells.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var sumRemoteness = function(grid) {
  const n = grid.length;
  const m = grid[0].length;
  const directions = [[-1, 0], [0, 1], [1, 0], [0, -1]];

  let totalSum = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < m; j++) {
      totalSum += Math.max(grid[i][j], 0);
    }
  }

  let result = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < m; j++) {
      if (grid[i][j] > 0) {
        const [componentSum, componentCount] = dfs(i, j, grid);
        result += (totalSum - componentSum) * componentCount;
      }
    }
  }

  return result;

  function dfs(x, y, grid) {
    if (x < 0 || x >= n || y < 0 || y >= m || grid[x][y] < 0) {
      return [0, 0];
    }

    const cellValue = grid[x][y];
    grid[x][y] = -1;
```

```
    let componentSum = cellValue;
    let componentCount = 1;

    for (const [dx, dy] of directions) {
      const [sum, count] = dfs(x + dx, y + dy, grid);
      componentSum += sum;
      componentCount += count;
    }

    return [componentSum, componentCount];
  }
};
```

2855 Minimum Right Shifts To Sort The Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2855. Minimum Right Shifts to Sort the Array
 * https://leetcode.com/problems/minimum-right-shifts-to-sort-the-array/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array nums of length n containing distinct positive integers.
 * Return the minimum number of right shifts required to sort nums and -1 if this is not
 * possible.
 *
 * A right shift is defined as shifting the element at index i to index (i + 1) % n, for
 * all indices.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumRightShifts = function(nums) {
  const n = nums.length;
  let breakPoint = 0;

  for (let i = 1; i < n; i++) {
    if (nums[i] < nums[i - 1]) {
      breakPoint++;
      if (breakPoint > 1) return -1;
    }
  }

  if (breakPoint === 0) return 0;
  if (nums[n - 1] > nums[0]) return -1;

  return n - (nums.indexOf(Math.min(...nums)));
};
```

2858 Minimum Edge Reversals So Every Node Is Reachable

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2858. Minimum Edge Reversals So Every Node Is Reachable
 * https://leetcode.com/problems/minimum-edge-reversals-so-every-node-is-reachable/
 * Difficulty: Hard
 *
 * There is a simple directed graph with n nodes labeled from 0 to n - 1. The graph would form
 * a tree if its edges were bi-directional.
 *
 * You are given an integer n and a 2D integer array edges, where edges[i] = [ui, vi]
represents
```

```
 * a directed edge going from node ui to node vi.
 *
 * An edge reversal changes the direction of an edge, i.e., a directed edge going from node ui
 to
 * node vi becomes a directed edge going from node vi to node ui.
 *
 * For every node i in the range [0, n - 1], your task is to independently calculate the
minimum
 * number of edge reversals required so it is possible to reach any other node starting from
node
 * i through a sequence of directed edges.
 *
 * Return an integer array answer, where answer[i] is the minimum number of edge reversals
required
 * so it is possible to reach any other node starting from node i through a sequence of
directed
 * edges.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number[]}
 */
var minEdgeReversals = function(n, edges) {
  const graph = Array.from({ length: n }, () => []);

  for (const [u, v] of edges) {
    graph[u].push([v, 0]);
    graph[v].push([u, 1]);
  }

  const result = new Array(n);
  const rootReversals = dfs(0, -1);

  reroot(0, -1, rootReversals);

  return result;
  function dfs(node, parent) {
    let reversals = 0;
    for (const [neighbor, cost] of graph[node]) {
      if (neighbor !== parent) {
        reversals += cost + dfs(neighbor, node);
      }
    }
    return reversals;
  }

  function reroot(node, parent, parentReversals) {
    result[node] = parentReversals;
    for (const [neighbor, cost] of graph[node]) {
      if (neighbor !== parent) {
        const childReversals = result[node] - cost + (1 - cost);
        reroot(neighbor, node, childReversals);
      }
    }
  }
};
```

2859 Sum Of Values At Indices With K Set Bits

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2859. Sum of Values at Indices With K Set Bits
 * https://leetcode.com/problems/sum-of-values-at-indices-with-k-set-bits/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums and an integer k.
 *
```

```
 * Return an integer that denotes the sum of elements in nums whose corresponding indices have
 * exactly k set bits in their binary representation.
 *
 * The set bits in an integer are the 1's present when it is written in binary.
 *
 * For example, the binary representation of 21 is 10101, which has 3 set bits.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var sumIndicesWithKSetBits = function(nums, k) {
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    if (i.toString(2).split('1').length - 1 === k) {
      result += nums[i];
    }
  }

  return result;
};
```

2860 Happy Students

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2860. Happy Students
 * https://leetcode.com/problems/happy-students/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of length n where n is the total number of
 * students in the class. The class teacher tries to select a group of students so that
 * all the students remain happy.
 *
 * The ith student will become happy if one of these two conditions is met:
 * - The student is selected and the total number of selected students is strictly greater
 *   than nums[i].
 * - The student is not selected and the total number of selected students is strictly less
 *   than nums[i].
 *
 * Return the number of ways to select a group of students so that everyone remains happy.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countWays = function(nums) {
  nums.sort((a, b) => a - b);
  let result = 0;
  let selected = 0;

  if (0 < nums[0]) result++;

  for (let i = 0; i < nums.length; i++) {
    selected++;
    if (selected > nums[i] && (i + 1 === nums.length || selected < nums[i + 1])) {
      result++;
    }
  }

  return result;
};
```

2862 Maximum Element Sum Of A Complete Subset Of Indices

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2862. Maximum Element-Sum of a Complete Subset of Indices
 * https://leetcode.com/problems/maximum-element-sum-of-a-complete-subset-of-indices/
 * Difficulty: Hard
 *
 * You are given a 1-indexed array nums. Your task is to select a complete subset from nums
where
 * every pair of selected indices multiplied is a perfect square,. i. e. if you select ai and
aj,
 * i * j must be a perfect square.
 *
 * Return the sum of the complete subset with the maximum sum.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumSum = function(nums) {
  const n = nums.length;
  const map = new Map();

  for (let i = 1; i <= n; i++) {
    let factors = 1;
    let num = i;

    for (let p = 2; p * p <= num; p++) {
      let count = 0;
      while (num % p === 0) {
        count++;
        num /= p;
      }
      if (count % 2 === 1) factors *= p;
    }
    if (num > 1) factors *= num;

    if (!map.has(factors)) map.set(factors, []);
    map.get(factors).push(nums[i - 1]);
  }

  let result = 0;
  for (const group of map.values()) {
    const sum = group.reduce((a, b) => a + b, 0);
    result = Math.max(result, sum);
  }

  return result;
};
```

2863 Maximum Length Of Semi Decreasing Subarrays

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2863. Maximum Length of Semi-Decreasing Subarrays
 * https://leetcode.com/problems/maximum-length-of-semi-decreasing-subarrays/
 * Difficulty: Medium
 *
 * You are given an integer array nums.
```

```
 *
 * Return the length of the longest semi-decreasing subarray of nums, and 0 if there are no
 * such subarrays.
 * - A subarray is a contiguous non-empty sequence of elements within an array.
 * - A non-empty array is semi-decreasing if its first element is strictly greater than its
 *   last element.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubarrayLength = function(nums) {
  const n = nums.length;
  const stack = [];

  for (let i = n - 1; i >= 0; i--) {
    if (stack.length === 0 || nums[i] < nums[stack[stack.length - 1]]) {
      stack.push(i);
    }
  }

  let result = 0;
  let maxSeen = -Infinity;
  for (let i = 0; i < n && stack.length > 0; i++) {
    while (stack.length > 0 && stack[stack.length - 1] <= i) {
      stack.pop();
    }

    if (nums[i] > maxSeen) {
      maxSeen = nums[i];
      while (stack.length > 0 && nums[stack[stack.length - 1]] < maxSeen) {
        result = Math.max(result, stack[stack.length - 1] - i + 1);
        stack.pop();
      }
    }
  }

  return result;
};
```

2864 Maximum Odd Binary Number

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2864. Maximum Odd Binary Number
 * https://leetcode.com/problems/maximum-odd-binary-number/
 * Difficulty: Easy
 *
 * You are given a binary string s that contains at least one '1'.
 *
 * You have to rearrange the bits in such a way that the resulting binary number is the
maximum
 * odd binary number that can be created from this combination.
 *
 * Return a string representing the maximum odd binary number that can be created from the
given
 * combination.
 *
 * Note that the resulting string can have leading zeros.
 */

/**
 * @param {string} s
 * @return {string}
 */
var maximumOddBinaryNumber = function(s) {
  let ones = 0;
  for (const bit of s) {
```

```
    if (bit === '1') ones++;
  }
  return '1'.padStart(ones, '1').padEnd(s.length, '0').slice(1) + '1';
};
```

2865 Beautiful Towers I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2865. Beautiful Towers I
 * https://leetcode.com/problems/beautiful-towers-i/
 * Difficulty: Medium
 *
 * You are given an array heights of n integers representing the number of bricks in n
 * consecutive towers. Your task is to remove some bricks to form a mountain-shaped tower
 * arrangement. In this arrangement, the tower heights are non-decreasing, reaching a
 * maximum peak value with one or multiple consecutive towers and then non-increasing.
 *
 * Return the maximum possible sum of heights of a mountain-shaped tower arrangement.
 */

/**
 * @param {number[]} heights
 * @return {number}
 */
var maximumSumOfHeights = function(heights) {
  const n = heights.length;
  let result = 0;

  for (let peak = 0; peak < n; peak++) {
    let currentSum = heights[peak];
    let prevHeight = heights[peak];

    for (let i = peak - 1; i >= 0; i--) {
      const currentHeight = Math.min(heights[i], prevHeight);
      currentSum += currentHeight;
      prevHeight = currentHeight;
    }

    prevHeight = heights[peak];

    for (let i = peak + 1; i < n; i++) {
      const currentHeight = Math.min(heights[i], prevHeight);
      currentSum += currentHeight;
      prevHeight = currentHeight;
    }

    result = Math.max(result, currentSum);
  }

  return result;
};
```

2868 The Wording Game

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2868. The Wording Game
 * https://leetcode.com/problems/the-wording-game/
 * Difficulty: Hard
 *
 * Alice and Bob each have a lexicographically sorted array of strings named a and b
respectively.
```

```
 *
 * They are playing a wording game with the following rules:
 * - On each turn, the current player should play a word from their list such that the new
word is
 *   closely greater than the last played word; then it's the other player's turn.
 * - If a player can't play a word on their turn, they lose.
 *
 * Alice starts the game by playing her lexicographically smallest word.
 *
 * Given a and b, return true if Alice can win knowing that both players play their best, and
 * false otherwise.
 *
 * A word w is closely greater than a word z if the following conditions are met:
 * - w is lexicographically greater than z.
 * - If w1 is the first letter of w and z1 is the first letter of z, w1 should either be equal
 *   to z1 or be the letter after z1 in the alphabet.
 * - For example, the word "care" is closely greater than "book" and "car", but is not closely
 *   greater than "ant" or "cook".
 *
 * A string s is lexicographically greater than a string t if in the first position where s
and
 * t differ, string s has a letter that appears later in the alphabet than the corresponding
 * letter in t. If the first min(s.length, t.length) characters do not differ, then the longer
 * string is the lexicographically greater one.
 */

/**
 * @param {string[]} a
 * @param {string[]} b
 * @return {boolean}
 */
var canAliceWin = function(a, b) {
  const dictionaries = [{}, {}];

  [a, b].forEach((words, playerIndex) => {
    for (let i = words.length - 1; i >= 0; i--) {
      const firstChar = words[i][0];
      if (!(firstChar in dictionaries[playerIndex])) {
        dictionaries[playerIndex][firstChar] = words[i];
      }
    }
  });

  const memo = new Map();

  return !canCurrentPlayerWin(a[0], 1);

  function canCurrentPlayerWin(previousWord, playerIndex) {
    const key = `${previousWord},${playerIndex}`;
    if (memo.has(key)) return memo.get(key);

    const initialChar = previousWord[0];
    const nextChar = String.fromCharCode(initialChar.charCodeAt(0) + 1);

    if (initialChar in dictionaries[playerIndex]
        && dictionaries[playerIndex][initialChar] > previousWord) {
      if (!canCurrentPlayerWin(dictionaries[playerIndex][initialChar], playerIndex ^ 1)) {
        memo.set(key, true);
        return true;
      }
    }

    if (nextChar in dictionaries[playerIndex]) {
      if (!canCurrentPlayerWin(dictionaries[playerIndex][nextChar], playerIndex ^ 1)) {
        memo.set(key, true);
        return true;
      }
    }

    memo.set(key, false);
    return false;
  }
};
```

2869 Minimum Operations To Collect Elements

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2869. Minimum Operations to Collect Elements
 * https://leetcode.com/problems/minimum-operations-to-collect-elements/
 * Difficulty: Easy
 *
 * You are given an array nums of positive integers and an integer k.
 *
 * In one operation, you can remove the last element of the array and add it to your
collection.
 *
 * Return the minimum number of operations needed to collect elements 1, 2, ..., k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minOperations = function(nums, k) {
  const set = new Set();
  let result = 0;

  for (let i = nums.length - 1; i >= 0; i--) {
    result++;
    if (nums[i] <= k) set.add(nums[i]);
    if (set.size === k) return result;
  }

  return result;
};
```

2870 Minimum Number Of Operations To Make Array Empty

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2870. Minimum Number of Operations to Make Array Empty
 * https://leetcode.com/problems/minimum-number-of-operations-to-make-array-empty/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums consisting of positive integers.
 *
 * There are two types of operations that you can apply on the array any number of times:
 * - Choose two elements with equal values and delete them from the array.
 * - Choose three elements with equal values and delete them from the array.
 *
 * Return the minimum number of operations required to make the array empty, or -1 if it is
 * not possible.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minOperations = function(nums) {
  const map = new Map();
  for (const num of nums) {
    map.set(num, (map.get(num) || 0) + 1);
  }

  let result = 0;
```

```
  for (const count of map.values()) {
    if (count === 1) return -1;
    result += Math.ceil(count / 3);
  }

  return result;
};
```

2871 Split Array Into Maximum Number Of Subarrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2871. Split Array Into Maximum Number of Subarrays
 * https://leetcode.com/problems/split-array-into-maximum-number-of-subarrays/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of non-negative integers.
 *
 * We define the score of subarray nums[l..r] such that l <= r as nums[l] AND nums[l + 1]
 * AND ... AND nums[r] where AND is the bitwise AND operation.
 *
 * Consider splitting the array into one or more subarrays such that the following conditions
 * are satisfied:
 * - Each element of the array belongs to exactly one subarray.
 * - The sum of scores of the subarrays is the minimum possible.
 *
 * Return the maximum number of subarrays in a split that satisfies the conditions above.
 *
 * A subarray is a contiguous part of an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubarrays = function(nums) {
  const total = nums.reduce((acc, num) => acc & num, nums[0]);
  if (total !== 0) return 1;

  let result = 0;
  let current = -1;

  for (const num of nums) {
    current = current === -1 ? num : current & num;
    if (current === 0) {
      result++;
      current = -1;
    }
  }

  return result;
};
```

2872 Maximum Number Of K Divisible Components

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2872. Maximum Number of K-Divisible Components
 * https://leetcode.com/problems/maximum-number-of-k-divisible-components/
 * Difficulty: Hard
 *
 * There is an undirected tree with n nodes labeled from 0 to n - 1. You are given the integer
 * n and a 2D integer array edges of length n - 1, where edges[i] = [ai, bi] indicates that
```

```
 * there is an edge between nodes ai and bi in the tree.
 *
 * You are also given a 0-indexed integer array values of length n, where values[i] is the
value
 * associated with the ith node, and an integer k.
 *
 * A valid split of the tree is obtained by removing any set of edges, possibly empty, from
the
 * tree such that the resulting components all have values that are divisible by k, where the
 * value of a connected component is the sum of the values of its nodes.
 *
 * Return the maximum number of components in any valid split.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[]} values
 * @param {number} k
 * @return {number}
 */
var maxKDivisibleComponents = function(n, edges, values, k) {
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  let result = 0;
  dfs(0, -1);
  return result;

  function dfs(node, parent) {
    let total = values[node];
    for (const neighbor of graph[node]) {
      if (neighbor !== parent) {
        total += dfs(neighbor, node);
      }
    }
    if (total % k === 0) {
      result++;
      return 0;
    }
    return total;
  }
};
```

2873 Maximum Value Of An Ordered Triplet I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2873. Maximum Value of an Ordered Triplet I
 * https://leetcode.com/problems/maximum-value-of-an-ordered-triplet-i/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums.
 *
 * Return the maximum value over all triplets of indices (i, j, k) such that i < j < k.
 * If all such triplets have a negative value, return 0.
 *
 * The value of a triplet of indices (i, j, k) is equal to (nums[i] - nums[j]) * nums[k].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumTripletValue = function(nums) {
  let result = 0;
```

```
  for (let i = 0; i < nums.length - 2; i++) {
    for (let j = i + 1; j < nums.length - 1; j++) {
      const diff = nums[i] - nums[j];
      if (diff <= 0) continue;
      for (let k = j + 1; k < nums.length; k++) {
        result = Math.max(result, diff * nums[k]);
      }
    }
  }

  return result;
};
```

2874 Maximum Value Of An Ordered Triplet Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2874. Maximum Value of an Ordered Triplet II
 * https://leetcode.com/problems/maximum-value-of-an-ordered-triplet-ii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums.
 *
 * Return the maximum value over all triplets of indices (i, j, k) such that i < j < k. If all
 * such triplets have a negative value, return 0.
 *
 * The value of a triplet of indices (i, j, k) is equal to (nums[i] - nums[j]) * nums[k].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumTripletValue = function(nums) {
  let result = 0;
  let maxNum = nums[0];
  let maxDiff = 0;

  for (let i = 1; i < nums.length; i++) {
    result = Math.max(result, maxDiff * nums[i]);
    maxDiff = Math.max(maxDiff, maxNum - nums[i]);
    maxNum = Math.max(maxNum, nums[i]);
  }

  return result;
};
```

2892 Minimizing Array After Replacing Pairs With Their Product

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2892. Minimizing Array After Replacing Pairs With Their Product
 * https://leetcode.com/problems/minimizing-array-after-replacing-pairs-with-their-product/
 * Difficulty: Medium
 *
 * Given an integer array nums and an integer k, you can perform the following operation on
the
 * array any number of times:
 * - Select two adjacent elements of the array like x and y, such that x * y <= k, and replace
 *   both of them with a single element with value x * y (e.g. in one operation the array
 *   [1, 2, 2, 3] with k = 5 can become [1, 4, 3] or [2, 2, 3], but can't become [1, 2, 6]).
 *
```

```
 * Return the minimum possible length of nums after any number of operations.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minArrayLength = function(nums, k) {
  if (nums.some(num => num === 0)) {
    return 1;
  }

  const segments = [];
  let currentProduct = 1;
  let currentLength = 0;

  for (const num of nums) {
    if (num > k) {
      if (currentLength > 0) {
        segments.push(currentLength);
        currentProduct = 1;
        currentLength = 0;
      }
      segments.push(1);
    } else if (currentProduct * num <= k) {
      currentProduct *= num;
      currentLength++;
    } else {
      segments.push(currentLength);
      currentProduct = num;
      currentLength = 1;
    }
  }

  if (currentLength > 0) {
    segments.push(currentLength);
  }

  return segments.length;
};
```

2894 Divisible And Non Divisible Sums Difference

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2894. Divisible and Non-divisible Sums Difference
 * https://leetcode.com/problems/divisible-and-non-divisible-sums-difference/
 * Difficulty: Easy
 *
 * You are given positive integers n and m.
 *
 * Define two integers as follows:
 * - num1: The sum of all integers in the range [1, n] (both inclusive) that are not divisible
 by m.
 * - num2: The sum of all integers in the range [1, n] (both inclusive) that are divisible by
m.
 *
 * Return the integer num1 - num2.
 */

/**
 * @param {number} n
 * @param {number} m
 * @return {number}
 */
var differenceOfSums = function(n, m) {
  let nonDivisibleSum = 0;
  let divisibleSum = 0;
```

```
  for (let i = 1; i <= n; i++) {
    if (i % m === 0) {
      divisibleSum += i;
    } else {
      nonDivisibleSum += i;
    }
  }

  return nonDivisibleSum - divisibleSum;
};
```

2895 Minimum Processing Time

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2895. Minimum Processing Time
 * https://leetcode.com/problems/minimum-processing-time/
 * Difficulty: Medium
 *
 * You have a certain number of processors, each having 4 cores. The number of tasks to be
executed
 * is four times the number of processors. Each task must be assigned to a unique core, and
each
 * core can only be used once.
 *
 * You are given an array processorTime representing the time each processor becomes available
 and
 * an array tasks representing how long each task takes to complete. Return the minimum time
needed
 * to complete all tasks.
 */

/**
 * @param {number[]} processorTime
 * @param {number[]} tasks
 * @return {number}
 */
var minProcessingTime = function(processorTime, tasks) {
  tasks.sort((a, b) => b - a);
  processorTime.sort((a, b) => a - b);
  let result = 0;

  for (let i = 0; i < processorTime.length; i++) {
    for (let j = 0; j < 4; j++) {
      const taskIndex = i * 4 + j;
      result = Math.max(result, processorTime[i] + tasks[taskIndex]);
    }
  }

  return result;
};
```

2897 Apply Operations On Array To Maximize Sum Of Squares

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2897. Apply Operations on Array to Maximize Sum of Squares
 * https://leetcode.com/problems/apply-operations-on-array-to-maximize-sum-of-squares/
 * Difficulty: Hard
 *
 * You are given a 0-indexed integer array nums and a positive integer k.
 *
```

```
 * You can do the following operation on the array any number of times:
 * - Choose any two distinct indices i and j and simultaneously update the values of nums[i]
 *   to (nums[i] AND nums[j]) and nums[j] to (nums[i] OR nums[j]). Here, OR denotes the
bitwise
 *   OR operation, and AND denotes the bitwise AND operation.
 *
 * You have to choose k elements from the final array and calculate the sum of their squares.
 *
 * Return the maximum sum of squares you can achieve.
 *
 * Since the answer can be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxSum = function(nums, k) {
  const MOD = 1e9 + 7;
  const bitCounts = new Array(30).fill(0);

  for (const num of nums) {
    for (let i = 0; i < 30; i++) {
      if (num & (1 << i)) bitCounts[i]++;
    }
  }

  let result = 0;
  for (let i = 0; i < k; i++) {
    let current = 0;
    for (let j = 29; j >= 0; j--) {
      if (bitCounts[j] > 0) {
        current |= (1 << j);
        bitCounts[j]--;
      }
    }
    result = (result + Number((BigInt(current) * BigInt(current)) % BigInt(MOD))) % MOD;
  }

  return result;
};
```

2898 Maximum Linear Stock Score

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2898. Maximum Linear Stock Score
 * https://leetcode.com/problems/maximum-linear-stock-score/
 * Difficulty: Medium
 *
 * Given a 1-indexed integer array prices, where prices[i] is the price of a particular stock
 * on the ith day, your task is to select some of the elements of prices such that your
 * selection is linear.
 *
 * A selection indexes, where indexes is a 1-indexed integer array of length k which is a
 * subsequence of the array [1, 2, ..., n], is linear if:
 * - For every 1 < j <= k,
 *   prices[indexes[j]] - prices[indexes[j - 1]] == indexes[j] - indexes[j - 1].
 *
 * A subsequence is an array that can be derived from another array by deleting some or no
 * elements without changing the order of the remaining elements.
 *
 * The score of a selection indexes, is equal to the sum of the following array:
 * [prices[indexes[1]], prices[indexes[2]], ..., prices[indexes[k]].
 *
 * Return the maximum score that a linear selection can have.
 */
```

```
/**
 * @param {number[]} prices
 * @return {number}
 */
var maxScore = function(prices) {
  const values = prices.map((price, index) => price - index);
  const map = new Map();

  for (let i = 0; i < prices.length; i++) {
    const value = values[i];
    map.set(value, (map.get(value) || 0) + prices[i]);
  }

  return Math.max(...map.values());
};
```

2899 Last Visited Integers

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2899. Last Visited Integers
 * https://leetcode.com/problems/last-visited-integers/
 * Difficulty: Easy
 *
 * Given an integer array nums where nums[i] is either a positive integer or -1. We need to
find
 * for each -1 the respective positive integer, which we call the last visited integer.
 *
 * To achieve this goal, let's define two empty arrays: seen and ans.
 *
 * Start iterating from the beginning of the array nums.
 * - If a positive integer is encountered, prepend it to the front of seen.
 * - If -1 is encountered, let k be the number of consecutive -1s seen so far (including the
 *   current -1),
 *   - If k is less than or equal to the length of seen, append the k-th element of seen to
ans.
 *   - If k is strictly greater than the length of seen, append -1 to ans.
 *
 * Return the array ans.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var lastVisitedIntegers = function(nums) {
  const seen = [];
  const result = [];
  let consecutiveNegatives = 0;

  for (const num of nums) {
    if (num > 0) {
      seen.unshift(num);
      consecutiveNegatives = 0;
    } else {
      consecutiveNegatives++;
      result.push(consecutiveNegatives <= seen.length ? seen[consecutiveNegatives - 1] : -1);
    }
  }

  return result;
};
```

2900 Longest Unequal Adjacent Groups Subsequence I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2900. Longest Unequal Adjacent Groups Subsequence I
 * https://leetcode.com/problems/longest-unequal-adjacent-groups-subsequence-i/
 * Difficulty: Easy
 *
 * You are given a string array words and a binary array groups both of length n, where
words[i]
 * is associated with groups[i].
 *
 * Your task is to select the longest alternating subsequence from words. A subsequence of
words
 * is alternating if for any two consecutive strings in the sequence, their corresponding
elements
 * in the binary array groups differ. Essentially, you are to choose strings such that
adjacent
 * elements have non-matching corresponding bits in the groups array.
 *
 * Formally, you need to find the longest subsequence of an array of indices [0, 1, ..., n -
1]
 * denoted as [i0, i1, ..., ik-1], such that groups[ij] != groups[ij+1] for each 0 <= j < k -
1
 * and then find the words corresponding to these indices.
 *
 * Return the selected subsequence. If there are multiple answers, return any of them.
 *
 * Note: The elements in words are distinct.
 */

/**
 * @param {string[]} words
 * @param {number[]} groups
 * @return {string[]}
 */
var getLongestSubsequence = function(words, groups) {
  const result = [words[0]];
  let lastGroup = groups[0];

  for (let i = 1; i < words.length; i++) {
    if (groups[i] !== lastGroup) {
      result.push(words[i]);
      lastGroup = groups[i];
    }
  }

  return result;
};
```

2901 Longest Unequal Adjacent Groups Subsequence Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2901. Longest Unequal Adjacent Groups Subsequence II
 * https://leetcode.com/problems/longest-unequal-adjacent-groups-subsequence-ii/
 * Difficulty: Medium
 *
 * You are given a string array words, and an array groups, both arrays having length n.
 *
 * The hamming distance between two strings of equal length is the number of positions at
which the
 * corresponding characters are different.
```

```
 *
 * You need to select the longest subsequence from an array of indices [0, 1, ..., n - 1],
such
 * that for the subsequence denoted as [i0, i1, ..., ik-1] having length k, the following
holds:
 * - For adjacent indices in the subsequence, their corresponding groups are unequal, i.e.,
 *   groups[ij] != groups[ij+1], for each j where 0 < j + 1 < k.
 * - words[ij] and words[ij+1] are equal in length, and the hamming distance between them is
1,
 *   where 0 < j + 1 < k, for all indices in the subsequence.
 *
 * Return a string array containing the words corresponding to the indices (in order) in the
 * selected subsequence. If there are multiple answers, return any of them.
 *
 * Note: strings in words may be unequal in length.
 */

/**
 * @param {string[]} words
 * @param {number[]} groups
 * @return {string[]}
 */
var getWordsInLongestSubsequence = function(words, groups) {
  const n = words.length;
  const dp = new Array(n).fill(1);
  const prev = new Array(n).fill(-1);
  let maxLen = 1;
  let lastIndex = 0;

  for (let i = 1; i < n; i++) {
    for (let j = 0; j < i; j++) {
      if (groups[i] !== groups[j] && helper(words[i], words[j])) {
        if (dp[j] + 1 > dp[i]) {
          dp[i] = dp[j] + 1;
          prev[i] = j;
        }
      }
    }
    if (dp[i] > maxLen) {
      maxLen = dp[i];
      lastIndex = i;
    }
  }

  const result = [];
  while (lastIndex !== -1) {
    result.push(words[lastIndex]);
    lastIndex = prev[lastIndex];
  }

  return result.reverse();

  function helper(s1, s2) {
    if (s1.length !== s2.length) return false;
    let diff = 0;
    for (let i = 0; i < s1.length; i++) {
      if (s1[i] !== s2[i]) diff++;
      if (diff > 1) return false;
    }
    return diff === 1;
  }
};
```

2903 Find Indices With Index And Value Difference I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2903. Find Indices With Index and Value Difference I
 * https://leetcode.com/problems/find-indices-with-index-and-value-difference-i/
```

```
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums having length n, an integer indexDifference,
 * and an integer valueDifference.
 *
 * Your task is to find two indices i and j, both in the range [0, n - 1], that satisfy the
 * following conditions:
 * - abs(i - j) >= indexDifference, and
 * - abs(nums[i] - nums[j]) >= valueDifference
 *
 * Return an integer array answer, where answer = [i, j] if there are two such indices, and
 * answer = [-1, -1] otherwise. If there are multiple choices for the two indices, return
 * any of them.
 *
 * Note: i and j may be equal.
 */

/**
 * @param {number[]} nums
 * @param {number} indexDifference
 * @param {number} valueDifference
 * @return {number[]}
 */
var findIndices = function(nums, indexDifference, valueDifference) {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i; j < nums.length; j++) {
      if (Math.abs(i - j) >= indexDifference && Math.abs(nums[i] - nums[j]) >=
valueDifference) {
        return [i, j];
      }
    }
  }
  return [-1, -1];
};
```

2904 Shortest And Lexicographically Smallest Beautiful String

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2904. Shortest and Lexicographically Smallest Beautiful String
 * https://leetcode.com/problems/shortest-and-lexicographically-smallest-beautiful-string/
 * Difficulty: Medium
 *
 * You are given a binary string s and a positive integer k.
 *
 * A substring of s is beautiful if the number of 1's in it is exactly k.
 *
 * Let len be the length of the shortest beautiful substring.
 *
 * Return the lexicographically smallest beautiful substring of string s with length equal to
 * len. If s doesn't contain a beautiful substring, return an empty string.
 *
 * A string a is lexicographically larger than a string b (of the same length) if in the first
 * position where a and b differ, a has a character strictly larger than the corresponding
 * character in b.
 * - For example, "abcd" is lexicographically larger than "abcc" because the first position
they
 *   differ is at the fourth character, and d is greater than c.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {string}
 */
var shortestBeautifulSubstring = function(s, k) {
  let minLength = Infinity;
  let result = '';
```

```
  for (let start = 0; start < s.length; start++) {
    let onesCount = 0;
    for (let end = start; end < s.length; end++) {
      if (s[end] === '1') onesCount++;
      if (onesCount === k) {
        const currentLength = end - start + 1;
        const currentSubstring = s.slice(start, end + 1);
        if (currentLength < minLength
            || (currentLength === minLength && currentSubstring < result)) {
          minLength = currentLength;
          result = currentSubstring;
        }
      }
    }
  }

  return result;
};
```

2907 Maximum Profitable Triplets With Increasing Prices I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2907. Maximum Profitable Triplets With Increasing Prices I
 * https://leetcode.com/problems/maximum-profitable-triplets-with-increasing-prices-i/
 * Difficulty: Medium
 *
 * Given the 0-indexed arrays prices and profits of length n. There are n items in an store
 * where the ith item has a price of prices[i] and a profit of profits[i].
 *
 * We have to pick three items with the following condition:
 * - prices[i] < prices[j] < prices[k] where i < j < k.
 *
 * If we pick items with indices i, j and k satisfying the above condition, the profit
 * would be profits[i] + profits[j] + profits[k].
 *
 * Return the maximum profit we can get, and -1 if it's not possible to pick three items
 * with the given condition.
 */

/**
 * @param {number[]} prices
 * @param {number[]} profits
 * @return {number}
 */
var maxProfit = function(prices, profits) {
  const n = prices.length;
  let result = -1;

  for (let j = 1; j < n - 1; j++) {
    let maxLeftProfit = 0;
    let maxRightProfit = 0;
    let hasLeft = false;
    let hasRight = false;

    for (let i = 0; i < j; i++) {
      if (prices[i] < prices[j]) {
        maxLeftProfit = Math.max(maxLeftProfit, profits[i]);
        hasLeft = true;
      }
    }

    for (let k = j + 1; k < n; k++) {
      if (prices[j] < prices[k]) {
        maxRightProfit = Math.max(maxRightProfit, profits[k]);
        hasRight = true;
      }
    }
```

```
    if (hasLeft && hasRight) {
      const tripletProfit = maxLeftProfit + profits[j] + maxRightProfit;
      result = Math.max(result, tripletProfit);
    }
  }

  return result;
};
```

2908 Minimum Sum Of Mountain Triplets I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2908. Minimum Sum of Mountain Triplets I
 * https://leetcode.com/problems/minimum-sum-of-mountain-triplets-i/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array nums of integers.
 *
 * A triplet of indices (i, j, k) is a mountain if:
 * - i < j < k
 * - nums[i] < nums[j] and nums[k] < nums[j]
 *
 * Return the minimum possible sum of a mountain triplet of nums. If no such triplet exists,
 * return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumSum = function(nums) {
  let minSum = Infinity;

  for (let i = 0; i < nums.length - 2; i++) {
    for (let j = i + 1; j < nums.length - 1; j++) {
      for (let k = j + 1; k < nums.length; k++) {
        if (nums[i] < nums[j] && nums[k] < nums[j]) {
          minSum = Math.min(minSum, nums[i] + nums[j] + nums[k]);
        }
      }
    }
  }

  return minSum === Infinity ? -1 : minSum;
};
```

2909 Minimum Sum Of Mountain Triplets Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2909. Minimum Sum of Mountain Triplets II
 * https://leetcode.com/problems/minimum-sum-of-mountain-triplets-ii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array nums of integers.
 *
 * A triplet of indices (i, j, k) is a mountain if:
 * - i < j < k
 * - nums[i] < nums[j] and nums[k] < nums[j]
 *
 * Return the minimum possible sum of a mountain triplet of nums. If no such triplet
 * exists, return -1.
```

```
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumSum = function(nums) {
  const n = nums.length;
  const leftMin = new Array(n).fill(Infinity);
  const rightMin = new Array(n).fill(Infinity);

  leftMin[0] = nums[0];
  for (let i = 1; i < n; i++) {
    leftMin[i] = Math.min(leftMin[i - 1], nums[i]);
  }

  rightMin[n - 1] = nums[n - 1];
  for (let i = n - 2; i >= 0; i--) {
    rightMin[i] = Math.min(rightMin[i + 1], nums[i]);
  }

  let minSum = Infinity;
  for (let j = 1; j < n - 1; j++) {
    if (nums[j] > leftMin[j - 1] && nums[j] > rightMin[j + 1]) {
      minSum = Math.min(minSum, leftMin[j - 1] + nums[j] + rightMin[j + 1]);
    }
  }

  return minSum === Infinity ? -1 : minSum;
};
```

2912 Number Of Ways To Reach Destination In The Grid

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2912. Number of Ways to Reach Destination in the Grid
 * https://leetcode.com/problems/number-of-ways-to-reach-destination-in-the-grid/
 * Difficulty: Hard
 *
 * You are given two integers n and m which represent the size of a 1-indexed grid. You are
 * also given an integer k, a 1-indexed integer array source and a 1-indexed integer array
 * dest, where source and dest are in the form [x, y] representing a cell on the given grid.
 *
 * You can move through the grid in the following way:
 * - You can go from cell [x1, y1] to cell [x2, y2] if either x1 == x2 or y1 == y2.
 * - Note that you can't move to the cell you are already in e.g. x1 == x2 and y1 == y2.
 *
 * Return the number of ways you can reach dest from source by moving through the grid exactly
 * k times.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number} n
 * @param {number} m
 * @param {number} k
 * @param {number[]} source
 * @param {number[]} dest
 * @return {number}
 */
var numberOfWays = function(n, m, k, source, dest) {
  const MOD = 1e9 + 7;
  const dp = new Array(k + 1).fill().map(() => new Array(4).fill(0));

  if (source[0] === dest[0] && source[1] === dest[1]) {
    dp[0][0] = 1;
  } else if (source[0] === dest[0]) {
    dp[0][1] = 1;
```

```
  } else if (source[1] === dest[1]) {
    dp[0][2] = 1;
  } else {
    dp[0][3] = 1;
  }

  for (let i = 1; i <= k; i++) {
    dp[i][0] = (dp[i - 1][1] + dp[i - 1][2]) % MOD;
    dp[i][1] = (BigInt(dp[i - 1][0]) * BigInt(m - 1) + BigInt(dp[i - 1][1])
      * BigInt(m - 2) + BigInt(dp[i - 1][3])) % BigInt(MOD);
    dp[i][2] = (BigInt(dp[i - 1][0]) * BigInt(n - 1) + BigInt(dp[i - 1][2])
      * BigInt(n - 2) + BigInt(dp[i - 1][3])) % BigInt(MOD);
    dp[i][3] = (BigInt(dp[i - 1][1]) * BigInt(n - 1) + BigInt(dp[i - 1][2])
      * BigInt(m - 1) + BigInt(dp[i - 1][3]) * BigInt(m + n - 4)) % BigInt(MOD);
    dp[i][1] = Number(dp[i][1]);
    dp[i][2] = Number(dp[i][2]);
    dp[i][3] = Number(dp[i][3]);
  }

  return dp[k][0];
};
```

2913 Subarrays Distinct Element Sum Of Squares I

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2913. Subarrays Distinct Element Sum of Squares I
 * https://leetcode.com/problems/subarrays-distinct-element-sum-of-squares-i/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums.
 *
 * The distinct count of a subarray of nums is defined as:
 * - Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such
 *   that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is
 *   called the distinct count of nums[i..j].
 *
 * Return the sum of the squares of distinct counts of all subarrays of nums.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumCounts = function(nums) {
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    const set = new Set();
    for (let j = i; j < nums.length; j++) {
      set.add(nums[j]);
      result += set.size * set.size;
    }
  }

  return result;
};
```

2914 Minimum Number Of Changes To Make Binary String Beautiful

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2914. Minimum Number of Changes to Make Binary String Beautiful
 * https://leetcode.com/problems/minimum-number-of-changes-to-make-binary-string-beautiful/
 * Difficulty: Medium
 *
 * You are given a 0-indexed binary string s having an even length.
 *
 * A string is beautiful if it's possible to partition it into one or more substrings such
that:
 * - Each substring has an even length.
 * - Each substring contains only 1's or only 0's.
 *
 * You can change any character in s to 0 or 1.
 *
 * Return the minimum number of changes required to make the string s beautiful.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minChanges = function(s) {
  let result = 0;

  for (let i = 0; i < s.length; i += 2) {
    if (s[i] !== s[i + 1]) result++;
  }

  return result;
};
```

2917 Find The K Or Of An Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2917. Find the K-or of an Array
 * https://leetcode.com/problems/find-the-k-or-of-an-array/
 * Difficulty: Easy
 *
 * You are given an integer array nums, and an integer k. Let's introduce K-or operation
 * by extending the standard bitwise OR. In K-or, a bit position in the result is set to
 * 1 if at least k numbers in nums have a 1 in that position.
 *
 * Return the K-or of nums.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var findKOr = function(nums, k) {
  let result = 0;

  for (let bit = 0; bit < 31; bit++) {
    let count = 0;
    for (const num of nums) {
      if (num & (1 << bit)) count++;
    }
    if (count >= k) result |= (1 << bit);
  }

  return result;
};
```

2918 Minimum Equal Sum Of Two Arrays After Replacing Zeros

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2918. Minimum Equal Sum of Two Arrays After Replacing Zeros
 * https://leetcode.com/problems/minimum-equal-sum-of-two-arrays-after-replacing-zeros/
 * Difficulty: Medium
 *
 * You are given two arrays nums1 and nums2 consisting of positive integers.
 *
 * You have to replace all the 0's in both arrays with strictly positive integers such that
the
 * sum of elements of both arrays becomes equal.
 *
 * Return the minimum equal sum you can obtain, or -1 if it is impossible.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minSum = function(nums1, nums2) {
  let sum1 = 0;
  let zeros1 = 0;
  for (const num of nums1) {
    sum1 += num;
    if (num === 0) zeros1++;
  }

  let sum2 = 0;
  let zeros2 = 0;
  for (const num of nums2) {
    sum2 += num;
    if (num === 0) zeros2++;
  }

  const minSum1 = sum1 + zeros1;
  const minSum2 = sum2 + zeros2;

  if (minSum1 > sum2 && zeros2 === 0 || minSum2 > sum1 && zeros1 === 0) return -1;

  return Math.max(minSum1, minSum2);
};
```

2921 Maximum Profitable Triplets With Increasing Prices Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2921. Maximum Profitable Triplets With Increasing Prices II
 * https://leetcode.com/problems/maximum-profitable-triplets-with-increasing-prices-ii/
 * Difficulty: Hard
 *
 * Given the 0-indexed arrays prices and profits of length n. There are n items in an store
 * where the ith item has a price of prices[i] and a profit of profits[i].
 *
 * We have to pick three items with the following condition:
 * - prices[i] < prices[j] < prices[k] where i < j < k.
 *
 * If we pick items with indices i, j and k satisfying the above condition, the profit would
 * be profits[i] + profits[j] + profits[k].
 *
 * Return the maximum profit we can get, and -1 if it's not possible to pick three items with
```

```
 * the given condition.
 */

/**
 * @param {number[]} prices
 * @param {number[]} profits
 * @return {number}
 */
var maxProfit = function(prices, profits) {
  const bit1 = new Array(5001).fill(0);
  const bit2 = new Array(5001).fill(0);
  let result = -1;

  for (let i = 0; i < prices.length; i++) {
    const price = prices[i];
    const profit = profits[i];
    const maxLeft = query(bit1, price - 1);
    const maxPair = query(bit2, price - 1);

    if (maxPair > 0) {
      result = Math.max(result, maxPair + profit);
    }

    update(bit1, price, profit);

    if (maxLeft > 0) {
      update(bit2, price, profit + maxLeft);
    }
  }

  return result;

  function query(bit, price) {
    let maxValue = 0;
    while (price > 0) {
      maxValue = Math.max(maxValue, bit[price]);
      price &= price - 1;
    }
    return maxValue;
  }

  function update(bit, price, value) {
    while (price < 5001) {
      bit[price] = Math.max(bit[price], value);
      price += price & (-price);
    }
  }
};
```

2923 Find Champion I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2923. Find Champion I
 * https://leetcode.com/problems/find-champion-i/
 * Difficulty: Easy
 *
 * There are n teams numbered from 0 to n - 1 in a tournament.
 *
 * Given a 0-indexed 2D boolean matrix grid of size n * n. For all i, j that 0 <= i, j <= n -
1
 * and i != j team i is stronger than team j if grid[i][j] == 1, otherwise, team j is stronger
 * than team i.
 *
 * Team a will be the champion of the tournament if there is no team b that is stronger than
 * team a.
 *
 * Return the team that will be the champion of the tournament.
 */
```

```
/**
 * @param {number[][]} grid
 * @return {number}
 */
var findChampion = function(grid) {
  const n = grid.length;

  for (let i = 0; i < n; i++) {
    let isChampion = true;
    for (let j = 0; j < n; j++) {
      if (i !== j && grid[j][i] === 1) {
        isChampion = false;
        break;
      }
    }
    if (isChampion) return i;
  }

  return -1;
};
```

2924 Find Champion Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2924. Find Champion II
 * https://leetcode.com/problems/find-champion-ii/
 * Difficulty: Medium
 *
 * There are n teams numbered from 0 to n - 1 in a tournament; each team is also a node in a
DAG.
 *
 * You are given the integer n and a 0-indexed 2D integer array edges of length m representing
 the
 * DAG, where edges[i] = [ui, vi] indicates that there is a directed edge from team ui to team
 vi
 * in the graph.
 *
 * A directed edge from a to b in the graph means that team a is stronger than team b and team
 b
 * is weaker than team a.
 *
 * Team a will be the champion of the tournament if there is no team b that is stronger than
team a.
 *
 * Return the team that will be the champion of the tournament if there is a unique champion,
 * otherwise, return -1.
 *
 * Notes:
 * - A cycle is a series of nodes a1, a2, ..., an, an+1 such that node a1 is the same node as
node
 *   an+1, the nodes a1, a2, ..., an are distinct, and there is a directed edge from the node
ai
 *   to node ai+1 for every i in the range [1, n].
 * - A DAG is a directed graph that does not have any cycle.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var findChampion = function(n, edges) {
  const inDegree = new Array(n).fill(0);

  for (const [, v] of edges) {
    inDegree[v]++;
  }
```

```
  let champion = -1;
  for (let i = 0; i < n; i++) {
    if (inDegree[i] === 0) {
      if (champion !== -1) return -1;
      champion = i;
    }
  }

  return champion;
};
```

2925 Maximum Score After Applying Operations On A Tree

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 2925. Maximum Score After Applying Operations on a Tree
 * https://leetcode.com/problems/maximum-score-after-applying-operations-on-a-tree/
 * Difficulty: Medium
 *
 * There is an undirected tree with n nodes labeled from 0 to n - 1, and rooted at node 0. You
 are
 * given a 2D integer array edges of length n - 1, where edges[i] = [ai, bi] indicates that
there
 * is an edge between nodes ai and bi in the tree.
 *
 * You are also given a 0-indexed integer array values of length n, where values[i] is the
value
 * associated with the ith node.
 *
 * You start with a score of 0. In one operation, you can:
 * - Pick any node i.
 * - Add values[i] to your score.
 * - Set values[i] to 0.
 *
 * A tree is healthy if the sum of values on the path from the root to any leaf node is
different
 * than zero.
 *
 * Return the maximum score you can obtain after performing these operations on the tree any
 * number of times so that it remains healthy.
 */

/**
 * @param {number[][]} edges
 * @param {number[]} values
 * @return {number}
 */
var maximumScoreAfterOperations = function(edges, values) {
  const n = values.length;
  const graph = new Array(n).fill().map(() => []);

  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  const totalSum = values.reduce((sum, val) => sum + val, 0);
  const minToKeep = dfs(0, -1);

  return totalSum - minToKeep;

  function dfs(node, parent) {
    const children = graph[node].filter(child => child !== parent);

    if (children.length === 0) {
      return values[node];
    }

    let childrenSum = 0;
```

```
    for (const child of children) {
      childrenSum += dfs(child, node);
    }

    return Math.min(values[node], childrenSum);
  }
};
```

2927 Distribute Candies Among Children Iii

HARD

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2927. Distribute Candies Among Children III
 * https://leetcode.com/problems/distribute-candies-among-children-iii/
 * Difficulty: Hard
 *
 * You are given two positive integers n and limit.
 *
 * Return the total number of ways to distribute n candies among 3 children such that no child
 * gets more than limit candies.
 */

/**
 * @param {number} n
 * @param {number} limit
 * @return {number}
 */
var distributeCandies = function(n, limit) {
  if (n > 3 * limit) return 0;

  const overLimit = limit + 1;
  return countWays(n) - 3 * countWays(n - overLimit)
    + 3 * countWays(n - 2 * overLimit) - countWays(n - 3 * overLimit);

  function countWays(candies) {
    if (candies < 0) return 0;
    return Math.floor((candies + 2) * (candies + 1) / 2);
  }
};
```

2928 Distribute Candies Among Children I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2928. Distribute Candies Among Children I
 * https://leetcode.com/problems/distribute-candies-among-children-i/
 * Difficulty: Easy
 *
 * You are given two positive integers n and limit.
 *
 * Return the total number of ways to distribute n candies among 3 children such that no child
 * gets more than limit candies.
 */

/**
 * @param {number} n
 * @param {number} limit
 * @return {number}
 */
var distributeCandies = function(n, limit) {
  let result = 0;

  for (let i = 0; i <= Math.min(n, limit); i++) {
```

```
    for (let j = 0; j <= Math.min(n - i, limit); j++) {
      const k = n - i - j;
      if (k >= 0 && k <= limit) result++;
    }
  }

  return result;
};
```

2929 Distribute Candies Among Children Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2929. Distribute Candies Among Children II
 * https://leetcode.com/problems/distribute-candies-among-children-ii/
 * Difficulty: Medium
 *
 * You are given two positive integers n and limit.
 *
 * Return the total number of ways to distribute n candies among 3 children such that no child
 * gets more than limit candies.
 */

/**
 * @param {number} n
 * @param {number} limit
 * @return {number}
 */
var distributeCandies = function(n, limit) {
  const minCandies = Math.max(0, n - 2 * limit);
  const maxCandies = Math.min(n, limit);
  let result = 0;

  for (let i = minCandies; i <= maxCandies; i++) {
    const remaining = n - i;
    const minSecond = Math.max(0, remaining - limit);
    const maxSecond = Math.min(remaining, limit);
    result += maxSecond - minSecond + 1;
  }

  return result;
};
```

2932 Maximum Strong Pair Xor I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2932. Maximum Strong Pair XOR I
 * https://leetcode.com/problems/maximum-strong-pair-xor-i/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums. A pair of integers x and y is called a strong
 pair
 * if it satisfies the condition:
 * - |x - y| <= min(x, y)
 *
 * You need to select two integers from nums such that they form a strong pair and their
bitwise
 * XOR is the maximum among all strong pairs in the array.
 *
 * Return the maximum XOR value out of all possible strong pairs in the array nums.
 *
 * Note that you can pick the same integer twice to form a pair.
```

```
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumStrongPairXor = function(nums) {
  let result = 0;
  nums.sort((a, b) => a - b);

  for (let i = 0; i < nums.length; i++) {
    for (let j = i; j < nums.length; j++) {
      if (nums[j] - nums[i] <= Math.min(nums[i], nums[j])) {
        result = Math.max(result, nums[i] ^ nums[j]);
      } else {
        break;
      }
    }
  }

  return result;
};
```

2933 High Access Employees

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2933. High-Access Employees
 * https://leetcode.com/problems/high-access-employees/
 * Difficulty: Medium
 *
 * You are given a 2D 0-indexed array of strings, accessTimes, with size n. For each i where
 * 0 <= i <= n - 1, accessTimes[i][0] represents the name of an employee, and
accessTimes[i][1]
 * represents the access time of that employee. All entries in accessTimes are within the same
 day.
 *
 * The access time is represented as four digits using a 24-hour time format, for example,
"0800"
 * or "2250".
 *
 * An employee is said to be high-access if he has accessed the system three or more times
within
 * a one-hour period.
 *
 * Times with exactly one hour of difference are not considered part of the same one-hour
period.
 * For example, "0815" and "0915" are not part of the same one-hour period.
 *
 * Access times at the start and end of the day are not counted within the same one-hour
period.
 * For example, "0005" and "2350" are not part of the same one-hour period.
 *
 * Return a list that contains the names of high-access employees with any order you want.
 */

/**
 * @param {string[][]} accessTimes
 * @return {string[]}
 */
var findHighAccessEmployees = function(accessTimes) {
  const employeeAccess = new Map();

  for (const [name, time] of accessTimes) {
    const minutes = parseInt(time.slice(0, 2)) * 60 + parseInt(time.slice(2));
    if (!employeeAccess.has(name)) {
      employeeAccess.set(name, []);
    }
```

```
    employeeAccess.get(name).push(minutes);
  }

  const result = [];
  for (const [name, times] of employeeAccess) {
    times.sort((a, b) => a - b);
    if (times.length >= 3) {
      for (let i = 0; i <= times.length - 3; i++) {
        if (times[i + 2] - times[i] < 60) {
          result.push(name);
          break;
        }
      }
    }
  }

  return result;
};
```

2934 Minimum Operations To Maximize Last Elements In Arrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2934. Minimum Operations to Maximize Last Elements in Arrays
 * https://leetcode.com/problems/minimum-operations-to-maximize-last-elements-in-arrays/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer arrays, nums1 and nums2, both having length n.
 *
 * You are allowed to perform a series of operations (possibly none).
 *
 * In an operation, you select an index i in the range [0, n - 1] and swap the values of
 * nums1[i] and nums2[i].
 *
 * Your task is to find the minimum number of operations required to satisfy the following
 * conditions:
 * - nums1[n - 1] is equal to the maximum value among all elements of nums1, i.e.,
 *   nums1[n - 1] = max(nums1[0], nums1[1], ..., nums1[n - 1]).
 * - nums2[n - 1] is equal to the maximum value among all elements of nums2, i.e.,
 *   nums2[n - 1] = max(nums2[0], nums2[1], ..., nums2[n - 1]).
 *
 * Return an integer denoting the minimum number of operations needed to meet both conditions,
 * or -1 if it is impossible to satisfy both conditions.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minOperations = function(nums1, nums2) {
  const n = nums1.length;
  const last1 = nums1[n - 1];
  const last2 = nums2[n - 1];

  let swapsNoChange = 0;
  let swapsChange = 0;

  for (let i = 0; i < n - 1; i++) {
    if (nums1[i] > last1 || nums2[i] > last2) {
      if (nums2[i] > last1 || nums1[i] > last2) {
        return -1;
      }
      swapsNoChange++;
    }
    if (nums1[i] > last2 || nums2[i] > last1) {
      if (nums2[i] > last2 || nums1[i] > last1) {
        return -1;
      }
```

```
      swapsChange++;
    }
  }

  if (last1 !== last2) {
    swapsChange++;
  }

  return Math.min(swapsNoChange, swapsChange);
};
```

2936 Number Of Equal Numbers Blocks

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 2936. Number of Equal Numbers Blocks
 * https://leetcode.com/problems/number-of-equal-numbers-blocks/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of integers, nums. The following property holds for nums:
 * - All occurrences of a value are adjacent. In other words, if there are two indices i < j
 *   such that nums[i] == nums[j], then for every index k that i < k < j, nums[k] == nums[i].
 *
 * Since nums is a very large array, you are given an instance of the class BigArray which has
 * the following functions:
 * - int at(long long index): Returns the value of nums[i].
 * - void size(): Returns nums.length.
 *
 * Let's partition the array into maximal blocks such that each block contains equal values.
 * Return the number of these blocks.
 *
 * Note that if you want to test your solution using a custom test, behavior for tests with
 * nums.length > 10 is undefined.
 */

/**
 * Definition for BigArray.
 * class BigArray {
 *     @param {number[]} elements
 *     constructor(elements);
 *
 *     @param {number} index
 *     @return {number}
 *     at(index);
 *
 *     @return {number}
 *     size();
 * }
 */
/**
 * @param {BigArray} nums
 * @return {number}
 */
var countBlocks = function(nums) {
  const n = nums.size();
  if (n === 0) return 0;

  let blocks = 1;
  let currentIndex = 0;

  while (currentIndex < n - 1) {
    const nextBlockStart = findNextBlockStart(nums, currentIndex, n);
    if (nextBlockStart < n) {
      blocks++;
      currentIndex = nextBlockStart;
    } else {
      break;
    }
  }
```

```
  return blocks;

  function findNextBlockStart(nums, startIndex, n) {
    const currentValue = nums.at(startIndex);
    let left = startIndex + 1;
    let right = n;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (nums.at(mid) === currentValue) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }

    return left;
  }
};
```

2937 Make Three Strings Equal

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2937. Make Three Strings Equal
 * https://leetcode.com/problems/make-three-strings-equal/
 * Difficulty: Easy
 *
 * You are given three strings: s1, s2, and s3. In one operation you can choose one of these
 * strings and delete its rightmost character. Note that you cannot completely empty a string.
 *
 * Return the minimum number of operations required to make the strings equal. If it is
 * impossible to make them equal, return -1.
 */

/**
 * @param {string} s1
 * @param {string} s2
 * @param {string} s3
 * @return {number}
 */
var findMinimumOperations = function(s1, s2, s3) {
  const minLength = Math.min(s1.length, s2.length, s3.length);

  let commonPrefixLength = 0;
  for (let i = 0; i < minLength; i++) {
    if (s1[i] !== s2[i] || s2[i] !== s3[i]) {
      break;
    }
    commonPrefixLength++;
  }

  if (commonPrefixLength === 0) {
    return -1;
  }

  return s1.length + s2.length + s3.length - 3 * commonPrefixLength;
};
```

2938 Separate Black And White Balls

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2938. Separate Black and White Balls
 * https://leetcode.com/problems/separate-black-and-white-balls/
 * Difficulty: Medium
 *
 * There are n balls on a table, each ball has a color black or white.
 *
 * You are given a 0-indexed binary string s of length n, where 1 and 0 represent black and
white
 * balls, respectively.
 *
 * In each step, you can choose two adjacent balls and swap them.
 *
 * Return the minimum number of steps to group all the black balls to the right and all the
white
 * balls to the left.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumSteps = function(s) {
  let result = 0;
  let whiteCount = 0;

  for (let i = 0; i < s.length; i++) {
    if (s[i] === '0') {
      result += i - whiteCount;
      whiteCount++;
    }
  }

  return result;
};
```

2941 Maximum Gcd Sum Of A Subarray

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2941. Maximum GCD-Sum of a Subarray
 * https://leetcode.com/problems/maximum-gcd-sum-of-a-subarray/
 * Difficulty: Hard
 *
 * You are given an array of integers nums and an integer k.
 *
 * The gcd-sum of an array a is calculated as follows:
 * - Let s be the sum of all the elements of a.
 * - Let g be the greatest common divisor of all the elements of a.
 * - The gcd-sum of a is equal to s * g.
 *
 * Return the maximum gcd-sum of a subarray of nums with at least k elements.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxGcdSum = function(nums, k) {
  const prefixSum = [0];
  let gcdPairs = [];
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    prefixSum.push(prefixSum[prefixSum.length - 1] + nums[i]);

    const newGcdPairs = [];
    const candidates = [...gcdPairs, [i, nums[i]]];
```

```
    for (const [startIndex, currentGcd] of candidates) {
      const newGcd = gcd(currentGcd, nums[i]);
      if (newGcdPairs.length === 0 || newGcdPairs[newGcdPairs.length - 1][1] !== newGcd) {
        newGcdPairs.push([startIndex, newGcd]);
      }
    }

    gcdPairs = newGcdPairs;

    for (const [startIndex, gcdValue] of gcdPairs) {
      if (i - startIndex + 1 < k) break;
      const subarraySum = prefixSum[prefixSum.length - 1] - prefixSum[startIndex];
      result = Math.max(result, subarraySum * gcdValue);
    }
  }

  return result;

  function gcd(a, b) {
    while (b !== 0) {
      const temp = b;
      b = a % b;
      a = temp;
    }
    return a;
  }
};
```

2942 Find Words Containing Character

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2942. Find Words Containing Character
 * https://leetcode.com/problems/find-words-containing-character/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array of strings words and a character x.
 *
 * Return an array of indices representing the words that contain the character x.
 *
 * Note that the returned array may be in any order.
 */

/**
 * @param {string[]} words
 * @param {character} x
 * @return {number[]}
 */
var findWordsContaining = function(words, x) {
  const result = [];

  for (let i = 0; i < words.length; i++) {
    if (words[i].includes(x)) {
      result.push(i);
    }
  }

  return result;
};
```

2946 Matrix Similarity After Cyclic Shifts

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```javascript
/**
 * 2946. Matrix Similarity After Cyclic Shifts
 * https://leetcode.com/problems/matrix-similarity-after-cyclic-shifts/
 * Difficulty: Easy
 *
 * You are given an m x n integer matrix mat and an integer k. The matrix rows are 0-indexed.
 *
 * The following proccess happens k times:
 * - Even-indexed rows (0, 2, 4, ...) are cyclically shifted to the left.
 * - Odd-indexed rows (1, 3, 5, ...) are cyclically shifted to the right.
 *
 * Return true if the final modified matrix after k steps is identical to the original matrix,
 * and false otherwise.
 */

/**
 * @param {number[][]} mat
 * @param {number} k
 * @return {boolean}
 */
var areSimilar = function(mat, k) {
  const m = mat.length;
  const n = mat[0].length;

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      const shift = i % 2 === 0 ? (j - k % n + n) % n : (j + k % n) % n;
      if (mat[i][j] !== mat[i][shift]) {
        return false;
      }
    }
  }

  return true;
};
```

2947 Count Beautiful Substrings I

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```javascript
/**
 * 2947. Count Beautiful Substrings I
 * https://leetcode.com/problems/count-beautiful-substrings-i/
 * Difficulty: Medium
 *
 * You are given a string s and a positive integer k.
 *
 * Let vowels and consonants be the number of vowels and consonants in a string.
 *
 * A string is beautiful if:
 * - vowels == consonants.
 * - (vowels * consonants) % k == 0, in other terms the multiplication of vowels and
 *   consonants is divisible by k.
 *
 * Return the number of non-empty beautiful substrings in the given string s.
 *
 * A substring is a contiguous sequence of characters in a string.
 *
 * Vowel letters in English are 'a', 'e', 'i', 'o', and 'u'.
 *
 * Consonant letters in English are every letter except vowels.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var beautifulSubstrings = function(s, k) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u']);
```

```
  let result = 0;

  for (let start = 0; start < s.length; start++) {
    let vowelCount = 0;
    let consonantCount = 0;

    for (let end = start; end < s.length; end++) {
      if (vowels.has(s[end])) {
        vowelCount++;
      } else {
        consonantCount++;
      }

      if (vowelCount === consonantCount && (vowelCount * consonantCount) % k === 0) {
        result++;
      }
    }
  }

  return result;
};
```

2948 Make Lexicographically Smallest Array By Swapping Elements

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2948. Make Lexicographically Smallest Array by Swapping Elements
 * https://leetcode.com/problems/make-lexicographically-smallest-array-by-swapping-elements/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of positive integers nums and a positive integer limit.
 *
 * In one operation, you can choose any two indices i and j and swap nums[i] and nums[j]
 * if |nums[i] - nums[j]| <= limit.
 *
 * Return the lexicographically smallest array that can be obtained by performing the
 * operation any number of times.
 *
 * An array a is lexicographically smaller than an array b if in the first position where
 * a and b differ, array a has an element that is less than the corresponding element in b.
 * For example, the array [2,10,3] is lexicographically smaller than the array [10,2,3]
 * because they differ at index 0 and 2 < 10.
 */

/**
 * @param {number[]} nums
 * @param {number} limit
 * @return {number[]}
 */
var lexicographicallySmallestArray = function(nums, limit) {
  const keys = new Array(nums.length).fill(0).map((_, i) => i);
  keys.sort((i, j) => nums[i] - nums[j]);

  const result = new Array(nums.length).fill(0);
  for (let i = 0; i < nums.length;) {
    let j = i + 1;

    while (j < nums.length && nums[keys[j]] - nums[keys[j - 1]] <= limit) {
      j++;
    }

    const keyRange = keys.slice(i, j).sort((a, b) => a - b);
    for (let k = 0; k < keyRange.length; k++) {
      result[keyRange[k]] = nums[keys[i + k]];
    }
    i = j;
  }
```

```
  return result;
};
```

2950 Number Of Divisible Substrings

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2950. Number of Divisible Substrings
 * https://leetcode.com/problems/number-of-divisible-substrings/
 * Difficulty: Medium
 *
 * Each character of the English alphabet has been mapped to a digit as shown below.
 *
 * A string is divisible if the sum of the mapped values of its characters is divisible
 * by its length.
 *
 * Given a string s, return the number of divisible substrings of s.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {string} word
 * @return {number}
 */
var countDivisibleSubstrings = function(word) {
  let result = 0;

  for (let i = 1; i < 10; i++) {
    const prefixCount = new Map([[0, 1]]);
    let prefixSum = 0;

    for (const char of word) {
      const charValue = 9 - Math.floor((122 - char.charCodeAt(0)) / 3);
      prefixSum += charValue - i;
      result += prefixCount.get(prefixSum) || 0;
      prefixCount.set(prefixSum, (prefixCount.get(prefixSum) || 0) + 1);
    }
  }

  return result;
};
```

2951 Find The Peaks

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2951. Find the Peaks
 * https://leetcode.com/problems/find-the-peaks/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array mountain. Your task is to find all the peaks in the
 * mountain array.
 *
 * Return an array that consists of indices of peaks in the given array in any order.
 *
 * Notes:
 * - A peak is defined as an element that is strictly greater than its neighboring elements.
 * - The first and last elements of the array are not a peak.
 */

/**
 * @param {number[]} mountain
```

```
 * @return {number[]}
 */
var findPeaks = function(mountain) {
  const result = [];

  for (let i = 1; i < mountain.length - 1; i++) {
    if (mountain[i] > mountain[i - 1] && mountain[i] > mountain[i + 1]) {
      result.push(i);
    }
  }

  return result;
};
```

2952 Minimum Number Of Coins To Be Added

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2952. Minimum Number of Coins to be Added
 * https://leetcode.com/problems/minimum-number-of-coins-to-be-added/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array coins, representing the values of the coins
available,
 * and an integer target.
 *
 * An integer x is obtainable if there exists a subsequence of coins that sums to x.
 *
 * Return the minimum number of coins of any value that need to be added to the array so that
every
 * integer in the range [1, target] is obtainable.
 *
 * A subsequence of an array is a new non-empty array that is formed from the original array
by
 * deleting some (possibly none) of the elements without disturbing the relative positions of
the
 * remaining elements.
 */

/**
 * @param {number[]} coins
 * @param {number} target
 * @return {number}
 */
var minimumAddedCoins = function(coins, target) {
  coins.sort((a, b) => a - b);
  let result = 0;
  let currentMax = 0;
  let index = 0;

  while (currentMax < target) {
    if (index < coins.length && coins[index] <= currentMax + 1) {
      currentMax += coins[index];
      index++;
    } else {
      currentMax += currentMax + 1;
      result++;
    }
  }

  return result;
};
```

2955 Number Of Same End Substrings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2955. Number of Same-End Substrings
 * https://leetcode.com/problems/number-of-same-end-substrings/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string s, and a 2D array of integers queries, where
 * queries[i] = [li, ri] indicates a substring of s starting from the index li and
 * ending at the index ri (both inclusive), i.e. s[li..ri].
 *
 * Return an array ans where ans[i] is the number of same-end substrings of queries[i].
 *
 * A 0-indexed string t of length n is called same-end if it has the same character at
 * both of its ends, i.e., t[0] == t[n - 1].
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {string} s
 * @param {number[][]} queries
 * @return {number[]}
 */
var sameEndSubstringCount = function(s, queries) {
  const n = s.length;
  const prefixCounts = new Array(26).fill().map(() => new Array(n + 1).fill(0));

  for (let i = 0; i < n; i++) {
    const charIndex = s.charCodeAt(i) - 97;
    for (let j = 0; j < 26; j++) {
      prefixCounts[j][i + 1] = prefixCounts[j][i];
    }
    prefixCounts[charIndex][i + 1]++;
  }

  const result = [];

  for (const [left, right] of queries) {
    let count = 0;

    for (let charIndex = 0; charIndex < 26; charIndex++) {
      const charCount = prefixCounts[charIndex][right + 1] - prefixCounts[charIndex][left];
      count += Math.floor(charCount * (charCount + 1) / 2);
    }

    result.push(count);
  }

  return result;
};
```

2956 Find Common Elements Between Two Arrays

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2956. Find Common Elements Between Two Arrays
 * https://leetcode.com/problems/find-common-elements-between-two-arrays/
 * Difficulty: Easy
 *
 * You are given two integer arrays nums1 and nums2 of sizes n and m, respectively.
 * Calculate the following values:
```

```
 * - answer1 : the number of indices i such that nums1[i] exists in nums2.
 * - answer2 : the number of indices i such that nums2[i] exists in nums1.
 *
 * Return [answer1,answer2].
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var findIntersectionValues = function(nums1, nums2) {
  const set1 = new Set(nums1);
  const set2 = new Set(nums2);

  let count1 = 0;
  let count2 = 0;

  for (const num of nums1) {
    if (set2.has(num)) {
      count1++;
    }
  }

  for (const num of nums2) {
    if (set1.has(num)) {
      count2++;
    }
  }

  return [count1, count2];
};
```

2957 Remove Adjacent Almost Equal Characters

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2957. Remove Adjacent Almost-Equal Characters
 * https://leetcode.com/problems/remove-adjacent-almost-equal-characters/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string word.
 *
 * In one operation, you can pick any index i of word and change word[i] to any lowercase
 * English letter.
 *
 * Return the minimum number of operations needed to remove all adjacent almost-equal
 * characters from word.
 *
 * Two characters a and b are almost-equal if a == b or a and b are adjacent in the alphabet.
 */

/**
 * @param {string} word
 * @return {number}
 */
var removeAlmostEqualCharacters = function(word) {
  let result = 0;

  for (let i = 1; i < word.length; i++) {
    if (Math.abs(word.charCodeAt(i) - word.charCodeAt(i - 1)) <= 1) {
      result++;
      i++;
    }
  }

  return result;
};
```

2958 Length Of Longest Subarray With At Most K Frequency

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2958. Length of Longest Subarray With at Most K Frequency
 * https://leetcode.com/problems/length-of-longest-subarray-with-at-most-k-frequency/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k.
 *
 * The frequency of an element x is the number of times it occurs in an array.
 *
 * An array is called good if the frequency of each element in this array is less than or
 * equal to k.
 *
 * Return the length of the longest good subarray of nums.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxSubarrayLength = function(nums, k) {
  const map = new Map();
  let result = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    map.set(nums[right], (map.get(nums[right]) || 0) + 1);

    while (map.get(nums[right]) > k) {
      map.set(nums[left], map.get(nums[left]) - 1);
      left++;
    }

    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

2960 Count Tested Devices After Test Operations

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2960. Count Tested Devices After Test Operations
 * https://leetcode.com/problems/count-tested-devices-after-test-operations/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array batteryPercentages having length n, denoting the
 * battery percentages of n 0-indexed devices.
 *
 * Your task is to test each device i in order from 0 to n - 1, by performing the following
 * test operations:
 * - If batteryPercentages[i] is greater than 0:
 *   - Increment the count of tested devices.
 *   - Decrease the battery percentage of all devices with indices j in the range [i + 1, n -
1]
```

```
 *     by 1, ensuring their battery percentage never goes below 0, i.e,
 *     batteryPercentages[j] = max(0, batteryPercentages[j] - 1).
 *   - Move to the next device.
 * - Otherwise, move to the next device without performing any test.
 *
 * Return an integer denoting the number of devices that will be tested after performing the
test
 * operations in order.
 */

/**
 * @param {number[]} batteryPercentages
 * @return {number}
 */
var countTestedDevices = function(batteryPercentages) {
  let result = 0;

  for (let i = 0; i < batteryPercentages.length; i++) {
    if (batteryPercentages[i] > 0) {
      result++;
      for (let j = i + 1; j < batteryPercentages.length; j++) {
        batteryPercentages[j] = Math.max(0, batteryPercentages[j] - 1);
      }
    }
  }

  return result;
};
```

2961 Double Modular Exponentiation

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2961. Double Modular Exponentiation
 * https://leetcode.com/problems/double-modular-exponentiation/
 * Difficulty: Medium
 *
 * You are given a 0-indexed 2D array variables where variables[i] = [ai, bi, ci, mi], and an
 * integer target.
 *
 * An index i is good if the following formula holds:
 * - 0 <= i < variables.length
 * - ((aibi % 10)ci) % mi == target
 *
 * Return an array consisting of good indices in any order.
 */

/**
 * @param {number[][]} variables
 * @param {number} target
 * @return {number[]}
 */
var getGoodIndices = function(variables, target) {
  const result = [];

  for (let i = 0; i < variables.length; i++) {
    const [base, exp1, exp2, modulus] = variables[i];

    let inner = 1;
    for (let j = 0; j < exp1; j++) {
      inner = (inner * base) % 10;
    }

    let count = 1;
    for (let j = 0; j < exp2; j++) {
      count = (count * inner) % modulus;
    }

    if (count === target) {
```

```
      result.push(i);
    }
  }

  return result;
};
```

2962 Count Subarrays Where Max Element Appears At Least K Times

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2962. Count Subarrays Where Max Element Appears at Least K Times
 * https://leetcode.com/problems/count-subarrays-where-max-element-appears-at-least-k-times/
 * Difficulty: Medium
 *
 * You are given an integer array nums and a positive integer k.
 *
 * Return the number of subarrays where the maximum element of nums appears at least k times
in
 * that subarray.
 *
 * A subarray is a contiguous sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countSubarrays = function(nums, k) {
  const maxNum = Math.max(...nums);
  let maxCount = 0;
  let left = 0;
  let result = 0;

  for (let right = 0; right < nums.length; right++) {
    if (nums[right] === maxNum) maxCount++;
    while (maxCount >= k) {
      result += nums.length - right;
      if (nums[left] === maxNum) maxCount--;
      left++;
    }
  }

  return result;
};
```

2963 Count The Number Of Good Partitions

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2963. Count the Number of Good Partitions
 * https://leetcode.com/problems/count-the-number-of-good-partitions/
 * Difficulty: Hard
 *
 * You are given a 0-indexed array nums consisting of positive integers.
 *
 * A partition of an array into one or more contiguous subarrays is called good if no two
 * subarrays contain the same number.
 *
 * Return the total number of good partitions of nums.
 *
 * Since the answer may be large, return it modulo 109 + 7.
```

```
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var numberOfGoodPartitions = function(nums) {
  const mod = 1e9 + 7;
  const map = new Map();
  let segments = 0;
  let end = -1;

  for (let i = 0; i < nums.length; i++) {
    map.set(nums[i], i);
  }

  for (let i = 0; i < nums.length; i++) {
    if (i > end) {
      segments++;
    }
    end = Math.max(end, map.get(nums[i]));
  }

  let result = 1;
  for (let i = 1; i < segments; i++) {
    result = (result * 2) % mod;
  }

  return result;
};
```

2964 Number Of Divisible Triplet Sums

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2964. Number of Divisible Triplet Sums
 * https://leetcode.com/problems/number-of-divisible-triplet-sums/
 * Difficulty: Medium
 *
 * Given a 0-indexed integer array nums and an integer d, return the number of triplets
 * (i, j, k) such that i < j < k and (nums[i] + nums[j] + nums[k]) % d == 0.
 */

/**
 * @param {number[]} nums
 * @param {number} d
 * @return {number}
 */
var divisibleTripletCount = function(nums, d) {
  const n = nums.length;
  const pairSums = new Map();

  for (let i = 0; i < n - 1; i++) {
    for (let j = i + 1; j < n; j++) {
      const sumMod = (nums[i] + nums[j]) % d;
      if (!pairSums.has(sumMod)) {
        pairSums.set(sumMod, []);
      }
      pairSums.get(sumMod).push([i, j]);
    }
  }

  let result = 0;
  for (let k = 0; k < n; k++) {
    const targetKey = (d - nums[k] % d) % d;
    if (pairSums.has(targetKey)) {
      for (const [i, j] of pairSums.get(targetKey)) {
        if (j < k) {
          result++;
```

```
        }
      }
    }
  }

  return result;
};
```

2965 Find Missing And Repeated Values

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 2965. Find Missing and Repeated Values
 * https://leetcode.com/problems/find-missing-and-repeated-values/
 * Difficulty: Easy
 *
 * You are given a 0-indexed 2D integer matrix grid of size n * n with values in the range [1,
 n2].
 * Each integer appears exactly once except a which appears twice and b which is missing. The
task
 * is to find the repeating and missing numbers a and b.
 *
 * Return a 0-indexed integer array ans of size 2 where ans[0] equals to a and ans[1] equals
to b.
 */

/**
 * @param {number[][]} grid
 * @return {number[]}
 */
var findMissingAndRepeatedValues = function(grid) {
  const sum = grid.flat().reduce((a, b) => a + b, 0);
  const expectedSum = (grid.length * grid.length * (grid.length * grid.length + 1)) / 2;
  const repeated = sum - [...new Set(grid.flat())].reduce((a, b) => a + b, 0);
  return [repeated, expectedSum - sum + repeated];
};
```

2966 Divide Array Into Arrays With Max Difference

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2966. Divide Array Into Arrays With Max Difference
 * https://leetcode.com/problems/divide-array-into-arrays-with-max-difference/
 * Difficulty: Medium
 *
 * You are given an integer array nums of size n where n is a multiple of 3 and a positive
 * integer k.
 *
 * Divide the array nums into n / 3 arrays of size 3 satisfying the following condition:
 * - The difference between any two elements in one array is less than or equal to k.
 *
 * Return a 2D array containing the arrays. If it is impossible to satisfy the conditions,
return
 * an empty array. And if there are multiple answers, return any of them.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[][]}
 */
var divideArray = function(nums, k) {
  nums.sort((a, b) => a - b);
```

```
  const result = [];

  for (let i = 0; i < nums.length; i += 3) {
    if (nums[i + 2] - nums[i] > k) {
      return [];
    }
    result.push([nums[i], nums[i + 1], nums[i + 2]]);
  }

  return result;
};
```

2969 Minimum Number Of Coins For Fruits Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2969. Minimum Number of Coins for Fruits II
 * https://leetcode.com/problems/minimum-number-of-coins-for-fruits-ii/
 * Difficulty: Hard
 *
 * You are at a fruit market with different types of exotic fruits on display.
 *
 * You are given a 1-indexed array prices, where prices[i] denotes the number of coins
 * needed to purchase the ith fruit.
 *
 * The fruit market has the following offer:
 * - If you purchase the ith fruit at prices[i] coins, you can get the next i fruits for free.
 *
 * Note that even if you can take fruit j for free, you can still purchase it for prices[j]
 * coins to receive a new offer.
 *
 * Return the minimum number of coins needed to acquire all the fruits.
 */

/**
 * @param {number[]} prices
 * @return {number}
 */
var minimumCoins = function(prices) {
  const n = prices.length;
  const dp = new Array(n);
  const deque = [];

  dp[0] = prices[0];
  deque.push(0);

  for (let i = 1; i < n; i++) {
    dp[i] = dp[deque[0]] + prices[i];

    while (deque.length > 0 && deque[0] + deque[0] + 1 < i) {
      deque.shift();
    }
    while (deque.length > 0 && dp[deque[deque.length - 1]] >= dp[i]) {
      deque.pop();
    }

    deque.push(i);
  }

  return dp[deque[0]];
};
```

2970 Count The Number Of Incremovable Subarrays I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2970. Count the Number of Incremovable Subarrays I
 * https://leetcode.com/problems/count-the-number-of-incremovable-subarrays-i/
 * Difficulty: Easy
 *
 * You are given a 0-indexed array of positive integers nums.
 *
 * A subarray of nums is called incremovable if nums becomes strictly increasing on removing
the
 * subarray. For example, the subarray [3, 4] is an incremovable subarray of [5, 3, 4, 6, 7]
because
 * removing this subarray changes the array [5, 3, 4, 6, 7] to [5, 6, 7] which is strictly
 * increasing.
 *
 * Return the total number of incremovable subarrays of nums.
 *
 * Note that an empty array is considered strictly increasing.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var incremovableSubarrayCount = function(nums) {
  let result = 0;

  for (let start = 0; start < nums.length; start++) {
    for (let end = start; end < nums.length; end++) {
      let isIncreasing = true;
      let prev = -Infinity;

      for (let i = 0; i < nums.length; i++) {
        if (i >= start && i <= end) continue;
        if (nums[i] <= prev) {
          isIncreasing = false;
          break;
        }
        prev = nums[i];
      }

      if (isIncreasing) result++;
    }
  }

  return result;
};
```

2971 Find Polygon With The Largest Perimeter

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 2971. Find Polygon With the Largest Perimeter
 * https://leetcode.com/problems/find-polygon-with-the-largest-perimeter/
 * Difficulty: Medium
 *
 * You are given an array of positive integers nums of length n.
 *
 * A polygon is a closed plane figure that has at least 3 sides. The longest side of a polygon
 is
 * smaller than the sum of its other sides.
 *
 * Conversely, if you have k (k >= 3) positive real numbers a1, a2, a3, ..., ak where
 * a1 <= a2 <= a3 <= ... <= ak and a1 + a2 + a3 + ... + ak-1 > ak, then there always exists
 * a polygon with k sides whose lengths are a1, a2, a3, ..., ak.
```

```
 *
 * The perimeter of a polygon is the sum of lengths of its sides.
 *
 * Return the largest possible perimeter of a polygon whose sides can be formed from nums, or
-1
 * if it is not possible to create a polygon.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var largestPerimeter = function(nums) {
  nums.sort((a, b) => b - a);
  for (let i = 0; i < nums.length - 2; i++) {
    if (nums[i] < nums.slice(i + 1).reduce((sum, num) => sum + num, 0)) {
      return nums.slice(i).reduce((sum, num) => sum + num, 0);
    }
  }
  return -1;
};
```

2976 Minimum Cost To Convert String I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2976. Minimum Cost to Convert String I
 * https://leetcode.com/problems/minimum-cost-to-convert-string-i/
 * Difficulty: Medium
 *
 * You are given two 0-indexed strings source and target, both of length n and consisting of
 * lowercase English letters. You are also given two 0-indexed character arrays original and
 * changed, and an integer array cost, where cost[i] represents the cost of changing the
character
 * original[i] to the character changed[i].
 *
 * You start with the string source. In one operation, you can pick a character x from the
string
 * and change it to the character y at a cost of z if there exists any index j such that
 * cost[j] == z, original[j] == x, and changed[j] == y.
 *
 * Return the minimum cost to convert the string source to the string target using any number
of
 * operations. If it is impossible to convert source to target, return -1.
 *
 * Note that there may exist indices i, j such that original[j] == original[i] and
 * changed[j] == changed[i].
 */

/**
 * @param {string} source
 * @param {string} target
 * @param {character[]} original
 * @param {character[]} changed
 * @param {number[]} cost
 * @return {number}
 */
var minimumCost = function(source, target, original, changed, cost) {
  const graph = new Array(26).fill().map(() => new Array(26).fill(Infinity));
  for (let i = 0; i < 26; i++) graph[i][i] = 0;

  for (let i = 0; i < original.length; i++) {
    const from = original[i].charCodeAt(0) - 97;
    const to = changed[i].charCodeAt(0) - 97;
    graph[from][to] = Math.min(graph[from][to], cost[i]);
  }

  for (let k = 0; k < 26; k++) {
    for (let i = 0; i < 26; i++) {
```

```
      for (let j = 0; j < 26; j++) {
        graph[i][j] = Math.min(graph[i][j], graph[i][k] + graph[k][j]);
      }
    }
  }

  let result = 0;
  for (let i = 0; i < source.length; i++) {
    if (source[i] !== target[i]) {
      const from = source[i].charCodeAt(0) - 97;
      const to = target[i].charCodeAt(0) - 97;
      if (graph[from][to] === Infinity) return -1;
      result += graph[from][to];
    }
  }

  return result;
};
```

2979 Most Expensive Item That Can Not Be Bought

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 2979. Most Expensive Item That Can Not Be Bought
 * https://leetcode.com/problems/most-expensive-item-that-can-not-be-bought/
 * Difficulty: Medium
 *
 * You are given two distinct prime numbers primeOne and primeTwo.
 *
 * Alice and Bob are visiting a market. The market has an infinite number of items, for any
 * positive integer x there exists an item whose price is x. Alice wants to buy some items
 * from the market to gift to Bob. She has an infinite number of coins in the denomination
 * primeOne and primeTwo. She wants to know the most expensive item she can not buy to gift
 * to Bob.
 *
 * Return the price of the most expensive item which Alice can not gift to Bob.
 */

/**
 * @param {number} primeOne
 * @param {number} primeTwo
 * @return {number}
 */
var mostExpensiveItem = function(primeOne, primeTwo) {
  return primeOne * primeTwo - primeOne - primeTwo;
};
```

2980 Check If Bitwise Or Has Trailing Zeros

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2980. Check if Bitwise OR Has Trailing Zeros
 * https://leetcode.com/problems/check-if-bitwise-or-has-trailing-zeros/
 * Difficulty: Easy
 *
 * You are given an array of positive integers nums.
 *
 * You have to check if it is possible to select two or more elements in the array such
 * that the bitwise OR of the selected elements has at least one trailing zero in its
 * binary representation.
 *
 * For example, the binary representation of 5, which is "101", does not have any trailing
 * zeros, whereas the binary representation of 4, which is "100", has two trailing zeros.
```

```
 *
 * Return true if it is possible to select two or more elements whose bitwise OR has trailing
 * zeros, return false otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var hasTrailingZeros = function(nums) {
  let evenCount = 0;

  for (const num of nums) {
    if (num % 2 === 0) {
      evenCount++;
      if (evenCount >= 2) {
        return true;
      }
    }
  }

  return false;
};
```

2981 Find Longest Special Substring That Occurs Thrice I

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2981. Find Longest Special Substring That Occurs Thrice I
 * https://leetcode.com/problems/find-longest-special-substring-that-occurs-thrice-i/
 * Difficulty: Medium
 *
 * You are given a string s that consists of lowercase English letters.
 *
 * A string is called special if it is made up of only a single character. For example, the
 * string "abc" is not special, whereas the strings "ddd", "zz", and "f" are special.
 *
 * Return the length of the longest special substring of s which occurs at least thrice,
 * or -1 if no special substring occurs at least thrice.
 *
 * A substring is a contiguous non-empty sequence of characters within a string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maximumLength = function(s) {
  const map = new Map();
  let result = -1;

  for (let i = 0; i < s.length; i++) {
    for (let len = 1; len <= s.length - i; len++) {
      const substr = s.slice(i, i + len);
      if (substr.split('').every(c => c === substr[0])) {
        const count = (map.get(substr) || 0) + 1;
        map.set(substr, count);
        if (count >= 3) {
          result = Math.max(result, len);
        }
      }
    }
  }

  return result;
};
```

2992 Number Of Self Divisible Permutations

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2992. Number of Self-Divisible Permutations
 * https://leetcode.com/problems/number-of-self-divisible-permutations/
 * Difficulty: Medium
 *
 * Given an integer n, return the number of permutations of the 1-indexed array
 * nums = [1, 2, ..., n], such that it's self-divisible.
 *
 * A 1-indexed array a of length n is self-divisible if for every 1 <= i <= n, gcd(a[i], i) ==
 1.
 *
 * A permutation of an array is a rearrangement of the elements of that array, for example
here
 * are all of the permutations of the array [1, 2, 3]:
 * - [1, 2, 3]
 * - [1, 3, 2]
 * - [2, 1, 3]
 * - [2, 3, 1]
 * - [3, 1, 2]
 * - [3, 2, 1]
 */

/**
 * @param {number} n
 * @return {number}
 */
var selfDivisiblePermutationCount = function(n) {
  const graph = new Array(n + 1).fill().map(() => []);

  for (let position = 1; position <= n; position++) {
    for (let value = 1; value <= n; value++) {
      if (gcd(value, position) === 1) {
        graph[position].push(value);
      }
    }
  }

  return countPermutations(1, 0);

  function countPermutations(position, usedMask) {
    if (position > n) return 1;

    let count = 0;
    for (const value of graph[position]) {
      if ((usedMask & (1 << value)) === 0) {
        count += countPermutations(position + 1, usedMask | (1 << value));
      }
    }

    return count;
  }

  function gcd(a, b) {
    while (b !== 0) {
      const temp = b;
      b = a % b;
      a = temp;
    }
    return a;
  }
};
```

2997 Minimum Number Of Operations To Make Array Xor Equal To K

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2997. Minimum Number of Operations to Make Array XOR Equal to K
 * https://leetcode.com/problems/minimum-number-of-operations-to-make-array-xor-equal-to-k/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums and a positive integer k.
 *
 * You can apply the following operation on the array any number of times:
 * - Choose any element of the array and flip a bit in its binary representation. Flipping a
bit
 *   means changing a 0 to 1 or vice versa.
 *
 * Return the minimum number of operations required to make the bitwise XOR of all elements of
 the
 * final array equal to k.
 *
 * Note that you can flip leading zero bits in the binary representation of elements. For
example,
 * for the number (101)2 you can flip the fourth bit and obtain (1101)2.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minOperations = function(nums, k) {
  let xorResult = 0;
  for (const num of nums) {
    xorResult ^= num;
  }

  let result = 0;
  let diff = xorResult ^ k;
  while (diff > 0) {
    result += diff & 1;
    diff >>= 1;
  }

  return result;
};
```

2998 Minimum Number Of Operations To Make X And Y Equal

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2998. Minimum Number of Operations to Make X and Y Equal
 * https://leetcode.com/problems/minimum-number-of-operations-to-make-x-and-y-equal/
 * Difficulty: Medium
 *
 * You are given two positive integers x and y.
 *
 * In one operation, you can do one of the four following operations:
 * 1. Divide x by 11 if x is a multiple of 11.
 * 2. Divide x by 5 if x is a multiple of 5.
 * 3. Decrement x by 1.
 * 4. Increment x by 1.
 *
```

```
 * Return the minimum number of operations required to make x and y equal.
 */

/**
 * @param {number} x
 * @param {number} y
 * @return {number}
 */
var minimumOperationsToMakeEqual = function(x, y) {
  const queue = [[x, 0]];
  const visited = new Set([x]);

  while (queue.length) {
    const [current, steps] = queue.shift();

    if (current === y) return steps;

    const nextStates = [];
    if (current % 11 === 0) nextStates.push(current / 11);
    if (current % 5 === 0) nextStates.push(current / 5);
    nextStates.push(current - 1, current + 1);

    for (const next of nextStates) {
      if (next >= 0 && next <= 10000 && !visited.has(next)) {
        visited.add(next);
        queue.push([next, steps + 1]);
      }
    }
  }

  return -1;
};
```

2999 Count The Number Of Powerful Integers

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 2999. Count the Number of Powerful Integers
 * https://leetcode.com/problems/count-the-number-of-powerful-integers/
 * Difficulty: Hard
 *
 * You are given three integers start, finish, and limit. You are also given a 0-indexed
string
 * s representing a positive integer.
 *
 * A positive integer x is called powerful if it ends with s (in other words, s is a suffix of
 x)
 * and each digit in x is at most limit.
 *
 * Return the total number of powerful integers in the range [start..finish].
 *
 * A string x is a suffix of a string y if and only if x is a substring of y that starts from
 * some index (including 0) in y and extends to the index y.length - 1. For example, 25 is a
 * suffix of 5125 whereas 512 is not.
 */

/**
 * @param {number} start
 * @param {number} finish
 * @param {number} limit
 * @param {string} suffix
 * @return {number}
 */
var numberOfPowerfulInt = function(start, finish, limit, suffix) {
  start--;
  const startCount = countValidNums(start, limit, suffix);
  const finishCount = countValidNums(finish, limit, suffix);
  return finishCount - startCount;
```

```
  function countValidNums(number, maxDigit, suffix) {
    const suffixNum = parseInt(suffix);
    const suffixMod = 10 ** suffix.length;
    const suffixPart = number % suffixMod;
    let baseNum = Math.floor(number / suffixMod);
    if (suffixPart < suffixNum) baseNum--;

    if (baseNum <= 0) return baseNum + 1;

    const digitStr = baseNum.toString();
    let count = digitStr.charCodeAt(0) - 48;
    let isExact = 1;

    if (count > maxDigit) {
      return (maxDigit + 1) ** digitStr.length;
    }

    for (let i = 1; i < digitStr.length; i++) {
      count *= (maxDigit + 1);
      if (isExact) {
        const currentDigit = digitStr.charCodeAt(i) - 48;
        if (currentDigit > maxDigit) {
          isExact = 0;
          count += maxDigit + 1;
        } else {
          count += currentDigit;
        }
      }
    }

    return count + isExact;
  }
};
```

3000 Maximum Area Of Longest Diagonal Rectangle

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3000. Maximum Area of Longest Diagonal Rectangle
 * https://leetcode.com/problems/maximum-area-of-longest-diagonal-rectangle/
 * Difficulty: Easy
 *
 * You are given a 2D 0-indexed integer array dimensions.
 *
 * For all indices i, 0 <= i < dimensions.length, dimensions[i][0] represents the length and
 * dimensions[i][1] represents the width of the rectangle i.
 *
 * Return the area of the rectangle having the longest diagonal. If there are multiple
rectangles
 * with the longest diagonal, return the area of the rectangle having the maximum area.
 */

/**
 * @param {number[][]} dimensions
 * @return {number}
 */
var areaOfMaxDiagonal = function(dimensions) {
  let maxDiagonal = 0;
  let result = 0;

  for (const [length, width] of dimensions) {
    const diagonal = length * length + width * width;
    const area = length * width;

    if (diagonal > maxDiagonal || (diagonal === maxDiagonal && area > result)) {
      maxDiagonal = diagonal;
      result = area;
    }
  }
```

```
  return result;
};
```

3002 Maximum Size Of A Set After Removals

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3002. Maximum Size of a Set After Removals
 * https://leetcode.com/problems/maximum-size-of-a-set-after-removals/
 * Difficulty: Medium
 *
 * You are given two 0-indexed integer arrays nums1 and nums2 of even length n.
 *
 * You must remove n / 2 elements from nums1 and n / 2 elements from nums2. After the
removals,
 * you insert the remaining elements of nums1 and nums2 into a set s.
 *
 * Return the maximum possible size of the set s.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var maximumSetSize = function(nums1, nums2) {
  const n = nums1.length;
  const set1 = new Set(nums1);
  const set2 = new Set(nums2);

  let unique1 = set1.size;
  let unique2 = set2.size;
  let common = 0;

  for (const num of set1) {
    if (set2.has(num)) common++;
  }

  unique1 -= common;
  unique2 -= common;

  const max1 = Math.min(unique1, n / 2);
  const max2 = Math.min(unique2, n / 2);
  const remaining = n / 2 - max1 + n / 2 - max2;
  const maxCommon = Math.min(common, remaining);

  return max1 + max2 + maxCommon;
};
```

3004 Maximum Subtree Of The Same Color

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 3004. Maximum Subtree of the Same Color
 * https://leetcode.com/problems/maximum-subtree-of-the-same-color/
 * Difficulty: Medium
 *
 * You are given a 2D integer array edges representing a tree with n nodes, numbered from
 * 0 to n - 1, rooted at node 0, where edges[i] = [ui, vi] means there is an edge between
 * the nodes vi and ui.
 *
 * You are also given a 0-indexed integer array colors of size n, where colors[i] is the
```

```
 * color assigned to node i.
 *
 * We want to find a node v such that every node in the subtree of v has the same color.
 *
 * Return the size of such subtree with the maximum number of nodes possible.
 */

/**
 * @param {number[][]} edges
 * @param {number[]} colors
 * @return {number}
 */
var maximumSubtreeSize = function(edges, colors) {
  const n = colors.length;
  const graph = new Array(n).fill().map(() => []);

  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u);
  }

  let result = 1;

  dfs(0, -1);

  return result;

  function dfs(node, parent) {
    let subtreeSize = 1;
    let isValidSubtree = true;

    for (const child of graph[node]) {
      if (child !== parent) {
        const childSize = dfs(child, node);

        if (colors[child] === colors[node] && childSize > 0) {
          subtreeSize += childSize;
        } else {
          isValidSubtree = false;
        }
      }
    }

    if (isValidSubtree) {
      result = Math.max(result, subtreeSize);
      return subtreeSize;
    }

    return 0;
  }
};
```

3005 Count Elements With Maximum Frequency

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3005. Count Elements With Maximum Frequency
 * https://leetcode.com/problems/count-elements-with-maximum-frequency/
 * Difficulty: Easy
 *
 * You are given an array nums consisting of positive integers.
 *
 * Return the total frequencies of elements in nums such that those elements all have the
 * maximum frequency.
 *
 * The frequency of an element is the number of occurrences of that element in the array.
 */

/**
```

```
 * @param {number[]} nums
 * @return {number}
 */
var maxFrequencyElements = function(nums) {
  const map = new Map();
  let maxFreq = 0;
  let result = 0;

  for (const num of nums) {
    const count = (map.get(num) || 0) + 1;
    map.set(num, count);
    if (count > maxFreq) {
      maxFreq = count;
      result = count;
    } else if (count === maxFreq) {
      result += count;
    }
  }

  return result;
};
```

3009 Maximum Number Of Intersections On The Chart

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3009. Maximum Number of Intersections on the Chart
 * https://leetcode.com/problems/maximum-number-of-intersections-on-the-chart/
 * Difficulty: Hard
 *
 * There is a line chart consisting of n points connected by line segments. You are given a
 * 1-indexed integer array y. The kth point has coordinates (k, y[k]). There are no horizontal
 * lines; that is, no two consecutive points have the same y-coordinate.
 *
 * We can draw an infinitely long horizontal line. Return the maximum number of points of
 * intersection of the line with the chart.
 */

/**
 * @param {number[]} y
 * @return {number}
 */
var maxIntersectionCount = function(y) {
  const n = y.length;
  const coordinateEvents = new Map();

  for (let segmentIndex = 1; segmentIndex < n; segmentIndex++) {
    const segmentStart = 2 * y[segmentIndex - 1];
    const segmentEnd = 2 * y[segmentIndex]
      + (segmentIndex === n - 1 ? 0 : y[segmentIndex] > y[segmentIndex - 1] ? -1 : 1);
    const intervalStart = Math.min(segmentStart, segmentEnd);
    const intervalEnd = Math.max(segmentStart, segmentEnd);

    coordinateEvents.set(intervalStart, (coordinateEvents.get(intervalStart) || 0) + 1);
    coordinateEvents.set(intervalEnd + 1, (coordinateEvents.get(intervalEnd + 1) || 0) - 1);
  }

  const sortedEvents = [...coordinateEvents.entries()].sort((a, b) => a[0] - b[0]);
  let activeSegments = 0;
  let result = 0;
  for (const [coordinate, segmentChange] of sortedEvents) {
    activeSegments += segmentChange;
    result = Math.max(result, activeSegments);
  }

  return result;
};
```

3010 Divide An Array Into Subarrays With Minimum Cost I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3010. Divide an Array Into Subarrays With Minimum Cost I
 * https://leetcode.com/problems/divide-an-array-into-subarrays-with-minimum-cost-i/
 * Difficulty: Easy
 *
 * You are given an array of integers nums of length n.
 *
 * The cost of an array is the value of its first element. For example, the cost of [1,2,3]
 * is 1 while the cost of [3,4,1] is 3.
 *
 * You need to divide nums into 3 disjoint contiguous subarrays.
 *
 * Return the minimum possible sum of the cost of these subarrays.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumCost = function(nums) {
  const n = nums.length;
  let result = Infinity;

  for (let i = 1; i < n - 1; i++) {
    for (let j = i + 1; j < n; j++) {
      const cost = nums[0] + nums[i] + nums[j];
      result = Math.min(result, cost);
    }
  }

  return result;
};
```

3011 Find If Array Can Be Sorted

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3011. Find if Array Can Be Sorted
 * https://leetcode.com/problems/find-if-array-can-be-sorted/
 * Difficulty: Medium
 *
 * You are given a 0-indexed array of positive integers nums.
 *
 * In one operation, you can swap any two adjacent elements if they have the same number of
 * set bits. You are allowed to do this operation any number of times (including zero).
 *
 * Return true if you can sort the array in ascending order, else return false.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var canSortArray = function(nums) {
  const sorted = [...nums].sort((a, b) => a - b);
  const bitGroups = [];
  let currentBits = countSetBits(nums[0]);
  let group = [nums[0]];
```

```
  for (let i = 1; i < nums.length; i++) {
    const bits = countSetBits(nums[i]);
    if (bits === currentBits) {
      group.push(nums[i]);
    } else {
      bitGroups.push(group);
      group = [nums[i]];
      currentBits = bits;
    }
  }
  bitGroups.push(group);

  let index = 0;
  for (const group of bitGroups) {
    const groupSorted = [...group].sort((a, b) => a - b);
    for (const num of groupSorted) {
      if (num !== sorted[index]) {
        return false;
      }
      index++;
    }
  }

  return true;

  function countSetBits(num) {
    let count = 0;
    while (num) {
      count += num & 1;
      num >>= 1;
    }
    return count;
  }
};
```

3014 Minimum Number Of Pushes To Type Word I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3014. Minimum Number of Pushes to Type Word I
 * https://leetcode.com/problems/minimum-number-of-pushes-to-type-word-i/
 * Difficulty: Easy
 *
 * You are given a string word containing distinct lowercase English letters.
 *
 * Telephone keypads have keys mapped with distinct collections of lowercase English letters,
 * which can be used to form words by pushing them. For example, the key 2 is mapped with
 * ["a","b","c"], we need to push the key one time to type "a", two times to type "b", and
 * three times to type "c".
 *
 * It is allowed to remap the keys numbered 2 to 9 to distinct collections of letters. The
 * keys can be remapped to any amount of letters, but each letter must be mapped to exactly
 * one key. You need to find the minimum number of times the keys will be pushed to type
 * the string word.
 *
 * Return the minimum number of pushes needed to type word after remapping the keys.
 *
 * An example mapping of letters to keys on a telephone keypad is given below. Note that
 * 1, *, #, and 0 do not map to any letters.
 */

/**
 * @param {string} word
 * @return {number}
 */
var minimumPushes = function(word) {
  const frequency = new Array(26).fill(0);
  for (const char of word) {
```

```
    frequency[char.charCodeAt(0) - 97]++;
  }
  frequency.sort((a, b) => b - a);

  let result = 0;
  for (let i = 0; i < frequency.length; i++) {
    if (frequency[i] === 0) break;
    result += frequency[i] * (Math.floor(i / 8) + 1);
  }

  return result;
};
```

3015 Count The Number Of Houses At A Certain Distance I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3015. Count the Number of Houses at a Certain Distance I
 * https://leetcode.com/problems/count-the-number-of-houses-at-a-certain-distance-i/
 * Difficulty: Medium
 *
 * You are given three positive integers n, x, and y.
 *
 * In a city, there exist houses numbered 1 to n connected by n streets. There is a street
 * connecting the house numbered i with the house numbered i + 1 for all 1 <= i <= n - 1.
 * An additional street connects the house numbered x with the house numbered y.
 *
 * For each k, such that 1 <= k <= n, you need to find the number of pairs of houses (house1,
 * house2) such that the minimum number of streets that need to be traveled to reach house2
 * from house1 is k.
 *
 * Return a 1-indexed array result of length n where result[k] represents the total number
 * of pairs of houses such that the minimum streets required to reach one house from the
 * other is k.
 *
 * Note that x and y can be equal.
 */

/**
 * @param {number} n
 * @param {number} x
 * @param {number} y
 * @return {number[]}
 */
var countOfPairs = function(n, x, y) {
  const distances = new Array(n + 1).fill().map(() => new Array(n + 1).fill(Infinity));

  for (let i = 1; i <= n; i++) {
    distances[i][i] = 0;
    if (i < n) distances[i][i + 1] = distances[i + 1][i] = 1;
  }

  distances[x][y] = distances[y][x] = 1;

  for (let k = 1; k <= n; k++) {
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= n; j++) {
        distances[i][j] = Math.min(distances[i][j], distances[i][k] + distances[k][j]);
      }
    }
  }

  const result = new Array(n + 1).fill(0);
  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= n; j++) {
      if (i !== j && distances[i][j] !== Infinity) {
        result[distances[i][j]]++;
      }
    }
```

```
  }

  return result.slice(1);
};
```

3016 Minimum Number Of Pushes To Type Word Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3016. Minimum Number of Pushes to Type Word II
 * https://leetcode.com/problems/minimum-number-of-pushes-to-type-word-ii/
 * Difficulty: Medium
 *
 * You are given a string word containing lowercase English letters.
 *
 * Telephone keypads have keys mapped with distinct collections of lowercase English letters,
 * which can be used to form words by pushing them. For example, the key 2 is mapped with
 * ["a","b","c"], we need to push the key one time to type "a", two times to type "b", and
 * three times to type "c".
 *
 * It is allowed to remap the keys numbered 2 to 9 to distinct collections of letters. The
 * keys can be remapped to any amount of letters, but each letter must be mapped to exactly
 * one key. You need to find the minimum number of times the keys will be pushed to type the
 * string word.
 *
 * Return the minimum number of pushes needed to type word after remapping the keys.
 *
 * An example mapping of letters to keys on a telephone keypad is given below. Note that
 * 1, *, #, and 0 do not map to any letters.
 */

/**
 * @param {string} word
 * @return {number}
 */
var minimumPushes = function(word) {
  const frequency = new Array(26).fill(0);
  for (const char of word) {
    frequency[char.charCodeAt(0) - 97]++;
  }
  frequency.sort((a, b) => b - a);

  let result = 0;
  for (let i = 0; i < frequency.length && frequency[i] > 0; i++) {
    result += frequency[i] * (Math.floor(i / 8) + 1);
  }

  return result;
};
```

3018 Maximum Number Of Removal Queries That Can Be Processed I

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3018. Maximum Number of Removal Queries That Can Be Processed I
 * https://leetcode.com/problems/maximum-number-of-removal-queries-that-can-be-processed-i/
 * Difficulty: Hard
 *
 * You are given a 0-indexed array nums and a 0-indexed array queries.
 *
 * You can do the following operation at the beginning at most once:
 * - Replace nums with a subsequence of nums.
 *
```

```
 * We start processing queries in the given order; for each query, we do the following:
 * - If the first and the last element of nums is less than queries[i], the processing of
 *   queries ends.
 * - Otherwise, we choose either the first or the last element of nums if it is greater
 *   than or equal to queries[i], and we remove the chosen element from nums.
 *
 * Return the maximum number of queries that can be processed by doing the operation
optimally.
 */

/**
 * @param {number[]} nums
 * @param {number[]} queries
 * @return {number}
 */
var maximumProcessableQueries = function(nums, queries) {
  const queryCount = queries.length;
  const arrayLength = nums.length;
  const dp = new Array(arrayLength + 1).fill().map(() => new Array(arrayLength + 1).fill(0));

  nums.push(-1);
  let result = 0;

  for (let segmentLength = arrayLength - 1; segmentLength >= 0; segmentLength--) {
    for (let leftIndex = 0; leftIndex < arrayLength; leftIndex++) {
      const rightIndex = leftIndex + segmentLength;

      if (rightIndex < arrayLength) {
        dp[leftIndex][rightIndex] = Math.max(
          dp[leftIndex - 1] ? dp[leftIndex - 1][rightIndex] : 0,
          dp[leftIndex][rightIndex + 1] || 0
        );

        const prevLeftQueries = dp[leftIndex - 1] ? dp[leftIndex - 1][rightIndex] : 0;
        const prevRightQueries = dp[leftIndex][rightIndex + 1] || 0;
        if (prevLeftQueries < queryCount && nums[leftIndex - 1] >= queries[prevLeftQueries]) {
          dp[leftIndex][rightIndex] = Math.max(dp[leftIndex][rightIndex], prevLeftQueries +
1);
        }

        if (prevRightQueries < queryCount && nums[rightIndex + 1] >=
queries[prevRightQueries]) {
          dp[leftIndex][rightIndex] = Math.max(dp[leftIndex][rightIndex], prevRightQueries +
1);
        }

        if (dp[leftIndex][rightIndex] === queryCount) {
          return queryCount;
        }
      } else {
        break;
      }
    }
  }

  for (let singleIndex = 0; singleIndex < arrayLength; singleIndex++) {
    const currentQueries = dp[singleIndex][singleIndex];
    const canProcessCurrent = currentQueries < queryCount
      && nums[singleIndex] >= queries[currentQueries] ? 1 : 0;
    result = Math.max(result, currentQueries + canProcessCurrent);
  }

  return result;
};
```

3019 Number Of Changing Keys

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 3019. Number of Changing Keys
 * https://leetcode.com/problems/number-of-changing-keys/
 * Difficulty: Easy
 *
 * You are given a 0-indexed string s typed by a user. Changing a key is defined as using a
key
 * different from the last used key. For example, s = "ab" has a change of a key while s =
"bBBb"
 * does not have any.
 *
 * Return the number of times the user had to change the key.
 *
 * Note: Modifiers like shift or caps lock won't be counted in changing the key that is if a
user
 * typed the letter 'a' and then the letter 'A' then it will not be considered as a changing
of key.
 */

/**
 * @param {string} s
 * @return {number}
 */
var countKeyChanges = function(s) {
  let result = 0;

  for (let i = 1; i < s.length; i++) {
    if (s[i].toLowerCase() !== s[i - 1].toLowerCase()) {
      result++;
    }
  }

  return result;
};
```

3021 Alice And Bob Playing Flower Game

MEDIUM

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3021. Alice and Bob Playing Flower Game
 * https://leetcode.com/problems/alice-and-bob-playing-flower-game/
 * Difficulty: Medium
 *
 * Alice and Bob are playing a turn-based game on a circular field surrounded by flowers.
 * The circle represents the field, and there are x flowers in the clockwise direction
 * between Alice and Bob, and y flowers in the anti-clockwise direction between them.
 *
 * The game proceeds as follows:
 * 1. Alice takes the first turn.
 * 2. In each turn, a player must choose either the clockwise or anti-clockwise direction
 *    and pick one flower from that side.
 * 3. At the end of the turn, if there are no flowers left at all, the current player
 *    captures their opponent and wins the game.
 *
 * Given two integers, n and m, the task is to compute the number of possible pairs (x, y)
 * that satisfy the conditions:
 * - Alice must win the game according to the described rules.
 * - The number of flowers x in the clockwise direction must be in the range [1,n].
 * - The number of flowers y in the anti-clockwise direction must be in the range [1,m].
 *
 * Return the number of possible pairs (x, y) that satisfy the conditions mentioned in the
 * statement.
 */

/**
 * @param {number} n
 * @param {number} m
 * @return {number}
 */
var flowerGame = function(n, m) {
```

```
  const evenN = Math.floor(n / 2);
  const evenM = Math.floor(m / 2);

  return evenN * (m - evenM) + (n - evenN) * evenM;
};
```

3023 Find Pattern In Infinite Stream I

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3023. Find Pattern in Infinite Stream I
 * https://leetcode.com/problems/find-pattern-in-infinite-stream-i/
 * Difficulty: Medium
 *
 * You are given a binary array pattern and an object stream of class InfiniteStream
 * representing a 0-indexed infinite stream of bits.
 *
 * The class InfiniteStream contains the following function:
 * - int next(): Reads a single bit (which is either 0 or 1) from the stream and returns it.
 *
 * Return the first starting index where the pattern matches the bits read from the stream.
 * For example, if the pattern is [1, 0], the first match is the highlighted part in the
 * stream [0, 1, 0, 1, ...].
 */

/**
 * Definition for an infinite stream.
 * class InfiniteStream {
 *     @param {number[]} bits
 *     constructor(bits);
 *
 *     @return {number}
 *     next();
 * }
 */
/**
 * @param {InfiniteStream} stream
 * @param {number[]} pattern
 * @return {number}
 */
var findPattern = function(stream, pattern) {
  const patternLength = pattern.length;
  const buffer = [];
  let currentIndex = 0;

  while (buffer.length < patternLength) {
    buffer.push(stream.next());
  }

  while (true) {
    if (isPatternMatch(buffer, pattern)) {
      return currentIndex;
    }

    buffer.shift();
    buffer.push(stream.next());
    currentIndex++;
  }

  function isPatternMatch(buffer, pattern) {
    for (let i = 0; i < pattern.length; i++) {
      if (buffer[i] !== pattern[i]) {
        return false;
      }
    }
    return true;
  }
};
```

3024 Type Of Triangle

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3024. Type of Triangle
 * https://leetcode.com/problems/type-of-triangle/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums of size 3 which can form the sides of a
triangle.
 * - A triangle is called equilateral if it has all sides of equal length.
 * - A triangle is called isosceles if it has exactly two sides of equal length.
 * - A triangle is called scalene if all its sides are of different lengths.
 *
 * Return a string representing the type of triangle that can be formed or "none" if it cannot
 * form a triangle.
 */

/**
 * @param {number[]} nums
 * @return {string}
 */
var triangleType = function(nums) {
  const [sideA, sideB, sideC] = nums.sort((a, b) => a - b);

  if (sideA + sideB <= sideC) return 'none';
  if (sideA === sideB && sideB === sideC) return 'equilateral';
  if (sideA === sideB || sideB === sideC) return 'isosceles';

  return 'scalene';
};
```

3025 Find The Number Of Ways To Place People I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3025. Find the Number of Ways to Place People I
 * https://leetcode.com/problems/find-the-number-of-ways-to-place-people-i/
 * Difficulty: Medium
 *
 * You are given a 2D array points of size n x 2 representing integer coordinates of some
points
 * on a 2D plane, where points[i] = [xi, yi].
 *
 * Count the number of pairs of points (A, B), where
 * - A is on the upper left side of B, and
 * - there are no other points in the rectangle (or line) they make (including the border).
 *
 * Return the count.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var numberOfPairs = function(points) {
  points.sort((a, b) => a[0] === b[0] ? b[1] - a[1] : a[0] - b[0]);
  let result = 0;

  for (let i = 0; i < points.length; i++) {
    for (let j = i + 1; j < points.length; j++) {
      if (points[i][1] >= points[j][1]) {
        let isValid = true;
```

```
        for (let k = 0; k < points.length; k++) {
          if (k !== i && k !== j) {
            if (points[k][0] >= points[i][0] && points[k][0] <= points[j][0]
                && points[k][1] <= points[i][1] && points[k][1] >= points[j][1]) {
              isValid = false;
              break;
            }
          }
        }
        if (isValid) result++;
      }
    }
  }

  return result;
};
```

3027 Find The Number Of Ways To Place People Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3027. Find the Number of Ways to Place People II
 * https://leetcode.com/problems/find-the-number-of-ways-to-place-people-ii/
 * Difficulty: Hard
 *
 * You are given a 2D array points of size n x 2 representing integer coordinates of some
points
 * on a 2D-plane, where points[i] = [xi, yi].
 *
 * We define the right direction as positive x-axis (increasing x-coordinate) and the left
 * direction as negative x-axis (decreasing x-coordinate). Similarly, we define the up
direction
 * as positive y-axis (increasing y-coordinate) and the down direction as negative y-axis
 * (decreasing y-coordinate)
 *
 * You have to place n people, including Alice and Bob, at these points such that there is
exactly
 * one person at every point. Alice wants to be alone with Bob, so Alice will build a
rectangular
 * fence with Alice's position as the upper left corner and Bob's position as the lower right
 * corner of the fence (Note that the fence might not enclose any area, i.e. it can be a
line).
 * If any person other than Alice and Bob is either inside the fence or on the fence, Alice
will
 * be sad.
 *
 * Return the number of pairs of points where you can place Alice and Bob, such that Alice
does not
 * become sad on building the fence.
 *
 * Note that Alice can only build a fence with Alice's position as the upper left corner, and
Bob's
 * position as the lower right corner. For example, Alice cannot build either of the fences in
 the
 * picture below with four corners (1, 1), (1, 3), (3, 1), and (3, 3), because:
 * - With Alice at (3, 3) and Bob at (1, 1), Alice's position is not the upper left corner and
 Bob's
 *   position is not the lower right corner of the fence.
 * - With Alice at (1, 3) and Bob at (1, 1), Bob's position is not the lower right corner of
the
 *   fence.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var numberOfPairs = function(points) {
  points.sort((a, b) => a[0] === b[0] ? b[1] - a[1] : a[0] - b[0]);
```

```
  let result = 0;

  for (let i = 0; i < points.length; i++) {
    const [, maxY] = points[i];
    let minY = -Infinity;

    for (let j = i + 1; j < points.length; j++) {
      const [, y] = points[j];
      if (y <= maxY && y > minY) {
        let isValid = true;
        for (let k = 0; k < points.length; k++) {
          if (k !== i && k !== j) {
            const [x, ky] = points[k];
            if (x >= points[i][0] && x <= points[j][0]
                && ky <= points[i][1] && ky >= points[j][1]) {
              isValid = false;
              break;
            }
          }
        }
        if (isValid) {
          result++;
          minY = y;
        }
      }
    }
  }

  return result;
};
```

3028 Ant On The Boundary

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3028. Ant on the Boundary
 * https://leetcode.com/problems/ant-on-the-boundary/
 * Difficulty: Easy
 *
 * An ant is on a boundary. It sometimes goes left and sometimes right.
 *
 * You are given an array of non-zero integers nums. The ant starts reading nums from the
 * first element of it to its end. At each step, it moves according to the value of the
 * current element:
 * - If nums[i] < 0, it moves left by -nums[i] units.
 * - If nums[i] > 0, it moves right by nums[i] units.
 *
 * Return the number of times the ant returns to the boundary.
 *
 * Notes:
 * - There is an infinite space on both sides of the boundary.
 * - We check whether the ant is on the boundary only after it has moved |nums[i]| units.
 *   In other words, if the ant crosses the boundary during its movement, it does not count.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var returnToBoundaryCount = function(nums) {
  let position = 0;
  let result = 0;

  for (const step of nums) {
    position += step;
    if (position === 0) {
      result++;
    }
  }
```

```
  return result;
};
```

3029 Minimum Time To Revert Word To Initial State I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3029. Minimum Time to Revert Word to Initial State I
 * https://leetcode.com/problems/minimum-time-to-revert-word-to-initial-state-i/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string word and an integer k.
 *
 * At every second, you must perform the following operations:
 * - Remove the first k characters of word.
 * - Add any k characters to the end of word.
 *
 * Note that you do not necessarily need to add the same characters that you removed.
 * However, you must perform both operations at every second.
 *
 * Return the minimum time greater than zero required for word to revert to its initial state.
 */

/**
 * @param {string} word
 * @param {number} k
 * @return {number}
 */
var minimumTimeToInitialState = function(word, k) {
  const n = word.length;

  for (let i = 1; i * k <= n; i++) {
    if (word.slice(i * k) === word.slice(0, n - i * k)) {
      return i;
    }
  }

  return Math.ceil(n / k);
};
```

3030 Find The Grid Of Region Average

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3030. Find the Grid of Region Average
 * https://leetcode.com/problems/find-the-grid-of-region-average/
 * Difficulty: Medium
 *
 * You are given m x n grid image which represents a grayscale image, where image[i][j]
 * represents a pixel with intensity in the range [0..255]. You are also given a non-negative
 * integer threshold.
 *
 * Two pixels are adjacent if they share an edge.
 *
 * A region is a 3 x 3 subgrid where the absolute difference in intensity between any two
 * adjacent pixels is less than or equal to threshold.
 *
 * All pixels in a region belong to that region, note that a pixel can belong to multiple
regions.
 *
 * You need to calculate a m x n grid result, where result[i][j] is the average intensity of
the
```

```
 * regions to which image[i][j] belongs, rounded down to the nearest integer. If image[i][j]
 * belongs to multiple regions, result[i][j] is the average of the rounded-down average
intensities
 * of these regions, rounded down to the nearest integer. If image[i][j] does not belong to
any
 * region, result[i][j] is equal to image[i][j].
 *
 * Return the grid result.
 */

/**
 * @param {number[][]} image
 * @param {number} threshold
 * @return {number[][]}
 */
var resultGrid = function(image, threshold) {
  const rows = image.length;
  const cols = image[0].length;
  const result = Array.from({ length: rows }, () => new Array(cols).fill(0));
  const regionCount = Array.from({ length: rows }, () => new Array(cols).fill(0));

  for (let i = 0; i <= rows - 3; i++) {
    for (let j = 0; j <= cols - 3; j++) {
      if (isValidRegion(i, j)) {
        const regionAvg = calculateRegionAverage(i, j);

        for (let r = i; r < i + 3; r++) {
          for (let c = j; c < j + 3; c++) {
            result[r][c] += regionAvg;
            regionCount[r][c]++;
          }
        }
      }
    }
  }

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (regionCount[i][j] === 0) {
        result[i][j] = image[i][j];
      } else {
        result[i][j] = Math.floor(result[i][j] / regionCount[i][j]);
      }
    }
  }

  return result;

  function calculateRegionAverage(startRow, startCol) {
    let sum = 0;
    for (let i = startRow; i < startRow + 3; i++) {
      for (let j = startCol; j < startCol + 3; j++) {
        sum += image[i][j];
      }
    }
    return Math.floor(sum / 9);
  }

  function isValidRegion(startRow, startCol) {
    const directions = [[-1, 0], [1, 0], [0, -1], [0, 1]];

    for (let i = startRow; i < startRow + 3; i++) {
      for (let j = startCol; j < startCol + 3; j++) {
        for (const [dr, dc] of directions) {
          const newRow = i + dr;
          const newCol = j + dc;

          if (newRow >= startRow && newRow < startRow + 3
              && newCol >= startCol && newCol < startCol + 3) {
            if (Math.abs(image[i][j] - image[newRow][newCol]) > threshold) {
              return false;
            }
          }
        }
      }
```

```
    }
    return true;
  }
};
```

3032 Count Numbers With Unique Digits Ii

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3032. Count Numbers With Unique Digits II
 * https://leetcode.com/problems/count-numbers-with-unique-digits-ii/
 * Difficulty: Easy
 *
 * Given two positive integers a and b, return the count of numbers having unique digits
 * in the range [a, b] (inclusive).
 */

/**
 * @param {number} a
 * @param {number} b
 * @return {number}
 */
var numberCount = function(a, b) {
  let result = 0;

  for (let number = a; number <= b; number++) {
    if (hasUniqueDigits(number)) {
      result++;
    }
  }

  return result;

  function hasUniqueDigits(number) {
    const set = new Set();

    while (number > 0) {
      const digit = number % 10;
      if (set.has(digit)) {
        return false;
      }
      set.add(digit);
      number = Math.floor(number / 10);
    }

    return true;
  }
};
```

3033 Modify The Matrix

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3033. Modify the Matrix
 * https://leetcode.com/problems/modify-the-matrix/
 * Difficulty: Easy
 *
 * Given a 0-indexed m x n integer matrix matrix, create a new 0-indexed matrix called answer.
 * Make answer equal to matrix, then replace each element with the value -1 with the maximum
 * element in its respective column.
 *
 * Return the matrix answer.
```

```
 */

/**
 * @param {number[][]} matrix
 * @return {number[][]}
 */
var modifiedMatrix = function(matrix) {
  const m = matrix.length;
  const n = matrix[0].length;
  const result = matrix.map(row => [...row]);
  const colMaxes = new Array(n).fill(-Infinity);

  for (let j = 0; j < n; j++) {
    for (let i = 0; i < m; i++) {
      colMaxes[j] = Math.max(colMaxes[j], matrix[i][j]);
    }
  }

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      if (result[i][j] === -1) {
        result[i][j] = colMaxes[j];
      }
    }
  }

  return result;
};
```

3034 Number Of Subarrays That Match A Pattern I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3034. Number of Subarrays That Match a Pattern I
 * https://leetcode.com/problems/number-of-subarrays-that-match-a-pattern-i/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums of size n, and a 0-indexed integer array
pattern
 * of size m consisting of integers -1, 0, and 1.
 *
 * A subarray nums[i..j] of size m + 1 is said to match the pattern if the following
conditions
 * hold for each element pattern[k]:
 * - nums[i + k + 1] > nums[i + k] if pattern[k] == 1.
 * - nums[i + k + 1] == nums[i + k] if pattern[k] == 0.
 * - nums[i + k + 1] < nums[i + k] if pattern[k] == -1.
 *
 * Return the count of subarrays in nums that match the pattern.
 */

/**
 * @param {number[]} nums
 * @param {number[]} pattern
 * @return {number}
 */
var countMatchingSubarrays = function(nums, pattern) {
  let result = 0;
  const m = pattern.length;

  for (let i = 0; i <= nums.length - m - 1; i++) {
    let isMatch = true;
    for (let j = 0; j < m; j++) {
      const diff = nums[i + j + 1] - nums[i + j];
      if ((pattern[j] === 1 && diff <= 0) || (pattern[j] === 0 && diff !== 0)
          || (pattern[j] === -1 && diff >= 0)) {
        isMatch = false;
        break;
      }
```

```
    }
    if (isMatch) result++;
  }

  return result;
};
```

3035 Maximum Palindromes After Operations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3035. Maximum Palindromes After Operations
 * https://leetcode.com/problems/maximum-palindromes-after-operations/
 * Difficulty: Medium
 *
 * You are given a 0-indexed string array words having length n and containing 0-indexed
strings.
 *
 * You are allowed to perform the following operation any number of times (including zero):
 * - Choose integers i, j, x, and y such that 0 <= i, j < n, 0 <= x < words[i].length,
 *   0 <= y < words[j].length, and swap the characters words[i][x] and words[j][y].
 *
 * Return an integer denoting the maximum number of palindromes words can contain, after
performing
 * some operations.
 *
 * Note: i and j may be equal during an operation.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var maxPalindromesAfterOperations = function(words) {
  const charCount = new Array(26).fill(0);
  let pairCount = 0;
  let result = 0;

  for (const word of words) {
    for (const char of word) {
      charCount[char.charCodeAt(0) - 97]++;
    }
  }

  for (const count of charCount) {
    pairCount += Math.floor(count / 2);
  }

  const lengths = words.map(word => word.length).sort((a, b) => a - b);
  for (const length of lengths) {
    if (pairCount >= Math.floor(length / 2)) {
      pairCount -= Math.floor(length / 2);
      result++;
    }
  }

  return result;
};
```

3037 Find Pattern In Infinite Stream Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 3037. Find Pattern in Infinite Stream II
 * https://leetcode.com/problems/find-pattern-in-infinite-stream-ii/
 * Difficulty: Hard
 *
 * You are given a binary array pattern and an object stream of class InfiniteStream
 * representing a 0-indexed infinite stream of bits.
 *
 * The class InfiniteStream contains the following function:
 * - int next(): Reads a single bit (which is either 0 or 1) from the stream and returns it.
 *
 * Return the first starting index where the pattern matches the bits read from the stream.
 * For example, if the pattern is [1, 0], the first match is the highlighted part in the
 * stream [0, 1, 0, 1, ...].
 */

/**
 * Definition for an infinite stream.
 * class InfiniteStream {
 *     @param {number[]} bits
 *     constructor(bits);
 *
 *     @return {number}
 *     next();
 * }
 */
/**
 * @param {InfiniteStream} stream
 * @param {number[]} pattern
 * @return {number}
 */
var findPattern = function(stream, pattern) {
  const patternLength = pattern.length;
  const failureFunction = buildFailureFunction(pattern);
  let streamIndex = 0;
  let patternIndex = 0;

  while (true) {
    const streamBit = stream.next();

    while (patternIndex > 0 && pattern[patternIndex] !== streamBit) {
      patternIndex = failureFunction[patternIndex - 1];
    }

    if (pattern[patternIndex] === streamBit) {
      patternIndex++;
    }

    if (patternIndex === patternLength) {
      return streamIndex - patternLength + 1;
    }

    streamIndex++;
  }

  function buildFailureFunction(pattern) {
    const failure = new Array(pattern.length).fill(0);
    let prefixLength = 0;

    for (let suffixEnd = 1; suffixEnd < pattern.length; suffixEnd++) {
      while (prefixLength > 0 && pattern[prefixLength] !== pattern[suffixEnd]) {
        prefixLength = failure[prefixLength - 1];
      }

      if (pattern[prefixLength] === pattern[suffixEnd]) {
        prefixLength++;
      }

      failure[suffixEnd] = prefixLength;
    }

    return failure;
  }
};
```

3038 Maximum Number Of Operations With The Same Score I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3038. Maximum Number of Operations With the Same Score I
 * https://leetcode.com/problems/maximum-number-of-operations-with-the-same-score-i/
 * Difficulty: Easy
 *
 * You are given an array of integers nums. Consider the following operation:
 * - Delete the first two elements nums and define the score of the operation as the sum of
 *   these two elements.
 *
 * You can perform this operation until nums contains fewer than two elements. Additionally,
 * the same score must be achieved in all operations.
 *
 * Return the maximum number of operations you can perform.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxOperations = function(nums) {
  let result = 0;
  const targetScore = nums[0] + nums[1];

  for (let i = 0; i < nums.length - 1; i += 2) {
    if (nums[i] + nums[i + 1] === targetScore) {
      result++;
    } else {
      break;
    }
  }

  return result;
};
```

3039 Apply Operations To Make String Empty

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3039. Apply Operations to Make String Empty
 * https://leetcode.com/problems/apply-operations-to-make-string-empty/
 * Difficulty: Medium
 *
 * You are given a string s.
 *
 * Consider performing the following operation until s becomes empty:
 * - For every alphabet character from 'a' to 'z', remove the first occurrence of that
 *   character in s (if it exists).
 *
 * For example, let initially s = "aabcbbca". We do the following operations:
 * - Remove the underlined characters s = "aabcbbca". The resulting string is s = "abbca".
 * - Remove the underlined characters s = "abbca". The resulting string is s = "ba".
 * - Remove the underlined characters s = "ba". The resulting string is s = "".
 *
 * Return the value of the string s right before applying the last operation. In the
 * example above, answer is "ba".
 */

/**
 * @param {string} s
 * @return {string}
```

```
 */
var lastNonEmptyString = function(s) {
  const charCount = new Array(26).fill(0);
  let maxFrequency = 0;

  for (const char of s) {
    const index = char.charCodeAt(0) - 97;
    charCount[index]++;
    maxFrequency = Math.max(maxFrequency, charCount[index]);
  }

  let result = '';
  for (let i = s.length - 1; i >= 0; i--) {
    const index = s.charCodeAt(i) - 97;
    if (charCount[index] === maxFrequency) {
      result = s[i] + result;
      charCount[index]--;
    }
  }

  return result;
};
```

3042 Count Prefix And Suffix Pairs I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3042. Count Prefix and Suffix Pairs I
 * https://leetcode.com/problems/count-prefix-and-suffix-pairs-i/
 * Difficulty: Easy
 *
 * You are given a 0-indexed string array words.
 *
 * Let's define a boolean function isPrefixAndSuffix that takes two strings, str1 and str2:
 * - isPrefixAndSuffix(str1, str2) returns true if str1 is both a prefix and a suffix of
 *   str2, and false otherwise.
 *
 * For example, isPrefixAndSuffix("aba", "ababa") is true because "aba" is a prefix of
 * "ababa" and also a suffix, but isPrefixAndSuffix("abc", "abcd") is false.
 *
 * Return an integer denoting the number of index pairs (i, j) such that i < j, and
 * isPrefixAndSuffix(words[i], words[j]) is true.
 */

/**
 * @param {string[]} words
 * @return {number}
 */
var countPrefixSuffixPairs = function(words) {
  let count = 0;

  for (let i = 0; i < words.length; i++) {
    for (let j = i + 1; j < words.length; j++) {
      if (words[j].startsWith(words[i]) && words[j].endsWith(words[i])) {
        count++;
      }
    }
  }

  return count;
};
```

3043 Find The Length Of The Longest Common Prefix

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3043. Find the Length of the Longest Common Prefix
 * https://leetcode.com/problems/find-the-length-of-the-longest-common-prefix/
 * Difficulty: Medium
 *
 * You are given two arrays with positive integers arr1 and arr2.
 *
 * A prefix of a positive integer is an integer formed by one or more of its digits, starting
 * from its leftmost digit. For example, 123 is a prefix of the integer 12345, while 234 is
not.
 *
 * A common prefix of two integers a and b is an integer c, such that c is a prefix of both a
 * and b. For example, 5655359 and 56554 have common prefixes 565 and 5655 while 1223 and
43456
 * do not have a common prefix.
 *
 * You need to find the length of the longest common prefix between all pairs of integers (x,
y)
 * such that x belongs to arr1 and y belongs to arr2.
 *
 * Return the length of the longest common prefix among all pairs. If no common prefix exists
 * among them, return 0.
 */

/**
 * @param {number[]} arr1
 * @param {number[]} arr2
 * @return {number}
 */
var longestCommonPrefix = function(arr1, arr2) {
  const prefixSet = new Set();
  let result = 0;

  for (const num of arr1) {
    let prefix = num;
    while (prefix > 0) {
      prefixSet.add(prefix);
      prefix = Math.floor(prefix / 10);
    }
  }

  for (const num of arr2) {
    let prefix = num;
    while (prefix > 0) {
      if (prefixSet.has(prefix)) {
        result = Math.max(result, String(prefix).length);
      }
      prefix = Math.floor(prefix / 10);
    }
  }

  return result;
};
```

3044 Most Frequent Prime

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3044. Most Frequent Prime
 * https://leetcode.com/problems/most-frequent-prime/
 * Difficulty: Medium
 *
 * You are given a m x n 0-indexed 2D matrix mat. From every cell, you can create numbers
 * in the following way:
```

```
 * - There could be at most 8 paths from the cells namely: east, south-east, south,
south-west,
 *   west, north-west, north, and north-east.
 * - Select a path from them and append digits in this path to the number being formed by
 *   traveling in this direction.
 * - Note that numbers are generated at every step, for example, if the digits along the path
 *   are 1, 9, 1, then there will be three numbers generated along the way: 1, 19, 191.
 *
 * Return the most frequent prime number greater than 10 out of all the numbers created by
 * traversing the matrix or -1 if no such prime number exists. If there are multiple prime
 * numbers with the highest frequency, then return the largest among them.
 *
 * Note: It is invalid to change the direction during the move.
 */

/**
 * @param {number[][]} mat
 * @return {number}
 */
var mostFrequentPrime = function(mat) {
  const m = mat.length;
  const n = mat[0].length;
  const frequency = new Map();
  const directions = [[0, 1], [1, 1], [1, 0], [1, -1], [0, -1], [-1, -1], [-1, 0], [-1, 1]];

  const isPrime = num => {
    if (num <= 1) return false;
    for (let i = 2; i * i <= num; i++) {
      if (num % i === 0) return false;
    }
    return true;
  };

  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      for (const [dx, dy] of directions) {
        let x = i;
        let y = j;
        let num = 0;
        while (x >= 0 && x < m && y >= 0 && y < n) {
          num = num * 10 + mat[x][y];
          if (num > 10 && isPrime(num)) {
            frequency.set(num, (frequency.get(num) || 0) + 1);
          }
          x += dx;
          y += dy;
        }
      }
    }
  }

  let maxFreq = 0;
  let result = -1;
  for (const [num, freq] of frequency) {
    if (freq > maxFreq || (freq === maxFreq && num > result)) {
      maxFreq = freq;
      result = num;
    }
  }

  return result;
};
```

3046 Split The Array

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3046. Split the Array
 * https://leetcode.com/problems/split-the-array/
```

```
 * Difficulty: Easy
 *
 * You are given an integer array nums of even length. You have to split the array into two
 * parts nums1 and nums2 such that:
 * - nums1.length == nums2.length == nums.length / 2.
 * - nums1 should contain distinct elements.
 * - nums2 should also contain distinct elements.
 *
 * Return true if it is possible to split the array, and false otherwise.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isPossibleToSplit = function(nums) {
  const map = new Map();

  for (const num of nums) {
    map.set(num, (map.get(num) || 0) + 1);
    if (map.get(num) > 2) return false;
  }

  return true;
};
```

3047 Find The Largest Area Of Square Inside Two Rectangles

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3047. Find the Largest Area of Square Inside Two Rectangles
 * https://leetcode.com/problems/find-the-largest-area-of-square-inside-two-rectangles/
 * Difficulty: Medium
 *
 * There exist n rectangles in a 2D plane with edges parallel to the x and y axis. You
 * are given two 2D integer arrays bottomLeft and topRight where bottomLeft[i] = [a_i, b_i]
 * and topRight[i] = [c_i, d_i] represent the bottom-left and top-right coordinates of the
 * ith rectangle, respectively.
 *
 * You need to find the maximum area of a square that can fit inside the intersecting
 * region of at least two rectangles. Return 0 if such a square does not exist.
 */

/**
 * @param {number[][]} bottomLeft
 * @param {number[][]} topRight
 * @return {number}
 */
var largestSquareArea = function(bottomLeft, topRight) {
  const n = bottomLeft.length;
  let maxSide = 0;

  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      const xLeft = Math.max(bottomLeft[i][0], bottomLeft[j][0]);
      const yBottom = Math.max(bottomLeft[i][1], bottomLeft[j][1]);
      const xRight = Math.min(topRight[i][0], topRight[j][0]);
      const yTop = Math.min(topRight[i][1], topRight[j][1]);

      if (xLeft < xRight && yBottom < yTop) {
        const side = Math.min(xRight - xLeft, yTop - yBottom);
        maxSide = Math.max(maxSide, side);
      }
    }
  }

  return maxSide * maxSide;
};
```

3062 Winner Of The Linked List Game

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3062. Winner of the Linked List Game
 * https://leetcode.com/problems/winner-of-the-linked-list-game/
 * Difficulty: Easy
 *
 * You are given the head of a linked list of even length containing integers.
 *
 * Each odd-indexed node contains an odd integer and each even-indexed node contains an
 * even integer.
 *
 * We call each even-indexed node and its next node a pair, e.g., the nodes with indices
 * 0 and 1 are a pair, the nodes with indices 2 and 3 are a pair, and so on.
 *
 * For every pair, we compare the values of the nodes in the pair:
 * - If the odd-indexed node is higher, the "Odd" team gets a point.
 * - If the even-indexed node is higher, the "Even" team gets a point.
 *
 * Return the name of the team with the higher points, if the points are equal, return "Tie".
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {string}
 */
var gameResult = function(head) {
  let evenScore = 0;
  let oddScore = 0;
  let currentNode = head;

  while (currentNode && currentNode.next) {
    const evenValue = currentNode.val;
    const oddValue = currentNode.next.val;

    if (evenValue > oddValue) {
      evenScore++;
    } else if (oddValue > evenValue) {
      oddScore++;
    }

    currentNode = currentNode.next.next;
  }

  if (evenScore > oddScore) {
    return 'Even';
  } else if (oddScore > evenScore) {
    return 'Odd';
  } else {
    return 'Tie';
  }
};
```

3063 Linked List Frequency

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop

invariant.

SOURCE CODE

```
/**
 * 3063. Linked List Frequency
 * https://leetcode.com/problems/linked-list-frequency/
 * Difficulty: Easy
 *
 * Given the head of a linked list containing k distinct elements, return the head to a linked
 * list of length k containing the frequency of each distinct element in the given linked list
 * in any order.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var frequenciesOfElements = function(head) {
  const map = new Map();
  let currentNode = head;

  while (currentNode) {
    map.set(currentNode.val, (map.get(currentNode.val) || 0) + 1);
    currentNode = currentNode.next;
  }

  const frequencies = Array.from(map.values());

  if (frequencies.length === 0) return null;

  const resultHead = new ListNode(frequencies[0]);
  let resultCurrent = resultHead;

  for (let i = 1; i < frequencies.length; i++) {
    resultCurrent.next = new ListNode(frequencies[i]);
    resultCurrent = resultCurrent.next;
  }

  return resultHead;
};
```

3064 Guess The Number Using Bitwise Questions I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3064. Guess the Number Using Bitwise Questions I
 * https://leetcode.com/problems/guess-the-number-using-bitwise-questions-i/
 * Difficulty: Medium
 *
 * There is a number n that you have to find.
 *
 * There is also a pre-defined API int commonSetBits(int num), which returns the number of
bits
 * where both n and num are 1 in that position of their binary representation. In other words,
 * it returns the number of set bits in n & num, where & is the bitwise AND operator.
 *
 * Return the number n.
 */

/**
 * Definition of commonSetBits API.
 * @param {number} num
```

```
 * @return {integer}
 * var commonSetBits = function(num) {}
 */

/**
 * @return {number}
 */
var findNumber = function() {
  let result = 0;

  for (let bitPosition = 0; bitPosition < 30; bitPosition++) {
    const powerOfTwo = 1 << bitPosition;
    if (commonSetBits(powerOfTwo) === 1) {
      result |= powerOfTwo;
    }
  }

  return result;
};
```

3065 Minimum Operations To Exceed Threshold Value I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3065. Minimum Operations to Exceed Threshold Value I
 * https://leetcode.com/problems/minimum-operations-to-exceed-threshold-value-i/
 * Difficulty: Easy
 *
 * You are given a 0-indexed integer array nums, and an integer k.
 *
 * In one operation, you can remove one occurrence of the smallest element of nums.
 *
 * Return the minimum number of operations needed so that all elements of the array are
greater
 * than or equal to k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minOperations = function(nums, k) {
  let operations = 0;

  for (const num of nums) {
    if (num < k) operations++;
  }

  return operations;
};
```

3066 Minimum Operations To Exceed Threshold Value Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3066. Minimum Operations to Exceed Threshold Value II
 * https://leetcode.com/problems/minimum-operations-to-exceed-threshold-value-ii/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer array nums, and an integer k.
 *
 * You are allowed to perform some operations on nums, where in a single operation, you can:
```

```
 * - Select the two smallest integers x and y from nums.
 * - Remove x and y from nums.
 * - Insert (min(x, y) * 2 + max(x, y)) at any position in the array.
 *
 * Note that you can only apply the described operation if nums contains at least two
elements.
 *
 * Return the minimum number of operations needed so that all elements of the array are
greater
 * than or equal to k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minOperations = function(nums, k) {
  const queue = new MinPriorityQueue();
  let operations = 0;

  nums.forEach(n => queue.enqueue(n));
  while (queue.size() >= 2 && queue.front().element < k) {
    queue.enqueue(queue.dequeue().element * 2 + queue.dequeue().element);
    operations++;
  }

  return operations;
};
```

3067 Count Pairs Of Connectable Servers In A Weighted Tree Network

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 3067. Count Pairs of Connectable Servers in a Weighted Tree Network
 *
https://leetcode.com/problems/count-pairs-of-connectable-servers-in-a-weighted-tree-network/
 * Difficulty: Medium
 *
 * You are given an unrooted weighted tree with n vertices representing servers numbered from
0
 * to n - 1, an array edges where edges[i] = [ai, bi, weighti] represents a bidirectional edge
 * between vertices ai and bi of weight weighti. You are also given an integer signalSpeed.
 *
 * Two servers a and b are connectable through a server c if:
 * - a < b, a != c and b != c.
 * - The distance from c to a is divisible by signalSpeed.
 * - The distance from c to b is divisible by signalSpeed.
 * - The path from c to b and the path from c to a do not share any edges.
 *
 * Return an integer array count of length n where count[i] is the number of server pairs that
 are
 * connectable through the server i.
 */

/**
 * @param {number[][]} edges
 * @param {number} signalSpeed
 * @return {number[]}
 */
var countPairsOfConnectableServers = function(edges, signalSpeed) {
  const n = edges.length + 1;
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v, w] of edges) {
    graph[u].push([v, w]);
    graph[v].push([u, w]);
  }

  const result = new Array(n).fill(0);
```

```
  for (let i = 0; i < n; i++) {
    result[i] = countValidPairs(i);
  }
  return result;

  function countValidPairs(root) {
    function dfs(node, parent, distance) {
      let count = distance % signalSpeed === 0 ? 1 : 0;
      for (const [next, weight] of graph[node]) {
        if (next !== parent) {
          count += dfs(next, node, distance + weight);
        }
      }
      return count;
    }

    let totalPairs = 0;
    const counts = [];
    for (const [child, weight] of graph[root]) {
      const count = dfs(child, root, weight);
      counts.push(count);
    }

    let sum = 0;
    for (const count of counts) {
      totalPairs += sum * count;
      sum += count;
    }
    return totalPairs;
  }
};
```

3068 Find The Maximum Sum Of Node Values

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3068. Find the Maximum Sum of Node Values
 * https://leetcode.com/problems/find-the-maximum-sum-of-node-values/
 * Difficulty: Hard
 *
 * There exists an undirected tree with n nodes numbered 0 to n - 1. You are given a 0-indexed
 * 2D integer array edges of length n - 1, where edges[i] = [ui, vi] indicates that there is
an
 * edge between nodes ui and vi in the tree. You are also given a positive integer k, and a
 * 0-indexed array of non-negative integers nums of length n, where nums[i] represents the
 * value of the node numbered i.
 *
 * Alice wants the sum of values of tree nodes to be maximum, for which Alice can perform the
 * following operation any number of times (including zero) on the tree:
 * - Choose any edge [u, v] connecting the nodes u and v, and update their values as follows:
 *   - nums[u] = nums[u] XOR k
 *   - nums[v] = nums[v] XOR k
 *
 * Return the maximum possible sum of the values Alice can achieve by performing the operation
 * any number of times.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @param {number[][]} edges
 * @return {number}
 */
var maximumValueSum = function(nums, k, edges) {
  let totalSum = 0;
  let minChange = Infinity;
  let changeCount = 0;

  for (const num of nums) {
```

```
    const xored = num ^ k;
    const change = xored - num;

    if (change > 0) {
      totalSum += xored;
      changeCount++;
    } else {
      totalSum += num;
    }

    minChange = Math.min(minChange, Math.abs(change));
  }

  if (changeCount % 2 === 0) {
    return totalSum;
  }

  return totalSum - minChange;
};
```

3069 Distribute Elements Into Two Arrays I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3069. Distribute Elements Into Two Arrays I
 * https://leetcode.com/problems/distribute-elements-into-two-arrays-i/
 * Difficulty: Easy
 *
 * You are given a 1-indexed array of distinct integers nums of length n.
 *
 * You need to distribute all the elements of nums between two arrays arr1 and arr2 using n
 * operations. In the first operation, append nums[1] to arr1. In the second operation,
 * append nums[2] to arr2. Afterwards, in the ith operation:
 * - If the last element of arr1 is greater than the last element of arr2, append nums[i] to
arr1.
 *   Otherwise, append nums[i] to arr2.
 *
 * The array result is formed by concatenating the arrays arr1 and arr2. For example, if
 * arr1 == [1,2,3] and arr2 == [4,5,6], then result = [1,2,3,4,5,6].
 *
 * Return the array result.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var resultArray = function(nums) {
  const arr1 = [nums[0]];
  const arr2 = [nums[1]];

  for (let i = 2; i < nums.length; i++) {
    if (arr1[arr1.length - 1] > arr2[arr2.length - 1]) {
      arr1.push(nums[i]);
    } else {
      arr2.push(nums[i]);
    }
  }

  return [...arr1, ...arr2];
};
```

3070 Count Submatrices With Top Left Element And Sum Less Than K

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3070. Count Submatrices with Top-Left Element and Sum Less Than k
 * https://leetcode.com/problems/count-submatrices-with-top-left-element-and-sum-less-than-k/
 * Difficulty: Medium
 *
 * You are given a 0-indexed integer matrix grid and an integer k.
 *
 * Return the number of submatrices that contain the top-left element of the grid, and have a
sum
 * less than or equal to k.
 */

/**
 * @param {number[][]} grid
 * @param {number} k
 * @return {number}
 */
var countSubmatrices = function(grid, k) {
  const rows = grid.length;
  const cols = grid[0].length;
  const prefixSum = Array.from({ length: rows + 1 }, () => new Array(cols + 1).fill(0));

  for (let i = 1; i <= rows; i++) {
    for (let j = 1; j <= cols; j++) {
      prefixSum[i][j] = grid[i-1][j-1] + prefixSum[i-1][j]
        + prefixSum[i][j-1] - prefixSum[i-1][j-1];
    }
  }

  let result = 0;
  for (let i = 1; i <= rows; i++) {
    for (let j = 1; j <= cols; j++) {
      if (prefixSum[i][j] <= k) {
        result++;
      }
    }
  }

  return result;
};
```

3071 Minimum Operations To Write The Letter Y On A Grid

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3071. Minimum Operations to Write the Letter Y on a Grid
 * https://leetcode.com/problems/minimum-operations-to-write-the-letter-y-on-a-grid/
 * Difficulty: Medium
 *
 * You are given a 0-indexed n x n grid where n is odd, and grid[r][c] is 0, 1, or 2.
 *
 * We say that a cell belongs to the Letter Y if it belongs to one of the following:
 * - The diagonal starting at the top-left cell and ending at the center cell of the grid.
 * - The diagonal starting at the top-right cell and ending at the center cell of the grid.
 * - The vertical line starting at the center cell and ending at the bottom border of the
grid.
 *
 * The Letter Y is written on the grid if and only if:
 * - All values at cells belonging to the Y are equal.
 * - All values at cells not belonging to the Y are equal.
 * - The values at cells belonging to the Y are different from the values at cells not
belonging
 *   to the Y.
 *
 * Return the minimum number of operations needed to write the letter Y on the grid given that
 * in one operation you can change the value at any cell to 0, 1, or 2.
```

```
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minimumOperationsToWriteY = function(grid) {
  const n = grid.length;
  const center = Math.floor(n / 2);
  const yCells = [];
  const nonYCells = [];

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if ((i === j && i <= center) || (i === n - 1 - j && i <= center)
          || (j === center && i >= center)) {
        yCells.push(grid[i][j]);
      } else {
        nonYCells.push(grid[i][j]);
      }
    }
  }

  const yCounts = [0, 0, 0];
  const nonYCounts = [0, 0, 0];

  for (const val of yCells) yCounts[val]++;
  for (const val of nonYCells) nonYCounts[val]++;

  let minOperations = Infinity;

  for (let yVal = 0; yVal <= 2; yVal++) {
    for (let nonYVal = 0; nonYVal <= 2; nonYVal++) {
      if (yVal !== nonYVal) {
        const yOps = yCells.length - yCounts[yVal];
        const nonYOps = nonYCells.length - nonYCounts[nonYVal];
        minOperations = Math.min(minOperations, yOps + nonYOps);
      }
    }
  }

  return minOperations;
};
```

3073 Maximum Increasing Triplet Value

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3073. Maximum Increasing Triplet Value
 * https://leetcode.com/problems/maximum-increasing-triplet-value/
 * Difficulty: Medium
 *
 * Given an array nums, return the maximum value of a triplet (i, j, k) such that i < j < k
 * and nums[i] < nums[j] < nums[k].
 *
 * The value of a triplet (i, j, k) is nums[i] - nums[j] + nums[k].
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumTripletValue = function(nums) {
  const n = nums.length;
  const maxRight = new Array(n).fill(-1);
  let currentMax = 0;
  let result = 0;
```

```
  for (let i = n - 1; i >= 0; i--) {
    if (nums[i] >= currentMax) {
      currentMax = nums[i];
    } else {
      maxRight[i] = currentMax;
    }
  }

  const sortedIndices = Array.from({ length: n }, (_, i) => i)
    .sort((a, b) => nums[a] - nums[b] || b - a);

  const stack = [];

  for (const currentIndex of sortedIndices) {
    while (stack.length > 0 && stack[stack.length - 1] > currentIndex) {
      stack.pop();
    }

    if (stack.length > 0 && maxRight[currentIndex] >= 0) {
      const leftIndex = stack[stack.length - 1];
      const tripletValue = nums[leftIndex] - nums[currentIndex] + maxRight[currentIndex];
      result = Math.max(result, tripletValue);
    }

    stack.push(currentIndex);
  }

  return result;
};
```

3074 Apple Redistribution Into Boxes

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3074. Apple Redistribution into Boxes
 * https://leetcode.com/problems/apple-redistribution-into-boxes/
 * Difficulty: Easy
 *
 * You are given an array apple of size n and an array capacity of size m.
 *
 * There are n packs where the ith pack contains apple[i] apples. There are m boxes as well,
 * and the ith box has a capacity of capacity[i] apples.
 *
 * Return the minimum number of boxes you need to select to redistribute these n packs of
 * apples into boxes.
 *
 * Note that, apples from the same pack can be distributed into different boxes.
 */

/**
 * @param {number[]} apple
 * @param {number[]} capacity
 * @return {number}
 */
var minimumBoxes = function(apple, capacity) {
  const totalApples = apple.reduce((sum, num) => sum + num, 0);
  const sortedCapacities = capacity.sort((a, b) => b - a);
  let currentCapacity = 0;
  let result = 0;

  for (const box of sortedCapacities) {
    if (currentCapacity < totalApples) {
      currentCapacity += box;
      result++;
    } else {
      break;
    }
  }
```

```
  return result;
};
```

3075 Maximize Happiness Of Selected Children

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3075. Maximize Happiness of Selected Children
 * https://leetcode.com/problems/maximize-happiness-of-selected-children/
 * Difficulty: Medium
 *
 * You are given an array happiness of length n, and a positive integer k.
 *
 * There are n children standing in a queue, where the ith child has happiness value
 * happiness[i]. You want to select k children from these n children in k turns.
 *
 * In each turn, when you select a child, the happiness value of all the children that
 * have not been selected till now decreases by 1. Note that the happiness value cannot
 * become negative and gets decremented only if it is positive.
 *
 * Return the maximum sum of the happiness values of the selected children you can
 * achieve by selecting k children.
 */

/**
 * @param {number[]} happiness
 * @param {number} k
 * @return {number}
 */
var maximumHappinessSum = function(happiness, k) {
  const sortedHappiness = happiness.sort((a, b) => b - a);
  let result = 0;

  for (let i = 0; i < k; i++) {
    const currentHappiness = sortedHappiness[i] - i;
    if (currentHappiness <= 0) break;
    result += currentHappiness;
  }

  return result;
};
```

3076 Shortest Uncommon Substring In An Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3076. Shortest Uncommon Substring in an Array
 * https://leetcode.com/problems/shortest-uncommon-substring-in-an-array/
 * Difficulty: Medium
 *
 * You are given an array arr of size n consisting of non-empty strings.
 *
 * Find a string array answer of size n such that:
 * - answer[i] is the shortest substring of arr[i] that does not occur as a substring in any
 *   other string in arr. If multiple such substrings exist, answer[i] should be the
 *   lexicographically smallest. And if no such substring exists, answer[i] should be an
 *   empty string.
 *
 * Return the array answer.
 */

/**
 * @param {string[]} arr
```

```
 * @return {string[]}
 */
var shortestSubstrings = function(arr) {
  const n = arr.length;
  const result = new Array(n).fill('');

  for (let i = 0; i < n; i++) {
    const str = arr[i];
    let minLen = Infinity;
    let minSubstr = '';

    for (let len = 1; len <= str.length; len++) {
      for (let start = 0; start <= str.length - len; start++) {
        const substr = str.slice(start, start + len);
        if (isUnique(substr, i)) {
          if (len < minLen || (len === minLen && substr < minSubstr)) {
            minLen = len;
            minSubstr = substr;
          }
        }
      }
      if (minSubstr) break;
    }

    result[i] = minSubstr;
  }

  return result;

  function isUnique(substr, strIdx) {
    for (let i = 0; i < n; i++) {
      if (i !== strIdx && arr[i].includes(substr)) {
        return false;
      }
    }
    return true;
  }
};
```

3078 Match Alphanumerical Pattern In Matrix I

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3078. Match Alphanumerical Pattern in Matrix I
 * https://leetcode.com/problems/match-alphanumerical-pattern-in-matrix-i/
 * Difficulty: Medium
 *
 * You are given a 2D integer matrix board and a 2D character matrix pattern.
 * Where 0 <= board[r][c] <= 9 and each element of pattern is either a digit or a
 * lowercase English letter.
 *
 * Your task is to find a submatrix of board that matches pattern.
 *
 * An integer matrix part matches pattern if we can replace cells containing letters in
 * pattern with some digits (each distinct letter with a unique digit) in such a way that
 * the resulting matrix becomes identical to the integer matrix part. In other words,
 * - The matrices have identical dimensions.
 * - If pattern[r][c] is a digit, then part[r][c] must be the same digit.
 * - If pattern[r][c] is a letter x:
 *   - For every pattern[i][j] == x, part[i][j] must be the same as part[r][c].
 *   - For every pattern[i][j] != x, part[i][j] must be different than part[r][c].
 *
 * Return an array of length 2 containing the row number and column number of the upper-left
 * corner of a submatrix of board which matches pattern. If there is more than one such
submatrix,
 * return the coordinates of the submatrix with the lowest row index, and in case there is
still
 * a tie, return the coordinates of the submatrix with the lowest column index. If there are
no
```

```
 * suitable answers, return [-1, -1].
 */

/**
 * @param {number[][]} board
 * @param {string[]} pattern
 * @return {number[]}
 */
var findPattern = function(board, pattern) {
  const boardRows = board.length;
  const boardCols = board[0].length;
  const patternRows = pattern.length;
  const patternCols = pattern[0].length;

  for (let startRow = 0; startRow <= boardRows - patternRows; startRow++) {
    for (let startCol = 0; startCol <= boardCols - patternCols; startCol++) {
      if (isPatternMatch(board, pattern, startRow, startCol)) {
        return [startRow, startCol];
      }
    }
  }

  return [-1, -1];

  function isPatternMatch(board, pattern, startRow, startCol) {
    const letterToDigit = new Map();
    const digitToLetter = new Map();
    const usedDigits = new Set();

    for (let patternRow = 0; patternRow < pattern.length; patternRow++) {
      for (let patternCol = 0; patternCol < pattern[0].length; patternCol++) {
        const patternChar = pattern[patternRow][patternCol];
        const boardDigit = board[startRow + patternRow][startCol + patternCol];

        if (isDigit(patternChar)) {
          if (parseInt(patternChar) !== boardDigit) {
            return false;
          }
        } else {
          if (letterToDigit.has(patternChar)) {
            if (letterToDigit.get(patternChar) !== boardDigit) {
              return false;
            }
          } else {
            if (usedDigits.has(boardDigit) || digitToLetter.has(boardDigit)) {
              return false;
            }
            letterToDigit.set(patternChar, boardDigit);
            digitToLetter.set(boardDigit, patternChar);
            usedDigits.add(boardDigit);
          }
        }
      }
    }

    return true;
  }

  function isDigit(char) {
    return char >= '0' && char <= '9';
  }
};
```

3079 Find The Sum Of Encrypted Integers

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3079. Find the Sum of Encrypted Integers
 * https://leetcode.com/problems/find-the-sum-of-encrypted-integers/
```

```
 * Difficulty: Easy
 *
 * You are given an integer array nums containing positive integers. We define a function
 * encrypt such that encrypt(x) replaces every digit in x with the largest digit in x.
 * For example, encrypt(523) = 555 and encrypt(213) = 333.
 *
 * Return the sum of encrypted elements.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var sumOfEncryptedInt = function(nums) {
  let result = 0;

  for (const num of nums) {
    let maxDigit = 0;
    let temp = num;
    let digitCount = 0;

    while (temp > 0) {
      maxDigit = Math.max(maxDigit, temp % 10);
      temp = Math.floor(temp / 10);
      digitCount++;
    }

    let encrypted = 0;
    while (digitCount > 0) {
      encrypted = encrypted * 10 + maxDigit;
      digitCount--;
    }

    result += encrypted;
  }

  return result;
};
```

3083 Existence Of A Substring In A String And Its Reverse

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3083. Existence of a Substring in a String and Its Reverse
 * https://leetcode.com/problems/existence-of-a-substring-in-a-string-and-its-reverse/
 * Difficulty: Easy
 *
 * Given a string s, find any substring of length 2 which is also present in the reverse of s.
 *
 * Return true if such a substring exists, and false otherwise.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var isSubstringPresent = function(s) {
  const reversed = s.split('').reverse().join('');

  for (let i = 0; i < s.length - 1; i++) {
    const substr = s.slice(i, i + 2);
    if (reversed.includes(substr)) {
      return true;
    }
  }

  return false;
};
```

3084 Count Substrings Starting And Ending With Given Character

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3084. Count Substrings Starting and Ending with Given Character
 * https://leetcode.com/problems/count-substrings-starting-and-ending-with-given-character/
 * Difficulty: Medium
 *
 * You are given a string s and a character c. Return the total number of substrings of s
 * that start and end with c.
 */

/**
 * @param {string} s
 * @param {character} c
 * @return {number}
 */
var countSubstrings = function(s, c) {
  let count = 0;
  for (const char of s) {
    if (char === c) count++;
  }
  return (count * (count + 1)) / 2;
};
```

3085 Minimum Deletions To Make String K Special

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3085. Minimum Deletions to Make String K-Special
 * https://leetcode.com/problems/minimum-deletions-to-make-string-k-special/
 * Difficulty: Medium
 *
 * You are given a string word and an integer k.
 *
 * We consider word to be k-special if |freq(word[i]) - freq(word[j])| <= k for all indices i
 * and j in the string.
 *
 * Here, freq(x) denotes the frequency of the character x in word, and |y| denotes the
absolute
 * value of y.
 *
 * Return the minimum number of characters you need to delete to make word k-special.
 */

/**
 * @param {string} word
 * @param {number} k
 * @return {number}
 */
var minimumDeletions = function(word, k) {
  const freq = new Array(26).fill(0);

  for (const char of word) {
    freq[char.charCodeAt(0) - 97]++;
  }

  const counts = freq.filter(x => x > 0).sort((a, b) => a - b);
  let minDeletions = Infinity;
  for (let i = 0; i < counts.length; i++) {
    let deletions = 0;
    for (let j = 0; j < i; j++) {
      deletions += counts[j];
```

```
    }
    for (let j = i; j < counts.length; j++) {
      if (counts[j] - counts[i] > k) {
        deletions += counts[j] - (counts[i] + k);
      }
    }
    minDeletions = Math.min(minDeletions, deletions);
  }

  return minDeletions;
};
```

3088 Make String Anti Palindrome

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3088. Make String Anti-palindrome
 * https://leetcode.com/problems/make-string-anti-palindrome/
 * Difficulty: Hard
 *
 * We call a string s of even length n an anti-palindrome if for each index
 * 0 <= i < n, s[i] != s[n - i - 1].
 *
 * Given a string s, your task is to make s an anti-palindrome by doing any number of
 * operations (including zero).
 *
 * In one operation, you can select two characters from s and swap them.
 *
 * Return the resulting string. If multiple strings meet the conditions, return the
 * lexicographically smallest one. If it can't be made into an anti-palindrome, return "-1".
 */

/**
 * @param {string} s
 * @return {string}
 */
var makeAntiPalindrome = function(s) {
  const n = s.length;
  const result = Array.from(s).sort();
  let leftIndex = Math.floor((n + 1) / 2);
  let rightIndex = Math.floor((n + 1) / 2);

  while (rightIndex < n && result[rightIndex] === result[leftIndex]) {
    rightIndex++;
  }
  while (result[leftIndex] === result[n - leftIndex - 1]) {
    if (rightIndex === n) {
      return '-1';
    }
    [result[leftIndex], result[rightIndex]] = [result[rightIndex], result[leftIndex]];
    leftIndex++;
    rightIndex++;
  }

  return result.join('');
};
```

3090 Maximum Length Substring With Two Occurrences

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3090. Maximum Length Substring With Two Occurrences
```

```
 * https://leetcode.com/problems/maximum-length-substring-with-two-occurrences/
 * Difficulty: Easy
 *
 * Given a string s, return the maximum length of a substring such that it contains at most
 * two occurrences of each character.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maximumLengthSubstring = function(s) {
  let result = 0;
  const map = new Map();

  for (let left = 0, right = 0; right < s.length; right++) {
    map.set(s[right], (map.get(s[right]) || 0) + 1);

    while (map.get(s[right]) > 2) {
      map.set(s[left], map.get(s[left]) - 1);
      if (map.get(s[left]) === 0) map.delete(s[left]);
      left++;
    }

    result = Math.max(result, right - left + 1);
  }

  return result;
};
```

3091 Apply Operations To Make Sum Of Array Greater Than Or Equal To K

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3091. Apply Operations to Make Sum of Array Greater Than or Equal to k
 * https://leetcode.com/problems/apply-operations-to-make-sum-of-array-greater-than-or-equal-t
o-k/
 * Difficulty: Medium
 *
 * You are given a positive integer k. Initially, you have an array nums = [1].
 *
 * You can perform any of the following operations on the array any number of times (possibly
zero):
 * - Choose any element in the array and increase its value by 1.
 * - Duplicate any element in the array and add it to the end of the array.
 *
 * Return the minimum number of operations required to make the sum of elements of the final
array
 * greater than or equal to k.
 */

/**
 * @param {number} k
 * @return {number}
 */
var minOperations = function(k) {
  if (k === 1) return 0;

  let result = Infinity;
  for (let increments = 0; increments <= Math.ceil(Math.sqrt(k)); increments++) {
    const value = 1 + increments;
    const duplicates = Math.ceil(k / value) - 1;
    if (duplicates >= 0) {
      result = Math.min(result, increments + duplicates);
    }
  }

  return result;
};
```

3094 Guess The Number Using Bitwise Questions Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3094. Guess the Number Using Bitwise Questions II
 * https://leetcode.com/problems/guess-the-number-using-bitwise-questions-ii/
 * Difficulty: Medium
 *
 * There is a number n between 0 and 230 - 1 (both inclusive) that you have to find.
 *
 * There is a pre-defined API int commonBits(int num) that helps you with your mission.
 * But here is the challenge, every time you call this function, n changes in some way.
 * But keep in mind, that you have to find the initial value of n.
 *
 * commonBits(int num) acts as follows:
 * - Calculate count which is the number of bits where both n and num have the same value
 *   in that position of their binary representation.
 * - n = n XOR num
 * - Return count.
 *
 * Return the number n.
 *
 * Note: In this world, all numbers are between 0 and 230 - 1 (both inclusive), thus for
counting
 * common bits, we see only the first 30 bits of those numbers.
 */

/**
 * Definition of commonBits API.
 * @param {number} num
 * @return {integer}
 * var commonBits = function(num) {}
 */
/**
 * @return {number}
 */
var findNumber = function() {
  let result = 0;

  for (let bitMask = 1; bitMask < 1073741824; bitMask <<= 1) {
    if (commonBits(bitMask) > commonBits(bitMask)) {
      result |= bitMask;
    }
  }

  return result;
};
```

3095 Shortest Subarray With Or At Least K I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3095. Shortest Subarray With OR at Least K I
 * https://leetcode.com/problems/shortest-subarray-with-or-at-least-k-i/
 * Difficulty: Easy
 *
 * You are given an array nums of non-negative integers and an integer k.
 *
 * An array is called special if the bitwise OR of all of its elements is at least k.
 *
 * Return the length of the shortest special non-empty subarray of nums, or return -1 if
 * no special subarray exists.
 */
```

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minimumSubarrayLength = function(nums, k) {
  if (k === 0) return 1;

  let minLength = Infinity;
  for (let start = 0; start < nums.length; start++) {
    let orResult = 0;
    for (let end = start; end < nums.length; end++) {
      orResult |= nums[end];
      if (orResult >= k) {
        minLength = Math.min(minLength, end - start + 1);
        break;
      }
    }
  }

  return minLength === Infinity ? -1 : minLength;
};
```

3097 Shortest Subarray With Or At Least K Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3097. Shortest Subarray With OR at Least K II
 * https://leetcode.com/problems/shortest-subarray-with-or-at-least-k-ii/
 * Difficulty: Medium
 *
 * You are given an array nums of non-negative integers and an integer k.
 *
 * An array is called special if the bitwise OR of all of its elements is at least k.
 *
 * Return the length of the shortest special non-empty subarray of nums, or return -1 if no
 * special subarray exists.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minimumSubarrayLength = function(nums, k) {
  if (k === 0) return 1;

  const bitCounts = new Array(32).fill(0);
  let minLength = Infinity;
  let orValue = 0;
  let left = 0;

  for (let right = 0; right < nums.length; right++) {
    const num = nums[right];
    for (let bit = 0; bit < 32; bit++) {
      if (num & (1 << bit)) bitCounts[bit]++;
      if (bitCounts[bit] > 0) orValue |= 1 << bit;
    }

    while (orValue >= k && left <= right) {
      minLength = Math.min(minLength, right - left + 1);
      const leftNum = nums[left];
      for (let bit = 0; bit < 32; bit++) {
        if (leftNum & (1 << bit)) {
          bitCounts[bit]--;
          if (bitCounts[bit] === 0) orValue &= ~(1 << bit);
        }
      }
```

```
      left++;
    }
  }

  return minLength === Infinity ? -1 : minLength;
};
```

3099 Harshad Number

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3099. Harshad Number
 * https://leetcode.com/problems/harshad-number/
 * Difficulty: Easy
 *
 * An integer divisible by the sum of its digits is said to be a Harshad number. You are given
 * an integer x. Return the sum of the digits of x if x is a Harshad number, otherwise, return
 -1.
 */

/**
 * @param {number} x
 * @return {number}
 */
var sumOfTheDigitsOfHarshadNumber = function(x) {
  let digitSum = 0;
  let num = x;
  while (num > 0) {
    digitSum += num % 10;
    num = Math.floor(num / 10);
  }
  return x % digitSum === 0 ? digitSum : -1;
};
```

3100 Water Bottles Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3100. Water Bottles II
 * https://leetcode.com/problems/water-bottles-ii/
 * Difficulty: Medium
 *
 * You are given two integers numBottles and numExchange.
 *
 * numBottles represents the number of full water bottles that you initially have. In one
 * operation, you can perform one of the following operations:
 * - Drink any number of full water bottles turning them into empty bottles.
 * - Exchange numExchange empty bottles with one full water bottle. Then, increase
 *   numExchange by one.
 *
 * Note that you cannot exchange multiple batches of empty bottles for the same value of
 * numExchange. For example, if numBottles == 3 and numExchange == 1, you cannot exchange
 * 3 empty water bottles for 3 full bottles.
 *
 * Return the maximum number of water bottles you can drink.
 */

/**
 * @param {number} numBottles
 * @param {number} numExchange
 * @return {number}
 */
var maxBottlesDrunk = function(numBottles, numExchange) {
```

```
  let result = 0;
  let full = numBottles;
  let empty = 0;

  while (full > 0) {
    result += full;
    empty += full;
    full = 0;
    if (empty >= numExchange) {
      full = 1;
      empty -= numExchange;
      numExchange++;
    }
  }

  return result;
};
```

3104 Find Longest Self Contained Substring

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3104. Find Longest Self-Contained Substring
 * https://leetcode.com/problems/find-longest-self-contained-substring/
 * Difficulty: Hard
 *
 * Given a string s, your task is to find the length of the longest self-contained substring
 * of s.
 *
 * A substring t of a string s is called self-contained if t != s and for every character in
 * t, it doesn't exist in the rest of s.
 *
 * Return the length of the longest self-contained substring of s if it exists, otherwise,
 * return -1.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxSubstringLength = function(s) {
  const firstOccurrences = new Map();
  const lastOccurrences = new Map();

  for (let i = 0; i < s.length; i++) {
    const char = s[i];
    if (!firstOccurrences.has(char)) {
      firstOccurrences.set(char, i);
      lastOccurrences.set(char, i);
    } else {
      lastOccurrences.set(char, i);
    }
  }

  let result = -1;

  for (const [startChar, startIndex] of firstOccurrences) {
    const currentStart = startIndex;
    let currentEnd = lastOccurrences.get(startChar);

    for (let j = currentStart; j < s.length; j++) {
      const currentChar = s[j];

      if (firstOccurrences.get(currentChar) < currentStart) {
        break;
      }

      currentEnd = Math.max(currentEnd, lastOccurrences.get(currentChar));
```

```
      if (currentEnd === j && currentEnd - currentStart + 1 !== s.length) {
        result = Math.max(result, currentEnd - currentStart + 1);
      }
    }
  }

  return result;
};
```

3105 Longest Strictly Increasing Or Strictly Decreasing Subarray

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3105. Longest Strictly Increasing or Strictly Decreasing Subarray
 * https://leetcode.com/problems/longest-strictly-increasing-or-strictly-decreasing-subarray/
 * Difficulty: Easy
 *
 * You are given an array of integers nums. Return the length of the longest subarray of nums
 * which is either strictly increasing or strictly decreasing.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var longestMonotonicSubarray = function(nums) {
  let result = nums.length > 0 ? 1 : 0;
  for (let i = 1, down = 1, up = 1; i < nums.length; i++) {
    nums[i - 1] > nums[i] ? down++ : down = 1;
    nums[i - 1] < nums[i] ? up++ : up = 1;
    result = Math.max(result, down, up);
  }
  return result;
};
```

3108 Minimum Cost Walk In Weighted Graph

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3108. Minimum Cost Walk in Weighted Graph
 * https://leetcode.com/problems/minimum-cost-walk-in-weighted-graph/
 * Difficulty: Hard
 *
 * There is an undirected weighted graph with n vertices labeled from 0 to n - 1.
 *
 * You are given the integer n and an array edges, where edges[i] = [ui, vi, wi] indicates
that
 * there is an edge between vertices ui and vi with a weight of wi.
 *
 * A walk on a graph is a sequence of vertices and edges. The walk starts and ends with a
vertex,
 * and each edge connects the vertex that comes before it and the vertex that comes after it.
 * It's important to note that a walk may visit the same edge or vertex more than once.
 *
 * The cost of a walk starting at node u and ending at node v is defined as the bitwise AND of
 the
 * weights of the edges traversed during the walk. In other words, if the sequence of edge
weights
 * encountered during the walk is w0, w1, w2, ..., wk, then the cost is calculated as w0 & w1
& w2
 * & ... & wk, where & denotes the bitwise AND operator.
 *
 * You are also given a 2D array query, where query[i] = [si, ti]. For each query, you need to
```

```
 find
 * the minimum cost of the walk starting at vertex si and ending at vertex ti. If there exists
 no
 * such walk, the answer is -1.
 *
 * Return the array answer, where answer[i] denotes the minimum cost of a walk for query i.
 */

/**
 * @param {number} n
 * @param {number[][]} edges
 * @param {number[][]} query
 * @return {number[]}
 */
var minimumCost = function(n, edges, query) {
  const parent = new Array(n).fill().map((_, i) => i);
  const costs = new Array(n).fill(2 ** 17 - 1);

  for (const [u, v, w] of edges) {
    const [p1, p2] = [find(u), find(v)];
    parent[p1] = p2;
    costs[p1] = costs[p2] = costs[p1] & costs[p2] & w;
  }

  for (let i = 0; i < n; i++) {
    parent[i] = find(i);
  }

  return query.map(([s, t]) => {
    if (s === t) return 0;
    return parent[s] === parent[t] ? costs[parent[s]] : -1;
  });

  function find(key) {
    if (parent[key] !== key) {
      parent[key] = find(parent[key]);
    }
    return parent[key];
  }
};
```

3109 Find The Index Of Permutation

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3109. Find the Index of Permutation
 * https://leetcode.com/problems/find-the-index-of-permutation/
 * Difficulty: Medium
 *
 * Given an array perm of length n which is a permutation of [1, 2, ..., n], return the index
 * of perm in the lexicographically sorted array of all of the permutations of [1, 2, ..., n].
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} perm
 * @return {number}
 */
var getPermutationIndex = function(perm) {
  const smallerToRight = perm.map(() => 0);

  mergeSort(0, perm.length - 1);

  const MOD = 10 ** 9 + 7;
  const factorials = computeFactorials(perm.length);

  return perm.reduce((total, _, i) => {
    return (total + smallerToRight[i] * factorials[perm.length - 1 - i]) % MOD;
```

```
  }, 0);

  function mergeSort(left, right) {
    if (left >= right) return [left];

    const mid = (left + right) >> 1;
    const leftIndices = mergeSort(left, mid);
    const rightIndices = mergeSort(mid + 1, right);
    const merged = [];
    let [leftPointer, rightPointer] = [0, 0];

    while (leftPointer < leftIndices.length || rightPointer < rightIndices.length) {
      if (rightPointer === rightIndices.length
          || (leftPointer < leftIndices.length
             && perm[leftIndices[leftPointer]] < perm[rightIndices[rightPointer]])) {
        smallerToRight[leftIndices[leftPointer]] += rightPointer;
        merged.push(leftIndices[leftPointer++]);
      } else {
        merged.push(rightIndices[rightPointer++]);
      }
    }

    return merged;
  }

  function computeFactorials(n) {
    const MOD = 10n ** 9n + 7n;
    const factorials = [1n, 1n];

    for (let i = 2n; i <= BigInt(n); i++) {
      factorials.push(i * factorials[factorials.length - 1] % MOD);
    }

    return factorials.map(val => parseInt(val));
  }
};
```

3110 Score Of A String

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3110. Score of a String
 * https://leetcode.com/problems/score-of-a-string/
 * Difficulty: Easy
 *
 * You are given a string s. The score of a string is defined as the sum of the absolute
 * difference between the ASCII values of adjacent characters.
 *
 * Return the score of s.
 */

/**
 * @param {string} s
 * @return {number}
 */
var scoreOfString = function(s) {
  let sum = 0;
  for (let i = 0; i < s.length - 1; i++) {
    sum += Math.abs(s[i].charCodeAt() - s[i + 1].charCodeAt());
  }
  return sum;
};
```

3119 Maximum Number Of Potholes That Can Be Fixed

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3119. Maximum Number of Potholes That Can Be Fixed
 * https://leetcode.com/problems/maximum-number-of-potholes-that-can-be-fixed/
 * Difficulty: Medium
 *
 * You are given a string road, consisting only of characters "x" and ".", where each "x"
denotes
 * a pothole and each "." denotes a smooth road, and an integer budget.
 *
 * In one repair operation, you can repair n consecutive potholes for a price of n + 1.
 *
 * Return the maximum number of potholes that can be fixed such that the sum of the prices of
all
 * of the fixes doesn't go over the given budget.
 */

/**
 * @param {string} road
 * @param {number} budget
 * @return {number}
 */
var maxPotholes = function(road, budget) {
  const potholeSections = [];
  let currentLength = 0;

  for (const char of road) {
    if (char === 'x') {
      currentLength++;
    } else {
      if (currentLength > 0) {
        potholeSections.push(currentLength);
        currentLength = 0;
      }
    }
  }

  if (currentLength > 0) {
    potholeSections.push(currentLength);
  }

  potholeSections.sort((a, b) => (a + 1) / a - (b + 1) / b);

  let result = 0;
  let remainingBudget = budget;
  for (const sectionLength of potholeSections) {
    const costToFixAll = sectionLength + 1;

    if (costToFixAll <= remainingBudget) {
      result += sectionLength;
      remainingBudget -= costToFixAll;
    } else {
      const maxFixable = Math.max(0, remainingBudget - 1);
      result += Math.min(maxFixable, sectionLength);
      break;
    }
  }

  return result;
};
```

3125 Maximum Number That Makes Result Of Bitwise And Zero

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3125. Maximum Number That Makes Result of Bitwise AND Zero
 * https://leetcode.com/problems/maximum-number-that-makes-result-of-bitwise-and-zero/
 * Difficulty: Medium
 *
 * Given an integer n, return the maximum integer x such that x <= n, and the bitwise AND of
 * all the numbers in the range [x, n] is 0.
 */

/**
 * @param {number} n
 * @return {number}
 */
var maxNumber = function(n) {
  let bit = 1;

  while (bit <= n) {
    bit *= 2;
  }

  return bit / 2 - 1;
};
```

3135 Equalize Strings By Adding Or Removing Characters At Ends

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3135. Equalize Strings by Adding or Removing Characters at Ends
 * https://leetcode.com/problems/equalize-strings-by-adding-or-removing-characters-at-ends/
 * Difficulty: Medium
 *
 * Given two strings initial and target, your task is to modify initial by performing a series
 * of operations to make it equal to target.
 *
 * In one operation, you can add or remove one character only at the beginning or the end of
 * the string initial.
 *
 * Return the minimum number of operations required to transform initial into target.
 */

/**
 * @param {string} initial
 * @param {string} target
 * @return {number}
 */
var minOperations = function(initial, target) {
  const initialLength = initial.length;
  const targetLength = target.length;
  let previousRow = new Array(targetLength).fill(0);
  let maxCommonSubsequence = 0;

  for (let i = 0; i < initialLength; i++) {
    const currentRow = new Array(targetLength).fill(0);

    for (let j = 0; j < targetLength; j++) {
      if (initial[i] === target[j]) {
        currentRow[j] = 1 + (j - 1 >= 0 ? previousRow[j - 1] : 0);
        maxCommonSubsequence = Math.max(maxCommonSubsequence, currentRow[j]);
      }
    }
```

```
    previousRow = currentRow;
  }

  return initialLength + targetLength - 2 * maxCommonSubsequence;
};
```

3136 Valid Word

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3136. Valid Word
 * https://leetcode.com/problems/valid-word/
 * Difficulty: Easy
 *
 * A word is considered valid if:
 * - It contains a minimum of 3 characters.
 * - It contains only digits (0-9), and English letters (uppercase and lowercase).
 * - It includes at least one vowel.
 * - It includes at least one consonant.
 *
 * You are given a string word.
 *
 * Return true if word is valid, otherwise, return false.
 *
 * Notes:
 * - 'a', 'e', 'i', 'o', 'u', and their uppercases are vowels.
 * - A consonant is an English letter that is not a vowel.
 */

/**
 * @param {string} word
 * @return {boolean}
 */
var isValid = function(word) {
  if (word.length < 3) return false;

  const vowels = new Set('aeiouAEIOU');
  const special = new Set('@#$');
  const digits = new Set('0123456789');
  const wordSet = new Set(word);

  if (hasIntersection(wordSet, special)) return false;
  if (!hasIntersection(wordSet, vowels)) return false;

  const result = new Set([...wordSet].filter(char => {
    return !vowels.has(char) && !digits.has(char);
  }));

  return result.size > 0;

  function hasIntersection(set1, set2) {
    for (const item of set1) {
      if (set2.has(item)) return true;
    }
    return false;
  }
};
```

3141 Maximum Hamming Distances

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
```

```
 * 3141. Maximum Hamming Distances
 * https://leetcode.com/problems/maximum-hamming-distances/
 * Difficulty: Hard
 *
 * Given an array nums and an integer m, with each element nums[i] satisfying 0 <= nums[i] <
2m,
 * return an array answer. The answer array should be of the same length as nums, where each
 * element answer[i] represents the maximum Hamming distance between nums[i] and any other
element
 * nums[j] in the array.
 *
 * The Hamming distance between two binary integers is defined as the number of positions at
which
 * the corresponding bits differ (add leading zeroes if needed).
 */

/**
 * @param {number[]} nums
 * @param {number} m
 * @return {number[]}
 */
var maxHammingDistances = function(nums, m) {
  const totalSize = 1 << m;
  const maxDistances = new Array(totalSize).fill(-Infinity);

  for (const num of nums) {
    maxDistances[num] = 0;
  }
  for (let bitPosition = 0; bitPosition < m; bitPosition++) {
    const previousDistances = [...maxDistances];
    for (let number = 0; number < totalSize; number++) {
      const flippedNumber = number ^ (1 << bitPosition);
      maxDistances[number] = Math.max(maxDistances[number], previousDistances[flippedNumber] +
 1);
    }
  }

  return nums.map(num => maxDistances[num]);
};
```

3147 Taking Maximum Energy From The Mystic Dungeon

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3147. Taking Maximum Energy From the Mystic Dungeon
 * https://leetcode.com/problems/taking-maximum-energy-from-the-mystic-dungeon/
 * Difficulty: Medium
 *
 * In a mystic dungeon, n magicians are standing in a line. Each magician has an attribute
 * that gives you energy. Some magicians can give you negative energy, which means taking
 * energy from you.
 *
 * You have been cursed in such a way that after absorbing energy from magician i, you will
 * be instantly transported to magician (i + k). This process will be repeated until you
 * reach the magician where (i + k) does not exist.
 *
 * In other words, you will choose a starting point and then teleport with k jumps until you
 * reach the end of the magicians' sequence, absorbing all the energy during the journey.
 *
 * You are given an array energy and an integer k. Return the maximum possible energy you can
gain.
 *
 * Note that when you are reach a magician, you must take energy from them, whether it is
negative
 * or positive energy.
 */

/**
 * @param {number[]} energy
```

```
 * @param {number} k
 * @return {number}
 */
var maximumEnergy = function(energy, k) {
  const n = energy.length;
  const dp = new Array(n).fill(0);
  let result = -Infinity;

  for (let i = n - 1; i >= 0; i--) {
    dp[i] = energy[i] + (i + k < n ? dp[i + k] : 0);
    result = Math.max(result, dp[i]);
  }

  return result;
};
```

3151 Special Array I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3151. Special Array I
 * https://leetcode.com/problems/special-array-i/
 * Difficulty: Easy
 *
 * An array is considered special if every pair of its adjacent elements contains
 * two numbers with different parity.
 *
 * You are given an array of integers nums. Return true if nums is a special array,
 * otherwise, return false.
 */

/**
 * @param {number[]} nums
 * @return {boolean}
 */
var isArraySpecial = function(nums) {
  for (let i = 0; i < nums.length - 1; i++) {
    if ((nums[i] + nums[i + 1]) % 2 === 0) {
      return false;
    }
  }
  return true;
};
```

3155 Maximum Number Of Upgradable Servers

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3155. Maximum Number of Upgradable Servers
 * https://leetcode.com/problems/maximum-number-of-upgradable-servers/
 * Difficulty: Medium
 *
 * You have n data centers and need to upgrade their servers.
 *
 * You are given four arrays count, upgrade, sell, and money of length n, which show:
 * - The number of servers
 * - The cost of upgrading a single server
 * - The money you get by selling a server
 * - The money you initially have
 * - for each data center respectively.
 *
 * Return an array answer, where for each data center, the corresponding element in answer
 * represents the maximum number of servers that can be upgraded.
```

```
 *
 * Note that the money from one data center cannot be used for another data center.
 */

/**
 * @param {number[]} count
 * @param {number[]} upgrade
 * @param {number[]} sell
 * @param {number[]} money
 * @return {number[]}
 */
var maxUpgrades = function(count, upgrade, sell, money) {
  const n = count.length;
  const result = [...count];

  for (let i = 0; i < n; i++) {
    const totalUpgradeCost = count[i] * upgrade[i] - money[i];

    if (totalUpgradeCost > 0) {
      const netGainPerSale = sell[i] + upgrade[i];
      const serversToSell = Math.ceil(totalUpgradeCost / netGainPerSale);
      result[i] -= serversToSell;
    }
  }

  return result;
};
```

3157 Find The Level Of Tree With Minimum Sum

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3157. Find the Level of Tree with Minimum Sum
 * https://leetcode.com/problems/find-the-level-of-tree-with-minimum-sum/
 * Difficulty: Medium
 *
 * Given the root of a binary tree root where each node has a value, return the level of the
 * tree that has the minimum sum of values among all the levels (in case of a tie, return the
 * lowest level).
 *
 * Note that the root of the tree is at level 1 and the level of any other node is its
distance
 * from the root + 1.
 */

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {number}
 */
var minimumLevel = function(root) {
  if (!root) return 1;

  const queue = [root];
  let currentLevel = 1;
  let minSum = Infinity;
  let result = 1;

  while (queue.length > 0) {
    const levelSize = queue.length;
    let levelSum = 0;
```

```
    for (let i = 0; i < levelSize; i++) {
      const node = queue.shift();
      levelSum += node.val;

      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }

    if (levelSum < minSum) {
      minSum = levelSum;
      result = currentLevel;
    }

    currentLevel++;
  }

  return result;
};
```

3160 Find The Number Of Distinct Colors Among The Balls

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3160. Find the Number of Distinct Colors Among the Balls
 * https://leetcode.com/problems/find-the-number-of-distinct-colors-among-the-balls/
 * Difficulty: Medium
 *
 * You are given an integer limit and a 2D array queries of size n x 2.
 *
 * There are limit + 1 balls with distinct labels in the range [0, limit]. Initially,
 * all balls are uncolored. For every query in queries that is of the form [x, y],
 * you mark ball x with the color y. After each query, you need to find the number
 * of distinct colors among the balls.
 *
 * Return an array result of length n, where result[i] denotes the number of distinct
 * colors after ith query.
 *
 * Note that when answering a query, lack of a color will not be considered as a color.
 */

/**
 * @param {number} limit
 * @param {number[][]} queries
 * @return {number[]}
 */
var queryResults = function(limit, queries) {
  const result = [];

  for (let i = 0, colors = new Map(), counts = new Map(); i < queries.length; i++) {
    const [index, color] = queries[i];
    if (colors.has(index)) {
      const prev = colors.get(index);
      counts.set(prev, counts.get(prev) - 1);
      if (!counts.get(prev)) counts.delete(prev);
    }
    colors.set(index, color);
    counts.set(color, (counts.get(color) ?? 0) + 1);
    result.push(counts.size);
  }

  return result;
};
```

3167 Better Compression Of String

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3167. Better Compression of String
 * https://leetcode.com/problems/better-compression-of-string/
 * Difficulty: Medium
 *
 * You are given a string compressed representing a compressed version of a string. The format
 * is a character followed by its frequency. For example, "a3b1a1c2" is a compressed version
 * of the string "aaabacc".
 *
 * We seek a better compression with the following conditions:
 * 1. Each character should appear only once in the compressed version.
 * 2. The characters should be in alphabetical order.
 *
 * Return the better compression of compressed.
 *
 * Note: In the better version of compression, the order of letters may change, which is
acceptable.
 */

/**
 * @param {string} compressed
 * @return {string}
 */
var betterCompression = function(compressed) {
  const map = new Map();
  let i = 0;

  while (i < compressed.length) {
    const char = compressed[i];
    i++;

    let frequency = '';
    while (i < compressed.length && !isNaN(compressed[i])) {
      frequency += compressed[i];
      i++;
    }

    const count = parseInt(frequency);
    map.set(char, (map.get(char) || 0) + count);
  }

  const sortedChars = [...map.keys()].sort();
  const result = sortedChars.map(char => char + map.get(char)).join('');

  return result;
};
```

3169 Count Days Without Meetings

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3169. Count Days Without Meetings
 * https://leetcode.com/problems/count-days-without-meetings/
 * Difficulty: Medium
 *
 * You are given a positive integer days representing the total number of days an employee is
 * available for work (starting from day 1). You are also given a 2D array meetings of size n
 * where, meetings[i] = [start_i, end_i] represents the starting and ending days of meeting
```

```
 * i (inclusive).
 *
 * Return the count of days when the employee is available for work but no meetings are
scheduled.
 *
 * Note: The meetings may overlap.
 */

/**
 * @param {number} days
 * @param {number[][]} meetings
 * @return {number}
 */
var countDays = function(days, meetings) {
  meetings.sort((a, b) => a[0] - b[0]);

  let result = 0;
  let currentEnd = 0;

  for (const [start, end] of meetings) {
    if (start > currentEnd + 1) {
      result += start - currentEnd - 1;
    }
    currentEnd = Math.max(currentEnd, end);
  }

  if (currentEnd < days) {
    result += days - currentEnd;
  }

  return result;
};
```

3170 Lexicographically Minimum String After Removing Stars

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3170. Lexicographically Minimum String After Removing Stars
 * https://leetcode.com/problems/lexicographically-minimum-string-after-removing-stars/
 * Difficulty: Medium
 *
 * You are given a string s. It may contain any number of '*' characters. Your task is to
 * remove all '*' characters.
 *
 * While there is a '*', do the following operation:
 * - Delete the leftmost '*' and the smallest non-'*' character to its left. If there are
several
 *   smallest characters, you can delete any of them.
 *
 * Return the lexicographically smallest resulting string after removing all '*' characters.
 */

/**
 * @param {string} s
 * @return {string}
 */
var clearStars = function(s) {
  const chars = s.split('');
  const deleted = new Set();
  const stacks = new Array(26).fill().map(() => []);

  for (let i = 0; i < s.length; i++) {
    if (s[i] === '*') {
      for (let j = 0; j < 26; j++) {
        if (stacks[j].length) {
          deleted.add(stacks[j].pop());
          deleted.add(i);
          break;
        }
```

```
      }
    } else {
      stacks[s[i].charCodeAt(0) - 97].push(i);
    }
  }

  return chars.filter((_, i) => !deleted.has(i)).join('');
};
```

3173 Bitwise Or Of Adjacent Elements

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3173. Bitwise OR of Adjacent Elements
 * https://leetcode.com/problems/bitwise-or-of-adjacent-elements/
 * Difficulty: Easy
 *
 * Given an array nums of length n, return an array answer of length n - 1 such that
 * answer[i] = nums[i] | nums[i + 1] where | is the bitwise OR operation.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var orArray = function(nums) {
  const result = [];

  for (let i = 0; i < nums.length - 1; i++) {
    result.push(nums[i] | nums[i + 1]);
  }

  return result;
};
```

3174 Clear Digits

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3174. Clear Digits
 * https://leetcode.com/problems/clear-digits/
 * Difficulty: Easy
 *
 * You are given a string s.
 *
 * Your task is to remove all digits by doing this operation repeatedly:
 * - Delete the first digit and the closest non-digit character to its left.
 *
 * Return the resulting string after removing all digits.
 */

/**
 * @param {string} s
 * @return {string}
 */
var clearDigits = function(s) {
  const stack = [];

  for (const character of s) {
    if (!isNaN(character) && stack.length) {
      stack.pop();
    } else {
      stack.push(character);
```

```
    }
  }

  return stack.join('');
};
```

3178 Find The Child Who Has The Ball After K Seconds

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3178. Find the Child Who Has the Ball After K Seconds
 * https://leetcode.com/problems/find-the-child-who-has-the-ball-after-k-seconds/
 * Difficulty: Easy
 *
 * You are given two positive integers n and k. There are n children numbered from 0 to n - 1
 * standing in a queue in order from left to right.
 *
 * Initially, child 0 holds a ball and the direction of passing the ball is towards the right
 * direction. After each second, the child holding the ball passes it to the child next to
 * them. Once the ball reaches either end of the line, i.e. child 0 or child n - 1, the
direction
 * of passing is reversed.
 *
 * Return the number of the child who receives the ball after k seconds.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var numberOfChild = function(n, k) {
  const position = k % (2 * (n - 1));
  return position < n ? position : 2 * n - position - 2;
};
```

3183 The Number Of Ways To Make The Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3183. The Number of Ways to Make the Sum
 * https://leetcode.com/problems/the-number-of-ways-to-make-the-sum/
 * Difficulty: Medium
 *
 * You have an infinite number of coins with values 1, 2, and 6, and only 2 coins with value
4.
 *
 * Given an integer n, return the number of ways to make the sum of n with the coins you have.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 *
 * Note that the order of the coins doesn't matter and [2, 2, 3] is the same as [2, 3, 2].
 */

/**
 * @param {number} n
 * @return {number}
 */
var numberOfWays = function(n) {
  const MOD = 1e9 + 7;
  const dp = new Array(n + 1).fill(0);
  dp[0] = 1;
```

```
  for (const coin of [1, 2, 6]) {
    for (let amount = coin; amount <= n; amount++) {
      dp[amount] = (dp[amount] + dp[amount - coin]) % MOD;
    }
  }

  let result = dp[n];
  if (n >= 4) {
    result = (result + dp[n - 4]) % MOD;
  }
  if (n >= 8) {
    result = (result + dp[n - 8]) % MOD;
  }

  return result;
};
```

3186 Maximum Total Damage With Spell Casting

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3186. Maximum Total Damage With Spell Casting
 * https://leetcode.com/problems/maximum-total-damage-with-spell-casting/
 * Difficulty: Medium
 *
 * A magician has various spells.
 *
 * You are given an array power, where each element represents the damage of a spell. Multiple
 * spells can have the same damage value.
 *
 * It is a known fact that if a magician decides to cast a spell with a damage of power[i],
 * they cannot cast any spell with a damage of power[i] - 2, power[i] - 1, power[i] + 1, or
 * power[i] + 2.
 *
 * Each spell can be cast only once.
 *
 * Return the maximum possible total damage that a magician can cast.
 */

/**
 * @param {number[]} power
 * @return {number}
 */
var maximumTotalDamage = function(power) {
  const frequency = new Map();
  for (const p of power) {
    frequency.set(p, (frequency.get(p) || 0) + 1);
  }

  const uniquePowers = Array.from(frequency.keys()).sort((a, b) => a - b);
  const n = uniquePowers.length;

  if (n === 0) return 0;
  if (n === 1) return uniquePowers[0] * frequency.get(uniquePowers[0]);

  const dp = new Array(n).fill(0);
  dp[0] = uniquePowers[0] * frequency.get(uniquePowers[0]);

  for (let i = 1; i < n; i++) {
    const currentPower = uniquePowers[i];
    const currentDamage = currentPower * frequency.get(currentPower);

    let j = i - 1;
    while (j >= 0 && uniquePowers[j] >= currentPower - 2) {
      j--;
    }

    const withCurrent = currentDamage + (j >= 0 ? dp[j] : 0);
```

```
    const withoutCurrent = dp[i - 1];

    dp[i] = Math.max(withCurrent, withoutCurrent);
  }

  return dp[n - 1];
};
```

3189 Minimum Moves To Get A Peaceful Board

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3189. Minimum Moves to Get a Peaceful Board
 * https://leetcode.com/problems/minimum-moves-to-get-a-peaceful-board/
 * Difficulty: Medium
 *
 * Given a 2D array rooks of length n, where rooks[i] = [xi, yi] indicates the position of a
 * rook on an n x n chess board. Your task is to move the rooks 1 cell at a time vertically
 * or horizontally (to an adjacent cell) such that the board becomes peaceful.
 *
 * A board is peaceful if there is exactly one rook in each row and each column.
 *
 * Return the minimum number of moves required to get a peaceful board.
 *
 * Note that at no point can there be two rooks in the same cell.
 */

/**
 * @param {number[][]} rooks
 * @return {number}
 */
var minMoves = function(rooks) {
  const rows = rooks.map(rook => rook[0]).sort((a, b) => a - b);
  const cols = rooks.map(rook => rook[1]).sort((a, b) => a - b);

  let result = 0;
  for (let i = 0; i < rooks.length; i++) {
    result += Math.abs(rows[i] - i) + Math.abs(cols[i] - i);
  }

  return result;
};
```

3190 Find Minimum Operations To Make All Elements Divisible By Three

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3190. Find Minimum Operations to Make All Elements Divisible by Three
 *
https://leetcode.com/problems/find-minimum-operations-to-make-all-elements-divisible-by-three/
 * Difficulty: Easy
 *
 * You are given an integer array nums. In one operation, you can add or subtract 1 from any
 * element of nums.
 *
 * Return the minimum number of operations to make all elements of nums divisible by 3.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumOperations = function(nums) {
```

```
  return nums.reduce((operations, num) => {
    const remainder = num % 3;
    return operations + Math.min(remainder, 3 - remainder);
  }, 0);
};
```

3191 Minimum Operations To Make Binary Array Elements Equal To One I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3191. Minimum Operations to Make Binary Array Elements Equal to One I
 *
https://leetcode.com/problems/minimum-operations-to-make-binary-array-elements-equal-to-one-i/
 * Difficulty: Medium
 *
 * You are given a binary array nums.
 *
 * You can do the following operation on the array any number of times (possibly zero):
 * - Choose any 3 consecutive elements from the array and flip all of them.
 *
 * Flipping an element means changing its value from 0 to 1, and from 1 to 0.
 *
 * Return the minimum number of operations required to make all elements in nums equal to 1.
 * If it is impossible, return -1.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minOperations = function(nums) {
  const result = [...nums];
  let operations = 0;

  for (let i = 0; i < nums.length - 2; i++) {
    if (result[i] === 0) {
      result[i] ^= 1;
      result[i + 1] ^= 1;
      result[i + 2] ^= 1;
      operations++;
    }
  }

  return result.every(num => num === 1) ? operations : -1;
};
```

3195 Find The Minimum Area To Cover All Ones I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3195. Find the Minimum Area to Cover All Ones I
 * https://leetcode.com/problems/find-the-minimum-area-to-cover-all-ones-i/
 * Difficulty: Medium
 *
 * You are given a 2D binary array grid. Find a rectangle with horizontal and vertical
 * sides with the smallest area, such that all the 1's in grid lie inside this rectangle.
 *
 * Return the minimum possible area of the rectangle.
 */

/**
 * @param {number[][]} grid
 * @return {number}
```

```
 */
var minimumArea = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  let minRow = rows;
  let maxRow = -1;
  let minCol = cols;
  let maxCol = -1;

  for (let i = 0; i < rows; i++) {
    for (let j = 0; j < cols; j++) {
      if (grid[i][j] === 1) {
        minRow = Math.min(minRow, i);
        maxRow = Math.max(maxRow, i);
        minCol = Math.min(minCol, j);
        maxCol = Math.max(maxCol, j);
      }
    }
  }

  return (maxRow - minRow + 1) * (maxCol - minCol + 1);
};
```

3197 Find The Minimum Area To Cover All Ones Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3197. Find the Minimum Area to Cover All Ones II
 * https://leetcode.com/problems/find-the-minimum-area-to-cover-all-ones-ii/
 * Difficulty: Hard
 *
 * You are given a 2D binary array grid. You need to find 3 non-overlapping rectangles having
 * non-zero areas with horizontal and vertical sides such that all the 1's in grid lie inside
 * these rectangles.
 *
 * Return the minimum possible sum of the area of these rectangles.
 *
 * Note that the rectangles are allowed to touch.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minimumSum = function(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  let minSum = Infinity;

  for (let i = 0; i < rows; i++) {
    const area1 = calculateArea(0, i, 0, cols - 1);
    for (let j = 0; j < cols; j++) {
      const area2 = calculateArea(i + 1, rows - 1, 0, j);
      const area3 = calculateArea(i + 1, rows - 1, j + 1, cols - 1);
      minSum = Math.min(minSum, area1 + area2 + area3);
    }
  }

  for (let j = 0; j < cols; j++) {
    const area1 = calculateArea(0, rows - 1, 0, j);
    for (let i = 0; i < rows; i++) {
      const area2 = calculateArea(0, i, j + 1, cols - 1);
      const area3 = calculateArea(i + 1, rows - 1, j + 1, cols - 1);
      minSum = Math.min(minSum, area1 + area2 + area3);
    }
  }

  for (let j = cols - 1; j >= 0; j--) {
    const area1 = calculateArea(0, rows - 1, j + 1, cols - 1);
```

```
    for (let i = 0; i < rows; i++) {
      const area2 = calculateArea(0, i, 0, j);
      const area3 = calculateArea(i + 1, rows - 1, 0, j);
      minSum = Math.min(minSum, area1 + area2 + area3);
    }
  }

  for (let i = rows - 1; i >= 0; i--) {
    const area1 = calculateArea(i + 1, rows - 1, 0, cols - 1);
    for (let j = 0; j < cols; j++) {
      const area2 = calculateArea(0, i, 0, j);
      const area3 = calculateArea(0, i, j + 1, cols - 1);
      minSum = Math.min(minSum, area1 + area2 + area3);
    }
  }

  for (let i = 0; i < rows; i++) {
    for (let j = i + 1; j < rows; j++) {
      const area1 = calculateArea(0, i, 0, cols - 1);
      const area2 = calculateArea(i + 1, j, 0, cols - 1);
      const area3 = calculateArea(j + 1, rows - 1, 0, cols - 1);
      minSum = Math.min(minSum, area1 + area2 + area3);
    }
  }

  for (let i = 0; i < cols; i++) {
    for (let j = i + 1; j < cols; j++) {
      const area1 = calculateArea(0, rows - 1, 0, i);
      const area2 = calculateArea(0, rows - 1, i + 1, j);
      const area3 = calculateArea(0, rows - 1, j + 1, cols - 1);
      minSum = Math.min(minSum, area1 + area2 + area3);
    }
  }

  return minSum;

  function calculateArea(startRow, endRow, startCol, endCol) {
    let minRow = Infinity;
    let maxRow = -1;
    let minCol = Infinity;
    let maxCol = -1;
    let found = false;

    for (let i = startRow; i <= endRow; i++) {
      for (let j = startCol; j <= endCol; j++) {
        if (grid[i][j] === 1) {
          minRow = Math.min(minRow, i);
          maxRow = Math.max(maxRow, i);
          minCol = Math.min(minCol, j);
          maxCol = Math.max(maxCol, j);
          found = true;
        }
      }
    }

    return found ? (maxRow - minRow + 1) * (maxCol - minCol + 1) : 0;
  }
};
```

3199 Count Triplets With Even Xor Set Bits I

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3199. Count Triplets with Even XOR Set Bits I
 * https://leetcode.com/problems/count-triplets-with-even-xor-set-bits-i/
 * Difficulty: Easy
 *
 * Given three integer arrays a, b, and c, return the number of triplets (a[i], b[j], c[k]),
 * such that the bitwise XOR of the elements of each triplet has an even number of set bits.
```

```
 */

/**
 * @param {number[]} a
 * @param {number[]} b
 * @param {number[]} c
 * @return {number}
 */
var tripletCount = function(a, b, c) {
  let result = 0;

  for (const numA of a) {
    for (const numB of b) {
      for (const numC of c) {
        const xorResult = numA ^ numB ^ numC;
        if (hasEvenSetBits(xorResult)) {
          result++;
        }
      }
    }
  }

  return result;

  function hasEvenSetBits(num) {
    let setBitsCount = 0;

    while (num > 0) {
      setBitsCount += num & 1;
      num >>= 1;
    }

    return setBitsCount % 2 === 0;
  }
};
```

3201 Find The Maximum Length Of Valid Subsequence I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3201. Find the Maximum Length of Valid Subsequence I
 * https://leetcode.com/problems/find-the-maximum-length-of-valid-subsequence-i/
 * Difficulty: Medium
 *
 * You are given an integer array nums.
 *
 * A subsequence sub of nums with length x is called valid if it satisfies:
 * - (sub[0] + sub[1]) % 2 == (sub[1] + sub[2]) % 2 == ... == (sub[x - 2] + sub[x - 1]) % 2.
 *
 * Return the length of the longest valid subsequence of nums.
 *
 * A subsequence is an array that can be derived from another array by deleting some or no
 * elements without changing the order of the remaining elements.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maximumLength = function(nums) {
  const evenCount = nums.filter(num => num % 2 === 0).length;
  const oddCount = nums.length - evenCount;

  let alternatingCount = 0;
  let lastParity = -1;
  for (const num of nums) {
    const currentParity = num % 2;
    if (lastParity === -1 || currentParity !== lastParity) {
      alternatingCount++;
```

```
      lastParity = currentParity;
    }
  }

  return Math.max(evenCount, oddCount, alternatingCount);
};
```

3202 Find The Maximum Length Of Valid Subsequence Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3202. Find the Maximum Length of Valid Subsequence II
 * https://leetcode.com/problems/find-the-maximum-length-of-valid-subsequence-ii/
 * Difficulty: Medium
 *
 * You are given an integer array nums and a positive integer k.
 *
 * A subsequence sub of nums with length x is called valid if it satisfies:
 * - (sub[0] + sub[1]) % k == (sub[1] + sub[2]) % k == ... == (sub[x - 2] + sub[x - 1]) % k.
 *
 * Return the length of the longest valid subsequence of nums.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maximumLength = function(nums, k) {
  let result = 0;

  for (let target = 0; target < k; target++) {
    const dp = new Array(k).fill(0);

    for (const num of nums) {
      const mod = num % k;
      const prev = (target - mod + k) % k;
      dp[mod] = Math.max(dp[mod], dp[prev] + 1);
    }

    result = Math.max(result, Math.max(...dp));
  }

  return result;
};
```

3205 Maximum Array Hopping Score I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3205. Maximum Array Hopping Score I
 * https://leetcode.com/problems/maximum-array-hopping-score-i/
 * Difficulty: Medium
 *
 * Given an array nums, you have to get the maximum score starting from index 0 and
 * hopping until you reach the last element of the array.
 *
 * In each hop, you can jump from index i to an index j > i, and you get a score of
 * (j - i) * nums[j].
 *
 * Return the maximum score you can get.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var maxScore = function(nums) {
  const n = nums.length;
  const dp = new Array(n).fill(-Infinity);
  dp[0] = 0;

  for (let i = 0; i < n; i++) {
    if (dp[i] === -Infinity) continue;

    for (let j = i + 1; j < n; j++) {
      const jumpScore = (j - i) * nums[j];
      dp[j] = Math.max(dp[j], dp[i] + jumpScore);
    }
  }

  return dp[n - 1];
};
```

3208 Alternating Groups Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3208. Alternating Groups II
 * https://leetcode.com/problems/alternating-groups-ii/
 * Difficulty: Medium
 *
 * There is a circle of red and blue tiles. You are given an array of integers colors and
 * an integer k. The color of tile i is represented by colors[i]:
 * - colors[i] == 0 means that tile i is red.
 *  -colors[i] == 1 means that tile i is blue.
 *
 * An alternating group is every k contiguous tiles in the circle with alternating colors
 * (each tile in the group except the first and last one has a different color from its
 * left and right tiles).
 *
 * Return the number of alternating groups.
 *
 * Note that since colors represents a circle, the first and the last tiles are considered
 * to be next to each other.
 */

/**
 * @param {number[]} colors
 * @param {number} k
 * @return {number}
 */
var numberOfAlternatingGroups = function(colors, k) {
  const extended = colors.concat(colors.slice(0, k - 1));
  let result = 0;
  let invalid = 0;

  for (let i = 1; i < k; i++) {
    if (extended[i] === extended[i - 1]) {
      invalid++;
    }
  }
  if (invalid === 0) {
    result++;
  }
  for (let i = 1; i < colors.length; i++) {
    if (extended[i] === extended[i - 1]) invalid--;
    if (extended[i + k - 1] === extended[i + k - 2]) invalid++;
    if (invalid === 0) result++;
  }

  return result;
```

```
};
```

3215 Count Triplets With Even Xor Set Bits Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3215. Count Triplets with Even XOR Set Bits II
 * https://leetcode.com/problems/count-triplets-with-even-xor-set-bits-ii/
 * Difficulty: Medium
 *
 * Given three integer arrays a, b, and c, return the number of triplets (a[i], b[j], c[k]),
 * such that the bitwise XOR between the elements of each triplet has an even number of set
bits.
 */

/**
 * @param {number[]} a
 * @param {number[]} b
 * @param {number[]} c
 * @return {number}
 */
var tripletCount = function(a, b, c) {
  const countA = [0, 0];
  const countB = [0, 0];
  const countC = [0, 0];

  for (const num of a) {
    countA[countSetBits(num) % 2]++;
  }
  for (const num of b) {
    countB[countSetBits(num) % 2]++;
  }
  for (const num of c) {
    countC[countSetBits(num) % 2]++;
  }

  let result = 0;
  result += countA[0] * countB[0] * countC[0];
  result += countA[1] * countB[1] * countC[0];
  result += countA[1] * countB[0] * countC[1];
  result += countA[0] * countB[1] * countC[1];

  return result;

  function countSetBits(num) {
    let count = 0;
    while (num > 0) {
      count += num & 1;
      num >>= 1;
    }
    return count;
  }
};
```

3217 Delete Nodes From Linked List Present In Array

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3217. Delete Nodes From Linked List Present in Array
 * https://leetcode.com/problems/delete-nodes-from-linked-list-present-in-array/
 * Difficulty: Medium
 *
 * You are given an array of integers nums and the head of a linked list. Return the head
```

```
 * of the modified linked list after removing all nodes from the linked list that have
 * a value that exists in nums.
 */

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {number[]} nums
 * @param {ListNode} head
 * @return {ListNode}
 */
var modifiedList = function(nums, head) {
  const set = new Set(nums);
  const temp = new ListNode(0, head);
  let current = temp;

  while (current.next) {
    if (set.has(current.next.val)) {
      current.next = current.next.next;
    } else {
      current = current.next;
    }
  }

  return temp.next;
};
```

3221 Maximum Array Hopping Score Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3221. Maximum Array Hopping Score II
 * https://leetcode.com/problems/maximum-array-hopping-score-ii/
 * Difficulty: Medium
 *
 * Given an array nums, you have to get the maximum score starting from index 0 and
 * hopping until you reach the last element of the array.
 *
 * In each hop, you can jump from index i to an index j > i, and you get a score of
 * (j - i) * nums[j].
 *
 * Return the maximum score you can get.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxScore = function(nums) {
  let result = 0;
  let max = 0;

  for (let i = nums.length - 1; i >= 1; i--) {
    max = Math.max(max, nums[i]);
    result += max;
  }

  return result;
};
```

3223 Minimum Length Of String After Operations

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 3223. Minimum Length of String After Operations
 * https://leetcode.com/problems/minimum-length-of-string-after-operations/
 * Difficulty: Medium
 *
 * You are given a string s.
 *
 * You can perform the following process on s any number of times:
 * - Choose an index i in the string such that there is at least one character to the left of
 *   index i that is equal to s[i], and at least one character to the right that is also equal
 *   to s[i].
 * - Delete the closest character to the left of index i that is equal to s[i].
 * - Delete the closest character to the right of index i that is equal to s[i].
 *
 * Return the minimum length of the final string s that you can achieve.
 */

/**
 * @param {string} s
 * @return {number}
 */
var minimumLength = function(s) {
  if (s.length < 3) return s.length;

  const map = new Map();
  s.split('').forEach(c => map.set(c, (map.get(c) ?? 0) + 1));

  return [...map.values()].reduce((count, n) => count + (n % 2 ? n % 2 : 2), 0);
};
```

3227 Vowels Game In A String

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3227. Vowels Game in a String
 * https://leetcode.com/problems/vowels-game-in-a-string/
 * Difficulty: Medium
 *
 * Alice and Bob are playing a game on a string.
 *
 * You are given a string s, Alice and Bob will take turns playing the following game where
 * Alice starts first:
 * - On Alice's turn, she has to remove any non-empty substring from s that contains an odd
 *   number of vowels.
 * - On Bob's turn, he has to remove any non-empty substring from s that contains an even
 *   number of vowels.
 *
 * The first player who cannot make a move on their turn loses the game. We assume that both
 * Alice and Bob play optimally.
 *
 * Return true if Alice wins the game, and false otherwise.
 *
 * The English vowels are: a, e, i, o, and u.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var doesAliceWin = function(s) {
```

```
  const vowels = new Set(['a', 'e', 'i', 'o', 'u']);
  for (const char of s) {
    if (vowels.has(char)) {
      return true;
    }
  }
  return false;
};
```

3231 Minimum Number Of Increasing Subsequence To Be Removed

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3231. Minimum Number of Increasing Subsequence to Be Removed
 * https://leetcode.com/problems/minimum-number-of-increasing-subsequence-to-be-removed/
 * Difficulty: Hard
 *
 * Given an array of integers nums, you are allowed to perform the following operation any
 * number of times:
 * - Remove a strictly increasing subsequence from the array.
 *
 * Your task is to find the minimum number of operations required to make the array empty.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minOperations = function(nums) {
  const dp = [nums[nums.length - 1]];

  for (let i = nums.length - 2; i >= 0; i--) {
    const currentNum = nums[i];

    if (currentNum >= dp[dp.length - 1]) {
      dp.push(currentNum);
    } else {
      const insertionIndex = binarySearchRight(dp, currentNum);
      dp[insertionIndex] = currentNum;
    }
  }

  return dp.length;

  function binarySearchRight(arr, target) {
    let left = 0;
    let right = arr.length;

    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (arr[mid] <= target) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }

    return left;
  }
};
```

3237 Alt And Tab Simulation

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3237. Alt and Tab Simulation
 * https://leetcode.com/problems/alt-and-tab-simulation/
 * Difficulty: Medium
 *
 * There are n windows open numbered from 1 to n, we want to simulate using alt + tab to
 * navigate between the windows.
 *
 * You are given an array windows which contains the initial order of the windows (the
 * first element is at the top and the last one is at the bottom).
 *
 * You are also given an array queries where for each query, the window queries[i] is
 * brought to the top.
 *
 * Return the final state of the array windows.
 */

/**
 * @param {number[]} windows
 * @param {number[]} queries
 * @return {number[]}
 */
var simulationResult = function(windows, queries) {
  const result = [];
  const set = new Set();

  for (let i = queries.length - 1; i >= 0; i--) {
    const window = queries[i];

    if (!set.has(window)) {
      result.push(window);
      set.add(window);
    }
  }

  for (const window of windows) {
    if (!set.has(window)) {
      result.push(window);
    }
  }

  return result;
};
```

3247 Number Of Subsequences With Odd Sum

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3247. Number of Subsequences with Odd Sum
 * https://leetcode.com/problems/number-of-subsequences-with-odd-sum/
 * Difficulty: Medium
 *
 * Given an array nums, return the number of subsequences with an odd sum of elements.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var subsequenceCount = function(nums) {
  const MOD = 1e9 + 7;
  let oddCount = 0;
  let evenCount = 1;

  for (const num of nums) {
    if (num % 2 === 1) {
```

```
      const newOddCount = (oddCount + evenCount) % MOD;
      const newEvenCount = (evenCount + oddCount) % MOD;
      oddCount = newOddCount;
      evenCount = newEvenCount;
    } else {
      oddCount = (oddCount * 2) % MOD;
      evenCount = (evenCount * 2) % MOD;
    }
  }

  return oddCount;
};
```

3253 Construct String With Minimum Cost Easy

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3253. Construct String with Minimum Cost (Easy)
 * https://leetcode.com/problems/construct-string-with-minimum-cost-easy/
 * Difficulty: Medium
 *
 * You are given a string target, an array of strings words, and an integer array costs,
 * both arrays of the same length.
 *
 * Imagine an empty string s.
 *
 * You can perform the following operation any number of times (including zero):
 * - Choose an index i in the range [0, words.length - 1].
 * - Append words[i] to s.
 * - The cost of operation is costs[i].
 *
 * Return the minimum cost to make s equal to target. If it's not possible, return -1.
 */

/**
 * @param {string} target
 * @param {string[]} words
 * @param {number[]} costs
 * @return {number}
 */
var minimumCost = function(target, words, costs) {
  const n = target.length;
  const dp = new Array(n + 1).fill(Infinity);
  dp[0] = 0;

  for (let i = 0; i < n; i++) {
    if (dp[i] === Infinity) continue;

    for (let j = 0; j < words.length; j++) {
      const word = words[j];
      const cost = costs[j];

      if (i + word.length <= n && target.substring(i, i + word.length) === word) {
        dp[i + word.length] = Math.min(dp[i + word.length], dp[i] + cost);
      }
    }
  }

  return dp[n] === Infinity ? -1 : dp[n];
};
```

3263 Convert Doubly Linked List To Array I

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3263. Convert Doubly Linked List to Array I
 * https://leetcode.com/problems/convert-doubly-linked-list-to-array-i/
 * Difficulty: Easy
 *
 * You are given the head of a doubly linked list, which contains nodes that have a next
 * pointer and a previous pointer.
 *
 * Return an integer array which contains the elements of the linked list in order.
 */

/**
 * // Definition for a _Node.
 * function _Node(val,prev,next) {
 *    this.val = val;
 *    this.prev = prev;
 *    this.next = next;
 * };
 */
/**
 * @param {_Node} head
 * @return {number[]}
 */
var toArray = function(head) {
  const result = [];
  let currentNode = head;

  while (currentNode) {
    result.push(currentNode.val);
    currentNode = currentNode.next;
  }

  return result;
};
```

3269 Constructing Two Increasing Arrays

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3269. Constructing Two Increasing Arrays
 * https://leetcode.com/problems/constructing-two-increasing-arrays/
 * Difficulty: Hard
 *
 * Given 2 integer arrays nums1 and nums2 consisting only of 0 and 1, your task is to
calculate
 * the minimum possible largest number in arrays nums1 and nums2, after doing the following.
 *
 * Replace every 0 with an even positive integer and every 1 with an odd positive integer.
 * After replacement, both arrays should be increasing and each integer should be used at most
 once.
 *
 * Return the minimum possible largest number after applying the changes.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var minLargest = function(nums1, nums2) {
  const n1 = nums1.length;
  const n2 = nums2.length;
  const dp = new Array(n1 + 1).fill().map(() => new Array(n2 + 1).fill(-Infinity));

  dp[0][0] = 0;

  for (let i = 0; i < n1; i++) {
    dp[i + 1][0] = getNextValue(dp[i][0], nums1[i]);
```

```
  }
  for (let j = 0; j < n2; j++) {
    dp[0][j + 1] = getNextValue(dp[0][j], nums2[j]);
  }
  for (let i = 0; i < n1; i++) {
    for (let j = 0; j < n2; j++) {
      dp[i + 1][j + 1] = Math.min(
        getNextValue(dp[i][j + 1], nums1[i]),
        getNextValue(dp[i + 1][j], nums2[j])
      );
    }
  }

  return dp[n1][n2];

  function getNextValue(currentValue, targetParity) {
    const nextValue = currentValue + 1;
    return nextValue % 2 === targetParity % 2 ? nextValue : nextValue + 1;
  }
};
```

3272 Find The Count Of Good Integers

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3272. Find the Count of Good Integers
 * https://leetcode.com/problems/find-the-count-of-good-integers/
 * Difficulty: Hard
 *
 * You are given two positive integers n and k.
 *
 * An integer x is called k-palindromic if:
 * - x is a palindrome.
 * - x is divisible by k.
 *
 * An integer is called good if its digits can be rearranged to form a k-palindromic integer.
 * For example, for k = 2, 2020 can be rearranged to form the k-palindromic integer 2002,
 * whereas 1010 cannot be rearranged to form a k-palindromic integer.
 *
 * Return the count of good integers containing n digits.
 *
 * Note that any integer must not have leading zeros, neither before nor after rearrangement.
 * For example, 1010 cannot be rearranged to form 101.
 */

/**
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
function countGoodIntegers(n, k) {
  if (n === 1) return Math.floor(9 / k);

  let result = 0;
  const halfLength = Math.ceil(n / 2);
  const start = Math.pow(10, halfLength - 1);
  const end = Math.pow(10, halfLength);

  const seen = new Set();
  for (let i = start; i < end; i++) {
    const palindromeStr = generatePalindrome(i);
    if (palindromeStr.length !== n) continue;
    const num = parseInt(palindromeStr);
    if (num % k !== 0) continue;

    const sortedDigits = palindromeStr.split('').sort().join('');
    if (seen.has(sortedDigits)) continue;
    seen.add(sortedDigits);
```

```
    result += calculateArrangements(palindromeStr);
  }

  return result;

  function generatePalindrome(firstHalf) {
    const str = firstHalf.toString();
    const isOdd = n % 2 === 1;
    return isOdd
      ? str + str.slice(0, -1).split('').reverse().join('')
      : str + str.split('').reverse().join('');
  }

  function calculateArrangements(numStr) {
    const freq = new Map();
    for (const digit of numStr) {
      freq.set(digit, (freq.get(digit) || 0) + 1);
    }
    const total = multinomial(n, freq);
    if (!freq.has('0')) return total;

    freq.set('0', freq.get('0') - 1);
    if (freq.get('0') === 0) freq.delete('0');
    return total - multinomial(n - 1, freq);
  }

  function multinomial(length, freq) {
    const numerator = factorial(length);
    let denominator = 1;
    for (const count of freq.values()) {
      denominator *= factorial(count);
    }
    return numerator / denominator;
  }

  function factorial(num) {
    let res = 1;
    for (let i = 2; i <= num; i++) res *= i;
    return res;
  }
}
```

3279 Maximum Total Area Occupied By Pistons

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3279. Maximum Total Area Occupied by Pistons
 * https://leetcode.com/problems/maximum-total-area-occupied-by-pistons/
 * Difficulty: Hard
 *
 * There are several pistons in an old car engine, and we want to calculate the maximum
 * possible area under the pistons.
 *
 * You are given:
 * - An integer height, representing the maximum height a piston can reach.
 * - An integer array positions, where positions[i] is the current position of piston i,
 *   which is equal to the current area under it.
 * - A string directions, where directions[i] is the current moving direction of piston
 *   i, 'U' for up, and 'D' for down.
 *
 * Each second:
 * - Every piston moves in its current direction 1 unit. e.g., if the direction is up,
 *   positions[i] is incremented by 1.
 * - If a piston has reached one of the ends, i.e., positions[i] == 0 or
 *   positions[i] == height, its direction will change.
 *
 * Return the maximum possible area under all the pistons.
 */
```

```
/**
 * @param {number} height
 * @param {number[]} positions
 * @param {string} directions
 * @return {number}
 */
var maxArea = function(height, positions, directions) {
  const n = directions.length;
  let upCount = directions.split('').filter(dir => dir === 'U').length;
  let result = positions.reduce((sum, pos) => sum + pos, 0);
  let currentArea = result;
  const vertices = new Map();
  vertices.set(0, 0);

  for (let i = 0; i < n; i++) {
    const direction = directions[i];
    const position = positions[i];

    if (direction === 'U') {
      vertices.set(height - position, (vertices.get(height - position) || 0) - 1);
      vertices.set(2 * height - position, (vertices.get(2 * height - position) || 0) + 1);
    } else {
      vertices.set(height + position, (vertices.get(height + position) || 0) - 1);
      vertices.set(position, (vertices.get(position) || 0) + 1);
    }
  }

  const sortedTimes = Array.from(vertices.keys()).sort((a, b) => a - b);
  for (let i = 0; i < sortedTimes.length - 1; i++) {
    const leftTime = sortedTimes[i];
    const rightTime = sortedTimes[i + 1];

    currentArea += (rightTime - leftTime) * (2 * upCount - n);
    if (currentArea > result) {
      result = currentArea;
    }
    upCount += vertices.get(rightTime) || 0;
  }

  return result;
};
```

3284 Sum Of Consecutive Subarrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3284. Sum of Consecutive Subarrays
 * https://leetcode.com/problems/sum-of-consecutive-subarrays/
 * Difficulty: Medium
 *
 * We call an array arr of length n consecutive if one of the following holds:
 * - arr[i] - arr[i - 1] == 1 for all 1 <= i < n.
 * - arr[i] - arr[i - 1] == -1 for all 1 <= i < n.
 *
 * The value of an array is the sum of its elements.
 *
 * For example, [3, 4, 5] is a consecutive array of value 12 and [9, 8] is another of
 * value 17. While [3, 4, 3] and [8, 6] are not consecutive.
 *
 * Given an array of integers nums, return the sum of the values of all consecutive subarrays.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 *
 * Note that an array of length 1 is also considered consecutive.
 */

/**
 * @param {number[]} nums
```

```
 * @return {number}
 */
var getSum = function(nums) {
  const MOD = 1e9 + 7;
  let result = nums[0];
  let currentContribution = nums[0];
  let leftBoundary = 0;
  let previousDifference = 0;

  for (let rightIndex = 1; rightIndex < nums.length; rightIndex++) {
    const currentDifference = nums[rightIndex] - nums[rightIndex - 1];
    if (Math.abs(currentDifference) !== 1) {
      previousDifference = 0;
      leftBoundary = rightIndex;
      currentContribution = 0;
    } else if (previousDifference !== currentDifference) {
      currentContribution = nums[rightIndex - 1];
      leftBoundary = rightIndex - 1;
      previousDifference = currentDifference;
    }

    currentContribution += nums[rightIndex] * (rightIndex - leftBoundary + 1);
    result = (result + currentContribution) % MOD;
  }

  return result;
};
```

3289 The Two Sneaky Numbers Of Digitville

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3289. The Two Sneaky Numbers of Digitville
 * https://leetcode.com/problems/the-two-sneaky-numbers-of-digitville/
 * Difficulty: Easy
 *
 * In the town of Digitville, there was a list of numbers called nums containing integers
 * from 0 to n - 1. Each number was supposed to appear exactly once in the list, however,
 * two mischievous numbers sneaked in an additional time, making the list longer than usual.
 *
 * As the town detective, your task is to find these two sneaky numbers. Return an array of
 * size two containing the two numbers (in any order), so peace can return to Digitville.
 */

/**
 * @param {number[]} nums
 * @return {number[]}
 */
var getSneakyNumbers = function(nums) {
  const set = new Set();
  const result = [];

  for (const n of nums) {
    if (set.has(n)) {
      result.push(n);
    } else {
      set.add(n);
    }
  }

  return result;
};
```

3294 Convert Doubly Linked List To Array Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3294. Convert Doubly Linked List to Array II
 * https://leetcode.com/problems/convert-doubly-linked-list-to-array-ii/
 * Difficulty: Medium
 *
 * You are given an arbitrary node from a doubly linked list, which contains nodes that
 * have a next pointer and a previous pointer.
 *
 * Return an integer array which contains the elements of the linked list in order.
 */

/**
 * // Definition for a _Node.
 * function _Node(val,prev,next) {
 *    this.val = val;
 *    this.prev = prev;
 *    this.next = next;
 * };
 */
/**
 * @param {_Node} node
 * @return {number[]}
 */
var toArray = function(node) {
  const result = [];
  let headNode = node;

  while (headNode.prev) {
    headNode = headNode.prev;
  }

  let currentNode = headNode;
  while (currentNode) {
    result.push(currentNode.val);
    currentNode = currentNode.next;
  }

  return result;
};
```

3299 Sum Of Consecutive Subsequences

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3299. Sum of Consecutive Subsequences
 * https://leetcode.com/problems/sum-of-consecutive-subsequences/
 * Difficulty: Hard
 *
 * We call an array arr of length n consecutive if one of the following holds:
 * - arr[i] - arr[i - 1] == 1 for all 1 <= i < n.
 * - arr[i] - arr[i - 1] == -1 for all 1 <= i < n.
 *
 * The value of an array is the sum of its elements.
 *
 * For example, [3, 4, 5] is a consecutive array of value 12 and [9, 8] is another of
 * value 17. While [3, 4, 3] and [8, 6] are not consecutive.
 *
 * Given an array of integers nums, return the sum of the values of all consecutive
 * non-empty subsequences.
 *
```

```
 * Since the answer may be very large, return it modulo 109 + 7.
 *
 * Note that an array of length 1 is also considered consecutive.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var getSum = function(nums) {
  const MOD = 1e9 + 7;
  const singleElementSum = nums.reduce((sum, num) => (sum + num) % MOD, 0);

  return (getSubSum(nums, 1) + getSubSum(nums, -1) + singleElementSum) % MOD;

  function getSubSum(nums, direction) {
    const count = new Array(100002).fill(0);
    const sum = new Array(100002).fill(0);
    let result = 0;

    for (const num of nums) {
      const prev = num - direction;
      const sumAtNum = (sum[prev] + num * (count[prev] + 1)) % MOD;
      result = (result + sumAtNum - num + MOD) % MOD;
      sum[num] = (sum[num] + sumAtNum) % MOD;
      count[num] = (count[num] + count[prev] + 1) % MOD;
    }

    return result;
  }
};
```

3304 Find The K Th Character In String Game I

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3304. Find the K-th Character in String Game I
 * https://leetcode.com/problems/find-the-k-th-character-in-string-game-i/
 * Difficulty: Easy
 *
 * Alice and Bob are playing a game. Initially, Alice has a string word = "a".
 *
 * You are given a positive integer k.
 *
 * Now Bob will ask Alice to perform the following operation forever:
 * - Generate a new string by changing each character in word to its next character in the
 *   English alphabet, and append it to the original word.
 *
 * For example, performing the operation on "c" generates "cd" and performing the operation
 * on "zb" generates "zbac".
 *
 * Return the value of the kth character in word, after enough operations have been done for
 * word to have at least k characters.
 *
 * Note that the character 'z' can be changed to 'a' in the operation.
 */

/**
 * @param {number} k
 * @return {character}
 */
var kthCharacter = function(k) {
  let word = 'a';

  while (word.length < k) {
    let nextPart = '';
    for (const char of word) {
      nextPart += String.fromCharCode(((char.charCodeAt(0) - 97 + 1) % 26) + 97);
    }
```

```
    word += nextPart;
  }

  return word[k - 1];
};
```

3306 Count Of Substrings Containing Every Vowel And K Consonants Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3306. Count of Substrings Containing Every Vowel and K Consonants II
 *
https://leetcode.com/problems/count-of-substrings-containing-every-vowel-and-k-consonants-ii/
 * Difficulty: Medium
 *
 * You are given a string word and a non-negative integer k.
 *
 * Return the total number of substrings of word that contain every vowel ('a', 'e', 'i', 'o',
 and
 * 'u') at least once and exactly k consonants.
 */

/**
 * @param {string} word
 * @param {number} k
 * @return {number}
 */
function countOfSubstrings(word, k) {
  return helper(word, k) - helper(word, k - 1);

  function helper(word, k) {
    const vowels = { a: 0, e: 0, i: 0, o: 0, u: 0 };
    let count = 0;
    let left = 0;
    let consonants = k;

    for (let right = 0; right < word.length; right++) {
      const character = word[right];
      if (character in vowels) {
        vowels[character]++;
      } else {
        consonants--;
      }

      while (vowels.a > 0 && vowels.e > 0 && vowels.i > 0 && vowels.o > 0
             && vowels.u > 0 && consonants < 0) {
        const leftCharacter = word[left];
        if (leftCharacter in vowels) {
          vowels[leftCharacter]--;
        } else {
          consonants++;
        }
        left++;
      }
      count += right - left + 1;
    }
    return count;
  }
}
```

3307 Find The K Th Character In String Game Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3307. Find the K-th Character in String Game II
 * https://leetcode.com/problems/find-the-k-th-character-in-string-game-ii/
 * Difficulty: Hard
 *
 * Alice and Bob are playing a game. Initially, Alice has a string word = "a".
 *
 * You are given a positive integer k. You are also given an integer array operations,
 * where operations[i] represents the type of the ith operation.
 *
 * Now Bob will ask Alice to perform all operations in sequence:
 * - If operations[i] == 0, append a copy of word to itself.
 * - If operations[i] == 1, generate a new string by changing each character in word to
 *   its next character in the English alphabet, and append it to the original word.
 *   For example, performing the operation on "c" generates "cd" and performing the
 *   operation on "zb" generates "zbac".
 *
 * Return the value of the kth character in word after performing all the operations.
 *
 * Note that the character 'z' can be changed to 'a' in the second type of operation.
 */

/**
 * @param {number} k
 * @param {number[]} operations
 * @return {character}
 */
var kthCharacter = function(k, operations) {
  const validOperations = [];
  let length = 1;

  for (const op of operations) {
    if (length >= k) break;
    validOperations.push(op);
    length *= 2;
  }

  return findCharacter(k, validOperations.length - 1, 0);

  function findCharacter(position, operationIndex, shifts) {
    if (operationIndex < 0) {
      return String.fromCharCode(97 + (shifts % 26));
    }

    const halfLength = Math.pow(2, operationIndex);

    if (position <= halfLength) {
      return findCharacter(position, operationIndex - 1, shifts);
    } else {
      const newShifts = shifts + validOperations[operationIndex];
      return findCharacter(position - halfLength, operationIndex - 1, newShifts);
    }
  }
};
```

3313 Find The Last Marked Nodes In Tree

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3313. Find the Last Marked Nodes in Tree
 * https://leetcode.com/problems/find-the-last-marked-nodes-in-tree/
 * Difficulty: Hard
 *
 * There exists an undirected tree with n nodes numbered 0 to n - 1. You are given a 2D
 * integer array edges of length n - 1, where edges[i] = [ui, vi] indicates that there
 * is an edge between nodes ui and vi in the tree.
 *
 * Initially, all nodes are unmarked. After every second, you mark all unmarked nodes which
 * have at least one marked node adjacent to them.
```

```
 *
 * Return an array nodes where nodes[i] is the last node to get marked in the tree, if you
 * mark node i at time t = 0. If nodes[i] has multiple answers for any node i, you can
 * choose any one answer.
 */

/**
 * @param {number[][]} edges
 * @return {number[]}
 */
var lastMarkedNodes = function(edges) {
  const n = edges.length + 1;
  const tree = new Array(n).fill().map(() => []);

  for (const [u, v] of edges) {
    tree[u].push(v);
    tree[v].push(u);
  }

  const diameterEndpointA = findFarthestNode(0);
  const diameterEndpointB = findFarthestNode(diameterEndpointA);
  const distancesFromA = getDistances(diameterEndpointA);
  const distancesFromB = getDistances(diameterEndpointB);
  const result = [];

  for (let i = 0; i < n; i++) {
    result.push(distancesFromA[i] > distancesFromB[i] ? diameterEndpointA :
diameterEndpointB);
  }

  return result;

  function findFarthestNode(startNode) {
    const visited = new Array(n).fill(false);
    visited[startNode] = true;
    const stack = [[startNode, 0]];
    let maxSteps = 0;
    let farthestNode = startNode;

    while (stack.length > 0) {
      const [node, steps] = stack.pop();

      if (steps > maxSteps) {
        farthestNode = node;
        maxSteps = steps;
      }

      for (const neighbor of tree[node]) {
        if (!visited[neighbor]) {
          visited[neighbor] = true;
          stack.push([neighbor, steps + 1]);
        }
      }
    }

    return farthestNode;
  }

  function getDistances(startNode) {
    const visited = new Array(n).fill(false);
    visited[startNode] = true;
    const stack = [[startNode, 0]];
    const distances = new Array(n);

    while (stack.length > 0) {
      const [node, steps] = stack.pop();
      distances[node] = steps;

      for (const neighbor of tree[node]) {
        if (!visited[neighbor]) {
          visited[neighbor] = true;
          stack.push([neighbor, steps + 1]);
        }
      }
    }
```

```
    return distances;
  }
};
```

3318 Find X Sum Of All K Long Subarrays I

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3318. Find X-Sum of All K-Long Subarrays I
 * https://leetcode.com/problems/find-x-sum-of-all-k-long-subarrays-i/
 * Difficulty: Easy
 *
 * You are given an array nums of n integers and two integers k and x.
 *
 * The x-sum of an array is calculated by the following procedure:
 * - Count the occurrences of all elements in the array.
 * - Keep only the occurrences of the top x most frequent elements. If two elements
 *   have the same number of occurrences, the element with the bigger value is
 *   considered more frequent.
 * - Calculate the sum of the resulting array.
 *
 * Note that if an array has less than x distinct elements, its x-sum is the sum of the array.
 *
 Return an integer array answer of length n - k + 1 where answer[i] is the x-sum of the
 subarray nums[i..i + k - 1].
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @param {number} x
 * @return {number[]}
 */
var findXSum = function(nums, k, x) {
  const n = nums.length;
  const result = [];

  for (let i = 0; i <= n - k; i++) {
    const subarray = nums.slice(i, i + k);
    result.push(helper(subarray));
  }

  return result;

  function helper(subarray) {
    const frequency = new Map();

    for (const num of subarray) {
      frequency.set(num, (frequency.get(num) || 0) + 1);
    }

    const elements = Array.from(frequency.entries());
    elements.sort((a, b) => {
      if (a[1] !== b[1]) return b[1] - a[1];
      return b[0] - a[0];
    });

    const topX = elements.slice(0, x);
    let sum = 0;

    for (const [value, count] of topX) {
      sum += value * count;
    }

    return sum;
  }
};
```

3323 Minimize Connected Groups By Inserting Interval

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3323. Minimize Connected Groups by Inserting Interval
 * https://leetcode.com/problems/minimize-connected-groups-by-inserting-interval/
 * Difficulty: Medium
 *
 * You are given a 2D array intervals, where intervals[i] = [starti, endi] represents the
 * start and the end of interval i. You are also given an integer k.
 *
 * You must add exactly one new interval [startnew, endnew] to the array such that:
 * - The length of the new interval, endnew - startnew, is at most k.
 * - After adding, the number of connected groups in intervals is minimized.
 *
 * A connected group of intervals is a maximal collection of intervals that, when considered
 * together, cover a continuous range from the smallest point to the largest point with no
gaps
 * between them. Here are some examples:
 * - A group of intervals [[1, 2], [2, 5], [3, 3]] is connected because together they cover
the
 *   range from 1 to 5 without any gaps.
 * - However, a group of intervals [[1, 2], [3, 4]] is not connected because the segment (2,
3)
 *   is not covered.
 *
 * Return the minimum number of connected groups after adding exactly one new interval to the
array.
 */

/**
 * @param {number[][]} intervals
 * @param {number} k
 * @return {number}
 */
var minConnectedGroups = function(intervals, k) {
  intervals.sort((a, b) => a[0] - b[0]);

  const groupStarts = [intervals[0][0]];
  const groupEnds = [intervals[0][1]];

  for (let i = 1; i < intervals.length; i++) {
    const [start, end] = intervals[i];

    if (start > groupEnds[groupEnds.length - 1]) {
      groupStarts.push(start);
      groupEnds.push(end);
    } else if (start <= groupEnds[groupEnds.length - 1] && groupEnds[groupEnds.length - 1] <
end) {
      groupEnds[groupEnds.length - 1] = end;
    }
  }

  let maxGroupsToMerge = 0;
  for (let i = 0; i < groupEnds.length; i++) {
    const searchTarget = groupEnds[i] + k + 1;
    const rightmostIndex = binarySearchLeft(groupStarts, searchTarget) - 1;
    const groupsCanReach = rightmostIndex - i;
    maxGroupsToMerge = Math.max(maxGroupsToMerge, groupsCanReach);
  }

  return groupStarts.length - maxGroupsToMerge;

  function binarySearchLeft(arr, target) {
    let left = 0;
    let right = arr.length;

    while (left < right) {
```

```
      const mid = Math.floor((left + right) / 2);
      if (arr[mid] < target) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }

    return left;
  }
};
```

3329 Count Substrings With K Frequency Characters Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3329. Count Substrings With K-Frequency Characters II
 * https://leetcode.com/problems/count-substrings-with-k-frequency-characters-ii/
 * Difficulty: Hard
 *
 * Given a string s and an integer k, return the total number of substrings of s where at
 * least one character appears at least k times.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var numberOfSubstrings = function(s, k) {
  const n = s.length;
  const map = new Array(26).fill(0);
  let result = 0;
  let leftPointer = 0;

  for (let rightPointer = 0; rightPointer < n; rightPointer++) {
    const rightCharIndex = s.charCodeAt(rightPointer) - 97;
    map[rightCharIndex]++;

    while (map[rightCharIndex] >= k) {
      result += n - rightPointer;
      const leftCharIndex = s.charCodeAt(leftPointer) - 97;
      map[leftCharIndex]--;
      leftPointer++;
    }
  }

  return result;
};
```

3330 Find The Original Typed String I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3330. Find the Original Typed String I
 * https://leetcode.com/problems/find-the-original-typed-string-i/
 * Difficulty: Easy
 *
 * Alice is attempting to type a specific string on her computer. However, she tends to be
 * clumsy and may press a key for too long, resulting in a character being typed multiple
times.
 *
 * Although Alice tried to focus on her typing, she is aware that she may still have done this
```

```
 * at most once.
 *
 * You are given a string word, which represents the final output displayed on Alice's screen.
 *
 * Return the total number of possible original strings that Alice might have intended to
type.
 */

/**
 * @param {string} word
 * @return {number}
 */
var possibleStringCount = function(word) {
  let result = 1;
  let consecutiveCount = 1;

  for (let i = 1; i < word.length; i++) {
    if (word[i] === word[i - 1]) {
      consecutiveCount++;
    } else {
      if (consecutiveCount > 1) {
        result += consecutiveCount - 1;
      }
      consecutiveCount = 1;
    }
  }

  if (consecutiveCount > 1) {
    result += consecutiveCount - 1;
  }

  return result;
};
```

3333 Find The Original Typed String Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3333. Find the Original Typed String II
 * https://leetcode.com/problems/find-the-original-typed-string-ii/
 * Difficulty: Hard
 *
 * Alice is attempting to type a specific string on her computer. However, she tends to be
 * clumsy and may press a key for too long, resulting in a character being typed multiple
times.
 *
 * You are given a string word, which represents the final output displayed on Alice's screen.
 * You are also given a positive integer k.
 *
 * Return the total number of possible original strings that Alice might have intended to
type,
 * if she was trying to type a string of size at least k.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {string} word
 * @param {number} k
 * @return {number}
 */
var possibleStringCount = function(word, k) {
  const MOD = 1e9 + 7;
  const groups = [];
  let count = 1;

  for (let i = 1; i < word.length; i++) {
    if (word[i] === word[i - 1]) {
      count++;
```

```
    } else {
      groups.push(count);
      count = 1;
    }
  }
  groups.push(count);

  const n = groups.length;

  let totalWays = 1;
  for (const groupSize of groups) {
    totalWays = (totalWays * groupSize) % MOD;
  }

  if (k <= n) {
    return totalWays;
  }

  let dp = new Array(k).fill(0);
  dp[0] = 1;

  for (let i = 0; i < n; i++) {
    const newDp = new Array(k).fill(0);
    const groupSize = groups[i];

    let sum = 0;
    for (let j = 0; j < Math.min(groupSize, k); j++) {
      sum = (sum + dp[j]) % MOD;
      newDp[j + 1] = sum;
    }

    for (let j = groupSize; j < k; j++) {
      sum = (sum + dp[j] - dp[j - groupSize] + MOD) % MOD;
      if (j + 1 < k) {
        newDp[j + 1] = sum;
      }
    }

    dp = newDp;
  }

  let invalidWays = 0;
  for (let j = 0; j < k; j++) {
    invalidWays = (invalidWays + dp[j]) % MOD;
  }

  return (totalWays - invalidWays + MOD) % MOD;
};
```

3335 Total Characters In String After Transformations I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3335. Total Characters in String After Transformations I
 * https://leetcode.com/problems/total-characters-in-string-after-transformations-i/
 * Difficulty: Medium
 *
 * You are given a string s and an integer t, representing the number of transformations to
 * perform. In one transformation, every character in s is replaced according to the following
 * rules:
 * - If the character is 'z', replace it with the string "ab".
 * - Otherwise, replace it with the next character in the alphabet. For example, 'a' is
replaced
 *   with 'b', 'b' is replaced with 'c', and so on.
 *
 * Return the length of the resulting string after exactly t transformations.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */
```

```
/**
 * @param {string} s
 * @param {number} t
 * @return {number}
 */
var lengthAfterTransformations = function(s, t) {
  const MOD = 1e9 + 7;
  const freq = new Array(26).fill(0);

  for (const char of s) {
    freq[char.charCodeAt(0) - 97]++;
  }

  for (let i = 0; i < t; i++) {
    const newFreq = new Array(26).fill(0);
    for (let j = 0; j < 25; j++) {
      newFreq[j + 1] = freq[j];
    }
    newFreq[0] = (newFreq[0] + freq[25]) % MOD;
    newFreq[1] = (newFreq[1] + freq[25]) % MOD;
    freq.splice(0, 26, ...newFreq);
  }

  let result = 0;
  for (const count of freq) {
    result = (result + count) % MOD;
  }

  return result;
};
```

3337 Total Characters In String After Transformations Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3337. Total Characters in String After Transformations II
 * https://leetcode.com/problems/total-characters-in-string-after-transformations-ii/
 * Difficulty: Hard
 *
 * You are given a string s consisting of lowercase English letters, an integer t representing
 * the number of transformations to perform, and an array nums of size 26. In one
transformation,
 * every character in s is replaced according to the following rules:
 * - Replace s[i] with the next nums[s[i] - 'a'] consecutive characters in the alphabet. For
 *   example, if s[i] = 'a' and nums[0] = 3, the character 'a' transforms into the next 3
 *   consecutive characters ahead of it, which results in "bcd".
 * - The transformation wraps around the alphabet if it exceeds 'z'. For example, if s[i] =
'y'
 *   and nums[24] = 3, the character 'y' transforms into the next 3 consecutive characters
ahead
 *   of it, which results in "zab".
 *
 * Return the length of the resulting string after exactly t transformations.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {string} s
 * @param {number} t
 * @param {number[]} nums
 * @return {number}
 */
var lengthAfterTransformations = function(s, t, nums) {
  const MOD = 1000000007n;
  const freq = new Array(26).fill(0n);

  for (const char of s) {
```

```
    freq[char.charCodeAt(0) - 97]++;
  }

  const matrix = new Array(26).fill().map(() => new Array(26).fill(0n));
  for (let i = 0; i < 26; i++) {
    for (let j = 0; j < nums[i]; j++) {
      matrix[(i + j + 1) % 26][i] = 1n;
    }
  }

  const finalMatrix = matrixPower(matrix, t);
  let total = 0n;

  for (let i = 0; i < 26; i++) {
    let sum = 0n;
    for (let j = 0; j < 26; j++) {
      sum = (sum + finalMatrix[i][j] * freq[j]) % MOD;
    }
    total = (total + sum) % MOD;
  }

  return Number(total);

  function matrixMultiply(a, b) {
    const result = new Array(26).fill().map(() => new Array(26).fill(0n));
    for (let i = 0; i < 26; i++) {
      for (let j = 0; j < 26; j++) {
        for (let k = 0; k < 26; k++) {
          result[i][j] = (result[i][j] + a[i][k] * b[k][j]) % MOD;
        }
      }
    }
    return result;
  }

  function matrixPower(mat, power) {
    let result = new Array(26).fill().map(() => new Array(26).fill(0n));
    for (let i = 0; i < 26; i++) result[i][i] = 1n;

    while (power > 0) {
      if (power & 1) result = matrixMultiply(result, mat);
      mat = matrixMultiply(mat, mat);
      power >>= 1;
    }
    return result;
  }
};
```

3339 Find The Number Of K Even Arrays

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3339. Find the Number of K-Even Arrays
 * https://leetcode.com/problems/find-the-number-of-k-even-arrays/
 * Difficulty: Medium
 *
 * You are given three integers n, m, and k.
 *
 * An array arr is called k-even if there are exactly k indices such that, for each
 * of these indices i (0 <= i < n - 1):
 * - (arr[i] * arr[i + 1]) - arr[i] - arr[i + 1] is even.
 *
 * Return the number of possible k-even arrays of size n where all elements are in the range
[1, m].
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
```

```
 * @param {number} n
 * @param {number} m
 * @param {number} k
 * @return {number}
 */
var countOfArrays = function(n, m, k) {
  const MOD = 1e9 + 7;
  const oddCount = Math.floor((m + 1) / 2);
  const evenCount = Math.floor(m / 2);
  const dpOdd = new Array(k + 1).fill(0);
  const dpEven = new Array(k + 1).fill(0);

  dpOdd[0] = oddCount;
  dpEven[0] = evenCount;

  for (let position = 1; position < n; position++) {
    for (let evenPairs = k; evenPairs >= 0; evenPairs--) {
      const tempOdd = dpOdd[evenPairs];
      dpOdd[evenPairs] = ((dpEven[evenPairs] + dpOdd[evenPairs]) * oddCount) % MOD;
      if (evenPairs > 0) {
        dpEven[evenPairs] = ((dpEven[evenPairs - 1] + tempOdd) * evenCount) % MOD;
      } else {
        dpEven[evenPairs] = (tempOdd * evenCount) % MOD;
      }
    }
  }

  return (dpOdd[k] + dpEven[k]) % MOD;
};
```

3341 Find Minimum Time To Reach Last Room I

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3341. Find Minimum Time to Reach Last Room I
 * https://leetcode.com/problems/find-minimum-time-to-reach-last-room-i/
 * Difficulty: Medium
 *
 * There is a dungeon with n x m rooms arranged as a grid.
 *
 * You are given a 2D array moveTime of size n x m, where moveTime[i][j] represents the
minimum
 * time in seconds when you can start moving to that room. You start from the room (0, 0) at
 * time t = 0 and can move to an adjacent room. Moving between adjacent rooms takes exactly
 * one second.
 *
 * Return the minimum time to reach the room (n - 1, m - 1).
 *
 * Two rooms are adjacent if they share a common wall, either horizontally or vertically.
 */

/**
 * @param {number[][]} moveTime
 * @return {number}
 */
var minTimeToReach = function(moveTime) {
  const rows = moveTime.length;
  const cols = moveTime[0].length;
  const distances = Array.from({ length: rows }, () => new Array(cols).fill(Infinity));
  const pq = new PriorityQueue((a, b) => a[0] - b[0]);
  pq.enqueue([0, 0, 0]);
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  distances[0][0] = 0;

  while (!pq.isEmpty()) {
    const [time, row, col] = pq.dequeue();

    if (row === rows - 1 && col === cols - 1) return time;
```

```
    if (time > distances[row][col]) continue;

    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;

      if (newRow < 0 || newRow >= rows || newCol < 0 || newCol >= cols) continue;

      const newTime = Math.max(time, moveTime[newRow][newCol]) + 1;

      if (newTime < distances[newRow][newCol]) {
        distances[newRow][newCol] = newTime;
        pq.enqueue([newTime, newRow, newCol]);
      }
    }
  }

  return distances[rows - 1][cols - 1];
};
```

3342 Find Minimum Time To Reach Last Room Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3342. Find Minimum Time to Reach Last Room II
 * https://leetcode.com/problems/find-minimum-time-to-reach-last-room-ii/
 * Difficulty: Medium
 *
 * There is a dungeon with n x m rooms arranged as a grid.
 *
 * You are given a 2D array moveTime of size n x m, where moveTime[i][j] represents the
minimum
 * time in seconds when you can start moving to that room. You start from the room (0, 0) at
 * time t = 0 and can move to an adjacent room. Moving between adjacent rooms takes one
 * second for one move and two seconds for the next, alternating between the two.
 *
 * Return the minimum time to reach the room (n - 1, m - 1).
 *
 * Two rooms are adjacent if they share a common wall, either horizontally or vertically.
 */

/**
 * @param {number[][]} moveTime
 * @return {number}
 */
var minTimeToReach = function(moveTime) {
  const rows = moveTime.length;
  const cols = moveTime[0].length;
  const distances = Array.from({ length: rows }, () => new Array(cols).fill(Infinity));
  const pq = new PriorityQueue((a, b) => a[0] - b[0]);
  pq.enqueue([0, 0, 0, 0]);
  const directions = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  distances[0][0] = 0;

  while (!pq.isEmpty()) {
    const [time, row, col, moveCount] = pq.dequeue();

    if (row === rows - 1 && col === cols - 1) return time;

    if (time > distances[row][col]) continue;

    for (const [dr, dc] of directions) {
      const newRow = row + dr;
      const newCol = col + dc;

      if (newRow < 0 || newRow >= rows || newCol < 0 || newCol >= cols) continue;
```

```
      const moveCost = moveCount % 2 === 0 ? 1 : 2;
      const startTime = Math.max(time, moveTime[newRow][newCol]);
      const newTime = startTime + moveCost;

      if (newTime < distances[newRow][newCol]) {
        distances[newRow][newCol] = newTime;
        pq.enqueue([newTime, newRow, newCol, moveCount + 1]);
      }
    }
  }

  return distances[rows - 1][cols - 1];
};
```

3343 Count Number Of Balanced Permutations

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3343. Count Number of Balanced Permutations
 * https://leetcode.com/problems/count-number-of-balanced-permutations/
 * Difficulty: Hard
 *
 * You are given a string num. A string of digits is called balanced if the sum of the digits
 * at even indices is equal to the sum of the digits at odd indices.
 *
 * Return the number of distinct permutations of num that are balanced.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 *
 * A permutation is a rearrangement of all the characters of a string.
 */

/**
 * @param {string} num
 * @return {number}
 */
var countBalancedPermutations = function(num) {
  const MOD = 1e9 + 7;
  const n = num.length;
  const digitCounts = new Array(10).fill(0);
  let totalSum = 0;

  for (const char of num) {
    const digit = parseInt(char);
    digitCounts[digit]++;
    totalSum += digit;
  }

  if (totalSum % 2) return 0;

  const memo = new Map();

  function combination(n, r) {
    if (r > n) return 0;
    if (r === 0 || r === n) return 1;
    if (r > n - r) r = n - r;

    let result = 1n;
    for (let i = 0; i < r; i++) {
      result = result * BigInt(n - i) / BigInt(i + 1);
    }
    return Number(result % BigInt(MOD));
  }

  function exploreDigits(digit, oddSlots, evenSlots, targetBalance) {
    if (oddSlots === 0 && evenSlots === 0 && targetBalance === 0) return 1;
    if (digit < 0 || oddSlots < 0 || evenSlots < 0 || targetBalance < 0) return 0;

    const key = `${digit},${oddSlots},${evenSlots},${targetBalance}`;
```

```
    if (memo.has(key)) return memo.get(key);

    let result = 0;
    for (let oddCount = 0; oddCount <= digitCounts[digit]; oddCount++) {
      const evenCount = digitCounts[digit] - oddCount;
      const ways = (BigInt(combination(oddSlots, oddCount))
        * BigInt(combination(evenSlots, evenCount))) % BigInt(MOD);
      const next = BigInt(exploreDigits(digit - 1, oddSlots - oddCount,
        evenSlots - evenCount, targetBalance - digit * oddCount));
      result = (result + Number((ways * next) % BigInt(MOD))) % MOD;
    }

    memo.set(key, result);
    return result;
  }

  return exploreDigits(9, n - Math.floor(n / 2), Math.floor(n / 2), totalSum / 2);
};
```

3344 Maximum Sized Array

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3344. Maximum Sized Array
 * https://leetcode.com/problems/maximum-sized-array/
 * Difficulty: Medium
 *
 * Given a positive integer s, let A be a 3D array of dimensions n × n × n, where each
 * element A[i][j][k] is defined as:
 * - A[i][j][k] = i * (j OR k), where 0 <= i, j, k < n.
 *
 * Return the maximum possible value of n such that the sum of all elements in array A
 * does not exceed s.
 */

/**
 * @param {number} s
 * @return {number}
 */
var maxSizedArray = function(s) {
  let left = Math.floor(Math.pow(4 * s, 0.2) / 2);
  let right = Math.floor(Math.sqrt(2 * Math.sqrt(s))) + 2;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);

    if (calculateArraySum(mid) > s) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left - 1;

  function getBitOnesCount(n, bitPosition) {
    const blockSize = 1 << bitPosition;
    const cycleSize = 1 << (bitPosition + 1);
    const fullCycles = Math.floor(n / cycleSize);
    const remainder = n % cycleSize;

    return fullCycles * blockSize + (remainder > blockSize ? remainder - blockSize : 0);
  }

  function calculateArraySum(n) {
    let totalSum = 0;

    for (let bitPos = 0; (1 << bitPos) < n; bitPos++) {
      const onesCount = getBitOnesCount(n, bitPos);
```

```
      const zerosCount = n - onesCount;

      totalSum += (n * n - zerosCount * zerosCount) * (1 << bitPos);
    }

    return totalSum * (n - 1) * n / 2;
  }
};
```

3346 Maximum Frequency Of An Element After Performing Operations I

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3346. Maximum Frequency of an Element After Performing Operations I
 *
https://leetcode.com/problems/maximum-frequency-of-an-element-after-performing-operations-i/
 * Difficulty: Medium
 *
 * You are given an integer array nums and two integers k and numOperations.
 *
 * You must perform an operation numOperations times on nums, where in each operation you:
 * - Select an index i that was not selected in any previous operations.
 * - Add an integer in the range [-k, k] to nums[i].
 *
 * Return the maximum possible frequency of any element in nums after performing the
operations.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @param {number} numOperations
 * @return {number}
 */
var maxFrequency = function(nums, k, numOperations) {
  const n = nums.length;
  nums.sort((a, b) => a - b);

  const count = new Map();
  for (const num of nums) {
    count.set(num, (count.get(num) || 0) + 1);
  }

  let result = 0;
  let left = 0;
  let right = 0;
  for (let mid = 0; mid < n; mid++) {
    while (nums[mid] - nums[left] > k) {
      left++;
    }

    while (right < n - 1 && nums[right + 1] - nums[mid] <= k) {
      right++;
    }

    const total = right - left + 1;
    result = Math.max(
      result,
      Math.min(total - count.get(nums[mid]), numOperations) + count.get(nums[mid])
    );
  }

  left = 0;
  for (right = 0; right < n; right++) {
    let mid = Math.floor((nums[left] + nums[right]) / 2);

    while (mid - nums[left] > k || nums[right] - mid > k) {
      left++;
```

```
      mid = Math.floor((nums[left] + nums[right]) / 2);
    }

    result = Math.max(result, Math.min(right - left + 1, numOperations));
  }

  return result;
};
```

3347 Maximum Frequency Of An Element After Performing Operations Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3347. Maximum Frequency of an Element After Performing Operations II
 *
https://leetcode.com/problems/maximum-frequency-of-an-element-after-performing-operations-ii
 * Difficulty: Hard
 *
 * You are given an integer array nums and two integers k and numOperations.
 *
 * You must perform an operation numOperations times on nums, where in each operation you:
 * - Select an index i that was not selected in any previous operations.
 * - Add an integer in the range [-k, k] to nums[i].
 *
 * Return the maximum possible frequency of any element in nums after performing the
operations.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @param {number} numOperations
 * @return {number}
 */
var maxFrequency = function(nums, k, numOperations) {
  const n = nums.length;
  nums.sort((a, b) => a - b);

  const count = new Map();
  for (const num of nums) {
    count.set(num, (count.get(num) || 0) + 1);
  }

  let result = 0;
  let left = 0;
  let right = 0;
  for (let mid = 0; mid < n; mid++) {
    while (nums[mid] - nums[left] > k) {
      left++;
    }

    while (right < n - 1 && nums[right + 1] - nums[mid] <= k) {
      right++;
    }

    const total = right - left + 1;
    result = Math.max(
      result,
      Math.min(total - count.get(nums[mid]), numOperations) + count.get(nums[mid])
    );
  }

  left = 0;
  for (right = 0; right < n; right++) {
    let mid = Math.floor((nums[left] + nums[right]) / 2);

    while (mid - nums[left] > k || nums[right] - mid > k) {
      left++;
```

```
      mid = Math.floor((nums[left] + nums[right]) / 2);
    }

    result = Math.max(result, Math.min(right - left + 1, numOperations));
  }

  return result;
};
```

3349 Adjacent Increasing Subarrays Detection I

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3349. Adjacent Increasing Subarrays Detection I
 * https://leetcode.com/problems/adjacent-increasing-subarrays-detection-i/
 * Difficulty: Easy
 *
 * Given an array nums of n integers and an integer k, determine whether there exist two
 * adjacent subarrays of length k such that both subarrays are strictly increasing.
 * Specifically, check if there are two subarrays starting at indices a and b (a < b),
 * where:
 * - Both subarrays nums[a..a + k - 1] and nums[b..b + k - 1] are strictly increasing.
 * - The subarrays must be adjacent, meaning b = a + k.
 *
 * Return true if it is possible to find two such subarrays, and false otherwise.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {boolean}
 */
var hasIncreasingSubarrays = function(nums, k) {
  for (let i = 0; i <= nums.length - 2 * k; i++) {
    if (helper(i, k) && helper(i + k, k)) {
      return true;
    }
  }

  return false;

  function helper(start, length) {
    for (let i = start; i < start + length - 1; i++) {
      if (nums[i] >= nums[i + 1]) return false;
    }
    return true;
  }
};
```

3350 Adjacent Increasing Subarrays Detection Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3350. Adjacent Increasing Subarrays Detection II
 * https://leetcode.com/problems/adjacent-increasing-subarrays-detection-ii/
 * Difficulty: Medium
 *
 * Given an array nums of n integers, your task is to find the maximum value of k for which
 * there exist two adjacent subarrays of length k each, such that both subarrays are strictly
 * increasing. Specifically, check if there are two subarrays of length k starting at indices
 * a and b (a < b), where:
 * - Both subarrays nums[a..a + k - 1] and nums[b..b + k - 1] are strictly increasing.
 * - The subarrays must be adjacent, meaning b = a + k.
```

```
 *
 * Return the maximum possible value of k.
 *
 * A subarray is a contiguous non-empty sequence of elements within an array.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxIncreasingSubarrays = function(nums) {
  const n = nums.length;
  const lengths = new Array(n).fill(1);

  for (let i = n - 2; i >= 0; i--) {
    if (nums[i] < nums[i + 1]) {
      lengths[i] = lengths[i + 1] + 1;
    }
  }

  let result = 0;
  for (let i = 0; i < n; i++) {
    const currentLength = lengths[i];
    result = Math.max(result, Math.floor(currentLength / 2));

    const nextIndex = i + currentLength;
    if (nextIndex < n) {
      const minLength = Math.min(currentLength, lengths[nextIndex]);
      result = Math.max(result, minLength);
    }
  }

  return result;
};
```

3353 Minimum Total Operations

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3353. Minimum Total Operations
 * https://leetcode.com/problems/minimum-total-operations/
 * Difficulty: Easy
 *
 * Given an array of integers nums, you can perform any number of operations on this array.
 *
 * In each operation, you can:
 * - Choose a prefix of the array.
 * - Choose an integer k (which can be negative) and add k to each element in the chosen
prefix.
 *
 * A prefix of an array is a subarray that starts from the beginning of the array and extends
to
 * any point within it.
 *
 * Return the minimum number of operations required to make all elements in arr equal.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minOperations = function(nums) {
  let result = 0;

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] !== nums[i - 1]) {
      result++;
    }
  }
```

```
  return result;
};
```

3354 Make Array Elements Equal To Zero

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3354. Make Array Elements Equal to Zero
 * https://leetcode.com/problems/make-array-elements-equal-to-zero/
 * Difficulty: Easy
 *
 * You are given an integer array nums.
 *
 * Start by selecting a starting position curr such that nums[curr] == 0, and choose a
movement
 * direction of either left or right.
 *
 * After that, you repeat the following process:
 * - If curr is out of the range [0, n - 1], this process ends.
 * - If nums[curr] == 0, move in the current direction by incrementing curr if you are moving
 *   right, or decrementing curr if you are moving left.
 * - Else if nums[curr] > 0:
 *   - Decrement nums[curr] by 1.
 *   - Reverse your movement direction (left becomes right and vice versa).
 *   - Take a step in your new direction.
 *
 * A selection of the initial position curr and movement direction is considered valid if
 * every element in nums becomes 0 by the end of the process.
 *
 * Return the number of possible valid selections.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countValidSelections = function(nums) {
  const n = nums.length;
  let result = 0;

  for (let i = 0; i < n; i++) {
    if (nums[i] === 0) {
      if (helper(i, -1)) result++;
      if (helper(i, 1)) result++;
    }
  }

  return result;

  function helper(startIndex, direction) {
    const arr = [...nums];
    let current = startIndex;
    let dir = direction;

    while (current >= 0 && current < n) {
      if (arr[current] === 0) {
        current += dir;
      } else {
        arr[current]--;
        dir = -dir;
        current += dir;
      }
    }

    return arr.every(val => val === 0);
  }
};
```

3355 Zero Array Transformation I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3355. Zero Array Transformation I
 * https://leetcode.com/problems/zero-array-transformation-i/
 * Difficulty: Medium
 *
 * You are given an integer array nums of length n and a 2D array queries, where
 * queries[i] = [li, ri].
 *
 * For each queries[i]:
 * - Select a subset of indices within the range [li, ri] in nums.
 * - Decrement the values at the selected indices by 1.
 *
 * A Zero Array is an array where all elements are equal to 0.
 *
 * Return true if it is possible to transform nums into a Zero Array after processing all the
 * queries sequentially, otherwise return false.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} queries
 * @return {boolean}
 */
var isZeroArray = function(nums, queries) {
  const maxDecrements = new Array(nums.length).fill(0);

  for (const [left, right] of queries) {
    maxDecrements[left]++;
    if (right + 1 < nums.length) {
      maxDecrements[right + 1]--;
    }
  }

  let currentDecrements = 0;
  for (let i = 0; i < nums.length; i++) {
    currentDecrements += maxDecrements[i];
    if (nums[i] > currentDecrements) {
      return false;
    }
  }

  return true;
};
```

3356 Zero Array Transformation Ii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3356. Zero Array Transformation II
 * https://leetcode.com/problems/zero-array-transformation-ii/
 * Difficulty: Medium
 *
 * You are given an integer array nums of length n and a 2D array queries where
 * queries[i] = [li, ri, vali].
 *
 * Each queries[i] represents the following action on nums:
 * - Decrement the value at each index in the range [li, ri] in nums by at most vali.
 * - The amount by which each value is decremented can be chosen independently for each index.
 *
 * A Zero Array is an array with all its elements equal to 0.
```

```
 *
 * Return the minimum possible non-negative value of k, such that after processing the first
 * k queries in sequence, nums becomes a Zero Array. If no such k exists, return -1.
 */

/**
 * @param {number[]} nums
 * @param {number[][]} queries
 * @return {number}
 */
var minZeroArray = function(nums, queries) {
  const diff = new Array(nums.length + 1).fill(0);
  const total = nums.reduce((sum, num) => sum + num, 0);
  let left = 0;
  let right = queries.length - 1;
  let result = -1;

  if (total === 0) {
    return 0;
  }

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (canZeroOut(mid)) {
      result = mid + 1;
      right = mid - 1;
    } else {
      left = mid + 1;
    }
  }

  return result;

  function canZeroOut(k) {
    const tempDiff = new Array(nums.length + 1).fill(0);
    for (let i = 0; i <= k; i++) {
      const [left, right, val] = queries[i];
      tempDiff[left] += val;
      if (right + 1 < nums.length) tempDiff[right + 1] -= val;
    }

    let current = 0;
    let reduction = 0;
    for (let i = 0; i < nums.length; i++) {
      current = Math.max(0, current + tempDiff[i]);
      reduction += Math.min(nums[i], current);
      if (reduction >= total) return true;
    }
    return reduction >= total;
  }
};
```

3359 Find Sorted Submatrices With Maximum Element At Most K

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3359. Find Sorted Submatrices With Maximum Element at Most K
 * https://leetcode.com/problems/find-sorted-submatrices-with-maximum-element-at-most-k/
 * Difficulty: Hard
 *
 * You are given a 2D matrix grid of size m x n. You are also given a non-negative integer k.
 *
 * Return the number of submatrices of grid that satisfy the following conditions:
 * - The maximum element in the submatrix less than or equal to k.
 * - Each row in the submatrix is sorted in non-increasing order.
 *
 * A submatrix (x1, y1, x2, y2) is a matrix that forms by choosing all cells grid[x][y]
 * where x1 <= x <= x2 and y1 <= y <= y2.
 */
```

```
/**
 * @param {number[][]} grid
 * @param {number} k
 * @return {number}
 */
var countSubmatrices = function(grid, k) {
  const rows = grid.length;
  const cols = grid[0].length;
  const groups = new Array(cols).fill().map(() => []);
  const sums = new Array(cols).fill(0);
  let result = 0;

  for (let row = 0; row < rows; row++) {
    const rowLengths = new Array(cols).fill(0);

    for (let col = 0; col < cols; col++) {
      if (grid[row][col] <= k) {
        rowLengths[col] = (col > 0 && grid[row][col - 1] >= grid[row][col])
          ? rowLengths[col - 1] + 1
          : 1;
        let lengthCount = 1;
        const stack = groups[col];

        while (stack.length > 0 && stack[stack.length - 1][0] > rowLengths[col]) {
          const [stackLength, stackCount] = stack.pop();
          lengthCount += stackCount;
          sums[col] -= stackLength * stackCount;
        }

        if (stack.length === 0 || stack[stack.length - 1][0] !== rowLengths[col]) {
          stack.push([rowLengths[col], 0]);
        }

        stack[stack.length - 1][1] += lengthCount;
        sums[col] += rowLengths[col] * lengthCount;
        result += sums[col];
      } else {
        groups[col].length = 0;
        sums[col] = 0;
      }
    }
  }

  return result;
};
```

3362 Zero Array Transformation Iii

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3362. Zero Array Transformation III
 * https://leetcode.com/problems/zero-array-transformation-iii/
 * Difficulty: Medium
 *
 * You are given an integer array nums of length n and a 2D array queries where
 * queries[i] = [li, ri].
 *
 * Each queries[i] represents the following action on nums:
 * - Decrement the value at each index in the range [li, ri] in nums by at most 1.
 * - The amount by which the value is decremented can be chosen independently for each index.
 *
 * A Zero Array is an array with all its elements equal to 0.
 *
 * Return the maximum number of elements that can be removed from queries, such that nums can
 * still be converted to a zero array using the remaining queries. If it is not possible to
 * convert nums to a zero array, return -1.
 */
```

```
/**
 * @param {number[]} nums
 * @param {number[][]} queries
 * @return {number}
 */
var maxRemoval = function(nums, queries) {
  queries.sort((a, b) => a[0] - b[0]);
  const endIndexHeap = new MaxPriorityQueue();
  const expiredQueries = new Array(nums.length + 1).fill(0);
  let operationCount = 0;
  let queryIndex = 0;

  for (let numIndex = 0; numIndex < nums.length; numIndex++) {
    operationCount -= expiredQueries[numIndex];

    while (queryIndex < queries.length && queries[queryIndex][0] === numIndex) {
      endIndexHeap.push(queries[queryIndex][1]);
      queryIndex++;
    }

    while (
      operationCount < nums[numIndex] && !endIndexHeap.isEmpty() && endIndexHeap.front() >=
numIndex
    ) {
      operationCount++;
      expiredQueries[endIndexHeap.pop() + 1]++;
    }

    if (operationCount < nums[numIndex]) {
      return -1;
    }
  }

  return endIndexHeap.size();
};
```

3363 Find The Maximum Number Of Fruits Collected

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3363. Find the Maximum Number of Fruits Collected
 * https://leetcode.com/problems/find-the-maximum-number-of-fruits-collected/
 * Difficulty: Hard
 *
 * There is a game dungeon comprised of n x n rooms arranged in a grid.
 *
 * You are given a 2D array fruits of size n x n, where fruits[i][j] represents the number of
fruits
 * in the room (i, j). Three children will play in the game dungeon, with initial positions at
 the
 * corner rooms (0, 0), (0, n - 1), and (n - 1, 0).
 *
 * The children will make exactly n - 1 moves according to the following rules to reach the
 * room (n - 1, n - 1):
 * - The child starting from (0, 0) must move from their current room (i, j) to one of the
rooms
 *   (i + 1, j + 1), (i + 1, j), and (i, j + 1) if the target room exists.
 * - The child starting from (0, n - 1) must move from their current room (i, j) to one of the
 rooms
 *   (i + 1, j - 1), (i + 1, j), and (i + 1, j + 1) if the target room exists.
 * - The child starting from (n - 1, 0) must move from their current room (i, j) to one of the
 rooms
 *   (i - 1, j + 1), (i, j + 1), and (i + 1, j + 1) if the target room exists.
 *
 * When a child enters a room, they will collect all the fruits there. If two or more children
 * enter the same room, only one child will collect the fruits, and the room will be emptied
after
 * they leave.
```

```
 *
 * Return the maximum number of fruits the children can collect from the dungeon.
 */

/**
 * @param {number[][]} fruits
 * @return {number}
 */
var maxCollectedFruits = function(fruits) {
  const n = fruits.length;
  const dp = Array(n).fill().map(() => Array(n).fill(-1));
  const dp2 = Array(n).fill().map(() => Array(n).fill(-1));

  let diagonalSum = 0;
  for (let i = 0; i < n; i++) {
    diagonalSum += fruits[i][i];
  }

  const child2Max = solveChild2(0, n - 1);
  const child3Max = solveChild3(n - 1, 0);

  return diagonalSum + child2Max + child3Max;

  function solveChild2(row, col) {
    if (row === n - 1 && col === n - 1) return 0;
    if (row >= n || col < 0 || col >= n || row > col + 1) return -Infinity;
    if (dp[row][col] !== -1) return dp[row][col];

    const value = (row === col) ? 0 : fruits[row][col];
    let maxNext = -Infinity;

    for (const [dr, dc] of [[1, -1], [1, 0], [1, 1]]) {
      const newRow = row + dr;
      const newCol = col + dc;
      if (newRow < n && newCol >= 0 && newCol < n) {
        maxNext = Math.max(maxNext, solveChild2(newRow, newCol));
      }
    }

    return dp[row][col] = value + maxNext;
  }

  function solveChild3(row, col) {
    if (row === n - 1 && col === n - 1) return 0;
    if (row < 0 || row >= n || col >= n || row + col < n - 1) return -Infinity;
    if (dp2[row][col] !== -1) return dp2[row][col];

    const value = (row === col) ? 0 : fruits[row][col];
    let maxNext = -Infinity;

    for (const [dr, dc] of [[-1, 1], [0, 1], [1, 1]]) {
      const newRow = row + dr;
      const newCol = col + dc;
      if (newRow >= 0 && newRow < n && newCol < n) {
        maxNext = Math.max(maxNext, solveChild3(newRow, newCol));
      }
    }

    return dp2[row][col] = value + maxNext;
  }
};
```

3369 Design An Array Statistics Tracker

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3369. Design an Array Statistics Tracker
 * https://leetcode.com/problems/design-an-array-statistics-tracker/
 * Difficulty: Hard
```

```
 *
 * Design a data structure that keeps track of the values in it and answers some queries
regarding
 * their mean, median, and mode.
 *
 * Implement the StatisticsTracker class.
 * - StatisticsTracker(): Initialize the StatisticsTracker object with an empty array.
 * - void addNumber(int number): Add number to the data structure.
 * - void removeFirstAddedNumber(): Remove the earliest added number from the data structure.
 * - int getMean(): Return the floored mean of the numbers in the data structure.
 * - int getMedian(): Return the median of the numbers in the data structure.
 * - int getMode(): Return the mode of the numbers in the data structure. If there are
multiple
 *   modes, return the smallest one.
 *
 * Note:
 * - The mean of an array is the sum of all the values divided by the number of values in the
array.
 * - The median of an array is the middle element of the array when it is sorted in
non-decreasing
 *   order. If there are two choices for a median, the larger of the two values is taken.
 * - The mode of an array is the element that appears most often in the array.
 */

var StatisticsTracker = function() {
  this.deque = [];
  this.count = new Map();
  this.frequencyHeap = new PriorityQueue((a, b) => {
    if (a[0] !== b[0]) return b[0] - a[0];
    return a[1] - b[1];
  });
  this.total = 0;

  this.smallHeap = new PriorityQueue((a, b) => b - a);
  this.largeHeap = new PriorityQueue((a, b) => a - b);
  this.largeRemove = new Map();
  this.smallRemove = new Map();
  this.balance = 0;
};

/**
 * @param {number} number
 * @return {void}
 */
StatisticsTracker.prototype.addNumber = function(number) {
  this.deque.push(number);
  this.total += number;

  const currentCount = this.count.get(number) || 0;
  this.count.set(number, currentCount + 1);
  this.frequencyHeap.enqueue([this.count.get(number), number]);

  if (this.largeHeap.isEmpty() || number >= this.largeHeap.front()) {
    this.largeHeap.enqueue(number);
    this.balance += 1;
  } else {
    this.smallHeap.enqueue(number);
    this.balance -= 1;
  }

  this.keepBalance();
};

/**
 * @return {void}
 */
StatisticsTracker.prototype.removeFirstAddedNumber = function() {
  const value = this.deque.shift();

  const currentCount = this.count.get(value);
  this.count.set(value, currentCount - 1);
  if (this.count.get(value) > 0) {
    this.frequencyHeap.enqueue([this.count.get(value), value]);
  }
```

```
  this.total -= value;

  if (!this.largeHeap.isEmpty() && value >= this.largeHeap.front()) {
    this.largeRemove.set(value, (this.largeRemove.get(value) || 0) + 1);
    this.balance -= 1;
  } else {
    this.smallRemove.set(value, (this.smallRemove.get(value) || 0) + 1);
    this.balance += 1;
  }

  this.keepBalance();
};

/**
 * @return {void}
 */
StatisticsTracker.prototype.keepBalance = function() {
  if (this.balance > 1) {
    this.smallHeap.enqueue(this.largeHeap.dequeue());
    this.balance -= 2;
  }
  if (this.balance < 0) {
    this.largeHeap.enqueue(this.smallHeap.dequeue());
    this.balance += 2;
  }

  while (!this.smallHeap.isEmpty() && (this.smallRemove.get(this.smallHeap.front()) || 0) > 0)
 {
    const removed = this.smallHeap.dequeue();
    this.smallRemove.set(removed, this.smallRemove.get(removed) - 1);
  }

  while (!this.largeHeap.isEmpty() && (this.largeRemove.get(this.largeHeap.front()) || 0) > 0)
 {
    const removed = this.largeHeap.dequeue();
    this.largeRemove.set(removed, this.largeRemove.get(removed) - 1);
  }
};

/**
 * @return {number}
 */
StatisticsTracker.prototype.getMean = function() {
  return Math.floor(this.total / this.deque.length);
};

/**
 * @return {number}
 */
StatisticsTracker.prototype.getMedian = function() {
  return this.largeHeap.front();
};

/**
 * @return {number}
 */
StatisticsTracker.prototype.getMode = function() {
  while (!this.frequencyHeap.isEmpty()) {
    const [frequency, value] = this.frequencyHeap.front();
    if (this.count.get(value) === frequency) {
      return value;
    } else {
      this.frequencyHeap.dequeue();
    }
  }
};
```

3370 Smallest Number With All Set Bits

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3370. Smallest Number With All Set Bits
 * https://leetcode.com/problems/smallest-number-with-all-set-bits/
 * Difficulty: Easy
 *
 * You are given a positive number n.
 *
 * Return the smallest number x greater than or equal to n, such that the binary
 * representation of x contains only set bits
 */

/**
 * @param {number} n
 * @return {number}
 */
var smallestNumber = function(n) {
  const bits = n.toString(2).length;
  const result = (1 << bits) - 1;
  return result >= n ? result : (1 << (bits + 1)) - 1;
};
```

3372 Maximize The Number Of Target Nodes After Connecting Trees I

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3372. Maximize the Number of Target Nodes After Connecting Trees I
 * https://leetcode.com/problems/maximize-the-number-of-target-nodes-after-connecting-trees-i/
 * Difficulty: Medium
 *
 * There exist two undirected trees with n and m nodes, with distinct labels in ranges [0, n -
 1]
 * and [0, m - 1], respectively.
 *
 * You are given two 2D integer arrays edges1 and edges2 of lengths n - 1 and m - 1,
respectively,
 * where edges1[i] = [ai, bi] indicates that there is an edge between nodes ai and bi in the
first
 * tree and edges2[i] = [ui, vi] indicates that there is an edge between nodes ui and vi in
the
 * second tree. You are also given an integer k.
 *
 * Node u is target to node v if the number of edges on the path from u to v is less than or
equal
 * to k. Note that a node is always target to itself.
 *
 * Return an array of n integers answer, where answer[i] is the maximum possible number of
nodes
 * target to node i of the first tree if you have to connect one node from the first tree to
 * another node in the second tree.
 *
 * Note that queries are independent from each other. That is, for every query you will remove
 * the added edge before proceeding to the next query.
 */

/**
 * @param {number[][]} edges1
 * @param {number[][]} edges2
 * @param {number} k
 * @return {number[]}
 */
var maxTargetNodes = function(edges1, edges2, k) {
  const graph1 = buildGraph(edges1);
  const graph2 = buildGraph(edges2);
  const n = edges1.length + 1;
  const m = edges2.length + 1;
```

```
  let tree2Max = 0;
  if (k > 0) {
    tree2Max = Math.max(...Array.from({ length: m }, (_, i) =>
      countReachable(graph2, i, k - 1)
    ));
  }

  const result = [];
  for (let i = 0; i < n; i++) {
    const tree1Count = countReachable(graph1, i, k);
    result.push(tree1Count + tree2Max);
  }

  return result;

  function buildGraph(edges) {
    const graph = {};
    for (const [u, v] of edges) {
      if (!graph[u]) graph[u] = [];
      if (!graph[v]) graph[v] = [];
      graph[u].push(v);
      graph[v].push(u);
    }
    return graph;
  }

  function countReachable(graph, start, maxDist) {
    const queue = [start];
    const visited = new Set([start]);
    let count = 1;
    let dist = 0;

    while (queue.length > 0 && dist < maxDist) {
      const size = queue.length;
      dist++;
      for (let i = 0; i < size; i++) {
        const node = queue.shift();
        if (graph[node]) {
          for (const neighbor of graph[node]) {
            if (!visited.has(neighbor)) {
              visited.add(neighbor);
              queue.push(neighbor);
              count++;
            }
          }
        }
      }
    }
    return count;
  }
};
```

3373 Maximize The Number Of Target Nodes After Connecting Trees Ii

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant. Look for depth-first traversal and recursion.

SOURCE CODE

```
/**
 * 3373. Maximize the Number of Target Nodes After Connecting Trees II
 *
https://leetcode.com/problems/maximize-the-number-of-target-nodes-after-connecting-trees-ii/
 * Difficulty: Hard
 *
 * There exist two undirected trees with n and m nodes, labeled from [0, n - 1] and [0, m -
1],
 * respectively.
 *
 * You are given two 2D integer arrays edges1 and edges2 of lengths n - 1 and m - 1,
respectively,
 * where edges1[i] = [ai, bi] indicates that there is an edge between nodes ai and bi in the
first
```

```
 * tree and edges2[i] = [ui, vi] indicates that there is an edge between nodes ui and vi in
the
 * second tree.
 *
 * Node u is target to node v if the number of edges on the path from u to v is even. Note
that
 * a node is always target to itself.
 *
 * Return an array of n integers answer, where answer[i] is the maximum possible number of
nodes
 * that are target to node i of the first tree if you had to connect one node from the first
tree
 * to another node in the second tree.
 *
 * Note that queries are independent from each other. That is, for every query you will remove
 the
 * added edge before proceeding to the next query.
 */

/**
 * @param {number[][]} edges1
 * @param {number[][]} edges2
 * @return {number[]}
 */
var maxTargetNodes = function(edges1, edges2) {
  const graph1 = buildGraph(edges1);
  const graph2 = buildGraph(edges2);
  const n = graph1.length;
  const { color: color1 } = getBipartiteGroups(graph1);
  const { maxGroup: maxGroup2 } = getBipartiteGroups(graph2);
  const group0Count = color1.filter(c => c === 0).length;
  const group1Count = n - group0Count;
  const result = new Array(n);

  for (let i = 0; i < n; i++) {
    result[i] = (color1[i] === 0 ? group0Count : group1Count) + maxGroup2;
  }

  return result;

  function buildGraph(edges) {
    const n = edges.length + 1;
    const graph = Array.from({ length: n }, () => []);
    for (const [u, v] of edges) {
      graph[u].push(v);
      graph[v].push(u);
    }
    return graph;
  }

  function getBipartiteGroups(graph) {
    const n = graph.length;
    const color = new Array(n).fill(-1);
    const groups = [0, 0];

    for (let i = 0; i < n; i++) {
      if (color[i] === -1) {
        dfs(i, 0);
      }
    }

    return { color, maxGroup: Math.max(groups[0], groups[1]) };

    function dfs(node, c) {
      color[node] = c;
      groups[c]++;
      for (const neighbor of graph[node]) {
        if (color[neighbor] === -1) {
          dfs(neighbor, 1 - c);
        }
      }
    }
  }
};
```

3375 Minimum Operations To Make Array Values Equal To K

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3375. Minimum Operations to Make Array Values Equal to K
 * https://leetcode.com/problems/minimum-operations-to-make-array-values-equal-to-k/
 * Difficulty: Easy
 *
 * You are given an integer array nums and an integer k.
 *
 * An integer h is called valid if all values in the array that are strictly greater than
 * h are identical.
 *
 * For example, if nums = [10, 8, 10, 8], a valid integer is h = 9 because all nums[i] > 9
 * are equal to 10, but 5 is not a valid integer.
 *
 * You are allowed to perform the following operation on nums:
 * - Select an integer h that is valid for the current values in nums.
 * - For each index i where nums[i] > h, set nums[i] to h.
 *
 * Return the minimum number of operations required to make every element in nums equal to k.
 * If it is impossible to make all elements equal to k, return -1.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minOperations = function(nums, k) {
  if (nums.some(x => x < k)) return -1;

  const uniqueAbove = [...new Set(nums.filter(x => x > k))].sort((a, b) => b - a);
  if (uniqueAbove.length === 0) return 0;

  let result = 0;
  let current = uniqueAbove.slice();

  while (current.length > 0) {
    const threshold = current[0] - 1;
    current = current.filter(x => x <= threshold);
    result++;
    if (current.every(x => x === k)) break;
  }

  return result;
};
```

3383 Minimum Runes To Add To Cast Spell

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3383. Minimum Runes to Add to Cast Spell
 * https://leetcode.com/problems/minimum-runes-to-add-to-cast-spell/
 * Difficulty: Hard
 *
 * Alice has just graduated from wizard school, and wishes to cast a magic spell to celebrate.
 * The magic spell contains certain focus points where magic needs to be concentrated, and
some
 * of these focus points contain magic crystals which serve as the spell's energy source.
Focus
```

```
 * points can be linked through directed runes, which channel magic flow from one focus point
 * to another.
 *
 * You are given a integer n denoting the number of focus points and an array of integers
crystals
 * where crystals[i] indicates a focus point which holds a magic crystal. You are also given
two
 * integer arrays flowFrom and flowTo, which represent the existing directed runes. The ith
rune
 * allows magic to freely flow from focus point flowFrom[i] to focus point flowTo[i].
 *
 * You need to find the number of directed runes Alice must add to her spell, such that each
 * focus point either:
 * - Contains a magic crystal.
 * - Receives magic flow from another focus point.
 *
 * Return the minimum number of directed runes that she should add.
 */

/**
 * @param {number} n
 * @param {number[]} crystals
 * @param {number[]} flowFrom
 * @param {number[]} flowTo
 * @return {number}
 */
var minRunesToAdd = function(n, crystals, flowFrom, flowTo) {
  const graph = Array.from({ length: n }, () => []);
  const reverseGraph = Array.from({ length: n }, () => []);

  for (let i = 0; i < flowFrom.length; i++) {
    graph[flowFrom[i]].push(flowTo[i]);
    reverseGraph[flowTo[i]].push(flowFrom[i]);
  }

  const visited = new Array(n).fill(false);
  const finishOrder = [];

  const dfsFinish = (node) => {
    if (visited[node]) return;
    visited[node] = true;
    for (const neighbor of graph[node]) {
      dfsFinish(neighbor);
    }
    finishOrder.push(node);
  };

  for (let i = 0; i < n; i++) {
    dfsFinish(i);
  }

  const componentId = new Array(n).fill(-1);
  let componentCount = 0;

  const dfsComponent = (node, id) => {
    if (componentId[node] >= 0) return;
    componentId[node] = id;
    for (const neighbor of reverseGraph[node]) {
      dfsComponent(neighbor, id);
    }
  };

  for (let i = n - 1; i >= 0; i--) {
    const node = finishOrder[i];
    if (componentId[node] < 0) {
      dfsComponent(node, componentCount++);
    }
  }

  const hasIncoming = new Array(componentCount).fill(false);
  for (let i = 0; i < flowFrom.length; i++) {
    if (componentId[flowFrom[i]] !== componentId[flowTo[i]]) {
      hasIncoming[componentId[flowTo[i]]] = true;
    }
  }
```

```
  const hasCrystal = new Array(componentCount).fill(false);
  for (const crystal of crystals) {
    hasCrystal[componentId[crystal]] = true;
  }

  let result = 0;
  for (let i = 0; i < componentCount; i++) {
    if (!hasIncoming[i] && !hasCrystal[i]) {
      result++;
    }
  }

  return result;
};
```

3385 Minimum Time To Break Locks Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3385. Minimum Time to Break Locks II
 * https://leetcode.com/problems/minimum-time-to-break-locks-ii/
 * Difficulty: Hard
 *
 * Bob is stuck in a dungeon and must break n locks, each requiring some amount of energy
 * to break. The required energy for each lock is stored in an array called strength where
 * strength[i] indicates the energy needed to break the ith lock.
 *
 * To break a lock, Bob uses a sword with the following characteristics:
 * - The initial energy of the sword is 0.
 * - The initial factor X by which the energy of the sword increases is 1.
 * - Every minute, the energy of the sword increases by the current factor X.
 * - To break the ith lock, the energy of the sword must reach at least strength[i].
 * - After breaking a lock, the energy of the sword resets to 0, and the factor X increases by
 1.
 *
 * Your task is to determine the minimum time in minutes required for Bob to break all n locks
 * and escape the dungeon.
 *
 * Return the minimum time required for Bob to break all n locks.
 */

/**
 * @param {number[]} strength
 * @return {number}
 */
var findMinimumTime = function(strength) {
  const n = strength.length;
  const cost = Array.from({ length: n + 1 }, () => new Array(n + 1).fill(0));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      cost[i + 1][j + 1] = Math.floor((strength[i] + j) / (j + 1));
    }
  }

  const rowPotential = new Array(n + 1).fill(0);
  const colPotential = new Array(n + 1).fill(0);
  const assignment = new Array(n + 1).fill(0);
  const parent = new Array(n + 1).fill(0);

  for (let row = 1; row <= n; row++) {
    assignment[0] = row;
    let col = 0;
    const minCol = new Array(n + 1).fill(Number.MAX_SAFE_INTEGER);
    const visited = new Array(n + 1).fill(false);

    do {
      visited[col] = true;
```

```
      const currentRow = assignment[col];
      let delta = Number.MAX_SAFE_INTEGER;
      let nextCol;

      for (let j = 1; j <= n; j++) {
        if (!visited[j]) {
          const reducedCost = cost[currentRow][j] - rowPotential[currentRow] -
colPotential[j];
          if (reducedCost < minCol[j]) {
            minCol[j] = reducedCost;
            parent[j] = col;
          }
          if (minCol[j] < delta) {
            delta = minCol[j];
            nextCol = j;
          }
        }
      }

      for (let j = 0; j <= n; j++) {
        if (visited[j]) {
          rowPotential[assignment[j]] += delta;
          colPotential[j] -= delta;
        } else {
          minCol[j] -= delta;
        }
      }

      col = nextCol;
    } while (assignment[col] !== 0);

    do {
      const prevCol = parent[col];
      assignment[col] = assignment[prevCol];
      col = prevCol;
    } while (col);
  }

  return -colPotential[0];
};
```

3391 Design A 3D Binary Matrix With Efficient Layer Tracking

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3391. Design a 3D Binary Matrix with Efficient Layer Tracking
 * https://leetcode.com/problems/design-a-3d-binary-matrix-with-efficient-layer-tracking/
 * Difficulty: Medium
 *
 * You are given a n x n x n binary 3D array matrix.
 *
 * Implement the Matrix3D class:
 * - Matrix3D(int n) Initializes the object with the 3D binary array matrix, where all
elements
 *   are initially set to 0.
 * - void setCell(int x, int y, int z) Sets the value at matrix[x][y][z] to 1.
 * - void unsetCell(int x, int y, int z) Sets the value at matrix[x][y][z] to 0.
 * - int largestMatrix() Returns the index x where matrix[x] contains the most number of 1's.
 *   If there are multiple such indices, return the largest x.
 */

/**
 * @param {number} n
 */
var Matrix3D = function(n) {
  this.n = n;
  this.matrix = new Set();
  this.layerCounts = new Array(n).fill(0);
};
```

```
/**
 * @param {number} x
 * @param {number} y
 * @param {number} z
 * @return {void}
 */
Matrix3D.prototype.setCell = function(x, y, z) {
  const key = `${x},${y},${z}`;
  if (!this.matrix.has(key)) {
    this.matrix.add(key);
    this.layerCounts[x]++;
  }
};

/**
 * @param {number} x
 * @param {number} y
 * @param {number} z
 * @return {void}
 */
Matrix3D.prototype.unsetCell = function(x, y, z) {
  const key = `${x},${y},${z}`;
  if (this.matrix.has(key)) {
    this.matrix.delete(key);
    this.layerCounts[x]--;
  }
};

/**
 * @return {number}
 */
Matrix3D.prototype.largestMatrix = function() {
  const maxCount = Math.max(...this.layerCounts);
  for (let i = this.n - 1; i >= 0; i--) {
    if (this.layerCounts[i] === maxCount) {
      return i;
    }
  }
  return this.n - 1;
};
```

3392 Count Subarrays Of Length Three With A Condition

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3392. Count Subarrays of Length Three With a Condition
 * https://leetcode.com/problems/count-subarrays-of-length-three-with-a-condition/
 * Difficulty: Easy
 *
 * Given an integer array nums, return the number of subarrays of length 3 such that
 * the sum of the first and third numbers equals exactly half of the second number.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countSubarrays = function(nums) {
  let count = 0;
  for (let i = 0; i < nums.length - 2; i++) {
    if (nums[i] + nums[i + 2] === nums[i + 1] / 2) {
      count++;
    }
  }
  return count;
};
```

3394 Check If Grid Can Be Cut Into Sections

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3394. Check if Grid can be Cut into Sections
 * https://leetcode.com/problems/check-if-grid-can-be-cut-into-sections/
 * Difficulty: Medium
 *
 * You are given an integer n representing the dimensions of an n x n grid, with the origin at
 the
 * bottom-left corner of the grid. You are also given a 2D array of coordinates rectangles,
where
 * rectangles[i] is in the form [startx, starty, endx, endy], representing a rectangle on the
grid.
 * Each rectangle is defined as follows:
 * - (startx, starty): The bottom-left corner of the rectangle.
 * - (endx, endy): The top-right corner of the rectangle.
 *
 * Note that the rectangles do not overlap. Your task is to determine if it is possible to
make
 * either two horizontal or two vertical cuts on the grid such that:
 * - Each of the three resulting sections formed by the cuts contains at least one rectangle.
 * - Every rectangle belongs to exactly one section.
 *
 * Return true if such cuts can be made; otherwise, return false.
 */

/**
 * @param {number} n
 * @param {number[][]} rectangles
 * @return {boolean}
 */
var checkValidCuts = function(n, rectangles) {
  return canPartition(rectangles.map(r => [r[0], r[2]]))
    || canPartition(rectangles.map(r => [r[1], r[3]]));
};

function canPartition(intervals, cuts = 0) {
  intervals.sort(([start1, end1], [start2, end2]) => start1 - start2 || end2 - end1);
  let maxReach = intervals[0][1];
  for (let i = 1; i < intervals.length; i++) {
    if (intervals[i][0] >= maxReach && ++cuts === 2) return true;
    maxReach = Math.max(intervals[i][1], maxReach);
  }
  return false;
}
```

3396 Minimum Number Of Operations To Make Elements In Array Distinct

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3396. Minimum Number of Operations to Make Elements in Array Distinct
 *
https://leetcode.com/problems/minimum-number-of-operations-to-make-elements-in-array-distinct/
 * Difficulty: Easy
 *
 * You are given an integer array nums. You need to ensure that the elements in the array are
 * distinct. To achieve this, you can perform the following operation any number of times:
 * - Remove 3 elements from the beginning of the array. If the array has fewer than 3
elements,
 *   remove all remaining elements.
 * Note that an empty array is considered to have distinct elements. Return the minimum number
 * of operations needed to make the elements in the array distinct.
```

```
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var minimumOperations = function(nums) {
  const unique = new Set();

  for (let i = nums.length - 1; i > -1; i--) {
    if (unique.has(nums[i])) {
      return Math.ceil((i + 1) / 3);
    }
    unique.add(nums[i]);
  }

  return 0;
};
```

3397 Maximum Number Of Distinct Elements After Operations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3397. Maximum Number of Distinct Elements After Operations
 * https://leetcode.com/problems/maximum-number-of-distinct-elements-after-operations/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k.
 *
 * You are allowed to perform the following operation on each element of the array at most
once:
 * - Add an integer in the range [-k, k] to the element.
 *
 * Return the maximum possible number of distinct elements in nums after performing the
operations.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var maxDistinctElements = function(nums, k) {
  nums.sort((a, b) => a - b);
  let last = nums[0] - k;
  let count = 1;

  for (let i = 1; i < nums.length; i++) {
    if (last + 1 > nums[i] + k) continue;
    last = Math.max(last + 1, nums[i] - k);
    count++;
  }

  return count;
};
```

3400 Maximum Number Of Matching Indices After Right Shifts

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3400. Maximum Number of Matching Indices After Right Shifts
 * https://leetcode.com/problems/maximum-number-of-matching-indices-after-right-shifts/
 * Difficulty: Medium
```

```
 *
 * You are given two integer arrays, nums1 and nums2, of the same length.
 *
 * An index i is considered matching if nums1[i] == nums2[i].
 *
 * Return the maximum number of matching indices after performing any number of right shifts
 * on nums1.
 *
 * A right shift is defined as shifting the element at index i to index (i + 1) % n, for
 * all indices.
 */

/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var maximumMatchingIndices = function(nums1, nums2) {
  const n = nums1.length;
  let result = 0;

  for (let shift = 0; shift < n; shift++) {
    let currentMatches = 0;
    for (let i = 0; i < n; i++) {
      if (nums1[(i - shift + n) % n] === nums2[i]) {
        currentMatches++;
      }
    }
    result = Math.max(result, currentMatches);
  }

  return result;
};
```

3402 Minimum Operations To Make Columns Strictly Increasing

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3402. Minimum Operations to Make Columns Strictly Increasing
 * https://leetcode.com/problems/minimum-operations-to-make-columns-strictly-increasing/
 * Difficulty: Easy
 *
 * You are given a m x n matrix grid consisting of non-negative integers.
 *
 * In one operation, you can increment the value of any grid[i][j] by 1.
 *
 * Return the minimum number of operations needed to make all columns of grid strictly
increasing.
 */

/**
 * @param {number[][]} grid
 * @return {number}
 */
var minimumOperations = function(grid) {
  let count = 0;

  for (let i = 1; i < grid.length; i++) {
    for (let j = 0; j < grid[i].length; j++) {
      const [previous, current] = [grid[i - 1][j], grid[i][j]];
      if (current <= previous) {
        const operations = previous - current + 1;
        grid[i][j] = operations + current;
        count += operations;
      }
    }
  }

  return count;
```

```
};
```

3403 Find The Lexicographically Largest String From The Box I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3403. Find the Lexicographically Largest String From the Box I
 * https://leetcode.com/problems/find-the-lexicographically-largest-string-from-the-box-i/
 * Difficulty: Medium
 *
 * You are given a string word, and an integer numFriends.
 *
 * Alice is organizing a game for her numFriends friends. There are multiple rounds in the
game,
 * where in each round:
 * - word is split into numFriends non-empty strings, such that no previous round has had the
 *   exact same split.
 * - All the split words are put into a box.
 *
 * Find the lexicographically largest string from the box after all the rounds are finished.
 */

/**
 * @param {string} word
 * @param {number} numFriends
 * @return {string}
 */
var answerString = function(word, numFriends) {
  if (numFriends === 1) return word;

  let result = '';
  const wordLength = word.length;

  for (let startIndex = 0; startIndex < wordLength; startIndex++) {
    const maxLength = wordLength - numFriends + 1;
    const endIndex = Math.min(startIndex + maxLength, wordLength);
    const substring = word.slice(startIndex, endIndex);

    if (substring > result) {
      result = substring;
    }
  }

  return result;
};
```

3405 Count The Number Of Arrays With K Matching Adjacent Elements

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3405. Count the Number of Arrays with K Matching Adjacent Elements
 * https://leetcode.com/problems/count-the-number-of-arrays-with-k-matching-adjacent-elements/
 * Difficulty: Hard
 *
 * You are given three integers n, m, k. A good array arr of size n is defined as follows:
 * - Each element in arr is in the inclusive range [1, m].
 * - Exactly k indices i (where 1 <= i < n) satisfy the condition arr[i - 1] == arr[i].
 *
 * Return the number of good arrays that can be formed.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */
```

```
/**
 * @param {number} n
 * @param {number} m
 * @param {number} k
 * @return {number}
 */
var countGoodArrays = function(n, m, k) {
  const MOD = 1e9 + 7;

  if (m === 1) {
    return k === n - 1 ? 1 : 0;
  }

  const choose = binomialCoeff(n - 1, k);
  const power = modPow(m - 1, n - 1 - k, MOD);
  const result = Number((BigInt(choose) * BigInt(m) * BigInt(power)) % BigInt(MOD));

  return result;

  function modPow(base, exp, mod) {
    let result = 1n;
    base = ((BigInt(base) % BigInt(mod)) + BigInt(mod)) % BigInt(mod);
    exp = BigInt(exp);
    mod = BigInt(mod);

    while (exp > 0n) {
      if (exp & 1n) result = (result * base) % mod;
      base = (base * base) % mod;
      exp >>= 1n;
    }
    return Number(result);
  }

  function modInverse(a, mod) {
    return modPow(a, mod - 2, mod);
  }

  function binomialCoeff(n, k) {
    if (k > n || k < 0) return 0;
    if (k === 0 || k === n) return 1;

    if (k > n - k) k = n - k;

    let numerator = 1n;
    let denominator = 1n;

    for (let i = 0; i < k; i++) {
      numerator = (numerator * BigInt(n - i)) % BigInt(MOD);
      denominator = (denominator * BigInt(i + 1)) % BigInt(MOD);
    }

    const invDenom = modInverse(Number(denominator), MOD);
    return Number((numerator * BigInt(invDenom)) % BigInt(MOD));
  }
};
```

3406 Find The Lexicographically Largest String From The Box Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3406. Find the Lexicographically Largest String From the Box II
 * https://leetcode.com/problems/find-the-lexicographically-largest-string-from-the-box-ii/
 * Difficulty: Hard
 *
 * You are given a string word, and an integer numFriends.
 *
 * Alice is organizing a game for her numFriends friends. There are multiple rounds in the
game,
 * where in each round:
```

```
 * - word is split into numFriends non-empty strings, such that no previous round has had the
 *   exact same split.
 * - All the split words are put into a box.
 *
 * Find the lexicographically largest string from the box after all the rounds are finished.
 *
 * A string a is lexicographically smaller than a string b if in the first position where a
 * and b differ, string a has a letter that appears earlier in the alphabet than the
corresponding
 * letter in b.
 *
 * If the first min(a.length, b.length) characters do not differ, then the shorter string is
the
 * lexicographically smaller one.
 */

/**
 * @param {string} word
 * @param {number} numFriends
 * @return {string}
 */
var answerString = function(word, numFriends) {
  if (numFriends === 1) return word;
  const n = word.length;
  const maxLength = n - numFriends + 1;

  const pairs = [];
  for (let i = 0; i < n - 1; i++) {
    pairs.push(word[i] + word[i + 1]);
  }
  const maxPair = pairs.reduce((a, b) => a > b ? a : b);
  let result = word[n - 1];
  let index = -1;

  while (index < n) {
    index = word.indexOf(maxPair, index + 1);
    if (index === -1) break;
    const candidate = word.substring(index, index + maxLength);
    if (candidate > result) {
      result = candidate;
    }
  }

  return result;
};
```

3408 Design Task Manager

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3408. Design Task Manager
 * https://leetcode.com/problems/design-task-manager/
 * Difficulty: Medium
 *
 * There is a task management system that allows users to manage their tasks, each associated
 * with a priority. The system should efficiently handle adding, modifying, executing, and
 * removing tasks.
 *
 * Implement the TaskManager class:
 * - TaskManager(vector<vector<int>>& tasks) initializes the task manager with a list of
 *   user-task-priority triples. Each element in the input list is of the form [userId,
 *   taskId, priority], which adds a task to the specified user with the given priority.
 * - void add(int userId, int taskId, int priority) adds a task with the specified taskId
 *   and priority to the user with userId. It is guaranteed that taskId does not exist in
 *   the system.
 * - void edit(int taskId, int newPriority) updates the priority of the existing taskId to
 *   newPriority. It is guaranteed that taskId exists in the system.
 * - void rmv(int taskId) removes the task identified by taskId from the system. It is
 *   guaranteed that taskId exists in the system.
```

```
 * - int execTop() executes the task with the highest priority across all users. If there
 *   are multiple tasks with the same highest priority, execute the one with the highest
 *   taskId. After executing, the taskId is removed from the system. Return the userId
 *   associated with the executed task. If no tasks are available, return -1.
 *
 * Note that a user may be assigned multiple tasks.
 */

/**
 * @param {number[][]} tasks
 */
var TaskManager = function(tasks) {
  this.taskRegistry = {};
  this.priorityHeap = new PriorityQueue((a, b) => {
    return a.priority !== b.priority ? b.priority - a.priority : b.taskId - a.taskId;
  });

  for (const [userId, taskId, priority] of tasks) {
    this.taskRegistry[taskId] = { userId, priority, version: 1 };
    this.priorityHeap.enqueue({ taskId, priority, version: 1 });
  }
};

/**
 * @param {number} userId
 * @param {number} taskId
 * @param {number} priority
 * @return {void}
 */
TaskManager.prototype.add = function(userId, taskId, priority) {
  this.taskRegistry[taskId] = { userId, priority, version: 1 };
  this.priorityHeap.enqueue({ taskId, priority, version: 1 });
};

/**
 * @param {number} taskId
 * @param {number} newPriority
 * @return {void}
 */
TaskManager.prototype.edit = function(taskId, newPriority) {
  const currentTask = this.taskRegistry[taskId];
  currentTask.priority = newPriority;
  currentTask.version++;
  this.priorityHeap.enqueue({ taskId, priority: newPriority, version: currentTask.version });
};

/**
 * @param {number} taskId
 * @return {void}
 */
TaskManager.prototype.rmv = function(taskId) {
  delete this.taskRegistry[taskId];
};


/**
 * @return {number}
 */
TaskManager.prototype.execTop = function() {
  while (!this.priorityHeap.isEmpty()) {
    const { taskId, priority, version } = this.priorityHeap.dequeue();
    const activeTask = this.taskRegistry[taskId];

    if (activeTask && activeTask.priority === priority && activeTask.version === version) {
      const executedUserId = activeTask.userId;
      delete this.taskRegistry[taskId];
      return executedUserId;
    }
  }
  return -1;
};
```

3416 Subsequences With A Unique Middle Mode Ii

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3416. Subsequences with a Unique Middle Mode II
 * https://leetcode.com/problems/subsequences-with-a-unique-middle-mode-ii/
 * Difficulty: Hard
 *
 * Given an integer array nums, find the number of subsequences of size 5 of nums with
 * a unique middle mode.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 *
 * A mode of a sequence of numbers is defined as the element that appears the maximum number
 * of times in the sequence.
 *
 * A sequence of numbers contains a unique mode if it has only one mode.
 *
 * A sequence of numbers seq of size 5 contains a unique middle mode if the middle element
 * (seq[2]) is a unique mode.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var subsequencesWithMiddleMode = function(nums) {
  const MOD = 1e9 + 7;
  const HALF = (MOD + 1) / 2;
  const [compressedNums, uniqueCount] = compress(nums);
  const totalCounts = new Array(uniqueCount).fill(0);
  compressedNums.forEach(id => totalCounts[id]++);

  const result = countValid(compressedNums, totalCounts, true);
  compressedNums.reverse();

  return add(result, countValid(compressedNums, totalCounts, false));

  function mul(x, y) {
    return Number((BigInt(x) * BigInt(y)) % BigInt(MOD));
  }

  function add(x, y) {
    return (x + y) % MOD;
  }

  function sub(x, y) {
    return (x - y + MOD) % MOD;
  }

  function choose2(n) {
    return mul(mul(n, n - 1), HALF);
  }

  function compress(arr) {
    const valueMap = new Map();
    let nextId = 0;
    arr.forEach(val => !valueMap.has(val) && valueMap.set(val, nextId++));
    return [arr.map(val => valueMap.get(val)), nextId];
  }

  function countValid(sequence, counts, includeEqual) {
    const length = sequence.length;
    const seenCounts = new Array(uniqueCount).fill(0);
    let result = 0;
    let squareSum = 0;
    let weightedSum = 0;
    let weightedSquareSum = 0;
```

```
    counts.forEach(count => squareSum = add(squareSum, mul(count, count)));

    for (let pos = 0; pos < length; pos++) {
      const valueId = sequence[pos];
      let remaining = counts[valueId] - seenCounts[valueId];
      const newSquareSum = sub(squareSum, mul(remaining, remaining));
      const newWeightedSum = sub(weightedSum, mul(remaining, seenCounts[valueId]));
      const newWeightedSquareSum = sub(
        weightedSquareSum,
        mul(mul(remaining, remaining), seenCounts[valueId])
      );
      const suffixSize = length - pos - remaining;
      const prefixSize = pos - seenCounts[valueId];

      let contribution = mul(sub(mul(suffixSize, suffixSize), newSquareSum), prefixSize);
      contribution = sub(contribution, mul(mul(2, suffixSize), newWeightedSum));
      contribution = add(contribution, mul(2, newWeightedSquareSum));
      contribution = mul(contribution, mul(seenCounts[valueId], HALF));
      result = add(result, contribution);
      result = add(result, mul(choose2(seenCounts[valueId]), choose2(suffixSize)));

      remaining--;
      result = add(result, mul(choose2(seenCounts[valueId]), mul(remaining, suffixSize)));

      if (includeEqual) {
        result = add(result, mul(mul(seenCounts[valueId], prefixSize), mul(remaining,
suffixSize)));
        result = add(result, mul(choose2(seenCounts[valueId]), choose2(remaining)));
      }

      seenCounts[valueId]++;
      squareSum = add(newSquareSum, mul(remaining, remaining));
      weightedSum = add(newWeightedSum, mul(remaining, seenCounts[valueId]));
      weightedSquareSum = add(
        newWeightedSquareSum,
        mul(mul(remaining, remaining), seenCounts[valueId])
      );
    }

    return result;
  }
};
```

3422 Minimum Operations To Make Subarray Elements Equal

MEDIUM

APPROACH NOTES

Uses a Set for unique membership. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3422. Minimum Operations to Make Subarray Elements Equal
 * https://leetcode.com/problems/minimum-operations-to-make-subarray-elements-equal/
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. You can perform the following
operation
 * any number of times:
 * - Increase or decrease any element of nums by 1.
 *
 * Return the minimum number of operations required to ensure that at least one subarray of
size
 * k in nums has all elements equal.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minOperations = function(nums, k) {
  const n = nums.length;
```

```
  const minHeap = new PriorityQueue((a, b) => a[0] < b[0] ? -1 : 1);
  const maxHeap = new PriorityQueue((a, b) => a[0] > b[0] ? -1 : 1);
  const maxHeapSize = Math.ceil(k / 2);
  const minHeapSize = k - maxHeapSize;

  let total = 0;
  for (let i = 0; i < k; i++) {
    total += nums[i];
    maxHeap.enqueue([nums[i], i]);
  }

  let minSum = 0;
  const minHeapIndices = new Set();
  for (let i = 0; i < minHeapSize; i++) {
    const [num, idx] = maxHeap.dequeue();
    minSum += num;
    minHeap.enqueue([num, idx]);
    minHeapIndices.add(idx);
  }

  let maxSum = total - minSum;
  let median = maxHeap.front()[0];
  let result = Math.abs(median * maxHeapSize - maxSum) + Math.abs(minSum - median *
minHeapSize);

  for (let i = k; i < n; i++) {
    const num = nums[i];
    const leftOut = i - k;
    total += num - nums[leftOut];

    while (!minHeap.isEmpty() && minHeap.front()[1] <= leftOut) minHeap.dequeue();
    while (!maxHeap.isEmpty() && maxHeap.front()[1] <= leftOut) maxHeap.dequeue();

    if (minHeapIndices.has(leftOut)) {
      minHeapIndices.delete(leftOut);
      minSum -= nums[leftOut];
      maxHeap.enqueue([num, i]);
      const [newNum, newIdx] = maxHeap.dequeue();
      minSum += newNum;
      minHeap.enqueue([newNum, newIdx]);
      minHeapIndices.add(newIdx);
    } else {
      minHeap.enqueue([num, i]);
      minSum += num;
      minHeapIndices.add(i);
      const [newNum, newIdx] = minHeap.dequeue();
      minHeapIndices.delete(newIdx);
      minSum -= newNum;
      maxHeap.enqueue([newNum, newIdx]);
    }

    maxSum = total - minSum;
    while (!maxHeap.isEmpty() && maxHeap.front()[1] <= leftOut) maxHeap.dequeue();
    median = maxHeap.front()[0];
    result = Math.min(
      result,
      Math.abs(median * maxHeapSize - maxSum) + Math.abs(minSum - median * minHeapSize)
    );
  }

  return result;
};
```

3423 Maximum Difference Between Adjacent Elements In A Circular Array

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3423. Maximum Difference Between Adjacent Elements in a Circular Array
 * https://leetcode.com/problems/maximum-difference-between-adjacent-elements-in-a-circular-ar
```

```
ray/
 * Difficulty: Easy
 *
 * Given a circular array nums, find the maximum absolute difference between adjacent
elements.
 *
 * Note: In a circular array, the first and last elements are adjacent.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxAdjacentDistance = function(nums) {
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    const next = (i + 1) % nums.length;
    const diff = Math.abs(nums[i] - nums[next]);
    result = Math.max(result, diff);
  }

  return result;
};
```

3431 Minimum Unlocked Indices To Sort Nums

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3431. Minimum Unlocked Indices to Sort Nums
 * https://leetcode.com/problems/minimum-unlocked-indices-to-sort-nums/
 * Difficulty: Medium
 *
 * You are given an array nums consisting of integers between 1 and 3, and a binary array
 * locked of the same size.
 *
 * We consider nums sortable if it can be sorted using adjacent swaps, where a swap between
 * two indices i and i + 1 is allowed if nums[i] - nums[i + 1] == 1 and locked[i] == 0.
 *
 * In one operation, you can unlock any index i by setting locked[i] to 0.
 *
 * Return the minimum number of operations needed to make nums sortable. If it is not possible
 * to make nums sortable, return -1.
 */

/**
 * @param {number[]} nums
 * @param {number[]} locked
 * @return {number}
 */
var minUnlockedIndices = function(nums, locked) {
  let currentMax = 1;
  let locks = 0;
  let result = 0;

  for (let i = 0; i < nums.length; i++) {
    if (currentMax < nums[i]) {
      currentMax = nums[i];
      locks = 0;
    }
    if (nums[i] < currentMax) {
      if (nums[i] + 1 < currentMax) {
        return -1;
      }
      result += locks;
      locks = 0;
    }
    locks += locked[i];
  }
```

```
  return result;
};
```

3432 Count Partitions With Even Sum Difference

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3432. Count Partitions with Even Sum Difference
 * https://leetcode.com/problems/count-partitions-with-even-sum-difference/
 * Difficulty: Easy
 *
 * You are given an integer array nums of length n.
 *
 * A partition is defined as an index i where 0 <= i < n - 1, splitting the array into two
 * non-empty subarrays such that:
 * - Left subarray contains indices [0, i].
 * - Right subarray contains indices [i + 1, n - 1].
 *
 * Return the number of partitions where the difference between the sum of the left and right
 * subarrays is even.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var countPartitions = function(nums) {
  const total = nums.reduce((sum, n) => sum + n, 0);
  let result = 0;

  for (let i = 0, left = 0; i < nums.length - 1; i++) {
    left += nums[i];
    result += (left - total - left) % 2 === 0 ? 1 : 0;
  }

  return result;
};
```

3433 Count Mentions Per User

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3433. Count Mentions Per User
 * https://leetcode.com/problems/count-mentions-per-user/
 * Difficulty: Medium
 *
 * You are given an integer numberOfUsers representing the total number of users and an
 * array events of size n x 3.
 *
 * Each events[i] can be either of the following two types:
 * 1. Message Event: ["MESSAGE", "timestampi", "mentions_stringi"]
 *    - This event indicates that a set of users was mentioned in a message at timestampi.
 *    - The mentions_stringi string can contain one of the following tokens:
 *      - id<number>: where <number> is an integer in range [0,numberOfUsers - 1]. There can
 *        be multiple ids separated by a single whitespace and may contain duplicates. This
 *        can mention even the offline users.
 *      - ALL: mentions all users.
 *      - HERE: mentions all online users.
 * 2. Offline Event: ["OFFLINE", "timestampi", "idi"]
 * - This event indicates that the user idi had become offline at timestampi for 60 time
units.
 *    The user will automatically be online again at time timestampi + 60.
```

```
 *
 * Return an array mentions where mentions[i] represents the number of mentions the user with
 * id i has across all MESSAGE events.
 *
 * All users are initially online, and if a user goes offline or comes back online, their
status
 * change is processed before handling any message event that occurs at the same timestamp.
 *
 * Note that a user can be mentioned multiple times in a single message event, and each
mention
 * should be counted separately.
 */

/**
 * @param {number} numberOfUsers
 * @param {string[][]} events
 * @return {number[]}
 */
var countMentions = function(numberOfUsers, events) {
  const result = new Array(numberOfUsers).fill(0);
  const onlineAt = new Array(numberOfUsers).fill(0);

  events.sort((a, b) => {
    const timeCompare = parseInt(a[1], 10) - parseInt(b[1], 10);
    if (timeCompare !== 0) return timeCompare;
    return a[0] === 'MESSAGE' ? 1 : -1;
  });

  for (const [type, timestampStr, data] of events) {
    const timestamp = parseInt(timestampStr, 10);

    if (type === 'OFFLINE') {
      const userId = parseInt(data, 10);
      onlineAt[userId] = timestamp + 60;
    } else {
      if (data === 'ALL') {
        for (let i = 0; i < numberOfUsers; i++) {
          result[i]++;
        }
      } else if (data === 'HERE') {
        for (let i = 0; i < numberOfUsers; i++) {
          if (onlineAt[i] <= timestamp) {
            result[i]++;
          }
        }
      } else {
        const ids = data.replace(/id/g, '').split(' ');
        for (const id of ids) {
          result[parseInt(id, 10)]++;
        }
      }
    }
  }

  return result;
};
```

3437 Permutations Iii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3437. Permutations III
 * https://leetcode.com/problems/permutations-iii/
 * Difficulty: Medium
 *
 * Given an integer n, an alternating permutation is a permutation of the first n positive
 * integers such that no two adjacent elements are both odd or both even.
 *
 * Return all such alternating permutations sorted in lexicographical order.
```

```
 */

/**
 * @param {number} n
 * @return {number[][]}
 */
var permute = function(n) {
  const result = [];
  const current = [];
  const used = new Array(n + 1).fill(false);

  backtrack(current, used, result, n);

  return result;

  function backtrack(current, used, result, n) {
    if (current.length === n) {
      result.push([...current]);
      return;
    }

    for (let num = 1; num <= n; num++) {
      if (used[num]) continue;

      if (current.length > 0) {
        const lastNum = current[current.length - 1];
        if ((lastNum % 2 === 0 && num % 2 === 0) || (lastNum % 2 === 1 && num % 2 === 1)) {
          continue;
        }
      }

      current.push(num);
      used[num] = true;
      backtrack(current, used, result, n);
      current.pop();
      used[num] = false;
    }
  }
};
```

3439 Reschedule Meetings For Maximum Free Time I

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3439. Reschedule Meetings for Maximum Free Time I
 * https://leetcode.com/problems/reschedule-meetings-for-maximum-free-time-i/
 * Difficulty: Medium
 *
 * You are given an integer eventTime denoting the duration of an event, where the event
occurs
 * from time t = 0 to time t = eventTime.
 *
 * You are also given two integer arrays startTime and endTime, each of length n. These
represent
 * the start and end time of n non-overlapping meetings, where the ith meeting occurs during
the
 * time [startTime[i], endTime[i]].
 *
 * You can reschedule at most k meetings by moving their start time while maintaining the same
 * duration, to maximize the longest continuous period of free time during the event.
 *
 * The relative order of all the meetings should stay the same and they should remain
 * non-overlapping.
 *
 * Return the maximum amount of free time possible after rearranging the meetings.
 *
 * Note that the meetings can not be rescheduled to a time outside the event.
 */
```

```
/**
 * @param {number} eventTime
 * @param {number} k
 * @param {number[]} startTime
 * @param {number[]} endTime
 * @return {number}
 */
var maxFreeTime = function(eventTime, k, startTime, endTime) {
  const n = startTime.length;
  const gaps = new Array(n + 1);

  gaps[0] = startTime[0];
  gaps[n] = eventTime - endTime[n - 1];
  for (let i = 1; i < n; i++) {
    gaps[i] = startTime[i] - endTime[i - 1];
  }

  let window = 0;
  for (let i = 0; i <= k; i++) {
    window += gaps[i];
  }

  let result = window;
  for (let i = k + 1; i <= n; i++) {
    window += gaps[i] - gaps[i - (k + 1)];
    result = Math.max(result, window);
  }

  return result;
};
```

3440 Reschedule Meetings For Maximum Free Time Ii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3440. Reschedule Meetings for Maximum Free Time II
 * https://leetcode.com/problems/reschedule-meetings-for-maximum-free-time-ii/
 * Difficulty: Medium
 *
 * You are given an integer eventTime denoting the duration of an event. You are also given
two
 * integer arrays startTime and endTime, each of length n.
 *
 * These represent the start and end times of n non-overlapping meetings that occur during the
 * event between time t = 0 and time t = eventTime, where the ith meeting occurs during the
time
 * [startTime[i], endTime[i]].
 *
 * You can reschedule at most one meeting by moving its start time while maintaining the same
 * duration, such that the meetings remain non-overlapping, to maximize the longest continuous
 * period of free time during the event.
 *
 * Return the maximum amount of free time possible after rearranging the meetings.
 *
 * Note that the meetings can not be rescheduled to a time outside the event and they should
 * remain non-overlapping.
 *
 * Note: In this version, it is valid for the relative ordering of the meetings to change
after
 * rescheduling one meeting.
 */

/**
 * @param {number} eventTime
 * @param {number[]} startTime
 * @param {number[]} endTime
 * @return {number}
 */
var maxFreeTime = function(eventTime, startTime, endTime) {
```

```
  const gap = [startTime[0]];
  for (let i = 1; i < startTime.length; i++) {
    gap.push(startTime[i] - endTime[i - 1]);
  }
  gap.push(eventTime - endTime[endTime.length - 1]);

  const largestRight = new Array(gap.length).fill(0);
  for (let i = gap.length - 2; i >= 0; i--) {
    largestRight[i] = Math.max(largestRight[i + 1], gap[i + 1]);
  }

  let result = 0;
  let largestLeft = 0;

  for (let i = 1; i < gap.length; i++) {
    const currentGap = endTime[i - 1] - startTime[i - 1];
    if (currentGap <= Math.max(largestLeft, largestRight[i])) {
      result = Math.max(result, gap[i - 1] + gap[i] + currentGap);
    }
    result = Math.max(result, gap[i - 1] + gap[i]);
    largestLeft = Math.max(largestLeft, gap[i - 1]);
  }

  return result;
};
```

3442 Maximum Difference Between Even And Odd Frequency I

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3442. Maximum Difference Between Even and Odd Frequency I
 * https://leetcode.com/problems/maximum-difference-between-even-and-odd-frequency-i/
 * Difficulty: Easy
 *
 * You are given a string s consisting of lowercase English letters.
 *
 * Your task is to find the maximum difference diff = freq(a1) - freq(a2) between the
frequency
 * of characters a1 and a2 in the string such that:
 * - a1 has an odd frequency in the string.
 * - a2 has an even frequency in the string.
 *
 * Return this maximum difference.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxDifference = function(s) {
  const map = new Map();

  for (const char of s) {
    map.set(char, (map.get(char) || 0) + 1);
  }

  let maxOdd = 0;
  let minEven = Infinity;

  for (const freq of map.values()) {
    if (freq % 2 === 1) {
      maxOdd = Math.max(maxOdd, freq);
    } else {
      minEven = Math.min(minEven, freq);
    }
  }

  return maxOdd - minEven;
};
```

3443 Maximum Manhattan Distance After K Changes

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3443. Maximum Manhattan Distance After K Changes
 * https://leetcode.com/problems/maximum-manhattan-distance-after-k-changes/
 * Difficulty: Medium
 *
 * You are given a string s consisting of the characters 'N', 'S', 'E', and 'W', where s[i]
 * indicates movements in an infinite grid:
 * - 'N' : Move north by 1 unit.
 * - 'S' : Move south by 1 unit.
 * - 'E' : Move east by 1 unit.
 * - 'W' : Move west by 1 unit.
 *
 * Initially, you are at the origin (0, 0). You can change at most k characters to any of the
 * four directions.
 *
 * Find the maximum Manhattan distance from the origin that can be achieved at any time while
 * performing the movements in order.
 *
 * The Manhattan Distance between two cells (xi, yi) and (xj, yj) is |xi - xj| + |yi - yj|.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var maxDistance = function(s, k) {
  const directions = {
    'N': [0, 1],
    'S': [0, -1],
    'E': [1, 0],
    'W': [-1, 0]
  };

  let x = 0;
  let y = 0;
  let result = 0;

  for (let i = 0; i < s.length; i++) {
    const [dx, dy] = directions[s[i]];
    x += dx;
    y += dy;

    const currentDistance = Math.abs(x) + Math.abs(y);
    const maxPossibleExtra = Math.min(2 * k, i + 1 - currentDistance);

    result = Math.max(result, currentDistance + maxPossibleExtra);
  }

  return result;
};
```

3445 Maximum Difference Between Even And Odd Frequency Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3445. Maximum Difference Between Even and Odd Frequency II
 * https://leetcode.com/problems/maximum-difference-between-even-and-odd-frequency-ii/
 * Difficulty: Hard
 *
```

```
 * You are given a string s and an integer k. Your task is to find the maximum difference
between
 * the frequency of two characters, freq[a] - freq[b], in a substring subs of s, such that:
 * - subs has a size of at least k.
 * - Character a has an odd frequency in subs.
 * - Character b has an even frequency in subs.
 *
 * Return the maximum difference.
 *
 * Note that subs can contain more than 2 distinct characters.
 */

/**
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var maxDifference = function(s, k) {
  const n = s.length;
  let result = -Infinity;

  const calculateParityStatus = (freqA, freqB) => {
    return ((freqA & 1) << 1) | (freqB & 1);
  };

  for (const charA of ['0', '1', '2', '3', '4']) {
    for (const charB of ['0', '1', '2', '3', '4']) {
      if (charA === charB) continue;

      const minDifferences = [Infinity, Infinity, Infinity, Infinity];
      let currentFreqA = 0;
      let currentFreqB = 0;
      let prefixFreqA = 0;
      let prefixFreqB = 0;
      let leftBoundary = -1;

      for (let rightIndex = 0; rightIndex < n; rightIndex++) {
        currentFreqA += s[rightIndex] === charA ? 1 : 0;
        currentFreqB += s[rightIndex] === charB ? 1 : 0;

        while (rightIndex - leftBoundary >= k && currentFreqB - prefixFreqB >= 2) {
          const prefixStatus = calculateParityStatus(prefixFreqA, prefixFreqB);
          minDifferences[prefixStatus] = Math.min(
            minDifferences[prefixStatus],
            prefixFreqA - prefixFreqB
          );
          leftBoundary++;
          prefixFreqA += s[leftBoundary] === charA ? 1 : 0;
          prefixFreqB += s[leftBoundary] === charB ? 1 : 0;
        }

        const currentStatus = calculateParityStatus(currentFreqA, currentFreqB);
        const targetStatus = currentStatus ^ 0b10;

        if (minDifferences[targetStatus] !== Infinity) {
          result = Math.max(
            result,
            currentFreqA - currentFreqB - minDifferences[targetStatus]
          );
        }
      }
    }
  }

  return result;
};
```

3446 Sort Matrix By Diagonals

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3446. Sort Matrix by Diagonals
 * https://leetcode.com/problems/sort-matrix-by-diagonals/
 * Difficulty: Medium
 *
 * You are given an n x n square matrix of integers grid. Return the matrix such that:
 * - The diagonals in the bottom-left triangle (including the middle diagonal) are sorted
 *   in non-increasing order.
 * - The diagonals in the top-right triangle are sorted in non-decreasing order.
 */

/**
 * @param {number[][]} grid
 * @return {number[][]}
 */
var sortMatrix = function(grid) {
  const n = grid.length;
  const map = new Map();

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      const key = i - j;
      if (!map.has(key)) {
        map.set(key, key < 0
          ? new PriorityQueue((a, b) => a - b)
          : new PriorityQueue((a, b) => b - a));
      }
      map.get(key).enqueue(grid[i][j]);
    }
  }

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      const key = i - j;
      grid[i][j] = map.get(key).dequeue();
    }
  }

  return grid;
};
```

3450 Maximum Students On A Single Bench

EASY

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3450. Maximum Students on a Single Bench
 * https://leetcode.com/problems/maximum-students-on-a-single-bench/
 * Difficulty: Easy
 *
 * You are given a 2D integer array of student data students, where
 * students[i] = [student_id, bench_id] represents that student student_id is
 * sitting on the bench bench_id.
 *
 * Return the maximum number of unique students sitting on any single bench. If no
 * students are present, return 0.
 *
 * Note: A student can appear multiple times on the same bench in the input, but
 * they should be counted only once per bench.
 */

/**
 * @param {number[][]} students
 * @return {number}
 */
var maxStudentsOnBench = function(students) {
  const map = new Map();

  for (const [studentId, benchId] of students) {
```

```
    if (!map.has(benchId)) {
      map.set(benchId, new Set());
    }
    map.get(benchId).add(studentId);
  }

  let result = 0;
  for (const studentSet of map.values()) {
    result = Math.max(result, studentSet.size);
  }

  return result;
};
```

3452 Sum Of Good Numbers

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3452. Sum of Good Numbers
 * https://leetcode.com/problems/sum-of-good-numbers/
 * Difficulty: Easy
 *
 * Given an array of integers nums and an integer k, an element nums[i] is considered good if
 * it is strictly greater than the elements at indices i - k and i + k (if those indices
exist).
 * If neither of these indices exists, nums[i] is still considered good.
 *
 * Return the sum of all the good elements in the array.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var sumOfGoodNumbers = function(nums, k) {
  return nums.reduce((sum, n, i) => {
    return n > (nums[i - k] ?? 0) && n > (nums[i + k] ?? 0) ? sum + n : sum;
  }, 0);
};
```

3460 Longest Common Prefix After At Most One Removal

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3460. Longest Common Prefix After at Most One Removal
 * https://leetcode.com/problems/longest-common-prefix-after-at-most-one-removal/
 * Difficulty: Medium
 *
 * You are given two strings s and t.
 *
 * Return the length of the longest common prefix between s and t after removing at most
 * one character from s.
 *
 * Note: s can be left without any removal.
 */

/**
 * @param {string} s
 * @param {string} t
 * @return {number}
 */
var longestCommonPrefix = function(s, t) {
```

```
  let i = 0;
  let j = 0;
  let result = 0;
  let isRemoved = false;

  while (i < s.length && j < t.length) {
    if (s[i] === t[j]) {
      i++;
      j++;
      result++;
    } else if (isRemoved) {
      return result;
    } else {
      isRemoved = true;
      i++;
    }
  }

  return result;
};
```

3461 Check If Digits Are Equal In String After Operations I

EASY

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3461. Check If Digits Are Equal in String After Operations I
 * https://leetcode.com/problems/check-if-digits-are-equal-in-string-after-operations-i/
 * Difficulty: Easy
 *
 * You are given a string s consisting of digits. Perform the following operation repeatedly
 * until the string has exactly two digits:
 * - For each pair of consecutive digits in s, starting from the first digit, calculate a new
 *   digit as the sum of the two digits modulo 10.
 * - Replace s with the sequence of newly calculated digits, maintaining the order in which
 *   they are computed.
 *
 * Return true if the final two digits in s are the same; otherwise, return false.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var hasSameDigits = function(s) {
  while (s.length > 2) {
    let next = '';
    for (let i = 0; i < s.length - 1; i++) {
      next += (parseInt(s[i]) + parseInt(s[i + 1])) % 10;
    }
    s = next;
  }
  return s[0] === s[1];
};
```

3462 Maximum Sum With At Most K Elements

MEDIUM

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3462. Maximum Sum With at Most K Elements
 * https://leetcode.com/problems/maximum-sum-with-at-most-k-elements/
 * Difficulty: Medium
 *
 * You are given a 2D integer matrix grid of size n x m, an integer array limits of length n,
```

```
and
 * an integer k. The task is to find the maximum sum of at most k elements from the matrix
grid
 * such that:
 * - The number of elements taken from the ith row of grid does not exceed limits[i].
 *
 * Return the maximum sum.
 */

/**
 * @param {number[][]} grid
 * @param {number[]} limits
 * @param {number} k
 * @return {number}
 */
var maxSum = function(grid, limits, k) {
  const result = [];
  grid.forEach((row, i) => {
    result.push(...row.slice().sort((a, b) => b - a).slice(0, limits[i]));
  });
  return result.sort((a, b) => b - a).slice(0, k).reduce((sum, num) => sum + num, 0);
};
```

3463 Check If Digits Are Equal In String After Operations Ii

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3463. Check If Digits Are Equal in String After Operations II
 * https://leetcode.com/problems/check-if-digits-are-equal-in-string-after-operations-ii/
 * Difficulty: Hard
 *
 * You are given a string s consisting of digits. Perform the following operation repeatedly
until
 * the string has exactly two digits:
 * - For each pair of consecutive digits in s, starting from the first digit, calculate a new
digit
 *   as the sum of the two digits modulo 10.
 * - Replace s with the sequence of newly calculated digits, maintaining the order in which
they
 *   are computed.
 *
 * Return true if the final two digits in s are the same; otherwise, return false.
 */

/**
 * @param {string} s
 * @return {boolean}
 */
var hasSameDigits = function(s) {
  const n = s.length;
  const binomialMod2 = (k, n) => (k & n) === k ? 1 : 0;
  const binomialMod5 = (k, n) => {
    if (k > n) return 0;
    let result = 1;
    const pascalMod5 = [
      [1, 0, 0, 0, 0],
      [1, 1, 0, 0, 0],
      [1, 2, 1, 0, 0],
      [1, 3, 3, 1, 0],
      [1, 4, 1, 4, 1]
    ];

    while (k > 0 || n > 0) {
      const ki = k % 5;
      const ni = n % 5;
      if (ki > ni) return 0;
      result = (result * pascalMod5[ni][ki]) % 5;
      k = Math.floor(k / 5);
      n = Math.floor(n / 5);
```

```
    }
    return result;
  };

  const binomialMod10 = (k, n) => {
    const mod2 = binomialMod2(k, n);
    const mod5 = binomialMod5(k, n);
    for (let i = 0; i < 10; i++) {
      if (i % 2 === mod2 && i % 5 === mod5) return i;
    }
    return 0;
  };

  let sum1 = 0;
  let sum2 = 0;
  for (let i = 0; i <= n - 2; i++) {
    const coeff = binomialMod10(i, n - 2);
    sum1 = (sum1 + coeff * +s[i]) % 10;
    sum2 = (sum2 + coeff * +s[i + 1]) % 10;
  }

  return sum1 === sum2;
};
```

3464 Maximize The Distance Between Points On A Square

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3464. Maximize the Distance Between Points on a Square
 * https://leetcode.com/problems/maximize-the-distance-between-points-on-a-square/
 * Difficulty: Hard
 *
 * You are given an integer side, representing the edge length of a square with corners
 * at (0, 0), (0, side), (side, 0), and (side, side) on a Cartesian plane.
 *
 * You are also given a positive integer k and a 2D integer array points, where
 * points[i] = [xi, yi] represents the coordinate of a point lying on the boundary of
 * the square.
 *
 * You need to select k elements among points such that the minimum Manhattan distance
 * between any two points is maximized.
 *
 * Return the maximum possible minimum Manhattan distance between the selected k points.
 *
 * The Manhattan Distance between two cells (xi, yi) and (xj, yj) is |xi - xj| + |yi - yj|.
 */

/**
 * @param {number} squareSide
 * @param {number[][]} coordinates
 * @param {number} pointsToPlace
 * @return {number}
 */
var maxDistance = function(squareSide, coordinates, pointsToPlace) {
  const pointCount = coordinates.length;
  const positions = new Array(pointCount);

  for (let i = 0; i < pointCount; i++) {
    const [x, y] = coordinates[i];
    positions[i] = y === 0 ? x : x === squareSide
      ? squareSide + y
      : y === squareSide
        ? 2 * squareSide + (squareSide - x)
        : 3 * squareSide + (squareSide - y);
  }
  positions.sort((a, b) => a - b);

  const perimeter = 4 * squareSide;
```

```
  const extendedPositions = new Array(pointCount * 2);
  for (let i = 0; i < pointCount; i++) {
    extendedPositions[i] = positions[i];
    extendedPositions[i + pointCount] = positions[i] + perimeter;
  }

  let result = 0;
  let maxDistance = 2 * squareSide;
  while (result < maxDistance) {
    const midDistance = Math.floor((result + maxDistance + 1) / 2);
    if (canPlaceAtDistance(midDistance)) result = midDistance;
    else maxDistance = midDistance - 1;
  }
  return result;

  function canPlaceAtDistance(distance) {
    for (let start = 0; start < pointCount; start++) {
      let current = start;
      let lastPos = extendedPositions[start];
      const limit = start + pointCount;
      let valid = true;

      for (let placed = 1; placed < pointsToPlace; placed++) {
        const nextTarget = lastPos + distance;
        let left = current + 1;
        let right = limit;

        while (left < right) {
          const mid = Math.floor((left + right) / 2);
          if (extendedPositions[mid] < nextTarget) left = mid + 1;
          else right = mid;
        }

        if (left === limit) {
          valid = false;
          break;
        }
        current = left;
        lastPos = extendedPositions[current];
      }

      if (valid && extendedPositions[start] + perimeter - lastPos >= distance) {
        return true;
      }
    }
    return false;
  }
};
```

3466 Maximum Coin Collection

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3466. Maximum Coin Collection
 * https://leetcode.com/problems/maximum-coin-collection/
 * Difficulty: Medium
 *
 * Mario drives on a two-lane freeway with coins every mile. You are given two integer arrays,
 * lane1 and lane2, where the value at the ith index represents the number of coins he gains
 * or loses in the ith mile in that lane.
 * - If Mario is in lane 1 at mile i and lane1[i] > 0, Mario gains lane1[i] coins.
 * - If Mario is in lane 1 at mile i and lane1[i] < 0, Mario pays a toll and loses
abs(lane1[i])
 *   coins.
 * - The same rules apply for lane2.
 *
 * Mario can enter the freeway anywhere and exit anytime after traveling at least one mile.
Mario
 * always enters the freeway on lane 1 but can switch lanes at most 2 times.
```

```
 *
 * A lane switch is when Mario goes from lane 1 to lane 2 or vice versa.
 *
 * Return the maximum number of coins Mario can earn after performing at most 2 lane switches.
 *
 * Note: Mario can switch lanes immediately upon entering or just before exiting the freeway.
 */

/**
 * @param {number[]} lane1
 * @param {number[]} lane2
 * @return {number}
 */
var maxCoins = function(lane1, lane2) {
  const n = lane1.length;
  const dp = new Array(n).fill().map(() => new Array(2).fill().map(() => new
Array(3).fill(0)));

  dp[0][0][2] = lane1[0];
  dp[0][0][0] = lane1[0];
  dp[0][1][1] = lane2[0];

  let result = Math.max(dp[0][0][2], dp[0][1][1]);
  for (let i = 1; i < n; i++) {
    dp[i][0][0] = Math.max(dp[i - 1][0][0], dp[i - 1][1][1]) + lane1[i];

    dp[i][0][2] = Math.max(dp[i - 1][0][2] + lane1[i], lane1[i]);

    dp[i][1][1] = Math.max(Math.max(dp[i - 1][0][2], dp[i - 1][1][1]), 0) + lane2[i];

    const currentMax = Math.max(Math.max(dp[i][0][0], dp[i][0][2]), dp[i][1][1]);
    result = Math.max(result, currentMax);
  }

  return result;
};
```

3476 Maximize Profit From Task Assignment

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3476. Maximize Profit from Task Assignment
 * https://leetcode.com/problems/maximize-profit-from-task-assignment/
 * Difficulty: Medium
 *
 * You are given an integer array workers, where workers[i] represents the skill level of
 * the ith worker. You are also given a 2D integer array tasks, where:
 * - tasks[i][0] represents the skill requirement needed to complete the task.
 * - tasks[i][1] represents the profit earned from completing the task.
 *
 * Each worker can complete at most one task, and they can only take a task if their skill
 * level is equal to the task's skill requirement. An additional worker joins today who can
 * take up any task, regardless of the skill requirement.
 *
 * Return the maximum total profit that can be earned by optimally assigning the tasks to
 * the workers.
 */

/**
 * @param {number[]} workers
 * @param {number[][]} tasks
 * @return {number}
 */
var maxProfit = function(workers, tasks) {
  const map = new Map();

  for (const [skill, profit] of tasks) {
    if (!map.has(skill)) {
```

```
      map.set(skill, []);
    }
    map.get(skill).push(profit);
  }

  for (const [skill, profits] of map) {
    profits.sort((a, b) => b - a);
  }

  let totalProfit = 0;

  for (const worker of workers) {
    if (map.has(worker) && map.get(worker).length > 0) {
      totalProfit += map.get(worker).shift();
      if (map.get(worker).length === 0) {
        map.delete(worker);
      }
    }
  }

  let maxRemainingProfit = 0;
  for (const profits of map.values()) {
    if (profits.length > 0) {
      maxRemainingProfit = Math.max(maxRemainingProfit, profits[0]);
    }
  }

  return totalProfit + maxRemainingProfit;
};
```

3477 Fruits Into Baskets Ii

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3477. Fruits Into Baskets II
 * https://leetcode.com/problems/fruits-into-baskets-ii/
 * Difficulty: Easy
 *
 * You are given two arrays of integers, fruits and baskets, each of length n, where
 * fruits[i] represents the quantity of the ith type of fruit, and baskets[j] represents
 * the capacity of the jth basket.
 *
 * From left to right, place the fruits according to these rules:
 * - Each fruit type must be placed in the leftmost available basket with a capacity greater
 *   than or equal to the quantity of that fruit type.
 * - Each basket can hold only one type of fruit.
 * - If a fruit type cannot be placed in any basket, it remains unplaced.
 *
 * Return the number of fruit types that remain unplaced after all possible allocations are
made.
 */

/**
 * @param {number[]} fruits
 * @param {number[]} baskets
 * @return {number}
 */
var numOfUnplacedFruits = function(fruits, baskets) {
  const availableBaskets = [...baskets];
  let result = 0;

  for (const fruitQuantity of fruits) {
    let placed = false;

    for (let i = 0; i < availableBaskets.length; i++) {
      if (availableBaskets[i] >= fruitQuantity) {
        availableBaskets[i] = 0;
        placed = true;
        break;
```

```
      }
    }

    if (!placed) {
      result++;
    }
  }

  return result;
};
```

3479 Fruits Into Baskets Iii

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3479. Fruits Into Baskets III
 * https://leetcode.com/problems/fruits-into-baskets-iii/
 * Difficulty: Medium
 *
 * You are given two arrays of integers, fruits and baskets, each of length n, where fruits[i]
 * represents the quantity of the ith type of fruit, and baskets[j] represents the capacity of
 * the jth basket.
 *
 * From left to right, place the fruits according to these rules:
 * - Each fruit type must be placed in the leftmost available basket with a capacity greater
 *   than or equal to the quantity of that fruit type.
 * - Each basket can hold only one type of fruit.
 * - If a fruit type cannot be placed in any basket, it remains unplaced.
 *
 * Return the number of fruit types that remain unplaced after all possible allocations are
made.
 */

/**
 * @param {number[]} fruits
 * @param {number[]} baskets
 * @return {number}
 */
var numOfUnplacedFruits = function(fruits, baskets) {
  const n = fruits.length;
  const bucketSize = Math.ceil(Math.sqrt(n));
  const buckets = Array.from({ length: bucketSize }, () => []);

  for (let i = 0; i < baskets.length; i++) {
    const bucketIndex = Math.floor(i / bucketSize);
    buckets[bucketIndex].push([baskets[i], i]);
  }

  for (const bucket of buckets) {
    bucket.sort((a, b) => a[0] - b[0]);
  }

  let result = 0;
  for (const fruitQuantity of fruits) {
    let placed = false;

    for (const bucket of buckets) {
      if (bucket.length > 0 && bucket[bucket.length - 1][0] >= fruitQuantity) {
        let chosenIndex = -1;
        let minBasketIndex = Infinity;

        for (let i = 0; i < bucket.length; i++) {
          if (bucket[i][0] >= fruitQuantity && bucket[i][1] < minBasketIndex) {
            chosenIndex = i;
            minBasketIndex = bucket[i][1];
          }
        }

        if (chosenIndex !== -1) {
```

```
          bucket.splice(chosenIndex, 1);
          placed = true;
          break;
        }
      }
    }

    if (!placed) {
      result++;
    }
  }

  return result;
};
```

3480 Maximize Subarrays After Removing One Conflicting Pair

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3480. Maximize Subarrays After Removing One Conflicting Pair
 * https://leetcode.com/problems/maximize-subarrays-after-removing-one-conflicting-pair/
 * Difficulty: Hard
 *
 * You are given an integer n which represents an array nums containing the numbers from
 * 1 to n in order. Additionally, you are given a 2D array conflictingPairs, where
 * conflictingPairs[i] = [a, b] indicates that a and b form a conflicting pair.
 *
 * Remove exactly one element from conflictingPairs. Afterward, count the number of
 * non-empty subarrays of nums which do not contain both a and b for any remaining
 * conflicting pair [a, b].
 *
 * Return the maximum number of subarrays possible after removing exactly one conflicting
pair.
 */

/**
 * @param {number} n
 * @param {number[][]} conflictingPairs
 * @return {number}
 */
var maxSubarrays = function(n, conflictingPairs) {
  const right = Array.from({ length: n + 1 }, () => []);

  for (const [a, b] of conflictingPairs) {
    right[Math.max(a, b)].push(Math.min(a, b));
  }

  let left = [0, 0];
  let total = 0;
  const bonus = new Array(n + 1).fill(0);
  for (let r = 1; r <= n; r++) {
    for (const l of right[r]) {
      if (l > left[0]) {
        left = [l, left[0]];
      } else if (l > left[1]) {
        left = [left[0], l];
      }
    }

    total += r - left[0];

    if (left[0] > 0) {
      bonus[left[0]] += left[0] - left[1];
    }
  }

  return total + Math.max(...bonus);
};
```

3481 Apply Substitutions

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 3481. Apply Substitutions
 * https://leetcode.com/problems/apply-substitutions/
 * Difficulty: Medium
 *
 * You are given a replacements mapping and a text string that may contain placeholders
 * formatted as %var%, where each var corresponds to a key in the replacements mapping.
 * Each replacement value may itself contain one or more such placeholders. Each placeholder
 * is replaced by the value associated with its corresponding replacement key.
 *
 * Return the fully substituted text string which does not contain any placeholders.
 */

/**
 * @param {string[][]} replacements
 * @param {string} text
 * @return {string}
 */
var applySubstitutions = function(replacements, text) {
  const replacementMap = new Map(replacements);
  const cache = new Map();

  return text.replace(/%([A-Z])%/g, (match, key) => helper(key));

  function helper(key) {
    if (cache.has(key)) {
      return cache.get(key);
    }

    const value = replacementMap.get(key);
    const result = value.replace(/%([A-Z])%/g, (match, innerKey) => helper(innerKey));
    cache.set(key, result);
    return result;
  }
};
```

3484 Design Spreadsheet

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups.

SOURCE CODE

```
/**
 * 3484. Design Spreadsheet
 * https://leetcode.com/problems/design-spreadsheet/
 * Difficulty: Medium
 *
 * A spreadsheet is a grid with 26 columns (labeled from 'A' to 'Z') and a given number
 * of rows. Each cell in the spreadsheet can hold an integer value between 0 and 105.
 *
 * Implement the Spreadsheet class:
 * - Spreadsheet(int rows) Initializes a spreadsheet with 26 columns (labeled 'A' to 'Z')
 *   and the specified number of rows. All cells are initially set to 0.
 * - void setCell(String cell, int value) Sets the value of the specified cell. The cell
 *   reference is provided in the format "AX" (e.g., "A1", "B10"), where the letter represents
 *   the column (from 'A' to 'Z') and the number represents a 1-indexed row.
 * - void resetCell(String cell) Resets the specified cell to 0.
 * - int getValue(String formula) Evaluates a formula of the form "=X+Y", where X and Y are
 *   either cell references or non-negative integers, and returns the computed sum.
 *
 * Note: If getValue references a cell that has not been explicitly set using setCell, its
 * value is considered 0.
 */
```

```
/**
 * @param {number} rows
 */
var Spreadsheet = function(rows) {
  this.cellValues = new Map();
  this.totalRows = rows;
};

/**
 * @param {string} cell
 * @param {number} value
 * @return {void}
 */
Spreadsheet.prototype.setCell = function(cell, value) {
  this.cellValues.set(cell, value);
};

/**
 * @param {string} cell
 * @return {void}
 */
Spreadsheet.prototype.resetCell = function(cell) {
  this.cellValues.delete(cell);
};

/**
 * @param {string} formula
 * @return {number}
 */
Spreadsheet.prototype.getValue = function(formula) {
  const expression = formula.slice(1);
  const operands = expression.split('+');

  const leftValue = this.parseOperand(operands[0]);
  const rightValue = this.parseOperand(operands[1]);

  return leftValue + rightValue;
};

/**
 * @param {string} operand
 * @return {number}
 */
Spreadsheet.prototype.parseOperand = function(operand) {
  if (/^\d+$/.test(operand)) {
    return parseInt(operand);
  }
  return this.cellValues.get(operand) || 0;
};
```

3487 Maximum Unique Subarray Sum After Deletion

EASY

APPROACH NOTES

Uses a Set for unique membership.

SOURCE CODE

```
/**
 * 3487. Maximum Unique Subarray Sum After Deletion
 * https://leetcode.com/problems/maximum-unique-subarray-sum-after-deletion/
 * Difficulty: Easy
 *
 * You are given an integer array nums.
 *
 * You are allowed to delete any number of elements from nums without making it empty.
 * After performing the deletions, select a subarray of nums such that:
 * 1. All elements in the subarray are unique.
 * 2. The sum of the elements in the subarray is maximized.
 *
 * Return the maximum sum of such a subarray.
 */
```

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSum = function(nums) {
  const uniqueValues = [...new Set(nums)];
  const positiveValues = uniqueValues.filter(val => val > 0);

  if (positiveValues.length === 0) {
    return Math.max(...uniqueValues);
  }

  return positiveValues.reduce((sum, val) => sum + val, 0);
};
```

3491 Phone Number Prefix

EASY

APPROACH NOTES

Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3491. Phone Number Prefix
 * https://leetcode.com/problems/phone-number-prefix/
 * Difficulty: Easy
 *
 * You are given a string array numbers that represents phone numbers. Return true if no phone
 * number is a prefix of any other phone number; otherwise, return false.
 */

/**
 * @param {string[]} numbers
 * @return {boolean}
 */
var phonePrefix = function(numbers) {
  numbers.sort();

  for (let i = 0; i < numbers.length - 1; i++) {
    if (numbers[i + 1].startsWith(numbers[i])) {
      return false;
    }
  }

  return true;
};
```

3494 Find The Minimum Amount Of Time To Brew Potions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3494. Find the Minimum Amount of Time to Brew Potions
 * https://leetcode.com/problems/find-the-minimum-amount-of-time-to-brew-potions/
 * Difficulty: Medium
 *
 * You are given two integer arrays, skill and mana, of length n and m, respectively.
 *
 * In a laboratory, n wizards must brew m potions in order. Each potion has a mana capacity
 * mana[j] and must pass through all the wizards sequentially to be brewed properly. The
 * time taken by the ith wizard on the jth potion is timeij = skill[i] * mana[j].
 *
 * Since the brewing process is delicate, a potion must be passed to the next wizard
 * immediately after the current wizard completes their work. This means the timing must
 * be synchronized so that each wizard begins working on a potion exactly when it arrives.
 *
 * Return the minimum amount of time required for the potions to be brewed properly.
 */
```

```
/**
 * @param {number[]} skill
 * @param {number[]} mana
 * @return {number}
 */
var minTime = function(skill, mana) {
  const result = new Array(skill.length + 1).fill(0);

  for (let j = 0; j < mana.length; j++) {
    for (let i = 0; i < skill.length; i++) {
      result[i + 1] = Math.max(result[i + 1], result[i]) + mana[j] * skill[i];
    }
    for (let i = skill.length - 1; i > 0; i--) {
      result[i] = result[i + 1] - mana[j] * skill[i];
    }
  }

  return result[skill.length];
};
```

3495 Minimum Operations To Make Array Elements Zero

HARD

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3495. Minimum Operations to Make Array Elements Zero
 * https://leetcode.com/problems/minimum-operations-to-make-array-elements-zero/
 * Difficulty: Hard
 *
 * You are given a 2D array queries, where queries[i] is of the form [l, r]. Each
 * queries[i] defines an array of integers nums consisting of elements ranging
 * from l to r, both inclusive.
 *
 * In one operation, you can:
 * - Select two integers a and b from the array.
 * - Replace them with floor(a / 4) and floor(b / 4).
 *
 * Your task is to determine the minimum number of operations required to reduce all
 * elements of the array to zero for each query. Return the sum of the results for
 * all queries.
 */

/**
 * @param {number[][]} queries
 * @return {number}
 */
var minOperations = function(queries) {
  let result = 0;

  for (const [start, end] of queries) {
    let operationsNeeded = 0;

    for (let depth = 1, previousBound = 1; depth < 17; depth++) {
      const currentBound = previousBound * 4;
      const intersectionLeft = Math.max(start, previousBound);
      const intersectionRight = Math.min(end, currentBound - 1);

      if (intersectionRight >= intersectionLeft) {
        operationsNeeded += (intersectionRight - intersectionLeft + 1) * depth;
      }
      previousBound = currentBound;
    }

    result += Math.ceil(operationsNeeded / 2);
  }

  return result;
};
```

3496 Maximize Score After Pair Deletions

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3496. Maximize Score After Pair Deletions
 * https://leetcode.com/problems/maximize-score-after-pair-deletions/
 * Difficulty: Medium
 *
 * You are given an array of integers nums. You must repeatedly perform one of the following
 * operations while the array has more than two elements:
 * - Remove the first two elements.
 * - Remove the last two elements.
 * - Remove the first and last element.
 *
 * For each operation, add the sum of the removed elements to your total score.
 *
 * Return the maximum possible score you can achieve.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxScore = function(nums) {
  const n = nums.length;
  const totalSum = nums.reduce((sum, num) => sum + num, 0);

  if (n % 2 === 1) {
    return totalSum - Math.min(...nums);
  } else {
    let minPairSum = Infinity;
    for (let i = 0; i < n - 1; i++) {
      minPairSum = Math.min(minPairSum, nums[i] + nums[i + 1]);
    }
    return totalSum - minPairSum;
  }
};
```

3506 Find Time Required To Eliminate Bacterial Strains

HARD

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3506. Find Time Required to Eliminate Bacterial Strains
 * https://leetcode.com/problems/find-time-required-to-eliminate-bacterial-strains/
 * Difficulty: Hard
 *
 * You are given an integer array timeReq and an integer splitTime.
 *
 * In the microscopic world of the human body, the immune system faces an extraordinary
 * challenge: combatting a rapidly multiplying bacterial colony that threatens the body's
survival.
 *
 * Initially, only one white blood cell (WBC) is deployed to eliminate the bacteria. However,
the
 * lone WBC quickly realizes it cannot keep up with the bacterial growth rate.
 *
 * The WBC devises a clever strategy to fight the bacteria:
 * - The ith bacterial strain takes timeReq[i] units of time to be eliminated.
 * - A single WBC can eliminate only one bacterial strain. Afterwards, the WBC is exhausted
and
 *   cannot perform any other tasks.
 * - A WBC can split itself into two WBCs, but this requires splitTime units of time. Once
split,
```

```
 *   the two WBCs can work in parallel on eliminating the bacteria.
 * - Only one WBC can work on a single bacterial strain. Multiple WBCs cannot attack one
strain
 *   in parallel.
 *
 * You must determine the minimum time required to eliminate all the bacterial strains.
 *
 * Note that the bacterial strains can be eliminated in any order.
 */

/**
 * @param {number[]} timeReq
 * @param {number} splitTime
 * @return {number}
 */
var minEliminationTime = function(timeReq, splitTime) {
  const heap = new PriorityQueue((a, b) => a - b);

  for (const time of timeReq) {
    heap.enqueue(time);
  }

  heap.dequeue();

  let result = 0;
  while (!heap.isEmpty()) {
    const bacteria = splitTime + heap.dequeue();
    if (!heap.isEmpty()) {
      heap.enqueue(bacteria);
      result = heap.dequeue();
    } else {
      result = bacteria;
    }
  }

  return result;
};
```

3508 Implement Router

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Uses a Set for unique membership. Contains a while loop: identify the progress condition.

SOURCE CODE

```
/**
 * 3508. Implement Router
 * https://leetcode.com/problems/implement-router/
 * Difficulty: Medium
 *
 * Design a data structure that can efficiently manage data packets in a network router.
 * Each data packet consists of the following attributes:
 * - source: A unique identifier for the machine that generated the packet.
 * - destination: A unique identifier for the target machine.
 * - timestamp: The time at which the packet arrived at the router.
 *
 * Implement the Router class:
 * Router(int memoryLimit): Initializes the Router object with a fixed memory limit.
 * - memoryLimit is the maximum number of packets the router can store at any given time.
 * - If adding a new packet would exceed this limit, the oldest packet must be removed to
 *   free up space.
 * bool addPacket(int source, int destination, int timestamp): Adds a packet with the
 * given attributes to the router.
 * - A packet is considered a duplicate if another packet with the same source, destination,
 *   and timestamp already exists in the router.
 * - Return true if the packet is successfully added (i.e., it is not a duplicate); otherwise
 *   return false.
 * int[] forwardPacket(): Forwards the next packet in FIFO (First In First Out) order.
 * - Remove the packet from storage.
 * - Return the packet as an array [source, destination, timestamp].
 * - If there are no packets to forward, return an empty array.
 * int getCount(int destination, int startTime, int endTime):
 * - Returns the number of packets currently stored in the router (i.e., not yet forwarded)
```

```
that
 *   have the specified destination and have timestamps in the inclusive range [startTime,
endTime].
 *
 * Note that queries for addPacket will be made in increasing order of timestamp.
 */

/**
 * @param {number} memoryLimit
 */
var Router = function(memoryLimit) {
  this.memoryCapacity = memoryLimit;
  this.packetQueue = [];
  this.packetSet = new Set();
  this.destinationTimestamps = new Map();
  this.removedPacketIndex = new Map();
};

/**
 * @param {number} source
 * @param {number} destination
 * @param {number} timestamp
 * @return {boolean}
 */
Router.prototype.addPacket = function(source, destination, timestamp) {
  const packetKey = `${source}-${destination}-${timestamp}`;

  if (this.packetSet.has(packetKey)) {
    return false;
  }

  if (this.packetQueue.length === this.memoryCapacity) {
    const [oldSource, oldDestination, oldTimestamp] = this.packetQueue.shift();
    const oldPacketKey = `${oldSource}-${oldDestination}-${oldTimestamp}`;
    this.packetSet.delete(oldPacketKey);
    this.removedPacketIndex.set(
      oldDestination,
      (this.removedPacketIndex.get(oldDestination) || 0) + 1,
    );
  }

  this.packetQueue.push([source, destination, timestamp]);
  this.packetSet.add(packetKey);

  if (!this.destinationTimestamps.has(destination)) {
    this.destinationTimestamps.set(destination, []);
  }
  this.destinationTimestamps.get(destination).push(timestamp);

  return true;
};

/**
 * @return {number[]}
 */
Router.prototype.forwardPacket = function() {
  if (this.packetQueue.length === 0) {
    return [];
  }

  const [source, destination, timestamp] = this.packetQueue.shift();
  const packetKey = `${source}-${destination}-${timestamp}`;
  this.packetSet.delete(packetKey);
  this.removedPacketIndex.set(destination, (this.removedPacketIndex.get(destination) || 0) +
1);

  return [source, destination, timestamp];
};

/**
 * @param {number} destination
 * @param {number} startTime
 * @param {number} endTime
 * @return {number}
 */
```

```
Router.prototype.getCount = function(destination, startTime, endTime) {
  if (!this.destinationTimestamps.has(destination)) {
    return 0;
  }

  const timestampArray = this.destinationTimestamps.get(destination);
  const removedCount = this.removedPacketIndex.get(destination) || 0;
  const totalLength = timestampArray.length;

  if (removedCount >= totalLength) {
    return 0;
  }

  const leftBound = this.binarySearchLeft(timestampArray, startTime, removedCount);
  const rightBound = this.binarySearchRight(timestampArray, endTime, removedCount) - 1;

  if (leftBound > rightBound) {
    return 0;
  }

  return rightBound - leftBound + 1;
};

/**
 * @param {number[]} array
 * @param {number} target
 * @param {number} startIndex
 * @return {number}
 */
Router.prototype.binarySearchLeft = function(array, target, startIndex) {
  let left = startIndex;
  let right = array.length;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (array[mid] < target) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
};

/**
 * @param {number[]} array
 * @param {number} target
 * @param {number} startIndex
 * @return {number}
 */
Router.prototype.binarySearchRight = function(array, target, startIndex) {
  let left = startIndex;
  let right = array.length;

  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (array[mid] <= target) {
      left = mid + 1;
    } else {
      right = mid;
    }
  }

  return left;
};
```

3511 Make A Positive Array

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3511. Make a Positive Array
 * https://leetcode.com/problems/make-a-positive-array/
 * Difficulty: Medium
 *
 * You are given an array nums. An array is considered positive if the sum of all numbers in
 * each subarray with more than two elements is positive.
 *
 * You can perform the following operation any number of times:
 * - Replace one element in nums with any integer between -1018 and 1018.
 *
 * Find the minimum number of operations needed to make nums positive.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var makeArrayPositive = function(nums) {
  let result = 0;
  let lowestSum = nums[0] + nums[1];

  for (let i = 2; i < nums.length; i++) {
    const currentValue = nums[i];
    const currentTriplet = currentValue + nums[i - 1] + nums[i - 2];
    lowestSum = Math.min(lowestSum + currentValue, currentTriplet);

    if (lowestSum <= 0) {
      nums[i] = lowestSum = 10 ** 18;
      result++;
    }
  }

  return result;
};
```

3512 Minimum Operations To Make Array Sum Divisible By K

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3512. Minimum Operations to Make Array Sum Divisible by K
 * https://leetcode.com/problems/minimum-operations-to-make-array-sum-divisible-by-k/
 * Difficulty: Easy
 *
 * You are given an integer array nums and an integer k. You can perform the following
 * operation any number of times:
 * - Select an index i and replace nums[i] with nums[i] - 1.
 *
 * Return the minimum number of operations required to make the sum of the array
 * divisible by k.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var minOperations = function(nums, k) {
  return nums.reduce((sum, n) => sum + n, 0) % k;
};
```

3516 Find Closest Person

EASY

APPROACH NOTES

Trace the function from its parameters to its return value; write down intermediate states.

SOURCE CODE

```
/**
 * 3516. Find Closest Person
 * https://leetcode.com/problems/find-closest-person/
 * Difficulty: Easy
 *
 * You are given three integers x, y, and z, representing the positions of three people
 * on a number line:
 * - x is the position of Person 1.
 * - y is the position of Person 2.
 * - z is the position of Person 3, who does not move.
 *
 * Both Person 1 and Person 2 move toward Person 3 at the same speed.
 *
 * Determine which person reaches Person 3 first:
 * - Return 1 if Person 1 arrives first.
 * - Return 2 if Person 2 arrives first.
 * - Return 0 if both arrive at the same time.
 *
 * Return the result accordingly.
 */

/**
 * @param {number} x
 * @param {number} y
 * @param {number} z
 * @return {number}
 */
var findClosest = function(x, y, z) {
  const distancePerson1 = Math.abs(x - z);
  const distancePerson2 = Math.abs(y - z);

  if (distancePerson1 < distancePerson2) {
    return 1;
  } else if (distancePerson2 < distancePerson1) {
    return 2;
  } else {
    return 0;
  }
};
```

3520 Minimum Threshold For Inversion Pairs Count

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3520. Minimum Threshold for Inversion Pairs Count
 * https://leetcode.com/problems/minimum-threshold-for-inversion-pairs-count/
 * Difficulty: Medium
 *
 * You are given an array of integers nums and an integer k.
 *
 * An inversion pair with a threshold x is defined as a pair of indices (i, j) such that:
 * - i < j
 * - nums[i] > nums[j]
 * - The difference between the two numbers is at most x (i.e. nums[i] - nums[j] <= x).
 *
 * Your task is to determine the minimum integer min_threshold such that there are at least
 * k inversion pairs with threshold min_threshold.
 *
 * If no such integer exists, return -1.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
```

```
var minThreshold = function(nums, k) {
  const maxThreshold = Math.max(...nums) - Math.min(...nums) + 1;

  if (!countInversions(maxThreshold)) return -1;

  let left = 0;
  let right = maxThreshold;
  while (left < right) {
    const mid = Math.floor((left + right) / 2);
    if (countInversions(mid)) {
      right = mid;
    } else {
      left = mid + 1;
    }
  }

  return left;

  function binarySearchLeft(arr, target) {
    let left = 0;
    let right = arr.length;
    while (left < right) {
      const mid = Math.floor((left + right) / 2);
      if (arr[mid] < target) {
        left = mid + 1;
      } else {
        right = mid;
      }
    }
    return left;
  }

  function insertSorted(arr, val) {
    const index = binarySearchLeft(arr, val);
    arr.splice(index, 0, val);
  }

  function countInversions(threshold) {
    let count = 0;
    const sortedList = [];

    for (let i = nums.length - 1; i >= 0; i--) {
      const num = nums[i];
      const leftBound = num - threshold;
      const rightBound = num;
      const leftIndex = binarySearchLeft(sortedList, leftBound);
      const rightIndex = binarySearchLeft(sortedList, rightBound);

      count += rightIndex - leftIndex;
      if (count >= k) return true;

      insertSorted(sortedList, num);
    }

    return false;
  }
};
```

3531 Count Covered Buildings

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant. Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3531. Count Covered Buildings
 * https://leetcode.com/problems/count-covered-buildings/
 * Difficulty: Medium
 *
 * You are given a positive integer n, representing an n x n city. You are also given
 * a 2D grid buildings, where buildings[i] = [x, y] denotes a unique building located
```

```
 * at coordinates [x, y].
 *
 * A building is covered if there is at least one building in all four directions:
 * left, right, above, and below.
 *
 * Return the number of covered buildings.
 */

/**
 * @param {number} n
 * @param {number[][]} buildings
 * @return {number}
 */
var countCoveredBuildings = function(n, buildings) {
  const rowBuildings = new Map();
  const colBuildings = new Map();
  let result = 0;

  for (const [x, y] of buildings) {
    if (!rowBuildings.has(x)) rowBuildings.set(x, []);
    if (!colBuildings.has(y)) colBuildings.set(y, []);
    rowBuildings.get(x).push(y);
    colBuildings.get(y).push(x);
  }
  for (const coords of rowBuildings.values()) {
    coords.sort((a, b) => a - b);
  }
  for (const coords of colBuildings.values()) {
    coords.sort((a, b) => a - b);
  }
  for (const [x, y] of buildings) {
    const rowCoords = rowBuildings.get(x);
    const colCoords = colBuildings.get(y);
    const hasLeft = rowCoords[0] < y;
    const hasRight = rowCoords[rowCoords.length - 1] > y;
    const hasAbove = colCoords[0] < x;
    const hasBelow = colCoords[colCoords.length - 1] > x;

    if (hasLeft && hasRight && hasAbove && hasBelow) {
      result++;
    }
  }

  return result;
};
```

3535 Unit Conversion Ii

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3535. Unit Conversion II
 * https://leetcode.com/problems/unit-conversion-ii/
 * Difficulty: Medium
 *
 * There are n types of units indexed from 0 to n - 1.
 *
 * You are given a 2D integer array conversions of length n - 1, where
 * conversions[i] = [sourceUniti, targetUniti, conversionFactori]. This indicates
 * that a single unit of type sourceUniti is equivalent to conversionFactori units
 * of type targetUniti.
 *
 * You are also given a 2D integer array queries of length q, where queries[i] = [unitAi,
unitBi].
 *
 * Return an array answer of length q where answer[i] is the number of units of type unitBi
 * equivalent to 1 unit of type unitAi, and can be represented as p/q where p and q are
coprime.
 * Return each answer[i] as pq-1 modulo 109 + 7, where q-1 represents the multiplicative
```

```
inverse
 * of q modulo 109 + 7.
 */

/**
 * @param {number[][]} conversions
 * @param {number[][]} queries
 * @return {number[]}
 */
var queryConversions = function(conversions, queries) {
  const n = conversions.length + 1;
  const adj = Array.from({ length: n }, () => []);
  const convMap = new Map();
  for (const [s, t, f] of conversions) {
    adj[s].push(t);
    adj[t].push(s);
    convMap.set(`${s}-${t}`, BigInt(f));
  }
  const MOD = 1000000007n;
  const ratio = new Array(n).fill(0n);
  ratio[0] = 1n;
  const visited = new Array(n).fill(false);
  visited[0] = true;
  const queue = [0];
  let queueIndex = 0;
  while (queueIndex < queue.length) {
    const current = queue[queueIndex++];
    for (const neighbor of adj[current]) {
      if (visited[neighbor]) continue;
      visited[neighbor] = true;
      queue.push(neighbor);
      const forwardKey = `${current}-${neighbor}`;
      if (convMap.has(forwardKey)) {
        const factor = convMap.get(forwardKey);
        ratio[neighbor] = (ratio[current] * factor) % MOD;
      } else {
        const backwardKey = `${neighbor}-${current}`;
        const factor = convMap.get(backwardKey);
        const inverseFactor = modInverse(factor, MOD);
        ratio[neighbor] = (ratio[current] * inverseFactor) % MOD;
      }
    }
  }
  const result = [];
  for (const [unitA, unitB] of queries) {
    const ratioA = ratio[unitA];
    const ratioB = ratio[unitB];
    const inverseRatioA = modInverse(ratioA, MOD);
    const conversion = (ratioB * inverseRatioA) % MOD;
    result.push(Number(conversion));
  }

  return result;

  function modPow(base, exponent, modulus) {
    let result = 1n;
    base %= modulus;
    while (exponent > 0n) {
      if (exponent % 2n === 1n) {
        result = (result * base) % modulus;
      }
      base = (base * base) % modulus;
      exponent /= 2n;
    }
    return result;
  }

  function modInverse(value, modulus) {
    return modPow(value, modulus - 2n, modulus);
  }
};
```

3539 Find Sum Of Array Product Of Magical Sequences

HARD

APPROACH NOTES

Uses a Map for keyed lookups. Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3539. Find Sum of Array Product of Magical Sequences
 * https://leetcode.com/problems/find-sum-of-array-product-of-magical-sequences/
 * Difficulty: Hard
 *
 * You are given two integers, m and k, and an integer array nums.
 *
 * A sequence of integers seq is called magical if:
 * - seq has a size of m.
 * - 0 <= seq[i] < nums.length
 * - The binary representation of 2seq[0] + 2seq[1] + ... + 2seq[m - 1] has k set bits.
 *
 * The array product of this sequence is defined as prod(seq) = (nums[seq[0]]
 * * nums[seq[1]] * ... * nums[seq[m - 1]]).
 *
 * Return the sum of the array products for all valid magical sequences.
 *
 * Since the answer may be large, return it modulo 109 + 7.
 *
 * A set bit refers to a bit in the binary representation of a number that has a value of 1.
 */

/**
 * @param {number} m
 * @param {number} k
 * @param {number[]} nums
 * @return {number}
 */
var magicalSum = function(m, k, nums) {
  const MOD = 1000000007n;
  const map = new Map();
  const n = nums.length;

  function bitCount(num) {
    let count = 0;
    while (num > 0) {
      count += num & 1;
      num >>= 1;
    }
    return count;
  }

  function modPow(base, exp) {
    let result = 1n;
    base = BigInt(base) % MOD;
    let e = BigInt(exp);
    while (e > 0n) {
      if (e & 1n) result = (result * base) % MOD;
      base = (base * base) % MOD;
      e >>= 1n;
    }
    return result;
  }

  const factorialCache = [1n];
  function factorial(n) {
    while (factorialCache.length <= n) {
      factorialCache.push(
        factorialCache[factorialCache.length - 1] * BigInt(factorialCache.length)
      );
    }
    return factorialCache[n];
  }

  function comb(n, r) {
```

```
    if (r > n || r < 0) return 0n;
    if (r === 0 || r === n) return 1n;
    return factorial(n) / (factorial(r) * factorial(n - r));
  }

  function dfs(remaining, oddNeeded, index, carry) {
    if (remaining < 0 || oddNeeded < 0 || remaining + bitCount(carry) < oddNeeded) {
      return 0n;
    }
    if (remaining === 0) {
      return bitCount(carry) === oddNeeded ? 1n : 0n;
    }
    if (index >= n) {
      return 0n;
    }

    const key = `${remaining},${oddNeeded},${index},${carry}`;
    if (map.has(key)) return map.get(key);

    let result = 0n;
    for (let take = 0; take <= remaining; take++) {
      const ways = (comb(remaining, take) * modPow(nums[index], take)) % MOD;
      const newCarry = carry + take;
      const contribution = dfs(
        remaining - take, oddNeeded - (newCarry % 2),
        index + 1, Math.floor(newCarry / 2),
      );
      result = (result + ways * contribution) % MOD;
    }

    map.set(key, result);
    return result;
  }

  return Number(dfs(m, k, 0, 0));
};
```

3541 Find Most Frequent Vowel And Consonant

EASY

APPROACH NOTES

Uses a Set for unique membership. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3541. Find Most Frequent Vowel and Consonant
 * https://leetcode.com/problems/find-most-frequent-vowel-and-consonant/
 * Difficulty: Easy
 *
 * You are given a string s consisting of lowercase English letters ('a' to 'z').
 *
 * Your task is to:
 * - Find the vowel (one of 'a', 'e', 'i', 'o', or 'u') with the maximum frequency.
 * - Find the consonant (all other letters excluding vowels) with the maximum frequency.
 *
 * Return the sum of the two frequencies.
 *
 * Note: If multiple vowels or consonants have the same maximum frequency, you may choose
 * any one of them. If there are no vowels or no consonants in the string, consider their
 * frequency as 0.
 *
 * The frequency of a letter x is the number of times it occurs in the string.
 */

/**
 * @param {string} s
 * @return {number}
 */
var maxFreqSum = function(s) {
  const vowels = new Set(['a', 'e', 'i', 'o', 'u']);
  const vowelFreq = {};
  const consonantFreq = {};
```

```
  for (const char of s) {
    if (vowels.has(char)) {
      vowelFreq[char] = (vowelFreq[char] || 0) + 1;
    } else {
      consonantFreq[char] = (consonantFreq[char] || 0) + 1;
    }
  }

  const maxVowelFreq = Math.max(0, ...Object.values(vowelFreq));
  const maxConsonantFreq = Math.max(0, ...Object.values(consonantFreq));

  return maxVowelFreq + maxConsonantFreq;
};
```

3577 Count The Number Of Computer Unlocking Permutations

MEDIUM

APPROACH NOTES

Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3577. Count the Number of Computer Unlocking Permutations
 * https://leetcode.com/problems/count-the-number-of-computer-unlocking-permutations/
 * Difficulty: Medium
 *
 * You are given an array complexity of length n.
 *
 * There are n locked computers in a room with labels from 0 to n - 1, each with its own
 * unique password. The password of the computer i has a complexity complexity[i].
 *
 * The password for the computer labeled 0 is already decrypted and serves as the root.
 * All other computers must be unlocked using it or another previously unlocked computer,
 * following this information:
 * - You can decrypt the password for the computer i using the password for computer j,
 *   where j is any integer less than i with a lower complexity. (i.e. j < i and
 *   complexity[j] < complexity[i])
 * - To decrypt the password for computer i, you must have already unlocked a computer j
 *   such that j < i and complexity[j] < complexity[i].
 *
 * Find the number of permutations of [0, 1, 2, ..., (n - 1)] that represent a valid order
 * in which the computers can be unlocked, starting from computer 0 as the only initially
 * unlocked one.
 *
 * Since the answer may be large, return it modulo 109 + 7.
 *
 * Note that the password for the computer with label 0 is decrypted, and not the computer
 * with the first position in the permutation.
 */

/**
 * @param {number[]} complexity
 * @return {number}
 */
var countPermutations = function(complexity) {
  const MOD = 1e9 + 7;
  const root = complexity[0];

  for (let i = 1; i < complexity.length; i++) {
    if (complexity[i] <= root) {
      return 0;
    }
  }

  let result = 1;
  for (let i = 2; i < complexity.length; i++) {
    result = (result * i) % MOD;
  }

  return result;
};
```

3578 Count Partitions With Max Min Difference At Most K

MEDIUM

APPROACH NOTES

Contains a while loop: identify the progress condition. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3578. Count Partitions With Max-Min Difference at Most K
 * https://leetcode.com/problems/count-partitions-with-max-min-difference-at-most-k
 * Difficulty: Medium
 *
 * You are given an integer array nums and an integer k. Your task is to partition nums
 * into one or more non-empty contiguous segments such that in each segment, the difference
 * between its maximum and minimum elements is at most k.
 *
 * Return the total number of ways to partition nums under this condition.
 *
 * Since the answer may be too large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var countPartitions = function(nums, k) {
  const MOD = 1e9 + 7;
  const n = nums.length;
  const dp = new Array(n + 1).fill(0);
  dp[0] = 1;

  let sum = 1;
  const minQueue = [];
  const maxQueue = [];

  for (let left = 0, right = 0; right < n; right++) {
    while (maxQueue.length && nums[right] > nums[maxQueue.at(-1)]) maxQueue.pop();
    maxQueue.push(right);

    while (minQueue.length && nums[right] < nums[minQueue.at(-1)]) minQueue.pop();
    minQueue.push(right);

    while (nums[maxQueue[0]] - nums[minQueue[0]] > k) {
      sum = (sum - dp[left++] + MOD) % MOD;
      if (minQueue[0] < left) minQueue.shift();
      if (maxQueue[0] < left) maxQueue.shift();
    }

    dp[right + 1] = sum;
    sum = (sum + dp[right + 1]) % MOD;
  }

  return dp[n];
};
```

3583 Count Special Triplets

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3583. Count Special Triplets
 * https://leetcode.com/problems/count-special-triplets/
 * Difficulty: Medium
 *
 * You are given an integer array nums.
 *
 * A special triplet is defined as a triplet of indices (i, j, k) such that:
```

```
 * - 0 <= i < j < k < n, where n = nums.length
 * - nums[i] == nums[j] * 2
 * - nums[k] == nums[j] * 2
 *
 * Return the total number of special triplets in the array.
 *
 * Since the answer may be large, return it modulo 109 + 7.
 */

/**
 * @param {number[]} nums
 * @return {number}
 */
var specialTriplets = function(nums) {
  const MOD = 1e9 + 7;
  let result = 0;

  const leftCount = new Map();
  const rightCount = new Map();
  for (const num of nums) {
    rightCount.set(num, (rightCount.get(num) || 0) + 1);
  }

  for (let j = 0; j < nums.length; j++) {
    const middle = nums[j];
    const target = middle * 2;

    rightCount.set(middle, rightCount.get(middle) - 1);
    if (rightCount.get(middle) === 0) {
      rightCount.delete(middle);
    }

    const left = leftCount.get(target) || 0;
    const right = rightCount.get(target) || 0;
    result = (result + left * right) % MOD;
    leftCount.set(middle, (leftCount.get(middle) || 0) + 1);
  }

  return result;
};
```

3606 Coupon Code Validator

EASY

APPROACH NOTES

Sorts data: inspect the comparator and sorting cost.

SOURCE CODE

```
/**
 * 3606. Coupon Code Validator
 * https://leetcode.com/problems/coupon-code-validator/
 * Difficulty: Easy
 *
 * You are given three arrays of length n that describe the properties of n
 * coupons: code, businessLine, and isActive. The ith coupon has:
 * - code[i]: a string representing the coupon identifier.
 * - businessLine[i]: a string denoting the business category of the coupon.
 * - isActive[i]: a boolean indicating whether the coupon is currently active.
 *
 * A coupon is considered valid if all of the following conditions hold:
 * 1. code[i] is non-empty and consists only of alphanumeric characters (a-z, A-Z, 0-9)
 *    and underscores (_).
 * 2. businessLine[i] is one of the following four categories: "electronics", "grocery",
 *    "pharmacy", "restaurant".
 * 3. isActive[i] is true.
 *
 * Return an array of the codes of all valid coupons, sorted first by their businessLine
 * in the order: "electronics", "grocery", "pharmacy", "restaurant", and then by code in
 * lexicographical (ascending) order within each category.
 */

/**
 * @param {string[]} code
```

```
 * @param {string[]} businessLine
 * @param {boolean[]} isActive
 * @return {string[]}
 */
var validateCoupons = function(code, businessLine, isActive) {
  const whitelist = ['electronics', 'grocery', 'pharmacy', 'restaurant'];
  const order = { electronics: 0, grocery: 1, pharmacy: 2, restaurant: 3 };

  return code
    .map((id, i) => ({ id, name: businessLine[i], active: isActive[i] }))
    .filter(({ id, name, active }) => active && whitelist.includes(name) &&
/^[\da-z_]+$/i.test(id))
    .sort((a, b) => {
      const compare = order[a.name] - order[b.name];
      if (compare !== 0) return compare;
      return a.id < b.id ? -1 : a.id > b.id ? 1 : 0;
    })
    .map(({ id }) => id);
};
```

3623 Count Number Of Trapezoids I

MEDIUM

APPROACH NOTES

Uses a Map for keyed lookups. Contains iteration: trace the loop invariant.

SOURCE CODE

```
/**
 * 3623. Count Number of Trapezoids I
 * https://leetcode.com/problems/count-number-of-trapezoids-i/
 * Difficulty: Medium
 *
 * You are given a 2D integer array points, where points[i] = [xi, yi] represents the
 * coordinates of the ith point on the Cartesian plane.
 *
 * A horizontal trapezoid is a convex quadrilateral with at least one pair of horizontal
 * sides (i.e. parallel to the x-axis). Two lines are parallel if and only if they have
 * the same slope.
 *
 * Return the number of unique horizontal trapezoids that can be formed by choosing any
 * four distinct points from points.
 *
 * Since the answer may be very large, return it modulo 109 + 7.
 */

/**
 * @param {number[][]} points
 * @return {number}
 */
var countTrapezoids = function(points) {
  const MOD = 1e9 + 7;

  const map = new Map();
  for (const [x, y] of points) {
    map.set(y, (map.get(y) || 0) + 1);
  }

  let result = 0n;
  let total = 0n;
  for (const count of map.values()) {
    const lines = BigInt(count) * BigInt(count - 1) / 2n;
    result = (result + total * lines) % BigInt(MOD);
    total = (total + lines) % BigInt(MOD);
  }

  return Number(result);
};
```